深度学习：基础与概念

Foundations and Concepts

作者:Christopher M. Bishop & Hugh Bishop

组织: 邹欣　阮思捷　刘志毅　王树良　译

时间:November 9, 2026

版本:ElegantBook-4.5

目录

第 1 章 深度学习革命 **1**

1.1 深度学习的影响 1

1.1.1 医疗诊断 1

1.1.2 蛋白质结构预测 3

1.1.3 图像合成 4

1.1.4 大语言模型 5

1.2 一个教学示例 6

1.2.1 合成数据 6

1.2.2 线性模型 7

1.2.3 误差函数 8

1.2.4 模型复杂度 10

1.2.5 正则化 13

1.2.6 模型选择 14

1.3 机器学习简史 17

1.3.1 单层网络 19

1.3.2 反向传播 21

1.3.3 深度网络 23

第 2 章 概率 **27**

2.1 概率法则 29

2.1.1 医学筛查示例 29

2.1.2 加和法则和乘积法则 30

2.1.3 Bayes 定理 34

2.1.4 再看医学筛查示例 35

2.1.5 先验概率和后验概率 38

2.1.6 独立变量 38

2.2 概率密度 39

2.2.1 分布的示例 42

2.2.2 期望和协方差 45

2.3 Gauss 分布 48

2.3.1 均值和方差 50

2.3.2 似然函数 51

2.3.3 最大似然的偏差 54

2.3.4 线性回归 56

2.4 密度变换 59

2.5 信息论 64

2.5.1 熵 64

2.5.2 物理学视角 67

2.5.3 微分熵 69

2.5.4 最大熵 70

2.5.5 Kullback-Leibler 散度 72

2.5.6 条件熵 75

2.5.7 互信息 . . . 76
2.6 Bayes 概率 . . . 77
2.6.1 模型参数 . . . 78
2.6.2 正则化 . . . 80
2.6.3 Bayes 机器学习 . . . 82
2.7 习题 . . . 83

第 3 章 标准分布 **86**
3.1 离散变量 . . . 86
3.1.1 Bernoulli 分布 . . . 86
3.1.2 二项分布 . . . 90
3.1.3 多项分布 . . . 91
3.2 多元 Gauss 分布 . . . 95
3.2.1 Gauss 几何 . . . 96
3.2.2 矩 . . . 99
3.2.3 局限性 . . . 101
3.2.4 条件分布 . . . 102
3.2.5 边缘分布 . . . 105
3.2.6 Bayes 定理 . . . 107
3.2.7 最大似然 . . . 110
3.2.8 序贯估计 . . . 112
3.2.9 Gauss 混合 . . . 113
3.3 周期变量 . . . 116
3.4 指数族分布 . . . 122
3.4.1 充分统计量 . . . 127
3.5 非参数化方法 . . . 129
3.5.1 直方图 . . . 129
3.5.2 核密度 . . . 131
3.5.3 最近邻 . . . 134
3.6 习题 . . . 137

第 4 章 单层网络: 回归 **141**
4.1 线性回归 . . . 141
4.1.1 基函数 . . . 141
4.1.2 似然函数 . . . 143
4.1.3 最大似然 . . . 145
4.1.4 最小二乘的几何表示 . . . 149
4.1.5 序贯学习 . . . 151
4.1.6 正则化最小二乘法 . . . 152
4.1.7 多重输出 . . . 153
4.2 决策理论 . . . 155
4.3 偏差-方差权衡 . . . 160
4.4 习题 . . . 164

第 5 章 单层网络: 分类 **167**
5.1 判别函数 . . . 168

5.1.1 二分类 . . . 168
5.1.2 多分类 . . . 170
5.1.3 1-of-K 编码方案 . . . 172
5.1.4 最小二乘分类 . . . 173
5.2 决策理论 . . . 176
5.2.1 误分类率 . . . 176
5.2.2 预期损失 . . . 179
5.2.3 拒绝选项 . . . 181
5.2.4 推理和决策 . . . 182
5.2.5 分类器精度 . . . 184
5.2.6 ROC 曲线 . . . 187
5.3 生成分类器 . . . 189
5.3.1 连续输入 . . . 191
5.3.2 最大似然解 . . . 194
5.3.3 离散特征 . . . 196
5.3.4 指数族分布 . . . 197
5.4 判别分类器 . . . 198
5.4.1 激活函数 . . . 199
5.4.2 固定基函数 . . . 199
5.4.3 logistic 回归 . . . 200
5.4.4 多类 logistic 回归 . . . 204
5.4.5 probit 回归 . . . 206
5.4.6 规范连接函数 . . . 209
5.5 习题 . . . 211

第 6 章 深度神经网络 **213**
6.1 固定基函数的局限性 . . . 213
6.1.1 维度诅咒 . . . 213
6.1.2 高维空间 . . . 216
6.1.3 数据流形 . . . 218
6.1.4 数据依赖的基函数 . . . 219
6.2 多层网络 . . . 221
6.2.1 参数矩阵 . . . 222
6.2.2 通用近似 . . . 223
6.2.3 隐藏单元激活函数 . . . 226
6.2.4 权重空间的对称性 . . . 229
6.3 深度网络 . . . 230
6.3.1 层次化表示 . . . 231
6.3.2 分布式表示 . . . 232
6.3.3 表示学习 . . . 233
6.3.4 迁移学习 . . . 234
6.3.5 对比学习 . . . 236
6.3.6 通用网络结构 . . . 239
6.3.7 张量 . . . 240
6.4 误差函数 . . . 241

6.4.1 回归 . . . 241
6.4.2 二分类 . . . 244
6.4.3 多分类 . . . 246
6.5 混合密度网络 . . . 248
6.5.1 机器人运动学示例 . . . 248
6.5.2 条件混合分布 . . . 249
6.5.3 梯度优化 . . . 252
6.5.4 预测分布 . . . 254
6.6 习题 . . . 256

第 7 章 梯度下降 **259**
7.1 错误平面 . . . 259
7.1.1 局部二次近似 . . . 261
7.2 梯度下降优化 . . . 265
7.2.1 梯度信息的使用 . . . 265
7.2.2 批量梯度下降 . . . 266
7.2.3 随机梯度下降 . . . 266
7.2.4 小批量方法 . . . 268
7.2.5 参数初始化 . . . 269
7.3 收敛 . . . 271
7.3.1 动量 . . . 274
7.3.2 学习率调度 . . . 277
7.3.3 AdaGrad、RMSProp 与 Adam 算法 . . . 278
7.4 正则化 . . . 281
7.4.1 数据归一化 . . . 282
7.4.2 批量归一化 . . . 283
7.4.3 层归一化 . . . 286
7.5 习题 . . . 286

第 8 章 反向传播 **289**
8.1 梯度计算 . . . 289
8.1.1 单层网络 . . . 289
8.1.2 一般前馈网络 . . . 290
8.1.3 简单示例 . . . 294
8.1.4 数值微分法 . . . 295
8.1.5 Jacobi 矩阵 . . . 297
8.1.6 黑塞矩阵 . . . 299
8.2 自动微分法 . . . 300
8.2.1 前向模式自动微分 . . . 302
8.2.2 逆模式自动微分 . . . 305
8.3 习题 . . . 307

第 9 章 正则化 **309**
9.1 归纳偏置 . . . 309
9.1.1 逆问题 . . . 309
9.1.2 无免费午餐定理 . . . 310

9.1.3 对称性和不变性 . . . 312
9.1.4 等变性 . . . 314
9.2 权重衰减 . . . 315
9.2.1 一致性正则化项 . . . 317
9.2.2 广义权重衰减 . . . 320
9.3 学习曲线 . . . 323
9.3.1 早停法 . . . 324
9.3.2 双重下降 . . . 326
9.4 参数共享 . . . 329
9.4.1 软权重共享 . . . 330
9.5 残差连接 . . . 333
9.6 模型平均 . . . 337
9.6.1 dropout . . . 340
9.7 习题 . . . 342

第 10 章 卷积网络 **345**
10.1 计算机视觉 . . . 345
10.1.1 图像数据 . . . 345
10.2 卷积滤波器 . . . 346
10.2.1 特征检测器 . . . 346
10.2.2 平移等变性 . . . 348
10.2.3 填充 . . . 351
10.2.4 跨步卷积 . . . 352
10.2.5 多维卷积 . . . 353
10.2.6 池化 . . . 355
10.2.7 多层卷积 . . . 357
10.2.8 网络架构示例 . . . 359
10.3 可视化训练好的 CNN . . . 361
10.3.1 视觉皮层 . . . 361
10.3.2 可视化训练好的滤波器 . . . 362
10.3.3 显著性图 . . . 364
10.3.4 对抗攻击 . . . 364
10.3.5 合成图像 . . . 366
10.4 目标检测 . . . 367
10.4.1 边界框 . . . 368
10.4.2 交并比 . . . 369
10.4.3 滑动窗口 . . . 370
10.4.4 跨尺度检测 . . . 373
10.4.5 非最大抑制 . . . 373
10.4.6 快速区域卷积神经网络 . . . 374
10.5 图像分割 . . . 374
10.5.1 卷积分割 . . . 375
10.5.2 上采样 . . . 375
10.5.3 全卷积网络 . . . 379
10.5.4 U-Net 架构 . . . 379

10.6 风格迁移 . . . 380
10.7 习题 . . . 382

第 11 章 结构化分布 384
11.1 概率图模型 . . . 384
11.1.1 有向图 . . . 384
11.1.2 分解 . . . 385
11.1.3 离散变量 . . . 388
11.1.4 Gauss 变量 . . . 391
11.1.5 二元分类器 . . . 393
11.1.6 参数和观测值 . . . 394
11.1.7 Bayes 定理 . . . 396
11.2 条件独立性 . . . 397
11.2.1 3 个示例图 . . . 398
11.2.2 相消解释 . . . 402
11.2.3 d 分离 . . . 403
11.2.4 朴素 Bayes . . . 405
11.2.5 生成式模型 . . . 407
11.2.6 Markov 毯 . . . 408
11.2.7 作为过滤器的图 . . . 410
11.3 序列模型 . . . 411
11.4 习题 . . . 417

第 12 章 Transformer 419
12.1 注意力 . . . 419
12.1.1 Transformer 处理 . . . 421
12.1.2 注意力系数 . . . 421
12.1.3 自注意力 . . . 423
12.1.4 网络参数 . . . 424
12.1.5 缩放自注意力 . . . 427
12.1.6 多头注意力 . . . 428
12.1.7 Transformer 层 . . . 430
12.1.8 计算复杂性 . . . 432
12.1.9 位置编码 . . . 433
12.2 自然语言 . . . 436
12.2.1 词嵌入 . . . 437
12.2.2 分词 . . . 438
12.2.3 词袋模型 . . . 439
12.2.4 自回归模型 . . . 440
12.2.5 递归神经网络 . . . 441
12.2.6 通过时间反向传播 . . . 443
12.3 Transformer 语言模型 . . . 444
12.3.1 解码器型 Transformer . . . 444
12.3.2 抽样策略 . . . 446
12.3.3 编码器型 Transformer . . . 448

12.3.4 序列到序列 Transformer . . . 450
12.3.5 大语言模型 . . . 450
12.4 多模态 Transformer . . . 453
12.4.1 视觉 Transformer . . . 454
12.4.2 图像生成 Transformer . . . 455
12.4.3 音频数据 . . . 457
12.4.4 文本语音转换 . . . 458
12.4.5 视觉和语言 Transformer . . . 458
12.5 习题 . . . 459

第 13 章 图神经网络 **464**
13.1 基于图的机器学习 . . . 464
13.1.1 图的属性 . . . 465
13.1.2 邻接矩阵 . . . 466
13.1.3 置换等变性 . . . 467
13.2 神经信息传递 . . . 468
13.2.1 卷积滤波器 . . . 469
13.2.2 图卷积网络 . . . 471
13.2.3 聚合算子 . . . 472
13.2.4 更新算子 . . . 475
13.2.5 节点分类 . . . 477
13.2.6 边分类 . . . 479
13.2.7 图分类 . . . 479
13.3 通用图网络 . . . 480
13.3.1 图注意力网络 . . . 480
13.3.2 边嵌入 . . . 482
13.3.3 图嵌入 . . . 483
13.3.4 过度平滑 . . . 484
13.3.5 正则化 . . . 485
13.3.6 几何深度学习 . . . 486
13.4 习题 . . . 487

第 14 章 采样 **489**
14.1 基本采样 . . . 489
14.1.1 期望 . . . 489
14.1.2 标准分布 . . . 491
14.1.3 拒绝采样 . . . 495
14.1.4 适应性拒绝采样 . . . 498
14.1.5 重要性采样 . . . 499
14.1.6 采样-重要性-重采样 . . . 502
14.2 Markov 链 Monte Carlo 采样 . . . 503
14.2.1 Metropolis 算法 . . . 503
14.2.2 Markov 链 . . . 505
14.2.3 Metropolis-Hastings 算法 . . . 508
14.2.4 Gibbs 采样 . . . 510

14.2.5 祖先采样 . . . 514
14.3 Langevin 采样 . . . 515
14.3.1 基于能量的模型 . . . 516
14.3.2 最大化似然 . . . 517
14.3.3 Langevin 动力学 . . . 520
14.4 习题 . . . 521

第 15 章 离散潜变量 **524**
15.1 K 均值聚类 . . . 524
15.2 Gauss 混合分布 . . . 531
15.2.1 似然函数 . . . 534
15.2.2 最大似然 . . . 536
15.3 EM 算法 . . . 541
15.3.1 Gauss 混合模型 . . . 543
15.3.2 EM 算法与 K 均值算法的关系 . . . 546
15.3.3 混合 Bernoulli 分布 . . . 547
15.4 证据下界 . . . 551
15.4.1 EM 算法回顾 . . . 554
15.4.2 独立同分布数据 . . . 556
15.4.3 参数先验 . . . 557
15.4.4 广义 EM 算法 . . . 558
15.4.5 顺序 EM 算法 . . . 559
15.5 习题 . . . 560

第 16 章 连续潜变量 **562**
16.1 主成分分析 . . . 562
16.1.1 最大方差表述 . . . 563
16.1.2 最小误差表述 . . . 565
16.1.3 数据压缩 . . . 569
16.1.4 数据白化 . . . 570
16.1.5 高维数据 . . . 571
16.2 概率潜变量 . . . 572
16.2.1 生成式模型 . . . 572
16.2.2 似然函数 . . . 573
16.2.3 最大似然法 . . . 575
16.2.4 因子分析 . . . 579
16.2.5 独立成分分析 . . . 581
16.2.6 Kalman 滤波器 . . . 582
16.3 证据下界 . . . 583
16.3.1 EM 算法 . . . 585
16.3.2 PCA 的 EM 算法 . . . 587
16.3.3 因子分析的 EM 算法 . . . 587
16.4 非线性潜变量模型 . . . 589
16.4.1 非线性流形 . . . 589
16.4.2 似然函数 . . . 592

16.4.3 离散数据 . . . 592
16.4.4 构建生成式模型的 4 种方法 . . . 593
16.5 习题 . . . 594

第 17 章 生成对抗网络 596
17.1 对抗训练 . . . 596
17.1.1 损失函数 . . . 597
17.1.2 实战中的 GAN 训练 . . . 600
17.2 图像的生成对抗网络 . . . 604
17.2.1 CycleGAN . . . 604
17.2.2 表示学习与潜空间插值 . . . 608
17.3 习题 . . . 609

第 18 章 标准化流 611
18.1 耦合流 . . . 614
18.2 自回归流 . . . 618
18.3 连续流 . . . 623
18.3.1 神经 ODE . . . 623
18.3.2 神经 ODE 的反向传播 . . . 625
18.3.3 神经 ODE 流 . . . 627
18.4 习题 . . . 631

第 19 章 自编码器 633
19.1 确定性的自编码器 . . . 633
19.1.1 线性自编码器 . . . 633
19.1.2 深度自编码器 . . . 636
19.1.3 稀疏自编码器 . . . 638
19.1.4 去噪自编码器 . . . 638
19.1.5 掩蔽自编码器 . . . 639
19.2 变分自编码器 . . . 641
19.2.1 摊销推理 . . . 644
19.2.2 重参数化技巧 . . . 646
19.3 习题 . . . 651

第 20 章 扩散模型 652
20.1 前向编码器 . . . 652
20.1.1 扩散核 . . . 653
20.1.2 条件分布 . . . 655
20.2 反向解码器 . . . 656
20.2.1 训练解码器 . . . 658
20.2.2 证据下界 . . . 658
20.2.3 重写 ELBO . . . 660
20.2.4 预测噪声 . . . 663
20.2.5 生成新的样本 . . . 664
20.3 得分匹配 . . . 666
20.3.1 得分损失函数 . . . 667

20.3.2 修改得分损失 . . . 667
20.3.3 噪声方差 . . . 669
20.3.4 随机微分方程 . . . 670
20.4 有引导的扩散 . . . 671
20.4.1 有分类器的引导 . . . 672
20.4.2 无分类器的引导 . . . 673
20.5 习题 . . . 675

附录 A 线性代数 **678**
A.1 矩阵恒等式 . . . 678
A.2 迹和行列式 . . . 681
A.3 矩阵导数 . . . 684
A.4 特征向量 . . . 687

附录 B 变分法 **695**

附录 C Lagrange 乘子 **700**

索引 **721**

第 1 章 深度学习革命

1.1 深度学习的影响

机器学习是当今最重要、发展最快的技术领域之一. 机器学习的应用遍及各个领域, 基于数据学习的解决方案正逐步取代传统的手工设计算法. 这不仅提升了现有技术的性能指标, 更为一系列全新能力的开发打开了大门. 而如果新算法仍需完全依赖人工设计, 这些能力的实现将是无法想象的.

作为机器学习的一个分支, 深度学习 (Deep Learning, DL) 已经成为一种异常强大且通用的数据学习框架. 深度学习的理论基础是被称为神经网络 (neural network) 的计算模型, 这些模型最初是受到人类大脑中学习和信息处理机制的启发而产生的. 人工智能 (Artificial Intelligence, AI) 领域致力于在机器中重现人脑的强大能力, 而今天“机器学习”和“人工智能”这两个术语常常被交替使用. 当前使用的许多人工智能系统, 实际上是机器学习的具体应用, 旨在解决特定领域的问题. 这些系统虽然非常有用, 但与人类大脑的巨大能力相比仍有很大差距. 人们引入了“通用人工智能” (Artificial General Intelligence, AGI) 这一概念, 来描述构建具有更高灵活性的智能系统的愿景. 经过几十年的稳步发展, 机器学习现在已进入快速发展阶段 (参见第 12 章). 最近, 一种称为大语言模型的庞大深度学习系统开始展现出卓越的能力, 人们把这些能力视为通用人工智能的初步迹象 (Bubeck et al., 2023).

本节从 4 个不同领域的示例来讨论机器学习, 以此说明这一技术应用的广泛性, 同时引入一些相关的基本概念和术语. 值得注意的是, 这些示例以及很多其他示例都是使用深度学习这一基本框架的不同变体来解决的. 这与传统方法形成了鲜明对比, 在传统方法中, 不同的应用需要采用各种迥异且高度专业的技术来处理. 应当强调的是, 我们选择的例子只代表了深度神经网络应用广度的一小部分, 事实上, 几乎所有需要计算的领域都正在或即将受益于深度学习的革命性影响.

1.1.1 医疗诊断

我们先来看看机器学习在皮肤癌诊断中的应用. 黑色素瘤 (melanoma) 是最危险的一种皮肤癌, 但如果发现得早, 是可治愈的.图 1.1展示了皮肤病变的示例图片, 上面的一行图像是危险的恶性黑色素瘤, 下面的一行图像是良性痣. 显然, 区分这两类图片具有极大的挑战性, 几乎不可能通过人工编写出一个算法来实现对这些图像的准确分类.

这个问题已经通过深度学习得到成功解决 (Esteva et al., 2017). 该解决方案是通过一组大型的病变图像集构建的, 该图集称为训练集, 其中每一幅图像都被标注为恶性或良性, 标注的依据是对病变活检后得到的真实分类. 训练集用来确定深度神经网络中约 2500 万个可调参数的取值. 这个根据数据反馈的结果来设置参数值的过程称为学习或训练. 训练的目标是使得训练好的神经网络根据新病变的图像就可以预测病变是良性还是恶性, 而无须执行耗时的活检步骤.

定义 1.1 (训练集、权重与训练)

设某学习任务中有一组带标注的样本 $\{(x_n, t_n)\}_{n=1}^{N}$, 其中 x_n 是第 n 个样本的输入,t_n 是它对应的标注. 这组样本称为**训练集** (training set),x_n 称为输入变量,t_n 称为目标变量或标签. 模型中可调的参数称为**权重** (weight), 记其全体为 $\mathbf{w}$. 依据训练集上的数据来确定 $\mathbf{w}$ 取值的过程称为**学习** (learning) 或**训练** (training). ♣

这是一个典型的监督学习问题的例子, 因为在训练过程中, 每个样本都标注了正确的类别标签. 这同时也是一个分类问题的例子, 因为每个输入经过处理后, 都需要被分配到离散的类别集合中 (本例中为“良性”或“恶性”). 如果输出包括一个或多个连续变量, 则称为回归问题. 在化工生产过程中, 根据温度、压力和反应物浓度预测产量, 就是回归问题的一个例子.

定义 1.2 (监督学习、分类与回归)

若训练过程中每个样本都带有正确的目标值 (例如类别标签), 则称相应的学习问题为**监督学习** (supervised learning). 在监督学习问题中, 若每个输入需要被分配到离散的类别集合中的某一类, 则称为**分类** (classification) 问题; 若输出包含一个或多个连续变量, 则称为**回归** (regression) 问题.

♣

这个应用的一个有趣之处是, 可用的带标注的训练图像数量相对较少, 大约只有 12.9 万幅. 因此, 研究人员首先要在一个包含 128 万幅日常物体 (如狗、建筑和蘑菇等) 图像的大型数据集上对深度神经网络进行预训练, 然后在皮肤病变图像数据集上对进行微调. 这是迁移学习的一个例子, 神经网络从日常物体的大规模数据集中学习自然图像的一般属性, 随后专门针对皮肤病变分类问题进行优化. 使用深度学习技术对皮肤病变图像进行分类的准确率已经超过专业皮肤科医生的水平 (Brinker et al., 2019).

定义 1.3 (预训练、微调与迁移学习)

设模型首先在一个大规模通用数据集上训练, 得到参数 $\mathbf{w}_0$; 随后以 $\mathbf{w}_0$ 为初值, 在目标任务的小规模数据集上继续训练. 第二步称为**微调** (fine-tune), 第一步称为**预训练** (pre-training). 把在源任务上获得的知识用于改进目标任务性能的这一类做法, 统称为**迁移学习** (transfer learning).

♣

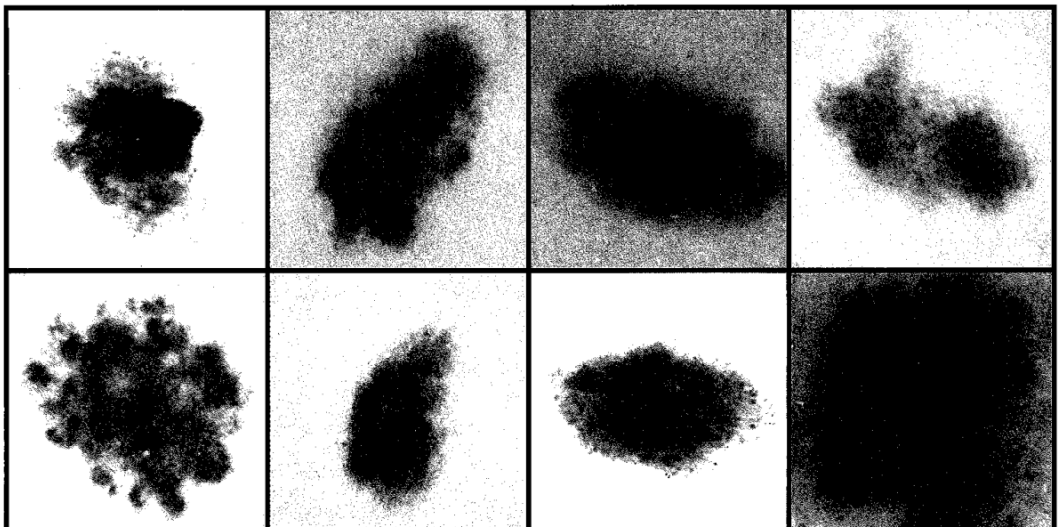


图 1.1: 上面的一行图像是危险的恶性黑色素瘤, 下面的一行图像是良性痣. 未经专业训练的人很难区分这两类皮肤病变类型.

注

(重构新增) 上面三个定义刻画了监督学习的三层结构:定义 1.1给出“数据与参数”,定义 1.2按标签形态把任务分为分类与回归,定义 1.3则说明了在标注数据稀缺时如何借助无关的大数据集.

“有标注”是监督学习的前提, 它把“学习什么”变成了“拟合什么”: 正因为每个 x_n 都配有 t_n, 训练才可以归结为让模型输出逼近已知目标值, 才有了后文可以最小化的误差函数. 分类与回归的区分不在于算法, 而在于输出空间的形态——离散类别给出的是“判定”, 连续变量给出的是“估计”, 两者的损失函数与评价方式因此不同 (详见第 5 章与第 4 章).

迁移学习之所以成立, 是因为自然图像的底层统计特性 (边缘、纹理、颜色分布) 在不同任务之间高度共享; 预训练阶段学到的正是这些共性, 微调阶段只需要调整与具体任务相关的部分. 因此, 微调的必要条件是两个任务的输入分布不能相差过远, 否则预训练得到的 $\mathbf{w}_0$ 不再是良好的初值.

1.1.2 蛋白质结构预测

蛋白质常被称为生物体的基石. 它们是由一个或多个氨基酸的长链组成的生物分子, 蛋白质的特性则由 22 种不同类型的氨基酸序列所决定. 当蛋白质在活细胞内合成后, 就会折叠成复杂的三维结构, 其行为和相互作用很大程度上取决于折叠的形态.

怎么根据氨基酸序列推导出蛋白质的三维结构呢? 这是生物学界几十年来一直想攻克的基础性难题, 但是进展甚微, 直到深度学习技术带来了突破性进展.

蛋白质的三维结构可以通过 X 射线晶体学、低温电子显微镜或核磁共振光谱等实验技术进行测定. 然而, 这个过程通常非常耗时, 而且对某些蛋白质来说, 这种方法极具挑战性. 例如, 很难得到蛋白质的纯净样本, 或者蛋白质的结构依赖于特定环境. 相比之下, 确定蛋白质的氨基酸序列的实验成本较低, 产出较高. 因此, 如何直接从氨基酸序列预测蛋白质的三维结构引起广泛关注, 这有助于更好地了解生物过程, 以及用于新药研发等实际应用. 我们可以以氨基酸序列为输入、以蛋白质的三维结构为输出来训练深度学习模型, 其中的训练数据包含了已知的氨基酸序列和对应的蛋白质三维结构. 因此, 蛋白质结构预测也是监督学习的一个例子. 经过训练的系统能够以新的氨基酸序列为输入, 预测对应的蛋白质三维结构 (Jumper et al., 2021).图 1.2比较了一种蛋白质的预测三维结构和通过 X 射线晶体学获得的实际三维结构.

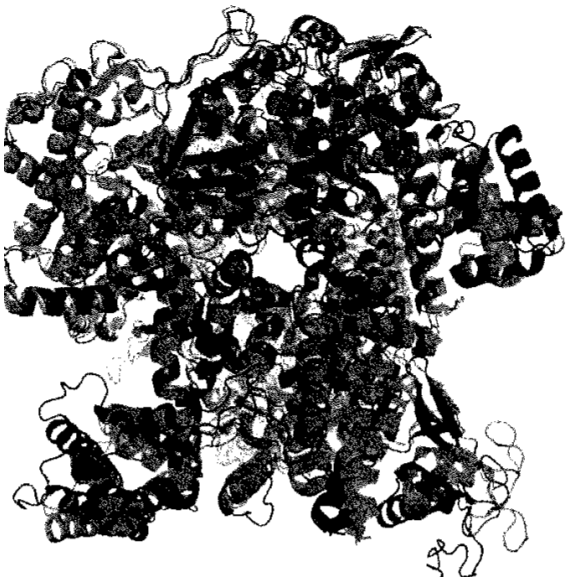


图 1.2: 一种名为 T1044/6VR4 的蛋白质的三维形状. 绿色结构展示了通过 X 射线晶体学确定的真实三维结构, 而叠加在上面的蓝色结构展示了深度学习模型 AlphaFold 预测的三维结构〔图片来源于 Jumper et al. (2021), 经许可使用〕.

注

(重构新增) 本工程的全部插图都直接裁自原书扫描件, 未重绘、未上色. 该扫描件是灰度图, 原书的彩色插图在此以灰度深浅呈现; 图题与正文中提到的颜色 (红、绿、蓝、橙等) 沿用原书表述, 请按原书的配色理解, 而形状、线型、箭头与坐标标注仍然可辨.

1.1.3 图像合成

在上面讨论的两个应用中, 神经网络学会了将输入 (皮肤图像或氨基酸序列) 转换成输出 (病变分类或蛋白质的三维结构). 现在我们来看一个不同的例子: 训练数据是一组样本图像, 训练后的神经网络能生成类似的新图像. 与病变分类和蛋白质结构预测的例子不同, 这些训练图像是没有标注的, 这样的训练属于无监督学习.图 1.3展示了一些由深度神经网络生成的合成图像, 该深度神经网络是在拍摄于摄影棚单色背景下的人脸图像集上进行训练的. 这些合成图像质量极高, 很难将它们和真人照片区分开来.

定义 1.4 (无监督学习)

若训练数据不带有任何人工标注的目标值, 训练过程只利用输入样本 $x_1,\ldots,x_N$ 本身的统计结构, 则称相应的学习问题为**无监督学习** (unsupervised learning).

♣

这种模型可以生成与训练数据不同但又有类似统计特性的结果, 所以被称为生成式模型. 这种方法的一个变体能根据输入的文本字符串 (称为提示词,prompt) 生成反映文本语义的图像 (参见第 10 章). 我们用生成式 AI(generative AI) 这个术语来描述那些能生成图像、视频、音频、文本、候选药物分子或其他形态信息的深度学习模型.

定义 1.5 (生成式模型、提示词与生成式 AI)

设训练数据服从某个未知分布 p_{data}. 若模型能够生成与训练数据不同、但具有与之相似统计特性的新样本, 则称该模型为**生成式模型** (generative model). 输入给生成式模型、用于描述所需输出内容的文本字符串称为**提示词** (prompt). 能够生成图像、视频、音频、文本、候选药物分子或其他形态信息的深度学习模型, 统称为**生成式 AI**(generative AI).

♣

图 1.3: 由无监督学习方法训练得到的深度神经网络生成的人脸图像〔来自 generated.photos 网站〕.

注

(重构新增) 定义 1.4与定义 1.5的差别, 正是“判别”与“生成”的差别: 前者只要求模型从数据中提取结构, 后者要求模型能够从结构中重新采出样本. 由于没有标签可以把“错”定义成“与 t_n 不一致”, 无监督学习必须以数据分布本身为学习目标, 这既是它的困难所在, 也是它能够被用于生成的原因.

对生成式模型而言, “与训练数据不同但统计特性相似”这一条不能弱化为“与训练数据相同”: 若模型只会复制训练样本, 它就退化为查表, 失去了泛化到新样本的能力; 也不能强化为“必须复现 p_{data} 的每一个细节”, 因为在有限样本下真实分布本身无法被精确恢复. 提示词的存在说明条件生成是一般情形——模型学到的实际上是条件分布 p(图像 | 提示词), 无条件生成只是提示词为空的特例.

1.1.4 大语言模型

近年来, 机器学习领域最重要的进展之一, 是研发人员开发出了强大的模型来处理自然语言和类似程序代码这样的序列数据. 大语言模型利用深度学习技术构建丰富的内部表征, 捕捉语言的语义属性.

定义 1.6 (大语言模型)

用于处理自然语言、程序代码等序列数据, 并借助深度学习技术为序列构建丰富的内部表征以捕捉其语义属性的模型, 称为**大语言模型** (Large Language Model, LLM).

♣

有一类重要的大语言模型称为自回归 (autoregressive) 语言模型, 它们能够输出语言, 因此属于生成式 AI 的一种. 这类模型以一个词序列作为输入, 生成这个词序列最可能的下一个词, 我们可以把末尾添加了新词的扩展序列再输入到模型中, 让它生成后续的词, 重复这个过程, 就会得到越来越长的词序列. 这样的模型还可以输出一个特殊的“停止”词, 来表示文本生成的结束, 这样就使得模型在输出一定长度的文本后停止. 此时, 用户可以在词序列的末尾添加自定义的词序列, 然后将完整的词序列重新输入模型以触发生成后续的词. 通过这种方式, 人们就可以和神经网络对话了.

定义 1.7 (自回归语言模型)

设 $w_{1:T} = (w_1, \ldots, w_T)$ 是一个词序列. 以 $w_{1:t}$ 为输入、输出该序列最可能的下一个词 w_{t+1} 的模型称为**自回归** (autoregressive) 语言模型. 把新生成的词接到序列末尾并重复输入同一模型, 即得到越来越长的词序列; 模型输出的特殊“停止”词表示生成结束.

♣

这类模型可以在大型文本数据集上通过提取训练对 (training pair) 进行训练. 训练对的输入是随机选定的词序列, 输出是已知的下一个词. 这是自监督学习的一个例子, 在自监督学习中, 模型学习从输入到输出映射函数, 有标注的输出是从输入训练数据中自动获取的, 无须进行另外的人工标注. 这种方法能把多种来源的大量文本作为大型训练集, 从而训练出超大规模神经网络.

定义 1.8 (训练对与自监督学习)

若训练样本由一个输入和一个“从输入本身自动获得的”目标值组成, 则称这样的样本对为**训练对** (training pair). 模型学习从输入到目标的映射、而目标值由输入数据自动生成、无需人工标注的学习方式, 称为**自监督学习** (self-supervised learning).

♣

一些专家认为, 展现出非凡能力的大语言模型是通用人工智能的雏形 (Bubeck et al., 2023), 本书后面将详细讨论这类模型 (参见第 12 章). 接下来, 我们给出一个基于 GPT-4(OpenAI, 2023) 的语言生成示例.GPT-4 能够针对如下提示做出响应: “请以莎士比亚戏剧风格, 通过两个角色对话展开争论的方式证明素数有无穷多个”, 其输出的译文见本页脚注.[1]

[1] 译者用大模型翻译的原文图 1.4(原书此处为截图):
甲: 汝未悟友啊, 真理难准, 素数无尽, 犹如天辰. 数不可分, 纯而且简, 虽觉奇迹, 实则无边.
乙: 可汝何以断定此论? 何证可显, 真理之根?
甲: 听我言, 疑心伴侣, 一证妙绝, 将疑云除. 设若终极素数存在, 为我们所见最大数领. 名之“P”字, 记于案头, 构思新数“Q”, 令人惊叹. 它将

注

(重构新增) 定义 1.6、定义 1.7与定义 1.8共同说明了为什么语言建模可以“无标签”地完成: 自回归模型把第 $t+1$ 个词当作前 t 个词的标签, 于是任何一个文本语料都自动变成了训练集. 条件“输入是随机选定的词序列”保证了模型见到的是文本的多种上下文, 而不是固定的前缀; “输出是已知的下一个词”则保证了每一步都有确定的目标, 这正是自监督学习的定义所指.

“自回归”中的“自”强调预测依赖于模型自己此前生成的输出: 推理时序列越接越长, 模型每一步的条件都在变化, 因此误差会逐步累积——这是后文讨论采样策略与大模型生成质量时必须回到的事实.

1.2 一个教学示例

对机器学习领域的新人来说, 许多基本概念和术语可以通过一个简单的例子来引入——用多项式拟合一个小型合成数据集 (Bishop, 2006). 这是一个监督学习问题, 在这个问题中, 我们希望根据输入的变量值, 对目标变量进行预测.

1.2.1 合成数据

我们用 x 表示输入变量, 用 t 表示目标变量, 并假设这两个变量在实数轴上取值连续. 给定一个训练集, 其中包含 N 个 x 的观测值, 记作 $x_1,\cdots,x_N$, 还包含相应 t 的观测值, 记作 $t_1,\cdots,t_N$. 我们的目标是根据 x 的某个新值来预测相应的 t 的值. 机器学习的一个关键目标是对以前未见过的输入进行准确预测, 这种能力称为泛化能力 (generalization).

定义 1.9 (泛化能力)

设 $y(\cdot,\mathbf{w})$ 是由训练数据确定的一个模型. 若该模型对训练集以外、此前未见过的输入也能给出准确预测, 则称该模型具有**泛化能力** (generalization). ♣

注

(重构新增) 这里的关键限定是“训练集以外”. 若一个输入本来就在训练数据之中, 模型对它给出正确回答只说明它记住了这个样本, 并不构成泛化; 因此本书后面的做法总是: 预留一部分与训练集不相交的数据, 只在这些数据上衡量预测是否准确. 还有一点需要澄清: 泛化能力是相对于“数据由什么规律产生、模型见过多少数据”而言的性质, 而不是模型单独具有的属性. 同一个模型在小训练集上可能泛化很差, 在更大的训练集上却泛化良好, 这正是小节 1.2.4中要定量考察的现象.

我们可以通过从正弦函数采样生成的合成数据集来说明这一点.

例题 1.1 正弦合成数据集 取 $N=10$. 输入值 $x_n (n=1,\cdots,N)$ 通过在区间 $[0,1]$ 上均匀采样生成; 对应的目标值 t_n 则是先计算每个 x 所对应的函数 $\sin(2\pi x)$ 的值, 然后向每个数据点添加少量随机噪声 (由 Gauss 分布控制, 参见节 2.3) 得到的.

通过这种方式生成数据, 我们可以捕获许多现实世界数据集的一个重要特性——它们具有我们希望了解的潜在规律, 但个别观测值会被随机噪声干扰. 这种噪声可能源于它们固有的随机过程 (例如放射性衰变), 但更常见的原因是存在未被观测到的变异源.

超越“P”, 成为素环.
乙: 此证何从? 友啊说明, 胜利之杯, 尚未高举.
甲: 如斯简单, 静待佳音, 乘素数列, 答案必寻. 自二至“P”, 逐一相连, 得“Q”新数, 正解现身. 加一以固, 使证更真, 举证眼前, 不再沉吟.
乙: 现明白矣, 如光闪莹, “Q”当素数, 结论清晰. 它不被小素整除有据, 时序内自身是最大. 既然选“P”作为终章, 预设破碎, 成为往昔. 无终素数, 真理已昭, 数字无穷, 如星海常在.

注

(重构新增) 例 1.1把“噪声”这一步写得更具体些: 若记 $t_n = \sin(2\pi x_n) + \varepsilon_n$, 其中 ε_n 是相互独立的随机变量, 则 $\sin(2\pi x_n)$ 是决定观测的**系统性成分**, 而 ε_n 是**随机性成分**. 本例中后者取为均值为 0 的 Gauss 噪声, 其标准差控制了圆点在曲线附近的散布程度. 这一分解的作用在于: 它说明“从数据中学习”不是去解释每一个观测点, 而是去估计那个系统性成分. 若没有噪声, 则在次数不超过 $N-1$ 的多项式中, 通过这 N 个点的那条曲线是唯一的; 正因为有噪声, 任何只依据有限样本的估计都会带有不确定性, 这也是后文要用概率论 (参见节 2.1) 来刻画预测的原因.

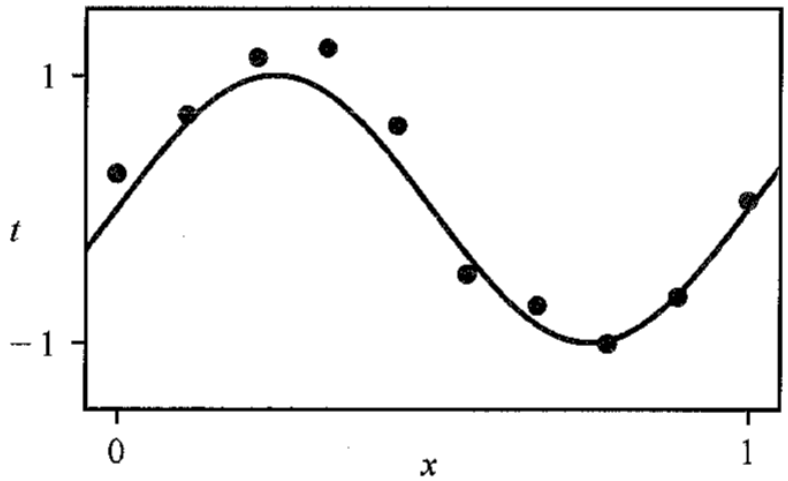


图 1.4: 一个由 $N = 10$ 个数据点组成的训练集, 以蓝色圆点显示, 其中每个数据点包含了输入变量 x 及其对应的目标变量 t 的观测值. 绿色曲线显示了用来生成数据的函数 $\sin(2\pi x)$. 我们的目标是在不知道绿色曲线的情况下, 预测新的输入变量 x 所对应的目标变量 t 的值.

在这个示例中, 我们事先知道真正生成数据是通过一个正弦函数. 在机器学习的实际应用中, 我们的目标是在有限的训练集中发现隐藏的规律. 不过, 了解数据的生成过程有助于我们阐明机器学习中的一些重要概念.

1.2.2 线性模型

现在让我们先从一种相对非正式的方式出发, 考虑一种基于曲线拟合的简单方法. 我们将使用多项式函数来拟合数据, 其形式如下:

定义 1.10 (多项式模型)

设 M 为非负整数,$w_0, \cdots, w_M$ 为实系数. 称由

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j \tag{1.1}$$

给出的函数 $y(\cdot, \mathbf{w})$ 为 M **阶多项式函数**, 并称以它为预测函数的模型为**多项式模型** (polynomial model). 其中 M 称为多项式的**阶数** (order),x^j 表示 x 的 j 次幂, 系数 $w_0, \cdots, w_M$ 统称为向量 $\mathbf{w}$.

♣

注

(重构新增) 引入多项式模型的动机是把“寻找一条曲线”换成“确定一个有限维向量”. 未知函数有无穷多个自由度, 而 M 阶多项式只由 $M+1$ 个实数决定, 于是“学习”就归结为从训练数据中把这 $M+1$ 个数定下来, 这使问题可以用微积分与线性代数处理. 条件方面要注意两点. 其一,M 是模型的**结构参数**而不是待拟合的系数: 给定 M, 可选的函数族才被固定, 因此 M 不能靠最小化训练误差来选 (见小节 1.2.6). 其二, 各阶函数族是**嵌套**的:M 阶族包含全部低阶多项式 (把高阶系数取 0 即可), 这一点在命题 1.2中会给出严格表述, 它也是理解“$M = 9$ 的训练误差不可能比 $M = 3$ 差、测试误差却差得多”这一表面矛盾的出发点.

注意, 尽管多项式函数 $y(x, \mathbf{w})$ 是 x 的非线性函数, 但它也是系数 $\mathbf{w}$ 的线性函数. 在式(1.1)中, 像这个多项式一样, 关于未知参数呈线性的函数具有重要的特性, 同时也存在明显的局限性, 它们被称为线性模型 (linear model).

定义 1.11 (线性模型)

设 $\mathbf{w}$ 为模型的未知参数向量. 若对每个 x, 函数值 $y(x, \mathbf{w})$ 都是 $\mathbf{w}$ 的仿射函数, 则称 y 为关于这些参数的**线性模型** (linear model).

♣

注

(重构新增) “线性”修饰的是**对参数的依赖**, 不是对输入 x 的依赖:x^M、$\sin x$、$\log x$ 都可以作为输入特征进入线性模型, 只要它们是已知的、不随 $\mathbf{w}$ 变化的函数. 反过来, 只要参数以乘积或复合的方式出现, 就不再是线性模型. 这一区分的实际作用在于它决定了误差函数的面貌: 若 y 关于 $\mathbf{w}$ 是仿射的, 则平方和误差是 $\mathbf{w}$ 的二次函数, 其梯度是 $\mathbf{w}$ 的仿射函数, 最小化问题有闭式解 (命题 1.1); 一旦失去对参数的线性, 就只剩下迭代优化这一条路, 这正是小节 1.2.6末尾所说的现实困难, 也是后续章节讨论梯度方法 (参见节 7.2) 的原因.

1.2.3 误差函数

多项式系数的值将通过拟合训练数据来确定, 这可以通过最小化误差函数 (error function) 来实现, 该误差函数度量了对于任意给定的 $\mathbf{w}$, 函数 $y(x, \mathbf{w})$ 与训练集中数据点之间的拟合误差.

定义 1.12 (误差函数)

设训练集为 $\{(x_n, t_n)\}_{n=1}^N$, 模型为 $y(\cdot, \mathbf{w})$. 称由训练集完全确定、以参数 $\mathbf{w}$ 为自变量的函数 $E(\mathbf{w})$ 为**误差函数** (error function), 若它度量了对于给定的 $\mathbf{w}$, 函数 $y(x, \mathbf{w})$ 与训练集中各数据点之间的拟合误差.

♣

注

(重构新增) 误差函数的作用是把“拟合得好不好”这一模糊说法压缩成一个可以比较大小的实数, 从而把学习问题转化为一个优化问题: 在所有候选参数中取使 $E(\mathbf{w})$ 最小者. 没有这一步, 就无法谈论“最优”模型. 条件上, 误差函数必须**只由训练集和模型决定**, 不能依赖测试数据或任何未知量; 同时, 为了能用基于梯度的算法 (节 7.2) 求解, 通常还要求它对 $\mathbf{w}$ 可微. 不同的误差函数对应不同的估计准则: 选择平方和误差, 等价于假定目标值上的噪声服从 Gauss 分布, 这一点稍后会再次提到 (参见小节 2.3.4).

有一个使用广泛的简单误差函数, 即每个数据点 x_n 的预测值 $y(x_n, \mathbf{w})$ 与相应目标值 t_n 之间的差的平方和的二分之一:

定义 1.13 (平方和误差函数)

对训练集 $\{(x_n, t_n)\}_{n=1}^N$ 与模型 $y(\cdot, \mathbf{w})$, 称

$$E(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(x_n, \mathbf{w}) - t_n\}^2 \tag{1.2}$$

为**平方和误差函数** (sum-of-squares error function).

♣

注

(重构新增) 平方和误差的动机有三层: 平方使正负偏差不会相互抵消; 平方对大偏差的惩罚远重于小偏差, 因而拟合会优先照顾偏差最大的点; 它是 $\mathbf{w}$ 的光滑 (可微) 函数, 便于求导. 其中系数 1/2 纯粹是为了后续计算方便: 对 E 求导时它会与平方带来的因子 2 相消, 使梯度中不出现常数, 且不改变极小点的位置, 因为正的常数倍不改变 $\arg\min$. 需要提防的是它的边界: 平方放大了离群点的作用, 一个被错误标注的样本可以显著拉动整条拟合曲线; 此外 E 的量纲是目标变量 t 的平方, 跨数据集比较时数值没有直观意义, 这也是定义 1.15要引入均方根误差的原因.

其中引入系数 1/2 是为了后续计算方便. 稍后我们将从概率论出发推导此误差函数 (参见小节 2.3.4). 注意, 这个误差函数是非负的, 当且仅当函数 $y(x_n, \mathbf{w})$ 正好通过每个训练数据点时, 其值等于零. 平方和误差函数的几何解

释如图 1.5所示.

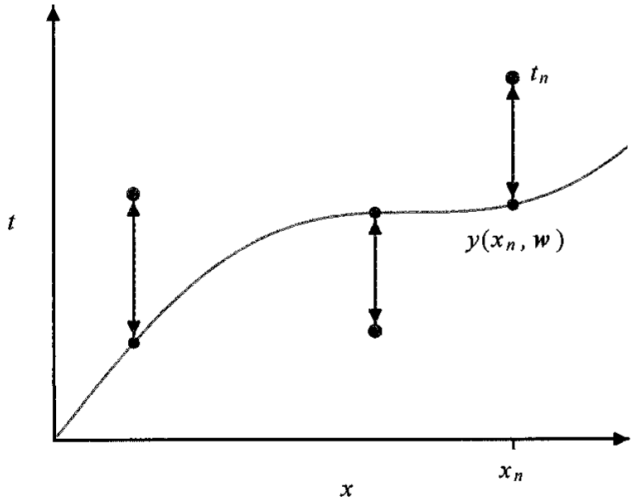


图 1.5: 平方和误差函数的几何解释 [该误差函数对应来自函数 $y(x, \mathbf{w})$ 的每个数据点的位移 (如垂直的绿色箭头所示) 平方和的一半]

我们可以通过选择能够使 $E(\mathbf{w})$ 尽可能小的 $\mathbf{w}$ 值来解决曲线拟合问题. 因为平方和误差函数是系数 $\mathbf{w}$ 的二次函数, 其对系数的导数是系数 $\mathbf{w}$ 的线性函数, 所以该误差函数的最小化有一个唯一解, 记为 $\mathbf{w}^*$, 可以通过解析形式求得封闭解 (也叫解析解)(参见节 4.1).

命题 1.1 (平方和误差函数的最小二乘解)

设训练集为 $\{(x_n, t_n)\}_{n=1}^N$,M 阶多项式函数如式(1.1)所示, 记 $\mathbf{\Phi}$ 为 $N \times (M+1)$ 矩阵, 其第 n 行第 j 列为 $x_n^j (j = 0, \cdots, M)$, 并记 $\mathbf{t} = (t_1, \cdots, t_N)^\mathsf{T}$. 若 $\mathbf{\Phi}$ 列满秩 (特别地, 这要求 $N \geqslant M+1$), 则式(1.2)中的 $E(\mathbf{w})$ 在 $\mathbb{R}^{M+1}$ 上有唯一全局极小点, 它由正规方程

$$\mathbf{w}^* = \left(\mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}\right)^{-1}\mathbf{\Phi}^\mathsf{T}\mathbf{t}$$

给出, 并且 $E(\mathbf{w}^*)$ 就是 E 在 $\mathbb{R}^{M+1}$ 上的最小值.

♠

证明

(重构新增) 以 $\mathbf{\Phi}$ 记设计矩阵, 则对任意 $\mathbf{w} \in \mathbb{R}^{M+1}$ 有 $y(x_n, \mathbf{w}) = (\mathbf{\Phi}\mathbf{w})_n$, 于是

$$E(\mathbf{w}) = \frac{1}{2}\|\mathbf{\Phi}\mathbf{w} - \mathbf{t}\|^2 = \frac{1}{2}\left(\mathbf{w}^\mathsf{T}\mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}\mathbf{w} - 2\,\mathbf{w}^\mathsf{T}\mathbf{\Phi}^\mathsf{T}\mathbf{t} + \mathbf{t}^\mathsf{T}\mathbf{t}\right).$$

对 $\mathbf{w}$ 求梯度, 得

$$\nabla E(\mathbf{w}) = \mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}\mathbf{w} - \mathbf{\Phi}^\mathsf{T}\mathbf{t}, \qquad \nabla^2 E(\mathbf{w}) = \mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}.$$

先证唯一性.$\mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}$ 是半正定矩阵; 若 $\mathbf{\Phi}$ 列满秩, 则对任意 $\mathbf{v} \neq \mathbf{0}$ 有 $\mathbf{\Phi}\mathbf{v} \neq \mathbf{0}$, 从而

$$\mathbf{v}^\mathsf{T}\mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}\mathbf{v} = \|\mathbf{\Phi}\mathbf{v}\|^2 > 0,$$

故 $\nabla^2 E$ 正定,E 是严格凸函数, 其极小点若存在则唯一. 再证存在性并求出极小点. 令 $\nabla E(\mathbf{w}) = \mathbf{0}$, 即得正规方程 $\mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}\mathbf{w} = \mathbf{\Phi}^\mathsf{T}\mathbf{t}$. 由 $\mathbf{\Phi}$ 列满秩知 $\mathbf{\Phi}^\mathsf{T}\mathbf{\Phi}$ 可逆, 故 $\mathbf{w}^* = (\mathbf{\Phi}^\mathsf{T}\mathbf{\Phi})^{-1}\mathbf{\Phi}^\mathsf{T}\mathbf{t}$ 是唯一的驻点. 最后, 严格凸函数在驻点处取全局极小值, 所以 $E(\mathbf{w}^*) \leqslant E(\mathbf{w})$ 对一切 $\mathbf{w}$ 成立, 且等号仅在 $\mathbf{w} = \mathbf{w}^*$ 时成立.

□

注

(重构新增) “唯一解”这一结论的真正前提是 $\boldsymbol{\Phi}$ 列满秩, 而它等价于:$N \geqslant M+1$, 且 $x_1, \cdots, x_N$ 中至少有 $M+1$ 个互不相同的取值 (否则 $\boldsymbol{\Phi}$ 的列线性相关). 若这一条件不成立 (例如 $N < M+1$, 或用到的 x_n 取值过少), 则正规方程的解构成一个仿射集, 最小化问题有无穷多个解,$\mathbf{w}^*$ 不再唯一, 但 E 的最小值仍然存在. 这解释了本例的一个细节:小节 1.2.4中 $M=9$、$N=10$ 恰好处在临界情形 $N=M+1$, 自由度正好等于数据点个数, 因此可以精确插值; 若数据点少于自由度, 插值解更多, 系数的选取将更加任意, 这也是表 1.1中 $M=9$ 一列出现巨大系数的原因.

最终的多项式由函数 $y(x, \mathbf{w}^*)$ 给出.

1.2.4 模型复杂度

我们还面临选择多项式的阶数 M 的问题, 这将引出模型比较 (model comparison) 或模型选择 (model selection) 这一重要概念. 在图 1.6中, 我们展示了 4 个拟合实例, 它们分别使用阶数 $M=0, 1, 3, 9$ 的多项式来拟合图 1.4所示的数据集.

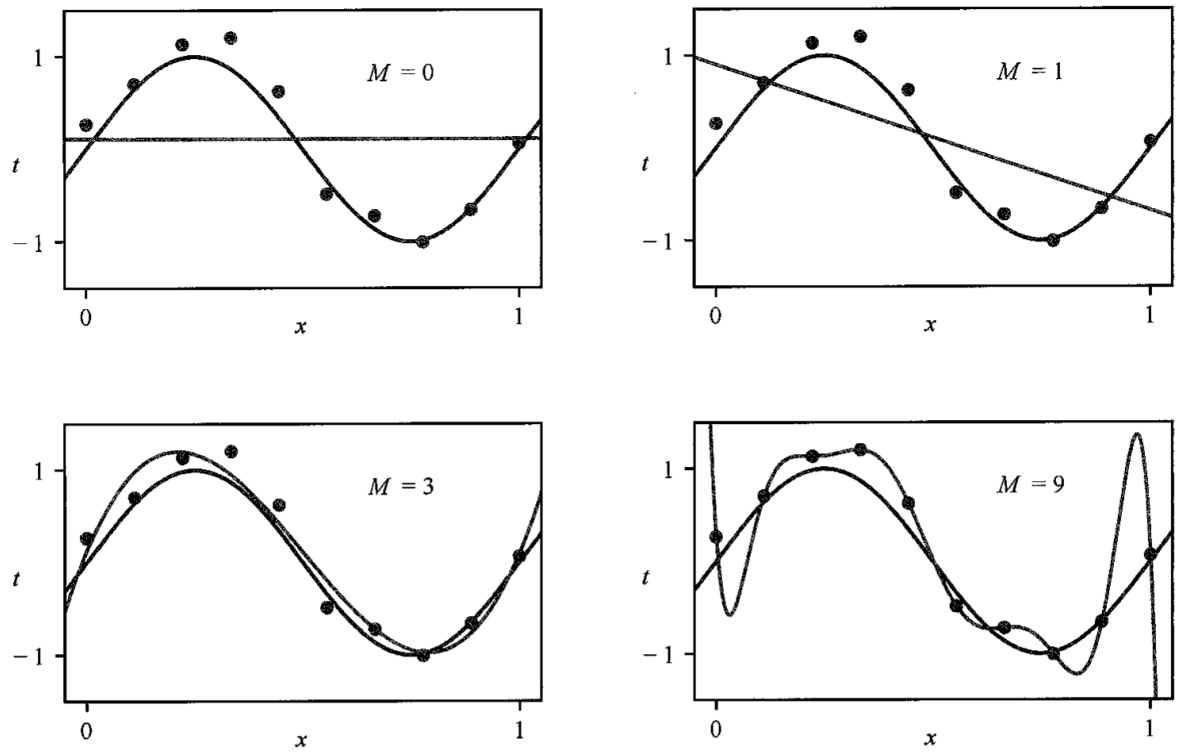


图 1.6: 具有不同阶数的多项式图示. 多项式如红色曲线所示. 这里通过最小化平方和误差函数 (见式(1.2)) 来拟合图 1.4所示的数据集

可以发现, 常数 ($M=0$) 和一阶 ($M=1$) 多项式对数据的拟合较差, 因此对函数 $\sin(2\pi x)$ 的表示较差. 三阶 ($M=3$) 多项式似乎对函数 $\sin(2\pi x)$ 给出了最佳拟合. 高阶 ($M=9$) 多项式得到了一个对训练数据完美的拟合. 事实上, 这个多项式曲线精确地穿过了每一个数据点, 使得误差 $E(\mathbf{w}^*)=0$. 然而, 拟合出的曲线却出现了剧烈的波动, 完全不能反映出函数 $\sin(2\pi x)$ 的真实形态. 这种现象称为过拟合 (over-fitting).

定义 1.14 (过拟合)

若随着模型复杂度的增加, 模型在训练集上的误差持续减小, 而在独立测试数据上的误差反而增大, 致使模型的预测能力下降, 则称该模型对训练数据发生了**过拟合** (over-fitting).

注

(重构新增) 过拟合的实质是模型把训练数据中的**随机噪声**也当作规律学习了.$M = 9$ 的多项式有 10 个可调系数, 而训练点恰有 10 个, 于是它可以用一条剧烈振荡的曲线精确穿过每一点 ($E(\mathbf{w}^*) = 0$), 而这条曲线在数据点之间、尤其是在区间两端完全偏离 $\sin(2\pi x)$. 条件是相对于数据量而言的: 同样的 $M = 9$, 在 $N = 15$ 或 $N = 100$ 时过拟合就明显减弱 (图 1.8). 因此“参数多必然过拟合”是不对的, 正确的说法是“参数相对于数据量过多时容易过拟合”;小节 1.2.5将说明如何在参数数量不变的前提下抑制这一现象, 而深度学习还会出现参数远超样本数却泛化良好的情形 (参见小节 9.3.2).

我们的目标是让模型获得良好的泛化能力, 使其能够对新的数据做出准确的预测. 为了定量地探究泛化性能与模型复杂度 M 之间的依赖关系, 我们可以引入一个独立的测试集. 该测试集包含 100 个数据点, 其生成方式与训练集相同. 针对每一个 M 值, 我们不仅可以计算出模型在训练集上的残差 $E(\mathbf{w}^*)$(式(1.2)), 还可以计算出其在测试集上的残差 $E(\mathbf{w}^*)$. 与评估误差函数 $E(\mathbf{w})$ 相比, 有时使用均方根 (Root Mean Square, RMS) 误差更为方便, 均方根误差定义如下:

定义 1.15 (均方根误差)

对包含 N 个数据点的数据集与模型 $y(\cdot, \mathbf{w})$, 称

$$E_{\mathrm{RMS}} = \sqrt{\frac{1}{N}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2} \tag{1.3}$$

为**均方根误差** (Root Mean Square error).

注

(重构新增) 两个修正各有明确目的: 除以 N 使不同大小的数据集可以在同一基准下比较 (否则误差和会随 N 增大而增大, 训练集与测试集的数值不可比); 开平方使 E_{RMS} 与目标变量 t 具有相同的量纲, 因而数值可以被直接解读为“平均偏差多少个单位”. 需要强调,E_{RMS} 与 E 只差一个由 N 决定的单调变换, 故对同一数据集而言两者的极小点相同; 改变的是**可读性**与**可比性**, 不是最优解.图 1.7与图 1.10中的纵轴都用 E_{RMS}, 正是为了把训练集与测试集放在同一把尺子上比较.

公式中的 $1/N$ 是为了让不同大小的数据集能够在相同的基准下进行比较, 而求平方根则是为了确保 E_{RMS} 是在与目标变量 t 相同的尺度上 (以相同的单位) 进行测量的.图 1.7展示了不同 M 值的训练集和测试集的 RMS 误差图. 测试集上的 RMS 误差反映了我们根据新观测数据 x 预测其对应 t 值的能力.

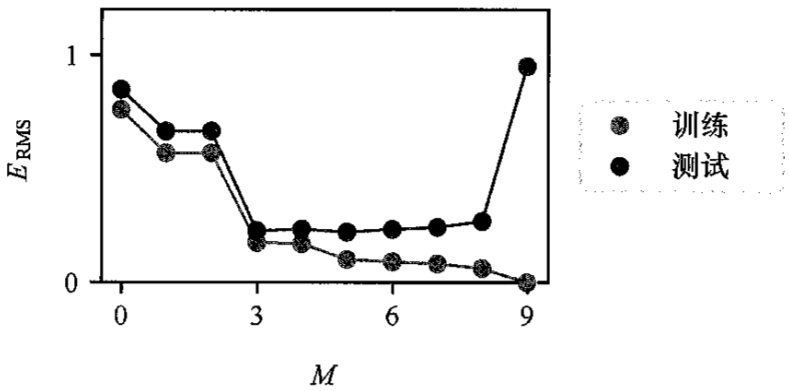


图 1.7: 由式(1.3)定义的均方根误差图 (在训练集和独立的测试集上对 M 的各个值进行评估)

从图 1.7中可以看出, 当 M 值较小时测试集误差较大, 这是因为此时的多项式模型灵活性不足, 无法捕捉函数 $\sin(2\pi x)$ 中的振荡. 当 M 取值在 $[3, 8]$ 时, 测试集误差较小, 同时这些模型也能合理地表示出数据的生成函数 $\sin(2\pi x)$, 如图 1.6中 $M = 3$ 时所示.

当 $M = 9$ 时, 训练集误差变为零. 这是符合预期的, 因为该多项式包含 10 个自由度 (对应 10 个系数 $w_0, \cdots, w_9$), 所以可以精确地调整到训练集中的 10 个数据点. 然而, 如图 1.6和图 1.7所示, 测试集误差变得极大,

函数 $y(x, \mathbf{w}^*)$ 表现出剧烈振荡.

这可能看起来很矛盾, 因为一个给定阶数的多项式包含了所有更低阶的多项式作为特例. 因此,$M = 9$ 的多项式理应能够产生至少与 $M = 3$ 的多项式一样好的结果. 此外, 我们或许会认为, 预测新数据的最佳模型就应该是生成这些数据的真实函数 $\sin(2\pi x)$ 本身 (我们后续将验证这一点). 同时, 我们知道 $\sin(2\pi x)$ 的幂级数展开式中包含了所有阶数的项, 所以我们会很自然地推断, 随着模型复杂度 M 的增加, 预测效果应该会持续提升.

命题 1.2 (训练误差随阶数单调不增)

设 $E_M^* = \min\limits_{\mathbf{w}\in\mathbb{R}^{M+1}} E(\mathbf{w})$ 为 M 阶多项式模型在给定训练集上的最小平方和误差. 则

$$E_0^* \geqslant E_1^* \geqslant E_2^* \geqslant \cdots,$$

即最小训练误差随阶数单调不增. 特别地,$E_9^* \leqslant E_3^*$.

♠

证明

(重构新增) 设 $M < M'$. 任取 M 阶系数 $\mathbf{w} = (w_0, \cdots, w_M)$, 令 $\tilde{\mathbf{w}} = (w_0, \cdots, w_M, 0, \cdots, 0) \in \mathbb{R}^{M'+1}$, 则对一切 x 有

$$\sum_{j=0}^{M'} \tilde{w}_j x^j = \sum_{j=0}^{M} w_j x^j,$$

即两个系数向量定义同一个多项式函数, 因而在训练集上给出同一组预测值,$E(\tilde{\mathbf{w}}) = E(\mathbf{w})$. 于是

$$\left\{E(\mathbf{w}) : \mathbf{w} \in \mathbb{R}^{M+1}\right\} \subseteq \left\{E(\mathbf{w}) : \mathbf{w} \in \mathbb{R}^{M'+1}\right\},$$

在更大的集合上取下确界不会变大, 故 $E_M^* \geqslant E_{M'}^*$. 由 M, M' 的任意性得单调不增; 取 $M = 3$、$M' = 9$ 即得 $E_9^* \leqslant E_3^*$.

□

注

(重构新增) 这条命题澄清了表面矛盾的确切含义:M 增大只会让**训练**误差不变或下降, 绝不会使它上升; 因此图 1.7中 $M = 9$ 时测试误差的剧烈上升, 只能来自泛化行为的恶化, 而不是训练集拟合能力的退步. 换言之, "高阶模型更灵活" 与 "高阶模型泛化更差" 并不冲突——灵活性保证它能把训练误差压得更低, 也正因为如此它更容易把噪声一起拟合进去. 条件的边界同样重要: 命题只比较训练误差, 并未断言测试误差也单调; 它也不要求解唯一 ($N < M + 1$ 时命题 1.1的唯一性条件失效, 但下确界仍存在, 命题的证明只用到了函数族的包含关系).

通过观察表 1.1中不同阶数多项式拟合得到的系数 $\mathbf{w}^*$, 我们可以更深入地了解这个问题.

表 1.1: 不同阶数多项式的系数 $\mathbf{w}^*$. 注意观察随着多项式阶数的增加, 系数的变化幅度是如何急剧增大的

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.11	0.90	0.12	0.26
w_1^*		−1.58	11.20	−66.13
w_2^*			−33.67	1,665.69
w_3^*			−22.43	−15,566.61
w_4^*				76,321.23
w_5^*				−217,389.15
w_6^*				370,626.48
w_7^*				−372,051.47
w_8^*				202,540.70
w_9^*				−46,080.94

我们注意到, 随着 M 的增加, 系数的幅度越来越大. 特别是当 $M = 9$ 时, 为了让对应的多项式曲线能精准地穿过每一个数据点, 这些系数被精细地调整到了很大的正值或负值. 但在数据点之间, 尤其是在数据范围的两端附近, 曲线却出现了大幅度的摆动, 正如我们在图 1.6中看到的那样. 直观地看, 当多项式模型具有较大的 M 值时, 它变得更加灵活, 从而更容易受到目标值上随机噪声的影响, 并过度拟合了这些噪声.

为进一步探究该现象, 我们观察随着数据集大小的变化, 模型学习效果的相应变化, 如图 1.8所示. 可以看出,

当模型复杂度固定时, 数据集越大, 过拟合现象就越不明显. 换言之, 数据量越大, 我们就能用越复杂 (即更灵活) 的模型去拟合数据.

经典统计学中有一条常用的启发式经验: 训练数据点的数量应至少是模型中可学习参数数量的若干倍 (比如 5 倍或 10 倍). 然而, 我们在本书继续探讨深度学习后会发现, 即使模型参数的数量远远超过训练数据点的数量, 也一样可以获得非常出色的结果 (参见小节 9.3.2).

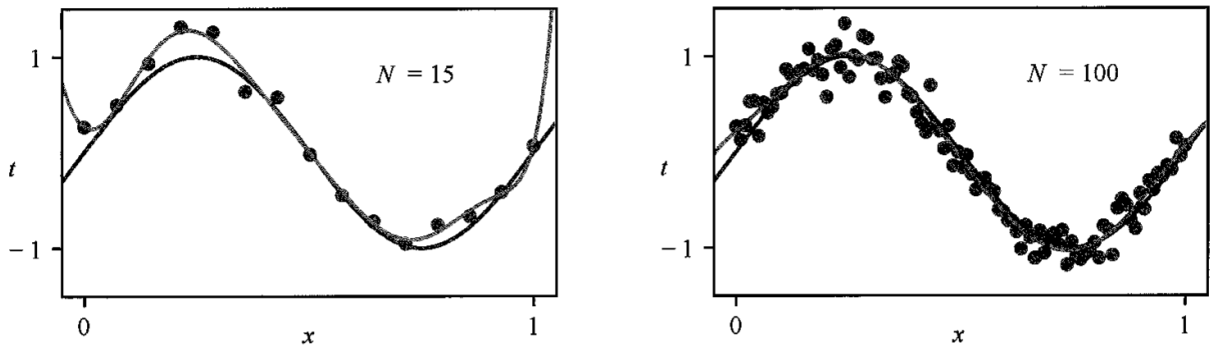


图 1.8: 使用 $M = 9$ 的多项式最小化平方和误差函数所获得的解决方案 (左图为拟合 $N = 15$ 个数据点所获得的解, 右图为拟合 $N = 100$ 个数据点所获得的解. 可以看到, 大的数据集可以减少过拟合)

1.2.5 正则化

根据可用训练集的大小来限制模型中参数的数量, 其结果有些不尽如人意. 而根据待解决问题的复杂性来选择模型的复杂性似乎更合理. 作为限制参数数量的替代方案, 正则化 (regularization) 技术经常被用于控制过拟合现象, 它通过向误差函数添加一个惩罚项来抑制系数取值过大. 最简单的惩罚项采用所有系数的平方和的形式, 误差函数变为

定义 1.16 (正则化)

设 $\lambda \geqslant 0$. 称通过向误差函数添加惩罚项来抑制系数取值过大的技术为**正则化** (regularization), 并称

$$\tilde{E}(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\lambda}{2}\sum_{j=0}^{M} w_j^2 \tag{1.4}$$

为**正则化误差函数**. 其中 $\|\mathbf{w}\|^2 = \mathbf{w}^{\mathrm{T}}\mathbf{w} = w_0^2 + w_1^2 + \cdots + w_M^2$, 系数 λ 控制着正则化项与平方和误差项之间的相对重要性.

♣

注

(重构新增) 动机是给“系数变大”本身标价: 既然过拟合表现为系数被调整到很大的正负值 (表 1.1), 就在目标函数中直接惩罚系数的幅度, 使模型在拟合数据与保持系数较小之间折中. λ 是这一折中的价格: $\lambda = 0$ 时, 式(1.4)退化为式(1.2); λ 增大时系数被压向零, 模型的有效复杂度下降. 条件上有两点容易忽略. 其一, 惩罚项通常**不含** w_0: 若惩罚 w_0, 则结果会依赖目标变量原点如何选取——把 t 整体平移一个常数, 本应只改变 w_0 而不改变曲线形状, 惩罚 w_0 却会使最优解随平移而变. 当然也可以惩罚 w_0, 但须为它单独配置系数 (参见节 9.2). 其二, 要求 $\lambda \geqslant 0$: 若允许 $\lambda < 0$, 惩罚项变成奖励项, 问题可能无下界.

注意, 正则化项中通常不包含系数 w_0, 因为如果包含 w_0, 就会导致最终结果受到目标变量所选原点的影响 (Hastie, Tibshirani, and Friedman, 2009). 当然也可以包含 w_0, 但需要单独为其配置一个正则化系数 (参见节 9.2). 同样, 我们可以求得式(1.4)中误差函数的精确闭式解 (见练习 4.2). 在统计学文献中, 这类方法被称为收缩方法 (shrinkage), 因为它们会使系数的值缩小. 在神经网络领域, 这种方法称为权重衰减 (weight decay), 因为神经网络中的参数通常称为权重, 而这种正则化手段会促使这些权重向零衰减.

图 1.9展示了采用正则化误差函数 (式(1.4)) 对 9 阶 ($M = 9$) 多项式进行拟合的结果, 所用数据集与之前相同. 观察发现, 当取 $\ln\lambda = -18$ 时, 过拟合现象被有效抑制, 此时的拟合曲线与目标函数 $\sin(2\pi x)$ 相当接近. 反之, 若 λ

取值过大, 则会导致欠拟合, 如图 1.9中 $\ln\lambda = 0$ 的情形.

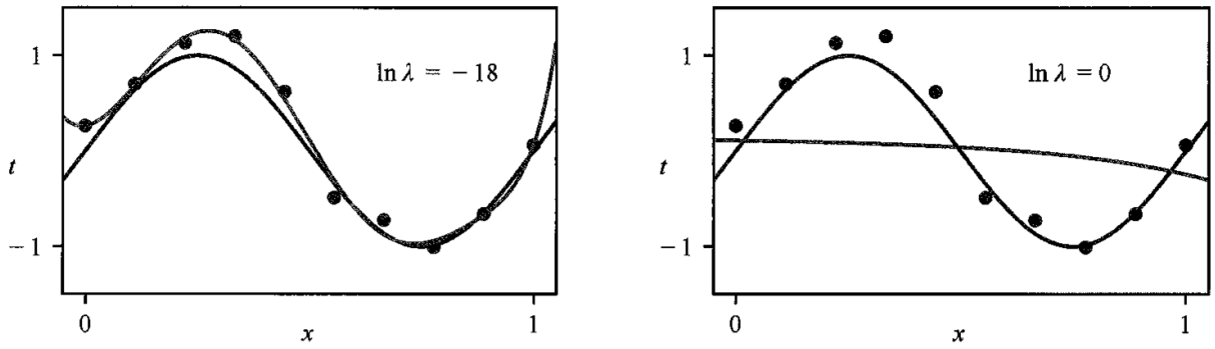


图 1.9: 使用正则化误差函数 [式(1.4)] 对图 1.4中数据集进行拟合所得的 $M = 9$ 阶多项式的曲线. 其中正则化参数 λ 取两个值, 分别对应于 $\ln\lambda = -\infty$ 和 $\ln\lambda = 0$. 无正则化项的情况, 即 $\lambda = 0$(对应 $\ln\lambda = -\infty$) 的情况, 显示在图 1.6的右下图中

表 1.2给出了不同 λ 值下拟合得到的多项式系数, 这些数值表明, 正则化确实发挥了预期作用, 有效减小了系数的幅度.

表 1.2: 正则化参数 λ 取不同值时,$M = 9$ 的多项式模型的系数 $\mathbf{w}^*$. 注意,$\ln\lambda = -\infty$ 对应未采用正则化的模型, 也就是图 1.6右下图中的拟合结果. 可见, 随着 λ 值的增大, 系数的量级会减小

	$\ln\lambda = -\infty$	$\ln\lambda = -18$	$\ln\lambda = 0$
w_0^*	0.26	0.26	0.11
w_1^*	−66.13	0.64	−0.07
w_2^*	1,665.69	43.68	−0.09
w_3^*	−15,566.61	−144.00	−0.07
w_4^*	76,321.23	57.90	−0.05
w_5^*	−217,389.15	117.36	−0.04
w_6^*	370,626.48	9.87	−0.02
w_7^*	−372,051.47	−90.02	−0.01
w_8^*	202,540.70	−70.90	−0.01
w_9^*	−46,080.94	75.26	0.00

通过绘制训练集和测试集的 RMS 误差 (式(1.3)) 与 $\ln\lambda$ 的关系, 可以看出正则化项对泛化误差的影响, 如图 1.10所示. 可以看到,λ 现在控制了模型的有效复杂性, 从而决定了过拟合的程度.

1.2.6 模型选择

在本例中,λ 作为一个超参数 (hyperparameter), 它的值在基于误差函数最小化来确定模型参数 $\mathbf{w}$ 的过程中始终保持不变.

定义 1.17 (超参数)

在依据训练数据最小化误差函数以确定模型参数 $\mathbf{w}$ 的过程中保持固定、而不由该最小化过程决定的量, 称为**超参数** (hyperparameter).

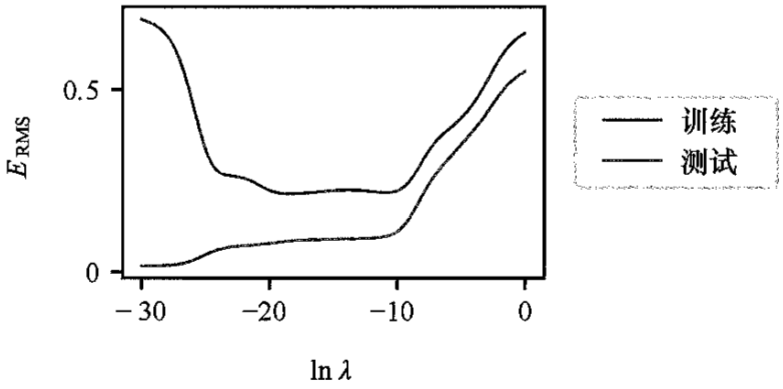


图 1.10: $M = 9$ 的多项式的 RMS 误差与 $\ln\lambda$ 的关系

注

(重构新增) 区分参数与超参数的标准是“由谁的最小化决定”, 而不是“是否可以被学习”:$\mathbf{w}$ 由最小化 E 或 $\tilde{E}$ 得到,λ 与阶数 M 则必须在优化之前就选定, 随后在整个训练过程中保持不变. 这一安排的必要性来自下一段: 若把 λ 也交给同一个最小化过程, 它会被推向 0(命题 1.3), 正则化随即失效. 因此超参数的合理取值只能借助**另一个**判据来确定, 例如在一份独立的验证数据上的误差, 这正是后面引入验证集与交叉验证的原因.

需要注意的是, 我们不能通过同时对 $\mathbf{w}$ 和 λ 最小化误差函数的方式来简单地确定 λ 的取值, 因为这样会导致 λ 趋近于 0, 从而产生一个在训练集上误差极小甚至为零的过拟合模型.

命题 1.3 (正则化系数不能与参数联合优化)

固定训练集, 并允许 $\lambda \geqslant 0$ 变动. 则

$$\min_{\mathbf{w}\in\mathbb{R}^{M+1}} \min_{\lambda\geqslant 0} \tilde{E}(\mathbf{w},\lambda) = \min_{\mathbf{w}\in\mathbb{R}^{M+1}} E(\mathbf{w}),$$

且除 $\mathbf{w}^* = \mathbf{0}$ 的退化情形外, 最优的 λ 只能取 0. 换言之, 把 λ 与 $\mathbf{w}$ 一起优化, 正则化项必然被完全取消.

证明

(重构新增) 注意 $\tilde{E}(\mathbf{w},\lambda) = E(\mathbf{w}) + \dfrac{\lambda}{2}\|\mathbf{w}\|^2$ 对固定的 $\mathbf{w}$ 关于 λ 是仿射函数, 其斜率为 $\dfrac{1}{2}\|\mathbf{w}\|^2 \geqslant 0$. 故对每个 $\mathbf{w}$ 有

$$\min_{\lambda\geqslant 0} \tilde{E}(\mathbf{w},\lambda) = \tilde{E}(\mathbf{w},0) = E(\mathbf{w}),$$

最小值在 $\lambda = 0$ 处取到; 若 $\mathbf{w} \neq \mathbf{0}$, 则该最小值只在 $\lambda = 0$ 处取到. 对 $\mathbf{w}$ 再取下确界, 即得所述等式与“最优 λ 只能为 0”.

□

注

(重构新增) 结论与原书的表述“λ 会趋近于 0”一致, 但这里的论证更彻底: 不是“趋近”, 而是在无约束的联合优化下最优值**恰为** 0——惩罚项只会增加目标函数值, 任何正的 λ 都不如 $\lambda = 0$ 划算. 这解释了为什么正则化系数必须被排除在训练目标之外. 条件方面, 论证只用到 $\|\mathbf{w}\|^2 \geqslant 0$ 与 $\tilde{E}$ 关于 λ 的单调性; 若把 λ 限制在一个正的区间并另加约束, 结论会改变, 但那样的约束已不属于“最小化误差函数”本身. 同理, 阶数 M 也不能与 $\mathbf{w}$ 联合优化: 由于 E^*_M 随 M 单调不增 (命题 1.2), 联合优化必然选出最大的 M.

类似地, 多项式的阶数 M 也是模型的一个超参数, 单纯地优化训练集误差关于 M 的取值会导致 M 过大, 同样会引发过拟合问题. 因此, 我们需要找到一种有效的方法来确定这些超参数的合理取值.节 1.1的讲述提供了一种简单的思路, 即将已有的数据集划分为训练集和验证集 (validation set) [也称为保留集 (hold-out set) 或开发集 (development set)], 其中训练集用于确定模型系数 $\mathbf{w}$, 而我们最终选择在验证集上误差最小的模型. 如果使用有限规模的数据集多次迭代模型设计, 也可能会出现对验证集的过拟合现象. 为此, 通常需要预留出测试集, 用于对最终选定模型的性能进行评估.

定义 1.18 (验证集与测试集)

把可用数据划分为训练集与**验证集** (validation set) [也称为**保留集** (hold-out set) 或**开发集** (development set)]: 训练集用于确定模型系数 $\mathbf{w}$, 验证集用于在若干候选模型或超参数设置中挑选在它上面误差最小的一个. 另外预留的、在模型选定之前不使用的那部分数据称为**测试集** (test set), 用于对最终选定模型的性能进行评估.

♣

注

(重构新增) 这种三分法的逻辑是: 每个数据集只承担一项职责. 若用训练集挑选模型, 由于训练误差随复杂度单调不增 (命题 1.2), 被选中的必然是最复杂的模型; 若用测试集挑选模型, 测试集就被“用过”了, 它给出的性能评估会偏乐观, 不再能代表未见数据上的表现. 因此验证集的作用是**承担选择的代价**, 测试集的作用是**给出一次性的评估**. 条件上, 验证集与测试集都必须与训练集独立同分布; 此外, 若反复用同一个验证集迭代模型设计, 验证集本身也会被过拟合, 此时需要再引入新的数据或改用交叉验证 (定义 1.19).

在某些实际应用场景中, 可用于模型训练和测试的数据量往往较为有限. 为了构建性能良好的模型, 我们希望尽量充分地利用一切可获取的数据进行训练. 然而, 如果验证集的规模过小, 则会导致对模型预测性能的评估存在较大偏差. 交叉验证 (cross-validation) 技术为解决这一困境提供了一种有效途径, 如图 1.11所示. 该方法允许将 $(S-1)/S$ 的数据用于模型训练, 与此同时, 利用全部数据来评估模型的性能. 当数据资源极度匮乏时, 还可以考虑 $S=N$ 的极端情况, 其中 N 表示数据点的总数, 这时的交叉验证就演变成了留一法 (leave-one-out).

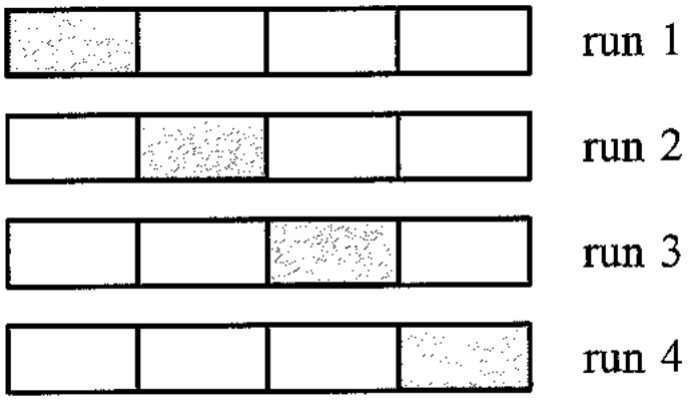


图 1.11: S 折交叉验证技术 (这里以 $S=4$ 为例) 的操作, 首先将可用数据分成 S 个等大小的子集. 然后使用 $S-1$ 个子集来训练一组模型, 并在剩余的子集上评估模型性能. 接下来, 将 S 个子集中的每个子集作为保留集 (图中红色块所示) 重复上述过程, 最后对 S 次运行的性能评分取平均值

定义 1.19 (交叉验证)

把可用数据分成 S 个等大小的子集. 对 $s=1,\cdots,S$, 用其余 $S-1$ 个子集训练模型, 并在第 s 个子集上评估其性能; 最后对 S 次运行的性能评分取平均值, 以此作为该模型或超参数设置的评分. 这一过程称为 S **折交叉验证** (S-fold cross-validation). 当 $S=N$(N 为数据点的总数) 时, 每次只留出一个数据点作评估, 称为**留一法** (leave-one-out).

♣

注

(重构新增) 交叉验证解决的是“验证集太小则评估偏差大、验证集太大则训练数据不足”这对矛盾: 每一折都让 $(S-1)/S$ 的数据参与训练, 同时 S 次评估覆盖了全部数据, 因此既充分利用了数据, 又给出了比单次划分更稳定的评分. 代价是训练次数增加为 S 倍 (图 1.11中的四次运行对应 $S=4$). 当 $S=N$ 时训练次数等于样本数, 评估偏差最小但计算量最大; 实际中常取 $S=5$ 或 $S=10$. 此外, 交叉验证假定各折数据可交换, 若数据本身有分组或时间顺序, 随机分折会破坏独立性, 需要改用按组或按时间的划分.

交叉验证的主要缺点在于所需的训练次数增加了 S 倍, 这对于训练过程本身计算成本较高的模型来说是一

个大问题. 交叉验证等使用独立数据评估性能的技术还存在另一个问题, 即对于单个模型可能存在多个复杂度超参数 (例如, 可能有多个正则化超参数). 在最坏的情况下, 探索这些超参数设置的最佳组合可能需要指数级数量的训练次数. 现代机器学习的前沿领域需要非常大的模型和大规模训练数据集. 因此, 超参数设置的探索空间有限, 很大程度上依赖于从小模型获得的经验和启发式方法.

这个将多项式模型拟合到基于正弦函数生成的合成数据集的简单示例, 阐释了机器学习领域的一些核心概念, 在后续章节我们将进一步用这个示例进行讨论. 然而, 现实中的机器学习应用往往要复杂得多: 首先, 用于模型训练的数据集规模可能会非常庞大, 数据量通常会高出几个数量级; 其次, 模型的输入变量的数量通常也会显著增加, 例如, 在图像分析领域, 输入变量的规模可达百万量级, 并且还可能伴随着多个输出变量. 在这些应用中, 将输出映射到输入的可学习函数通常由一类被称为神经网络的模型来表征, 这些模型往往具有海量的参数, 其数量甚至可以达到千亿的规模. 此时, 误差函数将表现为这些模型参数的复杂非线性函数, 无法再通过闭式解的方式进行最小化, 而必须借助迭代优化的方法, 利用误差函数关于模型参数的导数信息来逐步逼近最优解. 这对计算机硬件提出了更高的要求, 并且在实际应用中不可避免地会引入相当高的计算成本.

1.3 机器学习简史

机器学习拥有悠久且丰富的发展历史, 涌现了多种不同的研究路径. 本节将着重介绍基于神经网络的机器学习方法的发展脉络, 因为神经网络构成了深度学习的基石, 并且在实际应用中已被证明是最为有效的机器学习方法.

神经网络模型最初受到人类和其他哺乳动物大脑信息处理机制的启发. 大脑中处理信息的基本单元是称为神经元的电活性细胞, 如图 1.12所示. 当神经元放电时, 它会沿着轴突发送电脉冲, 到达称为突触的连接点, 这些连接点与其他神经元形成连接. 在突触处会释放称为神经递质的化学信号, 这些信号可以刺激或抑制后续神经元放电.

定义 1.20 (神经元与突触)

大脑中处理信息的基本单元是称为**神经元** (neuron) 的电活性细胞. 神经元放电时会沿**轴突** (axon) 发送电脉冲, 到达称为**突触** (synapse) 的连接点, 这些连接点与其他神经元形成连接; 在突触处释放的、可以刺激或抑制后续神经元放电的化学信号称为**神经递质**.

♣

注

(重构新增) 这里的描述是生物机制的极度简化: 真实的神经元有复杂的树突整合、多种离子通道与时间动态, 而本书只保留“加权输入—阈值放电—突触强度可变”这三件事. 这样做的动机是: 只有把生物细节剥离, 才能得到一个可以用少数几个实数 (权重) 描述、并且可能通过数据调整的数学模型. 需要注意“兴奋 / 抑制”的作用方向: 突触强度可以为正 (兴奋) 也可以为负 (抑制), 因此后文的权重 w_i 允许取负值, 这是模型能够表示“此消彼长”关系的前提. 至于“突触强度的变化是学习与记忆的机制”这一生物学论断, 本书只用它作为引入可调参数的动机, 并不把它当作需要证明的命题.

人脑总共包含约 900 亿个神经元, 每个神经元平均与其他数千个神经元通过突触相连, 形成了一个包含约 100 万亿 (10^{14}) 个突触连接的复杂网络. 如果某个神经元接收到来自其他神经元放电的足够强度的刺激, 那么它自身也可能被触发放电. 然而, 某些突触具有负向调节或抑制作用, 即输入神经元的放电会降低输出神经元放电的可能性. 一个神经元能否引发另一个神经元的放电取决于突触的连接强度, 而这些突触的连接强度的动态变化正是大脑存储信息和进行经验学习的关键机制.

神经元的这些属性已被抽象为简单的数学模型, 即人工神经网络 (Artificial Neural Network, ANN), 它们成为计算学习方法的基础 (McCulloch and Pitts, 1943). 诸多此类模型通过对其他神经元的输出的线性组合来描述单个神经元的特性, 然后使用非线性函数进行转换. 该过程可以用数学表达式表示为:

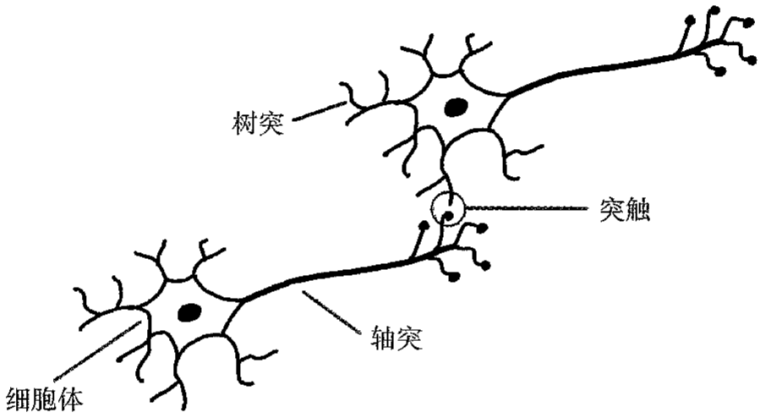


图 1.12: 人脑中两个神经元的示意图 (这些电活性细胞通过称为突触的连接点进行通信, 这些连接点随着神经网络的学习而变化)

定义 1.21 (人工神经网络)

把神经元的上述属性抽象成的数学模型称为**人工神经网络** (Artificial Neural Network, ANN). 其中每个单元接收 M 个输入 $x_1,\cdots,x_M$, 带有一组权重 $w_1,\cdots,w_M$ 与一个非线性函数 $f(\cdot)$, 并按

$$a=\sum_{i=1}^{M} w_i x_i \tag{1.5}$$

$$y=f(a) \tag{1.6}$$

计算其输出. 其中 $x_1,\cdots,x_M$ 表示与向该神经元发送连接的其他神经元的活动相对应的 M 个输入,$w_1,\cdots,w_M$ 是**权重** (weight), 表示相关突触的强度; 量 a 称为**预激活** (pre-activation), 非线性函数 $f(\cdot)$ 称为**激活函数** (activation function), 输出 y 称为**激活** (activation). ♣

注

(重构新增) 这个模型抓住了生物神经元的两件事: 先对所有输入作加权求和 (线性组合), 再作一次非线性变换. 前者对应“突触强度决定影响力”, 后者对应“达不到阈值就不放电”. 缺了非线性, 多层叠加仍等价于单层线性模型 (定义 1.11), 网络就失去了表达能力; 因此 f 的非线性是整个结构的要害. 条件上要注意式(1.5)中没有偏置项: 只有在补充一个恒为 1 的输入并把对应权重当作阈值时, 才能表示“总加权输入超过某个非零阈值才放电”的情形. 此外权重 w_i 可正可负, 这正是前面所说兴奋性与抑制性突触的数学对应.

可以看出, 式(1.1)可视为这个模型的一个特例, 其中输入 x_i 由单一变量 x 的各次幂构成, 而激活函数 $f(\cdot)$ 退化为恒等函数 $f(a)=a$.

命题 1.4 (多项式函数是神经元模型的特例)

在定义 1.21中取

$$x_i=x^i \quad (i=1,\cdots,M), \qquad f(a)=a,$$

则该单元的输出为 $y=\sum_{i=1}^{M} w_i x^i$, 即式(1.1)中常数项取 $w_0=0$ 时的多项式函数. 若再补充一个恒等于 1 的输入 $x_0 \triangleq 1$ 并把 w_0 取为该输入的权重, 则式(1.1)的完整形式也是这个模型的特例. ♠

证明

(重构新增) 把 $x_i = x^i$ 与 $f(a) = a$ 代入式(1.5)与式(1.6), 得

$$a = \sum_{i=1}^{M} w_i x^i, \qquad y = f(a) = a = \sum_{i=1}^{M} w_i x^i,$$

这正是式(1.1)中求和从 $j = 1$ 开始的特殊情形, 也就是 $w_0 = 0$ 的情形. 若把输入编号推广为 $i = 0, 1, \cdots, M$ 并令 $x_0 = 1$, 则和式变为 $\sum_{i=0}^{M} w_i x^i = w_0 + \sum_{i=1}^{M} w_i x^i$, 与式(1.1)完全一致, 故完整的多项式函数同样是该模型的特例.

□

注

(重构新增) 这条命题说明: 多项式拟合并不是与神经网络并列的另一类方法, 而是神经网络在“输入取幂次、激活取恒等”时的退化情形. 因此节 1.2中关于过拟合、正则化与模型选择的全部讨论, 都可以原样搬到神经网络上——那里出现的困难并不来自“神经网络”这一新结构, 而来自“用有限数据拟合带参数的模型”这一共同处境. 边界也要说清: 命题只要求 f 为恒等函数, 此时单元退化为线性模型; 一旦 f 取非线性函数 (如式(1.7)的阶跃函数),y 就不再是输入的线性函数, 多项式便不再是它的特例. 这也解释了为什么后文的表达能力提升总是伴随着非线性激活的引入.

式(1.5)和式(1.6)给出的简单数学公式构成了 20 世纪 60 年代至今神经网络模型的基础, 其图形表示如图 1.13所示.

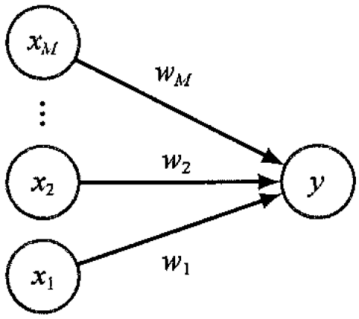


图 1.13: 一个简单的神经网络模型, 用于描述单个神经元的转换. 多项式函数可以视为这个模型的一个特例

1.3.1 单层网络

人工神经网络的历史大致可以根据其复杂程度划分为三个不同的阶段, 这种复杂性是通过处理的“层数”来衡量的. 由式(1.5)和式(1.6)描述的简单神经网络模型可以视为具有单层处理能力, 对应图 1.13中的单层连接结构. 在神经计算的历史长河中, 感知机模型 (Rosenblatt, 1962) 无疑占据着举足轻重的地位, 其激活函数 $f(\cdot)$ 是以下形式的阶跃函数:

定义 1.22 (感知机)

在定义 1.21中取激活函数为阶跃函数

$$f(a) = \begin{cases} 0, & a \leqslant 0, \\ 1, & a > 0, \end{cases} \tag{1.7}$$

所得的模型称为**感知机** (perceptron)(Rosenblatt, 1962).

♣

注

(重构新增) 阶跃函数把连续的预激活 a 压成两个取值, 因而感知机是一个“硬”判别器: 它只回答输入落在超平面 $a=0$ 的哪一侧, 而不给出置信程度. 这既符合“神经元要么放电要么不放电”的简化图像, 也使它的判别区域恰好是半空间, 这一点是后面“单层网络能力局限”的根源. 代价是它几乎处处不可微: f 在 $a\neq 0$ 处的导数恒为 0, 在 $a=0$ 处不可导, 因此无法用梯度信息来调整权重.Rosenblatt 的感知机学习算法靠的是“分错就按输入方向修正权重”这一离散规则, 而不是梯度下降; 这也解释了为什么小节 1.3.2要把激活函数换成连续可微的函数.

它可以视为一个简化的神经元放电模型, 在该模型中, 当且仅当总加权输入超过 0 的阈值时, 神经元才会放电. 感知机由 Rosenblatt(1962 年) 首创, 他开发了一种特定的训练算法, 该算法有一个有趣的特点, 即如果存在一组权重值使得感知机能够完美分类其训练数据, 则该算法保证在有限步数内收敛到这一解 (Bishop, 2006). 除了学习算法之外, 感知机还需要一个专用的模拟硬件, 如图 1.14所示. 典型的感知机配置有多个处理层, 但其中只有一层可以从数据中学习, 因此感知机被认为是一个“单层”神经网络.

图 1.14: Mark 1 感知机硬件示意图. 左图展示了使用简单的摄像系统获取输入, 其中输入场景 (在这种情况下是打印的字符) 被强光照射, 图像被聚焦在一个 20×20 的硫化镉光电管阵列上, 形成一幅原始的 400 像素大小的图像.Mark 1 感知机还配备了一个接线板, 如中间图所示, 它允许尝试不同的输入特征配置. 与现代数字计算机相比, 这些通常是随机连接的, 以展示感知机在没有精确连接的情况下进行学习的能力. 右图展示了一个可学习权重机架, 其中的每个权重都是使用旋转可变电阻器 (也称电位器) 实现的, 由电动机驱动, 从而允许通过学习算法自动调整权重值

起初, 大家惊叹于感知机能以类似大脑的方式从数据中学习. 然而, 这个模型也暴露出重大的局限性.Minsky and Papert(1969) 分析了感知机的特性, 并从数学上严格证明了单层网络的能力局限. 他们还推测, 这样的局限性将会扩展到具有多个可学习参数层的神经网络. 尽管这一猜想后来被证明是极其错误的, 但它在当时极大地打击了人们对神经网络模型的热情, 并直接导致了 20 世纪 70 年代和 80 年代初学术界和工业界对神经网络领域缺乏足够的兴趣和资金投入. 此外, 由于缺乏有效的训练神经网络的算法, 研究人员无法深入研究多层神经网络, 因此像感知机这样的技术仅适用于单层模型.

注

(重构新增) “单层网络的能力局限”可以用一个具体反例说清. 由式(1.5)与式(1.7), 感知机的判据是 $a=\sum_i w_i x_i$ 是否超过阈值, 若把阈值写成一个恒为 1 的输入的权重, 则判别边界是输入空间中的一张超平面, 因此单层感知机只能实现**线性可分**的分类. 考察异或问题: 输入 $(x_1,x_2)\in\{0,1\}^2$, 要求 $y=1$ 当且仅当 $x_1\neq x_2$. 设存在权重 w_1,w_2 与阈值 θ, 使判别“$a>\theta$ 判为正类”完全正确. 由 $(0,0)$ 属负类得 $0\leqslant\theta$; 由 $(1,0)$ 与 $(0,1)$ 属正类得 $w_1>\theta\geqslant 0$、$w_2>\theta\geqslant 0$, 从而 $w_1+w_2>2\theta\geqslant\theta$, 即 $(1,1)$ 必被判为正类, 与它属于负类矛盾. 故单层感知机无法表示异或. 这一反例说明 Minsky 与 Papert 所指的局限是真的, 但他们的外推——认为多层网络同样受限于此——是错的: 只要在输入与输出之间加入带非线性激活的隐藏层, 异或就可以被表示 (两个隐藏单元已足够). 这正是“深度”带来的表达能力, 也是小节 1.3.3的历史起点.

请注意, 尽管感知机已经从实际机器学习应用中消失, 但这个术语仍然流传下来, 因此现代神经网络有时也称为多层感知机 (Multi Layer Perceptron, MLP).

定义 1.23 (多层感知机)

把定义 1.21中的多个单元按层组织、使前一层的输出作为后一层的输入的模型, 整体称为**多层感知机** (Multi Layer Perceptron, MLP)——这是现代神经网络在历史上的另一名称.

♣

注

(重构新增) 这个名字容易引起误解: 现代的多层感知机既不是 Rosenblatt 的感知机 (Rosenblatt 的感知机单元取式(1.7)的阶跃函数), 也不限于单层; 它指的是由多层可调权重的单元堆叠而成、通常用反向传播训练的网络. "感知机"一词在这里只剩下"每个单元先加权求和再作非线性变换"这一层含义. 之所以仍沿用旧名, 是术语的历史惯性:小节 1.3.1中单层模型失宠之后, 多层版本沿用了同一名称以表明血统. 阅读时只需记住: 名字里的"感知机"不承诺任何能力上限, 真正的限制来自层数、激活函数与训练算法.

1.3.2 反向传播

对于训练具有多层可学习参数的神经网络问题, 其解决方案来自微分学的应用以及基于梯度的优化方法. 一项重要的改进是用具有非零梯度的连续可微激活函数取代了原有的阶跃函数 (式(1.7)). 另一项关键的改进是引入了可微的误差函数, 该函数能量化地评估在给定参数配置下模型对训练集中目标变量的预测效果. 我们在节 1.2中利用平方和误差函数 (式(1.2)) 进行多项式拟合时, 就已经接触到了此类误差函数的一个示例.

通过这些改进, 我们现在得到了一个误差函数, 它可以计算其关于网络模型中每一个参数的偏导数. 现在我们可以着手考虑具有多层参数的网络结构了.图 1.15展示了一个包含两层参数的简单网络模型. 位于中间层的节点称为隐藏单元 (hidden unit), 因为它们的值不会出现在训练数据中, 训练数据仅提供了模型的输入值和输出值.

定义 1.24 (隐藏单元)

位于网络中间层、其取值既不出现在训练数据的输入中也不出现在训练数据的目标中的单元, 称为**隐藏单元** (hidden unit).

♣

注

(重构新增) 隐藏单元的存在是"深度"与"可学习特征"的连接点: 训练数据只告诉网络输入是什么、输出应当是什么, 中间层应当表示什么完全由训练过程自行决定. 因此隐藏单元的值是模型内部的**隐变量**, 而不是被观测的量. 这一条件也带来解释上的困难: 正因为隐藏单元没有对应的监督信号, 我们无法直接检查"它学到的表示是否正确", 只能通过最终误差间接判断. 后文定义 1.30正是要从这个角度理解隐藏层的作用.

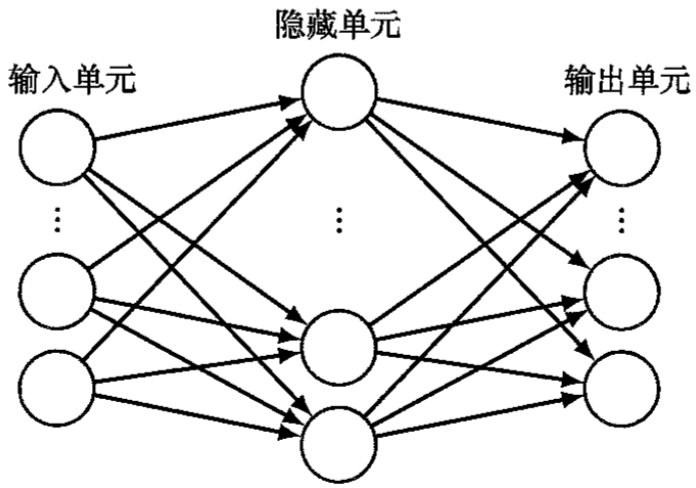


图 1.15: 一个具有两层参数的神经网络 (其中箭头表示神经网络中信息流的方向. 每个隐藏单元和每个输出单元的计算形式分别由式(1.5)和式(1.6)给出, 其中激活函数 $f(\cdot)$ 是可微的)

图 1.15中的每一个隐藏单元和每一个输出单元均计算形如式(1.5)和式(1.6)的函数值. 对于一组给定的输入值,

所有隐藏单元和输出单元的状态可以通过迭代地应用式(1.5)和式(1.6)进行前向计算, 在这个过程中, 信息沿着箭头所指的方向在网络中逐层向前传递. 由于这种信息流动的特点, 此类模型也被称作前馈神经网络 (feed-forward neural network).

定义 1.25 (前馈神经网络)

若网络中每个单元只从其前一层的单元接收输入、并把输出送往其后一层的单元, 信息沿单一方向逐层向前传递, 则称这样的网络为**前馈神经网络** (feed-forward neural network).

♣

注

(重构新增) "前馈"排除的是反馈连接: 一旦某一层的输出被送回到更靠前的层, 网络就具有了内部状态与时间动态, 那属于循环网络, 其计算与训练都要另行处理. 前馈结构保证给定输入后各单元的值可以被依次算出, 这正是前向计算能够"迭代地应用式(1.5)与式(1.6)"的原因. 这一结构条件在后文的误差反向传播中还会再用一次: 正因为信息流是单向的, 误差才可以沿相反方向逐层回传而不产生循环依赖. 若存在反馈连接, 反向传播就需要展开时间或改用其他算法.

为了训练这样的神经网络, 我们首先使用随机数生成器初始化参数, 然后使用基于梯度的优化技术进行迭代更新. 这需要计算误差函数的偏导数, 此操作可以通过误差反向传播 (error backpropagation) 过程中高效地完成 (参见节 8.1). 在反向传播中, 信息从输出端向后通过神经网络流向输入端 (Rumelhart, Hinton and Williams, 1986). 在机器学习领域, 有许多不同的优化算法利用了待优化的函数的梯度, 其中最简单和最常用的是随机梯度下降 (stochastic gradient descent) 法 (参见节 7.2).

定义 1.26 (误差反向传播)

计算误差函数关于网络各参数偏导数的一种高效方式称为**误差反向传播** (error backpropagation): 在该过程中, 信息从输出端向后通过神经网络流向输入端, 逐层给出误差函数对各层参数的偏导数 (Rumelhart, Hinton and Williams, 1986).

♣

注

(重构新增) 反向传播的实质是链式法则的系统化使用: 先算出误差对输出的导数, 再逐层用"上一层导数 × 本层局部导数"向前推进. 它的效率来自复用——同一条回传路径上的中间量被所有更靠前的参数共享, 因此一次反向遍历就能得到全部参数的梯度, 计算量与一次前向计算同阶, 而不是与参数个数同阶. 条件上, 它要求每个单元的输出对其输入可微, 这正是前面强调"用连续可微激活函数取代阶跃函数"的原因; 若激活函数像式(1.7)那样几乎处处导数为零, 回传的梯度会处处为零, 参数就无法更新. 此外反向传播只负责**算梯度**, 如何用梯度更新参数是优化算法的事 (参见节 7.2).

能够训练具有多层权重的神经网络是一项重大突破, 自 20 世纪 80 年代中期开始, 这一突破再度引发了对该领域的研究热潮. 这一时期, 研究重心逐渐超越了单纯对神经生物学机制的模拟, 而朝着构建更加严谨、更具原则性的理论基础的方向发展 (Bishop, 1995b). 特别是, 人们逐渐认识到, 概率论和统计学中的核心思想在神经网络及机器学习领域扮演着至关重要的角色. 一个关键的认识是, 从数据中进行学习的过程不可避免地会引入一些背景假设, 这些假设被称为先验知识 (prior knowledge) 或归纳偏置 (inductive bias). 这些先验知识可以显式地融入模型设计之中, 例如, 可以设计一种特定的神经网络结构, 使得皮肤病变的分类结果不依赖于图像中病变的具体位置; 或者, 它们也可以隐式地蕴含在模型的数学形式或其训练方法的选择中.

定义 1.27 (先验知识与归纳偏置)

从数据中进行学习的过程不可避免地会引入一些背景假设, 这些假设称为**先验知识** (prior knowledge) 或**归纳偏置** (inductive bias). 它们可以显式地融入模型设计 (例如设计一种使分类结果不依赖病变位置的网络结构), 也可以隐式地蕴含在模型的数学形式或其训练方法的选择中.

♣

注

(重构新增) 为什么学习必然需要假设？因为有限个数据点可以被无穷多个函数同时拟合，若不对“哪些函数更可取”作出约定，就无法从无穷多个候选里挑出一个. 先验知识正是这一约定的载体：它把假设空间从“所有函数”缩小到“我们愿意相信的函数”. 这也决定了偏置的两面性：偏置与数据规律相符时，学习效率大幅提高 (例如卷积网络的平移等变性之于图像)；不相符时，再多的数据也难以纠正. 因此“引入偏置”不是缺点而是必需，问题只在于偏置是否恰当——这一点在权重衰减的讨论中会以另一种形式出现 (参见节 9.2).

反向传播算法和基于梯度的优化技术显著提升了神经网络解决实际问题的能力. 然而，研究人员注意到，在具有多层结构的神经网络中，通常只有最后两层的权重参数能够学习到有效的信息. 除了少数例外，特别是用于图像分析的卷积神经网络模型 (LeCun et al., 1998)(详见节 10.1)，具有两层以上结构的网络模型鲜有成功的应用. 这意味着此类网络模型不能有效解决太复杂的问题. 为了在各种实际应用中获得令人满意的性能，人们通常要设计预处理步骤，将原始输入变量变换到一个新的特征空间中，希望在这个新的空间中机器学习任务能更容易解决. 这种预处理步骤有时也被称为特征提取 (feature extraction). 尽管这种人工特征提取的方法有时见效，但一个更理想的解决方案显然是直接从数据中自动学习出这些特征，而不是依赖于人工设计.

定义 1.28 (特征提取)

把原始输入变量变换到一个新的特征空间、以使机器学习任务在该空间中更容易解决的预处理步骤，称为**特征提取** (feature extraction).

♣

注

(重构新增) 特征提取的动机是“换一个坐标系看问题”：原始输入 (如像素) 与任务目标之间的关系往往极其复杂，而在经过恰当变换的空间里，同一任务可能近似线性可分. 历史上这一步由人手工设计，其质量直接决定了后续分类器的上限. 它的局限也正是深度学习登场的原因：人工设计特征需要领域知识，且难以针对每个任务重新设计；更理想的做法是让模型自己学出这层变换，也就是把特征提取纳入可学习参数之中. 条件上要注意，手工特征与学习到的表示并非互斥——前者可以作为后者的输入，或在数据极少时作为先验知识使用.

到了 2000 年，当时的神经网络方法再次达到了它们能力的极限. 研究人员开始探索替代神经网络的方法，如核方法、支持向量机、Gauss 过程等. 尽管一些热情的研究人员继续追求真正有效训练多层神经网络的目标，但神经网络再次失宠.

1.3.3 深度网络

神经网络发展的第三个阶段，也是当前阶段，始于 21 世纪的第二个十年. 一系列的突破使具有多层权重的神经网络得以有效训练，从而消除了这些技术先前的能力限制. 具有多层权重的网络称为深度神经网络，专注于这类网络的机器学习子领域称为深度学习 (deep learning)(LeCun, Bengio and Hinton, 2015).

定义 1.29 (深度神经网络与深度学习)

具有多层权重的网络称为**深度神经网络** (deep neural network)，专注于这类网络的机器学习子领域称为**深度学习** (deep learning).

♣

注

(重构新增) “深”是相对的说法：定义 1.23只要求有隐藏层，而“深度”强调的是隐藏层数量足够多、以致训练本身成为难点. 因此深度学习的核心问题不是“如何定义深度”，而是“如何让很多层的权重同时被有效训练”——这正是小节 1.3.2遗留、由本节后面的几项进展回答的问题. 条件上，深度并不自动带来好处：层数增加会使误差信号在反传中逐层衰减、使优化更难，也会放大过拟合风险. 后文提到的残差连接、大规模数据与正则化，正是为了让“深”在实践上可行而引入的配套手段.

深度学习发展历程中的一个关键性主题，是神经网络模型规模的显著扩张，这集中体现在模型参数数量的爆炸式增长上.20 世纪 80 年代，拥有数百或数千个参数的神经网络模型还较为普遍，但这一数字随后稳步攀升至数百万乃至数十亿规模. 而目前最前沿的模型，其参数数量更是达到了惊人的一万亿 (10^{12}) 量级. 拥有海量参数的神

经网络模型需要相应规模的庞大数据集进行训练, 才能确保在训练过程中为这些参数有效赋值. 这种庞大模型与海量数据的结合, 反过来也对模型训练过程的计算能力提出了更高的要求.

图形处理单元 (Graphics Processing Unit, GPU) 作为一种专为特定应用设计的处理器, 最初是为了满足视频游戏等应用中对高速图形渲染的需求而开发的. 随后人们发现 GPU 的架构能够很好地适配神经网络的训练需求. 具体而言, 神经网络中某一层的各个单元所执行的运算可以高度并行地进行, 这与 GPU 所提供的大规模并行计算架构高度契合 (Krizhevsky, Sutskever, and Hinton, 2012). 如今, 规模最大的那些模型的训练任务, 通常需要在包含数千块 GPU 的大规模计算集群上完成, 这些 GPU 之间通过专用的高速网络进行通信.

图 1.16描绘了历年来训练最先进的神经网络模型所需计算量的演变趋势, 图中显现出两个特征显著的增长阶段. 纵轴采用指数标度, 其单位为 petaflop/s-days. 其中, 一个 petaflop 代表每秒执行 10^{15} 次 (一千万亿次) 浮点运算, 而一个 petaflop/s 则表示每秒一千万亿次的浮点运算速率. 因此, 一个 petaflop/s-day 就代表以每秒一千万亿次的运算速率持续运行 24 小时所对应的计算量, 这大约等价于 10^{20} 次浮点运算. 由此可知, 图中最上方的水平线对应着高达 10^{24} 次的浮点运算量, 真是让人惊叹.

图 1.16中的两个直线段显示了两个指数级的增长阶段. 我们可以清楚地看到, 从感知机时代直至 2012 年前后, 计算需求的翻倍周期约为 2 年, 这与摩尔定律所预测的计算性能增长的总体趋势基本吻合. 然而, 自 2012 年步入深度学习时代以来, 计算需求再次呈现出指数级的增长态势, 但这一次的翻倍周期锐减至仅 3.4 个月, 这意味着计算能力每年增长高达 10 倍之多!

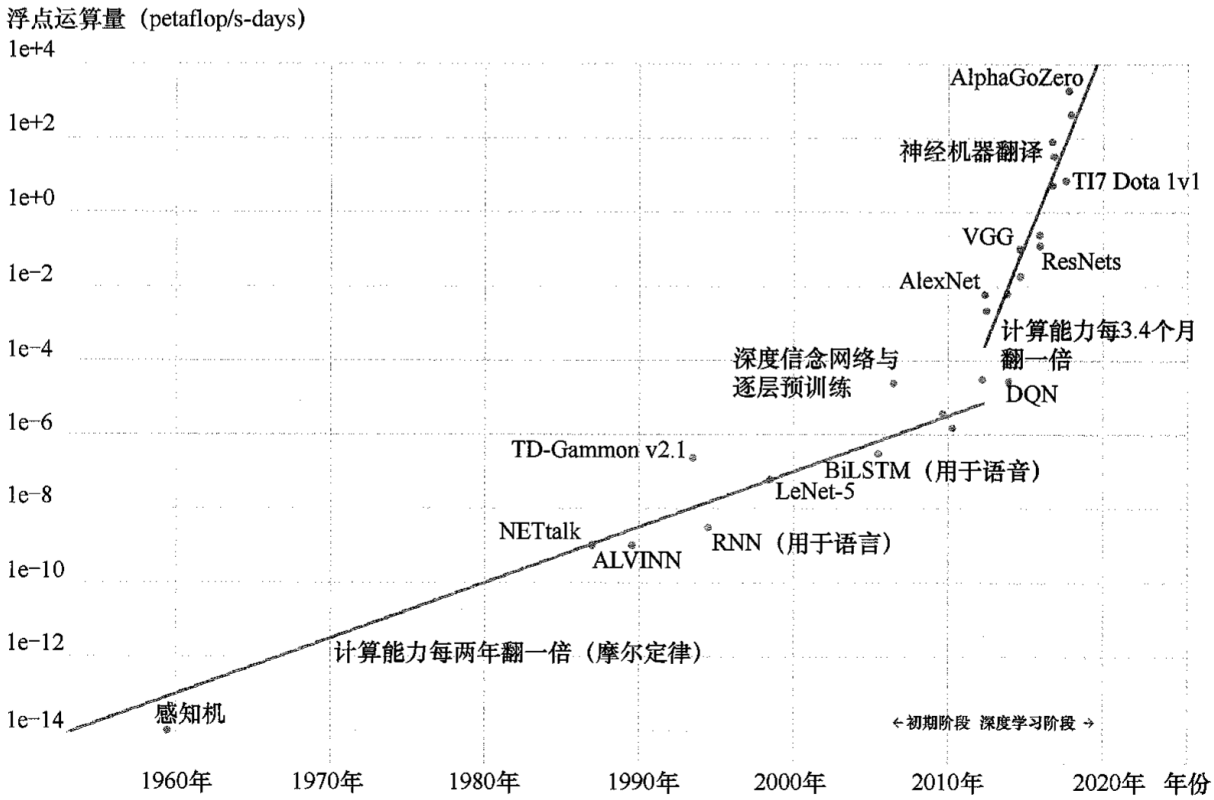


图 1.16: 训练最先进的神经网络所需的计算周期数 (以 petaflop/s-days 为单位) 与日期的关系, 这里显示了两个不同的指数增长阶段 (来自 OpenAI, 经许可使用)

人们常常发现, 通过架构创新或引入更复杂形式的归纳偏置而带来的性能改进, 很快就会被简单地通过扩大训练数据量替代, 同时伴随着相应的模型规模和用于训练的相关计算能力的扩展 (Sutton, 2019). 大语言模型不仅在特定任务上表现更优, 还可以用同一个训练好的神经网络解决更广泛范围的问题. 大语言模型就是一个显著的例子, 因为单个网络不仅拥有非凡的能力广度, 甚至还能够胜过为解决特定问题而设计的专业网络 (参见小节 12.3.5).

我们已经了解到, 网络的深度对于神经网络实现卓越性能至关重要. 要理解深度神经网络中隐藏层的作用, 一

种有效的视角是表示学习 (representation learning, 也称表征学习)(Bengio, Courville, and Vincent, 2012). 在这种视角下, 网络能够学习把原始输入数据转换成某种新的、富有语义信息的表示形式, 从而显著降低后续网络层需要解决的问题的难度. 这些学习到的内部表示还可以通过迁移学习技术被重新利用来解决其他相关问题, 正如前文在皮肤病变分类任务中所展示的那样. 一个有趣的发现是, 用于处理图像的神经网络模型所学习到的内部表示, 与在哺乳动物视觉皮层中观察到的神经表征惊人地相似 (详见节 10.3).

定义 1.30 (表示学习)

把原始输入数据转换成某种新的、富有语义信息的表示形式, 以降低后续网络层需要解决的问题的难度——这一视角与做法称为**表示学习** (representation learning, 也称表征学习).

♣

注

(重构新增) 表示学习把“特征工程”从人工搬到模型内部: 隐藏层输出的就是输入的一种新表示, 而它随参数一起被学习. 判断一种表示是否有用, 标准不是它像不像原始数据, 而是它是否让后续任务变得更简单——在理想情形下, 让原本高度非线性的判别问题在新空间中接近线性可分 (定义 1.11). 条件上, 表示学习依赖“多层的复合”与“任务驱动的目标函数”: 若没有足够深度, 表示只能作一次简单变换; 若没有明确的误差信号, 所学表示就未必对目标任务有用. 这也解释了为什么小节 1.3.3所强调的规模化与表示学习是同一件事的两面: 数据越多、模型越大, 学到的表示才越通用.

那些能够针对一系列下游任务进行适配或微调 (fine-tuned) 的大规模神经网络模型, 称为基础模型 (foundation model). 这些模型可以充分利用海量且多样化的数据集, 从而构建出具有广泛适用性的通用模型 (Bommasani et al., 2021).

定义 1.31 (基础模型)

能够针对一系列下游任务进行适配或微调 (fine-tuned) 的大规模神经网络模型, 称为**基础模型** (foundation model).

♣

注

(重构新增) 基础模型的要点是“一次训练、多处使用”: 它先在海量且多样化的数据上得到一个通用模型, 再通过适配或微调把它迁移到具体任务上 (定义 1.3). 因此它的价值不在某一项任务的精度, 而在可被复用的广度. 条件上, “大规模”与“多样化”缺一不可: 规模保证模型有足够容量吸收数据中的规律, 多样化保证这些规律不局限于单一分布, 从而使迁移到下游任务时仍然有效. 这也带来代价——训练与推理的计算成本、以及下游任务与预训练分布不匹配时的性能下降, 都需要另行处理.

除了规模化这一因素外, 还有其他若干进展也同样为深度学习的成功做出了重要贡献. 例如, 在传统的简单神经网络中, 训练信号随着其在深度网络中逐层反向传播而逐渐衰减 (详见节 9.5). 为了解决这一难题, 一种名为残差连接 (residual connection) 的技术被引入网络架构中 (He et al., 2015a), 它极大地提高了对数百层深度的网络的训练效率.

定义 1.32 (残差连接)

在深度网络中, 为缓解训练信号逐层反传时的衰减而引入的一类跨层连接技术, 称为**残差连接** (residual connection)(He et al., 2015a).

♣

注

(重构新增) 残差连接的典型做法是让某一层的输出与该层的输入相加, 使网络只需学习“在原有表示上需要改动多少”, 而不必从头学出一个完整的变换. 这样, 即使中间各层的变换接近恒等, 信息与梯度也能沿这条“直通”路径顺畅通过, 深层网络的训练因此变得可行. 条件上, 跨层相加要求两条通路的维数一致 (否则需要先做线性投影), 这也是实际架构中残差块通常保持通道数不变的原因. 它并不增加模型的表达能力上限, 改变的是**可优化性**——这一点与小节 1.2.4中“模型容量”与“能否训练好”的区分是同一回事.

另一项具有里程碑意义的进展是自动微分 (automatic differentiation) 方法的提出, 该方法能够基于描述网络前

向传播过程的代码, 自动生成用于执行反向传播以计算误差函数梯度的代码. 研究人员只需要显式地编写相对简洁的前向传播函数代码, 就能够快速地探索和尝试不同的神经网络架构, 并快速尝试不同的架构和多种组件的不同组合.

定义 1.33 (自动微分)

能够基于描述网络前向传播过程的代码, 自动生成用于执行反向传播以计算误差函数梯度的代码的方法, 称为**自动微分** (automatic differentiation). ♣

注

(重构新增) 自动微分既不是数值差分 (用差商近似导数, 精度与代价都受限), 也不是符号求导 (表达式会急剧膨胀), 而是把定义 1.26的链式法则作用在程序本身上: 前向计算被记录成一张计算图, 反向遍历该图即得全部偏导数. 它的实际意义在于把“求梯度”从手工推导变成框架的职责, 使研究者只需写好前向传播, 就能尝试不同的架构与组件组合. 条件上, 它要求前向过程可被记录 (即由可微的基本运算构成); 若代码中含有不可微或不可追踪的操作, 自动微分同样会失效, 这与定义 1.26对可微性的要求一致.

此外, 机器学习领域的很大一部分研究工作都是基于开源模式开展的, 广大研究人员能够借助“他山之石”进行构建和拓展, 从而进一步加快这个领域的创新步伐.

第 2 章 概率

几乎所有机器学习应用场景, 都需要处理不确定性. 例如, 区分皮肤病变是良性还是恶性的图像分类系统, 在实际中不可能万无一失. 不确定性可以分为两种. 第一种是认知不确定性 (epistemic uncertainty), 也称系统不确定性 (systematic uncertainty). 这种不确定性源自有限的数据集大小. 随着观测数据的增多, 例如更多良性和恶性皮肤病变图像样本, 系统可以更好地预测新样本的类别. 然而, 即使数据集无限大, 也仍然无法达到完美的准确率, 因为存在第二种不确定性——偶然不确定性 (aleatoric uncertainty), 也称内在 (intrinsic) 不确定性或随机 (stochastic) 不确定性, 有时简称噪声 (noise). 一般来说, 噪声源自人们只能观察到关于世界的部分信息. 因此, 一种从源头减小这种不确定性的方法是收集不同类型的数据.图 2.1用扩展到二维的正弦曲线为例说明了这个问题 (参见节 1.2).

定义 2.1 (认知不确定性与偶然不确定性)

设某学习系统由规模为 N 的数据集训练得到, 其预测结果所携带的不确定性可以划分为两类. 若这种不确定性源自数据集的规模有限, 即随着观测数据的增多 (N 增大) 它可以被减小, 则称之为**认知不确定性** (epistemic uncertainty), 也称系统不确定性 (systematic uncertainty). 若这种不确定性即使 $N \to \infty$ 也不会消失, 而源自人们只能观测到关于世界的部分信息, 则称之为**偶然不确定性** (aleatoric uncertainty), 也称内在 (intrinsic) 不确定性或随机 (stochastic) 不确定性, 有时简称噪声 (noise).

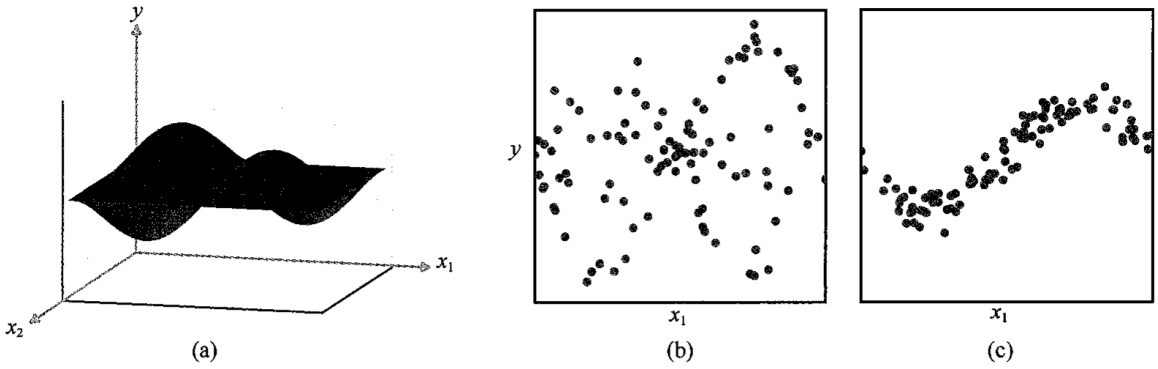


图 2.1: 扩展到二维的简单正弦曲线回归问题.(a) 函数 $y(x_1, x_2) = \sin(2\pi x_1)\sin(2\pi x_2)$ 的曲线图. 数据是通过选择 x_1 和 x_2 并计算对应的 $y(x_1, x_2)$ 值, 然后加上 Gauss 噪声得到的.(b) 100 个数据点的图示, 其中 x_2 未被观测到, 显示出了较高的噪声水平.(c) 100 个数据点的图示, 其中 x_2 固定为 $\pi/2$, 旨在模拟能够同时观测到 x_2 和 x_1 时的效果, 显示出了明显较低的噪声水平.

注

(重构新增) 这一区分的作用是给“不确定性还能不能继续减小”提供一个判据, 而不是给不确定性贴标签.图 2.1正说明了这一点: 图 (b) 只观测到 x_1、x_2 被边缘化掉, 因而数据点散布很宽; 图 (c) 把 x_2 固定为 $\pi/2$ 之后, 散布明显收窄. 两者之间的差别属于认知不确定性, 只要补测 x_2 就能消除; 而即使补充再多同一水平的数据, 图 (c) 中的数据点仍然围绕曲线 $y(x_1, \pi/2)$ 波动, 这一部分波动无法通过增加样本消除, 属于偶然不确定性.

条件“N 增大可减小”是判别认知不确定性的关键: 它要求不确定性来自对总体分布的估计不足, 而不是来自总体的内在随机性. 反过来, 若一个量的不确定性与数据量无关 (例如同一患者重复测量的生理波动), 那么无论怎样扩大数据集都不会改善, 只能通过引入新的观测变量 (也就是本节所说的“收集不同类型的数据”) 来缓解.

在实际的例子中, 皮肤病变的活检样本比图像本身的信息量要大得多, 这有可能极大提升判断新恶性病变的准确性. 如果同时考虑图像和活检数据, 则不确定性可能非常小, 而通过进一步收集较大的训练数据集, 我们或许能将系统不确定性降到一个较低的水平, 从而以较高的准确率来预测病变类别.

这两种不确定性都可以用概率论 (probability theory) 的框架来处理, 它为不确定性的量化和操作提供了统一

的范式, 因此成为机器学习的核心之一. 概率可以由两个简单的公式控制 (参见节 2.1), 即加和法则 (sum rule) 和乘积法则 (product rule). 结合决策论 (decision theory)(参见节 5.2), 这些规则理论上允许我们在给定的已知信息上做出最优预测——尽管我们获得的信息可能是不完整的或模糊的.

概率通常根据可重复事件的频率引入. 以图 2.2中的弯曲硬币为例, 假设该硬币的形状如此, 并被多次抛掷, 则落地时有 60% 的可能性凹面朝上, 有 40% 的可能性凸面朝上. 因此我们说, 弯曲硬币落地时凹面朝上的概率是 0.6 或 60%. 严格来说, 这时的概率是在无限次 "试验" 或抛掷硬币的极限情况下得出的. 因为硬币要么凹面朝上, 要么凸面朝上, 所以这两种情况的概率加和为 100% 或 1.0. 根据可重复事件的频率来定义概率, 是频率学派 (frequentist) 视角下统计学的基础.

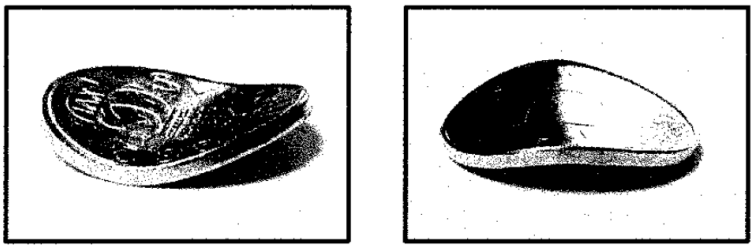


图 2.2: 概率既可以视为与可重复事件相关的频率, 也可以视为对不确定性的量化. 这里用弯曲的硬币抛掷后落地的情况来说明这两者之间的区别.

现在假设即使我们知道硬币落地时凹面朝上的概率是 0.6, 但我们不允许看硬币, 也不知道哪一面是正面、哪一面是反面. 此时如果让我们猜一下硬币抛掷后落地时正面朝上还是反面朝上, 那么硬币的对称性指示我们的猜测应该基于看到正面的概率是 0.5. 更细致的分析表明, 在没有任何附加信息的情况下, 这的确是一个理性的选择. 在这里, 我们使用的是更一般意义上的概率, 而不仅仅是事件发生的频率. 定义硬币的凸面是正面还是反面, 本身并不是一个可重复的事件, 它是未知的. 使用概率对不确定性进行量化是 Bayes 学派 (Bayesian) 的观点 (参见节 2.6). 这一观点更普遍, 而频率学派的概率只是这一观点下的一种特殊情况. 如果可以获知一系列抛掷硬币的结果, 我们就可以通过 Bayes 推理知道硬币的哪一面是正面 (见练习 2.40). 观测到的结果越多, 我们关于硬币哪一面朝上的不确定性就越低.

定义 2.2 (概率的频率学派解释与 Bayes 学派解释)

称一个事件是**可重复的**, 若它能在相同的条件下被反复试验. 把可重复事件 E 在 n 次试验中出现的次数记为 n_E. 若极限 $\lim_{n\to\infty} n_E/n$ 存在, 则把它定义为 E 的**概率**; 以这种 "长期频率" 的方式解释概率的立场称为**频率学派** (frequentist) 观点. 与之相对, 把概率解释为对命题成立这一事件的不确定性程度的量化、从而允许对不可重复的命题赋予概率的立场, 称为 **Bayes 学派** (Bayesian) 观点.

注

(重构新增) 两种解释的差别不在于公式, 而在于“概率从何而来”. 频率学派的定义依赖极限 $\lim_{n\to\infty} n_E/n$ 的存在, 因此必须先有一个可重复的试验机制;图 2.2的硬币满足这个条件,60% 与 40% 是抛掷次数趋于无穷时的比值.Bayes 学派则把这个极限当作特例: 当命题确实对应可重复事件时, 先验信息足够弱, 后验概率会收敛到频率, 二者一致; 当命题不可重复 (“这枚硬币的凸面是正面”) 时, 频率定义失效, 而“用概率量化不确定性”仍然有意义.

需要强调的是,Bayes 观点并不与频率观点矛盾, 它只是在更大的范围里给概率指派含义; 这也是本书后文 (例如节 2.6) 反复使用的立场. 条件“n_E/n 的极限存在”在频率解释中不可省: 若试验条件每次都在变化, 长期频率可能不存在, 此时只能采用 Bayes 解释.

在非正式地介绍完概率的概念之后, 我们现在要对概率进行更详细的探讨, 并讨论如何定量地使用概率. 本章接下来的内容所阐述的概念将构成本书所讨论的许多其他主题的核心基础.

2.1 概率法则

在本节, 我们将推导出两个控制概率行为的简单法则. 虽然这两个法则看上去很简单, 但我们将证明它们非常强大且具有广泛的适用性. 首先我们通过一个简单的例子来引出概率法则.

2.1.1 医学筛查示例

考虑对人群进行筛查以便尽早发现癌症患者的示例. 假设人群中有 1% 的人患有癌症. 理想情况下, 我们的癌症检查对任何患有癌症的人都会给出阳性结果, 而对任何未患癌症的人都会给出阴性结果. 然而, 癌症检查并不完美, 所以我们假设当对未患癌症的人进行检查时, 其中 3% 的人会被误判为阳性, 这就是所谓的假阳性 (false positive). 类似地, 当检查那些确实患有癌症的人时, 其中 10% 的人会被误判为阴性, 这就是所谓的假阴性 (false negative). 检查结果的准确性如图 2.3所示.

图 2.3: 癌症检查的准确性示意图. 如左图所示, 每 100 名接受检查的未患癌症的人中, 平均有 3 人检查结果呈阳性. 如右图所示, 每 100 名接受检查的患有癌症的人中, 平均有 90 人检查结果呈阳性.

有了这些信息, 我们需要考虑以下问题:(1) 如果我们对人群进行筛查, 某人的检查结果呈阳性的概率是多少? (2) 如果某人的检查结果呈阳性, 那么他确实患有癌症的概率是多少? 我们可以通过详细研究癌症筛查案例来回答这两个问题. 但现在我们先不对这个例子做进一步讨论, 而是首先推导出概率论的一般法则, 即概率的加和法则和乘积法则. 然后, 我们将通过回答这两个问题来说明这两个法则具体是如何使用的.

2.1.2 加和法则和乘积法则

为了推导概率法则,图 2.4展示了一个更一般的例子, 它涉及两个变量 X 和 Y. 在癌症检查的例子中,X 表示癌症存在与否,Y 表示检查结果变量. 因为这两个变量的值可能以某种未知的方式因人而异, 所以它们又称为随机变量 (random variable 或 stochastic variable). 假设 X 可以取任意值 x_i(其中 $i = 1, \cdots, L$), 并且 Y 取值 y_j(其中 $j = 1, \cdots, M$). 假设总共有 N 次试验, 我们同时对变量 X 和 Y 采样, 并将 $X = x_i$ 且 $Y = y_j$ 的试验次数记为 n_{ij}. 同样, 记 X 取值为 x_i(不考虑 Y 的取值) 的试验次数为 c_i, 记 Y 取值为 y_j 的试验次数为 r_j.

定义 2.3 (随机变量)

设一次试验的可能结果构成集合 Ω(称为样本空间). 若对每个可能结果都指定了一个取值, 则称这个取值为**随机变量** (random variable), 也称随机变量 (stochastic variable). 若随机变量 X 只能取有限或可数多个值 $x_1, x_2, \cdots$, 则称它为**离散的**; 若它可以在某个区间上取任意实数值, 则称它为**连续的**. 把 X 取值为 x_i 这一事件记为 $X = x_i$.

♣

注

(重构新增) 随机变量不是“随机的数”, 而是“从结果到数的映射”: 正因如此, 我们才能用 $X = x_i$ 这样的等式来叙述事件, 并谈论它的频率.定义 2.3中“离散”与“连续”的划分决定了后文的叙述方式: 离散情形用求和与计数 (本节), 连续情形用积分与概率密度 (节 2.2).

这里的关键条件是每一个结果都必须被指派一个确定的值——否则 n_{ij} 无法定义, 后面的频率解释也就无从建立. 癌症检查的例子中,C 与 T 都是二值离散随机变量, 这一点使得本节的所有论证都归结为对整数 n_{ij} 的计数.

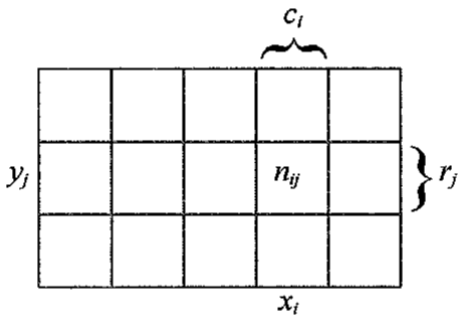


图 2.4: 通过考虑随机变量 X(X 取值为 $\{x_i\}$, 其中 $i = 1, \cdots, L$) 和 Y(Y 取值为 $\{y_j\}$, 其中 $j = 1, \cdots, M$) 来推导概率的加和法则和乘积法则. 其中,$L = 5$,$M = 3$. 如果考虑总共 N 个实例中的这些变量, 则用 n_{ij} 表示 $X = x_i$ 和 $Y = y_j$ 的实例数, 这是阵列中对应单元格的实例数. 用 c_i 表示第 i 列对应于 $X = x_i$ 的实例数, 用 r_j 表示第 j 行对应于 $Y = y_j$ 的实例数.

X 取值为 x_i 且 Y 取值为 y_j 的概率记为 $p(X = x_i, Y = y_j)$, 称作 $X = x_i$ 和 $Y = y_j$ 的联合 (joint) 概率. 它由落在单元格 (i, j) 中的点数占总点数的比例给出, 因此有

$$p(X = x_i, Y = y_j) = \frac{n_{ij}}{N} \tag{2.1}$$

定义 2.4 (联合概率与边缘概率)

设 X 取值于 $\{x_1, \cdots, x_L\}$,Y 取值于 $\{y_1, \cdots, y_M\}$. 称 $X = x_i$ 与 $Y = y_j$ 同时成立的概率 $p(X = x_i, Y = y_j)$ 为 X 与 Y 的**联合概率** (joint probability). 称只关心 X 取值、不考虑 Y 取值时 $X = x_i$ 的概率 $p(X = x_i)$ 为 X 的**边缘概率** (marginal probability). 把求边缘概率的过程 (即对另一个变量的所有取值求和) 称为**边缘化** (marginalization).

♣

注

(重构新增) 引入这两个概念是为了把“同时发生”与“单独发生”区分开. 联合概率 $p(X=x_i, Y=y_j)$ 描述的是图 2.4中单个单元格的占比 n_{ij}/N; 边缘概率 $p(X=x_i)$ 描述的是整列的占比 c_i/N. 正因为单元格的数目比列多得多,$p(X,Y)$ 的信息量比 $p(X)$ 大: 由 $p(X,Y)$ 可以算出 $p(X)$, 反之则不能.

条件“Y 的所有可能取值都被求和掉”体现了边缘化的含义: 边缘化是有损操作, 它会丢掉 X 与 Y 之间的依赖关系. 后文节 2.2中的连续情形把求和换成积分, 定义本身不变.

这里隐式地考虑了极限 $N \to \infty$. 类似地, 在不考虑 Y 取值的情况下,X 取值为 x_i 的概率记为 $p(X=x_i)$, 它由落在第 i 列中的点数占总点数的比例给出, 因此有

$$p(X=x_i) = \frac{c_i}{N} \tag{2.2}$$

由于 $\sum_i c_i = N$, 可以看出

$$\sum_{i=1}^{L} p(X=x_i) = 1 \tag{2.3}$$

命题 2.1 (边缘概率的归一化)

设离散随机变量 X 取值于 $\{x_1, \cdots, x_L\}$, 其边缘概率由(2.2)给出, 则(2.3)成立, 即 $\sum_{i=1}^{L} p(X=x_i) = 1$.

证明

由图 2.4,c_i 是第 i 列中全部单元格的实例数之和, 而各列互不重叠地覆盖了全部 N 个实例, 故 $\sum_{i=1}^{L} c_i = N$. 代入(2.2)得

$$\sum_{i=1}^{L} p(X=x_i) = \sum_{i=1}^{L} \frac{c_i}{N} = \frac{1}{N} \sum_{i=1}^{L} c_i = \frac{N}{N} = 1.$$

□

注

(重构新增) 归一化不是一条附加的技术假设, 而是“全部实例被各列无重复地划分”这一计数事实的直接翻译:$\sum_i c_i = N$. 它保证 $p(X)$ 是一个概率分布而不是任意的非负函数, 因而后文用 $p(X)$ 作权重求期望、用 $1/p(x)$ 度量信息量时才有意义.

条件“各列互不重叠且覆盖全体”不可省: 若同一个实例被计入两列 (例如允许一个人同时属于两个年龄组), 则 $\sum_i c_i > N$, 归一化失败. 这也是定义 2.3要求对每个结果指派唯一取值的原因.

因此, 概率之和为 1. 因为在图 2.4中, 第 i 列的实例数就是该列每个单元格中实例数的总和, 所以有 $c_i = \sum_j n_{ij}$. 根据(2.1)和(2.2), 有

$$p(X=x_i) = \sum_{j=1}^{M} p(X=x_i, Y=y_j) \tag{2.4}$$

这就是概率的加和法则. 请注意,$p(X=x_i)$ 有时称为边缘 (marginal) 概率, 它是通过边缘化或求和其他变量 (在这里是 Y) 得到的.

命题 2.2 (加和法则 (离散形式))

设 X 与 Y 为离散随机变量,X 取值于 $\{x_1, \cdots, x_L\}$,Y 取值于 $\{y_1, \cdots, y_M\}$, 则对每个 i,(2.4)成立, 即联合概率对 Y 的全部取值求和给出 X 的边缘概率.

证明

由 $c_i = \sum_{j=1}^{M} n_{ij}$(图 2.4中第 i 列的实例数等于该列各单元格实例数之和), 并代入(2.1)与(2.2),

$$\sum_{j=1}^{M} p(X = x_i, Y = y_j) = \sum_{j=1}^{M} \frac{n_{ij}}{N} = \frac{1}{N} \sum_{j=1}^{M} n_{ij} = \frac{c_i}{N} = p(X = x_i).$$

□

注

(重构新增) 加和法则的作用是把“高维的联合分布”压成“低维的边缘分布”, 它是后文所有模型中对隐变量求和 (例如节 2.6中计算分母) 的依据. 证明之所以只有一行, 是因为离散情形的求和恰好就是计数; 这一步在连续情形要换成积分, 并在 $N \to \infty$ 的极限下由频率过渡为密度, 见节 2.2.

条件“对 Y 的全部取值求和”不可省: 只对部分 y_j 求和得到的是 $p(X = x_i, Y \in A)$, 一般不等于 $p(X = x_i)$. 此外, 求和能够进行的前提是 Y 的取值至多可数; 否则必须换成积分.

如果我们只考虑 $X = x_i$ 的实例, 那么 $Y = y_j$ 的实例的占比可以记为 $p(Y = y_j \mid X = x_i)$, 称作给定 $X = x_i$ 下 $Y = y_j$ 的条件 (conditional) 概率. 它可以通过在第 i 列中寻找落在单元格 (i, j) 中的那些点的占比得到, 因此, 它可以由下式给出:

$$p(Y = y_j \mid X = x_i) = \frac{n_{ij}}{c_i} \tag{2.5}$$

定义 2.5 (条件概率)

设 $p(X = x_i) > 0$. 称在已知 $X = x_i$ 成立的条件下 $Y = y_j$ 成立的概率为**条件概率** (conditional probability), 记作 $p(Y = y_j \mid X = x_i)$, 它由(2.5)定义, 即把考察范围限制在第 i 列之内、再取单元格 (i, j) 在该列中所占的比例.

♣

注

(重构新增) 条件概率回答的是“已知 X 取某个值之后,Y 的分布如何”, 因此它把原来的样本空间 N 个点替换成了第 i 列的 c_i 个点. 这个“换分母”的动作是条件概率与联合概率的唯一差别, 也解释了为什么 $p(Y \mid X)$ 与 $p(X \mid Y)$ 一般不同: 两者的分母分别是列和与行和.

条件 $p(X = x_i) > 0$(即 $c_i > 0$) 不可省: 若第 i 列为空, 则(2.5)的分母为零, 条件概率无从定义. 这一要求在后文使用 Bayes 定理时反复出现 (例如(2.10)要求 $p(X) > 0$).

在(2.5)的两边对 j 求和, 并使用 $\sum_{j} n_{ij} = c_i$, 可以得到

$$\sum_{j=1}^{M} p(Y = y_j \mid X = x_i) = 1 \tag{2.6}$$

这表明条件概率得到了正确的归一化. 由(2.1)、(2.2)、(2.5)可以推导出以下关系:

$$p(X = x_i, Y = y_j) = \frac{n_{ij}}{N} = \frac{n_{ij}}{c_i} \cdot \frac{c_i}{N} = p(Y = y_j \mid X = x_i)\, p(X = x_i) \tag{2.7}$$

这就是概率的乘积法则.

命题 2.3 (条件概率的归一化)

设 $p(X = x_i) > 0$, 条件概率由(2.5)给出, 则对每个固定的 i,(2.6)成立, 即给定 $X = x_i$ 后 Y 的条件分布是合法的概率分布.

♠

证明

由(2.5)与 $c_i = \sum_{j=1}^{M} n_{ij}$,

$$\sum_{j=1}^{M} p(Y = y_j \mid X = x_i) = \sum_{j=1}^{M} \frac{n_{ij}}{c_i} = \frac{1}{c_i} \sum_{j=1}^{M} n_{ij} = \frac{c_i}{c_i} = 1.$$

□

注

(重构新增) 这条性质保证“条件化”不会破坏概率的规范性: 无论把哪一列单独拿出来看, 所得的都是一个总和为 1 的分布. 它使得我们可以把 $p(Y \mid X = x_i)$ 当作一个独立的分布去求期望、算熵 (见节 2.5).

条件 $p(X = x_i) > 0$ 仍然是必需的; 此外, 归一化是对 Y 的全部取值求和而言的, 若只对 Y 的部分取值求和, 结果一般小于 1.

命题 2.4 (乘积法则 (离散形式))

设 $p(X = x_i) > 0$, 则联合概率可以分解为条件概率与边缘概率的乘积, 即(2.7)成立.

证明

把(2.5)与(2.2)代入(2.7)的中间项:

$$p(Y = y_j \mid X = x_i)\, p(X = x_i) = \frac{n_{ij}}{c_i} \cdot \frac{c_i}{N} = \frac{n_{ij}}{N} = p(X = x_i, Y = y_j).$$

□

注

(重构新增) 乘积法则把“联合”拆成“先有 X、再在 X 的条件下生成 Y”两步, 这正是生成式模型 (节 2.6) 的写法, 也是 Bayes 定理的唯一来源. 它的证明只是约分, 但约分能够进行依赖于 $c_i \neq 0$, 即 $p(X = x_i) > 0$.

值得注意的是分解的方向不唯一: 同理有 $p(X = x_i, Y = y_j) = p(X = x_i \mid Y = y_j)p(Y = y_j)$, 两个方向给出同一个联合概率, 这一对称性稍后直接推出 Bayes 定理.

到目前为止, 我们一直都相当仔细地区分随机变量 (比如 X) 及其可取的值 (比如 x_i). 因此,X 取值为 x_i 的概率记为 $p(X = x_i)$. 这样写虽然可以避免歧义, 但也导致比较烦琐的符号表示, 而在很多情况下我们不需要写得如此复杂. 相反, 在上下文能解释清楚的场合下, 我们可以简单地用 $p(X)$ 来表示随机变量 X 上的分布, 或用 $p(x_i)$ 来表示针对特定值 x_i 的分布.

定义 2.6 (概率的简写记号)

在上下文不会引起歧义的场合, 用 $p(X)$ 表示随机变量 X 的分布 (即函数 $x_i \mapsto p(X = x_i)$), 用 $p(x_i)$ 表示该分布在特定值 x_i 处的取值; 类似地, 用 $p(X, Y)$、$p(Y \mid X)$ 分别表示联合分布与条件分布.

注

(重构新增) 这一约定纯粹是为了减轻书写负担, 不引入新的数学内容:$p(X)$ 是“函数”,$p(x_i)$ 是“函数值”, 二者在(2.8)、(2.9)这类法则中混用而不会出错, 是因为法则本身对自变量的每个取值都成立.

使用简写的前提是上下文能唯一确定变量的身份. 当同一个字母既作随机变量又作具体取值、或者出现 $p(x)$ 与 $p(y)$ 两个不同分布时, 就必须回到完整记号, 否则容易把 $p(X, Y)$ 与 $p(X)p(Y)$ 混为一谈——后者只在独立时才成立 (见定义 2.8).

使用这种更简单的记法, 我们可以将概率论的两个基本法则写成如下形式:

$$p(X) = \sum_{Y} p(X, Y) \quad \text{加和法则} \tag{2.8}$$

$$p(X, Y) = p(Y \mid X)\, p(X) \quad \text{乘积法则} \tag{2.9}$$

定理 2.1 (概率论的两个基本法则)

设 X 与 Y 为随机变量, 则**加和法则** (sum rule) 与**乘积法则** (product rule) 成立, 即(2.8)与(2.9). 这里 $p(X,Y)$ 是联合概率,$p(Y \mid X)$ 是条件概率,$p(X)$ 是边缘概率.

♡

证明

离散情形下,(2.8)就是(2.4)对每个 i 的陈述,(2.9)就是(2.7)对每对 (i,j) 的陈述, 二者已分别在命题 2.2与命题 2.4中证明. 把 X 的取值 x_i 换成一般的 x、把求和指标换成 Y 的全部取值, 证明逐字不变.

连续情形下, 把每个实变量划分为宽度为 Δ 的区间、在这些区间上使用离散形式的结论, 再令 $\Delta \to 0$ 并把求和换成积分, 即得到(2.8)与(2.9)的连续版本; 这一过渡的严格论证需要测度论, 见节 2.2的讨论.

□

注

(重构新增) 把这两条法则提升为一条定理, 是因为它们不是关于某个具体例子的结论, 而是全部概率推理的出发点: 后文的 Bayes 定理由乘积法则加对称性得到, 条件熵与互信息的恒等式由乘积法则展开得到, 边缘似然的计算则依赖加和法则.

两条法则的地位并不对称: 乘积法则用到条件概率 $p(Y \mid X)$, 因此隐含要求 $p(X) > 0$; 加和法则只要求对 Y 的全部取值求和, 不涉及除法. 使用(2.8)时还要注意, 求和号下的 Y 必须遍历其全部可能取值, 否则得到的不是边缘概率.

这里 $p(X,Y)$ 是联合概率, 可以表达为 “X 和 (and)Y 同时成立的概率” . 同样,$p(Y \mid X)$ 是条件概率, 可以表达为 “给定 (given)X 下 Y 的概率” . 最后,$p(X)$ 是边缘概率, 即 “X 的概率” . 这两个简单的法则构成了我们在本书中使用的所有概率机制的基础.

2.1.3 Bayes 定理

从乘积法则出发, 结合对称性 $p(X,Y) = p(Y,X)$, 可以得到以下条件概率之间的关系:

$$p(Y \mid X) = \frac{p(X \mid Y)\, p(Y)}{p(X)} \tag{2.10}$$

定理 2.2 (Bayes 定理)

设 $p(X) > 0$, 则条件概率满足(2.10).

♡

证明

由乘积法则(2.9), 一方面 $p(X,Y) = p(Y \mid X)p(X)$; 另一方面交换 X 与 Y 的角色得 $p(Y,X) = p(X \mid Y)p(Y)$. 由对称性 $p(X,Y) = p(Y,X)$, 两种分解相等:

$$p(Y \mid X)\, p(X) = p(X \mid Y)\, p(Y).$$

两边同除以 $p(X)$(这里用到 $p(X) > 0$) 即得(2.10).

□

注

(重构新增) Bayes 定理的价值在于把 “正向” 的条件概率 $p(X \mid Y)$ 翻转为 “反向” 的 $p(Y \mid X)$, 而后者往往才是我们真正想知道的量: 医学筛查中容易测得的是 $p(\text{检查阳性} \mid \text{患癌})$, 需要回答的却是 $p(\text{患癌} \mid \text{检查阳性})$.

定理中 $p(X) > 0$ 不可省, 因为证明的最后一步是除以 $p(X)$; 这也解释了为什么定义 2.5一开始就要求条件事件的概率为正. 此外, 定理本身不依赖 X,Y 是离散还是连续, 只要 $p(X,Y)$ 存在且 $p(X) > 0$ 即可.

这就是 Bayes 定理, 它在机器学习中扮演着至关重要的角色. 要注意 Bayes 定理是如何将(2.10)左边的条件分布 $p(Y \mid X)$ 与右边的 “反向” 条件分布 $p(X \mid Y)$ 联系起来的. 利用加和法则,Bayes 定理中的分母可以用分子中出现的量来表示:

$$p(X) = \sum_{Y} p(X \mid Y)\, p(Y) \tag{2.11}$$

命题 2.5 (Bayes 定理的分母)

设 Y 为离散随机变量, 则 Bayes 定理中的分母可以写成(2.11), 即 $p(X) = \sum_{Y} p(X \mid Y)p(Y)$.

♠

证明

对联合概率 $p(X,Y)$ 施用乘积法则(2.9)得 $p(X,Y) = p(X \mid Y)p(Y)$; 再对 Y 施用加和法则(2.8),

$$p(X) = \sum_{Y} p(X,Y) = \sum_{Y} p(X \mid Y)\, p(Y).$$

□

注

(重构新增) 这条推论把 (X,Y) 的联合分布"积分掉" Y, 从而只用分子中出现的两项就能算出分母, 这正是 Bayes 定理在实践中可用的原因: 分母不必单独建模. 连续情形下把求和换成对 Y 的积分, 结论形式不变 (见(2.32)).

条件是 Y 的取值可数 (离散情形); 若 Y 连续, 则(2.11)中的求和必须换成积分, 否则该式没有意义.

因此, 我们可以将 Bayes 定理中的分母看作归一化常数, 其确保了(2.10)左边的条件分布对所有 Y 值的和等于 1.

图 2.5展示了一个涉及两个变量联合分布的简单样例, 以说明边缘分布和条件分布的概念. 这里从联合分布中抽取了 $N = 60$ 个数据点的有限样本, 如图 2.5(a) 所示.图 2.5(b) 是 Y 的两种取值的数据点所占比例的直方图. 从概率的定义来看, 当样本量 $N \to \infty$ 时, 这些分数将等于极限下对应的概率 $p(Y)$. 我们可以将直方图视为一种仅从该分布中抽取有限数量的数据点来建模概率分布的简单方法 (参见小节 3.5.1).图 2.5(c) 和图 2.5(d) 分别展示了 $p(X)$ 和 $p(X \mid Y = 1)$ 所对应的直方图估计.

例题 2.1 用直方图估计联合、边缘与条件分布 考虑图 2.5中的样本:X 有 9 个可能取值,Y 有 2 个可能取值, 从联合分布中抽取 $N = 60$ 个点. 把样本点按 X 的取值分成 9 列、按 Y 的取值分成 2 行, 则:

(1)图 2.5(a) 中每个单元格内点的个数除以 60, 就是该单元格对应取值的联合概率的估计;

(2) 把每一行的点数相加再除以 60, 得到 $p(Y)$ 的估计, 即图 2.5(b) 中的两个高为 2/3 与 1/3 的长条;

(3) 把每一列的点数相加再除以 60, 得到 $p(X)$ 的估计, 即图 2.5(c) 中 9 个长条的高度;

(4) 只保留 $Y = 1$ 那一行 (图 (a) 的底部一行), 把该行每一列的点数除以该行的总点数, 得到条件分布 $p(X \mid Y = 1)$ 的估计, 即图 2.5(d).

当 $N \to \infty$ 时, 这些比例依大数定律收敛到相应的概率, 因此直方图是概率分布的一个一致的 (但分段常数的) 估计.

注

(重构新增) 这个例子把(2.8)、(2.9)与(2.11)的差别摆在了同一张图上: 图 (b)、(c) 是边缘化 (对另一变量求和) 的结果, 图 (d) 是条件化 (只在 $Y = 1$ 内重新归一) 的结果. 二者不可混用——图 (d) 的长条之和为 1, 而图 (a) 底部一行的点数之和只占全部的 1/3.

需要注意, 直方图估计的可靠性依赖于箱子的划分: 箱子越细, 偏差越小但方差越大; 这也是小节 3.5.1讨论非参数方法时的出发点.

2.1.4 再看医学筛查示例

让我们回到之前癌症检查的例子, 并应用概率的加和法则和乘积法则来回答小节 2.1.1开头提出的两个问题. 为了清楚起见, 在回答这两个问题时, 我们再一次明确区分随机变量和它们的实例. 用变量 C 来表示癌症存在与否, 它有两个取值:$C = 0$ 对应于"未患癌症",$C = 1$ 对应于"患有癌症". 我们假设人群中每 100 人里就有 1 人患有癌症, 则有

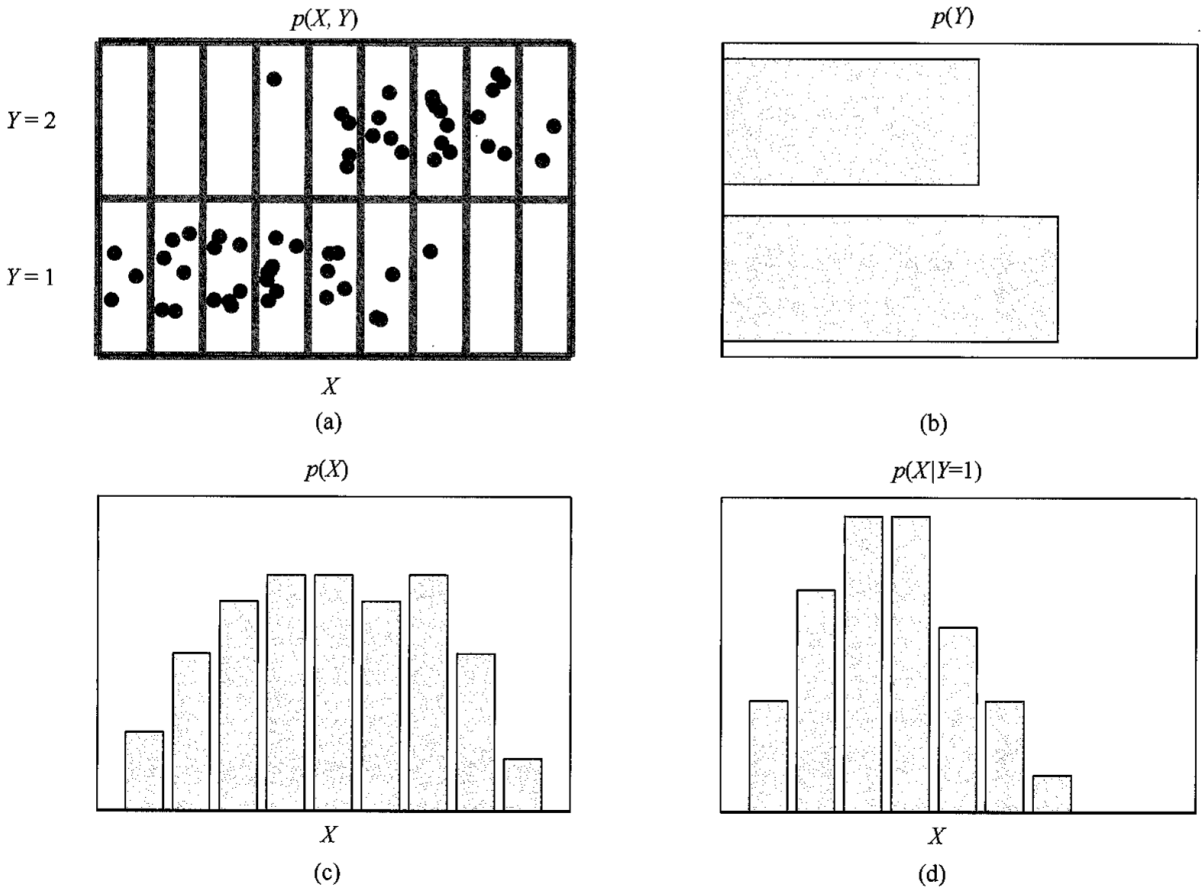


图 2.5: 变量 X 和 Y 分布的图示.X 有 9 个可能的取值,Y 有 2 个可能的取值. 图 (a) 展示了从变量 X 和 Y 的联合分布中抽取的 60 个样本点. 图 (b) 和图 (c) 分别展示了边缘分布 $p(Y)$ 和 $p(X)$ 的直方图估计, 图 (d) 展示了图 (a) 底部一行所对应的条件分布 $p(X \mid Y = 1)$.

$$p(C=1)=1/100=0.01 \tag{2.12}$$

$$p(C=0)=99/100=0.99 \tag{2.13}$$

注意 $p(C=0)+p(C=1)=1$. 下面我们引入第二个随机变量 T 来表示检查结果, 其中 $T=1$ 为阳性结果, 表示检查患有癌症;$T=0$ 为阴性结果, 表示检查未患癌症. 由图 2.3可知, 对于那些患有癌症的人, 检查结果呈阳性的概率是 90%; 而对于那些未患癌症的人, 检查结果呈阳性的概率是 3%. 因此, 我们可以把这 4 种条件概率都写出来:

$$p(T=1\mid C=1)=90/100=0.90 \tag{2.14}$$

$$p(T=0\mid C=1)=10/100=0.10 \tag{2.15}$$

$$p(T=1\mid C=0)=3/100=0.03 \tag{2.16}$$

$$p(T=1\mid C=1)=97/100=0.97 \tag{2.17}$$

再次注意, 这些概率是归一化的, 因此

$$p(T=1\mid C=1)+p(T=0\mid C=1)=1 \tag{2.18}$$

并且同样有

$$p(T=1\mid C=0)+p(T=0\mid C=0)=1 \tag{2.19}$$

例题 2.2 癌症检查的定量分析 我们现在可以使用概率的加和法则和乘积法则来回答第一个问题, 并估计一个人检查结果呈阳性的概率:

$$\begin{aligned} p(T=1) &= p(T=1\mid C=0)\,p(C=0)+p(T=1\mid C=1)\,p(C=1)\\ &= \frac{3}{100}\times\frac{99}{100}+\frac{90}{100}\times\frac{1}{100}=\frac{387}{10\,000}=0.0387 \end{aligned} \tag{2.20}$$

可以看到, 如果随机找一个人检查, 即使他实际患有癌症的概率只有 1%, 检查结果呈阳性的概率也大约为 4%. 进一步用加和法则可以得到 $p(T=0)=1-387/10\,000=9\,613/10\,000=0.9613$, 因此, 检查结果呈阴性的概率约为 96%.

再看第二个问题, 被检查者往往对此特别关注: 如果检查结果呈阳性, 那么这个人患有癌症的概率是多少? 这需要我们在已有检查结果的条件下估计这个人患有癌症的概率,(2.14)至(2.17)给出了在是否患有癌症的条件下检查结果的概率分布. 我们可以使用 Bayes 定理 [见(2.10)] 来解决逆向条件概率问题:

$$p(C=1\mid T=1)=\frac{p(T=1\mid C=1)\,p(C=1)}{p(T=1)} \tag{2.21}$$

$$=\frac{90}{100}\times\frac{1}{100}\times\frac{10\,000}{387}=\frac{90}{387}\approx 0.23 \tag{2.22}$$

因此, 如果一个人的检查结果是阳性, 那么他有 23% 的概率确实患有癌症. 应用加和法则, 可以得出 $p(C=0\mid T=1)=1-90/387=297/387\approx 0.77$, 即他有 77% 的概率实际未患癌症.

注

(重构新增) 原文问题说明 (2.17)在原书中印作 $p(T=1 \mid C=1)=97/100=0.97$, 这与(2.14)矛盾 (同一个条件概率不可能同时等于 0.90 与 0.97), 也与(2.18)矛盾 ($0.90+0.10=1$ 成立, 而 $0.97+0.10 \neq 1$). 结合“4 种条件概率”的上下文与 97/100 这一数值, 此处显然应为

$$p(T=0 \mid C=0)=97/100=0.97,$$

即未患癌症者被正确判为阴性的概率. 为保持与原书一致, 正文仍按原书字形照录(2.17); 自(2.19)起以及本节其余计算 (包括(2.20)) 均使用修正后的 $p(T=0 \mid C=0)=0.97$, 两处并不冲突, 因为(2.17)的左右两端并未在后续推导中被引用.

2.1.5 先验概率和后验概率

我们可以用癌症检查的例子对 Bayes 定理做出如下重要解释. 如果我们在某人接受检查之前就被问到他是否可能患有癌症, 那么我们所能获得的所有信息就是由概率 $p(C)$ 提供的. 我们称其为先验概率 (prior probability), 因为它是在我们观测到检查结果之前可知的概率. 一旦这个人的检查结果为阳性, 我们就可以使用 Bayes 定理来计算概率 $p(C \mid T)$, 我们称其为后验概率 (posterior probability), 因为它是我们观测到检查结果 T 之后得到的概率.

定义 2.7 (先验概率与后验概率)

在 Bayes 定理(2.10)中, 称 $p(Y)$ 为 Y 的**先验概率** (prior probability), 它是在观测到 X 之前对 Y 的分布的描述; 称 $p(Y \mid X)$ 为 Y 的**后验概率** (posterior probability), 它是在观测到 X 之后对 Y 的分布的描述. ♣

注

(重构新增)“先验”与“后验”是相对于一次观测而言的, 不是绝对的: 一次推断的后验可以成为下一次推断的先验 (这正是 Bayes 方法可以迭代使用的原因, 见节 2.6). 在癌症检查中, $p(C=1)=0.01$ 是先验、$p(C=1 \mid T=1) \approx 0.23$ 是后验, 二者相差二十余倍.

后验概率的定义依赖 Bayes 定理, 故隐含要求 $p(T=1)>0$: 若某种检查结果在总体中不可能出现, 就不能据此更新. 另外, 先验的选择会实质性地影响后验——若人群患癌率不是 1% 而是 0.1%, 同一检查给出的后验会显著降低 (见练习 2.1).

在这个示例中, 患有癌症的先验概率是 1%. 如果检查结果呈阳性, 那么患有癌症的后验概率为 23%. 正如我们预期的那样, 后者的患癌概率明显更高. 尽管从图 2.3来看, 检查方法似乎已经非常“准确”了, 但检查结果呈阳性的人实际患癌的概率仍然只有 23%(见练习 2.1). 这个结论在很多人看来似乎是反直觉的, 原因与患有癌症的先验概率较低有关. 虽然检查提供了患有癌症的有力证据, 但必须通过 Bayes 定理将其与先验概率结合, 才能得出正确的后验概率.

2.1.6 独立变量

如果两个变量的联合分布可以分解为边缘分布的乘积, 即 $p(X,Y)=p(X)p(Y)$, 则称变量 X 和 Y 是独立的. 连续抛掷硬币就是独立事件的一个例子. 由乘积法则, 有 $p(Y \mid X)=p(Y)$, 因此给定 X 下 Y 的条件分布确实独立于 X 的值. 在癌症检查样例中, 如果检查结果为阳性的概率与此人是否患有癌症无关, 则 $p(T \mid C)=p(T)$. 这意味着根据 Bayes 定理 [见(2.10)] 可得 $p(C \mid T)=p(C)$, 因此患癌的概率不会因观测到检查结果而改变. 当然, 这样的检查是没有用的, 因为检查的结果并不能告诉我们这个人是否患有癌症.

定义 2.8 (独立变量)

若两个随机变量 X 和 Y 的联合分布可以分解为它们边缘分布的乘积, 即

$$p(X,Y)=p(X)\,p(Y),$$

则称 X 和 Y 是**独立的**. 若对任意 $k \geqslant 2, X_1, \cdots, X_k$ 的联合分布都等于各自边缘分布之积, 则称这组变量相互独立.

注

(重构新增) 独立的定义用的是联合分布能否分解, 而不是“一个变量不影响另一个变量”这种模糊说法; 后者在连续情形下难以直接检验, 而分解式是可以计算的. 定义对变量的个数没有限制, 但两两独立 (每个二元组都分解) 并不等于相互独立, 故需要单独说明 k 元的情形.

定义本身不涉及条件概率, 因此不要求 $p(X) > 0$; 这也说明独立性比“$p(Y \mid X) = p(Y)$”更基本——后者只在 $p(X) > 0$ 时才有意义.

命题 2.6 (独立性的条件概率刻画)

设 $p(X) > 0$. 若 X 与 Y 独立, 则 $p(Y \mid X) = p(Y)$, 即给定 X 后 Y 的条件分布与 X 无关. 反之, 若对 X 的每个取值都有 $p(Y \mid X) = p(Y)$, 则 X 与 Y 独立.

证明

若 X 与 Y 独立, 则由乘积法则(2.9)与独立性的定义,

$$p(Y \mid X)\, p(X) = p(X, Y) = p(X)\, p(Y),$$

两边除以 $p(X) > 0$ 得 $p(Y \mid X) = p(Y)$. 反之, 若 $p(Y \mid X) = p(Y)$, 代回乘积法则即得 $p(X, Y) = p(X)p(Y)$, 故独立. □

注

(重构新增) 这条等价刻画解释了“独立”一词的直观含义: 知道 X 的取值不会改变 Y 的分布. 它也因此给出了独立性的一个实用判据——检查似然比 $p(Y \mid X)/p(Y)$ 是否恒等于 1.

在癌症检查中, 若检查完全无效 ($p(T \mid C) = p(T)$), 则由 Bayes 定理得 $p(C \mid T) = p(C)$, 观测结果不改变后验, 这正是“检查没有提供信息”的精确表述. 注意证明用到了 $p(X) > 0$; 若 X 的某个取值概率为零, 该取值上的条件概率本身就没有定义, 等价刻画只在正概率取值上成立.

2.2 概率密度

我们除了关注在离散数值集合上定义概率, 还关注连续变量的概率. 例如, 我们可能想预测给病人服用药物的剂量. 由于在预测中会有不确定性, 而我们想要量化这种不确定性, 因此就需要再次利用概率. 然而, 在无限精度下, 观测到连续变量为某个特定值的概率几乎为零. 为此, 需要引入概率密度 (probability density) 的概念. 这里我们只做相对非正式的讨论.

定义连续变量 x 上的概率密度 $p(x)$, 如图 2.6所示, 当 $\delta x \to 0$ 时,x 落在区间 $(x, x + \delta x)$ 的概率由 $p(x)\delta x$ 给出.x 落在区间 (a, b) 的概率为

$$p\big(x \in (a, b)\big) = \int_a^b p(x)\, \mathrm{d}x \tag{2.23}$$

定义 2.9 (概率密度)

设 x 为实值连续随机变量. 若存在非负函数 $p(x)$, 使得当 $\delta x \to 0$ 时 x 落在区间 $(x, x+\delta x)$ 内的概率为 $p(x)\delta x$, 则称 $p(x)$ 为 x 的**概率密度** (probability density). 此时 x 落在区间 (a, b) 内的概率由(2.23)给出.

注

(重构新增) 引入密度的动机是: 连续变量取任一确定值的概率为零, 因此不能像离散情形那样用 $p(X = x_i)$ 来描述分布, 只能用“落在一个小区间里的概率与区间长度之比”来描述.定义 2.9正是把这个比值固定在 x 处的极限值上, 于是 $p(x)$ 本身不是概率,$p(x)\delta x$ 才是概率.

条件“$\delta x \to 0$”不可省: 若只对某个固定的 δx 定义, 则得到的依赖于区间划分, 不构成 x 的函数. 也正因为 $p(x)$ 是密度而非概率, 它可以大于 1(例如 $\mathcal{N}(0, 0.01)$ 在 $x = 0$ 处约为 3.99), 只有(2.23)的积分才是概率.

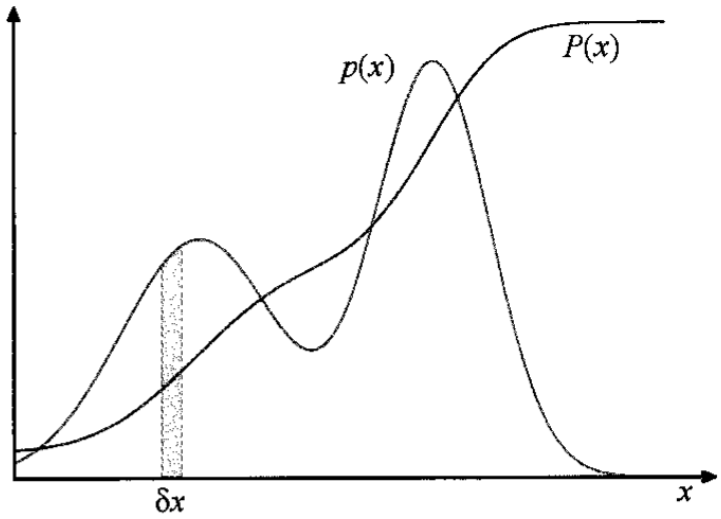


图 2.6: 离散变量下概率的概念可以推广到连续变量 x 上的概率密度 $p(x)$, 而当 $\delta x \to 0$ 时,x 落在区间 $(x, x + \delta x)$ 的概率由 $p(x)\delta x$ 给出. 概率密度可以表示为累积分布函数 $P(x)$ 的导数.

因为概率是非负的, 并且 x 的值必须落在实轴上的某个位置, 所以概率密度 $p(x)$ 必须同时满足以下两个条件:

$$p(x) \geqslant 0 \tag{2.24}$$

$$\int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1 \tag{2.25}$$

命题 2.7 (概率密度的两个基本性质)

设 $p(x)$ 是定义 2.9意义下的概率密度, 则它满足非负性(2.24)与归一化条件(2.25).

证明

非负性: 对任意 x 与充分小的 $\delta x > 0$,$p(x)\delta x$ 是事件 $\{x \in (x, x + \delta x)\}$ 的概率, 概率非负而 $\delta x > 0$, 故 $p(x) \geqslant 0$.

归一化: 事件“x 取实轴上的某个值”是必然事件, 其概率为 1. 把实轴分成互不重叠的小区间并求和, 再令区间长度趋于零, 由(2.23)即得 $\int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1$.

□

注

(重构新增) 这两条性质正好对应概率公理中的两条: 非负性保证不会算出负的概率, 归一化保证全体可能取值的概率之和为 1. 它们也是判断一个给定函数能否作为概率密度的唯一标准——(2.24)与(2.25)同时成立即可.

归一化条件不可省: 若只要求 $p \geqslant 0$, 则 $p(x) \equiv 1$ 也会被当作密度, 但它的积分发散, 无法给出概率. 原书把这类无法归一化的分布称为**反常** (improper) 分布; 在 Bayes 推断中, 先验常常只在形式上给定、其归一化常数留到分母中处理, 此时用的正是“未归一化的密度”.

如图 2.6所示,x 落在区间 $(-\infty, z)$ 的概率由累积分布函数 (cumulative distribution function) 给出, 定义为

$$P(z)=\int_{-\infty}^{z} p(x)\,\mathrm{d}x \tag{2.26}$$

且满足 $P'(x)=p(x)$.

定义 2.10 (累积分布函数)

称由(2.26)定义的函数 $P(z)$ 为随机变量 x 的**累积分布函数** (cumulative distribution function), 它给出 x 不超过 z 的概率.

注

(重构新增) 累积分布函数把“概率是多少”回答成“概率累积到哪里”:$P(z)$ 是密度在 $(-\infty,z]$ 上的积分, 因此它单调不减、取值于 $[0,1]$, 且 $z\to-\infty$ 时趋于 0、$z\to+\infty$ 时趋于 1. 这三个性质正好是概率测度的三条公理在实轴上的体现.

与密度相比,P 的好处是它总存在 (任何随机变量都有分布函数), 而密度不一定存在 (例如离散变量或奇异分布).(2.26)用积分定义 P, 因此隐含要求密度 p 可积; 对只有分布函数而没有密度的情形, 本节的定义需要推广为 Stieltjes 积分.

命题 2.8 (累积分布函数的导数)

设概率密度 p 在点 x 处连续,P 由(2.26)定义, 则 P 在 x 处可导且 $P'(x)=p(x)$.

证明

由(2.26)与(2.23), 对 $h>0$,

$$\frac{P(x+h)-P(x)}{h}=\frac{1}{h}\int_{x}^{x+h} p(t)\,\mathrm{d}t.$$

当 $h\to 0$ 时, 右端是 p 在 $[x,x+h]$ 上的平均值; 由 p 在 x 处连续, 该平均值趋于 $p(x)$.$h<0$ 的情形同理. 故 $P'(x)=p(x)$.

□

注

(重构新增) 这条性质说明“密度”与“分布函数”是同一对象的两种描述:P 是 p 的积分,p 是 P 的导数. 它的用途是把“求区间概率”转化为“求端点处的分布函数值之差”, 即 $p(x\in(a,b))=P(b)-P(a)$, 这在解析计算中往往更方便 (例如节 3.3中处理周期变量时).

条件“p 在 x 处连续”不可省: 若 p 在 x 处有跳跃,P 在该点不可导, 只能断言 P 在此处左、右导数分别等于 p 的左、右极限. 此外 P 总是单调不减且 $\lim_{z\to-\infty}P(z)=0$、$\lim_{z\to+\infty}P(z)=1$, 这两条由(2.24)与(2.25)直接得到.

假设有几个连续变量 $x_1,\cdots,x_D$, 整体用向量 $\mathbf{x}$ 表示, 我们可以定义联合概率密度 $p(\mathbf{x})=p(x_1,\cdots,x_D)$, 使得 $\mathbf{x}$ 落在包含点 $\mathbf{x}$ 的无穷小体积 $\delta\mathbf{x}$ 中的概率由 $p(\mathbf{x})\delta\mathbf{x}$ 给出. 该联合概率密度必须满足

$$p(\mathbf{x})\geqslant 0 \tag{2.27}$$

$$\int p(\mathbf{x})\,\mathrm{d}\mathbf{x}=1 \tag{2.28}$$

其中积分是在整个 $\mathbf{x}$ 的空间上进行的. 更一般地, 我们还可以考虑同时包含离散变量和连续变量的联合概率分布.

定义 2.11 (联合概率密度)

设 $\mathbf{x}=(x_1,\cdots,x_D)^{\mathsf{T}}$ 为 D 维连续随机向量. 若存在非负函数 $p(\mathbf{x})$, 使得 $\mathbf{x}$ 落在包含点 $\mathbf{x}$ 的无穷小体积 $\delta\mathbf{x}$ 中的概率为 $p(\mathbf{x})\delta\mathbf{x}$, 则称 $p(\mathbf{x})$ 为 $\mathbf{x}$ 的**联合概率密度**, 并要求它满足(2.27)与(2.28).

注

(重构新增) 联合密度是把定义 2.9逐维推广的结果: 一维时 δx 是长度,D 维时 $\delta\mathbf{x}$ 是体积, 因此 $p(\mathbf{x})$ 的量纲是“概率 / 体积”.(2.28)中的积分必须在整个 $\mathbf{x}$ 空间上进行, 这是它与一维情形的唯一差别.

当 D 很大时, 联合密度所需的参数个数随 D 迅速增长, 直接建模不可行; 这正是后文引入节 3.2中参数化形式 (以及节 3.4中指数族) 的动机.

概率的加和法则、乘积法则及 Bayes 定理, 也适用于概率密度以及离散变量和连续变量的组合. 如果 x 和 y 是两个实数变量, 那么加和法则和乘积法则将有如下形式:

$$p(x) = \int p(x, y)\,\mathrm{d}y \quad \text{加和法则} \tag{2.29}$$

$$p(x, y) = p(y \mid x)\, p(x) \quad \text{乘积法则} \tag{2.30}$$

类似地,Bayes 定理可以写成以下形式:

$$p(y \mid x) = \frac{p(x \mid y)\, p(y)}{p(x)} \tag{2.31}$$

其中分母由下式给出:

$$p(x) = \int p(x \mid y)\, p(y)\,\mathrm{d}y \tag{2.32}$$

定理 2.3 (连续形式的加和法则、乘积法则与 Bayes 定理)

设 x, y 为实值连续随机变量, 其联合密度存在, 则(2.29)、(2.30)、(2.31)与(2.32)成立 (其中(2.31)要求 $p(x) > 0$). ♡

证明

把实轴划分为宽度为 Δ 的区间, 将 x 在第 i 个区间、y 在第 j 个区间内的事件分别记为 $X = i$、$Y = j$. 由(2.23),$\mathbb{P}(X = i) \approx p(x_i)\Delta$、$\mathbb{P}(X = i, Y = j) \approx p(x_i, y_j)\Delta^2$, 其中 x_i, y_j 为相应区间的代表点. 这些离散事件满足(2.8)与(2.9), 即

$$\mathbb{P}(X = i) = \sum_j \mathbb{P}(X = i, Y = j), \qquad \mathbb{P}(X = i, Y = j) = \mathbb{P}(Y = j \mid X = i)\,\mathbb{P}(X = i).$$

把上述近似代入并约去公共因子 Δ, 得 $p(x_i) \approx \sum_j p(x_i, y_j)\Delta$ 与 $p(x_i, y_j) \approx p(y_j \mid x_i)p(x_i)$. 令 $\Delta \to 0$, 求和过渡为积分, 即得(2.29)与(2.30). 再把定理 2.2的推导逐字重复 (把求和换成积分) 即得(2.31)与(2.32).

□

注

(重构新增) 这条定理的作用是宣布: 前两节关于离散变量的全部法则, 在连续情形下形式完全不变, 只需把求和换成积分. 因此后文讨论 Gauss 分布、密度变换与信息论时, 可以不再区分离散与连续, 只按需要选择求和或积分.

证明中的关键是“先离散化、再取极限”, 而这个过渡的严格化需要测度论 (Feller, 1966), 超出了本书的范围; 本书只要求上述法则在所用密度足够正则 (例如分段连续) 时成立.(2.31)中的条件 $p(x) > 0$ 不可省, 理由与离散情形相同: 分母出现在除法中.

要对连续变量的加和法则和乘积法则做正式论证, 需要借助名为测度论 (measure theory)(Feller, 1966) 的数学分支, 这超出了本书的讨论范围. 然而, 通过将每个实变量划分为宽度为 Δ 的区间, 并考虑这些区间上的离散概率分布, 我们可以得知上述法则的正确性. 通过取极限 $\Delta \to 0$, 然后求和转为积分, 就可以得到我们想要的结果.

2.2.1 分布的示例

很多形式的概率密度都得到了广泛应用, 这些概率密度无论是它们本身还是作为更复杂概率模型的组件, 都很重要. 最简单的形式是 $p(x)$ 为常数, 其独立于 x, 但不能被归一化, 因为(2.28)中的积分是发散的. 不能被归一化

的分布称为反常 (improper) 分布. 然而, 我们有这样的均匀分布, 其在有限区间 (比如 (c,d)) 内是常数, 而在其他地方为零. 在这种情况下,(2.28)意味着

$$p(x)=\frac{1}{d-c},\quad x\in(c,d) \tag{2.33}$$

定义 2.12 (均匀分布)

设 $c<d$. 称由(2.33)给出的概率密度为区间 (c,d) 上的**均匀分布** (uniform distribution), 即在 (c,d) 内取常数、在区间外取零的密度. 若某个密度在有限区间外为零、在区间内为常数, 但其归一化常数尚未确定, 则称相应的分布为**反常** (improper) 分布.

♣

注

(重构新增) 均匀分布是“在已知取值范围、而对取值本身一无所知”时最自然的密度: 它把全部概率质量平均分配到区间上, 是节 2.5中最大熵原理在“支撑集有界”约束下给出的解.

常数 $1/(d-c)$ 完全由归一化确定, 不可任意选取: 若取其他常数,(2.25)便不成立. 定义中对“反常分布”的说明提醒我们, 常数密度在整条实轴上无法归一化, 因此均匀分布必须限定在有限区间内; 这也是后文使用无信息先验时需要格外小心的地方.

概率密度的另一种简单形式是指数分布 (exponential distribution), 由下式给出:

$$p(x\mid\lambda)=\lambda\exp(-\lambda x),\quad x\geqslant 0 \tag{2.34}$$

定义 2.13 (指数分布)

设 $\lambda>0$. 称由(2.34)给出的概率密度为参数 λ 的**指数分布** (exponential distribution), 其支撑集为 $x\geqslant 0$.

♣

指数分布有一种变体叫作 Laplace 分布 (Laplace distribution), 其允许将峰值移动到位置 μ, 由下式给出:

$$p(x\mid\mu,\gamma)=\frac{1}{2\gamma}\exp\left(-\frac{|x-\mu|}{\gamma}\right) \tag{2.35}$$

定义 2.14 (Laplace 分布)

设 $\gamma>0$. 称由(2.35)给出的概率密度为参数 μ,γ 的 **Laplace** 分布 (Laplace distribution), 其中 μ 决定峰值的位置 (对称中心),γ 决定峰值的宽度.

♣

注

(重构新增) 定义 2.12、定义 2.13、定义 2.14这三个密度构成一个便于对照的序列: 均匀分布在有界区间上“最平”, 指数分布在半直线上单调衰减,Laplace 分布则以 μ 为中心、两侧对称地指数衰减. 三者都由归一化常数唯一确定, 而常数恰好等于“密度曲线下的面积”的倒数, 因此(2.34)中的 λ 与(2.35)中的 2γ 都不是自由的.

条件 $\lambda>0$、$\gamma>0$ 不可省: 若 $\lambda\leqslant 0$,(2.34)在 $x\to+\infty$ 时不衰减, 积分为无穷;γ 的情形同理.Laplace 分布中的绝对值 $|x-\mu|$ 保证两侧对称, 若去掉绝对值, 所得函数在 μ 处不连续且不再是关于 μ 对称的密度. 指数分布可以看作 Laplace 分布在 $\mu=0$ 处只保留右半支的结果 (此时归一化常数变为 λ 而非 $1/(2\gamma)$).

均匀分布、指数分布和 Laplace 分布如图 2.7所示.

另一个重要的分布是 Dirac δ 函数 (Dirac delta function), 由下式给出:

$$p(x\mid\mu)=\delta(x-\mu) \tag{2.36}$$

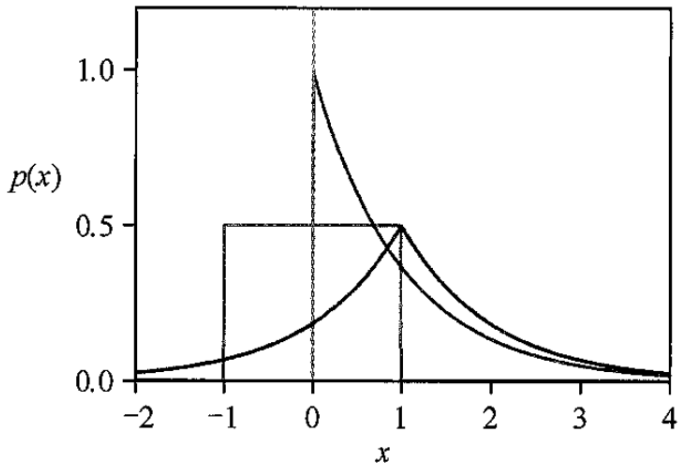


图 2.7: 区间 $(-1,1)$ 内的均匀分布 (红色),$\lambda=1$ 的指数分布 (蓝色), 以及 $\mu=1$ 和 $\gamma=1$ 的 Laplace 分布 (绿色) 的图示.

定义 2.15 (Dirac δ 函数)

称满足“在除 $x=\mu$ 外的任意位置均为零、且积分为 1”的对象 $\delta(x-\mu)$ 为 **Dirac** δ 函数 (Dirac delta function). 它可以视为位于 $x=\mu$ 处的无限窄且无限高的峰, 并具有单位面积的性质, 即

$$\delta(x-\mu)=0\ (x\neq\mu),\qquad \int_{-\infty}^{\infty}\delta(x-\mu)\,\mathrm{d}x=1.$$

♣

注

(重构新增) δ 函数把“概率质量集中在一点”这一极端情形也纳入了密度的语言: 它的“密度”在 μ 处无穷大, 但任何包含 μ 的区间上的积分都等于 1. 正因为如此, 离散分布与连续分布可以在同一个积分记号下统一处理, 这是定义 2.16与后文混合模型的基础.

严格说来 δ 不是函数而是分布 (广义函数), 因此它不能像普通密度那样逐点求值, 只能出现在积分号下. 使用时必须保证积分区间包含 μ, 否则(2.36)的积分为零而非 1.

最后, 如果我们有一个 x 的有限观测值集合 $\mathcal{D}=\{x_1,\cdots,x_N\}$, 则可以使用 Dirac δ 函数来构造经验分布 (empirical distribution), 由下式给出:

$$p(x\mid\mathcal{D})=\frac{1}{N}\sum_{n=1}^{N}\delta(x-x_n) \tag{2.37}$$

它由以每个数据点为中心的 Dirac δ 函数组成. 根据要求, 由(2.37)定义的概率密度积分为 1(见练习 2.6).

定义 2.16 (经验分布)

设 $\mathcal{D}=\{x_1,\cdots,x_N\}$ 为一组观测值. 称由(2.37)给出的密度为这组数据的**经验分布** (empirical distribution), 它把概率质量 $1/N$ 平均分配给每个观测点.

♣

注

(重构新增) 经验分布是“不对数据作任何分布假设”的建模方式: 它只记录观测到了什么, 而不假设数据来自某个参数族. 因此它天然是小节 2.2.1末尾所说的反常密度的对立面——它一定是归一化的, 但作为估计它并不光滑, 且在观测点之外没有任何概率质量.

它的重要性在于把“对分布求期望”化为“对样本求平均”: 由命题 2.9,$\mathbb{E}[f] \approx \frac{1}{N}\sum_n f(x_n)$. 这也解释了为什么可以用样本均值估计期望, 以及为什么样本量 N 越大估计越可靠.

2.2.2 期望和协方差

概率中最重要的操作之一是寻找函数的加权平均值. 某函数 $f(x)$ 在概率分布 $p(x)$ 下的加权平均称为 $f(x)$ 的期望 (expectation), 用 $\mathbb{E}[f]$ 表示. 对于离散分布, 期望可以通过对所有可能值求和得到:

$$\mathbb{E}[f] = \sum_x p(x)\, f(x) \tag{2.38}$$

其中平均值由不同 x 值的相对概率加权得到. 对于连续变量, 期望以相应的概率密度积分表示:

$$\mathbb{E}[f] = \int p(x)\, f(x)\, \mathrm{d}x \tag{2.39}$$

定义 2.17 (期望)

设 f 为随机变量 x 的函数. 称 $f(x)$ 在分布 $p(x)$ 下的加权平均为 $f(x)$ 的**期望** (expectation), 记作 $\mathbb{E}[f]$; 离散情形由(2.38)定义, 连续情形由(2.39)定义 (要求相应的和或积分绝对收敛).

注

(重构新增) 期望把“一个分布”压成一个数, 因而是描述分布位置的最基本信息; 它的定义方式 (以概率为权重求和或积分) 与中学的算术平均一致, 只是权重由 $1/N$ 换成了 $p(x)$.(2.38)与(2.39)的差别仅在求和与积分, 这正是定理 2.3所保证的“形式不变”.

条件“绝对收敛”不可省: 若 $\sum_x p(x)|f(x)|$ 发散, 则期望可能依赖于求和次序而不唯一 (例如 Cauchy 分布的均值), 此时称期望不存在. 本书讨论的分布都保证相应的和或积分收敛.

不论在哪种情况下, 如果我们从概率分布或概率密度中得到 N 个有限数量的点, 则期望可以近似为这些点的有限和 (见练习 2.7):

$$\mathbb{E}[f] \approx \frac{1}{N}\sum_{n=1}^{N} f(x_n) \tag{2.40}$$

命题 2.9 (期望的样本近似)

设 $x_1, \cdots, x_N$ 是来自分布 $p(x)$ 的样本,$\mathcal{D} = \{x_1, \cdots, x_N\}$, 则关于经验分布(2.37)的期望恰为样本平均, 即

$$\int f(x)\, p(x \mid \mathcal{D})\, \mathrm{d}x = \frac{1}{N}\sum_{n=1}^{N} f(x_n),$$

从而当 $N \to \infty$ 时它依概率收敛到 $\mathbb{E}[f]$, 这就是(2.40).

证明

把(2.37)代入并交换求和与积分 (有限和, 合法):

$$\int f(x)\, p(x \mid \mathcal{D})\, \mathrm{d}x = \frac{1}{N}\sum_{n=1}^{N} \int f(x)\, \delta(x - x_n)\, \mathrm{d}x = \frac{1}{N}\sum_{n=1}^{N} f(x_n),$$

最后一步用到 δ 函数的定义. 再由大数定律, 样本平均依概率收敛到 $\mathbb{E}[f]$.

□

注

(重构新增) 这条结果把“积分”变成了“求和”, 是 Monte Carlo 方法的雏形: 只要能从 $p(x)$ 中采样, 就可以用样本平均近似任意函数的期望, 而不必解析地计算积分.(2.40)中的“$\approx$”在 $N \to \infty$ 时变为精确, 误差的量级通常为 $O(N^{-1/2})$.

条件“样本来自 $p(x)$”不可省: 若样本来自另一个分布 $q(x)$, 则上式计算的是 $\mathbb{E}_q[f]$ 而非 $\mathbb{E}_p[f]$, 两者的差别需要重要性采样来修正 (参见节 14.1).

式(2.40)中的近似在极限 $N \to \infty$ 下是精确的.

有时我们会考虑多个变量的函数期望, 在这种情况下, 我们可以使用下标来表示哪个变量被平均, 例如

$$\mathbb{E}_x\big[f(x, y)\big] \tag{2.41}$$

这表示函数 $f(x, y)$ 相对于 x 的分布的平均值. 注意 $\mathbb{E}_x[f(x, y)]$ 是关于 y 的函数. 我们也可以考虑针对条件分布来说的条件期望 (conditional expectation):

$$\mathbb{E}_x\big[f \mid y\big] = \sum_{x} p(x \mid y)\, f(x) \tag{2.42}$$

$\mathbb{E}_x[f \mid y]$ 也是关于 y 的函数. 对于连续变量, 条件期望的形式为

$$\mathbb{E}_x\big[f \mid y\big] = \int p(x \mid y)\, f(x)\, \mathrm{d}x \tag{2.43}$$

定义 2.18 (条件期望)

设 $p(y) > 0$. 称 $f(x)$ 在给定 y 的条件分布 $p(x \mid y)$ 下的平均值为 f 关于 x 的**条件期望** (conditional expectation), 记作 $\mathbb{E}_x[f \mid y]$, 离散情形由(2.42)定义, 连续情形由(2.43)定义. 类似地,$\mathbb{E}_x[f(x, y)]$ 表示 f 关于 x 的边缘分布的平均, 见(2.41).

♣

注

(重构新增) 下标 x 指出“对哪个变量求平均”, 结果是剩余变量的函数; 这一记号使“先对 x 平均、再对 y 平均”这类双重平均得以无歧义地书写, 后文练习 2.11中的全期望公式与方差分解都要用到它.

条件期望与普通期望的差别在于所用的分布:(2.42)用的是 $p(x \mid y)$ 而不是 $p(x)$, 因此结果依赖 y. 定义要求 $p(y) > 0$, 否则(2.42)中的条件分布无定义——这与定义 2.5的要求一致.

$f(x)$ 的方差 (variance) 定义为

$$\operatorname{var}[f] = \mathbb{E}\Big[\big(f(x) - \mathbb{E}[f(x)]\big)^2\Big] \tag{2.44}$$

定义 2.19 (方差)

称由(2.44)定义的量为 $f(x)$ 的**方差** (variance), 它度量 $f(x)$ 围绕其均值 $\mathbb{E}[f(x)]$ 的分散程度.

♣

注

(重构新增) 方差是“二阶”的分散性度量: 它先取偏差、再平方、最后平均, 因此对大偏差的惩罚比对小偏差重. 选择平方而非绝对值, 是为了让方差可以展开成期望的代数式 (见命题 2.10), 从而便于计算.

(2.44)中的期望是对 x 的分布取的, 因此方差是一个数 (不是函数). 方差恒非负; 方差为零当且仅当 $f(x)$ 以概率 1 取常数.

式(2.44)度量了 $f(x)$ 围绕其平均值 $\mathbb{E}[f(y)]$ 的变化程度. 将平方项展开后, 可以看到, 方差也可以写成 $f(x)$ 和 $f(x)^2$ 的期望的形式 (见练习 2.8):

$$\operatorname{var}[f] = \mathbb{E}\left[f(x)^2\right] - \mathbb{E}\left[f(x)\right]^2 \tag{2.45}$$

命题 2.10 (方差的展开式)

设 $\mathbb{E}[f(x)^2]$ 存在, 则(2.45)成立.

♠

证明

记 $\mu = \mathbb{E}[f(x)]$, 它是不依赖于 x 的常数. 把(2.44)中的平方展开:

$$\mathbb{E}\left[(f-\mu)^2\right] = \mathbb{E}\left[f^2 - 2\mu f + \mu^2\right] = \mathbb{E}[f^2] - 2\mu\,\mathbb{E}[f] + \mu^2,$$

其中用到期望的线性性. 代入 $\mathbb{E}[f] = \mu$ 得 $\mathbb{E}[f^2] - 2\mu^2 + \mu^2 = \mathbb{E}[f^2] - \mu^2$, 即(2.45).

□

注

(重构新增) 此外,(2.44)前正文中的 $\mathbb{E}[f(y)]$ 应为 $\mathbb{E}[f(x)]$(原书笔误,y 在该句中并未出现), 这不影响(2.44)的表述. 展开式的实际价值在于把“先算偏差再平方”换成“两个矩相减”, 计算上少一次中心化; 它也是后文证明方差可加性 (练习 2.10) 与协方差公式 ((2.48)) 的工具.

条件“$\mathbb{E}[f^2]$ 存在”不可省: 若二阶矩不存在,(2.45)右端两项可能同时为无穷, 相减没有意义. 另外该式的推导只用到期望的线性性, 不要求 x 离散或连续.

变量 x 自身的方差, 其由下式给出:

$$\operatorname{var}[x] = \mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2 \tag{2.46}$$

对于两个随机变量 x 和 y, 协方差 (covariance) 度量了这两个变量一起变化的程度, 定义为

$$\begin{aligned}\operatorname{cov}[x,y] &= \mathbb{E}_{x,y}\left[\{x-\mathbb{E}[x]\}\{y-\mathbb{E}[y]\}\right] \\ &= \mathbb{E}_{x,y}[xy] - \mathbb{E}[x]\,\mathbb{E}[y]\end{aligned} \tag{2.47}$$

定义 2.20 (协方差)

称由(2.47)第一行定义的量为随机变量 x 与 y 的**协方差** (covariance), 记作 $\operatorname{cov}[x,y]$, 它度量这两个变量一起变化的程度. 当 $x = y$ 时 $\operatorname{cov}[x,x] = \operatorname{var}[x]$.

♣

注

(重构新增) 原文问题说明 (2.47)的第二行在原书中印作 $\mathbb{E}_{x,y}[xy]\,\mathbb{E}[x]\,\mathbb{E}[y]$, 其中缺少减号. 由第一行展开即可验证正确的等式为

$$\mathbb{E}_{x,y}\left[\{x-\mathbb{E}[x]\}\{y-\mathbb{E}[y]\}\right] = \mathbb{E}_{x,y}[xy] - \mathbb{E}[x]\,\mathbb{E}[y],$$

推导与命题 2.10完全相同 (把 f 换成 x、把平方换成乘积). 若按原书字形理解,$\operatorname{cov}[x,y]$ 将等于三个非负数之积, 与“协方差可以为负”这一基本事实矛盾, 故此处按修正后的形式排版; 下文(2.48)与命题 2.11均使用修正形式.

注

(重构新增) 协方差回答的是“两个变量是否同向变化”：当 x 偏大时 y 也偏大，乘积项为正，协方差为正；反向变化则为负. 它与方差的关系 $\operatorname{cov}[x,x]=\operatorname{var}[x]$ 说明方差只是协方差在对角线上的特例，因此(2.48)可以把两者放进同一个矩阵.

$\operatorname{cov}[x,y]$ 的数值依赖 x,y 的量纲，因此它的大小不能直接比较不同问题中的相关性强弱；这一点由相关系数(除以两个标准差)来弥补. 定义要求二阶矩存在，理由与命题 2.10相同.

如果 x 和 y 是独立的，那么它们的协方差为零 (见练习 2.9).

命题 2.11 (独立变量的协方差为零)

若随机变量 x 与 y 独立，且二者的二阶矩存在，则 $\operatorname{cov}[x,y]=0$.

♠

证明

由独立性，$p(x,y)=p(x)p(y)$，故

$$\mathbb{E}_{x,y}[xy]=\iint xy\,p(x)p(y)\,\mathrm{d}x\,\mathrm{d}y=\left(\int x\,p(x)\,\mathrm{d}x\right)\left(\int y\,p(y)\,\mathrm{d}y\right)=\mathbb{E}[x]\,\mathbb{E}[y].$$

代入(2.47)的第二行即得 $\operatorname{cov}[x,y]=0$.

□

注

(重构新增) 这条结论说明“独立”比“不相关”更强：独立可以推出协方差为零，反之不成立 (练习 2.39给出了协方差为零但彼此依赖的反例). 因此在建模时，看到协方差为零不能直接断定变量无关.

证明中把二重积分拆成两个单重积分，关键仍是 $p(x,y)=p(x)p(y)$；若只知 $\operatorname{cov}[x,y]=0$，则 $\mathbb{E}[xy]=\mathbb{E}[x]\mathbb{E}[y]$ 成立，但推不出联合分布可分解.

对于两个向量 $\mathbf{x}$ 和 $\mathbf{y}$，它们的协方差是由下式给出的矩阵：

$$\begin{aligned}\operatorname{cov}[\mathbf{x},\mathbf{y}]&=\mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\{\mathbf{x}-\mathbb{E}[\mathbf{x}]\}\{\mathbf{y}^{\mathsf{T}}-\mathbb{E}[\mathbf{y}^{\mathsf{T}}]\}\right]\\&=\mathbb{E}_{\mathbf{x},\mathbf{y}}\left[\mathbf{x}\mathbf{y}^{\mathsf{T}}\right]-\mathbb{E}[\mathbf{x}]\,\mathbb{E}[\mathbf{y}^{\mathsf{T}}]\end{aligned}\tag{2.48}$$

如果考虑向量 $\mathbf{x}$ 的分量彼此之间的协方差，则可以使用稍微简单一点的符号来表示：$\operatorname{cov}[\mathbf{x}]=\operatorname{cov}[\mathbf{x},\mathbf{x}]$.

定义 2.21 (协方差矩阵)

设 $\mathbf{x},\mathbf{y}$ 为随机向量. 称由(2.48)定义的矩阵为 $\mathbf{x}$ 与 $\mathbf{y}$ 的**协方差矩阵**；当 $\mathbf{x}=\mathbf{y}$ 时简记 $\operatorname{cov}[\mathbf{x}]=\operatorname{cov}[\mathbf{x},\mathbf{x}]$，其第 (i,j) 个元素是 $\operatorname{cov}[x_i,x_j]$.

♣

注

(重构新增) 矩阵形式把 $\binom{D}{2}+D$ 个标量协方差打包成一个 $D\times D$ 矩阵，使得“线性变换下的协方差”可以用矩阵乘法表达 (见节 3.2).$\operatorname{cov}[\mathbf{x}]$ 必然是对称半正定矩阵，这一事实在节 3.2中决定多元 Gauss 分布的合法性.

(2.48)中的转置不可省：$\mathbf{x}\mathbf{y}^{\mathsf{T}}$ 是 $D\times D$ 矩阵，而 $\mathbf{x}^{\mathsf{T}}\mathbf{y}$ 是标量，二者含义完全不同. 原书字形的第二行同样漏写了减号，修正方式与(2.47)相同，即第二行应为 $\mathbb{E}[\mathbf{x}\mathbf{y}^{\mathsf{T}}]-\mathbb{E}[\mathbf{x}]\,\mathbb{E}[\mathbf{y}^{\mathsf{T}}]$.

2.3 Gauss 分布

连续变量最重要的概率分布之一是 Gauss(Gaussian) 分布或称正态 (normal) 分布，我们将在本书的其余部分广泛使用这种分布. 对于单个实值变量 x，Gauss 分布定义为

$$\mathcal{N}(x\mid\mu,\sigma^2)=\frac{1}{(2\pi\sigma^2)^{1/2}}\exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\}\tag{2.49}$$

定义 2.22 (Gauss 分布)

称由(2.49)给出的概率密度为均值 μ、方差 σ^2 的 **Gauss** 分布 (Gaussian distribution), 也称**正态**分布 (normal distribution). 其中 μ 称为**均值** (mean),σ^2 称为**方差** (variance), 方差的平方根 σ 称为**标准差** (standard deviation), 方差的倒数 $\beta = 1/\sigma^2$ 称为**精度** (precision).

♣

注

(重构新增) Gauss 分布之所以成为全书的主角, 是因为它在两个意义上都是“最自然”的选择: 给定均值和方差, 它是使节 2.5中微分熵最大的分布 (小节 2.5.4), 也是独立随机变量之和在适当缩放下的极限分布 (中心极限定理, 参见节 3.2). 因此它既可作为“信息最少”的先验, 也可作为“大量微小扰动叠加”的模型.

参数 μ 与 σ^2 的地位不同:μ 决定位置,σ^2 决定宽度, 且 $\sigma^2 > 0$ 不可省 (否则(2.49)无定义). 精度 β 只是 σ^2 的另一种写法, 但在后文推导共轭先验时它比 σ^2 更方便——这正是“你很快就会看到这个术语的由来”所指.

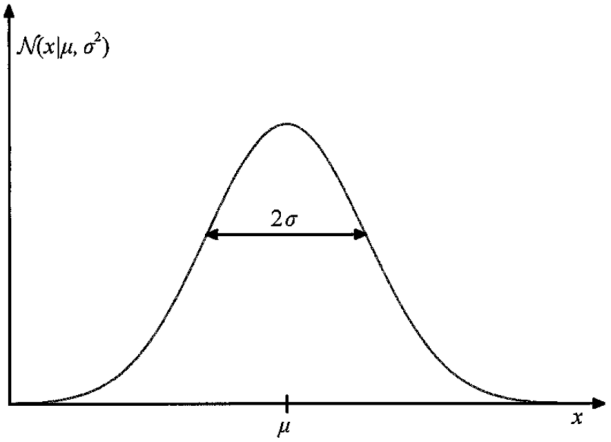


图 2.8: 单个连续变量 x 的 Gauss 分布, 其平均值为 μ, 标准差为 σ.

(2.49)给出的密度由两个参数 μ 和 σ^2 控制, 它们的名称与含义见定义 2.22. 下文你很快就会看到“精度”这一术语的由来.图 2.8展示了 Gauss 分布. 虽然 Gauss 分布的形式看起来是任意的, 但你稍后会看到, 它是从最大熵 (参见小节 2.5.4) 的概念和中心极限定理的角度自然产生的 (参见节 3.2).

由(2.49)可知,Gauss 分布满足

$$\mathcal{N}(x \mid \mu, \sigma^2) > 0 \tag{2.50}$$

可以很直观地看到 Gauss 分布是归一化的 (见练习 2.12), 即

$$\int_{-\infty}^{\infty} \mathcal{N}(x \mid \mu, \sigma^2)\,\mathrm{d}x = 1 \tag{2.51}$$

因此,(2.49)满足概率密度有效的两个要求.

命题 2.12 (Gauss 分布的正性与归一化)

设 $\sigma^2 > 0$, 则(2.49)定义的 $\mathcal{N}(x \mid \mu, \sigma^2)$ 满足(2.50)与(2.51), 因而确实是概率密度.

♠

证明

正性由指数函数恒正、系数 $(2\pi\sigma^2)^{-1/2}>0$ 直接得到. 归一化: 作换元 $u=(x-\mu)/\sigma$, 得 $\mathrm{d}x=\sigma\,\mathrm{d}u$, 于是

$$\int_{-\infty}^{\infty}\mathcal{N}(x\mid\mu,\sigma^2)\,\mathrm{d}x=\frac{1}{\sqrt{2\pi}}\int_{-\infty}^{\infty}e^{-u^2/2}\,\mathrm{d}u.$$

记 $I=\displaystyle\int_{-\infty}^{\infty}e^{-u^2/2}\,\mathrm{d}u$, 则

$$I^2=\int_{-\infty}^{\infty}\int_{-\infty}^{\infty}e^{-(u^2+v^2)/2}\,\mathrm{d}u\,\mathrm{d}v=\int_0^{2\pi}\int_0^{\infty}e^{-r^2/2}\,r\,\mathrm{d}r\,\mathrm{d}\theta=2\pi,$$

其中第二个等号用了极坐标 $(u,v)=(r\cos\theta,r\sin\theta)$、$\mathrm{d}u\,\mathrm{d}v=r\,\mathrm{d}r\,\mathrm{d}\theta$. 故 $I=\sqrt{2\pi}$, 代回得积分值为 1.

□

注

(重构新增) 归一化常数 $(2\pi\sigma^2)^{-1/2}$ 不是人为挑选的, 而是被(2.51)唯一确定的: 若把系数改成别的数, 密度下的面积就不再是 1. 证明中的关键一步是把二重积分化为极坐标, 这一步之所以可行, 是因为 $e^{-(u^2+v^2)/2}$ 只依赖 r.

条件 $\sigma^2>0$ 在整个证明中不可省:$\sigma=0$ 时换元 $u=(x-\mu)/\sigma$ 无意义, 而且此时分布退化为集中在 μ 处的 δ 函数 (定义 2.15), 不再是普通密度. 此外,μ 在归一化中不起作用——平移不改变面积, 这也预示了后文 μ 只影响位置而不影响形状.

2.3.1 均值和方差

我们可以很容易地找到 Gauss 分布下 x 的期望 (见练习 2.13).x 的平均值由下式给出:

$$\mathbb{E}[x]=\int_{-\infty}^{\infty}\mathcal{N}(x\mid\mu,\sigma^2)\,x\,\mathrm{d}x=\mu \tag{2.52}$$

由于参数 μ 表示 x 在该分布下的平均值, 因此称其为均值.(2.52)中的积分称为分布的一阶矩 (first-order moment), 因为是对 x 的期望取一次方. 我们可以类似地给出分布的二阶矩:

$$\mathbb{E}\left[x^2\right]=\int_{-\infty}^{\infty}\mathcal{N}(x\mid\mu,\sigma^2)\,x^2\,\mathrm{d}x=\mu^2+\sigma^2 \tag{2.53}$$

命题 2.13 (Gauss 分布的一阶矩与二阶矩)

设 $x\sim\mathcal{N}(x\mid\mu,\sigma^2)$, 则(2.52)与(2.53)成立, 即 $\mathbb{E}[x]=\mu$、$\mathbb{E}[x^2]=\mu^2+\sigma^2$.

♠

证明

作换元 $u=(x-\mu)/\sigma$, 则 $x=\mu+\sigma u$、$\mathrm{d}x=\sigma\,\mathrm{d}u$, 且 $\mathcal{N}(x\mid\mu,\sigma^2)\,\mathrm{d}x=\varphi(u)\,\mathrm{d}u$, 其中 $\varphi(u)=(2\pi)^{-1/2}e^{-u^2/2}$ 为标准正态密度. 由 φ 是偶函数, $\displaystyle\int u\,\varphi(u)\,\mathrm{d}u=0$, 故

$$\mathbb{E}[x]=\int(\mu+\sigma u)\,\varphi(u)\,\mathrm{d}u=\mu\int\varphi(u)\,\mathrm{d}u+\sigma\int u\,\varphi(u)\,\mathrm{d}u=\mu.$$

再由 $\displaystyle\int u^2\varphi(u)\,\mathrm{d}u=1$(分部积分: $\displaystyle\int u^2\varphi(u)\,\mathrm{d}u=\int\varphi(u)\,\mathrm{d}u=1$),

$$\mathbb{E}[x^2]=\int(\mu+\sigma u)^2\varphi(u)\,\mathrm{d}u=\mu^2+2\mu\sigma\int u\varphi(u)\,\mathrm{d}u+\sigma^2\int u^2\varphi(u)\,\mathrm{d}u=\mu^2+\sigma^2.$$

□

注

(重构新增) 这两条矩公式表明参数的名字名副其实:μ 就是一阶矩,σ^2 则由二阶矩与一阶矩之差给出 (见命题 2.14). 它们也是“矩”这一术语的来源——k 阶矩就是 x^k 的期望.

证明只用到标准正态密度的两个积分 $\displaystyle\int\varphi=1$、$\displaystyle\int u^2\varphi=1$, 而这两个积分又依赖命题 2.12中的 $I=\sqrt{2\pi}$. 条件 $\sigma^2>0$ 仍是换元的前提; 若 σ^2 未知而需要从数据估计, 则得到的是(2.57)中的 μ_{ML} 而不是 μ 本身.

由(2.52)和(2.53)可知,x 的方差为

$$\operatorname{var}[x] = \mathbb{E}\left[x^2\right] - \mathbb{E}[x]^2 = \sigma^2 \tag{2.54}$$

命题 2.14 (Gauss 分布的方差)

设 $x \sim \mathcal{N}(x \mid \mu, \sigma^2)$, 则(2.54)成立, 即 $\operatorname{var}[x] = \sigma^2$.

♠

证明

把命题 2.13给出的 $\mathbb{E}[x^2] = \mu^2 + \sigma^2$ 与 $\mathbb{E}[x] = \mu$ 代入方差展开式(2.45):

$$\operatorname{var}[x] = \mathbb{E}[x^2] - \mathbb{E}[x]^2 = (\mu^2 + \sigma^2) - \mu^2 = \sigma^2.$$

□

注

(重构新增) 这条结果说明 Gauss 分布的两个参数恰好就是它的前两阶矩, 因此“均值—方差”这一对参数已经完全刻画了分布的位置与宽度; 这也是后文用 $\mathcal{N}(x \mid \mu, \sigma^2)$ 直接以矩为参数书写的原因.

推导依赖(2.45), 而后者要求二阶矩存在——对 Gauss 分布而言这总是成立. 需要注意,$\operatorname{var}[x]$ 与 σ^2 相等并不是定义, 而是计算的结果; 若把(2.49)中的 σ^2 换成其他常数, 方差就会随之改变.

σ^2 称为方差参数, 分布的最大值称为众数. 对于 Gauss 分布, 众数与均值重合 (见练习 2.14).

定义 2.23 (众数)

称概率密度 (或概率质量函数) 取最大值的点为该分布的**众数** (mode).

♣

注

(重构新增) 众数描述分布的“峰在哪里”, 它与均值、中位数一起构成位置的三类刻画; 三者对对称分布重合, 对偏斜分布则分开.Gauss 分布的众数与均值重合, 正是因为它关于 μ 对称且单峰.

众数的定义要求最大值确实取到; 对某些密度 (例如在开区间上单调的密度) 最大值不存在, 此时众数无定义. 这与均值要求积分收敛是两类不同的存在性条件, 不可混用.

2.3.2 似然函数

假设我们有一个观测数据集, 记作行向量 $\mathbf{x} = (x_1, \cdots, x_N)$, 表示标量变量 x 的 N 个观测值. 特别值得注意的是, 我们用 $\mathbf{x}$ 这一特定的字体样式, 与 D 维向量值变量的单个观测值加以区分. 对于 D 维向量值变量的单个观测值, 我们则是以列向量 $\mathbf{x} = (x_1, \cdots, x_D)^{\mathsf{T}}$ 来表示. 假设观测值是从 Gauss 分布独立采样的, 其均值 μ 和方差 σ^2 是未知的, 我们希望从数据集中确定这些参数. 给定观测值的一个有限集合估计分布的问题叫作密度估计 (density estimation) 问题. 需要强调的是, 密度估计问题基本上是不适定的, 因为有可能存在无数个概率分布都能产生这一组观测到的有限数据集. 事实上, 任意在 $x_1, \cdots, x_N$ 处非零的分布 $p(x)$ 都有可能是真实的概率分布. 在这里, 我们将分布空间限制为 Gauss 分布, 以产生定义明确的解.

从相同分布中独立产生的数据点是独立同分布 (independent and identical distribution, 缩写为 i.i.d. 或 IID) 的. 你已经看到, 两个独立事件的联合概率由每个事件边缘概率的乘积给出. 因为数据集 $\mathbf{x}$ 是独立同分布的, 所以我们可以将给定 μ 和 σ^2 下数据集 $\mathbf{x}$ 的概率写成以下形式:

$$p(\mathbf{x} \mid \mu, \sigma^2) = \prod_{n=1}^{N} \mathcal{N}(x_n \mid \mu, \sigma^2) \tag{2.55}$$

定义 2.24 (独立同分布与似然函数)

若一组随机变量 $x_1,\cdots,x_N$ 相互独立, 且每个都服从同一个分布 $p(x)$, 则称它们是**独立同分布**的 (independent and identically distributed,i.i.d.). 此时它们的联合分布为 $\prod_{n=1}^{N} p(x_n)$. 若把(2.55)右端视为参数 μ,σ^2 的函数 (数据固定), 则称之为 **Gauss** 分布的**似然函数** (likelihood function). ♣

注

(重构新增) "独立同分布"这一假设使联合概率从一般的 $p(x_1,\cdots,x_N)$ 退化为 N 个相同因子的乘积, 这是似然函数能够写出显式形式的唯一原因; 没有它,(2.55)无从成立.

似然函数与概率密度在数学上是同一个表达式, 区别只在于"谁固定、谁变化": 作为 x 的函数它是密度 (对 x 积分为 1), 作为 μ,σ^2 的函数它是似然 (对 μ,σ^2 的积分不必为 1, 见小节 2.6.1). 这一区别是频率学派与 Bayes 学派分歧的技术根源之一.

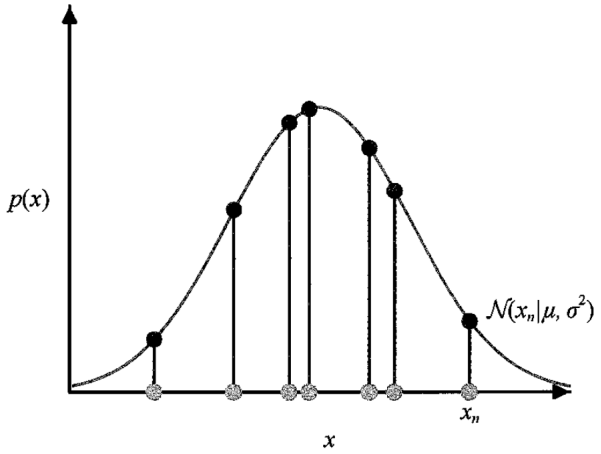


图 2.9: 红色曲线为 Gauss 分布似然函数的图示. 这里灰色的点表示数据集 $\{x_n\}$ 中的值, 似然函数 [见(2.55)] 由对应的 $p(x)$ 值 (蓝色的点) 的乘积给出. 最大似然法通过调整 Gauss 分布的均值和方差来最大化这个乘积 (参见小节 2.6.2).

如果将其视为 μ 和 σ^2 的函数, 则称之为 Gauss 分布的似然函数 (likelihood function), 如图 2.9所示.

一种常见的使用观测数据集确定概率分布中参数的方法就是最大似然 (maximum likelihood) 法, 其目的是找到使似然函数最大化的参数值. 这个准则似乎有些奇怪, 因为从我们之前对概率论的讨论来看, 在给定数据的情况下最大化参数的几率, 似乎比在给定参数的情况下最大化数据的概率更加自然. 事实上, 这两个准则是相关的.

定义 2.25 (最大似然估计)

设数据 $\mathbf{x}=(x_1,\cdots,x_N)$ 的似然函数为 $p(\mathbf{x}\mid\boldsymbol{\theta})$. 称使该似然函数取最大值的参数值

$$\boldsymbol{\theta}_{\mathrm{ML}}=\arg\max_{\boldsymbol{\theta}} p(\mathbf{x}\mid\boldsymbol{\theta})$$

为参数 $\boldsymbol{\theta}$ 的**最大似然估计** (maximum likelihood estimate), 简称最大似然解. ♣

注

(重构新增) 最大似然准则的动机是: 在已经观测到 $\mathbf{x}$ 的前提下, 取"使这组数据最可能出现"的参数值作为估计, 是把"数据已发生"这一事实反过来用于推断参数的最直接方式.

需要强调,$\boldsymbol{\theta}_{\mathrm{ML}}$ 是数据的函数, 因而本身是随机变量: 换一组数据就会得到不同的估计. 这正是小节 2.3.3讨论偏差时的出发点. 另外, 最大值若不存在 (例如似然在参数空间边界上取到上确界而不取到), 则最大似然解不存在, 此时只能说"最大似然估计在极限意义下取到".

首先, 我们将通过最大化似然函数 [见(2.55)] 来确定 Gauss 分布中未知参数 μ 和 σ^2 的值. 在实践中, 最大化似然函数的对数更为方便. 由于对数函数是单调递增函数, 最大化一个函数的对数等价于最大化这个函数本身.

取对数不仅可以简化后续的数学分析, 而且在数值计算上也有帮助, 因为大量小概率的乘积很容易因计算机数值精度不够而产生下溢, 而这个问题可以通过计算对数概率的和来解决. 基于(2.49)和(2.55), 对数似然函数可以写成以下形式:

$$\ln p(\mathbf{x} \mid \mu, \sigma^2) = -\frac{1}{2\sigma^2}\sum_{n=1}^{N}(x_n - \mu)^2 - \frac{N}{2}\ln\sigma^2 - \frac{N}{2}\ln(2\pi) \tag{2.56}$$

命题 2.15 (Gauss 分布的对数似然)

设数据 $\mathbf{x}$ 独立同分布于 $\mathcal{N}(x \mid \mu, \sigma^2)$, 则对数似然由(2.56)给出.

♠

证明

对(2.55)取对数, 把乘积化为求和:

$$\ln p(\mathbf{x} \mid \mu, \sigma^2) = \sum_{n=1}^{N}\ln \mathcal{N}(x_n \mid \mu, \sigma^2).$$

对每一项用(2.49),

$$\ln \mathcal{N}(x_n \mid \mu, \sigma^2) = -\frac{1}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}(x_n - \mu)^2.$$

把 N 项相加, 常数项 $-\frac{N}{2}\ln(2\pi)$ 与 $-\frac{N}{2}\ln\sigma^2$ 合并, 即得(2.56).

□

注

(重构新增) 取对数的合法性来自 ln 严格单调:$p \mapsto \ln p$ 保持大小关系, 故最大值点不变. 但要注意“等价”只对最大化成立, 最大值本身会变 (对数似然的最大值与似然的极大值不相等), 因此不能把对数似然的数值当作概率使用.

把乘积化为求和, 除了避免下溢, 还使求导变得可行:(2.56)右端对 μ 与 σ^2 的偏导数都很容易写出, 这正是下面两式能够显式求解的原因. 条件 $x_n \in \mathbb{R}$ 与 $\sigma^2 > 0$ 保证对数内的量有意义.

将(2.56)关于 μ 最大化, 可以得到最大似然解为 (见练习 2.15)

$$\mu_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}x_n \tag{2.57}$$

这是样本均值 (sample mean), 即观测值 $\{x_n\}$ 的均值. 类似地, 将(2.56)关于 σ^2 最大化, 可以得到方差的最大似然解为

$$\sigma^2_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}\left(x_n - \mu_{\mathrm{ML}}\right)^2 \tag{2.58}$$

定理 2.4 (Gauss 分布的最大似然解)

设数据 $\mathbf{x} = (x_1, \cdots, x_N)$ 独立同分布于 $\mathcal{N}(x \mid \mu, \sigma^2)$, 则对数似然(2.56)的驻点由(2.57)与(2.58)给出, 且该驻点是全局最大值点.

♡

证明

记 $L(\mu, \sigma^2) = \ln p(\mathbf{x} \mid \mu, \sigma^2)$. 先对 μ 求偏导并令其为零:

$$\frac{\partial L}{\partial \mu} = \frac{1}{\sigma^2}\sum_{n=1}^{N}(x_n - \mu) = 0 \quad \Longrightarrow \quad \sum_{n=1}^{N}x_n = N\mu,$$

即 $\mu = \frac{1}{N}\sum_{n}x_n$, 这就是(2.57). 又 $\partial^2 L/\partial\mu^2 = -N/\sigma^2 < 0$, 故该驻点关于 μ 是最大值点.

再对 σ^2 求偏导并令其为零:

$$\frac{\partial L}{\partial \sigma^2} = \frac{1}{2\sigma^4}\sum_{n=1}^{N}(x_n-\mu)^2 - \frac{N}{2\sigma^2} = 0 \quad \Longrightarrow \quad \sigma^2 = \frac{1}{N}\sum_{n=1}^{N}(x_n-\mu)^2.$$

由于 μ 的解与 σ^2 的解解耦 ($\partial L/\partial\sigma^2$ 中出现的 μ 已是上面求得的值), 把 $\mu = \mu_{\mathrm{ML}}$ 代入即得(2.58). 最后,L 关于 $(\mu, \ln\sigma^2)$ 是凹函数 (其 Hessian 为负定), 故驻点必为全局最大值点.

□

注

(重构新增) 定理的结论有两点值得注意. 其一,μ 的解与 σ^2 的解是解耦的: 可以先用(2.57)算出 μ_{ML}, 再把它代入(2.58), 而不必解联立方程. 其二,σ^2_{ML} 的分母是 N 而不是 $N-1$, 这正是小节 2.3.3要讨论的偏差来源.

证明中用 $\ln\sigma^2$(而非 σ^2) 作自变量使 Hessian 为负定, 从而把“驻点”升级为“最大值点”; 若直接对 σ^2 求二阶导, 需要额外验证. 条件 $N \geqslant 1$ 与 $\sigma^2 > 0$ 不可省:$N = 1$ 时(2.58)给出 $\sigma^2_{\mathrm{ML}} = 0$, 分布退化.

这是关于样本均值 μ_{ML} 测量的样本方差 (sample variance). 需要注意的是, 我们正在关于 μ 和 σ^2 进行(2.56)的联合最大化, 但对于 Gauss 分布,μ 的解与 σ^2 的解是解耦的, 因此我们可以先计算(2.57), 再用得到的结果计算(2.58).

2.3.3 最大似然的偏差

最大似然技术已广泛应用于深度学习中, 它是大多数机器学习算法的基础. 然而, 它也有一些局限性, 我们用单变量 Gauss 分布来说明.

我们首先注意到最大似然解 μ_{ML} 和 σ^2_{ML} 是数据集值 $x_1, \cdots, x_N$ 的函数. 假设这些值都是从一个真实参数为 μ 和 σ^2 的 Gauss 分布中独立产生的, 现在考虑 μ_{ML} 和 σ^2_{ML} 在这些数据集值上的期望, 显然有 (见练习 2.16)

$$\mathbb{E}\left[\mu_{\mathrm{ML}}\right] = \mu \tag{2.59}$$

$$\mathbb{E}\left[\sigma^2_{\mathrm{ML}}\right] = \left(\frac{N-1}{N}\right)\sigma^2 \tag{2.60}$$

定义 2.26 (偏差)

设 $\hat{\theta}$ 是参数 θ 的一个估计. 若 $\mathbb{E}[\hat{\theta}] \neq \theta$, 则称 $\hat{\theta}$ 是**有偏的** (biased), 并称 $\mathbb{E}[\hat{\theta}]-\theta$ 为它的**偏差** (bias); 若 $\mathbb{E}[\hat{\theta}] = \theta$, 则称它是**无偏的** (unbiased). ♣

注

(重构新增) 偏差描述的是“平均而言偏不偏”, 而不是“某一次估计准不准”: 一个估计可以有偏但方差很小, 也可以无偏但方差很大.(2.60)说明 σ^2_{ML} 系统性地偏小, 偏差为 $-\sigma^2/N$, 它与 N 成反比, 因此大样本下可以忽略.

需要区分偏差与小节 2.3.3中“最大似然低估方差”的直观原因: 方差的估计是相对于 μ_{ML} 而不是真实的 μ 计算的, 而 μ_{ML} 恰好是使 $\sum_n (x_n-\mu)^2$ 最小的那个点 (见练习 2.15), 因此用它中心化必然使平方和偏小.

命题 2.16 (最大似然估计的期望)

设 $x_1, \cdots, x_N$ 独立同分布于 $\mathcal{N}(x \mid \mu, \sigma^2)$,$\mu_{\mathrm{ML}}, \sigma^2_{\mathrm{ML}}$ 由(2.57)与(2.58)给出, 则(2.59)与(2.60)成立. ♠

证明

先算二阶混合矩: 由独立性, 当 $n \neq m$ 时 $\mathbb{E}[x_n x_m] = \mu^2$; 当 $n = m$ 时 $\mathbb{E}[x_n^2] = \mu^2 + \sigma^2$. 把两者统一写成

$$\mathbb{E}[x_n x_m] = \mu^2 + I_{nm}\sigma^2, \qquad I_{nm} = \begin{cases} 1, & n = m, \\ 0, & n \neq m. \end{cases}$$

于是由(2.57)与期望的线性性,$\mathbb{E}[\mu_{\mathrm{ML}}] = \dfrac{1}{N}\sum_n \mathbb{E}[x_n] = \mu$, 即(2.59).

再算 $\mathbb{E}[\mu_{\mathrm{ML}}^2]$. 由 $\mathrm{var}[\mu_{\mathrm{ML}}] = \dfrac{1}{N^2}\sum_n \mathrm{var}[x_n] = \sigma^2/N$(独立性使交叉项消失) 与命题 2.10,

$$\mathbb{E}\left[\mu_{\mathrm{ML}}^2\right] = \mathrm{var}\left[\mu_{\mathrm{ML}}\right] + \mu^2 = \frac{\sigma^2}{N} + \mu^2.$$

而 $\mathbb{E}[x_n \mu_{\mathrm{ML}}] = \dfrac{1}{N}\sum_m \mathbb{E}[x_n x_m] = \dfrac{1}{N}\left(N\mu^2 + \sigma^2\right) = \mu^2 + \dfrac{\sigma^2}{N}$. 把(2.58)展开并逐项取期望:

$$\begin{aligned} \mathbb{E}\left[\sigma_{\mathrm{ML}}^2\right] &= \frac{1}{N}\sum_{n=1}^{N}\left(\mathbb{E}[x_n^2] - 2\mathbb{E}[x_n\mu_{\mathrm{ML}}] + \mathbb{E}[\mu_{\mathrm{ML}}^2]\right) \\ &= \frac{1}{N}\sum_{n=1}^{N}\left((\mu^2+\sigma^2) - 2\left(\mu^2 + \frac{\sigma^2}{N}\right) + \left(\mu^2 + \frac{\sigma^2}{N}\right)\right) \\ &= \frac{1}{N}\sum_{n=1}^{N}\sigma^2\left(1 - \frac{1}{N}\right) = \frac{N-1}{N}\sigma^2, \end{aligned}$$

即(2.60).

□

注

(重构新增) 证明的关键是 I_{nm}: 它把“同一点”与“不同点”的二阶矩统一成一个公式, 从而使 $\mathbb{E}[x_n\mu_{\mathrm{ML}}]$ 与 $\mathbb{E}[\mu_{\mathrm{ML}}^2]$ 都能一次算出. 这个技巧在练习 2.16中原样出现.

结论中的 $(N-1)/N$ 因子解释了为什么样本方差通常除以 $N-1$: 除以 N 会低估 σ^2, 而(2.63) 的修正恰好抵消这一因子. 条件“独立同分布”不可省: 若 x_n 之间有相关性,$\mathrm{var}[\mu_{\mathrm{ML}}]$ 不再是 σ^2/N, 上述计算需改写.

可以看到, 当对给定大小的数据集进行平均时, 均值的最大似然解等于真实的均值. 然而, 方差的最大似然估计会变为真实方差的 $(N-1)/N$ 倍. 这是一个被称作偏差 (bias) 现象的例子, 即一个随机变量的估计与真实值存在系统性的差异. 这个结果的直觉解释如图 2.10所示.

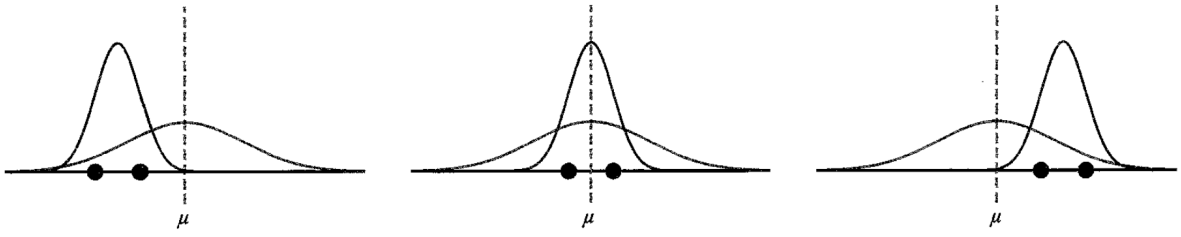


图 2.10: 当使用最大似然来确定 Gauss 分布的均值和方差时, 偏差如何产生的图示. 红色曲线表示所生成数据的真实 Gauss 分布, 三条蓝色曲线表示使用最大似然结果 [(2.57)和(2.58)] 对 3 个数据集进行拟合得到的 Gauss 分布, 每个数据集由绿色所示的两个数据点组成. 这 3 个数据集上的均值是正确的, 但方差被系统地低估了, 因为它是相对于样本均值而不是相对于真实均值测量的.

注意, 偏差的产生是因为方差是相对于均值的最大似然估计来测量的, 而均值本身是根据数据调整的. 相反, 假设我们可以获得真实的均值 μ, 并用它来估计方差:

$$\tilde{\sigma}^2 = \frac{1}{N}\sum_{n=1}^{N}(x_n - \mu)^2 \tag{2.61}$$

我们可以得到 (见练习 2.17)

$$\mathbb{E}\left[\tilde{\sigma}^2\right] = \sigma^2 \tag{2.62}$$

它是无偏的. 当然, 我们只有观测数据, 因而无法获得真正的均值. 根据(2.60), 对于 Gauss 分布, 方差参数的以下估计是无偏的:

$$\tilde{\sigma}^2 = \frac{N}{N-1}\sigma^2_{\mathrm{ML}} = \frac{1}{N-1}\sum_{n=1}^{N}\left(x_n - \mu_{\mathrm{ML}}\right)^2 \tag{2.63}$$

命题 2.17 (方差的无偏估计)

设 $x_1, \cdots, x_N$ 独立同分布于 $\mathcal{N}(x \mid \mu, \sigma^2)$,$\tilde{\sigma}^2$ 由(2.61)定义, 则由(2.63)定义的估计满足 $\mathbb{E}[\tilde{\sigma}^2] = \sigma^2$, 即(2.62)成立.

♠

证明

对(2.61)取期望, 并用 $\mathbb{E}[(x_n-\mu)^2] = \mathrm{var}[x_n] = \sigma^2$:

$$\mathbb{E}\left[\tilde{\sigma}^2\right] = \frac{1}{N}\sum_{n=1}^{N}\mathbb{E}\left[(x_n-\mu)^2\right] = \frac{1}{N}\cdot N\sigma^2 = \sigma^2.$$

这证明了(2.62). 再由(2.60),

$$\mathbb{E}\left[\frac{N}{N-1}\sigma^2_{\mathrm{ML}}\right] = \frac{N}{N-1}\cdot\frac{N-1}{N}\sigma^2 = \sigma^2,$$

故(2.63)中的估计是无偏的.

□

注

(重构新增) 两条估计的差别只在于中心:(2.61)用真实的 μ,(2.63)用 μ_{ML} 但把分母改为 $N-1$. 后者的代价是除以 $N-1$ 会使估计在 N 很小时方差较大, 因此“无偏”并不等于“更好”——这是偏差—方差权衡 (参见节 4.3) 的一个简单实例.

条件 $N \geqslant 2$ 不可省:$N=1$ 时 $N-1=0$,(2.63)无定义, 这与“单个观测无法估计方差”的直觉一致.

我们注意到, 随着数据点数量 N 的增加, 最大似然解的偏差变得不显著. 当 $N \to \infty$ 时, 方差的最大似然解等于所生成数据分布的真实方差. 在 Gauss 分布下, 除了 N 比较小的情况, 这种偏差不是很严重的问题. 然而, 在本书中, 我们会关注许多具备大量参数的复杂模型, 对它们而言, 与最大似然相关的偏差问题将会严重得多. 事实上, 最大似然中的偏差问题与过拟合 (over-fitting) 问题密切相关 (参见小节 2.6.3).

2.3.4 线性回归

我们已经知道可以利用误差最小化来表示线性回归问题. 下面让我们回到这个示例并从概率的角度看待, 从而进一步了解误差函数和正则化 (参见节 1.2).

回归问题的目标是, 通过使用包含 N 个输入值 $\mathbf{x} = (x_1, \cdots, x_N)$ 及对应目标值 $\mathbf{t} = (t_1, \cdots, t_N)$ 的训练数据集, 对于给定的新的输入变量 x, 能够对目标变量 t 进行预测. 我们可以用概率分布来表示对于目标变量值的不确定性. 为此, 我们假设给定 x 值, 对应的 t 值服从 Gauss 分布, 其均值为(1.1)中多项式曲线的值 $y(x, \mathbf{w})$, 方差为 σ^2, 其中 $\mathbf{w}$ 是多项式系数, 因此有

$$p(t \mid x, \mathbf{w}, \sigma^2) = \mathcal{N}\left(t \mid y(x, \mathbf{w}), \sigma^2\right) \tag{2.64}$$

图 2.11对此做了示意说明.

我们现在使用训练数据 $\{\mathbf{x}, \mathbf{t}\}$, 通过最大似然来确定未知参数 $\mathbf{w}$ 和 σ^2 的值. 假设数据从(2.64)所示的分布中独立产生, 则似然函数为

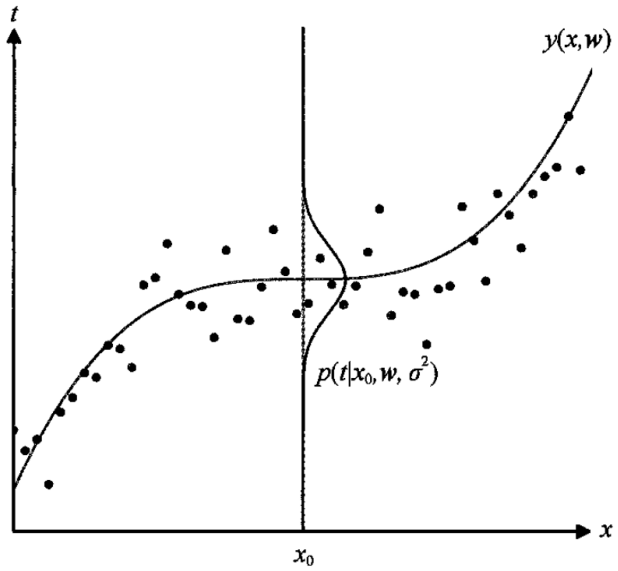


图 2.11: 通过(2.64)定义的给定 x 时 t 的 Gauss 条件分布, 其中均值由多项式函数 $y(x,\mathbf{w})$ 给出, 方差由参数 σ^2 给出.

$$p(\mathbf{t}\mid\mathbf{x},\mathbf{w},\sigma^2)=\prod_{n=1}^{N}\mathcal{N}\left(t_n\mid y(x_n,\mathbf{w}),\sigma^2\right) \tag{2.65}$$

正如我们之前对简单 Gauss 分布所做的那样, 我们可以很方便地最大化似然函数的对数. 代入(2.49)所示的 Gauss 分布, 得到对数似然函数的形式为

$$\ln p(\mathbf{t}\mid\mathbf{x},\mathbf{w},\sigma^2)=-\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left\{y(x_n,\mathbf{w})-t_n\right\}^2-\frac{N}{2}\ln\sigma^2-\frac{N}{2}\ln(2\pi) \tag{2.66}$$

首先对多项式系数的最大似然解进行估计, 用 $\mathbf{w}_{\mathrm{ML}}$ 表示, 它们是通过对 $\mathbf{w}$ 最大化(2.66)来确定的. 为此, 我们可以省略(2.66)右边的最后两项, 因为它们与 $\mathbf{w}$ 无关. 此外, 我们注意到使用一个正的常数系数将对数似然缩放后, 不会改变最大值相对于 $\mathbf{w}$ 的位置, 因此可以将系数 $1/2\sigma^2$ 替换为 $1/2$. 最后, 我们将最大化对数似然等价地替换为最小化负对数似然. 可以看到, 当 $\mathbf{w}$ 确定时, 最大化似然等价于最小化(2.67)定义的平方和误差函数 (sum-of-squares error function):

$$E(\mathbf{w})=\frac{1}{2}\sum_{n=1}^{N}\left\{y(x_n,\mathbf{w})-t_n\right\}^2 \tag{2.67}$$

命题 2.18 (Gauss 噪声下的最大似然等价于最小平方和误差)

设 $t_n=y(x_n,\mathbf{w})+\varepsilon_n$, 其中 ε_n 独立同分布于 $\mathcal{N}(0,\sigma^2)$, 则关于 $\mathbf{w}$ 最大化(2.66)等价于关于 $\mathbf{w}$ 最小化(2.67).

证明

(2.66)右端只有第一项依赖 $\mathbf{w}$, 且它以负号出现、系数为正. 因此关于 $\mathbf{w}$ 最大化该式等价于关于 $\mathbf{w}$ 最小化

$$\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left\{y(x_n,\mathbf{w})-t_n\right\}^2.$$

再乘以正常数 $2\sigma^2$(不改变最小值点) 即得(2.67). 反过来, 最小化(2.67)的点也使(2.66)取最大, 因为二者只相差一个正常数因子与一个与 $\mathbf{w}$ 无关的常数.

□

注

(重构新增) 这条等价性是全书把“误差函数”与“概率模型”联系起来的枢纽: 平方和误差并不是人为选定的, 它等价于“噪声服从 Gauss 分布”这一假设下的最大似然. 因此, 凡是使用平方损失的地方, 都可以解读为隐含地假定了 Gauss 噪声.

证明只用到一个事实:σ^2 固定时,(2.66)中依赖 $\mathbf{w}$ 的部分就是平方和. 若 σ^2 也随 $\mathbf{w}$ 变化 (例如异方差噪声), 这一等价性不再成立; 若噪声服从 Laplace 分布, 对应的则是绝对误差而非平方误差 (见定义 2.14).

因此, 平方和误差函数是在 Gauss 噪声分布的假设下最大化似然的结果.

也可以使用最大似然来确定方差参数 σ^2. 关于 σ^2 最大化(2.66)可得 (见练习 2.18)

$$\sigma^2_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}\left\{y(x_n, \mathbf{w}_{\mathrm{ML}}) - t_n\right\}^2 \tag{2.68}$$

命题 2.19 (噪声方差的最大似然解)

在命题 2.18的条件下, 关于 σ^2 最大化(2.66)得到的解由(2.68)给出.

证明

记 $S(\mathbf{w}) = \sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2$. 对(2.66)关于 σ^2 求偏导并令其为零:

$$\frac{\partial}{\partial \sigma^2}\left(-\frac{S(\mathbf{w})}{2\sigma^2} - \frac{N}{2}\ln\sigma^2\right) = \frac{S(\mathbf{w})}{2\sigma^4} - \frac{N}{2\sigma^2} = 0,$$

解得 $\sigma^2 = S(\mathbf{w})/N$. 由于 $\mathbf{w}$ 与 σ^2 的解解耦 (与定理 2.4的情形相同), 把 $\mathbf{w} = \mathbf{w}_{\mathrm{ML}}$ 代入即得(2.68).

□

注

(重构新增) 这条结果说明: 在拟合好均值函数之后, 噪声方差的最大似然估计就是残差的均方值. 它与(2.58)形式完全一致, 只是把 x_n 换成了 $y(x_n, \mathbf{w}_{\mathrm{ML}})$, 因此在样本量小时同样存在 $(N-1)/N$ 型的低估.

使用它时要注意 $\mathbf{w}_{\mathrm{ML}}$ 必须先确定: 若把 $\mathbf{w}$ 与 σ^2 的方程联立, 虽然解相同, 但推导会更繁琐. 此外,(2.68)只在 $\sigma^2 > 0$ 时有意义; 若残差全为零 (模型完全插值), 则 $\sigma^2_{\mathrm{ML}} = 0$, 分布退化.

我们可以首先确定控制均值的参数向量 $\mathbf{w}_{\mathrm{ML}}$, 然后与简单 Gauss 分布情况一样, 用它来找到方差 σ^2_{ML}.

在确定了参数 $\mathbf{w}$ 和 σ^2 后, 就可以对 x 的新值进行预测了. 我们现在有了一个预测分布 (predictive distribution) 方面的概率模型. 不同于简单地给出一个点估计, 该模型给出了 t 的概率分布, 它可以通过将最大似然参数代入(2.64)得到

$$p\left(t \mid x, \mathbf{w}_{\mathrm{ML}}, \sigma^2_{\mathrm{ML}}\right) = \mathcal{N}\left(t \mid y(x, \mathbf{w}_{\mathrm{ML}}), \sigma^2_{\mathrm{ML}}\right) \tag{2.69}$$

定义 2.27 (预测分布)

设已由训练数据确定参数, 则给定新输入 x 后目标变量 t 的**预测分布** (predictive distribution) 是条件密度 $p(t \mid x, \mathcal{D})$. 当参数取最大似然值、模型由(2.64)给定时, 预测分布由(2.69)给出.

注

(重构新增) 预测分布与点估计的差别在于它保留了不确定性:(2.69)不仅给出“预测值 $y(x,\mathbf{w}_{\mathrm{ML}})$”, 还给出围绕它的整个 Gauss 分布, 其宽度由 σ^2_{ML} 决定. 这样才可能回答“预测有多可靠”这类问题.

(2.69)只反映了噪声带来的不确定性 (偶然不确定性), 没有反映参数 $\mathbf{w}$ 本身的不确定性 (认知不确定性); 后者需要把 $\mathbf{w}$ 的后验也积分掉, 正是小节 2.6.3中(2.121)的做法.

2.4 密度变换

本节讨论概率密度在变量非线性变化下如何转换. 在讨论一类名为规范化流 (normalizing flow) 的生成式模型时 (参见第 18 章), 这个性质至关重要, 同时它还凸显了概率密度在此类变换下的行为不同于简单函数.

考虑单变量 x, 假设我们通过 $x=g(y)$ 进行变量变换, 那么函数 $f(x)$ 就变成了新的函数 $\tilde{f}(y)$, 定义为

$$\tilde{f}(y)=f\big(g(y)\big) \tag{2.70}$$

现在考虑概率密度 $p_x(x)$, 再次使用 $x=g(y)$ 变换变量, 产生关于新变量 y 的密度 $p_y(y)$, 其中下标表示 $p_x(x)$ 和 $p_y(y)$ 是不同的密度. 在 δx 很小的情况下, 落在区间 $(x,x+\delta x)$ 内的观测值会被变换到区间 $(y,y+\delta y)$ 内, 其中 $x=g(y)$, 并且 $p_x(x)\delta x\approx p_y(y)\delta y$. 因此, 如果我们取 $\delta x\to 0$ 的极限, 就会得到

$$p_y(y)=p_x(x)\left|\frac{\mathrm{d}x}{\mathrm{d}y}\right|=p_x\big(g(y)\big)\left|\frac{\mathrm{d}g}{\mathrm{d}y}\right| \tag{2.71}$$

定义 2.28 (变量变换与密度变换)

设 $x=g(y)$ 是可微的变量替换,f 为任一函数. 称由(2.70)定义的 $\tilde{f}$ 为 f 在变量变换 $x=g(y)$ 下的**像函数**. 若 p_x 是 x 的概率密度, 则称按(2.71)得到的 p_y 为 y 的**概率密度**; 称(2.71)为**密度变换**公式. ♣

注

(重构新增) 为什么函数直接复合、而密度必须乘一个导数因子? 因为密度是“单位长度上的概率”, 而长度的度量随变量改变:x 上的区间长度 δx 与 y 上对应区间长度 δy 之比是 $|\mathrm{d}x/\mathrm{d}y|$.(2.71)正是把这个长度比补齐, 使概率质量 $p_x\delta x$ 与 $p_y\delta y$ 相等.

在规范化流 (第 18 章) 中, 正是反复使用(2.71)把简单密度 (如标准 Gauss) 变成复杂密度, 因此这个公式是那类生成模型的全部数学基础.

这里引入模数 $|\cdot|$ 是因为导数 $\mathrm{d}y/\mathrm{d}x$ 可能为负, 而密度是通过两个长度的比值缩放的, 该比例恒为正.

命题 2.20 (一元密度变换公式)

设 $x=g(y)$ 是连续可微的严格单调函数,$g'(y)\neq 0$,p_x 为 x 的密度, 则由(2.71)定义的 p_y 是 y 的合法概率密度, 且对任意区间 (a,b) 有

$$\int_a^b p_y(y)\,\mathrm{d}y=p_x\big(x\in g((a,b))\big).$$

♠

证明

概率质量守恒:y 落在 $(y,y+\delta y)$ 内当且仅当 $x=g(y)$ 落在 g 的像中对应的区间内, 该区间的长度为 $|g'(y)|\delta y+o(\delta y)$. 由定义 2.9,

$$p_y(y)\delta y=p_x\big(g(y)\big)\big|g'(y)\big|\delta y+o(\delta y),$$

两边除以 δy 并令 $\delta y\to 0$ 即得(2.71). 非负性由 $p_x\geqslant 0$ 与 $|g'|>0$ 得到; 归一化由换元积分公式

$$\int p_y(y)\,\mathrm{d}y=\int p_x\big(g(y)\big)\big|g'(y)\big|\,\mathrm{d}y=\int p_x(x)\,\mathrm{d}x=1$$

得到.

□

注

(重构新增) 命题的实质是换元积分公式在概率语言中的重述:$\mathrm{d}x = g'(y)\,\mathrm{d}y$, 而绝对值保证 $\mathrm{d}y$ 的系数为正. 这也解释了为什么 p_y 必然是密度——它自动继承非负性与归一化.

条件“严格单调、$g' \neq 0$”不可省: 若 g 不单调, 同一个 y 可能对应多个 x, 此时必须把各支的贡献相加, 公式变为 $p_y(y) = \sum_i p_x(g_i(y))|g_i'(y)|$. 若在某点 $g' = 0$, 该点附近长度被压缩为零, 密度变换公式失效.

这种变换密度的方法非常强大. 任何密度 $p(y)$ 都可以通过一个处处非零的固定密度 $q(x)$ 对变量 $y = f(x)$ 进行非线性变量变换来获得, 其中 $f(x)$ 是一个单调函数, 满足 $0 \leqslant f'(x) < \infty$(见练习 2.19).

变换性质 [见(2.71)] 的一个结果是, 概率密度的极大值取决于变量的选择. 假设 $f(x)$ 在 $\hat{x}$ 处有一个众数 (即极大值), 使得 $f'(\hat{x}) = 0$. 在(2.70)的两边对 y 求导, 得到的 $\hat{y}$ 值对应 $\tilde{f}(y)$ 的众数:

$$\tilde{f}'(\hat{y}) = f'\big(g(\hat{y})\big)\, g'(\hat{y}) = 0 \tag{2.72}$$

命题 2.21 (像函数的众数)

设 g 可微、$g'(\hat{y}) \neq 0$,$\hat{x} = g(\hat{y})$ 是 f 的驻点, 则 $\hat{y}$ 是 $\tilde{f}(y) = f(g(y))$ 的驻点, 即(2.72)成立; 特别地, 若 $f'(\hat{x}) = 0$ 且 $g' \neq 0$, 则 f 的驻点与 $\tilde{f}$ 的驻点通过 $\hat{x} = g(\hat{y})$ 一一对应.

证明

由链式法则,$\tilde{f}'(y) = f'(g(y))g'(y)$. 在 $y = \hat{y}$ 处,$g(\hat{y}) = \hat{x}$ 且 $f'(\hat{x}) = 0$, 故 $\tilde{f}'(\hat{y}) = 0 \cdot g'(\hat{y}) = 0$, 即(2.72). 反之若 $\tilde{f}'(\hat{y}) = 0$ 而 $g'(\hat{y}) \neq 0$, 则必 $f'(g(\hat{y})) = 0$, 即 $g(\hat{y})$ 是 f 的驻点.

□

注

(重构新增) 这条结论说明: 对函数作变量变换时, 极值点的位置会随变量一起变换, 因此“函数 f 在哪里取最大”与“像函数 $\tilde{f}$ 在哪里取最大”是同一个几何事实的两种坐标描述.

条件 $g'(\hat{y}) \neq 0$ 不可省: 若 g' 在 $\hat{y}$ 处为零, 则 $\tilde{f}'(\hat{y}) = 0$ 可能由 g' 而非 f' 造成, 此时 $\hat{y}$ 对应的 x 未必是 f 的驻点; 这也正是下一段说明“密度的众数不随变量变换”的入口.

假设在众数处 $g'(\hat{y}) \neq 0$, 则有 $f'(g(\hat{y})) = 0$. 我们已经知道 $f'(\hat{x}) = 0$, 与期望的一致, 以变量 x 和 y 表示的众数的位置通过 $\hat{x} = g(\hat{y})$ 相关联. 因此, 找到对于变量 x 的众数等价于先变换到变量 y, 再找到对于 y 的众数, 最后变换回 x.

现在考虑概率密度 $p_x(x)$ 在变量变换 $x = g(y)$ 下的行为, 其中新变量的密度 $p_y(y)$ 由(2.71)给出. 为了处理(2.71)中的模数, 我们可以先写出 $g'(y) = s|g'(y)|$, 其中 $s \in \{-1, +1\}$, 则(2.71)可以写成

$$p_y(y) = p_x\big(g(y)\big)\, s\, g'(y) \tag{2.73}$$

其中用到了 $1/s = s$. 在(2.73)的两边对 y 求导, 可得

$$p_y'(y) = s\, p_x'\big(g(y)\big)\big\{g'(y)\big\}^2 + s\, p_x\big(g(y)\big)\, g''(y) \tag{2.74}$$

命题 2.22 (密度的众数依赖于变量的选择)

设 p_x 可微、g 二阶可微,$s = \operatorname{sgn} g'(y)$, 则 p_y 的导数由(2.74)给出. 因此, 即使 $\hat{x}$ 是 p_x 的众数,$\hat{y} = g^{-1}(\hat{x})$ 一般也不是 p_y 的众数, 除非(2.74)的右端第二项为零.

证明

由(2.73), 把 p_y 写成两个因子之积 $p_x(g(y)) \cdot sg'(y)$(s 在 g' 不变号处为常数). 由乘积法则与链式法则,

$$p'_y(y) = \frac{\mathrm{d}}{\mathrm{d}y}\left[p_x(g(y))\right] \cdot sg'(y) + p_x(g(y)) \cdot sg''(y) = s\,p'_x(g(y))\{g'(y)\}^2 + s\,p_x(g(y))\,g''(y),$$

即(2.74). 若 $\hat{x} = g(\hat{y})$ 是 p_x 的众数, 则 $p'_x(\hat{x}) = 0$, 于是

$$p'_y(\hat{y}) = s\,p_x(\hat{x})\,g''(\hat{y}).$$

只要 $g''(\hat{y}) \neq 0$ 且 $p_x(\hat{x}) > 0$, 就有 $p'_y(\hat{y}) \neq 0$, 故 $\hat{y}$ 不是 p_y 的驻点.

□

注

(重构新增) 这条结论是本节的要害: 众数不是概率密度的内禀性质, 而是依赖于所用坐标的. 原因就在(2.74)的第二项——它是“体积元变化率”的导数, 在非线性变换下不为零, 把峰的位置推偏了.

由此可知, 用“最大后验”作点估计时, 结果会随参数的参数化方式而变 (例如对 σ 还是对 σ^2 求最大会得到不同的答案), 这是 MAP 估计相对于后验均值的一个理论缺陷. 条件 $g'' = 0$(线性变换) 时第二项消失, 众数才随变量同步变换.

由于(2.74)右边第二项的存在, 相等关系 $\hat{x} = g(\hat{y})$ 不再成立. 因此, 通过最大化 $p_x(x)$ 获得的 x 值, 不是先变换到 $p_y(y)$, 再相对于 $p_y(y)$ 最大化, 最后变换回 x 获得的值. 这导致密度的众数依赖于变量的选择. 然而, 对于线性变换,(2.74)右边的第二项消失了, 在这种情况下, 最大值的位置须根据 $\hat{x} = g(\hat{y})$ 进行变换.

我们可以用图 2.12中的简单示例来加以说明. 首先考虑图 2.12中红色曲线所示的关于 x 的 Gauss 分布 $p_x(x)$. 接下来从这个分布中抽取 $N = 50\,000$ 个点的样本, 并绘制它们的值的直方图. 与预期一致, 该直方图符合分布 $p_x(x)$. 考虑如下从 x 到 y 的非线性变量变换:

$$x = g(y) = \ln(y) - \ln(1 - y) + 5 \tag{2.75}$$

这个函数的逆由下式给出:

$$y = g^{-1}(x) = \frac{1}{1 + \exp(-x + 5)} \tag{2.76}$$

它是一个逻辑斯谛 sigmoid 函数 (logistic sigmoid), 如图 2.12中的蓝色曲线所示. 如果我们简单地将 $p_x(x)$ 作为 x 的函数进行变换, 则可以得到图 2.12中的绿色曲线 $p_x(g(y))$, 并且密度 $p_x(x)$ 的众数可以通过 sigmoid 型函数转换为这个曲线的众数. 然而,y 上的密度转换是根据(2.71)进行的, 如图 2.12中左侧的洋红色曲线所示. 注意, 这使得其众数相对于绿色曲线的众数发生了偏移.

为了确认这个结果, 我们取 50 000 个 x 值的样本, 并使用(2.76)计算相应的 y 值, 然后绘制它们的值的直方图. 可以看到, 该直方图与图 2.12中左侧的洋红色曲线匹配, 而不是绿色曲线.

例题 2.3 非线性变换下众数的偏移 在图 2.12中,$p_x(x)$ 是 x 上的 Gauss 分布 (红色曲线),g^{-1} 是(2.76)给出的逻辑斯谛 sigmoid 函数 (蓝色曲线).

(1) 若只把函数复合, 得到绿色曲线 $p_x(g(y))$, 它的峰位于 $g^{-1}(\hat{x}) = 0.5$(取 $\hat{x} = 5$ 时), 即 x 的众数被 sigmoid 直接搬到 y 轴上.

(2) 按(2.71)计算 y 上的真实密度, 得到洋红色曲线. 由于(2.74)的第二项不为零, 它的峰相对于绿色曲线的峰发生了偏移.

(3) 从 p_x 抽 50 000 个样本、按(2.76)变换后作直方图, 所得的峰与洋红色曲线一致, 而与绿色曲线不一致. 这从数值上确认了(2.71)才是正确的密度变换公式.

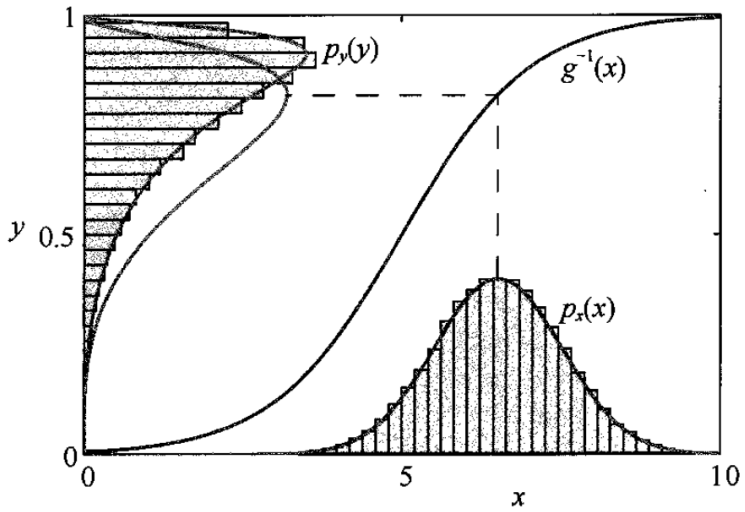


图 2.12: 非线性变量变换下密度众数的转换示例, 展示了与简单函数相比的不同行为.

注

(重构新增) 这个例子把“函数怎么变”与“密度怎么变”的差别具体化了: 绿色曲线只需要复合, 洋红色曲线还需要乘上 $|\mathrm{d}x/\mathrm{d}y|$. 若忽略这个因子 (即把 $p_x(g(y))$ 当作 $p_y(y)$), 则会得到一个积分不为 1 的函数, 它在 $y \to 0$ 或 $y \to 1$ 时会发散, 因为它没有补偿 sigmoid 在两端把区间压缩的效应.

这也说明“众数在哪个坐标系里取”必须事先约定; 在第 18 章中, 正因为密度的变换有这一因子, 才能用变量变换把简单分布精确地变成复杂分布.

多元分布

我们可以将(2.71)中的结果扩展到多个变量的密度函数上. 考虑 D 维变量 $\mathbf{x} = (x_1, \cdots, x_D)^{\mathsf{T}}$ 的密度 $p(\mathbf{x})$, 假设我们将其转换为一个新的变量 $\mathbf{y} = (y_1, \cdots, y_D)^{\mathsf{T}}$, 其中 $\mathbf{x} = \mathbf{g}(\mathbf{y})$. 这里我们限制 $\mathbf{x}$ 和 $\mathbf{y}$ 具有相同的维度. 转换后的密度由(2.71)的广义形式给出:

$$p_y(\mathbf{y}) = p_x(\mathbf{x}) \left| \det \mathbf{J} \right| \tag{2.77}$$

其中 $\mathbf{J}$ 是 Jacobi 矩阵 (Jacobian matrix), 其元素由偏导数 $J_{ij} = \partial g_i / \partial y_j$ 给出:

$$\mathbf{J} = \begin{bmatrix} \dfrac{\partial g_1}{\partial y_1} & \cdots & \dfrac{\partial g_1}{\partial y_D} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial g_D}{\partial y_1} & \cdots & \dfrac{\partial g_D}{\partial y_D} \end{bmatrix} \tag{2.78}$$

定义 2.29 (Jacobi 矩阵)

设 $\mathbf{g} = (g_1, \cdots, g_D)^{\mathsf{T}}$ 为 $\mathbf{y}$ 的可微向量函数. 称由(2.78)定义的 $D \times D$ 矩阵为 $\mathbf{g}$ 的 **Jacobi 矩阵** (Jacobian matrix), 其第 (i, j) 个元素为 $J_{ij} = \partial g_i / \partial y_j$. ♣

注

(重构新增) Jacobi 矩阵是多元函数的“导数”: 一维时它退化为 $g'(y)$, 多维时它把 D 个方向的偏导排列成矩阵, 从而可以像标量一样取行列式、求逆. 它在(2.77)中体现为体积伸缩因子, 在后文描述线性变换下协方差矩阵的变化规律时 (节 3.2) 也起同样作用.

定义要求 $\mathbf{g}$ 的每个分量都可微, 否则偏导数不存在; 这也是命题 2.23要求 $\mathbf{g}$ 连续可微的来源.

命题 2.23 (多元密度变换公式)

设 $\mathbf{x}=\mathbf{g}(\mathbf{y})$ 是连续可微、一一的变换, 且 $\det \mathbf{J} \neq 0$,p_x 为 $\mathbf{x}$ 的密度, 则由(2.77)定义的 p_y 是 $\mathbf{y}$ 的合法概率密度.

♠

证明

由多元换元积分公式, 对任意可积函数 h,

$$\int h(\mathbf{x})\, p_x(\mathbf{x})\, \mathrm{d}\mathbf{x} = \int h\big(\mathbf{g}(\mathbf{y})\big)\, p_x\big(\mathbf{g}(\mathbf{y})\big)\, \big|\det \mathbf{J}\big|\, \mathrm{d}\mathbf{y}.$$

取 $h \equiv 1$ 并用 $\int p_x(\mathbf{x})\, \mathrm{d}\mathbf{x} = 1$, 得 $\int p_y(\mathbf{y})\, \mathrm{d}\mathbf{y} = 1$, 即 p_y 归一化; 非负性由 $p_x \geqslant 0$ 与 $|\det \mathbf{J}| > 0$ 得到. 故 p_y 是密度, 且(2.77)就是换元公式的被积函数替换关系.

□

注

(重构新增) 一维时的因子 $|\mathrm{d}x/\mathrm{d}y|$ 在多元时升级为 $|\det \mathbf{J}|$, 它正是“体积元的伸缩比”: 一个无穷小立方体在变换后体积放大 $|\det \mathbf{J}|$ 倍. 取绝对值同样是为了保证密度非负, 因为 $\det \mathbf{J}$ 可以为负 (定向反转).

条件“$\mathbf{x}$ 与 $\mathbf{y}$ 维数相同、且 $\det \mathbf{J} \neq 0$”不可省: 维数不同时不存在方阵行列式, 必须引入低维流形上的密度 (参见小节 16.4.1);$\det \mathbf{J} = 0$ 时变换把正体积压成零体积, 公式失效.

直观上, 我们可以将变量的变换视为扩展某些区域的空间和收缩其他区域的空间, 同时一个围绕点 $\mathbf{x}$ 的无穷小区域 $\Delta\mathbf{x}$ 被变换到围绕点 $\mathbf{y}=\mathbf{g}(\mathbf{x})$ 的无穷小区域 $\Delta\mathbf{y}$.Jacobi 行列式的绝对值表示这些体积的比值, 同时也是在积分中改变变量时的因子.(2.78)遵循 $\Delta\mathbf{x}$ 区域内的概率质量等于 $\Delta\mathbf{y}$ 区域内的概率质量这一原则. 同样, 我们需要取模以确保密度是非负的.

如图 2.13中的上面一行所示, 我们可以通过在二维 Gauss 分布中改变变量来说明这一点. 这里从 $\mathbf{x}$ 到 $\mathbf{y}$ 的转换由下式给出 (见练习 2.20):

$$y_1 = x_1 + \tanh(5x_1) \tag{2.79}$$

$$y_2 = x_2 + \tanh(5x_2) + \frac{x_2^3}{3} \tag{2.80}$$

图 2.13中的下面一行展示了从 $\mathbf{x}$ 空间中 Gauss 分布里产生的那些样本经过转换后在 $\mathbf{y}$ 空间中对应的样本.

例题 2.4 二维非线性变换对密度的影响 在图 2.13中,$\mathbf{x}$ 空间里取标准 Gauss 分布.

(1) 左列: 把 $\mathbf{x}$ 平面上的规则网格按(2.79)、(2.80)变换到 $\mathbf{y}$ 平面, 可以看到网格在中心附近被拉伸、在边缘被压缩, 这正是 $\det \mathbf{J}$ 不为常数的几何表现.

(2) 中间列: 由于 y_1 只依赖 x_1、y_2 只依赖 x_2,Jacobi 矩阵是对角阵,$\det \mathbf{J} = \prod_i \mathrm{d}y_i/\mathrm{d}x_i$. 密度在中心被“摊平”, 在四角出现四个峰.

(3) 右列: 从 $\mathbf{x}$ 的 Gauss 分布中抽样并按同一变换映射, 得到的样本散布与中间列的密度形状一致, 从而在数值上验证了(2.77).

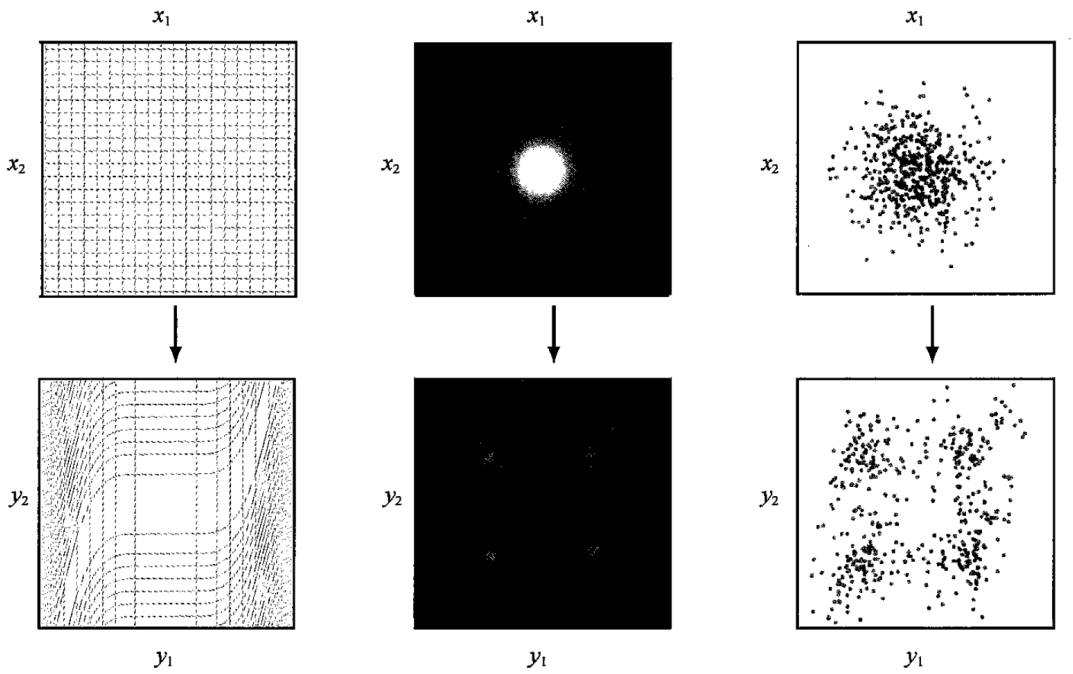


图 2.13: 在二维概率分布中改变变量的影响示意图. 左列展示了变量的变换, 中间列和右列则分别展示了对 Gauss 分布及来自该分布的样本的相应影响.

注

(重构新增) 这个例子说明多维情形下密度的形变比一维更剧烈: 一维只能拉伸或压缩, 二维还可以产生“一个峰变成多个峰”的效果, 因为 $\det \mathbf{J}$ 在平面上可以有很大的变化. 这正是规范化流能够把单峰分布变成多峰分布、从而建模复杂数据的原因.

需要注意(2.79)、(2.80)给出的映射是可逆的 (tanh 与三次项都单调), 否则(2.77)不能直接使用, 必须像一维非单调情形那样对各支求和.

2.5 信息论

概率论为另一个重要的框架——信息论 (information theory) 提供了基础. 它量化了数据集中存在的信息, 并在机器学习中扮演着重要角色. 这里简要介绍一些本书后面需要用到的信息论中的关键元素, 包括熵的重要概念及其各种变体. 关于信息论的更全面介绍及其与机器学习的联系, 参见 MacKay(2003).

2.5.1 熵

考虑一个离散随机变量 x, 我们想知道当观察到这个变量的某个特定值时能获得多少信息. 信息量可以视为我们在得知 x 的具体值时的“惊讶程度”. 如果我们被告知一个极不可能发生的事件刚刚发生了, 那么我们接受到的信息将多于某个极有可能发生的事件刚刚发生的信息. 如果我们知道某个事件确定会发生, 那么我们不会接收到任何信息. 信息内容的度量将依赖于概率分布 $p(x)$. 我们需要寻找一个量 $h(x)$, 它是概率 $p(x)$ 的单调函数, 并且表达了信息的内容. 注意, 如果两个事件 x 和 y 是无关的, 那么观察它们所获得的信息增益应该是分别观察它们所获得的信息增益之和, 即 $h(x, y) = h(x) + h(y)$. 两个无关的事件是统计独立的, 所以 $p(x, y) = p(x)p(y)$. 由这两个关系可知 $h(x)$ 必须由 $p(x)$ 的对数给出 (见练习 2.21), 所以有

$$h(x) = -\log_2 p(x) \tag{2.81}$$

定义 2.30 (信息量)

称由(2.81)定义的量 $h(x)$ 为事件 x 发生的**信息量**. 它以 2 为底的对数, 单位称为**比特** (bit, 即“二进制位”); 若改用自然对数, 单位称为**纳特** (nat).

♣

注

(重构新增) 定义中的负号是必要的:$0 < p(x) \leqslant 1$ 时 $\log_2 p(x) \leqslant 0$, 加负号后 $h(x) \geqslant 0$, 符合“信息量非负”的直觉. 同时,$p(x)$ 越小 $h(x)$ 越大, 恰好刻画了“越意外、信息越多”.

底数的选取只影响单位而不影响比较: 以 2 为底得比特, 以 e 为底得纳特, 二者相差常数因子 $\ln 2$, 即 1 纳特 ≈ 1.44 比特. 这一自由度在(2.81)中体现为“对数的底数选取是任意的”, 但一旦选定就必须在整节中保持一致.

其中负号确保了信息是正的或为零. 注意低概率事件 x 对应于高信息内容. 对数的底数选取是任意的, 此刻我们采用信息论中的惯例, 即以 2 为底数. 在这种情况下, 我们可以看到 $h(x)$ 的单位是比特 (“二进制位”).

现在假设发送者希望向接收者传输一个随机变量的值. 他们在这个过程中传输的平均信息量是通过对(2.81)关于分布 $p(x)$ 取期望得到的, 由下式给出:

$$H[x] = -\sum_x p(x)\log_2 p(x) \tag{2.82}$$

命题 2.24 (独立事件的信息量可加)

设事件 x 与 y 统计独立, 则 $h(x,y) = h(x) + h(y)$.

♠

证明

由独立性 $p(x,y) = p(x)p(y)$, 代入(2.81)并对数化积为和:

$$h(x,y) = -\log_2\big(p(x)p(y)\big) = -\log_2 p(x) - \log_2 p(y) = h(x) + h(y).$$

□

注

(重构新增) 可加性是对“信息量”这一概念的基本要求: 两个互不相关的事件所提供的总信息应当等于各自信息之和, 否则“信息”就不能像长度、时间那样累加, 编码长度的讨论也失去意义.

这条性质与(2.81)的对数形式互为因果: 正因为 log 把乘积变成和, 定义才能同时满足“单调”与“独立可加”.练习 2.21进一步证明, 在适当的连续性假设下,$-\log_2 p$ 是唯一满足这两条要求的函数形式.

这个重要的量称为随机变量 x 的熵 (entropy). 注意 $\lim_{\varepsilon\to 0}(\varepsilon\ln\varepsilon) = 0$, 所以当遇到 x 在某个取值下使 $p(x) = 0$ 时, 令 $p(x)\ln p(x) = 0$.

定义 2.31 (熵)

称由(2.82)定义的量 $H[x]$ 为离散随机变量 x 的**熵** (entropy), 它是信息量 $h(x)$ 在分布 $p(x)$ 下的期望, 即对 x 的不确定性的度量.

♣

注

(重构新增) 熵是“平均惊讶程度”: 它把每个取值的惊讶程度 $h(x)$ 按该取值出现的概率加权平均. 因此熵只依赖分布的形状, 不依赖取值的具体标签——把 x 的取值重新命名不会改变熵, 这一点与期望不同.

(2.82)中的约定 $0\ln 0 = 0$ 不可省: 否则 $p(x) = 0$ 的取值会使求和无定义. 该约定的合法性来自 $\lim_{\varepsilon\to 0}\varepsilon\ln\varepsilon = 0$, 它保证了熵对分布是连续的——概率的微小改变只会引起熵的微小改变.

到目前为止, 我们已经启发式地给出了信息 [(2.81)] 和相应的熵 [(2.82)] 的定义. 这些定义确实具有一些有用的性质. 考虑一个有 8 个可能状态的随机变量 x. 为了将 x 的值传输给接收者, 我们需要传输一条长度为 3 比特的消息. 注意, 这个变量的熵由下式给出:

$$H[x] = -8 \times \frac{1}{8}\log_2\frac{1}{8} = 3(\text{比特}) \tag{2.83}$$

举个例子 (Cover and Thomas, 1991), 一个变量有 8 个可能的状态 $\{a,b,c,d,e,f,g,h\}$, 各自的概率分别为 $\left(\frac{1}{2},\frac{1}{4},\frac{1}{8},\frac{1}{16},\frac{1}{64},\frac{1}{64},\frac{1}{64},\frac{1}{64}\right)$. 这种情况下的熵由下式给出:

$$H[x] = -\frac{1}{2}\log_2\frac{1}{2} - \frac{1}{4}\log_2\frac{1}{4} - \frac{1}{8}\log_2\frac{1}{8} - \frac{1}{16}\log_2\frac{1}{16} - 4\times\frac{1}{64}\log_2\frac{1}{64} = 2(\text{比特}) \tag{2.84}$$

可以看出, 非均匀分布的熵小于均匀分布的熵, 稍后在讨论熵的混乱度时, 我们将对此有所了解. 考虑如何将变量的状态传输给接收者. 我们既可以像之前那样使用一个 3 比特的数字, 也可以使用较短编码来表示较大可能发生的事件, 或用较长编码表示较小可能发生的事件, 从而利用非均匀分布的特点, 实现较短的平均编码长度. 这一目的可以通过使用以下一组代码字符串表示状态 $\{a,b,c,d,e,f,g,h\}$ 来实现, 例如 0、10、110、1110、111100、111101、111110 和 111111. 必须传输的代码的平均长度为

$$\text{代码平均长度} = \frac{1}{2}\times 1 + \frac{1}{4}\times 2 + \frac{1}{8}\times 3 + \frac{1}{16}\times 4 + 4\times\frac{1}{64}\times 6 = 2(\text{比特}) \tag{2.85}$$

这与随机变量的熵相同. 注意, 我们已经不能使用再短的代码字符串了, 因为必须保证由这些状态编码拼接起来的序列能被无歧义地分解为每个状态. 例如,11001110 只能唯一地解码为状态序列 c,a,d. 熵和最短编码长度之间的这种关系具有一般性.

例题 2.5 熵与平均编码长度 比较两个 8 状态随机变量.

(1) 等概率情形:$p(x_k) = 1/8$, 由(2.83),$H[x] = 3$ 比特, 恰等于用长度 3 的二进制串逐一编码所需的比特数.

(2) 非均匀情形: 概率为 (1/2, 1/4, 1/8, 1/16, 1/64, 1/64, 1/64, 1/64), 由(2.84),$H[x] = 2$ 比特, 小于等概率情形的 3 比特; 而(2.85)给出的前缀码 0, 10, 110, 1110, 111100, 111101, 111110, 111111 的平均长度也恰为 2 比特.

(3) 该前缀码可以唯一解码: 任一码字都不是另一码字的前缀, 故拼接后的串从左向右读时, 遇到第一个完整码字即可分割. 例如 11001110 → 110|0|1110 → c,a,d.

定理 2.5 (无噪编码定理)

设离散随机变量 x 取值 $x_1,\cdots,x_K$, 概率为 $p_1,\cdots,p_K$,$H[x] = -\sum_k p_k\log_2 p_k$. 则对 x 的任一前缀码, 其平均码长 $L = \sum_k p_k l_k$ 满足 $L \geqslant H[x]$; 且存在前缀码使 $L < H[x] + 1$.

♡

证明

先证下界. 设码字长度为 $l_1,\cdots,l_K$. 前缀码的 Kraft 不等式成立:$\sum_{k=1}^{K} 2^{-l_k} \leqslant 1$(前缀码对应一棵二叉树的叶, 第 k 个叶可分配长度 2^{-l_k} 的互不重叠的单位区间, 故这些长度之和不超过 1). 记 $q_k = 2^{-l_k}$、$K_0 = \sum_k q_k \leqslant 1$, 则由对数求和不等式 (见下面的说明) 取 $a_k = p_k$、$b_k = q_k$,

$$L - H[x] = \sum_{k=1}^{K} p_k\log_2\frac{p_k}{q_k} \geqslant \left(\sum_{k=1}^{K} p_k\right)\log_2\frac{\sum_{k=1}^{K} p_k}{\sum_{k=1}^{K} q_k} = \log_2\frac{1}{K_0} \geqslant 0,$$

故 $L \geqslant H[x]$. 这里用到的对数求和不等式为: 对 $a_k, b_k > 0$,

$$\sum_k a_k\ln\frac{a_k}{b_k} \geqslant \left(\sum_k a_k\right)\ln\frac{\sum_k a_k}{\sum_k b_k},$$

它由初等不等式 $\ln t \geqslant 1 - 1/t$ $(t > 0)$ 得到: 令 $A = \sum_k a_k$、$B = \sum_k b_k$, 则

$$\sum_k \frac{a_k}{A}\ln\frac{a_k B}{b_k A} \geqslant \sum_k \frac{a_k}{A}\left(1 - \frac{b_k A}{a_k B}\right) = \sum_k \frac{a_k}{A} - \sum_k \frac{b_k}{B} = 1 - 1 = 0.$$

再证上界: 取 $l_k = \lceil -\log_2 p_k \rceil$, 则 $2^{-l_k} \leqslant p_k$, 从而 $\sum_k 2^{-l_k} \leqslant \sum_k p_k = 1$, 由 Kraft 不等式的逆定理存在以这些长度为码长的前缀码; 而 $l_k < -\log_2 p_k + 1$, 故 $L = \sum_k p_k l_k < H[x] + 1$.

□

注

(重构新增) 定理把“熵”从一个启发式的量提升为一个可操作的下界: 任何无损压缩方案的平均码长都不可能低于熵, 而熵加 1 比特之内总能达到. 这也解释了(2.84)与(2.85)数值相同并非巧合.

证明的关键是 Kraft 不等式, 它把“前缀码存在”转化为“一组长度满足 $\sum 2^{-l_k} \leqslant 1$”. 条件“前缀码”不可省: 若允许非前缀码, 则可能出现歧义解码 (例如把 0 与 01 同时作为码字), 此时下界不再成立. 定理是 Shannon(1948) 的经典结果, 本书只引用其结论.

从现在开始, 我们将改用自然对数来定义熵, 以便与本书其他部分的观点联系起来. 在这种情况下, 熵以纳特 (nat)(源自“自然对数”) 而不是比特为单位度量,1 纳特约 1.44 比特.

2.5.2 物理学视角

我们已经以明确随机变量特定状态所需的平均信息量的形式介绍了熵的概念. 实际上, 熵的概念在物理学中有着更早的起源. 熵在平衡热力学的背景下被引入, 后续随着统计力学的发展, 熵被赋予更深层次的解释, 即作为无序程度的度量指标. 我们也可以从这种视角来理解熵. 假设有 N 个相同的物品要被分配到一组箱子中, 其中第 i 个箱子中有 n_i 个物品. 有多少种不同的方式可以将物品分配到箱子中. 选择第一个物品有 N 种方式, 选择第二个物品有 $(N-1)$ 种方式, 以此类推, 总共有 $N!$(读作“N 的阶乘”) 种方式将所有 N 个物品分配到箱子中, 其中 $N!$ 表示乘积 $N \times (N-1) \times \cdots \times 2 \times 1$. 然而, 我们无须区分每个箱子内的物品是否重新排列. 在第 i 个箱子中重新排列物品的方式有 $n_i!$ 种, 因此将 N 个物品分配到箱子中的总方式数由下式给出:

$$W = \frac{N!}{\prod_i n_i!} \tag{2.86}$$

W 表示多重数 (multiplicity). 然后, 熵被定义为多重数的对数并通过常数因子 $1/N$ 变换尺度, 所以有

$$H = \frac{1}{N} \ln W = \frac{1}{N} \ln N! - \frac{1}{N} \sum_i \ln n_i! \tag{2.87}$$

考虑极限 $N \to \infty$, 其中分数 n_i/N 保持固定, 并应用 Stirling 公式

$$\ln N! \approx N \ln N - N \tag{2.88}$$

可以得到

$$H = -\lim_{N \to \infty} \sum_i \frac{n_i}{N} \ln \frac{n_i}{N} = -\sum_i p_i \ln p_i \tag{2.89}$$

定义 2.32 (多重数、微观状态与宏观状态)

设 N 个物品被分配到若干箱子中, 第 i 个箱子中有 n_i 个物品. 称(2.86)给出的 W 为该分配的**多重数** (multiplicity). 在物理学中, 物品到箱子的具体分配称为**微观状态** (microstate), 而仅由各箱物品数 (用比例 n_i/N 表示) 决定的分配方式称为**宏观状态** (macrostate);W 表达给定宏观状态所包含的微观状态数量, 也称该宏观状态的**权重** (weight).

♣

注

(重构新增) 引入多重数的动机是把“熵”与“数数”联系起来: 一个宏观状态可以由越多的微观状态实现 (W 越大), 它就越“无序”, 熵也越大.(2.87)中的 $1/N$ 只是把总量归一为“每个物品的熵”, 使 H 与 N 无关, 从而在 $N \to \infty$ 时有极限.

定义区分微观与宏观状态是必要的: 同一宏观状态 (同样的 n_i) 对应许多微观状态 (物品的具体归属), 若把二者混同,W 就无从定义. 箱子数固定而 $N \to \infty$ 时,n_i/N 趋于稳定的分布 p_i, 这正是(2.89)的前提.

命题 2.25 (熵的统计力学表达式)

在 $N \to \infty$ 且 $n_i/N \to p_i$ 固定的极限下, 由(2.87)定义的 H 满足(2.89), 即 $H = -\sum_i p_i \ln p_i$.

证明

把 Stirling 公式(2.88)用于(2.87)中的两个阶乘:

$$H \approx \frac{1}{N}\left(N\ln N - N\right) - \frac{1}{N}\sum_i \left(n_i \ln n_i - n_i\right) = \ln N - 1 - \sum_i \frac{n_i}{N}\ln n_i + \sum_i \frac{n_i}{N}.$$

由 $\sum_i n_i = N$, 末项为 1, 与 -1 相消; 再把 $\ln n_i = \ln N + \ln(n_i/N)$ 代入,

$$H \approx \ln N - \sum_i \frac{n_i}{N}\left(\ln N + \ln\frac{n_i}{N}\right) = \ln N - \ln N \sum_i \frac{n_i}{N} - \sum_i \frac{n_i}{N}\ln\frac{n_i}{N} = -\sum_i \frac{n_i}{N}\ln\frac{n_i}{N}.$$

令 $N \to \infty$ 并用 $n_i/N \to p_i$, 即得(2.89).

□

注

(重构新增) 这条推导说明信息论的熵与统计力学的熵是同一个量 (相差一个常数因子, 即 Boltzmann 常数). 它的关键是 Stirling 公式把阶乘换成对数, 从而使乘积 $N!$ 化为求和——与命题 2.24中对数化积的技巧同源.

条件“n_i/N 保持固定”不可省: 若各箱物品比例随 N 变化, 则(2.89)右端的极限不存在,H 没有确定值.Stirling 公式本身的误差为 $O(\ln N)$, 相对误差在 $N \to \infty$ 时趋于零, 因此不影响极限.

其中,$\sum_i n_i = N$,$p_i = \lim(n_i/N)$ 是物品被分配到第 i 个箱子中的概率. 在物理学中, 物品到箱子的具体分配称为微观状态 (microstate), 而分配物品数的总体分布 (用比例 n_i/N 表示) 称为宏观状态 (macrostate). 多重数 W 表达了给定宏观状态中的微观状态数量, 也称宏观状态的权重 (weight).

我们可以将箱子解释为离散随机变量 X 的状态 x_i, 其中 $p(X = x_i) = p_i$, 则随机变量 X 的熵为

$$H[p] = -\sum_i p(x_i)\ln p(x_i) \tag{2.90}$$

如图 2.14所示, 密集围绕少数几个峰值的分布 $p(x_i)$ 将具有相对较低的熵, 而那些更均匀地散布在许多值上的分布将具有更高的熵.

因为 $0 \leqslant p_i \leqslant 1$, 所以熵是非负的, 并且当其中一个 $p_i = 1$ 且所有其他 $p_{j\neq i} = 0$ 时, 熵会等于其极小值 0. 通过使用 Lagrange 乘子来确保概率的归一化约束, 我们可以找到熵的极大值 (参见附录 C). 最大化下式:

$$\tilde{H} = -\sum_i p(x_i)\ln p(x_i) + \lambda\left(\sum_i p(x_i) - 1\right) \tag{2.91}$$

我们可以发现所有的 $p(x_i)$ 都是相等的, 且由 $p(x_i) = 1/M$ 给出, 其中 M 是状态 x_i 的总数. 然后熵的相应值是 $H = \ln M$(见练习 2.22). 这个结果也可以从 Jensen 不等式 (下文将很快讨论它) 得出 (见练习 2.23). 为了验证驻点确实是一个最大值, 可以计算熵的二阶导数, 得到

$$\frac{\partial^2 \tilde{H}}{\partial p(x_i)\,\partial p(x_j)} = -I_{ij}\frac{1}{p_i} \tag{2.92}$$

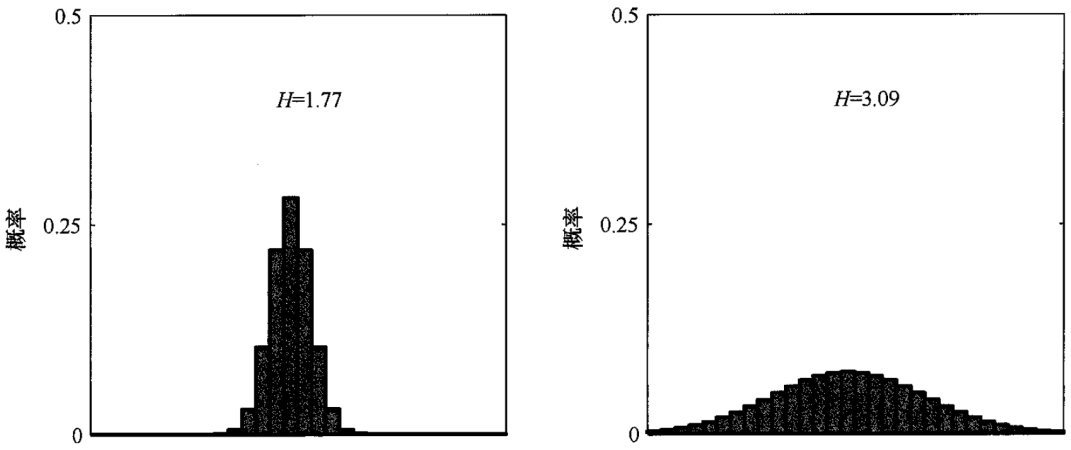


图 2.14: 30 个分箱上两种概率分布的直方图说明了更宽的分布具有更大的熵 H. 最大的熵来自均匀分布, 它使 $H = -\ln \dfrac{1}{30} = 3.40$.

其中,I_{ij} 是单位矩阵的元素. 可以看到这些值都是负的, 因此驻点确实是一个极大值.

命题 2.26 (均匀分布使熵最大)

设离散随机变量 x 有 M 个可能取值, 则 $\sum_i p_i = 1$、$p_i \geqslant 0$ 的约束下,(2.82)(以自然对数计) 在 $p_i = 1/M$(对一切 i) 处取最大值, 最大值为 $H = \ln M$.

证明

考虑 Lagrange 函数(2.91). 对 $p(x_i)$ 求偏导并令其为零:

$$\frac{\partial \tilde{H}}{\partial p(x_i)} = -\ln p(x_i) - 1 + \lambda = 0 \quad \Longrightarrow \quad p(x_i) = e^{\lambda - 1},$$

即所有 $p(x_i)$ 相等. 再由约束 $\sum_i p_i = 1$ 得 $Me^{\lambda-1} = 1$, 故 $p(x_i) = 1/M$. 此时

$$H = -\sum_{i=1}^{M} \frac{1}{M} \ln \frac{1}{M} = \ln M.$$

最后验证它是极大值: 由(2.92),Hessian 为对角矩阵, 对角元 $-1/p_i < 0$, 故为负定, 驻点即最大值点.

□

注

(重构新增) 这条结果给出了“熵最大”的精确含义: 在只知道取值个数、不知道其他任何信息时, 最不偏不倚的选择就是均匀分布. 这与图 2.14中右侧那个更平的直方图对应, 也是“最大熵原理”在离散情形下的全部内容.

条件的清单要看清: 约束只有两条——归一化 $\sum p_i = 1$ 与非负性 $p_i \geqslant 0$. 若再加约束 (例如规定均值), 最大值就会变成指数族分布而不是均匀分布 (见小节 2.5.4的连续版本). 二阶导验证不可省: 一阶条件只给出驻点,(2.92)才把它确认为最大值.

2.5.3 微分熵

我们可以将熵的定义扩展到连续变量 x 的分布 $p(x)$ 上. 首先将 x 分成宽度为 Δ 的区间, 然后假设 $p(x)$ 是连续的, 中值定理 (mean value theorem)(Weinstein, 1999) 告诉我们, 对于每个这样的区间, 必定存在一个值 x_i 在范围 $i\Delta \leqslant x_i \leqslant (i+1)\Delta$ 内, 使得

$$\int_{i\Delta}^{(i+1)\Delta} p(x)\,\mathrm{d}x = p(x_i)\Delta \tag{2.93}$$

现在我们可以通过将任何落入第 i 个区间的值 x 赋予 x_i 来量化连续变量 x. 观察到值 x_i 的概率是 $p(x_i)\Delta$. 这给出了一个离散分布, 其熵的形式为

$$H_\Delta = -\sum_i p(x_i)\Delta \ln\big(p(x_i)\Delta\big) = -\sum_i p(x_i)\Delta \ln p(x_i) - \ln \Delta \tag{2.94}$$

其中 $\sum_i p(x_i)\Delta = 1$, 这是由(2.93)和(2.25)得出的. 现在忽略(2.94)右侧的第二项 $-\ln\Delta$, 因为它与 $p(x)$ 无关, 然后考虑极限 $\Delta \to 0$. 在此极限下,(2.94)右侧的第一项将趋近于 $p(x)\ln p(x)$ 的积分, 因此有

$$\lim_{\Delta\to 0}\left\{-\sum_i p(x_i)\Delta \ln p(x_i)\right\} = -\int p(x)\ln p(x)\,\mathrm{d}x \tag{2.95}$$

定义 2.33 (微分熵)

称(2.95)右端的量 $-\int p(x)\ln p(x)\,\mathrm{d}x$ 为连续随机变量 x 的**微分熵** (differential entropy). 对于定义在多个连续变量上的密度 (统一用向量 $\mathbf{x}$ 表示), 微分熵为

$$H[\mathbf{x}] = -\int p(\mathbf{x})\ln p(\mathbf{x})\,\mathrm{d}\mathbf{x}.$$

♣

注

(重构新增) 微分熵是把熵的定义“先离散化、再取极限”得到的:(2.94)右侧的第一项在 $\Delta \to 0$ 时收敛, 第二项 $-\ln\Delta$ 却发散, 因此微分熵只保留了收敛的那部分. 代价是它不再具有离散熵的直观含义 (不是“比特数”), 并且可以为负.

这一点在(2.102)中会看到: 当 $\sigma^2 < 1/(2\pi e)$ 时微分熵为负. 尽管有这一缺陷, 微分熵在比较同一坐标系下的不同分布时仍然有用, 也是 KL 散度与最大熵推导的基础.

式(2.95)右侧的量称为微分熵 (differential entropy). 我们可以看到, 离散形式和连续形式的熵的不同之处在于 $\ln\Delta$, 而它在极限 $\Delta \to 0$ 下是发散的. 这表明非常精确地描述一个连续变量需要大量比特位.

2.5.4 最大熵

我们已经看到, 对于离散分布, 最大熵的情况对应变量概率在可能状态之间均匀分布的情况. 现在我们来考虑连续变量的对应结果. 如果要明确定义这个最大值, 则需要约束 $p(x)$ 的一阶矩和二阶矩, 并保持归一化约束. 因此, 我们需要在以下 3 个约束下最大化微分熵:

$$\int_{-\infty}^{\infty} p(x)\,\mathrm{d}x = 1 \tag{2.96}$$

$$\int_{-\infty}^{\infty} x\,p(x)\,\mathrm{d}x = \mu \tag{2.97}$$

$$\int_{-\infty}^{\infty} (x-\mu)^2 p(x)\,\mathrm{d}x = \sigma^2 \tag{2.98}$$

可以使用 Lagrange 乘子法来进行带约束的最大化 (参见附录 C), 为此需要最大化以下关于 $p(x)$ 的函数:

$$\begin{aligned} &-\int_{-\infty}^{\infty} p(x)\ln p(x)\,\mathrm{d}x + \lambda_1\left(\int_{-\infty}^{\infty} p(x)\,\mathrm{d}x - 1\right) \\ &+ \lambda_2\left(\int_{-\infty}^{\infty} x\,p(x)\,\mathrm{d}x - \mu\right) + \lambda_3\left(\int_{-\infty}^{\infty} (x-\mu)^2 p(x)\,\mathrm{d}x - \sigma^2\right) \end{aligned} \tag{2.99}$$

使用变分法 (参见附录 B), 令上述函数的导数为零, 可以得到

$$p(x) = \exp\left\{-1 + \lambda_1 + \lambda_2 x + \lambda_3 (x-\mu)^2\right\} \tag{2.100}$$

通过将这个结果代回 3 个约束方程可以得到 Lagrange 乘子, 最终得到如下结果 (见练习 2.24):

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{2.101}$$

定理 2.6 (最大熵分布是 Gauss 分布)

在约束(2.96)、(2.97)、(2.98)下, 使微分熵最大的密度由(2.101)给出, 即均值为 μ、方差为 σ^2 的 Gauss 分布. ♡

证明

对(2.99)关于 $p(x)$ 作变分, 令其为零:

$$\frac{\delta}{\delta p(x)}\left[-\int p\ln p\,\mathrm{d}x + \lambda_1\int p\,\mathrm{d}x + \lambda_2\int xp\,\mathrm{d}x + \lambda_3\int (x-\mu)^2 p\,\mathrm{d}x\right] = -\ln p(x) - 1 + \lambda_1 + \lambda_2 x + \lambda_3(x-\mu)^2 = 0,$$

即得(2.100). 把(2.100)代入三个约束: 由(2.98),λ_3 必须为负 (否则(2.96)的积分发散), 记 $\lambda_3 = -1/(2\sigma^2)$, 则(2.100)成为 Gauss 分布的二次指数形式; 再由(2.97), 线性项的中心为 μ(即 $\lambda_2 = \mu/\sigma^2$ 时均值为 μ); 最后由(2.96)定出归一化常数, 得(2.101). 约束(2.98)同时给出二次项的系数, 故方差确为 σ^2.

□

注

(重构新增) 这条定理解释了节 2.3开头那句话:Gauss 分布不是“形式任意”的, 而是在只固定前两阶矩时最不偏不倚的选择. 这也是为什么在许多模型中默认采用 Gauss 噪声——它在线索最少时引入了最少的额外假设.

三个约束缺一不可: 若去掉(2.98), 最大熵解不存在 (可把方差推向无穷使熵任意增大); 若去掉(2.97), 解仍为 Gauss 但中心不确定; 若去掉(2.96), 则连归一化都无法保证. 定理的证明只用到变分法, 但结果确实满足 $p(x) \geqslant 0$, 因此在最大化时无需额外施加非负约束.

因此, 最大化微分熵的分布是 Gauss 分布. 注意, 在最大化熵时我们没有限制分布是非负的. 然而, 结果分布确实是非负的, 事后看来这样的约束是不必要的.

如果我们计算 Gauss 分布的微分熵, 则可以得到 (见练习 2.25)

$$H[x] = \frac{1}{2}\left(1 + \ln(2\pi\sigma^2)\right) \tag{2.102}$$

命题 2.27 (Gauss 分布的微分熵)

设 $x \sim \mathcal{N}(x \mid \mu, \sigma^2)$, 则其微分熵由(2.102)给出. ♠

证明

把(2.49)代入微分熵的定义并利用 $\int p = 1$:

$$\begin{aligned} H[x] &= -\int p(x)\left[-\frac{1}{2}\ln(2\pi\sigma^2) - \frac{(x-\mu)^2}{2\sigma^2}\right]\mathrm{d}x \\ &= \frac{1}{2}\ln(2\pi\sigma^2)\int p(x)\,\mathrm{d}x + \frac{1}{2\sigma^2}\int (x-\mu)^2 p(x)\,\mathrm{d}x \\ &= \frac{1}{2}\ln(2\pi\sigma^2) + \frac{1}{2\sigma^2}\cdot\sigma^2 = \frac{1}{2}\left(1 + \ln(2\pi\sigma^2)\right), \end{aligned}$$

其中第三个等号用了(2.98).

□

注

(重构新增) 这条结果把“熵”与“方差”直接联系起来: 方差越大, 熵越大, 这与图 2.14的直观一致. 它也给出微分熵可以为负的明确阈值: 当 $\sigma^2 < 1/(2\pi e)$ 时 $H[x] < 0$, 而与离散熵的非负性形成对照.

推导依赖(2.98)(即二阶中心矩等于 σ^2), 这一步同时用到了命题 2.13的结论. 注意公式只依赖 σ^2 而不依赖 μ: 平移不改变熵, 这与离散熵对标签重命名不变是同一回事.

我们可以再次看到, 随着分布变得更宽 (即 σ^2 增加), 熵也增加. 这个结果还表明, 与离散熵不同, 微分熵可以是负的, 因为当 $\sigma^2 < 1/(2\pi e)$ 时,(2.102)中的 $H(x) < 0$.

2.5.5 Kullback-Leibler 散度

到目前为止, 我们已经介绍了信息论中的许多概念, 包括熵的关键符号表示. 下面让我们将这些想法与机器学习联系起来. 考虑某个未知分布 $p(x)$, 并假设我们使用了一个近似分布 $q(x)$ 来对它进行建模. 如果使用 $q(x)$ 来构建编码方案以便向接收者传输 x 的值, 那么由于使用 $q(x)$ 而不是真实分布 $p(x)$(假设我们选择了一种高效的编码方案), 描述 x 值所需的平均额外 (additional) 信息量 (单位: 纳特) 可由下式给出:

$$\mathrm{KL}(p\|q) = -\int p(x)\ln q(x)\,\mathrm{d}x - \left(-\int p(x)\ln p(x)\,\mathrm{d}x\right) = -\int p(x)\ln\frac{q(x)}{p(x)}\,\mathrm{d}x \tag{2.103}$$

定义 2.34 (Kullback-Leibler 散度)

称由(2.103)定义的量为分布 $p(x)$ 与 $q(x)$ 之间的**相对熵** (relative entropy), 又称 **Kullback-Leibler** 散度 (Kullback-Leibler divergence), 简称 **KL** 散度 (Kullback and Leibler, 1951). 它度量用 q 代替 p 时所需的平均额外信息量.

♣

注

(重构新增) KL 散度的定义来自“编码代价之差”: 用真实分布 p 编码的期望码长是 $-\int p\ln p$(即微分熵), 用近似分布 q 编码的期望码长是 $-\int p\ln q$, 两者之差就是(2.103). 因此它不是对称的:q 在 p 的支撑集上取小值会受到重罚, 反之则不会.

这种不对称性是 KL 散度的固有性质 ($\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$), 使用时必须明确“谁是真值、谁是近似”. 此外,(2.103)要求 p, q 都归一化, 否则右端会多出与归一化常数有关的项; 在变分推断中常见的 ELBO 就是在这种非归一化情形下使用的.

这就是分布 $p(x)$ 和 $q(x)$ 之间的相对熵 (relative entropy), 又称 Kullback-Leibler 散度 (Kullback-Leibler divergence), 简称 KL 散度 (Kullback and Leibler, 1951). 注意, 它不是一个对称量, 也就是说,$\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$.

Kullback-Leibler 散度满足 $\mathrm{KL}(p\|q) \geqslant 0$, 当且仅当 $p(x) = q(x)$ 时等式成立. 为了进行证明, 我们首先引入凸 (convex) 函数的概念. 如图 2.15所示, 如果函数 $f(x)$ 具有每条弦都位于函数上方或与函数重合的性质, 则称该函数是凸的.

定义 2.35 (凸函数与凹函数)

若函数 f 满足: 对任意 a, b 与任意 $0 \leqslant \lambda \leqslant 1$,

$$f\big(\lambda a + (1-\lambda)b\big) \leqslant \lambda f(a) + (1-\lambda)f(b),$$

即每条弦都位于函数图像上方或与图像重合, 则称 f 是**凸的** (convex). 若上式对 $\lambda \in (0,1)$ 严格成立 (除 $a = b$ 外), 则称 f 是**严格凸的** (strictly convex). 若 $-f$ 是凸的, 则称 f 是**凹的** (concave), 相应地有严格凹函数.

♣

注

(重构新增) 凸性刻画的是“函数图像向下弯”: 弦在图像上方意味着函数值的增长越来越快 (二阶导非负). 这一性质的价值在于它保证局部极小就是全局极小, 从而使优化问题变简单; 在信息论中, 它则是证明 KL 散度非负的关键.

定义中的 λ 可以解释为 a, b 之间的插值权重:$\lambda a + (1-\lambda)b$ 遍历区间 $[a, b]$. 注意凸性要求对**所有** λ 成立, 只在个别点成立不叫凸. 若 f 二阶可导, 凸性等价于 $f'' \geqslant 0$(见练习 2.32), 这也是实际检验时最常用的判据.

任何处在 $x = a$ 到 $x = b$ 区间中的 x 值都可以写成 $\lambda a + (1-\lambda)b$ 的形式, 其中 $0 \leqslant \lambda \leqslant 1$. 弦上对应的点由 $\lambda f(a) + (1-\lambda)f(b)$ 给出, 函数的对应值是 $f(\lambda a + (1-\lambda)b)$. 函数 $f(x)$ 是凸的意味着

$$f\big(\lambda a + (1-\lambda)b\big) \leqslant \lambda f(a) + (1-\lambda)f(b) \tag{2.104}$$

这等价于要求函数的二阶导数在任何地方都是正的 (见练习 2.32). 凸函数的例子有 $x \ln x (x > 0)$ 和 x^2. 如果一个函数仅在 $\lambda = 0$ 和 $\lambda = 1$ 时 $\mathrm{KL}(p\|q) = 0$ 成立, 则称其为严格凸的 (strictly convex). 如果一个函数具有相反的性质, 即每条弦都位于函数之下或与函数重合, 则称其为凹的 (concave), 对应也存在严格凹的 (strictly concave) 函数. 如果函数 $f(x)$ 是凸的, 那么 $-f(x)$ 是凹的.

注

(重构新增) 原文问题说明上一段中“如果一个函数仅在 $\lambda = 0$ 和 $\lambda = 1$ 时 $\mathrm{KL}(p\|q) = 0$ 成立, 则称其为严格凸的”一句, 原书把“等号”误印成了 $\mathrm{KL}(p\|q) = 0$: 严格凸的定义与 KL 散度无关, 正确表述应为“若(2.104)中的等号仅在 $\lambda = 0$ 和 $\lambda = 1$ 时成立, 则称 f 为严格凸的”. 正文照录原文, 特此说明; 后文凡涉及严格凸性的论证均按上述正确表述使用.

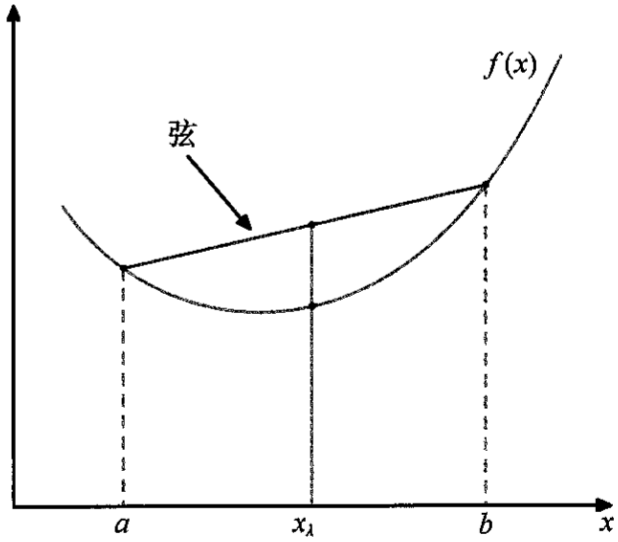


图 2.15: 凸函数 $f(x)$ 的每条弦 (蓝色) 都位于函数 (红色) 之上或与函数重合.

使用归纳法, 从(2.104)可以得到 (见练习 2.33) 如下结论: 对于任何一组点 $\{x_i\}$, 凸函数 $f(x)$ 须满足

$$f\left(\sum_{i=1}^{M} \lambda_i x_i\right) \leqslant \sum_{i=1}^{M} \lambda_i f(x_i) \tag{2.105}$$

其中 $\lambda_i \geqslant 0$ 且 $\sum_{i=1}^{M} \lambda_i = 1$.(2.105)称为 Jensen 不等式 (Jensen's inequality). 如果我们将 λ_i 解释为离散变量 x 在 $\{x_i\}$ 上取值的概率分布, 则(2.105)可以写成

$$f\big(\mathbb{E}[x]\big) \leqslant \mathbb{E}\big[f(x)\big] \tag{2.106}$$

其中 $\mathbb{E}[\cdot]$ 表示期望. 对于连续变量,Jensen 不等式具有以下形式:

$$f\left(\int x\,p(x)\,\mathrm{d}x\right) \leqslant \int f(x)\,p(x)\,\mathrm{d}x \tag{2.107}$$

定理 2.7 (Jensen 不等式)

设 f 为凸函数,$\lambda_i \geqslant 0$、$\sum_i \lambda_i = 1$, 则(2.105)成立; 若 x 为随机变量, 则(2.106)与(2.107)成立, 即 $f(\mathbb{E}[x]) \leqslant \mathbb{E}[f(x)]$.

证明

先对 M 作归纳证明(2.105).$M = 1$ 时两边相等;$M = 2$ 时即(2.104). 设结论对 $M-1$ 成立, 则对 M 项 (不妨设 $\lambda_M < 1$), 令 $\Lambda = \sum_{i=1}^{M-1} \lambda_i = 1 - \lambda_M$,

$$\begin{aligned} f\left(\sum_{i=1}^{M} \lambda_i x_i\right) &= f\left(\Lambda \cdot \sum_{i=1}^{M-1} \frac{\lambda_i}{\Lambda} x_i + \lambda_M x_M\right) \leqslant \Lambda f\left(\sum_{i=1}^{M-1} \frac{\lambda_i}{\Lambda} x_i\right) + \lambda_M f(x_M) \\ &\leqslant \Lambda \sum_{i=1}^{M-1} \frac{\lambda_i}{\Lambda} f(x_i) + \lambda_M f(x_M) = \sum_{i=1}^{M} \lambda_i f(x_i), \end{aligned}$$

其中第一个不等号用了 $M = 2$ 的凸性, 第二个用了归纳假设 (注意 λ_i/Λ 仍非负且和为 1).

(2.106)是(2.105)在 $\lambda_i = p(x_i)$、$M \to \infty$ 时的写法;(2.107)则把求和换成积分, 证明方式相同 (把积分看作和的极限).

□

注

(重构新增) Jensen 不等式是信息论中几乎所有非负性结论的统一来源:KL 散度非负、互信息非负、最大熵的解是 Gauss 分布, 都可由它推出. 直观上, 它说的是“先平均再取凸函数”不大于“先取凸函数再平均”.

条件“f 凸”不可省: 对凹函数不等号反向 ($f(\mathbb{E}[x]) \geqslant \mathbb{E}[f(x)]$). 条件“$\lambda_i \geqslant 0$、$\sum \lambda_i = 1$”也不可省, 它保证 $\sum \lambda_i x_i$ 是凸组合, 从而落在 f 的定义域内. 若 f 严格凸, 等号仅在 x 几乎必然为常数时成立.

我们可以将(2.107)所示形式的 Jensen 不等式应用于 Kullback-Leibler 散度 [见(2.103)] , 得到

$$\mathrm{KL}(p\|q) = -\int p(x) \ln \frac{q(x)}{p(x)}\,\mathrm{d}x \geqslant -\ln \int q(x)\,\mathrm{d}x = 0 \tag{2.108}$$

命题 2.28 (KL 散度的非负性)

设 p, q 为概率密度, 则 $\mathrm{KL}(p\|q) \geqslant 0$, 且等号成立当且仅当 $p(x) = q(x)$(几乎处处).

证明

在(2.107)中取 $f(u) = -\ln u$(它是凸函数, 见练习 2.32), 并取随机变量 $u = q(x)/p(x)$、其分布由 p 给出, 则

$$\mathrm{KL}(p\|q) = -\int p(x) \ln \frac{q(x)}{p(x)}\,\mathrm{d}x = \mathbb{E}_p\left[-\ln u\right] \geqslant -\ln \mathbb{E}_p[u] = -\ln \int p(x) \frac{q(x)}{p(x)}\,\mathrm{d}x = -\ln \int q(x)\,\mathrm{d}x = 0.$$

若 $p = q$, 被积函数 $\ln(q/p) = \ln 1 = 0$, 故 $\mathrm{KL} = 0$. 反之, 若 $\mathrm{KL} = 0$, 则 Jensen 不等式取等号; 由 $-\ln$ 严格凸, 这要求 $u = q(x)/p(x)$ 在 $p(x) > 0$ 处为常数, 结合双方归一化得该常数为 1, 即 $p = q$(几乎处处).

□

注

(重构新增) 非负性是 KL 散度被称为“散度”的资格: 它度量两个分布的不相似程度, 且最小值 0 只在两个分布相同时取到. 证明只用了一步——把 $-\ln$ 的凸性代入 Jensen 不等式——却给出了整个信息论中最常用的不等式.

条件“p, q 都归一化”在证明的最后一个等号中用到 ($\int q = 1$); 若 q 未归一化, 则 $\mathrm{KL}(p\|q) \geqslant -\ln \int q$, 可以为负. 此外, 若 $q(x) = 0$ 而 $p(x) > 0$ 在某个正测度集上成立, 则 $\mathrm{KL} = +\infty$, 此时不等式仍成立 (广义实数意义下).

其中利用了 $-\ln x$ 是凸函数这一性质，并且利用了归一化条件 $\int q(x)\,\mathrm{d}x = 1$. 实际上，$-\ln x$ 是严格凸函数，所以当且仅当对所有 x 有 $q(x) = p(x)$ 时等式成立. 可以将 Kullback-Leibler 散度解释为对两个分布 $p(x)$ 和 $q(x)$ 不相似度的度量.

可以看到，数据压缩和密度估计 (建模未知概率分布的问题) 之间有着密切的关系，因为当我们知道真实分布时，就可以实现最有效的压缩. 如果我们使用与真实分布不同的分布，那么最终只能得到一种效率较低的编码方式，并且平均来说，必须传输的额外信息 (至少) 等于两个分布之间的 Kullback-Leibler 散度.

假设数据是从一个我们希望建模的未知分布 $p(x)$ 中产生的. 我们可以尝试使用由一组可调参数 $\boldsymbol{\theta}$ 控制的参数化分布 $q(x \mid \boldsymbol{\theta})$ 来近似这个分布. 确定 $\boldsymbol{\theta}$ 的一种方法是最小化 $p(x)$ 和 $q(x \mid \boldsymbol{\theta})$ 之间的 Kullback-Leibler 散度. 但是由于我们不知道 $p(x)$，因此无法直接做到这一点. 然而，假设我们已经观察到一组有限的训练点——从 $p(x)$ 中产生的 x_n，其中 $n = 1, \cdots, N$，则可以使用(2.40)，通过这些训练点上的有限和来近似关于 $p(x)$ 的期望:

$$\mathrm{KL}(p\|q) \approx \frac{1}{N}\sum_{n=1}^{N}\big(-\ln q(x_n \mid \boldsymbol{\theta}) + \ln p(x_n)\big) \tag{2.109}$$

式(2.109)右侧的第二项与 $\boldsymbol{\theta}$ 无关，第一项是分布 $q(x \mid \boldsymbol{\theta})$ 使用训练集评估时 $\boldsymbol{\theta}$ 的负对数似然函数. 因此可以看到，最小化这个 Kullback-Leibler 散度等同于最大化对数似然函数 (见练习 2.34).

命题 2.29 (KL 散度与对数似然)

设 $q(x \mid \boldsymbol{\theta})$ 为参数化密度，则关于 $\boldsymbol{\theta}$ 最小化 $\mathrm{KL}(p\|q)$ 等价于关于 $\boldsymbol{\theta}$ 最大化期望对数似然 $\mathbb{E}_p[\ln q(x \mid \boldsymbol{\theta})]$; 在样本近似(2.109)下等价于最大化 $\sum_n \ln q(x_n \mid \boldsymbol{\theta})$.

♠

证明

把(2.103)拆开:

$$\mathrm{KL}(p\|q) = -\int p(x)\ln q(x \mid \boldsymbol{\theta})\,\mathrm{d}x + \int p(x)\ln p(x)\,\mathrm{d}x.$$

右端第二项与 $\boldsymbol{\theta}$ 无关，故最小化左端等价于最大化第一项，即 $\mathbb{E}_p[\ln q(x \mid \boldsymbol{\theta})]$. 再用(2.40)把该期望换成样本平均 $\frac{1}{N}\sum_n \ln q(x_n \mid \boldsymbol{\theta})$，乘以常数 N 不影响最优点，即得第二种表述.

□

注

(重构新增) 这条等价性是“最大似然 = 最小 KL”的精确版本，它把小节 2.3.2中的似然方法与信息论联系起来: 选择模型时，我们实际上是在让模型分布尽量靠近真实分布.

条件“p 固定”不可省:p 决定了 KL 散度中不可优化的那一项，因此最小化只对 $\boldsymbol{\theta}$ 进行. 若用有限样本近似(2.40)，则所得解是最大似然解，它的抽样误差由 N 控制——这也解释了为什么数据越多，学到的分布越接近真值.

2.5.6 条件熵

现在考虑两组变量 x 和 y 之间的联合分布 $p(x, y)$，从中采样 x 和 y 的值对. 如果 x 的值已知，那么明确对应 y 值所需的额外信息由 $-\ln p(y \mid x)$ 给出. 因此，明确 y 所需的平均额外信息可以写成

$$H[y \mid x] = -\iint p(y, x)\ln p(y \mid x)\,\mathrm{d}y\,\mathrm{d}x \tag{2.110}$$

定义 2.36 (条件熵)

称由(2.110)定义的量 $H[y \mid x]$ 为给定 x 的条件下 y 的**条件熵** (conditional entropy)，它表示在已知 x 后描述 y 平均所需的额外信息量.

♣

注

(重构新增) 条件熵是“在已知 x 之后还剩多少不确定性”, 因此它必然不大于无条件熵 (练习 2.29). 这个差别正是互信息 (小节 2.5.7) 要量化的东西.

(2.110)中的 $-\ln p(y \mid x)$ 用的是条件密度而不是联合密度, 这是它与联合熵的唯一差别; 求和 (积分) 仍然按联合分布 $p(x, y)$ 加权, 因为实际观测到的 (x, y) 就是按联合分布出现的. 条件 $p(x) > 0$ 不可省, 否则 $p(y \mid x)$ 无定义.

$H[y \mid x]$ 称为给定 x 的条件下 y 的条件熵 (conditional entropy). 应用乘积法则容易看出, 条件熵满足如下关系 (见练习 2.35):

$$H[x, y] = H[y \mid x] + H[x] \tag{2.111}$$

其中 $H[x, y]$ 是 $p(x, y)$ 的微分熵,$H[x]$ 是边缘分布 $p(x)$ 的微分熵. 因此, 描述 x 和 y 所需的信息实际就是描述 x 本身所需的信息再加上给定 x 描述 y 所需的额外信息.

命题 2.30 (熵的链式法则)

设 $p(x, y)$ 为联合密度,$p(x)$、$p(y \mid x)$ 为相应的边缘与条件密度, 则(2.111)成立.

证明

由 $p(x, y) = p(y \mid x)p(x)$((2.30)) 取对数得 $\ln p(x, y) = \ln p(y \mid x) + \ln p(x)$. 两边乘以 $-p(x, y)$ 并对 (x, y) 积分:

$$\begin{aligned} H[x, y] &= -\iint p(x, y) \ln p(x, y) \,\mathrm{d}x \,\mathrm{d}y \\ &= -\iint p(x, y) \ln p(y \mid x) \,\mathrm{d}x \,\mathrm{d}y - \iint p(x, y) \ln p(x) \,\mathrm{d}x \,\mathrm{d}y. \end{aligned}$$

第一项即 $H[y \mid x]$. 第二项中 $\ln p(x)$ 与 y 无关, 对 y 积分得 $\int p(x) \ln p(x) \,\mathrm{d}x$ 的相反数, 即 $H[x]$. 故(2.111)成立. □

注

(重构新增) 链式法则把“联合的不确定性”分解为“x 的不确定性”加“已知 x 后 y 剩余的不确定性”, 这与乘积法则把联合概率分解为两步完全平行——事实上证明就是乘积法则取对数.

它的直接推论是 $H[x, y] \leqslant H[x] + H[y]$(练习 2.29), 等号当且仅当 x, y 独立. 条件“$p(x) > 0$”用于保证 $p(y \mid x)$ 有定义; 若某个 x 取值的概率为零, 该处对积分的贡献也为零, 不影响结论.

2.5.7 互信息

当两个变量 x 和 y 独立时, 它们的联合分布便可以分解为它们边缘分布的乘积, 即 $p(x, y) = p(x)p(y)$. 如果这两个变量不独立, 则可以通过考虑联合分布和边缘分布乘积之间的 Kullback-Leibler 散度来了解它们是否“接近”独立, 其由下式给出:

$$I[x, y] = \mathrm{KL}\big(p(x, y) \,\|\, p(x)p(y)\big) = -\iint p(x, y) \ln \frac{p(x)p(y)}{p(x, y)} \,\mathrm{d}x \,\mathrm{d}y \tag{2.112}$$

定义 2.37 (互信息)

称由(2.112)定义的量 $I[x, y]$ 为变量 x 与 y 之间的**互信息** (mutual information), 它是联合分布 $p(x, y)$ 与其边缘分布之积 $p(x)p(y)$ 之间的 KL 散度.

注

(重构新增) 互信息回答的是“两个变量有多不独立”: 当且仅当 $p(x,y)=p(x)p(y)$ 时它为零, 因此它比协方差更强——协方差只捕捉线性相关, 而互信息捕捉任何形式的依赖 (练习 2.39中的例子协方差为零但互信息不为零).

由 KL 散度的性质,$I[x,y]$ 非负且不对称中的“方向”并不存在 ($I[x,y]=I[y,x]$, 因为 $p(x)p(y)$ 对称), 这与 KL 散度本身的不对称形成对比. 定义要求 $p(x,y)$ 与 $p(x)p(y)$ 都可归一化, 故 $p(x)>0$、$p(y)>0$ 在相应的支撑集上成立.

$I[x,y]$ 称为变量 x 和 y 之间的互信息 (mutual information). 从 Kullback-Leibler 散度的性质中我们可以看到 $I[x,y]\geqslant 0$, 当且仅当 x 和 y 独立时等式成立. 使用概率的加和法则和乘积法则, 我们可以看到互信息与条件熵有关 (见练习 2.38):

$$I[x,y]=H[x]-H[x\mid y]=H[y]-H[y\mid x] \tag{2.113}$$

命题 2.31 (互信息与熵的关系)

设 $H[x]$、$H[x\mid y]$ 存在, 则(2.113)成立, 且 $I[x,y]\geqslant 0$, 等号成立当且仅当 x 与 y 独立.

证明

把(2.112)中的对数拆开:$\ln\dfrac{p(x)p(y)}{p(x,y)}=\ln p(x)+\ln p(y)-\ln p(x,y)$. 于是

$$I[x,y]=-\iint p(x,y)\ln p(x)\,\mathrm{d}x\,\mathrm{d}y-\iint p(x,y)\ln p(y)\,\mathrm{d}x\,\mathrm{d}y+\iint p(x,y)\ln p(x,y)\,\mathrm{d}x\,\mathrm{d}y.$$

第一项等于 $H[x]$(对 y 积分得 $p(x)$), 第二项等于 $H[y]$, 第三项等于 $-H[x,y]$. 故

$$I[x,y]=H[x]+H[y]-H[x,y].$$

再由链式法则(2.111),$H[x,y]=H[x]+H[y\mid x]=H[y]+H[x\mid y]$, 代入即得(2.113). 非负性由 $I=\mathrm{KL}(p(x,y)\|p(x)p(y))\geqslant 0$(命题 2.28) 得到, 等号成立当且仅当 $p(x,y)=p(x)p(y)$, 即独立.

□

注

(重构新增) (2.113)把互信息写成“先验熵减去后验熵”:$H[x]$ 是观测 y 之前对 x 的不确定性,$H[x\mid y]$ 是观测之后剩余的不确定性, 二者之差就是 y 提供的关于 x 的信息量. 这与定义 2.7中先验、后验的关系完全平行.

非负性说明“观测不会增加不确定性”(信息不会有害), 等号成立当且仅当观测与 x 独立. 条件 $H[x]$ 存在不可省: 若熵发散, 则(2.113)两端可能同时为无穷, 差值无意义.

因此, 互信息表示由于被告知 y 值而减小的关于 x 的不确定性 (或反过来). 从 Bayes 视角来看, 可以将 $p(x)$ 视为 x 的先验分布, 而将 $p(x\mid y)$ 视为观测到新数据 y 后的后验分布. 综上, 互信息代表了由于新观测 y 而减小的关于 x 的不确定性.

2.6 Bayes 概率

在探讨图 2.2中的弯曲硬币示例时, 我们以随机、可重复事件的频率介绍了概率的概念, 例如硬币落地时凹面朝上的概率, 称为概率的经典 (classical) 或频率学派 (frequentist) 解释. 我们还介绍了更通用的 Bayes(Bayesian) 观点, 即概率提供了对不确定性的量化. 在这种情况下, 我们所说的不确定性是指硬币的凹面是正面还是反面.

使用概率来表示不确定性是不可避免的, 而非临时选择, 因为我们需要尊重常识并做出理性和连贯的推断. 例如,Cox(1946) 指出, 如果用数值来表示信念的程度, 那么通过一组编码了信念等常识性质的简单公理, 就可以唯一地导出一组操作信念程度的法则, 这些法则等同于概率的加和法则和乘积法则. 因此, 将这些量称为 (Bayes) 概率是自然的.

对于之前假设的弯曲硬币, 在没有更多信息的情况下, 我们假设硬币凹面为正面的概率是 0.5. 如果现在得知了几次抛硬币的结果, 那么在直觉上, 这似乎应该能提供一些信息来帮助我们判断硬币凹面是否为正面. 例如, 假

设我们看到硬币多次抛掷后反面朝上的次数远多于正面朝上的次数. 考虑到硬币更有可能凹面朝上, 这为凹面更有可能是反面提供了证据. 事实上, 这种直觉是正确的, 而且进一步讲, 我们可以使用概率法则来量化这一点 (见练习 2.40). 现在,Bayes 定理有了新的意义, 因为我们可以通过结合抛掷硬币提供的数据, 将凹面是正面的先验概率转换为后验概率. 而且这个过程是迭代式的, 这意味着当进一步结合抛掷硬币的数据时, 之前的后验概率就成了新的先验概率.

Bayes 观点的一个方面是, 先验知识的引入是自然的. 例如, 假设一个看起来正常的硬币被抛掷 3 次, 并且每次的结果都是正面朝上. 最大似然估计给出抛掷后硬币正面朝上的概率是 1(参见小节 3.1.2), 这表明未来所有的抛掷结果都会是正面朝上! 相比之下,Bayes 方法只要有任合理性的先验, 就不会给出如此极端的结论.

2.6.1 模型参数

Bayes 视角为机器学习的多个方面提供了有价值的见解, 我们可以使用正弦曲线回归的例子 (参见节 1.2) 来进行说明. 这里用 $\mathcal{D}$ 表示训练数据集. 我们在线性回归的问题中看到了可以使用最大似然 (maximum likelihood) 来选择参数, 其中 $\mathbf{w}$ 被估计为能使似然函数 $p(\mathcal{D} \mid \mathbf{w})$ 最大化的值. 这相当于选择 $\mathbf{w}$ 值, 使得观察到数据集的概率最大化. 在机器学习文献中, 似然函数的负对数称为误差函数 (error function). 因为似然函数的负对数是一个单调递减的函数, 所以最大化似然等同于最小化误差. 这样便确定了参数的值, 用 $\mathbf{w}_{\mathrm{ML}}$ 表示, 用来对新数据进行预测.

定义 2.38 (误差函数)

称似然函数的负对数 $-\ln p(\mathcal{D} \mid \mathbf{w})$ 为**误差函数** (error function). 由于 $-\ln$ 是单调递减的, 最小化误差函数与最大化似然函数等价.

♣

注

(重构新增) 把“最大化似然”改写成“最小化误差”, 好处是使优化问题符合“损失越小越好”的习惯, 并使后面的正则化项可以自然地加到误差函数上 (见(2.120)).节 2.3中的平方和误差(2.67)正是 Gauss 噪声下负对数似然的常数倍.

“单调递减”只是等价性的依据, 不能推出两者的数值相等: 误差函数的值与似然的极大值并不相同, 因此不能把误差函数当作概率使用. 此外, 误差函数的定义依赖数据 $\mathcal{D}$, 换一组数据就得到另一个函数.

可以看到, 不同训练数据集的选择 (例如包含不同数量的数据点) 会产生不同的 $\mathbf{w}_{\mathrm{ML}}$ 解. 从 Bayes 视角来看, 也可以使用概率论的工具来描述模型参数中的这种不确定性. 我们可以在观测数据之前以先验概率分布 $p(\mathbf{w})$ 的形式融入对 $\mathbf{w}$ 的假设. 观测到训练数据集 $\mathcal{D}$ 的概率可通过似然函数 $p(\mathcal{D} \mid \mathbf{w})$ 来表示,Bayes 定理在这里的形式如下:

$$p(\mathbf{w} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w})}{p(\mathcal{D})} \tag{2.114}$$

定理 2.8 (参数形式的 Bayes 定理)

设 $\mathbf{w}$ 为模型参数,$\mathcal{D}$ 为观测数据,$p(\mathcal{D}) > 0$, 则参数的后验分布由(2.114)给出, 即

$$p(\mathbf{w} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w})}{p(\mathcal{D})},$$

其中 $p(\mathbf{w})$ 是先验分布,$p(\mathcal{D} \mid \mathbf{w})$ 是似然函数,$p(\mathcal{D})$ 是证据 (evidence).

♡

证明

把定理 2.2中的 X、Y 分别取为 $\mathcal{D}$ 与 $\mathbf{w}$, 即得(2.10)在参数情形下的形式(2.114). 条件 $p(\mathcal{D}) > 0$ 保证分母不为零.

□

注

(重构新增) 这条定理把 Bayes 定理从“随机变量”搬到了“参数”上: 参数不再是固定的未知数, 而是带有分布的随机量, 于是“推断参数”就变成了“计算后验分布”. 这一步是 Bayes 学派与频率学派的分水岭.

条件 $p(\mathcal{D})>0$ 不可省, 它是分母; 在实践中 $p(\mathcal{D})$ 往往难以计算, 但它与 $\mathbf{w}$ 无关, 因此在求后验的众数或比较不同 $\mathbf{w}$ 时可以直接略去 (见(2.115)).

这使得我们能够在观测到 $\mathcal{D}$ 之后, 以后验概率 $p(\mathbf{w}\mid\mathcal{D})$ 的形式评估 $\mathbf{w}$ 的不确定性.

需要强调的是, 量 $p(\mathcal{D}\mid\mathbf{w})$ 在被看作参数向量 $\mathbf{w}$ 的函数时, 称为似然函数. 它表达了对于不同的 $\mathbf{w}$ 值, 观测到训练数据集 $\mathcal{D}$ 的可能性有多大. 注意, 似然函数 $p(\mathcal{D}\mid\mathbf{w})$ 不是 $\mathbf{w}$ 上的概率分布, 其对 $\mathbf{w}$ 的积分不 (一定) 等于 1.

有了似然的定义, 我们可以将 Bayes 定理表述为

$$\text{后验}\propto\text{似然}\times\text{先验} \tag{2.115}$$

命题 2.32 (后验正比于似然乘先验)

在定理 2.8的条件下, 后验分布与“似然乘先验”只相差一个与 $\mathbf{w}$ 无关的常数, 即(2.115).

证明

(2.114)右端的分母 $p(\mathcal{D})$ 不依赖于 $\mathbf{w}$, 故

$$p(\mathbf{w}\mid\mathcal{D})=\frac{1}{p(\mathcal{D})}\cdot p(\mathcal{D}\mid\mathbf{w})p(\mathbf{w})\propto p(\mathcal{D}\mid\mathbf{w})p(\mathbf{w}).$$

□

注

(重构新增) (2.115)是全部 Bayes 推断的工作形式: 我们只需写出“似然 × 先验”, 不必计算分母, 就能确定后验的形状; 需要归一化时再除以它的积分. 这也解释了为什么(2.114)中的证据在实践中常被省略.

“$\propto$”是对 $\mathbf{w}$ 而言的, 不能理解为对数据也成立; 此外, 比例常数依赖于 $\mathcal{D}$, 因此在比较不同数据集的后验时不能随意略去.

其中所有这些量都被看作关于 $\mathbf{w}$ 的函数.(2.114)中的分母是归一化常数, 其确保了(2.114)左侧的后验分布是一个有效的概率密度并且积分为 1. 实际上, 通过对(2.114)的两边关于 $\mathbf{w}$ 进行积分, 就可以用先验分布和似然函数表示 Bayes 定理中的分母:

$$p(\mathcal{D})=\int p(\mathcal{D}\mid\mathbf{w})\,p(\mathbf{w})\,\mathrm{d}\mathbf{w} \tag{2.116}$$

命题 2.33 (证据的边缘化表示)

设 $\mathbf{w}$ 为连续参数, 则 Bayes 定理中的分母 (证据) 由(2.116)给出.

证明

对(2.114)两边关于 $\mathbf{w}$ 积分. 左端为 $\int p(\mathbf{w}\mid\mathcal{D})\,\mathrm{d}\mathbf{w}=1$(后验是密度), 右端的分母可提出积分号外:

$$1=\frac{1}{p(\mathcal{D})}\int p(\mathcal{D}\mid\mathbf{w})\,p(\mathbf{w})\,\mathrm{d}\mathbf{w},$$

两边乘以 $p(\mathcal{D})$ 即得(2.116).

□

注

(重构新增) (2.116)说明证据就是“在所有可能的参数值上对似然加权平均”, 权重是先验. 它因此可以解释为模型对数据的**边缘似然**: 模型越能预测数据, 证据越大. 这一解释是小节 2.6.3中“Bayes 方法自动惩罚复杂度”的根据.

积分的可行性依赖参数维数:(2.116)是一个 M 维积分 (M 为 $\mathbf{w}$ 的维数), 一般没有解析形式, 这正是小节 2.6.3所说的计算困难.

在 Bayes 和频率学派范式中, 似然函数 $p(\mathcal{D} \mid \mathbf{w})$ 发挥着核心作用. 然而, 这两种学派使用它的方式有根本的不同. 在频率学派的设定中,$\mathbf{w}$ 是一个固定的参数, 其值由某种形式的“估计器”确定, 并且这个估计的误差界限(至少在概念上) 是通过考虑可能的训练数据集 $\mathcal{D}$ 的分布来确定的. 相反, 从 Bayes 视角来看, 只有一个训练数据集 $\mathcal{D}$(即实际观测到的那个数据集), 并且参数中的不确定性可通过 $\mathbf{w}$ 上的概率分布来表示.

2.6.2 正则化

我们可以从 Bayes 视角来深入理解用于减少过拟合的正则化技术 (该技术在正弦曲线回归示例中使用过)(参见小节 1.2.5). 我们不再通过最大化关于 $\mathbf{w}$ 的似然函数来选择模型参数, 而是最大化后验概率 [见(2.114)] . 这种技术称为最大后验 (maximum a posteriori) 估计, 简称 MAP 估计. 同样, 也可以最小化后验概率的负对数. 对(2.114)的两边取负对数, 则有

$$-\ln p(\mathbf{w} \mid \mathcal{D}) = -\ln p(\mathcal{D} \mid \mathbf{w}) - \ln p(\mathbf{w}) + \ln p(\mathcal{D}) \tag{2.117}$$

定义 2.39 (最大后验估计)

称使后验概率取最大值的参数值

$$\mathbf{w}_{\mathrm{MAP}} = \arg\max_{\mathbf{w}} p(\mathbf{w} \mid \mathcal{D})$$

为参数 $\mathbf{w}$ 的**最大后验估计** (maximum a posteriori estimate), 简称 **MAP** 估计; 等价地, 它是使负对数后验 $-\ln p(\mathbf{w} \mid \mathcal{D})$ 取最小值的点. ♣

注

(重构新增) MAP 估计介于最大似然与完全 Bayes 之间: 它像 Bayes 方法一样引入先验, 但最终仍只给出一个点, 而不给出整个后验分布. 因此它没有解决“参数不确定性”的问题, 只把先验当作一种偏好写进了目标函数.

定义中的“等价地”依赖 $-\ln$ 严格单调; 但要注意 MAP 估计不是后验均值, 二者在后验不对称时不同, 而且 MAP 会随参数化方式改变 (见命题 2.22).

(2.117)右侧的第一项是常见的对数似然; 第三项因为不依赖于 $\mathbf{w}$, 所以可以省略; 第二项是一个 $\mathbf{w}$ 的函数, 它在这里被加进对数似然中, 可以当作一种形式的正则化. 为了使这一点更明确, 我们假设 $\mathbf{w}$ 内的每个元素独立且服从均值为 0、方差为 s^2 的 Gauss 分布, 则先验分布 $p(\mathbf{w})$ 可表示为一系列分布的乘积, 即

$$p(\mathbf{w} \mid s) = \prod_{i=0}^{M} \mathcal{N}(w_i \mid 0, s^2) = \prod_{i=0}^{M} \frac{1}{\sqrt{2\pi s^2}} \exp\left(-\frac{w_i^2}{2s^2}\right) \tag{2.118}$$

定义 2.40 (各向同性 Gauss 先验)

若参数向量的每个分量 w_i 相互独立且都服从 $\mathcal{N}(0, s^2)$, 则称 $\mathbf{w}$ 的先验分布为**各向同性 Gauss 先验**, 它由(2.118)给出. ♣

注

(重构新增) 选择 Gauss 先验的动机有两条: 一是它把“参数应当接近零”这一偏好表达为对 $|w_i|$ 的二次惩罚, 二是它在数学上使后验可以解析处理 (Gauss 先验与 Gauss 似然共轭). 这两点正是(2.117)第二项能化成 w_i^2 之和的原因.

条件“各分量独立、均值 0、方差相同”都是可放松的建模选择: 若均值非零, 二次项的中心随之平移; 若各分量方差不同, 则得到加权 (广义) 权重衰减; 若改用 Laplace 先验, 对应的惩罚是 $|w_i|$ 而非 w_i^2.$s^2 > 0$ 不可省, 否则先验退化为 δ 函数.

代入(2.117), 可得

$$-\ln p(\mathbf{w} \mid \mathcal{D}) = -\ln p(\mathcal{D} \mid \mathbf{w}) + \frac{1}{2s^2}\sum_{i=0}^{M} w_i^2 + \text{const} \tag{2.119}$$

命题 2.34 (负对数后验的分解)

在(2.118)的先验下,(2.119)成立, 即负对数后验等于负对数似然加上参数的二次惩罚, 再加一个与 $\mathbf{w}$ 无关的常数.

证明

把(2.118)代入 $-\ln p(\mathbf{w})$:

$$-\ln p(\mathbf{w} \mid s) = -\sum_{i=0}^{M} \ln \mathcal{N}(w_i \mid 0, s^2) = \sum_{i=0}^{M}\left(\frac{1}{2}\ln(2\pi s^2) + \frac{w_i^2}{2s^2}\right) = \frac{1}{2s^2}\sum_{i=0}^{M} w_i^2 + \text{const},$$

其中常数项 $\dfrac{M+1}{2}\ln(2\pi s^2)$ 与 $\mathbf{w}$ 无关. 把它代入(2.117), 并注意 $\ln p(\mathcal{D})$ 也与 $\mathbf{w}$ 无关, 即得(2.119).

□

注

(重构新增) 这条分解是“正则化 = 先验”的定量表述:$-\ln p(\mathbf{w})$ 就是正则项, 其系数 $1/(2s^2)$ 由先验的方差决定——先验越窄 (s^2 越小), 惩罚越重, 解越接近零. 反之,$s^2 \to \infty$ 时惩罚消失,MAP 退化为最大似然.

“const”虽然不影响最优点, 但它影响后验的归一化; 在比较不同模型的证据时不能随意丢弃. 此外, 该分解只对 Gauss 先验成立, 换用其他先验时正则项的形式也随之改变.

如果考虑线性回归模型的特定情况, 其对数似然由(2.66)给出, 则可以看到最大化后验分布等价于最小化如下函数 (见练习 2.41):

$$E(\mathbf{w}) = \frac{1}{2\sigma^2}\sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{1}{2s^2}\mathbf{w}^{\mathsf{T}}\mathbf{w} \tag{2.120}$$

命题 2.35 (MAP 估计等价于正则化平方和)

设似然由(2.66)给出、先验由(2.118)给出, 则关于 $\mathbf{w}$ 最大化后验等价于关于 $\mathbf{w}$ 最小化(2.120).

证明

把(2.66)与(2.119)合并, 并乘以正的常数 $2\sigma^2$(不改变最小值点):

$$-2\sigma^2 \ln p(\mathbf{w} \mid \mathcal{D}) = \sum_{n=1}^{N}\{y(x_n, \mathbf{w}) - t_n\}^2 + \frac{\sigma^2}{s^2}\mathbf{w}^{\mathsf{T}}\mathbf{w} + \text{const}.$$

这里用到 $\sum_{i=0}^{M} w_i^2 = \mathbf{w}^{\mathsf{T}}\mathbf{w}$. 上式右端与(2.120)只相差正的常数因子 $2\sigma^2$, 故最小化点相同.

□

注

(重构新增) 这条结果把节 1.2中“为了抑制过拟合而人为添加”的二次正则项解释成了先验的产物, 其中的正则化系数 $\lambda = \sigma^2/s^2$ 不再是需要凭经验调节的超参数, 而由噪声方差与先验方差之比确定.

条件“Gauss 似然 + Gauss 先验”同时不可省: 若噪声不是 Gauss, 第一项就不是平方和; 若先验不是 Gauss, 第二项就不是二次型.(2.120)与(1.4)形式一致, 差别只在于系数是否被显式写出.

这正是我们之前遇到的正则化的平方和误差函数的形式 [见(1.4)] .

2.6.3 Bayes 机器学习

我们从 Bayes 视角说明了使用正则化的动机, 并推导出了正则化项的具体形式. 然而, 单独使用 Bayes 定理并不构成真正的 Bayes 机器学习方法, 因为它仍然是为 $\mathbf{w}$ 寻找单一解而没有考虑 $\mathbf{w}$ 值的不确定性. 假设我们有训练数据集 $\mathcal{D}$, 目标是预测给定新的输入值 x 时的目标变量 t. 此刻我们就会对给定 x 和 $\mathcal{D}$ 时 t 的分布感兴趣. 根据概率的加和法则和乘积法则, 有

$$p(t \mid x, \mathcal{D}) = \int p(t \mid x, \mathbf{w})\, p(\mathbf{w} \mid \mathcal{D})\, \mathrm{d}\mathbf{w} \tag{2.121}$$

命题 2.36 (Bayes 预测分布)

设 $p(t \mid x, \mathbf{w})$ 为给定参数的预测分布,$p(\mathbf{w} \mid \mathcal{D})$ 为参数后验, 则给定 x 与 $\mathcal{D}$ 时 t 的预测分布由(2.121)给出.

证明

由乘积法则,$p(t, \mathbf{w} \mid x, \mathcal{D}) = p(t \mid x, \mathbf{w})\, p(\mathbf{w} \mid \mathcal{D})$(给定 $\mathbf{w}$ 后 t 与 $\mathcal{D}$ 条件独立). 再对 $\mathbf{w}$ 施用加和法则 (连续情形为积分) 即把 $\mathbf{w}$ 边缘化:

$$p(t \mid x, \mathcal{D}) = \int p(t, \mathbf{w} \mid x, \mathcal{D})\, \mathrm{d}\mathbf{w} = \int p(t \mid x, \mathbf{w})\, p(\mathbf{w} \mid \mathcal{D})\, \mathrm{d}\mathbf{w}.$$

□

注

(重构新增) (2.121)与(2.69)的差别在于: 后者把 $\mathbf{w}$ 固定为 $\mathbf{w}_{\mathrm{ML}}$, 前者则对全部 $\mathbf{w}$ 按后验加权平均. 因此(2.121)同时包含了噪声带来的偶然不确定性与参数不确定性带来的认知不确定性.

条件“给定 $\mathbf{w}$ 后 t 与 $\mathcal{D}$ 条件独立”不可省, 否则被积函数不能写成两项之积. 这个积分一般没有解析解, 维数等于参数个数, 正是下段所说的计算困难所在.

可以看到, 通过对 $p(t \mid x, \mathbf{w})$ 按照所有可能的 $\mathbf{w}$ 值进行加权平均可以获得预测, 其中加权函数由后验概率分布 $p(\mathbf{w} \mid \mathcal{D})$ 给出. 相比其他方法,Bayes 方法的重要不同之处在于对参数空间做了积分. 而传统的频率学派使用的方法是通过优化损失函数 (如正则化平方和) 来获得参数的点估计.

这种面向机器学习的完全 Bayes 处理方法给我们提供了一些强大的见解. 例如, 我们在多项式回归中遇到的过拟合问题 (参见节 1.2), 就是使用最大似然导致的, 而在使用 Bayes 方法对参数进行边缘化时不会出现该问题. 同样, 我们可能有多个潜在的模型用于解决给定问题, 比如回归示例中不同阶数的多项式. 最大似然方法简单地选择能够给出最高数据概率的模型, 但这倾向于选择越来越复杂的模型, 从而导致过拟合. 完全 Bayes 方法涉及对所有可能的模型进行平均, 每个模型的贡献由其后验概率加权 (参见节 9.6). 另外, 通常, 中等复杂度的模型这种概率最高; 非常简单的模型 (如低阶多项式) 概率较低, 因为它们无法很好地拟合数据; 而非常复杂的模型 (如阶数非常高的多项式) 概率也较低, 因为 Bayes 方法对参数的积分自动且优雅地惩罚了复杂度. 关于应用于机器学习的 Bayes 方法的全面概述, 包括神经网络, 参见 Bishop(2006).

遗憾的是,Bayes 框架有一个主要的缺点——涉及对参数空间的积分, 这在(2.121)中显而易见. 现代深度学习模型可以有数百万甚至数十亿的参数, 即使是对这种积分的简单近似通常也不可行. 实际上, 若给定有限的计算预算和大量的训练数据, 通常最好使用最大似然技术, 一般还需要增加一种或多种形式的正则化到一个大型神经网络中, 而不是对一个规模小得多的模型应用 Bayes 方法.

2.7 习题

✍ **练习 2.1** (★) 在癌症检查的示例中, 我们使用了癌症的先验概率 $p(C = 1) = 0.01$. 实际上, 癌症的流行程度通常要低得多. 考虑 $p(C = 1) = 0.001$ 的情况, 并重新计算在检查结果为阳性的情况下某人患有癌症的概率 $p(C = 1 \mid T = 1)$. 结果可能会让许多人感到惊讶, 因为检查似乎具有很高的准确性, 但检查结果呈阳性情况下患有癌症的概率却很低.

✍ **练习 2.2** (★★) 确定的数满足传递性 (transitivity), 即如果 $x > y$ 且 $y > z$, 则可以推出 $x > z$. 然而, 随机数不满足传递性.图 2.16展示了 4 个立方体骰子, 我们规定了它们的循环顺序. 请证明这 4 个骰子中, 每一个掷出的数字比循环中前一个骰子掷出数字大的概率为 2/3. 这样的骰子称为非传递骰子 (non-transitive dice), 这里显示的具体例子称为 Efron 骰子 (Efron dice).

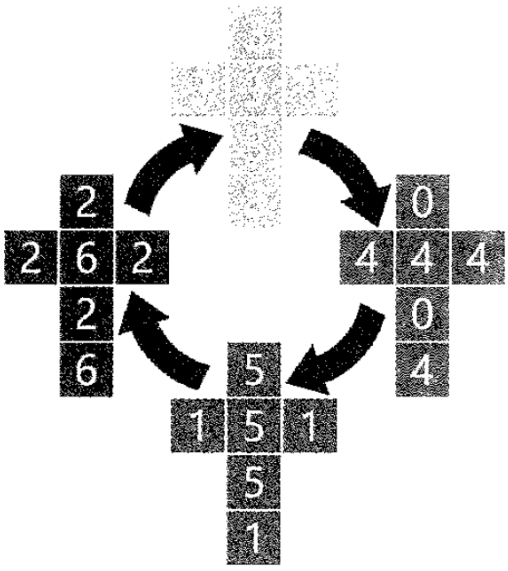


图 2.16: 非传递立方体骰子的一个示例, 其中每个骰子都已被 “扁平展开” 以展示每个面上的数字. 骰子按循环顺序排列, 使得每个骰子掷出的数字比循环中前一个骰子掷出数字大的概率为 2/3.

✍ **练习 2.3** (★) 考虑由两个独立随机变量之和给出的变量 $y = u + v$, 其中 $u \sim p_u(u)$ 且 $v \sim p_v(v)$. 证明 y 的分布 $p_y(y)$ 可由下式给出:

$$p(y) = \int p_u(u)\, p_v(y - u)\, \mathrm{d}u \tag{2.122}$$

这称为 $p_u(u)$ 和 $p_v(v)$ 的卷积 (convolution).

✍ **练习 2.4** (★) 验证均匀分布 [(2.33)] 被正确地归一化了, 并找出其均值和方差的表达式.

✍ **练习 2.5** (★★) 验证指数分布 [(2.34)] 和 Laplace 分布 [(2.35)] 被正确地归一化了.

✍ **练习 2.6** (★) 利用 Dirac δ 函数的性质, 验证经验密度 [(2.37)] 被正确地归一化了.

✍ **练习 2.7** (★) 通过使用经验密度 [(2.37)], 证明可以对从密度中抽取的有限样本集合进行求和 [(2.40)], 并近似由(2.39)给出的期望.

✍ **练习 2.8** (★) 使用方差定义 [(2.44)] 证明 $\mathrm{var}[f(x)]$ 满足(2.45).

✍ **练习 2.9** (★) 证明如果两个变量 x 和 y 相互独立, 则它们的协方差为零.

✍ **练习 2.10** (★) 假设两个变量 x 和 z 统计独立, 证明它们的和的均值和方差满足

$$\mathbb{E}[x + z] = \mathbb{E}[x] + \mathbb{E}[z] \tag{2.123}$$

$$\mathrm{var}[x + z] = \mathrm{var}[x] + \mathrm{var}[z] \tag{2.124}$$

✍ **练习 2.11** (★) 考虑具有联合分布 $p(x,y)$ 的两个变量 x 和 y. 证明以下两个结论:

$$\mathbb{E}[x] = \mathbb{E}_y\left[\mathbb{E}_x[x \mid y]\right] \tag{2.125}$$

$$\mathrm{var}[x] = \mathbb{E}_y\left[\mathrm{var}_x[x \mid y]\right] + \mathrm{var}_y\left[\mathbb{E}_x[x \mid y]\right] \tag{2.126}$$

其中 $\mathbb{E}_x[x \mid y]$ 表示条件分布 $p(x \mid y)$ 下 x 的期望, 条件方差的符号同理.

✍ **练习 2.12** (★) 证明单变量 Gauss 分布的归一化条件 [(2.51)] . 为此, 考虑积分

$$I = \int_{-\infty}^{\infty} \exp\left(-\frac{1}{2\sigma^2}x^2\right) \mathrm{d}x \tag{2.127}$$

首先将其平方写成以下形式:

$$I^2 = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} \exp\left(-\frac{1}{2\sigma^2}(x^2+y^2)\right) \mathrm{d}x\,\mathrm{d}y \tag{2.128}$$

然后从直角坐标 (x,y) 转换到极坐标 (r,θ), 并代入 $u = r^2$. 证明通过对 θ 和 u 进行积分, 然后对两边取平方根, 可以得到

$$I = \left(2\pi\sigma^2\right)^{1/2} \tag{2.129}$$

最后, 用这个结果来证明 Gauss 分布 $\mathcal{N}(x \mid \mu, \sigma^2)$ 是归一化的.

✍ **练习 2.13** (★★) 通过改变变量, 验证单变量 Gauss 分布 [(2.49)] 满足(2.52). 接下来, 对归一化条件

$$\int_{-\infty}^{\infty} \mathcal{N}(x \mid \mu, \sigma^2)\,\mathrm{d}x = 1 \tag{2.130}$$

关于 σ^2 微分, 验证 Gauss 分布满足(2.53). 最后, 证明(2.54)成立.

✍ **练习 2.14** (★) 证明 Gauss 分布 [(2.49)] 的众数 (即极大值) 由 μ 给出.

✍ **练习 2.15** (★) 通过将对数似然函数 [(2.56)] 关于 μ 和 μ^2 的导数置为零, 验证结果(2.57)和(2.58).

✍ **练习 2.16** (★★) 使用结论(2.52)和(2.53), 证明

$$\mathbb{E}\left[x_n x_m\right] = \mu^2 + I_{nm}\sigma^2 \tag{2.131}$$

其中 x_n 和 x_m 表示从均值为 μ 和方差为 σ^2 的 Gauss 分布中抽取的数据点,I_{nm} 满足如果 $n = m$,$I_{nm} = 1$, 否则 $I_{nm} = 0$, 从而证明结论(2.59)和(2.60).

✍ **练习 2.17** (★) 使用定义(2.61), 证明结果(2.62),(2.62)表明基于真实均值的 Gauss 分布的方差估计器的期望由真实方差 σ^2 给出.

✍ **练习 2.18** (★) 证明对 σ^2 最大化(2.66)可以得到结果(2.68).

✍ **练习 2.19** (★★) 使用改变量下概率密度的变换性质(2.71), 证明任何密度 $p(y)$ 都可以通过对处处非零的固定密度 $q(x)$ 进行非线性变量变换 $y = f(x)$ 获得, 其中 $f(x)$ 是单调函数, 即 $0 \leqslant f'(x) < \infty$. 写出 $f(x)$ 满足的微分方程, 并绘图说明密度变换.

✍ **练习 2.20** (★) 计算(2.79)和(2.80)所定义变换的 Jacobi 矩阵的元素.

✍ **练习 2.21** (★)节 2.5介绍了熵 $h(x)$ 的概念——作为观测具有分布 $p(x)$ 的随机变量 x 的值所获得的信息. 可以看到, 对于独立变量 x 和 y, 有 $p(x,y) = p(x)p(y)$. 熵函数是可加的, 即 $h(x,y) = h(x) + h(y)$. 在这个练习中, 推导 h 和 p 之间的关系, 形式为函数 $h(p)$. 首先证明 $h(p^2) = 2h(p)$, 并通过归纳法证明 $h(p^n) = nh(p)$, 其中 n 是正整数. 从而证明 $h(p^{n/m}) = (n/m)h(p)$, 其中 m 也是正整数. 这意味着 $h(p^x) = xh(p)$, 其中 x 是正有理数, 并且由于连续性, 当 x 是正实数时也成立. 最后, 证明这意味着 $h(p)$ 必须采用 $h(p) \propto \ln p$ 的形式.

✍ **练习 2.22** (★) 使用 Lagrange 乘子证明最大化离散变量的熵 [(2.90)] 会给出所有概率 $p(x_i)$ 相等的分布, 并且相应的熵为 $\ln M$.

✍ **练习 2.23** (★) 考虑一个包含 M 个状态的离散随机变量 x, 并使用(2.105)中 Jensen 不等式的形式证明其分布 $p(x)$ 的熵满足 $H[x] \leqslant \ln M$.

✍ **练习 2.24** (★★) 使用变分法证明函数(2.99)的驻点由(2.100)给出. 然后使用约束(2.96) ~ (2.98)来消除 Lagrange 乘子, 从而证明最大熵解由 Gauss 分布(2.101)给出.

✍ **练习 2.25** (★) 使用结果(2.97)和(2.98)证明单变量 Gauss 分布(2.101)的熵由(2.102)给出.

✍ **练习 2.26** (★★) 假设 $p(x)$ 是某个固定分布, 我们希望使用 Gauss 分布 $q(x) = \mathcal{N}(x \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 来近似它. 通过写出 Gauss 分布 $q(x)$ 的 Kullback-Leibler 散度 $\mathrm{KL}(p\|q)$ 的形式, 然后微分, 证明关于 $\boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 最小化 $\mathrm{KL}(p\|q)$ 的结果如下:$\boldsymbol{\mu}$ 由 $p(x)$ 下 x 的期望给出, 而 $\boldsymbol{\Sigma}$ 由协方差给出.

✍ **练习 2.27** (★★) 估计两个 Gauss 分布 $p(x) = \mathcal{N}(x \mid \mu, \sigma^2)$ 和 $q(x) = \mathcal{N}(x \mid m, s^2)$ 之间的 Kullback-Leibler 散度 [(2.103)] .

✍ **练习 2.28** (★★)α 家族 (alpha family) 的散度被定义为

$$D_\alpha(p\|q) = \frac{4}{1-\alpha^2}\left(1 - \int p(x)^{\frac{1+\alpha}{2}} q(x)^{\frac{1-\alpha}{2}} \,\mathrm{d}x\right) \tag{2.132}$$

其中 $\alpha(-\infty < \alpha < \infty)$ 是一个连续参数. 证明 Kullback-Leibler 散度 $\mathrm{KL}(p\|q)$ 对应于 $\alpha \to 1$. 这可以通过写出 $p^\varepsilon = \exp(\varepsilon \ln p) = 1 + \varepsilon \ln p + O(\varepsilon^2)$, 然后取 $\varepsilon \to 0$ 来完成. 类似地, 证明 $\mathrm{KL}(q\|p)$ 对应于 $\alpha \to -1$.

✍ **练习 2.29** (★★) 考虑具有联合分布 $p(x, y)$ 的两个变量 x 和 y. 证明这对变量的微分熵满足

$$H[x, y] \leqslant H[x] + H[y] \tag{2.133}$$

当且仅当 x 和 y 统计独立时等式成立.

✍ **练习 2.30** (★) 考虑具有分布 $p(\mathbf{x})$ 和相应熵 $H[\mathbf{x}]$ 的连续变量向量 $\mathbf{x}$. 假设进行非奇异线性变换, 得到新变量 $\mathbf{y} = \mathbf{A}\mathbf{x}$. 证明相应的熵由 $H[\mathbf{y}] = H[\mathbf{x}] + \ln\det(\mathbf{A})$ 给出, 其中 $\det(\mathbf{A})$ 表示 $\mathbf{A}$ 的行列式.

✍ **练习 2.31** (★★) 假设两个离散随机变量 x 和 y 之间的条件熵 $H[y \mid x]$ 为零. 证明对于所有 $p(x) > 0$ 的 x 值, 变量 y 必然是 x 的函数. 换句话说, 对于每个 x 值, 只有一个 y 值使得 $p(y \mid x) \neq 0$.

✍ **练习 2.32** (★) 严格凸函数被定义为每条弦都位于函数之上的函数. 证明这等价于函数的二阶导数为正的条件.

✍ **练习 2.33** (★★) 使用归纳法证明对于凸函数的不等式(2.104)蕴含了结果(2.105).

✍ **练习 2.34** (★) 证明在某个加性常数范围内, 经验分布(2.37)与模型分布 $q(x \mid \boldsymbol{\theta})$ 之间的 Kullback-Leibler 散度(2.103)等于负对数似然函数.

✍ **练习 2.35** (★) 使用定义(2.110)以及概率的乘积法则, 证明结果(2.111).

✍ **练习 2.36** (★★★) 考虑两个二元变量 x 和 y, 它们具有下列联合分布:

	$y = 0$	$y = 1$
$x = 0$	$1/3$	$1/3$
$x = 1$	0	$1/3$

计算以下量:(a) $H[x]$ (b) $H[y]$ (c) $H[y \mid x]$ (d) $H[x \mid y]$ (e) $H[x, y]$ (f) $I[x, y]$. 绘制一个 Venn 图来显示这些不同量之间的关系.

✍ **练习 2.37** (★) 通过应用 Jensen 不等式(2.105)与 $f(x) = \ln x$, 证明一组实数的算术平均值不小于它们的几何平均值.

✍ **练习 2.38** (★) 使用概率的加和法则与乘积法则, 证明互信息 $I(x, y)$ 满足关系式(2.113).

✍ **练习 2.39** (★★) 假设两个变量 z_1 和 z_2 独立, 可知 $p(z_1, z_2) = p(z_1)p(z_2)$. 证明这两个变量之间的协方差矩阵是对角矩阵. 这表明独立性是两个变量无关的充分条件. 考虑两个变量 y_1 和 y_2, 其中 y_1 围绕 0 对称分布,$y_2 = y_1^2$. 写出条件分布 $p(y_2 \mid y_1)$ 并观察到这与 y_1 相关, 从而证明这两个变量不是独立的. 证明这两个变量之间的协方差矩阵也是对角矩阵. 为此, 使用关系 $p(y_1, y_2) = p(y_1)p(y_2 \mid y_1)$ 证明非对角线项为零. 这个反例表明零相关性不是两个变量独立的充分条件.

✍ **练习 2.40** (★) 考虑图 2.2中的弯曲硬币. 假设凹面是正面 (头像面) 的先验概率为 0.1. 现在假设该硬币被抛掷 10 次, 得知有 8 次正面朝上,2 次反面朝上. 使用 Bayes 定理计算凹面是正面的后验概率, 并计算下一次抛掷后正面朝上的概率.

✍ **练习 2.41** (★) 通过将(2.118)代入(2.117)并利用线性回归模型的对数似然结果(2.66), 导出正则化误差函数的结果(2.120).

第 3 章 标准分布

在本章中, 我们将通过具体示例来讨论一些概率分布和它们的性质. 这些分布除了常规的用途之外, 还是构建更复杂模型的基石, 因此本书将广泛使用它们.

本章所讨论的分布有一个作用, 即在给定随机变量 x 的有限个观测值 $x_1, \cdots, x_N$ 的情况下, 对其分布 $p(x)$ 进行建模. 这就是密度估计 (density estimation) 问题. 需要强调的是, 密度估计问题基本上是不适定的, 因为可能存在无数个概率分布都能产生这样一组观测值. 事实上, 任意在 $x_1, \cdots, x_N$ 处非零的分布 $p(x)$ 都有可能是真实的概率分布. 选择合适分布的问题与模型选择有关, 而该问题我们已经在多项式曲线拟合这一机器学习核心问题中遇到过 (参见节 1.2).

在对连续变量的 Gauss 分布进行考察之前, 首先考虑离散变量的分布. 这些分布是参数分布的典型例子, 之所以如此命名, 是因为这些分布仅由很少的可调参数决定, 如 Gauss 分布中的均值和方差等. 为了将这些模型应用于密度估计问题, 我们需要一种在给定观测数据集的情况下确定合适参数值的方法 (主要聚焦于最大化似然函数).

在本章中, 我们假设观测数据是独立同分布的 (independent and identically distributed,i.i.d.). 在后续章节中, 我们还将探索包含结构化数据的更复杂情形, 到那时, 这一假设将不再成立.

参数化方法的一个局限在于, 它将分布假设为一种特定的函数形式, 而这一做法可能在特定的应用中并不合适. 另一种方法是非参数 (nonparametric) 密度估计方法, 此时分布的形式通常取决于数据集的大小. 这类模型仍然包含一些参数, 但这些参数控制的是模型的复杂度而不是分布的形式. 在本章的最后, 我们将简述 3 种典型的非参数化方法, 它们分别基于直方图、最近邻和核函数. 此类非参数化技术的一个主要局限在于它们涉及存储所有训练数据; 换言之, 参数的数量将随数据量的增大而增加, 因此这些方法在面对大型数据集时会变得非常低效. 神经网络拥有大量但数量固定的参数, 基于神经网络的灵活分布使深度学习同时具备参数模型的高效和非参数模型的泛化性.

3.1 离散变量

首先考虑离散变量的简单分布, 以二元变量为起点, 随后扩展到多状态变量.

3.1.1 Bernoulli 分布

考虑一个二元随机变量 $x \in \{0, 1\}$. 例如,x 可能描述的是抛掷硬币的结果, 其中 $x = 1$ 表示 “正面”、$x = 0$ 表示 “反面” . 如果这枚硬币有所损坏, 则落地时正面朝上的概率和反面朝上的概率未必相等.

定义 3.1 (Bernoulli 分布)

设 x 为只取 0 与 1 两个值的随机变量. 若 $x = 1$ 的概率由参数 μ 给出, 即

$$p(x = 1 \mid \mu) = \mu \tag{3.1}$$

其中 $0 \leqslant \mu \leqslant 1$, 则 $p(x = 0 \mid \mu) = 1 - \mu$, 于是 x 的概率分布可以写成如下形式:

$$\mathrm{Bern}(x \mid \mu) = \mu^x (1 - \mu)^{1-x}. \tag{3.2}$$

上式称为参数为 μ 的 Bernoulli 分布, 记作 $\mathrm{Bern}(x \mid \mu)$.

♣

注

(重构新增) 引入 Bernoulli 分布的目的是为最简单的二值观测建立参数化模型: 只要指定一个标量 μ, 就完全确定了分布, 而 μ 可以从数据中估计. 定义式(3.2)采用“指数指示”的写法, 把 $x=1$ 与 $x=0$ 两种情形压缩进同一个表达式: 当 $x=1$ 时因子 $(1-\mu)^{1-x}$ 的指数为 0, 只留下 μ; 当 $x=0$ 时 μ^x 的指数为 0, 只留下 $1-\mu$. 这种写法便于求似然、取对数并把分布改写成指数族形式 (见节 3.4).

条件 $0\leqslant\mu\leqslant 1$ 是分布良定义的必要条件: 若 $\mu<0$ 或 $\mu>1$, 则 μ 与 $1-\mu$ 中至少有一个为负, $\mu^x(1-\mu)^{1-x}$ 将取负值或非实数, 无法作为概率. 端点 $\mu=0$ 与 $\mu=1$ 对应退化分布 (概率质量全部集中在一个取值上), 此时 0^0 按组合惯例取为 1, 定义仍然自洽. 注意 μ 是刻画“正面”这一取值的概率, 一旦交换两个取值的标签, μ 就应替换为 $1-\mu$, 这也是练习 3.2讨论对称写法的原因.

命题 3.1 (Bernoulli 分布的归一化、均值与方差)

设 $x\sim\mathrm{Bern}(x\mid\mu)$, $0\leqslant\mu\leqslant 1$. 则

$$\sum_{x=0}^{1} p(x\mid\mu)=1, \tag{3.3}$$

并且 x 的均值与方差分别为

$$\mathbb{E}[x]=\mu, \tag{3.4}$$

$$\mathrm{var}[x]=\mu(1-\mu). \tag{3.5}$$

证明

(重构新增) 由(3.2), x 只取 0 与 1 两个值, 且 $p(x=1\mid\mu)=\mu$、$p(x=0\mid\mu)=1-\mu$, 故

$$\sum_{x=0}^{1} p(x\mid\mu)=p(0\mid\mu)+p(1\mid\mu)=(1-\mu)+\mu=1,$$

即(3.3)成立. 类似地,

$$\mathbb{E}[x]=\sum_{x=0}^{1} x\,p(x\mid\mu)=0\cdot(1-\mu)+1\cdot\mu=\mu,$$

即(3.4)成立. 对于方差, 先计算二阶矩:

$$\mathbb{E}[x^2]=\sum_{x=0}^{1} x^2 p(x\mid\mu)=0^2\cdot(1-\mu)+1^2\cdot\mu=\mu,$$

由于 $x^2=x$(x 只取 $0,1$), 于是

$$\mathrm{var}[x]=\mathbb{E}[x^2]-\mathbb{E}[x]^2=\mu-\mu^2=\mu(1-\mu),$$

即(3.5)成立.

□

注

(重构新增) 这三条结论刻画了 Bernoulli 分布的“全部”低阶信息: 归一化说明它是一个合法的概率分布, 均值为 μ 说明参数 μ 就是“$x=1$ 的概率”这一直观含义, 方差 $\mu(1-\mu)$ 说明当 $\mu=1/2$ 时不确定性最大、当 $\mu\to 0$ 或 $\mu\to 1$ 时不确定性趋于零.

条件 $0\leqslant\mu\leqslant 1$ 在证明中只用到了“$1-\mu\geqslant 0$”这一事实, 用以保证两项都是非负的概率; 对均值和方差的计算本身在形式上并不需要该条件, 但若不满足它,$\mathbb{E}[x]=\mu$ 就不再是“概率的加权平均”, 方差也可能为负, 失去意义. 方差公式中的 $\mu(1-\mu)$ 在 $\mu\in[0,1]$ 上非负且以 1/4 为最大值, 这一上界在后面讨论 Bernoulli 分布的熵 (练习 3.1) 时还会再次出现.

假设我们有一个包含 x 的观测值的数据集 $\mathcal{D}=\{x_1,\cdots,x_N\}$. 基于所有观测值都是独立地从 $p(x\mid\mu)$ 生成的, 我们可以建立以 μ 为参数的似然方程, 使得

$$p(\mathcal{D}\mid\mu)=\prod_{n=1}^{N}p(x_n\mid\mu)=\prod_{n=1}^{N}\mu^{x_n}(1-\mu)^{1-x_n}. \tag{3.6}$$

我们可以通过最大化似然方程, 或等价地最大化对数似然函数 (对数似然函数是单调函数) 来估计 μ 的值.Bernoulli 分布的对数似然函数形式如下:

$$\ln p(\mathcal{D}\mid\mu)=\sum_{n=1}^{N}\ln p(x_n\mid\mu)=\sum_{n=1}^{N}\big\{x_n\ln\mu+(1-x_n)\ln(1-\mu)\big\}. \tag{3.7}$$

注意对数似然函数仅依赖 x_n 的 N 个观测值的和 $\sum_{n=1}^{N}x_n$. 该值给出了数据在这一分布下的充分统计量 (sufficient statistic)(参见节 3.4).

命题 3.2 (Bernoulli 分布的最大似然估计)

设 $\mathcal{D}=\{x_1,\cdots,x_N\}$ 是来自 $\mathrm{Bern}(x\mid\mu)$ 的独立同分布观测, 其中 $0<\mu<1$. 则对数似然(3.7)的唯一驻点, 即 μ 的最大似然估计, 为

$$\mu_{\mathrm{ML}}=\frac{1}{N}\sum_{n=1}^{N}x_n. \tag{3.8}$$

若记观测到 $x=1$(正面) 的次数为 m, 则(3.8)又可写作

$$\mu_{\mathrm{ML}}=\frac{m}{N}. \tag{3.9}$$

♠

证明

(重构新增) 记 $\ell(\mu)=\ln p(\mathcal{D}\mid\mu)$. 由(3.7),

$$\ell(\mu)=\Big(\sum_{n=1}^{N}x_n\Big)\ln\mu+\Big(N-\sum_{n=1}^{N}x_n\Big)\ln(1-\mu),$$

其中用到了 $\sum_n(1-x_n)=N-\sum_n x_n$. 对 μ 求导得

$$\frac{\mathrm{d}\ell}{\mathrm{d}\mu}=\frac{1}{\mu}\sum_{n=1}^{N}x_n-\frac{1}{1-\mu}\Big(N-\sum_{n=1}^{N}x_n\Big).$$

令导数为零, 两边乘以 $\mu(1-\mu)$(在 $0<\mu<1$ 时为正), 得

$$(1-\mu)\sum_{n=1}^{N}x_n=\mu\Big(N-\sum_{n=1}^{N}x_n\Big),$$

即 $\sum_n x_n=\mu N$, 故驻点唯一, 且 $\mu_{\mathrm{ML}}=\dfrac{1}{N}\sum_n x_n$, 这就是(3.8).

再验证它给出最大值. 二阶导数为

$$\frac{\mathrm{d}^2\ell}{\mathrm{d}\mu^2}=-\frac{1}{\mu^2}\sum_{n=1}^{N}x_n-\frac{1}{(1-\mu)^2}\Big(N-\sum_{n=1}^{N}x_n\Big)<0,$$

因为两项都非正, 且当 $0<\sum_n x_n<N$ 时两项都严格为负. 故 ℓ 在 $(0,1)$ 上严格凹, 唯一驻点即为全局最大值点.

最后, 若 $x=1$ 出现 m 次, 则 $\sum_n x_n=m$, 于是 $\mu_{\mathrm{ML}}=m/N$, 即(3.9).

□

注

(重构新增) 最大似然估计的推导说明了“对数把乘积变成求和”这一技巧为何重要: 取对数后数据只通过 $\sum_n x_n$ 进入似然, 这就是充分统计量的雏形 (节 3.4将给出一般定义).

证明中要求 $0<\mu<1$, 原因是 $\ln\mu$ 与 $\ln(1-\mu)$ 只在开区间上有定义; 端点情形应直接处理: 若全部观测为 $x_n=1$, 则似然 μ^N 在 $\mu=1$ 处取最大值 $\mu_{\mathrm{ML}}=1$; 若全部观测为 $x_n=0$, 则 $\mu_{\mathrm{ML}}=0$. 这两种退化情形下 $\mu_{\mathrm{ML}}=m/N$ 仍然成立, 只是不再由求导得到. 此外, 若 $m=0$ 或 $m=N$, 二阶导数在驻点处退化, 但由 0^0 的惯例直接代入即可验证似然在该点的取值最大.

这样硬币落地时正面朝上的概率就可以在上述最大化似然的框架下, 通过在数据集中计算正面朝上次数的比例得到.

定义 3.2 (样本均值)

设 $x_1,\cdots,x_N$ 是一组观测值, 则称

$$\bar{x}\triangleq\frac{1}{N}\sum_{n=1}^{N}x_n \tag{3.10}$$

为这组观测值的样本均值 (sample mean).

注

(重构新增) 样本均值是(3.8)的一般形式, 它并不依赖 Bernoulli 假设. 引入这一术语是为了在后面反复使用: 多元 Gauss 分布的均值最大似然解(3.109)正是向量形式的样本均值, 序贯估计(3.114)则给出了它的递推算法. 注意样本均值只对“可以相加”的量有意义, 因此它适用于实值观测, 而不直接适用于周期变量——这正是节 3.3要另行定义“圆周均值”的原因.

3.1.2 二项分布

我们同样可以给出在包含 N 个观测值的数据集中, 有 m 个观测值为 $x=1$ 的二元变量 x 的分布. 这一分布称为二项 (binomial) 分布, 由eq. (3.6)可以看出, 它与 $\mu^m(1-\mu)^{N-m}$ 成比例. 为得到归一化系数, 注意在抛掷硬币 N 次的过程中, 我们需要将所有可能掷出 m 次正面朝上的情况加总起来, 因此二项分布可写为如下形式:

$$\mathrm{Bin}(m \mid N,\mu) = \binom{N}{m}\mu^m(1-\mu)^{N-m}, \tag{3.11}$$

其中

$$\binom{N}{m} \equiv \frac{N!}{(N-m)!\,m!} \tag{3.12}$$

是从 N 个相同物品中无放回地选择 m 个物品的方法总数.

定义 3.3 (二项分布)

设进行了 N 次相互独立的 Bernoulli 试验, 每次“成功”的概率为 μ, 并记 m 为 N 次试验中成功的次数. 则 m 的分布为

$$\mathrm{Bin}(m \mid N,\mu) = \binom{N}{m}\mu^m(1-\mu)^{N-m}, \qquad m=0,1,\cdots,N,$$

称为参数为 N 与 μ 的二项分布, 其中组合数 $\binom{N}{m}$ 由(3.12)给出.

♣

注

(重构新增) 二项分布回答的是“计数”的问题: 把 N 次独立的 Bernoulli 观测相加, 就得到成功次数 m. 这正是eq. (3.6)中“数据只通过 $\sum_n x_n$ 进入似然”的另一面——当数据是 N 次试验时, $\sum_n x_n$ 恰好就是 m, 其分布自然由二项分布描述.

条件有三项: N 为固定正整数、各次试验相互独立、每次试验的成功概率同为 μ. N 固定保证了 m 的取值范围 $0 \leqslant m \leqslant N$ 有限; 独立性与同分布性是 $\mu^m(1-\mu)^{N-m}$ 这一乘积形式成立的前提, 若各次试验的概率不同, 则分布不再是二项分布 (而是一般的 Poisson 二项分布). 组合数因子 $\binom{N}{m}$ 处理的是“顺序”: m 次成功可以出现在 N 个位置中的任意 m 个位置上, 且这些情形互斥, 故概率相加.

图 3.1展示了一个 $N=10$、$\mu=0.25$ 的二项分布 (见练习 3.3).

对于独立的事件, 和的均值就是均值的和, 而和的方差就是方差的和. 用这些结果就能找到二项分布的均值和方差 (见命题 3.3的证明). 由于 $m = x_1 + \cdots + x_N$ 且任意观测的均值和方差均可由eq. (3.4)和eq. (3.5)给出, 故有

$$\mathbb{E}[m] \equiv \sum_{m=0}^{N} m\,\mathrm{Bin}(m \mid N,\mu) = N\mu, \tag{3.13}$$

$$\mathrm{var}[m] \equiv \sum_{m=0}^{N} (m-\mathbb{E}[m])^2\,\mathrm{Bin}(m \mid N,\mu) = N\mu(1-\mu). \tag{3.14}$$

命题 3.3 (二项分布的均值与方差)

设 $m \sim \mathrm{Bin}(m \mid N, \mu), 0 \leqslant \mu \leqslant 1$. 则

$$\mathbb{E}[m] = N\mu, \qquad \mathrm{var}[m] = N\mu(1-\mu).$$

♠

证明

(重构新增) 设 $x_1, \cdots, x_N$ 是 N 次独立 Bernoulli 试验的指示变量, 即 $x_n = 1$ 表示第 n 次成功,$x_n = 0$ 表示失败, 则 $m = x_1 + \cdots + x_N$, 且每个 x_n 都服从参数为 μ 的 Bernoulli 分布.

由期望的线性性 (它不要求独立性),

$$\mathbb{E}[m] = \sum_{n=1}^{N} \mathbb{E}[x_n] = \sum_{n=1}^{N} \mu = N\mu,$$

其中用到(3.4).

对于方差, 由独立性可知协方差为零, 即 $\mathrm{cov}[x_i, x_j] = 0 (i \neq j)$, 于是

$$\mathrm{var}[m] = \sum_{n=1}^{N} \mathrm{var}[x_n] + 2\sum_{i<j} \mathrm{cov}[x_i, x_j] = \sum_{n=1}^{N} \mu(1-\mu) = N\mu(1-\mu),$$

其中用到(3.5).

□

注

(重构新增) 这一证明的关键是“把计数拆成指示变量之和”: 一旦 $m = x_1 + \cdots + x_N$, 期望与方差的计算就归结为单次试验的结论. 期望的线性性无需独立性, 而方差的相加必须依赖独立性 (严格地说是两两不相关); 若各次试验正相关, 则和的方差会大于 $N\mu(1-\mu)$.

$N\mu$ 与 $N\mu(1-\mu)$ 也解释了二项分布的直观形状: 当 N 增大时, 均值线性增长而标准差按 $\sqrt{N}$ 增长, 因此相对波动 $\sqrt{\mu(1-\mu)/N}$ 越来越小, 分布相对其均值越来越集中——这正是图 3.1以及中心极限定理所描述的现象. 这些结论也可以用微积分直接对(3.11)求和得到 (见练习 3.4).

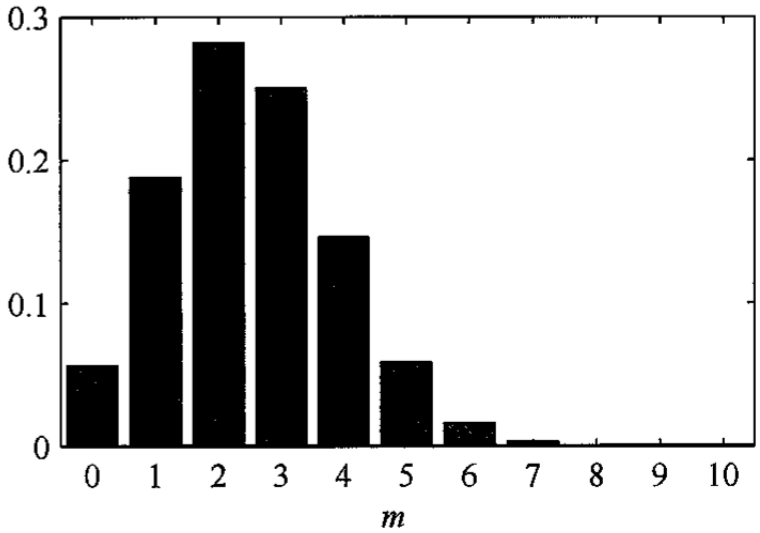


图 3.1: 当 $N = 10$ 且 $\mu = 0.25$ 时, 二项分布(3.11)作为 m 的函数的直方图.

3.1.3 多项分布

二元变量可以用于描述只有两种可能取值的量. 然而通常情况下, 我们遇到的离散变量可能包含 K 个相互无关的状态. 尽管存在多种表示这些变量的方法, 但我们很快将会看到一种非常便捷的 “K 中之一” (1-of-K) 的表示方案, 有时又称 “独热编码” .

定义 3.4 (1-of-K 表示与独热编码)

设离散变量有 K 个互不相同的状态. 称 K 维向量 $\boldsymbol{x} = (x_1, \cdots, x_K)^\mathsf{T}$ 为该变量的 1-of-K 表示或独热编码 (one-hot encoding), 如果 $x_1, \cdots, x_K$ 中有且仅有一个取值为 1, 其余全部取值为 0, 即 $\sum_{k=1}^{K} x_k = 1$ 且 $x_k \in \{0, 1\}$.

♣

注

(重构新增) 引入 1-of-K 表示是为了把“取哪一个状态”这种类别信息写成向量, 从而使概率、求和、线性代数运算都能直接施加于它. 例如, 若变量存在 $K = 6$ 种状态, 且某次观测刚好对应状态 $x_3 = 1$, 则

$$\boldsymbol{x} = (0, 0, 1, 0, 0, 0)^\mathsf{T}. \tag{3.15}$$

这样一来, “第 k 个状态”在数值上就对应“$\boldsymbol{x}$ 的第 k 个分量为 1”, 而 $\sum_k x_k = 1$ 恰好保证了每次观测只落在一个状态上. 代价是维度随状态数线性增长:K 个状态用 K 维向量表示, 其中 $K - 1$ 个自由度是冗余的 (节 3.4会利用这一点把参数减到 $K - 1$ 个).

如果我们将 $x_k = 1$ 的概率记为 μ_k, 则 $\boldsymbol{x}$ 的分布可以由下式给出:

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{x_k}, \tag{3.16}$$

其中 $\boldsymbol{\mu} = (\mu_1, \cdots, \mu_K)^\mathsf{T}$, 且因为参数 μ_k 表示的是概率, 它们满足约束 $\mu_k \geqslant 0$ 和 $\sum_{k=1}^{K} \mu_k = 1$.

定义 3.5 (类别分布)

设 $\boldsymbol{x}$ 为具有 K 个状态的离散变量的 1-of-K 表示, 参数向量 $\boldsymbol{\mu} = (\mu_1, \cdots, \mu_K)^\mathsf{T}$ 满足 $\mu_k \geqslant 0$ 与 $\sum_{k=1}^{K} \mu_k = 1$, 则称

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{x_k}$$

为参数为 $\boldsymbol{\mu}$ 的类别分布 (categorical distribution).

♣

注

(重构新增) 类别分布是 Bernoulli 分布在 $K > 2$ 时的直接推广: 把(3.2)中的 $\mu^x(1-\mu)^{1-x}$ 换成 $\prod_k \mu_k^{x_k}$, 就把“一次试验的两个结果”扩展为“一次试验的 K 个结果”. 由于 $\boldsymbol{x}$ 是独热向量, 乘积中只有第 k 个因子 (对应 $x_k = 1$) 存活下来, 其值为 μ_k, 其余因子都是 $\mu_j^0 = 1$.

参数空间的约束 $\mu_k \geqslant 0$ 与 $\sum_k \mu_k = 1$ 缺一不可: 前者保证非负性, 后者保证总概率为 1. 注意这 K 个参数只有 $K - 1$ 个自由度, 这一约束在作最大似然估计时必须用 Lagrange 乘子处理 (见命题 3.5), 也是后面把它写成指数族时只保留 $M - 1$ 个自然参数的原因.

eq. (3.16)所示的分布可以看作将 Bernoulli 分布扩展到可以描述超过两种状态. 可以看出, 这一分布是归一化的:

$$\sum_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \sum_{k=1}^{K} \mu_k = 1, \tag{3.17}$$

且

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{\mu}] = \sum_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{\mu}) \boldsymbol{x} = \boldsymbol{\mu}. \tag{3.18}$$

命题 3.4 (类别分布的归一化与均值)

设 $p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{x_k}$ 为类别分布,$\mu_k \geqslant 0, \sum_{k=1}^{K} \mu_k = 1$. 则

$$\sum_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{\mu}) = 1, \qquad \mathbb{E}[\boldsymbol{x} \mid \boldsymbol{\mu}] = \boldsymbol{\mu}.$$

♠

证明

(重构新增) 对 $\boldsymbol{x}$ 求和是在全部 K 个可能的独热向量上进行的. 由于每个独热向量恰好对应一个状态 k, 且在该向量处 $p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \mu_k$, 故

$$\sum_{\boldsymbol{x}} p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \sum_{k=1}^{K} \mu_k = 1,$$

这就是(3.17).

对于均值, 由定义

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{\mu}] = \sum_{k=1}^{K} \mu_k \boldsymbol{e}_k,$$

其中 $\boldsymbol{e}_k$ 是第 k 个分量为 1、其余为 0 的独热向量. 上式第 j 个分量为 $\sum_k \mu_k \delta_{jk} = \mu_j$, 故整个向量等于 $\boldsymbol{\mu}$, 即(3.18).

□

注

(重构新增) 这两条结论说明类别分布的参数 $\boldsymbol{\mu}$ 有明确的概率解释: 它既是各状态的概率, 又是独热向量的期望. 归一化把 K 个参数约束在一个单纯形上, 这解释了为什么 $K = 2$ 时只剩一个自由参数 μ, 与 Bernoulli 分布一致.

证明中“对 $\boldsymbol{x}$ 求和等于对状态 k 求和”这一步依赖于 $\boldsymbol{x}$ 的取值集合恰好是 K 个独热向量, 即依赖于 1-of-K 表示; 若改用 $x \in \{1, \cdots, K\}$ 的标量表示, 同一结论也成立, 只是求和的书写方式不同.

考虑包括 N 个独立观测值 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$ 的数据集 $\mathcal{D}$. 相应的似然函数有如下形式:

$$p(\mathcal{D} \mid \boldsymbol{\mu}) = \prod_{n=1}^{N} \prod_{k=1}^{K} \mu_k^{x_{nk}} = \prod_{k=1}^{K} \mu_k^{(\sum_n x_{nk})} = \prod_{k=1}^{K} \mu_k^{m_k}, \tag{3.19}$$

从中可以看出, 包含 N 个数据点的似然函数其实可以只通过 K 个量来定义:

$$m_k = \sum_{n=1}^{N} x_{nk}, \tag{3.20}$$

它表示观测值 $x_k = 1$ 的数量, 称为分布的充分统计量 (sufficient statistic)(参见节 3.4). 注意变量 m_k 满足如下约束:

$$\sum_{k=1}^{K} m_k = N. \tag{3.21}$$

为找到 $\boldsymbol{\mu}$ 的最大似然解, 我们需要在(3.17)的约束下 (也就是让 μ_k 的和为 1), 关于 μ_k 最大化 $\ln p(\mathcal{D} \mid \boldsymbol{\mu})$(Lagrange 乘子法的推导参见附录). 这可以通过使用 Lagrange 乘子 λ 并最大化

$$\sum_{k=1}^{K} m_k \ln \mu_k + \lambda \Big(\sum_{k=1}^{K} \mu_k - 1 \Big) \tag{3.22}$$

来实现求解. 令(3.22)关于 μ_k 的导数为零, 可得

$$\mu_k = -m_k / \lambda. \tag{3.23}$$

将(3.23)代入约束 $\sum_k \mu_k = 1$ 来求 Lagrange 乘子 λ. 由此可得 μ_k 的最大似然解为

$$\mu_k^{\mathrm{ML}} = \frac{m_k}{N}, \tag{3.24}$$

它实际上是 N 个观测值中 $x_k = 1$ 的观测值所占的比例.

命题 3.5 (类别分布的最大似然估计)

设 $\mathcal{D}=\{\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N\}$ 是来自类别分布的独立同分布观测,$m_k=\sum_n x_{nk}$,$\sum_k m_k=N$, 且各 $m_k>0$. 则在对 $\mu_k\geqslant 0$、$\sum_k \mu_k=1$ 的约束下, 对数似然 $\ln p(\mathcal{D}\mid\boldsymbol{\mu})=\sum_k m_k\ln\mu_k$ 的最大值点唯一, 且由(3.24)给出.

证明

(重构新增) 由于 $\sum_k \mu_k=1$, 可把约束吸收进目标函数, 构造 Lagrange 函数

$$L(\boldsymbol{\mu},\lambda)=\sum_{k=1}^{K} m_k\ln\mu_k+\lambda\Big(\sum_{k=1}^{K}\mu_k-1\Big),$$

这就是(3.22). 对 μ_k 求偏导并令其为零:

$$\frac{\partial L}{\partial \mu_k}=\frac{m_k}{\mu_k}+\lambda=0 \quad\Longrightarrow\quad \mu_k=-\frac{m_k}{\lambda},$$

即(3.23). 代入约束 $\sum_k \mu_k=1$ 得 $-\frac{1}{\lambda}\sum_k m_k=1$, 由(3.21)有 $\sum_k m_k=N$, 故 $\lambda=-N$. 回代即得 $\mu_k^{\mathrm{ML}}=m_k/N$, 这就是(3.24).

再证唯一性与最大值性. 在单纯形 $\{\mu_k\geqslant 0,\ \sum_k\mu_k=1\}$ 的内部,Hessian 为对角矩阵 $\mathrm{diag}(-m_k/\mu_k^2)$, 由于各 $m_k>0$, 它负定, 故 L 关于 $\boldsymbol{\mu}$ 严格凹, 驻点必为唯一全局最大值点. 当某个 $m_k=0$ 时, 目标函数不含 μ_k, 最大值可在 $\mu_k=0$ 处取到, 此时 $\mu_k^{\mathrm{ML}}=0=m_k/N$ 仍成立.

□

注

(重构新增) 最大似然解 $\mu_k^{\mathrm{ML}}=m_k/N$ 就是“经验频率”, 与 Bernoulli 情形(3.9)完全一致, 这说明参数估计在这一族分布中是“数频数”的操作.

这里必须使用 Lagrange 乘子而不能直接令梯度为零, 原因是约束 $\sum_k\mu_k=1$ 使 K 个参数中只有 $K-1$ 个自由; 若忽略约束直接对各 μ_k 求导, 得到的方程组无解 (梯度条件会要求所有 $m_k=0$). 证明中用到 $m_k>0$ 来保证 $\ln\mu_k$ 可微且 Hessian 负定; 若某个 $m_k=0$, 最优点落在单纯形的边界上, 此时严格的 KKT 条件应写成互补松弛形式, 但结论不变. 此外, 这一估计在 N 较小时会把某些 μ_k 估成 0, 从而对其他未观测状态给出零概率, 这是后面讨论先验平滑 (Dirichlet 先验) 的动机.

已知参数向量 $\boldsymbol{\mu}$ 和观测样本总数 N, 可以考虑量 $m_1,\cdots,m_K$ 的联合条件概率分布. 由(3.19)可得, 这一联合条件概率分布有如下形式:

$$\mathrm{Mult}(m_1,m_2,\cdots,m_K\mid\boldsymbol{\mu},N)=\binom{N}{m_1m_2\cdots m_K}\prod_{k=1}^{K}\mu_k^{m_k},\tag{3.25}$$

这就是多项分布. 归一化系数是将 N 个对象分为 K 组的方法数, 其中每组分别有 $m_1,\cdots,m_K$ 个对象, 由下式给出:

$$\binom{N}{m_1m_2\cdots m_K}=\frac{N!}{m_1!\,m_2!\cdots m_K!}.\tag{3.26}$$

定义 3.6 (多项分布)

设进行了 N 次相互独立的类别试验, 每次试验落到第 k 个状态的概率为 $\mu_k(k=1,\cdots,K,\mu_k\geqslant 0,\sum_k\mu_k=1)$, 记 m_k 为 N 次试验中落到第 k 个状态的次数, 则 $(m_1,\cdots,m_K)$ 的联合分布为

$$\mathrm{Mult}(m_1,\cdots,m_K\mid\boldsymbol{\mu},N)=\frac{N!}{m_1!\cdots m_K!}\prod_{k=1}^{K}\mu_k^{m_k},$$

其中 $m_k \geqslant 0$ 且 $\sum_k m_k = N$. 该分布称为多项 (multinomial) 分布, 它是二项分布在 $K > 2$ 个状态时的推广.

♣

注

(重构新增) 多项分布与二项分布的关系是“计数向量”与“单个计数”的关系: 当 $K = 2$ 时, 记 $m_1 = m$、$m_2 = N - m$, 则(3.25)化为 $\frac{N!}{m!(N-m)!}\mu_1^m \mu_2^{N-m}$, 令 $\mu_1 = \mu$、$\mu_2 = 1 - \mu$ 即回到二项分布(3.11). 归一化系数(3.26)是多项式系数, 它把 N 个可分辨的试验结果按状态分组, 组内不计顺序.

条件与二项分布相同 (N 固定、独立、同分布), 只是“成功/失败”被替换为“K 个互斥状态”. $\sum_k m_k = N$ 保证了计数向量位于一个 $K-1$ 维的单纯形格点上, 因此多项分布的支撑集只有 $K-1$ 个自由度.

注意, 包含两个状态的量可以表示为二元变量并用二项分布(3.11)建模, 或表示为二分之一 (1-of-2) 变量并用分布(3.16)建模, 其中令 $K = 2$.

注

(重构新增) 最后这句话澄清了三个层次的关系:Bernoulli 分布描述“一次二值试验”, 二项分布描述“N 次二值试验的成功次数”, 而类别分布与多项分布分别是它们在 K 个状态上的推广. 三者的参数个数不同 (μ;N, μ;$\boldsymbol{\mu}$;$N, \boldsymbol{\mu}$), 但共享同一个“独立重复试验”的生成机制, 因此后续在指数族框架下 (节 3.4) 可以把它们统一处理.

3.2 多元 Gauss 分布

Gauss 分布又称正态分布, 已广泛用于建模连续变量的分布. 我们已经看到对于单个变量 x(参见节 2.3),Gauss 分布可以写为如下形式:

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\}, \tag{3.27}$$

其中 μ 是均值,σ^2 是方差. 对于一个 D 维向量 $\boldsymbol{x}$, 多元 Gauss 分布有如下形式:

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right\}, \tag{3.28}$$

其中 $\boldsymbol{\mu}$ 是 D 维均值向量,$\boldsymbol{\Sigma}$ 是 $D \times D$ 的协方差矩阵,$|\boldsymbol{\Sigma}|$ 表示 $\boldsymbol{\Sigma}$ 的行列式.

定义 3.7 (多元 Gauss 分布)

设 $\boldsymbol{\mu} \in \mathbb{R}^D$,$\boldsymbol{\Sigma}$ 为 $D \times D$ 的对称正定矩阵. 则称由(3.28)定义的密度为均值 $\boldsymbol{\mu}$、协方差 $\boldsymbol{\Sigma}$ 的多元 Gauss 分布, 记作 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$.

♣

注

(重构新增) 多元 Gauss 分布是把一元 Gauss 分布(3.27)中的“标量平方”换成“二次型”得到的:$(x-\mu)^2/\sigma^2$ 变为 $(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})$, 而 σ^2 则被 $\boldsymbol{\Sigma}$ 取代. 这一替换并非任意: 它使得等密度面成为椭球 (小节 3.2.1), 并使 $\boldsymbol{\Sigma}$ 恰为 $\boldsymbol{x}$ 的协方差矩阵 (小节 3.2.2).

定义要求 $\boldsymbol{\Sigma}$ 对称正定, 有两点理由. 其一是归一化: 归一化系数 $(2\pi)^{-D/2}|\boldsymbol{\Sigma}|^{-1/2}$ 要求 $|\boldsymbol{\Sigma}| > 0$; 若 $\boldsymbol{\Sigma}$ 奇异, 则逆矩阵不存在, 密度无法定义. 其二是唯一性: 若 $\boldsymbol{\Sigma}$ 不对称, 则只有它的对称部分 $(\boldsymbol{\Sigma} + \boldsymbol{\Sigma}^{\mathsf{T}})/2$ 会出现在二次型中, 非对称部分对密度毫无影响, 因此把它纳入参数只会造成不可辨识 (练习 3.11给出了严格论证).

在很多不同的场景中都可以看到 Gauss 分布, 因此可以从各种不同的角度来理解它的作用 (参见节 2.5). 例如, 我们已经看到, 对于单个实数变量, 能够使它的熵最大化的分布就是 Gauss 分布. 这一性质对于多元 Gauss 分布同样适用 (见练习 3.8).

当我们考虑多个随机变量之和的时候, 同样会用到 Gauss 分布. 中心极限定理 (central limit theorem) 告诉我们, 在某些温和的条件下, 一组随机变量的和 (它本身也是一个随机变量) 的分布会随着其中随机变量数量的增加而越来越接近于 Gauss 分布 (Walker, 1969). 我们可以这样解释这一现象: 有 N 个随机变量 $x_1, \cdots, x_N$, 每个随机变量都在 $[0, 1]$ 区间上服从均匀分布, 考虑均值 $(x_1 + \cdots + x_N)/N$ 的分布. 对于较大的 N, 如图 3.2所示, 这一分布趋向

于 Gauss 分布. 现实中, 随着 N 的增大, 分布会很快地收敛到 Gauss 分布. 这一结果揭示了eq. (3.11)中关于变量 m 的二项分布, 其中 m 是二元随机变量 x 的 N 个观测样本的和, 分布会在 $N \to \infty$ 时趋向于 Gauss 分布 (见图 3.1中 $N = 10$ 的情形).

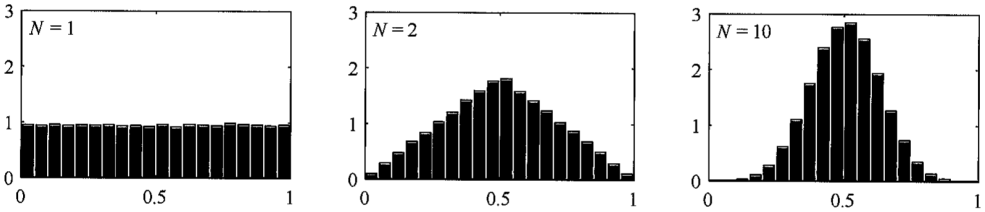


图 3.2: N 个均匀分布的数字的均值直方图. 我们可以看到, 随着 N 的增加, 分布趋向于 Gauss 分布.

Gauss 分布有很多重要的分析性质, 我们将会考虑其中一些性质的具体细节. 因此, 本节比前几节更具技术性, 读者需要熟悉多种矩阵相关的等式 (参见节 A.1).

3.2.1 Gauss 几何

下面我们考虑 Gauss 分布的几何形式.Gauss 分布对 $\boldsymbol{x}$ 的函数依赖是通过二次形式来呈现的:

$$\Delta^2 = (\boldsymbol{x} - \boldsymbol{\mu})^{\mathsf{T}} \boldsymbol{\Sigma}^{-1} (\boldsymbol{x} - \boldsymbol{\mu}). \tag{3.29}$$

定义 3.8 (Mahalanobis 距离)

设 $\boldsymbol{\Sigma}$ 为对称正定矩阵,$\boldsymbol{\mu} \in \mathbb{R}^D$. 对 $\boldsymbol{x} \in \mathbb{R}^D$, 称由(3.29)定义的 Δ^2 的算术平方根 Δ 为 $\boldsymbol{\mu}$ 到 $\boldsymbol{x}$ 的 Mahalanobis 距离.

♣

注

(重构新增) 引入 Mahalanobis 距离是为了把“距离”从各向同性的欧氏度量推广到由数据分布决定度量的情形: 在概率密度高的方向上, 同样的坐标差对应更小的 Δ, 因此“偏离多少个标准差”才是它度量的东西. 当 $\boldsymbol{\Sigma}$ 为单位矩阵时 $\Delta^2 = \|\boldsymbol{x} - \boldsymbol{\mu}\|^2$, 它退化为欧氏距离; 当 $\boldsymbol{\Sigma}$ 为对角矩阵时,$\Delta^2 = \sum_i (x_i - \mu_i)^2 / \sigma_i^2$, 即各坐标按标准差的加权平方和.

Gauss 分布在 $\boldsymbol{x}$ 空间中的等值面就是 Δ 为常数的曲面, 因为该二次型为常数. 定义中的正定性保证了 $\boldsymbol{\Sigma}^{-1}$ 存在且 $\Delta^2 \geqslant 0$, 从而 Δ 是良定义的实数量.

首先, 注意可以不失一般性地假设矩阵 $\boldsymbol{\Sigma}$ 为对称矩阵, 因为任何非对称的成分都会从指数中消失 (见练习 3.11). 考虑协方差矩阵的特征方程:

$$\boldsymbol{\Sigma} \boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i, \tag{3.30}$$

其中 $i = 1, \cdots, D$. 由于 $\boldsymbol{\Sigma}$ 是实对称矩阵, 其特征值是实数, 并且其特征向量可以正交化 (见练习 3.12), 故有

$$\boldsymbol{u}_i^{\mathsf{T}} \boldsymbol{u}_j = I_{ij}, \tag{3.31}$$

其中 I_{ij} 是单位矩阵中的元素, 满足

$$I_{ij} = \begin{cases} 1, & \text{当} i = j \text{ 时}, \\ 0, & \text{其他}. \end{cases} \tag{3.32}$$

命题 3.6 (实对称协方差矩阵的谱性质)

设 $\boldsymbol{\Sigma}$ 为 $D \times D$ 实对称矩阵. 则 $\boldsymbol{\Sigma}$ 的全部特征值 λ_i 都是实数, 且存在一组标准正交的特征向量 $\{\boldsymbol{u}_i\}$ 使(3.30)与(3.31)同时成立. 进一步,$\boldsymbol{\Sigma}$ 可以表示为其特征向量的展开

$$\boldsymbol{\Sigma} = \sum_{i=1}^{D} \lambda_i \boldsymbol{u}_i \boldsymbol{u}_i^{\mathsf{T}}, \tag{3.33}$$

而当 $\boldsymbol{\Sigma}$ 可逆时, 其逆矩阵可以表示为

$$\boldsymbol{\Sigma}^{-1}=\sum_{i=1}^{D}\frac{1}{\lambda_i}\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}}. \tag{3.34}$$

证明

(重构新增) 特征值为实数、特征向量可正交化这两点就是练习 3.12与练习 3.13的结论, 这里只补出(3.33)与(3.34)的推导.

记 $\boldsymbol{U}$ 为以 $\boldsymbol{u}_i^{\mathsf{T}}$ 为第 i 行的矩阵. 由(3.31),$\boldsymbol{U}$ 是正交矩阵, 即 $\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}}=\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}=\boldsymbol{I}$. 把(3.30)按列拼成矩阵形式得 $\boldsymbol{\Sigma}\boldsymbol{U}^{\mathsf{T}}=\boldsymbol{U}^{\mathsf{T}}\boldsymbol{\Lambda}$, 其中 $\boldsymbol{\Lambda}=\operatorname{diag}(\lambda_1,\cdots,\lambda_D)$, 于是

$$\boldsymbol{\Sigma}=\boldsymbol{U}^{\mathsf{T}}\boldsymbol{\Lambda}\boldsymbol{U}=\sum_{i=1}^{D}\lambda_i\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}},$$

这就是(3.33).

若 $\boldsymbol{\Sigma}$ 可逆, 则所有 $\lambda_i\neq 0$. 直接验证

$$\Big(\sum_{i=1}^{D}\lambda_i\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}}\Big)\Big(\sum_{j=1}^{D}\frac{1}{\lambda_j}\boldsymbol{u}_j\boldsymbol{u}_j^{\mathsf{T}}\Big)=\sum_{i,j}\frac{\lambda_i}{\lambda_j}\boldsymbol{u}_i(\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j)\boldsymbol{u}_j^{\mathsf{T}}=\sum_{i}\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}}=\boldsymbol{I},$$

其中第二个等号用到(3.31), 最后一个等号是(3.33)在 $\lambda_i\equiv 1$ 时的特例. 故(3.34)成立.

□

注

(重构新增) 谱分解把“矩阵求逆”化归为“标量取倒数”, 这是 Gauss 分布几乎所有解析计算的起点: 正因为有(3.34), 二次型才能写成(3.35)那样的对角和, 进而把多元密度分解为 D 个一元密度的乘积.

这里要求 $\boldsymbol{\Sigma}$ 对称, 是为了保证特征向量构成一组正交基; 若 $\boldsymbol{\Sigma}$ 不对称, 其特征值可能为复数,(3.33)不再成立. 要求 $\boldsymbol{\Sigma}$ 可逆 (等价于所有 $\lambda_i\neq 0$) 才能写出(3.34), 这也正是 Gauss 分布良定义的条件. 若某些 $\lambda_i=0$, 则 $\boldsymbol{\Sigma}$ 奇异, 分布退化到低维子空间上 (见本节末尾的讨论).

将(3.34)代入(3.29), 二次型变为

$$\Delta^2=\sum_{i=1}^{D}\frac{y_i^2}{\lambda_i}, \tag{3.35}$$

其中

$$y_i=\boldsymbol{u}_i^{\mathsf{T}}(\boldsymbol{x}-\boldsymbol{\mu}). \tag{3.36}$$

我们可以将 $\{y_i\}$ 解释为一个由正交向量定义的新坐标系, 它由原始 $\boldsymbol{x}$ 的坐标经过平移和旋转得到. 构造向量 $\boldsymbol{y}=(y_1,\cdots,y_D)^{\mathsf{T}}$, 有

$$\boldsymbol{y}=\boldsymbol{U}(\boldsymbol{x}-\boldsymbol{\mu}), \tag{3.37}$$

其中 $\boldsymbol{U}$ 是以 $\boldsymbol{u}_i^{\mathsf{T}}$ 为行向量的矩阵. 由(3.31)可知 $\boldsymbol{U}$ 是正交矩阵, 满足 $\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}}=\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}=\boldsymbol{I}$.

命题 3.7 (二次型的对角化)

设 $\boldsymbol{y}=\boldsymbol{U}(\boldsymbol{x}-\boldsymbol{\mu})$, 其中 $\boldsymbol{U}$ 的行由 $\boldsymbol{\Sigma}$ 的标准正交特征向量构成. 则(3.29)中的二次型可写成(3.35), 即 $\Delta^2=\sum_i y_i^2/\lambda_i$.

证明

(重构新增) 由(3.37),$\boldsymbol{x}-\boldsymbol{\mu}=\boldsymbol{U}^{\mathrm{T}}\boldsymbol{y}$(因为 $\boldsymbol{U}$ 正交). 代入(3.29):

$$\Delta^2=(\boldsymbol{U}^{\mathrm{T}}\boldsymbol{y})^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{U}^{\mathrm{T}}\boldsymbol{y})=\boldsymbol{y}^{\mathrm{T}}\left(\boldsymbol{U}\boldsymbol{\Sigma}^{-1}\boldsymbol{U}^{\mathrm{T}}\right)\boldsymbol{y}.$$

由(3.34),$\boldsymbol{U}\boldsymbol{\Sigma}^{-1}\boldsymbol{U}^{\mathrm{T}}=\mathrm{diag}(1/\lambda_1,\cdots,1/\lambda_D)$, 因为 $\boldsymbol{U}\boldsymbol{u}_i=\boldsymbol{e}_i$. 故内层矩阵为对角矩阵, 展开即得 $\Delta^2=\sum_i y_i^2/\lambda_i$, 其中 y_i 由(3.36)给出.

□

注

(重构新增) 这条结论的几何含义是: 在由 $\{\boldsymbol{u}_i\}$ 张成的坐标系中,Gauss 分布的等密度面 $\Delta^2=$ 常数 是一个与坐标轴对齐的椭球, 其第 i 个半轴长为 $\sqrt{\lambda_i}\,\Delta$. 旋转矩阵 $\boldsymbol{U}$ 把“相关的”原始坐标变换为“不相关的”新坐标, 因此这一步常被称为白化 (whitening) 的几何版本.

条件 $\boldsymbol{U}$ 正交是结论成立的关键: 只有正交变换才保持二次型的“形状”(长度与角度), 从而把 $\boldsymbol{\Sigma}^{-1}$ 精确对角化而不引入额外的交叉项. 若 $\boldsymbol{\Sigma}$ 有零特征值, 则对应的 $1/\lambda_i$ 无定义, 说明该方向上的方差为零、分布已被压扁到低维子空间.

由于(3.35)为常数, 因此二次型 (即 Gauss 密度) 在曲面上也为常数. 如果所有特征值 λ_i 均为正, 则这些曲面表示以 $\boldsymbol{\mu}$ 为中心、以 $\boldsymbol{u}_i$ 的方向为坐标轴且以 $\lambda_i^{1/2}$ 为半轴长的椭球, 如图 3.3所示.

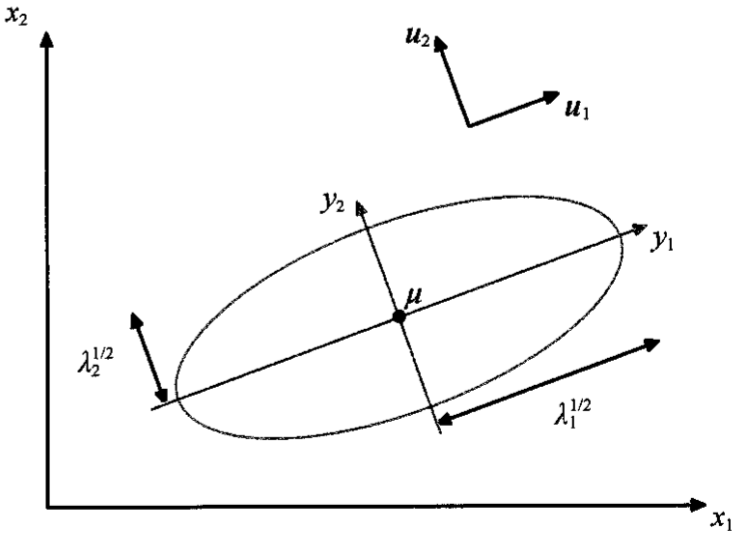


图 3.3: 红色曲线表示 Gauss 分布在二维空间 $\boldsymbol{x}=(x_1,x_2)$ 中具有恒定概率密度的椭圆面, 椭圆面上的密度是其在 $\boldsymbol{x}=\boldsymbol{\mu}$ 处取值的 $\exp(-1/2)$ 倍. 椭圆的轴由协方差矩阵的特征向量 $\boldsymbol{u}_i$ 定义, 具有相应的特征值 λ_i.

为了给出良定的 Gauss 分布, 需要保证协方差矩阵的所有特征值 λ_i 均严格为正, 否则分布就无法被正确归一化. 特征值全部严格为正的矩阵是正定的 (参见第 16 章). 在讨论隐变量模型时, 我们将遇到 Gauss 分布的一个或多个特征值为零的情形. 在这种情况下, 分布是奇异的且限制在一个更低维的子空间中. 如果所有的特征值非负, 则称这样的协方差矩阵为半正定的.

考虑 Gauss 分布在由 y_i 定义的新坐标系下的形式. 为了从 $\boldsymbol{x}$ 坐标系转换到 $\boldsymbol{y}$ 坐标系, 我们使用了 Jacobi 矩阵 $\boldsymbol{J}$, 其元素由下式给出:

$$J_{ij}=\frac{\partial x_i}{\partial y_j}=U_{ji}, \tag{3.38}$$

其中 U_{ji} 是矩阵 $\boldsymbol{U}^{\mathrm{T}}$ 中的元素. 运用矩阵 $\boldsymbol{U}$ 的正交性质, 可以得到 Jacobi 矩阵行列式的平方为

$$|\boldsymbol{J}|^2=|\boldsymbol{U}^{\mathrm{T}}|^2=|\boldsymbol{U}^{\mathrm{T}}|\,|\boldsymbol{U}|=|\boldsymbol{U}^{\mathrm{T}}\boldsymbol{U}|=|\boldsymbol{I}|=1, \tag{3.39}$$

从而有 $|\boldsymbol{J}| = 1$. 同样, 协方差矩阵的行列式 $|\boldsymbol{\Sigma}|$ 可以写成特征值的积, 于是有

$$|\boldsymbol{\Sigma}|^{1/2} = \prod_{j=1}^{D} \lambda_j^{1/2}. \tag{3.40}$$

因此, 在 y_i 的坐标系下,Gauss 分布有如下形式:

$$p(\boldsymbol{y}) = p(\boldsymbol{x})|\boldsymbol{J}| = \prod_{j=1}^{D} \frac{1}{(2\pi\lambda_j)^{1/2}} \exp\left\{-\frac{y_j^2}{2\lambda_j}\right\}. \tag{3.41}$$

命题 3.8 (旋转坐标下的 Gauss 密度)

在(3.37)定义的坐标变换 $\boldsymbol{y} = \boldsymbol{U}(\boldsymbol{x} - \boldsymbol{\mu})$ 下, 多元 Gauss 密度(3.28)化为(3.41), 即 D 个相互独立的一元 Gauss 密度的乘积, 其第 j 个因子的均值为 0、方差为 λ_j. 从而

$$\int p(\boldsymbol{y})\,\mathrm{d}\boldsymbol{y} = \prod_{j=1}^{D} \int_{-\infty}^{\infty} \frac{1}{(2\pi\lambda_j)^{1/2}} \exp\left\{-\frac{y_j^2}{2\lambda_j}\right\} \mathrm{d}y_j = 1. \tag{3.42}$$

♠

证明

(重构新增) 把(3.35)与(3.40)代入(3.28), 并乘以 Jacobi 行列式 $|\boldsymbol{J}| = 1$(由(3.39)), 即得

$$p(\boldsymbol{y}) = \frac{1}{(2\pi)^{D/2}\prod_j \lambda_j^{1/2}} \prod_{j=1}^{D} \exp\left\{-\frac{y_j^2}{2\lambda_j}\right\} = \prod_{j=1}^{D} \frac{1}{(2\pi\lambda_j)^{1/2}} \exp\left\{-\frac{y_j^2}{2\lambda_j}\right\},$$

这就是(3.41).

对(3.41)积分. 由于被积函数是乘积形式且积分变量可分离,

$$\int p(\boldsymbol{y})\,\mathrm{d}\boldsymbol{y} = \prod_{j=1}^{D} \int_{-\infty}^{\infty} \frac{1}{(2\pi\lambda_j)^{1/2}} \exp\left\{-\frac{y_j^2}{2\lambda_j}\right\} \mathrm{d}y_j.$$

每个因子都是一元 Gauss 密度在 $\mathbb{R}$ 上的积分, 由eq. (2.49)之后给出的归一化结论 (一元 Gauss 密度的归一性), 其值为 1. 故乘积为 1, 即(3.42), 从而验证了多元 Gauss 分布(3.28)确实是归一化的.

□

注

(重构新增) 这条结论是本节的技术核心: 它说明多元 Gauss 分布在“特征向量坐标系”下就是 D 个独立的一元 Gauss 分布之积. 由此立刻得到两个推论: 其一, 归一化可以逐因子验证, 从而(3.28)的系数 $(2\pi)^{-D/2}|\boldsymbol{\Sigma}|^{-1/2}$ 是正确的; 其二,$\boldsymbol{y}$ 的各分量相互独立, 而 $\boldsymbol{x} = \boldsymbol{U}^{\mathsf{T}}\boldsymbol{y} + \boldsymbol{\mu}$ 的分量一般并不独立——相关性完全由 $\boldsymbol{U}$ 的非对角结构承载.

条件 $\lambda_j > 0$ 在证明中通过 $\prod_j \lambda_j^{1/2}$ 与 $1/\lambda_j$ 两处使用: 若某个 $\lambda_j = 0$, 相应的因子不再是可积的一元 Gauss 密度,(3.42)失效, 这正是“奇异 Gauss 分布不能归一化”的解析表述.

3.2.2 矩

下面考虑 Gauss 分布的矩, 并给出对参数 $\boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 的一种解释.$\boldsymbol{x}$ 在 Gauss 分布下的期望由下式给出:

$$\mathbb{E}[\boldsymbol{x}] = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \int \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right\} \boldsymbol{x}\,\mathrm{d}\boldsymbol{x} = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \int \exp\left\{-\frac{1}{2}\boldsymbol{z}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{z}\right\} (\boldsymbol{z}+\boldsymbol{\mu})\,\mathrm{d}\boldsymbol{z}, \tag{3.43}$$

这里进行了变量替换 $\boldsymbol{z} = \boldsymbol{x} - \boldsymbol{\mu}$. 注意指数项是 $\boldsymbol{z}$ 的偶函数, 且由于是在 $(-\infty, \infty)$ 上求积分, 因此 $(\boldsymbol{z} + \boldsymbol{\mu})$ 中的 $\boldsymbol{z}$ 将由于对称性而消去. 于是有

$$\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu}. \tag{3.44}$$

接下来考虑 Gauss 分布的二阶矩. 在单变量的情形下, 我们知道二阶矩由 $\mathbb{E}[x^2]$ 给出. 对于多变量的 Gauss 分布, 存在由 $\mathbb{E}[x_i x_j]$ 给出的 D^2 个二阶矩. 我们可以将它们组合成矩阵 $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^{\mathsf{T}}]$, 该矩阵可以写作

$$\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^{\mathsf{T}}] = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \int \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right\} \boldsymbol{x}\boldsymbol{x}^{\mathsf{T}}\,\mathrm{d}\boldsymbol{x}. \tag{3.45}$$

这里再一次进行了变量替换 $z = x - \mu$. 注意交叉项中涉及的 μz^{T} 和 $z\mu^{\mathsf{T}}$ 也会由于对称性而消去. 项 $\mu\mu^{\mathsf{T}}$ 取常值, 因此可以拿到积分号之外. 由于 Gauss 分布是归一化的, 因此它是单位矩阵. 可以再一次利用(3.30)给出的协方差矩阵的特征向量扩展, 以及特征向量集的完备性, 写出

$$\mathbb{E}[xx^{\mathsf{T}}] = \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \int \exp\left\{-\frac{1}{2}z^{\mathsf{T}}\Sigma^{-1}z\right\}\left(zz^{\mathsf{T}} + \mu\mu^{\mathsf{T}}\right) \mathrm{d}z, \tag{3.46}$$

其中 $z = \sum_i y_i u_i$, 从而有

$$\mathbb{E}[xx^{\mathsf{T}}] = \mu\mu^{\mathsf{T}} + \sum_{i,j} u_i u_j^{\mathsf{T}} \int \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left\{-\frac{1}{2}\sum_k \frac{y_k^2}{\lambda_k}\right\} y_i y_j \, \mathrm{d}y. \tag{3.47}$$

这里用到了特征向量等式(3.30), 以及对称性导致中间行除 $i = j$ 之外的积分项消去的事实.(3.47)的最后一行用到了eq. (3.41)和eq. (3.40)的结果, 并结合了(3.33). 于是有

$$\mathbb{E}[xx^{\mathsf{T}}] = \mu\mu^{\mathsf{T}} + \Sigma. \tag{3.48}$$

命题 3.9 (多元 Gauss 分布的一阶矩与二阶矩)

设 $x \sim \mathcal{N}(x \mid \mu, \Sigma)$,$\Sigma$ 对称正定. 则

$$\mathbb{E}[x] = \mu, \qquad \mathbb{E}[xx^{\mathsf{T}}] = \mu\mu^{\mathsf{T}} + \Sigma.$$

♠

证明

(重构新增) 第一式的推导即(3.43)至(3.44): 作变量替换 $z = x - \mu$ 后, 被积函数 $\exp\{-\frac{1}{2}z^{\mathsf{T}}\Sigma^{-1}z\}z$ 关于 z 是奇函数, 而积分区域 $\mathbb{R}^D$ 关于原点对称, 故这部分积分为零; 剩下 μ 乘以归一化积分, 其值为 μ.

第二式: 由(3.37),$z = U^{\mathsf{T}}y$, 即 $z = \sum_i y_i u_i$, 其中 y 的各分量相互独立, 且由(3.41),$\mathbb{E}[y_i] = 0$、$\mathbb{E}[y_i^2] = \lambda_i$. 于是

$$\mathbb{E}[zz^{\mathsf{T}}] = \sum_{i,j} u_i u_j^{\mathsf{T}} \mathbb{E}[y_i y_j] = \sum_i \lambda_i u_i u_i^{\mathsf{T}} = \Sigma,$$

其中第二步用到独立性 ($i \neq j$ 时 $\mathbb{E}[y_i y_j] = \mathbb{E}[y_i]\mathbb{E}[y_j] = 0$), 最后一步用到(3.33). 再由 $x = z + \mu$ 展开 $\mathbb{E}[xx^{\mathsf{T}}] = \mathbb{E}[zz^{\mathsf{T}}] + \mu\mathbb{E}[z^{\mathsf{T}}] + \mathbb{E}[z]\mu^{\mathsf{T}} + \mu\mu^{\mathsf{T}}$, 并利用 $\mathbb{E}[z] = 0$, 即得(3.48).

□

注

(重构新增) 这两条结论给出了“μ 与 Σ 就是均值与协方差”这一说法:$\mathbb{E}[x] = \mu$ 说明第一个参数是位置参数,$\mathbb{E}[xx^{\mathsf{T}}] = \mu\mu^{\mathsf{T}} + \Sigma$ 说明第二个参数承担了扣除均值后的全部二阶波动.

推导中两次用到“对称性消去”: 一次是对奇函数在对称区域上的积分为零, 一次是 $i \neq j$ 时 y_i 与 y_j 独立而使交叉项的期望为零. 前者只依赖于积分区域的对称性与被积函数的奇偶性, 后者依赖(3.41)给出的独立性; 若 Σ 不是正定的 (例如奇异), 这些积分未必存在, 结论也就不再成立.

在定义单个随机变量的方差时, 我们在计算二阶矩之前减去了均值. 类似地, 在多元的情形中, 我们再次减去均值, 从而将随机变量 x 的协方差定义为

$$\mathrm{cov}[x] = \mathbb{E}\left[(x - \mathbb{E}[x])(x - \mathbb{E}[x])^{\mathsf{T}}\right]. \tag{3.49}$$

定义 3.9 (随机向量的协方差矩阵)

设 x 为 D 维随机向量且其二阶矩有限, 则称由(3.49)定义的 $D \times D$ 矩阵为 x 的协方差矩阵. 其 (i, j) 元素为 $\mathrm{cov}[x_i, x_j] = \mathbb{E}[(x_i - \mathbb{E}[x_i])(x_j - \mathbb{E}[x_j])]$.

♣

注

(重构新增) 把方差推广为矩阵, 是为了同时记录各分量的波动 (对角线) 与分量之间的线性相关 (非对角线). 由定义立即可见 $\operatorname{cov}[\boldsymbol{x}]$ 是对称矩阵, 且对任意实向量 $\boldsymbol{a}$ 有 $\boldsymbol{a}^{\mathsf{T}}\operatorname{cov}[\boldsymbol{x}]\boldsymbol{a} = \operatorname{var}[\boldsymbol{a}^{\mathsf{T}}\boldsymbol{x}] \geqslant 0$, 因此协方差矩阵总是半正定的; 只有当某个线性组合 $\boldsymbol{a}^{\mathsf{T}}\boldsymbol{x}$ 几乎必然为常数时才是奇异的.

定义要求二阶矩有限, 否则期望不存在, 协方差无从谈起. 这一条件在 Gauss 分布情形由 $\boldsymbol{\Sigma}$ 正定自动满足.

对于特定的 Gauss 分布的情形, 我们可以利用 $\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu}$ 并结合(3.48), 得到

$$\operatorname{cov}[\boldsymbol{x}] = \boldsymbol{\Sigma}. \tag{3.50}$$

命题 3.10 (Gauss 分布的协方差)

设 $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$, 则 $\operatorname{cov}[\boldsymbol{x}] = \boldsymbol{\Sigma}$.

证明

(重构新增) 由(3.49)与期望的线性性,

$$\operatorname{cov}[\boldsymbol{x}] = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^{\mathsf{T}}] - \mathbb{E}[\boldsymbol{x}]\,\mathbb{E}[\boldsymbol{x}]^{\mathsf{T}} = \left(\boldsymbol{\mu}\boldsymbol{\mu}^{\mathsf{T}} + \boldsymbol{\Sigma}\right) - \boldsymbol{\mu}\boldsymbol{\mu}^{\mathsf{T}} = \boldsymbol{\Sigma},$$

其中第二个等号用到(3.48)与(3.44).

□

注

(重构新增) 由于参数矩阵 $\boldsymbol{\Sigma}$ 决定了 Gauss 分布下 $\boldsymbol{x}$ 的协方差, 因此我们称其为协方差矩阵. 这条结论把(3.28)中的 $\boldsymbol{\Sigma}$ 从“指数里的一个矩阵”提升为“可解释的统计量”, 从而使最大似然估计 (小节 3.2.7) 有了明确的估计目标.

注意证明只用到一阶、二阶矩, 因此它对任意均值和协方差为 $\boldsymbol{\mu}, \boldsymbol{\Sigma}$ 的分布都给出“二阶矩形式”; 反过来,Gauss 分布的特殊性在于它是唯一由 $\boldsymbol{\mu}$ 与 $\boldsymbol{\Sigma}$ 完全确定的分布 (见练习 3.8的最大熵刻画).

3.2.3 局限性

尽管 Gauss 分布(3.28)经常用作简单的密度模型, 但它也存在一些明显的局限性. 考虑分布中自由参数的数量: 一个常规的对称协方差矩阵 $\boldsymbol{\Sigma}$ 含有 $D(D+1)/2$ 个独立的参数 (见练习 3.15),$\boldsymbol{\mu}$ 中还另外存在 D 个独立的参数, 因此总共有 $D(D+3)/2$ 个参数. 对于较大的 D, 参数量按 D 的平方增长, 对较大矩阵的操作和取逆计算代价巨大.

解决这个问题的一种方法是使用协方差矩阵的受限形式. 考虑到协方差矩阵是对角的,$\boldsymbol{\Sigma} = \operatorname{diag}(\sigma_i^2)$, 因此密度模型中总共有 $2D$ 个独立参数. 相应的概率密度等值线由轴对齐的椭球体给出. 我们可以进一步将协方差矩阵限制为与单位矩阵成正比,$\boldsymbol{\Sigma} = \sigma^2\boldsymbol{I}$, 即所谓各向同性的 (isotropic) 协方差, 这时模型中有 $D+1$ 个独立参数, 且概率密度等值线是一个球面. 常规协方差矩阵、对角协方差矩阵和各向同性协方差矩阵这 3 种情形如图 3.4所示. 遗憾的是, 虽然这些方法限制了分布中的自由度, 并使协方差矩阵的求逆变得更快, 但它们也极大限制了概率密度的形式, 并限制了模型从数据中捕获有趣相关性的能力.

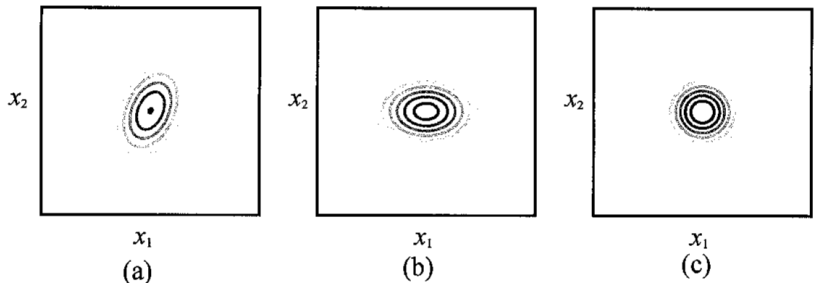


图 3.4: 二维 Gauss 分布的概率密度等值线, 其中协方差矩阵为:(a) 常规协方差矩阵;(b) 对角协方差矩阵, 在这种情况下, 椭圆等值线与坐标轴对齐;(c) 各向同性协方差矩阵, 与单位矩阵成正比, 在这种情况下, 等值线是同心圆.

Gauss 分布的另一个局限性是, 它本质上是单峰的 (即具有单个最大值), 因此无法为多峰分布提供良好的近似.Gauss 分布既存在由于具有太多参数而过于灵活的问题, 又存在所能表示的分布太过局限的问题. 稍后我们

将看到, 引入潜变量 (也称隐变量或无法观测的变量) 可以同时解决上述问题. 特别地, 通过引入离散潜变量实现 Gauss 混合, 可以获得一大类多峰分布 (参见小节 3.2.9). 同样, 连续潜变量的引入将使模型可以独立于数据空间的维度 D, 控制自由参数的数量, 同时仍然允许模型捕获数据集中的主要相关性 (参见第 16 章).

注

(重构新增) 这一小节指出了 Gauss 分布的“两难”: 完整协方差有 $D(D+3)/2$ 个参数, 随 D 二次增长, 在高维时既难以估计 (需要远多于参数个数的样本) 又难以计算 (求逆为 $O(D^3)$); 而对角或各向同性的限制虽然把参数降到 $O(D)$, 却丢掉了分量之间的相关性, 等值线只能是轴对齐椭球或球面.图 3.4的三个面板正是这三种情形的几何对照.

注意“参数多”与“表达力弱”并不矛盾: 完整协方差虽然参数多, 但仍然是单峰的, 无法表示多峰结构; 受限协方差参数少, 表达力更弱. 解决方向不是继续调整协方差的形式, 而是改变模型类——这正是下一小节之后引入混合模型的出发点.

3.2.4 条件分布

多元 Gauss 分布的一个重要性质是, 如果两组变量服从联合 Gauss 分布, 那么以其中一组变量为条件的另一组变量的条件分布是 Gauss 分布. 同样, 其中任意一组变量的边缘分布也是 Gauss 分布.

首先考虑条件分布的情况. 假设 D 维向量 $\boldsymbol{x}$ 服从 Gauss 分布 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$, 将 $\boldsymbol{x}$ 划分为两个不相交的子集 $\boldsymbol{x}_a$ 和 $\boldsymbol{x}_b$. 在不失一般性的情况下, 我们可以取 $\boldsymbol{x}_a$ 为前 M 个分量, 并取 $\boldsymbol{x}_b$ 作为剩余的 $D-M$ 个分量, 于是

$$\boldsymbol{x} = \begin{pmatrix} \boldsymbol{x}_a \\ \boldsymbol{x}_b \end{pmatrix}. \tag{3.51}$$

对均值向量 $\boldsymbol{\mu}$ 定义相应的分量:

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix}. \tag{3.52}$$

对协方差矩阵 $\boldsymbol{\Sigma}$ 也定义相应的分量:

$$\boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}. \tag{3.53}$$

注意, 协方差矩阵的对称性 $\boldsymbol{\Sigma}^{\mathsf{T}} = \boldsymbol{\Sigma}$ 意味着 $\boldsymbol{\Sigma}_{aa}$ 和 $\boldsymbol{\Sigma}_{bb}$ 也是对称的, 且 $\boldsymbol{\Sigma}_{ba} = \boldsymbol{\Sigma}_{ab}^{\mathsf{T}}$.

在许多情况下, 使用协方差矩阵的逆矩阵会更方便:

$$\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}. \tag{3.54}$$

定义 3.10 (精度矩阵)

设 $\boldsymbol{\Sigma}$ 为可逆的协方差矩阵, 则称其逆矩阵 $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ 为精度矩阵 (precision matrix). 精度矩阵的分量形式为

$$\boldsymbol{\Lambda} = \begin{pmatrix} \boldsymbol{\Lambda}_{aa} & \boldsymbol{\Lambda}_{ab} \\ \boldsymbol{\Lambda}_{ba} & \boldsymbol{\Lambda}_{bb} \end{pmatrix}. \tag{3.55}$$

♣

注

(重构新增) 引入精度矩阵是为了让某些性质写起来更简单: 本节将看到条件分布的均值和协方差用分量精度矩阵表示最简洁, 而边缘分布用分量协方差矩阵表示最简洁. 直观上, 精度矩阵是“方差的倒数”在矩阵情形的推广——精度越大, 分布在该方向上越集中.

注意 $\boldsymbol{\Lambda}_{aa}$ 与 $\boldsymbol{\Sigma}_{aa}^{-1}$ 一般并不相等 (练习 3.18、练习 3.23讨论的正是二者的关系). 这是因为求逆会把分块之间的耦合“混合”进来:$\boldsymbol{\Sigma}_{aa}$ 只描述 $\boldsymbol{x}_a$ 自身的波动, 而 $\boldsymbol{\Lambda}_{aa}$ 还包含了 $\boldsymbol{x}_a$ 与 $\boldsymbol{x}_b$ 的相关性对条件化的影响. 定义中的可逆性要求与 $\boldsymbol{\Sigma}$ 正定一致.

这对应于向量 $\boldsymbol{x}$ 的分量形式 [(3.51)]. 因为对称矩阵的逆矩阵也是对称的 (见练习 3.16), 所以 $\boldsymbol{\Lambda}_{aa}$ 和 $\boldsymbol{\Lambda}_{bb}$ 也是对称的, 且有 $\boldsymbol{\Lambda}_{ba} = \boldsymbol{\Lambda}_{ab}^{\mathsf{T}}$. 这里需要强调的是,$\boldsymbol{\Lambda}_{aa} = \boldsymbol{\Sigma}_{aa}^{-1}$ 不是简单地由 $\boldsymbol{\Sigma}_{aa}$ 的逆矩阵给出. 事实上, 我们将很快

研究分量矩阵的逆矩阵与其分量的逆矩阵之间的关系.

首先找到条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的表达式. 从概率的乘积法则中, 我们看到只需要将 $\boldsymbol{x}_b$ 固定为观测值, 并归一化表达式以获得关于 $\boldsymbol{x}_a$ 的有效概率分布, 就可以基于联合分布 $p(\boldsymbol{x}) = p(\boldsymbol{x}_a, \boldsymbol{x}_b)$ 得到条件分布. 有别于显式地执行归一化, 我们可以通过考虑(3.29)给出的 Gauss 分布指数中的二次形式, 然后在计算结束时恢复归一化系数, 从而更高效地获得解. 如果使用分量式(3.51)、(3.52)、(3.55), 则可以得到

$$\begin{aligned} -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) = &-\frac{1}{2}(\boldsymbol{x}_a-\boldsymbol{\mu}_a)^{\mathsf{T}}\boldsymbol{\Lambda}_{aa}(\boldsymbol{x}_a-\boldsymbol{\mu}_a) - \frac{1}{2}(\boldsymbol{x}_a-\boldsymbol{\mu}_a)^{\mathsf{T}}\boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b-\boldsymbol{\mu}_b) \\ &- \frac{1}{2}(\boldsymbol{x}_b-\boldsymbol{\mu}_b)^{\mathsf{T}}\boldsymbol{\Lambda}_{ba}(\boldsymbol{x}_a-\boldsymbol{\mu}_a) - \frac{1}{2}(\boldsymbol{x}_b-\boldsymbol{\mu}_b)^{\mathsf{T}}\boldsymbol{\Lambda}_{bb}(\boldsymbol{x}_b-\boldsymbol{\mu}_b). \end{aligned} \tag{3.56}$$

可以看到, 作为 $\boldsymbol{x}_a$ 的函数, 这又是一个二次形式. 因此, 相应的条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 将是 Gauss 分布. 由于这种分布完全由其均值和协方差确定, 因此我们的目标是通过检查(3.56)来确定 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的均值和协方差的表达式.

这是一个与 Gauss 分布相关的相当常见的运算示例, 有时称为 “完全平方” , 我们得到了一个定义了 Gauss 分布中指数项的二次形式. 我们需要确定相应的均值和协方差. 注意, 一般 Gauss 分布 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 中的指数可以写成

$$-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu}) = -\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} + \boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu} + \text{const}, \tag{3.57}$$

其中 “const” 表示独立于 $\boldsymbol{x}$ 的常数项. 再利用 $\boldsymbol{\Sigma}$ 的对称性, 这些问题就可以得到解决. 因此, 如果我们采用一般的二次形式, 并用(3.57)右侧给出的形式表示它, 则可以立即使 $\boldsymbol{x}$ 中二阶项的系数矩阵等于协方差矩阵的逆矩阵 $\boldsymbol{\Sigma}^{-1}$, 然后使 $\boldsymbol{x}$ 中线性项的系数等于 $\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}$, 即可得到 $\boldsymbol{\mu}$.

将此过程应用于条件 Gauss 分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$, 指数中的二次形式由(3.56)给出. 分别用 $\boldsymbol{\mu}_{a|b}$ 和 $\boldsymbol{\Sigma}_{a|b}$ 表示该分布的均值和协方差. 考虑式(3.56)对 $\boldsymbol{x}_a$ 的函数依赖性, 其中 $\boldsymbol{x}_b$ 可以视为常数. 如果我们挑选出 $\boldsymbol{x}_a$ 中的所有二阶项, 则得到

$$-\frac{1}{2}\boldsymbol{x}_a^{\mathsf{T}}\boldsymbol{\Lambda}_{aa}\boldsymbol{x}_a, \tag{3.58}$$

从中我们可以得出一个结论,$p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的协方差 (逆精度) 由下式给出:

$$\boldsymbol{\Sigma}_{a|b} = \boldsymbol{\Lambda}_{aa}^{-1}. \tag{3.59}$$

考虑(3.56)中所有在 $\boldsymbol{x}_a$ 中呈线性的项:

$$\boldsymbol{x}_a^{\mathsf{T}}\{\boldsymbol{\Lambda}_{aa}\boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b-\boldsymbol{\mu}_b)\}, \tag{3.60}$$

这里用到了表达式 $\boldsymbol{\Lambda}_{ba}^{\mathsf{T}} = \boldsymbol{\Lambda}_{ab}$. 根据我们对一般形式 [(3.57)] 的讨论, 该表达式中 $\boldsymbol{x}_a$ 的系数必须等于 $\boldsymbol{\Sigma}_{a|b}^{-1}\boldsymbol{\mu}_{a|b}$, 故有

$$\begin{aligned} \boldsymbol{\mu}_{a|b} &= \boldsymbol{\Sigma}_{a|b}\{\boldsymbol{\Lambda}_{aa}\boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b-\boldsymbol{\mu}_b)\} \\ &= \boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{aa}^{-1}\boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b-\boldsymbol{\mu}_b). \end{aligned} \tag{3.61}$$

这里用到了表达式(3.59).

命题 3.11 (条件 Gauss 分布: 精度形式)

设 $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 并按(3.51)至(3.55)分块. 则给定 $\boldsymbol{x}_b$ 时 $\boldsymbol{x}_a$ 的条件分布为 Gauss 分布, 其协方差与均值分别为

$$\boldsymbol{\Sigma}_{a|b} = \boldsymbol{\Lambda}_{aa}^{-1}, \qquad \boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{aa}^{-1}\boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b-\boldsymbol{\mu}_b).$$

且该条件分布与 $\boldsymbol{x}_b$ 的取值无关 (协方差不依赖 $\boldsymbol{x}_b$).

证明

(重构新增) 由(3.56), 联合密度的指数 (忽略常数 $-1/2$ 与不依赖 $\boldsymbol{x}_a$ 的项) 是 $\boldsymbol{x}_a$ 的二次函数, 故条件密度 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b) \propto p(\boldsymbol{x}_a, \boldsymbol{x}_b)$(把 $\boldsymbol{x}_b$ 视为常数) 也是 $\boldsymbol{x}_a$ 的二次指数形式, 即为 Gauss 分布.

将其与(3.57)对照: $\boldsymbol{x}_a$ 的二阶项系数矩阵为 $\boldsymbol{\Lambda}_{aa}$, 故条件分布的精度矩阵为 $\boldsymbol{\Lambda}_{aa}$, 即协方差为 $\boldsymbol{\Lambda}_{aa}^{-1}$, 这就是(3.59). $\boldsymbol{x}_a$ 的线性项系数由(3.60)给出, 为 $\boldsymbol{\Lambda}_{aa}\boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b - \boldsymbol{\mu}_b)$, 它必须等于 $\boldsymbol{\Sigma}_{a|b}^{-1}\boldsymbol{\mu}_{a|b} = \boldsymbol{\Lambda}_{aa}\boldsymbol{\mu}_{a|b}$, 两边左乘 $\boldsymbol{\Lambda}_{aa}^{-1}$ 即得(3.61). 由于指数中只出现 $\boldsymbol{x}_a$ 的二次项与线性项, 条件密度必定是归一化的 Gauss 密度, 其均值与协方差如上.

□

注

(重构新增) "完全平方" 是处理 Gauss 分布二次型的标准技巧: 只要把指数写成 $-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{x} + \boldsymbol{x}^{\mathsf{T}}\boldsymbol{b} + \text{const}$ 的形式, 就能立刻读出协方差 $\boldsymbol{A}^{-1}$ 与均值 $\boldsymbol{A}^{-1}\boldsymbol{b}$. 这里 $\boldsymbol{A} = \boldsymbol{\Lambda}_{aa}$、$\boldsymbol{b} = \boldsymbol{\Lambda}_{aa}\boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b - \boldsymbol{\mu}_b)$.

条件分布的协方差 $\boldsymbol{\Lambda}_{aa}^{-1}$ 与 $\boldsymbol{x}_b$ 无关, 这一事实非常重要: 它说明 "观测到 $\boldsymbol{x}_b$ 只移动条件分布的均值, 不改变其形状". 均值是 $\boldsymbol{x}_b$ 的线性函数, 这正是线性-Gauss 结构的来源. 注意这里用到 $\boldsymbol{\Lambda}_{aa}$ 可逆, 而它由 $\boldsymbol{\Sigma}$ 正定保证.

式(3.59)和式(3.61)给出的结果可以用原始联合分布 $p(\boldsymbol{x}_a, \boldsymbol{x}_b)$ 的分量精度矩阵来表示. 也可以用相应的分量协方差矩阵来表示这些结果. 为此, 对分量矩阵的逆矩阵使用以下恒等式 (见练习 3.18):

$$\begin{pmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{C} & \boldsymbol{D} \end{pmatrix}^{-1} = \begin{pmatrix} \boldsymbol{M} & -\boldsymbol{M}\boldsymbol{B}\boldsymbol{D}^{-1} \\ -\boldsymbol{D}^{-1}\boldsymbol{C}\boldsymbol{M} & \boldsymbol{D}^{-1} + \boldsymbol{D}^{-1}\boldsymbol{C}\boldsymbol{M}\boldsymbol{B}\boldsymbol{D}^{-1} \end{pmatrix}, \tag{3.62}$$

其中

$$\boldsymbol{M} = (\boldsymbol{A} - \boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C})^{-1}. \tag{3.63}$$

量 $\boldsymbol{M}^{-1}$ 称为(3.62)左侧矩阵相对于子矩阵 $\boldsymbol{D}$ 的 Schur 补 (Schur complement). 利用定义

$$\begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}^{-1} = \begin{pmatrix} \boldsymbol{\Lambda}_{aa} & \boldsymbol{\Lambda}_{ab} \\ \boldsymbol{\Lambda}_{ba} & \boldsymbol{\Lambda}_{bb} \end{pmatrix}, \tag{3.64}$$

并使用(3.62), 有

$$\boldsymbol{\Lambda}_{aa} = \left(\boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}\right)^{-1}, \tag{3.65}$$

$$\boldsymbol{\Lambda}_{ab} = -\left(\boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}\right)^{-1}\boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}. \tag{3.66}$$

条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的均值和协方差可以通过以下表达式获得:

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\boldsymbol{x}_b - \boldsymbol{\mu}_b), \tag{3.67}$$

$$\boldsymbol{\Sigma}_{a|b} = \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}. \tag{3.68}$$

比较(3.59)和(3.68), 可以看到条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 在用分量精度矩阵表示时, 比用分量协方差矩阵表示时更简单.

命题 3.12 (条件 Gauss 分布: 协方差形式)

在命题 3.11的设定下, 条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的均值与协方差亦可写成(3.67)与(3.68), 即

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\boldsymbol{x}_b - \boldsymbol{\mu}_b), \qquad \boldsymbol{\Sigma}_{a|b} = \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}.$$

♠

证明

(重构新增) 由(3.64)与(3.62)(取 $\boldsymbol{A} = \boldsymbol{\Sigma}_{aa}, \boldsymbol{B} = \boldsymbol{\Sigma}_{ab}, \boldsymbol{C} = \boldsymbol{\Sigma}_{ba}, \boldsymbol{D} = \boldsymbol{\Sigma}_{bb}$), 可得(3.65)与(3.66).

由(3.59), $\boldsymbol{\Sigma}_{a|b} = \boldsymbol{\Lambda}_{aa}^{-1}$, 对(3.65)取逆即得(3.68).

由(3.61),

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{aa}^{-1}\boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b - \boldsymbol{\mu}_b).$$

把(3.65)的逆 $\boldsymbol{\Lambda}_{aa}^{-1} = \boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}$ 与(3.66)代入, 得

$$-\boldsymbol{\Lambda}_{aa}^{-1}\boldsymbol{\Lambda}_{ab} = \left(\boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}\right)\left(\boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba}\right)^{-1}\boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1} = \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1},$$

于是 $\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a + \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}(\boldsymbol{x}_b - \boldsymbol{\mu}_b)$, 即(3.67).

□

注

(重构新增) 两种写法各有用途: 协方差形式(3.67)(3.68)只用联合分布的分量协方差矩阵, 直观地显示“条件均值是从先验均值出发、沿回归系数 $\boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}$ 修正”; 精度形式则说明条件协方差就是 $\boldsymbol{\Lambda}_{aa}^{-1}$, 无需计算 Schur 补.

这里出现了两个不同的 Schur 补且互为逆矩阵:$\boldsymbol{\Lambda}_{aa} = (\boldsymbol{\Sigma}_{aa} - \boldsymbol{\Sigma}_{ab}\boldsymbol{\Sigma}_{bb}^{-1}\boldsymbol{\Sigma}_{ba})^{-1}$. 证明中两次用到 $\boldsymbol{\Sigma}_{bb}$ 可逆, 这由 $\boldsymbol{\Sigma}$ 正定保证 (正定矩阵的主子块正定). 若 $\boldsymbol{\Sigma}_{bb}$ 奇异, 则 $\boldsymbol{x}_b$ 的分布在某个方向上退化, 条件化需要另行处理.

注意, 由(3.67)给出的条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的均值是 $\boldsymbol{x}_b$ 的线性函数, 而(3.68)给出的协方差与 $\boldsymbol{x}_b$ 无关. 这是线性-Gauss(linear-Gaussian) 模型的一个示例 (参见小节 11.1.4).

定义 3.11 (线性-Gauss 模型)

设 $\boldsymbol{x}$ 与 $\boldsymbol{y}$ 为随机向量. 若给定 $\boldsymbol{x}$ 时 $\boldsymbol{y}$ 的条件分布为 Gauss 分布, 且其均值是 $\boldsymbol{x}$ 的线性函数、其协方差与 $\boldsymbol{x}$ 无关, 即

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}\left(\boldsymbol{y} \mid \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}, \boldsymbol{L}^{-1}\right), \tag{3.69}$$

则称 $(\boldsymbol{x}, \boldsymbol{y})$ 服从线性-Gauss 模型, 其中 $\boldsymbol{A}, \boldsymbol{b}$ 为控制均值的参数,$\boldsymbol{L}$ 为精度矩阵.

♣

注

(重构新增) 线性-Gauss 模型之所以重要, 是因为它同时具备两个良好性质: 条件分布的均值对 $\boldsymbol{x}$ 线性 (便于解释为“回归”), 协方差不随 $\boldsymbol{x}$ 变化 (便于计算). 这两点正是(3.67)与(3.68)所显示的结构, 因此任何联合 Gauss 分布按分量分块后都自动成为线性-Gauss 模型.

条件 $\boldsymbol{L}$ 为正定 (即精度矩阵可逆) 保证条件密度良定义; 若 $\boldsymbol{A}$ 不是满秩, 则 $\boldsymbol{y}$ 的均值只落在 $\boldsymbol{x}$ 的一个线性子空间上, 但分布本身仍然良定义. 下一小节将看到, 线性-Gauss 结构在 Bayes 定理下封闭: 先验 Gauss、似然线性-Gauss 时, 后验仍是 Gauss(小节 3.2.6).

3.2.5 边缘分布

我们已经看到, 如果联合分布 $p(\boldsymbol{x}_a, \boldsymbol{x}_b)$ 是 Gauss 分布, 则条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 也是 Gauss 分布. 现在我们接着讨论由下式给出的边缘分布:

$$p(\boldsymbol{x}_a) = \int p(\boldsymbol{x}_a, \boldsymbol{x}_b)\,\mathrm{d}\boldsymbol{x}_b. \tag{3.70}$$

正如我们将看到的, 它也是 Gauss 分布. 与之前相同, 我们计算这种分布的策略是关注联合分布指数中的二次形式, 从而确定边缘分布 $p(\boldsymbol{x}_a)$ 的均值和协方差.

联合分布指数中的二次形式可以用(3.56)中的分量精度矩阵来表示. 我们的目标是通过积分消去 $\boldsymbol{x}_b$, 而最容易实现的是首先考虑涉及 $\boldsymbol{x}_b$ 的项, 然后完全平方以完成积分. 通过选出那些涉及 $\boldsymbol{x}_b$ 的项, 有

$$-\frac{1}{2}\boldsymbol{x}_b^{\mathsf{T}}\boldsymbol{\Lambda}_{bb}\boldsymbol{x}_b + \boldsymbol{x}_b^{\mathsf{T}}\boldsymbol{m} = -\frac{1}{2}\left(\boldsymbol{x}_b - \boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{m}\right)^{\mathsf{T}}\boldsymbol{\Lambda}_{bb}\left(\boldsymbol{x}_b - \boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{m}\right) + \frac{1}{2}\boldsymbol{m}^{\mathsf{T}}\boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{m}, \tag{3.71}$$

其中

$$\boldsymbol{m} = \boldsymbol{\Lambda}_{bb}\boldsymbol{\mu}_b - \boldsymbol{\Lambda}_{ba}(\boldsymbol{x}_a - \boldsymbol{\mu}_a). \tag{3.72}$$

可以看到, 对 $\boldsymbol{x}_b$ 的依赖性已转换为 Gauss 分布的标准二次形式, 对应于(3.71)右侧的第一项加上一个不依赖于 $\boldsymbol{x}_b$(但依赖于 $\boldsymbol{x}_a$) 的项. 因此, 当我们取这个二次形式的指数时, 可以看到(3.70)所需的对 $\boldsymbol{x}_b$ 的积分将采用以下形式:

$$\int \exp\left\{-\frac{1}{2}\left(\boldsymbol{x}_b - \boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{m}\right)^{\mathsf{T}}\boldsymbol{\Lambda}_{bb}\left(\boldsymbol{x}_b - \boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{m}\right)\right\}\mathrm{d}\boldsymbol{x}_b. \tag{3.73}$$

这种积分很容易实现, 由于该分布是未归一化的 Gauss 分布, 因此积分的结果将是归一化系数的倒数. 由(3.28)给出的归一化 Gauss 形式可知, 该系数与均值无关, 而仅取决于协方差矩阵的行列式. 因此, 通过计算完全关于 $\boldsymbol{x}_b$ 的平方, 可以积分消去 $\boldsymbol{x}_b$, 于是(3.71)左侧唯一剩下的依赖于 $\boldsymbol{x}_a$ 的项就是(3.71)右侧的最后一项, 其中 $\boldsymbol{m}$ 由(3.72)给

出. 将该项与(3.56)中依赖于 $\boldsymbol{x}_a$ 的其余项相结合, 可以得到

$$\begin{aligned}&\frac{1}{2}\left[\boldsymbol{\Lambda}_{bb}\boldsymbol{\mu}_b-\boldsymbol{\Lambda}_{ba}(\boldsymbol{x}_a-\boldsymbol{\mu}_a)\right]^{\mathsf{T}}\boldsymbol{\Lambda}_{bb}^{-1}\left[\boldsymbol{\Lambda}_{bb}\boldsymbol{\mu}_b-\boldsymbol{\Lambda}_{ba}(\boldsymbol{x}_a-\boldsymbol{\mu}_a)\right]\\&\quad-\frac{1}{2}\boldsymbol{x}_a^{\mathsf{T}}\boldsymbol{\Lambda}_{aa}\boldsymbol{x}_a+\boldsymbol{x}_a^{\mathsf{T}}\left(\boldsymbol{\Lambda}_{aa}\boldsymbol{\mu}_a+\boldsymbol{\Lambda}_{ab}\boldsymbol{\mu}_b\right)+\text{const}\\&\quad=-\frac{1}{2}\boldsymbol{x}_a^{\mathsf{T}}\left(\boldsymbol{\Lambda}_{aa}-\boldsymbol{\Lambda}_{ab}\boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{\Lambda}_{ba}\right)\boldsymbol{x}_a+\boldsymbol{x}_a^{\mathsf{T}}\left(\boldsymbol{\Lambda}_{aa}-\boldsymbol{\Lambda}_{ab}\boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{\Lambda}_{ba}\right)\boldsymbol{\mu}_a+\text{const},\end{aligned}\tag{3.74}$$

其中 “const” 表示独立于 $\boldsymbol{x}_a$ 的常数项. 同样, 通过与(3.57)进行比较, 可以看到边缘分布 $p(\boldsymbol{x}_a)$ 的协方差由下式给出:

$$\boldsymbol{\Sigma}_a=\left(\boldsymbol{\Lambda}_{aa}-\boldsymbol{\Lambda}_{ab}\boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{\Lambda}_{ba}\right)^{-1}.\tag{3.75}$$

均值则由下式给出:

$$\boldsymbol{\Sigma}_a\left(\boldsymbol{\Lambda}_{aa}-\boldsymbol{\Lambda}_{ab}\boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{\Lambda}_{ba}\right)\boldsymbol{\mu}_a=\boldsymbol{\mu}_a.\tag{3.76}$$

这里用到了(3.75). 协方差式(3.75)是用(3.55)给出的分量精度矩阵来表示的. 就像我们对条件分布所做的那样, 我们可以根据(3.53)给出的协方差矩阵的相应分量来重写它. 这些分量矩阵有如下关系:

$$\begin{pmatrix}\boldsymbol{\Lambda}_{aa}&\boldsymbol{\Lambda}_{ab}\\\boldsymbol{\Lambda}_{ba}&\boldsymbol{\Lambda}_{bb}\end{pmatrix}^{-1}=\begin{pmatrix}\boldsymbol{\Sigma}_{aa}&\boldsymbol{\Sigma}_{ab}\\\boldsymbol{\Sigma}_{ba}&\boldsymbol{\Sigma}_{bb}\end{pmatrix},\tag{3.77}$$

利用(3.62), 有

$$\left(\boldsymbol{\Lambda}_{aa}-\boldsymbol{\Lambda}_{ab}\boldsymbol{\Lambda}_{bb}^{-1}\boldsymbol{\Lambda}_{ba}\right)^{-1}=\boldsymbol{\Sigma}_{aa}.\tag{3.78}$$

这样我们就得到了直观上令人满意的结果, 即边缘分布 $p(\boldsymbol{x}_a)$ 具有下式给出的均值和协方差:

$$\mathbb{E}[\boldsymbol{x}_a]=\boldsymbol{\mu}_a,\tag{3.79}$$

$$\operatorname{cov}[\boldsymbol{x}_a]=\boldsymbol{\Sigma}_{aa}.\tag{3.80}$$

命题 3.13 (边缘 Gauss 分布)

设 $\boldsymbol{x}\sim\mathcal{N}(\boldsymbol{x}\mid\boldsymbol{\mu},\boldsymbol{\Sigma})$ 并按(3.51)至(3.53)分块, 则边缘分布 $p(\boldsymbol{x}_a)=\int p(\boldsymbol{x}_a,\boldsymbol{x}_b)\,\mathrm{d}\boldsymbol{x}_b$ 是 Gauss 分布, 且

$$\mathbb{E}[\boldsymbol{x}_a]=\boldsymbol{\mu}_a,\qquad\operatorname{cov}[\boldsymbol{x}_a]=\boldsymbol{\Sigma}_{aa}.$$

♠

证明

(重构新增) 推导即(3.71)至(3.78). 其中(3.71)是对 $\boldsymbol{x}_b$ 配方 (把含 $\boldsymbol{x}_b$ 的项写成 “完全平方 + 余项”),(3.73)中的积分因为被积函数是未归一化 Gauss 密度而等于 $(\det\boldsymbol{\Lambda}_{bb})^{-1/2}$ 乘以常数, 与 $\boldsymbol{x}_a$ 无关; 代回后剩下的 $\boldsymbol{x}_a$ 的二次形式由(3.74)给出, 与(3.57)对比读出协方差(3.75)与均值(3.76); 最后用分块逆公式(3.77)与(3.62)把精度形式化为协方差形式(3.78), 得 $\mathbb{E}[\boldsymbol{x}_a]=\boldsymbol{\mu}_a$、$\operatorname{cov}[\boldsymbol{x}_a]=\boldsymbol{\Sigma}_{aa}$.

□

注

(重构新增) 边缘分布 “只看对角线” 这一结论非常直观: 把 $\boldsymbol{x}_b$ 积分掉之后, $\boldsymbol{x}_a$ 的均值当然还是 $\boldsymbol{\mu}_a$, 其协方差当然还是联合协方差的左上块 $\boldsymbol{\Sigma}_{aa}$. 值得注意的是, 这一结论并非对所有分布成立, 它依赖于联合 Gaussss 分布的特殊形式.

证明中的两个条件值得强调: 一是 $\boldsymbol{\Lambda}_{bb}$ 可逆 (由 $\boldsymbol{\Sigma}$ 正定保证), 否则无法对 $\boldsymbol{x}_b$ 配方; 二是积分可分离为 “与 $\boldsymbol{x}_a$ 无关的常数 × 归一化积分”, 这正是二次型结构带来的便利. 若 $\boldsymbol{\Sigma}$ 奇异, 边缘分布仍可能是 Gauss 分布, 但需要处理退化情形.

可以看到, 对于边缘分布, 均值和协方差用分量协方差矩阵表达最简单; 而对于条件分布, 均值和协方差用分量精度矩阵表达最简单.

我们可以将分量形式 Gauss 分布的边缘分布和条件分布的结果总结如下. 对于联合 Gauss 分布 $\mathcal{N}(\boldsymbol{x}\mid\boldsymbol{\mu},\boldsymbol{\Sigma})$,

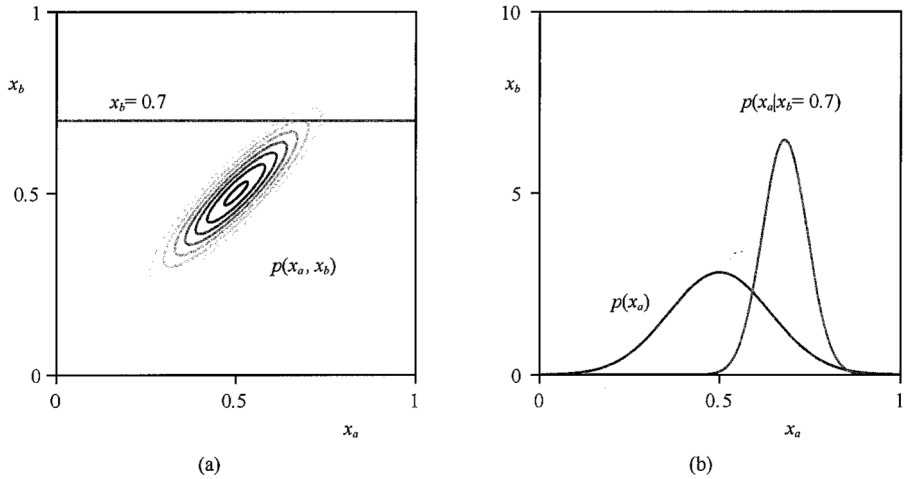


图 3.5: (a) 一个涉及两个变量的 Gauss 分布 $p(x_a, x_b)$ 的等值线.(b) $p(x_a)$ 的边缘分布 (蓝色曲线) 和 $x_b = 0.7$ 时的条件分布 $p(x_a \mid x_b)$(红色曲线).

给定 $\boldsymbol{\Lambda} = \boldsymbol{\Sigma}^{-1}$ 和下列分量矩阵:

$$\boldsymbol{x} = \begin{pmatrix} \boldsymbol{x}_a \\ \boldsymbol{x}_b \end{pmatrix}, \qquad \boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix}, \tag{3.81}$$

$$\boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} \end{pmatrix}, \qquad \boldsymbol{\Lambda} = \begin{pmatrix} \boldsymbol{\Lambda}_{aa} & \boldsymbol{\Lambda}_{ab} \\ \boldsymbol{\Lambda}_{ba} & \boldsymbol{\Lambda}_{bb} \end{pmatrix}, \tag{3.82}$$

则条件分布由下式给出:

$$p(\boldsymbol{x}_a \mid \boldsymbol{x}_b) = \mathcal{N}\left(\boldsymbol{x} \mid \boldsymbol{\mu}_{a|b}, \boldsymbol{\Lambda}_{aa}^{-1}\right), \tag{3.83}$$

$$\boldsymbol{\mu}_{a|b} = \boldsymbol{\mu}_a - \boldsymbol{\Lambda}_{aa}^{-1}\boldsymbol{\Lambda}_{ab}(\boldsymbol{x}_b - \boldsymbol{\mu}_b), \tag{3.84}$$

边缘分布由下式给出:

$$p(\boldsymbol{x}_a) = \mathcal{N}\left(\boldsymbol{x}_a \mid \boldsymbol{\mu}_a, \boldsymbol{\Sigma}_{aa}\right). \tag{3.85}$$

图 3.5用一个涉及两个变量的示例, 对多元 Gauss 分布的边缘分布和条件分布的思想进行了阐释.

3.2.6 Bayes 定理

在小节 3.2.4和小节 3.2.5中, 我们考虑了 Gauss 分布 $p(\boldsymbol{x})$, 其中我们将向量 $\boldsymbol{x}$ 分成了两个子向量 $\boldsymbol{x} = (\boldsymbol{x}_a, \boldsymbol{x}_b)$, 然后找到了条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 和边缘分布 $p(\boldsymbol{x}_a)$ 的表达式. 注意条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的均值是 $\boldsymbol{x}_b$ 的线性函数. 在这里, 假设我们有 Gauss 边缘分布 $p(\boldsymbol{x})$ 和 Gauss 条件分布 $p(\boldsymbol{y} \mid \boldsymbol{x})$, 其中 $p(\boldsymbol{y} \mid \boldsymbol{x})$ 的均值是 $\boldsymbol{x}$ 的线性函数, 协方差与 $\boldsymbol{x}$ 独立. 这是线性-Gauss 模型的一个例子 (Roweis and Ghahramani, 1999)(参见小节 11.1.4). 我们希望找到边缘分布 $p(\boldsymbol{y})$ 和条件分布 $p(\boldsymbol{x} \mid \boldsymbol{y})$. 这种结构在好几种生成式模型中都有出现 (参见第 16 章), 因此在这里推导出一般结论会很有帮助.

我们将边缘分布和条件分布取如下形式:

$$p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Lambda}^{-1}), \tag{3.86}$$

$$p(\boldsymbol{y} \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}, \boldsymbol{L}^{-1}), \tag{3.87}$$

其中 $\boldsymbol{\mu}$、$\boldsymbol{A}$ 和 $\boldsymbol{b}$ 是控制均值的参数,$\boldsymbol{\Lambda}$ 和 $\boldsymbol{L}$ 是精度矩阵. 如果 $\boldsymbol{x}$ 的维度为 M,$\boldsymbol{y}$ 的维度为 D, 则矩阵 $\boldsymbol{A}$ 的大小为 $D \times M$.

首先找到 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 上联合分布的表达式. 为此, 我们定义

$$\boldsymbol{z} = \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{pmatrix}, \tag{3.88}$$

然后考虑联合分布的对数:

$$\begin{aligned} \ln p(\boldsymbol{z}) &= \ln p(\boldsymbol{x}) + \ln p(\boldsymbol{y} \mid \boldsymbol{x}) \\ &= -\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Lambda}(\boldsymbol{x}-\boldsymbol{\mu}) - \frac{1}{2}(\boldsymbol{y}-\boldsymbol{A}\boldsymbol{x}-\boldsymbol{b})^{\mathsf{T}}\boldsymbol{L}(\boldsymbol{y}-\boldsymbol{A}\boldsymbol{x}-\boldsymbol{b}) + \text{const}, \end{aligned} \tag{3.89}$$

其中 “const” 表示独立于 $\boldsymbol{x}$ 和 $\boldsymbol{y}$ 的常数项. 如前所述, 这是 $\boldsymbol{z}$ 分量的二次函数, 因此 $p(\boldsymbol{z})$ 是 Gauss 分布. 为了找到这个 Gauss 分布的精度矩阵,(3.89)中的二阶项可以写成

$$\begin{aligned} &-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\left(\boldsymbol{\Lambda}+\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A}\right)\boldsymbol{x} - \frac{1}{2}\boldsymbol{y}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{y} + \frac{1}{2}\boldsymbol{y}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A}\boldsymbol{x} + \frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{y} \\ &= -\frac{1}{2}\begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{pmatrix}^{\mathsf{T}} \begin{pmatrix} \boldsymbol{\Lambda}+\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A} & -\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L} \\ -\boldsymbol{L}\boldsymbol{A} & \boldsymbol{L} \end{pmatrix} \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{pmatrix} = -\frac{1}{2}\boldsymbol{z}^{\mathsf{T}}\boldsymbol{R}\boldsymbol{z}, \end{aligned} \tag{3.90}$$

因此,$\boldsymbol{z}$ 上的 Gauss 分布的精度 (逆协方差) 矩阵由下式给出:

$$\boldsymbol{R} = \begin{pmatrix} \boldsymbol{\Lambda}+\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A} & -\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L} \\ -\boldsymbol{L}\boldsymbol{A} & \boldsymbol{L} \end{pmatrix}. \tag{3.91}$$

协方差矩阵是通过取精度矩阵的逆矩阵来求的, 这可以通过使用逆矩阵公式(3.62)来实现 (见练习 3.23), 即

$$\text{cov}[\boldsymbol{z}] = \boldsymbol{R}^{-1} = \begin{pmatrix} \boldsymbol{\Lambda}^{-1} & \boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\mathsf{T}} \\ \boldsymbol{A}\boldsymbol{\Lambda}^{-1} & \boldsymbol{L}^{-1}+\boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\mathsf{T}} \end{pmatrix}. \tag{3.92}$$

同样, 也可以通过识别(3.89)中的线性项来计算关于 $\boldsymbol{z}$ 的 Gauss 分布的均值, 它由下式给出:

$$\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Lambda}\boldsymbol{\mu} - \boldsymbol{x}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{b} + \boldsymbol{y}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{b} = \begin{pmatrix} \boldsymbol{x} \\ \boldsymbol{y} \end{pmatrix}^{\mathsf{T}} \begin{pmatrix} \boldsymbol{\Lambda}\boldsymbol{\mu} - \boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{b} \\ \boldsymbol{L}\boldsymbol{b} \end{pmatrix}. \tag{3.93}$$

使用我们之前通过在多元 Gauss 分布的二次型上完全平方得到的结果(3.57), 可以发现 $\boldsymbol{z}$ 的均值由下式给出:

$$\mathbb{E}[\boldsymbol{z}] = \boldsymbol{R}^{-1} \begin{pmatrix} \boldsymbol{\Lambda}\boldsymbol{\mu} - \boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{b} \\ \boldsymbol{L}\boldsymbol{b} \end{pmatrix}. \tag{3.94}$$

利用(3.92)(见练习 3.24), 可以得到

$$\mathbb{E}[\boldsymbol{z}] = \begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b} \end{pmatrix}. \tag{3.95}$$

接下来, 我们在 $\boldsymbol{x}$ 上进行边缘化, 得到 $p(\boldsymbol{y})$ 的表达式. 回想一下, 当使用分量协方差矩阵来表达服从 Gauss 分布的随机向量子集的边缘分布时, 我们可以采用非常简单的形式 (参见小节 3.2.5). 具体来说, 其均值和协方差分别由(3.79)和(3.80)给出. 利用(3.92)和(3.95)可以看到, 边缘分布 $p(\boldsymbol{y})$ 的均值和协方差由下式给出:

$$\mathbb{E}[\boldsymbol{y}] = \boldsymbol{A}\boldsymbol{\mu} + \boldsymbol{b}, \tag{3.96}$$

$$\text{cov}[\boldsymbol{y}] = \boldsymbol{L}^{-1} + \boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\mathsf{T}}. \tag{3.97}$$

这个结果的一个特例出现在 $\boldsymbol{A} = \boldsymbol{I}$ 时, 在这种情况下, 边缘分布退化为两个 Gauss 分布的卷积. 由此可以看到, 卷积的均值是两个 Gauss 分布的均值之和, 卷积的协方差则是它们的协方差之和.

最后推导条件分布 $p(\boldsymbol{x} \mid \boldsymbol{y})$ 的表达式. 回想一下, 条件分布的结果可以很容易地通过(3.59)和(3.61), 用分量精度矩阵来表示 (参见小节 3.2.4). 将这些结果应用于(3.92)和(3.95), 可以看到条件分布 $p(\boldsymbol{x} \mid \boldsymbol{y})$ 具有以下均值和协方差:

$$\mathbb{E}[\boldsymbol{x} \mid \boldsymbol{y}] = \left(\boldsymbol{\Lambda}+\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A}\right)^{-1}\left\{\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}(\boldsymbol{y}-\boldsymbol{b}) + \boldsymbol{\Lambda}\boldsymbol{\mu}\right\}, \tag{3.98}$$

$$\text{cov}[\boldsymbol{x} \mid \boldsymbol{y}] = \left(\boldsymbol{\Lambda}+\boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A}\right)^{-1}. \tag{3.99}$$

命题 3.14 (线性-Gauss 模型中的边缘分布与后验分布)

设 $p(\boldsymbol{x})=\mathcal{N}(\boldsymbol{x}\mid\boldsymbol{\mu},\boldsymbol{\Lambda}^{-1})$ 与 $p(\boldsymbol{y}\mid\boldsymbol{x})=\mathcal{N}(\boldsymbol{y}\mid\boldsymbol{A}\boldsymbol{x}+\boldsymbol{b},\boldsymbol{L}^{-1})$ 由(3.86)与(3.87)给出, 且 $\boldsymbol{\Lambda},\boldsymbol{L}$ 正定. 则边缘分布与条件分布分别为

$$p(\boldsymbol{y})=\mathcal{N}\big(\boldsymbol{y}\mid\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b},\ \boldsymbol{L}^{-1}+\boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\top}\big),$$

$$p(\boldsymbol{x}\mid\boldsymbol{y})=\mathcal{N}\big(\boldsymbol{x}\mid\boldsymbol{\Sigma}\{\boldsymbol{\Lambda}\boldsymbol{\mu}+\boldsymbol{A}^{\top}\boldsymbol{L}(\boldsymbol{y}-\boldsymbol{b})\},\ \boldsymbol{\Sigma}\big),$$

其中

$$\boldsymbol{\Sigma}=\big(\boldsymbol{\Lambda}+\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{A}\big)^{-1}. \tag{3.100}$$

♠

证明

(重构新增) 联合分布的对数由(3.89)给出, 它是 $\boldsymbol{z}=(\boldsymbol{x},\boldsymbol{y})$ 的二次函数, 故联合分布为 Gauss 分布.

由(3.90)读出精度矩阵 $\boldsymbol{R}$((3.91)), 再由分块逆公式(3.62)求逆. 取 $\boldsymbol{A}_{\text{block}}=\boldsymbol{\Lambda}+\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{A}$、$\boldsymbol{B}_{\text{block}}=-\boldsymbol{A}^{\top}\boldsymbol{L}$、$\boldsymbol{C}_{\text{block}}=-\boldsymbol{L}\boldsymbol{A}$、$\boldsymbol{D}_{\text{block}}=\boldsymbol{L}$, 则 Schur 补为 $\boldsymbol{M}=(\boldsymbol{\Lambda}+\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{A}-\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{L}^{-1}\boldsymbol{L}\boldsymbol{A})^{-1}=\boldsymbol{\Lambda}^{-1}$, 代入(3.62)即得(3.92).

由(3.57), 均值等于协方差矩阵乘以线性项系数. 把(3.92)与(3.93)相乘:

$$\boldsymbol{R}^{-1}\begin{pmatrix}\boldsymbol{\Lambda}\boldsymbol{\mu}-\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{b}\\ \boldsymbol{L}\boldsymbol{b}\end{pmatrix}=\begin{pmatrix}\boldsymbol{\Lambda}^{-1}(\boldsymbol{\Lambda}\boldsymbol{\mu}-\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{b})+\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{b}\\ \boldsymbol{A}\boldsymbol{\Lambda}^{-1}(\boldsymbol{\Lambda}\boldsymbol{\mu}-\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{b})+(\boldsymbol{L}^{-1}+\boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\top})\boldsymbol{L}\boldsymbol{b}\end{pmatrix}=\begin{pmatrix}\boldsymbol{\mu}\\ \boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b}\end{pmatrix},$$

即(3.95).

于是联合分布为 $\mathcal{N}(\boldsymbol{z}\mid(\boldsymbol{\mu},\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b}),\operatorname{cov}[\boldsymbol{z}])$. 按命题 3.13对 $\boldsymbol{x}$ 取边缘, 均值为 $\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b}$, 协方差为 $\boldsymbol{R}^{-1}$ 的右下块 $\boldsymbol{L}^{-1}+\boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\top}$, 即(3.96)与(3.97).

最后按命题 3.11计算 $p(\boldsymbol{x}\mid\boldsymbol{y})$: 在分块 $\boldsymbol{z}=(\boldsymbol{x},\boldsymbol{y})$ 下,$\boldsymbol{x}$ 的地位相当于 "$\boldsymbol{x}_a$" ,$\boldsymbol{y}$ 相当于 "$\boldsymbol{x}_b$" . 联合精度矩阵为(3.91), 故 $\boldsymbol{\Lambda}_{aa}=\boldsymbol{\Lambda}+\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{A}$、$\boldsymbol{\Lambda}_{ab}=-\boldsymbol{A}^{\top}\boldsymbol{L}$、$\boldsymbol{\mu}_a=\boldsymbol{\mu}$、$\boldsymbol{\mu}_b=\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b}$. 代入(3.59)得 $\operatorname{cov}[\boldsymbol{x}\mid\boldsymbol{y}]=(\boldsymbol{\Lambda}+\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{A})^{-1}$, 即(3.99)与(3.100); 代入(3.61)得

$$\begin{aligned}\mathbb{E}[\boldsymbol{x}\mid\boldsymbol{y}]&=\boldsymbol{\mu}-(\boldsymbol{\Lambda}+\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{A})^{-1}(-\boldsymbol{A}^{\top}\boldsymbol{L})\big(\boldsymbol{y}-(\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b})\big)\\&=\boldsymbol{\Sigma}\big\{\boldsymbol{\Lambda}\boldsymbol{\mu}+\boldsymbol{A}^{\top}\boldsymbol{L}(\boldsymbol{y}-\boldsymbol{b})\big\},\end{aligned}$$

其中 $\boldsymbol{\Sigma}=(\boldsymbol{\Lambda}+\boldsymbol{A}^{\top}\boldsymbol{L}\boldsymbol{A})^{-1}$, 这就是(3.98).

□

注

(重构新增) 这组公式是线性-Gauss 模型的 "闭合性" 定理:Gauss 先验与线性-Gauss 似然的组合在边缘化与条件化下都仍在 Gauss 族内.(3.97)中的两项有清晰含义:$\boldsymbol{L}^{-1}$ 是观测噪声的协方差,$\boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\top}$ 是先验不确定性经 $\boldsymbol{A}$ 传播后的协方差, 二者相加即 "不确定性叠加" .

后验协方差(3.99)不依赖于观测值 $\boldsymbol{y}$, 这说明观测只改变我们对 $\boldsymbol{x}$ 的 "中心位置", 不改变后验的 "形状" —— 这正是线性-Gauss 结构可解析计算的根本原因. 后验均值(3.98)可以看作先验均值与数据信息的加权折中:$\boldsymbol{\Lambda}\boldsymbol{\mu}$ 把先验 "拉回" $\boldsymbol{\mu}$,$\boldsymbol{A}^{\top}\boldsymbol{L}(\boldsymbol{y}-\boldsymbol{b})$ 把估计 "拉向" 与观测一致的方向.

推导用到 $\boldsymbol{\Lambda}$ 与 $\boldsymbol{L}$ 正定 (保证各精度矩阵可逆、Schur 补可逆). 若 $\boldsymbol{A}$ 不是满秩, 后验仍然良定义, 只是某些方向上的信息完全来自先验.

这些结果可以总结如下. 给定关于 $\boldsymbol{x}$ 的边缘分布和给定 $\boldsymbol{x}$ 时关于 $\boldsymbol{y}$ 的条件分布:

$$p(\boldsymbol{x})=\mathcal{N}(\boldsymbol{x}\mid\boldsymbol{\mu},\boldsymbol{\Lambda}^{-1}), \tag{3.101}$$

$$p(\boldsymbol{y}\mid\boldsymbol{x})=\mathcal{N}(\boldsymbol{y}\mid\boldsymbol{A}\boldsymbol{x}+\boldsymbol{b},\boldsymbol{L}^{-1}), \tag{3.102}$$

则 $\boldsymbol{y}$ 的边缘分布和给定 $\boldsymbol{y}$ 时 $\boldsymbol{x}$ 的条件分布由下式给出:

$$p(\boldsymbol{y})=\mathcal{N}\big(\boldsymbol{y}\mid\boldsymbol{A}\boldsymbol{\mu}+\boldsymbol{b},\ \boldsymbol{L}^{-1}+\boldsymbol{A}\boldsymbol{\Lambda}^{-1}\boldsymbol{A}^{\top}\big), \tag{3.103}$$

$$p(\boldsymbol{x}\mid\boldsymbol{y})=\mathcal{N}\big(\boldsymbol{x}\mid\boldsymbol{\Sigma}\{\boldsymbol{A}^{\top}\boldsymbol{L}(\boldsymbol{y}-\boldsymbol{b})+\boldsymbol{\Lambda}\boldsymbol{\mu}\},\ \boldsymbol{\Sigma}\big), \tag{3.104}$$

其中

$$\boldsymbol{\Sigma} = \left(\boldsymbol{\Lambda} + \boldsymbol{A}^{\mathsf{T}}\boldsymbol{L}\boldsymbol{A}\right)^{-1}. \tag{3.105}$$

对这种条件分布进行估计可以看作 Bayes 定理的一个示例. 在 Bayes 定理中, 我们将 $p(\boldsymbol{x})$ 称为 $\boldsymbol{x}$ 的先验分布. 如果观测到变量 $\boldsymbol{y}$, 则条件分布 $p(\boldsymbol{x} \mid \boldsymbol{y})$ 表示 $\boldsymbol{x}$ 所对应的后验分布. 得到边缘分布和条件分布后, 我们可以有效地将联合分布 $p(\boldsymbol{z}) = p(\boldsymbol{x})p(\boldsymbol{y} \mid \boldsymbol{x})$ 表示为 $p(\boldsymbol{x} \mid \boldsymbol{y})p(\boldsymbol{y})$ 的形式.

注

(重构新增) 这一小节把“先验—似然—后验”的语言落实为具体的 Gauss 公式:(3.104)就是后验分布,(3.103)是证据 (边缘似然), 而后验均值与协方差正是(3.98)与(3.99). 这一结构在连续潜变量模型中反复出现 (第 16 章), 因为在那里 $\boldsymbol{x}$ 是潜变量、$\boldsymbol{y}$ 是观测, 二者正是线性-Gauss 关系.

条件 $\boldsymbol{\Lambda}$、$\boldsymbol{L}$ 正定保证所有协方差可逆; 若先验方差趋于无穷 ($\boldsymbol{\Lambda} \to 0$), 后验由数据主导; 若观测噪声趋于零 ($\boldsymbol{L} \to \infty$), 后验退化为对 $\boldsymbol{A}\boldsymbol{x} + \boldsymbol{b} = \boldsymbol{y}$ 的精确求解.

3.2.7 最大似然

给定数据集 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 假设其中的观测值 $\{\boldsymbol{x}_n\}$ 服从多元 Gauss 分布且相互独立, 则可以通过最大似然来估计分布的参数. 对数似然函数由下式给出:

$$\ln p(X \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{ND}{2}\ln(2\pi) - \frac{N}{2}\ln|\boldsymbol{\Sigma}| - \frac{1}{2}\sum_{n=1}^{N}(\boldsymbol{x}_n - \boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}). \tag{3.106}$$

通过简单地重排, 可以看到似然函数与数据集相关的部分仅依赖于这两个量:

$$\sum_{n=1}^{N}\boldsymbol{x}_n \quad \text{和} \quad \sum_{n=1}^{N}\boldsymbol{x}_n\boldsymbol{x}_n^{\mathsf{T}}, \tag{3.107}$$

这称为 Gauss 分布的充分统计量 (sufficient statistic). 使用节 A.3中的矩阵求导法则, 对数似然关于 $\boldsymbol{\mu}$ 的导数由下式给出:

$$\frac{\partial}{\partial\boldsymbol{\mu}}\ln p(X \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^{N}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}). \tag{3.108}$$

令导数为零, 即可得到均值的最大似然估计解:

$$\boldsymbol{\mu}_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n, \tag{3.109}$$

这是观测数据点集的平均值. 关于 $\boldsymbol{\Sigma}$ 最大化(3.106)可以起到更大的作用. 最简单的方法是忽略对称约束, 正如所要求的那样, 得到的解是对称的 (参见练习 3.28). 我们在 Magnus and Neudecker(1999) 中可以找到对这一结果的替代推导, 这些推导显式地施加了对称性和正定性约束. 结果正如预期的那样, 具有如下形式:

$$\boldsymbol{\Sigma}_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})^{\mathsf{T}}. \tag{3.110}$$

其中涉及 $\boldsymbol{\mu}_{\mathrm{ML}}$, 因为这是相对于 $\boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 联合最大化的结果. 注意,$\boldsymbol{\mu}_{\mathrm{ML}}$ 的解 [(3.109)] 并不依赖于 $\boldsymbol{\Sigma}_{\mathrm{ML}}$, 因此我们可以首先估计 $\boldsymbol{\mu}_{\mathrm{ML}}$, 然后用它来估计 $\boldsymbol{\Sigma}_{\mathrm{ML}}$.

命题 3.15 (多元 Gauss 分布的最大似然估计)

设 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 为来自 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 的独立同分布观测,$N \geqslant 1$. 则对数似然(3.106)关于 $(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ 的最大值点由(3.109)与(3.110)给出, 其中 $\boldsymbol{\Sigma}$ 限制在正定矩阵集合上. ♠

证明

(重构新增) 关于 $\boldsymbol{\mu}$: 由(3.108), 令其为零得 $\sum_n \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}) = 0$. 由于 $\boldsymbol{\Sigma}^{-1}$ 可逆, 两边左乘 $\boldsymbol{\Sigma}$ 得 $\sum_n (\boldsymbol{x}_n - \boldsymbol{\mu}) = 0$, 即 $\boldsymbol{\mu}_{\mathrm{ML}} = \frac{1}{N}\sum_n \boldsymbol{x}_n$, 这就是(3.109).

关于 $\boldsymbol{\Sigma}$: 记 $\boldsymbol{S} = \sum_n (\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})^{\mathsf{T}}$. 利用 $\sum_n (\boldsymbol{x}_n - \boldsymbol{\mu})(\boldsymbol{x}_n - \boldsymbol{\mu})^{\mathsf{T}} = \boldsymbol{S} + N(\boldsymbol{\mu}_{\mathrm{ML}} - \boldsymbol{\mu})(\boldsymbol{\mu}_{\mathrm{ML}} - \boldsymbol{\mu})^{\mathsf{T}}$(交叉项因 $\sum_n (\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}}) = 0$ 而消失), 并利用迹的循环性 $\boldsymbol{a}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{a} = \mathrm{tr}(\boldsymbol{\Sigma}^{-1}\boldsymbol{a}\boldsymbol{a}^{\mathsf{T}})$, 可把(3.106)写成

$$\ln p(X \mid \boldsymbol{\mu}_{\mathrm{ML}}, \boldsymbol{\Sigma}) = -\frac{ND}{2}\ln(2\pi) - \frac{N}{2}\ln|\boldsymbol{\Sigma}| - \frac{1}{2}\mathrm{tr}\left(\boldsymbol{\Sigma}^{-1}\boldsymbol{S}\right) - \frac{N}{2}(\boldsymbol{\mu}_{\mathrm{ML}} - \boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_{\mathrm{ML}} - \boldsymbol{\mu}).$$

在 $\boldsymbol{\mu} = \boldsymbol{\mu}_{\mathrm{ML}}$ 处最后一项为零. 对 $\boldsymbol{\Sigma}$ 求导 (忽略对称约束, 把它当作一般矩阵), 利用 $\partial \ln|\boldsymbol{\Sigma}|/\partial\boldsymbol{\Sigma} = \boldsymbol{\Sigma}^{-\mathsf{T}}$ 与 $\partial\,\mathrm{tr}(\boldsymbol{\Sigma}^{-1}\boldsymbol{S})/\partial\boldsymbol{\Sigma} = -\boldsymbol{\Sigma}^{-\mathsf{T}}\boldsymbol{S}^{\mathsf{T}}\boldsymbol{\Sigma}^{-\mathsf{T}}$, 令导数为零得

$$-\frac{N}{2}\boldsymbol{\Sigma}^{-\mathsf{T}} + \frac{1}{2}\boldsymbol{\Sigma}^{-\mathsf{T}}\boldsymbol{S}^{\mathsf{T}}\boldsymbol{\Sigma}^{-\mathsf{T}} = 0 \quad\Longrightarrow\quad \boldsymbol{\Sigma} = \frac{1}{N}\boldsymbol{S}^{\mathsf{T}} = \frac{1}{N}\boldsymbol{S},$$

最后一步因为 $\boldsymbol{S}$ 对称. 这正是(3.110).

由于 $\boldsymbol{S}$ 半正定, 得到的 $\boldsymbol{\Sigma}_{\mathrm{ML}}$ 半正定; 当 $N \geqslant D$ 且数据张满 $\mathbb{R}^D$ 时它正定, 落在约束集内部. 此时 $\ln p$ 关于 $\boldsymbol{\Sigma}$ 在约束集上是凹的 ($-\ln|\boldsymbol{\Sigma}|$ 严格凸、$-\mathrm{tr}(\boldsymbol{\Sigma}^{-1}\boldsymbol{S})$ 凸), 故该驻点是最大值点. 若 $\boldsymbol{S}$ 奇异, 则最大值在正定矩阵集合的边界上取到 (不存在), 此时应把 $\boldsymbol{\Sigma}$ 限制在 $\boldsymbol{S}$ 的值域上, 或加入正则化.

□

注

(重构新增) 这两个估计量有明确的统计含义:$\boldsymbol{\mu}_{\mathrm{ML}}$ 就是样本均值 (定义 3.2),$\boldsymbol{\Sigma}_{\mathrm{ML}}$ 就是样本协方差 (未作无偏修正). 二者可以分两步计算, 因为 $\boldsymbol{\mu}_{\mathrm{ML}}$ 的解不依赖 $\boldsymbol{\Sigma}$, 这一“解耦”性质在练习 3.28中被进一步利用.

证明中“先忽略对称约束再验证解对称”是一个常用技巧: 若把 $\boldsymbol{\Sigma}$ 当作一般矩阵求导, 得到的驻点自动对称, 因此在对称子空间上也是驻点. 但正定性约束不能忽略:$\boldsymbol{S}$ 可能奇异, 此时 $\boldsymbol{\Sigma}_{\mathrm{ML}}$ 不可逆, 对数似然无定义 ($\ln|\boldsymbol{\Sigma}| \to -\infty$), 这也是高维 ($N < D$) 时最大似然失效的原因. 此外,$\boldsymbol{S}$ 用的是 $\boldsymbol{\mu}_{\mathrm{ML}}$ 而非真实 $\boldsymbol{\mu}$, 这一替换使估计量产生 $-\frac{1}{N}\boldsymbol{\Sigma}$ 量级的偏差 (见下一段).

如果我们估计真实分布下最大似然解的期望 (见练习 3.29), 则可以得到以下结果:

$$\mathbb{E}[\boldsymbol{\mu}_{\mathrm{ML}}] = \boldsymbol{\mu}, \tag{3.111}$$

$$\mathbb{E}[\boldsymbol{\Sigma}_{\mathrm{ML}}] = \frac{N-1}{N}\boldsymbol{\Sigma}. \tag{3.112}$$

可以看到, 均值的最大似然估计期望等于真实均值. 但是, 协方差的最大似然估计期望小于真实值, 因此它是有偏的. 通过定义一个不同的估计量 $\tilde{\boldsymbol{\Sigma}}$, 可以修正这一偏差, 其由下式给出:

$$\tilde{\boldsymbol{\Sigma}} = \frac{1}{N-1}\sum_{n=1}^{N}(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})^{\mathsf{T}}. \tag{3.113}$$

显然, 从(3.110)和(3.112)可以看出,$\tilde{\boldsymbol{\Sigma}}$ 的期望与 $\boldsymbol{\Sigma}$ 的期望相等.

命题 3.16 (最大似然估计量的期望与无偏修正)

设 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$ 独立同分布于 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$, 则(3.111)与(3.112)成立, 从而(3.113)定义的 $\tilde{\boldsymbol{\Sigma}}$ 是 $\boldsymbol{\Sigma}$ 的无偏估计, 即 $\mathbb{E}[\tilde{\boldsymbol{\Sigma}}] = \boldsymbol{\Sigma}$.

证明

(重构新增) 第一式: 由期望的线性性,$\mathbb{E}[\boldsymbol{\mu}_{\mathrm{ML}}] = \frac{1}{N}\sum_n \mathbb{E}[\boldsymbol{x}_n] = \boldsymbol{\mu}$.

第二式: 记 $\boldsymbol{z}_n = \boldsymbol{x}_n - \boldsymbol{\mu}$, 则 $\boldsymbol{z}_n$ 相互独立、均值为零、协方差为 $\boldsymbol{\Sigma}$, 且 $\boldsymbol{\mu}_{\mathrm{ML}} - \boldsymbol{\mu} = \frac{1}{N}\sum_n \boldsymbol{z}_n$. 于是

$$\boldsymbol{S} \triangleq \sum_{n=1}^{N}(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})(\boldsymbol{x}_n - \boldsymbol{\mu}_{\mathrm{ML}})^{\mathsf{T}} = \sum_{n=1}^{N}\left(\boldsymbol{z}_n - \frac{1}{N}\sum_m \boldsymbol{z}_m\right)\left(\boldsymbol{z}_n - \frac{1}{N}\sum_m \boldsymbol{z}_m\right)^{\mathsf{T}}$$

$$= \sum_n z_n z_n^{\mathsf{T}} - \frac{1}{N}\Big(\sum_n z_n\Big)\Big(\sum_m z_m\Big)^{\mathsf{T}}.$$

取期望, 利用 $\mathbb{E}[z_n z_n^{\mathsf{T}}] = \boldsymbol{\Sigma}$ 与独立性 ($n \neq m$ 时 $\mathbb{E}[z_n z_m^{\mathsf{T}}] = 0$), 得

$$\mathbb{E}[\boldsymbol{S}] = N\boldsymbol{\Sigma} - \frac{1}{N}\cdot N\boldsymbol{\Sigma} = (N-1)\boldsymbol{\Sigma}.$$

于是 $\mathbb{E}[\boldsymbol{\Sigma}_{\mathrm{ML}}] = \frac{1}{N}\mathbb{E}[\boldsymbol{S}] = \frac{N-1}{N}\boldsymbol{\Sigma}$, 即(3.112).

最后由 $\tilde{\boldsymbol{\Sigma}} = \frac{N}{N-1}\boldsymbol{\Sigma}_{\mathrm{ML}}$ 得 $\mathbb{E}[\tilde{\boldsymbol{\Sigma}}] = \frac{N}{N-1}\cdot\frac{N-1}{N}\boldsymbol{\Sigma} = \boldsymbol{\Sigma}$.

□

注

(重构新增) 偏差的来源是“用 $\boldsymbol{\mu}_{\mathrm{ML}}$ 代替真实 $\boldsymbol{\mu}$”: $\boldsymbol{\mu}_{\mathrm{ML}}$ 本身是数据的函数, 会向数据中心“靠近”, 从而系统性地压小平方离差, 压小的量恰好是 $\frac{1}{N}\boldsymbol{\Sigma}$. 除以 $N-1$ 而不是 N 正是为了补偿这一项, 这也是统计学中“样本方差分母为 $N-1$”的来源.

条件 $N \geqslant 2$ 是 $\tilde{\boldsymbol{\Sigma}}$ 有意义的前提, 否则分母为零. 注意无偏性只保证期望相等, 并不保证 $\tilde{\boldsymbol{\Sigma}}$ 一定正定 (当 $N-1 < D$ 时它必然奇异), 因此在 N 较小、D 较大时仍需正则化.

3.2.8 序贯估计

我们对最大似然解的讨论采用了批处理 (batch processing) 方法, 即一次性考虑整个训练集. 也可以使用序贯 (sequential) 方法, 该方法允许一次处理一个数据点, 然后将其丢弃. 这对于在线应用和大数据非常重要, 因为一次性批量处理所有数据在这种情况下是不可行的.

考虑(3.109)给出的最大似然估计的均值 $\boldsymbol{\mu}_{\mathrm{ML}}$, 当它建立在 N 个观测值之上时, 我们将它表示为 $\boldsymbol{\mu}_{\mathrm{ML}}^{(N)}$. 如果我们分解出最末尾的数据点 $\boldsymbol{x}_N$ 的贡献, 则可以得到

$$\begin{aligned}\boldsymbol{\mu}_{\mathrm{ML}}^{(N)} &= \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n \\ &= \frac{1}{N}\boldsymbol{x}_N + \frac{1}{N}\sum_{n=1}^{N-1}\boldsymbol{x}_n \\ &= \frac{1}{N}\boldsymbol{x}_N + \frac{N-1}{N}\boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)} \\ &= \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)} + \frac{1}{N}\big(\boldsymbol{x}_N - \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)}\big).\end{aligned} \tag{3.114}$$

命题 3.17 (样本均值的序贯更新)

设 $\boldsymbol{\mu}_{\mathrm{ML}}^{(N)} = \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n$, 则对 $N \geqslant 1$ 有递推式(3.114), 即 $\boldsymbol{\mu}_{\mathrm{ML}}^{(N)} = \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)} + \frac{1}{N}\big(\boldsymbol{x}_N - \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)}\big)$.

♠

证明

(重构新增) 由定义,

$$\boldsymbol{\mu}_{\mathrm{ML}}^{(N)} = \frac{1}{N}\Big(\sum_{n=1}^{N-1}\boldsymbol{x}_n + \boldsymbol{x}_N\Big) = \frac{N-1}{N}\cdot\frac{1}{N-1}\sum_{n=1}^{N-1}\boldsymbol{x}_n + \frac{1}{N}\boldsymbol{x}_N = \frac{N-1}{N}\boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)} + \frac{1}{N}\boldsymbol{x}_N,$$

再把它改写为

$$\frac{N-1}{N}\boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)} + \frac{1}{N}\boldsymbol{x}_N = \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)} + \frac{1}{N}\big(\boldsymbol{x}_N - \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)}\big),$$

即得(3.114). 初值可取 $\boldsymbol{\mu}_{\mathrm{ML}}^{(0)} = \boldsymbol{0}$(此时 $N=1$ 的递推给出 $\boldsymbol{\mu}_{\mathrm{ML}}^{(1)} = \boldsymbol{x}_1$).

□

注

(重构新增) 这个结果可以很容易地按如下方式解释. 在观测到 $N-1$ 个数据点后, 我们用 $\boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)}$ 来估计 $\boldsymbol{\mu}$. 而在观测到数据点 $\boldsymbol{x}_N$ 后, 我们可以通过将旧的估计值在“误差信号” $(\boldsymbol{x}_N - \boldsymbol{\mu}_{\mathrm{ML}}^{(N-1)})$ 的方向上移动一个较小的量来获得修订后的估计值, 这个较小的量与 $1/N$ 成正比. 注意, 随着 N 的增加, 来自后续数据点的贡献会越来越小.

这一递推的实用价值在于它只需保存当前的估计值与已见样本数, 而不必保存全部历史数据, 因此适用于在线学习与流式数据. 它与随机梯度下降的更新形式相似, 区别在于步长 $1/N$ 是确定性的、由样本数给出, 而不是人为设定的学习率. 注意 $N=1$ 时若取初值 $\mathbf{0}$, 更新式给出的正是 $\boldsymbol{x}_1$, 因此递推可以从空集启动.

3.2.9 Gauss 混合

尽管 Gauss 分布具有一些重要的分析特性, 但在用于对真实数据集建模时, 它存在很大的局限性. 考虑图 3.6(a) 中的示例, 它又称为 “Old Faithful” 数据集, 其中包括美国黄石国家公园老忠实喷泉喷发的 272 个观测值, 每个观测值都以分钟为单位给出喷发的持续时间 (横轴) 和下一次喷发的时间 (以分钟为单位)(纵轴). 我们可以看到该数据集形成了两个主要的簇, 并且使用简单的 Gauss 分布无法捕捉这种结构.

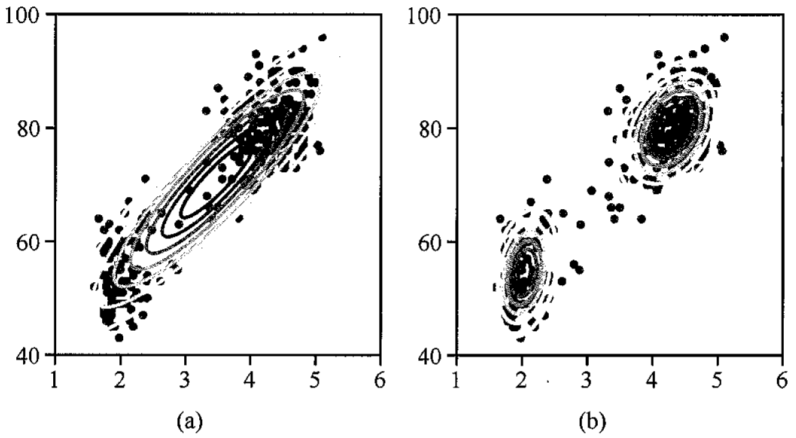


图 3.6: 老忠实喷泉数据的图示, 其中红色曲线是等概率密度的等值线.(a) 在单一 Gauss 分布上使用最大似然拟合数据得到的分布. 注意, 此分布无法捕捉数据中的两个簇, 并且确实将其大部分概率质量放置在了数据相对稀疏的团块间的中心区域.(b) 由两个 Gauss 分布的线性组合得到的分布, 其也由最大似然拟合, 这样可以更好地反映数据.

可以预期, 叠加两个 Gauss 分布将能够更好地表示该数据集的结构, 情况也确实如此, 如图 3.6(b) 所示. 这种叠加是通过对更基本的分布 (如 Gauss 分布等) 进行线性组合来实现的, 可以称作混合分布 (mixture distribution) 的概率模型 (参见第 15 章). 在本节中, 我们将考虑用 Gauss 分布来说明混合模型的框架. 更一般地, 混合模型可以包含其他分布的线性组合, 例如二元变量的 Bernoulli 分布的混合分布等. 在图 3.7中, 我们看到 Gauss 分布的线性组合可以产生非常复杂的密度. 通过使用足够数量的 Gauss 分布, 并调整它们的均值、协方差以及线性组合中的系数, 我们可以把几乎任何连续分布近似到任意准确度.

考虑形如下式的 K 个 Gauss 密度的叠加:

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \tag{3.115}$$

定义 3.12 (Gauss 混合分布)

设 $\pi_1, \cdots, \pi_K$ 满足 $\pi_k \geqslant 0$ 与 $\sum_{k=1}^{K} \pi_k = 1$,$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ 为第 k 个 Gauss 分量. 则称由(3.115)定义的密度为 Gauss 混合 (mixture of Gaussians). 其中每个 Gauss 密度 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ 称为混合的一个分量 (component),π_k

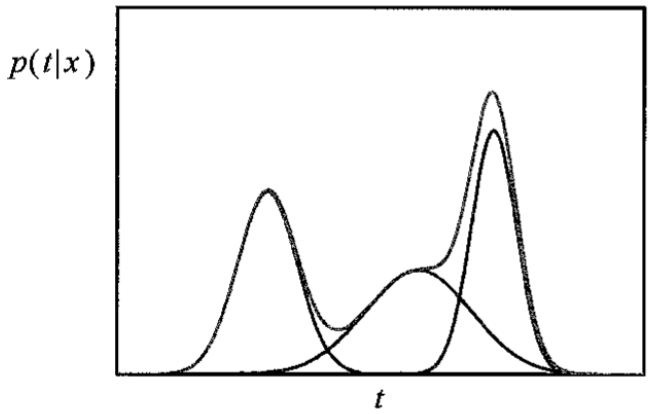


图 3.7: 一维 Gauss 混合分布, 蓝色曲线显示了 3 个 Gauss 分布 (每个 Gauss 分布按系数缩放), 红色曲线显示了它们的叠加效果.

称为混合系数 (mixing coefficient). ♣

注

(重构新增) 引入混合分布是为了突破单一 Gauss 分布“单峰”的限制: 把若干个 Gauss 密度加权相加, 就可以得到多峰的密度, 从而描述像 Old Faithful 那样含有多个簇的数据 (图 3.6). 每个分量负责一个簇的位置与形状, 混合系数负责各簇的相对权重, 因此模型在保持分量形式简单的同时获得了很强的表达能力.

条件 $\pi_k \geqslant 0$ 与 $\sum_k \pi_k = 1$ 保证 $p(\boldsymbol{x})$ 是合法的概率密度 (见命题 3.18), 各 $\boldsymbol{\Sigma}_k$ 正定保证每个分量良定义. 需要注意的是, 混合模型的参数个数为 $K-1+K(D+D(D+1)/2)$, 随 K 线性增长, 因此 K 的选择本身就是一个模型选择问题 (第 15 章将用最大似然与 EM 算法处理).

图 3.8显示了具有 3 个分量的二元 Gauss 混合分布的等值线和曲面图.

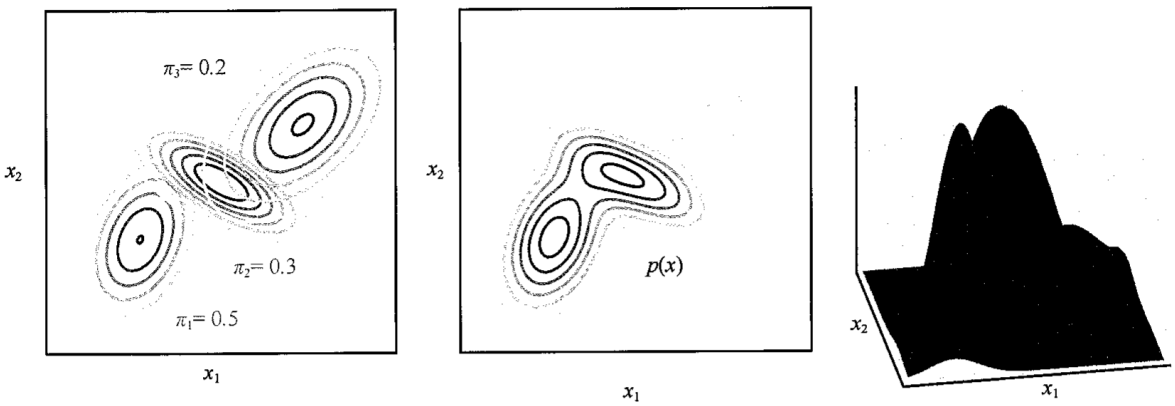


图 3.8: 3 个二元 Gauss 混合分布的示意图.(a) 每种混合情况的密度等值线,3 种情况分别表示为红色曲线、蓝色曲线和绿色曲线, 混合系数的值显示在每个分量的下方.(b) 混合分布的边缘概率密度 $p(\boldsymbol{x})$ 的等值线.(c) 分布 $p(\boldsymbol{x})$ 的曲面图.

(3.115)中的参数 π_k 称为混合系数 (mixing coefficient). 如果在(3.115)的两边对 $\boldsymbol{x}$ 进行积分, 并注意到 $p(\boldsymbol{x})$ 和单个 Gauss 分量都可以归一化, 便可得到

$$\sum_{k=1}^{K} \pi_k = 1. \tag{3.116}$$

此外, 给定 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \geqslant 0$, 则 $p(\boldsymbol{x}) \geqslant 0$ 的充分条件是对所有 k 都有 $\pi_k \geqslant 0$. 将其与条件(3.116)相结合, 可以得

到

$$0 \leqslant \pi_k \leqslant 1, \tag{3.117}$$

因此, 混合系数满足将其作为概率的要求.

命题 3.18 (混合系数的归一性与取值范围)

设 $p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ 为 Gauss 混合, 各分量归一化. 则 $\sum_k \pi_k = 1$; 若进一步要求 $p(\boldsymbol{x}) \geqslant 0$ 对一切 $\boldsymbol{x}$ 成立且各 $\pi_k \geqslant 0$, 则 $0 \leqslant \pi_k \leqslant 1$.

证明

(重构新增) 对(3.115)两边关于 $\boldsymbol{x}$ 积分, 左边得 1(p 是密度), 右边由积分的线性性得 $\sum_k \pi_k \int \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\,\mathrm{d}\boldsymbol{x} = \sum_k \pi_k$, 故(3.116)成立.

若各 $\pi_k \geqslant 0$, 则 $p(\boldsymbol{x})$ 是若干非负函数之和, 必非负; 反之若 $p \geqslant 0$ 对一切 $\boldsymbol{x}$ 成立, 取某个分量的密度远大于其余分量的点即可推出对应 $\pi_k \geqslant 0$(严格论证需分量之间线性无关,Gauss 分量一般满足). 结合 $\sum_k \pi_k = 1$ 与 $\pi_k \geqslant 0$ 得 $0 \leqslant \pi_k \leqslant 1$, 即(3.117).

□

注

(重构新增) 这两条结论说明混合系数确实可以当作“先验概率”来用: 它们非负且和为 1, 恰好是一个定义在离散变量 $k \in \{1, \cdots, K\}$ 上的概率分布. 这正是下一段把 $p(\boldsymbol{x})$ 写成 $\sum_k p(k)p(\boldsymbol{x} \mid k)$ 的依据.

注意“$\pi_k \geqslant 0$ 是 $p(\boldsymbol{x}) \geqslant 0$ 的充分条件”而非必要条件: 在一般情况下, 让某些 $\pi_k < 0$ 而整体仍非负是可能的(只要分量之间相互抵消), 但那样混合系数就不再具有概率含义. 因此(3.117)实际上是“把混合系数解释为概率”这一要求的结果, 而不是密度非负性的必要条件.

根据概率的加和法则与乘积法则, 边缘密度可以写为

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} p(k)\, p(\boldsymbol{x} \mid k), \tag{3.118}$$

这等价于(3.115), 其中可以将 $\pi_k = p(k)$ 视为选择第 k 个分量的先验概率, 并将密度 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = p(\boldsymbol{x} \mid k)$ 视为以 k 为条件时 $\boldsymbol{x}$ 的概率. 我们将在后面的章节中看到, 相应的后验概率 $p(k \mid \boldsymbol{x})$ 会起到重要作用, 因此也称责任. 根据 Bayes 定理, 这些可以由下式给出:

$$\gamma_k(\boldsymbol{x}) \equiv p(k \mid \boldsymbol{x}) = \frac{p(k)\, p(\boldsymbol{x} \mid k)}{\sum_l p(l)\, p(\boldsymbol{x} \mid l)} = \frac{\pi_k\, \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_l \pi_l\, \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)}. \tag{3.119}$$

定义 3.13 (责任)

在 Gauss 混合(3.115)中, 称由(3.119)定义的后验概率 $\gamma_k(\boldsymbol{x}) = p(k \mid \boldsymbol{x})$ 为分量 k 在点 $\boldsymbol{x}$ 处的责任 (responsibility).

注

(重构新增) “责任”这一名称来自它的解释:$\gamma_k(\boldsymbol{x})$ 度量的是“观测到 $\boldsymbol{x}$ 之后, 把这一点归因于第 k 个分量的可信程度”. 它满足 $\gamma_k(\boldsymbol{x}) \in [0, 1]$ 与 $\sum_k \gamma_k(\boldsymbol{x}) = 1$, 因此可以看作对数据点的一种“软分配” (soft assignment), 与 K 均值算法中的硬分配相对应 (第 15 章).

责任的定义只用到 Bayes 定理与全概率公式, 因此对任意混合分布都成立, 并不限于 Gauss 分量. 条件 $\sum_l \pi_l \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l) > 0$ 是分母非零的保证; 由于各分量密度严格为正, 只要某个 $\pi_l > 0$ 该条件就满足.

通过使用足够数量的 Gauss 分布, 并调整它们的均值、协方差以及线性组合中的系数, 我们可以把几乎任何连续分布近似到任意准确度. 下面我们将证明这种对混合分布的概率解释是非常强大的 (参见第 15 章).

命题 3.19 (混合密度的对数似然)

设 $X=\{\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N\}$ 为独立观测, 混合参数为 $\boldsymbol{\pi}=\{\pi_1,\cdots,\pi_K\}$、$\boldsymbol{\mu}=\{\boldsymbol{\mu}_1,\cdots,\boldsymbol{\mu}_K\}$、$\boldsymbol{\Sigma}=\{\boldsymbol{\Sigma}_1,\cdots,\boldsymbol{\Sigma}_K\}$, 则对数似然为

$$\ln p(X\mid\boldsymbol{\pi},\boldsymbol{\mu},\boldsymbol{\Sigma})=\sum_{n=1}^{N}\ln\left\{\sum_{k=1}^{K}\pi_k\,\mathcal{N}(\boldsymbol{x}_n\mid\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)\right\}. \tag{3.120}$$

♠

证明

(重构新增) 由独立性与(3.115),

$$p(X\mid\boldsymbol{\pi},\boldsymbol{\mu},\boldsymbol{\Sigma})=\prod_{n=1}^{N}p(\boldsymbol{x}_n)=\prod_{n=1}^{N}\sum_{k=1}^{K}\pi_k\mathcal{N}(\boldsymbol{x}_n\mid\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k),$$

两边取对数即得(3.120).

□

注

(重构新增) 与单个 Gauss 分布的情形相比,(3.120)的困难在于对数之内还有对 k 的求和, 因此对数不能逐项拆开, 参数的最大似然解不再具有闭式表达. 这正是混合模型必须借助迭代数值优化 (尤其是期望最大化算法) 的原因, 也是第 15 章引入潜变量视角的动机: 把“每个点属于哪个分量”视为缺失数据后, 完全数据的对数似然就能重新分解为逐分量之和.

条件“各观测独立”只用于把联合密度写成乘积; 混合结构本身 (求和号在乘积之内) 才是非线性的来源. 若 $K=1$,(3.120)退化为单个 Gauss 分布的对数似然(3.106).

3.3 周期变量

尽管 Gauss 分布本身具有重要的实际意义, 并且作为构建更复杂概率模型的基础也十分有用, 但在某些情况下, 它们并不适合作为连续变量的密度模型. 实际应用中存在的周期变量就属于这种情况.

特定地理位置的风向是周期变量的一个例子. 例如, 我们在测量多个地理位置的风向后, 希望使用参数分布来总结这些数据. 另一个例子是日历时间, 我们想建模那些可能在 24 小时或一年内具有周期性的量. 这样的量可以使用 $0\leqslant\theta<2\pi$ 范围内的角 (极) 坐标来方便地表示.

我们可以选择某个方向作为起始方向, 然后应用 Gauss 分布等常规分布来处理周期变量. 然而, 这种方法所产生的结果极其依赖于我们随意选择的起始方向. 例如, 假设有两个观测值 $\theta_1=1°$ 和 $\theta_2=359°$, 并且我们使用标准的单元 Gauss 分布对它们建模. 如果将起始方向设置为 0°, 那么该数据集的样本均值为 180°, 标准差为 179°; 而如果将起始方向设置为 180°, 那么该数据集的样本均值为 0°, 标准差为 1°. 显然, 我们需要一种专门用来处理周期变量的方法.

考虑估计周期变量 θ 的一组观测值 $\mathcal{D}=\{\theta_1,\cdots,\theta_N\}$ 的均值, 其中 θ 的单位为弧度. 简单计算均值 $(\theta_1+\cdots+\theta_N)/N$ 将与坐标系强相关. 为了找到均值的不变度量, 我们注意到观测值可以视为单位圆上的点, 因此可以用二维单位向量 $\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N$ 来描述它们, 其中对于 $\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N,\|\boldsymbol{x}_n\|=1$, 如图 3.9所示. 对向量 $\{\boldsymbol{x}_n\}$ 求平均值, 可得

$$\bar{\boldsymbol{x}}=\frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n,\qquad \boldsymbol{x}_n=(\cos\theta_n,\sin\theta_n)^{\mathsf{T}}. \tag{3.121}$$

定义 3.14 (圆周均值)

设 $\theta_1,\cdots,\theta_N$ 为周期变量的观测值,$\boldsymbol{x}_n=(\cos\theta_n,\sin\theta_n)^{\mathsf{T}}$. 称由(3.121)给出的 $\bar{\boldsymbol{x}}$ 的平均方向所对应的角度 $\bar{\theta}$ 为这组观测值的圆周均值 (circular mean), 其中 $\bar{\boldsymbol{x}}=\bar{r}(\cos\bar{\theta},\sin\bar{\theta})^{\mathsf{T}},\bar{r}\geqslant 0$.

♣

注

(重构新增) 引入圆周均值的目的是消除“起始方向”这一人为选择的影响: 把角度换成单位圆上的点之后再平均, 结果就与 0° 定在哪里无关. 这正是开头“1° 与 359°”例子的解药——按向量平均, 两个几乎重合的点会给出接近 0° 的均值, 而不是荒谬的 180°.

定义中的 $\bar{r}$ 称为平均合成长度 (resultant length), 它度量观测在圆周上的集中程度:$\bar{r} \approx 1$ 表示方向高度一致,$\bar{r} \approx 0$ 表示方向弥散. 只有当 $\bar{\boldsymbol{x}} \neq \mathbf{0}$ 时 $\bar{\theta}$ 才唯一确定; 若 $\bar{\boldsymbol{x}} = \mathbf{0}$(例如两个方向正好相反), 则均值方向无定义, 这是与实数均值的重要区别.

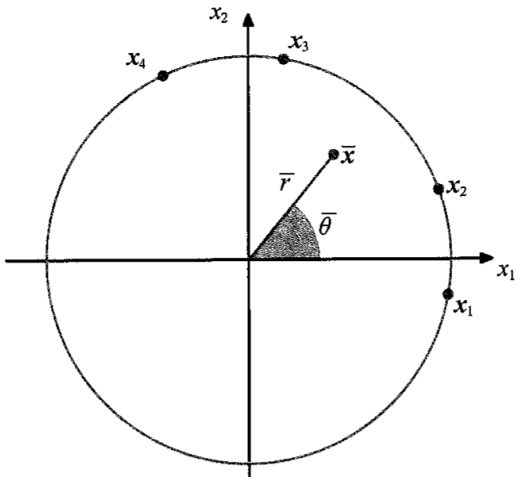


图 3.9: 将周期变量的值 θ_n 表示为位于单位圆上的二维向量 $\boldsymbol{x}_n$ 的示意图, 同时还展示了这些向量的平均值 $\bar{\boldsymbol{x}}$.

然后找到与平均值对应的角度 $\bar{\theta}$. 显然, 上述定义将确保均值的位置与角坐标的起始方向无关. 注意 $\bar{\boldsymbol{x}}$ 通常位于单位圆内. 观测值的笛卡儿坐标由 $\boldsymbol{x}_n = (\cos\theta_n, \sin\theta_n)$ 给出, 样本均值的笛卡儿坐标可以写成 $\bar{\boldsymbol{x}} = (\bar{r}\cos\bar{\theta}, \bar{r}\sin\bar{\theta})$. 代入(3.121)并使 x_1 和 x_2 分量相等, 则有

$$\bar{x}_1 = \bar{r}\cos\bar{\theta} = \frac{1}{N}\sum_{n=1}^{N}\cos\theta_n, \qquad \bar{x}_2 = \bar{r}\sin\bar{\theta} = \frac{1}{N}\sum_{n=1}^{N}\sin\theta_n. \tag{3.122}$$

对二者取比值, 并使用恒等式 $\tan\theta = \sin\theta/\cos\theta$,$\bar{\theta}$ 可由下式求解:

$$\bar{\theta} = \tan^{-1}\left(\frac{\sum_n \sin\theta_n}{\sum_n \cos\theta_n}\right). \tag{3.123}$$

命题 3.20 (圆周均值的显式表达)

设 $\bar{\boldsymbol{x}} \neq \mathbf{0}$, 则圆周均值 $\bar{\theta}$ 由(3.123)给出 (按象限取 $\tan^{-1}$ 的相应分支), 且它不依赖于角坐标起始方向的选择. ♠

证明

(重构新增) 由(3.122),$\bar{r}\cos\bar{\theta} = \frac{1}{N}\sum_n \cos\theta_n$、$\bar{r}\sin\bar{\theta} = \frac{1}{N}\sum_n \sin\theta_n$. 由于 $\bar{\boldsymbol{x}} \neq \mathbf{0}$ 意味着 $\bar{r} > 0$, 两式相除得

$$\tan\bar{\theta} = \frac{\bar{r}\sin\bar{\theta}}{\bar{r}\cos\bar{\theta}} = \frac{\sum_n \sin\theta_n}{\sum_n \cos\theta_n},$$

按象限取分支即得(3.123).

至于与起始方向无关: 若把起始方向旋转 α, 则每个观测变为 $\theta_n + \alpha$, 对应向量变为 $\boldsymbol{R}_\alpha \boldsymbol{x}_n$($\boldsymbol{R}_\alpha$ 为旋转矩阵), 于是 $\bar{\boldsymbol{x}}$ 变为 $\boldsymbol{R}_\alpha \bar{\boldsymbol{x}}$, 其方向角变为 $\bar{\theta} + \alpha$. 可见均值随坐标系同步旋转, 即相对位置不变.

□

注

(重构新增) (3.123)与普通的“算术平均”有本质区别: 它不是角度的线性平均, 而是先映射到单位圆、再平均、再取角度. 这一“映射—平均—反映射”的模式在周期数据分析中普遍适用 (例如周期数据的方差、相关系数都如此定义).

公式(3.123)只给出比值, 因此必须结合 $\sum_n \cos\theta_n$ 与 $\sum_n \sin\theta_n$ 的符号确定象限, 否则会相差 π. 当分母 $\sum_n \cos\theta_n = 0$ 时比值无定义, 此时 $\bar{\theta} = \pm\pi/2$, 应由 $\bar{x}_1 = 0$ 与 $\bar{x}_2$ 的符号直接判定.

读者很快就会看到这个结果是如何作为最大似然估计量而产生的.

按照惯例, 考虑分布 $p(\theta)$, 其具有 2π 周期. 任何定义在 θ 上的概率密度 $p(\theta)$ 都必须是非负的且积分等于 1, 同时还必须具有周期性. 因此,$p(\theta)$ 必须满足以下 3 个条件:

$$p(\theta) \geqslant 0, \tag{3.124}$$

$$\int_0^{2\pi} p(\theta)\,\mathrm{d}\theta = 1, \tag{3.125}$$

$$p(\theta + 2\pi) = p(\theta). \tag{3.126}$$

定义 3.15 (周期分布)

设 $p(\theta)$ 为定义在实轴上的非负函数, 若它满足(3.124)至(3.126), 则称 $p(\theta)$ 为周期分布 (或圆周分布) 的密度. ♣

注

(重构新增) 这三个条件把“概率密度”与“周期性”两件事同时固定下来: 前两条是密度的通用要求, 第三条是周期变量特有的要求. 有了第三条,p 就可以看作定义在单位圆上的函数, 因此“把实轴上的分布卷到圆周上”这种构造才有意义.

注意积分区间取 $[0, 2\pi)$ 即可, 周期性保证了在任意长度为 2π 的区间上积分都相同. 由(3.126)还可推出, 对任何整数 M 有 $p(\theta + 2M\pi) = p(\theta)$, 这说明圆周上的每一点都对应实轴上无穷多个等价点, 也解释了为什么周期分布不能直接沿用实轴上的矩 (平均值、方差) 概念.

从(3.126)可知, 对于任何整数 M,$p(\theta + 2M\pi) = p(\theta)$.

我们可以很容易地获得满足上述 3 个条件的类 Gauss 分布: 考虑二元变量 $\boldsymbol{x} = (x_1, x_2)$ 上的 Gauss 分布, 其均值为 $\boldsymbol{\mu} = (\mu_1, \mu_2)$, 协方差矩阵为 $\boldsymbol{\Sigma} = \sigma^2 \boldsymbol{I}$, 其中 $\boldsymbol{I}$ 是 2×2 的单位矩阵, 故有

$$p(x_1, x_2) = \frac{1}{2\pi\sigma^2} \exp\left\{-\frac{(x_1 - \mu_1)^2 + (x_2 - \mu_2)^2}{2\sigma^2}\right\}. \tag{3.127}$$

常数 $p(\boldsymbol{x})$ 的等值线为圆形, 如图 3.10所示.

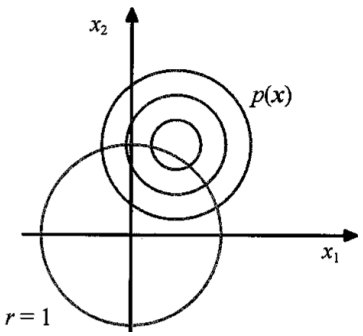


图 3.10: von Mises 分布可通过(3.127)所示的二维 Gauss 分布导出. 图中蓝色的圆是其密度等值线, 红色的单位圆是其条件.

考虑沿着固定半径的圆来取这个分布上的值. 通过这种构造方法, 尽管没有被归一化, 但这个分布仍将是周期

性的. 我们可以通过从笛卡儿坐标 (x_1, x_2) 转换到极坐标 (r, θ) 来确定该分布的形式:

$$x_1 = r\cos\theta, \qquad x_2 = r\sin\theta. \tag{3.128}$$

同时将均值 $\boldsymbol{\mu}$ 也映射到极坐标:

$$\mu_1 = r_0\cos\theta_0, \qquad \mu_2 = r_0\sin\theta_0. \tag{3.129}$$

接下来, 我们设单位圆 $r = 1$, 将这些变换代入二维 Gauss 分布(3.127), 以研究该分布与 θ 的依赖关系. 关注 Gauss 分布中的指数部分, 则有

$$\begin{aligned} &-\frac{1}{2\sigma^2}\left\{(r\cos\theta - r_0\cos\theta_0)^2 + (r\sin\theta - r_0\sin\theta_0)^2\right\} \\ &= -\frac{1}{2\sigma^2}\left\{1 + r_0^2 - 2r_0\cos\theta\cos\theta_0 - 2r_0\sin\theta\sin\theta_0\right\} \\ &= \frac{r_0}{\sigma^2}\cos(\theta - \theta_0) + \text{const}, \end{aligned} \tag{3.130}$$

其中 “const” 表示独立于 θ 的常数项. 此处的推导使用了以下三角恒等式:

$$\cos^2 A + \sin^2 A = 1, \tag{3.131}$$

$$\cos A\cos B + \sin A\sin B = \cos(A - B). \tag{3.132}$$

如果定义 $m = r_0/\sigma^2$, 则可以得到 $p(\theta)$ 沿单位圆 $r = 1$ 这个分布的最终表达式, 形式为

$$p(\theta \mid \theta_0, m) = \frac{1}{2\pi I_0(m)}\exp\{m\cos(\theta - \theta_0)\}. \tag{3.133}$$

定理 3.1 (von Mises 分布)

设 $p(\theta)$ 为单位圆上的周期密度, 它由二维各向同性 Gauss 分布(3.127)限制在 $r = 1$ 上得到. 则 $p(\theta)$ 具有(3.133)的形式, 其中 θ_0 为均值方向,$m = r_0/\sigma^2 > 0$, 归一化系数为

$$I_0(m) = \frac{1}{2\pi}\int_0^{2\pi}\exp\{m\cos\theta\}\,\mathrm{d}\theta. \tag{3.134}$$

♡

证明

(重构新增) 把(3.128)与(3.129)代入(3.127)并取 $r = 1$, 得

$$p(\theta) \propto \exp\left\{-\frac{1}{2\sigma^2}\left[(r\cos\theta - r_0\cos\theta_0)^2 + (r\sin\theta - r_0\sin\theta_0)^2\right]\right\}_{r=1}.$$

展开括号内的平方和:

$$\begin{aligned} &\cos^2\theta - 2r_0\cos\theta\cos\theta_0 + r_0^2\cos^2\theta_0 + \sin^2\theta - 2r_0\sin\theta\sin\theta_0 + r_0^2\sin^2\theta_0 \\ &= 1 + r_0^2 - 2r_0(\cos\theta\cos\theta_0 + \sin\theta\sin\theta_0) = 1 + r_0^2 - 2r_0\cos(\theta - \theta_0), \end{aligned}$$

其中用到(3.131)与(3.132). 于是

$$p(\theta) \propto \exp\left\{\frac{r_0}{\sigma^2}\cos(\theta - \theta_0)\right\}\exp\left\{-\frac{1 + r_0^2}{2\sigma^2}\right\} \propto \exp\{m\cos(\theta - \theta_0)\},$$

第二步把与 θ 无关的常数并入比例常数, 并令 $m = r_0/\sigma^2$. 这就是(3.130).

最后确定归一化常数. 由于 $\cos(\theta - \theta_0)$ 关于 θ 以 2π 为周期, 故 $\int_0^{2\pi}\exp\{m\cos(\theta - \theta_0)\}\,\mathrm{d}\theta = \int_0^{2\pi}\exp\{m\cos\theta\}\,\mathrm{d}\theta = 2\pi I_0(m)$, 其中 $I_0(m)$ 由(3.134)定义. 因此归一化后的密度为

$$p(\theta \mid \theta_0, m) = \frac{1}{2\pi I_0(m)}\exp\{m\cos(\theta - \theta_0)\},$$

即(3.133). 由于被积函数严格为正,$I_0(m) > 0$, 归一化常数良定义.

□

注

(重构新增) 这一定理说明 von Mises 分布不是凭空构造的, 而是“二维各向同性 Gauss 分布限制在单位圆上”的自然结果:Gauss 分布在平面上是单峰、等向的, 切到单位圆上就得到圆周上的单峰分布, 峰值方向由 $\boldsymbol{\mu}$ 的极角 θ_0 决定, 峰的尖锐程度由 $m = r_0/\sigma^2$ 决定.

推导中有两个关键步骤: 一是把 r 固定为 1(只关心单位圆上的值), 二是把与 θ 无关的因子并入归一化常数. 注意这里得到的密度是关于 θ 的, 而不是关于弧长的“密度乘以 r”; 取 $r = 1$ 恰好使二者一致. 若取其他半径, 得到的函数仍是周期的, 但归一化常数会依赖 r.

这称为 von Mises 分布或圆形正态 (circular normal) 分布. 这里的参数 θ_0 对应于分布的均值,m 则称为聚焦 (concentration) 参数, 类似于 Gauss 分布的逆方差 (即精度).(3.133)中的归一化系数用 $I_0(m)$ 表示, 它是第一类零阶修正 Bessel 函数 (modified Bessel function), 其定义即(3.134).

定义 3.16 (von Mises 分布)

设 $\theta_0 \in [0, 2\pi)$、$m > 0$. 称由(3.133)定义的周期密度为参数为 θ_0 与 m 的 von Mises 分布 (又称圆形正态分布), 其中 $I_0(m)$ 为第一类零阶修正 Bessel 函数(3.134),θ_0 为均值方向,m 为聚焦参数. ♣

注

(重构新增) von Mises 分布是圆周上的“Gauss 分布”:θ_0 控制峰值位置,m 控制峰的宽度.m 越大,$\exp\{m\cos(\theta-\theta_0)\}$ 在 $\theta = \theta_0$ 附近越尖锐, 因此 m 确实扮演了“精度”(方差的倒数) 的角色; 当 $m \to 0$ 时密度趋于常数 $1/(2\pi)$, 即圆周上的均匀分布.

条件 $m > 0$ 保证分布是单峰且峰在 θ_0 处 (练习 3.33);$m = 0$ 时退化为均匀分布,θ_0 不再可辨识.θ_0 的取值范围按模 2π 理解, 因为 $\cos(\theta - \theta_0)$ 只依赖 $\theta - \theta_0$ 的模 2π 值. 归一化常数 $I_0(m) > 0$ 对一切 m 成立, 因此分布对任意 $m \geqslant 0$ 都良定义.

当 m 较大时, 该分布近似为 Gauss 分布 (见练习 3.31).von Mises 分布如图 3.11所示, 函数 $I_0(m)$ 如图 3.12所示.

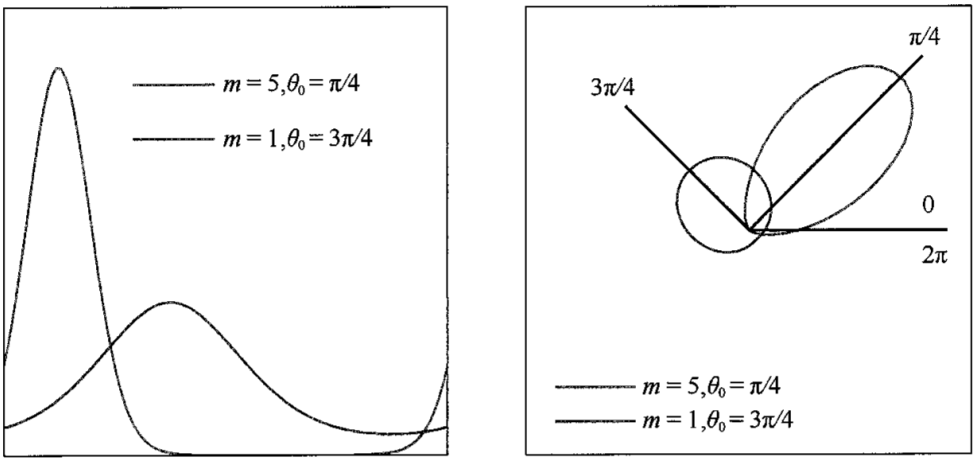


图 3.11: 两个不同参数值下的 von Mises 分布. 左图在笛卡儿坐标下, 右图在相应的极坐标下.

考虑 von Mises 分布中参数 θ_0 和 m 的最大似然估计量. 对数似然函数由下式给出:

$$\ln p(\mathcal{D} \mid \theta_0, m) = -N\ln(2\pi) - N\ln I_0(m) + m\sum_{n=1}^{N}\cos(\theta_n - \theta_0). \tag{3.135}$$

令(3.135)对 θ_0 的导数为零, 可得

$$\sum_{n=1}^{N}\sin(\theta_n - \theta_0) = 0. \tag{3.136}$$

为了求解 θ_0, 使用如下三角恒等式:

$$\sin(A - B) = \cos B\sin A - \cos A\sin B. \tag{3.137}$$

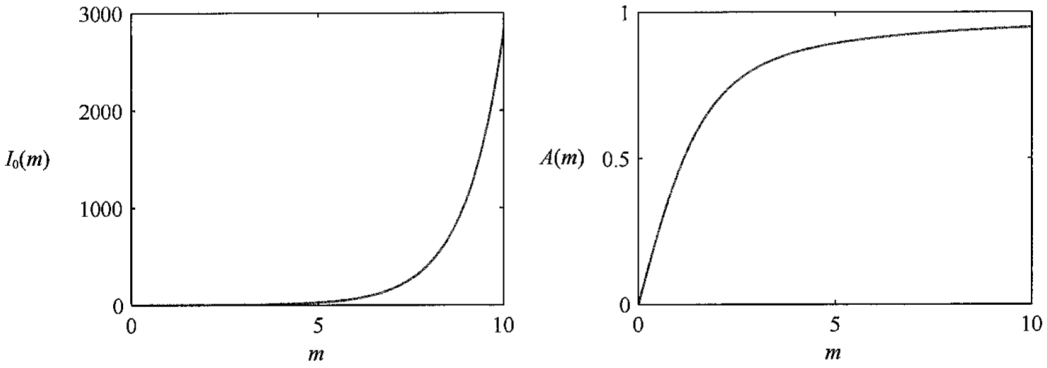


图 3.12: Bessel 函数 $I_0(m)$((3.134)) 和函数 $A(m)$((3.140)) 的图示.

由此可得 (见练习 3.32)

$$\theta_0^{\mathrm{ML}} = \tan^{-1}\left(\frac{\sum_n \sin\theta_n}{\sum_n \cos\theta_n}\right). \tag{3.138}$$

可以发现, 这与之前二维笛卡儿空间中观测值均值的结果 [(3.123)] 一致.

类似地, 关于 m 最大化(3.135), 并利用 $I_0'(m) = I_1(m)$(Abramowitz and Stegun, 1965), 可得

$$A(m_{\mathrm{ML}}) = \frac{1}{N}\sum_{n=1}^{N}\cos(\theta_n - \theta_0^{\mathrm{ML}}). \tag{3.139}$$

此处已经代入了 θ_0 的最大似然解 (注意我们现在正在对 θ 和 m 进行联合优化), 并定义了

$$A(m) = \frac{I_1(m)}{I_0(m)}. \tag{3.140}$$

函数 $A(m)$ 如图 3.12所示. 利用(3.132)中的三角恒等式, 可以将(3.139)写成如下形式:

$$A(m_{\mathrm{ML}}) = \left(\frac{1}{N}\sum_{n=1}^{N}\cos\theta_n\right)\cos\theta_0^{\mathrm{ML}} + \left(\frac{1}{N}\sum_{n=1}^{N}\sin\theta_n\right)\sin\theta_0^{\mathrm{ML}}. \tag{3.141}$$

命题 3.21 (von Mises 分布的最大似然估计)

设 $\theta_1, \cdots, \theta_N$ 为来自 von Mises 分布的独立观测,$N \geqslant 1$. 则 θ_0 的最大似然估计由(3.138)给出,m 的最大似然估计满足(3.139),(二者合起来要求分母不为零).

♠

证明

(重构新增) 关于 θ_0: 对(3.135)求导, 注意 $\dfrac{\mathrm{d}}{\mathrm{d}\theta_0}\cos(\theta_n - \theta_0) = \sin(\theta_n - \theta_0)$, 得 $\partial \ln p/\partial\theta_0 = m\sum_n \sin(\theta_n - \theta_0)$. 令其为零 ($m > 0$) 即得(3.136).

用(3.137)展开(3.136):

$$\sum_n \sin(\theta_n - \theta_0) = \cos\theta_0 \sum_n \sin\theta_n - \sin\theta_0 \sum_n \cos\theta_n = 0,$$

于是 $\tan\theta_0 = \dfrac{\sum_n \sin\theta_n}{\sum_n \cos\theta_n}$, 按象限取分支即得(3.138). 这与(3.123)完全一致.

关于 m: 对(3.135)求导, 得

$$\frac{\partial \ln p}{\partial m} = -N\frac{I_0'(m)}{I_0(m)} + \sum_n \cos(\theta_n - \theta_0).$$

令其为零并用 $I_0'(m) = I_1(m)$ 与 $A(m) = I_1(m)/I_0(m)$((3.140)), 得 $A(m_{\mathrm{ML}}) = \dfrac{1}{N}\sum_n \cos(\theta_n - \theta_0^{\mathrm{ML}})$, 即(3.139).

最后用(3.132)把 $\cos(\theta_n - \theta_0^{\mathrm{ML}})$ 展开即得(3.141). 注意该式右端只依赖 θ_0^{ML} 与数据, 而 $A(m)$ 在 $(0, \infty)$ 上严格单调递增且值域为 $(0, 1)$, 故当右端落在 $(0, 1)$ 内时 m_{ML} 唯一存在.

□

注

(重构新增) 这里出现了一个漂亮的巧合:von Mises 分布的均值方向的最大似然估计, 与“把观测当作单位圆上的向量再求平均”的圆周均值(3.123)完全相同. 这解释了前面那句“读者很快就会看到这个结果是如何作为最大似然估计量而产生的”: 圆周均值不是权宜之计, 而是 von Mises 模型下的最大似然解.

m 的估计则没有闭式解:(3.139)只给出 $A(m_{\mathrm{ML}})$ 的值, 需要数值反演 A 才能得到 m_{ML}(这正是图 3.12右图的作用). 条件 $N \geqslant 1$ 与分母 $\sum_n \cos\theta_n$、$\sum_n \sin\theta_n$ 不同时为零, 保证 θ_0^{ML} 有定义; 若数据在圆周上完全均匀, 则 $\sum_n \cos\theta_n = \sum_n \sin\theta_n = 0$, 均值方向不可辨识, 此时 $m_{\mathrm{ML}} = 0$ 对应均匀分布.

(3.141)的右侧部分很容易计算, 而函数 $A(m)$ 可以通过数值反演得到.von Mises 分布的一个局限性是, 它是单峰的. 我们可以通过构造一个包含多个 von Mises 分布的混合分布来获得一个更为灵活的周期变量建模框架, 以处理多峰数据.

为了讨论的完整性, 下面简要提及一些构造周期分布的替代技术. 最简单的方法是使用观测值的直方图, 其中角坐标被划分为固定的区间. 这种方法的优点是简单且灵活, 但也存在显著的局限性, 这一点读者在后文对直方图方法的详细讨论中将会看到 (参见小节 3.5.1). 另一种方法类似于 von Mises 分布: 从欧氏空间上的 Gauss 分布开始, 但现在边缘化到单位圆上, 而不是条件化 (Mardia and Jupp, 2000). 然而, 这会导致分布形式更加复杂, 因此我们不再做进一步讨论. 最后, 实轴上的任何有效分布 (如 Gauss 分布) 都可以通过将宽度为 2π 的连续区间映射到周期变量 $(0, 2\pi)$ 上, 成为周期分布, 这相当于将实轴“包裹”到单位圆上 (环绕分布). 同样, 由此得到的分布比 von Mises 分布更复杂.

注

(重构新增) 这段列举了三种替代构造, 它们分别对应三种“把直线上的工具搬到圆周上”的思路: 直方图法直接对角度分箱, 不做参数假设; 边缘化法把平面 Gauss 分布的 r 积分掉, 得到圆周上的边缘密度 (其形式不再是 von Mises); 包裹法把实轴按 2π 取模卷到圆上, 得到“环绕”分布 (如包裹正态分布).

选择哪种构造取决于问题: 若只需快速可视化, 直方图足够; 若需要解析上可处理的单峰模型,von Mises 最方便; 若数据可能多峰, 则应使用 von Mises 混合模型. 所有这些构造的共同点是它们都满足(3.124)至(3.126), 区别只在于密度的具体形式与计算复杂度.

3.4 指数族分布

到目前为止, 本章所研究的概率分布 (混合模型除外) 都是指数族 (exponential family) 分布 (Duda and Hart, 1973; Bernardo and Smith, 1994) 这一大类里的具体例子. 指数族分布的成员具有许多共同的重要性质, 对这些性质的一般性讨论可以带来不少启发.

定义 3.17 (指数族分布)

称一族由参数 $\boldsymbol{\eta}$ 控制的分布属于指数族 (exponential family), 如果其中的分布都可以写成如下形式:

$$p(\boldsymbol{x} \mid \boldsymbol{\eta}) = h(\boldsymbol{x})\, g(\boldsymbol{\eta}) \exp\{\boldsymbol{\eta}^{\mathsf{T}} \boldsymbol{u}(\boldsymbol{x})\}, \tag{3.142}$$

其中 $\boldsymbol{x}$ 可以是标量或矢量, 也可以是离散的或连续的;$\boldsymbol{\eta}$ 称为分布的自然参数 (natural parameters),$\boldsymbol{u}(\boldsymbol{x})$ 是 $\boldsymbol{x}$ 的某个函数,$h(\boldsymbol{x})$ 是取非负值的基底函数. 函数 $g(\boldsymbol{\eta})$ 是确保分布被归一化的系数, 因此满足

$$g(\boldsymbol{\eta}) \int h(\boldsymbol{x}) \exp\{\boldsymbol{\eta}^{\mathsf{T}} \boldsymbol{u}(\boldsymbol{x})\}\, \mathrm{d}\boldsymbol{x} = 1, \tag{3.143}$$

其中 $\boldsymbol{x}$ 若为离散变量, 则积分改为求和. ♣

注

(重构新增) 引入指数族的意义在于“提取公因子”: 本章前面遇到的 Bernoulli、二项、多项 (类别) 与 Gauss 分布, 尽管形式各异, 却都可以写成(3.142)这一统一形式. 一旦写成这种形式, 许多性质 (矩、最大似然、充分统计量、共轭先验) 就可以一次性对整族证明, 而不必逐个分布重复推导.

三个组成部分各司其职:$h(\boldsymbol{x})$ 只依赖数据、与参数无关;$g(\boldsymbol{\eta})$ 只依赖参数、由(3.143)确定;$\exp\{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}(\boldsymbol{x})\}$ 是数据与参数的“交互项”, 它必须是 $\boldsymbol{\eta}$ 与 $\boldsymbol{u}(\boldsymbol{x})$ 的内积这一特殊形式. 条件(3.143)要求(3.142)右端的积分有限且可积, 这实际上限制了 $\boldsymbol{\eta}$ 的取值范围 (自然参数空间); 对某些 $\boldsymbol{\eta}$, 积分发散,$g(\boldsymbol{\eta})$ 无定义, 该参数值就不被允许.

下面我们从本章前面介绍的分布示例入手, 说明它们确实是指数族分布的成员. 首先考虑 Bernoulli 分布:

$$p(x \mid \mu) = \mathrm{Bern}(x \mid \mu) = \mu^{x}(1-\mu)^{1-x}. \tag{3.144}$$

将(3.144)的右侧表示为对数的指数, 可得

$$\begin{aligned} p(x \mid \mu) &= \exp\big\{x \ln \mu + (1-x)\ln(1-\mu)\big\} \\ &= (1-\mu)\exp\left\{\ln\left(\frac{\mu}{1-\mu}\right)x\right\}. \end{aligned} \tag{3.145}$$

与(3.142)比较后, 我们可以确定

$$\eta = \ln\left(\frac{\mu}{1-\mu}\right). \tag{3.146}$$

对 μ 进行求解可得 $\mu = \sigma(\eta)$, 其中

$$\sigma(\eta) = \frac{1}{1+\exp(-\eta)}. \tag{3.147}$$

定义 3.18 (sigmoid 函数)

称由(3.147)定义的函数 $\sigma : \mathbb{R} \to (0,1)$ 为 sigmoid 函数 (又称逻辑斯谛函数).

注

(重构新增) sigmoid 函数在这里充当“自然参数到均值参数”的桥梁: 它把无约束的实数 $\eta \in \mathbb{R}$ 一一映射到概率 $\mu \in (0,1)$, 其反函数正是(3.146)的 logit 变换. 这一对互逆变换是指数族理论中的典型结构——自然参数可以取遍整个实数轴, 而均值参数被限制在概率单纯形内.

σ 严格单调递增、值域为 $(0,1)$、且满足 $\sigma(-\eta) = 1-\sigma(\eta)$, 最后这一对称性在下一段把 Bernoulli 分布写成标准形式时会被直接用到. 注意 σ 是光滑函数, 因此 $\eta \mapsto \mu$ 的变换不会破坏可微性, 这为后面用梯度方法求最大似然提供了便利.

它又称为 sigmoid 函数. 因此, 我们可以使用以下标准表达式来描述 Bernoulli 分布:

$$p(x \mid \eta) = \sigma(-\eta)\exp(\eta x). \tag{3.148}$$

这里使用了等式 $1-\sigma(\eta) = \sigma(-\eta)$, 其易由(3.147)证得.

命题 3.22 (Bernoulli 分布的指数族形式)

Bernoulli 分布(3.144)可以写成(3.142)的标准形式, 其中

$$u(x) = x, \qquad h(x) = 1, \qquad g(\eta) = \sigma(-\eta), \tag{3.149}$$

而自然参数 η 与均值参数 μ 由(3.146)相联系.

证明

(重构新增) 由(3.145),$p(x\mid\mu)=(1-\mu)\exp\{\eta x\}$, 其中 $\eta=\ln\dfrac{\mu}{1-\mu}$. 把 $\mu=\sigma(\eta)$ 代入系数 $1-\mu$, 利用 $1-\sigma(\eta)=\sigma(-\eta)$ 得 $1-\mu=\sigma(-\eta)$, 于是

$$p(x\mid\eta)=\sigma(-\eta)\exp(\eta x),$$

即(3.148). 与(3.142)逐项对照:$h(x)=1$、$u(x)=x$、$g(\eta)=\sigma(-\eta)$, 即(3.149).

最后验证归一化条件(3.143):x 只取 0, 1, 故

$$\sum_{x=0}^{1}h(x)\exp\{\eta u(x)\}=1+\exp(\eta),$$

而 $g(\eta)=\sigma(-\eta)=\dfrac{1}{1+\exp(\eta)}$, 两者相乘为 1, 条件满足.

□

注

(重构新增) 这条结论说明 Bernoulli 分布之所以“简单”, 是因为它属于指数族: 整个分布只通过 ηx 这一项依赖数据, 因此数据的作用被压缩为“x 的线性函数”. 这也解释了为什么它的最大似然解只依赖 $\sum_n x_n$——指数族的一般理论 (命题 3.27) 会给出同一个结论.

注意均值参数 μ 与自然参数 η 的对应不是线性的:$\eta=\ln\dfrac{\mu}{1-\mu}$ 在 $\mu\to0$ 或 $\mu\to1$ 时发散, 因此 $\mu\in(0,1)$ 对应 $\eta\in\mathbb{R}$, 自然参数空间是整条实轴. 若允许 $\mu=0$ 或 $\mu=1$, 则 $\eta=\pm\infty$, 不在自然参数空间内, 这提示退化分布处于指数族的边界.

接下来考虑多项分布. 对于单个观测 $\boldsymbol{x}$, 其形式为

$$p(\boldsymbol{x}\mid\boldsymbol{\mu})=\prod_{k=1}^{M}\mu_k^{x_k}=\exp\left\{\sum_{k=1}^{M}x_k\ln\mu_k\right\},\tag{3.150}$$

其中 $\boldsymbol{x}=(x_1,\cdots,x_M)^{\mathsf{T}}$.(3.150)同样可以写成标准表达式:

$$p(\boldsymbol{x}\mid\boldsymbol{\eta})=\exp\left(\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{x}\right),\tag{3.151}$$

其中 $\eta_k=\ln\mu_k$, 并且我们定义 $\boldsymbol{\eta}=(\eta_1,\cdots,\eta_M)^{\mathsf{T}}$. 与(3.142)比较可得

$$\boldsymbol{u}(\boldsymbol{x})=\boldsymbol{x},\qquad h(\boldsymbol{x})=1,\qquad g(\boldsymbol{\eta})=1.\tag{3.152}$$

注意, 参数 η_k 不是独立的, 因为参数 μ_k 受到下式的约束:

$$\sum_{k=1}^{M}\mu_k=1.\tag{3.153}$$

因此, 对于给定的任意 $M-1$ 个参数 μ_k, 剩余参数的值被固定. 在某些情况下, 通过仅用 $M-1$ 个参数表达分布, 可以方便地去除这一约束. 这可以通过使用关系式(3.153)将 μ_M 用剩余的 $\{\mu_k\}$(其中 $k=1,\cdots,M-1$) 表示, 从而留下 $M-1$ 个参数来实现. 请注意, 这些剩余参数仍然受到下式的约束:

$$0\leqslant\mu_k<1,\qquad\sum_{k=1}^{M-1}\mu_k<1.\tag{3.154}$$

利用约束(3.153), 多项分布变为

$$\begin{aligned}&\exp\left\{\sum_{k=1}^{M}x_k\ln\mu_k\right\}\\&=\exp\left\{\sum_{k=1}^{M-1}x_k\ln\mu_k+\left(1-\sum_{k=1}^{M-1}x_k\right)\ln\left(1-\sum_{k=1}^{M-1}\mu_k\right)\right\}\\&=\exp\left\{\sum_{k=1}^{M-1}x_k\ln\left(\frac{\mu_k}{1-\sum_{j=1}^{M-1}\mu_j}\right)+\ln\left(1-\sum_{k=1}^{M-1}\mu_k\right)\right\}.\end{aligned}\tag{3.155}$$

现在我们确定了

$$\ln\left(\frac{\mu_k}{1-\sum_j \mu_j}\right)=\eta_k. \tag{3.156}$$

可以通过首先对(3.156)两边的 k 求和, 然后重新排列和反向代入来求解 k:

$$\mu_k=\frac{\exp(\eta_k)}{1+\sum_j \exp(\eta_j)}. \tag{3.157}$$

定义 3.19 (softmax 函数)

称由(3.157)给出的映射 $\boldsymbol{\eta}\mapsto\boldsymbol{\mu}$ 为 softmax 函数或归一化指数 (normalized exponential), 它把 $\mathbb{R}^{M-1}$ 中的自然参数映射为概率向量 $\boldsymbol{\mu}$.

注

(重构新增) softmax 是 sigmoid 在多类别情形下的推广:$M=2$ 时(3.157)退化为 $\mu_1=\sigma(\eta_1)$、$\mu_2=\sigma(-\eta_1)$, 正是 Bernoulli 的 sigmoid 形式. 它的作用是“把任意实数打分变成概率”: 先取指数保证非负, 再除以总和保证归一.

由于 μ_k 只由 $M-1$ 个 η_k 决定 (第 M 个由归一化补足),softmax 的参数是冗余的: 把所有 η_k 同时加上常数 c,μ_k 不变. 这一“平移不变性”是不使用全部 M 个自然参数的原因, 也解释了为什么多项分布的指数族表示只保留 $M-1$ 个自然参数.

因此在这种表示中, 多项分布将采用以下形式:

$$p(\boldsymbol{x}\mid\boldsymbol{\eta})=\left(1+\sum_{k=1}^{M-1}\exp(\eta_k)\right)^{-1}\exp\left(\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{x}\right). \tag{3.158}$$

这是带有参数向量 $\boldsymbol{\eta}=(\eta_1,\cdots,\eta_{M-1})^{\mathsf{T}}$ 的指数族分布的标准形式. 与(3.142)比较可得

$$\boldsymbol{u}(\boldsymbol{x})=\boldsymbol{x},\qquad h(\boldsymbol{x})=1,\qquad g(\boldsymbol{\eta})=\left(1+\sum_{k=1}^{M-1}\exp(\eta_k)\right)^{-1}. \tag{3.159}$$

命题 3.23 (多项 (类别) 分布的指数族形式)

多项分布(3.150)可以写成(3.142)的标准形式. 若保留全部 M 个自然参数 $\eta_k=\ln\mu_k$, 则 $g(\boldsymbol{\eta})=1$、$\boldsymbol{u}(\boldsymbol{x})=\boldsymbol{x}$、$h(\boldsymbol{x})=1$; 若消去归一化约束(3.153)而只保留 $M-1$ 个自然参数, 则形式为(3.158), 其中 g 由(3.159)给出.

证明

(重构新增) 第一种写法: 由(3.150),$p(\boldsymbol{x}\mid\boldsymbol{\mu})=\exp\{\sum_k x_k\ln\mu_k\}$. 取 $\eta_k=\ln\mu_k$, 则 $p=\exp\{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{x}\}$, 即(3.151), 故 $\boldsymbol{u}(\boldsymbol{x})=\boldsymbol{x}$、$h(\boldsymbol{x})=1$、$g(\boldsymbol{\eta})=1$, 即(3.152). 注意此时 $\boldsymbol{\eta}$ 落在约束 $\sum_k\exp(\eta_k)=1$ 上, 并非自由参数.

第二种写法: 由(3.153),$\mu_M=1-\sum_{k=1}^{M-1}\mu_k$, 且 $\sum_k x_k=1$, 故(3.155)成立. 把(3.156)代入(3.155), 得

$$p(\boldsymbol{x}\mid\boldsymbol{\eta})=\Big(1-\sum_{k=1}^{M-1}\mu_k\Big)\exp\Big\{\sum_{k=1}^{M-1}\eta_k x_k\Big\}.$$

再由(3.156)解出 μ_k: 对 k 求和得 $\sum_k\mu_k/(1-\sum_j\mu_j)=\sum_k\exp(\eta_k)$, 即 $\frac{\sum_k\mu_k}{1-\sum_j\mu_j}=\sum_k\exp(\eta_k)$, 而 $\sum_k\mu_k=1-\mu_M$, 解得 $1-\sum_{k=1}^{M-1}\mu_k=\big(1+\sum_k\exp(\eta_k)\big)^{-1}$, 代回即得(3.158). 对照(3.142)读出(3.159).

□

注

(重构新增) 两种写法各有取舍: 保留 M 个参数时 $g\equiv 1$、形式最简洁, 但自然参数被约束在 $\sum_k e^{\eta_k}=1$ 上, 不是"标准"的指数族 (标准形式要求自然参数在开集内自由取值); 改用 $M-1$ 个参数后, 自然参数可以自由取值, 代价是 $g(\boldsymbol{\eta})$ 变成非平凡的归一化函数. 指数族的一般理论 (例如命题 3.26) 要求自然参数自由, 因此第二种写法才是范式.

这也解释了为什么小节 3.1.3中类别分布的参数只有 $K-1$ 个自由度: 冗余不是书写习惯, 而是分布本身的几何性质 (概率向量位于单纯形上).

最后考虑 Gauss 分布. 对于一元 Gauss 分布, 我们有

$$\begin{aligned} p(x\mid\mu,\sigma^2) &= \frac{1}{(2\pi\sigma^2)^{1/2}}\exp\left\{-\frac{1}{2\sigma^2}(x-\mu)^2\right\} \\ &= \frac{1}{(2\pi\sigma^2)^{1/2}}\exp\left\{-\frac{1}{2\sigma^2}x^2+\frac{\mu}{\sigma^2}x-\frac{1}{2\sigma^2}\mu^2\right\}. \end{aligned} \tag{3.160}$$

经过一些简单的重排, 就可以得到标准指数族分布的形式 [(3.142)](见练习 3.35):

$$\boldsymbol{\eta}=\begin{pmatrix}\mu/\sigma^2\\ -1/(2\sigma^2)\end{pmatrix}, \tag{3.161}$$

$$\boldsymbol{u}(x)=\begin{pmatrix}x\\ x^2\end{pmatrix}, \tag{3.162}$$

$$h(x)=(2\pi)^{-1/2}, \tag{3.163}$$

$$g(\boldsymbol{\eta})=(-2\eta_2)^{1/2}\exp\left(\frac{\eta_1^2}{4\eta_2}\right). \tag{3.164}$$

命题 3.24 (一元 Gauss 分布的指数族形式)

一元 Gauss 分布(3.27)可以写成(3.142)的标准形式, 其中 $\boldsymbol{\eta}$、$\boldsymbol{u}(x)$、$h(x)$、$g(\boldsymbol{\eta})$ 分别由(3.161)至(3.164)给出.

证明

(重构新增) 记 $\eta_1=\mu/\sigma^2$、$\eta_2=-1/(2\sigma^2)$, 则 $\sigma^2=-1/(2\eta_2)$, 且 $\mu=\eta_1\sigma^2=-\eta_1/(2\eta_2)$. 由(3.160),

$$\begin{aligned} p(x\mid\mu,\sigma^2) &= (2\pi)^{-1/2}(\sigma^2)^{-1/2}\exp\left\{\eta_2x^2+\eta_1x-\frac{\mu^2}{2\sigma^2}\right\} \\ &= (2\pi)^{-1/2}\Big(-\frac{1}{2\eta_2}\Big)^{-1/2}\exp\left\{-\frac{\mu^2}{2\sigma^2}\right\}\exp\{\eta_1x+\eta_2x^2\}. \end{aligned}$$

第一项与 x 无关, 可并入 $g(\boldsymbol{\eta})$. 计算常数因子:

$$(-\tfrac{1}{2\eta_2})^{-1/2}=(-2\eta_2)^{1/2},\qquad -\frac{\mu^2}{2\sigma^2}=-\frac{1}{2\sigma^2}\cdot\frac{\eta_1^2\sigma^4}{1}=-\frac{\eta_1^2\sigma^2}{2}=\frac{\eta_1^2}{4\eta_2},$$

故 $g(\boldsymbol{\eta})=(-2\eta_2)^{1/2}\exp\big(\eta_1^2/(4\eta_2)\big)$, 即(3.164). 于是

$$p(x\mid\boldsymbol{\eta})=(2\pi)^{-1/2}g(\boldsymbol{\eta})\exp\{\eta_1x+\eta_2x^2\}=h(x)g(\boldsymbol{\eta})\exp\{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}(x)\},$$

其中 $\boldsymbol{\eta},\boldsymbol{u}(x),h(x)$ 由(3.161)至(3.163)给出.

□

注

(重构新增) 一元 Gauss 分布的自然参数是二维的, 且第二个分量 $\eta_2=-1/(2\sigma^2)$ 必须为负——这正是它的自然参数空间是"下半平面"而非整个 $\mathbb{R}^2$ 的原因. 充分统计量 $\boldsymbol{u}(x)=(x,x^2)^{\mathsf{T}}$ 说明: 要知道 Gauss 分布的似然, 只需知道样本的和与平方和, 而不必保存全部样本.

这条结论也解释了为什么 Gauss 分布的共轭先验 (正态-逆 Gamma) 形式复杂: 自然参数空间是受约束的区域, 先验必须支撑在这个区域上. 当 $\sigma^2\to\infty$ 时 $\eta_2\to 0^-$, 自然参数趋于边界,g 的表达式出现退化.

有时我们会使用(3.142)所示的带约束形式, 其中我们选择 $\boldsymbol{u}(\boldsymbol{x}) = \boldsymbol{x}$. 不过, 我们也可以将其稍微推广一下. 如果 $f(x)$ 是一个归一化的密度函数, 则

$$\frac{1}{s} f\left(\frac{x}{s}\right) \tag{3.165}$$

也是一个归一化的密度函数, 其中 $s > 0$ 是尺度参数. 结合这些, 我们可以得到指数族分布的类-条件密度的一个约束集, 形式为

$$p(x \mid \lambda_k, s) = \frac{1}{s} h\left(\frac{x}{s}\right) g(\lambda_k) \exp\left\{\frac{1}{s} \lambda_k^{\mathsf{T}} x\right\}. \tag{3.166}$$

注意, 我们允许每个类别都有自己的参数向量 λ_k, 并且我们假设这些类别共享相同的尺度参数 s.

命题 3.25 (尺度变换保持归一化)

设 $f(x)$ 为归一化密度,$s > 0$, 则(3.165)也是归一化密度.

♠

证明

(重构新增) 作变量替换 $y = x/s$, 则 $\mathrm{d}x = s\,\mathrm{d}y$, 于是

$$\int \frac{1}{s} f\left(\frac{x}{s}\right) \mathrm{d}x = \int \frac{1}{s} f(y)\, s\, \mathrm{d}y = \int f(y)\, \mathrm{d}y = 1,$$

且被积函数非负, 故(3.165)是归一化密度.

□

注

(重构新增) (3.165)是“把一个密度横向拉伸 s 倍、纵向压缩 s 倍”的操作, 它保持总积分为 1, 因此仍是合法密度.(3.166)把这一尺度参数引入指数族, 使不同类别可以共享同一个“宽度”而各自拥有位置参数 λ_k, 这在分类问题中常用于建模各类条件密度形状相似、尺度相同的情形.

条件 $s > 0$ 保证变换不翻转坐标轴;$s \to 0$ 或 $s \to \infty$ 会使密度趋于奇异或平坦, 不在正常模型范围内. 注意这里各类共享 s 是一个实质性假设, 若各类尺度不同则应把 s 也加上下标 k.

3.4.1 充分统计量

定义 3.20 (充分统计量)

设数据 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 来自以 $\boldsymbol{\eta}$ 为参数的分布族. 若存在函数 $\boldsymbol{T}(X)$ 使得在给定 $\boldsymbol{T}(X)$ 的条件下 X 的分布与 $\boldsymbol{\eta}$ 无关, 则称 $\boldsymbol{T}(X)$ 为该分布族的充分统计量 (sufficient statistic). 对指数族(3.142),$\sum_n \boldsymbol{u}(\boldsymbol{x}_n)$ 是充分统计量.

♣

注

(重构新增) 充分统计量的直观含义是“数据中关于参数的全部信息都压缩在 $\boldsymbol{T}(X)$ 里”: 一旦知道 $\boldsymbol{T}(X)$, 再保留原始数据也不会提供关于 $\boldsymbol{\eta}$ 的额外信息. 因此它可以用于数据压缩、在线学习与分布式计算.

这一定义在指数族情形下有一个非常具体的体现: 似然只通过 $\sum_n \boldsymbol{u}(\boldsymbol{x}_n)$ 依赖数据 (命题 3.27), 故该和式必然是充分的. 对 Bernoulli 分布 $\boldsymbol{u}(x) = x$, 充分统计量是 $\sum_n x_n$; 对一元 Gauss 分布 $\boldsymbol{u}(x) = (x, x^2)^{\mathsf{T}}$, 充分统计量是 $\sum_n x_n$ 与 $\sum_n x_n^2$. 注意充分性依赖于分布族的选择: 同一组数据对不同模型可能有不同的充分统计量.

考虑使用最大似然估计的方法来估计一般指数族分布 [(3.142)] 中的参数向量 $\boldsymbol{\eta}$. 在(3.143)的两边对 $\boldsymbol{\eta}$ 求导, 可得

$$\nabla g(\boldsymbol{\eta}) \int h(\boldsymbol{x}) \exp\{\boldsymbol{\eta}^{\mathsf{T}} \boldsymbol{u}(\boldsymbol{x})\} \mathrm{d}\boldsymbol{x} + g(\boldsymbol{\eta}) \int h(\boldsymbol{x}) \exp\{\boldsymbol{\eta}^{\mathsf{T}} \boldsymbol{u}(\boldsymbol{x})\} \boldsymbol{u}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = 0. \tag{3.167}$$

重排并再次使用(3.143), 可得

$$\frac{1}{g(\boldsymbol{\eta})} \nabla g(\boldsymbol{\eta}) = g(\boldsymbol{\eta}) \int h(\boldsymbol{x}) \exp\{\boldsymbol{\eta}^{\mathsf{T}} \boldsymbol{u}(\boldsymbol{x})\} \boldsymbol{u}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = \mathbb{E}[\boldsymbol{u}(\boldsymbol{x})], \tag{3.168}$$

从而得到

$$-\nabla \ln g(\boldsymbol{\eta}) = \mathbb{E}[\boldsymbol{u}(\boldsymbol{x})]. \tag{3.169}$$

命题 3.26 (指数族的矩)

设 $p(\boldsymbol{x} \mid \boldsymbol{\eta}) = h(\boldsymbol{x})g(\boldsymbol{\eta})\exp\{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}(\boldsymbol{x})\}$ 为指数族分布,g 可微且(3.143)成立. 则

$$-\nabla \ln g(\boldsymbol{\eta}) = \mathbb{E}[\boldsymbol{u}(\boldsymbol{x})],$$

即 g 的对数的负梯度给出充分统计量的期望.

证明

(重构新增) 对(3.143)两边关于 $\boldsymbol{\eta}$ 求导. 左边用乘积法则得(3.167), 其中第二项来自 $\exp\{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}(\boldsymbol{x})\}$ 对 $\boldsymbol{\eta}$ 的导数 $\boldsymbol{u}(\boldsymbol{x})\exp\{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}(\boldsymbol{x})\}$.

由(3.143), $\int h\exp\{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}\}\mathrm{d}\boldsymbol{x} = 1/g(\boldsymbol{\eta})$, 代入(3.167)的第一项得 $\nabla g/g + g\int h e^{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}}\boldsymbol{u}\,\mathrm{d}\boldsymbol{x} = 0$. 而

$$g(\boldsymbol{\eta})\int h(\boldsymbol{x})e^{\boldsymbol{\eta}^{\mathsf{T}}\boldsymbol{u}(\boldsymbol{x})}\boldsymbol{u}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = \int p(\boldsymbol{x}\mid\boldsymbol{\eta})\boldsymbol{u}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = \mathbb{E}[\boldsymbol{u}(\boldsymbol{x})],$$

即(3.168). 故 $\dfrac{1}{g}\nabla g = -\mathbb{E}[\boldsymbol{u}]$, 即 $-\nabla \ln g(\boldsymbol{\eta}) = \mathbb{E}[\boldsymbol{u}(\boldsymbol{x})]$.

□

注

(重构新增) 这条结论给出了一个极其便利的“矩的微分公式”: 只要知道归一化函数 g, 就能通过对 $\ln g$ 求导得到充分统计量的期望, 而无需做任何积分. 若对(3.143)再求一次导数, 还能得到 $\mathrm{cov}[\boldsymbol{u}(\boldsymbol{x})] = -\nabla^2 \ln g(\boldsymbol{\eta})$(练习 3.36), 于是所有低阶矩都可以由 g 的导数读出.

推导只用到“积分与求导可交换”这一分析事实, 其合法性由 g 可微、积分在 $\boldsymbol{\eta}$ 的邻域内一致收敛保证. 若 $\boldsymbol{\eta}$ 取在自然参数空间的边界上, 积分可能发散, 公式失效.

因此, 如果可以对指数族分布进行归一化, 则总是可以通过对归一化函数进行简单的微分来找到它的矩.

考虑一组由 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 表示的独立同分布数据, 其似然函数为

$$p(X \mid \boldsymbol{\eta}) = \left(\prod_{n=1}^{N} h(\boldsymbol{x}_n)\right) g(\boldsymbol{\eta})^N \exp\left\{\boldsymbol{\eta}^{\mathsf{T}}\sum_{n=1}^{N}\boldsymbol{u}(\boldsymbol{x}_n)\right\}. \tag{3.170}$$

令 $\ln p(X \mid \boldsymbol{\eta})$ 对 $\boldsymbol{\eta}$ 的梯度为零, 可以得出最大似然估计量 $\boldsymbol{\eta}_{\mathrm{ML}}$ 必须满足以下条件:

$$-\nabla \ln g(\boldsymbol{\eta}_{\mathrm{ML}}) = \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{u}(\boldsymbol{x}_n). \tag{3.171}$$

命题 3.27 (指数族分布的最大似然估计)

设 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 为来自指数族(3.142)的独立同分布观测. 则最大似然估计 $\boldsymbol{\eta}_{\mathrm{ML}}$ 满足(3.171), 即“充分统计量的样本平均等于其期望”. 当 $N \to \infty$ 时,$\boldsymbol{\eta}_{\mathrm{ML}}$ 收敛到真实参数 $\boldsymbol{\eta}$.

证明

(重构新增) 对(3.170)取对数:

$$\ln p(X \mid \boldsymbol{\eta}) = \sum_n \ln h(\boldsymbol{x}_n) + N\ln g(\boldsymbol{\eta}) + \boldsymbol{\eta}^{\mathsf{T}}\sum_n \boldsymbol{u}(\boldsymbol{x}_n).$$

对 $\boldsymbol{\eta}$ 求梯度并令其为零:

$$N\frac{\nabla g(\boldsymbol{\eta})}{g(\boldsymbol{\eta})} + \sum_n \boldsymbol{u}(\boldsymbol{x}_n) = 0 \quad\Longrightarrow\quad -\nabla \ln g(\boldsymbol{\eta}) = \frac{1}{N}\sum_n \boldsymbol{u}(\boldsymbol{x}_n),$$

即(3.171).

关于 $N \to \infty$: 由大数定律,(3.171)右端几乎必然收敛到 $\mathbb{E}[\boldsymbol{u}(\boldsymbol{x})]$; 而由命题 3.26, 左端 $-\nabla \ln g(\boldsymbol{\eta})$ 正是 $\mathbb{E}[\boldsymbol{u}(\boldsymbol{x})]$, 即真实参数 $\boldsymbol{\eta}$ 满足同一方程. 若 $-\nabla \ln g$ 在自然参数空间上单射 (这是指数族自然参数化的标准性质), 则解唯一,

故 $\boldsymbol{\eta}_{\mathrm{ML}} \to \boldsymbol{\eta}$.

□

注

(重构新增) 可以看到, 最大似然估计量的解仅通过 $\sum_n \boldsymbol{u}(\boldsymbol{x}_n)$ 依赖于数据, 因此 $\boldsymbol{u}(\boldsymbol{x})$ 又称为分布(3.142)的充分统计量 (定义 3.20). 我们不需要存储整个数据集本身, 而只需要存储充分统计量的值.

例如, 对于 Bernoulli 分布, 函数 $\boldsymbol{u}(x)$ 仅由 x 给出, 因此只需要保留数据点 $\{x_n\}$ 的和; 而对于 Gauss 分布,$\boldsymbol{u}(x) = (x, x^2)^{\mathsf{T}}$, 因此应该保留 $\{x_n\}$ 的和与 $\{x_n^2\}$ 的和. 这一“压缩”性质是指数族在工程上极为常用的原因之一.

注意(3.171)一般没有闭式解: 左端是 $\boldsymbol{\eta}$ 的非线性函数, 需要数值求解. 但对若干具体分布 (如 Bernoulli、Gauss), 该方程恰好可解, 因而得到小节 3.1.1与小节 3.2.7中的显式公式.

考虑极限 $N \to \infty$,(3.171)的右边将变为 $\mathbb{E}[\boldsymbol{u}(\boldsymbol{x})]$, 通过与(3.169)进行比较, 我们可以看到在此极限下,$\boldsymbol{\eta}_{\mathrm{ML}}$ 将等于真实值 $\boldsymbol{\eta}$.

3.5 非参数化方法

本章专注于使用由少量参数决定的、具有特定函数形式的概率分布, 这些参数的值可从数据集中确定. 这称为参数化 (parametric) 的密度估计方法. 这种方法的一个关键局限是, 所选择的密度模型可能不适合数据分布, 这可能导致预测性能不佳. 例如, 如果生成数据的过程是多峰的, 那么 Gauss 分布永远无法捕捉到分布这个角度的特征, 因为 Gauss 分布必然是单峰的. 本节将考虑一些非参数化 (nonparametric) 的密度估计方法, 这些方法对分布的形式几乎不作任何假设.

3.5.1 直方图

首先讨论密度估计的直方图方法. 我们在前面关于边缘分布与条件分布 (见图 3.5) 以及中心极限定理 (见图 3.2) 的讨论中已经见到过这种方法. 下面将更详细地探讨直方图密度模型的特性, 并重点关注具有单个连续变量 x 的情况. 标准直方图简单将 x 划分为互不相交的宽度为 Δ_i 的分箱, 然后计算落在不同分箱中的观测值 x 的数量 n_i. 为了将此计数值转换为归一化的概率密度, 我们可以将其简单地除以观测值总数 N 和分箱宽度 Δ_i 的乘积, 以获得落入每个分箱的概率值:

$$p_i = \frac{n_i}{N\Delta_i}. \tag{3.172}$$

定义 3.21 (直方图密度估计)

设 $x_1, \cdots, x_N$ 为一元观测值, 将实轴划分为互不相交的分箱 $B_i = [b_{i-1}, b_i)$, 其宽度 $\Delta_i = b_i - b_{i-1}$, 并记 n_i 为落入第 i 个分箱的观测数. 称分段常数函数

$$\hat{p}(x) = \frac{n_i}{N\Delta_i}, \qquad x \in B_i$$

为数据的直方图密度估计 (histogram density estimate).

♣

注

(重构新增) 直方图是最古老的密度估计方法, 其思想是“用频率近似概率、用体积归一化”:$\frac{n_i}{N}$ 是落入第 i 个分箱的经验概率, 再除以箱宽 Δ_i 就把它变成密度. 这样得到的 $\hat{p}$ 在每个箱内为常数, 因此是阶梯函数, 天然不连续.

箱宽 Δ_i 是唯一的调节参数:Δ_i 越小, 阶梯越细、越能捕捉局部结构, 但每箱内的样本越少、噪声越大;Δ_i 越大, 估计越平滑, 但会抹平真实的多峰结构 (见图 3.13). 这与多项式回归中阶数 M 的选择属于同一类模型复杂度问题 (节 1.2).

可以看出, $\int p(x)\,\mathrm{d}x = 1$. 这种方法就给出了密度 $p(x)$ 的一种模型, 在每个分箱的宽度上, 密度是恒定的. 通常情况下, 我们选择具有相同宽度的分箱,$\Delta_i = \Delta$.

命题 3.28 (直方图密度的归一性)

设 $\hat{p}(x) = n_i/(N\Delta_i)$ 对 $x \in B_i$, 诸 B_i 互不相交且覆盖全部观测所在的区间, $\sum_i n_i = N$. 则 $\int \hat{p}(x)\,\mathrm{d}x = 1$.

证明

(重构新增) 由定义,

$$\int \hat{p}(x)\,\mathrm{d}x = \sum_i \int_{B_i} \frac{n_i}{N\Delta_i}\,\mathrm{d}x = \sum_i \frac{n_i}{N\Delta_i}\cdot\Delta_i = \frac{1}{N}\sum_i n_i = 1,$$

因为每个观测恰好落入一个分箱, 故 $\sum_i n_i = N$.

□

注

(重构新增) 归一化成立的前提是“分箱覆盖了全部观测”, 若把观测范围之外的部分也算入积分, 则需要额外规定 $\hat{p} = 0$, 此时归一化仍然成立. 这条性质说明直方图是一个合法的密度模型, 而不是任意的高度堆叠.

注意归一化并不要求各箱等宽: Δ_i 可以不同, 只要每箱按自己的宽度作除法即可. 等宽只是常见约定, 便于比较各箱高度.

图 3.13展示了直方图密度估计的示例. 这里的数据来自绿色曲线所对应的分布, 该分布由两个 Gauss 分布混合形成. 这 3 个直方图密度估计的例子对应着 3 种不同的分箱宽度 Δ. 我们可以看到, 当 Δ 非常小时 (图 3.13的顶部图), 得到的密度模型非常毛糙, 具有许多在生成数据集的底层分布中不存在的结构. 相反, 如果 Δ 太大 (图 3.13的底部图), 得到的密度模型将太过平滑, 因此无法捕捉到绿色曲线的双峰特性. 最佳结果是 Δ 取某个中间值 (图 3.13的中间图). 原则上, 直方图密度模型还取决于分箱边界的选择, 但这相较于分箱宽度 Δ 的选择来说通常不那么重要.

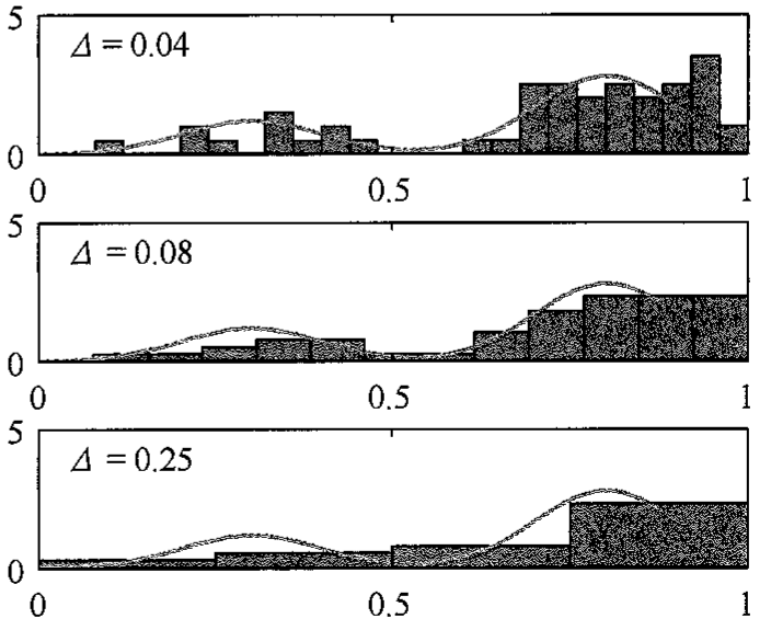


图 3.13: 直方图密度估计的示例, 其中根据绿色曲线所示的分布生成含有 50 个数据点的数据集. 这里的直方图密度估计基于(3.172), 具有相同的分箱宽度 Δ_i. 图中展示了不同分箱宽度的结果.

注意直方图方法具有一个特性 (与稍后讨论的其他密度估计方法不同), 即一旦直方图计算完成, 数据集本身就可以丢弃, 这一点在数据集很大时会十分有利. 如果数据点是顺序到达的, 则应用直方图方法也将比较容易.

在实践中, 直方图技术可以用于快速可视化一维或二维数据, 但其不适用于大多数密度估计应用. 一个明显的问题是, 分箱边界导致估计的密度是不连续的, 而这并不是由生成数据的底层分布的任何性质引起的. 直方图方法

的一个主要局限是, 规模会随维度变化显著改变. 如果将 D 维空间中的每个变量划分为 M 个区间, 那么分箱将有 M^D 个. 这种随 D 指数级扩展的特性是维度诅咒 (curse of dimensionality) 的一个例子 (参见小节 6.1.1). 在高维空间中, 为了提供有意义的局部概率密度估计, 所需的数据量将是我们无法承受的.

然而, 直方图方法也给密度估计带来了两条重要的经验. 首先, 要在特定位置估计概率密度, 就应该考虑落在该位置某个局部邻域内的数据点. 注意局部性的概念要求我们假设某种距离度量, 而在这里我们假设的是欧氏距离. 对于直方图来说, 这种邻域性质是由分箱定义的, 并且存在一个自然的 "平滑" 参数用于描述局部区域的空间大小, 本例中即为分箱宽度. 其次, 为了获得良好的结果, 平滑参数的值既不应该太大也不应该太小. 这会让人联想到多项式回归中模型复杂性的选择 (参见节 1.2), 其中多项式的阶数 M 或者正则化参数 λ 在某些中间值上是最优的, 既不太大也不太小. 有了这些洞察, 下面我们来讨论两种使用广泛的非参数化密度估计技术——核密度和最近邻, 它们在维度变化时的扩展性比简单的直方图模型要好.

注

(重构新增) 这一段实际上给出了非参数密度估计的 "方法论总结": 任何局部方法都由两件事决定——邻域的定义方式 (距离、形状、大小) 与平滑参数的选取. 直方图把邻域固定为分箱, 因此其平滑参数就是箱宽; 核密度把邻域固定为形状给定、宽度为 h 的核; 最近邻则让邻域大小随数据密度自适应变化.

分箱边界造成的不连续是直方图的固有问题: 它不是数据本身的性质, 而是人为网格的产物. 改进方向有两个: 用光滑核代替指示函数 (消除不连续), 或让邻域随位置自适应 (缓解高维稀疏). 这两种改进分别对应小节 3.5.2与小节 3.5.3.

3.5.2 核密度

假设观测值是从某个 D 维空间 (这里将其视为欧氏空间) 中的某个未知概率密度 $p(\boldsymbol{x})$ 中产生的, 我们希望估计 $p(\boldsymbol{x})$ 的值. 根据先前关于局部性的讨论, 考虑包含 $\boldsymbol{x}$ 的一个小区域 $\mathcal{R}$. 与该区域相关联的概率质量为

$$P = \int_{\mathcal{R}} p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}. \tag{3.173}$$

假设我们已经收集了一个包含 N 个从 $p(\boldsymbol{x})$ 中产生的观测值的数据集. 因为每个数据点在 $\mathcal{R}$ 内的概率为 P, 所以 $\mathcal{R}$ 内的点的总数 K 将服从二项分布 (参见小节 3.1.2):

$$\mathrm{Bin}(K \mid N, P) = \frac{N!}{K!\,(N-K)!} P^K (1-P)^{N-K}. \tag{3.174}$$

利用(3.13)可以看出, 落入该区域的点的平均占比为 $\mathbb{E}[K/N] = P$. 类似地, 利用(3.14)可以看出, 此均值对应的方差为 $\mathrm{var}[K/N] = P(1-P)/N$. 对于较大的 N, 该分布在均值处将变得十分尖锐, 因此

$$K \approx NP. \tag{3.175}$$

然而, 如果我们同时假设区域 $\mathcal{R}$ 足够小, 以至于概率密度 $p(\boldsymbol{x})$ 在该区域内大致保持恒定, 则有

$$P \approx p(\boldsymbol{x})V, \tag{3.176}$$

其中 V 是区域 $\mathcal{R}$ 的体积. 结合(3.175)和(3.176), 可以得到密度估计的形式为

$$p(\boldsymbol{x}) = \frac{K}{NV}. \tag{3.177}$$

命题 3.29 (局部密度估计)

设区域 $\mathcal{R}$ 包含 $\boldsymbol{x}$、体积为 V,N 个独立样本中有 K 个落入 $\mathcal{R}$. 若 p 在 $\mathcal{R}$ 内近似为常数且 N 充分大, 则

$$p(\boldsymbol{x}) \approx \frac{K}{NV}.$$

证明

(重构新增) 把落入 $\mathcal{R}$ 视为一次“成功”概率为 P 的 Bernoulli 试验, 则 $K \sim \mathrm{Bin}(K \mid N, P)$, 即(3.174). 由命题 3.3,$\mathbb{E}[K/N] = P$、$\mathrm{var}[K/N] = P(1-P)/N$. 当 $N \to \infty$ 时方差趋于零, 故 K/N 依概率收敛到 P, 即 $K \approx NP$((3.175)).

另一方面, 若 p 在 $\mathcal{R}$ 内近似为常数, 则(3.173)给出 $P = \int_{\mathcal{R}} p\,\mathrm{d}\boldsymbol{x} \approx p(\boldsymbol{x})V$((3.176)). 两式联立消去 P 得 $p(\boldsymbol{x}) \approx K/(NV)$, 即(3.177).

□

注

(重构新增) (3.177)是本节所有非参数方法的共同出发点: 它把“估计密度”转化为“数一数邻域里有多少点、再除以体积”. 核密度与最近邻的区别只在于固定 V 还是固定 K.

注意式(3.177)的有效性依靠两个相互矛盾的假设: 区域 $\mathcal{R}$ 足够小, 以至于密度在该区域内近似恒定; 但区域 $\mathcal{R}$ 又足够大 (与该区域内密度的值相关), 使得落入该区域的点的数量 K 足以使二项分布变得十分尖锐.V 的选取就是在这两个要求之间折中, 这正是平滑参数 h(或 K) 要解决的问题. 此外,$N \to \infty$ 时还需 $V \to 0$ 而 $NV \to \infty$, 才能同时保证偏差与方差都趋于零.

我们可以通过两种不同的方式利用(3.177). 一种是固定 K 并根据数据确定 V 的值, 这将引出后面讨论的 K 近邻技术; 另一种是固定 V 并根据数据确定 K 的值, 从而引出核密度估计技术. 可以证明, 当 $N \to \infty$ 时, 只要 V 随着 N 以合适的速率缩小, 而 K 随着 N 以合适的速率增长,K 近邻密度估计和核密度估计都会收敛到真实概率密度 (Duda and Hart, 1973).

下面详细讨论核密度估计技术. 首先, 对于希望确定概率密度的点 $\boldsymbol{x}$, 将区域 $\mathcal{R}$ 取为以点 $\boldsymbol{x}$ 为中心的超小立方体. 为了计算落入该区域的点的数量 K, 定义如下函数:

$$k(\boldsymbol{u}) = \begin{cases} 1, & |u_i| \leqslant 1/2,\ i = 1, \cdots, D, \\ 0, & \text{其他}. \end{cases} \tag{3.178}$$

定义 3.22 (核函数与 Parzen 窗)

称满足 $k(\boldsymbol{u}) \geqslant 0$ 且 $\int k(\boldsymbol{u})\,\mathrm{d}\boldsymbol{u} = 1$ 的函数 k 为核函数 (kernel function). 由(3.178)给出的 k 表示以原点为中心的单位立方体, 又称为 Parzen 窗 (Parzen window).

♣

注

(重构新增) 核函数的作用是把“点是否落在邻域内”这一硬判定换成可加权的“相似度”:Parzen 窗是 0/1 的硬判定 (在立方体内为 1, 否则为 0), 因此它对应的密度估计仍是阶梯状的、在立方体边界上不连续; 若把 k 换成光滑函数 (如 Gauss 核), 得到的密度也就光滑了.

定义中的两个条件各有含义: 非负性保证估计出的密度非负; 归一化 $\int k = 1$ 保证每个数据点对总概率质量的贡献恰好为 1, 从而整体密度可归一 (见命题 3.30). 注意 k 不必对称, 但对称核在直观上更自然.

它表示以原点为中心的单位立方体. 函数 $k(\boldsymbol{u})$ 是一个核函数, 在这里, 它又称为 Parzen 窗. 根据(3.178), 如果数据点 $\boldsymbol{x}_n$ 位于以点 $\boldsymbol{x}$ 为中心、边长为 h 的立方体内, 那么量 $k((\boldsymbol{x} - \boldsymbol{x}_n)/h)$ 将为 1, 否则为 0. 因此, 位于该立方体内的数据点的总数为

$$K = \sum_{n=1}^{N} k\left(\frac{\boldsymbol{x} - \boldsymbol{x}_n}{h}\right). \tag{3.179}$$

将(3.179)代入(3.177), 可得点 $\boldsymbol{x}$ 处估计的密度为

$$p(\boldsymbol{x}) = \frac{1}{N}\sum_{n=1}^{N} \frac{1}{h^D} k\left(\frac{\boldsymbol{x} - \boldsymbol{x}_n}{h}\right), \tag{3.180}$$

这里使用了 D 维空间中边长为 h 的超小立方体的体积公式 $V = h^D$. 利用函数 $k(\boldsymbol{u})$ 的对称性, 我们可以不再将其视为以点 $\boldsymbol{x}$ 为中心的单个立方体, 而是视为以 N 个数据点 $\boldsymbol{x}_n$ 为中心的 N 个立方体的总和, 进而重新解释这个方程.

定义 3.23 (核密度估计)

设 k 为核函数,$h>0$ 为带宽 (平滑参数). 称由(3.180)定义的函数 $\hat{p}$ 为数据 $\{\boldsymbol{x}_n\}$ 的核密度估计 (kernel density estimate), 又称 Parzen 估计.

♣

注

(重构新增) (3.180)的两种读法都很有用: 从“以 $\boldsymbol{x}$ 为中心”看, 它是“数邻域内的点数再除以体积”; 从“以数据点为中心”看, 它是“在每个数据点上放一个核、再把 N 个核平均”. 后一种读法直接说明了为什么核密度估计总是光滑的 (只要核光滑)、为什么它的计算量是 $O(N)$、以及为什么它天然是一个合法密度 (核的非负性与归一性会被继承).

带宽 h 扮演平滑参数的角色:h 太小则每个核过于尖锐, 估计出现许多虚假尖峰 (对噪声敏感);h 太大则核相互重叠过度, 多峰结构被抹平. 这一权衡与直方图的箱宽选择完全平行 (图 3.14).

就目前情况来看, 核密度估计 [(3.180)] 将遇到与直方图方法相同的一个问题, 即存在人为的不连续性, 此处体现在立方体的边界上. 如果我们选择更平滑的核函数, 则可以获得更平滑的密度模型. 我们通常选择使用 Gauss 函数, 导出的核密度模型为

$$p(\boldsymbol{x})=\frac{1}{N}\sum_{n=1}^{N}\frac{1}{(2\pi h^2)^{D/2}}\exp\left\{-\frac{\|\boldsymbol{x}-\boldsymbol{x}_n\|^2}{2h^2}\right\}. \tag{3.181}$$

其中 h 代表所使用 Gauss 分量的标准差. 因此, 我们得到密度模型的方式是, 首先在每个数据点上放置一个 Gauss 分布, 并将整个数据集的贡献相加, 然后除以 N 来正确地归一化密度. 在图 3.14中, 核密度模型 [(3.181)] 被应用于之前展示直方图方法的数据集. 正如预期的那样, 我们可以看到参数 h 充当了平滑参数的角色, 并且 h 较小时对噪声敏感, 与 h 较大时过度平滑之间存在权衡. 同样, 类似于直方图密度估计中的分箱选择或曲线拟合中所使用多项式的阶数选择,h 的优化是模型复杂性的问题.

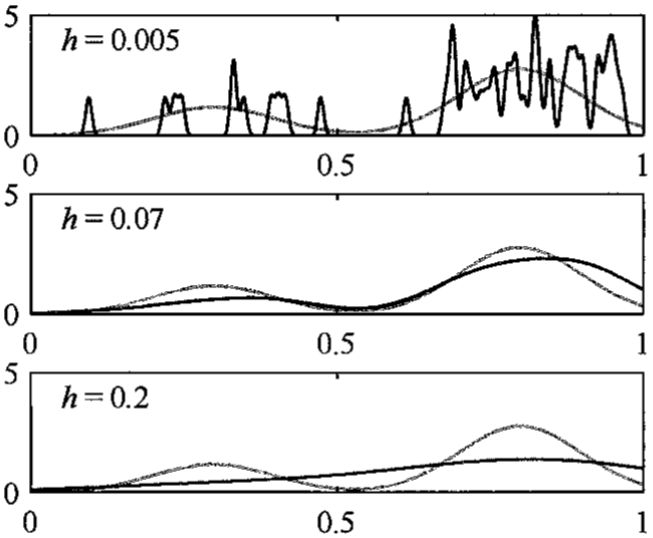


图 3.14: 核密度模型 [(3.181)] 的图示. 这里使用与前面展示直方图方法时相同的数据集. 我们可以看到 h 充当了平滑参数的角色, 如果 h 设置得太小 (顶部图), 则结果是噪声很大的核密度模型; 而如果 h 设置得太大 (底部图), 那么所生成数据的底层分布 (绿色曲线) 的双峰性质就会被淡化. 通过对 h 取某个中间值, 可以获得最佳的核密度模型 (中间图).

我们可以在(3.142)中选择任何其他核函数 $k(\boldsymbol{u})$, 只要满足以下两个条件即可.

$$k(\boldsymbol{u})\geqslant 0, \tag{3.182}$$

$$\int k(\boldsymbol{u})\,\mathrm{d}\boldsymbol{u}=1. \tag{3.183}$$

这两个条件确保了得到的概率分布在任何地方都是非负的, 并且积分为 1. 由(3.180)给出的这类密度模型称为核

密度估计器或 Parzen 估计器.

命题 3.30 (核密度估计的归一性)

设 k 满足(3.182)与(3.183), 则由(3.180)定义的 $\hat{p}$ 满足 $\hat{p}(\boldsymbol{x}) \geqslant 0$ 与 $\int \hat{p}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = 1$.

♠

证明

(重构新增) 非负性: 由(3.182),$k((\boldsymbol{x}-\boldsymbol{x}_n)/h) \geqslant 0$, 故(3.180)右端是若干非负项之和, 必非负.

归一性: 作变量替换 $\boldsymbol{u} = (\boldsymbol{x}-\boldsymbol{x}_n)/h$, 则 $\mathrm{d}\boldsymbol{x} = h^D\mathrm{d}\boldsymbol{u}$, 于是

$$\int \hat{p}(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{h^D}\int k\left(\frac{\boldsymbol{x}-\boldsymbol{x}_n}{h}\right)\mathrm{d}\boldsymbol{x} = \frac{1}{N}\sum_{n=1}^{N}\int k(\boldsymbol{u})\,\mathrm{d}\boldsymbol{u} = \frac{1}{N}\cdot N\cdot 1 = 1,$$

其中用到(3.183).

□

注

(重构新增) 这两条结论说明“核的条件”正是“估计量是密度”的充分条件: 核非负 $\Rightarrow$ 估计非负, 核归一 $\Rightarrow$ 估计归一. 因此, 任何满足这两个条件的函数都可以拿来当核, 而不必是立方体或 Gauss 函数——常见的选择还有 Epanechnikov 核、均匀核、三角核等.

注意归一化依赖于因子 $1/h^D$(D 维体积缩放), 少了它就只对 k 的某个特定形状成立. 这也解释了为什么带宽 h 同时控制“平滑程度”与“密度的高度”.

它们有一个很大的优点, 就是在“训练”阶段不涉及计算, 因为只需要将训练集存储起来即可. 然而, 这同时也是这类密度模型的一个巨大缺点, 因为评估密度的计算成本会随着数据集的增大而线性增长.

3.5.3 最近邻

使用核方法进行密度估计的一个难点在于, 对所有核来说控制核宽度的参数 h 是固定的. 在数据密度较高的区域, 较大的 h 值可能导致过度平滑, 并且可能淡化从数据中本应提取出的结构. 然而, 减小 h 值可能会导致在其他数据密度较低的区域产生噪声估计. 因此,h 的最佳选择可能取决于数据空间中的位置. 这个问题可以通过使用最近邻方法进行密度估计来解决.

因此, 下面我们重新回到局部密度估计的一般结果 [(3.177)]. 我们不使用固定 V 值并从数据中确定 V 值的方式, 而是考虑固定 K 值并使用数据找到适当的 V 值. 为此, 对于希望估计密度 $p(\boldsymbol{x})$ 的点 $\boldsymbol{x}$, 考虑以点 $\boldsymbol{x}$ 为中心的小球, 并允许小球的半径增长, 直至恰好包含 K 个数据点. 然后, 密度 $p(\boldsymbol{x})$ 的估计由(3.177)给出, 其中 V 设置为小球的体积.

定义 3.24 (K 近邻密度估计)

设 K 为正整数, 对给定的点 $\boldsymbol{x}$, 记 $V(\boldsymbol{x})$ 为以 $\boldsymbol{x}$ 为中心、恰好包含 K 个训练数据点的最小球的体积. 称

$$\hat{p}(\boldsymbol{x}) = \frac{K}{N\,V(\boldsymbol{x})}$$

为 K 近邻 (K nearest neighbor) 密度估计.

♣

注

(重构新增) 最近邻方法把“固定体积”换成“固定点数”, 从而使邻域大小随数据密度自适应: 在数据稠密处, 只要很小的球就能凑够 K 个点, 因此 V 小、估计更细致; 在数据稀疏处, 球必须张得很大,V 大、估计被平滑. 这正好解决了核方法“全局固定 h”的困难.

K 扮演平滑参数的角色:K 太小则对噪声敏感 (V 由最近的几个点决定, 波动大),K 太大则过度平滑 (V 覆盖太大区域). 注意 $\hat{p}$ 一般不是真正的密度——它在整个空间上的积分发散 (练习 3.38), 这是它作为“局部估计”的代价.

图 3.15展示了在使用与图 3.13和图 3.14中相同的数据集的情况下, 选择不同参数 K 时的 K 近邻密度估计. 我们可以看到 K 值决定了模型的平滑程度, 而且同样存在一个既不太大也不太小的最佳 K 值. 注意 K 近邻产生的模型并不是一个真正的密度模型, 因为其在整个空间上的积分是发散的 (见练习 3.38).

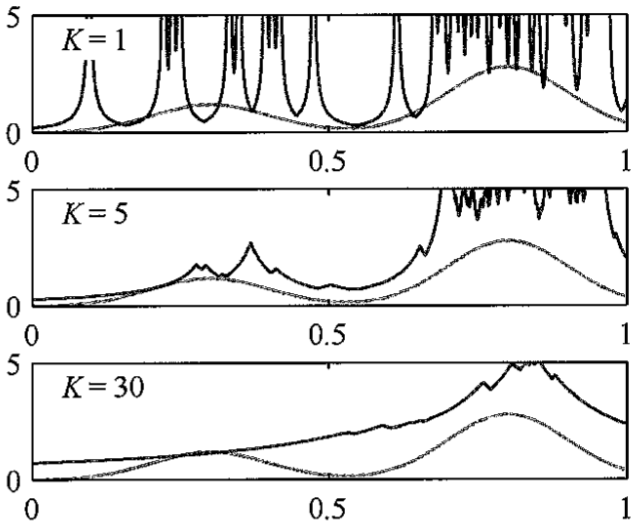


图 3.15: K 近邻密度估计的图示, 使用与前面展示直方图方法和核密度方法时相同的数据集. 可以看到参数 K 控制了模型的平滑程度, 因此较小的 K 值会导致噪声极大的密度模型 (顶部图), 而较大的 K 值则会平滑掉所生成数据的真实分布 (绿色曲线) 的双峰性质 (底部图).

下面通过展示 K 近邻技术如何扩展到分类问题来结束本章. 为此, 我们需要将 K 近邻密度估计分别应用于每个类别, 然后利用 Bayes 定理. 假设数据集包含 N_k 个属于类别 C_k 的点, 总共有 N 个点, 即 $\sum_k N_k = N$. 如果希望对一个新点 $\boldsymbol{x}$ 进行分类, 则可以在这个点的周围绘制一个恰好包含 K 个点的球体, 这里不考虑它们的类别. 假设这个球体的体积为 V, 并且包含来自类别 C_k 的 K_k 个点.(3.177)提供了与每个类别相关联的密度估计值:

$$p(\boldsymbol{x} \mid C_k) = \frac{K_k}{N_k V}. \tag{3.184}$$

类似地, 无条件密度由下式给出:

$$p(\boldsymbol{x}) = \frac{K}{NV}. \tag{3.185}$$

类别先验由下式给出:

$$p(C_k) = \frac{N_k}{N}. \tag{3.186}$$

我们现在可以使用 Bayes 定理结合(3.184)至(3.186)得到类别成员的后验概率:

$$p(C_k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_k)p(C_k)}{p(\boldsymbol{x})} = \frac{K_k}{K}. \tag{3.187}$$

命题 3.31 (K 近邻分类的后验概率)

设按(3.184)至(3.186)定义各类密度、总密度与类先验, 则后验概率为 $p(C_k \mid \boldsymbol{x}) = K_k/K$, 其中 K_k 是球体内属于类别 C_k 的点数,$K = \sum_k K_k$.

证明

(重构新增) 由 Bayes 定理,

$$p(C_k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_k)p(C_k)}{p(\boldsymbol{x})} = \frac{\frac{K_k}{N_k V} \cdot \frac{N_k}{N}}{\frac{K}{NV}} = \frac{K_k}{K},$$

其中 V 与 N 在分子分母中相消,N_k 也相消.

□

注

(重构新增) 这个结果极其简洁: 后验概率就是“球内该类点数占球内总点数的比例”. 它说明 K 近邻分类完全不需要显式估计密度, 只需要数点数——这正是非参数方法“把估计问题化为计数问题”的典型体现.

注意推导中假设各类共享同一个球体与同一个 V(即同一个 K), 否则三个式子中的 V 不能相消.K_k/K 的估计误差随 K 增大而减小, 但 K 增大会使球体跨越类别边界、引入偏差, 因此 K 的选择仍是偏差—方差权衡.

我们可以通过将测试点 $\boldsymbol{x}$ 分配给具有最大后验概率的类别来最小化错误分类的概率, 这对应于 K_k/K 的最大值. 因此, 在对一个新点进行分类时, 我们首先需要识别训练集中与新点最近的 K 个点, 然后将新点分配给在这组点中拥有最多代表的类别. 如果多个类别拥有的代表数量一致, 则可以将新点随机分配给其中一个类别.$K = 1$ 的特殊情况称为最近邻法则, 因为测试点被简单地分配给了训练集中离新点最近的点所属的那个类别.图 3.16对这些概念进行了说明.

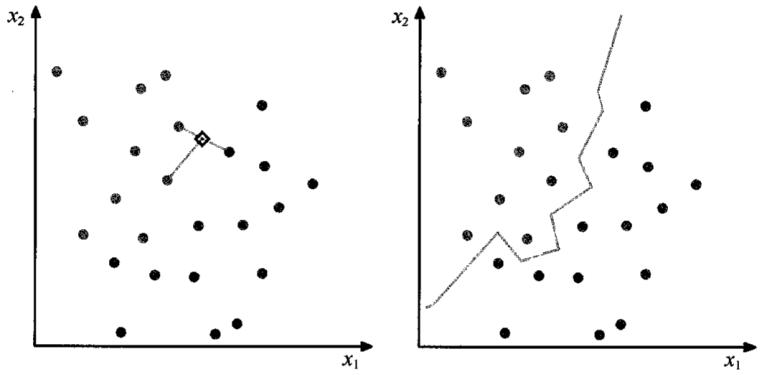


图 3.16: (a) 在 K 近邻分类器中, 一个用黑色菱形表示的新点根据 K 个最接近的训练数据点被归类为类别成员最多的那个类别, 在本例中 $K = 3$.(b) 在最近邻 ($K = 1$) 分类器中, 所得到的决策边界由超平面组成, 每个超平面是由来自不同类别点对的垂直平分线构造的.

最近邻 ($K = 1$) 分类器的一个有趣特性是, 在 $N \to \infty$ 的情况下, 错误率永远不会超过最优分类器 (即使用真实类别分布的分类器) 的最小可实现错误率的两倍 (Cover and Hart, 1967).

到目前为止, 我们已经讨论了 K 近邻方法和核密度估计, 它们都需要存储整个训练集. 如果数据集很大, 计算成本将十分高昂. 为了抵消这种影响, 可以构建基于树的搜索结构, 引入一些一次性的额外计算, 以便高效地找到 (近似的) 近邻, 而无须对整个数据集进行全量搜索. 然而, 这些非参数化方法的使用场景仍然受到严重制约. 另外, 我们已经看到简单的参数化模型仅能表示非常有限的分布形式. 因此我们需要找到一些非常灵活的密度模型, 它们的复杂度可以独立于训练集的大小而进行控制, 深度神经网络可以实现这一点.

注

(重构新增) 这一收尾段把全章的两条线索接了起来: 参数化模型 (如单个 Gauss 分布) 表达力有限, 非参数化模型 (直方图、核密度、最近邻) 表达力强却需要保存全部数据、计算随 N 线性增长. 深度神经网络提供了第三条路——用固定数量的参数表示高度灵活的分布, 从而同时获得参数模型的效率与非参数模型的适应性.

“Cover-Hart 定理”给出的 2 倍上界是最近邻方法最重要的理论保证, 它说明哪怕最简单的 $K = 1$ 规则在样本无限时也接近最优; 但其前提是 $N \to \infty$ 且数据独立同分布, 在有限样本、高维情形下这一保证并不提供实际性能的承诺.

3.6 习题

✍ **练习 3.1** (★) 验证 Bernoulli 分布 [(3.2)] 满足下列性质:

$$\sum_{x=0}^{1} p(x \mid \mu) = 1, \qquad \mathbb{E}[x] = \mu, \qquad \mathrm{var}[x] = \mu(1-\mu),$$

并证明符合 Bernoulli 分布的随机二元变量 x 的熵 $H[x]$ 由下式给出:

$$H[x] = -\mu \ln \mu - (1-\mu)\ln(1-\mu).$$

✍ **练习 3.2** (★★)(3.2)给出的 Bernoulli 分布形式在两个 x 值之间不对称. 在某些情况下, 使用另一种等价的形式会更方便, 其中 $x \in \{-1, 1\}$, 此时该分布可以写成

$$p(x \mid \mu) = \left(\frac{1-\mu}{2}\right)^{(1-x)/2} \left(\frac{1+\mu}{2}\right)^{(1+x)/2},$$

其中 $\mu \in [-1, 1]$. 证明该式给出的分布是归一化的, 并计算它的均值、方差和熵.

✍ **练习 3.3** (★★) 证明二项分布 [(3.11)] 是归一化的. 为此, 首先使用定义(3.12), 即从总数为 N 的物体中选择 m 个相同物体的组合数量, 证明下式:

$$\binom{N}{m} + \binom{N}{m-1} = \binom{N+1}{m}.$$

然后利用上述结论, 通过归纳证明以下结论:

$$(1+x)^N = \sum_{m=0}^{N} \binom{N}{m} x^m,$$

这个称为二项式定理 (binomial theorem) 的结论对所有实数 x 都成立. 最后, 证明二项分布是归一化的, 从而使

$$\sum_{m=0}^{N} \binom{N}{m} \mu^m (1-\mu)^{N-m} = 1.$$

✍ **练习 3.4** (★★) 证明二项分布的均值由(3.13)给出. 为此, 在归一化条件

$$\sum_{m=0}^{N} \binom{N}{m} \mu^m (1-\mu)^{N-m} = 1$$

的两边对 μ 求导, 并重排以获得 μ 的均值表达式. 类似地, 通过对上式关于 μ 进行二次求导, 并利用二项分布均值的结果, 证明二项分布方差的结果 [(3.14)].

✍ **练习 3.5** (★) 证明多元 Gauss 分布(3.28)的众数为 $\boldsymbol{\mu}$.

✍ **练习 3.6** (★★) 假设 $\boldsymbol{x}$ 服从均值为 $\boldsymbol{\mu}$、协方差为 $\boldsymbol{\Sigma}$ 的 Gauss 分布. 证明线性变换后的变量 $\boldsymbol{Ax} + \boldsymbol{b}$ 也服从 Gauss 分布, 并给出它的均值和协方差.

✍ **练习 3.7** (★★★) 证明两个 Gauss 分布 $q(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_q, \boldsymbol{\Sigma}_q)$ 和 $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_p, \boldsymbol{\Sigma}_p)$ 之间的 Kullback-Leibler 散度为

$$\mathrm{KL}(q \,\|\, p) = \frac{1}{2}\left\{\ln \frac{|\boldsymbol{\Sigma}_p|}{|\boldsymbol{\Sigma}_q|} - D + \mathrm{tr}\left(\boldsymbol{\Sigma}_p^{-1}\boldsymbol{\Sigma}_q\right) + (\boldsymbol{\mu}_p - \boldsymbol{\mu}_q)^{\mathsf{T}}\boldsymbol{\Sigma}_p^{-1}(\boldsymbol{\mu}_p - \boldsymbol{\mu}_q)\right\},$$

其中 $\mathrm{tr}(\cdot)$ 表示矩阵的迹,D 为 $\boldsymbol{x}$ 的维度.

✍ **练习 3.8** (★★) 这个习题表明, 对于给定的协方差, 具有最大熵的多元分布是 Gauss 分布. 分布 $p(\boldsymbol{x})$ 的熵由下式给出:

$$H[\boldsymbol{x}] = -\int p(\boldsymbol{x}) \ln p(\boldsymbol{x}) \,\mathrm{d}\boldsymbol{x}.$$

我们希望在所有分布 $p(\boldsymbol{x})$ 中最大化 $H[\boldsymbol{x}]$, 约束如下: 满足 $p(\boldsymbol{x})$ 是归一化的, 且具有特定的均值和协方差, 使得

$$\int p(\boldsymbol{x}) \,\mathrm{d}\boldsymbol{x} = 1, \qquad \int p(\boldsymbol{x})\boldsymbol{x} \,\mathrm{d}\boldsymbol{x} = \boldsymbol{\mu}, \qquad \int p(\boldsymbol{x})(\boldsymbol{x}-\boldsymbol{\mu})(\boldsymbol{x}-\boldsymbol{\mu})^{\mathsf{T}} \mathrm{d}\boldsymbol{x} = \boldsymbol{\Sigma}.$$

通过对熵进行变分最大化, 并使用 Lagrange 乘子来确保约束条件, 证明最大熵分布由 Gauss 分布(3.28)给出.

✍ **练习 3.9** (★★★) 证明多元 Gauss 分布 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 的熵由下式给出:

$$H[\boldsymbol{x}] = \frac{1}{2}\ln|\boldsymbol{\Sigma}| + \frac{D}{2}\big(1 + \ln(2\pi)\big),$$

其中 D 为 $\boldsymbol{x}$ 的维度.

✍ **练习 3.10** (★★★) 考虑两个随机变量 x_1 和 x_2, 它们的分布都是 Gauss 分布, 均值分别为 μ_1 和 μ_2, 精度分别为 τ_1 和 τ_2. 推导变量 $x = x_1 + x_2$ 的微分熵表达式. 为此, 首先利用以下关系找到 x 的分布:

$$p(x) = \int_{-\infty}^{\infty} p(x \mid x_2) p(x_2)\, \mathrm{d}x_2,$$

并完成指数中的平方项. 然后观察到这表示两个 Gauss 分布的卷积, 而卷积本身也是 Gauss 分布. 最后利用式(3.27)中一元 Gauss 分布的熵的计算结果.

✍ **练习 3.11** (★) 考虑(3.28)给出的多元 Gauss 分布. 通过将精度矩阵 (逆协方差矩阵) 写成对称矩阵和反对称矩阵的和, 证明反对称项不会出现在 Gauss 分布的指数中, 所以精度矩阵可以不失一般性地假定为对称的. 又因为对称矩阵的逆矩阵也是对称的 (见练习 3.16), 所以协方差矩阵也可以不失一般性地选为对称的.

✍ **练习 3.12** (★★★) 考虑一个实对称矩阵 $\boldsymbol{\Sigma}$, 其特征方程由(3.30)给出. 通过取这个方程的复共轭, 并减去原方程, 然后计算与特征向量 $\boldsymbol{u}_i$ 的内积, 证明特征值 λ_i 是实数. 类似地, 利用 $\boldsymbol{\Sigma}$ 的对称性, 证明当 $\lambda_i \neq \lambda_j$ 时, 特征向量 $\boldsymbol{u}_i$ 和 $\boldsymbol{u}_j$ 是正交的. 最后, 请证明特征向量集可以不失一般性地选择为正交的, 使得它们满足(3.31), 即使一些特征值为零.

✍ **练习 3.13** (★) 证明(3.30)这样一个具有特征向量方程的实对称矩阵 $\boldsymbol{\Sigma}$ 可以表示为形如(3.33)的特征向量的展开形式, 且系数由特征值给出. 类似地, 证明逆矩阵 $\boldsymbol{\Sigma}^{-1}$ 具有形如(3.34)的表示形式.

✍ **练习 3.14** (★★) 正定矩阵 $\boldsymbol{\Sigma}$ 可以定义如下: 对于任意实向量 $\boldsymbol{a}$, 其二次型

$$\boldsymbol{a}^{\mathsf{T}}\boldsymbol{\Sigma}\boldsymbol{a}$$

是正的. 证明 $\boldsymbol{\Sigma}$ 为正定的一个充要条件是,$\boldsymbol{\Sigma}$ 的所有特征值 λ_i 都是正的, 其中特征值由(3.30)定义.

✍ **练习 3.15** (★) 证明一个大小为 $D \times D$ 的实对称矩阵具有 $D(D+1)/2$ 个独立的参数.

✍ **练习 3.16** (★) 证明对称矩阵的逆矩阵也是对称的.

✍ **练习 3.17** (★★) 通过使用特征向量展开式(3.33)对坐标系进行对角化, 证明与常数 Mahalanobis 距离 Δ 对应的超椭球的体积为

$$V_D\, |\boldsymbol{\Sigma}|^{1/2}\, \Delta^D,$$

其中 V_D 是 D 维单位球的体积, 而 Mahalanobis 距离由(3.29)定义.

✍ **练习 3.18** (★★) 利用定义(3.63), 并通过对它的两边乘以下列矩阵, 证明恒等式(3.62)成立.

$$\begin{pmatrix} \boldsymbol{A} & \boldsymbol{B} \\ \boldsymbol{C} & \boldsymbol{D} \end{pmatrix}$$

✍ **练习 3.19** (★★★)小节 3.2.4和小节 3.2.5介绍了多元 Gauss 分布的条件分布和边缘分布. 更一般地, 可以将 $\boldsymbol{x}$ 的分量划分为三组——$\boldsymbol{x}_a$、$\boldsymbol{x}_b$ 和 $\boldsymbol{x}_c$, 对应地, 将均值向量 $\boldsymbol{\mu}$ 和协方差矩阵 $\boldsymbol{\Sigma}$ 也按照如下形式进行划分:

$$\boldsymbol{\mu} = \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \\ \boldsymbol{\mu}_c \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \begin{pmatrix} \boldsymbol{\Sigma}_{aa} & \boldsymbol{\Sigma}_{ab} & \boldsymbol{\Sigma}_{ac} \\ \boldsymbol{\Sigma}_{ba} & \boldsymbol{\Sigma}_{bb} & \boldsymbol{\Sigma}_{bc} \\ \boldsymbol{\Sigma}_{ca} & \boldsymbol{\Sigma}_{cb} & \boldsymbol{\Sigma}_{cc} \end{pmatrix}.$$

利用节 3.2的结论, 给出条件分布 $p(\boldsymbol{x}_a \mid \boldsymbol{x}_b)$ 的表达式, 其中 $\boldsymbol{x}_c$ 已被边缘化.

✍ **练习 3.20** (★★) 线性代数中一个非常有用的结论是 Woodbury 矩阵求逆公式, 即

$$(\boldsymbol{A} + \boldsymbol{BCD})^{-1} = \boldsymbol{A}^{-1} - \boldsymbol{A}^{-1}\boldsymbol{B}(\boldsymbol{C}^{-1} + \boldsymbol{D}\boldsymbol{A}^{-1}\boldsymbol{B})^{-1}\boldsymbol{D}\boldsymbol{A}^{-1}.$$

通过对(3.62)的两边乘以 $(\boldsymbol{A} + \boldsymbol{BCD})$, 证明这个结论的正确性.

✍ **练习 3.21** (★) 假设 $\boldsymbol{x}$ 和 z 是两个独立的随机向量, 故 $p(\boldsymbol{x}, z) = p(\boldsymbol{x})p(z)$. 证明这两个随机向量的和 $\boldsymbol{y} = \boldsymbol{x} + z$ 的均值等于其中每个随机向量的均值之和. 类似地, 证明 $\boldsymbol{y}$ 的协方差矩阵等于 $\boldsymbol{x}$ 与 z 的协方差矩阵之和.

✍ **练习 3.22** (★★★) 考虑变量

$$z = \begin{pmatrix} x \\ y \end{pmatrix}$$

的联合分布, 它的均值和协方差分别由(3.95)和(3.92)给出. 利用(3.79)和(3.80), 证明边缘分布 $p(\boldsymbol{x})$ 由(3.86)给出. 类似地, 利用(3.67)和(3.68), 证明条件分布 $p(\boldsymbol{y} \mid \boldsymbol{x})$ 由(3.87)给出.

✍ **练习 3.23** (★★) 请利用分块矩阵求逆公式 [(3.62)], 证明精度矩阵 [(3.91)] 的逆矩阵由协方差矩阵 [(3.92)] 给出.

✍ **练习 3.24** (★★) 从(3.94)开始, 利用(3.92), 验证(3.95).

✍ **练习 3.25** (★★) 考虑两个多维随机向量 $\boldsymbol{x}$ 和 z, 它们分别服从 Gauss 分布 $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_x, \boldsymbol{\Sigma}_x)$ 和 $p(z) = \mathcal{N}(z \mid \mu_z, \Sigma_z)$, 定义 $\boldsymbol{y} = \boldsymbol{x} + z$. 通过考虑由边缘分布 $p(\boldsymbol{x})$ 和条件分布 $p(\boldsymbol{y} \mid \boldsymbol{x})$ 的乘积组成的线性-Gauss 模型, 并利用(3.96)和(3.97), 证明边缘分布 $p(\boldsymbol{y})$ 由下式给出:

$$p(\boldsymbol{y}) = \mathcal{N}(\boldsymbol{y} \mid \boldsymbol{\mu}_x + \boldsymbol{\mu}_z, \boldsymbol{\Sigma}_x + \boldsymbol{\Sigma}_z).$$

✍ **练习 3.26** (★★★) 本习题和下一习题提供了关于操作线性-Gauss 模型中出现的二次型的实践, 它们也可以作为对正文中结论的独立检查. 考虑由(3.86)和(3.87)给出的边缘分布和条件分布定义的联合分布 $p(\boldsymbol{x}, \boldsymbol{y})$. 通过检查联合分布指数中的二次型, 并使用小节 3.2.4讨论的“完成平方”技术, 给出边缘分布 $p(\boldsymbol{y})$ 的均值表达式和协方差表达式, 其中变量 $\boldsymbol{x}$ 已经被积分消除. 为此, 利用 Woodbury 矩阵求逆公式 [(3.62)], 验证这些结果与(3.96)和(3.97)的一致性.

✍ **练习 3.27** (★★★) 考虑与练习 3.26中相同的联合分布, 但使用“完成平方”技术来找到条件分布 $p(\boldsymbol{x} \mid \boldsymbol{y})$ 的均值表达式和协方差表达式. 再次验证这些结果与(3.98)和(3.99)的一致性.

✍ **练习 3.28** (★★) 为了找到多元 Gauss 分布的协方差矩阵的最大似然解, 需要最大化对于 $\boldsymbol{\Sigma}$ 的对数似然函数 [(3.106)], 注意协方差矩阵必须是对称且正定的. 此处忽略这些约束, 直接进行最大化. 利用节 A.3中的矩阵求导法则, 证明使对数似然函数 [(3.106)] 最大化的协方差矩阵 $\boldsymbol{\Sigma}$ 是由样本协方差 [(3.110)] 给出的. 注意最终的结果必然是对称且正定的 (假设样本协方差是非奇异的).

✍ **练习 3.29** (★★) 使用结果(3.44)证明(3.48). 然后, 利用(3.44)和(3.48)证明

$$\mathbb{E}\left[\boldsymbol{x}_n \boldsymbol{x}_m^{\mathsf{T}}\right] = \boldsymbol{\mu}\boldsymbol{\mu}^{\mathsf{T}} + I_{nm}\boldsymbol{\Sigma},$$

其中 $\boldsymbol{x}_n$ 表示从均值为 $\boldsymbol{\mu}$、协方差为 $\boldsymbol{\Sigma}$ 的 Gauss 分布中采样得到的数据点, 而 I_{nm} 表示单位矩阵中位于 (n, m) 的元素. 最后证明(3.112).

✍ **练习 3.30** (★) 本章在讨论周期变量时使用的各种三角恒等式可以很容易地从下面的关系中证明:

$$\exp(\mathrm{i}A) = \cos A + \mathrm{i}\sin A,$$

其中 $\mathrm{i} = \sqrt{-1}$. 通过考虑恒等式 $\exp(\mathrm{i}A)\exp(-\mathrm{i}A) = 1$ 证明(3.131). 类似地, 使用恒等式 $\cos(A - B) = \Re\exp\{\mathrm{i}(A - B)\}$ 证明(3.132), 其中 $\Re$ 表示实部. 最后, 使用 $\sin(A - B) = \Im\exp\{\mathrm{i}(A - B)\}$(其中 $\Im$ 表示虚部) 证明(3.137).

✍ **练习 3.31** (★★) 对于较大的 m,von Mises 分布 [(3.133)] 将会在众数 θ_0 处变得十分尖锐. 通过定义 $\xi = m^{1/2}(\theta - \theta_0)$, 并利用余弦函数的 Taylor 展开式, 即

$$\cos\alpha = 1 - \frac{\alpha^2}{2} + O(\alpha^4),$$

证明当 $m \to \infty$ 时,von Mises 分布趋向于 Gauss 分布.

✍ **练习 3.32** (★) 利用三角恒等式(3.137), 证明(3.136)对于 θ_0 的解由(3.138)给出.

✍ **练习 3.33** (★) 通过计算(3.133)给出的 von Mises 分布的一阶导数和二阶导数, 并利用 $I_0(m) > 0 (m > 0)$, 证明该分布的极大值发生在 $\theta = \theta_0$ 处, 极小值发生在 $\theta = \theta_0 + \pi \pmod{2\pi}$ 处.

✍ **练习 3.34** (★) 通过利用结果(3.122), 并结合(3.138)和三角恒等式(3.132), 证明 von Mises 分布的聚焦参数的最大似然解 m_{ML} 满足 $A(m_{\mathrm{ML}}) = \bar{r}$, 其中 $\bar{r}$ 是观测值的均值的半径. 如图 3.9所示, 它可以视为二维欧氏平面上的单位向量.

✍ **练习 3.35** (★) 验证多元 Gauss 分布可以表示为指数族分布的形式 [(3.142)], 并为 $\boldsymbol{\eta}$、$\boldsymbol{u}(\boldsymbol{x})$、$h(\boldsymbol{x})$ 和 $g(\boldsymbol{\eta})$ 推导类似于(3.161)至(3.164)的表达式.

✍ **练习 3.36** (★)(3.169)表明, 指数族分布中 $\ln g(\boldsymbol{\eta})$ 的负梯度由 $\boldsymbol{u}(\boldsymbol{x})$ 的期望给出. 通过对(3.143)求二阶导数, 证明

$$-\nabla\nabla \ln g(\boldsymbol{\eta}) = \mathbb{E}\left[\boldsymbol{u}(\boldsymbol{x})\boldsymbol{u}(\boldsymbol{x})^{\mathsf{T}}\right] - \mathbb{E}[\boldsymbol{u}(\boldsymbol{x})]\,\mathbb{E}[\boldsymbol{u}(\boldsymbol{x})]^{\mathsf{T}} = \operatorname{cov}[\boldsymbol{u}(\boldsymbol{x})].$$

✍ **练习 3.37** (★★) 考虑一个类似直方图的密度模型, 其中空间 $\boldsymbol{x}$ 被划分为固定的区域, 密度 $p(\boldsymbol{x})$ 在第 i 个区域上取常数值 h_i. 第 i 个区域的体积表示为 Δ_i. 假设有一个集合, 里面包含 N 个 $\boldsymbol{x}$ 的观测值, 其中 n_i 个观测值落在第 i 个区域. 利用 Lagrange 乘子确保密度的归一化约束, 并推导 $\{h_i\}$ 的最大似然估计量的表达式.

✍ **练习 3.38** (★) 证明 K 近邻密度模型定义了一个反常分布, 其在整个空间上的积分是发散的.

第 4 章 单层网络: 回归

在之前的多项式曲线拟合问题中, 我们曾简单了解过线性回归 (参见节 1.2). 而在本章中, 我们将继续通过它的框架来讨论神经网络背后的一些基本思想. 你将看到, 线性回归模型对应简单的具有单层可学习参数的神经网络. 虽然单层网络的应用场景非常有限, 但它们具有一些简单的解析性质, 并为后续引入许多核心概念提供了一个优秀的框架, 这些概念将为我们在后续章节中讨论深度神经网络奠定基础.

4.1 线性回归

给定 D 维输入变量 $\boldsymbol{x}$, 回归的目标是预测一个或多个连续的目标变量 t. 通常我们会有一个包含 N 个观测值的训练数据集 $\{\boldsymbol{x}_n\}$, 其中 $n = 1, \cdots, N$, 以及相应的目标值 $\{t_n\}$. 目标是为新的 $\boldsymbol{x}$ 值预测对应的 t 值. 为此, 我们定义了函数 $y(\boldsymbol{x}, \boldsymbol{w})$ 来预测新的输入变量 $\boldsymbol{x}$ 所对应的 t 值, 其中 $\boldsymbol{w}$ 表示可以从训练数据中学习的参数向量.

4.1.1 基函数

最简单的回归模型是输入变量的线性组合:

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_0 + w_1 x_1 + \cdots + w_D x_D \tag{4.1}$$

定义 4.1 (线性回归模型)

在(4.1)中,$\boldsymbol{x} = (x_1, \cdots, x_D)^{\mathsf{T}}$ 是输入变量,$\boldsymbol{w} = (w_0, \cdots, w_D)^{\mathsf{T}}$ 是从训练数据中学习的参数向量. 形如(4.1)的模型称为**线性回归** (linear regression) 模型.

♣

注

(重构新增) 定义 4.1把“回归”这一任务限定在参数线性的一族函数上, 这一点是本章全部解析结果的来源.y 关于参数 $w_0, \cdots, w_D$ 是线性的, 因此平方和误差关于 $\boldsymbol{w}$ 是二次函数, 其驻点由一组线性方程 (正规方程) 刻画, 从而有唯一的闭式解(4.16).

需要强调的是,(4.1)中“线性”一词指的是对参数 $\boldsymbol{w}$ 线性, 而不是对输入 $\boldsymbol{x}$ 线性;定义 4.2将明确这一点. 若 y 对 $\boldsymbol{w}$ 不是线性的, 则误差函数的驻点方程不再是线性方程组,(4.16)乃至本章后续的所有闭式结论都将失效.

这种模型的关键性质在于它不仅是参数 $w_0, \cdots, w_D$ 的线性函数, 而且也是输入变量 x_i 的线性函数. 然而, 正是其作为输入变量 x_i 的线性函数这一特性, 给模型带来了显著的限制.

注

(重构新增) “对输入也线性”这一附加性质带来了定义 4.1的两重限制. 其一, 模型无法表示输入之间的交互作用: 若真实规律含有 $x_1 x_2$ 项, 则任何 $w_0 + w_1 x_1 + w_2 x_2$ 都无法逼近. 其二, 当输入维数 D 较大时, 可用的模型族仍然只有 $D + 1$ 个自由度, 表达能力与维数无关地停留在同一水平上.

正是为了摆脱第一重限制, 下面用固定非线性基函数的线性组合来替换输入本身; 而“对参数线性”这一条被完整保留下来, 从而全部解析性质得以保持. 这一“特征变换 + 参数线性”的折中, 是本章其余各节共同的前提.

我们可以通过输入变量的固定非线性函数的线性组合来扩展(4.1)定义的模型:

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_0 + \sum_{j=1}^{M-1} w_j \phi_j(\boldsymbol{x}) \tag{4.2}$$

定义 4.2 (基函数与偏置参数)

(4.2)中的固定函数 $\phi_j(\boldsymbol{x})$ 称为**基函数** (basis function). 若用 $M-1$ 表示索引 j 的最大值, 则模型中参数的总数为 M. 参数 w_0 代表数据中一个任意的固定偏移量, 有时称为**偏置参数** (bias)(注意不要与统计学意义上的偏差弄混淆, 参见节 4.3). 通过定义一个额外的虚拟基函数 $\phi_0(\boldsymbol{x}) = 1$,(4.2)可以统一为

$$y(\boldsymbol{x}, \boldsymbol{w}) = \sum_{j=0}^{M-1} w_j \phi_j(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}) \tag{4.3}$$

其中 $\boldsymbol{w} = (w_0, \cdots, w_{M-1})^{\mathsf{T}}$,$\boldsymbol{\phi} = (\phi_0, \cdots, \phi_{M-1})^{\mathsf{T}}$. 形如(4.3)、关于参数 $\boldsymbol{w}$ 为线性的模型称为**线性模型** (linear model).

♣

注

(重构新增) 定义 4.2逐项规定了本章模型的三个组成部分, 每一项都在后文被实际使用.

第一, 基函数 ϕ_j 必须是固定的, 即在看到数据之前就已选定; 它不含可学习参数, 因此不进入 $E_D(\boldsymbol{w})$ 的优化变量. 正是这一条使(4.3)关于 $\boldsymbol{w}$ 保持线性.

第二, 参数总数恰为 M(索引 j 取 $0, \cdots, M-1$), 这个 M 既是(4.15)中 $\boldsymbol{\Phi}$ 的列数, 也是小节 4.1.4中线性子空间 $\mathcal{S}$ 的维数; 本章反复出现的“基函数的个数”指的都是它.

第三, 虚拟基函数 $\phi_0 \equiv 1$ 把偏置 w_0 吸收进内积形式(4.3), 使“带偏置的线性模型”与“不带偏置的线性模型”可以用同一套公式处理. 代价是 $\boldsymbol{w}$ 与 $\boldsymbol{\phi}(\boldsymbol{x})$ 的维数都增加 1; 若不引入 ϕ_0, 则(4.15)中的第一列恒为 1 这一事实就必须单独处理.

通过使用非线性基函数, 函数 $y(\boldsymbol{x}, \boldsymbol{w})$ 变为关于输入变量 $\boldsymbol{x}$ 的非线性函数. 尽管如此, 形如(4.2)的函数仍然称为线性模型, 因为它们关于 $\boldsymbol{w}$ 是线性的. 正是这种关于参数线性的性质大大简化了对这类模型的分析. 然而, 它也带来了一些显著的限制 (参见节 6.1).

我们可以用神经网络图来表示该模型 [式(4.3)], 如图 4.1所示.

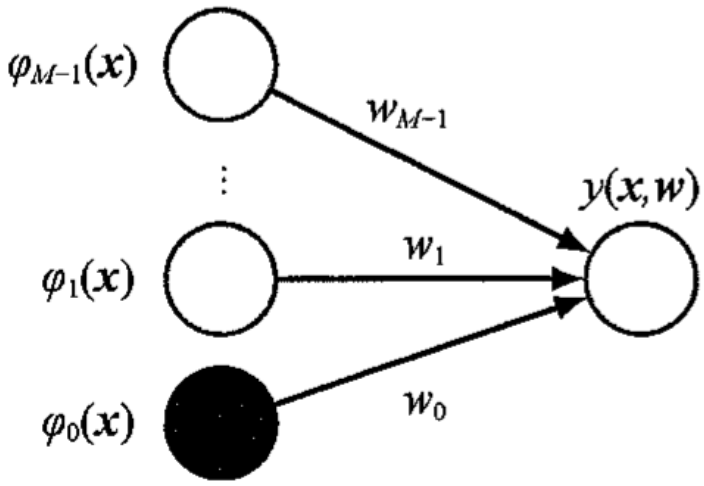


图 4.1: 线性回归模型 [式(4.3)] 可以表示为一个简单的由单层参数构成的神经网络. 这里的每个基函数 $\phi_j(\boldsymbol{x})$ 用一个输入节点表示, 实心节点表示“偏置”基函数 ϕ_0, 函数 $y(\boldsymbol{x}, \boldsymbol{w})$ 用输出节点表示. 每个参数 w_j 由一条连接相应基函数和输出的线表示

在深度学习出现之前, 机器学习中通常会对输入变量 $\boldsymbol{x}$ 进行某种形式的固定预处理, 称为**特征提取** (feature extraction), 其可以用一组基函数 $\{\phi_j(\boldsymbol{x})\}$ 来表示. 特征提取的目标是选择一组足够强大的基函数, 从而可以使用简单的网络模型解决学习任务. 遗憾的是, 除了最简单的应用之外, 人工选取合适的基函数是非常困难的. 深度学习仅通过数据集学习所需的数据非线性变换来避开这个问题.

之前讨论使用多项式进行曲线拟合时, 我们已经见到了一个回归问题的例子 (参见节 1.2). 如果我们考虑单个输入变量 x, 并选择由 $\phi_j(x) = x^j$ 定义的基函数, 则多项式函数(1.1)可以表示为(4.3)的形式. 此外, 基函数还有其他

的形式, 例如:

$$\phi_j(x) = \exp\left\{-\frac{(x-\mu_j)^2}{2s^2}\right\} \tag{4.4}$$

其中 μ_j 控制基函数在输入空间中的位置, 参数 s 控制它们的空间尺度. 这种形式的基函数通常叫作 Gauss 基函数, 虽然它们在形式上可能类似于 Gauss 分布, 但我们在使用时并不需要概率化的解释. 特别地, 这些基函数会被乘以可学习参数 w_j, 因此归一化系数不重要.

基函数的另一种形式是 sigmoid 型基函数:

$$\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right) \tag{4.5}$$

其中 $\sigma(a)$ 是按如下方式定义的 sigmoid 函数:

$$\sigma(a) = \frac{1}{1+\exp(-a)} \tag{4.6}$$

我们可以等价地使用 tanh 函数, 因为其可以通过 $\tanh(a) = 2\sigma(2a) - 1$ 与 sigmoid 函数相关联.sigmoid 函数的一般线性组合在某种意义上等价于 tanh 函数的一般线性组合, 故它们可以表示同一类输入—输出函数 (见练习 4.3). 不同的基函数选择如图 4.2所示.

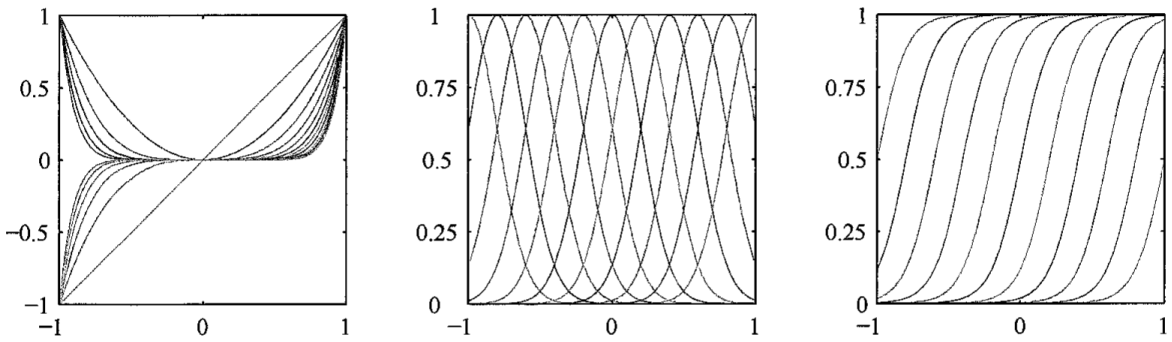


图 4.2: 左图是多项式, 中间图是式(4.4)形式的 Gauss 基函数, 右图是式(4.5)形式的 sigmoid 型基函数

基函数的另一种选择是 Fourier 基函数, 它对应正弦函数的展开. 每个基函数代表无限空间范围内一个特定的频率. 相比之下, 位于输入空间有限区域的基函数必然包含不同空间频率的频谱. 在信号处理的相关应用中, 通常需要考虑在空间和频率上都局部化的一类基函数, 即小波 (wavelet) 基函数 (Ogden, 1997; Mallat, 1999; Vidakovic, 1999). 为了简化应用, 这些函数被定义为相互正交的. 当输入值在规则的格 (lattice) 上时, 小波基函数最为适用, 例如时间序列中连续的时间点或图像中的像素.

然而, 本章后续大部分的讨论与基函数的选择无关, 所以除了数值说明外, 我们不会明确基函数的具体形式. 此外, 为了让符号标记简单, 我们将重点关注单目标变量 t 的情况, 也会简要概述处理多目标变量时所需要做的修改 (参见小节 4.1.7).

4.1.2 似然函数

我们已经通过最小化平方和误差函数解决了多项式拟合数据的问题, 并且我们还发现, 平方和误差函数可以用 Gauss 噪声模型假设下的最大似然解来导出 (参见小节 2.3.4). 下面我们将更详细地介绍最小二乘法, 以及它与最大似然的关系.

定义 4.3 (加性 Gauss 噪声模型)

设目标变量 t 由具有加性 Gauss 噪声的确定性函数 $y(\boldsymbol{x},\boldsymbol{w})$ 给出, 即

$$t = y(\boldsymbol{x},\boldsymbol{w}) + \varepsilon \tag{4.7}$$

其中 ε 是方差为 σ^2 的零均值 Gauss 随机变量. 于是 t 的条件分布为

$$p(t \mid \boldsymbol{x},\boldsymbol{w},\sigma^2) = \mathcal{N}\left(t \mid y(\boldsymbol{x},\boldsymbol{w}),\sigma^2\right) \tag{4.8}$$

♣

注

(重构新增) 定义 4.3的两条假设——噪声可加、噪声为 Gauss 且同方差——是本节全部结论的前提, 缺一不可.

"可加"意味着 t 的均值恰为 $y(\boldsymbol{x},\boldsymbol{w})$, 即模型只平移分布而不改变其形状;"零均值"保证 y 就是条件期望, 否则 y 会系统性地偏离 $\mathbb{E}[t \mid \boldsymbol{x}]$,(4.11)的最小值点也将随之偏移;(4.8)中的方差 σ^2 与 $\boldsymbol{x}$ 无关 (同方差), 这一点是(4.9)能写成连乘、并使 $\boldsymbol{w}$ 与 σ^2 的优化相互分离的关键.

若噪声是异方差的 (方差随 $\boldsymbol{x}$ 变化), "最小化平方和"就不再等价于最大似然, 而应改为加权平方和, 见练习 4.5; 若噪声服从重尾分布, 对应的则是 L_q 误差而非平方误差, 见练习 4.11.

考虑输入数据集 $X = \{\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N\}$ 以及对应的目标值 $t_1,\cdots,t_N$. 将目标变量 $\{t_n\}$ 构造为列向量, 用 $\boldsymbol{t}$ 表示. 假设这些数据点是从分布(4.8)中独立产生的, 我们得到如下似然函数的表达式, 它是一个关于可调参数 $\boldsymbol{w}$ 和 σ^2 的函数:

$$p(\boldsymbol{t} \mid X,\boldsymbol{w},\sigma^2) = \prod_{n=1}^{N} \mathcal{N}\left(t_n \mid \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n),\sigma^2\right) \tag{4.9}$$

其中用到了(4.3). 取似然函数的对数, 并使用 Gauss 分布的标准形式 (参见节 2.3), 我们有

命题 4.1 (回归问题的对数似然)

设 $X = \{\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N\}$,$\boldsymbol{t} = (t_1,\cdots,t_N)^{\mathsf{T}}$, 且各观测按(4.8)独立产生. 则对数似然为

$$\ln p(\boldsymbol{t} \mid X,\boldsymbol{w},\sigma^2) = -\frac{N}{2}\ln\sigma^2 - \frac{N}{2}\ln(2\pi) - \frac{1}{\sigma^2}E_D(\boldsymbol{w}) \tag{4.10}$$

其中平方和误差函数定义为

$$E_D(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\left\{t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right\}^2 \tag{4.11}$$

♠

证明

(重构新增) 记 $\mu_n \triangleq \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)$.Gauss 密度为

$$\mathcal{N}(t \mid \mu,\sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}}\exp\left\{-\frac{(t-\mu)^2}{2\sigma^2}\right\},$$

故对每个 n 有

$$\ln\mathcal{N}(t_n \mid \mu_n,\sigma^2) = -\frac{1}{2}\ln(2\pi\sigma^2) - \frac{(t_n-\mu_n)^2}{2\sigma^2}.$$

在独立性假设下,(4.9)的对数等于各项对数之和, 于是

$$\begin{aligned}\ln p(\boldsymbol{t} \mid X,\boldsymbol{w},\sigma^2) &= \sum_{n=1}^{N}\ln\mathcal{N}(t_n \mid \mu_n,\sigma^2)\\ &= -\frac{N}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}(t_n-\mu_n)^2\\ &= -\frac{N}{2}\ln(2\pi) - \frac{N}{2}\ln\sigma^2 - \frac{1}{\sigma^2}\cdot\frac{1}{2}\sum_{n=1}^{N}(t_n-\mu_n)^2,\end{aligned}$$

其中最后一步用了 $\ln(2\pi\sigma^2) = \ln(2\pi) + \ln\sigma^2$. 右端最后一项的求和正是(4.11)中的 $E_D(\boldsymbol{w})$, 代入即得(4.10).

□

注

(重构新增) 命题 4.1把“独立同分布假设”兑换成了一个具体的代数结构: 对数似然是 $-\ln\sigma^2$ 项、常数项与 $-(1/\sigma^2)E_D(\boldsymbol{w})$ 三项之和. 这一结构说明, 参数 $\boldsymbol{w}$ 只通过 E_D 进入似然, 因此 $\boldsymbol{w}$ 的最优值与 σ^2 无关; 反之 σ^2 的最优值依赖 $\boldsymbol{w}$. 这种“单向耦合”正是下面可以依次求 $\boldsymbol{w}_{\mathrm{ML}}$ 与 σ^2_{ML} 的原因.

若数据点不独立 (例如时间序列), 乘积(4.9)就不能分解为逐点之积,$E_D(\boldsymbol{w})$ 也不再是充分统计量.

命题 4.2 (最大似然等价于最小二乘)

在命题 4.1的条件下, 对任意固定的 $\sigma^2 > 0$, 使对数似然(4.10)最大的 $\boldsymbol{w}$ 与使平方和误差(4.11)最小的 $\boldsymbol{w}$ 相同.

♠

证明

(重构新增) 固定 $\sigma^2 > 0$ 时,(4.10)中的前两项 $-\dfrac{N}{2}\ln\sigma^2$ 与 $-\dfrac{N}{2}\ln(2\pi)$ 都不含 $\boldsymbol{w}$, 因此

$$\ln p(\boldsymbol{t} \mid X, \boldsymbol{w}, \sigma^2) = C - \frac{1}{\sigma^2}E_D(\boldsymbol{w}),$$

其中 C 是与 $\boldsymbol{w}$ 无关的常数. 由于 $1/\sigma^2 > 0$, 对任意 $\boldsymbol{w}_1, \boldsymbol{w}_2$ 成立

$$E_D(\boldsymbol{w}_1) \leqslant E_D(\boldsymbol{w}_2) \iff \ln p(\boldsymbol{t} \mid X, \boldsymbol{w}_1, \sigma^2) \geqslant \ln p(\boldsymbol{t} \mid X, \boldsymbol{w}_2, \sigma^2),$$

即两个目标函数在同一自变量处取到各自的极值, 故两者 argmin 相同.

□

注

(重构新增) 命题 4.2说明了(4.11)之所以被广泛使用, 并不是因为它是一个“自然”的误差度量, 而是因为它在 Gauss 噪声假设下恰是负对数似然 (相差常数与正因子).

反过来看, 这条等价性也界定了平方和的适用边界: 一旦噪声不是 Gauss, 平方和就不再是最大似然的产物, 其最小化点也不再是条件均值. 此外, 等价性只对 $\boldsymbol{w}$ 成立, 对 σ^2 并不成立——σ^2 的最大似然估计是有偏的, 见命题 4.5的注记.

4.1.3 最大似然

在写出似然函数后, 我们可以通过最大似然来确定 $\boldsymbol{w}$ 和 σ^2. 首先对于 $\boldsymbol{w}$ 最大化上述似然函数, 其对数似然函数 [式(4.10)] 关于 $\boldsymbol{w}$ 的梯度为如下形式:

$$\nabla_{\boldsymbol{w}} \ln p(\boldsymbol{t} \mid X, \boldsymbol{w}, \sigma^2) = \frac{1}{\sigma^2}\sum_{n=1}^{N}\big(t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\big)\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}} \tag{4.12}$$

将梯度设置为 $\boldsymbol{0}$, 可得

$$\boldsymbol{0} = \sum_{n=1}^{N} t_n\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}} - \boldsymbol{w}^{\mathsf{T}}\left(\sum_{n=1}^{N}\boldsymbol{\phi}(\boldsymbol{x}_n)\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}}\right) \tag{4.13}$$

这称为最小二乘问题的正规方程 (normal equation).

定义 4.4 (设计矩阵)

设 M 为基函数的个数. 由

$$\Phi_{nj} = \phi_j(\boldsymbol{x}_n), \qquad n = 1, \cdots, N, \quad j = 0, \cdots, M-1 \tag{4.14}$$

定义的 $N \times M$ 矩阵称为**设计矩阵** (design matrix), 即

$$\boldsymbol{\Phi} = \begin{pmatrix} \phi_0(\boldsymbol{x}_1) & \phi_1(\boldsymbol{x}_1) & \cdots & \phi_{M-1}(\boldsymbol{x}_1) \\ \phi_0(\boldsymbol{x}_2) & \phi_1(\boldsymbol{x}_2) & \cdots & \phi_{M-1}(\boldsymbol{x}_2) \\ \vdots & \vdots & & \vdots \\ \phi_0(\boldsymbol{x}_N) & \phi_1(\boldsymbol{x}_N) & \cdots & \phi_{M-1}(\boldsymbol{x}_N) \end{pmatrix} \tag{4.15}$$

它的第 n 行是 $\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}}$, 第 j 列是基函数 ϕ_j 在全部数据点上的取值.

♣

注

(重构新增) 定义 4.4的作用是把“对 N 个数据点求和”这一逐点运算一次性打包成矩阵乘法, 从而使(4.13)可以写成 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{w} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}$.

矩阵 $\boldsymbol{\Phi}$ 的规模为 $N \times M$, 两个方向的行列数分别对应“数据点的个数”与“基函数的个数”. 这两个数的相对大小决定了小节 4.1.4中的几何图像 ($M < N$ 时存在 M 维子空间), 也决定了 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 是否可逆: 当 $M > N$ 时 $\boldsymbol{\Phi}$ 的列必定线性相关,$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 奇异,(4.16)中的逆不存在, 此时必须改用伪逆(4.17)或引入正则化项 (见小节 4.1.6).

命题 4.3 (最大似然解)

设 $\boldsymbol{\Phi}$ 是定义 4.4给出的 $N \times M$ 设计矩阵,$\boldsymbol{t} = (t_1, \cdots, t_N)^{\mathsf{T}}$, 并设各列 $\phi_0, \cdots, \phi_{M-1}$ 线性无关 (从而 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 可逆). 则使对数似然(4.10)最大的参数向量为

$$\boldsymbol{w}_{\mathrm{ML}} = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t} \tag{4.16}$$

♠

证明

(重构新增) 由命题 4.2, 只需最小化 $E_D(\boldsymbol{w})$, 即解(4.12)为零的方程.

第一步, 把(4.12)中的每一项写成矩阵形式. 由于 $t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)$ 是标量,(4.12)右端的第 n 项等于 $\big(t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\big)\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}}$; 而 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}} = \boldsymbol{w}^{\mathsf{T}}\big(\boldsymbol{\phi}(\boldsymbol{x}_n)\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}}\big)$. 令梯度为零并乘以非零常数 σ^2, 得(4.13).

第二步, 把(4.13)中的两个求和用 $\boldsymbol{\Phi}$ 表示. 按定义 4.4,$\boldsymbol{\Phi}$ 的第 n 行为 $\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}}$, 因此

$$\sum_{n=1}^{N} \boldsymbol{\phi}(\boldsymbol{x}_n)\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}, \qquad \sum_{n=1}^{N} t_n\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}} = \boldsymbol{t}^{\mathsf{T}}\boldsymbol{\Phi}.$$

第一步中右端的第一个求和是二元和 $\sum_n t_n\phi_j(\boldsymbol{x}_n)$, 它确实等于 $(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t})^{\mathsf{T}} = \boldsymbol{t}^{\mathsf{T}}\boldsymbol{\Phi}$.

第三步, 把(4.13)转置. 等式左端 $\boldsymbol{0}$ 转置后仍为 $\boldsymbol{0}$, 故

$$\boldsymbol{0} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t} - \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{w},$$

其中用到了 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 的对称性. 移项得

$$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{w} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}.$$

第四步, 求解. 由假设 $\boldsymbol{\Phi}$ 的 M 列线性无关,$M \times M$ 矩阵 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 正定, 从而可逆, 两端左乘 $(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}$ 即得(4.16).

最后说明它给出的是最小值而非最大值.E_D 的 Hesse 矩阵为

$$\nabla_{\boldsymbol{w}}^2 E_D = \sum_{n=1}^{N} \boldsymbol{\phi}(\boldsymbol{x}_n)\boldsymbol{\phi}(\boldsymbol{x}_n)^{\mathsf{T}} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi},$$

它在列线性无关的假设下正定, 故 E_D 是严格凸二次函数,(4.16)是其唯一全局最小值点; 由命题 4.2, 它也是对数似然的唯一最大值点.

□

注

(重构新增) 命题 4.3的结论(4.16)成立的前提有三条, 需要分别核对.

其一,$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 必须可逆. 这只依赖于“M 个基函数向量在 $\mathbb{R}^N$ 中线性无关”, 而不要求 $N \geqslant M$ 以外的任何条件; 当 $M > N$ 时该条件必然失败. 此时(4.16)无意义, 最小二乘问题仍有无穷多个解 (均给出相同的拟合值), 通常取范数最小者, 即由伪逆(4.17)给出.

其二,$\sigma^2 > 0$ 必须成立, 否则(4.12)中的因子 $1/\sigma^2$ 无定义. 这一条件在实际数据中自然满足; 若 $\sigma^2 \to 0$, 则似然退化为对每个数据点的硬约束.

其三,(4.16)只用到 $\boldsymbol{\Phi}$, 与 $\boldsymbol{t}$ 的具体取值无关. 这意味着设计矩阵可以在看到目标值之前就预先算好其伪逆, 这在小节 4.1.7中将被用到.

最后,(4.16)在数值上并不总是可靠的: 当 $\boldsymbol{\Phi}$ 的列近似共线时,$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 接近奇异, 其逆的元素会非常大, 微小的数据扰动会导致 $\boldsymbol{w}_{\mathrm{ML}}$ 剧烈变化. 这正是小节 4.1.4末尾引入 SVD 与小节 4.1.6引入正则化的动机.

定义 4.5 (Moore-Penrose 伪逆)

矩阵

$$\boldsymbol{\Phi}^{\dagger} \equiv (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}} \tag{4.17}$$

称为矩阵 $\boldsymbol{\Phi}$ 的 Moore-Penrose 伪逆 (Moore-Penrose pseudo-inverse)(Rao and Mitra, 1971; Golub and Van Loan, 1996). 这可以看作矩阵逆的概念被推广到非方阵的情况. 如果 $\boldsymbol{\Phi}$ 是方阵且可逆, 则利用性质 $(\boldsymbol{AB})^{-1} = \boldsymbol{B}^{-1}\boldsymbol{A}^{-1}$ 可得 $\boldsymbol{\Phi}^{\dagger} = \boldsymbol{\Phi}^{-1}$.

注

(重构新增) (4.17)把“求解线性方程组”改写成了“乘以一个固定的矩阵”, 这是本节在计算上最有价值的一步: 一旦设计矩阵确定,$\boldsymbol{\Phi}^{\dagger}$ 就可以预先算好, 对任意新的目标向量 $\boldsymbol{t}$ 只需一次矩阵乘法.

称其为“推广”的依据是: 当 $\boldsymbol{\Phi}$ 为可逆方阵时,$\boldsymbol{\Phi}^{\dagger} = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}} = \boldsymbol{\Phi}^{-1}(\boldsymbol{\Phi}^{\mathsf{T}})^{-1}\boldsymbol{\Phi}^{\mathsf{T}} = \boldsymbol{\Phi}^{-1}$, 即伪逆退化为通常的逆. 当 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 奇异时, 上式中的逆不存在, 此时 Moore-Penrose 伪逆需改用奇异值分解来定义; 本节的公式(4.17)只覆盖列满秩的情形.

至此, 我们对偏置参数 w_0 的作用有了进一步的了解. 如果我们显式考虑偏置参数, 则误差函数 [式(4.11)] 就会变成

$$E_D(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\left\{t_n - w_0 - \sum_{j=1}^{M-1} w_j\phi_j(\boldsymbol{x}_n)\right\}^2 \tag{4.18}$$

命题 4.4 (偏置参数的显式解)

设 E_D 由(4.18)给出, 并定义

$$\bar{t} = \frac{1}{N}\sum_{n=1}^{N} t_n, \qquad \bar{\phi}_j = \frac{1}{N}\sum_{n=1}^{N}\phi_j(\boldsymbol{x}_n) \tag{4.19}$$

为训练集上目标值与基函数值的平均值. 则使上述误差函数关于 w_0 的导数为 0 的 w_0 为

$$w_0 = \bar{t} - \sum_{j=1}^{M-1} w_j\bar{\phi}_j \tag{4.20}$$

证明

(重构新增) 记 $r_n \triangleq t_n - w_0 - \sum_{j=1}^{M-1} w_j \phi_j(\boldsymbol{x}_n)$, 则(4.18)为 $E_D = \frac{1}{2}\sum_n r_n^2$.

第一步, 求导. 由链式法则,

$$\frac{\partial E_D}{\partial w_0} = \sum_{n=1}^{N} r_n \frac{\partial r_n}{\partial w_0} = -\sum_{n=1}^{N} r_n,$$

因为 $\partial r_n / \partial w_0 = -1$.

第二步, 令导数为零:

$$\sum_{n=1}^{N} \left(t_n - w_0 - \sum_{j=1}^{M-1} w_j \phi_j(\boldsymbol{x}_n) \right) = 0.$$

把求和拆开, 注意 w_0 与 j 都不依赖 n:

$$\sum_{n=1}^{N} t_n - N w_0 - \sum_{j=1}^{M-1} w_j \sum_{n=1}^{N} \phi_j(\boldsymbol{x}_n) = 0.$$

第三步, 两边除以 N 并解出 w_0:

$$w_0 = \frac{1}{N}\sum_{n=1}^{N} t_n - \sum_{j=1}^{M-1} w_j \left(\frac{1}{N} \sum_{n=1}^{N} \phi_j(\boldsymbol{x}_n) \right) = \bar{t} - \sum_{j=1}^{M-1} w_j \bar{\phi}_j,$$

其中最后一步用了(4.19).

第四步, 验证这是最小值.$\partial^2 E_D / \partial w_0^2 = N > 0$, 故 E_D 作为 w_0 的函数是开口向上的二次函数,(4.20)给出其唯一最小值点.

□

注

(重构新增) 命题 4.4给出的(4.20)是本章中唯一一个“只需一次求导就能读出全部含义”的结论:w_0 补偿了目标值的样本均值 $\bar{t}$ 与基函数加权均值 $\sum_j w_j \bar{\phi}_j$ 之间的差.

这条公式解释了为什么在定义 4.2中把 w_0 单独拿出来讨论是合理的: 其余参数 $w_1, \cdots, w_{M-1}$ 的最小值完全不受 w_0 影响 (在(4.18)中先对 w_0 最优化, 等价于把数据与基函数都中心化), 因此 w_0 可以被“消去”, 只留下一个 $M-1$ 维的优化问题.

由此也可以看出,$\bar{t}$ 依赖目标变量的原点选取: 若把所有 t_n 平移一个常数,w_0 就相应平移同样的量, 而 $w_1, \cdots, w_{M-1}$ 不变. 这正是小节 1.2.5中“正则化项通常不包含 w_0”的理由——对 w_0 施加惩罚会使结果依赖于目标变量原点的选择.

因此, 偏置 w_0 补足了目标值平均值 (在训练集上) 的加权和与基函数值平均值的加权和之间的差异.

我们还可以使对数似然函数(4.10)关于方差 σ^2 最大化.

命题 4.5 (方差的最大似然解)

设 $\boldsymbol{w}_{\mathrm{ML}}$ 由(4.16)给出. 则对数似然(4.10)关于 σ^2 的最大值点为

$$\sigma^2_{\mathrm{ML}} = \frac{1}{N}\sum_{n=1}^{N} \left\{ t_n - \boldsymbol{w}_{\mathrm{ML}}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}_n) \right\}^2 \tag{4.21}$$

♠

证明

(重构新增) 把(4.10)看作 σ^2 的函数 ($\boldsymbol{w}$ 固定), 记 $s \triangleq \sigma^2, E_D \triangleq E_D(\boldsymbol{w})$, 则

$$L(s) = -\frac{N}{2}\ln s - \frac{N}{2}\ln(2\pi) - \frac{E_D}{s}.$$

第一步, 求导:

$$\frac{\mathrm{d}L}{\mathrm{d}s} = -\frac{N}{2s} + \frac{E_D}{s^2}.$$

第二步, 令导数为零并乘以 $2s^2 > 0$:

$$-Ns + 2E_D = 0 \quad \Longrightarrow \quad s = \frac{2E_D}{N} = \frac{1}{N}\sum_{n=1}^{N}\{t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\}^2.$$

第三步, 验证是最大值. 二阶导数为

$$\frac{\mathrm{d}^2 L}{\mathrm{d}s^2} = \frac{N}{2s^2} - \frac{2E_D}{s^3},$$

在 $s = 2E_D/N$ 处等于 $\frac{N}{2}\cdot\frac{N^2}{4E_D^2} - \frac{2E_D N^3}{8E_D^3} = \frac{N^3}{8E_D^2} - \frac{N^3}{4E_D^2} = -\frac{N^3}{8E_D^2} < 0$(设 $E_D > 0$), 故该驻点是最大值点.

第四步, 代入 $\boldsymbol{w} = \boldsymbol{w}_{\mathrm{ML}}$. 对数似然对 $\boldsymbol{w}$ 与 σ^2 的驻点条件互相独立 (见命题 4.1后的注记), 故把(4.16)代入上式即得(4.21).

□

注

(重构新增) 命题 4.5的结论(4.21)说明: 方差的最大似然估计就是“回归函数周围目标值的残差方差”.

需要强调两点. 第一,(4.21)中的分母是 N 而不是 $N - M$, 因此 σ^2_{ML} 是 σ^2 的有偏估计, 它系统性地偏小; 无偏估计应把分母改为 $N - M$, 因为拟合时已经用掉了 M 个自由度. 第二, 只有当 $\boldsymbol{w}$ 取最大似然值时才可以把(4.21)代回似然, 否则 σ^2 的最优值应使用相应的残差平方和.

这两点与(4.16)的有偏性无关:$\boldsymbol{w}_{\mathrm{ML}}$ 本身是无偏的 (在定义 4.3的模型下 $\mathbb{E}[\boldsymbol{w}_{\mathrm{ML}}] = \boldsymbol{w}$), 只有 σ^2_{ML} 有偏.

可以看到, 方差参数的最大似然值是由回归函数周围目标值的残差方差给出的.

4.1.4 最小二乘的几何表示

下面讨论最小二乘解的几何解释. 考虑一个 N 维空间, 其轴由 t_n 给出, 因此 $\boldsymbol{t} = (t_1, \cdots, t_N)^{\mathsf{T}}$ 是这个空间中的一个向量. 如图 4.3所示, 每个在 N 个数据点上求值的基函数 $\phi_j(\boldsymbol{x})$ 可以表示为相同空间中的一个向量, 用 $\boldsymbol{\phi}_j$ 表示.$\boldsymbol{\phi}_j$ 是 $\boldsymbol{\Phi}$ 的第 j 列, 而 $\boldsymbol{\phi}(\boldsymbol{x}_n)$ 是 $\boldsymbol{\Phi}$ 的第 n 行的转置. 如果基函数的个数 M 小于数据点的个数 N, 则 M 个 $\boldsymbol{\phi}_j$ 将张成一个维度为 M 的线性子空间 $\mathcal{S}$. 我们定义 $\boldsymbol{y}$ 是一个 N 维向量, 其第 n 个元素由 $y(\boldsymbol{x}_n, \boldsymbol{w})$ 给出, 其中 $n = 1, \cdots, N$, 因为 $\boldsymbol{y}$ 是向量 $\boldsymbol{\phi}_j$ 任意的线性组合, 所以它可以存在于 M 维子空间中的任意位置. 平方和误差 [式(4.11)] 等于 (最高 0.5 倍)$\boldsymbol{y}$ 和 $\boldsymbol{t}$ 之间的欧氏距离的平方.$\boldsymbol{w}$ 的最小二乘解对应于子空间 $\mathcal{S}$ 中最接近 $\boldsymbol{t}$ 的 $\boldsymbol{y}$.

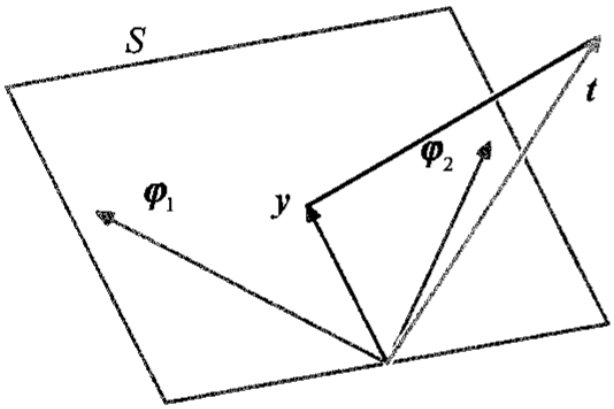


图 4.3: 在 N 维空间中, 不同轴的值分别为 $t_1, \cdots, t_N$ 时最小二乘解的几何解释. 最小二乘回归函数是通过确定数据向量 $\boldsymbol{t}$ 在基函数 $\phi_j(\boldsymbol{x})$ 张成的子空间中的正交投影得到的, 其中每个基函数都视为一个长度为 N 的向量 $\boldsymbol{\phi}_j$, 元素为 $\phi_j(\boldsymbol{x}_n)$

从图 4.3中可以直观地看出, 这个解对应于 $\boldsymbol{t}$ 在子空间 $\mathcal{S}$ 中的正交投影. 由于 $\boldsymbol{y}$ 的解是由 $\boldsymbol{\Phi}\boldsymbol{w}_{\mathrm{ML}}$ 给出的, 我们通过正交投影的形式可以很容易地验证这一点 (见练习 4.4).

命题 4.6 (最小二乘解的正交投影解释)

设 $\boldsymbol{\Phi}$ 列满秩,$\mathcal{S}=\{\boldsymbol{\Phi}\boldsymbol{w}:\boldsymbol{w}\in\mathbb{R}^M\}$ 为 $\boldsymbol{\Phi}$ 的列所张成的子空间,$\boldsymbol{w}_{\mathrm{ML}}$ 由(4.16)给出. 则 $\boldsymbol{y}=\boldsymbol{\Phi}\boldsymbol{w}_{\mathrm{ML}}$ 是 $\boldsymbol{t}$ 在 $\mathcal{S}$ 上的正交投影, 即

$$\boldsymbol{y}=\boldsymbol{\Phi}(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t},\qquad \boldsymbol{t}-\boldsymbol{y}\perp\mathcal{S}, \tag{4.22}$$

并且对任意 $\boldsymbol{y}'\in\mathcal{S}$ 成立

$$\|\boldsymbol{t}-\boldsymbol{y}\|^2\leqslant\|\boldsymbol{t}-\boldsymbol{y}'\|^2, \tag{4.23}$$

其中 $\|\cdot\|$ 为 $\mathbb{R}^N$ 中的欧氏范数. 此外,$E_D(\boldsymbol{w}_{\mathrm{ML}})=\dfrac{1}{2}\|\boldsymbol{t}-\boldsymbol{y}\|^2$.

♠

证明

(重构新增) 记 $\boldsymbol{P}\triangleq\boldsymbol{\Phi}(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}$, 则 $\boldsymbol{y}=\boldsymbol{P}\boldsymbol{t}$. 分四步.

第一步,$\boldsymbol{y}\in\mathcal{S}$. 由定义 $\boldsymbol{y}=\boldsymbol{\Phi}\boldsymbol{w}_{\mathrm{ML}}$, 而 $\boldsymbol{w}_{\mathrm{ML}}\in\mathbb{R}^M$, 故 $\boldsymbol{y}$ 是 $\boldsymbol{\Phi}$ 各列的线性组合.

第二步,$\boldsymbol{t}-\boldsymbol{y}\perp\mathcal{S}$. 任取 $\boldsymbol{s}=\boldsymbol{\Phi}\boldsymbol{a}\in\mathcal{S}$, 则

$$\begin{aligned}\boldsymbol{s}^{\mathsf{T}}(\boldsymbol{t}-\boldsymbol{y})&=\boldsymbol{a}^{\mathsf{T}}\boldsymbol{\Phi}^{\mathsf{T}}\left(\boldsymbol{t}-\boldsymbol{\Phi}(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}\right)\\&=\boldsymbol{a}^{\mathsf{T}}\left(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}-\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}\right)\\&=\boldsymbol{a}^{\mathsf{T}}\left(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}-\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}\right)=0,\end{aligned}$$

其中第三步用了 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 可逆 (由列满秩假设).

第三步,(4.23). 对任意 $\boldsymbol{y}'=\boldsymbol{\Phi}\boldsymbol{w}'\in\mathcal{S}$, 令 $\boldsymbol{e}\triangleq\boldsymbol{t}-\boldsymbol{y}$,$\boldsymbol{d}\triangleq\boldsymbol{y}-\boldsymbol{y}'$. 由第一步、第二步,$\boldsymbol{e}\perp\mathcal{S}$ 而 $\boldsymbol{d}\in\mathcal{S}$, 故 $\boldsymbol{e}^{\mathsf{T}}\boldsymbol{d}=0$. 于是

$$\|\boldsymbol{t}-\boldsymbol{y}'\|^2=\|\boldsymbol{e}+\boldsymbol{d}\|^2=\|\boldsymbol{e}\|^2+2\boldsymbol{e}^{\mathsf{T}}\boldsymbol{d}+\|\boldsymbol{d}\|^2=\|\boldsymbol{t}-\boldsymbol{y}\|^2+\|\boldsymbol{d}\|^2\geqslant\|\boldsymbol{t}-\boldsymbol{y}\|^2,$$

等号当且仅当 $\boldsymbol{y}'=\boldsymbol{y}$.

第四步, 与 E_D 的关系. 由(4.11),

$$E_D(\boldsymbol{w})=\frac{1}{2}\sum_{n=1}^{N}\left(t_n-\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right)^2=\frac{1}{2}\|\boldsymbol{t}-\boldsymbol{\Phi}\boldsymbol{w}\|^2,$$

取 $\boldsymbol{w}=\boldsymbol{w}_{\mathrm{ML}}$ 即 $E_D(\boldsymbol{w}_{\mathrm{ML}})=\dfrac{1}{2}\|\boldsymbol{t}-\boldsymbol{y}\|^2$; 由第三步, 这恰是 E_D 在 $\mathcal{S}$ 上的最小值.

□

注

(重构新增) 命题 4.6把(4.16)这个纯代数的公式翻译成了一个几何命题: 最小二乘就是在 M 维子空间 $\mathcal{S}$ 中找离 $\boldsymbol{t}$ 最近的点.

三条假设各有作用.$\mathcal{S}$ 的定义依赖“$\boldsymbol{\Phi}$ 的列张成子空间”, 这只需要 $\boldsymbol{\Phi}$ 的列数有限; 而 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 可逆 (列满秩) 则保证 $\mathcal{S}$ 中的表示 $\boldsymbol{y}=\boldsymbol{\Phi}\boldsymbol{w}$ 与 $\boldsymbol{w}$ 一一对应, 从而 $\boldsymbol{P}$ 是到 $\mathcal{S}$ 上的正交投影而不是斜投影. 若列不满秩,$\boldsymbol{y}$ 仍然唯一 (它就是投影), 但 $\boldsymbol{w}$ 不唯一.

命题中的“最高 0.5 倍”来自(4.11)中的人为因子 1/2:E_D 并不等于欧氏距离的平方, 而是它的一半.

最后,$\boldsymbol{P}=\boldsymbol{\Phi}(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}$ 是对称幂等矩阵 ($\boldsymbol{P}^{\mathsf{T}}=\boldsymbol{P}$,$\boldsymbol{P}^2=\boldsymbol{P}$), 这正是正交投影矩阵的特征性质;练习 4.4要求直接验证这一点.

在实践中, 当 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 接近奇异值时, 直接求解正规方程在数值上可能会比较困难. 特别是当两个或两个以上的基向量 $\boldsymbol{\phi}_j$ 共线或近似共线时, 所得到的参数值数量级可能较大. 在处理真实数据集时, 这种近似退化现象并不少见. 由这一现象产生的数值困难可以使用奇异值分解 (Singular Value Decomposition, SVD) 来解决 (Deisenroth, Faisal, and Ong, 2020). 值得注意的是, 我们可以通过添加正则项来确保矩阵是非奇异的, 即使在存在退化的情况下也可以如此.

4.1.5 序贯学习

定义 4.6 (批量学习与序贯学习)

若学习算法一次性处理整个训练集来确定参数, 则称为**批量学习** (batch learning); 若算法一个个地考虑数据点, 并在考虑每个数据点时更新模型参数, 则称为**序贯学习** (sequential learning), 也称**在线算法** (on-line algorithm).

♣

注

(重构新增) 定义 4.6区分的是计算方式而不是目标函数: 两种学习方式优化的是同一个误差函数, 只是逼近它的路径不同.

这一区分在两种情形下具有实际意义. 其一是数据规模:(4.16)中的 $\boldsymbol{\Phi}$ 是 $N\times M$ 矩阵, 当 N 极大时既无法一次性存入内存, 也无法承受一次矩阵求逆的代价. 其二是数据到达方式: 若观测是流式到达的, 则在全部数据到齐之前就必须给出预测, 批量学习在这种场景下根本无法执行.

代价是序贯学习放弃了闭式解, 转而依赖步长 η 的选取与迭代次数 τ 的控制; 它给出的只是近似解, 且不保证单调收敛到(4.16).

最大似然解(4.16)涉及一次性处理整个训练集, 称为批量学习. 对于大型数据集来说, 批处理的计算成本很高. 如果数据集非常大, 则使用序贯 (sequential) 学习算法 (也称在线算法) 是更为合适的. 这种算法一个个地考虑数据点, 在考虑每个数据点时模型参数得到更新. 序贯学习也适用于实时应用场景, 在这种场景中, 观测数据连续地流式到达, 我们需要在所有数据可获得前就进行预测.

如下所示, 我们可以通过随机梯度下降 (stochastic gradient descent) 技术 (参见节 7.2), 也称序贯梯度下降 (sequential gradient descent) 技术, 得到序贯学习算法. 如果误差函数包含 n 个数据点误差的总和,$E=\sum_n E_n$, 那么在观测 n 个数据点之后, 随机梯度下降法将使用下式更新参数向量 $\boldsymbol{w}$:

$$\boldsymbol{w}^{(\tau+1)}=\boldsymbol{w}^{(\tau)}-\eta\nabla E_n \tag{4.24}$$

其中 τ 表示迭代次数,η 是合适的学习率.$\boldsymbol{w}$ 的值被初始化为某个起始向量 $\boldsymbol{w}^{(0)}$. 通过平方和误差函数(4.11), 可得

命题 4.7 (LMS 更新公式)

设单个数据点的误差为 $E_n=\frac{1}{2}\left(t_n-\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right)^2$. 则更新(4.24)等价于

$$\boldsymbol{w}^{(\tau+1)}=\boldsymbol{w}^{(\tau)}+\eta\left(t_n-\boldsymbol{w}^{(\tau)\mathsf{T}}\boldsymbol{\phi}_n\right)\boldsymbol{\phi}_n \tag{4.25}$$

其中 $\boldsymbol{\phi}_n=\boldsymbol{\phi}(\boldsymbol{x}_n)$.

♠

这称为最小二乘 (least-mean-square) 法或 LMS 算法.

证明

(重构新增) 第一步, 求 E_n 关于 $\boldsymbol{w}$ 的梯度. 记 $r_n\triangleq t_n-\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n$, 则 $E_n=\frac{1}{2}r_n^2$. 由于 r_n 关于 $\boldsymbol{w}$ 是线性的,$\nabla_{\boldsymbol{w}} r_n=-\boldsymbol{\phi}_n$, 故由链式法则

$$\nabla_{\boldsymbol{w}} E_n=r_n\nabla_{\boldsymbol{w}} r_n=-\left(t_n-\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n\right)\boldsymbol{\phi}_n.$$

第二步, 代入(4.24). 将上式在 $\boldsymbol{w}=\boldsymbol{w}^{(\tau)}$ 处取值,

$$\begin{aligned}\boldsymbol{w}^{(\tau+1)}&=\boldsymbol{w}^{(\tau)}-\eta\left[-\left(t_n-\boldsymbol{w}^{(\tau)\mathsf{T}}\boldsymbol{\phi}_n\right)\boldsymbol{\phi}_n\right]\\&=\boldsymbol{w}^{(\tau)}+\eta\left(t_n-\boldsymbol{w}^{(\tau)\mathsf{T}}\boldsymbol{\phi}_n\right)\boldsymbol{\phi}_n,\end{aligned}$$

此即(4.25).

□

注

(重构新增) 命题 4.7的结论(4.25)表明, 一次更新只沿当前数据点的基向量 $\boldsymbol{\phi}_n$ 方向移动, 移动量正比于该点的预测残差 $t_n - \boldsymbol{w}^{(\tau)\mathsf{T}}\boldsymbol{\phi}_n$.

由此可以读出三点. 第一, 若当前点的预测已经精确, 则残差为零, 参数不再移动. 第二, 更新方向总在 $\mathrm{span}\{\boldsymbol{\phi}_n\}$ 内, 因此 LMS 算法从 $\boldsymbol{w}^{(0)}$ 出发, 其轨迹始终落在 $\boldsymbol{w}^{(0)}$ 加上所有 $\boldsymbol{\phi}_n$ 张成的子空间之内. 第三,η 必须足够小: 过大的 η 会使单步修正越过极小点而产生振荡甚至发散, 其收敛的步长条件将在讨论梯度下降时给出 (参见节 7.3).

LMS 的名称来自这样一个事实: 当 η 取得足够小并且按适当方式让数据循环出现时,$\boldsymbol{w}^{(\tau)}$ 会收敛到由(4.16)给出的最小二乘解.

4.1.6 正则化最小二乘法

我们之前介绍过在误差函数中添加正则化项来控制过拟合的思想 (参见小节 1.2.5), 因此我们可以采用下列形式来最小化总误差函数:

$$E_D(\boldsymbol{w}) + \lambda E_W(\boldsymbol{w}) \tag{4.26}$$

定义 4.7 (正则化项与权重衰减)

(4.26)中,$E_D(\boldsymbol{w})$ 是数据相关的误差,$E_W(\boldsymbol{w})$ 是正则化项,λ 是控制二者相对重要性的**正则化系数**. 正则化项最为简单的形式之一是权重向量的平方和

$$E_W(\boldsymbol{w}) = \frac{1}{2}\sum_j w_j^2 = \frac{1}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w}, \tag{4.27}$$

相应的正则化称为**权重衰减** (weight decay).

♣

注

(重构新增) 定义 4.7中的三项各司其职, 缺一不可.

E_D 拟合数据,E_W 惩罚参数的大小,λ 则是二者的交换率. 当 $\lambda \to 0$ 时(4.29)退化为(4.11), 正则化消失; 当 $\lambda \to \infty$ 时最小值点趋于 $\boldsymbol{w} = \boldsymbol{0}$, 模型退化为常数预测. 因此 λ 是一个超参数: 它不能与 $\boldsymbol{w}$ 一起通过最小化 E 来确定, 因为那样只会得到 $\lambda = 0$(这一点在节 4.3开头还会重申).

(4.27)采用平方和而非绝对值之和, 原因是平方项使 E 保持为 $\boldsymbol{w}$ 的二次函数, 从而仍有闭式解(4.30); 若改用 $\sum_j |w_j|$, 则问题变为非光滑的凸优化, 不再有(4.16)那样的线性方程.

最后,(4.27)中的求和指标 j 与(4.11)一致地遍历全部参数 (包括 $w_0 = \phi_0$ 对应的分量); 若按小节 1.2.5的惯例把 w_0 排除在外, 则(4.30)中 $\lambda\boldsymbol{I}$ 作用于 w_0 的那一个对角元应改为 0.

如果我们还考虑如下平方和误差函数:

$$E_D(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\left\{t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right\}^2 \tag{4.28}$$

那么总误差函数将变成

$$E(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N}\left\{t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right\}^2 + \frac{\lambda}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w} \tag{4.29}$$

在统计学中, 正则化方法会将参数值向 0 减小, 这为参数收缩 (parameter shrinkage) 提供了一个示例. 这一方法的优势是, 由于误差函数仍然是 $\boldsymbol{w}$ 的二次函数, 因此它具有闭式解. 具体来说, 将式(4.29)关于 $\boldsymbol{w}$ 的梯度设为 $\boldsymbol{0}$, 并像之前一样求解 $\boldsymbol{w}$(见练习 4.6), 可得

命题 4.8 (正则化最小二乘的闭式解)

设 $\lambda > 0$. 则(4.29)的唯一最小值点为

$$\boldsymbol{w} = (\lambda \boldsymbol{I} + \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}, \tag{4.30}$$

其中 $\boldsymbol{\Phi}$ 为定义 4.4的设计矩阵,$\boldsymbol{I}$ 为 $M \times M$ 单位矩阵. 该最小值点是全局最小值, 且不要求 $\boldsymbol{\Phi}$ 列满秩.

证明

(重构新增) 第一步, 把 E 写成矩阵形式. 由定义 4.4,$\sum_n \big(t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\big)^2 = \|\boldsymbol{t} - \boldsymbol{\Phi}\boldsymbol{w}\|^2$, 故

$$E(\boldsymbol{w}) = \frac{1}{2}\|\boldsymbol{t} - \boldsymbol{\Phi}\boldsymbol{w}\|^2 + \frac{\lambda}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w}.$$

第二步, 求梯度. 用 $\|\boldsymbol{t} - \boldsymbol{\Phi}\boldsymbol{w}\|^2 = \boldsymbol{t}^{\mathsf{T}}\boldsymbol{t} - 2\boldsymbol{t}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{w} + \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\boldsymbol{w}$, 并注意 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 对称, 得

$$\nabla_{\boldsymbol{w}} E = -\boldsymbol{\Phi}^{\mathsf{T}}(\boldsymbol{t} - \boldsymbol{\Phi}\boldsymbol{w}) + \lambda \boldsymbol{w} = \big(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi} + \lambda \boldsymbol{I}\big)\boldsymbol{w} - \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}.$$

第三步, 令梯度为零:

$$\big(\lambda \boldsymbol{I} + \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\big)\boldsymbol{w} = \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}.$$

第四步, 说明系数矩阵可逆. 对任意非零 $\boldsymbol{v}$,

$$\boldsymbol{v}^{\mathsf{T}}\big(\lambda \boldsymbol{I} + \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}\big)\boldsymbol{v} = \lambda\|\boldsymbol{v}\|^2 + \|\boldsymbol{\Phi}\boldsymbol{v}\|^2 \geqslant \lambda\|\boldsymbol{v}\|^2 > 0,$$

故 $\lambda \boldsymbol{I} + \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 正定, 从而可逆, 且这一论证不需要 $\boldsymbol{\Phi}$ 列满秩. 两端左乘其逆即得(4.30).

第五步, 说明是全局最小值. 由第四步,Hesse 矩阵 $\lambda \boldsymbol{I} + \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 正定, 故 E 为严格凸二次函数,(4.30)是唯一全局最小值点.

□

注

(重构新增) 命题 4.8的结论(4.30)与(4.16)只差一处:$\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 被换成了 $\lambda \boldsymbol{I} + \boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$.

这一处改动的意义超出“数值稳定”本身. 因为 $\lambda > 0$ 时该矩阵必然正定,(4.30)对任何设计矩阵都有定义, 包括 $M > N$ 或基函数共线的情形; 而在这些情形下(4.16)根本不存在. 代价是解被系统性地“收缩”了:λ 越大,$\boldsymbol{w}$ 越靠近原点, 因此正则化解是有偏的 (参见节 4.3对偏差与方差的讨论).

(4.30)还给出了一个连续过渡: 当 $\lambda \to 0^+$ 时它趋于 $\boldsymbol{\Phi}^{+}\boldsymbol{t}$(在列满秩时即(4.16)); 当 $\lambda \to \infty$ 时它趋于 $\boldsymbol{0}$.

λ 的选取不属于本节的优化问题. 若把 λ 也当作自由参数一起最小化 E, 则最优值必为 $\lambda = 0$, 正则化随之失效; 因此 λ 必须由独立于训练集的信息 (如验证集) 确定, 见小节 1.2.6.

式(4.30)可以看作对最小二乘解式(4.16)的简单扩展.

4.1.7 多重输出

到目前为止, 我们已经考虑了单目标变量 t 的情况. 在一些应用中, 我们可能希望预测 $K > 1$ 个目标变量, 它们可以用目标向量 $\boldsymbol{t} = (t_1, \cdots, t_K)^{\mathsf{T}}$ 来表示. 通过为 $\boldsymbol{t}$ 的每个分量引入一组不同的基函数, 我们得到多个独立的回归问题. 然而, 更常见的方法是使用相同的一组基函数来对目标向量的所有分量建模.

定义 4.8 (多重输出回归)

设目标向量 $\boldsymbol{t} = (t_1, \cdots, t_K)^{\mathsf{T}}$ 的 K 个分量共用同一组基函数 $\phi_0, \cdots, \phi_{M-1}$, 则模型取

$$\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{W}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}), \tag{4.31}$$

其中 $\boldsymbol{y}$ 是一个 K 维的列向量;$\boldsymbol{W}$ 是一个 $M \times K$ 大小的参数矩阵;$\boldsymbol{\phi}(\boldsymbol{x})$ 是一个 M 维的列向量,$\phi_j(\boldsymbol{x})$ 则是其中的元素, 且 $\phi_0(\boldsymbol{x}) = 1$. 这样的模型称为**多重输出** (multiple outputs) 的线性回归模型.

注

(重构新增) 定义 4.8的关键约定是“共用同一组基函数”:$\boldsymbol{W}$ 的每一列 $\boldsymbol{w}_k$ 对应一个输出分量 t_k, 但所有分量共享同一个 $\boldsymbol{\phi}(\boldsymbol{x})$.

若为每个分量各配一组基函数, 问题就退化为 K 个互不相关的节 4.1, 没有任何新内容. 共用一个 $\boldsymbol{\phi}$ 则带来实质性的节省: 设计矩阵 $\boldsymbol{\Phi}$ 只需计算一次, 其伪逆也只需计算一次, 由命题 4.9可见全部 K 个分量共用同一个 $\boldsymbol{\Phi}^{\dagger}$.

模型(4.31)仍然关于参数是线性的, 只是参数从向量扩展成了矩阵. 这一“向量 $\to$ 矩阵”的推广是本章其余结论可以直接平移的原因.

如图 4.4所示, 线性回归模型同样可以表示为只有单层参数的神经网络.

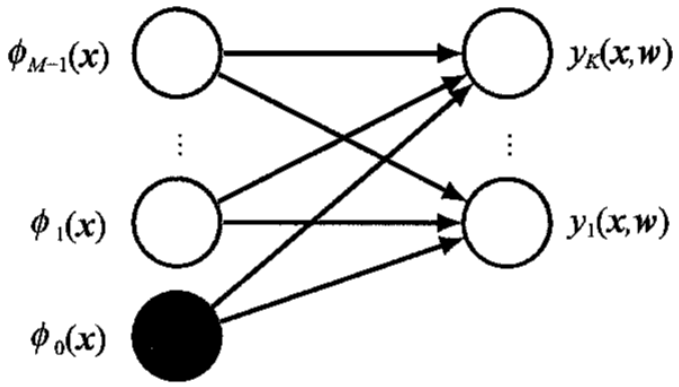


图 4.4: 将线性回归模型表示为只有单层参数的神经网络. 每个基函数由一个节点表示, 实心节点表示“偏置”基函数 $\phi_0(\boldsymbol{x})$. 同样, 每个输出也用一个节点表示. 节点之间的链接表示对应的权重和偏置参数

假设我们取目标向量的条件分布为如下形式的各向同性的 Gauss 分布:

$$p(\boldsymbol{t} \mid \boldsymbol{x}, \boldsymbol{W}, \sigma^2) = \mathcal{N}\left(\boldsymbol{t} \mid \boldsymbol{W}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}), \sigma^2 \boldsymbol{I}\right) \tag{4.32}$$

一组观测 $\boldsymbol{t}_1, \cdots, \boldsymbol{t}_N$ 可以用一个大小为 $N \times K$ 的矩阵 $\boldsymbol{T}$ 来表示,$\boldsymbol{t}_n^{\mathsf{T}} (n = 1, \cdots, N)$ 代表其中的第 n 行. 类似地, 我们可以将输入向量 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$ 转换为矩阵 X, 此时对数似然函数可以表示为如下形式:

$$\begin{aligned} \ln p(\boldsymbol{T} \mid X, \boldsymbol{W}, \sigma^2) &= \sum_{n=1}^{N} \ln \mathcal{N}\left(\boldsymbol{t}_n \mid \boldsymbol{W}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n), \sigma^2 \boldsymbol{I}\right) \\ &= -\frac{NK}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{n=1}^{N}\left\|\boldsymbol{t}_n - \boldsymbol{W}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right\|^2 \end{aligned} \tag{4.33}$$

和之前一样, 我们可以关于 $\boldsymbol{W}$ 最大化这个函数.

命题 4.9 (多重输出的最大似然解)

设对数似然由(4.33)给出,$\boldsymbol{\Phi}$ 为定义 4.4的设计矩阵,$\boldsymbol{T}$ 为第 n 行为 $\boldsymbol{t}_n^{\mathsf{T}}$ 的 $N \times K$ 矩阵. 若 $\boldsymbol{\Phi}$ 列满秩, 则

$$\boldsymbol{W}_{\mathrm{ML}} = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{T}, \tag{4.34}$$

其中, 我们已经用输入的特征向量 $\boldsymbol{\phi}(\boldsymbol{x}_1), \cdots, \boldsymbol{\phi}(\boldsymbol{x}_N)$ 组成了矩阵 $\boldsymbol{\Phi}$. 等价地,$\boldsymbol{W}_{\mathrm{ML}}$ 的第 k 列为

$$\boldsymbol{w}_k = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}_k = \boldsymbol{\Phi}^{\dagger}\boldsymbol{t}_k, \tag{4.35}$$

其中 $\boldsymbol{t}_k$ 是一个 N 维的列向量, 分量为 t_{nk}, 其中 $n = 1, \cdots, N$.

证明

(重构新增) 第一步, 把(4.33)按输出分量拆开. 记 $\boldsymbol{w}_k$ 为 $\boldsymbol{W}$ 的第 k 列,t_{nk} 为 $\boldsymbol{t}_n$ 的第 k 个分量. 由 $\boldsymbol{W}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)$ 的第 k 个分量为 $\boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)$, 有

$$\left\|\boldsymbol{t}_n - \boldsymbol{W}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right\|^2 = \sum_{k=1}^{K}\left(t_{nk} - \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right)^2.$$

第二步, 重写目标函数. 代入(4.33)的第一项 (它不含 $\boldsymbol{W}$), 得

$$\ln p(\boldsymbol{T} \mid X, \boldsymbol{W}, \sigma^2) = C - \frac{1}{2\sigma^2}\sum_{k=1}^{K}\sum_{n=1}^{N}\left(t_{nk} - \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right)^2,$$

其中 C 与 $\boldsymbol{W}$ 无关. 右端的求和是 K 个互不耦合的部分, 第 k 部分只含 $\boldsymbol{w}_k$. 因此最大化总和对每个 k 分别最大化

$$-\frac{1}{2\sigma^2}\sum_{n=1}^{N}\left(t_{nk} - \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\right)^2,$$

也就是最小化以 $\boldsymbol{t}_k = (t_{1k}, \cdots, t_{Nk})^{\mathsf{T}}$ 为目标向量的平方和误差, 而这正是命题 4.3所处理的问题.

第三步, 引用单输出结果. 由(4.16), 第 k 个最小化点为 $\boldsymbol{w}_k = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{t}_k = \boldsymbol{\Phi}^{\dagger}\boldsymbol{t}_k$, 即(4.35).

第四步, 合并成矩阵形式. 把 $k = 1, \cdots, K$ 的(4.35)并排作为列, 右端 $\boldsymbol{\Phi}^{\dagger}\boldsymbol{t}_k$ 并排后恰为 $\boldsymbol{\Phi}^{\dagger}\boldsymbol{T}$, 故 $\boldsymbol{W}_{\mathrm{ML}} = \boldsymbol{\Phi}^{\dagger}\boldsymbol{T} = (\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{T}$, 即(4.34).

□

注

(重构新增) 命题 4.9的证明实际上给出了比(4.34)更强的信息: 不同目标变量之间是完全解耦的,$\boldsymbol{W}$ 的每一列可以独立求出.

由此有两点推论. 其一, 伪逆矩阵 $\boldsymbol{\Phi}^{\dagger}$ 只需计算一次, 即可供全部 K 个分量共享, 这是定义 4.8共用基函数所带来的计算节省. 其二,(4.32)中协方差取 $\sigma^2\boldsymbol{I}$ 这一假设并不影响 $\boldsymbol{W}_{\mathrm{ML}}$: 只要协方差矩阵与 $\boldsymbol{W}$ 无关, 第一项就与 $\boldsymbol{W}$ 无关, 解耦论证照常成立. 协方差取一般形式 $\boldsymbol{\Sigma}$ 的情形见练习 4.7.

与单输出情形一样,(4.34)要求 $\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi}$ 可逆; 若 $M > N$, 同样需改用伪逆或引入正则化.

因此, 回归问题的解在不同的目标变量之间是解耦的, 我们只需要计算被所有向量 $\boldsymbol{w}_k$ 共享的单个伪逆矩阵 $\boldsymbol{\Phi}^{\dagger}$ 即可.

我们可以直接将其推广到具有任意协方差矩阵的一般 Gauss 噪声分布上 (参见练习 4.7). 这也再次将问题解耦成了 K 个独立回归问题. 由于参数 $\boldsymbol{W}$ 只定义了 Gauss 噪声分布的均值, 而多元 Gauss 均值的最大似然解与协方差无关 (参见小节 3.2.7), 因此这并不令人惊讶. 从现在开始, 为了简便起见, 我们将只考虑单目标变量 t.

4.2 决策理论

我们将回归任务看作条件概率分布 $p(t \mid \boldsymbol{x})$ 来建模, 并且选择了条件概率的特定形式, 即 Gauss 分布 [式(4.8)], 其中与 $\boldsymbol{x}$ 相关的均值 $y(\boldsymbol{x}, \boldsymbol{w})$ 由参数 $\boldsymbol{w}$ 控制, 方差由参数 σ^2 给出.$\boldsymbol{w}$ 和 σ^2 都可以使用最大似然从数据中学习得到. 预测分布 (predictive distribution) 的结果如下:

$$p(t \mid \boldsymbol{x}, \boldsymbol{w}_{\mathrm{ML}}, \sigma^2_{\mathrm{ML}}) = \mathcal{N}\left(t \mid y(\boldsymbol{x}, \boldsymbol{w}_{\mathrm{ML}}), \sigma^2_{\mathrm{ML}}\right) \tag{4.36}$$

预测分布表达了我们对某些新输入 $\boldsymbol{x}$ 所对应 t 值的不确定性. 然而, 对于许多实际应用, 我们需要预测 t 的具体值, 而不是返回整个分布, 特别是在需要采取特定行动的情况下. 例如, 如果我们想要确定用于治疗肿瘤的最佳辐射水平, 而我们的模型预测了辐射剂量的概率分布, 那么我们必须使用该分布来决定给予治疗的具体剂量. 因此, 我们的任务分为两个阶段. 第一个阶段称为推理 (inference) 阶段, 我们使用训练数据来确定预测分布 $p(t \mid \boldsymbol{x})$. 第二个阶段称为决策 (decision) 阶段, 我们使用这个预测分布来确定特定的值 $f(\boldsymbol{x})$. 这个特定的值依赖于输入向量 $\boldsymbol{x}$, 并且根据某些准则是最优的. 我们可以通过最小化一个既依赖于预测分布 $p(t \mid \boldsymbol{x})$ 又依赖于特定值 $f(\boldsymbol{x})$ 的损失函数 (也称成本函数) 来实现这一点.

直觉上, 我们可以选择条件分布的均值, 即 $f(\boldsymbol{x}) = y(\boldsymbol{x}, \boldsymbol{w}_{\mathrm{ML}})$. 在某些情况下, 这种直觉是正确的; 但在其他情况下, 它可能会给出非常糟糕的结果. 因此, 我们需要将其形式化, 这样我们就可以理解它应该何时应用以及在什么假设下应用. 该框架称为决策理论 (decision theory).

假设真实值为 t, 我们为预测选择了一个值 $f(\boldsymbol{x})$, 从而得到某种形式的惩罚或成本. 惩罚是由损失 (loss) 决定的, 用 $L(t, f(\boldsymbol{x}))$ 表示. 当然, 由于我们不知道 t 的真实值, 因此我们并不最小化 L 本身, 而是最小化平均或期望的损失.

定义 4.9 (损失函数与期望损失)

设 $L(t, f(\boldsymbol{x}))$ 为预测值 $f(\boldsymbol{x})$ 与真实值 t 之差的**损失函数** (也称成本函数). 由于 t 未知, 我们最小化的不是 L 本身, 而是以联合分布 $p(\boldsymbol{x}, t)$ 为权重的平均或期望损失

$$\mathbb{E}[L] = \iint L(t, f(\boldsymbol{x}))\, p(\boldsymbol{x}, t)\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}t, \tag{4.37}$$

其中, 我们以输入变量和目标变量的联合分布 $p(\boldsymbol{x}, t)$ 作为权重对它们的分布进行加权平均.

♣

注

(重构新增) 定义 4.9把“预测得好不好”这件事拆成了两个独立的成分: 损失函数 L 刻画错多少算多严重, 联合分布 $p(\boldsymbol{x}, t)$ 刻画什么样的 $(\boldsymbol{x}, t)$ 更常出现.(4.37)把二者以加权平均的方式合成一个标量, 从而使“选择 f”变成一个可以优化的目标.

这一形式化的代价是: 最小化 $\mathbb{E}[L]$ 要求知道 $p(\boldsymbol{x}, t)$, 而实际只掌握训练数据和节 4.2开头给出的预测分布. 因此(4.37)在理论上定义了“最优”, 在下文中只作为推导最优决策规则的出发点, 并不直接作为可计算的训练目标.

注意 $\mathbb{E}[L]$ 中的期望是对 $(\boldsymbol{x}, t)$ 的联合分布取的, 因而同时包含了 t 的随机性和 $\boldsymbol{x}$ 的分布. 若 $p(\boldsymbol{x})$ 在某个区域为零, 则该区域上的 f 取值不影响 $\mathbb{E}[L]$——这一点在讨论最优解的唯一性时必须注意.

回归问题中常见的损失函数是平方损失函数:$L(t, f(\boldsymbol{x})) = \{f(\boldsymbol{x}) - t\}^2$. 在这种情况下, 期望损失函数可以写成

$$\mathbb{E}[L] = \iint \{f(\boldsymbol{x}) - t\}^2 p(\boldsymbol{x}, t)\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}t \tag{4.38}$$

注意不要将平方损失函数与前面介绍的平方和误差函数弄混淆. 误差函数用于在训练过程中设置参数, 以便确定条件概率分布 $p(t \mid \boldsymbol{x})$, 而损失函数则控制如何使用条件分布来达到预测函数 $f(\boldsymbol{x})$, 以预测 $\boldsymbol{x}$ 的每个值.

我们的目标是找到能使 $\mathbb{E}[L]$ 最小的 $f(\boldsymbol{x})$. 假设 $f(\boldsymbol{x})$ 是一个完全任意的函数, 则可以使用变分法 (参见定义 B.3) 得到

$$\frac{\delta \mathbb{E}[L]}{\delta f(\boldsymbol{x})} = 2 \int \{f(\boldsymbol{x}) - t\} p(\boldsymbol{x}, t)\, \mathrm{d}t = 0 \tag{4.39}$$

命题 4.10 (平方损失的最优决策规则)

设 $\mathbb{E}[L]$ 由(4.38)给出,f 在所有满足所需可积性的函数中选取. 则 $\mathbb{E}[L]$ 的驻点满足(4.39), 且最优解为条件平均

$$f^*(\boldsymbol{x}) = \frac{1}{p(\boldsymbol{x})} \int t\, p(\boldsymbol{x}, t)\, \mathrm{d}t = \int t\, p(t \mid \boldsymbol{x})\, \mathrm{d}t = \mathbb{E}_t[t \mid \boldsymbol{x}], \tag{4.40}$$

即可取 $f^*(\boldsymbol{x}) = \mathbb{E}_t[t \mid \boldsymbol{x}]$.

♠

证明

(重构新增) 第一步, 写出变分条件. 令 f 作扰动 $f+\varepsilon\eta$, 由(4.38),

$$\begin{aligned}\mathbb{E}[L] &= \iint \big\{f(\boldsymbol{x})+\varepsilon\eta(\boldsymbol{x})-t\big\}^2 p(\boldsymbol{x},t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t \\ &= \mathbb{E}[L]\big|_f + \varepsilon\iint 2\{f(\boldsymbol{x})-t\}\eta(\boldsymbol{x})p(\boldsymbol{x},t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t + O(\varepsilon^2),\end{aligned}$$

与泛函导数的定义 (见定义 B.3) 比较, 得

$$\frac{\delta\mathbb{E}[L]}{\delta f(\boldsymbol{x})} = 2\int\{f(\boldsymbol{x})-t\}p(\boldsymbol{x},t)\,\mathrm{d}t,$$

令其为零即得(4.39).

第二步, 解出 $f(\boldsymbol{x})$. 在(4.39)中把含 $f(\boldsymbol{x})$ 的项与不含的项分开 ($f(\boldsymbol{x})$ 对 t 的积分是常数):

$$f(\boldsymbol{x})\int p(\boldsymbol{x},t)\,\mathrm{d}t = \int t\,p(\boldsymbol{x},t)\,\mathrm{d}t.$$

由概率的加和法则, $\int p(\boldsymbol{x},t)\,\mathrm{d}t = p(\boldsymbol{x})$, 故当 $p(\boldsymbol{x})>0$ 时

$$f(\boldsymbol{x}) = \frac{1}{p(\boldsymbol{x})}\int t\,p(\boldsymbol{x},t)\,\mathrm{d}t.$$

第三步, 化为条件平均. 由乘积法则 $p(\boldsymbol{x},t)=p(t\mid\boldsymbol{x})p(\boldsymbol{x})$,

$$\frac{1}{p(\boldsymbol{x})}\int t\,p(\boldsymbol{x},t)\,\mathrm{d}t = \frac{1}{p(\boldsymbol{x})}\int t\,p(t\mid\boldsymbol{x})p(\boldsymbol{x})\,\mathrm{d}t = \int t\,p(t\mid\boldsymbol{x})\,\mathrm{d}t = \mathbb{E}_t[t\mid\boldsymbol{x}],$$

此即(4.40).

第四步, 说明这是最小值而非最大值. 由(4.43)(其证明见命题 4.11),$\mathbb{E}[L]$ 等于一个只依赖于 f 的非负项加上一个与 f 无关的常数项, 而该非负项在 $f=f^*$ 处取零, 故 f^* 给出全局最小值.

□

注

(重构新增) 命题 4.10的结论(4.40)是本章从“概率”走向“决策”的关键一步: 一旦损失取平方, 最优预测就与 $p(t\mid\boldsymbol{x})$ 的全部信息无关, 只依赖它的一阶矩.

这解释了为什么在定义 4.3这样的模型下,小节 4.1.3给出的 $y(\boldsymbol{x},\boldsymbol{w}_{\mathrm{ML}})$ 恰好就是最优决策——因为 Gauss 分布的均值就是它的唯一参数. 但它同时暴露了平方损失的局限: 若 $p(t\mid\boldsymbol{x})$ 是多峰的, 条件均值可能落在两个峰之间的低密度区, 此时 f^* 虽然在期望意义下最优, 却几乎不可能是一个“典型”的 t 值.

推导中 $\mathbb{E}[L]$ 对 f 的驻点条件是逐点成立的:(4.39)对每个 $\boldsymbol{x}$ 分别给出一个方程, 这正是变分法“逐点变分”的含义. 因此(4.40)只在 $p(\boldsymbol{x})>0$ 处被确定; 在 $p(\boldsymbol{x})=0$ 处 f 可以任意取值而不影响 $\mathbb{E}[L]$.

定义 4.10 (回归函数)

(4.40)给出的条件平均 $\mathbb{E}_t[t\mid\boldsymbol{x}]$ 称为**回归函数** (regression function).

♣

注

(重构新增) 回归函数 $\mathbb{E}_t[t\mid\boldsymbol{x}]$ 只依赖条件分布 $p(t\mid\boldsymbol{x})$, 不依赖任何特定的损失函数, 因此可以作为“给定 $\boldsymbol{x}$ 时 t 的典型值”的通用刻画.命题 4.10说明, 在平方损失下它同时是最优预测; 而在其他损失下最优预测可能是别的泛函 ($q=1$ 时为条件中位数,$q\to 0$ 时为条件众数, 见定义 4.11与练习 4.12).

区分“回归函数”与“模型 $y(\boldsymbol{x},\boldsymbol{w})$”是本章的一个基本区分: 前者由数据生成机制 $p(t\mid\boldsymbol{x})$ 决定, 是拟合的对象; 后者是拟合的工具, 其参数 $\boldsymbol{w}$ 由有限数据估计得到. 二者的差距正是节 4.3要量化的偏差.

结果如图 4.5所示, 这一形式可以很容易地扩展到由向量 $\boldsymbol{t}$ 表示的多个目标变量. 这时, 最优解是条件平均 $f^*(\boldsymbol{x})=\mathbb{E}_t[\boldsymbol{t}\mid\boldsymbol{x}]$(见练习 4.8). 对于形如(4.8)的 Gauss 条件分布, 条件均值可以简化为

$$\mathbb{E}[t\mid\boldsymbol{x}] = \int t\,p(t\mid\boldsymbol{x})\,\mathrm{d}t = y(\boldsymbol{x},\boldsymbol{w}) \tag{4.41}$$

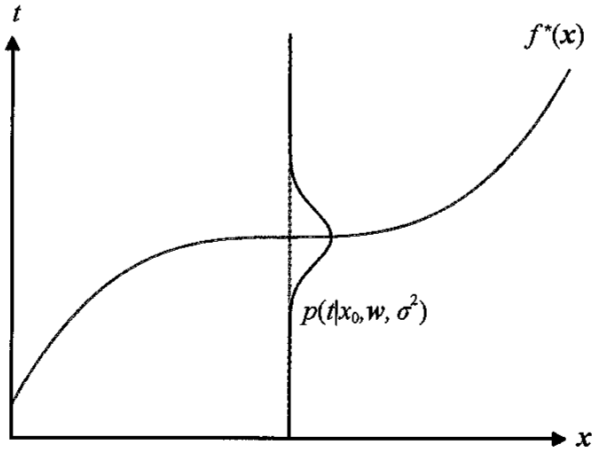


图 4.5: 用于最小化期望平方损失的回归函数 $f^*(\boldsymbol{x})$ 是由条件分布 $p(t \mid \boldsymbol{x})$ 的均值给出的

我们也可以用另一种推导方式得到这个结果, 这种推导方式将阐明回归问题的本质. 在知道最优解是条件期望的情况下, 我们可以将平方项展开为

$$\begin{aligned}\{f(\boldsymbol{x}) - t\}^2 &= \big\{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}] + \mathbb{E}[t \mid \boldsymbol{x}] - t\big\}^2 \\ &= \{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 + 2\{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\}\{\mathbb{E}[t \mid \boldsymbol{x}] - t\} + \{\mathbb{E}[t \mid \boldsymbol{x}] - t\}^2 \end{aligned} \tag{4.42}$$

其中, 为了保持符号整洁, 我们使用 $\mathbb{E}[t \mid \boldsymbol{x}]$ 来表示 $\mathbb{E}_t[t \mid \boldsymbol{x}]$. 代入损失函数(4.38)并对 t 进行积分后, 交叉项消失了, 得到如下形式的损失函数的表达式:

命题 4.11 (平方损失的期望损失分解)

设 $\mathbb{E}[L]$ 由(4.38)给出, 且 $p(\boldsymbol{x}, t)$ 使所涉及的积分绝对收敛. 则

$$\mathbb{E}[L] = \int \big\{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\big\}^2 p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} + \int \mathrm{var}[t \mid \boldsymbol{x}]\, p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \tag{4.43}$$

其中 $\mathrm{var}[t \mid \boldsymbol{x}] \triangleq \mathbb{E}\big[\{t - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 \mid \boldsymbol{x}\big]$ 为给定 $\boldsymbol{x}$ 时 t 的条件方差.

证明

(重构新增) 第一步, 把(4.42)代回(4.38). 三项分别记为 A、B、C:

$$\mathbb{E}[L] = \iint \{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 p(\boldsymbol{x}, t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t + \iint B\, p(\boldsymbol{x}, t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t + \iint \{\mathbb{E}[t \mid \boldsymbol{x}] - t\}^2 p(\boldsymbol{x}, t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t.$$

第二步, 第一项对 t 积分. 被积函数不含 t, 且 $\int p(\boldsymbol{x}, t)\,\mathrm{d}t = p(\boldsymbol{x})$, 故

$$\iint \{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 p(\boldsymbol{x}, t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t = \int \{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}.$$

第三步, 交叉项 B 为零. 先把 B 对 t 积分, 注意 $2\{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\}$ 不含 t:

$$\begin{aligned}\int B\, p(\boldsymbol{x}, t)\,\mathrm{d}t &= 2\{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\} \int \{\mathbb{E}[t \mid \boldsymbol{x}] - t\} p(\boldsymbol{x}, t)\,\mathrm{d}t \\ &= 2\{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\} p(\boldsymbol{x}) \int \{\mathbb{E}[t \mid \boldsymbol{x}] - t\} p(t \mid \boldsymbol{x})\,\mathrm{d}t \\ &= 2\{f(\boldsymbol{x}) - \mathbb{E}[t \mid \boldsymbol{x}]\} p(\boldsymbol{x}) \big\{\mathbb{E}[t \mid \boldsymbol{x}] - \mathbb{E}[t \mid \boldsymbol{x}]\big\} = 0,\end{aligned}$$

其中第三步用了 $\int p(t \mid \boldsymbol{x})\,\mathrm{d}t = 1$ 与 $\int t\, p(t \mid \boldsymbol{x})\,\mathrm{d}t = \mathbb{E}[t \mid \boldsymbol{x}]$. 再对 $\boldsymbol{x}$ 积分,B 的贡献为零.

第四步, 第三项 C. 由于 $\{\mathbb{E}[t \mid \boldsymbol{x}] - t\}^2 = \{t - \mathbb{E}[t \mid \boldsymbol{x}]\}^2$, 且

$$\iint \{t - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 p(\boldsymbol{x}, t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t = \int \left(\int \{t - \mathbb{E}[t \mid \boldsymbol{x}]\}^2 p(t \mid \boldsymbol{x})\,\mathrm{d}t \right) p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x},$$

而内层积分按定义就是 $\mathrm{var}[t \mid \boldsymbol{x}]$.

把第二步、第三步、第四步的结果代回第一步即得(4.43).

□

注

(重构新增) 命题 4.11把(4.43)的右端分成“可约”与“不可约”两部分, 这一点对理解本章其余部分至关重要.

第一项只依赖于 f, 且非负, 因此最小值为 0, 在 $f(\boldsymbol{x}) = \mathbb{E}[t \mid \boldsymbol{x}]$ 处取到——这正是命题 4.10的另一种证法. 第二项 $\int \mathrm{var}[t \mid \boldsymbol{x}] p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}$ 完全不含 f, 故任何预测函数都无法把它变小, 它代表数据本身的内在噪声, 是损失函数不可约的最小值.

推导中“交叉项为零”一步是全部技巧所在, 而它成立的原因只有一条:$\mathbb{E}[t \mid \boldsymbol{x}]$ 是 t 在给定 $\boldsymbol{x}$ 下的期望, 故 $t - \mathbb{E}[t \mid \boldsymbol{x}]$ 的条件期望为零. 若把 $\mathbb{E}[t \mid \boldsymbol{x}]$ 换成任何别的函数 (例如条件中位数), 交叉项一般不为零, 分解式随之失效.

最后, 本命题的绝对收敛假设不可省: 若 $\mathbb{E}[t^2] = \infty$, 则 $\mathbb{E}[L]$ 可能取 $+\infty$, 此时(4.43)两端同时为 $+\infty$, 讨论最小值就失去了意义.

我们需要确定的函数 $f(\boldsymbol{x})$ 只出现在(4.43)右侧的第一项中, 当 $f(\boldsymbol{x})$ 等于 $\mathbb{E}[t \mid \boldsymbol{x}]$ 时, 该项达到最小值. 此时,(4.43)右侧的这一项就消失了. 这只是我们之前推导出来的结果, 这表明最优最小二乘的预测是由条件均值给出的.(4.43)右侧的第二项是分布 t 的方差在 $\boldsymbol{x}$ 上取平均后的结果, 代表目标数据的内在变化, 可以视为噪声. 因为它独立于 $f(\boldsymbol{x})$, 所以它代表损失函数不可约的最小值.

平方损失函数并不是回归中唯一可选的损失函数. 平方损失的一种泛化形式称为 Minkowski 损失, 其期望由下式给出:

定义 4.11 (Minkowski 损失)

设 $q > 0$. 称

$$\mathbb{E}[L_q] = \iint |f(\boldsymbol{x}) - t|^q\, p(\boldsymbol{x}, t)\,\mathrm{d}\boldsymbol{x}\,\mathrm{d}t \tag{4.44}$$

为 **Minkowski 损失**的期望, 它是平方损失的一种泛化形式. ♣

注

(重构新增) 定义 4.11以一个参数 q 统一了一族损失:$q=2$ 时(4.44)退化为(4.38),$q=1$ 时给出绝对误差,$q>2$ 时对大误差的惩罚更重,$q<1$ 时对大误差的惩罚反而变轻.

q 的作用可以从图 4.6直接读出:q 越小, 曲线在 $f=t$ 处的尖点越尖锐、在两侧越平坦, 因此 $q\to 0$ 的极限只关心“t 是否恰等于 f”, 这正是众数出现的机制;q 越大, 曲线越平坦而两侧越陡, 因此对离群点越敏感.

需要说明的是,$q<1$ 时 $|f-t|^q$ 不再是凸函数,(4.44)的最小化问题可能没有唯一解; 本节的结论默认 $q\geqslant 1$, 而练习 4.12讨论的 $q\to 0$ 极限应理解为 $q\downarrow 0$ 的极限行为.

当 $q=2$ 时即为期望平方损失.图 4.6给出了当 q 取不同值时, 函数 $|f-t|^q$ 关于 $f-t$ 的曲线.$\mathbb{E}[L_q]$ 的最小值由 $q=2$ 时的条件均值、$q=1$ 时的条件中位数和 $q\to 0$ 时的条件众数给出 (见练习 4.12).

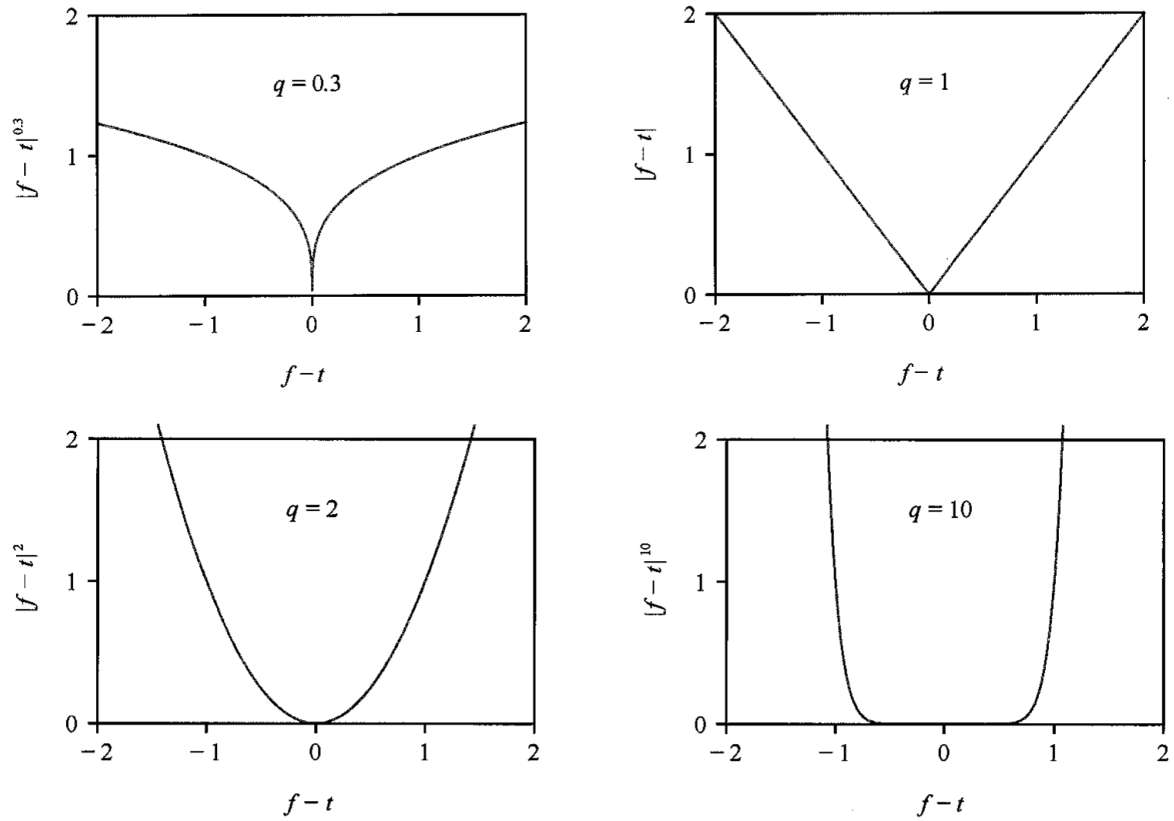


图 4.6: 当 q 取不同值时, 曲线 $L_q=|f-t|^q$ 的示意图

Gauss 噪声假设意味着给定 $\boldsymbol{x}$ 时 t 的条件分布是单峰的, 这对于某些应用来说可能是不合适的. 在这种情况下, 平方损失可能会导致结果变差, 我们需要开发更复杂的其他方法. 例如, 我们可以通过使用混合 Gauss 分布来扩展这个模型, 使其成为多峰条件分布 (参见节 6.5). 这种情况在求解逆问题时会经常出现. 本节重点讨论的是回归问题的决策理论, 在下一章中, 我们将在分类任务中引入类似的概念 (参见节 5.2).

4.3 偏差-方差权衡

到目前为止, 在对回归问题的线性模型进行讨论时, 我们假设基函数的形式和数量都是给定的 (参见节 1.2). 我们还看到, 如果使用非常有限的数据集训练复杂模型, 使用最大似然估计可能会导致严重的过拟合问题. 然而, 通过限制基函数的数量来避免过拟合有一个副作用, 就是限制了模型捕捉数据中有趣和重要趋势的灵活性. 尽管正则化项可以控制包含大量参数的模型的过拟合问题, 但这又引出了如何确定合适的正则化系数 λ 的问题. 同时关于权重向量 $\boldsymbol{w}$ 和正则化系数 λ 最小化正则化误差函数显然是不正确的, 因为这会导致未正则化的解, 即 $\lambda=0$.

我们可以从频率学派的视角讨论模型复杂性的问题, 称为偏差-方差 (bias-variance) 权衡. 虽然我们将在线性

基函数模型的背景下介绍这个概念, 因为在该背景下可以使用简单的例子很容易地说明这些想法, 但我们的讨论具有非常广泛的适用性. 过拟合确实是最大似然的缺点, 但是当我们在 Bayes 设定中边缘化参数时, 则不会出现这种情况 (Bishop, 2006).

在讨论回归问题的决策理论时 (参见节 4.2), 我们考虑了各种损失函数. 一旦知道条件分布 $p(t \mid \boldsymbol{x})$, 每个损失函数就都会得到对应的最优预测. 一个普遍的选择是平方损失函数, 其最优预测由条件期望给出, 用 $h(\boldsymbol{x})$ 表示.

$$h(\boldsymbol{x}) = \mathbb{E}[t \mid \boldsymbol{x}] = \int t\, p(t \mid \boldsymbol{x})\, \mathrm{d}t \tag{4.45}$$

平方损失的期望可以写成如下形式:

$$\mathbb{E}[L] = \int \{f(\boldsymbol{x}) - h(\boldsymbol{x})\}^2 p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x} + \iint \{h(\boldsymbol{x}) - t\}^2 p(\boldsymbol{x}, t)\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}t \tag{4.46}$$

前面我们曾提到, 与 $f(\boldsymbol{x})$ 无关的(4.46)右侧的第二项来自数据上的固有噪声, 代表了期望损失的最小值.(4.46)右侧的第一项取决于如何选择函数 $f(\boldsymbol{x})$. 我们将寻找一个 $f(\boldsymbol{x})$ 来最小化这一项. 因为这一项是非负的, 所以能达到的最小值是 0. 如果有无限的数据供应 (和无限的计算资源), 则原则上可以找到任何准确度下的回归函数 $h(\boldsymbol{x})$, 它可以代表 $f(\boldsymbol{x})$ 的最优选择. 然而在实践中, 数据集 $\mathcal{D}$ 只包含 N 个有限的数据点, 因此我们无法精确地知道回归函数 $h(\boldsymbol{x})$.

如果使用参数向量 $\boldsymbol{w}$ 控制的函数对 $h(\boldsymbol{x})$ 进行建模, 那么从 Bayes 学派的视角来看, 模型中的不确定性可以通过 $\boldsymbol{w}$ 上的后验分布来表示. 但是频率学派会基于数据集 $\mathcal{D}$ 对 $\boldsymbol{w}$ 进行点估计, 并试图通过以下思想实验 (thought experiment) 来解释该估计的不确定性. 假设我们有大量的数据集, 每个数据集的大小为 N 且都独立地从分布 $p(t \mid \boldsymbol{x})$ 中产生. 对于任何给定的数据集 $\mathcal{D}$, 都可以通过我们的算法得到预测函数 $f(\boldsymbol{x}; \mathcal{D})$. 集合中的不同数据集将给出不同的函数以及平方损失值. 一个特定学习算法的性能, 可以通过对它在集合中每个数据集上的性能表现取平均值来评估.

定理 4.1 (偏差-方差分解) 设训练集 $\mathcal{D}$ 的大小为 N, 其中的数据点独立地取自分布 $p(t \mid \boldsymbol{x})$, $f(\boldsymbol{x}; \mathcal{D})$ 表示在 $\mathcal{D}$ 上训练得到的预测函数, $h(\boldsymbol{x}) = \mathbb{E}[t \mid \boldsymbol{x}]$ 为(4.45)给出的回归函数. 若所涉及的各积分均绝对收敛, 则平方损失的期望可以分解为

$$\text{期望损失} = \text{平方偏差} + \text{方差} + \text{噪声}, \tag{4.47}$$

其中

$$\text{平方偏差} = \int \left\{\mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x}; \mathcal{D})] - h(\boldsymbol{x})\right\}^2 p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}, \tag{4.48}$$

$$\text{方差} = \int \mathbb{E}_{\mathcal{D}}\left[\{f(\boldsymbol{x}; \mathcal{D}) - \mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x}; \mathcal{D})]\}^2\right] p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}, \tag{4.49}$$

$$\text{噪声} = \iint \{h(\boldsymbol{x}) - t\}^2 p(\boldsymbol{x}, t)\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}t. \tag{4.50}$$

♡

证明

(重构新增) 由命题 4.11,(4.46)已经把 $\mathbb{E}[L]$ 写成两项之和, 其中第二项恰为(4.50)右端的积分, 它不含 f. 因此只需处理第一项.

第一步, 写出单个数据集上的被积函数.(4.46)右侧第一项的被积函数, 在一个特定的数据集 $\mathcal{D}$ 上具有以下形式:

$$\{f(\boldsymbol{x}; \mathcal{D}) - h(\boldsymbol{x})\}^2 \tag{4.51}$$

第二步, 在花括号内加减 $\mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x}; \mathcal{D})]$ 并展开. 为简洁起见记 $\bar{f}(\boldsymbol{x}) \triangleq \mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x}; \mathcal{D})]$, 则

$$\begin{aligned}\{f(\boldsymbol{x}; \mathcal{D}) - h(\boldsymbol{x})\}^2 &= \left\{f(\boldsymbol{x}; \mathcal{D}) - \bar{f}(\boldsymbol{x}) + \bar{f}(\boldsymbol{x}) - h(\boldsymbol{x})\right\}^2 \\ &= \left\{f(\boldsymbol{x}; \mathcal{D}) - \bar{f}(\boldsymbol{x})\right\}^2 + \left\{\bar{f}(\boldsymbol{x}) - h(\boldsymbol{x})\right\}^2 \\ &\quad + 2\left\{f(\boldsymbol{x}; \mathcal{D}) - \bar{f}(\boldsymbol{x})\right\}\left\{\bar{f}(\boldsymbol{x}) - h(\boldsymbol{x})\right\}.\end{aligned} \tag{4.52}$$

第三步, 对 $\mathcal{D}$ 取期望. 由于 $\bar{f}(\boldsymbol{x})$ 与 $h(\boldsymbol{x})$ 都不依赖 $\mathcal{D}$, 第二项在取期望后不变; 第一项的期望按定义就是方差的被积函数; 交叉项则消失, 因为

$$\mathbb{E}_{\mathcal{D}}\big[f(\boldsymbol{x};\mathcal{D})-\bar{f}(\boldsymbol{x})\big]=\mathbb{E}_{\mathcal{D}}\big[f(\boldsymbol{x};\mathcal{D})\big]-\bar{f}(\boldsymbol{x})=0$$

而 $\{\bar{f}(\boldsymbol{x})-h(\boldsymbol{x})\}$ 关于 $\mathcal{D}$ 是常数. 于是

$$\mathbb{E}_{\mathcal{D}}\big[\{f(\boldsymbol{x};\mathcal{D})-h(\boldsymbol{x})\}^2\big]=\big\{\mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x};\mathcal{D})]-h(\boldsymbol{x})\big\}^2+\mathbb{E}_{\mathcal{D}}\big[\{f(\boldsymbol{x};\mathcal{D})-\mathbb{E}_{\mathcal{D}}[f(\boldsymbol{x};\mathcal{D})]\}^2\big]. \tag{4.53}$$

第四步, 对 $\boldsymbol{x}$ 积分并代回(4.46). 在绝对收敛的假设下 $\mathbb{E}_{\mathcal{D}}$ 与 $\int(\cdot)p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}$ 可以交换次序, 于是

$$\begin{aligned}&\int\mathbb{E}_{\mathcal{D}}\big[\{f(\boldsymbol{x};\mathcal{D})-h(\boldsymbol{x})\}^2\big]p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}\\&=\int\{\bar{f}(\boldsymbol{x})-h(\boldsymbol{x})\}^2p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}+\int\mathbb{E}_{\mathcal{D}}\big[\{f(\boldsymbol{x};\mathcal{D})-\bar{f}(\boldsymbol{x})\}^2\big]p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x}.\end{aligned}$$

把它与(4.46)的第二项 (即(4.50)) 相加, 即得(4.47)与(4.48)–(4.50).

□

注

(重构新增) 定理 4.1把期望损失拆成三个来源不同的部分, 每一项的含义如下.

平方偏差(4.48)衡量“平均模型”与真实回归函数 h 的差距: 它把 $\mathcal{D}$ 的所有可能取值上的 f 先平均, 再与 h 比较, 因此刻画的是模型族的系统性误差. 若模型族足够丰富, 存在某个 f 等于 h, 则平方偏差可降为零.

方差(4.49)衡量 $f(\boldsymbol{x};\mathcal{D})$ 随训练集 $\mathcal{D}$ 的波动: 它只关心各个 f 相对其均值 $\bar{f}$ 偏离多少, 而不管 $\bar{f}$ 是否等于 h. 高方差的模型对数据的微小扰动极为敏感, 这正是过拟合的表现.

噪声(4.50)与 f 无关, 是数据本身的内在随机性; 任何预测函数都无法把它降低, 它是期望损失的下界.

三项之间的关系构成本节的核心权衡: 模型越灵活, 平方偏差越小而方差越大; 模型越保守, 方差越小而平方偏差越大.λ(或基函数的个数 M) 正是控制这一权衡的旋钮, 见例 4.1.

关于条件, 需要说明三点. 第一, “数据点独立地取自 $p(t\mid\boldsymbol{x})$”这一假设使(4.49)中的期望有定义: 没有它, 就不存在“大量独立数据集”这一思想实验. 第二, 各积分绝对收敛的假设保证第三步中期望与积分的交换合法; 若 f 的二阶矩不存在, 则(4.47)的两端可能同时为 $+\infty$. 第三, 本分解建立在“对数据集求平均”之上, 而实际只观测到一个 $\mathcal{D}$, 因此它解释损失的结构, 并不能被直接计算; 见本节末尾的讨论.

我们的目标是最小化期望损失, 我们已经将其分解为平方偏差项、方差项和常数噪声项的和. 偏差和方差之间存在权衡: 非常灵活的模型具有低偏差和高方差, 而相对稳定的模型具有高偏差和低方差. 具有最优预测能力的模型能够在偏差和方差之间达到最佳平衡. 这可以用前面介绍过的正弦数据集来说明 (参见节 1.2).

例题 4.1 正弦数据集上的偏差与方差 独立生成 $L=100$ 个数据集, 每个数据集包含 $N=25$ 个数据点, 数据点由正弦曲线 $h(x)=\sin(2\pi x)$ 产生. 数据集的索引是 $l=1,\cdots,L$, 其中 $L=100$. 对于每个数据集 $\mathcal{D}^{(l)}$, 拟合一个包含 $M=24$ 个 Gauss 基函数和一个常数“偏差”基函数的模型, 总共 25 个参数. 如图 4.7所示, 通过最小化正则化误差函数(4.29), 可以得到预测函数 $f^{(l)}(x)$.

图 4.7的最上面一行对应的正则化系数 λ 很大, 即方差很低 (因为左图中的红色曲线彼此之间相似), 但偏差很高 (因为右图中的两条曲线差异很大). 相反, 在 λ 较小的最下面一行, 方差较高 (表现为左图中红色曲线之间的高变异性), 但偏差较低 (表现为平均模型与原始正弦函数之间拟合良好). 我们发现复杂模型 ($M=25$) 多次拟合平均后的结果对回归函数的拟合效果非常好, 这说明平均确实颇有裨益. 虽然是关于参数的后验分布的平均, 而不是关于多个数据集的平均, 但是多个解的加权平均是 Bayes 方法的核心.

我们还可以定量地检查这个示例中的偏差-方差权衡. 平均预测是用下式估计的:

$$\bar{f}(x)=\frac{1}{L}\sum_{l=1}^{L}f^{(l)}(x) \tag{4.54}$$

而平方偏差和方差则由下式给出:

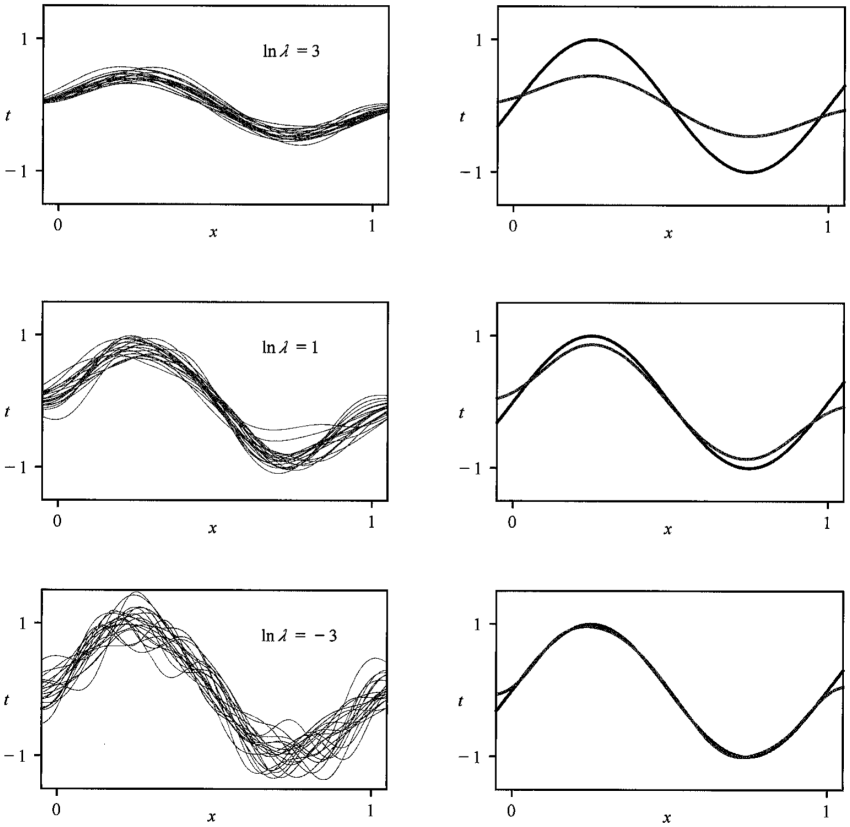


图 4.7: 使用节 1.2的正弦数据集展示模型复杂度对偏差和方差的依赖关系, 模型复杂度由正则化参数 λ 控制. 这里有 $L = 100$ 个数据集, 每个数据集有 $N = 25$ 个数据点, 模型中有 24 个 Gauss 基函数, 因此包括偏差参数在内的参数总数为 $M = 25$. 左列显示了模型拟合结果随 $\ln\lambda$ 变化的结果 (为了清晰起见, 只显示了 100 次拟合中的 20 次). 右列显示了 100 次拟合的平均结果 (红色曲线) 以及生成数据集的正弦函数 (绿色曲线)

$$\text{平方偏差} = \frac{1}{N}\sum_{n=1}^{N}\{\bar{f}(x_n) - h(x_n)\}^2 \tag{4.55}$$

$$\text{方差} = \frac{1}{N}\sum_{n=1}^{N}\frac{1}{L}\sum_{l=1}^{L}\{f^{(l)}(x_n) - \bar{f}(x_n)\}^2 \tag{4.56}$$

其中, 由分布 $p(x)$ 加权的对 x 的积分, 可通过该分布产生的数据点的有限和来近似.图 4.8给出了这些量在不同 $\ln\lambda$ 取值时的曲线. 可以看到, 较小的 λ 值允许模型对每个单独数据集上的噪声进行微调, 从而导致较大的方差. 相反, 较大的 λ 值会使权重参数向 0 靠近, 从而导致较大的偏差.

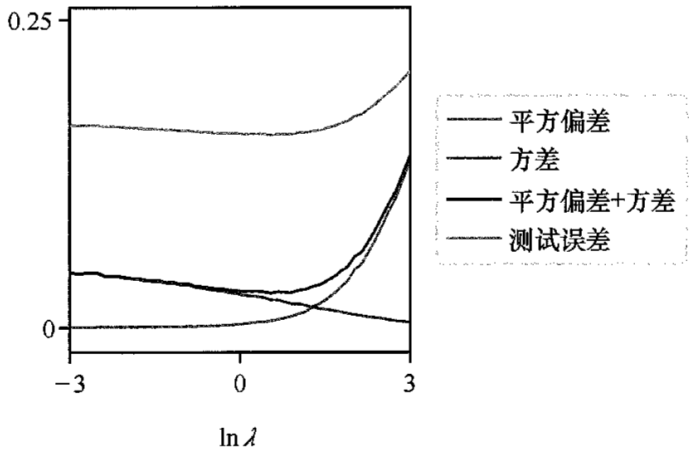


图 4.8: 根据图 4.7所示的结果, 绘制的平方偏差、方差以及它们的和的示意图. 图中还显示了测试数据集大小为 1000 个数据点时的平均测试集误差. 平方偏差 + 方差的最小值出现在 $\ln\lambda = 0.43$ 附近, 接近于在测试数据上给出最小误差的值

偏差-方差分解的实际价值是有限的, 因为它基于数据集的集合的平均, 而实际上我们只有单个观测数据集. 如果我们有大量给定大小的独立训练集, 则更好的方法是将它们组合成单个更大的训练集, 从而降低给定复杂度模型的过拟合风险. 但尽管如此, 偏差-方差分解仍经常提供对模型复杂性问题的有用洞察. 虽然我们在本章中只是从回归问题的角度介绍了它, 但其底层逻辑具有广泛的适用性.

4.4 习题

✍ **练习 4.1** (★) 考虑式(1.2)给出的平方和误差函数, 其中函数 $y(x, \boldsymbol{w})$ 由多项式(1.1)给出. 证明能使该误差函数最小的系数 $\boldsymbol{w} = \{w_i\}$ 是由下列线性方程组的解给出的:

$$\sum_{j=0}^{M} A_{ij} w_j = T_i \tag{4.57}$$

其中

$$A_{ij} = \sum_{n=1}^{N}(x_n)^{i+j}, \qquad T_i = \sum_{n=1}^{N}(x_n)^i t_n. \tag{4.58}$$

✍ **练习 4.2** (★) 写出类似于式(4.57)的耦合线性方程组, 其中系数 w_i 能使式(1.4)给出的正则化平方和误差函数最小化.

✍ **练习 4.3** (★) 证明定义如下的 tanh 函数

$$\tanh(a) = \frac{e^a - e^{-a}}{e^a + e^{-a}} \tag{4.59}$$

和式(4.6)定义的 sigmoid 函数有如下关系:

$$\tanh(a) = 2\sigma(2a) - 1 \tag{4.60}$$

并进一步证明 sigmoid 函数的一般线性组合形式

$$y(x, \boldsymbol{w}) = w_0 + \sum_{j=1}^{M} w_j \sigma\Big(\frac{x - \mu_j}{s}\Big) \tag{4.61}$$

等价于如下形式的 tanh 函数的线性组合:

$$y(x, \boldsymbol{u}) = u_0 + \sum_{j=1}^{M} u_j \tanh\Big(\frac{x - \mu_j}{2s}\Big) \tag{4.62}$$

最后找到将新参数 $\{u_1, \cdots, u_M\}$ 与原始参数 $\{w_1, \cdots, w_M\}$ 关联起来的表达式.

✍ **练习 4.4** (★★★) 证明矩阵

$$\boldsymbol{\Phi}(\boldsymbol{\Phi}^{\mathsf{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathsf{T}} \tag{4.63}$$

可将任何向量 $\boldsymbol{v}$ 投影到 $\boldsymbol{\Phi}$ 的列所张成的空间中. 使用此结果证明, 最小二乘解式(4.16)对应向量 $\boldsymbol{t}$ 在流形 $\mathcal{S}$ 上的正交投影 (见图 4.3).

✍ **练习 4.5** (★) 考虑一个每个数据点 t_n 都与加权因子 $r_n > 0$ 相关联的数据集, 平方和误差函数变为

$$E_D(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N} r_n \big\{t_n - \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\big\}^2. \tag{4.64}$$

给出最小化该误差函数的解 $\boldsymbol{w}^*$ 的表达式. 从与数据相关的噪声方差和复制的数据点两方面, 给出加权平方和误差函数的两种替代解释.

✍ **练习 4.6** (★) 通过将式(4.29)对 $\boldsymbol{w}$ 的梯度设置为 $\boldsymbol{0}$, 证明在线性回归中, 正则化平方和误差函数的精确最小值由式(4.30)给出.

✍ **练习 4.7** (★★) 假设多元目标变量 $\boldsymbol{t}$ 的线性基函数回归模型具有下式给出的 Gauss 分布:

$$p(\boldsymbol{t} \mid \boldsymbol{W}, \boldsymbol{\Sigma}) = \mathcal{N}\big(\boldsymbol{t} \mid \boldsymbol{y}(\boldsymbol{x}, \boldsymbol{W}), \boldsymbol{\Sigma}\big) \tag{4.65}$$

其中

$$\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{W}) = \boldsymbol{W}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}). \tag{4.66}$$

训练数据集由输入基向量 $\boldsymbol{\phi}(\boldsymbol{x}_n)$ 和对应的目标向量 $\boldsymbol{t}_n$ 组成, 其中 $n = 1, \cdots, N$. 证明参数矩阵 $\boldsymbol{W}$ 的最大似然解 $\boldsymbol{W}_{\mathrm{ML}}$ 具有如下性质: 每列由形如式(4.16)的表达式给出, 其是各向同性噪声分布的解. 注意, 它独立于协方差矩阵 $\boldsymbol{\Sigma}$. 证明 $\boldsymbol{\Sigma}$ 的最大似然解由下式给出:

$$\boldsymbol{\Sigma} = \frac{1}{N}\sum_{n=1}^{N} \big(\boldsymbol{t}_n - \boldsymbol{W}_{\mathrm{ML}}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\big)\big(\boldsymbol{t}_n - \boldsymbol{W}_{\mathrm{ML}}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}_n)\big)^{\mathsf{T}}. \tag{4.67}$$

✍ **练习 4.8** (★) 考虑将单个目标变量 t 的平方损失函数式(4.38)推广到下式给出的多目标变量 (用向量 $\boldsymbol{t}$ 表示), 则有

$$\mathbb{E}\big[L(\boldsymbol{t}, \boldsymbol{f}(\boldsymbol{x}))\big] = \iint \big\|\boldsymbol{f}(\boldsymbol{x}) - \boldsymbol{t}\big\|^2 p(\boldsymbol{x}, \boldsymbol{t})\, \mathrm{d}\boldsymbol{x}\, \mathrm{d}\boldsymbol{t}. \tag{4.68}$$

使用变分演算, 证明使预期损失最小化的函数 $\boldsymbol{f}(\boldsymbol{x})$ 由式(4.69)给出:

$$\boldsymbol{f}(\boldsymbol{x}) = \mathbb{E}_{\boldsymbol{t}}[\boldsymbol{t} \mid \boldsymbol{x}]. \tag{4.69}$$

✍ **练习 4.9** (★) 通过对式(4.68)中的平方项进行展开, 推导类似于式(4.43)的结果. 进一步地, 证明使多元目标变量 $\boldsymbol{t}$ 的预期平方损失最小化的函数 $\boldsymbol{f}(\boldsymbol{x})$ 同样以式(4.69)的形式由 $\boldsymbol{t}$ 的条件期望给出.

✍ **练习 4.10** (★★) 类比式(4.43), 通过扩展式(4.68)来重新推导结果式(4.69).

✍ **练习 4.11** (★★) 以下分布

$$p(x \mid \sigma^2, q) = \frac{q}{2(2\sigma^2)^{1/q}\Gamma(1/q)} \exp\Big(-\frac{|x|^q}{2\sigma^2}\Big) \tag{4.70}$$

是一元 Gauss 分布的推广形式. 其中 $\Gamma(x)$ 是由下式定义的伽马函数:

$$\Gamma(x) = \int_{-\infty}^{\infty} u^{x-1} \mathrm{e}^{-u} \,\mathrm{d}u. \tag{4.71}$$

证明此分布已经归一化, 从而有

$$\int_{-\infty}^{\infty} p(x \mid \sigma^2, q) \,\mathrm{d}x = 1, \tag{4.72}$$

且当 $q = 2$ 时, 它退化为 Gauss 分布. 考虑一个目标变量由 $t = y(\boldsymbol{x}, \boldsymbol{w}) + \varepsilon$ 给出的回归模型,ε 是从分布(4.70)中产生的随机噪声变量. 证明对于输入向量 $X = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 和相应的目标变量 $\boldsymbol{t} = (t_1, \cdots, t_N)^{\mathsf{T}}$ 的数据集, 定义在 $\boldsymbol{w}$ 和 σ^2 上的对数似然函数由下式给出:

$$\ln p(\boldsymbol{t} \mid X, \boldsymbol{w}, \sigma^2) = -\frac{1}{2\sigma^2} \sum_{n=1}^{N} \left| y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n \right|^q - \frac{N}{q} \ln(2\sigma^2) + \text{const}, \tag{4.73}$$

其中 “const” 表示独立于 $\boldsymbol{w}$ 和 σ^2 的常数项. 注意, 作为 $\boldsymbol{w}$ 的函数, 该对数似然函数是节 4.2中的 L_q 误差函数.

注

(重构新增: 原文问题说明) 原页把伽马函数印成 $\Gamma(x) = \int_{-\infty}^{\infty} u^{x-1}\mathrm{e}^{-u}\,\mathrm{d}u$(下限为 $-\infty$), 这一写法有误: 当 $u \to -\infty$ 且 $x-1$ 非整数时 u^{x-1} 无定义, 该积分并不收敛. 本译本在正文中照录了这一印刷形式, 但在归一化证明中采用了下限为 0 的标准定义, 即 $\Gamma(x) = \int_{0}^{\infty} u^{x-1}\mathrm{e}^{-u}\,\mathrm{d}u$; 归一化证明(4.72)正是在这一形式下进行的. 除此之外原题的条件与结论不受影响.

✍ **练习 4.12** (★★) 考虑由 L_q 损失函数定义的回归问题的期望损失式(4.44). 写出在最小化 $\mathbb{E}[L_q]$ 时,$y(\boldsymbol{x})$ 必须满足的条件. 证明当 $q = 1$ 时, 该解表示条件中位数, 即函数 $y(\boldsymbol{x})$ 满足 $t < y(\boldsymbol{x})$ 的概率与 $t \geqslant y(\boldsymbol{x})$ 的概率相同; 并证明当 $q \to 0$ 时最小期望损失 L_q 由条件众数给出, 即函数 $y(\boldsymbol{x})$ 等于对每个 $\boldsymbol{x}$ 而言, 能使 $p(t \mid \boldsymbol{x})$ 最大化的 t 值.

第 5 章 单层网络: 分类

第 4 章探讨了一类回归模型, 其中输出变量是模型参数的线性函数, 因此可表示为具有单层权重和偏置参数的简单神经网络. 接下来, 我们将目光转向分类问题. 本章将重点关注类似的一类模型, 这类模型同样可以表示为单层神经网络. 由此, 本章先介绍诸多有关分类的关键概念, 以便在后面各章讨论更为通用的深度神经网络.

分类旨在取一个输入向量 $\boldsymbol{x} \in \mathbb{R}^D$, 并将其归入 K 个离散类 C_k 中的一个类, 其中 $k = 1, \cdots, K$. 在最常见的场景中, 这些类是不相交的, 因此每个输入向量都只能归入一个类. 本章使用线性模型进行分类, 这意味着决策面是输入向量 $\boldsymbol{x}$ 的线性函数, 由 D 维输入空间内的 $(D-1)$ 维超平面定义. 线性分类模型可应用于非线性可分数据集, 但并非所有的输入向量都能进行正确的分类.

定义 5.1 (决策区域、决策边界与线性可分)

将输入空间划分为若干区域 $\mathcal{R}_k$, 并令每个区域对应一个类 C_k, 即约定当且仅当 $\boldsymbol{x} \in \mathcal{R}_k$ 时把 $\boldsymbol{x}$ 归入 C_k 类. 这些区域称为决策区域 (decision region). 决策区域之间的边界称为决策边界 (decision boundary) 或决策面 (decision surface). 若存在一个线性决策面, 使得数据集中的每个输入向量都被归入其所属的类, 则称该数据集是线性可分 (linearly separable) 的. ♣

注

(重构新增) 引入决策区域是为了把分类器的工作方式几何化: 一个分类器等价于一个从输入空间到类标签的映射, 而该映射的等值面就是决策面. 注意定义并不要求 $\mathcal{R}_k$ 连通, 这一点在小节 5.2.1 中会明确写出: 一个决策区域可以是若干互不相交的区域之并. 定义中的“线性”一词限定的是决策面的形状, 而不是类-条件密度的形状; 因此同一个线性决策面既可以由线性判别函数给出, 也可以由共享协方差矩阵的 Gauss 类-条件密度经 Bayes 定理导出 (见小节 5.3.1).$D = 1$ 时超平面的维数为 0, 即决策面退化为单个点, 此时“线性可分”等价于存在一个阈值把两类数据分开. 线性可分的定义依赖于给定的决策面族: 一个数据集对线性决策面不可分, 对二次决策面却可能可分 (见定义 5.19).

我们可以大致确定解决分类问题的 3 种方法 (参见小节 5.2.4). 其中, 最简单的方法是构建一个判别函数 (discriminant function), 该函数直接将每一个输入向量 $\boldsymbol{x}$ 归入一个特定类. 然而, 一种更有效的方法是在推理阶段依据条件概率 $p(C_k \mid \boldsymbol{x})$ 构建分布模型, 然后利用这些分布模型做出最优决策. 将推理和决策分开能带来很多好处.

定义 5.2 (判别函数)

判别函数 (discriminant function) 是把每个输入向量 $\boldsymbol{x}$ 直接映射到某个类标签上的函数 $f(\boldsymbol{x})$; 等价地说, 它不借助任何概率模型, 直接把输入空间划分为若干决策区域. ♣

注

(重构新增) 判别函数的动机是把小节 5.2.4 中的“推理”与“决策”两个阶段合并为一个学习问题: 只需求出 f, 无需知道任何后验概率. 其代价是丢失了概率信息: 当损失矩阵改变、需要设置拒绝选项或需要补偿类的先验时, 都必须重新训练 (这些理由在小节 5.2.4 中逐条给出). 注意“判别函数”与“判别式”在本书中同义使用: 前者强调它是一般的映射, 后者强调它确定决策边界. 对二分类问题, 判别函数可以取为 $y(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0$ 的符号, 此时 f 的值域只有两个元素; 对 K 类问题, 判别函数常取为 $\arg\max_k y_k(\boldsymbol{x})$(见小节 5.1.2).

有两种不同的方法可以确定条件概率 $p(C_k \mid \boldsymbol{x})$, 它们的区别在于对哪一组分布做参数假设.

定义 5.3 (判别概率模型与生成概率模型)

设 $p(C_k \mid \boldsymbol{x})$ 为给定输入 $\boldsymbol{x}$ 时类 C_k 的后验概率.

(i) 直接对后验概率 $p(C_k \mid \boldsymbol{x})$ 建模, 例如把它表示为参数模型并用训练集优化参数, 这样得到的方法称为判别概率模型 (discriminative probabilistic model).

(ii) 先对类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 以及类的先验概率 $p(C_k)$ 建模, 再使用 Bayes 定理计算所需的后验概

率, 这样得到的方法称为生成概率模型 (generative probabilistic model).

$$p(C_k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_k)\, p(C_k)}{p(\boldsymbol{x})} \tag{5.1}$$

注

(重构新增) 两种建模方式的分界在于“对谁做参数假设”. 判别概率模型只假设后验概率的函数形式, 可学习参数通常较少; 生成概率模型要对输入空间的密度做假设, 参数通常更多, 但它带来两个额外能力: 其一, 可从 $p(\boldsymbol{x} \mid C_k)$ 采样生成合成数据; 其二, 可由(5.1)的分母 $p(\boldsymbol{x}) = \sum_k p(\boldsymbol{x} \mid C_k)p(C_k)$ 得到边缘密度, 从而识别概率很低的新数据点 (离群值检测). 式(5.1)本身只是概率的乘积法则与加和法则的推论, 对 $p(\boldsymbol{x}) > 0$ 成立; 它并不假定任何具体的密度形式.$p(\boldsymbol{x})$ 的作用只是归一化, 因此在比较大小时可以约去.

此为生成概率模型, 它提供了从每个类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 生成样本的方法. 本章仅关注线性判别式 (linear discriminant), 即决策面为超平面的判别式.

定义 5.4 (线性判别式)

若一个判别式 (判别函数) 的决策面是输入空间中的超平面, 则称它为线性判别式 (linear discriminant).

注

(重构新增) 线性判别式在本章中会出现三种等价的外形: 单个线性函数 $y(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0$ 的符号 (二分类, 见小节 5.1.1);K 个线性函数中取最大者 (多分类, 见小节 5.1.2); 以及后验概率经 logistic sigmoid 或 softmax 变换后的最大值 (见小节 5.3.1与小节 5.4.3). 之所以值得单独研究, 是因为决策面为超平面时, 其几何性质 (法向量、到原点的距离、决策区域的凸性) 可以完全由参数 $\boldsymbol{w}$ 与 w_0 读出, 而且这些性质不依赖于用哪种方式得到该超平面.

5.1 判别函数

判别式是一种函数, 它取输入向量 $\boldsymbol{x}$ 并将其归入 K 个类 (用 C_k 表示) 中的一个类, 也就是定义 5.2意义下的判别函数. 本章仅关注线性判别式 (linear discriminant), 即决策面为超平面的判别式 (定义 5.4). 为了简化讨论, 在研究多分类 ($K > 2$) 之前, 我们首先分析二分类.

5.1.1 二分类

线性判别函数的最简单表示是通过取输入向量的线性函数得出的, 即

$$y(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0 \tag{5.2}$$

其中,$\boldsymbol{w}$ 为权重向量,w_0 为偏置参数 (请勿与统计学意义上的偏差混淆). 如果 $y(\boldsymbol{x}) \geqslant 0$, 输入向量 $\boldsymbol{x}$ 将被归入 C_1 类, 否则归入 C_2 类. 因此, 相应的决策边界由关系 $y(\boldsymbol{x}) = 0$ 定义, 对应于 D 维输入空间中的 $(D-1)$ 维超平面.

命题 5.1 (决策面的几何性质)

设 $\boldsymbol{w} \neq \boldsymbol{0}$,$y(\boldsymbol{x}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0$.

(i) 决策面 $\{\boldsymbol{x} : y(\boldsymbol{x}) = 0\}$ 是 $\mathbb{R}^D$ 中的一个 $(D-1)$ 维超平面, 且 $\boldsymbol{w}$ 是它的一个法向量;

(ii) 原点到该决策面的带号距离为 $-w_0/\|\boldsymbol{w}\|$, 其中正号取在 $\boldsymbol{w}$ 所指的方向上.

证明

(原书论证) 考虑两个点 $\boldsymbol{x}_A$ 和 $\boldsymbol{x}_B$, 它们都位于决策面上. 因为 $y(\boldsymbol{x}_A) = y(\boldsymbol{x}_B) = 0$, 所以得出 $\boldsymbol{w}^{\mathsf{T}}(\boldsymbol{x}_A - \boldsymbol{x}_B) = 0$. 由于 $\boldsymbol{x}_A - \boldsymbol{x}_B$ 是决策面内的任意方向, 故 $\boldsymbol{w}$ 与位于决策面内的每个向量都正交,$\boldsymbol{w}$ 决定了决策面的朝向.

(重构新增) 下面补足维数与距离两部分. 记 $H = \{\boldsymbol{x} : \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0 = 0\}$. 若 $\boldsymbol{w} \neq \boldsymbol{0}$, 则 H 非空: 取 $\boldsymbol{x}_0 = -w_0\boldsymbol{w}/\|\boldsymbol{w}\|^2$ 即可验证 $y(\boldsymbol{x}_0) = 0$.H 是仿射子空间, 其方向空间为 $\{\boldsymbol{v} : \boldsymbol{w}^{\mathsf{T}}\boldsymbol{v} = 0\}$, 即 $\boldsymbol{w}$ 的正交补; 由于 $\boldsymbol{w} \neq \boldsymbol{0}$, 该正交补的维数为 $D-1$, 故 H 的维数为 $D-1$. 又对任意 $\boldsymbol{x} \in H$, 有 $\boldsymbol{x} - \boldsymbol{x}_0 \in \boldsymbol{w}^{\perp}$, 故 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}_0 = -w_0$, 于是

$$\frac{\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}}{\|\boldsymbol{w}\|} = -\frac{w_0}{\|\boldsymbol{w}\|} \tag{5.3}$$

左端正是 $\boldsymbol{x}$ 在 $\boldsymbol{w}$ 方向上的投影长度, 它与 $\boldsymbol{x} \in H$ 的选取无关, 因此它就是原点到决策面的法向距离, 符号由 $\boldsymbol{w}$ 的方向确定.

□

注

(重构新增) 命题 5.1的两个结论分别说明了参数的几何含义:$\boldsymbol{w}$ 只决定决策面的朝向 (改变 $\boldsymbol{w}$ 的方向会旋转超平面), 而 w_0 只决定决策面的位置 (改变 w_0 会平移超平面, 且平移量为 $-w_0/\|\boldsymbol{w}\|$). 条件 $\boldsymbol{w} \neq \boldsymbol{0}$ 不可去掉: 若 $\boldsymbol{w} = \boldsymbol{0}$, 则 $y \equiv w_0$ 为常数, 决策面或者为空集 ($w_0 \neq 0$) 或者为整个空间 ($w_0 = 0$), 此时既没有法向量也没有到原点的距离. 结论 (ii) 中的“带号”不能省略: 若只关心距离, 则应为 $|w_0|/\|\boldsymbol{w}\|$; 带号版本在下一段讨论点到决策面的带号距离时是必要的. 注意 $\boldsymbol{w}$ 与 w_0 同时乘以一个正数不改变决策面, 因此线性判别式的参数并不唯一, 这一点在讨论最大似然的奇异性时还会出现 (见命题 5.21).

可以看到, 偏置参数 w_0 决定了决策面的位置.图 5.1展示了 $D = 2$ 情况下的此类特性.

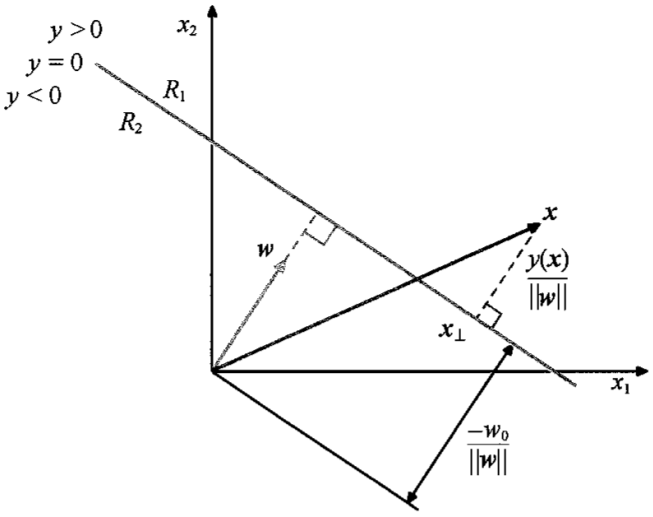


图 5.1: 二维线性判别函数的几何图解. 红色所示的决策面垂直于 $\boldsymbol{w}$, 它离原点的位移由偏置参数 w_0 控制. 同样, 点 $\boldsymbol{x}$ 与决策面的带号正交距离由 $y(\boldsymbol{x})/\|\boldsymbol{w}\|$ 给出

此外, 请注意,$y(\boldsymbol{x})$ 的值为点 $\boldsymbol{x}$ 到决策面的垂直距离 r 的带号测度. 为了方便理解, 考虑任意一个点 $\boldsymbol{x}$, 设 $\boldsymbol{x}_{\perp}$ 是它在决策面上的正交投影.

命题 5.2 (点到决策面的带号距离)

设 $\boldsymbol{w} \neq \boldsymbol{0}$,$\boldsymbol{x}_{\perp}$ 是 $\boldsymbol{x}$ 在决策面 $y(\boldsymbol{x}) = 0$ 上的正交投影,r 为 $\boldsymbol{x}$ 沿 $\boldsymbol{w}$ 方向到该决策面的带号距离. 则

$$\boldsymbol{x} = \boldsymbol{x}_{\perp} + r\frac{\boldsymbol{w}}{\|\boldsymbol{w}\|} \tag{5.4}$$

并且

$$r = \frac{y(\boldsymbol{x})}{\|\boldsymbol{w}\|} \tag{5.5}$$

♠

证明

(重构新增) 因为 $\boldsymbol{x}_\perp$ 是 $\boldsymbol{x}$ 在决策面上的正交投影, 所以 $\boldsymbol{x} - \boldsymbol{x}_\perp$ 与决策面正交. 由命题 5.1(i), 决策面的法向量为 $\boldsymbol{w}$, 故 $\boldsymbol{x} - \boldsymbol{x}_\perp$ 平行于 $\boldsymbol{w}$, 即存在实数 r 使(5.4)成立; 这里 r 的符号约定为沿 $\boldsymbol{w}$ 方向为正, 其绝对值就是 $\boldsymbol{x}$ 到决策面的距离.

在(5.4)的两边左乘 $\boldsymbol{w}^\mathsf{T}$ 并加上 w_0, 利用 $y(\boldsymbol{x}) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + w_0$ 以及 $\boldsymbol{x}_\perp$ 位于决策面上即 $y(\boldsymbol{x}_\perp) = \boldsymbol{w}^\mathsf{T}\boldsymbol{x}_\perp + w_0 = 0$, 可得

$$y(\boldsymbol{x}) = y(\boldsymbol{x}_\perp) + r\,\frac{\boldsymbol{w}^\mathsf{T}\boldsymbol{w}}{\|\boldsymbol{w}\|} = r\,\|\boldsymbol{w}\| \tag{5.6}$$

由于 $\boldsymbol{w} \neq \boldsymbol{0}$, 两边除以 $\|\boldsymbol{w}\|$ 即得(5.5).

□

注

(重构新增) 命题 5.2说明 $y(\boldsymbol{x})$ 本身就是一个“带尺度”的距离: 它等于真实距离乘以 $\|\boldsymbol{w}\|$. 因此 $y(\boldsymbol{x})$ 的符号给出分类结果, 而 $y(\boldsymbol{x}) = 0$ 恰好是决策面. 这个解释在小节 5.3.1中很重要: 当后验概率写成 $p(C_1 \mid \boldsymbol{x}) = \sigma(y(\boldsymbol{x}))$ 时,y 的绝对值越大, 分类的置信度越高, 但置信度与到决策面的几何距离之间相差因子 $\|\boldsymbol{w}\|$. 若 $\|\boldsymbol{w}\|$ 很小, 则离决策面很远的数据点也可能有很小的 $|y|$, 这说明“用 $|y|$ 衡量置信度”依赖于参数的尺度, 而参数的尺度又可以通过重新缩放任意改变.

如同线性回归模型 (参见小节 4.1.1), 有时使用更简洁的符号较为方便, 即给出一个额外的虚拟“输入”值 $x_0 = 1$, 然后定义 $\tilde{\boldsymbol{w}} = (w_0, \boldsymbol{w})$ 和 $\tilde{\boldsymbol{x}} = (x_0, \boldsymbol{x})$, 从而有

$$y(\boldsymbol{x}) = \tilde{\boldsymbol{w}}^\mathsf{T}\tilde{\boldsymbol{x}} \tag{5.7}$$

在此情况下, 决策面是通过 $(D+1)$ 维扩展输入空间原点的 D 维超平面.

定义 5.5 (增广权重与增广输入)

在输入向量 $\boldsymbol{x} \in \mathbb{R}^D$ 之前添加虚拟输入 $x_0 = 1$, 得到增广输入向量 $\tilde{\boldsymbol{x}} = (x_0, \boldsymbol{x}) = (1, \boldsymbol{x}) \in \mathbb{R}^{D+1}$; 相应地, 在权重向量 $\boldsymbol{w}$ 之前添加偏置 w_0, 得到增广权重向量 $\tilde{\boldsymbol{w}} = (w_0, \boldsymbol{w}) \in \mathbb{R}^{D+1}$.

♣

注

(重构新增) 增广记号的作用是把仿射函数 $y = \boldsymbol{w}^\mathsf{T}\boldsymbol{x} + w_0$ 写成线性函数 $y = \tilde{\boldsymbol{w}}^\mathsf{T}\tilde{\boldsymbol{x}}$, 从而使偏置与权重在求导、矩阵化时享有同一地位;小节 5.1.4中的设计矩阵 $\tilde{\boldsymbol{X}}$ 正是由此得到的. 其几何代价是: 决策面在 $\mathbb{R}^D$ 中是仿射的 (不过原点), 而在 $\mathbb{R}^{D+1}$ 中是过原点的线性子空间, 维数为 D. 由于虚拟输入恒为 1, 增广向量只张成 $\mathbb{R}^{D+1}$ 中的一个 D 维仿射子集, 因此 $\tilde{\boldsymbol{w}}$ 并不唯一: 任何与所有训练增广输入都正交的方向都可以加到 $\tilde{\boldsymbol{w}}$ 上, 而不改变训练点处的预测 (但在新的 $\boldsymbol{x}$ 处预测可能改变). 这一观察在讨论最小二乘解需要伪逆时还会用到 (见命题 5.4).

5.1.2 多分类

下面将线性判别函数扩展到多分类 ($K > 2$). 我们可能会试图通过组合一些二分类判别函数来建立多分类判别式. 然而, 这将导致一些困难 (Duda and Hart, 1973).

假设一个模型有 $K-1$ 个分类器, 每个分类器都要解决一个二分类问题, 也就是将特定 C_k 类中的点与其他的点分开. 这就是所谓的一对多分类器.图 5.2的左图给出了一个涉及 3 个类的例子, 在该例中, 这种方法会导致输入空间的区域分类不明确.

另一种方法是使用 $K(K-1)/2$ 个二元判别函数, 且每一对可能的类都有一个二元判别函数. 这就是所谓的一对一分类器. 然后根据判别函数的多数票对每个点进行分类. 然而, 这也会导致区域分类不明确的问题, 如图 5.2的右图所示.

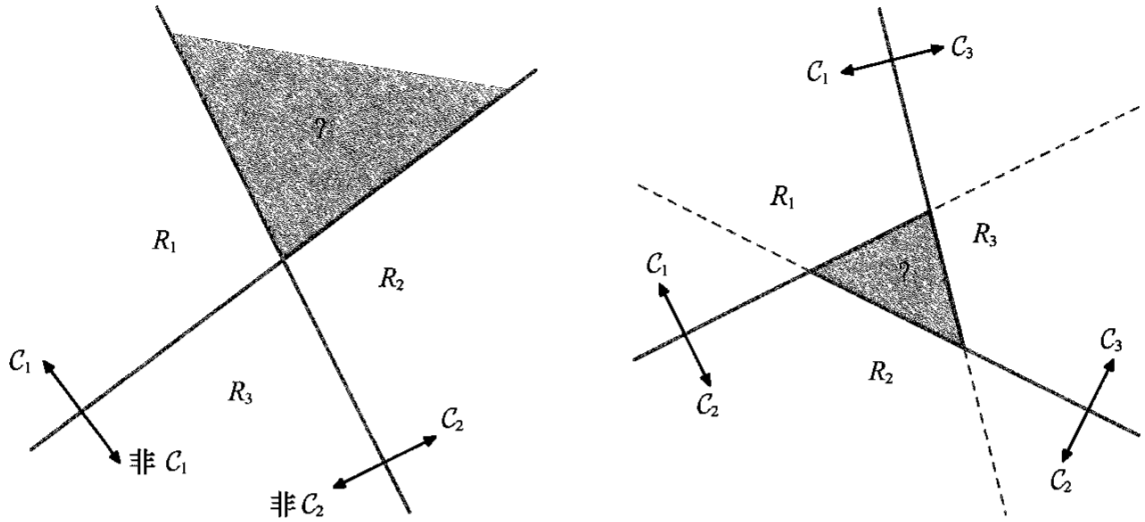


图 5.2: 尝试从一组二分类判别函数中构建一个 K 类判别式会导致分类不明确的区域, 如图中绿色所示. 左图给出了一个涉及两个判别函数的例子, 旨在对 C_1 类中的点与非 C_1 类中的点进行区分. 右图给出了一个涉及 3 个判别函数的例子, 其中的每个判别函数用于区分一对 C_k 类和 C_j 类

可通过采用 K 个线性函数组成的单个 K 类判别式来避免这些困难, 其形式如下:

$$y_k(\boldsymbol{x}) = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0} \tag{5.8}$$

然后在所有 $j \neq k$ 的情况下, 如果 $y_k(\boldsymbol{x}) > y_j(\boldsymbol{x})$, 就将点 $\boldsymbol{x}$ 归入 C_k 类. 因此,C_k 类和 C_j 类之间的决策边界由 $y_k(\boldsymbol{x}) = y_j(\boldsymbol{x})$ 给出, 且对应于一个 $(D-1)$ 维的超平面, 该超平面定义如下:

$$(\boldsymbol{w}_k - \boldsymbol{w}_j)^{\mathsf{T}}\boldsymbol{x} + (w_{k0} - w_{j0}) = 0 \tag{5.9}$$

由于它的形式与小节 5.1.1中讨论的二分类案例的决策边界相同, 因此其具有类似的几何特性.

这种判别式的决策区域总是单连且凸出的. 考虑两个点 $\boldsymbol{x}_A$ 和 $\boldsymbol{x}_B$, 它们都位于决策区域 $\mathcal{R}_k$ 内, 如图 5.3所示. 连接 $\boldsymbol{x}_A$ 和 $\boldsymbol{x}_B$ 的直线上的任何点 $\hat{\boldsymbol{x}}$ 都可以用如下形式来表示:

$$\hat{\boldsymbol{x}} = \lambda\boldsymbol{x}_A + (1-\lambda)\boldsymbol{x}_B \tag{5.10}$$

其中 $0 \leqslant \lambda \leqslant 1$. 根据判别函数的线性关系, 可得

$$y_k(\hat{\boldsymbol{x}}) = \lambda y_k(\boldsymbol{x}_A) + (1-\lambda)y_k(\boldsymbol{x}_B) \tag{5.11}$$

命题 5.3 (多类线性判别式的决策区域是凸的)

设 $y_k(\boldsymbol{x}) = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0}, k = 1, \cdots, K$, 并令 $\mathcal{R}_k = \{\boldsymbol{x} : y_k(\boldsymbol{x}) > y_j(\boldsymbol{x}) \text{ 对一切 } j \neq k\}$. 则每个 $\mathcal{R}_k$ 都是凸集; 若 $\mathcal{R}_k \neq \varnothing$, 则它还是单连通的.

证明

(原书论证) 因为 $\boldsymbol{x}_A$ 和 $\boldsymbol{x}_B$ 都在决策区域 $\mathcal{R}_k$ 内, 所以在所有 $j \neq k$ 的情况下,$y_k(\boldsymbol{x}_A) > y_j(\boldsymbol{x}_A)$ 且 $y_k(\boldsymbol{x}_B) > y_j(\boldsymbol{x}_B)$. 对(5.11)两端作同样的展开, 并对每个 $j \neq k$ 用这两个不等式, 得

$$y_k(\hat{\boldsymbol{x}}) = \lambda y_k(\boldsymbol{x}_A) + (1-\lambda) y_k(\boldsymbol{x}_B) > \lambda y_j(\boldsymbol{x}_A) + (1-\lambda) y_j(\boldsymbol{x}_B) = y_j(\hat{\boldsymbol{x}}) \tag{5.12}$$

(重构新增) 其中中间的不等号对 $\lambda \in [0,1]$ 成立: 当 $\lambda = 0$ 或 $\lambda = 1$ 时它退化为端点处已证的严格不等式, 当 $0 < \lambda < 1$ 时两个严格不等式分别乘以正系数 λ 与 $1-\lambda$ 后相加. 于是 $\hat{\boldsymbol{x}} \in \mathcal{R}_k$, 即 $\mathcal{R}_k$ 对任意两点之间的线段封闭, 故为凸集. 凸集是单连通的, 因此 $\mathcal{R}_k$ 单连且凸出. 注意 $\mathcal{R}_k$ 是有限个开半空间之交, 故它还是开集; 边界 $\partial\mathcal{R}_k$ 由若干超平面片组成, 这些超平面片正是形如(5.9)的决策边界的部分.

□

注

(重构新增) 凸性来自“取最大”这一运算与线性函数之间的相容性:$\mathcal{R}_k$ 是 $K-1$ 个开半空间 $\{\boldsymbol{x} : (\boldsymbol{w}_k - \boldsymbol{w}_j)^{\mathsf{T}}\boldsymbol{x} + (w_{k0} - w_{j0}) > 0\}$ 的交, 而半空间是凸集. 命题中的严格不等号意味着决策面上的点不属于任何 $\mathcal{R}_k$, 这些点可以任意归入相邻类之一而不改变决策区域的凸性. 需要强调, 凸性是这种“K 个线性函数取最大”的判别式所特有的: 若改用一对一或一对多组合, 决策区域一般不是凸的, 甚至可能出现图 5.2所示的既不连通也不单值的区域. 此外,$\mathcal{R}_k$ 可以为空: 当某个类在参数上被其他类完全“压倒”时, 没有任何输入会被归入该类.

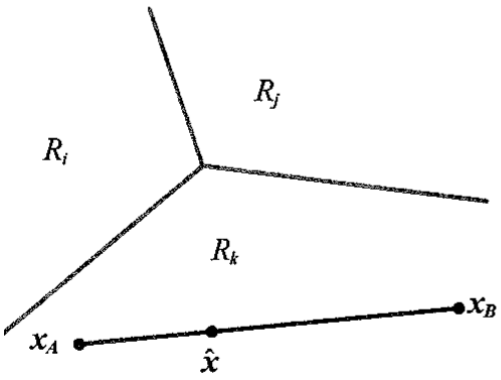


图 5.3: 多分类线性判别式的决策区域, 决策边界如红色所示. 如果两点 $\boldsymbol{x}_A$ 和 $\boldsymbol{x}_B$ 都在同一决策区域 $\mathcal{R}_k$ 内, 那么连接这两点的直线上的任何一点也必须在决策区域 $\mathcal{R}_k$ 内, 从而决策区域必须是单连且凸出的

请注意, 对于二分类问题, 我们既可以采用书中讨论的基于两个判别函数 $y_1(\boldsymbol{x})$ 和 $y_2(\boldsymbol{x})$ 的形式, 也可以采用基于单个判别函数 $y(\boldsymbol{x})$ 的更简单但本质上等价的形式 (参见小节 5.1.1).

5.1.3 1-of-K 编码方案

对于回归问题, 多元目标变量 $\boldsymbol{t}$ 只是我们希望预测其值的实数向量. 在分类问题中, 有多种使用目标值来表示类标签的方法. 对于二分类问题, 最方便的是单目标变量 $t \in \{0,1\}$ 的二进制表示, 所以 $t=1$ 表示 C_1 类,$t=0$ 表示 C_2 类. 我们可将 t 的值理解为该类是 C_1 类的概率, 概率值仅取 0 和 1 的极值. 对于多分类 ($K > 2$) 问题, 使用 1-of-K 编码方案较为方便, 其也称独热编码方案.

定义 5.6 (1-of-K 编码方案)

设共有 K 个类. 若把类标签编码为长度为 K 的向量 $\boldsymbol{t} = (t_1, \cdots, t_K)^{\mathsf{T}}$, 使得当且仅当样本属于 C_k 类时 $t_k = 1$ 而其余分量 $t_j = 0 (j \neq k)$, 则称这种编码为 1-of-K 编码方案, 也称独热 (one-hot) 编码方案.

♣

例如, 假设有 $K=5$ 个类, 则来自 C_2 类的数据点将得到目标向量

$$\boldsymbol{t} = (0, 1, 0, 0, 0)^{\mathsf{T}} \tag{5.13}$$

同样, 我们可以把 t_k 的值理解为该类是 C_k 类的概率, 其中概率仅有 0 和 1 两种取值.

注

(重构新增) 1-of-K 编码的意义在于把“类别”这一离散对象嵌入 $\mathbb{R}^K$, 从而使线性模型 $y_k(\boldsymbol{x}) = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0}$ 可以逐分量地作用于它. 该编码有两个直接推论. 其一, 各分量之和恒为 1(这正是命题 5.5所讨论的线性约束的特例). 其二, 把 t_k 读作“属于 C_k 的概率”只在真实标签意义上成立:$t_k \in \{0, 1\}$, 而模型输出 $y_k(\boldsymbol{x})$ 是实值的, 未必落在 $[0, 1]$ 内, 故不能直接解释为概率 (见小节 5.1.4). 若改用 $t \in \{0, 1\}$ 的二分类编码, 则两种类别的地位不对称, 编码依赖于任意选定的参照类;1-of-K 编码避免了这种不对称, 代价是参数个数增加为 K 倍. 注意编码向量的分量个数 K 是固定的, 不能把不同 K 的编码混用.

5.1.4 最小二乘分类

在线性回归模型中, 通过最小化平方和误差函数, 可以得到参数值的简单闭式解 (参见小节 4.1.3). 因此, 我们期待将同样的最小二乘形式应用于分类问题. 考虑一个涉及 K 个类的一般分类问题, 以及目标向量 $\boldsymbol{t}$ 的 1-of-K 二进制编码方案. 在此种情况下使用最小二乘法的一个理由是, 它近似于给定输入向量时目标值的条件期望 $\mathbb{E}[\boldsymbol{t} \mid \boldsymbol{x}]$. 对于二进制编码方案, 该条件期望由类的后验概率向量给出 (见练习 5.1). 遗憾的是, 这些概率的近似值通常很低, 而且近似值的取值范围实际上可能超出 $(0, 1)$. 但尽管如此, 研究这些简单的模型并了解这些局限性是如何产生的仍具有指导意义.

每个 C_k 类都由各自的线性模型描述, 所以

$$y_k(\boldsymbol{x}) = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0} \tag{5.14}$$

其中 $k = 1, \cdots, K$. 我们可以方便地使用向量表示法将它们归为一组, 从而

$$\boldsymbol{y}(\boldsymbol{x}) = \tilde{\boldsymbol{W}}^{\mathsf{T}}\tilde{\boldsymbol{x}} \tag{5.15}$$

其中 $\tilde{\boldsymbol{W}}$ 是一个矩阵, 其第 k 列包含 $(D+1)$ 维向量 $\tilde{\boldsymbol{w}}_k = (w_{k0}, \boldsymbol{w}_k)^{\mathsf{T}}$,$\tilde{\boldsymbol{x}}$ 是相应的增广输入向量 $(1, \boldsymbol{x}^{\mathsf{T}})^{\mathsf{T}}$, 虚拟输入 $x_0 = 1$. 然后将新输入 $\boldsymbol{x}$ 归入输出 $y_k = \tilde{\boldsymbol{w}}_k^{\mathsf{T}}\tilde{\boldsymbol{x}}$ 最大的类.

下面我们通过将平方和误差函数最小化来确定参数矩阵 $\tilde{\boldsymbol{W}}$. 考虑一个训练数据集 $\{\boldsymbol{x}_n, \boldsymbol{t}_n\}$, 其中 $n = 1, \cdots, N$, 并定义一个第 n 行为向量 $\boldsymbol{t}_n^{\mathsf{T}}$ 的矩阵 $\boldsymbol{T}$ 和另一个第 n 行为向量 $\tilde{\boldsymbol{x}}_n^{\mathsf{T}}$ 的矩阵 $\tilde{\boldsymbol{X}}$. 因此, 平方和误差函数可以写成

$$E_D(\tilde{\boldsymbol{W}}) = \frac{1}{2}\operatorname{tr}\left\{(\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}} - \boldsymbol{T})^{\mathsf{T}}(\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}} - \boldsymbol{T})\right\} \tag{5.16}$$

将与 $\tilde{\boldsymbol{W}}$ 有关的导数设为 0 并进行重新排列, 从而求得 $\tilde{\boldsymbol{W}}$ 的解, 其形式为

$$\tilde{\boldsymbol{W}} = (\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}})^{-1}\tilde{\boldsymbol{X}}^{\mathsf{T}}\boldsymbol{T} = \tilde{\boldsymbol{X}}^{\dagger}\boldsymbol{T} \tag{5.17}$$

其中 $\tilde{\boldsymbol{X}}^{\dagger}$ 是矩阵 $\tilde{\boldsymbol{X}}$ 的伪逆 (参见小节 4.1.3).

命题 5.4 (最小二乘分类解的极小性)

设 $\tilde{\boldsymbol{X}}$ 为 $N \times (D+1)$ 矩阵,$\boldsymbol{T}$ 为 $N \times K$ 矩阵. 若 $\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}}$ 可逆, 则由(5.17)给出的 $\tilde{\boldsymbol{W}}$ 是(5.16)中 E_D 的唯一全局极小点.

证明

(重构新增) 记 $\boldsymbol{R}=\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}}-\boldsymbol{T}$. 由迹的性质 $\operatorname{tr}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})=\sum_{ij}A_{ij}^2$ 知 $E_D=\frac{1}{2}\|\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}}-\boldsymbol{T}\|_F^2$, 其中 $\|\cdot\|_F$ 为 Frobenius 范数. 对 $\tilde{\boldsymbol{W}}$ 求导 (分母布局), 利用 $\mathrm{d}\operatorname{tr}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})=2\operatorname{tr}(\boldsymbol{A}^{\mathsf{T}}\mathrm{d}\boldsymbol{A})$, 得

$$\frac{\partial E_D}{\partial \tilde{\boldsymbol{W}}}=\tilde{\boldsymbol{X}}^{\mathsf{T}}(\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}}-\boldsymbol{T}) \tag{5.18}$$

令其为零, 得到正规方程 $\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}}=\tilde{\boldsymbol{X}}^{\mathsf{T}}\boldsymbol{T}$. 当 $\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}}$ 可逆时, 该方程有唯一解(5.17). 为说明它是极小点, 把 E_D 在 $\tilde{\boldsymbol{W}}_0=\tilde{\boldsymbol{X}}^{\dagger}\boldsymbol{T}$ 处展开: 由正规方程 $\tilde{\boldsymbol{X}}^{\mathsf{T}}(\tilde{\boldsymbol{X}}\tilde{\boldsymbol{W}}_0-\boldsymbol{T})=\boldsymbol{0}$, 交叉项消失, 于是对任意 $\boldsymbol{\Delta}$,

$$E_D(\tilde{\boldsymbol{W}}_0+\boldsymbol{\Delta})=E_D(\tilde{\boldsymbol{W}}_0)+\frac{1}{2}\operatorname{tr}\left\{(\tilde{\boldsymbol{X}}\boldsymbol{\Delta})^{\mathsf{T}}(\tilde{\boldsymbol{X}}\boldsymbol{\Delta})\right\} \tag{5.19}$$

而 $\operatorname{tr}\{(\tilde{\boldsymbol{X}}\boldsymbol{\Delta})^{\mathsf{T}}(\tilde{\boldsymbol{X}}\boldsymbol{\Delta})\}=\|\tilde{\boldsymbol{X}}\boldsymbol{\Delta}\|_F^2\geqslant 0$. 又 $\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}}$ 可逆蕴含 $\tilde{\boldsymbol{X}}$ 列满秩, 故 $\tilde{\boldsymbol{X}}\boldsymbol{\Delta}=\boldsymbol{0}$ 仅当 $\boldsymbol{\Delta}=\boldsymbol{0}$, 即增量严格为正, 故 $\tilde{\boldsymbol{W}}_0$ 是唯一全局极小点.

□

注

(重构新增) 命题中的可逆性条件等价于 $\tilde{\boldsymbol{X}}$ 列满秩, 即 $N\geqslant D+1$ 且增广输入向量线性无关; 若该条件失败 (例如样本数少于参数个数, 或某些特征线性相关), 则极小点不唯一, 此时(5.17)中的逆应换为 Moore–Penrose 伪逆 $\tilde{\boldsymbol{X}}^{\dagger}$, 它给出所有极小点中范数最小的那一个.(5.17)的第二个等号 $\tilde{\boldsymbol{X}}^{\dagger}=(\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}})^{-1}\tilde{\boldsymbol{X}}^{\mathsf{T}}$ 只在列满秩时成立. 还要注意, 最小二乘的目标函数是 $\boldsymbol{y}$ 与 $\boldsymbol{t}$ 的平方误差, 它并不是分类问题的似然: 只有当 $\boldsymbol{t}$ 的条件分布为 Gauss 分布时, 最小化 E_D 才等价于最大似然 (参见小节 4.1.3). 这正是本节末将要讨论的局限性的来源.

将(5.17)代回(5.15), 即得如下形式的判别函数:

$$\boldsymbol{y}(\boldsymbol{x})=\tilde{\boldsymbol{W}}^{\mathsf{T}}\tilde{\boldsymbol{x}}=\boldsymbol{T}^{\mathsf{T}}(\tilde{\boldsymbol{X}}^{\dagger})^{\mathsf{T}}\tilde{\boldsymbol{x}} \tag{5.20}$$

具有多元目标变量的最小二乘解拥有一个有趣的特性, 即如果训练集中的每个目标向量都满足某些线性约束条件, 如

$$\boldsymbol{a}^{\mathsf{T}}\boldsymbol{t}_n+b=0 \tag{5.21}$$

则对于某些常数 $\boldsymbol{a}$ 和 b,$\boldsymbol{x}$ 的任意值的模型预测都将满足相同的约束条件, 从而 (参见练习 5.3)

$$\boldsymbol{a}^{\mathsf{T}}\boldsymbol{y}(\boldsymbol{x})+b=0 \tag{5.22}$$

命题 5.5 (最小二乘预测保持目标向量的线性约束)

设 $\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}}$ 可逆,$\tilde{\boldsymbol{W}}$ 由(5.17)给出, 且训练集的每个目标向量都满足(5.21). 则对任意 $\boldsymbol{x}$,(5.20)给出的 $\boldsymbol{y}(\boldsymbol{x})$ 满足(5.22).

♠

证明

(重构新增) 记 $\mathbf{1}=(1,\cdots,1)^{\mathsf{T}}\in\mathbb{R}^N$. 条件(5.21)对 $n=1,\cdots,N$ 成立, 等价于矩阵等式 $\boldsymbol{T}\boldsymbol{a}+b\mathbf{1}=\mathbf{0}$. 于是 $\boldsymbol{T}\boldsymbol{a}=-b\mathbf{1}$; 转置得 $\boldsymbol{a}^{\mathsf{T}}\boldsymbol{T}^{\mathsf{T}}=-b\mathbf{1}^{\mathsf{T}}$. 由(5.20),

$$\boldsymbol{a}^{\mathsf{T}}\boldsymbol{y}(\boldsymbol{x})=\boldsymbol{a}^{\mathsf{T}}\boldsymbol{T}^{\mathsf{T}}(\tilde{\boldsymbol{X}}^{\dagger})^{\mathsf{T}}\tilde{\boldsymbol{x}}=-b\mathbf{1}^{\mathsf{T}}(\tilde{\boldsymbol{X}}^{\dagger})^{\mathsf{T}}\tilde{\boldsymbol{x}} \tag{5.23}$$

另一方面, 由 $\tilde{\boldsymbol{X}}^{\dagger}=(\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}})^{-1}\tilde{\boldsymbol{X}}^{\mathsf{T}}$ 以及 $\tilde{\boldsymbol{X}}$ 的第一列 (由虚拟输入 $x_0=1$ 组成) 为 $\mathbf{1}$, 有 $\tilde{\boldsymbol{X}}^{\mathsf{T}}\mathbf{1}=\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}}\boldsymbol{e}_1$, 其中 $\boldsymbol{e}_1=(1,0,\cdots,0)^{\mathsf{T}}$; 两边左乘 $(\tilde{\boldsymbol{X}}^{\mathsf{T}}\tilde{\boldsymbol{X}})^{-1}$ 得 $\tilde{\boldsymbol{X}}^{\dagger}\mathbf{1}=\boldsymbol{e}_1$. 故 $\mathbf{1}^{\mathsf{T}}(\tilde{\boldsymbol{X}}^{\dagger})^{\mathsf{T}}\tilde{\boldsymbol{x}}=(\tilde{\boldsymbol{X}}^{\dagger}\mathbf{1})^{\mathsf{T}}\tilde{\boldsymbol{x}}=\tilde{\boldsymbol{x}}_1=1$. 代入上式得 $\boldsymbol{a}^{\mathsf{T}}\boldsymbol{y}(\boldsymbol{x})=-b$, 即(5.22).

□

注

(重构新增) 证明中用到了“某个基函数恒为 1”这一条件, 它保证了 $\tilde{\boldsymbol{X}}$ 的第一列为 $\mathbf{1}$, 从而使常数项 b 可以被表示出来; 这正是原书在练习 5.3中要求“假设其中一个基函数 $\phi_0(\boldsymbol{x})=1$”的原因. 若没有这一列, 则 $\tilde{\boldsymbol{X}}^{\dagger}\mathbf{1}$ 不再是 $\boldsymbol{e}_1$, 结论一般不成立. 该命题的一个重要特例是 1-of-K 编码: 此时每个 $\boldsymbol{t}_n$ 都满足 $\mathbf{1}^{\mathsf{T}}\boldsymbol{t}_n=1$, 取 $\boldsymbol{a}=\mathbf{1}$、$b=-1$ 即得 $\sum_k y_k(\boldsymbol{x})=1$. 但如正文所述, 仅有求和为 1 并不能保证 $y_k(\boldsymbol{x})\in(0,1)$, 因此这些输出仍不能解释为概率.

因此, 如果我们针对多分类问题使用 1-of-K 编码方案, 那么模型所做的预测将具有这样的特性: 对于任意 $\boldsymbol{x}$ 值,$\boldsymbol{y}(\boldsymbol{x})$ 的元素之和都为 1. 然而, 仅凭该求和约束条件还不足以将模型输出理解为概率, 因为它们并没有被限制在区间 $(0,1)$ 内.

最小二乘法给出了判别函数参数的精确闭式解. 然而, 即使作为一个判别函数 (我们直接用它来做决定, 且不需要做任何概率解释), 它也存在一些严重的问题. 你已经看到, 在 Gauss 噪声分布的假设下, 平方和误差函数可以看作负对数似然 (参见小节 2.3.4). 如果数据的真实分布明显不同于 Gauss 分布, 则最小二乘法的结果就会很差. 特别是, 最小二乘法对离群值 (与大部分数据点相距甚远的数据点) 的存在非常敏感, 如图 5.4所示. 我们可以看到,图 5.4右图中的附加数据点使决策边界的位置发生了显著变化, 尽管这些数据点会被左图中的原始决策边界正确分类. 平方和误差函数对距离决策边界较远的数据点赋予了过高的权重, 即使这些数据点是被正确分类的. 离群值可能由于罕见事件, 或仅仅由于数据集中的错误而产生. 由此, 对极少数数据点敏感的技术是缺乏鲁棒性的. 为便于比较,图 5.4还显示了使用一种称为 logistic 回归 (logistic regression) 的技术 (参见小节 5.4.3) 得出的结果, 该技术对离群值的鲁棒性更强.

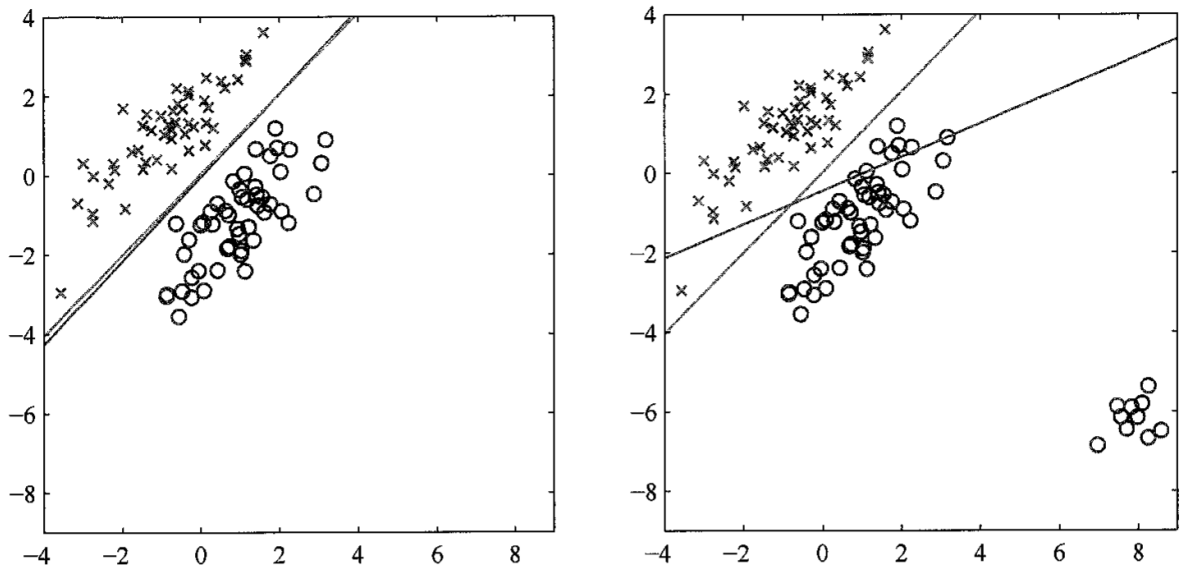


图 5.4: 左图显示了来自两个类的数据 (分别用红色十字和蓝色圆圈表示), 以及通过使用最小二乘法 (洋红色曲线) 和 logistic 回归模型 (绿色曲线) 得到的决策边界. 右图展示了在图右下方添加额外数据点后得到的相应结果. 与 logistic 回归不同的是, 最小二乘法对离群值高度敏感

回想一下, 最小二乘法对应的是 Gauss 条件分布假设下的最大似然, 而二进制目标向量的分布显然远非 Gauss 分布, 于是最小二乘法的失败也就不足为奇了. 通过采用更合适的概率模型, 我们可以获得相比最小二乘法性能更好的分类技术, 而且还可以将其扩展以得到灵活的非线性神经网络模型, 详见后续章节.

5.2 决策理论

在讨论线性回归时, 机器学习中的预测过程可分为推理和决策 (参见节 4.2) 两个阶段. 下面我们特别结合分类器对这一观点进行更深入的探讨.

假设有一个输入向量 $\boldsymbol{x}$ 和一个相应的目标变量 $\boldsymbol{t}$, 我们想要在给定 $\boldsymbol{x}$ 的新值时预测 $\boldsymbol{t}$. 对于回归问题,$\boldsymbol{t}$ 将由连续变量组成且通常是一个向量, 因为我们可能希望预测几个相关的量. 对于分类问题,$\boldsymbol{t}$ 将代表类标签. 同样, 如果有两个以上的类,$\boldsymbol{t}$ 通常是一个向量. 联合概率分布 $p(\boldsymbol{x}, \boldsymbol{t})$ 完整总结了与这些变量相关的不确定性. 作为推理的一个例子, 从一组训练数据中确定 $p(\boldsymbol{x}, \boldsymbol{t})$ 通常是一个解决起来非常困难的问题, 本书大部分篇幅讨论的就是如何解决这个问题. 然而在实际应用中, 我们往往必须对 $\boldsymbol{t}$ 的值做出具体预测, 或者更一般地说, 我们必须根据对 $\boldsymbol{t}$ 可能取值的理解而采取具体行动. 这正是决策理论的主题.

例如, 在之前的医学诊断问题中, 我们拍摄了病人皮肤病变的图像, 并希望确定病人是否患有癌症. 在这种情况下, 输入向量 $\boldsymbol{x}$ 是图像中像素强度的集合, 而输出变量 t 要么代表未患癌症, 用 C_1 类表示, 要么代表患有癌症, 用 C_2 类表示. 例如, 我们可以选择 t 作为二进制变量, 使得 $t = 0$ 对应 C_1 类,$t = 1$ 对应 C_2 类. 稍后我们将看到, 在计算概率时, 这种标签值的选择特别方便. 一般情况下, 推理问题涉及确定联合分布 $p(\boldsymbol{x}, C_k)$, 它等价于 $p(\boldsymbol{x}, t)$, 其对变量进行了完整的概率描述. 虽然这是一个非常有用的信息, 但归根结底, 我们必须决定是否对患者进行治疗, 而且我们希望能够根据某种适当的准则, 做出最佳选择 (Duda and Hart, 1973). 此为决策步骤, 而决策理论旨在告诉我们如何在适当的概率条件下做出最优决策. 一旦解决了推理问题, 你就会发现决策阶段通常非常简单. 在此, 我们介绍了学习本书其余部分所需了解的决策理论的主要观点. 要想了解更多的背景知识和更详细的信息, 请参阅 Berger(1985) 和 Bather(2000).

在进行更详细的分析之前, 有必要非正式地分析一下我们期望概率在制定决策时如何发挥作用. 当获得新患者的皮肤病变图像 $\boldsymbol{x}$ 时, 需要决定将图像归入两类中的哪一类. 因此我们感兴趣的是, 在给定图像的情况下,$p(C_k \mid \boldsymbol{x})$ 给出这两类的概率. 利用 Bayes 定理, 这些概率可用如下形式表示:

$$p(C_k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_k)\, p(C_k)}{p(\boldsymbol{x})} \tag{5.24}$$

请注意,Bayes 定理中出现的任何量都可通过将适当变量边缘化或条件化而从联合分布 $p(\boldsymbol{x}, C_k)$ 中得到. 现在, 我们可以将 $p(C_k)$ 解释为 C_k 类的先验概率, 并将 $p(C_k \mid \boldsymbol{x})$ 解释为相应的后验概率. 因此,$p(C_2)$ 代表在拍摄皮肤病变图像之前某人患癌的概率. 同样,$p(C_1 \mid \boldsymbol{x})$ 是后验概率, 我们对其根据图像所含的信息已使用 Bayes 定理进行了修正. 如果想要尽量降低将 $\boldsymbol{x}$ 归入错误类的概率, 则可以根据直觉选择后验概率较高的类. 接下来我们将证明这种直觉是正确的并讨论更普遍的决策制定准则.

5.2.1 误分类率

假设我们的目标只是尽可能减少误分类, 则需要一条规则, 以便将 $\boldsymbol{x}$ 的每个值归入一个可用的类. 该规则会将输入空间划分为若干决策区域 $\mathcal{R}_k$, 每个类拥有一个决策区域, 从而 $\mathcal{R}_k$ 中的所有点都会被归入 C_k 类. 决策区域之间的边界称为决策边界或决策面. 需要注意的是, 每个决策区域不需要是连续的, 但可包含若干不相交的区域.

定义 5.7 (误分类率)

设决策区域为 $\mathcal{R}_k, k = 1, \cdots, K$, 即约定 $\boldsymbol{x} \in \mathcal{R}_k$ 时把 $\boldsymbol{x}$ 归入 C_k 类. 称

$$p(\text{错误}) = p(\boldsymbol{x} \in \mathcal{R}_1, C_2) + p(\boldsymbol{x} \in \mathcal{R}_2, C_1) \tag{5.25}$$

为二分类情形下的误分类率 (misclassification rate), 即属于 C_1 类的输入向量被归入 C_2 类、或属于 C_2 类的输入向量被归入 C_1 类的概率. ♣

注

(重构新增) 定义中的两个事件 $\{\boldsymbol{x} \in \mathcal{R}_1, C_2\}$ 与 $\{\boldsymbol{x} \in \mathcal{R}_2, C_1\}$ 互不相交, 因此概率可以直接相加; 这正是(5.25)写成两项之和的原因. 误分类率只区分“对”与“错”, 不区分错误的方向与后果, 因此它只适用于两类错误代价相同的情形 (对比小节 5.2.2中的损失矩阵). 此外,$\mathcal{R}_k$ 允许不连通不会影响定义:$\{\boldsymbol{x} \in \mathcal{R}_k\}$ 始终是一个可测集, 概率仍按 $p(\boldsymbol{x}, C_k)$ 在该集合上的积分计算 (见命题 5.6的证明).

为得到最优决策规则, 首先考虑二分类, 例如癌症检查. 把(5.25)写成积分形式, 即

$$\begin{aligned} p(\text{错误}) &= p(\boldsymbol{x} \in \mathcal{R}_1, C_2) + p(\boldsymbol{x} \in \mathcal{R}_2, C_1) \\ &= \int_{\mathcal{R}_1} p(\boldsymbol{x}, C_2)\,\mathrm{d}\boldsymbol{x} + \int_{\mathcal{R}_2} p(\boldsymbol{x}, C_1)\,\mathrm{d}\boldsymbol{x} \end{aligned} \tag{5.26}$$

我们可以自由选择将每个点 $\boldsymbol{x}$ 归入两个类之一的决策规则. 显然, 为了将 p(错误) 最小化, 我们应该将每个点 $\boldsymbol{x}$ 归入(5.26)中被积函数值较小的类. 因此, 对于给定的 $\boldsymbol{x}$ 值, 如果 $p(\boldsymbol{x}, C_1) > p(\boldsymbol{x}, C_2)$, 则归入 C_1 类. 根据概率的乘积法则, 可以得到 $p(\boldsymbol{x}, C_k) = p(C_k \mid \boldsymbol{x})p(\boldsymbol{x})$. 由于 $p(\boldsymbol{x})$ 是两个项的公因子, 因此我们可以将这一结果重新表述为: 如果将 $\boldsymbol{x}$ 的每个值归入后验概率 $p(C_k \mid \boldsymbol{x})$ 最大的类, 则犯错的概率最小.

命题 5.6 (最小误分类率规则)

设 $p(\boldsymbol{x}) > 0$. 则使(5.26)中 p(错误) 最小的决策规则为: 对每个 $\boldsymbol{x}$, 把它归入使 $p(C_k \mid \boldsymbol{x})$ 最大的类. 任何其他规则所对应的误分类率都不小于该规则, 且当两个后验概率对某个正概率集合上的 $\boldsymbol{x}$ 不相等时, 其他规则的误分类率严格更大. ♠

证明

(重构新增) 把(5.26)的两个积分合并写为在整个输入空间上的积分. 对每个 $\boldsymbol{x}$, 定义“错误指示” $e(\boldsymbol{x})$: 若把 $\boldsymbol{x}$ 归入 C_1 类, 则 $e(\boldsymbol{x}) = p(\boldsymbol{x}, C_2)$; 若归入 C_2 类, 则 $e(\boldsymbol{x}) = p(\boldsymbol{x}, C_1)$. 于是

$$p(\text{错误}) = \int e(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} \tag{5.27}$$

积分是逐点相加的, 故要使积分最小, 只需使被积函数对每个 $\boldsymbol{x}$ 都最小. 对给定的 $\boldsymbol{x}$, 把 $\boldsymbol{x}$ 归入 C_1 类的代价是 $p(\boldsymbol{x}, C_2)$, 归入 C_2 类的代价是 $p(\boldsymbol{x}, C_1)$, 因此最优规则是选择二者中较小者所对应的类, 即当 $p(\boldsymbol{x}, C_1) > p(\boldsymbol{x}, C_2)$ 时归入 C_1 类, 否则归入 C_2 类. 由于 $p(\boldsymbol{x}, C_k) = p(C_k \mid \boldsymbol{x})p(\boldsymbol{x})$ 且 $p(\boldsymbol{x}) > 0$, 比较 $p(\boldsymbol{x}, C_1)$ 与 $p(\boldsymbol{x}, C_2)$ 等价于比较 $p(C_1 \mid \boldsymbol{x})$ 与 $p(C_2 \mid \boldsymbol{x})$. 逐点最优性给出全局最优性, 且当最优规则不唯一时 (即两个后验概率相等的 $\boldsymbol{x}$ 构成的集合具有正概率), 任意别的规则都在该集合上取到更大的被积函数, 故误分类率严格更大.

□

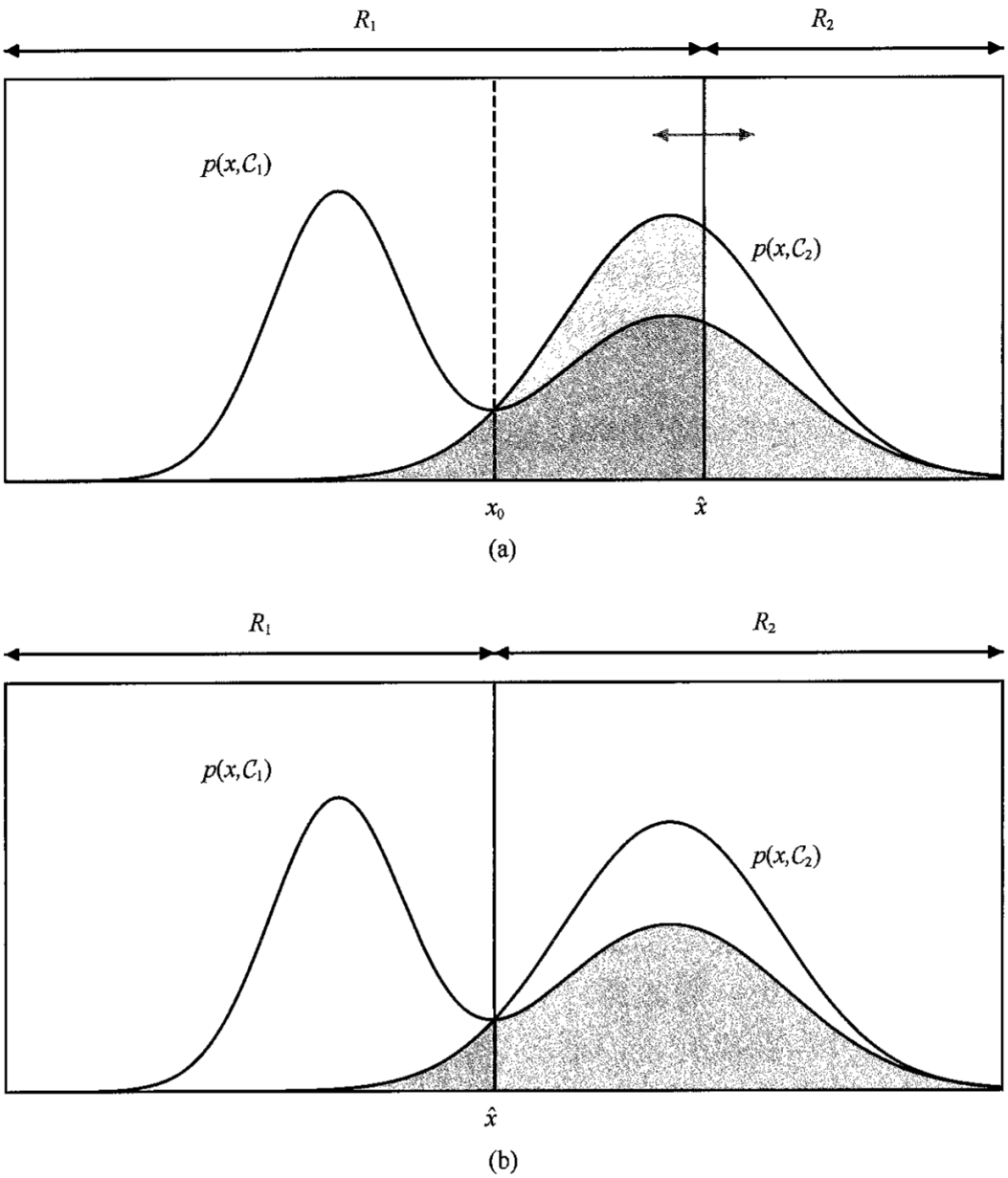


图 5.5: 二分类中每个类的联合概率 $p(\boldsymbol{x}, C_1)$ 与 $p(\boldsymbol{x}, C_2)$ 与 x 的关系以及决策边界 $x = \hat{x}$ 的示意图.$x \geqslant \hat{x}$ 的点被归入 C_2 类, 因此属于决策区域 $\mathcal{R}_2$; 而 $x < \hat{x}$ 的点被归入 C_1 类, 因此属于决策区域 $\mathcal{R}_1$. 误差来自蓝色、绿色和红色区域, 因此当 $x < \hat{x}$ 时, 误差是由于属于 C_2 类的点被错误分类为 C_1 类 (由红色和绿色区域之和表示) 而产生的; 而当 $x \geqslant \hat{x}$ 时, 误差是由于属于 C_1 类的点被错误分类为 C_2 类 (由蓝色区域表示) 而产生的. 通过改变决策边界的位置 $\hat{x}$, 如图 (a) 中红色双箭头所示, 可使蓝色和绿色区域的总面积保持不变, 而红色区域的面积则有所变化. 如图 (b) 所示, 对应于 $\hat{x} = x_0$,$\hat{x}$ 的最佳选择是 $p(\boldsymbol{x}, C_1)$ 和 $p(\boldsymbol{x}, C_2)$ 曲线的相交处, 因为红色区域在这种情况下消失了. 这相当于最小化误分类率决策规则,x 的每个值被归入后验概率 $p(C_k \mid x)$ 较高的类

注

(重构新增) 命题的证明揭示了这类决策问题的共同结构: 总风险是一个逐点代价的积分, 因此“全局最优”可以化归为“逐点最优”. 这一化归要求代价函数不依赖于其他 $\boldsymbol{x}$ 的决策, 即各点的决策相互独立; 在小节 5.2.2与小节 5.2.3中我们会反复使用它. 条件 $p(\boldsymbol{x}) > 0$ 只是为了能约去公因子; 若 $p(\boldsymbol{x}) = 0$, 该点出现的概率为零, 其决策不影响误分类率. 结论“选择后验概率最大的类”也解释了一个常见说法:Bayes 分类器就是最大后验分类器. 注意该结论只对 0-1 损失成立; 一旦两类错误的代价不同, 最优规则就要由命题 5.8给出.

图 5.5展示了二分类和单一输入变量 x 的结果.

在更一般的多分类情况下, 将正确概率最大化稍显容易, 其计算公式如下:

$$p(\text{正确}) = \sum_{k=1}^{K} p(\boldsymbol{x} \in \mathcal{R}_k, C_k) = \sum_{k=1}^{K} \int_{\mathcal{R}_k} p(\boldsymbol{x}, C_k)\,\mathrm{d}\boldsymbol{x} \tag{5.28}$$

当决策区域 $\mathcal{R}_k$ 使得每个 $\boldsymbol{x}$ 被归入 $p(\boldsymbol{x}, C_k)$ 最大的类时, 正确概率就会最高. 同样, 利用概率的乘积法则 $p(\boldsymbol{x}, C_k) = p(C_k \mid \boldsymbol{x})p(\boldsymbol{x})$,$p(\boldsymbol{x})$ 是所有项的公因子, 每个 $\boldsymbol{x}$ 都应被归入后验概率 $p(C_k \mid \boldsymbol{x})$ 最大的类.

命题 5.7 (最大正确率规则)

设 $p(\boldsymbol{x}) > 0$ 且 $\sum_k p(C_k \mid \boldsymbol{x}) = 1$. 则使(5.28)中 $p(\text{正确})$ 最大的决策规则为: 对每个 $\boldsymbol{x}$, 把它归入使 $p(C_k \mid \boldsymbol{x})$ 最大的类.

证明

(重构新增) 由于对每个 $\boldsymbol{x}$ 都有 $\sum_k p(C_k \mid \boldsymbol{x}) = 1$, 二分类情形下 $p(\text{正确}) = 1 - p(\text{错误})$, 故最大化正确率与最小化误分类率等价, 结论由命题 5.6得到. 对一般的 K, 把(5.28)写成 $\int \sum_k p(\boldsymbol{x}, C_k)\mathbb{I}\{\boldsymbol{x} \in \mathcal{R}_k\}\,\mathrm{d}\boldsymbol{x}$, 其中 $\mathbb{I}\{\cdot\}$ 为示性函数. 被积函数对给定的 $\boldsymbol{x}$ 等于 $p(\boldsymbol{x}, C_{k(\boldsymbol{x})})$, 其中 $C_{k(\boldsymbol{x})}$ 是 $\boldsymbol{x}$ 被归入的类; 它显然不超过 $\max_k p(\boldsymbol{x}, C_k)$, 而取 $k(\boldsymbol{x}) = \arg\max_k p(\boldsymbol{x}, C_k)$ 即可达到该上界. 故逐点取最大后验的规则使正确率最大, 且由 $p(\boldsymbol{x}) > 0$, 比较 $p(\boldsymbol{x}, C_k)$ 与比较 $p(C_k \mid \boldsymbol{x})$ 等价.

□

注

(重构新增) 两个命题互为补充:命题 5.6从“最小化错误”出发,命题 5.7从“最大化正确”出发, 而它们给出同一条规则, 原因是 $\sum_k p(C_k \mid \boldsymbol{x}) = 1$ 使正确与错误互补. 证明中对示性函数逐点取上界的论证是这类问题的标准技巧, 它与命题 5.6中“被积函数逐点最小”的论证是同一件事的两种说法. 需要强调, 逐点最优性依赖于 $\mathcal{R}_k$ 可以任意取 (即决策规则没有任何限制); 若对决策规则加约束 (例如要求决策面为线性), 则逐点最优一般无法实现, 此时“最优”只能在该约束类中取, 这正是线性判别式与 Bayes 分类器的差别.

5.2.2 预期损失

在许多应用中, 我们的目标相比仅减少误分类数量更为复杂. 让我们再次回到医疗诊断问题. 我们注意到, 如果一名未患癌症的病人被误诊为患有癌症, 后果可能是他会经历一些痛苦, 而且还需要接受进一步的检查. 相反, 如果癌症患者被误诊为身体健康, 后果可能是他由于没有接受相应治疗而过早死亡. 因此, 这两种错误可能导致截然不同的后果. 显然, 即使冒着犯下更多第一种错误的风险, 也不要犯第二种错误.

我们可以通过采用损失函数来将这些问题形式化, 损失函数是对采取任何可供选择之决策或行动所造成损失的单一、总体衡量. 因此, 我们的目标是最大限度地减少总损失. 请注意, 有些学者考虑使用效用函数 (utility function), 并希望最大化效用函数的值. 如果我们把效用简单地理解为损失的负数, 那么这两个概念是等价的.

定义 5.8 (损失矩阵)

设共有 K 个类. 假设一个新的 $\boldsymbol{x}$ 值实际上属于 C_k 类, 而我们却将 $\boldsymbol{x}$ 归入 C_j 类 (其中 j 可能等于也可能不等于 k), 在此过程中遭受的损失记为 L_{kj}, 并称之为损失矩阵 (loss matrix) 的 (k, j) 元素. 所有 L_{kj} 组成的 $K \times K$ 矩阵 $\boldsymbol{L}$ 称为损失矩阵, 其行对应真实类, 列对应决策所归入的类.

例如, 在癌症检查示例中, 我们可能得到图 5.6所示形式的一个损失矩阵. 这个特定的损失矩阵表示, 如果决策正确, 就不会有任何损失; 如果一个身体健康的人被误诊患有癌症, 则会遭受 1 的损失; 而如果一个癌症患者被误诊为身体健康, 则会遭受 100 的损失.

	normal	cancer
normal	0	1
cancer	100	0

图 5.6: 癌症检查示例中元素为 L_{kj} 的损失矩阵示例. 行代表真实类, 而列代表根据决策准则所做的分类

注

(重构新增) 损失矩阵把“错误的类型”与“错误的代价”分开: 误分类率只记录 $j \neq k$ 的事件, 而损失矩阵为每个 (k, j) 指定权重, 因此它可以表达“漏诊比误诊严重 100 倍”这类非对称代价. 矩阵不要求对称 (本例中 $L_{12} = 1 \neq L_{21} = 100$), 对角元通常取 0, 但也可以取非零值 (例如检查本身有成本). 损失与效用只差一个符号: 若把效用记为 $U_{kj} = -L_{kj}$, 则最小化预期损失等价于最大化预期效用. 注意 L_{kj} 是决策者给定的, 而不是从数据中学到的;命题 5.8的结论会随 $\boldsymbol{L}$ 的改变而改变, 这一点在小节 5.2.4中作为“需要后验概率”的第一条理由出现.

将损失函数最小化的解为最优解. 然而, 损失函数取决于真实类, 而真实类是未知的. 对于给定的输入向量 $\boldsymbol{x}$, 我们使用联合概率分布 $p(\boldsymbol{x}, C_k)$ 来表示真实类的不确定性. 我们可以转而寻求将预期损失最小化.

定义 5.9 (预期损失)

设决策区域为 $\mathcal{R}_j$, 即把 $\mathcal{R}_j$ 中的点归入 C_j 类. 称

$$\mathbb{E}[L] = \sum_k \sum_j \int_{\mathcal{R}_j} L_{kj}\, p(\boldsymbol{x}, C_k)\, \mathrm{d}\boldsymbol{x} \tag{5.29}$$

为决策规则 $\{\mathcal{R}_j\}$ 的预期损失 (expected loss).

注

(重构新增) (5.29)是对两个离散变量 (C_k, 决策 j) 与连续变量 $\boldsymbol{x}$ 取期望的结果: 先固定 k 与 j, 在 $\mathcal{R}_j$ 上对 $p(\boldsymbol{x}, C_k)$ 积分得到“真实类为 C_k 而被归入 C_j”的概率, 再乘以损失 L_{kj} 并对 (k, j) 求和. 它与定义 5.7的关系是: 取 $L_{kj} = 1 - I_{kj}$ (I 为单位矩阵) 时, $\mathbb{E}[L]$ 恰好等于误分类率 (见练习 5.7). 因此预期损失是误分类率的加权推广, 而定义 5.7只是它的一个特例. 定义不要求 $\mathcal{R}_j$ 连通, 也不要求 $L_{kj} \geqslant 0$; 若某些 $L_{kj} < 0$, 则“损失”实际上是收益, 公式仍然成立.

每个 $\boldsymbol{x}$ 可被独立归入决策区域 $\mathcal{R}_j$ 中的一个类. 我们想要通过选择决策区域 $\mathcal{R}_j$ 来将预期损失 [即(5.29)] 最小化, 这意味着对于每个 $\boldsymbol{x}$, 我们都应该将 $\sum_k L_{kj} p(\boldsymbol{x}, C_k)$ 最小化. 和前面一样, 我们可以使用概率的乘积法则 $p(\boldsymbol{x}, C_k) = p(C_k \mid \boldsymbol{x}) p(\boldsymbol{x})$ 来消除公因子 $p(\boldsymbol{x})$. 因此, 能够最小化预期损失的决策规则会将每个新的 $\boldsymbol{x}$ 分配给能使下式最小的类别 j:

$$\sum_k L_{kj}\, p(C_k \mid \boldsymbol{x}) \tag{5.30}$$

命题 5.8 (最小预期损失决策规则)

设 $p(\boldsymbol{x}) > 0$. 则使(5.29)中 $\mathbb{E}[L]$ 最小的决策规则为: 对每个 $\boldsymbol{x}$, 把它归入使(5.30)取最小值的类 j.

证明

(重构新增) 对给定的 $\boldsymbol{x}$, 若把它归入 C_j, 则它对 $\mathbb{E}[L]$ 的贡献 (在被积函数意义下) 为 $\sum_k L_{kj} p(\boldsymbol{x}, C_k) = p(\boldsymbol{x}) \sum_k L_{kj} p(C_k \mid \boldsymbol{x})$. 由于 $p(\boldsymbol{x}) > 0$ 且各点的决策互不影响, 最小化 $\mathbb{E}[L]$ 等价于对每个 $\boldsymbol{x}$ 独立地最小化 $\sum_k L_{kj} p(C_k \mid \boldsymbol{x})$. 故最优规则为逐点取 $\arg\min_j \sum_k L_{kj} p(C_k \mid \boldsymbol{x})$.

□

注

(重构新增) 命题 5.8把“最小化预期损失”化归为对每个 $\boldsymbol{x}$ 计算 K 个数并取最小, 其计算量为 $O(K^2)$(每类一次内积). 当 $L_{kj} = 1 - I_{kj}$ 时, $\sum_k L_{kj} p(C_k \mid \boldsymbol{x}) = 1 - p(C_j \mid \boldsymbol{x})$, 最小化它等价于最大化后验概率, 于是命题 5.6成为本命题的特例 (见练习 5.7). 若某个 L_{kj} 很大, 最优规则会倾向于避开把真实类 C_k 判为 C_j 的错误, 即使这样做会增加总误分类数——这正是图 5.6所刻画的情形. 注意 $p(\boldsymbol{x})$ 在比较中约去, 因此本命题只要求后验概率, 不要求输入密度, 这也说明判别式模型足以支持最小风险决策.

5.2.3 拒绝选项

我们已经看到, 分类误差产生于输入空间的区域, 其中最大后验概率 $p(C_k \mid \boldsymbol{x})$ 明显小于 1, 或等价地, 联合分布 $p(\boldsymbol{x}, C_k)$ 存在相近的概率值. 在这些区域, 我们对归属关系相对不确定. 在某些应用中, 应避免对疑难案例做出决策, 以期在做出分类决策的案例中获得较低的误差率. 这就是所谓的拒绝选项 (rejection option). 例如, 在癌症检查示例中, 我们使用自动系统对那些几乎不需要怀疑是否被正确分类的图像进行分类, 而要求通过活检来对较为可疑的病例进行分类, 这可能是较为合适的做法.

定义 5.10 (拒绝选项与拒绝阈值)

给定阈值 θ, 拒绝那些后验概率 $p(C_k \mid \boldsymbol{x})$ 的最大值小于或等于 θ 的输入变量 $\boldsymbol{x}$, 即当 $\max_k p(C_k \mid \boldsymbol{x}) \leqslant \theta$ 时不对 $\boldsymbol{x}$ 做出分类决策. 这样的决策方式称为拒绝选项 (rejection option),θ 称为拒绝阈值.

♣

为此, 我们可以引入这个阈值 θ.图 5.7以二分类和一个连续输入变量 x 为例进行了说明. 请注意, 通过设置 $\theta = 1$, 可以确保拒绝所有示例; 而如果有 K 个类, 那么设置 $\theta < 1/K$ 可以确保不拒绝任何示例. 因此, 被拒绝示例的比例受阈值 θ 的控制.

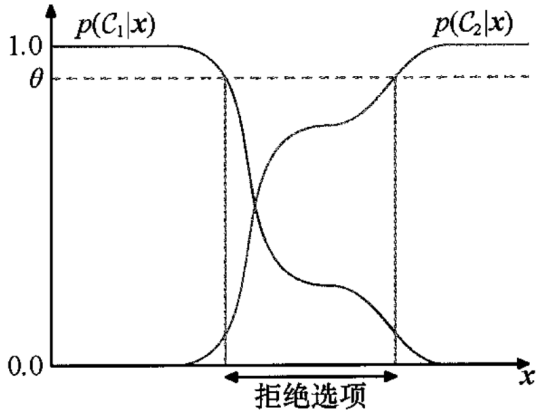


图 5.7: 拒绝选项的示意图. 如果输入变量 x 的两个后验概率中的较大值小于或等于某个阈值 θ, 则 x 会被拒绝

注

(重构新增) 阈值的两个极端值可以由后验概率的基本性质验证:$\max_k p(C_k \mid \boldsymbol{x}) \leqslant 1$ 恒成立, 故 $\theta = 1$ 时所有 $\boldsymbol{x}$ 都被拒绝; 又 $\sum_k p(C_k \mid \boldsymbol{x}) = 1$ 蕴含 $\max_k p(C_k \mid \boldsymbol{x}) \geqslant 1/K$, 故当 $\theta < 1/K$ 时没有任何 $\boldsymbol{x}$ 被拒绝.θ 越大, 被拒绝的样本越多, 被接受样本的误分类率越低, 但拒绝率越高; 这一权衡正是小节 5.2.6中 ROC 曲线所刻画的权衡的另一表现形式 (拒绝相当于把一部分样本从统计中剔除).

在给定损失矩阵的情况下, 通过考虑做出拒绝决定时产生的损失, 我们可以很容易地扩展拒绝准则以使预期损失最小化 (见练习 5.10).

5.2.4 推理和决策

解决分类问题的过程分为两个独立的阶段: 在推理阶段, 我们使用训练数据学习 $p(C_k \mid \boldsymbol{x})$ 模型; 在随后的决策阶段, 我们使用这些后验概率来进行最优分类. 另一种可能性是同时解决这两个问题, 只需要学习一个能将输入变量 $\boldsymbol{x}$ 直接映射为决策的函数即可. 这个函数就是所谓的判别函数.

事实上, 我们可以找出三种不同的方法来解决决策问题, 且每种方法都已得到实际应用, 按复杂程度递减的顺序排列如下.

(a) 首先解决确定每个 C_k 类的类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 的推理问题. 分别推断类的先验概率 $p(C_k)$. 然后使用 Bayes 定理, 其形式为

$$p(C_k \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid C_k)\, p(C_k)}{p(\boldsymbol{x})} \tag{5.31}$$

求出类的后验概率 $p(C_k \mid \boldsymbol{x})$. 按照惯例, 使用如下公式,(5.31)中的分母可以根据分子中项的数量求出:

$$p(\boldsymbol{x}) = \sum_k p(\boldsymbol{x} \mid C_k)\, p(C_k) \tag{5.32}$$

同样, 我们可以直接建立联合分布 $p(\boldsymbol{x}, C_k)$, 然后进行归一化处理以得出后验概率. 在得出后验概率后, 利用决策理论确定每个新输入的 $\boldsymbol{x}$ 归入哪个类. 对输入和输出的分布进行显式或隐式建模的方法叫作生成式模型, 因为通过从它们中采样, 我们可以在输入空间中生成合成数据点.

(b) 首先解决确定类的后验概率 $p(C_k \mid \boldsymbol{x})$ 的推理问题, 然后使用决策理论将每个新输入的 $\boldsymbol{x}$ 归入其中一个类. 直接对后验概率进行建模的方法称为判别模型.

(c) 计算出一个称为判别函数的函数 $f(\boldsymbol{x})$, 它能将每个新输入的 $\boldsymbol{x}$ 直接映射到类标签上. 例如, 对于二分类问题,$f(\cdot)$ 可能是二进制值, 从而 $f = 0$ 代表 C_1 类, 而 $f = 1$ 代表 C_2 类. 在这种情况下, 概率不起任何作用.

定义 5.11 (生成式模型与判别式模型)

(i) 对输入和输出的分布 (即 $p(\boldsymbol{x} \mid C_k)$ 与 $p(C_k)$, 或联合分布 $p(\boldsymbol{x}, C_k)$) 进行显式或隐式建模, 再经 Bayes 定理得到后验概率的方法, 称为生成式模型 (generative model).

(ii) 直接对后验概率 $p(C_k \mid \boldsymbol{x})$ 建模的方法, 称为判别式模型 (discriminative model).

♣

注

(重构新增) 这两种模型与定义 5.3中的两个定义相对应, 区别只在措辞: 前者强调“是否对输入分布建模”, 后者强调“对哪个条件分布建模”. 方法 (a) 的生成式模型需要 $p(\boldsymbol{x} \mid C_k)$, 因而隐含地对 $\boldsymbol{x}$ 的分布作了假设; 方法 (b) 只需 $p(C_k \mid \boldsymbol{x})$. 方法 (c) 连后验概率也不要, 它是定义 5.2的直接实现. 三者所需的数据量与计算量依次递减, 而可提供的信息也依次递减; 下面的四条理由具体说明了丢失后验概率会带来哪些限制.

让我们来看看这三种方法的相对优势. 方法 (a) 要求最高, 因为它需要求出 $\boldsymbol{x}$ 和 C_k 的联合分布. 在许多应用中,$\boldsymbol{x}$ 是高维度的, 因此我们可能需要大量的训练集才能合理、准确地确定类-条件概率密度. 请注意, 类的先验概率 $p(C_k)$ 通常可以通过每个类中的训练集数据点的比例简单估算出来. 不过, 方法 (a) 的一个优点是, 它可以根据(5.32)求出 $p(\boldsymbol{x})$ 的边缘密度. 这有助于检测模型中概率较低的新数据点, 针对这些数据点的预测可能准确性较

低. 我们将此类检测称作离群值检测 (outlier detection) 或奇异值检测 (novelty detection)(Bishop, 1994; Tarassenko, 1995).

但是, 如果我们只想做分类决策, 那么寻找联合分布 $p(\boldsymbol{x}, C_k)$ 可能会浪费计算资源, 而且对数据的要求过高, 实际上我们只需要后验概率 $p(C_k \mid \boldsymbol{x})$, 其可通过方法 (b) 直接获得. 如图 5.8所示, 类-条件概率密度可能包含大量对后验概率影响不大的结构. 人们一直在对探索机器学习的生成法和判别法的相对优势, 以及寻找将它们结合起来的方法有着浓厚的兴趣 (Jebara, 2004; Lasserre, Bishop, and Minka, 2006).

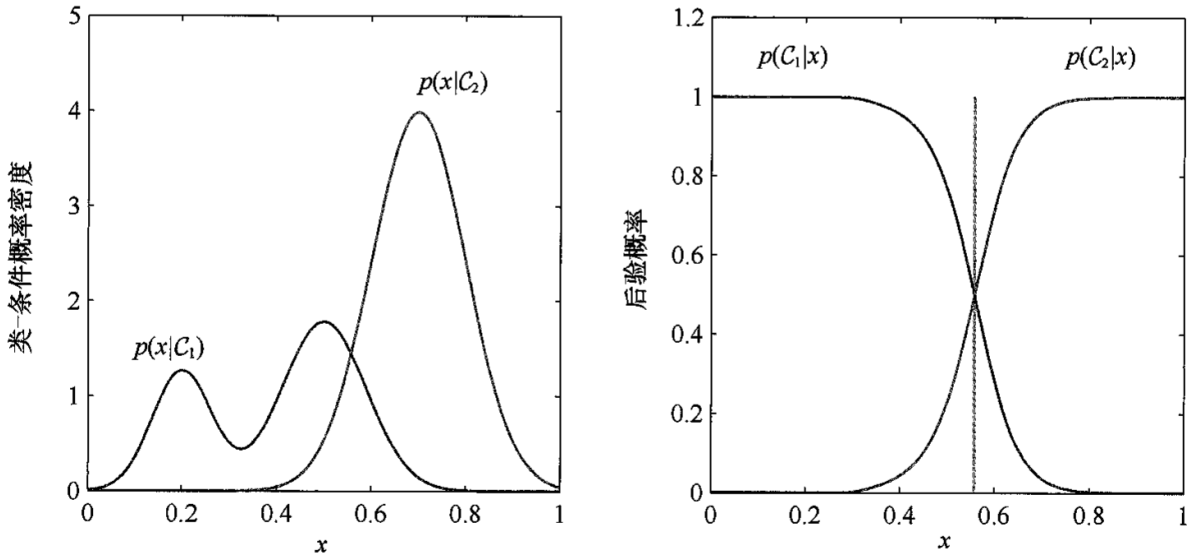


图 5.8: 具有单一输入变量 x 的两个类的类-条件概率密度 (左图) 和相应的后验概率 (右图). 请注意, 左图中的类-条件概率密度 $p(x \mid C_1)$(由蓝色曲线所示) 的左侧模式对后验概率没有影响. 右图中的垂直绿线表示 x 的决策边界, 假定先验概率 $p(C_1)$ 和 $p(C_2)$ 相等, 此时决策边界的误分类率最小

方法 (c) 更简单: 利用训练数据得出一个将每个新输入的 $\boldsymbol{x}$ 直接映射到一个类标签上的判别函数 $f(\boldsymbol{x})$, 从而将推理阶段和决策阶段合并为一个单一学习问题. 在图 5.8中, 这相当于求得垂直绿线所示的 x 值, 因为这是误分类率最小的决策边界.

然而, 如果采用方法 (c), 我们就无法再获得后验概率 $p(C_k \mid \boldsymbol{x})$. 我们有许多有力的理由计算后验概率, 这些理由如下.

(1) 降低风险. 考虑这样一个问题: 损失矩阵的元素时常会被修改 (例如修改可能发生在财务应用中). 如果我们知道后验概率, 就可以通过适当修改(5.30)来修订最小风险决策准则. 如果我们仅有一个判别函数, 那么损失矩阵的任何变化都要求我们返回训练数据并重新解决推理问题.

(2) 拒绝选项. 通过后验概率, 我们可以确定一个拒绝准则. 在给定拒绝数据点比例的情况下, 该拒绝准则能使误分类率或预期损失最小化.

(3) 补偿类的先验. 仍以癌症检查为例 (参见小节 2.1.1), 假设我们已从普通人群中收集大量皮肤病变图像作为训练数据, 并用这些数据建立了一个自动筛查系统. 由于癌症在普通人群中罕见, 我们可能会发现, 每 1000 个人中只有 1 人患有癌症.

如果使用这样一个数据集来训练自适应模型, 我们可能会遇到严重的困难, 因为癌症类数据的比例较小. 例如, 将每个点都归入正常类的分类器将达到 99.9% 的准确率, 而这种无效解可能很难避免. 此外, 即使是大型数据集, 其中包含的与癌症相对应的皮肤病变图像也非常少, 学习算法不会接触到很多此类图像, 因而不可能达到很好的泛化效果. 通过一个均衡的数据集 (其中的每个类都有相同数量的示例), 我们可以找到更精确的模型. 不过, 我们还必须对训练数据的修改效果进行补偿. 假设我们使用了这样一个修改过的数据集, 并找到了后验概率模型. 根据 Bayes 定理 [即(5.31)], 后验概率与先验概率成正比, 我们可以将后验概率解释为每个类中数据点的比例. 因此, 只需要将我们从人为平衡的数据集中得到的后验概率除以该数据集中的类比例, 再乘以我们希望应用该模型的人群中的类比例即可. 最后, 我们需要进行归一化处理, 以确保新的后验概

率的和为 1. 请注意, 如果我们直接学习判别函数, 而不是确定后验概率, 则无法应用这一方法.

(4) 模型整合. 在复杂的应用中, 我们可能希望将问题分解成若干较小的子问题, 每个子问题都可以用一个单独的模块来解决. 例如, 在医疗诊断问题中, 我们可以从血液检测和皮肤病变图像中获得信息. 与其将所有这些异构信息整合到一个巨大的输入空间中, 不如建立一个系统来解释图像, 并建立另一个系统来解释血液数据, 这样可能会更有效. 如果两个模型都给出了类的后验概率, 则可以利用概率规则系统地整合输出结果. 一种简单的方法是, 假设每个类的图像 (用 $\boldsymbol{x}_I$ 表示) 和血液数据 (用 $\boldsymbol{x}_B$ 表示) 的输入分布是独立的, 于是有

$$p(\boldsymbol{x}_I, \boldsymbol{x}_B \mid C_k) = p(\boldsymbol{x}_I \mid C_k)\, p(\boldsymbol{x}_B \mid C_k) \tag{5.33}$$

这是条件独立性 (参见节 11.2) 的一个例子, 因为当分布以 C_k 类为条件时, 独立性保持不变. 在给定图像和血液数据的条件下, 后验概率为

$$\begin{aligned} p(C_k \mid \boldsymbol{x}_I, \boldsymbol{x}_B) &\propto p(\boldsymbol{x}_I, \boldsymbol{x}_B \mid C_k)\, p(C_k) \\ &\propto p(\boldsymbol{x}_I \mid C_k)\, p(\boldsymbol{x}_B \mid C_k)\, p(C_k) \\ &\propto \frac{p(C_k \mid \boldsymbol{x}_I)\, p(C_k \mid \boldsymbol{x}_B)}{p(C_k)} \end{aligned} \tag{5.34}$$

因此, 我们需要类的先验概率 $p(C_k)$, 其可以很容易地从每个类的数据点比例中估算出来. 然后, 我们需要将得到的后验概率归一化, 使它们的和为 1. 特殊的条件独立性假设 [即(5.33)] 是朴素 Bayes 模型 (参见小节 11.2.4) 的一个例子. 注意, 在这个模型下, 联合边缘分布 $p(\boldsymbol{x}_I, \boldsymbol{x}_B)$ 通常不会分解. 后续章节将展示如何构建不需要条件独立性假设的数据组合模型. 使用输出概率而不是决策的模型的另一个优点是, 它们可以很容易地对任何可调参数 (例如多项式回归示例中的权重系数) 进行微分, 从而使用基于梯度 (参见第 7 章) 的优化方法来组合和联合训练它们.

命题 5.9 (条件独立性下的后验组合)

设(5.33)成立, 且 $p(C_k) > 0$、$p(\boldsymbol{x}_I \mid C_k) > 0$、$p(\boldsymbol{x}_B \mid C_k) > 0$. 则(5.34)成立, 即两个后验概率之积除以前验概率与组合后验概率成正比.

♠

证明

(重构新增) 由 Bayes 定理,$p(C_k \mid \boldsymbol{x}_I, \boldsymbol{x}_B) \propto p(\boldsymbol{x}_I, \boldsymbol{x}_B \mid C_k) p(C_k)$, 比例因子 $p(\boldsymbol{x}_I, \boldsymbol{x}_B)$ 与 k 无关. 代入(5.33)得第二个正比关系. 再由 Bayes 定理,$p(\boldsymbol{x}_I \mid C_k) = p(C_k \mid \boldsymbol{x}_I) p(\boldsymbol{x}_I) / p(C_k)$,$p(\boldsymbol{x}_B \mid C_k) = p(C_k \mid \boldsymbol{x}_B) p(\boldsymbol{x}_B) / p(C_k)$, 两式相乘得

$$p(\boldsymbol{x}_I \mid C_k)\, p(\boldsymbol{x}_B \mid C_k) = \frac{p(C_k \mid \boldsymbol{x}_I)\, p(C_k \mid \boldsymbol{x}_B)\, p(\boldsymbol{x}_I)\, p(\boldsymbol{x}_B)}{p(C_k)^2} \tag{5.35}$$

再乘以 $p(C_k)$, 与 k 无关的因子 $p(\boldsymbol{x}_I) p(\boldsymbol{x}_B)$ 和 $p(C_k)^{-1}$ 可一并吸收进正比号, 即得(5.34).

□

注

(重构新增) 命题的关键是分母 $p(C_k)$ 而不是常数: 它说明两个“独立来源”的后验概率不能直接相乘当作联合后验, 必须除以先验一次, 否则重复计入了先验. 这也解释了为什么在类别不平衡时, 先验的补偿必须显式进行 (见上面理由 (3)). 条件独立假设的强度在于它只要求以 C_k 为条件独立, 而不要求边缘独立; 因此(5.34)对 $p(\boldsymbol{x}_I, \boldsymbol{x}_B)$ 是否分解不作任何断言. 若去掉条件独立假设,(5.34)一般不成立, 需要显式建模两个来源的联合分布, 这正是后续章节讨论的“数据组合模型”要解决的问题.

5.2.5 分类器精度

衡量分类器性能的最简单方法是计算测试集中被正确分类的数据点所占的比例. 然而, 不同类型的误差会导致不同的后果, 正如损失矩阵所表现的那样. 因此, 我们往往不只是希望尽量减少误分类量. 通过改变决策边界的

位置, 我们可以在不同类型的错误之间做出权衡, 例如以最小化预期损失为目标. 考虑到该概念的重要性, 我们将介绍一些定义和术语, 以便更好地描述分类器的性能.

让我们再次回到癌症检查示例 (参见小节 2.1.1). 每个受测者都有一个是否患有癌症的 “真实标签” 以及分类器做出的预测.

定义 5.12 (混淆矩阵与真/假阳性)

设 N 是参加测试的总人数. 如果分类器对某个特定的人预测出癌症, 而这实际上是真实标签, 那么称这种预测为真阳性 (True Positive, TP); 但是, 如果这个人未患癌症, 则称这种预测为假阳性 (False Positive, FP). 同样, 如果分类器预测一个人未患癌症, 而且预测是正确的, 则称这种预测为真阴性 (True Negative, TN), 否则就称为假阴性 (False Negative, FN). 假阳性也称第一类误差 (type 1 error), 而假阴性则称第二类误差 (type 2 error). 设 N_{TP} 是真阳性人数,N_{FP} 是假阳性人数,N_{TN} 是真阴性人数,N_{FN} 是假阴性人数, 则它们满足

$$N = N_{\mathrm{TP}} + N_{\mathrm{FP}} + N_{\mathrm{TN}} + N_{\mathrm{FN}} \tag{5.36}$$

按行代表真实类、列代表根据决策准则所做的分类排成的 2×2 表, 称为混淆矩阵 (confusion matrix).

♣

如图 5.9所示, 这可以用混淆矩阵来表示.

	normal	cancer
normal	N_{TN}	N_{FP}
cancer	N_{FN}	N_{TP}

图 5.9: 癌症治疗问题的混淆矩阵, 其中行代表真实类, 列代表根据决策准则所做的分类. 该混淆矩阵的元素分别表示真阴性、假阳性、假阴性和真阳性的数量

注

(重构新增) 混淆矩阵把 “真实类” 与 “预测类” 交叉列表, 因此它保留了错误的方向信息, 而单一的误分类率会把两类错误混在一起. 四个计数之所以重要, 是因为几乎所有常用的评价指标都是它们的函数 (见定义 5.13). 注意 N_{TP} 与 N_{TN} 的和是正确分类数,N_{FP} 与 N_{FN} 的和是错误分类数, 故(5.36)只是把总人数按四种情形分解. 矩阵的行和分别等于两个类的人数 ($N_{\mathrm{TN}}+N_{\mathrm{FP}}$ 为真实正常人数), 列和分别等于被判为两类的数量; 这一事实在小节 5.2.6中用于解释阈值变化对混淆矩阵的影响.

定义 5.13 (准确率、精确率、召回率与假阳性率)

沿用定义 5.12的记号. 用正确分类率来衡量的准确率 (accuracy) 为

$$\text{准确率} = \frac{N_{\mathrm{TP}} + N_{\mathrm{TN}}}{N_{\mathrm{TP}} + N_{\mathrm{FP}} + N_{\mathrm{TN}} + N_{\mathrm{FN}}} \tag{5.37}$$

此外还定义精确率 (precision)、召回率 (recall, 也称真阳性率)、假阳性率与假发现率:

$$\text{精确率} = \frac{N_{\mathrm{TP}}}{N_{\mathrm{TP}} + N_{\mathrm{FP}}} \tag{5.38}$$

$$\text{召回率} = \frac{N_{\mathrm{TP}}}{N_{\mathrm{TP}} + N_{\mathrm{FN}}} \tag{5.39}$$

$$\text{假阳性率} = \frac{N_{\mathrm{FP}}}{N_{\mathrm{FP}} + N_{\mathrm{TN}}} \tag{5.40}$$

$$\text{假发现率} = \frac{N_{\mathrm{FP}}}{N_{\mathrm{FP}} + N_{\mathrm{TP}}} \tag{5.41}$$

♣

注

(重构新增) 这些指标的分母各不相同, 因而各自回答不同的问题: 准确率以全体为分母, 精确率以“被判为阳性者”为分母, 召回率以“真实阳性者”为分母, 假阳性率以“真实阴性者”为分母, 假发现率以“被判为阳性者”为分母 (故假发现率 = 1− 精确率). 在类别严重失衡时, 准确率会被多数类主导: 若每 1000 人中只有 1 人患癌, 把所有人都判为正常即可得到 99.9% 的准确率, 但召回率为 0. 正因为如此, 才需要精确率与召回率这类分别以行和与列和为分母的指标. 各分母都可能为零 (例如没有预测为阳性的样本时精确率无定义), 使用时应约定此时指标无意义而不是取 0. 另注意“假阳性率”与“假发现率”虽然都含“假阳性”, 但分母不同, 不能混用.

正如我们所看到的, 如果存在严重失衡的类, 则准确率可能会误导人. 例如, 在癌症检查示例中, 每 1000 人中只有 1 人患有癌症, 一个简单的判定无人患有癌症的朴素分类器将达到 99.9% 的准确率, 但它实际上毫无用处.

上述各量从不同侧面刻画混淆矩阵, 其中精确率与召回率最为常用. 在癌症检查示例中, 精确率代表对检测结果呈阳性的人确实患有癌症概率的预估, 而召回率则是对正确检测出癌症患者概率的预估. 假阳性率是对一个身体健康的人被归类为癌症患者概率的预估, 而假发现率则代表检查结果呈阳性但实际上没有患癌的比例——注意它等于 1 减去精确率.

通过改变决策边界的位置, 我们可以改变这两种误差之间的权衡. 为理解这种权衡, 让我们重温一下图 5.5. 然而, 如图 5.10所示, 我们现在对各个区域进行标记. 我们可以将标记的区域与如下各种真假率联系起来:

$$N_{\mathrm{FP}}/N = E \tag{5.42}$$

$$N_{\mathrm{TP}}/N = D + E \tag{5.43}$$

$$N_{\mathrm{FN}}/N = B + C \tag{5.44}$$

$$N_{\mathrm{TN}}/N = A + C \tag{5.45}$$

其中我们隐式地考虑极限 $N \to \infty$, 从而可以将观测值的数量与概率联系起来.

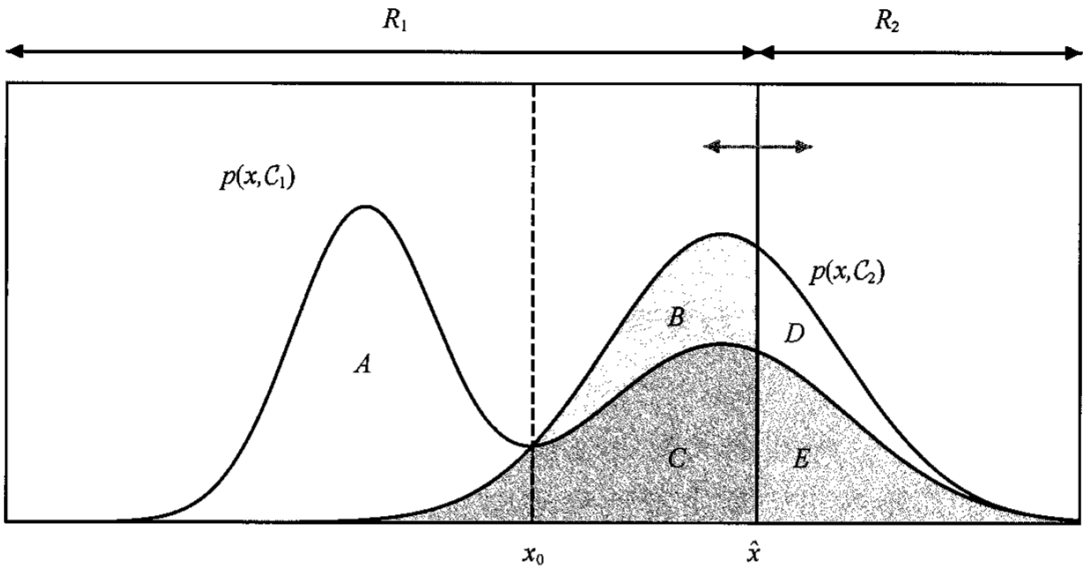


图 5.10: 标记各个区域. 在癌症治疗问题中, 区域 $\mathcal{R}_1$ 被归入正常类, 而区域 $\mathcal{R}_2$ 被归入癌症类

命题 5.10 (区域面积与计数比例的关系)

设 x 为单一输入变量,$\mathcal{R}_1 = (-\infty, \hat{x}), \mathcal{R}_2 = [\hat{x}, \infty), x_0 < \hat{x}$ 为两条曲线 $p(x, C_1)$ 与 $p(x, C_2)$ 的交点, 并按图 5.10记 $A = \int_{-\infty}^{x_0} p(x, C_1)\,\mathrm{d}x, C = \int_{x_0}^{\hat{x}} p(x, C_1)\,\mathrm{d}x, E = \int_{\hat{x}}^{\infty} p(x, C_1)\,\mathrm{d}x, B = \int_{x_0}^{\hat{x}} \{p(x, C_2) - p(x, C_1)\}\,\mathrm{d}x, D = \int_{\hat{x}}^{\infty} \{p(x, C_2) - p(x, C_1)\}\,\mathrm{d}x$. 若 $p(x, C_2)$ 在 $(-\infty, x_0)$ 上的积分为零, 则

$$\frac{N_{\mathrm{TN}}}{N} = A + C, \qquad \frac{N_{\mathrm{FP}}}{N} = E, \qquad \frac{N_{\mathrm{FN}}}{N} = B + C, \qquad \frac{N_{\mathrm{TP}}}{N} = D + E \tag{5.46}$$

其中各比例按 $N \to \infty$ 解释为概率.

证明

(重构新增) 按定义,N_{TN}/N 是“真实类为 C_1 且被判为正常”的频率, 当 $N \to \infty$ 时趋于 $\int_{\mathcal{R}_1} p(x, C_1)\,\mathrm{d}x$. 而 $\mathcal{R}_1 = (-\infty, \hat{x})$ 被 x_0 分成两段, 故该积分等于 $A + C$(A 对应 $(-\infty, x_0)$,C 对应 $(x_0, \hat{x})$). 同理,N_{FP}/N 趋于真实类为 C_1 而被判为癌症的概率 $\int_{\mathcal{R}_2} p(x, C_1)\,\mathrm{d}x = E$;$N_{\mathrm{FN}}/N$ 趋于真实类为 C_2 而被判为正常的概率 $\int_{\mathcal{R}_1} p(x, C_2)\,\mathrm{d}x$, 由 $p(x, C_2)$ 在 $(-\infty, x_0)$ 上积分为零, 它等于 $\int_{x_0}^{\hat{x}} p(x, C_2)\,\mathrm{d}x = B + C$;$N_{\mathrm{TP}}/N$ 趋于真实类为 C_2 而被判为癌症的概率 $\int_{\hat{x}}^{\infty} p(x, C_2)\,\mathrm{d}x = D + E$. 这里用到 $\int_{x_0}^{\hat{x}} p(x, C_2)\,\mathrm{d}x = B + C$ 与 $\int_{\hat{x}}^{\infty} p(x, C_2)\,\mathrm{d}x = D + E$, 它们正是 B、C、D、E 的定义的直接变形.

□

注

(重构新增) 四个恒等式合起来给出 $N_{\mathrm{TN}}/N + N_{\mathrm{FP}}/N + N_{\mathrm{FN}}/N + N_{\mathrm{TP}}/N = (A+C+E) + (B+C+D+E) = p(C_1) + p(C_2) = 1$, 与(5.36)一致; 这里 $A + C + E$ 是 $p(x, C_1)$ 的全积分,$B + C + D + E$ 是 $p(x, C_2)$ 在 (x_0, ∞) 上的积分, 后者等于其全积分用了 $p(x, C_2)$ 在 $(-\infty, x_0)$ 上积分为零的假设. 若该假设不成立, 则 N_{FN} 还须加上 $p(x, C_2)$ 在 $(-\infty, x_0)$ 上的质量, 恒等式要作相应修改. 证明的关键是: 计数比例在 $N \to \infty$ 时收敛到相应的概率, 而概率可以按区域拆成积分. 图中 B 与 C 之和恰是 $p(x, C_2)$ 在 $(x_0, \hat{x})$ 上的积分, 这正是“B 与 C 堆叠”画法的含义.

5.2.6 ROC 曲线

概率分类器会输出一个后验概率, 通过设置阈值可以将其转换为决策. 随着阈值的变化, 我们可以通过增加第二类误差来减少第一类误差, 反之亦然. 为更好地理解这种权衡, 绘制 ROC(Receiver Operating Characteristic, 受试者工作特征) 曲线 (Fawcett, 2006) 是非常有用的.

定义 5.14 (ROC 曲线)

把分类器的决策阈值从 $-\infty$ 移动到 ∞, 以假阳性率为横坐标、以真阳性率 (召回率) 为纵坐标, 把阈值每取一个值时得到的点 (假阳性率, 真阳性率) 连成的曲线, 称为该分类器的 ROC 曲线 (Receiver Operating Characteristic curve).

图 5.11是真阳性率与假阳性率的对比图. 当我们把图 5.10中的决策边界从 $-\infty$ 移动到 ∞ 时, 通过绘制 y 轴上癌症检查正确率的累积比例与 x 轴上癌症检查错误率的累积比例的对比图, 就能描绘并生成 ROC 曲线. 请注意, 特定的混淆矩阵代表 ROC 曲线上的一个点. 最佳分类器将由 ROC 曲线图左上角的一个点表示.ROC 曲线图的左下角表示一个将每个点都归入正常类的简单分类器, 该分类器虽无真阳性, 但也无假阳性. 同样,ROC 曲线图的右上角代表一个将所有点都归入癌症类的分类器, 该分类器虽无假阴性, 但也无真阴性. 在图 5.11中, 无论选择哪种假阳性率, 蓝色曲线表示的分类器都优于红色曲线表示的分类器. 不过, 这些曲线也有可能交叉, 在这种情况下, 具体选择哪条曲线更好则取决于操作点的选择.

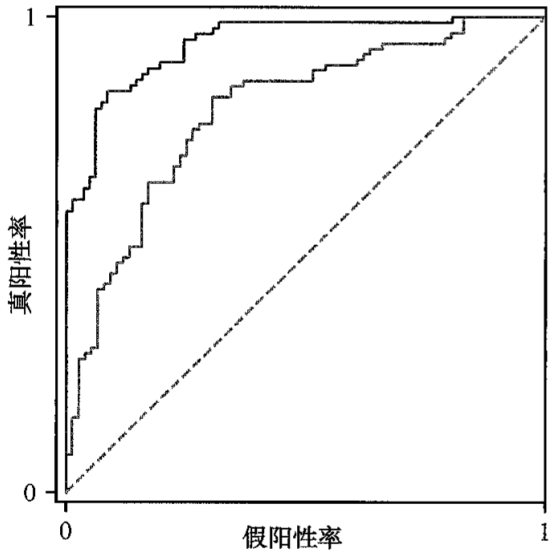


图 5.11: ROC 曲线图是真阳性率与假阳性率的对比图, 它描述了分类问题中第一类误差和第二类误差之间的权衡. 上方的蓝色曲线代表比下方的红色曲线更好的分类器. 此处, 虚线表示简单随机分类器的性能

注

(重构新增) ROC 曲线的每一点都对应一个阈值, 因此曲线整体刻画了分类器在所有可能的操作点上的性能, 而不依赖于某一个具体阈值的选择. 曲线必然经过 $(0,0)$ 与 $(1,1)$ 两点: 阈值取 $-\infty$ 时所有样本都被判为阳性, 假阳性率与真阳性率都为 1; 阈值取 $+\infty$ 时所有样本都被判为阴性, 两者都为 0(此处按“大于阈值判为阳性”的约定). 曲线位于对角线以上的分类器优于随机猜测; 位于对角线以下则说明把决策反号即可改善. 两条曲线交叉时不存在一致的优劣, 必须结合具体的代价或先验来选择操作点, 这也正是命题 5.8中损失矩阵所起的作用. 注意 ROC 曲线只适用于二分类; 对多分类需要对每个类做一对多处理, 随类数增加而迅速变得烦琐, 这正是本节末尾讨论的局限.

我们可以考虑使用随机分类器作为基线, 该分类器可以简单地将每个数据点以概率 p 归为癌症类, 以概率 $1-p$ 归为正常类. 如图 5.11所示, 当我们改变 p 值时, 它将描绘出一条由对角线构成的 ROC 曲线. 任何低于对角线的分类器都比不上随机猜测.

有时, 用一个数字来描述整条 ROC 曲线是非常有用的. 一种方法是测量 ROC 曲线下面积 (Area Under the Curve, AUC).

定义 5.15 (AUC 与 F 分数)

ROC 曲线下面积 (Area Under the Curve, AUC) 是指 ROC 曲线与横轴之间区域的面积, 其取值在 0 与 1 之间.AUC 值为 0.5 代表随机猜测, 而 AUC 值为 1.0 代表完美分类器. 另一种用一个数字概括分类器性能的量是 F 分数, 它是精确率和召回率的几何平均, 由下式定义:

$$F\text{分数} = \frac{2\times \text{精确率}\times \text{召回率}}{\text{精确率}+\text{召回率}} \tag{5.47}$$

$$= \frac{2N_{\mathrm{TP}}}{2N_{\mathrm{TP}}+N_{\mathrm{FP}}+N_{\mathrm{FN}}} \tag{5.48}$$

♣

注

(重构新增) AUC 的概率解释是: 随机抽取一个正样本与一个负样本, 分类器把正样本判得比负样本更“像正类”的概率 (即正样本得分高于负样本得分的概率), 因此 AUC 与阈值无关, 适合比较分类器的整体排序能力.F 分数实际上是精确率与召回率的调和平均: 两数之积除以两数之和的两倍 (正文按原书称为几何平均, 但由(5.47)可见它是调和平均, 这不影响其作为综合指标的用途), 当两者之一为 0 时 F 分数为 0, 因此它惩罚“只擅长一件事”的分类器.AUC 与 F 分数回答的问题不同: 前者衡量排序质量, 后者衡量某个具体阈值下的综合性能, 故不能互相替代. 它们的取值范围都在 $[0,1]$ 内, 但 F 分数没有“0.5 对应随机”这样的基准.

命题 5.11 (F 分数的显式形式)

由定义 5.13中的精确率与召回率,(5.47)可化为(5.48).

证明

(重构新增) 记 $P = N_{\mathrm{TP}}/(N_{\mathrm{TP}} + N_{\mathrm{FP}})$,$R = N_{\mathrm{TP}}/(N_{\mathrm{TP}} + N_{\mathrm{FN}})$. 则

$$\frac{2PR}{P+R} = \frac{2\,\dfrac{N_{\mathrm{TP}}^2}{(N_{\mathrm{TP}}+N_{\mathrm{FP}})(N_{\mathrm{TP}}+N_{\mathrm{FN}})}}{\dfrac{N_{\mathrm{TP}}}{N_{\mathrm{TP}}+N_{\mathrm{FP}}} + \dfrac{N_{\mathrm{TP}}}{N_{\mathrm{TP}}+N_{\mathrm{FN}}}} = \frac{2N_{\mathrm{TP}}}{(N_{\mathrm{TP}}+N_{\mathrm{FN}}) + (N_{\mathrm{TP}}+N_{\mathrm{FP}})} \tag{5.49}$$

其中第二个等号是分子分母同乘 $(N_{\mathrm{TP}} + N_{\mathrm{FP}})(N_{\mathrm{TP}} + N_{\mathrm{FN}})/N_{\mathrm{TP}}$ 得到的 ($N_{\mathrm{TP}} > 0$). 整理分母即得(5.48). 若 $N_{\mathrm{TP}} = 0$, 则 $P = R = 0$, 按 F 分数的定义其值为 0, 而(5.48)的右端也为 0, 故公式仍然成立.

□

注

(重构新增) 命题的推导只用了代数变形, 但它说明了 F 分数的一个实用优点: 无需分别计算精确率与召回率, 直接用混淆矩阵的计数即可求出, 从而避免了两个分母分别为零时的定义问题 (只要 $N_{\mathrm{TP}} > 0$).(5.48)还显示 F 分数对 N_{TP} 的依赖是“双倍加权”的: 分子与分母都含 $2N_{\mathrm{TP}}$, 因此 N_{TP} 的增加会同时提高分子并使分母相对不变. 当 $N_{\mathrm{FP}} = N_{\mathrm{FN}} = 0$ 时 $F = 1$; 当 $N_{\mathrm{TP}} = 0$ 时 $F = 0$.

当然, 我们也可以将图 5.9中的混淆矩阵与图 5.6中的损失矩阵结合起来, 从而通过将元素逐点相乘求和来计算预期损失. 下面的命题把这一算法形式化.

命题 5.12 (由混淆矩阵与损失矩阵计算预期损失)

设 N_{kj} 表示真实类为 C_k 而被归入 C_j 的样本数, 即定义 5.12意义下混淆矩阵的 (k, j) 元素,$N = \sum_{k,j} N_{kj}$. 则对该决策规则, 当 $N \to \infty$ 时预期损失(5.29)可由

$$\mathbb{E}[L] = \frac{1}{N} \sum_{k} \sum_{j} L_{kj}\, N_{kj} \tag{5.50}$$

估计.

证明

(重构新增) 由(5.29),$\mathbb{E}[L] = \sum_{k} \sum_{j} L_{kj}\, P(\text{真实类为}C_k,\ \text{归入}C_j)$. 在决策规则固定的条件下,$N_{kj}/N$ 是事件“真实类为 C_k 且被归入 C_j”的频率, 由大数定律, 当 $N \to \infty$ 时它依概率收敛到该事件的概率. 代入即得(5.50).

□

注

(重构新增) (5.50)说明: 混淆矩阵一旦确定, 任何损失矩阵下的预期损失都只是逐元素相乘再求和, 无需重新训练分类器. 这正是小节 5.2.4中“理由 (1): 降低风险”的定量版本——保存后验概率 (或混淆矩阵) 就能应对损失矩阵的变化; 反之, 若只保存判别函数, 损失矩阵一变就必须重新学习. 该式也把 ROC 曲线与损失联系起来:ROC 曲线上的每一点对应一个混淆矩阵, 因而对应一个预期损失; 改变阈值就是在这些点之间移动 (见图 5.11). 注意该估计假设测试集与部署分布一致; 若先验发生偏移,N_{kj} 不再代表部署时的频率, 需要按理由 (3) 的方法补偿.

虽然 ROC 曲线可以扩展到两个以上的类, 但随着类数量的增加, 它很快就会变得十分烦琐.

5.3 生成分类器

本节将从概率的角度分析分类问题并展示如何通过对数据分布进行简单的假设来建立具有线性决策边界的模型. 我们已经讨论过分类的判别法和生成法的区别 (参见小节 5.2.4). 在这里, 我们将采用对类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 以及类的先验概率 $p(C_k)$ 进行建模的生成法, 然后通过 Bayes 定理使用这些量来计算后验概率 $p(C_k \mid \boldsymbol{x})$.

首先考虑二分类问题.C_1 类的后验概率可以写成

$$\begin{aligned} p(C_1 \mid \boldsymbol{x}) &= \frac{p(\boldsymbol{x} \mid C_1)\, p(C_1)}{p(\boldsymbol{x} \mid C_1)\, p(C_1) + p(\boldsymbol{x} \mid C_2)\, p(C_2)} \\ &= \frac{1}{1+\exp(-a)} = \sigma(a) \end{aligned} \tag{5.51}$$

其中:

$$a = \ln \frac{p(\boldsymbol{x} \mid C_1)\, p(C_1)}{p(\boldsymbol{x} \mid C_2)\, p(C_2)} \tag{5.52}$$

定义 5.16 (logistic sigmoid 函数)

函数

$$\sigma(a) = \frac{1}{1+\exp(-a)} \tag{5.53}$$

称为 logistic sigmoid 函数.

♣

logistic sigmoid 函数 $\sigma(a)$ 如图 5.12所示, 这类函数有时也称 “压缩函数” , 因为它们能将整个实轴映射到一个有限区间内.logistic sigmoid 函数 $\sigma(a)$ 在许多分类算法中发挥着重要作用, 其满足如下对称性:

$$\sigma(-a) = 1 - \sigma(a) \tag{5.54}$$

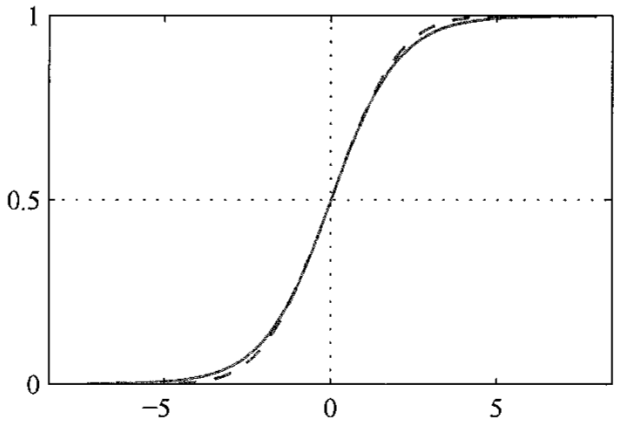


图 5.12: 红色实线表示由(5.53)定义的 logistic sigmoid 函数 $\sigma(a)$, 蓝色虚线表示 $\lambda^2 = \pi/8$ 时的缩放 probit 函数 $\Phi(\lambda a)$, 其中 $\Phi(a)$ 由(5.122)定义. 选择缩放因子 $\pi/8$ 是为了使 $a = 0$ 时这两条曲线的导数相等

注

(重构新增) 定义 5.16中的 σ 把 $\mathbb{R}$ 双射到 $(0,1)$, 其反函数即下面的 logit 函数, 因此它是 “把线性函数的输出转换为概率” 的最自然选择之一.(5.54)可由定义直接验证:$\sigma(-a) = 1/(1+\exp a) = \exp(-a)/(1+\exp(-a)) = 1-\sigma(a)$. 该对称性说明 $\sigma(0) = 1/2$, 即 “两个类概率相等” 对应预激活为 0, 这正是决策面的位置.σ 的导数满足 $\sigma'(a) = \sigma(a)(1-\sigma(a))$(见命题 5.18), 这一性质使交叉熵的梯度化简为 “误差乘以特征” , 是小节 5.4.3的出发点. 注意 σ 没有封闭形式的反函数初等表达式以外的简化, 其逆就是 logit; 在选择激活函数时,logistic 与 probit 的差别主要在于尾部衰减速度 (见小节 5.4.5).

logistic sigmoid 函数 $\sigma(a)$ 的倒数为

$$a = \ln\left(\frac{\sigma}{1-\sigma}\right) \tag{5.55}$$

定义 5.17 (logit 函数)

(5.55)定义的函数称为 logit 函数. 它表示两个类的概率之比的对数 $\ln[p(C_1 \mid \boldsymbol{x})/p(C_2 \mid \boldsymbol{x})]$, 也称对数几率. ♣

注

(重构新增) “几率”指事件发生与不发生的概率之比,logit 即几率的对数, 故 logit 把 $(0,1)$ 映到 $\mathbb{R}$, 与 σ 互为反函数.(5.51)与(5.52)合起来说明: 后验概率的对数几率恰等于两个“类-条件密度乘先验”之比的对数. 这一等式的用处在于, 只要该对数比值是 $\boldsymbol{x}$ 的线性函数, 后验就自动具有 logistic 形式;$\boldsymbol{x}$ 本身的分布形式并不重要, 重要的只是比值的函数形式. 这就是下一小节要研究的“线性”从何而来的问题.

请注意, 在(5.51)中, 我们只是以等价形式重写了后验概率, 因此 logistic sigmoid 函数的出现似乎是人为的. 不过, 只要 $a(\boldsymbol{x})$ 具有约束函数形式, 它就具有重要意义. 我们很快就会讨论 $a(\boldsymbol{x})$ 是 $\boldsymbol{x}$ 的线性函数的情况, 在此情况下, 后验概率受广义线性模型的控制.

如果存在 $K > 2$ 个类, 则有

$$\begin{aligned} p(C_k \mid \boldsymbol{x}) &= \frac{p(\boldsymbol{x} \mid C_k)\, p(C_k)}{\sum_j p(\boldsymbol{x} \mid C_j)\, p(C_j)} \\ &= \frac{\exp(a_k)}{\sum_j \exp(a_j)} \end{aligned} \tag{5.56}$$

这就是归一化指数 (normalized exponential), 它可以视为 logistic sigmoid 函数的多类泛化. 在这里, 量 a_k 由下式定义:

$$a_k = \ln\big(p(\boldsymbol{x} \mid C_k)\, p(C_k)\big) \tag{5.57}$$

定义 5.18 (归一化指数与 softmax 函数)

对实数 $a_1, \cdots, a_K$, 由

$$y_k = \frac{\exp(a_k)}{\sum_j \exp(a_j)}, \qquad k = 1, \cdots, K, \tag{5.58}$$

定义的变换称为归一化指数 (normalized exponential), 也称 softmax 函数, 其中 a_k 称为预激活. ♣

归一化指数也叫 softmax 函数, 因为它代表了 max 函数的平滑版本. 如果 $a_k \gg a_j$, 则对于所有 $j \neq k$,$p(C_k \mid \boldsymbol{x}) \approx 1$ 且 $p(C_j \mid \boldsymbol{x}) \approx 0$.

注

(重构新增) softmax 与 logistic sigmoid 的关系是:$K = 2$ 时, 令 $a = a_1 - a_2$, 则 $y_1 = \exp(a_1)/(\exp(a_1) + \exp(a_2)) = 1/(1 + \exp(-a)) = \sigma(a)$, 故 sigmoid 是 softmax 的二类特例.softmax 的一个重要性质是平移不变性: 把所有 a_k 同时加上常数 c,y_k 不变, 因此(5.56)中的 a_k 只需确定到一个共同的加性常数, 这也说明多类模型的参数存在冗余 (见小节 5.4.4). 另一个性质是 $\sum_k y_k = 1$ 且 $y_k > 0$, 因此输出可以直接解释为概率, 这与最小二乘分类的输出不同 (见小节 5.1.4). “softmax”之名来自它是对 max 的平滑近似: 当某个 a_k 远大于其他时,$y_k \to 1$, 其余趋于 0.

接下来研究选择特定形式的类-条件概率密度的后果. 首先讨论连续输入变量 $\boldsymbol{x}$, 然后简要讨论离散输入.

5.3.1 连续输入

假设类-条件概率密度是 Gauss 密度. 我们将探讨由此产生的后验概率形式. 假设所有类共享相同的协方差矩阵 $\boldsymbol{\Sigma}$, 则 C_k 类的密度为

$$p(\boldsymbol{x} \mid C_k) = \frac{1}{(2\pi)^{D/2}|\boldsymbol{\Sigma}|^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_k)\right\} \tag{5.59}$$

假设我们有两个类. 根据(5.51)与(5.52), 可得

$$p(C_1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0) \tag{5.60}$$

其中

$$\boldsymbol{w} = \boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) \tag{5.61}$$

$$w_0 = -\frac{1}{2}\boldsymbol{\mu}_1^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln\frac{p(C_1)}{p(C_2)} \tag{5.62}$$

命题 5.13 (Gauss 类-条件密度导出的线性后验)

设两个类的类-条件密度由(5.59)给出且共享同一协方差矩阵 $\boldsymbol{\Sigma}$($\boldsymbol{\Sigma}$ 正定), 先验概率满足 $p(C_k) > 0$. 则 C_1 的后验概率具有(5.60)的形式, 其中 $\boldsymbol{w}$ 与 w_0 分别由(5.61)与(5.62)给出. 特别地, 后验概率的对数几率是 $\boldsymbol{x}$ 的线性函数.

♠

证明

(重构新增) 由(5.52),

$$a(\boldsymbol{x}) = \ln\frac{p(C_1)}{p(C_2)} + \ln p(\boldsymbol{x} \mid C_1) - \ln p(\boldsymbol{x} \mid C_2) \tag{5.63}$$

两个密度的归一化常数相同 (因为 $\boldsymbol{\Sigma}$ 相同、维数相同), 相减时消去, 故只需计算指数部分之差. 展开

$$-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_1)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_1) = -\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} + \boldsymbol{\mu}_1^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} - \frac{1}{2}\boldsymbol{\mu}_1^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 \tag{5.64}$$

对 $k = 2$ 有同样的展开. 两式相减, 二次项 $-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x}$ 完全抵消 (这正是“共享协方差”假设的作用), 剩下

$$a(\boldsymbol{x}) = (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} - \frac{1}{2}\boldsymbol{\mu}_1^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_1 + \frac{1}{2}\boldsymbol{\mu}_2^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_2 + \ln\frac{p(C_1)}{p(C_2)} \tag{5.65}$$

由于 $\boldsymbol{\Sigma}^{-1}$ 对称,$(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} = [\boldsymbol{\Sigma}^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2)]^{\mathsf{T}}\boldsymbol{x} = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}$, 其中 $\boldsymbol{w}$ 如(5.61); 其余与 $\boldsymbol{x}$ 无关的项即 w_0, 如(5.62). 再由定义 5.16与(5.51),$p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x}))$. 最后,$a$ 是 $\boldsymbol{x}$ 的仿射函数, 故对数几率 $\ln[p(C_1 \mid \boldsymbol{x})/p(C_2 \mid \boldsymbol{x})] = a(\boldsymbol{x})$ 是线性的.

□

注

(重构新增) 命题中三个条件各有作用:$\boldsymbol{\Sigma}$ 正定保证 $\boldsymbol{\Sigma}^{-1}$ 存在且密度归一化; 两个类共享 $\boldsymbol{\Sigma}$ 保证二次项相消, 这是后验成为线性的关键 (若各自协方差不同, 见定义 5.19);$p(C_k) > 0$ 保证对数有意义.w_0 中的 $\ln[p(C_1)/p(C_2)]$ 说明先验只影响偏置, 因此改变先验只会使决策面平行移动, 而不改变其朝向, 这与命题 5.1中“w_0 决定位置、$\boldsymbol{w}$ 决定朝向”的解释一致. 注意 $\boldsymbol{w}$ 不依赖于先验, 也不依赖于 $\boldsymbol{x}$; 它只由两个均值与共享协方差决定. 这一结果的意义在于: 不必假设 $\boldsymbol{x}$ 的分布是 Gauss 的, 只要两个类的对数密度比是线性的, 后验就具有 logistic 形式.

可以看到, 来自 Gauss 密度指数的 $\boldsymbol{x}$ 中的二次项已被消去 (缘于共享协方差矩阵的假设), 从而在 logistic sigmoid 函数的变量中形成了 $\boldsymbol{x}$ 的线性函数.图 5.13展示了二维输入空间 $\boldsymbol{x}$ 的结果. 由此得出的决策边界对应于后验概率 $p(C_k \mid \boldsymbol{x})$ 恒定的曲面, 因此后验概率 $p(C_k \mid \boldsymbol{x})$ 将由 $\boldsymbol{x}$ 的线性函数给出, 决策边界在输入空间中是线性的. 先验概率 $p(C_k)$ 仅通过偏置参数 w_0 进入, 因此先验的变化会使决策边界发生平行位移, 更一般地说, 这会使恒定后验概率的轮廓发生平行位移.

对于 K 个类的一般情况, 后验概率由(5.56)得出. 根据(5.57)和(5.59), 得出

$$a_k(\boldsymbol{x}) = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0} \tag{5.66}$$

其中

$$\boldsymbol{w}_k = \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k \tag{5.67}$$

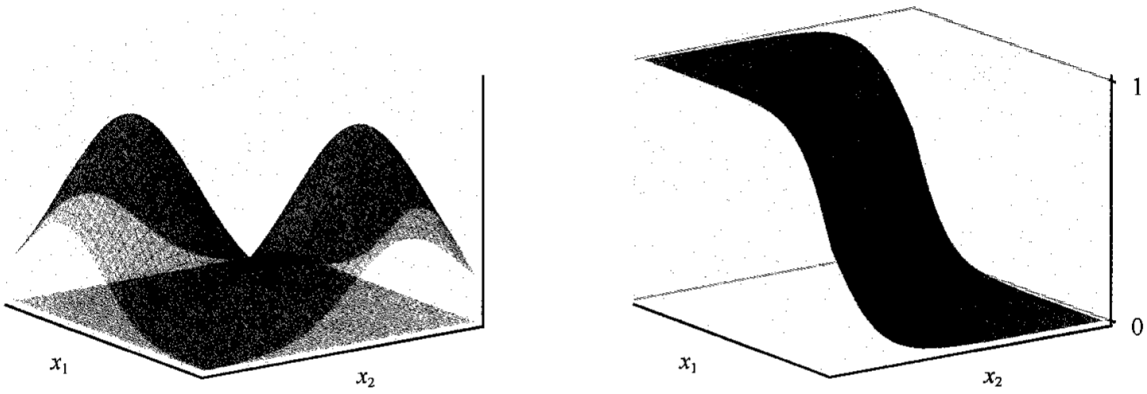


图 5.13: 左图显示了两个类的类-条件概率密度, 分别用红色和蓝色表示. 右图是相应的后验概率 $p(C_1 \mid \boldsymbol{x})$, 由作用于 $\boldsymbol{x}$ 的线性函数的 logistic sigmoid 函数给出. 右图中的面是用红色和蓝色涂色的, 红色的比例为 $p(C_1 \mid \boldsymbol{x})$, 蓝色的比例为 $p(C_2 \mid \boldsymbol{x}) = 1 - p(C_1 \mid \boldsymbol{x})$

$$w_{k0} = -\frac{1}{2}\boldsymbol{\mu}_k^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k + \ln p(C_k) \tag{5.68}$$

命题 5.14 (多类情形下的线性预激活)

设 K 个类的类-条件密度由(5.59)给出且共享同一正定协方差矩阵 $\boldsymbol{\Sigma}$. 则(5.57)中的 a_k 与(5.66)只相差一个与 k 无关的常数; 特别地,$a_k(\boldsymbol{x})$ 可以取为 $\boldsymbol{x}$ 的线性函数, 且(5.56)中的后验概率只依赖于这些线性函数.

证明

(重构新增) 由(5.57),

$$a_k = \ln p(C_k) - \frac{D}{2}\ln(2\pi) - \frac{1}{2}\ln|\boldsymbol{\Sigma}| - \frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_k)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_k) \tag{5.69}$$

展开二次型得

$$a_k = -\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} + \boldsymbol{\mu}_k^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x} - \frac{1}{2}\boldsymbol{\mu}_k^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k + \ln p(C_k) - \frac{D}{2}\ln(2\pi) - \frac{1}{2}\ln|\boldsymbol{\Sigma}| \tag{5.70}$$

其中第一项与 k 无关. 把它与其余与 k 无关的常数项一并记为 $c(\boldsymbol{x})$, 则 $a_k = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0} + c(\boldsymbol{x})$, 其中 $\boldsymbol{w}_k = \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k$、$w_{k0} = -\frac{1}{2}\boldsymbol{\mu}_k^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k + \ln p(C_k)$. 由于 softmax 对每个 a_k 同时加上同一个量不变 (见定义 5.18后的注记),$c(\boldsymbol{x})$ 可以从(5.56)中约去, 故后验概率由线性函数 $\boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{x} + w_{k0}$ 决定.

□

注

(重构新增) 与二分类情形相比, 多类情形多了一个“公共二次项” $-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}\boldsymbol{x}$: 它并不消失, 而是对所有 k 相同, 因而在 softmax 中被约去. 这说明“线性”是相对于 softmax 而言的, 而不是说 a_k 本身没有二次项.$\boldsymbol{w}_k = \boldsymbol{\Sigma}^{-1}\boldsymbol{\mu}_k$ 表明各类的权重向量是均值经同一线性变换得到的, 因此当两个类的均值之差与 $\boldsymbol{\Sigma}^{-1}$ 作用后方向相同时, 两类之间的决策面与二分类情形一致.w_{k0} 中的 $\ln p(C_k)$ 说明先验只平移各类的预激活, 从而改变决策面到原点的位置.

可以看到,$a_k(\boldsymbol{x})$ 是 $\boldsymbol{x}$ 的线性函数, 因为共享协方差矩阵消去了二次项. 当两个后验概率 (后验概率中最大的两个) 相等时, 就会出现与最小误分类率对应的决策边界, 因此决策边界将由 $\boldsymbol{x}$ 的线性函数定义. 我们再次得到了一个广义线性模型.

如果我们放宽共享协方差矩阵的假设, 并允许每个类-条件概率密度 $p(\boldsymbol{x}, C_k)$ 有自己的协方差矩阵 $\boldsymbol{\Sigma}_k$, 那么先前的二次项消去将不再发生, 我们将得到 $\boldsymbol{x}$ 的二次函数, 从而产生二次判别式 (quadratic discriminant). 线性决策边界和二次方决策边界如图 5.14所示.

定义 5.19 (二次判别式)

若各类的类-条件密度为具有各自协方差矩阵 $\boldsymbol{\Sigma}_k$ 的 Gauss 密度, 则后验概率的预激活 $a_k(\boldsymbol{x})$ 是 $\boldsymbol{x}$ 的二次函数, 相应的决策面为二次曲面. 这样的判别式称为二次判别式 (quadratic discriminant).

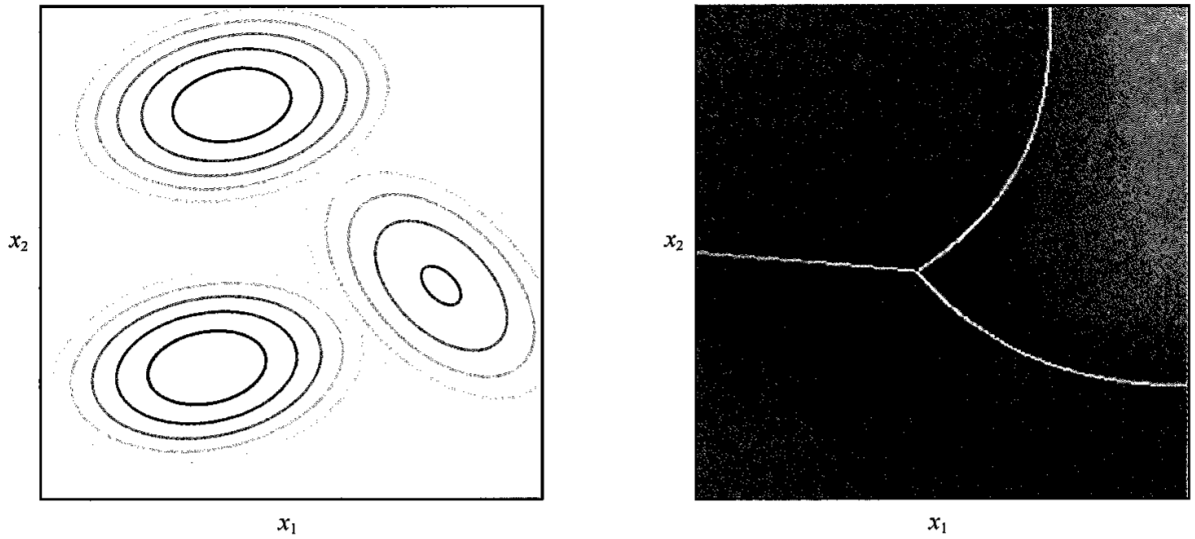


图 5.14: 左图显示了三个类的类-条件概率密度, 每个类都符合 Gauss 分布, 颜色分别为红色、绿色和蓝色, 其中红色类和蓝色类的协方差矩阵相同. 右图展示了相应的后验概率, 其中图像上的每个点都用红、蓝、绿三种颜色按比例着色, 分别对应三个类的后验概率. 右图还展示了决策边界, 请注意, 具有相同协方差矩阵的红色类和蓝色类之间的边界是线性的, 而红色类和绿色类以及蓝色类和绿色类之间的边界是二次的

注

(重构新增) 二次判别式比线性判别式多出的自由度来自“协方差可以不同”这一放宽:(5.66)的推导中, 二次项之所以能约去, 靠的是所有类共用同一个 $\boldsymbol{\Sigma}$; 一旦 $\boldsymbol{\Sigma}_k$ 随 k 变化, 二次项 $-\frac{1}{2}\boldsymbol{x}^{\mathsf{T}}\boldsymbol{\Sigma}_k^{-1}\boldsymbol{x}$ 就依赖于 k, 无法在 softmax 中约去, 于是 a_k 成为 $\boldsymbol{x}$ 的二次函数. 参数数量随之增加 (每个类多出 $D(D+1)/2$ 个参数), 所需训练数据也更多. 二次判别式仍然是“生成式”的: 它由类-条件密度的形式推出, 而不是直接假设后验的形式.图 5.14中“红色类与蓝色类的边界是线性的”正是它们协方差相同的结果, 这为“共享协方差 $\Rightarrow$ 线性”提供了直观证据.

5.3.2 最大似然解

在为类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 指定了一个参数函数形式后, 就可以使用最大似然法确定参数值和类的先验概率 $p(C_k)$. 此时需要一个包含 $\boldsymbol{x}$ 的观测值及其相应类标签的数据集.

假设我们有两个类, 每个类都有一个共享协方差矩阵的 Gauss 类-条件概率密度. 此外, 假设我们有一个数据集 $\{\boldsymbol{x}_n, t_n\}$, 其中 $n = 1, \cdots, N$.$t_n = 1$ 表示 C_1 类,$t_n = 0$ 表示 C_2 类. 将类的先验概率记为 $p(C_1) = \pi$, 因此 $p(C_2) = 1 - \pi$. 对于属于 C_1 类的数据点 $\boldsymbol{x}_n$, 我们有 $t_n = 1$, 因此

$$p(\boldsymbol{x}_n, C_1) = p(C_1)\, p(\boldsymbol{x}_n \mid C_1) = \pi\, \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}) \tag{5.71}$$

类似地, 对于 C_2 类, 我们有 $t_n = 0$, 因此

$$p(\boldsymbol{x}_n, C_2) = p(C_2)\, p(\boldsymbol{x}_n \mid C_2) = (1 - \pi)\, \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) \tag{5.72}$$

似然函数由下式给出:

$$p(\boldsymbol{t}, \boldsymbol{X} \mid \pi, \boldsymbol{\mu}_1, \boldsymbol{\mu}_2, \boldsymbol{\Sigma}) = \prod_{n=1}^{N} \left[\pi \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma})\right]^{t_n} \left[(1-\pi)\mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_2, \boldsymbol{\Sigma})\right]^{1-t_n} \tag{5.73}$$

其中 $\boldsymbol{t} = (t_1, \cdots, t_N)^{\mathsf{T}}$. 按照惯例, 最大化似然函数的对数是很方便的. 首先考虑关于变量 π 的最大值. 对数似然函数中取决于变量 π 的项为

$$\sum_{n=1}^{N} \{t_n \ln \pi + (1-t_n)\ln(1-\pi)\} \tag{5.74}$$

设变量 π 的导数为 0 并重新排列, 可得

$$\pi = \frac{1}{N}\sum_{n=1}^{N} t_n = \frac{N_1}{N} = \frac{N_1}{N_1 + N_2} \tag{5.75}$$

其中,N_1 表示 C_1 类数据点的总数,N_2 表示 C_2 类数据点的总数. 因此, 变量 π 的最大似然估计值就是预期中 C_1 类数据点的比例. 我们可以很容易地将这一结果扩展到多类情况, 在这种情况下, 与 C_k 类相关的先验概率的最大似然估计值同样是由归入该类的训练集数据点的比例计算得出的 (见练习 5.13).

下面考虑 $\boldsymbol{\mu}_1$ 的最大值. 同样, 我们可以从对数似然函数中挑出那些取决于 $\boldsymbol{\mu}_1$ 的项:

$$\sum_{n=1}^{N} t_n \ln \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_1, \boldsymbol{\Sigma}) = -\frac{1}{2}\sum_{n=1}^{N} t_n (\boldsymbol{x}_n - \boldsymbol{\mu}_1)^{\mathsf{T}} \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}_n - \boldsymbol{\mu}_1) + \text{const} \tag{5.76}$$

设 $\boldsymbol{\mu}_1$ 的导数为 0 并重新排列, 可得

$$\boldsymbol{\mu}_1 = \frac{1}{N_1}\sum_{n=1}^{N} t_n \boldsymbol{x}_n \tag{5.77}$$

此为归入 C_1 类的所有输入向量 $\boldsymbol{x}_n$ 的平均值. 经过类似的论证,$\boldsymbol{\mu}_2$ 的相应结果的计算公式为

$$\boldsymbol{\mu}_2 = \frac{1}{N_2}\sum_{n=1}^{N} (1-t_n)\boldsymbol{x}_n \tag{5.78}$$

此为归入 C_2 类的所有输入向量 $\boldsymbol{x}_n$ 的平均值.

最后, 考虑共享协方差矩阵 $\boldsymbol{\Sigma}$ 的最大似然解. 选出对数似然函数中依赖于 $\boldsymbol{\Sigma}$ 的项:

$$\begin{aligned} &-\frac{1}{2}\sum_{n=1}^{N} t_n \ln|\boldsymbol{\Sigma}| - \frac{1}{2}\sum_{n=1}^{N} t_n (\boldsymbol{x}_n - \boldsymbol{\mu}_1)^{\mathsf{T}} \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}_n - \boldsymbol{\mu}_1) \\ &\quad -\frac{1}{2}\sum_{n=1}^{N} (1-t_n) \ln|\boldsymbol{\Sigma}| - \frac{1}{2}\sum_{n=1}^{N} (1-t_n)(\boldsymbol{x}_n - \boldsymbol{\mu}_2)^{\mathsf{T}} \boldsymbol{\Sigma}^{-1} (\boldsymbol{x}_n - \boldsymbol{\mu}_2) \\ &= -\frac{N}{2}\ln|\boldsymbol{\Sigma}| - \frac{N}{2}\operatorname{tr}\{\boldsymbol{\Sigma}^{-1}\boldsymbol{S}\} \end{aligned} \tag{5.79}$$

其中

$$\boldsymbol{S} = \frac{N_1}{N}\boldsymbol{S}_1 + \frac{N_2}{N}\boldsymbol{S}_2 \tag{5.80}$$

$$\boldsymbol{S}_1 = \frac{1}{N_1}\sum_{n \in C_1} (\boldsymbol{x}_n - \boldsymbol{\mu}_1)(\boldsymbol{x}_n - \boldsymbol{\mu}_1)^{\mathsf{T}} \tag{5.81}$$

$$\boldsymbol{S}_2 = \frac{1}{N_2}\sum_{n \in C_2} (\boldsymbol{x}_n - \boldsymbol{\mu}_2)(\boldsymbol{x}_n - \boldsymbol{\mu}_2)^{\mathsf{T}} \tag{5.82}$$

利用 Gauss 分布最大似然解的标准结果, 我们可以得出 $\boldsymbol{\Sigma}=\boldsymbol{S}$, 它表示分别与两个类相关的协方差矩阵的加权平均值.

命题 5.15 (生成式模型的最大似然解)

设模型由(5.73)给出, 其中 $\mathcal{N}(\cdot\mid\boldsymbol{\mu}_k,\boldsymbol{\Sigma})$ 为均值 $\boldsymbol{\mu}_k$、协方差 $\boldsymbol{\Sigma}$(正定) 的 Gauss 密度,$0<\pi<1$. 则对数似然的驻点由(5.75)、(5.77)、(5.78)与 $\boldsymbol{\Sigma}=\boldsymbol{S}$ 给出, 其中 $\boldsymbol{S}$ 由(5.80)、(5.81)、(5.82)定义.

证明

(重构新增) 记 $N_1=\sum_n t_n, N_2=N-N_1$. 对数似然 (略去与参数无关的项) 为

$$\begin{aligned}\ell=&\sum_{n=1}^{N}\big[t_n\ln\pi+(1-t_n)\ln(1-\pi)\big]-\frac{N}{2}\ln|\boldsymbol{\Sigma}|\\&-\frac{1}{2}\sum_{n=1}^{N}t_n(\boldsymbol{x}_n-\boldsymbol{\mu}_1)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_n-\boldsymbol{\mu}_1)\\&-\frac{1}{2}\sum_{n=1}^{N}(1-t_n)(\boldsymbol{x}_n-\boldsymbol{\mu}_2)^{\mathsf{T}}\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}_n-\boldsymbol{\mu}_2)\end{aligned}\tag{5.83}$$

关于 π 求导并令其为零, 得 $N_1/\pi-N_2/(1-\pi)=0$, 即 $\pi=N_1/N$, 这就是(5.75). 关于 $\boldsymbol{\mu}_1$ 求导, 得 $\boldsymbol{\Sigma}^{-1}\sum_n t_n(\boldsymbol{x}_n-\boldsymbol{\mu}_1)=\boldsymbol{0}$, 左乘 $\boldsymbol{\Sigma}$ 并用 $N_1=\sum_n t_n$ 即得(5.77); 对 $\boldsymbol{\mu}_2$ 同理可得(5.78). 关于 $\boldsymbol{\Sigma}$ 求导, 用 $\partial\ln|\boldsymbol{\Sigma}|/\partial\boldsymbol{\Sigma}=\boldsymbol{\Sigma}^{-1}$ 与 $\partial\,\mathrm{tr}\{\boldsymbol{\Sigma}^{-1}\boldsymbol{S}\}/\partial\boldsymbol{\Sigma}=-\boldsymbol{\Sigma}^{-1}\boldsymbol{S}\boldsymbol{\Sigma}^{-1}$, 得

$$\frac{\partial\ell}{\partial\boldsymbol{\Sigma}}=-\frac{N}{2}\boldsymbol{\Sigma}^{-1}+\frac{N}{2}\boldsymbol{\Sigma}^{-1}\boldsymbol{S}\boldsymbol{\Sigma}^{-1}\tag{5.84}$$

令其为零并左乘 $\boldsymbol{\Sigma}$、右乘 $\boldsymbol{\Sigma}$, 得 $\boldsymbol{\Sigma}=\boldsymbol{S}$. 由于 ℓ 关于 $\boldsymbol{\Sigma}^{-1}$ 是凹的且驻点唯一, 上述解确为最大似然解.

□

注

(重构新增) 命题中各估计量都有直观含义:π 是类频率,$\boldsymbol{\mu}_k$ 是类内样本均值,$\boldsymbol{S}$ 是类内协方差的加权平均, 权重为类频率. 估计 $\boldsymbol{\Sigma}$ 时用的是"已知均值"的样本协方差 (分母为 N_1、N_2), 因为 $\boldsymbol{\mu}_k$ 已由同一批数据估计; 这与小节 3.2.7中一元 Gauss 情形一致. 条件 $0<\pi<1$ 保证对数的自变量为正, 也保证 $N_1,N_2>0$, 否则某个类的均值无定义. 与命题 5.13合起来看, 生成式方法先用训练数据估计 $O(KD^2)$ 量级的参数, 再由它们算出 $\boldsymbol{w}_k, w_{k0}$; 而小节 5.4.3将直接估计 $K(D+1)$ 个参数, 参数更少但不再能生成数据.

该结果可以很容易地扩展到多类问题中, 从而得出相应参数的最大似然解, 其中每个类-条件概率密度都是 Gauss 密度且具有共享协方差矩阵 (见练习 5.14). 请注意, 将 Gauss 分布拟合到类中的方法对离群值不具有鲁棒性, 因为 Gauss 分布的最大似然估计并不具有鲁棒性 (参见小节 5.1.4).

5.3.3 离散特征

接下来让我们看看离散特征值 x_i. 为简单起见, 我们首先考虑二进制特征值 $x_i\in\{0,1\}$, 然后讨论如何扩展到更一般的离散特征. 如果有 D 个输入, 那么一般的分布会对应该每个类的一张包含 2^D 个数字的表, 并且有 2^D-1 个自变量 (缘于求和约束). 由于这种情况会随着特征数量的增加而呈指数增长, 因此我们可以寻求一种限制性更强的表示方法. 在这里, 我们将做出朴素 Bayes 假设 (参见小节 11.2.4), 将特征值视为独立的并以 C_k 类为条件.

定义 5.20 (朴素 Bayes 假设)

设输入特征为 $\boldsymbol{x}=(x_1,\cdots,x_D)^{\mathsf{T}}$. 若对每个类 C_k, 各特征在给定 C_k 的条件下相互独立, 即

$$p(\boldsymbol{x}\mid C_k)=\prod_{i=1}^{D}p(x_i\mid C_k),\tag{5.85}$$

则称该假设为朴素 Bayes 假设 (naive Bayes assumption). ♣

注

(重构新增) 朴素 Bayes 假设把 D 个特征的条件联合密度化为 D 个一维密度的乘积, 从而使每类的参数量从指数级 2^D 降到线性级 D. 它的强度在于“以类为条件”独立, 而不要求特征在边缘上独立: 两个特征可以高度相关, 只要在给定类之后条件独立即可 (这与命题 5.9所用的条件独立性是同一类假设). 当特征实际上高度相关时, 该假设会使后验概率被过度自信地估计 (接近 0 或 1), 这是朴素 Bayes 分类器著名的局限; 但在许多文本分类任务中它仍然有效, 因为分类只需要后验的排序, 而不需要其精确数值. 注意假设的对象是 $p(\boldsymbol{x} \mid C_k)$ 而不是 $p(C_k \mid \boldsymbol{x})$.

于是, 我们可以得到如下形式的类-条件概率分布:

$$p(\boldsymbol{x} \mid C_k) = \prod_{i=1}^{D} \mu_{ki}^{x_i} (1-\mu_{ki})^{1-x_i} \tag{5.86}$$

其中包含每个类的 D 个独立参数. 代入(5.57), 可得

$$a_k(\boldsymbol{x}) = \sum_{i=1}^{D} \{x_i \ln \mu_{ki} + (1-x_i)\ln(1-\mu_{ki})\} + \ln p(C_k) \tag{5.87}$$

命题 5.16 (离散特征的线性预激活)

设 $p(\boldsymbol{x} \mid C_k)$ 由(5.86)给出, 其中 $0 < \mu_{ki} < 1$. 则(5.57)中的 a_k 可写成 $\boldsymbol{x}$ 的线性函数, 即(5.87), 其对 x_i 的系数为 $\ln[\mu_{ki}/(1-\mu_{ki})]$. ♠

证明

(重构新增) 由(5.86),

$$\ln p(\boldsymbol{x} \mid C_k) = \sum_{i=1}^{D} \{x_i \ln \mu_{ki} + (1-x_i)\ln(1-\mu_{ki})\} \tag{5.88}$$

将括号展开并按 x_i 整理:

$$\ln p(\boldsymbol{x} \mid C_k) = \sum_{i=1}^{D} x_i \ln \frac{\mu_{ki}}{1-\mu_{ki}} + \sum_{i=1}^{D} \ln(1-\mu_{ki}) \tag{5.89}$$

其中第一项是 $\boldsymbol{x}$ 的线性函数, 第二项与 $\boldsymbol{x}$ 无关. 加上 $\ln p(C_k)$ 即得(5.87). 由 $0 < \mu_{ki} < 1$, 各对数均有定义. □

注

(重构新增) 命题说明: 在朴素 Bayes 假设下, 无论特征的个数 D 多大, 后验概率的预激活都是线性的, 且参数只有 KD 个 (每个类、每个特征一个 μ_{ki}), 远小于完整表格所需的 $K(2^D-1)$ 个参数. 这就是“限制性更强的表示”的含义: 它用独立性换取了参数量的指数级下降. 条件 $0 < \mu_{ki} < 1$ 保证对数几率有限; 若某个 $\mu_{ki} = 0$ 或 1, 则相应的特征在该类中从不出现或总是出现, 系数发散, 此时需要平滑或把该特征视为确定性证据.

此为输入值 x_i 的线性函数. 对于二分类, 我们也可以考虑(5.51)给出的 logistic sigmoid 公式. 对于有 $L > 2$ 个状态的离散变量, 也可以得到类似的结果 (见练习 5.16).

5.3.4 指数族分布

正如我们所看到的, 对于符合 Gauss 分布且离散的输入, 类的后验概率都是由具有 logistic sigmoid($K = 2$) 或 softmax($K \geqslant 2$) 激活函数的广义线性模型得出的. 这些假设类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 都是如下公式给出的指数族分布 (参见节 3.4) 子集的成员, 从而得出的更为普遍结果的特殊情况.

$$p(\boldsymbol{x} \mid \boldsymbol{\lambda}_k, s) = \frac{1}{s} h\left(\frac{1}{s}\boldsymbol{x}\right) g(\boldsymbol{\lambda}_k) \exp\left\{\frac{1}{s}\boldsymbol{\lambda}_k^{\mathsf{T}}\boldsymbol{x}\right\} \tag{5.90}$$

在这里, 缩放参数 s 可在所有类之间共享.

命题 5.17 (指数族类-条件密度导出的广义线性后验)

设各类的类-条件密度由(5.90)给出, 其中 $\boldsymbol{\lambda}_k$ 为自然参数,g 为归一化函数, 缩放参数 $s > 0$ 对所有类相同. 则二分类情形的预激活 $a(\boldsymbol{x})$ 为 $\boldsymbol{x}$ 的线性函数, 即

$$a(\boldsymbol{x}) = (\boldsymbol{\lambda}_1 - \boldsymbol{\lambda}_2)^{\mathsf{T}}\boldsymbol{x} + \ln g(\boldsymbol{\lambda}_1) - \ln g(\boldsymbol{\lambda}_2) + \ln p(C_1) - \ln p(C_2), \tag{5.91}$$

从而 $p(C_1 \mid \boldsymbol{x}) = \sigma(a(\boldsymbol{x}))$; 多分类情形的预激活为

$$a_k(\boldsymbol{x}) = \boldsymbol{\lambda}_k^{\mathsf{T}}\boldsymbol{x} + \ln g(\boldsymbol{\lambda}_k) + \ln p(C_k), \tag{5.92}$$

从而后验概率由作用于线性函数的 softmax 变换给出.

♠

证明

(重构新增) 对二分类, 由(5.52),

$$a(\boldsymbol{x}) = \ln p(C_1) - \ln p(C_2) + \ln p(\boldsymbol{x} \mid \boldsymbol{\lambda}_1, s) - \ln p(\boldsymbol{x} \mid \boldsymbol{\lambda}_2, s) \tag{5.93}$$

由(5.90), 两个密度的 $h(\boldsymbol{x}/s)/s$ 因子相同, 相减时消去, 只剩

$$\ln p(\boldsymbol{x} \mid \boldsymbol{\lambda}_k, s) = \ln g(\boldsymbol{\lambda}_k) + \frac{1}{s}\boldsymbol{\lambda}_k^{\mathsf{T}}\boldsymbol{x} + \ln h\left(\frac{\boldsymbol{x}}{s}\right) - \ln s \tag{5.94}$$

两项相减,h 与 s 的项消去, 得

$$a(\boldsymbol{x}) = \frac{1}{s}(\boldsymbol{\lambda}_1 - \boldsymbol{\lambda}_2)^{\mathsf{T}}\boldsymbol{x} + \ln g(\boldsymbol{\lambda}_1) - \ln g(\boldsymbol{\lambda}_2) + \ln p(C_1) - \ln p(C_2) \tag{5.95}$$

将因子 $1/s$ 吸收进参数 (即用 $s\boldsymbol{\lambda}_k$ 代替 $\boldsymbol{\lambda}_k$, 这等价于重新标度自然参数), 即得(5.91). 多分类情形由(5.57)直接得到 $a_k = \ln p(C_k) + \ln p(\boldsymbol{x} \mid \boldsymbol{\lambda}_k, s)$, 其中与 k 无关的项在 softmax 中约去, 留下列(5.92).

□

注

(重构新增) 命题统一了前面两个特例:Gauss 密度 (固定协方差) 与 Bernoulli 乘积都是指数族的成员, 因此它们导出线性后验并非巧合, 而是指数族结构的结果.(5.90)中的 s 只影响参数的标度, 不影响“后验预激活是线性的”这一结论, 这正是可以在所有类之间共享 s 的原因; 正文中的(5.91)与(5.92)省略了 $1/s$, 相当于把 s 并入 $\boldsymbol{\lambda}_k$. 注意这里假设的是类-条件密度属于指数族, 而小节 5.4.6将假设目标变量的条件分布属于指数族, 两者的角色不同. 不是所有类-条件密度都能导出线性后验; 一旦密度比不是指数形式, 后验就可能出现非线性的预激活 (例如混合密度).

对于二分类问题, 将类-条件概率密度的表达式代入(5.52), 我们发现类的后验概率由作用于线性函数 $a(\boldsymbol{x})$ 的 logistic sigmoid 函数给出, 其中 $a(\boldsymbol{x})$ 即(5.91). 同样, 对于多分类问题 (简称多类问题), 可将类-条件概率密度的表达式代入(5.57), 得到(5.92), 此为 $\boldsymbol{x}$ 的线性函数.

5.4 判别分类器

对于二分类情况, 我们已经看到, 针对指数族分布中多种类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 的选择问题,C_1 类的后验概率可以写成作用于 $\boldsymbol{x}$ 的线性函数的 logistic sigmoid 函数. 同样, 对于多分类情况,C_k 类的后验概率由作用于 $\boldsymbol{x}$ 的线性函数的 softmax 变换给出. 对于类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 的具体选择, 我们首先使用最大似然法确定密度参数和类的先验概率 $p(C_k)$, 然后使用 Bayes 定理求出类的后验概率. 此为生成式模型的一个例子, 我们可以利用该模型, 通过从边缘分布 $p(\boldsymbol{x})$ 或任何类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 中取 $\boldsymbol{x}$ 值而生成合成数据.

不过, 还有一种方法是明确使用广义线性模型的函数形式并通过最大似然法直接确定其参数. 在这种直接方法中, 我们将似然函数 (由条件分布 $p(C_k \mid \boldsymbol{x})$ 定义) 最大化, 此为一种判别概率建模形式. 我们很快就能看到, 判别法的一个优点是, 需要确定的可学习参数通常较少. 此外, 判别法还可以提高预测性能, 尤其当类-条件概率密度的

假定形式与真实分布的近似程度较低时.

5.4.1 激活函数

在线性回归 (参见节 4.1) 中, 模型预测值 $y(\boldsymbol{x}, \boldsymbol{w})$ 由参数的线性函数给出:

$$y(\boldsymbol{x}, \boldsymbol{w}) = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0 \tag{5.96}$$

它给出了范围为 $(-\infty, \infty)$ 的连续值输出. 然而, 对于分类问题, 我们希望预测离散类标签, 或者更一般地说, 预测 $(0, 1)$ 范围内的后验概率. 为此, 我们考虑对这一模型进行泛化——用非线性函数 $f(\cdot)$ 对 $\boldsymbol{w}$ 和 w_0 的线性函数进行转换, 从而得到

$$y(\boldsymbol{x}, \boldsymbol{w}) = f\left(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0\right) \tag{5.97}$$

定义 5.21 (激活函数与连接函数)

在(5.97)中, $f(\cdot)$ 在机器学习文献中称为激活函数 (activation function), 它的反函数 $f^{-1}(\cdot)$ 在统计学文献中则称为连接函数 (link function).

♣

注

(重构新增) 激活函数的作用是把无界的线性输出 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0$ 压缩到后验概率所需的区间 $(0, 1)$ 内; 连接函数则是从概率出发的同一对象: 它把均值 y 映回线性预激活. 两者互为反函数, 因此“选激活函数”与“选连接函数”是同一件事的两种说法. f 必须是单调的, 否则同一个 y 可能对应多个预激活, 决策面将不再是单个超平面. f 的值域必须是 $(0, 1)$ 的子集, 否则输出不能解释为概率; 若允许端点, 则需额外约定. 注意 f 的非线性不改变决策面的形状: 决策面仍由 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0 =$ 常数 给出, 因为 f 单调 (见下面的讨论).

决策面对应于 $y(\boldsymbol{x}) =$ 常数, 因此 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} =$ 常数, 决策面是 $\boldsymbol{x}$ 的线性函数, 即使函数 $f(\cdot)$ 是非线性的. 因此,(5.97)描述的这一类模型名为广义线性模型 (McCullagh and Nelder, 1989).

定义 5.22 (广义线性模型)

若模型对输入 $\boldsymbol{x}$ 的依赖只通过线性组合 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0$ 体现, 而输出由非线性 (单调) 函数 f 作用于该线性组合得到, 即具有(5.97)的形式, 则称它为广义线性模型 (generalized linear model).

♣

注

(重构新增) “广义线性”中的“线性”指线性预激活, “广义”指允许 f 非线性; 因此它既不同于线性回归 (f 为恒等映射), 也不同于一般非线性模型 (f 的自变量不是线性的). 这种结构带来两个后果: 其一, 决策面仍是超平面 (因为 f 单调, y 的等值面对应 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x} + w_0$ 的等值面); 其二, 对参数的依赖是非线性的, 故不能像线性最小二乘那样用一次线性方程求解, 必须用迭代方法 (见小节 5.4.3). 条件“f 单调”不是定义的字面要求, 而是使决策面为超平面所必需的; 若 f 不单调, $y =$ 常数 可能对应多个超平面.

然而, 与用于回归的模型相比, 由于非线性函数 $f(\cdot)$ 的存在, 这些模型的参数不再是线性的. 这将导致比线性回归模型更复杂的分析和计算特性. 不过, 与更为灵活的非线性模型 (后续章节将对它们展开研究) 相比, 这些模型仍然相对简单.

5.4.2 固定基函数

前面介绍了直接使用原始输入向量 $\boldsymbol{x}$ 的分类模型. 不过, 如果我们首先使用基函数 $\boldsymbol{\phi}(\boldsymbol{x})$ 的向量对输入进行固定的非线性变换, 那么所有算法都同样适用. 如图 5.15所示, 由此产生的决策边界在特征空间 $\boldsymbol{\phi}$ 中是线性的, 与原始观测空间 $\boldsymbol{x}$ 中的非线性决策边界相对应. 特征空间 $\boldsymbol{\phi}$ 中可线性分离的类在原始观测空间 $\boldsymbol{x}$ 中不必线性可分.

定义 5.23 (固定基函数模型)

设 $\boldsymbol{\phi}(\boldsymbol{x}) = (\phi_1(\boldsymbol{x}), \cdots, \phi_M(\boldsymbol{x}))^{\mathsf{T}}$ 是给定的、不随数据改变的非线性变换. 若分类模型只通过线性组合 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}(\boldsymbol{x}) + w_0$ 使用输入, 则称它为固定基函数模型 (fixed basis function model),ϕ_i 称为基函数. ♣

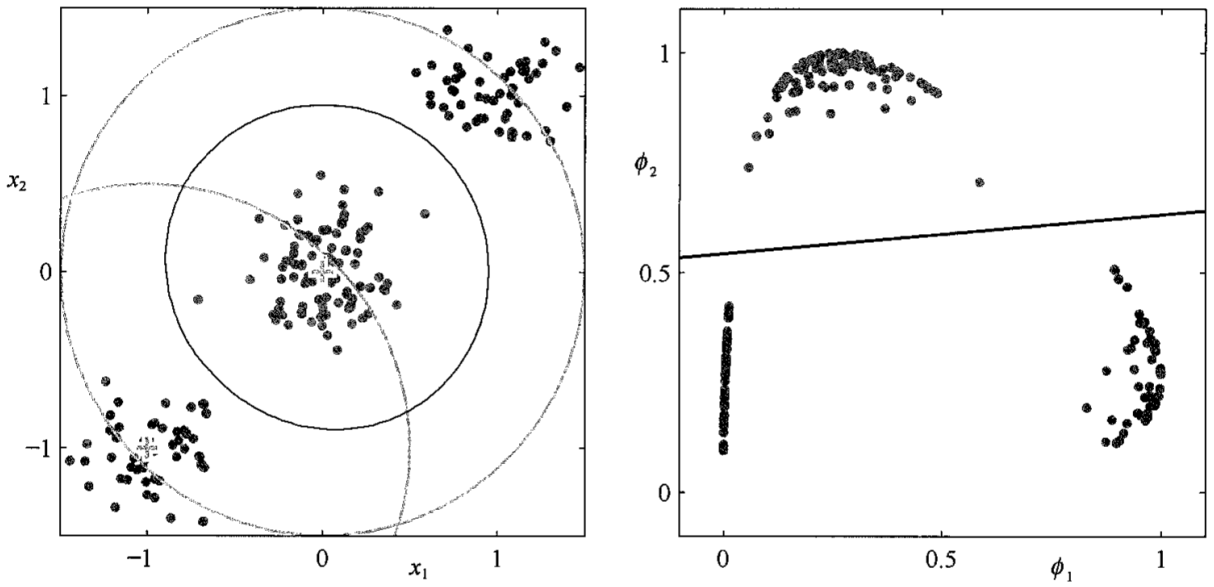


图 5.15: 非线性基函数在线性分类模型中发挥作用的示意图. 左图展示了原始输入空间 (x_1, x_2) 以及两个类 (标记为红色和蓝色) 中的数据点. 此空间中定义了两个 "Gauss" 基函数 $\phi_1(\boldsymbol{x})$ 和 $\phi_2(\boldsymbol{x})$, 中心用绿色十字表示, 轮廓用绿色圆圈表示. 右图展示了相应的特征空间 (ϕ_1, ϕ_2), 以及由小节 5.4.3所述形式的 logistic 回归模型得到的线性决策边界. 这与原始输入空间中的非线性决策边界相对应, 如左图中的黑色曲线所示

请注意, 在线性回归模型中, 有一个基函数通常设为常数, 例如 $\phi_0(\boldsymbol{x}) = 1$, 因此相应的参数 w_0 起到偏置的作用.

注

(重构新增) 固定基函数模型的意义在于: 线性分类算法的全部理论 (线性决策面、凸决策区域、最大似然) 都可以原样搬到一个新的特征空间中, 从而在原始空间中产生非线性边界. 代价是基函数的个数 M 与形式必须事先选定: 若 M 太小则表达能力不足, 若 M 太大则参数增多、容易过拟合.图 5.15中, 左图用两个径向基函数把二维输入映到二维特征空间, 右图中的直线在左图中对应一条闭曲线, 正好把中心的一类与外围的另一类分开. 条件 "基函数固定、不随数据改变" 是关键: 一旦让基函数依赖于数据或参数, 模型就不再是广义线性的, 而成为后续章节讨论的神经网络.

在许多实际问题中, 类-条件概率密度 $p(\boldsymbol{x} \mid C_k)$ 在 $\boldsymbol{x}$ 空间中存在明显的重叠. 这相当于后验概率 $p(C_k \mid \boldsymbol{x})$, 至少对于 $\boldsymbol{x}$ 的某些值, 后验概率 $p(C_k \mid \boldsymbol{x})$ 不是 0 或 1. 在这种情况下, 通过对后验概率进行精确建模并应用标准决策理论 (参见节 5.2), 便可得到最优解. 请注意, 非线性变换 $\phi_k(\boldsymbol{x})$ 并不能消除这种重叠, 尽管它们可以增加重叠程度, 或在原始观测空间中不存在重叠的地方产生重叠. 非线性的适当选择可以使后验概率建模过程变得更容易. 然而, 这种固定的基函数模型有很大的局限性 (参见节 6.1), 在后续章节中, 我们将通过让基函数适应数据来解决这些问题.

5.4.3 logistic 回归

首先考虑二分类问题. 在节 5.3关于生成方法的讨论中, 我们看到了在一般假设条件下,C_1 的后验概率可以写成作用于特征空间 $\boldsymbol{\phi}$ 的线性函数的 logistic sigmoid 函数, 从而得到

$$p(C_1 \mid \boldsymbol{\phi}) = y(\boldsymbol{\phi}) = \sigma\left(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}\right) \tag{5.98}$$

其中 $p(C_2 \mid \boldsymbol{\phi}) = 1 - p(C_1 \mid \boldsymbol{\phi})$ 由定义 5.16定义的 logistic sigmoid 函数给出.

定义 5.24 (logistic 回归模型)

若二分类模型的后验概率取(5.98)的形式, 其中 σ 为 logistic sigmoid 函数、$\boldsymbol{\phi} = \boldsymbol{\phi}(\boldsymbol{x})$ 为特征向量, 则称该模型为 logistic 回归 (logistic regression) 模型. ♣

注

(重构新增) 名称中的“回归”是历史遗留: 它来自统计学中对二值响应变量做广义线性建模的传统, 但模型输出的是类后验概率, 因此它实际上是一个分类模型, 而不是连续变量模型. 这一名称容易引起误解, 阅读时应以“输出为 $(0,1)$ 内的概率、决策面为超平面”为准. 模型的可调参数是 $\boldsymbol{w}$(若含偏置则增加一个分量), 共 M 个 (M 为特征维数). 与生成式方法相比,logistic 回归不假设 $\boldsymbol{\phi}$ 的分布, 因此对类-条件密度的假设错误不敏感; 但它也不能生成数据, 不能直接用于离群值检测.

对于 M 维特征空间 $\boldsymbol{\phi}$, 该模型有 M 个可调参数. 相比之下, 如果我们使用最大似然法拟合 Gauss 类-条件概率密度, 那么我们将使用 $2M$ 个参数表示均值, 并使用 $M(M+1)/2$ 个参数表示 (共享) 协方差矩阵. 再加上类的先验概率 $p(C_1)$, 总共有 $M(M+5)/2+1$ 个参数. 与 logistic 回归模型中参数数量与 M 成线性关系不同, 该模型的参数数量随 M 呈二次增长. 对于较大的 M 值, 直接使用 logistic 回归模型有明显的优势.

下面我们使用最大似然法确定 logistic 回归模型的参数. 为此, 我们将利用 logistic sigmoid 函数的导数, 它可以方便地用 sigmoid 函数本身来表示 (见练习 5.18):

$$\frac{\mathrm{d}\sigma}{\mathrm{d}a} = \sigma(1-\sigma) \tag{5.99}$$

命题 5.18 (logistic sigmoid 函数的导数)

设 $\sigma(a) = 1/(1+\exp(-a))$, 则 σ 可微且 $\sigma'(a) = \sigma(a)\big(1-\sigma(a)\big)$ 对一切 $a \in \mathbb{R}$ 成立. ♠

证明

(重构新增) 由 $\sigma(a) = (1+e^{-a})^{-1}$, 用链式法则:

$$\sigma'(a) = -(1+e^{-a})^{-2}\cdot(-e^{-a}) = \frac{e^{-a}}{(1+e^{-a})^2} = \frac{1}{1+e^{-a}}\cdot\frac{e^{-a}}{1+e^{-a}} = \sigma(a)\big(1-\sigma(a)\big) \tag{5.100}$$

其中最后一个等号用了 $1-\sigma(a) = e^{-a}/(1+e^{-a})$. 由于 $e^{-a}>0$, 右端恒为正, 故 σ 严格单调递增.

□

注

(重构新增) $\sigma' = \sigma(1-\sigma)$ 是 logistic 回归计算上便利的根本原因: 交叉熵的导数中会出现因子 $y_n(1-y_n)$, 正好被这个恒等式消去, 从而得到命题 5.19中“误差乘特征”的简洁形式. 该恒等式也说明 $\sigma' \leqslant 1/4$(因为 $\sigma(1-\sigma)$ 在 $\sigma = 1/2$ 处取最大值 1/4), 因此 σ 是 Lipschitz 常数为 1/4 的压缩映射, 这保证了梯度下降的稳定性. 注意 σ' 在 $|a|$ 很大时趋于 0, 这就是“梯度饱和”现象的来源.

对于数据集 $\{\boldsymbol{\phi}_n, t_n\}$, 其中 $\boldsymbol{\phi}_n = \boldsymbol{\phi}(\boldsymbol{x}_n)$, 且 $t_n \in \{0,1\}$,$n = 1,\cdots,N$. 似然函数可写成

$$p(\boldsymbol{t} \mid \boldsymbol{w}) = \prod_{n=1}^{N} y_n^{t_n}(1-y_n)^{1-t_n} \tag{5.101}$$

其中 $\boldsymbol{t} = (t_1,\cdots,t_N)^{\mathsf{T}}$, 且 $y_n = p(C_1 \mid \boldsymbol{\phi}_n)$. 按照惯例, 我们可以通过取似然的负对数来定义误差函数, 从而得到交叉熵 (cross-entropy) 误差函数:

$$E(\boldsymbol{w}) = -\ln p(\boldsymbol{t} \mid \boldsymbol{w}) = -\sum_{n=1}^{N}\big\{t_n \ln y_n + (1-t_n)\ln(1-y_n)\big\} \tag{5.102}$$

其中,$y_n = \sigma(a_n)$ 且 $a_n = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n$. 利用误差函数关于 $\boldsymbol{w}$ 的梯度, 可得 (见练习 5.19)

$$\nabla E(\boldsymbol{w}) = \sum_{n=1}^{N}(y_n - t_n)\boldsymbol{\phi}_n \tag{5.103}$$

命题 5.19 (交叉熵误差的梯度)

设 $E(\boldsymbol{w})$ 由(5.102)定义,$y_n = \sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}_n)$. 则 E 可微, 且其梯度由(5.103)给出.

证明

(重构新增) 记 $a_n = \boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}_n$. 由命题 5.18,

$$\frac{\partial E}{\partial a_n} = -\frac{t_n}{y_n}y_n(1-y_n) + \frac{1-t_n}{1-y_n}y_n(1-y_n) = -t_n(1-y_n) + (1-t_n)y_n = y_n - t_n \tag{5.104}$$

再由 $\partial a_n/\partial \boldsymbol{w} = \boldsymbol{\phi}_n$ 与链式法则,$\nabla E = \sum_n(\partial E/\partial a_n)\boldsymbol{\phi}_n = \sum_n(y_n - t_n)\boldsymbol{\phi}_n$, 即(5.103).

□

注

(重构新增) 命题中的推导显示:sigmoid 的导数因子 $y_n(1-y_n)$ 在分子与分母中同时出现而被约去, 这正是 logistic 回归“梯度形式简单”的原因. 若把交叉熵换成平方误差 $E = \frac{1}{2}\sum(y_n - t_n)^2$, 则梯度中会残留因子 $y_n(1-y_n)$, 当 y_n 接近 0 或 1 时梯度消失, 收敛变慢, 这就是分类问题不宜用平方误差的理论依据. 梯度的形式 $\sum_n(y_n - t_n)\boldsymbol{\phi}_n$ 与线性回归中平方和误差的梯度形式完全相同 (见eq. (4.12)), 差别只在于 y_n 是 $\boldsymbol{w}$ 的非线性函数.$\nabla E = 0$ 等价于 $\sum_n y_n\boldsymbol{\phi}_n = \sum_n t_n\boldsymbol{\phi}_n$, 即“预测的充分统计量等于观测的充分统计量”.

其中, 我们使用了(5.99). 你可以看到, 涉及 logistic sigmoid 导数的因子已被消去, 从而简化了对数似然梯度的形式. 具体来说, 数据点 n 对梯度的贡献由目标值与模型预测值之间的“误差” $(y_n - t_n)$ 乘以 $\boldsymbol{\phi}_n$ 给出. 此外, 通过与eq. (4.12)进行比较, 我们发现这与线性回归模型的平方和误差函数梯度的形式完全相同.

最大似然解 (参见小节 4.1.3) 对应于 $\nabla E(\boldsymbol{w}) = 0$. 然而, 从(5.103)中可以看出, 由于 $y(\cdot)$ 的非线性, 这不再对应于线性方程组, 因此没有闭式解.

注

(重构新增) “没有闭式解”的确切含义是: 驻点方程 $\sum_n\big(\sigma(\boldsymbol{w}^\mathsf{T}\boldsymbol{\phi}_n) - t_n\big)\boldsymbol{\phi}_n = \boldsymbol{0}$ 中, 未知量 $\boldsymbol{w}$ 出现在 sigmoid 的自变量内, 因而它不是关于 $\boldsymbol{w}$ 的线性方程组, 不能像(5.17)那样用一次矩阵求逆写出解. 这并不意味着解不存在 (凸性保证解存在, 见命题 5.20), 而是说不存在由有限次四则运算与初等函数组成的通用表达式. 数值上可用 Newton 法 (即 IRLS) 或梯度下降求解.

求最大似然解的一种方法是使用随机梯度下降法 (参见第 7 章), 其中 ∇E_n 是(5.103)右侧的第 n 项. 随机梯度下降法是训练高度非线性神经网络 (后续章节将展开讨论) 的主要方法. 然而, 最大似然方程只是“略微”非线性的. 事实上, 由(5.98)定义的模型, 其误差函数(5.102)是参数的凸函数, 这使得误差函数可以通过一种称为迭代重加权最小二乘法 (Iterative Reweighted Least Squares, IRLS) 的简单算法来最小化 (Bishop, 2006). 然而, 这很难泛化到更复杂的模型, 如深度神经网络.

命题 5.20 (交叉熵误差的凸性)

设 $E(\boldsymbol{w})$ 由(5.102)定义. 则 E 是 $\boldsymbol{w}$ 的凸函数; 若 $\{\boldsymbol{\phi}_n\}$ 张成整个参数空间, 则 E 还是严格凸的.

证明

(重构新增) 由 $y_n = \sigma(a_n)$, 把(5.102)改写为

$$E(\boldsymbol{w}) = \sum_{n=1}^{N} \{\ln(1 + e^{a_n}) - t_n a_n\} \tag{5.105}$$

其中用到 $-\ln\sigma(a) = \ln(1+e^{-a})$ 与 $-\ln(1-\sigma(a)) = \ln(1+e^{a})$. 对 a_n 求二阶导:$\dfrac{\mathrm{d}^2}{\mathrm{d}a_n^2}\ln(1+e^{a_n}) = \sigma(a_n)\big(1-\sigma(a_n)\big) \geqslant 0$, 而 $-t_n a_n$ 关于 a_n 是线性的, 二阶导为零. 于是

$$\nabla^2 E(\boldsymbol{w}) = \sum_{n=1}^{N} \sigma(a_n)\big(1-\sigma(a_n)\big)\boldsymbol{\phi}_n \boldsymbol{\phi}_n^{\mathsf{T}} \succeq \mathbf{0} \tag{5.106}$$

即 Hessian 半正定, 故 E 凸. 若 $\{\boldsymbol{\phi}_n\}$ 张成整个空间, 则对任意 $\boldsymbol{v} \neq \mathbf{0}$ 存在 n 使 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{\phi}_n \neq 0$; 又 $0 < \sigma(1-\sigma)$, 故 $\boldsymbol{v}^{\mathsf{T}}\nabla^2 E\boldsymbol{v} > 0$,Hessian 正定,$E$ 严格凸.

□

注

(重构新增) 凸性保证任何局部极小都是全局极小, 因此梯度下降不会陷入局部最优; 严格凸性还保证极小点唯一(若存在). 证明的关键恒等式是“logistic 损失的二阶导等于方差 $\sigma(1-\sigma)$”, 这也是 IRLS 中“权重”的来源: 每步用当前 $y_n(1-y_n)$ 作为样本权重做加权最小二乘. 注意凸性是关于 $\boldsymbol{w}$ 的, 而不是关于 y 的;$\boldsymbol{\phi}_n$ 固定时结论成立, 若基函数依赖于 $\boldsymbol{w}$(如神经网络), 凸性一般不再成立, 这正是深度网络优化更困难的原因之一.

请注意, 对于线性可分的数据集, 最大似然法可能会表现出严重的过拟合. 这是因为当 $\sigma = 0.5$(相当于 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi} = 0$) 所对应的超平面将两个类分开, 且 $\boldsymbol{w}$ 的大小趋于无穷时, 就会出现最大似然解. 在这种情况下,logistic sigmoid 函数在特征空间中变得无限陡峭, 相当于一个单位阶跃函数, 因此来自每个类的每个训练点都被赋予一个后验概率 $p(C_k \mid \boldsymbol{x}) = 1$(见练习 5.20). 此外, 这样的解通常是连续的, 因为任何分离超平面在训练点处都会产生相同的后验概率. 最大似然法无法提供最优解, 在实践中找到那个解将取决于优化算法的选择和参数是如何初始化的. 注意, 只要训练数据集是线性可分的, 即使数据点的数量相比模型中参数的数量很大, 问题也会出现. 在误差函数中加入正则化项 (参见第 9 章) 可以避免奇异解.

命题 5.21 (线性可分数据的最大似然问题无有限解)

设训练集线性可分, 即存在 $\boldsymbol{w}^*$ 使 $t_n = 1$ 时 $(\boldsymbol{w}^*)^{\mathsf{T}}\boldsymbol{\phi}_n > 0$、$t_n = 0$ 时 $(\boldsymbol{w}^*)^{\mathsf{T}}\boldsymbol{\phi}_n < 0$. 则似然(5.101)在 $\boldsymbol{w}$ 上的上确界为 1, 但任何有限的 $\boldsymbol{w}$ 都取不到该值; 特别地, 最大似然问题没有有限解, 且沿 $\boldsymbol{w} = c\boldsymbol{w}^*(c \to \infty)$ 的任一方向都趋向该上确界.

♠

证明

(重构新增) 对任意有限的 $\boldsymbol{w}$ 与任意 n,$a_n = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n$ 有限, 故 $y_n = \sigma(a_n) \in (0,1)$ 且 $1 - y_n \in (0,1)$. 于是(5.101)的每个因子都在 $(0,1)$ 内, 乘积严格小于 1, 即任何有限 $\boldsymbol{w}$ 处的似然都小于 1.

另一方面, 令 $\boldsymbol{w} = c\boldsymbol{w}^*$,$c > 0$. 对 $t_n = 1$ 的数据点,$a_n = c(\boldsymbol{w}^*)^{\mathsf{T}}\boldsymbol{\phi}_n \to +\infty$, 故 $y_n = \sigma(a_n) \to 1$; 对 $t_n = 0$ 的数据点,$a_n \to -\infty$, 故 $1 - y_n = \sigma(-a_n) \to 1$. 于是(5.101)的每个因子都趋于 1, 有限个因子的乘积也趋于 1(N 固定). 因此似然的上确界为 1 且不可达, 最大似然解不存在; 又因该论证对任何使数据分开的方向 $\boldsymbol{w}^*$ 都成立, 故这样的方向构成一个连续统.

□

注

(重构新增) 命题说明“过拟合”在这里表现为参数发散而非某个有限的坏解: 模型对训练点的后验概率都趋于 1, 训练误差为零, 但 $\|\boldsymbol{w}\| \to \infty$ 使决策面在特征空间中变得极陡, 任何靠近训练点的测试点都会被赋予接近 0 或 1 的概率, 泛化性能由 $\boldsymbol{w}$ 的初始值与优化路径决定. 条件“线性可分”不可去掉: 若数据不可分, 则最优 $\boldsymbol{w}$ 有限, 似然的最大值在驻点处取到 (由命题 5.20, 该驻点唯一). 加入正则化项 $\lambda\|\boldsymbol{w}\|^2$(见第 9 章) 后, 目标函数变为强凸, 即使数据线性可分也存在唯一有限解.

5.4.4 多类 logistic 回归

在讨论用于多分类的生成式模型 (参见节 5.3) 时, 我们已经看到, 对于指数族分布, 后验概率是由作用于特征变量的线性函数的 softmax 变换给出的, 于是有

$$p(C_k \mid \boldsymbol{\phi}) = y_k(\boldsymbol{\phi}) = \frac{\exp(a_k)}{\sum_j \exp(a_j)} \tag{5.107}$$

其中, 预激活 a_k 为

$$a_k = \boldsymbol{w}_k^{\mathsf{T}}\boldsymbol{\phi} \tag{5.108}$$

此处, 我们使用最大似然法分别确定了类-条件概率密度和类的先验, 然后使用 Bayes 定理找到了相应的后验概率, 从而隐式地确定了参数 $\{\boldsymbol{w}_k\}$. 下面考虑使用最大似然法直接确定参数 $\{\boldsymbol{w}_k\}$. 为此, 我们需要 y_k 相对于所有预激活 a_j 的导数, 计算公式为 (见练习 5.21)

$$\frac{\partial y_k}{\partial a_j} = y_k\left(I_{kj} - y_j\right) \tag{5.109}$$

其中,I_{kj} 是单位矩阵的元素.

命题 5.22 (softmax 函数的导数)

设 $y_k = e^{a_k}/\sum_j e^{a_j}, k = 1, \cdots, K$. 则 $\dfrac{\partial y_k}{\partial a_j} = y_k(I_{kj} - y_j)$, 其中 I_{kj} 为 $K \times K$ 单位矩阵的元素.

证明

(重构新增) 记 $S = \sum_j e^{a_j}$, 则 $y_k = e^{a_k}/S$. 当 $j \neq k$ 时,e^{a_k} 与 S 对 a_j 的偏导分别为 0 与 e^{a_j}, 故

$$\frac{\partial y_k}{\partial a_j} = -\frac{e^{a_k}e^{a_j}}{S^2} = -y_k y_j = y_k(0 - y_j) \tag{5.110}$$

当 $j = k$ 时,e^{a_k} 对 a_k 的偏导为 e^{a_k}, 故

$$\frac{\partial y_k}{\partial a_k} = \frac{e^{a_k}S - e^{a_k}e^{a_k}}{S^2} = y_k - y_k^2 = y_k(1 - y_k) \tag{5.111}$$

两式合并即得 $\partial y_k/\partial a_j = y_k(I_{kj} - y_j)$.

□

注

(重构新增) 该导数矩阵 $\partial y/\partial a$ 是 softmax 的 Jacobian, 它是半正定的 (对任意 $\boldsymbol{v}$,$\boldsymbol{v}^{\mathsf{T}}\partial y/\partial a\,\boldsymbol{v} = \sum_k y_k v_k^2 - (\sum_k y_k v_k)^2 \geqslant 0$, 即方差非负).$K = 2$ 时令 $a = a_1 - a_2$, 该式退化为 $\sigma'(a) = \sigma(1-\sigma)$, 与命题 5.18一致. 由于 softmax 具有平移不变性,Jacobian 的零空间至少包含全 1 向量, 这与多类模型参数的冗余一致 (把所有权重同时加上同一个向量不改变模型).

接下来让我们写下似然函数. 使用 1-of-K 编码方案最容易实现这一目标, 在这种编码方案下, 属于 C_k 类的 $\boldsymbol{\phi}_n$ 的目标向量 $\boldsymbol{t}_n$ 是一个二进制向量, 除元素 k 等于 1 外, 其他元素均为 0. 似然函数为

$$p(\boldsymbol{T} \mid \boldsymbol{w}_1, \cdots, \boldsymbol{w}_K) = \prod_{n=1}^{N}\prod_{k=1}^{K} p(C_k \mid \boldsymbol{\phi}_n)^{t_{nk}} = \prod_{n=1}^{N}\prod_{k=1}^{K} y_{nk}^{t_{nk}} \tag{5.112}$$

其中 $y_{nk} = y_k(\boldsymbol{\phi}_n)$, 且 $\boldsymbol{T}$ 是一个包含 $N \times K$ 个目标变量的矩阵, 其中的元素为 t_{nk}. 取负对数可得

$$E(\boldsymbol{w}_1, \cdots, \boldsymbol{w}_K) = -\ln p(\boldsymbol{T} \mid \boldsymbol{w}_1, \cdots, \boldsymbol{w}_K) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk} \ln y_{nk} \tag{5.113}$$

此为多分类问题的交叉熵误差函数.

下面求误差函数相对于参数向量 $\boldsymbol{w}_j$ 的梯度. 利用(5.109)对 softmax 函数求导可得 (见练习 5.22)

$$\nabla_{\boldsymbol{w}_j} E(\boldsymbol{w}_1, \cdots, \boldsymbol{w}_K) = \sum_{n=1}^{N} \left(y_{nj} - t_{nj}\right) \boldsymbol{\phi}_n \tag{5.114}$$

命题 5.23 (多类交叉熵误差的梯度)

设 E 由(5.113)定义,y_{nk} 由(5.107)与(5.108)给出, 且每个 $\boldsymbol{t}_n$ 满足 $\sum_k t_{nk} = 1$. 则 E 关于 $\boldsymbol{w}_j$ 的梯度由(5.114)给出.

证明

(重构新增) 由链式法则与(5.109),

$$\frac{\partial E}{\partial a_{nj}} = -\sum_{k=1}^{K} \frac{t_{nk}}{y_{nk}} \frac{\partial y_{nk}}{\partial a_{nj}} = -\sum_{k=1}^{K} \frac{t_{nk}}{y_{nk}} y_{nk}\left(I_{kj} - y_{nj}\right) = -\sum_{k=1}^{K} t_{nk} I_{kj} + y_{nj}\sum_{k=1}^{K} t_{nk} = -t_{nj} + y_{nj} \tag{5.115}$$

其中最后一步用了 $\sum_k t_{nk} = 1$. 又 $a_{nj} = \boldsymbol{w}_j^{\mathsf{T}}\boldsymbol{\phi}_n$, 故 $\partial a_{nj}/\partial \boldsymbol{w}_j = \boldsymbol{\phi}_n$, 于是 $\nabla_{\boldsymbol{w}_j} E = \sum_n (y_{nj} - t_{nj})\boldsymbol{\phi}_n$.

□

注

(重构新增) 命题与二分类的命题 5.19形式完全一致, 这不是巧合:softmax 与交叉熵的配对使 sigmoid 的导数因子被约去, 从而每个参数只“感受”到本类的误差. 条件 $\sum_k t_{nk} = 1$ 在推导中用于把 $\sum_k t_{nk} I_{kj}$ 化为 t_{nj}; 对 1-of-K 编码它自动成立, 若目标不是 1-of-K(例如软标签), 该条件仍成立但 t_{nk} 可以取 $(0,1)$ 内的值. 多类模型中参数存在冗余: 把每个 $\boldsymbol{w}_j$ 同时加上同一向量 $\boldsymbol{v}$ 不改变任何 a_{nj} 的差, 故最优解不唯一; 实践中通常靠正则化或固定某个参考类来消除冗余. 原书(5.116)左端对 w_{ij} 求导, 而右端误差的下标写作 k, 这是同一对象的两种记法 (w_{ij} 把第 i 个基函数连接到第 k 个输出单元), 阅读时以右端的 k 为准.

其中, 我们利用了 $\sum_k t_{nk} = 1$. 同样, 我们可以利用随机梯度下降法 (参见第 7 章) 来优化参数.

我们再次发现, 梯度的形式与线性模型的平方和误差函数和 logistic 回归模型的交叉熵误差函数的形式相同, 即误差 $(y_{nj} - t_{nj})$ 与基函数激活度 $\boldsymbol{\phi}_n$ 的乘积. 我们稍后将探讨更普遍的例子 (参见小节 5.4.6).

线性分类模型可表示为单层神经网络, 如图 5.16所示. 如果我们使用误差函数相对于权重 w_{ik} 的导数 (以便将基函数 $\phi_i(\boldsymbol{x})$ 与输出单元 t_k 连接起来), 则可以得出

$$\frac{\partial E(\boldsymbol{w}_1,\cdots,\boldsymbol{w}_K)}{\partial w_{ij}} = \sum_{n=1}^{N}(y_{nk}-t_{nk})\,\phi_i(\boldsymbol{x}_n) \tag{5.116}$$

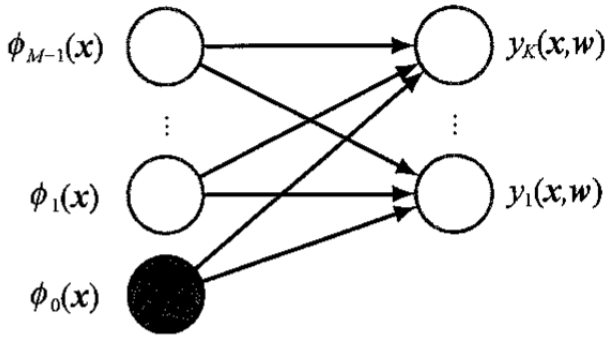


图 5.16: 将线性分类模型表示为具有单层连接的神经网络. 每个基函数由一个节点表示, 实心节点表示“偏置”基函数 ϕ_0, 而每个输出 $y_1,\cdots,y_K$ 也由一个节点表示. 节点之间的连接代表相应的权重和偏置参数

与图 5.16相比, 我们可以看到, 对于每个数据点 n, 梯度的形式是权重链路输入端的基函数输出与输出端“误差” $(y_{nk}-t_{nk})$ 的乘积.

5.4.5 probit 回归

我们已经看到, 对于各种类-条件分布, 得到的后验类率 (posterior class probability) 是由作用于特征变量线性函数的 logistic(或 softmax) 变换给出的. 然而, 并不是类-条件概率密度的所有选择都能为后验概率生成如此简单的形式, 这表明也许值得探索其他类型的判别概率模型. 在广义线性模型的框架内, 再次考虑二分类情况, 得到

$$p(t=1\mid a) = f(a) \tag{5.117}$$

其中,$a=\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}$ 且 $f(\cdot)$ 为激活函数.

激发替代连接函数的一种方法是使用噪声阈值模型 (noisy threshold model). 对于每个输入 $\boldsymbol{\phi}_n$, 评估 $a_n=\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n$, 然后根据以下公式设定目标值:

$$\begin{cases} t_n=1, & a_n \geqslant \theta \\ t_n=0, & \text{其他} \end{cases} \tag{5.118}$$

定义 5.25 (噪声阈值模型)

设每个输入 $\boldsymbol{\phi}_n$ 对应一个线性预激活 $a_n=\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n$, 并设阈值 θ 是一个随机变量, 其概率密度为 $p(\theta)$. 若目标值按(5.118)设定, 即当 $a_n \geqslant \theta$ 时取 $t_n=1$、否则取 $t_n=0$, 则称该模型为噪声阈值模型 (noisy threshold model).

♣

注

(重构新增) 噪声阈值模型的动机是: 真实的类标签往往由某个未观测的阈值决定, 而阈值本身带有随机性 (测量误差、个体差异等), 把它建模为随机变量就得到一条平滑的激活函数. 阈值 θ 的分布不同, 得到的 f 就不同:θ 取 logistic 分布时得到 logistic sigmoid, 取 Gauss 分布时得到 probit(见定义 5.26). 条件“θ 与 $\boldsymbol{\phi}_n$ 独立”是隐含的, 否则 f 还要依赖于 $\boldsymbol{\phi}_n$, 模型不再是广义线性的.(5.118)中 $a_n=\theta$ 的取法 (大于等于还是大于) 只影响零概率事件, 不影响下面的积分表达式.

如果 θ 值来自概率密度 $p(\theta)$, 那么相应的激活函数将由累积分布函数 (cumulative distribution function) 给出:

$$f(a) = \int_{-\infty}^{a} p(\theta)\,\mathrm{d}\theta \tag{5.119}$$

如图 5.17所示.

命题 5.24 (噪声阈值模型的激活函数)

设 θ 的概率密度为 $p(\theta)$, 目标值按(5.118)设定, 且 θ 与 $\boldsymbol{\phi}$ 独立. 则 $p(t=1 \mid a) = f(a)$, 其中 f 由(5.119)给出.

证明

(重构新增) 由全概率公式并对 θ 的边缘分布积分,

$$p(t=1 \mid a) = \int_{-\infty}^{\infty} p(t=1 \mid a, \theta)\, p(\theta)\,\mathrm{d}\theta \tag{5.120}$$

由(5.118), 给定 θ 时 $t=1$ 当且仅当 $a \geqslant \theta$, 故 $p(t=1 \mid a, \theta) = \mathbb{I}\{\theta \leqslant a\}$. 于是

$$p(t=1 \mid a) = \int_{-\infty}^{\infty} \mathbb{I}\{\theta \leqslant a\}\, p(\theta)\,\mathrm{d}\theta = \int_{-\infty}^{a} p(\theta)\,\mathrm{d}\theta = f(a) \tag{5.121}$$

即(5.119).

□

注

(重构新增) 命题说明: 任何概率分布函数都可以充当激活函数, 因此“选择激活函数”等价于“选择阈值的噪声分布”. 这解释了为什么 probit 与 logistic 都合理: 它们分别对应 Gauss 噪声与 logistic 噪声. 累积分布函数自动满足单调、值域为 $[0,1]$, 这正是定义 5.21所要求的. 若 $p(\theta)$ 关于 0 对称, 则 $f(0) = 1/2$, 决策面过原点 (在增广空间中).

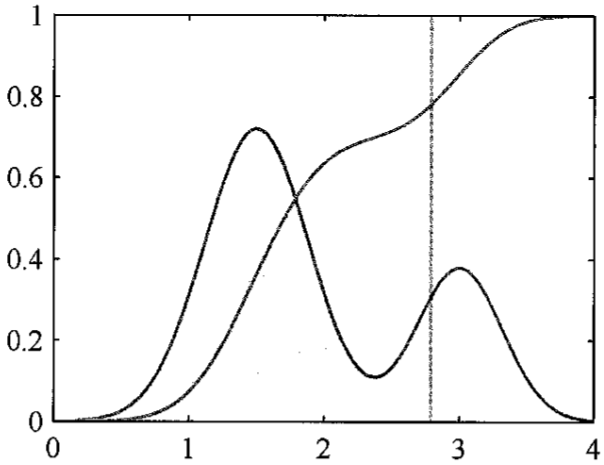


图 5.17: 蓝色曲线所示为概率密度 $p(\theta)$, 在本例中由两个 Gauss 混合分布及其累积分布函数 $f(a)$(用红色曲线表示) 给出. 请注意, 蓝色曲线在任何一点的值, 例如垂直绿线处的值, 对应于红色曲线在同一点的斜率. 相反, 红色曲线在此点的值对应于绿色阴影区域内蓝色曲线下的面积. 在随机阈值模型中, 如果 $a = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}$ 的值超过阈值, 则类标签取 $t=1$, 否则取 $t=0$, 这等价于由累积分布函数 $f(a)$ 给出的激活函数

举个具体的例子, 假设密度 $p(\theta)$ 由一个零均值、单位方差的 Gauss 分布给出. 相应的累积分布函数为

$$\Phi(a) = \int_{-\infty}^{a} \mathcal{N}(\theta \mid 0, 1)\,\mathrm{d}\theta \tag{5.122}$$

定义 5.26 (probit 函数与 erf 函数)

(5.122)定义的函数 Φ 称为 probit 函数. 函数

$$\operatorname{erf}(a) = \frac{2}{\sqrt{\pi}} \int_0^a \exp\left(-\theta^2/2\right) \mathrm{d}\theta \tag{5.123}$$

称为 erf 函数.

♣

注意, 使用具有一般均值和方差的 Gauss 分布并不会改变模型, 因为这等同于重新缩放线性系数 $\boldsymbol{w}$. 许多数值计算程序库都支持计算(5.123)中的 erf 函数, 它与 probit 函数的关系如下 (见练习 5.23):

$$\Phi(a) = \frac{1}{2}\left\{1 + \frac{1}{\sqrt{2}} \operatorname{erf}(a)\right\} \tag{5.124}$$

命题 5.25 (probit 函数与 erf 函数的关系)

设 Φ 与 erf 分别由(5.122)与(5.123)定义, 则(5.124)成立.

♠

证明

(重构新增) 记 $I(a) = \int_0^a e^{-\theta^2/2} \mathrm{d}\theta$. 由 $\mathcal{N}(\theta \mid 0, 1) = e^{-\theta^2/2}/\sqrt{2\pi}$ 与 $\Phi(0) = 1/2$,

$$\Phi(a) = \frac{1}{2} + \frac{1}{\sqrt{2\pi}} I(a) \tag{5.125}$$

而由(5.123),$\operatorname{erf}(a) = \frac{2}{\sqrt{\pi}} I(a)$, 故 $I(a) = \frac{\sqrt{\pi}}{2} \operatorname{erf}(a)$. 代入得

$$\Phi(a) = \frac{1}{2} + \frac{1}{\sqrt{2\pi}} \cdot \frac{\sqrt{\pi}}{2} \operatorname{erf}(a) = \frac{1}{2} + \frac{1}{2\sqrt{2}} \operatorname{erf}(a) = \frac{1}{2}\left\{1 + \frac{1}{\sqrt{2}} \operatorname{erf}(a)\right\} \tag{5.126}$$

即(5.124).

□

注

(重构新增) 命题的证明只用到两件事:Φ 关于 0 对称 (故 $\Phi(0) = 1/2$) 以及两个积分只相差一个常数因子.(5.123)中的因子 $1/\sqrt{2}$ 使 erf 的自变量按标准差缩放, 若改用常见的 $\operatorname{erf}(a) = \frac{2}{\sqrt{\pi}} \int_0^a e^{-\theta^2} \mathrm{d}\theta$, 则关系式变为 $\Phi(a) = \frac{1}{2}\{1 + \operatorname{erf}(a/\sqrt{2})\}$, 两者等价但常数位置不同. 使用 erf 的实际原因是许多数值库只提供 erf 而无 probit; 两者可以互相换算, 因此计算上无实质差别.

基于 probit 函数的广义线性模型称为 probit 回归模型. 通过对前面讨论的思路直接进行扩展, 我们可以利用最大似然法确定模型的参数. 实际上, 使用 probit 回归得出的结果往往与使用 logistic 回归得到的结果相似.

定义 5.27 (probit 回归)

若二分类模型的后验概率取 $p(C_1 \mid \boldsymbol{\phi}) = \Phi(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi})$ 的形式, 其中 Φ 为(5.122)定义的 probit 函数, 则称该模型为 probit 回归 (probit regression) 模型.

♣

注

(重构新增) probit 回归与 logistic 回归同属广义线性模型, 区别只在连接函数:probit 用标准正态分布函数,logistic 用 sigmoid. 二者的决策面都是超平面, 参数个数相同, 估计方法也相同 (最大似然 + 迭代). 差别体现在尾部:probit 的尾部按 $e^{-a^2/2}$ 衰减,logistic 按 e^{-a} 衰减, 因此远离决策面的点对 probit 的似然贡献更小,probit 对离群值更敏感 (见命题 5.26). 在实际数据上, 两者的预测往往非常接近, 因为两条曲线在中心区域几乎重合 (见图 5.12).

实际应用中可能出现的一个问题是离群值. 例如, 在测量输入向量 $\boldsymbol{x}$ 时出现的误差或对目标值 t 的错误标记, 都可能导致离群值的出现. 因为这些点可能离理想的决策边界很远, 它们会严重扭曲分类器.logistic 回归模型和 probit 回归模型在这方面的表现不同, 因为在 $|x| \to \infty$ 的情况下,logistic sigmoid 函数的尾部衰减为 $\exp(-x)$, 而 probit 函数的尾部衰减为 $\exp(-x^2)$, 因此 probit 模型对离群值更为敏感.

命题 5.26 (尾部衰减的比较)

对 $a>0$, 有

$$\sigma(-a)=\frac{e^{-a}}{1+e^{-a}},\qquad \frac{e^{-a}}{2}\leqslant\sigma(-a)\leqslant e^{-a}, \tag{5.127}$$

而对 probit 函数有

$$1-\Phi(a)=\int_a^{\infty}\mathcal{N}(\theta\mid 0,1)\,\mathrm{d}\theta\leqslant\frac{1}{a\sqrt{2\pi}}e^{-a^2/2}. \tag{5.128}$$

因此 logistic 的尾部按 e^{-a} 衰减,probit 的尾部按 $e^{-a^2/2}$ 衰减.

证明

(重构新增) 由 $\sigma(-a)=1/(1+e^{a})=e^{-a}/(1+e^{-a})$, 且 $a\geqslant 0$ 时 $1\leqslant 1+e^{-a}\leqslant 2$, 得 $\dfrac{e^{-a}}{2}\leqslant\sigma(-a)\leqslant e^{-a}$. 对 probit, 记 $\varphi(\theta)=\mathcal{N}(\theta\mid 0,1)$, 则 $\varphi'(\theta)=-\theta\varphi(\theta)$. 当 $\theta\geqslant a>0$ 时 $\theta/a\geqslant 1$, 故

$$1-\Phi(a)=\int_a^{\infty}\varphi(\theta)\,\mathrm{d}\theta\leqslant\int_a^{\infty}\frac{\theta}{a}\varphi(\theta)\,\mathrm{d}\theta=\frac{1}{a}\int_a^{\infty}\theta\varphi(\theta)\,\mathrm{d}\theta=\frac{\varphi(a)}{a}=\frac{1}{a\sqrt{2\pi}}e^{-a^2/2} \tag{5.129}$$

其中倒数第二个等号用了 $-\varphi'=\theta\varphi$, 即 $\displaystyle\int_a^{\infty}\theta\varphi(\theta)\mathrm{d}\theta=\varphi(a)$. 故 probit 的尾部至多按 $e^{-a^2/2}$ 衰减, 比 e^{-a} 快得多.

□

注

(重构新增) 两个尾部估计都是上界, 但足以说明量级差别: 当 a 从 10 增到 20 时,e^{-a} 缩小约 $e^{-10}\approx 4.5\times 10^{-5}$ 倍, 而 $e^{-a^2/2}$ 缩小约 e^{-150} 倍. 因此一个远离决策面的离群点对 probit 的似然几乎“不可见”, 而对 logistic 仍有非零影响, 这就是“probit 对离群值更敏感”的定量含义. 注意这里的 a 是带符号的预激活, 讨论离群值时应按 $|a|$ 理解; 若离群点被正确分类且 a 很大, 其似然接近 1, 对参数的拉动反而小, 真正有害的是被错标或位于边界远处的点.

5.4.6 规范连接函数

对于 Gauss 噪声分布的线性回归模型来说, 与负对数似然对应的误差函数就是平方和误差函数 (其梯度见eq. (4.12)). 如果我们在关于数据点 n 对误差函数的贡献方面对参数向量 $\boldsymbol{w}$ 求导, 则得到的形式为“误差” (y_n-t_n) 乘以 $\boldsymbol{\phi}_n$, 其中 $y_n=\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n$. 同样, 对于 logistic sigmoid 函数与交叉熵误差函数 [即(5.102)] 的组合以及 softmax 函数与多类交叉熵误差函数 [即(5.113)] 的组合, 我们也能得到同样的简单形式. 需要说明的是, 这是假定目标变量的条件分布来自指数族分布以及相应的激活函数选择 [即规范连接函数 (canonical link function)] 的一般结果.

让我们再次使用指数族分布的限制形式 (其一般形式见eq. (3.142)). 请注意, 这里我们对目标变量 t 采用了指数族分布假设, 而在小节 5.3.4中, 我们对输入向量 $\boldsymbol{x}$ 采用了指数族分布假设. 因此, 考虑目标变量的条件分布, 其形式为

$$p(t\mid\eta,s)=\frac{1}{s}h\left(\frac{t}{s}\right)g(\eta)\exp\left\{\frac{\eta t}{s}\right\} \tag{5.130}$$

利用推导eq. (3.169)时的相同论证思路, 我们可以看到,t 的条件均值 (用 y 表示) 由下式给出:

$$y=\mathbb{E}[t\mid\eta]=-s\frac{\mathrm{d}}{\mathrm{d}\eta}\ln g(\eta) \tag{5.131}$$

因此,y 和 η 必须相关, 它们之间的关系为 $\eta=\psi(y)$.

根据 Nelder and Wedderburn(1972), 我们将广义线性模型定义为 y 是输入 (或特征) 变量线性组合的非线性函数的模型, 从而有

$$y=f\left(\boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}\right) \tag{5.132}$$

其中 $f(\cdot)$ 在机器学习文献中称为激活函数, 而 $f^{-1}(\cdot)$ 在统计学中称为连接函数.

定义 5.28 (规范连接函数)

设目标变量 t 的条件分布为(5.130),$y = \mathbb{E}[t \mid \eta]$ 由(5.131)给出,ψ 为 $\eta = \psi(y)$ 所定义的关系. 若连接函数取 $f^{-1}(y) = \psi(y)$, 即使得线性预激活 $a = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}$ 恰等于自然参数 η, 则称该连接函数为规范连接函数 (canonical link function).

♣

注

(重构新增) 规范连接的含义是“让线性预激活直接等于自然参数”, 从而使对数似然对参数的导数中不出现额外的链式因子 (见命题 5.27). 它不是唯一的选择, 但在给定噪声分布时是使推导最简洁的选择, 因此称为“规范”. 对 Bernoulli 分布,$\eta = \ln[\mu/(1-\mu)]$ 即 logit, 故 logistic 回归使用规范连接; 对 Gauss 分布,$\eta = \mu$, 故线性回归使用恒等连接; 对 Poisson 分布,$\eta = \ln\mu$. 注意这里的“分布”是对目标变量 t 的假设, 而不是对输入 $\boldsymbol{x}$ 的假设, 这与小节 5.3.4的生成式视角不同.

考虑该模型的对数似然函数, 作为 η 的函数, 其计算公式为

$$\ln p(\boldsymbol{t} \mid \eta, s) = \sum_{n=1}^{N} \ln p(t_n \mid \eta_n, s) = \sum_{n=1}^{N} \left\{ \ln g(\eta_n) + \frac{\eta_n t_n}{s} \right\} + \text{const} \tag{5.133}$$

其中, 我们假设所有观测值都有一个共同的尺度参数 (相当于 Gauss 分布的噪声方差), 因此 s 与 n 无关. 对数似然关于模型参数 $\boldsymbol{w}$ 的导数为

$$\begin{aligned} \nabla_{\boldsymbol{w}} \ln p(\boldsymbol{t} \mid \eta, s) &= \sum_{n=1}^{N} \left\{ \frac{\mathrm{d}}{\mathrm{d}\eta_n} \ln g(\eta_n) + \frac{t_n}{s} \right\} \frac{\mathrm{d}\eta_n}{\mathrm{d}y_n} \frac{\mathrm{d}y_n}{\mathrm{d}a_n} \nabla_{\boldsymbol{w}} a_n \\ &= \sum_{n=1}^{N} \frac{1}{s} \{t_n - y_n\} \psi'(y_n) f'(a_n) \boldsymbol{\phi}_n \end{aligned} \tag{5.134}$$

其中 $a_n = \boldsymbol{w}^{\mathsf{T}}\boldsymbol{\phi}_n$, 我们使用了 $y_n = f(a_n)$ 和 $\mathbb{E}[t \mid \eta]$ 的结果(5.131). 可以看到, 如果我们为连接函数 $f^{-1}(y)$ 选择一种特殊的形式, 就可以极大地简化计算过程.

$$f^{-1}(y) = \psi(y) \tag{5.135}$$

由于 $f(\psi(y)) = y$, 因此 $f'(\psi)\psi'(y) = 1$. 同样, 由于 $a = f^{-1}(y)$, 得出 $a = \psi$, 因此 $f'(a)\psi'(y) = 1$. 在这种情况下, 误差函数的梯度减小为

$$\nabla \ln E(\boldsymbol{w}) = \frac{1}{s} \sum_{n=1}^{N} \{y_n - t_n\} \boldsymbol{\phi}_n \tag{5.136}$$

命题 5.27 (规范连接下的梯度化简)

设目标变量的条件分布为(5.130), 模型为(5.132), 且连接函数为规范连接, 即 $f^{-1} = \psi$. 则对数似然关于 $\boldsymbol{w}$ 的梯度为

$$\nabla_{\boldsymbol{w}} \ln p(\boldsymbol{t} \mid \eta, s) = \frac{1}{s} \sum_{n=1}^{N} (t_n - y_n) \boldsymbol{\phi}_n. \tag{5.137}$$

♠

证明

(重构新增) 由(5.131),$\dfrac{\mathrm{d}}{\mathrm{d}\eta_n} \ln g(\eta_n) = -\dfrac{y_n}{s}$. 代入(5.134)的第一个等号, 括号内变为 $(t_n - y_n)/s$. 在规范连接下 $a_n = \eta_n = \psi(y_n)$, 故 $\psi'(y_n)f'(a_n) = 1$. 于是(5.134)化为 $\dfrac{1}{s}\sum_n (t_n - y_n)\boldsymbol{\phi}_n$.

□

注

(重构新增) 命题说明了“规范连接”这一名称的由来: 正是它使链式因子 $\psi'(y)f'(a)$ 恒等于 1, 从而梯度只剩“误差乘特征”. 三个特例:Gauss + 恒等连接给出线性回归的平方和梯度;Bernoulli + logit 给出(5.103); 多项分布 + softmax 给出(5.114). 关于原书(5.136)的写法, 需要说明两点: 其一, 按(5.134), 对数似然的梯度应为 $\frac{1}{s}\sum_n (t_n - y_n)\boldsymbol{\phi}_n$, 而(5.136)印为 $\{y_n - t_n\}$, 两者相差一个符号; 其二, 左端的记号“$\nabla \ln E(\boldsymbol{w})$”应为 $\nabla_{\boldsymbol{w}} \ln p$ 或负对数似然的梯度 ∇E, 因为 E 本身已是对数似然的负值. 若取 E 为负对数似然, 则 $\nabla E = \frac{1}{s}\sum_n (y_n - t_n)\boldsymbol{\phi}_n$, 与印出的右端一致. 因此这里按原书照译, 同时指出其符号与记号的确切含义, 以免读者在推导中混淆.

我们已经看到, 误差函数的选择与输出-单元激活函数的选择之间存在自然的配对. 尽管我们是在单层网络的背景下推导出这一结果的, 但同样的考虑因素也适用于后续章节讨论的深度神经网络.

5.5 习题

✍ **练习 5.1** (⋆) 考虑一个涉及 K 个类和目标向量 $\boldsymbol{t}$ 的分类问题, 使用 1-of-K 二进制编码方案. 证明条件期望 $\mathbb{E}[\boldsymbol{t} \mid \boldsymbol{x}]$ 由后验概率 $p(C_k \mid \boldsymbol{x})$ 给出.

✍ **练习 5.2** (⋆⋆) 在给定一组数据点 $\{\boldsymbol{x}_n\}$ 的情况下, 我们可以将凸包定义为所有点 $\boldsymbol{x}$ 的集合

$$\boldsymbol{x} = \sum_n \alpha_n \boldsymbol{x}_n \tag{5.138}$$

其中 $\alpha_n \geqslant 0$ 且 $\sum_n \alpha_n = 1$. 考虑另一组数据点 $\{\boldsymbol{y}_n\}$ 及相应的凸包. 根据定义, 如果存在一个向量 $\boldsymbol{w}$ 和一个标量 w_0, 使得对所有 $\boldsymbol{x}_n$ 而言 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{x}_n + w_0 > 0$, 并且对所有 $\boldsymbol{y}_n$ 而言 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{y}_n + w_0 < 0$, 那么这两组数据点就是线性可分的. 证明如果它们的凸包相交, 则这两组数据点不可能是线性可分的; 反之, 如果它们是线性可分的, 则它们的凸包不相交.

✍ **练习 5.3** (⋆⋆) 考虑最小化平方和误差函数(5.16), 并假设训练集中所有的目标向量都满足线性约束条件

$$\boldsymbol{a}^{\mathsf{T}}\boldsymbol{t}_n + b = 0 \tag{5.139}$$

其中 $\boldsymbol{t}_n$ 相当于(5.16)中矩阵 $\boldsymbol{T}$ 的第 n 行. 证明作为以上约束条件的结果, 由最小二乘解(5.20)给出的模型预测 $\boldsymbol{y}(\boldsymbol{x})$ 的元素也满足该约束条件, 即

$$\boldsymbol{a}^{\mathsf{T}}\boldsymbol{y}(\boldsymbol{x}) + b = 0 \tag{5.140}$$

为此, 假设其中一个基函数 $\phi_0(\boldsymbol{x}) = 1$, 这样相应的参数 w_0 就起到了偏置的作用.

✍ **练习 5.4** (⋆⋆) 扩展练习 5.3的结果, 证明如果目标向量同时满足多个线性约束条件, 那么线性模型的最小二乘预测也将满足相同的约束条件.

✍ **练习 5.5** (⋆) 利用定义(5.47)以及(5.38)和(5.39)推导 F 分数的结果(5.48).

✍ **练习 5.6** (⋆⋆) 考虑两个非负数 a 和 b, 并证明如果 $a \leqslant b$, 则 $a \leqslant (ab)^{1/2}$. 利用这一结果可以证明, 如果选择二分类问题的决策区域是为了最小化误分类概率, 则该概率将满足

$$p(\text{错误}) \leqslant \int \{p(\boldsymbol{x}, C_1)\, p(\boldsymbol{x}, C_2)\}^{1/2} \mathrm{d}\boldsymbol{x} \tag{5.141}$$

✍ **练习 5.7** (⋆) 给定一个含有元素 L_{kj} 的损失矩阵, 如果我们为每个 $\boldsymbol{x}$ 选择最小化(5.30)的类, 则预期风险最小. 验证一下, 当损失矩阵为 $L_{kj} = 1 - I_{kj}$(其中 I_{kj} 为单位矩阵的元素) 时, 则可简化为选择后验概率最大的类的准则. 如何解释这种形式的损失矩阵?

✍ **练习 5.8** (⋆) 在有关的一般损失矩阵和一般先验概率时, 推导最小化预期损失的准则.

✍ **练习 5.9** (⋆) 考虑一组 N 个数据点的后验概率的平均值, 其形式为

$$\frac{1}{N}\sum_{n=1}^{N} p(C_k \mid \boldsymbol{x}_n) \tag{5.142}$$

通过取极限 $N \to \infty$, 证明这个量接近类的先验概率 $p(C_k)$.

✍ **练习 5.10** (★★) 考虑一个分类问题, 其中当来自 C_k 类的输入向量被归入 C_j 类时, 产生的损失由损失矩阵 L_{kj} 给出, 而选择拒绝选项所产生的损失为 λ. 推导预期损失最小的决策准则. 验证一下, 当损失矩阵由 $L_{kj} = 1 - I_{kj}$ 给出时, 所要推导的决策准则可简化为小节 5.2.3讨论的拒绝准则.λ 和拒绝阈值 θ 之间有何关系?

✍ **练习 5.11** (★) 证明 logistic sigmoid 函数 [即(5.53)] 满足性质 $\sigma(-a) = 1 - \sigma(a)$, 其逆函数由 $\sigma^{-1}(y) = \ln\{y/(1-y)\}$ 给出.

✍ **练习 5.12** (★) 利用(5.51)和(5.52)得出具有 Gauss 密度的二类生成式模型的类的后验概率结果(5.60), 并对参数 $\boldsymbol{w}$ 和 w_0 的结果(5.61)和(5.62)进行验证.

✍ **练习 5.13** (★) 考虑一个由类的先验概率 $p(C_k) = \pi_k$ 和类-条件概率密度 $p(\boldsymbol{\phi} \mid C_k)$ 定义的涉及 K 个类的分类模型, 其中 $\boldsymbol{\phi}$ 是输入特征. 假设我们得到一个训练数据集 $\{\boldsymbol{\phi}_n, \boldsymbol{t}_n\}$, 其中 $n = 1, \cdots, N$,$\boldsymbol{t}_n$ 是长度为 K 且采用 1-of-K 编码方案的二进制目标向量. 如果数据点 n 属于 C_k 类, 则 $t_{nj} = I_{jk}$. 假设数据点是从该模型中独立提取的, 证明先验概率的最大似然解为

$$\pi_k = \frac{N_k}{N} \tag{5.143}$$

其中 N_k 是归入 C_k 类的数据点数量.

✍ **练习 5.14** (★★) 考虑练习 5.13中的分类模型, 假设类-条件概率密度由具有共享协方差矩阵的 Gauss 分布给出, 因此

$$p(\boldsymbol{\phi} \mid C_k) = \mathcal{N}(\boldsymbol{\phi} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}) \tag{5.144}$$

证明 C_k 类的 Gauss 分布均值的最大似然解由下式给出:

$$\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{n=1}^{N} t_{nk} \boldsymbol{\phi}_n \tag{5.145}$$

它表示归入 C_k 类的那些特征向量的均值. 类似地, 证明共享协方差矩阵的最大似然解由下式给出:

$$\boldsymbol{\Sigma} = \sum_{k=1}^{K} \frac{N_k}{N} \boldsymbol{S}_k \tag{5.146}$$

其中

$$\boldsymbol{S}_k = \frac{1}{N_k} \sum_{n=1}^{N} t_{nk} (\boldsymbol{\phi}_n - \boldsymbol{\mu}_k)(\boldsymbol{\phi}_n - \boldsymbol{\mu}_k)^{\mathsf{T}} \tag{5.147}$$

因此,$\boldsymbol{\Sigma}$ 由与每个类相关的数据协方差的加权平均值给出, 其中加权系数由类的先验概率给出.

✍ **练习 5.15** (★★) 推导小节 5.3.3所讲的具有离散二元特征的概率朴素 Bayes 分类器参数 $\{\mu_{ki}\}$ 的最大似然解.

✍ **练习 5.16** (★★) 考虑一个涉及 K 个类的分类问题, 其中输入特征 $\boldsymbol{\phi}$ 有 M 个组件, 每个组件有 L 个离散状态. 将组件的值用 1-of-L 二进制编码方案表示. 进一步假设, 以 C_k 类为条件,$\boldsymbol{\phi}$ 的 M 个组件是独立的, 因此关于特征向量组件的类-条件概率密度可以分解.

证明由(5.57)给出的量 a_k(出现在描述类的后验概率的 softmax 函数的自变量中) 是 $\boldsymbol{\phi}$ 的组件的线性函数. 请注意, 这是一个朴素 Bayes 模型 (参见小节 11.2.4) 的示例.

✍ **练习 5.17** (★★) 推导练习 5.16中描述的概率朴素 Bayes 分类器参数的最大似然解.

✍ **练习 5.18** (★) 验证由(5.53)定义的 logistic sigmoid 函数导数的关系式(5.99).

✍ **练习 5.19** (★) 利用 logistic sigmoid 函数的导数结果(5.99), 证明 logistic 回归模型的误差函数(5.102)由(5.103)给出.

✍ **练习 5.20** (★) 证明对于线性可分数据集,logistic 回归模型的最大似然解可以通过计算一个向量 $\boldsymbol{w}$ 来获得, 该向量的决策边界 $\boldsymbol{w}^{\mathsf{T}} \boldsymbol{\phi}(\boldsymbol{x}) = 0$ 可将类分开, 然后将 $\boldsymbol{w}$ 的大小取到无穷大.

✍ **练习 5.21** (★) 证明 softmax 激活函数 [即(5.107)] 的导数 [其中 a_k 由(5.66)定义] 由(5.109)给出.

✍ **练习 5.22** (★) 利用 softmax 激活函数的导数结果 [即(5.109)], 证明交叉熵误差 [即(5.113)] 的梯度由(5.114)给出.

✍ **练习 5.23** (★) 证明 probit 函数 [即(5.122)] 和 erf 函数 [即(5.123)] 的关系式(5.124).

✍ **练习 5.24** (★★) 假设我们希望用缩放 probit 函数 $\Phi(\lambda a)$ 来近似由(5.53)定义的 logistic sigmoid 函数 $\sigma(a)$, 其中 $\Phi(a)$ 由(5.122)定义. 证明如果选择的 λ 能使这两个函数的导数在 $a = 0$ 处相等, 则 $\lambda^2 = \pi/8$.

第 6 章 深度神经网络

近年来, 神经网络已迅速成为实际应用中最重要的机器学习技术, 因此我们在本书中专门花了大量篇幅来介绍它. 之前的章节已经为我们奠定了许多必要的基础. 特别是, 我们已经看到, 由固定非线性基函数的线性组合构成的线性回归模型可以表达为单层权重和偏置参数的神经网络 (参见节 4.1). 同样, 基于基函数线性组合的分类模型也可以视作单层 (神经) 网络 (参见节 5.1). 这使我们能在讨论更复杂的多层 (神经) 网络之前引入几个重要概念.

给定足够数量的适当选择的基函数, 这样的线性模型可以以任意所需精度近似任何给定的从输入到输出的非线性变换, 因此似乎足以应对任何实际应用. 然而, 这些模型具有一些严重的局限性, 因此我们将通过探索这些局限性并理解为什么需要从数据中学习得到基函数, 来开始我们对神经网络的讨论. 这自然地引出了对具有多层可学习参数的神经网络的讨论 (参见小节 6.3.6), 它们被称为前馈网络 (feed-forward network) 或多层感知机 (multi-layer perceptron). 我们还将讨论多层处理的好处, 并引出目前支配了机器学习领域的关键概念——深度神经网络 (Deep Neural Network, DNN).

6.1 固定基函数的局限性

用于分类的线性基函数模型是基于基函数 $\phi_j(\boldsymbol{x})$ 的线性组合 (参见节 5.1), 形式如下:

定义 6.1 (线性基函数模型)

设 $\{\phi_j(\boldsymbol{x})\}_{j=1}^M$ 是一组给定的非线性基函数, $w_0, w_1, \cdots, w_M$ 为可学习参数, $f(\cdot)$ 为输出激活函数. 形如

$$y(\boldsymbol{x}, \boldsymbol{w}) = f\left(\sum_{j=1}^{M} w_j \phi_j(\boldsymbol{x}) + w_0\right) \tag{6.1}$$

的模型称为**线性基函数模型** (linear basis function model), 其中 $\boldsymbol{w} = (w_1, \cdots, w_M)^{\mathsf{T}}$.

其中 $f(\cdot)$ 是一个非线性输出激活函数. 用于回归的线性模型具有相同的形式, 但用恒等函数替换了 $f(\cdot)$(参见小节 4.1.1). 这些模型允许使用任意一组非线性基函数 $\{\phi_j(\boldsymbol{x})\}$, 并且由于这些基函数的一般性, 这样的模型原则上可以解决任何回归或分类问题. 我们可以简单地这么理解, 如果其中一个基函数对应希望的输入–输出转换, 则可学习的线性层只需要将这个基函数的值复制到模型的输出即可.

注

(重构新增)定义 6.1把“线性”限定在参数 $\boldsymbol{w}$ 上, 而不限定在输入 $\boldsymbol{x}$ 上: 只要 f 与 $\{\phi_j\}$ 中至少有一个是非线性的, $y(\boldsymbol{x}, \boldsymbol{w})$ 就是 $\boldsymbol{x}$ 的非线性函数. 这一区分是理解本章的关键: 模型的表达能力由固定基函数族 $\{\phi_j\}$ 决定, 而训练只在这个固定族上做线性组合. 也正因如此, 参数估计是凸问题 (最小二乘) 或单峰问题 (交叉熵), 但表达能力被基函数族事先锁死.

更一般地, 我们期望有一个足够大且丰富的基函数集合, 从而能以任意精度拟合需要的函数. 因此, 看似这样的线性模型构成了解决机器学习问题的通用框架. 遗憾的是, 线性模型存在一些重大缺陷, 因为它们假设基函数 $\phi_j(\boldsymbol{x})$ 是固定的且与训练数据无关. 为了理解这些局限性, 我们首先观察随着输入变量数量的增加线性模型的行为有何变化.

6.1.1 维度诅咒

考虑由如下 M 阶多项式给出的单输入变量的简单回归模型 (参见节 1.2):

$$y(x, \boldsymbol{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M. \tag{6.2}$$

假设增加输入变量的数量, 如果有 D 个输入变量 $\{x_1, \cdots, x_D\}$, 那么至多三阶系数的通用多项式形式如下:

$$y(\boldsymbol{x},\boldsymbol{w}) = w_0 + \sum_{i=1}^{D} w_i x_i + \sum_{i=1}^{D}\sum_{j=1}^{D} w_{ij} x_i x_j + \sum_{i=1}^{D}\sum_{j=1}^{D}\sum_{k=1}^{D} w_{ijk} x_i x_j x_k. \tag{6.3}$$

随着 D 的增大, 独立系数的数量增长为 $O(D^3)$, 而对于 M 阶多项式, 系数的数量增长为 $O(D^M)$(Bishop,2006). 我们看到, 在高维空间中, 多项式可以迅速变得笨重并且几乎不具有实用性.

命题 6.1 (多项式系数的数量)

D 个变量的所有次数不超过 M 的单项式所构成的线性模型, 其独立系数 (包括常数项) 的个数为

$$\binom{D+M}{M} = \frac{(D+M)!}{D!\,M!}. \tag{6.4}$$

特别地, 当 $M=3$ 时该个数为 $(D+3)(D+2)(D+1)/6 = O(D^3)$; 对固定的 M, 一般地有 $O(D^M)$. ♠

证明

(重构新增) 次数不超过 M 的单项式与 "把 M 个不可区分的球放入 $D+1$ 个盒子" 的方案一一对应:D 个盒子分别记录 $x_1,\cdots,x_D$ 的指数, 第 $D+1$ 个盒子记录 "剩余次数", 它使总次数补足到 M. 由隔板法, 方案数为 $\binom{(D+1)+M-1}{M} = \binom{D+M}{M}$, 即(6.4). 当 $M=3$ 时, $\binom{D+3}{3} = (D+3)(D+2)(D+1)/6$; 而对固定的 M, 分子是 D 的 M 次多项式、分母是常数 $M!$, 故为 $O(D^M)$.

□

注

(重构新增)命题 6.1说明了 "维度诅咒" 的代数根源: 需要估计的参数个数随维度 D 呈多项式增长, 而 M 阶多项式在 D 增大时迅速失去实用性. 注意这里统计的是独立系数: 由于 $x_i x_j = x_j x_i$, 若把(6.3)中的 w_{ij} 与 w_{ji} 视为同一个参数, 二阶项只有 $D(D+1)/2$ 个而非 D^2 个, 但阶数不变. 换言之, 对称性只能把常数因子减半, 不能改变指数增长的趋势.

高维空间中出现的这种巨大困难有时称为维度诅咒 (Bellman,1961). 这是一个相当普遍的问题, 不仅限于多项式回归. 我们来考虑使用线性模型解决分类问题:图 6.1展示了来自鸢尾花数据集的数据, 其中包括来自三种鸢尾花的各 50 个观测值.

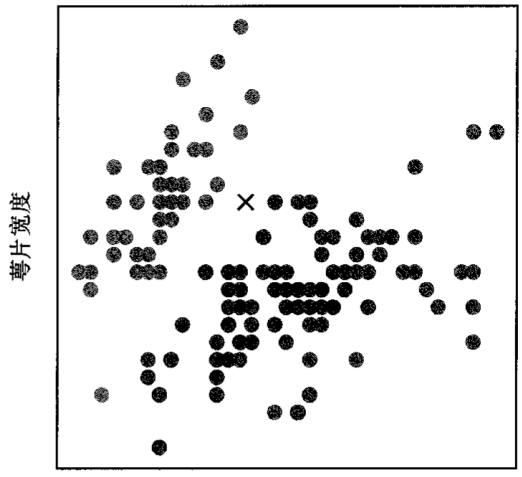


图 6.1: 鸢尾花数据的散点图. 其中红色、绿色和蓝色的点分别代表鸢尾花的三个亚种, 轴代表萼片长度和宽度的测量值. 我们的目标是对一个新的测试点 (图中用叉号表示) 进行分类

每个观测值有 4 个变量, 分别代表萼片长度、萼片宽度、花瓣长度和花瓣宽度的测量值. 对于这个示例, 我们只考虑萼片长度和萼片宽度. 给定这 150 个观测值作为训练数据, 我们的目标是将新的测试点, 如图 6.1中的叉号

所示, 归于其中一类. 我们观察到叉号靠近几个红点, 因此我们可能假设它属于红色类别. 然而, 叉号的附近也有一些绿点, 所以我们也可能认为它属于绿色类别. 而它属于蓝色类别的可能性似乎较小. 我们的直觉是, 叉号的类别应该更可能由训练集中较为靠近的点决定, 而不是由更远的点决定, 这个直觉是合理的.

将这个直觉转为学习算法的一种非常简单的方法是将输入空间划分为规则的单元, 如图 6.2所示.

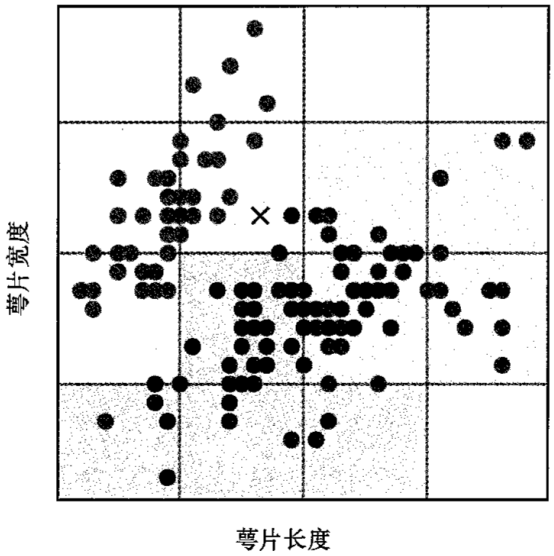


图 6.2: 解决分类问题的一种简单方法的示意图. 该方法将输入空间划分为规则的单元, 新测试点的类别被推断为同一单元内训练点最多的类别. 正如我们很快将看到的, 这种方法存在一些严重的缺陷

当我们得到一个测试点并希望预测其类别时, 可以首先确定它属于哪个单元, 然后找到所有落在同一单元内的训练点. 测试点的类别被推断为同一单元内训练点数量最多的那一类 (各类别如果训练点的数量相同, 则随机推断). 我们可以将其视为一个基函数模型, 其中每个单元有一个基函数 $\phi_j(\boldsymbol{x})$, 如果 $\boldsymbol{x}$ 位于单元外部, 则返回零, 否则返回所在单元内的训练点的主要类别. 该模型的输出由所有基函数的输出之和给出.

例题 6.1 网格分类器 把输入空间 $\mathbb{R}^D$ 按边长 h 划分为规则网格, 每个单元对应基函数

$$\phi_{\boldsymbol{m}}(\boldsymbol{x}) = \begin{cases} c_{\boldsymbol{m}}, & \boldsymbol{x} \text{ 落在第} \boldsymbol{m} \text{ 个单元内}, \\ 0, & \text{其他}, \end{cases} \tag{6.5}$$

其中 $c_{\boldsymbol{m}}$ 是该单元内训练点最多的类别. 把 $\mathbb{R}^D$ 中边长为 L 的立方体划分为规则单元, 所需单元数为 $(L/h)^D$.

这种朴素的方法存在许多问题, 当我们考虑将其扩展到具有更多输入变量的分类问题, 即考虑更高维度的输入空间时, 其中一个很严重的问题将变得更加明显.图 6.3说明了这个问题的起源.

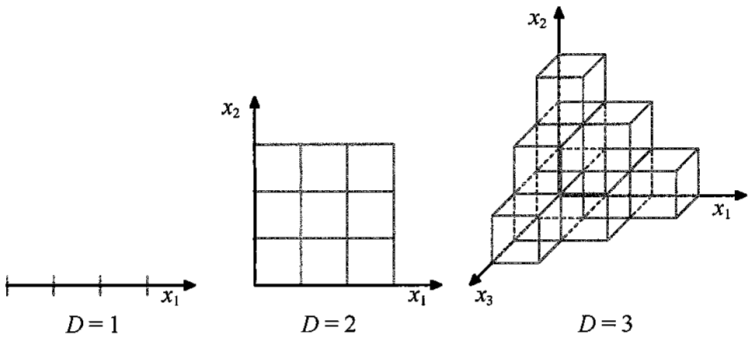


图 6.3: 维度诅咒的示意图. 这里展示了单元数量如何随空间维度 D 的增大呈指数级增长. 简便起见, 图中仅展示了 $D = 3$ 时的部分单元

它表明如果我们将输入空间划分为规则的单元, 那么这样的单元数量将随空间维度的增大呈指数级增长. 呈

指数级增长的单元数量带来的挑战是，我们需要指数级数量的训练数据来确保单元不是空的. 我们已经在图 6.2中看到，一些单元不包含任何训练点. 因此，这些单元中的测试点将得不到分类. 显然，我们无法在具有较多变量的输入空间中应用这种技术. 这种方法对多项式回归示例和鸢尾花数据分类示例而言都是困难的，因为基函数的选择没有考虑要解决的问题. 为了避免维度诅咒，我们需要在选择基函数时更加精细 (参见小节 6.1.4).

命题 6.2 (网格单元数的指数增长)

设输入空间为 $\mathbb{R}^D$ 中的单位立方体 $[0,1]^D$，用边长 h 的规则网格划分，则所需单元数为 h^{-D}. 若希望每个单元平均至少含有一个训练点、且训练点数为 N，则必须 $N \geqslant h^{-D}$，即 N 必须随 D 呈指数增长.

证明

(重构新增) 每个坐标轴被分成 $1/h$ 段，D 个坐标轴的独立选择给出 h^{-D} 个单元. 若每个单元至少含一个训练点，则 $N \geqslant h^{-D}$. 取 $h < 1$ 固定，$h^{-D} = e^{-D\ln h}$ 随 D 指数增长；例如 $h = 0.1$、$D = 10$ 时已需 10^{10} 个训练点.

□

注

(重构新增)命题 6.2给出了维度诅咒最朴素的定量形式，但它也提示了出路：真正起作用的不是数据空间的维度 D，而是数据实际占据的自由度. 如果数据位于一个 d 维流形上 ($d \ll D$)，那么把单元建在流形上而非整个空间上，所需单元数只随 d 指数增长. 这正是小节 6.1.3与小节 6.1.4的出发点.

6.1.2 高维空间

首先，我们将更细致地观察高维空间的性质. 在高维空间中，我们平日在三维空间中形成的几何直觉可能会严重地失效. 举个简单的例子，考虑 D 维空间中一个半径 $r = 1$ 的超球体，我们想知道介于半径 $r = 1-\varepsilon$ 和 $r = 1$ 之间的超球体体积的占比是多少. 我们注意到，半径为 r 的 D 维超球体的体积 $V_D(r)$ 必须按 r^D 缩放，因此有

$$V_D(r) = K_D r^D, \tag{6.6}$$

其中常数 K_D 仅取决于 D. 这个问题的答案由下式给出 (见练习 6.1):

$$\frac{V_D(1) - V_D(1-\varepsilon)}{V_D(1)} = 1 - (1-\varepsilon)^D. \tag{6.7}$$

命题 6.3 (高维超球体的薄壳效应)

设 $V_D(r) = K_D r^D$ 为 D 维超球体体积，则半径介于 $r = 1-\varepsilon$ 与 $r = 1$ 之间的薄壳占总体积的比例为 $1-(1-\varepsilon)^D$. 对固定的 $\varepsilon \in (0,1)$，当 $D \to \infty$ 时该比例单调递增地趋于 1.

证明

(重构新增) 由(6.6)，$V_D(1-\varepsilon) = K_D(1-\varepsilon)^D$，故

$$\frac{V_D(1) - V_D(1-\varepsilon)}{V_D(1)} = \frac{K_D - K_D(1-\varepsilon)^D}{K_D} = 1 - (1-\varepsilon)^D. \tag{6.8}$$

因为 $0 < 1-\varepsilon < 1$，序列 $(1-\varepsilon)^D$ 关于 D 单调递减且趋于 0，所以比例 $1-(1-\varepsilon)^D$ 单调递增地趋于 1.

□

注

(重构新增) 这个结论常被概括为“高维球的体积几乎全部集中在表面附近”. 它的实践含义是：在高维空间中，体积与概率质量的直觉不再一致，均匀分布于球内的点几乎必然落在薄壳里. 对机器学习而言，这意味着基于“局部邻域”的方法在高维下会失效：任何有限半径的邻域都只能覆盖极少的数据点.

图 6.4显示了不同维度 D 取值下体积占比关于 ε 的函数. 可以看到，对于较大的 D 值，即使 ε 值较小，这个占比也倾向于 1. 因此，我们有了一个重要的发现：在高维空间中，超球体的大部分体积集中在靠近表面的薄壳中!

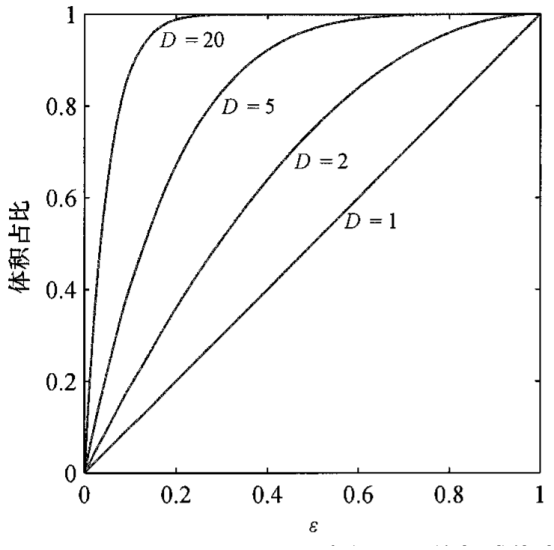


图 6.4: 不同维度 D 取值下, 半径 $r = 1$ 的超球体内, 半径介于 $r = 1 - \varepsilon$ 和 $r = 1$ 之间的超球体体积占比

接下来, 我们考虑高维空间中 Gauss 分布的行为, 作为与机器学习直接相关的另一个示例. 如果我们从笛卡儿坐标变换到极坐标, 然后积分消去方向变量, 则可以得到密度 $p(r)$ 的表达式, 它是一个关于从原点出发的半径为 r 的函数 (见练习 6.3). 因此,$p(r)\delta r$ 是位于半径 r 处的厚度为 δr 的薄壳内的概率质量. 在不同维度 D 取值下, 将这个分布绘制在图 6.5中. 可以看到, 对于较大的 D 值,Gauss 分布的大部分概率质量位于特定半径的薄壳内.

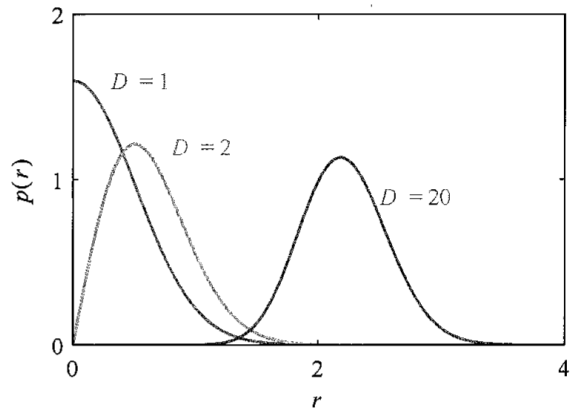


图 6.5: 不同维度 D 取值下, 关于半径为 r 的 Gauss 分布的概率密度图. 在高维空间中,Gauss 分布的大部分概率质量位于特定半径的薄壳内

命题 6.4 (高维 Gauss 的径向集中)

设 $\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{0}, \sigma^2 \boldsymbol{I})$ 取值于 $\mathbb{R}^D$,$r = \|\boldsymbol{x}\|$, 则径向密度为

$$p(r) = \frac{S_D r^{D-1}}{(2\pi\sigma^2)^{D/2}} \exp\left(-\frac{r^2}{2\sigma^2}\right), \tag{6.9}$$

其中 S_D 是 $\mathbb{R}^D$ 中单位超球体的表面积. 当 D 较大时,$p(r)$ 在 $r \approx \sqrt{D}\sigma$ 处取得唯一最大值, 且概率质量集中在该半径附近厚度为 $O(\sigma)$ 的薄壳内.

证明

(重构新增) 把 Gauss 密度在极坐标下积分掉方向变量即得(6.9)(详细推导见练习 6.3). 为求驻点, 对 $\ln p(r)$ 求导:

$$\frac{\mathrm{d}}{\mathrm{d}r}\ln p(r) = \frac{D-1}{r} - \frac{r}{\sigma^2},$$

令其为零得 $r^2 = (D-1)\sigma^2$, 故驻点为 $\hat{r} = \sqrt{D-1}\,\sigma \approx \sqrt{D}\sigma$; 又因二阶导数在 $r>0$ 上为负,$\hat{r}$ 是唯一最大值点. 在 $\hat{r}$ 附近作二阶展开可得 $p(\hat{r}+\varepsilon) \approx p(\hat{r})\exp\{-3\varepsilon^2/(2\sigma^2)\}$, 即密度在尺度 σ 上指数衰减, 故概率质量集中在半径 $\hat{r}$ 附近厚度为 $O(\sigma)$ 的薄壳内.

□

注

(重构新增)命题 6.4与命题 6.3是同一现象的两个侧面: 前者说的是体积集中在表面, 后者说的是概率质量集中在某个半径. 二者并不矛盾——Gauss 密度在原点最高, 但原点附近的体积元 $S_D r^{D-1}\mathrm{d}r$ 在 r 很小时几乎为零, 两者相乘的峰值因此被推到 $r \approx \sqrt{D}\sigma$ 处. 这提醒我们: 在高维空间中, "密度最大的点" 与 "最可能出现的半径" 是两个不同的概念.

在本书中, 我们广泛使用了涉及单个或两个变量的例子, 因为这样特别容易图形化地展示这些空间. 然而, 读者应该注意, 不是所有在低维空间中获得的直觉都可以推广到高维空间中.

最后, 虽然我们讨论了维度诅咒, 但在高维空间中建模也具有优势. 考虑图 6.6所示的数据集, 其中的每个数据点由一对值 (x_1, x_2) 组成.

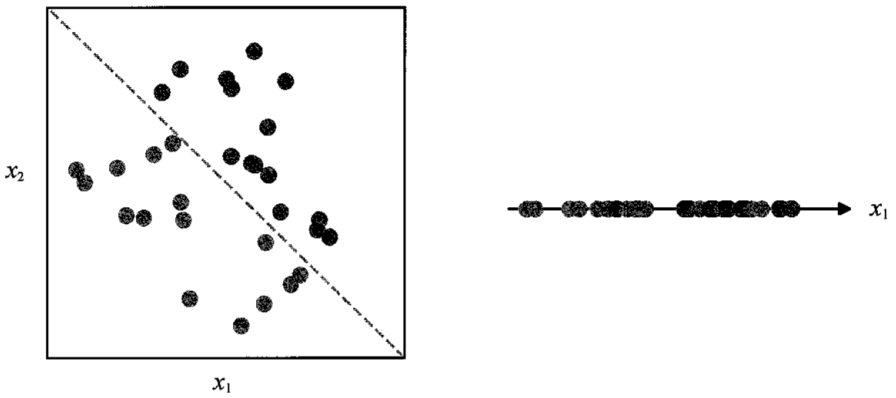


图 6.6: 二维数据集 (x_1, x_2) 的示意图. 其中, 使用绿色和红色圆圈表示的两类数据点可以被线性决策面分开, 如图 (a) 所示. 然而, 如果仅考虑变量 x_1, 则类不再可分, 如图 (b) 所示

我们看到这个数据集是线性可分的, 但是当我们仅观察 x_1 的值时, 发现类有严重的重叠. 因此, 在更高维的空间中, 分类问题解决起来要容易得多.

例题 6.2 升维使问题线性可分 设两类点分别位于直线 $x_2 = x_1$ 的两侧, 则它们在 $\mathbb{R}^2$ 中线性可分; 但把它们投影到 x_1 轴上后, 两类的取值范围完全重叠, 一维空间中没有阈值能把它们分开. 若再增加一个特征 x_1x_2, 则在同一维数更高的空间中仍可保持线性可分.

6.1.3 数据流形

在多项式回归模型和基于网格的分类器 (见图 6.2) 中, 我们看到随着维度的增加, 基函数的数量迅速增长, 导致这些方法甚至不适用于涉及几十个变量的应用, 更不用说常常出现的涉及数百万输入的应用了, 比如图像处理. 问题在于, 基函数是提前固定的, 不依赖于数据, 甚至不依赖于正在解决的特定问题. 我们需要找到一种方法来创建能够针对特定应用加以调整的基函数.

虽然维度诅咒确实为机器学习应用带来了严重问题, 但这并不妨碍我们找到适用于高维空间的有效技术. 这背后的一个原因是, 真实数据通常会被限制在具有较低有效维度的数据空间区域内. 考虑图 6.7中的图像.

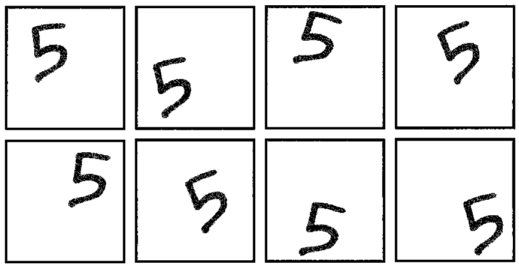


图 6.7: 手写数字图像的示例. 在这些图像中, 数字在图像内的位置以及它们的朝向有所不同. 这些图像存在于高维图像空间中的一个非线性三维流形上

其中的每幅图像都是高维空间中的一个点, 其维度由像素数决定. 因为对象可以在图像内不同的垂直和水平位置以不同的朝向出现, 所以图像之间有三个可变自由度. 一组图像将大致处在高维空间中的一个三维**流形** (manifold) 上. 由于对象位置或方向与像素强度之间存在复杂关系, 这个流形是高度非线性的.

定义 6.2 (数据流形)

设数据空间为 $\mathbb{R}^D$. 若存在整数 $d \ll D$ 与映射 $\boldsymbol{g}: \mathbb{R}^d \to \mathbb{R}^D$, 使得 (至少局部地) 每个数据点都可写成 $\boldsymbol{x} = \boldsymbol{g}(\boldsymbol{z})$, 其中 $\boldsymbol{z} \in \mathbb{R}^d$, 则称数据位于一个 d 维**流形** (manifold) 上,d 称为该流形的**内在维度** (intrinsic dimension),$\boldsymbol{z}$ 称为流形坐标. ♣

注

(重构新增)定义 6.2的关键在于区分外在维度 D(像素数、采样点数) 与内在维度 d(真正的自由度). 手写数字的例子中 D 可以是 10^4 量级而 $d = 3$(水平位置、垂直位置、朝向). 流形假设并不要求数据严格落在流形上, 只要求它集中在流形附近; 实际数据总带有噪声, 因此更准确的说法是“数据分布集中在低维流形的一个管状邻域内”.

实际上, 像素数量是图像生成过程的人为产物, 因为像素数量是对连续世界的测量方式. 以更高的分辨率捕获相同的图像会增加数据空间的维度 D, 但并不改变图像仍然存在于三维流形上这一事实. 如果我们能将局部化的基函数与数据流形相关联, 而不是与整个高维数据空间相关联, 则所需的基函数数量将随着流形的维度呈指数级增长, 而不是随着数据空间的维度呈指数级增长. 由于流形的维度通常比数据空间的维度低得多, 这将是一个巨大的改进. 实际上, 神经网络学习了一组适应于数据流形的基函数. 此外, 对于特定的应用, 并不是流形内的所有方向都是重要的. 例如, 如果我们只希望确定图 6.7中对象的朝向而不是位置, 那么在流形上将只有一个而不是三个相关的自由度. 神经网络还能够学习流形上的哪些朝向与预测想要的输出是相关的.

另一种看到真实数据被限制在低维流形上的方式是考虑随机图像生成任务.图 6.8展示了自然图像的示例以及相同分辨率的合成图像的示例.

这些合成图像是通过在每个像素独立随机地从均匀分布中采样红色、绿色和蓝色强度得到的. 可以看到, 没有任何一幅合成图像看起来像自然图像. 原因是这些合成图像缺乏自然图像所展示的像素之间的强相关性. 例如, 自然图像与合成图像相比, 两个相邻像素具有相同或非常相似的颜色的概率要高得多.图 6.8中的每幅图像对应于高维空间中的一个点, 然而自然图像只覆盖了这个空间的一少部分.

6.1.4 数据依赖的基函数

我们已经看到, 独立于所解决问题而选择的简单基函数在高维空间中可能具有严重的局限性. 如果我们想在这种情况下使用基函数, 那么一种方法是使用专家知识手工设计针对每个应用的特定基函数. 多年来, 这是机器学习中的主流方法. 基函数通常称为**特征**, 是通过结合领域知识和试错法而确定的. 这种方法取得了一定的成功, 但是数据驱动的方法替代了它. 在数据驱动的方法中, 基函数是从训练数据中学习得来的. 正如我们将在后面的章节中所看到的, 领域知识在现代机器学习中仍然发挥作用, 但是更多地体现在设计网络架构的定性层面上, 以此来建模适当的**归纳偏置** (inductive bias).

由于高维空间中的数据可能被限制在低维流形上, 因此我们不需要基函数密集地填充整个输入空间, 而是

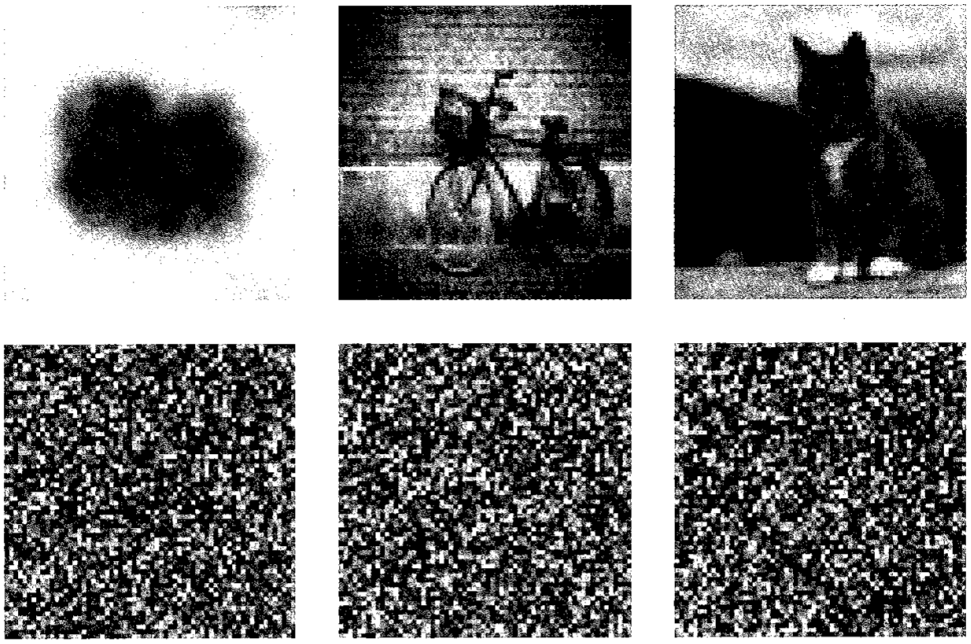


图 6.8: 第 1 行展示了 64×64 像素大小的自然图像的示例, 第 2 行展示了相同大小的随机生成图像 (即合成图像) 的示例. 第 2 行图像是通过从包含所有可能像素颜色的均匀概率分布中采样像素值得到的

可以使用与数据流形本身相关的基函数. 一种做法是将每个训练集中的数据点都与一个基函数相关联, 这确保了基函数自动适应于底层的数据流形. 这种模型的一个例子是**径向基函数** (radial basis function)(Broomhead and Lowe,1988), 其性质是每个基函数仅依赖于到中心向量的径向距离 (通常是欧氏距离). 如果基中心选择为输入数据值 $\{\boldsymbol{x}_n\}$, 那么对于每个数据点都有一个基函数 $\phi_n(\boldsymbol{x})$, 它将建模整个数据流形. 径向基函数的一个典型选择是

$$\phi_n(\boldsymbol{x}) = \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}_n\|^2}{s^2}\right), \tag{6.10}$$

其中 s 是控制基函数宽度的参数.

定义 6.3 (径向基函数)

设 $\{\boldsymbol{x}_n\}_{n=1}^N \subset \mathbb{R}^D$ 为训练输入,$s > 0$ 为宽度参数. 称(6.10)定义的 $\phi_n(\boldsymbol{x})$ 为以 $\boldsymbol{x}_n$ 为中心的**径向基函数** (radial basis function, RBF): 它的取值只依赖于 $\boldsymbol{x}$ 到中心 $\boldsymbol{x}_n$ 的欧氏距离 $\|\boldsymbol{x} - \boldsymbol{x}_n\|$. ♣

注

(重构新增)定义 6.3把“基函数应随数据分布走”这一想法落实为最直接的构造: 每个训练点放一个基函数. 它的优点是不需要先验地选择基函数形状, 缺点是基函数个数等于训练点个数 N, 且每个测试点的预测需要计算 N 个指数函数——计算量为 $O(ND)$. 更重要的是, 当 N 很大时, 模型参数与样本数同阶, 必须配合正则化才能避免过拟合. 这正是支持向量机试图解决的问题.

尽管可以很快建立起这样的模型, 但这种技术的一个主要问题是, 它对于大型数据集来说计算较为困难. 此外, 这样的模型还需要经过仔细的正则化以避免严重的过拟合问题.

有一种相关的方法, 叫作**支持向量机** (Support Vector Machine, SVM)(Vapnik,1995;Schölkopf and Smola,2002;Bishop,2006), 它通过再次定义以每个训练数据点为中心的基函数, 在训练过程中自动选择这些基函数的一个子集. 因此, 最终模型中有效基函数的数量通常远少于训练点的数量, 尽管它通常仍然相对较大, 并且会随训练集大小的增加而增大. 支持向量机不产生概率化的输出, 因此无法自然地泛化到多类情况. 像径向基函数和支持向量机这样的方法已被深度神经网络取代, 后者在有效利用非常大的数据集方面要好得多. 此外, 正如我们稍后将看到的, 神经网络能够学习深度层次化 (hierarchical) 的表示, 这对于在更复杂的应用中实现高预测

准确性至关重要.

定义 6.4 (支持向量机)

称如下分类器为**支持向量机** (SVM): 它以核函数 $k(\boldsymbol{x},\boldsymbol{x}_n)$(等价地, 以每个训练点为中心的基函数) 展开决策函数

$$y(\boldsymbol{x}) = \sum_{n\in\mathcal{S}} \alpha_n t_n k(\boldsymbol{x},\boldsymbol{x}_n) + b, \tag{6.11}$$

其中 $t_n \in \{-1,+1\}$ 为类标签,$\mathcal{S}$ 是训练过程中自动选出的训练点子集 (支持向量),$\alpha_n > 0$.

♣

注

(重构新增) 与定义 6.3相比,定义 6.4的基函数集合仍然以训练点为中心, 但只有一部分系数 α_n 非零, 这就是“自动选择子集”的含义.SVM 的判别式只给出类别 (或到超平面的距离), 不给出 $p(C_k \mid \boldsymbol{x})$, 因此难以直接用于需要概率输出的下游任务, 也不能像 softmax 那样自然地给出多类后验——这两点是它被深度网络取代的重要原因之一.

6.2 多层网络

为了将形如(6.1)的线性模型应用于涉及大规模数据集和高维空间的问题, 我们需要找到一组针对所要解决问题的基函数. 神经网络背后的关键思想是选择具有可学习参数的基函数 $\phi_j(\boldsymbol{x})$, 然后在训练过程中调整这些参数以及系数 $\{w_j\}$. 我们可以通过使用基于梯度的优化方法最小化误差函数来优化整个模型 (参见节 7.1), 如随机梯度下降法, 其中误差函数是在模型的所有参数上共同定义的.

当然, 有许多方法可以构造参数化的非线性基函数. 但有一个关键要求, 就是这些函数必须是关于可学习参数的可微函数, 这样我们就可以应用基于梯度的优化方法. 最成功的选择是使用与(6.1)形式相同的基函数, 从而每个基函数本身是输入的线性组合的非线性函数, 其中线性组合中的系数是可学习参数. 注意, 这种构造显然可以递归扩展以给出一个具有多层的层次化模型, 这构成了深度神经网络的基础 (参见节 6.3).

考虑具有两层可学习参数的基本神经网络. 首先, 我们用以下形式构造输入变量 $x_1,\cdots,x_D$ 的 M 个线性组合:

$$a_j^{(1)} = \sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}, \tag{6.12}$$

其中 $j = 1,\cdots,M$, “(1)” 表示相应的参数位于神经网络的第一层. 参数 $w_{ji}^{(1)}$ 称为**权重** (weight), 参数 $w_{j0}^{(1)}$ 称为**偏置参数** (bias), 量 $a_j^{(1)}$ 则称为**预激活** (pre-activation)(参见小节 4.1.1). 然后, 每个量 a_j 通过一个可微的、非线性的激活函数 $h(\cdot)$ 被转换为

$$z_j^{(1)} = h\left(a_j^{(1)}\right), \tag{6.13}$$

它代表了(6.1)中基函数的输出. 在神经网络的上下文中, 这些基函数 $z_j^{(1)}$ 称为**隐藏单元** (hidden unit). 我们将很快探索非线性函数 $h(\cdot)$ 的各种选择, 但在这里我们注意到, 只要导数 $h'(\cdot)$ 是可以计算的, 整个网络函数就将是可微的. 遵循(6.1), 这些值可以再次被线性组合为

$$a_k^{(2)} = \sum_{j=1}^{M} w_{kj}^{(2)} z_j^{(1)} + w_{k0}^{(2)}, \tag{6.14}$$

其中 $k = 1,\cdots,K$,K 是输出的总数. 这个转换对应于神经网络的第二层,$w_{k0}^{(2)}$ 同样是偏置参数. 最后,$a_k^{(2)}$ 则通过一个适当的输出单元激活函数 $f(\cdot)$ 被转换为一组网络输出 y_k. 两层的神经网络可以用图 6.9来表示.

定义 6.5 (两层神经网络)

由(6.12)、(6.13)与(6.14)依次复合而成的映射

$$y_k(\boldsymbol{x},\boldsymbol{w}) = f\left(\sum_{j=1}^{M} w_{kj}^{(2)}\, h\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i + w_{j0}^{(1)}\right) + w_{k0}^{(2)}\right), \qquad k = 1,\cdots,K \tag{6.15}$$

称为具有 M 个隐藏单元的**两层神经网络** (two-layer neural network). 集合 $\{w_{ji}^{(1)}, w_{j0}^{(1)}, w_{kj}^{(2)}, w_{k0}^{(2)}\}$ 统称为网络的**权重** (weights), 其中下标为 0 的项为偏置参数.

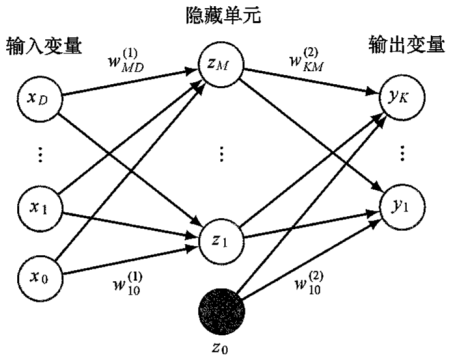


图 6.9: 两层神经网络的示意图. 输入变量、隐藏单元和输出变量由节点表示, 权重参数由节点之间的连接表示. 偏置参数由来自额外输入 x_0 和隐变量 z_0 的连接表示, 其中隐变量本身由实心节点表示. 箭头表示前向传播过程中信息流动的方向

注

(重构新增)定义 6.5中“层数”只计入可学习权重的层数: 输入层不参与计数, 因此带有一个隐藏层的网络称为两层网络. 这一点在本章后文讨论深度网络时会反复用到. 另一处需要留意的是记号: 偏置在这里记为 $w_{j0}^{(1)}$ 而不是单独的 b_j, 目的是让偏置与权重共享同一个下标结构, 从而在小节 6.2.1中自然地并入矩阵.

6.2.1 参数矩阵

我们在讨论线性回归模型时谈到 (参见小节 4.1.1), 通过定义一个额外的值固定为 1 的输入变量 x_0, 可以将(6.12)中的偏置参数吸收到权重参数集合中, 使(6.12)变为

$$a_j = \sum_{i=0}^{D} w_{ji}^{(1)} x_i. \tag{6.16}$$

定义 6.6 (增广输入)

在输入向量中追加分量 $x_0 \triangleq 1$, 得到 $\boldsymbol{x} = (x_0, x_1, \cdots, x_D)^{\mathsf{T}}$, 称为**增广输入** (augmented input). 类似地, 在隐藏单元中追加 $z_0 \triangleq 1$, 可把第二层的偏置 $w_{k0}^{(2)}$ 吸收进第二层权重.

注

(重构新增) 增广输入只是一个记号技巧, 不引入任何新的表达能力: 常数输入 $x_0 = 1$ 对应的权重就是原来的偏置. 它的好处是让(6.16)成为一个纯粹的线性变换, 从而可以写成矩阵乘法; 代价是每次写维度时要记得把 D 换成 $D+1$. 本书在推导反向传播和讨论正则化时都采用这一约定, 因此不应把 x_0 视为真正的数据特征, 也不应对其权重施加与数据特征相同的正则化.

我们同样也可以将第二层的偏置参数吸收到第二层的权重参数集合中, 于是整个网络函数变为

$$y_k(\boldsymbol{x}, \boldsymbol{w}) = f\left(\sum_{j=0}^{M} w_{kj}^{(2)} h\left(\sum_{i=0}^{D} w_{ji}^{(1)} x_i\right)\right). \tag{6.17}$$

一种方便的表示法是将输入表示为一个列向量 $\boldsymbol{x} = (x_1, \cdots, x_N)^{\mathsf{T}}$, 然后将(6.17)中的权重和偏置参数聚集成矩阵, 得到

$$\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{w}) = f\left(\boldsymbol{W}^{(2)} h\left(\boldsymbol{W}^{(1)} \boldsymbol{x}\right)\right), \tag{6.18}$$

其中 $f(\cdot)$ 和 $h(\cdot)$ 分别用于在每个向量元素上求值.

命题 6.5 (偏置吸收的等价性)

设 $\boldsymbol{W}^{(1)} \in \mathbb{R}^{M\times(D+1)}$ 的第一列取为原网络的偏置 $w_{j0}^{(1)}$, 其余列为 $w_{ji}^{(1)}$, 并令 $\boldsymbol{x} = (1, x_1, \cdots, x_D)^{\mathsf{T}}$, 则(6.17)与(6.15)定义同一个网络函数. 若再令 $\boldsymbol{W}^{(2)}$ 的第一列取为 $w_{k0}^{(2)}$, 并对隐藏单元追加 $z_0 = 1$, 则(6.18)与(6.17)等价.

♠

证明

(重构新增) 由矩阵乘法的定义, $\left(\boldsymbol{W}^{(1)}\boldsymbol{x}\right)_j = \sum_{i=0}^{D} w_{ji}^{(1)} x_i$, 这正是(6.16)的右端; 逐元素作用 h 后得到 $z_j^{(1)}$. 再左乘 $\boldsymbol{W}^{(2)}$ 得 $\sum_{j=0}^{M} w_{kj}^{(2)} z_j^{(1)}$, 其中 $z_0 = 1$ 的项即偏置 $w_{k0}^{(2)}$. 最后逐元素作用 f 即得(6.17). 由于每一步都是恒等变形, 两个网络对任意输入给出相同的输出.

□

注

(重构新增) 把网络写成(6.18)之后, 一次前向传播只需要两次矩阵–向量乘法, 计算量为 $O(MD + MK)$, 并且可以整批数据并行处理: 把 N 个输入按列排成 $D \times N$ 的矩阵 $\boldsymbol{X}$, 则 $\boldsymbol{Z}^{(1)} = h(\boldsymbol{W}^{(1)}\boldsymbol{X})$ 一次算出所有样本的隐藏单元. 这是深度网络能利用 GPU 的前提, 也是小节 6.3.7中“张量”概念的雏形.

6.2.2 通用近似

图 6.10展示了两层神经网络近似 4 个不同函数的能力. 该图还展示了各个隐藏单元是如何协作从而近似最终函数的.图 6.11说明了简单分类问题中隐藏单元的作用.

20 世纪 80 年代, 两层前馈网络的近似性质得到了广泛研究. 各种定理表明, 对于大多数激活函数而言, 这样的网络可以以任意精度近似定义在 $\mathbb{R}^D$ 的连续子集上的任何函数 (Funahashi,1989;Cybenko,1989;Hornik,Stinchcombe,and White,1989;Leshno et al.,1993). 对于从任何有限维离散空间到另一个空间的映射函数, 也有类似的结论. 因此, 神经网络可以视为**通用近似器** (universal approximator).

定理 6.1 (通用近似定理)

设 $K \subset \mathbb{R}^D$ 为紧集, $h : \mathbb{R} \to \mathbb{R}$ 连续且不是多项式 (例如 sigmoid、tanh、ReLU). 则对任意连续函数 $f : K \to \mathbb{R}$

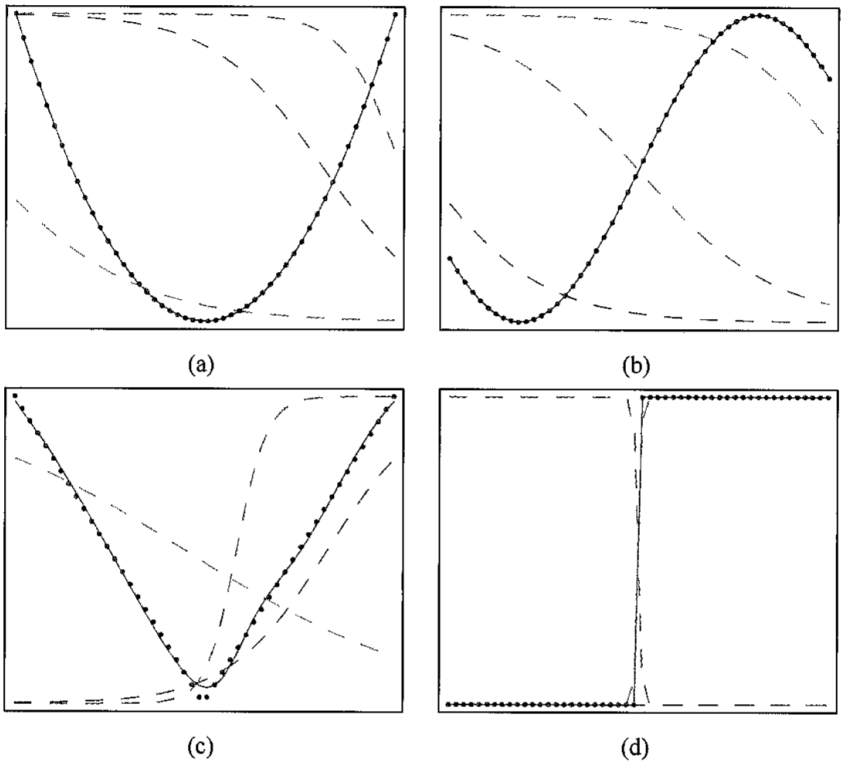


图 6.10: 两层神经网络近似 4 个不同函数的能力示意图.(a) $f(x) = x^2$;(b) $f(x) = \sin(x)$;(c) $f(x) = |x|$;(d) $f(x) = H(x)$, 其中 $H(x)$ 是 Heaviside 阶跃函数. 在每种情况下, 对 x 在 $(-1, 1)$ 区间均匀采样 $N = 50$ 个数据点 (以蓝点显示), 并评估相应的 $f(x)$ 的值. 然后使用这些数据点训练一个具有三个隐藏单元的两层神经网络, 该网络具有 tanh 激活函数和线性输出单元. 得到的网络函数由红色曲线显示, 三个隐藏单元的输出由三条虚线显示

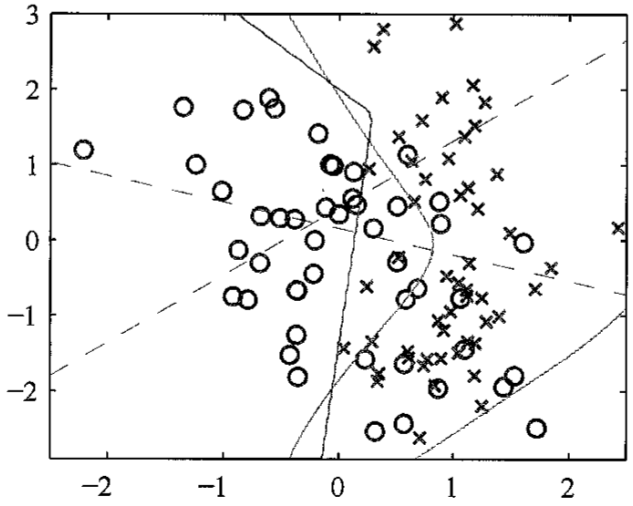


图 6.11: 使用神经网络解决在合成数据上进行简单二分类的示例. 该神经网络包含两个输入、两个由 tanh 函数激活的隐藏单元, 以及一个由 sigmoid 函数激活的输出. 蓝色虚线显示了每个隐藏单元 $z = 0.5$ 的等值线, 红色实线显示了网络 $y = 0.5$ 的决策面, 绿色实线表示从用于生成数据的分布中计算出的最优决策边界

与任意 $\varepsilon>0$, 存在整数 M、实数 w_j、$\boldsymbol{u}_j\in\mathbb{R}^D$ 与 b_j, 使得

$$\sup_{\boldsymbol{x}\in K}\left|f(\boldsymbol{x})-\sum_{j=1}^{M}w_j\,h\!\left(\boldsymbol{u}_j^{\top}\boldsymbol{x}+b_j\right)\right|<\varepsilon. \tag{6.19}$$

换言之, 具有一个隐藏层、激活函数为 h 的两层网络在 $(C(K),\|\cdot\|_\infty)$ 中稠密.

♡

证明

(重构新增) 先证 $D=1$、$K=[0,1]$、$h=\tanh$ 的情形, 再说明一般情形.

(i) 用两个隐藏单元构造区间指示函数. 对 $a<b$ 与 $\lambda>0$, 令

$$g_\lambda(x)=\frac{1}{2}\left[\tanh\big(\lambda(x-a)\big)-\tanh\big(\lambda(x-b)\big)\right].$$

当 $x\in(a,b)$ 时两项分别趋于 $+1$ 与 -1, 故 $g_\lambda(x)\to 1$; 当 $x\notin[a,b]$ 时两项趋于同一个值, 故 $g_\lambda(x)\to 0$. 并且在 $[0,1]\setminus(a-\delta,b+\delta)$ 上该收敛是一致的, 收敛速度为 $O(e^{-\lambda\delta})$.

(ii) 用阶梯函数逼近连续函数. 设 f 在 $[0,1]$ 上连续, 取 m 等分点 $x_i=i/m$, 令阶梯函数 $s_m(x)=\sum_{i=1}^{m}f(x_i)\,\mathbf{1}_{(x_{i-1},x_i]}(x)$, 则由一致连续性,$\|f-s_m\|_\infty\leqslant\omega_f(1/m)$, 其中 ω_f 是 f 的连续模, 且 $\omega_f(1/m)\to 0$.

(iii) 组合. 把 s_m 中的每个指示函数用上式替换, 得到形如(6.19)的网络函数 $\hat f$, 其隐藏单元数为 $2m$、输出权重为 $f(x_i)$. 于是

$$\|f-\hat f\|_\infty\leqslant\omega_f(1/m)+2\|f\|_\infty\,m\,e^{-\lambda\delta}. \tag{6.20}$$

先取 m 使第一项小于 $\varepsilon/2$, 再取 λ 充分大使第二项小于 $\varepsilon/2$, 即得 $D=1$ 的结论.

(iv) 一般 D 与一般 h. 多元情形不能直接沿用上述区间构造, 但结论仍成立. 证明的关键是 Cybenko 的判别准则: 若形如(6.19)的函数族在 $C(K)$ 中不稠密, 则由 Hahn–Banach 定理与 Riesz 表示定理, 存在非零有限符号测度 μ 使 $\int h(\boldsymbol{u}^{\top}\boldsymbol{x}+b)\,\mathrm{d}\mu(\boldsymbol{x})=0$ 对所有 $\boldsymbol{u},b$ 成立; 再令 $\boldsymbol{u}$ 的尺度趋于无穷并利用 h 的单调性 (sigmoid 型), 可推出 μ 在一切半空间上的取值为零, 从而 $\mu=0$, 矛盾.Leshno 等 (1993) 进一步把条件减弱为 “h 连续且不是多项式”. 故(6.19)对一般的 D 与上述 h 均成立.

□

注

(重构新增)定理 6.1只刻画表达能力: 单隐藏层网络的线性组合在紧集上可以逼近任意连续函数, 但它对所需的宽度 M 没有任何定量控制, 也不保证基于梯度的优化能找到这些权重. 证明中 “$D=1$ 的显式构造 + Cybenko 判别准则” 的结构也说明了结论为何对几乎任何非常数连续激活函数都成立: 起作用的不是 h 的具体形状, 而是它与平移、伸缩复合后张成的函数族的稠密性.

定义 6.7 (通用近似器)

若某类函数族 $\mathcal{F}$ 在紧集上的连续函数空间 $(C(K),\|\cdot\|_\infty)$ 中稠密, 即对任意 $f\in C(K)$ 与 $\varepsilon>0$ 都存在 $g\in\mathcal{F}$ 使 $\|f-g\|_\infty<\varepsilon$, 则称该函数族为**通用近似器** (universal approximator).定理 6.1说明, 单隐藏层的两层网络 (在适当激活函数下) 是通用近似器.

♣

注

(重构新增)定理 6.1是一个存在性定理, 它有三个容易被误读的地方. 第一, 它只保证存在某个宽度 M, 并未给出 M 的大小; 对某些函数,M 必须随 D 指数增长 (见后文关于分段线性区域数的讨论). 第二, 它没有说明梯度下降能否找到这组权重: 误差函数关于权重是非凸的, 定理对此不置一词. 第三, 稠密性是在紧集 K 上、以一致范数度量的; 在无界区域或要求导数也逼近时, 结论不再成立. 因此 “通用近似” 并不能推出 “深度网络可以解决任意问题”, 它只排除了表达能力上的根本障碍.

虽然这样的定理让我们安心, 但现有研究只告诉我们存在可以表示所需函数的网络. 在某些情况下, 它们可能需要指数级数量规模的隐藏单元. 更重要的是, 它们没有说明这样的网络是否可以通过学习算法找到. 此外, 我们

永远无法找到一个真正通用的机器学习算法 (参见小节 9.1.2). 最后, 尽管具有两层权重的网络是通用近似器, 但在实际应用中, 远超两层的网络可能带来巨大的好处, 因为其可以学习分层的内部表示. 以上所有这些论点都支持了深度学习的发展.

6.2.3 隐藏单元激活函数

你已经看到, 输出单元的激活函数由被建模分布的类型决定. 然而, 对隐藏单元的唯一要求是, 它们需要是可微的, 这为网络结构设计留下了广泛的可能性. 虽然原则上不同部分的网络可以应用不同的激活函数, 但大多数情况下, 网络中所有的隐藏单元将被赋予相同的激活函数.

最简单的隐藏单元激活函数是恒等函数, 这意味着所有的隐藏单元都是线性的. 然而, 对于任何这样的网络, 我们总能找到一个没有隐藏单元的等价网络. 这是因为连续线性变换的组合本身也是线性变换, 所以其表示能力不会强于单个线性层. 然而, 如果隐藏单元的数量少于输入或输出单元的数量, 那么此类网络所能生成的变换将不是从输入到输出最一般可能的线性变换, 因为隐藏单元处的降维丢失了信息. 考虑一个有 N 个输入、M 个隐藏单元和 K 个输出的网络, 其中所有激活函数都是线性的. 这样的网络有 $M(N+K)$ 个参数, 而直接将输入转换为输出的线性变换有 NK 个参数. 如果 M 相对于 N 或 K 较小, 或者相对于它们都小, 这将得到一个比直接的线性映射包含更少参数的两层线性网络, 对应一个秩亏变换. 这种线性单元的 “瓶颈” 网络对应一种标准的数据分析技术, 称为主成分分析 (Principal Component Analysis, PCA)(参见节 16.1). 然而, 一般来说, 使用线性单元的多层网络意义有限, 因为用其计算的整体函数仍然是线性的.

命题 6.6 (线性隐藏单元的等价性)

若两层网络的隐藏单元激活函数 h 为恒等函数, 则(6.15)等价于单个线性映射 $\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}$, 其中 $\boldsymbol{W} = \boldsymbol{W}^{(2)}\boldsymbol{W}^{(1)}$. 当 $M < \min(D, K)$ 时,$\operatorname{rank}\boldsymbol{W} \leqslant M$, 即该网络只能实现秩不超过 M 的线性变换, 其参数个数为 $M(D+K)$, 小于一般线性映射的 $(D+1)(K+1)$.

♠

证明

(重构新增) 恒等激活下 h 是线性映射, 故 $\boldsymbol{W}^{(2)}h(\boldsymbol{W}^{(1)}\boldsymbol{x}) = \boldsymbol{W}^{(2)}\boldsymbol{W}^{(1)}\boldsymbol{x}$, 即复合后仍是线性映射. 秩的不等式由 $\operatorname{rank}(\boldsymbol{AB}) \leqslant \min\{\operatorname{rank}\boldsymbol{A}, \operatorname{rank}\boldsymbol{B}\} \leqslant M$ 得到. 参数个数为 $MD + MK$(不含偏置) 或 $M(D+1) + M(K+1)$(含偏置), 在 $M < \min(D, K)$ 时小于一般线性映射的 $(D+1)(K+1)$.

□

注

(重构新增)命题 6.6说明: 多层线性网络的价值不在于增加表达能力 (它始终是线性的), 而在于参数压缩与低秩约束. 当 $M < \min(D, K)$ 时它是一个秩 M 的瓶颈, 其最优解在均方误差下由数据的主成分给出——这正是 PCA 的神经网络解释. 反过来, 只要激活函数是非线性的, 复合就不再塌缩为单层, 深度才真正带来表达能力的提升.

有一个简单的、非线性的、可微的函数, 叫作 sigmoid 激活函数, 由下式给出:

$$\sigma(a) = \frac{1}{1+\exp(-a)}. \tag{6.21}$$

图 6.12展示了多种非线性激活函数, 它们在早期的多层神经网络研究中使用广泛, 且部分受到生物神经元性质研究的启发. 一个与 sigmoid 激活函数密切相关的函数是 tanh 激活函数, 由下式给出:

$$\tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}. \tag{6.22}$$

定义 6.8 (sigmoid 与 tanh 激活函数)

称(6.21)定义的 $\sigma(a)$ 为 **sigmoid** 激活函数, 称(6.22)定义的 $\tanh(a)$ 为 **tanh** 激活函数 (双曲正切). 二者都把实轴映到有界区间:$0 < \sigma(a) < 1, -1 < \tanh(a) < 1$.

♣

注

(重构新增)sigmoid 与 tanh 的共同点是“饱和”: 当 $|a|$ 很大时函数值趋于常数, 导数趋于零, 这是小节 6.2.3后文所说的梯度消失问题的根源.tanh 相对 sigmoid 的优势在于输出关于原点对称、均值接近零, 因而隐藏单元的激活不会系统性地偏向某一侧; 在输入为零附近 tanh 的导数达到最大值 1, 而 sigmoid 的最大导数只有 1/4.

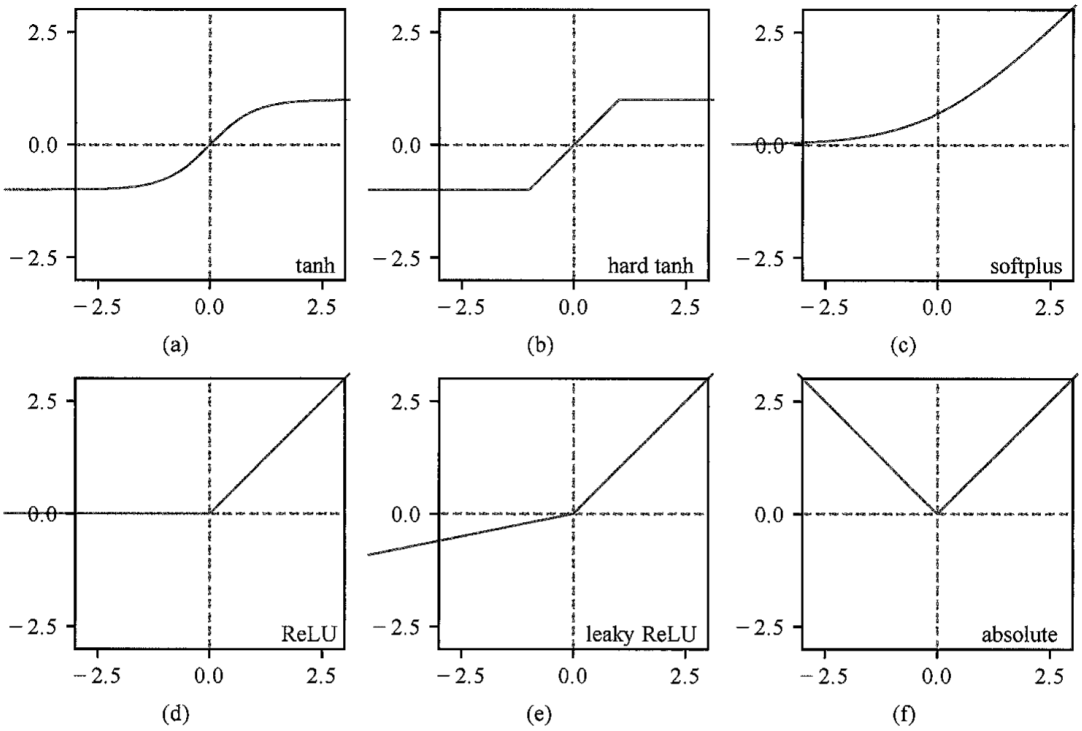


图 6.12: 多种非线性激活函数

如图 6.12(a) 所示,tanh 激活函数与 sigmoid 激活函数的不同之处在于输入和输出的线性变换, 因此对于任何具有 sigmoid 隐藏单元激活函数的网络, 都存在具有 tanh 激活函数的等价网络 (见练习 6.4). 然而在训练网络时, 这些并不一定是等价的, 因为对于基于梯度的优化方法, 需要初始化网络权重和偏置. 因此, 如果激活函数变了, 那么初始化方案也必须做相应调整.

命题 6.7 (tanh 网络与 sigmoid 网络的等价性)

对任意 $a \in \mathbb{R}$ 有

$$\tanh(a) = 2\sigma(2a) - 1. \tag{6.23}$$

从而对任何隐藏单元激活函数为 sigmoid 的两层网络(6.15), 都存在一个隐藏单元激活函数为 tanh、且对一切输入给出相同输出的两层网络; 二者的权重由线性变换互相转换.

证明

(重构新增) 由定义

$$2\sigma(2a) - 1 = \frac{2}{1+e^{-2a}} - 1 = \frac{1-e^{-2a}}{1+e^{-2a}} = \frac{e^{a}-e^{-a}}{e^{a}+e^{-a}} = \tanh(a). \tag{6.24}$$

设原网络第 j 个隐藏单元的预激活为 $a_j = \sum_i w_{ji}x_i + w_{j0}$, 输出为 $\sigma(a_j)$. 把该单元的输入权重与偏置同乘 2, 则新预激活为 $2a_j$, 由(6.23),$\tanh(2a_j) = 2\sigma(2a_j) - 1 = 2\sigma(a_j) - 1$. 于是新隐藏单元的输出等于原输出的仿射变换. 再把从该单元到输出层的所有权重乘以 1/2, 并在输出层偏置中补上相应的常数 (每个隐藏单元贡献 −1 的偏移), 即可完全抵消这一仿射变换, 使网络函数逐点相等.

□

注

(重构新增)命题 6.7是“函数层面等价、优化层面不等价”的典型例子: 两者的假设空间完全相同 (只差一个可逆的仿射重参数化), 但损失曲面在参数空间中的形状不同, 因此梯度下降的轨迹和收敛速度也不同. 这也解释了为什么更换激活函数时必须重新调整权重的初始化尺度——初始化分布应当与激活函数的“线性工作区”相匹配, 否则信号在深层网络中会逐层放大或衰减.

tanh 激活函数的 hard 版本 (Collobert,2024) 由下式给出:

$$h(a) = \max\big(-1, \min(1, a)\big). \tag{6.25}$$

函数曲线如图 6.12(b) 所示.

定义 6.9 (hard tanh)

称(6.25)定义的函数为 **hard tanh** 激活函数: 它在 $[-1, 1]$ 上是恒等映射, 在此区间外被截断为 ± 1.

注

(重构新增)hard tanh 是 tanh 的分段线性近似, 除 $a = \pm 1$ 两点外处处可微, 导数在 $(-1, 1)$ 内恒为 1、在外恒为 0. 与 tanh 相比它的计算只涉及比较与截断, 适合低精度硬件; 代价是区间外梯度恒为零, 因此同样存在“神经元失活”的风险.

sigmoid 激活函数和 tanh 激活函数的一个主要缺点, 是当输入具有很大的正值或很小的负值时, 梯度会呈指数级趋于零 (参见节 7.2). 我们稍后将讨论“梯度消失”问题, 就目前而言, 我们注意到使用非零梯度的激活函数通常会更好, 至少当输入为很大的正值时是这样. 其中一个激活函数的选择是 softplus 激活函数 (见练习 6.7), 由下式给出:

$$h(a) = \ln\big(1 + \exp(a)\big). \tag{6.26}$$

函数曲线如图 6.12(c) 所示. 对于 $a \gg 1$, 有 $h(a) \approx a$, 因此即使当 softplus 激活函数的输入很大且为正时, 梯度也仍然非零, 从而有助于缓解梯度消失问题.

定义 6.10 (softplus)

称(6.26)定义的函数为 **softplus** 激活函数. 它满足 $h'(a) = \sigma(a)$, 且在 $a \to -\infty$ 时 $h(a) \to 0$、在 $a \to +\infty$ 时 $h(a) \approx a$.

注

(重构新增)softplus 可以看作 ReLU 的光滑版本: 它处处可微, 且在 a 很大时与 ReLU 的差别只有指数小的一项. 它的导数恰好是 sigmoid, 因此在 a 很负时导数仍然很小 (趋于 0), 只是不像 sigmoid/tanh 那样双侧饱和——正半轴上梯度恒为 1, 这正是它缓解梯度消失的原因. 代价是需要计算指数与对数, 运算量高于 ReLU.

另一个更简单的选择是 ReLU(Rectified Linear Unit) 激活函数, 由下式给出:

$$h(a) = \max(0, a). \tag{6.27}$$

函数曲线如图 6.12(d) 所示. 从经验上看, 它是表现最好的激活函数, 得到了广泛使用. 注意, 严格来说,ReLU 激活函数的导数在 $a = 0$ 处是未定义的, 但实践中这可以安全地忽略.softplus 激活函数 [(6.26)] 可以视为 ReLU 激活函数的平滑版本, 因此有时也称 soft ReLU 激活函数 (见练习 6.5).

定义 6.11 (ReLU)

称(6.27)定义的函数为**修正线性单元** (Rectified Linear Unit, ReLU) 激活函数. 它在 $a > 0$ 时取 a、在 $a \leqslant 0$ 时取 0, 除 $a = 0$ 外处处可微, 且 $h'(a) = \mathbf{1}\{a > 0\}$.

注

(重构新增)ReLU 的非零梯度在正半轴上恒为 1, 因此深网络中梯度不会因激活函数本身而逐层衰减; 同时它的计算只需一次比较, 且对权重的随机初始化不敏感. 它的代价是负半轴梯度恒为零: 若某个单元的预激活在训练中长期为负, 该单元将不再接收任何误差信号 (“死亡 ReLU”). 这一缺陷正是下一段 leaky ReLU 的动机.

尽管 ReLU 激活函数对于正的输入值有非零梯度, 但对于负的输入值却不是这样, 这意味着某些隐藏单元在训练期间可能无法接收到“误差信号”. 避免该问题的一个办法是使用 ReLU 激活函数的修改版, 称为 leaky ReLU 激活函数, 由下式给出:

$$h(a) = \max(0, a) + \alpha \min(0, a), \tag{6.28}$$

其中 $0 < \alpha < 1$. 函数曲线如图 6.12(e) 所示. 与 ReLU 激活函数不同, 这个激活函数对于 $a < 0$ 的输入值有非零梯度, 以确保有信号驱动训练. 这个激活函数的一个变体使用 $\alpha = -1$, 在这种情况下,$h(a) = |a|$, 如图 6.12(f) 所示; 另一个变体则允许每个隐藏单元有自己的值 α_j, 从而可以在网络训练期间通过计算关于 α_j 以及权重和偏置的梯度来学习.

定义 6.12 (leaky ReLU)

称(6.28)定义的函数为 **leaky ReLU** 激活函数, 其中 $\alpha \in (0, 1)$ 为泄漏系数. 其导数为 $h'(a) = 1(a > 0)$ 与 $h'(a) = \alpha(a < 0)$.

♣

注

(重构新增)leaky ReLU 用一条斜率 α 很小的直线替换了 ReLU 在负半轴的零值, 从而保证每个单元始终有非零梯度.$\alpha = -1$ 的特例给出 $h(a) = |a|$, 它在原点不可微; 若允许 α_j 逐单元学习, 则相当于把“泄漏多少”也交给优化过程决定. 需要强调, 这些变体在实践中的收益依赖于任务与数据, 并非总是优于 ReLU.

ReLU 激活函数的引入在训练效率上较之前的 sigmoid 激活函数有了巨大提升 (Krizhevsky,Sutskever,and Hinton,2012). 除了能够高效地训练更深的网络外, 它对权重的随机初始化也不太敏感. 它非常适合低精度计算, 如 8 位定点数而非 64 位浮点数, 并且计算成本较低. 在许多实际应用中,ReLU 激活函数是默认选择, 除非明确地需要探索不同激活函数选择的效果.

6.2.4 权重空间的对称性

前馈网络的一个性质是, 多个不同的权重向量 $\boldsymbol{w}$ 的选择都可以产生相同的从输入到输出的映射函数 (Chen,Lu,and Hecht-Nielsen,1993). 考虑图 6.9所示的两层神经网络, 其具有 M 个由 tanh 函数激活的隐藏单元, 每一层都是全连接结构. 如果我们改变输入特定隐藏单元的所有权重和偏置的符号, 那么对于给定的输入数据点, 隐藏单元的预激活的符号将被反转, 从而激活也将被反转, 这是因为 tanh 函数是一个奇函数, 所以 $\tanh(-a) = -\tanh(a)$. 这种转变可以通过改变从隐藏单元输出的所有权重的符号来准确补偿. 因此, 通过改变一个特定组的权重 (和偏置) 符号, 网络所代表的输入–输出映射函数将保持不变. 我们找到了两个不同的权重向量, 它们可以产生相同的映射函数. 对于 M 个隐藏单元, 将存在 M 种这样的“符号翻转”对称性, 因此任何给定的权重向量都将是 2^M 个等价权重向量的集合中的一个.

类似地, 假设现在我们将输入到和输出自特定隐藏单元的所有权重 (和偏置) 值与不同隐藏单元所关联的权重 (和偏置) 值交换. 同样, 这显然没有改变网络的输入–输出映射函数, 但它对应于权重向量的不同选择. 对于 M 个隐藏单元, 任何给定的权重向量都将属于具有这种交换对称性的 $M \times (M-1) \times \cdots \times 2 \times 1 = M!$ 个等价权重向量的集合, 对应于隐藏单元的 $M!$ 种不同排序. 因此, 网络将有一个权重空间的对称性因子 $M!2^M$. 对于具有超过两层权重的网络, 总对称性水平将由这样的因子的乘积给出, 每一个隐藏单元层对应一个这样的因子.

定理 6.2 (权重空间的对称性)

考虑(6.15)定义的两层网络, 设所有隐藏单元使用奇激活函数 h(例如 $h = \tanh$ 或 $h(a) = a$). 对任意符号向量 $\boldsymbol{s} = (s_1, \cdots, s_M) \in \{\pm 1\}^M$ 与任意置换 $\pi \in S_M$, 定义新权重

$$\tilde{w}^{(1)}_{\pi(j)i} = s_j w^{(1)}_{ji}, \quad \tilde{w}^{(1)}_{\pi(j)0} = s_j w^{(1)}_{j0}, \quad \tilde{w}^{(2)}_{k\pi(j)} = s_j w^{(2)}_{kj}, \quad \tilde{w}^{(2)}_{k0} = w^{(2)}_{k0}, \tag{6.29}$$

则新网络与原网络定义同一个输入–输出映射. 若权重处于一般位置 (各隐藏单元的预激活两两不同、且没有任何单元恒为零), 则这 $M!\,2^M$ 个权重向量互不相同, 即对称群的作用是自由的.

♡

证明

(重构新增) 记原网络第 j 个隐藏单元的输出为 $z_j = h(a_j)$. 由(6.29), 新网络的第 $\pi(j)$ 个单元的预激活为

$$\tilde{a}_{\pi(j)} = \sum_i s_j w^{(1)}_{ji} x_i + s_j w^{(1)}_{j0} = s_j a_j, \tag{6.30}$$

故由 h 为奇函数得 $\tilde{z}_{\pi(j)} = h(s_j a_j) = s_j h(a_j) = s_j z_j$. 于是新网络第 k 个输出为

$$\tilde{y}_k = f\left(\sum_j \tilde{w}^{(2)}_{k\pi(j)} \tilde{z}_{\pi(j)} + w^{(2)}_{k0}\right) = f\left(\sum_j s_j w^{(2)}_{kj} \cdot s_j z_j + w^{(2)}_{k0}\right) = f\left(\sum_j w^{(2)}_{kj} z_j + w^{(2)}_{k0}\right) = y_k, \tag{6.31}$$

因为 $s_j^2 = 1$. 故映射不变. 至于自由性: 若两个变换 $(\boldsymbol{s}, \pi)$ 与 $(\boldsymbol{s}', \pi')$ 给出同一权重向量, 则对每个 j 必有 $\pi(j) = \pi'(j)$ 且 $s_j = s'_j$(否则某个权重分量或某个单元的输出会变号, 而在一般位置下预激活非零、各单元可区分), 故只有恒等变换才使权重不变.

□

注

(重构新增)定理 6.2解释了为什么损失曲面在权重空间中存在大量完全等价的“副本”: 对称群把参数空间分成大小为 $M!\,2^M$ 的轨道, 轨道上每一点的损失值相同. 这对优化本身影响不大 (找到一个副本就够了), 但对 Bayes 方法影响很大: 在计算参数的后验分布或模型证据时, 必须把这些等价模式正确地计入, 否则会重复计数 (Bishop,2006). 此外, 该结论不依赖于 tanh 的具体形式, 只依赖于 h 为奇函数; 对非奇激活函数 (如 sigmoid), 符号翻转不再是对称性, 但置换对称性仍然成立 (Kurková and Kainen,1994).

事实证实, 这些因子解释了权重空间中所有的对称性 (权重值的特定选择可能导致的偶然对称性除外). 此外, 这些对称性的存在并不是 tanh 激活函数的特定性质, 而是适用于大部分激活函数 (Kurková and Kainen,1994). 一般来说, 权重空间中的这些对称性在实践中几乎没有什么用, 因为网络训练旨在找到参数的特定设置, 而其他等价设置的存在几乎没有什么用. 然而, 当使用 Bayes 方法评估不同规模网络的概率分布时, 权重空间的对称性确实发挥了作用 (Bishop,2006).

6.3 深度网络

我们通过使用可学习参数控制线性回归模型或分类模型的基函数, 阐述了发展神经网络的动机, 从而产生了图 6.9所示的两层神经网络. 多年来, 它一直是最广为使用的架构——主要是因为很难有效训练超过两层的网络. 然而, 我们将很快讨论到, 将神经网络扩展到两层以上会带来许多优势, 并且最近神经网络训练技术的发展对多层网络是有效的 (参见节 7.1).

我们可以轻松地将两层网络架构 [(6.18)] 扩展到任意有限层数 L, 其中第 $l = 1, \cdots, L$ 层计算以下函数:

$$\boldsymbol{z}^{(l)} = h^{(l)}\left(\boldsymbol{W}^{(l)} \boldsymbol{z}^{(l-1)}\right), \tag{6.32}$$

其中 $h^{(l)}$ 表示与第 l 层相关联的激活函数,$\boldsymbol{W}^{(l)}$ 表示相应的权重和偏置参数矩阵. 此外,$\boldsymbol{z}^{(0)} = \boldsymbol{x}$ 表示输入向量,$\boldsymbol{z}^{(L)} = \boldsymbol{y}$ 表示输出向量.

定义 6.13 (深度神经网络)

称由(6.32)递推定义、以 $z^{(0)} = x$ 为输入、以 $z^{(L)} = y$ 为输出的复合映射为具有 L 层可学习权重的**深度神经网络** (deep neural network). 当 $L > 2$ 时通常称为**深度**网络, 其层数 L 称为网络**深度** (depth), 各层向量的维数称为该层的**宽度** (width).

♣

注

(重构新增)定义 6.13把“深度”定义为可学习权重的层数, 与定义 6.5的约定一致: 输入层和输出层的激活函数本身不是权重层, 但输出层的变换仍计入 L. 由此,图 6.9的网络是 $L = 2$ 的特例. 深度与宽度的区分很重要:“深”指的是复合的次数,“宽”指的是单层单元数; 后文的通用近似讨论关心的是宽度, 而表达效率的讨论关心的是深度.

注意, 此类网络中计算层数的术语存在一些混乱. 因此,图 6.9所示的网络有时描述为“三层网络”(对单元的层数进行计数, 并将输入视为单元), 有时描述为“单隐藏层网络”(对隐藏单元的层数进行计数). 建议将图 6.9所示的网络称为“两层网络”, 因为可学习权重的层数对于网络的性质是十分重要的.

我们已经看到,图 6.9所示的具有两层可学习参数的网络具有通用近似能力. 然而, 具有超过两层的网络有时可以用比两层网络少得多的参数来表示给定的函数.Montúfar 等人 (2014) 指出, 网络函数将输入空间划分成了多个区域, 这些区域在网络深度上是指数级的, 但在隐藏层的宽度上却是多项式级的. 使用两层网络表示相同的函数需要指数级数量规模的隐藏单元.

命题 6.8 (分段线性区域数随深度指数增长)

考虑使用 ReLU 激活的全连接网络, 其输入维数为 D、每层宽度为 W、深度为 L. 则该网络把输入空间划分成的线性区域数 $\mathcal{R}$ 满足

$$\mathcal{R} = O\left(W^{DL}\right), \tag{6.33}$$

即对固定的输入维数 D,$\mathcal{R}$ 关于宽度 W 是 DL 次多项式、关于深度 L 是指数级; 若改用深度为 2 的网络表示同一个分段线性函数, 则所需隐藏单元数在最坏情况下随 L 指数增长.

♠

证明

(重构新增, 思路) 每增加一层, 该层的超平面集会把上一层的每个线性区域继续细分, 故区域数至多乘以该层能产生的细分份数; 逐层累乘即得 Montúfar 等人 (2014) 给出的上界 $O(W^{DL})$, 其中指数含 D 与 L. 反之, 深度为 2 的网络只有一层隐藏单元 M 个超平面, 它把 $\mathbb{R}^D$ 划分成的区域数至多按 M^D 增长; 若要达到 L 层网络所能产生的 W^{DL} 个区域, 就必须 $M^D \gtrsim W^{DL}$, 即 $M \gtrsim W^L$, 随 L 指数增长. 严格的计数论证见原文及其中引用的组合几何结果.

□

注

(重构新增)命题 6.8给出了“深度为何有用”的一个纯表达能力的论据: 在参数量相同的情况下, 深网络能表达的分段线性函数比浅网络多得多. 但要注意这是最坏情形下的计数结论, 并不说明实际任务一定需要这些区域, 也不说明梯度下降能找到它们; 它只说明存在需要深度才能高效表示的函数. 真正让深度在实践中胜出的, 是下一小节讨论的层次化归纳偏置.

6.3.1 层次化表示

虽然这是一个有趣的结果, 但另一个更令人信服地探索深度神经网络的理由是, 网络架构编码了一种特定形式的归纳偏置, 即输出空间与输入空间之间通过层次化表示相关联. 这方面一个很好的例子是识别图像中物体的任务 (参见节 10.1). 图像的像素与高级概念 (比如“猫”) 之间的关系非常复杂且非线性, 对于两层网络而言, 这将是一个极具挑战性的识别任务. 然而, 深度神经网络会首先学习在浅层中检测低级特征 (比如边缘), 然后在后续层中将它们结合起来以得到更高级的特征 (比如眼睛或胡须), 之后继续在深层中将它们组合起来以检测猫的存在.

定义 6.14 (层次化表示)

若一个 L 层网络的第 l 层输出 $\boldsymbol{z}^{(l)}$ 可以理解为对第 $l-1$ 层特征的一次组合, 即存在 (在训练后学到的) 映射使

$$\boldsymbol{z}^{(l)} = \boldsymbol{h}^{(l)} \circ \boldsymbol{g}^{(l)}\left(\boldsymbol{z}^{(l-1)}\right), \qquad l = 1, \cdots, L, \tag{6.34}$$

且低层特征在输入空间中具有较小的感受野、高层特征由低层特征组合而成, 则称该网络学习到了**层次化表示** (hierarchical representation). 相应的归纳偏置称为**组合** (compositional) 归纳偏置.

♣

注

(重构新增)定义 6.14中的“组合”不是数学上的新运算, 而是对网络复合结构的语义解释: 它把“输出是输入的复杂非线性函数”这一事实, 替换成“输出是若干简单函数的逐层复合”. 这一归纳偏置的有效性依赖于数据本身具有层次结构——自然图像、语音、文本都满足这一点; 若数据没有这种结构 (例如各特征之间完全独立), 深度带来的先验反而可能有害.

这可以看作一种组合 (compositional) 归纳偏置, 其中更高级的对象 (比如猫) 由低级对象 (比如眼睛) 组成, 而这些低级对象又有着更低级的要素 (比如边缘). 我们也可以从相反的角度来考虑图像生成过程, 即从低级特征 (比如边缘) 开始, 然后将这些组合起来形成简单的形状 (比如圆), 之后进一步将这些形状组合起来形成更高级的对象 (比如猫). 在每个阶段都有许多不同的方式组合不同组件, 随着深度的增加, 可能性的数量呈指数级增长.

6.3.2 分布式表示

神经网络可以利用另一种称为**分布式表示** (distributed representation) 的组合形式. 在概念上, 隐藏层中的每个单元都可以视为表示了网络在该层的一个“特征”, 激活值高表示相应的特征存在, 激活值低表示相应的特征不存在. 当给定层中有 M 个单元时, 网络可以表示 M 个不同的特征. 然而, 网络也可能会学习到不同的表示, 其中隐藏单元的组合表示特征, 从而使具有 M 个单元的隐藏层能够表示 2^M 个不同的特征, 其随着单元数量的增加呈指数增长.

定义 6.15 (分布式表示)

若某层 M 个单元中的每一个都编码一个独立的属性, 且该层所表示的“概念”由这些属性的组合给出, 则称该层使用了**分布式表示** (distributed representation). 此时该层可区分的概念数为 2^M, 而若采用“每个概念一个单元”的局部表示 (local representation), 同样的 M 个单元只能表示 M 个概念.

♣

注

(重构新增) 分布式表示的核心是“用 M 个二值单元编码 2^M 种组合”, 这与用 M 位二进制数编码 2^M 个整数是同一件事. 它的代价是: 单个单元的含义不再是“某个概念存在”, 而是“某个属性存在”, 因此表示的可解释性下降; 好处是样本效率与泛化能力显著提高——从未见过的组合可以由已知属性的新搭配自然得到. 深度网络的隐藏层通常同时具有层次化与分布式两种结构.

例如, 考虑一个用于处理人脸图像的网络. 每个特定的人脸图像可能有 (或没有) 眼镜, 有 (或没有) 帽子, 有 (或没有) 胡须, 从而导致 8 种不同的组合. 尽管这可以用 8 个单元来表示, 当检测到相应的组合时对应的单元会“打开”, 但也可以更紧凑地只使用 3 个单元来表示, 每个单元代表一个属性. 这些属性可以独立存在 (尽管统计上它们的存在可能一定程度上相关). 稍后我们将详细探讨深度学习网络在训练过程中自行发现的内部表示类型 (参见节 10.1).

例题 6.3 人脸属性的分布式编码 设三个属性分别为“戴眼镜”“戴帽子”“有胡须”, 用三个二值单元 $z_1, z_2, z_3 \in \{0, 1\}$ 编码, 则 8 种外观对应 8 个不同的三元组. 若改用局部表示, 则需要 8 个单元、每个单元对应一种外观组合, 参数量与所需样本数都随属性数指数增长.

6.3.3 表示学习

我们可以将连续经过深度神经网络的不同层视为对数据进行转换, 以便更容易地完成任务. 例如, 成功学习将皮肤病变分类为良性或恶性 (参见小节 1.1.1) 的神经网络必须学会将原始图像数据转换为由最终隐藏单元层的输出表示的新空间, 以便网络的最终层可以区分这两类数据. 这个最终层可以视为一个简单的线性分类器, 因此在最后隐藏层的表示中, 这两类数据必须能被一个线性面很好地分开.

定义 6.16 (表示学习)

设 $z = \boldsymbol{\phi}_{\boldsymbol{\theta}}(\boldsymbol{x})$ 是由参数 $\boldsymbol{\theta}$ 控制的特征映射 (例如深度网络某个隐藏层的输出). 若 $\boldsymbol{\theta}$ 是通过在数据上优化某个目标而学到的, 且学到的映射使后续任务 (如分类、回归) 更容易完成, 则称这一过程为**表示学习** (representation learning),z 称为数据的一个**表示**.

♣

注

(重构新增)定义 6.16与传统的“特征工程”形成对照: 后者的 $\boldsymbol{\phi}$ 由人根据领域知识固定, 前者的 $\boldsymbol{\phi}$ 由数据决定. 表示学习并不神秘——它只是把“选择基函数”这一步也纳入优化. 其代价是模型不再凸、需要更多数据与更细致的正则化; 其收益是表示的可迁移性: 同一个 $\boldsymbol{\phi}$ 可以服务于多个下游任务 (见小节 6.3.4).

这种发现数据的非线性变换使后续任务更容易完成的能力称为**表示学习** (representation learning)(Bengio,Courville,and Vincent,2012). 学习到的表示有时称为**嵌入空间** (embedding space), 它们是由网络中一个隐藏层的输出给出的, 从而对于任何的输入向量, 无论是来自训练集还是来自某些新的数据集, 都可以通过网络的前向传播转换为这个表示.

定义 6.17 (嵌入空间)

设 $\boldsymbol{\phi} : \mathbb{R}^D \to \mathbb{R}^d$ 为某隐藏层定义的特征映射, 则称其像空间 $\{\boldsymbol{\phi}(\boldsymbol{x}) : \boldsymbol{x} \in \mathbb{R}^D\} \subseteq \mathbb{R}^d$ 为**嵌入空间** (embedding space),$\boldsymbol{\phi}(\boldsymbol{x})$ 称为 $\boldsymbol{x}$ 的**嵌入** (embedding).

♣

注

(重构新增) 嵌入空间的维数 d 通常远小于输入维数 D, 因此嵌入可以看作一种有监督或有约束的降维: 与 PCA 等线性降维不同, 嵌入的映射由任务目标驱动, 可以是非线性且高度任务相关的. 也正因如此, 同一个数据集针对不同任务会得到不同的嵌入空间, 嵌入之间没有跨任务的统一坐标意义.

表示学习异常强大的原因是, 它使得我们能够利用无标签数据. 通常, 收集大量的无标签数据很容易, 但获取相关的标签可能更加困难. 例如, 当车辆行驶在城市周边时, 车载摄像头可以收集大量城市场景的图像, 但是获取这些图像并识别相关的物体, 如行人和路标, 却需要进行成本高昂且耗时的人工标注.

定义 6.18 (无监督学习)

只使用输入数据 $\{\boldsymbol{x}_n\}$、不使用目标值 $\{t_n\}$ 的学习方式称为**无监督学习** (unsupervised learning). 与之相对, 使用成对 $(\boldsymbol{x}_n, t_n)$ 的学习方式称为**监督学习** (supervised learning).

♣

注

(重构新增) 无监督学习之所以对表示学习重要, 是因为它把“标签”这一稀缺资源替换成了数据自身的结构 (相关性、可压缩性、可预测性). 但也应看到: 无监督目标只是表示学习的代理目标, 最小化重构误差并不保证得到的表示对下游分类有用; 实践中往往需要在下游任务上微调, 或设计与下游任务更匹配的自监督目标 (如小节 6.3.5).

从无标签数据中学习称为**无监督学习** (unsupervised learning), 人们已经开发出许多不同的算法来实现这一点. 例如, 可以训练神经网络将图像作为输入并创建相同的图像作为输出. 为了使其成为一项具有挑战性的任务, 网络可能会使用比图像中像素数量更少的隐藏层单元数, 从而要求网络学习对图像进行某种压缩. 只需要无标签数据的原因是, 训练集中的每个图像都同时充当输入向量和目标向量. 这种网络又称为**自编码器** (参见节 19.1). 这种训练的目标是使网络发现一些对解决其他任务 (比如图像分类) 有用的数据的内部表示.

定义 6.19 (自编码器)

设编码器 $z = \phi(x)$ 与解码器 $\hat{x} = \psi(z)$ 均由神经网络实现, 且 $\dim z < \dim x$. 以重构误差

$$E(\theta, \phi) = \sum_{n=1}^{N} \left\| x_n - \psi\left(\phi(x_n)\right) \right\|^2 \tag{6.35}$$

为目标的网络称为**自编码器** (autoencoder), 其中 z 称为**编码** (code) 或潜表示.

♣

注

(重构新增)定义 6.19中的瓶颈 $\dim z < \dim x$ 是迫使网络学习有用表示的关键: 若容量不受限制, 网络可以学会恒等映射而毫无收获. 自编码器与 PCA 的关系是: 当编码器与解码器都取线性、误差取平方和时, 最优解张成的子空间正是数据的主成分子空间 (参见节 16.1); 非线性自编码器则给出 PCA 的非线性推广.

无监督学习在成功训练第一个深度网络 (卷积网络除外) 方面扮演了重要的角色. 网络的每一层首先使用无监督学习进行预训练, 然后整个网络进一步使用基于梯度的监督训练方式来训练. 人们后来发现预训练阶段可以省略, 并且在适当的条件下, 可以纯粹使用监督学习从头开始训练深度网络.

然而, 在讨论其他内容时, 预训练和表示学习仍然是深度学习的核心. 最值得一提的预训练是在自然语言处理中进行的 (参见节 12.1),Transformer 模型在大量文本上进行训练的, 从而学习到了高度复杂的语言内部表示, 促进其各项能力达到甚至超越人类水平.

6.3.4 迁移学习

从一个特定任务学习到的内部表示也可能对相关的任务有帮助. 例如, 在一个大型有标签的常见对象数据集上训练的网络可以学习到如何将图像表示转换为更适合于对象分类的表示. 然后可以使用一个较小的有标签数据集对网络的最终分类层重新进行训练, 以创建一个病变分类器 (参见小节 1.1.1). 这是**迁移学习** (transfer learning)(Hospedales et al.,2021) 的一个示例, 它相较于仅使用病变图像数据训练可以获得更高的准确率, 因为网络可以利用自然图像的通用共性. 迁移学习如图 6.13所示.

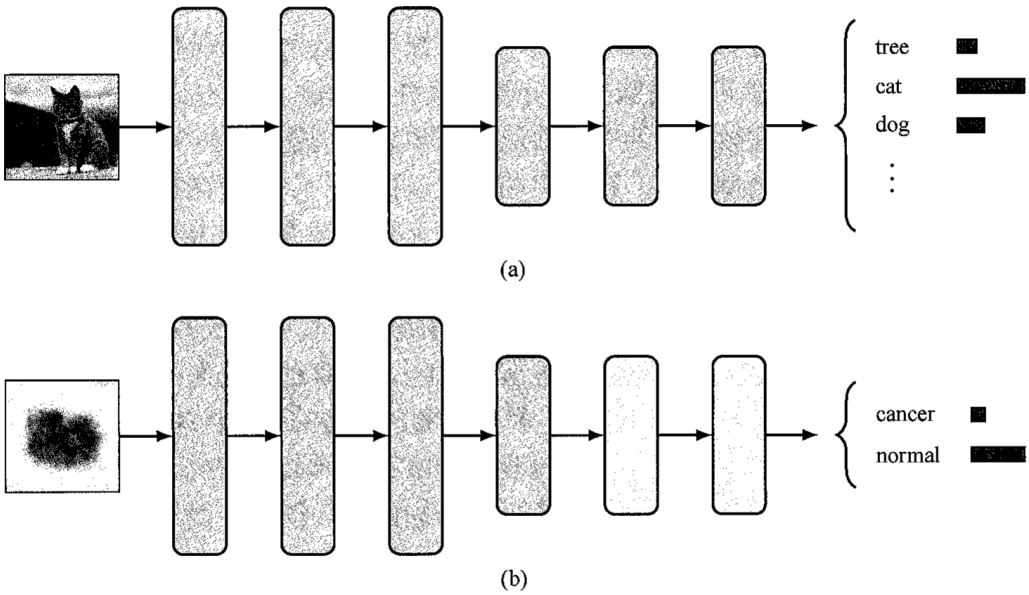


图 6.13: 迁移学习的示意图.(a) 首先在具有丰富数据的任务上对网络进行训练, 比如自然图像的对象分类.(b) 网络的早期层 (以红色表示) 从第一个任务中复制, 然后对网络的最后几层 (以蓝色表示) 在新任务上重新进行训练, 比如训练数据更稀缺的皮肤病变分类

定义 6.20 (迁移学习)

设任务 A 的有标签数据稀少, 任务 B 的有标签数据丰富, 且两任务的输入类型相同. 若把在任务 B 上学到的表示或参数用于任务 A 的训练, 从而提升 A 的性能, 则称这一过程为**迁移学习** (transfer learning). 其中任务 B 称为**源任务** (source task), 任务 A 称为**目标任务** (target task).

♣

注

(重构新增) 迁移学习能够成立的前提是两任务共享某些低层结构: 图像任务的边缘、纹理, 语音任务的频谱包络, 文本任务的词法模式, 都具有跨任务的通用性; 而高层语义 (“猫”与“皮肤病变”) 则不通用. 因此, 迁移的通常做法是固定早期层、只重训后几层, 这与定义 6.14中“浅层学通用特征、深层学任务特异特征”的图景完全一致.

一般来说, 迁移学习可以通过使用来自相关任务 B 的数据来提升某个任务 A 的性能, 其中任务 A 的训练数据较为匮乏, 而任务 B 的训练数据则更为丰富. 这两个任务应该具有相同类型的输入, 并且这两个任务之间应该存在一些共性, 使得从任务 B 学到的低级特征或内部表示对任务 A 有帮助. 当我们研究卷积网络 (参见节 10.1) 时, 我们会发现许多图像处理任务在深度神经网络的早期层中需要相似的低级特征, 而后继层则专门用于特定任务, 因此这样的网络非常适合用于迁移学习应用.

当任务 A 的训练数据非常稀缺时, 我们可能只需要简单地重新训练网络的最终层. 相比之下, 如果训练数据更多, 则重新训练多个层也是可行的. 使用一个任务学习参数, 然后应用于一个或多个其他任务的过程称为**预训练** (pre-training). 需要注意的是, 对于新任务, 相较于对整个网络应用随机梯度下降, 更高效的方式是将新的训练数据一次通过参数固定的预训练网络, 以便在新的表示中评估训练输入. 然后在仅包含最终层的较小网络上应用基于迭代梯度的优化. 除了使用预训练网络作为不同任务的固定预处理器之外, 也可以应用**微调** (fine-tuning), 即将整个网络适配到任务 A 的训练数据上. 这通常是通过非常小的学习率和有限次迭代来完成的, 以确保网络不会过拟合于新任务中相对较小的可用数据集.

定义 6.21 (预训练与微调)

在源任务上训练好参数 $\boldsymbol{\theta}_B$ 后, 将其作为目标任务 A 的初始参数 (或固定为特征提取器) 的过程称为**预训练** (pre-training). 若在任务 A 的数据上以较小的学习率继续更新全部 (或大部分) 参数, 则称为**微调** (fine-tuning).

♣

注

(重构新增)定义 6.21的关键区别在于是否更新预训练参数: 固定参数时, 预训练网络只是一个固定的非线性特征映射, 训练成本低、不易过拟合; 微调时, 网络可以调整内部表示以适配新任务, 但小数据集上容易过拟合, 因此需要小学习率与早停. 两者的选择取决于目标任务的数据量: 数据越少, 越倾向于只重训最后一层.

有一个相关的方法叫作**多任务学习** (multitask learning)(Caruana,1997), 其中网络同时联合学习多个相关任务. 例如, 我们可能希望建立一个垃圾邮件过滤器, 其允许不同用户有不同的分类器以适应他们各自的偏好. 训练数据可能包括许多不同用户的垃圾邮件和非垃圾邮件的样本, 但是任何一个用户的样本数量都可能相当有限, 因此为每个用户训练一个单独的分类器可能效果不佳. 相反, 我们可以将数据集组合并训练一个单一的较大网络, 该网络可以共享早期层, 但对于不同用户, 后续层具有单独的可学习参数. 跨任务共享数据允许网络利用任务之间的共性, 从而提高对所有用户的准确率. 大量的训练样本使得我们能够使用具有更多参数的更深层网络, 这可以进一步提高性能.

定义 6.22 (多任务学习)

设共有 T 个任务, 第 t 个任务的训练数据为 $\mathcal{D}_t$. 若网络由共享的底层表示 $\boldsymbol{\phi}$ 与任务专用层 $\{\boldsymbol{\psi}_t\}$ 组成, 并通过最小化

$$E(\boldsymbol{\theta}) = \sum_{t=1}^{T} \sum_{(\boldsymbol{x},t)\in\mathcal{D}_t} \ell_t\big(\boldsymbol{\psi}_t(\boldsymbol{\phi}(\boldsymbol{x})), t\big) \tag{6.36}$$

联合训练, 则称该过程为**多任务学习** (multitask learning), 其中 ℓ_t 是第 t 个任务的损失函数.

♣

注

(重构新增) 多任务学习与迁移学习的区别在于“谁是主、谁是辅”: 迁移学习有一个明确的目标任务, 源任务只是手段; 多任务学习则同时优化所有任务, 共享表示必须在所有任务上都表现良好. 这在数据不平衡时会产生“负迁移”的风险——某个任务的数据量过大时, 共享表示会被它主导. 实践中常通过对各任务损失加权或使用梯度归一化来缓解.

跨多个任务学习可以扩展到**元学习** (meta-learning), 也称学会学习 (learning to learn). 多任务学习旨在对一组固定任务进行预测, 而元学习的目标是对我们在训练期间未见过的未来任务进行预测. 这不仅可以通过跨任务学习共享的内部表示来实现, 还可以通过学习学习算法本身来实现 (Hospedales et al.,2021). 例如, 当新类的有标签样本非常少时, 元学习可以用于促进分类模型泛化到新类上, 称为**小样本学习** (few-shot learning). 当只使用一个有标签样本时, 则称为**单样本学习** (one-shot learning).

定义 6.23 (元学习)

设任务分布为 $p(\mathcal{T})$, 每个任务 $\mathcal{T}$ 带有训练集 $\mathcal{D}_{\mathcal{T}}^{\text{tr}}$ 与测试集 $\mathcal{D}_{\mathcal{T}}^{\text{te}}$. 若学习算法 $\mathcal{A}$ 通过在一批任务上优化

$$\min_{\mathcal{A}} \mathbb{E}_{\mathcal{T}\sim p(\mathcal{T})}\left[\mathcal{L}\left(\mathcal{A}(\mathcal{D}_{\mathcal{T}}^{\text{tr}}), \mathcal{D}_{\mathcal{T}}^{\text{te}}\right)\right] \tag{6.37}$$

而得到, 则称该过程为**元学习** (meta-learning). 若每个任务只提供 K 个 (或 1 个) 有标签样本, 则相应的问题称为 K **样本学习** (K-shot learning) 或**单样本学习** (one-shot learning).

注

(重构新增)定义 6.23与普通学习的差别在于优化的对象: 普通学习优化“模型参数”, 元学习优化“产生模型参数的算法”. 这也解释了为什么元学习需要“任务分布”: 只有在任务层面存在共同结构时, 学会学习才可能. 小样本学习的成功主要来自这种任务层面的先验, 而不是来自单个任务内的少量样本.

6.3.5 对比学习

对比学习 (contrastive learning) 是最常见和强大的表示学习方法之一 (Gutmann and Hyvärinen,2010;Oord,Li and Vinyals,2018;Chen,Kornblith,et al.,2020). 它背后的思想是学习一种表示, 使得在嵌入空间中, 某些输入对 (称为**正对**) 之间的距离近, 而其他输入对 (称为**负对**) 之间的距离远. 直觉上, 如果我们选择语义上相似的为正对, 并选择语义上不相似的为负对, 那么我们将学习到一个表示空间, 在这个表示空间中, 相似的输入之间的距离近, 从而使得完成下游任务 (比如分类) 变得更加容易. 与其他形式的表示学习一样, 我们通常不直接使用训练好的网络的输出, 而是使用某个早期层的激活来构建嵌入空间. 对比学习任务与其他大多数机器学习任务不同, 因为给定输入的误差函数仅取决于其他输入, 而非取决于每个输入单独的标签或目标输出.

定义 6.24 (对比学习)

设 $\boldsymbol{z} = f_{\boldsymbol{w}}(\boldsymbol{x})$ 为表示映射. 给定由正对 $(\boldsymbol{x}, \boldsymbol{x}^+)$ 与负对 $(\boldsymbol{x}, \boldsymbol{x}^-)$ 组成的训练数据, 若通过最小化某个只依赖于这些对之间表示距离的损失函数来学习 $\boldsymbol{w}$, 则称该学习方法为**对比学习** (contrastive learning).

注

(重构新增)定义 6.24的“误差函数只取决于其他输入”这一点值得强调: 它是自监督 (self-supervised) 学习的标志——标签不是人工给出的, 而是由数据本身的结构 (哪些样本应当相似) 构造出来的. 因此对比学习的关键设计不在损失函数, 而在“如何定义正对与负对”, 这等价于把关于“什么是不变因素”的先验知识编码进训练目标.

给定数据点 $\boldsymbol{x}$, 称为**锚点** (anchor), 然后指定另一个数据点 $\boldsymbol{x}^+$, 它与 $\boldsymbol{x}$ 一起组成一个正对. 我们还必须指定一组数据点 $\{\boldsymbol{x}_1^-, \cdots, \boldsymbol{x}_N^-\}$, 其中的每个数据点都与 $\boldsymbol{x}$ 组成一个负对. 我们现在需要一个损失函数, 如果 $\boldsymbol{x}$ 和 $\boldsymbol{x}^+$ 的表示较为接近, 则给予奖励, 同时拉大每一对 $\{\boldsymbol{x}, \boldsymbol{x}_n^-\}$ 之间的距离. 这种函数的一个例子, 也是对比学习中最常用的损失函数, 称为 InfoNCE 损失函数 (Gutmann and Hyvärinen,2010;Oord,Li and Vinyals,2018), 其中 NCE 表示“噪声对比估计” (noise contrastive estimation). 假设我们有一个神经网络函数 $f_{\boldsymbol{w}}(\boldsymbol{x})$, 用于将来自输入空间的点 $\boldsymbol{x}$ 映射到一个由可学习参数 $\boldsymbol{w}$ 控制的表示空间. 这个表示是归一化的, 即 $\|f_{\boldsymbol{w}}(\boldsymbol{x})\| = 1$. 对于数据点 $\boldsymbol{x}$,InfoNCE 损失函数定义

如下:

$$E(\boldsymbol{w}) = -\ln \frac{\exp\{f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}^+)\}}{\exp\{f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}^+)\} + \sum_{n=1}^{N} \exp\{f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}_n^-)\}}. \tag{6.38}$$

定义 6.25 (InfoNCE 损失)

设 $\|f_{\boldsymbol{w}}(\boldsymbol{x})\| = 1$, 温度参数 $\tau > 0$. 称

$$E(\boldsymbol{w}) = -\ln \frac{\exp\left(f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}^+)/\tau\right)}{\exp\left(f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}^+)/\tau\right) + \sum_{n=1}^{N} \exp\left(f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}_n^-)/\tau\right)} \tag{6.39}$$

为 **InfoNCE 损失** (information noise contrastive estimation loss), 其中 τ 控制分布的锐度. ♣

注

(重构新增)定义 6.25中温度 τ 的作用与统计物理中的温度一致:$\tau \to 0$ 时 softmax 趋于 one-hot, 损失只关心最难的负样本;τ 较大时损失对所有负样本一视同仁. 原书(6.38)的写法相当于 $\tau = 1$. 此外, 由于表示已归一化, 内积 $f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}^+)$ 就是余弦相似度, 取值范围为 $[-1, 1]$, 这使得相似度在不同样本之间可比.

在这个函数中, 锚点的表示 $f_{\boldsymbol{w}}(\boldsymbol{x})$ 和正样本的表示 $f_{\boldsymbol{w}}(\boldsymbol{x}^+)$ 之间的余弦相似度 $f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}^+)$ 提供了我们在习得空间中度量正对样本有多近的方法, 同样的方法也可以用于评估锚点与负对样本之间的接近程度. 注意这个函数类似于分类交叉熵误差函数, 其中正对的余弦相似度给出了标签类的对数概率, 而负对的余弦相似度给出了错误类的对数概率. 负对是非常重要的, 因为如果没有它们, 嵌入空间将简单地学习将每个点映射到相同的表示的退化解.

命题 6.9 (InfoNCE 是 $(N+1)$ 类交叉熵)

设 $\tau = 1$, 记 $s^+ = f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}^+), s_n^- = f_{\boldsymbol{w}}(\boldsymbol{x})^{\mathsf{T}} f_{\boldsymbol{w}}(\boldsymbol{x}_n^-)$, 并令 $p^+ = \mathrm{softmax}(s^+, s_1^-, \cdots, s_N^-)$ 的第一分量为正对的预测概率. 则(6.38)恰为以“正对为正确类、N 个负对为错误类”的 $(N+1)$ 类交叉熵. ♠

证明

(重构新增) 把 $(s^+, s_1^-, \cdots, s_N^-)$ 视为 $(N+1)$ 个类别的 logit,softmax 给出

$$p^+ = \frac{\exp(s^+)}{\exp(s^+) + \sum_{n=1}^{N} \exp(s_n^-)}. \tag{6.40}$$

正确类的 one-hot 目标为 $(1, 0, \cdots, 0)$, 故交叉熵为 $-\ln p^+$, 正是(6.38). 当 s^+ 相对所有 s_n^- 都很大时 $p^+ \to 1$、损失趋于 0; 当 s^+ 很小时损失增大.

□

注

(重构新增)命题 6.9说明 InfoNCE 并不是新奇的损失, 而是把“正样本应被认出来”表述成一个分类问题:N 个负样本就是 N 个干扰类. 由此可以立刻看出负样本为何不可缺少——若 $N = 0$, 损失恒为零, 网络可以学到常数表示. 也正因如此, 负样本的数量与质量决定了表示的好坏, 后续许多方法 (如记忆库、大批量训练) 都在设法增加有效负样本.

一个特定的对比学习算法主要由正对和负对的选择方式定义, 也就是如何使用先验知识来指定什么是好的表示. 例如, 考虑学习图像表示的问题. 在这里, 一个常见的选择是通过破坏输入图像来创建正对, 这种方式保留了图像的语义信息, 同时极大地改变了像素空间中的图像 (Wu et al.,2018;He et al.,2019;Chen,Kornblith,et al.,2020). 输入图像的破坏与**数据增强** (data augmentation)(参见节 9.1) 密切相关, 可以是旋转、平移和颜色转换. 然后数据集中的其他图像可以用来创建负对. 这种对比学习方法称为**个体判别** (instance discrimination).

定义 6.26 (个体判别)

若正对由同一幅图像的两种随机增强版本 $(\boldsymbol{x}, \mathcal{T}(\boldsymbol{x}))$ 构成、负对由数据集中其他图像构成, 则称该对比学习方案为**个体判别** (instance discrimination). 它不需要任何类别标签. ♣

注

(重构新增) 个体判别的有效性完全取决于增强方式的选择: 增强必须保持语义不变 (旋转、平移、颜色抖动), 否则网络会把语义不同的两个版本强行拉近, 学到错误的表示. 因此“选择哪些增强”实际上等价于“声明哪些变换不应改变表示”, 这是一种很强的先验, 也解释了个体判别在不同数据类型上效果差异很大的原因.

然而, 如果我们可以访问类标签, 则可以使用同一类别的图像作为正对, 而使用不同类别的图像作为负对. 这放宽了对指定表征应保持不变的增强方法的依赖, 并且避免了将两个语义相似的图像视为负对. 这称为**监督对比学习** (Khosla et al.,2020), 因为它依赖于类标签, 并且它通常能得到比仅使用交叉熵分类学习的表示更好的结果.

定义 6.27 (监督对比学习)

若正对由属于同一类别的两幅不同图像构成、负对由属于不同类别的图像构成, 则称该对比学习方案为**监督对比学习** (supervised contrastive learning).

注

(重构新增) 监督对比学习与普通交叉熵分类的差别在于损失作用的位置: 交叉熵直接约束输出层的类后验, 而对比损失约束的是嵌入空间中样本间的相对距离, 因此它对类内紧凑性有更强的要求. 代价是它需要成对采样, 计算量随批大小增长; 同时, 若数据集中存在标注噪声, 同一“类别”内的错误标签会被强行拉近, 伤害比交叉熵更明显.

正对和负对的成员不一定非要来自同一数据模态. 在对比语言–图像预训练 (Contrastive-Language Image Pre-training, CLIP)(Radford et al.,2021) 中, 一个正对由一幅图像和对应的文本说明组成. 用两个单独的函数将每个模态输入分别映射到相同的表示空间. 负对则由不匹配的图像和文本说明组成. 这通常称为**弱监督学习**, 因为它依赖于带文本说明的图像. 这些数据通常是从互联网上爬取的, 比手动标注图像类别更容易. 在这种情况下, 损失函数由下式给出:

$$
\begin{aligned}
E(\boldsymbol{w}) = &-\frac{1}{2}\ln\frac{\exp\left\{f_{\boldsymbol{w}}(\boldsymbol{x}^+)^{\mathsf{T}} g_{\boldsymbol{\theta}}(\boldsymbol{y}^+)\right\}}{\exp\left\{f_{\boldsymbol{w}}(\boldsymbol{x}^+)^{\mathsf{T}} g_{\boldsymbol{\theta}}(\boldsymbol{y}^+)\right\} + \sum_{n=1}^{N}\exp\left\{f_{\boldsymbol{w}}(\boldsymbol{x}_n^-)^{\mathsf{T}} g_{\boldsymbol{\theta}}(\boldsymbol{y}^+)\right\}} \\
&-\frac{1}{2}\ln\frac{\exp\left\{f_{\boldsymbol{w}}(\boldsymbol{x}^+)^{\mathsf{T}} g_{\boldsymbol{\theta}}(\boldsymbol{y}^+)\right\}}{\exp\left\{f_{\boldsymbol{w}}(\boldsymbol{x}^+)^{\mathsf{T}} g_{\boldsymbol{\theta}}(\boldsymbol{y}^+)\right\} + \sum_{m=1}^{M}\exp\left\{f_{\boldsymbol{w}}(\boldsymbol{x}^+)^{\mathsf{T}} g_{\boldsymbol{\theta}}(\boldsymbol{y}_m^-)\right\}},
\end{aligned}
\tag{6.41}
$$

其中 $\boldsymbol{x}^+$ 和 $\boldsymbol{y}^+$ 表示一个正对, 这里的 $\boldsymbol{x}$ 是一幅图像,$\boldsymbol{y}$ 是对应的文本说明,$f_{\boldsymbol{w}}$ 表示从图像到表示空间的映射, 而 $g_{\boldsymbol{\theta}}$ 则是从文本输入到表示空间的映射. 我们还需要从数据集中获取一组图像 $\{\boldsymbol{x}_1^-,\cdots,\boldsymbol{x}_N^-\}$, 并假设对它们来说, 文本说明 $\boldsymbol{y}^+$ 是不恰当的, 还需一组同样不匹配输入图像 $\boldsymbol{x}$ 的文本说明 $\{\boldsymbol{y}_1^-,\cdots,\boldsymbol{y}_M^-\}$. 损失函数中的两项确保了 (a) 图像的表示与对应的文本说明表示相对于其他图像表示而言是接近的, 以及 (b) 文本说明表示与它所描述的图像表示相对于其他文本说明表示而言是接近的. 虽然 CLIP 使用文本–图像对, 但任何具有配对模态的数据集都可以用来学习表示.图 6.14对我们讨论过的三种不同对比学习方法做了比较.

定义 6.28 (CLIP)

设图像编码器 $f_{\boldsymbol{w}}$ 与文本编码器 $g_{\boldsymbol{\theta}}$ 都把输入映到同一单位球面, 损失取(6.41), 则称该对比学习框架为**对比语言–图像预训练** (Contrastive-Language Image Pretraining, CLIP).

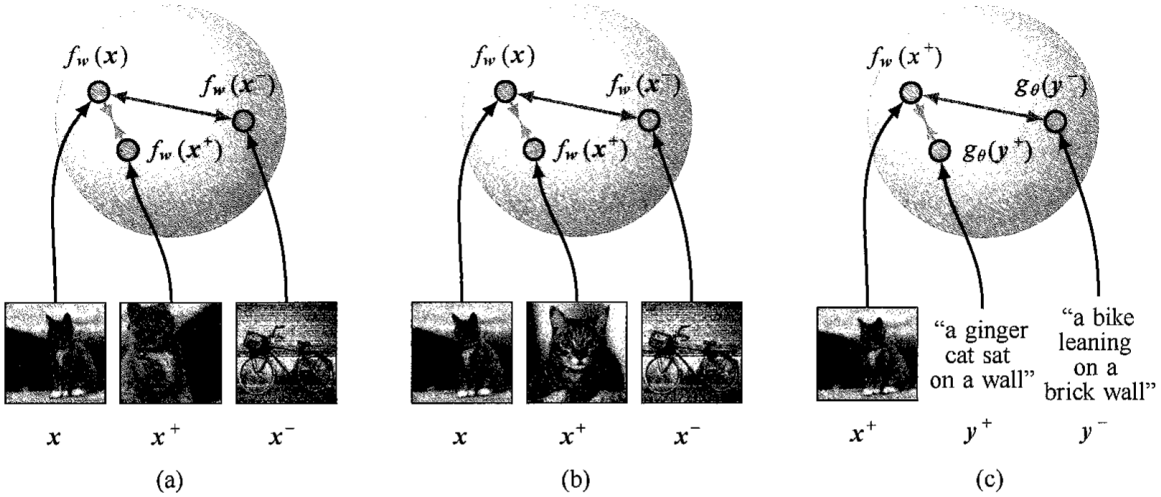


图 6.14: 对三种不同的对比学习方法的说明.(a) 个体判别方法, 其中正对由锚点和同一图像的增强版本组成. 它们会被映射到归一化空间 (可以认为是单位超球体) 中的点. 彩色箭头显示这种损失促使正对的表示更靠近, 但会将负对进一步分开.(b) 监督对比学习, 其中正对由来自同一类别的两幅不同图像组成.(c) CLIP 模型, 其中正对由一幅图像和与之关联的文本说明组成

注

(重构新增)定义 6.28中的两项互为“对称”的 InfoNCE: 第一项以图找文, 第二项以文找图, 因子 1/2 只是取平均. 这样设计的原因是对称化能避免模型只在一个方向上对齐 (例如把文本编码器退化成常数).(6.41)与(6.38)的关系是: 若把 $g_\theta(\boldsymbol{y}^+)$ 视为“正样本的表示”、把 $g_\theta(\boldsymbol{y}_m^-)$ 视为负样本, 则第二项就是(6.38); 第一项则是在图像侧做同样的构造.

6.3.6 通用网络结构

到目前为止, 我们已经探讨了组织成一系列全连接层的神经网络架构. 然而, 由于网络图与其数学函数之间存在直接对应关系, 我们可以通过考虑更复杂的网络图来设计更通用的网络映射. 这些网络必须限制为**前馈** (feed forward) 网络, 换句话说, 不能有闭合的有向环, 以确保输出是输入的确定性函数.图 6.15用一个简单的示例对此进行了说明. 在这样的网络中, 每个 (隐藏或输出) 单元计算的函数为

$$z_k = h\left(\sum_{j\in\mathcal{A}(k)} w_{kj} z_j + b_k\right), \tag{6.42}$$

其中 $\mathcal{A}(k)$ 表示节点 k 的祖先 (ancestor) 集, 换句话说, 就是指向单元 k 的单元所构成的集合, 而 b_k 表示相关联的偏置参数. 对于应用于网络输入的一组给定值, 连续应用(6.42)可以计算网络中所有单元的激活, 包括输出单元的激活.

定义 6.29 (前馈网络)

设网络的单元集合为 V, 有向边集合为 E, 边 $(j,k)\in E$ 表示单元 j 的输出送入单元 k. 若 (V,E) 作为有向图不含任何有向环, 则称该网络为**前馈网络** (feed-forward network). 此时单元 k 的**祖先集**定义为

$$\mathcal{A}(k) = \{j \in V : \text{存在从 } j \text{ 到 } k \text{ 的有向路径}\}. \tag{6.43}$$

注

(重构新增)定义 6.29中的“无有向环”不是为了简化, 而是为了保证 z_k 的定义是良定的: 若存在环,(6.42)就变成一个关于 $\{z_k\}$ 的隐式方程, 可能无解或多解, 输出也就不再是输入的确定性函数. 无环条件等价于存在一个拓扑序, 使得每个单元只依赖于排在它前面的单元, 这正是下一命题的内容.

命题 6.10 (前向传播的可计算性)

设网络为定义 6.29意义下的前馈网络. 则存在单元的一个排序 $k_1,\cdots,k_{|V|}$, 使得对每个 i,z_{k_i} 只依赖于输入 $\boldsymbol{x}$ 与 $\{z_{k_1},\cdots,z_{k_{i-1}}\}$. 因此给定 $\boldsymbol{x}$, 按该顺序依次应用(6.42)即可在 $O(|V|+|E|)$ 时间内算出所有单元 (包括输出) 的激活.

♠

证明

(重构新增) 有限无环有向图必存在拓扑序: 反复删除入度为零的节点即可, 每步至少存在一个这样的节点 (否则沿入边回溯必得环). 把删除顺序记为 $k_1,\cdots,k_{|V|}$, 则 k_i 的所有祖先都在 $\{k_1,\cdots,k_{i-1}\}$ 中, 故按序计算(6.42) 时右端各项已知. 每个单元计算一次、每条边贡献一次乘法, 总时间为 $O(|V|+|E|)$.

□

注

(重构新增)命题 6.10是“网络图即计算图”这一观点的形式化: 它不仅保证前向传播可行, 也说明后向传播可以按逆拓扑序进行 (参见节 8.1). 因此, 任意前馈拓扑 (卷积、残差、分支) 都可以用同一套自动微分机制训练, 网络设计的自由度只受“无环”这一条限制.

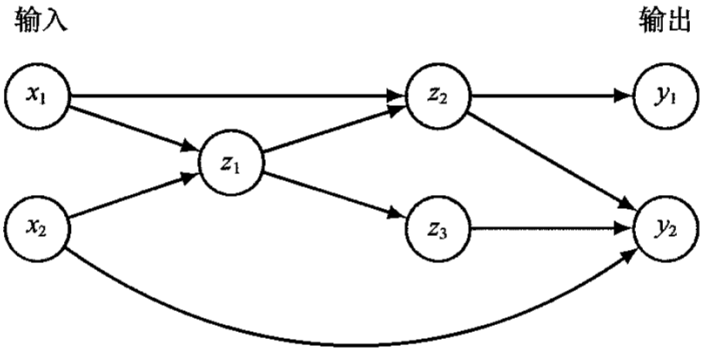


图 6.15: 具有一般前馈拓扑结构的神经网络示例. 注意每个隐藏单元和输出单元都有一个相关的偏置参数 (此处为简单起见而省略)

6.3.7 张量

可以看到, 线性代数在神经网络中发挥着核心作用, 其中数据集、激活和网络参数等表示为标量、向量和矩阵. 此外, 我们还遇到了更高维度的变量. 例如, 考虑一个包含 N 张彩色图像的数据集, 每张图像的高度为 I 像素, 宽度为 J 像素. 每个像素可通过其在图像内所处的行和列进行索引, 并具有红色、绿色和蓝色值. 对于数据集中的每个图像, 我们都有一个这样的值, 因此我们可以用一个四维数组 $\mathcal{X}$ 来表示特定的强度值, 数组中的元素表示为

$$x_{ijkn}, \qquad i\in\{1,\cdots,I\},\ j\in\{1,\cdots,J\},\ k\in\{1,2,3\},\ n\in\{1,\cdots,N\}, \tag{6.44}$$

其中 i 和 j 是图像内行和列的索引,$k\in\{1,2,3\}$ 是红色、绿色和蓝色强度的索引,$n\in\{1,\cdots,N\}$ 是数据集中特定图像的索引.

定义 6.30 (张量)

称多重指标数组 $(x_{i_1 i_2\cdots i_K})$ 为一个 K 阶**张量** (tensor), 其中每个指标 i_k 取值于有限集合.0 阶张量是标量,1 阶张量是向量,2 阶张量是矩阵.

♣

注

(重构新增)定义 6.30采用的是“多维数组”这一工程含义, 与微分几何中“张量是多重线性映射”的数学含义不同: 在深度学习中, 张量只是把数据按多个索引排布, 并约定各阶指标的语义 (样本、通道、行、列). 这一约定之所以重要, 是因为它决定了卷积、池化等算子在哪些指标上共享参数——这也是节 10.1的核心.

这些高维数组称为**张量** (tensor), 而标量、向量和矩阵是张量的特例. 等到后面讨论更为复杂的神经网络架构时, 我们会看到许多张量的例子. 像 GPU 这样的大规模并行处理器特别适合处理张量.

6.4 误差函数

在之前的章节中, 我们探讨了用于回归和分类的线性模型 (参见节 4.1), 并在此过程中推导出了误差函数的适当形式 (参见节 5.1) 以及相应的输出单元激活函数. 选择误差函数考虑的因素同样也适用于多层 (神经) 网络, 为了方便起见, 我们将在总结关键点.

6.4.1 回归

我们首先讨论回归问题. 此处我们暂时考虑可以取任意实值的单目标变量 t. 让我们回到对单层网络中回归的讨论 (参见小节 2.3.4), 假设 t 服从 Gauss 分布, 其均值是以 $\boldsymbol{x}$ 为输入的神经网络的输出, 则有

$$p(t \mid \boldsymbol{x}, \boldsymbol{w}) = \mathcal{N}\left(t \,\middle|\, y(\boldsymbol{x}, \boldsymbol{w}), \sigma^2\right), \tag{6.45}$$

其中 σ^2 是 Gauss 噪声的方差. 当然, 这是一个有点严格的假设, 在某些应用中, 我们需要扩展这个方法以支持更一般的分布 (参见节 6.5). 对于由(6.45)给出的条件分布, 将输出单元的激活函数取为恒等函数就足够了, 因为这样的网络可以近似从 $\boldsymbol{x}$ 到 y 的任意连续函数.

定义 6.31 (回归的条件 Gauss 模型)

若目标变量 $t \in \mathbb{R}$ 在给定输入 $\boldsymbol{x}$ 与网络参数 $\boldsymbol{w}$ 时的条件分布由(6.45)给出, 其中 $y(\boldsymbol{x}, \boldsymbol{w})$ 是神经网络的输出、$\sigma^2 > 0$ 是噪声方差, 则称该模型为回归的**条件 Gauss 模型** (conditional Gaussian model). ♣

注

(重构新增)定义 6.31把“网络输出”解释为条件均值而不是条件众数或中位数, 这一解释只在噪声对称 (此处为 Gauss) 时才与最小二乘一致. 若噪声是重尾的 (如 Laplace 分布), 条件均值不再是最优预测, 相应的误差函数也会从平方和变为绝对值和. 因此, 误差函数的选择本质上是对噪声分布的假设, 这是本节反复出现的主题.

给定一个由 N 个独立同分布的观测值 $\boldsymbol{X} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 和相应的目标值 $\boldsymbol{t} = \{t_1, \cdots, t_N\}$ 组成的数据集, 我们可以构造出相应的似然函数:

$$p(\boldsymbol{t} \mid \boldsymbol{X}, \boldsymbol{w}, \sigma^2) = \prod_{n=1}^{N} p\left(t_n \,\middle|\, y(\boldsymbol{x}_n, \boldsymbol{w}), \sigma^2\right). \tag{6.46}$$

值得注意的是, 在机器学习文献中, 通常考虑最小化误差函数而不是最大化似然函数, 所以此处我们将遵循这个惯例. 对似然函数(6.46)取负对数, 得到误差函数为

$$\frac{1}{2\sigma^2} \sum_{n=1}^{N} \left\{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\right\}^2 + \frac{N}{2} \ln \sigma^2 + \frac{N}{2} \ln(2\pi). \tag{6.47}$$

命题 6.11 (回归的负对数似然)

在定义 6.31与独立同分布假设下, 负对数似然 (忽略与参数无关的常数) 为(6.47), 即

$$-\ln p(\boldsymbol{t} \mid \boldsymbol{X}, \boldsymbol{w}, \sigma^2) = \frac{1}{2\sigma^2} \sum_{n=1}^{N} \{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\}^2 + \frac{N}{2} \ln \sigma^2 + \frac{N}{2} \ln(2\pi). \tag{6.48}$$

证明

(重构新增) 由 Gauss 密度 $\mathcal{N}(t \mid y, \sigma^2) = (2\pi\sigma^2)^{-1/2} \exp\{-(t-y)^2/(2\sigma^2)\}$, 逐个代入(6.46)并取负对数:

$$\begin{aligned} -\ln p &= \sum_{n=1}^{N} \left[\frac{1}{2} \ln(2\pi\sigma^2) + \frac{(y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n)^2}{2\sigma^2} \right] \\ &= \frac{1}{2\sigma^2} \sum_{n=1}^{N} \{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\}^2 + \frac{N}{2} \ln \sigma^2 + \frac{N}{2} \ln(2\pi), \end{aligned} \tag{6.49}$$

其中用到 $\sum_{n=1}^{N} \frac{1}{2} \ln(2\pi\sigma^2) = \frac{N}{2} \ln \sigma^2 + \frac{N}{2} \ln(2\pi)$.

□

注

(重构新增)(6.47)的后两项只依赖于 σ^2, 与 $\boldsymbol{w}$ 无关. 因此“对 $\boldsymbol{w}$ 最大化似然”与“最小化平方和误差”在 σ^2 固定时完全等价; 而当 σ^2 也被当作待估参数时, 两项会相互制衡——增大 σ^2 会减小第一项但增大第二项, 最优值恰为残差均方 (见命题 6.13).

它可以用来学习参数 $\boldsymbol{w}$ 和 σ^2. 首先考虑确定 $\boldsymbol{w}$. 最大化似然函数等价于最小化下式给出的平方和误差函数:

$$E(\boldsymbol{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\}^2. \tag{6.50}$$

命题 6.12 (最大似然等价于最小平方和)

在 σ^2 固定时,(6.46)的最大似然解与(6.50)的最小化解相同, 即

$$\boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} \frac{1}{2} \sum_{n=1}^{N} \{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\}^2. \tag{6.51}$$

证明

(重构新增) 由(6.47),$-\ln p$ 关于 $\boldsymbol{w}$ 只通过第一项依赖 $\boldsymbol{w}$, 且该项等于 $E(\boldsymbol{w})/\sigma^2$. 因 $\sigma^2 > 0$, 最小化 $-\ln p$ 等价于最小化 $E(\boldsymbol{w})$, 二者有相同的极小点集.

□

注

(重构新增) 该等价性依赖于 σ^2 固定这一前提:σ^2 只以正因子的形式出现在 $E(\boldsymbol{w})$ 之前, 因此不改变极小点的位置. 若把 σ^2 也视为待估参数并与之联合优化,$\boldsymbol{w}$ 的最优解仍然不变, 但目标函数的最优值会不同——这正是下一段要分别处理 $\boldsymbol{w}^*$ 与 σ^{2*} 的原因.

这里舍弃了加性的和乘性的常数. 我们把通过最小化 $E(\boldsymbol{w})$ 找到的 $\boldsymbol{w}$ 的值记为 $\boldsymbol{w}^*$. 值得注意的是, 它通常并不对应于似然函数的全局最大值, 因为网络函数 $y(\boldsymbol{x}_n, \boldsymbol{w})$ 的非线性导致误差 $E(\boldsymbol{w})$ 是非凸的, 通常很难找到全局最优解. 此外, 由于可以向误差函数添加正则化项 (参见节 9.1) 或者对其训练过程做出其他修改, 因此得到的网络参数的解可能与最大似然解有很大差异.

找到 $\boldsymbol{w}^*$ 后, 可以通过最小化误差函数(6.47)来找到 σ^2 的值 (见练习 6.8), 记为 σ^{2*}.

$$\sigma^{2*} = \frac{1}{N} \sum_{n=1}^{N} \{y(\boldsymbol{x}_n, \boldsymbol{w}^*) - t_n\}^2. \tag{6.52}$$

命题 6.13 (噪声方差的最大似然解)

固定 $\boldsymbol{w}=\boldsymbol{w}^*$, 则使(6.47)关于 σ^2 最小的解由(6.52)给出, 即噪声方差的最优估计是残差的均方值.

证明

(重构新增) 记 $S=\sum_n\{y(\boldsymbol{x}_n,\boldsymbol{w}^*)-t_n\}^2$, 把(6.47)视为 σ^2 的函数:

$$f(\sigma^2)=\frac{S}{2\sigma^2}+\frac{N}{2}\ln\sigma^2+\frac{N}{2}\ln(2\pi). \tag{6.53}$$

对 σ^2 求导并令其为零:

$$\frac{\mathrm{d}f}{\mathrm{d}\sigma^2}=-\frac{S}{2\sigma^4}+\frac{N}{2\sigma^2}=0 \quad\Longrightarrow\quad \sigma^2=\frac{S}{N}, \tag{6.54}$$

即(6.52). 又因二阶导数在驻点处为 $N/(2\sigma^4)>0$, 该驻点是极小点.

□

注

(重构新增) 值得注意的是,σ^{2*} 依赖 $\boldsymbol{w}^*$, 而 $\boldsymbol{w}^*$ 又是通过迭代优化得到的, 因此 σ^{2*} 只能在训练收敛之后计算. 若把 σ^2 与 $\boldsymbol{w}$ 联合优化, 通常采用交替或重参数化 (例如令 $\sigma=\exp(a)$) 的方式, 以避免 σ^2 趋零导致的数值不稳定.

值得注意的是, 一旦完成找到 $\boldsymbol{w}^*$ 所需的迭代优化, 就可以计算该值了.

如果有多个目标变量, 并假设它们在给定 $\boldsymbol{x}$ 和 $\boldsymbol{w}$ 的条件下是独立的, 且共享噪声方差 σ^2, 那么目标值的条件分布可以表示为

$$p(\boldsymbol{t}\mid\boldsymbol{x},\boldsymbol{w})=\mathcal{N}\big(\boldsymbol{t}\,\big|\,\boldsymbol{y}(\boldsymbol{x},\boldsymbol{w}),\sigma^2\boldsymbol{I}\big). \tag{6.55}$$

遵照与单目标变量相同的论证, 则关于权重最大化似然函数等价于最小化平方和误差函数 (见练习 6.9):

$$E(\boldsymbol{w})=\frac{1}{2}\sum_{n=1}^{N}\big\|\boldsymbol{y}(\boldsymbol{x}_n,\boldsymbol{w})-\boldsymbol{t}_n\big\|^2. \tag{6.56}$$

噪声方差则由下式给出:

$$\sigma^{2*}=\frac{1}{NK}\sum_{n=1}^{N}\big\|\boldsymbol{y}(\boldsymbol{x}_n,\boldsymbol{w}^*)-\boldsymbol{t}_n\big\|^2, \tag{6.57}$$

其中 K 是目标变量的维度. 目标变量的条件独立性假设, 在所承受的优化问题变得稍微复杂一些的代价下, 也是可以舍弃的 (见练习 6.10).

命题 6.14 (多输出回归的误差函数与噪声方差)

设 $\boldsymbol{t}\in\mathbb{R}^K$ 的条件分布由(6.55)给出, 且各观测独立. 则负对数似然 (忽略常数) 为

$$\frac{1}{2\sigma^2}\sum_{n=1}^{N}\big\|\boldsymbol{y}(\boldsymbol{x}_n,\boldsymbol{w})-\boldsymbol{t}_n\big\|^2+\frac{NK}{2}\ln\sigma^2, \tag{6.58}$$

故关于 $\boldsymbol{w}$ 的最大似然等价于最小化(6.56), 而关于 σ^2 的最优解为(6.57).

证明

(重构新增) 由 $\mathcal{N}(\boldsymbol{t}\mid\boldsymbol{y},\sigma^2\boldsymbol{I})=(2\pi\sigma^2)^{-K/2}\exp\{-\|\boldsymbol{t}-\boldsymbol{y}\|^2/(2\sigma^2)\}$, 对 n 求和后取负对数得(6.58). 关于 $\boldsymbol{w}$ 的依赖只在前一项, 故与最小化(6.56)等价. 关于 σ^2 求导:$-\frac{S}{2\sigma^4}+\frac{NK}{2\sigma^2}=0$, 其中 $S=\sum_n\|\boldsymbol{y}_n-\boldsymbol{t}_n\|^2$, 解得 $\sigma^2=S/(NK)$, 即(6.57).

□

注

(重构新增)(6.58)中的 $\ln\sigma^2$ 系数是 $NK/2$ 而不是 $N/2$, 这正对应“每个输出分量各贡献一个噪声维度”. 若各输出分量的噪声方差不同 (协方差为一般正定矩阵 $\boldsymbol{\Sigma}$), 则误差函数变为带权重的马氏距离形式, 且 $\boldsymbol{w}$ 与 $\boldsymbol{\Sigma}$ 的优化相互耦合——这正是练习 6.10要讨论的内容.

回想一下, 误差函数 (由负对数似然给出) 和输出单元激活函数之间存在自然的配对关系 (参见小节 5.4.5). 在回归问题中, 我们可以将网络视为具有恒等的输出单元激活函数, 即 $y_k = a_k$. 相应的平方和误差函数因此具有以下性质:

$$\frac{\partial E}{\partial a_k} = y_k - t_k. \tag{6.59}$$

命题 6.15 (恒等输出单元的误差导数)

设输出单元激活函数为恒等函数 $y_k = a_k$, 误差函数取(6.56)(单输出时取(6.50)). 则对第 n 个样本,

$$\frac{\partial E_n}{\partial a_k} = y_k - t_k, \tag{6.60}$$

即误差对预激活的导数恰为“预测减目标”的残差.

证明

(重构新增) 由 $E_n = \frac{1}{2}\sum_k (y_k - t_k)^2$ 与 $y_k = a_k$, 链式法则给出

$$\frac{\partial E_n}{\partial a_k} = \frac{\partial E_n}{\partial y_k}\cdot\frac{\partial y_k}{\partial a_k} = (y_k - t_k)\cdot 1 = y_k - t_k. \tag{6.61}$$

□

注

(重构新增)(6.59)之所以被称为“自然配对”, 是因为它把误差函数与激活函数的选择绑定在一起: 只有当输出激活与噪声模型匹配时, 导数才会简化成残差. 这一简化不仅是计算上的便利, 更是数值稳定性的保证——若用平方和误差配 sigmoid 输出, 导数中会多出一个 $\sigma'(a) = y(1-y)$ 因子, 在饱和区趋于零, 导致学习停滞.

6.4.2 二分类

接下来考虑二分类问题, 其中我们有一个单目标变量 t,$t = 1$ 表示类别 C_1,$t = 0$ 表示类别 C_2. 让我们回到对规范连接函数的讨论 (参见小节 5.4.5), 考虑具有单个输出的网络, 其激活函数是 sigmoid 函数 [(6.21)], 因此有 $0 \leqslant y(\boldsymbol{x}, \boldsymbol{w}) \leqslant 1$. 我们可以将 $y(\boldsymbol{x}, \boldsymbol{w})$ 解释为条件概率 $p(C_1 \mid \boldsymbol{x})$, 而 $p(C_2 \mid \boldsymbol{x})$ 由 $1 - y(\boldsymbol{x}, \boldsymbol{w})$ 给出. 给定输入下, 目标的条件分布是 Bernoulli 分布, 形式为

$$p(t \mid \boldsymbol{x}, \boldsymbol{w}) = y(\boldsymbol{x}, \boldsymbol{w})^t \left\{1 - y(\boldsymbol{x}, \boldsymbol{w})\right\}^{1-t}. \tag{6.62}$$

定义 6.32 (二分类的 Bernoulli 模型)

若二分类网络以 sigmoid 为输出激活、且目标的条件分布取(6.62), 则称该模型为**二分类的 Bernoulli 模型**, 其中 $y(\boldsymbol{x}, \boldsymbol{w}) = p(C_1 \mid \boldsymbol{x})$.

注

(重构新增)(6.62)的两点分布形式自动满足归一化:$t=1$ 时取 y,$t=0$ 时取 $1-y$. 这也解释了为什么二分类只需要一个输出单元而不是两个:$p(C_2 \mid \boldsymbol{x})$ 由 $1-y$ 给出, 若用两个 sigmoid 输出则二者之和不必为 1, 会造成表示上的冗余与不一致.

如果考虑一个观测值独立的训练集, 则由负对数似然给出的误差函数为以下形式的交叉熵误差函数:

$$E(\boldsymbol{w}) = -\sum_{n=1}^{N}\{t_n \ln y_n + (1-t_n)\ln(1-y_n)\}, \tag{6.63}$$

其中 y_n 表示 $y(\boldsymbol{x}_n, \boldsymbol{w})$.Simard、Steinkraus and Platt(2003) 对于分类问题, 使用交叉熵误差函数而不是平方和误差函数能加速训练并提高泛化能力.

命题 6.16 (二分类的交叉熵误差)

在定义 6.32与样本独立假设下, 负对数似然为

$$-\ln p(\boldsymbol{t} \mid \boldsymbol{X}, \boldsymbol{w}) = -\sum_{n=1}^{N}\{t_n \ln y_n + (1-t_n)\ln(1-y_n)\}, \tag{6.64}$$

即(6.63).

证明

(重构新增) 由(6.62)与独立性,

$$p(\boldsymbol{t} \mid \boldsymbol{X}, \boldsymbol{w}) = \prod_{n=1}^{N} y_n^{t_n}(1-y_n)^{1-t_n}, \tag{6.65}$$

取负对数并用 $\ln\prod = \sum \ln$ 即得(6.64).

□

注

(重构新增) 交叉熵只惩罚“把正确类预测得很不自信”的情形: 若 $t_n=1$ 而 $y_n \to 1$, 则 $-\ln y_n \to 0$; 若 $y_n \to 0$, 则损失发散. 这与平方和误差不同——后者在 y_n 饱和时梯度反而消失, 因而在分类任务上收敛更慢. 这一差别正是 Simard 等人观察到交叉熵训练更快的原因.

注意在(6.62)中, 因为假设目标值被正确打了标签, 所以不存在类似于噪声方差 σ^2 的概念. 可通过引入一个概率 ε 来表示目标值 t 被错误翻转的可能性, 该模型可以很容易地扩展为允许标签错误 (Opper and Winther,2000)(见练习 6.11). 这里的 ε 可以预先设定, 或者视为一个超参数, 其值可以从数据中推断得出.

如果我们有 K 个单独的二分类任务要执行, 则可以使用具有 K 个输出的网络, 每个输出都有一个 sigmoid 激活函数. 与每个输出相关联的是一个二元类标签 $t_k \in \{0,1\}$, 其中 $k=1,\cdots,K$. 如果我们假设在给定输入向量的情况下类标签是独立的, 则目标的条件分布为

$$p(\boldsymbol{t} \mid \boldsymbol{x}, \boldsymbol{w}) = \prod_{k=1}^{K} y_k(\boldsymbol{x}, \boldsymbol{w})^{t_k}\left[1-y_k(\boldsymbol{x}, \boldsymbol{w})\right]^{1-t_k}. \tag{6.66}$$

取相应似然函数的负对数, 可以得出以下误差函数 (见练习 6.13):

$$E(\boldsymbol{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K}\{t_{nk}\ln y_{nk} + (1-t_{nk})\ln(1-y_{nk})\}, \tag{6.67}$$

其中 y_{nk} 表示 $y_k(\boldsymbol{x}_n, \boldsymbol{w})$.

命题 6.17 (多标签交叉熵)

在(6.66)与样本独立假设下, 负对数似然为(6.67). 它等于 K 个独立二分类交叉熵之和.

证明

(重构新增) 对(6.66)取负对数并对 n 求和:

$$-\ln p(\boldsymbol{t} \mid \boldsymbol{X}, \boldsymbol{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K}\Big\{t_{nk}\ln y_{nk} + (1-t_{nk})\ln(1-y_{nk})\Big\}, \tag{6.68}$$

逐项整理即得(6.67).

□

注

(重构新增) 多标签情形与多分类情形的区别在于标签是否互斥: 这里 K 个标签可以同时为 1(例如一幅图像同时含“人”和“狗”), 因此用 K 个独立 sigmoid 而不是一个 softmax. 若错误地使用 softmax, 就人为加入了“只有一个标签为真”的约束, 属于模型设定错误.

同样, 误差函数相对于特定输出单元的预激活的导数采取了与回归情况相同的形式 [(6.59)] (见练习 6.14).

命题 6.18 (sigmoid 输出单元的误差导数)

设 $y_k = \sigma(a_k)$, 误差函数取(6.67). 则

$$\frac{\partial E_n}{\partial a_k} = y_k - t_k. \tag{6.69}$$

证明

(重构新增) 单个输出分量的损失为 $E_n^{(k)} = -\{t_k \ln y_k + (1-t_k)\ln(1-y_k)\}$. 利用 $\sigma'(a) = \sigma(a)(1-\sigma(a)) = y_k(1-y_k)$,

$$\begin{aligned}\frac{\partial E_n^{(k)}}{\partial a_k} &= -\left\{\frac{t_k}{y_k} - \frac{1-t_k}{1-y_k}\right\}\cdot y_k(1-y_k)\\ &= -\Big\{t_k(1-y_k) - (1-t_k)y_k\Big\} = y_k - t_k.\end{aligned} \tag{6.70}$$

各分量相加即得(6.69).

□

注

(重构新增)(6.69)与命题 6.15的形式完全一致, 这不是巧合:sigmoid 是 Bernoulli 分布的规范连接函数, 交叉熵是其负对数似然, 二者配对后导数中的 σ' 因子被精确消去. 这一“误差 = 残差”的结构使得输出层的梯度不会因激活函数饱和而消失, 是深度网络能够训练的重要前提 (参见节 8.1).

6.4.3 多分类

最后考虑多分类问题, 其中每个输入被分配到 K 个互斥的类别中. 二元目标变量 $t_k \in \{0,1\}$ 采用 1-of-K 编码方案来表示类别 (参见小节 5.1.3), 而网络输出则解释为 $y_k(\boldsymbol{x},\boldsymbol{w}) = p(t_k = 1 \mid \boldsymbol{x})$, 从而引出误差函数:

$$E(\boldsymbol{w}) = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk}\ln y_k(\boldsymbol{x}_n,\boldsymbol{w}). \tag{6.71}$$

对应于规范连接的输出单元激活函数由 softmax 函数给出 (参见小节 5.4.4):

$$y_k(\boldsymbol{x},\boldsymbol{w}) = \frac{\exp\big(a_k(\boldsymbol{x},\boldsymbol{w})\big)}{\sum_{j=1}^{K}\exp\big(a_j(\boldsymbol{x},\boldsymbol{w})\big)}. \tag{6.72}$$

定义 6.33 (softmax 输出模型)

称(6.72)定义的映射为 **softmax** 函数, 称以 softmax 为输出激活、以(6.71)为误差函数的网络为**多分类模型** (multiclass model). 该模型给出互斥类别的后验概率 $y_k = p(C_k \mid \boldsymbol{x})$.

注

(重构新增)softmax 满足 $0 \leqslant y_k \leqslant 1$ 且 $\sum_k y_k = 1$, 因此它是一个合法的概率向量; 但它对预激活的平移不变:$a_k \mapsto a_k + c$ 不改变输出. 这既是优点 (数值上可以减去 $\max_k a_k$ 以防止溢出), 也是缺点 (权重空间中存在一条不改变模型的方向, 需要正则化来固定尺度). 这一退化性与小节 6.2.1 中增广输入的冗余不是同一回事, 后者不会影响输出.

(6.72)满足 $0 \leqslant y_k \leqslant 1$ 且 $\sum_k y_k = 1$. 注意, 如果将常数项添加到所有的 $a_k(\boldsymbol{x}, \boldsymbol{w})$ 上, 导致误差函数在某些权重空间方向上保持不变, 则 $y_k(\boldsymbol{x}, \boldsymbol{w})$ 也不会改变. 如果在误差函数中添加适当的正则化项, 这种退化性将被消除 (参见节 9.1). 再次强调, 对于特定输出单元的预激活, 误差函数的导数采用了我们熟悉的形式 [(6.59)] (见练习 6.15).

命题 6.19 (多分类交叉熵是负对数似然)

设 t_n 为 1-of-K 编码, 且 $p(\boldsymbol{t}_n \mid \boldsymbol{x}_n, \boldsymbol{w}) = \prod_{k=1}^{K} y_{nk}^{t_{nk}}$, 则样本独立时的负对数似然为(6.71).

♠

证明

(重构新增) 若样本 n 属于类别 c, 则 $t_{nk} = \delta_{kc}$, 于是 $\prod_k y_{nk}^{t_{nk}} = y_{nc}$, 即模型对正确类给出的概率. 对 n 取负对数并求和:

$$-\ln \prod_{n=1}^{N} y_{n,c(n)} = -\sum_{n=1}^{N} \ln y_{n,c(n)} = -\sum_{n=1}^{N}\sum_{k=1}^{K} t_{nk} \ln y_{nk}, \tag{6.73}$$

即(6.71).

□

注

(重构新增)(6.71)只对正确类求和, 等价于“最大化正确类的对数概率”. 这与二分类交叉熵(6.63)是一致的: 当 $K = 2$ 且 $y_2 = 1 - y_1$ 时, 两式完全相同. 因此在实现中, 二分类既可以写成单个 sigmoid, 也可以写成两个输出的 softmax, 二者在数学上等价, 数值上则略有差异.

命题 6.20 (softmax 输出单元的误差导数)

设 y_k 由(6.72)给出, 误差函数取(6.71). 则

$$\frac{\partial E_n}{\partial a_k} = y_k - t_k. \tag{6.74}$$

♠

证明

(重构新增) 先求 softmax 的 Jacobi 矩阵. 由 $y_k = e^{a_k}/S, S = \sum_j e^{a_j}$,

$$\frac{\partial y_k}{\partial a_j} = \frac{e^{a_k}}{S}\delta_{kj} - \frac{e^{a_k} e^{a_j}}{S^2} = y_k(\delta_{kj} - y_j). \tag{6.75}$$

于是

$$\begin{aligned} \frac{\partial E_n}{\partial a_j} &= -\sum_k \frac{t_k}{y_k}\frac{\partial y_k}{\partial a_j} = -\sum_k \frac{t_k}{y_k} y_k(\delta_{kj} - y_j) = -\sum_k t_k(\delta_{kj} - y_j) \\ &= -t_j + \Big(\sum_k t_k\Big) y_j = y_j - t_j, \end{aligned} \tag{6.76}$$

其中用到 $\sum_k t_k = 1$(1-of-K 编码).

□

注

(重构新增) 注意证明中关键的一步是 $\sum_k t_k = 1$: 若标签不是 one-hot(例如标签平滑或软标签), 则(6.74)变为 $y_j \sum_k t_k - t_j$, 不再是简单的残差. 这也从另一侧面说明“输出激活 + 误差函数”必须成对选择:softmax 配平方和误差时, 导数中会残留一个非平凡的 Jacobi 因子, 训练效率显著下降.

总之, 根据所要解决问题的类型, 输出单元激活函数和匹配的误差函数都有自然的选择. 对于回归问题, 我们使用线性输出和平方和误差函数; 对于多个独立的二分类问题, 我们使用 sigmoid 输出和交叉熵误差函数; 对于多分类问题, 我们使用 softmax 输出和相应的多元交叉熵误差函数. 对于涉及两个类别的分类问题 (即二分类问题), 我们可以使用单个 sigmoid 输出, 也可以使用由 softmax 函数激活的有两个输出的网络.

注

(重构新增) 把本节的配对关系汇总起来, 可以看成“分布 → 负对数似然 → 输出激活”的一条链:Gauss 分布给出平方和误差与恒等输出;Bernoulli 分布给出二分类交叉熵与 sigmoid 输出; 类别分布给出多分类交叉熵与 softmax 输出. 任何新的输出分布 (如 Poisson、负二项、混合 Gauss) 都可以按同样的方式导出误差函数与激活函数, 这也是节 6.5的出发点.

这个过程相当通用, 通过考虑其他形式的条件分布, 我们可以推导出相应的误差函数作为对应的负对数似然函数.

6.5 混合密度网络

到目前为止, 我们已经讨论了输出表示简单概率分布的神经网络, 包括连续变量的 Gauss 分布以及离散变量的二元分布. 在本章的最后一节, 我们将展示神经网络如何通过将网络的输出视为更复杂分布 (如 Gauss 混合分布) 的参数来表示更一般的条件概率, 我们称此类网络为**混合密度网络** (mixture density network), 你将看到如何定义关联的误差函数和相应的输出单元激活函数.

6.5.1 机器人运动学示例

监督学习的目的是建模条件分布 $p(\boldsymbol{t} \mid \boldsymbol{x})$. 对于许多简单的回归问题, 选择 Gauss 分布较为常见. 然而, 实际的机器学习问题往往具有明显的非 Gauss 分布特征. 例如, 在分布可能是多模态的**逆问题** (inverse problem) 中,Gauss 假设可能导致非常糟糕的预测.

定义 6.34 (正问题与逆问题) 设物理过程由映射 $\boldsymbol{g}: \boldsymbol{\theta} \mapsto \boldsymbol{x}$ 描述. 由参数 $\boldsymbol{\theta}$(如关节角) 计算输出 $\boldsymbol{x}$(如末端执行器位置) 的问题称为**正问题** (forward problem); 反过来, 由观测 $\boldsymbol{x}$ 推断 $\boldsymbol{\theta}$ 的问题称为**逆问题** (inverse problem). 当 $\boldsymbol{g}$ 不是单射时, 逆问题的解不唯一. ♣

注

(重构新增)定义 6.34中点明的关系是本节全部动机的来源: 正问题通常由因果律给出、且往往一对一; 逆问题则把多个原因映射到同一观测, 因此其条件分布天然是多峰的. 若强行用单峰的 Gauss 模型拟合逆问题, 模型只能给出各峰之间的“平均值”, 而这个值往往不对应任何一个真实解. 注意“多对一”是逆问题多峰性的充分条件而非必要条件: 即使 $\boldsymbol{g}$ 是单射, 观测噪声也会使后验出现多个模式.

作为逆问题的简单说明, 如图 6.16所示, 考虑机械臂的运动学. 正问题涉及在给定关节角度的情况下找到末端执行器的位置, 并具有唯一解. 然而在实践中, 我们希望将末端执行器移动到特定位置. 为此, 我们必须设置适当的关节角度. 我们需要求解一个逆问题, 如图 6.16所示, 该问题有两个解 (见练习 6.16).

正问题通常对应于物理系统中的因果关系, 并且通常具有唯一解. 例如, 人体的特定症状可能是由于存在特定疾病引起的. 然而, 在机器学习中, 我们通常需要求解一个逆问题, 例如尝试在给定一组症状的情况下预测疾病的存在. 如果正问题涉及多对一映射, 则逆问题将具有许多解. 例如, 多种不同疾病可能导致相同的症状.

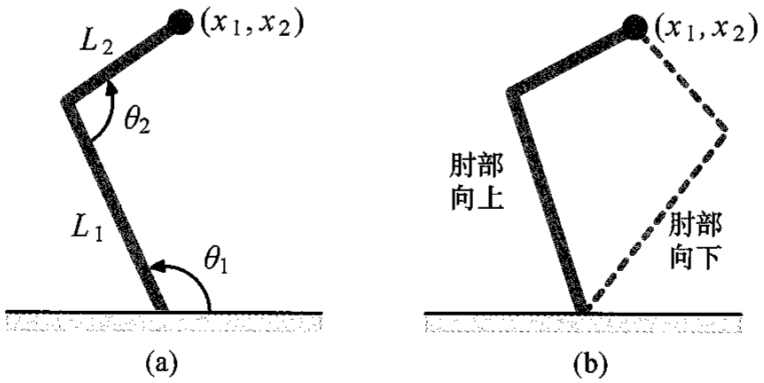


图 6.16: (a) 一种双连杆机械臂, 其中末端执行器的笛卡儿坐标 (x_1, x_2) 由两个关节角 θ_1 和 θ_2 以及臂的 (固定) 长度 L_1 和 L_2 唯一确定. 这称为机械臂的正运动学.(b) 在实践中, 我们必须找到能够产生所需末端执行器位置的关节角度. 这种逆运动学 (inverse kinematics) 问题有两个对应于“肘部向上”和“肘部向下”的解决方案

在机器人运动学示例中, 运动学由几何方程定义, 其多模态性显而易见. 然而, 在许多机器学习问题中, 多模态的存在可能不太明显, 特别是涉及高维空间的问题. 然而, 出于教学的目的, 下面考虑一个简单的小问题, 其可以很容易实现多模态可视化. 该问题的数据是通过在区间 $(0, 1)$ 上均匀采样变量 x 而产生的, 我们得到一组值 $\{x_n\}$. 通过计算函数 $x_n + 0.3\sin(2\pi x_n)$, 然后添加在区间 $(-0.1, 0.1)$ 上均匀采样时产生的噪声, 得到相应的目标值 t_n.图 6.17显示了正问题和逆问题的数据集, 以及通过最小化平方和误差函数得到的两层神经网络的结果, 该网络具有 6 个隐藏单元和一个线性输出单元. 最小二乘法对应于 Gauss 假设下的最大似然. 可以看到, 它对于正问题是一个很好的模型, 但它对于高度非 Gauss 性的逆问题却是一个很差的模型.

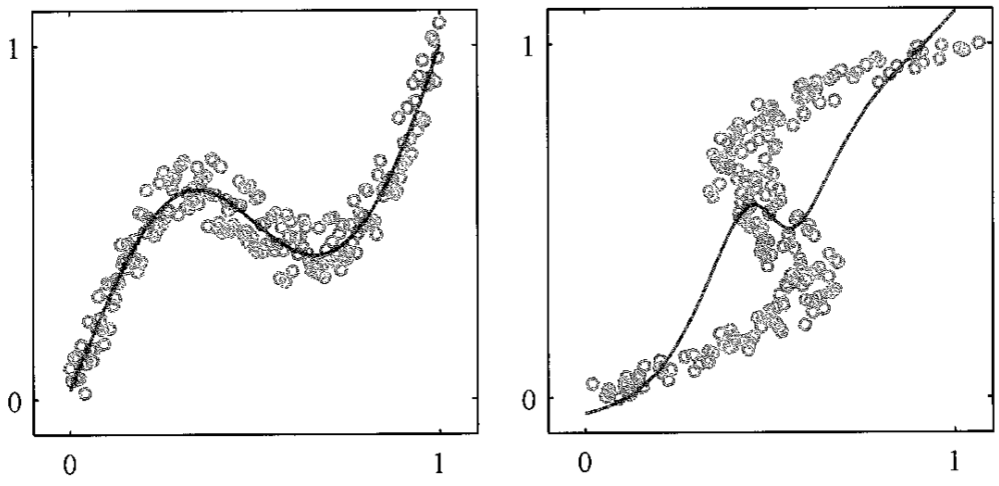


图 6.17: 左图展示了一个简单的正问题的数据集, 其中红色曲线显示了通过最小化平方和误差函数来拟合两层神经网络的结果. 如右图所示, 通过交换 x 和 t 的角色得到相应的逆问题. 在这里, 再次通过最小化平方和误差函数训练同样的网络, 由于数据集的多模态性, 网络对数据的拟合效果较差

例题 6.4 多模态的条件分布 设正问题的数据由 $t = x + 0.3\sin(2\pi x) + \varepsilon$ 生成, 其中 $x \sim \mathcal{U}(0, 1)$、$\varepsilon \sim \mathcal{U}(-0.1, 0.1)$. 逆问题把 t 视为输入、x 视为目标, 此时对同一个 t 往往存在多个 x 与之对应, 条件分布 $p(x \mid t)$ 因而具有多个峰; 最小二乘拟合只能给出这些峰之间的平滑曲线.

6.5.2 条件混合分布

我们想要找到建模条件概率分布的一般框架. 这可以通过使用专为建模 $p(\boldsymbol{t} \mid \boldsymbol{x})$ 设计的混合模型来实现, 其中混合系数和分量密度都是输入向量 $\boldsymbol{x}$ 的灵活函数, 从而得到**混合密度网络** (mixture density network). 对于任何

给定的 $\boldsymbol{x}$ 值, 混合模型为建模任意条件密度函数 $p(\boldsymbol{t} \mid \boldsymbol{x})$ 提供了一般形式. 如果有一个足够灵活的网络, 我们就有了一个可以近似任意条件概率分布的框架.

在这里, 我们显式地为 Gauss 分量设计模型如下:

$$p(\boldsymbol{t} \mid \boldsymbol{x}) = \sum_{k=1}^{K} \pi_k(\boldsymbol{x}) \mathcal{N}\left(\boldsymbol{t} \,\middle|\, \boldsymbol{\mu}_k(\boldsymbol{x}), \sigma_k^2(\boldsymbol{x})\right). \tag{6.77}$$

定义 6.35 (混合密度网络)

若条件密度由(6.77)给出, 其中混合系数 $\pi_k(\boldsymbol{x})$、分量均值 $\boldsymbol{\mu}_k(\boldsymbol{x})$ 与分量方差 $\sigma_k^2(\boldsymbol{x})$ 全部由以 $\boldsymbol{x}$ 为输入、以 $\boldsymbol{w}$ 为参数的同一个神经网络输出, 则称该模型为**混合密度网络** (mixture density network, MDN).

♣

注

(重构新增)定义 6.35与普通回归网络的区别不在于网络结构, 而在于输出的解释: 普通网络输出 L 个数并解释为条件均值,MDN 输出 $L + 2K$ 个数并解释为 K 个 Gauss 分量的参数. 因此 MDN 的误差函数不是人为选定的平方和, 而是(6.77)的负对数似然; 这也意味着训练 MDN 会自动地同时学习“有几个模式”“每个模式在哪里”“每个模式有多宽”.

这是一个**异方差** (heteroscedastic) 模型的示例, 数据上的噪声方差是输入向量 $\boldsymbol{x}$ 的函数. 除了 Gauss 分布, 我们还可以为分量使用其他分布. 例如, 如果目标变量是二元变量而不是连续变量, 则可以使用 Bernoulli 分布. 虽然这里使用了各向同性协方差, 但也可以通过使用 Cholesky 因式分解表示协方差来使混合密度网络支持一般的协方差矩阵 (Williams,1996). 即使使用各向同性的分量, 由于混合分布的影响, 条件分布 $p(\boldsymbol{t} \mid \boldsymbol{x})$ 也不具备对于 $\boldsymbol{t}$ 各分量的可分解性 (与标准平方和回归模型不同).

注

(重构新增) 异方差性在这里是“免费”得到的:$\sigma_k(\boldsymbol{x})$ 是网络的输出, 因此模型可以自动学到“在某些 $\boldsymbol{x}$ 处数据很嘈杂、在另一些处很干净”. 这与标准最小二乘回归形成鲜明对比——后者隐含假设噪声方差处处相同 (同方差), 若数据实际异方差, 最小二乘会给所有样本相同的权重, 从而在高噪声区域产生系统性偏差.

令混合模型的各个参数, 即混合系数 $\pi_k(\boldsymbol{x})$、均值 $\boldsymbol{\mu}_k(\boldsymbol{x})$ 和方差 $\sigma_k^2(\boldsymbol{x})$, 由以 $\boldsymbol{x}$ 为输入的神经网络的输出决定. 混合密度网络的结构如图 6.18所示. 混合密度网络与混合专家模型密切相关 (Jacobs et al.,1991). 它们的主要区别在于, 混合专家模型对自身的每个分量模型都有独立的参数; 而在混合密度网络中, 我们使用相同的函数来预测所有分量密度的参数和混合系数, 于是, 非线性隐藏单元在依赖于输入的函数之间共享.

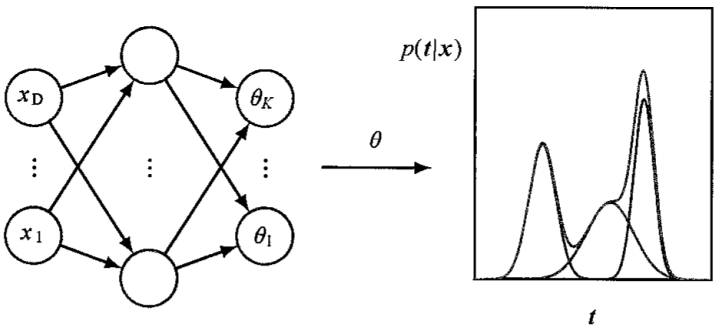


图 6.18: 混合密度网络可以通过考虑 t 分布的参数化混合模型来表示一般条件概率密度 $p(t \mid \boldsymbol{x})$, 其参数由以 $\boldsymbol{x}$ 为输入向量的神经网络的输出确定

例如,图 6.18中的神经网络可以是具有 S 形 tanh 隐藏单元的两层网络. 如果混合模型(6.77)中有 K 个分量, 并且 $\boldsymbol{t}$ 有 L 个分量, 则网络将具有 K 个输出单元预激活, 这些输出单元用 a_k^{π} 表示, 用于确定混合系数 $\pi_k(\boldsymbol{x})$, 用 a_k^{σ} 表示的 K 个输出用于确定 Gauss 标准差 $\sigma_k(\boldsymbol{x})$,$K \times L$ 个用 a_{kj}^{μ} 表示的输出则用于确定 Gauss 均值 $\boldsymbol{\mu}_k(\boldsymbol{x})$ 的分量 $\mu_{kj}(\boldsymbol{x})$. 网络输出的总数由 $L + 2K$ 给出, 这与具有 L 个输出的用来预测目标变量条件均值的网络不同.

混合系数必须满足下列约束条件:

$$\sum_{k=1}^{K}\pi_k(\boldsymbol{x})=1,\qquad 0\leqslant\pi_k(\boldsymbol{x})\leqslant 1, \tag{6.78}$$

这可以使用一组 softmax 输出来实现:

$$\pi_k(\boldsymbol{x})=\frac{\exp\left(a_k^{\pi}\right)}{\sum_{j=1}^{K}\exp\left(a_j^{\pi}\right)}. \tag{6.79}$$

同样, 方差必须满足 $\sigma_k^2(\boldsymbol{x})\geqslant 0$, 因而 $\sigma_k(\boldsymbol{x})$ 可以使用下列相应网络预激活的指数来表示:

$$\sigma_k(\boldsymbol{x})=\exp\left(a_k^{\sigma}\right). \tag{6.80}$$

最后, 由于均值 $\boldsymbol{\mu}_k(\boldsymbol{x})$ 具有实分量, 因此可以用网络输出直接表示:

$$\mu_{kj}(\boldsymbol{x})=a_{kj}^{\mu}, \tag{6.81}$$

其中输出单元激活函数由恒等式 $f(a)=a$ 给出.

定义 6.36 (MDN 的输出参数化)

称(6.79)、(6.80)与(6.81)为混合密度网络的**输出参数化**: 混合系数经 softmax 保证(6.78), 标准差经指数变换保证非负, 均值经恒等变换保持无约束. ♣

注

(重构新增) 三种变换的选择不是任意的, 而是与各参数的取值域一一对应:π_k 位于概率单纯形上, 故用 softmax;$\sigma_k>0$, 故用指数 (也可以用以 softplus 代替指数, 数值上更稳定);$\mu_{kj}\in\mathbb{R}$, 故用恒等. 需要注意的是, 指数变换虽然保证了非负, 却在 a_k^{σ} 很大时使 σ_k 增长极快, 容易造成数值溢出, 因此实现中通常对 a_k^{σ} 做截断或改用 softplus.

混合密度网络的可学习参数包括神经网络中的权重和偏置向量 $\boldsymbol{w}$, 它可以通过最大似然来确定, 也可以等价地通过最小化定义为似然负对数的误差函数来确定. 对于彼此独立的数据, 误差函数采用以下形式:

$$E(\boldsymbol{w})=-\sum_{n=1}^{N}\ln\left\{\sum_{k=1}^{K}\pi_k(\boldsymbol{x}_n,\boldsymbol{w})\,\mathcal{N}\left(\boldsymbol{t}_n\,\middle|\,\boldsymbol{\mu}_k(\boldsymbol{x}_n,\boldsymbol{w}),\sigma_k^2(\boldsymbol{x}_n,\boldsymbol{w})\right)\right\}, \tag{6.82}$$

其中, 我们显式地给出了有关 $\boldsymbol{w}$ 的依赖关系.

命题 6.21 (MDN 的误差函数)

在定义 6.35与样本独立假设下, 负对数似然为(6.82). 该误差函数是 $\boldsymbol{w}$ 的非凸函数, 且其梯度可以逐样本分解为 $\nabla E(\boldsymbol{w})=\sum_{n=1}^{N}\nabla E_n(\boldsymbol{w})$, 其中 $E_n=-\ln p(\boldsymbol{t}_n\mid\boldsymbol{x}_n,\boldsymbol{w})$. ♠

证明

(重构新增) 由独立性,$p(\{\boldsymbol{t}_n\}\mid\{\boldsymbol{x}_n\},\boldsymbol{w})=\prod_n p(\boldsymbol{t}_n\mid\boldsymbol{x}_n,\boldsymbol{w})$, 取负对数即得(6.82). 非凸性来自 $\boldsymbol{\mu}_k$ 与 σ_k 对 $\boldsymbol{w}$ 的非线性依赖以及 $\ln\sum_k$ 的结构. 梯度分解由求导的线性性直接得到.

□

注

(重构新增)(6.82)中的对数是“对求和取对数”, 这与命题 6.16中对乘积取对数不同: 它不能拆成每个分量的损失之和, 因此 MDN 无法像普通分类那样逐分量独立优化. 这一结构还带来一个著名的病态: 当某个分量的方差趋于零且恰好覆盖一个样本时, 似然可以趋于无穷、损失趋于 $-\infty$. 实践中通过给 σ_k 设下界或加入正则项来避免这种退化.

6.5.3 梯度优化

为了最小化误差函数, 需要计算误差 $E(\boldsymbol{w})$ 关于分量 $\boldsymbol{w}$ 的导数. 稍后我们将看到如何自动计算这些导数 (参见节 8.1). 然而, 显式地为输出单元预激活的误差导数导出合适的表达式是很有帮助的, 因为这强调了这些量的概率解释. 由于误差函数(6.82)由一些项的和组成, 其中每一项对应一个训练数据点, 因此我们可以考虑特定输入向量 $\boldsymbol{x}_n$ 与相关目标向量 $\boldsymbol{t}_n$ 的导数. 总误差 E 的导数是通过对所有数据点求和而获得的, 每个数据点自身的梯度也可以直接用于基于梯度的优化算法 (参见节 7.1).

引入以下变量将有助于推导:

$$\gamma_{nk} = \gamma_k(\boldsymbol{t}_n \mid \boldsymbol{x}_n) = \frac{\pi_k \mathcal{N}_{nk}}{\sum_{l=1}^{K} \pi_l \mathcal{N}_{nl}}, \tag{6.83}$$

其中 $\mathcal{N}_{nk}$ 表示 $\mathcal{N}\left(\boldsymbol{t}_n \,\middle|\, \boldsymbol{\mu}_k(\boldsymbol{x}_n), \sigma_k^2(\boldsymbol{x}_n)\right)$. 这些量可被自然地解释为混合分布分量的后验概率, 其中混合系数 $\pi_k(\boldsymbol{x})$ 可以视为与 $\boldsymbol{x}$ 相关的先验概率 (见练习 6.17).

定义 6.37 (分量后验概率)

称(6.83)定义的 γ_{nk} 为第 k 个混合分量对第 n 个样本的**后验概率** (responsibility), 它满足 $\gamma_{nk} \geqslant 0$ 且 $\sum_{k=1}^{K} \gamma_{nk} = 1$.

注

(重构新增)定义 6.37是 EM 算法中“责任度”的同一定义:γ_{nk} 度量“第 n 个观测由第 k 个分量生成”的后验概率. 它与先验 π_k 的关系正是 Bayes 公式: 后验正比于“先验与似然”的乘积. 注意 γ_{nk} 依赖于 $\boldsymbol{x}_n$(通过 $\pi_k(\boldsymbol{x}_n)$ 与 $\mu_k(\boldsymbol{x}_n)$) 而不依赖于 $\boldsymbol{t}$ 之外的标签, 这正是 MDN 把“软分配”内嵌进网络的方式.

关于控制混合系数的网络输出预激活的误差函数的导数由下式给出 (参见练习 6.18):

$$\frac{\partial E_n}{\partial a_k^{\pi}} = \pi_k - \gamma_{nk}. \tag{6.84}$$

命题 6.22 (混合系数预激活的导数)

在(6.79)的参数化下,(6.84)成立.

证明

(重构新增) 记 $S_n = \sum_j \pi_j \mathcal{N}_{nj}$, 则 $E_n = -\ln S_n$. 由 softmax 的 Jacobi $\partial \pi_j / \partial a_k^{\pi} = \pi_j(\delta_{jk} - \pi_k)$,

$$\begin{aligned} \frac{\partial E_n}{\partial a_k^{\pi}} &= -\frac{1}{S_n} \sum_j \mathcal{N}_{nj} \frac{\partial \pi_j}{\partial a_k^{\pi}} = -\frac{1}{S_n} \sum_j \mathcal{N}_{nj} \pi_j (\delta_{jk} - \pi_k) \\ &= -\frac{\pi_k \mathcal{N}_{nk}}{S_n} + \pi_k \frac{\sum_j \pi_j \mathcal{N}_{nj}}{S_n} = -\gamma_{nk} + \pi_k. \end{aligned} \tag{6.85}$$

□

注

(重构新增)(6.84)有非常直观的解释:π_k 是模型先验地给第 k 个分量的权重,γ_{nk} 是看到数据后后验地分配给它的权重,二者的差就是“应当增大还是减小该分量的混合系数”. 当某分量对某样本解释得比先验预期更好时 $\gamma_{nk} > \pi_k$,梯度为负,梯度下降会增大 π_k.

类似地,关于控制分量均值的网络输出预激活的误差函数的导数由下式给出 (见练习 6.19):

$$\frac{\partial E_n}{\partial a_{kj}^{\mu}} = \gamma_{nk} \frac{\mu_{kj}(\boldsymbol{x}_n) - t_{nj}}{\sigma_k^2(\boldsymbol{x}_n)}. \tag{6.86}$$

命题 6.23 (分量均值预激活的导数)

在(6.81)的参数化下,(6.86)成立,其中 $j = 1, \cdots, L$ 是目标向量的分量指标.

证明

(重构新增) 由 $E_n = -\ln S_n$ 与 $\mu_{kj} = a_{kj}^{\mu}$,

$$\frac{\partial E_n}{\partial a_{kj}^{\mu}} = -\frac{\pi_k}{S_n} \frac{\partial \mathcal{N}_{nk}}{\partial \mu_{kj}}. \tag{6.87}$$

对 Gauss 分量取对数再求导:$\ln \mathcal{N}_{nk} = -\frac{L}{2}\ln(2\pi) - \sum_j \ln \sigma_k - \frac{1}{2\sigma_k^2}\sum_j (t_{nj} - \mu_{kj})^2$,故 $\partial \ln \mathcal{N}_{nk}/\partial \mu_{kj} = (t_{nj} - \mu_{kj})/\sigma_k^2$,即 $\partial \mathcal{N}_{nk}/\partial \mu_{kj} = \mathcal{N}_{nk}(t_{nj} - \mu_{kj})/\sigma_k^2$. 代入并利用 $\gamma_{nk} = \pi_k \mathcal{N}_{nk}/S_n$ 得

$$\frac{\partial E_n}{\partial a_{kj}^{\mu}} = -\gamma_{nk}\frac{t_{nj} - \mu_{kj}}{\sigma_k^2} = \gamma_{nk}\frac{\mu_{kj} - t_{nj}}{\sigma_k^2}. \tag{6.88}$$

□

注

(重构新增)(6.86)与普通回归的残差形式 $y - t$ 完全平行,只是每个样本的残差被后验概率 γ_{nk} 加权、并被方差 σ_k^2 归一化:方差越小的分量对残差越敏感. 需要说明的是,印刷本此处下标写作 $\mu_{kl} - t_{nl}$;由于该式是对分量指标 j 求导、且 t_{nj} 与 μ_{kj} 必须逐分量对应,本重构统一记为 $\mu_{kj} - t_{nj}$.

最后,关于控制分量方差的网络输出预激活的误差函数的导数由下式给出 (见练习 6.20):

$$\frac{\partial E_n}{\partial a_k^{\sigma}} = \gamma_{nk}\left\{L - \frac{\|\boldsymbol{t}_n - \boldsymbol{\mu}_k\|^2}{\sigma_k^2}\right\}. \tag{6.89}$$

命题 6.24 (分量方差预激活的导数)

在(6.80)的参数化下,(6.89)成立,其中 L 是目标向量的维数.

证明

(重构新增) 由 $\sigma_k = \exp(a_k^{\sigma})$ 得 $\partial \sigma_k/\partial a_k^{\sigma} = \sigma_k$. 又

$$\ln \mathcal{N}_{nk} = -\frac{L}{2}\ln(2\pi) - L\ln\sigma_k - \frac{\|\boldsymbol{t}_n - \boldsymbol{\mu}_k\|^2}{2\sigma_k^2}, \tag{6.90}$$

对 σ_k 求导得 $\partial \ln \mathcal{N}_{nk}/\partial \sigma_k = -L/\sigma_k + \|\boldsymbol{t}_n - \boldsymbol{\mu}_k\|^2/\sigma_k^3$. 于是

$$\frac{\partial E_n}{\partial a_k^{\sigma}} = -\gamma_{nk}\frac{\partial \ln \mathcal{N}_{nk}}{\partial \sigma_k}\frac{\partial \sigma_k}{\partial a_k^{\sigma}} = -\gamma_{nk}\left(-\frac{L}{\sigma_k} + \frac{\|\boldsymbol{t}_n - \boldsymbol{\mu}_k\|^2}{\sigma_k^3}\right)\sigma_k = \gamma_{nk}\left\{L - \frac{\|\boldsymbol{t}_n - \boldsymbol{\mu}_k\|^2}{\sigma_k^2}\right\}. \tag{6.91}$$

□

注

(重构新增)(6.89)中括号内的两项相互竞争:L 使方差增大 (熵项,防止分量过窄),$\|\boldsymbol{t}_n - \boldsymbol{\mu}_k\|^2/\sigma_k^2$ 使方差减小 (拟合项,使分量覆盖样本). 平衡点是 $\sigma_k^2 = \|\boldsymbol{t}_n - \boldsymbol{\mu}_k\|^2/L$,即该分量的残差均方. 这一形式与命题 6.13完全一致,说明 MDN 的方差学习是普通噪声方差估计的“逐分量、逐样本”版本.

6.5.4 预测分布

让我们回到图 6.17所示的逆问题示例来说明混合密度网络的使用方法.图 6.19显示了混合系数 $\pi_k(\boldsymbol{x})$、均值 $\boldsymbol{\mu}_k(\boldsymbol{x})$ 和对应于 $p(\boldsymbol{t} \mid \boldsymbol{x})$ 的条件密度等值线. 神经网络的输出, 即混合模型中的参数, 必然是输入变量的连续单值函数. 然而, 我们从图 6.19(c) 中可以看到, 该模型能够通过调制混合分量 $\pi_k(\boldsymbol{x})$ 的振幅来产生条件密度, 该密度对 x 的某些值是单峰的, 而对其他值是三峰的.

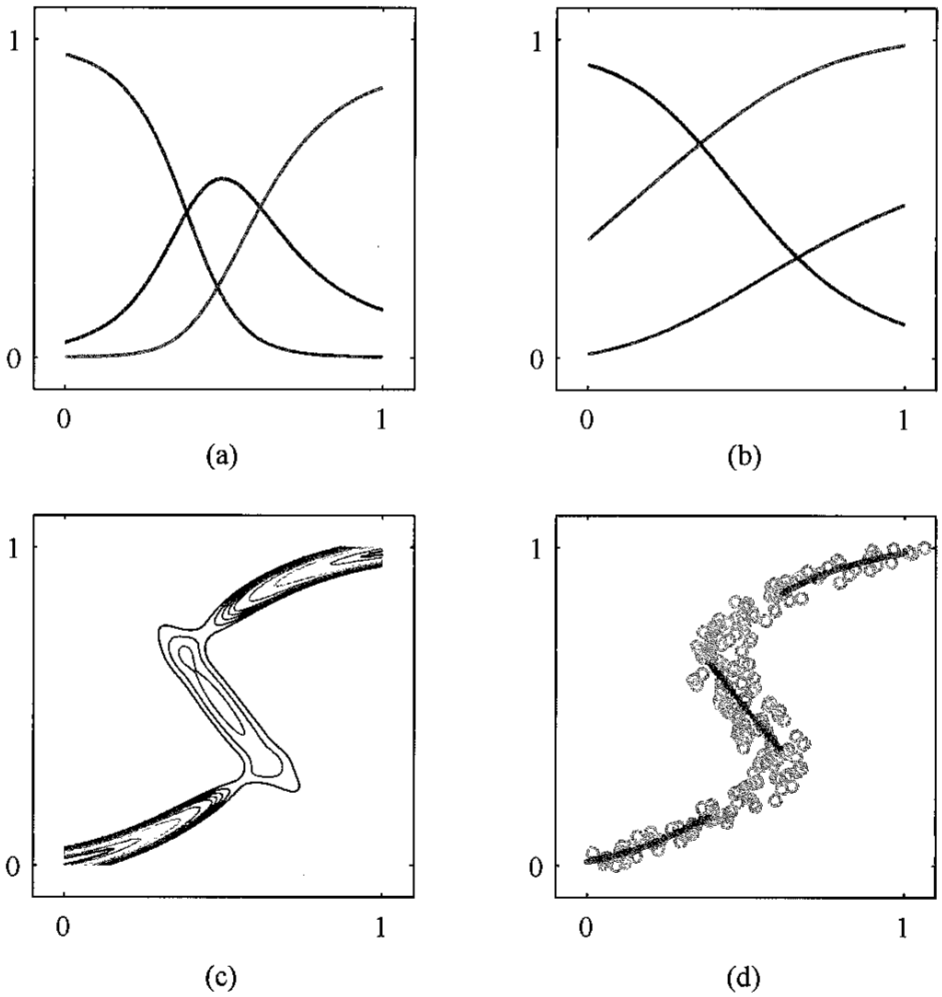


图 6.19: (a) 在根据同一逆问题数据训练的混合密度网络中, 将 3 种混合分量的混合系数 $\pi_k(\boldsymbol{x})$ 作为 x 的函数. 该模型有 3 个 Gauss 分量, 并使用一个两层神经网络, 该网络有 5 个 S 形 tanh 隐藏单元和 9 个输出 (分别对应于 Gauss 分量的 3 个均值、3 个方差以及 3 个混合系数). 在 x 取小值或大值的情况下, 目标数据的条件概率密度是单峰的, 只有一个 Gauss 分量的先验概率很高; 而在 x 取中间值时, 条件概率密度是三峰的,3 个混合系数具有差不多的值.(b) 为混合系数使用相同颜色方案绘制的均值 $\mu_k(\boldsymbol{x})$.(c) 为相同的混合密度网络绘制目标数据的条件概率密度等值线.(d) 用红点表示的条件概率密度的近似条件众数图

混合密度网络训练好后, 给定任意输入向量值, 就可以预测目标数据的条件密度函数. 就预测输出向量值的问题而言, 该条件密度表示对数据生成器的完整描述. 从这个密度函数中, 我们可以计算出不同应用可能关注的更具体的量. 其中最简单的一个量是平均值, 它对应于目标数据的条件平均值, 由下式给出:

$$\mathbb{E}[\boldsymbol{t} \mid \boldsymbol{x}] = \int \boldsymbol{t}\, p(\boldsymbol{t} \mid \boldsymbol{x})\, \mathrm{d}\boldsymbol{t} = \sum_{k=1}^{K} \pi_k(\boldsymbol{x})\, \boldsymbol{\mu}_k(\boldsymbol{x}). \tag{6.92}$$

命题 6.25 (条件均值)

设 $p(\boldsymbol{t}\mid\boldsymbol{x})$ 由(6.77)给出, 则条件均值为(6.92).

证明

(重构新增) 由积分的线性性与 Gauss 分布的归一化,

$$\int \boldsymbol{t}\sum_k \pi_k \mathcal{N}\big(\boldsymbol{t}\mid\boldsymbol{\mu}_k,\sigma_k^2\big)\mathrm{d}\boldsymbol{t} = \sum_k \pi_k \int \boldsymbol{t}\,\mathcal{N}\big(\boldsymbol{t}\mid\boldsymbol{\mu}_k,\sigma_k^2\big)\mathrm{d}\boldsymbol{t} = \sum_k \pi_k\boldsymbol{\mu}_k, \tag{6.93}$$

其中用到 $\int \boldsymbol{t}\,\mathcal{N}(\boldsymbol{t}\mid\boldsymbol{\mu}_k,\sigma_k^2)\mathrm{d}\boldsymbol{t}=\boldsymbol{\mu}_k$. 注意 π_k 与 $\boldsymbol{\mu}_k$ 都依赖于 $\boldsymbol{x}$, 但它们对 $\boldsymbol{t}$ 的积分而言是常数.

□

由于使用最小二乘法训练的标准网络会近似条件均值, 因此我们看到混合密度网络可以作为一个特例再现传统的最小二乘结果. 当然, 正如我们已经指出的, 对于多峰分布, 条件均值的值是有限的.

注

(重构新增)(6.92)说明 MDN 并不排斥最小二乘, 而是把它包含为 $K=1$、σ 为常数时的特例. 这也提供了检验 MDN 实现是否正确的一个简单方法: 把 K 设为 1 并固定 σ, 训练结果应当与普通回归网络一致. 对多峰分布而言, 条件均值落在两个峰之间、可能对应概率极低的位置, 这正是图 6.16(b) 中“两种解的平均值不是解”的情形.

我们可以类似地评估关于条件均值密度函数的方差 (见练习 6.21):

$$s^2(\boldsymbol{x}) = \mathbb{E}\Big[\big\|\boldsymbol{t}-\mathbb{E}[\boldsymbol{t}\mid\boldsymbol{x}]\big\|^2\,\Big|\,\boldsymbol{x}\Big], \tag{6.94}$$

展开后为

$$s^2(\boldsymbol{x}) = \sum_{k=1}^{K}\pi_k(\boldsymbol{x})\left\{\sigma_k^2(\boldsymbol{x}) + \Big\|\boldsymbol{\mu}_k(\boldsymbol{x}) - \sum_{i=1}^{K}\pi_i(\boldsymbol{x})\boldsymbol{\mu}_i(\boldsymbol{x})\Big\|^2\right\}. \tag{6.95}$$

命题 6.26 (条件方差分解)

设 $p(\boldsymbol{t}\mid\boldsymbol{x})$ 由(6.77)给出, 则(6.94)的方差可分解为(6.95), 即“分量内方差”与“分量间方差”之和.

证明

(重构新增) 记 $\boldsymbol{m}=\mathbb{E}[\boldsymbol{t}\mid\boldsymbol{x}]=\sum_i \pi_i\boldsymbol{\mu}_i$. 由全期望公式与各分量的协方差 $\sigma_k^2\boldsymbol{I}$,

$$\begin{aligned}\mathbb{E}\big[\|\boldsymbol{t}-\boldsymbol{m}\|^2\mid\boldsymbol{x}\big] &= \sum_k \pi_k\,\mathbb{E}\big[\|\boldsymbol{t}-\boldsymbol{m}\|^2\mid\boldsymbol{x},k\big]\\ &= \sum_k \pi_k\Big(\mathbb{E}\big[\|\boldsymbol{t}-\boldsymbol{\mu}_k\|^2\mid\boldsymbol{x},k\big] + \|\boldsymbol{\mu}_k-\boldsymbol{m}\|^2\Big)\\ &= \sum_k \pi_k\Big(\sigma_k^2 + \|\boldsymbol{\mu}_k-\boldsymbol{m}\|^2\Big),\end{aligned} \tag{6.96}$$

其中第二行用到恒等式 $\mathbb{E}\|\boldsymbol{t}-\boldsymbol{m}\|^2=\mathbb{E}\|\boldsymbol{t}-\boldsymbol{\mu}_k\|^2+\|\boldsymbol{\mu}_k-\boldsymbol{m}\|^2$(交叉项期望为零), 第三行用到各向同性 Gauss 的协方差为 $\sigma_k^2\boldsymbol{I}$. 把 $\boldsymbol{m}=\sum_i\pi_i\boldsymbol{\mu}_i$ 代入即得(6.95).

□

这里用到了(6.77)和(6.92). 该结果比相应的最小二乘结果更普遍, 因为方差是关于 $\boldsymbol{x}$ 的函数.

注

(重构新增)(6.95)是典型的“全方差公式”: 总方差 = 分量内方差的加权平均 + 分量均值之间的离散程度. 第二项在单峰时为零, 在多峰时可以非常大——这提醒我们, 多峰分布的“方差”往往由峰间距离主导, 而不能用单个 Gauss 的 σ^2 解释. 因此在逆问题中报告条件方差时, 应同时报告各分量的均值与混合系数, 否则会严重误导.

我们已经看到, 对于多峰分布, 条件均值给出了较差的数据表示. 例如, 在图 6.16所示的简单机械臂的控制中, 我们需要从两种可能的关节角度设置中选择一种来获得所需的末端执行器位置, 但两种解的平均值本身并不是

一个解. 在这种情况下, 条件众数可能更有价值. 由于混合密度网络的条件众数没有简单的解析解, 因此进行数值迭代是必要的. 一种简单的替代方法是在每个 $\boldsymbol{x}$ 值处取最可能的分量 (即混合系数最大的分量) 的平均值. 这一做法已在图 6.19(d) 所示的示例数据集上给出.

6.6 习题

✍ **练习 6.1** (★★★) 使用极坐标下的 Gauss 积分推导 D 维空间中单位半径超球体的表面积 S_D 和体积 V_D 的计算表达式. 为此, 请考虑以下结果, 该结果是通过从笛卡儿坐标转换为极坐标获得的:

$$\prod_{i=1}^{D} \int_{-\infty}^{\infty} e^{-x_i^2} \, \mathrm{d}x_i = S_D \int_0^{\infty} e^{-r^2} r^{D-1} \, \mathrm{d}r. \tag{6.97}$$

使用 Gamma 函数, 定义如下:

$$\Gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} \, \mathrm{d}t, \tag{6.98}$$

以及标准 Gauss 积分 $\int_{-\infty}^{\infty} e^{-x^2} \mathrm{d}x = \sqrt{\pi}$, 计算(6.97)的两边, 从而证明

$$S_D = \frac{2\pi^{D/2}}{\Gamma(D/2)}. \tag{6.99}$$

接下来, 通过对从 0 到 1 的半径进行积分, 证明 D 维空间中单位超球体的体积由下式给出:

$$V_D = \frac{S_D}{D}. \tag{6.100}$$

最后, 使用结果 $\Gamma(1) = 1$ 和 $\Gamma(3/2) = \sqrt{\pi}/2$ 证明(6.99)和(6.100)可以简化为 $D = 2$ 和 $D = 3$ 的常用表达式.

✍ **练习 6.2** (★★★) 考虑 D 维空间中半径为 a 的超球体与边长为 $2a$ 的同心超立方体, 使超球体在其每条边的中心与超立方体相交. 通过使用练习 6.1的结果, 证明超球体的体积与超立方体的体积之比由下式给出:

$$\frac{\text{超球体的体积}}{\text{超立方体的体积}} = \frac{\pi^{D/2}}{D\, 2^{D-1}\, \Gamma(D/2)}. \tag{6.101}$$

以如下形式使用 Stirling 公式:

$$\Gamma(x+1) \approx (2\pi)^{1/2} e^{-x} x^{x+1/2}, \tag{6.102}$$

它对 $x \gg 1$ 有效, 证明当 $D \to \infty$ 时,(6.101)等于零. 同时证明从超立方体的中心到其中一个角的距离, 与从超立方体的中心到其中一条边的垂直距离之比为 $\sqrt{D}$, 且在 $D \to \infty$ 时趋向于 ∞. 从这些结果中可以看到, 在高维空间中, 立方体的大部分体积都集中在大量的角上, 这些角本身变成了很长的 “尖刺” !

✍ **练习 6.3** (★★★) 探讨 Gauss 分布在高维空间中的行为. 考虑 D 维空间中的 Gauss 分布, 它由下式给出:

$$p(\boldsymbol{x}) = \frac{1}{(2\pi\sigma^2)^{D/2}} \exp\left(-\frac{\|\boldsymbol{x}\|^2}{2\sigma^2}\right). \tag{6.103}$$

我们希望在极坐标中找到以半径为参数的密度函数, 其中方向变量已被积分消去. 为此, 证明半径为 r 且厚度为 $\varepsilon(\varepsilon \ll 1)$ 的薄壳上的概率质量由 $p(r)\varepsilon$ 给出, 其中

$$p(r) = \frac{S_D r^{D-1}}{(2\pi\sigma^2)^{D/2}} \exp\left(-\frac{r^2}{2\sigma^2}\right), \tag{6.104}$$

S_D 是 D 维空间中单位超球体的表面积. 对于较大的 D 值, 证明函数 $p(r)$ 有单个位于 $\hat{r} \approx \sqrt{D}\sigma$ 的驻点. 通过考虑 $p(\hat{r}+\varepsilon)$, 其中 $\varepsilon \ll \hat{r}$, 证明对于较大的 D 值, 有

$$p(\hat{r}+\varepsilon) \approx p(\hat{r}) \exp\left(-\frac{\varepsilon^2}{\sigma^2}\right). \tag{6.105}$$

这表明 $\hat{r}$ 是径向概率密度的最大值, 且 $p(r)$ 在长度尺度为 σ 的最大值 $\hat{r}$ 处开始指数衰减. 对于较大的 D 值来说,$\sigma \ll \hat{r}$, 因此我们看到大部分概率质量集中在半径较大的薄壳内. 最后, 证明原点处的概率密度 $p(\boldsymbol{x})$ 大于半径 $\hat{r}$ 处的概率密度, 它们之间相差一个因子 $\exp(D/2)$. 于是, 我们可以看到在高维空间中,Gauss 分布中的大多数概率质量位于与高概率密度区域不同的半径处.

✍ **练习 6.4** (★★) 考虑(6.17)给出的网络函数, 其中隐藏单元非线性激活函数 $h(\cdot)$ 由 sigmoid 函数给出:

$$\sigma(a) = \left\{1 + \exp(-a)\right\}^{-1}. \tag{6.106}$$

证明存在一个等价网络, 它计算的是完全相同的网络函数, 但具有由 $\tanh(a)$ 给出的隐藏单元激活函数, 其中 tanh 函数由(6.22)定义. 提示: 首先找到 $\sigma(a)$ 和 $\tanh(a)$ 之间的关系, 然后证明这两个网络的参数可以通过线性变换相互转换.

✍ **练习 6.5** (★★)swish 激活函数 (Ramachandran,Zoph,and Le,2017) 定义如下:

$$h(x) = x\,\sigma(\beta x), \tag{6.107}$$

其中 $\sigma(x)$ 是(6.21)定义的 sigmoid 激活函数. 当在神经网络中使用时,β 可以视为可学习参数. 手绘或用软件画出 swish 激活函数及其在 $\beta = 0.1$、$\beta = 1.0$ 和 $\beta = 10$ 时的一阶导数. 证明当 $\beta \to \infty$ 时,swish 函数变为 ReLU 函数.

✍ **练习 6.6** (★) 我们在 sigmoid 激活函数的讨论中看到, 其导数可以用函数值本身来表示. 推导(6.22)定义的 tanh 激活函数的相应结果.

✍ **练习 6.7** (★★) 证明(6.26)给出的 softplus 激活函数 $\zeta(a)$ 满足以下性质:

$$\zeta(a) - \zeta(-a) = a, \tag{6.108}$$

$$\ln \sigma(a) = -\zeta(-a), \tag{6.109}$$

$$\frac{\mathrm{d}\zeta(a)}{\mathrm{d}a} = \sigma(a), \tag{6.110}$$

$$\zeta^{-1}(a) = \ln\big(\exp(a) - 1\big), \tag{6.111}$$

其中 $\sigma(a)$ 是(6.21)给出的 sigmoid 激活函数.

✍ **练习 6.8** (★) 证明对方差 σ^2 最小化误差函数 [(6.47)] 可以得到(6.52).

✍ **练习 6.9** (★) 证明在多输出神经网络的条件分布[(6.55)]下最大化似然函数等价于最小化平方和误差函数[(6.56)]. 此外, 证明最小化平方和误差函数的噪声方差由(6.57)给出.

✍ **练习 6.10** (★★) 考虑一个涉及多个目标变量的回归问题, 其中假设以输入向量 $\boldsymbol{x}$ 为条件的目标分布是具有以下形式的 Gauss 分布:

$$p(\boldsymbol{t} \mid \boldsymbol{x}, \boldsymbol{w}) = \mathcal{N}\big(\boldsymbol{t} \mid \boldsymbol{y}(\boldsymbol{x}, \boldsymbol{w}), \boldsymbol{\Sigma}\big), \tag{6.112}$$

其中 $\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{w})$ 是具有输入向量 $\boldsymbol{x}$ 和权重向量 $\boldsymbol{w}$ 的神经网络的输出,$\boldsymbol{\Sigma}$ 是假定的目标上 Gauss 噪声的协方差. 给定一组 $\boldsymbol{x}$ 和 $\boldsymbol{t}$ 的独立观测值, 假设 $\boldsymbol{\Sigma}$ 是固定的且已知, 写出为找到 $\boldsymbol{w}$ 的最大似然解而必须最小化的误差函数. 假设 $\boldsymbol{\Sigma}$ 也要从数据中确定, 写出 $\boldsymbol{\Sigma}$ 最大似然解的表达式. 注意, 现在 $\boldsymbol{w}$ 和 $\boldsymbol{\Sigma}$ 的优化是耦合的, 这与小节 6.4.1讨论的独立目标变量情况相反.

✍ **练习 6.11** (★★) 考虑一个二分类问题, 其中目标值为 $t \in \{0, 1\}$, 用网络输出 $y(\boldsymbol{x}, \boldsymbol{w})$ 表示 $p(t = 1 \mid \boldsymbol{x})$, 并假设训练数据点上的类标签设置不正确的概率为 ε. 假设数据独立同分布, 写出对应于负对数似然的误差函数. 验证当 $\varepsilon = 0$ 时将得到误差函数 [(6.63)] . 注意, 与交叉熵误差函数相比, 此误差函数使得模型对错误标记的数据具有鲁棒性.

✍ **练习 6.12** (★★) 二分类问题的误差函数 [(6.63)] 是针对具有 sigmoid 输出单元激活函数的网络推导出来的, 故有 $0 \leqslant y(\boldsymbol{x}, \boldsymbol{w}) \leqslant 1$, 且目标值为 $t \in \{0, 1\}$. 考虑一个具有输出 $-1 \leqslant y(\boldsymbol{x}, \boldsymbol{w}) \leqslant 1$ 的网络, 对类 C_1 目标值为 $t = 1$, 而对类 C_2 目标值为 $t = -1$, 推导出相应的误差函数. 进一步地, 输出单元激活函数应该如何选择?

✍ **练习 6.13** (★) 证明最大化多分类神经网络模型的似然, 其中网络输出可解释为 $y_k(\boldsymbol{x}, \boldsymbol{w}) = p(t_k = 1 \mid \boldsymbol{x})$, 等价于最小化交叉熵误差函数 [(6.71)] .

✍ **练习 6.14** (★) 证明对于具有 sigmoid 激活函数 $y_k = \sigma(a_k)$ 的输出单元, 误差函数 [(6.63)] 对其预激活 a_k 的导数满足(6.59), 其中 $\sigma(a)$ 由(6.21)给出.

✍ **练习 6.15** (★) 证明对于具有 softmax 激活函数 [(6.72)] 的输出单元, 误差函数 [(6.71)] 对其预激活 a_k 的导数满足(6.59).

✍ **练习 6.16** (★★) 用一对关于关节角 θ_1 和 θ_2 以及连接长度 L_1 和 L_2 的方程表示图 6.16所示机械臂的笛卡儿坐标 (x_1, x_2). 假设坐标系的原点由下臂的连接点给出. 这些方程定义了机械臂的正运动学.

✍ **练习 6.17** (★★) 证明(6.83)定义的变量 γ_{nk} 可以看作混合分布［(6.77)］分量的后验概率 $p(k \mid \boldsymbol{t})$, 其中混合系数 $\pi_k(\boldsymbol{x})$ 可以视为与 $\boldsymbol{x}$ 相关的先验概率 $p(k)$.

✍ **练习 6.18** (★★) 对于控制混合密度网络中混合系数的网络输出预激活, 推导误差函数对其的导数［(6.84)］.

✍ **练习 6.19** (★★) 对于控制混合密度网络中分量均值的网络输出预激活, 推导误差函数对其的导数［(6.86)］.

✍ **练习 6.20** (★★) 对于控制混合密度网络中分量方差的网络输出预激活, 推导误差函数对其的导数［(6.89)］.

✍ **练习 6.21** (★★★) 针对混合密度网络模型的条件均值和方差, 验证(6.92)和(6.95)给出的结果.

第 7 章 梯度下降

在上一章中, 我们看到神经网络是一类非常宽泛且灵活的函数类别, 原则上可以使用足够多的隐藏单元以高精度逼近所期望的函数. 此外, 我们发现深度神经网络能够编码对应于层次化表示的归纳偏置, 这在广泛的实际应用中很有价值. 在本章中, 我们的任务是基于一组训练数据, 为网络参数 (权重和偏置) 寻找合适的设置.

与前面讨论的回归模型和分类模型一样, 我们通过优化误差函数来选择模型参数.节 6.4介绍了如何基于最大似然法为具体的应用场景构建合适的误差函数. 尽管原则上可以通过一系列直接的误差函数数值计算来实现误差函数的最小化, 但实践表明这种方法的效率极其低下. 因此, 我们要靠深度学习中的另一项核心概念, 即利用梯度信息来更高效地优化误差函数. 具体而言, 就是计算误差函数关于网络模型中各个参数的偏导数. 这正是我们为什么需要精心设计神经网络模型以确保其所表征的函数是处处可微的. 相应地, 所采用的误差函数自身也要是可微的.

网络中的每个参数对误差函数的所需偏导数, 可以借助于反向传播 (backpropagation) 这一算法来高效地计算 (参见节 8.1). 该算法涉及一系列在网络中逆向传递的递推计算, 这与网络输出计算过程中的前向信息流动在形式上颇为相似.

尽管似然函数常被用于构建误差函数, 但在神经网络的训练过程中, 优化误差函数的终极目标是使模型在独立的测试数据上获得良好的泛化能力. 在经典统计学中, 最大似然估计被广泛应用于基于有限数据集进行参数模型的拟合, 在这种情况下, 数据点的数量通常远超模型中参数的数量. 最优解对应似然函数的最大值, 并且所得到的参数估计值本身就是关注的核心. 与之形成鲜明对比的是, 现代深度学习所处理的模型通常具有极其丰富的内部结构, 并包含海量的可学习参数 (参见小节 9.3.2), 其训练目标也从来都不局限于单纯地追求误差函数的极值. 相反, 学习算法自身的特性与行为, 以及各种正则化技术 (详见正则化一章) 的引入, 对模型在新数据上的泛化能力起着至关重要的作用.

7.1 错误平面

在模型训练过程中, 我们的目标是为神经网络中的权重和偏置参数确定合适的取值, 从而使其能对新的输入数据做出有效的预测. 为了便于后续的讨论, 我们首先将这些参数统一表示为一个向量 $\mathbf{w}$, 并基于预先选定的误差函数 $E(\mathbf{w})$ 来优化 $\mathbf{w}$. 为了直观地理解这一优化过程, 我们可以建立起误差函数的几何图像, 即将其视为一个位于“权重空间” 之上的曲面, 如图 7.1所示.

首先注意, 如果我们在权重空间中从 $\mathbf{w}$ 向 $\mathbf{w}+\delta\mathbf{w}$ 迈出一小步, 那么误差函数的变化由下式给出:

$$\delta E \approx \delta\mathbf{w}^{\mathsf{T}}\nabla E(\mathbf{w}), \tag{7.1}$$

其中 $\nabla E(\mathbf{w})$ 指向误差函数增长率最大的方向. 只要 $E(\mathbf{w})$ 是 $\mathbf{w}$ 的平滑连续函数, 它的最小值就会出现在权重空间中梯度为零的点处, 即

$$\nabla E(\mathbf{w}) = \mathbf{0}. \tag{7.2}$$

我们可以朝着 $-\nabla E(\mathbf{w})$ 的方向迈出一小步, 从而进一步减小误差. 梯度为零的点称为驻点 (stationary point, 参见小节 7.1.1), 驻点可以进一步分为极小点、极大点和鞍点.

定义 7.1 (驻点)

设 $E(\mathbf{w})$ 是权重向量 $\mathbf{w}\in\mathbb{R}^W$ 的平滑连续函数. 若 $\mathbf{w}$ 满足(7.2), 则称 $\mathbf{w}$ 为误差函数 E 的一个**驻点** (stationary point). 驻点可进一步分为极小点、极大点和鞍点.

♣

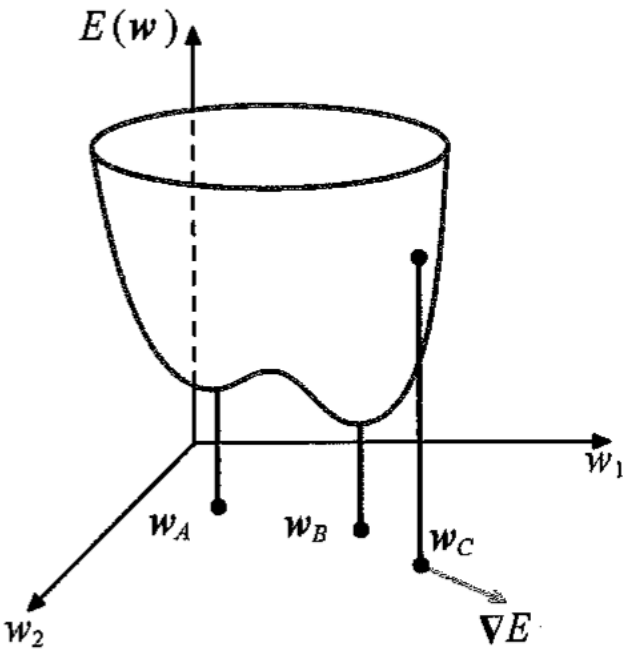


图 7.1: 误差函数 $E(\mathbf{w})$ 的几何视图, 它可以视为一个坐落在权重空间上方的表面. 点 $\mathbf{w}_A$ 是一个局部极小值点, 而点 $\mathbf{w}_B$ 是全局最小值点, 因此有 $E(\mathbf{w}_A) > E(\mathbf{w}_B)$. 在任意点 $\mathbf{w}_C$, 误差曲面的局部梯度由 ∇E 给出.

注

(重构新增) (7.2)之所以能刻画极小值点的候选位置, 是因为若 $\nabla E(\mathbf{w}) \neq \mathbf{0}$, 则沿 $-\nabla E(\mathbf{w})$ 方向作足够小的移动必使 E 严格减小,$\mathbf{w}$ 便不可能是极小值点; 反过来, 梯度为零只是一阶必要条件, 它把极小点、极大点和鞍点放进同一个候选集合, 必须借助二阶信息才能区分. 定义中“平滑连续”的要求不可省略: 若 E 只在有限个点处不可微 (例如由 ReLU 单元构成的分段线性网络), 最小值也可能出现在梯度不存在或梯度不连续的折点上, 此时(7.2)不再适用. 深度网络的误差函数通常只在局部意义下满足光滑性假设, 本章借助二次近似得到的结论都应理解为光滑区域内部的局部结论.

我们需要找到这样一个向量 $\mathbf{w}$, 使得 $E(\mathbf{w})$ 取最小值. 然而, 误差函数通常对权重和偏置参数有高度非线性的依赖关系, 因此权重空间中能使梯度为零 (或数值非常小) 的点有很多. 实际上, 对于任意局部极小值点, 权重空间中通常还会存在其他等效的局部极小值点. 例如, 在小节 6.2.4所讨论的、具有 M 个隐藏单元的两层网络中, 每个点都是 $M! \times 2^M$ 个等效点集合中的一员.

定义 7.2 (全局最小值点与局部极小值点)

在整个权重空间 $\mathbf{w}$ 中, 对应于误差函数最小取值的那个极小值点称为**全局最小值点** (global minimum), 而其他所有对应于相对较高误差函数值的极小值点则称为**局部极小值点** (local minima). ♣

注

(重构新增) 这个定义把“最小”分成两种强度: 全局最小值点要求 E 在整个权重空间上取到最小值, 局部极小值点只要求在一个邻域内不被超越. 定义隐含了一个存在性前提: 全局最小值点未必存在, 例如 $E(\mathbf{w}) = e^{w_1}$ 在 $\mathbb{R}^W$ 上下方有界却取不到最小值, 此时只能讨论下确界; 深度网络的误差函数一般下方有界, 但最小值是否取到并无先验保证. 另一要点是“等效”: 由于网络输出对隐藏单元的置换和符号翻转不变 (见小节 6.2.4), 一个局部极小值点会连带 $M! \times 2^M$ 个函数等价的点, 它们不是彼此独立的解, 因此比较不同极小值时应按网络所表示的函数而非按参数向量计数.

不仅如此, 权重空间中还可能存在多个本质上并不等效的驻点, 尤其是多个非等效的局部极小值点. 深度神经网络模型的误差曲面形态可能极为复杂, 人们曾认为基于梯度的优化方法可能会陷入性能欠佳的局部极小值区域. 然而, 在实践中, 大型神经网络模型在各种不同的初始条件下通常都能够收敛到性能相近的解 (见小节 9.3.2).

7.1.1 局部二次近似

我们可以通过考虑对误差函数进行局部二次近似来深入理解优化问题以及各种优化技术. 在权重空间中某点 $\hat{\mathbf{w}}$ 的周围对 $E(\mathbf{w})$ 进行 Taylor 展开, 得到

$$E(\mathbf{w}) \approx E(\hat{\mathbf{w}}) + (\mathbf{w}-\hat{\mathbf{w}})^{\mathsf{T}}\mathbf{b} + \frac{1}{2}(\mathbf{w}-\hat{\mathbf{w}})^{\mathsf{T}}\mathbf{H}(\mathbf{w}-\hat{\mathbf{w}}), \tag{7.3}$$

其中省略了三阶及更高阶的项. 这里的 $\mathbf{b}$ 定义为我们在 $\hat{\mathbf{w}}$ 处评估的 E 的梯度:

$$\mathbf{b} \triangleq \nabla E(\mathbf{w})|_{\mathbf{w}=\hat{\mathbf{w}}}. \tag{7.4}$$

定义 7.3 (黑塞矩阵)

设 $E(\mathbf{w})$ 二阶连续可微, 则称二阶导数矩阵

$$\mathbf{H}(\hat{\mathbf{w}}) = \nabla\nabla E(\mathbf{w})|_{\mathbf{w}=\hat{\mathbf{w}}} \tag{7.5}$$

为 E 在 $\hat{\mathbf{w}}$ 处的**黑塞矩阵** (Hessian).

♣

注

(重构新增) 一阶导数只给出斜率, 二阶导数则给出曲率; 黑塞矩阵把“沿各个方向弯曲得多快”编码成一个矩阵, 从而把(7.3)变成一个可以完全对角化的二次型. 定义要求 E 二阶连续可微, 此时 $\mathbf{H}$ 对称 (混合偏导数可交换), 独立的元素个数为 $W(W+1)/2$; 若只假设二阶可微而不要求连续性, 对称性可能失效. 矩阵规模由可学习参数总数 W 决定, 深度网络中 W 可达 10^6 以上, 显式构造和存储 $\mathbf{H}$ 都不可行, 因此实际算法只通过“黑塞矩阵—向量乘积”使用它的作用. 几何上,$\mathbf{H}$ 在极小值点附近正定、在极大值点附近负定、在鞍点处不定, 在平坦方向上奇异, 这几种情形正是本章后续收敛性分析的分类依据.

如果网络中总共有 W 个权重和偏置, 那么 $\mathbf{w}$ 和 $\mathbf{b}$ 的长度为 W,$\mathbf{H}$ 的维度为 $W \times W$. 根据(7.3), 对应的局部梯度近似由下式给出:

$$\nabla E(\mathbf{w}) = \mathbf{b} + \mathbf{H}(\mathbf{w}-\hat{\mathbf{w}}). \tag{7.6}$$

对于足够接近 $\hat{\mathbf{w}}$ 的点 $\mathbf{w}$, 这些表达式将给出误差及其梯度的合理近似.

考虑一种特定情况, 即围绕一个误差函数最小值点 $\mathbf{w}^*$ 进行局部二次近似. 在这种情况下, 因为在点 $\mathbf{w}^*$ 处 $\nabla E = \mathbf{0}$, 所以没有线性项,(7.3)变为

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2}(\mathbf{w}-\mathbf{w}^*)^{\mathsf{T}}\mathbf{H}(\mathbf{w}-\mathbf{w}^*), \tag{7.7}$$

其中黑塞矩阵 $\mathbf{H}$ 在点 $\mathbf{w}^*$ 处求得. 为了从几何角度解释这一点, 考虑黑塞矩阵的特征方程:

$$\mathbf{H}\mathbf{u}_i = \lambda_i \mathbf{u}_i, \tag{7.8}$$

其中特征向量 $\mathbf{u}_i$ 形成一组完整的正交规范集 (参见节 A.4), 所以有

$$\mathbf{u}_i^{\mathsf{T}}\mathbf{u}_j = \delta_{ij}. \tag{7.9}$$

将 $(\mathbf{w}-\mathbf{w}^*)$ 展开为特征向量线性组合的形式:

$$\mathbf{w}-\mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i. \tag{7.10}$$

这可以视为坐标系的变换, 原点被平移到点 $\mathbf{w}^*$, 轴与特征向量对齐 (见节 A.4). 这是通过一个正交矩阵进行的, 它的列由 $\mathbf{u}_1, \cdots, \mathbf{u}_W$ 组成. 将(7.10)代入(7.7)并使用(7.8)和(7.9), 误差函数可以写成如下形式 (见练习 7.1).

命题 7.1 (特征向量坐标下的误差函数)

设 $E(\mathbf{w})$ 在 $\mathbf{w}^*$ 处由二次函数(7.7)精确给出,$\mathbf{H}$ 的特征值为 λ_i、对应的正交规范特征向量为 $\mathbf{u}_i$, 并设 $\mathbf{w}-\mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i$. 则

$$E(\mathbf{w}) = E(\mathbf{w}^*) + \frac{1}{2}\sum_i \lambda_i \alpha_i^2. \tag{7.11}$$

♠

证明

(重构新增) 由(7.10)与(7.8),

$$\begin{aligned}(\mathbf{w}-\mathbf{w}^*)^{\mathsf{T}}\mathbf{H}(\mathbf{w}-\mathbf{w}^*) &= \Big(\sum_i \alpha_i \mathbf{u}_i\Big)^{\mathsf{T}}\mathbf{H}\Big(\sum_j \alpha_j \mathbf{u}_j\Big)\\ &= \sum_i\sum_j \alpha_i\alpha_j\, \mathbf{u}_i^{\mathsf{T}}\mathbf{H}\mathbf{u}_j\\ &= \sum_i\sum_j \alpha_i\alpha_j\lambda_j\, \mathbf{u}_i^{\mathsf{T}}\mathbf{u}_j.\end{aligned}$$

再由正交规范性(7.9),$\mathbf{u}_i^{\mathsf{T}}\mathbf{u}_j = \delta_{ij}$, 故上式等于 $\sum_i \lambda_i\alpha_i^2$. 代入(7.7)即得(7.11).

□

注

(重构新增) 这个命题把耦合的二次型化为 W 个独立的一元二次函数之和: 在特征向量坐标下, 沿 $\mathbf{u}_i$ 方向的运动只改变 α_i, 不同方向互不干扰, 这一点是后续收敛性分析能够逐个方向讨论的前提. 结论依赖三个条件. 其一,$\mathbf{H}$ 为实对称矩阵, 从而特征值全为实数且特征向量可选取为正交规范基 (节 A.4). 其二,$\mathbf{w}^*$ 是驻点, 因此展开式没有线性项; 若 $\mathbf{w}^*$ 不是驻点, 对角化后每个方向还会残留一次项, 极小点位置随之平移, 这正是 Newton 法要处理的形态. 其三,$\mathbf{u}_i$ 取为单位向量, 于是 $\sum_i \alpha_i^2 = \|\mathbf{w}-\mathbf{w}^*\|^2$,命题 7.2的证明要用到这个等式.

假设对于所有的 $i \neq j$, 设置 $\alpha_i = 0$, 然后变化 α_j, 这相当于把 $\mathbf{w}$ 沿着 $\mathbf{u}_j$ 方向移动, 远离 $\mathbf{w}^*$. 由(7.11)可以直接读出驻点的类型与特征值符号之间的关系.

命题 7.2 (驻点类型与特征值符号)

在命题 7.1的设定下, 设全部特征值非零, 则:

1. 若所有 $\lambda_i > 0$, 则 $\mathbf{w}^*$ 是 E 的严格局部极小值点;
2. 若所有 $\lambda_i < 0$, 则 $\mathbf{w}^*$ 是 E 的严格局部极大值点;
3. 若特征值有正有负, 则 $\mathbf{w}^*$ 是一个**鞍点** (saddle point).

♠

证明

记 $\lambda_{\min} = \min_i \lambda_i, \lambda_{\max} = \max_i \lambda_i$. 由(7.11)及正交规范性,

$$E(\mathbf{w}) - E(\mathbf{w}^*) = \frac{1}{2}\sum_i \lambda_i\alpha_i^2, \qquad \|\mathbf{w}-\mathbf{w}^*\|^2 = \sum_i \alpha_i^2.$$

(1) 若所有 $\lambda_i > 0$, 则对 $\mathbf{w} \neq \mathbf{w}^*$ 有 $\sum_i \lambda_i\alpha_i^2 \geqslant \lambda_{\min}\sum_i \alpha_i^2 = \lambda_{\min}\|\mathbf{w}-\mathbf{w}^*\|^2 > 0$, 故 $E(\mathbf{w}) > E(\mathbf{w}^*)$, 即 $\mathbf{w}^*$ 是严格局部极小值点.

(2) 若所有 $\lambda_i < 0$, 同理 $\sum_i \lambda_i\alpha_i^2 \leqslant \lambda_{\max}\|\mathbf{w}-\mathbf{w}^*\|^2 < 0$, 故 $E(\mathbf{w}) < E(\mathbf{w}^*)$, 即 $\mathbf{w}^*$ 是严格局部极大值点.

(3) 设 $\lambda_p > 0$、$\lambda_q < 0$. 取 $\mathbf{w} = \mathbf{w}^* + t\,\mathbf{u}_p$(即 $\alpha_p = t$, 其余 $\alpha_i = 0$), 则 $E(\mathbf{w}) - E(\mathbf{w}^*) = \frac{1}{2}\lambda_p t^2 > 0$ 对一切 $t \neq 0$ 成立; 再取 $\mathbf{w} = \mathbf{w}^* + t\,\mathbf{u}_q$, 则 $E(\mathbf{w}) - E(\mathbf{w}^*) = \frac{1}{2}\lambda_q t^2 < 0$ 对一切 $t \neq 0$ 成立. 因此 $\mathbf{w}^*$ 的任意邻域内既有函数值大于 $E(\mathbf{w}^*)$ 的点, 也有函数值小于 $E(\mathbf{w}^*)$ 的点,$\mathbf{w}^*$ 是鞍点.

□

注

(重构新增) 该结论的适用范围是二次模型(7.7)本身: 在二次模型中, 特征值的符号完全决定几何形态. 对一般光滑函数, 只要所有特征值非零, 同样的分类可由带余项的 Taylor 定理证明 (参见定理 7.1); 一旦出现零特征值, 二次项在该方向上消失, 形态由三阶及更高阶项决定, 此时不能只凭二次近似下结论. 两个典型例子说明了这一边界:$E(x) = x^3$ 在 0 处梯度与二阶导数同时为零, 却既非极小也非极大;$E(x) = x^4$ 在 0 处是严格局部极小, 相应的黑塞矩阵为零矩阵, 并不正定. 这也说明“黑塞矩阵正定”是局部极小的充分条件而非普遍的必要条件.

定义 7.4 (正定矩阵)

如果对于所有的非零向量 $\mathbf{v}$ 有

$$\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} > 0, \tag{7.12}$$

则称黑塞矩阵 $\mathbf{H}$ 为**正定矩阵** (positive definite).

♣

注

(重构新增) 正定性刻画的是“沿任意非零方向都向上弯曲”: 只有当二次型在所有方向上严格为正时,(7.7)中的二次项才能压制任意方向上的高阶余项. 定义中必须排除 $\mathbf{v}=\mathbf{0}$, 因为 $\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v}$ 在原点恒为零, 若不排除就无矩阵能满足严格不等式. 对一般方阵应把 $\mathbf{H}$ 换为其对称部分 $(\mathbf{H}+\mathbf{H}^{\mathsf{T}})/2$, 因为 $\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v}$ 只依赖对称部分; 本章的 $\mathbf{H}$ 由二阶偏导数构成, 本身对称, 故不必额外处理. 把“>0”放宽为“$\geqslant 0$”得到半正定矩阵, 它对应“某些方向上一阶不弯曲”的退化情形, 此时二次近似无法判定极小性 (参见定理 7.1的注记).

因为特征向量 $\mathbf{u}_i$ 形成一个完整的集合, 所以任意向量 $\mathbf{v}$ 可以写成

$$\mathbf{v} = \sum_i c_i \mathbf{u}_i, \tag{7.13}$$

且 $\mathbf{v}\neq\mathbf{0}$ 当且仅当某个 $c_i \neq 0$. 由(7.8)和(7.9), 有

$$\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} = \sum_i c_i^2 \lambda_i. \tag{7.14}$$

由此可以得到正定性的特征值判据.

命题 7.3 (正定性的特征值判据)

设 $\mathbf{H}$ 为实对称矩阵, 其特征值为 λ_i、对应的正交规范特征向量为 $\mathbf{u}_i$. 则 $\mathbf{H}$ 正定当且仅当所有特征值 λ_i 都是正的.

♠

证明

(重构新增) 由(7.13)与(7.14),$\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} = \sum_i c_i^2\lambda_i$. (必要性) 设 $\mathbf{H}$ 正定. 若某个 $\lambda_k \leqslant 0$, 取 $\mathbf{v}=\mathbf{u}_k\neq\mathbf{0}$, 则(7.14)给出 $\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v}=\lambda_k\leqslant 0$, 与(7.12)矛盾. 故所有 $\lambda_i>0$.

(充分性) 设所有 $\lambda_i>0$. 对任意 $\mathbf{v}\neq\mathbf{0}$, 由(7.14)及 $\sum_i c_i^2 = \|\mathbf{v}\|^2 > 0$,

$$\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} = \sum_i c_i^2\lambda_i \geqslant \lambda_{\min}\sum_i c_i^2 = \lambda_{\min}\|\mathbf{v}\|^2 > 0,$$

其中 $\lambda_{\min} = \min_i \lambda_i > 0$. 故 $\mathbf{H}$ 正定.

□

注

(重构新增) 这个判据把“对一切非零 $\mathbf{v}$ 成立的不等式”化归为“有限个数的符号检验”, 是本章后面反复使用的工具. 证明的两个方向各有一个关键点: 必要性方向只需沿单个特征向量 $\mathbf{u}_k$ 取 $\mathbf{v}$, 这说明正定性是逐方向的要求, 任何一个方向上的非正曲率都会破坏它; 充分性方向则用到正交规范基下 $\sum_i c_i^2 = \|\mathbf{v}\|^2$, 从而可以用最小特征值给出统一的下界. 条件“$\mathbf{H}$ 实对称”不可省: 非对称矩阵的特征值可能为复数, 判据失去意义; 此时应先对称化. 判据对半正定性有平行版本:$\mathbf{H}$ 半正定当且仅当所有 $\lambda_i\geqslant 0$.

由此可以得到局部极小值的二阶判别条件.

定理 7.1 (局部极小值的二阶条件)

设 $E:\mathbb{R}^W\to\mathbb{R}$ 在 $\mathbf{w}^*$ 的某邻域内二阶连续可微,$\nabla E(\mathbf{w}^*)=\mathbf{0}$, 并记 $\mathbf{H}=\nabla\nabla E(\mathbf{w}^*)$. 则:

1. 若 $\mathbf{H}$ 正定, 则 $\mathbf{w}^*$ 是 E 的严格局部极小值点;
2. 若 $\mathbf{w}^*$ 是 E 的局部极小值点且 $\mathbf{H}$ 非奇异, 则 $\mathbf{H}$ 正定.

♡

证明

(重构新增)(1) 由带余项的 Taylor 公式, 对 $\mathbf{v} \to \mathbf{0}$ 有

$$E(\mathbf{w}^* + \mathbf{v}) = E(\mathbf{w}^*) + \frac{1}{2}\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} + o(\|\mathbf{v}\|^2).$$

$\mathbf{H}$ 正定, 故存在 $\lambda_{\min} > 0$ 使 $\mathbf{v}^{\mathsf{T}}\mathbf{H}\mathbf{v} \geqslant \lambda_{\min}\|\mathbf{v}\|^2$, 于是

$$E(\mathbf{w}^* + \mathbf{v}) - E(\mathbf{w}^*) \geqslant \frac{\lambda_{\min}}{2}\|\mathbf{v}\|^2 + o(\|\mathbf{v}\|^2) > 0$$

对一切充分小的 $\mathbf{v} \neq \mathbf{0}$ 成立, 故 $\mathbf{w}^*$ 是严格局部极小值点.

(2) 设 $\mathbf{w}^*$ 是局部极小值点. 若存在特征值 $\lambda < 0$, 取对应的单位特征向量 $\mathbf{u}$, 则由 Taylor 公式

$$E(\mathbf{w}^* + t\mathbf{u}) = E(\mathbf{w}^*) + \frac{1}{2}\lambda t^2 + o(t^2) < E(\mathbf{w}^*)$$

对一切充分小的 $t \neq 0$ 成立, 与局部极小性矛盾; 故不存在负特征值. 又因 $\mathbf{H}$ 非奇异, 零不是特征值, 故全部特征值都为正. 由命题 7.3,$\mathbf{H}$ 正定.

□

注

(重构新增) 逐项来看条件的作用:$\nabla E(\mathbf{w}^*) = \mathbf{0}$ 使一阶项消失, 否则 $\mathbf{w}^*$ 沿梯度反方向还可下降; 二阶连续可微保证余项可写成 $o(\|\mathbf{v}\|^2)$, 从而二次项在 $\|\mathbf{v}\|$ 足够小时占优;$\mathbf{H}$ 正定则保证二次项本身严格为正. 第 (2) 款必须附加“$\mathbf{H}$ 非奇异”, 原书把“梯度为零且黑塞矩阵正定”径直表述为充分必要条件, 这一说法在二次模型(7.7)内成立, 但对一般光滑函数并不成立:$E(x) = x^4$ 在 $x = 0$ 处是严格局部极小, 而 $E''(0) = 0$, 黑塞矩阵为零矩阵, 既非正定也非负定. 出现零特征值时, 该方向上的形态由三阶及更高阶项决定, 需要另行分析; 深度网络中大量“平坦”极小值正是这种退化情形, 这也是不能仅凭黑塞矩阵判定所有极小值的原因.

在新的坐标系中, 基向量由特征向量 $\mathbf{u}_i$ 给出, 恒定误差 $E(\mathbf{w})$ 的等高线是轴对齐的椭圆, 以 $\mathbf{w}^*$ 为中心, 如图 7.2所示 (见练习 7.6).

命题 7.4 (恒定误差等高线的椭圆结构)

设 E 由(7.7)给出且 $\mathbf{H}$ 正定, 特征值为 λ_i、对应的正交规范特征向量为 $\mathbf{u}_i$. 则对每个 $c > E(\mathbf{w}^*)$, 水平集 $\{\mathbf{w} : E(\mathbf{w}) = c\}$ 是以 $\mathbf{w}^*$ 为中心、各轴沿 $\mathbf{u}_i$ 方向、半轴长为

$$r_i = \sqrt{\frac{2\big(c - E(\mathbf{w}^*)\big)}{\lambda_i}} \tag{7.15}$$

的椭圆.

证明

(重构新增) 由(7.11),$E(\mathbf{w}) = c$ 等价于 $\sum_i \lambda_i \alpha_i^2 = 2(c - E(\mathbf{w}^*))$. 令

$$\beta_i = \alpha_i \sqrt{\frac{\lambda_i}{2(c - E(\mathbf{w}^*))}},$$

则方程化为 $\sum_i \beta_i^2 = 1$, 即 $\boldsymbol{\beta}$ 空间中的单位球面. 由 $\mathbf{w} - \mathbf{w}^* = \sum_i \alpha_i \mathbf{u}_i$ 与 $\alpha_i = \beta_i r_i$ 可知, 该水平集是单位球面在正交变换 $\{\mathbf{u}_i\}$ 下、再沿各轴按 r_i 伸缩所得的像, 因而是以 $\mathbf{w}^*$ 为中心、轴沿 $\mathbf{u}_i$、半轴长为 r_i 的椭圆.

□

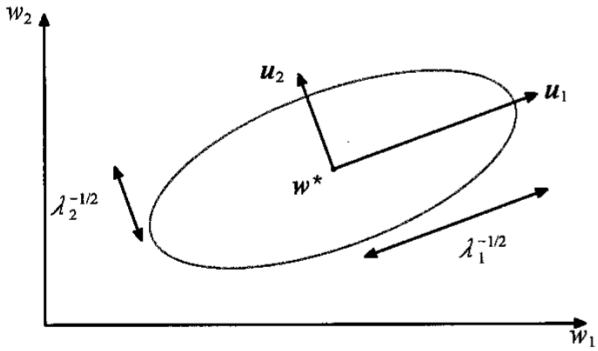


图 7.2: 在局部极小值 $\mathbf{w}^*$ 的邻域内, 误差函数可以被二次函数近似. 恒定误差的等高线变为椭圆形, 其轴与黑塞矩阵的特征向量 $\mathbf{u}_i$ 对齐, 长度则与相应特征值 λ_i 的平方根成反比.

注

(重构新增) 半轴长(7.15)表明: 曲率大的方向 (λ_i 大) 等高线挤得紧、半轴短, 曲率小的方向等高线拉得长. 当 $\lambda_{\max}/\lambda_{\min} \gg 1$ 时, 椭圆极度狭长, 形成节 7.3所分析的“山谷”几何, 比值 $\lambda_{\max}/\lambda_{\min}$ 也因此成为衡量优化难度的指标. 结论依赖 $\mathbf{H}$ 正定 (否则左端不是正定二次型, 水平集可能是双曲面而非椭圆) 以及 $c > E(\mathbf{w}^*)$(否则水平集为空集;$c = E(\mathbf{w}^*)$ 时退化为单点 $\mathbf{w}^*$). 若某些特征值相等, 对应半轴等长, 椭圆在相应子空间内退化为圆.

7.2 梯度下降优化

对于神经网络这种具有复杂结构的误差函数而言, 试图找到方程 $\nabla E(\mathbf{w}) = \mathbf{0}$ 的解析解是不现实的, 因此我们需要借助于迭代式的数值优化方法. 连续非线性函数的优化问题一直是学术界广泛研究的课题, 有大量的文献详细阐述了各种高效求解的方法. 大多数优化技术首先要为权重向量选取一个初始值 $\mathbf{w}^{(0)}$, 然后按照如下所示的迭代规则在权重空间中逐步搜索:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} + \Delta\mathbf{w}^{(\tau-1)}, \tag{7.16}$$

其中 τ 标记了迭代步骤. 不同的算法对权重向量如何更新 $\Delta\mathbf{w}^{(\tau)}$ 有不同选择.

除了最简单的神经网络模型之外, 绝大多数神经网络的误差曲面都具有极其复杂的形态, 因此最终求得的解将会很大程度上依赖于初始参数值 $\mathbf{w}^{(0)}$ 的选取等因素. 为了获得一个足够好的解, 通常需要多次运行基于梯度的优化算法, 每次运行都选用一个不同的随机初始点, 最终通过在一个独立的验证集上进行性能评估来确定最优解.

7.2.1 梯度信息的使用

深度神经网络的误差函数梯度可以通过误差反向传播技术高效计算 (参见节 8.1), 应用这些梯度信息可以显著提高网络训练速度. 请看下面的解释.

在(7.3)所给出的误差函数二次近似表达式中, 误差曲面的形态由向量 $\mathbf{b}$ 和矩阵 $\mathbf{H}$ 共同决定, 它们总共包含了 $W(W+3)/2$ 个独立的元素 (由于矩阵 $\mathbf{H}$ 具有对称性)(参见练习 7.7), 其中 W 表示权重向量 $\mathbf{w}$ 的维度 (即网络中可学习参数的总数). 因此, 该二次近似所对应的最小值位置取决于 $O(W^2)$ 个参数, 这意味着我们至少需要收集到 $O(W^2)$ 个相互独立的信息片段, 才能准确定位到该最小值. 如果不考虑梯度信息, 我们预计需要进行 $O(W^2)$ 次误差函数的数值评估, 而每一次评估又需要进行 $O(W)$ 步计算操作. 这样一来, 采用这种策略来寻找最小值所需的总计算量将高达 $O(W^3)$.

现在, 我们将上述情况与利用梯度信息的优化算法进行对比. 由于梯度向量 ∇E 的长度为 W, 因此每次计算 ∇E 都能提供 W 个独立的信息片段. 据此推断, 我们有望仅通过 $O(W)$ 次梯度评估操作就能找到函数的最小值 (参见节 8.1). 正如后续章节将要展示的那样, 利用误差反向传播算法, 每次梯度评估的计算复杂度仅为 $O(W)$, 因此, 现在我们能够在 $O(W^2)$ 的总计算复杂度内找到最小值. 尽管二次近似的有效性仅局限于最小值点附近的局部区

域, 但这种效率上的提升是具有普遍性的. 正是基于这一关键优势, 利用梯度信息成了当前所有实用的神经网络训练算法的基础.

7.2.2 批量梯度下降

利用梯度信息指导权重更新的一种最朴素的方法是在(7.16)中按照负梯度方向进行小步长的迭代调整, 即

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E(\mathbf{w}^{(\tau-1)}), \tag{7.17}$$

其中参数 $\eta(\eta > 0)$ 被定义为学习率 (learning rate). 在完成每次权重更新后, 需要基于新的权重向量 $\mathbf{w}^{(\tau)}$ 重新计算梯度, 并重复上述过程. 在每一次迭代过程中, 权重向量都朝着当前位置处误差函数下降最快的方向进行更新.

定义 7.5 (学习率)

在权重更新规则(7.17)中, 决定每一步沿负梯度方向移动幅度的正参数 $\eta > 0$ 称为**学习率** (learning rate).

注

(重构新增) 学习率是梯度下降中唯一不由误差函数本身决定的正参数, 因而也是最重要的超参数. 它的量纲必须与“梯度乘以参数”的量纲相消, 所以固定 η 的做法隐含了一种尺度约定: 当误差函数整体乘以常数、或参数被重新缩放时, 合适的 η 会随之改变, 这正是小节 7.4.1要讨论输入归一化的原因之一.η 过小会使每一步的下降量被二阶项淹没, 收敛缓慢;η 过大会使迭代在极小值两侧来回跳跃甚至发散. 本章节 7.3将在二次近似下给出定量边界 $\eta < 2/\lambda_{\max}$, 其中 $\lambda_{\max}$ 是黑塞矩阵的最大特征值, 这个边界说明了“学习率必须与曲率匹配”的确切含义.

定义 7.6 (梯度下降与最速下降)

按(7.17)沿当前位置的负梯度方向迭代更新权重向量的方法称为**梯度下降** (gradient descent), 也称**最速下降** (steepest descent).

注

(重构新增) “最速下降”这一名称来自如下事实: 在所有单位长度的方向 $\mathbf{d}$ 中, 方向导数 $\mathbf{d}^{\mathrm{T}}\nabla E$ 在 $\mathbf{d} = -\nabla E/\|\nabla E\|$ 处取到最小值, 因此负梯度方向是局部下降最快的方向. 但“局部最快”不等于“全局最优路径”:图 7.3所示的狭长山谷中, 负梯度方向几乎垂直于通向极小值的方向, 于是梯度下降会在山谷两侧来回振荡而沿谷底前进极慢. 理解这一点是理解动量法、Adam 等改进算法的前提: 它们改动的都不是“下降方向最快”这一局部性质, 而是对该局部性质的累积与修正.

定义 7.7 (批处理方法)

每次计算误差函数或其梯度都遍历全部训练样本的技术称为**批处理** (batch) 方法.

注

(重构新增) 由于误差函数是基于整个训练集定义的 (见(7.18)), 批处理是(7.17)最直接的实现方式, 它给出的梯度是误差函数梯度的精确值, 不含随机误差. 代价是单次迭代的计算量与训练集规模 N 成正比: 当 N 达到 10^6 量级时, 一次参数更新就要处理上百万个样本, 这在数据规模增长时很快变得不可接受. 批处理的这一缺点直接催生了后面的随机梯度下降与小批量方法; 反过来, 当数据集很小或需要精确梯度 (例如做数值诊断) 时, 批处理仍是首选.

7.2.3 随机梯度下降

深度学习方法的性能通常会随着训练数据集规模的扩大而显著提升. 然而, 当训练集中包含海量数据时, 批处理方法的效率会急剧下降, 因为每一次误差函数或梯度的计算都需要遍历整个数据集. 为了寻求一种更为高效的优化策略, 我们注意到, 对于一组独立同分布的观测样本, 基于最大似然估计构建的误差函数可以自然地分解为每

个数据点对应项的和:

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}). \tag{7.18}$$

在处理大规模数据集时, 应用最为广泛的训练算法是基于梯度下降的一种称为随机梯度下降 (Stochastic Gradient Descent, SGD) 的逐次迭代算法 (Bottou, 2010). 该算法每次仅基于一个数据点来更新权重向量, 具体的更新规则如下:

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta \nabla E_n(\mathbf{w}^{(\tau-1)}), \tag{7.19}$$

其中索引 n 在每次迭代后循环递增.

定义 7.8 (随机梯度下降)

按(7.19)每次仅用单个数据点的误差函数 E_n 计算梯度并更新权重的方法称为**随机梯度下降** (Stochastic Gradient Descent, SGD). 其中索引 n 按 $1, 2, \cdots, N$ 循环递增; 对整个训练集进行一次完整的遍历称为一个**训练轮次** (epoch); 当数据以流的形式持续生成时, 这种技术也称为**在线梯度下降** (online gradient descent). ♣

注

(重构新增) 随机梯度下降的合理性在于 ∇E_n 是 ∇E 的无偏估计: 若 n 从 $\{1, \cdots, N\}$ 中均匀抽取, 则 $\mathbb{E}[\nabla E_n(\mathbf{w})] = \frac{1}{N}\sum_{n} \nabla E_n(\mathbf{w}) = \nabla E(\mathbf{w})$. 因此单步梯度虽然带有随机噪声, 但其期望指向真实梯度方向, 噪声在迭代平均中会被部分抵消. 这一性质依赖“数据点独立同分布”这一前提 (见(7.18)上方); 若数据高度相关, 单点梯度会产生系统性偏差, 此时必须先打乱数据 (见小节 7.2.4). 另外, 单个数据点对应的误差函数 E_n 与整体误差函数 E 的驻点一般不同, 正是这一差异使 SGD 的噪声具有逃离较浅局部极小值的作用, 这一点与批处理梯度下降有本质区别.

上述更新过程通过循环遍历训练集中的数据点来反复执行. 对整个训练集进行一次完整的遍历称为一个训练轮次 (epoch). 当数据以流的形式持续生成时, 这种技术有时也称为在线梯度下降 (online gradient descent). 下面的算法对随机梯度下降做了总结.

算法 7.1　随机梯度下降

Input: 由 $n \in \{1, \cdots, N\}$ 索引的数据点构成的训练集
　每个数据点的误差函数 $E_n(\mathbf{w})$
　学习率参数 η
　初始权重向量 $\mathbf{w}$
Output: 最终的权重向量 $\mathbf{w}$
$n \leftarrow 1$
repeat
　$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_n(\mathbf{w})$　// 更新权重向量
　$n \leftarrow n + 1 \pmod N$　// 遍历所有数据
until 收敛
return w

例题 7.1 数据冗余下的计算量对比　考虑一个极端的例子: 复制某数据集的每一个数据点, 把它的规模扩大一倍. 此时比较批处理梯度下降与随机梯度下降完成同样多“有效”更新所需的工作量.

解

设原数据集含 N 个数据点，复制后含 $2N$ 个. 复制不改变数据的经验分布，因此整体误差函数满足 $E_{新}(\mathbf{w}) = 2E_{旧}(\mathbf{w})$，其梯度也恰好是原来的两倍. 对批处理梯度下降，把学习率取为原来的一半即可保持更新量不变，然而评估一次批梯度所需遍历的样本数由 N 变成 $2N$，单次更新的计算量随之加倍，故完成同样多迭代的总计算量加倍. 对随机梯度下降，每次更新只用到单个数据点，更新规则(7.19)与 N 无关；若把遍历顺序安排为在每个原始数据点及其副本上各更新一次，则相邻两次更新的梯度相同 (因为 E_n 被复制)，其效果等价于把原数据集上的那一步重复执行两次，而单步代价不变. 也就是说,SGD 完全不受数据冗余的影响，它把重复数据带来的额外计算“浪费”在重复的信息上，而不是浪费在计算批梯度上.

□

随机梯度下降的另一个重要特性是其具备逃离局部极小值的能力，因为整个数据集的误差函数的驻点，通常并不会是每个单独数据点对应误差函数的驻点.

7.2.4 小批量方法

随机梯度下降的一个缺点是，基于单个数据点计算出的梯度，可能大大偏离整个数据集上误差函数的梯度. 为了兼顾计算效率和估计精度，我们折衷一下，每次迭代时使用一小批数据点 (称为小批量,mini-batch) 来估计梯度. 在确定小批量的合适大小时，需要注意，利用 N 个样本估计均值的误差为 $\sigma/\sqrt{N}$(参见练习 7.8)，其中 σ 为生成这些数据的分布的标准差. 这表明，通过增加小批量的规模来提升梯度估计精度的收益是递减的. 例如，将小批量的规模增加 100 倍，估计误差仅能降低到原先的十分之一. 选择小批量规模时还需要考虑的另一个因素是，充分利用运行代码的底层硬件架构. 例如，在某些硬件平台上，小批量的规模如果是 2 的幂次 (如 64、128、256 等) 就可以获得更好的性能.

定义 7.9 (小批量)

在每次迭代中从整个训练集中随机抽取的、用于估计整体误差函数梯度的一小组数据点称为**小批量** (mini-batch).

♣

注

(重构新增) 小批量方法在批处理与单点 SGD 之间插值: 批量规模 $B=1$ 时退化为(7.19),$B=N$ 时退化为批处理. 它的两个极端分别对应“梯度噪声大、但单步代价低”和“梯度精确、但单步代价高”, 而 B 取中间值时, 梯度估计的方差按 $1/B$ 下降, 单步代价按 B 增长, 二者的乘积大致不变——这正是“收益递减”的确切含义: 把 B 扩大 100 倍只能把标准差降到 1/10. 定义中“随机抽取”不是可有可无的措辞: 若数据按类别或时间排序, 连续取出的 B 个样本会高度相关, 梯度估计将产生系统偏差, 因此实践中必须先打乱数据, 并在每个训练轮次重新打乱.

在使用小批量方法时, 要注意, 数据点应当从整个数据集中随机选取. 因为在原始数据集中, 由于数据采集方式 (例如数据点是按照字母顺序或日期排序), 相邻的数据点之间可能存在相关性. 为了消除这种相关性的影响, 通常的做法是先将整个数据集进行随机打乱, 然后按顺序划分为多个小批量. 此外, 在每次完整遍历数据集后, 还可以再次进行数据打乱, 以确保每个小批量的数据组合都是全新的, 这有助于模型跳出局部最优解. 下面的算法总结了这种算法的变体. 值得注意的是, 即使采用了小批量方法, 这种学习算法通常仍被称为“随机梯度下降”.

算法 7.2　带有小批量的随机梯度下降

Input: 由 $n \in \{1, \cdots, N\}$ 索引的数据点构成的训练集
　　批量规模 B
　　每个小批量的误差函数 $E_{n:n+B-1}(\mathbf{w})$
　　学习率参数 η
　　初始权重向量 $\mathbf{w}$
Output: 最终的权重向量 $\mathbf{w}$
$n \leftarrow 1$
repeat
　　$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla E_{n:n+B-1}(\mathbf{w})$　// 更新权重向量
　　$n \leftarrow n + B$
　　if $n > N$ **then**
　　　　打乱数据
　　　　$n \leftarrow 1$
　　end if
until 收敛
return $\mathbf{w}$

7.2.5 参数初始化

像梯度下降这样的迭代式优化算法, 都要求我们为需要学习的参数设定初始值. 参数初始化的具体策略会对算法收敛的速度以及训练得到的网络的泛化能力产生显著的影响. 遗憾的是, 目前还没有能够系统性地指导参数初始化过程的完善理论.

尽管如此, 在参数初始化过程中一个关键的考虑因素是对称性破缺 (symmetry breaking). 考虑网络中的一组隐藏单元或输出单元, 它们都接收相同的输入信号. 如果这些单元的所有参数都被赋予了相同的值, 例如全部初始化为零, 那么这些单元的参数在训练过程中将始终步调一致地进行更新, 并且每个单元都计算出相同的函数, 导致功能上的冗余. 为了解决这一问题, 可以从某个预先定义的分布中随机采样来初始化参数, 以打破这种对称性. 在计算资源允许的情况下, 可以采用不同的随机初始化策略来多次训练网络, 最终选择在独立验证集上性能最佳的模型.

定义 7.10 (对称性破缺)

在参数初始化时, 通过给本可互换的单元赋予互不相同的随机参数值, 使这些单元在训练中不再保持同步更新的做法, 称为**对称性破缺** (symmetry breaking).

注

(重构新增) 对称性的根源是网络函数对隐藏单元的置换不变 (见小节 6.2.4): 若同一层的两个单元具有完全相同的输入连接与参数, 则它们的输出相同, 反向传播传给它们的梯度也相同, 于是无论训练多久二者始终保持相同. 此时这两个单元在功能上等价于一个单元, 网络的表达能力被白白浪费. 随机初始化只是打破对称性的一种手段, 其要点是“各不相同”而非“接近零”: 初始化值过大同样会破坏训练, 因为过大的预激活会使 sigmoid、tanh 等饱和激活函数的导数趋近于零 (参见小节 7.4.2关于梯度消失的讨论). 初始化尺度与激活函数、网络深度之间的定量关系由下面的 He 初始化给出.

用于初始化权重的分布通常要么是范围为 $[-\varepsilon,\varepsilon]$ 的均匀分布, 要么是形式为 $\mathcal{N}(0,\varepsilon^2)$ 的零均值 Gauss 分布.ε 值的选择很重要, 在各种经验方法中,He 初始化 (He et al., 2015b) 得到了广泛使用. 考虑一个网络, 其中第 l 层计算以下转换:

$$a_i^{(l)} = \sum_{j=1}^{M} w_{ij}^{(l)} z_j^{(l-1)}, \tag{7.20}$$

$$z_i^{(l)} = \mathrm{ReLU}\big(a_i^{(l)}\big), \tag{7.21}$$

其中 M 是发送连接到单元 i 的单元数,ReLU 激活函数由(6.27)给出. 假设我们使用 Gauss 分布 $\mathcal{N}(0,\varepsilon^2)$ 初始化权重, 并假设第 $(l-1)$ 层的预激活 $a_j^{(l-1)}$ 具有零均值和方差 λ^2(参见练习 7.9), 则可以容易地证明

$$\mathbb{E}\big[a_i^{(l)}\big] = 0, \tag{7.22}$$

$$\mathrm{var}\big[a_i^{(l)}\big] = \frac{M}{2}\varepsilon^2\lambda^2. \tag{7.23}$$

命题 7.5 (He 初始化的方差传播)

设第 $l-1$ 层的预激活 $a_j^{(l-1)}$ 相互独立、分布关于原点对称, 并满足 $\mathbb{E}[a_j^{(l-1)}]=0$、$\mathrm{var}[a_j^{(l-1)}]=\lambda^2$; 设 $z_j^{(l-1)} = \mathrm{ReLU}(a_j^{(l-1)})$ 相互独立、与权重独立, 权重 $\{w_{ij}^{(l)}\}$ 相互独立且服从 $\mathcal{N}(0,\varepsilon^2)$, 第 l 层的预激活由(7.20)给出. 则(7.22)与(7.23)成立; 特别地, 若要求第 l 层的预激活也具有方差 λ^2, 则初始化分布的标准差应取

$$\varepsilon = \sqrt{\frac{2}{M}}. \tag{7.24}$$

♠

证明

(重构新增) 由 $\mathbb{E}[w_{ij}^{(l)}]=0$ 及独立性,

$$\mathbb{E}\big[a_i^{(l)}\big] = \sum_{j=1}^{M} \mathbb{E}\big[w_{ij}^{(l)}\big]\mathbb{E}\big[z_j^{(l-1)}\big] = 0,$$

即(7.22). 再由各项独立且均值为零,

$$\mathrm{var}\big[a_i^{(l)}\big] = \sum_{j=1}^{M} \mathrm{var}\big(w_{ij}^{(l)} z_j^{(l-1)}\big) = \sum_{j=1}^{M} \mathbb{E}\big[(w_{ij}^{(l)})^2\big]\mathbb{E}\big[(z_j^{(l-1)})^2\big] = M\varepsilon^2\,\mathbb{E}\big[(z_j^{(l-1)})^2\big].$$

由于 $z_j^{(l-1)} = \mathrm{ReLU}(a_j^{(l-1)})$, 而 $a = a_j^{(l-1)}$ 的分布关于原点对称, 故 a 与 $-a$ 同分布; 又 $\mathrm{ReLU}(a)^2 = a^2$ 当 $a>0$、为 0 当 $a\leqslant 0$, 于是

$$\mathbb{E}\big[\mathrm{ReLU}(a)^2\big] = \frac{1}{2}\mathbb{E}\big[a^2\big] = \frac{1}{2}\lambda^2,$$

其中最后一步用到 $\mathbb{E}[a]=0$. 于是 $\mathrm{var}[a_i^{(l)}] = M\varepsilon^2\lambda^2/2$, 即(7.23). 令其等于 λ^2 并解出 ε, 即得(7.24).

□

注

(重构新增) 推导中每一步的条件都不可省. 独立性保证方差可以逐项相加, 且交叉项 $\mathbb{E}[w_{ij}z_j w_{ik}z_k] = \mathbb{E}[w_{ij}]\mathbb{E}[w_{ik}]\mathbb{E}[z_j z_k]$ 因零均值而消失;$\mathbb{E}[w_{ij}^{(l)}] = 0$ 使 $\mathbb{E}[wz] = 0$, 从而 $\mathrm{var}(wz) = \mathbb{E}[w^2]\mathbb{E}[z^2]$;$a_j^{(l-1)}$ 的分布关于原点对称则是因子 1/2 的来源——ReLU 把负半边截成零, 恰有一半的概率质量被保留, 故 $\mathbb{E}[\mathrm{ReLU}(a)^2] = \mathbb{E}[a^2]/2$. 若把激活函数换成 tanh 或恒等映射, 该因子变为 1, 相应的尺度是 $\sqrt{1/M}$ 型 (Glorot 初始化的取值即介于 $\sqrt{1/M}$ 与 $\sqrt{2/M}$ 之间, 由前后层单元数决定). 原书把 λ^2 归于第 $l-1$ 层单元的输出 $z_j^{(l-1)}$; 由于 ReLU 输出的二阶矩只有预激活二阶矩的一半, 因子 1/2 只有在把 λ^2 理解为预激活方差时才成立, 这里按后一理解陈述, 二者在 $\mathbb{E}[z] \approx 0$ 的近似下一致. 结论的实际意义是: 若每层都按(7.24)初始化, 则前向传播中预激活的方差逐层保持不变; 若 ε 偏小, 方差逐层衰减到零, 信号无法传到深层; 若 ε 偏大, 方差逐层放大, 最终饱和或溢出. 这正是“既不衰减也不爆炸”的定量表述, 也是后面小节 7.4.2讨论梯度消失与梯度爆炸的出发点. 需要注意, 推导假定各层之间严格独立同分布, 真实网络中这一假定只是近似, 因此(7.24)应视为尺度选择的指导而非精确公式.

理想情况下, 我们希望确保在从一层传播到下一层时, 预激活的方差既不会衰减到零, 也不会显著增长. 如果我们因此要求第 l 层的单元也具有方差 λ^2, 那么对于一个具有 M 个输入的单元, 用于初始化其权重的 Gauss 分布的标准差由(7.24)给出.

我们还可以将初始化分布的尺度 ε 视为一个超参数, 并在多次训练中探索不同的 ε 值. 偏置参数通常设置为小的正值, 以确保大多数预激活在学习开始时处于激活状态. 这对 ReLU 特别有帮助, 我们希望预激活为正值, 以便有非零的梯度来驱动学习过程.

除了上述方法之外, 另一类重要的神经网络参数初始化策略, 是利用在其他相关任务上预训练得到的网络权重, 或者利用各种无监督学习方法. 这些技术都属于迁移学习 (transfer learning) 的范畴 (参见小节 6.3.4).

7.3 收敛

在实际应用梯度下降算法时, 我们需要谨慎地选择参数学习率 η 的值. 考虑图 7.3所示的情形: 这是一个假想的二维权重空间中的简化误差曲面, 该曲面的曲率在不同方向上存在显著差异, 形成了一个狭长的“山谷”结构. 对误差曲面上的大多数点来说, 批量梯度下降算法计算出的局部梯度向量 (与该点的等高线垂直) 并不直接指向最小值点. 直观上我们可能会认为增大学习率 η 应该能让算法在权重空间中迈开更大的步伐, 从而更快地收敛到最小值. 然而, 如果一味地加大步长, 权重更新会在山谷两侧来回震荡, 当 η 超过某个阈值时, 这种震荡甚至会变得越来越剧烈, 导致算法无法收敛. 为了避免这种发散的振荡, 学习率 η 必须设定得足够小, 那么, 沿着山谷方向向最小值靠近的速度又可能会变得非常缓慢. 在这种情况下, 梯度下降算法需要经过相当多次的小步迭代才能逐渐逼近最小值, 导致整体的优化效率非常低下.

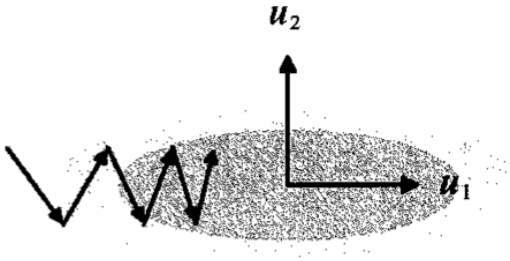


图 7.3: 步长固定的梯度下降的示意图, 这种梯度下降通常用于在不同方向上曲率大不相同的误差函数. 误差表面 E 形如一个“山谷”, 如椭圆所示. 注意, 对于权重空间中的大多数点来说, 局部负梯度 $-\nabla E$ 并不指向误差函数的最小值. 因此, 梯度下降的连续步骤可能会在“山谷”中来回振荡, 导致沿“山谷”向最小值前进得非常缓慢. 向量 $\mathbf{u}_1$ 和 $\mathbf{u}_2$ 是黑塞矩阵的特征向量.

通过考虑误差函数在最小值附近的二次近似, 我们可以深入了解这个问题的本质 (参见小节 7.1.1). 根据(7.7)、

(7.8)和(7.10), 在这种近似中, 误差函数的梯度可以写为

$$\nabla E = \sum_i \alpha_i \lambda_i \mathbf{u}_i. \tag{7.25}$$

再次使用(7.10), 我们可以将权重向量的变化表示为对应系数 α_i 的变化:

$$\Delta \mathbf{w} = \sum_i \Delta\alpha_i \mathbf{u}_i. \tag{7.26}$$

结合(7.25)、(7.26)与梯度下降公式(7.17), 并利用黑塞矩阵特征向量的正交性关系(7.9), 可以得到梯度下降算法每次迭代步中 α_i 的变化量表达式:

$$\Delta\alpha_i = -\eta \lambda_i \alpha_i. \tag{7.27}$$

由此可以直接推导出 (见练习 7.10) 如下收敛性结论.

定理 7.2 (二次近似下的线性收敛)

设 $E(\mathbf{w})$ 由(7.7)精确给出, 其中 $\mathbf{H}$ 正定, 特征值为 $0 < \lambda_{\min} = \lambda_1 \leqslant \cdots \leqslant \lambda_W = \lambda_{\max}$, 对应的正交规范特征向量为 $\mathbf{u}_i$, 并设 E 的极小值点为 $\mathbf{w}^*$. 若按梯度下降(7.17)从任意初值 $\mathbf{w}^{(0)}$ 出发迭代, 则对每个 i,

$$\alpha_i^{(\tau)} = (1 - \eta\lambda_i)\, \alpha_i^{(\tau-1)}, \tag{7.28}$$

从而经过总共 T 步之后

$$\alpha_i^{(T)} = (1 - \eta\lambda_i)^T \alpha_i^{(0)}. \tag{7.29}$$

并且 $\mathbf{w}^{(T)} \to \mathbf{w}^*$ 对一切初值成立, 当且仅当学习率满足

$$0 < \eta < \frac{2}{\lambda_{\max}}. \tag{7.30}$$

此时收敛是线性的, 收缩因子为 $\rho(\eta) = \max_i |1 - \eta\lambda_i|$.

♡

证明

由(7.27),α_i 在第 τ 步的新旧取值满足

$$\alpha_i^{(\tau)} = \alpha_i^{(\tau-1)} + \Delta\alpha_i = (1 - \eta\lambda_i)\, \alpha_i^{(\tau-1)},$$

即(7.28); 反复代入 T 次即得(7.29).

其次说明 α_i 的几何含义. 在(7.10)两端左乘 $\mathbf{u}_i^{\mathsf{T}}$ 并用正交规范性(7.9), 得

$$\mathbf{u}_i^{\mathsf{T}}(\mathbf{w} - \mathbf{w}^*) = \alpha_i,$$

即 α_i 是 $\mathbf{w} - \mathbf{w}^*$ 在 $\mathbf{u}_i$ 方向上的分量, 可解释为沿着方向 $\mathbf{u}_i$ 到极小值点的 “距离” .

于是

$$\|\mathbf{w}^{(T)} - \mathbf{w}^*\|^2 = \sum_i \big(\alpha_i^{(T)}\big)^2 = \sum_i (1 - \eta\lambda_i)^{2T} \big(\alpha_i^{(0)}\big)^2.$$

因此 $\mathbf{w}^{(T)} \to \mathbf{w}^*$ 当且仅当每个分量都趋于零, 即 $|1 - \eta\lambda_i| < 1$ 对所有 i 成立. 由于 $\lambda_i > 0$, 不等式 $|1 - \eta\lambda_i| < 1$ 等价于 $-1 < 1 - \eta\lambda_i < 1$, 亦即 $0 < \eta\lambda_i < 2$, 或 $0 < \eta < 2/\lambda_i$; 该条件对全部 i 成立当且仅当 $0 < \eta < 2/\lambda_{\max}$. 最后, 由上式还得到

$$\|\mathbf{w}^{(T)} - \mathbf{w}^*\| \leqslant \rho(\eta)^T \, \|\mathbf{w}^{(0)} - \mathbf{w}^*\|,$$

其中 $\rho(\eta) = \max_i |1 - \eta\lambda_i|$, 即误差按几何级数衰减, 故为线性收敛.

□

注

(重构新增) 这个定理把 “学习率必须与曲率匹配” 变成了一个可计算的判据. 逐条看条件:$\mathbf{H}$ 正定保证所有 $\lambda_i > 0$, 从而 $2/\lambda_i > 0$、上界有意义; 若某个 $\lambda_i \leqslant 0$, 则 $\mathbf{w}^*$ 不是极小值点, 梯度下降没有理由向其收敛.$|1 - \eta\lambda_i| < 1$ 允许因子取负值, 此时 α_i 在极小值两侧振荡但振幅逐次减半式衰减; 一旦 $\eta > 2/\lambda_i$, 因子绝对值大于 1, 该分量发散, 这正是图 7.3中山谷两侧振荡越来越剧烈的定量解释. 定理给出的只是二次模型下的局部结论: 对一般非线性

误差函数, 只有在 $\mathbf{w}^*$ 的足够小邻域内二次近似才有效, 因此它描述的是“进入收敛域之后”的行为, 而不保证从任意远处出发都能到达该邻域. 最后, 最优学习率并不是 $2/\lambda_{\max}$: 使 $\rho(\eta)$ 最小的是 $\eta^* = 2/(\lambda_{\max}+\lambda_{\min})$, 此时 $\rho = (1-\eta^*\lambda_{\min}) = (1-\eta^*\lambda_{\max})$ 的绝对值都等于 $(\kappa-1)/(\kappa+1)$, 其中 $\kappa = \lambda_{\max}/\lambda_{\min}$(见定义 7.11). 原书只讨论了 η 取最大允许值这一端点, 因为它直接显示了最慢方向上的困难.

由(7.30)可以看到, 学习率的上界由最大特征值决定, 而实际的收敛速度却由收缩最慢的方向决定. 虽然允许 $1-\eta\lambda_i$ 为负 (这会产生 α_i 的振荡值), 但只要所有因子的绝对值都小于 1, 振荡仍会衰减; 一旦某个因子越过 -1, 对应的分量就会发散. 于是 η 的值需要限制为 $\eta < 2/\lambda_{\max}$, 其中 $\lambda_{\max}$ 是最大的特征值. 然而, 收敛速度主要由最小的特征值控制, 因此当 η 设定为最大允许值时, 沿着对应于最小特征值方向的收敛 (图 7.3中椭圆的长轴) 由下式控制:

$$1-\frac{2\lambda_{\min}}{\lambda_{\max}}, \tag{7.31}$$

其中 $\lambda_{\min}$ 是最小的特征值.

定义 7.11 (条件数)

设 $\mathbf{H}$ 正定, 其最大、最小特征值分别为 $\lambda_{\max}$ 与 $\lambda_{\min}$, 则称比值

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} \tag{7.32}$$

为黑塞矩阵 $\mathbf{H}$ 的**条件数** (condition number).κ 的倒数 $\lambda_{\min}/\lambda_{\max}$ 越接近 1, 各方向曲率越接近;κ 越大, 各方向曲率差异越悬殊.

♣

注

(重构新增) 条件数衡量的是误差曲面在不同方向上“被压扁”的程度, 它不依赖坐标系的旋转与平移, 因而是误差曲面本身的几何不变量. 它之所以成为优化难度的指标, 是因为梯度下降对各特征方向的收缩因子 $|1-\eta\lambda_i|$ 不可能同时接近零: 为了不让最大特征值方向发散, 必须取 $\eta < 2/\lambda_{\max}$, 于是最小特征值方向的因子至少是 $1-2\lambda_{\min}/\lambda_{\max} = 1-2/\kappa$, 当 $\kappa = 10^4$ 时该因子约为 0.9998, 需要上万步才能把对应分量缩小一个数量级. 更精确地说, 达到精度 ε 所需迭代次数约为 $\frac{\kappa}{2}\ln(1/\varepsilon)$, 即与条件数成正比. 这个结论解释了为什么输入归一化 (小节 7.4.1) 与动量法、Adam 等算法 (小节 7.3.1) 在实践中如此重要: 它们或者直接减小 κ, 或者改变迭代对 κ 的依赖方式.

推论 7.1 (最慢方向的收敛速度)

在定理 7.2的设定下, 当 $\eta = 2/\lambda_{\max}$ 时, 沿 $\mathbf{u}_{\min}$ 方向的收缩因子为(7.31), 即 $1-2/\kappa$. 因此 κ 越大, 向极小值收敛越慢.

♡

证明

把 $\eta = 2/\lambda_{\max}$ 代入(7.28), 对 $\lambda_i = \lambda_{\min}$ 得

$$1-\eta\lambda_{\min} = 1-\frac{2\lambda_{\min}}{\lambda_{\max}} = 1-\frac{2}{\kappa},$$

即(7.31); 由定义 7.11,$\lambda_{\min}/\lambda_{\max} = 1/\kappa$. 该因子随 κ 增大而增大并趋于 1, 故收敛速度随 κ 增大而减慢.

□

注

(重构新增) 这个推论给出的正是“狭长山谷”的代价:κ 越大, 沿长轴方向每步只能把距离缩短 $2/\kappa$ 的比例, 而梯度下降又必须沿长轴走完大部分路程. 需要强调的是,$\eta = 2/\lambda_{\max}$ 是允许区间的端点, 此时沿 $\mathbf{u}_{\max}$ 方向的因子恰为 -1, 该分量不衰减而持续振荡, 因此实践中不会真的取到这个端点, 只会取略小的值; 推论的意义在于给出端点的极限行为, 说明为什么单纯调小学习率无法解决病态问题. 若把定理 7.2之后的注记中给出的最优学习率 $\eta^* = 2/(\lambda_{\max} + \lambda_{\min})$ 代回, 收缩因子为 $(\kappa - 1)/(\kappa + 1) \approx 1 - 2/\kappa$, 与(7.31)同阶, 这从另一侧面说明条件数造成的困难不能靠“选对学习率”消除.

7.3.1 动量

解决特征值差异悬殊问题的一种简单技巧是, 在梯度下降公式中引入动量 (momentum) 项. 这实际上相当于给权重向量在权重空间中的运动增加了惯性, 从而缓解了图 7.3中所示的振荡现象.

定义 7.12 (动量)

引入动量项后的梯度下降更新公式为

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E\left(\mathbf{w}^{(\tau-1)}\right) + \mu \Delta \mathbf{w}^{(\tau-2)}, \tag{7.33}$$

其中 μ 称为**动量参数** (momentum parameter), 权重向量仍按(7.16)更新. 这种迭代方法称为**动量法** (momentum).

注

(重构新增) (7.33)把上一步的更新量按比例 μ 保留下来, 于是每一步的位移都带有“历史速度”的记忆: 在梯度方向持续一致的区域, 历次更新相互叠加, 等效步长被放大; 在梯度方向反复改变的区域, 历次更新相互抵消, 等效步长被压缩. 这正是“惯性”的数学含义, 也解释了为什么动量法能在不增大 η 的前提下加快沿山谷方向的推进. 条件方面,μ 必须满足 $|\mu| < 1$, 否则历史项会被无限放大 (见命题 7.6);$\mu = 0$ 时(7.33)退化为普通梯度下降.μ 与 η 不能独立调节: 由(7.34)可见真正起作用的是组合 $\eta/(1-\mu)$, 因此增大 μ 与增大 η 在低曲率方向上效果相似, 但在高曲率方向上二者截然不同.

为了理解动量项的作用, 我们首先考虑权重向量在误差曲面上曲率相对较小的区域中的运动情况, 如图 7.4所示. 如果我们近似地认为梯度在该区域内保持不变, 那么我们可以对(7.33)进行多次迭代, 并将由此产生的一系列权重更新值累加起来, 得到

$$\Delta \mathbf{w} = -\eta \nabla E \left\{1 + \mu + \mu^2 + \cdots\right\} = -\frac{\eta}{1-\mu} \nabla E. \tag{7.34}$$

命题 7.6 (动量法的有效学习率)

设在迭代过程中梯度保持为常向量 $\mathbf{g} = \nabla E$, 且 $|\mu| < 1$. 则从任意初始更新量出发, 按(7.33)产生的更新量收敛于

$$\Delta \mathbf{w} = -\frac{\eta}{1-\mu} \mathbf{g}, \tag{7.35}$$

即动量项把学习率由 η 放大为 $\eta/(1-\mu)$.

证明

记 $\boldsymbol{\delta}_k = \Delta\mathbf{w}^{(k)}$. 由(7.33),$\boldsymbol{\delta}_k = -\eta\mathbf{g} + \mu\boldsymbol{\delta}_{k-1}$. 反复代入得

$$\boldsymbol{\delta}_k = -\eta\mathbf{g}\sum_{j=0}^{k-1}\mu^j + \mu^k\boldsymbol{\delta}_0.$$

当 $|\mu| < 1$ 时 $\mu^k \to 0$, 且几何级数 $\sum_{j=0}^{\infty}\mu^j = 1/(1-\mu)$ 收敛, 故

$$\boldsymbol{\delta}_k \longrightarrow -\eta\mathbf{g}\cdot\frac{1}{1-\mu} = -\frac{\eta}{1-\mu}\mathbf{g},$$

即(7.35).

□

注

(重构新增) 结论依赖两个条件.“梯度在迭代期间保持不变”是一个局部化的假设: 只有在曲率很低、梯度随位置变化很慢的区域, 连续若干步的梯度才近似相同, 因此(7.35)描述的是低曲率区域的行为, 而不是全局结论. “$|\mu| < 1$”则是级数收敛的充要条件,$\mu \to 1$ 时有效学习率趋于无穷, 必然发散. 由此可以看出动量法与单纯增大学习率的区别: 在低曲率区域二者都等价于把步长乘以 $1/(1-\mu)$, 但在高曲率区域, 梯度方向逐步反转,(7.33)中的历史项相互抵消, 等效学习率回落到接近 η, 因此不会像增大 η 那样立刻触发(7.30)所禁止的振荡. 原书据此把 μ 的取值范围取为 $0 \leqslant \mu \leqslant 1$, 实践中常用值为 $\mu = 0.9$, 对应约 10 倍的有效步长放大.

可以看到, 添加动量项能有效将学习率从 η 提高到 $\eta/(1-\mu)$. 在曲率较低的表面上, 使用固定学习率的梯度下降会导致逐渐变小的步长, 对应于线性收敛; 在这种情况下, 添加动量项的效果类似于提高学习率, 向误差函数谷底的收敛更快.

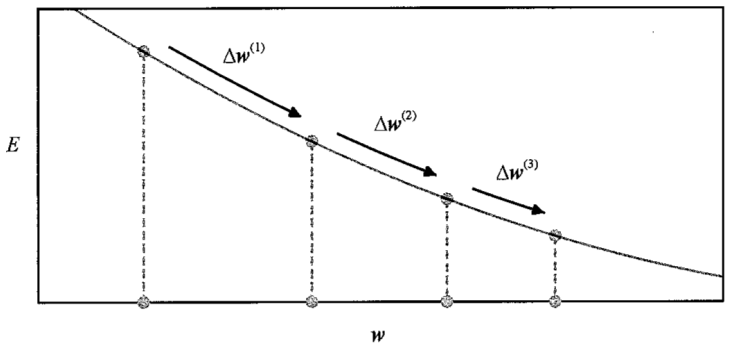


图 7.4: 在一个曲率较低的表面上, 使用固定学习率的梯度下降会导致逐渐变小的步长, 对应于线性收敛. 在这种情况下, 添加动量项的效果类似于提高学习率.

相反, 在误差曲面有较高曲率的区域, 如果不加动量项, 梯度下降会出现振荡现象, 如图 7.5所示. 在这种情况下, 动量项的连续贡献将会趋于相互抵消, 使得有效的学习率接近于 η. 因此, 动量项能够加速模型向最小值点收敛的速度, 同时又避免了发散的振荡.图 7.6对动量项的作用进行了示意性的说明.

尽管引入动量项可以改善梯度下降算法的性能, 但它也引入了一个新的参数 μ, 它的值需要与学习率参数 η 一起进行合理的选择. 从(7.34)可以看出,μ 的取值范围应为 $0 \leqslant \mu \leqslant 1$. 实践中常用的值是 $\mu = 0.9$.

研究者提出了称作 Nesterov 动量的动量改进版方法, 可以进一步加速模型的收敛 (Nesterov, 2004; Sutskever et al., 2013). 在传统的带动量项的随机梯度下降算法中, 我们首先计算当前位置的梯度, 然后利用该梯度与前一步的动量项的加权组合来确定当前的更新步长. 而在 Nesterov 方法中, 这两步的顺序被交换. 具体来说, 我们首先基于前一步的动量项进行一次预更新, 然后在这个预更新后的位置计算梯度, 并利用该梯度来确定最终的更新步长.

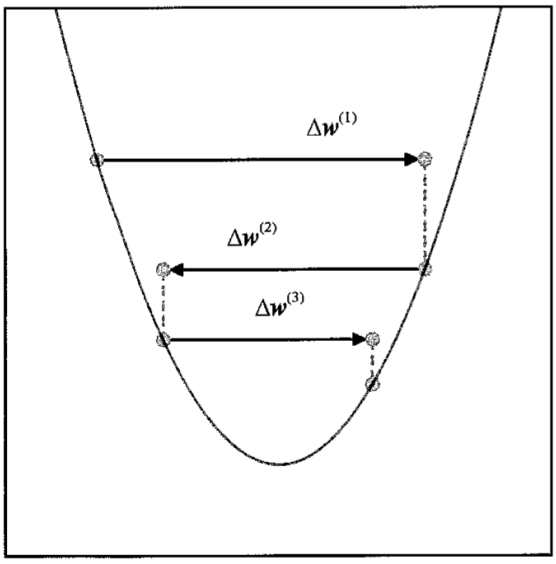


图 7.5: 在梯度下降的连续步骤呈振荡性的情况下, 动量项对学习率 η 的有效值影响不大.

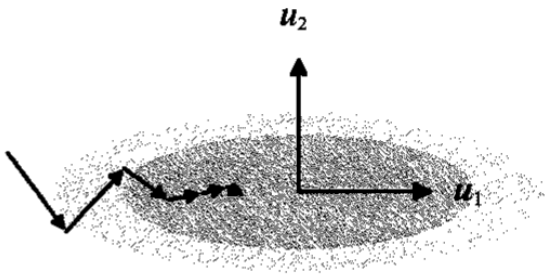


图 7.6: 在梯度下降算法中添加动量项的效果, 与图 7.3中未改进的梯度下降相比, 向误差函数的谷底的收敛更快.

定义 7.13 (Nesterov 动量)

设 μ 为动量参数,η 为学习率, 则称如下更新规则为 **Nesterov 动量** (Nesterov momentum):

$$\Delta \mathbf{w}^{(\tau-1)} = -\eta \nabla E\left(\mathbf{w}^{(\tau-1)} + \mu \Delta \mathbf{w}^{(\tau-2)}\right) + \mu \Delta \mathbf{w}^{(\tau-2)}. \tag{7.36}$$

♣

注

(重构新增) 与(7.33)相比,(7.36)把梯度取在“预更新位置” $\mathbf{w}^{(\tau-1)} + \mu\Delta\mathbf{w}^{(\tau-2)}$ 而不是当前位置, 因此它用未来的梯度修正当前的更新, 具有“前瞻”效果: 当动量项把参数带向一个即将上升的方向时, 预更新位置上的梯度会提前给出反向的修正, 从而抑制过冲. 条件方面, 预更新位置必须仍落在二次近似有效的邻域内, 否则前瞻失去意义;μ 同样需满足 $0 \leqslant \mu < 1$.$|\mu\delta|$ 较小时,$\nabla E(\mathbf{w} + \mu\boldsymbol{\delta}) \approx \nabla E(\mathbf{w}) + \mu\mathbf{H}\boldsymbol{\delta}$, 代回(7.36)可见它与(7.33)的差别是 $O(\mu^2)$ 的高阶项, 这解释了为什么在小学习率、小动量下二者近似等价 (见练习 7.11), 而这种差别在高曲率方向上恰恰提供了额外的阻尼.

对于批量梯度下降算法,Nesterov 动量通常能够提高收敛速度, 然而在随机梯度下降的场景下, 其加速效果可能并不明显.

算法 7.3　带有动量的随机梯度下降

Input: 由 $n \in \{1, \cdots, N\}$ 索引的数据点构成的训练集
　　批量规模 B
　　每个小批量的误差函数 $E_{n:n+B-1}(\mathbf{w})$
　　学习率参数 η
　　动量参数 μ
　　初始权重向量 $\mathbf{w}$
Output: 最终的权重向量 $\mathbf{w}$
$n \leftarrow 1$
$\Delta\mathbf{w} \leftarrow 0$
repeat
　　$\Delta\mathbf{w} \leftarrow -\eta\nabla E_{n:n+B-1}(\mathbf{w}) + \mu\Delta\mathbf{w}$　// 计算将要更新的动量项
　　$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w}$　// 更新权重向量
　　$n \leftarrow n + B$
　　if $n > N$ **then**
　　　　打乱数据
　　　　$n \leftarrow 1$
　　end if
until 收敛
return $\mathbf{w}$

7.3.2 学习率调度

在随机梯度下降学习算法(7.19)中, 我们需要确定学习率 η 的值. 如果 η 值过小, 那么学习过程将异常缓慢 (参见节 7.3中的收敛性分析); 如果 η 值设置过大, 则可能导致算法不稳定. 尽管一定程度的振荡是可以接受的, 但这种振荡不应导致发散现象.

在实际应用中, 我们通常在训练初期使用较大的 η 值, 然后随着迭代的进行逐渐减小学习率, 即让学习率成为迭代步数 τ 的函数.

定义 7.14 (学习率调度)

若把学习率取为迭代步数 τ 的函数 $\eta^{(\tau)}$, 并按

$$\mathbf{w}^{(\tau)} = \mathbf{w}^{(\tau-1)} - \eta^{(\tau-1)} \nabla E_n\left(\mathbf{w}^{(\tau-1)}\right) \tag{7.37}$$

更新权重, 则称这种安排为**学习率调度** (learning rate schedule).

♣

注

(重构新增) 学习率调度针对的是定理 7.2所刻画的矛盾: 训练初期参数离极小值较远, 误差曲面在大尺度上较为平滑, 较大的 η 有助于快速下降; 接近极小值时局部曲率变大,(7.30)要求 η 变小, 否则迭代会在极小值附近持续振荡而无法进一步降低误差. 把 η 写成 τ 的递减函数, 相当于让算法随时间自动“退火”. 条件方面, 调度必须保证 $\eta^{(\tau)}$ 始终为正且最终趋于零; 若衰减过快, 参数会在到达极小值之前就“冻结”, 若衰减过慢则无法消除随机梯度带来的噪声. 需要强调, 调度只改变步长而不改变梯度方向, 因此它不能克服条件数造成的病态, 只能缓解收敛后期的振荡.

学习率衰减的例子包括线性衰减、幂律衰减和指数衰减, 公式依次如下:

$$\eta^{(\tau)} = \left(1 - \frac{\tau}{K}\right)\eta^{(0)} + \frac{\tau}{K}\eta^{(K)}, \qquad \tau \leqslant K, \tag{7.38}$$

$$\eta^{(\tau)} = \eta^{(0)}\left(1 + \frac{\tau}{s}\right)^{c}, \tag{7.39}$$

$$\eta^{(\tau)} = \eta^{(0)} c^{\tau/s}. \tag{7.40}$$

注

(重构新增) 三式的适用场景不同. 线性衰减(7.38)在 K 步内把学习率从 $\eta^{(0)}$ 线性降到 $\eta^{(K)}$ 并在此后保持不变, 参数少、行为可预测, 适合训练轮次固定的场景. 幂律衰减(7.39)与指数衰减(7.40)都是无终点的渐近衰减: 前者按多项式速率下降, 后者按几何速率下降. 要使三者确实起到“衰减”作用, 参数的取值范围必须相应受限:(7.38)要求 $\eta^{(K)} \leqslant \eta^{(0)}$ 且 $\tau \leqslant K$;(7.40)要求 $0 < c < 1$; 而(7.39)作为衰减策略应取负指数, 即 $c < 0$(等价地写成 $(1+\tau/s)^{-c}$、$c > 0$). 原书此处把指数印作正的 c, 若照字面取 $c > 0$, 学习率将随迭代增大而非减小, 与“衰减”的名称相反; 这里忠实保留原印刷形式, 读者在实际使用时必须按上述符号约定取值.

在(7.38)中,η 的值在 K 步内线性减小, 之后保持为 $\eta^{(K)}$. 超参数 $\eta^{(0)}$、$\eta^{(K)}$、K、s 和 c 的最优值通常需要借助经验才能找到. 在实践中, 监督学习曲线 (learning curve), 即误差函数随梯度下降迭代过程的变化情况, 对调整学习率衰减策略很有好处, 可以确保误差函数以恰当的速率减小.

7.3.3 AdaGrad、RMSProp 与 Adam 算法

前文阐述了最优学习率取决于误差曲面在局部的曲率, 并且这种曲率会随着参数空间中方向的不同而发生变化. 这就启发了一系列针对网络中各个参数采用不同学习率的优化算法. 这些学习率的值在训练过程中会自动进行调整. 我们在本小节介绍几个应用最为广泛的算法. 然而, 需要注意的是, 这种直观理解实际上仅在主曲率方向与参数空间坐标轴对齐的情况下才真正适用, 这对应于黑塞矩阵局部为对角矩阵的情形, 尽管在实际应用中这种情况不太可能发生, 但这几个算法仍然被广泛使用并发挥作用.

AdaGrad 是“自适应梯度” (adaptive gradient) 的缩写, 其算法核心思想是随着时间的推移不断减小每个参数的学习率, 而减小的幅度则取决于该参数对应的所有历史梯度值的平方和 (Duchi, Hazan, and Singer, 2011). 因此, 与较大曲率相关联的参数, 其对应的学习率会下降得更快.

定义 7.15 (AdaGrad)

设 η 为学习率,δ 为稳定化常数,$E(\mathbf{w})$ 为当前小批量的误差函数, 并记 $r_i^{(0)} = 0$.**AdaGrad** 算法的更新规则为

$$r_i^{(\tau)} = r_i^{(\tau-1)} + \Big(\frac{\partial E(\mathbf{w})}{\partial w_i}\Big)^2, \tag{7.41}$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)} + \delta}} \frac{\partial E(\mathbf{w})}{\partial w_i}. \tag{7.42}$$

♣

注

(重构新增) AdaGrad 用累积平方梯度 r_i 为每个参数单独缩放步长: 历史梯度一直很大的参数 (对应高曲率方向)r_i 增长快, 其有效学习率 $\eta/\sqrt{r_i+\delta}$ 迅速变小; 历史梯度一直很小的参数则保持较大的步长. 这相当于用 $\sqrt{r_i}$ 近似该方向曲率的量级, 从而在没有显式计算黑塞矩阵的情况下实现了“按曲率缩放步长”的效果, 缓解了条件数造成的困难. 常数 δ 的作用是避免 $r_i \to 0$ 时除以零, 原书取 $\delta = 10^{-8}$ 量级. 定义中“$E(\mathbf{w})$ 是特定小批量的误差函数”这一条很重要:r_i 累积的是随机梯度的平方, 因此它具有随批量规模变化的噪声, 批量越小噪声越大.AdaGrad 的固有缺陷也来自这里:r_i 单调不减, 故有效学习率单调下降, 训练后期步长会变得过小, 即使尚未到达极小值也难以继续前进.

其中 η 是学习率,δ 是一个小的常数 (比如 10^{-8}), 旨在确保 r_i 接近零时的数值稳定性. 该算法的初始值是 $r_i^{(0)} = 0$.$E(\mathbf{w})$ 是特定小批量的误差函数,(7.42)是标准的随机梯度下降公式, 但学习率针对每个参数进行了调整.AdaGrad 算法的缺点是, 它从训练一开始就累积平方梯度, 因此相关的权重更新可能变得非常小, 这可能会在后期大大减缓训练进度.

RMSProp 算法背后的思想是均方根传播 (root mean square propagation), 使用梯度平方的指数加权平均来替代 AdaGrad 中的简单累加 (Hinton, 2012).

定义 7.16 (RMSProp)

设 η 为学习率,δ 为稳定化常数,$0 < \beta < 1$ 为衰减率.**RMSProp** 算法的更新规则为

$$r_i^{(\tau)} = \beta r_i^{(\tau-1)} + (1-\beta)\Big(\frac{\partial E(\mathbf{w})}{\partial w_i}\Big)^2, \tag{7.43}$$

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \frac{\eta}{\sqrt{r_i^{(\tau)} + \delta}} \frac{\partial E(\mathbf{w})}{\partial w_i}. \tag{7.44}$$

♣

注

(重构新增) RMSProp 把 AdaGrad 的累加换成指数加权平均, 从而只“记住”最近约 $1/(1-\beta)$ 步的梯度平方: 旧信息按 β 的幂次衰减,r_i 不再单调增长, 有效学习率也就不会无限减小, 这正好修补了 AdaGrad 后期停滞的缺陷. 参数 β 控制记忆长度, 原书给出的典型值为 $\beta = 0.9$, 对应约 10 步的有效窗口;β 越接近 1, 估计越平滑但对曲率变化的响应越慢. 条件 $0 < \beta < 1$ 保证权重 $(1-\beta)\beta^k$ 构成一个总和为 1 的几何分布, 从而 r_i 是梯度平方的加权平均而非无界累积. 与 AdaGrad 相同,δ 只起数值稳定作用, 不影响算法的渐近行为.

其中 $0 < \beta < 1$, 典型值为 $\beta = 0.9$.

Adam 优化算法 (Kingma and Ba, 2014) 的名字来源于“自适应矩估计” (adaptive moments). 它是 RMSProp 算法与动量法的结合, 为每个参数分别维护一个动量项.

定义 7.17 (Adam)

设 η 为学习率,δ 为稳定化常数,$0 < \beta_1, \beta_2 < 1$ 为衰减率, 并记 $s_i^{(0)} = r_i^{(0)} = 0$.**Adam** 算法对梯度及其平方分别作指数加权移动平均:

$$s_i^{(\tau)} = \beta_1 s_i^{(\tau-1)} + (1-\beta_1)\frac{\partial E(\mathbf{w})}{\partial w_i}, \tag{7.45}$$

$$r_i^{(\tau)} = \beta_2 r_i^{(\tau-1)} + (1-\beta_2)\Big(\frac{\partial E(\mathbf{w})}{\partial w_i}\Big)^2, \tag{7.46}$$

并作偏差修正

$$\hat{s}_i^{(\tau)} = \frac{s_i^{(\tau)}}{1-\beta_1^{\tau}}, \qquad \hat{r}_i^{(\tau)} = \frac{r_i^{(\tau)}}{1-\beta_2^{\tau}}, \tag{7.47}$$

然后按

$$w_i^{(\tau)} = w_i^{(\tau-1)} - \eta \frac{\hat{s}_i^{(\tau)}}{\sqrt{\hat{r}_i^{(\tau)}} + \delta} \tag{7.48}$$

更新权重. 权重参数的典型值为 $\beta_1 = 0.9$、$\beta_2 = 0.99$.

♣

注

(重构新增) Adam 可以看作“带一阶动量的 RMSProp”:s_i 提供梯度的平滑方向 (对应动量),r_i 提供逐参数的尺度 (对应 RMSProp 的曲率估计), 二者相除使更新量对梯度的整体缩放不敏感. 偏差修正(7.47)针对的是零初值: 若 $s_i^{(0)} = 0$, 则 $s_i^{(\tau)} = (1-\beta_1)\sum_{k=1}^{\tau}\beta_1^{\tau-k} g_i^{(k)}$, 其期望是真实梯度的 $(1-\beta_1^{\tau})$ 倍, 因此除以 $1-\beta_1^{\tau}$ 才能得到无偏估计 (见练习 7.12). 由于 $\beta_1^{\tau} \to 0$, 这个因子在若干步之后趋于 1, 所以实践中有时省略修正, 但在训练最初几十步内不修正会使步长系统性偏小. 需要再次强调本节开头的限制:AdaGrad、RMSProp 与 Adam 的“按曲率缩放”直观只在黑塞矩阵近似对角、主曲率方向与坐标轴对齐时严格成立; 一般情形下它们仍是启发式方法, 其有效性来自逐参数自适应尺度对病态条件的缓解, 而非精确的曲率估计.

这里的系数 $1/(1-\beta_1^{\tau})$ 和 $1/(1-\beta_2^{\tau})$ 纠正了初始设置 $s_i^{(0)}$ 和 $r_i^{(0)}$ 为零引入的偏差 (见练习 7.12). 请注意, 由于 $\beta_i < 1$ 且随着 τ 变大, 偏差趋于零, 因此我们在实践中有时会省略这种偏差修正. 权重参数的典型值为 $\beta_1 = 0.9$ 和 $\beta_2 = 0.99$.Adam 算法是深度学习应用中最常用的学习算法.

算法 7.4　Adam 算法

Input: 由 $n \in \{1, \cdots, N\}$ 索引的数据点构成的训练集
　　批量规模 B
　　每个小批量的误差函数 $E_{n:n+B-1}(\mathbf{w})$
　　学习率参数 η
　　衰减参数 β_1 与 β_2
　　稳定化参数 δ
Output: 最终的权重向量 $\mathbf{w}$
$n \leftarrow 1$
$\mathbf{s} \leftarrow \mathbf{0}$
$\mathbf{r} \leftarrow \mathbf{0}$
repeat
　　从 $\mathcal{D}$ 中随机选取一个小批量
　　$\mathbf{g} \leftarrow -\nabla E_{n:n+B-1}(\mathbf{w})$　// 计算梯度向量
　　$\mathbf{s} \leftarrow \beta_1 \mathbf{s} + (1-\beta_1)\mathbf{g}$
　　$\mathbf{r} \leftarrow \beta_2 \mathbf{r} + (1-\beta_2)\mathbf{g} \odot \mathbf{g}$　// 逐元素相乘
　　$\hat{\mathbf{s}} \leftarrow \mathbf{s}/(1-\beta_1^\tau)$　// 偏差校正
　　$\hat{\mathbf{r}} \leftarrow \mathbf{r}/(1-\beta_2^\tau)$　// 偏差校正
　　$\Delta\mathbf{w} \leftarrow -\eta\,\hat{\mathbf{s}}/(\sqrt{\hat{\mathbf{r}}} + \delta)$　// 逐元素操作
　　$\mathbf{w} \leftarrow \mathbf{w} + \Delta\mathbf{w}$　// 更新权重向量
　　$n \leftarrow n + B$
　　if $n + B > N$ **then**
　　　　打乱数据
　　　　$n \leftarrow 1$
　　end if
until 收敛
return w

7.4 正则化

在神经网络的前向传播过程中, 对计算得到的变量进行归一化处理, 可以避免网络处理过大或过小的值. 尽管原则上神经网络的权重和偏置可以适应各种不同的输入和隐藏变量的值, 但在实践中, 归一化对于确保有效的训练至关重要. 根据归一化操作所针对的对象不同, 我们将归一化技术分为三种类型: 数据归一化、批量归一化和层归一化.

注

(重构新增) 本节标题在原书中印作“正则化”, 但全节讨论的对象是**归一化** (normalization): 重标度输入变量、隐藏单元预激活以及单个样本内部隐藏单元取值的尺度. 中文里“规范化”“归一化”“正则化”常被混用, 而这三者在本书中含义不同: 归一化指对变量作平移与伸缩的预处理, 正则化 (regularization) 指在误差函数中加入惩罚项或采取早停等手段以控制模型复杂性 (见正则化一章). 正文保留原书标题以便与目录对应, 读者应按下文实际内容理解其含义.

7.4.1 数据归一化

有时我们会遇到这样的数据集, 其中不同的输入变量的取值范围差异很大. 例如, 在医疗健康数据中, 患者的身高可能以米为单位进行测量, 如 1.8 米, 而他们的血小板计数可能以每微升血液中的血小板数量来衡量, 例如每微升 300 000 个血小板. 这种差异会让梯度下降训练过程更困难. 考虑一个具有两个权重的单层回归网络, 其两个对应的输入变量具有截然不同的取值范围. 其中一个权重的微小变化, 相比于另一个权重的类似变化, 会对输出乃至误差函数产生大得多的影响. 这会产生一个沿着不同坐标轴曲率差异极大的误差曲面, 如图 7.3所示.

因此, 对于连续型输入变量, 将输入值重新缩放使其具有相似的取值范围通常是非常有益的. 这很容易做到: 首先分别计算每个输入变量的均值和方差,

$$\mu_i = \frac{1}{N}\sum_{n=1}^{N} x_{ni}, \tag{7.49}$$

$$\sigma_i^2 = \frac{1}{N}\sum_{n=1}^{N}(x_{ni} - \mu_i)^2, \tag{7.50}$$

这是在开始任何训练之前都需要进行的计算. 然后使用以下方法缩放输入值:

$$\tilde{x}_{ni} = \frac{x_{ni} - \mu_i}{\sigma_i}, \tag{7.51}$$

这样, 缩放的值 $\tilde{x}_{ni}$ 具有零均值和单位方差 (见练习 7.14).

定义 7.18 (数据归一化)

对第 i 个连续型输入变量, 按(7.49)与(7.50)在训练集上计算其均值 μ_i 与标准差 σ_i, 再按(7.51)把每个输入值变换为 $\tilde{x}_{ni}$, 这种预处理称为**数据归一化** (data normalization). ♣

注

(重构新增) 数据归一化的动机可以从误差曲面的几何直接读出: 若两个输入变量的尺度相差 10^3 倍, 则误差函数对相应权重的敏感度也相差 10^3 倍, 黑塞矩阵的两个特征值随之相差 10^6 倍, 条件数急剧增大, 梯度下降沿小曲率方向前进极慢 (见定义 7.11). 把每个输入缩放到单位方差, 相当于在输入坐标上作一次对角伸缩, 使各方向曲率量级接近, 从而改善条件数. 条件方面:$\sigma_i > 0$ 必须成立, 若某个输入变量在训练集上为常数 ($\sigma_i = 0$), 变换无定义, 实践中应删除该变量或给它一个非零的默认尺度. 另一个容易忽视的要点是,μ_i 与 σ_i 只能由训练集计算, 并原样用于开发集、验证集与测试集; 若对每个数据集各自标准化, 就相当于让测试数据“偷看”自己的统计量, 破坏了训练与测试条件一致的前提.

请注意, 必须使用相同的 μ_i 值和 σ_i 值来预处理任何开发集、验证集或测试集数据, 以确保对所有输入都以相同方式进行缩放.图 7.7对数据归一化进行了说明.

命题 7.7 (标准化变量的矩)

设 $x_n (n = 1, \cdots, N)$ 为某个输入变量的 N 个观测值,μ 与 σ^2 由(7.49)与(7.50)定义且 $\sigma > 0$, 并令 $\tilde{x}_n = (x_n - \mu)/\sigma$. 则

$$\frac{1}{N}\sum_{n=1}^{N}\tilde{x}_n = 0, \qquad \frac{1}{N}\sum_{n=1}^{N}\tilde{x}_n^2 = 1. \tag{7.52}$$

♠

证明

(重构新增) 由定义及求和的线性性,

$$\frac{1}{N}\sum_{n=1}^{N}\tilde{x}_n = \frac{1}{N}\sum_{n=1}^{N}\frac{x_n-\mu}{\sigma} = \frac{1}{\sigma}\Big(\frac{1}{N}\sum_{n=1}^{N}x_n-\mu\Big) = \frac{1}{\sigma}(\mu-\mu) = 0.$$

同理,

$$\frac{1}{N}\sum_{n=1}^{N}\tilde{x}_n^2 = \frac{1}{\sigma^2}\cdot\frac{1}{N}\sum_{n=1}^{N}(x_n-\mu)^2 = \frac{\sigma^2}{\sigma^2} = 1.$$

□

注

(重构新增) 命题的结论是“样本矩”意义下的: 所用的均值与方差都是对同一批 N 个观测值按 $1/N$ 平均得到的, 而不是总体矩, 因此不需要任何概率假设. 这一点与练习 7.8不同, 那里讨论的是样本均值作为总体均值估计量的期望误差. 条件 $\sigma > 0$ 是本质的: 若 $\sigma = 0$, 所有 x_n 相同, 变换把全部点映射为 0/0, 归一化失去意义. 把(7.52)与(7.51)合起来看, 归一化后的数据落在以原点为中心、各方向尺度相当的区域内, 这既改善了误差曲面的条件数, 也使后续的权重初始化尺度 (命题 7.5) 具有统一含义.

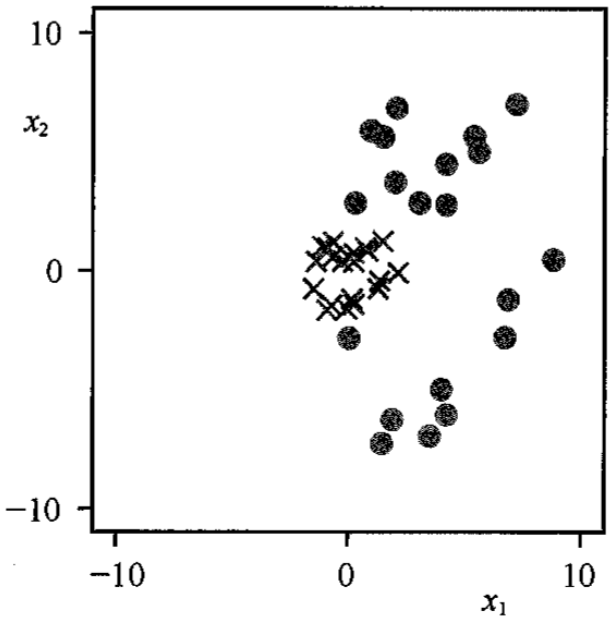


图 7.7: 对输入数据进行归一化. 红色圆圈表示一个具有两个变量的数据集的原始数据点. 蓝色叉号显示了这个数据集经过归一化处理后的情况: 每个变量在数据集中都具有零均值和单位方差.

7.4.2 批量归一化

我们已经了解了数据归一化的重要性, 类似的思路也可以应用于深度网络中每一隐藏层的变量. 如果某个特定隐藏层的激活值范围存在很大差异, 那么将这些值归一化为零均值和单位方差应该可以简化后续网络层的学习问题. 然而, 与数据归一化可以在训练开始前一次性完成不同, 隐藏单元值的归一化需要在训练过程中每次权重更新后都重复进行. 这种方法称为**批量归一化** (batch normalization)(Ioffe and Szegedy, 2015).

引入批量归一化也是为了解决**梯度消失**和**梯度爆炸**问题, 它们通常出现在训练非常深的神经网络的时候. 根据微积分中的链式法则, 误差函数 E 相对于网络第一层中参数的梯度由下式给出:

$$\frac{\partial E}{\partial w_i} = \sum_m \cdots \sum_l \sum_j \frac{\partial z_m^{(l)}}{\partial w_i} \cdots \frac{\partial z_j^{(K)}}{\partial z_l^{(K-1)}} \frac{\partial E}{\partial z_j^{(K)}}, \tag{7.53}$$

其中 $z_j^{(K)}$ 表示第 K 层中节点 j 的激活值,(7.53)右侧的每个偏导数 (参见小节 8.1.5) 代表那一层的 Jacobi 矩阵的元素. 如果这些偏导数项中大多数的绝对值小于 1, 那么大量此类项的乘积将趋于 0; 反之, 如果大多数项的绝对值大于 1, 乘积将趋于无穷大. 因此, 随着网络深度的增加, 误差函数的梯度可能会变得非常大或非常小. 批量归一化方法在很大程度上解决了这个问题.

为了明确批量归一化的具体定义, 我们考虑多层网络中的某个特定层. 该层中的每个隐藏单元都对其输入的预激活值 $z_j = h(a_j)$ 计算一个非线性函数, 因此我们可以选择对预激活值 a_j 或激活值 z_j 进行归一化. 在实践中, 二者均可, 这里我们以归一化预激活值为例来说明这个过程. 由于权重值在每个小批量之后都会更新, 因此我们需要对每个小批量分别归一化. 具体来说, 对于一个大小为 K 的小批量, 定义

$$\mu_i = \frac{1}{K}\sum_{n=1}^{K} a_{ni}, \tag{7.54}$$

$$\sigma_i^2 = \frac{1}{K}\sum_{n=1}^{K}(a_{ni} - \mu_i)^2, \tag{7.55}$$

$$\hat{a}_{ni} = \frac{a_{ni} - \mu_i}{\sqrt{\sigma_i^2 + \delta}}, \tag{7.56}$$

其中, 对 $n = 1, \cdots, K$ 的求和操作是针对小批量中的元素进行的.δ 是一个很小的常数, 引入它是为了避免当 σ_i^2 很小时出现因为除数趋于零而导致的各种数值问题.

通过对网络中某一层的预激活值进行归一化, 我们减少了该层参数的自由度, 从而降低了它的表达能力. 我们可以通过将预激活值重新缩放为具有平均值 β_i 和标准差 γ_i 来补偿这一点:

$$\tilde{a}_{ni} = \gamma_i \hat{a}_{ni} + \beta_i, \tag{7.57}$$

其中 β_i 和 γ_i 是可学习的自适应参数, 它们与网络的权重和偏置一起通过梯度下降法进行联合学习.

定义 7.19 (批量归一化)

设某一层第 i 个隐藏单元在一个大小为 K 的小批量上的预激活为 $a_{1i}, \cdots, a_{Ki}$. 按(7.54)与(7.55)计算小批量均值与小批量方差, 按(7.56)得到标准化预激活 $\hat{a}_{ni}$, 再按(7.57)用可学习参数 γ_i、β_i 重新缩放, 这一整组变换称为**批量归一化** (batch normalization). ♣

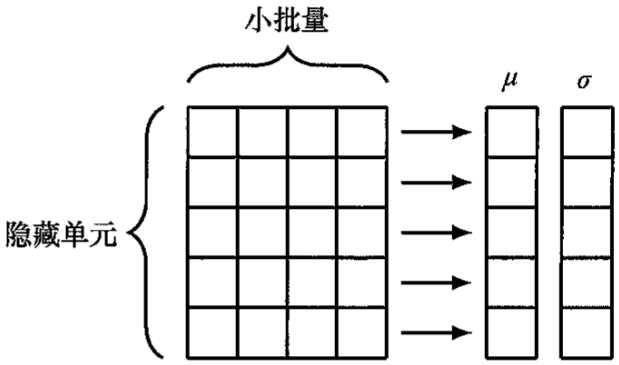


图 7.8: 批量归一化的示意图. 在批量归一化中, 均值和方差是分别针对每个隐藏单元跨小批量计算的.

注

(重构新增) 批量归一化把“归一化”从输入层推广到每个隐藏层, 其关键创新不是标准化本身, 而是两点. 其一, 统计量按小批量计算, 因此每步迭代都在更新, 不需要遍历整个数据集. 其二, 标准化之后立即用可学习参数 γ_i, β_i 重新缩放, 使该层仍能表示任意均值与尺度的分布——若不加这一步, 归一化会强行把预激活限制在单位尺度上, 削减该层的表达能力. 条件方面:K 不能太小, 否则 μ_i、σ_i^2 的估计噪声很大 (见小节 7.4.3);$\delta > 0$ 保证 $\sigma_i^2 = 0$(该单元在小批量上取常值) 时除数不为零;γ_i、β_i 与网络权重一同训练, 因此批量归一化层本身也是可微的, 可以像普通层一样反向传播. 需要注意, 训练阶段用当前小批量的统计量, 推理阶段则用训练期间累积的移动平均值 ((7.58)、(7.59)), 二者必须区分, 否则同一模型在训练与测试时的行为不一致.

式(7.57)可能看起来只是抵消了批量归一化的效果, 因为均值和方差现在又可以适应任意值了. 然而, 关键的区别在于参数在训练过程中的演变方式. 对于原始网络, 一个小批量的均值和方差由该层所有权重和偏置的复杂函数决定; 而在(7.57)中, 它们直接由独立的参数 β_i 和 γ_i 决定, 这些参数在梯度下降过程中更容易学习. 式(7.56)至(7.57)描述了一个相对于可学习参数 β_i 和 γ_i 可微的变量的转换. 这可以视为神经网络中的附加层, 因此每个标准隐藏层的后面都可以加一个批量归一化层, 参见图 7.8.

当我们完成网络训练, 想要对新数据进行预测时, 我们并没有训练时的小批量数据可用, 也不能仅从一个数据样本确定均值和方差. 为了解决这个问题, 原则上可以在对权重和偏置进行最后的更新之后, 针对每个层在整个训练集上计算 μ_i 和 σ_i^2. 然而, 为了计算这些统计量, 就需要对整个数据集再进行处理, 计算成本太高了. 于是, 我们选择在整个训练阶段计算移动平均值:

$$\bar{\mu}_i^{(\tau)} = \alpha\, \bar{\mu}_i^{(\tau-1)} + (1-\alpha)\mu_i, \tag{7.58}$$

$$\bar{\sigma}_i^{(\tau)} = \alpha\, \bar{\sigma}_i^{(\tau-1)} + (1-\alpha)\sigma_i, \tag{7.59}$$

其中 $0 \leqslant \alpha \leqslant 1$. 这些移动平均值在训练期间不起作用, 仅用于在推理阶段处理新的数据点.

尽管批量归一化在实践中非常有效, 但其内在原理目前还不明确. 批量归一化提出的原因是, 人们观察到, 网络中较早层权重的更新会改变后续层所看到的数值分布, 这种现象称为**内部协变量偏移** (internal covariate shift). 然而, 后来的研究 (Santurkar et al., 2018) 表明, 协变量偏移并不是一个关键因素, 批量归一化所带来的训练改进, 实际上在于它平滑了误差函数的崎岖程度.[1]

定义 7.20 (内部协变量偏移)

在深层网络训练过程中, 较早层权重的更新使后续层所接收到的输入数值分布不断发生变化, 这种现象称为**内部协变量偏移** (internal covariate shift). ♣

[1] 如果误差函数的图形看上去很崎岖, 即存在很多尖锐的高峰和深谷, 梯度下降算法就很可能会陷入局部最低点, 而无法找到真正的全局最低点. ——译者注

注

(重构新增) 这个概念描述的是一个真实存在的现象: 第 l 层的输入是第 $l-1$ 层的输出, 而第 $l-1$ 层的参数在训练中不断更新, 因此第 l 层面对的输入分布 (均值、方差、高阶矩) 随时间漂移, 它必须一边适应任务一边适应不断移动的输入分布. 原书随即指出, 把批量归一化的效果归因于消除协变量偏移并不成立 (Santurkar et al., 2018): 实验表明, 即使人为注入协变量偏移, 批量归一化仍然有效; 更可信的解释是它使误差曲面更平滑、Lipschitz 常数更小, 从而允许更大的学习率. 这段讨论的价值在于提醒读者, 一个方法的经验成功与其最初提出的理论解释可能是两回事; 条件“较早层参数在更新”是现象成立的前提, 若网络各层独立训练 (例如逐层预训练后冻结), 则不存在这种漂移.

7.4.3 层归一化

在批量归一化中, 如果批量规模太小, 那么均值和方差的估计会包含太多噪声. 另外, 对于非常大的训练集, 小批量可能会被分散在不同的 GPU 上处理, 使得跨小批量的全局归一化效率低下. 一种替代方案是分别针对每个数据点对隐藏单元的值进行归一化, 而不是在小批量中的样本间进行归一化. 这称为**层归一化** (layer normalization)(Ba, Kiros, and Hinton, 2016)(参见小节 12.2.5). 层归一化最初是在递归神经网络的背景下提出的, 因为递归神经网络中数值的分布在每个时间步后都会发生变化, 这使得批量归一化难以实施 (参见节 12.2). 另外, 层归一化在其他网络架构中也是有效的, 例如 Transformer 网络. 类似于批量归一化, 我们进行如下变换:

$$\mu_n = \frac{1}{M}\sum_{i=1}^{M} a_{ni}, \tag{7.60}$$

$$\sigma_n^2 = \frac{1}{M}\sum_{i=1}^{M}(a_{ni}-\mu_n)^2, \tag{7.61}$$

$$\hat{a}_{ni} = \frac{a_{ni}-\mu_n}{\sqrt{\sigma_n^2+\delta}}, \tag{7.62}$$

其中, 对 $i = 1, \cdots, M$ 的求和是针对该层中的所有隐藏单元进行的.

定义 7.21 (层归一化)

对第 n 个数据点, 设该层 M 个隐藏单元的预激活为 $a_{n1}, \cdots, a_{nM}$. 按(7.60)与(7.61)在该数据点内部跨隐藏单元计算均值与方差, 按(7.62)得到标准化预激活, 并如同(7.57)那样用每个隐藏单元各自的可学习参数 γ_i、β_i 重新缩放, 这组变换称为**层归一化** (layer normalization). ♣

注

(重构新增) 层归一化与批量归一化的区别只在“沿哪个方向求统计量”: 批量归一化对固定的隐藏单元跨小批量求均值与方差 ((7.54)、(7.55)), 层归一化对固定的数据点跨隐藏单元求均值与方差 ((7.60)、(7.61)). 这一转置带来三个直接后果. 其一, 层归一化不依赖批量规模,$K = 1$ 时仍可正常工作, 因此适合批量很小的场景. 其二, 它不需要跨设备同步统计量, 适合模型并行. 其三, 训练与推理使用完全相同的归一化函数, 无须存储移动平均值, 这消除了(7.58)、(7.59)在训练与推理之间的不一致. 代价是: 它假设同一层内不同隐藏单元的尺度应当可比, 若各单元的量纲本就不同 (例如最后一层的多个输出对应不同物理量), 层归一化可能不合适. 条件 $\delta > 0$ 与批量归一化相同, 用于处理 $\sigma_n^2 = 0$ 的退化情形.

与批量归一化一样, 也需要为每个隐藏单元分别引入额外的可学习的均值参数和标准差参数, 形式如(7.57)所示. 值得注意的是, 在训练阶段和推理阶段可以使用相同的归一化函数, 因此我们无须存储移动平均值.图 7.8和图 7.9对批量归一化和层归一化进行了比较.

7.5 习题

✍ **练习 7.1** (★) 通过将(7.10)代入(7.7), 并使用(7.8)和(7.9), 证明误差函数(7.7)可以写成(7.11)的形式.

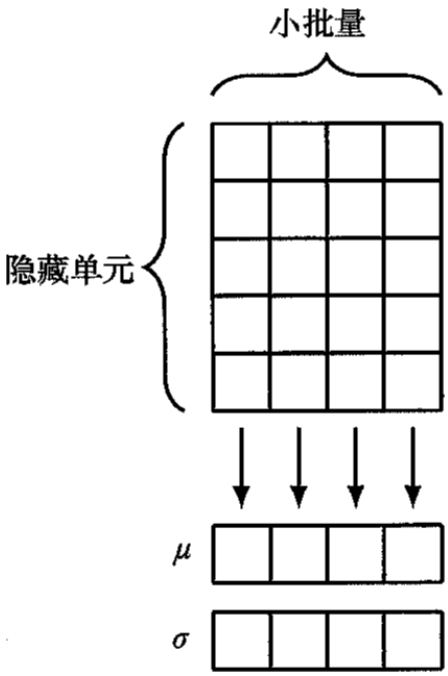


图 7.9: 层归一化的示意图. 在层归一化中, 均值和方差是跨隐藏单元分别针对每个数据点计算的.

✍ **练习 7.2** (★) 考虑具有特征向量方程(7.8)的黑塞矩阵 $\mathbf{H}$. 通过将(7.14)中的向量 $\mathbf{v}$ 依次设置为每个特征向量 $\mathbf{u}_i$, 证明黑塞矩阵 $\mathbf{H}$ 是正定的——当且仅当它的所有特征值都是正的.

✍ **练习 7.3** (★★) 通过考虑关于驻点 $\mathbf{w}^*$ 的误差函数的局部 Taylor 展开(7.7), 证明该驻点是误差函数的局部极小值的充分必要条件是, 由(7.5)定义的黑塞矩阵 $\mathbf{H}$ 在 $\hat{\mathbf{w}} = \mathbf{w}^*$ 的情况下是正定的.

✍ **练习 7.4** (★★) 考虑一个具有单一输入变量 x 和单一输出变量 y 的线性回归模型:

$$y(x, w, b) = wx + b, \tag{7.63}$$

平方和误差函数为

$$E(w, b) = \frac{1}{2}\sum_{n=1}^{N}\{y(x_n, w, b) - t_n\}^2. \tag{7.64}$$

推导出由误差函数对权重参数 w 和偏置参数 b 的二阶导数给出的 2×2 黑塞矩阵的元素表达式. 证明这个黑塞矩阵的迹和行列式都是正的. 由于矩阵的迹代表特征值之和, 而行列式对应特征值的乘积 (参见节 A.2), 两个特征值都是正的, 因此误差函数在驻点达到最小值.

✍ **练习 7.5** (★★) 考虑一个带有单一输入变量 x 和单一输出变量 y 的单层分类模型:

$$y(x, w, b) = \sigma(wx + b), \tag{7.65}$$

其中 $\sigma(\cdot)$ 是定义 5.16中定义的逻辑斯谛 sigmoid 函数. 交叉熵误差函数为

$$E(w, b) = -\sum_{n=1}^{N}\{t_n \ln y(x_n, w, b) + (1 - t_n)\ln\big(1 - y(x_n, w, b)\big)\}. \tag{7.66}$$

推导出由误差函数对权重参数 w 和偏置参数 b 的二阶导数给出的 2×2 黑塞矩阵的元素表达式. 证明这个黑塞矩阵的迹和行列式都是正的. 由于矩阵的迹代表特征值之和, 而行列式对应特征值的乘积 (参见节 A.2), 两个特征值都是正的, 因此误差函数在驻点达到最小值.

✍ **练习 7.6** (★★) 考虑一个由(7.7)定义的二次误差函数, 在该误差函数中, 黑塞矩阵 $\mathbf{H}$ 有一个由(7.8)给出的特征方程. 证明常数误差的等高线呈椭圆形, 其轴与特征向量 $\mathbf{u}_i$ 对齐, 长度则与相应特征值 λ_i 的平方根成反比.

✍ **练习 7.7** (★) 证明由于黑塞矩阵 $\mathbf{H}$ 的对称性, 二次误差函数(7.3)中独立元素的数量由 $W(W + 3)/2$ 给出.

✍ **练习 7.8** (★) 考虑一组值 $x_1, \cdots, x_N$, 它们是从均值为 μ、方差为 σ^2 的分布中抽取的, 并定义样本均值为

$$\bar{x} = \frac{1}{N}\sum_{n=1}^{N} x_n. \tag{7.67}$$

证明关于数据来源分布的平方误差 $(\bar{x} - \mu)^2$ 的期望由 σ^2/N 给出. 这说明样本的均方根误差由 $\sigma/\sqrt{N}$ 给出, 并且会随着样本数量的增加而相对缓慢地减小.

✍ **练习 7.9** (★★) 考虑一个在第 l 层计算函数(7.20)和(7.21)的分层网络. 假设我们使用 Gauss 分布 $\mathcal{N}(0, \varepsilon^2)$ 初始化权重, 并假设第 $(l-1)$ 层单元的输出 $z_j^{(l-1)}$ 具有方差 λ^2. 通过使用 ReLU 激活函数的形式, 证明第 l 层输出的平均值和方差分别由(7.22)和(7.23)给出, 进而证明如果我们想要第 l 层的单元的预激活也有方差 λ^2, 则 ε 的值应该由(7.24)给出.

✍ **练习 7.10** (★★) 通过使用(7.7)、(7.8)和(7.10), 推导出结果(7.25)和(7.26), 这两个结果以黑塞矩阵的特征向量展开形式来表达梯度向量和一般的权重更新. 利用这两个结果, 并结合特征向量的正交性关系(7.9)和梯度下降公式(7.17), 导出以系数 α_i 表示的批量梯度下降更新结果(7.27).

✍ **练习 7.11** (★) 考虑一个平滑变化、曲率较低的误差表面, 梯度随位置缓慢变化. 证明对于较小的学习率和动量参数, 由(7.36)定义的 Nesterov 动量梯度更新等价于由(7.33)定义的标准带动量的梯度下降.

✍ **练习 7.12** (★★) 考虑变量 x 的一系列值 $\{x_1, \cdots, x_N\}$, 并假设我们使用下式计算指数加权移动平均值:

$$\mu_n = \beta\mu_{n-1} + (1-\beta)x_n, \tag{7.68}$$

其中 $0 \leqslant \beta \leqslant 1$. 通过使用有限几何级数求和的以下结果:

$$\sum_{k=1}^{n} \beta^{k-1} = \frac{1-\beta^n}{1-\beta}, \tag{7.69}$$

证明如果使用 $\mu_0 = 0$ 初始化平均序列, 则估计器存在偏差, 并且可以使用下式校正偏差:

$$\hat{\mu}_n = \frac{\mu_n}{1-\beta^n}. \tag{7.70}$$

✍ **练习 7.13** (★) 在梯度下降法中, 权重向量 $\mathbf{w}$ 是通过在权重空间中沿由学习率参数 η 控制的负梯度方向进行更新的. 假设我们选择权重空间中的一个方向 $\mathbf{d}$, 沿着该方向在给定当前权重向量 $\mathbf{w}^{(\tau)}$ 的情况下最小化误差函数

$$E\left(\mathbf{w}^{(\tau)} + \lambda\mathbf{d}\right), \tag{7.71}$$

进行最小化, 以获得新权重向量 $\mathbf{w}^{(\tau+1)}$ 所对应的值 λ^*. 证明在 $\mathbf{w}^{(\tau+1)}$ 处,$E(\mathbf{w})$ 的梯度垂直于向量 $\mathbf{d}$. 这称为“线搜索”方法, 它是各种数值优化算法的基础 (Bishop, 1995b).

✍ **练习 7.14** (★) 证明由(7.51)定义的重新归一化的输入变量, 其中 μ_i 由(7.49)定义,σ_i^2 由(7.50)定义, 具有零均值和单位方差.

第 8 章 反向传播

本章旨在寻找一种高效的技术, 用于评估前馈神经网络误差函数 $E(\mathbf{w})$ 的梯度. 我们将看到, 这可以通过一种本地消息传递方案来实现, 在该方案中, 信息会通过网络向后发送, 称为**误差反向传播** (error backpropagation).

在过去, 反向传播方程需要手工推导, 然后在软件中与前向传播方程一起实现; 这两个步骤都很耗费时间, 且都容易出错. 然而, 在现代神经网络软件环境中, 我们只需要在原有网络函数的编码基础上稍作扩展, 就能高效地计算出任何感兴趣的导数. 这种想法称为**自动微分** (automatic differentiation)(参见节 8.2), 其在现代深度学习中发挥着关键作用. 然而, 理解如何进行计算是有价值的, 这样我们就不必依赖所谓的 “黑盒子” 软件解决方案. 因此, 本章将对反向传播的关键概念进行解释, 并详细探讨自动微分的框架.

请注意, “反向传播” 一词在神经计算文献中有多种不同的含义. 例如, 有的神经计算文献可能会把前馈结构称作反向传播网络. 此外, “反向传播” 一词有时也用来描述神经网络的端到端训练程序, 包括梯度下降参数更新. 本书将专门使用 “反向传播” 来描述用于对导数进行数值计算的计算程序, 例如计算关于网络权重和偏置的误差函数梯度. 这一程序也可用于计算其他重要导数, 如 Jacobi 矩阵和黑塞矩阵.

8.1 梯度计算

下面让我们为具有任意前馈拓扑结构、任意可微分非线性激活函数和一大类误差函数的通用网络推导出反向传播算法. 然后, 我们将使用一个简单的分层网络结构 (具有单层 S 形隐藏单元以及平方和误差) 来说明所得公式.

许多具有实际意义的误差函数 (例如由一组独立同分布数据的最大似然定义的误差函数) 都包含一些项 (其中的每一项对应训练集中的一个数据点) 的总和, 因此

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}). \tag{8.1}$$

在此, 我们将考虑如何为误差函数中这样一项的 $\nabla E_n(\mathbf{w})$ 求值. 这可以直接用于随机梯度下降法, 也可以将结果累积到一组训练数据点上, 用于批量处理或小批量处理方法.

8.1.1 单层网络

考虑一个简单的线性模型, 其中输出 y_k 是输入变量 x_i 的线性组合, 因此

$$y_k = \sum_i w_{ki} x_i. \tag{8.2}$$

对于特定的输入数据点 n, 平方和误差函数的形式为

$$E_n = \frac{1}{2} \sum_k (y_{nk} - t_{nk})^2, \tag{8.3}$$

其中 $y_{nk} = y_k(\mathbf{x}_n, \mathbf{w})$,$t_{nk}$ 是关联的目标值. 该误差函数相对于权重 w_{ji} 的梯度为

$$\frac{\partial E_n}{\partial w_{ji}} = \left(y_{nj} - t_{nj}\right) x_{ni}. \tag{8.4}$$

这可以理解为 “本地” 计算, 涉及与链路输出端 w_{ji} 相关的 “误差信号” $(y_{nj} - t_{nj})$ 和与链路输入端相关的变量 x_{ni} 的乘积. 逻辑斯谛 sigmoid 激活函数与交叉熵误差函数、以及 softmax 激活函数和与之匹配的多元交叉熵误差函数是如何产生类似公式的, 参见小节 5.4.3. 接下来, 我们将探究这一简单结果如何扩展到更复杂的多层前馈网络情境.

8.1.2 一般前馈网络

一般来说, 前馈网络由一组单元组成, 每个单元计算其输入的加权和:

$$a_j = \sum_i w_{ji} z_i, \tag{8.5}$$

其中 z_i 是另一个单元的激活值, 或是向单元 j 发送连接的输入单元, 而 w_{ji} 是与该连接相关的权重. 此外, 可以通过引入一个额外的单元或输入 (激活值固定为 +1) 来将偏置包含在该总和中, 因此我们不需要显式地处理偏置 (参见节 6.2). 在通过非线性激活函数 $h(\cdot)$ 进行转换后,(8.5)中的和 (称为预激活) 给出单元 j 的激活值 z_j, 其形式为

$$z_j = h(a_j). \tag{8.6}$$

请注意,(8.5)中参与求和的一个或多个变量 z_i 可以是一个输入变量. 同样,(8.6)中的单元 j 也可以是一个输出变量.

定义 8.1 (计算图)

把前馈网络中的每个单元画成一个节点, 把每条权重为 w_{ji} 的连接画成从节点 i 指向节点 j 的有向边, 所得的有向无环图称为该网络的**计算图** (computational graph). 在计算图中, 节点 i 的激活值 z_i 沿边的方向送入节点 j 并参与(8.5)的求和, 因此(8.5)与(8.6)就是信息沿边正向流动一次的结果; 反过来, 若把误差函数对某个节点的预激活的导数看作该节点携带的信号, 那么这个信号可以沿边反向送回它的全部前驱节点. ♣

注

(重构新增) 计算图之所以值得单独定义, 是因为它把“网络里谁影响谁”这件事从公式中剥离出来, 只留下节点与有向边. 有了它, 前向传播与反向传播就不是两套彼此独立的公式, 而是同一张图上的两次遍历: 一次顺着边的方向求值, 一次逆着边的方向累积导数. 定义要求图是有向无环图, 这保证两次遍历都能按拓扑序进行、不会出现“节点依赖自己”的循环, 因而每个节点的量都能由它的前驱 (或后继) 先算出来; 若网络中存在反馈连接, 遍历顺序就不再存在, 必须改用迭代求解. 定义还隐含要求每个节点上的激活函数可微, 否则反向遍历时 $\partial z_j/\partial a_j$ 不存在 ((8.6)的导数即 $h'(a_j)$). 求值轨迹 (参见节 8.2) 是计算图的一个特例: 那里的节点是初等运算而不是网络单元, 但“沿边正向求值、反向累积”的结构完全相同, 这正是自动微分能够统一处理前向模式与逆模式的原因.

对于训练集中的每个数据点, 我们假设已经向网络提供了相应的输入向量, 并通过连续应用(8.5)和(8.6)计算出了网络中所有隐藏单元和输出单元的激活值.

定义 8.2 (前向传播)

从输入单元出发, 沿定义 8.1中边的方向依次对每个单元计算(8.5)与(8.6), 从而得到网络中所有隐藏单元和输出单元的激活值, 这一过程叫作**前向传播** (forward propagation), 因为它可以被视为信息通过网络的前向流动. ♣

注

(重构新增) 前向传播是唯一需要真正“跑一遍网络”的步骤: 它把输入 $\mathbf{x}$ 变成输出 $\mathbf{y}$, 同时把所有中间激活值 z_j 留在内存中. 定义中的“沿边的方向”不是修辞: 由于计算图无环, 每个单元的预激活只依赖它前驱的激活值, 因此可以逐个单元地求值, 而不必解联立方程. 需要注意, 前向传播本身并不涉及误差函数, 也不产生任何导数; 它只是为反向传播准备好两样东西——各单元的预激活 a_j(用于计算 $h'(a_j)$) 和各单元的输入 z_i(用于计算权重梯度). 这也解释了为什么反向传播的存储开销与前向传播相当:(8.12)需要 z_i,(8.11)需要 a_j, 二者都必须保留到反向阶段.

下面让我们来计算 E_n 相对于权重 w_{ji} 的导数. 各单元的输出将取决于特定输入数据点 n. 不过, 为了使符号简洁, 我们将省略网络变量的下标 n. 首先要注意的是,E_n 对 w_{ji} 的依赖仅体现在输入 a_j 对单元 j 的求和上.

命题 8.1 (偏导数的链式法则)

设误差函数 E 关于所有权重可微, 并且 E 对权重的依赖全部通过有限个中间变量 $a_1, \cdots, a_J$(即各单元的

预激活) 传递. 若权重 w_{ji} 只出现在(8.5)中 a_j 的表达式里, 而对其他 $a_l (l \neq j)$ 没有影响, 则

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial a_j}\frac{\partial a_j}{\partial w_{ji}}. \tag{8.7}$$

若某个权重同时出现在多个预激活的表达式中, 则右端应改为对这些预激活求和.

证明

(重构新增) 固定除 w_{ji} 之外的全部权重, 把 E 看成 w_{ji} 的一元函数. 由(8.5),a_j 是 w_{ji} 的可微函数, 而其余 a_l 与 w_{ji} 无关, 于是复合函数

$$g(t) = E\left(a_1, \cdots, a_{j-1}, a_j(t), a_{j+1}, \cdots, a_J\right)$$

在 $t = w_{ji}$ 处的导数由一元复合函数求导法则给出:

$$g'(w_{ji}) = \frac{\partial E}{\partial a_j}\bigg|_{\mathbf{a}} \cdot \frac{\mathrm{d}a_j}{\mathrm{d}w_{ji}}.$$

另一方面,$g'(w_{ji})$ 按定义就是 E 沿坐标 w_{ji} 的偏导数 $\partial E/\partial w_{ji}$, 而 $\mathrm{d}a_j/\mathrm{d}w_{ji}$ 就是偏导数 $\partial a_j/\partial w_{ji}$, 故(8.7)成立. 若 w_{ji} 同时影响若干个 a_l, 则 g 是多个一元函数的复合, 一元链式法则给出的是各项之和, 即右端对全部受影响的 a_l 求和.

□

注

(重构新增) 这条法则的内容可以概括为“上游导数乘局部导数”:$\partial E/\partial a_j$ 描述误差函数对单元 j 输出的敏感程度,$\partial a_j/\partial w_{ji} = z_i$ 描述权重对单元 j 预激活的敏感程度, 两者相乘即得权重对误差函数的影响. 它的条件必须逐条看清. 第一,E 与 a_j 都必须可微; 对于 ReLU 这类在原点不可微的激活函数, 法则在折点处失效, 实践中只能取某一侧的导数 (次梯度). 第二, 命题要求 w_{ji} 只影响一个预激活; 深度网络中同一权重通常只属于一条连接, 所以这一条自动满足, 但若做权重共享 (如卷积核在多个位置复用), 同一权重就会出现在多个 a_j 中, 此时必须求和而不能只取一项, 这是实现共享参数网络时最容易出错的地方. 第三, 法则只用到“局部”信息, 不要求知道 E 的全局形式——正是这一点使反向传播可以对任意拓扑的网络逐个单元地递推, 而不必展开整个复合函数.

接下来引入一个有用的符号.

定义 8.3 (误差项)

称单元 j 的预激活对误差函数的偏导数为该单元的**误差** (error), 记作

$$\delta_j \triangleq \frac{\partial E_n}{\partial a_j}, \tag{8.8}$$

其中 δ_j 通常简称为误差, 缘由后续将很快揭晓.

注

(重构新增) 把 δ_j 定义在预激活 a_j 而不是激活值 z_j 上, 是一个刻意的选择:a_j 是权重线性进入单元的位置,$\partial a_j/\partial w_{ji} = z_i$ 与单元自身无关, 因此所有“与单元有关”的信息都被集中到 δ_j 里, 权重梯度就简化为 $\delta_j z_i$ 这样一个纯粹的乘积. 这也解释了“误差”这个名称: 对输出单元,$\delta_k = y_k - t_k$ 恰是预测值与目标值之差; 对隐藏单元,δ_j 是把下游各单元的误差按权重 w_{kj} 加权后、再乘上本单元激活函数的斜率 $h'(a_j)$ 得到的量, 它不是某个可直接观测的“误差”, 而是误差信号反向流经该单元时的强度. 注意 δ_j 与 $\partial E_n/\partial z_j$ 只相差一个因子:$\delta_j = h'(a_j)\,\partial E_n/\partial z_j$, 当 $h'(a_j) = 0$(例如 tanh 饱和) 时该单元对上游完全没有影响, 这正是梯度消失现象的来源.

利用(8.5), 我们可以写出

$$\frac{\partial a_j}{\partial w_{ji}} = z_i. \tag{8.9}$$

将(8.8)和(8.9)代入(8.7), 即得到权重 w_{ji} 的梯度. 与它并列, 输出单元的误差由误差函数的具体形式直接给出, 隐藏单元的误差则可以对(8.8)再用一次链式法则而递归地得到. 整理起来, 就得到反向传播的四个基本方程.

定理 8.1 (反向传播的四个基本方程)

在(8.5)–(8.6)的前馈网络、(8.8)的误差项与(8.9)的记号下, 给定一个输入数据点, 误差函数的全部一阶导数可由下列四个方程求得.

(i) (输出单元的误差) 若输出单元采用与误差函数相匹配的规范连接函数作为激活函数 (参见小节 5.4.6), 则输出单元 k 的误差为

$$\delta_k = y_k - t_k, \tag{8.10}$$

其中 y_k 是输出单元的激活值,t_k 是相应的目标值;

(ii) (隐藏单元的误差) 隐藏单元 j 的误差由它向其发送连接的所有单元 k 的误差反向传播得到:

$$\delta_j = h'(a_j)\sum_k w_{kj}\delta_k, \tag{8.11}$$

其中求和遍及所有满足“单元 j 向单元 k 发送连接”的单元 k, 这些单元既可以是其他隐藏单元, 也可以是输出单元;

(iii) (权重的梯度) 权重 w_{ji} 的梯度为

$$\frac{\partial E_n}{\partial w_{ji}} = \delta_j z_i; \tag{8.12}$$

(iv) (偏置的梯度) 若把单元 j 的偏置 b_j 显式地写入预激活, 即 $a_j = \sum_i w_{ji}z_i + b_j$, 则

$$\frac{\partial E_n}{\partial b_j} = \delta_j. \tag{8.13}$$

♡

证明

下面按推导的先后次序逐条验证这四个方程.

权重的梯度(8.12)**.** 由命题 8.1与(8.9),

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial E_n}{\partial a_j}\frac{\partial a_j}{\partial w_{ji}} = \delta_j z_i.$$

输出单元的误差(8.10)**.** 以平方和误差(8.3)与线性输出 $y_k = a_k$ 为例,

$$\delta_k = \frac{\partial E_n}{\partial a_k} = \frac{\partial}{\partial a_k}\left[\frac{1}{2}\sum_l (y_l - t_l)^2\right] = (y_k - t_k)\frac{\partial y_k}{\partial a_k} = y_k - t_k.$$

一般地, 只要输出单元的激活函数取为与误差函数相匹配的规范连接函数, 同样的计算给出同一结果: 此时 $\partial E_n/\partial a_k$ 恰好等于“输出值减目标值”. 交叉熵与 sigmoid、交叉熵与 softmax 这两种常用组合满足这一条件, 具体的验证参见小节 5.4.6.

隐藏单元的误差(8.11)**.** 隐藏单元 j 的预激活 a_j 并不直接出现在误差函数中, 它只能通过它所影响的那些预激活 a_k(即满足“j 向 k 发送连接”的全部 k) 去影响 E_n. 因此对(8.8)再用一次链式法则:

$$\delta_j = \frac{\partial E_n}{\partial a_j} = \sum_k \frac{\partial E_n}{\partial a_k}\frac{\partial a_k}{\partial a_j} = \sum_k \delta_k \frac{\partial a_k}{\partial a_j},$$

这里用到了这样一个事实: 只有通过改变变量 a_k 的值,a_j 的变化才会引起误差函数的变化. 再由(8.5)与(8.6),$a_k = \sum_l w_{kl}h(a_l)$, 于是

$$\frac{\partial a_k}{\partial a_j} = w_{kj}h'(a_j),$$

代回上式即得

$$\delta_j = h'(a_j)\sum_k w_{kj}\delta_k.$$

偏置的梯度(8.13)(重构新增)**.** 原书把偏置吸收为激活值恒为 1 的额外输入, 没有单列这一方程; 若把偏置显

式写入预激活, 则 $\partial a_j/\partial b_j = 1$, 于是

$$\frac{\partial E_n}{\partial b_j} = \frac{\partial E_n}{\partial a_j}\frac{\partial a_j}{\partial b_j} = \delta_j.$$

若按原书的约定令 $z_0 = 1$、$b_j = w_{j0}$, 则(8.12)在 $z_i = 1$ 时也给出(8.13), 两种写法完全一致.

□

注

(重构新增) 四个方程可以分成“算误差”与“算梯度”两组:(8.10)与(8.11)负责从输出层出发、逆着计算图逐层求出每个单元的 δ;(8.12)与(8.13)负责把已经求出的 δ 就地转换为参数的梯度. 这样分组的意义在于复用: 同一个 δ_j 既用于 j 的全部入边权重 $\{w_{ji}\}_i$, 也用于偏置 b_j, 还参与前驱单元的 δ 的递推, 因此不必对每个参数单独做一次前向传播. 条件的边界要分清.(8.11)只要求 h 在 a_j 处可微; 若 h 是 ReLU 且在 $a_j = 0$ 处不可微, 需要约定一个次梯度值, 这会使结果依赖于实现约定.(8.10)的形式依赖于输出激活函数与误差函数的匹配 (规范连接函数); 若换成不匹配的组合, 例如 sigmoid 输出配平方和误差,δ_k 中会多出一个因子 $h'(a_k)$, 平方和误差在输出饱和时收敛极慢, 这正是分类问题普遍改用交叉熵的原因.(8.13)只是(8.12)在常数输入下的特例, 它说明偏置与权重的梯度在计算上没有本质区别. 最后, 两个“梯度”方程给出的都是单个数据点的梯度; 对整个数据集求和 ((8.14)) 即得总误差的梯度, 但求和必须在 δ 已经算好之后进行, 不能先把各数据点的 δ 平均掉再反传, 否则得到的是平均误差的梯度而不是梯度之和.

根据(8.12)可知, 只需要将权重输出端单元的 δ 值乘以权重输入端单元的 z 值 (其中偏置的 $z = 1$), 即可得到所需的导数. 请注意, 这与(8.4)中简单线性模型的形式相同. 因此, 为了计算导数, 我们只需要计算网络中每个隐藏单元和输出单元的 δ 值, 然后应用(8.12)即可.

由此可知, 特定隐藏单元的 δ 值可以通过从网络中较高的单元后向传播误差信息来获得, 如图 8.1所示.

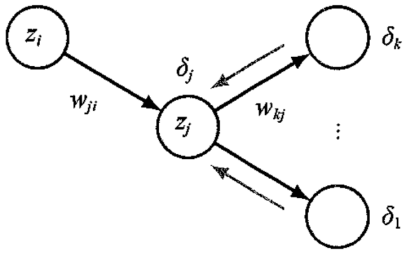


图 8.1: 通过从单元 k(单元 j 会向其发送连接) 对 δ 值进行反向传播, 计算隐藏单元 j 的 δ_j 值的示意图. 黑色箭头表示前向传播中的信息流方向, 红色箭头表示误差信息的反向传播

注意, 反向传播 [(8.11)] 中的求和取自 w_{kj} 上的第一个索引 (对应于信息通过网络后向传播), 而在前向传播 [(8.5)] 中, 求和取自第二个索引 (见练习 8.2). 由于已知输出单元的 δ 值, 因此通过递归应用(8.11), 我们可以计算前馈网络中所有隐藏单元的 δ 值, 而无须考虑其拓扑结构. 算法 8.1 对反向传播做了总结.

算法 8.1　反向传播

Input: 输入向量 $\mathbf{x}_n$
　网络参数 $\mathbf{w}$
　关于输入 $\mathbf{x}_n$ 的误差函数 $E_n(\mathbf{w})$
　激活函数 $h(a)$
Output: 误差函数的导数 $\{\partial E_n/\partial w_{ji}\}$
// 前向传播
for $j \in$ 所有隐藏单元和输出单元 **do**
　$a_j \leftarrow \sum_i w_{ji} z_i$　// $\{z_i\}$ 包括输入 $\{x_i\}$
　$z_j \leftarrow h(a_j)$　// 激活函数
end for
// 误差评估
for $k \in$ 所有输出单元 **do**
　$\delta_k \leftarrow \partial E_n/\partial a_k$　// 计算误差
end for
// 以相反顺序进行反向传播
for $j \in$ 所有隐藏单元 **do**
　$\delta_j \leftarrow h'(a_j) \sum_k w_{kj}\delta_k$　// 递归地进行反向计算
　$\partial E_n/\partial w_{ji} \leftarrow \delta_j z_i$　// 计算导数
end for
return $\{\partial E_n/\partial w_{ji}\}$

在上述推导中, 我们隐式地假设网络中的每个隐藏单元或输出单元都具有相同的激活函数 $h(\cdot)$. 不过, 只需要跟踪 $h(\cdot)$ 的哪种形式与哪个单元相匹配, 就可以轻松地将推导结果拓展到不同的单元, 使其拥有单独的激活函数.

对于批量处理方法, 对训练集中的每个数据点重复上述步骤, 然后对批量处理或小批量处理中的所有数据点求和, 即可得到总误差 E 的导数:

$$\frac{\partial E}{\partial w_{ji}} = \sum_n \frac{\partial E_n}{\partial w_{ji}}. \tag{8.14}$$

8.1.3 简单示例

上述推导采用了误差函数、激活函数和网络拓扑的一般形式. 为了演示反向传播的应用, 考虑图 6.9所示的两层网络以及平方和误差.

例题 8.1 两层网络的误差反向传播　输出单元具有线性激活函数, 因此 $y_k = a_k$; 而隐藏单元具有 sigmoid 型激活函数, 其计算公式为

$$h(a) = \tanh(a), \tag{8.15}$$

其中 $\tanh(a)$ 由(6.22)定义. 该函数的一个有用特征是, 它的导数可以用一种特别简单的形式来表示:

$$h'(a) = 1 - h(a)^2. \tag{8.16}$$

考虑平方和误差函数, 对于数据点 n, 误差的计算公式为

$$E_n = \frac{1}{2}\sum_{k=1}^{K} (y_k - t_k)^2, \tag{8.17}$$

其中 y_k 是输出单元 k 的激活值,t_k 是特定输入向量 $\mathbf{x}$ 的相应目标值. 对于训练集中的每个数据点, 首先使用以下

方法依次进行前向传播:

$$a_j = \sum_{i=0}^{D} w_{ji}^{(1)} x_i, \tag{8.18}$$

$$z_j = \tanh(a_j), \tag{8.19}$$

$$y_k = \sum_{j=0}^{M} w_{kj}^{(2)} z_j, \tag{8.20}$$

其中 D 是输入向量 $\mathbf{x}$ 的维度,M 是隐藏单元的总数. 此外, 我们还利用了 $x_0 = z_0 = 1$, 以便在权重中包含偏置参数.

解

用以下方法计算每个输出单元的 δ 值: 由(8.10), 输出单元为线性激活, 故

$$\delta_k = y_k - t_k.$$

然后利用如下方法对误差进行反向传播, 从而得出隐藏单元的 δ 值:

$$\delta_j = \left(1 - z_j^2\right) \sum_{k=1}^{K} w_{kj}^{(2)} \delta_k.$$

上式可根据(8.11)和(8.16)得出: 把 $h'(a_j) = 1 - \tanh^2(a_j) = 1 - z_j^2$ 代入(8.11), 并注意此处单元 j 只向 K 个输出单元发送连接, 即得. 最后, 第一层和第二层权重的导数为

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i, \qquad \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j,$$

它们分别是(8.12)作用于第一层连接 (输入端激活值为 x_i) 与第二层连接 (输入端激活值为 z_j) 的结果.

□

注

(重构新增) 这个例子之所以选 tanh 与线性输出, 是因为这两处导数都能写成闭合而极简的形式:tanh 的导数可以用自身表示 ((8.16)), 于是反向传播时不必再存储 a_j, 直接用前向阶段留下的 z_j 即可; 线性输出的导数为 1, 于是输出端不需要额外乘因子,δ_k 就直接等于残差 $y_k - t_k$. 这也说明网络中真正决定反向传播实现细节的只有两件事: 每个单元的 $h'(a_j)$ 如何计算, 以及输出单元与误差函数是否匹配. 条件方面,(8.18)–(8.20)把偏置写成了下标为 0 的权重, 因此求和从 0 开始,$w_{j0}^{(1)}$ 与 $w_{k0}^{(2)}$ 就是原来的偏置参数; 若按(8.13)显式处理偏置, 则求和应从 1 开始并另外加上 b_j, 两种写法给出相同的梯度.

8.1.4 数值微分法

反向传播最重要的方面之一是计算效率. 为此, 让我们来看看计算误差函数导数所需的计算操作数是如何随着网络中权重和偏置的总数 W 的变化而变化的.

在 W 足够大的情况下, 计算误差函数导数 (给定输入数据点) 需要 $O(W)$ 次运算. 这是因为除连接非常稀疏的网络外, 权重的数量通常远大于单元的数量, 因此前向传播的大部分计算量来自(8.5)中的求和操作, 而计算激活函数的工作量却很小.(8.5)中参与求和的每一项都需要进行一次乘法和一次加法运算, 因此总计算成本为 $O(W)$.

命题 8.2 (反向传播的计算复杂度)

设网络中有 W 个权重与偏置, 且有 U 个单元. 若 W 足够大 (典型情形是 W 远大于 U), 则给定一个输入数据点, 用反向传播计算误差函数关于全部 W 个参数的一阶导数所需的运算次数为 $O(W)$, 即与一次前向传播同一量级.

♠

证明

一次前向传播的开销由(8.5)的求和主导: 每条连接贡献一次乘法与一次加法, 故前向传播需要 $O(W)$ 次运算; 激活函数只对每个单元作用一次, 合计 $O(U)$, 在 W 远大于 U 时可并入 $O(W)$. 反向传播分两步. 第一步按(8.11)求各单元的 δ: 每个隐单元要对它发出的每条连接做一次乘加, 合计仍是每条连接一次, 故为 $O(W)$; 第二步按(8.12)与(8.13)把每个 δ 乘以对应的 z_i, 每个参数一次乘法, 共 $O(W)$. 三步相加仍为 $O(W)$, 其中常数因子约为前向传播的 2 至 3 倍.

□

注

(重构新增) 这条结论是反向传播全部实用价值的来源: 它说明“一次前向 + 一次反向”就能拿到全部 W 个偏导数, 平均到每个参数只花常数次运算. 若改用有限差分法逐个扰动参数, 同样的信息需要 W 次前向传播, 代价是 $O(W^2)$(命题 8.3). 结论中的“W 足够大”不是修辞: 当网络极稀疏、W 与 U 同阶甚至更小时, 激活函数的 $O(U)$ 开销不能再被忽略, 此时 $O(W)$ 的写法应理解为 $O(W+U)$. 另外, 这里的计数只统计算术运算, 不统计访存: 实际实现中反向传播常常受内存带宽限制, 实测加速比往往低于运算次数的比值, 这也是工程上要尽量让中间结果留在高速缓存中的原因.

计算误差函数导数的另一种反向传播方法是使用有限差分法. 具体做法是依次扰动每个权重, 并利用该表达式近似求导来实现:

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n\left(w_{ji}+\varepsilon\right) - E_n\left(w_{ji}\right)}{\varepsilon} + O(\varepsilon), \tag{8.21}$$

其中 $\varepsilon \ll 1$. 在软件模拟中, 可以通过将 ε 最小化来提高导数近似值的精度, 直到出现数值舍入问题为止. 使用对称中心差分法可以显著提高有限差分法的精度, 形式如下:

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n\left(w_{ji}+\varepsilon\right) - E_n\left(w_{ji}-\varepsilon\right)}{2\varepsilon} + O\left(\varepsilon^2\right). \tag{8.22}$$

在这种情况下, 正如(8.22)右侧的 Taylor 展开所验证的,$O(\varepsilon)$ 修正被抵消了, 因此残差修正为 $O(\varepsilon^2)$(见练习 8.3). 但请注意, 计算步骤的数量相比(8.21)增加了大约一倍.图 8.2显示了利用有限差分法 [(8.21)] 和对称中心差分法 [(8.22)] 对梯度进行数值计算的结果与分析结果之间的误差.

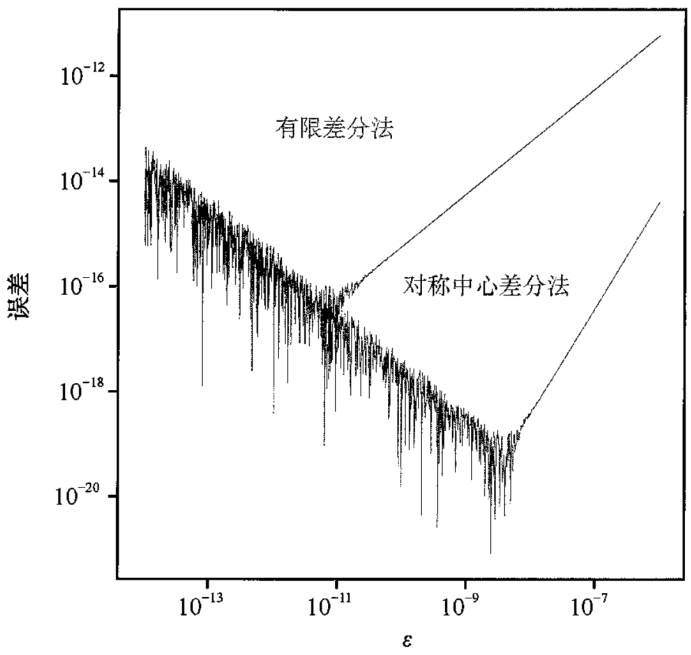


图 8.2: 红色曲线显示了使用有限差分法对梯度进行数值计算的结果与分析结果之间的误差. 随着 ε 不断减小, 误差呈线性减小, 且由于坐标轴是对数, 因此这代表了一种幂律行为. 红色曲线的斜率为 1, 这表明误差为 $O(\varepsilon)$. 在某一点上, 所计算的梯度达到数值四舍五入的极限, 且 ε 的进一步减小会导致出现一条噪声线, 该噪声线同样遵循幂律, 但误差会随着 ε 的减小而增大. 蓝色曲线显示了使用对称中心差分法对梯度进行数值计算的结果与分析结果之间的误差. 与有限差分法相比, 对称中心差分法的误差要小得多, 而且蓝色曲线的斜率为 2, 这表明误差为 $O(\varepsilon^2)$

数值微分法的主要问题是失去了非常理想的 $O(W)$ 缩放. 每次前向传播需要执行 $O(W)$ 个步骤, 而网络中有 W 个权重, 每个权重都必须单独扰动, 因此总计算成本为 $O(W^2)$.

命题 8.3 (数值微分的计算复杂度)

用有限差分法(8.21)或中心差分法(8.22)计算误差函数关于全部 W 个参数的梯度, 需要 $O(W^2)$ 次运算, 即比反向传播贵一个因子 W.

♠

证明

有限差分(8.21)每估计一个参数的偏导数需要两次误差函数求值 (一次在原参数处、一次在扰动后的参数处), 中心差分(8.22)需要两次扰动求值; 无论哪一种, 对全部 W 个参数逐一处理都要进行 $\Theta(W)$ 次误差函数求值. 每次求值都是一次前向传播, 代价为 $O(W)$, 故总代价为 $O(W)\times O(W) = O(W^2)$. 中心差分只把常数因子提高约一倍, 不改变量级.

□

注

(重构新增) $O(W^2)$ 与 $O(W)$ 的差别不是常数因子, 而是量级: 当 W 达到 10^6 时, 两者相差一百万倍, 因此把数值微分当作训练算法在实际中不可行. 还要注意近似精度与舍入误差的权衡:(8.21)的截断误差为 $O(\varepsilon)$,(8.22)为 $O(\varepsilon^2)$, 减小 ε 似乎总能提高精度; 但 ε 小到一定程度后,$E_n(w_{ji}+\varepsilon) - E_n(w_{ji})$ 是两个相近数之差, 舍入误差被 ε 放大, 总误差随 ε 减小反而上升——图 8.2中红色曲线与蓝色曲线在极小 ε 处的噪声线正是这一现象. 因此中心差分除了公式本身的二阶精度, 还能把“精度开始劣化”的 ε 推得更小. 它真正的用途不是训练, 而是检验: 把反向传播或自动微分得到的导数与中心差分的结果比较, 可以在不重写反向代码的前提下发现实现错误, 这也是本书建议的调试手段.

不过, 数值微分法在实践中也能发挥重要作用, 因为将直接实现反向传播或自动微分法计算出的导数与使用中心差分法计算出的导数进行比较, 可以有力地检验软件的正确性.

8.1.5 Jacobi 矩阵

我们已经了解了如何通过网络反向传播误差来获得误差函数相对于权重的导数. 反向传播也可用于计算其他导数. 在此, 我们对 Jacobi 矩阵进行求值, 其元素由网络输出相对于输入的导数给出.

定义 8.4 (Jacobi 矩阵)

设网络输出 y_k 与输入 x_i 均可微, 则称由网络输出相对于输入的导数构成的矩阵

$$J_{ki} = \frac{\partial y_k}{\partial x_i} \tag{8.23}$$

为网络所表示函数的 **Jacobi 矩阵** (Jacobian matrix), 其中每个导数都是在所有其他输入值保持不变的情况下求得的.

♣

注

(重构新增) Jacobi 矩阵回答的是与梯度不同的问题: 梯度问“参数怎样变能使误差下降”,Jacobi 矩阵问“输入怎样变会使输出怎样变”. 它是一个 $K\times D$ 的矩阵 (K 为输出维数、D 为输入维数), 而不是一个与参数同维的向量, 因此“反向传播能算 Jacobi 矩阵”并不是同义反复——它需要把误差反向传播的程序改造成对每个输出单元分别反向一次的循环. 定义要求 y_k 对 x_i 可微, 这排除了含硬阈值或取整操作的网络; 定义还强调“其他输入保持不变”, 即 J_{ki} 是偏导数而不是全导数, 若输入之间存在相关性 (例如做了归一化), 改变 x_i 时其他输入并不会真的保持不变, 此时用(8.25)解释灵敏度必须谨慎. 由于网络映射是非线性的,J 依赖于具体的输入点, 必须在每个新输入处重新计算.

如图 8.3所示,Jacobi 矩阵在由多个不同模块构建的系统中发挥着重要作用. 每个模块都可以包含一个固定的或可学习的函数, 该函数可以是线性的或非线性的, 但必须是可微分的. 假设我们希望最小化图 8.3中与参数 $\mathbf{w}$ 有

关的误差函数 E, 该误差函数的导数为

$$\frac{\partial E}{\partial \mathbf{w}} = \sum_{k,j} \frac{\partial E}{\partial y_k} \frac{\partial y_k}{\partial z_j} \frac{\partial z_j}{\partial \mathbf{w}}. \tag{8.24}$$

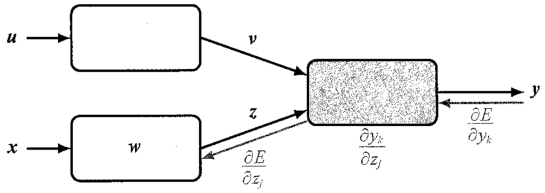


图 8.3: 模块化深度学习架构示意图, 其中 Jacobi 矩阵可用于将误差信号从输出反向传播至系统的早期模块

图 8.3中红色模块所代表的 Jacobi 矩阵出现在(8.24)右侧的中间项.

由于 Jacobi 矩阵度量了输出变量对每个输入变量变化的局部灵敏度, 因此它还允许与输入相关的任何已知误差 Δx_i 通过经训练的网络进行传播, 从而预估它们对输出误差的贡献 Δy_k:

$$\Delta y_k \approx \sum_i \frac{\partial y_k}{\partial x_i} \Delta x_i. \tag{8.25}$$

这里假设 $|\Delta x_i|$ 很小. 一般来说, 训练后的神经网络所代表的网络映射是非线性的, 因此 Jacobi 矩阵的元素不是常数, 而取决于所使用的特定输入向量. 因此,(8.25)只对输入的微小扰动有效, 而且必须为每个新输入向量重新计算 Jacobi 矩阵.

我们可以使用反向传播程序来计算 Jacobi 矩阵, 该程序与前面计算误差函数相对于权重的导数的程序类似. 首先写下元素 J_{ki}, 形式如下:

$$J_{ki} = \frac{\partial y_k}{\partial x_i} = \sum_j \frac{\partial y_k}{\partial a_j} \frac{\partial a_j}{\partial x_i} = \sum_j w_{ji} \frac{\partial y_k}{\partial a_j}, \tag{8.26}$$

其中使用了(8.5).(8.26)中的求和操作涉及所有单元 j(例如前面提到的分层拓扑结构中第一隐藏层的所有单元), 且输入单元 i 向单元 j 发送连接. 导数 $\partial y_k/\partial a_j$ 的递归反向传播公式如下:

$$\frac{\partial y_k}{\partial a_j} = \sum_l \frac{\partial y_k}{\partial a_l} \frac{\partial a_l}{\partial a_j} = h'(a_j) \sum_l w_{lj} \frac{\partial y_k}{\partial a_l}, \tag{8.27}$$

其中的求和操作涉及所有单元 l, 且单元 j 向单元 l 发送连接 (对应于 w_{lj} 的第一个索引). 我们再次使用了(8.5)和(8.6). 这种反向传播从输出单元开始, 所需导数可直接从输出单元激活函数的函数形式中得到. 对于线性输出单元, 我们可以得出

$$\frac{\partial y_k}{\partial a_l} = \delta_{kl}, \tag{8.28}$$

其中 δ_{kl} 是单位矩阵的元素, 定义如下:

$$\delta_{kl} = \begin{cases} 1, & k = l, \\ 0, & \text{其他}. \end{cases} \tag{8.29}$$

如果我们在每个输出单元上都有独立的逻辑斯谛 sigmoid 激活函数 (参见节 3.4), 则有

$$\frac{\partial y_k}{\partial a_l} = \delta_{kl}\, \sigma'(a_l). \tag{8.30}$$

而对于 softmax 输出 (参见节 3.4), 可以得出

$$\frac{\partial y_k}{\partial a_l} = \delta_{kl}\, y_k - y_k y_l. \tag{8.31}$$

我们可以将 Jacobi 矩阵的计算程序总结如下. 首先, 应用与输入空间 (在该空间中需要计算 Jacobi 矩阵) 中的点相对应的输入向量, 并按照常规方法进行前向传播, 以得到网络中所有隐藏单元和输出单元的状态. 接下来, 对于 Jacobi 矩阵中与输出单元 k 相对应的每一行 k, 使用递归关系式(8.27), 从(8.28)、(8.30)或(8.31)开始, 对网络中的所有隐藏单元进行反向传播. 最后, 使用(8.26)对输入进行反向传播.Jacobi 矩阵也可以用另一种前向传播形式来

计算, 其推导方法与此处的反向传播方法类似 (见练习 8.5).

同样, 此类算法的实现可以用形式如下的数值微分来检验:

$$\frac{\partial y_k}{\partial x_i} = \frac{y_k(x_i+\varepsilon) - y_k(x_i-\varepsilon)}{2\varepsilon} + O\left(\varepsilon^2\right). \tag{8.32}$$

对于有 D 个输入的网络来说, 这涉及 $2D$ 次前向传播, 因此总共需要 $O(DW)$ 个步骤.

8.1.6 黑塞矩阵

我们已经展示了如何利用反向传播来获得误差函数相对于网络权重的一阶导数. 反向传播也可用于计算误差的二阶导数, 表达式如下

$$\frac{\partial^2 E}{\partial w_{ji}\partial w_{lk}} \tag{8.33}$$

方便起见, 我们通常将所有权重和偏置参数视为单个向量 (用 $\mathbf{w}$ 表示) 的元素 w_i.

定义 8.5 (黑塞矩阵)

设误差函数 $E(\mathbf{w})$ 关于参数向量 $\mathbf{w} = (w_1, \cdots, w_W)^{\mathsf{T}}$ 二阶可微. 此时二阶导数构成**黑塞矩阵** (Hessian matrix)$\mathbf{H}$ 的元素

$$H_{ij} = \frac{\partial^2 E}{\partial w_i \partial w_j}, \tag{8.34}$$

其中 $i, j \in \{1, \cdots, W\}$,$W$ 是权重和偏置的总数. ♣

注

(重构新增) 黑塞矩阵把误差曲面在一点处的全部二阶信息打包成一个 $W \times W$ 的对称矩阵: 对角元 H_{ii} 是沿坐标方向的曲率, 非对角元 H_{ij} 刻画两个方向之间的耦合. 它与定义 8.4的区别在于对谁求导:Jacobi 矩阵对输入求导, 黑塞矩阵对参数求二阶导, 因此后者的规模由参数个数决定, 在深度网络中可达 $10^6 \times 10^6$, 显式存储都不可能, 更不用说求逆. 定义要求 E 二阶可微, 这保证 $H_{ij} = H_{ji}$; 若只二阶可微而不连续, 混合偏导数可能不相等,$\mathbf{H}$ 就不对称, 基于 $\mathbf{H}$ 的优化算法 (Newton 法、共轭梯度) 所依赖的性质也随之失效.$\mathbf{H}$ 的用途有三类: 判断驻点的类型 (正定、负定、不定分别对应极小、极大、鞍点)、构造二次近似以加速优化、以及在 Bayes 近似中充当后验精度的度量.

基于误差表面二阶特性的考虑, 黑塞矩阵出现在了用于训练神经网络的几种非线性优化算法中 (Bishop, 2006). 此外, 它还在神经网络的某些 Bayes 处理中发挥作用 (MacKay, 1992; Bishop, 2006), 并用于降低大语言模型中权重的精度, 从而减少内存占用 (Shen et al., 2019).

在黑塞矩阵的众多应用中, 评估黑塞矩阵的效率是十分重要的. 如果网络中有 W 个参数 (权重和偏置), 那么黑塞矩阵的维度为 $W \times W$, 因此对于数据集中的每个数据点, 计算黑塞矩阵所需的计算量将缩放为 $O(W^2)$(见练习 8.6). 通过扩展反向传播程序 (Bishop, 1992), 可以有效地计算黑塞矩阵, 缩放比例实际为 $O(W^2)$. 有时, 我们并非直接需要黑塞矩阵, 而只需要黑塞矩阵与某个向量 $\mathbf{v}$ 的乘积 $\mathbf{v}^{\mathsf{T}}\mathbf{H}$; 通过扩展反向传播程序, 我们可以在 $O(W)$ 个步骤内高效地计算出该乘积 (Møller, 1993; Pearlmutter, 1994).

由于神经网络可能包含数百万个甚至数十亿个参数, 因此对许多模型的全黑塞矩阵进行计算, 甚至只进行存储, 都是不可行的. 由于计算量被缩放为 $O(W^3)$, 求黑塞矩阵的逆将变得更加困难. 业内人士对寻找有效的全黑塞矩阵近似值很感兴趣.

一种近似法是只对黑塞矩阵的对角元素进行计算, 并隐式地将非对角元素设为零. 这需要 $O(W)$ 的存储空间, 并允许在 $O(W)$ 个步骤内进行求逆, 但仍需要 $O(W^2)$ 个计算步骤 (Ricotti, Ragazzini, and Martinelli, 1988), 不过通过进一步近似, 计算步骤可以减少到 $O(W)$ 个 (Becker and LeCun, 1989; LeCun, Denker, and Solla, 1990). 但实际上, 黑塞矩阵通常具有重要的非对角项, 因此必须谨慎对待这一近似值.

另一种更有说服力的方法是外积近似法, 原理如下. 考虑使用平方和误差函数的回归应用, 平方和误差函数如

下:

$$E = \frac{1}{2}\sum_{n=1}^{N}(y_n - t_n)^2, \tag{8.35}$$

其中只使用了一个单一输出, 以保持符号简洁 (扩展到多个输出也很简单). 然后写下黑塞矩阵 (见练习 8.8), 形式如下:

$$\mathbf{H} = \nabla\nabla E = \sum_{n=1}^{N}\nabla y_n(\nabla y_n)^{\mathsf{T}} + \sum_{n=1}^{N}(y_n - t_n)\nabla\nabla y_n, \tag{8.36}$$

其中 ∇ 表示相对于 $\mathbf{w}$ 的梯度. 如果网络已在数据集上经过训练, 则输出 y_n 将非常接近目标值 t_n,(8.36)中末尾的项很小, 可以忽略不计. 不过, 从更广泛的意义上讲, 根据以下推论, 忽略这一项可能是合适的. 回顾节 4.2, 最优函数是使平方和损失最小化的目标数据的条件平均值. 量 $(y_n - t_n)$ 是一个均值为零的随机变量. 如果假设它的值与(8.36)右侧二阶导数项的值无关, 那么在对 n 求和时, 整个导数项的平均值将为零 (见练习 8.9).

忽略(8.36)右侧的二阶导数项, 即可得到 Levenberg–Marquardt 近似法, 也叫作外积近似法, 因为黑塞矩阵是由向量的外积之和构成的.

定义 8.6 (外积近似 (Levenberg–Marquardt 近似)) 对形如(8.35)的平方和误差函数, 略去(8.36)中的二阶导数项, 用一阶导数 (梯度) 的外积之和作为黑塞矩阵的近似:

$$\mathbf{H} \approx \sum_{n=1}^{N}\nabla a_n \nabla a_n^{\mathsf{T}}. \tag{8.37}$$

这一近似称为**外积近似** (outer product approximation), 也称 Levenberg–Marquardt 近似. ♣

注

(重构新增) 外积近似用“梯度的外积”代替“二阶导数”, 好处是只依赖一阶信息: 一阶导数已由反向传播在 $O(W)$ 内算出, 再对每个数据点做一次外积, 就得到 $\mathbf{H}$ 的全部元素, 总代价 $O(W^2)$, 与显式求二阶导的代价同阶但常数小得多; 而且每个外积 $\nabla a_n \nabla a_n^{\mathsf{T}}$ 都是半正定的秩一矩阵, 故近似后的 $\mathbf{H}$ 自动半正定, 这使基于它的迭代 (如 Levenberg–Marquardt 的阻尼步) 不会因负曲率而失稳. 代价是它丢掉了二阶项, 因此近似的好坏取决于 $|y_n - t_n|$ 是否足够小: 网络训练充分时残差接近零, 近似成立; 网络远未训练好时残差很大, 二阶项不可忽略, 此时用外积近似可能严重低估曲率 (见练习 8.9的讨论). 还要注意,(8.37)写的是 ∇a_n 而不是 ∇y_n, 这是因为在平方和误差的标准配置中输出单元是线性的,$y_n = a_n$, 两者相同; 若输出单元带非线性激活, 则必须换回 ∇y_n.

求黑塞矩阵的外积近似值非常简单, 因为仅涉及误差函数的一阶导数, 使用标准反向传播法即可在 $O(W)$ 个步骤内高效地计算出误差函数的一阶导数. 而通过进行简单的乘法运算, 只需要 $O(W^2)$ 个步骤即可求出矩阵的元素. 需要强调的是, 这种近似法可能只适用于经过适当训练的网络, 且对于一般的网络映射,(8.36)右侧的二阶导数项通常是不可忽略的.

对于具有逻辑斯谛 sigmoid 输出单元激活函数的网络, 其交叉熵误差函数的相应近似值为

$$\mathbf{H} \approx \sum_{n=1}^{N} y_n(1 - y_n)\nabla a_n \nabla a_n^{\mathsf{T}}. \tag{8.38}$$

对于具有 softmax 输出单元激活函数的多类网络, 均可以得到类似的结果 (见练习 8.11). 外积近似法还可用于开发一种高效的序列程序, 用于近似黑塞矩阵的逆 (Hassibi and Stork, 1993)(见练习 8.12).

8.2 自动微分法

我们已经了解到利用梯度信息高效训练神经网络的重要性 (参见小节 7.2.1). 评估神经网络误差函数梯度的方法主要有 4 种.

第 1 种方法是手工推导反向传播方程, 然后在软件中将其显式地实现. 这种方法多年来一直是神经网络的中

流砥柱. 如果能认真地实施这种方法, 就能产生高效的代码, 从而给出达到数值精度要求的结果. 然而, 推导方程和编码的过程都十分耗费时间且容易出错. 前向传播方程与反向传播方程是分开编码的, 这会导致代码中的一些冗余. 又由于其中往往涉及重复计算, 因此如果模型发生变化, 就需要同步改变前向实现和反向实现, 这容易限制对不同架构进行经验探索的速度和效率.

第 2 种方法是利用有限差分法对梯度进行数值计算 (参见小节 8.1.4). 这种方法只需要用软件实现前向传播方程. 数值微分法的一个问题是计算精度有限, 不过这对于网络训练来说可能不是问题, 因为我们可能会使用随机梯度下降法. 在随机梯度下降法中, 每次计算都只是对局部梯度非常粗略的估计. 这种方法的主要缺点是在网络规模的缩放上表现很差. 不过, 这种技术对于调试其他方法也很有用, 因为梯度的计算只使用了前向传播代码, 因此可以用来确认反向传播的正确性, 以及确认其他用于计算梯度的代码的正确性.

第 3 种方法是符号微分法, 这种方法能利用专业软件将我们在第 1 种方法中手工完成的分析操作自动化. 作为计算机代数或符号计算的一个例子, 这一过程涉及在一个完全机械化的过程中自动应用微积分规则 (如链式法则). 符号微分法的一个明显优势是避免了人工推导反向传播方程时引入的人为误差. 此外, 梯度的计算再次达到了机器精度, 避免了数值微分时出现的缩放性差的问题.

然而, 符号微分法的主要缺点是导数的表达式可能比原始函数的表达式长很多, 其长度呈指数级增长, 相应的计算时间也很长. 考虑由 $u(x)$ 和 $v(x)$ 的乘积给出的函数 $f(x)$, 该函数及其导数的计算公式为

$$f(x) = u(x)v(x), \tag{8.39}$$

$$f'(x) = u'(x)v(x) + u(x)v'(x). \tag{8.40}$$

可以看到, 在计算 $f(x)$ 和 $f'(x)$ 时, 必须同时求出 $u(x)$ 和 $v(x)$, 因此存在冗余计算. 如果因子 $u(x)$ 和 $v(x)$ 本身涉及因子, 就会出现表达式嵌套重复, 复杂性会迅速增加.

定义 8.7 (表达式膨胀)

在符号微分法中, 若被微分的表达式含有重复出现的子表达式, 则求导后得到的表达式会因这些子表达式被反复复制而急剧变长, 这种“导数表达式远长于原表达式、且含大量重复计算”的现象叫作**表达式膨胀** (expression swell). ♣

注

(重构新增) 表达式膨胀的根源在于符号微分只做“字符串替换式”的变换: 它按链式法则把导数写成原表达式的组合, 却不判断哪些子表达式相同、更不会把它们提取成公共变量. 于是每求一次导, 表达式的规模大约按因子个数倍增, n 层复合之后长度可达指数级; 即使长度尚可接受, 重复出现的子表达式也会被反复计算, 实际运行时间随之膨胀. 条件方面, 膨胀的严重程度取决于表达式的“共享结构”: 若把重复出现的子表达式显式记为中间变量 (例如令 $u = w_1x + b_1$), 导数表达式立刻变短, 这正是自动微分法与符号微分法的分水岭——自动微分法始终以中间变量为单位工作, 从不展开复合结构. 需要强调, 膨胀是符号微分法的固有代价, 不是实现不佳; 只要坚持输出“导数的封闭形式表达式”, 就无法彻底避免它.

为了进一步说明, 考虑一个结构类似于两层神经网络的函数 (Grosse, 2018), 该函数有单个输入 x、带激活 z 的一个隐藏单元和输出 y.

例题 8.2 软 ReLU 网络的表达式膨胀 该函数由

$$z = h(w_1x + b_1), \tag{8.41}$$

$$y = h(w_2z + b_2) \tag{8.42}$$

给出, 其中 $h(a)$ 是软 ReLU:

$$\zeta(a) = \ln\left(1 + \exp(a)\right). \tag{8.43}$$

于是有

$$y(x) = h(w_2h(w_1x + b_1) + b_2). \tag{8.44}$$

网络输出 y 相对于 w_1 的导数为 (见练习 8.13)

$$\frac{\partial y}{\partial w_1} = \frac{w_2 x \exp\left(w_1 x + b_1 + b_2 + w_2 \ln\left[1 + e^{w_1 x + b_1}\right]\right)}{\left(1 + e^{w_1 x + b_1}\right)\left(1 + \exp\left(b_2 + w_2 \ln\left[1 + e^{w_1 x + b_1}\right]\right)\right)}. \tag{8.45}$$

除了比原始函数复杂得多之外, 还存在冗余计算, 比如像 $w_1 x + b_1$ 这样的表达式就出现在了多个地方.

注

(重构新增) 这个例子把表达式膨胀的规模具体化了: 原函数(8.44)只含 $w_1 x + b_1$ 这一个复合结构, 而它的导数(8.45)中, 子表达式 $w_1 x + b_1$、由它构成的 $e^{w_1 x + b_1}$ 以及 $\ln\left[1 + e^{w_1 x + b_1}\right]$ 都反复出现了多次, 表达式长度约为原来的数倍; 若网络再深几层, 长度会继续按层数倍增. 注意原书在(8.43)中把这个函数记作 $\zeta(a)$, 而(8.41)、(8.42)、(8.44)与(8.45)都用 h 表示同一个函数; 从紧邻的说明 "$h(a)$ 是软 ReLU" 以及后文只出现 h 来看, 二者指同一对象, 本书按原书排版保留 ζ 的写法, 阅读时把它等同于 h 即可.

符号微分法的另一个主要缺点是, 它需要以封闭形式表示被微分的表达式. 因此, 这种方法排除了重要的控制流操作, 如循环、递归、条件执行和过程调用, 而这些都是我们在定义网络函数时可能希望使用的有价值结构.

第 4 种方法是自动微分法, 也称 "算法微分法" (Baydin et al., 2018). 与符号微分法不同的是, 自动微分法旨在让计算机在仅给定前向传播方程代码的条件下自动生成实现梯度计算的代码, 而非得到导数的数学表达式. 与符号微分法一样, 自动微分法也达到了机器精度, 但效率更高, 因为它能够利用我们在定义前向传播方程时使用的中间变量, 从而避免冗余计算. 值得注意的是, 自动微分法不仅能处理传统的封闭式数学表达式, 还能处理流程控制元素, 如分支、循环、递归和过程调用. 因此, 自动微分法的功能比符号微分法强大得多. 与此同时, 自动微分法具有广泛的适用性, 是在机器学习领域之外发展起来的. 现代深度学习在很大程度上是一个经验过程, 涉及对不同架构的计算和比较. 因此, 自动微分法在准确高效地完成实验方面发挥着关键作用.

定义 8.8 (自动微分法)

自动微分法 (automatic differentiation) 是采用计算函数的代码 (例如计算神经网络误差函数的前向传播方程代码), 并用附加变量来扩充该代码, 使这些附加变量的值在代码执行过程中累加, 从而获得所需导数的一类方法. 它并不给出导数的封闭形式表达式, 而是给出一个与原代码同样可执行的、计算导数的程序. 自动微分法主要有两种形式: **前向模式**和**逆模式**.

♣

注

(重构新增) 自动微分法与另外三种方法的分界可以用一句话概括: 它微分的是 "程序", 不是 "公式". 正因如此, 它能穿过分支、循环与函数调用——这些结构在符号微分中没有对应的表达式形式, 在数值微分中又只能整体当作黑箱. 它的精度是机器精度, 因为每一步都只是对已有的浮点值做乘加, 不做任何近似展开; 它的效率接近一次函数求值, 因为每条初等运算的导数都可以用该运算的输入输出值直接算出, 不必重复计算子表达式. 定义中的 "附加变量" 是全部机制的关键: 对每个中间变量 v_j 附加一个导数变量 (前向模式中记作 $\dot{v}_j$, 逆模式中记作 $\bar{v}_j$), 原代码的每一行都被改写为 "原值 + 导数值" 的一行, 于是求导被转化为一次普通的数值执行. 需要说明, 本定义并不要求原函数有解析表达式, 只要求它由可微的初等运算按确定的顺序复合而成.

我们首先来看概念上较为简单的前向模式.

8.2.1 前向模式自动微分

定义 8.9 (前向模式自动微分)

在前向模式自动微分中, 我们在函数 (如神经网络的误差函数) 计算过程中涉及的每个中间变量 z_i(称为 "原始" 变量) 上添加一个附加变量, 该附加变量代表中间变量的某个导数值. 我们将其记作 $\dot{z}_i$, 即 "正切" 变量. 正切变量及其相关代码由软件环境自动生成. 现在, 代码不再简单地通过前向传播来计算 z_i, 而是传播元组 $\{z_i, \dot{z}_i\}$, 从而同时计算变量和导数.

♣

注

(重构新增) 前向模式的直觉是“把导数带着一起往前走”: 每算出一个中间量, 就同时算出它对所关心的那个输入(或参数) 的导数, 两者沿同一条求值顺序推进. 定义中的“正切”一词来自几何: 若把输入看作曲线参数, $\dot{z}_i$ 就是曲线在 z_i 处的切向量分量, 因此前向模式算的正是沿某个输入方向的“方向导数”. 它只要求原函数由初等算子 (算术运算、取负) 与指数、对数、三角函数等超越函数复合而成, 因为这些函数的导数都有简单公式, 与链式法则结合即可自动生成代码. 条件方面, 前向模式一次传播只能携带一个方向的种子: $\dot{\mathbf{x}}$ 取定之后, $\dot{z}_i$ 就是 $\partial z_i/\partial$(该方向), 所以要得到完整的 Jacobi 矩阵必须对每个输入方向各跑一次. 这一限制正是后文转向逆模式的理由.

举例来说, 如下函数有两个输入变量:

$$f(x_1, x_2) = x_1 x_2 + \exp(x_1 x_2) - \sin(x_2). \tag{8.46}$$

在软件中实现时, 代码由一系列操作组成, 这些操作可以用底层基本操作的求值轨迹 (evaluation trace) 来表示.

定义 8.10 (求值轨迹)

把函数计算过程中依次执行的每个初等运算的结果记为一个**原始变量** (primal variable), 并把它们以及“每个原始变量由哪些原始变量算出”这一依赖关系按执行顺序记录下来; 这些原始变量作为节点、依赖关系作为有向边构成的有向无环图, 称为该计算的**求值轨迹** (evaluation trace). 轨迹中的节点是原始变量, 边表示“某个原始变量由另一些原始变量通过一次初等运算得到”这一依赖关系; 它也就是定义 8.1所说的计算图在“运算”这一粒度上的具体化. ♣

注

(重构新增) 求值轨迹把“一段代码”变成“一张可以正反两向遍历的图”, 这是自动微分能够同时支持两种模式的结构基础. 定义要求按执行顺序排列, 是因为原始变量只有在它的全部父节点求值之后才有定义; 这条顺序在前向模式中就是计算顺序, 在逆模式中取反序即为反向传播的顺序. 轨迹的粒度取“一次初等运算”, 既不能粗到整个矩阵乘法 (那样无法逐元素套用链式法则), 也不必细到每个浮点加法 (那样开销过大); 实际框架通常以“算子”为单位, 这与定义中的初等运算一致. 需要注意, 轨迹依赖于控制流: 同一段代码对不同输入可能走不同分支, 因而产生不同的轨迹, 逆模式要求重新执行时仍走同一条轨迹, 这也是它必须保存中间变量的原因之一.

如图 8.4所示, 这种轨迹可以用图形来可视化. 此处对以下原始变量进行了定义:

$$\begin{aligned} v_1 &= x_1, \\ v_2 &= x_2, \\ v_3 &= v_1 v_2, \\ v_4 &= \sin(v_2), \\ v_5 &= \exp(v_3), \\ v_6 &= v_3 - v_4, \\ v_7 &= v_5 + v_6. \end{aligned} \tag{8.47, 8.48}$$

假设要求导数 $\partial f/\partial x_1$. 我们用 $\dot{v}_i = \partial v_i/\partial x_1$ 来定义正切变量. 利用微积分中的链式法则, 可以自动构造出用于计算这些值的表达式:

$$\dot{v}_i = \frac{\partial v_i}{\partial x_1} = \sum_{j \in \mathrm{pa}(i)} \frac{\partial v_j}{\partial x_1} \frac{\partial v_i}{\partial v_j} = \sum_{j \in \mathrm{pa}(i)} \dot{v}_j \frac{\partial v_i}{\partial v_j}, \tag{8.49}$$

其中 $\mathrm{pa}(i)$ 表示求值轨迹中节点 i 的父节点的集合, 即指向节点 i 的变量的集合. 例如, 在图 8.4中, 节点 v_3 的父节点是节点 v_1 和 v_2. 将(8.49)应用于求值轨迹方程 [(8.47) ~ (8.48)], 可以得到以下正切变量的求值轨迹方程:

$$\begin{aligned} \dot{v}_1 &= 1, \\ \dot{v}_2 &= 0, \\ \dot{v}_3 &= \dot{v}_1 v_2 + \dot{v}_2 v_1, \end{aligned} \tag{8.50}$$

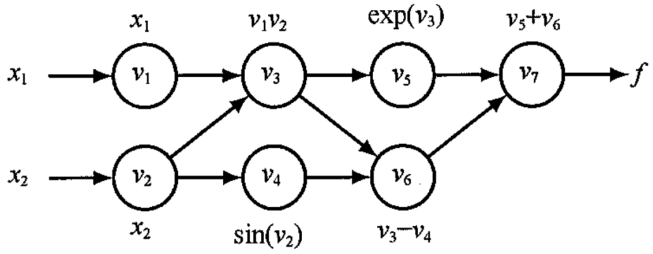


图 8.4: 利用(8.47) ~ (8.48)对函数(8.46)进行数值计算的求值轨迹的可视化表示

$$\begin{aligned}
\dot{v}_4 &= \dot{v}_2 \cos(v_2), \\
\dot{v}_5 &= \dot{v}_3 \exp(v_3), \\
\dot{v}_6 &= \dot{v}_3 - \dot{v}_4, \\
\dot{v}_7 &= \dot{v}_5 + \dot{v}_6.
\end{aligned} \tag{8.51}$$

我们可以将本例的自动微分总结如下. 首先, 我们编写代码来对(8.47) ~ (8.48)给出的原始变量进行求值. 用于计算正切变量 [(8.50) ~ (8.51)] 的相关方程和代码会自动生成. 为了求导数 $\partial f/\partial x_1$, 我们需要输入 x_1 和 x_2 的特定值, 然后代码会执行原始方程和正切方程, 依次对元组 $(v_i, \dot{v}_i)$ 进行数值计算, 直至得到所需的导数 $\dot{v}_7$(见练习 8.17).

注

(重构新增) 之所以最终取的是 $\dot{v}_7$, 是因为按(8.47)~(8.48)的定义 $f = v_7$, 故 $\partial f/\partial x_1 = \partial v_7/\partial x_1 = \dot{v}_7$; 中间量 $\dot{v}_5$ 只是 $\exp(v_3)$ 对 x_1 的导数, 并不是整个函数的导数. 原书此处印作 $\dot{v}_5$, 从求值轨迹的构造看应为 $\dot{v}_7$. 另外要注意种子 $\dot{v}_2 = 0$ 的来历: 本例求的是对 x_1 的导数,x_2 与 x_1 无关, 故其对 x_1 的导数为零; 若要改求 $\partial f/\partial x_2$, 只需把种子换成 $\dot{v}_1 = 0$、$\dot{v}_2 = 1$ 再跑一遍, 这正是前向模式“一次一个方向”的代价.

考虑一个有两个输出 $f_1(x_1, x_2)$ 和 $f_2(x_1, x_2)$ 的示例, 其中 $f_1(x_1, x_2)$ 由(8.46)定义, 且

$$f_2(x_1, x_2) = (x_1 x_2 - \sin(x_2)) \exp(x_1 x_2). \tag{8.52}$$

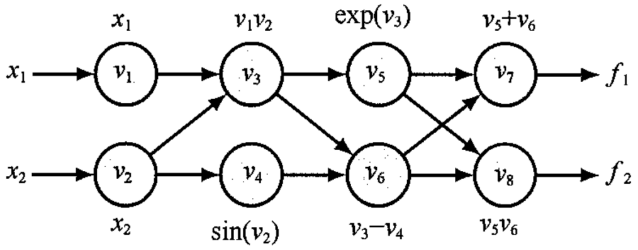


图 8.5: 将图 8.4中的示例扩展为具有两个输出 f_1 和 f_2

从图 8.5中可以看出, 这只涉及对原始变量和正切变量求值方程的一个小扩展, 因此 $\partial f_1/\partial x_i$ 和 $\partial f_2/\partial x_i$ 都可以在一次前向传播中一并求出. 不过, 这样做的缺点是, 如果我们想计算相对于不同输入变量 x_2 的导数, 就必须进行一次单独的前向传播.

一般来说, 如果一个函数有 D 个输入和 K 个输出, 那么一次前向模式自动微分就会产生 $K \times D$ 大小的 Jacobi 矩阵中的一列:

$$\mathbf{J} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_D} \\ \vdots & & \vdots \\ \frac{\partial f_K}{\partial x_1} & \cdots & \frac{\partial f_K}{\partial x_D} \end{bmatrix}. \tag{8.53}$$

若要计算 Jacobi 矩阵的第 j 列, 则需要对正切变量求值方程的前向传播进行初始化, 也就是在 $i \neq j$ 时设置

$\dot{x}_j = 1$ 和 $\dot{x}_i = 0$. 我们可以将其写成向量形式, 即 $\dot{\mathbf{x}} = \mathbf{e}_j$, 其中 $\mathbf{e}_j$ 是第 j 个单位向量. 若要计算全 Jacobi 矩阵, 则需要进行 D 次前向传播. 然而, 若要计算 Jacobi 矩阵与向量 $\mathbf{r} = (r_1, \cdots, r_D)^{\mathsf{T}}$ 的乘积:

$$\mathbf{Jr} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_D} \\ \vdots & & \vdots \\ \frac{\partial f_K}{\partial x_1} & \cdots & \frac{\partial f_K}{\partial x_D} \end{bmatrix} \begin{bmatrix} r_1 \\ \vdots \\ r_D \end{bmatrix}, \tag{8.54}$$

那么只需要设置 $\dot{\mathbf{x}} = \mathbf{r}$, 就可以通过一次前向传播来完成 (见练习 8.18).

我们可以看到, 前向模式自动微分可以通过 D 次前向传播, 求出导数的 $K \times D$ 大小的全 Jacobi 矩阵. 对于具有少量输入和较多输出 ($K \gg D$) 的网络, 这种方法非常有效. 然而, 我们经常在这样的环境中进行操作: 我们通常只有一个函数, 即用于训练的误差函数, 以及大量我们想要对其进行微分的变量 (包括网络中的权重和偏置), 这些变量可能有数百万个或数十亿个. 在这种情况下, 前向模式自动微分具有极低的效率. 因此, 我们转而采用另一种自动微分法——逆模式自动微分.

8.2.2 逆模式自动微分

逆模式自动微分可以看作误差反向传播程序的拓展.

定义 8.11 (逆模式自动微分)

逆模式自动微分的做法与前向模式相反: 先完成一次前向传播并把求值轨迹上的全部中间变量 v_j 保存下来, 再从输出端开始沿轨迹反向地、逐个节点地计算函数对各中间变量的偏导数, 一次传播即得到函数对全部中间变量 (因而对全部输入与参数) 的导数. 这种自动微分形式称为**逆模式自动微分** (reverse-mode automatic differentiation), 也称反向模式.

♣

注

(重构新增) 逆模式之所以适合训练神经网络, 是因为它把“代价”放在了输出个数而不是输入个数上: 误差函数只有一个输出 (标量), 却有数以百万计的参数, 于是逆模式只需一次反向传播就能拿到全部参数的梯度, 而前向模式要按输入维数传播同样多次. 这一优势的代价是内存: 反向阶段用到的中间变量必须在正向阶段全部保留, 而前向模式算完一个就可以丢掉. 定义中“先保存再反向”的顺序不可颠倒, 因为 $\bar{v}_i$ 的递推(8.56)需要用到正向阶段算出的 v_j; 若轨迹依赖于运行时的分支, 反向时还必须重放同一分支, 否则会用到不存在的中间量. 与前向模式相同, 它只要求各初等运算可微, 因而同样能处理分支、循环与函数调用.

和前向模式自动微分一样, 我们也在每个中间变量 v_j 中添加附加变量; 附加变量在此处称为**伴随** (adjoint) 变量, 用 $\bar{v}_j$ 表示. 考虑具有单一输出函数 f 的情况, 其伴随变量定义如下.

定义 8.12 (伴随变量)

设函数 f 由求值轨迹上的原始变量 v_j 复合而成, 则称

$$\bar{v}_j \triangleq \frac{\partial f}{\partial v_j} \tag{8.55}$$

为原始变量 v_j 的**伴随变量** (adjoint variable).

♣

注

(重构新增) 伴随变量与前向模式的正切变量恰好“对偶”: 正切变量固定一个输入、对每个中间变量求“它对该输入”的导数, 伴随变量固定唯一的输出、对每个中间变量求“输出对它的导数”. 二者都定义在求值轨迹的全部节点上, 都通过链式法则递推, 区别只在递推的方向与固定的对象. 定义要求 f 是单一输出, 这并不是技术限制, 而是为了让“伴随”有唯一含义: 若有多个输出, 就必须对每个输出各定义一套伴随变量, 这也正是后文说“多个输出需要多次反向传播”的原因. 需要注意,$\bar{v}_j$ 与 δ_j 是同一思想在两套记号下的表述: 把 f 取为误差函数、把 v_j 取为预激活 a_j,(8.55)就是(8.8). 伴随变量还有一个有用的读法: 它度量“若在节点 j 处把 v_j 抬高一个单位, 输出

会增加多少”, 因此它天然是灵敏度系数.

可以利用微积分中的链式法则, 从输出开始, 依次反向地对它们进行求值:

$$\bar{v}_i = \frac{\partial f}{\partial v_i} = \sum_{j\in \mathrm{ch}(i)} \frac{\partial f}{\partial v_j}\frac{\partial v_j}{\partial v_i} = \sum_{j\in \mathrm{ch}(i)} \bar{v}_j \frac{\partial v_j}{\partial v_i}, \tag{8.56}$$

其中 ch(i) 表示求值轨迹中节点 i 的子节点的集合, 即从节点 i 指向这些子节点的变量的集合. 如前所述, 对伴随变量的连续求值代表了通过求值轨迹进行的反向信息流动 (参见图 8.1).

再次考虑(8.47) ~ (8.48)给出的具体函数示例, 可以得出以下用于计算伴随变量的方程 (见练习 8.16):

$$\bar{v}_7 = 1, \tag{8.57}$$

$$\begin{aligned}
\bar{v}_6 &= \bar{v}_7,\\
\bar{v}_5 &= \bar{v}_7,\\
\bar{v}_4 &= -\bar{v}_6,\\
\bar{v}_3 &= \bar{v}_5 v_5 + \bar{v}_6,\\
\bar{v}_2 &= \bar{v}_3 v_1 + \bar{v}_4 \cos(v_2),\\
\bar{v}_1 &= \bar{v}_3 v_2.
\end{aligned} \tag{8.58}$$

请注意, 它们从输出端开始, 然后通过求值轨迹反向流至输入端. 即使有多个输入, 也只需要一次反向传播就能求出导数. 对于神经网络误差函数,E 关于权重和偏置的导数可作为相应的伴随变量. 但是, 如果出现一个以上的输出变量, 则需要为每个输出变量进行一次单独的反向传播 (参见图 8.5).

逆模式通常比前向模式占用更多内存, 因为必须存储所有中间变量, 以便在反向传播过程中根据需要使用中间变量计算伴随变量. 与此相反, 在前向模式下, 原始变量和正切变量在前向传播过程中被一起计算, 所以变量一旦用完就可以丢弃. 与逆模式相比, 前向模式一般更容易实现.

命题 8.4 (前向模式与逆模式的代价比较)

设函数有 D 个输入和 K 个输出, 其求值轨迹含 N_{op} 个初等运算.

(i) 前向模式的一次传播给出 Jacobi 矩阵的一列; 求全 Jacobi 矩阵需 D 次传播, 总代价与 D 成正比.

(ii) 逆模式的一次传播给出 Jacobi 矩阵的一行 (一个输出对全部输入的导数); 求全 Jacobi 矩阵需 K 次传播, 总代价与 K 成正比.

(iii) 逆模式必须把求值轨迹上的全部中间变量保留到反向阶段, 其额外内存与轨迹长度 N_{op} 同阶; 前向模式则在每个节点用过之后即可丢弃它的值, 额外内存只与当前节点有关.

(iv) 无论是前向模式还是逆模式, 通过网络的单次传播成本都不会超过单次函数计算成本的 6 倍. 实际上, 开销通常接近单次函数计算成本的 2 倍或 3 倍 (Griewank and Walther, 2008).

♠

证明

(重构新增)(i)(ii) 由(8.49), 前向模式的每一次传播对轨迹上每个节点 i 计算一次 $\dot{v}_i$, 其代价正比于 i 的父节点个数; 全部节点的父节点个数之和不超过轨迹中边的条数, 而每条边对应一次初等运算, 故一次传播为 $O(N_{\mathrm{op}})$. 由于一次传播只能携带一个种子方向, 得到 $K \times D$ 的 Jacobi 矩阵需要 D 个方向, 故总代价为 $O(DN_{\mathrm{op}})$. 逆模式同理: 由(8.56), 反向一次遍历的代价正比于子节点的个数, 合计仍为 $O(N_{\mathrm{op}})$; 一次反向传播只对一个输出给出全部导数, 故求全 Jacobi 矩阵需要 K 次. 两种模式的差别只在 D 与 K 谁更大, 这正是神经网络 ($K = 1$、$D = W$) 选择逆模式的原因.

(iii) 前向模式在计算 $\dot{v}_i$ 时只需要父节点的 $(v_j, \dot{v}_j)$, 算完即可覆盖, 故额外内存与轨迹的宽度同阶 (常数个变量); 逆模式要计算(8.56), 必须同时掌握全部 v_j, 故额外内存与轨迹的长度 N_{op} 同阶.

(iv) 每条初等运算的导数都可由该运算的输入输出值用常数次运算算出 (例如 exp 的导数是它自身的值, 乘法 $v_i v_j$ 的两个偏导数就是另一因子), 因此一次传播至多在每个节点上增加常数次运算; 把这些常数逐个累加, 得到的就是前向与反向各不超过函数求值的常数倍.Griewank 与 Walther 的经典分析给出的上界是 6 倍, 而实测的常

见值是 2 至 3 倍.

□

注

(重构新增) 这张“代价表”解释了深度学习框架为何一律默认逆模式, 也说明了前向模式并非无用: 当输出维数远大于输入维数 ($K \gg D$) 时, 前向模式反而更省. 选择模式的判据不是“哪个更先进”, 而是 D 与 K 的比值: 想拿 Jacobi 的一列用前向, 想拿一行用逆模式. 条件方面,(iii) 的结论依赖于“轨迹长度”这一概念: 循环被展开多少次, 轨迹就有多长, 逆模式的内存随之线性增长, 这也是长序列模型在训练时显存紧张的结构性原因. 最后,(iv) 给出的是运算次数的界, 不含访存开销; 逆模式的实际减速常常来自对中间变量的反复读取, 因此“不超过 6 倍”应理解为运算次数的上界, 而不是端到端墙钟时间的上界.

前向模式和逆模式的融合也很有趣: 它在计算黑塞矩阵与向量乘积的情况下很有用 (Pearlmutter, 1994). 在此, 我们可以使用逆模式来计算代码的梯度, 而代码本身是由前向模式生成的. 下面让我们从一个向量 $\mathbf{v}$ 和一个点 $\mathbf{x}$ 开始计算黑塞矩阵与向量的乘积. 通过设置 $\dot{\mathbf{x}} = \mathbf{v}$ 并使用前向模式, 可以得出方向导数 $\mathbf{v}^{\mathsf{T}}\nabla f$. 然后使用逆模式进行微分, 得到 $\nabla^2 f\,\mathbf{v} = \mathbf{H}\mathbf{v}$. 如果 W 是神经网络中参数的数量, 那么即使黑塞矩阵的大小为 $W \times W$, 计算复杂度也为 $O(W)$. 黑塞矩阵本身也可以通过自动微分法进行显式的计算, 但计算复杂度为 $O(W^2)$.

8.3 习题

- **练习 8.1** (★) 利用(8.5)、(8.6)、(8.8), 以及在定理 8.1的证明中出现的链式分解, 验证计算误差函数导数的反向传播公式 [(8.11)] .
- **练习 8.2** (★★) 采用一个多层网络, 并从前向传播公式 [(6.18)] 开始, 用矩阵符号重写反向传播公式 [(8.11)] . 请注意, 结果涉及转置矩阵.
- **练习 8.3** (★) 使用 Taylor 展开, 验证 $O(\varepsilon)$ 在(8.22)的右侧被抵消了.
- **练习 8.4** (★★) 考虑图 6.9所示的两层网络, 其中额外增加了与直接从输入到输出的跳层连接相对应的参数. 通过拓展小节 8.1.3所讨论的内容, 写出误差函数关于这些额外参数的导数方程.
- **练习 8.5** (★★★) 在小节 8.1.5中, 我们推导出了一种使用反向传播程序计算神经网络 Jacobi 矩阵的方法. 请推导出一种基于前向传播方程计算 Jacobi 矩阵的替代形式.
- **练习 8.6** (★★★) 考虑双层神经网络并定义以下数据:

$$\delta_k = \frac{\partial E_n}{\partial a_k}, \qquad M_{kk'} = \frac{\partial^2 E_n}{\partial a_k \partial a_{k'}}. \tag{8.59}$$

对于 (i) 两个权重都在第二层、(ii) 两个权重都在第一层、(iii) 每一层都有一个权重的元素, 推导出用 δ_k 和 $M_{kk'}$ 表示的黑塞矩阵的元素表达式.

- **练习 8.7** (★★★) 将练习 8.6中关于双层神经网络的精确黑塞矩阵的结果进行扩展, 使其包括直接从输入层到输出层的跳层连接.
- **练习 8.8** (★★) 使用平方和误差函数的神经网络黑塞矩阵的外积近似值由(8.37)给出. 将这一结果扩展到多个输出.
- **练习 8.9** (★★) 考虑如下形式的平方损失函数:

$$E(\mathbf{w}) = \frac{1}{2}\iint \{y(\mathbf{x}, \mathbf{w}) - t\}^2\, p(\mathbf{x}, t)\, \mathrm{d}\mathbf{x}\, \mathrm{d}t, \tag{8.60}$$

其中 $y(\mathbf{x}, \mathbf{w})$ 为参数化函数, 如神经网络.命题 4.10表明, 能使该误差最小化的函数 $y(\mathbf{x}, \mathbf{w})$ 是由给定 $\mathbf{x}$ 情况下 t 的条件期望给出的. 该结果可证明 E 相对于向量 $\mathbf{w}$ 的两个元素 w_r 和 w_s 的二阶导数为

$$\frac{\partial^2 E}{\partial w_r \partial w_s} = \int \frac{\partial y}{\partial w_r}\frac{\partial y}{\partial w_s} p(\mathbf{x})\, \mathrm{d}\mathbf{x}. \tag{8.61}$$

注意, 对于 $p(\mathbf{x})$ 的有限样本, 我们可以得到(8.37).

- **练习 8.10** (★★) 推导出具有单输出、逻辑斯谛 sigmoid 输出单元激活函数和交叉熵误差函数的网络的黑塞矩阵的外积近似表达式 [(8.38)] , 对应于采用平方误差函数时的结果 [(8.37)] .

✍ **练习 8.11** (★★) 推导出一个具有 K 个输出、softmax 输出单元激活函数和交叉熵误差函数的网络的黑塞矩阵的外积近似表达式, 对应于采用平方和误差函数时的结果.

✍ **练习 8.12** (★★★) 考虑矩阵等式

$$\left(\mathbf{M}+\mathbf{v}\mathbf{v}^{\mathrm{T}}\right)^{-1}=\mathbf{M}^{-1}-\frac{\left(\mathbf{M}^{-1}\mathbf{v}\right)\left(\mathbf{v}^{\mathrm{T}}\mathbf{M}^{-1}\right)}{1+\mathbf{v}^{\mathrm{T}}\mathbf{M}^{-1}\mathbf{v}}. \tag{8.62}$$

它只是 Woodbury 恒等式 [(A.7)] 的一个特例. 将(8.62)应用于黑塞矩阵的外积近似表达式 [(8.37)], 推导出一个公式, 该公式允许通过训练数据进行一次传播并用每个数据点更新黑塞矩阵, 从而计算出黑塞矩阵的逆. 需要注意的是, 该算法可以采用 $\mathbf{H}=\alpha\mathbf{I}$ 进行初始化, 其中 α 是一个小的常数, 而且结果对 α 的精确值并不特别敏感.

✍ **练习 8.13** (★★) 验证(8.44)的导数由(8.45)给出.

✍ **练习 8.14** (★★) 逻辑斯蒂映射是由 $L_{n+1}(x)=4L_n(x)\left(1-L_n(x)\right)$ 和 $L_1(x)=x$ 的迭代关系定义的函数. 写出 $L_2(x)$、$L_3(x)$、$L_4(x)$ 的求值轨迹方程, 然后写出相应的导数 $L_1'(x)$、$L_2'(x)$、$L_3'(x)$、$L_4'(x)$ 的表达式. 请勿简化这些表达式, 但需要注意导数公式的复杂度是如何比函数本身的表达式的复杂度增长得更快的.

✍ **练习 8.15** (★★) 从示例函数(8.46)的求值轨迹方程(8.47) ~ (8.48)开始, 利用(8.49)推导出前向模式正切变量求值方程(8.50) ~ (8.51).

✍ **练习 8.16** (★★) 从示例函数(8.46)的求值轨迹方程(8.47) ~ (8.48)开始, 并参考图 8.4, 利用(8.56)推导出逆模式伴随变量求值方程(8.57) ~ (8.58).

✍ **练习 8.17** (★★★) 考虑示例函数(8.46). 写出 $\partial f/\partial x_1$ 的表达式, 并在 $x_1=1$ 和 $x_2=2$ 时对该函数进行求值. 使用求值轨迹方程(8.47) ~ (8.48)求出变量 $v_1\sim v_7$, 然后使用前向模式自动微分的求值轨迹方程求出正切变量 $\dot{v}_1\sim\dot{v}_7$, 并确认求得的 $\partial f/\partial x_1$ 值与直接求得的 $\partial f/\partial x_1$ 值一致. 同样, 使用逆模式自动微分的求值轨迹方程求出伴随变量 $\bar{v}_1\sim\bar{v}_7$, 并再次确认求得的 $\partial f/\partial x_1$ 值与直接求得的 $\partial f/\partial x_1$ 值一致.

✍ **练习 8.18** (★★) 将任意向量 $\mathbf{r}=(r_1,\cdots,r_D)^{\mathrm{T}}$ 表示为单位向量 $\mathbf{e}_i$ 的线性组合, 其中 $i=1,\cdots,D$. 证明可以通过设置 $\dot{\mathbf{x}}=\mathbf{r}$, 只需要通过单次前向模式自动微分传播就可以求出形式为(8.54)的函数与 $\mathbf{r}$ 的 Jacobi 矩阵的乘积.

第 9 章 正则化

在讨论多项式曲线拟合时, 我们引入了正则化的概念, 其通过防止模型参数取值过大来减少过拟合 (参见小节 1.2.5). 这涉及向误差函数中添加一个简单的惩罚项, 从而得到正则化的误差函数, 形式如下:

$$\widetilde{E}(\boldsymbol{w}) = E(\boldsymbol{w}) + \frac{\lambda}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w}, \tag{9.1}$$

其中 $\boldsymbol{w}$ 是模型参数向量,$E(\boldsymbol{w})$ 是未正则化的误差函数, 正则化超参数 λ 控制正则化效应的强度. 在以增加一些偏差为代价的情况下减小解的方差可以改善预测准确性, 这可以从偏差–方差权衡的角度来解释 (参见节 4.3). 在本章中, 我们将深入探讨正则化, 并讨论几种不同的正则化方法. 我们还将看到, 能从有限训练数据集中获得良好泛化能力的偏差所起的重要作用.

定义 9.1 (正则化误差函数)

设 $E(\boldsymbol{w})$ 是模型参数 $\boldsymbol{w} \in \mathbb{R}^M$ 的未正则化误差函数,$\lambda \geqslant 0$ 是正则化超参数. 称由(9.1)定义的 $\widetilde{E}(\boldsymbol{w})$ 为**正则化误差函数** (regularized error function), 称 $\frac{\lambda}{2}\|\boldsymbol{w}\|^2$ 为**正则化项** (regularization term) 或**惩罚项** (penalty term),λ 为**正则化系数** (regularization coefficient). 当 $\lambda = 0$ 时正则化误差函数退化为原来的误差函数 $E(\boldsymbol{w})$. ♣

注

(重构新增)定义 9.1把“防止参数过大”这一直观要求写成可优化的目标: 在原有误差 E 上加一项只依赖于参数大小的惩罚. 因子 1/2 只是为了让求导后的系数为 1(见eq. (9.5)), 不影响最优解的位置.λ 的作用是双重的:$\lambda \to 0$ 时惩罚消失, 模型退化为最大似然/最小二乘解, 方差大而偏差小;$\lambda \to \infty$ 时最优解被压向 $\boldsymbol{w} = \boldsymbol{0}$, 偏差大而方差小. 因此 λ 并不是“越大约好”或“越小越好”的固定选择, 而是控制有效模型复杂度的连续旋钮, 需要由验证集或 Bayes 证据确定. 二次惩罚之所以常用, 一是它可微且导数线性, 二是它等价于零均值 Gauss 先验的负对数 (见命题 9.1), 从而把“偏好小参数”这一归纳偏置纳入概率框架.

在实际应用中, 产生数据的过程不太能够精确对应于特定的神经网络架构, 因此任何给定的神经网络都只是对真实数据生成器的一种近似. 更大的网络可以提供更接近的近似, 但这会带来过拟合的风险. 实践中我们发现, 最佳的泛化结果几乎总是通过使用较大的网络结合某种形式的正则化来获得的. 在本章中, 我们将探索几种可替代的正则化技术, 包括早停法、模型平均、dropout、数据增强和参数共享. 如果需要, 也可以结合使用多种正则化技术. 例如, 正则化的误差函数 [(9.1)] 通常可以与 dropout 一起使用.

9.1 归纳偏置

在比较正弦合成数据问题中各阶多项式的预测误差时 (参见小节 1.2.4), 我们发现使用中等复杂度的多项式能够获得最小的泛化误差, 既不过于简单也不过于灵活. 当使用形如(9.1)的正则化项来控制模型复杂度时, 我们也发现了类似的结果: 当正则化系数 λ 大小中等时, 我们实现了对新输入值的最佳预测 (参见小节 1.2.5).

对这一结果的洞察来自偏差–方差分解. 我们看到, 模型中适当的偏差水平对于从有限数据集中学习泛化能力是比较重要的 (参见节 4.3). 具有高偏差的简单模型无法捕捉到潜在数据生成过程中的变化, 而具有低偏差的高度灵活模型则容易过拟合, 导致泛化能力差. 随着数据集大小的增加, 我们可以使用具有更小偏差的更灵活的模型, 而不会引起过大的方差, 从而改善泛化能力. 请注意, 在实际设置中, 模型的选取也可能受到内存使用或执行速度等因素的影响. 在此我们忽略这些附加因素, 而专注于实现良好的预测性能, 也就是良好的泛化能力.

9.1.1 逆问题

模型选择问题是机器学习的核心, 可以追溯到大多数机器学习任务都是**逆问题** (inverse problem) 这一事实. 给定条件分布 $p(t \mid \boldsymbol{x})$ 以及一组有限的输入点 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 原则上至少可以从该分布中抽样相应的值 $\{t_1, \cdots, t_N\}$. 然而, 在机器学习中, 我们必须解决这个问题的逆问题, 即只给定有限数量的样本就推断整个分布. 这本质上是不

适定的 (ill-posed), 因为有无限多的分布可能生成我们所观测到的训练数据. 实际上, 任何在观测目标值上具有非零概率密度的分布都是候选分布.

定义 9.2 (逆问题与不适定性)

设条件分布 $p(t \mid \boldsymbol{x})$ 生成有限样本 $\{(\boldsymbol{x}_n, t_n)\}_{n=1}^N$. 由有限样本推断 $p(t \mid \boldsymbol{x})$(或更一般地, 由观测结果反推生成机制、由输出反推输入) 的任务称为**逆问题** (inverse problem). 若该问题的解不唯一, 或解不连续依赖于观测数据 (观测的微小扰动可导致解的剧烈变化), 则称它是**不适定的** (ill-posed). ♣

注

(重构新增)定义 9.2中的两个“不”对应 Hadamard 意义下适定性的三条要求 (解存在、唯一、稳定) 中后两条的失效. 这里的困难不是数值算法不够好, 而是问题本身的信息不足: 给定 N 个样本点, 任何在这些点上与观测目标值吻合、而在其余点任意取值的分布都与数据相容, 候选集合因此是无限维的. 正因如此, 学习算法必须额外引入选择准则; 这个准则就是下一小节定义的归纳偏置. 反过来, 适定的正问题 (由分布生成样本) 在计算上也可能很难, 但那属于采样与数值计算问题, 与这里讨论的“解不唯一”不是一回事.

然而, 为了使机器学习有实用价值, 我们需要对新的 $\boldsymbol{x}$ 值进行预测, 因此需要一种方法来从无限多可能分布中选择一个特定的分布. 选择这个特定分布而非其他分布的偏好称为**归纳偏置** (inductive bias) 或**先验知识** (prior knowledge), 其在机器学习中起着核心作用.

定义 9.3 (归纳偏置)

在从与训练数据相容的无限多个候选分布 (或候选函数) 中选择一个特定分布 (或函数) 时, 所采用的、不依赖于具体训练样本的偏好, 称为**归纳偏置** (inductive bias). 当这种偏好以关于问题领域的背景知识的形式给出时, 也称为**先验知识** (prior knowledge). ♣

注

(重构新增)定义 9.3强调“不依赖于具体训练样本”: 归纳偏置是学习算法在见到数据之前就带有的选择倾向, 而不是从数据中估计出来的量. 它可以是显式的 (如(9.1)的二次惩罚、定义 9.5所刻画的对称性约束), 也可以是隐式的 (如网络层数、卷积的局部连接、参数初始化). 没有归纳偏置就无法从有限数据中泛化: 训练集上完全拟合的函数有无穷多个, 算法必须靠某种偏好挑出一个. 但偏置也不是越多越好——若偏置与真实数据生成过程不一致, 再多的数据也无法弥补系统性偏差. 下一小节的无免费午餐定理正是这一点的极端表述.

先验知识可能来自有助于约束解空间的背景信息. 对于许多应用, 输入值的微小变化应该导致输出值的微小变化, 因此应该使解偏向于那些具有平滑变化函数的解决方案. 形如(9.1)的正则化项鼓励模型权重取值较小, 因此引入了对输入变化较慢的函数的偏好. 同样, 当我们在图像中检测对象时, 可以利用对象本体通常与对象在图像中的位置无关的先验知识 (参见第 10 章). 这称为**平移不变性** (translation invariance), 将其纳入我们的解可以显著简化构建具有良好泛化能力的系统的任务. 然而, 必须注意不要引入与数据生成过程不一致的偏差或约束. 例如, 当输出与输入之间实际存在显著的非线性关系时, 如果假设它们的关系是线性的, 则可能导致得到一个产生不准确答案的系统.

像迁移学习和多任务学习这样的技术也可以从正则化的角度来看 (参见小节 6.3.4). 当某个特定任务的训练数据有限时, 可以使用来自不同但相关任务的额外数据来帮助确定神经网络中的可学习参数. 任务之间的相似性假设代表了一种比简单正则化更复杂的归纳偏置形式, 这解释了使用额外数据所带来的性能提升.

9.1.2 无免费午餐定理

本书的核心关注点是一类重要的机器学习模型——深度神经网络. 这些模型高度灵活, 并已经在计算机视觉、语音识别和自然语言处理等多个领域引发了革命. 事实上, 它们已成为大多数机器学习应用的首选框架. 因此, 它们看起来像是能完成所有任务的“通用”学习算法. 然而, 即便是非常灵活的神经网络也包含重要的归纳偏置 (参见第 10 章). 例如, 卷积神经网络编码了特定形式的归纳偏置, 包括对图像应用特别有用的平移不变性.

无免费午餐定理 (no free lunch theorem)(Wolpert,1996) 源自“天下没有免费的午餐”这一说法, 其表明在所有可能的问题上, 平均来看每个学习算法与其他学习算法一样好. 如果某个特定模型或算法在某些问题上优于平均水平, 那么它在其他问题上必定低于平均水平. 然而, 这更多是一个理论概念, 因为这里所说的可能问题空间, 包括了在任何合理的实际应用中都非常不典型的输入与输出之间的关系. 例如, 我们已经注意到, 大多数具有实践背景的例子显示出了某种程度的平滑性, 其中输入值的微小变化大多与目标值的微小变化相关联. 像神经网络这样的模型, 以及大多数广泛使用的机器学习技术, 由于表现出了这种形式的归纳偏置, 因此在某种程度上具有广泛的适用性.

定理 9.1 (无免费午餐定理)

设输入空间 $\mathcal{X}$ 与输出空间 $\mathcal{Y}$ 均为有限集, 目标函数 $f:\mathcal{X}\to\mathcal{Y}$ 在全体 $|\mathcal{Y}|^{|\mathcal{X}|}$ 个函数上等可能地选取. 给定训练集 $S=\{(\boldsymbol{x}_n,f(\boldsymbol{x}_n))\}_{n=1}^N$, 任一学习算法根据 S 输出假设 $h_S:\mathcal{X}\to\mathcal{Y}$. 则对任意未在训练集中出现的输入 $x_0\notin\{x_1,\cdots,x_N\}$, 该算法在 x_0 上的期望 0–1 误差为

$$\mathbb{E}_f\left[\mathbf{1}\{h_S(x_0)\neq f(x_0)\}\right]=\frac{|\mathcal{Y}|-1}{|\mathcal{Y}|},$$

它与学习算法无关. ♡

证明

(重构新增) 固定训练输入 $\{x_1,\cdots,x_N\}$ 与 $x_0\notin\{x_1,\cdots,x_N\}$. 对任意与 S 相容的目标函数 f, 算法输出同一个假设 h_S, 因为算法的输入只有 S. 在全体目标函数中, 与 S 相容的函数共有 $|\mathcal{Y}|^{|\mathcal{X}|-N}$ 个: 它们在 N 个训练点上的取值已被固定, 其余 $|\mathcal{X}|-N$ 个点可任意取值. 其中在 x_0 处取值不等于 $h_S(x_0)$ 的函数有 $(|\mathcal{Y}|-1)|\mathcal{Y}|^{|\mathcal{X}|-N-1}$ 个:x_0 处有 $|\mathcal{Y}|-1$ 种“错误”选择, 其余 $|\mathcal{X}|-N-1$ 个点任意. 由于 f 等可能, 所求期望误差等于上述两类函数个数之比

$$\frac{(|\mathcal{Y}|-1)|\mathcal{Y}|^{|\mathcal{X}|-N-1}}{|\mathcal{Y}|^{|\mathcal{X}|-N}}=\frac{|\mathcal{Y}|-1}{|\mathcal{Y}|}.$$

该比值只依赖于 $|\mathcal{Y}|$, 与 h_S 无关, 而 h_S 是任一算法由 S 产生的, 故期望误差与算法无关. 若算法本身带随机性, 则对随机性再取一次期望, 上式仍成立.

□

注

(重构新增)定理 9.1把“平均来看每个算法一样好”精确化为: 在目标函数等可能的先验下, 任何算法对未见点的期望误差都等于随机猜测的误差 $(|\mathcal{Y}|-1)/|\mathcal{Y}|$. 证明的关键是“算法只看到 S”这一信息约束——在 x_0 处,f 的取值与 S 独立, 算法无法提供任何信息. 必须注意这个结论的适用范围: 它依赖“全体函数等可能”这一极强且不现实的先验, 而真实任务的目标函数高度集中 (平滑、具有层次结构、满足对称性等). 因此它并不否定某个算法在特定任务上更好, 而是说明“脱离任务分布的普遍最优算法”不存在; 一个算法在某一类任务上的优势, 必然以在另一类任务上的劣势为代价. 这也解释了为什么引入与任务一致的归纳偏置 (如卷积的平移等变性) 能带来实际收益.

尽管无免费午餐定理在某种程度上是理论性的, 但它确实突出了偏差在确定机器学习算法性能方面的重要性. 在没有任何偏差的情况下, 不可能“纯粹从数据中”学习. 在实践中, 偏差可能是隐式的. 例如, 每个神经网络都有有限数量的参数, 这限制了它所能表示的函数. 然而, 偏差也可以作为解决特定问题的先验知识被显式编码.

在寻找通用学习算法的过程中, 我们实际上是在寻找适合的归纳偏置, 它广泛适用于我们在实践中遇到的不同应用. 然而, 对于任何给定的应用, 如果能够融入特定于该应用的更强的归纳偏置, 则可以获得更好的结果. 基于模型的机器学习的观点 (Winn et al.,2023) 主张使机器学习模型中的所有假设显式化, 以便为归纳偏置做出适当的选择.

我们已经看到, 可以通过分布的形式将归纳偏置融入其中, 例如通过指定输出是一组基函数固定的线性函数. 也可以在训练期间, 通过在使用的误差函数中添加正则化项来融入归纳偏置. 我们可以通过训练过程本身来控制神经网络的复杂性 (参见第 7 章). 你将看到, 即使可调参数的数量超过训练数据点的数量, 只要训练过程设置得当, 深度神经网络也可以实现良好的泛化. 将深度学习应用于现实世界问题的部分技巧在于归纳偏置的仔细设计和先验知识的融入.

9.1.3 对称性和不变性

在机器学习的许多应用中, 预测应该在输入变量经过一种或多种变换时保持不变, 称为具有**不变性** (invariance). 例如, 在对二维图像中的对象进行分类时, 如猫或狗, 无论对象在图像中的位置如何, 都应赋予对象相同的分类, 称为**平移不变性** (translation invariance). 同样, 图像中对象大小的变化也不应改变对象的分类, 称为**尺度不变性** (scale invariance). 利用这种对称性来创建归纳偏置可以大大提高机器学习模型的性能, 这是**几何深度学习** (geometric deep learning)(Bronstein et al.,2021) 所要研究的主题.

定义 9.4 (不变性与对称性)

设 $\mathcal{T}$ 是一族作用在输入空间上的变换,C 是将输入 I 映射为预测 $C(I)$ 的预测器. 若对每个 $T \in \mathcal{T}$ 与每个输入 I 都有 $C(T(I)) = C(I)$, 则称 C 关于 $\mathcal{T}$ 具有**不变性** (invariance). 当 $\mathcal{T}$ 为平移变换族时, 称相应的不变性为**平移不变性** (translation invariance); 当 $\mathcal{T}$ 为缩放变换族时, 称为**尺度不变性** (scale invariance). 若一族变换保持某个属性不变, 则称该属性具有相应的**对称性** (symmetry).

♣

注

(重构新增)定义 9.4把“对称性”从几何直观转成了对预测器的函数方程约束: 对变换族中的每个 T,$C \circ T = C$. 这一约束越强, 候选函数空间越小, 所需样本越少; 但若真实任务并不具有该对称性 (例如预测图像中对象的位置, 而不是类别), 强行施加不变性会丢失关键信息. 平移不变性适合分类, 却不适合定位与分割, 这正是后文定义 9.6要把“不变”推广为“按已知方式变换”的动机. 还要注意, 不变性是相对于给定的变换族而言的, 脱离变换族谈“某个模型具有不变性”没有意义.

平移或缩放等变换保持特定属性不变的现象称为对称性. 所有对应于特定对称性 (如平移或缩放) 的变换的集合构成了一个称为**群** (group) 的数学结构. 群由一组元素 $A, B, C, \cdots$ 以及一个用于组合元素对的二元运算 (使用符号 $A \circ B$ 表示) 组成. 以下 4 个公理对于群是成立的.

定义 9.5 (群)

设 G 是非空集合,$\circ : G \times G \to G$ 是定义在 G 上的二元运算. 若下列四条公理成立, 则称 $(G, \circ)$ 为**群** (group):

1. **封闭性**: 对于集合中的任何两个元素 A 和 B,$A \circ B$ 也必须在集合中;
2. **结合性**: 对于集合中的任何三个元素 A、B 和 C, 有 $(A \circ B) \circ C = A \circ (B \circ C)$;
3. **单位元**: 集合中存在一个称为单位元的元素 I, 满足对于集合中的每个元素 A, 有 $A \circ I = I \circ A = A$;
4. **逆元**: 对于集合中的每个元素 A, 存在集合中的另一个元素, 用 A^{-1} 表示, 称为逆元, 满足 $A \circ A^{-1} = A^{-1} \circ A = I$.

♣

注

(重构新增)定义 9.5的作用是把“变换可以复合、可以撤销、可以什么都不做”这三件直观事情公理化. 封闭性保证复合仍是允许的变换; 结合性保证多步复合不必加括号; 单位元对应恒等变换; 逆元保证每个变换可撤销, 从而变换族不会把不同的输入永久粘在一起. 四条公理中, 封闭性与逆元是最实质的: 只满足封闭性与结合性的结构是半群, 加上单位元是幺半群, 只有再加上逆元才成为群. 正因为有逆元,“在变换下不变”才能等价地写成“先变换再预测”与“先预测再逆变换”的一致性, 这也是定义 9.6能用 $S \circ T = \tilde{T} \circ S$ 表述的前提. 群只刻画变换的代数结构, 不涉及变换作用在具体数据上的连续性或可微性; 后文的卷积网络利用的是平移群的离散版本.

简单的群如以 90° 的倍数旋转正方形的变换集合, 或在二维平面上连续平移对象的变换集合, 都满足群公理 (见练习 9.1).

原则上, 神经网络对输入空间变换做出预测的不变性可以从数据中学习, 而无须对网络或训练程序进行特殊修改. 然而, 实际上这可能极其具有挑战性, 因为这样的变换可以在原始数据中产生显著改变. 例如, 图像中对象的相对较小的平移, 即使只有几个像素, 也可以导致像素值发生显著变化. 此外, 多种不变性通常需要同时保持, 例如同时要求对二维空间中的平移、缩放、旋转、强度变化、色彩平衡变化等保持不变. 这样的变换组合的数量是指数级的, 这使得学习所有这些不变性所需的训练集大得难以承受.

因此, 我们需要寻求更有效的方法来促使自适应模型展现出所需的不变性. 这些方法大致可以分为 4 类.

1. **预处理**: 通过在预处理阶段计算在所需变换下保持不变的数据特征来构建不变性. 任何后续使用这些特征作为输入的回归或分类系统必然也保持了这些不变性.
2. **正则化的误差函数**: 向误差函数中添加一个正则化项, 以惩罚输入仅受不变性变换改变时模型输出的变化.
3. **数据增强**: 通过使用所需的不变性对训练数据点进行变换, 从而扩展训练集, 同时保持它们的目标值与变换前相同.
4. **网络架构**: 通过适当选择网络架构, 将不变性融入神经网络的结构中.

上述第 1 类方法面临的一个挑战是设计呈现所需不变性的特征, 同时不丢弃可能有助于确定网络输出的信息. 我们已经看到, 固定的、手工提取的特征表示能力有限, 并且已被使用深度神经网络习得的表示所取代 (参见第 6 章).

第 2 类方法的一个例子是在训练过程中添加正则化项到误差函数中的**切线传播** (tangent propagation) 技术 (Simard et al.,1992), 旨在直接惩罚由某种不变性变换下输入变量的变化引起的输出变化. 除了训练的额外复杂性外, 这种技术的一个局限性是只能处理小的变换 (例如少于一个像素的平移).

第 3 类方法又称**数据集增强** (data set augmentation). 它通常相对容易实现, 并且在实践中被证明非常有效. 它通常应用在图像分析场景中, 因为创建变换的训练数据非常直接.图 9.1显示了对猫的图像应用这些变换的示例. 对于软组织的医疗图像, 数据增强也可以包含连续的“橡皮布”变形 (Ronneberger,Fischer,and Brox,2015).

图 9.1: 数据集增强示意图

对于序贯训练算法, 如随机梯度下降, 我们可以在输入模型之前变换每个数据点来增强数据集, 以便如果数据点被循环使用, 则每次都会应用不同的变换 (从适当的分布中抽取变换方式). 对于批训练方法, 通过多次复制每个数据点并独立变换每个副本, 可以达到类似的效果.

通过对原始示例做小改动的变换, 然后按变换幅度的幂次对误差函数进行 Taylor 展开, 可以分析使用增强数据的效果 (Bishop,1995c;Leen,1995;Bishop,2006), 并得到一个正则化的误差函数, 其中的正则化项惩罚了网络输出相对于输入变量在变换方向上投影的梯度. 这与前面讨论的切线传播技术相关. 添加随机噪声的变换是一种特殊情况, 在这种情况下, 正则化项惩罚网络输出相对于输入的导数 (见练习 9.2). 因为我们鼓励即便在输入变量添加噪声的情况下网络输出也保持不变, 所以该惩罚是非常合理的.

最后, 第 4 类方法, 也就是在网络结构中融入不变性, 已被证明非常强大且有效, 并且还带来了其他关键的好处. 我们将在计算机视觉中的卷积神经网络背景下详细讨论这类方法 (参见第 10 章).

9.1.4 等变性

不变性的一个重要推广称为**等变性** (equivariance). 在这种情况下, 当输入变换时, 网络的输出以特定的方式变换, 而不是保持不变. 例如, 考虑一个接收图像作为输入并返回图像分割结果的网络, 其中每个像素分类为属于前景对象或背景. 此时, 如果图像中对象的位置发生平移, 那么我们希望对象的相应分割结果也类似地发生平移. 假设我们用 I 表示图像, 用 S 表示分割网络的操作, 那么对于一个平移操作 T, 我们有

$$S(T(I)) = T(S(I)). \tag{9.2}$$

这表明被平移图像的分割结果是通过对原始图像的分割结果进行平移得到的.图 9.2说明了这种关系.

定义 9.6 (等变性)

设 S 是将输入 I 映射为输出 $S(I)$ 的算子,$\mathcal{T}$ 是作用在输入空间上的一族变换. 若对每个 $T \in \mathcal{T}$ 都存在作用在输出空间上的变换 $\tilde{T}$, 使得

$$S(T(I)) = \tilde{T}(S(I))$$

对所有 I 成立, 则称 S 关于 $\mathcal{T}$ 是**等变的** (equivariant),$\tilde{T}$ 称为 T 在输出空间的**诱导变换**. 当 $\tilde{T}$ 为恒等变换时, 等变性退化为不变性.

♣

注

(重构新增)定义 9.6比定义 9.4更精细: 它不要求输出“不动”, 而要求输出“按与输入相同 (或已知) 的方式动”. 这一区分对分割、检测、姿态估计等结构化输出任务是本质的——把输入图像平移几个像素, 分割掩码也应相应平移, 而不是保持不变. 等变性的强度取决于诱导变换 $\tilde{T}$: 若 $\tilde{T} = T$, 称为同变; 若 $\tilde{T}$ 是降采样网格上的平移, 则等变性只在离散网格上近似成立. 实现等变性的架构 (如卷积网络对平移) 把这一约束硬编码进模型, 因此无需从数据中学习变换的复合规律, 样本效率远高于让全连接网络自行发现. 但等变性是相对于给定变换群而言的: 旋转等变性需要旋转卷积或群等变网络, 普通卷积并不具备.

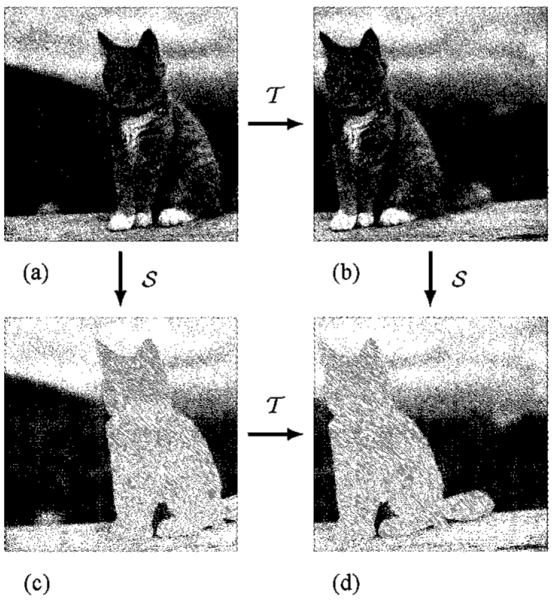


图 9.2: 对应(9.2)的等变性示意图. 如果图像 (a) 首先被平移得到图像 (b), 然后被分割得到图像 (d), 则结果与先将图像 (a) 分割得到图像 (c), 再平移得到图像 (d) 相同

更一般地, 当应用于输出的变换与应用于输入的变换不同时, 等变性也成立:

$$S(T(I)) = \tilde{T}(S(I)). \tag{9.3}$$

例如, 假设分割后的图像的分辨率低于原始图像,T 代表原始图像空间中的平移,$\tilde{T}$ 代表较低维度分割空间中相应

的平移. 同样, 如果 S 代表计算图像中对象方向的操作, 而 T 代表旋转 (这是图像中所有像素值的复杂非线性变换), 那么 $\tilde{T}$ 将依据 S 生成一个标量方向值增大或减小.

我们还看到, 不变性是等变性的一种特殊情况, 其中输出变换是简单的恒等变换. 例如, 如果 C 是一个神经网络, 用于分类图像中的一个对象, 而 T 是一个平移操作, 则有

$$C(T(I)) = C(I), \tag{9.4}$$

这表明对象的类别不依赖于该对象在图像中的位置.

9.2 权重衰减

我们在线性回归的背景下引入了正则化 (参见小节 1.2.5) 来控制模型的复杂性. 这是一种限制模型中参数数量的替代方法. 最简单的正则化项由模型参数的平方和构成, 从而形成(9.1)的正则化误差函数. 该误差函数会对较大的参数值施加惩罚. 有效的模型复杂性由正则化系数 λ 的选择决定.

我们还看到, 这种加性的正则化项可以解释为权重向量 $\boldsymbol{w}$ 上零均值 Gauss 先验分布的负对数 (参见节 2.3). 这为将先验知识纳入模型训练过程提供了概率性的视角. 遗憾的是, 这种先验是通过模型参数表示的, 而我们具有的关于待解决问题的领域知识更可能是通过从输入到输出的网络函数形式表示的. 然而, 参数与网络函数之间的关系极其复杂, 因此只有非常有限类型的先验知识可以直接轻松地表示为模型参数的先验.

命题 9.1 (权重衰减的 Gauss 先验解释)

设 $\boldsymbol{w} \in \mathbb{R}^M$ 的先验分布为零均值各向同性 Gauss 分布 $p(\boldsymbol{w}) = \mathcal{N}(\boldsymbol{w} \mid \boldsymbol{0}, \alpha^{-1}\boldsymbol{I})$, 其中 $\alpha > 0$ 为精度. 则负对数先验 (忽略与 $\boldsymbol{w}$ 无关的常数) 为

$$-\ln p(\boldsymbol{w}) = \frac{\alpha}{2}\|\boldsymbol{w}\|^2 + \text{常数}.$$

因此, 最小化(9.1)等价于在平方和误差下求最大后验 (MAP) 解, 且 $\lambda = \alpha$.

证明

(重构新增) 由多元 Gauss 密度的定义,

$$p(\boldsymbol{w}) = (2\pi)^{-M/2}\alpha^{M/2}\exp\left(-\frac{\alpha}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w}\right),$$

取负对数得

$$-\ln p(\boldsymbol{w}) = \frac{\alpha}{2}\boldsymbol{w}^{\mathsf{T}}\boldsymbol{w} - \frac{M}{2}\ln\alpha + \frac{M}{2}\ln(2\pi).$$

后两项与 $\boldsymbol{w}$ 无关, 记为常数. 于是

$$-\ln p(\boldsymbol{w} \mid \mathcal{D}) = -\ln p(\mathcal{D} \mid \boldsymbol{w}) - \ln p(\boldsymbol{w}) + \text{常数} = E(\boldsymbol{w}) + \frac{\alpha}{2}\|\boldsymbol{w}\|^2 + \text{常数},$$

其中 $E(\boldsymbol{w}) = -\ln p(\mathcal{D} \mid \boldsymbol{w})$ 是负对数似然 (对平方和误差即为其一半). 比较(9.1)即知 MAP 解与 $\lambda = \alpha$ 的正则化最小化解相同.

□

注

(重构新增)命题 9.1说明二次惩罚不是随意的技巧, 而是“参数先验为 Gauss 分布”这一假设的推论. 精度 α 与正则化系数 λ 一一对应:α 大 (先验窄) 等价于 λ 大 (强惩罚). 这也暴露了该先验的局限: 它假设各参数独立、同方差、均值为零, 而多层网络中不同层的权重尺度往往不同, 因此实际使用分组或逐层的正则化系数 (见定义 9.9). 此外, 把先验放在参数上而非函数上, 是 Bayes 神经网络的主要困难: 参数的对称性与缩放不变性会使先验在参数空间中“看起来”很窄, 但在函数空间中未必如此.

(9.1)中的平方和正则化项在机器学习文献中称为**权重衰减** (weight decay), 因为在序贯学习算法中, 它鼓励权重值向零衰减 (除非数据支持其不衰减)(参见练习 9.3). 这种正则化项的一个优点是, 它的导数很容易计算, 可用于梯度下降训练. 具体来说, 对于(9.1), 梯度由下式给出:

$$\nabla\widetilde{E}(\boldsymbol{w}) = \nabla E(\boldsymbol{w}) + \lambda\boldsymbol{w}. \tag{9.5}$$

可以看到,(9.1)中的因子 1/2 在取导数时会消失.

定义 9.7 (权重衰减)

称(9.1)中的二次正则化项 $\frac{\lambda}{2}\|\boldsymbol{w}\|^2$ 为**权重衰减** (weight decay), 称相应的正则化训练为**权重衰减正则化**. 其梯度由(9.5)给出, 即在原梯度上叠加 $\lambda\boldsymbol{w}$; 在梯度下降更新中它表现为与参数成正比的回复项 $-\eta\lambda\boldsymbol{w}$.

♣

注

(重构新增) "衰减" 一词来自连续时间极限: 若只保留正则化项并用梯度下降更新, 则 $\boldsymbol{w}^{(\tau+1)} = (1-\eta\lambda)\boldsymbol{w}^{(\tau)}$, 每一步都把参数按同一比例缩短, 以步数 τ 为时间,$\boldsymbol{w}^{(\tau)} = (1-\eta\lambda)^{\tau}\boldsymbol{w}^{(0)} \approx \boldsymbol{w}^{(0)}e^{-\eta\lambda\tau}$; 若以 $t = \eta\tau$ 为时间, 则化为 $\boldsymbol{w}(t) = \boldsymbol{w}(0)e^{-\lambda t}$(见练习 9.3). 因此 λ 的量纲是 "逆时间", 与学习率 η 一起决定衰减速度.定义 9.7只对参数的大小施加惩罚, 不含任何数据相关项; 真正决定某个参数是否被压向零的, 是数据项在该方向上的曲率与 λ 的相对大小——这正是下面定理 9.2要量化的.

通过考虑二维参数空间以及未正则化的误差函数 $E(\boldsymbol{w})$, 可以看到二次正则化项的一般效果, 如图 9.3所示. 该误差函数是 $\boldsymbol{w}$ 的二次函数 (对应于具有平方和误差函数的简单线性回归模型)(参见小节 4.1.3). 参数空间中的轴已经旋转以与 Hessian 矩阵的特征向量对齐, 对应于椭圆误差函数等值线的轴 (参见第 8 章).

定理 9.2 (L_2 正则化的闭式解)

设 $\boldsymbol{H} \in \mathbb{R}^{M\times M}$ 对称正定, 未正则化误差函数为

$$E(\boldsymbol{w}) = \frac{1}{2}(\boldsymbol{w}-\boldsymbol{w}^*)^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{w}-\boldsymbol{w}^*) + E_0,$$

其中 $\boldsymbol{w}^*$ 是 E 的唯一最小点. 则对 $\lambda > 0$, 正则化误差函数 $\widetilde{E}(\boldsymbol{w}) = E(\boldsymbol{w}) + \frac{\lambda}{2}\|\boldsymbol{w}\|^2$ 有唯一最小点

$$\boldsymbol{w}_\lambda = (\boldsymbol{H}+\lambda\boldsymbol{I})^{-1}\boldsymbol{H}\boldsymbol{w}^*.$$

若 $\{(\eta_j, \boldsymbol{u}_j)\}_{j=1}^{M}$ 是 $\boldsymbol{H}$ 的正交特征对, 即 $\boldsymbol{H}\boldsymbol{u}_j = \eta_j\boldsymbol{u}_j$、$\boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{u}_k = \delta_{jk}$, 并记 $w_j^* = \boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{w}^*$, 则

$$\boldsymbol{w}_\lambda = \sum_{j=1}^{M}\frac{\eta_j}{\eta_j+\lambda}w_j^*\boldsymbol{u}_j.$$

特别地, 若 $E(\boldsymbol{w}) = \frac{1}{2}\|\boldsymbol{X}\boldsymbol{w}-\boldsymbol{t}\|^2$ 且 $\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}$ 可逆, 则

$$\boldsymbol{w}_\lambda = (\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}+\lambda\boldsymbol{I})^{-1}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{t}.$$

♡

证明

(重构新增) 先证一般二次情形. 求梯度:

$$\nabla\widetilde{E}(\boldsymbol{w}) = \boldsymbol{H}(\boldsymbol{w}-\boldsymbol{w}^*) + \lambda\boldsymbol{w} = (\boldsymbol{H}+\lambda\boldsymbol{I})\boldsymbol{w} - \boldsymbol{H}\boldsymbol{w}^*.$$

令其为零得 $(\boldsymbol{H}+\lambda\boldsymbol{I})\boldsymbol{w} = \boldsymbol{H}\boldsymbol{w}^*$. 由于 $\boldsymbol{H}$ 正定、$\lambda > 0$,$\boldsymbol{H}+\lambda\boldsymbol{I}$ 正定从而可逆, 故驻点唯一, 为 $\boldsymbol{w}_\lambda = (\boldsymbol{H}+\lambda\boldsymbol{I})^{-1}\boldsymbol{H}\boldsymbol{w}^*$; 又 $\widetilde{E}$ 的 Hessian 为 $\boldsymbol{H}+\lambda\boldsymbol{I}$ 正定, 故该驻点是全局最小点.

再证特征基形式. 把 $\boldsymbol{w}^*$ 与 $\boldsymbol{w}_\lambda$ 都按正交特征基展开:$\boldsymbol{w}^* = \sum_j w_j^*\boldsymbol{u}_j$,$\boldsymbol{w}_\lambda = \sum_j w_j^\lambda\boldsymbol{u}_j$. 由 $\boldsymbol{H}\boldsymbol{u}_j = \eta_j\boldsymbol{u}_j$ 及特征向量的正交性, 方程 $(\boldsymbol{H}+\lambda\boldsymbol{I})\boldsymbol{w}_\lambda = \boldsymbol{H}\boldsymbol{w}^*$ 的第 j 个分量为 $(\eta_j+\lambda)w_j^\lambda = \eta_j w_j^*$, 即 $w_j^\lambda = \frac{\eta_j}{\eta_j+\lambda}w_j^*$. 代回展开即得. 由于 $\eta_j > 0$、$\lambda > 0$, 每个收缩因子 $\frac{\eta_j}{\eta_j+\lambda}$ 都严格介于 0 与 1 之间.

最后证线性回归特例. 此时 $E(\boldsymbol{w}) = \frac{1}{2}\|\boldsymbol{X}\boldsymbol{w}-\boldsymbol{t}\|^2$,$\nabla E = \boldsymbol{X}^{\mathsf{T}}(\boldsymbol{X}\boldsymbol{w}-\boldsymbol{t})$,Hessian 为 $\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}$. 在 $\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}$ 可逆时 $\boldsymbol{w}^* = (\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X})^{-1}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{t}$, 故 $\boldsymbol{H}\boldsymbol{w}^* = \boldsymbol{X}^{\mathsf{T}}\boldsymbol{t}$. 代入一般公式得 $\boldsymbol{w}_\lambda = (\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}+\lambda\boldsymbol{I})^{-1}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{t}$.

□

注

(重构新增)定理 9.2是本章后续讨论的定量基础, 包含三层信息. 第一, 闭式解只对二次误差成立: 神经网络对参数是非线性的, 一般没有闭式解, 只能用迭代法; 但局部二次近似仍可用该公式解释正则化的作用. 第二, 收缩因子 $\frac{\eta_j}{\eta_j+\lambda}$ 表明惩罚是“按曲率分配”的:Hessian 特征值 η_j 大 (数据对该方向约束强) 的方向几乎不被收缩,η_j 小 (数据几乎不约束) 的方向被强烈压向零. 这解释了图 9.3中最小值沿 w_1 方向移动较大、沿 w_2 方向移动较小. 第三, 定义有效参数数

$$\gamma(\lambda) \triangleq \sum_{j=1}^{M} \frac{\eta_j}{\eta_j+\lambda},$$

则 $\gamma(0)=M$、$\gamma(\lambda)\downarrow 0(\lambda\to\infty)$, 且 γ 关于 λ 严格单调递减. 这把“正则化控制模型复杂度”从定性说法变成了可计算的量:$\gamma(\lambda)$ 就是被数据“有效约束”的方向数. 注意 γ 依赖 $\boldsymbol{H}$ 的谱, 因而依赖数据与参数化方式; 它不是模型参数的个数, 也不能脱离数据集谈论.

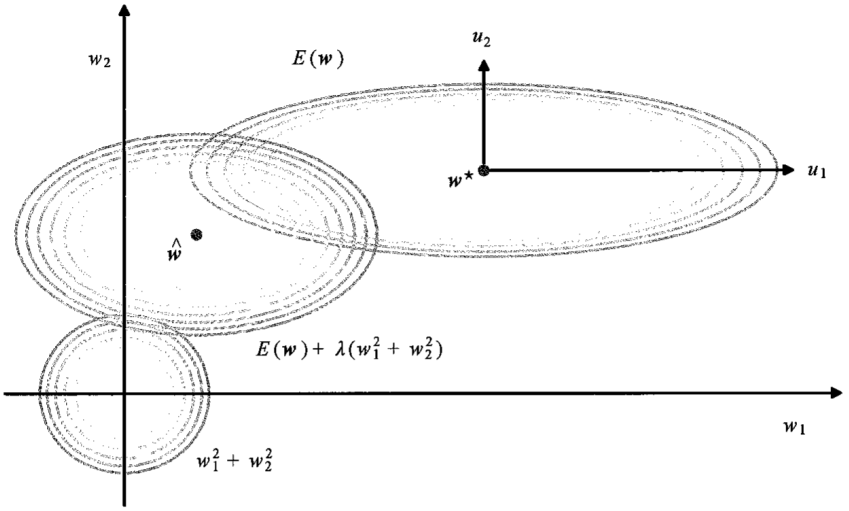


图 9.3: 二次误差函数与平方和正则化项 $\lambda(w_1^2+w_2^2)$ 的误差函数等值线 (红色)、正则化项等值线 (绿色), 以及两者的线性组合等值线 (蓝色). 这里参数空间中的轴已经旋转以与未正则化误差函数的椭圆等值线的轴对齐. 对于 $\lambda=0$, 最小误差由 $\boldsymbol{w}^*$ 表示. 当 $\lambda>0$ 时, 正则化误差函数 $E(\boldsymbol{w})+\lambda(w_1^2+w_2^2)$ 的最小值向原点移动. 这种移动在 w_1 方向上较大 (因为未正则化误差对参数值相对不敏感), 在 w_2 方向上较小, 这是误差与参数值高度相关导致的. 正则化项有效地抑制了对网络预测准确性影响较小的参数

可以看到, 正则化项的效果是缩小权重参数的大小. 然而, 正则化项对于参数 w_1 的效果更大, 因为未正则化的误差对 w_1 值的敏感性远低于对 w_2 值的敏感性. 直观上, 只有参数 w_2 是“活跃的”, 因为输出对 w_1 相对不敏感, 正则化项会将 w_1 推向 0. 有效参数的数量指的是正则化后仍然活跃的参数的数目, 这个概念可以从 Bayes 学派或频率学派的角度形式化 (Bishop,2006;Hastie,Tibshirani,and Friedman,2009). 对于 $\lambda\to\infty$, 所有参数都被推向 0, 有效参数的数量因此为 0. 随着 λ 的减小, 有效参数的数量开始增加, 直至 $\lambda=0$ 时, 有效参数的数量等于模型中实际参数的总数. 我们发现, 通过正则化控制模型复杂性与通过限制参数数量控制模型复杂性有相似之处.

9.2.1 一致性正则化项

简单权重衰减形式 [(9.1)] 的一个缺陷是, 它破坏了网络映射的某些我们想要的变换特性. 为了说明这一点, 考虑一个有两层权重和线性输出单元的多层感知机网络, 该网络从一组输入变量 $\{x_i\}$ 映射到一组输出变量 $\{y_k\}$. 第一隐藏层 (简称隐层) 的隐藏单元的激活由下式给出:

$$z_j = h\left(\sum_i w_{ji}x_i + w_{j0}\right), \tag{9.6}$$

而输出单元的激活由下式给出:

$$y_k = \sum_j w_{kj} z_j + w_{k0}. \tag{9.7}$$

假设我们对输入数据进行如下线性变换:

$$x_i \longrightarrow \tilde{x}_i = a x_i + b. \tag{9.8}$$

然后,我们可以通过对从输入到隐藏层单元的权重和偏置进行相应的线性变换,来保证网络执行的映射不变(见练习 9.4). 下面把这一要求形式化.

定义 9.8 (线性变换一致性)

设 $\mathcal{G}$ 是由输入、输出变量的可逆线性 (仿射) 变换诱导的一族参数变换: 对每个 $g \in \mathcal{G}$, 用 $g(\boldsymbol{w})$ 表示使网络所表示的输入—输出映射保持不变的参数. 若正则化项 Ω 满足

$$\Omega(g(\boldsymbol{w})) = \Omega(\boldsymbol{w}), \qquad g \in \mathcal{G},$$

则称 Ω 与这些线性变换**一致** (consistent); 否则称 Ω 破坏了相应的变换特性. ♣

注

(重构新增)定义 9.8的要求不是“网络输出不变”, 而是“正则化项在等价的参数表示上取相同的值”. 这两件事必须分开: 网络映射在参数变换下保持不变, 是模型结构本身的性质; 正则化项若不保持不变, 就会在两个等价的网络之间人为地偏爱一个. 由于参数化本身存在冗余 (把第一层权重乘以 a、偏置作相应平移, 可以表示同一个函数), 任何只依赖参数的先验都会对这种冗余敏感. 一致性正是要求正则化项对冗余“视而不见”. 这也解释了为什么简单权重衰减会失败: 它把权重和偏置一视同仁地惩罚, 而偏置的平移与权重的缩放并不是同一回事.

命题 9.2 (网络映射的线性变换不变性)

在由(9.6)与(9.7)定义的两层网络中, 若输入按(9.8)变换, 第一层参数按

$$w_{ji} \longrightarrow \tilde{w}_{ji} = \frac{1}{a} w_{ji}, \tag{9.9}$$

$$w_{j0} \longrightarrow \tilde{w}_{j0} = w_{j0} - \frac{b}{a} \sum_i w_{ji} \tag{9.10}$$

变换, 则隐层激活 z_j 保持不变. 进一步, 若输出按

$$y_k \longrightarrow \tilde{y}_k = c y_k + d \tag{9.11}$$

变换, 第二层参数按

$$w_{kj} \longrightarrow \tilde{w}_{kj} = c w_{kj}, \tag{9.12}$$

$$w_{k0} \longrightarrow \tilde{w}_{k0} = c w_{k0} + d \tag{9.13}$$

变换, 则新网络的输出恰为 $\tilde{y}_k = c y_k + d$. ♠

证明

(重构新增) 对第一层，把(9.8)至(9.10)代入：

$$\begin{aligned}\sum_i \tilde{w}_{ji}\tilde{x}_i + \tilde{w}_{j0} &= \sum_i \frac{w_{ji}}{a}(ax_i + b) + w_{j0} - \frac{b}{a}\sum_i w_{ji} \\ &= \sum_i w_{ji}x_i + \frac{b}{a}\sum_i w_{ji} + w_{j0} - \frac{b}{a}\sum_i w_{ji} \\ &= \sum_i w_{ji}x_i + w_{j0}.\end{aligned}$$

故 $z_j = h(\sum_i w_{ji}x_i + w_{j0})$ 不变. 对第二层，由 z_j 不变及(9.12)、(9.13)，

$$\sum_j \tilde{w}_{kj}z_j + \tilde{w}_{k0} = \sum_j cw_{kj}z_j + cw_{k0} + d = c\Big(\sum_j w_{kj}z_j + w_{k0}\Big) + d = cy_k + d,$$

这正是(9.11).

□

注

(重构新增)命题 9.2中的两组变换是“同一函数的两种参数化”的显式构造：第一组把输入仿射变换吸收进第一层，第二组把输出仿射变换吸收进第二层. 注意第一层权重按 $1/a$ 缩放而不是按 a 缩放：输入被放大 a 倍后，要得到相同的加权和，权重必须缩小 a 倍. 偏置的修正项 $-\frac{b}{a}\sum_i w_{ji}$ 则抵消输入平移 b 带来的常数偏移. 这些公式是命题 9.3的直接前提；它们也说明，若对权重和偏置施加各向同性的二次惩罚，两个等价的参数化会得到不同的惩罚值，从而破坏一致性.

如果使用原始数据训练一个网络，并使用输入和/或目标变量经过上述某种线性变换的数据训练另一个网络，那么一致性要求我们应该得到等价网络，它们区别仅仅在于是否对权重做了线性变换. 任何正则化项都应该与这种性质一致，否则就会任意地偏好一种解而不是另一种等价的解. 显然，简单权重衰减形式［(9.1)］因为平等地惩罚所有权重和偏置，所以不满足这一性质.

为此，我们需要寻找一个在线性变换［(9.9)、(9.10)、(9.12)和(9.13)］下不变的正则化项. 这要求正则化项对权重的重新缩放和偏置的移动保持不变. 这样的正则化项由下式给出：

$$\frac{\lambda_1}{2}\sum_{w\in\mathcal{W}_1} w^2 + \frac{\lambda_2}{2}\sum_{w\in\mathcal{W}_2} w^2, \tag{9.14}$$

其中 $\mathcal{W}_1$ 表示第一层的权重集合，$\mathcal{W}_2$ 表示第二层的权重集合，且偏置被排除在求和之外. 原书接着指出，只要使用 $\lambda_1 \to a^{1/2}\lambda_1$ 和 $\lambda_2 \to c^{-1/2}\lambda_2$ 来重新缩放正则化参数，这个正则化项在权重变换下就会保持不变.

注

(重构新增，原文问题说明) 上述缩放指数按原书照译，但按(9.9)与(9.12)逐项计算，它并不能使(9.14)保持不变：变换后第一层项为 $\frac{\lambda_1}{2a^2}\sum_{w\in\mathcal{W}_1} w^2$，要等于原来的 $\frac{\lambda_1}{2}\sum w^2$ 必须取 $\lambda_1' = a^2\lambda_1$；同理第二层项变为 $\frac{\lambda_2 c^2}{2}\sum w^2$，必须取 $\lambda_2' = c^{-2}\lambda_2$. 原书印作 $a^{1/2}$ 与 $c^{-1/2}$，按上述计算指数应为 2 与 −2，应属排版或翻译中的笔误. 下面命题 9.3给出修正后的陈述与证明；正文其余部分不受影响.

命题 9.3 (一致性正则项)

在命题 9.2的记号下，正则化项(9.14)在参数变换(9.9)、(9.12)下保持不变，当且仅当正则化系数按

$$\lambda_1 \longmapsto a^2\lambda_1, \qquad \lambda_2 \longmapsto c^{-2}\lambda_2$$

重新缩放. 特别地，偏置不出现在(9.14)中，因此对偏置的平移不变.

证明

(重构新增) 记变换后的系数为 λ_1', λ_2'. 由 $\tilde{w}_{ji} = w_{ji}/a$,

$$\frac{\lambda_1'}{2} \sum_{w \in \mathcal{W}_1} \tilde{w}^2 = \frac{\lambda_1'}{2a^2} \sum_{w \in \mathcal{W}_1} w^2.$$

要使其等于 $\frac{\lambda_1}{2} \sum w^2$, 必须且只需 $\lambda_1' = a^2 \lambda_1$. 同理, 由 $\tilde{w}_{kj} = c w_{kj}$,

$$\frac{\lambda_2'}{2} \sum_{w \in \mathcal{W}_2} \tilde{w}^2 = \frac{\lambda_2' c^2}{2} \sum_{w \in \mathcal{W}_2} w^2,$$

要使其等于 $\frac{\lambda_2}{2} \sum w^2$, 必须且只需 $\lambda_2' = c^{-2} \lambda_2$. 两个条件互相独立, 故命题成立. 偏置不在求和中, 其变换(9.10)、(9.13)不改变(9.14)的值.

□

注

(重构新增)命题 9.3给出了“按层分组、只惩罚权重、不惩罚偏置”这一做法的理由: 只有这样才能与输入、输出的仿射重参数化相容. 但一致性是有代价的: 由于偏置完全不受惩罚,(9.15)所对应的先验是反常的 (不可归一化), 因为偏置可以在 $\mathbb{R}$ 上任意取值而不影响惩罚. 实际使用中通常再给偏置单独加一个较弱的先验 (从而主动打破平移不变性), 其超参数与权重的超参数分开. 这也说明“一致性”不是越彻底越好: 它是一个建模选择, 需要与任务真正具有的对称性相匹配.

正则化项 [(9.14)] 对应于参数的先验分布形式:

$$p(\boldsymbol{w} \mid \alpha_1, \alpha_2) \propto \exp\left(-\frac{\alpha_1}{2} \sum_{w \in \mathcal{W}_1} w^2 - \frac{\alpha_2}{2} \sum_{w \in \mathcal{W}_2} w^2 \right). \tag{9.15}$$

注意, 这种形式的先验是反常的 (它们不能被规范化), 因为偏置参数是不受约束的. 使用反常的先验可能会对选择正则化系数, 以及在 Bayes 框架内进行模型比较造成困难. 因此, 我们通常会为偏置设置单独的先验 (从而打破平移不变性), 这些先验都有自己的超参数.

我们可以通过从先验中抽取样本并绘制相应的网络函数来展示这 4 个超参数的效果, 如图 9.4所示. 先验由 4 个超参数控制:α_1^b、α_1^w、α_2^b 和 α_2^w, 它们分别代表第一层偏置、第一层权重、第二层偏置和第二层权重所服从的 Gauss 分布的精度. 可以看到, 超参数 α_2^w 控制函数的竖直尺度 (注意图 9.4(a) 和图 9.4(b) 的竖直轴具有不同的取值范围),α_1^w 控制函数值变化的水平尺度,α_1^b 控制函数值变化的水平范围. 超参数 α_2^b 的效果此处未展示, 其控制函数的竖直偏移范围.

9.2.2 广义权重衰减

更一般地, 考虑将权重分成任意多组 $\mathcal{W}_k$ 的正则化项, 使得

$$\Omega(\boldsymbol{w}) = \frac{1}{2} \sum_k \alpha_k \|\boldsymbol{w}\|_k^2, \tag{9.16}$$

其中

$$\|\boldsymbol{w}\|_k^2 = \sum_{j \in \mathcal{W}_k} w_j^2. \tag{9.17}$$

例如, 我们可以为多层网络中的每一层使用不同的正则化项.

定义 9.9 (分组权重衰减)

设参数被划分为若干互不相交的组 $\mathcal{W}_k$,$\boldsymbol{w}_k$ 表示第 k 组的权重子向量. 称由(9.16)与(9.17)定义的 $\Omega(\boldsymbol{w})$ 为**分组权重衰减** (grouped weight decay),$\alpha_k > 0$ 为第 k 组的正则化系数 (等价地, 第 k 组先验的精度).

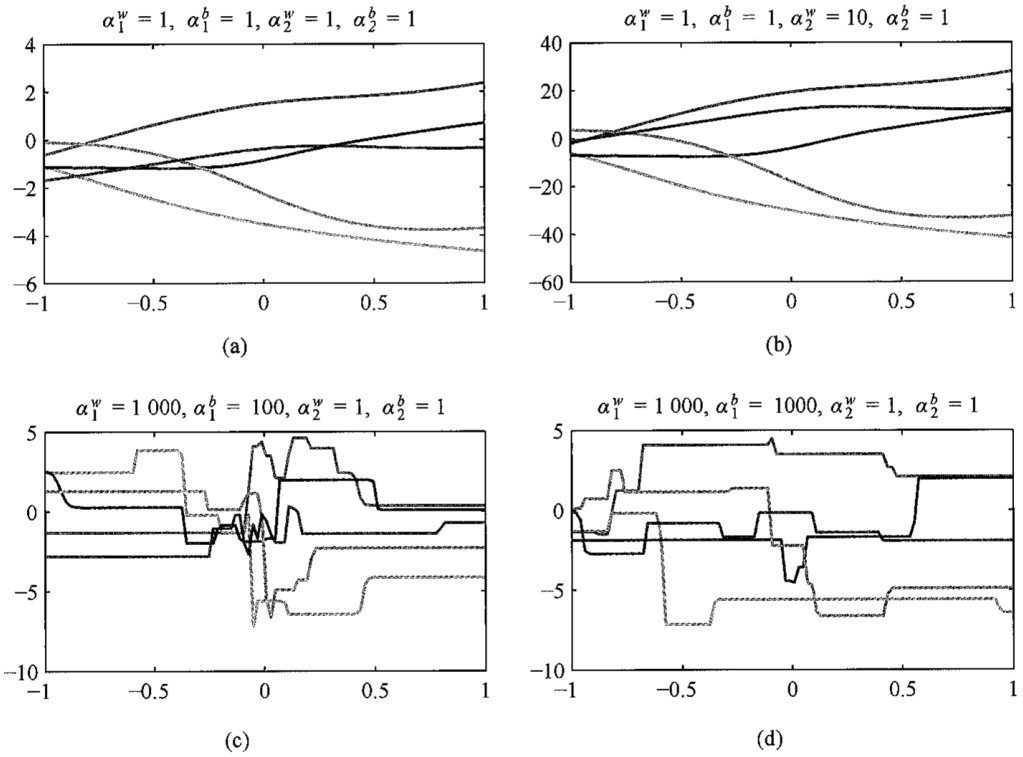


图 9.4: 在具有单一输入、单一线性输出和 12 个由 tanh 函数激活的隐藏单元的双层神经网络中, 旨在控制权重和偏置先验分布的超参数的效果示意图

注

(重构新增)定义 9.9把“一个全局 λ”推广为“每组一个 α_k”, 其必要性来自不同层权重的尺度差异: 输入层权重的量级、隐藏层权重的量级和输出层权重的量级往往相差很大, 用同一个 λ 会使某些层被过度收缩而另一些层几乎不受约束. 分组也带来新的选择问题: 组划分本身是一种结构先验, 划得过细等价于给每组单独调参, 划得过粗又退化为全局权重衰减.(9.14)就是按“层”分组的特例, 它同时满足线性变换一致性; 一般的分组则未必具有这种不变性.

我们有时使用简单的二次正则化项的一般形式:

$$\Omega(\boldsymbol{w}) = \frac{\lambda}{2}\sum_{j=1}^{M}|w_j|^q. \tag{9.18}$$

定义 9.10 (q-范数正则化与 lasso)

对 $q > 0$, 称(9.18)为 **q-范数正则化项** (q-norm regularizer). 当 $q = 1$ 时, 它在统计学领域称为 **lasso**(least absolute shrinkage and selection operator)(Tibshirani,1996); 当 $q = 2$ 时, 它退化为(9.1)中的二次正则化项. ♣

注

(重构新增)定义 9.10中的 q 控制等值线的几何形状, 从而控制解的稀疏性:$q = 2$ 的等值线是光滑的球面,$q = 1$ 的是有尖角的菱形,$0 < q < 1$ 的是向坐标轴凹陷的星形. 稀疏性来自“尖角”与误差椭圆相切的位置: 切点落在坐标轴上, 就使相应分量为零.$q < 1$ 时目标函数非凸, 存在多个局部极小, 优化更困难;$q = 1$ 是仍保持凸性的最小指数, 因此成为变量选择的标准选择. 注意 q-范数记法在 $q \neq 2$ 时并不是真正的范数 (不满足三角不等式), 只是对 $\sum|w_j|^q$ 的简称.

其中 $q = 2$ 对应于(9.1)中的二次正则化项.图 9.5显示了不同 q 值的正则化函数的等值线.

当 $q = 1$ 时, 形如(9.18)的正则化项在统计学领域称为 lasso(Tibshirani,1996). 对于二次误差函数, 它的性质是, 如果 λ 足够大, 系数 w_j 将趋于零, 从而得到一个稀疏的模型, 对应的基函数不起作用. 为了说明这一点, 考虑最小

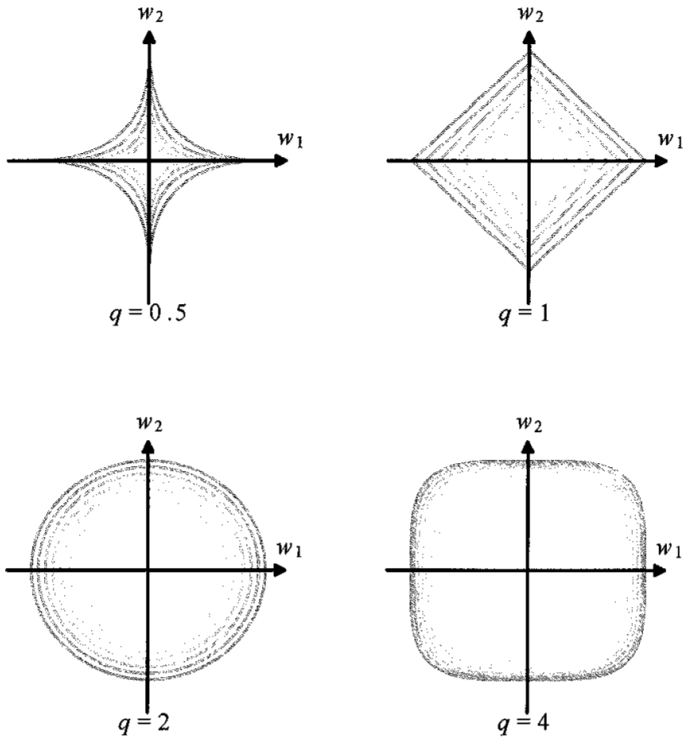


图 9.5: 不同 q 值的正则化函数的等值线

化下式给出的正则化误差函数:

$$E(\boldsymbol{w}) + \frac{\lambda}{2}\sum_{j=1}^{M}|w_j|^q. \tag{9.19}$$

这等价于在约束条件 (见练习 9.5)

$$\sum_{j=1}^{M}|w_j|^q \leqslant \eta \tag{9.20}$$

下最小化未正则化的误差函数 $E(\boldsymbol{w})$, 其中 η 有一个合适的取值. 这两种方法可以通过 Lagrange 乘子法关联起来 (参见附录 C).

命题 9.4 (lasso 的约束等价形式)

设 $E : \mathbb{R}^M \to \mathbb{R}$ 为凸函数,$q \geqslant 1$, 并设约束问题

$$\min_{\boldsymbol{w}} E(\boldsymbol{w}) \quad \text{使得} \quad \sum_{j=1}^{M}|w_j|^q \leqslant \eta$$

满足 Slater 条件 (存在严格可行点). 则对每个 $\lambda > 0$, 存在 $\eta = \eta(\lambda) \geqslant 0$, 使得(9.19)的任一最小点都是该约束问题的最小点; 反之, 对每个 $\eta > 0$, 存在 $\lambda = \lambda(\eta) \geqslant 0$, 使得约束问题的任一最小点都是(9.19)的最小点.

证明

(重构新增) 记 $g(\boldsymbol{w})=\sum_j |w_j|^q$.(9.19)是 $\min_{\boldsymbol{w}} E(\boldsymbol{w})+\frac{\lambda}{2}g(\boldsymbol{w})$, 其 Lagrange 函数可写为

$$\mathcal{L}(\boldsymbol{w},\mu)=E(\boldsymbol{w})+\frac{\mu}{2}\big(g(\boldsymbol{w})-\eta\big),\qquad \mu\geqslant 0.$$

由凸性及 Slater 条件,KKT 条件是最优性的充要条件: 存在 $\mu\geqslant 0$ 使

$$\mathbf{0}\in\partial E(\boldsymbol{w})+\frac{\mu}{2}\partial g(\boldsymbol{w}),\qquad \mu\big(g(\boldsymbol{w})-\eta\big)=0.$$

取 $\lambda=\mu$ 即知约束问题的 KKT 点满足(9.19)的一阶最优性条件; 凸情形下这也是充分条件, 故约束问题的最小点是(9.19)的最小点. 反过来, 给定 $\lambda>0$, 令 $\boldsymbol{w}_\lambda$ 为(9.19)的最小点, 取 $\eta=g(\boldsymbol{w}_\lambda)$, 则 $\boldsymbol{w}_\lambda$ 满足约束且 KKT 条件中的互补松弛成立, 从而是约束问题的最小点.η 与 λ 的对应由 $g(\boldsymbol{w}_\lambda)$ 关于 λ 的单调性给出:λ 增大时最优解的 g 值不增, 故 $\eta(\lambda)$ 单调不增; 在 g 严格凸且解唯一的区域可逆.

□

注

(重构新增)命题 9.4把“惩罚形式”和“约束形式”视为同一个问题的两种参数化, 这在几何上解释了图 9.6: 约束形式把解限制在半径为 η 的 q-球内, 惩罚形式则用等值线去“推”解. 两者只在凸情形下等价; 若 E 非凸或 $q<1$,KKT 只是必要条件, 可能出现局部解与全局解不一致. 还要注意 η 与 λ 的对应依赖数据 (通过 E 的曲率), 并不是一个固定的换算公式;练习 9.5要求讨论这一关系.

模型稀疏的原因可以通过图 9.6说明: 该图显示了误差函数在约束条件(9.20)下的最小值. 随着 λ 增大, 更多参数将趋于零. 相比之下, 二次正则化项不会要求权重参数为零, 只会将 w_j 减小到一个较小值.lasso 正则化项提供了一个稀疏解, 其中 $w_j=0$.

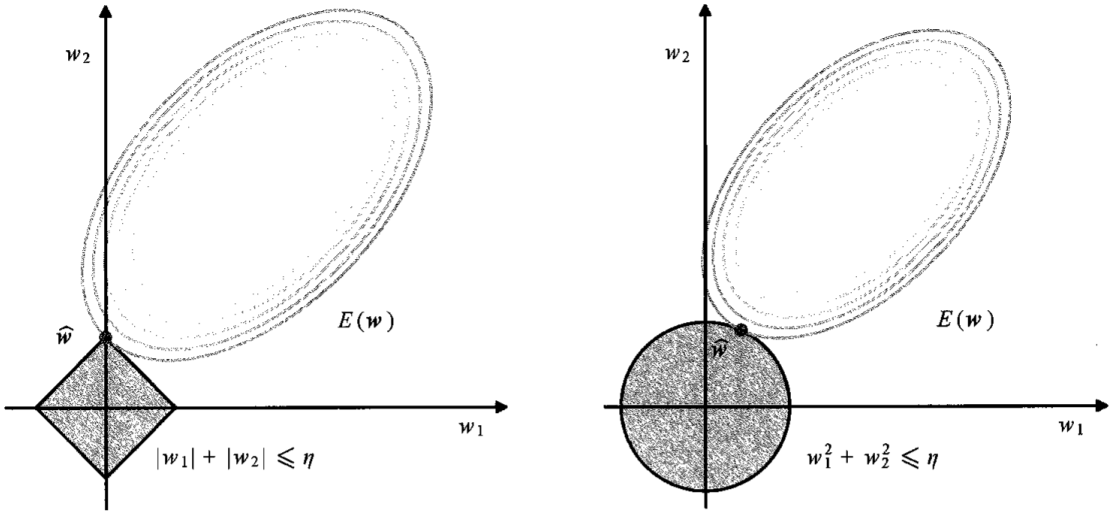


图 9.6: 未正则化的误差函数的等值线 (红色) 以及两种情况的约束条件 [(9.20)] 的示意图. 左侧是 $q=1$ 时的 lasso 正则化项, 右侧是 $q=2$ 时的二次正则化项. 其中参数向量 $\boldsymbol{w}$ 的最优值由 $\widehat{\boldsymbol{w}}$ 表示.lasso 正则化项提供了一个稀疏解, 其中 $\widehat{w}_1=0$, 而二次正则化项仅将 w_1 减至一个较小的值

正则化使得我们在有限大小的数据集上训练复杂模型的同时不会造成严重的过拟合, 这本质上是通过限制有效模型的复杂度来实现的. 这就让确定最优的模型复杂度问题从寻找适当数量的可学习参数变为确定合适的正则化系数 λ. 我们将在下一节讨论模型复杂度问题.

9.3 学习曲线

我们已经探讨了模型的泛化能力如何随模型中参数的数量、数据集的大小以及权重衰减正则化项的系数而变化. 这些都允许我们在偏差和方差之间进行权衡, 以最小化泛化误差. 另一个影响这种权衡的因素是学习过程本

身. 在通过梯度下降优化误差函数期间, 训练误差通常随着模型参数的更新而降低, 而留出数据 (hold-out data) 的误差可能是非单调的. 这种现象可以通过使用学习曲线来可视化. 学习曲线显示了在迭代式学习过程 (如随机梯度下降) 中, 性能指标 (如训练集和验证集误差) 关于迭代次数的变化. 学习曲线不仅提供了对训练进程的洞察, 还提供了一种控制模型有效复杂度的实用方法.

定义 9.11 (学习曲线)

设模型由迭代式优化算法训练,$\tau = 0, 1, 2, \cdots$ 为迭代次数 (或训练轮次). 称函数

$$\tau \mapsto \big(E_{\text{train}}(\tau), E_{\text{val}}(\tau)\big)$$

为**学习曲线** (learning curve), 其中 $E_{\text{train}}(\tau)$ 与 $E_{\text{val}}(\tau)$ 分别是第 τ 步时在训练集与验证集 (留出数据) 上计算的误差. 若横轴改为训练集大小 N, 相应的曲线称为**样本量学习曲线**.

♣

注

(重构新增)定义 9.11把“训练过程”本身当作一个可调节模型复杂度的维度: 固定模型容量而改变训练多久, 等价于在“欠拟合—过拟合”之间移动. 这与调 λ、调参数个数是三种不同的复杂度控制方式, 但它们在偏差–方差意义下有相似的效应. 读学习曲线时要注意两点: 验证误差的非单调性是过拟合的直接信号, 但它的最小值位置依赖验证集的大小与划分, 因此早停的轮次本身是一个需要估计的超参数; 训练误差通常单调下降, 若它下降而验证误差同时上升, 说明模型开始记忆训练集中的噪声, 而不是学到了可迁移的结构.

9.3.1 早停法

除正则化外, 控制网络有效复杂度的一种替代方法是**早停法** (early stopping). 深度学习模型的训练涉及迭代下降定义在一组训练数据上的误差函数. 虽然在训练集上计算的误差大致上关于迭代次数单调递减, 但在留出数据 (通常称为验证集) 上计算的误差通常首先下降, 然后由于网络开始过拟合而增加. 因此, 为了获得具有良好泛化能力的网络, 应在验证集上的最小误差点处停止训练, 如图 9.7所示.

定义 9.12 (早停法)

设 $\{\boldsymbol{w}^{(\tau)}\}_{\tau \geqslant 0}$ 是迭代训练产生的参数序列,$E_{\text{val}}(\tau)$ 为相应的验证误差. 若训练在

$$\tau^\star \in \arg\min_{\tau \geqslant 0} E_{\text{val}}(\tau)$$

处停止, 并以 $\boldsymbol{w}^{(\tau^\star)}$ 作为最终模型, 则称这一训练策略为**早停法** (early stopping).$\tau^\star$ 称为**停止轮次**.

♣

注

(重构新增)定义 9.12看起来只是“提前结束训练”, 但它实质上是一种正则化: 它限制了参数从初始化点出发所能走过的距离, 从而限制了模型的有效复杂度. 与权重衰减不同, 早停几乎不增加计算成本 (本来就要监控验证误差), 也不需要修改误差函数; 代价是它引入了新的超参数 $\tau^\star$, 而且这个超参数依赖验证集的划分. 还需注意,arg min 可能不唯一, 实践中通常取第一个达到最小值的轮次, 并在其附近做平滑或 patience 机制, 以避免验证误差的随机波动导致过早停止.

学习曲线的这种现象有时可以从网络中有效参数数量的角度进行定性解释. 这个数量开始时很少, 然后在训练过程中增长, 对应模型有效复杂度的稳定增加. 在训练误差达到最小值之前, 停止训练是限制网络有效复杂度的一种方法.

我们可以在二次误差函数上验证这一洞察, 结果显示早停法与使用简单权重衰减项的正则化 (Bishop,1995a) 作用相似.图 9.8对此做了说明, 权重空间中的轴已经旋转以与 Hessian 矩阵的特征向量平行. 如果在没有权重衰减的情况下 (参见节 7.1), 权重向量从原点开始, 并在训练期间沿着局部负梯度向量的路径前进, 那么权重向量将从最初与 $\boldsymbol{w}_2$ 轴平行, 大约朝着 $\boldsymbol{w}^*$ 移动, 最后向误差函数的最小值移动. 这种移动方法遵循误差表面的形状, 并且与 Hessian 矩阵的特征值有很大不同. 因此, 在 $\hat{\boldsymbol{w}}$ 附近的某个点停止训练类似于权重衰减. 早停法和权重衰减之间的关系可以被量化 (参见练习 9.6), 从而显示出量 $\rho\tau$(其中 τ 是迭代次数,ρ 是学习率) 可以像正则化参数 λ 的倒数一

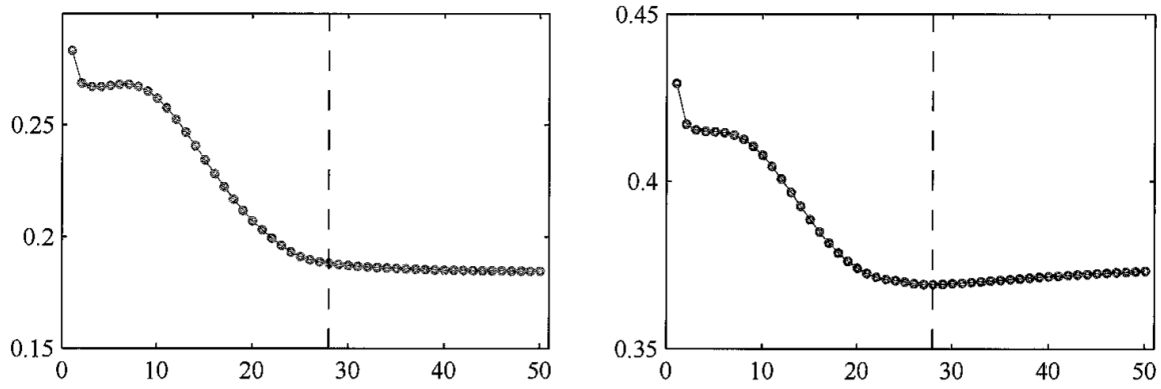


图 9.7: 在正弦数据集上进行的典型训练过程中, 训练误差 (左) 和验证误差 (右) 关于迭代次数的函数的示意图. 为了达到最佳的泛化能力, 应在垂直虚线所示的迭代轮次停止训练, 此处对应验证误差的最小值

样发挥作用. 因此, 网络中有效参数的数量在训练期间会增长.

定理 9.3 (早停与权重衰减的等价性)

设未正则化误差函数为

$$E(\boldsymbol{w}) = E_0 + \frac{1}{2}(\boldsymbol{w}-\boldsymbol{w}^*)^{\mathsf{T}}\boldsymbol{H}(\boldsymbol{w}-\boldsymbol{w}^*),$$

其中 $\boldsymbol{H}$ 对称正定,$\boldsymbol{w}^*$ 为最小值点. 取初始值 $\boldsymbol{w}^{(0)} = \boldsymbol{0}$, 并用学习率 $\rho > 0$ 作梯度下降

$$\boldsymbol{w}^{(\tau)} = \boldsymbol{w}^{(\tau-1)} - \rho\nabla E(\boldsymbol{w}^{(\tau-1)}).$$

则

$$\boldsymbol{w}^{(\tau)} = \left[\boldsymbol{I} - (\boldsymbol{I}-\rho\boldsymbol{H})^{\tau}\right]\boldsymbol{w}^*.$$

若 $\{(\eta_j, \boldsymbol{u}_j)\}$ 是 $\boldsymbol{H}$ 的正交特征对,$w_j^{(\tau)} = \boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{w}^{(\tau)}$、$w_j^* = \boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{w}^*$, 则

$$w_j^{(\tau)} = \left[1-(1-\rho\eta_j)^{\tau}\right]w_j^*.$$

进一步, 设 $0 < \rho\eta_j < 1$ 对所有 j 成立且 τ 充分大. 则对 $\eta_j \gg (\rho\tau)^{-1}$ 的分量有 $w_j^{(\tau)} \approx w_j^*$, 对 $\eta_j \ll (\rho\tau)^{-1}$ 的分量有 $|w_j^{(\tau)}| \ll |w_j^*|$. 这正是权重衰减解

$$\boldsymbol{w}_\lambda = (\boldsymbol{H}+\lambda\boldsymbol{I})^{-1}\boldsymbol{H}\boldsymbol{w}^*$$

在 $\lambda = (\rho\tau)^{-1}$ 处的逐分量行为.

♡

证明

(重构新增) 令 $\boldsymbol{u}^{(\tau)} = \boldsymbol{w}^{(\tau)} - \boldsymbol{w}^*$. 由 $\nabla E(\boldsymbol{w}) = \boldsymbol{H}(\boldsymbol{w}-\boldsymbol{w}^*)$, 更新式变为

$$\boldsymbol{u}^{(\tau)} = \boldsymbol{u}^{(\tau-1)} - \rho\boldsymbol{H}\boldsymbol{u}^{(\tau-1)} = (\boldsymbol{I}-\rho\boldsymbol{H})\boldsymbol{u}^{(\tau-1)}.$$

初始条件为 $\boldsymbol{u}^{(0)} = \boldsymbol{w}^{(0)} - \boldsymbol{w}^* = -\boldsymbol{w}^*$. 递推得 $\boldsymbol{u}^{(\tau)} = -(\boldsymbol{I}-\rho\boldsymbol{H})^{\tau}\boldsymbol{w}^*$, 故

$$\boldsymbol{w}^{(\tau)} = \boldsymbol{w}^* - (\boldsymbol{I}-\rho\boldsymbol{H})^{\tau}\boldsymbol{w}^* = \left[\boldsymbol{I}-(\boldsymbol{I}-\rho\boldsymbol{H})^{\tau}\right]\boldsymbol{w}^*.$$

在特征基下,$\boldsymbol{H}^{\tau}\boldsymbol{u}_j = \eta_j^{\tau}\boldsymbol{u}_j$, 且 $(\boldsymbol{I}-\rho\boldsymbol{H})^{\tau}\boldsymbol{u}_j = (1-\rho\eta_j)^{\tau}\boldsymbol{u}_j$, 故第 j 个分量为 $w_j^{(\tau)} = [1-(1-\rho\eta_j)^{\tau}]w_j^*$. 当 $|1-\rho\eta_j| < 1$ 即 $0 < \rho\eta_j < 2$ 时,$(\boldsymbol{I}-\rho\boldsymbol{H})^{\tau} \to \boldsymbol{0}$, 从而 $\boldsymbol{w}^{(\tau)} \to \boldsymbol{w}^*$.

下面比较有限 τ 的行为. 若 $\eta_j \gg (\rho\tau)^{-1}$, 则 $\rho\eta_j\tau \gg 1$, 且

$$(1-\rho\eta_j)^{\tau} \approx e^{-\rho\eta_j\tau} \approx 0,$$

故 $w_j^{(\tau)} \approx w_j^*$. 若 $\eta_j \ll (\rho\tau)^{-1}$, 则 $\rho\eta_j\tau \ll 1$. 由二项式展开

$$(1-\rho\eta_j)^{\tau} = 1 - \rho\eta_j\tau + O\big((\rho\eta_j\tau)^2\big),$$

故 $w_j^{(\tau)} \approx \rho\eta_j\tau\, w_j^*$, 从而 $|w_j^{(\tau)}| \ll |w_j^*|$. 另一方面, 由定理 9.2, 权重衰减的第 j 个分量为

$$w_j^{\lambda} = \frac{\eta_j}{\eta_j + \lambda} w_j^*.$$

取 $\lambda = (\rho\tau)^{-1}$: 当 $\eta_j \gg \lambda$ 时 $\dfrac{\eta_j}{\eta_j+\lambda} \approx 1$, 当 $\eta_j \ll \lambda$ 时 $\dfrac{\eta_j}{\eta_j+\lambda} \approx \dfrac{\eta_j}{\lambda} = \rho\eta_j\tau$. 这与上面的两个渐近式逐一吻合.

□

注

(重构新增)定理 9.3说明早停不是“另一种正则化”的类比, 而是在二次误差、从原点出发、学习率足够小的条件下与权重衰减的精确对应: 迭代次数 τ 扮演 $1/(\rho\lambda)$ 的角色, 训练越久等价于 λ 越小. 由此可以解释图 9.8: 梯度下降的轨迹先沿 Hessian 大特征值方向快速下降, 再沿小特征值方向缓慢前进; 在 $\hat{\boldsymbol{w}}$ 处停止, 相当于让小特征值方向的分量还来不及长大, 这与权重衰减压制小曲率方向的效果一致. 必须注意这个等价性的边界: 它依赖二次误差、固定学习率、从原点初始化; 对非线性网络只是局部近似, 且学习率过大时 $|1-\rho\eta_j| < 1$ 可能不成立, 训练会发散而不是正则化. 此外, 早停的有效参数数随 τ 增长, 这与 λ 固定时有效参数数固定的情形不同: 早停对应一条“随时间变化的正则化路径”.

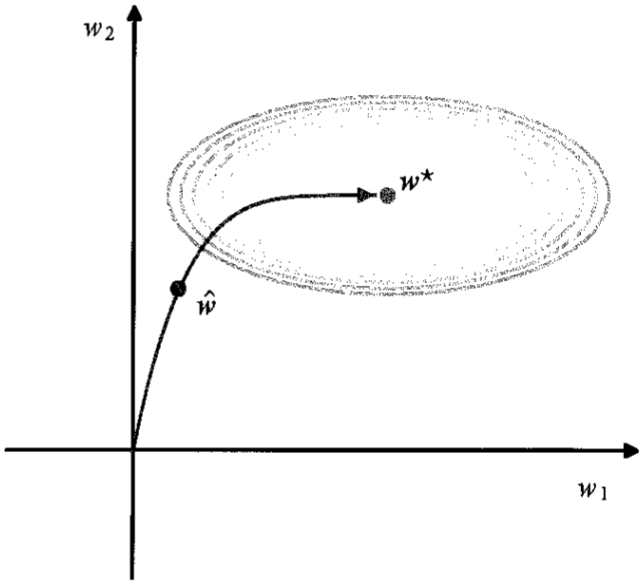


图 9.8: 对于二次误差函数, 早停法可以给出与权重衰减相似结果的示意图. 椭圆显示了恒定误差的等值线,$\boldsymbol{w}^*$ 表示对应于未正则化误差函数最小值的最大似然解. 如果权重向量从原点开始, 并根据局部负梯度方向移动, 那么权重向量将沿着图中所示的路径前进. 通过与图 9.3进行比较可以看出, 提前停止训练后, 就可以找到一个权重向量 $\hat{\boldsymbol{w}}$, 从而发现这与使用简单的权重衰减正则化项, 并训练到正则化误差最小时的结果类似

9.3.2 双重下降

偏差–方差权衡为可学习模型的泛化能力会随着模型参数数量的变化而变化提供了解释 (参见节 4.3). 参数太少的模型会因为有限的表现能力 (高偏差) 而有较大的测试误差; 随着参数数量的增加, 可以预期测试误差会降低. 然而, 随着参数数量的进一步增加, 由于过拟合 (高方差), 测试误差会再次上升. 这引出了经典统计学中一个普遍的观念, 即模型中参数的数量需要根据数据集的大小进行限制: 对于给定的训练集, 非常大的模型预期表现不佳.

然而, 与这种预期相反, 对于现代深度神经网络, 即使参数数量远远超过对训练数据完美拟合所需, 也可以表现出色 (Zhang et al.,2016). 深度学习社区的普遍看法是, 更大的模型表现会更好. 虽然有时提前停止训练, 但模型也可能被训练到零误差, 并且在测试数据上表现良好.

这些看似矛盾的观点可以通过检查学习曲线和其他模型复杂度对泛化能力的图表来解释, 这些图表揭示了一种称为**双重下降** (double descent) 的更微妙的现象 (Belkin et al.,2019).图 9.9显示了一个名为 ResNet18 的大型神

经网络模型 (He et al.,2015a) 中, 训练误差和测试误差与模型复杂度 (由可学习参数的数量决定) 的关系. 该网络有 18 层参数, 是在图像分类任务上进行训练的. 通过改变控制每层中隐藏单元数量的 “宽度参数”, 网络中的权重和偏置数量可以变化. 我们看到, 与预期一致, 训练误差随着模型复杂度的增加而单调减小. 然而, 测试误差最初下降, 然后再次上升, 最后再次下降. 虽然训练误差已经达到零, 但测试误差仍旧持续减小.

定义 9.13 (双重下降)

设模型复杂度由参数数量或某个连续控制参数 (如宽度、训练轮次、$1/\lambda$) 度量. 若测试误差作为复杂度的函数先下降、再上升、最后再次下降, 即出现两个下降段与中间一个峰值, 则称这一现象为**双重下降** (double descent). 第一个下降段对应经典的偏差–方差权衡, 峰值出现在模型刚好能够插值训练数据的临界区域附近, 第二个下降段对应过参数化区域.

♣

注

(重构新增)定义 9.13中的 “双重” 指的是测试误差曲线上有两个下降段, 而不是两个峰值. 峰值的位置称为插值阈值: 在它左侧, 模型欠参数化, 增加容量主要降低偏差; 在它右侧, 模型可以零误差拟合训练集, 继续增加容量反而使隐式正则化 (如随机梯度下降的偏好) 挑选出更平滑的插值解, 测试误差重新下降. 这个现象并不否定偏差–方差权衡, 而是说明 “参数数量” 不是复杂度的好度量: 在插值阈值附近, 有效复杂度急剧变化, 单看参数个数会得出错误结论. 还要注意, 双重下降的第二个下降段依赖优化算法与初始化; 同样的过参数化模型若训练不充分, 可能停留在峰值附近而表现很差.

两种模型拟合模式之间的过渡大约发生在模型中参数的数量足够多以至于能够精确拟合训练数据时 (Belkin et al.,2019).Nakkiran 等人 (2019) 定义了模型有效复杂度为模型可以在上面达到零训练误差的最大训练集大小, 因此当模型有效复杂度超过训练集中数据点的数量时, 就会出现双重下降.

定义 9.14 (模型有效复杂度)

对给定的模型族与训练算法, 称该模型能够在其上达到零训练误差的最大训练集大小为模型的**有效复杂度** (effective complexity)(Nakkiran et al.,2019). 当有效复杂度超过当前训练集的数据点数时, 模型处于插值区域.

♣

注

(重构新增)定义 9.14把复杂度从 “参数个数” 改为 “能记住多少样本”, 因而同时依赖模型族、优化算法与训练轮次. 这个定义解释了为什么同一网络在不同训练轮次下表现出不同的有效复杂度: 训练越久, 模型能插值的样本越多, 有效复杂度越高. 它也给出双重下降的统一图景: 无论横轴是参数个数、训练轮次还是 $1/\lambda$, 只要该轴能推动模型越过插值阈值, 就可能看到第二次下降. 使用时要注意 “零训练误差” 是理想化条件; 实际中应理解为低于某个很小的阈值, 因此有效复杂度是一个依赖于阈值的量.

这种令人惊讶的现象比通常从经典统计学的偏差–方差讨论中期望的现象更复杂, 并呈现出两种不同的模型拟合模式, 正如图 9.9中所显示的, 对应于小到中等复杂度的经典偏差–方差权衡, 随后当我们进入大型模型的阶段时, 测试误差进一步减小.

如图 9.10所示, 如果使用早停法来控制模型复杂度, 则可以看到类似的现象. 增加训练轮次会增加模型有效复杂度, 对于足够大的模型, 我们将再一次观察到双重下降现象.

当我们在误差函数中使用正则化项来控制模型复杂度时, 可以得到类似的结果. 在这里, 一个训练到收敛的大型模型的测试误差显示出关于 $1/\lambda$(即正则化参数的倒数) 的双重下降, 因为较高的 λ 对应于较低的模型复杂度 (Yilmaz and Heckel,2022).

双重下降的一个讽刺结果是, 在某种情况下, 增加训练集的大小实际上可能会降低性能, 这与常识相悖, 因为一般认为更多数据总是好事. 对于图 9.9中显示的处于临界阶段的模型, 增加训练集的大小可以将插值阈值向右推, 导致更高的测试误差. 这在图 9.11中得到了确认,图 9.11显示了 Transformer 模型的测试误差关于输入空间维度 (称为嵌入维度) 的函数 (参见第 12 章). 增加嵌入维度会增加模型中权重和偏置的数量, 从而增加模型复杂度. 我们看到, 将训练集中数据点的数量从 4 000 增加到 18 000 可以得到整体上更低的曲线. 然而, 对于一定范围内的

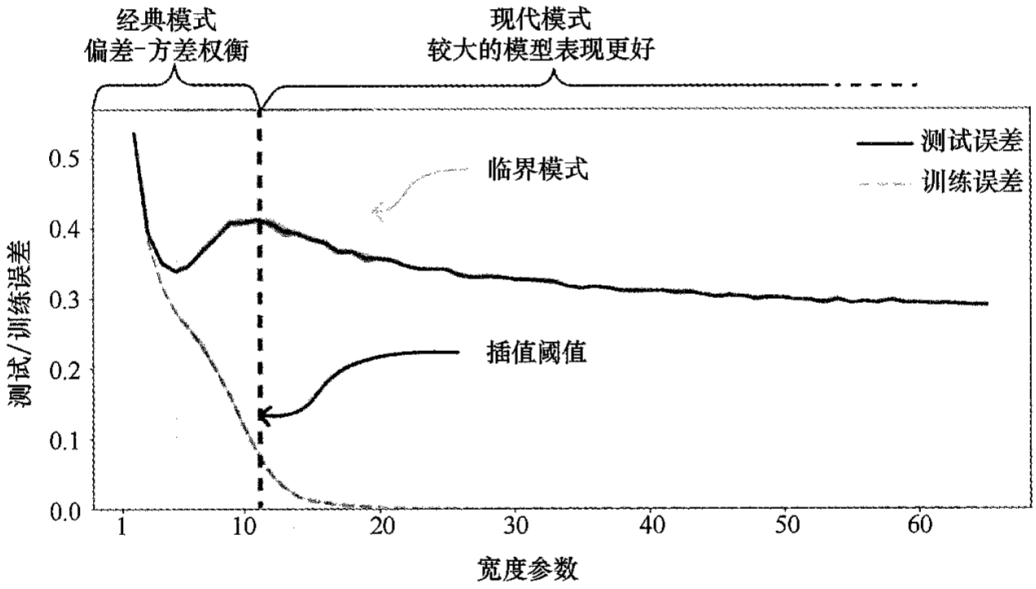


图 9.9: 一个名为 ResNet18 的大型神经网络模型在图像分类问题上训练误差和测试误差与模型复杂度的关系图. 水平轴代表一个控制隐藏单元数量的超参数, 因此也控制了网络中权重和偏置的总数. 标记为“插值阈值”的竖直虚线表示原则上能够在训练集上达到零误差的模型复杂度水平［经 Nakkiran et al.(2019) 授权使用］

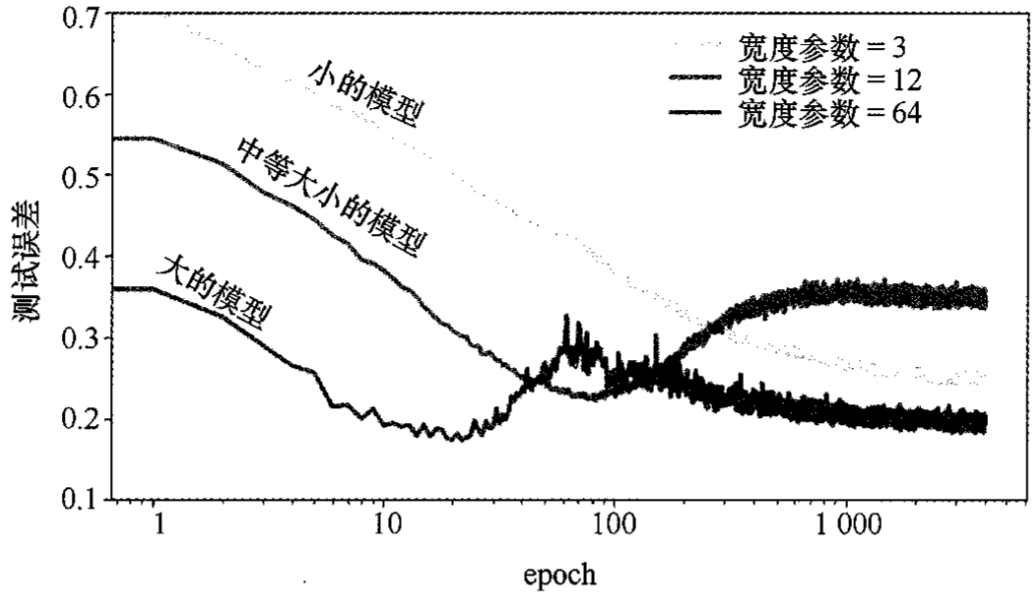


图 9.10: ResNet18 模型的测试误差与梯度下降训练的训练轮次的关系图. 模型有效复杂度随训练轮次的增加而增加, 对于足够大的模型, 可以观察到双重下降现象［经 Nakkiran et al.(2019) 授权使用］

嵌入维度 (对应处于临界复杂度阶段的模型), 增加数据集的大小实际上可能会降低模型的泛化能力.

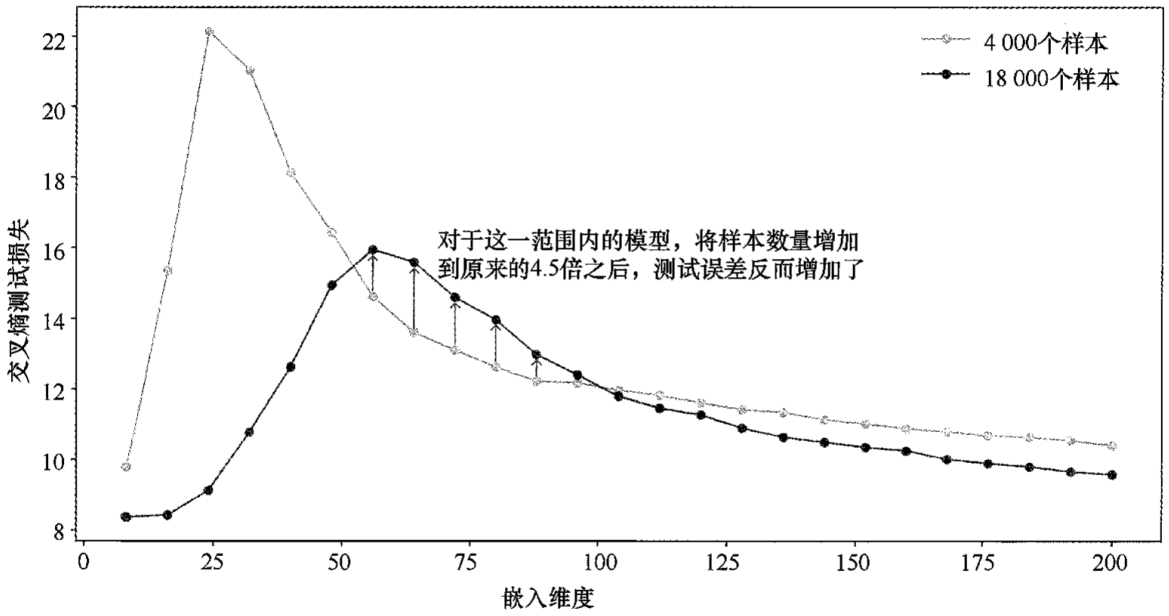


图 9.11: 大型 Transformer 模型的测试误差与控制模型中参数数量的嵌入维度的关系图. 将训练集的大小从 4 000 增加到 18 000 通常会使测试误差减小, 但对于一些中等复杂度的模型, 测试误差反而会增加, 如垂直红色箭头所示 [经 Nakkiran et al.(2019) 授权使用]

对于这样的模型, 存在许多可能的解, 包括那些过度拟合数据的解. 因此, 似乎是随机梯度下降引入的隐性偏见产生了良好的泛化能力.

9.4 参数共享

正则化项, 如 L_2 正则化项 $\|\boldsymbol{w}\|^2$, 通过促使权值接近于零, 从而减少过拟合. 另一种降低网络复杂度的方法是对权值施加硬约束. 该方法将权重分组, 要求每组内的所有权重共享相同的值, 其中共享的值是从数据中学习得到的.

定义 9.15 (权重共享)

设参数向量被划分为若干互不相交的组 $\{\mathcal{W}_k\}$. 若对每个 k 及所有 $i, j \in \mathcal{W}_k$ 都强制 $w_i = w_j$, 则称这些权重**共享** (weight sharing), 又称**参数共享** (parameter sharing) 或**参数捆绑** (parameter tying). 每个等价类中的公共值才是独立可学习参数, 模型的自由度数量等于等价类的个数而非连接数. ♣

注

(重构新增)定义 9.15与定义 9.7的区别是“硬约束”与“软惩罚”: 前者把参数空间限制在一个线性子空间上, 后者只是把解往原点拉. 硬约束的优势是它精确表达了已知的不变性, 不需要调节惩罚强度; 代价是约束形式必须事先给定, 模型无法从数据中学习“哪些权重应该相等”. 卷积网络是这一思想的典型: 同一个卷积核在图像的所有位置重复使用, 自由度从“位置 × 通道”降为“通道”, 从而把平移等变性写进架构 (参见第 10 章). 实现上, 共享权重的梯度不是某一副本的梯度, 而是所有副本梯度之和 (见练习 9.7); 这一点在自动微分框架中由计算图自动处理.

这意味着自由度的数量少于网络中连接的数量. 通常, 这是一种将归纳偏置编码到网络中的方法, 从而表达一些已知不变性. 评估此类网络误差函数的梯度可以通过稍微修改反向传播来完成 (见练习 9.7), 虽然在实践中这是通过自动微分隐式处理的. 我们讨论卷积神经网络 (参见第 10 章) 时, 将广泛使用参数共享. 然而, 参数共享只适用于可以预先指定约束形式的特定问题.

9.4.1 软权重共享

Nowlan 和 Hinton(1992) 并未使用强制模型参数集相等的硬约束, 而是引入了一种称为**软权重共享** (soft weight sharing) 的形式, 其中正则化项促使权重组具有相似的值. 此外, 权重的分组、每组的平均权值以及组内权值的传播都是作为学习过程的一部分来确定的.

定义 9.16 (软权重共享)

设每个权重 w_i 独立地服从同一个 K 分量 Gauss 混合先验

$$p(w_i) = \sum_{j=1}^{K} \pi_j \mathcal{N}\left(w_i \mid \mu_j, \sigma_j^2\right),$$

其中混合系数 π_j、分量均值 μ_j 与方差 σ_j^2 都是可学习参数. 称以该先验的负对数作为正则化项、从而促使权重组具有相似值 (而非强制相等) 的训练方式为**软权重共享** (soft weight sharing).

♣

注

(重构新增)定义 9.16把“分组”从离散的、事先指定的约束变成连续的、由数据推断的聚类: 若某个 w_i 的后验几乎只落在第 j 个分量上, 它就相当于被“分配”到第 j 组, 并以 μ_j 为中心被收缩; 若后验在多个分量间分摊, 则它同时受到多个中心的拉扯. 与硬共享相比, 软共享允许组内方差不等于零, 也允许一个权重在训练过程中改变所属的组, 因此更灵活; 代价是多了约 $3K$ 个超参数 (π_j, μ_j, σ_j), 并且目标函数关于这些参数不再凸, 可能收敛到不同的局部解.K 的选择本身是一个模型选择问题:K 太小退化为单峰先验,K 太大则容易把噪声也拟合成“组”.

前面曾提到,(9.1)中的简单权重衰减正则化项可以看作权重 Gauss 先验分布的负对数. 这促使所有的权重都收敛到 0. 但我们可以通过考虑 Gauss 混合概率分布来促使权重形成几个组, 而非仅仅一个组.Gauss 分量的均值 μ_j 和方差 σ_j^2, 以及混合系数 π_j, 则视为学习过程中需要确定的可调参数 (参见小节 3.2.9). 这样我们就有了如下概率密度形式:

$$p(\boldsymbol{w}) = \prod_i \left\{ \sum_{j=1}^{K} \pi_j \mathcal{N}\left(w_i \mid \mu_j, \sigma_j^2\right) \right\}. \tag{9.21}$$

定义 9.17 (权重的 Gauss 混合先验)

称由(9.21)给出的、关于各分量独立同分布的权重先验为**权重的 Gauss 混合先验** (Gaussian mixture prior), 其中 K 为分量数,π_j 为混合系数,μ_j 与 σ_j^2 分别为第 j 个分量的均值与方差.

♣

注

(重构新增)定义 9.17中的“独立同分布”是相对于各权重下标 i 而言的: 给定混合参数后, 各 w_i 条件独立. 这个假设使似然可以写成乘积, 从而负对数变成对 i 的求和, 计算与实现都很简单. 但它也意味着先验不知道权重的空间位置或层归属; 若希望不同层使用不同的混合参数, 应把 i 的求和分组, 得到分层的混合先验. 另一个要点是: 混合模型的对数似然不是凸的, 且存在“分量塌缩”的退化解 (某个分量只拟合一个权重、方差趋于零), 实践中需要对方差设下界或使用最大似然的重参数化来避免.

其中 K 是 Gauss 分量的数量. 对(9.21)的右侧取负对数就会得到如下形式的正则化函数:

$$\Omega(\boldsymbol{w}) = -\sum_i \ln \left\{ \sum_{j=1}^{K} \pi_j \mathcal{N}\left(w_i \mid \mu_j, \sigma_j^2\right) \right\}. \tag{9.22}$$

总误差函数由下式给出:

$$\widetilde{E}(\boldsymbol{w}) = E(\boldsymbol{w}) + \lambda \Omega(\boldsymbol{w}), \tag{9.23}$$

其中 λ 为正则化系数.

该误差关于混合模型的权重 w_i 和参数 $\{\pi_j, \mu_j, \sigma_j\}$ 被联合最小化. 这可以使用梯度下降来完成, 但需要我们计算 $\Omega(\boldsymbol{w})$ 对所有可学习参数的导数. 为此, 我们将 $\{\pi_j\}$ 视为已经生成权值的每个分量的先验概率, 并引入相应的

后验概率. 后验概率由 Bayes 定理给出 (见练习 9.8).

命题 9.5 (软权重共享的后验概率)

在(9.21)的模型中, 把 π_j 视为分量 j 的先验概率 $p(j)$. 则给定权重 w_i 时分量 j 的后验概率为

$$\gamma_j(w_i) = p(j \mid w_i) = \frac{\pi_j \mathcal{N}\left(w_i \mid \mu_j, \sigma_j^2\right)}{\sum_k \pi_k \mathcal{N}\left(w_i \mid \mu_k, \sigma_k^2\right)}. \tag{9.24}$$

并且对每个 i 有 $\sum_j \gamma_j(w_i) = 1$.

证明

(重构新增) 由 Bayes 定理,

$$p(j \mid w_i) = \frac{p(w_i \mid j)p(j)}{p(w_i)} = \frac{\mathcal{N}(w_i \mid \mu_j, \sigma_j^2)\pi_j}{\sum_k \mathcal{N}(w_i \mid \mu_k, \sigma_k^2)\pi_k},$$

其中分母是全概率公式给出的边缘密度 $p(w_i)$. 这正是(9.24). 对 j 求和时分子之和等于分母, 故 $\sum_j \gamma_j(w_i) = 1$.

□

注

(重构新增)命题 9.5中的 $\gamma_j(w_i)$ 就是混合模型中的“责任度” (responsibility): 它衡量第 j 个分量对权重 w_i 的解释占多大比例. 与硬分配不同,$\gamma_j(w_i)$ 是 $[0, 1]$ 中的连续值, 因此一个权重可以同时“属于”多个组, 只是权重不同. 这个量在后续梯度中反复出现:(9.25)中的拉力以 $\gamma_j(w_i)$ 加权,(9.31)则让 π_j 趋向于 γ_j 的平均值. 责任度依赖当前参数, 因此每步梯度下降都要重新计算它; 这正是 EM 算法与梯度下降在这里的联系.

总误差函数关于权重的导数由下式给出 (见练习 9.9).

命题 9.6 (软权重共享的梯度)

在(9.21)至(9.23)的记号下, 记 $\gamma_j(w_i)$ 由(9.24)给出. 则

$$\frac{\partial \widetilde{E}}{\partial w_i} = \frac{\partial E}{\partial w_i} + \lambda \sum_j \gamma_j(w_i) \frac{w_i - \mu_j}{\sigma_j^2}, \tag{9.25}$$

$$\frac{\partial \widetilde{E}}{\partial \mu_j} = \lambda \sum_i \gamma_j(w_i) \frac{\mu_j - w_i}{\sigma_j^2}. \tag{9.26}$$

为处理方差的正性约束, 引入新变量 ξ_j 使

$$\sigma_j^2 = \exp(\xi_j), \tag{9.27}$$

并对 ξ_j 作无约束最小化. 相应的导数为

$$\frac{\partial \widetilde{E}}{\partial \xi_j} = \frac{\lambda}{2} \sum_i \gamma_j(w_i) \left(1 - \frac{(w_i - \mu_j)^2}{\sigma_j^2}\right). \tag{9.28}$$

证明

(重构新增) 记 $S_i = \sum_k \pi_k \mathcal{N}(w_i \mid \mu_k, \sigma_k^2)$, 则 $\Omega = -\sum_i \ln S_i$, 且 $\pi_j \mathcal{N}(w_i \mid \mu_j, \sigma_j^2)/S_i = \gamma_j(w_i)$.

先证(9.25). 由

$$\frac{\partial}{\partial w_i} \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = -\frac{w_i - \mu_j}{\sigma_j^2} \mathcal{N}(w_i \mid \mu_j, \sigma_j^2),$$

得

$$\frac{\partial \Omega}{\partial w_i} = -\frac{1}{S_i} \sum_j \pi_j \left(-\frac{w_i - \mu_j}{\sigma_j^2}\right) \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = \sum_j \gamma_j(w_i) \frac{w_i - \mu_j}{\sigma_j^2}.$$

乘以 λ 并与 $\partial E/\partial w_i$ 相加即得(9.25).

再证(9.26). 由

$$\frac{\partial}{\partial \mu_j}\mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = \frac{w_i - \mu_j}{\sigma_j^2}\mathcal{N}(w_i \mid \mu_j, \sigma_j^2),$$

得

$$\frac{\partial \Omega}{\partial \mu_j} = -\sum_i \frac{1}{S_i}\pi_j \frac{w_i - \mu_j}{\sigma_j^2}\mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = -\sum_i \gamma_j(w_i)\frac{w_i - \mu_j}{\sigma_j^2} = \sum_i \gamma_j(w_i)\frac{\mu_j - w_i}{\sigma_j^2},$$

乘以 λ 即得(9.26).

最后证(9.28). 由 $\sigma_j^2 = e^{\xi_j}$,

$$\ln \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = -\frac{1}{2}\ln(2\pi) - \frac{1}{2}\xi_j - \frac{(w_i - \mu_j)^2}{2e^{\xi_j}}.$$

对 ξ_j 求导得

$$\frac{\partial}{\partial \xi_j}\ln \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = -\frac{1}{2} + \frac{(w_i - \mu_j)^2}{2\sigma_j^2} = \frac{1}{2}\left(\frac{(w_i - \mu_j)^2}{\sigma_j^2} - 1\right).$$

于是

$$\frac{\partial \Omega}{\partial \xi_j} = -\sum_i \gamma_j(w_i)\frac{\partial}{\partial \xi_j}\ln \mathcal{N}(w_i \mid \mu_j, \sigma_j^2) = \frac{1}{2}\sum_i \gamma_j(w_i)\left(1 - \frac{(w_i - \mu_j)^2}{\sigma_j^2}\right),$$

乘以 λ 即得(9.28).

□

注

(重构新增)命题 9.6的四个公式可以统一读成“以责任度为权重的加权矩估计”:(9.25)把每个 w_i 拉向第 j 个中心, 拉力与 $\gamma_j(w_i)$ 成正比、与方差 σ_j^2 成反比;(9.26)让 μ_j 移向被分配到该分量的权重的加权平均;(9.28)让 σ_j^2 移向 $(w_i - \mu_j)^2$ 的加权平均. 取 $\xi_j = \ln \sigma_j^2$ 而不是直接优化 σ_j^2, 是为了把正性约束变成无约束问题, 同时使梯度在 $\sigma_j^2 \to 0$ 时仍保持有限. 注意这些公式只给出驻点条件, 不保证全局最优; 混合模型的对数似然非凸, 初值对结果影响很大.

因此, 正则化项的作用是将每个权值拉向第 j 个 Gauss 分布的中心, 其力度与给定权重的 Gauss 分布的后验概率成正比. 这正是我们想要的结果.

关于 Gauss 分布中心的误差导数也很容易计算得出 (见练习 9.10), 即(9.26). 这个过程会驱使 μ_j 趋向于相应中心 μ_j 周围的权重偏差平方的加权平均值, 其中权重系数由每个权重的分量 j 生成的后验概率给出.

为了确保方差 σ_j^2 为正, 我们引入(9.27)定义的新变量 ξ_j, 并且对 ξ_j 进行无约束最小化. 相关导数为(9.28)(见练习 9.11).

关于混合系数 π_j 的导数, 我们需要考虑如下约束条件:

$$\sum_j \pi_j = 1, \qquad 0 \leqslant \pi_j \leqslant 1. \tag{9.29}$$

这些约束条件源于对 π_j 作为先验概率的解释. 我们可以通过使用下式给出的 softmax 函数, 将混合系数表示为一组辅助变量 η_j 来实现对它的约束:

$$\pi_j = \frac{\exp(\eta_j)}{\sum_k \exp(\eta_k)}. \tag{9.30}$$

命题 9.7 (混合系数的梯度)

在(9.29)与(9.30)下, 记 $\gamma_j(w_i)$ 由(9.24)给出, 则总误差函数关于辅助变量 η_j 的导数为

$$\frac{\partial \widetilde{E}}{\partial \eta_j} = \lambda \sum_i \left(\pi_j - \gamma_j(w_i)\right). \tag{9.31}$$

并且 π_j 将趋向于混合分量 j 的平均后验概率 $\dfrac{1}{M}\sum_i \gamma_j(w_i)$.

证明

(重构新增) 先求 softmax 的导数. 由 $\pi_j = e^{\eta_j} / \sum_k e^{\eta_k}$,

$$\frac{\partial \pi_k}{\partial \eta_j} = \frac{\delta_{kj} e^{\eta_j} \sum_l e^{\eta_l} - e^{\eta_k} e^{\eta_j}}{\left(\sum_l e^{\eta_l}\right)^2} = \delta_{kj}\pi_j - \pi_j \pi_k.$$

再求 Ω 对 π_k 的导数. 由 $\Omega = -\sum_i \ln S_i$、$S_i = \sum_l \pi_l \mathcal{N}_{il}$,

$$\frac{\partial \Omega}{\partial \pi_k} = -\sum_i \frac{\mathcal{N}_{ik}}{S_i} = -\sum_i \frac{\gamma_k(w_i)}{\pi_k}.$$

由链式法则及 $\sum_k \gamma_k(w_i) = 1$,

$$\begin{aligned}\frac{\partial \widetilde{E}}{\partial \eta_j} &= \lambda \sum_k \frac{\partial \Omega}{\partial \pi_k} \frac{\partial \pi_k}{\partial \eta_j} = \lambda \sum_i \sum_k \left(-\frac{\gamma_k(w_i)}{\pi_k}\right)\left(\delta_{kj}\pi_j - \pi_j \pi_k\right) \\ &= \lambda \sum_i \left(-\gamma_j(w_i) + \pi_j \sum_k \gamma_k(w_i)\right) = \lambda \sum_i \left(\pi_j - \gamma_j(w_i)\right).\end{aligned}$$

这正是(9.31). 令其为零得 $\pi_j = \frac{1}{M}\sum_i \gamma_j(w_i)$($M$ 为权重总数), 即 π_j 的驻点是责任度的平均值.

□

注

(重构新增)命题 9.7表明混合系数的学习规则与 EM 的 M 步完全一致:π_j 更新为平均责任度. 把 π_j 参数化为 softmax 而不是直接约束, 好处是梯度无约束、自动满足 $\pi_j \in (0,1)$ 且和为 1; 代价是参数化不可识别——给所有 η_j 同时加常数不改变 π_j, 因此 Hessian 至少有一个零特征值, 优化时需固定一个参考分量或使用带正则的 softmax. 责任度 $\gamma_j(w_i)$ 同时出现在权重、均值、方差和混合系数的梯度中, 这解释了为什么软权重共享是一个“自洽”的聚类过程: 参数更新改变责任度, 责任度又反过来改变参数更新.

可以看到,π_j 将趋向于混合分量 j 的平均后验概率.

软权重共享的另一种应用 (Lasserre,Bishop and Minka,2006) 介绍了一种原则性方法: 将生成式模型的无监督训练与相应判别模型的有监督训练相结合. 这在有大量未标记数据而标记数据短缺的情况下是有用的. 生成式模型的优势在于, 所有的数据都可以用来确定模型参数, 而判别模型只有标记数据直接为其提供参数. 当存在模型设定误差时, 也就是当模型没能准确描述所生成数据的真实分布时 (通常情况下是这样), 判别模型具备更好的泛化能力. 通过引入两个模型参数的软捆绑, 我们得到了一个良定的生成式和判别式的混合方法, 从而可以鲁棒地对模型设定误差进行建模. 此外, 还能够从对模型在未标记数据上进行训练中受益.

9.5 残差连接

深度神经网络 (简称深度网络) 的表征能力 (也称表示能力) 在很大程度上源于模型的多层处理. 据观察, 增加网络层数可以显著提高泛化能力. 我们也看到了批量归一化, 以及对权重和偏置进行初始化, 可以帮助解决深度神经网络中梯度消失或梯度爆炸的问题 (参见小节 7.4.2和小节 7.2.5). 然而即使使用批量归一化, 训练难度也仍然会随着网络层数的增多而增加.

这种现象的一种解释称为**破碎梯度** (shattered gradient)(Balduzzi et al.,2017). 我们已经看到, 神经网络的表征能力随着网络深度呈指数级增长. 使用 ReLU 激活函数, 网络可以表示的线性区域的数量呈指数级增长. 然而, 这样做的一个后果是误差函数梯度中的不连续现象激增.图 9.12在具有单输入变量和单输出变量的网络上对这一现象进行了说明. 在这里, 输出变量对输入变量的导数 (网络的 Jacobian 矩阵) 是输入变量的函数. 从微积分的链式法则来看, 这些导数决定了误差表面的梯度. 我们看到, 对于深度神经网络, 网络浅层的权重参数即便发生极其微小的变化, 也会产生显著的梯度变化. 基于梯度的迭代式优化算法假设梯度在参数空间中平滑变化, 因此这种“破碎梯度”效应会使训练对非常深的网络无效 (参见节 6.3).

定义 9.18 (破碎梯度)

设网络的输出 $\boldsymbol{y}$ 对输入 $\boldsymbol{x}$ 的 Jacobian 矩阵记为 $\boldsymbol{J}(\boldsymbol{x}) = \partial \boldsymbol{y}/\partial \boldsymbol{x}$. 若 $\boldsymbol{J}(\boldsymbol{x})$ 作为 $\boldsymbol{x}$ 的函数在输入空间中存在大量不连续点, 以致误差函数关于浅层参数的梯度随参数的微小变化而剧烈变化, 则称这一现象为**破碎梯度** (shattered gradient).

♣

注

(重构新增)定义 9.18描述的是误差曲面在参数空间中的“高频振荡”, 而不是梯度消失或爆炸. 三者的区别是: 梯度消失/爆炸指梯度的幅度随深度指数衰减或增长; 破碎梯度指梯度的方向与符号在相邻参数点之间剧烈跳变, 幅度未必异常.ReLU 网络把输入空间划分成指数多个线性区域, 跨越区域边界时 Jacobian 不连续, 浅层参数的一点点移动就可能把某个输入点从“一侧”推到“另一侧”, 从而改变该点的梯度贡献. 基于梯度的优化隐含假设局部梯度能代表邻域内的下降方向; 当这个假设失效时, 步长再小也可能无效.

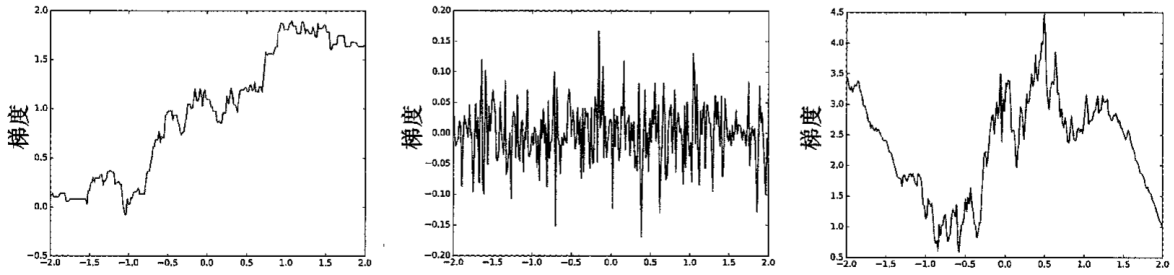


图 9.12: 单输入单输出网络的 Jacobian 矩阵的示意图.(a) 两层权重网络;(b) 25 层权重网络;(c) 带有残差连接的 51 层权重网络 [经 Balduzzi et al.(2017) 授权使用]

残差连接 (residual connection)(He et al.,2015a) 是对神经网络架构的一个重要修改, 它对训练极深的网络有很大帮助. 残差连接是一种特殊形式的**跳层连接** (skip-layer connection). 考虑一个包含如下三层的神经网络:

$$\boldsymbol{z}_1 = \boldsymbol{F}_1(\boldsymbol{x}), \tag{9.32}$$

$$\boldsymbol{z}_2 = \boldsymbol{F}_2(\boldsymbol{z}_1), \tag{9.33}$$

$$\boldsymbol{y} = \boldsymbol{F}_3(\boldsymbol{z}_2). \tag{9.34}$$

这里的函数 $\boldsymbol{F}_i(\cdot)$ 可能仅由一个线性变换和一个 ReLU 激活函数组成, 也可能很复杂, 比如包含多个线性层、激活函数和归一化层. 残差连接会将每个函数的输入添加到函数的输出, 从而有

$$\boldsymbol{z}_1 = \boldsymbol{F}_1(\boldsymbol{x}) + \boldsymbol{x}, \tag{9.35}$$

$$\boldsymbol{z}_2 = \boldsymbol{F}_2(\boldsymbol{z}_1) + \boldsymbol{z}_1, \tag{9.36}$$

$$\boldsymbol{y} = \boldsymbol{F}_3(\boldsymbol{z}_2) + \boldsymbol{z}_2. \tag{9.37}$$

定义 9.19 (残差连接与残差网络)

设某层的输入为 $\boldsymbol{z}$, 该层实现的映射为 $\boldsymbol{F}$. 若该层的输出取为 $\boldsymbol{z}' = \boldsymbol{F}(\boldsymbol{z}) + \boldsymbol{z}$, 则称 $\boldsymbol{F}(\boldsymbol{z}) + \boldsymbol{z}$ 为**残差块** (residual block),$\boldsymbol{F}$ 为**残差函数**, 恒等通路 $\boldsymbol{z} \mapsto \boldsymbol{z}$ 为**残差连接** (residual connection), 它是**跳层连接** (skip-layer connection) 的一种特殊形式. 由多个残差块顺序堆叠而成的网络称为**残差网络** (residual network), 也称 ResNet.

♣

注

(重构新增)定义 9.19的关键是恒等通路不经过任何非线性变换: 若 $\boldsymbol{F}$ 的输出接近零, 整个块就接近恒等映射. 这带来两个后果. 其一是“深网络至少不劣于浅网络”: 把新增的块设成恒等映射, 深层网络的表示能力包含浅层网络, 因此增加深度不会降低训练集上的最优误差. 其二是梯度的“高速公路”: 反向传播时梯度可以经由恒等通路直接流过, 不必逐层乘以 Jacobian, 从而缓解梯度消失. 注意残差连接与定义 9.15是两类不同的结构先验: 前者改变网络的函数形式 (增加并行恒等通路), 后者减少独立参数的数量.

每个函数和残差连接的组合, 如 $\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x}$, 称为残差块. 残差网络也称 ResNet, 由多层这样的块按顺序组成. 通过残差连接修改后的网络如图 9.13所示. 如果非线性函数中的参数足够小, 使函数输出接近于零, 则残差块可以很容易地生成恒等变换.

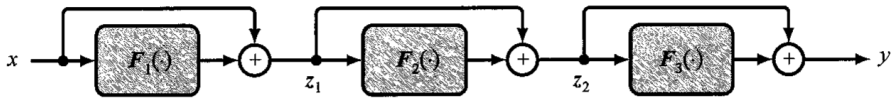


图 9.13: 一个由三个残差块组成的残差网络, 对应于(9.35) ~ (9.37)所示的转换序列

“残差”一词指的是, 在每个块中, 函数学习恒等映射和期望输出之间的残差, 我们可以通过整理残差变换的等式得到:

$$\boldsymbol{F}_i(\boldsymbol{z}_{i-1})=\boldsymbol{z}_i-\boldsymbol{z}_{i-1}. \tag{9.38}$$

与标准的深度网络相比, 具有残差连接的网络中的梯度对输入值的敏感度要低得多, 如图 9.12(c) 所示.

Li 等人 (2017) 开发了一种直接可视化误差表面的方法, 结果表明, 残差连接的作用是创建更平滑的误差表面, 如图 9.14所示. 残差网络中通常包含批量归一化层, 它们一同显著地减少了梯度消失和梯度爆炸的问题.He 等人 (2015a) 证明了残差连接可以使非常深的网络 (可能有数百层) 得到有效的训练.

如果我们结合(9.35) ~ (9.37)给出整个网络的统一总体方程, 就可以进一步深入了解残差连接促使误差表面变得平滑的方式:

$$\boldsymbol{y}=\boldsymbol{F}_3(\boldsymbol{F}_2(\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})+\boldsymbol{z}_1)+\boldsymbol{z}_2. \tag{9.39}$$

命题 9.8 (残差网络的展开)

在(9.35)至(9.37)中, 输出可先写成(9.39), 进而写成

$$\begin{aligned}\boldsymbol{y}=&\boldsymbol{F}_3(\boldsymbol{F}_2(\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})+\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})\\&+\boldsymbol{F}_2(\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})+\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x}.\end{aligned} \tag{9.40}$$

即 $\boldsymbol{y}$ 是四个并行子网络的输出之和: 深度为 3 的 $\boldsymbol{F}_3\circ(\boldsymbol{F}_2+\mathrm{id})\circ(\boldsymbol{F}_1+\mathrm{id})$、深度为 2 的 $\boldsymbol{F}_2\circ(\boldsymbol{F}_1+\mathrm{id})$、深度为 1 的 $\boldsymbol{F}_1$ 以及恒等映射 $\boldsymbol{x}$.

证明

(重构新增) 由(9.35),$\boldsymbol{z}_1=\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x}$; 由(9.36),$\boldsymbol{z}_2=\boldsymbol{F}_2(\boldsymbol{z}_1)+\boldsymbol{z}_1$. 把 $\boldsymbol{z}_1$ 代入 $\boldsymbol{z}_2$ 得

$$\boldsymbol{z}_2=\boldsymbol{F}_2(\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})+\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x}.$$

再把 $\boldsymbol{z}_2$ 代入(9.37),

$$\begin{aligned}\boldsymbol{y}&=\boldsymbol{F}_3(\boldsymbol{z}_2)+\boldsymbol{z}_2\\&=\boldsymbol{F}_3(\boldsymbol{F}_2(\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})+\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})+\boldsymbol{F}_2(\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x})+\boldsymbol{F}_1(\boldsymbol{x})+\boldsymbol{x},\end{aligned}$$

这正是(9.40). 逐项看, 第一项是 $\boldsymbol{F}_3$ 作用于 $\boldsymbol{z}_2$, 第二项是 $\boldsymbol{F}_2$ 作用于 $\boldsymbol{z}_1$, 第三项是 $\boldsymbol{F}_1$ 作用于 $\boldsymbol{x}$, 第四项是 $\boldsymbol{x}$ 本身, 因此可读成四个深度不同的并行子网络之和.

□

注

(重构新增)命题 9.8是一个纯代数恒等式, 不是近似: 它说明残差网络可以精确地写成“多个浅层子网络之和”. 这解释了两件事. 第一, 普通深度网络是它的特例: 令 $\boldsymbol{F}_i(z) = \boldsymbol{G}_i(z) - z$, 则残差网络退化为普通复合 $\boldsymbol{G}_3 \circ \boldsymbol{G}_2 \circ \boldsymbol{G}_1$. 第二, 误差曲面被“平均”得更平滑: 每个子网络的深度都较小, 其梯度不易破碎; 把它们相加后, 总梯度是各子网络梯度的和, 浅层子网络的梯度不经过深层 Jacobian, 从而为优化提供了更直接的方向. 但要注意, 这个展开并不意味着残差网络等价于一个浅层网络:$\boldsymbol{F}_3$ 仍然复合了 $\boldsymbol{F}_2$ 与 $\boldsymbol{F}_1$, 参数在各子网络之间共享, 因此表示能力并未退化为四个独立浅层网络的和.

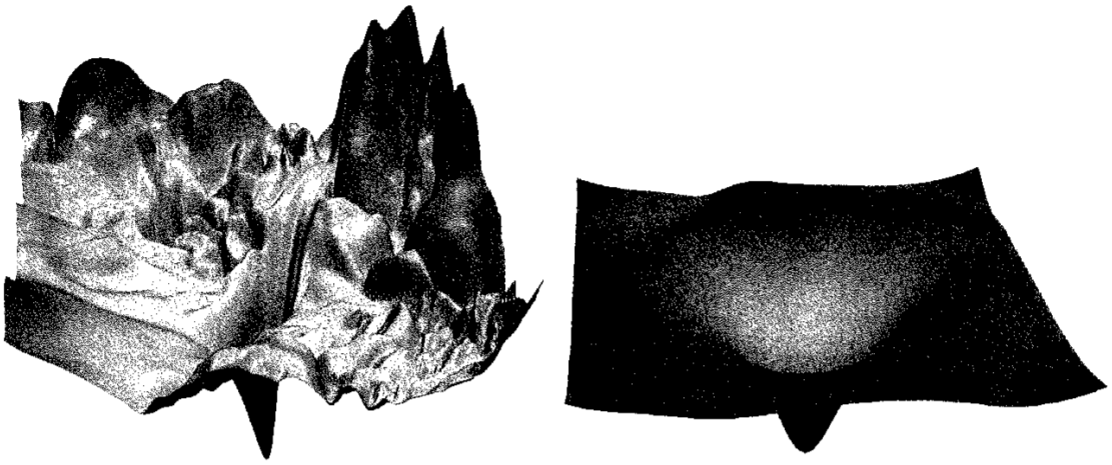


图 9.14: (a) 一个 56 层网络的误差表面的可视化效果.(b) 包含残差连接的相同网络的误差表面, 此处显示了残差连接的平滑效果 [经 Li et al.(2017) 授权使用]

图 9.15描述了残差网络的这种扩展形式. 可以看到, 函数整体由多个并行的网络组成, 每个网络层数较少. 该网络具有深度网络的表征能力, 因为深度网络是它的一个特例. 误差表面被浅层和深层子网络的组合所平滑.

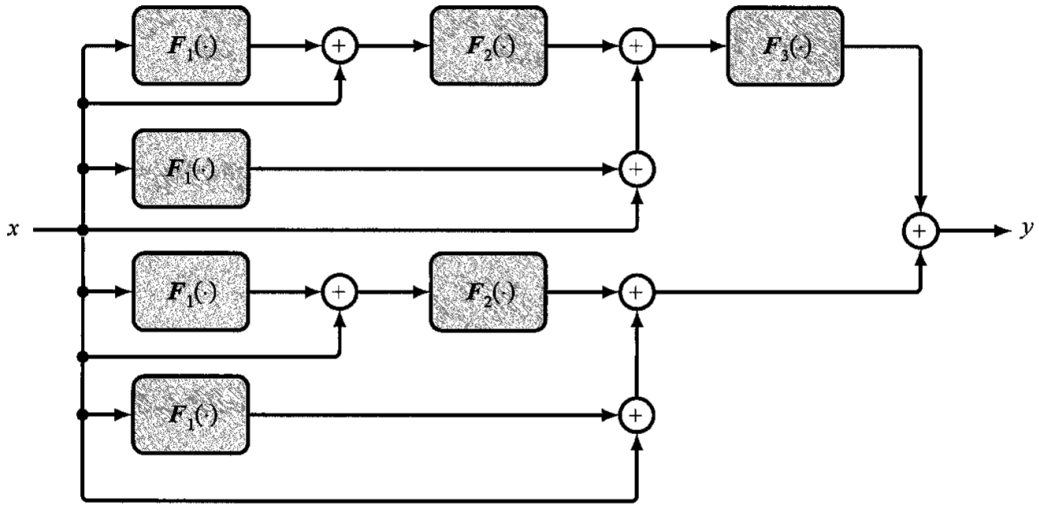


图 9.15: 以扩展形式显示图 9.13所示的残差网络

(9.40)定义的跳层连接要求输入变量和所有中间变量具有相同的维度, 以使它们能够相加. 我们可以通过添加以下形式的可学习参数的非方阵 $\boldsymbol{W}_s$ 来改变网络中某处的维度:

$$z_i = \boldsymbol{F}_i(z_{i-1}) + \boldsymbol{W}_s z_{i-1}. \tag{9.41}$$

定义 9.20 (投影跳层连接)

当残差通路的输入与输出维数不一致时, 引入可学习矩阵 $\boldsymbol{W}_s$, 把(9.40)中的恒等通路替换为线性映射 $z_{i-1} \mapsto \boldsymbol{W}_s z_{i-1}$, 其中 $\boldsymbol{W}_s$ 的形状使 $\boldsymbol{W}_s z_{i-1}$ 与 $\boldsymbol{F}_i(z_{i-1})$ 同维. 称 $\boldsymbol{W}_s$ 为**投影矩阵**, 相应的连接为**投影跳层连接** (projection shortcut).

♣

注

(重构新增)定义 9.20是残差连接在维数不匹配时的最小修改: 用线性投影把跳层通路的向量映射到与残差同维, 再相加. 它引入的额外参数很少 (一个矩阵), 且当 $\boldsymbol{W}_s$ 取为单位阵的嵌入时退化为恒等连接. 选择线性投影而不是非线性变换, 是为了保持"恒等通路尽量简单"的初衷: 投影只负责对齐坐标, 不负责提取特征. 在卷积网络中,$\boldsymbol{W}_s$ 通常实现为 1×1 卷积, 用于在通道数变化时对齐通道维.

到目前为止, 我们还没有指定可学习非线性函数 $\boldsymbol{F}_i(\cdot)$ 的具体形式. 最简单的选择是使用一个标准的前馈网络, 它由可学习的线性变换层和固定的非线性激活函数 (如 ReLU 函数) 层交替构成. 这为残差连接的放置提供了两种可能, 如图 9.16所示: 在版本 (a) 中, 被加的量总是非负的, 因为它们是由 ReLU 层的输出给出的, 因此为了允许被加的量可正可负, 版本 (b) 更常用.

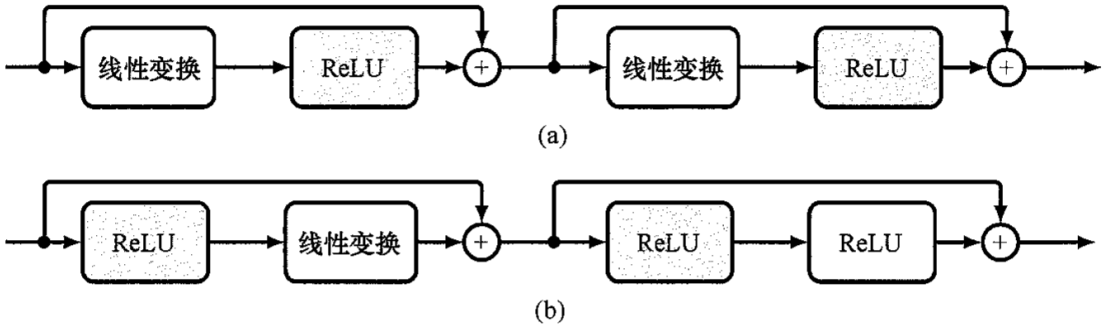


图 9.16: 将残差连接加入标准前馈网络的两种可选方法, 该网络由可学习的线性变换层和非线性激活函数 (如 ReLU 函数) 层交替构成

9.6 模型平均

如果我们训练了几个不同的模型来解决相同的问题, 而不是尝试选择单一的最佳模型, 那么我们通常可以通过平均单个模型所做的预测来提高泛化能力. 这种模型整合方式有时称为**委员会**或**集成** (ensemble). 对于产生概率输出的模型, 预测分布是每个模型预测的平均值:

$$p(y \mid \boldsymbol{x}) = \frac{1}{L} \sum_{l=1}^{L} p_l(y \mid \boldsymbol{x}), \tag{9.42}$$

其中 $p_l(y \mid \boldsymbol{x})$ 是模型 l 的输出,L 是模型的总数.

定义 9.21 (集成与委员会)

设 L 个模型 $p_l(y \mid \boldsymbol{x})(l = 1, \cdots, L)$ 针对同一预测任务训练. 若以(9.42)的算术平均作为组合模型的预测分布, 则称该组合为**集成** (ensemble) 或**委员会** (committee), 每个 p_l 称为一个**基模型** (base model) 或**成员** (member).

♣

注

(重构新增)定义 9.21只规定了“如何组合”, 没有规定“成员从何而来”; 不同的成员生成方式对应不同的集成方法 (袋装、不同架构、不同初始化、dropout 等). 平均预测之所以有用, 是因为各成员的误差中有一部分相互抵消: 若误差完全独立, 平均后的期望平方误差按 $1/L$ 缩小 (见定理 9.4); 若误差高度相关, 收益就小得多. 这也说明集成的代价: 训练 L 个模型并使它们“足够不同”需要额外的计算与设计, 而成员越相似, 收益越接近零.

这种平均过程的动机源于偏差和方差之间的权衡. 回想一下前面关于偏差–方差的示意图 (参见节 4.3), 当我们使用正弦数据训练多个多项式, 然后对得到的函数进行平均时, 方差项产生的贡献几乎会被抵消, 从而提升预测性能. 当然, 在实践中, 我们只有一个数据集, 因此我们必须找到一种方法, 以便在集成模型 (一些旨在解决相同问题的模型的集合) 内部的不同模型之间引入可变性. 我们可以使用**自助** (bootstrap) 数据集, 其中多个数据集的创建过程如下. 假设我们的原始数据集由 N 个数据点 $\boldsymbol{X} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 组成. 我们可以通过从 $\boldsymbol{X}$ 中有放回地抽样 N 个数据点来创建一个新的数据集 $\boldsymbol{X}_B$, 这样 $\boldsymbol{X}$ 中的一些数据点可能在 $\boldsymbol{X}_B$ 中出现多次, 而 $\boldsymbol{X}$ 中的其他数据点则可能不包含在 $\boldsymbol{X}_B$ 中. 这个过程可以重复 L 次, 从而生成 L 个大小为 N 的数据集, 其中的每个数据集都是通过从原始数据集 $\boldsymbol{X}$ 中采样得到的. 每个数据集可以用来训练一个模型, 并对得到的模型的预测进行平均.

定义 9.22 (自助聚合与袋装)

设原始数据集为 $\boldsymbol{X} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$. 从 $\boldsymbol{X}$ 中有放回地独立抽取 N 个样本, 得到的多重集 $\boldsymbol{X}_B$ 称为**自助样本** (bootstrap sample). 重复 L 次得到 $\boldsymbol{X}_{B_1}, \cdots, \boldsymbol{X}_{B_L}$, 分别训练模型并对预测取平均, 称这一过程为**自助聚合** (bootstrap aggregation) 或**袋装** (bagging)(Breiman,1996). ♣

注

(重构新增)定义 9.22的要点是“有放回”: 每个自助样本平均包含原数据中约 $1 - 1/e \approx 63.2\%$ 的不同样本点, 其余约 36.8% 的点被遗漏. 这些被遗漏的点构成所谓的袋外 (out-of-bag) 样本, 可用于估计泛化误差而无需单独的验证集. 袋装主要降低方差, 对偏差几乎没有影响, 因此它最适合高方差、低偏差的基模型 (如深决策树); 对本身偏差很大的模型 (如线性回归), 袋装的收益有限. 成员之间的相关性来自共享的训练数据, 这是袋装收益通常小于 $1/L$ 的原因.

形成集成模型的另一种方法是使用原始数据集训练多个具有不同架构的模型.

我们可以通过考虑一个仅涉及输入向量 $\boldsymbol{x}$ 和单输出变量 y 的回归问题来分析集成预测的好处. 假设我们有一组训练好的模型 $y_1(\boldsymbol{x}), \cdots, y_M(\boldsymbol{x})$, 构造由下式给出的集成模型预测:

$$y_{\mathrm{COM}}(\boldsymbol{x}) = \frac{1}{M}\sum_{m=1}^{M} y_m(\boldsymbol{x}). \tag{9.43}$$

如果我们试图预测的真实函数由 $h(\boldsymbol{x})$ 给出, 那么每个模型的输出都可以写成真实值加上一个误差的形式:

$$y_m(\boldsymbol{x}) = h(\boldsymbol{x}) + \varepsilon_m(\boldsymbol{x}). \tag{9.44}$$

平均平方和误差则采用以下形式:

$$\mathbb{E}_{\boldsymbol{x}}\left[\{y_m(\boldsymbol{x}) - h(\boldsymbol{x})\}^2\right] = \mathbb{E}_{\boldsymbol{x}}[\varepsilon_m(\boldsymbol{x})^2], \tag{9.45}$$

其中 $\mathbb{E}_{\boldsymbol{x}}[\cdot]$ 表示关于输入向量 $\boldsymbol{x}$ 的分布的期望. 因此, 集合中模型分别预测产生的平均误差为

$$E_{\mathrm{AV}} = \frac{1}{M}\sum_{m=1}^{M} \mathbb{E}_{\boldsymbol{x}}[\varepsilon_m(\boldsymbol{x})^2]. \tag{9.46}$$

类似地, 来自集成模型(9.43)的期望误差由下式给出:

$$\begin{aligned} E_{\mathrm{COM}} &= \mathbb{E}_{\boldsymbol{x}}\left[\left\{\frac{1}{M}\sum_{m=1}^{M} y_m(\boldsymbol{x}) - h(\boldsymbol{x})\right\}^2\right] \\ &= \mathbb{E}_{\boldsymbol{x}}\left[\left\{\frac{1}{M}\sum_{m=1}^{M} \varepsilon_m(\boldsymbol{x})\right\}^2\right]. \end{aligned} \tag{9.47}$$

如果我们假设误差均值为零且不相关, 则有

$$\mathbb{E}_{\boldsymbol{x}}[\varepsilon_m(\boldsymbol{x})] = 0, \tag{9.48}$$

$$\mathbb{E}_{\boldsymbol{x}}[\varepsilon_m(\boldsymbol{x})\varepsilon_l(\boldsymbol{x})] = 0, \qquad m \neq l. \tag{9.49}$$

定理 9.4 (集成误差的分解)

在(9.43)至(9.49)的记号与假设下, 集成的期望误差与成员的平均误差满足

$$E_{\mathrm{COM}} = \frac{1}{M} E_{\mathrm{AV}}. \tag{9.50}$$

♡

证明

(重构新增) 由(9.47),

$$E_{\mathrm{COM}} = \mathbb{E}_{\boldsymbol{x}}\left[\left(\frac{1}{M}\sum_{m=1}^{M}\varepsilon_m\right)^2\right] = \frac{1}{M^2}\sum_{m=1}^{M}\sum_{l=1}^{M}\mathbb{E}_{\boldsymbol{x}}[\varepsilon_m\varepsilon_l].$$

把双重和拆成对角项与非对角项. 对角项 $m = l$ 共 M 个, 每个为 $\mathbb{E}_{\boldsymbol{x}}[\varepsilon_m^2]$; 非对角项 $m \neq l$ 由(9.49)全为零. 于是

$$E_{\mathrm{COM}} = \frac{1}{M^2}\sum_{m=1}^{M}\mathbb{E}_{\boldsymbol{x}}[\varepsilon_m^2] = \frac{1}{M}\cdot\frac{1}{M}\sum_{m=1}^{M}\mathbb{E}_{\boldsymbol{x}}[\varepsilon_m^2] = \frac{1}{M}E_{\mathrm{AV}}.$$

其中最后一个等号用了(9.46);(9.48)在推导中未直接使用; 它保证 $\mathbb{E}[y_m] = h$, 从而在把 y_m 写成 $h + \varepsilon_m$ 时交叉项的期望为零. 在本推导中, 由于 h 与 $\boldsymbol{x}$ 无关且 $\frac{1}{M}\sum_{m} 1 = 1$, 该交叉项也自动消失.

□

注

(重构新增) 本节前段用 L 表示模型数, 回归分析部分改用 M, 二者含义相同.定理 9.4给出了集成收益的“理想上限”: 只有当成员误差两两不相关时, 平均才能把期望误差降到 $1/M$. 现实中成员共享训练数据与模型结构, 误差正相关, 收益远小于 $1/M$; 这也解释了为什么集成方法要刻意制造多样性 (不同的数据子集、初始化、架构或超参数). 定理的另一个含义是: 集成不改变偏差项 (各成员误差的均值仍可含系统性偏差), 它主要缩小方差. 因此不能用集成来挽救一个系统性偏差过大的模型; 若所有成员都以同样的方式错, 平均之后仍然错.

这个结果表明, 只需要对 M 个不同版本的模型进行平均, 就可以将模型的平均误差降低到原来的 M 分之一. 遗憾的是, 它依赖于一个关键假设, 即各个模型造成的误差是不相关的. 在实践中, 误差通常是高度相关的, 总体误差的减小通常要小得多. 然而可以证明, 预期的集成模型误差不会超过集合中模型的预期误差, 因此

$$E_{\mathrm{COM}} \leqslant E_{\mathrm{AV}}.$$

命题 9.9 (集成误差的 Jensen 上界)

对任意 $M \geqslant 1$ 与任意平方可积的误差 $\varepsilon_1, \cdots, \varepsilon_M$,(9.47)与(9.46)给出的 E_{COM} 与 E_{AV} 满足 $E_{\mathrm{COM}} \leqslant E_{\mathrm{AV}}$, 无需误差不相关或均值为零的假设.

♠

证明

(重构新增) 令 $Z = \frac{1}{M}\sum_{m=1}^{M}\varepsilon_m(\boldsymbol{x})$. 由凸函数 $f(z) = z^2$ 的 Jensen 不等式,

$$E_{\mathrm{COM}} = \mathbb{E}_{\boldsymbol{x}}[Z^2] \leqslant \mathbb{E}_{\boldsymbol{x}}\left[\frac{1}{M}\sum_{m=1}^{M}\varepsilon_m^2\right] = \frac{1}{M}\sum_{m=1}^{M}\mathbb{E}_{\boldsymbol{x}}[\varepsilon_m^2] = E_{\mathrm{AV}},$$

其中第一个不等号就是 $(\mathbb{E}Z)^2 \leqslant \mathbb{E}[Z^2]$ 的加权形式, 即 $\left(\frac{1}{M}\sum \varepsilon_m\right)^2 \leqslant \frac{1}{M}\sum \varepsilon_m^2$ 对每个 $\boldsymbol{x}$ 逐点成立后再取期望.

□

注

(重构新增)命题 9.9说明“平均不会比成员更差”是一个只依赖凸性的普遍事实, 与误差是否独立无关. 它与定理 9.4的区别在于: 后者给出 $1/M$ 的定量缩小, 但需要不相关假设; 前者只给不等式, 却在任意相关结构下成立. 上界通常不紧: 当所有 ε_m 完全相同时,$E_{\mathrm{COM}} = E_{\mathrm{AV}}$, 平均没有任何收益. 因此实际集成设计的目标不是“平均”本身, 而是让成员误差尽量去相关, 使实际值接近 $1/M$ 的理想值.

另一种称为**提升** (boosting)(Freund and Schapire,1996) 的模型组合方法, 将多个“基”分类器组合起来, 产生了一种性能明显优于任何基分类器的集成模型. 即使基分类器的表现只比随机好一点点, 提升也可以给出好的结果. 提升方法和前面讨论的集成模型方法 (如袋装) 的主要区别在于: 基分类器是按顺序训练的, 且每个基分类器都是使用数据集的加权形式进行训练的, 其中与每个数据点相关的权重系数取决于前一个分类器的性能. 特别是, 被其中一个基分类器误分类的点, 在训练序列中的下一个分类器时, 会被赋予更大的权重. 一旦所有分类器都训练完毕, 就可以对它们的预测通过加权多数投票方式进行整合 (见练习 9.15).

在实践中, 所有模型组合方法的主要缺点是必须训练多个模型, 然后必须对所有模型的预测进行评估, 因而增加了训练和推理的计算开销, 开销的大小取决于具体的应用场景.

9.6.1 dropout

dropout(Srivastava et al.,2014) 是一种使用广泛且非常有效的正则化形式, 它可以视为一种在不需要单独训练多个模型的情况下, 对指数级数量规模的模型隐式地进行模型平均的方法. 它具有广泛的适用性, 并且计算开销低.

定义 9.23 (dropout)

设网络有 M 个非输出节点. 对每个训练数据点与每个非输出节点, 独立地抽取掩码 $R_i \in \{0,1\}$, 其中 $R_i \sim \mathrm{Bernoulli}(p)$, 并把节点 i 的输出乘以 R_i(等价于删除该节点及其全部连接). 称这种在训练时随机剪枝网络、并以保留概率 p 控制剪枝强度的正则化方法为 **dropout**.p 称为**保留概率** (keep probability). ♣

注

(重构新增)定义 9.23中的随机性只出现在训练阶段; 测试阶段通常使用完整网络并做相应的缩放 (见命题 9.10). 保留概率 p 是每个节点独立的, 因此一次前向传播对应 2^M 个可能剪枝网络中的一个; 训练过程相当于在这个指数级模型集合上共享参数地做随机优化. 这也解释了为什么 dropout 训练时梯度噪声大、需要更多迭代: 每步只在一个随机子网络上计算梯度, 而参数要同时服务于所有子网络.p 的选择是一个偏差–方差权衡:p 太小 (剪枝太多) 会使网络欠拟合,p 太大则正则化效果减弱; 隐藏节点常用 $p = 0.5$, 输入节点常用 $p = 0.8$.

dropout 的核心思想是在训练过程中随机删除网络中的节点, 包括它们之间的连接. 每当一个新的数据点进入网络, 模型就会随机选择一些节点并忽略它们.图 9.17展示了一个简单的神经网络以及两个通过忽略节点子集剪枝网络的例子.

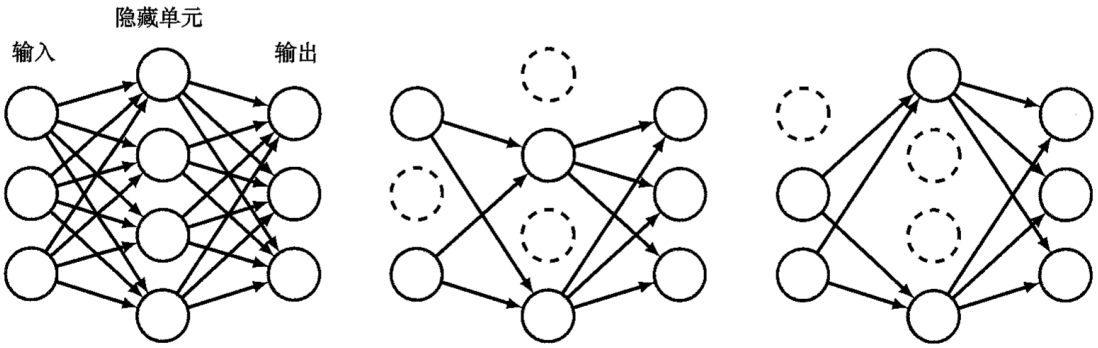


图 9.17: 一个简单的神经网络以及两个通过忽略节点子集剪枝网络的例子 (其中忽略了一个随机的节点子集)

dropout 可以同时应用于隐藏节点和输入节点, 但不能应用于输出节点, 这相当于将被删节点的输出设置为零.

它可以通过定义一个掩码向量 $\boldsymbol{R}_i \in \{0,1\}$ 来实现: 对于数据点 n, 该掩码将被乘以非输出节点 i, 其值以概率 p 设置为 1. 对于隐藏节点,$p = 0.5$ 很有效; 而对于输入节点, 通常使 $p = 0.8$.

在训练过程中, 当每个数据点进入网络时, 就会创建一个新的掩码, 并在该剪枝网络上应用前向和后向传播步骤来创建误差函数梯度, 然后用于更新权重, 例如采用随机梯度下降法. 如果数据点被分组成多个小批次, 则对梯度在每个小批次 (参见小节 7.2.4) 中的数据点上进行平均后, 再应用权重更新. 对于具有 M 个非输出节点的网络, 有 2^M 个剪枝网络, 因此在训练期间只会考虑这些网络中的一少部分. 这与传统的集成方法不同, 在传统的集成方法中, 集成中的每个网络都是独立训练至收敛的. 它们之间的另一个区别是, 指数级数量规模的网络隐式地用 dropout 进行训练, 它们不是独立的, 而是与完整网络共享它们的参数值, 因此剪枝网络间的参数也是彼此共享的. 需要注意的是, 使用 dropout 训练后, 单个参数更新包含较大噪声, 因此训练可能需要花费更长的时间. 此外, 由于误差函数本身是有噪声的, 仅通过在训练过程中减小误差函数, 很难确保优化算法正常工作.

一旦训练完成, 原则上就可以通过应用集成规则 [(9.42)] 来进行预测, 这种情况下的集成规则如下:

$$p(y \mid \boldsymbol{x}) = \sum_R p(R)\, p(y \mid \boldsymbol{x}, R), \tag{9.51}$$

其中, 求和是在掩码的指数级空间上进行的,$p(y \mid \boldsymbol{x}, R)$ 是来自具有掩码 R 的网络的预测分布. 因为这种求和难以处理, 所以可以通过对少量掩码进行采样来近似. 在实践中, 只需要 10 或 20 个掩码就足以获得良好的结果. 这一过程称为蒙特卡洛 dropout.

一种更简单的方法是在不掩盖训练好的网络的节点的情况下进行预测, 并重新缩放网络中的权重, 使每个节点在测试期间的预期输入与训练期间的输入大致相同, 以补偿在训练中会忽略一部分节点的事实. 因此, 如果一个节点在训练期间以概率 p 的形式存在, 那么在测试期间, 在使用网络进行预测之前, 需要对来自该节点的输出权重乘以 p.

命题 9.10 (dropout 的测试时缩放)

考虑一个节点, 其输出为 h, 并以权重 w 连接到下一层. 设训练时该节点以概率 p 保留, 即下一层收到的输入为 Rwh, 其中 $R \sim \text{Bernoulli}(p)$. 则在测试时把权重改为 $w' = pw$ 并关闭随机掩码, 下一层收到的输入期望与训练时相同, 即

$$\mathbb{E}[Rwh] = w'h = pwh.$$

因此, 按保留概率 p 缩放输出权重 (等价地, 缩放节点输出) 可以补偿训练时的随机丢弃.

证明

(重构新增) 由 R 的分布,$\mathbb{E}[R] = p$, 故 $\mathbb{E}[Rwh] = pwh$. 而使用缩放权重 $w' = pw$ 且不丢弃时, 下一层收到 $w'h = pwh$, 与训练时的期望相同. 若一个节点以多条边连接到下一层, 则对每条边的权重同时乘 p, 由线性性, 所有输入的期望之和保持不变.

□

注

(重构新增)命题 9.10只保证一阶矩 (期望) 匹配, 不保证方差匹配: 训练时下一层输入是随机变量 Rwh, 其方差为 $p(1-p)w^2h^2$; 测试时输入是确定值 pwh, 方差为零. 因此这种缩放是一种近似, 而不是精确的集成. 它也解释了为什么测试时不能直接使用未缩放的权重: 那样下一层输入的期望会变成 p 倍, 导致激活值整体偏小、逐层累积后分布严重偏移. 缩放的实现有两种等价写法: 训练时把保留节点的输出除以 p(inverted dropout), 测试时保持不变; 或训练时不动、测试时把权重乘 p. 现代框架通常采用前者, 因为它把缩放放在训练阶段、测试阶段无额外操作.

命题 9.11 (dropout 的期望误差)

考虑线性模型 $y_k = \sum_{i=1}^{D} w_{ki}x_i$ 与平方和误差

$$E(\boldsymbol{w}) = \sum_{n=1}^{N}\sum_{k=1}^{K}\left\{t_{nk} - \sum_{i=1}^{D} w_{ki}R_{ni}x_{ni}\right\}^2,$$

其中 $R_{ni} \sim \text{Bernoulli}(\rho)$ 相互独立. 则

$$\mathbb{E}[E(\boldsymbol{w})] = \sum_{n=1}^{N}\sum_{k=1}^{K}\left\{t_{nk} - \rho\sum_{i=1}^{D} w_{ki}x_{ni}\right\}^2 + \rho(1-\rho)\sum_{n=1}^{N}\sum_{k=1}^{K}\sum_{i=1}^{D} w_{ki}^2x_{ni}^2.$$

即 dropout 的期望误差是一个带输入依赖二次正则项的平方和误差: 第 i 个输入方向上的有效正则化系数为 $\rho(1-\rho)\sum_n x_{ni}^2$.

证明

(重构新增) 令 $\mu_{nk} = \sum_i w_{ki}x_{ni}$、$A_{nk} = t_{nk} - \rho\mu_{nk}$、$B_{nk} = \sum_i w_{ki}(R_{ni}-\rho)x_{ni}$. 则

$$t_{nk} - \sum_i w_{ki}R_{ni}x_{ni} = A_{nk} - B_{nk}, \qquad \mathbb{E}[B_{nk}] = 0.$$

由 $\mathbb{E}[(R_{ni}-\rho)(R_{nj}-\rho)] = \delta_{ij}\rho(1-\rho)$($i=j$ 时 $\mathbb{E}[R_{ni}^2] = \rho$,$i\neq j$ 时由独立性得 ρ^2, 故差为 $\delta_{ij}\rho(1-\rho)$),

$$\mathbb{E}[B_{nk}^2] = \sum_i\sum_j w_{ki}w_{kj}x_{ni}x_{nj}\,\mathbb{E}[(R_{ni}-\rho)(R_{nj}-\rho)] = \rho(1-\rho)\sum_i w_{ki}^2x_{ni}^2.$$

于是

$$\mathbb{E}\left[(A_{nk}-B_{nk})^2\right] = A_{nk}^2 + \mathbb{E}[B_{nk}^2],$$

对 n,k 求和即得所证.

□

注

(重构新增)命题 9.11说明 dropout 不是“无正则化的随机模型平均”, 而是在期望意义下等价于一个二次正则化问题: 正则项按输入分量的平方和 $\sum_n x_{ni}^2$ 加权, 因此输入尺度大的方向受到更强的收缩. 这也解释了为什么 dropout 对输入特征尺度敏感: 若某些输入的数值范围远大于其他输入, 它们对应的权重会被更重地惩罚. 把 R_{ni} 的独立性换成相关性会改变交叉项的系数, 从而改变有效正则化强度; 这是 dropout 与一般集成在理论上的主要差别.

应用 dropout 的另一种动机来自 Bayes 学派. 在按照 Bayes 学派的观点处理问题时, 我们将通过对所有的 2^M 个网络模型进行平均来预测, 其中的每个网络模型都由其后验概率加权 (参见节 2.6). 这样做的计算成本很高, 无论是在评估后验概率的训练期间, 还是在计算加权预测的测试期间.dropout 通过给每个可能的模型赋予相同的权重来近似这个模型平均.

dropout 具有良好表现背后的更深层次原因, 在于 dropout 在减少过拟合方面发挥了重要作用. 在一个标准的神经网络中, 参数可以调整为错误适应单个数据点, 其隐藏节点被过度特化. 给定其他节点的输出, 每个节点会调整其权重以最小化误差, 导致节点以一种可能无法泛化到新数据的方式共同调整参数. 使用 dropout, 每个节点不能依赖其他特定节点的存在, 而必须在广泛的场景下做出有用的贡献, 从而减少共同调整和特化. 对于使用最小二乘法训练的简单线性回归模型,dropout 正则化等价于二次正则化的一种修改形式 (见练习 9.18).

9.7 习题

练习 9.1 (★) 通过依次考虑小节 9.1.3介绍的 4 个公理, 证明以复合为运算, 绕正方形中心旋转 90° 的整数倍的变换集合构成一个群. 同样, 证明在二维平面上连续平移对象的变换也构成一个群 (参见小节 9.1.3).

✍ **练习 9.2** (★★) 考虑如下形式的线性模型:

$$y(\boldsymbol{x}, \boldsymbol{w}) = w_0 + \sum_{i=1}^{D} w_i x_i, \tag{9.52}$$

并考虑如下形式的平方和误差函数:

$$E_D(\boldsymbol{w}) = \frac{1}{2} \sum_{n=1}^{N} \{y(\boldsymbol{x}_n, \boldsymbol{w}) - t_n\}^2 . \tag{9.53}$$

将均值为 0、方差为 σ^2 的 Gauss 噪声 ε_i 独立地添加到每个输入变量 x_i 中. 利用 $\mathbb{E}[\varepsilon_i] = 0$ 和 $\mathbb{E}[\varepsilon_i \varepsilon_j] = \delta_{ij}\sigma^2$, 证明最小化在噪声分布上平均的 E_D 等价于最小化无噪声输入变量的平方和误差以及权重衰减正则化项的和, 其中偏置参数 w_0 已从正则化项中省略.

✍ **练习 9.3** (★★) 考虑一个仅由二次正则化项组成的误差函数

$$\Omega(\boldsymbol{w}) = \frac{1}{2} \boldsymbol{w}^{\mathsf{T}} \boldsymbol{w} \tag{9.54}$$

以及梯度下降更新公式

$$\boldsymbol{w}^{(\tau+1)} = \boldsymbol{w}^{(\tau)} - \eta \nabla \Omega(\boldsymbol{w}). \tag{9.55}$$

通过考虑无穷小更新, 写出 $\boldsymbol{w}$ 迭代的相应微分方程. 从一个初值 $\boldsymbol{w}_0$ 开始, 写出这个微分方程的解, 并证明 $\boldsymbol{w}$ 的元素指数衰减到零.

✍ **练习 9.4** (★) 验证由(9.6)和(9.7)定义的网络函数在应用于输入的转换 [(9.8)] 下是不变的, 前提是同时使用(9.9)和(9.10)转换权重和偏置. 同样, 通过将(9.12)和(9.13)应用到第二层权重和偏置, 证明网络输出可以根据(9.11)进行变换.

✍ **练习 9.5** (★★) 通过使用 Lagrange 乘子, 证明最小化(9.19)给出的正则化误差函数等价于在约束条件 [(9.20)] 下最小化未正则化的误差函数 $E(\boldsymbol{w})$, 并讨论参数 η 和 λ 之间的关系 (参见附录 C).

✍ **练习 9.6** (★★★) 考虑如下形式的二次误差函数:

$$E = E_0 + \frac{1}{2}(\boldsymbol{w} - \boldsymbol{w}^*)^{\mathsf{T}} \boldsymbol{H} (\boldsymbol{w} - \boldsymbol{w}^*), \tag{9.56}$$

其中 $\boldsymbol{w}^*$ 代表最小值,Hessian 矩阵 $\boldsymbol{H}$ 为正定常矩阵 (不随 $\boldsymbol{w}$ 变化). 假设选择初始权重向量 $\boldsymbol{w}^{(0)}$ 位于原点, 并使用简单梯度下降法进行更新:

$$\boldsymbol{w}^{(\tau)} = \boldsymbol{w}^{(\tau-1)} - \rho \nabla E, \tag{9.57}$$

其中 τ 表示步数,ρ 表示学习率 (假设学习率较小). 证明在 τ 步之后, 与 $\boldsymbol{H}$ 的特征向量平行的权重向量的分量可以写成

$$w_j^{(\tau)} = \left\{1 - (1 - \rho\eta_j)^{\tau}\right\} w_j^*, \tag{9.58}$$

其中 $w_j = \boldsymbol{u}_j^{\mathsf{T}} \boldsymbol{w}$、$w_j^* = \boldsymbol{u}_j^{\mathsf{T}} \boldsymbol{w}^*$, 而 $\boldsymbol{u}_j$ 和 η_j 分别是 $\boldsymbol{H}$ 的特征向量和特征值, 由下式给出:

$$\boldsymbol{H}\boldsymbol{u}_j = \eta_j \boldsymbol{u}_j. \tag{9.59}$$

证明当 $\tau \to \infty$ 时, 有 $\boldsymbol{w}^{(\tau)} \to \boldsymbol{w}^*$, 前提是 $|1 - \rho\eta_j| < 1$. 假设训练在有限个 τ 步之后停止, 证明与 $\boldsymbol{H}$ 的特征向量平行的权重向量的分量满足

$$w_j^{(\tau)} \approx w_j^*, \qquad \text{当} \eta_j \gg (\rho\tau)^{-1} \text{ 时}, \tag{9.60}$$

$$|w_j^{(\tau)}| \ll |w_j^*|, \qquad \text{当} \eta_j \ll (\rho\tau)^{-1} \text{ 时}. \tag{9.61}$$

这个结果表明 $(\rho\tau)^{-1}$ 在权重衰减中起着与正则化参数 λ 类似的作用.

✍ **练习 9.7** (★★) 考虑一个其中多个权重被约束为具有相同值的神经网络. 讨论在计算误差函数关于网络中可调参数的导数时, 如何修改标准的反向传播算法以确保满足此类约束.

✍ **练习 9.8** (★) 考虑下式定义的混合分布:

$$p(w) = \sum_{j=1}^{M} \pi_j \mathcal{N}\left(w \mid \mu_j, \sigma_j^2\right), \tag{9.62}$$

其中 π_j 可以视为对应 Gauss 分量的先验概率 $p(j)$. 使用 Bayes 定理证明相应的后验概率 $p(j \mid w)$ 由(9.24)给出.

练习 9.9 (★★) 使用(9.21) ~ (9.24)验证(9.25)的结果.

练习 9.10 (★★) 使用(9.21) ~ (9.24)验证(9.26)的结果.

练习 9.11 (★★) 使用(9.21) ~ (9.24)验证(9.28)的结果.

练习 9.12 (★★) 证明(9.30)定义的混合系数 π_k 相对于辅助参数 η_j 的导数由下式给出:

$$\frac{\partial \pi_k}{\partial \eta_j} = \delta_{kj}\pi_j - \pi_j\pi_k. \tag{9.63}$$

因此, 通过对所有 i 使用约束 $\sum_k \gamma_k(w_i) = 1$, 可以得到(9.31)的结果.

练习 9.13 (★) 验证(9.35) ~ (9.37)可以组合出一个形为(9.40)的涉及整个网络的单一整体方程.

练习 9.14 (★★) 对于一个简单的集成模型, 平均期望平方和误差 E_{AV} 可以用(9.46)来定义, 集成模型本身的期望误差由(9.47)给出. 假设个体误差满足(9.48)和(9.49), 请推导出(9.50)的结果.

练习 9.15 (★★) 利用 Jensen 不等式针对凸函数 $f(x) = x^2$ 的特殊情况, 证明(9.46)给出的简单集成模型成员的平均期望平方和误差 E_{AV} 和(9.47)给出的集成模型本身的期望误差 E_{COM} 满足

$$E_{\mathrm{COM}} \leqslant E_{\mathrm{AV}}. \tag{9.64}$$

练习 9.16 (★★) 利用 Jensen 不等式, 证明(9.64)适用于任何误差函数 $E(y)$, 而非仅仅适用于平方和误差函数, 只要它是 y 的凸函数即可.

练习 9.17 (★★) 考虑一个集成模型, 其允许各组成模型的权重不相等, 于是有

$$y_{\mathrm{COM}}(\boldsymbol{x}) = \sum_{m=1}^{M} \alpha_m y_m(\boldsymbol{x}). \tag{9.65}$$

为了确保预测 $y_{\mathrm{COM}}(\boldsymbol{x})$ 保持在合理的范围内, 假设我们要求其在 $\boldsymbol{x}$ 的每个取值上都由最小值和最大值限定, 这些边界可以由集成模型的任何成员给出, 从而

$$y_{\min}(\boldsymbol{x}) \leqslant y_{\mathrm{COM}}(\boldsymbol{x}) \leqslant y_{\max}(\boldsymbol{x}), \tag{9.66}$$

其中 $y_{\min}(\boldsymbol{x}) = \min_m y_m(\boldsymbol{x})$、$y_{\max}(\boldsymbol{x}) = \max_m y_m(\boldsymbol{x})$. 证明这一约束的充要条件是系数 α_m 满足

$$\alpha_m \geqslant 0 \quad \text{且} \quad \sum_{m=1}^{M} \alpha_m = 1. \tag{9.67}$$

练习 9.18 (★★★) 探索 dropout 正则化对使用最小二乘法训练的简单线性回归模型的影响. 考虑如下形式的模型:

$$y_k = \sum_{i=1}^{D} w_{ki} x_i, \tag{9.68}$$

并考虑下式给出的平方和误差函数:

$$E(\boldsymbol{w}) = \sum_{n=1}^{N}\sum_{k=1}^{K}\left\{t_{nk} - \sum_{i=1}^{D} w_{ki} R_{ni} x_{ni}\right\}^2, \tag{9.69}$$

其中 dropout 矩阵的元素 $R_{ni} \in \{0, 1\}$ 是从参数 ρ 的 Bernoulli 分布中随机采样得到的. 对随机 dropout 参数的分布取期望. 证明

$$\mathbb{E}[R_{ni}] = \rho, \tag{9.70}$$

$$\mathbb{E}[R_{ni}R_{nj}] = \delta_{ij}\rho + (1 - \delta_{ij})\rho^2. \tag{9.71}$$

然后证明 dropout 模型的期望误差函数由下式给出:

$$\begin{aligned}\mathbb{E}[E(\boldsymbol{w})] = &\sum_{n=1}^{N}\sum_{k=1}^{K}\left\{y_{nk} - \rho\sum_{i=1}^{D} w_{ki} x_{ni}\right\}^2 \\ &+ \rho(1-\rho)\sum_{n=1}^{N}\sum_{k=1}^{K}\sum_{i=1}^{D} w_{ki}^2 x_{ni}^2.\end{aligned} \tag{9.72}$$

可以看到,dropout 模型的期望误差函数对应于具有二次正则化项的平方和误差函数, 后者的正则化系数已根据输入值分别为每个输入变量做了缩放. 最后, 写下用于最小化这个正则化误差函数的权重矩阵的闭式解.

第 10 章 卷积网络

最简单的机器学习模型假设观测数据值是非结构化的, 这意味着我们事先对数据向量 $\mathbf{x} = (x_1, \cdots, x_D)$ 中各个元素之间可能存在的关系一无所知. 如果我们对这些变量进行随机排列, 并在所有训练数据和测试数据上一致地应用这种固定排列, 那么到目前为止我们所考虑过的模型的性能将不会有任何差异.

然而, 机器学习的许多应用涉及结构化数据 (structured data), 其中输入变量之间存在额外的关系. 例如, 自然语言中的单词形成一个序列 (sequence)(参见第 12 章): 如果我们将语言建模为一个生成式自回归过程, 那么我们将期望每个单词与直接邻近的单词更相关, 而不是依赖于序列中更早出现的单词. 同样, 图像的像素之间具有良定的空间关系, 其中输入变量被安排在二维网格中, 相邻像素具有高度相关的值.

我们已经可以通过在训练目标的误差函数中添加正则化项 (参见第 9 章), 以及通过数据增强或修改模型架构来利用我们对特定数据模态结构的知识. 这些方法可以帮助引导模型尊重输入数据在变换方面的某些属性, 如不变性和等变性 (参见小节 9.1.3). 在本章中, 我们将介绍一种称为卷积神经网络 (Convolutional Neural Network, CNN) 的架构方法.CNN 可以视为具有参数共享的稀疏连接多层网络, 并用于编码特定于图像数据的不变性和等变性.

10.1 计算机视觉

图像数据的自动分析和解释是计算机视觉 (computer vision) 领域的焦点, 代表了机器学习的一个主要应用领域 (Szeliski, 2022). 计算机视觉主要基于三维投影几何, 人们曾手工构建特征并用作简单学习算法的输入 (Hartley and Zisserman, 2004). 然而, 计算机视觉是最早被深度学习革命改变的领域之一, 这主要归功于 CNN 架构. 虽然该架构最初是在图像分析场景下开发的, 但它也被应用于序列数据分析等其他场景. 最近, 基于 Transformer(参见第 12 章) 的替代架构在一些应用中已经与 CNN 架构形成竞争态势.

机器学习在计算机视觉领域的应用有很多, 其中最常见的一些应用如下:

1. 图像分类, 例如将一幅皮肤病变的图像分类为良性或恶性 (参见图 1.1), 这有时称为 “图像识别” ;
2. 目标检测, 检测图像中的目标并确定它们在图像中的位置, 例如从自动驾驶汽车收集的相机数据中检测行人 (参见图 10.19);
3. 图像分割, 其中每个像素被单独分类, 从而将图像划分为共享相同标签的区域. 例如, 自然场景可能分割成天空、草地、树木和建筑物 (参见图 10.26), 而医学扫描图像可以分割成癌变组织和正常组织;
4. 标题生成, 即从图像中自动生成文本描述 (参见第 12 章);
5. 图像合成, 例如生成人脸图像 (参见图 1.3), 也可以根据描述所需图像内容的文本输入来合成图像;
6. 图像修补, 将图像的一个区域替换为与图像其余部分一致的合成像素, 这可用于在图像编辑过程中删除不需要的对象 (参见第 20 章);
7. 风格迁移, 即将一种风格的输入图像 (例如一张照片) 转换为不同风格的相应图像, 例如一幅油画 (参见图 10.32);
8. 图像超分, 通过增加像素数量以及与合成相关的高频信息来提高图像的分辨率 (参见第 20 章);
9. 深度预测, 即使用一个或多个视图来预测目标图像中每个像素处场景与相机的距离;
10. 场景重建, 即使用一个或多个场景的二维图像来重建三维表示.

10.1.1 图像数据

图像由像素的矩形数组组成, 其中每个像素要么具有灰度强度, 要么具有更常见的红、绿、蓝通道的三元组强度值. 这些强度值是非负数, 同时也有最大值, 对应于相机或用于捕捉图像的其他硬件设备的限制. 大多数情况下, 我们将强度视为连续变量, 但在实践中它们是用有限精度表示的, 例如用 8 比特表示范围 0 ~ 255 内的整数. 一些图像, 例如医学诊断中使用的磁共振成像 (Magnetic Resonance Imaging, MRI) 扫描图像, 是包含体素的三维网

格. 类似地, 视频包含一系列二维图像, 因此也可以视为三维结构, 其中连续的帧随着时间堆叠在一起.

定义 10.1 (图像数据)

称由像素 (pixel) 构成的矩形数组为**图像**, 其中每个像素携带一个灰度强度值, 或携带红 (R)、绿 (G)、蓝 (B) 三个通道的强度三元组. 若图像有 J 行、K 列、C 个通道, 则可用尺寸为 $J\times K\times C$ 的张量描述它, 其中 $C=1$ 对应灰度图像、$C=3$ 对应彩色图像. 强度取非负值且有上界, 实践中以有限精度表示, 例如以 8 比特整数表示 0 ~ 255 内的值. 若图像元素排列成三维网格而不是二维网格, 则称其元素为**体素** (voxel); 视频是沿时间轴堆叠的二维图像序列, 因而也构成三维结构. ♣

注

(重构新增) 这个定义把“图像”从直观的图片转成可直接送入网络的对象: 一个三阶张量. 定义中的三件事各有用处. 第一, 通道维 C 与空间维 $J\times K$ 分离, 使定义 10.8可以把“在空间上共享权重、在通道上求和”写成四阶滤波器张量. 第二, 强度有界 (如 0 ~ 255) 意味着输入落在紧集中, 于是梯度下降与梯度符号法 (定义 10.14) 中的像素更新必须再投影回该集合, 否则会越界. 第三, “视为连续变量还是有限精度”只是建模约定: 把强度视为连续变量是为了可微, 存储上的离散化不改变前向计算. 注意体素网格与视频序列虽然都是三维结构, 但第三维的语义分别是空间深度与时间, 滤波器在这两维上是否共享权重必须另行规定.

考虑在图像数据上应用神经网络实现上述应用所面临的挑战. 图像通常具有高维特性, 通过相机捕获的图像包含数千万像素. 因此, 将图像数据视为非结构化数据可能需要一个具有大量参数的模型, 而这会导致模型难以训练. 更重要的是, 这样的方法没有考虑到图像数据的高度结构化性质, 其中不同像素的相对位置起着至关重要的作用. 我们之所以关注这一点, 是因为如果我们将图像的像素随机排列, 那么结果就不再像自然图像了. 如果我们为每个像素独立地随机采样像素强度来生成合成图像, 那么生成的合成图像看起来像自然图像 (参见图 6.8) 的可能性基本为零. 局部相关性很重要: 在自然图像中, 与两个相距很远的像素相比, 两个邻近像素具有相似颜色和强度的概率要高得多. 这表明我们可以将图像的先验知识通过强大的归纳偏置编码到神经网络中, 从而得到参数少得多、泛化精度高得多的模型.

10.2 卷积滤波器

引入卷积神经网络的一个动机是, 图像数据是卷积神经网络处理的模态, 由于其高维特性, 标准的全连接架构将需要大量的参数. 要了解这一点, 考虑具有 $10^3\times 10^3$ 像素的彩色图像, 每个像素都有三个值分别对应于红色、绿色和蓝色的强度. 如果网络的第一个隐藏层 (简称第一层) 有 1000 个隐藏单元, 那么我们在第一层就已经有 3×10^9 个权重. 此外, 这样的网络必须通过示例来学习任何不变性和等变性, 这将需要一些庞大的数据集. 通过设计一个包含图像结构归纳偏置的架构, 我们可以大大降低对数据集的要求, 并提升模型关于图像空间对称性的泛化能力.

为了利用图像数据的二维结构来创建归纳偏置, 我们可以使用 4 个相互关联的概念: 层次性、局部性、等变性和不变性. 在检测图像中人脸的任务中, 存在一个自然的层次结构. 因为一个图像可能包含几张人脸, 每张人脸都包含眼睛等元素, 而每只眼睛都有虹膜等结构, 虹膜本身则具有边缘等结构. 在层次结构的最低级别, 神经网络中的节点可以使用图像小区域的局部信息来检测特征 (例如边缘) 的存在, 因此只需要看到图像像素的一少部分即可. 更复杂的结构可以通过组合在前层发现的多个特征来检测. 然而, 一个关键点是, 尽管我们希望在模型中构建层次结构的一般概念, 但我们希望包括在每个级别提取的特征类型在内的层次结构细节, 都是从数据中学习的, 而不是手动编码的. 分层模型可以很自然地与深度学习框架结合, 其已经能够通过一系列 (可能非常多的) 处理“层”从原始数据中提取非常复杂的概念. 整个系统是端到端训练的.

10.2.1 特征检测器

为简单起见, 我们最初将重点聚焦在灰度图像 (即具有单个通道的图像) 上. 考虑神经网络第一层中的单个单元, 该单元仅将图像中的一个小矩形区域或小块内的像素值作为输入, 如图 10.1(a) 所示. 这个小矩形区域或小块称为该单元的感受野 (receptive field), 它体现了局部性的概念. 我们希望与此单元相关的权重值 (简称权重) 能够

学习检测一些有用的低级特征. 该单元的输出以函数的形式给出. 该函数首先对输入值进行加权线性组合, 然后使用非线性激活函数进行变换:

$$z = \mathrm{ReLU}\left(\mathbf{w}^{\mathsf{T}}\mathbf{x} + w_0\right) \tag{10.1}$$

其中 $\mathbf{x}$ 是感受野的像素值向量, 我们假设使用了 ReLU 激活函数. 由于每个输入像素都有一个权重相关联, 因此权重本身会形成一个小的、称为滤波器 (filter) 的二维网格, 有时也称核 (kernel), 其本身可以像图像一样可视化, 如图 10.1(b) 所示.

定义 10.2 (感受野)

设某隐藏单元只接收图像中一个矩形区域内的像素值作为输入, 则称该矩形区域为该单元的**感受野** (receptive field). 若感受野的尺寸为 $M \times M$, 则它的像素值可排成向量 $\mathbf{x} \in \mathbb{R}^{M^2}$, 该单元的输出由(10.1)给出.

♣

注

(重构新增) 感受野是把“局部性”这一先验写进网络结构的最小装置: 单元只看到图像的一小块, 于是它的参数个数只取决于 M, 与整幅图像的像素数无关. 定义中“矩形区域”是必要的, 因为只有这样像素才能按固定顺序排成向量并与权重一一对应; 若感受野是任意形状的集合, 权重共享与后续的等变性论证都不再成立.M 的取值是一个建模选择:M 太小则单元只能看到极短程的相关性,M 太大则参数量与计算量随 M^2 增长, 且局部性先验被削弱.

定义 10.3 (滤波器与核)

在定义 10.2的记号下, 把与感受野内各像素相关联的权重按空间位置排成 $M \times M$ 的二维网格, 称该网格为**滤波器** (filter), 也称**核** (kernel); 称 w_0 为该单元的**偏置**参数.

♣

注

(重构新增) 滤波器与感受野是一对互补的概念: 感受野描述“单元看图像的哪一块”, 滤波器描述“单元怎样看这一块”. 把权重排成与像素网格同形的矩阵, 使我们可以把滤波器本身当作一幅小图像来可视化,图 10.1(b) 正是这样做的. 这个定义还隐含了一个约定: 滤波器的空间索引与感受野的空间索引使用同一套坐标, 后面的卷积公式(10.2)才能写成平移内积的形式. 注意 w_0 不属于滤波器网格, 它在同一特征图的所有位置上共享, 但不参与空间结构的表达.

假设(10.1)中的 $\mathbf{w}$ 和 w_0 是固定的, 我们想知道对于哪一个输入图像小块 $\mathbf{x}$, 这个隐藏单元将给出最大输出响应. 为了回答这个问题, 我们需要以某种方式约束 $\mathbf{x}$. 假设它的范数 $\|\mathbf{x}\|^2$ 是固定的, 那么对于某个系数 α, 使 $\mathbf{w}^{\mathsf{T}}\mathbf{x}$ 最大化并因此最大化隐藏单元响应的 $\mathbf{x}$ 的解的形式为 $\mathbf{x} = \alpha\mathbf{w}$(见练习 10.1). 也就是说, 当检测到一个在整体缩放下像核图像的图像小块时, 就会产生来自这个隐藏单元的最大输出响应. 注意, 只有当 $\mathbf{w}^{\mathsf{T}}\mathbf{x}$ 超过 $-w_0$ 的阈值时,ReLU 才会生成非零输出. 因此该隐藏单元充当了特征检测器的角色, 它会在找到与其卷积核匹配足够好的图像小块时发出信号.

命题 10.1 (最大响应方向)

设 $\mathbf{w} \in \mathbb{R}^n$ 非零,$c > 0$, 在约束 $\|\mathbf{x}\|^2 = c$ 下考虑函数 $f(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x}$. 则 f 在约束集上的最大值在 $\mathbf{x} = \alpha\mathbf{w}$ 处取得, 其中 $\alpha = \sqrt{c}/\|\mathbf{w}\|$; 若 α 可取任意实数, 则所有满足 $\mathbf{x} = \alpha\mathbf{w}(\alpha > 0)$ 的点都是最大值点.

♠

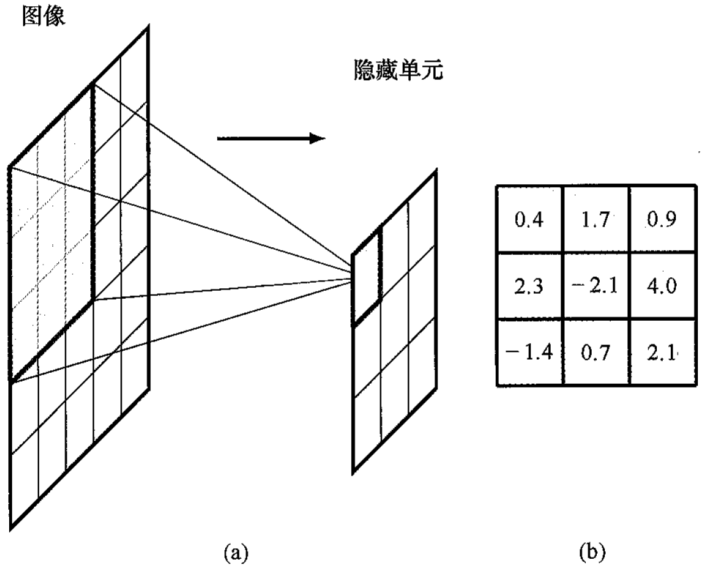


图 10.1: 感受野的图示. 这里显示了网络隐藏层中的一个单元, 该隐藏单元接收来自图像的 3×3 小块像素输入, 此小块中的像素构成了该隐藏单元的感受野.(b) 与该隐藏单元相关的权重值可以可视化为一个小的 3 × 3 矩阵, 称为卷积核. 注意, 还有一个额外的偏置参数, 此处未显示.

证明

由 Cauchy-Schwarz 不等式, 对任意满足 $\|\mathbf{x}\|^2 = c$ 的 $\mathbf{x}$ 有

$$\mathbf{w}^\mathsf{T}\mathbf{x} \leqslant \|\mathbf{w}\|\,\|\mathbf{x}\| = \|\mathbf{w}\|\sqrt{c}.$$

取 $\mathbf{x} = \alpha\mathbf{w}$、$\alpha = \sqrt{c}/\|\mathbf{w}\|$, 则 $\|\mathbf{x}\|^2 = \alpha^2\|\mathbf{w}\|^2 = c$, 且

$$\mathbf{w}^\mathsf{T}\mathbf{x} = \alpha\|\mathbf{w}\|^2 = \sqrt{c}\,\|\mathbf{w}\|,$$

故上界可取到.Cauchy-Schwarz 不等式的等号成立条件恰为 $\mathbf{x}$ 与 $\mathbf{w}$ 共线且同向, 即 $\mathbf{x} = \alpha\mathbf{w}$,$\alpha > 0$. 当 α 可在实数中任取时,$\alpha > 0$ 的共线点都取到同一个最大值 $\sqrt{c}\|\mathbf{w}\|$. 证毕.

□

注

(重构新增) 这个命题说明了“隐藏单元是特征检测器”的精确含义: 在输入能量固定的条件下, 单元响应最大的输入小块恰是滤波器本身的一个正倍数. 条件 $\|\mathbf{x}\|^2 = c$ 不能去掉: 若 $\mathbf{x}$ 可任意放大,$\mathbf{w}^\mathsf{T}\mathbf{x}$ 无上界, 最大值不存在, “最匹配的小块”也就无从谈起.$\alpha > 0$ 也不可换成 $\alpha \neq 0$:$\mathbf{x} = -\alpha\mathbf{w}$ 给出的是最小响应, 它对应“与核反相”的小块. 命题只处理预激活 $\mathbf{w}^\mathsf{T}\mathbf{x} + w_0$; 由于 ReLU 单调不减, 最大化预激活与最大化 ReLU 输出等价. 但“何时输出非零”由阈值决定: 仅当 $\mathbf{w}^\mathsf{T}\mathbf{x} > -w_0$ 时才有非零响应, 故偏置 w_0 控制着检测器的触发门限, 而不是匹配的方向.

10.2.2 平移等变性

接下来请注意, 如果人脸图像中的小块表示处在该位置的眼睛, 那么图像不同部分的同一组像素值表示新位置的眼睛. 我们的神经网络需要能够将其在一个位置学到的知识泛化到图像中的所有可能位置, 而不需要在训练集中每个可能位置见过对应特征的示例. 为了实现这一点, 我们可以简单地在图像的多个位置复制相同的隐藏单元权值, 如图 10.2中的一维输入空间所示.

隐藏层的单元形成一个特征图 (feature map), 其中, 所有单元共享相同的权重. 因此, 如果图像的局部小块在连接到该小块的单元中产生特定响应, 则不同位置的同一组像素值将在特征图中的相应变换位置产生相同的响应. 这是等变性 (equivariance) 的一个例子 (参见小节 9.1.4). 可以看到这个网络中的连接是稀疏的 (sparse), 因为大多数连接不存在. 此外, 权值由所有隐藏单元共享, 如连接的颜色所示. 这一变换是卷积 (convolution) 的一个例子.

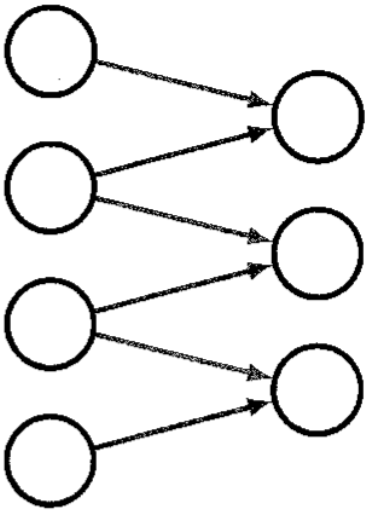


图 10.2: 输入值的一维数组以及宽度为 2 的核卷积示意图. 连接是稀疏的, 由隐藏单元共享, 如红色和蓝色箭头所示, 其中具有相同颜色的连接具有相同的权值. 因此, 该网络有 6 个连接, 但只有两个独立的学习参数.

我们可以将卷积的想法扩展到二维图像 (Dumoulin and Visin, 2016). 对于像素强度为 $I(j,k)$ 的图像 I 和像素值为 $K(l,m)$ 的滤波器 K, 特征图 C 的激活值由下式给出:

$$C(j,k) = \sum_l \sum_m I(j+l,\, k+m)\, K(l,m). \tag{10.2}$$

定义 10.4 (特征图)

设一层中若干隐藏单元以相同方式作用于输入的不同空间位置, 则称这些单元的激活值按空间位置排成的二维数组为该层的**特征图** (feature map). 特征图中所有单元共享同一组权重; 若一层有多个特征图, 则称每个特征图为该层的一个**通道** (channel).

♣

注

(重构新增) 特征图把“位置”与“内容”分开存放: 数组的两个下标是位置, 数组元素是内容. 定义中“以相同方式作用”就是权重共享, 它是等变性 (命题 10.2) 成立的结构性前提; 如果不同位置使用不同权重, 同一小块像素平移到新位置后响应会改变, 等变性立即失效. 把多个特征图称为通道, 是为了与图像的 RGB 通道使用同一套张量语言, 从而让定义 10.8中的四阶滤波器张量有意义. 注意特征图是“对一幅输入图像的一层输出”, 不是网络的参数, 也不参与训练.

定义 10.5 (卷积 (互相关))

设 I 为 $J\times K$ 的图像,K 为 $M\times M$ 的滤波器. 称由(10.2)定义的数组 C 为 I 与 K 的**卷积**, 并记 $C = I * K$. 严格地说,(10.2)实现的是**互相关** (cross-correlation); 机器学习文献惯例上仍称之为卷积.

♣

注

(重构新增) 这个定义有两处需要说明. 第一, 求和范围由“滤波器完全落在图像内”决定, 即 l,m 取使 $1\leqslant j+l\leqslant J$、$1\leqslant k+m\leqslant K$ 的全部整数, 故 C 的尺寸比 I 小 (命题 10.3将精确写出). 第二, 互相关与数学上的卷积相差一次核翻转 (见练习 10.4), 但在卷积网络中滤波器是学习得到的, 翻转与否只是把参数重新编号, 不改变模型族, 因此沿用“卷积”这一名称不会损失任何表达能力. 真正重要的是公式中的两个性质: 对 l,m 求和使输出对输入的局部平移等变, 系数 $K(l,m)$ 与 j,k 无关使权重在所有位置共享.

命题 10.2 (卷积的平移等变性)

设 I 为图像,K 为 $M\times M$ 滤波器,$C=I*K$ 由(10.2)给出,且不施加边界填充 (即只对使滤波器完全落在图像内的位置定义 C). 对整数 δ_j,δ_k,定义平移算子 $(T_{\delta_j,\delta_k}I)(j,k)=I(j-\delta_j,k-\delta_k)$. 则在两边都有定义的位置上成立

$$\big(T_{\delta_j,\delta_k}I\big)*K=T_{\delta_j,\delta_k}\big(I*K\big). \tag{10.3}$$

即卷积与平移可交换: 输入平移多少,特征图就在相同方向上平移多少.

证明

记 $\tilde{I}=T_{\delta_j,\delta_k}I$,即 $\tilde{I}(j,k)=I(j-\delta_j,k-\delta_k)$. 按(10.2),

$$\big(\tilde{I}*K\big)(j,k)=\sum_l\sum_m\tilde{I}(j+l,k+m)K(l,m)=\sum_l\sum_m I(j-\delta_j+l,\,k-\delta_k+m)K(l,m).$$

另一方面,令 $j'=j-\delta_j$、$k'=k-\delta_k$,由同一公式

$$\big(I*K\big)(j',k')=\sum_l\sum_m I(j'+l,\,k'+m)K(l,m),$$

故 $\big(\tilde{I}*K\big)(j,k)=\big(I*K\big)(j-\delta_j,k-\delta_k)=\big(T_{\delta_j,\delta_k}(I*K)\big)(j,k)$. 由于 j,k 任意,(10.3)成立. 证毕.

□

注

(重构新增) 等变 (equivariance) 与不变 (invariance) 是两件不同的事,这个命题给出的是前者: 输出随输入一起平移,而不是保持不变. 条件"不施加边界填充"只是为了回避边界效应: 在补零填充下,靠近边界的位置会看到人为的零,平移后看到的零的个数改变,等式只在离开边界足够远的位置成立. 这正是卷积网络需要定义 10.6的原因之一,也是池化 (定义 10.10) 之后才能得到近似平移不变性的原因. 命题对任意滤波器成立,与 K 的取值无关,因此等变性是架构的性质而非学习的结果; 网络中真正从数据中学到的是"检测什么特征",而不是"是否等变".

通过将这种卷积结构与标准的全连接网络作比较,我们看到了前者的 3 个优点: 首先,连接是稀疏的,即使对于较大的图像,权重也要少得多; 其次,权值是共享的,这大大减少了独立参数的数量,从而减小了学习这些参数所需的训练集大小; 最后,相同的网络可以应用于不同大小的图像,而无需重新训练. 我们稍后将重新讨论最后一点,但目前,你只需要注意,改变输入图像的大小只会更改特征图的大小,而不会改变模型中的权重数量或独立可学习参数的数量 (参见小节 10.4.3). 关于卷积网络的最后一个观察是,通过利用图形处理单元 (Graphics Processing Unit, GPU) 高计算吞吐量的大规模并行特性,卷积在实现中计算效率非常高.

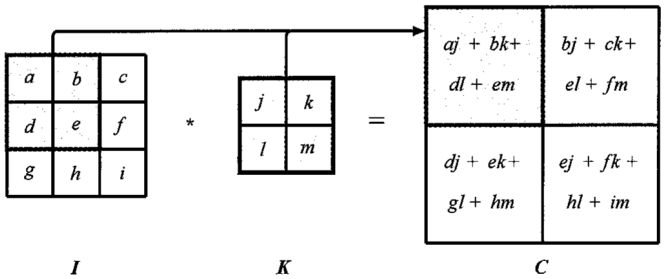


图 10.3: 在 3×3 的图像上使用 2×2 滤波器卷积的示例,以及得到的 2×2 特征图.

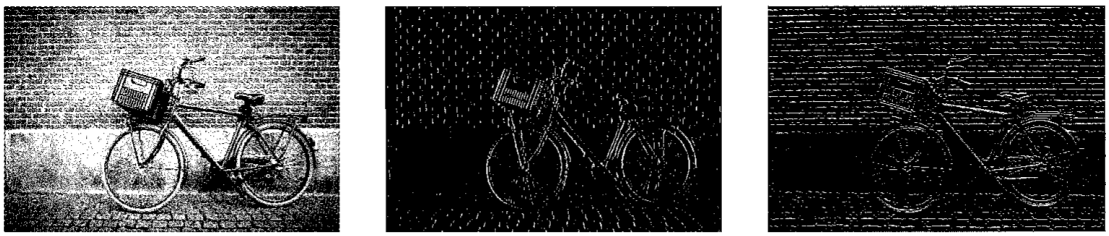
作为卷积应用的一个例子,考虑使用固定的、手工制作的卷积滤波器检测图像边缘的问题. 直观上,可以认为垂直边缘是我们在图像上水平移动时,因像素之间的强度发生显著的局部变化而产生的. 我们可以通过使用以下形式的 3×3 滤波器对图像进行卷积来测量这一点:

$$K=\begin{bmatrix}-1&0&1\\-1&0&1\\-1&0&1\end{bmatrix}. \tag{10.4}$$

类似地, 我们也可以使用此滤波器的转置来对图像进行卷积, 从而检测图像的水平边缘:

$$K = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}. \tag{10.5}$$

图 10.4显示了将这两个卷积滤波器应用于示例图像的结果. 注意在图 10.4(b) 中, 如果竖直边缘对应于像素强度的增加, 则特征图上的相应点为正 (用浅色表示); 而如果竖直边缘对应于像素强度的降低, 则特征图上的相应点为负 (用深色表示).图 10.4(c) 所示水平边缘的结果具有类似的性质.



(a) (b) (c)

图 10.4: 使用卷积滤波器检测图像边缘的示意图:(a) 原始图像;(b) 使用滤波器(10.4)卷积检测竖直边缘的结果;(c) 使用滤波器(10.5)卷积检测水平边缘的结果.

10.2.3 填充

从图 10.3中可以看出, 卷积图比原始图像小. 如果原始图像的维度为 $J \times K$, 并且我们用维度为 $M \times M$ 的核对其进行卷积 (滤波器通常是正方形的), 则生成的特征图的维度为 $(J - M + 1) \times (K - M + 1)$. 在某些情况下, 我们希望特征图具有与原始图像相同的尺寸. 这可以通过在原始图像外部填充 (padding) 额外像素来实现, 如图 10.5所示. 如果我们用 P 个像素填充原始图像, 则输出的特征图的维度为 $(J + 2P - M + 1) \times (K + 2P - M + 1)$. 如果没有对原始图像进行填充, 即 $P = 0$, 则称为有效 (valid) 卷积. 当选择的 P 值使得输出数组的大小与输入数组的大小相同时, 对应于 $P = (M - 1)/2$, 则称为等大 (same) 卷积, 因为原始图像和特征图具有相同的维度 (见练习 10.6). 在计算机视觉中, 滤波器的 M 通常为奇数, 因此填充可以在原始图像的所有边上对称进行, 并且滤波器具有一个与位置关联的明确定义的中心像素. 最后, 我们需要为填充像素选择一个合适的强度值. 典型的选择是将填充值设置为零 (在先从每个图像中减去平均值之后), 这样零就是像素强度的平均值. 填充也可以应用于特征图, 以便由后续卷积层处理.

0	0	0	0	0	0
0	X_{11}	X_{12}	X_{13}	X_{14}	0
0	X_{21}	X_{22}	X_{23}	X_{24}	0
0	X_{31}	X_{32}	X_{33}	X_{34}	0
0	X_{41}	X_{42}	X_{43}	X_{44}	0
0	0	0	0	0	0

图 10.5: 4×4 图像的填充示意图, 该图像已填充了额外像素以创建 6×6 图像.

定义 10.6 (填充、有效卷积与等大卷积)

在图像 (或特征图) 的每条边外侧各补上 P 个取固定值的像素, 称为**填充** (padding),P 称为填充宽度. 若 $P=0$, 称相应的卷积为**有效** (valid) 卷积; 若 $P=(M-1)/2$(M 为奇数), 称相应的卷积为**等大** (same) 卷积. 填充值通常取 0, 且取零之前先从每幅图像中减去像素强度的均值.

注

(重构新增) 填充的唯一目的是控制输出尺寸, 但它同时改变了边界处看到的内容: 位于边界附近的单元在原图上只有半个感受野, 补零后它“看到”的是人为的零, 这与图像内部单元看到的统计量并不一致. 于是“先减去均值再补零”成为一个有实质内容的约定: 若像素均值不为零而直接补零, 边界单元会持续收到一个偏置信号, 等变性(10.3)在边界附近被破坏得更严重.$P=(M-1)/2$ 只对奇数 M 给出整数, 这也是计算机视觉中滤波器尺寸常取奇数的原因之一; 若 M 为偶数, 对称填充无法使输出与输入同尺寸. 注意“等大卷积”不改变参数量, 只是让空间分辨率得以在深层网络中保持.

命题 10.3 (填充卷积的输出尺寸)

设输入为 $J\times K$, 滤波器为 $M\times M$, 填充宽度为 P, 步幅为 1. 则输出特征图的尺寸为

$$(J+2P-M+1)\times(K+2P-M+1). \tag{10.6}$$

特别地, 当 M 为奇数且 $P=(M-1)/2$ 时输出尺寸等于输入尺寸 $J\times K$.

证明

填充后数组的边长为 $J+2P$ 与 $K+2P$. 滤波器在水平方向可放置的位置数为 $(J+2P)-M+1$: 第一个位置使滤波器的第 1 列对齐填充数组的第 1 列, 最后一个位置使滤波器的第 M 列对齐填充数组的第 $J+2P$ 列, 中间每个整数偏移给出一个位置, 故共有 $J+2P-M+1$ 个. 竖直方向同理为 $K+2P-M+1$. 每个位置对应输出特征图的一个元素, 故(10.6)成立. 取 $P=(M-1)/2$ 得 $J+2P-M+1=J+(M-1)-M+1=J$, 竖直方向同理. 证毕.

□

注

(重构新增) 这个公式是后面所有尺寸计算的基石, 使用时要注意三点. 第一, 它要求 $J+2P\geqslant M$, 否则滤波器放不进填充后的数组, 输出尺寸为零或负, 卷积无定义. 第二, 公式对特征图同样适用, 只需把 J,K 换成上一层的输出尺寸, 这正是逐层追踪网络形状的依据. 第三, 等大填充保持的是空间尺寸而不是信息量: 边界处的输出仍由补出来的零参与计算, 因此“输出与输入同尺寸”不等于“边界处理无损失”.

10.2.4 跨步卷积

在典型的图像处理应用中, 图像可以有非常多的像素, 并且由于卷积核通常相对较小, 因此 $M\ll J,K$, 特征图的大小将与原始图像相似 (如果进行等大填充, 则大小相同). 有时, 我们希望使用比原始图像小得多的特征图, 以便在设计卷积网络时更灵活. 实现此目的的一种方法是使用跨步卷积 (strided convolution), 即不再将滤波器一次移动一个像素, 而是以较大的步长 S 移动滤波器. 我们称 S 为步幅 (stride). 如果在水平和竖直方向上使用相同的步幅, 那么特征图中的元素数量为 (见练习 10.7):

$$\left\lfloor\frac{J+2P-M}{S}+1\right\rfloor\times\left\lfloor\frac{K+2P-M}{S}+1\right\rfloor, \tag{10.7}$$

其中 $\lfloor x\rfloor$ 表示将 x 向下取整, 得到小于或等于 x 的最大整数. 对于大图像和小尺寸滤波器, 特征图将是原始图像大小的 $1/S$.

定义 10.7 (跨步卷积与步幅)

在卷积中, 若滤波器每次沿水平或竖直方向移动 S 个像素而不是 1 个像素, 则称该运算为**跨步卷积** (strided convolution), 称 S 为**步幅** (stride). 若两个方向使用相同的步幅, 则称步幅为 S.

注

(重构新增) 步幅提供了一种与池化不同的下采样手段: 它不引入任何额外的池化参数, 而是在卷积本身中跳着取样. 定义要求 S 为正整数, 因为滤波器位置必须落在像素网格上;$S=1$ 退化为普通卷积. 步幅带来的代价是信息丢失的不可逆性: 被跳过的位置完全不参与输出, 这一点与池化相同, 但跨步卷积同时仍在做特征变换, 因此它在降分辨率的同时保留了“学到的”表示, 而池化只做固定的聚合. 步幅与填充是两套独立机制, 前者控制采样密度, 后者控制边界.

命题 10.4 (跨步卷积的输出尺寸)

设输入为 $J\times K$, 滤波器为 $M\times M$, 填充宽度为 P, 步幅为 S, 且 $J+2P\geqslant M$、$K+2P\geqslant M$. 则输出特征图的尺寸由(10.7)给出.

♠

证明

考察水平方向. 填充后数组长度为 $J+2P$. 滤波器的左端位置依次为 $0,S,2S,\cdots$, 第 t 个位置 ($t=0,1,2,\cdots$) 要求滤波器完全落在数组内, 即 $tS+M\leqslant J+2P$, 等价于 $t\leqslant (J+2P-M)/S$. 满足该不等式的非负整数 t 的个数为 $\lfloor (J+2P-M)/S\rfloor+1$, 这正是输出特征图的列数. 竖直方向同理给出 $\lfloor (K+2P-M)/S\rfloor+1$ 行. 两方向独立, 故总元素个数为二者之积. 当 $J,K\gg M$ 且 P 相对 J,K 可忽略时, 该数约为 J/S 与 K/S, 即特征图边长约为原图的 $1/S$. 证毕.

□

注

(重构新增) 向下取整是公式的要害: 它说明当 $(J+2P-M)$ 不能被 S 整除时, 最后一段像素被丢弃, 输出尺寸不是简单的除法. 若把 S 增大而不同时调整 P, 可能出现某个位置被完全跳过、或边界像素从未被任何滤波器位置覆盖的情形, 因此实际实现中常配合 P 使 $J+2P-M$ 尽量被 S 整除. 条件 $J+2P\geqslant M$ 不可省, 否则连第一个位置都放不下. 公式对多层网络逐层使用即可追踪形状, 这一点在命题 10.7中还会用到.

10.2.5 多维卷积

到目前为止, 我们已经考虑了单个灰度图像上的卷积. 彩色图像有三个通道, 对应于红色、绿色和蓝色. 通过扩展滤波器的维度, 我们可以轻松地扩展卷积以支持多个通道. 具有 $J\times K$ 个像素和 C 个通道的图像可以用维度为 $J\times K\times C$ 的张量来描述 (参见节 6.3). 我们可以引入一个用维度为 $M\times M\times C$ 的张量来描述的滤波器, 该张量对于 C 个通道中的每一个通道, 都有一个独立的 $M\times M$ 滤波器. 假设对原始图像没有进行填充且步幅为 1, 则同样可以得到大小为 $(J-M+1)\times(K-M+1)$ 的特征图, 如图 10.6所示.

定义 10.8 (多通道卷积与通道数)

设输入张量为 $I\in\mathbb{R}^{J\times K\times C_{\text{in}}}$, 滤波器张量为 $W\in\mathbb{R}^{M\times M\times C_{\text{in}}\times C_{\text{out}}}$. 对 $c=1,\cdots,C_{\text{out}}$, 定义第 c 个输出通道为

$$O_c(j,k)=\sum_{c'=1}^{C_{\text{in}}}\sum_{l}\sum_{m} I_{c'}(j+l,\,k+m)\,W(l,m,c',c)+b_c, \tag{10.8}$$

其中 b_c 为第 c 个输出通道的偏置,$I_{c'}$ 表示输入的第 c' 个通道. 称 C_{in} 为输入通道数,C_{out} 为输出通道数, 称 $O_1,\cdots,O_{C_{\text{out}}}$ 为输出的各个通道.

♣

注

(重构新增) 这个定义把“在每个输入通道上各有一个 $M\times M$ 滤波器”与“把各通道的结果相加”两件事合成一个张量运算: 滤波器张量的最后一维 C_{out} 取不同值时, 取到的是完全独立的一组 C_{in} 个二维滤波器. 求和指标 c' 遍历全部输入通道而不是某一个, 因此单个输出通道看到的是所有输入通道的联合信息, 这正是彩色图像第一层能检测“颜色-方向”组合特征的原因. 若去掉对 c' 的求和、只保留单通道, 就退化为一维卷积在通道上的堆叠, 各通道之间不再交互, 表达能力显著下降.

下面让我们对卷积进一步进行重要扩展. 目前, 我们已经创建了单一的特征图, 并且特征图中的所有点共享同

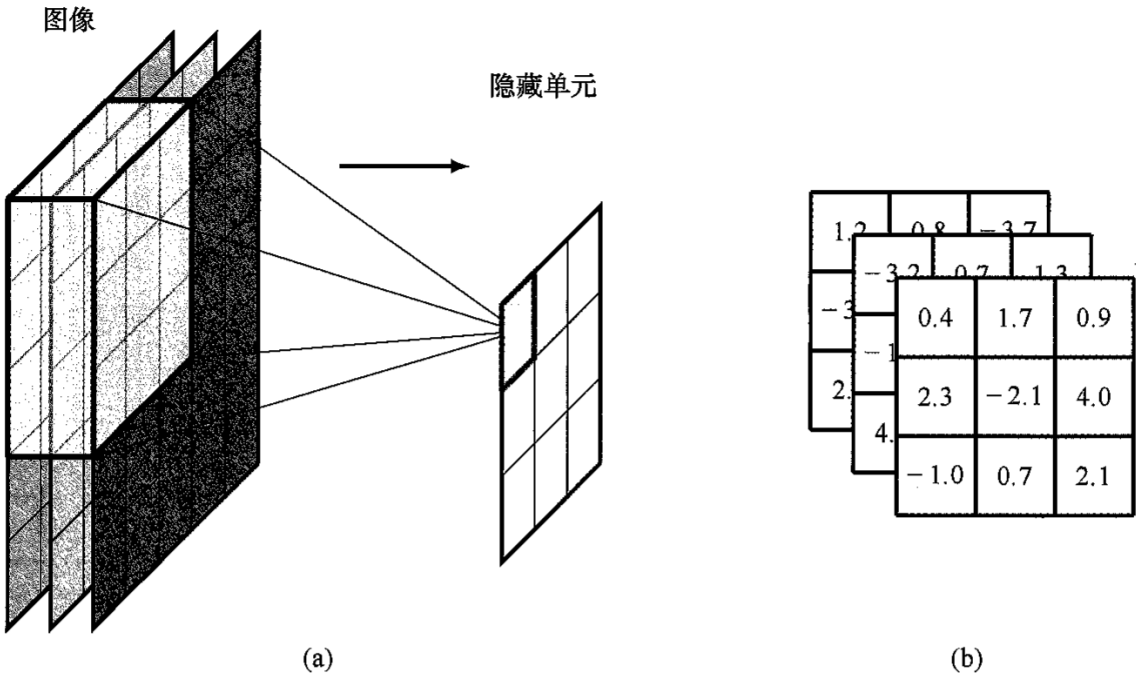


图 10.6: (a) 从 R、G 和 B 通道接收输入的多维滤波器示意图;(b) 这里的核有 27 个权重 (加上 1 个未显示的偏置参数), 它可以可视化为一个 $3 \times 3 \times 3$ 大小的张量.

一组可学习参数. 对于维度为 $M \times M \times C$ 的滤波器, 无论图像大小如何, 都将有 M^2C 个权重参数. 此外, 还有一个与该单元相关的偏置参数. 这种滤波器类似于全连接网络中的单个隐藏节点, 它只能学习检测一种特征, 因此作用非常有限. 为了构建更灵活的模型, 我们需要使用多个这样的滤波器, 其中每个滤波器都有互不相关的参数集, 从而产生独立的特征图, 如图 10.7所示. 我们再次将这些独立的特征图称为通道 (channel). 滤波器张量现在的维度为 $M \times M \times C \times C_{\text{out}}$, 其中 C 是输入通道数,C_{out} 是输出通道数. 每个输出通道都有自己的相关偏置参数, 因此参数总数为 $(M^2C + 1)C_{\text{out}}$.

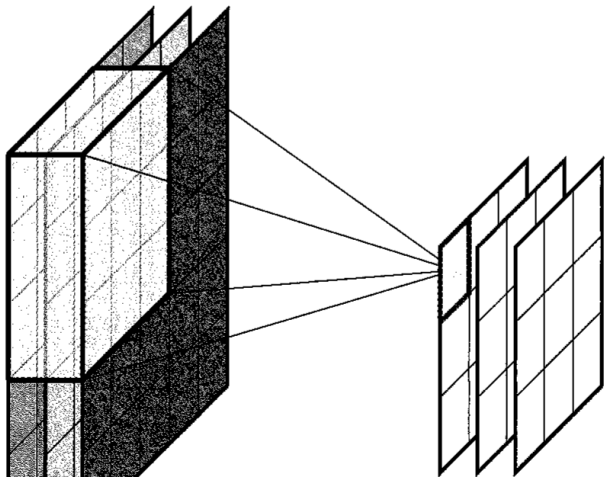


图 10.7: 图 10.6所示的多维卷积滤波层可以扩展为包括多个独立的滤波通道.

命题 10.5 (卷积层参数量)

设一个卷积层的输入有 C_{in} 个通道, 滤波器空间尺寸为 $M \times M$, 输出有 C_{out} 个通道, 每个输出通道带一个偏

置. 则该层的可学习参数总数为

$$\left(M^2C_{\text{in}}+1\right)C_{\text{out}}. \tag{10.9}$$

该数目与输入的空间尺寸 $J\times K$ 无关; 该层的连接数为 $M^2C_{\text{in}}C_{\text{out}}J'K'$, 其中 $J'\times K'$ 为输出特征图的空间尺寸.

♠

证明

由定义 10.8, 第 c 个输出通道用到权重 $W(\cdot,\cdot,\cdot,c)$, 共 $M\cdot M\cdot C_{\text{in}}=M^2C_{\text{in}}$ 个, 再加一个偏置 b_c, 故每个输出通道有 $M^2C_{\text{in}}+1$ 个参数.C_{out} 个输出通道的参数集互不相关, 故总数为 $(M^2C_{\text{in}}+1)C_{\text{out}}$. 该表达式不含 J,K, 故与输入空间尺寸无关. 连接数方面, 输出特征图的每个元素通过 M^2C_{in} 条边连到输入, 输出元素共 $J'K'C_{\text{out}}$ 个, 故连接总数为 $M^2C_{\text{in}}C_{\text{out}}J'K'$. 证毕.

□

注

(重构新增) 这个命题量化了节 10.2开头关于参数量的对比: 第一层 3×3、$C_{\text{in}}=3$、$C_{\text{out}}=64$ 的卷积层只有 $(9\cdot 3+1)\cdot 64=1792$ 个参数, 而同尺寸输入上的全连接层需要 $10^6\cdot 3\cdot 1000$ 量级的权重. 注意“参数少”与“连接多”并不矛盾: 连接数仍与空间尺寸成正比 (乘上了 $J'K'$), 减少的是独立自由度而非计算量. 条件“每个输出通道带一个偏置”是本书采用的约定; 若某些实现共享偏置或使用无偏置卷积, 公式相应变为 $M^2C_{\text{in}}C_{\text{out}}$ 或 $M^2C_{\text{in}}C_{\text{out}}+C_{\text{out}}$, 这不影响数量级结论.

设计卷积网络时的一个有用概念是 1×1 卷积 (Lin, Chen, and Yan, 2013), 它是一个滤波器大小为单个像素的卷积层. 滤波器有 C 个权重 (每个输入通道一个权重), 外加一个偏置.1×1 卷积通过将输出通道数设置为与输入通道数不同, 可以简单地用来改变通道数 (通常用来减少通道数), 而不改变特征图的大小. 因此, 它是对跨步卷积或池化的补充, 因为它减少了通道的数量而不是通道的维度.

定义 10.9 (1×1 卷积)

称空间尺寸为 1×1 的卷积为 1×1 **卷积**, 即对每个空间位置 (j,k) 独立地施加同一个线性映射

$$O_c(j,k)=\sum_{c'=1}^{C_{\text{in}}}W(1,1,c',c)\,I_{c'}(j,k)+b_c,\qquad c=1,\cdots,C_{\text{out}}. \tag{10.10}$$

♣

注

(重构新增) 1×1 卷积是“逐像素的全连接层”: 它在每个空间位置上使用同一组权重, 因而完全不看邻域, 只做通道间的线性混合. 它的参数量为 $(C_{\text{in}}+1)C_{\text{out}}$, 与空间尺寸无关, 因此常被用来在不改变分辨率的前提下压缩或扩展通道数, 例如在 U-Net 的最后一层把通道数降到类别数 (小节 10.5.4). 把它与定义 10.8比较可以看出, 它只是 $M=1$ 的特例, 并不是新的运算; 把它单列出来是因为它在网络设计中承担“通道变换”这一独立职能. 若在 1×1 卷积后再接非线性激活, 它就成为逐像素的两层感知机.

10.2.6 池化

卷积层编码了平移等变性 (equivariance): 如果将一小块像素 (代表隐藏单元的感受野) 移动到图像中的不同位置, 则特征图的关联输出将移动到特征图中的相应位置. 这对于在图像中找到目标的位置等应用很有价值. 对于其他应用, 例如图像分类, 我们希望在输入平移时, 输出保持不变. 然而, 在所有情况下, 我们都希望网络能够学习分层结构, 其中特定级别的复杂特征是由前一级的简单特征构建而成的. 在许多情况下, 这些简单特征之间的空间关系非常重要. 例如, 眼睛、鼻子和嘴巴的相对位置有助于确定面部的存在, 而非仅仅表明这些特征在图像中存在. 但是, 相对位置的微小变化不会影响分类, 我们希望对单个特征的这种小平移保持输出不变. 这可以通过在卷积层输出之后, 应用池化 (pooling) 来实现.

池化与卷积有相似之处, 因为单元阵列排列在网格中, 且其中的每个单元都从前一个特征图的感受野中获取输入. 此外, 池化同样需要选择滤波器大小和步幅. 然而, 池化与卷积的不同之处在于, 池化单元的输出是其输

入的简单、固定的函数, 因此池化中没有可学习参数. 池化的一个常见示例是最大池化 (max-pooling)(Zhou and Chellappa, 1988), 其中每个单元仅输出对于输入值的“取最大值”函数.图 10.8所示的简单示例说明了这一过程. 这里的步幅等于滤波器的宽度, 因此感受野没有重叠.

定义 10.10 (池化与最大池化)

设特征图为 A, 池化窗口为 $M\times M$、步幅为 S. 若池化单元的输出为窗口内全部输入值的最大值, 则称该运算为**最大池化** (max-pooling); 若为窗口内全部输入值的平均值, 则称**平均池化** (average pooling). 池化不含可学习参数, 且通常对每个通道独立施加.

♣

注

(重构新增) 池化把“特征是否存在”与“特征在哪里”分开: 最大池化保留窗口内的最大响应 (存在性与强度), 但丢弃最大值的位置 (位置信息). 这正是它在图像分类中有用、在语义分割中却造成麻烦的原因 (小节 10.5.4). 定义强调“对每个通道独立施加”, 否则不同通道的响应会被混在一起, 通道语义被破坏; 也强调“无可学习参数”, 因此池化不增加模型容量, 只改变表示. 窗口与步幅都等于 M 时窗口互不重叠, 输出尺寸恰为输入的 $1/M$; 若步幅小于窗口, 则窗口重叠, 输出更大但仍小于输入.

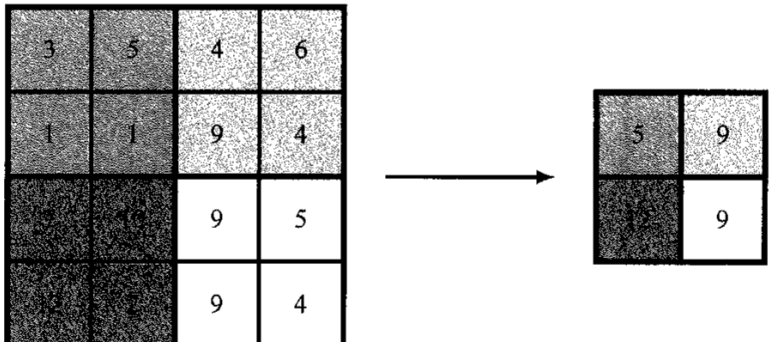


图 10.8: 最大池化示意图, 其中使用“max”运算符将特征图中的 2×2 像素块组合在一起, 从而生成较小维度的新特征图.

除了构建一些局部平移不变性外, 池化通过对特征图下采样, 还可用于降低表示的维度. 注意, 在卷积层中使用大于 1 的步幅也可以实现对特征图下采样的效果.

我们可以将特征图中单元的激活解释为检测相应特征强度的度量, 因此最大池化保留了有关特征是否存在以及特征强度的信息, 但丢弃了一些位置信息. 池化还有许多其他选择, 例如平均池化 (average pooling), 其中池化函数计算特征图中相应感受野值的平均值. 这些都引入了一定程度的局部平移不变性.

命题 10.6 (池化的输出尺寸与局部不变性)

设特征图的空间尺寸为 $J\times K$, 池化窗口为 $M\times M$、步幅为 S. 则池化输出的空间尺寸为

$$\left\lfloor \frac{J-M}{S} \right\rfloor + 1 \times \left\lfloor \frac{K-M}{S} \right\rfloor + 1. \tag{10.11}$$

当 $S=M$ 时输出尺寸为 $\lfloor J/M\rfloor \times \lfloor K/M\rfloor$, 即窗口互不重叠. 此外, 若输入在窗口内发生不超过窗口范围的平移, 且窗口内最大值唯一, 则最大池化的输出在相应窗口内保持不变.

♠

证明

尺寸公式与命题 10.4在 $P=0$ 时的情形完全相同: 窗口左端可放置的位置为 $0, S, 2S, \cdots$, 第 t 个位置要求 $tS+M \leqslant J$, 故 $t \leqslant (J-M)/S$, 位置个数为 $\lfloor (J-M)/S\rfloor + 1$. 取 $S=M$ 得 $\lfloor (J-M)/M\rfloor + 1 = \lfloor J/M\rfloor$(当 J 为 M 的整数倍时等号成立, 一般情形由取整给出). 关于不变性: 设某窗口内的输入为 $a_1, \cdots, a_{M^2}$, 最大值唯一且等于 a^*. 若输入发生平移, 只要平移后 a^* 仍落在同一窗口内, 该窗口的最大值仍是 a^*, 故对应池化单元的输出不变. 证毕.

□

注

(重构新增) 池化只带来“局部”平移不变性, 这一点必须与命题 10.2的全局等变性区分开: 卷积使输出随输入平移, 池化则把一个小范围内的平移“吸收”掉, 但平移超过一个窗口后输出仍会改变. 命题中“最大值唯一”的条件不能省: 若窗口内有两个相等的最大值, 平移可能使池化输出相同, 也可能因取到不同位置的最大值而在后续层产生不同结果.$S < M$ 时窗口重叠, 同一输入会参与多个输出, 不变性变弱但信息损失减少, 这是设计时的取舍. 最后, 池化不改变通道数, 它降低的是空间分辨率.

例题 10.1 8 通道特征图的最大池化 设特征图有 8 个通道, 每个通道的空间尺寸为 64×64, 施加窗口 2×2、步幅 2 的最大池化. 由(10.11), 每个通道输出 32×32, 故池化结果为 $32 \times 32 \times 8$ 的张量, 元素个数降为原来的四分之一, 而通道数不变.

我们还可以在特征图的多个通道中应用池化, 这将使网络有潜力学习简单平移不变性之外的其他不变性. 例如, 如果卷积层中的多个通道学习检测不同方向上的相同特征, 则这些特征图的最大池化将对于旋转变换 (几乎) 保持不变. 池化还允许卷积网络处理不同大小的图像. 归根结底, 卷积网络的输出, 以及通常的一些中间层, 必须具有固定尺寸. 根据图像大小改变池化的步幅大小, 可以使卷积网络适应可变大小的输入, 以确保池化输出的数量保持不变.

10.2.7 多层卷积

到目前为止我们描述的卷积网络结构类似于标准全连接神经网络中的单层网络结构. 为了允许网络发现和表示数据中的分层结构, 现在我们通过使用多个上述类型的卷积层来扩展架构. 每个卷积层由维度为 $M \times M \times C_{\text{in}} \times C_{\text{out}}$ 的滤波器张量表示, 其中独立权重和偏置参数的数量为 $(M^2 C_{\text{in}} + 1)C_{\text{out}}$. 每个这样的卷积层都允许我们选择性地在通道数量上进行扩展, 后续加上一个池化层. 现在, 我们可以连续应用多个这样的卷积层和池化层, 其中某个层的输出通道 C_{out}(类似于输入图像的 RGB 通道) 形成下一层的输入通道. 注意, 特征图中的通道数有时称为特征图的“深度”, 但我们更愿意用“深度”来表示网络中的层数.

卷积框架的一个关键性质是局部性, 特征图中的给定单元仅从前一层的一小块 (即感受野) 中获取信息. 当我们构建一个每一层都是卷积层的深度神经网络时, 后面层中单元的有效感受野会比前面层中单元的有效感受野大得多, 如图 10.9所示.

命题 10.7 (有效感受野的递推)

考虑 L 层卷积堆叠, 第 l 层的滤波器空间尺寸为 $M_l \times M_l$、步幅为 S_l. 记第 l 层单元在输入图像上的有效感受野边长为 R_l, 并约定 $R_0 = 1$. 则

$$R_l = R_{l-1} + (M_l - 1)\prod_{i=1}^{l-1} S_i, \qquad l = 1, \cdots, L, \tag{10.12}$$

其中空乘积约定为 1. 特别地, 当所有 $S_l = 1$、$M_l = M$ 时 $R_L = 1 + L(M-1)$, 随深度线性增长. ♠

证明

对 l 归纳.$l = 1$ 时, 第 1 层单元直接连接输入的 $M_1 \times M_1$ 小块, 而 $R_0 + (M_1 - 1) \cdot 1 = 1 + M_1 - 1 = M_1$, 成立. 设(10.12)对 $l-1$ 成立. 第 l 层的一个单元连接第 $l-1$ 层中 $M_l \times M_l$ 个相邻单元, 这些单元在第 $l-1$ 层坐标系中覆盖的长度为 $(M_l - 1)S_{l-1} + 1$(步幅为 S_{l-1} 时,M_l 个采样点跨过的距离为 $(M_l - 1)S_{l-1}$, 再加一个单元自身的宽度). 把这段长度换算到输入图像坐标, 需要乘以第 $l-1$ 层之前的累计步幅 $\prod_{i=1}^{l-1} S_i$ 中的相应部分. 更直接地, 记 $P_{l-1} = \prod_{i=1}^{l-1} S_i$ 为输入到第 $l-1$ 层的累计步幅, 则第 $l-1$ 层中相邻单元在输入上相距 P_{l-1}, 故第 l 层单元覆盖的输入范围为 $R_{l-1} + (M_l - 1)P_{l-1}$, 即(10.12). 所有 $S_l = 1$ 时 $P_{l-1} = 1$, 逐层相加得 $R_L = 1 + \sum_{l=1}^{L}(M_l - 1) = 1 + L(M-1)$. 证毕.

□

注

(重构新增) 这个递推解释了图 10.9的现象, 也解释了为什么“小滤波器堆叠”可以替代“大滤波器”: 三层 3×3 卷积 ($S=1$) 的感受野为 $1+3\cdot2=7$, 与一层 7×7 卷积相同, 但参数为 $3\cdot9C^2$ 而不是 $49C^2$, 且中间多出两次非线性. 条件“每层步幅为 S_l”必须逐层记录, 因为一旦某层 $S_l>1$, 后续每增加一层感受野的增量就被放大 S_l 倍, 这就是跨步卷积能快速扩大感受野的原因. 公式给出的是理论上界: 由于边界补零与滤波器权重可能为零, 实际对输出有影响的输入区域 (有效感受野) 通常小于 R_L.R_L 也不等于“一个单元能看见多少像素”的精确描述, 它给出的是边长的紧上界.

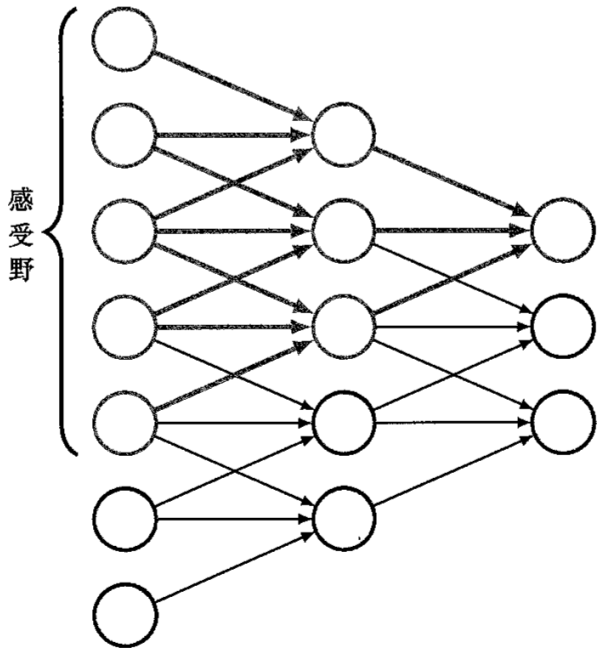


图 10.9: 多层卷积网络中有效感受野如何随深度的增加而增大的示意图. 在这里, 我们看到输出层顶部的红色单元从中间层单元的感受野中获取输入, 每个中间层单元在第一层单元中都有一个感受野. 因此, 输出层中红色单元的激活取决于中间层中 3 个单元的输出以及输入层中的 5 个单元的输出.

早期的 CNN 通常具有较少的卷积层, 因为它们具有较大的感受野. 例如,AlexNet(Krizhevsky, Sutskever, and Hinton, 2012) 具有步幅为 4、尺寸为 11×11 的感受野. 从图 10.9中可以看出, 通过使用每层只有较小感受野的多层结构, 也可以隐式地实现较大的感受野. 后者的优点是需要的参数要少得多, 并且有效地对较大的滤波器施加了归纳偏置, 因为它们必定由卷积子滤波器组成. 尽管这是一个高度复杂的架构, 但只需要对网络函数本身进行显式编码即可, 因为我们可以使用自动微分 (参见节 8.2) 来计算损失函数的导数, 并使用随机梯度下降来优化损失函数.

在许多应用中, 网络的输出单元需要预测图像整体的信息. 例如在图像分类任务中, 网络需要组合来自整个输入图像的信息. 这通常是通过在网络的最后阶段引入一个或两个标准的全连接层来实现的, 其中每个单元都连接到前一层中的每个单元. 这种架构中的参数数量是可控的, 因为中间池化层的存在, 最终卷积层的维度通常比输入层的维度少得多. 然而, 即使网络中 (共享的) 连接数量比卷积层中的多, 最终全连接层也可能包含网络中大多数独立自由度. 因此, 一个完整的 CNN 包括多层卷积, 这些卷积穿插着池化操作, 并且通常在网络的最后阶段有传统的全连接层. 在设计这样的架构时, 需要做出许多选择, 包括层数、每层中的通道数、滤波器尺寸、步幅大小以及多个其他此类超参数. 在实践中, 训练每个候选配置的计算成本很高, 很难使用留出法对超参数的值进行系统的比较, 研究人员为此探索了各种不同的架构.

10.2.8 网络架构示例

卷积网络是第一个成功部署在应用中的深度神经网络 (即具有两个以上可学习参数层的神经网络). 一个早期的例子是 LeNet, 它用于对低分辨率的单色手写数字图像进行分类 (LeCun et al., 1989; LeCun et al., 1998). 通过引入名为 ImageNet(Deng et al., 2009) 的大规模基准数据集, 开发更强大的卷积网络得到了加速. 该数据集包含约 1 400 万幅自然图像, 其中每幅图像都被手工归为近 22 000 个类别之一. 这是一个比我们以前使用的所有数据集都大得多的数据集,ImageNet 推动的领域进步强调大规模数据以及设计具有适当归纳偏置的良好模型在构建成功深度学习方案时的重要性.

ImageNet 中包含 1 000 个非重叠类别的图像子集构成了年度 ImageNet 大规模视觉识别挑战赛 (ImageNet Large Scale Visual Recognition Challenge) 的基础. 同样, 这比之前通常考虑几十个类别要多得多. 拥有如此多的类别使问题变得更加棘手, 因为如果类别分布均匀, 随机猜测的错误率将达到 99.9%. 该数据集包含超过 128 万幅训练图像、5 万幅验证图像和 10 万幅测试图像. 分类器旨在为测试图像生成预测输出类别的排序列表, 并根据 Top 1 和 Top 5 的错误率报告结果. 这意味着如果真实类别出现在列表顶部或位于 5 个排名最靠前的类别预测之中, 则视为图像分类正确. 该数据集的早期结果实现了 Top 5 的错误率约为 25.5%.AlexNet 卷积网络架构 (Krizhevsky, Sutskever, and Hinton, 2012) 取得了重要进展, 赢得了 2012 年的比赛, 并将 Top 5 的错误率降低到 15.3% 的新纪录.AlexNet 模型的关键之处在于使用 ReLU 函数作为激活函数、应用 GPU 训练网络以及使用 dropout 正则化 (参见小节 9.6.1). 该模型随后几年取得了进一步的成功, 错误率降低至约 3%, 在某些数据上, 比人类水平 (约 5%) 表现得要好一些 (Dodge and Kara, 2017). 这可以归因于人类难以区分只有微妙差异的不同类别 (例如不同的蘑菇品种).

作为典型卷积网络架构的一个示例, 我们来详细了解一下 VGG-16 模型 (Simonyan and Zisserman, 2014), 其中 VGG 代表开发该模型的 Visual Geometry Group,16 表示模型中可学习层的数量.VGG-16 模型有一些简单的设计原则, 旨在得到相对一致的架构, 如图 10.10所示, 即最大限度地减少需要选择的超参数的数量.

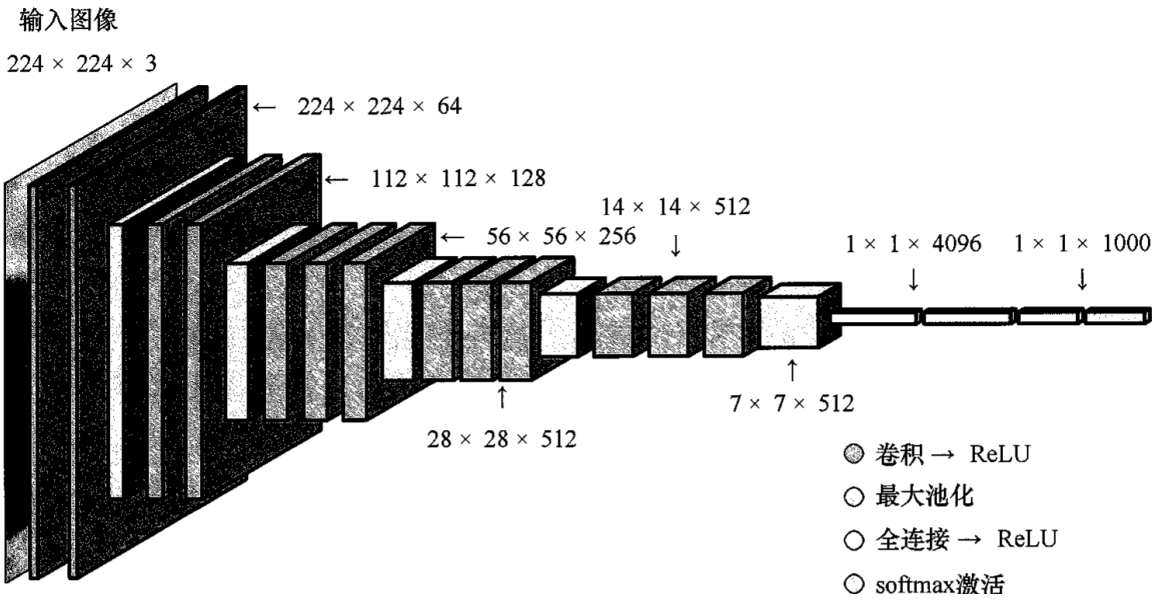


图 10.10: 一个典型卷积网络 (在本例中是 VGG-16 模型) 的架构.

该模型以 224×224 像素和三颜色通道的图像作为输入, 然后接入穿插有下采样的卷积层集合. 所有卷积层都具有尺寸为 3×3、步幅为 1 的滤波器, 执行等大填充并使用 ReLU 激活函数; 而最大池化操作都使用步幅为 2、尺寸为 2×2 的滤波器, 从而将单元数下采样为原先的四分之一. 第 1 个可学习层 (简称第 1 层) 是一个卷积层, 其中每个单元从输入通道堆叠得到的 3×3×3 的“立方体”获取输入, 因此包括偏置在内共有 28 个参数, 这些参数在该通道的特征图中的所有单元之间共享. 第 1 层有 64 个这样的特征通道, 从而使输出张量的大小为 224×224×64. 第 2 层也是卷积层, 同样有 64 个通道. 紧接着是最大池化层, 给出的特征图大小为 112 × 112. 第 3 层和第 4 层又

是卷积层, 维度为 112×112, 每层都有 128 个通道. 通道数量的增加在一定程度上抵消了最大池化层中的下采样, 以确保每层表示中的变量数量不会随着所通过网络层数的增加而过快地减少. 同样, 接下来是最大池化操作, 以产生大小为 56×56 的特征图. 接下来是另外三个卷积层, 每个卷积层有 256 个通道, 从而再次将与下采样相关的通道数量增加一倍. 接下来是另一个最大池化操作, 得到大小为 28×28 的特征图. 紧接着又是三个卷积层, 每个卷积层有 512 个通道, 后面跟着另一个最大池化操作, 下采样得到大小为 14×14 的特征图. 接下来是另外三个卷积层, 尽管这三个卷积层中的特征图数量保持在 512, 但接下来再次进行最大池化操作, 从而将特征图的大小减小到 7×7. 最后, 还有三层是全连接层, 这意味着它们是具有完全连接且不共享参数的标准神经网络层. 最终的最大池化层有 512 个通道, 每个通道的大小为 7×7, 总共 25 088 个单元. 第一个全连接层有 4 096 个单元, 每个单元都连接到每个最大池化单元. 接下来是第二个全连接层, 它同样有 4 096 个单元. 最后是第三个全连接层, 它有 1 000 个单元, 以便将网络应用于包含 1 000 个类别的分类问题. 网络中除输出层外的所有可学习层都具有非线性 ReLU 激活函数, 输出层具有 softmax 激活函数.

总体而言,VGG-16 模型中总共有大约 1.38 亿个可独立学习的参数, 其中大部分 (近 1.03 亿个) 位于第一个全连接层中, 而大多数连接位于第一个卷积层中 (参见练习 10.8).

例题 10.2 VGG-16 的逐层参数量 按命题 10.5逐个计算 VGG-16 的参数.13 个卷积层的参数依次为

$$(3 \cdot 3 \cdot 3 + 1) \cdot 64 = 1\,792, \qquad (3 \cdot 3 \cdot 64 + 1) \cdot 64 = 36\,928,$$
$$(3 \cdot 3 \cdot 64 + 1) \cdot 128 = 73\,856, \qquad (3 \cdot 3 \cdot 128 + 1) \cdot 128 = 147\,584,$$
$$(3 \cdot 3 \cdot 128 + 1) \cdot 256 = 295\,168, \qquad 2 \times \left[(3 \cdot 3 \cdot 256 + 1) \cdot 256\right] = 1\,180\,160,$$
$$(3 \cdot 3 \cdot 256 + 1) \cdot 512 = 1\,180\,160, \qquad 5 \times \left[(3 \cdot 3 \cdot 512 + 1) \cdot 512\right] = 11\,799\,040,$$

三个全连接层分别为 $7 \cdot 7 \cdot 512 \cdot 4096 + 4096 = 102\,764\,544$、$4096 \cdot 4096 + 4096 = 16\,781\,312$ 与 $4096 \cdot 1000 + 1000 = 4\,097\,000$.

命题 10.8 (VGG-16 的总参数量)

按图 10.10的结构 (13 个卷积层、3 个全连接层、池化层无可学习参数),VGG-16 的可学习参数总数为

$$N_{\text{VGG16}} = 138\,357\,544 \approx 1.38 \times 10^8, \tag{10.13}$$

其中第一个全连接层贡献 102 764 544 个参数, 约占总数的 74%.

♠

证明

卷积层部分由例 10.2逐层相加:

$$1\,792 + 36\,928 + 73\,856 + 147\,584 + 295\,168 + 1\,180\,160 + 1\,180\,160 + 11\,799\,040$$
$$= 14\,714\,688.$$

全连接层部分为 $102\,764\,544 + 16\,781\,312 + 4\,097\,000 = 123\,642\,856$. 故总数为 $14\,714\,688 + 123\,642\,856 = 138\,357\,544$. 第一个全连接层所占比例为 $102\,764\,544/138\,357\,544 \approx 0.743$, 即约 74%. 证毕.

□

注

(重构新增) 这个计算说明了一个反直觉的事实:VGG-16 的绝大多数参数并不在卷积层, 而在最后三个全连接层, 因为 $7\times7\times512$ 与 4096 之间的全连接矩阵没有任何权重共享. 由此可以理解后续架构 (如 ResNet) 为何用全局平均池化取代大全连接层: 不是为了减少连接, 而是为了减少独立自由度. 计算时逐层核对三件事: 输入通道数是否等于上一层的输出通道数、3×3 卷积是否保持空间尺寸 (等大填充,$P=1$)、池化是否把边长减半. 若把某个 3×3 卷积误当作 5×5, 或漏掉偏置, 总数就会偏离 1.38 亿; 因此这个数字可以作为架构描述的校验和. 注意题目所说的“权重数量 (连接数)”与“独立可学习参数”是两个不同的量, 前者还要乘以输出特征图的空间尺寸, 后者与 J,K 无关.

10.3 可视化训练好的 CNN

本节介绍探索现代深度 CNN 所学习到的特征, 你将看到它们与哺乳动物视觉皮层性质的一些显著相似之处.

10.3.1 视觉皮层

研发 CNN 的大部分动机来自神经科学的开创性研究, 这些研究深入了解了包括人类在内的哺乳动物视觉处理的本质. 来自视网膜的电信号通过位于大脑后部的视觉皮层中的一系列处理层进行转换, 其中神经元组织成二维片, 每个二维片形成二维视野的映射. 在这些开创性研究中,Hubel and Wiesel (1959) 测量了当向猫的眼睛呈现视觉刺激时, 猫的视觉皮层中单个神经元的电响应. 他们发现, 一些称为“简单细胞”的神经元对视觉输入有强烈的响应, 这些视觉输入朝向特定角度的简单边缘, 位于视野内的特定位置. 而其他刺激在这些神经元中产生的响应相对较少. 更详细的研究表明, 这些简单细胞的响应可以使用 Gabor 滤波器来建模,Gabor 滤波器是由下式定义的二维函数:

$$G(x,y)=A\exp\left(-\alpha\tilde{x}^2-\beta\tilde{y}^2\right)\sin\left(\omega\tilde{x}+\phi\right), \tag{10.14}$$

其中

$$\tilde{x}=(x-x_0)\cos(\theta)+(y-y_0)\sin(\theta), \tag{10.15}$$

$$\tilde{y}=-(x-x_0)\sin(\theta)+(y-y_0)\cos(\theta). \tag{10.16}$$

(10.15)和(10.16)表示坐标系旋转角度 θ, 因此(10.14)中的 $\sin(\cdot)$ 项表示朝向极角 θ 方向的正弦空间振荡, 频率为 ω, 相位角为 ϕ.(10.14)中的指数因子旨在创建一个衰减包络, 将滤波器置于点 (x_0,y_0) 的邻域内, 衰减速率由 α 和 β 控制.(10.14)定义的 Gabor 滤波器示例如图 10.11所示.

定义 10.11 (Gabor 滤波器)

称由(10.14)–(10.16)定义的二元函数 $G:\mathbb{R}^2\to\mathbb{R}$ 为 **Gabor 滤波器**, 其中 $A>0$ 为振幅,$\alpha,\beta>0$ 为包络在 $\tilde{x},\tilde{y}$ 方向上的衰减速率,$\omega>0$ 为空间频率,ϕ 为相位角,θ 为朝向角,(x_0,y_0) 为滤波器的中心位置,$(\tilde{x},\tilde{y})$ 是绕 (x_0,y_0) 旋转 θ 后的局部坐标.

注

(重构新增) Gabor 滤波器是“Gauss 包络乘正弦条纹”, 这个形式的两部分各自对应一个可验证的性质. 正弦因子 $\sin(\omega\tilde{x}+\phi)$ 沿 $\tilde{x}$ 方向振荡、沿 $\tilde{y}$ 方向不变, 因此它是一个方向选择性滤波器: 只有与朝向 θ 垂直的边缘才会产生强响应, 这正是简单细胞的朝向选择性.Gauss 因子 $\exp(-\alpha\tilde{x}^2-\beta\tilde{y}^2)$ 使响应局限在 (x_0,y_0) 附近, 因此它是位置选择性的;$\alpha\neq\beta$ 时包络是椭圆的, 允许滤波器在振荡方向上延伸更长 (更窄的频带). 参数条件 $\alpha,\beta>0$ 不可省, 否则包络发散、函数不再可积;$\omega>0$ 只是约定振荡的“正方向”, 改变 ω 的符号等价于把 ϕ 平移 π.(10.16)中的负号保证 $(\tilde{x},\tilde{y})$ 是右手正交坐标, 若写成正号则旋转方向相反, 滤波器的朝向会与 θ 差一个镜像.

Hubel 和 Wiesel 还发现了“复杂细胞”的存在, 这些细胞会响应更复杂的刺激, 并且该刺激似乎是通过组合和处理简单细胞的输出得到的. 这些响应对输入中的微小变化 (例如位置偏移) 表现出一定程度的不变性, 类似于

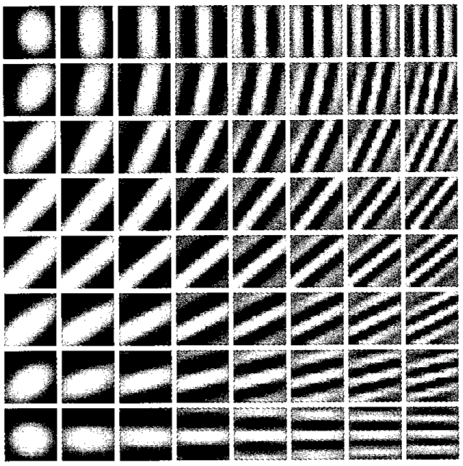


图 10.11: (10.14)定义的 Gabor 滤波器示例. 朝向角 θ 从顶行的 0 向底行的 $\pi/2$ 变化, 频率从左列的 $\omega = 1$ 向右列的 $\omega = 10$ 变化.

深度卷积网络中的池化单元. 哺乳动物视觉处理系统的更深层对视觉输入的变换具有更具体的响应和更大的不变性. 这种细胞称为“祖母细胞”, 因为不论场景的位置、大小、照明或其他变换如何, 当且仅当视觉输入与某些特定的概念 (如祖母的头像) 相对应时, 这种细胞才能在理论上做出响应. 这项工作直接启发了一种称为新认知机 (neocognitron)(Fukushima, 1980) 的深度神经网络的早期形式, 它是卷积神经网络的先驱. 新认知机支持多层处理, 包括了具有共享权重的局部感受野和紧接着的局部平均或最大池化, 以确保位置不变性. 然而, 新认知机缺乏端到端的训练过程, 而依赖于无监督聚类算法进行贪婪的逐层学习.

定义 10.12 (新认知机)

称满足下列条件的多层网络为**新认知机** (neocognitron): 每一层由若干二维单元片组成, 每个单元只连接前一层的一个局部感受野, 同一片内的单元共享权重; 每个卷积型的层之后紧跟一个局部平均或局部最大池化层, 以获得位置不变性; 各层以无监督聚类算法逐层贪婪训练, 而非端到端地最小化统一损失. ♣

注

(重构新增) 新认知机在结构上已经具备现代 CNN 的三要素: 局部感受野、权重共享、池化; 它缺少的是训练方式——贪婪逐层聚类无法把“分类错误”这一全局目标反传到前面的层, 因此每一层学到的表示只能局部最优. 定义把“逐层贪婪训练”写进对象本身, 是为了说明它与 CNN 的差别不在前向结构而在学习规则. 这也解释了为什么在反向传播 (第 8 章) 与大规模数据都具备之后, 同样的结构才真正发挥作用. 把池化紧接在卷积之后这一安排, 正是后文小节 10.2.6所沿用的模式.

10.3.2 可视化训练好的滤波器

假设我们有一个训练好的深度 CNN, 我们希望了解隐藏单元学会了检测什么. 对于第一个卷积层中的滤波器, 这是相对直接的, 因为它们对应于原始输入图像空间中的小块, 我们可以将这些滤波器的网络权重直接可视化为小的图像块. 第一个卷积层计算滤波器和相应图像块的内积, 当内积较大时, 隐藏单元将具有较大的激活.

图 10.12展示了一些在 ImageNet 数据集上训练好的 CNN 第一层滤波器示例. 我们看到这些滤波器与图 10.11中的 Gabor 滤波器之间存在显著的相似性. 然而, 这并不意味着卷积神经网络是大脑工作的良好模型, 因为我们可以从各种统计方法中获得非常相似的结果 (Hyvärinen, Hurri, and Hoyer, 2009). 这些特征滤波器表示的是统计自然图像的一般性质, 它们对于自然和人工系统中的图像理解都被证明是有用的.

虽然我们可以直接可视化第一层中的滤波器, 但网络中的后续层更难解释, 因为它们的输入不是图像小块, 而是滤波器响应. 一种方法 (类似于 Hubel 和 Wiesel 使用的方法) 是向网络提供大量图像小块, 并观察哪些图像小块

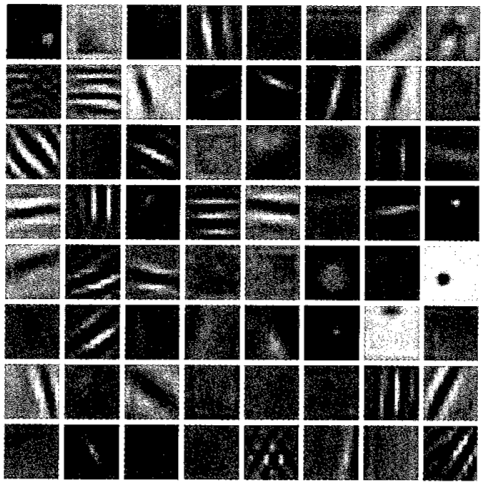


图 10.12: AlexNet 第一层学习到的滤波器示例. 注意, 许多学习到的滤波器与图 10.11中的 Gabor 滤波器具有惊人的相似性, 它们对应于哺乳动物视觉皮层中活跃神经元检测到的特征.

在任何特定隐藏单元中产生最大的激活值.图 10.13显示了在网络的隐藏单元中产生最强激活的图像小块示例. 该网络有 5 个卷积层, 紧接着是两个全连接层, 在包含 1 000 个类别的 130 万幅 ImageNet 图像上完成训练. 我们看到一个自然的分层结构, 第 1 层对边缘响应, 第 2 层对纹理和简单形状响应, 第 3 层显示对象的组成部分 (如轮子), 第 5 层显示整个对象.

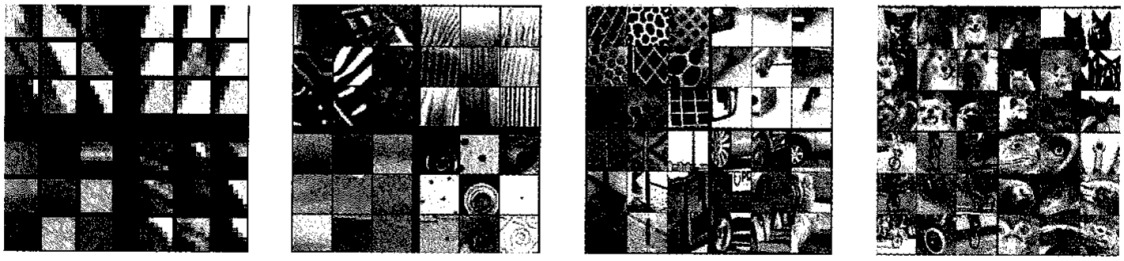


图 10.13: 在网络的隐藏单元中产生最强激活的图像小块示例 (取自验证集). 该网络有 5 个卷积层, 在 ImageNet 图像上完成训练. 针对 4 个随机选择的通道, 每个对应层的特征图中的前 9 个激活排成了一个 3×3 的网格. 我们看到复杂性会随着深度的增加稳步增长——从第 1 层的简单边缘到第 5 层的完整对象 [经 Zeiler and Fergus (2013) 授权使用] .

我们可以扩展这种技术, 使其不仅仅从验证集中选择图像小块, 而且对输入变量进行数值优化, 以最大限度地激活特定单元 (Zeiler and Fergus, 2013; Simonyan, Vedaldi, and Zisserman, 2013; Yosinski et al., 2015). 如果我们选择某单元作为输出之一, 则可以寻找最能代表相应类标签的图像. 由于输出单元通常具有 softmax 激活函数, 因此最好最大化输入 softmax 激活函数的预激活值, 而不是直接最大化类概率, 因为这样可以确保优化仅依赖于一个类. 例如, 假设我们想要寻找对类别 “狗” 产生最强烈响应的图像, 那么如果我们优化 softmax 输出, 由于 softmax 激活函数中的分母的存在, 可能会使图像变得不那么像猫. 然而, 输出单元激活的无约束优化会导致单个像素值趋于无穷大, 并且还会产生难以解释的高频结构, 因此需要某种形式的正则化来找到更接近自然图像的解.Yosinski et al. (2015) 使用了一种正则化函数, 该函数涉及像素值的平方和以及一个变换过程. 该过程将基于梯度的更新替换为删除高频结构的模糊操作, 并引入剪裁操作来将那些对类标签贡献很小的像素值设置为零, 示例结果如图 10.14所示. 这种方法与对抗训练有关 (参见第 17 章).

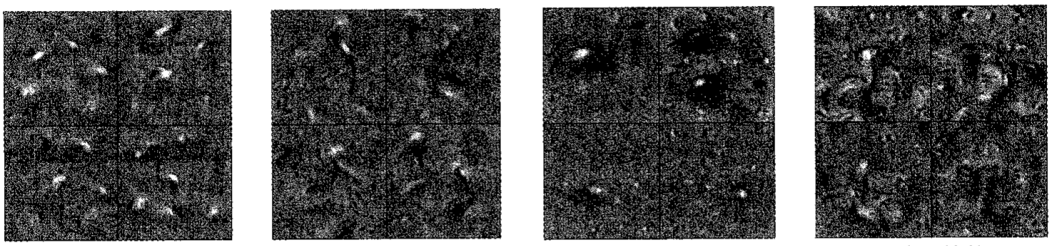


图 10.14: 通过训练好的卷积分类器, 对图像像素通道值最大化类概率生成的合成图像示例. 图中显示了对于每一对象类别, 通过不同正则化参数设置得到的 4 个不同的解［经 Yosinski et al. (2015) 授权使用］.

10.3.3 显著性图

深入了解卷积网络所用特征的另一种方法是使用显著性图 (saliency map), 目的是在确定类标签时识别图像中最重要的区域. 这最好通过研究最终卷积层来完成, 因为该卷积层仍然保留了空间定位信息, 该信息在随后的全连接层中会丢失, 但它们具有最高级别的语义表示. 对于给定的输入图像,Grad-CAM(Gradient Class Activation Mapping, 梯度类激活图) 方法 (Selvaraju et al., 2016) 首先在 softmax 输出之前, 计算对于给定类别 c 的输出单元预激活 $a^{(c)}$ 关于最终卷积层通道 k 中所有单元预激活 $a_{ij}^{(k)}$ 的导数. 对于该层中的每个通道, 这些导数的平均值形式如下:

$$\alpha_k = \frac{1}{M_k} \sum_i \sum_j \frac{\partial a^{(c)}}{\partial a_{ij}^{(k)}}, \tag{10.17}$$

其中 i 和 j 用于索引通道 k 的行和列,M_k 是通道 k 中的单元总数. 然后, 这些平均值被用于形成以下形式的加权和:

$$L = \sum_k \alpha_k A^{(k)}, \tag{10.18}$$

其中 $A^{(k)}$ 是包含元素 $a_{ij}^{(k)}$ 的矩阵. 生成的数组与最终卷积层具有相同的维度, 例如图 10.10所示的 VGG-16 模型的维度为 14×14, 并且可以用 “热力图” 的形式叠加在原始图像上, 如图 10.15所示.

定义 10.13 (显著性图与 Grad-CAM)

设 $a^{(c)}$ 为类别 c 在 softmax 之前的预激活,$a_{ij}^{(k)}$ 为最后一个卷积层第 k 个通道在位置 (i, j) 处的预激活. 称按(10.17)算出的 α_k 为通道 k 对类别 c 的**权重**, 称按(10.18)算出的矩阵 L 为类别 c 的 **Grad-CAM 显著性图** (saliency map).

♣

注

(重构新增) 显著性图的定义把 “哪些像素重要” 转化为 “哪些通道重要、以及这些通道在哪里激活”.(10.17)对空间位置取平均, 是为了让权重与位置无关 (否则热力图会退化成一堆逐点导数);(10.18)再用该权重把各通道的激活图加权求和, 于是 L 与最后一个卷积层同尺寸. 定义中要求 $a^{(c)}$ 是 softmax 之前的预激活而不是概率, 因为概率经 softmax 归一化后所有类别的梯度相互耦合, 单看某一类的概率导数会混入其他类的信息. 注意 L 可以取负值: 负值表示该位置对类别 c 起反作用, 实际可视化时常截断为 $\max(L, 0)$ 再归一化, 这只是显示上的处理, 不改变定义.

10.3.4 对抗攻击

与输入图像像素值变化相关的梯度也可用于创建针对卷积网络的对抗攻击 (adversarial attack)(Szegedy et al., 2013). 这类攻击以人类无法察觉的水平对图像进行非常小的修改, 从而导致神经网络对图像分类错误. 创建对抗性图像的一种简单方法是快速梯度符号 (fast gradient sign) 法 (Goodfellow, Shlens, and Szegedy, 2014). 这涉及将图像 $\mathbf{x}$ 中的每个像素值更改为固定量 ε, 符号由误差函数 $E(\mathbf{x}, t)$ 相对于像素值的梯度确定. 我们将得到一幅由下式

原始图像　　“狗”的显著性图　　“猫”的显著性图

图 10.15: VGG-16 模型关于类别“狗”和“猫”的显著性图［经 Selvaraju et al. (2016) 授权使用］.

定义的修改后的图像:

$$\tilde{\mathbf{x}} = \mathbf{x} + \varepsilon \operatorname{sign}(\nabla_{\mathbf{x}} E(\mathbf{x}, t)), \tag{10.19}$$

这里的 t 是 $\mathbf{x}$ 的真实标签, 误差 $E(\mathbf{x}, t)$ 举例来说可以是 $\mathbf{x}$ 的负对数似然. 使用反向传播可以有效地计算所需梯度(参见第 8 章). 在神经网络的常规训练中, 可以通过调整网络参数来最小化误差, 而(10.19)定义的修改会改变图像(同时保持训练好的网络参数固定), 从而导致误差增加. 通过保持 ε 为较小的值, 可以确保人眼无法察觉图像的变化. 值得注意的是, 这可以使网络以较高的置信度将图像分类错误, 如图 10.16中的示例所示.

定义 10.14 (对抗攻击与快速梯度符号法)

设已训练好的分类器在输入 $\mathbf{x}$ 上的误差为 $E(\mathbf{x}, t)$,t 为真实标签. 若以人类难以察觉的幅度修改输入, 使分类器输出错误类别, 则称这种修改为**对抗攻击** (adversarial attack), 修改后的输入称为**对抗样本**. 若修改按(10.19)进行 ($\varepsilon > 0$ 为步长,sign 为逐元素符号函数), 则称该方法为**快速梯度符号法** (fast gradient sign method).

♣

注

(重构新增) 对抗攻击的定义强调两件事: 扰动幅度要小 (否则不算“攻击”而只是换了一张图), 以及攻击目标是让分类器出错而不是让误差函数本身取到某个指定值.(10.19)之所以用符号函数而不用梯度本身, 是因为在梯度各分量大小悬殊时, 逐分量取符号可以把有限的总扰动预算 ε 平均分配到所有像素上, 从而使误差的一阶增量最大 (见命题 10.9). 这也解释了为什么扰动看起来像均匀噪声: 它把预算用满了, 却几乎不改变图像的低频结构. 注意 ε 必须相对于像素取值范围足够小, 且修改后通常还要把像素投影回合法区间 $[0, 255]$.

命题 10.9 (快速梯度符号法的一阶效应)

设 E 在 $\mathbf{x}$ 的一个邻域内可微,$\mathbf{g} = \nabla_{\mathbf{x}} E(\mathbf{x}, t)$,$\boldsymbol{\delta}$ 为满足 $\|\boldsymbol{\delta}\|_\infty \leqslant \varepsilon$ 的扰动. 则一阶近似下

$$E(\mathbf{x} + \boldsymbol{\delta}, t) \approx E(\mathbf{x}, t) + \mathbf{g}^{\mathsf{T}} \boldsymbol{\delta}, \tag{10.20}$$

且在约束 $\|\boldsymbol{\delta}\|_\infty \leqslant \varepsilon$ 下,$\mathbf{g}^{\mathsf{T}} \boldsymbol{\delta}$ 的最大值在 $\boldsymbol{\delta} = \varepsilon \operatorname{sign}(\mathbf{g})$ 处取得, 最大值为 $\varepsilon \|\mathbf{g}\|_1$.

♠

证明

由可微性,$E(\mathbf{x} + \boldsymbol{\delta}, t) = E(\mathbf{x}, t) + \mathbf{g}^{\mathsf{T}} \boldsymbol{\delta} + o(\|\boldsymbol{\delta}\|)$, 即(10.20). 对任意满足 $|\delta_i| \leqslant \varepsilon$ 的 $\boldsymbol{\delta}$,

$$\mathbf{g}^{\mathsf{T}} \boldsymbol{\delta} = \sum_i g_i \delta_i \leqslant \sum_i |g_i|\,|\delta_i| \leqslant \varepsilon \sum_i |g_i| = \varepsilon \|\mathbf{g}\|_1.$$

取 $\delta_i = \varepsilon \operatorname{sign}(g_i)$(约定 $\operatorname{sign}(0) = 0$) 时 $|\delta_i| \leqslant \varepsilon$ 且 $g_i \delta_i = \varepsilon |g_i|$, 故等号成立. 证毕.

□

注

(重构新增) 这个命题说明快速梯度符号法不是启发式技巧, 而是在无穷范数预算下对一阶误差增量最优的扰动. 两个条件都不能丢:$\|\boldsymbol{\delta}\|_\infty \leqslant \varepsilon$ 规定了“人眼不可察觉”的度量, 若换成二范数约束, 最优扰动就变成沿梯度方向而非逐像素取符号;E 可微则是线性化的前提, 在 ReLU 的折点处梯度不连续, 一阶近似只在局部有效. 命题只保证误差的一阶增量最大, 不保证分类一定翻转: 若 $\varepsilon\|\mathbf{g}\|_1$ 小于当前正确类与错误类之间的间隔, 攻击就不会成功, 这也是“增大 ε 提高成功率、但增大到可见”这一取舍的来源.

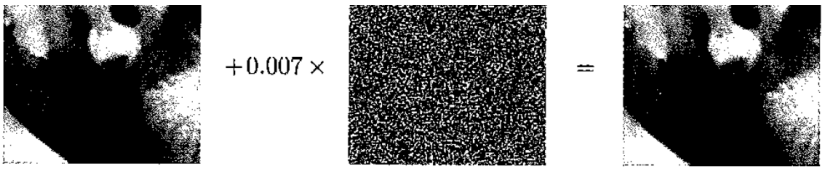


图 10.16: 针对训练好的卷积网络的攻击示例. 左边的图像以 57.7% 的置信度归类为熊猫. 添加少量随机扰动 (这些扰动本身以 8.2% 的概率归类为线虫), 导致右边的图像以 99.3% 的置信度归类为长臂猿 [经 Goodfellow, Shlens, and Szegedy (2014) 授权使用].

以这种方式欺骗神经网络会引发潜在的安全问题, 因为它为攻击训练好的分类器创造了机会. 这个问题似乎是由过拟合引起的, 在过拟合中, 具有较高拟合能力的模型已经精确地适应了特定的图像, 使得输入中的微小变化会导致预测类概率的大幅变化. 然而事实证明, 一幅经过调整能使特定训练好的网络产生虚假输出的图像, 在被输入其他网络时也会导致类似的虚假输出 (Goodfellow, Shlens, and Szegedy, 2014). 此外, 使用灵活性低得多的线性模型可以得到类似的对抗结果. 人工对图像进行物理修改也可能导致这种现象. 当把一幅规则的、未损坏的人工图像呈现给训练好的神经网络时, 该网络也会给出错误的预测, 如图 10.17所示. 虽然这些对抗攻击的基本类型可以通过对网络训练过程进行简单修改来解决, 但更复杂的对抗攻击方法处理起来依然较为困难. 了解这些结果的含义并缓解隐患仍然是一个开放的研究领域.

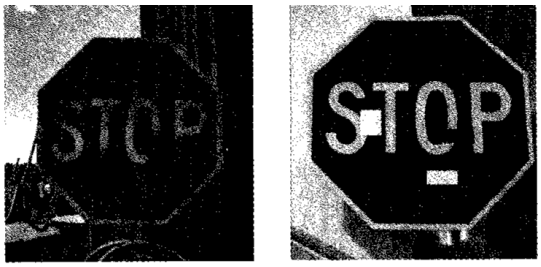


图 10.17: 两个修改过的物理停车标志, 它们会被 CNN 以极高的概率归类为限速标志 [经 Eykholt et al. (2018) 授权使用].

10.3.5 合成图像

在关于图像修改的最后一个示例中, 我们将讨论一种称为 DeepDream 的技术 (Mordvintsev, Olah, and Tyka, 2015), 它为运行训练好的卷积网络提供了额外的见解. 它的目标是生成具有夸张特征的合成图像. 为此, 我们需要确定网络特定隐藏层中的哪些节点对特定图像有较强响应, 然后修改图像从而放大这些响应. 例如, 我们将一些云的图像输入一个在对象检测任务上训练好的网络, 并且如果一个特定节点在图像的特定区域检测到类似猫的模式, 则将该区域修改得更像“猫”.

为此, 我们首先将图像输入网络, 并前向传播到某个特定层. 然后将该层的反向传播 δ 变量设置为等于节点的预激活, 并向输入层反向传播, 从而获得图像像素的梯度向量 (参见节 8.1). 最后沿着梯度向量前进一小步来修改

图像. 该过程可以视作利用一种基于梯度的方法来增大函数的值 (见练习 10.10). 要最大化的函数为

$$F(I) = \sum_{i,j,k} a_{ijk}(I)^2, \tag{10.21}$$

其中 $a_{ijk}(I)$ 是当输入图像为 I 时, 所选网络层中通道 k 的第 i 行和第 j 列单元的预激活, 求和操作则涉及该层中所有的单元和通道. 为了生成平滑的图像, 可以使用空间平滑和像素裁剪形式的某些正则化. 如果需要更强的增强效果, 可以多次重复此过程. 将 DeepDream 技术应用于图像的示例如图 10.18所示. 有趣的是, 即使卷积网络被训练以区分对象类别, 但它们似乎至少能够捕获从这些类生成图像所需的一些信息. 这种技术可以应用于照片, 或者我们也可以将随机噪声作为输入, 从而完全由训练好的网络生成图像. 尽管 DeepDream 技术提供了一些关于训练好的网络运行的见解, 但它主要用于生成艺术风格的有趣图像.

定义 10.15 (DeepDream)

设训练好的网络在输入 I 上给出某层预激活 $a_{ijk}(I)$,F 由(10.21)定义. 从图像 I_0 出发, 按下式迭代

$$I_{\tau+1} = I_\tau + \eta \nabla_I F(I_\tau), \qquad \tau = 0, 1, 2, \cdots, \tag{10.22}$$

其中 $\eta > 0$ 为步长, 并对结果施加空间平滑与像素裁剪. 称这一过程为 **DeepDream**.

♣

注

(重构新增) DeepDream 的定义把“让某个单元更兴奋”写成一个无约束的梯度上升问题, 其目标函数(10.21)是所选层全部预激活的平方和, 因此它不针对单一类别, 而是放大该层已经学到的所有模式. 平方的作用是让大幅激活主导目标 (大响应的梯度也大), 从而在合成图中产生夸张的纹理. 定义中必须包含正则化: 若不约束, 梯度上升会把像素推到区间外并生成高频棋盘状结构, 图像不再像自然图像; 空间平滑与裁剪正是把迭代限制在“看起来像图像”的集合内. 该过程不更新网络参数, 只更新输入, 这一点与训练有本质区别.

命题 10.10 (DeepDream 的上升方向)

设 F 在 I 处可微,$\nabla_I F(I) \neq \mathbf{0}$. 则对充分小的 $\eta > 0$, 沿(10.22)更新后目标函数严格增加:

$$F\big(I + \eta \nabla_I F(I)\big) > F(I). \tag{10.23}$$

♠

证明

由可微性,$F(I + \eta \mathbf{g}) = F(I) + \eta\, \mathbf{g}^{\mathsf{T}}\mathbf{g} + o(\eta\|\mathbf{g}\|)$, 其中 $\mathbf{g} = \nabla_I F(I)$. 于是

$$\frac{F(I + \eta\mathbf{g}) - F(I)}{\eta} = \|\mathbf{g}\|^2 + \frac{o(\eta\|\mathbf{g}\|)}{\eta} \longrightarrow \|\mathbf{g}\|^2 > 0 \qquad (\eta \to 0^+).$$

故存在 $\eta_0 > 0$, 使 $0 < \eta < \eta_0$ 时差商为正, 即 $F(I + \eta\mathbf{g}) > F(I)$. 证毕.

□

注

(重构新增) 这个命题说明 DeepDream 的每一步都在局部增加目标函数, 前提是步长足够小且梯度非零. 梯度为零时命题不适用: 此时 I 已是驻点, 继续迭代不会带来一阶改进, 实际算法会停止或需要更大的步长跳出. 命题只保证“局部增加”, 不保证 F 有上界:(10.21)是预激活的平方和, 若网络输出无界则 F 无上界, 迭代会不断放大像素, 这正是需要正则化的数学理由. 把该结论与命题 10.9对比可以看出两者是同一事实的两面: 一个沿梯度上升以最大化目标, 一个沿符号梯度上升以在无穷范数预算内最大化误差.

10.4 目标检测

前面主要以图像分类问题为例介绍了 CNN 的设计动机, 其中整幅图像被分类为单一类别, 例如“猫”或“自行车”. 这在诸如 ImageNet 的数据集上是合理的, 因为每幅图像都设计为主要包含单一对象. 然而, 还有许多其他应用可以利用 CNN 内建的归纳偏置. 更一般地说, 如果 CNN 中的卷积层已经在大型图像数据集的特定任务上训练好, 那么 CNN 就可以学习具有广泛适用性的内部表示. 因此, 我们可以针对大量的特定任务对 CNN 进行微调

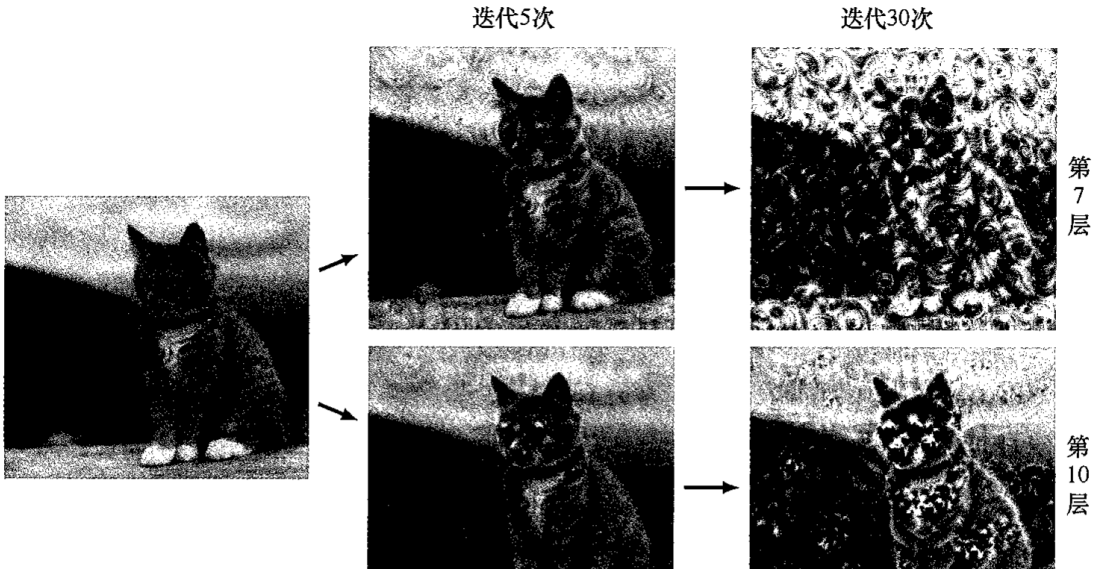


图 10.18: 将 DeepDream 技术应用于图像的示例. 上方显示了在经过 5 次迭代和 30 次迭代后, 应用算法到 VGG-16 模型的第 7 个卷积层激活的结果. 类似地, 下方显示了同样在经过 5 次迭代和 30 次迭代后, 应用算法到 VGG-16 模型的第 10 个卷积层激活的结果.

(参见节 6.3). 你已经看到了一个在 ImageNet 数据上训练好的卷积网络的例子, 其通过迁移学习能够在皮肤病变图像的分类上达到人类水平 (参见节 1.1).

图像中有多个对象属于一个或多个类别的情况很常见, 我们可能希望检测其中每个对象的存在及其所属的类别. 此外, 在许多计算机视觉应用中, 我们还需要确定在图像中检测到的任何对象的位置. 例如, 使用 RGB 摄像头的自动驾驶车辆可能需要检测行人的存在和位置, 并识别道路标志、其他车辆等.

10.4.1 边界框

考虑明确图像中对象位置的问题. 一种广泛使用的方法是定义一个边界框 (bounding box), 它是一个紧密贴合对象边界的矩形, 如图 10.19所示. 边界框可以由其中心坐标以及宽度和高度定义, 形式为向量 $\mathbf{b} = (b_x, b_y, b_w, b_h)$. 这里 $\mathbf{b}$ 中的元素可以以像素为单位指定, 也可以用连续数字表示. 按照惯例, 图像的左上角对应坐标 $(0, 0)$, 右下角对应坐标 $(1, 1)$.

定义 10.16 (边界框)

称图像中一个紧密贴合对象边界的矩形为**边界框** (bounding box), 并用向量 $\mathbf{b} = (b_x, b_y, b_w, b_h)$ 描述它, 其中 (b_x, b_y) 为框中心坐标,b_w 与 b_h 为宽度与高度. 若采用归一化坐标, 则图像左上角为 $(0, 0)$、右下角为 $(1, 1)$, 此时 $b_x, b_y, b_w, b_h \in [0, 1]$; 若采用像素坐标, 则各分量以像素为单位.

♣

注

(重构新增) 把边界框参数化为“中心 + 宽高”而不是“左上角 + 右下角”, 是为了让四个分量在尺度变换下具有统一的行为: 平移只改变 b_x, b_y, 缩放同时改变四个分量, 回归时不会出现“一个角动、另一个角不动”的耦合. 定义中两种坐标制的并存也有实际后果: 归一化坐标使不同分辨率的图像可以共用同一套回归目标, 像素坐标则便于与图像数组直接对齐; 二者之间的换算是固定的线性变换, 不改变框的几何. 矩形“紧密贴合”只是语义要求, 实际标注总会留下余量, 因此预测框与真值框几乎不可能完全重合, 这正是需要定义 10.17的原因.

当假设图像有且仅有一个对象, 且该对象来自一个预定义的类别集合 (其中的类别数量为 C) 时,CNN 通常会有 C 个输出单元, 其激活函数由 softmax 函数定义 (参见第 5 章). 对象可以由 4 个额外的具有线性激活函数的输出来定位, 用于训练预测边界框坐标 (b_x, b_y, b_w, b_h). 由于这些量是连续的, 因此可以使用对应输出的平方和误差

图 10.19: 包含不同类别的多个对象的图像, 其中每个对象的位置由一个称为边界框的紧密贴合矩形标记. 蓝色框对应“汽车”类别, 红色框对应“行人”类别, 橙色框对应“交通信号灯”类别 [原始图像由 Wayve Technologies Ltd. 提供] .

函数. 例如,Redmon et al. (2015) 使用此方法首先将图像划分成一个 7×7 的网格; 然后对于每个网格单元, 使用卷积网络并基于从整个图像中提取的特征, 输出与网格单元关联的任意对象的类别和边界框坐标.

10.4.2 交并比

我们需要一种有意义的方式来衡量训练好的网络在预测边界框方面的性能. 在图像分类中, 网络的输出是类别标签的概率分布, 我们可以通过查看测试集上真实类别标签的对数似然来衡量性能. 然而, 在对象定位中, 我们需要某种方法来衡量边界框相对于某些真值的准确性, 其中后者可以通过像人工标注这样的方式获得. 预测框与目标框的重合程度可以用作这种度量的基础, 但重叠区域的面积会取决于图像中对象的大小. 此外, 如果预测的边界框超出实际的边界框, 则应该受到惩罚. 一种能够较好地解决这两个问题的度量方法称为交并比 (Intersection-over-Union, IoU), 它是这两个边界框的交集区域面积与并集区域面积的比值, 如图 10.20所示. 注意,IoU 度量的取值范围是 0 ~ 1. 如果 IoU 度量超过某个阈值 (通常为 0.5), 则可以将预测标记为正确. 注意,IoU 通常不直接用作训练的损失函数, 因为想要通过梯度下降优化它很困难, 因此通常使用中心对象来训练, 而 IoU 分数主要用作评价指标.

定义 10.17 (交并比)

设 B 与 B' 是两幅同坐标系下的边界框, 记 $|B|$ 为框 B 所围区域的面积. 称

$$\mathrm{IoU}(B, B') = \frac{|B \cap B'|}{|B \cup B'|} \tag{10.24}$$

为 B 与 B' 的**交并比** (Intersection-over-Union, IoU), 并约定当分母为零 (两框面积均为零) 时该值无定义. ♣

注

(重构新增) IoU 的设计回应了两个具体问题: 用比值而不是交集面积, 消除了对象大小对度量的影响 (大对象天然有大交集); 用并集作分母而不是用真值框面积, 使“预测框超出真值框”受到惩罚 (超出部分增大并集、压低比值). 定义中“同坐标系”不能省: 若一个是归一化坐标、另一个是像素坐标, 比值会算出无意义的数. 分母为零的退化情形必须排除, 否则出现 0/0.IoU 不是损失函数的好选择, 原因在定义之外: 它是分段有理函数, 在两框不相交时梯度为零, 无法告诉模型“该往哪个方向移动框”, 这也是实际训练常用平滑替代损失 (如 1 – IoU 的可微变体或平方和误差) 而只在评价时用 IoU 的原因.

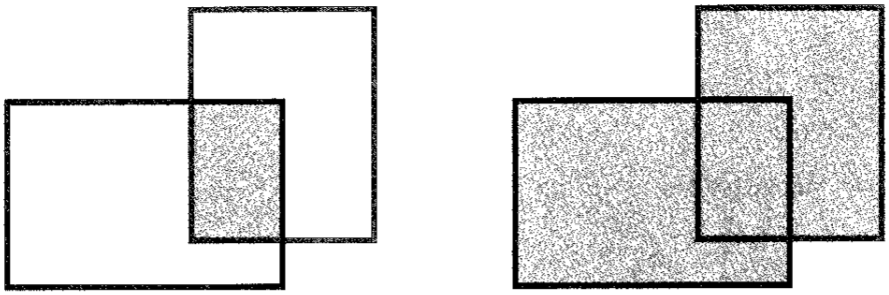


图 10.20: 用于量化边界框预测准确性的交并比示意图. 如果预测的边界框如蓝色矩形所示, 而实际的边界框如红色矩形所示, 则将交并比定义为这两个框的交集区域面积 (左图的绿色部分) 与并集区域面积 (右图的绿色部分) 的比值.

命题 10.11 (交并比的取值范围与退化情形)

设 B, B' 为面积非零的边界框. 则 $0 \leqslant \mathrm{IoU}(B, B') \leqslant 1$, 且 $\mathrm{IoU}(B, B') = 1$ 当且仅当 $B = B'$(忽略零面积的边界重叠); 当 $B \cap B'$ 的内部为空时 $\mathrm{IoU}(B, B') = 0$. 此外, 若把两框同时平移同一向量, 则 IoU 不变.

♠

证明

由 $B \cap B' \subseteq B \cup B'$ 及面积单调性,$0 \leqslant |B \cap B'| \leqslant |B \cup B'|$, 而分母 $|B \cup B'| > 0$, 故 $0 \leqslant \mathrm{IoU} \leqslant 1$. 若 $\mathrm{IoU} = 1$, 则 $|B \cap B'| = |B \cup B'|$, 而 $B \cap B' \subseteq B \cup B'$, 故二者至多相差一个零面积集, 即 $B = B'$(忽略边界). 反之 $B = B'$ 时交集等于并集, 比值为 1. 若两框内部不相交, 则 $|B \cap B'| = 0$, 故 IoU 为 0. 平移不变性: 设 T 为平移, 则 $T(B \cap B') = T(B) \cap T(B')$、$T(B \cup B') = T(B) \cup T(B')$, 且平移保持面积, 故比值不变. 证毕.

□

注

(重构新增) 这个命题给出的是 IoU 作为度量的最低要求: 有界、只在完全重合时取上界、不相交时取下界、对位置平移不敏感 (它只关心相对重合). 注意“当且仅当 $B = B'$”排除了仅边界接触的情形, 因为边界面积为零、不影响比值, 这说明 IoU 对“贴合但不重叠”和“完全分离”给出相同的分数 0, 这是它作为评价指标的粗糙之处. 平移不变性有实际用途: 它说明 IoU 不依赖于对象在图像中的绝对位置, 因此可以跨图像比较. 命题没有说 IoU 对旋转或缩放不变, 事实上矩形框在旋转下不再是矩形, 定义本身就不再适用.

10.4.3 滑动窗口

一种进行目标检测和目标定位的方法是, 首先创建一个训练集, 其中既包括所要检测对象的紧密裁剪的样本, 也包括不包含任何对象 (“背景”类别) 的图像类似裁剪区域的样本. 这个训练集用于训练分类器, 例如一个深度 CNN, 其输出表示输入窗口中存在特定对象类别的概率. 然后, 训练好的模型通过在图像上“扫描”输入窗口来检测新图像中的对象, 并针对每个位置, 将图像的结果集合用作分类器的输入. 这称为滑动窗口 (sliding window). 当以较高概率检测到对象时, 窗口位置就是相应的边界框.

定义 10.18 (滑动窗口)

设已用“目标窗口/背景窗口”训练好一个窗口分类器 f(输出窗口内存在各类对象的概率). 在图像上按固定步幅依次放置尺寸固定的输入窗口, 对每个窗口计算 f, 并把概率超过阈值的窗口位置作为检测结果, 称此检测方法为**滑动窗口** (sliding window). 窗口的尺寸与步幅统称为该方法的尺度参数.

♣

注

(重构新增) 滑动窗口把“检测”化归为“逐窗口分类”, 因此它不需要新的学习算法, 只需要一个已经训练好的分类器; 代价是计算: 窗口位置数约为 $(J/s)\times(K/s)$, 每个位置都要跑一次完整前向传播. 定义中的步幅是一个显式的取舍参数: 步幅小则定位精度高但计算量大, 步幅大则相反. 阈值则控制查全与查准的平衡. 注意定义要求分类器在“背景”类上训练过, 否则网络会对不含对象的窗口也给出高置信度的某一类, 产生大量假阳性. 后文小节 10.4.3的卷积化实现之所以能大幅降低计算量, 是因为它把所有这些窗口的前向传播合并成了一次卷积, 而不是因为改变了定义.

这种方法的一个明显缺点是, 由于图像中潜在窗口位置的数量十分庞大, 因此计算成本可能非常高. 此外, 为了适应图像中不同大小的对象, 可能需要使用不同尺度的窗口重复该过程. 我们可以通过在图像上以大于 1 像素的步幅在水平和垂直方向上移动输入窗口来节省计算成本. 然而, 使用较小步幅提高定位精度与使用较大步幅降低计算成本之间存在权衡. 滑动窗口方法的计算成本对于简单分类器而言可能是合理的, 但对于潜在包含数百万参数的深度神经网络来说, 计算成本难以承受.

幸运的是, 神经网络的卷积结构可以大大提高效率 (Sermanet et al., 2013). 我们注意到, 这种网络中的卷积层本身就涉及在输入图像中以跨步的方式滑动共享权重的特征检测器. 因此, 当使用滑动窗口通过卷积网络进行多次前向传播时, 计算中存在大量冗余, 如图 10.21所示.

由于滑动窗口的计算结构与卷积的计算结构相似, 因此在卷积网络中高效地实现滑动窗口非常简单. 考虑图 10.22中一个简化的卷积网络, 它包括卷积层, 后跟最大池化层, 然后是全连接层. 为简便起见, 我们在每层中只显示一个通道, 但扩展到多个通道也很容易. 网络的输入图像大小为 6×6, 卷积层中的滤波器大小为 3×3, 步幅为 1, 最大池化层具有大小为 2×2、步幅为 1 的非重叠感受野. 然后是一个具有单个输出单元的全连接层. 注意, 我们也可以将这个最终层视为另一个滤波器大小为 2×2 的卷积层, 以使滤波器只有一个位置, 从而只产生一个输出.

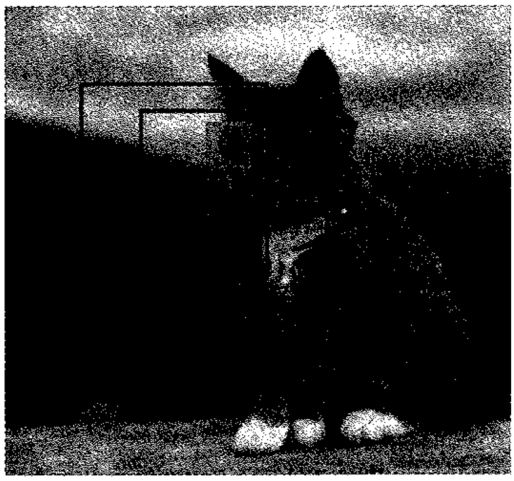


图 10.21: 当用 CNN 处理来自滑动输入窗口的数据时, 出现重复计算的示意图. 其中红色框和蓝色框显示了有部分位置重叠的两个输入窗口. 绿色框表示第一卷积层中隐藏单元感受野的位置之一, 对应隐藏单元激活的计算在这两个窗口位置可以共享.

假设该网络是在对象的中心图像上训练的, 然后应用于 8×8 大小的更大图像, 如图 10.23所示. 其中, 我们只是通过增大卷积层和最大池化层来扩大网络. 现在, 卷积层的大小为 6×6, 池化层的大小为 3×3, 有 4 个输出单元, 每个输出单元都有自己的 softmax 函数. 输入其中一个输出单元的权重将在这 4 个输出单元之间共享. 我们看到, 处理位于输入图像左上角的窗口位置所需的计算与处理训练阶段原始 6×6 输入的计算相同. 对于其余的窗口位置, 则只需要少量的额外计算, 如蓝色方块所示. 与简单地重复应用完整卷积网络相比, 这显著提高了效率 (见练习 10.12). 注意, 现在全连接层本身具有卷积结构.

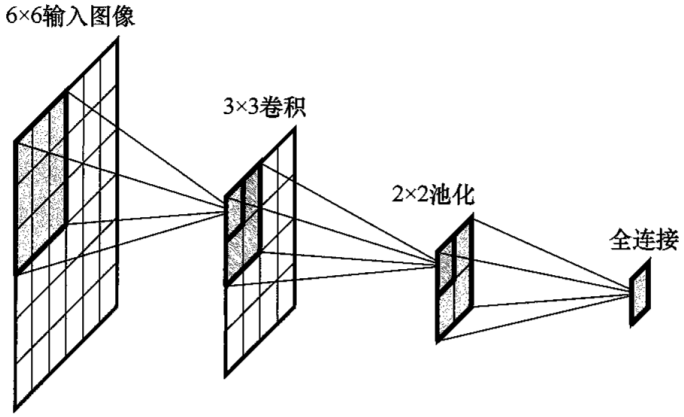


图 10.22: 每层具有单通道的简单卷积网络的示例, 用以说明用于目标检测的滑动窗口的概念.

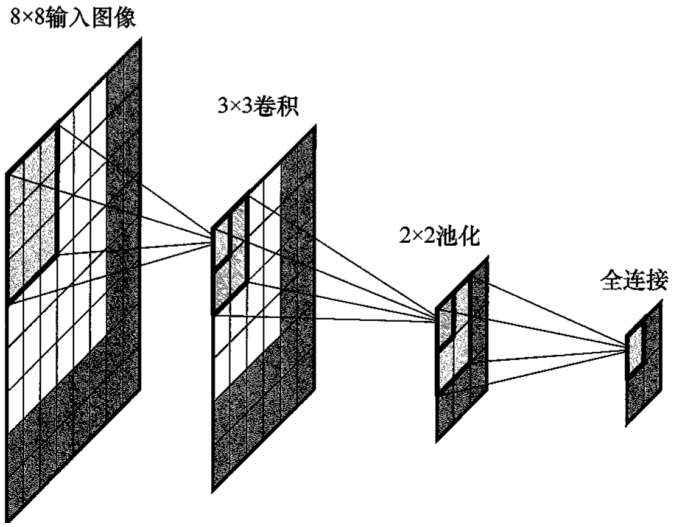


图 10.23: 将图 10.22所示的简单卷积网络应用于较大的图像, 所需的额外计算对应于蓝色区域.

10.4.4 跨尺度检测

除了在图像中寻找不同位置的对象之外, 还需要在不同尺度和不同的宽高比下寻找对象. 例如, 猫直立时为其绘制的边界框, 与其躺下时为其绘制的边界框具有不同的宽高比. 与使用输入窗口具有不同大小和形状的多个检测器相比, 使用固定输入窗口并制作多个具有不同水平和垂直缩放因子的输入图像副本的方式更加简单, 但是效果相同. 后者在每个图像副本上扫描输入窗口以检测对象, 并使用关联的缩放因子将边界框坐标转换回原始图像空间, 如图 10.24所示.

图 10.24: 使用固定输入窗口在多个尺度和宽高比下检测和定位对象的示意图. 原始图像 (a) 被复制多次, 并且每个副本在水平和/或垂直方向上被缩放, 如 (b) 中的水平缩放所示. 然后在缩放的图像上以固定大小的窗口进行扫描. 当以高概率检测到对象时, 如 (b) 中的红框所示, 可以将相应的窗口坐标投影回原始图像空间, 从而确定相应的边界框, 如 (c) 中所示.

10.4.5 非最大抑制

通过在图像上扫描训练好的卷积网络, 可以检测图像中同一类别对象的多个实例, 以及来自其他类别对象的实例. 然而, 这也倾向于在相似位置产生同一对象的多个检测结果, 如图 10.25所示. 可以使用非最大抑制 (non-max suppression) 来解决这个问题, 对于每个对象类别, 非最大抑制的工作方式如下: 首先在整个图像上运行滑动窗口, 并计算每个位置存在该对象类别的概率; 然后消除所有概率低于某个阈值 (例如 0.7) 的边界框, 得到类似于图 10.25所示的结果——具有最高概率的框被认为是有效检测, 并且将相应的边界框记录作为预测结果; 接下来, 将任何与有效检测的边界框的 IoU 超过某个阈值 (例如 0.5) 的其他框丢弃, 从而消除针对同一对象的多个邻近检测结果; 最后在剩余的框中, 将具有最高概率的框选为另一个有效检测, 并且重复上述消除步骤. 将这个过程持续下去, 直至所有边界框要么丢弃, 要么选为有效检测.

定义 10.19 (非最大抑制)

给定同一类别的一组候选边界框及其得分 (例如属于该类别的概率), 以及得分阈值 p 与重合阈值 ι, **非最大抑制** (non-max suppression) 按如下步骤给出保留集合 $\mathcal{K}$:

1. 丢弃所有得分低于 p 的候选框;
2. 在剩余框中取得分最高者 B^*, 将其加入 $\mathcal{K}$ 并从候选集中删除;
3. 删除候选集中所有与 B^* 的 IoU 超过 ι 的框;
4. 重复第 2、3 步, 直至候选集为空.

♣

注

(重构新增) 非最大抑制处理的是滑动窗口方法的固有冗余: 同一对象会被多个位置、多个尺度同时检出, 且这些框高度重合. 算法用两个阈值把两类错误分开: p 控制“要不要这个检测”(低于 p 的直接丢弃, 减少假阳性), ι 控制“两个检测是否指同一对象”(IoU 高则视为重复). 两个阈值的作用不同, 不能合并: 把 ι 调大可以保留密集排列的不同对象 (如人群), 把 ι 调小则更激进地合并. 定义对每个类别独立执行, 因为不同类别的框可以合法地重合. 算法必然终止: 每一步至少删除一个框, 候选集有限. 注意它不保证找到全局最优的框组合, 只是一个贪心过程.

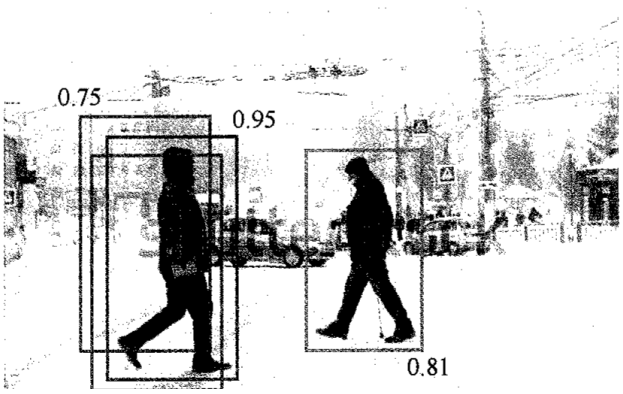


图 10.25: 同一对象在临近位置被多次检测及关联概率的示意图. 红色边界框对应于最高的总体概率. 非最大抑制消除了以蓝色显示的其他重合候选边界框, 同时保留了以绿色边界框显示的对同一对象类别的另一实例的检测结果.

10.4.6 快速区域卷积神经网络

扫描窗口方法将深度卷积网络的全部力量应用于图像的所有区域, 即使一些区域不太可能包含对象. 另一种方法是应用一些计算成本较低的技术, 例如分割算法, 来识别图像中存在对象可能性较高的区域, 然后仅对这些区域应用完整的卷积网络. 这种方法称为带有 CNN 的快速区域提议 (fast Region proposals with CNN, fast R-CNN)(Girshick, 2015). 也可以使用区域提议 (region proposal) 卷积网络来识别最有可能的区域, 这种方法称为 faster R-CNN(Ren et al., 2015), 该方法允许对区域提议网络和检测定位网络进行端到端训练.

滑动窗口方法的一个缺点是, 如果想要非常精确地定位对象, 则需要考虑大量的只有细微间隔的窗口位置, 这将使计算成本变得很高. 一种更高效的方法是将滑动窗口与直接边界框预测结合起来 (Sermanet et al., 2013). 在这种情况下, 可以连续输出预测边界框相对于窗口位置的位置, 以便对预测位置进行一些微调.

定义 10.20 (区域提议)

称一个低成本的预处理过程为**区域提议** (region proposal), 若它从图像中给出少量候选区域, 使得真实对象以高概率落在其中, 从而可以把昂贵的分类与定位网络只作用于这些区域. 若提议过程本身由一个卷积网络实现并与检测网络联合训练, 则称相应的检测框架为 **faster R-CNN**; 若提议由外部算法给出、只对区域做分类与框回归, 则称 **fast R-CNN**.

♣

注

(重构新增) 区域提议的要点是“用便宜的计算换掉昂贵的穷举”: 滑动窗口对所有位置一视同仁, 而区域提议先用颜色、边缘或浅层特征排除明显不含对象的区域. 定义区分 fast 与 faster 的关键在于提议是否可学习: 前者把提议当作固定输入, 检测网络无法通过提议的错误反向传播; 后者把提议网络也纳入训练, 两个网络共享特征, 因此可以端到端优化. 这不是命名上的细节, 而是训练目标是否一致的问题: 提议与检测若分开训练, 二者的召回率与定位精度可能相互错配.

10.5 图像分割

在图像分类问题中, 整个图像被分配了单一类别标签. 我们已经看到, 如果检测多个对象并使用边界框记录它们的位置, 则可以提供更详细的信息 (参见节 10.4). 语义分割 (semantic segmentation) 可以让我们获得更详细的分析, 其中图像的每个像素被分配给预设的类别之一. 这意味着输出空间的维度与输入图像的维度相同, 因此可以方便地表示为具有相同像素数量的图像. 尽管输入图像通常具有 R、G、B 三个通道, 但输出数组具有 C 个通道 (假设有 C 个类别) 以表示每个类别的概率. 如果我们将每个类别与不同的 (任意选择的) 颜色关联起来, 那么分割网络的预测可以表示为一幅图像, 其中每个像素根据具有最高概率的类别着色, 如图 10.26所示.

定义 10.21 (语义分割)

设图像有 $J \times K$ 个像素、预设类别集合的大小为 C. 若网络对每个像素 (j,k) 输出一个概率向量 $\mathbf{p}(j,k) \in [0,1]^C (\sum_{c=1}^{C} p_c(j,k) = 1)$, 并把该像素判给概率最大的类别, 则称这一任务为**语义分割** (semantic segmentation). 此时网络输出是一个 $J \times K \times C$ 的数组, 其空间尺寸与输入相同, 通道数为类别数. ♣

注

(重构新增) 语义分割与目标检测的根本差别在于输出空间的形状: 检测输出的是“少量框 + 类别”, 分割输出的是“与输入同尺寸的逐像素标签”. 因此分割网络的最后一层必须有 C 个通道而不是 C 个单元, 且空间尺寸必须恢复——这正是后文所有上采样技术 (定义 10.22) 存在的理由. 定义要求概率向量归一化, 是为了让“取最大”有明确含义; 若某像素的分布接近均匀, 则该像素的分类本质上是不确定的, 实际系统常用置信度阈值把这部分像素标为“忽略”. 注意定义是逐像素独立的, 因此它不区分同一类别的不同实例 (两只狗会得到同一个标签), 这是“语义”分割与“实例”分割的分界.

图 10.26: 图像及相应语义分割的示例, 其中每个像素根据具有最高概率的类别着色. 例如, 蓝色像素对应“汽车”类别, 红色像素对应“行人”类别, 橙色像素对应“交通信号灯”类别 [由 Wayve Technologies Ltd. 提供].

10.5.1 卷积分割

一种解决语义分割问题的简单方法是构建一个卷积网络, 该网络将图像上以某个像素为中心的矩形部分作为输入, 并借助单一 softmax 输出对该像素进行分类. 通过将这样的网络轮流应用于每个像素, 便可以分割整个图像 (这需要根据输入窗口的大小在图像周围进行边缘填充). 然而重叠部分会导致冗余计算, 使得这种方法十分低效 (参见图 10.21). 我们可以通过将不同输入位置的前向传播计算组合输入单一网络来消除这种低效性, 得到一个最终的全连接层也是卷积层的模型 (参见图 10.23). 因此, 我们可以构建一个在每层都使用步幅为 1、做了等大填充且没有池化的 CNN, 从而使得每一层的维度都与输入图像的维度相同. 每个输出单元都具有 softmax 激活函数, 权重在所有输出之间共享. 虽然可能有效, 但这样的网络仍然需要许多层, 且每层中有多个通道, 以学习实现高精度所需的复杂内部表示. 如果这样做, 即使对于常规分辨率的图像, 计算成本也是难以承受的.

10.5.2 上采样

大多数卷积网络使用多层下采样, 从而在通道数量增加的同时, 使特征图的大小减小, 以及在确保网络整体大小和计算成本可控的同时, 允许网络从图像中提取高阶语义特征. 我们可以利用这个概念来创建一个更高效的语义分割架构. 该架构采用标准的深度卷积网络, 并添加了一些额外的可学习层, 这些额外的可学习层用于将低维的内部表示转换回原始图像分辨率 (Long, Shelhamer, and Darrell, 2014; Noh, Hong, and Han, 2015; Badrinarayanan, Kendall, and Cipolla, 2015), 如图 10.27所示.

为了做到这一点, 我们需要一种方法来逆转跨步卷积和池化操作的下采样效果. 首先考虑类比于池化的上采样操作, 其中输出层的单元数比输入层的单元数多, 例如, 每个输入单元对应一个 2×2 的输出单元块. 接下来的问

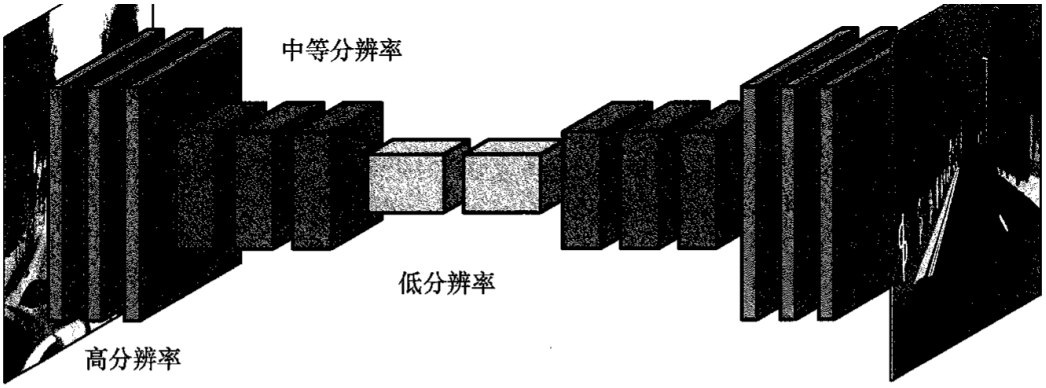


图 10.27: 用于图像语义分割的卷积神经网络的示意图. 这里首先通过一系列跨步卷积和/或池化操作降低特征图的维度, 紧接着通过一系列转置卷积和/或上采样操作, 将维度还原为原始图像的维度.

题是输出应该使用什么值. 为了找到类比于平均池化的上采样操作, 我们可以简单地将每个输入值复制到所有对应的输出单元中, 如图 10.28(a) 所示. 可以看到, 对这个操作的输出应用平均池化即可重新生成输入.

对于最大池化, 我们可以考虑图 10.28(b) 中的操作, 其中将每个输入值复制到相应输出块的第一个单元中, 并将每个输出块中的其余值置零. 同样, 我们看到对输出层应用最大池化操作会重新得到输入层. 这有时称为最大上采样 (max-unpooling). 将非零值分配给输出块的第一个单元似乎是任意的, 因此可以使用一种保留了更多来自下采样层的空间信息的改进方法 (Badrinarayanan, Kendall, and Cipolla, 2015). 这是通过设计一种网络架构来实现的, 该架构在每个最大池化下采样层之后都有一个相应的上采样层. 在下采样过程中, 记录每个输出块中的哪个单元具有最大值, 接着在相应的上采样层中, 将非零元素设置到相同的位置, 如图 10.29中的 2×2 最大池化所示.

定义 10.22 (上采样与最大上采样)

称把每个输入单元映射为一个 $R\times R$ 输出单元块、从而使输出单元数变为输入 R^2 倍的运算为**上采样** (up-sampling). 若每个输入值被复制到对应输出块的全部单元, 则称**平均上采样** (平均池化的类比); 若每个输入值只写入对应输出块的一个单元、其余置零, 则称**最大上采样** (max-unpooling). 若最大上采样写入的位置由对应下采样层记录的最大值位置决定, 则称其为**带位置记录的最大上采样**. ♣

注

(重构新增) 上采样是下采样的“伴随”而不是“逆”: 一般情形下它丢失了被池化丢弃的信息, 只是把少量数值铺回原来的网格. 定义中 $R\times R$ 的块结构对应池化窗口, 因此上采样的输出尺寸恰好是池化输入的尺寸, 这使二者可以逐层配对. 平均上采样与最大上采样的差别只在写入模式: 前者把所有位置填满, 后者只填一个位置, 于是输出是稀疏的. 稀疏性本身不是缺陷, 但“填哪个位置”携带信息——这正是位置记录版上采样存在的原因, 它把池化时丢掉的空间位置从下采样层搬到上采样层, 代价是需要额外保存索引.

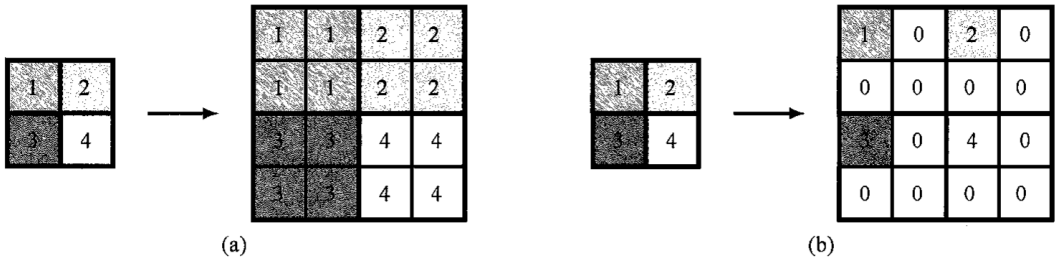


图 10.28: 上采样操作的示意图:(a) 类比于平均池化的上采样操作;(b) 类比于最大池化的上采样操作.

图 10.29: 左边显示的最大池化层的一些空间信息可以通过如下方式保留: 首先记录输入数组中每个 2×2 块最大值的位置, 然后在相应的上采样层中将非零元素放置在输出数组中相同的位置.

命题 10.12 (上采样与池化的复合)

设池化窗口与上采样块均为 $R\times R$、步幅均为 R(互不重叠), 输入特征图的尺寸是 R 的整数倍.

1. 若先做平均上采样再做平均池化, 则复合运算为恒等映射;
2. 若先做最大上采样再做最大池化, 且各输出块恰有一个非零元, 则复合运算为恒等映射.

♠

证明

(1) 设输入单元值为 v, 它被复制到对应输出块的 R^2 个单元. 对该块做平均池化得 $\dfrac{1}{R^2}\sum_{t=1}^{R^2} v = v$, 即恢复原值. 由于块与块互不重叠且覆盖整个输出, 逐块结论即整体结论.

(2) 设输入单元值为 v, 它被写入对应输出块的某一个单元, 其余单元为零. 对该块取最大值: 若 $v\geqslant 0$, 最大值为 v; 若 $v<0$, 由于其余为零, 最大值为 0 而不是 v. 因此要得到恒等映射, 需 $v\geqslant 0$(例如激活值经 ReLU 之后). 在此条件下逐块取最大即得 v, 故复合为恒等映射. 证毕.

□

注

(重构新增) 这个命题说明了上采样与池化“配对”的确切含义: 它们互为逆运算是有条件的. 第 (2) 条中的非负性条件常被忽略: 若特征图允许负值, 最大上采样后块内最大值是零而不是原来那个负数, 信息已经丢失, 复合不再恒等. 这也解释了为什么最大上采样通常用在 ReLU 之后. 两条结论都依赖“窗口不重叠、步幅等于窗口”的约定; 若步幅小于窗口, 块与块重叠, 同一个输入单元会参与多个输出块, 复合结果不再逐块独立. 命题只讨论形状匹配的一对一情形, 实际网络中上采样层与池化层之间往往还隔着若干卷积层, 此时复合当然不再是恒等映射.

前面讨论的上采样操作都是固定的函数, 类似于平均池化和最大池化的下采样操作. 我们也可以使用一种称为学习到的 (learned) 上采样操作, 类比于用于下采样的跨步卷积. 在跨步卷积中, 输出图上的每个单元通过共享可学习权重与输入图上的一小块相连. 当我们在输出数组中移动一步时, 滤波器在输入数组上将移动两步或更多步, 因此输出数组的维度小于输入数组的维度. 对于上采样, 我们使用一个滤波器将输入数组中的一个像素连接到输出数组中的一小块. 然后选择架构, 使得当在输入数组中移动一步时, 在输出数组中移动两步或更多步 (Dumoulin and Visin, 2016).图 10.30说明了应用 3×3 滤波器和输出步幅为 2 的情况. 注意, 存在多个滤波器位置重合的输出单元, 在这种情况下, 相应的输出值可以通过对来自各个滤波器位置的贡献进行求和或求平均来计算. 这种上采样称为转置卷积 (transpose convolution), 因为如果将下采样卷积表示为矩阵形式, 则相应的上采样卷积可以由转置矩阵给出 (见练习 10.13). 注意, 有时也称这种上采样为“反卷积”, 但最好避免使用这个术语, 因为在数学中, “反卷积”通常表示泛函分析中卷积的逆操作, 概念完全不同. 转置卷积又称“分数跨步卷积”, 因为标准卷积的步幅是输出层步长与输入层步长的比值. 例如, 在图 10.30中, 这个比值是 0.5.

定义 10.23 (转置卷积)

设某下采样 (跨步) 卷积层可写成矩阵形式 $\mathbf{y}=\mathbf{A}\mathbf{x}$, 其中 $\mathbf{x}$ 与 $\mathbf{y}$ 分别是输入、输出按通道展开成的向量. 对这个层的**转置卷积** (transpose convolution) 是用同一组可学习权重、以转置矩阵作用在 $\mathbf{y}$ 上:

$$\mathbf{x}' = \mathbf{A}^{\top}\mathbf{y}. \tag{10.25}$$

等价地, 转置卷积把输入数组中的每个元素通过一个滤波器连接到输出数组中的一小块, 并使输出数组的

相对步长是输入数组的 $1/S$.

注

(重构新增) 转置卷积之所以叫“转置”, 是因为(10.25)中的矩阵确实是下采样卷积矩阵的转置, 而不是说它计算了卷积的逆. 两者的差别是本质的:$\mathbf{A}$ 通常是“高瘦”的 (输出维数小于输入维数), 它不可逆,$\mathbf{A}^{\mathsf{T}}$ 只是把梯度方向上的伴随映射作用于 $\mathbf{y}$, 因此转置卷积的输出尺寸虽与下采样输入相同, 数值却不是下采样的逆. 这正是文本强调不应称其为“反卷积”的原因: 数学中的反卷积要求先有卷积算子并求其逆, 而这里根本没有可逆性. 定义中的“等价地”一句给出了实现视角: 转置卷积可以看作用同一个滤波器在放大后的网格上做普通卷积, 重叠位置的贡献相加, 这解释了为什么它的输出会有棋盘状伪影.

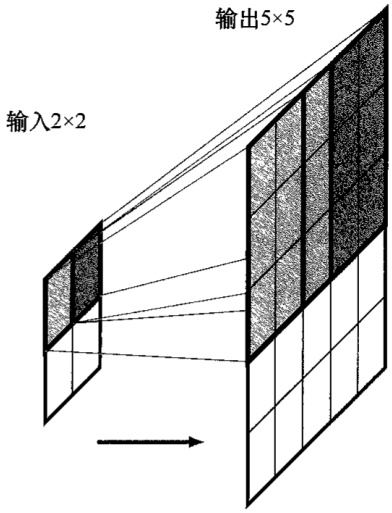


图 10.30: 带有 3×3 滤波器、输出步幅为 2 的转置卷积示意图. 这可以看作 3×3 卷积的逆运算. 红色输出小块是通过将卷积核乘以输入层中红色单元的激活得到的, 类似地, 对于蓝色输出小块也是如此. 小块重合单元的激活是通过对单个小块的贡献进行求和或求平均来计算的.

命题 10.13 (转置卷积是下采样卷积的伴随算子)

设下采样卷积层为线性映射 $\mathbf{A}:\mathbb{R}^n\to\mathbb{R}^m$,$\mathbf{y}=\mathbf{A}\mathbf{x}$. 则对任意 $\mathbf{x}\in\mathbb{R}^n$、$\mathbf{z}\in\mathbb{R}^m$ 成立

$$\langle\mathbf{A}\mathbf{x},\mathbf{z}\rangle=\langle\mathbf{x},\mathbf{A}^{\mathsf{T}}\mathbf{z}\rangle, \tag{10.26}$$

即 $\mathbf{A}^{\mathsf{T}}$ 是 $\mathbf{A}$ 关于标准内积的伴随算子; 并且当 $\mathbf{A}$ 为满秩方阵时 $\mathbf{A}^{\mathsf{T}}$ 与 $\mathbf{A}^{-1}$ 一般不同.

证明

按标准内积展开:$\langle\mathbf{A}\mathbf{x},\mathbf{z}\rangle=(\mathbf{A}\mathbf{x})^{\mathsf{T}}\mathbf{z}=\mathbf{x}^{\mathsf{T}}\mathbf{A}^{\mathsf{T}}\mathbf{z}=\langle\mathbf{x},\mathbf{A}^{\mathsf{T}}\mathbf{z}\rangle$, 即(10.26). 若 $\mathbf{A}$ 为满秩方阵, 则 $\mathbf{A}^{\mathsf{T}}=\mathbf{A}^{-1}$ 当且仅当 $\mathbf{A}^{\mathsf{T}}\mathbf{A}=\mathbf{I}$, 即 $\mathbf{A}$ 为正交阵; 取 $\mathbf{A}=2\mathbf{I}$ 得 $\mathbf{A}^{\mathsf{T}}=2\mathbf{I}\neq\frac{1}{2}\mathbf{I}=\mathbf{A}^{-1}$, 故一般不同. 证毕.

□

注

(重构新增) 这个命题给出转置卷积的严格身份: 它是伴随, 而不是逆. 伴随的意义在反向传播中才显现——卷积层反向传播时, 输入梯度正是用同一个矩阵的转置作用在输出梯度上, 因此“转置卷积”这个名字来自它在前向中的形状与反向中的算子一致. 命题第二句提醒: 即便 $\mathbf{A}$ 可逆,$\mathbf{A}^{\mathsf{T}}$ 也只在 $\mathbf{A}$ 为正交阵时才等于 $\mathbf{A}^{-1}$, 因此“转置卷积恢复了下采样的信息”这一说法只在极特殊的情形成立. 条件“标准内积”不能省: 若换用加权内积, 伴随算子就要相应改写, 转置矩阵不再正确.

10.5.3 全卷积网络

如果我们的网络架构没有池化层, 即下采样和上采样纯粹使用卷积来完成, 则称这样的网络为全卷积网络 (fully convolutional network)(Long, Shelhamer, and Darrell, 2014). 全卷积网络可以接收任意大小的图像, 并输出相同大小的分割图. 这可以看作 3×3 卷积的逆运算.

定义 10.24 (全卷积网络)

若一个网络的所有层都是卷积层与转置卷积层 (不含池化层与全连接层), 从而下采样与上采样完全由卷积完成, 则称其为**全卷积网络** (fully convolutional network). 全卷积网络对任意空间尺寸的输入都有定义, 并输出与输入同空间尺寸的分割图. ♣

注

(重构新增) 全卷积网络的关键性质是“对任意输入尺寸都有定义”: 卷积核的参数量与输入尺寸无关 (命题 10.5), 因此在比训练图像更大的输入上, 网络只是输出更大的特征图, 而不需要改变任何参数. 若网络中夹有全连接层, 这一点立刻失效: 全连接层的权重矩阵维数被输入尺寸固定, 换尺寸就无法计算. 定义排除池化层, 是为了让“尺寸只由步幅决定”这一性质保持可控; 实际上更常见的做法是保留池化但配合上采样, 这就是定义 10.25的思路. 最后一句“可看作 3×3 卷积的逆运算”只是形状层面的类比, 不应按命题 10.13的严格意义理解.

10.5.4 U-Net 架构

我们已经看到, 与跨步卷积和池化相关的下采样允许增加通道的数量, 而不会使网络的规模变得过大. 但这也会降低空间分辨率, 导致信号流过网络时丢失位置信息. 虽然这对于图像分类来说是可以接受的, 但对于图像语义分割来说, 丢失空间信息是一个大问题, 因为我们想要分类每个像素. 解决这个问题的一种方法是使用 U-Net 架构 (Ronneberger, Fischer, and Brox, 2015), 如图 10.31所示, 其名称来自示意图所示的 U 形结构. 其核心概念是, 对于每个下采样层, 都有一个对应的上采样层, 并且每个下采样层的输出被拼接到相应的上采样层, 从而使这些上采样层能够获得更高分辨率的空间信息. 注意, 可以在 U-Net 架构的最后一层使用 1×1 卷积将通道数量减少至类别数量, 然后接一个 softmax 激活函数.

定义 10.25 (U-Net)

称满足下列结构的语义分割网络为 **U-Net**: 网络由一条下采样路径与一条上采样路径组成; 对下采样路径中的每一层, 上采样路径中存在一层与之空间尺寸相同、并通过**拼接** (concatenation) 接收其输出的对应层; 上采样路径逐层恢复空间分辨率, 最后以 1×1 卷积把通道数降到类别数 C, 再接 softmax 激活函数. ♣

注

(重构新增) U-Net 的定义把“对称”与“拼接”两件事都写进了对象. 对称保证每个上采样阶段都有匹配尺寸的特征图可供拼接; 拼接 (而不是相加) 则保留了浅层的高分辨率细节与深层的语义信息两条独立通道, 让后续卷积自己学习如何融合——若改为相加, 两路信息在进入网络前就被强行对齐, 浅层的边缘信息容易被深层的语义响应淹没. 定义中最后的 1×1 卷积不是装饰: 它把任意多的通道数压到类别数, 使输出可以直接与逐像素标签比较 (参见定义 10.9). 注意 U-Net 不要求下采样只用池化, 也可以用跨步卷积, 只要尺寸对应关系成立.

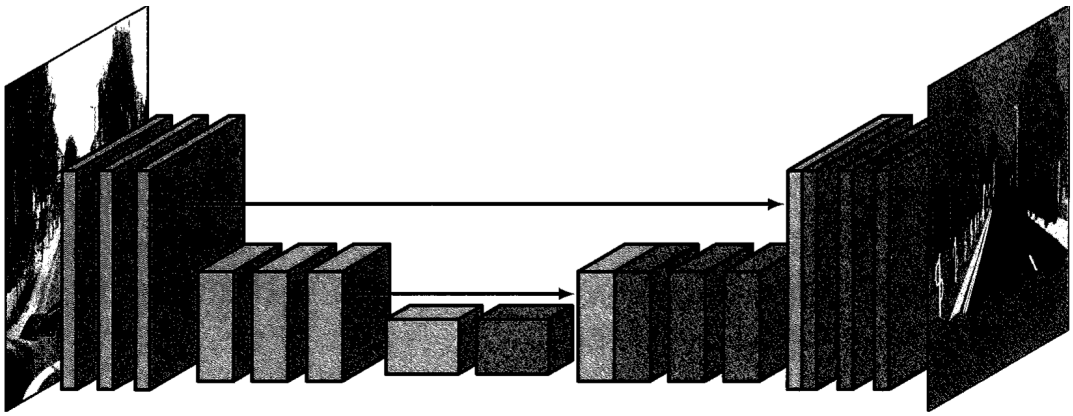


图 10.31: U-Net 架构具有下采样层和上采样层的对称排列结构, 每个下采样层的输出被拼接到相应的上采样层.

10.6 风格迁移

深度卷积网络中的早期层学习检测边缘和纹理等简单特征, 而后续层则学习检测更复杂的对象等实体 (参见节 10.3). 我们可以利用这一性质, 使用一种称为神经风格迁移 (neural style transfer) 的过程来以另一种图像的风格重新渲染图像 (Gatys, Ecker, and Bethge, 2015), 如图 10.32所示.

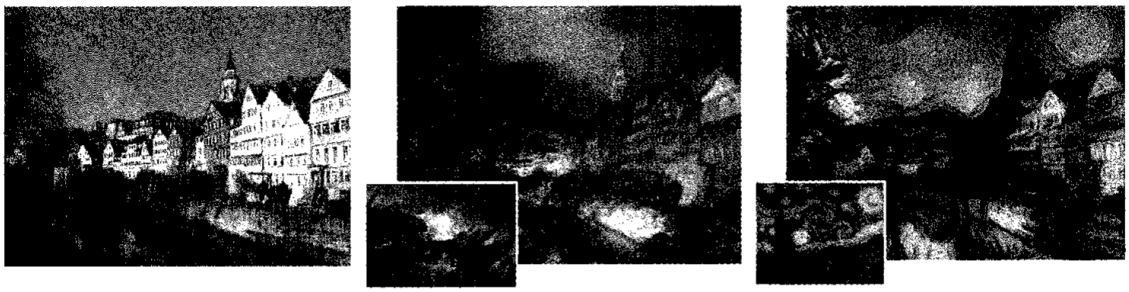


图 10.32: 一个神经风格迁移的示例, 展示了一张运河场景的照片 (左) 以 J. M. W. Turner 的《运输船遇难》(中) 风格和 Vincent van Gogh 的《星月夜》(右) 风格渲染后的效果, 用于提供样式的图像都显示在小图中 [经 Gatys, Ecker and Bethge (2015) 授权使用] .

我们的目标是生成合成图像 G, 其“内容”由图像 C 定义, 其“风格”来自另一幅图像 S. 这可以通过定义由以下两项的和组成的误差函数 $E(G)$ 来实现, 其中一项促使 G 具有与 C 相似的内容, 另一项则促使 G 具有与 S 相似的风格:

$$E(G) = E_{\text{content}}(G, C) + E_{\text{style}}(G, S). \tag{10.27}$$

图像内容和风格的概念由这两项的函数形式隐含定义. 然后, 我们可以通过从随机初始化的图像开始, 并使用梯度下降最小化 $E(G)$ 来得到 G.

定义 10.26 (神经风格迁移)

设 C 为内容图像、S 为风格图像,E_{content} 与 E_{style} 为给定的内容误差与风格误差,E 由(10.27)定义. 称从随机初始化的图像出发、用梯度下降最小化 $E(G)$ 而得到的图像 G 为以 C 为内容、S 为风格的**神经风格迁移**结果, 称这一过程为神经风格迁移 (neural style transfer).

注

(重构新增) 这个定义的关键在于它并不定义“内容”与“风格”这两个词本身, 而是通过两个误差项的函数形式隐含地定义它们: 凡是被 E_{content} 约束的, 就是内容; 凡是被 E_{style} 约束的, 就是风格. 因此换一种误差项的函数形式, 就等于换了一种对“风格”的理解, 这不是实现细节而是模型定义的一部分. 定义要求从随机图像出发并使用梯度下降, 说明 G 是优化变量而不是网络参数: 网络本身是固定的特征提取器 (通常是预训练的 VGG), 被优化的是像素. 这个选择带来一个实践上的后果: 每生成一幅图都要重新求解一次优化问题, 速度远慢于训练一个前馈生成网络.

为了定义 $E_{\text{content}}(G, C)$, 我们可以选择网络中特定的卷积层, 并将图像 G 用作输入, 计算该层中单元的激活, 对图像 C 也执行相同的操作. 然后我们可以促使相应的预激活值变得相似, 方法是使用如下形式的平方和误差函数:

$$E_{\text{content}}(G, C) = \sum_{i,j,k} \big(a_{ijk}(G) - a_{ijk}(C)\big)^2, \tag{10.28}$$

其中 $a_{ijk}(G)$ 表示当输入图像为 G 时, 该层中通道 k 位置 (i, j) 处单元的预激活值; 类似地,$a_{ijk}(C)$ 表示当输入图像为 C 时, 该层中通道 k 位置 (i, j) 处单元的预激活值. 选择哪层用于定义预激活会影响最终结果, 早期层旨在匹配边缘等低级特征, 后续层则旨在匹配更复杂的结构甚至整个对象.

在定义 $E_{\text{style}}(G, S)$ 时, 直觉上图像风格是由卷积层中不同通道的特征共现来确定的. 例如, 如果风格图像 S 中的垂直边缘通常与橙色斑块相关联, 那么我们希望生成的图像 G 也是如此. 然而, 尽管 $E_{\text{content}}(G, C)$ 尝试匹配 G 的特征与 C 中相同位置的相应特征, 但对于风格误差 $E_{\text{style}}(G, S)$, 我们希望 G 具有与 S 相匹配的特征, 因此我们选择对特征图中的位置取平均. 与之前一样, 考虑一个特定的卷积层. 我们可以通过构建互相关矩阵来衡量输入图像 G 的通道 k 中的特征与通道 k' 中相应特征的共现程度:

$$F_{kk'}(G) = \sum_{i=1}^{I} \sum_{j=1}^{J} a_{ijk}(G)\, a_{ijk'}(G), \tag{10.29}$$

其中,I 和 J 是这个特定卷积层中特征图的维度, 乘积 $a_{ijk}a_{ijk'}$ 在两个特征均激活时值较大. 如果该层有 K 个通道, 则 $F_{kk'}$ 形成一个 $K \times K$ 的矩阵, 称为风格矩阵 (style matrix). 我们可以通过使用下式比较它们的风格矩阵来衡量图像 G 和 S 具有相同风格的程度:

$$E_{\text{style}}(G, S) = \frac{1}{(2IJK)^2} \sum_{k=1}^{K} \sum_{k'=1}^{K} \big(F_{kk'}(G) - F_{kk'}(S)\big)^2. \tag{10.30}$$

虽然也可以使用单层, 但通过使用以下形式多层的贡献可以获得更令人满意的结果:

$$E_{\text{style}}(G, S) = \sum_{l} \lambda_l E^{(l)}_{\text{style}}(G, S), \tag{10.31}$$

其中,l 表示卷积层. 系数 λ_l 不仅确定了不同层之间的相对权重, 也确定了相对于内容误差项的权重. 这些权重系数是由主观判断经验性地调整的.

定义 10.27 (风格矩阵 (Gram 矩阵))

设某卷积层有 K 个通道、每个通道的空间尺寸为 $I \times J$,a_{ijk} 为通道 k 在位置 (i, j) 的预激活. 称由(10.29)定义的 $K \times K$ 矩阵 $F(G) = (F_{kk'}(G))$ 为输入图像 G 在该层的**风格矩阵** (style matrix), 亦称 **Gram 矩阵**. ♣

注

(重构新增) 风格矩阵把“某层特征图”压缩成一个 $K \times K$ 的通道间相关矩阵, 其代价是彻底丢弃空间信息:$F_{kk'}$ 对全部位置求和, 因此两幅图只要通道间的共现统计相同, 风格矩阵就相同, 哪怕图案排布完全不同. 这正是“风格”与“内容”的分界: 内容误差(10.28)逐位置比较, 风格误差(10.30)只比较统计量. 定义中“对位置求和”而不是取平均, 使 F 的数值随特征图尺寸增大而增大, 故(10.30)前的归一化因子 $(2IJK)^2$ 不是装饰: 没有它, 不同层的误差量级会随分辨率剧烈变化,λ_l 就无法跨层比较.

命题 10.14 (Gram 矩阵的基本性质)

设 $A \in \mathbb{R}^{IJ\times K}$ 的列是各通道展平后的向量, 则 $F = A^{\mathsf{T}}A$. 于是

1. F 对称:$F^{\mathsf{T}} = F$;
2. F 半正定: 对任意 $\mathbf{v} \in \mathbb{R}^K$ 有 $\mathbf{v}^{\mathsf{T}}F\mathbf{v} \geqslant 0$, 且等号成立当且仅当 $A\mathbf{v} = \mathbf{0}$;
3. $\operatorname{rank}(F) = \operatorname{rank}(A) \leqslant \min\{IJ, K\}$.

♠

证明

把 $F_{kk'} = \sum_{i,j} a_{ijk}a_{ijk'}$ 写成向量内积, 正是 A 的第 k 列与第 k' 列的内积, 故 $F = A^{\mathsf{T}}A$.(1) $F^{\mathsf{T}} = (A^{\mathsf{T}}A)^{\mathsf{T}} = A^{\mathsf{T}}A = F$.(2) $\mathbf{v}^{\mathsf{T}}F\mathbf{v} = \mathbf{v}^{\mathsf{T}}A^{\mathsf{T}}A\mathbf{v} = \|A\mathbf{v}\|^2 \geqslant 0$, 而 $\|A\mathbf{v}\|^2 = 0$ 当且仅当 $A\mathbf{v} = \mathbf{0}$.(3) 由 $\operatorname{rank}(A^{\mathsf{T}}A) = \operatorname{rank}(A)$ 及 $\operatorname{rank}(A) \leqslant \min\{IJ, K\}$ 即得. 证毕.

□

注

(重构新增) 这三个性质各有用途. 对称性使 F 只有 $K(K+1)/2$ 个独立元素, 风格误差实际比较的自由度比 K^2 少. 半正定性保证了 F 可以看作某个特征分布的协方差矩阵, 因此“风格距离”在统计意义上是有依据的; 但要注意 F 通常奇异 (当 $IJ > K$ 时秩至多为 K, 更常见的是通道间高度相关), 所以不能直接求逆, 任何用到 F^{-1} 的写法都需要正则化. 秩的上界说明风格矩阵能表达的相关性受通道数限制: 通道太少时, 丰富的纹理统计无法被完整编码. 第 (2) 条的等号条件在数值上意味着: 若某线性组合 $A\mathbf{v}$ 在全部位置上为零, 则 F 在该方向上退化, 这通常出现在某通道恒为零 (例如 ReLU 之后整片失活) 的情形.

10.7 习题

✍ **练习 10.1**★ 考虑一个固定的权重向量 $\mathbf{w}$, 证明使得标量积 $\mathbf{w}^{\mathsf{T}}\mathbf{x}$ 最大化的输入向量 $\mathbf{x}$, 在约束 $\|\mathbf{x}\|^2$ 为常数时, 满足 $\mathbf{x} = \alpha\mathbf{w}$, 其中 α 为某个标量. 这通过使用 Lagrange 乘子法最容易完成 (参见附录 C).

✍ **练习 10.2**★★ 考虑一个具有一维输入数组和一维特征图的卷积网络层. 如图 10.2所示, 其中输入数组的维度为 5, 滤波器宽度为 3, 步幅为 1. 通过写出权重矩阵, 证明该层可以表示为全连接层的一个特例, 其中缺失的连接用 0 替换, 共享参数用重复元素表示, 忽略任何偏置参数.

✍ **练习 10.3**★ 显式计算在以下 4×4 输入矩阵上应用 2×2 滤波器的卷积输出:

$$\begin{bmatrix} 2 & 5 & 3 & 0 \\ 0 & 6 & 0 & -4 \\ -1 & -3 & 0 & 2 \\ 5 & 0 & 0 & 3 \end{bmatrix} * \begin{bmatrix} -2 & 0 \\ 4 & 6 \end{bmatrix} = \begin{bmatrix} ? & ? & ? \\ ? & ? & ? \\ ? & ? & ? \end{bmatrix}. \tag{10.32}$$

✍ **练习 10.4**★★ 如果图像 I 有 $J\times K$ 个像素, 而滤波器 K 有 $L\times M$ 个元素, 写出(10.2)中两个求和的极限. 在数学文献中,(10.2)所示的操作称为互相关, 卷积则由下式定义:

$$C(j,k) = \sum_l \sum_m I(j-l,\, k-m)K(l,m). \tag{10.33}$$

写出(10.33)中求和的极限. 证明(10.33)可以写成如下等价的“翻转”形式:

$$C(j,k) = \sum_l \sum_m I(j+l,\, k+m)K(L-1-l,\, M-1-m), \tag{10.34}$$

并同样写出(10.34)中求和的极限.

✍ **练习 10.5**★ 在数学中, 对连续变量 x 进行卷积的定义如下:

$$F(x) = \int_{-\infty}^{\infty} G(y)\, k(x-y)\, \mathrm{d}y, \tag{10.35}$$

其中 $k(x-y)$ 是核函数. 通过对该积分进行离散近似, 解释由(10.33)定义的 CNN 中卷积层与它的关系.

✍ **练习 10.6**★ 考虑一个大小为 $J\times K$ 的图像, 在所有边上都填充额外的 P 个像素, 然后使用大小为 $M\times M$ 的卷积核

进行卷积, 其中 M 是一个奇数. 证明如果我们选择 $P = (M-1)/2$, 则得到的特征图大小为 $J \times K$, 即特征图与原始图像大小相同.

✍ **练习 10.7**⋆ 证明如果对大小为 $M \times M$ 的卷积核与大小为 $J \times K$ 的图像进行卷积, 且填充深度为 P, 步幅为 S, 则得到的特征图维度由(10.7)给出.

✍ **练习 10.8**⋆⋆ 对于图 10.10中显示的 VGG-16 模型中的每一层, 计算包括偏置在内的权重数量 (即连接数) 和独立可学习参数的数量. 证实网络中可学习参数的总数约为 1.38 亿.

✍ **练习 10.9**⋆⋆ 考虑形为(10.2)的卷积, 并假设卷积核是可分离的, 则对于某些函数 $F(\cdot)$ 和 $G(\cdot)$, 有

$$K(l,m) = F(l)\,G(m). \tag{10.36}$$

证明可以不采用单个二维卷积, 而是使用两个一维卷积计算得到相同的结果, 从而显著提高效率.

✍ **练习 10.10**⋆ DeepDream 的更新过程涉及将用于反向传播的损失变量 δ 设置为所选层中节点的预激活值, 然后将损失反向传播到输入层以得到图像像素的梯度向量. 证明这可以看作函数式(10.21)关于图像 I 的像素进行梯度优化的过程.

✍ **练习 10.11**⋆⋆ 当设计一个神经网络来检测图像中 C 个不同类别的对象时, 我们可以使用一个 "$C+1$ 中之一" 形式的类别标签, 即每个对象类别使用一个变量, 另一个变量表示 "背景" 类别 (不包含任何定义对象类别的输入图像区域). 网络将输出一个长度为 $(C+1)$ 的概率向量. 也可以使用单个二元变量来表示对象存在与否, 然后使用一个单独的 "C 中之一" 向量来表示对象类别. 在这种情况下, 网络将输出单个表示对象存在可能性的概率和一组独立的类别标签概率. 写出这两组概率之间的关系.

✍ **练习 10.12**⋆⋆ 计算图 10.22中的卷积网络进行一次前向传递所需的计算步骤数量, 忽略偏置以及激活函数的计算. 同样, 计算通过图 10.23中的扩展网络进行一次前向传递所需的计算步骤数量. 最后, 计算在 8×8 图像上重复应用图 10.22中的卷积网络 9 次与应用图 10.23中的扩展网络 1 次所需的计算步骤数量之比. 这个比值隐含了使用卷积实现滑动窗口技术所带来的效率提升.

✍ **练习 10.13**⋆⋆ 使用一维向量来说明为什么有时候上采样卷积称为转置卷积. 考虑一维跨步卷积层, 输入具有 4 个单元, 激活值为向量 (x_1, x_2, x_3, x_4), 用 0 填充后得到 $(0, x_1, x_2, x_3, x_4, 0)$, 滤波器的值为 (w_1, w_2, w_3). 假设步幅为 2, 写出输出层的一维激活向量. 将这个输出用矩阵 $\mathbf{A}$ 乘以向量 $(0, x_1, x_2, x_3, x_4, 0)$ 的形式表示. 考虑上采样卷积, 其中输入层具有激活值 (z_1, z_2), 滤波器的值为 (w_1, w_2, w_3), 输出步幅为 2. 假设重叠的滤波器值被求和, 并且激活函数仅是恒等函数, 写出 6 维输出向量. 证明这可以表示为使用转置矩阵 $\mathbf{A}^{\mathsf{T}}$ 的矩阵乘法.

第 11 章 结构化分布

我们已经看到, 概率是深度学习最重要的基础概念之一. 例如, 用于二元分类的神经网络可以用一个条件概率分布来描述, 其形式为

$$p(t \mid \mathbf{x}, \mathbf{w}) = y(\mathbf{x}, \mathbf{w})^t \left(1 - y(\mathbf{x}, \mathbf{w})\right)^{1-t}, \tag{11.1}$$

其中,$y(\mathbf{x}, \mathbf{w})$ 表示神经网络函数, 它以向量 $\mathbf{x}$ 作为输入, 并受可学习参数向量 $\mathbf{w}$ 的控制. 相应的交叉熵似然是构建用于训练神经网络的误差函数的基础. 虽然神经网络函数可能极其复杂, 但(11.1)中的条件分布形式却很简单. 有许多重要的深度学习模型都具有更丰富的概率结构, 如大语言模型、归一化流、变分自编码器、扩散模型等. 为了描述和利用这种结构, 我们引入了一个强大的框架, 称为概率图模型 (probabilistic graphical model) 或简称图模型 (graphical model), 它允许以图的形式表达结构化概率分布. 当与神经网络相结合来定义相关的概率分布时, 图模型为创建可以端到端训练的复杂模型提供了巨大的灵活性, 其中梯度可通过自动微分来进行有效计算, 并使用随机梯度下降进行训练. 本章将重点讨论应用深度学习所需的概率图模型的核心概念, 而对机器学习的概率图模型更为全面的讨论可参阅 Bishop (2006).

11.1 概率图模型

概率可以用两个简单的法则来表示, 即加和法则和乘积法则 (参见节 2.1). 本书讨论的所有概率操作, 无论多么复杂, 都相当于重复应用这两个法则. 因此, 原则上我们可以纯粹通过代数操作来制定和使用复杂的概率模型. 不过, 我们发现使用概率分布的图示法来扩充分析是有好处的, 因为图示法具有以下 3 个有用的特性:

(1) 它们提供了一种可视化概率模型结构的简单方法, 可用于设计和构思新的模型;
(2) 通过对图进行检查, 可以深入了解模型的特性, 包括条件独立性;
(3) 在复杂模型中进行推理和学习所需的复杂计算可以用图形操作来表示, 如消息传递, 其中隐式地进行了基本的数学运算.

虽然这种概率图模型的节点和边与神经网络图很相似, 但它们具有特别的概率性解释, 并含有更丰富的语义. 为了避免混淆, 本书用蓝色表示神经网络图, 用红色表示概率图模型 (后文简称图模型).

11.1.1 有向图

图由节点 (也称顶点) 构成, 这些节点由边来连接. 在图模型中, 每个节点代表一个随机变量, 而连接则表示这些变量之间的概率关系. 然后, 图就可以记录所有随机变量的联合分布是如何分解成多个因子 (每个因子都仅依赖变量的一个子集) 的乘积的. 本章将重点讨论图模型, 在图模型中, 图的连接具有箭头指示的特定方向.

定义 11.1 (有向图模型)

设 $G = (\mathcal{V}, \mathcal{E})$ 是一个有向图, 其中节点集 $\mathcal{V} = \{1, \cdots, K\}$ 中的每个节点 k 对应一个随机变量 x_k, 边集 $\mathcal{E}$ 由有序对 (j, k) 组成. 若 $(j, k) \in \mathcal{E}$, 则称节点 j 是节点 k 的**父节点**, 节点 k 是节点 j 的**子节点**, 并记 $\mathrm{pa}(k) = \{j : (j, k) \in \mathcal{E}\}$ 为节点 k 的父节点集. 若一个联合分布可以写成每个节点的条件分布的乘积, 其中每个条件分布以该节点的父节点所对应的变量为条件, 则称该分布由 G 表示, 并称 G 为**有向图模型** (directed graphical model), 也称 **Bayes 网络**.

♣

注

(重构新增) 定义 11.1给出了全章的核心对象: 一个有向图加上每个节点的一个条件分布. 定义中的求和约定是“节点即变量”, 即不区分节点与它所对应的随机变量, 日后写 $p(x_k)$ 与写“节点 k 的边缘分布”是同一件事. 定义没有限制变量的类型 (离散、连续、向量值均可), 这正是图模型的适用范围很广的原因; 它要求的只是“存在一种按父节点组织的条件分解”.(11.8)中的条件分布 $p(x_k \mid \mathrm{pa}(k))$ 称为局部因子, 它们是把图翻译成代数表达式的唯一载体.

图模型的另一大类是 Markov 随机场 (Markov random field), 也称无向图模型 (undirected graphical model), 其中的连接不带箭头, 无方向意义. 有向图适用于表达随机变量之间的因果关系, 而无向图更适合表达随机变量之间的软约束. 有向图和无向图都可以看作一种叫作因子图 (factor graph) 表示法的特例.

定义 11.2 (无向图模型)

设 $G = (\mathcal{V}, \mathcal{E})$ 是一个无向图, 其中每条边 $\{j,k\} \in \mathcal{E}$ 没有方向. 若一个联合分布可以写成定义在图的团 (clique) 上的非负函数 ψ_C 的乘积, 即

$$p(\mathbf{x}) = \frac{1}{Z}\prod_{C\in C}\psi_C(\mathbf{x}_C), \qquad Z = \int \prod_{C\in C}\psi_C(\mathbf{x}_C)\,\mathrm{d}\mathbf{x}, \tag{11.2}$$

其中 C 是图中所有极大团的集合,Z 是归一化常数 (配分函数), 则称 G 为**无向图模型** (undirected graphical model) 或 **Markov 随机场** (Markov random field).

注

(重构新增) 无向图模型与有向图模型的差别不只是“箭头的有无”. 有向分解(11.8)自动归一化 (每个局部因子都是条件分布), 而无向分解(11.2)中的因子 ψ_C 只是非负函数, 必须额外除以配分函数 Z 才能成为概率分布.Z 一般没有解析形式, 需要对所有变量积分或求和, 这在小节 11.1.3的参数计数与第 14 章的采样方法中都会成为核心困难. 另一方面, 无向图能表达“软约束”:ψ_C 大表示该团上的取值组合更受偏好, 而不必指明谁是“因”、谁是“果”.

定义 11.3 (因子图)

因子图 (factor graph) 是一个二分图, 它有两类节点: 变量节点 (对应随机变量 x_k) 与因子节点 (对应联合分布分解式中的每个因子 f_s). 若因子 f_s 依赖于变量 x_k, 则在因子节点 s 与变量节点 k 之间连一条无向边. 因子图表示的联合分布为

$$p(\mathbf{x}) = \frac{1}{Z}\prod_{s} f_s(\mathbf{x}_{\mathcal{N}(s)}), \tag{11.3}$$

其中 $\mathcal{N}(s)$ 是因子节点 s 的邻接变量节点集.

注

(重构新增) 因子图的价值在于它把“图的形状”和“因子的划分”彻底解耦: 有向图和无向图都能无损地转写成因子图, 而反过来不一定. 例如有向图中的父子关系 $p(x_k \mid \mathrm{pa}(k))$ 在因子图里就是一个以 x_k 与 $\mathrm{pa}(k)$ 为邻接的因子节点, 父节点与子节点的地位因此变得对称. 这种对称性正是信念传播等消息传递算法的前提: 算法只需在变量节点与因子节点之间交替传递消息, 不必关心原图是有向还是无向.定义 11.1与定义 11.2中“因子”一词的用法在因子图框架下得到统一.

接下来, 我们将重点关注有向图. 不过, 请注意, 在我们讨论图神经网络 (Graph Neural Network, GNN)(参见第 13 章) 时 (其中节点代表标准神经网络中的确定性变量), 也会出现无概率解释的无向图.

11.1.2 分解

为了有利于使用有向图描述概率分布, 首先考虑关于 3 个变量 a、b 和 c 的任意联合分布 $p(a,b,c)$. 请注意, 在这个阶段, 我们不需要进一步说明它们都是些什么变量, 比如它们是离散变量还是连续变量. 事实上, 图模型的一个强大之处在于, 一个特定的图可以为多个分布类别做出概率说明. 通过应用概率的乘积法则 [式(2.9)], 我们可

以写下联合分布, 形式如下:

$$p(a,b,c) = p(c \mid a,b)\, p(a,b). \tag{11.4}$$

再次应用乘积法则对(11.4)右边的第二项进行计算, 得到

$$p(a,b,c) = p(c \mid a,b)\, p(b \mid a)\, p(a). \tag{11.5}$$

请注意, 无论选择何种联合分布, 以上分解都是成立的. 下面让我们用一个简单的图模型来表示(11.5)的右侧, 如图 11.1所示. 首先为随机变量 a、b 和 c 分别引入一个节点, 并将每个节点与(11.5)右侧相应的条件分布联系起来. 然后为每个条件分布添加有向连接 (如箭头所示), 这些连接来自与分布条件变量相对应的节点. 因此, 对于因子 $p(c \mid a,b)$, 将存在从节点 a 和 b 到节点 c 的连接; 而对于因子 $p(a)$, 则不存在传入连接. 如果节点 a 和节点 b 之间存在一个连接, 那么节点 a 是节点 b 的父节点, 相应地, 节点 b 是节点 a 的子节点. 请注意, 我们不会对节点以及节点所对应的变量进行任何形式上的区分, 而只是使用相同的符号指代两者.

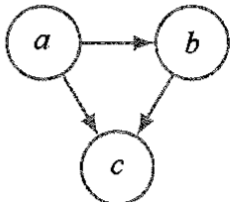


图 11.1: 用于表示变量 a、b、c 的联合概率分布的有向图模型, 对应于(11.5)右侧的分解

对于(11.5), 重要的一点是, 它的左侧变量与变量 a、b、c 是对称的, 而右侧变量则不对称. 在对(11.5)进行分解时, 我们隐式地选择了一种特定的排序, 即 a、b、c; 如果我们选择不同的排序, 就会得到不同的分解, 从而得到不同的图形化表示.

下面让我们通过 $p(x_1,\cdots,x_K)$ 给出的 K 个变量的联合分布来扩展图 11.1中的示例. 通过反复应用乘积法则, 该联合分布可以写成条件分布的乘积,K 个变量中的每一个变量都有一个条件分布:

$$p(x_1,\cdots,x_K) = p(x_K \mid x_1,\cdots,x_{K-1})\cdots p(x_2 \mid x_1)\, p(x_1). \tag{11.6}$$

对于给定的 K, 我们可以再次将这表示为一个有向图, 其中有 K 个节点.(11.6)右侧的每个条件分布都有一个节点, 且每个节点都有来自所有较低编号节点的输入连接. 我们称这个图是全连接的 (fully connected), 因为每一对节点之间都有一个连接.

到目前为止, 我们使用的都是完全通用的联合分布, 因此它们的分解和相关的全连接图表示适用于任何分布. 我们很快就会发现, 正是图中没有 (absence) 的连接传达了关于图所代表的分布类属性的有趣信息. 参见图 11.2所示的有向图. 请注意, 这并不是一个全连接图, 因为从节点 x_1 到 x_3 或者从节点 x_3 到 x_4 并不存在连接. 从这个有向图中提取出联合 (概率) 分布的相应表示, 即图 11.2给出了联合分布的相应分解. 例如, 节点 x_5 将以节点 x_1 和 x_3 为条件. 因此, 所有 7 个变量的联合分布为

$$p(x_1,\cdots,x_7) = p(x_1)\, p(x_2)\, p(x_3)\, p(x_4 \mid x_1,x_2,x_3)\, p(x_5 \mid x_1,x_3)\, p(x_6 \mid x_4)\, p(x_7 \mid x_4,x_5). \tag{11.7}$$

建议读者花点时间仔细研究一下(11.7)与图 11.2之间的对应关系.

现在, 我们可以用一般性术语来说明给定有向图与相应变量分布之间的关系.

命题 11.1 (有向图分解定理)

设 G 是一个有 K 个节点的有向无环图, 节点 k 对应变量 x_k, 且每个条件分布 $p(x_k \mid \mathrm{pa}(k))$ 给定. 则由 G 定义的联合分布为

$$p(\mathbf{x}) = \prod_{k=1}^{K} p\left(x_k \mid \mathrm{pa}(k)\right), \tag{11.8}$$

其中 $\mathrm{pa}(k)$ 表示节点 x_k 的父节点集. 只要各个条件分布被归一化,(11.8)的右端就总是被正确归一化的.

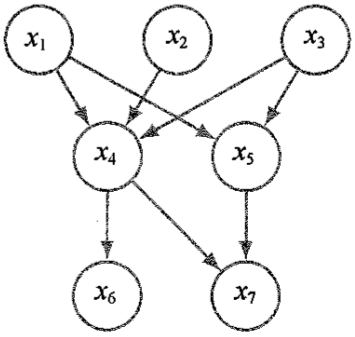


图 11.2: 一个描述变量 $x_1,\cdots,x_7$ 联合分布的有向图.(11.7)给出了联合分布的相应分解

证明

(重构新增) 设节点编号是拓扑序, 即每个节点的编号大于其父节点的编号 (存在性见练习 11.2). 对 k 归纳证明: 前 k 个变量的联合分布为 $\prod_{i=1}^{k} p(x_i \mid \mathrm{pa}(i))$.$k=1$ 时, 节点 1 没有父节点,$p(x_1)=p(x_1\mid\varnothing)$, 成立. 设结论对 $k-1$ 成立. 由乘积法则,

$$p(x_1,\cdots,x_k)=p(x_k\mid x_1,\cdots,x_{k-1})\,p(x_1,\cdots,x_{k-1}). \tag{11.9}$$

由于 $\mathrm{pa}(k)\subseteq\{1,\cdots,k-1\}$, 条件分布 $p(x_k\mid\mathrm{pa}(k))$ 是良定义的; 把 $\{x_j: j<k,\ j\notin\mathrm{pa}(k)\}$ 在给定 $\mathrm{pa}(k)$ 下边缘化, 由加和法则得 $p(x_k\mid x_1,\cdots,x_{k-1})=p(x_k\mid\mathrm{pa}(k))$. 代入归纳假设即得 k 的情形. 取 $k=K$ 便得到(11.8).

归一化: 设每个条件分布满足 $\int p(x_k\mid\mathrm{pa}(k))\,\mathrm{d}x_k=1$(离散情形为求和). 按 $k=K,K-1,\cdots,1$ 的顺序依次对 $x_K,x_{K-1},\cdots,x_1$ 积分. 当对 x_K 积分时, 因子 $p(x_K\mid\mathrm{pa}(K))$ 是唯一含 x_K 的因子, 其积分为 1, 其余因子与 x_K 无关而被提出积分号; 同理逐步消去全部变量, 最终结果为 1.

□

注

(重构新增) 命题 11.1是本章的“分解定理”, 它把图与分布的联系固定下来: 图给出因子的组织方式, 条件分布给出因子的具体数值. 证明中“拓扑序”这一条件不可省略: 若节点编号不是拓扑序, 则父亲可能出现在孩子之后, 条件分布 $p(x_k\mid\mathrm{pa}(k))$ 无法用前 $k-1$ 个变量表达, 归纳无法进行. 归一化结论依赖每个因子都是归一化的条件分布; 若某个因子只是非负函数, 则乘积的积分一般不等于 1, 这正是无向图必须引入配分函数的原因. 还需要强调:(11.8)只是说“由图定义的分布具有这种形式”, 并不排除同一个分布被多个不同的图表示, 这一点在小节 11.2.7中会以过滤器的语言重新表述.

我们现在所讨论的有向图有一个重要的限制条件, 即禁止存在有向环 (directed cycle). 换句话说, 图中不存在闭环路径, 这意味着我们不能沿着箭头的方向从一个节点移动到另一个节点, 最后回到起始节点.

定义 11.4 (有向无环图)

若一个有向图中不存在有向环, 即不存在节点序列 $v_1,v_2,\cdots,v_n$ 使得 $(v_i,v_{i+1})\in\mathcal{E}(i=1,\cdots,n-1)$ 且 $(v_n,v_1)\in\mathcal{E}$, 则称该图为**有向无环图** (Directed Acyclic Graph), 简称 DAG.

♣

注

(重构新增) 无环性是命题 11.1能够成立的前提, 也是“父节点”“祖先”“后代”这些概念不会自我指涉的保证. 若无环性被破坏, 例如 $x_1 \to x_2 \to x_1$, 则 $p(x_1 \mid x_2)$ 与 $p(x_2 \mid x_1)$ 互相定义, 乘积一般不再是归一化的联合分布, 甚至可能不存在满足二者的分布.DAG 的等价刻画是“存在节点排序使每条边都从小号指向大号” (练习 11.2), 这个刻画在实现中就是拓扑排序, 它同时给出了按顺序采样和按顺序边缘化的可行算法.

这表示存在一种节点排序, 使得不存在从任何节点到任何较低编号节点的连接 (见练习 11.2).

11.1.3 离散变量

我们已经讨论过作为指数族分布成员的概率分布的重要性, 并看到指数族分布中包括许多知名分布的特例 (参见节 3.4). 虽然这些分布相对简单, 但它们构成了构建更复杂概率分布的有用构件, 而图模型非常有助于表达这些构件是如何相互连接在一起的. 对于广泛使用的组件分布, 有两种特殊的选择, 分别对应于离散变量和 Gauss 变量. 我们先来看看离散变量.

具有 K 种可能状态 (使用 1-of-K 表示法) 的单个离散变量 x 的概率分布 $p(x \mid \boldsymbol{\mu})$ 由下式给出并由参数 $\boldsymbol{\mu} = (\mu_1, \cdots, \mu_K)^{\mathsf{T}}$ 控制:

$$p(x \mid \boldsymbol{\mu}) = \prod_{k=1}^{K} \mu_k^{x_k}. \tag{11.10}$$

由于约束 $\sum\limits_k \mu_k = 1$, 因此只需要指定 μ_k 的 $K-1$ 个值即可定义分布.

假设有两个离散变量 x_1 和 x_2, 其中每个变量都有 K 种状态, 希望模拟它们的联合分布. 用参数 μ_{kl} 表示同时观测到 $x_{1k} = 1$ 和 $x_{2l} = 1$ 的概率, 其中 x_{1k} 表示 x_1 的第 k 个组件,x_{2l} 表示 x_2 的第 l 个组件. 联合分布可以写为

$$p(x_1, x_2 \mid \boldsymbol{\mu}) = \prod_{k=1}^{K} \prod_{l=1}^{K} \mu_{kl}^{x_{1k} x_{2l}}. \tag{11.11}$$

由于参数 μ_{kl} 受到 $\sum\limits_k \sum\limits_l \mu_{kl} = 1$ 的约束, 因此以上联合分布受 $K^2 - 1$ 个参数的控制. 不难看出, 对于 M 个变量的任意联合分布, 必须指定的参数总数为 $K^M - 1$, 因此参数总数会随着变量数量 M 的增加而呈指数增长.

利用乘积法则, 我们可以将联合分布 $p(x_1, x_2)$ 分解为 $p(x_2 \mid x_1)\, p(x_1)$ 的形式, 对应于一个两节点图, 其中有一个从节点 x_1 到节点 x_2 的连接, 如图 11.3(a) 所示. 边缘分布 $p(x_1)$ 和之前一样受 $K-1$ 个参数的控制. 同样, 条件分布 $p(x_2 \mid x_1)$ 需要为 x_1 的 K 个可能取值中的每个值指定 $K-1$ 个参数. 因此, 在联合分布中必须指定的参数总数依然是 $(K-1) + K(K-1) = K^2 - 1$.

假设变量 x_1 和 x_2 是独立的, 对应于图 11.3(b) 所示的图模型. 然后, 将每个变量都用一个独立的离散分布来描述, 参数总数为 $2(K-1)$. 对于 M 个独立离散变量的分布 (每个变量有 K 个状态), 参数总数为 $M(K-1)$, 因此参数总数随着变量数量的增加而呈线性增长. 从图的角度来看, 我们已经通过放弃图中的连接来减少参数的数量, 但这样做的代价是分布的类更加有限了.

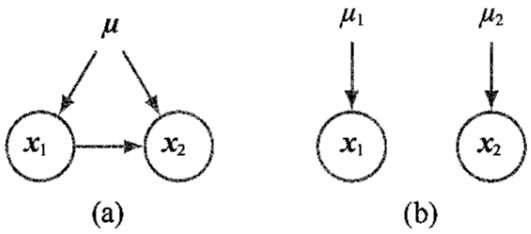


图 11.3: (a) 这个全连接图描述了两个 K 状态离散变量 (共有 $K^2 - 1$ 个参数) 的一般分布;(b) 通过放弃节点之间的连接, 参数数量减少至 $2(K-1)$

命题 11.2 (离散图模型的参数计数)

设 M 个离散变量各有 K 种状态, 联合分布按有向图分解为局部条件分布, 且每个条件分布用一般条件概率表给出.

(i) 若图为全连接图, 则参数总数为 $K^M - 1$;

(ii) 若图中没有连接, 则参数总数为 $M(K-1)$;

(iii) 若图为一条节点链 $x_1 \to x_2 \to \cdots \to x_M$, 则参数总数为 $K - 1 + (M-1)K(K-1)$.

♠

证明

(重构新增) 三种情形都按“每个节点的条件分布需多少个自由参数”相加得到, 全局归一化约束只消耗一个自由度.

(i) 全连接图中每个节点的父节点是全部较低编号节点, 故联合分布是一般的 K^M 格点分布, 除归一化约束外其余 $K^M - 1$ 个参数自由.

(ii) 无连接时 $p(\mathbf{x}) = \prod_i p(x_i)$, 每个 $p(x_i)$ 有 $K-1$ 个自由参数, 共 $M(K-1)$, 各因子分别归一化, 不再有额外的全局约束.

(iii) 节点 1 的分布 $p(x_1)$ 有 $K-1$ 个参数; 对其余每个节点, 条件分布 $p(x_i \mid x_{i-1})$ 是对 x_{i-1} 的 K 个取值分别给出的、长度为 K 的概率向量, 每个向量有 $K-1$ 个自由参数, 故为 $K(K-1)$. 于是总数为 $K-1+(M-1)K(K-1)$. 当 $M=2$ 时它等于 $K-1+K(K-1) = K^2-1$, 与 (i) 一致, 这符合“两个节点的全连接图就是一条链”的事实.

□

注

(重构新增) 命题 11.2量化了“图结构即先验假设”这一说法: 连接越少, 参数越少, 分布类越小.(i) 与 (iii) 的对比尤其重要: 全连接图的参数随 M 指数增长, 节点链却只随 M 线性增长. 注意 (iii) 中的常数 $K-1$ 来自第一个节点没有父节点, 这个“边界项”不能省略; 也正因为如此,(iii) 在 $M=1$ 时才退化为 $K-1$ 而不是 0. 这些计数都假设条件分布用完整条件概率表表示, 如果改用参数化形式 (如(11.12)), 计数会完全不同, 这一点在小节 11.1.3末尾会看到.

更一般地说, 如果有 M 个离散变量 $x_1, \cdots, x_M$, 则可以使用有向图对联合分布进行建模, 每个节点有一个变量. 每个节点上的条件分布由一组非负参数给出, 这些参数受限于归一化约束. 如果图是全连接的, 那么我们就有了一个完全通用的分布, 该分布有 $K^M - 1$ 个参数; 而如果图中没有连接, 联合分布就会分解为边缘分布的乘积, 参数总数为 $M(K-1)$. 具有中间水平连通性的图比完全分解的图更易于实现一般分布, 且比一般分布需要更少的参数. 如图 11.4所示, 边缘分布 $p(x_1)$ 需要 $K-1$ 个参数, 而对于 $i = 2, \cdots, M$, 条件分布 $p(x_i \mid x_{i-1})$ 中的每个分布都需要 $K(K-1)$ 个参数. 因此总的参数数量为 $K-1+(M-1)K(K-1)$, 与 K 成二次方并随节点链长度 M 的增加呈线性增长 (而非指数增长). 相比之下, 由 M 个节点组成的全连接图则有 $K^M - 1$ 个参数, 参数的数量随着 M 的增加呈指数增长.

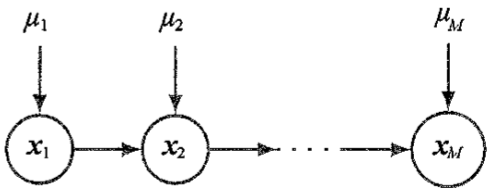


图 11.4: 这个具有 M 个离散节点 (每个节点有 K 个状态) 的节点链需要指定 $K-1+(M-1)K(K-1)$ 个参数, 参数的数量随着节点链长度 M 的增加呈线性增长. 相比之下, 由 M 个节点组成的全连接图则有 $K^M - 1$ 个参数, 参数的数量随着 M 的增加呈指数增长

共享参数 (也称捆绑参数) 是减少模型中独立参数数量的另一种方法. 例如, 在图 11.4中, 我们可以安排在 $i = 2, \cdots, M$ 的情况下, 所有条件分布 $p(x_i \mid x_{i-1})$ 都受同一组 $K(K-1)$ 个参数的控制, 从而得到图 11.5所示的模型. 再加上控制 x_1 分布的 $K-1$ 个参数, 总共需要指定 $K^2 - 1$ 个参数来定义联合分布.

定义 11.5 (共享参数)

若图中若干节点的条件分布 $p(x_k \mid \mathrm{pa}(k))$ 被约束为同一个参数化函数的不同取值, 即它们共用同一组参数 (而不各自拥有独立的参数), 则称这些参数是**共享参数** (也称**捆绑参数**,tied parameters). 共享参数不改变图的结构, 只改变因子的参数化方式.

♣

注

(重构新增) 共享参数与“删除连接”是两种互相独立的简化手段: 前者减少每个因子的自由参数, 后者减少因子的个数.图 11.5中把 $M-1$ 个转移分布捆绑在一起, 使参数从 $K-1+(M-1)K(K-1)$ 降到 K^2-1, 与 M 无关, 这正是“参数不随序列长度增长”的原型,第 12 章的循环网络与 Transformer 的权重共享思想都由此而来. 需要留意, 共享参数后联合分布仍然对每个 M 良定义, 但不同位置的条件分布不再能分别拟合, 这是一种更强的归纳偏置;定义 11.5只描述参数化的约束, 并不改变命题 11.1的分解形式.

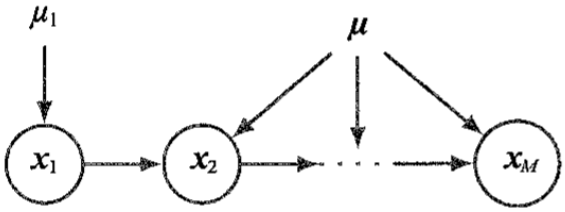


图 11.5: 与图 11.4相似, 但所有条件分布 $p(x_i \mid x_{i-1})$ 都共享一组参数 $\boldsymbol{\mu}$

在离散变量模型中, 控制参数数量增长的另一种方法是使用参数化的条件分布表, 而非使用完整的条件概率值表. 为了阐明这一观点, 请看图 11.6所示的模型, 其中所有节点都代表二进制变量. 每个父变量 x_i 由代表概率 $p(x_i=1)$ 的单个参数 μ_i 控制, 因此父节点共有 M 个参数. 然而, 条件分布 $p(y \mid x_1,\cdots,x_M)$ 需要 2^M 个参数, 用于表示父变量 2^M 种可能设置中每一种设置的概率 $p(y=1)$. 因此, 一般来说, 指定这一条件分布所需的参数数量将随着 M 的增加呈指数增长. 通过使用作用于父变量线性组合的逻辑斯谛 sigmoid 函数 (参见节 3.4), 我们可以得到如下更简洁的条件分布形式:

$$p(y=1 \mid x_1,\cdots,x_M)=\sigma\left(w_0+\sum_{i=1}^{M} w_i x_i\right)=\sigma\left(\mathbf{w}^{\mathsf{T}}\mathbf{x}\right), \tag{11.12}$$

其中,$\sigma(a)=(1+\exp(-a))^{-1}$ 是逻辑斯谛 sigmoid 函数,$\mathbf{x}=(x_0,x_1,\cdots,x_M)^{\mathsf{T}}$ 是一个 $(M+1)$ 维的父状态向量, 其中增加了一个变量 x_0, 其值限制为 1, 而 $\mathbf{w}=(w_0,w_1,\cdots,w_M)^{\mathsf{T}}$ 是一个包含 $M+1$ 个参数的向量. 与一般情况相比, 这种条件分布的形式更受限制, 但它现在受一些随 M 的增加呈线性增长的参数的控制. 从这个意义上讲, 这类似于在多元 Gauss 分布中选择协方差矩阵的限制形式 (例如对角矩阵).

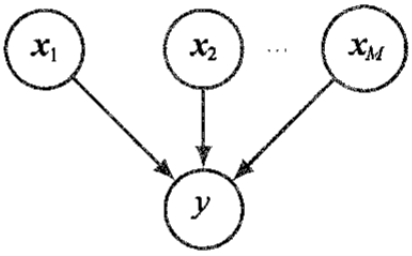


图 11.6: 一个由 M 个父变量 $x_1,\cdots,x_M$ 和一个子变量 y 组成的图, 用于说明离散变量参数化条件分布的概念

注

(重构新增) (11.12)与命题 11.2形成对照: 同一个图 (图 11.6) 配完整条件概率表需要 2^M 个参数, 配逻辑斯谛参数化则只需 $M+1$ 个. 代价是条件分布被限制为一类“单峰、单调”的形状:$p(y=1 \mid \mathbf{x})$ 只能随 $\mathbf{w}^{\mathsf{T}}\mathbf{x}$ 单调变化, 无法表示异或 (XOR) 那样的非单调依赖. 因此参数化是一个建模折中, 而不是纯粹的近似: 图结构表达的是条件独立假设, 参数化表达的是函数形式假设.小节 11.1.3最后提到的“协方差矩阵的对角限制”与此完全平行: 那里限制的是二阶矩的形状, 这里限制的是一阶响应的形状.

11.1.4 Gauss 变量

让我们回到图模型, 其中节点代表具有 Gauss 分布的连续变量. 每个分布都以图中父节点的状态为条件. 这种依赖关系可以有多种形式, 这里我们重点讨论一种特定的选择, 即每个 Gauss 分布的均值是 Gauss 父变量状态的某个线性函数. 这就产生了另一类模型——线性 Gauss 模型, 其中包括许多具有实际意义的案例, 如概率主成分分析 (见节 16.2)、因子分析和线性动态系统等 (Roweis and Ghahramani, 1999).

定义 11.6 (线性 Gauss 模型)

设 G 是一个有 D 个节点的有向无环图, 节点 i 代表连续随机变量 x_i. 若每个节点的条件分布为均值线性依赖于父节点取值的 Gauss 分布, 即

$$p\left(x_i \mid \mathrm{pa}(i)\right) = \mathcal{N}\left(x_i \,\middle|\, \sum_{j\in\mathrm{pa}(i)} w_{ij}x_j + b_i,\ v_i\right), \tag{11.13}$$

其中 w_{ij} 和 b_i 是控制均值的参数,v_i 是 x_i 条件分布的方差, 则称该图模型为**线性 Gauss 模型** (linear Gaussian model).

♣

注

(重构新增) 定义 11.6把“连续变量 + 线性依赖 +Gauss 噪声”三件事捆在一起. 它的动机是:Gauss 分布在线性变换与边缘化下封闭, 因此整张图的联合分布仍是 Gauss 分布, 从而可以用均值和协方差两个量完全描述 (命题 11.3). 条件中“均值线性”不可换成任意非线性函数, 否则联合分布不再是 Gauss, 矩递推随即失效; 方差 v_i 不依赖于父节点取值, 这是“同方差”假设, 若允许 v_i 随 $\mathrm{pa}(i)$ 变化, 模型就变成异方差回归, 仍可用但递推公式需改写. 节点编号必须是拓扑序, 否则 $j\in\mathrm{pa}(i)$ 可能大于 i, 递推无法从低到高进行.

考虑 D 个变量上的任意一个有向无环图, 其中节点 i 代表具有 Gauss 分布的单个连续随机变量 x_i. 该 Gauss 分布的均值是节点 i 的父节点 $\mathrm{pa}(i)$ 状态的线性组合, 如(11.13)所示. 联合分布的对数就是图中所有节点的这些条件式乘积的对数, 因此其形式为

$$\begin{aligned}\ln p(\mathbf{x}) &= \sum_{i=1}^{D} \ln p\left(x_i \mid \mathrm{pa}(i)\right) \\ &= -\sum_{i=1}^{D} \frac{1}{2v_i}\left(x_i - \sum_{j\in\mathrm{pa}(i)} w_{ij}x_j - b_i\right)^2 + \text{const},\end{aligned} \tag{11.14}$$

其中 $\mathbf{x}=(x_1,\cdots,x_D)^{\mathsf{T}}$, 且 const 表示与 $\mathbf{x}$ 无关的常数项. 可以看到, 这是 $\mathbf{x}$ 的组件的二次函数, 因此联合分布 $p(\mathbf{x})$ 是多元 Gauss 分布.

命题 11.3 (线性 Gauss 模型的矩递推)

在定义 11.6的记号下, 并设节点编号为拓扑序 (每个节点的编号大于其父节点的编号), 则联合分布的均值与协方差满足

$$\mathbb{E}[x_i] = \sum_{j\in\mathrm{pa}(i)} w_{ij}\,\mathbb{E}[x_j] + b_i, \tag{11.15}$$

$$\operatorname{cov}[x_i, x_j] = \sum_{k \in \mathrm{pa}(i)} w_{ik} \operatorname{cov}[x_k, x_j] + I_{ij} v_i, \tag{11.16}$$

其中 I_{ij} 是单位矩阵的元素 ($i = j$ 时为 1, 否则为 0). 因此 $\mathbb{E}[\mathbf{x}]$ 与协方差矩阵可以从最低编号的节点开始递推求得.

♠

证明

(重构新增) 把(11.13)写成显式的生成式:

$$x_i = \sum_{j \in \mathrm{pa}(i)} w_{ij} x_j + b_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, v_i), \tag{11.17}$$

且 ε_i 与 $\{\varepsilon_k : k \neq i\}$ 相互独立.

均值: 对(11.13)取期望并用期望的线性性,$\mathbb{E}[x_i \mid \mathrm{pa}(i)] = \sum_j w_{ij} x_j + b_i$, 再由全期望公式 $\mathbb{E}[x_i] = \mathbb{E}\big[\mathbb{E}[x_i \mid \mathrm{pa}(i)]\big]$ 即得(11.15).

协方差: 记 $\tilde{x}_i = x_i - \mathbb{E}[x_i]$, 则 $\tilde{x}_i = \sum_{k \in \mathrm{pa}(i)} w_{ik} \tilde{x}_k + \varepsilon_i$. 两边乘以 $\tilde{x}_j$ 并取期望:

$$\operatorname{cov}[x_i, x_j] = \sum_{k \in \mathrm{pa}(i)} w_{ik} \operatorname{cov}[x_k, x_j] + \mathbb{E}[\varepsilon_i \tilde{x}_j]. \tag{11.18}$$

由于节点编号是拓扑序,$j < i$ 时 x_j 只依赖 $\{\varepsilon_k : k \leqslant j\}$, 与 ε_i 独立, 故 $\mathbb{E}[\varepsilon_i \tilde{x}_j] = 0$;$j = i$ 时 $\mathbb{E}[\varepsilon_i \tilde{x}_i] = \mathbb{E}[\varepsilon_i^2] = v_i$. 两句合起来正是 $I_{ij} v_i$. 递推的可用性:(11.16)的右端只涉及编号更小的 k, 而协方差矩阵对称, 故按 $i = 1, 2, \cdots, D$ 的顺序逐行计算即可.

□

注

(重构新增) 原书把该递推式的右端印作 $\operatorname{cov}[x_i, x_k]$(本译本按推导结果写作 $\operatorname{cov}[x_k, x_j]$), 与左端的下标不一致, 也和下面(11.20)的矩阵不符. 上面的证明给出正确的递推: 右端应为 $\operatorname{cov}[x_k, x_j]$, 因为对 x_i 取协方差时展开的是 x_i 对父节点的线性组合, 被乘的 $\tilde{x}_j$ 保持不变. 用例 11.1的 3 变量链可以逐项验证:$\Sigma_{22} = w_{21} \operatorname{cov}[x_1, x_2] + v_2 = w_{21}^2 v_1 + v_2$, 只有用 $\operatorname{cov}[x_1, x_2]$(即 $\operatorname{cov}[x_k, x_j]$) 才能得到(11.20)中印出的结果. 条件 $I_{ij} v_i$ 中的 v_i(而不是 v_j) 也由证明确定: 噪声 ε_i 属于第 i 个方程. 这个例子说明, 递推公式的每一个下标都必须由推导决定, 不能凭对称性猜测.

例题 11.1 三变量线性 Gauss 模型 考虑图 11.7所示的 3 个 Gauss 变量的有向图, 即 $x_1 \to x_2 \to x_3$, 其中 x_1 与 x_3 之间缺少一个连接. 由(11.15)与(11.16)可得联合分布的均值与协方差为

$$\boldsymbol{\mu} = \big(b_1,\ b_2 + w_{21} b_1,\ b_3 + w_{32} b_2 + w_{32} w_{21} b_1\big)^{\mathsf{T}}, \tag{11.19}$$

$$\boldsymbol{\Sigma} = \begin{pmatrix} v_1 & w_{21} v_1 & w_{32} w_{21} v_1 \\ w_{21} v_1 & v_2 + w_{21}^2 v_1 & w_{32}\big(v_2 + w_{21}^2 v_1\big) \\ w_{32} w_{21} v_1 & w_{32}\big(v_2 + w_{21}^2 v_1\big) & v_3 + w_{32}^2\big(v_2 + w_{21}^2 v_1\big) \end{pmatrix}. \tag{11.20}$$

注

(重构新增) 例 11.1展示了两件事. 第一, 缺少连接 $x_1 \to x_3$ 的效果是 $\Sigma_{31} = w_{32} w_{21} v_1$, 即 x_1 对 x_3 的影响必须经由 x_2 传递, 系数是两个权重的乘积; 若存在直接的 $x_1 \to x_3$ 连接, 则 Σ_{31} 中还要加上 $w_{31} v_1$ 一项. 第二, 协方差矩阵的元素个数虽然很多, 但都由 $v_1, v_2, v_3, w_{21}, w_{32}$ 这 5 个参数决定, 而不是一般的 3×3 对称矩阵所需的 6 个参数, 这正是"图结构约束协方差"的具体含义. 注意 Σ 是正定的 (两个 w 取任何实数都成立), 这来自生成式 $x = (I - W)^{-1}(\mathbf{b} + \boldsymbol{\varepsilon})$ 的结构, 无需额外假设.

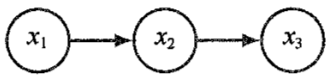


图 11.7: 3 个 Gauss 变量的有向图, 其中缺少一个连接

我们可以使用以下方法求出该联合分布的均值和协方差. 每个变量的均值由递推公式给出 (见练习 11.6), 即(11.15), 因此我们可以从最低编号的节点开始, 通过图递归地找到 $\mathbb{E}[\mathbf{x}] = (\mathbb{E}[x_1], \cdots, \mathbb{E}[x_D])^{\mathsf{T}}$ 的组件, 其中我们假设每个节点的编号大于其父节点的编号. 同样, 联合分布的协方差矩阵元素也满足(11.16)形式的递推公式 (见练习 11.7), 因此, 协方差同样可以从最低编号的节点开始递归求值.

考虑两个可能出现的图结构极端案例. 首先, 假设图中无连接, 因此图由 D 个孤立节点构成. 在这个情况下, 参数 w_{ij} 不存在, 因此只有 D 个参数 b_i 和 D 个参数 v_i. 从递推公式 [(11.15)和(11.16)] 中可以看出,$p(\mathbf{x})$ 的均值为 $(b_1, \cdots, b_D)^{\mathsf{T}}$, 协方差矩阵为 $\mathrm{diag}(v_1, \cdots, v_D)$ 形式的对角矩阵. 联合分布共有 $2D$ 个参数且代表一组 D 个独立的单变量 Gauss 分布. 其次, 考虑一个全连接图, 其中每个节点都将所有较低编号的节点作为其父节点. 在这种情况下, 协方差矩阵中独立参数 w_{ij} 和 v_i 的总数为 $D(D+1)/2$, 对应于一般的对称协方差 (见练习 11.8).

具有某种中等复杂程度的图对应于具有部分约束协方差矩阵的联合 Gauss 分布. 例如, 如图 11.7所示, 在 Gauss 变量 x_1 和 x_3 之间缺少一个连接, 联合分布的均值和协方差就由(11.19)和(11.20)给出 (见练习 11.9).

我们很容易将线性 Gauss 图模型扩展到图节点代表多元 Gauss 变量的情况. 在这种情况下, 我们可以写下节点 i 的条件分布, 形式如下:

$$p\big(\mathbf{x}_i \mid \mathrm{pa}(i)\big) = \mathcal{N}\left(\mathbf{x}_i \;\middle|\; \sum_{j \in \mathrm{pa}(i)} \mathbf{W}_{ij}\mathbf{x}_j + \mathbf{b}_i,\ \boldsymbol{\Sigma}_i\right), \tag{11.21}$$

其中 $\mathbf{W}_{ij}$ 是一个矩阵 (如果 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 的维度不同, 则这个矩阵是非正方形的). 同样, 我们很容易验证所有变量的联合分布是 Gauss 分布 (见练习 11.10).

11.1.5 二元分类器

下面我们用一个二元分类器来说明如何使用有向图对概率分布进行描述, 该模型的可学习参数具有 Gauss 先验 (参见小节 2.6.2). 我们可以将其写成如下形式:

$$p(\mathbf{t}, \mathbf{w} \mid \mathbf{X}, \lambda) = p(\mathbf{w} \mid \lambda) \prod_{n=1}^{N} p(t_n \mid \mathbf{w}, \mathbf{x}_n), \tag{11.22}$$

其中,$\mathbf{t} = (t_1, \cdots, t_N)^{\mathsf{T}}$ 是目标值的向量,$\mathbf{X}$ 是行 $\mathbf{x}_1^{\mathsf{T}}, \cdots, \mathbf{x}_N^{\mathsf{T}}$ 的数据矩阵. 分布 $p(t_n \mid \mathbf{x}_n, \mathbf{w})$ 由(11.1)给出. 我们还假设参数向量 $\mathbf{w}$ 的 Gauss 先验为

$$p(\mathbf{w} \mid \lambda) = \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \lambda\mathbf{I}). \tag{11.23}$$

该模型中的随机变量为 $\{t_1, \cdots, t_N\}$ 和 $\mathbf{w}$. 此外, 该模型还包含噪声方差 σ^2 和超参数 λ, 它们是模型的参数而非随机变量. 如果我们暂时只考虑随机变量, 那么(11.22)给出的分布可以用图 11.8所示的图模型来表示.

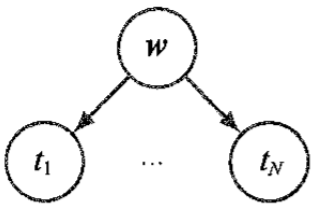


图 11.8: 二元分类器的有向图模型, 该模型仅显示随机变量 $\{t_1, \cdots, t_N\}$ 和 $\mathbf{w}$

当开始处理更复杂的模型时, 要像图 11.8那样明确写出 $t_1, \cdots, t_N$ 形式的多个节点就会变得很不方便. 因此, 我们引入了一种图形符号, 以便更简洁地表示这样的多个节点.

定义 11.7 (板块表示法)

板块 (plate) 是一种图形缩写: 把一组同类的、仅在索引上不同的节点画成一个代表性节点, 并用一个方框把它包围起来, 方框角上标注索引集合的大小 (如 N), 表示方框内的图形被重复 N 次, 每次取一个不同的

索引值.

注

(重构新增) 板块不是一种新的数学对象, 而是一种严格的书写约定:图 11.9与图 11.8表示同一个分布(11.22), 只是前者把 N 个同构的因子折叠成一个. 它的作用是让“数据独立同分布”这一假设在图上一眼可见: 方框内的 t_n 之间没有连接, 正因为它们在给定 $\mathbf{w}$ 后条件独立. 反之, 如果数据之间存在依赖 (例如序列数据), 就必须在方框内画出节点之间的边, 或者放弃板块而显式地画出整条链. 理解这一点是阅读后续章节中变分自编码器、扩散模型等“带板块”的图的前提.

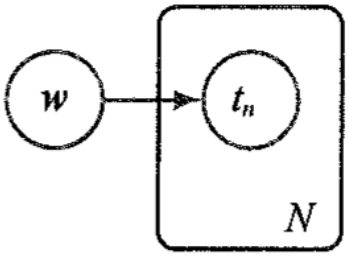


图 11.9: 图 11.8所示图模型的另一种更紧凑的表示法, 其中引入了一个板块 (标为 N 的方框) 来表示 N 个节点, 这里仅明确展现了一个示例节点 t_n

用此方式修改图 11.8, 便可得到图 11.9.

11.1.6 参数和观测值

有时我们会发现, 用图形化表示法明确表示模型的参数及随机变量是很有帮助的. 为此, 我们将遵照以下惯例: 随机变量用开圆表示, 确定性参数用浮动变量表示. 如果我们采用图 11.9所示的图模型并加入确定性参数, 即可得到图 11.10所示的图模型.

当我们将图模型应用于机器学习问题时, 通常会将一些随机变量设置为特定的观测值. 例如, 线性回归模型中的随机变量 t_n 将设置为等于训练集中给出的特定值. 在图模型中, 可通过给相应的节点添加阴影来表示这些观测变量. 因此, 与图 11.10所示的图模型 (其中变量 t_n 被观测) 相对应的图模型如图 11.11所示.

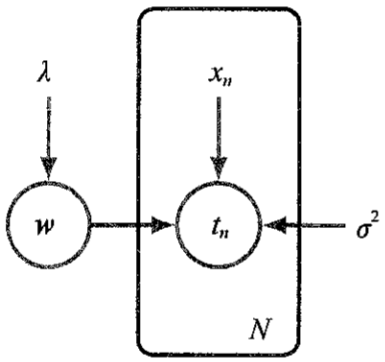


图 11.10: 与图 11.9所示的图模型相同, 但确定性参数用浮动变量来明确表示

请注意,$\mathbf{w}$ 的值是无法观测到的, 因此 $\mathbf{w}$ 是一个潜 (latent) 变量 (也称隐变量). 这些变量在本书讨论的许多模型中都发挥了至关重要的作用.

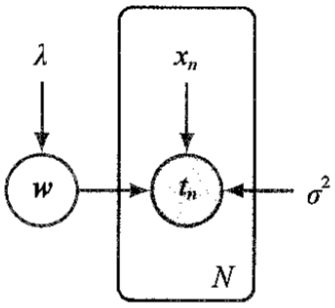


图 11.11: 与图 11.10所示的图模型相同, 但节点 t_n 被添加阴影以表示相应的随机变量已设置为训练集中给出的观测值

定义 11.8 (三类变量)

在有向图模型中有 3 种变量, 用 3 种画法区分:

(i) **未观测到的随机变量** (也称潜在变量或隐藏变量), 用红色圆圈表示;

(ii) **观测到的随机变量**, 即已被设置为特定值的随机变量, 用蓝色阴影的红圈表示;

(iii) **非随机参数**, 用浮动变量 (不带圆圈) 表示.

♣

注

(重构新增) 定义 11.8的区分决定了哪些量需要积分、哪些量可以当作常数. 随机变量 (无论是否观测) 都参与联合分布与边缘化; 参数不参与积分, 它们是条件分布的“旋钮”, 在频率学派的意义下由最大化似然而定, 在 Bayes 意义下则可以先给它们一个先验而升级为随机变量 (此时图 11.10中的 λ 就成为随机节点). 这种“升级”正是第 15 章与第 16 章中从最大似然过渡到变分推断的入口. 需要注意, 同一符号在不同语境下可能扮演不同角色:图 11.11中 t_n 被观测, 于是它的分布退化为一个点质量, 图中表现为阴影.

请注意, 由于我们最终的目标是对新的输入值进行预测, 因此诸如 $\mathbf{w}$ 的模型参数一般不会与它们自身有直接关系. 假设我们得到了一个新的输入值 $\hat{\mathbf{x}}$, 并希望以观测到的数据为条件找到 $\hat{t}$ 的相应概率分布. 以确定性参数为条件, 该模型中所有随机变量的联合分布由下式给出:

$$p(\hat{t}, \mathbf{t}, \mathbf{w} \mid \hat{\mathbf{x}}, \mathbf{X}, \lambda) = p(\mathbf{w} \mid \lambda)\, p(\hat{t} \mid \mathbf{w}, \hat{\mathbf{x}}) \prod_{n=1}^{N} p(t_n \mid \mathbf{w}, \mathbf{x}_n). \tag{11.24}$$

相应的图模型如图 11.12所示. 然后, 通过对模型参数 $\mathbf{w}$ 进行积分, 就能从加和法则中得到 $\hat{t}$ 所需的预测分布. 这种对参数的积分是一种完全 Bayes 式的处理方法, 在实践中很少使用, 尤其是在深度神经网络中. 相反, 我们通常首先找到能使后验分布最大化的最可能值 $\mathbf{w}_{\text{MAP}}$ 来近似这个积分, 然后仅使用这个单一值并应用 $p(\hat{t} \mid \hat{\mathbf{x}}, \mathbf{w}_{\text{MAP}})$ 进行预测.

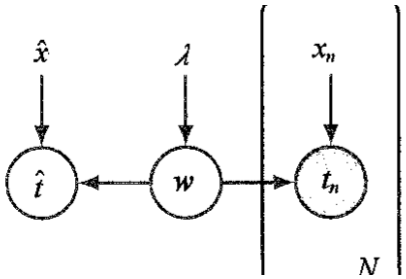


图 11.12: 与图 11.11所示的图模型相对应的分类模型, 显示了新输入值 $\hat{\mathbf{x}}$ 和相应的模型预测值 $\hat{t}$

11.1.7 Bayes 定理

当概率模型中的随机变量设定为与观测值相等时, 其他未观测到的随机变量的分布也会相应地改变.

定义 11.9 (推理)

推理 (inference) 是指: 在概率模型中某些随机变量被设定为观测值之后, 计算其余未观测变量的条件分布 (后验分布) 的过程. 在图中, 这相当于把被观测节点加上阴影, 然后重新计算未加阴影节点的分布. ♣

注

(重构新增) 定义 11.9把“学习”与“推理”分开: 学习是确定参数 (或参数的后验), 推理是在给定参数与观测后求隐变量或预测量的分布. 推理的计算复杂度完全由图的结构决定: 树状图上存在线性时间的精确算法 (消息传递), 有环图一般只能近似. 定义中“设定为观测值”在数学上是把联合分布在对应变量处取值并重新归一化, 因此后验分布与先验分布同属一个分布族的情形 (共轭) 是特例而非常态;第 15 章与第 16 章的变分推断正是为了绕开无法解析计算的后验.

我们可以通过 Bayes 定理的图形解释来说明这一点. 假设我们把两个变量 x 和 y 的联合分布 $p(x,y)$ 分解成因子的乘积, 形式为 $p(x,y) = p(x)\,p(y \mid x)$. 这可以用图 11.13(a) 中的有向图来表示. 假设我们观测到了 y 的值, 如图 11.13(b) 中的阴影节点所示. 我们可以把边缘分布 $p(x)$ 看作潜变量 x 的一个先验概率, 而我们的目标就是推断出相应的后验概率.

命题 11.4 (两个变量的 Bayes 更新)

设联合分布分解为 $p(x,y) = p(x)\,p(y \mid x)$,$p(x)$ 称为先验. 则边缘分布与后验分布分别为

$$p(y) = \sum_{x'} p(y \mid x')\,p(x'), \tag{11.25}$$

$$p(x \mid y) = \frac{p(y \mid x)\,p(x)}{p(y)}, \tag{11.26}$$

其中(11.25)右端的 x' 是与 x 同取值域的求和哑变量. 于是联合分布也可以用 $p(x \mid y)$ 与 $p(y)$ 表示为 $p(x,y) = p(x \mid y)\,p(y)$, 对应的图是把箭头反向后的 $x \leftarrow y$. ♠

证明

(重构新增) 由加和法则,$p(y) = \sum_{x'} p(y,x') = \sum_{x'} p(y \mid x')p(x')$, 即(11.25). 由乘积法则 $p(x,y) = p(y \mid x)p(x)$, 又 $p(x,y) = p(x \mid y)p(y)$, 两式相除 (设 $p(y) > 0$) 即得(11.26). 把(11.26)代回 $p(x \mid y)p(y) = p(y \mid x)p(x) = p(x,y)$, 可见用 $(p(x \mid y), p(y))$ 同样能重构联合分布, 而这一分解对应的分解式正是 $x \leftarrow y$ 方向上的 $p(y)\,p(x \mid y)$.

□

注

(重构新增) 命题 11.4是全书最简单的推理例子, 但它已经包含了推理的全部要素: 先验、似然、证据 (分母) 与后验. 分母 $p(y)$ 只依赖于观测值, 对 x 是常数, 因此实践中常写成 $p(x \mid y) \propto p(y \mid x)p(x)$, 把归一化留到最后. 条件 $p(y) > 0$ 不能省略: 若观测到的事件概率为零, 后验无定义, 这正是“零概率观测”在连续模型中必须用密度而非概率处理的原因. 图中箭头反向这一事实说明: 图的结构描述的是分解方式, 而不是因果方向;Bayes 定理允许我们在同一个联合分布上换一种分解, 从而换一张图 (图 11.13(c)).

因此, 联合分布现在可以用 $p(x \mid y)$ 和 $p(y)$ 来表示. 从图的角度看, 联合分布 $p(x,y)$ 可以用图 11.13(c) 所示的图来表示, 其中箭头的方向是相反的. 这是图模型推理问题的最简单示例.

对于捕获到丰富概率结构的复杂图模型来说, 一旦观测到某些随机变量, 计算后验分布的过程就会变得复杂而巧妙. 从概念上讲, 它只涉及系统地应用概率的加和法则与乘积法则, 或等同于 Bayes 定理. 然而在实践中, 利用图结构可以大大提高计算和管理的效率. 这些计算可以用图 (涉及在节点间发送本地信息) 上优雅的计算来表示. 这些方法不仅对树状结构的图给出了精确答案, 也对有环图给出了近似迭代算法. 我们不会在此进一步讨论这些问题, 感兴趣的读者请参阅 Bishop (2006), 以了解机器学习方面更全面的探讨.

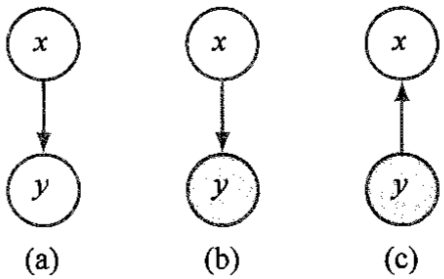


图 11.13: Bayes 定理的图示:(a) 以分解形式表示的两个变量 x 和 y 的联合分布;(b) y 设为观测值的情况;(c) Bayes 定理给出的 x 的后验分布

11.2 条件独立性

多变量概率分布的一个重要概念是条件独立性 (Dawid, 1980). 考虑 3 个变量 a、b、c, 假设在给定 b 和 c 的条件下,a 的条件分布不依赖于 b 的值, 即

$$p(a \mid b, c) = p(a \mid c). \tag{11.27}$$

定义 11.10 (条件独立性)

若对 c 的每一个可能取值都有(11.27)成立, 则称**在给定 c 的条件下,a 与 b 是条件独立的** (conditionally independent). 若存在某个 c 的取值使命题不成立, 则不能称二者条件独立. ♣

注

(重构新增) 定义 11.10中最容易被忽略的是“对 c 的每一个可能值”这一全称量词. 逐点成立才是条件独立; 只在某些 c 上成立通常是无意义的, 因为条件分布本来就是 c 的函数. 与之对照,a 与 b 的边缘独立只要求一个等式 $p(a,b) = p(a)p(b)$, 它是把 c 边缘化之后的整体性质, 二者互不蕴含: 本节例 11.2给出“条件独立但边缘不独立”的例子,例 11.4给出“边缘独立但条件不独立”的例子. 定义没有限制变量类型, 离散时 p 为概率, 连续时 p 为密度, 两种情形下等式的含义相同.

我们称之为, 在给定 c 的条件下,a 与 b 是条件独立的. 如果我们考虑以 c 为条件的 a 和 b 的联合分布, 就可以用稍微不同的方式来表示, 形式如下:

$$p(a, b \mid c) = p(a \mid b, c)\, p(b \mid c) = p(a \mid c)\, p(b \mid c), \tag{11.28}$$

其中利用了概率的乘积法则和(11.27).

命题 11.5 (条件独立性的等价刻画)

设 $p(b,c) > 0$. 则下列两条等价:

(i) $p(a \mid b, c) = p(a \mid c)$ 对 c 的每个取值成立;

(ii) $p(a, b \mid c) = p(a \mid c)\, p(b \mid c)$ 对 c 的每个取值成立.

也就是说, 在给定 c 的条件下,a 与 b 的联合分布分解为二者 (同样以 c 为条件的) 边缘分布的乘积. ♠

证明

(重构新增)(i)⇒(ii): 由乘积法则 $p(a,b \mid c) = p(a \mid b,c)p(b \mid c)$, 代入 (i) 即得 (ii).(ii)⇒(i): 在 $p(b \mid c) > 0$ 处, 由 (ii) 与乘积法则得 $p(a \mid b,c) = p(a,b \mid c)/p(b \mid c) = p(a \mid c)$; 在 $p(b \mid c) = 0$ 处条件分布 $p(a \mid b,c)$ 本身无定义, 此时约定 (i) 按“几乎处处”理解即可, 不影响任何积分.

□

注

(重构新增) 命题 11.5说明“条件独立”与“以 c 为条件后 a、b 统计独立”是同一件事, 这正是名称的由来. 两个方向的证明都只用到乘积法则, 因此条件独立性不是新的公理, 而是乘积法则在一个特殊分解下的推论, 这也是为什么它可以在图中被“读出来”. 条件 $p(b,c)>0$ 是技术性的: 若 c 的某个取值使 $p(b \mid c)=0$, 则条件概率本身无定义, 此时等式只在正概率处有意义. 该命题还提示了一个实用判据: 检验条件独立性等价于检验一个分解式, 而不必真的去算条件分布.

也就是说, 在给定 c 的条件下, 变量 a 和 b 在统计上是独立的. 请注意, 我们对条件独立性的定义要求(11.27)或(11.28)必须对 c 的每一个可能值都成立, 而不仅仅对 c 的某些值成立. 我们有时也会使用条件独立性的速记符号 (Dawid, 1979), 其中

$$a \perp b \mid c \tag{11.29}$$

表示在给定 c 的条件下,a 和 b 是条件独立的. 条件独立性在机器学习的概率模型中发挥着重要作用, 因为它既简化了模型的结构, 也简化了在模型下执行推理和学习所需的计算.

如果能用条件分布的乘积 (即有向图的数学表达式) 得出一组变量的联合分布表达式, 则原则上可以通过反复应用概率的加和法则与乘积法则来检验任何潜在的条件独立性是否成立. 实际上, 该方法十分耗时. 图模型的一个重要而优雅的特点是, 可以直接从图中读取联合分布的条件独立性, 而无须执行任何分析操作. 实现此过程的一般框架称为 d 分离 (d-separation), 其中“d”代表“有向”(Pearl, 1988). 本节将介绍 d 分离的概念并给出 d 分离准则的一般说明, 形式证法见 Lauritzen (1996).

11.2.1 3 个示例图

在开始讨论有向图的条件独立性之前, 我们先来看 3 个简单的例子, 每个例子都涉及只有 3 个节点的图. 这些将共同促进和说明 d 分离的关键概念.

例题 11.2 尾尾相接 第 1 个示例如图 11.14所示, 利用一般结果 [(11.8)] 可以很容易地写出该图所对应的联合分布, 即

$$p(a,b,c)=p(a \mid c)\, p(b \mid c)\, p(c). \tag{11.30}$$

如果未观测到任何变量, 则可以通过将(11.30)的两边相对于 c 边缘化来研究 a 和 b 是否独立, 即

$$p(a,b)=\sum_{c} p(a \mid c)\, p(b \mid c)\, p(c). \tag{11.31}$$

一般来说, 这不会分解为乘积 $p(a)p(b)$, 因此

$$a \not\perp b \mid \varnothing, \tag{11.32}$$

其中 $\varnothing$ 表示空集, 符号 $\not\perp$ 表示条件独立性一般不成立. 当然, 对于特定的分布来说, 条件独立性也可能由于与各种条件概率相关的特定数值而成立, 但一般来说, 这并不是由图的结构得出的.

假设我们以节点 c 为条件, 如图 11.15中的有向图所示. 根据(11.30), 我们可以很容易地写出在给定 c 的条件下 a 和 b 的条件分布, 其形式为

$$p(a,b \mid c)=\frac{p(a,b,c)}{p(c)}=p(a \mid c)\, p(b \mid c). \tag{11.33}$$

于是, 我们得到如下条件独立性:

$$a \perp b \mid c. \tag{11.34}$$

可以考虑从节点 a 经过节点 c 到达节点 b 的路径, 从而对这一结果进行简单的图解释. 节点 c 之所以相对于该路径称为**尾尾相接** (tail-to-tail) 节点, 就是因为该节点与两个箭头的尾部相连, 而连接节点 a 和 b 的路径的存在会导致这两个节点相互依赖. 然而, 如图 11.15所示, 当我们以节点 c 为条件时, 条件节点会“阻塞”从 a 到 b 的路径, 并使 a 和 b 变得 (条件) 独立.

注

(重构新增)例 11.2是 d 分离的第一个基本构件. 它的结构含义是: 若两个变量共享一个共同的“原因” c, 则它们在边缘上相关 (因为 c 的取值同时影响二者), 但一旦知道了 c, 这种通过 c 的相关就被切断. 这就是“以共同原因为条件可以去除伪相关”的形式化, 也是小节 11.2.4中朴素 Bayes 假设的原型. 注意(11.31)中的求和: 只有当 c 被边缘化掉时,$p(a \mid c)p(b \mid c)$ 才会通过 $p(c)$ 把二者耦合起来; 这一步不可省略, 否则会误以为尾尾相接总是导致独立.

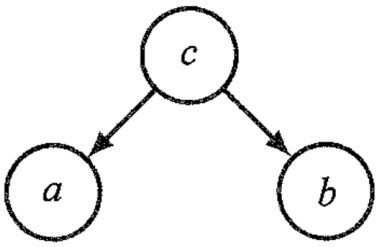


图 11.14: 第 1 个示例用于讨论有向图的条件独立性

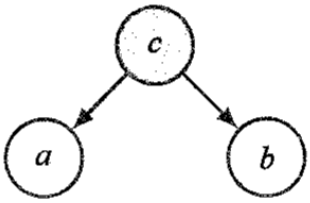


图 11.15: 与图 11.14相同, 但以节点 c 为条件

例题 11.3 头尾相接 同样, 我们也可以考虑图 11.16所示的有向图. 根据(11.8), 我们可以得到与该有向图相对应的联合分布, 即

$$p(a,b,c) = p(a)\,p(c \mid a)\,p(b \mid c). \tag{11.35}$$

假设未观测到任何变量. 我们可以通过对 c 进行边缘化来检验 a 和 b 是否独立,

$$p(a,b) = p(a)\sum_{c} p(c \mid a)\,p(b \mid c) = p(a)\,p(b \mid a), \tag{11.36}$$

从而得出

$$a \not\perp b \mid \varnothing. \tag{11.37}$$

假设以节点 c 为条件, 如图 11.17中的有向图所示. 利用 Bayes 定理和(11.35), 即可得出

$$p(a,b \mid c) = \frac{p(a,b,c)}{p(c)} = \frac{p(a)\,p(c \mid a)\,p(b \mid c)}{p(c)} = p(a \mid c)\,p(b \mid c). \tag{11.38}$$

于是, 我们再次得到如下条件独立性:

$$a \perp b \mid c. \tag{11.39}$$

和以前一样, 我们可以用图形来解释这些结果. 就节点 a 到节点 b 的路径而言, 节点 c 称为**头尾相接** (head-to-tail) 节点. 该路径连接了节点 a 和 b, 并使它们相互依赖. 假设我们现在观测到了节点 c, 如图 11.17所示, 那么这个观测结果就会“阻塞”从节点 a 到 b 的路径, 从而得到条件独立性 $a \perp b \mid c$.

注

(重构新增) 例 11.3是“中介”结构的原型:a 通过 c 影响 b, 因此 a 与 b 边缘相关; 一旦 c 被观测, 信息通道被截断, 二者条件独立. 它与尾尾相接的区别在于阻塞的机制不同: 尾尾相接是“共同原因”, 这里是“链条”. 这一区别在小节 11.2.3被统一为同一条规则——两者都要求 c 被观测才阻塞; 而下面的头头相接恰好相反. 注意(11.35)的分解顺序与 $a \to c \to b$ 的箭头方向一致, 若把三个变量重新排序, 同一分布会写成另一种分解, 但条件独立性的结论不变.

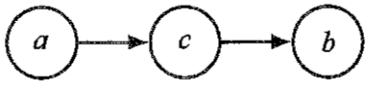


图 11.16: 第 2 个示例用于促进建立有向图的条件独立性框架

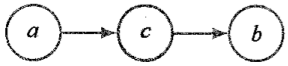


图 11.17: 与图 11.16相同, 但以节点 c 为条件

例题 11.4 头头相接 最后, 让我们来看看第 3 个示例, 如图 11.18中的有向图所示. 利用一般结果 [(11.8)] 写出该图所对应的联合分布, 即

$$p(a,b,c) = p(a)\,p(b)\,p(c \mid a,b). \tag{11.40}$$

首先考虑未观测到任何变量的情况. 在 c 上对(11.40)的两边进行边缘化, 得到

$$p(a,b) = p(a)\,p(b), \tag{11.41}$$

因此, 与前两个例子所不同的是,a 和 b 是独立的, 且未观测到任何变量. 我们可以将这一结果写成

$$a \perp b \mid \varnothing. \tag{11.42}$$

假设我们以节点 c 为条件, 如图 11.19所示, 则 a 和 b 的条件分布如下:

$$p(a,b \mid c) = \frac{p(a,b,c)}{p(c)} = \frac{p(a)\,p(b)\,p(c \mid a,b)}{p(c)}. \tag{11.43}$$

一般来说, 这不会分解为乘积 $p(a \mid c)\,p(b \mid c)$, 因此

$$a \not\perp b \mid c. \tag{11.44}$$

第 3 个示例的行为与前两个示例相反. 观察图 11.18, 节点 c 相对于从节点 a 到节点 b 的路径是**头头相接** (head-to-head) 的. 节点 c 有时也称**碰撞节点**. 当未观测到节点 c 时, 就会“阻塞”路径, 且导致 a 和 b 是独立的. 然而, 以 c 为条件会“疏通”路径, 并使 a 和 b 相互依赖.

注

(重构新增) 例 11.4与前两个例子的方向完全相反, 是 d 分离中最容易出错的一种结构. 其直觉是:c 是 a 与 b 的“共同结果”, 若不知道 c, 两个原因互不干扰; 一旦知道 c, 二者就通过“谁造成了 c”而竞争, 从而相关. 这种“以结果为条件导致原因相关”的现象称为相消解释 (explaining away), 是小节 11.2.2的主题. 数学上关键的一步是(11.40)中 $p(c \mid a,b)$ 不能分解为 $p(c \mid a)p(c \mid b)$: 若它恰好分解, 则(11.40)变成尾尾相接结构, 条件独立性的结论也会随之翻转. 因此, 图的结构与因子的形式必须匹配, 二者缺一不可.

第 3 个例子还有一个微妙之处. 首先, 我们来介绍一些术语.

定义 11.11 (后代与祖先)

如果存在一条从 x 到 y 的路径, 且这条路径上的每一步都遵循箭头的方向, 则称节点 y 是节点 x 的**后代** (descendant), 节点 x 是节点 y 的**祖先**.

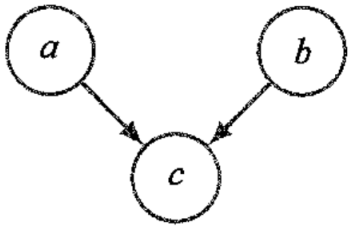


图 11.18: 第 3 个示例具有与前两个示例截然不同的性质

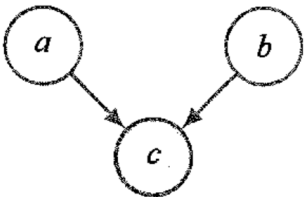


图 11.19: 与图 11.18相同, 但以节点 c 为条件, 且条件行为导致 a 和 b 之间的依赖关系

注

(重构新增) 定义 11.11把“路径”与“可达性”区分开: 路径不必沿箭头方向, 而后代关系要求每一步都沿箭头方向. 这个区分是 d 分离的关键, 因为阻塞规则只关心路径上每个节点的箭头朝向 (头对头、头对尾、尾对尾), 而后代关系决定了头对头节点被观测时路径是否被“疏通”. 由于图无环, 后代关系是严格偏序, 因此可以按拓扑序递推 (练习 11.2). 注意后代关系依赖整条路径而不只是相邻边, 所以“y 是 x 的后代”比“y 是 x 的子节点”弱得多.

然后可以证明, 如果节点或其任何后代被观测到, 则头头相接的路径就会变得畅通 (见练习 11.13).

总而言之, 在未观测到尾对尾节点或头对尾节点的情况下, 路径会变得畅通; 而一旦尾对尾节点或头对尾节点被观测到, 路径就会变得阻塞. 相比之下, 头对头节点在未被观测到时会阻塞路径, 而一旦头对头该节点和/或其至少一个后代被观测到, 路径就会变得畅通.

命题 11.6 (三种基本结构的条件独立性)

设 a 与 b 之间的路径只经过一个中间节点 c. 则

(i) 若 c 是尾尾相接或头尾相接节点 (即 $a \leftarrow c \rightarrow b$ 或 $a \rightarrow c \rightarrow b$), 则 $a \perp b \mid \varnothing$ 一般不成立, 而 $a \perp b \mid c$ 成立;

(ii) 若 c 是头头相接节点 (即 $a \rightarrow c \leftarrow b$), 则 $a \perp b \mid \varnothing$ 成立, 而 $a \perp b \mid c$ 一般不成立;

(iii) 在 (ii) 的情形下, 若以 c 的任一后代 d 为条件, 则 $a \perp b \mid d$ 一般不成立.

♠

证明

(重构新增)(i)(ii) 已分别在例 11.2、例 11.3、例 11.4中由联合分布的显式分解逐项验证.(iii) 设 c 有后代 d, 则存在有向路径 $c \rightarrow \cdots \rightarrow d$. 把该路径上的节点依次边缘化 (它们都是隐变量), 得到的 a, b, d 的联合分布仍可写成 $a \rightarrow c \leftarrow b$、$c \rightarrow \cdots \rightarrow d$ 的结构; 由 d 依赖 c、c 依赖 (a, b), 一般有 $p(a, b \mid d) \neq p(a \mid d)p(b \mid d)$.练习 11.13给出了一个具体的反例 (取 a, b 独立、c 为二者的确定性“与”、d 为 c 的含噪观测), 其中 $p(a = 1, b = 1 \mid d = 1) = 0.75$ 而 $p(a = 1 \mid d = 1)p(b = 1 \mid d = 1) = 25/36 \approx 0.694$, 故一般不独立.

□

注

(重构新增) 命题 11.6把本节三个例子压缩成一张“判定表”, 它是小节 11.2.3中一般判据的全部素材: 一条长路径只要在某个节点处满足 (i) 或 (ii) 的阻塞条件, 整条路径就被阻塞.(iii) 是唯一需要“后代”概念的地方, 也是实际建模中最容易漏掉的一条: 观测一个与隐变量相邻的下游变量, 效果等同于 (部分地) 观测该隐变量. 三条款中的“一般不成立”都应理解为“对图的几乎所有参数成立”, 而不是逻辑上的必然; 特殊参数下可能出现偶然的独立, 但那不是图结构所蕴含的结论.

11.2.2 相消解释

我们有必要花点时间进一步了解图 11.19中的有向图的异常行为. 如图 11.20所示, 考虑这样一个图, 该图对应的问题与汽车燃油系统有关的 3 个二进制随机变量. 其中, 变量 B 代表电池的状态, 即满电 ($B = 1$) 或没电 ($B = 0$); 变量 F 代表油箱的状态, 即满油 ($F = 1$) 或没油 ($F = 0$); 变量 G 代表电动燃油表的读数,$G = 1$ 表示油箱满油,$G = 0$ 表示油箱没油. 电池要么满电要么没电, 油箱要么满油要么没油, 且先验概率为

$$p(B = 1) = 0.9, \tag{11.45}$$

$$p(F = 1) = 0.9. \tag{11.46}$$

例题 11.5 汽车燃油系统 在给定油箱状态和电池状态的情况下, 电动燃油表读数为 1 的概率为

$$p(G = 1 \mid B = 1, F = 1) = 0.8, \tag{11.47}$$

$$p(G = 1 \mid B = 1, F = 0) = 0.2, \tag{11.48}$$

$$p(G = 1 \mid B = 0, F = 1) = 0.2, \tag{11.49}$$

$$p(G = 1 \mid B = 0, F = 0) = 0.1. \tag{11.50}$$

可见这是一个相当不靠谱的燃油表! 因为剩下的所有概率都由概率和为 1 这一要求决定.

在我们观测到任何数据之前, 油箱为空的先验概率为 $p(F = 0) = 0.1$. 假设我们观测燃油表并发现它的读数为零, 即 $G = 0$, 与图 11.20的中间图相对应. 我们可以使用 Bayes 定理来计算油箱为空的后验概率. 首先计算

$$p(G = 0) = \sum_{B \in \{0,1\}} \sum_{F \in \{0,1\}} p(G = 0 \mid B, F)\, p(B)\, p(F) = 0.315. \tag{11.51}$$

然后计算

$$p(G = 0 \mid F = 0) = \sum_{B \in \{0,1\}} p(G = 0 \mid B, F = 0)\, p(B) = 0.81, \tag{11.52}$$

利用这些结果, 我们得出

$$p(F = 0 \mid G = 0) = \frac{p(G = 0 \mid F = 0)\, p(F = 0)}{p(G = 0)} \approx 0.257. \tag{11.53}$$

所以 $p(F = 0 \mid G = 0) > p(F = 0)$, 从而只要测到燃油表读数为零, 就更有可能确定油箱确实是空的, 正如我们潜意识里所认为的那样. 接下来, 假设我们还检查了电池的状态并发现电池没电, 即 $B = 0$. 如图 11.20的右图所示, 我们现在已经观测到燃油表和电池的状态. 根据对燃油表状态和电池状态的观测, 油箱为空的后验概率为

$$p(F = 0 \mid G = 0, B = 0) = \frac{p(G = 0 \mid B = 0, F = 0)\, p(F = 0)}{\sum_{F \in \{0,1\}} p(G = 0 \mid B = 0, F)\, p(F)} \approx 0.111, \tag{11.54}$$

其中, 先验概率 $p(B = 0)$ 在(11.54)的分子和分母之间相互抵消. 因此, 由于观测到电池的状态, 油箱为空的概率降低了 (从 0.257 降至 0.111). 这与我们的直觉相吻合, 即电池没电可以相消解释 (explain away) 燃油表读数为零的观测结果. 通过观测燃油表的读数, 我们发现油箱和电池的状态确实是相互依赖的. 事实上, 如果我们不直接观测燃油表, 而是观测节点 G 的某个后代节点的状态, 例如一个相当不可靠的目击者说他看到燃油表的读数为零, 则情况也会如此 (见练习 11.14). 请注意, 概率 $p(F = 0 \mid G = 0, B = 0) \approx 0.111$ 大于先验概率 $p(F = 0) = 0.1$, 因为观测到燃油表读数为零仍然提供了一些支持油箱为空的证据.

注

(重构新增) 例 11.5把相消解释量化了: 三条后验概率 0.1 → 0.257 → 0.111 的走势说明, 证据的方向可以因新证据而反转. 中间一步 $p(F=0 \mid G=0) \approx 0.257$ 用到了(11.52)中“先对 B 求和”的边缘化, 这一步不能与 $p(F=0 \mid G=0, B=0)$ 混淆: 前者是对电池状态不确定, 后者是已知电池没电. 数值上两个后验都大于先验 0.1, 但后者小得多, 因为电池没电本身就能解释低读数, 从而削弱了“油箱空”这一解释的必要性. 这个例子还预告了小节 11.2.6的结论:F 与 B 在给定 G 后并不独立 (二者通过碰撞节点相关), 因此“条件独立”不能想当然地由“边缘独立”推出.

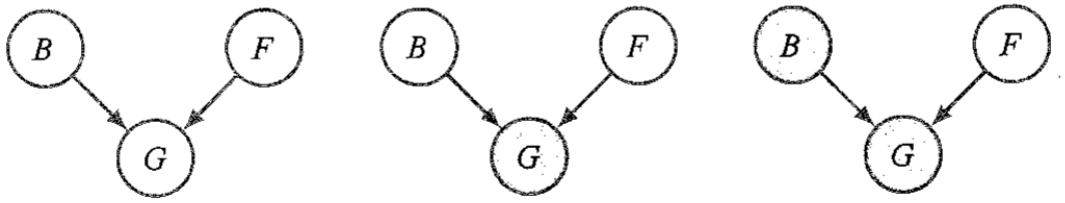


图 11.20: 用于说明相消解释的 3 节点图的示例.3 个节点分别代表电池的状态 (B)、油箱的状态 (F) 和电动燃油表的读数 (G)

11.2.3 d 分离

本小节将对有向图的 d 分离特性 (Pearl, 1988) 进行一般性阐述. 考虑一个有向图, 其中 **A**、**B**、**C** 是任意不相交的节点集 (但它们的并集可能小于图中的完整节点集). 我们希望确定给定的有向无环图是否隐含特定的条件独立性 $\mathbf{A} \perp \mathbf{B} \mid \mathbf{C}$. 为此, 我们需要考虑从 **A** 中任何节点到 **B** 中任何节点的所有可能路径.

定义 11.12 (阻塞与 d 分离)

设 **C** 是已观测节点的集合,π 是一条路径. 若 π 上存在节点 w 满足下列两条之一, 则称 π 被 **C 阻塞**:

(1) 路径上的箭头在 w 处头尾相接或尾尾相接, 且 $w \in \mathbf{C}$;

(2) 箭头在 w 处头头相接, 且 w 及其任何后代节点都不在 **C** 中.

若从 **A** 中任何节点到 **B** 中任何节点的所有路径都被 **C** 阻塞, 则称 **A** 与 **B** 被 **Cd 分离** (d-separated).

注

(重构新增) 定义 11.12把前三个例子的判定规则组装成一条可机械执行的算法: 枚举路径、检查每个中间节点的箭头朝向与该节点是否在 **C** 中. 它只依赖图的拓扑, 不依赖任何数值, 因此同一张图对所有满足该分解的分布给出同一组结论. 定义中“路径”指忽略箭头方向的连通序列 (允许重复节点但通常只考虑简单路径); 若某条路径本身以 **A** 或 **B** 中的节点为中间节点, 则按同样规则判断. 注意 (1) 与 (2) 是互补的两种情况: 一个节点要么不是碰撞节点 (此时只要它在 **C** 中就阻塞), 要么是碰撞节点 (此时只有它及其后代都不在 **C** 中才阻塞).小节 11.2.3的定理 11.1将说明, 这种纯图形的判定与“分解式蕴含的条件独立性”完全等价.

定理 11.1 (d 分离判定)

设 $p(\mathbf{x})$ 由有向无环图 G 按(11.8)分解, 且 p 对每个变量取值的组合都为正. 若 **A** 与 **B** 被 **C**d 分离, 则

$$\mathbf{A} \perp \mathbf{B} \mid \mathbf{C} \tag{11.55}$$

对 G 所表示的任何分布成立. 反之, 若 **A** 与 **B** 不被 **C**d 分离, 则存在满足该分解的分布使(11.55)不成立.

证明

(重构新增) 这里只给出正面方向的论证思路 (完整的形式证明见 Lauritzen (1996)). 把 G 按定义 11.3改写成因子图, 并把 $\mathbf{C}$ 中的节点固定. 对任一未被阻塞的路径, 逐点按定义 11.12的条件分类: 若路径上存在满足条件 (1) 的节点, 则该节点在 $\mathbf{C}$ 中, 路径被切断; 若存在满足条件 (2) 的节点, 则该节点是碰撞节点且其所有后代都不在 $\mathbf{C}$ 中, 路径在该处被“堵住”. 因此只要所有路径都被阻塞,$\mathbf{A}$ 与 $\mathbf{B}$ 之间就不存在既不被观测、又不被碰撞节点堵住的“有效通道”. 反方向: 对每条未被阻塞的路径, 可以构造一组使该路径产生相关的参数 (路径上每个非碰撞节点都取非平凡的条件分布, 碰撞节点取非平凡的条件分布), 由这些路径的独立性可逐条叠加, 得到使 $\mathbf{A}$ 与 $\mathbf{B}$ 相关的分布. □

注

(重构新增) 定理 11.1是本章的核心结论, 通常称为 d 分离定理或 Pearl 的因果图定理. 它的价值在于把“验证条件独立性”这一需要做高维积分的分析问题, 降为一个多项式时间的图论问题 (路径枚举可用可达性算法实现). 两个方向的意义不同: 正面方向是“图蕴含独立”, 对所有满足分解的分布成立, 无需任何数值信息; 反面方向是“图不蕴含独立”, 只保证存在反例, 并不排除某些特殊参数下仍然独立——这正是例 11.2中“特定数值下也可能独立”的严格表述. 定理要求分布严格为正, 否则某些条件概率无定义, 需要用极限或“几乎处处”的语言修补.

d 分离法如图 11.21所示. 在图 11.21(a) 中, 从 a 到 b 的路径既没有被节点 f 阻塞, 因为节点 f 是这条路径的尾对尾节点, 且未被观测到; 也没有被节点 e 阻塞, 因为尽管节点 e 是头对头节点, 但它在条件集中只有一个后代节点 c. 因此, 条件独立性 $\mathbf{A} \perp \mathbf{B} \mid \mathbf{C}$ 从该图中无法得出. 在图 11.21(b) 中, 从 a 到 b 的路径被节点 f 阻塞, 因为这是一个被观测到的尾对尾节点, 所以根据该图进行分解的任何分布都将满足条件独立性 $a \perp b \mid f$. 请注意, 这条路径也被节点 e 阻塞了, 因为节点 e 是一个头对头节点, 节点 e 及其子节点都不在条件集合中. 在 d 分离中,图 11.12中以浮动变量表示的 λ 等参数的行为与观测节点的行为相同. 然而, 这些节点不存在边缘分布, 因此参数节点本身永远不会有父节点, 经过这些节点的所有路径永远是尾尾相接的, 因此会被阻塞. 这些节点在 d 分离中不起作用.

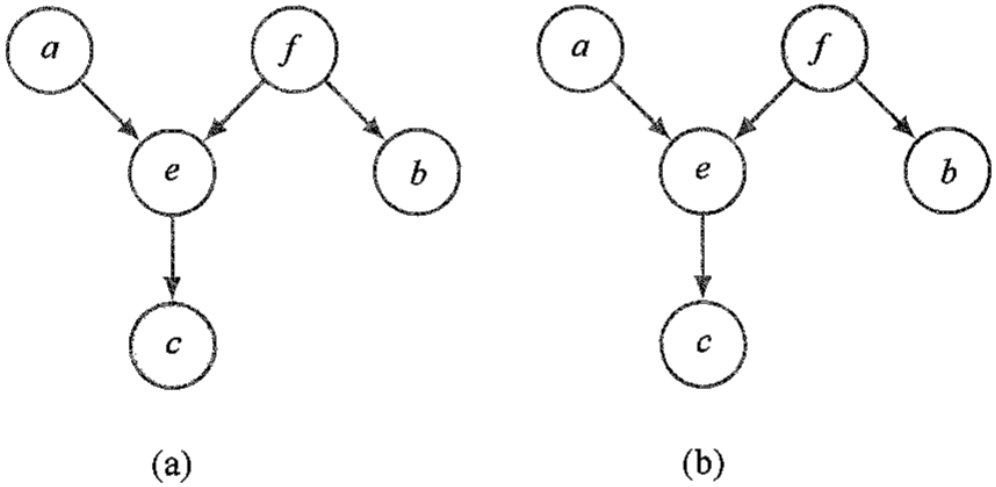


图 11.21: d 分离示意图

独立同分布数据是条件独立性和 d 分离的另一个示例 (参见小节 2.3.2). 观察图 11.12所示的二元分类模型. 这里的随机节点分别对应 t_n、$\mathbf{w}$ 和 $\hat{t}$. 我们可以看到, 对于从 $\hat{t}$ 到任何一个节点 t_n 的路径来说,$\mathbf{w}$ 的节点是尾对尾的, 因此我们具有以下条件独立性:

$$\hat{t} \perp t_n \mid \mathbf{w}. \tag{11.56}$$

命题 11.7 (以参数为条件的预测)

在(11.24)的模型中,$t_1, \cdots, t_N$ 在给定 $\mathbf{w}$ 后条件独立, 且预测分布满足

$$p(\hat{t} \mid \hat{\mathbf{x}}, \mathbf{w}, \mathbf{X}, \mathbf{t}) = p(\hat{t} \mid \hat{\mathbf{x}}, \mathbf{w}), \tag{11.57}$$

即以网络参数 $\mathbf{w}$ 为条件,$\hat{t}$ 的预测分布与训练数据 $\{t_1,\cdots,t_N\}$ 无关.

♠

证明

(重构新增) 由(11.24), 给定 $\mathbf{w}$ 后联合分布分解为 $p(\hat{t}\mid\mathbf{w},\hat{\mathbf{x}})\prod_n p(t_n\mid\mathbf{w},\mathbf{x}_n)$, 其中 $\hat{t}$ 的因子与 $t_1,\cdots,t_N$ 的因子相乘而无交互项, 故 $t_1,\cdots,t_N$ 条件独立, 且 $\hat{t}$ 与它们条件独立 (这正是(11.56)). 于是

$$p(\hat{t}\mid\hat{\mathbf{x}},\mathbf{w},\mathbf{X},\mathbf{t})=\frac{p(\hat{t},\mathbf{t}\mid\mathbf{w},\hat{\mathbf{x}},\mathbf{X})}{p(\mathbf{t}\mid\mathbf{w},\mathbf{X})}=\frac{p(\hat{t}\mid\mathbf{w},\hat{\mathbf{x}})\prod_n p(t_n\mid\mathbf{w},\mathbf{x}_n)}{\prod_n p(t_n\mid\mathbf{w},\mathbf{x}_n)}=p(\hat{t}\mid\mathbf{w},\hat{\mathbf{x}}). \tag{11.58}$$

□

注

(重构新增) 命题 11.7是“训练完就可以丢掉训练数据”这一工程实践的概率依据: 以参数为条件时, 训练集不再提供额外信息. 但它有一个容易被误用的前提——$\mathbf{w}$ 被视为已知的固定值. 若对 $\mathbf{w}$ 做 Bayes 处理并积分掉, 则 $\hat{t}$ 与训练数据通过 $\mathbf{w}$ 的后验仍然相关, 预测分布变成 $p(\hat{t}\mid\hat{\mathbf{x}},\mathbf{X},\mathbf{t})=\int p(\hat{t}\mid\hat{\mathbf{x}},\mathbf{w})p(\mathbf{w}\mid\mathbf{X},\mathbf{t})\,\mathrm{d}\mathbf{w}$, 此时“丢掉训练数据”就必须连同后验一起保留. 这一区别正是小节 11.1.6最后关于完全 Bayes 与 MAP 近似的讨论的伏笔.

因此, 以网络参数 $\mathbf{w}$ 为条件,$\hat{t}$ 的预测分布与训练数据 $\{t_1,\cdots,t_N,\mathbf{x}_n\}$ 无关. 我们可以首先使用训练数据来确定 $\mathbf{w}$ 的后验分布 (或后验分布的近似值), 然后丢弃训练数据, 并使用 $\mathbf{w}$ 的后验分布来预测新输入的观测结果中的 $\hat{t}$.

11.2.4 朴素 Bayes

有一种相关的图结构出现在朴素 Bayes(Naive Bayes) 模型中, 其中我们使用条件独立性假设来简化模型结构. 假设我们的数据由向量 $\mathbf{x}$ 的观测值组成, 我们希望将 $\mathbf{x}$ 的观测值归入 K 个类别中的一个类别 (后文简称类). 我们可以为每个类定义类-条件密度 $p(\mathbf{x}\mid C_k)$ 以及先验类概率 $p(C_k)$. 朴素 Bayes 模型的关键假设是, 以类 C_k 为条件, 将输入变量的分布分解为两个或多个密度的乘积.

定义 11.13 (朴素 Bayes 模型)

设输入向量被分成 L 个元素, 即 $\mathbf{x}=(x^{(1)},\cdots,x^{(L)})$. 若以类 C_k 为条件, 输入变量的分布分解为

$$p(\mathbf{x}\mid C_k)=\prod_{l=1}^{L}p\left(x^{(l)}\mid C_k\right), \tag{11.59}$$

则称该模型为**朴素 Bayes 模型** (Naive Bayes model). 假设(11.59)分别对每个类 C_k 成立.

♣

注

(重构新增) 定义 11.13的假设等价于说: 在给定类标签后,L 个特征彼此条件独立. 它把 L 维联合密度的估计问题化为 L 个一维密度估计问题, 参数数量从指数级降到线性级, 这是它在高维输入下仍然可用的原因. 假设的代价是它无法表示特征之间的交互: 若真实数据中两个特征高度相关 (例如图像的相邻像素),(11.59)会严重失真. 但需要注意, 分类决策只依赖后验 $p(C_k\mid\mathbf{x})$ 的相对大小, 而决策边界对类-条件密度的某些细节并不敏感, 因此即使假设不成立, 分类精度也可能保持得不错——这是“模型错但决策对”的典型例子.

图 11.22是该模型的图形化表示. 我们可以看到, 在 $j\neq i$ 的情况下, 对 C_k 的观测会阻塞 $x^{(i)}$ 和 $x^{(j)}$ 之间的路径, 因为这种路径在节点 C_k 处是尾尾相接的, 所以当给定 C_k 时,$x^{(i)}$ 和 $x^{(j)}$ 是条件独立的. 然而, 如果我们将 C_k 去边缘化, 从 $x^{(i)}$ 到 $x^{(j)}$ 的尾尾相接路径将不再受阻. 所以一般来说, 边缘密度 $p(\mathbf{x})$ 不会相对于元素 $x^{(1)},\cdots,x^{(L)}$ 进行分解.

如果我们得到一个标记训练集, 其中包括观测值 $\{\mathbf{x}_1,\cdots,\mathbf{x}_N\}$ 以及它们的类标签, 我们就可以假设数据是从模型中独立提取的, 从而使用最大似然法将朴素 Bayes 模型拟合到训练数据中 (见练习 11.15). 通过使用相应的标记数据分别拟合每个类的模型, 然后将先验类概率 $p(C_k)$ 设置为等于每个类中训练数据点的比例, 从而得到解决

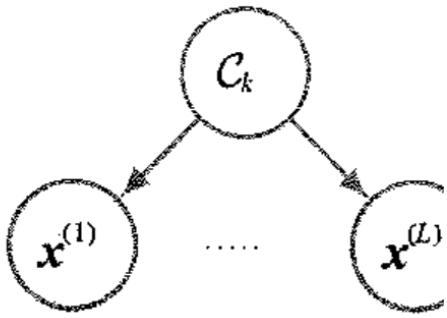


图 11.22: 用于分类的朴素 Bayes 模型的图示. 以类标签 C_k 为条件, 假设观测向量 $\mathbf{x} = (x^{(1)}, \cdots, x^{(L)})$ 中的元素是独立的

方案. 向量 $\mathbf{x}$ 属于类 C_k 的概率由 Bayes 定理给出, 其形式为

$$p(C_k \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid C_k)\, p(C_k)}{p(\mathbf{x})}, \tag{11.60}$$

其中 $p(\mathbf{x} \mid C_k)$ 由(11.59)给出,$p(\mathbf{x})$ 可以用下式求出:

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(\mathbf{x} \mid C_k)\, p(C_k). \tag{11.61}$$

命题 11.8 (朴素 Bayes 的分类后验)

在定义 11.13的假设下, 类后验满足

$$p(C_k \mid \mathbf{x}) = \frac{p(C_k) \prod_{l=1}^{L} p\left(x^{(l)} \mid C_k\right)}{\sum_{j=1}^{K} p(C_j) \prod_{l=1}^{L} p\left(x^{(l)} \mid C_j\right)}, \tag{11.62}$$

因此分类决策只需比较各分子, 无须计算分母.

证明

(重构新增) 把(11.59)代入(11.60), 再用(11.61)与(11.59)展开分母:

$$p(\mathbf{x}) = \sum_{j=1}^{K} p(\mathbf{x} \mid C_j) p(C_j) = \sum_{j=1}^{K} p(C_j) \prod_{l=1}^{L} p\left(x^{(l)} \mid C_j\right). \tag{11.63}$$

分母与 k 无关, 故分类时只需比较分子 $p(C_k) \prod_{l} p(x^{(l)} \mid C_k)$.

□

注

(重构新增) 命题 11.8的实用价值在于“分母可省”: 分类只依赖分子的大小顺序, 这既省计算也避免了下溢. 但省略分母的代价是无法得到校准的概率——若需要置信度或拒绝选项 (参见小节 5.2.3), 必须计算(11.62)的完整分母. 此外,(11.62)显示后验是各特征“证据”的乘积再乘以先验, 因此单个特征若给出零密度, 整个后验会塌缩为零; 实践中通常对每个特征密度做平滑 (加性平滑) 以避免这一退化.

用于二维数据空间的朴素 Bayes 模型如图 11.23所示, 其中 $\mathbf{x} = (x_1, x_2)$. 此处, 我们假设两个类中每个类的类-条件密度 $p(\mathbf{x} \mid C_k)$ 都是轴对齐的 Gauss 密度, 因此它们各自将对 x_1 和 x_2 进行分解, 从而

$$p(\mathbf{x} \mid C_k) = p(x_1 \mid C_k)\, p(x_2 \mid C_k). \tag{11.64}$$

然而, 边缘密度 $p(\mathbf{x})$ 由下式给出:

$$p(\mathbf{x}) = \sum_{k=1}^{K} p(\mathbf{x} \mid C_k)\, p(C_k), \tag{11.65}$$

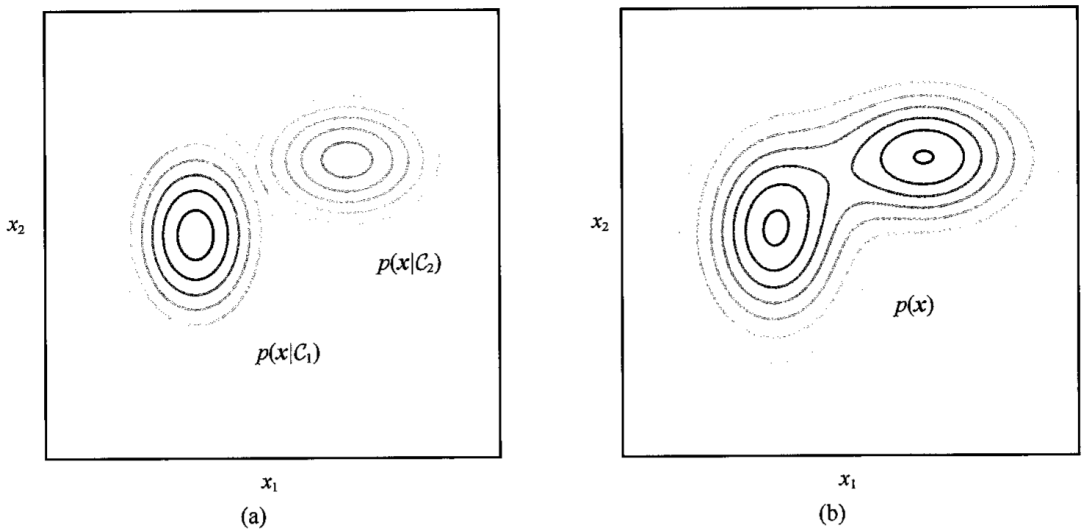


图 11.23: (a) 两个类中每个类的类-条件密度 $p(\mathbf{x} \mid C_k)$;(b) 边缘密度 $p(\mathbf{x})$, 其中我们假设了相等的类先验概率 $p(C_1) = p(C_2) = 0.5$. 请注意, 条件密度会对 x_1 和 x_2 进行分解, 而边缘密度不会

它现在是 Gauss 混合模型且不会对 x_1 和 x_2 进行分解.

注

(重构新增) 图 11.23用等值线直观地说明了一个容易混淆的事实: 条件密度分解, 不代表边缘密度分解.图 11.23(a) 中每个类的等值线是轴对齐的椭圆 (因为(11.64)把协方差限制为对角), 而图 11.23(b) 中的混合分布等值线是倾斜的“花生形”, 因为把 C 边缘化后两个分量叠加产生了相关性. 这与例 11.2是同一现象: 尾尾相接的路径在 C 未观测时把两个特征耦合起来. 因此, 若把朴素 Bayes 模型用于密度估计 (而不仅是分类), 得到的 $p(\mathbf{x})$ 并不是“各特征独立”的分布.

在融合不同来源的数据 (如用于医疗诊断的血液检测和皮肤图像) 时, 将会涉及朴素 Bayes 模型的简单应用 (参见小节 5.2.4). 当输入空间的维度 D 较高时, 在全 D 维空间中进行密度估计将更具挑战性, 朴素 Bayes 假设会有所帮助. 如果输入向量同时包含离散变量和连续变量, 这种方法也会很有用, 因为每种变量都可以用适当的模型来表示 (例如, 对于二元观测可以使用 Bernoulli 分布, 对于实值变量可以使用 Gauss 分布). 该模型的条件独立性假设显然是强假设, 可能会导致对类-条件密度的表示相当糟糕. 然而, 即使这一假设没有得到精确的满足, 模型在实际应用中也仍然可以提供良好的分类性能, 因为决策边界可能对类-条件密度中的某些细节不敏感.

11.2.5 生成式模型

机器学习的许多应用都可以看作逆问题 (inverse problem) 的示例, 其中有一个产生数据的底层过程 (通常是物理过程), 我们现在的目标是学习如何反转这个过程. 例如, 一个对象的图像可以看作一个生成过程的输出, 在这个生成过程中, 对象的类型是从一些可能的对象类分布中选择出来的. 对象的位置和方向也从一些先验分布中选择, 然后创建图像. 在给定一个大型图像数据集的情况下, 其中的图像已标记所含对象的类型、位置和大小, 我们的目标是训练一个机器学习模型, 该模型可以获取新的、未标记的图像, 并检测对象是否存在, 以及对象在图像中的位置和大小. 因此, 机器学习解决方案代表了数据生成过程的逆过程.

一种方法是训练深度神经网络, 如卷积网络, 从而将图像作为输入并生成描述对象的类型、位置和大小的输出. 因此, 作为判别模型的一个示例, 这种方法试图直接解决逆问题. 只要有足够多的标记图像, 这种方法就能达到很高的准确率. 在实践中, 未标记的图像往往很多, 而在获得训练集的过程中, 大部分工作量耗费在证明标记上, 这个证明过程可能需要手动完成. 我们的判别模型在训练过程中无法直接使用未标记的图像.

定义 11.14 (判别模型与生成式模型)

设 $\mathbf{x}$ 为观测,y 为待预测的目标 (如类标签、位置或大小). 若模型直接对条件分布 $p(y \mid \mathbf{x})$ 建模, 则称其为**判别模型** (discriminative model); 若模型先对联合分布 $p(\mathbf{x}, y)$(或等价的生成过程 $p(y)$ 与 $p(\mathbf{x} \mid y)$) 建模, 再通过 Bayes 定理得到 $p(y \mid \mathbf{x})$, 则称其为**生成式模型** (generative model). 生成式模型训练完成后可以从中采样合成新数据.

♣

注

(重构新增) 定义 11.14的区分标准是“是否对输入的分布建模”, 而不是“是否用神经网络”. 判别模型把建模能力集中在决策边界上, 样本效率高但无法生成数据; 生成式模型需要刻画 $p(\mathbf{x} \mid y)$, 任务更难, 但可以利用未标记数据, 并且能回答“这个样本是否合理”之类的问题. 图中的差别同样明显: 判别模型只有从输入指向输出的箭头, 生成式模型的箭头方向与数据的生成步骤一致 (图 11.24), 因此观测到数据后可以用定理 11.1读出“未观测时哪些隐变量独立、观测后又如何变得相关”. 需要注意, 生成式与判别式并非互斥: 同一个联合分布既可以写成生成式分解, 也可以写成判别式分解 (命题 11.4的箭头反转).

另一种方法是对生成过程进行建模, 然后进行计算反转. 在我们的图像示例中, 如果假设对象的类、位置和大小都是独立选择的, 则可以使用有向图模型来表示生成过程, 如图 11.24所示. 请注意, 箭头的方向与生成步骤的顺序相对应, 因此该模型代表了观测数据生成的因果过程 (Pearl, 1988). 这是生成式模型的一个示例, 因为一旦经过训练, 它就可以用来生成合成图像, 方法是首先从学习到的先验分布中选择对象的类、位置和大小, 然后从学习到的条件分布中抽取图像样本. 稍后我们将看到扩散模型和其他生成式模型如何根据对所需图像内容和风格的文字描述, 合成令人赞叹的高分辨率图像 (参见第 20 章).

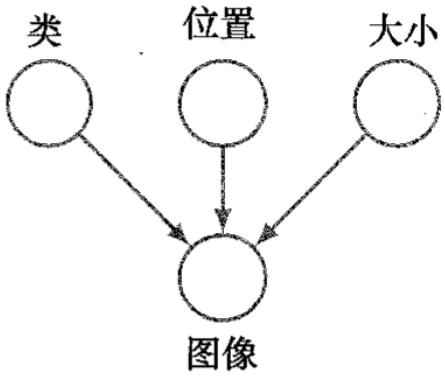


图 11.24: 表示创建对象图像过程的图模型. 对象所属的类 (离散变量) 与对象的位置和方向 (连续变量) 具有独立的先验概率. 图像 (像素强度阵列) 的概率分布取决于对象所属的类及其位置和方向

根据图 11.24中的图模型, 在未观测到图像时, 类、位置和大小变量是独立的. 这是因为这些变量中的任何两个变量之间的每条路径就图像变量而言是头头相接的, 这是未观测到的. 然而, 当我们观测到一幅图像时, 这些路径就会变得畅通, 类、位置和大小变量也不再独立. 从直觉上讲, 这是合理的, 因为被告知图像中对象所属的类可以为我们提供相关信息, 从而帮助我们确定对象的位置和大小.

然而, 概率模型中的隐变量并不需要有任何明确的物理解释, 而引入这些隐变量可能只是为了从更简单的组件中构造出更复杂的联合分布. 例如, 归一化流、变分自编码器和扩散模型等都使用深度神经网络, 从而通过转换具有简单 Gauss 分布的隐变量, 在数据空间中创建复杂的分布.

11.2.6 Markov 毯

在讨论更复杂的有向图时, 一个很有用的条件独立性称为 Markov 毯 (Markov blanket) 或 Markov 边界 (Markov boundary). 考虑由具有 D 个节点的有向图表示的联合分布 $p(x_1, \cdots, x_D)$, 并考虑以所有其余变量 $x_{j \neq i}$ 为条件的变

量 x_i 的特定节点的条件分布. 利用分解性质 [(11.8)], 我们可以将这一条件分布表示为如下形式:

$$p(x_i \mid x_{j\neq i}) = \frac{p(x_1,\cdots,x_D)}{\int p(x_1,\cdots,x_D)\,\mathrm{d}x_i} = \frac{\prod_k p(x_k \mid \mathrm{pa}(k))}{\int \prod_k p(x_k \mid \mathrm{pa}(k))\,\mathrm{d}x_i},$$

其中积分被离散变量的求和取代. 现在我们注意到, 任何与 x_i 无函数相关的因子 $p(x_k \mid \mathrm{pa}(k))$ 都可以在对 x_i 的积分之外取值, 因此分子和分母之间会抵消. 剩下的因子就是节点 x_i 本身的条件分布 $p(x_i \mid \mathrm{pa}(i))$, 以及任何其他节点 x_k 的条件分布, 从而节点 x_i 在 $p(x_k \mid \mathrm{pa}(k))$ 的条件集中, 即节点 x_i 是 x_k 的父节点. 条件分布 $p(x_i \mid \mathrm{pa}(i))$ 将取决于节点 x_i 的父节点, 而条件分布 $p(x_k \mid \mathrm{pa}(k))$ 将取决于 x_k 的子节点和共父节点 [即节点 x_k 的父节点 (节点 x_i 除外) 所对应的变量].

定义 11.15 (Markov 毯)

由父节点、子节点和共父节点组成的节点集称为节点 x_i 的 **Markov 毯** (Markov blanket), 即

$$\mathrm{MB}(i) = \mathrm{pa}(i) \cup \mathrm{ch}(i) \cup \mathrm{cp}(i), \tag{11.66}$$

其中 $\mathrm{ch}(i)$ 是 x_i 的子节点集,$\mathrm{cp}(i) = \bigcup_{k\in\mathrm{ch}(i)} \big(\mathrm{pa}(k) \setminus \{i\}\big)$ 是 x_i 的共父节点集, 如图 11.25所示.

♣

注

(重构新增) 定义 11.15中“共父节点”这一项最容易被遗漏, 而它恰恰是相消解释 (例 11.5) 的图形化表述: 若 x_i 与另一个父节点共同指向同一个子节点, 则观测到该子节点会使二者相关, 因此必须把共父节点一并纳入条件集.Markov 毯可以理解为“把 x_i 与图中其余节点隔离开来的最小节点集”, 它在图上是一个局部对象 (只涉及距离 x_i 不超过两步的节点), 却在概率上刻画了完整的条件分布, 这一“局部性”是 Markov 随机场与信念传播算法的基石.

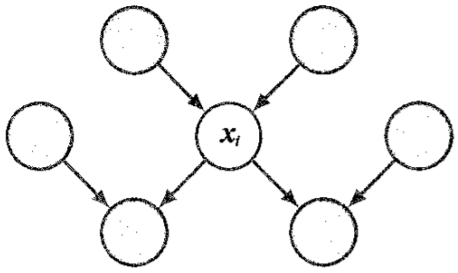


图 11.25: 节点 x_i 的 Markov 毯由该节点的父节点、子节点和共父节点组成. 它具有这样一个特性, 即节点 x_i 的条件分布 (以图中所有其余节点为条件) 仅取决于 Markov 毯中的变量

命题 11.9 (Markov 毯的分离性质)

设 $p(\mathbf{x})$ 由有向无环图按(11.8)分解. 则对任意 $j \notin \mathrm{MB}(i) \cup \{i\}$,

$$x_i \perp x_j \mid \mathbf{x}_{\mathrm{MB}(i)}, \tag{11.67}$$

即 x_i 在给定 Markov 毯后的条件分布与图中其余变量无关.

♠

证明

(重构新增) 用定理 11.1, 只需证明从 x_i 到任意 $j \notin \mathrm{MB}(i) \cup \{i\}$ 的每条路径都被 $\mathbf{C} = \mathbf{x}_{\mathrm{MB}(i)}$ 阻塞. 设路径为 $x_i = v_0, v_1, \cdots, v_m = j$.

先看第一步. 若 $v_0 \to v_1$, 则 $v_1 \in \mathrm{ch}(i) \subseteq \mathbf{C}$; 若 $v_1 \to v_0$, 则 $v_1 \in \mathrm{pa}(i) \subseteq \mathbf{C}$. 两种情形都有 $v_1 \in \mathbf{C}$, 且 $v_1 \neq j$(因 $j \notin \mathrm{MB}(i)$). 在 v_1 处, 路径的另一条边有两种可能: 若为 $v_2 \to v_1$(即 v_1 是头头相接), 则 $v_2 \in \mathrm{cp}(i) \cup \mathrm{pa}(i) \subseteq \mathbf{C}$, 且 $v_2 \neq j$; 若为 $v_1 \to v_2$(v_1 是头尾相接或尾尾相接), 则 $v_1 \in \mathbf{C}$ 直接使路径被阻塞. 故只需处理 $v_2 \in \mathbf{C}$ 的情形, 此时在 v_1 处路径是头头相接的, 尚未被阻塞.

于是路径可写为 $x_i \to v_1 \leftarrow v_2$ 或 $v_2 \to v_1 \leftarrow x_i$, 其中 $v_1 \in \mathbf{C}$、$v_2 \in \mathbf{C}$. 在 v_2 处, 路径经过的边是 $v_2 \to v_1$(出

边). 若 $m=2$, 则 $j=v_2\in\mathbf{C}$, 与 $j\notin \mathrm{MB}(i)$ 矛盾, 故 $m\geqslant 3$. 此时 v_2 处的另一条边为 $v_3\to v_2$ 或 $v_2\to v_3$, 两种情形下 v_2 都不是头头相接节点, 而 $v_2\in\mathbf{C}$, 故路径在 v_2 处被阻塞.

综上, 所有路径都被 $\mathbf{C}$ 阻塞, 由定理 11.1得(11.67).

□

注

(重构新增) 命题 11.9的证明只用到两步就完成了“局部性”的论证, 关键在于路径在离开 x_i 后的前两个节点必然落在父/子/共父之中. 这也说明 Markov 毯不能缩小: 若去掉共父节点,例 11.4的结构就会使路径在子节点处重新疏通; 若去掉子节点, 则父节点与子节点之间的依赖无法被条件化. 命题的实用形式是 Gibbs 采样的“全条件”:$p(x_i\mid \mathbf{x}_{-i})$ 只需用 Markov 毯中的变量计算, 因此每一次单变量更新都是局部的, 这正是第 14 章中 Gibbs 采样高效的原因.

我们可以考虑把节点 x_i 的 Markov 毯看作能将节点 x_i 与图中的其他节点隔离开来的最小节点集. 请注意, 仅包含节点 x_i 的父节点和子节点是不够的, 因为相消解释意味着观测到子节点不会阻塞通往共父节点的路径. 因此, 我们还必须观测到共父节点.

11.2.7 作为过滤器的图

我们已经看到, 一个特定的有向图不仅代表了将联合概率分布分解为条件概率的乘积, 还代表了通过 d 分离准则获得的一组条件独立性.d 分离定理实际上表达了这两个特性的等价性. 为了说明这一点, 不妨把有向图看作一个过滤器. 假设我们考虑的是图中 (未观测到的) 节点所对应的变量 $\mathbf{x}$ 的特定联合概率分布 $p(\mathbf{x})$. 只有当且仅当该分布可以用图所隐含的分解 [(11.8)] 表示时, 过滤器才会允许该分布通过. 将变量 $\mathbf{x}$ 上所有可能的分布 $p(\mathbf{x})$ 的集合提交给过滤器, 对于有向分解 (directed factorization), 将过滤器允许通过的分布子集记为 $\mathcal{D}_F$, 如图 11.26所示.

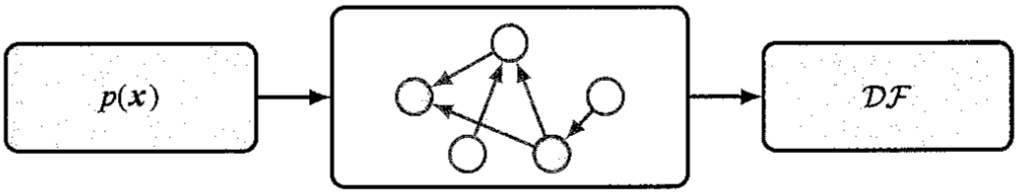


图 11.26: 我们可以把图模型 (此处为有向图模型) 看作一种过滤器, 其中的概率分布 $p(\mathbf{x})$ 只有在满足有向分解特性时才能通过过滤器. 将通过过滤器的所有可能概率分布 $p(\mathbf{x})$ 的集合记为 $\mathcal{D}_F$. 我们也可以根据是否满足图的 d 分离特性所隐含的所有条件独立性, 使用图 (作为第二种过滤器) 来过滤分布.d 分离定理表明, 同样的一组分布 $\mathcal{D}_F$ 将被允许通过第二种过滤器

我们也可以将图用作另一种过滤器, 首先列出通过对图应用 d 分离准则而得到的所有条件独立性, 然后只允许满足所有这些条件独立性的分布通过. 如果我们将所有可能的分布 $p(\mathbf{x})$ 呈现给第二种过滤器, 则 d 分离定理告诉我们, 允许通过的分布集合正是集合 $\mathcal{D}_F$.

定理 11.2 (d 分离定理的过滤器形式)

设 G 是定义在 $\mathbf{x}$ 上的有向无环图. 记

$$\mathcal{D}_F=\left\{p:\ p(\mathbf{x})=\textstyle\prod_k p(x_k\mid \mathrm{pa}(k))\ \text{对某组条件分布}\right\},\tag{11.68}$$

$$\mathcal{D}_I=\left\{p:\ p\ \text{满足}\ G\ \text{经 d 分离得到的全部条件独立性}\right\}.\tag{11.69}$$

则 $\mathcal{D}_F=\mathcal{D}_I$.

♡

证明

(重构新增) 两个包含关系分别对应定理 11.1的两个方向.$\mathcal{D}_F \subseteq \mathcal{D}_I$: 由该定理的正面方向, 任何按(11.8)分解的分布都满足该图经 d 分离得到的全部条件独立性.

$\mathcal{D}_I \subseteq \mathcal{D}_F$: 关键是“有序 Markov 性质”——对每个节点 k,

$$x_k \perp \{\text{非后代}\} \setminus \mathrm{pa}(k) \mid \mathrm{pa}(k), \tag{11.70}$$

它是 G 经 d 分离蕴含的条件独立性 (从 k 到非后代的任何路径必在某个父节点处以尾尾或头尾方式被 $\mathrm{pa}(k)$ 阻塞). 若 p 满足 G 经 d 分离得到的全部条件独立性, 则 p 满足这一性质; 由乘积法则按拓扑序逐次应用命题 11.5, 得

$$p(\mathbf{x}) = \prod_{k=1}^{K} p\big(x_k \mid \mathrm{pa}(k)\big), \tag{11.71}$$

即 $p \in \mathcal{D}_F$. 含有序 Markov 性质与 d 分离等价性的完整形式证明见 Lauritzen (1996), 与正文的说明一致.

□

注

(重构新增) 定理 11.2是本节的收束: 它说“按图分解”与“满足图的全部条件独立性”是同一组分布的两种刻画, 因此图既是分解的模板, 也是独立性的生成器. 正因为两个集合相等, 我们才能在建模时自由选择更方便的一侧: 想设计模型就从分解出发 (写出局部因子), 想检验模型就从独立性出发 (检查 d 分离). 需要强调的是,$\mathcal{D}_F$ 只包含“至少满足这些独立性”的分布, 因此一个完全分解的分布总能通过任何图所隐含的过滤器; 反过来, 图越稠密,$\mathcal{D}_F$ 越大, 全连接图的 $\mathcal{D}_F$ 就是全部分布.

需要强调的是, 从 d 分离得到的条件独立性适用于该特定有向图所描述的任何概率模型. 例如, 无论变量是离散的还是连续的, 抑或是这些变量的组合, 情况都是如此. 在一种极端情况下, 我们有一个全连接图, 它完全不显示条件独立性, 并且可以表示给定变量上任何可能的联合概率分布, 集合 $\mathcal{D}_F$ 将包含所有可能的分布 $p(\mathbf{x})$. 在另一种极端情况下, 我们有一个全不连接图, 即不包含任何连接的图. 这相当于联合分布可以分解为组成图节点的变量的边缘分布的乘积. 请注意, 对于任何给定的图, 集合 $\mathcal{D}_F$ 都将包括除图所描述的分布外, 还具有其他独立特性的任何分布. 例如, 一个全分解分布总能通过相应变量集上的任何图所隐含的过滤器.

11.3 序列模型

机器学习有许多重要的应用, 其中的数据都由一系列的值组成, 称为序列数据. 例如, 文本由一系列单词组成, 而蛋白质则由一系列氨基酸组成. 序列数据通常是按时间排序的, 例如麦克风的音频信号或特定地点的日降雨量. 有时, “时间”“过去”“未来”等术语也可以用于描述其他类型的序列数据, 而不仅仅是时间序列. 涉及序列数据的应用包括语音识别、不同语言间的自动翻译、DNA 中的基因检测、合成音乐、编写计算机代码、与现代搜索引擎进行对话等.

我们用 $x_1, \cdots, x_N$ 表示序列数据, 其中的每个元素 x_n 都包含一个值向量. 请注意, 我们可能会有好几个这样的序列数据 (后文简称序列), 它们分别独立地取自同一个分布. 在这种情况下, 所有序列的联合分布可以分解为每个序列分布的乘积. 由此, 我们可以仅关注其中一个序列的建模.

根据(11.6), 我们已经看到, 通过反复应用概率的乘积法则,N 个变量的一般分布可以写成条件分布的乘积, 而这种分解的形式取决于变量的特定排序. 对于向量值变量, 如果我们选择与序列中变量顺序相对应的排序, 则可以写出

$$p(x_1, \cdots, x_N) = \prod_{n=1}^{N} p(x_n \mid x_1, \cdots, x_{n-1}), \tag{11.72}$$

这对应于一个有向图, 其中每个节点都从序列中的每个前一节点接收一个连接, 如图 11.27中的 4 个节点所示.

定义 11.16 (自回归模型)

若序列的联合分布按(11.72)分解, 即每个条件分布以序列中它之前的全部变量为条件, 则称该模型为**自回归** (autoregressive) 模型. 由命题 11.1,(11.72)的每个因子都是归一化的条件分布, 因此该分解对任意序列分布都成立.

♣

注

(重构新增) 定义 11.16的分解与(11.6)是同一个式子, 只是把“变量”换成了“序列的元素”, 并把排序固定为时间顺序. 这一排序的选择是有代价的: 它假定了“第 n 个元素只依赖过去”这一非对称的建模方向, 因此自回归模型不能直接表示“未来的观测影响过去的观测”这类依赖. 另一方面, 一般性也是它的弱点:(11.72)对任何分布都成立, 因此它本身不表达任何假设, 参数数量随 N 指数增长, 必须通过后面的 Markov 假设或参数共享 (定义 11.5) 加以限制.

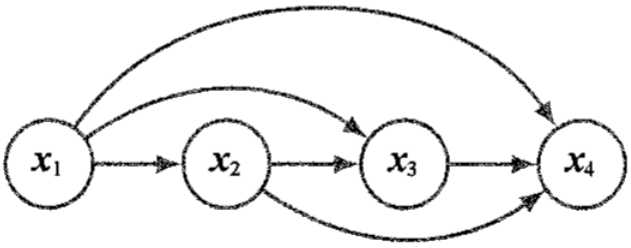


图 11.27: 具有 4 个节点且形式为(11.72)的一般自回归模型的示意图

这种表示法具有完全的通用性, 因此从建模的角度来看没有任何价值, 因为它没有表达任何假设. 我们可以通过从图中移除连接, 或者等效地从(11.72)右侧因子的条件集中去除变量, 来引入条件独立性, 从而限制模型的空间.

最有力的假设是去除所有条件变量, 从而得出联合分布, 其形式为

$$p(x_1, \cdots, x_N) = \prod_{n=1}^{N} p(x_n). \tag{11.73}$$

(11.73)将观测变量视为独立变量, 因此完全忽略了排序信息. 这对应于一个无连接的概率图模型, 如图 11.28所示.

图 11.28: 对一系列观测变量进行建模的最简单方法是将其视为独立变量, 这对应于一个无连接的图模型

既能捕获到顺序特性, 又能引入建模假设的有趣模型介于以上两个极端之间. 一个强有力的假设是, 每个条件分布都只依赖于序列中的前一个变量, 从而得出如下联合分布:

$$p(x_1, \cdots, x_N) = p(x_1) \prod_{n=2}^{N} p(x_n \mid x_{n-1}). \tag{11.74}$$

请注意, 序列中第一个变量的处理方式略有不同, 因为它没有条件变量.图 11.29所示图模型的函数形式[(11.74)] 称为 Markov 模型或 Markov 链, 其可用简单节点链组成的图来表示.

定义 11.17 (M 阶 Markov 模型)

若序列的联合分布可写成

$$p(x_1,\cdots,x_N)=p(x_1,\cdots,x_M)\prod_{n=M+1}^{N}p(x_n\mid x_{n-M},\cdots,x_{n-1}),\tag{11.75}$$

则称其为 M **阶 Markov 模型** (M-th order Markov model), 其中每个条件分布只依赖前面 M 个变量.$M=1$ 时(11.75)退化为(11.74), 称为**一阶 Markov 模型**或 **Markov 链**;$M=2$ 时称为二阶 Markov 模型.

♣

注

(重构新增) 定义 11.17的关键是“阶数” M: 它规定了条件分布的记忆长度, 从而把(11.72)中第 n 个因子的条件集从 $n-1$ 个变量削减到 M 个. 前 M 个变量需要单独处理, 因为它们的条件变量不足 M 个, 这不是技术细节而是定义的一部分: 若强行对 $n\leqslant M$ 使用同一个条件分布, 就需要引入“虚拟的”过去值.M 的选择直接决定模型的表达能力与参数规模, 二者由命题 11.11定量刻画. 另请注意,M 阶 Markov 性是对联合分布的一个约束, 而不是对单个条件分布的约束; 它等价于说“给定最近 M 个观测, 更早的观测不再提供信息”.

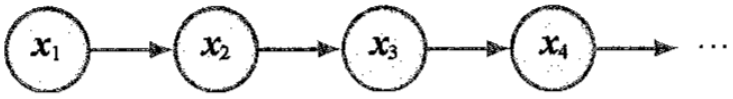


图 11.29: 观测值的一阶 Markov 链, 其中特定观测值 x_n 的分布以前一个观测值 x_{n-1} 为条件

通过使用 d 分离 (参见小节 11.2.3), 我们可以看到, 在时间 n 之前的所有观测值中, 观测值 x_{n-1} 的条件分布为

$$p(x_n\mid x_1,\cdots,x_{n-1})=p(x_n\mid x_{n-1}).\tag{11.76}$$

命题 11.10 (一阶 Markov 性质)

设 $p(x_1,\cdots,x_N)$ 由(11.74)给出, 则对 $n=2,\cdots,N$,(11.76)成立, 即

$$x_n\perp\{x_1,\cdots,x_{n-2}\}\mid x_{n-1},\tag{11.77}$$

其中约定 $n=2$ 时 $\{x_1,\cdots,x_{n-2}\}$ 为空集 (此时条件独立性平凡成立).

♠

证明

(重构新增) 由乘积法则与(11.74),

$$p(x_n\mid x_1,\cdots,x_{n-1})=\frac{p(x_1,\cdots,x_n)}{p(x_1,\cdots,x_{n-1})}=\frac{p(x_1)\prod_{m=2}^{n}p(x_m\mid x_{m-1})}{p(x_1)\prod_{m=2}^{n-1}p(x_m\mid x_{m-1})}=p(x_n\mid x_{n-1}),\tag{11.78}$$

分子分母中 $m\leqslant n-1$ 的因子逐一相消, 只剩最后一个因子. 这同时说明 x_n 与 $x_1,\cdots,x_{n-2}$ 在给定 x_{n-1} 后条件独立 (见练习 11.16).

□

注

(重构新增) 命题 11.10给出了两种等价的验证方式: 代数上靠因子相消, 图形上靠 d 分离 (路径 $x_n\to x_{n-1}\to\cdots\to x_1$ 在 x_{n-1} 处头尾相接且 x_{n-1} 已观测, 故被阻塞). 相消过程中分子分母必须“逐因子对齐”, 这依赖(11.74)的乘积顺序与时间顺序一致; 若把序列倒序写出, 因子形式改变, 但条件独立性的结论不变. 该性质是“用前一个观测预测下一个观测”的全部理论依据, 因此在实际预测中, 只要模型是一阶 Markov 的, 就可以丢弃 x_{n-2} 及更早的全部历史.

从(11.76)开始, 利用概率的乘积法则, 直接求值即可对此轻松验证 (见练习 11.16). 因此, 如果我们使用这样一个模型来预测序列中的下一个观测值, 则预测值的分布将只取决于前一个观测值, 而与之前的所有观测值无关.

更具体地说, 形式为(11.74)的图模型称为一阶 Markov 模型, 因为每个条件分布中只出现了一个条件变量. 我们可以对这个模型进行扩展, 方法是让每个条件分布都依赖于前面的两个变量, 从而得到一个二阶 Markov 模型,

其形式为

$$p(x_1,\cdots,x_N)=p(x_1)\,p(x_2\mid x_1)\prod_{n=3}^{N}p(x_n\mid x_{n-1},x_{n-2}). \tag{11.79}$$

请注意, 前两个变量的处理方法不同, 因为它们的条件变量少于两个.图 11.30以有向图的形式对该模型进行了展示.

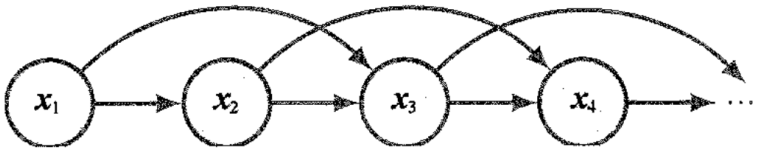


图 11.30: 一个二阶 Markov 链, 其中特定观测值 x_n 的条件分布取决于前两个观测值 x_{n-1} 和 x_{n-2}

通过使用 d 分离 (或使用概率规则进行直接计算), 我们可以看到, 在二阶 Markov 模型中, 给定所有先前观测值 $x_1,\cdots,x_{n-1}$ 的 x_n 的条件分布与观测值 $x_1,\cdots,x_{n-3}$ 无关 (见练习 11.17). 同样, 我们可以考虑扩展到 M 阶 Markov 链, 其中某个变量的条件分布取决于前 M 个变量.

假设观测值是具有 K 种状态的离散变量, 此时一阶 Markov 链中的条件分布 $p(x_n\mid x_{n-1})$ 将由一组参数 (共 $K-1$ 个) 指定; 在 x_{n-1} 的 K 个状态中, 每个状态都有 $K-1$ 个参数, 共有 $K(K-1)$ 个参数. 假设我们将模型扩展为 M 阶 Markov 模型, 这样联合分布便可用条件式 $p(x_n\mid x_{n-M},\cdots,x_{n-1})$ 来构造. 如果变量是离散的且条件分布用一般条件概率来表示, 模型将有 $K^{M-1}(K-1)$ 个参数. 因此, 参数数量会随着 M 的增加而呈指数增长, 一般来说,M 的值越大, 这种方法就越不实用.

命题 11.11 (M 阶 Markov 模型的参数计数)

设观测值为 K 状态的离散变量, 条件分布用完整条件概率表给出. 则 M 阶 Markov 模型(11.75)中, 每个条件分布 $p(x_n\mid x_{n-M},\cdots,x_{n-1})$ 需要 $K^M(K-1)$ 个自由参数, 整个模型 (忽略前 M 个变量的初始分布) 需要 $K^M(K-1)$ 个转移参数.

♠

证明

(重构新增) 条件分布 $p(x_n\mid x_{n-M},\cdots,x_{n-1})$ 对条件变量的每个取值组合给出一个长度为 K 的概率向量. 条件变量共有 M 个、每个 K 种取值, 故共有 K^M 个组合; 每个概率向量受归一化约束, 只有 $K-1$ 个自由参数. 相乘得 $K^M(K-1)$.$M=1$ 时它等于 $K(K-1)$, 与正文中一阶链的参数数目一致;$M=0$ 时它退化为 $K-1$, 即无记忆的独立模型. 前 M 个变量的初始分布 $p(x_1,\cdots,x_M)$ 另需 K^M-1 个参数, 正文所说的 “构造联合分布所需的参数” 通常只统计随 n 重复使用的转移参数.

□

注

(重构新增) 原文问题说明原文把 M 阶模型的参数数目印作 $K^{M-1}(K-1)$, 这与同一段前面给出的 “一阶链共有 $K(K-1)$ 个参数” 相矛盾: 代入 $M=1$ 会得到 $K-1$, 而不是 $K(K-1)$. 按命题 11.11的推导, 正确数目是 $K^M(K-1)$, 它在 $M=1$ 时给出 $K(K-1)$, 在 $M=0$ 时给出 $K-1$, 两个端点都吻合. 因此这里按修正后的 $K^M(K-1)$ 理解. 无论采用哪个式子, 结论 “参数随 M 指数增长” 都不变, 这也是高阶 Markov 模型在实践中很少直接使用的根本原因; 替代方案是定义 11.18的潜变量模型, 它用 $O(K^2)$ 个转移参数换取任意长的记忆.

潜变量

假设我们希望为序列建立一个模型, 这个模型不受 Markov 假设的任何顺序的限制, 但可以使用有限数量的自由参数来指定. 我们可以通过引入额外的潜变量 (又称隐变量) 来实现这一目标, 从而允许从简单组件中构建出类别丰富的模型. 对于每个观测值 x_n, 我们引入一个相应的潜变量 z_n(可能与观测变量的类型或维度不同). 现在,

我们假设潜变量构成了 Markov 链, 从而产生了状态空间模型的图结构, 如图 11.31所示. 它满足关键的条件独立性要求, 即给定 z_n 时,z_{n-1} 和 z_{n+1} 是独立的, 因此

$$z_{n+1} \perp z_{n-1} \mid z_n. \tag{11.80}$$

定义 11.18 (状态空间模型)

设 $z_1, \cdots, z_N$ 是潜变量,$x_1, \cdots, x_N$ 是观测变量, 且联合分布按有向图 11.31分解为

$$p(x_1, \cdots, x_N, z_1, \cdots, z_N) = p(z_1)\left[\prod_{n=2}^{N} p(z_n \mid z_{n-1})\right]\left[\prod_{n=1}^{N} p(x_n \mid z_n)\right], \tag{11.81}$$

则称该模型为**状态空间模型** (state space model). 其中 $p(z_1)$ 是初始分布,$p(z_n \mid z_{n-1})$ 是转移分布,$p(x_n \mid z_n)$ 是发射分布.

♣

注

(重构新增) 定义 11.18用“潜变量链 + 逐点发射”替换了(11.75)的“显式记忆 M 步”. 这样做的收益是参数数量与 N 无关、且记忆长度不受限制; 代价是观测变量之间不再有直接的边, 所有依赖都必须通过潜状态传递, 因此似然需要对潜变量求积分 (求和), 一般无法解析计算.(11.81)的三部分——初始、转移、发射——正是隐 Markov 模型与线性动态系统的共同骨架, 二者的区别仅在于 z_n 是离散还是连续、发射分布是离散表还是线性 Gauss(定义 11.6). 这种“同一图、不同分布族”的现象再次说明图结构与分布族是两个独立的建模维度.

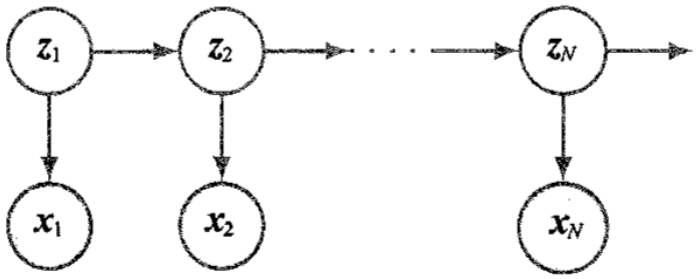


图 11.31: 状态空间模型用潜变量 $z_1, \cdots, z_N$ 表示一系列观测变量 $x_1, \cdots, x_N$ 的联合概率分布, 它的形式如(11.81)所示

通过使用 d 分离, 我们可以发现, 在状态空间模型中, 总有一条路径能通过潜变量连接任意两个观测变量 x_i 和 x_m, 而且这条路径永远不会阻塞. 因此, 在给定所有先前观测值的情况下, 观测值 x_{n+1} 的预测分布 $p(x_{n+1} \mid x_1, \cdots, x_n)$ 并不会表现出任何条件独立性, 因此我们对 x_{n+1} 的预测取决于所有先前的观测值. 观测变量在任何有限阶 (order) 都不符合 Markov 特性.

图 11.31描述了两个重要的序列模型. 如果潜变量是离散的, 我们就会得到一个隐 Markov 模型 (Elliott, Aggoun, and Moore, 1995). 请注意, 隐 Markov 模型中的观测变量既可以是离散的, 也可以是连续的, 而且可以使用多种不同的条件分布来进行建模. 如果潜变量和观测变量都是 Gauss 变量 (其条件分布与父变量之间存在线性-Gauss 依赖关系), 我们就可以得到一个线性动态系统, 也称 Kalman 滤波器 (Zarchan and Musoff, 2005).Bishop (2006) 详细讨论了隐 Markov 模型和 Kalman 滤波器, 以及训练它们的算法. 用深度神经网络取代用于定义 $p(x_n \mid z_n)$ 的简单离散概率表或线性 Gauss 分布, 便可以使这类模型变得更加灵活.

定义 11.19 (隐 Markov 模型)

在定义 11.18中, 若潜变量 z_n 取 K 个离散状态且转移与发射都是离散分布, 则称该模型为**隐 Markov 模型** (hidden Markov model, HMM). 若 z_n 与 x_n 都是 Gauss 变量且条件分布是线性 Gauss 的, 则称该模型为**线性动态系统** (linear dynamical system), 也称 **Kalman 滤波器**.

♣

注

(重构新增) 定义 11.19中“隐”字指状态 z_n 不可观测, 只有 x_n 可见; 这与定义 11.17的 (显式)Markov 链形成对比: 后者的状态就是观测本身.HMM 的表达能力来自“离散状态 + 任意发射分布”的组合: 状态转移只有 K^2 个参数, 而发射分布 $p(x_n \mid z_n)$ 可以是离散表、Gauss 密度, 甚至可以是由深度神经网络参数化的分布 (此时模型常称为神经 HMM). 需要注意,HMM 的似然 $p(x_1, \cdots, x_N)$ 需要对 K^N 条状态路径求和, 直接计算不可行, 必须使用下面的前向-后向递推.

注

(重构新增) 原书在此只给出了状态空间模型与 HMM 的图结构和定义, 并把前向 - 后向算法与参数学习留给了 Bishop (2006). 为使本章的序列模型部分自足, 下面补写 HMM 的似然计算与前向-后向递推, 以下内容均为重构新增, 不冒充原书内容.

命题 11.12 (HMM 的前向–后向递推 (重构新增))

设 $p(z_1, \cdots, z_N, x_1, \cdots, x_N)$ 由(11.81)给出,$z_n \in \{1, \cdots, K\}$. 定义**前向变量**与**后向变量**

$$\alpha_n(z_n) \triangleq p(x_1, \cdots, x_n, z_n), \tag{11.82}$$

$$\beta_n(z_n) \triangleq p(x_{n+1}, \cdots, x_N \mid z_n). \tag{11.83}$$

则它们满足递推

$$\alpha_1(z_1) = p(z_1)\, p(x_1 \mid z_1), \qquad \alpha_n(z_n) = p(x_n \mid z_n) \sum_{z_{n-1}} \alpha_{n-1}(z_{n-1})\, p(z_n \mid z_{n-1}), \tag{11.84}$$

$$\beta_N(z_N) = 1, \qquad \beta_n(z_n) = \sum_{z_{n+1}} \beta_{n+1}(z_{n+1})\, p(x_{n+1} \mid z_{n+1})\, p(z_{n+1} \mid z_n), \tag{11.85}$$

并且对任意 $n = 1, \cdots, N$, 似然可写成

$$p(x_1, \cdots, x_N) = \sum_{z_N} \alpha_N(z_N) = \sum_{z_n} \alpha_n(z_n)\, \beta_n(z_n), \tag{11.86}$$

潜状态的后验边缘为

$$\gamma_n(z_n) \triangleq p(z_n \mid x_1, \cdots, x_N) = \frac{\alpha_n(z_n)\, \beta_n(z_n)}{p(x_1, \cdots, x_N)}. \tag{11.87}$$

证明

(重构新增) 前向递推: 由乘积法则与(11.81),$p(x_1, \cdots, x_n, z_n)$ 可以写成“前 $n-1$ 步的全部变量”与“第 n 步”的乘积, 并对 z_{n-1} 求和:

$$\alpha_n(z_n) = \sum_{z_{n-1}} p(x_1, \cdots, x_n, z_{n-1}, z_n) = \sum_{z_{n-1}} p(x_1, \cdots, x_{n-1}, z_{n-1})\, p(z_n \mid z_{n-1})\, p(x_n \mid z_n) \tag{11.88}$$

$$= p(x_n \mid z_n) \sum_{z_{n-1}} \alpha_{n-1}(z_{n-1})\, p(z_n \mid z_{n-1}), \tag{11.89}$$

其中第二个等号用了条件独立性: 给定 z_{n-1}, z_n 后,$(x_1, \cdots, x_{n-1})$ 与 (x_n, z_n) 独立 (图 11.31中路径在 z_n 处被观测节点 z_n 阻塞). 基例 $\alpha_1(z_1) = p(z_1)p(x_1 \mid z_1)$ 由同一分解直接得到.

后向递推: 对 z_{n+1} 用全概率公式并再次利用条件独立性,

$$\beta_n(z_n) = \sum_{z_{n+1}} p(x_{n+1}, \cdots, x_N, z_{n+1} \mid z_n) = \sum_{z_{n+1}} p(z_{n+1} \mid z_n)\, p(x_{n+1} \mid z_{n+1})\, p(x_{n+2}, \cdots, x_N \mid z_{n+1}) \tag{11.90}$$

$$= \sum_{z_{n+1}} p(z_{n+1} \mid z_n)\, p(x_{n+1} \mid z_{n+1})\, \beta_{n+1}(z_{n+1}), \tag{11.91}$$

基例 $\beta_N(z_N) = p(\varnothing \mid z_N) = 1$.

似然:$\alpha_N(z_N) = p(x_1, \cdots, x_N, z_N)$, 对 z_N 求和即得第一个等号; 另一方面, 对任意 n,

$$\alpha_n(z_n)\beta_n(z_n) = p(x_1, \cdots, x_n, z_n)\, p(x_{n+1}, \cdots, x_N \mid z_n) = p(x_1, \cdots, x_N, z_n), \tag{11.92}$$

其中用了给定 z_n 时“过去”与“未来”条件独立. 对 z_n 求和即得(11.86)的第三个等号.

后验边缘: 由 Bayes 定理,$p(z_n \mid x_1, \cdots, x_N) = p(x_1, \cdots, x_N, z_n)/p(x_1, \cdots, x_N)$, 分子即上式, 代回即得(11.87). □

注

(重构新增) 命题 11.12的核心是“动态规划”: 直接计算似然要对 K^N 条状态路径求和, 而(11.84)把求和按时间分解, 每一步只保留 K 个数, 总代价为 $O(NK^2)$(每个 n 要做 K^2 次乘加), 从指数级降到线性级. 这一“把路径求和改写为逐时刻状态求和”的技巧与小节 11.2.7中消息传递的思想完全一致: 前向变量就是沿链从过去传向未来的消息, 后向变量是反向的消息.(11.86)中“对任意 n 求和都得到同一个数”是算法正确性的自检条件, 也是数值稳定化 (缩放) 的出发点:α_n 与 β_n 分别随 n 指数衰减与增长, 实现时必须每步归一化. 条件“z_n 离散”保证了求和可算; 若 z_n 连续 (如 Kalman 滤波器), 求和换成积分, 线性 Gauss 情形下积分仍有闭式解, 这正是命题 11.3在 Gauss 情形下的对应物.

11.4 习题

- **练习 11.1** (★) 通过按顺序将变量边缘化, 证明有向图的联合分布 [(11.8)] 是正确归一化的, 前提是每个条件分布也都是归一化的.
- **练习 11.2** (★) 证明有向图中不存在有向循环这一特性是由以下陈述得出的: 存在节点的有序编号, 使得每个节点都不存在通往低编号节点的连接.
- **练习 11.3** (★★) 考虑 3 个二进制变量 $a, b, c \in \{0, 1\}$, 其联合分布如表 11.1所示. 通过直接求值, 证明该分布具有以下性质:a 和 b 边缘相关, 因此 $p(a, b) \neq p(a)p(b)$; 但是当以 c 为条件时, 它们变得独立, 因此在 $c = 0$ 和 $c = 1$ 时,$p(a, b \mid c) = p(a \mid c)p(b \mid c)$.

表 11.1: 3 个二进制变量 a、b、c 的联合分布

a	b	c	$p(a, b, c)$
0	0	0	0.192
0	0	1	0.144
0	1	0	0.048
0	1	1	0.216
1	0	0	0.192
1	0	1	0.064
1	1	0	0.048
1	1	1	0.096

- **练习 11.4** (★★) 求出表 11.1中给出的联合分布所对应的分布 $p(a)$、$p(b \mid c)$ 和 $p(c \mid a)$. 然后通过直接求值证明 $p(a, b, c) = p(a)p(c \mid a)p(b \mid c)$, 并绘制相应的有向图.
- **练习 11.5** (★) 对于图 11.6所示的模型, 我们已经看到, 通过利用逻辑斯谛 sigmoid 表示法 [(11.12)], 我们可以将条件分布 $p(y \mid x_1, \cdots, x_M)$(其中 $x_i \in \{0, 1\}$) 所需的参数数量从 2^M 减少至 $M + 1$. 另一种表示法 (Pearl, 1988) 如下:

$$p(y = 1 \mid x_1, \cdots, x_M) = 1 - (1 - \mu_0) \prod_{i=1}^{M} (1 - \mu_i)^{x_i}, \tag{11.93}$$

其中, 参数 μ_i 代表概率 $p(x_i = 1)$,μ_0 是一个附加参数, 满足 $0 \leqslant \mu_0 \leqslant 1$. 此条件分布 [(11.93)] 称为噪声 OR. 证明这可以解释为逻辑 OR 函数的“软”(概率) 形式 (即只要至少有一个 $x_i = 1$, 函数就会给出 $y = 1$). 讨论 μ_0 的解释.
- **练习 11.6** (★★) 从条件分布的定义 [(11.13)] 出发, 推导出线性 Gauss 模型联合分布均值的递推公式 [(11.15)].
- **练习 11.7** (★★) 从条件分布的定义 [(11.13)] 出发, 推导出线性 Gauss 模型联合分布协方差矩阵的递推公式 [(11.16)].
- **练习 11.8** (★★) 证明由(11.13)定义的 D 个变量的全连接线性 Gauss 图模型的协方差矩阵中的参数数量为 $D(D + 1)/2$.

✍ **练习 11.9** (★★) 用递推公式 [(11.15)和(11.16)] 证明图 11.7所示图模型的联合分布的均值和协方差分别由(11.19)和(11.20)给出.

✍ **练习 11.10** (★) 验证由线性 Gauss 模型定义的一组向量值变量的联合分布本身就是 Gauss 分布, 其中每个节点对应于形式为(11.21)的分布.

✍ **练习 11.11** (★) 证明 $a \perp \{b, c\} \mid d$ 意味着 $a \perp b \mid d$.

✍ **练习 11.12** (★) 通过使用 d 分离, 证明有向图中节点 x 的条件分布 (以 Markov 毯中的所有节点为条件) 与该图中的其余变量无关.

✍ **练习 11.13** (★) 考虑图 11.32所示的有向图, 其中未观测到任何变量. 证明 $a \perp b \mid \varnothing$. 假设我们现在观测到变量 d, 证明一般情况下 $a \not\perp b \mid d$.

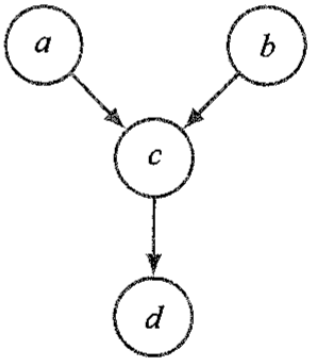


图 11.32: 当观测到节点 c 的后代节点 (即节点 d) 时, 用于研究头头相接路径 a-c-b 的条件独立性的图模型示例

✍ **练习 11.14** (★) 以图 11.20所示的汽车燃油系统为例, 假设驾驶员 D 未直接观测到燃油表 G 的状态, 而是看到了燃油表并向我们报告了燃油表读数. 该驾驶员报告指出, 燃油表要么显示油箱已满 ($D = 1$), 要么显示油箱已空 ($D = 0$). 该驾驶员有点不可靠, 具体由以下概率可以看出:

$$p(D = 1 \mid G = 1) = 0.9, \tag{11.94}$$

$$p(D = 0 \mid G = 0) = 0.9. \tag{11.95}$$

假设该驾驶员告诉我们燃油表显示油箱已空, 换句话说,$D = 0$. 仅根据这一条件, 计算油箱为空的概率. 同样, 在观测到电池没电的情况下, 计算相应的概率, 注意第二个概率更低. 讨论这一结果, 并将该结果与图 11.32联系起来.

✍ **练习 11.15** (★★) 用最大似然法训练一个朴素 Bayes 模型, 假设条件为(11.59), 并假设每个类-条件密度 $p(x^{(l)} \mid C_k)$ 都由它们各自独立的参数 $\mathbf{w}^{(l)}$ 控制. 通过最大化与相应类标签数据相关的似然, 然后将先验类概率 $p(C_k)$ 设置为每个类中训练数据点的比例, 证明最大似然求解涉及使用相应的观测数据向量 $\mathbf{x}_1^{(l)}, \cdots, \mathbf{x}_N^{(l)}$.

✍ **练习 11.16** (★★) 考虑与图 11.29所示的有向图相对应的联合概率分布 [(11.74)]. 利用概率的加和法则与乘积法则, 验证在 $n = 2, \cdots, N$ 时, 该联合概率分布满足条件独立性 [(11.76)]. 同样, 证明形式为(11.79)的联合分布所描述的二阶 Markov 模型满足 $n = 3, \cdots, N$ 时的条件独立性:

$$p(x_n \mid x_1, \cdots, x_{n-1}) = p(x_n \mid x_{n-1}, x_{n-2}). \tag{11.96}$$

✍ **练习 11.17** (★) 通过使用 d 分离, 验证图 11.29所示的一阶 Markov 模型 (其中有 N 个节点) 在 $n = 2, \cdots, N$ 时满足条件独立性 [(11.76)]. 同样, 证明图 11.30所示的二阶 Markov 模型 (其中也有 N 个节点) 在 $n = 3, \cdots, N$ 时满足条件独立性 [(11.96)].

✍ **练习 11.18** (★) 考虑图 11.30所示的二阶 Markov 过程, 通过合并相邻的变量对, 证明它可以用新变量的一阶 Markov 过程来表示.

✍ **练习 11.19** (★) 通过使用 d 分离, 证明图 11.31中的有向图所代表的状态空间模型的观测数据分布 $p(x_1, \cdots, x_N)$ 不满足任何条件独立性, 因此观测变量在任何有限阶都不符合 Markov 特性.

第 12 章 Transformer

Transformer 是深度学习领域最重要的进展之一. 基于一种称为注意力 (attention) 的处理概念,Transformer 能使网络对不同的输入赋予不同的权重, 而这些加权系数本身又取决于输入值, 从而捕捉到与序列数据和其他形式数据相关的强大归纳偏差.

这类模型之所以叫 Transformer, 是因为它们能将某个表示空间中的一组向量变换成某个新空间中具有相同维度的相应向量的集合. 变换旨在使新空间具有更丰富的内部表示, 该表示更适合解决下游任务. 对于 Transformer 而言, 其输入可以是非结构化的向量集、有序序列或其他更一般的表示形式, 从而使 Transformer 具有广泛的适用性.

Transformer 最初是在自然语言处理 (Natural Language Processing, NLP) 领域引入的, 这里的“自然”语言指的是像英语或汉语这样的语言, 它大大超越了之前基于递归神经网络 (Recurrent Neural Network, RNN) 的那些最先进的方法. 随后, 人们发现 Transformer 在许多其他领域也取得了优异成果. 例如, 视觉 Transformer 在图像处理任务中的表现往往优于卷积神经网络 (Convolutional Neural Network, CNN), 而结合文本、图像、音频和视频等多种类型数据的多模态 Transformer 则是最强大的深度学习模型之一.

Transformer 的一大优势在于迁移学习效果显著, 所以 Transformer 可以在大量数据上进行训练, 然后通过某种形式的微调, 将经过训练的模型应用于众多下游任务. 一个大规模模型随后可以进行调整以解决多个不同的任务, 它就是所谓的基础模型 (foundation model). 此外,Transformer 可以使用无标签数据以自监督的方式进行训练, 这对语言模型尤其有效, 因为 Transformer 可以利用来自互联网和其他来源的海量文本. 缩放假设 (scaling hypothesis) 认为, 只要扩大模型的规模 (以可学习参数的数量来衡量), 并在相应的大规模数据集上进行训练, 即使不改变架构, 也能显著提高性能. 此外,Transformer 特别适用于大规模并行处理硬件, 如图形处理单元 (Graphics Processing Unit, GPU), 从而能在合理的时间内训练出拥有万亿 (10^{12}) 规模参数的超大型神经网络语言模型. 这些模型具有非凡的能力, 并展现出被描述为通用人工智能早期迹象的涌现特性 (Bubeck et al., 2023).

对于初学者来说,Transformer 架构可能看起来很复杂, 甚至让人望而却步, 因为它涉及多个不同组件的协同工作, 其中各种设计构思可能看起来很随意. 因此, 本章将逐步全面地介绍 Transformer 背后的所有关键思想, 并提供清晰的直观认识以引出各种元素的设计. 在介绍完 Transformer 架构后, 我们将重点介绍自然语言处理, 并在最后探讨其他应用领域.

12.1 注意力

作为 Transformer 的核心概念, 注意力最初是作为机器翻译领域递归神经网络 (参见小节 12.2.5) 的增强版而开发的 (Bahdanau, Cho, and Bengio, 2014). 不过,Vaswani et al. (2017) 后来证明, 如果去除递归结构, 转而只专注于注意力机制, 就能显著提高性能. 如今, 在几乎所有应用中, 基于注意力的 Transformer 已完全取代递归神经网络.

虽然注意力具有非常广泛的适用性, 但我们仍将以自然语言为例引出注意力的应用. 请看如下两个句子.

I swam across the river to get to the other bank.

I walked across the road to get cash from the bank.

“bank”一词在这两个句子中具有不同的含义. 不过, 这只能通过查看句子中其他词提供的上下文才能发现. 我们还看到, 在决定“bank”如何翻译时, 有些词发挥着尤为重要的作用. 在第 1 个句子中, “swam”和“river”两个词最能说明“bank”指的是河边; 而在第 2 个句子中, “cash”一词最能说明“bank”指的是金融机构. 我们发现, 为了对“bank”进行适当的翻译, 处理这样一个句子的神经网络应该关注 (换句话说, 更依赖于) 句子中其余部分的特定词. 注意力的概念如图 12.1所示.

此外, 我们还发现, 哪些位置应获得更多注意力取决于输入序列本身: 在第 1 个句子中, 重要的是第 2 和第 5 个词; 而在第 2 个句子中, 重要的是第 8 个词. 在标准神经网络中, 不同的输入对输出产生的不同程度的影响, 由输入值乘以相应的权重系数来决定. 然而, 一旦神经网络经过训练, 这些权重及相关输入就会固定下来. 相比之下, 注

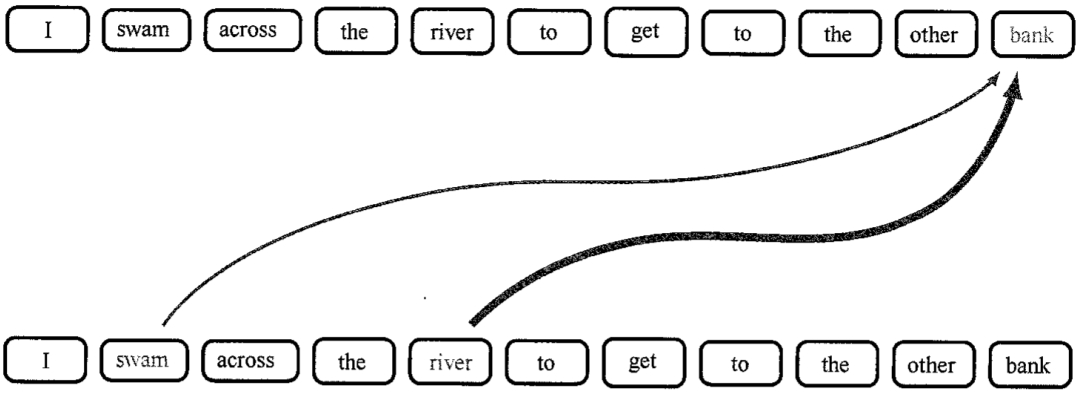


图 12.1: “bank” 一词的解释受 “river” 和 “swam” 影响的注意力示意图, 每条线的粗细表示受影响的程度.

意力使用的加权因子的值取决于具体的输入数据. 在自然语言上经过训练的一个 Transformer 网络的注意力权重示例见图 12.2.

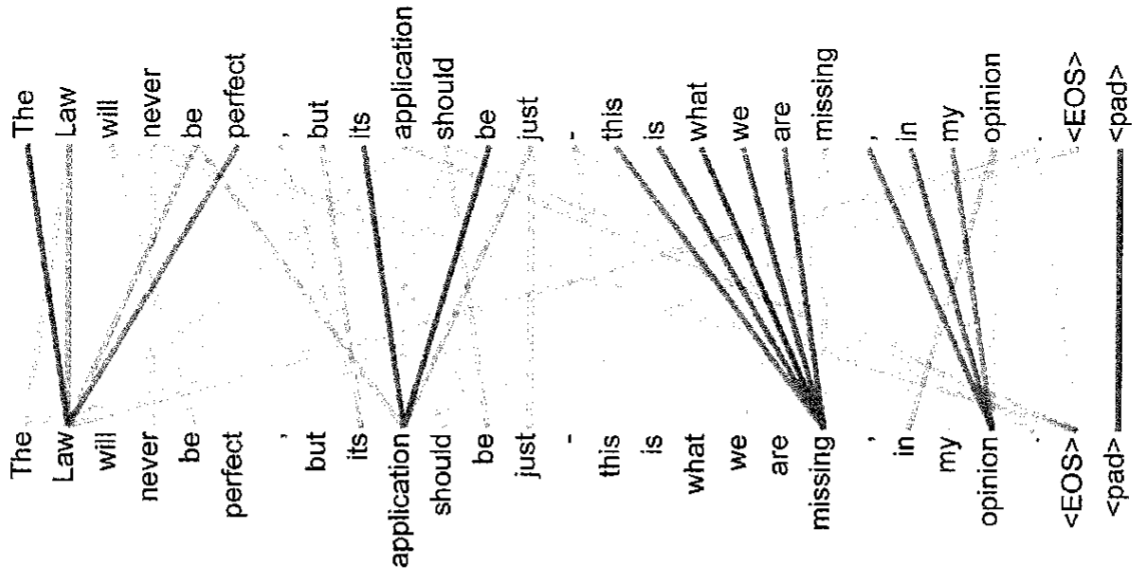


图 12.2: 习得的注意力权重示例 [经 Vaswani 等人授权使用].

在讨论自然语言处理时, 我们将了解如何使用词嵌入 (word embedding) 将词映射到嵌入空间中的向量. 这些向量可用作后续神经网络处理的输入. 这些词嵌入可以捕捉到基本的语义属性, 例如, 将具有相似含义的词映射到嵌入空间中的邻近位置. 这种嵌入的一个特点是, 给定的词总是被映射到同一个嵌入向量.

Transformer 可以视为一种更丰富的嵌入形式, 其中一个给定向量被映射到一个依赖序列中其他向量的位置. 因此, 上述示例中代表 “bank” 的向量可以映射到两个不同句子的新嵌入空间中的不同位置. 例如, 在第 1 个句子中, 变换后的表示可能会使 “bank” 在嵌入空间中靠近 “water” ; 而在第 2 个句子中, 变换后的表示可能会将 “bank” 放在靠近 “money” 的位置.

下面我们以蛋白质建模为例来说明注意力的重要性. 我们可以把蛋白质看作由分子单元 (称为氨基酸) 组成的一维序列. 一个蛋白质可能由数百或数千个这样的分子单元组成, 每个分子单元由 22 种可能性中的一种给定. 在活细胞中, 蛋白质折叠成三维结构, 一维序列中互相远离的氨基酸可以在三维空间中变得接近, 从而产生相互作用.Transformer 模型允许这些互相远离的氨基酸相互 “关注” , 从而大大提高了三维结构建模的准确性 (Vig et al., 2020).

12.1.1 Transformer 处理

Transformer 的输入数据是一组 D 维向量 $\{\mathbf{x}_n\}$, 其中 $n = 1, \cdots, N$. 我们将这些数据向量称为 token, 一个 token 可能对应句子中的一个词、图像中的一个小块或者蛋白质中的一个氨基酸.token 中的元素 x_{ni} 称为特征. 稍后我们将了解如何为自然语言数据和图像构建这些 token 向量.Transformer 的一个强大特性是, 我们无须设计新的网络架构来处理不同类型的数据, 而只需要将数据变量组合成一组联合 token 即可.

在清楚地了解 Transformer 的工作原理之前, 我们必须准确地使用符号. 我们将按照标准惯例, 将数据向量组合成一个 $N \times D$ 维的矩阵 $\mathbf{X}$, 其中第 n 行是 token 向量 $\mathbf{x}_n^{\mathsf{T}}, n = 1, \cdots, N$, 如图 12.3所示. 请注意, 该矩阵代表一组输入 token, 而在大多数应用中, 我们需要一个包含多组 token 的数据集, 例如每个词都是由一个 token 表示的独立文本段落.

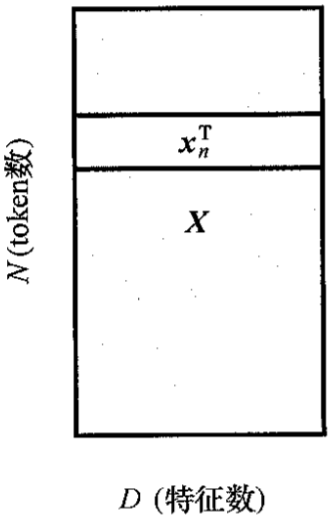


图 12.3: $N \times D$ 维的数据矩阵 $\mathbf{X}$ 的结构, 其中第 n 行代表转置后的数据向量 $\mathbf{x}_n^{\mathsf{T}}$.

Transformer 的基本构件是一个函数, 用于将数据矩阵作为输入并创建一个具有相同维度的变换矩阵 $\bar{\mathbf{X}}$ 作为输出. 我们可以写下该函数, 形式如下:

$$\bar{\mathbf{X}} = \text{TransformerLayer}[\mathbf{X}] \tag{12.1}$$

然后, 我们可以连续叠加多个 Transformer 层来构建能够学习丰富内部表示的深度网络. 每个 Transformer 层都有自己的权重和偏置, 可以通过梯度下降法使用适当的损失函数进行学习, 这将在本章后续部分 (参见节 12.3) 详细讨论.

Transformer 层本身包含两个阶段. 在实现注意力机制的第一阶段, 将数据矩阵各列中不同 token 向量的相应特征混合在一起; 第二阶段则独立作用于每一行, 并对每个 token 向量中的特征进行变换. 下面让我们先来看看注意力机制.

12.1.2 注意力系数

假设嵌入空间中有一组输入 token $\mathbf{x}_1, \cdots, \mathbf{x}_N$, 我们希望将它们映射到另一组具有相同数量 token 但位于一个新嵌入空间的 $\mathbf{y}_1, \cdots, \mathbf{y}_N$, 这一新的嵌入空间能捕捉到更丰富的语义结构. 考虑一个特定的输出向量 $\mathbf{y}_n$.$\mathbf{y}_n$ 的值不仅取决于相应的输入向量 $\mathbf{x}_n$, 还取决于集合中所有其他的输入向量. 凭借注意力, 对于那些在确定 $\mathbf{y}_n$ 的修正表示中发挥特别重要作用的输入向量 $\mathbf{x}_m$ 来说, 这种依赖性应该更强. 实现这一目标的简单方法是将每个输出向量 $\mathbf{y}_n$ 定义为输入向量 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 与加权系数 a_{nm} 的线性组合:

$$\mathbf{y}_n = \sum_{m=1}^{N} a_{nm}\mathbf{x}_m \tag{12.2}$$

定义 12.1 (注意力权重)

设 $\mathbf{x}_1,\cdots,\mathbf{x}_N\in\mathbb{R}^D$ 为一组输入 token,$\mathbf{y}_1,\cdots,\mathbf{y}_N$ 为对应的输出 token. 若每个输出向量按(12.2)写成输入向量的线性组合, 则系数 a_{nm} 称为**注意力权重** (attention weight), 又称注意力系数或权重系数; 它同时满足非负性

$$a_{nm}\geqslant 0 \tag{12.3}$$

与归一性

$$\sum_{m=1}^{N}a_{nm}=1 \tag{12.4}$$

其中 $n,m=1,\cdots,N$.

♣

注

(重构新增) 注意力的动机是让“哪些输入重要”由数据本身决定, 而不是像标准神经网络那样在训练后把权重固定下来. 为此,(12.2)把输出写成输入的线性组合, 把“重要性”完全交给系数 a_{nm}: 对 $\mathbf{y}_n$ 影响小的 token 取 $a_{nm}\approx 0$, 影响大的 token 取较大的 a_{nm}. 两条约束各有明确作用. 非负性(12.3)排除了“一个大正系数被另一个大负系数抵消”的情形——否则系数大小不再能直接读作重要程度, 而且 $\mathbf{y}_n$ 可能落在输入向量凸包之外, 与“加权平均”的直观解释不符. 归一性(12.4)则把注意力变成一种“预算分配”: 给某个输入更多注意力, 必然以减少对其他输入的注意力为代价, 从而使 a_{nm} 的量级在所有 n 之间可比. 两条约束合起来等价于说:$\mathbf{y}_n$ 是输入 token 的一个凸组合, 系数构成所谓的“单位分解”. 这里 N 与 D 都不受限制, 系数矩阵 $\mathbf{A}=[a_{nm}]$ 的每一行各自归一化, 不同输出位置使用互不相同的系数集合.

这意味着每个注意力系数都在 $0\leqslant a_{nm}\leqslant 1$ 的范围内, 因此这些注意力系数定义了一个“单位分解”(见练习 12.1). 在 $a_{nn}=1$ 且当 $n\neq m$ 时 $a_{nm}=0$ 的特殊情况下, 由于 $\mathbf{y}_n=\mathbf{x}_n$, 输入向量在变换后保持不变. 一般来说, 输出向量 $\mathbf{y}_n$ 是输入向量的混合体, 其中一些输入向量的权重系数高于其他输入向量.

命题 12.1 (注意力系数构成单位分解)

设 a_{nm} 满足(12.3)与(12.4), 并按(12.2)定义 $\mathbf{y}_n$. 则对每个 n 有 $0\leqslant a_{nm}\leqslant 1(m=1,\cdots,N)$, 且 $\mathbf{y}_n$ 落在输入向量的凸包之内; 特别地, 若 $a_{nn}=1$ 而 $n\neq m$ 时 $a_{nm}=0$, 则 $\mathbf{y}_n=\mathbf{x}_n$.

♠

证明

由非负性 $a_{nm}\geqslant 0$ 与归一性 $\sum_{m=1}^{N}a_{nm}=1$ 立即得到 $a_{nm}\leqslant\sum_{m'=1}^{N}a_{nm'}=1$. 凸包是包含全部 $\mathbf{x}_m$ 的最小凸集, 而 $\mathbf{y}_n$ 恰是这些点的系数非负、总和为 1 的线性组合, 故 $\mathbf{y}_n$ 属于凸包. 若 $a_{nn}=1$ 且 $n\neq m$ 时 $a_{nm}=0$, 则(12.2)中只有 $m=n$ 一项非零, 于是 $\mathbf{y}_n=\mathbf{x}_n$.

□

注

(重构新增) 该命题把两条看似技术性的约束翻译成几何语言: 注意力层的每个输出都是输入点集的凸组合, 因此输出永远不会“跑出”输入所张成的区域. 这正是后面残差连接与层归一化能够稳定训练的前提之一——注意力的输出与输入同量级. 需要强调,$a_{nm} \in [0,1]$ 只是两条约束的推论, 不是额外假设; 而 $\mathbf{y}_n = \mathbf{x}_n$ 的“恒等”情形要求整行为单位向量, 而不是仅仅 a_{nn} 较大: 只要 $a_{nn} < 1$, 输出就严格混合了其他 token 的信息, 这正是下面自注意力要刻画的现象.

请注意, 我们为每个输出向量 $\mathbf{y}_n$ 设置了一组不同的系数, 而约束条件 [(12.3)和(12.4)] 分别适用于每个 n 值. 这些系数取决于输入数据, 后续我们将展示如何计算它们.

12.1.3 自注意力

接下来的问题是如何确定权重系数 a_{nm}. 在展开详细讨论之前, 我们不妨先了解一下信息检索领域的一些术语. 以通过线上电影流媒体服务选择观看哪部电影为例. 一种方法是将每部电影与一系列属性关联起来, 这些属性描述了电影的类型 (喜剧、动作片等)、主演姓名、片长等. 用户可以通过目录搜索找到自己喜欢看的电影. 我们可以将每部电影的属性编码成一个称为键 (key) 的向量, 从而将该过程自动化. 相应的电影文件本身叫作值 (value). 同样, 用户也可以为所需属性提供自己的个人值向量, 我们称之为查询 (query). 然后, 线上电影流媒体服务可以将查询向量与所有键向量作比较, 从而找出最佳匹配, 并以值文件的形式将相应的电影发送给用户. 用户所“关注”的特定电影, 其键向量最匹配于用户的查询向量. 这可以视为硬注意力 (hard attention) 的一种形式, 其会返回一个单一值向量. 对于 Transformer, 我们可以将其泛化到软注意力 (soft attention) 中, 即使用连续变量来度量查询和键之间的匹配程度, 然后使用这些变量来权衡值向量对输出的影响. 这也将确保 Transformer 函数是可微分的, 从而可以通过梯度下降法进行训练.

参照信息检索的方法, 我们可以将每个输入 $\mathbf{x}_n$ 视为一个值向量, 用来创建输出 token. 我们还可以直接使用 $\mathbf{x}_n$ 作为输入 token n 的键向量. 这就好比用电影本身来概括电影的特点. 最后, 我们可以使用 $\mathbf{x}_n$ 作为输出 $\mathbf{y}_n$ 的查询向量, 然后将其与每个键向量作比较. 要想知道 $\mathbf{x}_n$ 代表的 token 与 $\mathbf{x}_m$ 代表的 token 有多大的相似度, 我们需要计算出这些向量的相似度. 度量向量相似度的一种简单方法是求它们的点积 $\mathbf{x}_n^\mathsf{T}\mathbf{x}_m$. 为了施加约束条件 [(12.3)和(12.4)], 使用 softmax 函数变换点积 (参见节 5.3), 以定义权重系数 a_{nm}:

$$a_{nm} = \frac{\exp\left(\mathbf{x}_n^\mathsf{T}\mathbf{x}_m\right)}{\sum_{m'=1}^{N}\exp\left(\mathbf{x}_n^\mathsf{T}\mathbf{x}_{m'}\right)} \tag{12.5}$$

定义 12.2 (点积自注意力)

设数据矩阵 $\mathbf{X} \in \mathbb{R}^{N\times D}$ 的第 n 行为 $\mathbf{x}_n^\mathsf{T}$. 若以 a_{nm} 按(12.5)由输入向量自身的点积给出, 并按(12.2)构造输出, 则称该变换为**自注意力** (self-attention); 由于相似度由点积给出, 又称**点积自注意力** (dot-product self-attention). 用矩阵记号可写为

$$\mathbf{Y} = \mathrm{Softmax}\left[\mathbf{X}\mathbf{X}^\mathsf{T}\right]\mathbf{X} \tag{12.6}$$

其中 Softmax[·] 是逐行作用的算子: 先对矩阵 $\mathbf{L}$ 中每个元素取指数, 再将每一行独立归一化使其和为 1. ♣

注

(重构新增) 定义中“查询、键、值三者都取自同一个序列”是自注意力区别于一般注意力的关键: 序列既是信息来源, 也是查询来源, 因此每个 token 的输出都依赖于同序列中其他 token 的表示. 这种“用自己检索自己”的做法使模型无须递归结构就能刻画长距离依赖——两个相距很远的 token 在一次矩阵乘法中直接相连, 路径长度为 1, 而递归神经网络需要经过中间所有时间步. 之所以用 softmax 而不是线性归一化, 是因为指数函数把点积的相对大小转化为权重的相对大小, 并且严格为正, 从而保证(12.3)与(12.4)同时成立 (见命题 12.2); 同时 softmax 可微, 使整个层可以用梯度下降训练. 这里 softmax 没有概率解释, 它只是归一化注意力权重的工具. 另外,$N \times N$ 的注意力矩阵随序列长度平方增长, 这一点在小节 12.1.8中会以计算复杂度的形式重现.

请注意, 在这种情况下,softmax 函数不存在概率解释, 它只是用来对注意力权重进行适当的归一化.

命题 12.2 (softmax 权重的归一性)

对任意 $\mathbf{x}_1, \cdots, \mathbf{x}_N$, 由(12.5)定义的 a_{nm} 满足 $a_{nm} > 0$ 与 $\sum_{m=1}^{N} a_{nm} = 1$, 从而(12.2)的 $\mathbf{y}_n$ 是输入向量的严格凸组合.

证明

指数函数取值恒正, 故分子 $\exp(\mathbf{x}_n^{\mathsf{T}}\mathbf{x}_m) > 0$, 分母是 N 个正数之和, 因而 $a_{nm} > 0$. 把分子对 m 求和即得分母, 故 $\sum_{m=1}^{N} a_{nm} = 1$. 非负性已由严格正性得到, 于是命题 12.1的条件全部满足,$\mathbf{y}_n$ 为严格凸组合.

□

注

(重构新增) 该命题说明(12.5)自动满足注意力系数的两条约束, 因而不需要任何额外的投影或截断. 它同时解释了 softmax 的“软化”作用: 当某个点积明显大于其余点积时, 对应权重趋近于 1, 注意力接近硬注意力的“检索”行为; 当所有点积相近时, 权重接近均匀分布 $1/N$, 输出接近所有输入的平均. 这两个极端分别对应“精确匹配”和“无偏好平均”, 中间的连续过渡由点积的尺度控制——这也是下一小节要引入 $1/\sqrt{D_k}$ 缩放的原因.

因此, 总的来说, 每个输入向量 $\mathbf{x}_m$ 都是通过形式为(12.2)的输入向量线性组合而转换为相应的输出向量 $\mathbf{y}_n$ 的, 其中应用于输入向量 $\mathbf{x}_m$ 的权重系数 a_{nm} 由 softmax 函数给出 [(12.5)], 该 softmax 函数是根据输入 n 的查询向量 $\mathbf{x}_n$ 与输入 m 相关的键向量 $\mathbf{x}_m$ 之间的点积 $\mathbf{x}_n^{\mathsf{T}}\mathbf{x}_m$ 进行定义的. 请注意, 如果所有输入向量都是正交的, 那么每个输出向量都等于相应的输入向量, 从而在 $m = 1, \cdots, N$ 的情况下,$\mathbf{y}_m = \mathbf{x}_m$(见练习 12.3).

注

(重构新增) 这里“正交即恒等”的说法需要精确化. 由(12.5), 正交性只使非对角点积为零, 而对角点积仍为 $\|\mathbf{x}_n\|^2 > 0$, 故 $a_{nn} = e^{\|\mathbf{x}_n\|^2} / \left(e^{\|\mathbf{x}_n\|^2} + N - 1\right) < 1$, 输出实际上是 $\mathbf{x}_n$ 与其余向量均值的凸组合, 只有当事先对输入作归一化并使 $\|\mathbf{x}_n\|$ 很大 (或注意力温度趋于零) 时,$\mathbf{y}_n$ 才趋于 $\mathbf{x}_n$. 严格的反例与可证的修正版本见练习 12.3的解答. 保留这一讨论的意义在于: 真实模型中注意力权重很少接近 one-hot, 输出总是“部分混合”的, 这正是注意力聚合上下文信息的方式.

我们可以使用数据矩阵 $\mathbf{X}$ 和类似的 $N \times D$ 输出矩阵 $\mathbf{Y}$(其行由 $\mathbf{y}_m$ 给出) 将(12.2)用矩阵符号写出, 即(12.6).

其中,Softmax[$\mathbf{L}$] 是一个算子, 作用是取矩阵 $\mathbf{L}$ 中每个元素的指数, 然后将每一行独立归一化, 使它们的和为 1. 自此, 为了清晰易懂, 我们将讨论的重点放在矩阵符号上.

这个过程叫作自注意力 (self-attention), 因为我们使用相同的序列来确定查询、键和值. 在后续内容中, 我们还会遇到这种注意力的变体. 此外, 由于查询向量和键向量之间的相似度度量是由点积给出的, 因此这个过程又叫作点积自注意力 (dot-product self-attention).

12.1.4 网络参数

目前, 由于没有可调整的参数, 从输入向量 $\mathbf{x}_n$ 到输出向量 $\mathbf{y}_n$ 的变换是固定的, 且不具备从数据中学习的能力. 此外,token 向量 $\mathbf{x}_n$ 的每个特征值在确定注意力系数时发挥相同的作用, 而我们希望网络在确定 token 相似度时能够灵活地将注意力更多地集中在某些特征上. 通过定义由原始向量 (形式如下) 的线性变换给出的修正特征向量, 我们可以解决这两个问题:

$$\tilde{\mathbf{X}} = \mathbf{X}\mathbf{U} \tag{12.7}$$

其中 $\mathbf{U}$ 是可学习权重参数的 $D \times D$ 矩阵, 类似于标准神经网络中的“层”, 从而得到一个修正后的变换, 其形式为

$$\mathbf{Y} = \text{Softmax}\left[\mathbf{X}\mathbf{U}\mathbf{U}^{\mathsf{T}}\mathbf{X}^{\mathsf{T}}\right]\mathbf{X}\mathbf{U} \tag{12.8}$$

虽然这种方法更灵活, 但它有一个特性, 即如下矩阵是对称的:

$$\mathbf{X}\mathbf{U}\mathbf{U}^{\mathsf{T}}\mathbf{X}^{\mathsf{T}} \tag{12.9}$$

而希望注意力机制支持明显的不对称. 例如, 我们可能会认为“斧头”与“工具”的关联性很强, 因为每一把斧头都是一个工具; 而“工具”与“斧头”的关联性很弱, 因为除了斧头, 还有许多其他种类的工具. 虽然 softmax 函数意味着得到的注意力权重矩阵本身并不对称, 但我们可以通过允许查询和键具有独立参数来创建一个更加灵活的网络. 此外,(12.8)使用相同的参数矩阵 $\mathbf{U}$ 来定义值向量和注意力系数, 这似乎又是一个不受欢迎的限制.

通过定义单独的查询矩阵、键矩阵和值矩阵, 并让每个矩阵都有自己独立的线性变换, 我们可以克服这些限制:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)} \tag{12.10}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^{(k)} \tag{12.11}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^{(v)} \tag{12.12}$$

其中, 权重矩阵 $\mathbf{W}^{(q)}$、$\mathbf{W}^{(k)}$ 和 $\mathbf{W}^{(v)}$ 表示在训练最终的 Transformer 架构时将学习到的参数. 矩阵 $\mathbf{W}^{(k)}$ 的维度为 $D \times D_k$, 其中 D_k 是键向量的长度. 矩阵 $\mathbf{W}^{(q)}$ 的维度 $D \times D_q$ 必须与矩阵 $\mathbf{W}^{(k)}$ 的维度相同, 因为只有这样才能在查询向量和键向量之间形成点积. 为此, 可以设置 $D_k = D$. 同样,$\mathbf{W}^{(v)}$ 是一个维度为 $D \times D_v$ 的矩阵, 其中 D_v 是输出向量的长度. 如果设置 $D_v = D$, 使输出表示和输入表示具有相同的维度, 则有助于加入残差连接, 我们稍后将对此进行讨论. 此外, 如果每个层的维度相同, 则多个 Transformer 层可以叠加在一起 (参见小节 12.1.7). 通过对(12.6)进行泛化, 我们可以得到

$$\mathbf{Y} = \text{Softmax}\left[\mathbf{Q}\mathbf{K}^{\mathsf{T}}\right]\mathbf{V} \tag{12.13}$$

定义 12.3 (查询、键与值)

设 $\mathbf{X} \in \mathbb{R}^{N\times D}$ 为数据矩阵,$\mathbf{W}^{(q)} \in \mathbb{R}^{D\times D_q}$、$\mathbf{W}^{(k)} \in \mathbb{R}^{D\times D_k}$、$\mathbf{W}^{(v)} \in \mathbb{R}^{D\times D_v}$ 为可学习的权重矩阵, 且 $D_q = D_k$. 称

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}^{(v)} \tag{12.14}$$

分别为**查询矩阵**、**键矩阵**与**值矩阵**; 它们的第 n 行分别是第 n 个 token 的查询向量、键向量与值向量. 由它们按(12.13)给出的映射称为 (单头) 注意力层的输出.

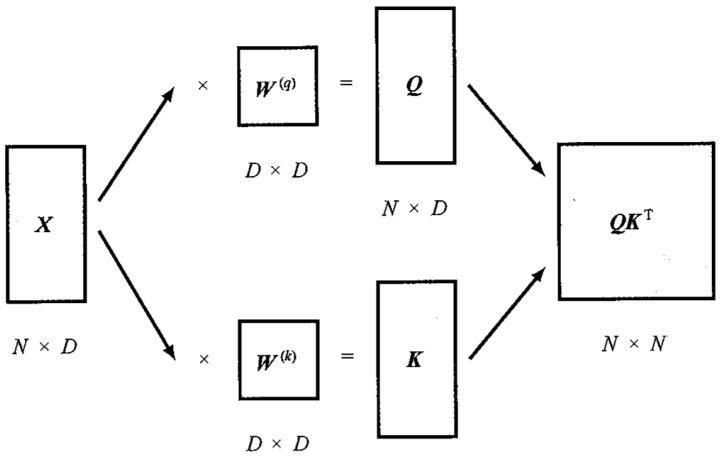


图 12.4: 决定 Transformer 中注意力系数的矩阵 $\mathbf{QK}^{\mathsf{T}}$ 的计算示意图. 分别通过(12.10)和(12.11)对输入 $\mathbf{X}$ 进行变换, 得到查询矩阵 $\mathbf{Q}$ 和键矩阵 $\mathbf{K}$, 然后将它们相乘.

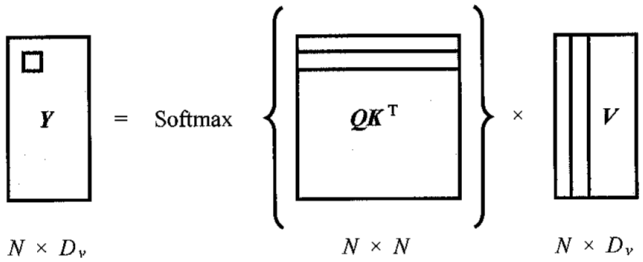


图 12.5: 在给定查询矩阵 $\mathbf{Q}$、键矩阵 $\mathbf{K}$ 和值矩阵 $\mathbf{V}$ 的情况下计算注意力层输出的示意图. 输出矩阵 $\mathbf{Y}$ 中突出显示位置的条目分别由 Softmax[$\mathbf{QK}^{\mathsf{T}}$] 和 $\mathbf{V}$ 矩阵中突出显示位置的行和列的点积得出.

注

(重构新增) 把“查询、键、值”拆成三个独立线性变换, 是为了让模型能够学习“用什么去检索”“被检索的东西如何被表示”“检索到之后取回什么”这三件事, 而不像(12.8)那样被迫用同一个 $\mathbf{U}$ 兼任. 若三者共用参数, 注意力矩阵 $\mathbf{XUU}^{\mathsf{T}}\mathbf{X}^{\mathsf{T}}$ 天然对称, 模型就无法表示“斧头依赖工具、工具不依赖斧头”这类单向关联; 三个独立矩阵打破了这一对称性. 条件 $D_q = D_k$ 是点积 $\mathbf{QK}^{\mathsf{T}}$ 有意义的前提: 查询向量与键向量必须等长才能逐分量相乘求和; 而 D_v 可以自由选择, 选 $D_v = D$ 只是为了让输出与输入同维, 从而能够相加形成残差连接——这是小节 12.1.7的需要, 而非注意力本身的要求. 最后要注意,$N \times D$ 的输入经线性变换后仍按 token 逐行处理, 参数在所有 token 之间共享, 这正是后面复杂度分析中参数量只有 $O(D^2)$ 的原因.

其中矩阵 $\mathbf{QK}^{\mathsf{T}}$ 的维度为 $N \times N$, 且矩阵 $\mathbf{Y}$ 的维度为 $N \times D_v$. 矩阵 $\mathbf{QK}^{\mathsf{T}}$ 的计算如图 12.4所示, 而矩阵 $\mathbf{Y}$ 的计算如图 12.5所示.

实际上, 我们还可以在这些线性变换中加入偏置参数. 不过, 偏置参数也可以包含在权重矩阵中, 就像我们对标准神经网络所做的那样 (参见小节 6.2.1), 方法是在数据矩阵 $\mathbf{X}$ 中增加一列 1, 然后在权重矩阵中增加额外一行参数来表示偏置. 自此, 我们将把偏置参数视为隐参数, 以免混淆符号.

与传统的神经网络相比, 信号路径的激活值之间存在乘法关系. 标准的神经网络会将激活值乘以固定权重, 而这里的激活值则被乘以依赖数据的注意力系数. 这意味着如果选择特定的输入向量, 其中一个注意力系数接近于零, 那么产生的信号路径将忽略相应的输入信号, 因此不会对网络输出产生任何影响. 相比之下, 如果一个标准的神经网络学会了忽略某个特定的输入或隐藏单元变量, 那么该神经网络就会对所有输入向量都这样做.

12.1.5 缩放自注意力

我们还可以对自注意力层进行最后一次改进. 回想一下, 当输入量较大时,softmax 函数的梯度会呈指数减小, 就像 tanh 函数或逻辑斯谛 sigmoid 函数一样. 为了避免出现这种情况, 我们可以在应用 softmax 函数之前重新缩放查询向量和键向量的乘积. 为了得到合适的缩放结果, 请注意, 如果查询向量和键向量的元素都是均值为 0 且方差为单位方差的独立随机数, 则点积的方差就是 D_k(见练习 12.4). 因此, 我们使用 D_k 的平方根所给出的标准差对 Softmax 的参数进行归一化处理, 从而注意力层的输出形式为

$$\text{Attention}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{Softmax}\left[\frac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{D_k}}\right]\mathbf{V} \tag{12.15}$$

这就是所谓的缩放点积自注意力 (scaled dot-product self-attention), 也是自注意力神经网络层的最终形式. 这一层的结构如图 12.6所示, 算法过程见下面的算法 12.1.

命题 12.3 (缩放点积注意力的输出形式)

设 $\mathbf{Q}=\mathbf{X}\mathbf{W}^{(q)}$、$\mathbf{K}=\mathbf{X}\mathbf{W}^{(k)}$、$\mathbf{V}=\mathbf{X}\mathbf{W}^{(v)}$ 如(12.14),$D_q=D_k$. 则按(12.15)定义的注意力层输出与(12.13)的差别仅在于 softmax 的自变量被除以常数 $\sqrt{D_k}$: 对每个 n, 其输出 $\mathbf{y}_n$ 仍是 $\mathbf{v}_1,\cdots,\mathbf{v}_N$ 的凸组合, 权重为

$$a_{nm} = \frac{\exp\left(\mathbf{q}_n^{\mathsf{T}}\mathbf{k}_m/\sqrt{D_k}\right)}{\sum_{m'=1}^{N}\exp\left(\mathbf{q}_n^{\mathsf{T}}\mathbf{k}_{m'}/\sqrt{D_k}\right)}. \tag{12.16}$$

♠

证明

记 $\mathbf{S}=\mathbf{Q}\mathbf{K}^{\mathsf{T}}$, 其 (n,m) 元为 $s_{nm}=\mathbf{q}_n^{\mathsf{T}}\mathbf{k}_m$.(12.15)中 softmax 的自变量为 $s_{nm}/\sqrt{D_k}$, 按 Softmax[·] 的逐行定义即得(12.16), 其分母为正且分子恒正, 故 $a_{nm}>0$、$\sum_m a_{nm}=1$. 又 $\mathbf{Y}$ 的第 n 行为 $\sum_m a_{nm}\mathbf{v}_m$, 即 $\mathbf{v}_m$ 的凸组合.

□

注

(重构新增) 缩放的目的是把 softmax 的自变量重新标准化到“单位量级”. 若 $\mathbf{q}_n$ 与 $\mathbf{k}_m$ 各分量独立、均值 0、方差 1, 则点积 s_{nm} 是 D_k 个独立同分布项之和, 其均值为 0、方差为 D_k;D_k 越大,s_{nm} 的典型绝对值越大,softmax 越接近“赢者通吃”的硬选择, 其 Jacobi 矩阵随之退化, 梯度几乎为零. 除以标准差 $\sqrt{D_k}$ 后自变量方差回到 1, 无论 D_k 取多大,softmax 都工作在梯度良好的区间. 这一改动不改变输出的凸组合结构, 只改变权重的尖锐程度, 因此命题 12.2的结论原样保留. 还要注意, 缩放常数由 D_k 而非 D 决定, 因为决定点积量级的是键向量的长度.

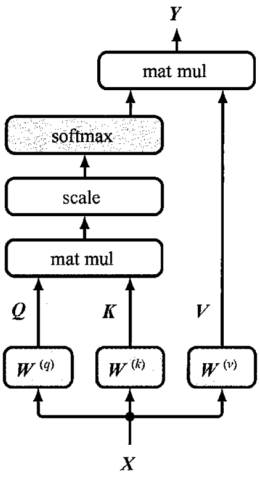


图 12.6: 缩放点积自注意力神经网络层中的信息流. 此处“mat mul”表示矩阵乘法, 而“scale”指的是使用 $\sqrt{D_k}$ 对 Softmax 的参数进行归一化处理. 该结构构成了一个单一注意力“头”.

算法 12.1　缩放点积自注意力

Input: 一组 token $\mathbf{X} \in \mathbb{R}^{N\times D} : \{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$
　　权重矩阵 $\{\mathbf{W}^{(q)}, \mathbf{W}^{(k)}\} \in \mathbb{R}^{D\times D_k}$ 与 $\mathbf{W}^{(v)} \in \mathbb{R}^{D\times D_v}$
Output: Attention$(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{N\times D} : \{\mathbf{y}_1, \cdots, \mathbf{y}_N\}$
$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}$　// 计算查询矩阵 $\mathbf{Q} \in \mathbb{R}^{N\times D_k}$
$\mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}$　// 计算键矩阵 $\mathbf{K} \in \mathbb{R}^{N\times D_k}$
$\mathbf{V} = \mathbf{X}\mathbf{W}^{(v)}$　// 计算值矩阵 $\mathbf{V} \in \mathbb{R}^{N\times D_v}$
return Attention$(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left[\dfrac{\mathbf{Q}\mathbf{K}^{\mathsf{T}}}{\sqrt{D_k}}\right]\mathbf{V}$

12.1.6 多头注意力

注意力层允许输出向量关注输入向量的依赖数据模式, 称作注意力头 (attention head). 然而, 可能同时存在多种相关的注意力模式. 例如, 在自然语言中, 有些注意力模式可能与时态有关, 而另一些注意力模式则可能与词汇有关. 使用单一注意力头会导致这些效应的平均化. 对此, 我们可以并行使用多个注意力头. 这些注意力头由结构相同的单个注意力头的副本组成, 具有独立的学习参数, 用于控制查询矩阵、键矩阵和值矩阵的计算. 这类似于在卷积网络的每一层中使用多个不同的过滤器.

假设我们有 H 个注意力头, 以 $h = 1, \cdots, H$ 进行编号, 形式为

$$\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h) \tag{12.17}$$

其中 Attention$(\cdot,\cdot,\cdot)$ 由(12.15)给出, 使用如下方法为每个注意力头定义单独的查询矩阵、键矩阵和值矩阵:

$$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(q)}, \qquad \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(k)}, \qquad \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(v)} \tag{12.18}$$

首先将各个注意力头合并成一个单一矩阵, 然后使用矩阵 $\mathbf{W}^{(o)}$ 对结果进行线性变换, 从而得到一个合并输出, 形式为

$$\mathbf{Y}(\mathbf{X}) = \text{Concat}\left[\mathbf{H}_1, \cdots, \mathbf{H}_H\right]\mathbf{W}^{(o)} \tag{12.19}$$

定义 12.4 (多头注意力)

设 H 为正整数, 对每个 $h = 1, \cdots, H$ 取独立的参数 $\mathbf{W}_h^{(q)}, \mathbf{W}_h^{(k)} \in \mathbb{R}^{D\times D_k}$ 与 $\mathbf{W}_h^{(v)} \in \mathbb{R}^{D\times D_v}$, 按(12.17)与(12.18)计算各头的输出 $\mathbf{H}_h$. 把 $\mathbf{H}_1, \cdots, \mathbf{H}_H$ 按列拼接得到 $N \times HD_v$ 的矩阵, 再右乘 $\mathbf{W}^{(o)} \in \mathbb{R}^{HD_v\times D}$, 所得(12.19)称为**多头注意力** (multi-head attention). ♣

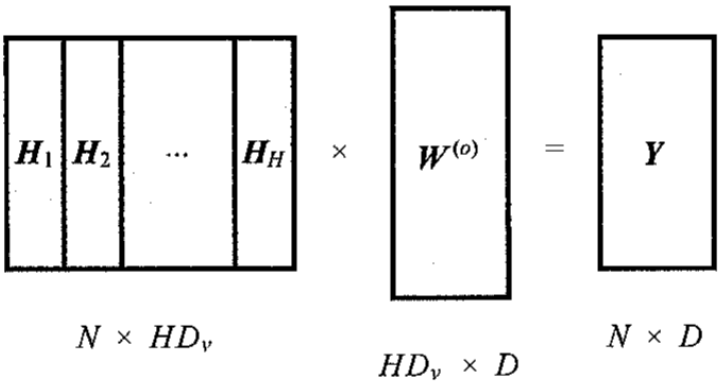


图 12.7: 多头注意力网络的结构. 每个注意力头都包含图 12.6所示的结构, 并有自己的键、查询和值参数. 将各个注意力头的输出合并起来, 然后线性投射回输入数据维度.

注

(重构新增) 多头的动机是“分工”: 不同的头可以用不同的线性投影去捕捉不同类型的依赖 (例如一个头跟踪时态、另一个头跟踪词性), 而单一头只能给出这些模式的加权平均, 无法同时精确表达. 因为每个头都有自己的一套 $\mathbf{W}_h^{(q)}, \mathbf{W}_h^{(k)}, \mathbf{W}_h^{(v)}$, 头与头之间不共享参数, 所以它们学到的是互不相同的注意力矩阵. 拼接后的维度为 HD_v, 再由 $\mathbf{W}^{(o)}$ 映射回 D 维, 这一“先拼后投”的结构保证了输出维度与输入维度一致, 从而多头注意力可以像单头一样嵌入残差连接. 通常取 $D_v = D/H$, 于是拼接矩阵的维度恰为 $N \times D$, 总参数量与单头情形保持同一量级——这也是多头并不显著增加计算量的原因.$\mathbf{W}^{(o)}$ 与各头的参数一起在训练中学习, 它不是固定的平均算子.

多头注意力网络的结构如图 12.7所示, 算法过程见下面的算法 12.2.

算法 12.2　多头注意力

Input: 一组 token $\mathbf{X} \in \mathbb{R}^{N\times D}$: $\{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$
　查询权重矩阵 $\{\mathbf{W}_1^{(q)}, \cdots, \mathbf{W}_H^{(q)}\} \in \mathbb{R}^{D\times D}$
　键权重矩阵 $\{\mathbf{W}_1^{(k)}, \cdots, \mathbf{W}_H^{(k)}\} \in \mathbb{R}^{D\times D}$
　值权重矩阵 $\{\mathbf{W}_1^{(v)}, \cdots, \mathbf{W}_H^{(v)}\} \in \mathbb{R}^{D\times D_v}$
　输出权重矩阵 $\mathbf{W}^{(o)} \in \mathbb{R}^{HD_v\times D}$
Output: $\mathbf{Y} \in \mathbb{R}^{N\times D}$: $\{\mathbf{y}_1, \cdots, \mathbf{y}_N\}$
// 为每个注意力头计算自注意力 (参见算法 12.1)
for $h = 1, \cdots, H$ **do**
　$\mathbf{Q}_h = \mathbf{X}\mathbf{W}_h^{(q)}, \mathbf{K}_h = \mathbf{X}\mathbf{W}_h^{(k)}, \mathbf{V}_h = \mathbf{X}\mathbf{W}_h^{(v)}$
　$\mathbf{H}_h = \text{Attention}(\mathbf{Q}_h, \mathbf{K}_h, \mathbf{V}_h)$　// $\mathbf{H}_h \in \mathbb{R}^{N\times D_v}$
end for
$\mathbf{H} = \text{Concat}[\mathbf{H}_1, \cdots, \mathbf{H}_H]$　// 合并注意力头
return $\mathbf{Y}(\mathbf{X}) = \mathbf{H}\mathbf{W}^{(o)}$

每个矩阵 $\mathbf{H}_h$ 的维度为 $N \times D_v$, 因此合并矩阵的维度为 $N \times HD_v$. 在使用维度为 $HD_v \times D$ 的线性矩阵 $\mathbf{W}^{(o)}$ 进行变换后, 最终得到维度为 $N \times D$ 的输出矩阵 $\mathbf{Y}$, 它与原始输入矩阵 $\mathbf{X}$ 的维度相同. 矩阵 $\mathbf{W}^{(o)}$ 的元素是在训练阶段与查询矩阵、键矩阵和值矩阵一起学习的. 通常情况下,D_v 选择为与 D/H 相等, 从而使得到的合并矩阵的维度为 $N \times D$.图 12.8展示了多头注意力层的信息流.

请注意, 前文给出的多头注意力表述与研究文献中使用的表述相同, 我们在每个注意力头的 $\mathbf{W}^{(v)}$ 矩阵与输出矩阵 $\mathbf{W}^{(o)}$ 的连续乘法中包含了一些冗余. 在消除这些冗余后, 多头自注意力层就可以写成每个注意力头的贡献之和 (见练习 12.5).

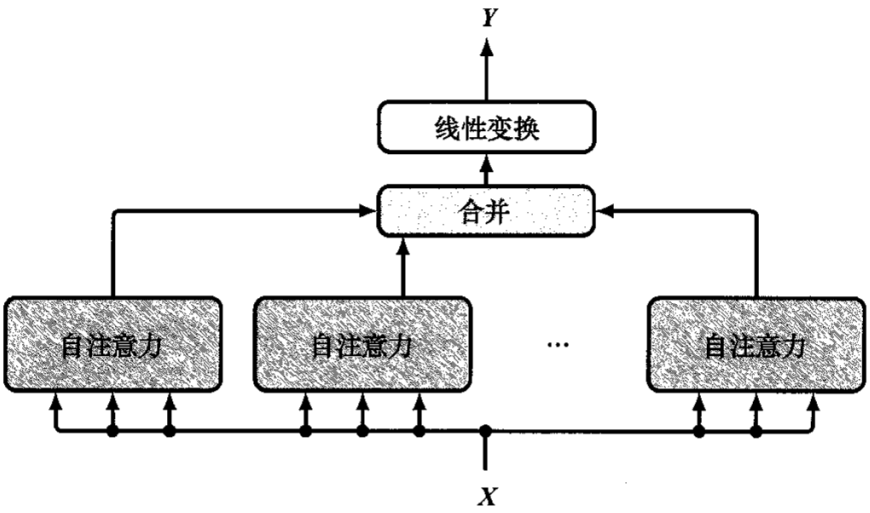


图 12.8: 多头注意力层的信息流.

12.1.7 Transformer 层

多头自注意力构成了 Transformer 网络的核心架构元素. 我们知道, 神经网络从深度结构中获益匪浅, 因此我们希望将多个自注意力层堆叠在一起. 为了提高训练效率, 我们可以引入绕过多头结构的残差连接 (参见节 9.5). 为此, 我们要求输出维度与输入维度相同 (参见小节 7.4.3). 然后进行层归一化 (layer normalization)(Ba, Kiros, and Hinton, 2016), 从而提高训练效率. 由此产生的变换可以写成

$$\mathbf{Z} = \text{LayerNorm}\,[\mathbf{Y}(\mathbf{X}) + \mathbf{X}] \tag{12.20}$$

其中 $\mathbf{Y}$ 由(12.19)定义. 有时, 层归一化会被预归一化取代, 即层归一化在多头注意力之前而不是之后进行, 因为这可以带来更有效的优化. 在这种情况下, 我们得出

$$\mathbf{Z} = \mathbf{Y}(\mathbf{X}') + \mathbf{X}, \qquad \text{其中} \quad \mathbf{X}' = \text{LayerNorm}\,[\mathbf{X}] \tag{12.21}$$

在所有情况下,$\mathbf{Z}$ 的维度都与输入矩阵 $\mathbf{X}$ 的维度相同.

我们已经看到, 注意力机制创建了值向量的线性组合, 然后用它产生输出向量. 此外, 这些值向量是输入向量的线性函数, 因此我们可以看到, 注意力层的输出受限于输入的线性组合. 非线性通过注意力权重引入, 因此输出将通过 softmax 函数非线性地依赖于输入, 但输出向量仍然受限于输入向量所跨的子空间, 这限制了注意力层的表达能力. 我们可以使用一个标准的非线性神经网络 (有 D 个输入和 D 个输出) 对每一层的输出进行后处理, 用 MLP[·] 表示 "多层感知机" , 从而提高 Transformer 的灵活性. 例如, 这可能包括一个具有 ReLU 隐藏单元的两层全连接网络. 当我们这样做时, 需要保持 Transformer 处理不同长度序列的能力. 为此, 我们需要对每个输出向量 (与 $\mathbf{Z}$ 的行相对应) 应用相同的共享网络. 同样, 也可以通过使用残差连接来改进该神经网络层. 它还包括层归一化, 从而 Transformer 层的最终输出形式为

$$\tilde{\mathbf{X}} = \text{LayerNorm}\,[\text{MLP}[\mathbf{Z}] + \mathbf{Z}] \tag{12.22}$$

这就产生了 Transformer 层的整体架构, 如图 12.9所示, 算法过程见下面的算法 12.3. 同样, 我们也可以使用预归一化, 在这种情况下,Transformer 层的最终输出形式为

$$\tilde{\mathbf{X}} = \text{MLP}[\mathbf{Z}'] + \mathbf{Z}, \qquad \text{其中} \quad \mathbf{Z}' = \text{LayerNorm}\,[\mathbf{Z}] \tag{12.23}$$

定义 12.5 (Transformer 层)

设 $\mathbf{X} \in \mathbb{R}^{N\times D}$,$\mathbf{Y}$ 为(12.19)给出的多头自注意力输出,MLP[·] 为逐行作用、参数在各 token 间共享的多层感知机. 由(12.20)与(12.22)(或预归一化形式(12.21)与(12.23)) 复合而成的映射 $\mathbf{X} \mapsto \tilde{\mathbf{X}}$ 称为一个 **Transformer 层**. 多个这样的层首尾相接即构成 Transformer 网络, 层与层之间不共享权重与偏置. ♣

注

(重构新增) Transformer 层由两个子层组成, 动机与条件都很明确. 第一, 残差连接 $\mathbf{Y}(\mathbf{X}) + \mathbf{X}$ 要求 $\mathbf{Y}$ 与 $\mathbf{X}$ 同维, 这正是前面要求 $D_v = D$ 的理由; 残差路径使梯度可以直接回传, 也让位置信息在层间传递时不被淹没. 第二, 层归一化在特征维度上对每个 token 独立地做标准化, 因此与序列长度无关, 可以在不同长度的序列上使用, 这正是它优于依赖批统计量的批量归一化之处. 第三, 注意力只做输入的线性组合加 softmax 加权, 单靠它无法表示特征之间的非线性交互; 逐行共享的 MLP 补上了这一表达能力, 而“逐行共享”又保证了输出不依赖序列长度、参数不随 N 增长. 预归一化把 LayerNorm 移到子层之前, 使残差路径上完全没有归一化算子, 梯度可以无损地沿残差路径回传, 因此深层模型更容易优化; 两种形式的输出维度都与 $\mathbf{X}$ 相同, 故可任意互换而不破坏堆叠.

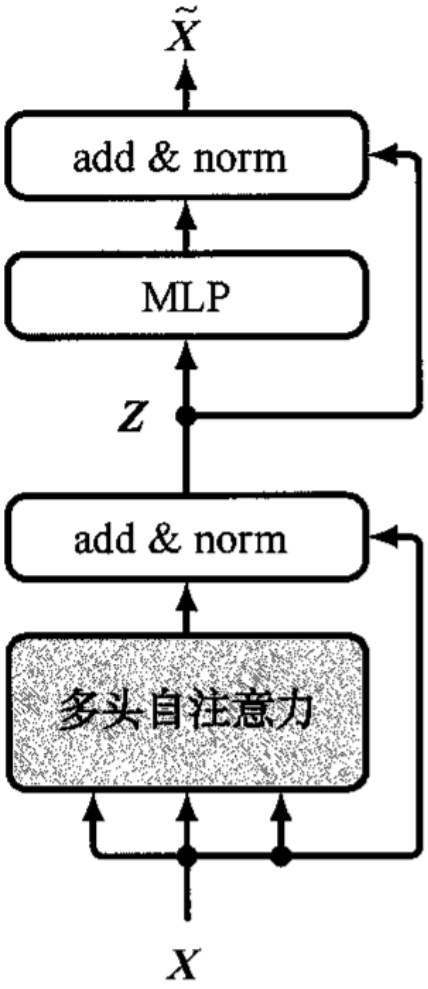


图 12.9: Transformer 层的整体架构. 此处“MLP”代表多层感知机, “add & norm”则代表残差连接和随后的层归一化.

算法 12.3　Transformer 层

Input: 一组 token $\mathbf{X} \in \mathbb{R}^{N\times D}$: $\{\mathbf{x}_1, \cdots, \mathbf{x}_N\}$
　　多头自注意力层参数
　　前馈网络参数
Output: $\tilde{\mathbf{X}} \in \mathbb{R}^{N\times D}$: $\{\tilde{\mathbf{x}}_1, \cdots, \tilde{\mathbf{x}}_N\}$
$\mathbf{Z}$ = LayerNorm [$\mathbf{Y}(\mathbf{X}) + \mathbf{X}$]　// $\mathbf{Y}(\mathbf{X})$ 由算法 12.2 给出
$\tilde{\mathbf{X}}$ = LayerNorm [MLP[$\mathbf{Z}$] + $\mathbf{Z}$]　// 共享网络
return $\tilde{\mathbf{X}}$

在典型的 Transformer 模型中, 有多个这样的层叠加在一起. 虽然不同层之间不共享权重和偏置, 但各层通常具有相同的结构.

12.1.8 计算复杂性

到目前为止我们所讨论的注意力层包含一个向量集 (其中有 N 个向量, 每个向量的长度为 D), 我们将它映射到了另一个具有相同维度的向量集 (其中也有 N 个向量). 因此, 输入和输出的总维度均为 ND. 如果我们使用标准的全连接神经网络将输入值映射到输出值, 就会有 $O(N^2D^2)$ 个独立参数. 同样, 计算通过这样一个网络进行一次前向传递的成本也是 $O(N^2D^2)$.

在注意力层中, 矩阵 $\mathbf{W}^{(q)}$、$\mathbf{W}^{(k)}$ 和 $\mathbf{W}^{(v)}$ 在输入 token 之间共享, 因此独立参数的数量为 $O(D^2)$, 这里假设 $D_k \approx D_v \approx D$. 由于有 N 个输入 token, 在自注意力层中计算点积的步骤数为 $O(N^2D)$. 我们可以把自注意力层看作一个稀疏矩阵, 其中的参数在矩阵的特定块之间共享. 随后的神经网络层 (它有 D 个输入和 D 个输出) 的成本为 $O(D^2)$. 由于它在 token 之间共享, 其复杂度与 N 成线性关系, 因此这一层的总成本为 $O(ND^2)$.

命题 12.4 (Transformer 层的计算复杂度)

设序列长度为 N、模型维度为 D, 且 $D_k \approx D_v \approx D$. 则单头自注意力层的参数量为 $O(D^2)$、计算量为 $O(N^2D)$; 逐行共享的两层 MLP(D 个输入、D 个输出) 的参数量与计算量均为 $O(D^2)$ 与 $O(ND^2)$; 整个 Transformer 层的计算量为 $O(N^2D + ND^2)$, 参数量为 $O(D^2)$. 相比之下, 把 ND 维输入映射到 ND 维输出的全连接网络需要 $O(N^2D^2)$ 个参数与同阶的计算量.

♠

证明

单个 $\mathbf{W}^{(q)}$ 有 $D \times D_k$ 个元素, 三个投影矩阵合计 $O(D^2)$ 个参数, 且与 N 无关. 计算 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 各需 $O(ND^2)$ 次乘法; 计算 $\mathbf{QK}^\mathsf{T}$ 需对 N^2 个元素各做长度为 D 的点积, 共 $O(N^2D)$;softmax 逐行处理 $N \times N$ 个元素, 为 $O(N^2)$; 右乘 $\mathbf{V}$ 再需 $O(N^2D)$. 故注意力部分为 $O(N^2D + ND^2)$.MLP 对每个 token 做固定的 $O(D^2)$ 运算,N 个 token 合计 $O(ND^2)$, 因参数共享其参数量与 N 无关, 为 $O(D^2)$. 两者相加即得 $O(N^2D + ND^2)$. 而全连接层有 ND 个输入与 ND 个输出, 权重矩阵为 $ND \times ND$, 故参数量与计算量均为 $O(N^2D^2)$.

□

注

(重构新增) 这个对比说明了 Transformer 效率优势的确切来源: 参数共享. 全连接层要为每一对 (输入位置, 输出位置) 单独学习权重, 参数量随 N^2 爆炸; 注意力层只用一套 $O(D^2)$ 的投影, 位置之间的交互由数据决定的内积完成. 代价是注意力部分仍随 N^2 增长, 因此长序列时 $O(N^2D)$ 项会主导——这正是后续各种稀疏化、低秩近似与线性注意力变体的出发点. 两项的相对大小由 N 与 D 的比值决定: 当 $N \ll D$ 时 MLP 项 $O(ND^2)$ 占优, 当 $N \gg D$ 时注意力项 $O(N^2D)$ 占优. 此外, 注意力层的计算可以按 token 完全并行, 而递归神经网络必须逐步串行, 这一硬件友好性才是“效率更高”在实际训练中的主要体现, 复杂度本身只说明渐近阶.

根据 N 和 D 的相对大小,Transformer 层或多层感知机层有可能支配计算成本. 与全连接网络相比,Transformer 层的计算效率更高.Transformer 架构的许多变体已被提出 (Lin et al., 2021; Phuong and Hutter, 2022), 包括旨在提高

效率的修正版 (Tay et al., 2020).

12.1.9 位置编码

在 Transformer 架构中, 矩阵 $\mathbf{W}^{(q)}$、$\mathbf{W}^{(k)}$ 和 $\mathbf{W}^{(v)}$ 由输入的所有 token 共用. 同样, 紧随其后的神经网络 (前馈神经网络) 也由输入的所有 token 共用. 因此,Transformer 具有这样的特性, 即对输入 token(也就是 $\mathbf{X}$ 的行) 进行排序会导致输出矩阵 $\bar{\mathbf{X}}$ 的行也被排序. 换句话说,Transformer 在输入排列方面是等变化的 (equivariant). 网络架构中的参数共享有利于 Transformer 的大规模并行处理, 也使网络能够像学习短距离依赖关系一样有效地学习远距离依赖关系. 然而, 当我们考虑序列数据 (如自然语言中的词) 时, 不依赖 token 顺序就成了一个主要的限制, 因为 Transformer 学习到的表示法将与输入 token 的顺序无关. 句子 “The food was bad, not good at all” 和 “The food was good, not bad at all” 包含相同的 token, 但由于 token 的排序不同, 这两个句子的含义也截然不同. 显然,token 顺序对于包括自然语言处理在内的大多数顺序处理任务至关重要, 因此我们需要找到一种方法来将有关 token 顺序的信息注入网络.

命题 12.5 (Transformer 层的排列等变性)

设 $\mathbf{P} \in \mathbb{R}^{N\times N}$ 为置换矩阵. 若 Transformer 层不含位置编码, 则对任意输入 $\mathbf{X}$ 有 $\mathrm{TL}[\mathbf{PX}] = \mathbf{P}\,\mathrm{TL}[\mathbf{X}]$, 即输出随输入的 token 排列同步置换.

♠

证明

置换成 $\mathbf{PX}$ 后,$\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 分别变为 $\mathbf{PQ}, \mathbf{PK}, \mathbf{PV}$. 注意力矩阵变为 $(\mathbf{PQ})(\mathbf{PK})^{\mathsf{T}} = \mathbf{PQK}^{\mathsf{T}}\mathbf{P}^{\mathsf{T}}$, 逐行 softmax 与左乘 $\mathbf{P}$ 可交换, 故注意力输出为 $\mathbf{P}\,\mathrm{Softmax}[\mathbf{QK}^{\mathsf{T}}]\,\mathbf{P}^{\mathsf{T}}\mathbf{PV} = \mathbf{PY}$. 残差与层归一化都是逐行运算, 与左乘置换矩阵可交换;MLP 亦逐行作用, 故 $\mathrm{MLP}[\mathbf{PZ}] = \mathbf{P}\,\mathrm{MLP}[\mathbf{Z}]$. 逐层归纳即得结论.

□

注

(重构新增) 等变性既是优点也是限制. 作为优点, 它意味着注意力对 “任意两个 token 之间的关系” 建模, 与它们相距多远无关, 因此长距离依赖与短距离依赖被同等对待——这是 Transformer 相对递归神经网络的根本改进. 作为限制, 它也意味着模型对词序完全无知: 把 “狗咬人” 与 “人咬狗” 的 token 集合原样输入, 输出只是同样被置换的两组表示, 模型无法区分二者. 解决方式不是修改架构 (那会破坏等变性带来的参数共享与并行性), 而是在数据上注入位置信息, 使原本相同的 token 表示在不同位置取不同的值, 从而打破等变性. 命题的条件 “不含位置编码” 是本质的: 一旦加上依赖位置的向量,$\mathbf{PX}$ 与 $\mathbf{X}$ 的位置编码不再对应, 等变性即被有意破坏.

由于想要保留精心构建的注意力层的强大特性, 因此我们希望在数据中对 token 顺序进行编码, 而不是在网络架构中表示 token 顺序. 为此, 我们将构建一个与每个输入位置 n 相关的位置编码向量 $\mathbf{r}_n$, 然后将其与相关输入 token 的嵌入向量 $\mathbf{x}_n$ 结合起来. 一种显而易见的方法是将这些向量合并起来, 但这会增加输入空间的维度, 进而增加所有后续注意力空间的维度, 从而显著增加计算成本. 相反, 我们可以简单地将位置向量加到 token 向量上, 从而得出

$$\tilde{\mathbf{x}}_n = \mathbf{x}_n + \mathbf{r}_n \tag{12.24}$$

这就要求位置编码向量的维度与 token 嵌入向量的维度相同.

初看起来, 在 token 向量上添加位置信息会破坏输入向量, 使得网络的任务变得更加难以完成. 不过, 从两个随机选择的不相关向量在高维空间中几乎是正交的这一现象中, 我们可以了解到这一方法为什么能够很好地发挥作用, 这表明网络能够相对独立地处理 token 的身份信息和位置信息. 另外, 请注意, 由于 Transformer 中的层与层之间都有残差连接, 因此位置信息从一层转到下一层的过程中不会丢失. 此外,Transformer 中的线性处理层能使合并表示法与加法表示法具有相似的特性.

定义 12.6 (位置编码)

设第 n 个 token 的嵌入向量为 $\mathbf{x}_n \in \mathbb{R}^D$. 若对每个位置 n 指定向量 $\mathbf{r}_n \in \mathbb{R}^D$, 并按(12.24)把 $\mathbf{r}_n$ 加到 $\mathbf{x}_n$ 上再送入 Transformer, 则称 $\mathbf{r}_n$ 为**位置编码** (positional encoding): 其元素由固定公式给出的称为正弦位置编码, 其元素作为可学习参数在训练中确定的称为习得的位置编码. ♣

注

(重构新增) 把位置信息“加”到词向量上而不是“拼接”, 首先是为了不改变模型维度: 拼接会使输入维度变为 $D + D'$, 后续所有权重矩阵都要随之增大, 计算成本上升; 相加则保持 D 不变. 相加之所以可行, 依赖高维空间的一个几何事实: 随机选取的两个无关向量内积近似为零, 因此 $\mathbf{x}_n$ 与 $\mathbf{r}_n$ 在 D 较大时近似正交, 网络可以分别从投影中读出“是哪个 token”与“在哪个位置”, 而不会严重相互干扰. 残差连接保证位置信息不会在多层堆叠中被逐层稀释, 线性层则使加法形式与拼接形式在表达能力上接近 (线性映射对拼接的分块作用等价于分别作用于两部分再相加). 需要注意, 位置编码不是注意力权重的一部分, 它只改变输入表示; 因此它不破坏命题 12.5 以外的任何性质, 却有意破坏了排列等变性本身.

接下来的任务是构建嵌入向量 $\mathbf{r}_n$. 一种简单的方法是为每个位置关联一个整数. 但这样做的问题是, 数值的大小会无限制地增加, 因此可能会严重破坏嵌入向量. 此外, 对于比训练时所用输入序列更长的新输入序列, 它可能无法很好地泛化, 因为这些输入序列涉及的编码值将超出训练时所用编码值的范围. 我们也可以为序列中的每个 token 分配一个介于 0 和 1 之间的数字, 从而保持表示法的有界性. 然而, 这种表示法对于特定位置并不是唯一的, 因为它取决于整个序列的长度.

理想的位置编码应该为每个位置提供唯一的表示, 其应该是有界的, 能泛化为较长的序列, 且应该有一致的方法来表示任意两个输入向量之间的步数, 而不管它们的绝对位置如何, 因为 token 的相对位置往往比绝对位置更重要.

位置编码的方法有很多 (Duffer, Schmitt, and Schütze, 2021). 在此, 我们介绍一种由 Vaswani et al. (2017) 引入的基于正弦函数的技术. 对于给定的位置 n, 相关位置的编码向量具有组件 r_{ni}, 其计算公式为

$$r_{ni} = \begin{cases} \sin\left(\dfrac{n}{L^{i/D}}\right), & \text{当} i \text{ 是偶数时,} \\ \cos\left(\dfrac{n}{L^{(i-1)/D}}\right), & \text{当} i \text{ 是奇数时,} \end{cases} \tag{12.25}$$

其中 L 为常数尺度.

如图 12.10(a) 所示, 嵌入向量 $\mathbf{r}_n$ 的元素由一系列波长逐渐增大的正弦和余弦函数给出.

这种编码的特点是, 向量 $\mathbf{r}_n$ 的元素都位于 $(-1, 1)$ 范围内. 这不禁让人联想到二进制数的表示方法, 最低阶位高频率地交替出现, 而随后的位则以逐渐降低的频率交替出现:

1:	0	0	0	1
2:	0	0	1	0
3:	0	0	1	1
4:	0	1	0	0
5:	0	1	0	1
6:	0	1	1	0
7:	0	1	1	1
8:	1	0	0	0
9:	1	0	0	1

不过, 对于(12.25)来说, 向量元素是连续变量而非二进制变量. 位置编码向量的热图说明见图 12.10(b).

(12.25)给出的正弦表示法有一个很好的特性, 即对于任何固定偏移 k, 位置 $n + k$ 处的编码可以表示为位置 n 处编码的线性组合, 其中系数不取决于绝对位置, 而只取决于 k 值 (见练习 12.10). 因此, 网络应该能够学会关注相对位置. 请注意, 这一特性要求编码同时使用正弦和余弦函数.

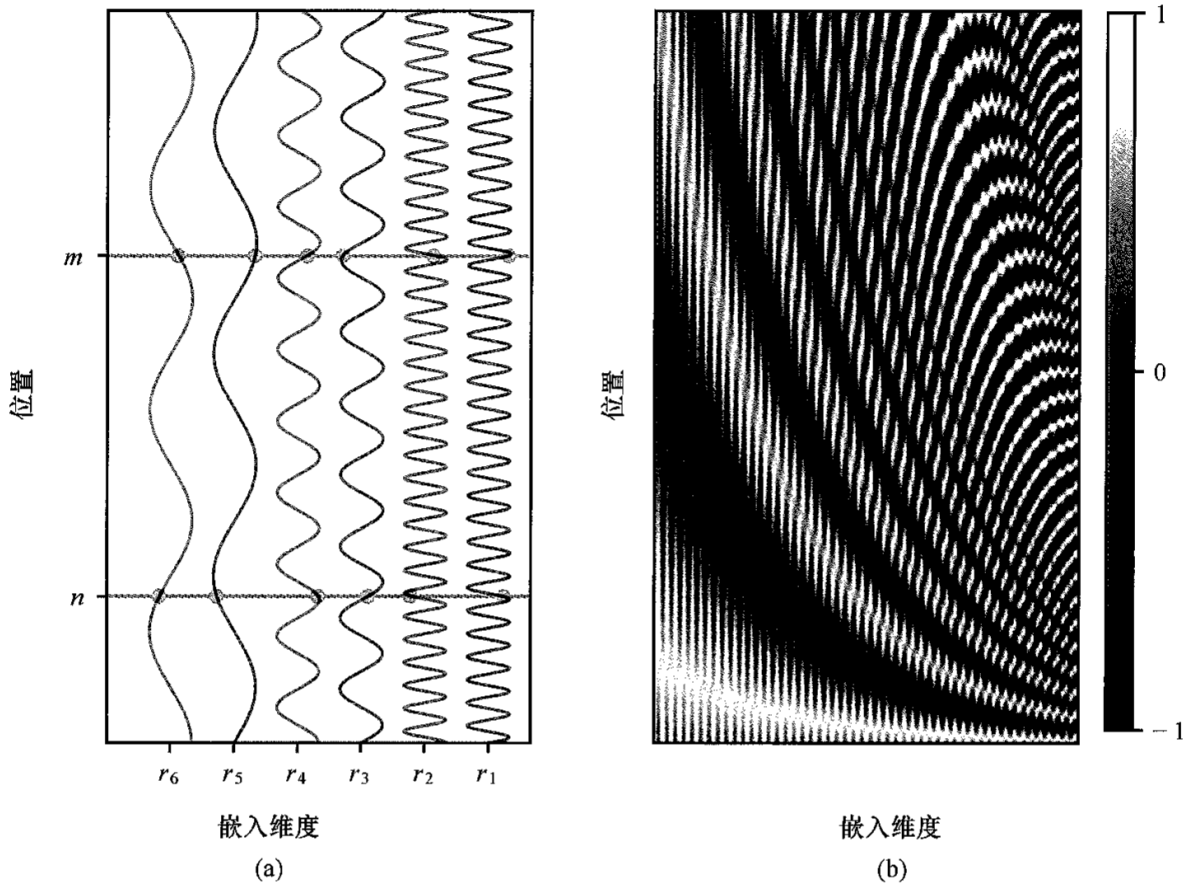


图 12.10: 由(12.25)定义并用于构建位置编码向量的函数示意图.(a) 图中横轴表示嵌入向量 $\mathbf{r}_n$ 的不同组件, 纵轴表示序列中的位置. 位置 n 和位置 m 的向量元素值均由相应的正弦和余弦曲线与水平灰线的交点表示.(b) 由(12.25)定义且对于前 $N = 200$ 个位置、$L = 30$ 的位置编码向量的热图说明, 其中维度为 $D = 100$.

命题 12.6 (正弦位置编码的平移线性性)

按(12.25)定义 r_{ni}, 并记 $\mathbf{r}_n = (r_{n1}, \cdots, r_{nD})$. 则对任意固定整数 k 与任意 n, 存在只依赖 k(而不依赖 n) 的矩阵 $\mathbf{M}_k$, 使得 $\mathbf{r}_{n+k} = \mathbf{M}_k \mathbf{r}_n$.

♠

证明

考察(12.25)中以 $\omega_i \triangleq L^{-i/D}$(奇数 i 时为 $L^{-(i-1)/D}$) 为角频率的一对分量. 由和角公式, 对同一角频率 ω 有

$$\sin((n+k)\omega) = \cos(k\omega)\sin(n\omega) + \sin(k\omega)\cos(n\omega),$$

$$\cos((n+k)\omega) = \cos(k\omega)\cos(n\omega) - \sin(k\omega)\sin(n\omega),$$

即位置 $n+k$ 处的正弦、余弦分量是位置 n 处同频率的正弦、余弦分量的线性组合, 系数 $\cos(k\omega)$、$\sin(k\omega)$ 只依赖 k 与 ω, 不依赖 n. 由于每个分量只与同频率的分量耦合, 把这些 2×2 的块按分量排列拼成分块对角矩阵, 即得只依赖 k 的 $\mathbf{M}_k$ 满足 $\mathbf{r}_{n+k} = \mathbf{M}_k \mathbf{r}_n$.

□

注

(重构新增) 该命题说明正弦位置编码把“相对位移”编码进了表示: 注意力层是线性的 (softmax 之前的部分), 若它能学会用 $\mathbf{M}_k$ 作用于查询或键, 就能识别“相距 k 步”这一关系, 而不必为每个绝对位置单独记忆参数, 因此对训练时未见过的更长序列也能泛化. 证明中同时使用正弦与余弦是关键: 只有两者成对出现时, 和角公式才能给出封闭的线性关系; 若只保留正弦,$\sin((n+k)\omega)$ 还依赖于 $\cos(n\omega)$, 无法写成单个分量的线性组合. 这也解释了为什么编码的相邻维度要成对取正弦与余弦、且共享同一角频率. 注意该性质是“表示层面”的: 它保证存在这样的线性变换, 但不保证网络一定学到它.

另一种常用的位置表示方法是使用习得的位置编码. 这是通过在每个 token 位置设置一个权重向量来实现的, 该权重向量可在训练过程中与模型的其他参数共同学习, 从而避免使用人工表示法. 由于 token 位置之间不共享参数, 因此 token 在排列时不再是不变的, 而这正是位置编码的目的. 不过, 这种方法并不符合前述对较长的输入序列进行泛化的标准, 因为训练过程中没有出现过的位置编码未经训练. 因此, 当输入序列的长度在训练和推理过程中都相对固定时, 这种方法通常最适用.

12.2 自然语言

在研究了 Transformer 架构之后, 我们将探讨如何用它来处理由词、句子和段落组成的语言数据. 尽管 Transformer 模型最初就是针对这种模态开发的, 但事实证明,Transformer 模型是一类非常通用的模型, 并已成为大多数输入数据类型的最先进模型. 本章稍后将探讨 Transformer 在其他领域的应用 (参见节 12.4).

包括英语在内的许多语言都由一系列词 (用空格分隔) 和标点符号组成, 语言数据是序列数据的典型代表. 我们目前将重点关注词, 稍后再讨论标点符号.

我们面临的第一个挑战是将词转换成适合用作深度神经网络输入的数字表示. 一种简单的方法是定义一个固定的字典, 然后引入长度与这个字典大小相等的向量以及每个词的“独热”表示, 其中字典中的第 k 个词用一个向量编码, 该向量在第 k 个位置为 1, 在其他位置为 0. 例如, 如果“aardwolf”是这个字典中的第三个词, 那么它的向量表示就是 $(0, 0, 1, 0, \cdots, 0)$.

独热编码有一个明显的问题, 即现实中的字典可能有几十万个条目, 从而导致向量的维度非常高. 此外, 独热编码无法捕捉词与词之间可能存在的任何相似性或关系. 这两个问题都可以通过将词映射到一个低维空间来解决, 这个过程叫作词嵌入 (word embedding). 在这个过程中, 每个词都可以表示为一个通常只有几百个维度的空间中的密集向量.

12.2.1 词嵌入

嵌入过程可以用一个大小为 $D \times K$ 的矩阵 $\mathbf{E}$ 来定义, 其中 D 是嵌入空间的维度,K 是字典的维度. 对于每个独热编码的输入向量 $\mathbf{x}_n$, 我们可以用下式计算出相应的嵌入向量:

$$\mathbf{v}_n = \mathbf{E}\mathbf{x}_n \tag{12.26}$$

由于 $\mathbf{x}_n$ 具有独热编码, 因此向量 $\mathbf{v}_n$ 可以由矩阵 $\mathbf{E}$ 的相应列给出.

定义 12.7 (词嵌入)

设字典含 K 个词, 嵌入空间的维度为 D,$\mathbf{E} \in \mathbb{R}^{D\times K}$. 对第 k 个词的独热编码 $\mathbf{x} \in \{0,1\}^K$(第 k 个分量为 1、其余为 0), 称 $\mathbf{v} = \mathbf{E}\mathbf{x} \in \mathbb{R}^D$ 为该词的**嵌入向量**, 映射 $\mathbf{x} \mapsto \mathbf{E}\mathbf{x}$ 称为**词嵌入** (word embedding); 当 $D \ll K$ 时称为低维嵌入.

♣

注

(重构新增) 词嵌入的动机直接来自独热编码的两个缺陷: 维度随字典线性膨胀 (K 可达数十万), 以及任意两个不同词的独热向量正交, 因而内积恒为零, 无法表达“相似”. 把词映射到 D 维 (通常几百) 的实向量, 既压缩了维度, 又使“相似”可以量化为内积或距离. 由于 $\mathbf{E}\mathbf{x}$ 取的是 $\mathbf{E}$ 的第 k 列, 词嵌入本质上就是一张“查表”: 每个词对应矩阵的一列, 训练只更新这些列. 这样做的代价是词与词之间不再有结构上的区别, 全部信息都必须由数据学出来; 好处是同一个词在任何位置都取同一个向量——这既是优点 (参数共享、样本效率高), 也是下一节 Transformer 要突破的限制 (同一个词在不同上下文中应有不同表示).

我们可以使用很多方法来从文本语料库 (即大型数据集) 中学习矩阵 $\mathbf{E}$. 在此, 我们将研究一种名为 word2vec 的流行技术 (Mikolov et al., 2013), 它可以视为一种简单的双层神经网络. 构建一个训练集, 其中每个样本都是通过考虑文本中 M 个相邻词的“窗口”获得的, 一个典型的值可能是 $M = 5$. 样本被认为是独立的, 误差函数则定义为每个样本的误差函数之和. 这种方法有两个模型: 连续词袋模型和“跳字模型”. 在连续词袋 (continuous bag of words) 模型中, 网络训练的目标变量是中间词, 其余的上下文 (context) 词构成输入, 因此网络被训练用于“填空”. 而“跳字 (skip-gram) 模型”将输入和输出互换了, 也就是将中心词作为输入, 而将目标值作为上下文词. 这两个模型如图 12.11所示.

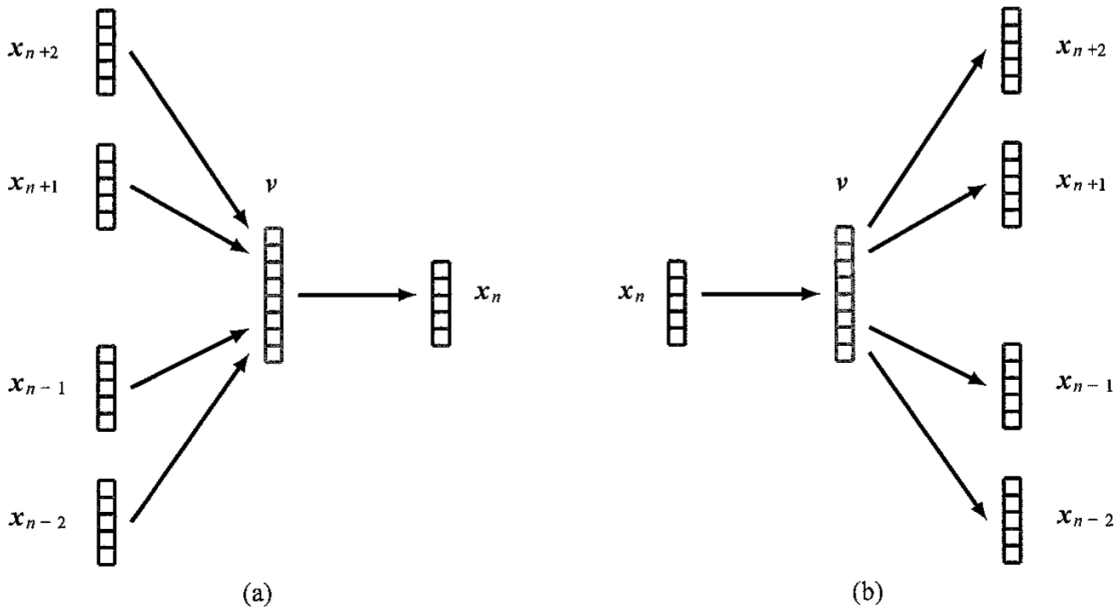


图 12.11: 用于学习词嵌入的双层神经网络:(a) 连续词袋模型;(b) “跳字模型”.

这种训练过程可以视为一种自监督 (self-supervised) 学习, 因为数据仅由大规模无标签文本语料组成, 可从中随机抽取许多词序列的小窗口. 通过“遮掩”那些网络试图预测其值的词, 便可从文本本身获取标签.

模型训练完成后, 嵌入矩阵 $\mathbf{E}$ 在连续词袋模型中由第二层权重矩阵的转置矩阵给出, 而在“跳字模型”中则由第一层权重矩阵给出. 在嵌入空间中, 语义相关的词会被映射到附近的位置. 这在意料之中, 因为与不相关的词相比, 相关的词更有可能与相似的上下文词一起出现. 例如, “城市”和“首都”两个词作为“巴黎”或“伦敦”等目标词的上下文词出现的频率可能较高, 而作为“橙色”或“多项式”的上下文词出现的频率较低. 如果将“巴黎”和“伦敦”映射到附近的嵌入向量上, 网络就能更容易地预测缺失词的概率.

事实证明, 习得的嵌入空间往往比相关词的邻近性具有更丰富的语义结构, 这使得进行简单的向量运算成为可能. 例如, “巴黎之于法国就像罗马之于意大利”这一概念可以通过对嵌入向量的运算来表达. 如果我们用 $\mathbf{v}(\text{word})$ 来表示“词”的嵌入向量, 则可以得到

$$\mathbf{v}(\text{Paris}) - \mathbf{v}(\text{France}) + \mathbf{v}(\text{Italy}) \approx \mathbf{v}(\text{Rome}) \tag{12.27}$$

词嵌入本身最初是作为自然语言处理工具开发的. 如今, 它们更多地用作深度神经网络的预处理步骤. 在这方面, 它们可以被视为深度神经网络的第一层. 它们可以通过一些标准的预训练嵌入矩阵来固定, 也可以作为一个自适应层来处理, 作为系统整体端到端训练的一部分来学习. 在后一种情况下, 可以使用随机权值或标准嵌入矩阵对嵌入层进行初始化.

12.2.2 分词

使用固定字典的一个问题是, 我们无法处理字典中没有的词或拼写错误的词. 此外, 这种方法不考虑标点符号或其他字符序列, 如计算机代码. 解决这些问题的另一种方法是在字符层面进行操作而不是使用词来操作, 这样我们的字典就包含了大小写字母、数字、标点符号以及其他符号 (如空格和制表符等). 但这种方法的缺点是, 它忽略了语言中语义上重要的词结构, 因此后续的神经网络必须学会从基本字符中重新组合词. 此外, 对于给定的文本, 还需要执行更多的连续步骤, 从而增加了处理序列的计算成本.

我们可以通过使用预处理步骤将一串词和标点符号转换成一串 token, 从而将字符级和词级表示的优势结合起来.token 通常是小的字符组, 其中可能包括完整的常用词、较长词的碎片以及可以组合成非常用词的单个字符 (Schuster and Nakajima, 2012). 通过进行这种分词, 系统便能处理其他类型的序列, 如计算机代码, 甚至处理其他模态, 如图像 (参见小节 12.4.1). 这也意味着同一个词的变体可以有相关的表示. 例如, “cook” “cooks” “cooked” “cooking” 和 “cooker” 都是相关的并共享共同元素 “cook”, 而 “cook” 本身也可以表示为 token.

定义 12.8 (分词)

设文本为字符序列. 若用一个预处理步骤把该序列切分为若干子串 (token) 的序列, 使得每个 token 取自一个预先确定的 token 字典, 则称这一过程为**分词** (tokenization), 所得子串序列称为该文本的 token 化表示. ♣

注

(重构新增) 分词要解决的是“词级表示”与“字符级表示”之间的两难: 词级字典无法覆盖未登录词与拼写变体, 且把标点、代码等排除在外; 字符级表示虽然完备, 却丢掉了词的形态结构, 并把序列拉得很长, 使注意力的 $O(N^2)$ 代价迅速上升 (见命题 12.4). 分词取两者的折中: 常见的完整词各占一个 token, 生僻词则拆成若干有意义的片段, 于是字典规模可控、序列长度可控、未登录词也能被表示. 定义中“取自预先确定的字典”是本质条件: 它保证了任何输入都能被编码为有限字典上的序列, 从而使嵌入矩阵与输出 softmax 的维度固定. 分词器与模型参数通常分开确定, 分词方式一旦改变, 嵌入层与输出层的维度都要随之改变, 因此它属于模型设计的一部分而不是纯粹的数据预处理.

有许多方法可以实现分词. 例如, 用于数据压缩的字节对编码 (byte pair encoding) 技术可以通过连接字符而非字节来适应文本分词 (Sennrich, Haddow, and Birch, 2015). 首先用单个字符列表对 token 列表进行初始化. 然后搜索文本中出现频率最高的相邻 token 对, 并用新 token 替换这些 token. 为确保词不被连接, 如果第二个 token 以空格开头, 则新 token 不是由两个 token 组成的. 反复进行上述过程, 如图 12.12所示.

最初,token 数量等于字符数量, 因而相对较少. 随着 token 的形成,token 的总数也会增加, 如果持续时间足够长,token 最终将与文本中的词集相对应. 作为字符级和词级表示之间的折中方案,token 的总数通常是预先确定的.

Peter Piper picked a peck of pickled peppers
Peter Piper picked a peck of pickledpeppers
Peter Piper picked a peck of pickled peppers
Peter Piper picked a peck of pickled peppers
Peter Piper picked a peck of pickled peppers

图 12.12: 通过类比字节对编码, 说明自然语言 token 化的过程. 此例中, 出现频率最高的一对字符是 “pe” , 共出现 4 次, 因此这对字符组成一个新的 token, 并取代了所有出现的 “pe” . 请注意, “Pe” 不包括在内, 因为大写 “P” 和小写 “p” 是两个不同的字符. 出现频率第二高的一对字符是 “ck” , 共出现 3 次. 接着是 “pi” “ed” 和 “per” , 它们都出现了两次, 以此类推.

当达到这个数量时, 算法就会停止.

在将深度学习运用到自然语言的实际操作中时, 输入文本通常首先映射为 token 化的表示. 不过, 在本章的其余部分, 我们将使用词级表示法, 因为这样更容易说明和引出关键概念.

12.2.3 词袋模型

现在, 我们要对自然语言中的词 (或 token) 等有序向量序列的联合分布 $p(x_1, \cdots, x_N)$ 进行建模. 最简单的方法是假定词是从同一分布中独立抽取的, 因此联合分布被全分解, 形式为

$$p(x_1, \cdots, x_N) = \prod_{n=1}^{N} p(x_n) \tag{12.28}$$

这可以表示为一个概率图模型, 其中节点是孤立的, 相互之间无连接.

变量之间共享分布 $p(x)$, 且在不失通用性的前提下, 共享分布 $p(x)$ 可以用一个简单的表格来表示, 该表格列出了 x 的每种可能状态 (对应于词或 token 字典) 出现的概率. 只需要将这些概率中的每个概率设置为该词在训练集中出现的次数, 就能得到该模型的最大似然解. 这就是所谓的词袋 (bag-of-words) 模型, 因为它完全忽略了词的排序.

定义 12.9 (词袋模型)

设 $x_1, \cdots, x_N$ 取自有限字典 $\mathcal{V}$, 且各变量独立同分布. 若联合分布取(12.28)的完全分解形式, 则称该模型为 **词袋模型** (bag-of-words model).

♣

注

(重构新增) 词袋模型的假设 “各词独立同分布” 在语言中显然不成立——词序、搭配、句法全部被丢弃——但它换来的是参数量的急剧下降: 联合分布从 $|\mathcal{V}|^N$ 个自由参数降为 $|\mathcal{V}| - 1$ 个. 这正是它在短文本分类 (如情感分析) 中长期可用的原因: 当 N 较大而 $|\mathcal{V}|$ 中等时, $|\mathcal{V}|^N$ 完全不可估计, 而词袋的参数可以被数据充分覆盖. 条件 “同分布” 还意味着同一个词在不同位置有相同的概率, 这在形式上与定义 12.7中 “同一个词总映射到同一向量” 的假设一致. 需要注意, 词袋模型仍然是一个概率模型,(12.28)只是分解假设, 不是对文本生成机制的真实刻画; 把它用于生成时会得到词序混乱的文本.

我们可以使用词袋法构建一个简单的文本分类器. 例如, 在情感分析中, 可以将一段代表餐厅评论的文本分为正面评论和负面评论. 朴素 Bayes 分类器假定每个类 C_k 中的词是独立的, 但每个类的词分布不同, 因此

$$p(x_1, \cdots, x_N \mid C_k) = \prod_{n=1}^{N} p(x_n \mid C_k) \tag{12.29}$$

在给定先验类概率 $p(C_k)$ 的前提下, 新序列的后验类概率为

$$p(C_k \mid x_1, \cdots, x_N) \propto p(C_k) \prod_{n=1}^{N} p(x_n \mid C_k) \tag{12.30}$$

类-条件密度 $p(x \mid C_k)$ 和先验概率 $p(C_k)$ 都可以使用训练集的频率进行估计. 对于一个新的序列, 将表格条目

相乘得到所需要的后验概率. 请注意, 如果测试集中出现一个训练集中没有的词, 那么相应的概率估计值将为零, 因此通常会在训练后对这些估计值进行“平滑”, 即在所有条目中均匀地重新分配一个较小的概率水平, 以避免出现零值.

命题 12.7 (词袋模型的最大似然解)

设训练集由 N 个独立抽取的词组成, 字典 $\mathcal{V}$ 有限, 记 n_k 为词 $k \in \mathcal{V}$ 出现的次数. 则词袋模型(12.28)的最大似然解为 $\hat{p}(k) = n_k/N$, 即训练集中的相对频率.

♠

证明

把(12.28)取对数, 得对数似然 $\sum_{k\in\mathcal{V}} n_k \ln p(k)$. 在约束 $\sum_k p(k) = 1$、$p(k) \geqslant 0$ 下用 Lagrange 乘子 $L = \sum_k n_k \ln p(k) + \lambda\left(1 - \sum_k p(k)\right)$, 对 $p(k)$ 求偏导并令其为零, 得 $n_k/p(k) = \lambda$, 即 $p(k) = n_k/\lambda$; 代入约束得 $\lambda = N$, 故 $\hat{p}(k) = n_k/N$. 又 ln 为凹函数, 该驻点即为全局最大值.

□

注

(重构新增) 这个命题说明“用频率估计概率”不是随手的选择, 而是词袋模型下的最大似然解——它与(12.30)的朴素 Bayes 分类器完全衔接: 类-条件密度就是在类 C_k 内部做同样的频率估计. 证明中的约束条件是本质的: 没有 $\sum_k p(k) = 1$, 目标函数无上界 (把某个 $p(k)$ 推向无穷即可). 命题同时暴露了该方法的一个结构性缺陷: 若某个词 k 在训练集中未出现, 则 $n_k = 0$, 最大似然解给出 $\hat{p}(k) = 0$, 于是任何含该词的新序列后验概率都为零, 分类器被“一票否决”. 这正需要文中所说的平滑来修补: 在计数上加一 (Laplace 平滑) 等价于把 n_k 换成 $n_k + 1$, 从而消除零概率.

12.2.4 自回归模型

词袋模型的一个主要局限在于完全忽略了词序. 为了解决这个问题, 我们可以采用自回归方法. 在不失通用性的前提下, 我们可以将词序列的分布分解为条件分布的乘积, 其形式为

$$p(x_1, \cdots, x_N) = \prod_{n=1}^{N} p(x_n \mid x_1, \cdots, x_{n-1}) \tag{12.31}$$

这可以用一个概率图模型 (参见节 11.1) 来表示, 其中序列里的每个节点都会从各自的前一个节点接收一个连接. 我们可以用一个表格来表示(12.31)右侧的每一项, 该表格中的条目也是通过训练集的简单频率计数估算出来的. 然而, 这些表格的大小会随着序列长度的增加呈指数增长 (见练习 12.12), 因此这种方法的成本会高得令人望而却步.

命题 12.8 (自回归分解)

对任意联合分布 $p(x_1, \cdots, x_N)$, 恒有(12.31)成立, 其中约定 $p(x_1 \mid x_1, \cdots, x_0) \triangleq p(x_1)$.

♠

证明

由概率的乘积法则, $p(x_1, \cdots, x_N) = p(x_1)p(x_2 \mid x_1)\cdots p(x_N \mid x_1, \cdots, x_{N-1})$. 把首个因子写成 $p(x_1 \mid \varnothing)$, 即得(12.31). 该分解对任意联合分布成立, 不依赖任何独立性假设, 故“不失通用性”.

□

注

(重构新增) 该命题说明自回归分解本身不含任何建模假设——它是恒等式, 因此“用自回归建模语言”并不损失表达能力, 真正的假设在于如何参数化每个条件分布: 用表格则参数随 N 指数增长, 用神经网络则参数固定. 这就是小节 12.2.5与 12.3 节转向参数化模型的理由. 与词袋模型(12.28)相比, 自回归分解保留了全部词序信息, 代价是每个条件分布的自变量个数随位置增长. 注意 $n=1$ 的项按约定为 $p(x_1)$, 这不是笔误: 把空条件视为“无信息”, 正是链式法则的起点.

我们可以假设(12.31)右侧的每个条件分布都与之前的所有观测值无关, 但最近的 L 个词除外, 从而大大简化模型. 例如, 如果 $L=2$, 那么在该模型下, 一组 N 个观测值的联合分布为

$$p(x_1,\cdots,x_N)=p(x_1)\,p(x_2\mid x_1)\prod_{n=3}^{N}p(x_n\mid x_{n-1},x_{n-2}) \tag{12.32}$$

在相应的图模型中, 每个节点都有前两个节点的连接 (参见节 11.1). 此处, 我们假设所有变量共享条件分布 $p(x_n\mid x_{n-L:n-1})$. 同样,(12.32)右侧的每个分布都可以用表格来表示, 其值是从抽取自训练语料的连续词的三元组统计中估算出来的.

定义 12.10 (n 元模型)

在自回归分解(12.31)中, 若对每个 n 都有 $p(x_n\mid x_1,\cdots,x_{n-1})=p(x_n\mid x_{n-L},\cdots,x_{n-1})$, 即条件分布只依赖最近的 L 个词, 则称所得模型为 L 阶 n **元模型** (n-gram model);$L=1$ 时称二元 (bi-gram) 模型,$L=2$ 时称三元 (tri-gram) 模型. ♣

注

(重构新增) n 元模型用“截断历史”换取可估计性: 条件分布的自变量从 $n-1$ 个减为固定的 L 个, 于是所有位置的 $p(x_n\mid x_{n-L:n-1})$ 共享同一张大小为 $|\mathcal{V}|^{L+1}$ 的表, 可以用频率计数估计. 这是一次典型的 Markov 假设, 其代价是超过 L 步的依赖被完全丢弃, 而自然语言中长距离依赖恰恰普遍存在 (例如代词与先行词).L 的选择因此是偏差-方差权衡:L 太小则模型无法表达语法,L 太大则表的大小指数增长、绝大多数条目在训练集中从未出现. 这也解释了为什么后续要用参数化的神经网络代替查表——把“所有可能的 L 元组”压缩为固定维度的参数.

$L=1$ 的情况称为二元 (bi-gram) 模型, 因为它取决于相邻的词对. 同样,$L=2$ 的情况涉及相邻词的三元组, 叫作三元 (tri-gram) 模型. 一般来说, 这些模型都叫作 n 元 (n-gram) 模型.

本节到目前为止讨论的所有模型都可以生成式运行, 以合成新文本. 例如, 如果我们提供一个序列中的第 1 和第 2 个词, 那么我们就可以从三元统计 $p(x_n\mid x_{n-1},x_{n-2})$ 中抽样生成第 3 个词, 然后我们可以使用第 2 和第 3 个词来抽样生成第 4 个词, 以此类推. 然而, 由于每个词都是根据前两个词预测出来的, 因此产生的文本将是不连贯的. 高质量的文本模型必须考虑语言中的远距离依赖关系. 另一方面, 我们不能简单地增加 L 的值, 因为概率表的大小会呈指数级增长, 所以成本远远超过三元模型. 不过, 当我们考虑现代语言模型时, 自回归表示法将发挥核心作用, 这些模型并不基于概率表, 而基于配置为 Transformer 的深度神经网络.

在避免 n 元模型参数数量呈指数增长的同时, 允许存在远距离依赖关系的一种方法是使用隐 Markov 模型 (hidden Markov model), 其图结构如节 11.3所示 (参见该节关于序列模型的讨论). 可学习参数的数量由潜变量的维度决定, 而给定观测值 x_n 的分布原则上取决于之前的所有观测值. 但是, 较远的观测值具有非常有限的影响, 因为它们的影响必须通过潜态链来实现, 而潜态链本身也在被较新的观测值更新.

12.2.5 递归神经网络

n 元模型等技术在序列长度上的缩放性很差, 因为它们存储的是完全通用的条件分布表. 通过使用基于神经网络的参数化模型, 我们可以实现更好的缩放. 设想我们仅仅用标准的前馈神经网络来处理自然语言中的词序列. 此时便会出现一个问题, 即网络有固定数量的输入和输出, 而我们需要网络能够处理训练集和测试集中长度可变的序列. 此外, 如果序列中某个位置的一个词或一组词代表某个概念, 那么处在不同位置的同一个词或同一组词很可能在新的位置代表相同的概念. 这会让我们想起之前处理图像数据时遇到的等变性 (参见第 10 章). 如果我们能

够构建一个能在序列中共享参数的网络架构, 那么我们不仅可以捕捉到等变性, 还可以大大减少模型中自由参数的数量, 并处理不同长度的序列.

为了解决这个问题, 我们可以借鉴隐 Markov 模型, 引入一个与序列中每一步 n 相关的显式隐变量 $\mathbf{z}_n$. 神经网络将当前词 $\mathbf{x}_n$ 和当前隐状态 $\mathbf{z}_{n-1}$ 作为输入, 并产生输出词 $\mathbf{y}_n$ 以及隐变量的下一个状态 $\mathbf{z}_n$. 然后, 我们可以将该网络的各个副本连接起来, 其中权重值在各个副本之间共享. 由此产生的架构称为递归神经网络 (Recurrent Neural Network, RNN), 如图 12.13所示. 此处隐状态的初始值可以初始化为某个默认值, 例如 $\mathbf{z}_0 = (0, 0, \cdots, 0)^{\mathsf{T}}$.

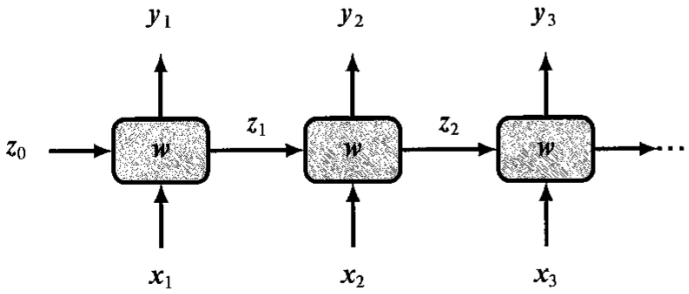


图 12.13: 带有参数 $\mathbf{w}$ 的一般递归神经网络. 它将 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 作为输入, 并生成一个序列 $\mathbf{y}_1, \cdots, \mathbf{y}_N$ 作为输出. 图中的每个方框对应一个具有非线性隐藏单元的多层网络.

定义 12.11 (递归神经网络)

设 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 为输入序列. 若网络按

$$\mathbf{z}_n = \mathbf{f}(\mathbf{x}_n, \mathbf{z}_{n-1}; \mathbf{w}), \qquad \mathbf{y}_n = \mathbf{g}(\mathbf{z}_n; \mathbf{w}) \tag{12.33}$$

逐时间步更新隐状态, 其中 $\mathbf{f}, \mathbf{g}$ 为以 $\mathbf{w}$ 为参数、在所有时间步上共享的非线性映射,$\mathbf{z}_0$ 为固定的初始状态, 则称该架构为**递归神经网络** (Recurrent Neural Network, RNN).

♣

注

(重构新增) 递归神经网络用“状态”代替“历史”: 不必为每个可能的词序列存一张表, 只需把迄今为止的全部信息压缩进 $\mathbf{z}_n$, 再用同一组参数 $\mathbf{w}$ 递推. 因此参数量与序列长度无关, 且能处理任意长度的序列——这两点正是 n 元模型做不到的. 参数在时间步之间共享还带来“平移等变”: 同一个词出现在不同位置时由同一套权重处理, 与卷积网络在空间上共享滤波器的思路一致. 代价是信息必须逐时间步传递: 从 t 到 $t+k$ 要经过 k 次复合, 梯度也随之连乘 k 次, 这正是小节 12.2.6中梯度消失/爆炸与瓶颈问题的来源. 此外, 递推的串行性使训练无法在时间维上并行, 这是 Transformer 取代它的直接原因.

关于如何在实践中使用递归神经网络, 请参考将句子从英语翻译成荷兰语的具体任务. 句子的长度是可变的, 且每个输出句子的长度可能与相应的输入句子的长度不同. 此外, 网络在开始生成输出句子之前, 可能需要看到整个输入句子. 我们可以使用递归神经网络解决这个问题, 方法是输入完整的英文句子, 然后输入一个特殊的 token(用 ⟨start⟩ 表示), 以触发翻译过程的开始. 在训练过程中, 网络将学会如何将 ⟨start⟩ 与输出句子的开头联系起来. 我们还可以将每个连续生成的词输入下一个时间步的输入中, 如图 12.14所示. 训练网络以生成特定的 ⟨stop⟩ token, 以表示翻译完成. 网络的前几个阶段用于吸收输入序列, 而相关的输出向量则被忽略. 网络的这一部分可以看作一个“编码器”, 其中整个输入句子都压缩成了隐变量的状态 $\mathbf{z}^*$. 其余的网络阶段则充当“解码器”, 旨在逐字生成翻译好的句子作为输出. 请注意, 每个输出词都会作为输入词输入网络的下一阶段, 因此这种方法具有类似于(12.31)的自回归结构.

解决梯度消失和梯度爆炸问题以及有限远距离依赖关系的一种方法是修正神经网络架构, 从而允许额外的信号路径, 该路径可以绕过网络每个阶段的许多处理步骤, 进而允许在更多的时间步内记忆信息. 长短时记忆 (Long Short-Term Memory, LSTM) 模型 (Hochreiter and Schmidhuber, 1997) 和门控循环单元 (Gated Recurrent Unit, GRU) 模型 (Cho et al., 2014) 是最广为人知的示例. 虽然与标准递归神经网络相比, 它们的性能有所提高, 但在远距离依赖关系建模方面, 它们的能力仍然有限, 而且每个单元的额外复杂性意味着长短时记忆模型的训练速度相

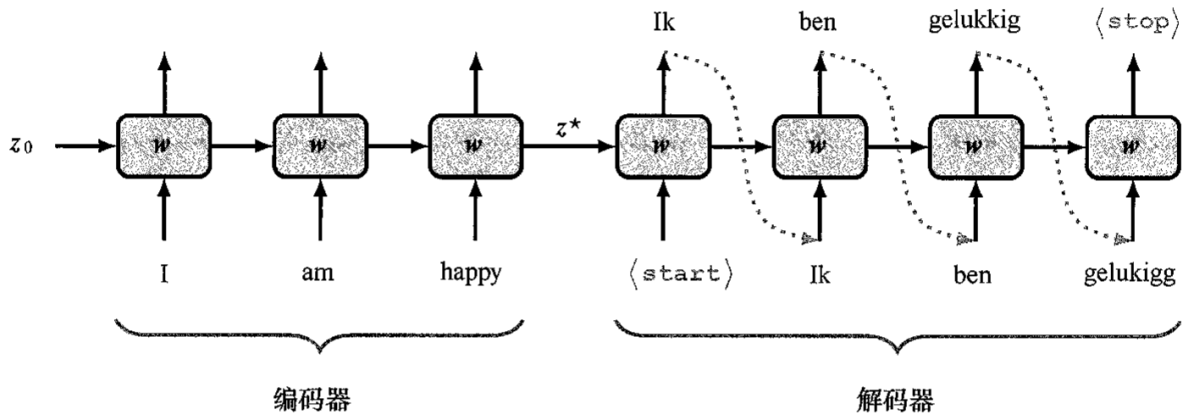


图 12.14: 用于语言翻译的递归神经网络示例.

比标准递归神经网络更慢. 此外, 所有递归神经网络的信号路径都与序列中的步数成线性关系. 最后, 由于处理过程具有连续性, 它们不支持在单个训练示例内进行并行计算. 特别是, 这意味着递归神经网络难以有效利用基于图形处理单元的现代高度并行硬件. 用 Transformer 取代递归神经网络可以解决这些问题.

12.2.6 通过时间反向传播

与普通神经网络一样, 递归神经网络可根据反向传播计算出的梯度, 通过随机梯度下降法进行训练, 并通过自动微分法进行计算. 误差函数包括所有输出单元对每个单元误差的求和, 其中每个输出单元都有一个 softmax 激活函数和一个相关的交叉熵误差函数. 在前向传播过程中, 激活值从序列中的第一个输入节点一直传播到序列中的所有输出节点, 然后误差信号沿着相同的路径反向传播. 这一过程称为通过时间反向传播 (backpropagation through time), 其原理简单明了. 然而在实践中, 由于深度网络架构会出现梯度消失或梯度爆炸的问题, 因此训练很长的序列可能会很困难.

标准递归神经网络的另一个问题是, 其无法很好地处理远距离依赖关系. 这在自然语言中尤为突出, 因为这种依赖关系非常普遍. 在较长的文本段落中, 不妨引入一个概念, 这个概念对预测文本后面出现的词起着重要作用. 在图 12.14所示的架构中, 英文句子的整个概念必须在长度固定的单个隐向量 $\mathbf{z}^*$ 内被捕捉到, 而如果序列较长, 这个问题就会越来越严重. 这就是所谓的瓶颈问题 (bottleneck problem), 因为任意长度的序列都必须总结为单个激活的隐向量, 而网络只有在处理完整个输入序列后才能开始生成输出.

命题 12.9 (通过时间反向传播的梯度连乘)

考虑(12.33)的递归网络, 设 $\mathbf{z}_n = \mathbf{f}(\mathbf{x}_n, \mathbf{z}_{n-1}; \mathbf{w})$. 则损失对早期状态的梯度满足

$$\frac{\partial L}{\partial \mathbf{z}_k} = \sum_{n>k} \frac{\partial L}{\partial \mathbf{z}_n} \prod_{j=k+1}^{n} \frac{\partial \mathbf{z}_j}{\partial \mathbf{z}_{j-1}}, \tag{12.34}$$

其中每个因子是 $\mathbf{f}$ 对 $\mathbf{z}_{j-1}$ 的 Jacobi 矩阵.

证明

对 $L(\mathbf{z}_1, \cdots, \mathbf{z}_N)$ 用链式法则, 把 $\mathbf{z}_k$ 到 $\mathbf{z}_n$ 的依赖拆成逐时间步的复合: 由 $\mathbf{z}_j = \mathbf{f}(\mathbf{x}_j, \mathbf{z}_{j-1}; \mathbf{w})$ 得 $\partial \mathbf{z}_j / \partial \mathbf{z}_{j-1}$ 为 Jacobi 矩阵. 对每个 $n > k$ 沿唯一路径复合, 再对所有 n 求和 (因为 $\mathbf{z}_k$ 通过每个后续状态影响损失), 即得(12.34).

□

注

(重构新增) 该命题把“梯度消失/爆炸”从一个经验说法变成可检验的陈述: 梯度是 $(n-k)$ 个 Jacobi 矩阵的连乘. 若这些矩阵的谱半径一致小于 1, 乘积随 $n-k$ 指数衰减; 若一致大于 1, 则指数增长. 因此长序列上早期时间步几乎收不到有效梯度, 这正是“难以学习远距离依赖”的定量含义.LSTM 与 GRU 的门控结构相当于把连乘中的一部分替换为接近恒等的路径 (遗忘门接近 1 时), 从而缓解衰减; 残差连接在 Transformer 中起的是同一作用——把连乘变成“恒等 + 小扰动”的和式展开. 注意公式里的求和意味着 $\mathbf{z}_k$ 有多条路径到达损失, 这与普通深度网络中同一变量被多处使用的处理完全一致.

12.3 Transformer 语言模型

Transformer 处理层是一个高度灵活的组件, 可用于构建具有广泛适用性的强大神经网络模型. 本节将探讨 Transformer 在自然语言中的应用. 这促进了称作大语言模型 (Large Language Model, LLM) 的大规模神经网络的发展, 事实证明, 这类模型具有非凡的能力 (Zhao et al., 2023).

Transformer 语言模型可用于多种不同的语言处理任务, 并可根据输入和输出数据的形式分为 3 类. 在情感分析等问题中, 我们将一个词序列作为输入, 并提供一个代表文本情感 (如快乐或悲伤) 的变量作为输出. 此处,Transformer 充当序列的“编码器”. 而在其他问题中, 我们可能会将单个向量作为输入, 并生成一个词序列作为输出. 例如, 假设我们希望在给定输入图像的情况下生成文本标题. 在这种情况下,Transformer 作为“解码器”产生一个序列作为输出. 最后, 在序列到序列的处理任务中, 输入和输出都由一个词序列组成. 例如, 假设我们想要将一种语言翻译成另一种语言. 在这种情况下,Transformer 同时扮演编码器和解码器的角色. 我们将使用模型架构的说明性示例依次讨论上述每一类 Transformer 语言模型.

12.3.1 解码器型 Transformer

我们首先考虑解码器型 Transformer. 此类 Transformer 语言模型可以用作生成式模型以创建 token 的输出序列. 作为说明性示例, 我们将重点讨论一种名为 GPT(Generative Pretrained Transformer, 生成式预训练 Transformer) 的模型 (Radford et al., 2019; Brown et al., 2020; OpenAI, 2023). 我们想要利用 Transformer 架构构建一个形式如(12.31)的自回归模型, 其中条件分布 $p(x_n \mid x_1, \cdots, x_{n-1})$ 是利用从数据中习得的 Transformer 神经网络来表示的.

该模型将由前 $n-1$ 个 token 组成的序列作为输入, 相应的输出表示 token n 的条件分布. 从这个分布中抽取一个样本, 即可将序列扩展到 n 个 token, 这个新序列可以通过模型反馈给 token $n+1$ 的分布, 以此类推. 可以重复该过程, 生成一个序列, 该序列的最大长度由 Transformer 输入的数量决定. 我们稍后将讨论从条件分布中采样的策略 (参见小节 12.3.2), 但目前我们讨论的重点是如何构建和训练网络.

GPT 模型的架构由 Transformer 层堆叠组成, 这些层采用 token 的序列 $\mathbf{x}_1, \cdots, \mathbf{x}_N$ 作为输入, 维度为 D, 并产生 token 的序列 $\tilde{\mathbf{x}}_1, \cdots, \tilde{\mathbf{x}}_N$ 作为输出, 维度同样为 D. 每个输出都需要代表该时间步的 token 字典的概率分布,token 字典的维度为 K, 而 token 的维度为 D. 因此, 我们使用维度为 $D \times K$ 的矩阵 $\mathbf{W}^{(o)}$ 对每个输出 token 进行线性变换, 然后使用 softmax 激活函数, 其形式为

$$\mathbf{Y} = \text{Softmax}\left(\tilde{\mathbf{X}}\mathbf{W}^{(o)}\right) \tag{12.35}$$

其中 $\mathbf{Y}$ 是第 n 行为 $\mathbf{y}_n^\mathsf{T}$ 的矩阵,$\tilde{\mathbf{X}}$ 是第 n 行为 $\tilde{\mathbf{x}}_n^\mathsf{T}$ 的矩阵. 每个 softmax 输出单元都有一个相关的交叉熵误差函数 (参见小节 5.4.4). 模型架构如图 12.15所示.

该模型可以采用自监督的方法并使用大规模的无标签自然语言语料进行训练. 每个训练样本由 token 的序列 $\mathbf{x}_1, \cdots, \mathbf{x}_n$ 组成, 构成网络的输入. 每个训练样本还包含序列中下一个 token 组成的相关目标值 $\mathbf{x}_{n+1}$. 序列可以认为是独立同分布的, 因此用于训练的误差函数是训练集上的交叉熵误差值之和, 并分成适当的小批次. 我们可以使用通过模型的前向传递来独立且朴素地处理每个训练样本. 不过, 我们可以通过一次性处理整个序列来实现更高的效率, 从而每个 token 既可以作为前一个 token 序列的目标值, 也可以作为后续 token 的输入值. 例如, 考虑如下词序列:

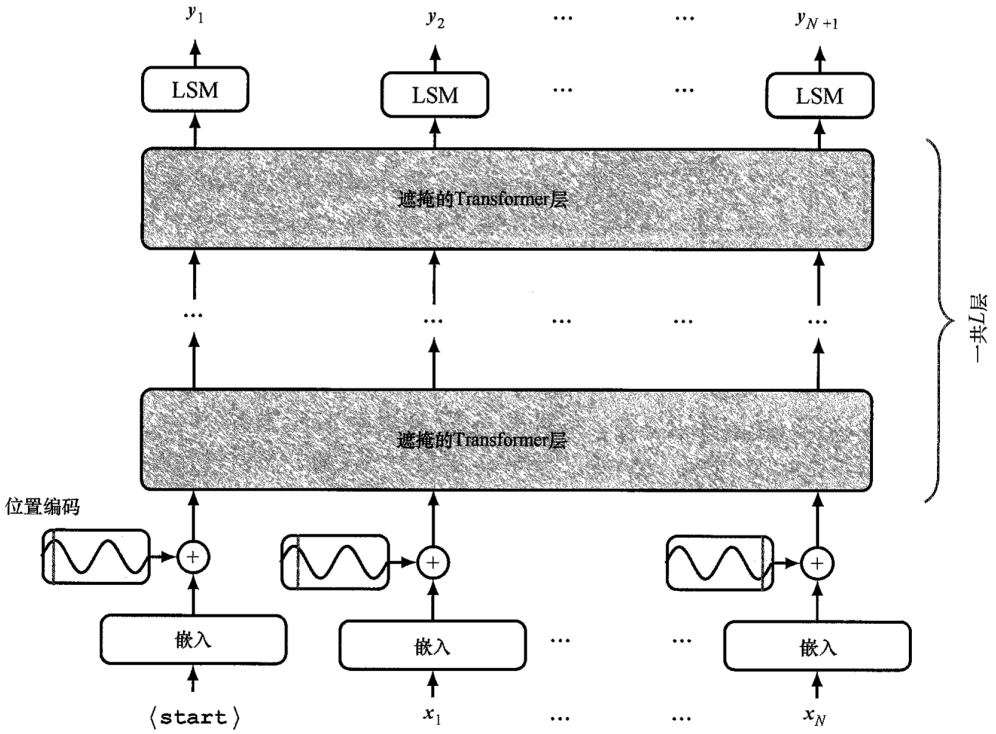


图 12.15: GPT 解码器型 Transformer 网络的架构. 此处 “LSM” 代表 linear-softmax, 记作一种线性变换, 它的可学习参数在 token 位置之间是共享的, 然后是一个 softmax 激活函数.

I swam across the river to get to the other bank.

我们既可以使用 “I swam across” 作为输入序列, 关联目标为 “the” ; 也可以使用 “I swam across the” 作为输入序列, 关联目标为 “river” , 以此类推. 不过, 为了并行处理这些数据, 我们必须确保网络无法通过提前查看序列来 “作弊” , 否则它只会学到将下一个输入直接复制到输出. 这样的话, 它就无法生成新的序列, 因为根据定义, 测试时后续 token 不可用. 为了解决这个问题, 我们做了两件事. 首先, 我们将输入序列向右移动一步, 使输入 $\mathbf{x}_n$ 对应于输出 $\mathbf{y}_{n+1}$, 目标为 $\mathbf{x}_{n+1}$, 并在输入序列的第一个位置预置一个额外的 token, 标记为 ⟨start⟩. 其次, 请注意,Transformer 中的 token 是独立处理的, 除非它们用于计算注意力权重, 此时它们会通过点积成对地相互作用. 因此, 我们在每个注意层中引入了遮掩注意力 (masked attention), 有时也称因果注意力 (causal attention). 在此过程中, 我们将每个 token 对于序列中之后的所有 token 的注意力权重设为零. 具体来说, 只需要将(12.15)定义的注意力矩阵 Attention($\mathbf{Q}, \mathbf{K}, \mathbf{V}$) 的所有相应元素置零, 然后对其余元素进行归一化, 使每一行的总和再次为 1 即可. 为达成这一目标, 在实际操作中, 可将相应的预激活值设置为 $-\infty$. 这样一来,softmax 函数对相关输出的计算结果为零, 并能对非零输出进行归一化处理. 遮掩注意力矩阵的结构如图 12.16所示.

定义 12.12 (遮掩自注意力)

设 $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ 由(12.14)给出. 若对每个 n 只允许位置 $m \leqslant n$ 的键参与注意力, 即把 $m > n$ 的 softmax 预激活值置为 $-\infty$, 其余按(12.15)归一化, 则称所得注意力为**遮掩自注意力** (masked self-attention). ♣

注

(重构新增) 遮掩的作用是 “在训练时模拟推理时的信息约束” : 推理时位置 n 只能看到前 n 个 token, 因此训练时也不许它看后面的内容, 否则模型会退化为 “复制下一个输入” 的恒等映射. 用 $-\infty$ 而不是 0 来屏蔽是有讲究的: 若把未归一化的权重置零,softmax 的分母仍包含这些项, 归一化后每行之和不再是原先的 1, 且被屏蔽项会分走概率质量; 置 $-\infty$ 后 $\exp(-\infty) = 0$, 这些项在分子与分母中同时精确消失, 归一化自动只在可见集合上进行. 由于屏蔽模式只依赖位置而不依赖数据, 它可以在每个注意力头、每一层复用同一张下三角掩码. 定义中 “$m \leqslant n$” 包含自身, 这是必要的: 位置 n 的表示必须能看见自己.

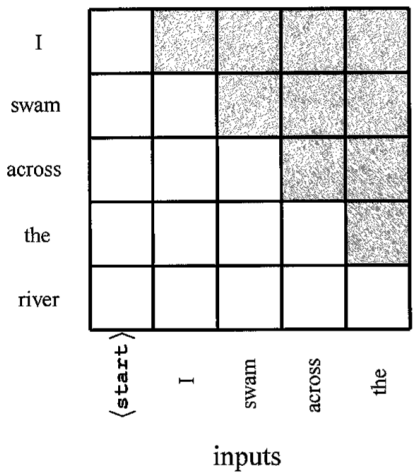


图 12.16: 遮掩注意力矩阵的结构. 红色元素对应的注意力权重为零. 因此, 在预测 token “across” 时, 输出只能取决于输入 tokens “⟨start⟩” “I” 和 “swam” .

在实践中, 我们希望有效利用 GPU 的大规模并行性, 因此可以将多个序列堆叠成一个输入张量, 以便在一个批次中进行并行处理. 不过, 这要求序列的长度相同, 而文本序列的长度是可变的. 要解决这个问题, 我们可以引入一个特定的 token(用 ⟨pad⟩ 表示), 用来填充未使用的位置, 从而使所有序列达到相同的长度, 以便将它们合并成一个单一张量. 然后在注意力权重中使用一个额外的掩码, 以确保输出向量不对 ⟨pad⟩ token 占用的任何输入给予注意力. 请注意, 该掩码的形式取决于特定的输入序列.

训练模型的输出是 token 空间的概率分布, 其由输出激活函数 softmax 给出, 表示当前 token 序列情况下的下一个 token 的概率. 一旦选择了下一个词, 就可以将包含新 token 的 token 序列再次输入模型, 以生成序列中的后续 token, 这个过程可以无限重复, 直至生成表示序列结束的 token. 这看起来似乎效率很低, 因为每个新生成的 token 都必须通过整个模型输入数据. 不过, 请注意, 由于遮掩注意力, 为特定 token 所学习到的嵌入向量只取决于该 token 本身和先前的 token, 因此在生成新的、后续的 token 时, 该嵌入向量不会发生变化. 在处理新的 token 时, 大部分计算可以循环使用.

12.3.2 抽样策略

我们已经看到, 解码器型 Transformer 的输出是序列中下一个 token 值的概率分布, 必须从中选择一个特定的 token 值来扩展序列. 根据计算出的概率, 选择 token 值的方法有好几种 (Holtzman et al., 2019). 一种显而易见的方法称为贪婪搜索, 即简单地选择概率最高的 token. 这使得模型具有确定性, 即给定的输入序列总是产生相同的输出序列. 请注意, 在每个阶段选择概率最高的 token 并不等于选择概率最高的 token 序列 (见练习 12.15). 要找到最有可能的序列, 我们需要最大化所有 token 的联合分布, 其计算公式为

$$p(\mathbf{y}_1, \cdots, \mathbf{y}_N) = \prod_{n=1}^{N} p(\mathbf{y}_n \mid \mathbf{y}_1, \cdots, \mathbf{y}_{n-1}) \tag{12.36}$$

如果序列中有 N 个步骤, 而字典中 token 值的数量为 K, 那么序列的总数为 $O(K^N)$, 这使得序列的长度呈指数增长, 因此寻找单一的最可能序列是不可行的. 相比之下, 贪婪搜索的成本为 $O(KN)$, 与序列长度成线性关系.

与贪婪搜索相比, 有一种技术有可能产生更高概率的序列, 即束搜索 (beam search). 我们不是在每一步都选择一个最可能的 token 值, 而是保留一组 B 假设 (其中的 B 称为束宽度 (beam width)), 每个假设由直到 n 步的 token 值序列组成. 然后, 我们通过网络将所有这些序列输入, 并为每个序列找到 B 个最可能的 token 值, 从而为扩展序列创建 B^2 个可能的假设. 最后, 根据扩展序列的总概率选择最有可能的 B 假设, 从而对该序列进行剪枝. 因此, 束搜索会保留 B 个备选序列, 跟踪它们的概率, 并从考虑过的序列中选出最有可能的序列. 由于序列的概率是由序列中每一步的概率相乘得到的, 而这些概率总是小于或等于 1, 因此长序列的概率通常低于短序列的概率, 从而使结果偏向于短序列. 在进行比较之前, 通常需要根据序列的相应长度对序列的概率进行归一化处理. 束搜索的成本

为 $O(BKN)$, 也与序列长度成线性关系. 然而, 生成一个序列的成本会是原来的 B 倍, 因此对于推理成本可能会变得很高的超大语言模型来说, 束搜索的吸引力将大打折扣.

贪婪搜索和束搜索等方法存在的一个问题是, 它们限制了潜在输出的多样性, 甚至可能导致生成过程陷入循环, 即重复出现相同的子序列. 从图 12.17中可以看出, 与自动生成的文本相比, 人工生成的文本可能概率较低, 因此对于给定的模型来说更不寻常.

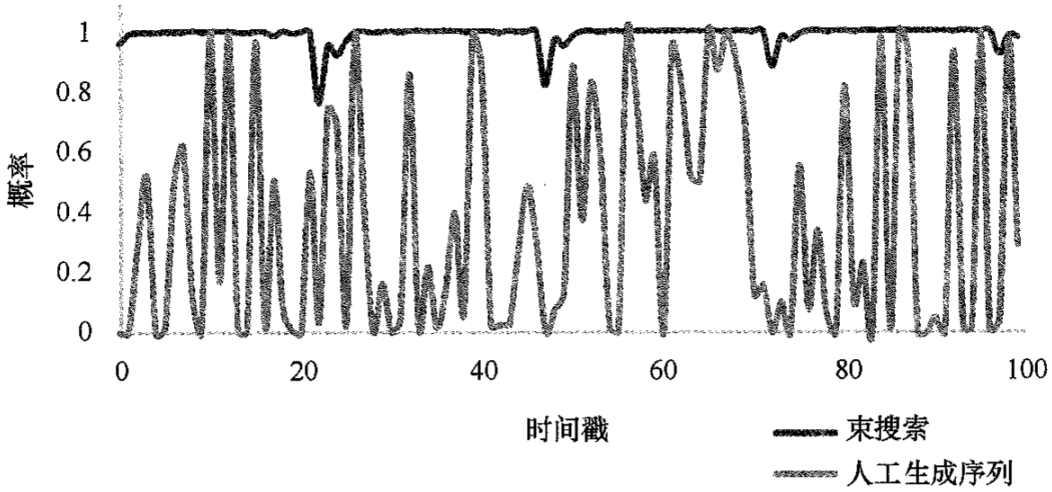


图 12.17: 在给定经训练 Transformer 语言模型和初始输入序列的情况下, 比较束搜索和人工生成序列的 token 概率, 得出人工生成序列的 token 概率要低得多 [经 Holtzman 等人 (2019) 授权使用].

我们可以在每一步从 softmax 分布中采样, 生成连续的 token, 而不是试图找到概率最高的序列. 然而, 这可能导致无意义的序列. 这是因为 token 字典通常非常庞大, 其中有许多 token 状态组成的长尾, 每个状态的概率都非常小, 但总体上却占总概率的很大一部分. 这就导致一个问题, 即系统很有可能对下一个 token 做出错误的选择.

为了在这两个极端之间取得平衡, 对于 K 的某个选择, 我们可以只考虑前 K 个概率的状态, 然后根据它们的重归一化概率对其进行采样, 称为 top-K 采样. 这种方法的一个变体叫作 top-p 采样或核采样 (nuclear sampling), 用于计算最高输出的累积概率, 直至达到一个阈值, 然后从这个受限的 token 状态集中采样.

top-K 采样的一个“更软”版本是在 softmax 函数的定义中引入一个称为温度的参数 T(Hinton, Vinyals, and Dean, 2015), 从而

$$y_i = \frac{\exp(a_i/T)}{\sum_j \exp(a_j/T)} \tag{12.37}$$

定义 12.13 (温度采样)

设 logits 为 $a_1, \cdots, a_K$. 若按(12.37)把 softmax 的输出 y_i 作为第 i 个 token 的采样概率, 则称参数 $T > 0$ 为**温度**, 相应过程称为**温度采样**. ♣

注

(重构新增) 温度把“确定性”与“随机性”放进同一个连续参数里:T 越小,logits 的差异被放大, 分布越尖锐;T 越大, 差异被压平, 分布越均匀.$T = 1$ 时恢复原始 softmax, 因此它不改变模型的训练, 只改变推理时的采样行为. 它的实际作用是对长尾的抑制程度可调:$T < 1$ 时低概率 token 的采样机会急剧下降,$T > 1$ 时则会引入更多罕见词. 注意定义要求 $T > 0$:$T = 0$ 时公式无定义, 应理解为极限, 而不是直接代入. 温度采样与 top-K、top-p 并不冲突, 实际系统常先截断再调温.

命题 12.10 (温度的极限行为)

设 $a_1, \cdots, a_K$ 中最大值唯一, 记为 a_{i^*},$y_i(T)$ 由(12.37)给出. 则当 $T \to 0^+$ 时 $y_{i^*}(T) \to 1$ 而对 $i \neq i^*$ 有 $y_i(T) \to 0$, 即贪婪选择; 当 $T \to \infty$ 时 $y_i(T) \to 1/K$, 即均匀分布; 当 $T = 1$ 时 y_i 为原始 softmax 输出. ♠

证明

写 $y_i = \exp((a_i - a_{i^*})/T) \Big/ \sum_j \exp\left((a_j - a_{i^*})/T\right)$. 对 $i \neq i^*$ 有 $a_i - a_{i^*} < 0$, 当 $T \to 0^+$ 时该指数趋于 0; 分子与分母中 i^* 项均趋于 1, 故 $y_{i^*} \to 1$, 其余趋于 0. 当 $T \to \infty$ 时所有 $(a_i - a_{i^*})/T \to 0$, 故 $y_i \to 1/K.T = 1$ 时(12.37)就是 softmax 的定义.

□

注

(重构新增) 该命题说明贪婪搜索不是独立于采样之外的算法, 而是温度采样在 $T \to 0$ 时的极限, 这解释了为什么 T 很小时生成文本接近确定、重复率高, 而 T 很大时言辞紊乱: 分布被拉平后, 长尾 token 与合理 token 的概率差距消失.$T \to \infty$ 的均匀极限给出了最坏情形的参照: 此时模型输出与输入完全无关, 退化为随机打字. 这两个极限也提示调温的合理区间是 $0 < T \leqslant 1$ 附近.

然后从修正过的分布中采样下一个 token. 当 $T = 0$ 时, 概率质量 (probability mass) 集中在最可能的状态上, 其他状态的概率为零, 因此这就变成了贪婪挑选. 当 $T = 1$ 时, 我们恢复未经修正的 softmax 分布; 当 $T \to \infty$ 时, 分布在所有状态下变得均匀. 在 $0 < T < 1$ 的范围内选择一个值, 概率就会向高值集中.

序列生成面临的一个挑战是, 在学习阶段, 模型是在人工生成的输入序列上进行训练的; 而在生成运行阶段, 输入序列本身就是由模型生成的. 这意味着模型可能会偏离我们在训练过程中看到的序列分布.

12.3.3 编码器型 Transformer

接下来, 我们将考虑基于编码器的 Transformer 语言模型, 这种模型将序列作为输入, 并生成长度固定的向量 (如 ⟨class⟩ 标签) 作为输出. 作为这种模型的典型代表,BERT(Bidirectional Encoder Representations from Transformers)(Devlin et al., 2018) 旨在利用大规模文本语料预先训练语言模型, 然后利用迁移学习 (transfer learning) 对模型进行微调, 以完成各种下游任务, 其中每种任务都需要较小的特定应用训练集. 模型架构如图 12.18所示. 这种方法是前面所讲的 Transformer 层的简单应用 (参见小节 12.1.7).

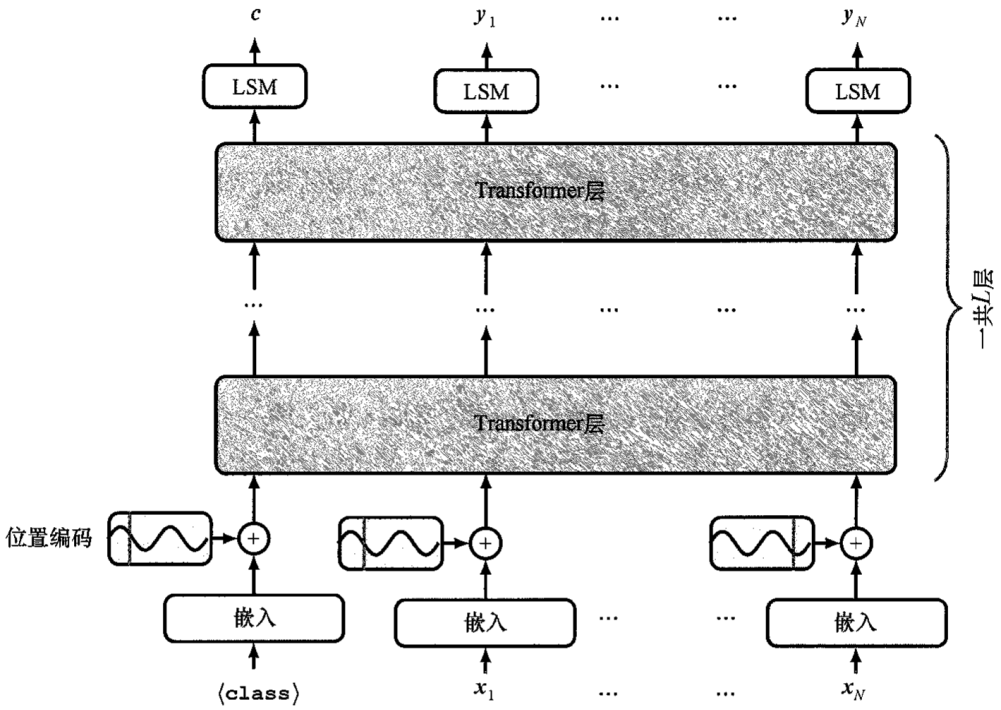


图 12.18: GPT 编码器型 Transformer 网络的架构. 标有 “LSM” 的方框表示线性变换, 其可学习参数在 token 位置之间是共享的, 然后是一个 softmax 激活函数. 与图 12.15相比, 主要区别在于输入序列没有向右移动, 且省略了遮掩矩阵. 因此, 在每个自注意力层中, 每个输出 token 都可以关注任何输入 token.

每个输入字符串的第一个 token 由一个特殊的 token ⟨class⟩ 给出, 模型的相应输出在预训练时则被忽略. 当我们讨论微调时, 它的作用将变得显而易见. 通过在输入端呈现 token 序列, 可以对模型进行预训练. 用一个特殊 token(用 ⟨mask⟩ 表示) 代替一个随机选择的 token 子集, 比例为 15%. 该模型经过训练后, 就可以预测相应输出节点上缺失的 token. 这类似于 word2vec 模型中用于学习词嵌入的遮掩 (参见小节 12.2.1). 例如, 输入序列可能如下:

I ⟨mask⟩ across the river to get to the ⟨mask⟩ bank.

网络应该预测输出节点 2 上的 “swam” 和输出节点 10 上的 “other” . 在这种情况下, 只有两个输出有益于误差函数, 其他输出则被忽略.

“双向” 一词是指网络可以同时看到掩词之前和之后的词, 并利用这两个信息源进行预测. 因此, 与解码器型 Transformer 不同的是, 编码器型 Transformer 不需要将输入向右移动一位, 也不需要遮掩每一层的输出, 以免看到序列中后续出现的输入 token. 与解码器模型相比, 编码器模型的效率较低, 因为只有一部分序列 token 用作训练标签. 此外, 编码器模型无法生成序列.

用 ⟨mask⟩ 替换随机选择 token 的过程意味着与后续的微调集相比, 训练集存在不匹配的问题, 因为后者不包含任何 ⟨mask⟩ token. 为了减少这种情况可能导致的问题,Devlin et al.(2018) 对随机选择 token 的过程稍微做了修改, 从而在 15% 的随机选择 token 中,80% 用 ⟨mask⟩ 代替,10% 用词汇表中随机选取的一个词代替,10% 将原始词保留在输入端 (但仍需要在输出端对其进行正确预测).

编码器模型经过训练后, 就可以针对各种不同的任务进行微调. 为此, 需要构建一个新的输出层, 其形式与所要解决的任务相匹配. 在文本分类任务中, 只使用第一个输出位置, 其对应于总是出现在输入序列中第一个位置的 ⟨class⟩ token. 如果输出矩阵的维度为 D, 那么第一个输出节点就会添加一个维度为 $D \times K$ 的参数矩阵, 其中 K 为类的数量, 然后将其依次输入一个 K 维的 softmax 函数或一个维度为 $D \times 1$ 的向量,$K = 2$ 时则将其输入一个逻辑斯谛 sigmoid 函数. 线性输出变换也可以用更复杂的可微分模型 (如多层感知机) 替代. 如果希望对输入字符串中的每个 token 进行分类, 例如将每个 token 归入一个类 (如人、地点、颜色等), 那么第一个输出将被忽略, 随后的输出会有一个共享的 “线性变换 + softmax” 层. 在微调过程中, 包括新输出矩阵在内的所有模型参数都是通过随机梯度下降法并利用正确标签的对数概率来学习的. 另外, 预训练模型的输出也可输入复杂的生成式深度学习模型, 用于文本到图像合成等应用 (参见第 20 章).

命题 12.11 (掩码语言建模的损失覆盖)

设输入序列长度为 N, 被选中遮掩的位置集合为 $\mathcal{M}$. 若只对被选中位置的输出计算交叉熵, 则训练损失为 $-\sum_{n\in\mathcal{M}} \ln p\left(x_n \mid \tilde{\mathbf{x}}\right)$, 其中 $\tilde{\mathbf{x}}$ 为遮掩后的输入; 若 $\mathcal{M}$ 以独立概率 q 选取每个位置, 则每个位置进入损失的期望次数为 q, 因而同样数目的梯度步所覆盖的 “监督信号” 约为自回归模型的一小部分.

♠

证明

损失是各被选中位置负对数似然之和, 只有这些位置的目标已知, 其余位置的输出不进入损失. 由期望的线性性, 位置 n 被选中的概率为 q, 故它进入损失的期望次数为 q;N 个位置合计为 qN 个有效标签, 而自回归模型在同一序列上有 N 个标签. 文中取 $q = 15\%$, 即得结论.

□

注

(重构新增) 该命题给出文中“编码器模型效率较低”的定量解释: 同一段文本, 自回归训练能提供 N 个监督信号 (每个位置都预测下一个 token), 而掩码训练只提供约 $qN = 0.15N$ 个. 这也是编码器模型无法直接用于生成的原因——它从未学习“给定前缀续写”的条件分布.80/10/10 的替换规则则是为了缓解训练与微调的不匹配: 若永远只用 ⟨mask⟩, 模型将只学会“看到 mask 就填词”, 而微调时输入中根本没有 mask; 保留 10% 原词、10% 随机词, 使模型对“输入可能被污染”具有鲁棒性.

12.3.4 序列到序列 Transformer

为完整起见, 我们简要讨论一下第三类 Transformer 模型, 此类 Transformer 模型将编码器与解码器做了结合, 正如 Vaswani et al.(2017) 在最初的 Transformer 论文中所讨论的那样. 请参阅将句子从英语翻译成荷兰语的任务. 如前所述, 我们可以使用解码器型 Transformer 逐个生成与荷兰语输出相对应的 token 序列 (参见小节 12.3.1). 主要区别在于, 该输出需要以与英文句子相对应的整个输入序列为条件. 编码器型 Transformer 可用于将输入 token 序列映射为合适的内部表示, 我们用 $\mathbf{Z}$ 表示. 为了将 $\mathbf{Z}$ 纳入输出序列的生成过程, 我们使用了一种经过修正的注意力机制, 称为交叉注意力 (cross attention). 交叉注意力与自注意力相似, 只是虽然查询向量来自正在生成的序列 (在本例中指的是荷兰语输出序列), 但键向量和值向量来自 $\mathbf{Z}$ 所代表的序列, 如图 12.19所示. 回到我们对视频流服务的类比, 这就好比用户将其查询向量发送给不同的流媒体公司, 流媒体公司会将其与自己的键向量集作比较, 以找到最佳匹配, 然后以电影的形式返回相关的值向量. 将编码器和解码器组件结合起来, 即得到图 12.20所示的模型架构. 该模型可以使用成对的输入句子和输出句子进行训练.

定义 12.14 (交叉注意力)

设编码器输出为 $\mathbf{Z} \in \mathbb{R}^{N' \times D}$, 解码器当前层的表示为 $\mathbf{X} \in \mathbb{R}^{N \times D}$. 若查询由解码器一侧给出而键、值由编码器一侧给出, 即

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}, \qquad \mathbf{K} = \mathbf{Z}\mathbf{W}^{(k)}, \qquad \mathbf{V} = \mathbf{Z}\mathbf{W}^{(v)}, \tag{12.38}$$

再按(12.15)计算输出, 则称该注意力为**交叉注意力** (cross attention).

♣

注

(重构新增) 交叉注意力与自注意力的唯一差别在于 $(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ 的来源: 自注意力三者同源, 故 $N \times N$ 的注意力矩阵刻画“序列内部各位置的关系”; 交叉注意力中 $\mathbf{Q}$ 来自解码器 (长度 N)、$\mathbf{K}, \mathbf{V}$ 来自编码器 (长度 N'), 注意力矩阵为 $N \times N'$, 刻画“输出每个位置应读取输入的哪些部分”. 这正是机器翻译中“对齐”的软实现: 生成第 n 个荷兰语词时, 模型可以按权重读取英语句子的任意位置. 条件 $D_q = D_k$ 仍然必要, 但两侧序列长度可以不同, 这也是它与自注意力在形状上最显著的区别. 编码器只计算一次 $\mathbf{Z}$, 所有解码层复用它, 因此交叉注意力不增加编码成本.

12.3.5 大语言模型

机器学习领域最近最重要的发展是为自然语言处理创建了基于 Transformer 的超大型神经网络——大语言模型 (Large Language Model, LLM). 此处的“大”指的是网络中权重和偏置参数的数量规模大, 在撰写本书时, 这些参数的数量可高达约一万亿 (10^{12}). 这种模型的训练成本很高, 但其所拥有的卓越能力促使着人们建立这种模型.

除了大型数据集的可用性, 基于图形处理单元和类似处理器的大规模并行训练硬件的出现也为训练更大型的模型提供了便利, 这些硬件在配备了快速互联机制和大量内置存储器的大型集群中紧密耦合.Transformer 架构在这些模型的开发过程中发挥了关键作用, 因为它能够非常有效地利用这些硬件. 通常情况下, 增加训练集的规模, 同时相应增加模型参数的数量, 所带来的性能改进会超过架构改进或纳入更多领域知识的方法 (Sutton, 2019; Kaplan et al., 2020). 例如,GPT 系列模型 (Radford et al., 2019; Brown et al., 2020; OpenAI, 2023) 连续几代性能的显著提升主要归功于参数规模的扩大. 这类性能改进推动了一种新摩尔定律的产生: 自 2012 年以来, 训练一个最先进的机器学习模型所需的计算运算次数呈指数增长, 倍增的时间约为 3.4 个月 (参见第 1 章). 此外, 这类模型的性能也在以惊人的速度不断提高.

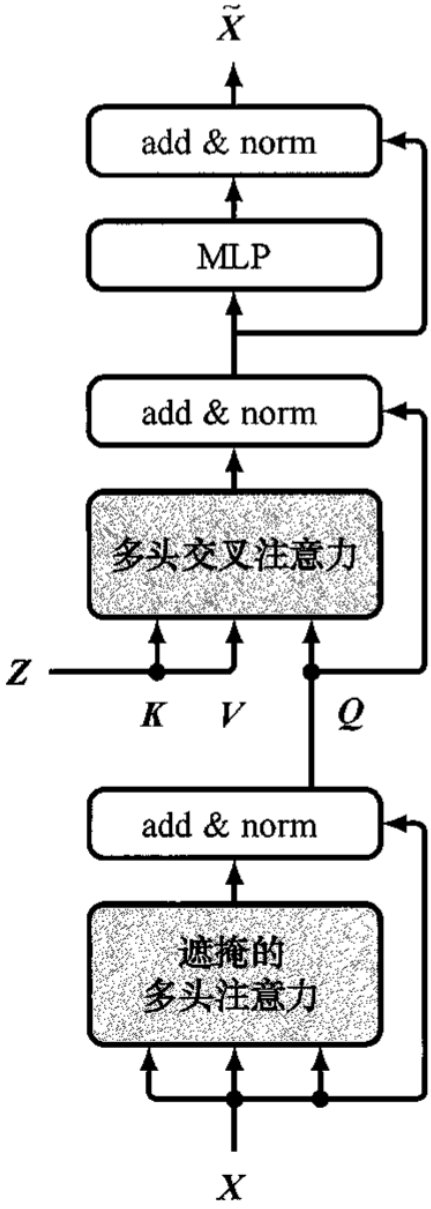


图 12.19: 序列到序列 Transformer 的解码器部分使用的交叉注意力层示意图. 此处 **Z** 表示编码器部分的输出, “add & norm” 代表残差连接和随后的层归一化, “MLP” 代表多层感知机.**Z** 还表示交叉注意力层的键向量和值向量, 而查询向量则由解码器部分决定.

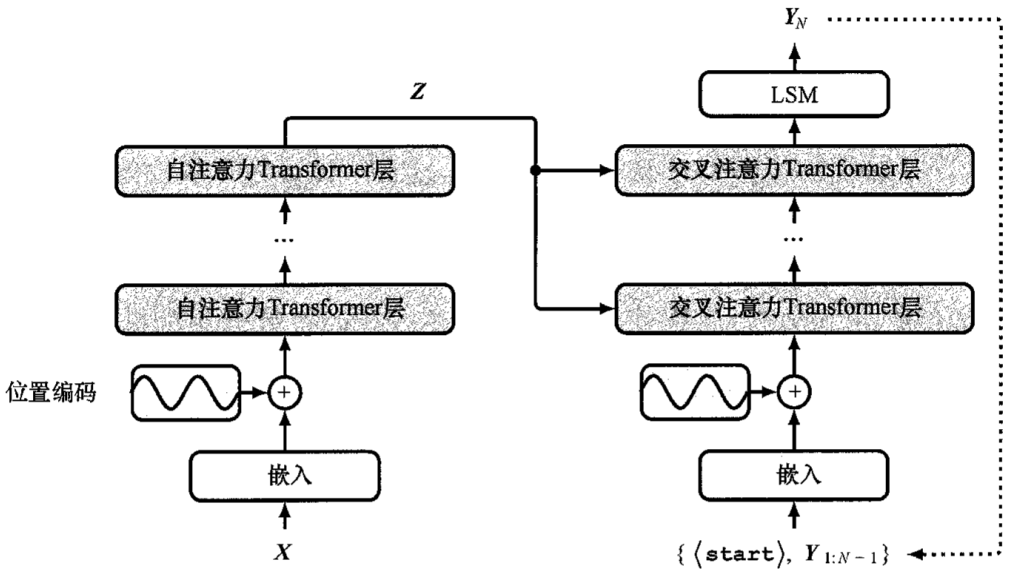


图 12.20: 序列到序列 Transformer 的示意图. 在这里, 输入 token 和输出 token 都显示为方框. 位置编码向量被添加到编码器和解码器部分的输入 token 中. 编码器中的每一层都与图 12.9所示的结构相对应, 每个交叉注意层的形式如图 12.19所示.

早期的语言模型采用监督学习的方法进行训练. 例如, 为了建立一个翻译系统, 训练集将由两种语言的配对句子组成. 然而, 监督学习的一个主要局限是, 数据通常必须经过人工整理, 以提供有标签数据, 这严重限制了可用数据的数量, 因此需要大量使用归纳偏置, 如特征工程和架构约束, 以实现合理的性能.

相反, 大语言模型是通过对超大型文本数据集进行自监督学习来训练的, 同时还可能包括计算机代码等其他 token 序列. 我们已经了解了如何在 token 序列上训练解码器型 Transformer(参见小节 12.3.1), 其中每个 token 都是一个有标签的目标示例, 而前面的序列则是输入, 从而学习条件概率分布. 这种 “自标签” 大大增加了可用的训练数据量, 因而可以利用具有大量参数的深度神经网络.

使用自监督学习引发了一种范式转变, 即首先使用无标签数据对一个大的模型进行预训练 (pre-trained), 然后使用基于小得多的有标签数据集的监督学习对模型进行微调 (fine-tuned). 这实际上是一种迁移学习, 同一个预训练模型可用于多个“下游”应用. 具有广泛功能且随后可以针对特定任务进行微调的模型称作基础模型 (foundation model)(Bommasani et al., 2021).

我们可以通过在网络输出中增加额外层, 或用新参数替换最后几层, 然后使用有标签数据训练最后几层来实现微调. 在微调阶段, 主模型中的权重和偏置既可以保持不变, 也可以进行小幅调整. 与预训练相比, 微调的成本通常较低.

一种非常有效的微调方法叫作低秩自适应 (Low-Rank Adaptation, LoRA)(Hu et al., 2021). 这种方法的灵感来自一些结果, 这些结果表明, 经过训练的过度参数化模型在微调方面具有较低的内在维度, 这意味着微调过程中模型参数的变化位于一个流形 (manifold) 上, 该流形的维度远远小于模型中可学习参数的总数 (Aghajanyan, Zettlemoyer, and Gupta, 2020). 低秩自适应利用了这一点, 做法是冻结原始模型的权重, 并以低秩乘积的形式在 Transformer 的每一层添加额外的可学习权重矩阵. 通常只修改注意力层的权重, 多层感知机层的权重则保持不变. 考虑一个维度为 $D \times D$ 的权重矩阵 $\mathbf{W}_0$, 它可能代表一个查询矩阵、键矩阵或值矩阵, 其中来自多个注意力头的矩阵则视为一个单独的矩阵. 如图 12.21所示, 我们引入了一个平行权重集, 它由两个矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 的乘积定义, 这两个矩阵的维度分别为 $D \times R$ 和 $R \times D$. 然后, 该层产生一个由 $\mathbf{XW}_0 + \mathbf{XAB}$ 给出的输出矩阵. 与原始权重矩阵 $\mathbf{W}_0$ 中的 D^2 个参数相比, 附加权重矩阵 $\mathbf{AB}$ 中的参数数量为 $2RD$. 因此, 如果 $R \ll D$, 则微调过程中需要调整的参数数量远少于原始 Transformer 中的参数数量. 实际上, 这可以将需要训练的参数数量减少到原来的万分之一. 一旦完成微调, 就可以将附加权重添加到原始权重矩阵中, 从而得到新的权重矩阵:

$$\tilde{\mathbf{W}} = \mathbf{W}_0 + \mathbf{AB} \tag{12.39}$$

在推理过程中, 与运行原始模型相比, 我们不会有额外的计算开销, 因为更新后的模型与原始模型大小相同.

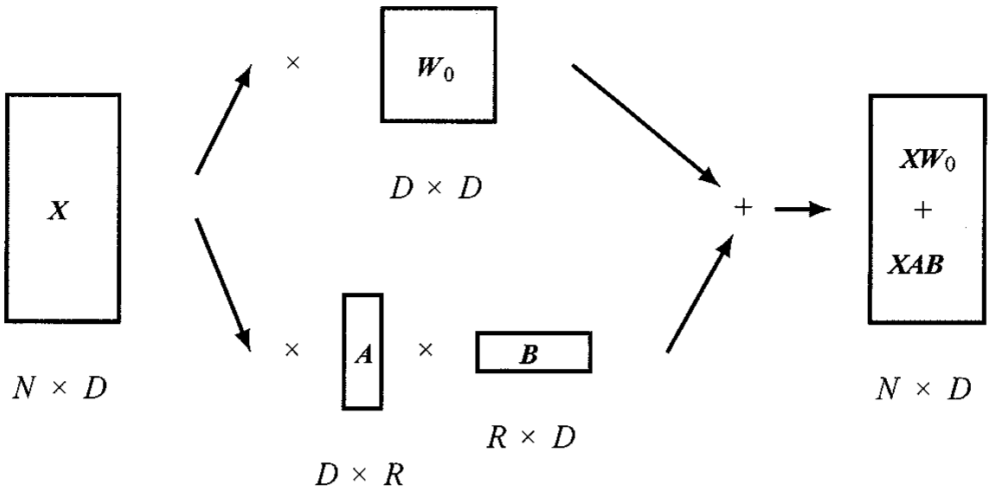


图 12.21: 低秩自适应的示意图, 这里展示了预训练 Transformer 中一个注意力层的权重矩阵 $\mathbf{W}_0$. 在微调过程中, 对矩阵 **A** 和 **B** 给出的附加权重进行调整, 然后将它们的乘积 **AB** 添加到原始权重矩阵中, 用于后续推理.

随着语言模型变得越来越大, 对微调的需求越来越少, 现在只需要通过基于文本的交互, 生成式语言模型就能执行广泛的任务. 例如, 假设给出一个文本串

English: the cat sat on the mat. French:

作为输入序列, 自回归语言模型就可以继续生成后续 token, 直至生成 ⟨stop⟩ token, 其中新生成的 token 代表法语翻译. 请注意, 该模型并不是专门为翻译而训练的, 而是在包括多种语言在内的大量数据语料上训练出来的.

用户可以使用自然语言与这些模型交互. 为了改善用户体验和所生成输出的质量, 研究人员使用基于人类反馈的强化学习 (Reinforcement Learning from Human Feedback RLHF) 等方法, 开发出了一些通过人工评估生成输出来微调大语言模型的技术 (Christiano et al., 2017). 这些技术支持创建具有简单易用对话界面的大语言模型, 其中最著名的是 OpenAI 的 ChatGPT.

用户给出的输入 token 序列称为提示 (prompt). 例如, 它可能包含一个故事的开场白, 而模型需要完成这个故事. 它也可能包含一个问题, 而模型应该提供答案. 通过使用不同的提示, 同一个经训练的神经网络可以完成多种任务, 例如根据简单的文本请求生成计算机代码, 或根据要求写出押韵的诗歌. 模型的性能取决于提示的形式, 这催生了一个叫作提示工程 (prompt engineering) 的新领域 (Liu et al., 2021), 其目的是为提示设计一种好的形式, 从而为下游任务带来高质量的输出. 在将用户提示输入语言模型之前, 还可以通过调整用户提示来修改模型的行为, 即在用户提示前预置一个称为前缀提示 (prefix prompt) 的额外 token 序列以修改输出的形式. 例如, 前缀提示可能包括用标准英语表达的指令, 以告诉网络不要在输出中包含攻击性语言.

这样, 只需要在提示中提供一些示例, 模型就能解决新任务, 而无须调整模型参数. 这是小样本学习 (few-shot learning) 的一个示例.

目前最先进的模型 (如 GPT-4) 已经变得十分强大, 以至于它们表现出一些非凡的特性, 这些特性被描述为通用人工智能的初步迹象 (Bubeck et al., 2023), 并正在推动新一轮的技术创新.

12.4 多模态 Transformer

虽然 Transformer 最初是作为处理顺序语言数据的递归网络的替代品而开发的, 但 Transformer 模型已在深度学习的几乎所有领域得到普及. 事实证明, 它们是通用模型, 因为它们对输入数据的假设很少, 而对卷积网络的等变性和局部性有很强的假设. 由于其通用性,Transformer 已成为处理许多不同模态数据 (包括文本、图像、视频、点云和音频数据) 的最先进技术, 并已用于这些领域中每个领域的判别和生成应用 (参见第 10 章).Transformer 层

的核心架构在不同时期和不同应用中保持相对稳定. 因此, 能够在自然语言以外的领域使用 Transformer 的关键创新主要集中在输入和输出的表示及编码方面.

能够处理多种不同数据的单一架构有一个巨大优势, 就是能使多模态计算变得相对简单. 在这种情况下, 多模态指的是将两种或两种以上类型的数据结合在一起. 例如, 我们可能希望根据文字提示生成图像, 或者设计一个能将摄像头、雷达和麦克风等多个传感器捕获的信息结合起来的机器人. 重要的是, 如果我们能对输入进行 token 化并对输出 token 进行解码, 那么我们就有可能使用 Transformer.

12.4.1 视觉 Transformer

Transformer 已成功应用于计算机视觉领域, 并在许多任务中有着不俗的表现. 判别任务最常见的选择是标准 Transformer, 其在视觉领域又称视觉 Transformer(Vision Transformer, 简称 ViT)(Dosovitskiy et al., 2020). 在使用 Transformer 时, 我们需要决定如何将输入图像转换为 token, 最简单的选择是将每个像素作为一个 token 并按照线性投影进行转换. 然而, 标准 Transformer 实现所需的内存与输入 token 的数量成二次方增长关系, 因此这种方法通常是不可行的. 相反, 最常见的分词方法是将图像分割成一组大小相同的小块. 假设图像的尺寸为 $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$, 其中 H 和 W 分别指的是图像的高度和宽度 (以像素为单位),C 是通道数 (R、G、B 三种颜色的通道数通常为 3). 每幅图像被分割成大小为 $P\times P$ 的非重叠小块 ($P = 16$ 是常见的选择), 然后 “平铺” 成一维向量, 这样就得到了图像尺寸的另一种表示 $\mathbf{x}_p \in \mathbb{R}^{N\times(P^2C)}$, 其中 $N = HW/P^2$ 是一幅图像的小块总数. 用于分类任务的视觉 Transformer 的架构如图 12.22所示.

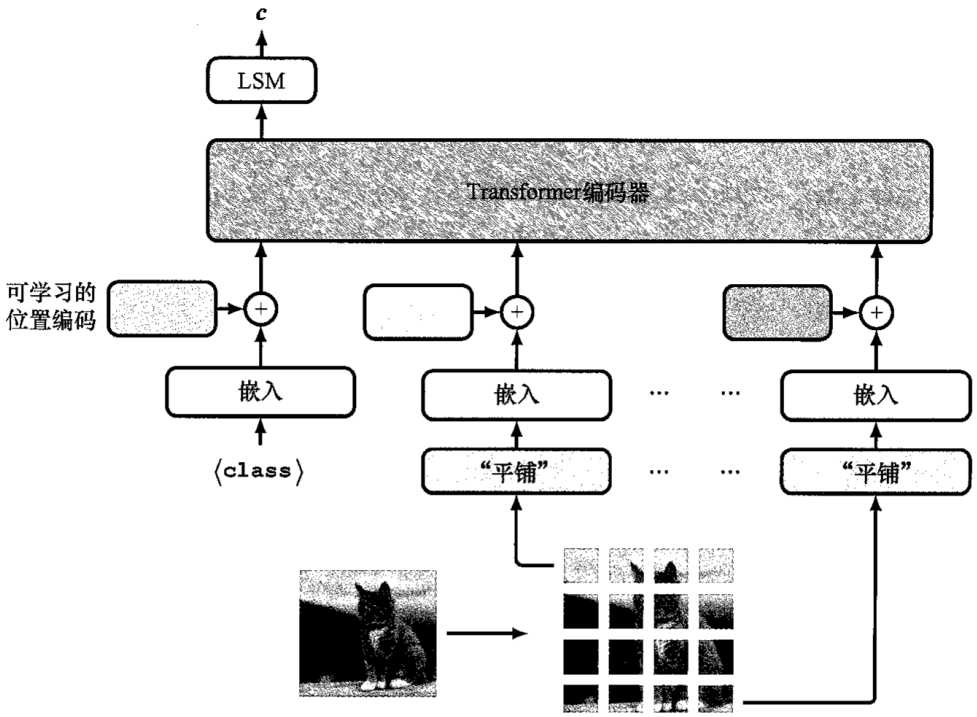


图 12.22: 用于分类任务的视觉 Transformer 的架构. 此处将一个可学习的 token ⟨class⟩ 作为额外的输入, 相关的输出由一个具有 softmax 激活函数的线性层 (记作 LSM) 进行转换, 从而得到最终的类-向量输出 $\mathbf{c}$.

另一种分词方法是将图像输入一个小型卷积神经网络 (参见第 10 章). 这样就可以对图像进行采样, 从而得到数量可控的 token, 每个 token 由一个网络输出表示. 例如, 一个典型的 ResNet18 编码器将在高度和宽度两个维度对图像进行 8 倍的下采样, 所得到的 token 数量是像素数量的 1/64.

我们还需要一种在 token 中编码位置信息的方法. 我们可以构建明确的位置嵌入, 对图像小块的二维位置信息进行编码. 但在实践中, 这一般不会提高性能, 因此最常见的做法是只使用习得的位置嵌入. 与用于自然语言的 Transformer 不同, 视觉 Transformer 通常将固定数量的 token 作为输入, 从而避免了习得的位置编码无法泛化到不

同大小的输入的问题.

视觉 Transformer 的架构设计与卷积神经网络截然不同. 虽然卷积神经网络中会产生强归纳偏置, 但视觉 Transformer 中唯一的二维归纳偏置是用于 token 化输入的小块造成的. 因此, 视觉 Transformer 通常比卷积神经网络需要更多的训练数据, 因为它必须从头开始学习图像的几何特性. 不过, 由于对输入的结构没有很强的假设, 视觉 Transformer 通常能够收敛到更高的精度. 这再次说明了归纳偏置与训练数据规模之间需要权衡 (Sutton, 2019).

定义 12.15 (图像块 token 化)

设图像 $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$, 小块边长为 P(P 整除 H, W). 把图像划分为 $N = HW/P^2$ 个互不重叠的 $P \times P \times C$ 小块, 并将每块展平为长度 P^2C 的向量, 得到矩阵 $\mathbf{x}_p \in \mathbb{R}^{N\times(P^2C)}$, 再经可学习的线性投影映射到 D 维, 则称这一过程为图像的**块 token 化**. ♣

注

(重构新增) 块 token 化的动机是复杂度: 若以像素为 token,$N = HW$ 可达 10^5 量级, 注意力的 $O(N^2D)$ 代价无法承受 (见命题 12.4); 以 16×16 小块为单位,224×224 的图像得到 $N = 196$ 个 token, 复杂度降到可行范围. 条件“P 整除 H, W”保证划分无剩余, 否则需要补齐. 这一设计也解释了 ViT 的归纳偏置为何弱于卷积网络: 卷积核天然编码了“局部性”与“平移等变”, 而块 token 化只保留了“局部小块”这一条, 其余几何关系必须由数据和位置编码学出来, 因此需要更大规模的预训练数据. 线性投影的维度 D 与语言模型一致, 使同一套 Transformer 层可以直接复用.

除了图像块的 token, 我们还为输入序列增加了一个可学习的分类 token(用 $\mathbf{c}$ 表示), 如图 12.22所示. 我们将整个序列输入一个仅编码器 Transformer(参见小节 12.3.3). 这个序列有 196 个图像 token 和 1 个分类 token, 其中分类 token 最终编码了图像的整体表示, 可用于下游分类. 在经过 L 个 Transformer 层处理后, 我们可以从最终输出序列中提取出分类 token, 并使用全连接层进行处理以执行分类任务.图 12.22所示的视觉 Transformer 架构使用了 $L = 12$ 层、维度为 $D = 768$ 和 12 个注意力头, 总共有大约 8600 万个可学习参数.

值得注意的是, 虽然视觉 Transformer 是在 ImageNet 数据集上进行预训练的, 但它们在小型数据集上的表现通常不如卷积网络 (例如 ResNet), 因为它们没有内建的图像结构归纳偏置. 当在大型数据集上进行预训练时, 视觉 Transformer 的性能优于卷积网络. 因此, 卷积网络通常可以作为特征提取器, 为 Transformer 提供输入.

12.4.2 图像生成 Transformer

在语言领域,Transformer 在用作合成文本的自回归生成式模型时取得了令人瞩目的成果. 因此, 人们自然会问是否也可以使用 Transformer 来合成逼真的图像. 由于自然语言本身具有顺序性, 因此其非常适合自回归框架, 而图像的像素没有自然顺序, 因此自回归解码是否有用并不那么直观. 然而, 只要首先定义变量的某种排序, 任何分布都可以分解为条件式的乘积 (参见节 11.3). 有序变量 $x_1, \cdots, x_N$ 的联合分布可以写成

$$p(x_1, \cdots, x_N) = \prod_{n=1}^{N} p(x_n \mid x_1, \cdots, x_{n-1}) \tag{12.40}$$

这种分解是完全通用的, 对各个条件分布 $p(x_n \mid x_1, \cdots, x_{n-1})$ 的形式没有任何限制.

对于一幅图像, 我们可以选择 x_n 作为 R、G、B 值的三维向量来表示第 n 个像素. 现在我们需要确定像素的排序, 其中一种使用广泛的排序方式叫栅格扫描 (raster scan), 如图 12.23所示. 使用基于栅格扫描排序的自回归模型生成图像如图 12.24所示.

请注意, 图像自回归生成式模型的使用早于 Transformer 的引入. 例如,PixelCNN(Oord et al., 2016) 和 PixelRNN(Oord, Kalchbrenner, and Kavukcuoglu, 2016) 使用了定制的遮掩卷积层, 其保留了(12.40)右侧相应项为每个像素定义的条件独立性.

使用连续值的图像表示可以很好地完成判别任务. 不过, 使用离散表示法生成图像的效果要好得多. 通过最大似然法习得的连续条件分布, 例如负对数似然函数为平方和误差函数的 Gauss 分布, 往往会学习到训练数据的均值, 从而导致图像模糊 (参见节 4.2). 相反, 离散分布可以轻松处理多模态问题. 例如,(12.40)中的条件分布 $p(x_n \mid x_1, \cdots, x_{n-1})$ 可能会得出一个像素应该是黑色或白色的结论, 而回归模型可能会得出该像素应该是灰色的结论.

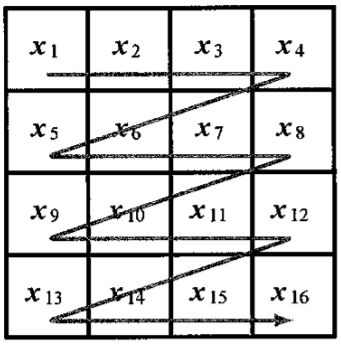


图 12.23: 定义二维图像中像素的特定线性排序的栅格扫描示意图.

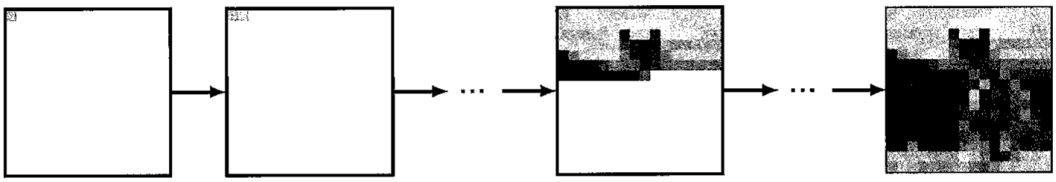


图 12.24: 从自回归模型中进行图像采样的示意图. 按栅格扫描顺序, 第一个像素从边缘分布 $p(x_1)$ 中采样, 第二个像素从条件分布 $p(x_2 \mid x_1)$ 中采样, 以此类推, 直至得到一幅完整的图像.

然而, 使用离散空间也会带来挑战. 图像像素的 R、G、B 值通常至少用 8 位精度来表示, 因此每个像素都有 $2^{24} \approx 1600$ 万个可能值. 在维度如此高的空间中学习条件 softmax 分布是不可行的.

解决高维度问题的一种方法是使用向量量化 (vector quantization) 技术, 它可以视为一种数据压缩形式. 假设我们有一组数据向量 $\mathbf{x}_1, \cdots, \mathbf{x}_N$, 其中每个向量的维度为 D, 它可能代表图像像素. 然后我们引入一组 K 个码本向量 $C = \mathbf{c}_1, \cdots, \mathbf{c}_K$, 它们的维度也是 D, 且通常情况下 $K \ll D$. 接下来, 我们根据某种相似度量 (通常是欧几里得距离), 将每个数据向量近似为其最近的码本向量 (codebook vector), 从而

$$\mathbf{x}_n \to \arg\min_{\mathbf{c}_k \in C} \|\mathbf{x}_n - \mathbf{c}_k\|^2 \tag{12.41}$$

由于有 K 个码本向量, 我们可以用一个独热编码的 K 维向量来表示每个 $\mathbf{x}_n$, 而且由于我们可以选择 K 的值, 因此我们可以通过使用较大的 K 值来更准确地表示数据, 或使用较小的 K 值来更大程度地压缩数据, 从而进行两者之间的权衡.

定义 12.16 (向量量化)

设码本 $C = \{\mathbf{c}_1, \cdots, \mathbf{c}_K\} \subset \mathbb{R}^D$. 映射 $Q : \mathbb{R}^D \to C, Q(\mathbf{x}) = \arg\min_{\mathbf{c} \in C} \|\mathbf{x} - \mathbf{c}\|$, 称为**向量量化** (vector quantization); 每个数据点用它最近码本的索引表示, 索引再经独热编码送入后续模型. ♣

注

(重构新增) 向量量化的作用是把“连续高维取值”变成“有限字典上的离散符号”, 从而让 softmax 型语言模型可以直接用于像素.K 的选择就是保真度与可学习性的权衡:K 太小, 量化误差大, 重建图像失真;K 太大, 每个码本被数据覆盖的机会变小,softmax 参数难以估计 (与命题 12.7中零频问题同源).arg min 不可微是这里的结构性障碍——梯度无法穿过“取最近邻”这一步, 因此实践中需要用直通估计 (straight-through estimator) 把梯度近似地复制到编码器一侧.(12.41)定义的是硬分配; 若改用 softmax 给出各码本的概率, 就得到软量化, 这与注意力的软硬之分是同一思想.

我们可以将原始图像的像素映射到低维码本空间, 然后训练自回归 Transformer 以生成一组码本向量, 并通过用相应的 D 维码本向量 $\mathbf{c}_k$ 替换每个码本索引 k, 将这组码本向量映射回原始图像空间.

自回归 Transformer 最早应用于 ImageGPT(Chen, Radford, et al., 2020) 中的图像. 此处, 每个像素都可以视为一组离散的三维颜色码本向量之一, 每个三维颜色码本向量都与颜色空间的 k 均值聚类中的一个聚类相对应 (参

见第 15 章). 因此, 独热编码可以提供离散 token(类似于语言 token), 并允许 Transformer 以与语言模型相同的方式进行训练, 以实现下一个 token 分类目标. 这是表示学习的一个强大目标 (参见小节 6.3.3), 可用于后续微调 (同样以类似于语言建模的方式进行).

然而, 直接使用单个像素作为 token 可能会导致计算成本过高, 因为每个像素都需要进行一次前向传递, 这意味着训练和推理的效率都会随着图像分辨率的提高而降低. 此外, 使用单个像素作为输入意味着必须使用低分辨率图像, 以便在栅格扫描后期解码像素时提供合理的上下文长度. 正如我们在视觉 Transformer 中看到的那样, 最好使用图像的小块而不是像素作为 token, 因为这样 token 的数量会大大减少, 从而有利于处理更高分辨率的图像. 和以前一样, 由于条件分布可能具有多模态性, 我们需要使用 token 值的离散空间. 于是再次出现维度挑战, 由于维度与小块中像素的数量成指数关系, 因此小块的维度比单个像素的维度大得多. 例如, 即使只有代表黑色和白色两种可能像素的 token, 以及大小为 16×16 的小块, 我们也会有一个内含 $2^{256} \approx 10^{77}$ 个小块的 token 字典.

我们再次使用向量量化技术来解决维度挑战. 简单的聚类 (如 k 均值聚类) 方法或更复杂的方法 (如全卷积网络)(Oord, Vinyals, and Kavukcuoglu, 2017; Esser, Rombach, and Ommer, 2020), 甚至包括视觉 Transformer(Yu et al., 2021), 都可以用来从图像小块数据集中习得码本向量. 将每个小块映射到离散的代码集, 之后再映射回来的一个问题是, 向量量化是一种不可微分操作. 幸运的是, 我们可以使用一种称为直通梯度估计的技术 (Bengio, Léonard, and Courville, 2013), 这是一种简单的近似法, 只需要在反向传播过程中通过不可微分函数复制梯度即可.

使用自回归 Transformer 生成图像也可以扩展到生成视频, 方法是将视频视为这类向量量化 token 的一个长序列 (Rakhimov et al., 2020; Yan et al., 2021; Hu et al., 2023).

12.4.3 音频数据

接下来让我们看看 Transformer 在音频数据中的应用. 声音通常以波形的形式存储, 它们可以通过在固定时间间隔内测量气压的振幅来获得. 虽然这种波形可以直接作为深度学习模型的输入, 但在实践中, 将其预处理为梅尔频谱 (mel spectrogram) 更为有效. 这是一个矩阵, 其中的列代表时间步, 行代表频率. 这些频段遵循标准惯例 (标准惯例是通过主观评估选择的), 从而使得连续频率之间的感知差异相等 (“mel” 一词来源于旋律). 梅尔频谱如图 12.25所示.

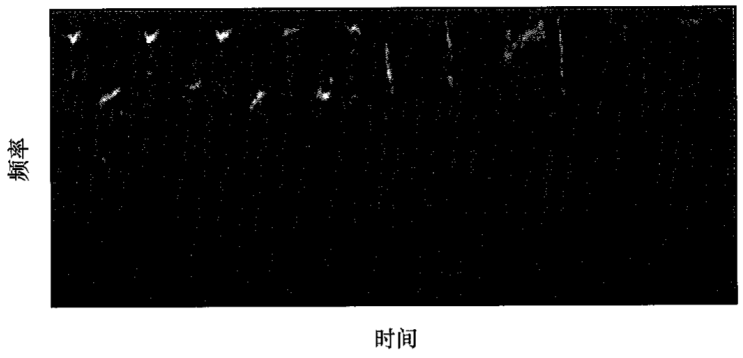


图 12.25: 座头鲸叫声的梅尔频谱 (librosa 开发团队版权所有 ©2013–2023).

Transformer 在音频领域的一个应用是分类, 旨在将音频片段归入若干预定义类别之一. 例如,AudioSet 数据集 (Gemmeke et al., 2017) 就是一个使用广泛的基准, 其中包含 “汽车” “动物” “笑声” 等类. 在 Transformer 出现之前, 最先进的音频分类方法是将梅尔频谱作为图像进行处理, 并将其作为卷积神经网络的输入 (参见第 10 章). 不过, 虽然卷积神经网络擅长理解局部关系, 但它的一个缺点是难以处理远距离依赖关系, 而这在处理音频时可能非常重要.

正如 Transformer 取代递归神经网络成为自然语言处理领域的最先进技术一样,Transformer 在音频分类等任务中也开始取代卷积神经网络. 例如, 如图 12.18所示, 编码器型 Transformer 与用于语言和视觉的 Transformer 结构相同, 可用于预测音频输入的类 (Gong, Chung, and Glass, 2021). 此处, 梅尔频谱可以视为图像, 然后进行 token 化. 具体做法是以类似于视觉 Transformer 的方式将图像分割成不同的小块, 可能会有一些重叠, 以避免丢失任何

重要的邻域关系. 然后对每个小块进行扁平化处理, 也就是将它们转换为一维数组, 在本例中长度为 256. 之后为每个 token 添加唯一的位置编码, 并附加特定的 token ⟨class⟩, 接下来通过编码器型 Transformer 输入 token. 我们可以使用一个线性层和一个 softmax 激活函数对最后一个转换层中与输入 token ⟨class⟩ 相对应的输出 token 进行解码, 并使用交叉熵损失对整个模型进行端到端训练.

12.4.4 文本语音转换

分类并不是深度学习 (更具体地说是 Transformer 架构) 在音频领域带来革命性变化的唯一任务.Transformer 成功地合成了模仿特定说话者声音的语音这一事实再次证明了 Transformer 的多功能性, 而 Transformer 在这项任务中的应用则是关于如何在新环境中应用 Transformer 的一项内容十分丰富的案例研究.

生成与给定文本段落相对应的语音叫作文本语音合成 (text-to-speech synthesis). 一种更传统的方法是收集特定说话者的录音, 然后训练一个监督回归模型, 从而根据相应的转录文本预测语音输出 (可能采用梅尔频谱的形式). 在推理过程中, 我们希望将合成语音的文本作为输入, 由此产生的梅尔频谱输出则可以解码回音频波形, 因为这是一个固定的映射.

不过, 这种方法有 3 个主要不足. 首先, 如果我们在低层次上预测语音, 例如使用叫作音素 (phoneme) 的子词组件, 则需要更大的上下文来使生成的句子听起来流畅. 但是, 如果我们预测更长的片段, 那么可能的输入空间就会显著增大; 要达到良好的泛化效果, 我们可能需要大量的训练数据. 其次, 这种方法不能在不同说话者之间传递知识, 因此每个新的说话者都需要大量数据. 最后, 这实际上是一个生成建模任务, 因为给定的说话者和文本对有多个正确的语音输出, 所以回归可能并不合适, 回归倾向于对目标值进行平均 (参见节 4.2).

相反, 如果我们将语音数据与自然语言同等对待, 并将文本语音合成作为一项条件语言建模任务, 那么我们就能以与基于文本的大语言模型大致相同的方式训练该模型. 此时我们需要解决两个主要的实现细节问题. 首先是如何对训练数据进行 token 化并对预测值进行解码, 其次是如何根据说话者的声音对模型进行调节.

Vall-E(Wang et al., 2023) 是一种利用 Transformer 和语言建模技术进行文本语音合成的方法. 只需要几秒样本语音, 就能将新文本映射成新说话者的语音. 语音数据通过向量量化获得的字典或码本 (codebook) 被转换成一组离散 token(参见小节 12.4.2), 我们可以将这类 token 视为类似于自然语言领域的独热编码 token. 输入由文本段落中的文本 token 组成, 而用于训练的目标输出则由相应的语音 token 组成. 如图 12.26所示, 来自同一说话者的一小段无关语音的附加语言 token 会被添加到输入文本 token 中. 通过加入来自许多不同说话者的额外语音 token, 系统可以在模仿这些额外语音 token 所代表的声音时学习朗读一个文本段落. 训练完成后, 系统就可以显示新的文本, 以及从新的说话者那里捕捉到的一段简短语音中的音频 token, 并使用训练时的相同码本对输出 token 进行解码, 以创建语音波形. 这样系统就能用新说话者的声音合成与输入文本相对应的语音.

12.4.5 视觉和语言 Transformer

我们已经了解了如何生成文本、音频和图像的离散 token, 接下来的问题是, 我们是否可以用一种模态的输入 token 和另一种模态的输出 token 来训练模型. 或者说, 我们是否可以将不同模态的输入或输出抑或它们两者结合起来, 在这里, 我们将重点讨论文本和视觉数据的组合, 因为这是研究最广泛的示例, 但原则上, 此处讨论的方法也可应用于其他输入和输出模态的组合.

第一个要求是, 我们要有一个庞大的数据集来进行训练.LAION-400M 数据集 (Schuhmann et al., 2021) 极大地促进了文生图和图生文的研究, 该数据集与 ImageNet 数据集一样, 对深度图像分类模型的发展起着关键作用. 文生图实际上与我们迄今为止研究过的无条件图像生成很相似, 只不过我们还允许模型将文本信息作为输入, 为生成过程提供条件. 使用 Transformer 时, 进行文生图非常简单, 因为我们只需要在解码每个图像 token 时提供文本 token 作为额外输入即可.

这种方法也可视为将文生图问题看作序列到序列的语言建模问题, 如机器翻译, 只不过目标 token 是离散的图像 token 而不是语言 token. 因此, 选择一个完整的编码器-解码器 Transformer 模型是合理的, 如图 12.20所示, 其中 $\mathbf{X}$ 对应于输入文本 token,$\mathbf{Y}$ 对应于输出图像 token. 一个名为 Parti 的模型 (Yu et al., 2022) 就采用了这种方法, 其中 Transformer 可扩展至 200 亿个参数, 同时随着模型规模的增大, 性能也得到了持续改善.

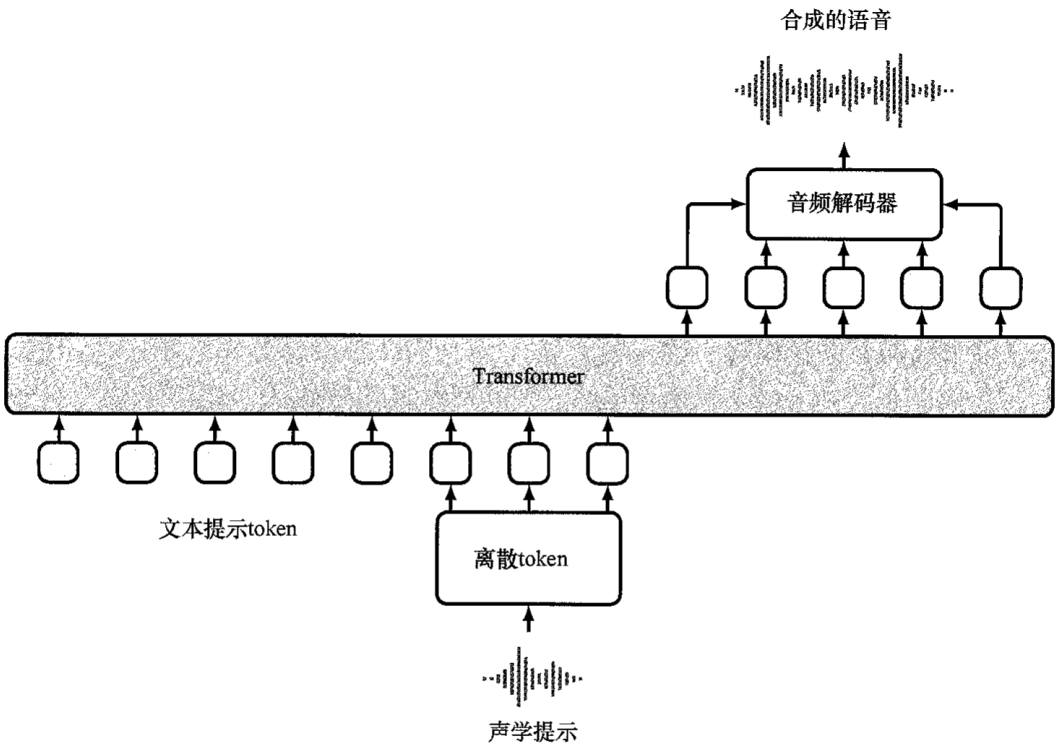


图 12.26: Vall-E 高级架构图.Transformer 模型的输入包括文本提示 token(提示模型合成的语音应包含哪些词) 和声学提示 token(确定说话者的风格和语调信息). 使用习得的解码器将采样的模型输出 token 解码为语音. 为简单起见, 这里没有显示位置编码和线性投影.

在使用预训练的语言模型并对其进行修改或微调, 以使其也能接收视觉数据作为输入方面, 业界也开展了大量研究 (Alayrac et al., 2022; Li et al., 2022). 这些研究主要使用定制架构和连续值图像 token, 因此并不适合同时生成视觉数据. 此外, 如果我们希望加入音频 token 等新模态, 也不能直接使用它们. 尽管这是向多模态迈出的一步, 但我们还是希望将文本 token 和图像 token 同时用作输入和输出. 最简单的方法是将所有内容都视为一组 token, 就像自然语言一样, 但使用的字典是语言 token 字典和图像 token 码本的组合. 这样我们就可以将任何视听数据流简单地视为一组 token 了.

在 CM3(Aghajanyan et al., 2022) 和 CM3Leon(Yu et al., 2023) 中, 语言建模的一种变体用于对 HTML 文档进行训练, 这些文档包含从在线资源中获取的图像和文本数据. 当大量的训练数据与可扩展的架构相结合时, 模型就会变得非常强大. 此外, 训练的多模态性质意味着模型非常灵活. 这类模型不仅能够完成许多原本需要特定任务模型架构和训练机制才能完成的任务, 如文生图、图生文、图像编辑、文本补全等, 还能够完成普通语言模型所能完成的任何任务.图 12.27给出了 CM3Leon 模型在文本和图像联合空间中执行各种不同任务的示例.

12.5 习题

✍ **练习 12.1(⋆⋆)** 考虑 $m = 1, \cdots, N$ 时的一组系数 a_{nm}, 它们必须满足

$$a_{nm} \geqslant 0 \tag{12.42}$$

$$\sum_m a_{nm} = 1 \tag{12.43}$$

使用 Lagrange 乘子证明它们还必须满足

$$a_{nm} \leqslant 1, \qquad \text{其中} n = 1, \cdots, N. \tag{12.44}$$

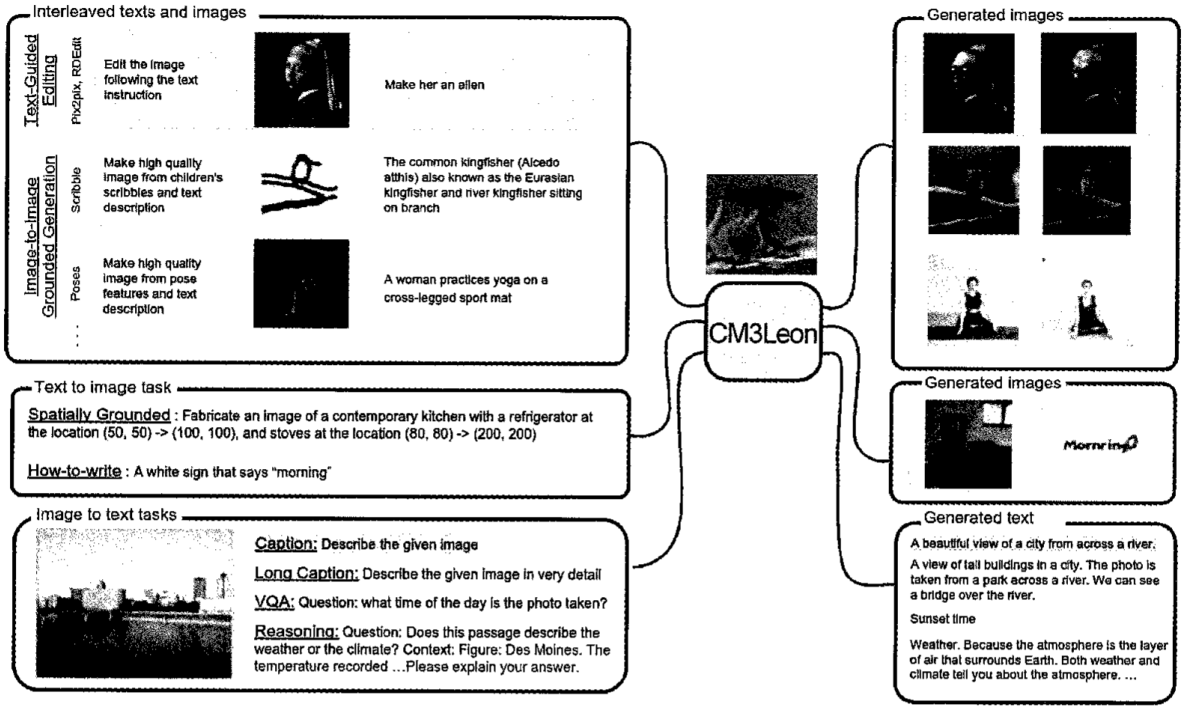


图 12.27: CM3Leon 模型在文本和图像联合空间中执行各种不同任务的示例 [经 Yu 等人 (2023) 授权使用].

注

(重构新增) 本题的真正内容是“归一化加非负把系数锁在 $[0,1]$ 内”,Lagrange 乘子只是把它形式化. 互补松弛条件在这里承担了关键作用: 它迫使除 a_n 外的所有系数取零, 从而把“最大”钉在顶点上. 这个上界是命题 12.1的直接来源, 也保证了注意力权重可以被解释为“比例”.

✍ **练习 12.2(⋆)** 验证对于向量 $\mathbf{x}_1,\cdots,\mathbf{x}_N$ 的任意值,softmax 函数 [(12.5)] 满足约束条件 [(12.3)和(12.4)].

注

(重构新增) 证明的关键是“分母恰为分子的和”, 这使得归一化是自动的而非外加的. 需要留意分母永远不为零: 即使所有 s_m 都很小, 每个指数项也严格大于零. 若改用线性归一化 $s_m/\sum s_{m'}$, 则在 s_m 有正有负时分母可能为零, 非负性也不成立——这正是选择指数函数的理由之一.

✍ **练习 12.3(⋆)** 考虑(12.2)定义的简单变换的输入向量 $\mathbf{x}_n$, 其中加权系数 a_{nm} 由(12.5)定义. 证明如果所有输入向量都是正交的, 即 $n\neq m$ 时 $\mathbf{x}_n^{\mathsf{T}}\mathbf{x}_m=0$, 则输出向量将等于输入向量, 即 $n=1,\cdots,N$ 时 $\mathbf{y}_n=\mathbf{x}_n$.

注

(重构新增) 原题把“正交”误当成“恒等映射”的充分条件, 实际起作用的量是点积的绝对尺度而非其为零. 正交只保证非对角元相等, 因而输出是“自身 + 其余向量等权平均”的混合; 要消除混合, 必须让对角元的指数远大于其余项, 也就是范数足够大或温度足够低. 这一辨析对理解实际模型很重要: 训练中注意力权重很少接近 one-hot, 模型正是靠这种“部分混合”来聚合上下文信息.

✍ **练习 12.4(⋆)** 考虑两个独立的随机向量 $\mathbf{a}$ 和 $\mathbf{b}$, 这两个向量的长度均为 D, 并且都取自均值为零、方差为单位方差的 Gauss 分布 $\mathcal{N}(\mathbf{0},\mathbf{I})$. 证明 $\left(\mathbf{a}^{\mathsf{T}}\mathbf{b}\right)^2$ 的期望值由 D 给出.

注

(重构新增) 本题是命题 12.3中“点积方差为 D_k”的严格来源:$\mathbb{E}[(\mathbf{a}^{\mathsf{T}}\mathbf{b})^2]$ 是零均值随机变量 $\mathbf{a}^{\mathsf{T}}\mathbf{b}$ 的二阶矩, 也就是它的方差, 因此点积的标准差为 $\sqrt{D}$. 这正是缩放因子取 $1/\sqrt{D_k}$ 的理由. 证明中“交叉项期望为零”依赖于两个向量的各个分量相互独立且均值为零; 若分量相关 (例如词向量各维高度相关), 方差会偏离 D, 缩放就不再精确.

✍ **练习 12.5(⋆⋆⋆)** 证明(12.19)定义的多头注意力可以重写为每个注意力头贡献之和的形式, 并给出相应的组合权重矩阵.

注

(重构新增) 该恒等式说明多头注意力在代数上就是“H 个带不同注意力矩阵的线性映射之和”, 并不比单头多出本质的表达类型; 多头的收益来自每个头拥有独立的 $\mathbf{W}_h^{(v)}, \mathbf{W}_h^{(o)}$ 与独立的注意力模式. 这里把 $\mathbf{W}_h^{(v)}$ 与 $\mathbf{W}_h^{(o)}$ 的连续乘积合并为一个 $\mathbf{W}^{(h)}$, 正是文中提到的“冗余”: 若不合并, 两个矩阵之间可以插入任意可逆矩阵而不改变乘积, 参数化不唯一. 合并后每个头只剩一个 $D \times D$ 矩阵, 参数量的记账也更清楚.

✍ **练习 12.6(★★)** 将自注意力函数 [(12.15)] 以矩阵的形式表达为一个全连接网络, 该矩阵可以将全部输入序列的合并词向量映射为相同维度的输出向量. 请注意, 这样的矩阵有 $O(N^2D^2)$ 个参数. 证明自注意力网络对应于参数共享矩阵的稀疏版本. 画出该矩阵的结构草图, 指出哪些参数块是共享的, 以及哪些参数块的所有元素都等于零.

注

(重构新增) 本题给出了命题 12.4的矩阵视角: 一般全连接层需要为每一对 (输入位置, 输出位置) 单独学习 D^2 个参数, 而注意力把这 N^2 个块用一个 $\mathbf{W}^{(v)}$ 统一起来, 块间的差异由只需 $O(ND^2)$ 计算量的点积给出. 这种“结构化的参数共享”正是归纳偏置的来源, 也解释了为什么注意力在 N 很大时远比全连接层经济. 需要注意的是,a_{nm} 虽不是参数, 却使整个映射对 $\mathbf{X}$ 非线性 (softmax 非线性), 因此它并不是一个线性层.

✍ **练习 12.7(★)** 证明如果省略输入向量的位置编码, 那么(12.19)定义的多头注意力层的输出对于输入序列的重新排序将是等变化的.

注

(重构新增) 证明的要点是“置换与逐行运算可交换”:softmax、层归一化、MLP 都在单个 token 内部进行, 因此重排 token 只是把结果一起重排. 这解释了为什么位置编码必须加在输入上而不是网络结构中——一旦加上依赖位置的向量,$\mathbf{PX}$ 与 $\mathbf{X}$ 的位置编码不再对应, 等变性被有意破坏, 模型才能区分词序. 反之, 若任务本身与顺序无关 (如集合上的分类), 保持等变性反而是优点.

✍ **练习 12.8(★★★)** 考虑两个 D 维单位向量 $\mathbf{a}$ 和 $\mathbf{b}$, 它们满足 $\|\mathbf{a}\| = 1$ 和 $\|\mathbf{b}\| = 1$ 并且是从随机分布中抽取的. 假设该随机分布是围绕原点的对称分布. 证明对于 D 的较大值, 这两个向量之间夹角的余弦值接近零, 因此这两个向量在高维空间中几乎是正交的. 为此, 考虑一个正交基 $\mathbf{u}_i$, 其中 $\mathbf{u}_i^\intercal \mathbf{u}_j = \delta_{ij}$ 并将 $\mathbf{a}$ 和 $\mathbf{b}$ 表示为这个正交基中的展开式.

注

(重构新增) 这个结论是位置编码可以“相加”而不严重破坏词向量的根据 (见定义 12.6): 把 $\mathbf{r}_n$ 加到 $\mathbf{x}_n$ 上, 相当于给向量一个近似正交的扰动, 网络可以分别从投影中读出身份与位置. 证明中没有用到 Gauss 性, 只用到“各方向同分布”的旋转对称性, 因此结论对球面上任何对称分布都成立. 注意 $1/D$ 是均方值, 典型绝对值约为 $1/\sqrt{D}$, 这正是高维几何中“几乎所有方向都近似正交”的定量表述.

✍ **练习 12.9(★★)** 考虑一种位置编码, 其中输入 token 向量 $\mathbf{x}$ 与位置编码向量 $\mathbf{e}$ 已合并为一个向量. 证明当这个合并向量在通过矩阵进行一般线性变换时, 结果可以表示为线性变换输入向量和线性变换位置向量的和.

注

(重构新增) 本题说明“相加”与“拼接”在线性层看来是等价的: 拼接后分块相乘再相加, 恰好等于两个独立线性变换之和. 这正是文中说“线性处理层能使合并表示法与加法表示法具有相似的特性”的含义. 差别只在于参数量与维数: 拼接需要 $D + D'$ 维输入, 而相加保持 D 维. 若变换是非线性的 (例如经过 ReLU), 这一分解就不再成立, 这也是为什么位置编码要在第一个线性层之前加入.

✍ **练习 12.10(★★)** 证明(12.25)定义的位置编码具有这样的特性: 对于固定偏移 k, 位置 $n + k$ 的编码可以表示为位置 n 的编码的线性组合, 其系数取决于 k 而非取决于 n. 为此, 使用以下三角恒等式:

$$\cos(A + B) = \cos A \cos B - \sin A \sin B \tag{12.45}$$

$$\sin(A + B) = \cos A \sin B + \sin A \cos B. \tag{12.46}$$

证明如果编码只基于正弦函数而非余弦函数, 这一特性将不再成立.

注

(重构新增) 该性质的意义在于“相对位置可被线性读取”：注意力层本身是线性的，若它能学会用 $\mathbf{M}_k$ 作用于查询或键，就能识别固定的位移 k，而不必为每个绝对位置单独记忆参数. 证明揭示了正弦与余弦必须成对出现的代数原因——它们构成一个二维旋转，位移 k 对应旋转角 $k\omega$，而旋转是线性的. 这也说明位置编码的频率集合 $\{\omega_i\}$ 决定了模型能分辨的位移尺度：频率越高 (i 越小) 分辨短位移，频率越低分辨长位移.

✍ **练习 12.11(★)** 考虑词袋模型 [(12.28)]，其中每个组件分布 $p(x_n)$ 都是由所有词共享的通用概率表给出的. 证明在给定向量训练集的情况下，最大似然解是由一个表格给出的，该表格中的条目是每个词在训练集中出现的次数.

注

(重构新增) 记忆要点是“词袋的充分统计量就是计数”：所有位置共享同一张表，因此每个词的计数把所有出现位置的信息压缩成一个数. 这也暴露了零频问题——若某词未出现，$n_k = 0$，最大似然给 $p(k) = 0$，任何含该词的新句子概率为零. 习题中的结论与命题 12.7一致，只是这里强调“表格条目即计数”这一实现形式.

✍ **练习 12.12(★)** 考虑(12.31)给出的自回归模型，并假设(12.31)右侧的项 $p\left(x_n \mid x_1, \cdots, x_{n-1}\right)$ 可用一般概率表来表示. 证明这些表格中的条目数会随着 n 值的增大呈指数增长.

注

(重构新增) 这正是自回归模型必须参数化的原因：不作假设时，等价于为每个可能的历史存一张表，参数量随长度爆炸. 词袋模型把参数降到 $K-1$，n 元模型降到与 N 无关的 $K^L(K-1)$，而 Transformer 用一个固定大小的网络在每种历史上共享参数——三者是同一分解在不同假设下的极端. 题目要求“证明条目数指数增长”，本质上是说明“查表法不可行”的定量版本.

✍ **练习 12.13(★)** 在使用 n 元模型时，通常会同时训练 n 元模型和 $(n-1)$ 元模型，然后使用概率的乘积法则计算条件概率，其形式为

$$p\left(x_n \mid x_{n-L+1}, \cdots, x_{n-1}\right)=\frac{p_L\left(x_{n-L+1}, \cdots, x_n\right)}{p_{L-1}\left(x_{n-L+1}, \cdots, x_{n-1}\right)} \tag{12.47}$$

解释为什么这样做更方便，并证明在求 $p_{L-1}(\cdot)$ 时，必须省略每个序列的最后一个 token 才能得到正确的概率.

注

(重构新增) 本题的实质是“分子分母必须基于同一组窗口计数”. 若不省略末尾 token，分母多计了没有后继的窗口，导致 $\sum_{x_n} p(x_n \mid \cdot) \neq 1$，概率模型不再自洽. 这一细节在实现中表现为：统计 $(L-1)$ 元组频次时只扫描那些后面至少还有一个 token 的位置. 回退语言模型 (如 Kneser-Ney) 正是在这一计数结构上做插值.

✍ **练习 12.14(★★)** 写出在具有图 12.13所示架构的经训练递归神经网络中进行推理的伪代码.

注

(重构新增) 伪代码体现了递归网络推理的两个结构性特征：一是串行，第 n 步必须等待第 $n-1$ 步的状态，无法在时间维并行；二是状态复用，编码器只需把 $\mathbf{z}^*$ 交给解码器，整个输入句子的信息都被压缩进这一个固定长度的向量——这正是小节 12.2.6中瓶颈问题的来源. 与 Transformer 的写法对比，自注意力的推理可以一次并行算出全部位置，代价是 $O(N^2)$ 的内存.

✍ **练习 12.15(★★)** 考虑一个由两个 token y_1 和 y_2 组成的序列，其中的每个 token 都可以有 A 和 B 两种状态. 联合概率分布 $p(y_1, y_2)$ 如下表所示.

	$y_1 = A$	$y_1 = B$
$y_2 = A$	0.0	0.4
$y_2 = B$	0.1	0.25

可以看到，最可能序列是 $y_1 = B, y_2 = A$，概率为 0.4. 利用概率的加法和乘法法则，写出边缘分布 $p(y_1)$ 和条件分布 $p(y_2 \mid y_1)$ 的值. 证明如果我们先最大化 $p(y_1)$ 而得出一个值 y_1^*，再最大化 $p(y_2 \mid y_1^*)$，那么我们得到的序列将与总体最可能序列不同. 求该序列的概率.

注

(重构新增) 本题是小节 12.3.2中“逐阶段取最大不等于最大化联合概率”的最小反例: 贪心在第一步就锁定了 y_1, 而联合概率的最大值可能位于 $p(y_1)$ 较小的分支上. 原因是 $p(y_1, y_2) = p(y_1)p(y_2 \mid y_1)$ 中两个因子此消彼长——$y_1 = A$ 的边缘概率大, 但其条件下的 y_2 分布较平;$y_1 = B$ 边缘概率小, 却几乎必然跟着 A. 这也说明为什么需要束搜索: 保留 B 个候选可以在后续步骤纠正第一步的“近视”. 原表不归一化属于印刷错误, 按 v14 规范已在解答中指出并给出可验证的修正版本.

练习 12.16(⋆) BERT-Large 模型 (Devlin et al., 2018) 的最大输入长度为 512 个 token, 每个 token 的维度 $D = 1024$, 取自 30 000 个词汇. 该模型有 24 个 Transformer 层, 每个 Transformer 层有 16 个自注意力头, 且 $D_q = D_k = D_v = 64$; 多层感知机定位网络有两层, 共有 4096 个隐节点. 证明 BERT 编码器型 Transformer 语言模型中的参数总数约为 3.4 亿.

注

(重构新增) 估算的关键是“参数量与序列长度无关”: 所有投影矩阵都在 token 之间共享,$N = 512$ 只影响计算量 $O(ND^2)$ 与注意力矩阵 $O(N^2D)$, 不影响参数个数. 这也解释了为什么大模型可以处理任意长度输入而参数不变. 注意 $D_q = D_k = D_v = 64$ 与 $H = 16$ 给出 $HD_v = 1024 = D$, 因此每个头的拼接恰好回到模型维度, 总参数与单头情形同量级——这是定义 12.4中“$D_v = D/H$”惯例的由来.

第 13 章 图神经网络

在前几章中, 我们学习了以序列和图像形式出现的结构化数据, 分别对应于用一维数组和二维数组存储的变量. 在更广泛的应用中, 有许多类型的结构化数据更适合通过图来描述, 如图 13.1所示. 通常, 一个图由一组称为节点的对象组成, 它们通过边来连接. 节点和边都可以与数据相关联. 例如, 在一个分子中, 节点和边分别对应于不同原子类型 (碳原子、氮原子、氢原子等) 和不同化学键类型 (单键、双键等) 的离散变量相关联. 对于铁路网络, 每条铁路线可能与两个城市间平均旅行时间的连续变量相关联. 这里我们假设边是对称的, 例如从伦敦到剑桥的旅行时间与从剑桥到伦敦的旅行时间相同. 这样的边可以表示为节点之间的无向连接. 对于万维网而言, 边是有向的, 因为页面 A 上有指向页面 B 的超链接, 并不一定意味着页面 B 上有指向页面 A 的超链接.

其他图结构化数据的例子有: 蛋白质相互作用网络, 其中节点是蛋白质, 边表示蛋白质对之间相互作用的强度; 电路图, 其中节点是组件, 边是导线; 社交网络, 其中节点是人, 边是“友谊”. 此外, 还有可能存在更复杂的图结构, 例如公司内部的知识图谱包含多种不同类型的节点 (如人员、文档和会议), 以及多种类型的边用于描述不同的属性 (如一个人出席会议或一个文档引用另一个文档).

在本章中, 我们将探索如何将深度学习应用于图结构化数据. 在讨论图像时, 我们已经遇到了结构化数据的一个例子. 在图像中, 数据向量 $\boldsymbol{x}$ 的单个元素对应于规则网格上的像素. 因此, 图像是图结构化数据的特殊实例, 其中节点是像素, 边描述了哪些像素是相邻的. 卷积神经网络 (CNN) 考虑了这种结构 (参见第 10 章), 并纳入了像素之间相对位置的先验知识及一些保持属性的等变性操作 (如分割), 以及保持属性的不变性的操作 (如分类). 我们将以图像的卷积神经网络作为启发, 构建更通用的称为图神经网络的深度学习方法 (Zhou et al., 2018; Wu et al., 2019; Hamilton, 2020; Veličković, 2023). 我们将看到, 把深度学习应用于图结构化数据时的一个关键思路是确保对图中节点重新排序的等变性或不变性.

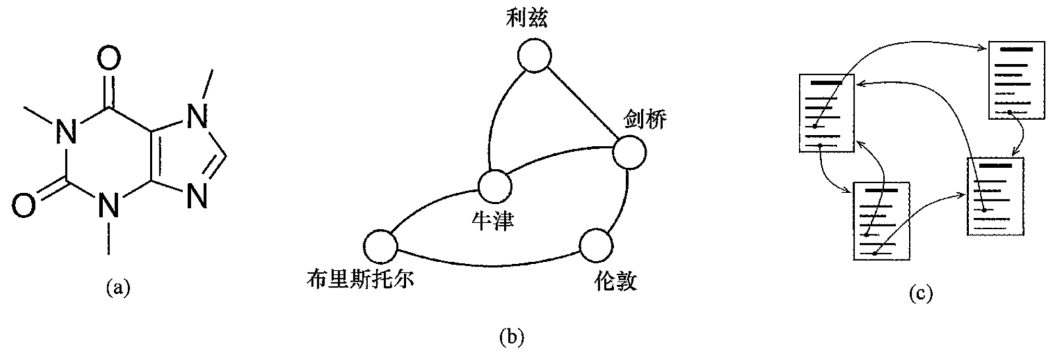


图 13.1: 图结构化数据的 3 个例子:(a) 咖啡因分子由化学键连接的原子组成;(b) 由铁路线连接的城市构成的铁路网络;(c) 由超链接连接的页面构成的万维网

13.1 基于图的机器学习

我们可以使用图结构化数据来完成许多类型的应用, 这些应用可以根据目标是预测节点、边还是整个图的属性来大致分组. 节点预测 (node prediction) 的一个例子是根据文档之间的超链接和引用关系对文档进行主题分类.

对于边来说, 举个例子, 我们可能已知蛋白质网络中的一些相互作用, 想要预测其他潜在的相互作用. 这类任务称为边预测 (edge prediction) 或图补全 (graph completion) 任务. 还有一些任务, 其中边是已知的, 我们的目标是在图中发现簇或“社群”.

最后一类是预测与整个图相关的属性. 例如, 我们可能希望预测特定分子是否能溶解于水. 这里, 我们得到的不是单个图, 而是不同图的数据集, 我们可以把它们看成从某个共同分布抽取而来, 换句话说, 我们假设图本身是独立同分布的 (independent and identically distributed). 这类任务可以视为图回归 (graph regression) 或图分类 (graph classification) 任务.

对于分子溶解性分类的例子, 我们可能会得到一个带标签的分子训练集, 以及需要预测溶解性的新分子测试集. 这是我们在前面章节中多次见到的典型归纳 (inductive) 任务的一个标准示例. 然而, 另一些图预测任务是直推式 (transductive) 的, 我们得到整个图的结构以及一些节点的标签, 要去预测剩余节点的标签. 假设我们要区分一个大型社交网络中的每个用户节点是真实人物还是自动化机器人. 在一个庞大且不断变化的社交网络中逐一核实并标注每个节点是不可行的, 我们只能手工标注少数节点. 因此, 在训练期间, 我们可以访问包含一部分节点标签的整个图, 并试图预测剩余节点的标签. 这是半监督学习的一种形式.

除了直接完成预测任务外, 我们还可以在图上使用深度学习来发现有用的内部表示, 这些表示随后可以促进一系列下游任务的完成. 这称为图表示学习 (graph representation learning). 例如, 我们可以通过在大量分子结构语料库上训练深度学习系统来寻求构建分子的基础模型 (foundation model). 这样的模型一旦训练成功, 我们就可以通过使用小型标注数据集对它进行微调以完成各种特定任务.

图神经网络为每个节点定义了一个嵌入向量 (embedding vector), 该嵌入向量通常使用观测到的节点属性进行初始化, 然后通过一系列可学习层进行转换, 以创建学习表示. 这与词嵌入或 token 在 Transformer 中的处理方式类似, 通过一系列层的处理 (参见第 12 章), 便可以更好地捕获单词在文本其余部分上下文中的语义. 图神经网络还可以学习表示图中每条边的特征向量 (边嵌入) 以及整个图的全局特征向量 (图嵌入).

13.1.1 图的属性

在本章中, 我们将关注简单图, 其中任意两个节点之间最多有一条边, 边是无向的, 并且没有连接到自身节点的环. 这足以让我们介绍图神经网络的关键概念, 同时也涵盖了图神经网络广泛的实际应用. 这些概念之后可以应用于更复杂的图结构.

我们首先引入与图相关的一些符号, 并定义一些重要的属性.

定义 13.1 (图、邻居与度)

一个图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 由一组节点或顶点构成, 用 $\mathcal{V}$ 表示, 其中还包含一组边或连接, 用 $\mathcal{E}$ 表示. 我们用 $n = 1, \cdots, N$ 来索引节点, 并将从节点 n 到节点 m 的边写为 (n, m). 如果两个节点被一条边连接, 则称它们为邻居节点, 所有节点 n 的邻居节点的集合用 $\mathcal{N}(n)$ 表示. 节点 n 的邻居节点的个数称为节点 n 的度 (degree), 记作 $d_n \triangleq |\mathcal{N}(n)|$.

♣

注

(重构新增) 把图写成二元组 $(\mathcal{V}, \mathcal{E})$ 的意义在于把两类信息分开: $\mathcal{V}$ 只说“有哪些对象”, $\mathcal{E}$ 只说“谁与谁相连”, 而后者正是图神经网络唯一能够利用的结构信息. 这也解释了为什么后面的每一层都必须以邻接矩阵 (或等价地, 邻居集合 $\mathcal{N}(n)$) 为输入之一: 若只给出节点变量 $\boldsymbol{X}$, 图就退化成一个无结构的集合, 此时任何层都无法区分两个在 $\boldsymbol{X}$ 上相同、但在图中位置不同的节点.

本章的“简单图”假设包含三条子条件, 它们各有各的作用. 无向性给出 $A_{nm} = A_{mn}$, 这是练习 13.2中“$\boldsymbol{A}^2$ 的对角元等于度”以及后面所有对称归一化写法的前提; 无重边性使 $A_{nm} \in \{0, 1\}$ 而不必取整数权, 从而 $\sum_{m \in \mathcal{N}(n)}$ 与 $\sum_m A_{nm}$ 完全一致; 无自环性给出 $A_{nn} = 0$, 于是 $\mathcal{N}(n)$ 不含 n 本身——这一点在(13.17)中会表现为“是否把节点自身并入求和”的显式区别. 三条条件都不是数学上的必然要求, 而是为了让记号在最简单的情形下保持干净; 去掉其中任何一条, 定义本身仍然成立, 只是相应的公式需要改写.

度 d_n 是本章反复使用的量: 它在(13.16)之后的初始化讨论中作为“节点没有观测数据”时的替代特征 (度数独热向量), 在(13.14)中作为对称归一化的分母, 并在过度平滑的分析中刻画一个节点在一次消息传递中平均了多少条边. 度只依赖 $\mathcal{E}$, 与 $\boldsymbol{X}$ 无关, 因此它是纯粹的结构量.

除了图结构, 通常还有与节点相关联的观测数据.

定义 13.2 (节点变量与数据矩阵) 对于每个节点 n, 我们可以将相应的节点变量表示为一个 D 维的列向量 $\boldsymbol{x}_n$, 将这些变量组合成一个 $N \times D$ 维的数据矩阵 $\boldsymbol{X}$, 其中第 n 行由 $\boldsymbol{x}_n^{\mathsf{T}}$ 表示. ♣

注

(重构新增) 这里把 N 个节点变量堆叠成 $\boldsymbol{X} \in \mathbb{R}^{N\times D}$ 而不是保留为列表, 是为了让“按节点作用”的变换可以写成矩阵乘法. 行索引与 $\boldsymbol{A}$ 的行索引使用同一套编号, 这一点是后面的矩阵形式(13.44)能够成立的前提: 只有当 $\boldsymbol{X}$ 的第 n 行、$\boldsymbol{A}$ 的第 n 行、第 n 列都指向同一个节点时,$\boldsymbol{AH}$ 的第 n 行才会恰好是节点 n 的邻居聚合量.

特征维数 D 与节点数 N 是两个独立的量:D 由数据决定,N 由具体的图决定. 正因为网络的参数只作用在 D 维向量上而不依赖 N, 同一个图神经网络才能处理节点数不同的图, 这正是节 13.2开头所说的“变长输入”能力.

需要强调的是,$\boldsymbol{X}$ 的行顺序完全由我们给节点指定的编号决定, 而这个编号是任意的.(13.6)与(13.7)要刻画的就是“网络的输出不应依赖这个任意选择”.

图中的边也可能与数据变量相关联, 但在当下我们只关注节点变量 (参见小节 13.3.2).

13.1.2 邻接矩阵

指定图中边的一种便捷方式是使用邻接矩阵.

定义 13.3 (邻接矩阵) 邻接矩阵 (adjacency matrix) 用 $\boldsymbol{A}$ 表示. 要定义邻接矩阵, 我们首先需要为节点分配一个顺序. 如果图中有 N 个节点, 我们可以使用 $n = 1, \cdots, N$ 来索引它们. 邻接矩阵的维度是 $N \times N$, 在其中的每个元素位置 n, m, 只要从节点 n 到节点 m 的边存在, 就在该处填入 1, 所有其他元素位置都填入 0. 对于具有无向边的图, 邻接矩阵是对称的, 因为存在从节点 n 到节点 m 的边意味着也存在从节点 m 到节点 n 的边, 对于所有的 n 和 m, 都有 $A_{nm} = A_{mn}$. ♣

注

(重构新增) 邻接矩阵把 $\mathcal{E}$ 编码成一个 $N \times N$ 的数值数组, 代价是引入了对节点编号的依赖: 同一个图换一种编号方式, 得到的矩阵并不相同, 而是相差一个相似变换 (见(13.5)). 这个代价是必须付的, 因为神经网络只能接收数值数组; 而它带来的问题正是下一小节“置换等变性”要解决的.

定义中的三条约定都可以在矩阵上读出:$A_{nm} = A_{mn}$ 对应无向,$A_{nn} = 0$ 对应无自环,$A_{nm} \in \{0, 1\}$ 对应无重边. 若是一般图, 则 A_{nm} 可以取边权,A_{nn} 可以非零,$A_{nm} \neq A_{mn}$ 表示有向边——这些推广都不改变后面的公式, 只改变 $\boldsymbol{A}$ 的性质.

按定义直接计算,$\boldsymbol{A}^2$ 的 (n, m) 元等于从 n 到 m 的长度为 2 的路径条数; 特别地, 对角元 $(\boldsymbol{A}^2)_{nn}$ 恰是节点 n 的度 (见练习 13.2). 这说明度并非额外引入的量, 它已经隐含在 $\boldsymbol{A}$ 中.

邻接矩阵的一个例子如图 13.2所示.

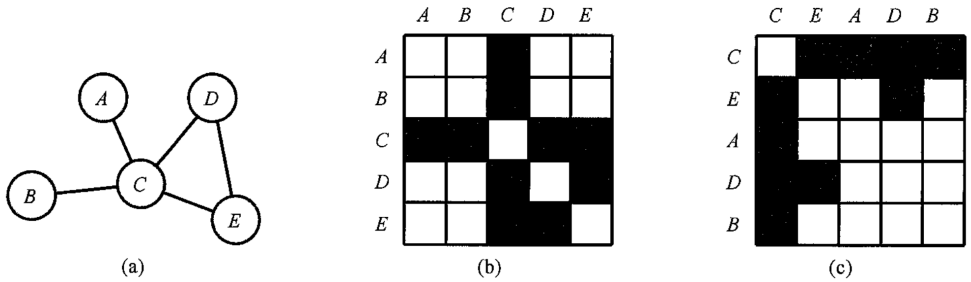


图 13.2: 邻接矩阵的一个例子:(a) 一个包含 5 个节点的图;(b) 对应于特定节点顺序的邻接矩阵;(c) 对应于另一不同节点顺序的邻接矩阵

邻接矩阵定义了图的结构, 我们可以考虑直接将其作为神经网络的输入. 为此, 我们可以“展平”邻接矩阵, 例如将列连接成一个长的列向量. 然而, 这种方法的一个主要问题在于邻接矩阵取决于节点排序的任意选择, 如图 13.2所示. 假设我们想要预测一个分子的溶解性, 这显然不应该取决于写下邻接矩阵时给节点分配的顺序. 由于节点的排序方式的数量随节点数的增加而阶乘式增长, 尝试通过使用大数据集或数据增强来学习置换不变性是不切实际的. 相反, 在构建网络架构时, 我们应当将这种不变性视为归纳偏置.

13.1.3 置换等变性

我们可以引入置换矩阵 $\boldsymbol{P}$ 的概念, 以便在数学上表达节点标签的排列.

定义 13.4 (置换矩阵)

置换矩阵与邻接矩阵大小相同, 并指定了节点排序的特定排列. 置换矩阵在每行和每列中各含有一个 1, 其余元素都是 0, 于是置换矩阵中元素位置 n,m 的 1 表示节点 n 在重新排列后将标记为 m. 例如, 考虑从 (A,B,C,D,E) 到 (C,E,A,D,B) 的排列, 它对应于图 13.2中的两种节点排序选择. 相应的置换矩阵形式为 (见练习 13.1)

$$\boldsymbol{P}=\begin{pmatrix}0&0&1&0&0\\0&0&0&0&1\\1&0&0&0&0\\0&0&0&1&0\\0&1&0&0&0\end{pmatrix}. \tag{13.1}$$

我们可以更加正式地定义置换矩阵如下. 首先引入标准单位向量 $\boldsymbol{u}_n$, 其中 $n=1,\cdots,N$. 这是一个列向量, 其中除了第 n 个元素等于 1 之外, 其他所有元素都等于 0. 在这种表示法中, 单位矩阵如下所示:

$$\boldsymbol{I}=\begin{pmatrix}\boldsymbol{u}_1^{\mathsf{T}}\\\boldsymbol{u}_2^{\mathsf{T}}\\\vdots\\\boldsymbol{u}_N^{\mathsf{T}}\end{pmatrix}. \tag{13.2}$$

引入一个置换函数 $\pi(\cdot)$, 将 n 映射到 $m=\pi(n)$. 对应的置换矩阵如下所示:

$$\boldsymbol{P}=\begin{pmatrix}\boldsymbol{u}_{\pi(1)}^{\mathsf{T}}\\\boldsymbol{u}_{\pi(2)}^{\mathsf{T}}\\\vdots\\\boldsymbol{u}_{\pi(N)}^{\mathsf{T}}\end{pmatrix}. \tag{13.3}$$

♣

注

(重构新增) 按(13.3)定义的 $\boldsymbol{P}$ 的行是标准基向量, 因此 $\boldsymbol{P}$ 是正交矩阵:$\boldsymbol{P}^{\mathsf{T}}\boldsymbol{P}=\boldsymbol{P}\boldsymbol{P}^{\mathsf{T}}=\boldsymbol{I}$. 这一点在后面反复使用——(13.5)中的 $\boldsymbol{P}^{\mathsf{T}}$ 是 $\boldsymbol{P}$ 的逆,(13.19)中的 $\boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}$ 保持 $\boldsymbol{A}$ 的对称性与特征值,定理 13.1的证明中对网络整体的归纳也依赖 $\boldsymbol{P}^{\mathsf{T}}\boldsymbol{P}=\boldsymbol{I}$.

置换函数 π 的读法是“节点 n 在重新编号之后被叫作 $\pi(n)$”. 由于 $\boldsymbol{P}$ 的第 i 行是 $\boldsymbol{u}_{\pi(i)}^{\mathsf{T}}$, 矩阵 $\boldsymbol{P}\boldsymbol{X}$ 的第 i 行就等于 $\boldsymbol{X}$ 的第 $\pi(i)$ 行 (见练习 13.4): 新矩阵的第 i 行记录的是“新编号为 i 的那个节点”的旧数据.π 必须是 $\{1,\cdots,N\}$ 到自身的双射, 因此全体置换矩阵构成一个群 (对称群 S_N 的矩阵表示), “对任意 $\boldsymbol{P}$ 成立”这句话才有确切含义.

以(13.1)为例,$\pi(1)=3,\pi(2)=5,\pi(3)=1,\pi(4)=4,\pi(5)=2$, 即编号 1,2,3,4,5(对应 A,B,C,D,E) 在新编号下依次成为 3,5,1,4,2, 于是新序列按 1 到 5 读出的节点是 C,E,A,D,B, 与题设一致. 把 $\boldsymbol{P}$ 作用在图 13.2(b) 的邻接矩阵上, 就得到同一图的 (c).

当我们重新排序一个图的节点上的标签时, 对应的节点数据矩阵 $\boldsymbol{X}$ 的效果是根据 $\pi(\cdot)$ 进行置换操作, 这可以

通过在其前面乘上 $\boldsymbol{P}$ 来实现 (见练习 13.4), 即

$$\widetilde{\boldsymbol{X}} = \boldsymbol{P}\boldsymbol{X}. \tag{13.4}$$

对于邻接矩阵, 行和列都会被置换. 同样, 行可以通过在前面乘上 $\boldsymbol{P}$ 来置换, 而列可以通过在后面乘上 $\boldsymbol{P}^{\mathsf{T}}$ 来置换 (见练习 13.5), 从而得到一个新的邻接矩阵:

$$\widetilde{\boldsymbol{A}} = \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}. \tag{13.5}$$

当我们将深度学习应用于图结构化数据时, 需要以数值形式表示图结构, 这样才能将它们输入神经网络, 这就要求我们为节点指定顺序. 然而, 我们选择的具体节点排序是任意的, 所以必须确保图的任何全局属性不依赖于这种排序方式.

定义 13.5 (置换不变性与置换等变性)

设 $y(\cdot,\cdot)$ 是网络的输出. 网络预测必须对节点标签的重新排序具有不变性, 即

$$y(\widetilde{\boldsymbol{X}}, \widetilde{\boldsymbol{A}}) = y(\boldsymbol{X}, \boldsymbol{A}) \qquad \text{不变性} \tag{13.6}$$

其中 $y(\cdot,\cdot)$ 是网络的输出. 有些情况下, 我们要对单个节点进行预测. 在这种情况下, 如果我们重新排序节点标签, 则相应的预测应显示相同的次序, 这样不论如何选择顺序, 给定的预测总是与同一个节点相关联. 换言之, 节点级预测应该对节点标签重新排序保持等变性, 即

$$\boldsymbol{y}(\widetilde{\boldsymbol{X}}, \widetilde{\boldsymbol{A}}) = \boldsymbol{P}\,\boldsymbol{y}(\boldsymbol{X}, \boldsymbol{A}) \qquad \text{等变性} \tag{13.7}$$

其中 $\boldsymbol{y}(\cdot,\cdot)$ 是网络输出的一个向量, 每个节点对应其中的一个元素. ♣

注

(重构新增) 不变性与等变性是两个强度不同的要求, 适用对象也不同. 当网络的输出是一个与整张图关联的量 (例如分子是否溶于水) 时, 输出中没有“节点”这一维度, 重排节点不应改变任何东西, 这就是(13.6); 当输出是逐节点的 (例如每个用户是否为机器人) 时, 输出本身是一个 N 维向量, 重排节点会使输出向量的分量跟着换位, 但“第 n 个节点的预测”必须还是同一个数, 这就是(13.7). 若把二者混淆——例如要求节点级输出不变——就会得到荒谬的结论: 重排之后第 1 个分量的含义已经变成了另一个节点.

两式中的 $\boldsymbol{P}$ 必须是同一个任意置换矩阵, 且必须同时作用在 $\boldsymbol{X}$ 与 $\boldsymbol{A}$ 上. 只置换 $\boldsymbol{X}$ 而不置换 $\boldsymbol{A}$(或反之) 描述的是“换了一种节点特征顺序但仍沿用旧图”, 那不是同一个图的重排, 因而不能作为对称性要求.

两个要求之间有确定的联系: 若 $\boldsymbol{y}$ 是等变的, 则任何对分量求和 (或取平均、取最大) 的读出函数都给出不变的输出, 因为 $\sum_n (\boldsymbol{P}\boldsymbol{y})_n = \sum_n y_n$. 这正是小节 13.2.7中用求和构造图级预测的依据, 也说明“等变”是比“不变”更基本的要求.

最后要注意,(13.6)与(13.7)是网络应当满足的性质, 而不是可以自动成立的事实: 把 $\boldsymbol{A}$ 展平后送入多层感知机所得到的函数一般不满足它们, 因此必须把对称性写进架构, 这正是节 13.2的出发点.

13.2 神经信息传递

确保节点标签置换下的不变性或等变性是我们将深度神经网络应用于图结构化数据时的一个关键考虑. 另一个关键考量是我们想要利用深度神经网络的表示能力, 所以我们保留了“层”的概念, 即可以对每一层进行重复的计算变换. 如果网络的每一层都在节点重新排序后保持等变性, 那么多层叠加后的网络也将保持等变性, 同时网络的每一层都受益于图结构信息.

对于输出代表节点级预测的网络, 整个网络将按要求表现出等变性. 如果网络用来预测图级属性, 那么可以在网络的最后添加一个对输入的排列顺序不敏感 (置换不变) 的层. 我们还想确保网络的每一层都是一个高度灵活的非线性函数, 并且对其参数具有可微性, 以便可以使用自动微分计算梯度, 从而通过随机梯度下降进行训练.

图的规模各不相同. 例如, 不同的分子可以有不同数量的原子, 因此标准神经网络使用固定长度表示是不合适的. 因此, 网络应该能够处理变长输入, 正如我们在 Transformer 网络中看到的那样 (参见第 12 章). 一些图可能非

常大, 例如有数百万参与者的社交网络, 所以我们也希望构建具有良好扩展性的模型. 为了将不变性和等变性内置于网络架构, 并促进模型在大型图上的应用, 参数共享将发挥重要作用, 这一点毋庸置疑.

13.2.1 卷积滤波器

为了开发符合所有这些要求的框架, 我们可以从使用卷积神经网络的图像处理中寻找灵感 (参见第 10 章). 首先我们注意到, 图像可以视为图结构化数据的一个特定实例, 在这个实例中, 节点是像素, 边代表图像中相邻的像素对, 这里的相邻包括对角相邻的节点以及水平或垂直相邻的节点.

在卷积网络中, 我们对图像域进行逐层变换, 某个层的一个像素的状态值是由前面层的相关像素的状态值通过计算决定的, 用于计算的局部函数称为滤波器 (参见节 10.2).

定义 13.6 (卷积滤波器)

考虑一个使用 3×3 滤波器的卷积层, 如图 13.3(a) 所示. 单个的滤波器在第 $(l+1)$ 层的一个像素上执行的计算可以表示为

$$z_i^{(l+1)} = f\left(\sum_j w_j\, z_j^{(l)} + b\right), \tag{13.8}$$

其中 $f(\cdot)$ 是一个可微分的非线性激活函数, 比如 ReLU 函数, 而对 j 的求和则是对第 l 层中一个小块内的所有 9 个像素进行的. 相同的函数可以应用于图像中的多个小块, 因此权重 w_j 和偏置 b 在各个小块间共享 (因此它们不带索引 i).

♣

注

(重构新增) (13.8)中有两点是后面推广的起点. 其一是求和指标 j 的取值范围: 它不是全部像素, 而只是与像素 i 相邻的那 9 个位置, 这个范围由图像网格的邻接关系决定, 换成一般的图就变成 $\mathcal{N}(i)$. 其二是参数不带位置索引:w_j 对所有小块相同, 因而全部像素共享同一组权重. 正是这一共享使参数量与图像大小无关, 也使(13.8)的卷积层具有平移等变性.

需要区分 w_j 的下标 j 与节点的下标:j 枚举的是一个 3×3 小块内部的相对位置, 它共有 9 个取值, 与图像中有多少个像素无关; 而 i 枚举的是像素. 正因为这两类索引的性质不同,(13.8)才可能在节点重排下失效——重排节点会打乱“相对位置”与“权重分量”的对应, 而权重向量本身并没有随之重排. 这一观察直接引出下面的图形式改写.

(13.8)在第 l 层节点置换下不是等变的, 因为权重向量中的 w_j 元素在排列上不是不变的. 然而, 我们可以通过进行一些简单的修改来实现等变性, 具体步骤如下. 首先将滤波器视为一个图, 如图 13.3(b) 所示, 并且从中分离出来自节点 i 的贡献. 其他 8 个节点构成节点 i 的邻居节点集合 $\mathcal{N}(i)$.

定义 13.7 (图形式的卷积滤波器)

假设单个权重参数 w_{neigh} 在邻居节点间共享, 则图形式的卷积滤波器定义为

$$z_i^{(l+1)} = f\left(w_{\text{neigh}} \sum_{j\in\mathcal{N}(i)} z_j^{(l)} + w_{\text{self}}\, z_i^{(l)} + b\right), \tag{13.9}$$

其中节点 i 有自己的权重参数 w_{self}.

♣

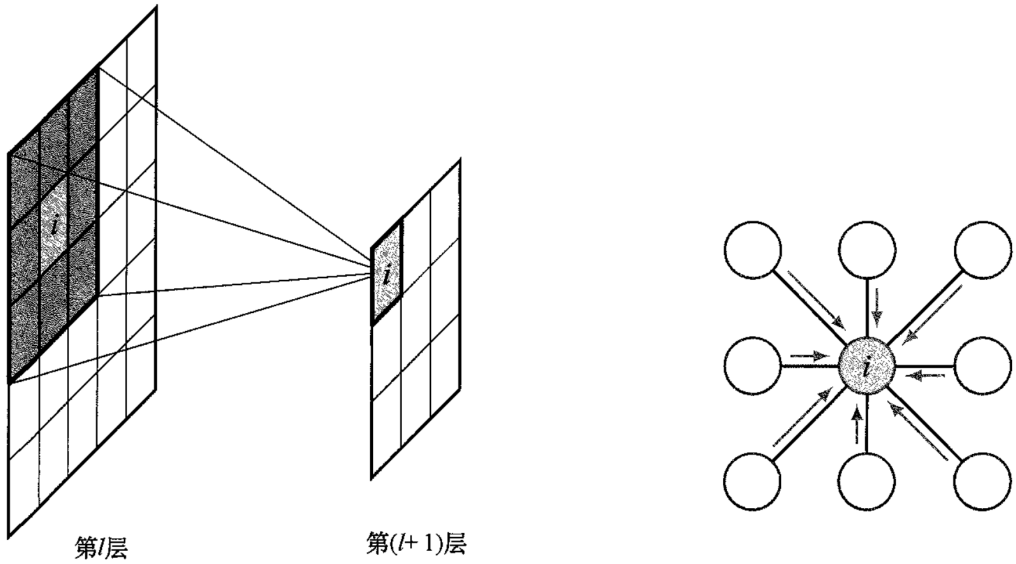


图 13.3: 图像的卷积滤波器可以表示为一种图结构计算:(a) 在深度卷积网络的第 $(l+1)$ 层, 节点 i 的值是由第 l 层局部像素区域激活值通过滤波器函数计算得来的;(b) 将同样的计算结构以图的形式表达, “消息” 从节点 i 的邻居节点流向节点 i

注

(重构新增) (13.9)相对(13.8)只做了两处改动, 但这两处改动的地位并不相同. 第一处是把 “3×3 小块内按相对位置编号的 9 个权重 w_j” 换成 “邻居共用的一个标量 w_{neigh}” : 这消除了权重分量与邻居编号之间的对应关系, 是获得等变性的关键. 第二处是把节点自身从求和中分离出来, 单独乘以 w_{self}: 这一步与等变性无关, 它只是补回了 “中心像素与邻居地位不同” 这一信息, 否则中心节点与邻居将无法区分.

代价也必须说清楚: 单个 w_{neigh} 的表达能力弱于 9 个独立的 w_j, 因为卷积层原本可以利用 “上、下、左、右” 等方向差异, 而图形式的滤波器把所有邻居一视同仁. 这正是为什么后续要引入对邻居嵌入先做变换的聚合算子((13.15)): 用共享的 MLP_ϕ 恢复表达力, 同时不破坏对称性.

系数 w_{neigh}、w_{self} 与偏置 b 必须在全部节点间共享; 若允许每个节点有自己的参数, 等变性立刻失效, 因为重排节点会改变 “哪个参数属于哪个节点” .

命题 13.1 (图形式卷积滤波器的置换等变性)

设 $\boldsymbol{z}^{(l)}=(z_1^{(l)},\cdots,z_N^{(l)})^{\mathsf{T}}$ 为第 l 层的激活向量,$\boldsymbol{P}$ 为任意 $N\times N$ 置换矩阵,$\widetilde{\boldsymbol{z}}^{(l)}=\boldsymbol{P}\boldsymbol{z}^{(l)}$, 并记 $\widetilde{\mathcal{N}}(i)$ 为按 $\boldsymbol{P}$ 重排后的邻居集合. 则按(13.9)逐节点计算得到的第 $(l+1)$ 层激活满足

$$\widetilde{\boldsymbol{z}}^{(l+1)}=\boldsymbol{P}\boldsymbol{z}^{(l+1)}.$$

♠

证明

我们可以将(13.9)解释为通过从邻居节点收集信息来更新节点 i 处的局部表示 z_i, 这通过从邻居节点传递给节点 i 的消息来实现. 在这种情况下, 消息直接是其他节点的激活状态. 随后将这些消息与节点 i 的信息结合, 并使用非线性函数进行转换. 来自邻居节点的信息通过(13.9)中的简单求和聚合起来, 这显然对于与这些节点相关联的标签的任何排列都是不变的. 将以上操作同步应用于图中的每个节点, 因此如果节点被排序, 则最终的计算将保持不变, 但它们也将被相应地置换, 因此这个计算在节点重新排序下是等变的. 把上述论证写成指标形式: 重排之后, 编号为 i 的节点实际上就是原来的节点 $\pi^{-1}(i)$, 于是它的邻居集合为 $\widetilde{\mathcal{N}}(i)=\pi\big(\mathcal{N}(\pi^{-1}(i))\big)$, 而 $\widetilde{z}_j^{(l)}=z_{\pi^{-1}(j)}^{(l)}$. 代入(13.9)得

$$\widetilde{z}_i^{(l+1)}=f\left(w_{\text{neigh}}\sum_{j\in\widetilde{\mathcal{N}}(i)}\widetilde{z}_j^{(l)}+w_{\text{self}}\widetilde{z}_i^{(l)}+b\right)$$

$$= f\left(w_{\text{neigh}} \sum_{j' \in \mathcal{N}(\pi^{-1}(i))} z_{j'}^{(l)} + w_{\text{self}} z_{\pi^{-1}(i)}^{(l)} + b\right) = z_{\pi^{-1}(i)}^{(l+1)}.$$

第二个等号只是把求和变量由 $j \in \widetilde{\mathcal{N}}(i)$ 换成 $j' = \pi^{-1}(j) \in \mathcal{N}(\pi^{-1}(i))$, 这是有限集合上的重指标, 不改变求和值;$w_{\text{neigh}}$、$w_{\text{self}}$、$b$ 与 i 无关, 故可原样保留. 最后一式表明 $\tilde{\boldsymbol{z}}^{(l+1)}$ 的第 i 个分量等于 $\boldsymbol{z}^{(l+1)}$ 的第 $\pi^{-1}(i)$ 个分量, 即 $\tilde{\boldsymbol{z}}^{(l+1)} = \boldsymbol{P}\boldsymbol{z}^{(l+1)}$. 证毕.

□

注

(重构新增) 这条命题把“卷积可以写成图上的消息传递而不破坏对称性”这句话变成了可检验的等式, 它的使用条件是三条: 求和只遍历邻居集合、每个邻居的贡献只通过同一个标量 w_{neigh} 进入、非线性函数 f 逐元素作用. 第三条容易被忽略: 若 f 依赖分量的位置 (例如对向量做固定的线性混合), 等变性就不再成立.

注意命题只断言等变性, 没有断言不变性. z_i 本身会随节点重排而换位, 这是节点级量的正确行为; 若要得到图级的不变量, 还需在最后对全部节点做一次置换不变的聚合, 这正是小节 13.2.7的内容.

最后, 本命题讨论的是标量激活 z_n, 它只是下一节向量值嵌入 $\boldsymbol{h}_n$ 的一维特例; 一般情形见定理 13.1.

13.2.2 图卷积网络

我们以卷积操作为例, 构建用于处理图结构数据的深度神经网络模型. 我们的目标是定义一种灵活的、非线性的节点嵌入转换, 它对一组权重和偏置参数可微分, 并且将第 l 层中的变量映射到第 $(l+1)$ 层的相应变量.

定义 13.8 (节点嵌入)

对于图中的每个节点 n 以及网络中的每一层 l, 引入一个 D 维的列向量 $\boldsymbol{h}_n^{(l)}$ 作为节点嵌入变量, 其中 $n = 1, \cdots, N$ 且 $l = 1, \cdots, L$.

♣

注

(重构新增) 节点嵌入是“把结构信息写进特征”的载体: $\boldsymbol{h}_n^{(l)}$ 的第 l 层版本应当同时编码节点 n 自身的属性与它 l 跳邻域内的结构. 因此 $\boldsymbol{h}_n^{(l)}$ 的维数 D 是一个建模选择, 它不必等于输入特征维数——(13.16)之后的初始化讨论给出了三种把 $\boldsymbol{x}_n$ 变成合适维数的方法.

上标 (l) 不是幂次而是层号, 这一点在(13.18)中尤其要注意: 那里 $\boldsymbol{H}^{(l)}$ 表示第 l 层的整块嵌入矩阵. 层号从 0 开始计 ($\boldsymbol{h}_n^{(0)} = \boldsymbol{x}_n$), 而层的变换共 L 次, 因此最终的表示是 $\boldsymbol{h}_n^{(L)}$.

(13.9)给出的转换首先从邻居节点收集和合并信息, 然后用当前节点嵌入和传入消息的函数更新节点. 因此, 我们可以将每一层的处理视为两个连续阶段. 首先是聚合阶段, 对于每个节点 n, 将消息从其邻居节点传递到该节点, 并以置换不变的方式组合成一个新的向量 $\boldsymbol{z}_n^{(l)}$. 然后是更新阶段, 将邻居节点的聚合信息与节点本身的局部信息结合, 计算出该节点更新后的嵌入向量.

定义 13.9 (消息传递神经网络)

考虑图中的一个特定节点 n. 首先聚合来自节点 n 的所有邻居节点的节点向量:

$$\boldsymbol{z}_n^{(l)} = \text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right). \tag{13.10}$$

如果聚合函数对变化的邻居节点数量有良好的定义, 并且不依赖于这些节点的排序, 那么该聚合函数的形式将非常灵活. 只要它是一个对这些参数可微分的函数, 便于进行梯度下降训练, 它就可以包含可学习参数. 然后使用另一种操作来更新节点 n 处的嵌入向量:

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)}\right). \tag{13.11}$$

同样, 这可以是一组可学习参数的可微分函数. 并行地对图中的每个节点执行聚合操作, 然后执行更新操作. 节点嵌入通常使用观测到的节点数据初始化, 因此 $\boldsymbol{h}_n^{(0)} = \boldsymbol{x}_n$. 注意, 每一层通常都有自己的参数, 尽管参数也可以跨层共享. 这个框架称为消息传递神经网络 (message-passing neural network)(Gilmer et al., 2017), 算

法 13.1 对它做了总结.

♣

注

(重构新增) 这个定义把“一层图神经网络”拆成两个可以分别设计的部件, 其意义在于: 置换等变性只由聚合函数决定, 而表达能力主要由更新函数决定. 具体地说,Aggregate 的输入是一个集合 $\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}$, 集合本身没有顺序, 因此任何以集合为定义域的函数自动是置换不变的; 而 Update 的输入是一个有序对 $(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)})$, 它逐节点作用、参数共享, 因此保持等变性 (定理 13.1).

定义中的三个条件缺一不可. 第一, “对变化的邻居节点数量有良好的定义”排除了把邻居向量拼接成定长向量的做法——那样一旦度数改变, 输入维数就变了. 第二, “不依赖于这些节点的排序”排除了带位置索引的加权求和. 第三, “对参数可微分”保证梯度可以通过 Aggregate 与 Update 反传, 这是整个框架能用随机梯度下降训练的前提. 注意可学习参数是允许的, 被禁止的只是对顺序的依赖.

$\boldsymbol{h}_n^{(0)} = \boldsymbol{x}_n$ 只是最常用的初始化; 当 $\boldsymbol{x}_n$ 的维数与所需的 D 不同, 或节点根本没有观测数据时, 需要另作处理 (见(13.17)之后的讨论).

算法 13.1: 一个简单的消息传递神经网络

输入: 无向图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; 初始节点嵌入 $\{\boldsymbol{h}_n^{(0)} = \boldsymbol{x}_n\}$; 聚合函数 Aggregate$(\cdot)$; 更新函数 Update$(\cdot, \cdot)$

输出: 最终节点嵌入 $\{\boldsymbol{h}_n^{(L)}\}$

// 迭代进行消息传递

for $l \in \{0, \cdots, L-1\}$ **do**

  $\boldsymbol{z}_n^{(l)} \leftarrow \text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right)$

  $\boldsymbol{h}_n^{(l+1)} \leftarrow \text{Update}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)}\right)$

end for

return $\{\boldsymbol{h}_n^{(L)}\}$

13.2.3 聚合算子

聚合函数有许多可能的形式, 但它必须仅依赖于输入的集合, 而不依赖于输入的顺序. 它还必须是任何可学习参数的可微分函数.

定义 13.10 (聚合算子)

根据(13.9), 最简单的聚合函数为求和

$$\text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} \boldsymbol{h}_m^{(l)}. \tag{13.12}$$

简单求和显然独立于邻居节点的排序, 并且无论邻居节点集合中有多少节点, 它都是明确定义的. 注意这里没有可学习参数.

♣

注

(重构新增) 求和之所以是“最简单”的聚合, 是因为它在所有置换不变的算子中形式最简, 并且同时满足三条要求: 对邻居集合的排序不变、对邻居个数不设上限、且与线性变换可交换 ($\sum_m \boldsymbol{W}\boldsymbol{h}_m = \boldsymbol{W}\sum_m \boldsymbol{h}_m$). 最后一条尤其重要——由于 $\sum_m \boldsymbol{h}_m$ 是线性的,(13.14)与(13.15)中“先变换再求和”与“先求和再变换”只差一个矩阵乘法次序, 这使整层可以写成矩阵形式 (见练习 13.6).

但求和也有明确的缺陷: 它的数值随度数增长. 一个度为 10^4 的节点, 其聚合量的量级会比度为 1 的节点大四个数量级, 训练时容易造成梯度尺度失衡.(13.13)与(13.14)都是针对这一点做的修正, 代价各不相同.

定义中“没有可学习参数”不是缺陷而是特性: 它保证聚合本身不会破坏对称性. 可学习参数应当放在逐节点共享的变换里 ((13.15)), 而不是放在对邻居的加权里——后者若权重依赖邻居的编号就会破坏等变性, 若权重依赖邻居的特征则变成注意力 (小节 13.3.1).

相对于那些邻居节点较少的节点, 拥有众多邻居的节点会被求和操作赋予更强的影响力, 这可能导致数值计算问题, 特别是在像社交网络这样的应用中, 邻居节点集合的大小可能会相差好几个数量级. 这种方法的一个变体是将聚合操作定义为邻居嵌入向量的平均值, 即

$$\text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \frac{1}{|\mathcal{N}(n)|} \sum_{m \in \mathcal{N}(n)} \boldsymbol{h}_m^{(l)}, \tag{13.13}$$

其中 $|\mathcal{N}(n)|$ 表示邻居节点集合 $\mathcal{N}(n)$ 中的节点数量. 然而, 这种归一化抛弃了关于网络结构的信息, 并且被证明比简单的求和 (Hamilton, 2020) 要弱, 因此是否使用它取决于节点特征和图结构相比谁更重要.

这种方法的另一个变体 (Kipf and Welling, 2016) 则考虑到了每个节点的邻居的数量:

$$\text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m \in \mathcal{N}(n)} \frac{\boldsymbol{h}_m^{(l)}}{\sqrt{|\mathcal{N}(n)|\,|\mathcal{N}(m)|}}. \tag{13.14}$$

最后一种可能性是取邻居嵌入向量的逐元素最大值 (或最小值), 这同样满足所需特性, 即对数量可变的邻居节点能作出良好定义, 并且和邻居节点顺序无关.

命题 13.2 (图卷积聚合的谱解释 (重构新增))

设 $\boldsymbol{D} \triangleq \operatorname{diag}(d_1, \cdots, d_N)$ 为度矩阵, 其中 $d_n = |\mathcal{N}(n)| > 0$,$\boldsymbol{A}$ 为邻接矩阵,$\boldsymbol{H} \in \mathbb{R}^{N \times D}$ 的第 n 行为 $\boldsymbol{h}_n^{\mathsf{T}}$. 则(13.14)给出的聚合量的转置恰为对称归一化矩阵 $\widehat{\boldsymbol{A}} \triangleq \boldsymbol{D}^{-1/2}\boldsymbol{A}\boldsymbol{D}^{-1/2}$ 所作用的结果, 即整层可以写成

$$\boldsymbol{Z} = \widehat{\boldsymbol{A}}\boldsymbol{H} = \boldsymbol{D}^{-1/2}\boldsymbol{A}\boldsymbol{D}^{-1/2}\boldsymbol{H}.$$

特别地,$\widehat{\boldsymbol{A}} = \boldsymbol{I} - \boldsymbol{L}_{\text{sym}}$, 其中 $\boldsymbol{L}_{\text{sym}} \triangleq \boldsymbol{I} - \boldsymbol{D}^{-1/2}\boldsymbol{A}\boldsymbol{D}^{-1/2}$ 是对称归一化 Laplacian.

证明

(重构新增) 按矩阵乘法展开,$\widehat{\boldsymbol{A}}$ 的 (n, m) 元为

$$\widehat{A}_{nm} = \left(\boldsymbol{D}^{-1/2}\right)_{nn} A_{nm} \left(\boldsymbol{D}^{-1/2}\right)_{mm} = \frac{A_{nm}}{\sqrt{d_n d_m}}.$$

于是 $(\widehat{\boldsymbol{A}}\boldsymbol{H})$ 的第 n 行为

$$\left(\widehat{\boldsymbol{A}}\boldsymbol{H}\right)_{n:} = \sum_{m=1}^{N} \widehat{A}_{nm}\boldsymbol{h}_m^{\mathsf{T}} = \sum_{m \in \mathcal{N}(n)} \frac{\boldsymbol{h}_m^{\mathsf{T}}}{\sqrt{d_n d_m}},$$

其中第二个等号用到 $A_{nm} = 1$ 当且仅当 $m \in \mathcal{N}(n)$. 这正是(13.14)的右端 (取转置), 故 $\boldsymbol{Z} = \widehat{\boldsymbol{A}}\boldsymbol{H}$. 又因 $\boldsymbol{L}_{\text{sym}} = \boldsymbol{I} - \boldsymbol{D}^{-1/2}\boldsymbol{A}\boldsymbol{D}^{-1/2}$ 逐项相减即得 $\widehat{\boldsymbol{A}} = \boldsymbol{I} - \boldsymbol{L}_{\text{sym}}$. 由于 $\boldsymbol{A}$ 对称、$\boldsymbol{D}$ 为对角正定矩阵,$\boldsymbol{L}_{\text{sym}}$ 是对称半正定矩阵, 故存在正交矩阵 $\boldsymbol{U}$ 与对角矩阵 $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \cdots, \lambda_N)$ 使 $\boldsymbol{L}_{\text{sym}} = \boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}}$, 于是

$$\boldsymbol{Z} = \boldsymbol{U}(\boldsymbol{I} - \boldsymbol{\Lambda})\boldsymbol{U}^{\mathsf{T}}\boldsymbol{H},$$

即该聚合层在 $\boldsymbol{L}_{\text{sym}}$ 的特征基下对节点特征逐模态地乘以响应 $1 - \lambda_i$. 证毕.

□

注

(重构新增) 这条命题补出的是原书仅以文献引用带过的 Kipf and Welling (2016) 归一化的矩阵含义:(13.14)的分母 $\sqrt{|\mathcal{N}(n)|\,|\mathcal{N}(m)|}$ 不是随手选的缩放, 它恰好使聚合算子成为一个对称矩阵 $\widehat{\boldsymbol{A}}$ 的乘法. 对称性带来两个直接后果:$\widehat{\boldsymbol{A}}$ 可正交对角化, 从而上一段的谱分解成立;$\widehat{\boldsymbol{A}}$ 的特征值落在 $[-1, 1]$ 内, 反复相乘不会指数爆炸, 这正是深堆叠图卷积比直接使用 $\boldsymbol{A}$ 更稳定的原因.

与(13.13)比较可以看出归一化的两种取舍: 平均按行归一, 只消除度数对量级的影响, 得到的是非对称算子 $\boldsymbol{D}^{-1}\boldsymbol{A}$; 对称归一同时按行与列缩放, 保留了 $\boldsymbol{A}$ 的对称性因而保留了谱结构. 原书说“平均化抛弃了网络结构的信息”, 在矩阵语言里就是 $\boldsymbol{D}^{-1}\boldsymbol{A}$ 与 $\boldsymbol{A}$ 不再相似, 其特征值不再直接反映图的结构.

需要指出, 谱解释只是理解该算子的一种视角: 它把一次聚合看作一次固定的低通滤波, 滤波器形状由 $1 - \lambda_i$ 决定, 其中不含可学习参数. 原书并未展开这一视角, 本命题与证明属于重构新增内容.$\boldsymbol{D}$ 可逆 (即每个节点至少有一个邻居) 是本命题的前提; 孤立节点会使 $d_n = 0$, 此时两种归一化都无定义, 实际实现中通常给孤立节点补一个自环.

图神经网络中给定层的每个节点通过聚合前一层的邻居节点信息来更新, 这就出现了感受野 (receptive field) 的概念, 类似于第 10 章中 CNN 所使用的滤波器的感受野 (见定义 10.2). 随着信息通过连续的层被处理, 对当前层某个节点的更新就会依赖于越来越多的早期层的其他节点, 直到有效的感受野几乎覆盖整个图为止, 如图 13.4所示. 然而, 大的稀疏图可能需要非常多的层, 才能使每个输出受每个输入的影响. 因此, 有些架构引入了一个额外的“超级节点”, 它直接连接到原始图中的每个节点, 使信息得到快速传播.

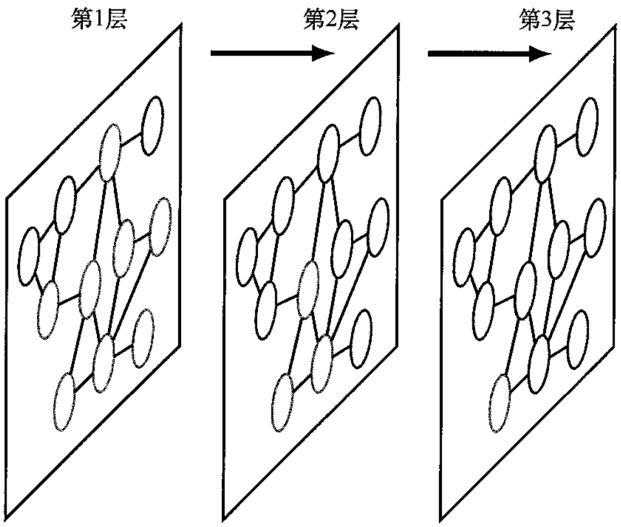


图 13.4: 通过图神经网络连续层的信息流示意图. 在第 3 层中, 只有一个节点以红色突出显示. 该节点从前一层的两个邻居节点接收信息, 而这两个邻居节点又从第 1 层的邻居节点接收信息. 就像用于图像的卷积神经网络一样, 有效感受野——对应于显示为红色的节点数量——会随着处理层数的增加而扩大

迄今为止我们所讨论的聚合算子都没有可学习参数. 如果我们在组合邻居节点的嵌入向量之前, 先使用一个多层神经网络 (用 MLP_ϕ 表示) 对每个嵌入向量进行变换, 那么我们就可以引入这样的参数, 其中 MLP 代表“多层感知机”, 其下标 ϕ 表示网络的参数. 只要网络具有跨节点共享的结构和参数值, 那么这个聚合算子就是排列不变的.

定义 13.11 (深度集合聚合算子)

还可以使用另一个参数为 θ 的神经网络 MLP_θ 来转换组合向量, 从而得到如下一般性的聚合操作:

$$\mathrm{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \mathrm{MLP}_\theta\left(\sum_{m\in\mathcal{N}(n)} \mathrm{MLP}_\phi(\boldsymbol{h}_m^{(l)})\right), \tag{13.15}$$

其中 MLP_ϕ 和 MLP_θ 在网络第 l 层是共享的. 由于 MLP 的灵活性,(13.15)定义的转换代表了任何置换不变函数的通用逼近器, 该函数能将一组嵌入映射到单个嵌入 (Zaheer et al., 2017). 注意, 这里的求和操作可以替换为其他的置换不变函数, 例如平均值, 逐元素取最大值或最小值. ♣

注

(重构新增) (13.15)的结构可以概括为“逐元素变换—求和—再变换”, 它是“先聚合、再更新”框架中最常用的一种聚合算子. 把 MLP_ϕ 放在求和之前的理由是可以逐节点独立计算, 从而可以并行; 把 MLP_θ 放在求和之后的理由是它需要看到整个邻域的信息, 而求和已经把邻域压成一个定长向量. 两个 MLP 在同一层的所有节点、所有位置之间共享参数, 这既是置换等变性的来源, 也是参数量与节点数无关的原因.

定义中“求和可以替换为平均值、逐元素最大值或最小值”这句话有条件: 替换后的函数仍必须是置换不变的. 求和、平均、逐元素最大/最小都满足这一点, 而按邻居编号加权求和、把邻居向量按序拼接都不满足. 另外, 用最大值替换求和会改变算子对邻居个数的敏感性 (最大值只看“最强的那个邻居”), 表达能力与稳定性都随之变化, 这不是无关紧要的实现细节.

定义把“通用逼近”作为一句引用的结论写在正文里. 它比“置换不变”强得多: 置换不变只说明函数不区分顺序, 通用逼近说明任何连续置换不变函数都能被这一族算子逼近. 完整的证明属于文献而不是本章, 下面的命题 13.3只证明其中可以直接验证的一半.

命题 13.3 (深度集合聚合的置换不变性)

设 $\boldsymbol{h}_1,\cdots,\boldsymbol{h}_K$ 为某节点邻居的嵌入向量,σ 为 $\{1,\cdots,K\}$ 的任意置换. 定义

$$F(\{\boldsymbol{h}_1,\cdots,\boldsymbol{h}_K\}) = \mathrm{MLP}_\theta\left(\sum_{k=1}^{K}\mathrm{MLP}_\phi(\boldsymbol{h}_k)\right),$$

则 $F(\{\boldsymbol{h}_{\sigma(1)},\cdots,\boldsymbol{h}_{\sigma(K)}\}) = F(\{\boldsymbol{h}_1,\cdots,\boldsymbol{h}_K\})$, 即(13.15)的聚合算子与邻居的排序无关.

♠

证明

(重构新增) 设 $\{\boldsymbol{h}'_1,\cdots,\boldsymbol{h}'_K\}$ 是 $\{\boldsymbol{h}_1,\cdots,\boldsymbol{h}_K\}$ 的一个重排, 即存在置换 σ 使 $\boldsymbol{h}'_k = \boldsymbol{h}_{\sigma(k)}$. 则内层求和满足

$$\sum_{k=1}^{K}\mathrm{MLP}_\phi(\boldsymbol{h}'_k) = \sum_{k=1}^{K}\mathrm{MLP}_\phi(\boldsymbol{h}_{\sigma(k)}) = \sum_{k'=1}^{K}\mathrm{MLP}_\phi(\boldsymbol{h}_{k'}),$$

其中第二个等号是有限和的重指标 (σ 是双射, 遍历 k 等价于遍历 $k'=\sigma(k)$). 这里用到 MLP_ϕ 逐元素作用、与 $\boldsymbol{h}$ 的下标无关这一事实. 由于内层求和的结果完全相同, 再经过同一个 MLP_θ 得到的输出也相同, 故 F 对邻居的任意重排不变. 证毕.

□

注

(重构新增) 命题的证明只有一步, 但它的条件值得逐条核对. 第一,MLP_ϕ 必须对所有邻居使用同一组参数; 若每个邻居有自己的 MLP, 重排会改变“哪个网络作用在哪个向量上”, 结论立刻失效. 第二, 内层必须是对全部邻居的求和 (或平均、最大), 而不能是带位置权重的加权和. 第三,MLP_θ 只作用于求和结果这一个向量, 因此它看不到邻居的顺序.

反向的问题是这条命题回答不了的: 满足置换不变的函数很多, 为什么偏偏选“变换—求和—变换”这一族? Zaheer et al. (2017) 的结论给出了回答: 在温和的连续性条件下, 任何置换不变的连续函数都可以写成 $\rho\left(\sum_k \phi(\boldsymbol{h}_k)\right)$ 的形式, 而 ρ 与 ϕ 可以用 MLP 逼近. 原书引用这一结论而未加证明, 它属于本章范围之外的逼近论结果; 本注记只说明它的地位, 不把它当作已证事实使用.

最后, 当图没有边时 $\mathcal{N}(n)$ 只含其他所有节点,(13.15)就退化为作用在无结构集合上的算子, 这解释了原书所说的“深度集合” (deep set) 是图神经网络的特例: 图神经网络严格包含深度集合, 反之不成立.

如果我们考虑一个没有边的图, 它就退化成了一个简单的无结构节点集合, 这时的图神经网络就成为一个特例. 在这种情况下, 如果我们对集合中的每个向量 $\boldsymbol{h}_n^{(l)}$ 应用(13.15), 其中求和是在除 $\boldsymbol{h}_n^{(l)}$ 外的所有其他向量上进行的, 那么我们就有了一个关于无结构变量集合 [称为深度集合 (deep set)] 的学习函数的通用框架.

13.2.4 更新算子

在选择了合适的聚合算子之后, 我们同样需要决定更新算子的形式. 类似于 CNN 的公式(13.9), 一种简单的更新算子形式如下:

$$\mathrm{Update}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)}\right) = f\left(\boldsymbol{W}_{\mathrm{self}}\boldsymbol{h}_n^{(l)} + \boldsymbol{W}_{\mathrm{neigh}}\boldsymbol{z}_n^{(l)} + \boldsymbol{b}\right), \tag{13.16}$$

其中 $f(\cdot)$ 是一个非线性激活函数, 例如可以逐元素应用到向量参数上的 ReLU 函数;$\boldsymbol{W}_{\mathrm{self}}$ 和 $\boldsymbol{W}_{\mathrm{neigh}}$ 是可学习权重;$\boldsymbol{b}$ 是偏置;$\boldsymbol{z}_n^{(l)}$ 由(13.10)定义.

定义 13.12 (更新算子)

若选择简单的求和(13.12)作为聚合函数, 并且在节点及其邻居节点之间共享相同的权重矩阵, 使得 $\boldsymbol{W}_{\text{self}} = \boldsymbol{W}_{\text{neigh}}$, 就能得到如下特别简单的更新算子形式:

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l)}\right) = f\left(\boldsymbol{W}_{\text{neigh}} \sum_{m\in\mathcal{N}(n),n} \boldsymbol{h}_m^{(l)} + \boldsymbol{b}\right). \tag{13.17}$$

♣

注

(重构新增) (13.16)与(13.17)的差别不只是“少了一个矩阵”. 前者对“节点自身”和“邻居聚合”使用两个不同的线性映射, 因而能够表达“我”与“我的邻域”在语义上的不同角色; 后者令两者共用 $\boldsymbol{W}_{\text{neigh}}$, 相当于把节点自身也当作一个邻居, 于是(13.17)中的求和写成了 $\sum_{m\in\mathcal{N}(n),n}$ ——这个逗号加 n 正是“给图补上自环”的记号版本.

(13.16)中的三个参数各有分工:$\boldsymbol{W}_{\text{self}}$ 决定如何保留节点自身的信息,$\boldsymbol{W}_{\text{neigh}}$ 决定如何吸收邻域信息,$\boldsymbol{b}$ 提供一个与节点无关的平移. 若去掉 $\boldsymbol{W}_{\text{self}}$ 这一项, 节点在更新后会“忘记”自己原来的特征, 多层之后所有节点的表示只由结构决定; 若去掉 $\boldsymbol{W}_{\text{neigh}}$, 图结构就完全不起作用.

两个矩阵的维数必须匹配:$\boldsymbol{h}_n^{(l)} \in \mathbb{R}^{D_l}$、$\boldsymbol{h}_n^{(l+1)} \in \mathbb{R}^{D_{l+1}}$ 时,$\boldsymbol{W}_{\text{self}}, \boldsymbol{W}_{\text{neigh}} \in \mathbb{R}^{D_{l+1}\times D_l}$、$\boldsymbol{b} \in \mathbb{R}^{D_{l+1}}$. 这与原书未显式写出维数的做法一致, 但在实现时必须核对.

消息传递算法通常通过设置 $\boldsymbol{h}_n^{(0)} = \boldsymbol{x}_n$ 来初始化. 然而, 有时我们可能希望每个节点都有一个内部表示向量, 其维度不同于 $\boldsymbol{x}_n$ 的维度. 这样的表示可以通过用额外的零填充节点向量 $\boldsymbol{x}_n$(以达到更高的维度), 或者简单地使用可学习的线性变换将节点向量转换到所需维度的空间来初始化. 当节点没有与之关联的数据变量时, 一种替代的初始化形式是使用一个标记了每个节点度数 (即邻居节点数量) 的独热向量.

总的来说, 我们可以将图神经网络表示为一系列连续变换节点嵌入的层. 如果我们将这些嵌入分组到一个矩阵 $\boldsymbol{H}$ 中, 其第 n 行是向量 $\boldsymbol{h}_n^{\mathsf{T}}$, 并初始化为数据矩阵 $\boldsymbol{X}$, 则可以将这些连续的变换写成以下形式:

定义 13.13 (图网络的层叠表示)

$$\begin{aligned}
\boldsymbol{H}^{(1)} &= \boldsymbol{F}\left(\boldsymbol{X}, \boldsymbol{A}, \boldsymbol{W}^{(1)}\right)\\
\boldsymbol{H}^{(2)} &= \boldsymbol{F}\left(\boldsymbol{H}^{(1)}, \boldsymbol{A}, \boldsymbol{W}^{(2)}\right)\\
&\vdots\\
\boldsymbol{H}^{(L)} &= \boldsymbol{F}\left(\boldsymbol{H}^{(L-1)}, \boldsymbol{A}, \boldsymbol{W}^{(L)}\right),
\end{aligned} \tag{13.18}$$

其中 $\boldsymbol{A}$ 是邻接矩阵,$\boldsymbol{W}^{(l)}$ 表示网络第 l 层中全部权重和偏置的集合.

♣

注

(重构新增) (13.18)把“深度”写成了同一个算子 $\boldsymbol{F}$ 的反复复合, 其中每一层只更换参数集合 $\boldsymbol{W}^{(l)}$, 图结构 $\boldsymbol{A}$ 在所有层中保持不变. 这个写法有两个直接好处: 一是它把“层数”与“图的规模”彻底解耦,L 是网络的超参数而 N 由数据决定; 二是它为归纳证明提供了合适的形式——只要证明单层 $\boldsymbol{F}$ 与置换矩阵可交换, 复合的等变性就自动成立 (定理 13.1).

注意 $\boldsymbol{F}$ 的第二个自变量始终是同一个 $\boldsymbol{A}$, 而不是 $\boldsymbol{A}^{(l)}$. 有些架构会随层改变图的连接 (例如池化后粗化图), 那就不再是(13.18)的形式, 需要另作分析. 另外,$\boldsymbol{W}^{(l)}$ 逐层不同是默认设定; 原书也提到参数可以跨层共享, 那相当于令所有 $\boldsymbol{W}^{(l)}$ 相等, 此时 $\boldsymbol{F}$ 的反复复合就是一个迭代映射, 其不动点与过度平滑现象直接相关 (小节 13.3.4).

在置换矩阵 $\boldsymbol{P}$ 定义的节点重新排序下, 第 l 层计算的节点嵌入的变换是等变的:

$$\boldsymbol{P}\boldsymbol{H}^{(l)} = \boldsymbol{F}\left(\boldsymbol{P}\boldsymbol{H}^{(l-1)}, \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}, \boldsymbol{W}^{(l)}\right). \tag{13.19}$$

定理 13.1 (图神经网络的置换等变性)

设单层变换 $\boldsymbol{F}$ 由(13.10)与(13.11)给出，其中聚合算子对邻居的排序不变、更新算子逐节点作用且参数在所有节点间共享. 若对某一层 l 有(13.19)，则对由(13.18)定义的整个网络，

$$\boldsymbol{P}\boldsymbol{H}^{(L)} = \boldsymbol{F}\left(\boldsymbol{P}\boldsymbol{H}^{(L-1)}, \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}, \boldsymbol{W}^{(L)}\right)$$

对任意置换矩阵 $\boldsymbol{P}$ 成立；换言之，整个网络计算出一个等变的转换.

♡

证明

(重构新增) 对层数 l 作归纳. 归纳假设为：当输入数据被 $\boldsymbol{P}$ 重排，即 $\widetilde{\boldsymbol{X}} = \boldsymbol{P}\boldsymbol{X}$、$\widetilde{\boldsymbol{A}} = \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}$ 时，第 l 层的嵌入满足 $\widetilde{\boldsymbol{H}}^{(l)} = \boldsymbol{P}\boldsymbol{H}^{(l)}$.

奠基 $l = 0$：由初始化 $\boldsymbol{H}^{(0)} = \boldsymbol{X}$ 及(13.4)，$\widetilde{\boldsymbol{H}}^{(0)} = \widetilde{\boldsymbol{X}} = \boldsymbol{P}\boldsymbol{X} = \boldsymbol{P}\boldsymbol{H}^{(0)}$. 归纳步：设 $\widetilde{\boldsymbol{H}}^{(l-1)} = \boldsymbol{P}\boldsymbol{H}^{(l-1)}$. 由(13.18)，$\widetilde{\boldsymbol{H}}^{(l)} = \boldsymbol{F}(\widetilde{\boldsymbol{H}}^{(l-1)}, \widetilde{\boldsymbol{A}}, \boldsymbol{W}^{(l)}) = \boldsymbol{F}(\boldsymbol{P}\boldsymbol{H}^{(l-1)}, \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}, \boldsymbol{W}^{(l)})$. 由单层等变性(13.19)，右端等于 $\boldsymbol{P}\boldsymbol{F}(\boldsymbol{H}^{(l-1)}, \boldsymbol{A}, \boldsymbol{W}^{(l)}) = \boldsymbol{P}\boldsymbol{H}^{(l)}$，归纳步完成. 还需说明(13.19)本身为什么成立，否则归纳是空的. 设 $\widetilde{\boldsymbol{H}} = \boldsymbol{P}\boldsymbol{H}$，即 $\widetilde{\boldsymbol{h}}_n = \boldsymbol{h}_{\pi^{-1}(n)}$，并记 $\widetilde{\mathcal{N}}(n) = \pi(\mathcal{N}(\pi^{-1}(n)))$. 聚合阶段：

$$\begin{aligned}\widetilde{\boldsymbol{z}}_n &= \mathrm{Aggregate}\left(\{\widetilde{\boldsymbol{h}}_m : m \in \widetilde{\mathcal{N}}(n)\}\right)\\ &= \mathrm{Aggregate}\left(\{\boldsymbol{h}_{m'} : m' \in \mathcal{N}(\pi^{-1}(n))\}\right) = \boldsymbol{z}_{\pi^{-1}(n)},\end{aligned}$$

第一个等号用 $\widetilde{\boldsymbol{A}} = \boldsymbol{P}\boldsymbol{A}\boldsymbol{P}^{\mathsf{T}}$ 所刻画的邻接关系 ($\widetilde{A}_{nm} = A_{\pi^{-1}(n)\pi^{-1}(m)}$)，第二个等号只是重指标 $m' = \pi^{-1}(m)$，最后一个等号用聚合对排序不变这一假设. 更新阶段：

$$\begin{aligned}\widetilde{\boldsymbol{h}}_n^{(l+1)} &= \mathrm{Update}\left(\widetilde{\boldsymbol{h}}_n, \widetilde{\boldsymbol{z}}_n\right)\\ &= \mathrm{Update}\left(\boldsymbol{h}_{\pi^{-1}(n)}, \boldsymbol{z}_{\pi^{-1}(n)}\right) = \boldsymbol{h}_{\pi^{-1}(n)}^{(l+1)},\end{aligned}$$

其中第二个等号用归纳假设与上一式，第三个等号用更新算子逐节点作用且参数共享. 合并得 $\widetilde{\boldsymbol{H}}^{(l)} = \boldsymbol{P}\boldsymbol{H}^{(l)}$，即(13.19). 取 $l = L$ 即得定理结论. 证毕.

□

注

(重构新增) 这个定理是本章的中心结论，它把“图神经网络尊重节点编号的任意性”从一句设计口号变成了对任意层数的严格断言. 证明的结构值得记住：等变性不是逐层“偶然成立”的，而是由单层性质加归纳传递的，因此只要单层满足(13.19)，无论堆多少层都不会破坏它. 这也解释了节 13.2开头那句“如果网络的每一层都在节点重新排序后保持等变性，那么多层叠加后的网络也将保持等变性”.

定理对聚合算子与更新算子的要求是充分的，但不是必要的：某些并不逐层等变的架构 (例如在中间层做全局池化) 仍可能给出不变的整体输出. 定理也不涉及数值稳定性、表达能力或泛化，它只断言对称性.

还要注意，等变的是 $\boldsymbol{H}^{(L)}$ 这一整块节点嵌入，而不是某个图级标量. 若要得到图级预测，必须在 $\boldsymbol{H}^{(L)}$ 之后接一个置换不变的读出函数，否则输出会随编号改变——这正是下一小节与小节 13.2.7的分工.

13.2.5 节点分类

图神经网络可以看作一系列层，其中的每一层都将一组节点嵌入向量 $\{\boldsymbol{h}_n^{(l)}\}$ 转换成同样大小和维度的新向量集合 $\{\boldsymbol{h}_n^{(l+1)}\}$. 在网络的最后一个卷积层之后，我们需要得到预测结果，以便定义一个用于训练的损失函数，以及使用训练好的网络对新数据进行预测.

考虑对图中节点分类的任务，这是图神经网络最常见的用途之一. 我们可以定义一个输出层，有时称为读出层 (readout layer)，该层为每个节点计算一个与 C 个分类相对应的 softmax 函数.

定义 13.14 (读出层与节点分类损失)

读出层的输出由 softmax 函数给出, 形式如下:

$$y_{ni} = \frac{\exp\left(\boldsymbol{w}_i^{\mathsf{T}} \boldsymbol{h}_n^{(L)}\right)}{\sum_j \exp\left(\boldsymbol{w}_j^{\mathsf{T}} \boldsymbol{h}_n^{(L)}\right)}, \tag{13.20}$$

其中 $\{\boldsymbol{w}_i\}$ 是一组可学习的权重向量,$i = 1, \cdots, C$. 我们可以将损失函数定义为所有节点和所有类别的交叉熵损失之和:

$$\mathcal{L} = -\sum_{n \in \mathcal{V}_{\text{train}}} \sum_{i=1}^{C} t_{ni} \ln y_{ni}, \tag{13.21}$$

其中 t_{ni} 是目标值, 每个 n 值都有一个独热编码.

♣

注

(重构新增) (13.20)与普通的 softmax 回归形式完全相同, 唯一的区别在于输入 $\boldsymbol{h}_n^{(L)}$ 已经通过 L 层消息传递融合了邻域信息. 因此“图神经网络做节点分类”并没有引入新的输出层, 它只是把特征提取换成了尊重图结构的那一种.

(13.21)中的求和范围是 $\mathcal{V}_{\text{train}}$ 而不是全部节点, 这是节点分类与普通分类的关键差别: 图是半监督设定的载体, 未标注节点仍然参与消息传递 (它们出现在(13.10)的邻居集合中), 只是不进入损失. 这一点在下一段的三类节点划分中被明确区分.

t_{ni} 取独热编码是一个约定, 它使(13.21)简化为 $-\sum_n \ln y_{n,t_n}$, 其中 t_n 是节点 n 的类别. 若标签有软分配 (例如标注者的分歧), 则保留一般的 t_{ni} 即可.

命题 13.4 (节点分类损失的置换不变性)

设 y_{ni} 由(13.20)给出,$\mathcal{L}$ 由(13.21)给出,t_{ni} 随节点一同重排. 则对任意置换矩阵 $\boldsymbol{P}$, 有 $\mathcal{L}(\widetilde{\boldsymbol{H}}^{(L)}, \widetilde{\boldsymbol{t}}) = \mathcal{L}(\boldsymbol{H}^{(L)}, \boldsymbol{t})$.

♠

证明

(重构新增) 由定理 13.1,$\widetilde{\boldsymbol{h}}_n^{(L)} = \boldsymbol{h}_{\pi^{-1}(n)}^{(L)}$. 由于权重向量 $\boldsymbol{w}_i$ 在所有节点间共享,(13.20)对每个节点逐点计算, 故

$$\begin{aligned} \widetilde{y}_{ni} &= \frac{\exp(\boldsymbol{w}_i^{\mathsf{T}} \widetilde{\boldsymbol{h}}_n^{(L)})}{\sum_j \exp(\boldsymbol{w}_j^{\mathsf{T}} \widetilde{\boldsymbol{h}}_n^{(L)})} \\ &= \frac{\exp(\boldsymbol{w}_i^{\mathsf{T}} \boldsymbol{h}_{\pi^{-1}(n)}^{(L)})}{\sum_j \exp(\boldsymbol{w}_j^{\mathsf{T}} \boldsymbol{h}_{\pi^{-1}(n)}^{(L)})} = y_{\pi^{-1}(n),i}, \end{aligned}$$

即输出随节点一同等变. 又 t_{ni} 随节点重排,$\widetilde{t}_{ni} = t_{\pi^{-1}(n),i}$, 于是每一项 $\widetilde{t}_{ni} \ln \widetilde{y}_{ni} = t_{\pi^{-1}(n),i} \ln y_{\pi^{-1}(n),i}$. 对 $n \in \mathcal{V}_{\text{train}}$ 求和时作重指标 $n' = \pi^{-1}(n)$, 训练集 $\mathcal{V}_{\text{train}}$ 也随节点一同重排为 $\widetilde{\mathcal{V}}_{\text{train}} = \pi(\mathcal{V}_{\text{train}})$, 故和值不变:$\mathcal{L}(\widetilde{\boldsymbol{H}}^{(L)}, \widetilde{\boldsymbol{t}}) = \mathcal{L}(\boldsymbol{H}^{(L)}, \boldsymbol{t})$. 证毕.

□

注

(重构新增) 这条命题说明了“损失不变”是怎么来的: 等变的输出乘上同样等变的目标, 再对节点求和, 重排只是把求和项重新排列. 三个条件都不能少——$\boldsymbol{w}_i$ 必须共享 (否则 $\widetilde{y}$ 不等于 y 的重排)、目标必须随节点重排 (否则求和项被错配)、求和必须遍历同一批节点 (否则重排会改变训练集).

如果目标是在输出处预测连续值而非类别, 则把(13.20)换成线性变换、把(13.21)换成平方和误差即可, 命题的论证逐字不变, 因为只用到“逐节点计算 + 共享参数 + 对节点求和”这三点.

式(13.21)中对 n 的求和是在用于训练的节点子集 $\mathcal{V}_{\text{train}}$ 上进行的. 我们需要区分以下 3 种类型的节点.

(1) 节点子集 $\mathcal{V}_{\text{train}}$ 中的节点是有标签的, 既参与图神经网络的消息传递操作, 也用于计算训练的损失函数.

(2) 可能还有一个名为 $\mathcal{V}_{\text{trans}}$ 的节点子集, 里面是未标记的直推式 (transductive) 节点并且不参与用于训练的损失函数的计算. 然而, 它们仍然参与训练和推理阶段的消息传递, 它们的标签可能会作为推理过程的一部分

被预测.

(3) 剩余的节点标记为 $\mathcal{V}_{\text{induct}}$, 是一组归纳节点, 它们不用于计算损失函数, 并且这些节点及其关联的边在训练阶段不参与消息传递. 然而, 它们在推理阶段参与消息传递, 它们的标签预测为推理的结果.

定义 13.15 (归纳学习与直推式学习)

如果没有直推式节点, 即测试节点 (及其关联的边) 在训练阶段不可用, 那么通常将这种训练称为归纳学习 (inductive learning), 这可以视为一种监督学习 (supervised learning) 的形式. 然而, 如果存在直推式节点, 则称这种训练为直推式学习 (transductive learning), 这可以视为半监督 (semi-supervised) 学习的一种形式. ♣

注

(重构新增) 这三类节点的划分不是术语游戏, 它决定了训练时哪些边可以出现在计算图中. $\mathcal{V}_{\text{train}}$ 与 $\mathcal{V}_{\text{trans}}$ 的节点在训练阶段参与消息传递, 因此它们的特征会通过邻居影响有标签节点的嵌入; $\mathcal{V}_{\text{induct}}$ 的节点及其边在训练阶段被完全移出, 因此模型在训练时从未见过它们.

这个区别对应两种不同的泛化问题. 直推式学习回答的是“在这张已经给定的图上, 未标注节点的标签是什么”, 模型可以 (也必然) 利用测试节点的结构位置; 归纳学习回答的是“对一张训练时从未出现过的新图, 如何预测”, 因此要求模型对任意图都能前向计算. 分子性质预测属于后者, 社交网络中的用户分类通常属于前者.

从工程角度看, 直推式的代价是模型与特定图绑定, 图一变就必须重新训练; 归纳式的代价是必须让所有参数与 N 无关, 而这正好由参数共享 (定理 13.1) 保证.

13.2.6 边分类

在某些应用中, 我们希望对图的边而不是节点进行预测. 一种常见的边分类任务是边补全. 在这种任务中, 目标是确定两个节点之间是否应该存在一条边. 给定一组节点嵌入, 可以使用嵌入向量之间的点积, 通过逻辑斯谛 sigmoid 函数定义两个节点 n 和 m 之间存在边的概率 $p(n,m)$.

定义 13.16 (边存在概率)

两个节点 n 和 m 之间存在边的概率定义为

$$p(n,m)=\sigma\left(\boldsymbol{h}_n^{\top}\boldsymbol{h}_m\right), \tag{13.22}$$

其中 $\sigma(\cdot)$ 是逻辑斯谛 sigmoid 函数, $\boldsymbol{h}_n,\boldsymbol{h}_m$ 为最终的节点嵌入. ♣

注

(重构新增) 用内积构造边得分是“嵌入空间几何”的直接体现: 两个节点的嵌入越同向、模越长, 内积越大, 判定存在边的概率越高. 它的一个好处是对称——$p(n,m)=p(m,n)$, 与本章的无向图假设一致; 若图是有向的, 就需要把 $\boldsymbol{h}_n^{\top}\boldsymbol{h}_m$ 换成 $\boldsymbol{h}_n^{\top}\boldsymbol{W}\boldsymbol{h}_m$ 之类的不对称形式.

(13.22)对节点重排是等变的: 重排只会把 $p(n,m)$ 换成 $p(\pi(n),\pi(m))$, 而不会改变任何一对节点之间的数值, 因此以边为单位计算损失时整体不变. 这一点与命题 13.4同理, 不再单列命题.

注意 $p(n,m)$ 只用了两个端点的嵌入, 没有显式使用边的特征. 若边本身带有观测数据, 应当把它们并入 $\boldsymbol{h}$ 或改用定义 13.21中的边嵌入.

例题 13.1 社交网络中的边预测 一个典型的应用实例是, 预测社交网络中任意两个用户是否拥有共同的兴趣爱好, 并愿意建立联系.

13.2.7 图分类

在图神经网络的某些应用中, 我们要在给定一组带标签的训练图 $\mathcal{G}_1,\cdots,\mathcal{G}_N$ 的情况下, 预测新图的属性. 这要求我们以不依赖于任意节点排序的方式组合最终层的嵌入向量, 从而确保输出的预测结果与节点的排序无关. 这任务有点像聚合函数, 只不过这里考虑的是图中所有节点, 而不仅仅是单个节点的邻域集.

定义 13.17 (图级读出函数)

最简单的方法是对节点嵌入向量进行求和:

$$y = f\left(\sum_{n\in\mathcal{V}} \boldsymbol{h}_n^{(L)}\right), \tag{13.23}$$

其中, 函数 f 可能包含可学习参数, 如线性变换或神经网络. 也可以使用其他不变的聚合函数, 例如平均值、逐元素的最小值或最大值.

♣

注

(重构新增) 读出函数是“等变 $\to$ 不变”的转换器:$\boldsymbol{H}^{(L)}$ 是等变的, 对它做一次置换不变的归约就得到与节点编号无关的 y.(13.23)选的是求和, 于是 y 只依赖全部节点嵌入的多重集合, 而不依赖它们分别属于哪个节点——这正是图级预测需要的语义.

求和与平均在这里的差别比在聚合算子中更实质: 求和使 y 随图的规模 (节点数) 增长, 因此同一个模型在不同大小的图上给出的 y 量级不同; 平均则消除了节点数的影响. 分子性质预测通常用求和或平均都能工作, 但若训练集与测试集的分子大小分布差异很大, 平均更稳妥. 逐元素最大值只保留每个维度上最突出的节点, 适合“是否存在某种子结构”这类问题.

f 中是否含可学习参数不影响不变性, 因为 f 作用在已经归约好的单个向量上, 看不到节点顺序.

例题 13.2 候选药物分子的图分类 对于分类问题, 例如将候选药物分子标记为有毒或安全, 通常采用交叉熵损失函数; 而对于回归问题, 例如预测候选药物分子的溶解度, 则通常采用平方误差损失函数. 图级别的预测对应于归纳任务, 因为必须有不同的图集合用于训练和推理.

13.3 通用图网络

迄今为止我们所考虑的图网络有许多变体和扩展. 这里我们概述一些关键概念以及一些实践考虑.

13.3.1 图注意力网络

注意力机制在作为 Transformer 架构的基础时展现出强大的能力 (参见节 12.1). 在图神经网络中, 注意力机制可以用于构建一种能够整合来自邻居节点信息的聚合函数.

定义 13.18 (图注意力网络)

传入的消息根据注意力系数 A_{nm} 加权, 从而得到

$$z_n^{(l)} = \text{Aggregate}\left(\{\boldsymbol{h}_m^{(l)} : m \in \mathcal{N}(n)\}\right) = \sum_{m\in\mathcal{N}(n)} A_{nm}\, \boldsymbol{h}_m^{(l)}, \tag{13.24}$$

其中, 注意力系数满足

$$A_{nm} \geqslant 0 \tag{13.25}$$

$$\sum_{m\in\mathcal{N}(n)} A_{nm} = 1. \tag{13.26}$$

♣

注

(重构新增) (13.25)与(13.26)两条约束的作用不同: 非负性使(13.24)成为邻居嵌入的一个凸组合, 因而聚合结果不会超出邻居嵌入的凸包, 数值上不会因为正负抵消而失控; 归一化使聚合量的尺度与邻居个数无关, 避免了(13.12)中“度数越大、量级越大”的问题. 两条合起来,A_{nm} 可以被读作“节点 n 把多少注意力分给邻居 m”的分配比例.

与(13.13)的固定权重 $1/|\mathcal{N}(n)|$ 相比, 注意力权重是数据相关的: 它由当前的嵌入计算得到, 因此同一张图上不同样本、不同层的权重都可以不同. 这正是原书所说的归纳偏置——某些相邻节点比其他节点更重要, 而重要性由数据决定.

代价是计算量与存储量: 每一条边都要算一个注意力系数, 稠密图上这是 $O(N^2)$ 的量. 此外,(13.26)的分母只对邻居求和, 因此孤立节点的分母为 0, 实际实现中需要补自环或特殊处理.

这种网络称为图注意力网络 (graph attention network)(Veličković et al., 2017), 它可以捕获一种归纳偏置, 该归纳偏置表明在确定最佳更新的过程中, 某些相邻节点比其他节点更重要, 而这种重要性取决于数据本身.

构建注意力系数的方法有很多种, 但通常都会使用 softmax 函数.

定义 13.19 (注意力系数)

例如, 我们可以使用双线性形式

$$A_{nm} = \frac{\exp\left(\boldsymbol{h}_n^{\mathsf{T}} \boldsymbol{W} \boldsymbol{h}_m\right)}{\sum_{m' \in \mathcal{N}(n)} \exp\left(\boldsymbol{h}_n^{\mathsf{T}} \boldsymbol{W} \boldsymbol{h}_{m'}\right)}, \tag{13.27}$$

其中 $\boldsymbol{W}$ 是一个 $D \times D$ 的可学习参数矩阵. 一种更通用的选择是使用神经网络组合边的两个端点的节点的嵌入向量:

$$A_{nm} = \frac{\exp\{\mathrm{MLP}(\boldsymbol{h}_n, \boldsymbol{h}_m)\}}{\sum_{m' \in \mathcal{N}(n)} \exp\{\mathrm{MLP}(\boldsymbol{h}_n, \boldsymbol{h}_{m'})\}}, \tag{13.28}$$

其中 MLP 具有单个的连续输出变量, 并且当输入向量交换时, 这个输出变量的值不变. 只要 MLP 在网络中的所有节点之间是共享的 (见练习 13.8), 这个聚合函数就会在节点重新排序下保持等变性.

注

(重构新增) (13.27)与(13.28)都是“先算得分、再在邻域上做 softmax”的两步结构, 区别只在得分函数. 双线性形式的得分是 $\boldsymbol{h}_n^{\mathsf{T}} \boldsymbol{W} \boldsymbol{h}_m$, 它对 $\boldsymbol{W}$ 线性, 因而训练稳定、参数量只有 D^2;MLP 形式的得分是任意可学的标量函数, 表达力更强但需要更多数据.

两条公式都自动满足(13.25)(指数函数为正) 与(13.26)(softmax 分母的定义), 因此定义中那两条约束不是额外假设, 而是构造方式的直接结果.

定义中对 MLP 提出了一个不显眼但关键的要求: 交换两个输入向量时输出不变. 若 MLP 不对称, 则 $A_{nm} \neq A_{mn}$, 聚合将依赖“谁是被更新的节点”, 这在无向图上没有意义, 也会破坏等变性. 双线性形式 $\boldsymbol{h}_n^{\mathsf{T}} \boldsymbol{W} \boldsymbol{h}_m$ 只有在 $\boldsymbol{W}$ 对称时才满足这一要求; 原书未强调这一点, 实现时若 $\boldsymbol{W}$ 取一般矩阵, 得到的是有向注意力.

命题 13.5 (图注意力聚合的置换等变性)

设注意力系数由(13.28)给出, 其中 MLP 在所有节点间共享且对两个输入向量对称,$\boldsymbol{P}$ 为任意置换矩阵. 则按(13.24)定义的聚合算子满足

$$\mathrm{Aggregate}\left(\{\boldsymbol{P}\boldsymbol{h}_m : m \in \mathcal{N}(n)\}\right) = \boldsymbol{P}\,\mathrm{Aggregate}\left(\{\boldsymbol{h}_m : m \in \mathcal{N}(n)\}\right),$$

即该聚合函数在节点重新排序下是等变的.

证明

(重构新增) 记 $\widetilde{\boldsymbol{h}}_m = \boldsymbol{h}_{\pi^{-1}(m)}$, 则按(13.28), 重排后的注意力系数为

$$\begin{aligned}\widetilde{A}_{nm} &= \frac{\exp\{\mathrm{MLP}(\widetilde{\boldsymbol{h}}_n, \widetilde{\boldsymbol{h}}_m)\}}{\sum_{m'\in\widetilde{\mathcal{N}}(n)} \exp\{\mathrm{MLP}(\widetilde{\boldsymbol{h}}_n, \widetilde{\boldsymbol{h}}_{m'})\}} \\ &= \frac{\exp\{\mathrm{MLP}(\boldsymbol{h}_{\pi^{-1}(n)}, \boldsymbol{h}_{\pi^{-1}(m)})\}}{\sum_{m'\in\widetilde{\mathcal{N}}(n)} \exp\{\mathrm{MLP}(\boldsymbol{h}_{\pi^{-1}(n)}, \boldsymbol{h}_{\pi^{-1}(m')})\}},\end{aligned}$$

把分母中的求和变量换成 $m'' = \pi^{-1}(m')$, 则 m'' 遍历 $\mathcal{N}(\pi^{-1}(n))$, 分母恰为 $\sum_{m''\in\mathcal{N}(\pi^{-1}(n))} \exp\{\mathrm{MLP}(\boldsymbol{h}_{\pi^{-1}(n)}, \boldsymbol{h}_{m''})\}$, 故 $\widetilde{A}_{nm} = A_{\pi^{-1}(n)\pi^{-1}(m)}$. 这里用到 MLP 在所有节点间共享 (分子分母中的 MLP 是同一个函数) 这一事实. 于是

$$\begin{aligned}\widetilde{\boldsymbol{z}}_n &= \sum_{m\in\widetilde{\mathcal{N}}(n)} \widetilde{A}_{nm}\widetilde{\boldsymbol{h}}_m = \sum_{m\in\widetilde{\mathcal{N}}(n)} A_{\pi^{-1}(n)\pi^{-1}(m)}\boldsymbol{h}_{\pi^{-1}(m)} \\ &= \sum_{m''\in\mathcal{N}(\pi^{-1}(n))} A_{\pi^{-1}(n)m''}\boldsymbol{h}_{m''} = \boldsymbol{z}_{\pi^{-1}(n)},\end{aligned}$$

第二个等号是重指标 $m'' = \pi^{-1}(m)$. 这恰是 $\widetilde{\boldsymbol{z}} = \boldsymbol{P}\boldsymbol{z}$, 即聚合算子与置换矩阵可交换. 证毕.

□

注

(重构新增) 证明中真正用到对称性的地方只有一处: 把 $\widetilde{A}_{nm}$ 的分子写成 $\mathrm{MLP}(\boldsymbol{h}_{\pi^{-1}(n)}, \boldsymbol{h}_{\pi^{-1}(m)})$ 时, 两个参数的位置必须与原式一致. 若 MLP 不对称, 重排会让“查询向量”与“键向量”的角色互换,A_{nm} 与 A_{mn} 不再相等, 聚合结果依赖于节点编号的方向, 等变性随之失效.

命题没有断言 $\boldsymbol{z}_n$ 对邻居的置换不变——它不是不变的, 而是等变的: 重排节点后, 第 n 个分量变成原来的第 $\pi^{-1}(n)$ 个分量. 这正是(13.7)要求的行为.

把本命题与定理 13.1结合可知, 用注意力聚合替换求和聚合不会破坏整个网络的等变性, 因此图注意力网络可以任意堆叠. 原书把这一验证留作练习 13.8, 这里给出完整论证以便后续引用.

定义 13.20 (多头图注意力)

图注意力网络可以通过引入多头注意力机制来进行扩展, 在这个机制中, 可以定义 H 组不同的注意力权重 $A^{(h)}_{nm}, h = 1, \cdots, H$. 每个注意力头都使用上面描述的机制之一进行计算, 并且具有自己的独立参数 (参见小节 12.1.6). 然后在聚合步骤中使用拼接和线性投影将它们组合起来. 需要注意的是, 对于全连接网络, 多头图注意力网络就变成标准的 Transformer 编码器 (见练习 13.9).

♣

注

(重构新增) 多头的意义在于让不同的头关注邻域中不同的关系模式: 一个头可能按化学键类型分配注意力, 另一个头可能按原子距离分配, 最后的拼接与线性投影把这些视角合并. 这与卷积网络中一层使用多个滤波器完全平行.

每个头必须有自己独立的参数集, 否则 H 个头会退化为同一个头, 只是把结果重复了 H 次. 拼接使输出维数变为 H 倍 (当每个头输出 D/H 维时恰好回到 D 维), 线性投影则负责把拼接结果映射回所需的维数并混合各头的信息.

“全连接图上多头图注意力即 Transformer 编码器”这一断言的确切含义与证明见练习 13.9: 在完全图上邻居集合就是除自身外的全部节点,softmax 注意力与 Transformer 的自注意力在形式上一致, 差别只在是否包含自身以及缩放因子的写法.

13.3.2 边嵌入

前面讨论的图神经网络使用的嵌入向量都与节点相关联. 我们已经看到一些网络也具有与边相关联的数据. 即使不存在与边相关联的观测值, 我们也仍然可以维护和更新基于边的隐藏变量, 这些变量可以为图神经网络学习的内部表示做出贡献. 除了由 $\boldsymbol{h}_n^{(l)}$ 给出的节点嵌入, 我们还引入了边嵌入 $\boldsymbol{e}_{nm}^{(l)}$.

定义 13.21 (带边嵌入的消息传递)

使用边嵌入 $\boldsymbol{e}_{nm}^{(l)}$, 可以用以下形式定义一般的消息传递方程:

$$\boldsymbol{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\boldsymbol{e}_{nm}^{(l)}, \boldsymbol{h}_{n}^{(l)}, \boldsymbol{h}_{m}^{(l)}\right) \tag{13.29}$$

$$z_{n}^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\boldsymbol{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.30}$$

$$\boldsymbol{h}_{n}^{(l+1)} = \text{Update}_{\text{node}}\left(\boldsymbol{h}_{n}^{(l)}, z_{n}^{(l+1)}\right). \tag{13.31}$$

最终层学习到的边嵌入 $\boldsymbol{e}_{nm}^{(l)}$ 可以直接用来预测与边相关的内容.

♣

注

(重构新增) 这三式相对(13.10)与(13.11)的推广在于多了一层“边”: 消息不再直接取邻居的节点嵌入, 而是先由两端节点的嵌入生成一个边向量, 再从边向量聚合到节点. 这样做的收益是表示能力的提升——边向量可以编码“这两个节点之间的关系类型”, 而仅用节点嵌入的求和无法区分“同一条消息来自不同类型的边”.

三式的执行顺序不能随意交换:(13.30)用的是 $\boldsymbol{e}_{nm}^{(l+1)}$ 而不是 $\boldsymbol{e}_{nm}^{(l)}$, 即先更新边、再聚合边、最后更新节点. 若改成先聚合旧边再更新边, 则本层的节点更新用不到本层的边信息, 表达力下降.

$\boldsymbol{e}_{nm}$ 的下标是有序对 (n, m), 但无向图中 (n, m) 与 (m, n) 是同一条边, 因此实现时必须保证 $\boldsymbol{e}_{nm} = \boldsymbol{e}_{mn}$, 否则等变性会被破坏. 原书写成 $\boldsymbol{e}_{nm}$ 是记号上的简化.

13.3.3 图嵌入

除了节点嵌入和边嵌入, 我们还可以维护和更新与整个图相关的嵌入向量 $\boldsymbol{g}^{(l)}$. 将所有这些方面结合在一起, 我们便能够为图结构化应用定义更通用的一组消息传递函数, 以及更丰富的学习表示集合.

定义 13.22 (通用图网络的消息传递方程)

具体来说, 可以定义如下消息传递方程 (Battaglia et al., 2018):

$$\boldsymbol{e}_{nm}^{(l+1)} = \text{Update}_{\text{edge}}\left(\boldsymbol{e}_{nm}^{(l)}, \boldsymbol{h}_{n}^{(l)}, \boldsymbol{h}_{m}^{(l)}, \boldsymbol{g}^{(l)}\right) \tag{13.32}$$

$$z_{n}^{(l+1)} = \text{Aggregate}_{\text{node}}\left(\{\boldsymbol{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.33}$$

$$\boldsymbol{h}_{n}^{(l+1)} = \text{Update}_{\text{node}}\left(\boldsymbol{h}_{n}^{(l)}, z_{n}^{(l+1)}, \boldsymbol{g}^{(l)}\right) \tag{13.34}$$

$$\boldsymbol{g}^{(l+1)} = \text{Update}_{\text{graph}}\left(\boldsymbol{g}^{(l)}, \{\boldsymbol{h}_{n}^{(l+1)} : n \in \mathcal{V}\}, \{\boldsymbol{e}_{nm}^{(l+1)} : (n, m) \in \mathcal{E}\}\right). \tag{13.35}$$

♣

注

(重构新增) 这四式把“节点—边—图”三个层次的变量放进同一个更新回路, 是本章最一般的消息传递形式; 前面所有的模型都是它的特例: 去掉 $\boldsymbol{g}$ 与边变量就回到(13.10), 保留边变量但去掉 $\boldsymbol{g}$ 就回到(13.29). 理解这一点比记住四式更重要——它说明这些模型之间的差别只是“哪些变量参与更新”, 而不是彼此无关的四种算法.

四个更新函数的输入各不相同, 这决定了信息流的顺序: 边只知道两端节点与全局状态, 节点知道自身、邻域聚合与全局状态, 全局变量则能看到全部节点与全部边. 因此 $\boldsymbol{g}$ 是唯一“全局可见”的变量, 它承担把远距离信息压缩传递的角色; 这也是为什么原书在讨论过度平滑时提到的“超级节点”与 $\boldsymbol{g}$ 有相似的作用.

(13.35)的输入是集合 $\{\boldsymbol{h}_{n}^{(l+1)}\}$ 与 $\{\boldsymbol{e}_{nm}^{(l+1)}\}$, 因此 $\text{Update}_{\text{graph}}$ 必须是置换不变的, 否则整个网络的图级表示会依赖节点编号. 这一点是(13.23)中求和要求的同一件事, 只是发生在每一层而不是只在最后一层.

这些消息传递方程首先更新边嵌入向量 $\boldsymbol{e}_{nm}^{(l+1)}$, 该操作是基于边嵌入向量的先前状态, 每条边连接的节点的嵌入向量, 以及图级嵌入向量 $\boldsymbol{g}^{(l)}$ 进行的, 参见(13.32). 然后将这些更新的边嵌入向量在连接到每个节点的每条边上进行聚合, 参见(13.33). 接下来转而利用当前节点的嵌入向量和图级嵌入向量更新节点嵌入向量 $\boldsymbol{h}_{n}^{(l+1)}$, 参见(13.34). 最后基于图中所有节点和所有边的信息, 以及从前一层获得的图级嵌入向量, 更新当前层的图级嵌入向量.图 13.5演示了上述过程, 算法 13.2 对图消息传递更新做了总结.

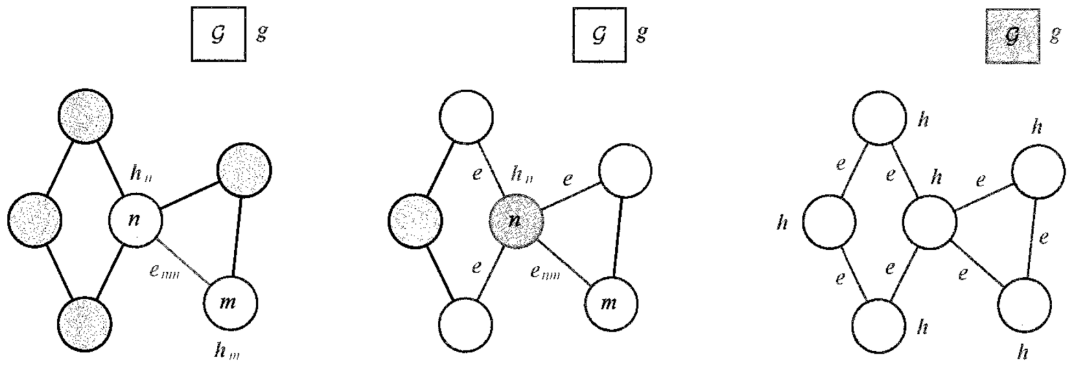


图 13.5: (13.32) ~ (13.35)定义的图消息传递更新的示意图, 展示了 (a) 边更新、(b) 节点更新, 以及 (c) 全局图更新. 在每种情况下, 正在更新的变量用红色显示, 而那些对该更新有贡献的变量则用蓝色显示

算法 13.2: 图消息传递更新

输入: 无向图 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$; 初始节点嵌入 $\{\boldsymbol{h}_n^{(0)}\}$; 初始边嵌入 $\{\boldsymbol{e}_{nm}^{(0)}\}$; 初始图嵌入 $\boldsymbol{g}^{(0)}$

输出: 最终节点嵌入 $\{\boldsymbol{h}_n^{(L)}\}$; 最终边嵌入 $\{\boldsymbol{e}_{nm}^{(L)}\}$; 最终图嵌入 $\boldsymbol{g}^{(L)}$

// 迭代地进行消息传递

for $l \in \{0, \cdots, L-1\}$ **do**

$\boldsymbol{e}_{nm}^{(l+1)} \leftarrow \text{Update}_{\text{edge}}\left(\boldsymbol{e}_{nm}^{(l)}, \boldsymbol{h}_n^{(l)}, \boldsymbol{h}_m^{(l)}, \boldsymbol{g}^{(l)}\right)$

$z_n^{(l+1)} \leftarrow \text{Aggregate}_{\text{node}}\left(\{\boldsymbol{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right)$

$\boldsymbol{h}_n^{(l+1)} \leftarrow \text{Update}_{\text{node}}\left(\boldsymbol{h}_n^{(l)}, z_n^{(l+1)}, \boldsymbol{g}^{(l)}\right)$

$\boldsymbol{g}^{(l+1)} \leftarrow \text{Update}_{\text{graph}}\left(\boldsymbol{g}^{(l)}, \{\boldsymbol{h}_n^{(l+1)}\}, \{\boldsymbol{e}_{nm}^{(l+1)}\}\right)$

end for

return $\{\boldsymbol{h}_n^{(L)}\}, \{\boldsymbol{e}_{nm}^{(L)}\}, \boldsymbol{g}^{(L)}$

13.3.4 过度平滑

某些图神经网络会出现的一个显著问题, 就是所谓的过度平滑 (over-smoothing).

定义 13.23 (过度平滑)

在多次迭代地进行消息传递后, 节点嵌入向量变得彼此非常相似, 这实际上限制了网络的深度. 这种现象称为过度平滑 (over-smoothing).

♣

注

(重构新增) 过度平滑的机制可以从(13.19)中读出: 每一层都把节点表示向其邻域的平均值推进一次, 反复作用相当于对邻接算子做幂迭代, 其极限由该算子的最大特征值对应的特征向量决定. 对于连通的非二部图, 这个极限使所有节点的表示趋于同一个向量, 于是节点之间不再可区分, 节点分类的判别信息随之消失.

因此“层数不能太深”不是训练技巧问题, 而是算子本身的性质: 堆叠 L 层等价于把感受野扩大到 L 跳, 当 L 接近图的直径时, 所有节点的感受野几乎覆盖整张图, 表示自然趋同. 图越小、越稠密, 过度平滑出现得越早.

定义中“彼此非常相似”需要一个度量才有确切含义, 常用的量是相邻节点表示之差的范数或表示矩阵的秩. 原书没有给出定量刻画, 本注记也不引入新的定理, 只指出这是本章余下两种缓解手段 (残差连接与多层读出) 所针对的现象.

缓解这个问题的一种方法是引入残差连接 (见节 9.5). 例如, 我们可以修改更新运算符 [(13.34)] .

定义 13.24 (残差式更新算子)

修改后的更新算子为

$$\boldsymbol{h}_n^{(l+1)} = \text{Update}_{\text{node}}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l+1)}, \boldsymbol{g}^{(l)}\right) + \boldsymbol{h}_n^{(l)}. \tag{13.36}$$

♣

注

(重构新增) (13.36)只加了一项 $\boldsymbol{h}_n^{(l)}$, 但它改变了整个迭代的定性行为: 无残差时第 $l+1$ 层的表示完全由邻域聚合决定, 节点自身的历史信息只能通过 $\boldsymbol{h}_n^{(l)}$ 这一个入口间接保留; 有残差时, 恒等映射成为默认路径, 网络需要“学习”的只是对恒等的修正量. 这与节 9.5中残差连接使深层网络可训练的理由相同.

从过度平滑的角度看, 残差项使每一层的表示至少保留上一层的全部信息, 因此即使聚合项把节点推向邻域平均, 表示也不会在有限层内被完全抹平. 但它只是缓解而不是根治: 若残差权重固定为 1, 聚合项的累积影响仍会随层数增长.

(13.36)要求 $\boldsymbol{h}_n^{(l)}$ 与 $\boldsymbol{h}_n^{(l+1)}$ 的维数相同, 因此带残差的层不能改变嵌入维数, 这一点在设计与(13.16)配合的网络结构时必须核对.

减少过度平滑的另一种方法是让输出层不仅从网络的最后一个卷积层获取信息, 也从所有先前层获取信息.

定义 13.25 (跳跃连接读出)

这可以通过拼接先前层的表示来完成:

$$\boldsymbol{y}_n = f\left(\boldsymbol{h}_n^{(1)} \oplus \boldsymbol{h}_n^{(2)} \oplus \cdots \oplus \boldsymbol{h}_n^{(L)}\right), \tag{13.37}$$

其中 $\boldsymbol{a} \oplus \boldsymbol{b}$ 表示对向量 $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 进行拼接. 这种方法的一个变体是使用最大池化 (max pooling) 而不是拼接来组合向量. 在这种情况下, 输出向量的每个元素由前几层中所有对应嵌入向量的元素的最大值决定.

♣

注

(重构新增) (13.37)把“用哪一层的表示”这个选择交给下游的分类器: 浅层的表示保留了更多局部、区分性的信息, 深层的表示融合了更大范围的结构, 拼接后两者都可用. 这相当于在深度方向上做了一次“多尺度”读取, 而不必赌某一层最好.

拼接与最大池化的差别在于是否保留层身份: 拼接后各层的分量位置固定, f 可以针对不同层学不同的权重; 最大池化则逐元素取最大, 输出维数不变, 但各层信息被混合, 无法再区分某个分量来自哪一层. 前者参数量随 L 线性增长, 后者不增长.

与(13.36)一样, 这种做法的代价是存储: 需要保留所有中间层的表示, 训练时的显存占用随层数线性增长.

13.3.5 正则化

标准的正则化技术可以用于图神经网络 (参见第 9 章), 包括将惩罚项 (例如参数值的平方和) 添加到损失函数中. 此外, 一些专门针对图神经网络开发的正则化方法也已经出现.

图神经网络已经采用权重共享来实现置换的等变性和不变性, 但通常它们在每层都有独立的参数. 然而, 权重和偏置也可以跨层共享, 以减少独立参数的数量.

定义 13.26 (图神经网络中的 dropout)

在图神经网络的背景下进行 dropout 指的是在训练期间省略图节点的随机子集, 对于每次前向传播则选择新的随机子集. 这同样适用于图中的边, 即在训练过程中, 随机移除或掩盖邻接矩阵中的部分元素.

♣

注

(重构新增) 图上的 dropout 与全连接网络中的 dropout 形似而神不同. 全连接网络中 dropout 随机丢弃的是单元, 作用在特征维上; 这里丢弃的是节点或边, 作用在图的索引集上. 丢弃一个节点等于同时删除它的全部特征与它的所有关联边, 因此一次 dropout 会改变图的拓扑, 而不只是屏蔽某些特征分量.

这种做法的正则化效果来自两个方面: 一是它迫使模型在缺失部分邻居的情况下仍能给出正确预测, 从而对噪声边与缺失节点稳健; 二是它相当于对图结构做数据增强, 缓解了“一张图只有一个样本”的窘境.

需要小心的是, 丢弃节点会改变归一化因子. 若使用(13.14)的对称归一化, 度 d_n 在训练与推理时不一致, 必须在实现中对保留的边重新归一化, 否则训练与推理的尺度不匹配.

13.3.6 几何深度学习

在设计针对图结构数据的深度学习模型时, 我们已经看到排列对称性是一个至关重要的因素. 它作为一种归纳偏置的形式, 在大幅减少数据需求的同时也提高了预测性能. 在与空间属性相关的图神经网络应用中, 如图形网格、流体流动模拟或分子结构, 我们还可以在网络架构中构建额外的等变性和不变性.

例如, 考虑预测分子属性的任务. 当探索候选药物的空间时, 分子可以表示为一个列表, 其中包括给定类型的原子 (碳原子、氢原子、氮原子等) 以及每个原子的空间坐标 (表示为三维的列向量). 我们可以为每个原子 n 在每一层 l 引入一个相关的嵌入向量, 表示为 $\boldsymbol{r}_n^{(l)}$, 并且这些向量可以用已知的原子坐标进行初始化. 然而, 这些向量的元素值取决于坐标系统的任意选择, 而分子的属性则不依赖于它. 例如, 如果分子在三维空间中旋转或相对于坐标系统原点移动到新位置, 或者如果坐标系统本身反射得到分子的镜像版本, 那么分子的溶解度保持不变. 因此, 分子的属性也应该在这样的变换下保持不变.

通过仔细选择更新操作和聚合操作的函数形式 (Satorras, Hoogeboom, and Welling, 2021), 新的嵌入向量 $\boldsymbol{r}_n^{(l)}$ 可以纳入图神经网络的更新方程 [(13.29) ~ (13.31)] 中, 以实现所需的对称性.

定义 13.27 (等变消息传递)

引入坐标嵌入 $\boldsymbol{r}_n^{(l)}$ 后的消息传递方程为

$$\boldsymbol{e}_{nm}^{(l+1)} = \mathrm{Update}_{\mathrm{edge}}\left(\boldsymbol{e}_{nm}^{(l)}, \boldsymbol{h}_n^{(l)}, \boldsymbol{h}_m^{(l)}, \|\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)}\|^2\right) \tag{13.38}$$

$$\boldsymbol{r}_n^{(l+1)} = \boldsymbol{r}_n^{(l)} + C \sum_{(n,m)\in\mathcal{E}} \left(\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)}\right) \phi\left(\boldsymbol{e}_{nm}^{(l+1)}\right) \tag{13.39}$$

$$\boldsymbol{z}_n^{(l+1)} = \mathrm{Aggregate}_{\mathrm{node}}\left(\{\boldsymbol{e}_{nm}^{(l+1)} : m \in \mathcal{N}(n)\}\right) \tag{13.40}$$

$$\boldsymbol{h}_n^{(l+1)} = \mathrm{Update}_{\mathrm{node}}\left(\boldsymbol{h}_n^{(l)}, \boldsymbol{z}_n^{(l+1)}\right). \tag{13.41}$$

其中 $\phi(\boldsymbol{e}_{nm}^{(l+1)})$ 是边嵌入的一般标量函数, 由神经网络表示, 系数 C 通常设置为求和项数目的倒数. ♣

注

(重构新增) 这四式是(13.29)～(13.31)的对称化版本, 改动集中在两处: 边更新多了一个输入 $\|\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)}\|^2$, 节点多了一个坐标变量 $\boldsymbol{r}_n^{(l)}$ 并把它按(13.39)更新. 两处改动都是为了让坐标只以“相对量”的形式进入计算.

把坐标嵌入与节点特征嵌入分开处理是一个关键设计: $\boldsymbol{h}_n$ 是标量型的语义特征 (原子类型等), 要求不变; $\boldsymbol{r}_n$ 是几何量 (位置), 要求等变. 若把二者混在同一个向量里, 就无法对它们施加不同的变换规则.

系数 C 取求和项数目的倒数, 是为了让坐标更新量的量级不随邻居个数增长; ϕ 输出标量则保证 $(\boldsymbol{r}_n - \boldsymbol{r}_m)\phi$ 仍是与 $\boldsymbol{r}$ 同型的向量, 可以相加.

注意, $\|\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)}\|^2$ 表示坐标 $\boldsymbol{r}_n^{(l)}$ 和 $\boldsymbol{r}_m^{(l)}$ 之间的平方距离, 它并不依赖于平移、旋转或反射. 同样, 坐标 $\boldsymbol{r}_n^{(l)}$ 可以通过相对差异 $(\boldsymbol{r}_n^{(l)} - \boldsymbol{r}_m^{(l)})$ 的线性组合进行更新. 因此在这种变换下, 消息传递方程 [(13.38)、(13.40)和(13.41)] 是不变的, 而(13.39)给出的坐标嵌入是等变的 (见练习 13.10).

命题 13.6 (几何消息传递的平移—旋转不变性与坐标等变性)

设坐标系统作平移与正交变换, 即对每个节点 n 有 $\boldsymbol{r}'_n = \boldsymbol{R}\boldsymbol{r}_n + \boldsymbol{c}$, 其中 $\boldsymbol{c} \in \mathbb{R}^3$ 固定,$\boldsymbol{R}$ 满足 $\boldsymbol{R}\boldsymbol{R}^\top = \boldsymbol{R}^\top\boldsymbol{R} = \boldsymbol{I}$. 设第 l 层的坐标满足 $\boldsymbol{r}'^{(l)}_n = \boldsymbol{R}\boldsymbol{r}^{(l)}_n + \boldsymbol{c}$, 且 $(\boldsymbol{e}^{(l)}, \boldsymbol{h}^{(l)})$ 在该变换下保持不变. 则按(13.38)〜(13.41)计算得到的第 $l+1$ 层满足:$\boldsymbol{e}'^{(l+1)}_{nm} = \boldsymbol{e}^{(l+1)}_{nm}$、$\boldsymbol{h}'^{(l+1)}_n = \boldsymbol{h}^{(l+1)}_n$(不变性), 以及 $\boldsymbol{r}'^{(l+1)}_n = \boldsymbol{R}\boldsymbol{r}^{(l+1)}_n + \boldsymbol{c}$(等变性).

证明

(重构新增) 先看边更新. 由 $\boldsymbol{R}^\top\boldsymbol{R} = \boldsymbol{I}$,

$$\begin{aligned}\|\boldsymbol{r}'^{(l)}_n - \boldsymbol{r}'^{(l)}_m\|^2 &= \|(\boldsymbol{R}\boldsymbol{r}^{(l)}_n + \boldsymbol{c}) - (\boldsymbol{R}\boldsymbol{r}^{(l)}_m + \boldsymbol{c})\|^2 = \|\boldsymbol{R}(\boldsymbol{r}^{(l)}_n - \boldsymbol{r}^{(l)}_m)\|^2 \\ &= (\boldsymbol{r}^{(l)}_n - \boldsymbol{r}^{(l)}_m)^\top\boldsymbol{R}^\top\boldsymbol{R}(\boldsymbol{r}^{(l)}_n - \boldsymbol{r}^{(l)}_m) = \|\boldsymbol{r}^{(l)}_n - \boldsymbol{r}^{(l)}_m\|^2,\end{aligned}$$

其中平移向量 $\boldsymbol{c}$ 在相减时抵消, 正交性 $\boldsymbol{R}^\top\boldsymbol{R} = \boldsymbol{I}$ 使长度不变. 于是(13.38)的四个输入中有三个不变、第四个 (平方距离) 也不变, 故 $\boldsymbol{e}'^{(l+1)}_{nm} = \boldsymbol{e}^{(l+1)}_{nm}$. 再看坐标更新. 由于 ϕ 作用在不变的边嵌入上,$\phi(\boldsymbol{e}'^{(l+1)}_{nm}) = \phi(\boldsymbol{e}^{(l+1)}_{nm})$; 又 $\boldsymbol{r}'^{(l)}_n - \boldsymbol{r}'^{(l)}_m = \boldsymbol{R}(\boldsymbol{r}^{(l)}_n - \boldsymbol{r}^{(l)}_m)$, 代入(13.39)得

$$\begin{aligned}\boldsymbol{r}'^{(l+1)}_n &= \boldsymbol{R}\boldsymbol{r}^{(l)}_n + \boldsymbol{c} + C\sum_{(n,m)\in\mathcal{E}} \boldsymbol{R}\left(\boldsymbol{r}^{(l)}_n - \boldsymbol{r}^{(l)}_m\right)\phi\left(\boldsymbol{e}^{(l+1)}_{nm}\right) \\ &= \boldsymbol{R}\left[\boldsymbol{r}^{(l)}_n + C\sum_{(n,m)\in\mathcal{E}}\left(\boldsymbol{r}^{(l)}_n - \boldsymbol{r}^{(l)}_m\right)\phi\left(\boldsymbol{e}^{(l+1)}_{nm}\right)\right] + \boldsymbol{c} = \boldsymbol{R}\boldsymbol{r}^{(l+1)}_n + \boldsymbol{c},\end{aligned}$$

第二个等号只是把 $\boldsymbol{R}$ 与 $\boldsymbol{c}$ 提到求和号外 ($\boldsymbol{R}$ 与 n, m 无关), 这正是等变性.

最后看节点更新.$\boldsymbol{z}'^{(l+1)}_n$ 由(13.40)对不变的 $\boldsymbol{e}'^{(l+1)}_{nm}$ 聚合得到, 故 $\boldsymbol{z}'^{(l+1)}_n = \boldsymbol{z}^{(l+1)}_n$; 再由(13.41), 其两个输入 $\boldsymbol{h}^{(l)}_n$ 与 $\boldsymbol{z}^{(l+1)}_n$ 都不变, 故 $\boldsymbol{h}'^{(l+1)}_n = \boldsymbol{h}^{(l+1)}_n$. 对层数归纳即得全部结论. 证毕.

□

注

(重构新增) 证明的关键只有两个等式: 平移在相减时抵消, 正交矩阵保持内积. 这两条恰好刻画了“坐标系统的改变不应影响物理量”这一直觉, 也说明为什么(13.38)必须用平方距离而不是坐标本身——若把 $\boldsymbol{r}^{(l)}_n$ 直接送进边更新, 平移会改变结果, 不变性立即失效.

$\boldsymbol{R}$ 只要求正交, 不要求 $\det\boldsymbol{R} = 1$, 因此镜像反射 ($\det\boldsymbol{R} = -1$) 也被包含在内, 这与原书“旋转和/或镜像反射”的说法一致. 如果分子属性本身是手性的 (例如旋光性), 那么反射下的不变性假设就不成立, 此时需要把 $\det\boldsymbol{R}$ 作为一个额外的输入或改用只对 SO(3) 不变的设计.

命题的归纳起点是 $\boldsymbol{r}^{(0)}_n$ 由已知原子坐标初始化, 因此初值满足 $\boldsymbol{r}'^{(0)}_n = \boldsymbol{R}\boldsymbol{r}^{(0)}_n + \boldsymbol{c}$; 若坐标是从数据中学出来的, 这个前提需要另行验证.

原书把这一验证留作练习 13.10, 本命题与证明是与之配套的完整版本, 二者结论一致.

我们已经看到了结构化数据中许多对称性的例子, 从图像中物体的平移和图中节点顺序的排列, 到三维空间中分子的旋转和平移. 将这些对称性捕获在深度神经网络的结构中是一种强大的归纳偏置形式, 也是进行丰富的几何深度学习 (geometric deep learning) 研究的基础 (Bronstein et al., 2017; Bronstein et al., 2021).

13.4 习题

✍ **练习 13.1**★ 证明: 对应于图 13.2中两种节点排序选择的置换 $(A, B, C, D, E) \to (C, E, A, D, B)$, 可以表示为(13.5)定义的形式, 其中置换矩阵由(13.1)给出.

✍ **练习 13.2**★★ 证明: 对应于图 13.2中两种节点排序选择的图 $\mathcal{G}$ 中与每个节点相连的边数可以由矩阵 $\boldsymbol{A}^2$ 中相应的对角元素给出, 其中 $\boldsymbol{A}$ 是邻接矩阵.

✍ **练习 13.3**⋆ 由下式给出的邻接矩阵, 画出其对应的图:

$$\boldsymbol{A}=\begin{pmatrix}0&1&1&0&1\\1&0&1&1&1\\1&1&0&1&0\\0&1&1&0&0\\1&1&0&0&0\end{pmatrix}. \tag{13.42}$$

✍ **练习 13.4**⋆⋆ 证明在数据矩阵 $\boldsymbol{X}$ 前乘以(13.3)定义的置换矩阵 $\boldsymbol{P}$ 的效果是创建一个新的数据矩阵 $\widetilde{\boldsymbol{X}}$ [由(13.4)给出], 其行已根据置换函数 $\pi(\cdot)$ 进行置换.

✍ **练习 13.5**⋆⋆ 证明转换后的邻接矩阵 $\widetilde{\boldsymbol{A}}$, 它由(13.5)定义, 其中 $\boldsymbol{P}$ 由(13.3)定义, 使得新矩阵的行和列都根据置换函数 $\pi(\cdot)$ 相对于原始邻接矩阵 $\boldsymbol{A}$ 进行置换.

✍ **练习 13.6**⋆⋆ 将(13.16)所示的更新方程写成使用矩阵的图级别方程. 为了保持符号简洁, 省略层索引 l. 为此, 首先将节点嵌入向量 $\boldsymbol{h}_n^{(l)}$ 收集到一个矩阵 $\boldsymbol{H}$ 中, 其中的行 n 由 $\boldsymbol{h}_n^{\mathsf{T}}$ 给出. 然后证明下式

$$\boldsymbol{z}_n=\sum_{m\in\mathcal{N}(n)}\boldsymbol{h}_m \tag{13.43}$$

给出的邻域聚合向量 $\boldsymbol{z}_n$ 可以以矩阵形式写成 $\boldsymbol{Z}=\boldsymbol{AH}$,$\boldsymbol{Z}$ 是一个 $N\times D$ 大小的矩阵, 其中的行 n 由 $\boldsymbol{z}_n^{\mathsf{T}}$ 给出,$\boldsymbol{A}$ 是邻接矩阵. 最后证明(13.16)中非线性激活函数的参数可以以矩阵形式写成

$$\boldsymbol{AHW}_{\text{neigh}}+\boldsymbol{HW}_{\text{self}}+\boldsymbol{1}_D\boldsymbol{b}^{\mathsf{T}}, \tag{13.44}$$

其中 $\boldsymbol{1}_D$ 是一个 D 维的列向量, 其中的所有元素都是 1.

注

(重构新增) 上面的推导与题面(13.44)有两处记号差异, 这里说明清楚, 以免读者误以为推导有误.

其一是转置.(13.16)中权重以“左乘列向量”的形式出现 ($\boldsymbol{W}_{\text{self}}\boldsymbol{h}_n$), 而把 N 个节点堆成 $\boldsymbol{H}$ 之后, 同一个线性映射必须写成右乘 $\boldsymbol{W}^{\mathsf{T}}$, 否则维数不匹配. 题面(13.44)省略了转置, 含义是: 其中的 $\boldsymbol{W}_{\text{self}}$、$\boldsymbol{W}_{\text{neigh}}$ 取(13.16)中矩阵的转置 (等价地, 把 $\boldsymbol{h}_n$ 当作行向量处理). 这不是数学错误, 而是两套约定的混用; 实现时只需固定一套.

其二是维数.$\boldsymbol{AHW}_{\text{neigh}}$ 与 $\boldsymbol{HW}_{\text{self}}$ 都是 $N\times D'$ 矩阵, 而 $\boldsymbol{b}^{\mathsf{T}}$ 是 $1\times D'$ 的行向量, 故左乘的全 1 向量必须是 N 维 (即 $\boldsymbol{1}_N$) 才能得到 $N\times D'$ 的矩阵; 题面说“$\boldsymbol{1}_D$ 是一个 D 维的列向量”相应地应理解为 N 维, 其中 N 是节点数. 若坚持用 D 维向量, 则该乘积为 $D\times D'$, 与其余两项无法相加.

除这两处约定外, 题面的结论是正确的: 把逐节点更新写成矩阵形式后, 一层消息传递的计算代价是一次稀疏矩阵乘法加两次稠密矩阵乘法, 与节点数成线性关系, 这正是图神经网络能用于大规模图的根本原因.

✍ **练习 13.7**⋆⋆ 利用深度图卷积网络第 l 层的等变性 [(13.19)] 和节点变量的置换性质 [(13.4)], 证明(13.18)定义的完整深度图卷积网络也是等变的.

✍ **练习 13.8**⋆⋆ 证明在(13.24)定义的聚合函数中 [注意力权重由(13.28)给出], 该聚合函数在图中节点重新排序下是等变的.

✍ **练习 13.9**⋆ 证明一个图注意力网络 (其中图是全连接的, 即每对节点之间都有一条边), 等同于标准的 Transformer 架构.

✍ **练习 13.10**⋆⋆ 当坐标系统被平移时, 对坐标系统定义的物体的位置向量使用

$$\boldsymbol{r}'=\boldsymbol{r}+\boldsymbol{c} \tag{13.45}$$

进行转换, 其中 $\boldsymbol{c}$ 是描述平移的固定向量. 类似地, 如果坐标系统发生旋转和/或镜像反射, 则对坐标系统定义的物体的位置向量使用

$$\boldsymbol{r}'=\boldsymbol{Rr} \tag{13.46}$$

进行转换, 其中 $\boldsymbol{R}$ 是一个正交矩阵, 它的逆由其转置给出, 所以

$$\boldsymbol{RR}^{\mathsf{T}}=\boldsymbol{R}^{\mathsf{T}}\boldsymbol{R}=\boldsymbol{I}. \tag{13.47}$$

利用这些属性, 证明在平移、旋转和反射下, 消息传递方程 [(13.38)、(13.40)和(13.41)] 是不变的, 并且(13.39)给出的坐标嵌入向量是等变的.

第 14 章 采样

在深度学习领域, 我们经常需要从概率分布 $p(z)$ 中生成 z 的样本. 这里的 z 可能是一个标量, 分布可能是单变量 Gauss 分布; 或者 z 可能是一幅高分辨率的图像, 而 $p(z)$ 可能是由深度神经网络定义的生成式模型. 对于许多简单的分布, 我们可以使用数值技术直接生成合适的样本; 而对于更复杂的分布, 包括那些隐式定义的分布, 我们则需要使用更复杂的方法.

定义 14.1 (采样与样本)

从概率分布 $p(z)$ 中生成取值 z 的过程称为采样 (sampling), 也称 Monte Carlo 采样 (Monte Carlo sampling); 其中每一个被实例化的取值称为一个样本 (sample). ♣

注

(重构新增) 本定义有两点约定值得说明. 第一, 本书把单个被实例化的取值称为样本, 而传统统计学常把一组观测称为一个样本; 采用前一种约定后,$z^{(l)}$ 这样的记号可以直接读作「第 l 个样本」, 本章的求和与平均才不会产生歧义. 第二, 定义只要求「生成取值」这一操作, 并未要求 $p(z)$ 有解析表达式: 这为本章后面处理未归一化分布 $p(z) = \tilde{p}(z)/Z_p$(其中 Z_p 未知) 留下空间, 因为从 p 采样并不需要知道 Z_p. 此外, 计算机给出的样本实际上是伪随机数, 它只保证通过统计检验意义上的随机性, 一切基于样本的结论都带有统计误差, 而小节 14.1.1将用方差来度量这一误差.

在本章中, 我们将重点关注与深度学习最相关的采样方面的内容. 关于 Monte Carlo 采样方法的更通用的介绍, 可以参考 Gilks, Richardson, and Spiegelhalter (1996) 以及 Robert and Casella (1999).

14.1 基本采样

在本节中, 我们将介绍一系列相对简单的采样策略, 用于从给定分布中生成随机样本. 由于样本是由计算机算法生成的, 它们是伪随机 (pseudo-random) 的, 也就是说, 它们将使用确定性算法来运算, 但仍必须通过适当的随机性测试. 这里我们假设已经存在一个确定性算法, 它能生成在 $(0, 1)$ 区间上均匀分布的伪随机数, 事实上, 大多数软件平台都内置了这样的功能.

14.1.1 期望

虽然在某些应用中我们可以直接使用样本本身, 但在更多情况下, 我们的目标是计算某个概率分布的期望. 假设我们希望找到函数 $f(z)$ 关于概率分布 $p(z)$ 的期望. 在这里,z 的分量可能包括离散变量、连续变量或它们两者的组合. 对于连续变量, 期望定义为

$$\mathbb{E}[f] = \int f(z)\, p(z)\, \mathrm{d}z, \tag{14.1}$$

其中, 积分在离散变量的情况下被求和替代.图 14.1展示了单个连续变量的情况.

我们在此假设这样的期望过于复杂, 无法使用解析方法准确计算, 这时候就需要用采样方法近似计算. 采样方法的基本思路是获得从分布 $p(z)$ 中独立地抽取的一组样本 $z^{(l)}$(其中 $l = 1, \cdots, L$), 使得期望(14.1)可以通过有限和的形式来近似:

$$\overline{f} = \frac{1}{L} \sum_{l=1}^{L} f\left(z^{(l)}\right). \tag{14.2}$$

如果样本 $z^{(l)}$ 是从概率分布 $p(z)$ 中抽取的, 则 $\mathbb{E}[\overline{f}] = \mathbb{E}[f(z)]$, 因此估计量 $\overline{f}$ 具有正确的均值 (见练习 14.1).

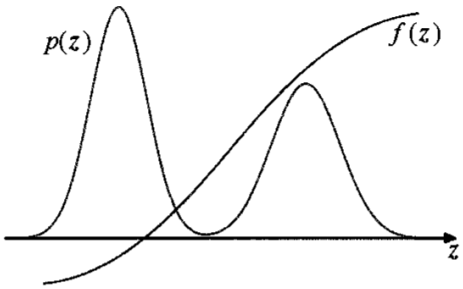


图 14.1: 一个函数 $f(z)$, 我们需要计算它关于概率分布 $p(z)$ 的期望 (示意图)

命题 14.1 (Monte Carlo 估计的无偏性)

设 $z^{(1)},\cdots,z^{(L)}$ 的边缘分布均为 $p(z)$, 且 $\mathbb{E}\left[|f(z)|\right]<\infty$. 则由(14.2)定义的估计量 $\overline{f}$ 是 $\mathbb{E}[f(z)]$ 的无偏估计, 即 $\mathbb{E}[\overline{f}]=\mathbb{E}[f(z)]$.

证明

由(14.2)与期望的线性性,

$$\mathbb{E}\left[\overline{f}\right]=\mathbb{E}\left[\frac{1}{L}\sum_{l=1}^{L}f\left(z^{(l)}\right)\right]=\frac{1}{L}\sum_{l=1}^{L}\mathbb{E}\left[f\left(z^{(l)}\right)\right]=\frac{1}{L}\sum_{l=1}^{L}\mathbb{E}[f(z)]=\mathbb{E}[f(z)].$$

上面的推导只用到每个 $z^{(l)}$ 的边缘分布为 $p(z)$, 并未用到样本之间的独立性.

□

注

(重构新增) 本命题说明了 Monte Carlo 估计「至少不会系统性地偏离真值」, 这是把有限和当作积分使用的全部依据. 条件 $\mathbb{E}[|f(z)|]<\infty$ 不可省: 若该期望不存在 (例如重尾情形), 则 $\mathbb{E}[\overline{f}]$ 本身没有定义, 无偏性也就无从谈起; 此时中心极限定理同样失效, 样本均值不再收敛. 值得注意的是, 命题只要求各样本的边缘分布相同而不要求独立, 因此它对 Markov 链产生的相关样本仍然成立 (见节 14.2); 独立性只在控制方差时起作用, 见命题 14.2. 此外,$\overline{f}$ 的分子是 L 项之和, 故无偏性依赖于权重 $1/L$ 的精确取值; 若改用依赖样本的自适应权重, 无偏性需要另行证明.

我们也可以把它改成

$$\mathbb{E}[f(z)]\simeq\frac{1}{L}\sum_{l=1}^{L}f\left(z^{(l)}\right),\tag{14.3}$$

其中符号 $\simeq$ 表示右侧是左侧的无偏估计量, 也就是说, 当对噪声分布进行平均时, 两侧是相等的 (见练习 14.1). 估计量(14.2)的方差由下式给出:

$$\operatorname{var}\left[\overline{f}\right]=\frac{1}{L}\mathbb{E}\left[(f(z)-\mathbb{E}[f(z)])^2\right].\tag{14.4}$$

命题 14.2 (Monte Carlo 估计的方差)

在命题 14.1的条件下, 若进一步假设 $z^{(1)},\cdots,z^{(L)}$ 相互独立, 且 $\operatorname{var}[f(z)]<\infty$, 则(14.4)成立, 即

$$\operatorname{var}\left[\overline{f}\right]=\frac{1}{L}\operatorname{var}[f(z)].\tag{14.5}$$

证明

记 $\sigma^2=\operatorname{var}[f(z)]=\mathbb{E}\left[(f(z)-\mathbb{E}[f(z)])^2\right]$. 由独立性, 各 $f(z^{(l)})$ 相互独立且方差同为 σ^2, 于是

$$\operatorname{var}\left[\overline{f}\right]=\operatorname{var}\left[\frac{1}{L}\sum_{l=1}^{L}f\left(z^{(l)}\right)\right]=\frac{1}{L^2}\sum_{l=1}^{L}\operatorname{var}\left[f\left(z^{(l)}\right)\right]=\frac{1}{L^2}\cdot L\sigma^2=\frac{\sigma^2}{L},$$

这正是(14.4)与(14.5)的右端.

□

注

(重构新增) 本命题给出了 Monte Carlo 方法最核心的定量结论: 误差的平方按 $1/L$ 衰减, 即标准差按 $1/\sqrt{L}$ 衰减, 而且这一速率与 z 的维数无关——这正是采样方法在高维问题中优于网格求和的原因. 两个条件各有作用: 独立性保证了交叉项 $\operatorname{cov}[f(\boldsymbol{z}^{(l)}), f(\boldsymbol{z}^{(l')})]$ 全部为零, 若样本相关 (例如 MCMC 的相邻状态), 则

$$\operatorname{var}\left[\overline{f}\right] = \frac{\sigma^2}{L}\left(1 + 2\sum_{k\geqslant 1}\rho_k\right),$$

其中 ρ_k 为滞后 k 的自相关系数, 有效样本量按因子 $\left(1 + 2\sum_{k}\rho_k\right)$ 缩小, 这就是节 14.2中「丢弃大部分序列、只保留每第 M 个样本」做法的依据; 有限方差则保证 σ^2 存在, 否则 $1/L$ 的速率不再成立. 最后提醒, 方差小并不等于估计准确: 若 f 在 p 的高密度区取值很小而在低密度区取值很大, 则 σ^2 会很大, 仍需要很大的 L 才能达到所需精度.

该方差是函数 $f(\boldsymbol{z})$ 在概率分布 $p(\boldsymbol{z})$ 下的方差. 注意, 这种方差会随着 L 的增加而线性减小, 并且不依赖于 $\boldsymbol{z}$ 的维度, 从原则上讲, 使用相对较小的样本集合 $\{\boldsymbol{z}^{(l)}\}$ 就有可能获得高精度. 然而, 问题在于样本集合 $\{\boldsymbol{z}^{(l)}\}$ 中的样本可能不是独立的, 所以有效样本量可能比表面上的样本量要小得多. 另外, 在图 14.1中, 如果 $f(\boldsymbol{z})$ 在 $p(\boldsymbol{z})$ 较大的区域很小, 在 $p(\boldsymbol{z})$ 较小的地方反而很大, 则期望可能被小概率区域主导, 这意味着为了达到足够的准确度, 可能需要较大的样本量.

14.1.2 标准分布

现在假设我们已经有了一个均匀分布随机数的来源, 我们讨论如何从简单的非均匀分布中生成随机数. 假设 z 在区间 $(0, 1)$ 上均匀分布, 并且我们使用某个函数 $g(\cdot)$ 来变换 z 的值, 使得 $y = g(z)$.y 的分布将由下式决定:

$$p(y) = p(z)\left|\frac{\mathrm{d}z}{\mathrm{d}y}\right|, \tag{14.6}$$

在这里,$p(z) = 1$. 我们的目标是选择合适的函数 $g(\cdot)$, 让生成的 y 具有某个特定的期望分布 $p(y)$. 对(14.6)进行积分, 可以得到

$$z = \int_{-\infty}^{y} p(\hat{y})\,\mathrm{d}\hat{y} \equiv h(y), \tag{14.7}$$

这是 $p(y)$ 的不定积分.$y = h^{-1}(z)$, 所以我们需要使用一个变换函数, 该函数是目标分布不定积分的反函数, 如图 14.2所示.

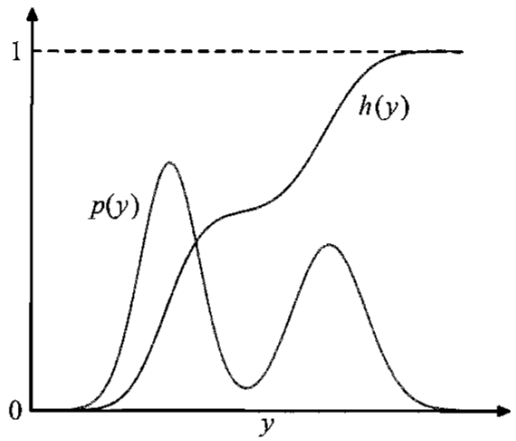


图 14.2: 生成非均匀分布随机数的变换方法的几何解释.$h(y)$ 是期望的目标分布 $p(y)$ 的不定积分. 如果用 $y = h^{-1}(z)$ 变换均匀分布的随机变量 z, 那么得到的变量 y 将服从 $p(y)$ 分布

定义 14.2 (逆变换采样)

设目标分布 $p(y)$ 的累积分布函数为 $h(y)=\int_{-\infty}^{y} p(\hat{y})\,\mathrm{d}\hat{y}$, 且 h 严格单调递增. 若 z 服从区间 $(0,1)$ 上的均匀分布, 则称由

$$y=h^{-1}(z)$$

生成样本的方法为逆变换采样 (inverse transform sampling).

♣

注

(重构新增) 逆变换采样的动机是: 均匀分布是最容易生成的分布, 而累积分布函数 h 恰好把任意分布「压平」成均匀分布——$h(y)\sim\mathcal{U}(0,1)$, 于是把这一过程反过来, 就能把均匀随机数「拉伸」成目标分布. 定义中的两个条件都不可省: 单调性保证 h^{-1} 存在且唯一 (若 p 有零密度区间,h 只能做到单调不减, 此时需在平台上取广义逆 $h^{-1}(z)=\inf\{y:h(y)\geqslant z\}$, 结论仍然成立);$(0,1)$ 上的均匀性保证 $P(z\leqslant h(y))=h(y)$. 定义并不要求 p 归一化常数已知, 但要求能对 p 积分并求反函数, 这一要求把方法限制在少数解析可积的分布上——这正是本节随后转向拒绝采样与重要性采样的原因.

命题 14.3 (逆变换采样的正确性)

设 $p(y)$ 为概率密度,$h(y)=\int_{-\infty}^{y} p(\hat{y})\,\mathrm{d}\hat{y}$ 严格单调递增且连续,$z\sim\mathcal{U}(0,1)$,$y=h^{-1}(z)$. 则 y 的密度函数恰为 $p(y)$.

♠

证明

任取 $y_0\in\mathbb{R}$. 由 h 严格单调递增,$h^{-1}(z)\leqslant y_0$ 与 $z\leqslant h(y_0)$ 等价, 于是

$$P(y\leqslant y_0)=P\left(h^{-1}(z)\leqslant y_0\right)=P\left(z\leqslant h(y_0)\right)=h(y_0)=\int_{-\infty}^{y_0} p(\hat{y})\,\mathrm{d}\hat{y},$$

这里用到 z 在 $(0,1)$ 上均匀分布且 $h(y_0)\in(0,1)$. 上式右端关于 y_0 可微且导数为 $p(y_0)$, 故 y 的累积分布函数是 p 的不定积分,y 的密度为 $p(y)$.

□

注

(重构新增) 证明的关键一步是把「y 的分布」转化为「z 的分布」: 由于 h^{-1} 单调, 事件 $\{h^{-1}(z)\leqslant y_0\}$ 与 $\{z\leqslant h(y_0)\}$ 完全相同, 而后者在均匀分布下的概率就是区间长度 $h(y_0)$. 这解释了为什么(14.7)中的下限取 $-\infty$:h 必须是完整的累积分布函数, 才能把 $(0,1)$ 映满. 若只取不定积分的某一个原函数 $H(y)+C$, 则 z 的取值范围不再是 $(0,1)$, 必须先用 H 的上下确界做仿射归一并相应修改反函数. 条件「h 连续」保证 p 没有原子, 否则 h 有跳跃,h^{-1} 在跳跃区间上不唯一, 但按广义逆取值时结论不变.

以指数分布为例

$$p(y)=\lambda\exp(-\lambda y),\qquad 0\leqslant y<\infty. \tag{14.8}$$

在这种情况下,(14.7)中的积分的下限是 0, 因此 $h(y)=1-\exp(-\lambda y)$. 如果我们使用 $y=-\lambda^{-1}\ln(1-z)$ 变换均匀分布的随机变量 z, 那么 y 将服从指数分布.

例题 14.1 指数分布的逆变换采样 设 $z\sim\mathcal{U}(0,1)$, 令

$$y=-\frac{1}{\lambda}\ln(1-z),\qquad \lambda>0.$$

验证 y 服从(14.8)给出的指数分布.

解

(重构新增) 由(14.8),$h(y)=\int_0^y \lambda e^{-\lambda\hat{y}}\,\mathrm{d}\hat{y}=1-e^{-\lambda y}$, 它在 $[0,\infty)$ 上严格递增, 值域为 $[0,1)$. 解 $z=1-e^{-\lambda y}$ 得 $y=-\lambda^{-1}\ln(1-z)$, 与题设一致; 又 $1-z$ 与 z 同为 $(0,1)$ 上的均匀随机变量, 故也可写 $y=-\lambda^{-1}\ln z$. 由命题 14.3,y 的密度为 $p(y)=\lambda e^{-\lambda y}(y\geqslant 0)$. 这里 h 的值域为 $[0,1)$ 而 $z\in(0,1)$, 端点 $z=0$ 的概率为零, 不影响分布.

□

可以应用变量变换方法的另一种分布是 Cauchy 分布:

$$p(y)=\frac{1}{\pi}\frac{1}{1+y^2}. \tag{14.9}$$

在这种情况下, 不定积分的反函数可以用正切函数表示 (见练习 14.4).

例题 14.2 Cauchy 分布的逆变换采样 设 $z\sim\mathcal{U}(0,1)$, 求变换 $y=f(z)$, 使 y 服从(14.9)给出的 Cauchy 分布.

解

(重构新增) 由(14.9),

$$h(y)=\int_{-\infty}^{y}\frac{1}{\pi}\frac{1}{1+\hat{y}^2}\,\mathrm{d}\hat{y}=\frac{1}{\pi}\arctan y+\frac{1}{2},$$

其值域为 $(0,1)$ 且严格递增. 令 $z=\frac{1}{\pi}\arctan y+\frac{1}{2}$, 解得 $y=\tan\left(\pi\left(z-\frac{1}{2}\right)\right)$. 由命题 14.3, 该 y 的密度为(14.9).Cauchy 分布没有有限的期望与方差, 这提醒我们命题 14.2中的有限方差条件是实质性的.

□

将该方法推广到多元变量的情形, 需要引入变量变换的 Jacobi 行列式, 于是有

$$p(y_1,\cdots,y_M)=p(z_1,\cdots,z_M)\left|\frac{\partial(z_1,\cdots,z_M)}{\partial(y_1,\cdots,y_M)}\right|. \tag{14.10}$$

作为变换方法的最后一个例子, 我们介绍用于生成 Gauss 分布样本的 Box-Muller 方法. 首先, 我们生成一对均匀分布的随机数 $z_1,z_2\in(-1,1)$, 这可以通过对在区间 $(0,1)$ 上均匀分布的变量 z 进行 $z\to 2z-1$ 线性变换来实现. 接下来丢弃所有不满足条件 $z_1^2+z_2^2\leqslant 1$ 的数对, 这将产生一个在单位圆内均匀分布的点集, 且其概率密度为 $p(z_1,z_2)=1/\pi$, 如图 14.3所示.

最后, 对于每对 z_1,z_2, 我们计算量

$$y_1=z_1\left(\frac{-2\ln r^2}{r^2}\right)^{1/2}, \tag{14.11}$$

$$y_2=z_2\left(\frac{-2\ln r^2}{r^2}\right)^{1/2}, \tag{14.12}$$

其中 $r^2=z_1^2+z_2^2$(见练习 14.5).y_1 和 y_2 的联合概率密度函数为

$$\begin{aligned}p(y_1,y_2)&=p(z_1,z_2)\left|\frac{\partial(z_1,z_2)}{\partial(y_1,y_2)}\right|\\&=\left[\frac{1}{\sqrt{2\pi}}\exp\left(-y_1^2/2\right)\right]\left[\frac{1}{\sqrt{2\pi}}\exp\left(-y_2^2/2\right)\right].\end{aligned} \tag{14.13}$$

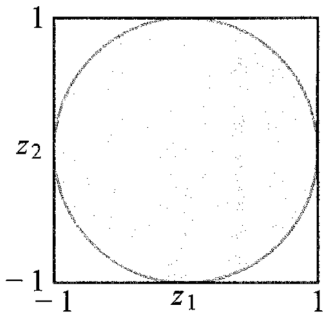


图 14.3: Box-Muller 方法用于生成 Gauss 分布的随机数, 为此, 首先在单位圆内生成均匀分布的样本

因此,y_1 和 y_2 是独立的, 并且每个变量都服从均值为零、方差为 1 的 Gauss 分布.

命题 14.4 (Box-Muller 方法的正确性)

设 (z_1, z_2) 在单位圆 $\{(z_1, z_2) : z_1^2 + z_2^2 \leqslant 1\}$ 内均匀分布,$r^2 = z_1^2 + z_2^2$,y_1, y_2 由(14.11)与(14.12)定义. 则 y_1, y_2 相互独立且均服从 $\mathcal{N}(0, 1)$, 即(14.13)成立.

证明

采用极坐标 $z_1 = r\cos\theta, z_2 = r\sin\theta$, 其中 $r \in [0, 1], \theta \in [0, 2\pi)$. 由 (z_1, z_2) 在单位圆上均匀分布 $(p(z_1, z_2) = 1/\pi)$ 及面积元 $\mathrm{d}z_1\mathrm{d}z_2 = r\,\mathrm{d}r\,\mathrm{d}\theta$,$(r, \theta)$ 的联合密度为

$$p(r, \theta) = \frac{r}{\pi}, \qquad 0 \leqslant r \leqslant 1, \quad 0 \leqslant \theta < 2\pi,$$

即 θ 在 $[0, 2\pi)$ 上均匀、r 的密度为 $2r$, 且二者独立. 由(14.11)与(14.12),$y_1 = z_1\varphi(r), y_2 = z_2\varphi(r)$, 其中 $\varphi(r) = \left(-2\ln r^2\right)^{1/2}/r$. 该变换是径向的: 方向角 θ 保持不变, 半径按

$$R \triangleq \left(y_1^2 + y_2^2\right)^{1/2} = r\,\varphi(r) = \left(-2\ln r^2\right)^{1/2}$$

映射, 故 $r^2 = \exp\left(-R^2/2\right)$, 即 $r = \exp\left(-R^2/4\right)$. 于是 (R, θ) 的联合密度为

$$p(R, \theta) = p(r, \theta)\left|\frac{\mathrm{d}r}{\mathrm{d}R}\right| = \frac{r}{\pi}\cdot\frac{R}{2}\exp\left(-\frac{R^2}{4}\right) = \frac{R}{2\pi}\exp\left(-\frac{R^2}{2}\right),$$

其中用到 $|\mathrm{d}r/\mathrm{d}R| = \dfrac{R}{2}\exp\left(-R^2/4\right)$. 最后换回直角坐标: 由 $\mathrm{d}y_1\mathrm{d}y_2 = R\,\mathrm{d}R\,\mathrm{d}\theta$,

$$p(y_1, y_2) = \frac{p(R, \theta)}{R} = \frac{1}{2\pi}\exp\left(-\frac{y_1^2 + y_2^2}{2}\right) = \frac{1}{\sqrt{2\pi}}e^{-y_1^2/2}\cdot\frac{1}{\sqrt{2\pi}}e^{-y_2^2/2},$$

它关于 y_1、y_2 分离, 故二者独立且均为标准 Gauss 分布, 这正是(14.13).

□

注

(重构新增)Box-Muller 方法绕开了「对 Gauss 累积分布函数求反函数」这一障碍:Gauss 的累积分布没有初等闭式, 但极坐标下的 r^2 服从指数分布、θ 服从均匀分布, 而这两者都可以用逆变换法生成, 于是先产生 (r, θ) 再变回直角坐标即可.(14.11)中的因子 $\left(-2\ln r^2\right)^{1/2}/r$ 正是把半径 r 拉到「Gauss 半径」的伸缩量, 它不是常数, 故不能把 y_1, y_2 看成 z_1, z_2 的线性变换,Jacobi 行列式必须按二维变换整体计算. 丢弃单位圆外的点对是必要的: 若在正方形 $(-1, 1)^2$ 上均匀取点而不筛选, 则 r 的密度不再是 $2r$, 得到的将不是标准 Gauss 分布.

如果 y 服从均值为零、方差为单位方差的 Gauss 分布, 那么 $\sigma y + \mu$ 将服从均值为 μ、方差为 σ^2 的 Gauss 分布. 为了生成具有多元 Gauss 分布的向量值变量, 假设多元 Gauss 分布的均值为 $\boldsymbol{\mu}$、协方差矩阵为 $\boldsymbol{\Sigma}$, 我们可以利用 Cholesky 分解, 其形式为 $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$, 其中 $\boldsymbol{L}$ 是一个下三角矩阵 (Deisenroth, Faisal, and Ong, 2020). 然后, 如果 $\boldsymbol{z}$ 是一个随机向量, 其分量是独立的且具有零均值、单位方差的 Gauss 分布 (见练习 14.6), 那么 $\boldsymbol{y} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{z}$ 将具有均值为 $\boldsymbol{\mu}$、协方差矩阵为 $\boldsymbol{\Sigma}$ 的 Gauss 分布.

命题 14.5 (多元 Gauss 分布的 Cholesky 采样)

设 $\boldsymbol{\Sigma} \in \mathbb{R}^{D\times D}$ 对称正定,$\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$ 为其 Cholesky 分解 ($\boldsymbol{L}$ 为对角元为正的下三角矩阵),$\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. 则 $\boldsymbol{y} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{z}$ 服从 $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

证明

由期望的线性性,$\mathbb{E}[\boldsymbol{y}] = \boldsymbol{\mu} + \boldsymbol{L}\mathbb{E}[\boldsymbol{z}] = \boldsymbol{\mu}$. 由 $\mathrm{cov}[\boldsymbol{z}] = \boldsymbol{I}$ 及协方差在仿射变换下的变换规则,

$$\mathrm{cov}[\boldsymbol{y}] = \boldsymbol{L}\,\mathrm{cov}[\boldsymbol{z}]\boldsymbol{L}^{\mathsf{T}} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}} = \boldsymbol{\Sigma}.$$

又 $\boldsymbol{y}$ 是 Gauss 向量 $\boldsymbol{z}$ 的仿射变换, 故仍为 Gauss 向量;Gauss 分布由均值与协方差唯一确定, 所以 $\boldsymbol{y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

□

注

(重构新增)Cholesky 分解的作用是把「协方差矩阵的相关结构」翻译成「下三角线性变换」:$\boldsymbol{L}$ 的对角元给出各方向的标准差, 非对角元给出方向之间的耦合, 因此只需 D 个独立的标准正态数就能生成具有完整协方差结构的样本, 计算代价为 $O(D^2)$. 存在性由 $\boldsymbol{\Sigma}$ 正定保证: 正定性等价于所有顺序主子式为正,Cholesky 分解的存在唯一性正是这一事实的构造性表述; 若 $\boldsymbol{\Sigma}$ 只是半正定 (例如某些方差为零), 则 $\boldsymbol{L}$ 仍存在但只有半正定对角元, 此时 $\boldsymbol{y}$ 落在低维仿射子空间上, 不再有密度. 另外,$\boldsymbol{z}$ 各分量必须独立: 若 $\boldsymbol{z}$ 分量相关,cov[$\boldsymbol{y}$] 将多出交叉项,$\boldsymbol{y}$ 的协方差不再是 $\boldsymbol{\Sigma}$.

显然, 变换技术的成功依赖于计算所需分布的不定积分与求得其反函数的能力. 这样的操作只对几种简单分布可行, 因此我们必须转向替代方法, 寻找更通用的策略. 这里我们考虑两种替代方法: 拒绝采样 (rejection sampling) 和重要性采样 (importance sampling). 尽管这两种方法受限于单变量分布, 并不能直接应用于多维的复杂问题, 但它们是更通用策略的重要组成部分.

14.1.3 拒绝采样

拒绝采样框架允许我们从相对复杂的分布中采样, 但有一定的限制条件. 首先考虑单变量分布, 然后考虑扩展到多维的情形.

假设我们想要从一个分布 $p(z)$ 中进行采样, 该分布不属于目前为止考虑过的简单标准分布, 并且直接从 $p(z)$ 中采样很困难. 此外, 假设我们能够轻易地计算任意给定值 z 的 $\widetilde{p}(z)$, 但计算结果需要乘以归一化常数 Z_p 的倒数才能得到真实的 $p(z)$, 即

$$p(z) = \frac{1}{Z_p}\widetilde{p}(z), \tag{14.14}$$

其中 $\widetilde{p}(z)$ 很容易计算, 但 Z_p 是未知的.

定义 14.3 (拒绝采样)

设目标分布 $p(z) = \widetilde{p}(z)/Z_p$, 其中 $\widetilde{p}$ 已知而 Z_p 未知. 设 $q(z)$ 是容易从中采样的提议分布 (proposal distribution), 常数 k 满足对一切 z 都有 $kq(z) \geqslant \widetilde{p}(z)$, 称 $kq(z)$ 为比较函数. 拒绝采样的每一步为:

(i) 从 $q(z)$ 中采样 z_0;

(ii) 从 $[0, kq(z_0)]$ 上的均匀分布中采样 u_0;

(iii) 若 $u_0 > \widetilde{p}(z_0)$ 则拒绝该样本, 否则保留 z_0.

♣

注

(重构新增) 拒绝采样的几何图像是「在比较函数曲线下均匀撒点、再按目标曲线的高度筛选」:(z_0, u_0) 在 kq 曲线下方均匀分布, 若 $u_0 \leqslant \widetilde{p}(z_0)$ 则它落在 $\widetilde{p}$ 曲线下方, 被保留的点在 $\widetilde{p}$ 下方仍然均匀, 故其横坐标服从 p. 这里有三个条件需要逐项确认. 其一,$kq(z) \geqslant \widetilde{p}(z)$ 对一切 z 成立, 否则 $\widetilde{p}$ 的某些区域无法被覆盖, 接受样本的分布会被截断; 其二,Z_p 不需要已知, 因为归一化在筛选后自动完成, 这是该方法相对逆变换采样的最大优势; 其三,q 必须有比 $\widetilde{p}$ 更重的尾部 (这正是存在有限 k 的含义), 否则 k 不存在. 常数 k 直接决定效率: 接受概率为 $\int \widetilde{p}\,\mathrm{d}z \Big/ \left(k\int q\,\mathrm{d}z\right) = Z_p/k$, 故 k 越接近最优值 $k^\star = \sup_z \widetilde{p}(z)/q(z)$ 越好.

要应用拒绝采样, 我们需要一些更简单的分布 $q(z)$, 它们有时称为提议分布, 我们可以容易地从中抽取样本. 接下来引入常数 k, 其值的选取需要满足对于所有的 z 都有 $kq(z) \geqslant \widetilde{p}(z)$. 函数 $kq(z)$ 称为比较函数,图 14.4针对一元分布的情况对它进行了说明. 拒绝采样的每一步都需要生成两个随机数: 先从分布 $q(z)$ 中生成一个随机数 z_0, 接着从区间 $\left[0, kq(z_0)\right]$ 上的均匀分布中生成另一个随机数 u_0. 这对随机数在函数 $kq(z)$ 的曲线下具有均匀分布. 最后, 如果 $u_0 > \widetilde{p}(z_0)$, 则拒绝该样本, 否则保留 z_0. 因此, 这对随机数如果位于图 14.4中的灰色阴影区域内, 则会被拒绝. 剩余的随机数对在 $\widetilde{p}(z)$ 曲线下具有均匀分布 (见练习 14.7), 因此相应的 z 值服从我们期望的分布 $p(z)$.

原始的 z 值是从分布 $q(z)$ 中生成的, 然后这些样本以 $\widetilde{p}(z)/kq(z)$ 的概率得以接受, 因此接受样本的概率由下

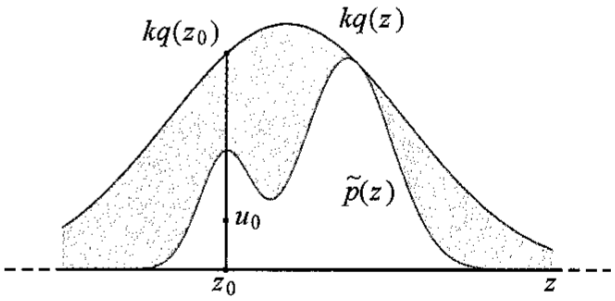


图 14.4: 在拒绝采样中, 样本是从一个简单的分布 $q(z)$ 中抽取的, 并且它们如果落在未归一化分布 $\tilde{p}(z)$ 和缩放分布 $kq(z)$ 之间的灰色区域内, 则被拒绝. 最终的样本是根据 $p(z)$ 分布的,$p(z)$ 是 $\tilde{p}(z)$ 的归一化版本

式给出:

$$\begin{aligned} p(\text{accept}) &= \int \min\left\{\frac{\tilde{p}(z)}{kq(z)}, 1\right\} q(z)\,\mathrm{d}z \\ &= \frac{1}{k}\int \tilde{p}(z)\,\mathrm{d}z. \end{aligned} \tag{14.15}$$

命题 14.6 (拒绝采样的正确性)

在定义 14.3的设定下, 设 $\tilde{p}$ 与 q 均为概率密度 (或均可积), $\int \tilde{p}\,\mathrm{d}z = Z_p \in (0,\infty)$, 且 $kq(z) \geqslant \tilde{p}(z)$ 对一切 z 成立. 则被接受样本的取值服从 $p(z) = \tilde{p}(z)/Z_p$, 且接受概率由(14.15)给出.

证明

设事件 A 表示样本被接受. 给定 $z_0 = z$ 时,u_0 在 $[0, kq(z)]$ 上均匀分布, 故

$$P(A \mid z_0 = z) = P(u_0 \leqslant \tilde{p}(z)) = \frac{\tilde{p}(z)}{kq(z)},$$

这里用到 $0 \leqslant \tilde{p}(z) \leqslant kq(z)$. 于是由概率的乘法规则,$z_0$ 落于 $\mathrm{d}z$ 且被接受的联合概率为

$$q(z)\,\mathrm{d}z \cdot \frac{\tilde{p}(z)}{kq(z)} = \frac{\tilde{p}(z)}{k}\,\mathrm{d}z,$$

它关于 z 的积分为 Z_p/k, 正是 $P(A)$; 因此给定 A 时 z_0 的条件密度为

$$p(z \mid A) = \frac{\tilde{p}(z)/k}{Z_p/k} = \frac{\tilde{p}(z)}{Z_p} = p(z).$$

又 $P(A) = \int \frac{\tilde{p}(z)}{kq(z)} q(z)\,\mathrm{d}z = \frac{1}{k}\int \tilde{p}(z)\,\mathrm{d}z$, 即(14.15).

□

注

(重构新增) 证明的实质是「先算联合、再条件化」:z_0 与 u_0 的联合分布已知, 接受事件 A 只是对 u_0 的一个截断, 于是 $(z_0 \mid A)$ 的密度就是把联合密度在 A 上积分后归一化. 注意这里没有要求 $\tilde{p}$ 归一化,Z_p 在分子分母中相消, 这是拒绝采样与重要性采样共有的性质. 条件 $kq \geqslant \tilde{p}$ 保证了接受概率不超过 1; 若某个 z 使 $\tilde{p}(z) > kq(z)$, 则上式给出大于 1 的「概率」, 说明该 z 邻域的样本永远不可能被正确筛选, 接受样本的分布将缺失这一块质量. 最后, 接受概率 Z_p/k 与 z 无关, 因此拒绝是「纯粹的浪费」, 算法效率完全由 k 决定.

因此, 这种方法拒绝的点的比例取决于未归一化分布 $\tilde{p}(z)$ 曲线下的面积与缩放分布 $kq(z)$ 曲线下的面积的比值. 由此可见, 常数 k 应尽可能小, 同时满足对于任意 z,$kq(z)$ 处处都不小于 $\tilde{p}(z)$ 的限制条件.

为了演示拒绝采样的用法, 考虑从 Gamma 分布中采样的任务:

$$\mathrm{Gam}(z \mid a, b) = \frac{b^a z^{a-1} \exp(-bz)}{\Gamma(a)}. \tag{14.16}$$

当 $a > 1$,Gamma 分布具有钟形形状, 如图 14.5所示. 因此, 合适的提议分布是 Cauchy 分布(14.9), 因为它也是钟形的, 而且我们可以使用前面讨论的变换方法从中采样. 我们需要稍微扩展一下 Cauchy 分布, 以确保其在任何

位置的值都不小于 Gamma 分布的值. 这可以通过使用 $z = b\tan y + c$ 变换均匀分布的随机变量 y 来实现, 这会产生服从下式分布的随机数 (见练习 14.8):

$$q(z) = \frac{k}{1 + (z-c)^2/b^2}. \tag{14.17}$$

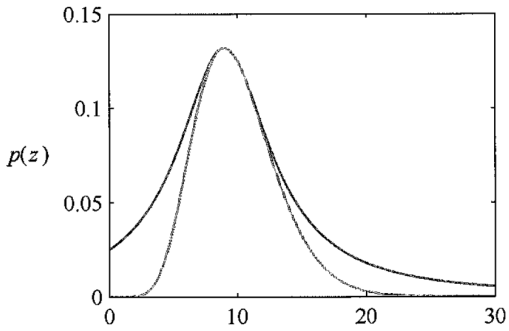


图 14.5: 式(14.16)给出的 Gamma 分布为绿色曲线, 缩放后的 Cauchy 提议分布为红色曲线. 可以通过从 Cauchy 分布中采样, 然后应用拒绝采样来获得 Gamma 分布的样本

通过设置 $c = a - 1$、$b^2 = 2a - 1$, 并选择尽可能小的常数 k 来满足 $kq(z) \geqslant \tilde{p}(z)$ 的要求, 可以获得最小的拒绝率. 最终得到的比较函数如图 14.5所示.

例题 14.3 用拒绝采样从 Gamma 分布中采样 设目标分布为(14.16)给出的 $\mathrm{Gam}(z \mid a, b)$, 取提议分布为(14.17), 其中 $c = a - 1$、$b^2 = 2a - 1$. 说明如何用拒绝采样得到 Gamma 分布的样本.

解

(重构新增) 由(14.17),q 是对称的钟形密度, 可用 $z = c + b\tan\left(\pi\left(u - \frac{1}{2}\right)\right)(u \sim \mathcal{U}(0,1))$ 采样, 且 q 的尾部按 z^{-2} 衰减, 比 Gamma 分布的指数尾部更重, 故存在有限常数 k 使 $kq \geqslant \tilde{p}$. 取

$$k = \sup_{z>0} \frac{\tilde{p}(z)}{q(z)},$$

其中 $\tilde{p}(z) = b^a z^{a-1} e^{-bz}/\Gamma(a)(z > 0)$. 由于 $a > 1$ 时 $\tilde{p}$ 在 $z = (a-1)/b = c/b$ 处取极大, 而 q 在同一位置取极大,k 由该点的比值给出 (精确的最优 k 需用求导或数值优化确定). 算法为: 从 q 采样 z_0, 从 $\mathcal{U}(0, kq(z_0))$ 采样 u_0, 若 $u_0 \leqslant \tilde{p}(z_0)$ 则保留 z_0, 否则拒绝. 由命题 14.6, 保留下来的 z_0 服从 $\mathrm{Gam}(z \mid a, b)$, 接受概率为 Z_p/k.

□

为了使拒绝采样具有实用价值, 我们要求比较函数与目标分布尽可能接近, 以便将拒绝率保持在最低限度. 下面看看当我们试图在高维空间中使用拒绝采样时会发生什么. 我们考虑一个有点儿故意设计的问题: 希望从一个均值为零、协方差为 $\sigma_p^2\boldsymbol{I}$ 的 Gauss 分布中采样 (其中 $\boldsymbol{I}$ 是单位矩阵), 而提议分布本身是一个均值为零、协方差为 $\sigma_q^2\boldsymbol{I}$ 的 Gauss 分布. 显然, 必须有 $\sigma_q^2 \geqslant \sigma_p^2$ 以确保存在一个常数 k 使得 $kq(z) \geqslant p(z)$. 在 D 维空间中,k 的最优值由 $k = \left(\sigma_q/\sigma_p\right)^D$ 给出, 参见图 14.6中 $D = 1$ 的情形. 样本接受率将是 $p(z)$ 和 $kq(z)$ 曲线下面积的比值, 因为这两个分布都是归一化的, 所以结果为 $1/k$. 样本接受率会随着维度的增大呈指数下降. 即使 σ_q 仅比 σ_p 高出 1%, 对于 $D = 1000$, 接受率大约为 $1/20\,000$.

在这个例子中, 比较函数接近目标分布. 目标分布可能是多模态且具有尖锐峰值的, 要找到一个好的提议分布和比较函数将极其困难. 此外, 接受率随维度呈指数下降是拒绝采样的一个普遍特性. 尽管拒绝采样在一维或两维空间中可能是一种有用的技术, 但它不适用于高维问题. 然而, 它可以作为在高维空间采样的更复杂算法中的一个子程序发挥作用.

命题 14.7 (各向同性 Gauss 提议下的最优比较常数)

设 $p = \mathcal{N}\left(\mathbf{0}, \sigma_p^2\boldsymbol{I}\right), q = \mathcal{N}\left(\mathbf{0}, \sigma_q^2\boldsymbol{I}\right), \sigma_q > \sigma_p, \boldsymbol{z} \in \mathbb{R}^D$. 则满足 $kq(\boldsymbol{z}) \geqslant p(\boldsymbol{z})$ 对一切 $\boldsymbol{z}$ 成立的最小常数为

$$k = \left(\frac{\sigma_q}{\sigma_p}\right)^D, \tag{14.18}$$

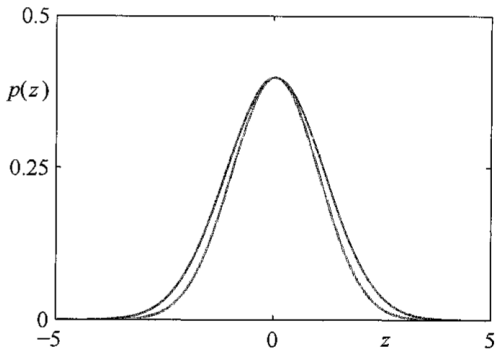


图 14.6: 用来突出拒绝采样局限性的示例. 可通过使用提议分布 $q(z)$ 进行拒绝采样, 并从 Gauss 分布 $p(z)$ 中抽取样本, 该分布用绿色曲线表示, 其缩放版本 $kq(z)$ 用红色曲线表示, 提议分布 $q(z)$ 也是 Gauss 分布

相应的接受概率为 $1/k = \left(\sigma_p/\sigma_q\right)^D$.

证明

两个密度的比值为

$$\frac{p(z)}{q(z)} = \left(\frac{\sigma_q}{\sigma_p}\right)^D \exp\left(-\frac{1}{2}\left(\frac{1}{\sigma_p^2} - \frac{1}{\sigma_q^2}\right)\|z\|^2\right),$$

由于 $\sigma_q > \sigma_p$, 指数上的系数 $\dfrac{1}{\sigma_p^2} - \dfrac{1}{\sigma_q^2} > 0$, 故该比值在 $z = \mathbf{0}$ 处取最大值, 最大值即(14.18). 取 k 等于该最大值时 $kq \geqslant p$ 处处成立, 且任何更小的 k 都会在 $z = \mathbf{0}$ 处失效, 故它是最小常数. 接受概率由(14.15), 因 p, q 均已归一化, 得 $1/k$.

□

注

(重构新增) 本命题定量地说明了「维数灾难」在拒绝采样中的具体形态:k 是每维比值 σ_q/σ_p 的 D 次幂, 因此即使单维上提议分布只比目标分布「胖」1%, 在 $D = 1000$ 维时 $k \approx 1.01^{1000} \approx 2 \times 10^4$, 接受率降至约 $1/20\,000$. 这里 k 的最优性依赖于两个分布同为各向同性 Gauss: 此时密度比只通过 $\|z\|^2$ 依赖 z, 最大值在原点取得; 若目标分布是多模态或有尖峰, 密度比的最大值会出现在某个难以预知的点,k 既难计算又往往极大. 条件 $\sigma_q > \sigma_p$ 是必要的: 若 $\sigma_q \leqslant \sigma_p$, 则 p/q 在 $\|z\| \to \infty$ 时无界, 不存在有限 k, 这正是「提议分布必须有更重的尾」的一维特例. 命题也解释了为什么拒绝采样在高维只能作为子程序: 把维度拆成若干低维块分别处理, 才能把 k 的指数增长控制在可接受范围内.

14.1.4 适应性拒绝采样

许多时候, 我们希望使用拒绝采样方法却很难确定包络分布 $q(z)$ 的合适解析形式. 一种替代方法是基于分布 $\tilde{p}(z)$ 的测量值构造包络函数 (Gilks and Wild, 1992). 当 $p(z)$ 是对数凹函数时, 即 $\ln p(z)$ 的导数是 z 的非增函数时, 构造合适的包络函数尤其简单. 构造的包络函数的说明见图 14.7.

定义 14.4 (对数凹函数)

设 $p(z) > 0$ 定义在区间 I 上. 若 $\ln p(z)$ 是凹函数, 即对任意 $z_1, z_2 \in I$ 与 $t \in [0, 1]$ 有

$$\ln p\left(tz_1 + (1-t)z_2\right) \geqslant t \ln p(z_1) + (1-t) \ln p(z_2),$$

则称 p 是对数凹 (log-concave) 的. 若 $\ln p$ 可微, 则等价于 $(\ln p)'$ 在 I 上非增.

注

(重构新增) 对数凹性是适应性拒绝采样能够「自动构造包络」的关键:$\ln p$ 凹意味着它的图像位于任意切线的下方, 于是取若干网格点处的切线、再取这些切线的逐点最大值, 就得到一个分段线性的上界, 其指数即为一个分段指数的包络密度, 且该包络可以精确地归一化与采样. 若 p 不是对数凹的, 切线可能落在 $\ln p$ 的下方, 包络不再成立, 算法必须加入 Metropolis-Hastings 修正 (见小节 14.2.3). 常见的对数凹分布包括 Gauss、指数、Gamma($a \geqslant 1$)、Beta(参数均 $\geqslant 1$) 等; 混合分布通常不是对数凹的, 因为 $\ln(\pi_1 p_1 + \pi_2 p_2)$ 一般不是凹函数. 注意定义只涉及 $\ln p$ 而不涉及 p 本身:p 本身不必是凹的 (例如 Gauss 密度在均值附近是凸的).

首先, 在一些初始网格点上计算 $\ln \tilde{p}(z)$ 及其梯度, 然后利用得到的切线交点来构造包络函数. 接下来, 从包络分布中抽取一个样本值 (见练习 14.10). 这一步很简单, 因为包络分布的对数是一系列线性函数, 因此包络分布本身是由以下形式的分段指数分布构成的:

$$q(z) = k_j \lambda_j \exp\left\{-\lambda_j \left(z - z_{j-1}\right)\right\}, \qquad z_{j-1} < z \leqslant z_j. \tag{14.19}$$

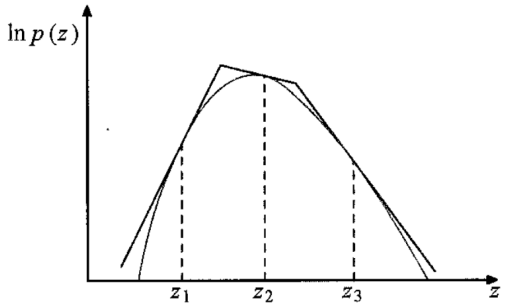


图 14.7: 在拒绝采样中, 如果一个分布是对数凹的, 则可以使用我们在一组网格点计算得到的切线来构造一个包络函数. 如果一个样本被拒绝, 就将它添加到网格点集合中, 并用于完善包络分布

一旦抽取了一个样本, 就可以应用常规的拒绝标准. 样本如果被接受, 它将是期望分布的一个抽取. 样本如果被拒绝, 它就被纳入网格点集合中, 用于计算新的切线, 并由此完善包络函数. 随着网格点数量的增加, 包络函数会越来越逼近期望分布 $p(z)$, 并且样本被拒绝的概率就会降低.

Gilks 提出了旨在避免计算导数的自适应拒绝采样 (Gilks, 1992). 自适应拒绝采样框架也可以扩展到非对数凹分布, 只需要在每个拒绝采样步骤之后跟随一个 Metropolis-Hastings 步骤 (将在小节 14.2.3中讨论), 从而产生自适应拒绝 Metropolis 采样 (Gilks, Best, and Tan, 1995).

14.1.5 重要性采样

我们希望从复杂概率分布中进行采样的一个原因是为了计算形如(14.1)的期望. 重要性采样技术提供了一个直接近似期望的框架, 但它本身并不提供从分布 $p(z)$ 中抽取样本的方法.

式(14.2)给出的有限和近似期望值, 依赖于能够从分布 $p(z)$ 中抽取样本. 然而, 假设直接从 $p(z)$ 中采样是不现实的, 但对于任意给定的 z 值, 我们能够容易地计算出 $\tilde{p}(z)$ 的值. 一种简单的计算期望方法是将 z 空间离散化为均匀网格, 并将期望表示成求和的形式:

$$\mathbb{E}[f] \approx \sum_{l=1}^{L} p\left(z^{(l)}\right) f\left(z^{(l)}\right). \tag{14.20}$$

这种方法的一个明显问题是, 求和项的数量会随着 z 的维度的增加呈指数增长. 此外, 正如我们已经注意到的, 我们感兴趣的概率分布通常会将它们的大部分概率密度限制在 z 空间的相对较小的区域内, 所以在高维问题中, 均匀采样将非常低效, 因为只有很少一部分样本会对求和做出显著贡献. 我们真正希望的是在 $p(z)$ 较大的区域, 或理想情况下, 从 $p(z)f(z)$ 较大的区域中选择样本点.

与拒绝采样一样, 重要性采样也基于易于抽取样本的提议分布 $q(z)$, 如图 14.8所示. 我们可以利用从 $q(z)$ 中

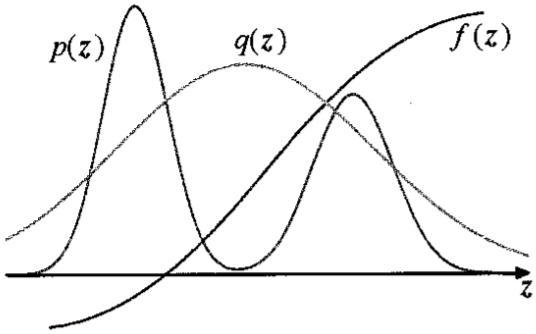


图 14.8: 重要性采样解决了这样一个问题: 当我们很难直接从分布 $p(z)$ 中采样时, 如何计算函数 $f(z)$ 关于该分布的期望. 此方法从更易采样的分布 $q(z)$ 中抽取样本 $\{z^{(l)}\}$, 并且在求和计算中为每个样本项赋予权重 $p\left(z^{(l)}\right)/q\left(z^{(l)}\right)$

抽取的样本集 $\{z^{(l)}\}$, 将期望表达为样本上的有限和形式:

$$\begin{aligned}\mathbb{E}[f] &= \int f(z)p(z)\,\mathrm{d}z \\ &= \int f(z)\frac{p(z)}{q(z)}q(z)\,\mathrm{d}z \\ &\approx \frac{1}{L}\sum_{l=1}^{L}\frac{p\left(z^{(l)}\right)}{q\left(z^{(l)}\right)}f\left(z^{(l)}\right).\end{aligned} \tag{14.21}$$

定义 14.5 (重要性权重)

在重要性采样中, 称

$$r_l = \frac{p\left(z^{(l)}\right)}{q\left(z^{(l)}\right)}$$

为样本 $z^{(l)}$ 的重要性权重 (importance weights), 其中 $z^{(l)} \sim q$,$\widetilde{p}$ 为与 p 成比例的未归一化分布.

♣

注

(重构新增) 重要性权重的动机是「换测度」:$\mathbb{E}_p[f] = \int f\,p\,\mathrm{d}z = \int f\frac{p}{q}q\,\mathrm{d}z = \mathbb{E}_q\left[f\frac{p}{q}\right]$, 于是只要能从 q 采样, 就可以通过对 f 乘上权重 p/q 来计算 p 下的期望, 权重正好纠正了「从错误分布采样」引入的偏差. 定义中的 $\widetilde{p}$ 不必归一化: 若用 $p = \widetilde{p}/Z_p$ 与 $q = \widetilde{q}/Z_q$, 则常数之比会在归一化权重中相消 (见(14.25)). 使用权重的前提是 $q(z) > 0$ 在 $p(z)f(z) \neq 0$ 的区域成立, 否则该区域被系统性漏掉, 权重再大也无法补救; 这是重要性采样最容易被忽视的条件, 也是它与拒绝采样共享的「支撑集覆盖」要求.

量 $r_l = p\left(z^{(l)}\right)/q\left(z^{(l)}\right)$ 称为重要性权重, 用于纠正从错误分布中采样而引入的偏差. 请注意, 与拒绝采样不同, 这里生成的所有样本都会被保留.

通常分布 $p(z)$ 的计算只能达到一个归一化常数的精度, 所以 $p(z) = \widetilde{p}(z)/Z_p$, 其中 $\widetilde{p}(z)$ 可以轻易计算出来, 而 Z_p 是未知的. 类似地, 我们可能希望使用的重要性采样分布 $q(z) = \widetilde{q}(z)/Z_q$ 也具有同样的性质:

$$\begin{aligned}\mathbb{E}[f] &= \int f(z)p(z)\,\mathrm{d}z \\ &= \frac{Z_q}{Z_p}\int f(z)\frac{\widetilde{p}(z)}{\widetilde{q}(z)}q(z)\,\mathrm{d}z \\ &\approx \frac{Z_q}{Z_p}\frac{1}{L}\sum_{l=1}^{L}\tilde{r}_l f\left(z^{(l)}\right),\end{aligned} \tag{14.22}$$

其中 $\tilde{r}_l = \widetilde{p}\left(z^{(l)}\right)/\widetilde{q}\left(z^{(l)}\right)$. 我们可以使用相同的样本集来计算比值 Z_p/Z_q, 结果为

$$\frac{Z_p}{Z_q} = \frac{1}{Z_q}\int \widetilde{p}(z)\,\mathrm{d}z = \int \frac{\widetilde{p}(z)}{\widetilde{q}(z)}q(z)\,\mathrm{d}z$$

$$\approx \frac{1}{L}\sum_{l=1}^{L}\tilde{r}_l. \tag{14.23}$$

因此,(14.22)中的期望可用加权和来表示:

$$\mathbb{E}[f] \approx \sum_{l=1}^{L} w_l f\left(z^{(l)}\right), \tag{14.24}$$

其中

$$w_l = \frac{\tilde{r}_l}{\sum_m \tilde{r}_m} = \frac{\widetilde{p}\left(z^{(l)}\right)/q\left(z^{(l)}\right)}{\sum_m \widetilde{p}\left(z^{(m)}\right)/q\left(z^{(m)}\right)}. \tag{14.25}$$

请注意,$\{w_l\}$ 都是非负数且和为 1.

命题 14.8 (重要性采样估计的无偏性)

设 $z^{(l)} \sim q$ 独立同分布,$q(z) > 0$ 在 $\{z : p(z)f(z) \neq 0\}$ 上成立, 且 $\mathbb{E}_q\left[\left|f(z)\frac{p(z)}{q(z)}\right|\right] < \infty$. 则由(14.21)定义的估计量

$$\overline{f}_q = \frac{1}{L}\sum_{l=1}^{L}\frac{p\left(z^{(l)}\right)}{q\left(z^{(l)}\right)} f\left(z^{(l)}\right)$$

是 $\mathbb{E}_p[f]$ 的无偏估计.

♠

证明

对每一项取关于 q 的期望, 并利用 p 为归一化密度:

$$\mathbb{E}_q\left[\frac{p(z)}{q(z)}f(z)\right] = \int \frac{p(z)}{q(z)}f(z)\,q(z)\,\mathrm{d}z = \int p(z)f(z)\,\mathrm{d}z = \mathbb{E}_p[f],$$

这里用到 $q(z) > 0$ 在 $p(z)f(z) \neq 0$ 处成立, 从而被积函数关于 $q\,\mathrm{d}z$ 可积. 由期望的线性性,$\mathbb{E}_q\left[\overline{f}_q\right] = \mathbb{E}_p[f]$. 对自归一化形式(14.22), 由大数定律,

$$\frac{1}{L}\sum_{l=1}^{L}\tilde{r}_l \longrightarrow \frac{Z_p}{Z_q},$$

故加权和几乎必然收敛到 $\mathbb{E}_p[f]$; 但它是两个随机量之比, 一般只有 $O(1/L)$ 的偏差.

□

注

(重构新增) 无偏性只对未归一化的形式(14.21)成立; 一旦使用自归一化权重(14.25), 估计量变成两个随机量之比, 其期望一般不等于 $\mathbb{E}_p[f]$, 偏差为 $O(1/L)$, 但它是相合的 ($L \to \infty$ 时收敛), 实践中因不需要 Z_p 而更常用. 条件 $\mathbb{E}_q\left[|f\,p/q|\right] < \infty$ 比命题 14.1中的条件更强: 即使 f 有界, 若 q 的尾部比 p 轻, 则 p/q 无界, 方差可能无穷大, 估计量虽无偏却极不稳定. 这正是下文强调「q 在 p 显著的区域不能太小」的原因, 也是有效样本量

$$\mathrm{ESS} = \frac{\left(\sum_l w_l\right)^2}{\sum_l w_l^2}$$

被用来诊断权重退化的原因.

与拒绝采样一样, 重要性采样的成功严重依赖于采样分布 $q(z)$ 与目标分布 $p(z)$ 的匹配程度. 如果 $p(z)f(z)$ 变化剧烈并且其大部分概率密度集中在 z 空间的相对较小的区域内 (这种情况很常见), 则一组重要性权重 $\{r_l\}$ 可能会受少数几个较大权重的支配, 剩余的权重则相对无关紧要. 因此, 有效样本的数量可能比表面上的样本数 L 少得多. 如果没有任何样本落在 $p(z)f(z)$ 较大的区域内, 问题将更为严重. 在那种情况下, 即使 r_l 和 $r_l f\left(z^{(l)}\right)$ 的表面方差可能很小, 对期望的估计也可能产生严重错误. 因此, 重要性采样的一个主要缺点是, 它可能产生任意错误的结果, 并且没有诊断指标. 这也突出了采样分布 $q(z)$ 的一个关键要求, 即在 $p(z)$ 可能显著的区域内, 它的值不应该太小或为零.

14.1.6 采样-重要性-重采样

小节 14.1.3讨论的拒绝采样部分地依赖于成功确定一个适当的 k 值. 对于分布 $p(z)$ 和 $q(z)$ 的各种组合, 想要确定一个适当的 k 值是不切实际的, 因为当 k 值大到能保证目标分布被包络的时候, 对应的样本接受率会小到没有实用价值.

与拒绝采样一样, 采样-重要性-重采样方案也使用一个采样分布 $q(z)$, 但避免了确定 k 值的要求. 该方案分两个阶段. 在第一阶段, 从 $q(z)$ 中抽取 L 个样本 $z^{(1)}, \cdots, z^{(L)}$. 然后在第二阶段, 使用(14.25)构建权重 $(w_1, \cdots, w_L)$. 最后从离散分布 $\left(z^{(1)}, \cdots, z^{(L)}\right)$ 中抽取第二组 L 个样本, 其被抽中的概率由权重 $(w_1, \cdots, w_L)$ 给出.

得到的 L 个样本只是近似按照 $p(z)$ 分布的, 但随着 L 趋于无穷大, 该分布将变得正确. 为了理解这一点, 考虑单变量情况, 并注意重采样值的累积分布由下式给出:

$$p(z < a) = \sum_l w_l \mathbb{I}\left(z^{(l)} \leqslant a\right) \simeq \int \mathbb{I}(z \leqslant a)\, p(z)\, \mathrm{d}z, \tag{14.26}$$

其中 $\mathbb{I}(\cdot)$ 是指示函数 (如果其参数为真, 则函数值为 1, 否则为 0). 使 L 趋于无穷大, 并假设分布的适当的正则性, 我们就可以用根据原始采样分布 $q(z)$ 加权的积分来替换求和:

$$p(z < a) \simeq \int \mathbb{I}(z \leqslant a) \frac{\widetilde{p}(z)}{q(z)} q(z)\, \mathrm{d}z. \tag{14.27}$$

这就是 $p(z)$ 的累积分布函数. 同样, 我们可以看到 $p(z)$ 的归一化并不是必需的.

命题 14.9 (重采样分布的一致性)

设 $z^{(l)} \sim q$ 独立同分布, 权重由(14.25)给出, 且 $q(z) > 0$ 在 $\{z : p(z) \neq 0\}$ 上成立. 则对任意 a, 当 $L \to \infty$ 时(14.26)的右端依概率收敛于 $p(z < a) = \int_{-\infty}^{a} p(z)\, \mathrm{d}z$.

证明

把(14.26)中的 w_l 写成自归一化形式, 并分子分母同乘 L:

$$\sum_l w_l \mathbb{I}\left(z^{(l)} \leqslant a\right) = \frac{\frac{1}{L}\sum_l \tilde{r}_l \mathbb{I}\left(z^{(l)} \leqslant a\right)}{\frac{1}{L}\sum_l \tilde{r}_l}.$$

由大数定律, 分母收敛于 $\mathbb{E}_q[\tilde{r}] = Z_p/Z_q$, 分子收敛于

$$\mathbb{E}_q\left[\tilde{r}\, \mathbb{I}(z \leqslant a)\right] = \int \frac{\widetilde{p}(z)}{\widetilde{q}(z)} \mathbb{I}(z \leqslant a)\, q(z)\, \mathrm{d}z = \frac{Z_p}{Z_q} \int \mathbb{I}(z \leqslant a) p(z)\, \mathrm{d}z.$$

两式相除即得 $p(z<a)$. 这里用到 $\mathbb{I}$ 有界, 从而上述期望有限, 大数定律适用.

□

注

(重构新增) 重采样的实质是「用样本经验分布 $\sum_l w_l \delta_{z^{(l)}}$ 逼近 p」: 它先允许用任意 q 生成候选点, 再用权重把候选点重新分配到正确的概率质量上, 因此不需要比较函数 k, 也不会像拒绝采样那样丢弃样本. 代价是重采样引入了额外的随机性: 同一批 L 个候选点经重采样后可能出现重复值, 样本之间不再独立, 且当权重高度集中时 (少数 w_l 接近 1), 重采样后的样本几乎全部来自少数几个候选点, 多样性急剧下降——这正是粒子滤波中「样本贫化」现象的来源. 命题中的一致性只保证 $L\to\infty$ 时的极限正确, 有限 L 的偏差与权重的方差成正比, 故实践中仍需监控有效样本量. 当 $q=p$ 时所有 $w_l=1/L$, 重采样等价于对原样本再做一次均匀抽样, 分布不变.

对于有限值 L 和给定的初始样本集, 重采样值只能近似地从目标分布中抽取. 与拒绝采样一样, 随着采样分布 $q(z)$ 更接近目标分布 $p(z)$, 近似结果的效果也会越来越好. 当 $q(z)=p(z)$ 时, 初始样本 $\{z^{(1)},\cdots,z^{(L)}\}$ 具有目标分布, 且权重 $w_l=1/L$, 因此重采样值也具有目标分布.

如果需要关于分布 $p(z)$ 的矩, 则可以直接使用原始样本连同权重来计算, 因为

$$\mathbb{E}[f(z)]=\int f(z)\,p(z)\,\mathrm{d}z. \tag{14.28}$$

14.2 Markov 链 Monte Carlo 采样

在上一节中, 我们讨论了用于计算函数期望的拒绝采样和重要性采样策略, 并且我们看到它们在高维空间中有严重局限性. 本节将讨论一种非常通用和强大的采样策略, 称为 Markov 链 Monte Carlo(Markov chain Monte Carlo,MCMC) 采样, 它允许从多种分布中进行采样并且可以很好地扩展到高维样本空间.Markov 链 Monte Carlo 采样起源于物理学 (Metropolis and Ulam, 1949), 直至 20 世纪 80 年代末期才开始在统计学领域产生显著影响.

与拒绝采样和重要性采样一样, 我们依然从提议分布中采样. 然而这一次, 我们保持当前状态 $z^{(\tau)}$ 的记录, 提议分布 $q\left(z\mid z^{(\tau)}\right)$ 以当前状态为条件, 因此样本序列 $z^{(1)},z^{(2)},\cdots$ 形成一个 Markov 链 (参见小节 14.2.2). 同样, 尽管 Z_p 的值可能是未知的, 如果我们将 $p(z)$ 写成 $p(z)=\widetilde{p}(z)/Z_p$, 我们假设对于任意给定的 z 值,$\widetilde{p}(z)$ 可以很容易计算出来. 我们的提议分布足够简单, 可以直接从中抽取样本. 在算法的每一个迭代中, 我们从提议分布中生成一个候选样本 $z^\star$, 然后根据一个适当的准则来接受该样本.

14.2.1 Metropolis 算法

在基础的 Metropolis 算法 (Metropolis et al., 1953) 中, 我们假设提议分布是对称的, 即对于所有的 z_A 和 z_B 值, 有 $q(z_A\mid z_B)=q(z_B\mid z_A)$. 然后候选样本以如下概率被接受:

$$A\left(z^\star,z^{(\tau)}\right)=\min\left(1,\frac{\widetilde{p}\left(z^\star\right)}{\widetilde{p}\left(z^{(\tau)}\right)}\right), \tag{14.29}$$

这可以通过选择一个在单位区间 $(0,1)$ 上均匀分布的随机数 u 来实现: 如果 $A\left(z^\star,z^{(\tau)}\right)>u$, 则接受该样本. 请注意, 如果从 $z^{(\tau)}$ 到 $z^\star$ 的步骤导致 $p(z)$ 的值增加, 那么候选样本一定会被保留.

定义 14.6 (Metropolis 算法)

设目标分布为 $p(z)=\widetilde{p}(z)/Z_p$,$Z_p$ 未知, 提议分布 q 关于其两个自变量对称.Metropolis 算法按下述方式生成 Markov 链: 给定当前状态 $z^{(\tau)}$, 从 $q\left(z\mid z^{(\tau)}\right)$ 中抽取候选样本 $z^\star$, 以(14.29)给出的概率 $A\left(z^\star,z^{(\tau)}\right)$ 接受它, 并令

$$z^{(\tau+1)}=\begin{cases}z^\star, & \text{若接受},\\ z^{(\tau)}, & \text{若拒绝}.\end{cases}$$

♣

注

(重构新增)Metropolis 算法用「按目标密度比决定去留」代替了拒绝采样中的比较函数: 分子分母中的未知常数 Z_p 自动相消, 因此无需知道归一化常数, 也无需寻找包络. 对称性条件不可省: 只有 $q\left(z^{\star} \mid z^{(\tau)}\right)=q\left(z^{(\tau)} \mid z^{\star}\right)$ 时, 接受率才恰为 $\tilde{p}(z^{\star})/\tilde{p}(z^{(\tau)})$; 否则必须补上提议分布之比, 这就是小节 14.2.3的推广. 注意接受率取 $\min(1,\cdot)$: 当候选状态的密度更高时总是接受, 当密度更低时按比例接受, 这一「有偏的爬山」正是让链停留在 p 的高密度区而不至于永远卡在局部极大值的原因. 最后, 拒绝时链原地不动 ($z^{(\tau+1)}=z^{(\tau)}$), 样本序列中会出现重复值, 这是 Metropolis 与拒绝采样的本质区别: 被拒绝的信息被保留而非丢弃.

下面给出 Metropolis 采样的完整流程.

算法 14.1:Metropolis 采样

输入: 未归一化分布 $\tilde{p}(z)$; 提议分布 $q\left(z \mid \tilde{z}\right)$; 初始状态 $z^{(0)}$; 迭代次数 T.

输出: $z \sim \tilde{p}(z)$.

$z_{\text{prev}} \leftarrow z^{(0)}$;

对 $\tau \in \{1, \cdots, T\}$:

从提议分布中采样 $z^{\star} \sim q\left(z \mid z_{\text{prev}}\right)$;

从均匀分布中采样 $u \sim \mathcal{U}(0,1)$;

若 $\tilde{p}\left(z^{\star}\right) / \tilde{p}\left(z_{\text{prev}}\right)>u$, 则 $z_{\text{prev}} \leftarrow z^{\star}$(即 $z^{(\tau)}=z^{\star}$); 否则保持 z_{prev} 不变 (即 $z^{(\tau)}=z^{(\tau-1)}$);

返回 z_{prev}(即 $z^{(T)}$).

如果候选样本被接受, 则 $z^{(\tau+1)}=z^{\star}$, 否则候选样本 $z^{\star}$ 将被丢弃: 将 $z^{(\tau+1)}$ 设置为 $z^{(\tau)}$, 并从分布 $q\left(z \mid z^{(\tau+1)}\right)$ 中抽取另一个候选样本. 这与拒绝采样形成了鲜明对比, 在拒绝采样中, 被拒绝的样本将直接被丢弃. 在 Metropolis 算法中, 当一个候选样本被拒绝时, 它的前一个样本将包含在最终的样本列表中, 导致样本有多个副本. 当然, 在实际应用中, 每个保留的样本只会有一个副本, 并附带一个整数权重因子, 用于记录该状态出现的次数.

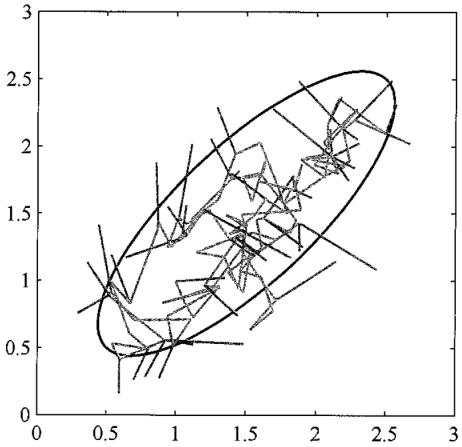


图 14.9: 使用 Metropolis 算法从 Gauss 分布中采样的简单示意图, 该 Gauss 分布的一个标准差等高线是用椭圆显示的. 提议分布是一个各向同性的 Gauss 分布, 其标准差为 0.2. 接受的候选样本以绿线显示, 拒绝的候选样本以红线显示. 共生成 150 个候选样本, 其中 43 个被拒绝

正如我们将看到的, 如果对于任何的 z_A 和 z_B 值,$q(z_A \mid z_B)$ 都是正值 (这是一个充分但非必要条件), 则 $z^{(\tau)}$ 的分布趋向于 $p(z)$(当 $\tau \rightarrow \infty$ 时). 然而, 应该强调的是, 序列 $z^{(1)}, z^{(2)}, \cdots$ 不是来自 $p(z)$ 的独立样本集, 因为连续的样本高度相关. 如果希望获得独立的样本, 那么我们可以丢弃大部分序列, 每次只保留第 M 个样本. 对于足够大的 M, 保留的样本将在实际意义上是独立的.图 14.9显示了一个简单的示例, 它使用 Metropolis 算法从二维 Gauss 分布中进行采样, 其中提议分布是各向同性的 Gauss 分布.

为了深入了解 Markov 链 Monte Carlo 采样算法的本质, 我们观察一个具体例子 (即随机游走) 的性质. 考虑一个由整数构成的状态空间 $\mathbb{Z}$, 已知概率

$$p\left(z^{(\tau+1)}=z^{(\tau)}\right)=0.5, \tag{14.30}$$

$$p\left(z^{(\tau+1)}=z^{(\tau)}+1\right)=0.25, \tag{14.31}$$

$$p\left(z^{(\tau+1)}=z^{(\tau)}-1\right)=0.25, \tag{14.32}$$

其中 $z^{(\tau)}$ 表示在时间步 τ 的状态.

例题 14.4 整数上的简单随机游走 设 $\{z^{(\tau)}\}$ 按(14.30) ~ (14.32)演化, 初始状态 $z^{(0)}=0$. 求 $\mathbb{E}\left[z^{(\tau)}\right]$ 与 $\mathbb{E}\left[\left(z^{(\tau)}\right)^2\right]$.

解

(重构新增) 由对称性, 给定 $z^{(\tau-1)}=z$ 时,

$$\mathbb{E}\left[z^{(\tau)}\mid z^{(\tau-1)}=z\right]=0.5z+0.25(z+1)+0.25(z-1)=z,$$

故 $\mathbb{E}\left[z^{(\tau)}\right]=\mathbb{E}\left[z^{(\tau-1)}\right]=\cdots=z^{(0)}=0$. 又

$$\mathbb{E}\left[\left(z^{(\tau)}\right)^2\mid z^{(\tau-1)}=z\right]=0.5z^2+0.25(z+1)^2+0.25(z-1)^2=z^2+\frac{1}{2},$$

故 $\mathbb{E}\left[\left(z^{(\tau)}\right)^2\right]=\mathbb{E}\left[\left(z^{(\tau-1)}\right)^2\right]+\frac{1}{2}$, 归纳得 $\mathbb{E}\left[\left(z^{(\tau)}\right)^2\right]=\tau/2$(见练习 14.11).

□

因此, 在时间步 τ 之后, 随机游走平均而言仅行进了与 $\sqrt{\tau}$ 成比例的距离. 这种平方根依赖性是随机游走行为的典型特征, 表明随机游走在探索状态空间时效率非常低. 正如我们将看到的, 设计 Markov 链 Monte Carlo 采样算法的一个核心目标就是避免随机游走行为.

14.2.2 Markov 链

在更详细地讨论 Markov 链 Monte Carlo 采样之前, 我们先研究一下 Markov 链的一般性质. 特别地, 我们想知道 Markov 链在何种情况下会收敛到期望的分布.

定义 14.7 (一阶 Markov 链)

称随机变量序列 $z^{(1)},\cdots,z^{(M)}$ 为一阶 Markov 链, 若对每个 $m\in\{1,\cdots,M-1\}$ 都有条件独立性

$$p\left(z^{(m+1)}\mid z^{(1)},\cdots,z^{(m)}\right)=p\left(z^{(m+1)}\mid z^{(m)}\right). \tag{14.33}$$

♣

注

(重构新增)(14.33)的含义是「给定现在, 未来与过去条件独立」: 链在时刻 $m+1$ 的状态只通过当前状态 m 依赖历史, 因此整条链的联合分布可由初始分布 $p\left(z^{(1)}\right)$ 与转移核 T_m 完全确定. 这一「无记忆性」正是 MCMC 能够只用当前状态就生成下一个样本的原因, 也是它相比拒绝采样、重要性采样能在高维工作而不需要全局包络的根本原因. 一阶性是建模选择而非数学必然: 二阶链需要保留两个历史状态, 状态空间随之增大; 本书只处理一阶链.

这可以用一个链式的有向图模型来表示 (见图 11.4). 然后, 我们可以通过给出初始变量的概率分布 $p\left(z^{(0)}\right)$ 和以转移概率 (transition probabilities) 形式表达的后续变量条件分布 $T_m\left(z^{(m)},z^{(m+1)}\right)\equiv p\left(z^{(m+1)}\mid z^{(m)}\right)$, 即可完整定义 Markov 链.

定义 14.8 (齐次 Markov 链)

若定义 14.7中的转移概率 T_m 对一切 m 都相同, 则称该 Markov 链是齐次的或同质 (homogeneous) 的, 此时记 $T\left(z,z'\right)=p\left(z'\mid z\right)$.

♣

注

(重构新增) 齐次性把「随时刻变化的转移规则」简化为单个转移核 T, 于是链的分布演化只依赖于 T 的幂次, 不变分布与遍历性的讨论才成为可能. 本章构造的 MCMC 算法 (Metropolis、Metropolis-Hastings、Gibbs) 全部是齐次链, 转移核不随 τ 改变; 变分推断中使用的「退火」或「自适应」方案则会破坏齐次性, 此时收敛性需要另行论证 (例如 Robbins-Monro 型的学习率条件). 注意定义只要求条件分布相同, 并不要求初始分布与 T 有任何关系——初始分布只影响链的暂态, 这正是 MCMC 需要 burn-in 的原因.

链中的某个特定变量的边缘概率可以用 Markov 链中前一个变量的边缘概率表示为

$$p\left(z^{(m+1)}\right)=\int p\left(z^{(m+1)}\mid z^{(m)}\right)p\left(z^{(m)}\right)\mathrm{d}z^{(m)}, \tag{14.34}$$

其中的积分对于离散变量来说可以替换为求和.

定义 14.9 (不变分布)

若分布 $p^\star(z)$ 满足

$$p^\star(z) = \int T(z', z)\, p^\star(z')\, \mathrm{d}z', \tag{14.35}$$

则称 $p^\star$ 对该 Markov 链是不变的, 也称 $p^\star$ 是平稳分布 (stationary distribution). 若一个分布在 Markov 链的每个步骤都保持该分布不变, 那么该分布对于这个 Markov 链来说就是不变的或者说稳定的.

注

(重构新增) 不变分布是 MCMC 的「目标」: 只要目标分布 p 对转移核不变, 链一旦进入 p 就会永远保持 p, 因此用链上的样本估计 p 下的期望才有意义.(14.35)是一个泛函不动点方程, 它并不保证 $p^\star$ 唯一: 若转移概率由恒等变换给出, 则任何分布都是不变的. 所以「构造一个以 p 为不变分布的链」只是第一步, 还需要定义 14.11保证从任意初始分布出发都收敛到同一个 $p^\star$. 对离散状态空间,(14.35)就是 $p^\star = p^\star T$, 即 $p^\star$ 是转移矩阵的左特征向量 (特征值 1).

请注意, 给定的 Markov 链可能有多个不变分布. 例如, 如果转移概率由恒等变换给出, 那么任何分布都将是不变的.

定义 14.10 (详细平衡)

若转移概率 T 与分布 $p^\star$ 满足

$$p^\star(z)\, T(z, z') = p^\star(z')\, T(z', z), \tag{14.36}$$

则称它们满足详细平衡 (detailed balance) 性质. 满足详细平衡性质的 Markov 链被称为可逆的 (reversible).

注

(重构新增) 详细平衡比不变性更强: 它要求每一对状态之间的「双向流量」逐点相等, 而不只是总流量相等. 验证详细平衡通常比直接验证(14.35)容易得多, 因为后者需要对整个状态空间积分, 而前者只需比较两个密度的乘积; 本章所有 MCMC 算法都是通过构造满足(14.36)的转移核来证明其正确性的. 可逆性一词来自如下事实: 在平稳状态下, 正向轨迹 $(z^{(1)}, z^{(2)}, \cdots)$ 与反向轨迹的联合分布相同, 即链在时间反演下统计等价. 注意详细平衡是充分而非必要条件: 存在不可逆的链 (例如循环式的 Gibbs 扫描) 仍以 p 为不变分布.

确保目标分布 $p(z)$ 是不变的一个充分 (但非必要) 条件是选择满足详细平衡性质的转移概率. 对于特定的分布 $p^\star(z)$, 容易看出, 如果转移概率满足关于该分布的详细平衡性质, 则保持该分布不变, 因为

$$\begin{aligned} \int p^\star(z')\, T(z', z)\, \mathrm{d}z' &= \int p^\star(z)\, T(z, z')\, \mathrm{d}z' \\ &= p^\star(z) \int p(z' \mid z)\, \mathrm{d}z' \\ &= p^\star(z). \end{aligned} \tag{14.37}$$

命题 14.10 (详细平衡蕴含不变性)

设转移核 T 与分布 $p^\star$ 满足(14.36), 且 $T(z, \cdot)$ 对每个 z 都是概率密度. 则 $p^\star$ 是 T 的不变分布, 即(14.35)成立.

证明

由(14.36), 对任意 z,

$$\int p^\star(z')\, T(z', z)\, \mathrm{d}z' = \int p^\star(z) T(z, z')\, \mathrm{d}z' = p^\star(z) \int T(z, z')\, \mathrm{d}z' = p^\star(z),$$

最后一步用到 $T(z, \cdot)$ 是概率密度, 即 $\int T(z, z')\, \mathrm{d}z' = 1$. 这正是(14.35).

□

注

(重构新增) 证明只做了一次「换元 + 提公因子」, 关键是把对 z' 的积分换成对 z' 的另一个方向的积分, 从而把 $p^\star(z)$ 提到积分号外. 条件「$T(z,\cdot)$ 是概率密度」不可省: 若 T 只是次随机核 (例如 Metropolis 中遗漏了「拒绝时原地不动」这一项), 则 $\int T\,\mathrm{d}z' < 1$, 上式给出的是 $p^\star$ 被衰减而不是保持不变. 这解释了算法描述中为何必须显式写出「拒绝则 $z^{(\tau+1)} = z^{(\tau)}$」: 正是这一项补齐了转移核的概率质量, 使(14.37)的归一化成立.

我们的目标是使用 Markov 链从给定分布中采样. 如果能建立一个 Markov 链使得目标分布是不变的, 就可以达到目的. 然而, 我们还必须要求不管初始分布 $p\left(z^{(0)}\right)$ 的选择如何, 当 $m \to \infty$ 时, 分布 $p\left(z^{(m)}\right)$ 都收敛到所需的不变分布 $p^\star(z)$.

定义 14.11 (遍历性)

若对任意初始分布 $p\left(z^{(0)}\right)$, 当 $m \to \infty$ 时 $p\left(z^{(m)}\right)$ 都收敛到同一个不变分布 $p^\star(z)$, 则称该 Markov 链是遍历的 (ergodic), 并称 $p^\star$ 为平衡 (equilibrium) 分布. ♣

注

(重构新增) 遍历性是「无论从哪里出发都能到达并停留在 $p^\star$」的保证, 它是 MCMC 可用性的另一半: 不变性只保证「到了就不走」, 遍历性才保证「一定会到」. 显然一个遍历的 Markov 链只能有一个平衡分布. 可以证明, 一个齐次 Markov 链是遍历的, 只需要对不变分布和转移概率施加较弱的约束条件 (Neal, 1993), 例如不可约 (任意两个正概率区域互相可达) 与非周期 (不会按固定周期振荡); 这两个条件合起来保证 T^m 收敛到秩一投影. 实践中的主要障碍是不可约性: 若目标分布有多个被零概率区域隔开的模态, 而提议分布无法跨越这些区域, 链就会被困在某个模态内, 此时不变性依然成立但遍历性失效 (见练习 14.13).

在实践中, 我们经常利用一组「基础」转移 $B_1, \cdots, B_K$ 来构造转移概率. 这可以通过混合分布的形式来实现:

$$T\left(z', z\right) = \sum_{k=1}^{K} \alpha_k B_k\left(z', z\right), \tag{14.38}$$

混合系数 $\alpha_1, \cdots, \alpha_K$ 必须满足 $\alpha_k \geqslant 0$ 且 $\sum\limits_k \alpha_k = 1$.「基础」转移可以通过连续应用结合起来, 于是有

$$T\left(z', z\right) = \sum_{z_1} \cdots \sum_{z_{K-1}} B_1\left(z', z_1\right) \cdots B_{K-1}(z_{K-2}, z_{K-1})\, B_K(z_{K-1}, z). \tag{14.39}$$

如果一个分布对每个基础转移都是不变的, 那么很明显它也会对(14.38)或(14.39)给出的 $T\left(z', z\right)$ 保持不变. 对于混合分布(14.38), 如果每个基础转移都满足详细平衡条件, 那么复合转移概率 T 也将满足详细平衡条件. 这对于使用(14.39)构造的转移概率并不成立, 尽管通过对基础转移的应用顺序进行对称化 (即 $B_1, B_2, \cdots, B_K, B_K, \cdots, B_2, B_1$), 可以恢复详细平衡. 使用复合转移概率的一个常见例子是让每个基础转移仅改变变量的一个子集.

命题 14.11 (混合转移保持不变性)

设 $p^\star$ 对每个基础转移 B_k 都不变,$\alpha_k \geqslant 0, \sum\limits_k \alpha_k = 1$. 则由(14.38)定义的 T 也不变 $p^\star$. 若进一步每个 B_k 都满足关于 $p^\star$ 的详细平衡, 则 T 也满足详细平衡. ♠

证明

由线性性,

$$\int T\left(z', z\right) p^\star\left(z'\right) \mathrm{d}z' = \sum_{k=1}^{K} \alpha_k \int B_k\left(z', z\right) p^\star\left(z'\right) \mathrm{d}z' = \sum_{k=1}^{K} \alpha_k p^\star(z) = p^\star(z),$$

即 T 不变 $p^\star$. 若每个 B_k 满足详细平衡, 则对每个 k 有 $p^\star(z) B_k\left(z, z'\right) = p^\star\left(z'\right) B_k\left(z', z\right)$, 两边乘 α_k 并对 k 求和即得 $p^\star(z) T\left(z, z'\right) = p^\star\left(z'\right) T\left(z', z\right)$.

□

注

(重构新增) 混合的直观含义是「每一步随机挑选一种更新方式」, 因此只要每种方式都不破坏 $p^\star$, 混合起来也不破坏; 这是 Metropolis-Hastings 中「在不同提议分布之间轮换」以及 Gibbs 采样中「随机挑选一个变量更新」的理论依据. 复合转移 (连续应用) 只保证不变性, 不保证可逆性: 两个各自可逆的核复合后一般不可逆, 因为正向与反向的复合顺序相反; 要对 $B_1,\cdots,B_K$ 的复合恢复详细平衡, 必须按 $B_1,\cdots,B_K,B_K,\cdots,B_1$ 的顺序对称地应用. 由于不变性已足以保证正确性, 多数算法并不追求可逆性, 但可逆性在证明收敛速度与构造控制变量时仍有用.

14.2.3 Metropolis-Hastings 算法

前面我们介绍了基础的 Metropolis 算法, 但没有实践证明它能从所需分布中进行采样. 在给出证明之前, 我们首先讨论一种更泛化的形式, 称为 Metropolis-Hastings 算法 (Hastings, 1970), 该算法适用于提议分布不再是其参数的对称函数的情况. 特别是在算法的第 τ 步, 当前状态为 $z^{(\tau)}$, 首先从分布 $q_k\left(z \mid z^{(\tau)}\right)$ 中抽取一个样本 $z^\star$, 然后以概率 $A_k\left(z^\star, z^{(\tau)}\right)$ 接受它, 其中

$$A_k\left(z^\star, z^{(\tau)}\right) = \min\left(1, \frac{\widetilde{p}\left(z^\star\right) q_k\left(z^{(\tau)} \mid z^\star\right)}{\widetilde{p}\left(z^{(\tau)}\right) q_k\left(z^\star \mid z^{(\tau)}\right)}\right), \tag{14.40}$$

这里的 k 标记了正在考虑的可能转换集合的成员. 同样, 计算接受准则不需要知道概率分布 $p(z) = \widetilde{p}(z)/Z_p$ 中的归一化常数 Z_p. 对于对称的提议分布,Metropolis-Hastings 准则(14.40)可简化为标准的 Metropolis 准则(14.29).

定义 14.12 (Metropolis-Hastings 算法)

给定未归一化目标分布 $\widetilde{p}$、一组提议分布 $\{q_k\}_{k=1}^K$ 以及迭代到分布下标的映射 $M(\cdot)$,Metropolis-Hastings 算法在第 τ 步取 $k = M(\tau)$, 从 $q_k\left(z \mid z^{(\tau)}\right)$ 中抽取 $z^\star$, 以(14.40)的概率接受它, 拒绝时令 $z^{(\tau+1)} = z^{(\tau)}$.

注

(重构新增)Metropolis-Hastings 相对 Metropolis 的唯一改动是接受率中多了一个提议分布之比 $q_k\left(z^{(\tau)} \mid z^\star\right)/q_k\left(z^\star \mid z^{(\tau)}\right)$. 这一项的来源是: 若提议分布偏向于某些方向 (例如向高密度区偏移), 则链会过度访问这些方向, 必须用「返回概率」把偏差校正回来. 它使算法对任意提议分布都成立, 因而允许使用梯度信息 (如 Langevin 提议) 或依赖当前状态的局部近似 (如自适应拒绝采样), 这是 Metropolis-Hastings 成为通用框架的原因. 条件仍然只是 $q_k\left(z^\star \mid z^{(\tau)}\right) > 0$ 当 $\widetilde{p}\left(z^\star\right) > 0$, 以保证支撑集覆盖; 归一化常数 Z_p 依旧相消.

算法 14.2:Metropolis-Hastings 采样

输入: 未归一化分布 $\widetilde{p}(z)$; 提议分布 $\{q_k(z \mid z) : k \in \{1,\cdots,K\}\}$; 迭代下标到分布下标的映射 $M(\cdot)$; 初始状态 $z^{(0)}$; 迭代次数 T.

输出: $z \sim \widetilde{p}(z)$.

$z_{\text{prev}} \leftarrow z^{(0)}$;

对 $\tau \in \{1,\cdots,T\}$:

$k \leftarrow M(\tau)$;

$z^\star \sim q_k\left(z \mid z_{\text{prev}}\right)$;

$u \sim \mathcal{U}(0,1)$;

若 $\widetilde{p}\left(z^\star\right) q_k\left(z_{\text{prev}} \mid z^\star\right) / \left[\widetilde{p}\left(z_{\text{prev}}\right) q_k\left(z^\star \mid z_{\text{prev}}\right)\right] > u$, 则 $z_{\text{prev}} \leftarrow z^\star$; 否则保持不变;

返回 z_{prev}(即 $z^{(T)}$).

我们可以通过证明(14.36)定义的详细平衡条件得以满足, 从而进一步证明 $p(z)$ 是由 Metropolis-Hastings 算法所定义的 Markov 链的不变分布. 利用(14.40), 我们有

$$\begin{aligned} p(z)\, q_k\left(z' \mid z\right) A_k\left(z, z'\right) &= \min\left(p(z) q_k\left(z' \mid z\right),\, p\left(z'\right) q_k\left(z \mid z'\right)\right) \\ &= \min\left(p\left(z'\right) q_k\left(z \mid z'\right),\, p(z) q_k\left(z' \mid z\right)\right) \\ &= p\left(z'\right) q_k\left(z \mid z'\right) A_k\left(z', z\right). \end{aligned} \tag{14.41}$$

这个表达式在交换 z 和 $z^\star$ 时是对称的, 因此详细平衡成立.

命题 14.12 (Metropolis-Hastings 的细致平衡)

设 $p(z) = \tilde{p}(z)/Z_p$，转移核为「以 q_k 提议、以(14.40)接受，拒绝时原地不动」，并记 $T_k(z,z') = q_k(z' \mid z)\, A_k(z,z')(z \neq z')$. 则 p 与 T_k 满足详细平衡(14.36)，从而 p 是链的不变分布.

证明

对 $z \neq z'$，把(14.40)代入并利用 $\tilde{p} = Z_p p$:

$$\begin{aligned} p(z)T_k(z,z') &= p(z)q_k(z' \mid z) \min\left(1, \frac{\tilde{p}(z')\, q_k(z \mid z')}{\tilde{p}(z) q_k(z' \mid z)}\right) \\ &= \min\left(p(z)q_k(z' \mid z)\,,\, p(z')\, q_k(z \mid z')\right), \end{aligned}$$

其中 Z_p 在第二个等式中相消. 右端关于 z 与 z' 对称，故 $p(z)T_k(z,z') = p(z')\, T_k(z',z)$. 对角项 $z = z'$ 两侧相同，因此(14.36)成立. 最后由命题 14.10,p 不变.

□

注

(重构新增) 证明的全部内容是「min 的两个自变量互换」: 接受率被设计成使两个方向的流量相等，因此任何满足该接受率的提议分布都自动正确. 这解释了为什么 Metropolis-Hastings 不需要知道 Z_p——它只需要比值 $\tilde{p}(z')/\tilde{p}(z)$，而这个比值与 Z_p 无关. 注意证明中把 $z \neq z'$ 与 $z = z'$ 分开处理: 对角项来自「拒绝时原地不动」，它保证 T 的行和为 1(见命题 14.10的 remark). 若提议分布对称，$q_k(z \mid z')/q_k(z' \mid z) = 1$，接受率退化为 Metropolis 形式(14.29)，因此 Metropolis 是 Metropolis-Hastings 的特例.

提议分布的具体选择对算法的性能有显著影响. 对于连续状态空间，一个常见的选择是以当前状态为中心的 Gauss 分布，在确定此分布的方差时有一个重要的权衡——如果方差较小，则接受转换的比例将会很高，但通过状态空间的过程表现为缓慢的随机游走，导致相关时间较长. 然而，如果方差较大，则拒绝率将会很高，因为在我们考虑的复杂问题中，许多被提出的步骤将处于概率 $p(z)$ 较低的状态. 考虑一个有强相关性分量的多变量分布 $p(z)$，如图 14.10所示. 提议分布的尺度 ρ 应尽可能大，同时不至于引起高拒绝率. 这表明 ρ 应该与最小的长度尺度 $\sigma_{\min}$ 同一个数量级. 随后系统通过随机游走沿着更延长的方向探索分布，因此到达一个与原始状态或多或少独立的状态所需的步长是 $(\sigma_{\max}/\sigma_{\min})^2$ 数量级的. 实际上，在二维问题中，随着 ρ 增加而导致的拒绝率增加已被接受转换的更大步长抵消. 更一般地说，对于多元 Gauss 分布，获得独立样本所需的步数和 $(\sigma_{\max}/\sigma_2)^2$ 成正比，其中 σ_2 是第二小的标准差 (Neal, 1993). 撇开这些细节不谈，如果分布变化的长度尺度在不同方向上差异很大，那么 Metropolis-Hastings 算法可能会收敛得非常慢.

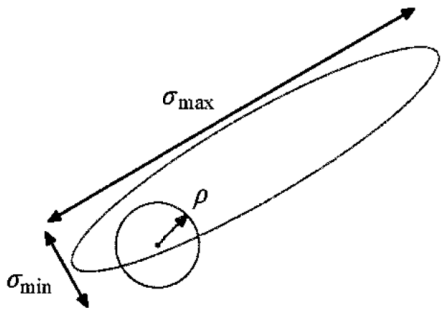


图 14.10: 使用各向同性的提议分布 (蓝色圆圈) 从相关多元 Gauss 分布 (红色椭圆) 中进行 Metropolis-Hastings 采样的示意图，后者在不同方向上具有完全不同的标准差. 为了保持较低的拒绝率，提议分布的尺度 σ 应该与最小标准差 $\sigma_{\min}$ 的数量级相当，这导致了随机游走行为，其中大致独立的状态之间分隔的步数是 $(\sigma_{\max}/\sigma_{\min})^2$ 阶的，其中 $\sigma_{\max}$ 是最大的标准差

14.2.4 Gibbs 采样

Gibbs 采样 (Geman and Geman, 1984) 是一种简单且使用广泛的采样策略, 可以看作 Metropolis-Hastings 算法的一个特例. 考虑分布 $p(\boldsymbol{z}) = p(z_1, \cdots, z_M)$, 我们希望从中采样, 假设我们已经为 Markov 链选择了一些初始状态.Gibbs 采样过程中的每一步都包含用我们从一个变量的条件分布中抽取的值替换该变量的值, 该条件分布是以其余变量的值为条件的. 因此, 可通过从分布 $p\left(z_i \mid \boldsymbol{z}_{\setminus i}\right)$ 中采样来替换 z_i, 其中 $\boldsymbol{z}_{\setminus i}$ 表示 $\{z_1, \cdots, z_M\}$ 但其中不包括 z_i, 这个过程要么按某种特定顺序循环遍历变量, 要么在每一步随机从某些分布中选择要更新的变量.

定义 14.13 (Gibbs 采样)

设 $p(\boldsymbol{z}) = p(z_1, \cdots, z_M)$ 为目标分布,$\boldsymbol{z}^{(0)}$ 为初始状态.Gibbs 采样按如下方式迭代: 在第 τ 步依次 (或按某个次序) 对每个分量 $i \in \{1, \cdots, M\}$, 从条件分布

$$p\left(z_i \mid \boldsymbol{z}_{\setminus i}\right) = p(z_i \mid z_1, \cdots, z_{i-1}, z_{i+1}, \cdots, z_M)$$

中抽取新值替换 z_i, 其余分量保持不变.

♣

注

(重构新增)Gibbs 采样的动机是「把联合采样化为一维条件采样」: 直接从 $p(\boldsymbol{z})$ 采样困难, 但从 $p\left(z_i \mid \boldsymbol{z}_{\setminus i}\right)$ 采样往往容易 (例如在图模型中,z_i 的条件分布只依赖其 Markov 毯, 见图 14.12). 它有两个前提: 其一, 所有条件分布都必须可以采样, 若某个条件分布本身难以采样,Gibbs 就无法进行 (可改用 Metropolis-Hastings 提议); 其二, 条件分布必须在支撑集上无零概率区域, 否则链被限制在状态空间的一部分、遍历性失效 (见练习 14.13). 定义中「依次」与「随机选择」两种扫描方式都正确, 因为它们分别对应(14.38)与(14.39)的转移核; 实践中按固定次序扫描更常用, 但随机扫描更容易保证可逆性.

例如, 假设我们有一个关于 3 个变量的分布 $p(z_1, z_2, z_3)$, 在时间步 τ, 我们选定了值 $z_1^{(\tau)}$、$z_2^{(\tau)}$ 和 $z_3^{(\tau)}$. 首先将 $z_1^{(\tau)}$ 替换为通过从如下条件分布中采样获得的新值 $z_1^{(\tau+1)}$:

$$p\left(z_1 \mid z_2^{(\tau)}, z_3^{(\tau)}\right). \tag{14.42}$$

然后将 $z_2^{(\tau)}$ 替换为通过从如下条件分布中采样获得的新值 $z_2^{(\tau+1)}$:

$$p\left(z_2 \mid z_1^{(\tau+1)}, z_3^{(\tau)}\right). \tag{14.43}$$

这样新的 z_1 值便可立即在后续的采样步骤中使用. 用我们从如下条件分布中抽取的样本 $z_3^{(\tau+1)}$ 更新 $z_3^{(\tau)}$:

$$p\left(z_3 \mid z_1^{(\tau+1)}, z_2^{(\tau+1)}\right). \tag{14.44}$$

以此类推, 依次循环遍历这 3 个变量.

算法 14.3:Gibbs 采样

输入: 初始值 $\{z_i : i \in \{1, \cdots, M\}\}$; 条件分布 $\left\{p\left(z_i \mid \{z_j\}_{j \neq i}\right) : i \in \{1, \cdots, M\}\right\}$; 迭代次数 T.
输出: 最终值 $\{z_i : i \in \{1, \cdots, M\}\}$.
对 $\tau \in \{1, \cdots, T\}$:
对 $i \in \{1, \cdots, M\}$:
从 $p\left(z_i \mid \{z_j\}_{j \neq i}\right)$ 中采样 z_i;
返回 $\{z_i : i \in \{1, \cdots, M\}\}$.

我们可以将 Gibbs 采样过程看作是 Metropolis-Hastings 算法的一个特定实例. 考虑一个涉及变量 z_i 的 Metropolis-Hastings 采样步骤, 其余变量 $\boldsymbol{z}_{\setminus i}$ 保持不变, 从 $\boldsymbol{z}$ 到 $\boldsymbol{z}^\star$ 的转移概率由 $q_k\left(\boldsymbol{z}^\star \mid \boldsymbol{z}\right) = p\left(z_i^\star \mid \boldsymbol{z}_{\setminus i}\right)$ 给出. 注意 $\boldsymbol{z}_{\setminus i}^\star = \boldsymbol{z}_{\setminus i}$, 因为这些分量在采样步骤中未发生改变. 同样,$p(\boldsymbol{z}) = p\left(z_i \mid \boldsymbol{z}_{\setminus i}\right) p\left(\boldsymbol{z}_{\setminus i}\right)$, 因此,Metropolis-Hastings 算法(14.40)中确定接受概率的因子由下式给出:

$$\begin{aligned} A\left(\boldsymbol{z}^\star, \boldsymbol{z}\right) &= \frac{p\left(\boldsymbol{z}^\star\right) q_k\left(\boldsymbol{z} \mid \boldsymbol{z}^\star\right)}{p(\boldsymbol{z}) q_k\left(\boldsymbol{z}^\star \mid \boldsymbol{z}\right)} \\ &= \frac{p\left(z_i^\star \mid \boldsymbol{z}_{\setminus i}^\star\right) p\left(\boldsymbol{z}_{\setminus i}^\star\right) p\left(z_i \mid \boldsymbol{z}_{\setminus i}^\star\right)}{p\left(z_i \mid \boldsymbol{z}_{\setminus i}\right) p\left(\boldsymbol{z}_{\setminus i}\right) p\left(z_i^\star \mid \boldsymbol{z}_{\setminus i}\right)} = 1, \end{aligned} \tag{14.45}$$

其中 $z^{\star}_{\setminus i} = z_{\setminus i}$. 因此,Metropolis-Hastings 采样步骤总是可以接受.

命题 14.13 (Gibbs 采样是 Metropolis-Hastings 的特例)

以 $q_k(z^{\star} \mid z) = p(z_i^{\star} \mid z_{\setminus i})$ 为提议分布、其余分量保持不变的 Metropolis-Hastings 步骤, 其接受概率恒为 1, 即(14.45)成立.

证明

因 $z^{\star}_{\setminus i} = z_{\setminus i}$, 有 $p(z^{\star}_{\setminus i}) = p(z_{\setminus i})$, 且 $p(z_i \mid z^{\star}_{\setminus i}) = p(z_i \mid z_{\setminus i})$、$p(z_i^{\star} \mid z_{\setminus i}) = p(z_i^{\star} \mid z^{\star}_{\setminus i})$. 把这些代入(14.40)的比值, 分子与分母逐项相同, 故比值等于 1, 接受概率 $\min(1,1) = 1$.

□

注

(重构新增) 这一命题说明 Gibbs 采样「无需拒绝」: 提议分布恰好是目标分布的一个条件分布, 于是(14.45)中的四个因子两两相消. 这带来两个好处: 每一步都产生新样本 (没有浪费), 且不需要调节步长; 代价是每个样本只改变一个分量, 当分量之间高度相关时, 链只能沿坐标方向移动, 形成缓慢的锯齿状随机游走 (见图 14.11). 命题也说明了 Gibbs 的适用边界: 只有当所有条件分布都能精确采样时, 接受率才恒为 1; 若用近似方法从条件分布采样, 则必须退回 Metropolis-Hastings 并显式使用其接受率.

为了证明该过程从目标分布中采样, 我们首先注意到分布 $p(z)$ 是 Gibbs 采样步骤中的一个单独的不变量, 因此它也是整个 Markov 链的不变量. 这是因为当我们从 $p(z_i \mid z_{\setminus i})$ 中采样时, 边缘分布 $p(z_{\setminus i})$ 显然是不变的, 因为 $z_{\setminus i}$ 的值没有发生改变. 同样, 定义的每一步采样都从正确的条件分布 $p(z_i \mid z_{\setminus i})$ 中采样. 因为这些条件分布和边缘分布共同指定了联合分布, 所以我们可以看到联合分布本身是不变的.

命题 14.14 (Gibbs 采样的不变性)

设 $p(z) > 0$ 在状态空间上成立. 则对每个分量 i, 以 $p(z_i \mid z_{\setminus i})$ 为条件分布的 Gibbs 更新以 p 为不变分布; 从而整个 Gibbs 扫描 (无论按固定次序还是随机次序) 也以 p 为不变分布.

证明

记 $z = (z_i, z_{\setminus i})$.Gibbs 更新把 z_i 换成 $z_i' \sim p(\cdot \mid z_{\setminus i})$, 故更新后的联合密度为

$$p(z_i', z_{\setminus i}) = p(z_i' \mid z_{\setminus i})\, p(z_{\setminus i}) = p(z_i', z_{\setminus i}),$$

即 p 不变. 若按固定次序依次更新全部分量, 则每一步都不变 p, 复合后仍不变 p; 若随机挑选分量, 则由命题 14.11, 混合后的转移核也不变 p.

□

注

(重构新增) 证明的关键是「条件分布乘边缘分布等于联合分布」:Gibbs 更新只重新抽取 z_i, 而新值恰好来自联合分布关于 z_i 的条件, 因此更新后的联合密度与原联合密度逐点相等——这比 Metropolis-Hastings 的「流量相等」更强, 是逐点的等式. 条件 $p(z) > 0$ 保证所有条件分布有定义 (否则 $p(z_i \mid z_{\setminus i})$ 可能分母为零). 注意这里论证的是不变性而非遍历性: 若状态空间被零概率区域分割, 不变性仍成立但链可能只在其中一个连通块内游走.

为了完整地描述该算法, 还需要指定初始状态的分布, 尽管在多次迭代后抽取的样本实际上独立于此分布. 当然, 从 Markov 链中连续抽取的样本将高度相关, 因此要获得几乎独立的样本, 就需要对序列进行子样本采样.

确保能从正确分布中进行 Gibbs 采样的第二个条件是要具有遍历性 (ergodic). 具有遍历性的一个充分条件是条件分布不存在零概率区域. 如果是这种情况, 那么 z 空间中的任意一点都可以通过有限步骤到达其他任意一点, 每一步都包括更新每个分量变量中的一个. 如果不满足这个要求, 即某些条件分布存在零概率区域, 那么遍历性 (如果适用) 必须被显式地证明.

与 Metropolis 算法类似, 我们可以通过研究 Gibbs 采样在特定分布 (例如 Gauss 分布) 上的应用来深入了解其行为. 考虑图 14.11所示的两个变量中相关 Gauss 分布, 其条件分布的宽度为 l, 边缘分布的宽度为 L. 典型的步长由条件分布决定, 并且将是 l 阶的. 由于状态根据随机游走演化, 因此为了获得来自分布的独立样本, 所需的步数

将是 $(L/l)^2$ 阶的. 当然, 如果 Gauss 分布是不相关的, 那么 Gibbs 采样过程将是最优的. 对于这个简单问题, 我们可以旋转坐标系, 使得新变量不相关. 但是, 在实际应用中, 我们通常不可能找到这样的转换.

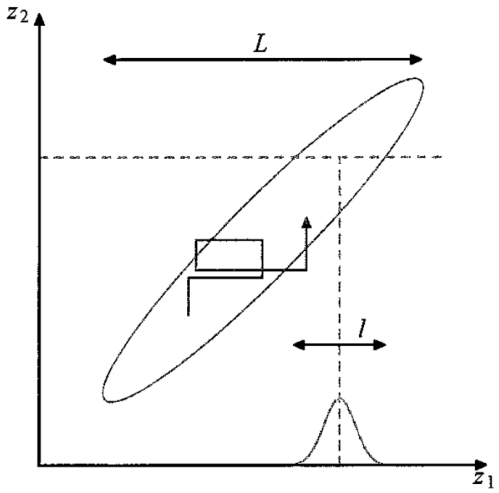


图 14.11: 交替更新两个变量的 Gibbs 采样示意图, 这两个变量的分布是相关 Gauss 分布. 步长由条件分布 (绿色曲线) 的标准差控制, 为 $O(l)$, 这导致在联合分布 (红色椭圆) 拉长方向上的进展缓慢. 从分布中获得一个独立样本所需的步数是 $O\left((L/l)^2\right)$

由于基本的 Gibbs 采样技术一次只考虑一个变量, 因此连续样本之间存在强依赖关系. 在另一种极端情况下, 如果我们能够直接从联合分布中采样 (实际上这种操作很难实现), 则连续样本将是独立的. 我们可以采用一种折中策略来改进简单的 Gibbs 采样, 即连续地从变量组而不是单个变量中采样. 这可以通过分块 Gibbs(blocking Gibbs) 采样算法实现, 该算法选择变量块 (不一定是互斥的), 然后依次对每个块中的变量进行联合采样, 条件是其他所有变量保持其当前值 (Jensen, Kong, and Kjaerulff, 1995).

减少 Gibbs 采样中随机游走行为的一种方法称为超松弛 (over-relaxation)(Adler, 1981). 在它的原始形式中, 它适用于条件分布是 Gauss 分布的问题, 就是说,「条件分布为 Gauss 分布」所涵盖的分布类型比「多元 Gauss 分布」更广泛. 例如, 非 Gauss 分布 $p(z,y) \propto \exp\left(-z^2y^2\right)$ 具有条件 Gauss 分布. 在 Gibbs 采样的每一步, 特定分量 z_i 具有均值为 μ_i、方差为 σ_i^2 的条件分布. 在超松弛框架中,z_i 的值被替换为

$$z_i' = \mu_i + \alpha_i\left(z_i - \mu_i\right) + \sigma_i\left(1-\alpha_i^2\right)^{1/2}\nu, \tag{14.46}$$

其中 ν 是一个有零均值和单位方差的 Gauss 随机变量,α 是一个参数, 满足 $-1 < \alpha < 1$. 当 $\alpha = 0$ 时, 该方法等同于标准的 Gibbs 采样; 当 $\alpha < 0$ 时, 该步骤偏向于均值的相反侧. 这一步骤保持了目标分布的不变性, 因为如果 z_i 有均值 μ_i 和方差 σ_i^2, 那么 z_i' 也将如此 (见练习 14.14). 超松弛的作用是在变量高度相关时, 鼓励状态空间中的定向运动. 有序超松弛框架 (ordered over-relaxation)(Neal, 1999) 将这种方法推广到了非 Gauss 分布.

命题 14.15 (超松弛更新保持条件分布)

设给定其余分量时 z_i 的条件分布为 $\mathcal{N}\left(\mu_i,\sigma_i^2\right)$,$\nu \sim \mathcal{N}(0,1)$ 且与 z_i 独立,$|\alpha_i| < 1$,z_i' 由(14.46)定义. 则 z_i' 的条件分布仍为 $\mathcal{N}\left(\mu_i,\sigma_i^2\right)$.

证明

记 $\xi = z_i - \mu_i$, 则 $\xi \sim \mathcal{N}\left(0, \sigma_i^2\right)$. 由(14.46),

$$z_i' - \mu_i = \alpha_i \xi + \sigma_i \left(1 - \alpha_i^2\right)^{1/2} v.$$

由独立性,$\mathbb{E}\left[z_i' - \mu_i\right] = \alpha_i \cdot 0 + 0 = 0$, 且

$$\operatorname{var}\left[z_i'\right] = \alpha_i^2 \sigma_i^2 + \sigma_i^2 \left(1 - \alpha_i^2\right) = \sigma_i^2.$$

两个独立 Gauss 变量的线性组合仍为 Gauss 变量, 故 $z_i' \sim \mathcal{N}\left(\mu_i, \sigma_i^2\right)$.

□

注

(重构新增) 超松弛的技巧是「保留均值与方差、但把新值推到均值的另一侧」:$\alpha < 0$ 时 $z_i' - \mu_i$ 与 $z_i - \mu_i$ 反号, 于是链不再沿坐标方向做小步随机游走, 而是跨过均值继续前进, 从而削弱相邻样本的相关性. 条件 $|\alpha_i| < 1$ 保证 $\left(1 - \alpha_i^2\right)^{1/2}$ 为实数且方差为正;$\alpha = 1$ 时更新退化为 $z_i' = \mu_i + z_i - \mu_i = z_i$(不动),$\alpha = -1$ 时退化为关于均值的镜像反射. 命题只说明了条件分布不变, 由于 Gibbs 的每一步都是条件更新, 这与联合分布不变等价 (见命题 14.14). 注意该方法要求条件分布为 Gauss 且能算出 μ_i, σ_i^2, 对一般分布需改用有序超松弛.

Gibbs 采样的实用性取决于从条件分布 $p\left(z_i \mid z_{\setminus i}\right)$ 中采样的难度. 对于使用有向图模型表示的概率分布, 单个节点的条件分布仅依赖于相应 Markov 毯中的变量, 如图 14.12所示. 对于有向图, 各个节点的基于其父节点的条件分布的广泛选择将导致 Gibbs 采样的条件分布是对数凹的. 因此,小节 14.1.4讨论的自适应拒绝采样方法为从有向图中进行 Monte Carlo 采样提供了一个具有广泛适用性的框架.

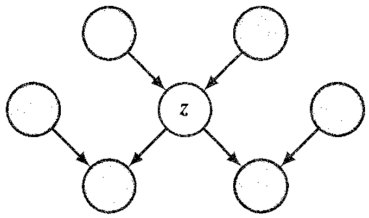


图 14.12: Gibbs 采样需要从一个变量 z 的条件分布中抽取样本, 该条件分布是基于其余变量的. 对于有向图模型, 这个条件分布只是 Markov 毯中节点状态的函数,Markov 毯在图中以蓝色阴影显示, 它包括了父节点、子节点以及共同父节点

定义 14.14 (Markov 毯)

在有向图模型中, 节点 i 的 Markov 毯 (Markov blanket) 由 i 的父节点、子节点以及子节点的其他父节点组成. 给定 Markov 毯中所有变量的取值,z_i 与图中其余变量条件独立, 即 $p\left(z_i \mid z_{\setminus i}\right) = p(z_i \mid \mathrm{mb}(i))$, 其中 $\mathrm{mb}(i)$ 表示 Markov 毯中的变量集合.

♣

注

(重构新增)Markov 毯把「计算条件分布需要哪些变量」这一全局问题化为局部问题: 对 Gibbs 采样而言, 更新 z_i 时只需知道其 Markov 毯的取值, 而不必访问整张图, 这使得高维图模型上的 Gibbs 采样在计算上可行. 毯中之所以还要包含「子节点的其他父节点」, 是因为这些变量与 z_i 通过共同子节点产生依赖 (解释消除效应); 若漏掉它们, 条件分布会被错误地简化. 对无向图模型,Markov 毯就是与节点相邻的所有节点, 这一差别源于有向图与无向图的因子分解方式不同.图 14.12中的蓝色阴影即示意这一集合.

14.2.5 祖先采样

对于许多模型来说, 联合分布 $p(z)$ 可以方便地用图模型的术语来表示. 对于一个没有观测变量的有向图, 可以直接使用祖先采样 (ancestral sampling) 方法从联合分布中采样. 联合分布由下式指定:

$$p(z)=\prod_{i=1}^{M} p\left(z_i \mid \mathrm{pa}(i)\right), \tag{14.47}$$

其中 z_i 是与节点 i 关联的一组变量,pa(i) 表示与节点 i 的父节点关联的一组变量.

定义 14.15 (祖先采样)

设联合分布具有(14.47)的因子分解形式. 按拓扑序 $z_1,\cdots,z_M$ 遍历各节点, 依次从条件分布 $p\left(z_i \mid \mathrm{pa}(i)\right)$ 中抽取样本, 则得到的 $(z_1,\cdots,z_M)$ 是联合分布 $p(z)$ 的一个样本. 这一方法称为祖先采样 (ancestral sampling). ♣

注

(重构新增) 祖先采样之所以正确, 是因为拓扑序保证每一步的所有父变量都已被实例化, 条件分布 $p\left(z_i \mid \mathrm{pa}(i)\right)$ 因而总是有定义的; 把各步抽样的条件密度相乘, 恰好得到(14.47), 即联合分布. 它要求每一步都能从条件分布中采样, 并且图中没有观测变量; 一旦存在证据 (被观测的节点), 条件分布就变成后验分布, 祖先采样不能直接使用, 必须引入重要性加权 (见下文似然加权采样). 祖先采样的计算代价是 $O(M)$ 次条件采样, 是图模型中最快的精确采样方法, 但它只适用于有向无环图.

为了获得联合分布的一个样本, 我们按照 $z_1,\cdots,z_M$ 的顺序遍历这组变量, 并从条件分布 $p\left(z_i \mid \mathrm{pa}(i)\right)$ 中采样. 这总是可能的, 因为在每个步骤, 所有父变量都会被实例化. 遍历图一次后, 我们将获得联合分布的一个样本. 这里假设我们可以从每个节点的各条件分布中采样.

考虑这样一个有向图, 其部分节点组成了证据集 ε, 并被观测值实例化过. 原则上可以扩展上述过程, 至少对于代表离散变量的节点, 我们可以给出以下逻辑采样方法 (Henrion, 1988), 它可以看作重要性采样的一种特殊情况 (见小节 14.1.5). 在每一步, 当为一个观测值已知的变量 z_i 获得一个采样值时, 对采样值与观测值进行比较: 如果它们一致, 则保留采样值, 并继续依次处理下一个变量; 然而, 如果它们不一致, 则丢弃到目前为止的整个样本, 重新从图的第一个节点开始. 这种算法可以正确地从后验分布中采样, 因为它仅仅相当于从隐变量和数据变量的联合分布中抽取样本, 然后丢弃那些与观测数据不一致的样本 (在发现一个不一致的值后就不再从联合分布中继续采样, 从而稍微提高了效率). 然而, 随着观测变量数量的增加和这些变量可以取的状态数量的增加, 从后验分布中接受一个样本的整体概率迅速降低, 因此这种方法在实践中很少使用.

这种方法的改进版称为似然加权采样 (likelihood weighted sampling)(Fung and Chang, 1990; Shachter and Peot, 1990), 它基于祖先采样并结合了重要性采样. 对于每一个变量, 如果该变量在证据集中, 那么把它设置为其实例化的值. 如果它不在证据集中, 则从条件分布 $p\left(z_i \mid \mathrm{pa}(i)\right)$ 中采样, 其中条件变量设置为其当前采样的值. 随后产生的样本 z 的权重由下式给出 (见练习 14.15):

$$r(z)=\prod_{z_i \notin \varepsilon} \frac{p\left(z_i \mid \mathrm{pa}(i)\right)}{p\left(z_i \mid \mathrm{pa}(i)\right)} \prod_{z_i \in \varepsilon} \frac{p\left(z_i \mid \mathrm{pa}(i)\right)}{1}=\prod_{z_i \in \varepsilon} p\left(z_i \mid \mathrm{pa}(i)\right). \tag{14.48}$$

定义 14.16 (似然加权采样)

给定有向图模型、证据集 ε 与观测值 $\hat{z}_i (i \in \varepsilon)$. 似然加权采样按拓扑序处理各节点: 若 $z_i \in \varepsilon$, 则令 $z_i = \hat{z}_i$; 否则从 $p\left(z_i \mid \mathrm{pa}(i)\right)$ 中采样. 最终样本 $\boldsymbol{z}$ 附带权重(14.48).

♣

注

(重构新增) 似然加权的动机是「不丢弃任何样本」: 逻辑采样在证据与采样值不一致时整条样本作废, 接受率随证据变量个数与状态数指数下降; 似然加权则把证据节点固定为观测值、把它本应出现的先验概率记入权重, 于是每条样本都被保留, 权重承担了「这条样本在证据下的合理性」. 它本质上是重要性采样, 提议分布取为「先验 + 证据上的 delta」, 目标分布取为后验. 使用条件有三: 其一, 必须能从每个非证据节点的先验条件分布 $p\left(z_i \mid \mathrm{pa}(i)\right)$ 中采样, 这与祖先采样相同; 其二, 对后验支撑上的任意取值都应有 $p\left(z_i \mid \mathrm{pa}(i)\right) > 0$, 否则权重为零或未定义, 后验的相应区域被系统性漏掉; 其三, 权重只依赖证据节点, 因此其方差通常远小于逻辑采样的接受率倒数. 当证据很多且每条证据的概率都很小时, 权重会高度集中, 有效样本量下降, 这是似然加权的主要局限.

命题 14.16 (似然加权采样的权重)

在定义 14.16的设定下, 若 $q(\boldsymbol{z})$ 表示该采样过程产生的分布, 则后验分布 $p(\boldsymbol{z} \mid \varepsilon)$ 关于 q 的重要性权重为(14.48), 即

$$\frac{p(\boldsymbol{z} \mid \varepsilon)}{q(\boldsymbol{z})} \propto \prod_{z_i \in \varepsilon} p\left(z_i \mid \mathrm{pa}(i)\right).$$

♠

证明

采样过程对非证据节点按先验抽样、对证据节点取固定值, 故

$$q(\boldsymbol{z}) = \prod_{z_i \notin \varepsilon} p\left(z_i \mid \mathrm{pa}(i)\right) \prod_{z_i \in \varepsilon} \delta(z_i - \hat{z}_i).$$

另一方面, 由(14.47), 联合分布为 $p(\boldsymbol{z}) = \prod_i p\left(z_i \mid \mathrm{pa}(i)\right)$, 而后验分布 $p(\boldsymbol{z} \mid \varepsilon) = p(\boldsymbol{z}) \mathbb{I}(\boldsymbol{z}_\varepsilon = \hat{\boldsymbol{z}}_\varepsilon) / p(\varepsilon)$ 与 $p(\boldsymbol{z}) \mathbb{I}(\boldsymbol{z}_\varepsilon = \hat{\boldsymbol{z}}_\varepsilon)$ 成比例. 于是

$$\frac{p(\boldsymbol{z} \mid \varepsilon)}{q(\boldsymbol{z})} \propto \frac{\prod_{z_i \notin \varepsilon} p\left(z_i \mid \mathrm{pa}(i)\right) \prod_{z_i \in \varepsilon} p\left(\hat{z}_i \mid \mathrm{pa}(i)\right)}{\prod_{z_i \notin \varepsilon} p\left(z_i \mid \mathrm{pa}(i)\right) \prod_{z_i \in \varepsilon} \delta(z_i - \hat{z}_i)},$$

在 $\boldsymbol{z}_\varepsilon = \hat{\boldsymbol{z}}_\varepsilon$ 上两个 δ 因子相消, 非证据节点的因子相消, 剩下的正是 $\prod_{z_i \in \varepsilon} p\left(\hat{z}_i \mid \mathrm{pa}(i)\right)$, 即(14.48).

□

注

(重构新增) 似然加权采样的意思是「照常按先验采样, 但把证据节点的采样概率记在权重里」: 证据节点不再被采样, 而是被固定, 于是它们的先验概率不再出现在样本分布 q 中, 却仍是后验分布的一部分, 权重正好补上这一缺口. 与逻辑采样相比, 它不会因为证据与采样值不一致而丢弃整条样本, 因此效率高得多; 代价是权重可能很小(当证据在模型下概率很低时), 有效样本量随之下降. 命题中的 δ 记号是连续情形下的形式写法, 离散情形应理解为指示函数 $\mathbb{I}(z_i = \hat{z}_i)$; 无论哪种情形, 结论都成立. 这一权重形式与小节 14.1.5的重要性权重完全一致, 只是在图模型上被因式分解了.

这种方法可以进一步使用自重要性采样 (self-importance sampling)(Shachter and Peot, 1990) 扩展, 其中重要性采样分布不断更新, 以反映当前估计的后验分布.

14.3 Langevin 采样

Metropolis-Hastings 算法通过使用提议分布创建候选样本的 Markov 链, 并使用(14.40)所示的准则接受或拒绝它们, 从而从概率分布中抽取样本. 这可能相对低效, 因为提议分布通常是一个简单且固定的分布, 它可以在数据空间的任何方向上生成更新, 从而导致随机游走的行为.

我们已经看到, 在训练神经网络时, 利用关于模型可学习参数的对数似然梯度来最大化似然函数是极其有利的. 类似地, 我们可以引入利用关于数据向量的概率密度梯度的 Markov 链采样算法, 以便优先向概率较大的区域移动. 这种技术称为 Hamiltonian Monte Carlo 采样, 也称混合 Monte Carlo(hybrid Monte Carlo) 采样. 该采样方法使用了 Metropolis 接受准则 (Duane et al., 1987; Bishop, 2006). 在这里, 我们研究一种在深度学习中使用广泛的新的采样方法——Langevin 采样. 尽管它避免了使用 Metropolis 接受准则, 但我们必须仔细设计算法, 以确保产生的结果样本是无偏的. 在基于能量函数的机器学习模型中,Langevin 采样发挥了至关重要的作用.

14.3.1 基于能量的模型

许多生成式模型可以表示为条件概率分布 $p(\boldsymbol{x} \mid \boldsymbol{w})$, 其中 $\boldsymbol{x}$ 是数据向量,$\boldsymbol{w}$ 代表可学习参数的向量. 这样的模型可以通过最大化相对于训练集定义的对应似然函数来进行训练. 然而, 为了表示一个有效的概率分布, 模型必须满足如下要求:

$$\int p(\boldsymbol{x} \mid \boldsymbol{w})\, p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x} = 1. \tag{14.49}$$

注

(重构新增) **原文问题说明.** 原书(14.49)印作 $\int p(\boldsymbol{x} \mid \boldsymbol{w}) p(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = 1$, 但归一化要求针对的是条件密度本身, 正确形式应为

$$\int p(\boldsymbol{x} \mid \boldsymbol{w})\, \mathrm{d}\boldsymbol{x} = 1.$$

上式多出的因子 $p(\boldsymbol{x})$ 会使积分依赖于 $\boldsymbol{x}$ 的先验分布, 从而无法对一切 $\boldsymbol{w}$ 恒等于 1, 与「模型是有效概率分布」这一要求不符. 本译本保留原式以忠实于原书, 但本章此后的一切推导 (特别是(14.50)与(14.51)) 均按修正后的归一化形式进行. 这一处属于排版或翻译层面的笔误, 不影响后续数学内容.

这样的归一化要求大大限制了适用模型的形式. 在不考虑归一化约束的情况下, 我们可以考虑一个更广泛的模型类别, 称为基于能量的模型 (LeCun et al., 2006). 假设我们有一个称为能量函数的函数 $E(\boldsymbol{x}, \boldsymbol{w})$, 它是其参数的实值函数, 但没有其他约束. 指数 $\exp\{-E(\boldsymbol{x}, \boldsymbol{w})\}$ 是一个非负数, 因此可以看作未归一化的 $\boldsymbol{x}$ 上的概率分布. 这里在指数中引入负号只是一个惯例, 它意味着能量的较高值对应于概率的较低值. 然后我们可以使用下式定义一个归一化的分布 (见练习 14.16):

$$p(\boldsymbol{x} \mid \boldsymbol{w}) = \frac{1}{Z(\boldsymbol{w})} \exp\{-E(\boldsymbol{x}, \boldsymbol{w})\}, \tag{14.50}$$

其中归一化常数 $Z(\boldsymbol{w})$ 称为配分函数 (partition function), 它由下式定义:

$$Z(\boldsymbol{w}) = \int \exp\{-E(\boldsymbol{x}, \boldsymbol{w})\}\, \mathrm{d}\boldsymbol{x}. \tag{14.51}$$

定义 14.17 (基于能量的模型)

设 $E(\boldsymbol{x}, \boldsymbol{w})$ 是数据向量 $\boldsymbol{x}$ 与参数 $\boldsymbol{w}$ 的实值函数, 称为能量函数, 且

$$0 < Z(\boldsymbol{w}) = \int \exp\{-E(\boldsymbol{x}, \boldsymbol{w})\}\, \mathrm{d}\boldsymbol{x} < \infty.$$

则由(14.50)定义的分布族 $\{p(\boldsymbol{x} \mid \boldsymbol{w})\}$ 称为基于能量的模型 (energy-based model,EBM),$Z(\boldsymbol{w})$ 称为配分函数 (partition function). ♣

注

(重构新增) 基于能量的模型把「概率建模」化为「设计一个打分函数」: 能量 E 可以任意选取 (例如用深度神经网络实现), 只要指数可积,(14.50)就自动给出一个合法的概率分布. 负号约定使「低能量=高概率」, 从而训练可以理解为「给数据点分配低能量、给模型样本分配高能量」. 定义中 $0 < Z(\boldsymbol{w}) < \infty$ 的条件不可省:$Z = 0$ 时密度无定义,$Z = \infty$ 时密度不可归一化, 此时 p 只是「未归一化的分数函数」而非概率分布. 配分函数是全部困难的来源——它是对整个 $\boldsymbol{x}$ 空间 (或全部离散状态) 的积分, 高维时无法精确计算,小节 14.3.2与小节 14.3.3的全部技巧都是为了绕开它. 最后, 任何归一化的概率模型都可以写成(14.50)的形式 (取 $E = -\ln p$), 因此 EBM 是一个包含极广的框架, 其代价是 Z 通常不可解.

定义 14.18 (配分函数)

对能量函数 $E(\boldsymbol{x}, \boldsymbol{w})$, 称

$$Z(\boldsymbol{w}) = \int \exp\{-E(\boldsymbol{x}, \boldsymbol{w})\} \, \mathrm{d}\boldsymbol{x}$$

为配分函数; 离散变量情形下积分替换为求和 $Z(\boldsymbol{w}) = \sum_{\boldsymbol{x}} \exp\{-E(\boldsymbol{x}, \boldsymbol{w})\}$.

♣

注

(重构新增) 配分函数之名来自统计物理:Z 是 Boltzmann 因子 e^{-E} 对所有状态的总和,$\ln Z$ 对应自由能. 它在数学上只起归一化作用, 却出现在对数似然的梯度中 (见(14.53)), 因此无法像对待普通常数那样忽略. 计算 Z 的困难有两个来源: 其一是积分区域是高维的, 网格法的代价随维数指数增长; 其二是 E 通常由神经网络给出, 没有解析结构. 这两种困难正是本章后文用 Markov 链 Monte Carlo 与 Langevin 动力学近似 Z 的梯度 (而非 Z 本身) 的动机. 注意 Z 只依赖 $\boldsymbol{w}$ 而不依赖 $\boldsymbol{x}$, 这一事实在(14.54)中被用来把 $-\nabla_{\boldsymbol{w}} \ln Z$ 移出关于 $\boldsymbol{x}$ 的期望.

能量函数通常使用深度神经网络进行建模, 输入向量为 $\boldsymbol{x}$, 标量输出为 $E(\boldsymbol{x}, \boldsymbol{w})$, 其中 $\boldsymbol{w}$ 代表网络中权重和偏置的集合.

注意配分函数依赖于 $\boldsymbol{w}$, 这给训练带来了一些问题. 例如, 一组独立同分布数据 $\mathcal{D} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 的对数似然函数具有以下形式:

$$\ln p(\mathcal{D} \mid \boldsymbol{w}) = -\sum_{n=1}^{N} E(\boldsymbol{x}_n, \boldsymbol{w}) - N \ln Z(\boldsymbol{w}). \tag{14.52}$$

要计算 $\ln p(\mathcal{D} \mid \boldsymbol{w})$ 关于 $\boldsymbol{w}$ 的梯度, 我们需要知道 $Z(\boldsymbol{w})$ 的形式. 然而, 对于具有许多选择的能量函数 $E(\boldsymbol{x}, \boldsymbol{w})$, 计算配分函数(14.51)是不切实际的, 因为这需要对整个 $\boldsymbol{x}$ 空间进行积分 (或对离散变量求和).「基于能量的模型」指的就是这种积分难以处理的模型. 然而请注意, 概率模型可以视为基于能量的模型的特例, 因此本书讨论的许多模型可以视为基于能量的模型. 基于能量的模型的最大优势是, 它们在不需要归一化的情况下具有灵活性. 相应的缺点是, 由于归一化常数未知, 它们可能更难以训练.

14.3.2 最大化似然

我们已经开发了各种近似方法来训练基于能量的模型, 而无须计算配分函数 (Song and Kingma, 2021). 本小节探讨基于 Markov 链 Monte Carlo 采样的技术. 另一种方法称为分数匹配, 我们将在扩散模型的背景下进行讨论 (见第 20 章).

我们已经看到, 对于基于能量的模型, 由于存在未知的配分函数 $Z(\boldsymbol{w})$, 似然函数不能显式求值. 然而, 我们可以利用 Monte Carlo 采样方法, 来计算对数似然函数关于模型参数的梯度. 一旦以任何方式训练了基于能量的模型, 就可以使用采样方法从模型中抽取样本了, 比如使用 Monte Carlo 采样方法.

使用(14.50), 对于一个基于能量的模型, 对数似然函数关于模型参数的梯度可以写成以下形式:

$$\nabla_{\boldsymbol{w}} \ln p(\boldsymbol{x} \mid \boldsymbol{w}) = -\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w}) - \nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}). \tag{14.53}$$

命题 14.17 (对数似然的梯度)

设 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 由(14.50)给出,E 关于 $\boldsymbol{w}$ 可微, 且可在积分号下求导. 则(14.53)成立.

♠

证明

对(14.50)取对数得 $\ln p(\boldsymbol{x} \mid \boldsymbol{w}) = -E(\boldsymbol{x}, \boldsymbol{w}) - \ln Z(\boldsymbol{w})$, 对 $\boldsymbol{w}$ 求梯度即得(14.53).

□

注

(重构新增) 这个恒等式本身只是「对数把乘积变和」的结果, 它的意义在于把梯度分成两项: 第一项 $-\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})$ 可以直接由自动微分算出; 第二项 $-\nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w})$ 涉及配分函数, 需要借助采样来估计 (见(14.55)). 条件「可在积分号下求导」保证了 $\nabla_{\boldsymbol{w}} Z = \int \nabla_{\boldsymbol{w}} e^{-E}\,\mathrm{d}\boldsymbol{x}$, 在 E 关于 $\boldsymbol{w}$ 局部 Lipschitz、e^{-E} 可积且有可积控制函数时成立, 这是后续一切推导的前提. 若该条件失效 (例如 E 的梯度无界), 梯度与积分不可交换, 公式需要重新论证.

这是针对单个数据点 $\boldsymbol{x}$ 的似然函数, 但实际上我们希望最大化训练数据集上的似然函数, 该数据集是从某个未知分布 $p_D(\boldsymbol{x})$ 中抽样得到的. 如果我们假设数据点是独立同分布的, 那么我们可以考虑与 $p_D(\boldsymbol{x})$ 相关的似然对数期望的梯度, 其由下式给出:

$$\mathbb{E}_{\boldsymbol{x}\sim p_D}\left[\nabla_{\boldsymbol{w}} \ln p(\boldsymbol{x} \mid \boldsymbol{w})\right] = -\mathbb{E}_{\boldsymbol{x}\sim p_D}\left[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right] - \nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}). \tag{14.54}$$

这里我们利用了 $-\nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w})$ 不依赖于 $\boldsymbol{x}$ 的性质, 因而可以把它移到期望之外. 假设配分函数 $Z(\boldsymbol{w})$ 是未知的, 但我们可以利用(14.51)并重组得到

$$-\nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}) = \int \left\{\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right\} p(\boldsymbol{x} \mid \boldsymbol{w})\,\mathrm{d}\boldsymbol{x}. \tag{14.55}$$

命题 14.18 (配分函数对数的梯度)

在命题 14.17的条件下,(14.55)成立, 即

$$-\nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}) = \mathbb{E}_{p(\boldsymbol{x}|\boldsymbol{w})}\left[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right].$$

♠

证明

由 $Z(\boldsymbol{w}) = \int e^{-E(\boldsymbol{x},\boldsymbol{w})}\mathrm{d}\boldsymbol{x}$ 及积分号下求导,

$$\nabla_{\boldsymbol{w}} Z(\boldsymbol{w}) = \int \nabla_{\boldsymbol{w}} e^{-E(\boldsymbol{x},\boldsymbol{w})}\,\mathrm{d}\boldsymbol{x} = -\int \left\{\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right\} e^{-E(\boldsymbol{x},\boldsymbol{w})}\,\mathrm{d}\boldsymbol{x}.$$

两边除以 $Z(\boldsymbol{w})$ 并用(14.50):

$$\nabla_{\boldsymbol{w}} \ln Z(\boldsymbol{w}) = \frac{\nabla_{\boldsymbol{w}} Z(\boldsymbol{w})}{Z(\boldsymbol{w})} = -\int \left\{\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right\} \frac{e^{-E(\boldsymbol{x},\boldsymbol{w})}}{Z(\boldsymbol{w})}\,\mathrm{d}\boldsymbol{x} = -\int \left\{\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right\} p(\boldsymbol{x} \mid \boldsymbol{w})\,\mathrm{d}\boldsymbol{x},$$

移项即得(14.55).

□

注

(重构新增) 这个结果通常称为「对数配分函数的梯度等于能量的负梯度在模型分布下的期望」, 它是全部对比散度方法的核心恒等式: 右侧只涉及模型分布 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 下的期望, 而不涉及 Z 本身, 因此只要能从模型采样就能估计. 证明中「积分号下求导」与「除以 Z」两步分别需要命题 14.17的条件与 $Z > 0$;$Z = 0$ 时对数无定义,$Z = \infty$ 时比值无意义, 这正是定义 14.17要求 $0 < Z < \infty$ 的原因. 注意该式对任意 $\boldsymbol{w}$ 都成立, 并不要求 $\boldsymbol{w}$ 已是最优参数.

(14.55)右侧的部分对应于模型分布 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 的期望, 它由下式给出 (见练习 14.18):

$$\int \left\{\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right\} p(\boldsymbol{x} \mid \boldsymbol{w})\,\mathrm{d}\boldsymbol{x} = \mathbb{E}_{\boldsymbol{x}\sim p_M}\left[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right], \tag{14.56}$$

其中 $p_M(\boldsymbol{x})$ 表示模型分布 $p(\boldsymbol{x} \mid \boldsymbol{w})$. 结合(14.54) ~ (14.56), 我们得到

$$\nabla_{\boldsymbol{w}}\mathbb{E}_{\boldsymbol{x}\sim p_D}\left[\ln p(\boldsymbol{x} \mid \boldsymbol{w})\right] = -\mathbb{E}_{\boldsymbol{x}\sim p_D}\left[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right] + \mathbb{E}_{\boldsymbol{x}\sim p_M(\boldsymbol{x})}\left[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right]. \tag{14.57}$$

定理 14.1 (对比散度梯度)

在命题 14.18的条件下, 记 $p_M = p(\cdot \mid \boldsymbol{w})$, 则对数似然期望的梯度由(14.57)给出.

♡

证明

把(14.55)代入(14.54), 并用(14.56)把积分写成关于 p_M 的期望:

$$\begin{aligned}\nabla_{\boldsymbol{w}}\mathbb{E}_{p_D}\left[\ln p(\boldsymbol{x}\mid\boldsymbol{w})\right] &= -\mathbb{E}_{p_D}\left[\nabla_{\boldsymbol{w}}E(\boldsymbol{x},\boldsymbol{w})\right]-\nabla_{\boldsymbol{w}}\ln Z(\boldsymbol{w})\\ &= -\mathbb{E}_{p_D}\left[\nabla_{\boldsymbol{w}}E(\boldsymbol{x},\boldsymbol{w})\right]+\mathbb{E}_{p_M}\left[\nabla_{\boldsymbol{w}}E(\boldsymbol{x},\boldsymbol{w})\right],\end{aligned}$$

即(14.57).

□

注

(重构新增)(14.57)把梯度写成两个期望之差: 一项在数据分布下、一项在模型分布下, 故称为「正相」与「负相」, 其差也常称为对比散度. 两相的作用相反: 正相 (第一项, 带负号) 降低数据点处的能量, 从而提高数据点的概率; 负相 (第二项, 带正号) 提高模型样本处的能量, 从而压低模型自身产生的样本的概率. 当且仅当 $p_M = p_D$ 时两相相等、梯度为零, 因此该式的驻点条件正是「模型分布与数据分布匹配」. 这一形式要求能从 p_M 采样——而 p_M 恰恰含有未知的 Z, 于是问题转化为「如何从 EBM 采样」, 这正是小节 14.3.3的主题. 注意(14.57)是期望的梯度, 与单个数据点的梯度(14.53)不同, 实际训练用 mini-batch 估计前者.

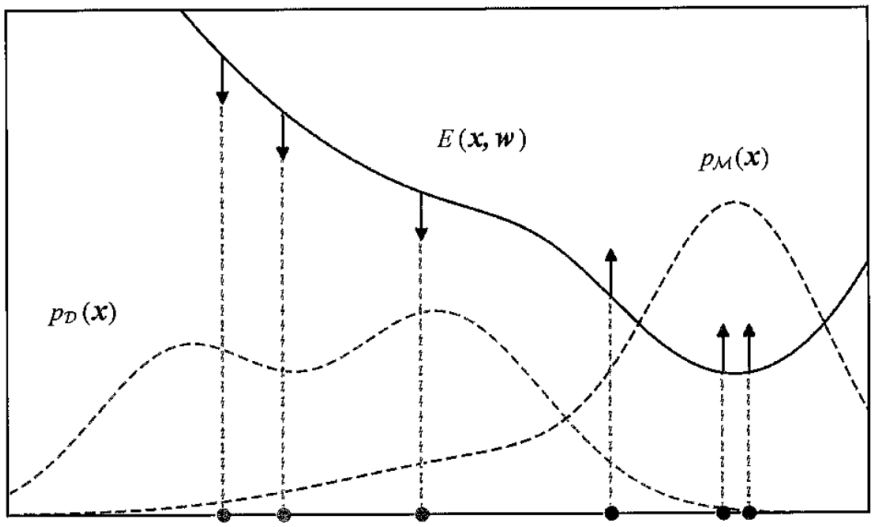


图 14.13: 通过最大化似然对基于能量的模型进行训练的示意图, 其中显示了能量函数 $E(\boldsymbol{x},\boldsymbol{w})$(绿色) 及相关的模型分布 $p_M(\boldsymbol{x})$ 和真实数据分布 $p_D(\boldsymbol{x})$. 利用(14.57)增加期望的对数似然, 会在与模型样本 (用蓝点表示) 对应的点处推高能量函数, 并在与数据集样本 (用红点表示) 对应的点处拉低能量函数

图 14.13对这个结果做了说明, 详细解释如下. 我们的目标是找到参数 $\boldsymbol{w}$ 的值, 让似然函数的值最大化. 为此, 我们朝梯度 $\nabla_{\boldsymbol{w}}\ln p(\boldsymbol{x}\mid\boldsymbol{w})$ 的方向对 $\boldsymbol{w}$ 进行了微小调整. 从(14.57)中可以看出, 这个梯度的期望值可以分解为两部分, 且这两部分符号相反.(14.57)右侧第一项的作用是减小 $E(\boldsymbol{x},\boldsymbol{w})$, 因而会增大模型定义的数据点 $\boldsymbol{x}$ 的概率密度, 这里的 $\boldsymbol{x}$ 是从 $p_D(\boldsymbol{x})$ 中抽取的.(14.57)右侧第二项的作用则是增大 $E(\boldsymbol{x},\boldsymbol{w})$, 从而降低由模型定义的来自模型本身的数据点的概率密度. 在模型密度超过训练数据密度的区域, 净效果将是增加能量, 因此降低了概率密度. 相反, 在训练数据密度超过模型密度的区域, 净效果将是降低能量, 因此增大了概率密度. 正如我们所希望的那样, 这两项一起将概率密度从训练数据密度低的区域移向了训练数据密度高的区域. 当模型分布与数据分布相匹配时, 这两项将在幅度上相等, 此时(14.57)左侧的梯度就等于零.

14.3.3 Langevin 动力学

当把(14.57)作为一种实际的训练方法应用时, 我们需要近似计算(14.57)右侧的两项. 对于任何给定的 $\boldsymbol{x}$ 值, 我们可以使用自动微分来评估 $\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})$. 对于(14.57)中的第一项, 我们可以使用训练集来估计关于 $\boldsymbol{x}$ 的期望:

$$\mathbb{E}_{\boldsymbol{x}\sim p_D}\left[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right] \approx \frac{1}{N}\sum_{n=1}^{N} \nabla_{\boldsymbol{w}} E(\boldsymbol{x}_n, \boldsymbol{w})\,. \tag{14.58}$$

(14.57)中的第二项更具挑战性, 因为我们需要从能量函数定义的模型分布中抽取样本, 而相应的配分函数难以处理. 这可以使用 Markov 链 Monte Carlo 采样来完成. 另一种受欢迎的采样方法称为随机梯度 Langevin 动力学, 简称 Langevin 采样 (Parisi, 1981; Welling and Teh, 2011). 这一项仅通过得分函数 (score function) 依赖于分布 $p(\boldsymbol{x} \mid \boldsymbol{w})$, 得分函数定义为对数似然函数关于数据向量 $\boldsymbol{x}$ 的梯度, 并由下式给出:

$$\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{w}). \tag{14.59}$$

定义 14.19 (得分函数)

设 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 关于 $\boldsymbol{x}$ 可微且为正, 称

$$\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{w})$$

为分布 $p(\cdot \mid \boldsymbol{w})$ 的得分函数 (score function).

♣

注

(重构新增) 得分函数是「对数密度关于数据的梯度」, 与通常的「关于参数 $\boldsymbol{w}$ 的梯度」完全不同: 前者指向「往哪个方向移动 $\boldsymbol{x}$ 能使密度增大」, 后者指向「怎样调整参数能使似然增大」. 这一区分是理解本节的钥匙: 训练需要后者, 而从模型采样需要前者. 得分函数不依赖归一化常数, 因为 $\nabla_{\boldsymbol{x}} \ln Z(\boldsymbol{w}) = 0$($Z$ 与 $\boldsymbol{x}$ 无关), 这使得它成为绕开配分函数的天然工具——它正是扩散模型中分数匹配方法的出发点. 定义要求 $p > 0$ 且可微, 否则对数梯度无定义; 在支撑集边界附近得分函数可能发散, 这是数值实现时需要裁剪的原因之一.

值得强调的是, 这是针对数据点 $\boldsymbol{x}$ 的梯度, 而非通常意义上针对可学习参数 $\boldsymbol{w}$ 的梯度. 将(14.50)代入(14.59), 我们可以得到

$$\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) = -\nabla_{\boldsymbol{x}} E(\boldsymbol{x}, \boldsymbol{w}). \tag{14.60}$$

可以看到, 配分函数不再出现, 因为它与 $\boldsymbol{x}$ 无关.

命题 14.19 (得分函数与能量函数)

设 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 由(14.50)给出,E 关于 $\boldsymbol{x}$ 可微. 则(14.60)成立.

♠

证明

对 $\ln p(\boldsymbol{x} \mid \boldsymbol{w}) = -E(\boldsymbol{x}, \boldsymbol{w}) - \ln Z(\boldsymbol{w})$ 关于 $\boldsymbol{x}$ 求梯度. 由于 $Z(\boldsymbol{w})$ 与 $\boldsymbol{x}$ 无关,$\nabla_{\boldsymbol{x}} \ln Z(\boldsymbol{w}) = \boldsymbol{0}$, 故 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{w}) = -\nabla_{\boldsymbol{x}} E(\boldsymbol{x}, \boldsymbol{w})$.

□

注

(重构新增) 这一恒等式说明 EBM 的得分函数就是能量的负梯度, 因而「沿得分方向爬升」等价于「沿能量下降方向移动」, 这正是下面 Langevin 动力学更新的直观来源. 它同时解释了为什么归一化常数在基于得分的模型中彻底消失:Z 只依赖 $\boldsymbol{w}$, 对 $\boldsymbol{x}$ 求导即为零. 反过来, 这也意味着得分函数只确定 p 的形状而不确定其归一化, 若要从得分恢复密度必须另做归一化 (例如用 Langevin 采样得到样本后估计 Z). 条件「E 关于 $\boldsymbol{x}$ 可微」是必要的, 若 E 含不可微项 (如 ℓ_1 正则), 得分函数在不可微点无定义, 需要次梯度或平滑近似.

让我们从一个先验分布中开始抽取一个初始值 $\boldsymbol{x}^{(0)}$, 然后执行以下 Markov 链步骤:

$$\boldsymbol{x}^{(\tau+1)} = \boldsymbol{x}^{(\tau)} + \eta \nabla_{\boldsymbol{x}} \ln p\left(\boldsymbol{x}^{(\tau)}, \boldsymbol{w}\right) + \sqrt{2\eta}\,\boldsymbol{\varepsilon}^{(\tau)}, \qquad \tau = 1, \cdots, \mathcal{T}, \tag{14.61}$$

其中 $\boldsymbol{\varepsilon}^{(\tau)} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ 是从零均值、单位协方差的 Gauss 分布中独立抽取的样本, 参数 η 控制步长的大小.Langevin

方程的每次迭代都会沿着对数似然的梯度方向迈出一步, 然后添加 Gauss 噪声. 可以证明, 当 $\eta \to 0$ 和 $\mathcal{T} \to \infty$ 时,$\boldsymbol{x}^{(\mathcal{T})}$ 的值是从分布 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 中独立抽取的样本.

命题 14.20 (Langevin 动力学的平稳分布)

设 $p(\boldsymbol{x})$ 为正的光滑概率密度, 考虑随机微分方程

$$\mathrm{d}\boldsymbol{x} = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})\,\mathrm{d}t + \sqrt{2}\,\mathrm{d}\boldsymbol{W}_t,$$

其中 $\boldsymbol{W}_t$ 为标准 Wiener 过程. 则 p 是该扩散过程的平稳分布;(14.61)是它在步长 η 下的 Euler-Maruyama 离散化.

♠

证明

记 $\boldsymbol{b}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$, 设 $\rho(\boldsymbol{x}, t)$ 为 t 时刻的密度, 则 Fokker-Planck 方程为

$$\frac{\partial \rho}{\partial t} = -\nabla_{\boldsymbol{x}} \cdot (\rho\, \boldsymbol{b}) + \Delta_{\boldsymbol{x}} \rho = \nabla_{\boldsymbol{x}} \cdot (\rho\,(-\boldsymbol{b} + \nabla_{\boldsymbol{x}} \ln \rho)).$$

取 $\rho = p$, 则 $-\boldsymbol{b} + \nabla_{\boldsymbol{x}} \ln p = -\nabla_{\boldsymbol{x}} \ln p + \nabla_{\boldsymbol{x}} \ln p = \boldsymbol{0}$, 故 $\partial \rho / \partial t = 0$, 即 p 是平稳解. 另一方面,(14.61)正是该随机微分方程以步长 η 做 Euler-Maruyama 离散、并把 $\sqrt{2\eta}\,\boldsymbol{\varepsilon}$ 解释为 $\sqrt{2}\left(\boldsymbol{W}_{(\tau+1)\eta} - \boldsymbol{W}_{\tau\eta}\right)$ 的结果.

□

注

(重构新增) 证明只用到「Fokker-Planck 算子可以写成散度形式」这一事实: 概率流 $\boldsymbol{J} = \rho\,(\boldsymbol{b} - \nabla \ln \rho)$ 在 $\rho = p$ 时恒为零, 因此 p 既是平稳分布、又满足细致平衡 (扩散过程可逆). 两个条件不能省:$p > 0$ 保证 $\ln p$ 处处有定义,$\nabla \ln p$ 的 Lipschitz 性保证随机微分方程有唯一强解且不会在有限时间爆炸; 若 p 有界支撑, 边界处需要反射或吸收边界条件, 平稳分布的形式会改变. 命题只对连续时间的扩散过程给出严格结论;(14.61)作为离散化, 只在 $\eta \to 0$ 且 $\mathcal{T} \to \infty$ 的极限下收敛到 p, 有限步长会带来 $O(\eta)$ 的偏差 (这正是「必须仔细设计算法以确保无偏」的含义, 也是实践中常对 Langevin 更新加 Metropolis 修正的原因). 此外,(14.61)中写的是 $p\left(\boldsymbol{x}^{(\tau)}, \boldsymbol{w}\right)$, 按(14.50)应理解为条件密度 $p\left(\boldsymbol{x}^{(\tau)} \mid \boldsymbol{w}\right)$; 两者只差一个与 $\boldsymbol{x}$ 无关的因子, 对 $\nabla_{\boldsymbol{x}}$ 无影响.

算法 14.4:Langevin 采样

输入: 初始值 $\boldsymbol{x}^{(0)}$; 概率密度 $p(\boldsymbol{x}, \boldsymbol{w})$; 学习率参数 η; 迭代次数 T.
输出: 最终值 $\boldsymbol{x}^{(T)}$.
$\boldsymbol{x} \leftarrow \boldsymbol{x}^{(0)}$;
对 $\tau \in \{1, \cdots, T\}$:
$\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$;
$\boldsymbol{x} \leftarrow \boldsymbol{x} + \eta \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}, \boldsymbol{w}) + \sqrt{2\eta}\,\boldsymbol{\varepsilon}$;
返回 $\boldsymbol{x}$(即最终值 $\boldsymbol{x}^{(T)}$).

我们可以通过重复这个过程来生成一组样本 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_M\}$, 然后使用下式来近似计算(14.57)中的第二项:

$$\mathbb{E}_{\boldsymbol{x} \sim p_M(\boldsymbol{x})}\left[\nabla_{\boldsymbol{w}} E(\boldsymbol{x}, \boldsymbol{w})\right] \approx \frac{1}{M} \sum_{m=1}^{M} \nabla_{\boldsymbol{w}} E(\boldsymbol{x}_m, \boldsymbol{w}). \tag{14.62}$$

运行长的 Markov 链来生成独立样本的计算成本可能很高, 所以我们需要考虑实际的近似方法. 有一种方法称为对比散度 (contrastive divergence)(Hinton, 2002). 在这里, 用来评估(14.62)的样本是通过从训练数据点 $\boldsymbol{x}_n$ 开始的 Monte Carlo 链获得的. 如果 Monte Carlo 链运行了大量步骤, 那么得出的值基本上就是来自模型分布的无偏样本. 相反, 对比散度方法建议只运行几步的 Monte Carlo 链, 甚至只有一步, 这在计算上成本要低得多. 相应的结果样本将远非无偏样本, 并且会靠近数据流形. 因此, 使用梯度下降的效果将是仅在数据流形的邻域内塑造能量曲面, 从而塑造概率密度. 这对于诸如分类的任务可能是有效的, 但预计在学习生成式模型方面效果会较差.

14.4 习题

✍ **练习 14.1★★** 证明(14.2)定义的 $\overline{f}$ 是无偏估计, 换句话说, 其右边项的期望等于 $\mathbb{E}[f(z)]$.

✍ **练习 14.2**⋆ 证明(14.2)定义的 $\overline{f}$ 的方差由(14.4)给出.

✍ **练习 14.3**⋆ 假设 z 是在区间 $(0,1)$ 上均匀分布的随机变量, 使用 $y = h^{-1}(z)$ 变换 z, 其中 $h(y)$ 由(14.7)给出. 证明 y 具有分布 $p(y)$.

✍ **练习 14.4**⋆⋆ 给定一个在区间 $(0,1)$ 上均匀分布的随机变量 z, 找到一个转换 $y = f(z)$, 使得 y 具有(14.9)给出的 Cauchy 分布.

✍ **练习 14.5**⋆⋆ 假设变量 z_1 和 z_2 在单位圆上均匀分布, 如图 14.3所示, 进行(14.11)和(14.12)给出的变量变换. 证明 (y_1, y_2) 具有(14.13)所示的联合分布.

✍ **练习 14.6**⋆⋆ 假设 $\boldsymbol{z}$ 是一个具有零均值和单位协方差矩阵的 D 维 Gauss 分布的随机变量, 并且假设正定对称矩阵 $\boldsymbol{\Sigma}$ 具有 Cholesky 分解 $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$, 其中 $\boldsymbol{L}$ 是一个下三角矩阵 (即主对角线以上的元素为零). 证明变量 $\boldsymbol{y} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{z}$ 服从均值为 $\boldsymbol{\mu}$ 和协方差为 $\boldsymbol{\Sigma}$ 的 Gauss 分布. 这提供了一种利用服从零均值、单位方差的一元 Gauss 样本来生成服从一般多元 Gauss 分布样本的技术.

✍ **练习 14.7**⋆⋆ 证明拒绝采样确实是从期望的分布 $p(z)$ 中抽取样本. 假设提议分布是 $q(z)$. 证明样本值 z 被接受的概率由 $\widetilde{p}(z)/kq(z)$ 给出, 其中 $\widetilde{p}$ 是与 $p(z)$ 成比例的任何未归一化的分布, 并且常数 k 设置为能够确保 $kq(z) \geqslant \widetilde{p}(z)$ 的最小值. 注意, 抽取值 z 的概率等于从 $q(z)$ 中抽取该值的概率乘以给定已经抽取了该值的情况下接受该值的概率. 利用这一点, 结合概率的加法和乘法规则, 写出关于 z 的归一化分布形式, 并证明它等于 $p(z)$.

✍ **练习 14.8**⋆ 假设变量 z 在区间 $[0,1]$ 上均匀分布. 证明变量 $y = b\tan(z) + c$ 具有(14.17)给出的 Cauchy 分布.

注

(重构新增) **原文问题说明.** 按题面字面,$z \in [0,1]$ 时 $\tan z$ 只在 $[0, \tan 1]$ 上取值,y 的密度是截断的 Cauchy 密度, 其支撑为 $[c, c + b\tan 1]$, 总质量集中在该区间内, 不可能等于(14.17)(后者在整个 $\mathbb{R}$ 上为正). 因此题面的变换应理解为 $y = b\tan\left(\pi\left(z - \frac{1}{2}\right)\right) + c$, 它与(14.17)一致; 本解答按修正后的变换证明, 并在末尾说明字面版本的结论.

$$y = b\tan\left(\pi\left(z - \frac{1}{2}\right)\right) + c.$$

✍ **练习 14.9**⋆⋆ 使用连续性和归一化的要求, 确定自适应拒绝采样中包络分布(14.19)的系数 k_j.

✍ **练习 14.10**⋆⋆ 通过使用小节 14.1.2讨论的从单一指数分布中采样的技术, 设计一种算法, 用来从(14.19)定义的分段指数分布中采样.

✍ **练习 14.11**⋆ 证明(14.30) ~ (14.32)定义的整数上的简单随机游走具有性质 $\mathbb{E}\left[\left(z^{(\tau)}\right)^2\right] = \mathbb{E}\left[\left(z^{(\tau-1)}\right)^2\right] + 1/2$, 并因此通过归纳得出 $\mathbb{E}\left[\left(z^{(\tau)}\right)^2\right] = \tau/2$.

✍ **练习 14.12**⋆⋆ 证明小节 14.2.4讨论的 Gibbs 采样满足(14.36)定义的详细平衡性质.

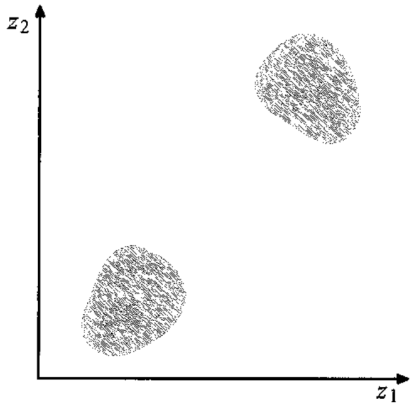


图 14.14: 变量 z_1 和 z_2 上的概率分布在阴影区域是均匀的, 在其他区域为零

✍ **练习 14.13**⋆ 考虑图 14.14所示的分布. 讨论标准的 Gibbs 采样是不是遍历的, 并由此判断是否能正确地从此分布中采样.

- **练习 14.14⋆** 验证超松弛更新［(14.46)］, 其中 z_i 具有均值 μ_i 和方差 σ_i^2,v 具有零均值和单位方差,z_i' 具有均值 μ_i 和方差 σ_i^2.
- **练习 14.15⋆** 证明在有向图的似然加权采样中, 重要性采样权重由(14.48)给出.
- **练习 14.16⋆** 证明只要 $Z(\boldsymbol{w})$ 满足(14.51),(14.50)所示的分布关于 $\boldsymbol{x}$ 就是归一化的.
- **练习 14.17⋆⋆** 利用(14.50), 证明基于能量的模型的对数似然函数的梯度可以写成(14.53)的形式.

  注

  (重构新增) **原文问题说明.** 题面把待证目标标为原书第 14.52 式, 但该编号给出的是对数似然本身, 其梯度才是(14.53). 据原书上下文, 题面所指应为(14.53), 本解答按此理解.
- **练习 14.18⋆⋆** 利用(14.54) ~ (14.56), 证明基于能量的模型的对数似然函数的梯度可以写成(14.57)的形式.

第 15 章 离散潜变量

我们已经了解到复杂的分布是如何通过组合多个简单的分布构建出来的, 以及这些生成的模型是怎样用有向图来表示的 (参见第 11 章). 除了数据集里的观测变量之外, 这类模型常常还会引入一些隐藏的变量 (即隐变量) 或者说潜在变量 (即潜变量). 这些潜变量, 可能对应于数据生成过程中的特定属性, 比如图像分析中物体在三维空间中的未知朝向; 也可能仅是作为建模构造被引入, 以便构建更丰富的模型. 如果定义了一个包括观测变量和潜变量在内的联合分布, 则可以通过边缘化 (marginalization) 来获得单独观测变量的分布. 这样一来, 就能用一个更容易处理的联合分布来描述观测变量上相对复杂的边缘分布, 其中联合分布是在观测变量和潜变量的扩展空间上定义的.

在本章中, 我们将看到通过对离散潜变量 (discrete latent variable) 进行边缘化会产生混合分布. 我们将重点了解 Gauss 混合模型, 它不仅很好地展示了混合分布的特点, 而且在机器学习中也得到了广泛应用. 混合模型的一个简单应用是帮助我们发现在数据中的聚类. 我们将首先讨论 K 均值算法 (K-means algorithm) 这一聚类技术, 它对应于 Gauss 混合模型的一个特定非概率极限. 然后我们将介绍混合分布的潜变量视角 (latent-variable view), 其中离散潜变量可以理解为给数据点指定归属, 以决定每个数据点属于混合模型中的哪一个特定成分.

在潜变量模型中找到最大似然估计的一种通用技术是期望最大化 (Expectation-Maximization,EM) 算法. 我们将首先利用 Gauss 混合分布以非正式的方式介绍 EM 算法, 然后我们会从潜变量的角度给出更详细的解释. 最后, 我们通过探讨证据下界 (Evidence Lower BOund,ELBO) 来展开话题,ELBO 在变分自编码器和扩散模型等生成式模型中起着重要作用.

15.1 K 均值聚类

我们首先考虑在多维空间中识别数据点群组或聚类的问题. 假设有一个数据集 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 其中包含了 N 个 D 维欧几里得变量 $\boldsymbol{x}$ 的观测值. 我们的目标是将该数据集划分成 K 个聚类, 可以暂定 K 的值是已知的. 直觉上, 我们会认为一个聚类包含一组数据点, 这些数据点之间的距离相较于聚类外的数据点之间的距离要短. 这里引入一组 D 维向量 $\boldsymbol{\mu}_k$, 其中 $k = 1, \cdots, K$, 并且 $\boldsymbol{\mu}_k$ 是第 k 个聚类的 “原型” . 可以认为 $\boldsymbol{\mu}_k$ 代表了聚类中心. 然后要找到一组聚类向量 $\{\boldsymbol{\mu}_k\}$, 以及数据点到聚类的分配, 使得每个数据点到与其最近的聚类向量 $\boldsymbol{\mu}_k$ 的距离平方和最小.

为了方便起见, 需要先约定一些符号来描述数据点到聚类的分配.

定义 15.1 (K 均值聚类)

对每个数据点 $\boldsymbol{x}_n$ 引入一组二进制指示变量 $r_{nk} \in \{0, 1\}$, 其中 $k = 1, \cdots, K$. 这些指示变量描述了数据点 $\boldsymbol{x}_n$ 会被分配到哪个聚类: 如果数据点 $\boldsymbol{x}_n$ 被分配到聚类 k, 那么 $r_{nk} = 1$, 并且对于 $j \neq k$,$r_{nj} = 0$. 这是 1-of-K 编码方案的一个例子. 定义误差函数

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{nk} \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2, \tag{15.1}$$

它代表每个数据点到分配给它的聚类向量 $\boldsymbol{\mu}_k$ 的距离平方和. 寻找使(15.1)最小的分配 $\{r_{nk}\}$ 与原型向量 $\{\boldsymbol{\mu}_k\}$ 的过程, 称为 K 均值聚类 (K-means clustering).

♣

注

(重构新增) 引入误差函数(15.1)的动机是把 “聚类” 这一几何直观翻译成一个可以优化的标量:r_{nk} 只负责 “归属” ,$\boldsymbol{\mu}_k$ 只负责 “位置” , 而 J 是每个点到其所属原型的距离平方的总和. 之所以取平方而不用距离本身, 是因为平方距离对 $\boldsymbol{\mu}_k$ 可微, 从而 M 步骤能写成闭式解(15.4); 若改用欧几里得距离,M 步骤就退化为求几何中位数, 一般没有闭式解. 条件 $r_{nk} \in \{0, 1\}$ 与 “每个 n 恰有一个 k 使 $r_{nk} = 1$” 合起来说明分配是**硬**的: 不允许一个点同时以 0.4 和 0.6 的权重属于两个聚类, 这正是节 15.2要用概率模型放松的地方.K 在这里被当作已知常数, 这是 K 均值作为 “给定 K 的划分算法” 的边界; 如何选 K 属于模型选择问题, 不在(15.1)的优化范围内. 此外,J 对联合变量

$(\{r_{nk}\},\{\boldsymbol{\mu}_k\})$ 不是凸函数, 所以命题 15.1只能保证交替最小化的每一步最优, 不能保证到达全局最优.

该误差函数代表每个数据点到分配给它的聚类向量 $\boldsymbol{\mu}_k$ 的距离平方和. 我们的目标是找到 $\{r_{nk}\}$ 和 $\{\boldsymbol{\mu}_k\}$ 的值, 让 J 最小. 这可以通过一个迭代过程来实现, 其中每次迭代包括两个连续步骤, 分别对应于对 $\{r_{nk}\}$ 和 $\{\boldsymbol{\mu}_k\}$ 进行的连续优化. 首先为 $\{\boldsymbol{\mu}_k\}$ 选择一些初始值. 然后在第一步, 保持 $\{\boldsymbol{\mu}_k\}$ 固定, 并优化 $\{r_{nk}\}$ 以最小化 J; 而在第二步, 保持 $\{r_{nk}\}$ 固定, 并优化 $\{\boldsymbol{\mu}_k\}$ 以最小化 J. 重复这两步优化直至收敛. 可以看到, 更新 $\{r_{nk}\}$ 和更新 $\{\boldsymbol{\mu}_k\}$ 的阶段分别对应于 EM 算法的 E(期望) 步骤和 M(最大化) 步骤 (参见节 15.3). 为强调这一点, 在 K 均值算法的背景下, 我们将使用 E 步骤和 M 步骤等术语.

在第一步, 保持 $\{\boldsymbol{\mu}_k\}$ 固定, 并关于 $\{r_{nk}\}$ 最小化 J.J 是关于 $\{r_{nk}\}$ 的线性函数, 所以对于这个优化可以轻松地给出闭合形式的解. 涉及不同 n 的项是独立的, 因此对于任何能给出 $\|\boldsymbol{x}_n-\boldsymbol{\mu}_k\|^2$ 最小值的 k 值, 可以通过选择 $r_{nk}=1$ 来单独为每个 n 进行优化. 换句话说, 我们只是简单地将第 n 个数据点分配给与其最近的聚类, 这可以表示为

$$r_{nk}=\begin{cases}1, & \text{如果}k=\arg\min\limits_j\|\boldsymbol{x}_n-\boldsymbol{\mu}_j\|^2,\\ 0, & \text{其他}.\end{cases}\tag{15.2}$$

接下来考虑在 $\{r_{nk}\}$ 固定的情况下优化 $\{\boldsymbol{\mu}_k\}$(M 步骤). 目标函数 J 是关于 $\boldsymbol{\mu}_k$ 的二次函数, 可通过将其关于 $\boldsymbol{\mu}_k$ 的导数设为零来最小化它, 从而得到

$$2\sum_{n=1}^{N}r_{nk}(\boldsymbol{x}_n-\boldsymbol{\mu}_k)=0,\tag{15.3}$$

我们可以很容易地解出 $\boldsymbol{\mu}_k$, 得到

$$\boldsymbol{\mu}_k=\frac{\sum_n r_{nk}\boldsymbol{x}_n}{\sum_n r_{nk}}.\tag{15.4}$$

命题 15.1 (K 均值算法的两步闭式更新)

固定 $\{\boldsymbol{\mu}_k\}$ 时, 使(15.1)最小的分配由(15.2)给出; 固定 $\{r_{nk}\}$ 且第 k 个聚类非空时, 使(15.1)最小的原型向量由(15.4)唯一给出. 因此交替执行“按最近中心分配”与“按分配求质心”这两个步骤, 每一步都使 J 单调不增.

证明

记 $d_{nk}=\|\boldsymbol{x}_n-\boldsymbol{\mu}_k\|^2\geqslant 0$. 固定 $\{\boldsymbol{\mu}_k\}$ 时,

$$J=\sum_{n=1}^{N}\sum_{k=1}^{K}r_{nk}d_{nk}.$$

由于每个 n 恰有一个 k 使 $r_{nk}=1$, 右端化为 $\sum\limits_n d_{n,k(n)}$, 其中 $k(n)$ 是分配给 $\boldsymbol{x}_n$ 的聚类下标, 故

$$J=\sum_{n=1}^{N}d_{n,k(n)}\geqslant\sum_{n=1}^{N}\min_k d_{nk},$$

且等号成立当且仅当每个 n 都被分到使 d_{nk} 最小的 k, 即(15.2). 这给出了固定 $\{\boldsymbol{\mu}_k\}$ 时的最小点及其最小值的下界.

固定 $\{r_{nk}\}$ 时,J 关于不同 $\boldsymbol{\mu}_k$ 的项可以分离:

$$J=\sum_{k=1}^{K}\left(\sum_{n=1}^{N}r_{nk}\|\boldsymbol{x}_n-\boldsymbol{\mu}_k\|^2\right).$$

第 k 个括号关于 $\boldsymbol{\mu}_k$ 的梯度为 $-2\sum\limits_n r_{nk}(\boldsymbol{x}_n-\boldsymbol{\mu}_k)$, 置零即得(15.3), 从而(15.4). 若该聚类非空, 记 $N_k=\sum\limits_n r_{nk}>0$, 则括号的海塞矩阵为 $2N_k\boldsymbol{I}$ 正定, 故(15.4)是唯一全局最小点. 若该聚类为空 ($N_k=0$),J 不依赖 $\boldsymbol{\mu}_k$, 它的取值不影响 J, 按惯例保持原值不变.

于是 E 步骤用一次分配把 J 降到固定中心下的最小点,M 步骤再用一次质心更新把 J 降到固定分配下的最小

点, 两步都不会使 J 增大.

□

注

(重构新增) 这条命题说明 K 均值算法是**分块坐标下降** (block coordinate descent): 把参数分成 $\{r_{nk}\}$ 与 $\{\boldsymbol{\mu}_k\}$ 两块, 轮流对每块做精确最小化. 需要注意三点. 第一, 每次精确最小化只保证 J 不增, 不保证到达全局最优——J 对联合变量不是凸函数, 不同的初始化会落入不同的局部极小值, 这正是定理 15.1只能断言“终止”而不能断言“终止于全局最优”的原因. 第二, 两个子问题的最优解都是**唯一**的 (在给 $\{r_{nk}\}$ 配上确定性的择一规则之后), 这一点是证明有限步收敛的关键: 如果某个子问题的最优解不唯一, 迭代就可能在同一目标值上无限游走. 第三,E 步骤(15.2)中出现 $\arg\min$, 当两个中心与 $\boldsymbol{x}_n$ 等距时最优分配不唯一, 此时必须事先约定一条确定性规则 (例如取下标最小的 k), 否则“算法”本身没有良好定义.

观察(15.4)的右侧, 分母等于分配给聚类 k 的数据点的数量, 运算结果 $\boldsymbol{\mu}_k$ 可以简单地看作等于所有分配给聚类 k 的数据点 $\boldsymbol{x}_n$ 的平均值. 这种方法因而得名 K 均值算法 (Lloyd, 1982), 详见算法 15.1. 因为 $\{r_{nk}\}$ 是离散的, 并且每次迭代都不会导致误差函数增大, 所以 K 均值算法一定可以在有限的步骤中收敛 (见练习 15.1).

定理 15.1 (K 均值算法的有限步终止)

设数据点 $\{\boldsymbol{x}_n\}$ 给定,K 固定. 取定一条确定性的择一规则 (在距离相同的中心中取下标最小者), 并约定当某个聚类为空时其原型向量保持不变. 则从任意初始原型向量 $\{\boldsymbol{\mu}_k\}$ 出发,K 均值算法在有限次迭代之后必有 $\{r_{nk}\} = \{r_{nk}^{(\text{old})}\}$, 即算法终止.

♡

证明

把一次完整的迭代看成状态之间的映射: 状态取为当前的分配 $\{r_{nk}\}$. 给定分配后, 原型向量被(15.4)唯一确定 (空聚类的原型保持上一轮的值), 故状态确实决定了整个算法的后续演化; 记一次“E 步骤 + M 步骤”给出的新状态为 $F(r)$, 并记 $g(r)$ 为在分配 r 及其对应原型下误差函数的取值.

由命题 15.1,E 步骤给出

$$J\big(F(r),\ \boldsymbol{\mu}(r)\big) \leqslant J\big(r,\ \boldsymbol{\mu}(r)\big) = g(r),$$

其中 $\boldsymbol{\mu}(r)$ 表示分配 r 对应的原型;M 步骤又给出

$$g\big(F(r)\big) = J\big(F(r),\ \boldsymbol{\mu}(F(r))\big) \leqslant J\big(F(r),\ \boldsymbol{\mu}(r)\big).$$

两式合并得 $g(F(r)) \leqslant g(r)$, 即 g 沿迭代单调不增.

若 $F(r) \neq r$ 而 $g(F(r)) = g(r)$, 则上面两个不等式都取等号. 第二个等号说明: 对固定分配 $F(r)$, 原型 $\boldsymbol{\mu}(F(r))$ 与 $\boldsymbol{\mu}(r)$ 给出相同的 J 值; 而固定分配时 J 关于各非空聚类的原型是严格凸的 (海塞矩阵 $2N_k\boldsymbol{I}$ 正定), 故最小值点唯一, 于是 $\boldsymbol{\mu}(F(r)) = \boldsymbol{\mu}(r)$. 既然原型相同、择一规则又是确定性的, 同一个分配就会再次被算出, 即 $F(F(r)) = F(r)$: 迭代在一步之后停住.

因此, 若算法永不终止, 则每次迭代都必须使 g 严格减小. 但分配 $\{r_{nk}\}$ 只有 K^N 种可能取值,g 至多取 K^N 个不同的值, 不可能严格下降无穷多次. 矛盾. 故算法必在有限步内终止.

□

注

(重构新增) 这条定理的结论只是“终止”, 不是“终止于全局最优”:J 沿迭代单调不增且有下界 0, 所以迭代值收敛, 但它收敛到的是局部极小值. 定理的三条前提都不可省. 其一, “确定性择一规则”保证了状态演化的确定性, 否则等距情形可能使分配在两个同样好的解之间来回跳动, 此时算法确实可能不终止. 其二, “空聚类保持原型”的约定使 g 仍是状态的函数; 若允许空聚类随意重置,g 的单调性论证就要重新检查. 其三,K 固定且有限, 才使分配的总数 K^N 有限——若把 K 也当作待优化变量, 状态空间不再有限, 本证明失效. 此外, “有限步”并不给出步数的实用上界:K^N 随 N 指数增长, 实际收敛速度取决于初始化, 这也解释了后面为什么常用多次随机重启来缓解局部极小值.

算法 15.1　K 均值算法

Input: 初始原型向量 $\boldsymbol{\mu}_1, \cdots, \boldsymbol{\mu}_K$
　　数据集 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$
Output: 最终原型向量 $\boldsymbol{\mu}_1, \cdots, \boldsymbol{\mu}_K$
$\{r_{nk} \leftarrow 0\}$　// 将所有分配初始化为零
repeat
　　$\{r_{nk}^{(\text{old})}\} \leftarrow \{r_{nk}\}$
　　// 更新分配
　　for $n \in \{1, \cdots, N\}$ **do**
　　　　$k \leftarrow \arg\min_j \|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2$
　　　　$r_{nk} \leftarrow 1$
　　　　$r_{nj} \leftarrow 0, \quad j \in \{1, \cdots, K\},\ j \neq k$
　　end for
　　// 更新原型向量
　　for $k \in \{1, \cdots, K\}$ **do**
　　　　$\boldsymbol{\mu}_k \leftarrow \sum_n r_{nk}\boldsymbol{x}_n \Big/ \sum_n r_{nk}$
　　end for
until $\{r_{nk}\} = \{r_{nk}^{(\text{old})}\}$　// 未发生变化的分配
return $\boldsymbol{\mu}_1, \cdots, \boldsymbol{\mu}_K, \{r_{nk}\}$

将数据点重新分配到聚类和重新计算聚类均值这两个阶段轮流重复进行, 直至分配不再变化 (或者直至迭代次数超过某预定值). 然而, 这种方法可能会收敛到目标函数 J 的局部极小值而不是全局最小值.MacQueen (1967) 研究了 K 均值算法的收敛性.

图 15.1使用源自美国黄石国家公园老忠实喷泉 (Old Faithful geyser) 数据集的喷发数据对 K 均值算法进行了说明 (参见小节 3.2.9). 该数据集包含 272 个数据点, 每个数据点都在水平轴上给出了一次喷发的持续时间, 并在垂直轴上给出了下一次喷发的持续时间. 在这里, 我们已经对数据进行了线性重新缩放, 也就是标准化, 使得每个变量都具有零均值和单位标准差.

例题 15.1 老忠实喷泉数据上的 K 均值迭代　对于这个例子, 我们选择了 $K = 2$, 因此这相当于是根据数据点位于两个聚类中心的垂直平分线的哪一边来对数据点进行分类的.图 15.2显示了(15.1)给出的用于老忠实喷泉例子的成本函数 J 的曲线图. 注意, 我们特意为聚类中心选择了较差的初始值, 这样算法就需要多步操作才能收敛. 在实践中, 更好的初始化程序会选择聚类中心 $\boldsymbol{\mu}_k$ 等于 K 个数据点的随机子集. 另外要注意,K 均值算法通常用于在应用 EM 算法之前初始化 Gauss 混合模型中的参数 (参见小节 15.2.2).

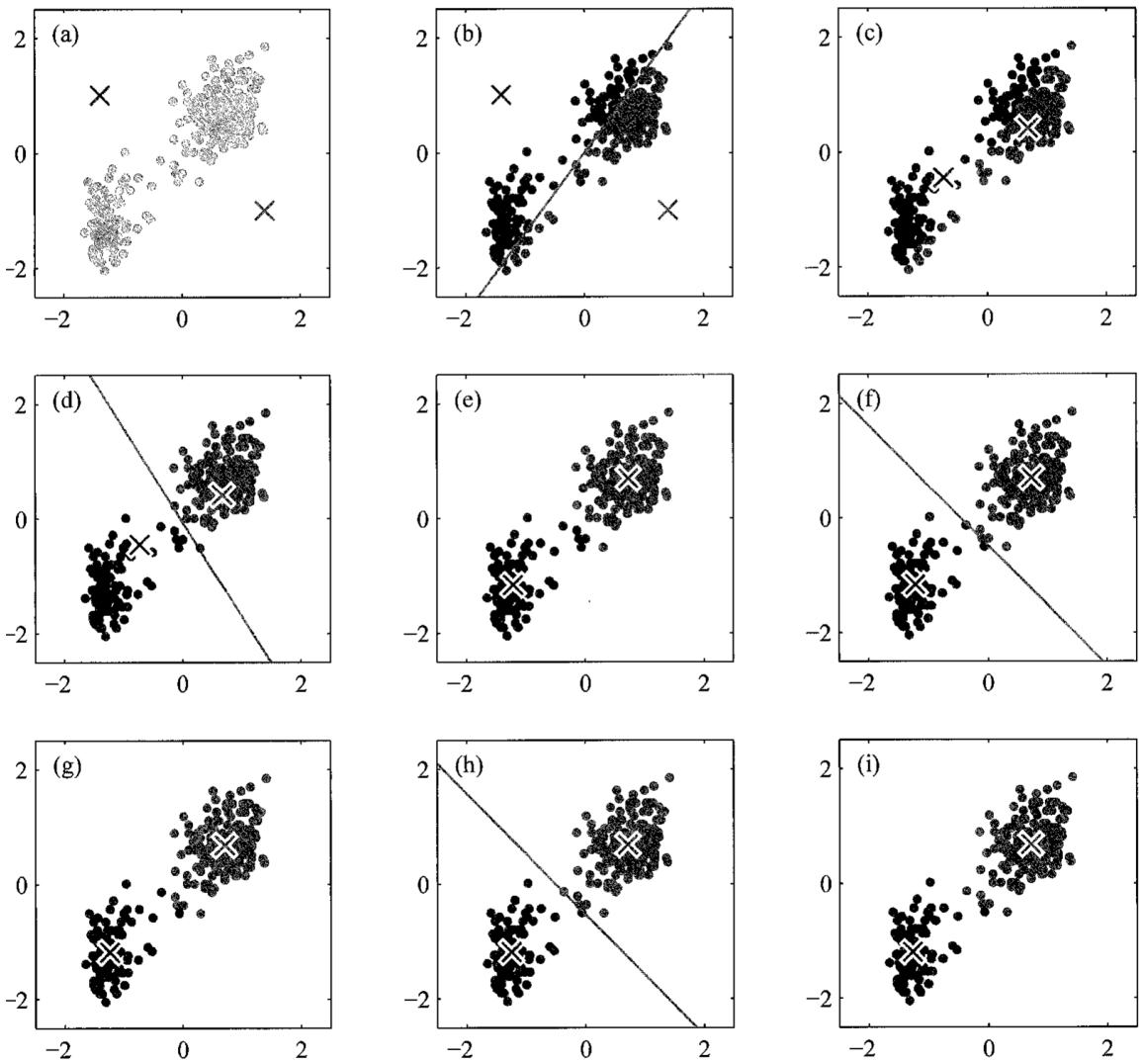


图 15.1: 利用线性重新缩放的老忠实喷泉数据集来说明 K 均值算法.(a) 绿色点表示二维欧几里得空间中的数据集. 初始选择的聚类中心 $\boldsymbol{\mu}_1$ 和 $\boldsymbol{\mu}_2$ 分别由红色和蓝色的叉号表示.(b) 在初始的 E 步骤中, 每个数据点要么分配给红色聚类, 要么分配给蓝色聚类, 这取决于数据点与哪个聚类中心更近. 这相当于根据数据点位于两个聚类中心的垂直平分线 (洋红色的线) 的哪一边来分类数据点.(c) 在随后的 M 步骤中, 每个聚类中心被重新计算为分配给对应聚类的所有数据点的平均值.(d)~(i) 显示了直至最终收敛前的连续 E 步骤和 M 步骤

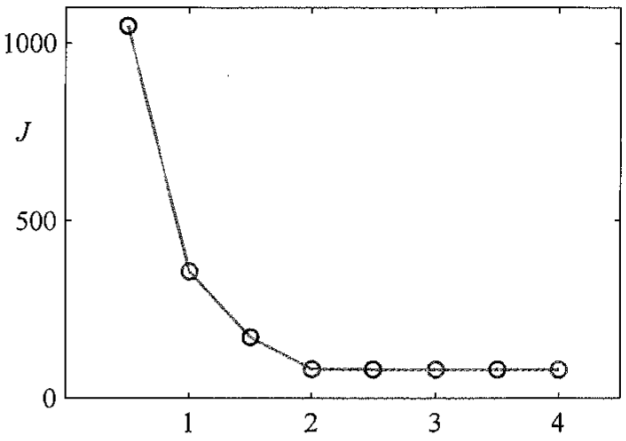


图 15.2: 根据(15.1)绘出的成本函数 J 的曲线图, 其中显示了成本函数 J 在用于图 15.1所示例子的 K 均值算法的每个 E 步骤 (蓝色点) 和 M 步骤 (红色点) 之后的变化情况. 在经过第三个 M 步骤之后, 算法已经收敛, 最后一轮 EM 周期既没有改变分配也没有改变原型向量

注

(重构新增) 图 15.2的横轴是“完整迭代 (一轮 E 步骤加一轮 M 步骤)”的序号, 纵轴是(15.1)的取值, 因此曲线的阶梯结构直接反映了命题 15.1的结论:J 只在每次 E 步骤或 M 步骤之后取值, 且从不上升. 曲线在第三次 M 步骤后变平, 对应定理 15.1所说的“分配不再变化”; 此时若继续迭代,J 的值不再改变, 因为分配与原型都已是不动点. 图中特意采用较差的初始中心, 是为了让读者看到 J 明显下降的若干步; 若初始中心已经接近最终的质心, 曲线会在第一步之后几乎水平.

到目前为止, 考虑的是批量版本的 K 均值算法, 其中整个数据集一起用来更新原型向量. 我们也可以推导出一个顺序更新的策略, 在这个策略中, 对于每个数据点 $\boldsymbol{x}_n$, 更新离它最近的原型向量 $\boldsymbol{\mu}_k$:

$$\boldsymbol{\mu}_k^{\text{new}} = \boldsymbol{\mu}_k^{\text{old}} + \frac{1}{N_k}(\boldsymbol{x}_n - \boldsymbol{\mu}_k^{\text{old}}), \tag{15.5}$$

其中 N_k 是到目前为止用来更新 $\boldsymbol{\mu}_k$ 的数据点的数量. 这允许每个数据点在使用一次后就被丢弃, 然后处理下一个数据点.

命题 15.2 (顺序 K 均值更新)

设 $\boldsymbol{\mu}_k^{\text{old}}$ 是把此前的 $N_k - 1$ 个数据点分配给聚类 k 后由(15.4)得到的均值, 现有一个新数据点 $\boldsymbol{x}_n$ 被分配给聚类 k. 则把 $\boldsymbol{x}_n$ 并入后由(15.4)给出的新均值为(15.5), 其中 N_k 是并入后聚类 k 中数据点的总数. 该更新没有作任何近似.

♠

证明

设并入前聚类 k 中的数据点下标集合为 $\mathcal{S}$,$|\mathcal{S}| = N_k - 1$, 则由(15.4),

$$\boldsymbol{\mu}_k^{\text{old}} = \frac{1}{N_k - 1}\sum_{n' \in \mathcal{S}} \boldsymbol{x}_{n'}, \quad \text{即} \quad \sum_{n' \in \mathcal{S}} \boldsymbol{x}_{n'} = (N_k - 1)\,\boldsymbol{\mu}_k^{\text{old}}.$$

并入 $\boldsymbol{x}_n$ 后,(15.4)的分母变为 N_k, 分子变为 $\sum_{n' \in \mathcal{S}} \boldsymbol{x}_{n'} + \boldsymbol{x}_n$, 于是

$$\begin{aligned}\boldsymbol{\mu}_k^{\text{new}} &= \frac{1}{N_k}\left[(N_k - 1)\,\boldsymbol{\mu}_k^{\text{old}} + \boldsymbol{x}_n\right] = \frac{N_k - 1}{N_k}\boldsymbol{\mu}_k^{\text{old}} + \frac{1}{N_k}\boldsymbol{x}_n \\ &= \boldsymbol{\mu}_k^{\text{old}} + \frac{1}{N_k}\left(\boldsymbol{x}_n - \boldsymbol{\mu}_k^{\text{old}}\right),\end{aligned}$$

即(15.5). 推导中只用到“旧均值等于旧集合的算术平均”这一事实, 未作任何近似, 故新的原型向量仍精确等于分配给它的所有数据向量的均值.

□

注

(重构新增) (15.5)就是均值估计的 Robbins–Monro 型递推: 新均值等于旧均值加上一个与“新数据带来的残差”成正比的修正, 而 $1/N_k$ 充当一个随 N_k 衰减的学习率. 这也说明了它与节 7.2中随机梯度下降的区别: 这里的学习率由分配计数精确给出, 并非人为设定, 因此更新是精确的而不是近似的. 条件在于“只增不减”——本命题讨论的是把一个新点并入某个聚类. 若在线算法允许把一个已分配的点改派到别的聚类, 那么除了给新聚类加上贡献, 还必须从旧聚类减去该点的贡献; 这正是小节 15.4.5中顺序 EM 的更新(15.60)比(15.5)多出一个 $-\gamma^{\text{old}}(z_{mk})$ 项的原因. 此外, 这样得到的是“单遍、一次一个点”的算法, 其最终原型依赖于数据点的处理顺序, 一般不等于批量 K 均值的结果.

K 均值算法的一个显著特点是, 在每次迭代中, 每个数据点都只被分配给一个聚类. 尽管有些数据点会比其他任何聚类中心 $\boldsymbol{\mu}_j$ 更接近特定聚类中心 $\boldsymbol{\mu}_k$, 但也可能有其他数据点大约处于聚类中心之间的中点位置. 在后一种情况下, 其实并不清楚将数据点硬性分配给最近的聚类是否是最合适的方法 (参见节 15.2). 我们将看到, 通过采用概率方法, 我们将能够以一种反映最适当分配的不确定性程度的方式, 获得数据点到聚类的“软”分配. 这种概率方法有许多好处.

图像分割

作为 K 均值算法应用的例子, 我们考虑图像分割和图像压缩这两个相关问题. 图像分割的目标是将图像划分成区域, 以便每个区域都具有相对一致的视觉外观或者对应于物体或物体的某一部分 (Forsyth and Ponce, 2003). 图像中的每个像素都是由红色、蓝色和绿色通道的强度组成的三维空间中的一个点, 图像分割算法简单地将图像中的每个像素作为一个单独的数据点来处理. 请注意, 严格来说这个空间并不是欧几里得空间, 因为通道强度受 $[0, 1]$ 区间的限制. 尽管如此, 我们仍然可以直接应用 K 均值算法. 我们展示了对任何特定的 K 值运行 K 均值算法直至收敛的结果, 方法是重新绘制图像, 并替换图像中的每个像素向量——用其分配到的聚类中心 $\boldsymbol{\mu}_k$ 给出的 $\{R, G, B\}$ 强度三元组进行替换. 各种不同 K 值的结果已显示在图 15.3中, 从中可以看到, 对于给定的 K 值, 该算法只使用 K 种颜色来表示图像. 应该强调的是, 就图像分割而言, 这种使用 K 均值的方式并不是特别复杂的方法, 最起码它没有考虑不同像素之间的空间邻近性. 总的来说, 图像分割非常困难, 但仍然是活跃的研究课题, 这里仅用来说明 K 均值算法的应用.

我们也可以使用聚类算法进行数据压缩. 首先我们要区分无损数据压缩 (其目标是从压缩表示中精确地重建原始数据) 和有损数据压缩 (也就是说, 我们接受数据重建过程中的一些误差以换取相比无损压缩情况下更高的压缩率). 我们可以将 K 均值算法应用于有损数据压缩问题. 对于 N 个数据点中的每一个数据点, 我们只存储其分配到的聚类的身份 k. 我们还需要存储 K 个聚类中心 $\{\boldsymbol{\mu}_k\}$ 的值, 在我们选择 $K \ll N$ 的前提下, 这种方法明显减少了需要的数据量, 然后每个数据点都可以通过与其最近的聚类中心 $\boldsymbol{\mu}_k$ 来近似表示. 新的数据点同样可以通过首先找到最近的 $\boldsymbol{\mu}_k$, 然后存储标签 k 而不是原始的数据向量来进行压缩. 这个框架通常称为向量量化 (vector quantization), 向量 $\{\boldsymbol{\mu}_k\}$ 称为码本向量 (codebook vector).

例题 15.2 图像压缩的比特数计算 上面讨论的图像分割问题也提供了聚类可以用于数据压缩的例证. 假设原始图像有 N 个像素, 其中的每个像素都由 $\{R, G, B\}$ 值组成, 每个值均以 8 位的精度存储. 直接传输整个图像将耗费 $24N$ 比特. 假设我们首先在图像数据上应用 K 均值算法, 然后传输最接近的向量 $\boldsymbol{\mu}_k$ 的身份而不是传输原始的像素强度向量. 因为有 K 个这样的向量, 所以每个像素需要 $\log_2 K$ 比特. 我们还必须传输 K 个码本向量 $\{\boldsymbol{\mu}_k\}$, 这需要 $24K$ 比特, 因此传输整个图像所需的比特数是 $24K + N \log_2 K$(向上取整).图 15.3中的原始图像有 240×180 像素 $= 43\,200$ 像素, 因此直接传输需要 $1\,036\,800$ 比特. 相比之下, 压缩后的图像分别需要传输 $43\,248$ 比特 ($K = 2$)、$86\,472$ 比特 ($K = 3$) 和 $173\,040$ 比特 ($K = 10$). 与原始图像相比, 它们分别代表了大约 4.2%、8.3% 和 16.7% 的压缩比. 当然, 压缩程度和图像质量之间存在权衡关系. 注意, 在这个例子中, 我们的目的是说明 K 均值算法的应用. 如果我们的目标是生成一个好的图像压缩器, 那么一个更有效的方法是, 考虑小块的相邻像素, 例如 5×5 的像素块, 并利用自然图像中近邻像素之间存在的相关性.

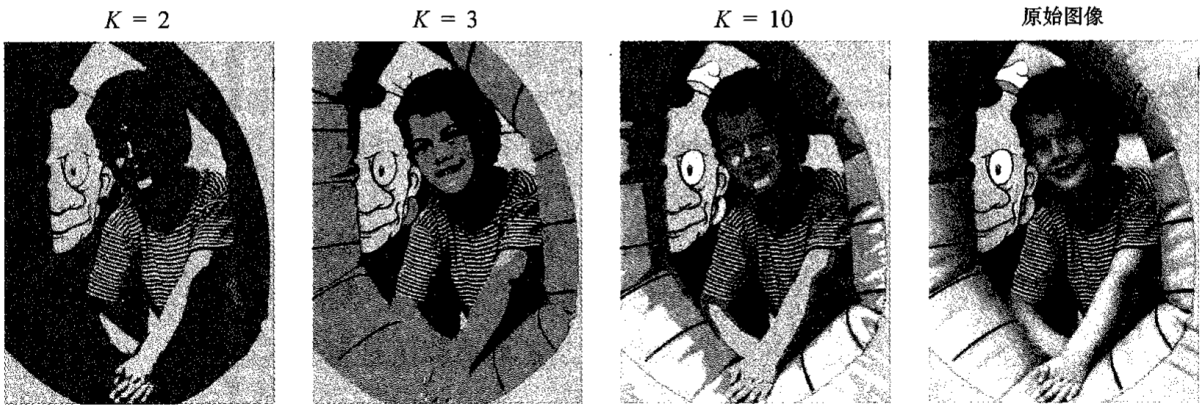


图 15.3: 一个将 K 均值算法应用于图像分割的例子, 这里展示了原始图像以及使用各种 K 值获得的 K 均值分割图像. 这也说明了向量量化在数据压缩中的使用效果, 其中较小的 K 值以牺牲图像质量为代价获得更高的压缩率

注

(重构新增) 这里的比特数可以逐项核对, 关键在于“向上取整”落在 $\lceil \log_2 K \rceil$ 上而不是落在乘积上: 每个像素的标号必须用整数个比特存储, 因此需要 $\lceil \log_2 K \rceil$ 比特, 而不是 $\log_2 K$ 比特.$K = 2, 3, 10$ 分别给出 $1, 2, 4$ 比特, 于是

$$K = 2: \quad 24 \times 2 + 43\,200 \times 1 = 48 + 43\,200 = 43\,248,$$

$$K = 3: \quad 24 \times 3 + 43\,200 \times 2 = 72 + 86\,400 = 86\,472,$$

$$K = 10: \quad 24 \times 10 + 43\,200 \times 4 = 240 + 172\,800 = 173\,040,$$

与印出的三个数字完全一致; 压缩比 $43\,248/1\,036\,800 \approx 4.17\%$、$86\,472/1\,036\,800 \approx 8.34\%$、$173\,040/1\,036\,800 \approx 16.69\%$ 也与正文的 4.2%、8.3%、16.7% 相符. 这也解释了为什么 K 从 2 增到 3 时比特数几乎翻倍而不是只增加 58.5%:$\lceil \log_2 3 \rceil = 2$, 与 $K = 4$ 所需的比特数相同. 码本本身的 $24K$ 比特在 $K \ll N$ 时可以忽略 ($K = 10$ 时仅占 $240/173\,040 \approx 0.14\%$), 所以压缩比近似等于 $1/\lceil \log_2 K \rceil$; 要提高压缩率, 更有效的途径是像正文最后所说那样利用相邻像素的相关性, 而不是单纯增大 K.

15.2 Gauss 混合分布

我们之前已经将 Gauss 混合分布描述为 Gauss 分量的简单线性叠加, 目的是提供比单个 Gauss 分布更丰富的密度模型类别 (参见小节 3.2.9). 在本节中, 将转向用离散潜变量的形式来描述 Gauss 混合分布. 这将帮助我们更深入地理解这个重要分布, 并有助于激励我们去探究 EM(Expectation–Maximization, 最大期望) 算法.

回想一下(3.115),Gauss 混合分布可以写成 Gauss 分量的线性叠加形式.

定义 15.2 (Gauss 混合分布)

设 $K \geqslant 1, \boldsymbol{\mu}_k \in \mathbb{R}^D, \boldsymbol{\Sigma}_k$ 为 $D \times D$ 正定对称矩阵,$k = 1, \cdots, K$. 称

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \tag{15.6}$$

为 K 个分量的 Gauss 混合分布 (Gaussian mixture distribution,GMM), 其中 $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ 称为第 k 个**分量**,π_k 称为第 k 个**混合系数**. 参数 $\{\pi_k\}$ 必须满足

$$0 \leqslant \pi_k \leqslant 1 \tag{15.7}$$

和

$$\sum_{k=1}^{K} \pi_k = 1. \tag{15.8}$$

注

(重构新增) 把若干 Gauss 密度加权相加, 动机是让每个分量只负责一个“局部团块”, 从而用简单的分量拼出多峰的复杂密度;节 15.1的 K 均值正是每个分量协方差都相同且趋于零时的极限情形 (见小节 15.3.2). 条件(15.7)与(15.8)的作用各不相同: 前者保证每一项都非负, 从而 $p(\boldsymbol{x}) \geqslant 0$; 后者保证 $\int p(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = \sum_k \pi_k = 1$, 即 p 是合法的概率密度. 二者缺一不可——只去掉非负性,π_k 为负时积分仍可能为 1, 但 p 在某些区域会取负值. 各 $\boldsymbol{\Sigma}_k$ 的正定性保证 $\mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ 良定义且可归一化. 注意分量个数 K 不是由(15.6)决定的参数, 而是模型结构的一部分;K 越大模型越灵活, 但参数个数 $K - 1 + K(D + D(D+1)/2)$ 也随之线性增长, 且似然函数会出现命题 15.4所述的奇异性.

让我们引入一个 K 维的二进制随机向量 $\boldsymbol{z}$, 它采用 1-of-K 表示形式, 其中一个元素等于 1, 其他所有元素等于 0. 因此 z_k 的值满足 $z_k \in \{0,1\}$ 以及 $\sum_k z_k = 1$, 并且根据哪个元素非零, 向量 $\boldsymbol{z}$ 有 K 种可能的状态. 将联合分布 $p(\boldsymbol{x}, \boldsymbol{z})$ 定义为边缘分布 $p(\boldsymbol{z})$ 和条件分布 $p(\boldsymbol{x}\,|\,\boldsymbol{z})$ 的形式.

定义 15.3 (1-of-K 离散潜变量表示)

设 $\boldsymbol{z} = (z_1, \cdots, z_K)^{\mathsf{T}}$ 满足 $z_k \in \{0,1\}$ 与 $\sum_{k=1}^{K} z_k = 1$. 关于 $\boldsymbol{z}$ 的边缘分布由混合系数 π_k 指定, 即

$$p(z_k = 1) = \pi_k,$$

其中参数 $\{\pi_k\}$ 满足(15.7)与(15.8). 因为 $\boldsymbol{z}$ 使用 1-of-K 表示形式, 该分布也可以写成

$$p(\boldsymbol{z}) = \prod_{k=1}^{K} \pi_k^{z_k}. \tag{15.9}$$

给定 $\boldsymbol{z}$ 的特定值时 $\boldsymbol{x}$ 的条件分布也是一个 Gauss 分布:

$$p(\boldsymbol{x}\,|\,z_k = 1) = \mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k),$$

它可以写成

$$p(\boldsymbol{x}\,|\,\boldsymbol{z}) = \prod_{k=1}^{K} \mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_k}. \tag{15.10}$$

注

(重构新增) 这个定义把“混合分布”改写成一个两层的生成过程: 先按 $p(\boldsymbol{z})$ 抽一个状态 (即选出分量), 再按对应的 Gauss 密度抽 $\boldsymbol{x}$. 定义中的两条乘积式(15.9)与(15.10)是该表示能工作的关键: 由于 $\boldsymbol{z}$ 是 1-of-K 向量,$\prod_k \pi_k^{z_k}$ 中只有第 k 个 ($z_k = 1$ 的那个) 因子存活, 值恰为 π_k, 其余因子都是 $\pi_j^0 = 1$; 同理(15.10)中只有第 k 个 Gauss 因子存活. 这样就把“按状态取分支”写成了统一的乘积形式, 从而可以直接取对数、求和与求导. 注意 $z_k \in \{0,1\}$ 与 $\sum_k z_k = 1$ 是硬约束:$\boldsymbol{z}$ 的取值只有 K 个, 而不是 2^K 个, 这正是“离散潜变量”中“离散”的含义.

联合分布由 $p(\boldsymbol{z})p(\boldsymbol{x}\,|\,\boldsymbol{z})$ 给出, 它可以用图 15.4中的图模型来描述 (参见练习 15.3). 可以通过对所有可能的 $\boldsymbol{z}$ 的状态求和联合分布来获得 $\boldsymbol{x}$ 的边缘分布:

$$p(\boldsymbol{x}) = \sum_{\boldsymbol{z}} p(\boldsymbol{z})p(\boldsymbol{x}\,|\,\boldsymbol{z}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \tag{15.11}$$

其中我们利用了(15.9)和(15.10). 因此,$\boldsymbol{x}$ 的边缘分布是形式为(15.6)的 Gauss 混合分布. 如果我们有多个观测值 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$, 那么因为边缘分布的形式是 $p(\boldsymbol{x}) = \sum_{\boldsymbol{z}} p(\boldsymbol{x}, \boldsymbol{z})$, 所以每个观测到的数据点 $\boldsymbol{x}_n$ 都对应一个潜变量 $\boldsymbol{z}_n$.

命题 15.3 (边缘化恢复混合分布)

设 $p(\boldsymbol{z})$ 由(15.9)给出,$p(\boldsymbol{x}\,|\,\boldsymbol{z})$ 由(15.10)给出, 则联合分布 $p(\boldsymbol{x}, \boldsymbol{z}) = p(\boldsymbol{z})p(\boldsymbol{x}\,|\,\boldsymbol{z})$ 关于 $\boldsymbol{z}$ 的边缘分布恰为(15.6).

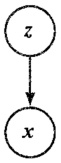


图 15.4: 联合分布的图模型表示, 其中联合分布以 $p(\boldsymbol{x},\boldsymbol{z})=p(\boldsymbol{z})p(\boldsymbol{x}\,|\,\boldsymbol{z})$ 的形式给出

证明

对 $\boldsymbol{z}$ 的所有 K 种可能取值求和. 由于 $\boldsymbol{z}$ 取遍 1-of-K 向量, 求和只有 K 个非零项, 且第 k 项对应 $z_k=1$、其余 $z_j=0$. 对这样的 $\boldsymbol{z}$,

$$p(\boldsymbol{z})=\prod_{j=1}^{K}\pi_j^{z_j}=\pi_k,\qquad p(\boldsymbol{x}\,|\,\boldsymbol{z})=\prod_{j=1}^{K}\mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_j,\boldsymbol{\Sigma}_j)^{z_j}=\mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k).$$

于是

$$p(\boldsymbol{x})=\sum_{\boldsymbol{z}}p(\boldsymbol{z})p(\boldsymbol{x}\,|\,\boldsymbol{z})=\sum_{k=1}^{K}\pi_k\mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k),$$

即(15.6).

□

注

(重构新增) 这条命题说明了“引入潜变量没有改变模型”:$\boldsymbol{z}$ 只是把(15.6)中的求和换了一种说法, 边缘分布仍然是同一个 Gauss 混合分布. 它的价值在于把**求和**搬到了联合分布这一层, 从而在完全数据情形下对数可以直接作用到每个分量 (见小节 15.3.1). 证明只用到 $\boldsymbol{z}$ 的 1-of-K 结构: 求和项数等于 K 而不是 2^K, 这既保证了对 $\boldsymbol{z}$ 求和的可行性, 也是后面 E 步骤只需计算 K 个责任值的原因. 若 $\boldsymbol{z}$ 的各分量独立 (而非 1-of-K 约束), 边缘化将产生 2^K 项, 模型就不再是混合分布了.

另一个起到重要作用的量是给定 $\boldsymbol{x}$ 时 $\boldsymbol{z}$ 的条件概率. 我们使用 $\gamma(z_k)$ 来表示 $p(z_k=1\,|\,\boldsymbol{x})$, 其值可以使用 Bayes 定理找到:

$$\begin{aligned}\gamma(z_k)\equiv p(z_k=1\,|\,\boldsymbol{x})&=\frac{p(z_k=1)p(\boldsymbol{x}\,|\,z_k=1)}{\sum_{j=1}^{K}p(z_j=1)p(\boldsymbol{x}\,|\,z_j=1)}\\&=\frac{\pi_k\mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K}\pi_j\mathcal{N}(\boldsymbol{x}\,|\,\boldsymbol{\mu}_j,\boldsymbol{\Sigma}_j)}.\end{aligned}\tag{15.12}$$

定义 15.4 (责任)

称(15.12)中的 $\gamma(z_k)=p(z_k=1\,|\,\boldsymbol{x})$ 为第 k 个分量对观测值 $\boldsymbol{x}$ 的**责任** (responsibility).

♣

注

(重构新增) π_k 可以视为 $z_k=1$ 的先验概率,$\gamma(z_k)$ 可以视为观测到 $\boldsymbol{x}$ 之后的相应后验概率. 正如我们稍后将看到的,$\gamma(z_k)$ 也可以视为分量 k 对“解释”观测值 $\boldsymbol{x}$ 所要承担的责任. 从定义可以直接读出两条性质:$\gamma(z_k)\in[0,1]$ 且 $\sum_{k=1}^{K}\gamma(z_k)=1$, 即责任是一个合法的“软分配”权重; 这正是它与 K 均值中 $r_{nk}\in\{0,1\}$ 的硬分配的本质区别. 责任依赖于 $\boldsymbol{x}$ 与全部分量参数的比较 (分母对所有 j 求和), 因此它不是某个分量的“局部”属性; 但当某个分量的密度远大于其他分量时,$\gamma(z_k)$ 会迅速趋近于 1, 这正是小节 15.3.2中 $\varepsilon\to 0$ 极限下退化为硬分配的原因. 名字“责任”的来历也在这里:$\gamma(z_k)$ 衡量的是“观测到 $\boldsymbol{x}$ 之后, 把这一点归因于第 k 个分量的可信程度”.

参见小节 14.2.5, 我们可以使用祖先采样 (ancestral sampling) 生成随机样本, 这些样本按照 Gauss 混合模型分布. 为此, 我们首先从边缘分布 $p(\boldsymbol{z})$ 中生成一个 $\boldsymbol{z}$ 的值, 记为 $\hat{\boldsymbol{z}}$, 然后从条件分布 $p(\boldsymbol{x}\,|\,\hat{\boldsymbol{z}})$ 中生成一个 $\boldsymbol{x}$ 的值. 我们可以通过在对应的 $\boldsymbol{x}$ 值处绘制点, 然后根据 $\boldsymbol{z}$ 的值 (即生成这些点的 Gauss 分量) 对点进行着色来描绘联合分布

$p(\boldsymbol{x}, z)$ 中的样本, 如图 15.5(a) 所示. 类似地, 边缘分布 $p(\boldsymbol{x})$ 中的样本是通过选取联合分布中的样本并忽略 z 的值获得的. 这些样本如图 15.5(b) 所示, 仅依据 $\boldsymbol{x}$ 值绘制出点而不带任何颜色标签.

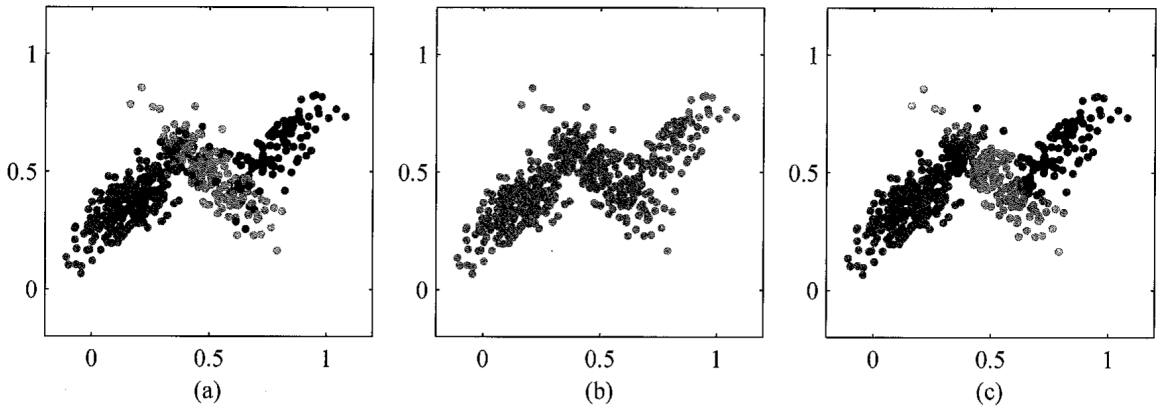


图 15.5: 从图 3.8所示的三个 Gauss 混合分布中抽取 500 个数据点的示例.(a) 来自联合分布 $p(z)p(\boldsymbol{x} \mid z)$ 的样本, 其中 z 的三种状态分别对应于混合的三个分量, 分别用红色、绿色和蓝色表示.(b) 来自边缘分布 $p(\boldsymbol{x})$ 的相应样本, 该边缘分布是通过简单地忽略 z 的值并仅绘制 $\boldsymbol{x}$ 的值获得的, 数据集在 (a) 中被认为是完整的, 而在 (b) 中被认为是不完整的 (参见节 15.3).(c) 相同的样本, 其中颜色代表与数据点 $\boldsymbol{x}_n$ 相关的责任 $\gamma(z_{nk})$, 当 k 取值分别为 $1, 2, 3$ 时, 可通过调配红色、蓝色和绿色比例来绘制相应的点来获得

我们还可以使用这个合成的数据集来说明 “责任”, 即评估每个数据点关于生成此数据集的 Gauss 混合分布中每个分量的后验概率. 特别地, 我们可以使用红色、蓝色和绿色墨水的比例来表示与数据点 $\boldsymbol{x}_n$ 相关的责任 $\gamma(z_{nk})$ 的值. 每个点所用的这三种墨水的比例由 $\gamma(z_{nk})$ 给出,$k = 1, 2, 3$, 如图 15.5(c) 所示. 例如, 一个 $\gamma(z_{n1}) = 1$ 的数据点将染成红色, 而一个 $\gamma(z_{n2}) = \gamma(z_{n3}) = 0.5$ 的数据点将以等比例的蓝色和绿色染色, 因此呈现为青色. 而在图 15.5(a) 中, 将数据点标记成什么颜色依据的是它们实际所属的分量.

我们找到了一个涉及显式潜变量的 Gauss 混合分布的等效表示, 其中联合分布以图模型的形式给出. 这么做的收益看起来并不大. 然而, 现在我们已经能够用联合分布 $p(\boldsymbol{x}, z) = p(z)p(\boldsymbol{x} \mid z)$ 代替边缘分布 $p(\boldsymbol{x})$ 工作, 这将带来显著的简化 (特别是在引入 EM 算法时).

15.2.1 似然函数

假设我们有一组观测数据 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 我们希望使用 Gauss 混合模型来对这些数据进行建模. 我们可以将这个数据集表示为一个 $N \times D$ 的矩阵 $\boldsymbol{X}$, 其中第 n 行由 $\boldsymbol{x}_n^{\mathrm{T}}$ 给出.

由(15.6)可知, 似然函数的对数由下式给出:

$$\ln p(\boldsymbol{X} \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}. \tag{15.13}$$

最大化这个对数似然函数比针对单个 Gauss 分布更复杂. 难点在于(15.13)中的对数内部的求和项会导致对数不再直接作用于 Gauss 分布. 如果将对数似然函数关于模型参数的导数设为零, 我们将不再获得闭合解, 这一点我们很快就会看到.

在讨论如何最大化这个函数之前, 值得强调的是, 在应用到 Gauss 混合模型时, 最大似然框架存在一个显著的问题, 就是有奇异性. 为了简单起见, 考虑一个 Gauss 混合模型, 其组成部分的协方差矩阵由 $\boldsymbol{\Sigma}_k = \sigma_k^2 \boldsymbol{I}$ 给出, 其中 $\boldsymbol{I}$ 是单位矩阵, 但所得结论对于一般的协方差矩阵也是成立的. 假设该 Gauss 混合模型的一个分量, 比如第 j 个分量, 其均值 $\boldsymbol{\mu}_j$ 恰好等于某个数据点, 即 $\boldsymbol{\mu}_j = \boldsymbol{x}_n$, 则这个数据点将在似然函数中贡献一个如下形式的项:

$$\mathcal{N}\left(\boldsymbol{x}_n \mid \boldsymbol{x}_n, \sigma_j^2 \boldsymbol{I}\right) = \frac{1}{(2\pi)^{1/2}} \frac{1}{\sigma_j}. \tag{15.14}$$

注

(重构新增: 原文问题说明) (15.14)印出的是**一元**情形 ($D = 1$) 的取值:$D = 1$ 时 $\mathcal{N}(x \mid x, \sigma_j^2) = (2\pi\sigma_j^2)^{-1/2} = \frac{1}{(2\pi)^{1/2}} \frac{1}{\sigma_j}$, 正是右端的形状. 但同一式左端写的是向量 $\boldsymbol{x}_n$ 与矩阵 $\boldsymbol{I}$, 按此记号应为 D 元密度

$$\mathcal{N}\left(\boldsymbol{x}_n \mid \boldsymbol{x}_n, \sigma_j^2 \boldsymbol{I}\right) = \frac{1}{(2\pi\sigma_j^2)^{D/2}},$$

它在 $\sigma_j \to 0$ 时以 σ_j^{-D} 的速度发散, 比一元情形更快. 两种读法在“$\sigma_j \to 0$ 时趋于无穷”这一定性结论上完全一致, 下文与命题 15.4的论证对一般 D 成立. 这里照译原式, 并指出右端应按 $D = 1$ 理解, 引用时请以 D 元形式为准.

考虑极限 $\sigma_j \to 0$, 我们会看到该项趋向于无穷大, 所以对数似然函数也将趋向于无穷大. 由此可见, 最大化对数似然函数并不是一个良定义的问题, 因为这种奇异性总是存在的, 并且每当一个 Gauss 分量“坍缩”到一个特定的数据点时就会发生.

命题 15.4 (似然函数的奇异性)

设 $K \geqslant 1, \pi_j > 0$, 且数据集中至少含有一个点 $\boldsymbol{x}_n$. 则对数似然(15.13)在参数空间上没有上界: 对任意 $M > 0$, 都存在参数使得 $\ln p(\boldsymbol{X} \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) > M$.

♠

证明

固定 n 与分量 j, 取 $\boldsymbol{\mu}_j = \boldsymbol{x}_n$、$\boldsymbol{\Sigma}_j = \sigma_j^2 \boldsymbol{I}$, 其余分量任意固定. 由于每一项 $\pi_k \mathcal{N}(\cdot) \geqslant 0$, 有

$$p(\boldsymbol{x}_n \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) \geqslant \pi_j \, \mathcal{N}\left(\boldsymbol{x}_n \mid \boldsymbol{x}_n, \sigma_j^2 \boldsymbol{I}\right) = \frac{\pi_j}{(2\pi\sigma_j^2)^{D/2}}.$$

当 $\sigma_j \to 0^+$ 时右端趋于 $+\infty$, 故 $p(\boldsymbol{x}_n \mid \cdot) \to +\infty$; 又对数似然是 N 项之和, 其中第 n 项为 $\ln p(\boldsymbol{x}_n \mid \cdot)$, 而其余各项非负项的对数可能为负但有限, 于是

$$\ln p(\boldsymbol{X} \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) \geqslant \ln p(\boldsymbol{x}_n \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}) \longrightarrow +\infty.$$

因此对任意 M 都可取 σ_j 足够小使对数似然超过 M, 即无上界.

□

注

(重构新增) 这条命题说明“最大似然在 Gauss 混合模型中不是一个良定义的问题”: 不是解不存在, 而是目标函数没有上界, 所谓“最大值”根本不存在 ($\sup = +\infty$). 奇异性只要求某一个分量的均值精确落在某个数据点上、同时其协方差退化到零, 因此它在参数空间中是一个低维但**稠密可达**的“坏集”: 只要允许 $\boldsymbol{\Sigma}_k$ 任意小, 优化过程就可能被吸引过去. 条件“$\pi_j > 0$”不可省——若某个混合系数严格为零, 该分量对密度没有贡献, 也就不会产生这个奇异项. 对一般的 (非各向同性) 协方差矩阵, 证明完全相同, 只需把 $\sigma_j^2 \boldsymbol{I}$ 换成一个行列式趋于零的正定矩阵, 此时密度按 $|\boldsymbol{\Sigma}_j|^{-1/2}$ 发散. 这一现象是最大似然过拟合的典型例子, 处理办法见下文的启发式重置与参数先验 (小节 15.4.3).

这种奇异性提供了一个最大似然方法中可能出现过拟合的例子. 当把最大似然方法应用到 Gauss 混合模型时, 我们必须采取措施以避免找到这样的病态 (pathological) 解, 比如转而寻求似然函数的局部最大值, 这些局部最大值通常表现良好. 我们可以试图通过使用适当的启发式方法来避免奇异性, 例如检测到 Gauss 分量在坍缩时将其均值重置为随机选择的值, 同时将其协方差重置为某个较大值, 然后继续优化. 通过添加一个对应于参数先验分布的正则化项到对数似然函数中, 也可以避免奇异性 (参见小节 15.4.3).

在寻找最大似然解时还有一个问题, 因为对于任何给定的最大似然解, 一个包含 K 个组分的 Gauss 混合模型将有总共 $K!$ 个等效解, 对应于将 K 组参数分配给 K 个组分的 $K!$ 种方式. 换句话说, 对于参数值空间中的任何给定 (非退化) 点, 都会存在额外的 $K! - 1$ 个点. 所有这些点都将产生完全相同的分布. 这个问题称为可识别性 (identifiability) 问题 (Casella and Berger, 2002), 当我们希望解释模型发现的参数值时, 这是一个重要问题. 当我们讨论具有连续潜变量的模型 (参见第 16 章) 时, 可识别性问题也会出现. 然而, 当我们寻找一个好的密度模型时, 这个问题无关紧要, 因为任何等效解都和其他解一样好.

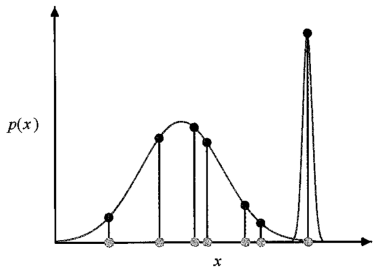


图 15.6: 在 Gauss 混合模型中, 似然函数中的奇异点是如何出现的. 请和图 2.9中显示的没有奇异点的情况进行比较

命题 15.5 (交换对称性导致的 $K!$ 个等价解)

设 $\boldsymbol{\theta} = (\pi_1, \cdots, \pi_K, \boldsymbol{\mu}_1, \cdots, \boldsymbol{\mu}_K, \boldsymbol{\Sigma}_1, \cdots, \boldsymbol{\Sigma}_K)$ 给出(15.6)的一个参数组, 且各分量互不相同 (非退化). 对 $\{1, \cdots, K\}$ 的任一置换 σ, 令 $\pi'_k = \pi_{\sigma(k)}$、$\boldsymbol{\mu}'_k = \boldsymbol{\mu}_{\sigma(k)}$、$\boldsymbol{\Sigma}'_k = \boldsymbol{\Sigma}_{\sigma(k)}$, 则 $\boldsymbol{\theta}'$ 与 $\boldsymbol{\theta}$ 给出同一个密度. 不同的置换给出不同的参数组, 因此等价参数组恰有 $K!$ 个.

证明

对任意 $\boldsymbol{x}$,

$$\sum_{k=1}^{K} \pi'_k \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}'_k, \boldsymbol{\Sigma}'_k) = \sum_{k=1}^{K} \pi_{\sigma(k)} \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_{\sigma(k)}, \boldsymbol{\Sigma}_{\sigma(k)}) = \sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j),$$

这是因为 σ 只是把求和指标 $\{1, \cdots, K\}$ 重新排列了一遍 (令 $j = \sigma(k)$). 故两参数组给出同一密度.

再证不同的置换给出不同的参数组. 置换群有 $K!$ 个元素; 若 $\sigma \neq \sigma'$, 则存在 k 使 $\sigma(k) \neq \sigma'(k)$. 若两参数组相同, 则 $\pi_{\sigma(k)} = \pi_{\sigma'(k)}$、$\boldsymbol{\mu}_{\sigma(k)} = \boldsymbol{\mu}_{\sigma'(k)}$、$\boldsymbol{\Sigma}_{\sigma(k)} = \boldsymbol{\Sigma}_{\sigma'(k)}$; 由非退化性 (各分量互不相同, 故分量由其参数三元组唯一标识), 这迫使 $\sigma(k) = \sigma'(k)$, 矛盾. 因此这 $K!$ 个参数组两两不同.

□

注

(重构新增) 这条命题是“可识别性”讨论的核心: 模型作为**密度**是可识别的 (不同分量组合给出不同密度, 在非退化点处), 但作为**参数化**不可识别 (同一个密度对应 $K!$ 个参数组).$K!$ 来自对称群而非参数的连续自由度, 所以它不是“近似解”而是**精确**的等价: 两参数组给出的 $p(\boldsymbol{x})$ 处处相等. 后果有两点. 其一, 任何以参数为对象的推断 (例如“第 3 个分量对应哪个簇”) 都必须先固定一个规范次序或事后重新排序, 否则结论没有意义. 其二,$K!$ 个等价解使似然函数的驻点成倍出现, 但它们的最优值相同, 故对“找好的密度模型”这一目标没有影响; 这也解释了为什么 EM 算法可以收敛到其中任意一个. 非退化条件不可省: 若两个分量参数完全相同, 则交换它们得到的是同一个参数组,$K!$ 的计数需要按重数修正.

15.2.2 最大似然

在为含有隐变量的模型寻找最大似然解的探索中,EM 算法是一种优雅而强大的方法 (Dempster, Laird and Rubin, 1977; McLachlan and Krishnan, 1997). 在本章中, 我们将先后给出三种不同的 EM 算法推导, 每一种都比前一种更加通用. 下面让我们在 Gauss 混合模型的背景下以相对非正式的方式开始. 首先需要强调的是,EM 算法具有广泛的适用性, 它的底层概念我们将在本书涉及的多个模型中遇到.

我们选择从写下似然函数最大值处必须满足的条件开始. 将(15.13)中的 $\ln p(\boldsymbol{X} \mid \boldsymbol{\pi}, \boldsymbol{\mu}, \boldsymbol{\Sigma})$ 关于 Gauss 分量均值 $\boldsymbol{\mu}_k$ 的导数设为零, 便可得到

$$0 = \sum_{n=1}^{N} \underbrace{\frac{\pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}}_{\gamma(z_{nk})} \boldsymbol{\Sigma}_k^{-1} (\boldsymbol{x}_n - \boldsymbol{\mu}_k), \tag{15.15}$$

其中我们利用了 Gauss 分布的形式(3.28). 注意, 后验概率或责任 $\gamma(z_{nk})$ 由(15.12)给出, 因而自然地出现在公式的右侧. 乘以 $\boldsymbol{\Sigma}_k$(我们假设它是非奇异的) 并重新排列后, 便可得到

$$\boldsymbol{\mu}_k = \frac{1}{N_k}\sum_{n=1}^{N}\gamma(z_{nk})\boldsymbol{x}_n, \tag{15.16}$$

其中我们定义了

$$N_k = \sum_{n=1}^{N}\gamma(z_{nk}). \tag{15.17}$$

可以将 N_k 解释为分配给聚类 k 的有效点数. 仔细观察这个解的形式, 可以看到, 第 k 个 Gauss 分量的均值 $\boldsymbol{\mu}_k$ 是通过对数据集中的所有点进行加权平均得到的. 在这个加权因子中, 数据点 $\boldsymbol{x}_n$ 的权重是由后验概率 $\gamma(z_{nk})$ 给出的, 即第 k 个 Gauss 分量负责生成 $\boldsymbol{x}_n$ 的概率.

如果将 $\ln p(\boldsymbol{X}\,|\,\boldsymbol{\pi},\boldsymbol{\mu},\boldsymbol{\Sigma})$ 关于 $\boldsymbol{\Sigma}_k$ 的导数设为零并遵循类似的推理线索, 利用单个 Gauss 分布的最大似然解的结果 (参见小节 3.2.7), 便可得到

$$\boldsymbol{\Sigma}_k = \frac{1}{N_k}\sum_{n=1}^{N}\gamma(z_{nk})(\boldsymbol{x}_n-\boldsymbol{\mu}_k)(\boldsymbol{x}_n-\boldsymbol{\mu}_k)^{\mathsf{T}}. \tag{15.18}$$

这与用单个 Gauss 分布拟合数据集的对应结果形式相同, 但每个数据点都由相应的后验概率加权, 并且分母由与相应分量相关联的有效点数给出.

最后, 我们调整混合系数 π_k 的值, 目的是最大化对数似然函数 $\ln p(\boldsymbol{X}\,|\,\boldsymbol{\pi},\boldsymbol{\mu},\boldsymbol{\Sigma})$. 此处我们必须考虑一个约束[(15.8)], 该约束要求混合系数的和为 1. 这可以通过使用 Lagrange 乘子 λ 并最大化以下量来实现:

$$\ln p(\boldsymbol{X}\,|\,\boldsymbol{\pi},\boldsymbol{\mu},\boldsymbol{\Sigma}) + \lambda\left(\sum_{k=1}^{K}\pi_k - 1\right). \tag{15.19}$$

这样便可得到

$$0 = \sum_{n=1}^{N}\frac{\mathcal{N}(\boldsymbol{x}_n\,|\,\boldsymbol{\mu}_k,\boldsymbol{\Sigma}_k)}{\sum_j \pi_j\mathcal{N}(\boldsymbol{x}_n\,|\,\boldsymbol{\mu}_j,\boldsymbol{\Sigma}_j)} + \lambda. \tag{15.20}$$

可以看到 “责任” 再次出现了. 如果对(15.20)的两边乘以 π_k 并利用(15.8)对 k 进行求和, 就会发现 $\lambda = -N$. 消除 λ 并重新排列后, 便可得到

$$\pi_k = \frac{N_k}{N}. \tag{15.21}$$

因此, 第 k 个 Gauss 分量的混合系数由该分量对解释数据点所要承担的平均责任给出.

命题 15.6 (Gauss 混合模型的最大似然方程)

设 $N_k = \sum_n \gamma(z_{nk}) > 0$,$\boldsymbol{\Sigma}_k$ 非奇异. 则对数似然(15.13)的任一驻点 $(\boldsymbol{\pi},\boldsymbol{\mu},\boldsymbol{\Sigma})$ 满足(15.16)、(15.18)与(15.21), 其中 $\gamma(z_{nk})$ 由(15.12)定义.

证明

(关于 $\boldsymbol{\mu}_k$) 记 $p_n = \sum_j \pi_j \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$. 由 $\partial \ln p_n / \partial \boldsymbol{\mu}_k = \pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \cdot \big(-\boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}_k)\big)/p_n$, 并注意 $\pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)/p_n = \gamma(z_{nk})$, 得

$$\frac{\partial \ln p(\boldsymbol{X} \mid \cdot)}{\partial \boldsymbol{\mu}_k} = -\sum_{n=1}^{N} \gamma(z_{nk})\, \boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}_k).$$

置零并左乘 $\boldsymbol{\Sigma}_k$ 得 $\sum_n \gamma(z_{nk})(\boldsymbol{x}_n - \boldsymbol{\mu}_k) = 0$, 即(15.15), 从而 $\boldsymbol{\mu}_k = N_k^{-1} \sum_n \gamma(z_{nk}) \boldsymbol{x}_n$.

(关于 $\boldsymbol{\Sigma}_k$) 由 $\partial \ln p_n / \partial \boldsymbol{\Sigma}_k = \gamma(z_{nk})\, \partial \ln \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)/\partial \boldsymbol{\Sigma}_k$ 以及单个 Gauss 密度的矩阵导数

$$\frac{\partial}{\partial \boldsymbol{\Sigma}_k} \ln \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{2}\boldsymbol{\Sigma}_k^{-1} - \frac{1}{2}\boldsymbol{\Sigma}_k^{-1}(\boldsymbol{x}_n - \boldsymbol{\mu}_k)(\boldsymbol{x}_n - \boldsymbol{\mu}_k)^{\mathsf{T}}\boldsymbol{\Sigma}_k^{-1},$$

置零得

$$\frac{1}{2}\boldsymbol{\Sigma}_k^{-1} \sum_{n=1}^{N} \gamma(z_{nk}) = \frac{1}{2}\boldsymbol{\Sigma}_k^{-1} \left[\sum_{n=1}^{N} \gamma(z_{nk})(\boldsymbol{x}_n - \boldsymbol{\mu}_k)(\boldsymbol{x}_n - \boldsymbol{\mu}_k)^{\mathsf{T}} \right] \boldsymbol{\Sigma}_k^{-1}.$$

左乘 $\boldsymbol{\Sigma}_k$、右乘 $\boldsymbol{\Sigma}_k$, 并用 $N_k = \sum_n \gamma(z_{nk})$, 即得(15.18). 这与单 Gauss 最大似然协方差 (命题 3.15) 的推导完全平行, 区别只在于每个数据点带着权重 $\gamma(z_{nk})$.

(关于 $\boldsymbol{\pi}_k$) 在约束(15.8)下引入 Lagrange 乘子, 得(15.19). 对 π_k 求偏导并置零:

$$0 = \sum_{n=1}^{N} \frac{\mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{p_n} + \lambda,$$

即(15.20). 两边乘 π_k, 注意 $\pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)/p_n = \gamma(z_{nk})$, 得 $\sum_n \gamma(z_{nk}) + \lambda \pi_k = 0$. 对 k 求和并用 $\sum_k \pi_k = 1$ 与 $\sum_k \gamma(z_{nk}) = 1$:

$$\sum_{k=1}^{K} \sum_{n=1}^{N} \gamma(z_{nk}) + \lambda \sum_{k=1}^{K} \pi_k = N + \lambda = 0,$$

故 $\lambda = -N$. 代回得 $\pi_k = N_k/N$, 即(15.21).

□

注

(重构新增) 这条命题给出了 EM 算法在 GMM 上的 M 步骤的全部内容, 但要注意它是一个**隐式**方程组: 右端的 $\gamma(z_{nk})$ 通过(15.12)依赖于 $\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k, \boldsymbol{\pi}_k$ 本身, 所以(15.16)、(15.18)、(15.21)并不是参数的闭合解, 而是一组“若把责任看作已知量则成立”的方程. 这正是 EM 的出发点: 把责任当作缺失数据代入, 方程就变成显式的. N_k 的解释是“有效点数”, 它不必是整数; 当责任退化为 0/1 时 N_k 就是第 k 个聚类中数据点的个数, 于是(15.16)退化为 K 均值的均值更新(15.4). 推导中用到的两个条件——$\boldsymbol{\Sigma}_k$ 非奇异 (否则 $\boldsymbol{\Sigma}_k^{-1}$ 无定义) 与 $N_k > 0$(否则均值无定义)——在数值实现中都要显式检查: 前者由命题 15.4说明可能失效, 后者意味着某个分量“没有分到任何质量”, 此时应删除该分量或重新初始化.

请注意,(15.16)、(15.18)和(15.21)并不构成 Gauss 混合模型参数的闭合解, 因为责任 $\gamma(z_{nk})$ 通过(15.12)以复杂的方式依赖于这些参数. 然而, 这些结果确实提出了一种简单的迭代方案来寻找最大似然问题的解. 正如我们将看到的, 对于 Gauss 混合模型的特定情况, 这实际上是 EM 算法的一个实例. 我们首先选择均值、协方差和混合系数的一些初始值, 然后交替进行以下两个步骤——期望步骤 (E 步骤) 和最大化步骤 (M 步骤). 在 E 步骤中, 我们使用参数的当前值来估计后验概率或责任, 其由(15.12)给出. 然后在 M 步骤中使用这些概率重新估计均值、协方差和混合系数, 它们分别由(15.16)、(15.18)和(15.21)给出. 请注意, 当决定这样做时, 我们首先使用(15.16)计算新的均值, 然后使用这些新值通过(15.18)找到协方差, 其与单个 Gauss 分布的相应结果一致. 我们将证明每次进行 E 步骤之后, 紧跟着进行的 M 步骤导致的参数更新都会增大对数似然函数的值. 在实践中, 当对数似然函数的值的变化, 或者说参数的值的变化低于某个阈值时, 算法就被认为已经收敛.

下面我们用重缩放的老忠实喷泉数据集来说明应用于 Gauss 混合模型的 EM 算法, 如图 15.7所示. 这里使用

了两个 Gauss 分量, 其聚类中心使用与图 15.1中 K 均值算法相同的值进行初始化, 协方差矩阵则初始化为与单位矩阵成比例.图 15.7(a) 显示了绿色的数据点以及 Gauss 混合模型的初始配置, 其中两个 Gauss 分量的一倍标准差轮廓线分别显示为蓝色和红色圆圈.图 15.7(b) 显示了 E 步骤的初始结果, 其中每个数据点使用与从蓝色分量生成的后验概率相等的蓝色墨水比例绘制, 并且绘制时还使用了由红色分量生成的后验概率所给出的相应比例的红色墨水. 因此, 那些属于两个聚类的概率大致相等的点呈现为紫色. 第一次进行 M 步骤后的情况如图 15.7(c) 所示, 蓝色分量的均值已移动到由每个数据点属于蓝色聚类的概率加权的数据集的均值上. 换句话说, 其已经移动到了蓝色墨水的中心位置. 同样, 蓝色分量的协方差已设定为蓝色墨水的协方差. 对于红色分量也有类似的结果.图 15.7(d)~图 15.7(f) 分别展示了进行 2 次、5 次和 20 次完整的 EM 循环后的结果. 在图 15.7(f) 中, 算法接近收敛.

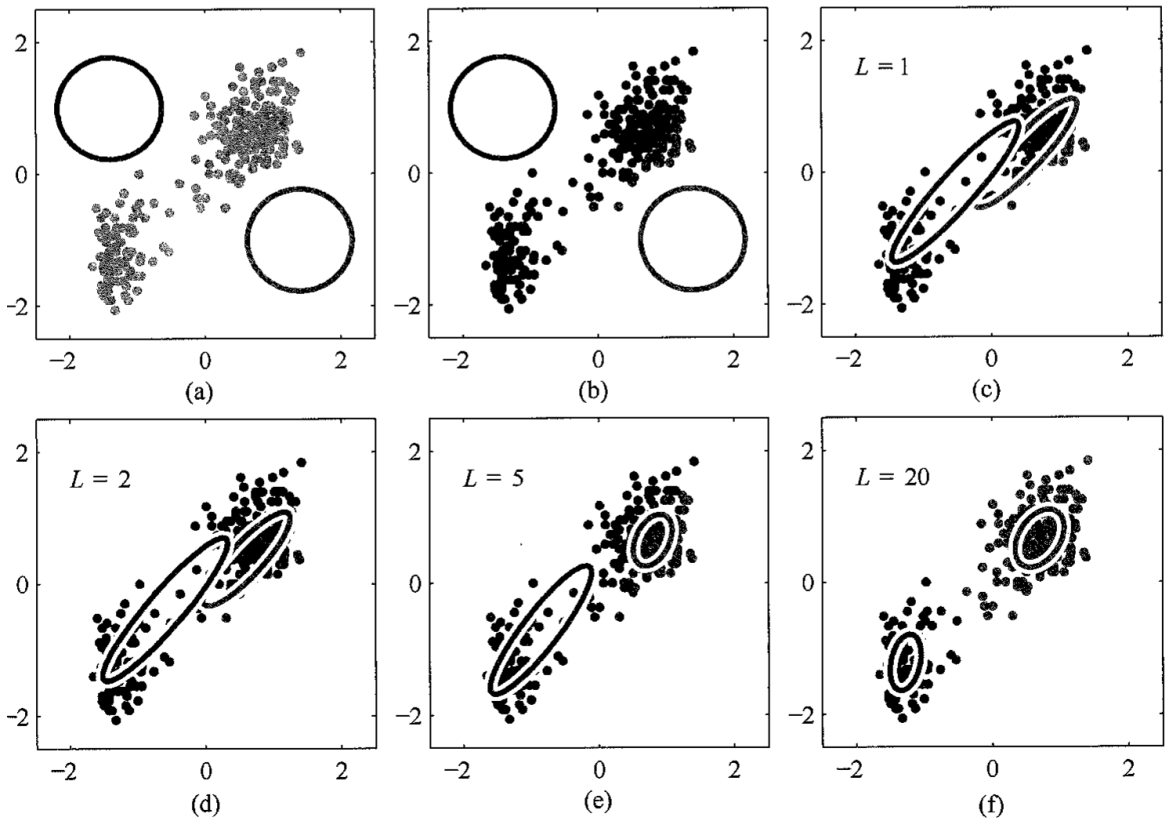


图 15.7: 将 EM 算法应用于老忠实喷泉数据集, 该数据集在图 15.1中用于说明 K 均值算法

注

(重构新增) 图 15.7中每个分图左上角的 L 表示已经完成的完整 EM 循环次数, 而每个分量画出的两条轮廓线分别是一倍标准差与两倍标准差的等密度线. 由于参数是在 E 步骤之后、M 步骤之前被固定的, 图中每一轮“颜色(责任)—轮廓 (参数)”的交替正好对应命题 15.6中方程组的迭代求根: 先按当前参数算责任, 再按责任算参数. 注意 (f) 中两个分量仍有部分重叠, 这说明 EM 收敛到的是对数似然的一个局部极大点, 而不是把两个簇完全分开的“理想”解; 初值若选得更差, 收敛结果可能完全不同 (见本节末的讨论).

请注意, 与 K 均值算法相比,EM 算法需要进行更多的迭代才能达到 (近似) 收敛, 而且每个循环都需要进行更多的计算. 因此, 人们通常会运行 K 均值算法来找到 Gauss 混合模型的初始状态, 随后使用 EM 算法进行调整. 一种便捷的做法就是把协方差矩阵初始化为 K 均值算法以找到各个聚类的样本协方差, 而混合系数可以设置为数据点被分配给相应聚类的比例. 为了规避似然函数的奇异性——Gauss 分量坍缩至特定数据点的情况——我们必须采取诸如参数正则化的技术. 需要强调的是, 对数似然函数通常会有多个局部最大值, 并且 EM 算法不能保证找到全局最大值. 考虑到应用于 Gauss 混合模型的 EM 算法的重要性, 我们在算法 15.2 中对它做了总结.

算法 15.2　应用于 Gauss 混合模型的 EM 算法

Input: 初始模型参数 $\{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}, \{\pi_k\}$
　　数据集 $\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$
Output: 最终模型参数 $\{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}, \{\pi_k\}$
repeat
　　// E 步骤
　　for $n \in \{1, \cdots, N\}$ **do**
　　　　for $k \in \{1, \cdots, K\}$ **do**

$$\gamma(z_{nk}) \leftarrow \pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \Big/ \sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$

　　　　end for
　　end for
　　// M 步骤
　　for $k \in \{1, \cdots, K\}$ **do**

$$N_k \leftarrow \sum_{n=1}^{N} \gamma(z_{nk})$$

$$\boldsymbol{\mu}_k \leftarrow \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) \boldsymbol{x}_n$$

$$\boldsymbol{\Sigma}_k \leftarrow \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) (\boldsymbol{x}_n - \boldsymbol{\mu}_k)(\boldsymbol{x}_n - \boldsymbol{\mu}_k)^{\mathsf{T}}$$

$$\pi_k \leftarrow \frac{N_k}{N}$$

　　end for
　　// 对数似然

$$\mathcal{L} \leftarrow \sum_{n=1}^{N} \ln \left\{ \sum_{k=1}^{K} \pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}$$

until 收敛
return $\{\boldsymbol{\mu}_k\}, \{\boldsymbol{\Sigma}_k\}, \{\pi_k\}$

Gauss 混合模型非常灵活, 如果模型参数选择得当, 只需要给定足够数量的分量, 就可以高精度地逼近复杂的数据分布. 然而在实践中, 尤其在高维空间中, 分量的数量可能非常大. 这个问题在图 15.8中有所体现. 尽管如此,Gauss 混合模型仍十分有用. 此外, 理解 Gauss 混合模型能为我们理解具有连续潜变量的模型和基于深度神经网络的生成式模型奠定基础, 后者对高维空间的扩展性要好得多 (参见第 16 章).

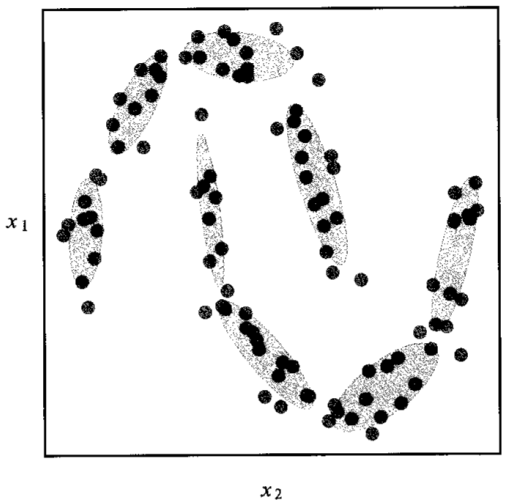


图 15.8: 拟合到“双月”数据集的 Gauss 混合模型, 这表明可能需要大量的混合分量才能准确表示复杂的数据分布. 这里椭圆表示对应混合分量的恒定密度等值线. 随着我们进入更高维空间, 精确建模分布所需的分量的数量可能会大到不可接受

15.3 EM 算法

本节将讨论更一般的 EM 算法, 并特别关注隐变量的作用. 如前所述, 我们用 $\boldsymbol{X}$ 表示所有观测数据点的集合, 其中的第 n 行代表 $\boldsymbol{x}_n^{\mathsf{T}}$. 同样, 相应的隐变量则用一个 $N \times K$ 的矩阵 $\boldsymbol{Z}$ 来表示, 每一行记为 $\boldsymbol{z}_n^{\mathsf{T}}$. 如果我们假设这些数据点是从分布中独立抽取的, 则可以使用图 15.9所示的图形来表示这个独立同分布数据集的 Gauss 混合模型. 所有模型参数的集合记为 θ, 因此对数似然函数由下式给出:

$$\ln p(\boldsymbol{X} \mid \theta) = \ln \left\{ \sum_{\boldsymbol{Z}} p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta) \right\}. \tag{15.22}$$

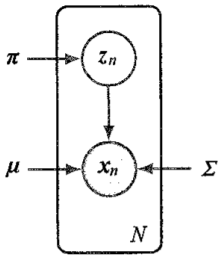


图 15.9: Gauss 混合模型的图形表示, 该 Gauss 混合模型可以用于一组 N 个独立同分布的数据点 $\boldsymbol{x}_n$ 及对应的隐变量 $\boldsymbol{z}_n$, 其中 $n = 1, \cdots, N$

注意, 我们的讨论同样适用于连续隐变量, 只需要将对 $\boldsymbol{Z}$ 的求和替换为积分即可. 另请注意, 对潜变量的求和出现在对数内部. 即使联合分布 $p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$ 属于指数族分布, 但由于这个求和项, 边缘分布 $p(\boldsymbol{X} \mid \theta)$ 通常也不会属于指数族分布. 求和的存在会阻止我们将对数直接作用于联合分布, 导致最大似然解的表达式复杂化.

现在假设对于 $\boldsymbol{X}$ 中的每个观测值, 我们都被告知对应的隐变量 $\boldsymbol{Z}$ 的值, 如图 15.5(a) 所示.

定义 15.5 (完整数据与不完整数据)

将被观测到的数据 $\boldsymbol{X}$ 与对应的隐变量 $\boldsymbol{Z}$ 合在一起称为**完整数据集** (complete data), 记作 $\{\boldsymbol{X}, \boldsymbol{Z}\}$; 相应地, 实际观测到的数据 $\boldsymbol{X}$ 称为**不完整数据** (incomplete data).

注

(重构新增) “完整/不完整” 这一对术语描述的不是数据量, 而是**信息**:$\boldsymbol{X}$ 是实际拿到的部分,$\boldsymbol{Z}$ 是生成过程中存在但未被记录的部分. 把它称为 “数据集” 是为了强调 $\{\boldsymbol{X},\boldsymbol{Z}\}$ 是一个可以写出联合似然 $p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)$ 的对象. 这一定义的作用在于区分两个优化问题: 完整数据的最大似然 (通常有闭式解, 见命题 15.7) 与不完整数据的最大似然 (对数内含有求和, 通常没有闭式解).EM 的全部技巧就是把后者换成前者在 $\boldsymbol{Z}$ 的后验下的期望. 注意这里的 “缺失” 是就建模而言的:$\boldsymbol{Z}$ 并非 “丢失的数据”, 它在现实中可能根本不可观测 (例如 “该像素属于哪个分量”). 条件在于我们确实写出了联合分布 $p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)$——若模型没有显式的隐变量结构, 就无从谈完整数据.

完整数据集的似然函数可以简单地采取 $\ln p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)$ 的形式, 我们假设这个完整数据集的对数似然函数的最大化是比较简单的.

然而在实践中, 我们没有得到完整的数据集 $\{\boldsymbol{X},\boldsymbol{Z}\}$, 而只有不完整的数据 $\boldsymbol{X}$. 隐变量 $\boldsymbol{Z}$ 的值的状态信息仅由后验分布 $p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)$ 给出. 由于无法使用完整数据集的对数似然函数, 我们转而考虑其在隐变量的后验分布下的期望值, 这对应于 (正如我们将看到的)EM 算法的 E 步骤. 在随后的 M 步骤中, 我们最大化这个期望值. 如果当前对参数的估计记为 θ^{old}, 那么一对连续的 E 步骤和 M 步骤将产生修正后的估计 θ^{new}.

EM 算法通过选择参数的某个起始值 θ_0 来进行初始化. 尽管这种期望的使用看起来有些武断, 但是当我们在节 15.4对 EM 算法进行更深入的讨论时, 我们将看到做出这个选择的动机. 在 E 步骤中, 我们使用当前的参数值 θ^{old} 来找出 $p(\boldsymbol{Z}\mid\boldsymbol{X},\theta^{\text{old}})$ 给出的隐变量的后验分布. 然后使用这个后验分布找出用于对一些通用参数值 θ 进行估计的完整数据对数似然函数的期望.

定义 15.6 (Q 函数与 EM 的两步)

将这个期望记为 $\boldsymbol{Q}(\theta,\theta^{\text{old}})$, 其由下式给出:

$$\boldsymbol{Q}(\theta,\theta^{\text{old}})=\sum_{\boldsymbol{Z}}p(\boldsymbol{Z}\mid\boldsymbol{X},\theta^{\text{old}})\ln p(\boldsymbol{X},\boldsymbol{Z}\mid\theta). \tag{15.23}$$

在 M 步骤中, 我们通过最大化(15.23)所示的函数来确定修正后的参数估计:

$$\theta^{\text{new}}=\arg\max_{\theta}\boldsymbol{Q}(\theta,\theta^{\text{old}}). \tag{15.24}$$

注

(重构新增) $Q(\theta, \theta^{\text{old}})$ 有两个参数位, 这一点是理解 EM 的关键: 第二个位置 θ^{old} 只决定用哪个后验来加权, 在 M 步骤中是固定的常数; 第一个位置 θ 才是被优化的变量. 因此 Q 不是一个函数而是“一族”函数, 每次 E 步骤换一个. 定义中把对数放在联合分布上 (而不是边缘分布), 正是为了避免求和出现在对数内部——这是定义(15.23)与前文不完整数据对数似然(15.22)的唯一区别, 也是 M 步骤“根据我们的假设是可行的”的原因. 注意 Q 对 θ 的依赖只通过 $\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$, 所以权重的归一化 ($\sum_{\boldsymbol{Z}} p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}}) = 1$) 保证了 Q 是 $\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$ 的一个加权平均, 从而 $\theta \mapsto Q(\theta, \theta^{\text{old}})$ 的光滑性与 $\theta \mapsto \ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$ 相同. 若 $\boldsymbol{Z}$ 连续, 求和应换成对 $p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}})$ 的积分, 其余不变.

请注意, 在函数 $Q(\theta, \theta^{\text{old}})$ 的定义中, 对数直接作用于联合分布 $p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$, 因此相应的 M 步骤根据我们的假设是可行的. 算法 15.3 对通用的 EM 算法做了总结. 正如我们稍后将展示的, 每个 EM 循环都会增大不完整数据对数似然函数的值 (除非它已经处于局部最大值)(参见小节 15.4.1).

算法 15.3　通用的 EM 算法

Input: 联合分布 $p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$
　初始参数 θ^{old}
　数据集 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$
Output: 最终参数 θ
repeat
　$Q(\theta, \theta^{\text{old}}) \leftarrow \sum_{\boldsymbol{Z}} p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}}) \ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$　// E 步骤
　$\theta^{\text{new}} \leftarrow \arg\max_{\theta} Q(\theta, \theta^{\text{old}})$　// M 步骤
　$\mathcal{L} \leftarrow p(\boldsymbol{X} \mid \theta^{\text{new}})$　// 计算对数似然
　$\theta^{\text{old}} \leftarrow \theta^{\text{new}}$　// 更新参数
until 收敛
return θ^{new}

EM 算法也可以用来寻找 MAP(最大后验概率) 解, 这适用于定义参数时引入先验分布 $p(\theta)$ 的模型 (见练习 15.5). 在这种情况下,E 步骤与最大似然示例中的情形相同, 而在 M 步骤中, 要最大化的量由 $Q(\theta, \theta^{\text{old}}) + \ln p(\theta)$ 给出. 适当的先验选择能消除图 15.6所示类型的奇异性.

这里我们考虑了当存在离散隐变量时如何使用 EM 算法最大化似然函数. 然而, 当未观测变量对应于数据集中缺失的值时, 也可以应用 EM 算法. 观测值的分布是通过取所有变量的联合分布, 然后在缺失的变量上进行边缘化得到的. 接下来便可以使用 EM 算法来最大化相应的似然函数. 如果数据值是随机缺失的, 则意味着导致缺失值的机制不依赖于未观测到的值. 但在许多情况下, 事实并非如此. 例如, 当传感器测量的量超过某个阈值时, 传感器就无法返回值.

15.3.1 Gauss 混合模型

现在我们考虑将这种隐变量视角的 EM 算法应用于 Gauss 混合模型的特定案例. 回顾一下, 我们的目标是最大化对数似然函数(15.13), 该函数可以使用观测数据集 $\boldsymbol{X}$ 来计算, 并且我们发现由于在对数内部进行了关于 k 的求和, 这比单个 Gauss 分布的情况要困难得多. 假设除了观测数据集 $\boldsymbol{X}$, 我们还得到了相应的离散变量 $\boldsymbol{Z}$ 的值. 回想一下,图 15.5(a) 显示了一个完整数据集 (即包含标签来显示每个数据点是由哪个分量生成的),图 15.5(b) 则显示了相应的不完整数据集.图 15.10展示了完整数据的图形表示.

现在考虑对完整数据集 $\{\boldsymbol{X}, \boldsymbol{Z}\}$ 的似然函数进行最大化的问题. 根据(15.9)和(15.10), 这个似然函数的形式如下:

$$p(\boldsymbol{X}, \boldsymbol{Z} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \prod_{n=1}^{N} \prod_{k=1}^{K} \pi_k^{z_{nk}} \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)^{z_{nk}}, \tag{15.25}$$

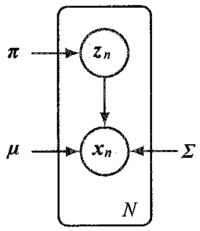


图 15.10: 这与图 15.9中显示的图形相同, 不同之处在于我们现在假设离散变量 $\boldsymbol{z}_n$ 和数据变量 $\boldsymbol{x}_n$ 都是可观测的

其中 z_{nk} 表示 $\boldsymbol{z}_n$ 的第 k 个分量. 取对数后, 可以得到

$$\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \left\{ \ln \pi_k + \ln \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right\}. \tag{15.26}$$

命题 15.7 (完全数据的对数似然与混合系数的更新)

设完整数据 $\{\boldsymbol{X}, \boldsymbol{Z}\}$ 的似然由(15.25)给出, 则其对数似然为(15.26); 固定 $\boldsymbol{\mu}, \boldsymbol{\Sigma}$ 并在约束 $\sum_k \pi_k = 1$ 下关于 $\boldsymbol{\pi}$ 最大化(15.26), 得到的解为

$$\pi_k = \frac{1}{N} \sum_{n=1}^{N} z_{nk}. \tag{15.27}$$

♠

证明

对(15.25)取对数, 乘积变为求和, 幂次变为系数:

$$\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \cdot) = \sum_{n=1}^{N} \sum_{k=1}^{K} \left[z_{nk} \ln \pi_k + z_{nk} \ln \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right],$$

即(15.26).

关于 $\boldsymbol{\pi}$ 最大化时, 含 π_k 的项只有 $\sum_n z_{nk} \ln \pi_k$. 引入 Lagrange 乘子 λ, 考虑

$$\sum_{k=1}^{K} \left(\sum_{n=1}^{N} z_{nk} \right) \ln \pi_k + \lambda \left(\sum_{k=1}^{K} \pi_k - 1 \right).$$

对 π_k 求偏导并置零得 $\sum_n z_{nk}/\pi_k + \lambda = 0$, 即 $\pi_k = -\sum_n z_{nk}/\lambda$. 对 k 求和并用 $\sum_k \pi_k = 1$ 与 $\sum_k z_{nk} = 1$:

$$1 = \sum_{k=1}^{K} \pi_k = -\frac{1}{\lambda} \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} = -\frac{N}{\lambda},$$

故 $\lambda = -N$, 代回即得(15.27).

□

注

(重构新增) 这条命题说明了为什么“完整数据”情形容易处理:(15.26)中求和与对数已经交换了位置,ln 直接作用在每个 Gauss 分量上, 而 Gauss 属于指数族, 其最大似然有显式解. 对比不完整数据的(15.13), 两者的差别只在于 z_{nk} 是否已知: 已知时 $z_{nk} \in \{0, 1\}$ 起到“选择器”的作用, 未知时它被替换为责任 $\gamma(z_{nk}) \in [0, 1]$(见命题 15.8).(15.27)的直观含义是“混合系数等于分配给该分量的数据点的比例”; 由于 z_{nk} 是硬标签, 右端是严格的频率, 而不是加权平均. 条件 $\sum_k z_{nk} = 1$ 在推导中用于把 N 提出来; 若某个数据点没有被分配到任何分量 ($\sum_k z_{nk} \neq 1$), 该式需按实际计数修正.

与不完整数据的对数似然函数 [(15.13)] 相比, 可以看出 k 的求和项与对数项已经交换了位置. 对数现在直接作用于 Gauss 分布, 而 Gauss 分布本身就是指数族分布的成员. 显然, 这使得最大似然问题的解决方案简单多了.

首先考虑均值和协方差的最大化问题. 因为 $\boldsymbol{z}_n$ 是一个 K 维向量, 除了其中一个元素的值为 1, 其他所有元素的值都为 0, 所以完整数据的对数似然函数仅仅是 K 个独立贡献的和, 每个分量对应一个贡献. 因此, 均值或协方差的最大化处理实际上就像对单个 Gauss 分布的处理一样, 只不过前者仅涉及 “分配” 给该分量的数据点子集. 对于混合系数的最大化, 注意, 由于(15.8)中的求和约束, 不同 k 值的这些系数是耦合的. 另外, 和之前一样, 这也可以使用 Lagrange 乘子来保证, 从而导致(15.27). 因此, 混合系数等于分配给相应分量的数据点的比例.

由此可见, 完整数据的对数似然函数可以在闭合形式中轻易地最大化. 然而, 实际上我们没有隐变量的值. 如前所述, 我们考虑了关于隐变量后验分布的完整数据对数似然函数的期望. 使用(15.9)和(15.10)并结合 Bayes 定理, 这个后验分布的形式如下:

$$p(\boldsymbol{Z} \mid \boldsymbol{X}, \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}) \propto \prod_{n=1}^{N} \prod_{k=1}^{K} \left[\pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\right]^{z_{nk}} . \tag{15.28}$$

可以看到这个分布在 n 上是可分解的, 所以在这个后验分布下,$\boldsymbol{z}_n$ 是独立的 (见练习 15.6), 这可以很容易地通过检查图 15.9中的有向图并使用 d 分离准则来验证 (参见节 11.2).

命题 15.8 (责任与完整数据对数似然的期望)

在后验(15.28)下, 指示变量的期望为

$$\begin{aligned} \mathbb{E}[z_{nk}] &= \frac{\sum_{\boldsymbol{z}_n} z_{nk} \prod_{k'} \left[\pi_{k'} \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_{k'}, \boldsymbol{\Sigma}_{k'})\right]^{z_{nk'}}}{\sum_{\boldsymbol{z}_n} \prod_{j} \left[\pi_j \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)\right]^{z_{nj}}} \\ &= \frac{\pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} = \gamma(z_{nk}), \end{aligned} \tag{15.29}$$

即第 k 个 Gauss 分量对数据点 $\boldsymbol{x}_n$ 所要承担的责任. 因此, 完整数据对数似然函数的期望为

$$\mathbb{E}_{\boldsymbol{Z}}\left[\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})\right] = \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma(z_{nk}) \left\{\ln \pi_k + \ln \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)\right\} . \tag{15.30}$$

♠

证明

由(15.28),$\boldsymbol{z}_n$ 之间相互独立, 故对 $\boldsymbol{Z}$ 求期望可逐 n 进行, 而每个 n 只涉及 $\boldsymbol{z}_n$ 的 K 种取值. 对固定的 n, 分子中只有 $z_{nk} = 1$ 的那一项非零 (其余项含因子 $z_{nk} = 0$), 且该项中只有 $k' = k$ 的因子存活, 值为 $\pi_k \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$; 分母是全部 K 项之和, 即 $\sum_j \pi_j \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$(公共的归一化常数在分子分母中约去). 相除即得(15.29).

对(15.26)取期望, 并把 $\mathbb{E}[z_{nk}] = \gamma(z_{nk})$ 代入 (期望是线性的,$\boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi}$ 在 E 步骤中固定), 即得(15.30).

□

注

(重构新增) 这条命题是“E 步骤到底算什么”的精确回答: 它算的是指示变量的后验期望, 也就是责任. 式(15.30)与完整数据的(15.26)形式完全一样, 唯一的改动是把不可观测的 $z_{nk} \in \{0, 1\}$ 换成它的条件期望 $\gamma(z_{nk}) \in [0, 1]$——这正是“把硬标签换成软标签”的数学含义, 也是 EM 与 K 均值在公式层面的唯一差别. 命题的成立依赖于后验在 n 上的可分解性 ((15.28)), 而后者又依赖于数据独立同分布这一假设; 若数据点之间存在依赖 (例如序列数据, 见图 15.11), 责任就不能逐点独立计算. 此外,$\gamma(z_{nk})$ 依赖全部 K 个分量的参数 (分母对所有 j 求和), 所以 E 步骤必须对每个数据点遍历所有分量, 代价是 $O(NK)$.

我们现在可以按照以下步骤进行操作. 首先为参数 $\boldsymbol{\mu}^{\text{old}}$、$\boldsymbol{\Sigma}^{\text{old}}$ 和 $\boldsymbol{\pi}^{\text{old}}$ 选择一些初始值, 并使用这些值来估计责任 (E 步骤). 然后保持责任固定, 并通过 $\boldsymbol{\mu}_k$、$\boldsymbol{\Sigma}_k$ 和 π_k 来最大化期望, 见(15.30)(M 步骤). 因此前面提到的闭合解 $\boldsymbol{\mu}_k^{\text{new}}$、$\boldsymbol{\Sigma}_k^{\text{new}}$ 和 π_k^{new} 分别由(15.16)、(15.18)和(15.21)给出. 这正是我们早些时候推导出的想要应用于 Gauss 混合模型的 EM 算法. 当我们在节 15.4讨论 EM 算法的收敛性时, 我们将更深入地了解完整数据 (一个数据集中同时包含了观测到的数据点以及与之相关联的潜变量) 对数似然函数的期望的作用.

在本章中, 我们假设观测值是独立同分布的. 对于形成序列的有序观测值, 可以通过将隐变量连接成一个 Markov 链来扩展混合模型, 以得到一个隐 Markov 模型 (hidden Markov model,HMM), 如图 15.11所示.EM 算法可以扩展到这个更复杂的模型, 在这个模型中,E 步骤涉及一个顺序计算, 其中信息沿着隐变量的链传递 (Bishop,

2006).

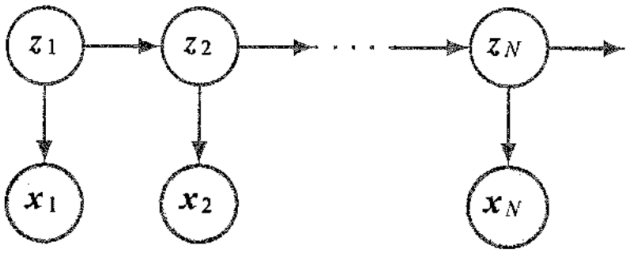


图 15.11: 对应于隐 Markov 模型的序列数据的概率图模型. 离散的隐变量不再是独立的, 而是形成了一个 Markov 链

15.3.2 EM 算法与 K 均值算法的关系

对 K 均值算法与 EM 算法进行比较后, 可以发现它们之间有很大的相似性.K 均值算法把数据点硬 (hard) 分配到聚类中, 每个数据点唯一地与一个聚类相关联, 而 EM 算法基于后验概率进行软 (soft) 分配. 事实上, 我们可以从 EM 算法推导出 K 均值算法.

考虑一个 Gauss 混合模型, 其中混合分量的协方差矩阵由 $\varepsilon \boldsymbol{I}$ 给出, 其中 ε 是所有分量共享的方差参数,$\boldsymbol{I}$ 是单位矩阵, 因此

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) = \frac{1}{(2\pi\varepsilon)^{D/2}} \exp\left\{-\frac{1}{2\varepsilon}\|\boldsymbol{x} - \boldsymbol{\mu}_k\|^2\right\}. \tag{15.31}$$

考虑将要应用于这种形式的包含 K 个分量的 Gauss 混合模型的 EM 算法, 其中我们将 ε 当作固定常数而不是重新估计的参数处理. 根据(15.29), 特定数据点 $\boldsymbol{x}_n$ 的后验概率或责任由下式给出:

$$\gamma(z_{nk}) = \frac{\pi_k \exp\{-\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2/2\varepsilon\}}{\sum_j \pi_j \exp\{-\|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2/2\varepsilon\}}. \tag{15.32}$$

考虑极限 $\varepsilon \to 0$,(15.32)右侧的分母中包含了以 j 索引的多个趋于零的项. 假设使得 $\|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2$ 最小的特定项 (例如 $j = l$ 的项) 将会最慢地趋于零并支配该平方和. 因此, 数据点 $\boldsymbol{x}_n$ 的责任 $\gamma(z_{nk})$ 除了第 l 项外都会趋于零, 第 l 项的责任 $\gamma(z_{nl})$ 将趋于 1. 注意, 这独立于 π_k 的值, 只要没有任何 π_k 为零即可.

命题 15.9 (EM 算法的 K 均值极限)

设 $\varepsilon > 0$, 各向同性协方差由(15.31)给出, 责任由(15.32)给出,$\pi_k > 0$. 设 $l(n)$ 是唯一使 $\|\boldsymbol{x}_n - \boldsymbol{\mu}_j\|^2$ 最小的分量 (即无并列). 则当 $\varepsilon \to 0^+$ 时,

$$\gamma(z_{nk}) \longrightarrow r_{nk} = \begin{cases} 1, & k = l(n), \\ 0, & k \neq l(n), \end{cases}$$

且 $\boldsymbol{\mu}_k$ 的重估方程(15.16)退化为 K 均值的均值更新(15.4); 完整数据对数似然期望(15.30)则趋于

$$\mathbb{E}_{\boldsymbol{Z}}\left[\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma}, \boldsymbol{\pi})\right] \longrightarrow -\frac{1}{2}\sum_{n=1}^{N}\sum_{k=1}^{K} r_{nk}\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2 + \text{常数}. \tag{15.33}$$

证明

记 $d_{nk} = \|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2/2$, 则(15.32)可写成

$$\gamma(z_{nk}) = \frac{\pi_k \mathrm{e}^{-d_{nk}/\varepsilon}}{\sum_j \pi_j \mathrm{e}^{-d_{nj}/\varepsilon}} = \frac{\pi_k}{\sum_j \pi_j \mathrm{e}^{-(d_{nj}-d_{nk})/\varepsilon}}.$$

设 $l = l(n)$ 使 d_{nl} 最小. 由唯一性, 对 $j \neq l$ 有 $d_{nj} - d_{nl} > 0$, 故 $\mathrm{e}^{-(d_{nj}-d_{nl})/\varepsilon} \to 0$. 于是分母 $\to \pi_l$, 而 $\pi_l > 0$. 当 $k = l$ 时 $\gamma \to \pi_l/\pi_l = 1$; 当 $k \neq l$ 时分子 $\to 0$ 而分母 $\to \pi_l > 0$, 故 $\gamma \to 0$. 这正是 r_{nk} 的取值.

把 $\gamma(z_{nk}) \to r_{nk}$ 代入(15.16), 分母 $N_k \to \sum_n r_{nk}$ 为分到第 k 类的点数, 分子 $\to \sum_n r_{nk}\boldsymbol{x}_n$, 故 $\boldsymbol{\mu}_k \to \sum_n r_{nk}\boldsymbol{x}_n / \sum_n r_{nk}$, 即(15.4).

最后, 把(15.31)取对数:

$$\ln \mathcal{N}(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k, \varepsilon \boldsymbol{I}) = -\frac{D}{2}\ln(2\pi\varepsilon) - \frac{1}{2\varepsilon}\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2.$$

代入(15.30):

$$\mathbb{E}_{\boldsymbol{Z}}[\ln p] = \sum_n \sum_k \gamma(z_{nk}) \ln \pi_k - \frac{ND}{2}\ln(2\pi\varepsilon) - \frac{1}{2\varepsilon}\sum_n \sum_k \gamma(z_{nk})\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2,$$

其中用到 $\sum_k \gamma(z_{nk}) = 1$. 前两项都与 $\boldsymbol{\mu}$ 无关, 可合并为 $\boldsymbol{\mu}$ 的一个函数 $C(\varepsilon, \boldsymbol{\pi})$, 于是

$$\mathbb{E}_{\boldsymbol{Z}}[\ln p] = -\frac{1}{2\varepsilon}\sum_n \sum_k \gamma(z_{nk})\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2 + C(\varepsilon, \boldsymbol{\pi}).$$

两边乘以 2ε 并令 $\varepsilon \to 0$:$2\varepsilon C \to 0$(因为 C 只含 $\ln\varepsilon$ 与 $\ln\pi_k$), 而 $\gamma(z_{nk}) \to r_{nk}$, 故

$$2\varepsilon\big(\mathbb{E}_{\boldsymbol{Z}}[\ln p] - C\big) \longrightarrow -\sum_n \sum_k r_{nk}\|\boldsymbol{x}_n - \boldsymbol{\mu}_k\|^2 = -J.$$

这就是(15.33)的精确含义: 其中的 "$\to$" 应读作 "在差一个与 $\boldsymbol{\mu}$ 无关的常数 (该常数可以依赖 ε) 的意义下趋于", 因为 $\mathbb{E}_{\boldsymbol{Z}}[\ln p]$ 本身随 $\varepsilon \to 0$ 发散.

□

注

(重构新增) 这条命题给出了 "K 均值是 GMM 的非概率极限" 的精确含义: 当所有分量共享同一个方差 ε 且 $\varepsilon \to 0$ 时, 责任从软分配退化为(15.2)的硬分配, 均值更新退化为质心公式, 而 Q 的主项正是 $-J/2$(乘上 2ε 之后). 因此在这个极限下 "最大化完整数据对数似然的期望" 等价于 "最小化 K 均值的误差度量 J" ((15.1)). 推导中有两处需要条件: 其一,$\pi_k > 0$ 必须成立, 否则分母可能为零; 其二,$l(n)$ 必须唯一: 若出现并列, 那些并列项的指数都不衰减,$\gamma(z_{nk})$ 的极限会按 π 的比例分摊, 而不再退化为 0/1——此时硬分配本身就失去了定义. 这也说明 K 均值不估计协方差——协方差被固定为 $\varepsilon\boldsymbol{I}$ 且不参与优化, 只估计均值; 混合系数(15.21)虽然仍按 N_k/N 更新, 但它已不再影响 γ 的极限, 因而在算法中不再起积极作用.

最后, 在极限 $\varepsilon \to 0$ 下, 用来给出完整数据对数似然函数期望的(15.30)(见练习 15.12), 就可以变为(15.33). 因此, 我们看到在这个极限下, 完整数据对数似然函数的最大化期望等价于最小化 K 均值算法的误差度量 J,J 由(15.1)给出. 注意,K 均值算法不估计聚类的协方差, 只估计聚类的均值.

15.3.3 混合 Bernoulli 分布

在本章中, 目前主要关注的是由 Gauss 混合分布描述的连续变量模型. 为了在不同的情境中阐述 EM 算法, 现在讨论混合建模的另一个例子——由 Bernoulli 分布描述的离散二元变量混合模型. 这个模型也称潜在类别分析模型 (latent class analysis model)(Lazarsfeld and Henry, 1968; McLachlan and Peel, 2000).

定义 15.7 (多元 Bernoulli 分布)

考虑一组 D 个二元变量 $x_i \in \{0,1\}, i = 1, \cdots, D$, 其中的每一个二元变量都服从参数为 μ_i 的 Bernoulli 分布, 因此

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}) = \prod_{i=1}^{D} \mu_i^{x_i} (1-\mu_i)^{1-x_i}, \tag{15.34}$$

其中 $\boldsymbol{x} = (x_1, \cdots, x_D)^{\mathsf{T}}, \boldsymbol{\mu} = (\mu_1, \cdots, \mu_D)^{\mathsf{T}}$. 可以看出, 在给定 $\boldsymbol{\mu}$ 的情况下, 各个变量 x_i 是独立的. 称(15.34)为参数为 $\boldsymbol{\mu}$ 的**多元 Bernoulli 分布**.

♣

注

(重构新增) 这个定义把 D 个独立的 Bernoulli 变量打包成一个向量值分布. 定义中的独立性假设 (给定 $\boldsymbol{\mu}$ 时各 x_i 独立) 是**分量内部**的假设, 它使联合概率分解为 D 个一维因子的乘积, 从而使对数似然变成 D 项之和——这是混合模型能够逐维解释参数 μ_{ki} 的前提. 参数空间为 $0 \leqslant \mu_i \leqslant 1$; 若某个 μ_i 取到端点 0 或 1, 对应维度的似然退化为示性函数, 取对数后为 $-\infty$(当观测与端点矛盾时). 注意单个多元 Bernoulli 只能描述“各维独立”的二值数据, 无法刻画像素之间的相关性; 下一段引入混合正是为了突破这一限制.

这个分布的均值和协方差可以很容易地得到 (见练习 15.13):

$$\mathbb{E}[\boldsymbol{x}] = \boldsymbol{\mu}, \tag{15.35}$$

$$\operatorname{cov}[\boldsymbol{x}] = \operatorname{diag}\{\mu_i(1-\mu_i)\}. \tag{15.36}$$

命题 15.10 (Bernoulli 分布的均值与协方差)

设 $\boldsymbol{x}$ 的分布由(15.34)给出, 则(15.35)与(15.36)成立.

♠

证明

由(15.34), 各分量独立且 $p(x_i = 1) = \mu_i$、$p(x_i = 0) = 1 - \mu_i$, 故 $\mathbb{E}[x_i] = \mu_i$, 即(15.35). 再由独立性, $i \neq j$ 时 $\operatorname{cov}[x_i, x_j] = 0$; 而 $\operatorname{var}[x_i] = \mathbb{E}[x_i^2] - \mu_i^2 = \mu_i - \mu_i^2 = \mu_i(1-\mu_i)$, 即(15.36).

□

注

(重构新增) 这两条结果与命题 3.1 完全一致, 只是逐分量并排写成向量形式. 注意协方差矩阵是对角的, 这直接反映了“给定 $\boldsymbol{\mu}$ 时各维独立”这一假设: 单个 Bernoulli 乘积模型无法刻画变量之间的相关性. 条件 $0 \leqslant \mu_i \leqslant 1$ 保证 $\operatorname{var}[x_i] = \mu_i(1-\mu_i) \geqslant 0$; 当 $\mu_i = 0$ 或 1 时该维退化, 方差为零. 这一“对角”性质正是下一段要引入混合的原因: 混合之后的协方差(15.40)一般不再是对角阵.

考虑这些分布的有限混合分布, 表示为

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}, \boldsymbol{\pi}) = \sum_{k=1}^{K} \pi_k \, p(\boldsymbol{x} \mid \boldsymbol{\mu}_k), \tag{15.37}$$

其中 $\boldsymbol{\mu} = \{\boldsymbol{\mu}_1, \cdots, \boldsymbol{\mu}_K\}, \boldsymbol{\pi} = \{\pi_1, \cdots, \pi_K\}$, 并且

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}_k) = \prod_{i=1}^{D} \mu_{ki}^{x_i} (1-\mu_{ki})^{1-x_i}. \tag{15.38}$$

混合系数满足(15.7)和(15.8)所示的条件.

定义 15.8 (混合 Bernoulli 分布)

称由(15.37)与(15.38)给出的分布为 K 个分量的**混合 Bernoulli 分布** (mixture of Bernoullis), 其中 π_k 为混合系数, $\boldsymbol{\mu}_k$ 为第 k 个分量的参数.

♣

注

(重构新增) 混合层的作用是让各维之间产生相关性: 每个分量内部各维仍独立, 但不同分量在不同位置给出高概率, 叠加后 $p(\boldsymbol{x}\mid\boldsymbol{\mu},\boldsymbol{\pi})$ 一般不再分解为逐维乘积 (见命题 15.11). 这与 Gauss 混合的构造完全平行, 只是把分量密度从 $\mathcal{N}$ 换成(15.38). 它适用的数据是二值特征向量, 如图像的黑白像素或文本的词袋表示; 由于分量内部独立, 它不能刻画“同一分量内像素之间的局部相关性”, 若要刻画需改用更结构化的分量. 约束(15.7)与(15.8)保证(15.37)是合法密度, 理由与定义 15.2处相同.

这个混合分布的均值和协方差分别如下 (见练习 15.14):

$$\mathbb{E}[\boldsymbol{x}]=\sum_{k=1}^{K}\pi_k\boldsymbol{\mu}_k, \tag{15.39}$$

$$\operatorname{cov}[\boldsymbol{x}]=\sum_{k=1}^{K}\pi_k\left\{\boldsymbol{\Sigma}_k+\boldsymbol{\mu}_k\boldsymbol{\mu}_k^{\mathsf{T}}\right\}-\mathbb{E}[\boldsymbol{x}]\mathbb{E}[\boldsymbol{x}]^{\mathsf{T}}, \tag{15.40}$$

其中 $\boldsymbol{\Sigma}_k=\operatorname{diag}\{\mu_{ki}(1-\mu_{ki})\}$. 由于协方差矩阵 $\operatorname{cov}[\boldsymbol{x}]$ 不再是对角矩阵, 这个混合分布能够捕捉变量之间的相关性, 这与单个 Bernoulli 分布不同.

命题 15.11 (混合 Bernoulli 分布的均值与协方差)

设 $p(\boldsymbol{x}\mid\boldsymbol{\mu},\boldsymbol{\pi})$ 由(15.37)给出, 第 k 个分量的均值与协方差为 $\boldsymbol{\mu}_k$ 与 $\boldsymbol{\Sigma}_k$. 则(15.39)与(15.40)成立.

证明

引入潜变量 $\boldsymbol{z}$, 则 $p(\boldsymbol{x}\mid\boldsymbol{\mu},\boldsymbol{\pi})=\sum_k p(z_k=1)p(\boldsymbol{x}\mid z_k=1)$. 由全期望公式,

$$\mathbb{E}[\boldsymbol{x}]=\sum_{k=1}^{K}\pi_k\,\mathbb{E}[\boldsymbol{x}\mid k]=\sum_{k=1}^{K}\pi_k\boldsymbol{\mu}_k,$$

即(15.39).

对协方差用全协方差公式:$\operatorname{cov}[\boldsymbol{x}]=\mathbb{E}\big[\operatorname{cov}[\boldsymbol{x}\mid\boldsymbol{z}]\big]+\operatorname{cov}\big[\mathbb{E}[\boldsymbol{x}\mid\boldsymbol{z}]\big]$. 第一项为 $\sum_k\pi_k\boldsymbol{\Sigma}_k$; 第二项为 $\sum_k\pi_k\boldsymbol{\mu}_k\boldsymbol{\mu}_k^{\mathsf{T}}-\mathbb{E}[\boldsymbol{x}]\mathbb{E}[\boldsymbol{x}]^{\mathsf{T}}$. 相加即得(15.40).

□

注

(重构新增) (15.40)的两项各有含义:$\sum_k\pi_k\boldsymbol{\Sigma}_k$ 是“分量内部”的方差 (各分量自身的对角阵), 而 $\sum_k\pi_k\boldsymbol{\mu}_k\boldsymbol{\mu}_k^{\mathsf{T}}-\mathbb{E}[\boldsymbol{x}]\mathbb{E}[\boldsymbol{x}]^{\mathsf{T}}$ 是“分量之间”的方差 (由各分量均值散布造成). 即使每个 $\boldsymbol{\Sigma}_k$ 都是对角的, 第二项一般含有非对角元 $\sum_k\pi_k\mu_{ki}\mu_{kj}-\mathbb{E}[x_i]\mathbb{E}[x_j]$, 因此总的协方差不再是——这就是混合模型能够刻画相关性的原因. 这一分解与 Gauss 混合的对应公式 (命题 3.9一类的恒等式) 同源, 只是把连续情形换成了离散情形. 注意 $\mu_{ki}\in[0,1]$ 保证了每个 $\boldsymbol{\Sigma}_k$ 半正定, 从而 $\operatorname{cov}[\boldsymbol{x}]$ 半正定.

如果我们得到一个数据集 $\boldsymbol{X}=\{\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N\}$, 那么该模型的对数似然函数可以表示为

$$\ln p(\boldsymbol{X}\mid\boldsymbol{\mu},\boldsymbol{\pi})=\sum_{n=1}^{N}\ln\left\{\sum_{k=1}^{K}\pi_k p(\boldsymbol{x}_n\mid\boldsymbol{\mu}_k)\right\}. \tag{15.41}$$

我们再次看到对数内部出现了求和, 因此最大似然解不再有闭合形式.

下面让我们为混合 Bernoulli 分布的似然函数最大化推导 EM 算法. 为此, 首先引入一个与每个实例 $\boldsymbol{x}_n$ 关联的显式离散隐变量 $\boldsymbol{z}_n$. 与 Gauss 混合模型一样,$\boldsymbol{z}_n$ 采用 1-of-K 编码方案, 所以 $\boldsymbol{z}_n=(z_{n1},\cdots,z_{nK})^{\mathsf{T}}$ 是一个二元的 K 维向量, 有一个分量等于 1, 所有其他分量等于 0. 我们可以将给定隐变量的 $\boldsymbol{x}$ 的条件分布写为

$$p(\boldsymbol{x}\mid\boldsymbol{z},\boldsymbol{\mu})=\prod_{k=1}^{K}p(\boldsymbol{x}\mid\boldsymbol{\mu}_k)^{z_k}, \tag{15.42}$$

而隐变量的先验分布与 Gauss 混合模型中的相同, 因此

$$p(\boldsymbol{z} \mid \boldsymbol{\pi}) = \prod_{k=1}^{K} \pi_k^{z_k}. \tag{15.43}$$

将 $p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{\mu})$ 和 $p(\boldsymbol{z} \mid \boldsymbol{\pi})$ 相乘, 然后对 $\boldsymbol{z}$ 进行边缘化, 即可回到(15.37)(见练习 15.16).

为了推导 EM 算法, 我们首先写下完整数据对数似然函数:

$$\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \boldsymbol{\mu}, \boldsymbol{\pi}) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \left\{ \ln \pi_k + \sum_{i=1}^{D} \left[x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki}) \right] \right\}, \tag{15.44}$$

其中 $\boldsymbol{X} = \{\boldsymbol{x}_n\}$,$\boldsymbol{Z} = \{\boldsymbol{z}_n\}$. 接下来取完整数据对数似然函数关于隐变量后验分布的期望, 得到

$$\mathbb{E}_{\boldsymbol{Z}} \left[\ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \boldsymbol{\mu}, \boldsymbol{\pi}) \right] = \sum_{n=1}^{N} \sum_{k=1}^{K} \gamma(z_{nk}) \left\{ \ln \pi_k + \sum_{i=1}^{D} \left[x_{ni} \ln \mu_{ki} + (1 - x_{ni}) \ln(1 - \mu_{ki}) \right] \right\}, \tag{15.45}$$

其中 $\gamma(z_{nk}) = \mathbb{E}[z_{nk}]$ 是给定数据点 $\boldsymbol{x}_n$ 时分量 k 的后验概率或责任. 在 E 步骤中, 使用 Bayes 定理计算这些责任, 形式如下:

$$\begin{aligned} \gamma(z_{nk}) &= \frac{\sum_{z_n} z_{nk} \prod_{k'} \left[\pi_{k'} p(\boldsymbol{x}_n \mid \boldsymbol{\mu}_{k'}) \right]^{z_{nk'}}}{\sum_{z_n} \prod_{j} \left[\pi_j p(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j) \right]^{z_{nj}}} \\ &= \frac{\pi_k p(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k)}{\sum_{j=1}^{K} \pi_j p(\boldsymbol{x}_n \mid \boldsymbol{\mu}_j)}. \end{aligned} \tag{15.46}$$

考虑(15.45)中关于 n 的求和, 我们可以看到责任仅存在于两个项, 它们可以分别写成

$$N_k = \sum_{n=1}^{N} \gamma(z_{nk}), \tag{15.47}$$

$$\bar{\boldsymbol{x}}_k = \frac{1}{N_k} \sum_{n=1}^{N} \gamma(z_{nk}) \boldsymbol{x}_n. \tag{15.48}$$

在 M 步骤中, 针对参数 $\boldsymbol{\mu}_k$ 和 π_k 最大化完整数据对数似然函数的期望. 如果我们把(15.45)关于 μ_{ki} 求导并将导数设为零 (见练习 15.17), 然后重新排列各项, 便可得到

$$\mu_{ki} = \frac{\sum_n \gamma(z_{nk}) x_{ni}}{N_k}, \tag{15.49}$$

其中 $N_k = \sum_n \gamma(z_{nk})$ 是与分量 k 相关联的数据点的有效数量. 分量 k 的均值设置为数据的加权均值, 权重系数由分量 k 对每个数据点所要承担的责任给出. 在关于 π_k 进行最大化的过程中, 我们需要引入一个 Lagrange 乘子来确保约束条件 $\sum_k \pi_k = 1$ 得到满足. 遵循与 Gauss 混合类似的步骤 (见练习 15.18), 我们得到

$$\pi_k = \frac{N_k}{N}. \tag{15.50}$$

这代表了一个直观合理的结果, 即分量 k 的混合系数由数据集中被该分量解释的数据点的有效比例给出.

命题 15.12 (混合 Bernoulli 分布的 M 步骤)

设 $N_k = \sum_n \gamma(z_{nk}) > 0$. 则在约束 $\sum_k \pi_k = 1$ 下关于 $(\boldsymbol{\mu}, \boldsymbol{\pi})$ 最大化(15.45)的解由(15.49)与(15.50)给出, 其中 $\bar{\boldsymbol{x}}_k$ 由(15.48)定义, 且 $\boldsymbol{\mu}_k = \bar{\boldsymbol{x}}_k$ 在分量意义下成立.

证明

关于 μ_{ki} 求导.(15.45)中与 μ_{ki} 有关的项为

$$\sum_{n=1}^{N}\gamma(z_{nk})\left[x_{ni}\ln\mu_{ki}+(1-x_{ni})\ln(1-\mu_{ki})\right].$$

对 μ_{ki} 求导并置零:

$$\sum_{n=1}^{N}\gamma(z_{nk})\left[\frac{x_{ni}}{\mu_{ki}}-\frac{1-x_{ni}}{1-\mu_{ki}}\right]=0.$$

乘以 $\mu_{ki}(1-\mu_{ki})$ 得

$$(1-\mu_{ki})\sum_{n}\gamma(z_{nk})x_{ni}-\mu_{ki}\sum_{n}\gamma(z_{nk})(1-x_{ni})=0,$$

整理即 $\mu_{ki}\sum_{n}\gamma(z_{nk})=\sum_{n}\gamma(z_{nk})x_{ni}$, 故 $\mu_{ki}=\sum_{n}\gamma(z_{nk})x_{ni}/N_k$, 即(15.49); 写成向量形式就是 $\boldsymbol{\mu}_k=\bar{\boldsymbol{x}}_k$.

关于 π_k, 与命题 15.7的证明完全相同: 含 π_k 的项只有 $\sum_{n}\gamma(z_{nk})\ln\pi_k$(把 z_{nk} 换成其期望), 用 Lagrange 乘子得 $\pi_k=N_k/N$, 即(15.50).

□

注

(重构新增) 这条命题与 Gauss 混合的命题 15.6结构完全同构: 均值更新是“责任加权的平均”, 混合系数更新是“责任的平均”. 差别在于 Bernoulli 分量的均值和方差不是独立的——一旦 μ_{ki} 确定, 方差就是 $\mu_{ki}(1-\mu_{ki})$, 所以 M 步骤只需求 $\boldsymbol{\mu}$, 不必求协方差. 这也是混合 Bernoulli 比混合 Gauss 少一组参数的原因. 条件 $N_k>0$ 不可省: 若某个分量在 E 步骤后没有得到任何质量, 则 μ_{ki} 的方程两边同时为零, 分母 $N_k=0$ 使(15.49)无定义; 数值上应删除该分量或重新初始化. 另外,$\mu_{ki}\in(0,1)$ 才使导数有意义 (端点处 $\ln\mu_{ki}$ 或 $\ln(1-\mu_{ki})$ 发散), 但若所有 x_{ni} 在该分量上的加权全为 1, 最优解会趋向端点 1, 此时应按约束 $0\leqslant\mu_{ki}\leqslant 1$ 取边界值.

注意, 与 Gauss 混合模型相比, 这里不存在使似然函数趋于无穷大的奇异点. 这可以通过似然函数上界有限(见练习 15.19) 看出来, 因为 $0\leqslant p(\boldsymbol{x}_n\mid\boldsymbol{\mu}_k)\leqslant 1$. 虽然存在使似然函数为零的解, 但只要 EM 算法不是初始化到一个病态起点, 这些解就不会被 EM 算法找到, 因为 EM 算法总是增大似然函数的值, 直至找到一个局部最大值 (参见节 15.4).

图 15.12用混合 Bernoulli 模型建模手写数字说明了这一点. 这里的数字图像已经转换成二元向量, 方法是将所有值超过 0.5 的元素设为 1, 其他剩余的元素设为 0. 拟合一个包含 $N=600$ 个这样的数字图像的数据集, 其中包括数字 2、3 和 4, 这里采用 $K=3$ 个 Bernoulli 分布的混合模型, 通过 EM 算法完成 10 次迭代. 将混合系数初始化为 $\pi_k=1/K$, 并将参数 μ_{kj} 设置为来自范围 $(0.25,0.75)$ 的随机值, 然后归一化以满足约束 $\sum_{j}\mu_{kj}=1$. 可以看到, 由三个 Bernoulli 分布构成的混合模型能够发现数据集中对应不同数字的三个聚类. 将对混合 Bernoulli 模型的分析扩展到具有 $M>2$ 个状态的多项式二元变量 (见练习 15.20), 方法是使用离散分布 (见小节 3.1.3).

15.4 证据下界

本节从更一般的角度介绍 EM 算法. 推导对数似然函数的一个下界, 这个下界称为证据下界 (Evidence Lower BOund,ELBO), 有时也称变分下界. 这里的“证据”是指 (对数) 似然函数, 在 Bayes 设置中有时也称“模型证据”, 因为它允许在不使用验证集的情况下比较不同的模型 (Bishop, 2006). 为了说明这个下界, 我们使用它从第三个角度重新推导 EM 算法.ELBO 将在后续章节讨论的几个深度生成式模型中发挥重要作用. 它还提供了一个关于变分框架的样板, 在这个框架中, 我们引入了一个关于隐变量的分布 $q(\boldsymbol{Z})$, 然后使用变分法针对这个分布进行优化 (见附录 B).

考虑一个概率模型, 在该模型中我们将所有观测变量统称为 $\boldsymbol{X}$, 并将所有隐变量统称为 $\boldsymbol{Z}$. 联合分布

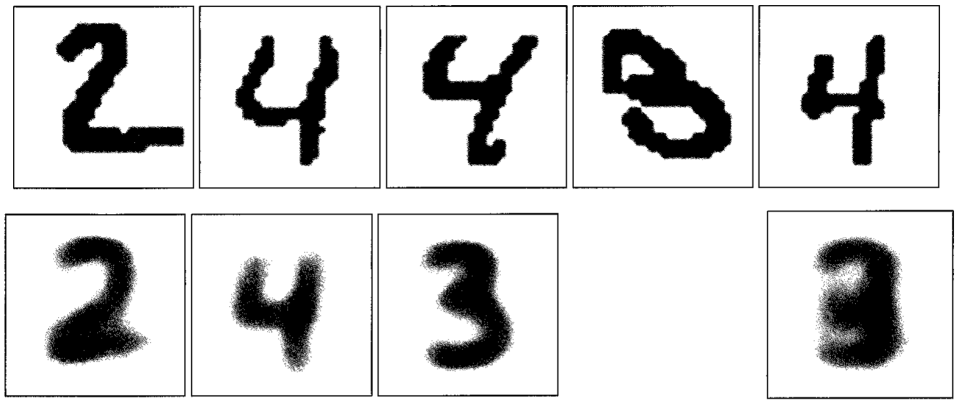


图 15.12: 用混合 Bernoulli 模型建模手写数字, 顶部行展示了数字的数据集中的样本, 在这些样本中使用 0.5 的阈值将像素值转换为二元向量. 底部行的前三个图像显示了混合模型中每个分量的参数 μ_{ki}. 作为比较, 我们也使用单一的多元 Bernoulli 分布拟合同一个数据集, 同样采用最大似然法. 这相当于简单地对每个像素的计数取平均值, 并通过底部行最右边的图像展现出来

$p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$ 由一组参数 θ 控制. 我们的目标是最大化似然函数:

$$p(\boldsymbol{X} \mid \theta) = \sum_{\boldsymbol{Z}} p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta). \tag{15.51}$$

在这里我们假设 $\boldsymbol{Z}$ 是离散的, 当 $\boldsymbol{Z}$ 包含连续变量, 或者包含离散变量与连续变量的组合时, 要讨论的内容是相同的, 将求和替换为适当的积分即可.

假设直接优化 $p(\boldsymbol{X} \mid \theta)$ 是困难的, 但优化完整数据似然函数 $p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$ 则简单得多. 接下来我们引入一个定义在隐变量上的分布 $q(\boldsymbol{Z})$, 我们观察到, 对所选择的任何分布 $q(\boldsymbol{Z})$, 以下分解成立:

$$\ln p(\boldsymbol{X} \mid \theta) = \mathcal{L}(q, \theta) + \mathrm{KL}(q(\boldsymbol{Z}) \,\|\, p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta)), \tag{15.52}$$

其中

$$\mathcal{L}(q, \theta) = \sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \ln \left\{ \frac{p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)}{q(\boldsymbol{Z})} \right\}, \tag{15.53}$$

而

$$\mathrm{KL}(q(\boldsymbol{Z}) \,\|\, p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta)) = -\sum_{\boldsymbol{Z}} q(\boldsymbol{Z}) \ln \left\{ \frac{p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta)}{q(\boldsymbol{Z})} \right\}. \tag{15.54}$$

定义 15.9 (证据下界与 KL 散度) 设 $q(\boldsymbol{Z})$ 是隐变量上的任一分布,$q(\boldsymbol{Z}) > 0$ 处取值有限. 称(15.53)中的 $\mathcal{L}(q, \theta)$ 为**证据下界** (ELBO), 称(15.54)中的 $\mathrm{KL}(q \,\|\, p)$ 为 $q(\boldsymbol{Z})$ 与后验 $p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta)$ 之间的 **Kullback–Leibler 散度**. ♣

注

(重构新增) 定义(15.53)与(15.54)的形式看起来不对称, 这一点值得专门说明:$\mathcal{L}$ 里出现的是**联合**分布 $p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$, 而 KL 里出现的是**条件**分布 $p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta)$; 两者相差的正是 $\ln p(\boldsymbol{X} \mid \theta)$(见命题 15.13). 名称“证据下界”来自两件事:“证据”指 $\ln p(\boldsymbol{X} \mid \theta)$(模型证据),“下界”指对任意 q 都有 $\mathcal{L}(q, \theta) \leqslant \ln p(\boldsymbol{X} \mid \theta)$(见命题 15.14).$\mathcal{L}$ 既是分布 $q(\boldsymbol{Z})$ 的**泛函** (输入是一个函数而不是有限维向量), 又是参数 θ 的普通函数; 这种“一半变分、一半参数”的双重身份是变分推断的典型结构, 也是后面交替优化 q 与 θ 的原因. 定义只要求 q 是归一化分布, 并不要求它属于某个参数族——这一点使 ELBO 比“用一族参数化分布逼近后验”的变分近似更一般.

注意 $\mathcal{L}(q, \theta)$ 既是分布 $q(\boldsymbol{Z})$ 的泛函, 也是参数 θ 的函数. 仔细研究(15.53)和(15.54)的形式, 并特别注意它们在符号上的差异,$\mathcal{L}(q, \theta)$ 包含 $\boldsymbol{X}$ 和 $\boldsymbol{Z}$ 的联合分布, 而 $\mathrm{KL}(q \,\|\, p)$ 包含给定 $\boldsymbol{X}$ 的 $\boldsymbol{Z}$ 的条件分布 (见练习 15.21).

命题 15.13 (ELBO 分解)

对任意归一化分布 $q(\boldsymbol{Z})$ 与任意参数 θ, 分解(15.52)成立, 其中 $\mathcal{L}$ 与 KL 分别由(15.53)与(15.54)定义.

证明

利用概率的乘积法则,

$$\ln p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)=\ln p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)+\ln p(\boldsymbol{X}\mid\theta), \tag{15.55}$$

然后将其代入 $\mathcal{L}(q,\theta)$ 的表达式(15.53):

$$\begin{aligned}\mathcal{L}(q,\theta)&=\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})\big[\ln p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)+\ln p(\boldsymbol{X}\mid\theta)-\ln q(\boldsymbol{Z})\big]\\&=\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})\ln p(\boldsymbol{X}\mid\theta)-\left[-\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})\ln\left\{\frac{p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)}{q(\boldsymbol{Z})}\right\}\right].\end{aligned}$$

右端第一项中 $\ln p(\boldsymbol{X}\mid\theta)$ 与 $\boldsymbol{Z}$ 无关, 可提出求和号, 而 $\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})=1$($q$ 是归一化分布), 故该项等于 $\ln p(\boldsymbol{X}\mid\theta)$; 第二项恰为 $-\mathrm{KL}(q\,\|\,p)$, 与 KL 的定义(15.54)中的抵消项一起给出 KL 本身. 于是

$$\mathcal{L}(q,\theta)=\ln p(\boldsymbol{X}\mid\theta)-\mathrm{KL}(q\,\|\,p),$$

移项即(15.52).

□

注

(重构新增) 这条命题也解释了“为什么必须用联合分布”: 拆开 $\ln p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)$ 后会多出一项 $\ln p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)$, 而 $-\sum_{\boldsymbol{Z}}q\ln\{p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)/q\}$ 恰好就是 KL 的定义, 于是联合分布中“多出来的那部分”被 KL 精确吸收, 剩下的正好是 $\ln p(\boldsymbol{X}\mid\theta)$. 这个证明只用到两条性质: 乘积法则(15.55)与 q 的归一化. 前者是恒等式, 后者是定义要求; 若 q 未归一化, 分解中会多出 $-\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})$ 这一项, 下界结论就不再成立. 注意分解对任意 q 成立 (不需要 q 接近后验), 这正是它可以作为“变分优化目标”的前提.

命题 15.14 (ELBO 是证据的下界)

对任意 $q(\boldsymbol{Z})$,$\mathrm{KL}(q\,\|\,p)\geqslant 0$, 且等号成立当且仅当 $q(\boldsymbol{Z})=p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)$ 处处成立. 因此

$$\mathcal{L}(q,\theta)\leqslant\ln p(\boldsymbol{X}\mid\theta),$$

即 $\mathcal{L}(q,\theta)$ 是 $\ln p(\boldsymbol{X}\mid\theta)$ 的一个下界.

证明

由 $\ln t\leqslant t-1(t>0$, 等号仅当 $t=1)$, 取 $t=p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)/q(\boldsymbol{Z})$ 并对 $\boldsymbol{Z}$ 用 q 加权求和:

$$\begin{aligned}-\mathrm{KL}(q\,\|\,p)&=\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})\ln\frac{p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)}{q(\boldsymbol{Z})}\leqslant\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})\left[\frac{p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)}{q(\boldsymbol{Z})}-1\right]\\&=\sum_{\boldsymbol{Z}}p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)-\sum_{\boldsymbol{Z}}q(\boldsymbol{Z})=1-1=0,\end{aligned}$$

故 $\mathrm{KL}(q\,\|\,p)\geqslant 0$. 等号成立当且仅当对每个 $q(\boldsymbol{Z})>0$ 的 $\boldsymbol{Z}$ 都有 $p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)/q(\boldsymbol{Z})=1$, 即 $q=p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)$. 再由命题 15.13,$\mathcal{L}=\ln p(\boldsymbol{X}\mid\theta)-\mathrm{KL}\leqslant\ln p(\boldsymbol{X}\mid\theta)$.

□

注

(重构新增) 下界成立的关键是 ln 的凹性 (等价地 $\ln t\leqslant t-1$ 或 Jensen 不等式), 而不是任何关于模型的假设, 因此它对任意 q 与任意 θ 都成立. 等号条件给出了 E 步骤的动机: 固定 θ 时, 使下界最大的 q 就是后验——这正好解释了小节 15.4.1 中“E 步骤选 q 为后验”的做法并非武断, 而是让下界在 θ^{old} 处与对数似然**相切** (图 15.13). 这里也可以从信息论角度重述:$\mathrm{KL}(q\,\|\,p)$ 度量的是用 q 近似后验时“多付出的编码长度”, 而 $\mathcal{L}$ 是“边际似然减去这部分多余的编码长度”. 注意 KL 不是对称的, 也不是距离; 本命题只用到 $\mathrm{KL}\geqslant 0$ 这一条性质.

从(15.54)中我们看到,KL($q \| p$) 是 $q(\boldsymbol{Z})$ 与后验分布 $p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)$ 之间的 Kullback–Leibler 散度.Kullback–Leibler 散度满足 KL($q \| p$) $\geqslant 0$, 当且仅当 $q(\boldsymbol{Z}) = p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta)$ 时等号成立. 因此, 从(15.52)可以得出 $\mathcal{L}(q, \theta) \leqslant \ln p(\boldsymbol{X} \mid \theta)$, 换句话说,$\mathcal{L}(q, \theta)$ 是 $\ln p(\boldsymbol{X} \mid \theta)$ 的一个下界.图 15.13对(15.52)所示的分解做了说明.

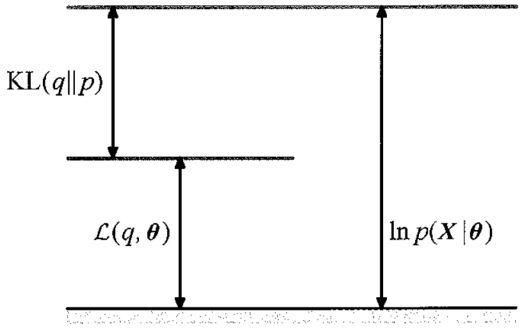


图 15.13: (15.52)所示的分解, 对于任何分布 $q(\boldsymbol{Z})$ 都成立.Kullback–Leibler 散度满足 KL($q \| p$) $\geqslant 0$,$\mathcal{L}(q, \theta)$ 是对数似然函数 $\ln p(\boldsymbol{X} \mid \theta)$ 的一个下界

15.4.1 EM 算法回顾

我们可以使用(15.52)来推导 EM 算法, 并证明它确实能够最大化对数似然函数. 假设当前的参数向量值为 θ^{old}. 在 E 步骤中, 固定 θ^{old} 并通过优化 $q(\boldsymbol{Z})$ 来最大化下界 $\mathcal{L}(q, \theta)$. 这个最大化问题的解可以很容易地看出来, 因为 $\ln p(\boldsymbol{X} \mid \theta^{\text{old}})$ 的值不依赖于 $q(\boldsymbol{Z})$, 所以当 Kullback–Leibler 散度消失时, 即 $q(\boldsymbol{Z})$ 等于后验分布 $p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}})$ 时,$\mathcal{L}(q, \theta^{\text{old}})$ 的值最大. 在这种情况下, 下界将等于对数似然函数的值, 如图 15.14所示.

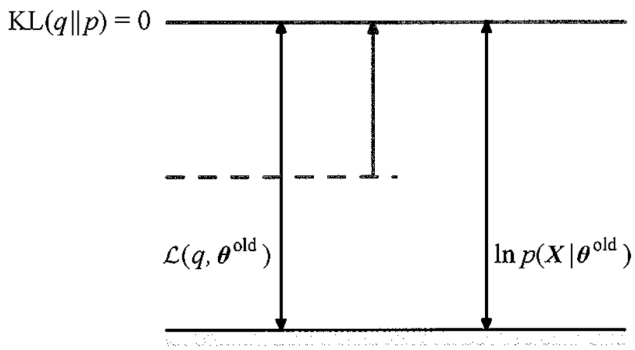


图 15.14: EM 算法的 E 步骤. 将分布 $q(\boldsymbol{Z})$ 设定为等于当前参数值 θ^{old} 的后验分布, 这导致下界上移至与对数似然函数相同的值, 同时 KL 散度消失

在随后的 M 步骤中, 固定分布 $q(\boldsymbol{Z})$ 并通过优化 θ 来最大化下界 $\mathcal{L}(q, \theta)$, 得到新的参数值 θ^{new}. 这将导致下界增大 (除非它已经达到最大值), 这必然也会导致相应的对数似然函数增大. 由于我们确定分布 $q(\boldsymbol{Z})$ 使用的是旧值而不是新值, 并且在 M 步骤中保持固定, 它不等于新的后验分布 $p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{new}})$, 因此会有一个非零的 Kullback–Leibler 散度. 对数似然函数的增量将大于下界的增量, 如图 15.15所示.

如果将 $q(\boldsymbol{Z}) = p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}})$ 代入(15.53), 则可以看到, 在 E 步骤之后, 下界采取以下形式:

$$\begin{aligned}\mathcal{L}(q, \theta) &= \sum_{\boldsymbol{Z}} p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}}) \ln p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta) \\ &\quad - \sum_{\boldsymbol{Z}} p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}}) \ln p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}}) = Q(\theta, \theta^{\text{old}}) + \text{const.}\end{aligned} \tag{15.56}$$

其中的常数项 const 只是分布 $q(\boldsymbol{Z})$ 的负熵, 因此与 θ 无关. 在这里我们将 $Q(\theta, \theta^{\text{old}})$ 识别为(15.23)定义的完整数据对数似然函数的期望, 因此它是 M 步骤中被最大化的量, 正如我们早先在 Gauss 混合模型中看到的那样. 注意, 我们正在优化的变量 θ 只出现在对数内部. 如果联合分布 $p(\boldsymbol{Z}, \boldsymbol{X} \mid \theta)$ 是指数族分布或者指数族分布的乘积, 对

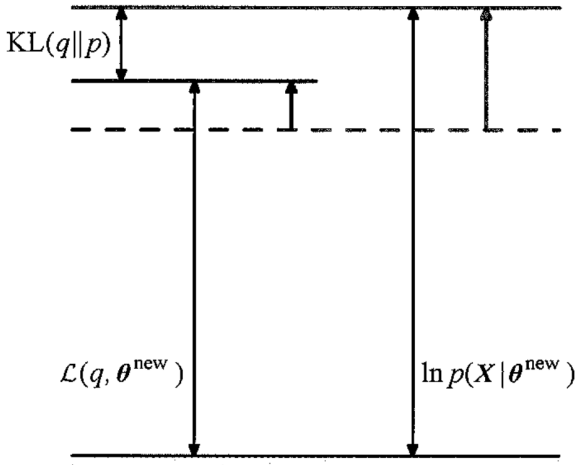


图 15.15: EM 算法的 M 步骤. 在 M 步骤中, 分布 $q(\boldsymbol{Z})$ 保持固定, 然后通过优化参数向量 θ 来最大化下界 $\mathcal{L}(q, \theta)$, 从而得到更新后的参数值 θ^{new}.Kullback–Leibler 散度是非负的, 这导致对数似然函数 $\ln p(\boldsymbol{X}|\theta)$ 的增量至少与下界的一样多

数运算会简化对指数的计算, 并使得 M 步骤通常比直接最大化相应的不完整数据对数似然函数 $p(\boldsymbol{X}|\theta)$ 简单得多.

EM 算法也可以从参数空间的角度来观察, 如图 15.16所示. 这里红色曲线描述了我们希望最大化的 (不完整数据的) 对数似然函数. 我们从某个初始参数值 θ^{old} 开始. 在第一次 E 步骤中, 我们评估隐变量的后验分布, 产生了一个下界 $\mathcal{L}(q, \theta)$, 其值在 θ^{old} 处等于对数似然函数的值, 如蓝色曲线所示. 注意, 这个下界曲线在 θ^{old} 处与对数似然函数曲线呈切线接触, 因此这两条曲线具有相同的梯度. 这个下界是一个凸函数, 具有唯一的最大值 (用于来自指数族分布的混合分量). 在 M 步骤中, 下界被最大化, 得到新的参数值 θ^{new}, 它能给出比 θ^{old} 更大的对数似然值. 随后的 E 步骤构建了一个在 θ^{new} 处相切的下界, 如绿色曲线所示.

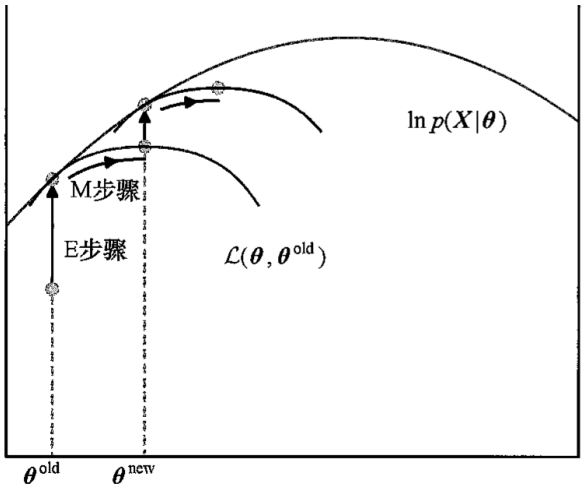


图 15.16: EM 算法涉及交替计算当前参数值的对数似然下界, 然后最大化这个下界以得到新的参数值

定理 15.2 (EM 算法的单调性)

设 $q(\boldsymbol{Z}) = p(\boldsymbol{Z}|\boldsymbol{X}, \theta^{\text{old}})$,$\theta^{\text{new}}$ 由 M 步骤(15.24)给出. 则

$$\ln p(\boldsymbol{X}|\theta^{\text{new}}) \geqslant \ln p(\boldsymbol{X}|\theta^{\text{old}}),$$

即一个完整的 EM 循环不会减小不完整数据的对数似然. 更准确地说, 等号成立当且仅当 θ^{new} 与 θ^{old} 给出同一个后验分布 (特别地, 当 θ^{old} 已是驻点、$\theta^{\text{new}} = \theta^{\text{old}}$ 时参数保持不变).

证明

记 $q^\star = p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}})$. 由命题 15.13与命题 15.14, 对任意 θ,

$$\ln p(\boldsymbol{X} \mid \theta) = \mathcal{L}(q, \theta) + \mathrm{KL}(q \,\|\, p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta)) \geqslant \mathcal{L}(q, \theta).$$

在 $\theta = \theta^{\text{old}}$ 处取 $q = q^\star$, 散度为零, 故

$$\mathcal{L}(q^\star, \theta^{\text{old}}) = \ln p(\boldsymbol{X} \mid \theta^{\text{old}}).$$

由(15.56),$\mathcal{L}(q^\star, \theta) = Q(\theta, \theta^{\text{old}}) + \text{const}$, 其中常数与 θ 无关; 而 θ^{new} 最大化 $Q(\cdot, \theta^{\text{old}})$, 故同时最大化 $\mathcal{L}(q^\star, \cdot)$, 于是

$$\mathcal{L}(q^\star, \theta^{\text{new}}) \geqslant \mathcal{L}(q^\star, \theta^{\text{old}}) = \ln p(\boldsymbol{X} \mid \theta^{\text{old}}).$$

最后再用一次下界不等式 (这次在 θ^{new} 处):

$$\ln p(\boldsymbol{X} \mid \theta^{\text{new}}) \geqslant \mathcal{L}(q^\star, \theta^{\text{new}}) \geqslant \ln p(\boldsymbol{X} \mid \theta^{\text{old}}),$$

即所需结论. 若 $\theta^{\text{new}} = \theta^{\text{old}}$, 则 $q^\star = p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{new}})$, 第二次不等式的散度为零, 两式都取等号, 对数似然不变.

□

注

(重构新增) 这条定理是 EM 算法“一定不会变差”的严格依据, 它的证明只用了两件事:ELBO 分解 (命题 15.13) 与 KL $\geqslant 0$(命题 15.14). 注意不等式的方向:M 步骤增大的是下界 $\mathcal{L}$, 而对数似然的增大至少同样多, 因为 KL 从 0 变成一个非负数, 把下界的增量“放大”了. 由此可以读出 EM 的两个极限行为: 若 θ^{old} 已是 $\ln p(\boldsymbol{X} \mid \cdot)$ 的驻点, 则 M 步骤无法再增大 Q,$\theta^{\text{new}} = \theta^{\text{old}}$, 算法停在不动点; 由于 $\ln p(\boldsymbol{X} \mid \theta)$ 有上界 (在离散情形下不超过 0, 因为它是概率的对数), 单调不降的序列必收敛, 但收敛到的可能只是局部极大点. 定理的结论是“不减小”, 不是“严格增大”, 也不是“收敛到全局最优”. 此外, 证明假定了 Q 的最大点存在; 若 Q 无上界 (例如命题 15.4的奇异方向), 则 M 步骤本身无定义, 这正是需要正则化的原因.

我们已经看到,EM 算法的 E 步骤和 M 步骤都会使良定义的对数似然函数的下界值增大, 并且完整的 EM 循环将以这样一种方式改变模型参数; 使对数似然值增大 (除非它已经达到最大值, 在这种情况下, 参数保持不变).

15.4.2 独立同分布数据

对于一个独立同分布的数据集,$\boldsymbol{X}$ 将包含 N 个数据点 $\{\boldsymbol{x}_n\}$, 而 $\boldsymbol{Z}$ 将包含相应的 N 个隐变量 $\{\boldsymbol{z}_n\}$, 其中 $n = 1, \cdots, N$. 根据独立性假设, 我们有 $p(\boldsymbol{X}, \boldsymbol{Z}) = \prod_n p(\boldsymbol{x}_n, \boldsymbol{z}_n)$, 通过对 $\{\boldsymbol{z}_n\}$ 进行边缘化, 我们得到 $p(\boldsymbol{X}) = \prod_n p(\boldsymbol{x}_n)$. 使用概率的求和法则和乘积法则, 则在 E 步骤中计算的后验概率具有以下形式:

$$p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta) = \frac{p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)}{\sum_{\boldsymbol{Z}} p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta)} = \frac{\prod_{n=1}^N p(\boldsymbol{x}_n, \boldsymbol{z}_n \mid \theta)}{\sum_{\boldsymbol{Z}} \prod_{n=1}^N p(\boldsymbol{x}_n, \boldsymbol{z}_n \mid \theta)} = \prod_{n=1}^N p(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \theta). \tag{15.57}$$

命题 15.15 (后验在数据点上的因子分解)

设数据独立同分布, 即 $p(\boldsymbol{X}, \boldsymbol{Z} \mid \theta) = \prod_{n=1}^N p(\boldsymbol{x}_n, \boldsymbol{z}_n \mid \theta)$. 则(15.57)成立, 即后验分布可以针对不同的数据点进行因子分解.

♠

证明

把乘积代入 Bayes 公式:

$$p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)=\frac{\prod_n p(\boldsymbol{x}_n,\boldsymbol{z}_n\mid\theta)}{\sum_{\boldsymbol{Z}}\prod_n p(\boldsymbol{x}_n,\boldsymbol{z}_n\mid\theta)}.$$

分母中求和是对 $\boldsymbol{Z}=(\boldsymbol{z}_1,\cdots,\boldsymbol{z}_N)$ 的所有取值求和; 由于被求和的乘积对 n 可分离, 求和可以逐 n 拆开:

$$\sum_{\boldsymbol{Z}}\prod_{n=1}^{N}p(\boldsymbol{x}_n,\boldsymbol{z}_n\mid\theta)=\prod_{n=1}^{N}\left(\sum_{\boldsymbol{z}_n}p(\boldsymbol{x}_n,\boldsymbol{z}_n\mid\theta)\right)=\prod_{n=1}^{N}p(\boldsymbol{x}_n\mid\theta).$$

于是

$$p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)=\prod_{n=1}^{N}\frac{p(\boldsymbol{x}_n,\boldsymbol{z}_n\mid\theta)}{p(\boldsymbol{x}_n\mid\theta)}=\prod_{n=1}^{N}p(\boldsymbol{z}_n\mid\boldsymbol{x}_n,\theta),$$

即(15.57).

□

注

(重构新增) 这条命题是从“独立同分布”到“E 步骤可以逐点并行计算”的桥梁: 后验一旦分解为 $\prod_n$, 则 $\boldsymbol{Q}$ 中的期望也分解为 N 个独立的小期望, 每个只涉及 $\boldsymbol{z}_n$, 于是责任 $\gamma(z_{nk})$ 只需用 $\boldsymbol{x}_n$ 与全部分量参数计算, 不需要其他数据点的信息. 证明中分母的拆分是关键一步, 它只在独立性假设下成立; 若数据点之间有依赖 (如图 15.11的 Markov 链结构), 后验不能这样分解,E 步骤就必须做前向—后向之类的前序计算. 对于 Gauss 混合模型, 这条结论简单地表明每个混合分量对特定数据点 $\boldsymbol{x}_n$ 所要承担的责任仅依赖于 $\boldsymbol{x}_n$ 的值以及混合分量的参数 θ, 而不依赖于其他数据点的值. 注意“独立同分布”包含两层含义: 独立使乘积分解成立, 同分布使各因子形式相同; 证明只用到前者.

15.4.3 参数先验

我们也可以使用 EM 算法来最大化引入了参数先验 $p(\theta)$ 的模型中的后验分布 $p(\theta\mid\boldsymbol{X})$. 为了理解这一点, 注意作为 θ 的函数, 我们有 $p(\theta\mid\boldsymbol{X})=p(\theta,\boldsymbol{X})/p(\boldsymbol{X})$, 所以

$$\ln p(\theta\mid\boldsymbol{X})=\ln p(\theta,\boldsymbol{X})-\ln p(\boldsymbol{X}). \tag{15.58}$$

利用(15.52), 我们有

$$\begin{aligned}\ln p(\theta\mid\boldsymbol{X})&=\mathcal{L}(q,\theta)+\mathrm{KL}(q\,\|\,p)+\ln p(\theta)-\ln p(\boldsymbol{X})\\&\geqslant\mathcal{L}(q,\theta)+\ln p(\theta)-\ln p(\boldsymbol{X}),\end{aligned} \tag{15.59}$$

其中 $\ln p(\boldsymbol{X})$ 是一个常数.

命题 15.16 (MAP-EM 的两步)

在引入参数先验 $p(\theta)$ 的模型中, 交替优化(15.59)右端的下界: 关于 q 的最优解与标准 EM 的 E 步骤相同, 即 $q(\boldsymbol{Z})=p(\boldsymbol{Z}\mid\boldsymbol{X},\theta^{\mathrm{old}})$; 关于 θ 的最优解由最大化 $\boldsymbol{Q}(\theta,\theta^{\mathrm{old}})+\ln p(\theta)$ 给出.

证明

由(15.58)与(15.52),

$$\ln p(\theta\mid\boldsymbol{X})=\mathcal{L}(q,\theta)+\mathrm{KL}(q\,\|\,p(\boldsymbol{Z}\mid\boldsymbol{X},\theta))+\ln p(\theta)-\ln p(\boldsymbol{X}).$$

关于 q 优化时,$\ln p(\boldsymbol{X})$ 与 $\ln p(\theta)$ 都是常数,q 只出现在 $\mathcal{L}$ 与 KL 中, 而这两项之和等于 $\ln p(\boldsymbol{X}\mid\theta)$(与 q 无关); 故关于 q 的最优解与最大似然情形完全一致, 即令 $\mathrm{KL}=0$,$q(\boldsymbol{Z})=p(\boldsymbol{Z}\mid\boldsymbol{X},\theta^{\mathrm{old}})$.

关于 θ 优化时,q 固定,$\mathrm{KL}\geqslant 0$ 可丢弃 (丢弃后得到(15.59)的下界); 又由(15.56),$\mathcal{L}(q,\theta)=\boldsymbol{Q}(\theta,\theta^{\mathrm{old}})+\text{const}$, 常数与 θ 无关. 因此关于 θ 最大化(15.59)的下界等价于最大化 $\boldsymbol{Q}(\theta,\theta^{\mathrm{old}})+\ln p(\theta)$.

□

注

(重构新增) 这条命题说明 MAP-EM 只在 M 步骤上做了修改:E 步骤不变,M 步骤的目标多了一个 $\ln p(\theta)$. 这个附加项的作用是**正则化**: 它在 θ 的先验概率小的地方给出很大的负值, 从而把最优解从命题 15.4那种“方差趋于零”的退化方向推开, 因此具有消除 Gauss 混合模型似然函数奇异性的作用 (参见第 9 章). 条件在于 $p(\theta)$ 必须是正常的 (可归一化的) 先验, 否则 $\ln p(\theta)$ 无界,M 步骤可能无解; 此外, 若先验在某些参数区域为零, 则最优点被限制在该区域的闭包内, 此时 M 步骤可能落在边界上. 注意这里最大化的不是后验本身而是它的一个下界, 因此 MAP-EM 同样只保证后验单调不减, 不保证到达全局最大.

我们可以再次交替地针对 q 和 θ 优化(15.59)右侧的表达式. 针对 q 的优化导致与标准 EM 算法相同的 E 步骤方程, 因为 q 只出现在 $\mathcal{L}(q,\theta)$ 中. 通过引入先验项 $\ln p(\theta)$,M 步骤方程被修改, 这通常只需要对标准最大似然的 M 步骤方程进行小幅度的修改即可 (参见第 9 章). 附加项代表着一种正则化形式, 并且具有消除 Gauss 混合模型似然函数奇异性的作用.

15.4.4 广义 EM 算法

EM 算法将最大化似然函数这个潜在的困难问题分解为两个阶段:E 步骤和 M 步骤, 每个步骤通常都更容易实现. 但尽管如此, 对于复杂模型来说,E 步骤或 M 步骤, 或者说两者都可能变得难以处理. 因此 EM 算法有两种扩展.

广义 EM(Generalized EM,GEM) 算法解决了难以处理的 M 步骤问题.GEM 算法不对 θ 最大化 $\mathcal{L}(q,\theta)$, 而是寻求以这样一种方式改变参数: 增大其值. 因为 $\mathcal{L}(q,\theta)$ 是对数似然函数的下界, 所以 GEM 算法的每一个完整循环都保证了能够增大对数似然函数的值 (除非参数已经对应于局部最大值).

命题 15.17 (GEM 的单调性)

设 $q(\boldsymbol{Z})=p(\boldsymbol{Z}\mid\boldsymbol{X},\theta^{\text{old}})$ 为 E 步骤的后验, 而 M 步骤只要求新参数 θ^{new} 满足

$$\mathcal{L}(q,\theta^{\text{new}})\geqslant\mathcal{L}(q,\theta^{\text{old}})$$

(不要求取到最大). 则仍有 $\ln p(\boldsymbol{X}\mid\theta^{\text{new}})\geqslant\ln p(\boldsymbol{X}\mid\theta^{\text{old}})$.

♠

证明

与定理 15.2的证明相同, 只是把 “θ^{new} 最大化 $\mathcal{L}(q,\cdot)$” 这一步换成假设的不等式:

$$\ln p(\boldsymbol{X}\mid\theta^{\text{new}})\geqslant\mathcal{L}(q,\theta^{\text{new}})\geqslant\mathcal{L}(q,\theta^{\text{old}})=\ln p(\boldsymbol{X}\mid\theta^{\text{old}}).$$

□

注

(重构新增) GEM 的要点是: 单调性只需要“下界不降”, 不需要“下界最大”. 这带来很大的自由度——M 步骤可以用一步梯度上升、一步 Newton 迭代, 或者坐标式的分块优化来近似最大化 $\mathcal{L}$, 只要保证 $\mathcal{L}$ 不减小即可. 代价是收敛变慢: 每步的增量可能很小, 且不再有“一步到位”的最优性. 利用 GEM 的一种方式是在 M 步骤中使用基于梯度的迭代优化算法. 另一种形式的 GEM 算法称为期望条件最大化算法 (expectation conditional maximization algorithm), 涉及在每个 M 步骤中进行多个约束优化 (Meng and Rubin, 1993): 参数可能划分为几组,M 步骤则拆分为多个步骤, 每个步骤涉及优化一个组, 同时固定其他组. 这一做法的正确性正是本命题: 每个子步骤只增大 $\mathcal{L}$, 于是整个循环仍单调. 条件在于子步骤必须真的使 $\mathcal{L}$ 不降 (例如用精确的线搜索或足够小的步长), 若用固定的粗糙近似而实际使 $\mathcal{L}$ 下降, 单调性就不再成立.

我们同样可以通过对 $q(\boldsymbol{Z})$ 进行部分优化而非完全优化来推广 EM 算法的 E 步骤 (Neal and Hinton, 1999). 正如我们所见, 对于任何给定的 θ 值,$\mathcal{L}(q,\theta)$ 关于 $q(\boldsymbol{Z})$ 有一个唯一的最大值, 对应于后验分布 $q_\star(\boldsymbol{Z})=p(\boldsymbol{Z}\mid\boldsymbol{X},\theta)$, 并且对于这个选择的 $q(\boldsymbol{Z})$, 下界 $\mathcal{L}(q,\theta)$ 等于对数似然函数 $\ln p(\boldsymbol{X}\mid\theta)$. 因此, 任何收敛到 $\mathcal{L}(q,\theta)$ 全局最大值的算法都能找到一个同时也是对数似然函数 $\ln p(\boldsymbol{X}\mid\theta)$ 全局最大值的 θ 值. 只要 $p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)$ 是 θ 的连续函数, 那么根据连续性,$\mathcal{L}(q,\theta)$ 的任何局部最大值也将是 $\ln p(\boldsymbol{X}\mid\theta)$ 的局部最大值.

注

(重构新增) 这段讨论给出了“部分 E 步骤”合法性的理由, 也说明了广义 EM 与标准 EM 的**区别**所在: 标准 EM 要求 E 步骤取到 $\mathcal{L}$ 关于 q 的精确最大 (即后验), 而 Neal–Hinton 的增量版本只要求 E 步骤使 $\mathcal{L}$ 增大; 同理,GEM 只要求 M 步骤使 $\mathcal{L}$ 增大. 两者的共同点是“每步都不让下界下降”, 因此命题 15.17的论证对它们同时适用, 这就是它们都被称为“广义 EM”的原因. 代价是: 标准 EM 的 E 步骤使下界与对数似然在 θ^{old} 处相切 (散度为零), 而部分优化的 E 步骤一般没有这一性质, 下界可能严格低于对数似然, 收敛因此变慢. 连续性条件不可省: 它保证 $\mathcal{L}$ 的局部最大点也是 $\ln p$ 的局部最大点; 若 p 关于 θ 不连续,$\mathcal{L}$ 的最大点可能落在 $\ln p$ 的跳跃处, 结论失效.

15.4.5 顺序 EM 算法

考虑 N 个独立的数据点 $\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N$ 及相应的隐变量 $\boldsymbol{z}_1,\cdots,\boldsymbol{z}_N$. 将联合分布 $p(\boldsymbol{X},\boldsymbol{Z}\mid\theta)$ 在数据点上分解, 这种结构可以在增量形式的 EM 算法中利用, 在每一个 EM 循环中, 一次只处理一个数据点. 在 E 步骤中, 我们不是重新计算所有数据点的责任, 而是只重新计算一个数据点的责任. 看起来随后的 M 步骤可能涉及对所有数据点的责任的计算. 然而, 如果混合分量是指数族分布, 则责任仅存在于简单的充分统计量 (sufficient statistic), 并且可以高效地更新. 例如, 考虑一个 Gauss 混合模型, 假设我们对数据点 $\boldsymbol{x}_m$ 进行更新, 其中相应的旧责任值和新责任值分别表示为 $\gamma^{\text{old}}(z_{mk})$ 和 $\gamma^{\text{new}}(z_{mk})$. 在 M 步骤中, 所需的充分统计量可以递增地更新. 例如, 对于均值, 充分统计量由(15.16)和(15.17)定义, 我们可以得到

$$\boldsymbol{\mu}_k^{\text{new}}=\boldsymbol{\mu}_k^{\text{old}}+\left(\frac{\gamma^{\text{new}}(z_{mk})-\gamma^{\text{old}}(z_{mk})}{N_k^{\text{new}}}\right)\left(\boldsymbol{x}_m-\boldsymbol{\mu}_k^{\text{old}}\right),\tag{15.60}$$

其中

$$N_k^{\text{new}}=N_k^{\text{old}}+\gamma^{\text{new}}(z_{mk})-\gamma^{\text{old}}(z_{mk}).\tag{15.61}$$

命题 15.18 (顺序 EM 的均值更新)

设 $N_k^{\text{old}}=\sum_n\gamma^{\text{old}}(z_{nk})$、$N_k^{\text{new}}=\sum_n\gamma^{\text{new}}(z_{nk})$, 且两个责任向量只在第 m 个数据点处不同 (即对 $n\neq m$ 有 $\gamma^{\text{new}}(z_{nk})=\gamma^{\text{old}}(z_{nk})$). 则(15.61)与(15.60)成立, 且更新后的均值精确等于用新责任加权的平均.

证明

对 $n\neq m$ 的责任相同, 故

$$N_k^{\text{new}}-N_k^{\text{old}}=\gamma^{\text{new}}(z_{mk})-\gamma^{\text{old}}(z_{mk}),$$

即(15.61). 再由(15.16),

$$\begin{aligned}N_k^{\text{new}}\boldsymbol{\mu}_k^{\text{new}}&=\sum_n\gamma^{\text{new}}(z_{nk})\boldsymbol{x}_n=\sum_n\gamma^{\text{old}}(z_{nk})\boldsymbol{x}_n+\left[\gamma^{\text{new}}(z_{mk})-\gamma^{\text{old}}(z_{mk})\right]\boldsymbol{x}_m\\&=N_k^{\text{old}}\boldsymbol{\mu}_k^{\text{old}}+\left[\gamma^{\text{new}}(z_{mk})-\gamma^{\text{old}}(z_{mk})\right]\boldsymbol{x}_m.\end{aligned}$$

两边减去 $\left[\gamma^{\text{new}}(z_{mk})-\gamma^{\text{old}}(z_{mk})\right]\boldsymbol{\mu}_k^{\text{old}}$, 并用(15.61)把 N_k^{old} 换成 $N_k^{\text{new}}-\left[\gamma^{\text{new}}-\gamma^{\text{old}}\right]$:

$$N_k^{\text{new}}\left(\boldsymbol{\mu}_k^{\text{new}}-\boldsymbol{\mu}_k^{\text{old}}\right)=\left[\gamma^{\text{new}}(z_{mk})-\gamma^{\text{old}}(z_{mk})\right]\left(\boldsymbol{x}_m-\boldsymbol{\mu}_k^{\text{old}}\right),$$

除以 N_k^{new} 即得(15.60). 由于推导只用到(15.16), 没有作任何近似, 新均值精确等于以新责任为权重的加权平均. □

注

(重构新增) 顺序 EM 与命题 15.2的增量更新是同一件事的两种写法: 后者只“加入”一个新点 ($\gamma^{\text{old}}=0$), 前者允许“改派”一个已有点 (旧责任非零, 故要减去旧贡献). 这正是(15.60)比分段 K 均值的(15.5)多出 $-\gamma^{\text{old}}(z_{mk})$ 项的原因. 命题的前提“只有第 m 个数据点的责任改变”必须成立: 若一次更新改动了多个点的责任, 就必须把每个变化量都累加上去, 否则(15.61)不成立. 协方差和混合系数的相应结果是类似的, 只需把充分统计量换成 $\sum_n\gamma(z_{nk})\boldsymbol{x}_n\boldsymbol{x}_n^{\mathsf{T}}$ 与 $\sum_n\gamma(z_{nk})$; 推导与上面完全相同, 只是把 $\boldsymbol{x}_m$ 换成 $\boldsymbol{x}_m\boldsymbol{x}_m^{\mathsf{T}}$.

协方差和混合系数的相应结果是类似的. 因此,E 步骤和 M 步骤花费的时间都是固定的, 与数据点的数量无关. 因为参数在每个数据点之后就修正了, 而不是等到整个数据集处理完毕之后才修正, 所以这种增量版本比批处理版本收敛得更快. 这种增量版本中的每个 E 步骤或 M 步骤都会增加 $\mathcal{L}(q,\theta)$ 的值, 如果算法收敛到 $\mathcal{L}(q,\theta)$ 的局部 (或全局) 最大值, 这将对应于对数似然函数 $\ln p(\boldsymbol{X}\,|\,\theta)$ 的局部 (或全局) 最大值.

15.5 习题

练习 15.1 (⋆) 考虑节 15.1讨论的 K 均值算法. 证明由于离散指示变量 r_{nk} 存在有限数量的可能取值, 对于每个这样的取值都有一个唯一的最优集合 $\{\boldsymbol{\mu}_k\}$, 因此 K 均值算法在有限次迭代后必定会收敛.

练习 15.2 (⋆⋆) 推导 K 均值算法的顺序形式. 在每一步, 考虑一个新的数据点 $\boldsymbol{x}_n$, 并且只更新与数据点 $\boldsymbol{x}_n$ 最近的原型向量. 从批处理设置中用于原型向量的(15.4)出发, 将来自最后一个数据点 $\boldsymbol{x}_n$ 的贡献单独分离出来. 通过重新排列各项, 证明这种更新采取的是(15.5)的形式. 注意, 由于在此推导过程中没有做任何近似, 得到的原型向量具有以下性质: 它们各自等于分配给它们的所有数据向量的均值.

练习 15.3 (⋆) 考虑一个 Gauss 混合模型, 其中潜变量的边缘分布 $p(\boldsymbol{z})$ 由(15.9)给出, 观测变量的条件分布 $p(\boldsymbol{x}\,|\,\boldsymbol{z})$ 由(15.10)给出. 证明边缘分布 $p(\boldsymbol{x})$(通过对所有可能的 $\boldsymbol{z}$ 值求和 $p(\boldsymbol{z})p(\boldsymbol{x}\,|\,\boldsymbol{z})$ 而获得), 是形式为(15.6)的 Gauss 混合分布.

练习 15.4 (⋆) 证明在具有 K 个分量的混合模型中, 交换对称性导致的等价参数设置的数量是 $K!$.

练习 15.5 (⋆⋆) 假设我们希望使用 EM 算法来最大化包含潜变量的模型的参数后验分布 $p(\theta\,|\,\boldsymbol{X})$, 其中 $\boldsymbol{X}$ 是观测数据集. 证明 E 步骤在最大似然情况下保持不变, 而 M 步骤中要最大化的量由 $Q(\theta,\theta^{\text{old}})+\ln p(\theta)$ 给出, 其中 $Q(\theta,\theta^{\text{old}})$ 由(15.23)定义.

练习 15.6 (⋆) 考虑图 15.9所示的 Gauss 混合模型的有向图. 利用 d 分离准则, 证明潜变量的后验分布可以关于不同的数据点进行因子分解, 即

$$p(\boldsymbol{Z}\,|\,\boldsymbol{X},\boldsymbol{\mu},\boldsymbol{\Sigma},\boldsymbol{\pi})=\prod_{n=1}^{N}p(\boldsymbol{z}_n\,|\,\boldsymbol{x}_n,\boldsymbol{\mu},\boldsymbol{\Sigma},\boldsymbol{\pi}). \tag{15.62}$$

练习 15.7 (⋆⋆) 考虑 Gauss 混合模型的一种特殊情况, 其中每个分量的协方差矩阵 $\boldsymbol{\Sigma}_k$ 都受限于它们所具有的一个共同的值 $\boldsymbol{\Sigma}$. 在这样的模型下推导用于最大化似然函数的 EM 方程.

练习 15.8 (⋆⋆) 验证对 Gauss 混合模型的完整数据对数似然函数 [(15.26)] 进行最大化给出的 EM 方程.

练习 15.9 (⋆⋆) 证明如果我们在关于 $\boldsymbol{\Sigma}_k$ 最大化(15.30)的同时保持 $\gamma(z_{nk})$ 固定, 我们将获得(15.16)给出的闭式解.

注

(重构新增: 原文问题说明) 本题印为“关于 $\boldsymbol{\Sigma}_k$ 最大化”, 却断言得到(15.16)——后者是均值的更新式. 二者必有一处笔误: 关于 $\boldsymbol{\Sigma}_k$ 求极值应得到协方差更新(15.18)(这恰是下一题 15.10 的内容), 而得到(15.16)的应是关于 $\boldsymbol{\mu}_k$ 求极值. 这里保持原文照译, 并在解答中把两种读法都完整推导一遍: 按“关于 $\boldsymbol{\mu}_k$”读, 结论(15.16)成立; 按“关于 $\boldsymbol{\Sigma}_k$”读, 得到的是(15.18). 读者应以“关于 $\boldsymbol{\mu}_k$”为准.

练习 15.10 (⋆⋆) 证明如果我们在关于 $\boldsymbol{\Sigma}_k$ 和 π_k 最大化(15.30)的同时保持 $\gamma(z_{nk})$ 固定, 我们将获得(15.18)和(15.21)给出的闭式解.

练习 15.11 (⋆⋆) 考虑由混合分布给出的概率密度模型:

$$p(\boldsymbol{x})=\sum_{k=1}^{K}\pi_k p(\boldsymbol{x}\,|\,k) \tag{15.63}$$

假设我们将向量 $\boldsymbol{x}$ 分割成两部分, 即 $\boldsymbol{x}=(\boldsymbol{x}_a,\boldsymbol{x}_b)$.

证明条件密度 $p(\boldsymbol{x}_a\,|\,\boldsymbol{x}_b)$ 本身就是一个混合分布, 并给出混合系数和分量密度的表达式.

练习 15.12 (⋆) 在小节 15.3.2小节中, 我们通过考虑一个所有分量都有协方差 $\varepsilon\boldsymbol{I}$ 的混合模型, 得到了 K 均值算法和 EM 算法之间的关系. 证明在 $\varepsilon\to 0$ 的情况下, 最大化这个模型的完整数据对数似然函数的期望 [(15.30)] 等同于最小化 K 均值算法给出的误差度量 J [(15.1)].

练习 15.13 (⋆⋆) 验证 Bernoulli 分布的均值和协方差结果 [(15.35)和(15.36)].

✍ **练习 15.14** (★★) 考虑如下形式的混合分布:

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k p(\boldsymbol{x} \mid k) \tag{15.64}$$

其中 $\boldsymbol{x}$ 的元素可以是离散变量或连续变量, 也可以是它们两者的组合.$p(\boldsymbol{x} \mid k)$ 的均值和协方差分别为 $\boldsymbol{\mu}_k$ 和 $\boldsymbol{\Sigma}_k$. 利用练习 15.13的结果, 证明该混合分布的均值和协方差分别由(15.39)和(15.40)给出.

✍ **练习 15.15** (★★) 使用 EM 算法的重新估计方程, 证明当其参数设置对应于似然函数的最大值时,Bernoulli 混合模型具有以下性质:

$$\mathbb{E}[\boldsymbol{x}] = \frac{1}{N} \sum_{n=1}^{N} \boldsymbol{x}_n = \bar{\boldsymbol{x}}. \tag{15.65}$$

继续证明如果这个模型的参数初始化使得所有分量都有相同的平均值 $\boldsymbol{\mu}_k = \hat{\boldsymbol{\mu}}, k = 1, \cdots, K$, 则 EM 算法将在任何初始混合系数的选择下, 在一次迭代后收敛, 且这个解具有 $\boldsymbol{\mu}_k = \bar{\boldsymbol{x}}$ 的性质. 注意, 这代表了混合模型的一种退化情况, 实践中我们可以尝试通过进行适当的初始化来避免这种解.

✍ **练习 15.16** (★) 考虑通过 $p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{\mu})$ [(15.42)] 和 $p(\boldsymbol{z} \mid \boldsymbol{\pi})$ [(15.43)] 的乘积而得到的 Bernoulli 分布的潜变量和观测变量的联合分布. 证明如果我们就 $\boldsymbol{z}$ 对这个联合分布进行边缘化, 我们将得到(15.37).

✍ **练习 15.17** (★) 证明如果针对 $\boldsymbol{\mu}_k$ 最大化 Bernoulli 混合模型的完整数据对数似然函数的期望 [(15.45)], 我们将得到 M 步骤方程 [(15.49)].

✍ **练习 15.18** (★) 证明如果我们针对混合系数 π_k 最大化 Bernoulli 混合模型的完整数据对数似然函数的期望[(15.45)], 并使用 Lagrange 乘子引入求和约束, 我们将得到 M 步骤方程 [(15.50)].

✍ **练习 15.19** (★) 证明由于离散变量 $\boldsymbol{x}_n$ 的概率 $p(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k)$ 满足约束条件 $0 \leqslant p(\boldsymbol{x}_n \mid \boldsymbol{\mu}_k) \leqslant 1$,Bernoulli 混合模型的不完整数据对数似然函数存在上界, 因此不存在使对数似然函数值趋于无穷大的奇异点.

✍ **练习 15.20** (★★★) 考虑一个 D 维变量 $\boldsymbol{x}$, 它的每个分量本身是一个 M 阶多项式变量, 因此 $\boldsymbol{x}$ 是一个二值向量, 其分量为 x_{ij}, 其中 $i = 1, \cdots, D$ 且 $j = 1, \cdots, M$, 约束条件 $\sum_j x_{ij} = 1$ 对所有 i 都成立. 假设这些变量的分布可以用一系列离散多项式分布的混合分布来描述 (见小节 3.1.3), 则有

$$p(\boldsymbol{x}) = \sum_{k=1}^{K} \pi_k \, p(\boldsymbol{x} \mid \boldsymbol{\mu}_k), \tag{15.66}$$

其中

$$p(\boldsymbol{x} \mid \boldsymbol{\mu}_k) = \prod_{i=1}^{D} \prod_{j=1}^{M} \mu_{kij}^{x_{ij}}. \tag{15.67}$$

参数 μ_{kij} 表示概率 $p(x_{ij} = 1 \mid \boldsymbol{\mu}_k)$, 并且必须满足 $0 \leqslant \mu_{kij} \leqslant 1$, 约束条件 $\sum_j \mu_{kij} = 1$ 对所有的 k 值和 i 值都成立.

给定一个观测数据集 $\{\boldsymbol{x}_n\}$, 其中 $n = 1, \cdots, N$, 推导出 EM 算法中的 E 步骤方程和 M 步骤方程, 以便通过最大似然法优化混合系数 π_k 和分量参数 μ_{kij}.

✍ **练习 15.21** (★) 验证(15.52)中的 $\mathcal{L}(q, \theta)$ 和 $\mathrm{KL}(q \,\|\, p)$ 分别由(15.53)和(15.54)定义.

✍ **练习 15.22** (★) 证明(15.53)给出的下界 $\mathcal{L}(q, \theta)$ 以 $q(\boldsymbol{Z}) = p(\boldsymbol{Z} \mid \boldsymbol{X}, \theta^{\text{old}})$ 的形式存在, 其关于 θ 的梯度与对数似然函数 $\ln p(\boldsymbol{X} \mid \theta)$ 在 $\theta = \theta^{\text{old}}$ 处的梯度相同.

✍ **练习 15.23** (★★) 考虑应用于 Gauss 混合模型的 EM 算法的增量形式, 在此形式中, 只针对特定数据点 $\boldsymbol{x}_m$ 重新计算责任. 从 M 步公式 [(15.16)和(15.17)] 出发, 推导更新分量均值的结果 [(15.60)和(15.61)].

✍ **练习 15.24** (★★) 当责任值以增量方式更新时, 推导出在 Gauss 混合模型中更新协方差矩阵和混合系数的 M 步骤公式, 这类似于更新分量均值的结果 [(15.60)].

第 16 章 连续潜变量

在第 15 章中, 我们讨论了拥有离散潜变量的概率模型, 例如 Gauss 混合模型. 本章探讨部分或全部潜变量为连续变量的模型. 研究这类模型的一个重要原因是很多数据集都有这样的特点: 数据点都挤在一个比起原始数据空间维度低得多的流形 (manifold) 上. 为什么会这样呢? 想象一下, 假如我们从 MNIST 数据集 (LeCun et al., 1998) 里选出一个显示手写数字的 64×64 像素灰度图, 将它嵌入一个更大的 100×100 像素的图像中, 图像边缘用数值为零的像素 (对应白色像素) 填充. 嵌入的同时, 随机改变数字的位置和方向, 就像图 16.1展示的那样, 那么每一个生成的图像由高达 10000 维 (因为是 100×100 像素) 的数据空间中的一个数据点来表示. 然而在这样的图像数据集中, 仅有三个自由度 (degree of freedom) 的变化, 分别对应于垂直移动、平移及旋转. 因此, 这些数据点实际上存在于一个内在维度 (intrinsic dimensionality) 为 3 的子空间中. 请注意, 这个流形是非线性的, 因为如果我们将数字平移经过一个特定像素, 那么这个像素的值将从 0(白色) 变成 1(黑色), 然后再变回 0, 这显然是关于数字位置的非线性函数. 在这个例子中, 平移和旋转的相关参数是潜变量, 因为我们只观察到图像向量本身, 而不知道用来生成它们的平移或旋转变量的具体值.

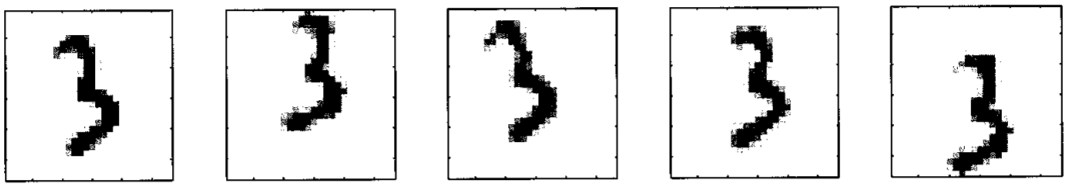


图 16.1: 一个合成的数据集, 从中获取一个手写数字图像并创建多个副本, 在每个副本中, 手写数字在某个更大的图像区域内进行随机移动和旋转

在真实的手写数字数据集中, 会有更多的自由度出现. 例如, 不同人的书写风格有差异, 而同一个人的书写风格也有变化, 这就会导致手写数字在尺寸和其他方面上的变化. 不过, 即使有这些自由度, 它们的数量相较于整个数据集的维度还是显得很少.

在实践中, 数据点不会精确地局限于一个光滑的低维流形上, 我们可以将数据点偏离流形的现象解释为“噪声”. 我们可以自然地从生成性视角 (generative view) 来理解此类模型. 首先根据某种潜变量分布在流形内选择一个点, 然后添加从给定潜变量的数据变量的某种条件分布中抽取的噪声, 生成一个观测数据点.

对于最简单的连续潜变量模型 (continuous latent-variable model), 可以假设潜变量和观测变量都遵循 Gauss 分布, 并利用线性-Gauss 依赖性 (linear-Gaussian dependence)(见小节 11.1.4) 来描述观测变量对潜变量状态的依赖. 这引出了著名的主成分分析 (Principal Component Analysis, PCA) 概率公式, 还有称为因子分析 (factor analysis) 的相关模型. 本章首先介绍如何使用标准非概率方法来实现 PCA, 随后揭示 PCA 如何自然而然地对应于线性-Gauss 潜变量模型的最大似然解 (maximum likelihood solution)(见小节 16.2.3). 这种概率化表述带来了许多优势 (见节 16.2), 比如使用 EM 算法进行参数估计, 规范化地扩展成 PCA 混合模型, 以及通过 Bayes 公式自动从数据中确定主成分的数量 (Bishop, 2006). 本章还为我们学习具有连续潜变量的非线性模型奠定了基础, 这些模型包括标准化流、变分自编码器和扩散模型.

16.1 主成分分析

主成分分析 (PCA) 已广泛应用于降维、有损数据压缩、特征提取和数据可视化等领域 (Jolliffe, 2002), 它也称为 Kosambi–Karhunen–Loève 变换. 考虑将一个数据集正交投影到一个低维的线性空间 [即主子空间 (principal subspace)], 如图 16.2所示.

PCA 可以定义为最大化投影数据方差的线性投影 (Hotelling, 1933). 同样,PCA 也可以定义为最小化平均投影代价的线性投影, 平均投影代价则定义为数据点与其投影之间的均方距离 (Pearson, 1901). 我们将依次学习这些定义.

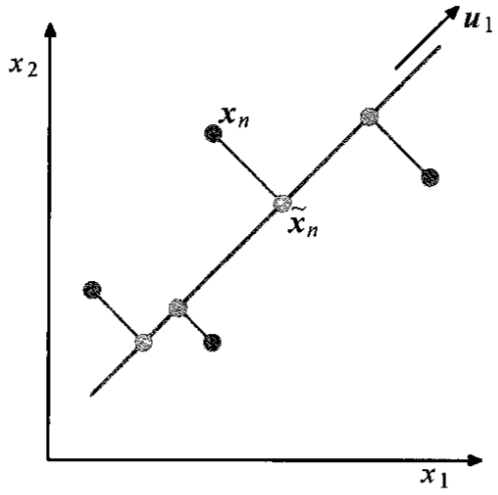


图 16.2: 利用 PCA 寻求一个低维空间, 称为主子空间 (principal subspace), 用洋红色线条表示, 以使数据点 (红色点) 在主子空间中的正交投影能够最大化投影点 (绿色点) 的方差.PCA 的另一种定义是基于最小化投影误差的平方和给出的, 这些误差用蓝色线条表示

16.1.1 最大方差表述

考虑一个观测数据集 $\{\boldsymbol{x}_n\}$, 其中 $n = 1, \cdots, N$, 并且 $\boldsymbol{x}_n$ 是一个维度为 D 的欧几里得变量. 我们想要把数据投影到维度为 $M < D$ 的空间, 同时最大化投影数据的方差. 目前我们假设 M 的值是给定的. 在本章的后面, 我们会学习从数据中确定 M 的合适值的技术.

首先考虑投影到一维空间 ($M = 1$). 我们可以使用一个 D 维向量 $\boldsymbol{u}_1$ 来定义这个空间的方向, 为了方便 (且不失一般性), 我们会选择单位向量, 因此 $\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{u}_1 = 1$(注意我们只对 $\boldsymbol{u}_1$ 定义的方向感兴趣). 然后将每个数据点 $\boldsymbol{x}_n$ 投影到一个标量值 $\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{x}_n$ 上. 投影数据的平均值是 $\boldsymbol{u}_1^{\mathsf{T}}\bar{\boldsymbol{x}}$, 其中 $\bar{\boldsymbol{x}}$ 是样本集的平均值, 由下式给出:

$$\bar{\boldsymbol{x}} = \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{x}_n. \tag{16.1}$$

投影数据的方差由下式给出:

$$\frac{1}{N}\sum_{n=1}^{N}\left\{\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{x}_n - \boldsymbol{u}_1^{\mathsf{T}}\bar{\boldsymbol{x}}\right\}^2 = \boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_1, \tag{16.2}$$

其中 $\boldsymbol{S}$ 是数据协方差矩阵, 定义为

$$\boldsymbol{S} = \frac{1}{N}\sum_{n=1}^{N}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}. \tag{16.3}$$

下面我们将投影数据的方差 $\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_1$ 相对于 $\boldsymbol{u}_1$ 最大化. 显然, 这必须是一个受约束的最大化操作, 以防止 $\|\boldsymbol{u}_1\| \to \infty$. 适当的约束来源于归一化条件 $\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{u}_1 = 1$. 为了强制执行这个约束, 我们引入一个 Lagrange 乘子 (见附录 C), 记作 λ_1, 并对下式进行无约束的最大化:

$$\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_1 + \lambda_1(1 - \boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{u}_1). \tag{16.4}$$

通过将(16.4)对 $\boldsymbol{u}_1$ 的导数设为零, 我们发现当以下条件满足时, 目标函数将取驻点:

$$\boldsymbol{S}\boldsymbol{u}_1 = \lambda_1\boldsymbol{u}_1. \tag{16.5}$$

这意味着 $\boldsymbol{u}_1$ 必须是 $\boldsymbol{S}$ 的一个特征向量. 如果将(16.5)左乘 $\boldsymbol{u}_1^{\mathsf{T}}$ 并利用归一化条件 $\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{u}_1 = 1$, 可得方差为

$$\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_1 = \lambda_1. \tag{16.6}$$

因此, 当我们将 $\boldsymbol{u}_1$ 设置为具有最大特征值 λ_1 的特征向量时, 方差将达到最大.

定义 16.1 (主成分与主子空间) 设 $\boldsymbol{S}$ 是数据集的 $D \times D$ 协方差矩阵, 其特征值为 $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_D$, 对应的单位特征向量为 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_D$. 称对应于最大特征值的特征向量 $\boldsymbol{u}_1$ 为第一主成分 (first principal component). 一般地, 在 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_{M-1}$ 已经选定之后, 称在所有与它们正交的单位向量中使投影方差 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}$ 取最大值的 $\boldsymbol{u}_M$ 为第 M 个主成分, 称由前 M 个主成分张成的 M 维线性子空间为主子空间 (principal subspace). ♣

注

(重构新增) PCA 要解决的问题是: 在 D 维数据空间中找出一个 M 维线性子空间, 使得把数据压缩到该子空间上所丢失的信息尽可能少. 这里把“信息”量化为投影后数据的方差, 理由是: 若某个方向上的投影几乎没有变化, 那么沿该方向的坐标对区分数据点毫无贡献, 删去它不会损失可分性; 反之, 投影方差大的方向保留了数据的主要变动. 因此“最大化投影方差”是对“保留最多信息”的一种可计算的具体化.

定义中的几个条件各有作用. 第一, 归一化约束 $\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{u}_1 = 1$ 不是数学上的限制而是必要的: 若不加约束,$\boldsymbol{u}_1^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_1$ 可以沿任一使二次型取正的方向无限增大, 最大化问题没有解; 归一化把可行集变成紧集 (单位球面), 从而最大值一定取到. 第二,M 被当作给定常数, 是因为主成分的个数是模型选择问题, 本书后面会用 Bayes 方法从数据中确定它; 此处先假定已知, 以免与优化问题混在一起. 第三, 协方差矩阵 $\boldsymbol{S}$ 对称半正定, 故其特征值非负且可取正交特征向量组, 这是(16.5)有实解并可按大小排序的前提. 此外,$\boldsymbol{S}$ 采用除以 N 的约定, 除以 $N-1$ 的无偏估计只与之相差一个常数因子, 不影响特征向量的结论.

我们可以通过逐步的方式定义更多的主成分, 方法就是选取一个新方向, 使其在所有与已考虑方向正交的方向中最大化投影后的方差. 考虑 M 维投影空间的一般情况, 投影数据方差最大化的最优线性投影由数据协方差矩阵 $\boldsymbol{S}$ 对应于最大特征值 $\lambda_1, \cdots, \lambda_M$ 的 M 个特征向量 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M$ 定义 (见练习 16.1). 这可以很容易地使用数学归纳法来证明.

命题 16.1 (最大方差表述的最优性) 设 $\boldsymbol{S}$ 为 $D \times D$ 对称半正定矩阵, 其特征值按 $\lambda_1 \geqslant \cdots \geqslant \lambda_D \geqslant 0$ 排列, 对应的单位特征向量为 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_D$ 且两两正交. 对任意 $1 \leqslant M \leqslant D$, 在全体满足 $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j = \delta_{ij}$ 的 D 维向量组 $\{\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M\}$ 上, 投影数据的方差之和

$$\sum_{i=1}^{M} \boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_i$$

的最大值等于 $\sum_{i=1}^{M} \lambda_i$, 且当且仅当 $\{\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M\}$ 张成由前 M 个特征向量张成的子空间时取到. ♠

证明

(重构新增) 对 M 作归纳.$M=1$ 的情形已由(16.5)与(16.6)给出: 单位球面是紧集,$\boldsymbol{u}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}$ 连续, 故最大值存在; 在约束 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{u}=1$ 下用乘子 λ, 驻点满足 $\boldsymbol{S}\boldsymbol{u}=\lambda\boldsymbol{u}$, 代回得目标值 λ, 故最大值是最大特征值 λ_1, 在 $\boldsymbol{u}=\boldsymbol{u}_1$(或其反向) 处取到.

设结论对 $M-1$ 成立, 即 $\boldsymbol{u}_1,\cdots,\boldsymbol{u}_{M-1}$ 必须张成前 $M-1$ 个特征向量张成的子空间 $\mathcal{U}_{M-1}$, 此时方差之和为 $\sum_{i=1}^{M-1}\lambda_i$. 现在固定这一部分, 只对 $\boldsymbol{u}_M$ 优化, 约束为

$$\boldsymbol{u}_M^{\mathsf{T}}\boldsymbol{u}_M=1,\qquad \boldsymbol{u}_M^{\mathsf{T}}\boldsymbol{u}_j=0\quad(j=1,\cdots,M-1).$$

引入乘子 λ 与 $\mu_1,\cdots,\mu_{M-1}$, 对下式求驻点:

$$\tilde{J}=\boldsymbol{u}_M^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_M+\lambda(1-\boldsymbol{u}_M^{\mathsf{T}}\boldsymbol{u}_M)+\sum_{j=1}^{M-1}\mu_j\boldsymbol{u}_M^{\mathsf{T}}\boldsymbol{u}_j.$$

对 $\boldsymbol{u}_M$ 求导并令其为零, 得

$$2\boldsymbol{S}\boldsymbol{u}_M-2\lambda\boldsymbol{u}_M+\sum_{j=1}^{M-1}\mu_j\boldsymbol{u}_j=\boldsymbol{0}.$$

左乘 $\boldsymbol{u}_k^{\mathsf{T}}(k\leqslant M-1)$, 利用 $\boldsymbol{u}_M^{\mathsf{T}}\boldsymbol{u}_k=0$ 与 $\boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{u}_k=\delta_{jk}$, 得 $2\boldsymbol{u}_k^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_M+\mu_k=0$. 由于 $\boldsymbol{u}_k$ 是 $\boldsymbol{S}$ 的特征向量,$\boldsymbol{u}_k^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_M=\lambda_k\boldsymbol{u}_k^{\mathsf{T}}\boldsymbol{u}_M=0$, 故 $\mu_k=0$. 于是驻点方程退化为 $\boldsymbol{S}\boldsymbol{u}_M=\lambda\boldsymbol{u}_M$:$\boldsymbol{u}_M$ 是 $\boldsymbol{S}$ 的特征向量, 且与 $\mathcal{U}_{M-1}$ 正交, 因此它必是 $\boldsymbol{u}_M,\cdots,\boldsymbol{u}_D$ 之一, 相应的目标值为其特征值. 在剩下的特征值 $\lambda_M\geqslant\cdots\geqslant\lambda_D$ 中取最大者, 即得 $\boldsymbol{u}_M=\boldsymbol{u}_{M+1}$ 之前的那一个, 也就是 $\boldsymbol{u}_M$, 目标值为 λ_M. 故 M 时的最大值为 $\sum_{i=1}^{M}\lambda_i$.

唯一性: 若某个最优解张成的子空间与 $\mathcal{U}_M$ 不同, 则它对应的特征值之和不可能是最大的 M 个 (除非出现重特征值, 此时不同的特征向量组合张成同一子空间, 命题所说的 “张成” 仍然成立). 因此最优解在子空间意义下唯一.

□

注

(重构新增) 本命题把 “逐步选取正交方向” 这一构造提升为全局最优性断言: 不必逐个求解 M 次约束优化, 只需对 $\boldsymbol{S}$ 作一次特征分解, 取最大的 M 个特征值对应的特征向量即可. 证明的关键是: 在已取定前 $M-1$ 个方向后, 新方向对旧方向的乘子全部为零, 因为旧方向本身就是 $\boldsymbol{S}$ 的特征向量、与任何候选新方向正交; 这一点使问题逐层解耦.

条件的作用如下.$\boldsymbol{S}$ 的对称性是 “特征向量可取为正交组” 的来源; 半正定性保证 $\lambda_i\geqslant 0$, 从而 “取最大的特征值” 等价于 “取最大的方差”. 若 $\boldsymbol{S}$ 有重特征值, 则对应的特征向量不唯一, 但张成的子空间唯一, 命题的结论仍成立; 此时主成分子空间仍然良定义, 只是单个主成分的方向不唯一. 若去掉正交约束, 问题会退化: 各 $\boldsymbol{u}_i$ 都取同一个最大特征向量时方差之和为 $M\lambda_1$, 这并没有给出 M 个不同的方向, 故正交约束是必要的.

综上,PCA 涉及计算数据集的均值 $\bar{\boldsymbol{x}}$ 和数据协方差矩阵 $\boldsymbol{S}$, 然后找到 $\boldsymbol{S}$ 对应于最大特征值的 M 个特征向量. 关于寻找特征向量和特征值的算法, 以及与特征向量分解相关的附加定理, 可以在 Golub and Van Loan (1996) 中找到. 请注意, 计算一个 $D\times D$ 大小矩阵的完整特征向量分解的计算成本是 $O(D^3)$. 如果打算将数据投影到前 M 个主成分上, 则只需要找到前 M 个特征值和特征向量. 这可以通过使用更有效的方法来完成, 比如幂方法 (Golub and Van Loan, 1996), 复杂度为 $O(MD^2)$, 当然也可以使用 EM 算法 (见小节 16.3.2).

16.1.2 最小误差表述

本小节讨论基于投影误差最小化的 PCA 的另一种表述 (见附录 A). 为此, 引入一组 D 维的完备正交规范的基向量 $\{\boldsymbol{u}_i\}$, 其中 $i=1,\cdots,D$, 它们满足

$$\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j=\delta_{ij}. \tag{16.7}$$

由于这组基向量是完备的, 每个数据点都可以通过基向量的线性组合精确表示为

$$\boldsymbol{x}_n = \sum_{i=1}^{D} \alpha_{ni} \boldsymbol{u}_i, \tag{16.8}$$

其中系数 α_{ni} 对于不同的数据点是不同的. 这只是相当于将坐标系旋转到 $\{\boldsymbol{u}_i\}$ 定义的新系统, 原始的 D 个分量 $\{x_{n1}, \cdots, x_{nD}\}$ 被等价的一组分量 $\{\alpha_{n1}, \cdots, \alpha_{nD}\}$ 替代. 与 $\boldsymbol{u}_j$ 内积并利用正交性质, 可以得到 $\alpha_{nj} = \boldsymbol{x}_n^{\mathsf{T}} \boldsymbol{u}_j$. 因此, 不失一般性地, 我们可以写作

$$\boldsymbol{x}_n = \sum_{i=1}^{D} \left(\boldsymbol{x}_n^{\mathsf{T}} \boldsymbol{u}_i \right) \boldsymbol{u}_i. \tag{16.9}$$

不过, 我们的目标是用一个较小的变量数 M(小于原始维度 D) 来近似这个数据点, 相当于将它投影到一个低维的子空间. 这个 M 维的线性子空间可以用前 M 个基向量表示, 这样我们就可以用这些基向量来近似每一个数据点 $\boldsymbol{x}_n$:

$$\tilde{\boldsymbol{x}}_n = \sum_{i=1}^{M} z_{ni} \boldsymbol{u}_i + \sum_{i=M+1}^{D} b_i \boldsymbol{u}_i, \tag{16.10}$$

其中 $\{z_{ni}\}$ 依赖于特定的数据点, 而 $\{b_i\}$ 是对所有数据点都相同的常数. 我们可以自由选择 $\{\boldsymbol{u}_i\}$、$\{z_{ni}\}$ 和 $\{b_i\}$, 以最小化由维度降低引入的误差.

作为误差度量, 我们将使用原始数据点 $\boldsymbol{x}_n$ 与其近似值 $\tilde{\boldsymbol{x}}_n$ 之间的平方距离, 并在数据集上取平均, 因此我们的目标是最小化

$$J = \frac{1}{N} \sum_{n=1}^{N} \|\boldsymbol{x}_n - \tilde{\boldsymbol{x}}_n\|^2. \tag{16.11}$$

首先考虑关于量 $\{z_{ni}\}$ 的最小化. 将 $\tilde{\boldsymbol{x}}_n$ 代入 J, 将 J 关于 z_{ni} 的导数设为零, 利用正交关系, 我们得到

$$z_{ni} = \boldsymbol{x}_n^{\mathsf{T}} \boldsymbol{u}_i, \tag{16.12}$$

其中 $i = 1, \cdots, M$. 类似地, 将 J 关于 b_i 的导数设为零, 并再次利用正交关系, 得到

$$b_i = \bar{\boldsymbol{x}}^{\mathsf{T}} \boldsymbol{u}_i, \tag{16.13}$$

其中 $i = M+1, \cdots, D$. 代入 z_{ni} 和 b_i, 并利用一般展开式 [(16.9)], 得到

$$\boldsymbol{x}_n - \tilde{\boldsymbol{x}}_n = \sum_{i=M+1}^{D} \left\{ (\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}} \boldsymbol{u}_i \right\} \boldsymbol{u}_i. \tag{16.14}$$

我们看到, 从 $\boldsymbol{x}_n$ 到 $\tilde{\boldsymbol{x}}_n$ 的位移向量位于主子空间的正交空间中, 因为它是 $i = M+1, \cdots, D$ 的 $\boldsymbol{u}_i$ 的线性组合, 如图 16.2所示. 这在预期之内, 因为投影点 $\tilde{\boldsymbol{x}}_n$ 必须位于主子空间内, 但我们可以在这个子空间内自由移动它们, 因此最小误差由正交投影给出.

在考虑正式的解之前, 让我们通过一个二维的特例来直观地理解 J 的最小化.

例题 16.1 二维数据的一维主子空间 考虑二维数据空间 ($D = 2$) 与一维主子空间 ($M = 1$). 此时误差度量只涉及一个方向 $\boldsymbol{u}_2$, 且约束为正交归一条件 $\boldsymbol{u}_2^{\mathsf{T}} \boldsymbol{u}_2 = 1$. 以乘子 λ_2 强制该约束, 考虑最小化

$$\tilde{J} = \boldsymbol{u}_2^{\mathsf{T}} \boldsymbol{S} \boldsymbol{u}_2 + \lambda_2 \left(1 - \boldsymbol{u}_2^{\mathsf{T}} \boldsymbol{u}_2 \right). \tag{16.15}$$

将 $\tilde{J}$ 关于 $\boldsymbol{u}_2$ 的导数设为零, 得到 $\boldsymbol{S}\boldsymbol{u}_2 = \lambda_2 \boldsymbol{u}_2$, 即 $\boldsymbol{u}_2$ 是 $\boldsymbol{S}$ 的一个特征向量, 具有特征值 λ_2. 因此, 任何特征向量都将定义误差度量的驻点. 要找到 J 在最小值点的值, 可以将方程对 $\boldsymbol{u}_2$ 的解回代到误差度量中, 得到 $J = \lambda_2$. 因此, 我们可以通过选择 $\boldsymbol{u}_2$ 为对应于两个特征值中较小的那个特征向量来获得 J 的最小值. 我们应该选择让主成分子空间与具有较大特征值的特征向量对齐. 这个结果符合我们的直觉, 即为了最小化平均平方投影距离, 应该选择主成分子空间, 使其通过数据点的平均值并与最大方差的方向对齐. 只要特征值相等, 任何选择的主方向都将产生相同的 J 值.

注

(重构新增) 这个二维例子把一般情形中的两个量分离清楚了:J 的驻点条件与最大方差表述的驻点条件是同一个特征方程, 区别只在于目标函数是 $\boldsymbol{u}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}$ 还是它的相反数, 因此驻点集合相同, 而“取最大”与“取最小”的差别就体现为选最大还是最小的特征值. 这正是两种表述等价的直观来源. 当两个特征值相等时,$\boldsymbol{S}$ 是单位矩阵的倍数, 所有方向的投影误差都相同,J 与方向无关, 这与“任何选择的主方向都产生相同的 J”一致.

误差度量 J 作为纯粹依赖于 $\{\boldsymbol{u}_i\}$ 的函数的表达式如下:

$$J = \frac{1}{N}\sum_{n=1}^{N}\sum_{i=M+1}^{D}\left(\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i - \bar{\boldsymbol{x}}^{\mathsf{T}}\boldsymbol{u}_i\right)^2 = \sum_{i=M+1}^{D}\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_i. \tag{16.16}$$

剩下的任务是调整 $\{\boldsymbol{u}_i\}$ 从而最小化 J, 这也必须是一个受约束的最小化操作, 否则我们将得到毫无意义的结果 $\boldsymbol{u}_i = \boldsymbol{0}$. 约束来源于正交条件, 正如我们将看到的, 解将以协方差矩阵的特征向量的形式表示出来.

针对任意 D 和任意 $M < D$ 最小化 J 的一般解, 均可以选择 $\boldsymbol{u}_i$ 为协方差矩阵的特征向量 (见练习 16.2), 由下式给出:

$$\boldsymbol{S}\boldsymbol{u}_i = \lambda_i\boldsymbol{u}_i, \tag{16.17}$$

其中 $i = 1, \cdots, D$, 并且像往常一样, 特征向量 $\boldsymbol{u}_i$ 选择为正交向量. 然后, 误差度量的相应值由下式给出:

$$J = \sum_{i=M+1}^{D}\lambda_i. \tag{16.18}$$

命题 16.2 (最小误差表述与最大方差表述的等价性)

设 $\boldsymbol{S}$ 为 $D\times D$ 对称半正定矩阵, 特征值按 $\lambda_1 \geqslant \cdots \geqslant \lambda_D \geqslant 0$ 排列. 在 $\tilde{\boldsymbol{x}}_n$ 由(16.10)给出、$\{\boldsymbol{u}_i\}$ 满足(16.7)的条件下, 误差度量 J 的最小值为 $\sum_{i=M+1}^{D}\lambda_i$, 且最小值在 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M$ 张成前 M 个特征向量张成的子空间时取到; 换言之, 最小误差表述与命题 16.1的最大方差表述给出同一个主子空间.

证明

(重构新增) 第一步, 关于系数的最优化. 把(16.10)代入(16.11), 并利用(16.7)展开范数:

$$J=\frac{1}{N}\sum_{n=1}^{N}\left\{\sum_{i=1}^{M}\left(z_{ni}-\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i\right)^2+\sum_{i=M+1}^{D}\left(b_i-\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i\right)^2\right\}.$$

右端关于 z_{ni} 与 b_i 分别是独立的二次函数, 逐项配方即知最小值在 $z_{ni}=\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i$ 与 $b_i=\dfrac{1}{N}\sum_n\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i=\bar{\boldsymbol{x}}^{\mathsf{T}}\boldsymbol{u}_i$ 处取到, 这正是(16.12)与(16.13); 此时每个平方项为零,J 只剩下被丢弃的分量, 即(16.14)与(16.16).

第二步, 关于基的最优化. 由(16.16), 记 $\tilde{\boldsymbol{U}}$ 为以 $\boldsymbol{u}_{M+1},\cdots,\boldsymbol{u}_D$ 为列的 $D\times(D-M)$ 矩阵, 则 $J=\mathrm{tr}(\tilde{\boldsymbol{U}}^{\mathsf{T}}\boldsymbol{S}\tilde{\boldsymbol{U}})$, 约束为 $\tilde{\boldsymbol{U}}^{\mathsf{T}}\tilde{\boldsymbol{U}}=\boldsymbol{I}$(即(16.7)中 $i,j>M$ 的部分). 引入对称乘子矩阵 $\boldsymbol{H}$, 考虑

$$\tilde{J}=\mathrm{tr}\left[\tilde{\boldsymbol{U}}^{\mathsf{T}}\boldsymbol{S}\tilde{\boldsymbol{U}}\right]+\mathrm{tr}\left[\boldsymbol{H}\left(\boldsymbol{I}-\tilde{\boldsymbol{U}}^{\mathsf{T}}\tilde{\boldsymbol{U}}\right)\right]. \tag{16.19}$$

(约束矩阵 $\boldsymbol{I}-\tilde{\boldsymbol{U}}^{\mathsf{T}}\tilde{\boldsymbol{U}}$ 对称, 故只需 $\boldsymbol{H}$ 的对称部分, 可设 $\boldsymbol{H}$ 对称.) 关于 $\tilde{\boldsymbol{U}}$ 求导并令其为零, 得

$$\boldsymbol{S}\tilde{\boldsymbol{U}}=\tilde{\boldsymbol{U}}\boldsymbol{H}.$$

由于 $\boldsymbol{H}$ 对称, 存在正交矩阵 $\boldsymbol{V}$ 使 $\boldsymbol{V}^{\mathsf{T}}\boldsymbol{H}\boldsymbol{V}=\boldsymbol{\Lambda}'$ 为对角矩阵. 右乘 $\boldsymbol{V}$ 得 $\boldsymbol{S}(\tilde{\boldsymbol{U}}\boldsymbol{V})=(\tilde{\boldsymbol{U}}\boldsymbol{V})\boldsymbol{\Lambda}'$. 又 $\tilde{\boldsymbol{U}}\boldsymbol{V}$ 的列仍两两正交且为单位向量, 故不妨设 $\tilde{\boldsymbol{U}}$ 的列就是 $\boldsymbol{S}$ 的特征向量; 此时 $\boldsymbol{H}=\tilde{\boldsymbol{U}}^{\mathsf{T}}\boldsymbol{S}\tilde{\boldsymbol{U}}=\boldsymbol{\Lambda}'$,$J=\mathrm{tr}(\boldsymbol{H})$ 等于所选 $D-M$ 个特征值之和.

第三步, 取最小的 $D-M$ 个特征值 $\lambda_{M+1},\cdots,\lambda_D$ 即得 $J=\sum_{i=M+1}^{D}\lambda_i$, 这就是 J 的最小值; 相应地, 主子空间由剩下的 M 个特征向量 $\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M$ 张成, 即前 M 个最大特征值对应的特征向量张成的子空间. 把它与命题 16.1对照: 那里最大化的正是 $\sum_{i=1}^{M}\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_i$, 而这里 $J=\sum_{i=M+1}^{D}\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_i=\mathrm{tr}(\boldsymbol{S})-\sum_{i=1}^{M}\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_i$(因为 $\{\boldsymbol{u}_i\}$ 是完备正交基,$\mathrm{tr}(\boldsymbol{S})=\sum_{i=1}^{D}\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_i$). 由于 $\mathrm{tr}(\boldsymbol{S})$ 与基的选取无关, 最小化 J 与最大化前 M 项之和是同一个问题, 两者的最优子空间相同.

□

注

(重构新增) 本命题说明 PCA 的两种经典表述——Hotelling 的最大方差表述与 Pearson 的最小误差表述——在最优化意义下完全等价, 因此可以按计算上的方便任选其一. 证明中真正起作用的结构是:$\{\boldsymbol{u}_i\}$ 是完备正交基, 于是迹可分解为两部分, 而被丢弃的那部分恰是 J. 这条恒等式 $\mathrm{tr}(\boldsymbol{S})=\sum_i\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{S}\boldsymbol{u}_i$ 依赖正交归一条件(16.7); 若基不正交, 交叉项不会消失,J 就不再等于 $\mathrm{tr}(\boldsymbol{S})$ 减去保留部分, 两种表述的等价性也随之失效.

需要强调的是,“J 的最小值为 $\sum_{i=M+1}^{D}\lambda_i$”这一结论中特征值必须按降序排列; 若把最小的特征值排在前面, 公式本身仍然成立, 但“主子空间由前 M 个特征向量张成”的说法就要改成“由最大的 M 个特征值对应的特征向量张成”. 此外,λ_i 是投影误差的逐方向贡献, 因此(16.18)也给出了选择 M 的定量依据: 当 $\lambda_{M+1},\cdots,\lambda_D$ 相对于 $\lambda_1,\cdots,\lambda_M$ 很小时, 取 M 维主子空间的误差可以忽略.

这仅仅是正交于主子空间的那些特征向量的特征值之和. 因此, 我们通过选择这些特征向量为具有 $D-M$ 个最小特征值的那些特征向量, 得到了 J 的最小值, 定义主子空间的特征向量是对应于 M 个最大特征值的那些特征向量. 这个结果符合我们的直觉: 为了最小化平均平方投影距离, 应该选择主成分子空间, 使其通过数据点的平均值并与最大方差的方向对齐.

虽然我们仅考虑了 $M<D$ 的情况, 但如果 $M=D$,PCA 仍然成立, 在这种情况下没有降维, 只是简单地旋转坐标轴以与主成分对齐.

最后请注意, 有一种相关的线性降维技术称为典型相关分析 (Canonical Correlation Analysis, CCA)(Hotelling, 1936; Bach and Jordan, 2002).PCA 只处理单个随机变量, 而典型相关分析则考虑多个变量, 并尝试找到具有高度互相关性的对应线性子空间对, 以使一个子空间内的分量与另一个子空间内的分量具有相关性. 它的解可以用广义特征向量问题来表示.

16.1.3 数据压缩

PCA 的另一个应用是数据压缩, 我们用手写数字图像数据集来举例. 因为协方差矩阵的每个特征向量都是原始 D 维空间中的向量, 所以我们可以将特征向量表示成与数据点大小相同的图像. 平均向量和前 4 个 PCA 特征向量及对应的特征值如图 16.3所示.

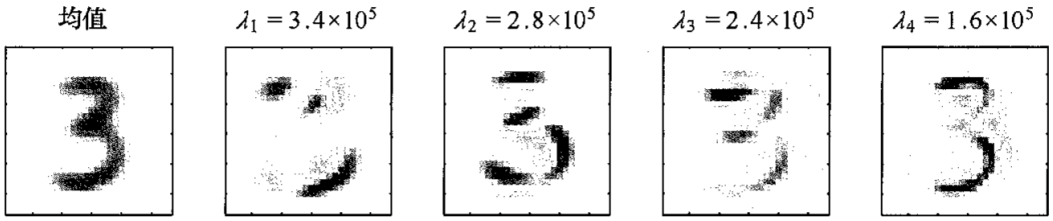


图 16.3: 将 PCA 应用于一个由 6000 个 28×28 像素的手写数字 “3” 的图像数据集, 这里展现了平均向量 $\bar{\boldsymbol{x}}$ 和前 4 个 PCA 特征向量 $\boldsymbol{u}_1,\cdots,\boldsymbol{u}_4$ 及对应的特征值

完整特征值谱的排序递减图如图 16.4中的 (a) 所示. 与选择特定的 M 值相关联的误差度量 J 由第 $M+1$ 到第 D 个特征值的和给出, 并且不同 M 值的情况已绘制在同一图的 (b) 中.

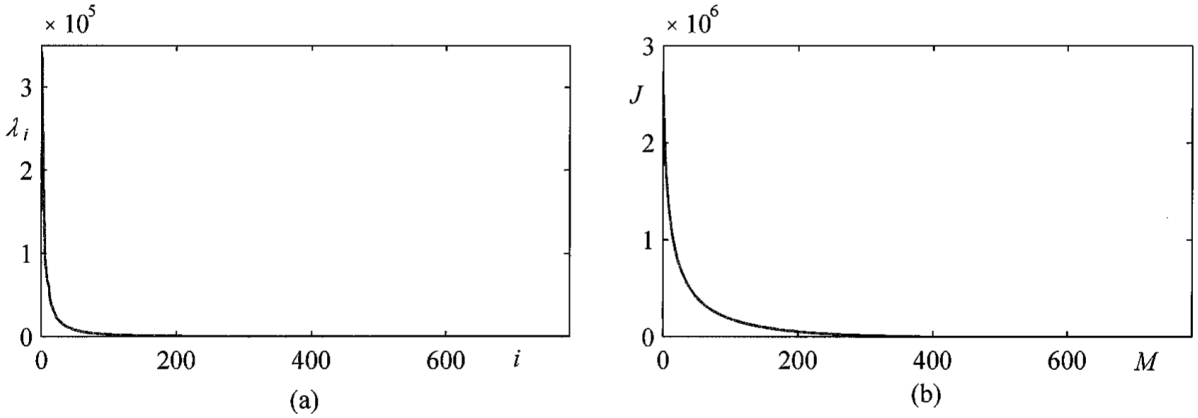


图 16.4: (a) 图 16.3中使用的手写数字图像数据集的特征值谱图;(b) 被抛弃特征值的和代表了通过将数据投影到维度为 M 的主子空间而引入的平方和误差 J

如果我们将在(16.12)和(16.13)代入(16.10), 就可以将 PCA 近似于数据向量 $\boldsymbol{x}_n$ 的形式写为

$$\begin{aligned}\tilde{\boldsymbol{x}}_n &= \sum_{i=1}^{M}\left(\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i\right)\boldsymbol{u}_i + \sum_{i=M+1}^{D}\left(\bar{\boldsymbol{x}}^{\mathsf{T}}\boldsymbol{u}_i\right)\boldsymbol{u}_i \\ &= \bar{\boldsymbol{x}} + \sum_{i=1}^{M}\left(\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i - \bar{\boldsymbol{x}}^{\mathsf{T}}\boldsymbol{u}_i\right)\boldsymbol{u}_i,\end{aligned} \tag{16.20}$$

其中我们利用了如下关系:

$$\bar{\boldsymbol{x}} = \sum_{i=1}^{D}\left(\bar{\boldsymbol{x}}^{\mathsf{T}}\boldsymbol{u}_i\right)\boldsymbol{u}_i. \tag{16.21}$$

式(16.21)成立是因为 $\{\boldsymbol{u}_i\}$ 的完备性. 这代表了数据集的压缩, 因为对于每个数据点, 我们都会将 D 维向量 $\boldsymbol{x}_n$ 替换为具有分量 $(\boldsymbol{x}_n^{\mathsf{T}}\boldsymbol{u}_i - \bar{\boldsymbol{x}}^{\mathsf{T}}\boldsymbol{u}_i)$ 的 M 维向量.M 值越小, 压缩程度越大. 手写数字图像数据集的 PCA 重建数据点示例见图 16.5.

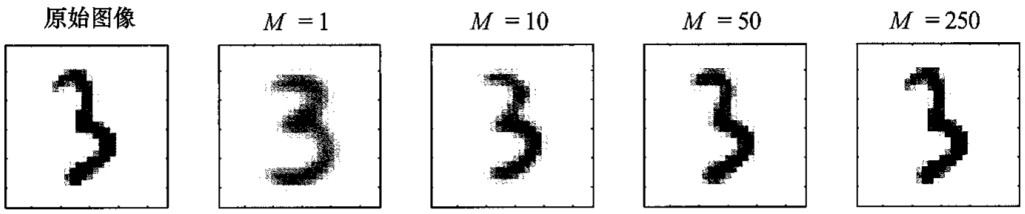


图 16.5: 手写数字图像数据集中的一个手写数字及其通过保留 M 个主成分而获得的 PCA 重构结果. 随着 M 值的增加, 重构变得更准确, 当 $M = D = 28 \times 28 = 784$ 时, 重构将会是完美的

注

(重构新增) 压缩与重建是同一件事的两种说法: 编码器把 $\boldsymbol{x}_n$ 映射为 M 维码字 $\boldsymbol{z}_n = (\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\boldsymbol{U}_M$, 解码器用(16.20)把它映回 $\tilde{\boldsymbol{x}}_n$. 由于 $\boldsymbol{U}_M$ 的列正交归一, 编码器是线性映射且不放大误差; 又由(16.18), 重建的均方误差恰是被丢弃特征值之和, 与码字分量无关. 因此"保留 M 个主成分"就是在给定码长的前提下使均方误差最小, 而误差随 M 的下降速度完全由特征值谱决定: 谱衰减得越快, 用很小的 M 就能达到很小的误差,图 16.4给出的正是这一衰减曲线.

16.1.4 数据白化

PCA 的另一个应用是数据预处理, 其目标不是降低维度, 而是转换数据集以标准化某些属性. 变化后的数据集能更好地为机器学习算法服务. 通常, 当原始变量以不同的单位测量或具有显著不同的可变性时, 就需要进行此类处理. 例如, 在老忠实喷泉数据集中, 两次喷发之间的间隔时间通常比一次喷发的持续时间长一个数量级(见节 15.1). 将 K 均值算法应用于这个数据集时, 我们首先对各个变量进行了单独的线性重缩放, 使得每个变量都具有零均值和单位方差. 这称为标准化数据, 标准化数据的协方差矩阵由以下元素构成:

$$\rho_{ij} = \frac{1}{N}\sum_{n=1}^{N}\frac{(x_{ni} - \bar{x}_i)}{\sigma_i}\frac{(x_{nj} - \bar{x}_j)}{\sigma_j}, \tag{16.22}$$

其中 σ_i 是 x_i 的标准差. 原始数据的相关性矩阵的特点是, 如果数据的两个分量 x_i 和 x_j 完全相关, 则 $\rho_{ij} = 1$; 如果它们不相关, 则 $\rho_{ij} = 0$.

利用 PCA, 可以对数据进行更实质性的标准化, 使其具有零均值和单位协方差, 从而使不同的变量变得不相关. 为此, 首先将特征向量方程 [(16.17)] 写成以下形式:

$$\boldsymbol{SU} = \boldsymbol{UL}, \tag{16.23}$$

其中 $\boldsymbol{L}$ 是一个 $D \times D$ 的对角矩阵, 矩阵元素为 λ_i;$\boldsymbol{U}$ 是一个 $D \times D$ 的正交矩阵, 其列由 $\boldsymbol{u}_i$ 给出. 然后我们定义, 对于每个数据点 $\boldsymbol{x}_n$, 转换后的值由下式给出:

$$\boldsymbol{y}_n = \boldsymbol{L}^{-1/2}\boldsymbol{U}^{\mathsf{T}}(\boldsymbol{x}_n - \bar{\boldsymbol{x}}), \tag{16.24}$$

其中 $\bar{\boldsymbol{x}}$ 是由(16.1)定义的样本均值. 显然, 集合 $\{\boldsymbol{y}_n\}$ 具有零均值, 并且其协方差由单位矩阵给出, 因为

$$\begin{aligned}\frac{1}{N}\sum_{n=1}^{N}\boldsymbol{y}_n\boldsymbol{y}_n^{\mathsf{T}} &= \frac{1}{N}\sum_{n=1}^{N}\boldsymbol{L}^{-1/2}\boldsymbol{U}^{\mathsf{T}}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}\boldsymbol{U}\boldsymbol{L}^{-1/2} \\ &= \boldsymbol{L}^{-1/2}\boldsymbol{U}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{U}\boldsymbol{L}^{-1/2} = \boldsymbol{L}^{-1/2}\boldsymbol{L}\boldsymbol{L}^{-1/2} = \boldsymbol{I}.\end{aligned} \tag{16.25}$$

定义 16.2 (白化)

设 $\boldsymbol{S} = \boldsymbol{U}\boldsymbol{L}\boldsymbol{U}^{\mathsf{T}}$ 为数据协方差矩阵的特征分解, 其中 $\boldsymbol{U}$ 正交、$\boldsymbol{L}$ 为对角矩阵且对角元 $\lambda_i > 0$. 称线性变换

$$\boldsymbol{x} \mapsto \boldsymbol{y} = \boldsymbol{L}^{-1/2}\boldsymbol{U}^{\mathsf{T}}(\boldsymbol{x} - \bar{\boldsymbol{x}})$$

为白化 (whitening), 也称球化 (sphering); 变换后的数据集 $\{\boldsymbol{y}_n\}$ 称为白化数据. ♣

注

(重构新增) 白化把“标准化”从逐坐标的缩放推广为整体的线性变换. 逐坐标标准化只令每个分量的方差为 1, 并不消除分量之间的相关性 ((16.22)中的非对角元一般不为零); 白化则同时做到零均值、单位协方差, 即各分量不相关且方差相同, 因此数据的协方差形状被“揉成”一个球.

定义要求所有 $\lambda_i > 0$. 若某个 $\lambda_i = 0$,$\boldsymbol{L}^{-1/2}$ 不存在, 说明数据落在低维子空间里, 此时只能在非零特征值对应的方向上白化, 其余方向应删去 (降维), 这正是小节 16.1.5讨论的情形. 白化的代价是它依赖 $\boldsymbol{S}$ 的完整特征分解, 复杂度为 $O(D^3)$; 当 D 很大时通常先降维再白化.

这个操作称为白化 (whitening) 或球化 (sphering) 数据,图 16.6展示了对老忠实喷泉数据集 (见节 15.1) 应用线性预处理的效果.

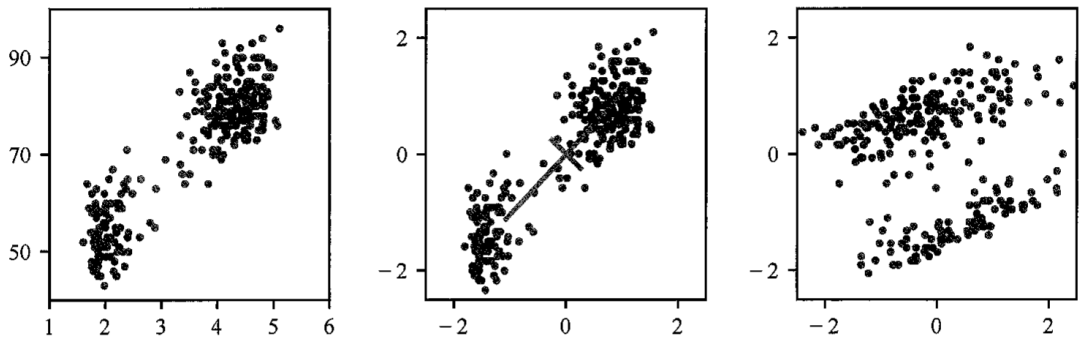


图 16.6: 对老忠实喷泉数据集应用线性预处理的效果. 左图显示了原始数据. 中图显示了将各个变量标准化为零均值和单位方差后的结果, 同时显示了这个规范化数据集的主轴, 它们绘制在范围 $\pm\lambda_i^{1/2}$ 内. 右图显示了白化数据并得到零均值和单位协方差后的结果

16.1.5 高维数据

在 PCA 的某些应用中, 数据点的数量小于数据空间的维度. 例如, 我们可能想对一个有几百张图像的数据集应用 PCA, 其中的每个图像对应于潜在的几百万维空间中的一个向量 (对应于图像中每个像素的三个颜色值). 注意, 在 D 维空间中,N 个点的集合 ($N < D$) 定义了一个线性子空间, 其维度最多为 $N-1$, 因此当 M 的值大于 $N-1$ 时, 应用 PCA 没有多大意义. 实际上, 如果我们执行 PCA, 就会发现至少有 $D-N+1$ 个特征值是零, 在这些特征值对应的特征向量方向上, 数据的方差为零.

此外, 寻找 $D \times D$ 矩阵的特征向量的典型算法的计算成本达到了 $O(D^3)$ 的规模, 因此对图像直接应用 PCA 在计算上是不可行的.

可以按照以下方式来解决这个问题. 首先将 $\boldsymbol{X}$ 定义为 $N \times D$ 维的中心化数据矩阵, 其第 n 行由 $(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^{\mathsf{T}}$ 给出. 然后可以将协方差矩阵 [(16.3)] 写作 $\boldsymbol{S} = N^{-1}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}$, 相应的特征向量方程变为

$$\frac{1}{N}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i. \tag{16.26}$$

对(16.26)的两边同时乘以 $\boldsymbol{X}$, 得到

$$\frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}(\boldsymbol{X}\boldsymbol{u}_i) = \lambda_i (\boldsymbol{X}\boldsymbol{u}_i). \tag{16.27}$$

如果定义 $\boldsymbol{v}_i = \boldsymbol{X}\boldsymbol{u}_i$, 则可以得到

$$\frac{1}{N}\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i. \tag{16.28}$$

这是 $N \times N$ 矩阵 $N^{-1}\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}$ 的特征向量方程, 它与原始协方差矩阵具有相同的 $N-1$ 个特征值 (它本身还有额外的 $D-N+1$ 个特征值为零). 因此, 我们可以在维度较低的空间中解决特征向量问题, 计算成本为 $O(N^3)$ 而不是 $O(D^3)$. 要确定特征向量, 我们可以对(16.28)的两边乘以 $\boldsymbol{X}^{\mathsf{T}}$, 得到

$$\left(\frac{1}{N}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}\right)\left(\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i\right) = \lambda_i \left(\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i\right). \tag{16.29}$$

从中可以看出 $(\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i)$ 是具有特征值 λ_i 的 $\boldsymbol{S}$ 的特征向量. 但请注意, 这些特征向量未必是归一化的. 为了对它们进行适当的归一化, 重新调整 $\boldsymbol{u}_i \propto \boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i$, 使得 $\|\boldsymbol{u}_i\| = 1$(见练习 16.3), 并且假设 $\boldsymbol{v}_i$ 已经归一化为单位长度, 同时有

$$\boldsymbol{u}_i = \frac{1}{(N\lambda_i)^{1/2}}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i. \tag{16.30}$$

总之, 要应用这种方法, 我们需要首先计算 $\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}$, 然后找到它的特征向量和特征值, 最后使用(16.30)在原始数据空间中计算特征向量.

注

(重构新增) 这一小节要解决的问题是“样本数少于维数”时 PCA 的可行性, 包含两层困难. 其一是统计上的:N 个数据点至多张成 $N-1$ 维的子空间, 因此当 $M \geqslant N$ 时, 超出 $N-1$ 的那些“主成分”对应的方差恒为零, 继续增加主成分数没有意义; 这解释了为什么要求 $M \leqslant N-1$. 其二是计算上的:$\boldsymbol{S}$ 是 $D \times D$ 矩阵, 当 D 达到百万量级时既无法存下也无法分解.

(16.27)—(16.28)给出的对偶技巧把问题从 $D \times D$ 转到 $N \times N$:$\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}$ 与 $\boldsymbol{X}^{\mathsf{T}}\boldsymbol{X}$ 的非零特征值完全相同 (这是 $\boldsymbol{A}\boldsymbol{B}$ 与 $\boldsymbol{B}\boldsymbol{A}$ 的非零谱相同这一事实的特例), 因此只需对 $N \times N$ 矩阵作特征分解, 代价从 $O(D^3)$ 降到 $O(N^3)$.(16.30)中的因子 $(N\lambda_i)^{-1/2}$ 来自 $\|\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i\|^2 = \boldsymbol{v}_i^{\mathsf{T}}\boldsymbol{X}\boldsymbol{X}^{\mathsf{T}}\boldsymbol{v}_i = N\lambda_i\boldsymbol{v}_i^{\mathsf{T}}\boldsymbol{v}_i = N\lambda_i$, 这一步用了 $\boldsymbol{v}_i$ 已是单位向量以及(16.28); 若 $\lambda_i = 0$ 则该方向无从归一化, 这与“零方差方向”是同一件事.

16.2 概率潜变量

我们在节 16.1中已经看到 PCA 可以定义为数据在低维子空间上的线性投影, 这个低维子空间的维度比原始数据空间的维度小. 每个数据点被映射到由(16.12)定义的唯一的量 z_{ni}, 我们可以将这些量视为确定性的潜变量. 为了引入和论证连续概率潜变量, 本节会展示,PCA 也可以表示为概率潜变量模型的最大似然解. 这种 PCA 的重新表述, 称为概率 PCA, 与传统 PCA 相比, 有如下优势.

- 概率 PCA 模型表示了 Gauss 分布的一种约束形式, 在仍然能够捕捉数据集中主要相关性的同时, 还可以限制自由参数的数量.
- 我们可以为 PCA 推导出一种 EM 算法, 该算法在只需要少数几个主特征向量的情况下计算效率很高, 而且能避免将计算数据协方差矩阵作为中间步骤进行计算 (见小节 16.3.2).
- 概率 PCA 模型与 EM 算法的结合使我们能够处理数据集中缺失的值.
- 可以原则性地构造概率 PCA 模型的混合模型并使用 EM 算法进行训练.
- 似然函数的存在使得我们可以与其他概率密度模型直接进行比较. 相比之下, 传统 PCA 模型会给予靠近主成分子空间 (也可简称主子空间) 且距离训练数据任意远的数据点一个较低的重建成本.
- 概率 PCA 可用于建模类-条件密度, 因此可应用于分类问题.
- 概率 PCA 模型可以生成性地运行以提供来自分布的样本.
- 概率 PCA 构成了基于 Bayes 方法处理 PCA 的基础, 我们可以从数据中自动找到主成分子空间的维度 (Bishop, 2006).

将 PCA 作为概率模型的这种表述是由 Tipping and Bishop (1997; 1999) 以及 Roweis (1998) 独立提出的. 正如我们稍后将看到的, 它与因子分析 (factor analysis)(Basilevsky, 1994) 密切相关.

16.2.1 生成式模型

概率 PCA 是线性-Gauss 框架的一个简单例子, 其中所有的边缘分布和条件分布都是 Gauss 分布. 我们可以首先通过引入对应于主成分子空间的、明确的 M 维潜变量 $\boldsymbol{z}$ 来构建概率 PCA. 接下来, 我们定义一个关于潜变量的 Gauss 先验分布 $p(\boldsymbol{z})$, 以及基于潜变量值的、D 维观测变量 $\boldsymbol{x}$ 的 Gauss 条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$. 具体来说, 关于 $\boldsymbol{z}$ 的先验分布由零均值、单位协方差的 Gauss 分布给出:

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I}). \tag{16.31}$$

同样, 观测变量 $\boldsymbol{x}$ 的条件分布, 基于潜变量 $\boldsymbol{z}$ 的值, 也符合 Gauss 分布:

$$p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \sigma^2\boldsymbol{I}). \tag{16.32}$$

把上述模型的形式正式表述如下.

定义 16.3 (概率 PCA)

设 $\boldsymbol{z}$ 是 M 维连续潜变量,$\boldsymbol{x}$ 是 D 维观测变量. 若它们的联合分布由

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I}), \qquad p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \sigma^2\boldsymbol{I})$$

给出, 其中 $\boldsymbol{W}$ 是 $D \times M$ 矩阵、$\boldsymbol{\mu}$ 是 D 维向量、$\sigma^2 > 0$ 是控制条件分布方差的标量, 则称 $(\boldsymbol{x}, \boldsymbol{z})$ 服从概率 PCA(probabilistic PCA, PPCA) 模型, 称 $\boldsymbol{W}$ 的列所跨越的数据空间中的线性子空间为该模型的子空间. ♣

注

(重构新增) 概率 PCA 的动机是给节 16.1中纯代数的 PCA 补上一个概率生成机制, 从而可以用最大似然而不是“方差最大”这类几何准则来确定参数. 生成图像的过程可以这样读: 先从 $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ 抽取一个低维码字 $\boldsymbol{z}$, 用 $\boldsymbol{W}$ 把它线性地搬到 D 维空间中, 再加上均值为 $\boldsymbol{\mu}$ 的位置偏移, 最后叠加各向同性的 Gauss 噪声 $\mathcal{N}(\boldsymbol{0}, \sigma^2\boldsymbol{I})$. 因此 $\boldsymbol{W}$ 的列张成的正是数据的“理想”低维子空间, 而 σ^2 度量数据偏离该子空间的程度.

条件的含义需要逐条说明. 第一,$\boldsymbol{z}$ 的先验取零均值、单位协方差, 这不是实质限制: 若先验是 $\mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$, 重新定义 $\boldsymbol{W}' = \boldsymbol{W}\boldsymbol{L}$、$\boldsymbol{\mu}' = \boldsymbol{\mu} + \boldsymbol{W}\boldsymbol{m}$(其中 $\boldsymbol{L}\boldsymbol{L}^{\mathrm{T}} = \boldsymbol{\Sigma}$) 即可把模型化为本定义的形式, 观测分布完全不变 (见练习 16.4). 第二, 条件协方差取 $\sigma^2\boldsymbol{I}$ 而不是一般的正定矩阵, 正是概率 PCA 区别于因子分析的地方, 它使 $\boldsymbol{x}$ 的各分量在给定 $\boldsymbol{z}$ 时条件独立, 并且使模型对潜空间的旋转不变 (见命题 16.4). 第三, 要求 $\sigma^2 > 0$: 若 $\sigma^2 = 0$,$\boldsymbol{x}$ 将以概率 1 落在 $\boldsymbol{W}$ 的列空间这个零测集上, 边缘密度退化而不再有密度, 最大似然问题也没有意义.

其中 $\boldsymbol{x}$ 的均值是 $\boldsymbol{z}$ 的一般线性函数, 受 $D \times M$ 维矩阵 $\boldsymbol{W}$ 和 D 维向量 $\boldsymbol{\mu}$ 的控制. 注意, 这关于 $\boldsymbol{x}$ 的元素是因子化的 (见小节 11.2.4). 换句话说, 这是一个朴素 Bayes 模型的例子. 正如我们稍后将看到的,$\boldsymbol{W}$ 的列跨越了数据空间内的线性子空间, 对应于主子空间. 这个模型中的另一个参数是控制条件分布方差的标量 σ^2. 请注意, 潜分布 $p(\boldsymbol{z})$ 是零均值、单位协方差的 Gauss 分布, 这一假设不失一般性, 因为更普遍的 Gauss 分布也会产生一个等价的概率模型 (见练习 16.4).

我们可以从生成性视角来看概率 PCA 模型. 首先选择一个潜变量, 然后基于这个潜变量的值采样观测变量, 得到观测变量的样本值. 具体来说,D 维观测变量 $\boldsymbol{x}$ 是通过为 M 维潜变量 $\boldsymbol{z}$ 加上加性 Gauss 噪声的线性变换来定义的, 所以有

$$\boldsymbol{x} = \boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}, \tag{16.33}$$

其中 $\boldsymbol{z}$ 是一个 M 维的 Gauss 潜变量,$\boldsymbol{\varepsilon}$ 是一个 D 维的零均值 Gauss 分布噪声变量, 协方差为 $\sigma^2\boldsymbol{I}$. 这个生成过程如图 16.7所示. 注意, 这个框架基于从潜空间到数据空间的映射, 而不是前面讨论的常规 PCA 视角. 稍后, 我们将利用 Bayes 定理得出从数据空间到潜空间的反向映射.

16.2.2 似然函数

假设我们希望通过最大似然法确定参数 $\boldsymbol{W}$、$\boldsymbol{\mu}$ 和 σ^2 的值. 为了写下似然函数, 我们需要观测变量的边缘分布 $p(\boldsymbol{x})$ 的表达式. 根据概率的加和法则与乘积法则, 这可以表示为

$$p(\boldsymbol{x}) = \int p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}. \tag{16.34}$$

命题 16.3 (概率 PCA 的边缘分布)

在定义 16.3的模型下, 观测变量 $\boldsymbol{x}$ 的边缘分布仍是 Gauss 分布:

$$p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{C}), \tag{16.35}$$

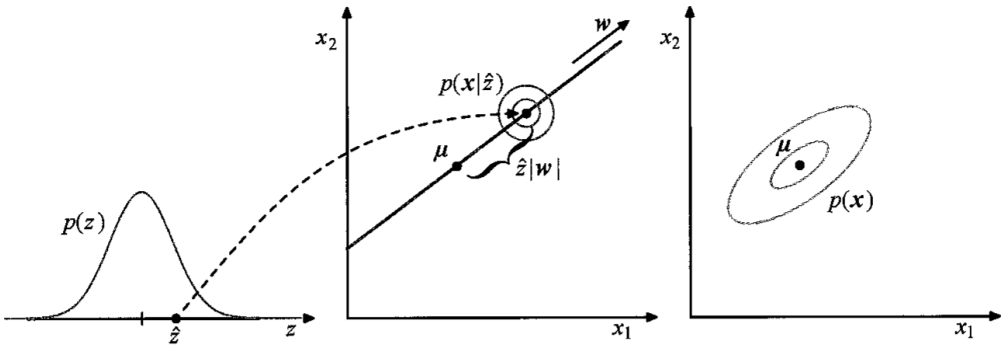


图 16.7: 从常规 PCA 视角看一个针对两维数据空间和一维潜空间的概率 PCA 模型的生成过程. 生成观测数据点 $\boldsymbol{x}$ 的过程是: 首先从其先验分布 $p(\boldsymbol{z})$ 中抽取潜变量的一个值 $\hat{\boldsymbol{z}}$, 然后从具有均值 $\boldsymbol{W}\hat{\boldsymbol{z}}+\boldsymbol{\mu}$ 和协方差 $\sigma^2\boldsymbol{I}$ 的各向同性 Gauss 分布 (见红色圆圈) 中抽取 $\boldsymbol{x}$ 的值. 绿色椭圆显示了边缘分布 $p(\boldsymbol{x})$ 的密度等高线

其中 $D\times D$ 的协方差矩阵 $\boldsymbol{C}$ 定义如下:

$$\boldsymbol{C}=\boldsymbol{W}\boldsymbol{W}^{\mathrm{T}}+\sigma^2\boldsymbol{I}. \tag{16.36}$$

证明

(重构新增) 把(16.32)与(16.31)代入(16.34), 并对指数配方. 记 $\boldsymbol{M}=\boldsymbol{W}^{\mathrm{T}}\boldsymbol{W}+\sigma^2\boldsymbol{I}$, 则被积函数为 $(2\pi\sigma^2)^{-D/2}(2\pi)^{-M/2}$ 乘以

$$\exp\left\{-\frac{1}{2\sigma^2}\|\boldsymbol{x}-\boldsymbol{\mu}-\boldsymbol{W}\boldsymbol{z}\|^2-\frac{1}{2}\boldsymbol{z}^{\mathrm{T}}\boldsymbol{z}\right\}.$$

把指数中 $\boldsymbol{z}$ 的二次型整理为

$$-\frac{1}{2}\boldsymbol{z}^{\mathrm{T}}\left(\frac{\boldsymbol{W}^{\mathrm{T}}\boldsymbol{W}}{\sigma^2}+\boldsymbol{I}\right)\boldsymbol{z}+\frac{1}{\sigma^2}\boldsymbol{z}^{\mathrm{T}}\boldsymbol{W}^{\mathrm{T}}(\boldsymbol{x}-\boldsymbol{\mu})-\frac{1}{2\sigma^2}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2=-\frac{1}{2\sigma^2}\boldsymbol{z}^{\mathrm{T}}\boldsymbol{M}\boldsymbol{z}+\frac{1}{\sigma^2}\boldsymbol{z}^{\mathrm{T}}\boldsymbol{W}^{\mathrm{T}}(\boldsymbol{x}-\boldsymbol{\mu})-\frac{1}{2\sigma^2}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2,$$

其中用到 $\boldsymbol{M}=\boldsymbol{W}^{\mathrm{T}}\boldsymbol{W}+\sigma^2\boldsymbol{I}=\sigma^2\left(\boldsymbol{W}^{\mathrm{T}}\boldsymbol{W}/\sigma^2+\boldsymbol{I}\right)$. 对 $\boldsymbol{z}$ 用标准 Gauss 积分

$$\int\exp\left\{-\frac{1}{2\sigma^2}\boldsymbol{z}^{\mathrm{T}}\boldsymbol{M}\boldsymbol{z}+\frac{1}{\sigma^2}\boldsymbol{z}^{\mathrm{T}}\boldsymbol{b}\right\}\mathrm{d}\boldsymbol{z}=(2\pi\sigma^2)^{M/2}|\boldsymbol{M}|^{-1/2}\exp\left\{\frac{1}{2\sigma^2}\boldsymbol{b}^{\mathrm{T}}\boldsymbol{M}^{-1}\boldsymbol{b}\right\},$$

取 $\boldsymbol{b}=\boldsymbol{W}^{\mathrm{T}}(\boldsymbol{x}-\boldsymbol{\mu})$, 得到 $p(\boldsymbol{x})$ 等于 $(2\pi)^{-D/2}(\sigma^2)^{-(D-M)/2}|\boldsymbol{M}|^{-1/2}$ 乘以

$$\exp\left\{-\frac{1}{2\sigma^2}\|\boldsymbol{x}-\boldsymbol{\mu}\|^2+\frac{1}{2\sigma^2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathrm{T}}\boldsymbol{W}\boldsymbol{M}^{-1}\boldsymbol{W}^{\mathrm{T}}(\boldsymbol{x}-\boldsymbol{\mu})\right\}.$$

再把指数写成单一二次型:

$$-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathrm{T}}\left[\frac{1}{\sigma^2}\boldsymbol{I}-\frac{1}{\sigma^2}\boldsymbol{W}\boldsymbol{M}^{-1}\boldsymbol{W}^{\mathrm{T}}\right](\boldsymbol{x}-\boldsymbol{\mu}).$$

方括号中的矩阵正是 $(\sigma^2\boldsymbol{I}+\boldsymbol{W}\boldsymbol{W}^{\mathrm{T}})^{-1}$: 这可以由矩阵求逆恒等式(A.5)取 $\boldsymbol{A}=\sigma^2\boldsymbol{I}$、$\boldsymbol{B}=\boldsymbol{W}$、$\boldsymbol{C}=\boldsymbol{I}$、$\boldsymbol{D}=\boldsymbol{W}^{\mathrm{T}}$ 得到, 于是指数中的矩阵就是 $\boldsymbol{C}^{-1}$. 最后核对归一化常数. 由行列式恒等式,

$$|\boldsymbol{C}|=\left|\sigma^2\boldsymbol{I}+\boldsymbol{W}\boldsymbol{W}^{\mathrm{T}}\right|=\sigma^{2D}\left|\boldsymbol{I}+\frac{\boldsymbol{W}^{\mathrm{T}}\boldsymbol{W}}{\sigma^2}\right|=\sigma^{2D}\cdot\sigma^{-2M}|\boldsymbol{M}|=\sigma^{2(D-M)}|\boldsymbol{M}|,$$

故 $(\sigma^2)^{-(D-M)/2}|\boldsymbol{M}|^{-1/2}=|\boldsymbol{C}|^{-1/2}$, 从而 $p(\boldsymbol{x})=(2\pi)^{-D/2}|\boldsymbol{C}|^{-1/2}\exp\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^{\mathrm{T}}\boldsymbol{C}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\}$, 即(16.35)与(16.36). □

注

(重构新增) 这个命题说明: 线性-Gauss 结构对边缘化是封闭的——把 Gauss 潜变量积掉之后仍然是 Gauss 分布, 且协方差是“信号项” $\boldsymbol{W}\boldsymbol{W}^{\mathsf{T}}$ 与“噪声项” $\sigma^2\boldsymbol{I}$ 之和. 这条可加性正是后面所有解释的基础.

条件的作用:$\sigma^2>0$ 保证 $\boldsymbol{C}$ 正定 ($\boldsymbol{W}\boldsymbol{W}^{\mathsf{T}}$ 半正定, 加上正定的 $\sigma^2\boldsymbol{I}$ 后严格正定), 因此 $|\boldsymbol{C}|>0$、$\boldsymbol{C}^{-1}$ 存在, 密度良定义; 若允许 $\sigma^2=0$ 且 $M<D$,$\boldsymbol{C}$ 奇异,$p(\boldsymbol{x})$ 退化到子空间上. 推导中唯二用到外部结果的地方是矩阵求逆恒等式与行列式恒等式, 两者都在附录 A中给出并证明, 因此本证明不依赖本章后面的内容.

注意预测分布也是 Gauss 分布, 因此通过使用(16.33)计算其均值和协方差就能更直接地得出同样的结果, 于是有

$$\mathbb{E}[\boldsymbol{x}]=\mathbb{E}[\boldsymbol{W}\boldsymbol{z}+\boldsymbol{\mu}+\boldsymbol{\varepsilon}]=\boldsymbol{\mu}, \tag{16.37}$$

$$\begin{aligned}\operatorname{cov}[\boldsymbol{x}]&=\mathbb{E}\left[(\boldsymbol{W}\boldsymbol{z}+\boldsymbol{\varepsilon})(\boldsymbol{W}\boldsymbol{z}+\boldsymbol{\varepsilon})^{\mathsf{T}}\right]\\&=\mathbb{E}\left[\boldsymbol{W}\boldsymbol{z}\boldsymbol{z}^{\mathsf{T}}\boldsymbol{W}^{\mathsf{T}}\right]+\mathbb{E}\left[\boldsymbol{\varepsilon}\boldsymbol{\varepsilon}^{\mathsf{T}}\right]\\&=\boldsymbol{W}\boldsymbol{W}^{\mathsf{T}}+\sigma^2\boldsymbol{I},\end{aligned} \tag{16.38}$$

这里我们利用了如下事实:$\boldsymbol{z}$ 和 $\boldsymbol{\varepsilon}$ 是独立随机变量, 因此它们不相关.

直观上, 我们可以认为分布 $p(\boldsymbol{x})$ 是通过取一个各向同性的 Gauss “喷雾罐”, 然后在主成分子空间上移动并喷洒以 σ^2 决定密度的 Gauss 墨水, 并通过先验分布来加权的. 累积的墨水密度将产生一个代表边缘密度 $p(\boldsymbol{x})$ 的“松饼”状分布.

16.2.3 最大似然法

预测分布 $p(\boldsymbol{x})$ 受参数 $\boldsymbol{\mu}$、$\boldsymbol{W}$ 和 σ^2 的控制. 然而, 这种参数化中存在冗余, 相当于潜空间坐标发生旋转. 为了说明这一点, 考虑一个矩阵 $\widetilde{\boldsymbol{W}}=\boldsymbol{W}\boldsymbol{R}$, 其中 $\boldsymbol{R}$ 是一个正交矩阵. 利用正交性质 $\boldsymbol{R}\boldsymbol{R}^{\mathsf{T}}=\boldsymbol{I}$, 我们看到出现在协方差矩阵 $\boldsymbol{C}$ 中的量 $\widetilde{\boldsymbol{W}}\widetilde{\boldsymbol{W}}^{\mathsf{T}}$ 具有以下形式:

$$\widetilde{\boldsymbol{W}}\widetilde{\boldsymbol{W}}^{\mathsf{T}}=\boldsymbol{W}\boldsymbol{R}\boldsymbol{R}^{\mathsf{T}}\boldsymbol{W}^{\mathsf{T}}=\boldsymbol{W}\boldsymbol{W}^{\mathsf{T}}. \tag{16.39}$$

正因为独立于 $\boldsymbol{R}$, 所以存在一整套矩阵 $\widetilde{\boldsymbol{W}}$, 所有这些矩阵都会导致相同的预测分布. 这种不变性可以理解为潜空间内的旋转. 稍后我们将返回讨论该模型中独立参数的数量问题.

当计算预测分布时, 我们需要 $\boldsymbol{C}^{-1}$, 它涉及计算一个 $D\times D$ 矩阵的逆 [参见(A.5)]:

$$\boldsymbol{C}^{-1}=\sigma^{-2}\boldsymbol{I}-\sigma^{-2}\boldsymbol{W}\boldsymbol{M}^{-1}\boldsymbol{W}^{\mathsf{T}}, \tag{16.40}$$

其中 $M\times M$ 矩阵 $\boldsymbol{M}$ 定义如下:

$$\boldsymbol{M}=\boldsymbol{W}^{\mathsf{T}}\boldsymbol{W}+\sigma^2\boldsymbol{I}. \tag{16.41}$$

因为我们求 $\boldsymbol{M}$ 的逆而不是直接求 $\boldsymbol{C}$ 的逆, 所以计算 $\boldsymbol{C}^{-1}$ 的成本从 $O(D^3)$ 降低到了 $O(M^3)$.

除了预测分布 $p(\boldsymbol{x})$, 我们还需要后验分布 $p(\boldsymbol{z}|\boldsymbol{x})$, 后者可以直接再次使用线性-Gauss 模型的结果 [命题 3.14] 写出:

$$p(\boldsymbol{z}|\boldsymbol{x})=\mathcal{N}\left(\boldsymbol{z}\,\middle|\,\boldsymbol{M}^{-1}\boldsymbol{W}^{\mathsf{T}}(\boldsymbol{x}-\boldsymbol{\mu}),\sigma^2\boldsymbol{M}^{-1}\right). \tag{16.42}$$

请注意, 后验分布的均值依赖于 $\boldsymbol{x}$, 而后验分布的协方差独立于 $\boldsymbol{x}$.

接下来考虑使用最大似然法确定模型参数. 给定一组观测数据点的数据集 $\boldsymbol{X}=\{\boldsymbol{x}_n\}$, 概率 PCA 模型可以表示为一个有向图, 如图 16.8所示. 相应的对数似然函数由(16.35)给出, 形式为

$$\begin{aligned}\ln p(\boldsymbol{X}|\boldsymbol{\mu},\boldsymbol{W},\sigma^2)&=\sum_{n=1}^{N}\ln p(\boldsymbol{x}_n|\boldsymbol{W},\boldsymbol{\mu},\sigma^2)\\&=-\frac{ND}{2}\ln(2\pi)-\frac{N}{2}\ln|\boldsymbol{C}|-\frac{1}{2}\sum_{n=1}^{N}(\boldsymbol{x}_n-\boldsymbol{\mu})^{\mathsf{T}}\boldsymbol{C}^{-1}(\boldsymbol{x}_n-\boldsymbol{\mu}).\end{aligned} \tag{16.43}$$

将对数似然函数关于 $\boldsymbol{\mu}$ 的导数设为零便可得到预期结果 $\boldsymbol{\mu}=\bar{\boldsymbol{x}}$, 其中 $\bar{\boldsymbol{x}}$ 是由(16.1)定义的数据均值. 由于对数

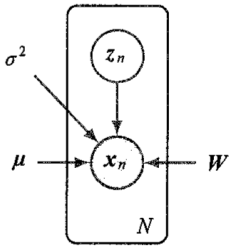


图 16.8: 对于包含 $\boldsymbol{x}$ 的 N 次观测的一个数据集, 概率 PCA 模型可以表示为一个有向图, 其中的每个观测值 $\boldsymbol{x}_n$ 与潜变量的一个值 $\boldsymbol{z}_n$ 相关联

似然函数是 $\boldsymbol{\mu}$ 的二次函数, 这个解代表了唯一的最大值, 可以通过计算二阶导数来验证 (见练习 16.9). 代入原式后, 我们可以将对数似然函数写成以下形式:

$$\ln p(\boldsymbol{X}|\boldsymbol{W},\boldsymbol{\mu},\sigma^2) = -\frac{N}{2}\left\{D\ln(2\pi) + \ln|\boldsymbol{C}| + \operatorname{tr}\left(\boldsymbol{C}^{-1}\boldsymbol{S}\right)\right\}, \tag{16.44}$$

其中 $\boldsymbol{S}$ 是由(16.3)定义的数据协方差矩阵.

虽然似然函数关于 $\boldsymbol{W}$ 和 σ^2 的最大化更为复杂, 但仍有精确的闭合解.Tipping and Bishop (1999) 指出, 对数似然函数的所有驻点都可以写成

$$\boldsymbol{W}_{\mathrm{ML}} = \boldsymbol{U}_M\left(\boldsymbol{L}_M - \sigma^2\boldsymbol{I}\right)^{1/2}\boldsymbol{R}, \tag{16.45}$$

其中 $\boldsymbol{U}_M$ 是一个 $D\times M$ 矩阵, 其列由数据协方差矩阵 $\boldsymbol{S}$ 的任意子集 (大小为 M) 的特征向量给出.$M\times M$ 对角矩阵 $\boldsymbol{L}_M$ 的元素由对应的特征值 λ_i 给出.$\boldsymbol{R}$ 是一个任意的 $M\times M$ 正交矩阵.

定理 16.1 (概率 PCA 的最大似然解)

设数据协方差矩阵 $\boldsymbol{S}$ 的特征值为 $\lambda_1 \geqslant \cdots \geqslant \lambda_D > 0$, 对应的单位特征向量为 $\boldsymbol{u}_1,\cdots,\boldsymbol{u}_D$. 对数似然函数(16.44)关于 $\boldsymbol{W}$ 与 σ^2 的所有驻点都由(16.45)与

$$\sigma^2_{\mathrm{ML}} = \frac{1}{D-M}\sum_{i=M+1}^{D}\lambda_i \tag{16.46}$$

给出, 其中 $\boldsymbol{U}_M$、$\boldsymbol{L}_M$ 取 $\boldsymbol{S}$ 的一个大小为 M 的特征向量子集及其特征值. 当且仅当该子集是前 M 个最大特征值时, 该驻点使对数似然函数达到最大值; 其余驻点 (原文称为鞍点) 的似然严格更小, 都不是最大值点. ♡

证明

(重构新增) 分三步.

第一步, 关于 $\boldsymbol{W}$ 的驻点方程. 由(16.44), 并利用 $\partial\ln|\boldsymbol{C}|/\partial\boldsymbol{W} = 2\boldsymbol{C}^{-1}\boldsymbol{W}$ 与 $\partial\operatorname{tr}(\boldsymbol{C}^{-1}\boldsymbol{S})/\partial\boldsymbol{W} = -2\boldsymbol{C}^{-1}\boldsymbol{S}\boldsymbol{C}^{-1}\boldsymbol{W}$(命题 A.17与命题 A.18), 得

$$\frac{\partial\ln p}{\partial\boldsymbol{W}} = -\frac{N}{2}\left(2\boldsymbol{C}^{-1}\boldsymbol{W} - 2\boldsymbol{C}^{-1}\boldsymbol{S}\boldsymbol{C}^{-1}\boldsymbol{W}\right) = N\boldsymbol{C}^{-1}\left(\boldsymbol{S}\boldsymbol{C}^{-1}\boldsymbol{W} - \boldsymbol{W}\right),$$

故驻点满足 $\boldsymbol{S}\boldsymbol{C}^{-1}\boldsymbol{W} = \boldsymbol{W}$.

第二步, 验证(16.45)满足该方程. 设 $\boldsymbol{S}\boldsymbol{U}_M = \boldsymbol{U}_M\boldsymbol{L}_M$,$\boldsymbol{U}_M^{\mathsf{T}}\boldsymbol{U}_M = \boldsymbol{I}$. 由 $\boldsymbol{C} = \boldsymbol{U}_M(\boldsymbol{L}_M - \sigma^2\boldsymbol{I})\boldsymbol{U}_M^{\mathsf{T}} + \sigma^2\boldsymbol{I}$(其中用到 $\boldsymbol{W}\boldsymbol{W}^{\mathsf{T}} = \boldsymbol{U}_M(\boldsymbol{L}_M - \sigma^2\boldsymbol{I})\boldsymbol{U}_M^{\mathsf{T}}$) 可知, 对 $\boldsymbol{U}_M$ 的每一列有 $\boldsymbol{C}\boldsymbol{u}_i = \lambda_i\boldsymbol{u}_i$, 而在 $\boldsymbol{U}_M$ 的正交补上 $\boldsymbol{C}$ 等于 $\sigma^2\boldsymbol{I}$. 于是

$$\boldsymbol{C}^{-1}\boldsymbol{W} = \boldsymbol{U}_M\boldsymbol{L}_M^{-1}\left(\boldsymbol{L}_M - \sigma^2\boldsymbol{I}\right)^{1/2}\boldsymbol{R},$$

从而

$$\boldsymbol{S}\boldsymbol{C}^{-1}\boldsymbol{W} = \boldsymbol{U}_M\boldsymbol{L}_M\boldsymbol{L}_M^{-1}\left(\boldsymbol{L}_M - \sigma^2\boldsymbol{I}\right)^{1/2}\boldsymbol{R} = \boldsymbol{U}_M\left(\boldsymbol{L}_M - \sigma^2\boldsymbol{I}\right)^{1/2}\boldsymbol{R} = \boldsymbol{W},$$

即驻点方程成立. 关于 $\boldsymbol{R}$ 的任意性见命题 16.4.

第三步, 关于 σ^2 的驻点方程与最大值的比较. 由 $\partial\ln|\boldsymbol{C}|/\partial\sigma^2 = \operatorname{tr}(\boldsymbol{C}^{-1})$ 与 $\partial\operatorname{tr}(\boldsymbol{C}^{-1}\boldsymbol{S})/\partial\sigma^2 = -\operatorname{tr}(\boldsymbol{C}^{-2}\boldsymbol{S})$, 驻点条件为 $\operatorname{tr}(\boldsymbol{C}^{-1}) = \operatorname{tr}(\boldsymbol{C}^{-2}\boldsymbol{S})$. 代入第二步的谱表示:$\operatorname{tr}(\boldsymbol{C}^{-1}) = \sum_{i\in J}\lambda_i^{-1} + (D-M)/\sigma^2$ 而 $\operatorname{tr}(\boldsymbol{C}^{-2}\boldsymbol{S}) = \sum_{i\in J}\lambda_i^{-1} + \sigma^{-4}\sum_{i\notin J}\lambda_i$,

其中 J 是所选取的特征向量下标集,$|J| = M$. 两式相等给出 $(D-M)/\sigma^2 = \sigma^{-4}\sum_{i\notin J}\lambda_i$, 即 $\sigma^2 = \dfrac{1}{D-M}\sum_{i\notin J}\lambda_i$, 这正是(16.46)(当 J 取前 M 个下标时).

把该 σ^2 代回(16.44), 并利用 $\mathrm{tr}(\boldsymbol{C}^{-1}\boldsymbol{S}) = M + (D-M) = D$, 得

$$\ln p(\boldsymbol{X}|\boldsymbol{W},\boldsymbol{\mu},\sigma^2) = -\frac{N}{2}\left\{D\ln(2\pi) + \sum_{i\in J}\ln\lambda_i + (D-M)\ln\bar{\lambda}_{J^c} + D\right\},$$

其中 $\bar{\lambda}_{J^c} = \dfrac{1}{D-M}\sum_{i\notin J}\lambda_i$ 是被丢弃特征值的平均值. 于是最大化似然等价于最小化

$$f(J) = \sum_{i\in J}\ln\lambda_i + (D-M)\ln\bar{\lambda}_{J^c}.$$

设 $a\in J$、$b\notin J$ 且 $\lambda_b > \lambda_a$, 把 J 换成 $J' = J\setminus\{a\}\cup\{b\}$. 记 $S_c = \sum_{i\notin J}\lambda_i$、$r = \lambda_b - \lambda_a > 0$, 则 $S_c \geqslant \lambda_b$, 于是

$$f(J') - f(J) = \ln\frac{\lambda_b}{\lambda_a} + (D-M)\ln\left(1-\frac{r}{S_c}\right) \leqslant \ln\frac{\lambda_b}{\lambda_a} + (D-M)\ln\frac{\lambda_a}{\lambda_b} = \big(1-(D-M)\big)\ln\frac{\lambda_b}{\lambda_a} \leqslant 0,$$

其中第二个不等号用了 $r/S_c \leqslant r/\lambda_b = 1-\lambda_a/\lambda_b$ 以及 ln 的单调性, 最后一个不等号用了 $D-M\geqslant 1$ 与 $\lambda_b > \lambda_a$. 因此, 只要保留的下标集合中存在比被丢弃的特征值更小的特征值, 把它换成更大的那个就不会使似然变小; 反复替换即知, 最大值在 J 取前 M 个最大特征值时达到. 若出现重特征值, 则替换前后似然相同 (最后一个不等号取等号), 此时不同的 J 给出相同的极大值, 对应同一子空间, 与命题中的 “当且仅当” 在子空间意义下一致. 其余驻点的似然严格更小, 不是最大值点.

□

注

(重构新增) 本定理回答了 “概率 PCA 到底在优化什么” 这一问题: 最大化似然自动把 $\boldsymbol{W}$ 的列对齐到样本协方差矩阵的前 M 个特征向量, 而 σ^2 恰好等于被丢弃特征值的平均值. 也就是说, 概率模型的最大似然解在子空间层面就是节 16.1中的主子空间, 两种完全不同的出发点给出同一个答案; 同时,σ^2 给出了一个内生的噪声估计, 这正是传统 PCA 所缺少的.

条件与边界需要说明. 第一, 要求所有 $\lambda_i > 0$, 否则 $\ln\lambda_i$ 无定义、$\boldsymbol{C}$ 奇异; 当 $N < D$ 时会出现零特征值, 此时应先用小节 16.1.5的对偶技巧把问题降到 N 维再应用本定理. 第二, 要求 $M \leqslant D-1$, 否则(16.46)的分母为零; 当 $M = D$ 时模型退化为无约束 Gauss 分布, 见(16.47)的讨论. 第三,$\boldsymbol{R}$ 的任意性说明 $\boldsymbol{W}$ 本身不可辨识, 可辨识的只是它张成的子空间与 σ^2; 这一点由命题 16.4给出精确表述. 最后, 本证明第三步只用到 ln 的单调性与一个初等不等式, 没有使用 EM 算法, 因此不依赖小节 16.3.1, 不存在循环论证.

此外,Tipping and Bishop (1999) 还指出, 当选择那些对应于前 M 个最大的特征值的特征向量时, 就会得到似然函数的最大值 (所有其他解都是鞍点).Roweis (1998) 也独立提出了类似的猜想, 尽管没有给出证明. 同样, 假设特征向量已按照对应特征值递减的顺序排列, 则 M 个主特征向量是 $\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M$. 在这种情况下,$\boldsymbol{W}$ 的列定义了标准 PCA 的主子空间.σ^2_{ML} 是与被丢弃维度相关联的平均方差.

$\boldsymbol{R}$ 是正交的, 因此可以解释为 M 维潜空间中的旋转矩阵. 如果我们将 $\boldsymbol{W}$ 的解代入 $\boldsymbol{C}$ 的表达式并利用正交性质 $\boldsymbol{R}\boldsymbol{R}^{\mathsf{T}} = \boldsymbol{I}$, 就可以看到 $\boldsymbol{C}$ 独立于 $\boldsymbol{R}$. 这表明预测密度不受潜空间内部旋转的影响. 对于特殊情况 $\boldsymbol{R} = \boldsymbol{I}$, 我们可以看到 $\boldsymbol{W}$ 的列是主成分特征向量, 按方差参数 $\lambda_i - \sigma^2$ 缩放. 一旦认识到独立 Gauss 分布 (在这种情况下是潜空间分布和噪声模型) 的卷积的方差是可加的, 这些缩放因子的解释也就清楚了. 因此, 沿着特征向量 $\boldsymbol{u}_i$ 方向的方差 λ_i 由以下两部分之和构成: 一是单位方差潜空间分布通过 $\boldsymbol{W}$ 的对应列投影到数据空间的贡献 $\lambda_i - \sigma^2$, 二是噪声模型在所有方向上添加的各向同性方差贡献 σ^2.

我们有必要花些时间研究(16.36)给出的数据协方差矩阵的形式. 考虑预测分布沿某个由单位向量 $\boldsymbol{v}$ 指定的方向的方差, 其中 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{v} = 1$, 由 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{v}$ 给出. 首先假设 $\boldsymbol{v}$ 与主子空间正交, 换句话说,$\boldsymbol{v}$ 是由被丢弃特征向量的某种线性组合给出的. 由于 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{U}_M = \boldsymbol{0}$, 因此 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{v} = \sigma^2$. 该模型预测了正交于主子空间的噪声方差. 从(16.46)来看, 它正是被丢弃特征值的平均值. 假设 $\boldsymbol{v} = \boldsymbol{u}_i$, 其中 $\boldsymbol{u}_i$ 是定义主子空间的保留特征向量之一, 则有 $\boldsymbol{v}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{v} = (\lambda_i - \sigma^2) + \sigma^2 = \lambda_i$.

换句话说, 这个模型正确地捕捉了数据沿主轴的方差, 并用单个平均值 σ^2 近似所有其余方向的方差.

命题 16.4 (潜空间旋转的不可识别性)

在概率 PCA 模型中, 对任意 $M \times M$ 正交矩阵 $\boldsymbol{R}$, 把 $\boldsymbol{W}$ 换成 $\boldsymbol{WR}$、把 $\boldsymbol{z}$ 换成 $\boldsymbol{R}^{\mathsf{T}}\boldsymbol{z}$ 后, 观测变量的边缘分布 $p(\boldsymbol{x})$ 与对数似然函数(16.44)完全不变; 因此仅由数据无法区分 $\boldsymbol{W}$ 与 $\boldsymbol{WR}$. 但 $\boldsymbol{W}$ 的列所张成的子空间与 σ^2 是可以由数据唯一确定的 (在特征值互不相同的条件下).

♠

证明

(重构新增) 由 $\boldsymbol{RR}^{\mathsf{T}} = \boldsymbol{I}$ 得 $(\boldsymbol{WR})(\boldsymbol{WR})^{\mathsf{T}} = \boldsymbol{WRR}^{\mathsf{T}}\boldsymbol{W}^{\mathsf{T}} = \boldsymbol{WW}^{\mathsf{T}}$, 故 $\boldsymbol{C}$ 不变; 由(16.35),$p(\boldsymbol{x})$ 只通过 $\boldsymbol{C}$ 依赖于 $\boldsymbol{W}$, 故边缘分布与似然函数都不变. 另一方面,$p(\boldsymbol{z}|\boldsymbol{x})$ 的均值变为 $\boldsymbol{M}'^{-1}(\boldsymbol{WR})^{\mathsf{T}}(\boldsymbol{x}-\boldsymbol{\mu})$, 其中 $\boldsymbol{M}' = \boldsymbol{R}^{\mathsf{T}}\boldsymbol{MR}$, 即后验分布被 $\boldsymbol{R}^{\mathsf{T}}$ 旋转, 这说明 "潜变量本身" 随 $\boldsymbol{R}$ 改变, 不可由数据辨认.

反过来说,$\boldsymbol{C} = \boldsymbol{WW}^{\mathsf{T}} + \sigma^2\boldsymbol{I}$ 的谱决定了一切: 它的前 M 个特征值是 $\lambda_1, \cdots, \lambda_M$、其余为 σ^2, 特征向量给出主子空间. 若 λ_i 互不相同且与 σ^2 不同, 则这个谱分解唯一, 于是主子空间与 σ^2 被数据唯一确定, 而 $\boldsymbol{W}$ 只被确定到右乘一个正交矩阵. 这正是(16.45)中 $\boldsymbol{R}$ 的来源.

□

注

(重构新增) 这条不可识别性与离散混合模型中 "分量重标记" 造成的不可识别性形成对照: 那里的不可识别性是离散的 ($K!$ 种重标记), 这里的不可识别性是连续的 (整个正交群 $O(M)$). 实践中的后果是: 数值优化得到的 $\boldsymbol{W}$ 的列一般不正交, 也不按特征值大小排列; 若需要正交的主成分方向, 可以对 $\boldsymbol{W}$ 作后处理 (例如对 $\boldsymbol{WW}^{\mathsf{T}}$ 作特征分解), 或者修改 EM 算法使其直接产生按特征值排序的正交方向.

需要注意的是, 不可识别的只是 "坐标", 不是 "模型": 所有 $\boldsymbol{WR}$ 给出同一个预测密度, 因此预测、似然、重构误差都不受影响. 这也解释了为什么在概率 PCA 中谈 "第 k 个主成分" 时必须先固定一个旋转约定 (例如取 $\boldsymbol{R} = \boldsymbol{I}$ 并按特征值降序排列).

构建最大似然密度模型的一种方法是, 简单地找到数据协方差矩阵的特征向量和特征值, 然后使用上述结果计算 $\boldsymbol{W}$ 和 σ^2. 在这种情况下, 选择 $\boldsymbol{R} = \boldsymbol{I}$ 会更为方便. 然而, 如果我们通过数值优化似然函数来找到最大似然解, 例如使用共轭梯度算法 (Fletcher, 1987; Nocedal and Wright, 1999) 或 EM 算法, 那么得到的 $\boldsymbol{R}$ 矩阵的值本质上是任意的 (见小节 16.3.2). 这意味着矩阵 $\boldsymbol{W}$ 的列不必是正交的. 如果需要正交基, 则可以适当对 $\boldsymbol{W}$ 矩阵进行后处理 (Golub and Van Loan, 1996). 也可以修改 EM 算法, 以直接产生按相应特征值大小排序的正交主成分方向 (Ahn and Oh, 2003).

潜空间中的旋转不变性表示了一种统计上的不可识别性, 类似于我们在离散潜变量混合模型中遇到的情况. 这里, 参数的连续性使得任何值都会导致相同的预测密度, 这与混合设置中组件重标记相关的离散不可识别性形成了对比.

考虑 $M = D$, 也就是没有降维的情况, 则 $\boldsymbol{U}_M = \boldsymbol{U}$ 且 $\boldsymbol{L}_M = \boldsymbol{L}$. 利用正交性质 $\boldsymbol{UU}^{\mathsf{T}} = \boldsymbol{I}$ 和 $\boldsymbol{RR}^{\mathsf{T}} = \boldsymbol{I}$,$\boldsymbol{x}$ 的边缘分布的数据协方差矩阵 $\boldsymbol{C}$ 将变为

$$\begin{aligned}\boldsymbol{C} &= \boldsymbol{U}\left(\boldsymbol{L}-\sigma^2\boldsymbol{I}\right)^{1/2}\boldsymbol{RR}^{\mathsf{T}}\left(\boldsymbol{L}-\sigma^2\boldsymbol{I}\right)^{1/2}\boldsymbol{U}^{\mathsf{T}} + \sigma^2\boldsymbol{I} \\ &= \boldsymbol{ULU}^{\mathsf{T}} = \boldsymbol{S}. \end{aligned} \tag{16.47}$$

于是我们得到无约束 Gauss 分布的标准最大似然解, 其中数据协方差矩阵由样本协方差给出.

传统 PCA 通常构造为从 D 维数据空间到 M 维线性子空间的点的投影. 然而, 概率 PCA 可以最自然地表示为从潜空间到数据空间的映射. 对于可视化和数据压缩等应用, 我们可以使用 Bayes 定理反转这个映射. 随后, 数据空间中的任何点就可以由其在潜空间中的后验均值和协方差来概括. 从(16.42)可知, 均值由下式给出:

$$\mathbb{E}[\boldsymbol{z}|\boldsymbol{x}] = \boldsymbol{M}^{-1}\boldsymbol{W}_{\mathrm{ML}}^{\mathsf{T}}(\boldsymbol{x}-\bar{\boldsymbol{x}}), \tag{16.48}$$

其中 $\boldsymbol{M}$ 由(16.41)给出. 这会将点投影到由下式给出的数据空间中:

$$\boldsymbol{W}\,\mathbb{E}[\boldsymbol{z}|\boldsymbol{x}] + \boldsymbol{\mu}. \tag{16.49}$$

请注意, 这与正则化线性回归的方程相同, 并且是最大化线性-Gauss 模型的似然函数的结果 (见小节 4.1.6). 同样, 从(16.42)可以看出, 后验协方差由 $\sigma^2\boldsymbol{M}^{-1}$ 给出, 且与 $\boldsymbol{x}$ 无关.

取极限 $\sigma^2 \to 0$, 后验均值可以简化为

$$\left(\boldsymbol{W}_{\mathrm{ML}}^{\mathsf{T}} \boldsymbol{W}_{\mathrm{ML}}\right)^{-1} \boldsymbol{W}_{\mathrm{ML}}^{\mathsf{T}}(\boldsymbol{x}-\bar{\boldsymbol{x}}). \tag{16.50}$$

这代表数据点在潜空间中的正交投影, 因此我们恢复了标准 PCA 模型 (见练习 16.11). 然而, 在这个极限下, 后验协方差为零, 密度变得奇异. 对于 $\sigma^2 > 0$, 潜投影相对于正交投影向原点移动 (见练习 16.12).

最后请注意, 概率 PCA 模型的一个重要作用是定义了一个多元 Gauss 分布, 其中自由度的数量 (也就是独立参数的数量) 是可以控制的, 同时仍允许模型捕获数据中的主要相关性. 请回想一下, 一个普通的 Gauss 分布在协方差矩阵中有 $D(D+1)/2$ 个独立参数 (另外还有 D 个参数在均值中)(见节 3.2). 因此, 参数的数量随着 D 的增加而按平方增长, 并且在高维空间中可能会变得过多. 如果我们限制协方差矩阵为对角矩阵, 那便只有 D 个独立参数, 参数的数量随维度线性增长. 然而, 这会将变量当作独立变量, 因此已无法再表达它们之间的任何相关性. 概率 PCA 模型提供了一个优雅的折中方案, 其中 M 个最重要的相关性可以捕获, 同时确保参数的数量只随维度线性增长. 我们可以通过计算概率 PCA 模型中自由度的量看到这一点. 数据协方差矩阵 $\boldsymbol{C}$ 取决于 $\boldsymbol{W}$, 大小为 $D \times M$, 再加上 σ^2, 总共有 $DM+1$ 个参数. 然而, 我们已经看到这种参数化中存在与潜空间坐标系的旋转相关的一些冗余. 表达这些旋转的正交矩阵 $\boldsymbol{R}$ 的大小为 $M \times M$. 在这个矩阵的第一列中, 有 $M-1$ 个独立参数 (因为列向量必须归一化为单位长度); 而在第二列中, 有 $M-2$ 个独立参数 (因为列必须归一化并且与前一列正交), 以此类推. 将这个等差数列求和后, 我们看到 $\boldsymbol{R}$ 总共有 $M(M-1)/2$ 个独立参数. 因此, 数据协方差矩阵 $\boldsymbol{C}$ 中自由度的数量由下式给出:

$$DM + 1 - M(M-1)/2. \tag{16.51}$$

在固定 M 的情况下, 该模型中独立参数的数量仅随维度线性增长. 取 $M = D-1$, 便可得到全协方差 Gauss 分布的标准结果 (见练习 16.14). 在这种情况下, 沿 $D-1$ 个线性无关方向的方差由 $\boldsymbol{W}$ 的列控制, 而沿剩余方向的方差由 σ^2 给出. 当 $M = 0$ 时, 该模型等价于各向同性协方差的情况.

16.2.4 因子分析

因子分析模型是一个线性-Gauss 潜变量模型, 它与概率 PCA 模型密切相关. 因子分析的独特性体现为, 其使得给定潜变量 $\boldsymbol{z}$ 的观测变量 $\boldsymbol{x}$ 的条件分布具有对角协方差, 而不是各向同性协方差, 即

$$p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z}+\boldsymbol{\mu}, \boldsymbol{\Psi}), \tag{16.52}$$

其中 $\boldsymbol{\Psi}$ 是一个 $D \times D$ 的对角矩阵.

下面把上述模型的形式正式表述如下.

定义 16.4 (因子分析模型)

设 $\boldsymbol{z}$ 是 M 维潜变量,$\boldsymbol{x}$ 是 D 维观测变量, 且

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I}), \qquad p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z}+\boldsymbol{\mu}, \boldsymbol{\Psi}),$$

其中 $\boldsymbol{W}$ 为 $D \times M$ 矩阵,$\boldsymbol{\Psi}$ 为 $D \times D$ 对角矩阵. 则称 $(\boldsymbol{x}, \boldsymbol{z})$ 服从因子分析 (factor analysis) 模型; 称 $\boldsymbol{W}$ 的列为因子载荷 (factor loading), 称 $\boldsymbol{\Psi}$ 的对角元素为唯一性 (uniquenesses). ♣

注

(重构新增) 因子分析与概率 PCA 的唯一差别是条件协方差由 $\sigma^2\boldsymbol{I}$ 换成了对角矩阵 $\boldsymbol{\Psi}$. 这一改动看似很小, 却改变了模型能表达的相关结构:$\boldsymbol{W}\boldsymbol{W}^{\mathsf{T}}$ 负责解释变量之间的相关性, 而 $\boldsymbol{\Psi}$ 的对角元 ψ_i 单独描述第 i 个变量自身的、无法由公共因子解释的方差. 因此因子分析把每个观测变量的方差分解为“公共部分” $\sum_j w_{ij}^2$ 与“独特部分” ψ_i, 这正是“因子载荷”“唯一性”这两个名称的含义.

条件方面:$\boldsymbol{\Psi}$ 必须正定 (对角元全为正), 否则条件密度退化;$\boldsymbol{W}$ 一般取满列秩 M, 否则因子之间可以线性表示、模型进一步不可辨识. 与概率 PCA 不同, 这里不再有 σ^2 这个共同的噪声水平, 因此模型对潜空间的旋转不变, 但对数据坐标的旋转不不变——这一点由练习 16.17精确刻画. 正如本节将要说明的,$\boldsymbol{W}$ 没有闭式的最大似然解, 需

用 EM 或迭代方法求解.

注意, 因子分析模型和概率 PCA 模型一样, 也假设观测变量 $x_1, \cdots, x_D$ 在给定潜变量 $\boldsymbol{z}$ 时是相互独立的. 因子分析模型的核心在于, 它通过用矩阵 $\boldsymbol{\Psi}$ 表示与每个坐标相关的独立方差, 并在矩阵 $\boldsymbol{W}$ 中捕获变量之间的协方差, 来解释观测到的数据协方差结构.

在 Everitt (1984)、Bartholomew (1987) 和 Basilevsky (1994) 等人的文献中可以找到关于因子分析的论述.Lawley (1953) 和 Anderson (1963) 研究了因子分析与 PCA 之间的联系, 他们揭示了在似然函数的驻点处, 对于一个 $\boldsymbol{\Psi} = \sigma^2\boldsymbol{I}$ 的因子分析模型,$\boldsymbol{W}$ 的列是样本协方差矩阵的缩放特征向量, 而 σ^2 是被丢弃特征值的平均值. 后来,Tipping and Bishop (1999) 指出, 在将组成 $\boldsymbol{W}$ 的特征向量选为主特征向量时, 对数似然函数的最大值便会出现.

利用(16.34), 我们可以看到观测变量的边缘分布是由 $p(\boldsymbol{x}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{C})$ 给出的, 其中

$$\boldsymbol{C} = \boldsymbol{W}\boldsymbol{W}^{\mathsf{T}} + \boldsymbol{\Psi}. \tag{16.53}$$

命题 16.5 (因子分析的旋转不变性)

对任意 $M\times M$ 正交矩阵 $\boldsymbol{R}$, 把因子分析模型中的 $\boldsymbol{W}$ 换成 $\boldsymbol{W}\boldsymbol{R}$ 后, 观测变量的边缘分布不变, 即 $\boldsymbol{C} = \boldsymbol{W}\boldsymbol{W}^{\mathsf{T}}+\boldsymbol{\Psi}$ 不变; 因此因子载荷矩阵 $\boldsymbol{W}$ 只能确定到右乘一个正交矩阵, 因子本身 (潜空间的坐标) 不可识别. ♠

证明

(重构新增) $(\boldsymbol{W}\boldsymbol{R})(\boldsymbol{W}\boldsymbol{R})^{\mathsf{T}} = \boldsymbol{W}\boldsymbol{R}\boldsymbol{R}^{\mathsf{T}}\boldsymbol{W}^{\mathsf{T}} = \boldsymbol{W}\boldsymbol{W}^{\mathsf{T}}$, 故(16.53)不变, 从而 $p(\boldsymbol{x})$ 与似然函数不变. 另一方面, 给定 $\boldsymbol{x}$ 时 $\boldsymbol{z}$ 的后验均值是 $\boldsymbol{G}\boldsymbol{W}^{\mathsf{T}}\boldsymbol{\Psi}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})$(其中 $\boldsymbol{G}$ 由(16.73)定义), 换成 $\boldsymbol{W}\boldsymbol{R}$ 后变为 $\boldsymbol{R}^{\mathsf{T}}$ 倍的相应量, 即后验分布随 $\boldsymbol{R}$ 旋转, 说明潜变量坐标确实改变.

反过来, 模型能被数据确定的量是 $\boldsymbol{C} = \boldsymbol{W}\boldsymbol{W}^{\mathsf{T}} + \boldsymbol{\Psi}$. 给定 $\boldsymbol{C}$ 与 $\boldsymbol{\Psi}$ 时 $\boldsymbol{W}\boldsymbol{W}^{\mathsf{T}} = \boldsymbol{C} - \boldsymbol{\Psi}$ 固定, 而满足该式的 $\boldsymbol{W}$ 恰好构成 $\{\boldsymbol{W}\boldsymbol{R} : \boldsymbol{R}^{\mathsf{T}}\boldsymbol{R} = \boldsymbol{I}\}$ 这一族 (因为 $\boldsymbol{W}$ 与 $\boldsymbol{W}\boldsymbol{R}$ 有相同的 Gram 矩阵, 且同一 Gram 矩阵的满秩分解在右乘正交矩阵的意义下唯一). 故 $\boldsymbol{W}$ 的不可识别性恰为正交群 $O(M)$.

□

注

(重构新增) 这条不可识别性比概率 PCA 的更“顽固”: 在概率 PCA 中可以通过固定 $\boldsymbol{R} = \boldsymbol{I}$ 并按特征值大小排序来消除冗余 (因为 σ^2 提供了额外的标量约束), 而在因子分析中, 任意正交旋转都给出相同的似然, 因此“第 j 个因子是什么”没有数据依据. 历史上人们曾试图通过旋转到某种“简单结构”(使载荷矩阵尽量稀疏) 来解释各个因子, 这种做法本质上是人为地添加了一个非统计的约定.

条件上要注意: 该命题的结论依赖于 $\boldsymbol{\Psi}$ 固定且为对角. 若允许同时变换 $\boldsymbol{\Psi}$, 则群会变大, 模型更加不可辨识 (见练习 16.17的 (i) 款, 那里对数据坐标作对角缩放会被 $\boldsymbol{\Psi}$ 吸收). 反之, 若限定潜变量非 Gauss 分布, 则旋转后的模型一般不再保持“各分量独立”, 这正是独立成分分析能消除旋转不确定性的原因 (见小节 16.2.5).

与概率 PCA 模型一样, 该模型对潜空间中的旋转是不变的 (见练习 16.16).

历史上, 当人们尝试用因子分析方法对各个因子 ($\boldsymbol{z}$ 空间中的坐标) 进行解释时, 曾引发争议. 因子分析与这个空间中的旋转相关联, 导致不可识别性, 因而这种尝试被证明是有问题的. 然而, 从我们的角度看, 我们将因子分析视为一种潜变量密度模型, 关注的重点是其中潜空间的结构形式, 而不是用于描述该空间的具体坐标. 如果希望消除与潜空间旋转所带来的不确定性, 则必须考虑非 Gauss 潜变量分布, 以产生独立成分分析模型 (参见小节 16.2.5).

概率 PCA 和因子分析的另一个区别是它们在数据集变换下的行为不同 (见练习 16.17). 对于 PCA 和概率 PCA, 如果旋转数据空间中的坐标系, 则可以得到与数据完全相同的拟合结果, 只不过 $\boldsymbol{W}$ 矩阵会由相应的旋转矩阵变换而成的. 然而, 对于因子分析, 相似的性质是: 如果我们对数据向量逐个分量地进行重新缩放, 则这个操作会被吸收进相应的 $\boldsymbol{\Psi}$ 元素的重新缩放中.

16.2.5 独立成分分析

线性 Gauss 潜变量模型的一种扩展是这样的模型: 观测变量与潜变量线性相关, 但潜变量分布是非 Gauss 分布. 这类模型的一个重要分支是独立成分分析 (Independent Component Analysis, ICA) 模型, 其潜变量的联合分布能够分解为各分量的独立分布之积, 即

$$p(\boldsymbol{z}) = \prod_{j=1}^{M} p(z_j). \tag{16.54}$$

定义 16.5 (独立成分分析) 设 $\boldsymbol{z} = (z_1, \cdots, z_M)$ 是潜变量, 其联合分布满足(16.54)(即各分量相互独立), 且各分量 z_j 的分布不是 Gauss 分布; 观测变量由线性映射给出,$\boldsymbol{x} = \boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu} + \boldsymbol{\varepsilon}$, 其中 $\boldsymbol{W}$ 为 $D \times M$ 矩阵、$\boldsymbol{\varepsilon}$ 为噪声. 则称该模型为独立成分分析 (Independent Component Analysis, ICA) 模型. ♣

注

(重构新增) ICA 要解决的问题是“盲源分离”: 只观测到若干路混合信号, 要求还原出彼此独立的原始源. 与 PCA 相比, 关键差别不在模型的形式 (同样是线性映射), 而在对潜变量的分布假设:PCA 默认潜变量是 Gauss 的,ICA 要求非 Gauss.

为什么非 Gauss 如此关键? 因为 Gauss 分布在正交变换下保持 Gauss 且协方差为 $\boldsymbol{I}$ 的形式不变, 所以只用二阶信息无法确定“哪个方向是源”; 而非 Gauss 分布 (尤其是重尾分布) 在旋转下不再保持“各分量独立”这一性质, 于是独立性本身成了可检验、可利用的约束. 条件方面:$M \leqslant D$(通常取 $M = D$),$\boldsymbol{W}$ 需满列秩, 否则源不可分离; 噪声可省略是因为当 $M = D$ 且 $\boldsymbol{W}$ 非奇异时观测分布是源分布的确定性线性像, 边缘分布通常非奇异.

例题 16.2 盲源分离 为了理解这些模型的作用, 可以设想以下场景: 有两个人在同时说话, 我们使用两个麦克风录制他们的声音. 如果忽略时间延迟和回声等效果, 则任何时刻麦克风接收到的信号都将由这两个声音的振幅的线性组合给出. 这个线性组合的系数是恒定的. 如果我们能从样本数据中推断出它们的值, 我们就可以反转混合过程 (假设它是非奇异的), 从而得到两个干净的信号, 每个信号只包含一个人的声音. 这是一个盲源分离的问题, “盲”指的是我们只给出了混合数据, 而没有观察到原始来源或混合系数 (Cardoso, 1998).

这类问题有时可以采取以下方法来解决 (MacKay, 2003). 该方法忽略信号的时间特性, 并将连续的样本视为独立同分布的. 考虑一个生成式模型, 其中有两个潜变量, 对应于未观测语音信号振幅; 还有两个观测变量, 由麦克风处信号值给出. 潜变量具有可以进行因子分解的联合分布, 并且观测变量由潜变量的线性组合给出. 没有必要包括噪声分布, 因为潜变量的数量等于观测变量的数量, 而观测变量的边缘分布通常不具有奇异性, 所以观测变量仅仅是潜变量的确定性线性组合. 给定一组观测数据, 该模型的似然函数是线性组合中系数的一个函数. 在使用基于梯度的优化方法最大化对数似然函数时, 会产生特定版本的 ICA.

命题 16.6 (Gauss 潜变量无法辨识独立成分) 若把 ICA 模型中的潜变量分布换成一般的 Gauss 分布 $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{m}, \boldsymbol{\Sigma})$, 则所得模型与某个概率 PCA 模型等价; 特别地, 对任意正交矩阵 $\boldsymbol{R}$, 把 $\boldsymbol{z}$ 换成 $\boldsymbol{R}^{\mathsf{T}}\boldsymbol{z}$、$\boldsymbol{W}$ 换成 $\boldsymbol{W}\boldsymbol{R}$ 都给出同一个观测分布, 而 $\boldsymbol{R}^{\mathsf{T}}\boldsymbol{z}$ 的各分量一般不再相互独立. 因此仅凭二阶信息无法确定独立成分. ♠

证明

(重构新增) 设 $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^{\mathsf{T}}$ 为 Cholesky 分解, 令 $\boldsymbol{\xi} = \boldsymbol{L}^{-1}(\boldsymbol{z} - \boldsymbol{m})$, 则 $\boldsymbol{\xi} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$, 且

$$\boldsymbol{x} = \boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu} + \boldsymbol{\varepsilon} = \boldsymbol{W}\boldsymbol{L}\boldsymbol{\xi} + (\boldsymbol{W}\boldsymbol{m} + \boldsymbol{\mu}) + \boldsymbol{\varepsilon}.$$

这正是概率 PCA 模型(16.31)—(16.32), 参数为 $\boldsymbol{W}' = \boldsymbol{W}\boldsymbol{L}$、$\boldsymbol{\mu}' = \boldsymbol{W}\boldsymbol{m} + \boldsymbol{\mu}$. 因此任何 Gauss 潜变量的线性模型都等价于一个概率 PCA 模型.

再由命题 16.4, 对任意正交 $\boldsymbol{R}$,$(\boldsymbol{W}'\boldsymbol{R}, \boldsymbol{R}^{\mathsf{T}}\boldsymbol{\xi})$ 与 $(\boldsymbol{W}', \boldsymbol{\xi})$ 给出相同的观测分布. 把 $\boldsymbol{R}$ 取为一般的旋转,$\boldsymbol{R}^{\mathsf{T}}\boldsymbol{\xi}$ 的分量是独立标准正态变量的正交组合, 仍是 Gauss 变量且彼此不相关; 而 Gauss 情形下“不相关”与“独立”重合, 因此任何正交组合都同时满足各分量独立, 无法由数据判断哪一个坐标对应真正的源. 若原模型中的源是非 Gauss

的独立分量, 那么混入一个正交旋转后各分量不再独立 (非 Gauss 情形下不相关推不出独立, 见练习 2.39), 正是这个差别使得 ICA 可解.

□

注

(重构新增) 这条命题把“非 Gauss 性是 ICA 的必要条件”讲清楚了. 直观上,Gauss 分布是唯一在正交变换下保持“独立同分布形式”的分布族: 对 Gauss 变量而言,“不相关”与“独立”重合, 因此旋转前后的两套坐标都满足独立假设, 模型无法区分. 非 Gauss 分布不具备这一旋转对称性, 独立性才成为可用的判据.

条件上要注意: 命题只说明 Gauss 情形不可辨识, 并不说明任意非 Gauss 分布都可辨识.ICA 的可辨识性还需要至多一个源是 Gauss 的 (否则仍可交换或旋转), 以及混合矩阵满秩等条件; 本书不展开这些充分条件, 但读者应避免把“非 Gauss”误当成“一定可解”.

对于为什么 Gauss 潜变量分布在线性模型中不足以找到独立成分, 我们还可以用另一种方式来理解. 我们注意到, 主成分代表了数据空间中坐标系的旋转, 目的是使协方差矩阵对角化, 从而使新坐标中的数据分布不再相关. 尽管零相关是独立性的必要条件, 但它不是充分条件 (见练习 2.39).

在实践中, 常用的潜变量分布由下式给出:

$$p(z_j) = \frac{1}{\pi \cos(z_j)} = \frac{2}{\pi \left(\mathrm{e}^{z_j} + \mathrm{e}^{-z_j}\right)}. \tag{16.55}$$

与 Gauss 分布相比, 它具有重尾 (heavy tail), 我们在许多真实世界的分布中也能观察到这种特性.

注

(重构新增, 原文问题说明) (16.55)的第一个等号按字面并不成立:$\frac{1}{\pi \cos z_j}$ 在 $z_j = \pm\pi/2$ 处发散, 且不是概率密度; 而右端 $\frac{2}{\pi(\mathrm{e}^{z_j} + \mathrm{e}^{-z_j})} = \frac{1}{\pi \cosh z_j}$ 才是有定义、可归一化的重尾密度. 原文此处印作 cos, 应为双曲余弦 cosh 的排印脱漏. 本式保留原样以便与原文对照, 但读者应按 cosh 理解: 该密度确实具有比 Gauss 分布更重的尾部, 这正是 ICA 所需要的性质.

最初的 ICA 模型 (Bell and Sejnowski, 1995) 是采用信息最大化定义的目标函数进行优化. 概率潜变量表述的一个优势是, 它有助于启发并构建 ICA 的扩展. 例如, 独立因子分析 (independent factor analysis)(Attias, 1999) 考虑了一个模型, 在该模型中, 潜变量和观测变量的数量可以不同, 观测变量是有噪声的, 潜变量则具有 Gauss 混合模型化的灵活分布. 可以使用 EM 算法最大化这个模型的对数似然函数, 并使用变分法近似重构潜变量. 研究人员考虑了许多其他类型的模型, 如关于独立成分分析及其应用的文献已经不胜枚举了 (Jutten and Herault, 1991; Comon, Jutten, and Herault, 1991; Amari, Cichocki, and Yang, 1996; Pearlmutter and Parra, 1997; Hyvärinen and Oja, 1997; Hinton et al., 2001; Miskin and MacKay, 2001; Hojen-Sorensen, Winther, and Hansen, 2002; Choudrey and Roberts, 2003; Chan, Lee, and Sejnowski, 2003; Stone, 2004).

16.2.6 Kalman 滤波器

到目前为止, 我们假设数据值是独立同分布的. 一种常见的非独立同分布情况是, 数据点形成一个有序序列. 我们已经看到, 隐 Markov 模型可以视为混合模型的扩展, 旨在允许数据中存在序列相关性 (参见小节 15.3.1). 类似地, 连续潜变量模型可以通过连接潜变量形成 Markov 链来扩展, 以处理序列数据, 参见图 16.9所示的图模型. 这种模型称为线性动态系统 (linear dynamical system) 或 Kalman 滤波器 (Zarchan and Musoff, 2005). 注意, 这与隐 Markov 模型的图结构相同 (参见小节 15.3.1). 值得注意的是, 隐 Markov 模型和线性动态系统是独立发展的. 然而, 一旦它们都表示为图模型, 它们之间的深层关系就会水落石出.Kalman 滤波器已广泛地应用在许多实时跟踪应用中, 如使用雷达信号跟踪飞机.

定义 16.6 (线性动态系统)

设观测序列 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$ 与潜变量序列 $z_1, \cdots, z_N$ 满足:(i) 给定 z_n 时 $\boldsymbol{x}_n$ 的条件分布是线性-Gauss 模型 $p(\boldsymbol{x}_n|z_n)$, 其均值是 z_n 的线性函数、协方差与 z_n 无关;(ii) 潜变量构成 Markov 链,$p(z_n|z_{n-1})$ 是 Gauss 分布且其均值是 z_{n-1} 的线性函数;(iii) 所有 $p(\boldsymbol{x}_n|z_n)$ 共享同一组参数, 所有 $p(z_n|z_{n-1})$ 也共享同一组参数. 则

称该模型为线性动态系统 (linear dynamical system), 也称 Kalman 滤波器. ♣

注

(重构新增) 线性动态系统把概率 PCA 从“独立同分布样本”推广到“有序序列”: 每个观测仍然由各自的潜变量经线性-Gauss 映射生成, 但潜变量之间用 Markov 链串起来, 从而让相邻时刻的状态可以相互传递信息. 第 (iii) 条“参数共享”是关键: 它使模型的参数个数与序列长度 N 无关, 因此无论序列多长, 都只有固定个数的参数需要估计, 这也是 Kalman 滤波可以实时运行的根本原因.

条件方面:$p(z_n|z_{n-1})$ 取线性-Gauss 形式保证了整个模型是线性-Gauss 的, 从而所有边缘与条件分布都是 Gauss 分布; 否则后验将不再有闭式, 只能退化为一般的状态空间模型. 若把 $p(\boldsymbol{x}_n|z_n)$ 换成离散分布、$p(z_n|z_{n-1})$ 换成离散转移, 就得到隐 Markov 模型——两者的图结构相同, 差别只在变量的取值域.

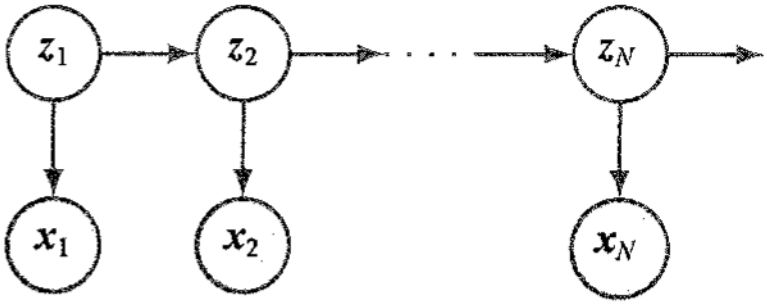


图 16.9: 用于序列数据的概率图模型称为线性动态系统或 Kalman 滤波器, 其中的潜变量形成了一个 Markov 链

在这样的最简单模型中,图 16.9中的分布 $p(\boldsymbol{x}_n|z_n)$ 代表了该特定观测结果的线性-Gauss 潜变量模型, 就像我们之前讨论的独立同分布数据一样. 然而, 潜变量 z_n 不再视为独立的, 而是形成了一个 Markov 链, 链中每个潜变量的分布 $p(z_n|z_{n-1})$ 取决于链中前一个潜变量的状态. 在线性-Gauss 潜变量模型中,z_n 的分布是 Gauss 分布, 均值则由 z_{n-1} 的线性函数给出. 通常, 分布 $p(\boldsymbol{x}_n|z_n)$ 的所有参数是共享的, 分布 $p(z_n|z_{n-1})$ 的所有参数也是共享的, 因此模型中总的参数数量是固定的, 不依赖于序列的长度. 这些参数可以通过最大似然法从数据中来学习. 这需要用到在图模型上传播消息的有效算法 (Bishop, 2006). 在本章的其余部分, 我们将专注于独立同分布数据.

16.3 证据下界

在讨论离散潜变量模型时, 我们推导出了边缘对数似然的证据下界 (ELBO), 并展示了它是如何构成期望最大化 (EM) 算法的基础的, 包括其扩展, 例如变分推断 (参见节 15.4). 相同的框架同样适用于连续潜变量模型, 以及结合了离散和连续变量的模型. 这里我们提供一个稍微不同的 ELBO 推导, 并且假设潜变量 $\boldsymbol{z}$ 是连续的.

考虑一个带有观测变量 $\boldsymbol{x}$、潜变量 $\boldsymbol{z}$ 和可学习参数向量 $\boldsymbol{w}$ 的模型 $p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{w})$. 只要引入潜变量上的任意一个分布 $q(\boldsymbol{z})$, 我们就可以将对数似然函数 $\ln p(\boldsymbol{x}|\boldsymbol{w})$ 写成两项之和的形式 (见练习 16.18):

$$\ln p(\boldsymbol{x}|\boldsymbol{w}) = \mathcal{L}(\boldsymbol{w}) + \mathrm{KL}\left(q(\boldsymbol{z}) \| p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w})\right), \tag{16.56}$$

其中

$$\mathcal{L}(q, \boldsymbol{w}) = \int q(\boldsymbol{z}) \ln \left\{ \frac{p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{w})}{q(\boldsymbol{z})} \right\} \mathrm{d}\boldsymbol{z}, \tag{16.57}$$

而

$$\mathrm{KL}\left(q(\boldsymbol{z}) \| p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w})\right) = -\int q(\boldsymbol{z}) \ln \left\{ \frac{p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w})}{q(\boldsymbol{z})} \right\} \mathrm{d}\boldsymbol{z}. \tag{16.58}$$

命题 16.7 (证据下界分解)

设 $q(z)$ 是潜变量上的任意一个概率密度, 且(16.57)与(16.58)中的积分绝对收敛. 则(16.56)成立, 其中 $\mathcal{L}(q,\boldsymbol{w})$ 由(16.57)给出、KL 由(16.58)给出; 并且

$$\ln p(\boldsymbol{x}|\boldsymbol{w}) \geqslant \mathcal{L}(q,\boldsymbol{w}), \tag{16.59}$$

等号成立当且仅当 $q(z) = p(z|\boldsymbol{x},\boldsymbol{w})$(几乎处处).

证明

(重构新增) 由概率的乘积法则,$p(\boldsymbol{x},z|\boldsymbol{w}) = p(z|\boldsymbol{x},\boldsymbol{w})p(\boldsymbol{x}|\boldsymbol{w})$. 于是

$$\begin{aligned}\mathcal{L}(q,\boldsymbol{w}) + \mathrm{KL}\left(q\|p(\cdot|\boldsymbol{x},\boldsymbol{w})\right) &= \int q(z)\ln\frac{p(\boldsymbol{x},z|\boldsymbol{w})}{q(z)}\mathrm{d}z - \int q(z)\ln\frac{p(z|\boldsymbol{x},\boldsymbol{w})}{q(z)}\mathrm{d}z \\ &= \int q(z)\ln\frac{p(\boldsymbol{x},z|\boldsymbol{w})}{p(z|\boldsymbol{x},\boldsymbol{w})}\mathrm{d}z = \int q(z)\ln p(\boldsymbol{x}|\boldsymbol{w})\,\mathrm{d}z = \ln p(\boldsymbol{x}|\boldsymbol{w}),\end{aligned}$$

其中最后一步用了 $\int q(z)\mathrm{d}z = 1$, 且 $\ln p(\boldsymbol{x}|\boldsymbol{w})$ 与 z 无关. 这就证明了(16.56).

再由 Kullback-Leibler 散度的非负性 (命题 2.28),KL $\geqslant 0$, 故(16.59)成立. 等号成立当且仅当两个密度几乎处处相等, 即 $q(z) = p(z|\boldsymbol{x},\boldsymbol{w})$.

□

注

(重构新增) 这个分解是整章的枢纽: 它把一个难以直接计算的量 $\ln p(\boldsymbol{x}|\boldsymbol{w})$ 拆成“可算的下界”与“非负的差距”两部分, 而差距恰是 q 与真实后验之间的 Kullback-Leibler 散度. 因此, 想让下界接近真值, 就要让 q 接近后验; 想提高似然, 就要提高下界.EM 算法的两个步骤正是分别对应这两件事.

条件方面: 分解本身只要求 q 是概率密度且相关积分绝对收敛, 对 q 的形式没有任何限制, 因此可以取任意易处理的分布族来做变分近似 (这正是变分推断的出发点). 等号成立的条件“q 等于后验”说明下界是紧的, 但它一般无法达到: 只有当后验恰好落在所选分布族内时才可能.

原文在(16.56)与(16.59)中把下界简记为 $\mathcal{L}(\boldsymbol{w})$, 此后又记作 $\mathcal{L}(q,\boldsymbol{w})$; 两者是同一个量 (下界同时依赖于 q 与 $\boldsymbol{w}$, 简记时省略了 q). 本译本保留原文的两种写法.

KL$(q(z)\|p(z|\boldsymbol{x},\boldsymbol{w}))$ 是一种 Kullback-Leibler 散度, 满足特性 KL$(\cdot\|\cdot) \geqslant 0$(参见小节 2.5.5), 从中可以得出(16.59).(16.57)给出的 $\mathcal{L}(q,\boldsymbol{w})$ 构成了对数似然的下界, 称为证据下界 (ELBO).$\mathcal{L}(q,\boldsymbol{w})$ 采用了与离散情况相同的形式 [(15.53)], 但是将求和替换成了积分.

我们可以使用 EM 算法的两阶段迭代过程来最大化对数似然函数. 在 EM 算法中, 我们可以交替地针对 $q(z)$(E 步骤) 和 $\boldsymbol{w}$(M 步骤) 最大化 $\mathcal{L}(q,\boldsymbol{w})$. 首先初始化参数 $\boldsymbol{w}^{\mathrm{old}}$. 然后在 E 步骤中保持 $\boldsymbol{w}$ 固定, 并针对 $q(z)$ 最大化下界. 我们注意到, 可以通过最小化(16.58)中的 Kullback-Leibler 散度来获得下界的最高值. 换言之, 当 Kullback-Leibler 散度为零时,$q(z) = p(z|\boldsymbol{x};\boldsymbol{w}^{\mathrm{old}})$. 在 M 步骤中, 我们保持所选的 $q(z)$ 固定, 并针对 $\boldsymbol{w}$ 最大化 $\mathcal{L}(q,\boldsymbol{w})$. 将 $q(z)$ 代入(16.57), 可以得到

$$\begin{aligned}\mathcal{L}(q,\boldsymbol{w}) = &\int p(z|\boldsymbol{x},\boldsymbol{w}^{\mathrm{old}})\ln p(\boldsymbol{x},z|\boldsymbol{w})\,\mathrm{d}z \\ &- \int p(z|\boldsymbol{x},\boldsymbol{w}^{\mathrm{old}})\ln p(z|\boldsymbol{x},\boldsymbol{w}^{\mathrm{old}})\,\mathrm{d}z.\end{aligned} \tag{16.60}$$

在 M 步骤中保持 $\boldsymbol{w}^{\mathrm{old}}$ 固定, 并针对 $\boldsymbol{w}$ 最大化 $\mathcal{L}(q,\boldsymbol{w})$. 请注意,(16.60)右侧的第二项独立于 $\boldsymbol{w}$, 因此在 M 步骤中可以忽略.(16.60)右侧的第一项则是完整数据对数似然的期望, 该期望来自我们用 $\boldsymbol{w}^{\mathrm{old}}$ 计算得到的 z 的后验分布 (参见节 15.3).

如果我们有一个由独立同分布观测值组成的数据集 $\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N$, 则似然函数的形式为

$$\ln p(\boldsymbol{X}|\boldsymbol{w}) = \sum_{n=1}^{N}\ln p(\boldsymbol{x}_n|\boldsymbol{w}), \tag{16.61}$$

其中数据矩阵 $\boldsymbol{X}$ 包含 $\boldsymbol{x}_1,\cdots,\boldsymbol{x}_N$, 并且参数 $\boldsymbol{w}$ 在所有数据点之间是共享的. 对于每一个数据点, 引入一个相应的潜变量 $\boldsymbol{z}_n$ 及其关联的分布 $q(\boldsymbol{z}_n)$, 并且通过遵循类似于推导(16.57)时使用的步骤, 我们得到了以下形式的 ELBO(参

见节 19.2):

$$\mathcal{L}(q, \boldsymbol{w}) = \sum_{n=1}^{N} \int q(\boldsymbol{z}_n) \ln \left\{ \frac{p(\boldsymbol{x}_n, \boldsymbol{z}_n | \boldsymbol{w})}{q(\boldsymbol{z}_n)} \right\} \mathrm{d}\boldsymbol{z}_n. \tag{16.62}$$

当讨论变分自编码器 (见第 19 章) 时, 我们将遇到一个模型, 其 E 步骤的精确解无法实现, 因此我们改为使用深度神经网络对 $q(\boldsymbol{z})$ 进行建模, 然后使用 ELBO 来学习网络的参数.

16.3.1 EM 算法

在本小节, 我们将使用 EM 算法, 通过迭代地最大化证据下界来学习概率 PCA 模型的参数. 这看起来有些多余, 因为我们已经得到了一个精确的闭式解来求解最大似然参数值, 然而在高维空间中, 与直接处理样本协方差矩阵相比, 使用迭代的 EM 过程可能在计算上有优势. 这个 EM 过程也可以扩展到没有闭式解的因子分析模型 (参见小节 16.2.4). 最后, 它允许以合理的方式处理缺失数据.

我们可以按照 EM 算法的通用框架来导出概率 PCA 的 EM 算法 (参见节 15.3). 为此, 我们写下完整数据对数似然函数并根据用 “旧” 参数值评估的潜变量分布的后验分布来获取期望, 然后最大化这个完整数据对数似然函数的期望以产生 “新” 的参数值. 因为我们假定数据点是独立的, 所以完整数据对数似然函数的形式为

$$\ln p(\boldsymbol{X}, \boldsymbol{Z} | \boldsymbol{\mu}, \boldsymbol{W}, \sigma^2) = \sum_{n=1}^{N} \{ \ln p(\boldsymbol{x}_n | \boldsymbol{z}_n) + \ln p(\boldsymbol{z}_n) \}, \tag{16.63}$$

其中矩阵 $\boldsymbol{Z}$ 的第 n 行由 $\boldsymbol{z}_n^{\mathsf{T}}$ 给出. 我们已经知道 $\boldsymbol{\mu}$ 的精确最大似然解是由样本均值 $\bar{\boldsymbol{x}}$[(16.1)] 给出的, 在这个阶段替代 $\boldsymbol{\mu}$ 是很方便的. 利用(16.31)和(16.32)所示的潜变量分布和条件分布, 并且考虑到对潜变量的后验分布的期望, 我们得到

$$\begin{aligned} \mathbb{E}\left[\ln p(\boldsymbol{X}, \boldsymbol{Z} | \boldsymbol{\mu}, \boldsymbol{W}, \sigma^2) \right] = -\sum_{n=1}^{N} \Bigg\{ & \frac{D}{2} \ln(2\pi\sigma^2) + \frac{1}{2} \operatorname{Tr}\left(\mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] \right) \\ & + \frac{1}{2\sigma^2} \| \boldsymbol{x}_n - \boldsymbol{\mu} \|^2 - \frac{1}{\sigma^2} \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}} \boldsymbol{W}^{\mathsf{T}} (\boldsymbol{x}_n - \boldsymbol{\mu}) \\ & + \frac{1}{2\sigma^2} \operatorname{Tr}\left(\mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] \boldsymbol{W}^{\mathsf{T}} \boldsymbol{W} \right) + \frac{M}{2} \ln(2\pi) \Bigg\}. \end{aligned} \tag{16.64}$$

注意, 此情况仅通过 Gauss 分布的充分统计量与后验分布产生关联. 因此, 在 E 步骤中, 我们使用旧的参数值来计算期望:

$$\mathbb{E}[\boldsymbol{z}_n] = \boldsymbol{M}^{-1} \boldsymbol{W}^{\mathsf{T}} (\boldsymbol{x}_n - \bar{\boldsymbol{x}}), \tag{16.65}$$

$$\mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] = \sigma^2 \boldsymbol{M}^{-1} + \mathbb{E}[\boldsymbol{z}_n] \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}}. \tag{16.66}$$

以上这些可以直接从后验分布 [(16.42)] 和标准结果 $\mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] = \operatorname{cov}[\boldsymbol{z}_n] + \mathbb{E}[\boldsymbol{z}_n] \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}}$ 中得出. 这里的 $\boldsymbol{M}$ 由(16.41)定义.

在 M 步骤中, 我们固定后验统计量, 并针对 $\boldsymbol{W}$ 和 σ^2 进行最大化. 针对 σ^2 的最大化相对简单, 针对 $\boldsymbol{W}$ 的最大化 (见练习 16.21) 则需要我们利用(A.27)获得 M 步骤的如下方程:

$$\boldsymbol{W}_{\text{new}} = \left[\sum_{n=1}^{N} (\boldsymbol{x}_n - \bar{\boldsymbol{x}}) \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}} \right] \left[\sum_{n=1}^{N} \mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] \right]^{-1}, \tag{16.67}$$

$$\begin{aligned} (\sigma^2)_{\text{new}} = \frac{1}{ND} \sum_{n=1}^{N} \Bigg\{ & \| \boldsymbol{x}_n - \bar{\boldsymbol{x}} \|^2 - 2 \mathbb{E}[\boldsymbol{z}_n]^{\mathsf{T}} \boldsymbol{W}_{\text{new}}^{\mathsf{T}} (\boldsymbol{x}_n - \bar{\boldsymbol{x}}) \\ & + \operatorname{tr}\left(\mathbb{E}[\boldsymbol{z}_n \boldsymbol{z}_n^{\mathsf{T}}] \boldsymbol{W}_{\text{new}}^{\mathsf{T}} \boldsymbol{W}_{\text{new}} \right) \Bigg\}. \end{aligned} \tag{16.68}$$

概率 PCA 的 EM 算法将首先初始化参数, 然后交替地在 E 步骤中使用(16.65)和(16.66)计算潜空间后验分布的充分统计量, 并在 M 步骤中使用(16.67)和(16.68)修正参数值.

命题 16.8 (概率 PCA 的 EM 算法的单调性)

设 $\boldsymbol{\theta}=(\boldsymbol{W},\sigma^2)$, 并记 $\ln p(\boldsymbol{X}|\boldsymbol{\theta})$ 为数据对数似然. 在小节 16.3.1的 EM 迭代中, 若第 t 轮参数为 $\boldsymbol{\theta}^{(t)}$, 则对每一轮 t 都有

$$\ln p\left(\boldsymbol{X}\middle|\boldsymbol{\theta}^{(t+1)}\right)\geqslant\ln p\left(\boldsymbol{X}\middle|\boldsymbol{\theta}^{(t)}\right),$$

即对数似然函数沿 EM 迭代单调不减; 若 $\boldsymbol{\theta}^{(t+1)}=\boldsymbol{\theta}^{(t)}$, 则对数似然不再增加, 迭代已到达驻点.

证明

(重构新增) 记第 t 轮 E 步骤选定的变分分布为 $q^{(t)}(\boldsymbol{Z})=p(\boldsymbol{Z}|\boldsymbol{X},\boldsymbol{\theta}^{(t)})$, 第 t 轮 M 步骤把 $\boldsymbol{\theta}$ 更新为 $\boldsymbol{\theta}^{(t+1)}=\arg\max_{\boldsymbol{\theta}}\mathcal{L}(q^{(t)},\boldsymbol{\theta})$. 由命题 16.7,

$$\ln p(\boldsymbol{X}|\boldsymbol{\theta})=\mathcal{L}(q,\boldsymbol{\theta})+\mathrm{KL}\left(q(\boldsymbol{Z})\|p(\boldsymbol{Z}|\boldsymbol{X},\boldsymbol{\theta})\right),\qquad\mathrm{KL}\geqslant 0.$$

在 E 步骤中取 $q=q^{(t)}$ 时 Kullback-Leibler 散度为零, 故

$$\mathcal{L}\left(q^{(t)},\boldsymbol{\theta}^{(t)}\right)=\ln p\left(\boldsymbol{X}\middle|\boldsymbol{\theta}^{(t)}\right).$$

在 M 步骤中 $\boldsymbol{\theta}^{(t+1)}$ 使 $\mathcal{L}(q^{(t)},\cdot)$ 取最大, 故

$$\mathcal{L}\left(q^{(t)},\boldsymbol{\theta}^{(t+1)}\right)\geqslant\mathcal{L}\left(q^{(t)},\boldsymbol{\theta}^{(t)}\right)=\ln p\left(\boldsymbol{X}\middle|\boldsymbol{\theta}^{(t)}\right).$$

再用一次下界不等式 (此时 $q=q^{(t)}$、$\boldsymbol{\theta}=\boldsymbol{\theta}^{(t+1)}$), 得

$$\ln p\left(\boldsymbol{X}\middle|\boldsymbol{\theta}^{(t+1)}\right)\geqslant\mathcal{L}\left(q^{(t)},\boldsymbol{\theta}^{(t+1)}\right)\geqslant\ln p\left(\boldsymbol{X}\middle|\boldsymbol{\theta}^{(t)}\right),$$

即对数似然单调不减.

若 $\boldsymbol{\theta}^{(t+1)}=\boldsymbol{\theta}^{(t)}$, 则上面两处不等号都取等号: 第二处取等号说明 $\mathrm{KL}(q^{(t)}\|p(\boldsymbol{Z}|\boldsymbol{X},\boldsymbol{\theta}^{(t)}))=0$, 第一处取等号说明 $\boldsymbol{\theta}^{(t)}$ 已是 $\mathcal{L}(q^{(t)},\cdot)$ 的最大值点. 由 $\mathcal{L}$ 与 $\ln p$ 在 $\boldsymbol{\theta}^{(t)}$ 处的值相等,$\boldsymbol{\theta}^{(t)}$ 也是 $\ln p(\boldsymbol{X}|\cdot)$ 的驻点.

□

注

(重构新增) 单调性是说 EM "不会把似然弄差", 这是 EM 能作为通用算法使用的最基本保证. 证明本身只用 ELBO 分解与非负性, 没有用到概率 PCA 的具体公式, 因此对任何满足正则性条件的潜变量模型都成立; 概率 PCA 的特殊之处只在于 E 步骤与 M 步骤都有闭式解, 所以每次迭代都可以精确执行.

条件方面: 证明要求 M 步骤的解存在 ((16.67)中的矩阵 $\sum_n\mathbb{E}[\boldsymbol{z}_n\boldsymbol{z}_n^{\mathsf{T}}]$ 可逆). 这个矩阵等于 $N(\sigma^2\boldsymbol{M}^{-1}+$ 后验均值的外积), 由 $\sigma^2>0$ 保证正定, 故可逆; 若 $\sigma^2=0$(即小节 16.3.2的极限情形), 需要另行论证, 那里用的是"类 EM"的表述. 另外, 单调性只保证收敛到驻点, 不保证收敛到全局最大值; 概率 PCA 的全局最大值由定理 16.1给出,EM 的极限点是否为其一取决于初值.

概率 PCA 的 EM 算法的一个好处是, 其在大规模应用中的计算效率较高 (Roweis, 1998). 与基于样本协方差矩阵的特征向量分解的传统 PCA 不同,EM 算法是迭代的, 因此可能看起来不那么吸引人. 然而,EM 算法的每个循环在高维空间中可能比传统 PCA 的计算效率要高得多. 为了理解这一点, 请注意, 协方差矩阵的特征分解需要 $O(D^3)$ 次计算. 通常我们只对前 M 个特征向量及对应的特征值感兴趣, 在这种情况下我们可以使用计算成本仅为 $O(MD^2)$ 的算法. 然而, 计算协方差矩阵需要 $O(ND^2)$ 的计算成本, 其中 N 是数据点的数量. 快照方法 (Sirovich, 1987) 等算法假设特征向量是数据向量的线性组合, 这样虽然避免了直接计算协方差矩阵, 但是需要 $O(N^3)$ 的计算成本, 因此不适用于大数据集. 这里描述的 EM 算法也没有显式构造协方差矩阵. 相反, 计算最密集的步骤涉及数据集的求和, 其复杂度为 $O(NDM)$. 对于很大的维度 D,M 要远小于 D, 与 $O(ND^2)$ 相比, 这可以显著节省计算成本, 并且可以抵消 EM 算法的迭代成本.

请注意, 这个 EM 算法可以通过在线方式实现, 依次读取、处理每个 D 维数据点, 然后丢弃并考虑下一个数据点. 请注意, 那些在 E 步骤中计算的量 (一个 M 维向量和一个 $M\times M$ 矩阵) 可以针对每个数据点单独计算. 在 M 步骤中, 需要对数据点进行累加求和, 这可以逐步进行. 如果 N 和 D 都很大, 这种方法可能会有优势.

我们现在有了一个完全概率化的 PCA 模型. 我们可以处理缺失数据, 前提是这些数据是随机缺失的 (missing

at random). 换句话说, 决定哪些值缺失的过程不依赖于任何观测到或未观测到的变量的值. 这样的数据集可以通过边缘化未观测变量的分布来处理, 生成的似然函数可以使用 EM 算法来最大化 (见练习 16.22).

16.3.2 PCA 的 EM 算法

EM 算法的另一个优雅特性是, 我们可以取极限 $\sigma^2 \to 0$, 这对应于传统 PCA, 并且仍然是一个有效的类 EM 算法 (Roweis, 1998). 从(16.66)中可以看到, 在 E 步骤中唯一需要计算的量是 $\mathbb{E}[\boldsymbol{z}_n]$. 此外, 因为 $\boldsymbol{M} = \boldsymbol{W}^\mathsf{T}\boldsymbol{W}$, 所以 M 步骤也简化了. 为了强调算法的简洁性, 定义 $\widetilde{\boldsymbol{X}}$ 为一个大小为 $N \times D$ 的矩阵, 其第 n 行由向量 $(\boldsymbol{x}_n - \bar{\boldsymbol{x}})^\mathsf{T}$ 给出. 类似地, 定义 $\boldsymbol{\Omega}$ 为一个大小为 $M \times N$ 的矩阵, 其第 n 列由 $\mathbb{E}[\boldsymbol{z}_n]$ 给出.PCA 的 EM 算法的 E 步骤 [(16.65)] 变为

$$\boldsymbol{\Omega} = \left(\boldsymbol{W}_{\text{old}}^\mathsf{T}\boldsymbol{W}_{\text{old}}\right)^{-1}\boldsymbol{W}_{\text{old}}^\mathsf{T}\widetilde{\boldsymbol{X}}^\mathsf{T}, \tag{16.69}$$

M 步骤 [(16.67)] 变为

$$\boldsymbol{W}_{\text{new}} = \widetilde{\boldsymbol{X}}^\mathsf{T}\boldsymbol{\Omega}^\mathsf{T}\left(\boldsymbol{\Omega}\boldsymbol{\Omega}^\mathsf{T}\right)^{-1}. \tag{16.70}$$

以上这些也可以通过在线学习的方式实现. 这些方程的简单解释如下. 根据我们之前的讨论,E 步骤涉及将数据点正交投影到当前估计的主子空间. 相应地,M 步骤表示在固定投影的情况下重新估计主子空间以最小化重建误差.

我们可以用一个简单的物理类比理解这个 EM 算法, 它在 $D = 2$ 和 $M = 1$ 时很容易描绘. 考虑二维空间中的一系列数据点, 用一根坚硬的棒子代表一维主子空间. 然后通过符合 Hooke 定律 (力与弹簧伸长量成正比, 因此储能与弹簧伸长量的平方成正比) 的弹簧将每个数据点连接到棒子上. 在 E 步骤中, 我们保持棒子固定并允许连接点沿着棒子上下滑动以最小化能量. 这会导致每个连接点 (独立地) 将自己定位在相应数据点到棒子的正交投影位置. 在 M 步骤中, 我们保持连接点固定, 然后释放棒子, 并允许棒子移动到最小能量位置. 然后重复 E 步骤和 M 步骤, 直至满足适当的收敛标准, 如图 16.10所示.

16.3.3 因子分析的 EM 算法

我们可以使用最大似然法来确定因子分析模型中的参数 $\boldsymbol{\mu}$、$\boldsymbol{W}$ 和 $\boldsymbol{\Psi}$.$\boldsymbol{\mu}$ 的解再次由样本均值给出 (参见小节 16.2.4). 然而, 与概率 PCA 不同的是,$\boldsymbol{W}$ 不再有闭式的最大似然解, 只能通过迭代来找到. 由于因子分析模型是一个潜变量模型, 因此可以使用类似于概率 PCA 的 EM 算法来完成这项工作 (见练习 16.24)(Rubin and Thayer, 1982). 具体来说,E 步骤方程为

$$\mathbb{E}[\boldsymbol{z}_n] = \boldsymbol{G}\boldsymbol{W}^\mathsf{T}\boldsymbol{\Psi}^{-1}(\boldsymbol{x}_n - \bar{\boldsymbol{x}}), \tag{16.71}$$

$$\mathbb{E}[\boldsymbol{z}_n\boldsymbol{z}_n^\mathsf{T}] = \boldsymbol{G} + \mathbb{E}[\boldsymbol{z}_n]\mathbb{E}[\boldsymbol{z}_n]^\mathsf{T}, \tag{16.72}$$

其中

$$\boldsymbol{G} = \left(\boldsymbol{I} + \boldsymbol{W}^\mathsf{T}\boldsymbol{\Psi}^{-1}\boldsymbol{W}\right)^{-1}. \tag{16.73}$$

注意这是以一种涉及对 $M \times M$ 而不是 $D \times D$ 的矩阵 (除了 $D \times D$ 对角矩阵 $\boldsymbol{\Psi}$, 其逆在 $O(D)$ 步内可以轻松计算) 求逆的形式来表示的, 这很方便, 因为通常情况下 $M \ll D$(见练习 16.25).

同样,M 步骤方程为

$$\boldsymbol{W}_{\text{new}} = \left[\sum_{n=1}^{N}(\boldsymbol{x}_n - \bar{\boldsymbol{x}})\mathbb{E}[\boldsymbol{z}_n]^\mathsf{T}\right]\left[\sum_{n=1}^{N}\mathbb{E}[\boldsymbol{z}_n\boldsymbol{z}_n^\mathsf{T}]\right]^{-1}, \tag{16.74}$$

$$\boldsymbol{\Psi}_{\text{new}} = \text{diag}\left\{\boldsymbol{S} - \boldsymbol{W}_{\text{new}}\frac{1}{N}\sum_{n=1}^{N}\mathbb{E}[\boldsymbol{z}_n](\boldsymbol{x}_n - \bar{\boldsymbol{x}})^\mathsf{T}\right\}, \tag{16.75}$$

其中 diag 运算符会将矩阵的所有非对角元素设置为零.

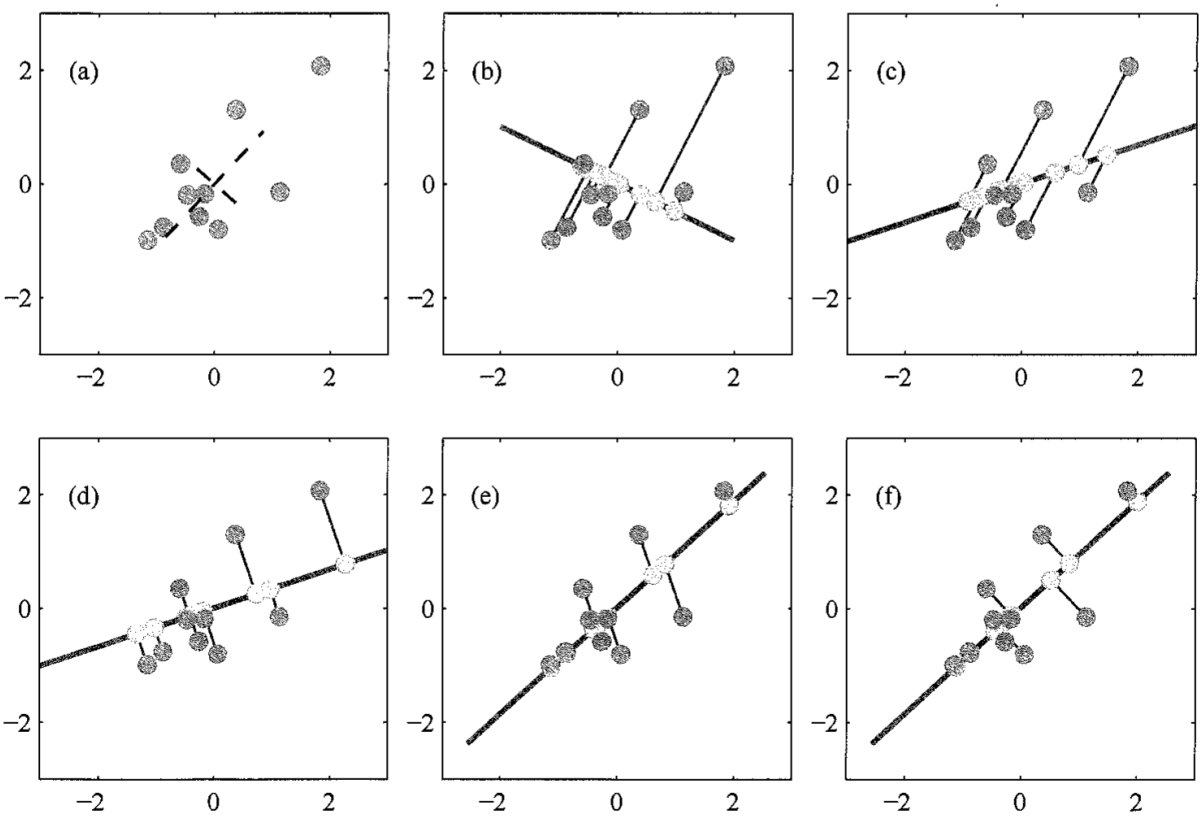


图 16.10: 用一个简单的物理类比来说明 PCA 的 EM 算法.(a) 一组绿色的数据点以及真实的主成分 (显示为按特征值平方根缩放的特征向量);(b) 由 $\boldsymbol{W}$ 定义的主子空间的初始配置 (显示为红色), 以及潜在点 $\boldsymbol{Z}$ 在数据空间中的投影 (由 $\boldsymbol{Z}\boldsymbol{W}^{\mathsf{T}}$ 给出, 显示为青色);(c) 经过第一个 M 步骤后,$\boldsymbol{W}$ 已经在 $\boldsymbol{Z}$ 保持固定的情况下得到更新;(d) 在随后的 E 步骤中,$\boldsymbol{Z}$ 的值已更新并给出了正交投影, 并且 $\boldsymbol{W}$ 保持不变;(e) 经过第二个 M 步骤之后的结果;(f) 收敛后的解

注

(重构新增) 因子分析的 M 步骤与概率 PCA 的唯一差别在 $\boldsymbol{\Psi}$ 的更新: 概率 PCA 对所有方向共用标量 σ^2, 所以只能取一个全局平均; 因子分析允许每个坐标有自己的噪声方差, 于是 $\boldsymbol{\Psi}_{\text{new}}$ 取残差协方差 $\boldsymbol{S}-\boldsymbol{W}_{\text{new}}\frac{1}{N}\sum_n \mathbb{E}[\boldsymbol{z}_n](\boldsymbol{x}_n-\bar{\boldsymbol{x}})^{\mathsf{T}}$ 的对角部分, 非对角元被强制置零. 这也解释了为什么因子分析没有闭式解:$\boldsymbol{\Psi}$ 与 $\boldsymbol{W}$ 的更新互相依赖, 只能交替进行.

条件方面:$\boldsymbol{\Psi}$ 必须保持正定, 因此 diag{·} 中若出现非正的对角元, 算法会失效, 实践中需要加下界或改用其他优化方法.E 步骤中 $\boldsymbol{G}$ 的定义依赖 $\boldsymbol{\Psi}^{-1}$, 这正是“对角”这一假设的价值所在: 对角矩阵求逆只需 $O(D)$, 而 $\boldsymbol{G}$ 只涉及 $M\times M$ 矩阵的逆, 从而整个算法的复杂度与 D 呈线性关系.

16.4 非线性潜变量模型

到目前为止, 在本章中, 我们关注的是基于从潜空间到数据空间的线性变换的潜变量模型. 自然地, 我们会问: 是否可以利用深度神经网络的灵活性来表示更复杂的变换, 同时利用深度神经网络的学习能力让得到的分布适配于一个数据集? 考虑一个简单的向量变量 $\boldsymbol{z}$ 上的分布, 例如 Gauss 分布:

$$p_z(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0},\boldsymbol{I}). \tag{16.76}$$

假设我们使用一个由深度神经网络给出的函数 $\boldsymbol{x}=\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$ 来变换 $\boldsymbol{z}$, 其中 $\boldsymbol{w}$ 代表权重和偏置.$\boldsymbol{z}$ 上的分布与神经网络的结合定义了 $\boldsymbol{x}$ 上的分布. 从这样的模型中采样是直截了当的, 因为我们可以从 $p_z(\boldsymbol{z})$ 中生成样本, 然后使用神经网络函数对每个样本进行变换, 以获得对应的 $\boldsymbol{x}$ 的样本. 这是一个高效的过程, 因为它不涉及迭代.

16.4.1 非线性流形

要求潜空间和数据空间具有相同的维度是一个重大的限制. 考虑这样一种情况:$\boldsymbol{z}$ 的维度是 M,$\boldsymbol{x}$ 的维度是 D, 其中 $M<D$. 在这种情况下,$\boldsymbol{x}$ 上的分布局限于维度为 M 的流形或子空间, 如图 16.11所示. 许多机器学习应用中都出现了低维流形, 例如在模拟自然图像的分布时. 非线性潜变量模型 (nonlinear latent variable model) 在建模此类数据时可能非常有用, 因为它们表达了强烈的归纳偏置, 即数据并不“填满”数据空间, 而是被限制在一个流形上, 尽管这个流形的形状和维度通常是未知的.

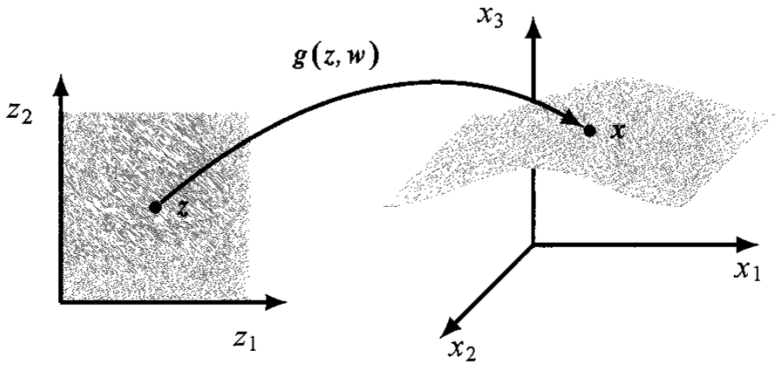


图 16.11: 使用一个带有参数向量 $\boldsymbol{w}$ 的神经网络所代表的非线性函数 $\boldsymbol{x}=\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$, 演示从二维潜空间 $\boldsymbol{z}=(z_1,z_2)$ 到三维数据空间 $\boldsymbol{x}=(x_1,x_2,x_3)$ 的一个映射

为了从数据中学习 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$, 考虑如何计算似然函数 $p(\boldsymbol{x}|\boldsymbol{w})$.$\boldsymbol{x}$ 上的分布由密度的变量变换公式给出:

$$p_x(\boldsymbol{x}) = p_z(\boldsymbol{z}(\boldsymbol{x}))\,|\det \boldsymbol{J}(\boldsymbol{x})|\,, \tag{16.77}$$

其中 $\boldsymbol{J}$ 是偏导数的 Jacobi 矩阵, 其元素由下式给出:

$$J_{ij}(\boldsymbol{x}) = \frac{\partial z_i}{\partial x_j}. \tag{16.78}$$

要计算(16.77)右侧的分布 $p_z(\boldsymbol{z}(\boldsymbol{x}))$(对于给定的数据向量 $\boldsymbol{x}$) 和(16.78)中同一 $\boldsymbol{x}$ 值的 Jacobi 矩阵, 我们需要使用神经网络函数的逆函数 $\boldsymbol{z}=\boldsymbol{g}^{-1}(\boldsymbol{x},\boldsymbol{w})$. 对于大多数神经网络来说, 这个逆函数将不是良好定义的. 例如, 神经网络可能实现一种多对一的函数关系, 在这种情况下, 不同的输入值会被映射到相同的输出值, 变量变换公式并不能给出良好定义的密度. 此外, 如果潜空间的维度与数据空间的维度不同, 那么变换将是不可逆的.

解决这个问题的一种方法是限制函数 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$ 是可逆的, 这要求 $\boldsymbol{z}$ 和 $\boldsymbol{x}$ 具有相同的维度. 我们将在介绍标准化流 (normalizing flow) 的技术时更详细地探索这种方法 (参见第 18 章).

命题 16.9 (流形上的分布是奇异的)

设 $\boldsymbol{g}:\mathbb{R}^M\to\mathbb{R}^D$ 是连续可微映射,$M<D$, 且潜变量 $\boldsymbol{z}$ 的分布有密度. 若取无噪声的确定性变换 $\boldsymbol{x}=\boldsymbol{g}(\boldsymbol{z})$, 则 $\boldsymbol{x}$ 的分布关于 $\mathbb{R}^D$ 上的 Lebesgue 测度是奇异的: 不存在密度函数 $p_x(\boldsymbol{x})$ 使得 $\boldsymbol{x}$ 的分布等于 p_x 的积分. 因此(16.77)在 $M<D$ 时不能作为密度的定义式.

证明

(重构新增) $\boldsymbol{g}$ 在 $\mathbb{R}^M$ 的每个紧集上 Lipschitz, 故 $\boldsymbol{g}(\mathbb{R}^M)$ 可表为可数个紧集之并的像, 而每个这样的像的 Hausdorff 维数不超过 M, 从而 $\mathbb{R}^D$ 中的 Lebesgue 测度为零. 形式地, 对任意 $\varepsilon>0$, 可用可数个边长之和任意小的立方体覆盖 $\boldsymbol{g}(\mathbb{R}^M)$, 故其 Lebesgue 测度为零. 于是

$$P\left(\boldsymbol{x}\in\boldsymbol{g}(\mathbb{R}^M)\right)=1,\qquad \left|\boldsymbol{g}(\mathbb{R}^M)\right|_D=0.$$

若存在密度 p_x 表示该分布, 则 $1=\int_{\boldsymbol{g}(\mathbb{R}^M)}p_x(\boldsymbol{x})\mathrm{d}\boldsymbol{x}=0$, 矛盾. 故不存在这样的密度.

另一方面,(16.77)中的 $\det\boldsymbol{J}(\boldsymbol{x})$ 要求 $\boldsymbol{J}$ 是方阵, 即 $\boldsymbol{z}$ 与 $\boldsymbol{x}$ 同维; 当 $M<D$ 时 $\boldsymbol{J}(\boldsymbol{x})$ 是 $M\times D$ 的长方阵, 行列式没有定义,(16.77)本身无法写成一个数.

□

注

(重构新增) 这个命题解释了为什么“纯流形模型”无法直接用最大似然训练: 数据在 $\mathbb{R}^D$ 中占据一个零测集, 似然要么无定义, 要么在流形外处处为零、在流形上无界, 基于梯度的优化得不到有用的信号. 原文所说的“这个框架将零概率密度赋予那些不完全位于流形上的任何数据向量”正是这一现象的非正式表述.

条件的作用: 结论依赖 $M<D$ 与 $\boldsymbol{g}$ 的正则性 (连续可微已足够). 若 $M=D$ 且 $\boldsymbol{g}$ 可逆、Jacobi 处处非奇异, 则(16.77)成立并给出 $\mathbb{R}^D$ 上的合法密度, 这正是标准化流所要求的结构 (见第 18 章). 解决办法有两个方向: 其一, 保持 $M<D$ 但显式加入噪声, 使边缘分布在所有方向上都有正的密度 (这正是(16.79)的做法); 其二, 令 $M=D$ 并约束 $\boldsymbol{g}$ 可逆.

然而, 这个框架将零概率密度赋予那些不完全位于流形上的任何数据向量, 这对基于梯度的学习算法是一个问题, 因为对于真实的数据集, 似然函数在每一个数据点都将为零, 并且对于小的 $\boldsymbol{w}$ 变化将保持不变. 为了解决这个问题, 我们使用之前用于解决回归和分类问题的策略, 定义了整个数据空间上的条件分布, 其参数由神经网络的输出决定. 例如, 如果 $\boldsymbol{x}$ 是连续变量的一个向量, 则可以选择条件分布为 Gauss 分布:

$$p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{w})=\mathcal{N}\left(\boldsymbol{x}\,\middle|\,\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w}),\sigma^2\boldsymbol{I}\right),\tag{16.79}$$

其中神经网络 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$ 的输出单元激活函数是线性的, 并且 $\boldsymbol{g}\in\mathbb{R}^D$. 生成式模型由 $\boldsymbol{z}$ 上的潜变量分布和 $\boldsymbol{x}$ 上的条件分布指定, 并且可以用图 16.12所示的简单图模型来表示.

定义 16.7 (非线性潜变量模型)

设 $\boldsymbol{z}$ 是 M 维连续潜变量,$\boldsymbol{g}(\cdot,\boldsymbol{w})$ 是由深度神经网络定义的、从潜空间到数据空间的非线性函数,$\boldsymbol{w}$ 为网络参数. 若

$$p(\boldsymbol{z})=p_z(\boldsymbol{z}),\qquad p(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{w})=\mathcal{N}\left(\boldsymbol{x}\,\middle|\,\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w}),\sigma^2\boldsymbol{I}\right),$$

则称该模型为非线性潜变量模型 (nonlinear latent variable model).

注

(重构新增) 非线性潜变量模型把节 16.2的线性映射 $\boldsymbol{W}\boldsymbol{z}+\boldsymbol{\mu}$ 换成神经网络 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$, 从而允许潜空间到数据空间的映射弯曲: 如图 16.11所示, 二维潜空间中的平面区域被映成三维空间中的一张曲面, 数据沿曲面分布而不是填满整个空间. 这就是“低维流形”这一归纳偏置的来源.

条件方面: 条件协方差取 $\sigma^2\boldsymbol{I}$ 有两个作用——它使边缘密度在整个 $\mathbb{R}^D$ 上为正 (于是似然处处有定义, 回避了命题 16.9的奇异问题), 并且它是各向同性的, 因而不引入额外的方向偏好;σ^2 同时充当“流形厚度”的尺度. 若把 σ^2 取得很小, 模型会逼近真正的流形模型, 但似然会变得非常尖锐 (见小节 16.4.2的讨论).

注意, 从这个分布中独立抽取样本不仅简单而且效率高. 首先使用标准方法从 Gauss 分布 [(16.76)] 中抽取一个样本. 然后将这个值作为神经网络的输入, 得到输出值 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$. 最后从均值为 $\boldsymbol{g}(\boldsymbol{z},\boldsymbol{w})$、协方差为 $\sigma^2\boldsymbol{I}$ 的 Gauss 分布中抽取一个样本 [(16.79)]. 重复上述过程以生成多个独立样本.

图 16.12: 由(16.76)和(16.79)定义的联合分布 $p(\boldsymbol{x},\boldsymbol{z})=p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})$ 的图模型

潜变量分布 $p(\boldsymbol{z})$ 与条件分布 $p(\boldsymbol{x}|\boldsymbol{z})$ 的组合定义了数据空间上的边缘分布:

$$p(\boldsymbol{x})=\int p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}. \tag{16.80}$$

我们可以用一个涉及一维潜空间和二维数据空间的简单例子来说明这一点, 见图 16.13.

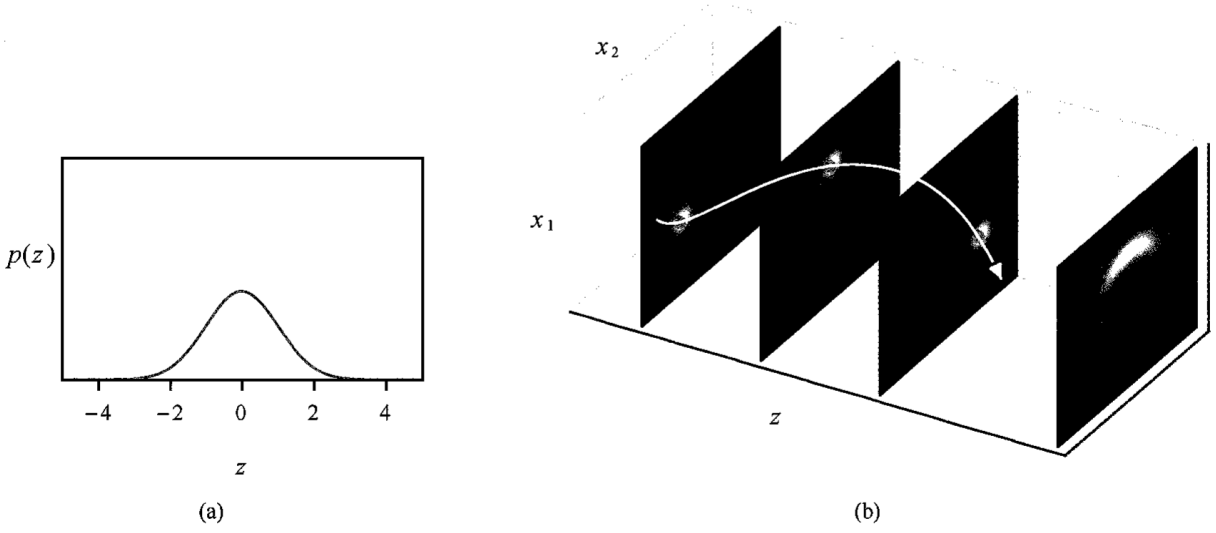


图 16.13: 一个涉及一维潜空间和二维数据空间的非线性潜变量模型的示意图.(a) 潜空间中的先验分布由零均值、单位方差的 Gauss 分布给出.(b) 最左侧的三个图显示了不同 z 值下的 Gauss 条件分布 $p(\boldsymbol{x}|z)$ 的示例, 最右侧的图显示了边缘分布 $p(\boldsymbol{x})$ 的示例. 定义条件分布均值的非线性函数 $\boldsymbol{g}(z)$ 由 $g_1(z)=\sin(z)$ 和 $g_2(z)=\cos(z)$ 给出, 条件分布的标准差由 $\sigma=0.3$ 给出 [经 Prince (2020) 授权使用]

例题 16.3 一维潜空间与圆周流形 在图 16.13的例子中,$M = 1$、$D = 2$,$p(z) = \mathcal{N}(z|0, 1)$, 条件分布均值为

$$\boldsymbol{g}(z) = \begin{bmatrix} \sin z \\ \cos z \end{bmatrix}, \qquad \sigma = 0.3.$$

由于 $\|\boldsymbol{g}(z)\| = 1$ 对一切 z 成立,$\boldsymbol{g}$ 的像是以原点为圆心、半径为 1 的圆周; 当 $\sigma \to 0$ 时, 边缘分布 $p(\boldsymbol{x})$ 集中在这条圆周上,z 到 $z + 2\pi$ 的平移给出同一个点, 因此该映射是无穷多对一的, 不存在全局逆. 由命题 16.9, 无噪声时 $\boldsymbol{x}$ 的分布关于平面 Lebesgue 测度奇异;$\sigma = 0.3$ 的噪声把圆周 “加粗” 成一条环带, 边缘密度才有定义, 且其等高线沿环带方向比沿径向平缓得多.

16.4.2 似然函数

我们已经看到从这个非线性潜变量模型中抽取样本是很容易的. 假设我们希望通过最大化似然函数来拟合观测到的数据集. 可以通过对 $\boldsymbol{z}$ 进行积分并使用概率的加和法则和乘积法则来得到似然值:

$$\begin{aligned} p(\boldsymbol{x}|\boldsymbol{w}) &= \int p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{w}) p(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z} \\ &= \int \mathcal{N}\left(\boldsymbol{x} \,\middle|\, \boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w}), \sigma^2 \boldsymbol{I}\right) \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I}) \, \mathrm{d}\boldsymbol{z}. \end{aligned} \tag{16.81}$$

尽管积分内部的两个分布都是 Gauss 分布, 但由神经网络定义的高度非线性函数 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$ 却使得积分在解析上是不可处理的. 一种计算似然函数的方法是从潜空间分布中抽取样本, 并用这些样本来近似(16.81), 从而得到

$$p(\boldsymbol{x}|\boldsymbol{w}) \approx \frac{1}{K} \sum_{i=1}^{K} p(\boldsymbol{x}|\boldsymbol{z}_i, \boldsymbol{w}), \tag{16.82}$$

其中 $\boldsymbol{z}_i \sim p(\boldsymbol{z})$. 这将 $\boldsymbol{z}$ 的分布表示为具有固定混合系数 $1/K$ 的 Gauss 混合分布, 当样本数量趋于无穷时, 该方法给出了真实的似然函数. 然而, 实际训练中所需的 K 值通常会大得不切实际. 为什么会这样? 请考虑图 16.14所示的三个手写数字图像, 并假设该图的 (a) 代表我们希望计算似然函数的向量 $\boldsymbol{x}$. 如果一个训练好的模型生成了该图的 (b), 我们会认为这是一个比较差的模型, 因为这个图像并不是数字 “2” 的一个很好的表示, 所以应该被赋予一个更低的似然值. 相反, 该图的 (c) 是通过将 (a) 中的数字向下和向右移动 0.5 个像素获得的, 是数字 “2” 的一个很好的表示, 因此应该被赋予一个更高的似然值. 由于是 Gauss 分布, 似然函数与网络输出和数据向量 $\boldsymbol{x}$ 之间负平方距离的指数成正比. 如果方差参数 σ^2 设置得足够小, 使得 (b) 具有低似然值, 那么 (c) 就会有更低的似然值. 即使模型在生成数字方面做得很好, 我们也必须抽取极大量的 $\boldsymbol{z}$ 样本, 才能找到一个足够接近 (a) 的数字图像. 因此, 我们需要寻求更精细的技术来训练非线性潜变量模型, 这些技术可以服务于实际应用. 在概述这些技术之前, 我们先简要讨论一些关于离散数据空间的考虑.

(a) (b) (c)

图 16.14: 三个手写数字图像, 旨在说明为什么从潜空间采样以计算似然函数需要大量的样本.(a) 原始图像;(b) 损坏的图像, 其中部分笔画被移除;(c) 将原始图像向下和向右各移动 0.5 个像素的结果. 尽管 (c) 在外观上更接近 (a), 但 (b) 在似然性方面更接近 (a) [经 Doersch (2016) 授权使用]

16.4.3 离散数据

如果观测到的数据集包含独立的二进制变量, 则可以使用以下形式的条件分布:

$$p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{w}) = \prod_{i=1}^{D} g_i(\boldsymbol{z}, \boldsymbol{w})^{x_i} \left(1 - g_i(\boldsymbol{z}, \boldsymbol{w})\right)^{1-x_i}, \tag{16.83}$$

其中 $g_i(\boldsymbol{z}, \boldsymbol{w}) = \sigma(a_i(\boldsymbol{z}, \boldsymbol{w}))$ 是输出单元 i 的激活值, 激活函数 $\sigma(\cdot)$ 由逻辑斯谛 sigmoid 函数给出,$a_i(\boldsymbol{z}, \boldsymbol{w})$ 是输出单元 i 的预激活值. 类似地, 对于独热编码 (one-hot encoded) 的分类变量, 可以使用多项式分布:

$$p(\boldsymbol{x}|\boldsymbol{z}, \boldsymbol{w}) = \prod_{i=1}^{D} g_i(\boldsymbol{z}, \boldsymbol{w})^{x_i}, \tag{16.84}$$

其中,softmax 激活函数为

$$g_i(\boldsymbol{z}, \boldsymbol{w}) = \frac{\exp\left(a_i(\boldsymbol{z}, \boldsymbol{w})\right)}{\sum_j \exp\left(a_j(\boldsymbol{z}, \boldsymbol{w})\right)}. \tag{16.85}$$

我们还可以通过构造相关条件分布的乘积来考虑离散变量和连续变量的组合.

在实际应用中, 连续变量通常以离散值表示, 例如在图像中, 红色、绿色和蓝色通道的强度可能取自 8 位二进制数字所能表示的数值集 $\{0, \cdots, 255\}$. 当我们采用基于深度神经网络的高度灵活模型时, 这可能会导致一个问题: 如果密度坍塌到一个或多个离散值上, 似然函数就有可能变为零, 导致异常解. 这个问题可以通过使用一种称为去量化 (dequantization) 的技术来解决, 该技术涉及向变量添加噪声, 这些噪声在连续离散值之间的区域内通常是均匀分布的, 如图 16.15所示. 训练集的去量化就是用随机抽取的样本替换每个观测值, 这些样本来自与特定离散值相关联的连续分布. 采用这种方法, 模型找到异常解的可能性就会降低.

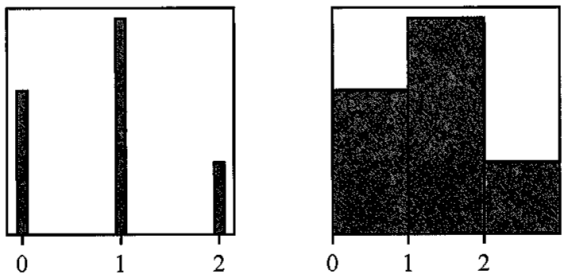


图 16.15: 去量化的示意图.(a) 单个变量上的离散分布;(b) 相关的去量化连续分布

16.4.4 构建生成式模型的 4 种方法

我们已经看到, 基于深度神经网络的非线性潜变量模型提供了一个高度灵活的框架来构建生成式模型. 由于神经网络变换的普适性, 这些模型原则上能够以高精度近似任何期望的分布. 此外, 一旦训练好, 这些模型还可以使用一种高效、非迭代的过程从分布中生成样本. 然而, 我们也面临着一些与训练此类模型相关的挑战, 迫切需要找到比线性模型更复杂的方法. 业内已经提出了不少方法, 每种方法都有自身的优势和局限性, 它们大致可以分为以下 4 种.

1. 基于生成对抗网络 (Generative Adversarial Network, GAN)(见第 17 章) 放宽 “网络映射必须是可逆的” 这一要求, 允许潜空间的维度低于数据空间的维度. 这里也放弃了似然函数的概念, 引入了第二个神经网络, 其功能是为生成网络提供训练信号. 由于缺乏明确定义的似然函数, 训练过程可能不稳定, 但一旦训练完成, 就很容易从模型中生成样本, 并且所得结果的质量也高.
2. 变分自编码器 (Variational AutoEncoder, VAE)(见第 19 章) 也使用了第二个神经网络, 其作用是近似潜变量的后验分布, 从而允许计算似然函数的近似值. 与 GAN 相比, 其能使训练更为稳健, 而且能直接从训练好的模型中采样, 但是要获得高质量的结果可能更困难.
3. 在标准化流 (normalizing flow)(见第 18 章) 中, 将潜空间的维度设置为等于数据空间的维度, 然后修改神经网络, 使其变得可逆. “网络必须是可逆的” 这一要求限制了其函数形式, 但也使得无须近似就能计算似然函数, 而且能进行高效采样.
4. 扩散模型 (diffusion model)(见第 20 章) 使用一个网络学习如何通过一系列去噪步骤将先验分布的样本转换为数据分布的样本. 这在许多应用中带来了先进的性能, 缺点是由于多次经历网络的去噪过程, 采样成本可能较高.

注

(重构新增) 这 4 种方法可以按“如何绕开命题 16.9造成的困难”来统一理解. 若坚持 $M < D$, 就必须处理边缘似然中的积分:GAN 干脆放弃似然、改用判别网络提供信号;VAE 用一个近似后验把积分化为可算的下界; 若坚持精确似然, 则必须让映射可逆, 于是取 $M = D$ 并约束网络结构, 这就是标准化流; 扩散模型则换了一个思路, 不去近似一次性的映射, 而是把生成分解为许多个小的去噪步骤, 每一步都是可逆的 Gaussian 扰动. 四者的取舍可以归纳为: 精确似然 (流)、近似似然 (VAE)、无似然 (GAN)、多步可逆变换 (扩散).

这一分类不是数学定理, 而是对建模策略的归纳, 因此不必逐条验证条件; 它的用处在于给出选择模型的判据: 若训练稳定性优先,VAE 或流更合适; 若样本质量优先且可以承受不稳定,GAN 更合适; 若追求当前最好的生成质量且能接受采样开销, 扩散模型更合适.

16.5 习题

练习 16.1 (★★) 使用归纳法证明, 将数据投影到 M 维子空间以最大化投影数据的方差所定义的线性投影是由数据协方差矩阵 $\boldsymbol{S}$[(16.3)] 的 M 个特征向量定义的, 对应于 M 个最大特征值. 在小节 16.1.1中, 这个结果已经在 $M = 1$ 的情况下得到证明. 假设该结果对某个一般的 M 值成立, 证明它因此对 $M + 1$ 也成立. 为此, 首先将投影数据的方差关于定义新方向的向量 $\boldsymbol{u}_{M+1}$ 的导数设置为零. 这应该在满足约束条件的情况下完成, 即 $\boldsymbol{u}_{M+1}$ 与现有向量 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M$ 正交, 并且已归一化至单位长度. 使用 Lagrange 乘子保证这些约束, 然后利用向量 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M$ 的正交性质证明新向量 $\boldsymbol{u}_{M+1}$ 是 $\boldsymbol{S}$ 的一个特征向量. 最后证明如果选择与第 $M + 1$ 个特征值相对应的特征向量 (特征值按降序排列), 则方差达到最大.

练习 16.2 (★★) 证明在满足正交性质 [(16.7)] 的情况下,PCA 误差度量 J[(16.16)] 的最小值是在 $\boldsymbol{u}_j$ 为数据协方差矩阵 $\boldsymbol{S}$ 的特征向量时获得的. 为此, 引入一个 Lagrange 乘子矩阵 $\boldsymbol{H}$, 每个约束对应一个 Lagrange 乘子, 因此修改后的误差度量 $\tilde{J}$ 用矩阵表示为

$$\tilde{J} = \operatorname{tr}\left[\widehat{\boldsymbol{U}}^{\mathsf{T}} \boldsymbol{S} \widehat{\boldsymbol{U}}\right] + \operatorname{tr}\left[\boldsymbol{H}\left(\boldsymbol{I} - \widehat{\boldsymbol{U}}^{\mathsf{T}} \widehat{\boldsymbol{U}}\right)\right], \tag{16.86}$$

其中 $\widehat{\boldsymbol{U}}$ 是一个 $D \times (D - M)$ 大小的矩阵, 其列由 $\boldsymbol{u}_j$ 给出. 对 $\widehat{\boldsymbol{U}}$ 求 $\tilde{J}$ 的最小值并证明解满足 $\boldsymbol{S}\widehat{\boldsymbol{U}} = \widehat{\boldsymbol{U}}\boldsymbol{H}$. 显然, 一个可能的解是,$\widehat{\boldsymbol{U}}$ 的列是 $\boldsymbol{S}$ 的特征向量. 在这种情况下,$\boldsymbol{H}$ 是一个包含对应特征值的对角矩阵. 假设 $\boldsymbol{H}$ 是一个对称矩阵, 并且通过使用特征向量将其展开, 证明 $\boldsymbol{S}\widehat{\boldsymbol{U}} = \widehat{\boldsymbol{U}}\boldsymbol{H}$ 的通用解 (对于给出的 $\tilde{J}$ 值) 与 $\widehat{\boldsymbol{U}}$ 的列是 $\boldsymbol{S}$ 的特征向量时的具体解相同. 因为这些解都是等效的, 所以选择特征向量解是很方便的.

练习 16.3 (★) 假设(16.30)中的特征向量 $\boldsymbol{v}_i$ 具有单位长度, 验证由该式定义的特征向量 $\boldsymbol{u}_i$ 已归一化至单位长度.

练习 16.4 (★) 假设在概率 PCA 模型中将零均值、单位协方差的潜空间分布 [(16.31)] 替换为一般形式的 Gauss 分布 $\mathcal{N}(\boldsymbol{z}|\boldsymbol{m}, \boldsymbol{\Sigma})$. 通过重新定义模型的参数, 证明这种替换会使得任意有效的 $\boldsymbol{m}$ 和 $\boldsymbol{\Sigma}$ 都无法改变观测变量边缘分布 $p(\boldsymbol{x})$ 的形式.

练习 16.5 (★★) 假设 $\boldsymbol{x}$ 是一个 D 维的随机变量, 具有 Gauss 分布 $\mathcal{N}(\boldsymbol{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma})$, 并考虑由 $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$ 给出的另一个 M 维随机变量, 其中 $\boldsymbol{A}$ 是一个 $M \times D$ 大小的矩阵. 证明 $\boldsymbol{y}$ 也具有 Gauss 分布, 并找出其均值和协方差. 讨论此 Gauss 分布在 $M < D$、$M = D$ 和 $M > D$ 情况下的形式.

练习 16.6 (★★) 利用一般分布的均值和协方差在仿射变换下的公式, 推导概率 PCA 模型中边缘分布 $p(\boldsymbol{x})$ 的结果 [(16.35), 协方差见(16.36)].

练习 16.7 (★) 绘制小节 16.2.1描述的概率 PCA 模型的有向图, 并将观测变量 $\boldsymbol{x}$ 的各个组成部分明确显示为单独的节点. 验证概率 PCA 模型具有与小节 11.2.4讨论的朴素 Bayes 模型相同的独立结构.

练习 16.8 (★★) 利用线性-Gauss 模型的结果, 证明概率 PCA 模型的后验分布 $p(\boldsymbol{z}|\boldsymbol{x})$ 可以由(16.42)给出.

练习 16.9 (★) 验证对概率 PCA 模型的对数似然函数 [(16.43)] 关于参数 $\boldsymbol{\mu}$ 进行最大化会得到结果 $\boldsymbol{\mu} = \bar{\boldsymbol{x}}$, 其中 $\bar{\boldsymbol{x}}$ 是数据向量的均值.

练习 16.10 (★★) 通过计算对数似然函数 [(16.43)] 对参数 $\boldsymbol{\mu}$ 的二阶导数, 证明驻点 $\boldsymbol{\mu} = \bar{\boldsymbol{x}}$ 代表唯一的最大值.

练习 16.11 (★★) 证明当 $\sigma^2 \to 0$ 时, 概率 PCA 模型的后验均值相当于对主子空间的正交投影, 这与传统 PCA 模型的结果一致.

✍ **练习 16.12** (★★) 对于 $\sigma^2 > 0$, 证明概率 PCA 模型中的后验均值会相对于正交投影向原点移动.

✍ **练习 16.13** (★★) 证明在概率 PCA 下, 根据传统 PCA 的最小二乘投影成本, 数据点的最优重建由下式给出:

$$\tilde{\boldsymbol{x}} = \boldsymbol{W}_{\mathrm{ML}} \left(\boldsymbol{W}_{\mathrm{ML}}^{\mathsf{T}} \boldsymbol{W}_{\mathrm{ML}} \right)^{-1} \boldsymbol{M}\, \mathbb{E}[\boldsymbol{z}|\boldsymbol{x}]. \tag{16.87}$$

✍ **练习 16.14** (★) 概率 PCA 模型中协方差矩阵的独立参数数量由(16.51)给出. 证明对于 $M = D - 1$, 独立参数的数量与一般 Gauss 分布中的相同; 而对于 $M = 0$, 独立参数的数量与各向同性协方差 Gauss 分布相同.

✍ **练习 16.15** (★) 推导出小节 16.2.4描述的因子分析模型中独立参数数量的计算表达式.

✍ **练习 16.16** (★★) 证明小节 16.2.4描述的因子分析模型不随潜空间坐标旋转而变化.

✍ **练习 16.17** (★★) 考虑一个线性-Gauss 潜变量模型, 它具有潜空间分布 $p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z}|\boldsymbol{0}, \boldsymbol{I})$ 和观测变量的条件分布 $p(\boldsymbol{x}|\boldsymbol{z}) = \mathcal{N}(\boldsymbol{x}|\boldsymbol{W}\boldsymbol{z} + \boldsymbol{\mu}, \boldsymbol{\Phi})$, 其中 $\boldsymbol{\Phi}$ 是任意对称且正定的噪声协方差矩阵. 对数据变量进行非奇异线性变换 $\boldsymbol{x} \to \boldsymbol{A}\boldsymbol{x}$, 其中 $\boldsymbol{A}$ 是一个 $D \times D$ 矩阵. 如果 $\boldsymbol{\mu}_{\mathrm{ML}}$、$\boldsymbol{W}_{\mathrm{ML}}$ 和 $\boldsymbol{\Phi}_{\mathrm{ML}}$ 代表原始未变换数据所对应的最大似然解, 证明 $\boldsymbol{A}\boldsymbol{\mu}_{\mathrm{ML}}$、$\boldsymbol{A}\boldsymbol{W}_{\mathrm{ML}}$ 和 $\boldsymbol{A}\boldsymbol{\Phi}_{\mathrm{ML}}\boldsymbol{A}^{\mathsf{T}}$ 代表变换后数据所对应的最大似然解. 最后证明在以下两种情况下模型的形式得以保留:(i) $\boldsymbol{A}$ 是对角矩阵且 $\boldsymbol{\Phi}$ 也是对角矩阵, 这对应于因子分析. 变换后的 $\boldsymbol{\Phi}$ 仍然是对角矩阵, 因此因子分析在各成分矩阵的变换下具有协变性.(ii) $\boldsymbol{A}$ 是正交矩阵且 $\boldsymbol{\Phi}$ 与单位矩阵成比例,$\boldsymbol{\Phi} = \sigma^2 \boldsymbol{I}$, 这对应于概率 PCA. 变换后的 $\boldsymbol{\Phi}$ 仍与单位矩阵成比例, 因此概率 PCA 在数据空间的轴旋转下具有协变性, 就像传统 PCA 一样.

✍ **练习 16.18** (★) 验证连续潜变量模型对数似然函数可以写成两项之和 [(16.56)], 这两项分别由(16.57)和(16.58)定义. 这可以通过使用概率的乘积法则来完成, 公式如下:

$$p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{w}) = p(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{w}) p(\boldsymbol{x}|\boldsymbol{w}), \tag{16.88}$$

然后将 $p(\boldsymbol{x}, \boldsymbol{z}|\boldsymbol{w})$ 代入(16.57)即可.

✍ **练习 16.19** (★) 证明对于一组独立同分布数据, 证据下界 (ELBO) 可以采用(16.62)的形式.

✍ **练习 16.20** (★★) 绘制一个有向图, 以表示概率 PCA 模型的离散混合模型, 其中的每个概率 PCA 模型都有自己的 $\boldsymbol{W}$、$\boldsymbol{\mu}$ 和 σ^2 值. 然后绘制一个修改后的有向图, 这些参数值在该有向图所表示的混合模型的各部分之间是共享的.

✍ **练习 16.21** (★★) 通过最大化(16.64)给出的完整数据对数似然函数的期望来推导概率 PCA 模型的 M 步骤方程 [(16.67)] 和 [(16.68)].

✍ **练习 16.22** (★★★) 概率 PCA 的一个优点是, 它可以应用于某些值缺失的数据集, 前提是这些值是随机缺失的. 在这种情况下, 推导出最大化概率 PCA 模型似然函数的 EM 算法. 注意,$\{\boldsymbol{z}_n\}$ 和向量 $\{\boldsymbol{x}_n\}$ 的缺失数据值现在都是潜变量. 证明在所有数据值都观测到的特殊情况下, 这可以简化为推导概率 PCA 的 EM 算法 (见小节 16.3.2).

✍ **练习 16.23** (★★) 设 $\boldsymbol{W}$ 是一个 $D \times M$ 矩阵, 其列定义了嵌入 D 维数据空间的 M 维线性子空间, 并且 $\boldsymbol{\mu}$ 是一个 D 维向量. 给定一个数据集 $\{\boldsymbol{x}_n\}$, 其中 $n = 1, \cdots, N$, 我们可以使用一组 M 维向量 $\{\boldsymbol{z}_n\}$ 的线性映射来近似数据点, 使得 $\boldsymbol{x}_n$ 能被 $\boldsymbol{W}\boldsymbol{z}_n + \boldsymbol{\mu}$ 近似. 相关的平方和重建成本由下式给出:

$$J = \sum_{n=1}^{N} \|\boldsymbol{x}_n - \boldsymbol{\mu} - \boldsymbol{W}\boldsymbol{z}_n\|^2 . \tag{16.89}$$

首先证明关于 $\boldsymbol{\mu}$ 最小化 J 会得到一个相似的表达式, 其中 $\boldsymbol{x}_n$ 和 $\boldsymbol{z}_n$ 分别替换为零均值变量 $\boldsymbol{x}_n - \bar{\boldsymbol{x}}$ 和 $\boldsymbol{z}_n - \bar{\boldsymbol{z}}$, 其中 $\bar{\boldsymbol{x}}$ 和 $\bar{\boldsymbol{z}}$ 表示样本均值. 然后证明在 $\boldsymbol{W}$ 保持不变的情况下关于 $\boldsymbol{z}_n$ 最小化 J 会产生 PCA EM 算法的 E 步骤 [(16.69)], 并且在 $\{\boldsymbol{z}_n\}$ 保持不变的情况下关于 $\boldsymbol{W}$ 最小化 J 会产生 PCA EM 算法的 M 步骤 [(16.70)].

✍ **练习 16.24** (★★) 推导因子分析 EM 算法的 E 步骤方程 [(16.71)和(16.72)]. 注意, 从练习 16.26的结果可知, 参数 $\boldsymbol{\mu}$ 可以用样本均值 $\bar{\boldsymbol{x}}$ 替换.

✍ **练习 16.25** (★★) 给出因子分析模型的完整数据对数似然函数的期望的表达式, 从而推导出相应的 M 步骤方程 [(16.74)和(16.75)].

✍ **练习 16.26** (★★) 通过考虑二阶导数, 证明小节 16.2.4讨论的因子分析模型的对数似然函数相对于参数 $\boldsymbol{\mu}$ 的唯一驻点是由(16.1)定义的样本均值给出的. 此外, 证明这个驻点是一个最大值.

第 17 章 生成对抗网络

生成式模型使用机器学习算法从一组训练数据中学习潜在的数据分布, 然后根据该分布生成新的样本. 例如, 一个生成式模型可以基于动物图像进行训练, 随后用于生成新的动物图像. 我们可以将这种生成式模型理解为一个条件分布 $p(x \mid w)$, 这里 x 是数据空间中的向量, 而 w 代表模型的可学习参数. 在很多情况下, 我们更倾向于使用形式为 $p(x \mid c, w)$ 的条件生成式模型, 其中 c 代表条件变量的向量. 以动物图像生成模型为例, 我们希望生成的图像应该是特定动物的, 比如猫或狗, 而这是由 c 的值指定的.

对于像图像生成这样的实际应用而言, 其分布极其复杂, 深度学习能显著提高生成式模型的性能. 在讨论基于 Transformer 的自回归大语言模型时, 我们已经了解了一些重要的深度生成式模型. 我们还概述了 4 种基于非线性潜变量模型的生成式模型 (参见第 12 章和小节 16.4.4), 本章将讨论其中的第一种——生成对抗网络 (Generative Adversarial Network, GAN), 其他三种将在后续各章中讨论.

17.1 对抗训练

考虑一个基于从潜空间 z 到数据空间 x 的非线性变换的生成式模型. 引入一个潜空间分布 $p(z)$, 它可能是一个简单的 Gauss 分布:

$$p(z) = \mathcal{N}(z \mid \mathbf{0}, \boldsymbol{I}). \tag{17.1}$$

同时定义一个由深度神经网络实现的非线性变换 $x = g(z, w)$, 其中可学习参数 w 对应的网络称为**生成器** (generator) 网络. 它们共同隐式地定义了 x 上的分布, 我们的目标是将这个分布拟合到一组训练样本 $\{x_n\}$ 上, 其中 $n = 1, \cdots, N$. 然而, 我们并不能通过优化似然函数来确定 w, 因为这通常无法以封闭形式来计算. 生成对抗网络 (Goodfellow et al., 2014; Ruthotto and Haber, 2021) 的关键思想是引入判别器 (critic) 网络, 判别器网络与生成器网络一起训练, 并为更新生成器的权重提供训练信号, 如图 17.1所示.

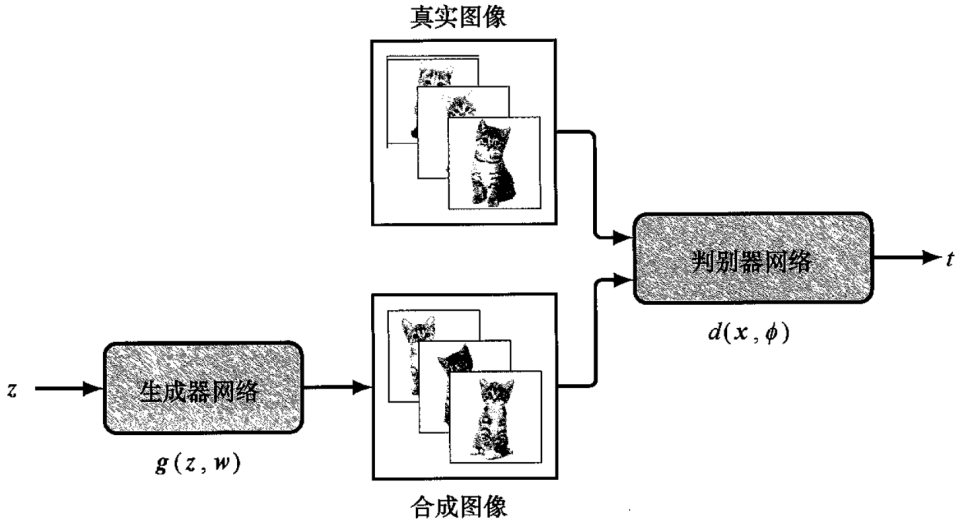


图 17.1: 生成对抗网络的示意图. 在这样的网络中, 训练判别器网络 $d(x, \phi)$ 以区分来自训练集的真实样本 (小猫的图像) 和由生成器网络 $g(z, w)$ 产生的合成样本. 生成器网络的目标是通过产生逼真的图像来让判别器网络产生更多的误判, 而判别器网络的目的是通过更好地区分真实样本和合成样本来减少误判

定义 17.1 (生成对抗网络)

设 $p(z)$ 是潜空间上的给定分布,$g(\cdot, w)$ 是参数为 w 的深度神经网络. 称 $g(\cdot, w)$ 为**生成器** (generator) 网络. 由 $z \sim p(z)$ 与 $x = g(z, w)$ 共同决定的 x 上的分布称为**生成分布**, 记作 $p_g(x)$. 称参数为 ϕ 的网络 $d(\cdot, \phi)$ 为**判别器** (critic) 网络, 它把数据向量 x 映射为 x 来自真实数据 (而不是来自生成分布) 的概率. 判别器网络与生成器网络一起训练、互为对手, 这一训练方式称为**对抗训练** (adversarial training).

♣

注

(重构新增) 这里把生成式模型写成“先取 $z \sim p(z)$、再作确定性变换 $x = g(z, w)$”的形式, 动机是绕开似然函数.GAN 并不要求能写出 p_g 的密度, 只要求能从 $p(z)$ 采样并把样本经 g 推送出去; 因此它是隐式 (implicit) 的生成式模型, 与第 16 章中以 $p(x \mid z, w)$ 的可求值为前提的非线性潜变量模型有本质区别: 后者通过最大化边缘似然来学习, 前者通过判别器提供的信号来学习.

定义中的每一项都有作用.$p(z)$ 被固定为一个简单分布 (如(17.1)的标准 Gauss 分布), 是把全部建模负担交给 g 的刻意选择: 若 $p(z)$ 也可学, 则它与 g 的重参数化不唯一, 模型不可识别.g 关于 w 的可微性是训练的前提, 因为生成器的梯度要经由 g 反传到 w. 判别器输出必须落在 $(0, 1)$ 内, 才能与(17.4)的概率解释相容, 并保证对数损失有定义.

最后要强调,p_g 一般没有关于 Lebesgue 测度的密度: 当 z 的维数低于 x 的维数时,p_g 集中在 g 的像 (一个低维流形) 上. 这正是理论分析中必须额外假定 p_g 与 p_{data} 处处有正密度的原因, 见命题 17.1.

判别器网络的目标是区分来自数据集的真实样本和由生成器网络的合成 (或“假”) 样本, 它通过最小化传统的分类误差函数来进行训练. 相反, 生成器网络的目标是通过生成与训练集同分布的样本来最大化这个误差. 因此, 生成器网络和判别器网络互相对抗, “生成对抗网络”由此得名.

定义 17.2 (零和博弈)

设两个参与者分别以 w 与 ϕ 为策略, 其收益分别为 $E(w, \phi)$ 与 $-E(w, \phi)$. 若一个参与者的收益恰好是另一个参与者的损失, 即两个收益之和恒为零, 则称这样的博弈为**零和博弈** (zero-sum game).

♣

注

(重构新增) 把对抗训练看作零和博弈, 是为了给“谁进步了”确立一个共同的标尺: 若双方的收益之和不为零, 就存在对双方都有利的方向, 对抗性随之消失,“生成器变强必然使判别器变难”这一直观也不再成立. 零和性还保证整个训练只有一个标量目标 $E(w, \phi)$, 于是(17.7)与(17.8)只是同一函数上的两个相反方向.

需要区分两件事: 零和是对训练目标的建模假设, 而收敛是动力学性质. 博弈为零和并不意味着梯度下降-上升一定收敛到鞍点; 恰恰相反,练习 17.2将给出一个零和目标 $E(a, b) = ab$ 的例子, 其梯度动力学绕鞍点作周期运动而永不收敛.

生成对抗网络通过允许判别器网络提供一个训练信号来训练生成器网络, 从而将无监督的密度建模问题转换成了一种监督学习的形式.

17.1.1 损失函数

为了明确这一点, 我们定义一个二元目标变量 t:

$$t = 1, \quad \text{代表真实数据} \tag{17.2}$$

$$t = 0, \quad \text{代表合成数据} \tag{17.3}$$

判别器网络拥有单个输出单元, 带有 sigmoid 激活函数, 其输出表示数据向量 x 为真实数据的概率:

$$P(t = 1) = d(x, \phi). \tag{17.4}$$

我们可以使用如下标准的交叉熵损失函数来训练判别器网络:

$$E(w, \phi) = -\frac{1}{N} \sum_{n=1}^{N} \Big\{ t_n \ln d_n + (1 - t_n) \ln(1 - d_n) \Big\}, \tag{17.5}$$

其中 $d_n = d(x_n, \phi)$ 是判别器网络的输出, 我们已经通过数据点的总数对它进行了归一化 (参见小节 1.2.4). 训练集包括标记为 x_n 的真实样本和由生成器网络 $g(z_n, w)$ 输出的合成样本, 其中 z_n 是从潜空间分布 $p(z)$ 中采样的随机样本. 对于真实样本,$t_n = 1$; 对于合成样本,$t_n = 0$. 我们可以将损失函数 [(17.5)] 写成以下形式:

$$E_{\text{GAN}}(w, \phi) = -\frac{1}{N_{\text{real}}} \sum_{n \in \text{real}} \ln d(x_n, \phi) - \frac{1}{N_{\text{synth}}} \sum_{n \in \text{synth}} \ln \big(1 - d(g(z_n, w), \phi)\big). \tag{17.6}$$

通常情况下, 真实数据点的数量 N_{real} 等于合成数据点的数量 N_{synth}. 这种生成器网络和判别器网络的组合可以使用随机梯度下降进行端到端的训练 (参见第 7 章), 并通过反向传播来计算梯度. 然而, 这种训练方式的独特之处在于其对抗性, 即误差相对于 ϕ 最小化, 而相对于 w 最大化.

这种最大化可以通过标准的基于梯度的方法来完成, 通过反转梯度的符号, 参数更新为

$$\Delta\phi = -\lambda \nabla_\phi E_n(w, \phi), \tag{17.7}$$

$$\Delta w = \lambda \nabla_w E_n(w, \phi), \tag{17.8}$$

其中 $E_n(w, \phi)$ 表示为数据点 n 定义的误差, 或者更一般地说, 是为小批量数据点定义的误差. 注意(17.7)和(17.8)右侧的符号不同, 因为判别器网络的训练目标是降低错误率, 而生成器网络的训练目标是提高错误率. 实际上, 训练交替地在更新生成器网络的参数和更新判别器网络的参数之间进行, 每次只使用一个小批量数据, 并采取一个梯度下降步骤, 之后生成一组新的合成样本.

如果生成器网络成功找到完美解决方案, 则判别器网络将无法区分真实数据和合成数据, 因此其输出始终为 0.5. 一旦 GAN 训练完毕, 判别器网络就会被舍弃, 生成器网络则用来从潜空间中采样并将这些样本经训练好的生成器网络加以传播, 从而在数据空间中合成新的样本. 我们可以证明, 对于具有无限灵活性的生成器网络和判别器网络, 完全优化的 GAN 将具有与数据分布完全匹配的生成分布 (见练习 17.1).图 1.3展示了一些由 GAN 生成的优秀的人脸图像.

到目前为止我们所讨论的 GAN 都是从无条件分布 $p(x)$ 中生成样本的. 例如, 如果我们用狗的图像对 GAN 进行训练, 它就会生成合成的狗的图像. 我们也可以创建**条件 GAN**(Mirza and Osindero, 2014), 它从条件分布 $p(x \mid c)$ 中采样, 其中条件向量 c 可能代表不同品种的狗.

定义 17.3 (条件 GAN)

设 c 是条件变量的向量. 若生成器网络与判别器网络都把 c 作为额外输入, 使生成器学习的是条件分布 $p_g(x \mid c)$、判别器判断的是“给定 c 时 x 是否来自真实数据”, 则称这样的生成对抗网络为**条件 GAN**(conditional GAN). 此时训练集由带标签的样本对 $\{(x_n, c_n)\}$ 组成. ♣

注

(重构新增) 条件 GAN 把“生成什么”从模型内部转移到输入上: 无条件 GAN 学的是边缘分布 $p(x)$, 它把数据中的所有模式混在一起; 条件 GAN 学的是 $p(x \mid c)$, 条件 c 指定了要生成哪一个模式. 这样做的好处不只是可控, 还包括参数共享: 所有类别共用同一个生成器, 因而样本少的类别也能从样本多的类别中学到通用的底层特征, 这正是原书所说的“跨所有类别学习共享的内部表示”.

条件方面要注意,c 必须同时进入生成器与判别器. 若只把 c 给生成器而不给判别器, 判别器就无法检查“生成的图像是否与 c 相符”, 生成器可以忽略 c 而仍然骗过判别器 (这与 CycleGAN 中“生成器无视输入”的退化同源, 见小节 17.2.1); 若只给判别器而不给生成器, 则生成器根本没有收到 c 的信息. 此外, 训练集必须带标签, 这一点与无条件 GAN 只需无标签样本不同.

为此, 生成器网络和判别器网络会将 c 作为额外输入, 并使用标有标签的图像实例 $\{x_n, c_n\}$ 对 GAN 进行训练. 一旦 GAN 训练好了, 我们就可以通过将 c 设置为相应的类向量来生成所需类别的图像. 与为每个类别训练单独的 GAN 相比, 这样做有一个优势, 就是可以跨所有类别学习共享的内部表示, 从而更有效地利用数据.

定义 17.4 (GAN 的极小极大目标)

在数据样本无限多的极限下,(17.6)中的两个经验平均收敛为相应的数学期望, 此时对抗训练的目标可写成

$$\min_{\phi}\max_{w} E_{\mathrm{GAN}}(w,\phi),$$

$$E_{\mathrm{GAN}}(w,\phi) = -\mathbb{E}_{x\sim p_{\mathrm{data}}}\big[\ln d(x,\phi)\big] - \mathbb{E}_{x\sim p_g}\big[\ln\big(1-d(x,\phi)\big)\big],$$

其中 p_{data} 是真实数据的分布,p_g 是由 $g(\cdot,w)$ 诱导的生成分布; 判别器相对于 ϕ 最小化 E_{GAN}, 生成器相对于 w 最大化 E_{GAN}. 称这样的目标为 GAN 的**极小极大目标** (minimax objective). ♣

注

(重构新增) 极小极大目标把“对抗”写成了一句话: 同一个标量函数 E_{GAN} 上, 一方求极小、一方求极大. 之所以要取极限形式, 是因为(17.6)是经验平均, 它的极小极大依赖于有限样本, 无法做解析分析; 取期望后, 问题才化为两个分布 p_{data} 与 p_g 之间的泛函优化, 从而可以讨论“最优判别器是什么”“生成器的最优值是多少”.

条件方面: 期望形式要求 p_{data} 与 p_g 都是概率分布且 $d(\cdot,\phi)$ 可测;min 与 max 不能交换, 因为 E_{GAN} 关于 ϕ 不凹、关于 w 不凸, 交换后会得到不同的值 (见练习 17.2的鞍点现象). 还要注意 $\max_w$ 是对生成器参数取的, 而 w 只通过 p_g 进入目标, 因此这一步实际上是“在所有可达的生成分布上取上确界”; 若把 $w\mapsto p_g$ 的像集扩大, 最优值只会变大而不会变小.

命题 17.1 (最优判别器与 Jensen-Shannon 散度)

设 p_{data} 与 p_g 是 $\mathbb{R}^D$ 上处处为正的连续密度, 记

$$E(p_g,d) = -\int p_{\mathrm{data}}(x)\ln d(x)\,\mathrm{d}x - \int p_g(x)\ln\big(1-d(x)\big)\,\mathrm{d}x,$$

其中 d 取遍 $\mathbb{R}^D$ 上取值于 $(0,1)$ 的可测函数. 则:

(i) 使 $E(p_g,d)$ 取最小值的判别器函数在几乎处处的意义下唯一, 由下式给出:

$$d^*(x) = \frac{p_{\mathrm{data}}(x)}{p_{\mathrm{data}}(x)+p_g(x)}; \tag{17.9}$$

(ii) 令 $M(x)=\frac{1}{2}\big(p_{\mathrm{data}}(x)+p_g(x)\big)$, 并记

$$\mathrm{JSD}(p_{\mathrm{data}}\|p_g) = \mathrm{KL}\big(p_{\mathrm{data}}\|M\big) + \mathrm{KL}\big(p_g\|M\big),$$

则把 d^* 代回后, 损失函数作为生成分布的函数可写成

$$C(p_g) \triangleq E(p_g,d^*) = \ln 4 - \mathrm{JSD}(p_{\mathrm{data}}\|p_g); \tag{17.10}$$

(iii)$\mathrm{JSD}(p_{\mathrm{data}}\|p_g)\geqslant 0$, 且等号成立当且仅当 $p_g = p_{\mathrm{data}}$ 几乎处处. 因此 $C(p_g)\leqslant \ln 4$, 生成器在最优判别器下的目标值 $\ln 4$ 当且仅当 $p_g = p_{\mathrm{data}}$ 时取到, 即生成器的最优策略是让 Jensen-Shannon 散度最小 (等于零). ♠

证明

(重构新增) 记 $V(p_g,d) = -E(p_g,d)$. 由于 E 与 V 只相差一个整体符号, 最小化 E 等价于最大化 V.

(i) 把 V 写成 $\int F\big(x,d(x)\big)\mathrm{d}x$, 其中

$$F(x,d) = p_{\mathrm{data}}(x)\ln d + p_g(x)\ln(1-d).$$

对固定的 x,F 关于 $d\in(0,1)$ 二阶可微且

$$\frac{\partial^2 F}{\partial d^2} = -\frac{p_{\mathrm{data}}(x)}{d^2} - \frac{p_g(x)}{(1-d)^2} < 0,$$

故 $F(x,\cdot)$ 严格凹, 一阶条件

$$\frac{\partial F}{\partial d} = \frac{p_{\mathrm{data}}(x)}{d} - \frac{p_g(x)}{1-d} = 0$$

给出唯一驻点 $d^*(x) = p_{\mathrm{data}}(x)/\big(p_{\mathrm{data}}(x)+p_g(x)\big)$, 它也就是最大值点. 又因为 p_{data}, p_g 处处为正,$d^*(x)\in(0,1)$ 确实落在可行范围内. 于是对任意可测的 d 与几乎处处的 x 都有 $F\big(x,d(x)\big)\leqslant F\big(x,d^*(x)\big)$, 两端积分得 $V(p_g,d)\leqslant$

$V(p_g, d^*)$, 即 $E(p_g, d) \geqslant E(p_g, d^*)$. 等号成立当且仅当 $d(x) = d^*(x)$ 几乎处处, 故最小值点在几乎处处的意义下唯一.

(ii) 把 d^* 与 $1 - d^* = \dfrac{p_g}{p_{\text{data}} + p_g}$ 代入 E. 先算第一项:

$$-\int p_{\text{data}} \ln d^* \,\mathrm{d}x = \int p_{\text{data}} \ln \frac{p_{\text{data}} + p_g}{p_{\text{data}}} \,\mathrm{d}x = \int p_{\text{data}} \ln \frac{2M}{p_{\text{data}}} \,\mathrm{d}x$$
$$= \ln 2 - \mathrm{KL}\left(p_{\text{data}} \| M\right),$$

其中末步用了 $\int p_{\text{data}} \,\mathrm{d}x = 1$ 与 $\mathrm{KL}(p_{\text{data}} \| M) = \int p_{\text{data}} \ln \dfrac{p_{\text{data}}}{M} \mathrm{d}x$. 同理

$$-\int p_g \ln(1 - d^*) \,\mathrm{d}x = \ln 2 - \mathrm{KL}\left(p_g \| M\right).$$

两式相加即得

$$C(p_g) = \ln 4 - \mathrm{KL}\left(p_{\text{data}} \| M\right) - \mathrm{KL}\left(p_g \| M\right) = \ln 4 - \mathrm{JSD}(p_{\text{data}} \| p_g).$$

(iii) 由 $\ln t \leqslant t - 1$(等号当且仅当 $t = 1$) 可得 $\mathrm{KL}(p \| q) \geqslant 0$, 且等号成立当且仅当 $p = q$ 几乎处处 (参见小节 2.5.5). 于是 $\mathrm{JSD}(p_{\text{data}} \| p_g) \geqslant 0$, 且

$$\mathrm{JSD}(p_{\text{data}} \| p_g) = 0 \iff \mathrm{KL}\left(p_{\text{data}} \| M\right) = \mathrm{KL}\left(p_g \| M\right) = 0 \iff p_{\text{data}} = M = p_g$$

几乎处处. 代回(17.10)即得 $C(p_g) \leqslant \ln 4$, 等号当且仅当 $p_g = p_{\text{data}}$. 这正是生成器 (相对于 w 最大化 E_{GAN}) 所能达到的最好结果.

□

注

(重构新增) 本命题回答了两个问题: 给定生成器, 判别器的最优解长什么样 ((17.9)); 以及把这个最优判别器代回后, 生成器到底在优化什么 ((17.10)). 答案是后者为 Jensen-Shannon 散度: 它非负、对称, 且只在两个分布相等时为零, 因此“最小化 GAN 损失”在最理想的无限容量极限下与“让生成分布等于数据分布”是同一件事. 这也解释了为什么完全训练好的 GAN 中判别器的输出恒为 1/2: 把 $p_g = p_{\text{data}}$ 代入(17.9)立即得到 $d^* \equiv 1/2$.

证明用到三个条件, 缺一不可. 第一,p_{data} 与 p_g 处处为正, 保证(17.9)的分母不为零、d^* 落在 $(0, 1)$ 内; 若某个 x 处 $p_{\text{data}}(x) = p_g(x) = 0$, 该点的 $d^*(x)$ 无定义, 但它在积分中权重为零, 不影响结论. 第二,d 取遍所有可测函数是“变分优化”的含义: 正是这一点使逐点最小化可以合法地搬到积分号内; 若把 d 限制在某个参数化族 (如某个神经网络) 中, 逐点最优解一般不可达,(17.10)只是下界. 第三,p_g 必须关于 Lebesgue 测度有密度, 否则 $\ln p_g$ 无意义——这正是一般 GAN 的生成分布 (集中低维流形上) 不满足的条件, 也是练习 17.1要求“模型具有无限灵活性”的技术含义.

最后说明符号约定. 按本节的(17.6),E_{GAN} 带负号, 判别器求极小、生成器求极大, 故生成器的最优值为 $\ln 4$, 而游戏值 $V = -E_{\text{GAN}}$ 的最优值为 $-\ln 4$. 文献中常见的写法 $C = -\ln 4 + 2\,\mathrm{JSD}$ 采用后一种 (最大化) 约定, 并把 JSD 定义中的因子 1/2 并入系数; 本节的 JSD 按练习 17.1的约定取两个 KL 散度之和, 故(17.10)中系数为 1. 两种写法的数学内容相同.

17.1.2 实战中的 GAN 训练

尽管 GAN 可以产生高质量的结果, 但由于对抗学习的特点, 其并不容易训练成功 (见练习 17.2). 此外, 与标准的损失函数最小化不同, 由于目标函数的值在训练过程中可能上升也可能下降, 因此并没有一个明确的进展指标.

定义 17.5 (模式崩溃)

在 GAN 的训练过程中, 生成器网络的权重可能发生适应性调整, 导致所有潜变量样本 z 都被映射到有效输出的一个子集上; 在极端情况下, 输出可能只对应一个或少数几个输出值 x, 判别器网络随后将这些实例的值设置为 0.5 并停止训练. 称这一现象为**模式崩溃** (mode collapse).

♣

注

(重构新增) 模式崩溃的根源在于生成器的目标只要求“骗过判别器”, 而不要求“覆盖数据分布”: 若 p_g 把全部质量堆在 p_{data} 的某个峰上, 判别器在该峰附近无法区分真假, 生成器的损失就已经很小, 尽管其余的模式完全没有被生成. 用命题 17.1的语言说, 此时 p_g 与 p_{data} 的 Jensen-Shannon 散度并不为零, 但在有限容量的参数化下, 生成器可以停在一个使判别器“局部失效”的驻点上.

条件方面: 模式崩溃是优化与参数化的联合现象, 而不是目标函数在无限容量下的性质——命题 17.1表明, 若生成器能在全部概率分布上优化, 最优解恰是 $p_g = p_{\text{data}}$, 不会崩溃. 此外, 判别器输出被置为 0.5 意味着它对真实样本与合成样本给出相同的概率, 这既可能来自 p_g 与 p_{data} 的局部重合, 也可能来自判别器自身的容量不足, 两者在训练中很难区分, 这也是 GAN 缺乏可靠进展指标的一个具体表现.

例题 17.1 手写数字生成中的模式崩溃 一个用于生成手写数字的 GAN 可能学会只生成数字“3”的实例, 而判别器网络无法将这些实例与真正的数字“3”的实例区分开来, 它没能认识到生成器网络并没有生成所有可能的数字.

通过图 17.2, 我们可以更深入地理解训练 GAN 的困难. 该图显示了一个简单的一维数据空间 x, 其中样本 $\{x_n\}$ 来自固定但未知的数据分布 $p_{\text{data}}(x)$. 该图还显示了初始生成分布 $p_g(x)$ 以及从该分布中抽取的样本. 因为数据分布和生成分布之间差异显著, 最优的判别器函数 $d(x)$ 很容易学习, 其函数值下降态势急剧, 在真实样本或合成样本的邻近区域梯度几乎为零. 考虑 GAN 损失函数 [(17.6)] 中的第二项, 由于在生成样本跨越的区域内 $d(g(z,w),\phi)$ 值等于零, 生成器网络参数 w 的微小变化在判别器网络的输出上将产生很小的变化, 因此梯度较小, 导致学习进程缓慢.

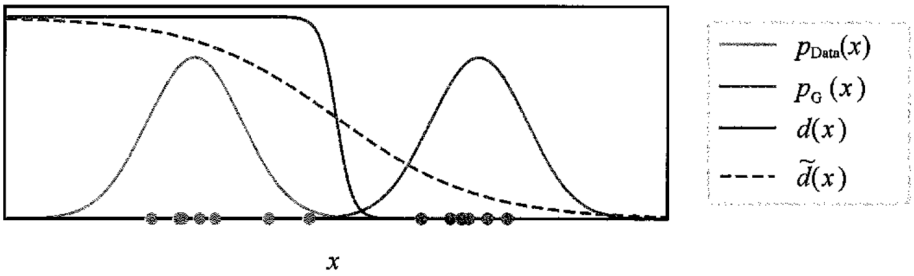


图 17.2: 为了从概念上说明为什么训练 GAN 可能会有难度, 这里展示了一个简单的一维数据空间 x, 其中有固定但未知的数据分布 $p_{\text{data}}(x)$ 和初始生成分布 $p_g(x)$. 最优的判别函数 $d(x)$ 在训练或合成数据点附近几乎没有梯度, 使学习变得非常缓慢. 判别函数的平滑版本 $\tilde{d}(x)$ 可以使学习变得更快

这个问题可以通过使用判别器函数的平滑版本 $\tilde{d}(x)$ 来解决 (见图 17.2), 从而为生成器网络提供更强的梯度以驱动训练. 最小二乘 GAN(least-squares GAN, Mao et al., 2016) 通过修改判别器网络输出一个实值而不是取值范围为 0 ~ 1 的概率, 并用平方和损失函数替换交叉熵损失函数来实现平滑. 另一种称为**实例噪声** (instance noise)(Sønderby et al., 2016) 的方法则向真实样本和合成样本中添加 Gauss 噪声, 这同样能使判别器函数更加平滑.

为了改进 GAN 的训练, 研究者们提出了许多对损失函数和训练过程的修改方案 (Mescheder, Geiger, and Nowozin, 2018). 其中常见的一个建议是将原始损失函数中的生成器网络

$$-\frac{1}{N_{\text{synth}}}\sum_{n\in\text{synth}}\ln\big(1-d(g(z_n,w),\phi)\big) \tag{17.11}$$

替换为

$$\frac{1}{N_{\text{synth}}}\sum_{n\in\text{synth}}\ln d(g(z_n,w),\phi). \tag{17.12}$$

(17.11)所示的第一种形式最小化了图像为假的概率,(17.12)所示的第二种形式则最大化了图像为真的概率. 这两种形式的不同属性可以通过图 17.3来加以理解. 当生成分布 $p_g(x)$ 与数据分布 $p_{\text{data}}(x)$ 完全不重叠时,$d(g(z,w))$ 接近于零, 因此第一种形式具有非常小的梯度, 而第二种形式具有较大的梯度, 从而能加快训练速度.

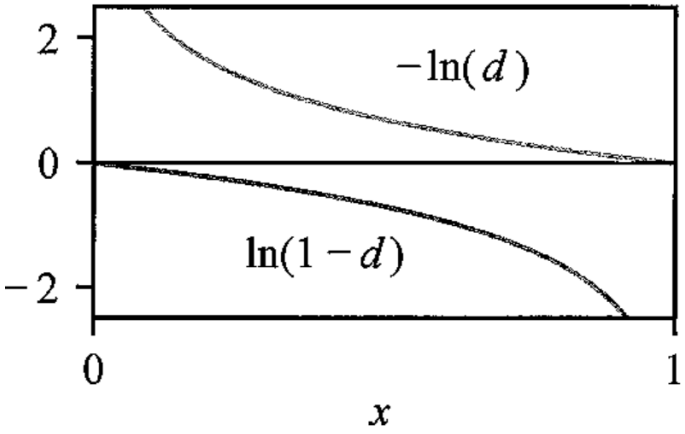


图 17.3: $-\ln(d)$ 和 $\ln(1-d)$ 的图像表明, 在 $d=0$ 和 $d=1$ 附近, 梯度的表现存在非常大的差异

为确保生成分布 $p_g(x)$ 向数据分布 $p_{\text{data}}(x)$ 靠拢, 更直接的方法是修改误差标准, 以反映这两个分布在数据空间中的距离. 这可以使用 **Wasserstein 距离** (也称推土机距离) 来衡量. 设想生成分布 $p_g(x)$ 是一堆土, 我们需要分批运输这堆土以构建数据分布 $p_{\text{data}}(x)$.Wasserstein 距离就是移动的土壤总量和平均移动距离的积. 在重排这堆土以构建 $p_{\text{data}}(x)$ 的许多方式中, 能产生最小平均移动距离的方式被用于定义度量.

定义 17.6 (Wasserstein 距离)

设 p 与 q 是 $\mathbb{R}^D$ 上的概率分布,$\Pi(p,q)$ 表示边缘分布分别为 p 与 q 的全部联合分布的集合. 称

$$W(p,q)=\inf_{\gamma\in\Pi(p,q)}\mathbb{E}_{(x,y)\sim\gamma}\left[\|x-y\|\right]$$

为 p 与 q 之间的 **Wasserstein 距离** (Wasserstein distance), 也称**推土机距离** (earth mover distance);γ 称为从 p 到 q 的一个**传输方案** (transport plan).

♣

注

(重构新增) 定义中的 γ 就是“运输方案”:$\gamma(A\times B)$ 表示从区域 A 运往区域 B 的土壤量, 边缘条件 $\gamma(A\times\mathbb{R}^D)=p(A)$、$\gamma(\mathbb{R}^D\times B)=q(B)$ 保证出发点与目的地的土量分别与 p、q 相符. 由于总质量已被归一化为 1, “移动的土壤总量与平均移动距离的积”就是平均移动距离本身, 即 $\mathbb{E}_\gamma[\|x-y\|]$; 取 inf 对应原书所说的“能产生最小平均移动距离的方式”.

与 Jensen-Shannon 散度相比,Wasserstein 距离的最大好处是它在两个分布的支撑集互不相交时仍然提供有意义的数值: 若 p 与 q 的支撑集之间的距离为 $d>0$, 则 $W(p,q)\geqslant d>0$, 而 $\text{JSD}(p\|q)$ 恒等于 $\ln 4$(与两个支撑集的距离无关), 其梯度为零. 这解释了为什么命题 17.1意义下的 GAN 在训练初期 (p_g 与 p_{data} 几乎不重叠) 没有可用的梯度信号, 而以 W 为目标的模型不会遇到这一问题.

条件方面:$\Pi(p,q)$ 非空 (乘积分布即为一个传输方案), 故 inf 有定义;$\|\cdot\|$ 取哪个范数会影响数值但不影响“$W(p,q)=0\iff p=q$”这一性质. 此外 W 是对称的、满足三角不等式, 因此是真正的度量, 而 $\text{JSD}^{1/2}$ 才是度量、JSD 本身不是.

在实践中, 这并不能直接实现, 需要通过使用一个具有实值输出的判别器网络来近似, 然后通过权重裁剪限制判别器函数关于 x 的梯度 $\nabla_x d(x,\phi)$, 从而产生了 Wasserstein GAN(Arjovsky, Chintala, and Bottou, 2017).

另一种改进的方法是对梯度引入惩罚, 从而产生了梯度惩罚 Wasserstein GAN(Gulrajani et al., 2017), 其损失函数由下式给出:

$$\begin{aligned}E_{\text{WGAN-GP}}(w,\phi)=&-\frac{1}{N_{\text{real}}}\sum_{n\in\text{real}}\left[\ln d(x_n,\phi)-\eta\left(\|\nabla_{x_n}d(x_n,\phi)\|^2-1\right)^2\right]_+\\&+\frac{1}{N_{\text{synth}}}\sum_{n\in\text{synth}}\ln d\left(g(z_n,w,\phi)\right),\end{aligned}\tag{17.13}$$

其中的 η 控制了惩罚项的相对重要性.

注

(重构新增) (17.13)的结构可以逐项读出: 第一项对真实样本要求 $\ln d(x_n,\phi)$ 尽量大, 同时用 $\left(\|\nabla_{x_n} d\|^2-1\right)^2$ 把判别器在真实样本处的梯度范数拉向 1; 第二项对合成样本要求 $\ln d$ 尽量小; 外层的 $[\cdot]_+$ 是 hinge(只惩罚超过零的部分), 使损失在达到间隔后不再继续下降.

需要指出原文的一处技术细节: 标准的梯度惩罚项写作 $\left(\|\nabla_{\hat{x}} d(\hat{x},\phi)\|-1\right)^2$, 其中 $\hat{x}$ 在真实样本与合成样本之间随机插值;(17.13)把范数写成了平方 $\|\cdot\|^2$, 并把取样点取在真实样本 x_n 上. 二者都是对"判别器应为 1-Lipschitz"这一约束的可计算松弛, 只是松弛的位置不同: 前者约束梯度范数等于 1, 后者约束梯度范数的平方等于 1, 由于 $\|\nabla d\|\geqslant 0$, 两式的零点都恰是 $\|\nabla d\|=1$, 故在最优点处二者一致. 此外,η 在原文中控制惩罚项的权重, 其作用与 Lagrange 乘子相同:η 越大, 约束越被强制满足, 但训练也越不稳定.

命题 17.2 (Kantorovich-Rubinstein 对偶)

设 $\mathcal{X}$ 是 $\mathbb{R}^D$ 中的紧集,$\|\cdot\|$ 是 $\mathbb{R}^D$ 上的范数,p,q 是 $\mathcal{X}$ 上的概率分布. 则

$$W(p,q)=\sup_{f\in \mathrm{Lip}_1}\left\{\mathbb{E}_{x\sim p}\left[f(x)\right]-\mathbb{E}_{y\sim q}\left[f(y)\right]\right\}, \tag{17.14}$$

其中 Lip_1 表示 $\mathcal{X}$ 上全部满足 $|f(x)-f(y)|\leqslant\|x-y\|$ 的实值函数 (称为 1-Lipschitz 函数) 的集合.

证明

(重构新增) 先证"$\geqslant$". 设 f 是 1-Lipschitz 函数,$\gamma\in\Pi(p,q)$ 是任一传输方案. 则

$$\begin{aligned}\mathbb{E}_{x\sim p}\left[f(x)\right]-\mathbb{E}_{y\sim q}\left[f(y)\right]&=\int_{\mathcal{X}\times\mathcal{X}}\big(f(x)-f(y)\big)\mathrm{d}\gamma(x,y)\\&\leqslant\int_{\mathcal{X}\times\mathcal{X}}\big|f(x)-f(y)\big|\,\mathrm{d}\gamma(x,y)\leqslant\int_{\mathcal{X}\times\mathcal{X}}\|x-y\|\,\mathrm{d}\gamma(x,y),\end{aligned}$$

第一个等号用到 γ 的边缘条件. 对 γ 取下确界得 $\mathbb{E}_p[f]-\mathbb{E}_q[f]\leqslant W(p,q)$, 再对 f 取上确界即得 $\sup\leqslant W(p,q)$.

再证"$\leqslant$". 先设 $\mathcal{X}$ 是有限点集 $\{x_1,\cdots,x_m\}$, 记 $p_i=p(\{x_i\})$、$q_j=q(\{x_j\})$、$C_{ij}=\|x_i-x_j\|$. 此时 $W(p,q)$ 是线性规划

$$\min\Big\{\sum_{i,j}\gamma_{ij}C_{ij}:\ \gamma_{ij}\geqslant 0,\ \sum_j\gamma_{ij}=p_i,\ \sum_i\gamma_{ij}=q_j\Big\}$$

的最优值. 由线性规划对偶定理, 它等于对偶问题

$$\max\Big\{\sum_i a_ip_i+\sum_j b_jq_j:\ a_i+b_j\leqslant C_{ij}\text{ 对一切 }i,j\Big\}$$

的最优值. 设 (a,b) 是对偶可行解, 令 $f_i=\min_j\left(C_{ij}-b_j\right)$. 由三角不等式, 对任意 i,k 及取得最小值的 j^* 有

$$f_i-f_k\leqslant\left(C_{ij^*}-b_{j^*}\right)-\left(C_{kj^*}-b_{j^*}\right)=C_{ij^*}-C_{kj^*}\leqslant C_{ik},$$

交换 i,k 可得反向不等式, 故 f 是 1-Lipschitz 的; 又 $f_j\leqslant C_{jj}-b_j=-b_j$, 故

$$\sum_i a_ip_i+\sum_j b_jq_j\leqslant\sum_i f_ip_i-\sum_j f_jq_j=\mathbb{E}_p[f]-\mathbb{E}_q[f]\leqslant\sup_{g\in\mathrm{Lip}_1}\left\{\mathbb{E}_p[g]-\mathbb{E}_q[g]\right\}.$$

于是 $W(p,q)\leqslant\sup$, 与前一方向合并即得(17.14).

对一般的紧集 $\mathcal{X}$, 上述论证可以这样移植. 取 $\mathcal{X}$ 的有限 ε-网 $\{x_1,\cdots,x_m\}$ 与投影 $\pi:\mathcal{X}\to\{x_1,\cdots,x_m\}$, 使 $\|x-\pi(x)\|\leqslant\varepsilon$ 对一切 x 成立, 并记 $p^\varepsilon,q^\varepsilon$ 为 p,q 在 π 下的像. 任取 (p,q) 的传输方案 γ, 其像 $(\pi(x),\pi(y))_\sharp\gamma$ 是 $(p^\varepsilon,q^\varepsilon)$ 的传输方案, 且 $\|x-y\|\leqslant\|\pi(x)-\pi(y)\|+2\varepsilon$, 故 $W(p,q)\leqslant W(p^\varepsilon,q^\varepsilon)+2\varepsilon$. 另一方面,$\mathcal{X}$ 上的 1-Lipschitz 函数限制在网上仍为 1-Lipschitz, 且 $|f(x)-f(\pi(x))|\leqslant\varepsilon$, 故 $\mathbb{E}_{p^\varepsilon}[f]-\mathbb{E}_{q^\varepsilon}[f]\leqslant\mathbb{E}_p[f]-\mathbb{E}_q[f]+2\varepsilon$. 把离散情形的结论用于 $p^\varepsilon,q^\varepsilon$ 并令 $\varepsilon\to 0$, 即得 $W(p,q)\leqslant\sup$. 这一论证的一般形式就是 Kantorovich-Rubinstein 定理, 它也可由 Hahn-Banach 定理在 $C(\mathcal{X})$ 上直接导出.

□

注

(重构新增) (17.14)是 Wasserstein GAN 的全部理论依据: 它把一个“在联合分布上求下确界”的传输问题换成了“在一个函数类上求上确界”的问题, 而后者可以直接用神经网络参数化——取 $f = d(\cdot, \phi)$, 就得到

$$\max_{\phi} \left\{ \mathbb{E}_{x \sim p_{\text{data}}} \left[d(x, \phi) \right] - \mathbb{E}_{z \sim p(z)} \left[d(g(z, w), \phi) \right] \right\},$$

这正是 WGAN 中判别器 (此时称 critic, 因为它输出实值而不是概率) 要最大化的量; 生成器则最小化同一量. 原书所说的“权重裁剪”就是对 Lip_1 这一约束的粗糙实现: 把每个权重的绝对值截断在 $[-c, c]$ 内, 可以使网络所表示的函数具有与 c 和层数有关的 Lipschitz 常数. 权重裁剪把约束变成硬投影, 容易造成梯度消失或爆炸;(17.13)的梯度惩罚则是把约束写成软惩罚, 因而更稳定.

条件方面: 命题要求 $\mathcal{X}$ 紧、p, q 是 Borel 概率测度, 这保证 $\Pi(p, q)$ 弱紧、inf 可达; 若 $\mathcal{X}$ 非紧, 结论对“有界 1-Lipschitz 函数”仍成立, 但上确界可能不可达. 还要注意(17.14)中的 f 不带符号约束, 因此 sup 一定非负 (取 $f \equiv 0$ 得 0), 这与 $W \geqslant 0$ 相符.

17.2 图像的生成对抗网络

GAN 的基本概念催生了大量的研究工作, 涵盖了许多算法和众多应用, 其中应用最广泛且成功的是图像生成. 早期的 GAN 使用全连接网络作为生成器网络和判别器网络 (参见第 10 章). 然而, 使用卷积网络有许多好处, 尤其是对于更高分辨率的图像而言. 判别器网络将图像作为输入, 并提供一个标量概率作为输出, 因此使用标准的卷积网络非常合适 (参见小节 10.5.3). 生成器网络需要将低维潜空间映射成高分辨率图像, 因此使用了基于转置卷积的网络, 如图 17.4所示.

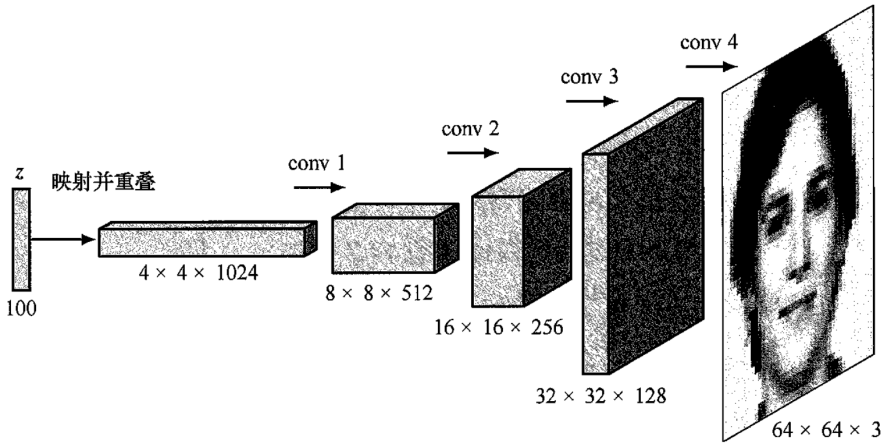


图 17.4: 深度卷积 GAN 的示例架构, 这里展示了如何在网络的连续模块中使用转置卷积来扩展维度

从低分辨率开始, 随着训练的进行, 逐步添加新的卷积层, 就能模拟越来越精细的细节 (Karras et al., 2017). 这种方法不仅加速了训练过程, 而且能够从 4×4 大小的图像开始, 最终合成 1024×1024 大小的高分辨率图像.图 17.5展示了用于类别条件图像生成的 GAN 模型——BigGAN, 这是体现 GAN 架构的规模和复杂性的一个典型例子.BigGAN 的生成器网络拥有超过 7000 万个参数, 而拥有 8800 万个参数的判别器网络具有类似的架构, 不同之处在于判别器网络使用平均池化层 (average pooling) 来降低维度, 而不是通过上采样 (up-sampling) 来增加维度 (Brock, Donahue, and Simonyan, 2018).

17.2.1 CycleGAN

作为 GAN 多样性的一个例子, 考虑一种称为 CycleGAN(Zhu et al., 2017) 的 GAN 架构, 我们将其作为示例以展示深度学习技术如何用于解决传统任务 (如分类和密度估计) 之外的不同类型的问题. 考虑如下问题: 如何将一

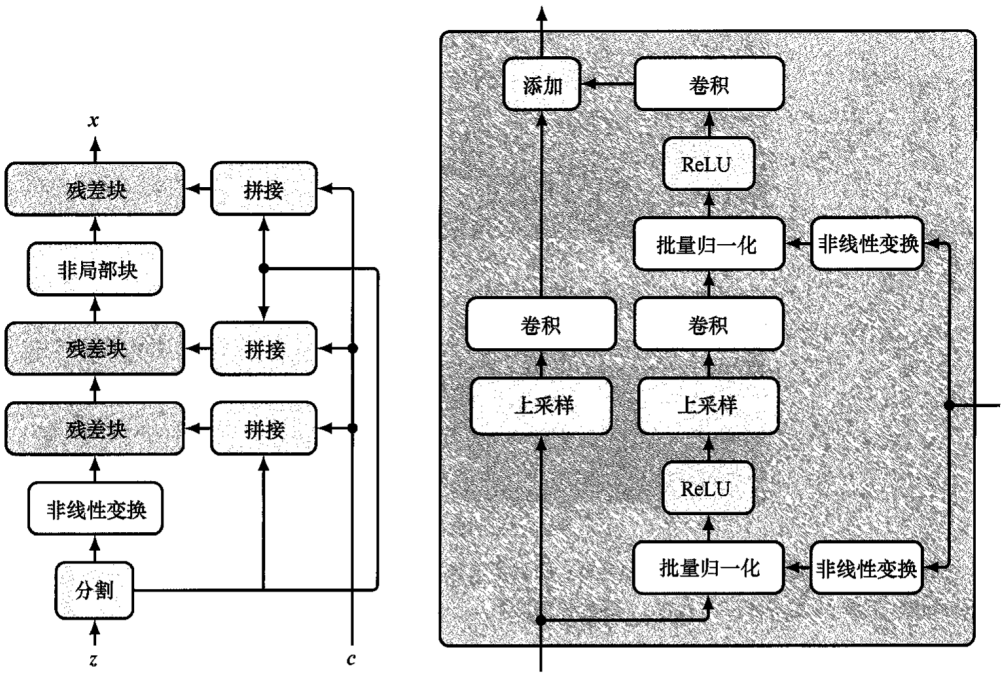


图 17.5: (a) BigGAN 模型中生成器网络的架构, 它拥有超过 7000 万个参数.(b) 生成器网络中每个残差块的细节. 拥有 8800 万个参数的判别器网络具有类似的架构, 不同之处在于判别器网络使用平均池化层 (average pooling) 来降低维度, 而不是通过上采样 (up-sampling) 来增加维度 [基于 Brock, Donahue, and Simonyan (2018)]

张照片转换为同一场景下的莫奈风格的画作 (或者反之)? 在图 17.6中, 我们展示了从一个经过训练的 CycleGAN 中得到的成对图像示例, 该 CycleGAN 已经学会执行此类图像到图像的转换.

CycleGAN 的目标是学习两个双射 (一对一映射), 一个是从照片的域 X 到莫奈风格画作的域 Y, 另一个则相反. 为了实现这一点,CycleGAN 利用了两个条件生成器 (g_X 和 g_Y) 以及两个判别器 (d_X 和 d_Y). 生成器 $g_X(y, w_X)$ 以一幅画作 $y \in Y$ 为输入, 并生成相应的合成照片, 判别器 $d_X(x, \phi_X)$ 则区分合成照片和真实照片. 类似地, 生成器 $g_Y(x, w_Y)$ 以一张照片 $x \in X$ 为输入并生成一幅合成画作 y, 判别器 $d_Y(y, \phi_Y)$ 则区分合成画作和真实画作. 因此, 对判别器 d_X 的训练, 需要结合 g_X 生成的合成照片和真实照片来进行, 而判别器 d_Y 则是在 g_Y 生成的合成画作和真实画作的组合上进行训练的.

定义 17.7 (CycleGAN)

设 X 是照片的域,Y 是画作的域,$g_X(\cdot, w_X) : Y \to X$ 与 $g_Y(\cdot, w_Y) : X \to Y$ 是两个条件生成器,$d_X(\cdot, \phi_X)$ 与 $d_Y(\cdot, \phi_Y)$ 分别是照片域与画作域上的判别器. 若训练目标是学习一对互逆的映射, 即

$$g_X\big(g_Y(x, w_Y), w_X\big) \approx x, \qquad g_Y\big(g_X(y, w_X), w_Y\big) \approx y,$$

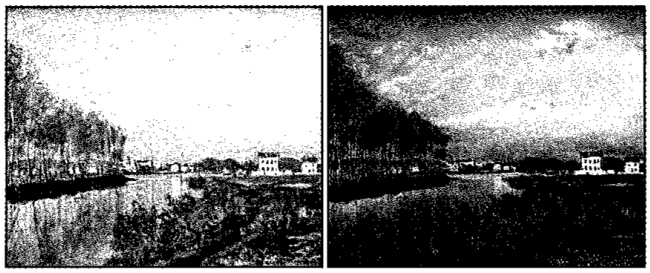
同时用 d_X 判别合成照片与真实照片、用 d_Y 判别合成画作与真实画作, 则称这样的架构为 **CycleGAN**. 称复合映射 $g_X \circ g_Y$ 与恒同映射 id_X 的偏差、以及 $g_Y \circ g_X$ 与 id_Y 的偏差为**循环一致性误差** (cycle consistency error).

♣

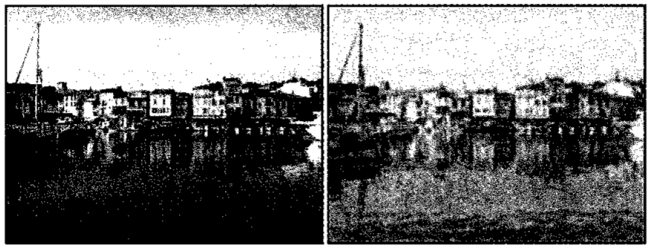
注

(重构新增) CycleGAN 要解决的是无配对数据的图像转换: 训练集中只有一堆照片和一堆画作, 并没有“这张照片对应这幅画”的配对信息. 若只用两个 GAN 损失, 生成器 g_Y 完全可以忽略输入 x 而输出任意一幅逼真的画作——判别器只检查“像不像画”, 不检查“是不是这张照片的画”, 这就是所谓生成器“无视输入”的退化. 循环一致性正是为堵住这一漏洞而加入的约束: 它要求转换可逆, 从而迫使 g_Y 保留 x 中足以重建 x 的信息.

条件方面: 定义中的“互逆”只能近似成立, 因为照片与画作的信息量不同 (画作丢失了照片的细节, 照片也没有笔触), 严格的互逆通常不存在; 因此循环一致性以误差而非硬约束的形式出现. 此外, 互逆性本身并不足以保



画作→合成照片



照片→合成画作

图 17.6: 使用 CycleGAN 进行图像转换的示例: 由莫奈风格画作合成摄影风格照片 (上排), 以及由照片合成莫奈风格画作 (下排) [Zhu et al. (2017), 经授权使用]

证语义对应——一个把照片编码成任意可逆码的 g_Y 也满足循环一致性, 只是生成的画作不像画; 所以 CycleGAN 必须把 GAN 损失与循环一致性误差同时使用, 二者缺一不可 (命题 17.3给出这一点的精确表述).

如果我们使用标准的 GAN 损失函数来训练这种架构,CycleGAN 就能学会生成逼真的合成画作和合成照片, 但没有什么机制能确保生成的画作看起来像对应的照片, 反之亦然. 因此, 我们在损失函数中引入了一个额外的术语, 称为**循环一致性误差** (cycle consistency error). 它包含两项, 其构建过程如图 17.7所示.

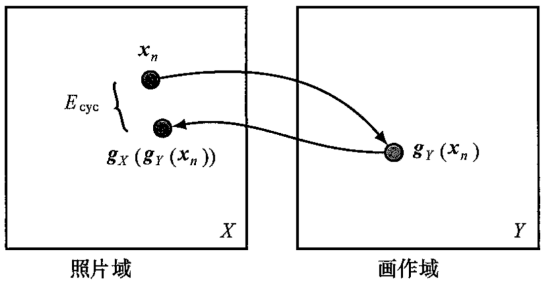


图 17.7: 为示例照片 x_n 计算循环一致性误差. 首先使用生成器 g_Y 将照片映射到画作域, 然后将得到的向量映射回照片域, 使用的是生成器 g_X. 结果照片与原始照片 x_n 之间的差异是由循环一致性误差贡献的. 类似地, 使用 g_X 将一幅画作 y_n 映射成一张照片, 然后使用 g_Y 再映射回一幅画作, 即可计算在这个方向上循环一致性误差的贡献

这一过程的目标是确保当一张照片先被转换成一幅画, 再被转换回照片时, 结果与原始照片接近, 从而确保生成的画作保留了足以恢复原始照片的初始信息. 类似地, 当一幅画作先被转换成一张合成照片, 再被转换回一幅画作时, 结果也应该与原始画作接近. 将这个理念应用到训练集中的所有照片和画作, 就会得到形式如下的循环一致性误差:

$$\begin{aligned} E_{\text{cyc}}(w_X, w_Y) = &\frac{1}{N_X}\sum_{n\in X}\left\|g_X\left(g_Y(x_n, w_Y), w_X\right) - x_n\right\|_1 \\ &+ \frac{1}{N_Y}\sum_{n\in Y}\left\|g_Y\left(g_X(y_n, w_X), w_Y\right) - y_n\right\|_1, \end{aligned} \tag{17.15}$$

其中 $\|\cdot\|_1$ 表示 L1 范数. 将循环一致性误差添加到(17.6)定义的 GAN 损失函数中, 便可得到总的损失函数:

$$E_{\text{total}} = E_{\text{GAN}}(w_X, \phi_X) + E_{\text{GAN}}(w_Y, \phi_Y) + \eta E_{\text{cyc}}(w_X, w_Y), \tag{17.16}$$

其中系数 η 决定了 GAN 误差和循环一致性误差的相对重要性. 在计算一个图像和一幅画作的损失函数时, 通过

CycleGAN 的信息流如图 17.8所示.

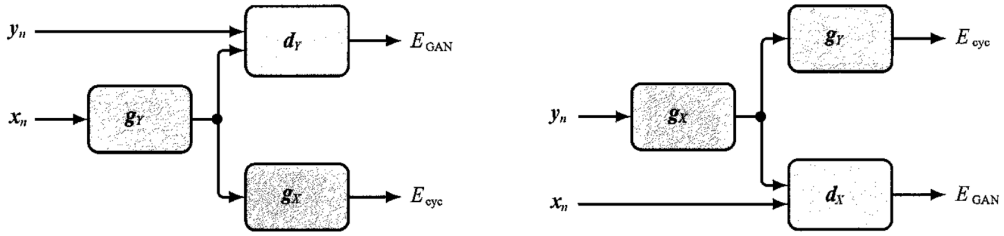


图 17.8: 通过 CycleGAN 的信息流. 数据点 x_n 和 y_n 的总误差是其 4 个组成部分误差的总和

命题 17.3 (循环一致性误差的性质)

设 $X, Y \subseteq \mathbb{R}^D$ 为 Borel 集,$g_X : Y \to X$、$g_Y : X \to Y$ 为可测映射,p_X 与 p_Y 分别是照片与画作在 X、Y 上的分布. 则:

(i) 对任意 w_X, w_Y 都有 $E_{\text{cyc}}(w_X, w_Y) \geqslant 0$;

(ii)$E_{\text{cyc}}(w_X, w_Y) = 0$ 当且仅当两个复合映射在训练集上等于恒同映射, 即

$$g_X\big(g_Y(x_n, w_Y), w_X\big) = x_n \quad (n \in X), \qquad g_Y\big(g_X(y_n, w_X), w_Y\big) = y_n \quad (n \in Y);$$

(iii) 若把(17.15)中的经验平均换成对 p_X、p_Y 的数学期望 (总体形式), 则相应的循环一致性误差为零, 当且仅当

$$g_X\big(g_Y(x, w_Y), w_X\big) = x \quad \text{对}p_X\text{-几乎处处的}x\text{ 成立}, \qquad g_Y\big(g_X(y, w_X), w_Y\big) = y \quad \text{对}p_Y\text{-几乎处处的}y\text{ 成立};$$

(iv) 特别地, 若 $g_X \circ g_Y = \mathrm{id}_X$ 且 $g_Y \circ g_X = \mathrm{id}_Y$(即两者互为逆映射), 则 $E_{\text{cyc}} = 0$.

证明

(重构新增) (i)(17.15)右端是两族非负项的平均: 每个 $\|\cdot\|_1 \geqslant 0, N_X, N_Y \geqslant 1$, 故 $E_{\text{cyc}} \geqslant 0$.

(ii) 由 (i),$E_{\text{cyc}} = 0$ 等价于两个平均值都为零, 而有限个非负项之和为零又等价于每一项为零. 于是

$$\big\|g_X\big(g_Y(x_n, w_Y), w_X\big) - x_n\big\|_1 = 0, \qquad \big\|g_Y\big(g_X(y_n, w_X), w_Y\big) - y_n\big\|_1 = 0$$

对全部 n 成立. 由范数的正定性,$\|u\|_1 = 0$ 当且仅当 $u = 0$, 故这两族等式分别等价于 “$g_X \circ g_Y$ 在全部照片样本上等于恒同映射” 与 “$g_Y \circ g_X$ 在全部画作样本上等于恒同映射”. 反之, 若这两族等式成立, 则每一项的范数为零, 从而 $E_{\text{cyc}} = 0$.

(iii) 总体形式下,E_{cyc} 是两个非负随机变量 $\big\|g_X\big(g_Y(x, w_Y), w_X\big) - x\big\|_1$ 与 $\big\|g_Y\big(g_X(y, w_X), w_Y\big) - y\big\|_1$ 的期望之和. 期望为零当且仅当相应的非负随机变量几乎处处为零, 即两个复合映射分别在 p_X-几乎处处的 x 与 p_Y-几乎处处的 y 上等于恒同映射.

(iv) 若 $g_X \circ g_Y = \mathrm{id}_X$, 则对每个 x 有 $g_X(g_Y(x, w_Y), w_X) = x$, 第一项为零; 同理第二项为零, 故 $E_{\text{cyc}} = 0$.

□

注

(重构新增) 本命题说明循环一致性误差是一个 “可加的非负惩罚”: 它的零点恰好刻画了 “两个生成器在数据支撑集上互逆”. 因此, 把 ηE_{cyc} 加入总损失(17.16)等价于在 GAN 目标上叠加一个软约束,$\eta \to \infty$ 时约束变为硬约束.

有三点条件需要强调. 第一, 范数的选择不影响零点的刻画: 任何范数都满足 “范数为零当且仅当向量为零”, 所以换成 L_2 范数命题仍然成立; 选 L_1 是为了让误差对大偏差不那么敏感, 从而在图像上产生更少的模糊. 第二, 结论只在数据分布的支撑集上成立, 离开支撑集 (例如输入一幅训练集中没有的画风) 时 $g_X \circ g_Y$ 可以任意, 这正是 CycleGAN 泛化能力有限的原因之一. 第三,$E_{\text{cyc}} = 0$ 并不意味着 g_X, g_Y 是双射: 本命题只给出 “在数据分布意义下几乎处处互逆”, 而两个生成器完全可以把支撑集之外的点映到同一个像点. 原书称目标是学习 “两个双射”, 应理解为对理想模型的期望而非可验证的结论.

最后, 循环一致性并不能单独保证语义正确. 若把 g_Y 取为任意可逆编码、g_X 取其逆, 则 $E_{\text{cyc}} = 0$, 但生成的“画作”可能完全不像画. 因此总损失(17.16)必须同时保留两个 GAN 误差: 前者负责“像画/像照片”, 后者负责“是这张照片的画”.

17.2.2 表示学习与潜空间插值

我们已经看到,GAN 能胜任生成式模型的工作, 它也可用于**表示学习** (representation learning)(参见小节 6.3.3), 其通过无监督学习揭示了数据集中丰富的统计结构. 当用图 17.4所示的深度卷积 GAN 在一个卧室图像数据集上进行训练, 并且从潜空间中随机抽取样本, 然后通过已训练的网络加以传播时, 生成的图像看起来也像卧室, 正如我们预期的那样 (Radford, Metz and Chintala, 2015). 此外, 潜空间已经以具有语义意义的方式组织起来. 例如, 如果我们沿着潜空间中的平滑轨迹进行跟踪, 并生成相应的一系列图像, 就可以实现从一个图像到另一个图像的平滑过渡, 如图 17.9所示.

图 17.9: 由在卧室图像上训练的深度卷积 GAN 生成的样本. 其中的每一行都是通过在潜空间内部随机生成的位置之间进行平滑移动得到的. 图像的过渡很平滑, 每幅图像看起来都像是一个合理的卧室. 在最下方一行, 我们看到墙上的电视逐渐变形成了窗户 [Radford, Metz, and Chintala (2015), 经授权使用]

此外, 识别潜空间中与具有语义意义的变换相对应的方向也是可能的. 例如, 对人脸图像, 某个方向可能对应面部朝向的变化, 而其他方向可能对应光照变化或面部微笑程度的变化, 这称为**解耦表示** (disentangled representation). 我们可以利用它们来合成具有特定属性的新图像.

定义 17.8 (解耦表示)

设 $z \mapsto g(z)$ 是训练好的生成器. 若潜空间中存在若干方向 $v_1, \cdots, v_m$, 使得沿每个 v_i 移动时, 生成的图像中只有第 i 个语义属性 (如面部朝向、光照、是否微笑) 发生系统性变化, 而其余属性近似保持不变, 则称该潜空间给出了这些属性的**解耦表示** (disentangled representation), 并称 v_i 为该属性对应的**语义方向**. ♣

注

(重构新增) 解耦表示的用途是“可控生成”: 找到语义方向之后, 就可以固定其余坐标、只沿该方向移动, 从而在保持其他属性不变的前提下改变目标属性. 这在数据空间中往往做不到——像素级的向量加减会破坏图像的对齐与光照,图 17.10最下一行正是这种失败的例子; 而潜空间中的移动天然保持“像一张人脸”, 因为整个潜空间都被生成器映到数据流形附近.

条件方面要强调: 解耦不是 GAN 训练目标的推论, 而是额外的结构假设.命题 17.1只保证 $p_g = p_{\text{data}}$, 它并不惩罚潜坐标与语义属性之间的纠缠. 若训练数据中“戴眼镜”与“男性”高度相关, 那么这两个因素在潜空间中对应的方向几乎重合, 任何沿其中一个方向的移动都会同时改变两者; 只有数据中的相关性足够弱、或额外加入独立性正则时, 才可能得到近似解耦的表示. 此外, 语义方向没有唯一性: 不同的随机初始化会给出不同的基, 它们能解释同一组属性, 因此“找到方向”总是指相对于某个已训练模型而言.

例题 17.2 人脸潜空间中的向量算术 图 17.10是一个关于在面部图像上训练的 GAN 的例子, 可以看到性别或“戴眼镜与否”等语义属性对应于潜空间中特定的方向.

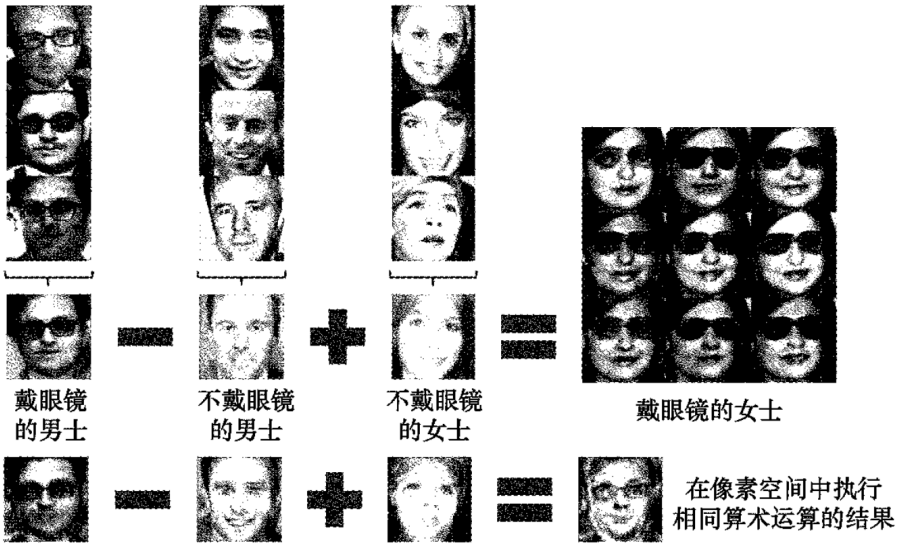


图 17.10: 在训练成功的 GAN 的潜空间中进行向量运算的例子. 对左侧三列中的每一列, 算法先对生成这些图像的潜空间向量取平均值, 再对具有中间值的结果进行向量运算, 这样就创建了右侧 3×3 阵列中正中的那个新向量. 然后向这个向量中添加噪声, 产生周围的 8 个样本图像. 最下方一行的 4 幅图像则展示了直接在数据空间中应用相同的运算会导致不对齐, 从而产生模糊的图像 [Radford, Metz, and Chintala (2015), 经授权使用]

17.3 习题

练习 17.1★★★ 我们希望 GAN 损失函数 [(17.6)] 具有如下性质: 在神经网络足够灵活的情况下, 当生成分布与真实数据分布匹配时, 就能获得驻点. 在这个习题中, 假设模型具有无限的灵活性, 我们将通过在与生成器网络和判别器网络相对应的全部概率分布 $p_g(x)$ 和全部函数 $d(x)$ 的空间上对其进行优化来证明这一结果. 具体来说, 我们假设判别模型在一个内循环中被优化, 从而产生用于生成式模型的有效外循环误差函数. 首先证明在数据样本无限多的情况下,GAN 损失函数 [(17.6)] 可以重写为如下形式:

$$E(p_g, d) = -\int p_{\text{data}}(x) \ln d(x)\, \mathrm{d}x - \int p_g(x) \ln\big(1 - d(x)\big)\, \mathrm{d}x, \tag{17.17}$$

其中 $p_{\text{data}}(x)$ 是真实数据点的固定分布. 考虑对所有函数 $d(x)$ 进行变分优化 (参见附录 B), 证明对于固定的生成器网络, 使 E 最小化的判别器函数 $d(x)$ 的解由下式给出:

$$d^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}. \tag{17.18}$$

进一步证明损失函数 E 可以作为生成器网络 $p_g(x)$ 的函数, 写成以下形式:

$$C(p_g) = -\int p_{\text{data}}(x) \ln\left\{\frac{p_{\text{data}}(x)}{p_{\text{data}}(x)+p_g(x)}\right\} \mathrm{d}x - \int p_g(x) \ln\left\{\frac{p_g(x)}{p_{\text{data}}(x)+p_g(x)}\right\} \mathrm{d}x, \tag{17.19}$$

(17.19)可以重写为以下形式:

$$C(p_g) = -\ln(4) + \mathrm{KL}\left(p_{\text{data}} \middle\| \frac{p_{\text{data}}+p_g}{2}\right) + \mathrm{KL}\left(p_g \middle\| \frac{p_{\text{data}}+p_g}{2}\right), \tag{17.20}$$

其中 Kullback-Leibler 散度 $\mathrm{KL}(p\|q)$ 由(2.103)定义 (参见小节 2.5.5). 最后, 根据只有在 $p(x)=q(x)$ 对所有 x 都成立时 $\mathrm{KL}(p\|q)$ 才等于零这一性质, 证明 $C(p_g)$ 的最小值发生在 $p_g(x)=p_{\text{data}}(x)$ 时. 注意,(17.20)中两个 Kullback-Leibler 散度的总和称为 p_{data} 和 p_g 之间的 Jensen-Shannon 散度. 和 Kullback-Leibler 散度一样,Jensen-Shannon 散度也是一个非负的量, 当且仅当两个分布相等时它会消失, 但不同于 KL 散度, 它对于两个分布是对称的.

注

(重构新增) **原文问题说明.** 本题的(17.19)与(17.20)在符号上不相容. 按(17.17)的定义,$E(p_g, d^*)$ 的值为 $\ln 4 - \mathrm{JSD}$, 而(17.20)的右端为 $-\ln 4 + \mathrm{JSD}$, 两者互为相反数. 这一点可以用一个具体的例子核对: 取 $p_{\text{data}}=(1/2,1/2)$、$p_g=(1,0)$ 的两点分布, 则 $d^*=(1/3,1)$, 直接代入(17.19)得 $E=\frac{1}{2}\ln 3+\ln\frac{3}{2}\approx 0.955$, 而 $-\ln 4+\mathrm{JSD}\approx -0.955$, 二者确实相差一个符号.

最小修正如下 (保留原题的条件与结论, 只修正符号约定): 把(17.19)改写为 "$C(p_g) \triangleq -E(p_g, d^*)$", 则(17.20)成立, 且 "$C$ 的最小值在 $p_g=p_{\text{data}}$ 处取到" 成立; 或者保留(17.19)不动, 把(17.20)写成 $C(p_g)=\ln 4-\mathrm{JSD}(p_{\text{data}}\|p_g)$, 并把结论改为 "$C$ 的最大值在 $p_g=p_{\text{data}}$ 处取到, 等价地 JSD 取最小值零". 本工程的正文采用后一种说法, 见命题 17.1.

✍ **练习 17.2**★★★ 在这个习题中, 我们将探讨 GAN 训练的对抗性可能会引起的问题. 考虑一个定义在两个参数 a 和 b 上的成本函数 $E(a,b)=ab$, 证明点 $a=0,b=0$ 是该成本函数的一个驻点. 考虑沿着直线 $b=a$ 和 $b=-a$ 的二阶导数, 其中 a 和 b 分别类比于生成网络和判别网络的参数. 证明点 $a=0,b=0$ 也是一个鞍点. 假设我们采取无穷小步骤来优化这个误差函数, 使得变量成为连续时间 $a(t)$ 和 $b(t)$ 的函数, 在此过程中生成器网络的参数 $a(t)$ 被更新以增加 $E(a,b)$, 参数 $b(t)$ 也被更新以减少 $E(a,b)$. 证明参数的演变受以下方程的控制:

$$\frac{\mathrm{d}a}{\mathrm{d}t}=\eta\frac{\partial E}{\partial a}, \qquad \frac{\mathrm{d}b}{\mathrm{d}t}=-\eta\frac{\partial E}{\partial b}. \tag{17.21}$$

然后证明 $a(t)$ 满足二阶微分方程:

$$\frac{\mathrm{d}^2a}{\mathrm{d}t^2}=-\eta^2 a(t). \tag{17.22}$$

验证以上二阶微分方程的解为

$$a(t)=C\cos(\eta t)+D\sin(\eta t), \tag{17.23}$$

其中 C 和 D 是任意常数. 如果系统在 $t=0$ 时初始化为 $a=1,b=0$, 找出 C 和 D 的值, 并由此证明得到的 $a(t)$ 和 $b(t)$ 的值会在以原点为中心的 a,b 空间中描绘出一个单位半径的圆, 并且它们因此永远不会收敛到鞍点.

✍ **练习 17.3**★★ 考虑一个 GAN 模型, 其训练集由相等数量的猫和狗的图像构成, 并且生成器网络已经学会了产生高质量的狗的图像. 证明当我们向判别器网络 (训练目标是输出一个图像为真的概率) 输入一张狗的图像时, 最优输出是 1/3.

第 18 章 标准化流

第 17 章介绍的生成对抗网络扩展了线性潜变量模型的框架, 其关键是使用深度神经网络来表示从潜空间到数据空间的高度灵活且可学习的非线性变换. 然而, 因为网络函数无法求逆, 似然函数通常很难处理, 当潜空间的维度低于数据空间的维度时, 似然函数甚至可能都无法定义. 因此, 研究人员在 GAN 中引入了一个新的判别器网络来支持对抗训练.

本章将讨论训练非线性潜变量模型的 4 种方法中的第 2 种方法 (参见小节 16.4.4), 它通过限制神经网络模型的形式, 在确保能够从训练好的模型中易于采样的前提下, 无须近似即可计算似然函数. 假设我们定义了一个分布 $p_z(\boldsymbol{z})$[有时也称**基础分布** (base distribution)], 这个分布定义在潜变量 $\boldsymbol{z}$ 上, 以及一个由深度神经网络给出的非线性函数 $\boldsymbol{x}=\boldsymbol{f}(\boldsymbol{z},\boldsymbol{w})$, 该函数要将潜空间转换成数据空间. 假设 $p_z(\boldsymbol{z})$ 是一个简单的分布, 比如 Gauss 分布, 那么从这样的模型中抽样就很容易, 因为每一个潜样本 $\boldsymbol{z}^*\sim p_z(\boldsymbol{z})$ 只需要通过神经网络就可以生成对应的数据样本 $\boldsymbol{x}^*=\boldsymbol{f}(\boldsymbol{z}^*,\boldsymbol{w})$.

为了计算这个模型的似然函数, 我们需要知道数据空间分布, 而它依赖于神经网络函数的逆函数, 记为 $\boldsymbol{z}=\boldsymbol{g}(\boldsymbol{x},\boldsymbol{w})$, 并且满足 $\boldsymbol{z}=\boldsymbol{g}(\boldsymbol{f}(\boldsymbol{z},\boldsymbol{w}),\boldsymbol{w})$. 这要求对于每一个 $\boldsymbol{w}$ 值, 函数 $\boldsymbol{f}(\boldsymbol{z},\boldsymbol{w})$ 和 $\boldsymbol{g}(\boldsymbol{x},\boldsymbol{w})$ 都是可逆的.

定义 18.1 (可逆映射与双射)

设 $S\subseteq\mathbb{R}^D$,$\boldsymbol{f}:S\to\mathbb{R}^D$ 是一映射. 若对每个 $\boldsymbol{x}\in\boldsymbol{f}(S)$ 都存在唯一的 $\boldsymbol{z}\in S$ 使得 $\boldsymbol{f}(\boldsymbol{z})=\boldsymbol{x}$, 则称 $\boldsymbol{f}$ 在 S 上是**可逆的** (invertible), 并称 $\boldsymbol{f}$ 是 S 与 $\boldsymbol{f}(S)$ 之间的**双射** (bijection), 即每个 $\boldsymbol{x}$ 值都对应唯一的 $\boldsymbol{z}$ 值, 反之亦然. 此时把 $\boldsymbol{x}$ 映到上述唯一 $\boldsymbol{z}$ 的映射称为 $\boldsymbol{f}$ 的**逆映射**, 记为 $\boldsymbol{f}^{-1}$.

注

(重构新增) 定义里的“唯一”二字是全部要害: 变量变换公式要用 $\boldsymbol{x}$ 唯一地决定 $\boldsymbol{z}$, 才能给 $p_x(\boldsymbol{x})$ 赋一个确定的值. 若映射是多对一的, 同一个 $\boldsymbol{x}$ 会来自多个 $\boldsymbol{z}$, 密度就无从定义;第 17 章中的生成对抗网络正是放弃了这一要求 (允许网络不可逆、允许潜空间维度低于数据空间), 代价是失去可计算的似然函数. 标准化流则反过来: 保留可逆性以换取精确似然.

逐项看条件. 其一, 定义域与值域的维数必须相同 (这里都是 D), 否则 Jacobian 矩阵不是方阵、行列式没有定义,命题 18.1根本无法写出. 其二, “双射”是一个整体性的要求, 它比“Jacobian 处处非奇异”更强: 由反函数定理,C^1 且 Jacobian 非奇异只保证局部可逆, 而流模型需要的是在整个数据空间上存在全局逆映射 $\boldsymbol{g}$. 其三,$\boldsymbol{f}$ 与 $\boldsymbol{g}$ 互为逆映射这一条在下一段的推导中被反复使用: 它把事件 $\{\boldsymbol{x}\in A\}$ 与 $\{\boldsymbol{z}\in\boldsymbol{g}(A)\}$ 等同起来.

一个容易被误解之处是术语“潜变量”:$\boldsymbol{x}$ 与 $\boldsymbol{z}$ 由确定性映射互相决定, 给定 $\boldsymbol{x}$ 后 $\boldsymbol{z}$ 不再有任何不确定性, 严格说来已不是通常意义下的隐变量. 但由于生成过程仍从 $\boldsymbol{z}$ 出发, 书中沿用了这一名称.

然后我们可以使用变量变换公式来计算数据密度 (参见节 2.4):

命题 18.1 (变量变换公式)

设 $\boldsymbol{f}(\cdot,\boldsymbol{w}):\mathbb{R}^D\to\mathbb{R}^D$ 是连续可微的双射, 其逆映射为 $\boldsymbol{g}(\cdot,\boldsymbol{w})$, 且 $\boldsymbol{g}$ 的 Jacobian 矩阵处处非奇异. 若潜变量 $\boldsymbol{z}$ 的密度为 p_z, 则数据变量 $\boldsymbol{x}=\boldsymbol{f}(\boldsymbol{z},\boldsymbol{w})$ 的密度为

$$p_x(\boldsymbol{x}\mid\boldsymbol{w})=p_z\big(\boldsymbol{g}(\boldsymbol{x},\boldsymbol{w})\big)\,|\det\boldsymbol{J}(\boldsymbol{x})|\,, \tag{18.1}$$

其中 $\boldsymbol{J}(\boldsymbol{x})$ 是偏导数的 Jacobian 矩阵, 其元素由下式给出:

$$J_{ij}(\boldsymbol{x})=\frac{\partial g_i(\boldsymbol{x},\boldsymbol{w})}{\partial x_j}. \tag{18.2}$$

证明

(重构新增) 设 $A \subseteq \mathbb{R}^D$ 为任意 Borel 集. 由于 $\boldsymbol{f}$ 与 $\boldsymbol{g}$ 互为逆映射, 事件 $\{\boldsymbol{x} \in A\}$ 与事件 $\{\boldsymbol{z} \in \boldsymbol{g}(A)\}$ 是同一个事件, 于是概率质量守恒给出

$$\int_A p_x(\boldsymbol{x})\,\mathrm{d}\boldsymbol{x} = \int_{\boldsymbol{g}(A)} p_z(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z}.$$

因为 $\boldsymbol{g}$ 连续可微且 Jacobian 处处非奇异, 由多元积分的变量代换定理, 右端可以改写为对 A 的积分:

$$\int_{\boldsymbol{g}(A)} p_z(\boldsymbol{z})\,\mathrm{d}\boldsymbol{z} = \int_A p_z\big(\boldsymbol{g}(\boldsymbol{x},\boldsymbol{w})\big)\left|\det \frac{\partial \boldsymbol{g}}{\partial \boldsymbol{x}}(\boldsymbol{x})\right|\mathrm{d}\boldsymbol{x}.$$

上式对一切 Borel 集 A 成立, 故两个被积函数几乎处处相等, 即得(18.1), 其中 $\boldsymbol{J}(\boldsymbol{x}) = \partial \boldsymbol{g}/\partial \boldsymbol{x}$ 的元素正是(18.2).

□

注

(重构新增) 这个公式的直观内容是一维情形 $p_x(x) = p_z(g(x))|g'(x)|$ 的多元推广:$\boldsymbol{f}$ 把 $\boldsymbol{z}$ 空间中的一个无穷小体积元拉伸或压缩了 $|\det \boldsymbol{J}|$ 倍, 概率质量必须守恒, 所以密度要乘以这个因子的倒数, 而 $\boldsymbol{J} = \partial \boldsymbol{z}/\partial \boldsymbol{x}$ 恰好就是该因子的倒数 (见练习 18.1). 这也是“非体积保持”(non-volume-preserving) 这一名称的来源: 一般的光滑双射并不保持体积, 唯当 $|\det \boldsymbol{J}| \equiv 1$ 时才保持 (如练习 18.3中的平移).

条件的作用. 第一, 双射与同维数是公式有意义的前提 (见定义 18.1的注记). 第二,$\boldsymbol{g}$ 的连续可微性保证 Jacobian 存在、变量代换定理可用; 仅有可微还不够, 还需要 $\boldsymbol{g}(A)$ 可测等测度论条件, 这些在最常见的 C^1 双射情形下自动成立. 第三,$\det \boldsymbol{J} \neq 0$ 保证密度不退化: 若某点处 $\det \boldsymbol{J} = 0$, 则该点附近 $\boldsymbol{f}$ 把 D 维体积压成低维集合,p_x 在那里会出现奇异性, 这正是命题 16.9所描述的退化情形.

公式中的绝对值不可省. 当 $\det \boldsymbol{J} < 0$ 时映射改变定向, 但密度必须非负, 所以只能取 $|\det \boldsymbol{J}|$; 换句话说, 公式只关心体积的缩放倍数, 不关心定向.

(18.1)中的 $|\cdot|$ 表示模或绝对值. 尽管确定性映射意味着任何给定的数据值 $\boldsymbol{x}$ 都对应一个唯一的 $\boldsymbol{z}$ 值,$\boldsymbol{z}$ 值因此不再不确定, 但我们仍继续将 $\boldsymbol{z}$ 称为“潜变量”.

映射函数 $\boldsymbol{f}(\boldsymbol{z},\boldsymbol{w})$ 则定义为一种特殊形式的神经网络, 我们稍后会讨论其结构. 要求可逆映射的一个结果是潜空间的维度必须与数据空间的维度相同, 这可能导致像图像这样的高维数据会需要很大的模型. 另外, 评估 $D \times D$ 矩阵的行列式的成本通常是 $O(D^3)$, 所以我们需要对模型施加一些额外的限制, 以便更高效地计算 Jacobian 行列式.

考虑一个训练集 $\mathcal{D} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 它由独立数据点组成, 根据(18.1)得到的对数似然函数为

$$\ln p(\mathcal{D} \mid \boldsymbol{w}) = \sum_{n=1}^{N} \ln p_x(\boldsymbol{x}_n \mid \boldsymbol{w}) \tag{18.3}$$

$$= \sum_{n=1}^{N} \left\{\ln p_z\big(\boldsymbol{g}(\boldsymbol{x}_n,\boldsymbol{w})\big) + \ln|\det \boldsymbol{J}(\boldsymbol{x}_n)|\right\}. \tag{18.4}$$

我们的目标是使用似然函数来训练神经网络. 为了能够建模各种各样的分布, 我们希望转换函数 $\boldsymbol{x} = \boldsymbol{f}(\boldsymbol{z},\boldsymbol{w})$ 高度灵活, 因而使用了深度神经网络架构. 如果能够保证网络的每一层都是可逆的, 那么整个函数就是可逆的 (见练习 18.2). 考虑三个连续的转换, 其中的每个转换对应于一个层, 形式如下:

$$\boldsymbol{x} = \boldsymbol{f}^A\big(\boldsymbol{f}^B(\boldsymbol{f}^C(\boldsymbol{z}))\big). \tag{18.5}$$

命题 18.2 (分层流的逆与 Jacobian 链)

设 $\boldsymbol{f}^A, \boldsymbol{f}^B, \boldsymbol{f}^C$ 都可逆, 其逆分别为 $\boldsymbol{g}^A, \boldsymbol{g}^B, \boldsymbol{g}^C$, 且(18.5)中的复合在相应点上连续可微. 则逆函数为

$$\boldsymbol{z} = \boldsymbol{g}^C\big(\boldsymbol{g}^B(\boldsymbol{g}^A(\boldsymbol{x}))\big), \tag{18.6}$$

并且 $\boldsymbol{z}$ 关于 $\boldsymbol{x}$ 的 Jacobian 矩阵的元素满足链式法则

$$J_{ij} = \frac{\partial z_i}{\partial x_j} = \sum_k \sum_l \frac{\partial g_i^C}{\partial g_k^B}\frac{\partial g_k^B}{\partial g_l^A}\frac{\partial g_l^A}{\partial x_j}, \tag{18.7}$$

从而 $\ln|\det \boldsymbol{J}|$ 等于三个层各自对数行列式之和.

♠

证明

(重构新增) 先证(18.6). 把(18.5)的右端逐层代入: 由 $\boldsymbol{g}^A$ 是 $\boldsymbol{f}^A$ 的逆、$\boldsymbol{g}^B$ 是 $\boldsymbol{f}^B$ 的逆、$\boldsymbol{g}^C$ 是 $\boldsymbol{f}^C$ 的逆, 依次有

$$\boldsymbol{g}^A(\boldsymbol{x}) = \boldsymbol{f}^B\big(\boldsymbol{f}^C(\boldsymbol{z})\big), \qquad \boldsymbol{g}^B\big(\boldsymbol{g}^A(\boldsymbol{x})\big) = \boldsymbol{f}^C(\boldsymbol{z}), \qquad \boldsymbol{g}^C\big(\boldsymbol{g}^B(\boldsymbol{g}^A(\boldsymbol{x}))\big) = \boldsymbol{z}.$$

反向代入同样成立, 故(18.6)确是(18.5)的逆 (亦可由练习 18.2的一般结论得到).

再证(18.7). 记 $\boldsymbol{y} = \boldsymbol{g}^A(\boldsymbol{x})$、$\boldsymbol{u} = \boldsymbol{g}^B(\boldsymbol{y})$, 则 $\boldsymbol{z} = \boldsymbol{g}^C(\boldsymbol{u})$, 由多元复合函数的链式法则

$$\frac{\partial z_i}{\partial x_j} = \sum_k \frac{\partial g_i^C}{\partial u_k}\frac{\partial u_k}{\partial x_j} = \sum_k \frac{\partial g_i^C}{\partial u_k}\sum_l \frac{\partial g_k^B}{\partial y_l}\frac{\partial y_l}{\partial x_j} = \sum_k\sum_l \frac{\partial g_i^C}{\partial u_k}\frac{\partial g_k^B}{\partial y_l}\frac{\partial g_l^A}{\partial x_j},$$

其中 $\partial g_i^C/\partial u_k$ 与 $\partial g_k^B/\partial y_l$ 分别就是(18.7)中写作 $\partial g_i^C/\partial g_k^B$ 与 $\partial g_k^B/\partial g_l^A$ 的偏导数 (即在相应中间变量处取值). 写成矩阵形式即 $\boldsymbol{J} = \boldsymbol{J}^C\boldsymbol{J}^B\boldsymbol{J}^A$, 每层 Jacobian 在相应的中间点上计算. 由行列式的乘积法则,

$$|\det \boldsymbol{J}| = \left|\det \boldsymbol{J}^C\right|\left|\det \boldsymbol{J}^B\right|\left|\det \boldsymbol{J}^A\right|,$$

取对数即得 $\ln|\det \boldsymbol{J}| = \ln|\det \boldsymbol{J}^C| + \ln|\det \boldsymbol{J}^B| + \ln|\det \boldsymbol{J}^A|$.

□

注

(重构新增) 动机很直接: 若把整个 $\boldsymbol{f}$ 当成一个黑箱, 算 $\det \boldsymbol{J}$ 要 $O(D^3)$; 改成分层结构后, 只需每层各算一次行列式再相乘, 总代价是各层代价之和, 而每层都可以设计得让行列式便宜 (例如三角矩阵, 只需 $O(D)$). 这一条是整章的结构性基础: 耦合流与自回归流都是在“让每层 Jacobian 成为三角矩阵”这一思路上做文章.

条件方面: 每一层都必须可逆 (否则复合不可逆), 且都要在相应的中间点连续可微; $\boldsymbol{J} = \boldsymbol{J}^C\boldsymbol{J}^B\boldsymbol{J}^A$ 中的三个矩阵在不同的点上取值, 不能误认为同一个点处的常数矩阵. 另外, 层数增加并不改变“可逆性由每层保证”这一逻辑, 但对数行列式的累计是相加的, 这正是深度流模型仍可训练的原因.

一个边界情形: 若某一层 Jacobian 奇异, 则整体 $\det \boldsymbol{J} = 0$,(18.1)在该点失效. 因此实际模型中每层的变换都被设计成行列式恒不为零 (耦合层与自回归流均如此).

(18.7)的右侧为三个矩阵的乘积, 而乘积的行列式是行列式的乘积. 因此, 整体 Jacobian 矩阵的对数行列式将是每层对应对数行列式之和 (参见附录 A).

定义 18.2 (标准化流)

设 p_z 是潜变量 $\boldsymbol{z}$ 上的简单分布 (基础分布), $\boldsymbol{f}(\cdot, \boldsymbol{w})$ 是由深度神经网络给出的可逆映射. 若数据分布 p_x 由命题 18.1的变量变换公式从 p_z 与 $\boldsymbol{f}$ 确定, 则称这种对灵活分布进行建模的方法为**标准化流** (normalizing flow), 并称复合成 $\boldsymbol{f}$ 的每个可逆映射为一层流.

♣

注

(重构新增) “流”这个名称来自直观图像: 通过一系列映射来转换概率分布的过程, 在某种程度上类似于流体的流动——密度沿着流线被搬运、被拉伸压缩, 而质量始终守恒. 逆映射方向的效果则相反: 它把复杂的数据分布“标准化”成简单形式, 通常就是 Gauss 分布, 这正是名称中“标准化”的含义.

这个定义与第 19 章的变分自编码器、第 17 章的生成对抗网络构成对照: 三者都用一个神经网络把简单分布搬到数据空间, 差别在于对“可逆”与“似然精确”的取舍. 标准化流要求可逆, 代价是潜空间与数据空间必须同维、网络结构受限; 回报是似然函数可以被精确计算, 而且采样只需一次前向传播.

条件方面, 定义要求 p_z 是“简单”分布, 这是为了让 p_z 的对数密度容易计算 (例如标准 Gauss 的对数密度就是 $-\|\boldsymbol{z}\|^2/2$ 加常数), 并且容易采样; 要求 $\boldsymbol{f}$ 可逆, 是为了让(18.1)成立. 这两条缺一不可.

还要注意, 定义并未要求 $\boldsymbol{f}$ 只有一层; 恰恰相反, 实际模型总是把许多层复合起来, 用命题 18.2把整体的对数行列式化归为各层之和. 单层负责“便宜且可逆”, 多层负责“灵活”, 这是标准化流的基本设计原则.

标准化流的详细介绍参见 Kobyzev, Prince, and Brubaker (2019) 以及 Papamakarios et al. (2019). 在本章中, 我们将讨论实践中经常使用的两个主要类型的标准化流的核心概念: 耦合流 (coupling flow) 和自回归流 (autoregressive

flow). 此外, 我们还将介绍使用神经微分方程定义可逆映射的方法, 并引入连续流 (continuous flow) 这一概念.

18.1 耦合流

我们的目标是设计一个单一的可逆函数层, 这样就可以将许多这样的层组合在一起, 从而定义一类高度灵活的可逆函数. 首先考虑以下形式的线性变换:

$$\boldsymbol{x} = a\boldsymbol{z} + \boldsymbol{b}, \tag{18.8}$$

这个变换很容易求逆, 得到

$$\boldsymbol{z} = \frac{1}{a}(\boldsymbol{x} - \boldsymbol{b}). \tag{18.9}$$

然而, 线性变换在组合下是封闭的, 这意味着一系列线性变换等同于一个整体的线性变换 (见练习 3.6). 因为 Gauss 分布的线性变换结果还是 Gauss 分布. 所以即使我们有许多这样的线性变换 "层", 我们也只能得到一个 Gauss 分布. 问题在于我们能否保持线性变换的可逆性, 同时允许额外的灵活性, 从而使得最终结果可以是非 Gauss 分布.

命题 18.3 (线性层的复合)

设 $a, a' \neq 0$ 为实数,$\boldsymbol{b}, \boldsymbol{b}'$ 为 D 维向量. 若 $\boldsymbol{x} = a\boldsymbol{z} + \boldsymbol{b}$ 且 $\boldsymbol{z} = a'\boldsymbol{u} + \boldsymbol{b}'$, 则 $\boldsymbol{x} = aa'\boldsymbol{u} + (a\boldsymbol{b}' + \boldsymbol{b})$, 仍是一个可逆的线性变换. 若进一步 $\boldsymbol{z} \sim \mathcal{N}(\boldsymbol{m}, \boldsymbol{\Sigma})$, 则 $\boldsymbol{x} \sim \mathcal{N}(a\boldsymbol{m} + \boldsymbol{b}, a^2\boldsymbol{\Sigma})$, 仍是 Gauss 分布.

证明

(重构新增) 第一个断言由直接代入得到:

$$\boldsymbol{x} = a(a'\boldsymbol{u} + \boldsymbol{b}') + \boldsymbol{b} = aa'\boldsymbol{u} + (a\boldsymbol{b}' + \boldsymbol{b}),$$

因为 $a, a' \neq 0$, 故 $aa' \neq 0$, 该线性变换仍可逆, 逆变换即由上式解出 $\boldsymbol{u}$.

第二个断言用特征函数验证. 设 $\boldsymbol{z}$ 的特征函数为 $\varphi_{\boldsymbol{z}}(\boldsymbol{t}) = \exp\left(\mathrm{i}\boldsymbol{t}^{\mathsf{T}}\boldsymbol{m} - \frac{1}{2}\boldsymbol{t}^{\mathsf{T}}\boldsymbol{\Sigma}\boldsymbol{t}\right)$, 则

$$\varphi_{\boldsymbol{x}}(\boldsymbol{t}) = \mathbb{E}\left[\mathrm{e}^{\mathrm{i}\boldsymbol{t}^{\mathsf{T}}(a\boldsymbol{z}+\boldsymbol{b})}\right] = \mathrm{e}^{\mathrm{i}\boldsymbol{t}^{\mathsf{T}}\boldsymbol{b}}\varphi_{\boldsymbol{z}}(a\boldsymbol{t}) = \exp\left(\mathrm{i}\boldsymbol{t}^{\mathsf{T}}(a\boldsymbol{m} + \boldsymbol{b}) - \frac{1}{2}\boldsymbol{t}^{\mathsf{T}}(a^2\boldsymbol{\Sigma})\boldsymbol{t}\right),$$

这正是均值为 $a\boldsymbol{m} + \boldsymbol{b}$、协方差为 $a^2\boldsymbol{\Sigma}$ 的 Gauss 分布的特征函数, 由特征函数的唯一性即得结论.

□

注

(重构新增) 这个命题解释了为什么单靠线性层无法构造灵活的生成模型: 线性变换的复合仍是线性变换, 而 Gauss 分布在仿射变换下保持不变, 所以无论堆多少层, 结果分布仍是 Gauss 分布. 若要得到非 Gauss 分布, 就必须引入非线性, 同时又不破坏可逆性——这正是下一段耦合层出场的原因.

条件方面:$a \neq 0$ 保证可逆 ($a = 0$ 时映射把整个空间压成一点, 不可逆, 行列式为零, 密度退化); 命题对 $\boldsymbol{\Sigma}$ 无正定性要求, 只要求它是合法的协方差矩阵,$a^2\boldsymbol{\Sigma}$ 自动对称半正定. 第二个断言依赖 a 为不随分量变化的常数; 若换成一般的可逆矩阵 $\boldsymbol{A}$, 结论仍成立, 协方差变为 $\boldsymbol{A}\boldsymbol{\Sigma}\boldsymbol{A}^{\mathsf{T}}$, 而 Jacobian 行列式因子变成 $|\det \boldsymbol{A}|$.

解决这个问题的一种方法是使用一个叫作实值 NVP(real-valued non-volume-preserving, RealNVP)(Dinh, Krueger, and Bengio, 2014; Dinh, Sohl-Dickstein, and Bengio, 2016) 的标准化流模型. 这里的 NVP 代表非体积保持 (non-volume-preserving).

定义 18.3 (实值 NVP 耦合层)

设潜变量向量 $\boldsymbol{z}$ 被划分为两部分 $\boldsymbol{z} = (\boldsymbol{z}_A, \boldsymbol{z}_B)$: 若 $\boldsymbol{z}$ 的维度为 D 且 $\boldsymbol{z}_A$ 的维度为 d, 则 $\boldsymbol{z}_B$ 的维度为 $D - d$. 类似地划分输出向量 $\boldsymbol{x} = (\boldsymbol{x}_A, \boldsymbol{x}_B)$, 其中 $\boldsymbol{x}_A$ 的维度为 d,$\boldsymbol{x}_B$ 的维度为 $D - d$. 对于输出向量的第一部分, 我们简单地复制输入

$$\boldsymbol{x}_A = \boldsymbol{z}_A, \tag{18.10}$$

输出向量的第二部分经历了一个线性变换, 但这个线性变换中的系数由 z_A 的非线性函数给出:

$$\boldsymbol{x}_B = \exp\big(\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})\big) \odot \boldsymbol{z}_B + \boldsymbol{b}(\boldsymbol{z}_A, \boldsymbol{w}), \tag{18.11}$$

其中 $\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})$ 和 $\boldsymbol{b}(\boldsymbol{z}_A, \boldsymbol{w})$ 是神经网络的实值输出, 指数函数确保了乘法项是非负的;$\odot$ 表示 Hadamard 积 (Hadamard product), 即对两个向量进行逐元素相乘,(18.11)中的指数也是逐元素计算的. 称由(18.10)与(18.11)定义的层为**实值 NVP 耦合层**.

♣

注

(重构新增) 动机:命题 18.3说明线性层堆叠不产生任何新的分布形状. 耦合层保留线性层“容易求逆”的优点, 但把缩放系数与平移量换成 z_A 的任意非线性函数 (由神经网络给出), 于是 $\boldsymbol{x}_B$ 可以是 z_A 的非常灵活的形变, 而 z_A 这一半原样保留. 这一“分而治之”的结构使得逆映射仍然显式可写 (见命题 18.4).

逐项条件:z_A 与 $\boldsymbol{x}_A$ 的维数必须相同 (都是 d), 否则(18.10)无从写起;$0 \leqslant d \leqslant D$, 而 $d = 0$ 或 $d = D$ 时该层退化为恒等映射、失去作用, 故实际取 $1 \leqslant d \leqslant D - 1$.$\boldsymbol{s}$ 与 $\boldsymbol{b}$ 可以是任意函数, 不需要可逆, 甚至不需要连续; 指数函数的作用只是把乘性因子约束为正, 从而 $\boldsymbol{x}_B$ 关于 $\boldsymbol{z}_B$ 的变换是保向的、单调的, 保证逆存在.$\boldsymbol{w}$ 是网络的参数向量.

“非体积保持”这个名字说的正是行列式不为 1: 由命题 18.5, 该层的体积缩放因子是 $\exp\left(\sum_i s_i\right)$, 一般不等于 1, 故称非体积保持.

注意我们在两个网络函数中展示了相同的向量 $\boldsymbol{w}$. 在实践中, 它们可能实现为具有不同参数的独立网络, 或者一种具有两套输出的网络.

由于使用了神经网络函数,$\boldsymbol{x}_B$ 的值可以是 z_A 的一个非常灵活的函数. 尽管如此, 整体的变换仍然很容易逆推. 给定 $\boldsymbol{x} = (\boldsymbol{x}_A, \boldsymbol{x}_B)$ 的值, 首先计算

$$\boldsymbol{z}_A = \boldsymbol{x}_A, \tag{18.12}$$

命题 18.4 (耦合层的逆)

设耦合层由(18.10)与(18.11)给出. 则对任意 $\boldsymbol{x} = (\boldsymbol{x}_A, \boldsymbol{x}_B)$, 先由(18.12)得到 z_A, 再计算 $\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})$ 和 $\boldsymbol{b}(\boldsymbol{z}_A, \boldsymbol{w})$, 最后使用下式计算 z_B:

$$\boldsymbol{z}_B = \exp\big(-\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})\big) \odot \big(\boldsymbol{x}_B - \boldsymbol{b}(\boldsymbol{z}_A, \boldsymbol{w})\big), \tag{18.13}$$

就得到唯一的原像 $\boldsymbol{z} = (\boldsymbol{z}_A, \boldsymbol{z}_B)$, 即(18.12)与(18.13)给出该耦合层的逆映射.

♠

证明

(重构新增) 由(18.10),$\boldsymbol{z}_A = \boldsymbol{x}_A$ 是被迫的, 且 $\boldsymbol{z}_A$ 一旦确定,$\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})$ 与 $\boldsymbol{b}(\boldsymbol{z}_A, \boldsymbol{w})$ 就完全确定 (它们不依赖 $\boldsymbol{z}_B$). 把(18.13)代回(18.11):

$$\exp(\boldsymbol{s}) \odot \boldsymbol{z}_B + \boldsymbol{b} = \exp(\boldsymbol{s}) \odot \big[\exp(-\boldsymbol{s}) \odot (\boldsymbol{x}_B - \boldsymbol{b})\big] + \boldsymbol{b} = (\boldsymbol{x}_B - \boldsymbol{b}) + \boldsymbol{b} = \boldsymbol{x}_B,$$

其中用到 Hadamard 积的逐元素结合律与 $\exp(\boldsymbol{s}) \odot \exp(-\boldsymbol{s}) = \boldsymbol{1}$. 反向代入 (把(18.11)代入(18.13)) 同样还原出 $\boldsymbol{z}_B$. 故两式互为逆映射; 唯一性来自(18.13)的显式性: 给定 $\boldsymbol{x}$ 后每一步都没有选择余地.

□

注

(重构新增) 这个命题是耦合流全部实用价值的来源: 正向 (采样方向) 与逆向 (似然方向) 都只需一次神经网络求值加逐元素运算, 没有任何迭代或数值求解, 代价与普通前馈网络同阶. 注意逆映射只用到 $\boldsymbol{s}$ 与 $\boldsymbol{b}$ 本身, 从不要求这两个网络可逆——这是耦合结构的关键技巧: 把一个 D 维的可逆性问题化归为两个任意函数的求值问题.

条件方面:(18.13)只用到 $\exp(-\boldsymbol{s})$ 的逐元素乘法与减法, 对 $\boldsymbol{s}, \boldsymbol{b}$ 没有额外要求, 因此结论对任意 $\boldsymbol{s}, \boldsymbol{b}$ 都成立; 实际使用中取神经网络只是为了灵活性. 若把(18.11)中的 $\exp(\boldsymbol{s})$ 换成任意逐元素正值函数, 逆的公式相应改为除以该函数, 结论不变.

一个值得注意的细节:$\boldsymbol{x}_A = \boldsymbol{z}_A$ 意味着这一半坐标在变换中完全没有改变, 因此单层耦合层的表达能力有限

——这正是要引入“第二层、交换两个部分的角色”的原因.

整体变换如图 18.1所示. 注意, 单个神经网络函数, 如 $\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})$ 和 $\boldsymbol{b}(\boldsymbol{z}_A, \boldsymbol{w})$, 并不要求是可逆的.

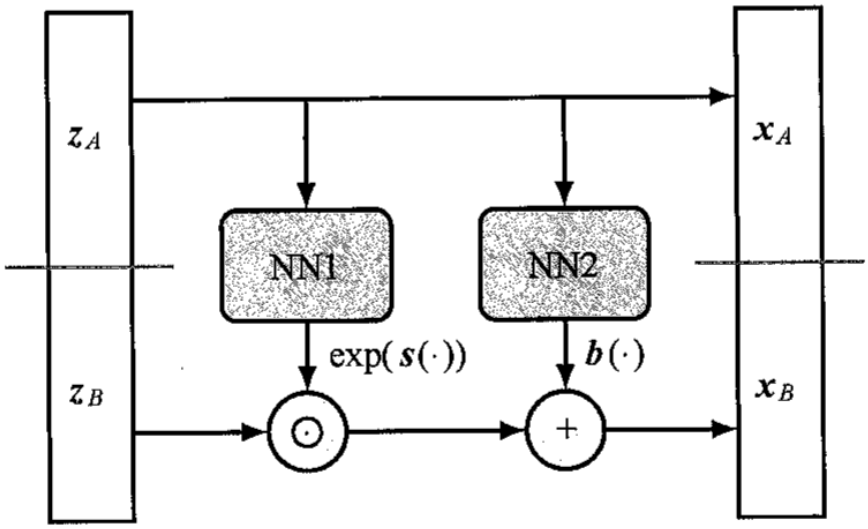


图 18.1: 实值 NVP 标准化流模型的单层结构. 在这里, 网络 NN1 计算函数 $\exp\{\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})\}$, 而网络 NN2 计算函数 $\boldsymbol{b}(\boldsymbol{z}_A, \boldsymbol{w})$. 输出向量由(18.10)和(18.11)定义

考虑对(18.2)定义的 Jacobian 矩阵及其行列式进行计算. 可以将该 Jacobian 矩阵分成块, 对应于 $\boldsymbol{z}$ 和 $\boldsymbol{x}$ 的划分.

命题 18.5 (耦合层 Jacobian 的行列式)

设耦合层由(18.10)与(18.11)给出,$\boldsymbol{J}(\boldsymbol{x}) = \partial \boldsymbol{z}/\partial \boldsymbol{x}$ 为(18.2)所定义的 Jacobian 矩阵. 则按 $\boldsymbol{z} = (\boldsymbol{z}_A, \boldsymbol{z}_B)$、$\boldsymbol{x} = (\boldsymbol{x}_A, \boldsymbol{x}_B)$ 的划分,

$$\boldsymbol{J} = \begin{bmatrix} \boldsymbol{I}_d & \boldsymbol{0} \\ \dfrac{\partial \boldsymbol{z}_B}{\partial \boldsymbol{x}_A} & \operatorname{diag}\big(\exp(-\boldsymbol{s})\big) \end{bmatrix}, \tag{18.14}$$

其中 $\boldsymbol{I}_d$ 是 $d \times d$ 单位矩阵,$\boldsymbol{0}$ 是 $d \times (D-d)$ 零矩阵,$\operatorname{diag}\big(\exp(-\boldsymbol{s})\big)$ 是以 $\exp(-s_i(\boldsymbol{z}_A, \boldsymbol{w}))$ 为对角元的 $(D-d) \times (D-d)$ 对角矩阵. 因此 $\boldsymbol{J}$ 是下三角块矩阵, 其行列式只由主对角线上的元素决定:

$$\det \boldsymbol{J}(\boldsymbol{x}) = \prod_{i=1}^{D-d} \exp\big(-s_i(\boldsymbol{z}_A, \boldsymbol{w})\big) = \exp\left(-\sum_{i=1}^{D-d} s_i(\boldsymbol{z}_A, \boldsymbol{w})\right). \tag{18.15}$$

证明

(重构新增) 逐块计算偏导数. 左上角的块对应于 $\boldsymbol{z}_A$ 关于 $\boldsymbol{x}_A$ 的导数, 根据(18.12),$\boldsymbol{z}_A = \boldsymbol{x}_A$, 故该块为 $d \times d$ 的单位矩阵 $\boldsymbol{I}_d$. 右上角的块对应于 $\boldsymbol{z}_A$ 关于 $\boldsymbol{x}_B$ 的导数, 根据(18.12),$\boldsymbol{z}_A$ 与 $\boldsymbol{x}_B$ 无关, 结果为 $\boldsymbol{0}$.

左下角的块对应于 $\boldsymbol{z}_B$ 关于 $\boldsymbol{x}_A$ 的导数, 根据(18.13), 这是关于神经网络函数的复杂表达式:$\boldsymbol{s}$ 与 $\boldsymbol{b}$ 都通过 $\boldsymbol{z}_A = \boldsymbol{x}_A$ 依赖 $\boldsymbol{x}_A$, 求导时要用链式法则, 其结果不需要显式写出.

最后, 右下角的块对应于 $\boldsymbol{z}_B$ 关于 $\boldsymbol{x}_B$ 的导数. 根据(18.13), 在固定 $\boldsymbol{z}_A = \boldsymbol{x}_A$ 时,

$$\frac{\partial (\boldsymbol{z}_B)_i}{\partial (\boldsymbol{x}_B)_j} = \exp\big(-s_i(\boldsymbol{z}_A, \boldsymbol{w})\big)\delta_{ij},$$

即这些导数由一个对角矩阵表示, 其对角元素是 $\exp\big(-s_i(\boldsymbol{z}_A, \boldsymbol{w})\big)$. 于是 $\boldsymbol{J}$ 具有(18.14)的形状, 按 $d \times d$ 分块是下三角矩阵, 这意味着其中所有主对角线以上的块元素都是零 (参见附录 A). 对于这样的分块三角矩阵, 其行列式只是沿主对角线各块行列式的乘积 (按行做消去的过程不改变行列式, 可把左下角的块消成零), 故

$$\det \boldsymbol{J} = \det(\boldsymbol{I}_d) \cdot \det\Big(\operatorname{diag}\big(\exp(-\boldsymbol{s})\big)\Big) = 1 \cdot \prod_{i=1}^{D-d} \exp\big(-s_i(\boldsymbol{z}_A, \boldsymbol{w})\big),$$

而不依赖于左下角的块中的复杂表达式, 即得(18.15).

□

注

(重构新增) 这条结论是耦合流能够实用的根本原因: 计算 $D \times D$ 一般矩阵的行列式需要 $O(D^3)$, 而(18.15)只需把 $D - d$ 个指数相加, 代价是 $O(D)$; 同时左下角那个“关于神经网络函数的复杂表达式”根本不必计算, 因为它不影响行列式. 更一般地, 只要每层的 Jacobian 是三角矩阵, 整层的对数行列式就是对角元对数之和, 这正是命题 18.2所要求的“每层便宜”.

条件方面: 结论只用到(18.13)的逐元素结构, 对 $\boldsymbol{s}$ 没有任何限制;$\exp(-s_i)$ 恒正, 故 $\det \boldsymbol{J} \neq 0$ 自动成立, 与命题 18.1的非奇异要求相容. 这也说明耦合层的可逆性是结构性的, 而非数值性的: 不需要求解线性方程组, 也不会出现病态.

注意(18.15)给出的是 $\partial \boldsymbol{z}/\partial \boldsymbol{x}$ 的行列式. 若改用正向映射的 Jacobian $\partial \boldsymbol{x}/\partial \boldsymbol{z}$, 由练习 18.1知二者互为倒数, 其行列式为 $\exp\left(\sum_i s_i\right)$. 两者互为倒数 (指数相差一个符号), 代入(18.1)时必须用前者.

因此,Jacobian 行列式可以简单地由 $\exp\left(\boldsymbol{s}(\boldsymbol{z}_A, \boldsymbol{w})\right)$ 中元素的乘积给出.

注

(重构新增, 原文问题说明) 上面这句译文忠实保留了原文, 但按字面它与(18.14)、(18.15)不一致, 需要说明清楚. 原文前一句还说“其对角元素是 $s(\boldsymbol{z}_A, \boldsymbol{w})$”, 而(18.14)中右下角块的对角元实际是 $\exp(-s_i)$, 并非 s_i 本身.

准确的对应关系是

$$\det \frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \prod_i \exp(-s_i) = \exp\left(-\sum_i s_i\right), \qquad \det \frac{\partial \boldsymbol{x}}{\partial \boldsymbol{z}} = \prod_i \exp(s_i) = \exp\left(\sum_i s_i\right),$$

二者互为倒数 (练习 18.1). 原文“由 $\exp(\boldsymbol{s})$ 中元素的乘积给出”说的其实是正向 Jacobian $\partial \boldsymbol{x}/\partial \boldsymbol{z}$ 的行列式; 而对密度公式(18.1)而言必须取 $\partial \boldsymbol{z}/\partial \boldsymbol{x}$ 的绝对值, 即 $\exp\left(-\sum_i s_i\right)$. 这不是文字游戏: 若误把 $\exp\left(+\sum_i s_i\right)$ 当作 $|\det \boldsymbol{J}|$, 算出的密度会相差一个因子 $\exp\left(2\sum_i s_i\right)$, 似然也就错了. 本书后文 (例如“每一项本身就是形式为 $-s_i(\boldsymbol{x}, \boldsymbol{w})$ 的项之和”) 用的正是负号, 与(18.15)一致.

这种方法的一个明显局限是,$\boldsymbol{z}_A$ 的值没有因变换而改变. 这可以通过添加另一层来轻松解决, 其中 $\boldsymbol{z}_A$ 和 $\boldsymbol{z}_B$ 的角色被颠倒过来, 如图 18.2所示. 这种双层结构可以重复多次, 从而能构建一类非常灵活的生成式模型.

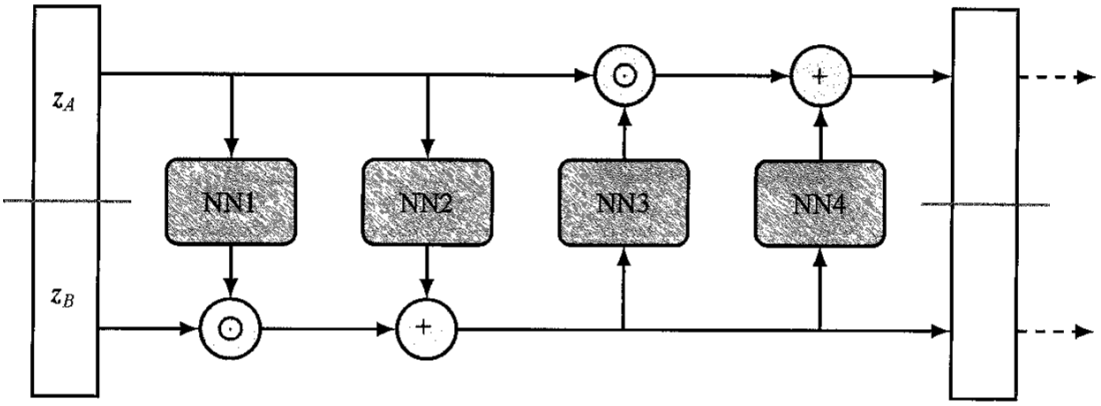


图 18.2: 通过组合两个图 18.1所示的层, 我们得到了一个更加灵活但仍然可逆的非线性层. 其中的每个子层都是可逆的, 并且拥有容易计算的 Jacobian 矩阵, 因此整个双层结构也具有相同的特性

整个训练过程包括创建数据点的一些小批量, 在这些小批量中, 每个数据点对对数似然函数的贡献可从(18.4)中获得. 对于形式为 $\mathcal{N}(\boldsymbol{z} \mid \boldsymbol{0}, \boldsymbol{I})$ 的潜在分布, 对数密度就是 $-\|\boldsymbol{z}\|^2/2$ 加一个加法常数. 逆变换 $\boldsymbol{z} = \boldsymbol{g}(\boldsymbol{x})$ 可以使用一系列形式为(18.13)的逆变换计算得出. 同样,Jacobian 行列式的对数由每层的对数行列式之和给出, 其中的每一项本身就是形式为 $-s_i(\boldsymbol{x}, \boldsymbol{w})$ 的项之和. 对数似然的梯度可以通过自动微分评估, 并且网络参数可以通过

随机梯度下降来更新.

实值 NVP 模型属于一类使用广泛的标准化流, 称为**耦合流** (coupling flows).

定义 18.4 (耦合流)

在耦合流中,(18.11)中的线性变换被如下更一般的形式替换:

$$\boldsymbol{x}_B = \boldsymbol{h}\big(\boldsymbol{z}_B, \boldsymbol{g}(\boldsymbol{z}_A, \boldsymbol{w})\big), \tag{18.16}$$

其中 $\boldsymbol{h}(\boldsymbol{z}_B, \boldsymbol{g})$ 是 $\boldsymbol{z}_B$ 的函数, 对于任何给定的 $\boldsymbol{g}$ 值, 它都可以高效地逆推, 称为**耦合函数** (coupling function);$\boldsymbol{g}(\boldsymbol{z}_A, \boldsymbol{w})$ 称为**条件函数** (conditioner), 通常用神经网络来表示.

♣

注

(重构新增) (18.16)把(18.11)中的“缩放加平移”抽象成任意的、对第二个变量可高效求逆的 $\boldsymbol{h}$. 这样做的动机是: 行列式的结论并不依赖 $\boldsymbol{h}$ 是线性形式, 只要 $\boldsymbol{h}(\cdot, \boldsymbol{g})$ 对每个固定的 $\boldsymbol{g}$ 是 $\boldsymbol{z}_B$ 的一一映射, 且其 Jacobian 关于 $\boldsymbol{z}_B$ 是三角 (在逐元素情形下就是对角) 结构, 整层的对数行列式仍可廉价计算; 而 $\boldsymbol{h}$ 取得越一般, 模型的表达力越强.

条件方面:$\boldsymbol{h}(\cdot, \boldsymbol{g})$ 必须对每个 $\boldsymbol{g}$ 可逆 (否则整层不可逆);$\boldsymbol{g}(\boldsymbol{z}_A, \boldsymbol{w})$ 可以是任意函数, 不需要可逆——它只提供“条件”, 不参与 $\boldsymbol{z}_B$ 方向的可逆性. 当 $\boldsymbol{h}(\boldsymbol{z}_B, \boldsymbol{g}) = \exp(\boldsymbol{g}) \odot \boldsymbol{z}_B + \boldsymbol{c}$ 时, 耦合流退化为实值 NVP; 当 $\boldsymbol{h}$ 取逐元素的单调递增函数时, 得到所谓“单调耦合流”. 这两类都保持“三角 Jacobian”这一关键性质.

与自回归流的关系值得记住: 若把(18.16)中的“划分成两组”改为“逐坐标条件化”, 就得到节 18.2的自回归流.

我们可以使用一个简单的数据集来说明实值NVP 标准化流, 这个数据集有时称为“双月”数据集, 如图 18.3所示.

例题 18.1 双月数据集 取二维潜空间 ($D = 2$) 与标准 Gauss 基础分布 $\mathcal{N}(\boldsymbol{z} \mid \boldsymbol{0}, \boldsymbol{I})$, 用两个连续的耦合层复合: 第一层取 $d = 1$, 令 $\boldsymbol{z}_A$ 为水平轴分量、$\boldsymbol{z}_B$ 为垂直轴分量, 于是 $\boldsymbol{x}_A = \boldsymbol{z}_A$ 保持水平轴不变, 而 $\boldsymbol{x}_B$ 只对垂直轴施加由水平轴决定的缩放与平移; 第二层交换两个分量的角色, 只对水平轴施加由垂直轴决定的变换. 于是每一层都只改变一个坐标, 两层交替之后两个坐标都被非线性地重排. 把基础分布的大批样本依次通过这两层, 就得到图 18.3中从 (a) 到 (e) 的演化, 最终样本分布逼近 (f) 所示的训练集形状. 整条路径上每一步都是可逆的, 故同样可以把数据点逆向送回潜空间并精确计算其对数密度.

18.2 自回归流

我们注意到一组变量的联合分布总是可以写成条件分布的乘积, 每个变量对应一个条件分布, 由此可以推导出相关的标准化流公式 (参见节 11.1). 首先选择向量 $\boldsymbol{x}$ 中变量的顺序, 我们可以在不失一般性的情况下写出:

$$p(x_1, \cdots, x_D) = \prod_{i=1}^{D} p(x_i \mid \boldsymbol{x}_{1:i-1}), \tag{18.17}$$

其中 $\boldsymbol{x}_{1:i-1}$ 表示 $(x_1, \cdots, x_{i-1})$.

命题 18.6 (条件分布的链式分解)

设随机向量 $\boldsymbol{x} = (x_1, \cdots, x_D)$ 的联合密度为 p, 且各条件密度 $p(x_i \mid \boldsymbol{x}_{1:i-1})$ 存在. 则(18.17)成立; 特别地, 当 $i = 1$ 时约定 $\boldsymbol{x}_{1:0}$ 为空,$p(x_1 \mid \boldsymbol{x}_{1:0}) = p(x_1)$.

♠

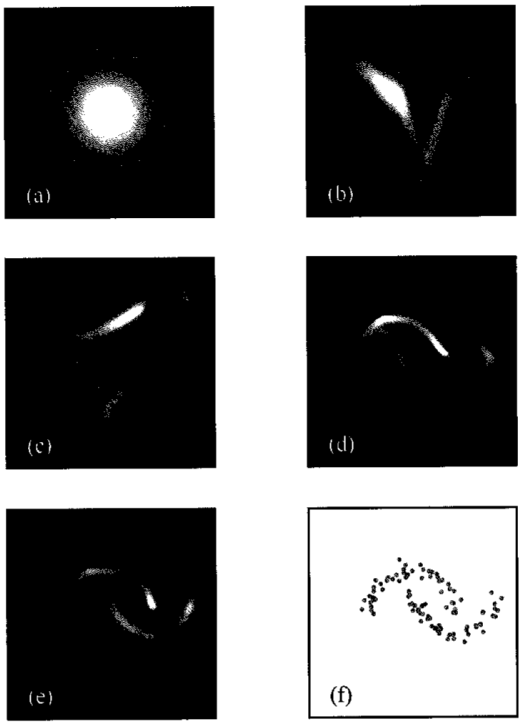


图 18.3: 在实值 NVP 标准化流模型中, 通过交替地对不同维度应用变换, 可以将基础的 Gauss 分布逐步转换为更复杂的数据分布.(a) 基础的 Gauss 分布: 初始的二维 Gauss 分布通常均值为零且协方差矩阵为单位矩阵, 表示潜变量空间中的点.(b) 垂直轴变换后的分布: 第一层变换只对垂直轴应用耦合函数和条件函数, 并保持水平轴不变.(c) 水平轴变换后的分布: 第二层变换只对水平轴进行变换, 并利用前一变换的输出作为输入, 而不改变垂直轴.(d) 第二次垂直轴变换后的分布: 经过前两次变换后, 再次对垂直轴进行变换, 进一步增加分布的复杂性.(e) 第二次水平轴变换后的分布: 最后再次对水平轴进行变换, 以此类推, 可以通过持续地交替进行多层变换来获得越来越复杂的数据分布.(f) 模型训练用的数据集: 最终, 这些连续的变换会将基础的 Gauss 分布转换成与训练集相匹配的复杂形状, 也就是所谓的“双月”数据集

证明

(重构新增) 对 D 归纳.$D=1$ 时(18.17)两端都是 $p(x_1)$, 按空条件的约定成立.

设结论对 $D-1$ 成立. 由条件密度的定义 (见节 2.2), 在 $p(x_1,\cdots,x_{D-1})>0$ 的点处有

$$p(x_1,\cdots,x_D)=p(x_D\mid \boldsymbol{x}_{1:D-1})\,p(x_1,\cdots,x_{D-1}).$$

右端第二个因子由归纳假设等于 $\prod_{i=1}^{D-1} p(x_i\mid \boldsymbol{x}_{1:i-1})$, 代入即得(18.17). 若某点处 $p(x_1,\cdots,x_{D-1})=0$, 则左端也为零, 等式仍成立.

□

注

(重构新增) 这个恒等式本身只是概率的乘积法则反复使用, 但它是自回归建模的全部起点: 它把一个 D 维联合分布的问题拆成 D 个一维条件分布的问题, 而每个一维条件分布都可以用一个神经网络来参数化 (例如取成均值和方差由网络给出的 Gauss 分布).节 11.3中的自回归模型正是按这个思路构造的.

条件方面: 结论只要求各条件密度存在, 对变量是离散还是连续、条件依赖哪一段前缀都没有限制. 所谓“不失一般性”指的是变量的排列顺序可以任意选定: 换一个顺序就换一种分解,(18.17)对任何顺序都成立, 但不同的顺序给出不同的模型族 (这与第 11 章中讨论的变量顺序问题同源).

与标准化流的联系在于: 若把每个条件分布的实现写成 $x_i=h(z_i,g_i(\boldsymbol{x}_{1:i-1}))$, 其中 z_i 取自简单的基础分布, 那么整条变换的 Jacobian 自动是三角的——因为 x_i 只依赖 $\boldsymbol{x}_{1:i-1}$ 与 z_i, 不依赖 $z_{i+1},\cdots,z_D$. 三角结构又让行列式变成对角元之积, 于是似然可以精确而廉价地计算. 这正是下面两个模型的共同机制.

这种因式分解可以用来构建一类称为**掩码自回归流** (Masked Autoregressive Flow, MAF)(Papamakarios, Pavlakou, and Murray, 2017) 的标准化流, 如下所示:

定义 18.5 (掩码自回归流)

设潜变量为 $\boldsymbol{z}=(z_1,\cdots,z_D)$、数据变量为 $\boldsymbol{x}=(x_1,\cdots,x_D)$. 掩码自回归流是形如

$$x_i=h\big(z_i,g_i(\boldsymbol{x}_{1:i-1},\boldsymbol{w}_i)\big) \tag{18.18}$$

的变换, 其中 $h(z_i,\cdot)$ 是耦合函数, 因为对每个给定的条件值都可以很容易地对 z_i 求逆;g_i 是条件函数, 通常用一个深度神经网络来表示,$\boldsymbol{w}_i$ 是它的参数.

♣

注

(重构新增) 动机:(18.17)把联合密度拆成条件密度之积, 而(18.18)正是“逐坐标地把基础分布的条件分位数搬运成数据的条件分布”的实现. 因为第 i 个输出只用到前 $i-1$ 个输入, 整条变换天然是三角的, 于是既能精确算似然, 又能保持可逆.

条件方面:$h(z_i,\cdot)$ 只需在“第一个变量”方向上可逆, 对条件函数 g_i 没有任何可逆性要求 (这与耦合流中的技巧完全相同);$\boldsymbol{w}_i$ 允许随 i 不同, 实际实现中常让所有 i 共享一个网络、靠掩码区分.

一个容易忽略的细节:(18.18)要求 x_i 只依赖 $\boldsymbol{x}_{1:i-1}$ 而不依赖 $\boldsymbol{x}_{i+1:D}$, 这正是“自回归”的含义, 也是三角性的来源. 若破坏了这一依赖结构, 行列式就不再是三角的, 计算代价立刻回到 $O(D^3)$.

图 18.4(a) 说明了这一点. 术语“掩码”指的是使用单个神经网络来实现一组形式为(18.18)的方程, 连同一个二进制掩码 (Germain et al., 2015) 来强制网络权重的一个子集为零, 以实现自回归约束 [(18.17)] .

在这种情况下, 计算似然函数所需的逆运算如下:

命题 18.7 (掩码自回归流的逆与三角 Jacobian)

设掩码自回归流由(18.18)给出, 且每个 $h(z_i,\cdot)$ 对固定的条件值都是 z_i 的可逆函数. 则计算似然函数所需的逆运算为

$$z_i=h^{-1}\big(x_i,g_i(\boldsymbol{x}_{1:i-1},\boldsymbol{w}_i)\big). \tag{18.19}$$

由于(18.19)中 $z_1,\cdots,z_D$ 这些分项函数只依赖已知的 $\boldsymbol{x}$, 它们可以并行计算, 因此评估以上似然函数的操作

在现代硬件上就能高效执行. 对应于这组变换的 Jacobian 矩阵的元素为 $\partial z_i/\partial x_j$, 这些元素形成一个三角矩阵, 其行列式由对角线元素的乘积给出, 因此也可以高效计算 (见练习 18.4).

证明

(重构新增) 先证(18.19). 由(18.18),$x_i = h\big(z_i, g_i(\boldsymbol{x}_{1:i-1}, \boldsymbol{w}_i)\big)$, 其中条件值 $g_i(\boldsymbol{x}_{1:i-1}, \boldsymbol{w}_i)$ 只依赖 $\boldsymbol{x}$ 的前 $i-1$ 个分量、与 z_i 无关. 对固定的条件值把 $h(\cdot, g_i)$ 反解, 即得(18.19); 由于每个 z_i 的右端只用到已知的 $\boldsymbol{x}$, 这 D 个式子之间没有依赖, 可以并行求值.

再证三角性. 由(18.19),z_i 只依赖 $x_1, \cdots, x_i$, 与 $x_{i+1}, \cdots, x_D$ 无关, 故当 $j > i$ 时

$$\frac{\partial z_i}{\partial x_j} = 0,$$

即 $\boldsymbol{J}$ 的主对角线以上元素全为零, 它是下三角矩阵. 下三角矩阵的行列式等于对角元之积,

$$\det \boldsymbol{J} = \prod_{i=1}^{D} \frac{\partial z_i}{\partial x_i},$$

其中 $\partial z_i/\partial x_i = \partial h^{-1}(x_i, g_i)/\partial x_i$(固定条件值求导), 于是只需 D 次一维求导即可得到行列式.

□

注

(重构新增) 这条结论说明掩码自回归流在“评估似然”这一方向上极其高效: 一次前向就能得到全部 z_i, 行列式只是 D 个对角元的乘积, 总代价与一次普通网络前向同阶. 代价出在反方向上: 要从 $\boldsymbol{z}$ 生成 $\boldsymbol{x}$, 必须按 $i = 1, 2, \cdots, D$ 的顺序逐个计算 x_i(因为 x_i 依赖 $\boldsymbol{x}_{1:i-1}$), 无法并行, 见本节末的比较.

条件方面: 结论只用到“h 对第一个变量可逆”这一条, 以及(18.18)的依赖结构; 对 g_i 没有额外要求. 若 h 不是逐元素的, 而是把 z_i 与条件值混合的一般可逆函数, 只要它仍只依赖 $\boldsymbol{x}_{1:i-1}$, 三角性与行列式结论都不变.

原文问题说明. 正文此处印作“这些元素形成一个上三角矩阵”, 这与练习 18.4的说法 (“下三角矩阵”) 以及(18.2)的约定 $J_{ij} = \partial z_i/\partial x_j$ 都不一致: 按该约定,$j > i$ 时元素为零, 得到的是下三角矩阵; 只有在把矩阵转置 (即用 $J_{ij} = \partial z_j/\partial x_i$ 排布) 时它才表现为上三角. 两种写法不影响“行列式等于对角元之积”这一实质结论, 也不影响计算代价, 但读者应以练习 18.4的“下三角”为准.

然而, 从该模型中抽样的任务必须通过计算(18.18)来完成, 这本质上是一个顺序执行的过程, 因为在计算 x_i 之前必须先计算 $x_1, \cdots, x_{i-1}$ 的值, 所以速度较慢. 为了避免这种效率低下的抽样, 我们可以定义**逆自回归流** (Inverse Autoregressive Flow, IAF)(Kingma et al., 2016), 如下所示:

定义 18.6 (逆自回归流)

逆自回归流是形如

$$x_i = h\big(z_i, \tilde{g}_i(\boldsymbol{z}_{1:i-1}, \boldsymbol{w}_i)\big) \tag{18.20}$$

的变换, 其中 $\tilde{g}_i$ 是条件函数,$h(z_i, \cdot)$ 是耦合函数. 它与(18.18)的唯一差别在于: 条件函数依赖的是潜变量的前缀 $\boldsymbol{z}_{1:i-1}$, 而不是数据变量的前缀 $\boldsymbol{x}_{1:i-1}$.

注

(重构新增) 把条件函数的输入从 $\boldsymbol{x}_{1:i-1}$ 换成 $\boldsymbol{z}_{1:i-1}$, 看似只是记号上的小改动, 实际上交换了“便宜的方向”:(18.20)的右端只用到 $\boldsymbol{z}$, 所以给定 $\boldsymbol{z}$ 时所有 x_i 可以一次算出; 而似然方向必须反解(18.20), 反解 z_i 需要 $\tilde{g}_i(\boldsymbol{z}_{1:i-1}, \boldsymbol{w}_i)$, 后者又依赖尚未求出的 $z_1, \cdots, z_{i-1}$, 于是只能顺序进行.

条件方面: 与掩码自回归流相同, 只要求 $h(\cdot, g)$ 可逆;$\tilde{g}_i$ 可以是任意网络. 注意 $\tilde{g}$ 上的波浪号只是为了与(18.18)中的 g 区分, 两者在结构上没有本质不同.

这个定义也解释了为什么两种流要做成“对偶”的样子: 可逆变换的两个方向 (采样、求似然) 中总有一个是顺序的, 除非像耦合流那样把坐标分成两组. 这种“一个方向快、另一个方向慢”的取舍无法同时消除, 只能依应用选择.

如图 18.4(b) 所示, 对于给定的 z, 可以使用(18.20)并行计算元素 $x_1, \cdots, x_{i-1}$, 因此采样效率提高了. 然而, 计算似然所需的逆函数需要一系列如下形式的计算:

命题 18.8 (逆自回归流的逆)

设逆自回归流由(18.20)给出, 且每个 $h(z_i, \cdot)$ 对固定的条件值都是 z_i 的可逆函数. 则似然方向所需的计算为

$$z_i = h^{-1}\left(x_i, \tilde{g}_i(z_{1:i-1}, w_i)\right), \tag{18.21}$$

这些计算本质上是顺序进行的:z_i 的右端依赖 $z_1, \cdots, z_{i-1}$, 必须逐个求解; 而采样方向给定 z 后可以并行.

证明

(重构新增) (18.21)由(18.20)对第一个变量反解得到, 因为 $\tilde{g}_i(z_{1:i-1}, w_i)$ 与 z_i 无关. 顺序性来自右端的依赖: 要算 z_i 必须先知道 $z_1, \cdots, z_{i-1}$, 故 z_1 先算, 之后 z_2, 依此类推, 共 D 步串行, 不能并行.

反过来, 给定 z 后,(18.20)的右端对每个 i 只用到已知的 $z_{1:i-1}$,D 个式子互不依赖, 可以一次并行算出全部 x_i.

□

注

(重构新增) 把这条与命题 18.7并排看, 就得到自回归流的“对偶表”: 掩码自回归流评估似然是 $O(1)$ 次并行前向、采样是 D 步串行; 逆自回归流恰好相反. 两者都需要 D 步串行完成其中一个方向, 因此当 D 很大 (例如高分辨率图像) 时, 串行的一侧会成为瓶颈.

条件方面: 两条命题的前提完全相同 (h 对第一个变量可逆), 差别只在条件函数的输入.

原文问题说明. 正文把可并行计算的元素写作 $x_1, \cdots, x_{i-1}$, 按上下文应为 $x_1, \cdots, x_D$: 给定 z 后,(18.20)右端对每个 i 都只用到已知的 $z_{1:i-1}$, 因此全部 x_i 都可一次并行算出, 而不只是前 $i-1$ 个. 这不影响“逆自回归流采样快、似然慢”的结论.

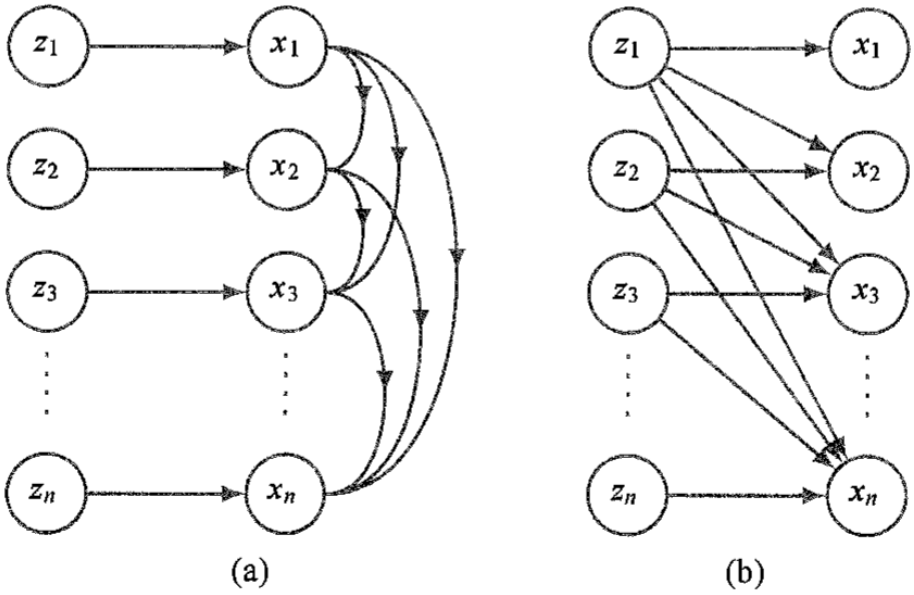


图 18.4: 两种自回归标准化流的结构示意图.(a) 中显示的掩码自回归流允许对似然函数进行有效评估, 而 (b) 中显示的替代逆自回归流允许进行高效抽样

这些计算本质上是顺序进行的, 因此速度很慢. 具体应该使用掩码自回归流还是逆自回归流取决于特定的应用.

耦合流和自回归流是密切相关的. 尽管自回归流引入了相当大的灵活性, 但由于需要顺序地依次采样, 这带来了随数据空间维数 D 线性增长的计算成本. 耦合流可以视为自回归流的一个特例, 它将变量分成两组而不是 D 组, 提升了效率, 但牺牲了一些通用性.

18.3 连续流

本章将要探讨的最后一种标准化流采用了常微分方程 (Ordinary Differential Equation, ODE) 定义的深度神经网络, 它可以看成具有无限多层数的深层网络. 我们首先介绍神经常微分方程 (简称神经 ODE) 的概念, 然后将其应用于标准化流模型的构建.

18.3.1 神经 ODE

神经网络包含的处理层越多, 神经网络就越有用, 我们不妨探究一下, 如果层数无限多会发生什么. 考虑一个残差网络, 其中每一层生成的输出由输入向量加上该输入向量的某个参数化非线性函数给出:

$$z^{(t+1)} = z^{(t)} + \boldsymbol{f}\left(z^{(t)}, \boldsymbol{w}\right), \tag{18.22}$$

其中 $t = 1, \cdots, T$ 标记了网络中的层. 注意, 我们在每一层都使用了相同的函数, 并且共享参数向量 $\boldsymbol{w}$, 因为这允许我们在参数数量有限的情况下考虑任意多的层. 想象一下, 增加层数, 同时确保在每一层引入的变化相应地变得更小. 在极限情况下, 隐藏单元激活向量变成了连续变量 t 的函数 $z(t)$, 我们可以将这个向量通过网络的演化表示为一个微分方程 (见练习 18.5):

定义 18.7 (神经 ODE)

形如

$$\frac{\mathrm{d}z(t)}{\mathrm{d}t} = \boldsymbol{f}\left(z(t), \boldsymbol{w}\right) \tag{18.23}$$

的方程称为**神经 ODE**(Chen et al., 2018), 其中 t 通常称为“时间”, $\boldsymbol{f}$ 是由神经网络给出的向量场, $\boldsymbol{w}$ 是它的参数. 常微分方程的“常”指的是只有单一变量 t.

♣

注

(重构新增) 动机: 把(18.22)的层数 T 推向无穷、同时把每层的变化量缩到无穷小, 离散的层索引就变成了连续的“时间”, 逐层的加法变成了积分. 这样做的收益是“深度”不再是一个整数超参数, 而由数值求解器的步数自适应决定: 求解器在函数变化剧烈处自动加密求值点, 在平缓处放稀, 相当于自动选择了非均匀的层间距.

条件方面:(18.23)要在整个区间 $[0, T]$ 上存在唯一解, 需要对 $\boldsymbol{f}$ 加正则性假设 (例如关于 z 局部 Lipschitz), 这在 $\boldsymbol{f}$ 由常见网络给出、且参数有界时成立; 若不加这类假设, 解可能在某时刻爆破. 此外, $\boldsymbol{f}$ 与 t 无关这一写法是最简形式, 允许显式依赖 t 的 $\boldsymbol{f}(z(t), t, \boldsymbol{w})$ 完全平行地适用 (本节末尾会用到).

与(18.22)的关系正是 Euler 前向积分: 取步长 $\varepsilon = 1$ 就回到残差网络, 取 $\varepsilon \to 0$ 就得到(18.23)(练习 18.5). 因此残差网络可以看作神经 ODE 的一个粗糙离散化, 而神经 ODE 可以看作“层数连续”的残差网络.

如果将网络的输入表示为向量 $z(0)$, 则输出 $z(T)$ 可以通过积分微分方程来获得.

命题 18.9 (神经 ODE 的积分形式)

设 $z(t)$ 在 $[0, T]$ 上连续可微且满足(18.23), $\boldsymbol{f}$ 连续. 则

$$z(T) = z(0) + \int_0^T \boldsymbol{f}\left(z(t), \boldsymbol{w}\right) \mathrm{d}t. \tag{18.24}$$

反之, 若连续可微的 z 满足(18.24)对一切 T 成立, 则它满足(18.23).

♠

证明

(重构新增) 由微积分基本定理,

$$z(T) - z(0) = \int_0^T \frac{\mathrm{d}z(t)}{\mathrm{d}t}\,\mathrm{d}t = \int_0^T \boldsymbol{f}\big(z(t), \boldsymbol{w}\big)\,\mathrm{d}t,$$

其中第二个等号用了(18.23). 这就给出(18.24).

反过来, 若(18.24)对一切 T 成立, 把 T 换成变量 t 并对 t 求导, 由 $\boldsymbol{f}$ 连续及微积分基本定理的逆命题, 左端导数为 $\mathrm{d}z(t)/\mathrm{d}t$, 右端导数为 $\boldsymbol{f}(z(t), \boldsymbol{w})$, 即得(18.23).

□

注

(重构新增) 这个命题说明“解 ODE”与“算一个积分”是同一件事: 给定初值 $z(0)$, 输出 $z(T)$ 就是向量场沿轨迹的累积. 于是任何数值积分器 (Euler、Runge–Kutta、自适应步长方法) 都可以直接用来求 $z(T)$, 而不需要显式地“展开”任何层.

条件方面: $\boldsymbol{f}$ 连续保证积分有意义; 解的存在唯一性仍需 Lipschitz 条件. 命题的逆命题要求(18.24)对一切 T 成立 (而不只是某个固定的 T), 否则只能在几乎处处意义下求导. 这一点在实际中不成问题, 因为求解器总是逐点推进的.

一个实用的推论:(18.24)表明 $z(T)$ 对 $z(0)$ 的依赖是连续的, 且当 $\boldsymbol{f}$ 关于 z Lipschitz 时还是 Lipschitz 的. 这为“神经 ODE 定义了一个良定的可逆映射”提供了基础 (反向只需把时间倒过来积分).

这个积分可以使用标准数值积分软件工具来计算. 求解微分方程最简单的方法是 Euler 前向积分法 (forward integration), 对应于(18.22). 在实际操作中, 更强大的数值积分算法可以通过调整其函数计算来实现目标. 特别是, 它们可以自适应地选择通常为非均匀分布的 t 的值. 这种计算次数取代了传统分层网络中“深度”的概念. 传统分层网络与神经 ODE 的比较如图 18.5所示.

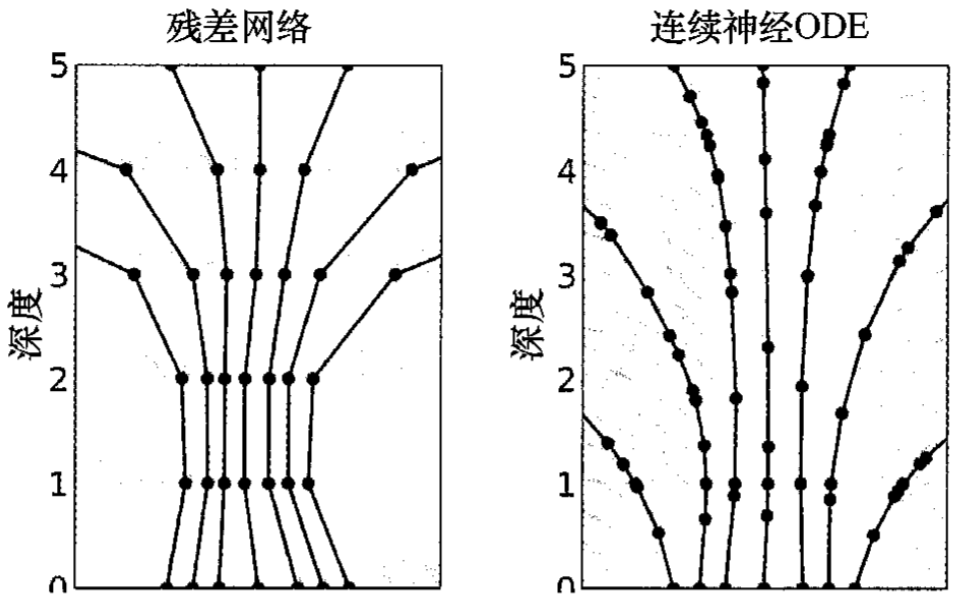


图 18.5: 传统分层网络与神经 ODE 的比较. 左图对应一个有 5 层的残差网络, 并展示了单标量输入的几个起始值的轨迹. 右图显示了连续神经 ODE 的数值积分结果, 同样是对单标量输入的几个起始值, 我们看到函数不是在等间隔的时间点进行求值的, 求值点由数值求解器自适应选择, 并且取决于输入值的选择 [经 Chen et al. (2018) 授权使用]

18.3.2 神经 ODE 的反向传播

我们现在需要解决如何训练神经 ODE 的问题, 即如何通过优化损失函数来确定 $\boldsymbol{w}$ 的值. 给定一个数据集, 其中包括输入向量 $\boldsymbol{z}(0)$ 的值以及相关的输出目标向量, 以及一个依赖于输出向量 $\boldsymbol{z}(T)$ 的损失函数 $L(\cdot)$. 一种方法是使用自动微分来微分 ODE 求解器在前向传播期间执行的所有操作 (参见节 8.2). 尽管这样做很直接, 但从内存角度来看却代价高昂, 并且在控制数值误差方面也不是最佳做法. 另一种方法 (Chen et al., 2018) 将 ODE 求解器视为一个黑箱, 并使用了一种称为**伴随灵敏度方法** (adjoint sensitivity method) 的技术, 这可以视为显式反向传播的连续类比.

回想一下, 反向传播涉及对每个数据点执行三个连续操作步骤 (参见第 8 章): 前向传播, 计算网络中每一层的激活向量; 反向传播, 从输出层开始利用微积分的链式法则计算损失函数对每一层激活的导数, 并向后传播; 参数梯度计算, 通过将前向传播的激活值和反向传播的梯度相乘来计算损失函数对网络参数的导数. 我们将看到, 在计算神经 ODE 的梯度时可以执行类似的步骤.

为了将反向传播应用于神经 ODE, 定义一个称为**伴随** (adjoint) 的量 (见练习 18.6):

定义 18.8 (伴随状态)

设 L 是依赖于 $\boldsymbol{z}(T)$ 的损失函数,$\boldsymbol{z}(t)$ 满足(18.23). 对每个 $t \in [0, T]$, 定义

$$\boldsymbol{a}(t) = \frac{\partial L}{\partial \boldsymbol{z}(t)}, \tag{18.25}$$

并称 $\boldsymbol{a}(t)$ 为**伴随** (adjoint); 特别地,$\boldsymbol{a}(T)$ 对应于损失函数关于输出向量的常规导数.

♣

注

(重构新增) 动机: 在离散网络中, 反向传播逐层保存的“误差信号” $\delta_j = \partial E/\partial a_j$ 正是这里伴随的离散对应物;(18.25)无非是把“损失对某一层激活的导数”沿时间轴连续化. 这个量之所以有用, 是因为一旦知道 $\boldsymbol{a}(t)$, 参数梯度就可以写成它对 $\boldsymbol{f}$ 的加权积分, 而不必逐个时间步地展开计算图.

条件方面:$\boldsymbol{a}(t)$ 要求 L 对 $\boldsymbol{z}(t)$ 可微, 且 $\boldsymbol{z}(T)$ 对 $\boldsymbol{z}(t)$ 的依赖可微——后者由 $\boldsymbol{f}$ 的 Lipschitz/可微性保证. 约定上,(18.25)中的 $\boldsymbol{a}(t)$ 按列向量 (梯度) 理解; 而(18.26)的右端写成 $\boldsymbol{a}(t)^{\mathsf{T}}\nabla_{\boldsymbol{z}}\boldsymbol{f}$ 是按行向量排布的形式——把该式两边转置, 就得到列向量形式 $\mathrm{d}\boldsymbol{a}/\mathrm{d}t = -(\nabla_{\boldsymbol{z}}\boldsymbol{f})^{\mathsf{T}}\boldsymbol{a}$, 两种写法完全等价.

一个容易混淆之处:$\boldsymbol{a}(t)$ 是“总导数”而非“显式偏导”, 它包含了通过后续所有时间步的间接依赖. 这正与离散反传中 δ_j 是总误差对预激活的导数一样.

$\boldsymbol{a}(t)$ 满足如下微分方程.

命题 18.10 (伴随方程)

设 $\boldsymbol{z}(t)$ 满足(18.23),$\boldsymbol{f}$ 连续可微,$\boldsymbol{a}(t)$ 由(18.25)定义. 则 $\boldsymbol{a}(t)$ 满足

$$\frac{\mathrm{d}\boldsymbol{a}(t)}{\mathrm{d}t} = -\boldsymbol{a}(t)^{\mathsf{T}}\,\nabla_{\boldsymbol{z}}\boldsymbol{f}\big(\boldsymbol{z}(t), \boldsymbol{w}\big), \tag{18.26}$$

其中 $(\nabla_{\boldsymbol{z}}\boldsymbol{f})_{ij} = \partial f_i/\partial z_j$ 是 $\boldsymbol{f}$ 关于 $\boldsymbol{z}$ 的 Jacobian 矩阵.

♠

证明

(重构新增) 记 $\boldsymbol{M}(t) = \partial \boldsymbol{z}(T)/\partial \boldsymbol{z}(t)$. 由链式法则,$\boldsymbol{a}(t) = \boldsymbol{M}(t)^{\mathsf{T}}\boldsymbol{a}(T)$($\boldsymbol{a}$ 按列向量理解). 由(18.24),

$$\boldsymbol{z}(T) = \boldsymbol{z}(t) + \int_t^T \boldsymbol{f}\big(\boldsymbol{z}(s), \boldsymbol{w}\big)\,\mathrm{d}s,$$

两边对 $\boldsymbol{z}(t)$ 求导 (注意积分下限也依赖 $\boldsymbol{z}(t)$) 得

$$\boldsymbol{M}(t) = \boldsymbol{I} + \int_t^T \boldsymbol{A}(s)\boldsymbol{M}(s)\,\mathrm{d}s, \qquad \boldsymbol{A}(s) = \nabla_{\boldsymbol{z}}\boldsymbol{f}\big(\boldsymbol{z}(s), \boldsymbol{w}\big).$$

对 t 求导, 由微积分基本定理,

$$\frac{\mathrm{d}\boldsymbol{M}(t)}{\mathrm{d}t} = -\boldsymbol{A}(t)\boldsymbol{M}(t), \qquad \boldsymbol{M}(T) = \boldsymbol{I}.$$

于是

$$\frac{\mathrm{d}\boldsymbol{a}(t)}{\mathrm{d}t} = \frac{\mathrm{d}\boldsymbol{M}(t)^{\mathsf{T}}}{\mathrm{d}t}\boldsymbol{a}(T) = -\boldsymbol{M}(t)^{\mathsf{T}}\boldsymbol{A}(t)^{\mathsf{T}}\boldsymbol{a}(T) = -\boldsymbol{A}(t)^{\mathsf{T}}\boldsymbol{a}(t),$$

两边转置即得(18.26)(左端写成行向量 $\mathrm{d}\boldsymbol{a}(t)^{\mathsf{T}}/\mathrm{d}t$, 右端为 $-\boldsymbol{a}(t)^{\mathsf{T}}\nabla_{\boldsymbol{z}}\boldsymbol{f}$).

□

注

(重构新增) (18.26)就是“微积分链式法则的连续版本”: 离散反传把它写成 $\boldsymbol{a}^{(t)} = \boldsymbol{a}^{(t+\varepsilon)}(\boldsymbol{I} + \varepsilon\nabla_{\boldsymbol{z}}\boldsymbol{f})$, 移项取极限 $\varepsilon \to 0$ 即得(18.26)(练习 18.6). 负号来自“时间倒流”: 随着 t 减小 (向过去推进), 伴随按 $-\boldsymbol{A}$ 演化.

条件方面: 需要 $\boldsymbol{f}$ 连续可微, 才能保证 $\boldsymbol{A}(s)$ 存在且 $\boldsymbol{M}$ 可微; 解的存在唯一性由 $\boldsymbol{A}$ 的有界性保证. 注意(18.26)是一个反向初值问题: 已知 $\boldsymbol{a}(T)$, 向 $t = 0$ 积分. 因此它不是“正向”可解的, 实践中总是从终点往回积.

一个重要的结构性事实:(18.26)右端的矩阵 $\boldsymbol{A}(t)$ 是同一个 $\nabla_{\boldsymbol{z}}\boldsymbol{f}$, 它在计算(18.25)时已经需要 (用于参数梯度), 因此伴随方法与自动微分共享大部分计算.

这是微积分链式法则的连续版本. 这可以通过从 $\boldsymbol{a}(T)$ 开始向后积分来求解, 并且同样可以使用一个黑箱 ODE 求解器来完成. 原则上, 这需要存储我们在前向传播阶段计算的轨迹 $\boldsymbol{z}(T)$, 但这可能会产生问题, 因为反向求解器可能希望在与前向求解器不同的 t 值处计算 $\boldsymbol{z}(T)$. 对此我们只需要允许反向求解器通过从 $\boldsymbol{z}(T)$ 的输出值开始, 同时积分(18.23)和(18.26)来重新计算任何所需的 $\boldsymbol{z}(T)$ 值即可.

反向传播的第三步通过计算激活值和梯度的适当乘积来计算损失函数对网络参数的导数. 当一个网络中的多个连接共享一个参数值时, 总导数由每个连接的导数相加而成 (见练习 9.7). 对于神经 ODE, 其中相同的参数向量 $\boldsymbol{w}$ 在整个网络中共享, 求和变成了对 t 的积分, 形式如下 (见练习 18.7).

命题 18.11 (参数梯度)

设 $\boldsymbol{z}(t)$ 满足(18.23),$L = L(\boldsymbol{z}(T))$,$\boldsymbol{a}(t)$ 由(18.25)定义. 则

$$\nabla_{\boldsymbol{w}} L = -\int_T^0 \boldsymbol{a}(t)^{\mathsf{T}}\nabla_{\boldsymbol{w}}\boldsymbol{f}\big(\boldsymbol{z}(t),\boldsymbol{w}\big)\,\mathrm{d}t = \int_0^T \boldsymbol{a}(t)^{\mathsf{T}}\nabla_{\boldsymbol{w}}\boldsymbol{f}\big(\boldsymbol{z}(t),\boldsymbol{w}\big)\,\mathrm{d}t. \tag{18.27}$$

♠

证明

(重构新增) 记 $\boldsymbol{P}(t) = \partial\boldsymbol{z}(t)/\partial\boldsymbol{w}$. 对(18.24)关于 $\boldsymbol{w}$ 求导 ($\boldsymbol{z}(0)$ 与 $\boldsymbol{w}$ 无关):

$$\frac{\mathrm{d}\boldsymbol{P}(t)}{\mathrm{d}t} = \nabla_{\boldsymbol{w}}\boldsymbol{f}\big(\boldsymbol{z}(t),\boldsymbol{w}\big) + \boldsymbol{A}(t)\boldsymbol{P}(t), \qquad \boldsymbol{P}(0) = \boldsymbol{0},$$

其中 $\boldsymbol{A}(t) = \nabla_{\boldsymbol{z}}\boldsymbol{f}(\boldsymbol{z}(t),\boldsymbol{w})$. 另一方面, 由 $\boldsymbol{a}(t) = \boldsymbol{a}(T)\boldsymbol{M}(t)$ 及 $\mathrm{d}\boldsymbol{M}/\mathrm{d}t = -\boldsymbol{A}\boldsymbol{M}$ 可得

$$\frac{\mathrm{d}}{\mathrm{d}t}\big[\boldsymbol{a}(t)\boldsymbol{P}(t)\big] = \frac{\mathrm{d}\boldsymbol{a}}{\mathrm{d}t}\boldsymbol{P} + \boldsymbol{a}\frac{\mathrm{d}\boldsymbol{P}}{\mathrm{d}t} = -\boldsymbol{a}\boldsymbol{A}\boldsymbol{P} + \boldsymbol{a}\big(\nabla_{\boldsymbol{w}}\boldsymbol{f} + \boldsymbol{A}\boldsymbol{P}\big) = \boldsymbol{a}\nabla_{\boldsymbol{w}}\boldsymbol{f},$$

其中 $-\boldsymbol{a}\boldsymbol{A}\boldsymbol{P}$ 与 $\boldsymbol{a}\boldsymbol{A}\boldsymbol{P}$ 恰好抵消. 在 $[0,T]$ 上积分:

$$\boldsymbol{a}(T)\boldsymbol{P}(T) - \boldsymbol{a}(0)\boldsymbol{P}(0) = \int_0^T \boldsymbol{a}(t)\nabla_{\boldsymbol{w}}\boldsymbol{f}\big(\boldsymbol{z}(t),\boldsymbol{w}\big)\,\mathrm{d}t.$$

因为 $\boldsymbol{P}(0) = \boldsymbol{0}$ 且 $\boldsymbol{a}(T)\boldsymbol{P}(T) = \partial L/\partial\boldsymbol{z}(T)\cdot\partial\boldsymbol{z}(T)/\partial\boldsymbol{w} = \nabla_{\boldsymbol{w}}L$, 即得(18.27). 反向积分形式 $-\int_T^0$ 与正向形式 $\int_0^T$ 只差一个积分限的翻转, 数值上相同.

□

注

(重构新增) (18.27)正是离散网络 $\nabla_{\boldsymbol{w}}L = \sum_t \boldsymbol{a}^{(t+\varepsilon)\mathsf{T}}\partial\boldsymbol{z}^{(t+\varepsilon)}/\partial\boldsymbol{w}$ 的连续极限: 共享参数使“对每个连接求和”变成“对时间积分”, 这也是伴随方法能一次算出全部参数梯度的原因——不需要对每个参数单独做一次反向传播.

条件方面: 需要 $\boldsymbol{f}$ 关于 $\boldsymbol{w}$ 可微、$\boldsymbol{a}$ 与 $\boldsymbol{P}$ 在 $[0,T]$ 上有界 (由 $\boldsymbol{A}$ 有界保证).$\boldsymbol{P}(0) = \boldsymbol{0}$ 用到了初值与 $\boldsymbol{w}$ 无关这一约定, 若 $\boldsymbol{z}(0)$ 也依赖 $\boldsymbol{w}$, 则要补上 $\boldsymbol{a}(0)\partial\boldsymbol{z}(0)/\partial\boldsymbol{w}$ 这一项.

原文问题说明. 原文把该式印为 $\nabla_{\boldsymbol{w}}L = -\int_0^T \boldsymbol{a}(t)^{\mathsf{T}}\nabla_{\boldsymbol{w}}\boldsymbol{f}\,\mathrm{d}t$, 负号与积分限 $0 \to T$ 不配套. 用一维反例即可检验: 取 $f(z,w) = w$(与 z 无关)、$z(0) = 0$、$T = 1$, 则 $z(1) = w$,$\boldsymbol{a}(t) \equiv L'(w)$,$\nabla_w f \equiv 1$, 正确梯度为 $L'(w)$, 而

$-\int_0^1 L'(w)\,\mathrm{d}t = -L'(w)$, 符号相反. 可见负号必须与反向积分限 $\int_T^0$ 搭配 (这正是 Chen et al. (2018) 的原始写法), 或者去掉负号而保留 $\int_0^T$;(18.27)把两种等价写法并列写出, 以免误用.

(18.26)中的 $\nabla_z \boldsymbol{f}$ 和(18.27)中的 $\nabla_w \boldsymbol{f}$ 可以使用自动微分高效地求值 (参见节 8.2). 注意, 上述结果同样适用于更普遍的神经网络函数 $\boldsymbol{f}(\boldsymbol{z}(t), t, \boldsymbol{w})$, 它除了通过 $\boldsymbol{z}(t)$ 的隐式依赖之外 (原文此处印作 $\boldsymbol{z}(T)$, 按上下文应为 $\boldsymbol{z}(t)$), 还显式地依赖于 t.

与传统分层网络相比, 使用伴随方法训练的神经 ODE 的一个好处是, 不需要存储前向传播的中间结果, 因此内存成本是恒定的.

命题 18.12 (伴随方法的存储与计算代价)

设用伴随方法训练(18.23). 则梯度计算所需存储与离散层数 (即求解器的步数) 无关: 只需保存端点 $\boldsymbol{z}(T)$ 与 $\boldsymbol{a}(T)$, 即可按(18.26)与(18.27)反向求出梯度. 代价是反向过程中需要重新积分(18.23)以重建所需的 $\boldsymbol{z}(t)$, 函数求值次数约为前向的常数倍; 若损失依赖轨迹上的多个时刻, 则需要对每一段分别求解, 代价按段数线性增长.

♠

证明

(重构新增) 先看存储. 离散网络中, 反向传播要逐层使用前向激活 $\boldsymbol{z}^{(t)}$, 故必须把它们全部保存, 存储量为 $O(T/\varepsilon)$ 个 D 维向量; 而在连续情形,(18.26)与(18.27)只用到 $\boldsymbol{a}(t)$、$\boldsymbol{z}(t)$ 与 $\nabla_w \boldsymbol{f}$, 其中 $\boldsymbol{z}(t)$ 可以由 $\boldsymbol{z}(T)$ 出发反向积分(18.23)重建 ((18.24)在 $[t, T]$ 上的形式), 因此除端点外无需保存任何中间状态, 存储量为 $O(1)$ 个 D 维向量.

再看计算. 反向一次积分同时求解 $\boldsymbol{z}$ 与 $\boldsymbol{a}$(把两者拼成增广状态), 再积分一次累积(18.27), 总函数求值次数与前向同阶乘以一个小的常数, 与层数无关. 若 L 依赖 $\boldsymbol{z}(t_1), \cdots, \boldsymbol{z}(t_k)$ 等多个时刻, 则需在每段 $[t_j, t_{j+1}]$ 上分别做一次反向求解, 代价乘以 k.

□

注

(重构新增) 这条代价结论是神经 ODE 的主要卖点: 传统深层网络的内存随深度线性增长, 而伴随方法把它压成常数, 使得“极深”(等价于很多求解步) 的模型在内存上可行. 代价是时间: 反向时要重新积分, 相当于用计算换内存; 在最坏情形下总计算量约为前向的两倍到数倍.

条件方面: 结论只依赖“$\boldsymbol{z}(t)$ 可由端点重建”这一条, 即(18.23)的解对初值连续依赖且可反向积分; 若向量场不满足 Lipschitz 条件, 反向重建可能不稳定, 这时需要额外技巧 (如检查点法, 把轨迹分段保存若干端点以限制误差累积).

此外, 神经 ODE 可以自然地处理连续时间数据, 其中观测可在任意时间发生. 如果损失函数 L 依赖于 $\boldsymbol{z}(T)$ 的其他值而不仅仅是输出值, 则需要进行多次逆模型求解, 对每一对连续的输出进行一次逆模型求解, 以便单个解决方案可以分解为多个连续解决方案以访问中间状态 (Chen et al., 2018). 请注意, 在训练阶段可以使用高精度的求解器; 而在推理阶段, 特别是在计算资源有限的应用场景中, 可以采用较低的精度, 从而减少所需的函数评估次数.

18.3.3 神经 ODE 流

我们可以利用神经 ODE 来定义一个旨在构建易处理的标准化流模型的替代方法. 神经 ODE 定义了一个从输入向量 $\boldsymbol{z}(0)$ 到输出向量 $\boldsymbol{z}(T)$ 的高度灵活的转换, 其形式为如下微分方程:

$$\frac{\mathrm{d}\boldsymbol{z}(t)}{\mathrm{d}t} = \boldsymbol{f}\big(\boldsymbol{z}(t), \boldsymbol{w}\big). \tag{18.28}$$

如果我们定义一个基于输入向量的基础分布 $p(\boldsymbol{z}(0))$, 神经 ODE 就会随时间向前传播, 从而得到每个 t 值的分布 $p(\boldsymbol{z}(t))$, 进而得到输出向量的分布 $p(\boldsymbol{z}(T))$. 有研究 [Chen et al. (2018)] 指出, 对于神经 ODE 来说, 密度转换可以通过积分如下微分方程来计算 (见练习 18.8):

定理 18.1 (连续流的瞬时变量替换)

设 $\boldsymbol{f}(\cdot,\boldsymbol{w})$ 在 $\mathbb{R}^D$ 上连续可微且导数有界,$\boldsymbol{z}(t)$ 是(18.28)以 $\boldsymbol{z}(0)$ 为初值的解,$p(\boldsymbol{z}(t),t)$ 表示 $\boldsymbol{z}(t)$ 的密度. 若 $\boldsymbol{z}(0)$ 的密度为 p_0, 则对 $\boldsymbol{z}(0)$ 的分布几乎处处的初值,

$$\frac{\mathrm{d}\ln p\big(\boldsymbol{z}(t)\big)}{\mathrm{d}t} = -\operatorname{tr}\left(\frac{\partial \boldsymbol{f}\big(\boldsymbol{z}(t),\boldsymbol{w}\big)}{\partial \boldsymbol{z}(t)}\right), \tag{18.29}$$

其中 $\partial \boldsymbol{f}/\partial \boldsymbol{z}(t)$ 表示 Jacobian 矩阵, 其元素为 $\partial f_i/\partial z_j$. 等价地,

$$\ln p\big(\boldsymbol{z}(T)\big) = \ln p\big(\boldsymbol{z}(0)\big) - \int_0^T \operatorname{tr}\left(\frac{\partial \boldsymbol{f}\big(\boldsymbol{z}(t),\boldsymbol{w}\big)}{\partial \boldsymbol{z}(t)}\right)\mathrm{d}t.$$

♡

证明

(重构新增) 记 $\boldsymbol{\varphi}_t$ 为(18.28)的流映射 ($\boldsymbol{z}(0)\mapsto \boldsymbol{z}(t)$), 并记

$$\boldsymbol{T}_t = \frac{\partial \boldsymbol{\varphi}_t}{\partial \boldsymbol{z}(0)}, \qquad \boldsymbol{A}(t) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{z}}\big(\boldsymbol{z}(t),\boldsymbol{w}\big).$$

由 $\boldsymbol{z}(t)$ 对初值可微 ($\boldsymbol{f}$ 的导数有界保证) 及链式法则,$\boldsymbol{T}_t$ 满足线性矩阵微分方程

$$\frac{\mathrm{d}\boldsymbol{T}_t}{\mathrm{d}t} = \boldsymbol{A}(t)\boldsymbol{T}_t, \qquad \boldsymbol{T}_0 = \boldsymbol{I}.$$

第一步, 计算 $\det \boldsymbol{T}_t$.$\boldsymbol{T}_t$ 作为流映射的 Jacobian 恒可逆, 故可用 Jacobi 公式 $(\det \boldsymbol{M})' = \det(\boldsymbol{M})\operatorname{tr}(\boldsymbol{M}^{-1}\boldsymbol{M}')$:

$$\frac{\mathrm{d}}{\mathrm{d}t}\det \boldsymbol{T}_t = \det(\boldsymbol{T}_t)\operatorname{tr}\left(\boldsymbol{T}_t^{-1}\boldsymbol{A}(t)\boldsymbol{T}_t\right) = \det(\boldsymbol{T}_t)\operatorname{tr}\boldsymbol{A}(t),$$

其中第二个等号用了迹的循环性. 解这个一阶方程 (初值 $\det \boldsymbol{T}_0 = 1$) 得

$$\det \boldsymbol{T}_t = \exp\left(\int_0^t \operatorname{tr}\boldsymbol{A}(s)\,\mathrm{d}s\right).$$

第二步, 用命题 18.1把 p_0 与 $p(\cdot,t)$ 联系起来.$\boldsymbol{\varphi}_t$ 是微分同胚, 其逆的 Jacobian 是 $\boldsymbol{T}_t^{-1}$, 故

$$p\big(\boldsymbol{z}(t),t\big) = \frac{p_0\big(\boldsymbol{z}(0)\big)}{|\det \boldsymbol{T}_t|}.$$

取对数并代入第一步的结果 ($\det \boldsymbol{T}_t > 0$):

$$\ln p\big(\boldsymbol{z}(t),t\big) = \ln p_0\big(\boldsymbol{z}(0)\big) - \int_0^t \operatorname{tr}\boldsymbol{A}(s)\,\mathrm{d}s.$$

两端对 t 求导 (左端沿轨迹求全导数,$\ln p_0(\boldsymbol{z}(0))$ 与 t 无关), 即得(18.29). 把(18.29)从 0 积分到 T 即得定理中的累积形式.

□

注

(重构新增) 这条定理是连续流的核心, 可以看成一维连续性方程 $\partial_t q + \partial_z(qf) = 0$ 的积分形式, 也可以看成离散流(18.1)的“无穷小版本”: 在极短时间 δt 内, 流映射近似为 $\boldsymbol{z} + \delta t\,\boldsymbol{f}(\boldsymbol{z})$, 其 Jacobian 是 $\boldsymbol{I} + \delta t\,\boldsymbol{A}$, 由 $\det(\boldsymbol{I}+\delta t\boldsymbol{A}) = 1 + \delta t\operatorname{tr}\boldsymbol{A} + O(\delta t^2)$ 得

$$\ln p\big(\boldsymbol{z}(t+\delta t)\big) - \ln p\big(\boldsymbol{z}(t)\big) \approx -\delta t\operatorname{tr}\boldsymbol{A}(t),$$

除以 δt 并取极限就是(18.29).练习 18.8在一维情形用“体积 (长度) 守恒”给出了这个推导.

条件方面:$\boldsymbol{f}$ 连续可微且导数有界, 保证流映射是微分同胚、$\boldsymbol{T}_t$ 恒可逆, 从而 Jacobi 公式可用; 若只假设 $\boldsymbol{f}$ Lipschitz, 解仍存在唯一, 但 $\boldsymbol{T}_t$ 的存在需要 $\boldsymbol{f}$ 可微. 定理的结论是“几乎处处”的, 因为命题 18.1中的密度只在几乎处处意义下确定. 定理中的累积形式还需要 $\operatorname{tr}\boldsymbol{A}(t)$ 在 $[0,T]$ 上可积, 这在导数有界时自动成立.

符号的含义:$\operatorname{tr}\boldsymbol{A} > 0$ 表示流在局部膨胀体积, 密度随之下降 (负号);$\operatorname{tr}\boldsymbol{A} < 0$ 表示压缩, 密度上升. 这与图 18.6中“流线分开处密度降低、靠拢处密度增加”的图像一致.

这个积分操作可以使用标准 ODE 求解器来完成. 同样, 可以通过从基础分布 $p(\boldsymbol{z}(0))$ 中采样来获得这个密度的样本, 这个基础分布可以是一个简单的分布, 如 Gauss 分布, 并支持使用 ODE 求解器再次整合 [(18.28)] , 以便将值传播到输出.

定义 18.9 (连续标准化流)

设 $p(z(0))$ 是输入向量上的基础分布,$z(t)$ 满足(18.28),$p(z(t))$ 按(18.29)演化. 称由(18.28)定义的、把 $p(z(0))$ 变换为 $p(z(T))$ 的标准化流为**连续标准化流** (continuous normalizing flow), 并把定理中的累积形式称为它的密度计算公式. ♣

注

(重构新增) 连续标准化流把定义 18.2中的“有限个可逆层”换成了“一个连续时间的过程”: 层数不再是整数, 而是由求解器自适应选择的求值点; 每层的对数行列式贡献被换成 $\operatorname{tr} \boldsymbol{A}(t)$ 在时间上的积分. 因此它是离散标准化流的“无穷深”极限, 也是命题 18.2中“各层对数行列式之和”的连续版本——求和变成了积分.

条件方面: 基础分布要简单、可采样、对数密度可算 (例如标准 Gauss);$\boldsymbol{f}$ 要满足定理 18.1的正则性条件, 使流映射是微分同胚, 从而(18.28)确实定义了一个可逆变换. 与离散流不同的是, 这里不需要显式构造逆映射: 反向只需把时间从 T 积分回 0, 同一个求解器即可完成.

一个实用上的差别: 离散流的层数固定, 计算图固定, 反传直接可用; 连续流的“层”由求解器决定, 其反向需要伴随方法 (见小节 18.3.2), 这正是下一段要说明的.

连续标准化流可以使用适用于神经 ODE 的伴随灵敏度方法进行训练, 该方法可以看作反向传播的连续时间等价物 (参见小节 18.3.2).

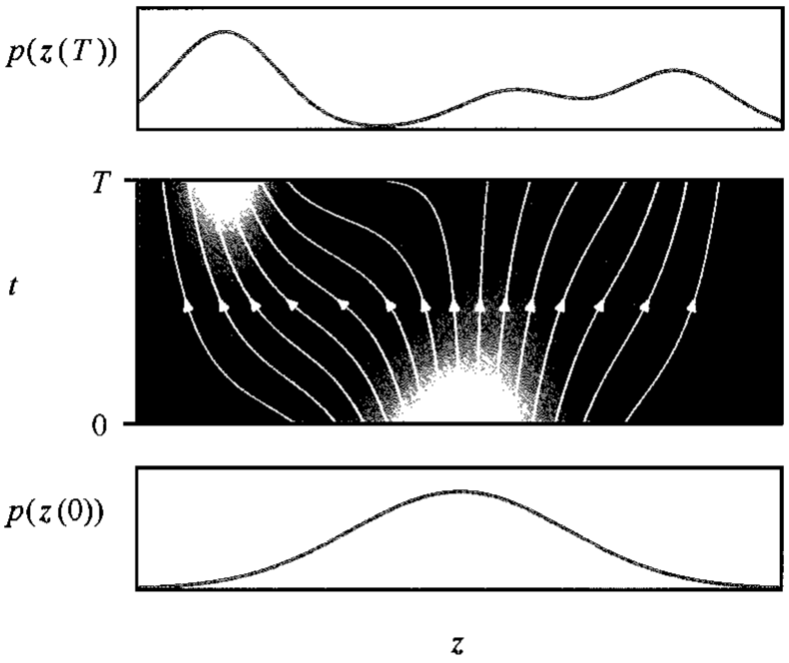


图 18.6: 连续标准化流的示意图, 这里展示了 $t = 0$ 时刻的一个简单 Gauss 分布是如何通过连续转换变成 $t = T$ 时刻的一个多峰分布的. 当流线分开时, 密度降低; 当流线靠拢时, 密度增加

由于(18.29)涉及 Jacobian 矩阵的**迹**而不是行列式 (行列式出现在离散标准化流中), 它可能看起来更具计算效率.

命题 18.13 (密度计算的代价)

设 D 为数据维数,$\boldsymbol{A}$ 为 $D \times D$ 矩阵. 则:(i) 计算 $\det \boldsymbol{A}$ 一般需要 $O(D^3)$ 次运算, 而计算 $\operatorname{tr}(\boldsymbol{A})$ 只需 $O(D)$ 次运算;(ii) 若 $\boldsymbol{A}$ 是三角矩阵, 则 $\det \boldsymbol{A}$ 是对角元之积, 也只需 $O(D)$ 次运算;(iii) 形成 $\boldsymbol{A} = \partial \boldsymbol{f}/\partial \boldsymbol{z}$ 需要 D 次 Jacobian-向量积, 每次的代价与前向求值 $\boldsymbol{f}$ 同阶 (记为 $O(D)$), 故精确计算 $\operatorname{tr}(\boldsymbol{A})$ 或三角矩阵的行列式共需 $O(D^2)$ 的成本;(iv) 若改用(18.31)的随机估计, 只需 M 次矩阵-向量积, 代价为 $O(MD)$, 取 $M = 1$ 时为 $O(D)$. ♠

证明

(重构新增) (i) 一般矩阵的行列式用 LU 分解计算, 分解本身是 $O(D^3)$, 再乘对角元是 $O(D)$; 迹只是把 D 个对角元相加, 是 $O(D)$.

(ii) 三角矩阵的 LU 分解不需要消元 (行列式即对角元之积), 故为 $O(D)$.

(iii)$\boldsymbol{A}$ 的第 j 列是 $\partial \boldsymbol{f}/\partial z_j$, 可用一次 Jacobian-向量积 (或反向的向量-Jacobian 积) 得到, 其代价与一次前向求值 $\boldsymbol{f}$ 同阶; 对 D 个方向各做一次即为 $O(D^2)$. 得到全部对角元后求和是 $O(D)$, 被 $O(D^2)$ 吸收.

(iv)(18.31)只需对每个样本计算 $\boldsymbol{\varepsilon}_m^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}_m$, 即一次向量-Jacobian 积再与 $\boldsymbol{\varepsilon}_m$ 内积, 代价 $O(D)$;M 个样本共 $O(MD)$.

□

注

(重构新增) 这条结论解释了连续流相对于离散流的计算优势与代价. 优势是: 离散流要对每层算一次行列式, 即便每层是三角的也要 $O(D)$, 层数一多就累积; 连续流把“每层的行列式”换成“迹的时间积分”, 而迹的精确计算只需 $O(D^2)$(主要开销是形成 Jacobian), 与层数无关. 代价是: 迹仍然需要 D 次 Jacobian-向量积, 当 D 很大时 $O(D^2)$ 仍不可忽视, 于是有了下面的随机估计.

条件方面:(i) 中的 $O(D^3)$ 是一般稠密矩阵的代价, 若 $\boldsymbol{A}$ 有特殊结构 (三角、低秩、对角加低秩) 则可大幅降低;(iii) 中“每次 Jacobian-向量积与前向同阶”对逐层网络成立, 对一般黑箱向量场则以实际求值代价为准. 原文把“计算 Jacobian 的各个元素需要单独的前向传播”计为 $O(D)$ 次运算, 与这里的记账一致:D 次求值, 每次 $O(D)$.

然而, 通过使用 Hutchinson 的迹估计器 (Grathwohl et al., 2018), 我们可以将计算迹的成本降低到 $O(D)$. 对于矩阵 $\boldsymbol{A}$, 迹的形式为

$$\operatorname{tr}(\boldsymbol{A}) = \mathbb{E}_{\boldsymbol{\varepsilon}}\left[\boldsymbol{\varepsilon}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}\right], \tag{18.30}$$

其中 $\boldsymbol{\varepsilon}$ 是一个零均值、单位协方差的随机向量, 例如 Gauss 分布 $\mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. 对于一个特定的 $\boldsymbol{\varepsilon}$, 矩阵-向量乘积 $\boldsymbol{A}\boldsymbol{\varepsilon}$ 可以使用反向的自动微分在单次传递中高效计算. 然后我们可以使用有限数量的样本来近似矩阵的迹, 形式如下:

$$\operatorname{tr}(\boldsymbol{A}) \approx \frac{1}{M}\sum_{m=1}^{M}\boldsymbol{\varepsilon}_m^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}_m. \tag{18.31}$$

命题 18.14 (迹估计量的无偏性)

设 $\boldsymbol{\varepsilon}$ 是零均值、协方差为 $\boldsymbol{I}$ 的 D 维随机向量,$\boldsymbol{A}$ 为 $D\times D$ 实矩阵. 则

$$\mathbb{E}\left[\boldsymbol{\varepsilon}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}\right] = \operatorname{tr}(\boldsymbol{A}).$$

因此对任意 $M \geqslant 1$ 与独立同分布的 $\boldsymbol{\varepsilon}_1, \cdots, \boldsymbol{\varepsilon}_M$,(18.31)右端的期望恰为 $\operatorname{tr}(\boldsymbol{A})$, 即该估计量是无偏的. ♠

证明

(重构新增) 由二次型的展开,

$$\mathbb{E}\left[\boldsymbol{\varepsilon}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}\right] = \mathbb{E}\left[\sum_{i,j}A_{ij}\varepsilon_i\varepsilon_j\right] = \sum_{i,j}A_{ij}\,\mathbb{E}\left[\varepsilon_i\varepsilon_j\right].$$

$\boldsymbol{\varepsilon}$ 零均值、协方差为 $\boldsymbol{I}$, 即 $\mathbb{E}[\varepsilon_i]=0$、$\mathbb{E}[\varepsilon_i\varepsilon_j]=\delta_{ij}$, 故上式等于 $\sum_i A_{ii} = \operatorname{tr}(\boldsymbol{A})$.

对 M 个独立样本, 由期望的线性性,

$$\mathbb{E}\left[\frac{1}{M}\sum_{m=1}^{M}\boldsymbol{\varepsilon}_m^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}_m\right] = \frac{1}{M}\sum_{m=1}^{M}\mathbb{E}\left[\boldsymbol{\varepsilon}_m^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}_m\right] = \operatorname{tr}(\boldsymbol{A}),$$

与 M 无关, 故估计量无偏.

□

注

(重构新增) 无偏性只用到前两阶矩, 与 $\boldsymbol{\varepsilon}$ 的具体分布无关; 取 Gauss 只是为了让采样方便、并便于分析方差. 这里不需要 $\boldsymbol{A}$ 对称, 也不需要它正定——二次型的期望只依赖对角元, 非对角元在 $\mathbb{E}[\varepsilon_i\varepsilon_j]=\delta_{ij}$ 处被消掉.

关于“单位协方差”. 原文举例写作 $\mathcal{N}(0,1)$, 而在 D 维情形应理解为每个分量独立的标准 Gauss 分布, 即 $\boldsymbol{\varepsilon}\sim\mathcal{N}(\boldsymbol{0},\boldsymbol{I})$; 若各分量方差不等于 1, 则 $\mathbb{E}[\boldsymbol{\varepsilon}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon}]=\operatorname{tr}(\boldsymbol{A}\boldsymbol{\Sigma})$, 不再是 $\operatorname{tr}(\boldsymbol{A})$, 这会引入系统偏差.

方差方面:$\operatorname{Var}(\boldsymbol{\varepsilon}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{\varepsilon})$ 与 $\|\boldsymbol{A}\|_F^2$ 同阶, 因此 M 个样本把标准差降到 $1/\sqrt{M}$ 量级.$M=1$ 时估计有噪声, 但它是无偏的, 且噪声会与随机梯度下降的其它噪声一起被平均掉, 这正是下一段所说的“不太重要”.

实践中, 我们可以设置 $M=1$ 并只使用一个样本对每个新的数据点进行刷新. 虽然这是一个有噪声的估计, 但由于它是噪声随机梯度下降过程的一部分, 这可能显得不太重要. 重要的是, 它是无偏的, 这意味着估计器的期望等于真实值 (参见练习 18.11).

通过使用一种名为流匹配 (flow matching) 的技术 (参见第 20 章), 可以显著提高连续标准化流的训练效率 (Lipman et al., 2022). 这使得标准化流更接近于扩散模型, 避免了在积分器中进行反向传播, 同时大幅减少内存需求, 实现了更快的推理和更稳定的训练.

18.4 习题

✍ **练习 18.1** (★★) 考虑变换 $\boldsymbol{x}=\boldsymbol{f}(\boldsymbol{z})$ 及其逆变换 $\boldsymbol{z}=\boldsymbol{g}(\boldsymbol{x})$. 通过对 $\boldsymbol{x}=\boldsymbol{f}(\boldsymbol{g}(\boldsymbol{x}))$ 进行求导, 证明

$$\boldsymbol{J}\boldsymbol{K}=\boldsymbol{I}, \tag{18.32}$$

其中 $\boldsymbol{I}$ 是单位矩阵,$\boldsymbol{J}$ 和 $\boldsymbol{K}$ 则是元素分别为如下矩阵

$$J_{ij}=\frac{\partial g_i}{\partial x_j}\quad\text{和}\quad K_{ij}=\frac{\partial f_i}{\partial z_j} \tag{18.33}$$

的矩阵. 利用矩阵乘积的行列式是矩阵各自行列式的乘积这一结论, 证明

$$\det(\boldsymbol{J})=\frac{1}{\det(\boldsymbol{K})}. \tag{18.34}$$

然后证明密度在变量变换下的转换公式 [(18.1)] 可以重写为

$$p_x(\boldsymbol{x})=p_z\big(\boldsymbol{g}(\boldsymbol{x})\big)\,|\det\boldsymbol{K}|^{-1}, \tag{18.35}$$

其中 $\boldsymbol{K}$ 是在 $\boldsymbol{z}=\boldsymbol{g}(\boldsymbol{x})$ 处进行计算的.

✍ **练习 18.2** (★) 考虑一个由形式如下的可逆变换序列构成的变换:

$$\boldsymbol{x}=\boldsymbol{f}_1\big(\boldsymbol{f}_2(\cdots\boldsymbol{f}_{M-1}(\boldsymbol{f}_M(\boldsymbol{z}))\cdots)\big). \tag{18.36}$$

证明其逆函数由下式给出:

$$\boldsymbol{z}=\boldsymbol{f}_M^{-1}\big(\boldsymbol{f}_{M-1}^{-1}(\cdots\boldsymbol{f}_2^{-1}(\boldsymbol{f}_1^{-1}(\boldsymbol{x}))\cdots)\big). \tag{18.37}$$

✍ **练习 18.3** (★) 考虑形式如下的线性变量变换:

$$\boldsymbol{x}=\boldsymbol{z}+\boldsymbol{b}. \tag{18.38}$$

证明此变换的 Jacobian 矩阵是单位矩阵. 通过对 $\boldsymbol{z}$ 空间中一个小区域的体积与 $\boldsymbol{x}$ 空间中对应区域的体积进行比较, 解释这一结果.

✍ **练习 18.4** (★★) 证明(18.19)给出的自回归标准化流变换的 Jacobian 矩阵是一个下三角矩阵. 矩阵的行列式由主对角线上项的乘积给出, 因此容易计算.

✍ **练习 18.5** (★) 考虑残差网络的前向传播方程 [(18.22)] , 并考虑时间变量 t 的一个小的增量:

$$\boldsymbol{z}^{(t+\varepsilon)}=\boldsymbol{z}^{(t)}+\varepsilon\boldsymbol{f}\big(\boldsymbol{z}^{(t)},\boldsymbol{w}\big), \tag{18.39}$$

来自神经网络的加性贡献被 ε 缩放. 注意(18.22)对应于 $\varepsilon=1$ 的情况. 通过取极限 $\varepsilon\to0$, 推导出(18.23)所示的前向传播微分方程.

✍ **练习 18.6** (★★) 在这个习题和下一个习题中, 我们将为神经 ODE 的反向传播方程和梯度计算方程提供非正式的推

导. 对于这些结果更正式的推导可以在 Chen et al. (2018) 中找到. 写出与前向传播方程 [(18.39)] 相对应的反向传播方程. 通过取极限 $\varepsilon \to 0$, 推导出反向传播方程 [(18.26)] , 其中 $\boldsymbol{a}(t)$ 由(18.25)定义.

✍ **练习 18.7** (★★) 利用(8.12), 写出损失函数 $L(\boldsymbol{z}(T))$ 的梯度表达式, 该损失函数针对的是(18.39)定义的多层残差网络, 其中所有层共享相同的参数向量 $\boldsymbol{w}$. 通过取极限 $\varepsilon \to 0$, 推导出计算损失函数导数的方程 [(18.27)] .

✍ **练习 18.8** (★★) 在这个习题中, 我们将为(18.29)的一维版本提供非正式的推导. 考虑时间 t 的分布 $q(z)$, 从 z 到 x 的变换在时间 $t + \delta t$ 变成了新的分布 $p(x)$. 同时考虑附近值 z 和 $z + \Delta z$ 以及相应的值 x 和 $x + \Delta x$, 如图 18.7所示. 首先写出一个方程来表达区间 Δz 中的概率质量与区间 Δx 中的概率质量相同. 然后写出一个方程来显示概率密度是如何从 t 变化到 $t + \delta t$ 的, 用导数 $\mathrm{d}q(t)/\mathrm{d}t$ 表示. 接下来通过引入函数 $f(z) = \mathrm{d}z/\mathrm{d}t$ 写下一个关于 Δz 的 Δx 的方程. 最后通过结合以上三个方程并取极限 $\delta t \to 0$, 证明

$$\frac{\mathrm{d}}{\mathrm{d}t} \ln q(z) = -f'(z), \tag{18.40}$$

这就是(18.29)的一维版本.

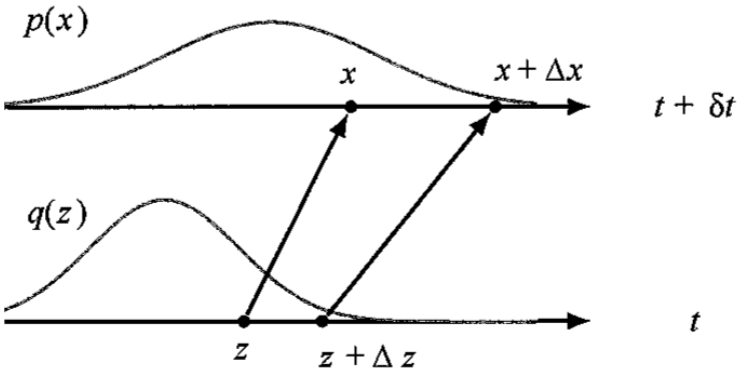


图 18.7: 概率密度变换的示意图, 用于推导一维连续标准化流的方程

✍ **练习 18.9** (★★)图 18.6中的流线是通过取一组等间距的值, 并利用每个 t 值处累积分布函数的逆函数, 通过绘制对应的 $\boldsymbol{z}$ 空间中的点而得到的. 证明这等同于使用微分方程 [(18.28)] 计算流线, 其中 $\boldsymbol{f}$ 由(18.29)定义.

✍ **练习 18.10** (★★) 使用微分方程 [(18.28)] 写下连续标准化流的基础密度表达式, 该表达式以输出密度形式表示, 并在 t 上进行积分. 利用改变定积分符号等同于交换积分限的事实, 证明反转连续标准化流的计算成本与评估正向流的计算成本相同.

✍ **练习 18.11** (★) 证明 Hutchinson 的迹估计器 [(18.31)] 右侧的期望等于 $\mathrm{Tr}(\boldsymbol{A})$, 这对任何的 M 值都成立, 从而证明该估计器是无偏的.

第 19 章 自编码器

深度学习的一个核心目标是发现数据的表示方法, 为后续应用提供价值.

定义 19.1 (自编码器)

自联想神经网络 (auto-associative neural network) 或**自编码器** (autoencoder) 是获得广泛认可的学习数据内部表示的方法, 其中神经网络的输入单元和输出单元的数量相同, 训练目的是让输出 $\boldsymbol{y}$ 尽可能接近输入 $\boldsymbol{x}$. 一旦训练完成, 神经网络中的一个内部层就为每个新的输入提供一个表示 $\boldsymbol{z}(\boldsymbol{x})$. 这样的网络可以划分成两部分: 首先是**编码器** (encoder), 它将输入 $\boldsymbol{x}$ 映射到一个隐式表示 $\boldsymbol{z}(\boldsymbol{x})$; 其次是**解码器** (decoder), 它将隐式表示映射到输出 $\boldsymbol{y}(\boldsymbol{z})$.

♣

注

(重构新增) 定义 19.1的目标函数是“重建误差”, 因此它属于无监督学习: 训练时不需要标签, 输入本身就充当目标. 定义中“输入单元与输出单元数量相同”这一条不是装饰——若两者维数不同,$\boldsymbol{y}$ 与 $\boldsymbol{x}$ 根本无法逐分量比较, 平方和误差也就无从定义. 与此相对, 中间的隐式表示 $\boldsymbol{z}(\boldsymbol{x})$ 的维数可以与输入不同, 这正是自编码器能压缩或扩维的原因.

定义本身没有对 $\boldsymbol{z}$ 加任何限制, 这一点很关键: 若不加约束, 取 $M \geqslant D$ 并令编码器、解码器都等于恒等映射, 就有 $\boldsymbol{y} = \boldsymbol{x}$、误差为零, 网络什么也没学到. 原书随后列举的三种约束——限制 $\boldsymbol{z}$ 的维数、要求 $\boldsymbol{z}$ 稀疏、改变训练方式使网络必须纠正被损坏的输入——分别对应小节 19.1.1与小节 19.1.2、小节 19.1.3和小节 19.1.4至小节 19.1.5. 这些约束的共同作用是排除恒等映射, 从而迫使网络把数据中重复出现的结构 (相关性、低维流形) 编码进 $\boldsymbol{z}$.

一个网络把自己的输入复制到输出, 这就是一个很一般的自编码器, 不解决任何实际问题. 如果自编码器要找到非平凡解, 就需要引入某种形式的约束. 例如, 这种约束可以要求 $\boldsymbol{z}$ 的维度小于 $\boldsymbol{x}$ 的维度, 或者要求 $\boldsymbol{z}$ 具有稀疏表示. 我们也可以改变训练的方式, 迫使网络去探索非平凡解. 例如, 训练网络学会纠正输入向量的错误, 比如清除加入的噪声或者填补遗漏的信息. 这样的限制性条件能激励神经网络挖掘数据内部有价值的结构, 从而获得更好的训练效果.

在本章中, 我们将从确定性的自编码器开始学习, 然后推广到学习编码器分布 $p(\boldsymbol{z} \mid \boldsymbol{x})$ 以及解码器分布 $p(\boldsymbol{y} \mid \boldsymbol{z})$ 的随机模型. 这些概率模型称为**变分自编码器** (variational autoencoder), 它们代表了我们训练非线性潜变量模型的 4 种方法中的第 3 种 (参见小节 16.4.4).

19.1 确定性的自编码器

在研究主成分分析 (Principal Component Analysis,PCA) 时, 我们遇到了一种形式简单的自编码器. 这种模型通过对输入向量进行线性变换, 将其映射到一个低维的流形上, 然后通过再次线性变换, 将其近似地重构回原始数据空间中 (参见节 16.1). 我们可以利用神经网络的非线性特点来定义一种非线性 PCA, 其中的潜在流形不再是数据空间的线性子空间. 这是通过使用输出数量与输入数量相同的网络, 并针对每一组训练数据, 通过优化权重以最小化输入与输出之间的某种重建误差来实现的.

简单的自编码器在现代深度学习中很少直接使用, 因为它们在潜空间中既不能提供语义上有意义的表示, 也不能直接从数据分布中生成新的样本. 然而, 它们为一些更强大的深度生成式模型 (如变分自编码器) 提供了重要的概念基础 (参见节 19.2).

19.1.1 线性自编码器

首先考虑图 19.1所示形式的多层感知机, 它有 D 个输入单元、D 个输出单元和 M 个隐藏单元, 其中 $M < D$. 用来训练网络的目标向量就是输入向量本身, 因此该网络试图将每个输入向量映射到其自身. 由于隐藏单元的数

量少于输入单元的数量, 它通常无法完美地重构所有输入向量; 因此, 我们先定义一个误差函数, 用来衡量输入向量及其重建之间的不匹配程度, 然后通过最小化这个误差函数来确定网络参数 $\boldsymbol{w}$.

定义 19.2 (线性自编码器)

若定义 19.1所定义网络的隐藏单元采用线性激活函数, 则称它为**线性自编码器** (linear autoencoder). 它的训练通过最小化如下形式的平方和误差函数来实现:

$$E(\boldsymbol{w}) = \frac{1}{2}\sum_{n=1}^{N} \left\|\boldsymbol{y}(\boldsymbol{x}_n, \boldsymbol{w}) - \boldsymbol{x}_n\right\|^2, \tag{19.1}$$

其中 $\boldsymbol{y}(\boldsymbol{x}_n, \boldsymbol{w})$ 是网络对输入 $\boldsymbol{x}_n$ 的重建.

♣

注

(重构新增) 定义 19.2中的式(19.1)取 $\frac{1}{2}$ 的因子并不是数学上的需要, 而是为了让求导后消去平方项的系数 2, 使梯度形式更干净; 把它换成任意正常数只会整体缩放误差曲面, 不改变极小点. 这里把“线性”加在隐藏单元上而不是加在输出单元上, 是因为输出单元本来就是线性的 (否则 $\boldsymbol{y}(\boldsymbol{x}_n, \boldsymbol{w})$ 无法与 $\boldsymbol{x}_n$ 直接比较): 真正决定模型容量的是隐藏层. 条件 $M < D$ 是这一节的核心约束, 它使 $\boldsymbol{BA}$ 的秩不超过 M, 从而排除了 $\boldsymbol{BA} = \boldsymbol{I}$ 这种把输入原样复制的平凡解; 若去掉它, 取 $\boldsymbol{A} = \boldsymbol{I}$、$\boldsymbol{B} = \boldsymbol{I}$ 就有 $E(\boldsymbol{w}) = 0$, 训练失去意义. 注意误差中并没有显式地减去数据均值, 这一点在定理 19.1之后的注记中还会讨论.

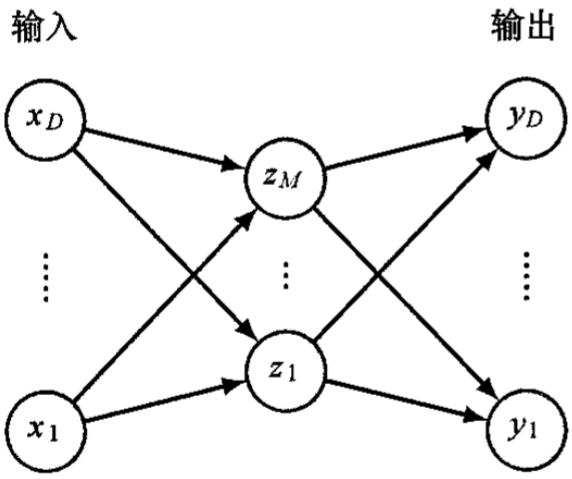


图 19.1: 一个具有两层权重的多层感知机. 这样的网络是通过最小化平方和误差来训练的, 使得输入向量被映射到它们自身. 即使隐藏层中有非线性单元, 这样的网络也相当于线性主成分分析. 为了清晰起见, 表示偏置参数的连接已省略

定理 19.1 (线性自编码器的最优解与主成分分析)

设训练数据 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N \in \mathbb{R}^D$, 记 $\boldsymbol{S} = \sum_{n=1}^{N} \boldsymbol{x}_n \boldsymbol{x}_n^{\mathsf{T}}$. 设 $\boldsymbol{S}$ 的特征值为 $\lambda_1 \geqslant \lambda_2 \geqslant \cdots \geqslant \lambda_D \geqslant 0$, 对应的单位正交特征向量为 $\boldsymbol{u}_1, \cdots, \boldsymbol{u}_D$. 设线性自编码器的隐藏层宽为 $M < D$, 隐藏层与输出层都是线性的, 即存在 $\boldsymbol{A} \in \mathbb{R}^{M\times D}$ 与 $\boldsymbol{B} \in \mathbb{R}^{D\times M}$, 使得

$$\boldsymbol{z}(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}, \qquad \boldsymbol{y}(\boldsymbol{x}) = \boldsymbol{B}\boldsymbol{A}\boldsymbol{x}.$$

则:

(i) 误差函数(19.1)的全局最小值唯一, 且等于 $\frac{1}{2}\sum_{i=M+1}^{D} \lambda_i$;

(ii) 当 $\lambda_M > \lambda_{M+1}$ 且 $\boldsymbol{S}$ 正定时, 该最小值当且仅当 $\boldsymbol{BA}$ 是到 $\operatorname{span}\{\boldsymbol{u}_1, \cdots, \boldsymbol{u}_M\}$ 的正交投影时取到;

(iii) 此时网络执行一个投影操作, 把数据映射到由数据的前 M 个主成分所张成的 M 维子空间上, 通向隐

藏单元的权重向量 (即 $\boldsymbol{A}$ 的各行) 构成该主子空间的一组基; 这些向量不必是正交或归一化的. ♡

证明

(重构新增) 记 $\boldsymbol{P}=\boldsymbol{B}\boldsymbol{A}$, 则 $\operatorname{rank}\boldsymbol{P}\leqslant M$, 且

$$E(\boldsymbol{w})=\frac{1}{2}\sum_{n=1}^{N}\left\|\boldsymbol{x}_n-\boldsymbol{P}\boldsymbol{x}_n\right\|^2.$$

分四步证明.

(1) 任意 $\boldsymbol{P}$ 都可替换为其值域上的正交投影. 设 $\boldsymbol{Q}$ 是到 $\operatorname{range}(\boldsymbol{P})$ 的正交投影, 则 $\boldsymbol{Q}\boldsymbol{P}=\boldsymbol{P}$, 从而 $(\boldsymbol{I}-\boldsymbol{Q})\boldsymbol{x}_n$ 与 $\operatorname{range}(\boldsymbol{P})$ 正交, 而 $(\boldsymbol{Q}-\boldsymbol{P})\boldsymbol{x}_n=\boldsymbol{Q}\boldsymbol{x}_n-\boldsymbol{P}\boldsymbol{x}_n$ 落在 $\operatorname{range}(\boldsymbol{P})$ 中. 于是二者正交, 由勾股定理,

$$\left\|\boldsymbol{x}_n-\boldsymbol{P}\boldsymbol{x}_n\right\|^2=\left\|\boldsymbol{x}_n-\boldsymbol{Q}\boldsymbol{x}_n\right\|^2+\left\|\boldsymbol{Q}\boldsymbol{x}_n-\boldsymbol{P}\boldsymbol{x}_n\right\|^2\geqslant\left\|\boldsymbol{x}_n-\boldsymbol{Q}\boldsymbol{x}_n\right\|^2.$$

因此只需在秩不超过 M 的正交投影中极小化 E, 而且 (1) 中的等号成立当且仅当 $\boldsymbol{Q}\boldsymbol{x}_n=\boldsymbol{P}\boldsymbol{x}_n$ 对每个 n 成立.

(2) 对秩为 $r\leqslant M$ 的正交投影 $\boldsymbol{Q}$, 设 $\boldsymbol{V}=[\boldsymbol{v}_1,\cdots,\boldsymbol{v}_r]$ 是 $\operatorname{range}(\boldsymbol{Q})$ 的标准正交基, 则 $\boldsymbol{Q}=\boldsymbol{V}\boldsymbol{V}^{\mathsf{T}}$. 由 $\boldsymbol{Q}$ 的对称性与幂等性,

$$\sum_{n=1}^{N}\left\|\boldsymbol{x}_n-\boldsymbol{Q}\boldsymbol{x}_n\right\|^2=\sum_{n=1}^{N}\|\boldsymbol{x}_n\|^2-\sum_{n=1}^{N}\|\boldsymbol{Q}\boldsymbol{x}_n\|^2=\operatorname{tr}\boldsymbol{S}-\operatorname{tr}\left(\boldsymbol{V}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{V}\right).$$

由谱分解 (见附录 A),$\boldsymbol{S}=\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}}$, 其中 $\boldsymbol{U}=[\boldsymbol{u}_1,\cdots,\boldsymbol{u}_D]$ 正交、$\boldsymbol{\Lambda}=\operatorname{diag}(\lambda_1,\cdots,\lambda_D)$. 记 $\boldsymbol{C}=\boldsymbol{U}^{\mathsf{T}}\boldsymbol{V}$, 则 $\boldsymbol{C}^{\mathsf{T}}\boldsymbol{C}=\boldsymbol{I}_r$, 故 $c_i\triangleq\|\boldsymbol{C}_{i\cdot}\|^2$ 满足 $0\leqslant c_i\leqslant 1$ 且 $\sum_{i=1}^{D}c_i=\|\boldsymbol{C}\|_F^2=r$, 并且

$$\operatorname{tr}\left(\boldsymbol{V}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{V}\right)=\operatorname{tr}\left(\boldsymbol{C}^{\mathsf{T}}\boldsymbol{\Lambda}\boldsymbol{C}\right)=\sum_{i=1}^{D}\lambda_i c_i\leqslant\sum_{i=1}^{r}\lambda_i,$$

最后一个不等号是因为把总量为 r 的权重 $c_i\in[0,1]$ 分配给 λ_i 时, 最大值出现在把权重全部分配给最大的 r 个 λ_i 处; 当 $\lambda_r>\lambda_{r+1}$ 时等号成立当且仅当 $c_1=\cdots=c_r=1$、$c_{r+1}=\cdots=c_D=0$, 即 $\operatorname{range}(\boldsymbol{Q})=\operatorname{span}\{\boldsymbol{u}_1,\cdots,\boldsymbol{u}_r\}$. 于是

$$E\geqslant\frac{1}{2}\Big(\operatorname{tr}\boldsymbol{S}-\sum_{i=1}^{r}\lambda_i\Big)=\frac{1}{2}\sum_{i=r+1}^{D}\lambda_i\geqslant\frac{1}{2}\sum_{i=M+1}^{D}\lambda_i.$$

(3) 取 $\boldsymbol{A}=[\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M]^{\mathsf{T}}$、$\boldsymbol{B}=[\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M]$, 则 $\boldsymbol{B}\boldsymbol{A}=\sum_{i=1}^{M}\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}}$ 正是到主子空间 $\operatorname{span}\{\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M\}$ 的正交投影, 此时 $E=\frac{1}{2}\sum_{i=M+1}^{D}\lambda_i$, 下界可以取到, 故最小值等于 $\frac{1}{2}\sum_{i=M+1}^{D}\lambda_i$, 它由 $\lambda_{M+1},\cdots,\lambda_D$ 唯一确定.

(4) 等号情形的刻画. 由 (1)(2), 当 $\boldsymbol{S}$ 正定且 $\lambda_M>\lambda_{M+1}$ 时,E 取到最小值必须有 $r=M$、$\operatorname{range}(\boldsymbol{Q})=\operatorname{span}\{\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M\}$ 且 $\boldsymbol{P}\boldsymbol{x}_n=\boldsymbol{Q}\boldsymbol{x}_n$ 对一切 n 成立;$\boldsymbol{S}$ 正定意味着 $\{\boldsymbol{x}_n\}$ 张成 $\mathbb{R}^D$, 故 $\boldsymbol{P}=\boldsymbol{Q}$. 此时 $\operatorname{range}(\boldsymbol{A}^{\mathsf{T}})=\operatorname{range}(\boldsymbol{P})=\operatorname{span}\{\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M\}$, 所以 $\boldsymbol{A}$ 的各行线性无关并构成主子空间的一组基; 而 $\boldsymbol{A}$ 只受“行空间等于主子空间”这一条件约束, 各行一般既不正交也未归一化. 最后, 使 E 取到最小值的参数 $(\boldsymbol{A},\boldsymbol{B})$ 并不唯一: 对任意可逆的 $\boldsymbol{R}\in\mathbb{R}^{M\times M}$, 把 $\boldsymbol{A}$ 换成 $\boldsymbol{R}\boldsymbol{A}$、$\boldsymbol{B}$ 换成 $\boldsymbol{B}\boldsymbol{R}^{-1}$ 时 $\boldsymbol{B}\boldsymbol{A}$ 不变, 故误差不变.

□

注

(重构新增) 定理 19.1解释了为什么“线性自编码器不提供超出 PCA 的东西”: 两者的目标函数是同一个平方和误差, 而 PCA 的主成分正是该误差在秩约束下的最优解, 所以训练一个线性自编码器等价于用迭代法计算主成分, 结论与 Bourlard and Kamp(1988)、Baldi and Hornik(1989) 一致.

逐项说明条件的作用与边界:

(1) $M<D$ 与“秩不超过 M”是结论的全部内容所在. 若 $M\geqslant D$, 结论中的下界 $\frac{1}{2}\sum_{i=M+1}^{D}\lambda_i$ 成为空和 0, 而 $\boldsymbol{B}\boldsymbol{A}=\boldsymbol{I}$ 确实使 $E=0$, 此时自编码器退化为恒等映射, 与“复制输入不解决任何实际问题”的说法相符.

(2) “隐藏层与输出层都是线性的”是证明中把网络写成 $\boldsymbol{B}\boldsymbol{A}\boldsymbol{x}$ 的前提. 若隐藏单元取非线性激活函数, 上述证明

不再适用; 但 Bourlard and Kamp(1988) 证明了即使隐藏单元非线性, 最小误差解仍会被投影到主成分子空间——这是引用结果, 本书未给证明, 故本定理不覆盖该情形.

(3) "平方和误差"使 E 成为 $\boldsymbol{P}$ 的二次函数, 才能用谱分解把它化为 $\operatorname{tr}\boldsymbol{S} - \operatorname{tr}(\boldsymbol{V}^{\mathsf{T}}\boldsymbol{S}\boldsymbol{V})$. 若换成交叉熵等其它损失, 最优子空间一般不再是主子空间; 原书在小节 19.1.2中正是通过加入非线性层而不是更换损失来突破线性限制.

(4) 关于数据中心化:(19.1)没有减去均值, 证明中的 $\boldsymbol{S}$ 是二阶矩矩阵.图 19.1为清晰起见省略了偏置连接, 而偏置的作用恰好等价于先减去均值再重构, 因此含偏置的网络对应的是协方差矩阵的谱分解. 两种写法只在"是否把均值方向计入子空间"上有差别, 不影响 $M < D$ 时的秩约束论证.

(5) 关于唯一性: 最小值唯一 (它只依赖 $\lambda_{M+1}, \cdots, \lambda_D$), 但取到最小值的参数不唯一 (见证明 (4)); 当 $\lambda_M = \lambda_{M+1}$ 时主子空间本身也不唯一, 此时结论 (3) 中的"该主子空间"应理解为任一由前 M 个特征向量张成的子空间.

(6) 关于 $\boldsymbol{S}$ 不满秩: 若 $N < D$ 或数据退化,$\boldsymbol{S}$ 只有 $r_0 < D$ 个正特征值. 此时下界仍为 $\frac{1}{2}\sum_{i=M+1}^{D}\lambda_i$(其中 $\lambda_i = 0$ 的项不贡献), 但等号条件只在 $\operatorname{span}\{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$ 上成立, 这正是定理 19.1中附加"$\boldsymbol{S}$ 正定"的原因.

这个结果并不在意料之外, 因为 PCA 和神经网络都依赖于线性降维以及最小化相同的平方和误差函数.

有人可能会觉得, 对图 19.1所示网络的隐藏单元使用非线性激活函数, 可以克服线性流形的局限性. 然而, 即使有非线性隐藏单元, 最小误差解也仍然会被投影到主成分子空间 (Bourlard and Kamp,1988). 因此, 使用两层神经网络进行降维没有任何优势. 基于奇异值分解 (SVD) 的标准 PCA 技术可以保证在有限时间内给出正确解, 并且生成一组有序的特征值及对应的正交归一化特征向量.

19.1.2 深度自编码器

然而, 如果在网络中加入额外的非线性层, 情况就有所不同了.

定义 19.3 (深度自编码器)

称图 19.2所示形式的四层自联想网络为**深度自编码器** (deep autoencoder): 它的输出单元仍然是线性的, 第二层中的 M 个单元也可以是线性的, 而第一层和第三层具有非线性激活函数; 它仍然通过最小化损失函数(19.1)来训练. ♣

注

(重构新增) 定义 19.3中"两层非线性夹一层线性"的结构不是随意的: 若第一层也是线性的, 则整个网络仍是 $\boldsymbol{B}\boldsymbol{A}\boldsymbol{x}$ 的形式,定理 19.1立即适用, 非线性层就白加了; 若第二层也非线性, 则中间的 M 个单元只是又一层非线性特征, "M 维子空间的坐标"这一几何解释便不再适用, 但网络作为非线性降维器仍然成立. 因此"第一层、第三层非线性、第二层线性"恰好保证了 F_1 可以是任意非线性映射而 F_2 在其参数化下仍把 M 维潜空间嵌入 $\mathbb{R}^D$.

条件的代价在于优化:(19.1)不再是 $\boldsymbol{w}$ 的二次函数, 全局最小值一般无法用闭式解或凸优化得到, 只能依赖局部搜索, 因而存在收敛到次优局部极小值的风险; 此外潜空间维数 M 必须在训练之前人为指定, 不能像概率 PCA 那样由数据自动确定 (参见节 16.2).

我们可以将这个网络视为两个连续的函数映射 (下文简称映射)F_1 和 F_2, 如图 19.2所示. 第一个映射 F_1 将原始的 D 维数据投影到了由第二层单元的激活定义的 M 维子空间. 因为第一层非线性单元的存在, 这个映射非常通用, 不受限于线性映射. 类似地, 该网络的后半部分定义了从 M 维隐藏空间回到原始 D 维输入空间的任意映射. 这在几何上有一个简单的解释, 参见图 19.3, 其中 $D = 3$ 且 $M = 2$.

这样的网络实际上有效地执行了一种非线性形式的 PCA. 它具有不受限于线性变换的优势, 尽管标准 PCA 是它的一个特例. 然而, 现在训练网络需要进行非线性优化, 因为误差函数(19.1)不再是网络参数的二次函数. 因此必须使用计算密集型的非线性优化技术, 并且存在找到误差函数的次优局部极小值的风险. 此外, 在训练网络之前必须指定子空间的维数.

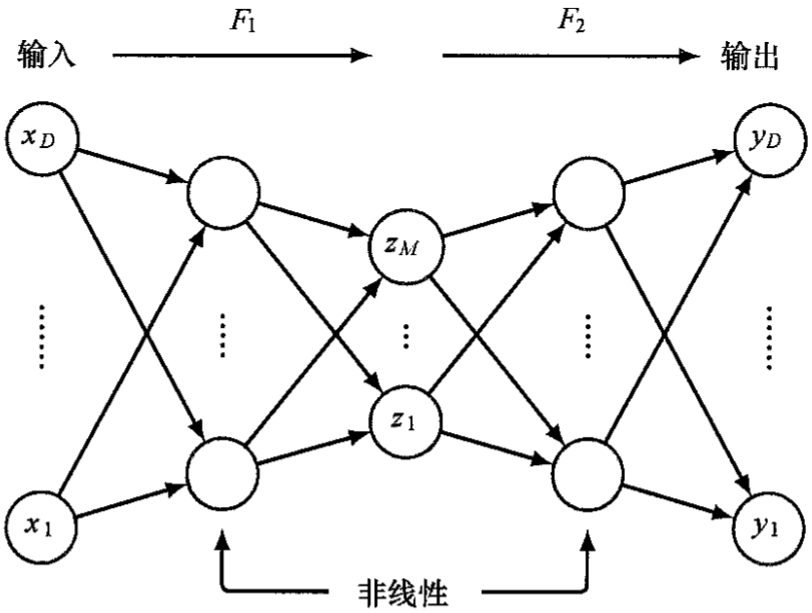


图 19.2: 添加额外的非线性单元隐藏层会产生一个自联想网络, 它可以执行非线性降维

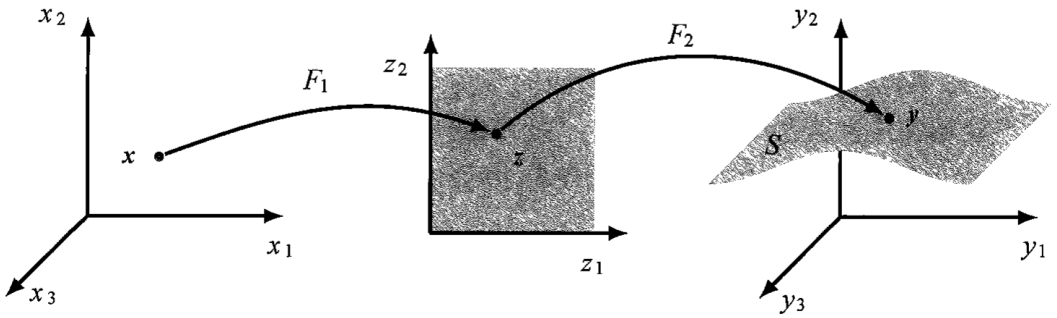


图 19.3: 图 19.2所示网络执行的映射的几何解释 (具有 $D = 3$ 个输入且第二层中有 $M = 2$ 个单元的模型) 从潜空间到数据空间的映射 F_2 定义了流形 S 在更高维数据空间中的嵌入方式. 由于映射 F_2 可以是非线性的, 因此流形 S 的嵌入可以是非平面的. 映射 F_1 则定义了从原始 D 维数据空间到 M 维潜空间的投影

19.1.3 稀疏自编码器

与其限制网络中某个隐藏层的节点数量, 不如考虑另一种约束内部表示的方法: 使用正则化器鼓励稀疏表示, 从而降低有效维度. 一个简单的选择是 L_1 正则化器, 它能促进数据的稀疏性, 相应的正则化误差函数如下 (参见小节 9.2.2):

定义 19.4 (稀疏自编码器)

在自编码器的误差函数中加入对隐藏单元激活值的 L_1 惩罚, 得到**正则化误差函数**

$$\tilde{E}(\boldsymbol{w}) = E(\boldsymbol{w}) + \lambda \sum_{k=1}^{K} |z_k|, \tag{19.2}$$

其中 z_k 是隐藏层中第 k 个单元的激活值,K 是隐藏层单元数,$\lambda > 0$ 是正则化系数. 用(19.2)训练的自编码器称为**稀疏自编码器** (sparse autoencoder).

♣

注

(重构新增) 定义 19.4最容易被忽略的一点是: 正则化施加在单元激活值 z_k 上, 而不是像通常的权重衰减那样施加在网络参数 $\boldsymbol{w}$ 上 (参见小节 9.2.2). 两者作用的对象不同, 效果也不同: 对 $\boldsymbol{w}$ 的 L_1 惩罚压缩的是权重, 而对 z_k 的 L_1 惩罚直接推动隐藏层在给定输入下只有少数单元被激活, 从而“降低有效维度”而不必真的删掉单元. 这也是它被称为“另一种约束内部表示的方法”的原因——它不是通过减少单元数 M 来限制容量, 而是在 M 不变的前提下让表示变稀疏.

λ 控制稀疏性与重建精度之间的取舍:λ 太小则(19.2)退化为原来的平方和误差, 稀疏性无从产生;λ 太大则网络倾向于令所有 z_k 都接近于零, 重建误差随之上升. 注意 $|z_k|$ 在 $z_k = 0$ 处不可微, 实践中要么使用自动微分框架的次梯度约定, 要么把 $|z_k|$ 换成光滑近似 (如 $\sqrt{z_k^2 + \varepsilon}$); 原书只说明“导数可以像往常一样使用自动微分来计算”, 并未讨论这一细节. 最后,(19.2)对 z_k 的依赖使得误差不再只是 $\boldsymbol{w}$ 的二次函数, 即使在“线性”自编码器中也不能再用定理 19.1的谱分解方法求闭式解.

其中 $E(\boldsymbol{w})$ 是未正则化的误差函数, 求和操作是在隐藏层中所有单元的激活值上进行的. 注意, 正则化通常应用于网络的参数, 而在这里它被用于单元激活值. 所需的用于梯度下降训练的导数可以像往常一样使用自动微分来计算.

19.1.4 去噪自编码器

我们已经看到了在简单的自编码器中限制潜空间层的维度的重要性, 这样能避免模型直接学习恒等映射. 去噪自编码器 (denoising autoencoder)(Vincent,2008) 也能迫使模型发现数据内部有趣的结构.

定义 19.5 (去噪自编码器)

去噪自编码器 (denoising autoencoder) 的训练方式如下: 用噪声破坏每个输入向量 $\boldsymbol{x}_n$, 得到一个修改后的向量 $\tilde{\boldsymbol{x}}_n$, 然后把它输入自编码器, 产生输出 $\boldsymbol{y}(\tilde{\boldsymbol{x}}_n, \boldsymbol{w})$; 网络通过最小化平方和损失函数

$$E(\boldsymbol{w}) = \sum_{n=1}^{N} \left\| \boldsymbol{y}(\tilde{\boldsymbol{x}}_n, \boldsymbol{w}) - \boldsymbol{x}_n \right\|^2 \tag{19.3}$$

来训练, 其目的是重建原始的无噪声输入向量 $\boldsymbol{x}_n$.

♣

注

(重构新增) 定义 19.5的损失(19.3)与(19.1)相比有两处实质差别: 输入换成了被损坏的 $\tilde{\boldsymbol{x}}_n$, 而目标仍是干净的 $\boldsymbol{x}_n$. 正是这一“输入—目标不对称”阻止了恒等映射: 网络若原样复制输入, 只能得到 $\tilde{\boldsymbol{x}}_n$, 与目标 $\boldsymbol{x}_n$ 的差恰是噪声, 因此要把损失降下来, 网络必须学会去掉损坏. 这也解释了为什么它不需要像线性自编码器那样靠 $M < D$ 来排除平凡解——即使 $M \geqslant D$, 恒等映射也不再是最优的. 原书这里没有写 $\frac{1}{2}$ 的因子 (与(19.1)不同), 这不影响极小点, 只影响梯度尺度.

定义中“噪声”的具体形式由使用者选择, 原书随后给出两种: 随机置零 (掩蔽噪声, 比例 v) 与加性 Gauss 噪声. 前者对应后文的掩蔽自编码器, 后者使(19.3)的最优解与得分匹配联系起来 (Vincent,2011). 需要注意,(19.3)的期望是对噪声分布取的, 而定义中写的是单个样本上的求和; 只有在噪声实现固定时它才是一个确定的数, 实际的训练目标应理解为对该式的期望.

添加噪声的一种方法是随机地把一些输入变量设置为零. 此类输入所占的比例 v 代表噪声水平, 并且范围是 $0 \leqslant v \leqslant 1$. 添加噪声的另一种方法是为每个输入变量添加独立的零均值 Gauss 噪声, 其中噪声的规模由 Gauss 分布的方差设定. 通过学习去除输入数据中的噪声, 网络被迫学习数据方方面面的结构. 例如, 如果数据包含图像, 网络只需要学习到相邻像素值之间的强相关性, 就可以纠正被噪声损坏的像素.

更正式地讲, 去噪自编码器的训练与得分匹配 (score matching)(Vincent,2011) 相关, 得分由 $s(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$ 定义.

定义 19.6 (得分)

设 $p(\boldsymbol{x})$ 是数据空间上的概率密度, 且 $\ln p$ 关于 $\boldsymbol{x}$ 可微. 称向量场

$$s(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$$

为 p 的**得分** (score).

♣

注

(重构新增) 定义 19.6所说的得分, 是“对数密度增长最快的方向”, 它指向数据密度更高的区域, 因而与去噪有直观联系: 去噪自编码器学习的是把损坏点拉回流形的向量场, 而得分场也指向高密度区域, 两者在流形附近方向一致. 这一联系的确切形式 (Vincent,2011) 是: 当噪声为方差很小的加性 Gauss 噪声时, 最小化(19.3)的最优去噪函数 $\boldsymbol{y}^*$ 满足 $\boldsymbol{y}^*(\tilde{\boldsymbol{x}}) - \tilde{\boldsymbol{x}} = \sigma^2 \nabla_{\tilde{\boldsymbol{x}}} \ln p(\tilde{\boldsymbol{x}})$, 即去噪方向恰好是得分乘以噪声方差, 因此训练去噪自编码器等价于 (在适当意义下) 估计得分. 原书只给出直观解释而未陈述这一定理, 故本节不把它写成命题;节 20.3讨论扩散模型时会回到这一关系.

条件方面:$\ln p$ 必须可微, 这要求 p 在其支撑内部为正; 若数据分布在低维流形上,$\mathbb{R}^D$ 中的密度并不存在 (只能相对于流形上的测度定义), 此时得分的定义需要按流形上的密度理解, 这也是原书图 19.4只画“指向流形”的箭头而不写公式的原因.

图 19.4说明了我们对这种关系的一些直观的理解. 自编码器通过学习逆转失真向量 $\tilde{\boldsymbol{x}}_n - \boldsymbol{x}_n$, 学习了数据空间中每个点的一个向量, 该向量指向流形, 从而指向高数据密度区域. 得分向量 $\nabla \ln p(\boldsymbol{x})$ 同样是一个指向高数据密度区域的向量. 等到讨论扩散模型时, 我们将更深入地探索得分匹配和去噪之间的关系. 扩散模型也会学习从那些被噪声破坏的输入中去除噪声 (参见节 20.3).

19.1.5 掩蔽自编码器

我们已经看到, 借助随机遮盖输入子集的自监督学习方式, 像 BERT 这样的 Transformer 模型能学习到自然语言的深层次内部表示 (参见节 12.2). 人们很自然会问, 类似的方法能应用于自然图像吗?

定义 19.7 (掩蔽自编码器)

掩蔽自编码器 (masked autoencoder)(He,2021) 利用深度神经网络, 根据输入图像的损坏版本重建图像; 这里的损坏 (corruption) 形式是掩蔽或删除输入图像的一部分.

♣

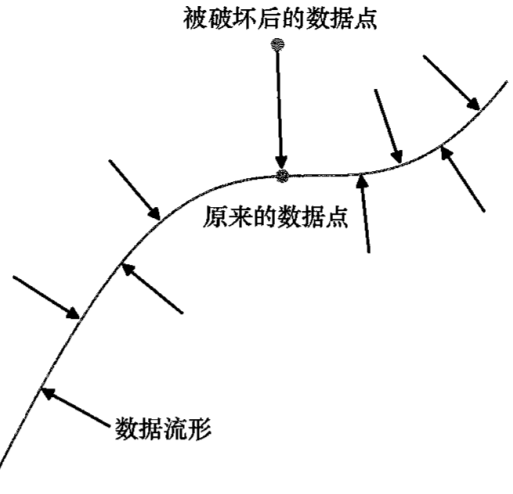


图 19.4: 在去噪自编码器中, 数据点被假定存在于数据空间中的一个低维流形上, 并且被加性噪声破坏了. 自编码器会学习将损坏的数据点映射回它们原来的值, 因此自编码器为数据空间中的每个点学习了一个指向流形的向量

注

(重构新增) 定义 19.7是定义 19.5的一个特例: 损坏算子不再是加性噪声或随机置零, 而是“删除一个补丁块”. 这个特例之所以重要, 是因为它把“去噪”与 Transformer 的 token 序列表示对接起来——被保留的补丁成为编码器的输入 token, 被删除的补丁不再需要计算, 于是编码器的计算量随掩蔽比例线性下降. 这也解释了原书随后强调的“掩蔽的区块简单地被省略”与 BERT 用固定掩蔽符号替换的区别.

条件方面: 损坏算子必须与后续任务无关 (否则网络可以学会“猜出”掩蔽模式而不是图像结构); 掩蔽比例要足够高才能迫使网络学习全局结构, 但过高会使可用信息不足; 此外解码器必须能处理被恢复的完整 token 序列, 这要求位置编码在掩蔽后仍能对齐原图网格.

这种技术通常可以与视觉 Transformer 架构结合起来使用 (参见小节 12.4.1), 因为在这种情况下, 仅仅通过传递随机选定的输入补丁标记 (input patch token) 子集给编码器, 就可以轻松实现对输入图像的部分进行掩蔽.图 19.5总结了整个算法.

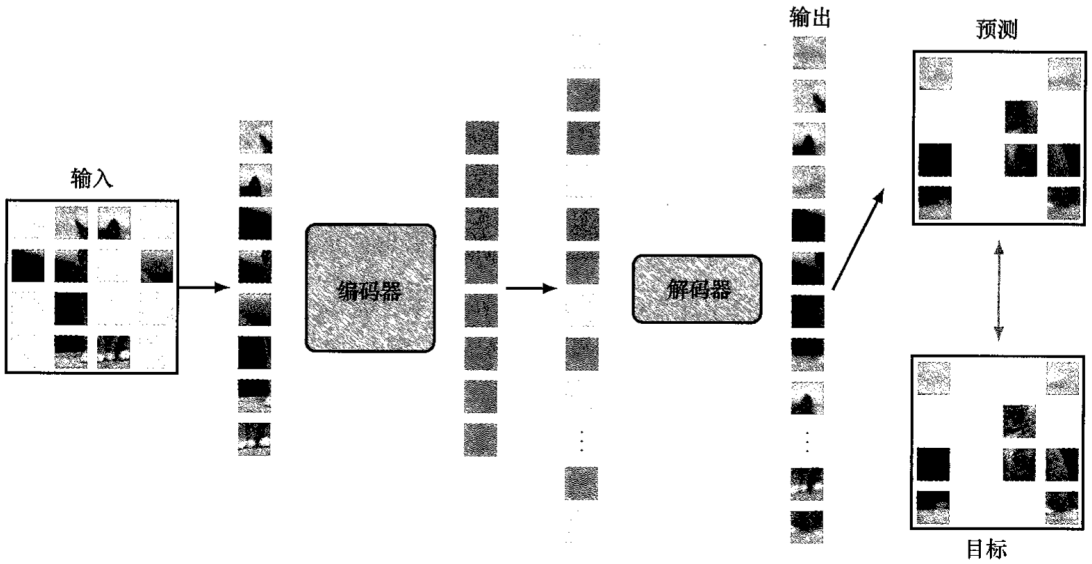


图 19.5: 掩蔽自编码器在训练阶段的架构. 注意, 目标是输入的补集, 因为损失仅应用于被掩蔽的区块. 训练结束后, 解码器被丢弃, 编码器用于将图像映射到内部表示, 以便在后续任务中使用

和语言不同, 图像具有更多的冗余, 并伴有强烈的局部相关性. 在一句话中省略一个单词可能会大大增加歧义, 然而随机移除图像中的一小块通常对图像的语义影响很小. 当输入图像有较高掩蔽比例 (通常为 75%) 时, 算法能学到最佳的内部表示, 而 BERT 中的掩蔽比例仅为 15%. 在 BERT 中, 掩蔽的输入被固定的掩蔽符号替换; 而在掩蔽自编码器中, 掩蔽的区块简单地被省略. 考虑到 Transformer 的计算效率随着输入序列长度的增加而下降, 通过省略大量的输入区块, 我们可以节省大量的计算资源, 因此掩蔽自编码器成为预训练大型 Transformer 编码器的优选.

由于解码器层同样是 Transformer, 它需要按照原始图像的维度来运作. 因为 Transformer 的输出与输入具有相同的维度, 我们需要在编码器输出和解码器输入之间恢复图像的维度. 这可以通过重新引入被掩蔽的区块来实现, 每个片段标记都增加了位置编码信息, 并且由固定的掩蔽标记向量表示. 由于解码器表示的维度远高于编码器, 解码器 Transformer 拥有的可学习参数远少于编码器 Transformer. 解码器的输出对接的是一个可学习的线性层, 用于将输出表示映射到像素值空间, 而训练损失函数的作用就是简单地对每个图像缺失区块的均方误差取平均值.

例题 19.1 掩蔽自编码器的图像重构 图 19.6展示了使用训练成功的掩蔽自编码器重建图像的 4 个例子, 展示了训练后的自编码器在生成合理语义上可信的重构能力. 在每个例子中, 被掩蔽的输入在左侧, 重建的图像在中间, 原始图像在右侧; 被掩蔽的输入只剩少量可见补丁, 而重建结果已经恢复了物体的整体轮廓与主要纹理.

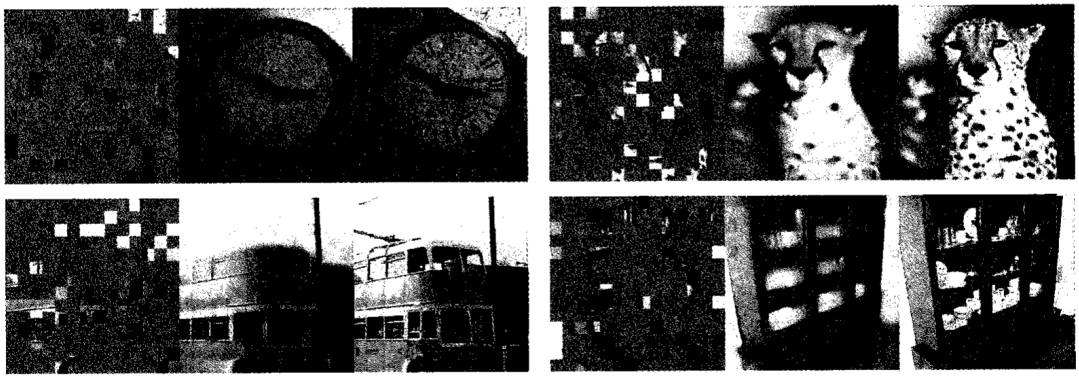


图 19.6: 使用训练成功的掩蔽自编码器重建图像的 4 个例子. 在这些例子中, 有 80% 的输入区块被掩蔽. 在每个例子中, 被掩蔽的图像在左侧, 重建的图像在中间, 原始图像在右侧 [图片来源于 He et al.(2021), 经授权使用]

然而, 我们最终的目标是学习对后续下游任务有用的内部表示, 因此我们舍弃了解码器, 而把编码器应用于没有遮蔽的完整图像, 并配备了一组针对所需应用而微调的新输出层. 请注意, 虽然这一算法最初是为图像数据而设计的, 但理论上可以应用于任何模态的数据.

19.2 变分自编码器

潜变量模型的似然函数由下式给出:

$$p(\boldsymbol{x} \mid \boldsymbol{w}) = \int p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})\, p(\boldsymbol{z})\, \mathrm{d}\boldsymbol{z}, \tag{19.4}$$

其中 $p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$ 由一个深度神经网络定义. 这个似然函数是难以处理的, 因为关于 $\boldsymbol{z}$ 的积分无法解析计算. 变分自编码器 (Variational AutoEncoder,VAE)(Kingma and Welling,2013;Rezende、Mohamed 和 Wierstra,2014;Doersch,2016;Kingma and Welling,2019) 在训练模型时使用了这个似然函数的近似表示来解决这一难题.VAE 背后的关键思想如下:

(1) 使用证据下界来近似似然函数, 从而与 EM 算法形成紧密联系;

(2) 借助摊销推断 (即使用编码器网络) 来近似 E 步骤中潜变量上的后验分布, 而不是精确评估每个数据点的后验分布;

(3) 使用重参数化 (reparameterization) 技巧实现编码器模型的可微分训练 (参见节 15.3).

定义 19.8 (变分自编码器)

考虑一个生成式模型, 其条件分布 $p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$ 覆盖了由深度神经网络输出控制的 D 维数据变量 $\boldsymbol{x}$, 例如 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$ 可以表示 Gauss 条件分布的均值. 同时考虑 M 维潜变量 $\boldsymbol{z}$ 的分布, 它由零均值、单位方差的 Gauss 分布给出:

$$p(\boldsymbol{z}) = \mathcal{N}(\boldsymbol{z} \mid \boldsymbol{0}, \boldsymbol{I}). \tag{19.5}$$

称以(19.4)为似然、以(19.5)为先验, 并用一个编码器网络近似后验 $p(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{w})$ 的潜变量模型为**变分自编码器** (variational autoencoder,VAE).

♣

注

(重构新增) 定义 19.8的出发点是把第 16 章中的线性潜变量模型换成非线性模型:$p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$ 不再由矩阵乘法给出, 而由深度神经网络给出, 因此先验仍然简单 ((19.5)), 但边缘分布 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 可以逼近非常复杂的形状. 代价是(19.4)的积分不可解: 被积函数关于 $\boldsymbol{z}$ 的依赖由神经网络决定, 既没有闭式原函数, 也不能像线性-Gauss 情形那样配方, 这正是需要变分近似的根本原因.

逐项说明条件:D 是数据维数,M 是潜空间维数, 二者不必相等 (通常 $M \ll D$); 先验取标准 Gauss 分布是一个建模选择, 它使采样和 KL 项都有闭式表达, 但并不是唯一选择, 后文的“条件 VAE”就把先验换成 $p(\boldsymbol{z} \mid \boldsymbol{c})$.$p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$ 的具体形式 (Gauss、Bernoulli 等) 决定了(19.7)中被积函数的具体表达式, 而本章的推导只用到它的可微性, 因此对输出分布的形式并不敏感 (参见节 6.5).

为了推导出 VAE 的近似, 首先回想一下, 对于由潜变量 $\boldsymbol{z}$ 描述的空间上的任意概率分布 $q(\boldsymbol{z})$, 以下关系成立 (参见节 15.4):

$$\ln p(\boldsymbol{x} \mid \boldsymbol{w}) = \mathcal{L}(\boldsymbol{w}) + \mathrm{KL}\left(q(\boldsymbol{z}) \,\|\, p(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{w})\right), \tag{19.6}$$

其中 $\mathcal{L}$ 与 KL 的定义如下:

定义 19.9 (证据下界与 KL 散度)

设 $q(\boldsymbol{z})$ 是潜变量空间上的任意概率密度. 称

$$\mathcal{L}(\boldsymbol{w}) = \int q(\boldsymbol{z}) \ln \left\{ \frac{p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})\, p(\boldsymbol{z})}{q(\boldsymbol{z})} \right\} \mathrm{d}\boldsymbol{z} \tag{19.7}$$

为**证据下界** (evidence lower bound) 或 ELBO, 也称**变分下界** (variational lower bound).Kullback-Leibler 散度 (简称 KL 散度)$\mathrm{KL}(\cdot \,\|\, \cdot)$ 定义为

$$\mathrm{KL}\left(q(\boldsymbol{z}) \,\|\, p(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{w})\right) = -\int q(\boldsymbol{z}) \ln \left\{ \frac{p(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{w})}{q(\boldsymbol{z})} \right\} \mathrm{d}\boldsymbol{z}. \tag{19.8}$$

♣

注

(重构新增) 定义 19.9中的(19.7)之所以叫“下界”, 马上会由命题 19.1说明; 之所以叫“证据”, 是因为在 Bayes 语境中 $\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 被称为模型证据 (model evidence). 它与小节 2.5.5中的 KL 散度是同一个量, 只换成了连续潜变量的积分写法:(19.8)右端的负号使 $\mathrm{KL} \geqslant 0$, 这与小节 2.5.5的约定一致, 因此不能把负号理解成“负的散度”.

这里的 $q(\boldsymbol{z})$ 是任意的: 它既可以依赖 $\boldsymbol{x}$(后文取 $q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi})$), 也可以与 $\boldsymbol{x}$ 无关. 下界对任意 q 都成立, 但对不同的 q 松紧不同——q 越接近真实后验 $p(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{w})$, 下界越紧. 这既是变分推断的核心, 也是 VAE 中“训练编码器”的目标: 编码器的任务就是让 q 逼近后验, 从而把下界抬高到接近真实对数似然. 最后, 两个定义都要求相应的积分存在; 若 q 与 $p(\cdot \mid \boldsymbol{x}, \boldsymbol{w})$ 不满足绝对连续性,KL 取 $+\infty$,(19.6)仍按扩展实数理解.

命题 19.1 (证据下界分解)

设 $q(\boldsymbol{z})$ 是潜变量空间上的概率密度, 且 $\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 有限. 则(19.6)成立; 又因 KL 散度满足 $\mathrm{KL}(q \,\|\, p) \geqslant 0$, 可以得到

$$\ln p(\boldsymbol{x} \mid \boldsymbol{w}) \geqslant \mathcal{L}, \tag{19.9}$$

即 $\mathcal{L}$ 是 $\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 的证据下界.

证明

(重构新增) 由于 $p(\boldsymbol{x} \mid \boldsymbol{w})$ 与 z 无关, 且 $\int q(z)\,\mathrm{d}z = 1$, 有

$$\begin{aligned}\ln p(\boldsymbol{x} \mid \boldsymbol{w}) &= \int q(z) \ln p(\boldsymbol{x} \mid \boldsymbol{w})\,\mathrm{d}z \\ &= \int q(z) \ln \frac{p(\boldsymbol{x}, z \mid \boldsymbol{w})}{p(z \mid \boldsymbol{x}, \boldsymbol{w})}\,\mathrm{d}z \\ &= \int q(z) \ln \left\{\frac{p(\boldsymbol{x} \mid z, \boldsymbol{w})\, p(z)}{q(z)}\right\}\mathrm{d}z - \int q(z) \ln \left\{\frac{p(z \mid \boldsymbol{x}, \boldsymbol{w})}{q(z)}\right\}\mathrm{d}z \\ &= \mathcal{L}(\boldsymbol{w}) + \mathrm{KL}\left(q(z) \,\|\, p(z \mid \boldsymbol{x}, \boldsymbol{w})\right),\end{aligned}$$

其中第三步把 $\ln\{p(\boldsymbol{x}, z \mid \boldsymbol{w})/p(z \mid \boldsymbol{x}, \boldsymbol{w})\}$ 拆成 $\ln\{p(\boldsymbol{x} \mid z, \boldsymbol{w})p(z)/q(z)\} - \ln\{p(z \mid \boldsymbol{x}, \boldsymbol{w})/q(z)\}$, 两处都用了 Bayes 定理 $p(\boldsymbol{x}, z \mid \boldsymbol{w}) = p(\boldsymbol{x} \mid z, \boldsymbol{w})p(z) = p(z \mid \boldsymbol{x}, \boldsymbol{w})p(\boldsymbol{x} \mid \boldsymbol{w})$. 第四步按(19.7)与(19.8)的定义. 由于对任意两个概率密度有 $\mathrm{KL} \geqslant 0$, 由(19.6)立得(19.9).

□

注

(重构新增) 命题 19.1给出了一条清晰的改进路径: 对数似然固定, 因此“抬高下界”与“缩小 KL”是同一件事.EM 算法之所以能让下界最终等于对数似然, 是因为它取 q 为精确后验, 使 KL 恰为零;VAE 中 q 被限制在一个神经网络参数化的分布族里,KL 一般不为零, 这就留下了原书后文所说的“残差”.

证明只用到 Bayes 定理与 $\int q = 1$, 因此结论对任意 q 成立, 并不要求 q 是 Gauss 分布或因子分解的. 条件“$\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 有限”用于避免 $\infty - \infty$; 若 q 在 $p(z \mid \boldsymbol{x}, \boldsymbol{w}) = 0$ 的区域为正,KL 为 $+\infty$, 此时(19.6)应理解为扩展实数等式, 下界自动失效. 这也解释了为什么编码器的输出方差必须保持为正: 若某个 $\sigma_j \to 0$,KL 中的 $-\ln \sigma_j^2$ 会发散, 下界随之失去意义.

所以 $\mathcal{L}$ 是 $\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 的证据下界. 虽然对数似然函数 $\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 难以直接计算, 但我们可以使用 Monte Carlo 估计方法来评估这个下界, 从而使它近似于真实对数似然函数的值.

考虑从模型分布 $p(\boldsymbol{x})$ 中独立采样的训练数据集 $\mathcal{D} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$. 这个数据集的对数似然函数由下式给出:

$$\ln p(\mathcal{D} \mid \boldsymbol{w}) = \sum_{n=1}^{N} \mathcal{L}_n + \sum_{n=1}^{N} \mathrm{KL}\left(q_n(\boldsymbol{z}_n) \,\|\, p(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{w})\right), \tag{19.10}$$

其中

$$\mathcal{L}_n = \int q_n(\boldsymbol{z}_n) \ln \left\{\frac{p(\boldsymbol{x}_n \mid \boldsymbol{z}_n, \boldsymbol{w})\, p(\boldsymbol{z}_n)}{q_n(\boldsymbol{z}_n)}\right\}\mathrm{d}\boldsymbol{z}_n. \tag{19.11}$$

请注意, 这引入了与每个数据向量 $\boldsymbol{x}_n$ 相对应的单独潜变量 $\boldsymbol{z}_n$, 就像我们在混合模型和概率 PCA 模型中看到的那样. 因此, 每个潜变量都有自己的独立分布 $q_n(\boldsymbol{z}_n)$, 且都可以单独优化 (参见节 15.2和节 16.2).

由于(19.10)对任意选取的分布 $q_n(\boldsymbol{z})$ 都成立, 我们可以选择能使下限 $\mathcal{L}_n$ 最大化的分布, 或等价地选择最小化 KL 散度 $\mathrm{KL}(q_n(\boldsymbol{z}_n) \| p(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{w}))$ 的分布. 在先前的简单 Gauss 混合模型和概率 PCA 模型中, 我们可以在 EM 算法的 E 步骤中精确计算这些后验分布, 即将每个 $q_n(\boldsymbol{z}_n)$ 设置为相应的后验分布 $p(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{w})$. 此时 KL 散度降为 0, 因此下界等于真实的对数似然.图 19.7使用我们之前在生成对抗网络相关章节所举的简单例子解释了后验分布 (参见小节 16.4.1).

$\boldsymbol{z}_n$ 的精确后验分布由 Bayes 定理给出:

$$p(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{w}) = \frac{p(\boldsymbol{x}_n \mid \boldsymbol{z}_n, \boldsymbol{w})\, p(\boldsymbol{z}_n)}{p(\boldsymbol{x}_n \mid \boldsymbol{w})}. \tag{19.12}$$

对于深度生成式模型,(19.12)中的分子是可以直接计算的. 然而分母是由似然函数给出的, 难以解析计算. 因此, 我们需要找到一个对后验分布的近似. 原则上, 我们可以考虑为每个分布 $q_n(\boldsymbol{z}_n)$ 单独建立参数化模型, 并进行数值优化, 但计算成本非常高, 特别是对于大数据集来说, 此外还需要在每次更新 $\boldsymbol{w}$ 后重新计算分布. 因此, 我们

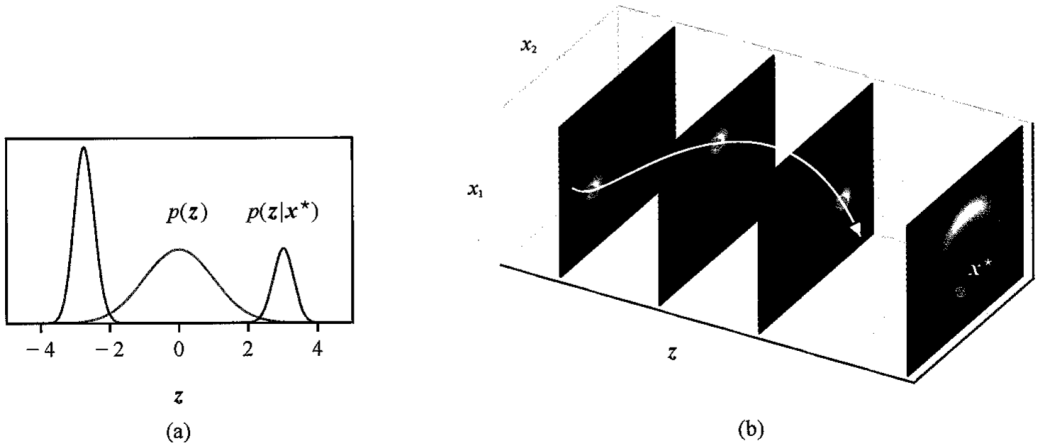


图 19.7: 与图 16.13相似, 计算非线性潜变量模型的后验分布. 边缘分布 $p(\boldsymbol{x})$ 显示在 (b) 的最右侧图中, 其具有香蕉形状, 数据点 $\boldsymbol{x}^*$ 更接近香蕉形状的尖端. 后验分布 $p(\boldsymbol{z} \mid \boldsymbol{x}^*)$ 显示在 (a) 中, 它是双模态的, 即使先验分布 $p(\boldsymbol{z})$ 是单模态的 [来源于 Prince (2020), 经授权使用]

决定换一个方向, 引入第二个神经网络来探索一个更有效的近似框架.

19.2.1 摊销推理

在变分自编码器中, 我们不再单独为每个数据点 $\boldsymbol{x}_n$ 计算一个后验分布 $p(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{w})$, 而是通过训练一个称为编码器网络的神经网络来近似所有这些分布.

定义 19.10 (摊销推理与编码器网络) 用单个以 $\boldsymbol{x}$ 为输入、以 $\boldsymbol{\phi}$ 为参数的神经网络输出条件分布 $q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi})$, 并用它同时近似所有数据点的后验分布 $p(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{w})$, 这种技术称为**摊销推理** (amortized inference), 该神经网络称为**编码器网络** (encoder network),$\boldsymbol{\phi}$ 称为**编码器参数**. ♣

注

(重构新增) 定义 19.10中“摊销”一词来自计算成本的分摊: 逐个数据点优化 q_n 的代价是每个数据点都要单独做一次优化, 且每次更新 $\boldsymbol{w}$ 后旧结果全部作废; 摊销推理把这份代价一次性“摊”到一个共享的编码器上——训练一次, 之后对任何 $\boldsymbol{x}$ 只需一次前向传播就能得到近似后验. 代价是表达能力受限: 一个网络必须同时拟合所有数据点的后验, 遇到多模态或强非 Gauss 的后验时只能给出平均意义上的近似 (见图 19.7中双模态的例子).

条件方面:q 必须以 $\boldsymbol{x}$ 为条件 (否则退化为与数据无关的先验, 即后文的“后验崩塌”);$\boldsymbol{\phi}$ 与解码器参数 $\boldsymbol{w}$ 相互独立, 才能联合优化; 摊销推理本身并不要求 q 是 Gauss 分布, 只要求它可采样、可求 KL 或可估计其梯度——第 (3) 点“重参数化”正是为此服务.

由证据下界给出的目标函数现在同时依赖于 $\boldsymbol{\phi}$ 和 $\boldsymbol{w}$, 我们可以使用基于梯度的优化方法来联合优化这两组参数.

因此,VAE 包含两个参数独立但能联合训练的神经网络: 一个编码器网络, 用于将数据向量映射到潜空间; 一个解码器网络, 用于将潜空间向量映射回数据空间. 这类似于简单的神经网络自编码器模型, 所不同的是, 我们在潜空间中定义了一个概率分布. 我们将看到, 编码器根据 Bayes 定理计算解码器的近似概率逆过程——即根据给定观测数据推断潜变量的分布.

定义 19.11 (对角 Gauss 编码器) 在变分自编码器中, 编码器的一个典型选择是具有对角协方差矩阵的 Gauss 分布, 其均值 μ_j 和方差 σ_j^2 由

一个以 $\boldsymbol{x}$ 作为输入的神经网络给出:

$$q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi}) = \prod_{j=1}^{M} \mathcal{N}\left(z_j \mid \mu_j(\boldsymbol{x}, \boldsymbol{\phi}), \sigma_j^2(\boldsymbol{x}, \boldsymbol{\phi})\right). \tag{19.13}$$

♣

注

(重构新增) 定义 19.11选择因子分解的 Gauss 分布有两个直接好处: 其一,q 与标准 Gauss 先验之间的 KL 有闭式(19.15), 无需 Monte Carlo; 其二, 采样只需 M 个独立的一维 Gauss 样本, 重参数化(19.17)逐分量成立. 代价是排除了后验各分量之间的相关性, 原书后文列出的“真实后验未必服从因子分解的 Gauss 分布”正是这一限制;练习 19.3给出了用下三角矩阵 $\boldsymbol{L}$ 引入一般协方差矩阵的推广.

注意 $\mu_j(\boldsymbol{x}, \boldsymbol{\phi})$ 取值于 $(-\infty, \infty)$, 而 $\sigma_j^2(\boldsymbol{x}, \boldsymbol{\phi})$ 必须非负, 所以两者的输出单元不能用同一种激活函数: 前者可以用线性单元, 后者通常用指数函数 $\exp(\cdot)$ 以保证为正. 用 exp 的代价是数值上可能溢出, 实践中常改用 softplus 或对 $\ln \sigma_j^2$ 直接建模.

请注意, 均值 $\mu_j(\boldsymbol{x}, \boldsymbol{\phi})$ 位于区间 $(-\infty, \infty)$, 因此相应的输出单元激活函数可以是线性的; 而方差 $\sigma_j^2(\boldsymbol{x}, \boldsymbol{\phi})$ 必须是非负的, 所以相关的输出单元通常使用指数函数 $\exp(\cdot)$ 作为激活函数.

我们的优化目标是使用基于梯度的优化方法, 针对参数 $\boldsymbol{\phi}$ 和 $\boldsymbol{w}$ 最大化这个下界, 这通常使用基于小批量数据的随机梯度下降法来实现. 尽管是联合优化参数, 但从概念上我们可以交替地优化 $\boldsymbol{\phi}$ 和 $\boldsymbol{w}$, 如图 19.8所示.

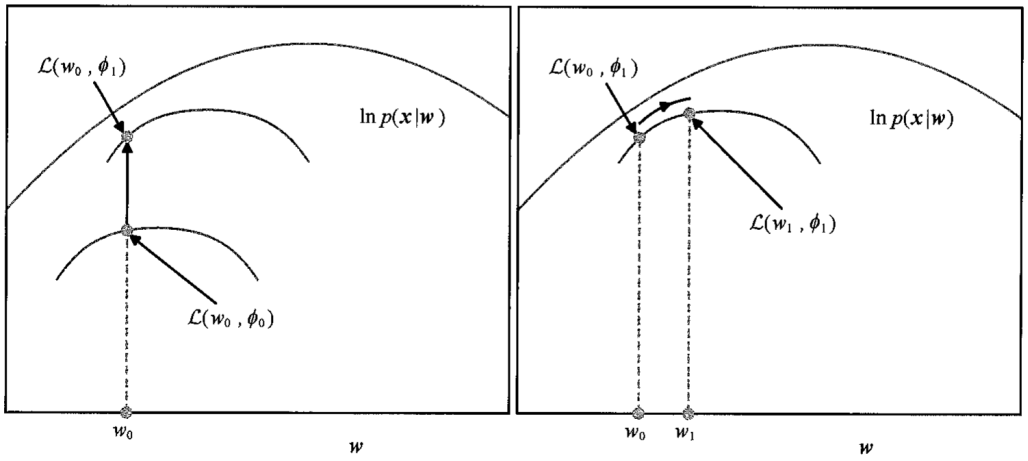


图 19.8: 对证据下界 (ELBO) 进行优化的示意图.(a) 对于解码器网络参数 $\boldsymbol{w}$ 的给定值 $\boldsymbol{w}_0$, 我们可以通过优化编码器网络的参数 $\boldsymbol{\phi}$ 来提高这个下界的值.(b) 对于给定的 $\boldsymbol{\phi}$ 值, 我们可以通过优化 $\boldsymbol{w}$ 来增大 ELBO 函数的值. 请注意, 蓝色曲线所示的 ELBO 函数始终位于红色曲线所示的对数似然函数之下, 这是因为编码器网络通常无法完全匹配真实的后验分布

ELBO 优化与 EM 算法的一个关键区别是, 对于给定的 $\boldsymbol{w}$ 值, 相对于编码器参数 $\boldsymbol{\phi}$ 的优化通常不会将 KL 散度降低到零, 因为编码器网络并不能完美地预测后验潜在分布, 所以下界和真实对数似然之间仍然存在残差. 尽管基于深度神经网络的编码器非常灵活, 但它并不能精确地模拟真实的后验分布, 原因如下:

(1) 真实的条件后验分布未必服从因子分解的 Gauss 分布;

(2) 即使是一个大型神经网络, 也存在灵活性方面的限制;

(3) 训练过程只是一个近似的优化.

图 19.9对 EM 算法和 ELBO 优化之间的关系做了总结.

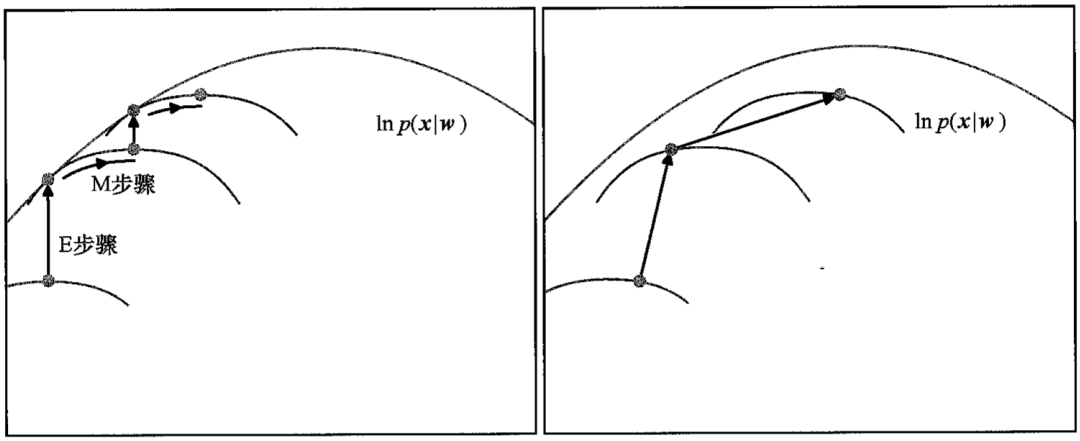


图 19.9: EM 算法和 ELBO 优化之间的关系.(a) 在 EM 算法中, 我们在 E 步骤中交替更新变分后验分布, 并在 M 步骤中更新模型参数.E 步骤是精确的, 在每个 E 步骤之后, 下界与对数似然之间的差距将减小到零.(b) 在 ELBO 优化中, 我们执行的是编码器网络参数 $\boldsymbol{\phi}$(类似于 E 步骤) 和解码器网络参数 $\boldsymbol{w}$(类似于 M 步骤) 的联合优化

19.2.2 重参数化技巧

遗憾的是, 到目前为止, 下界 [式(19.11)] 仍然难以直接计算, 因为在潜变量 $\boldsymbol{z}$ 的积分过程中, 解码器网络会导致被积函数对潜变量存在复杂的依赖关系. 对于数据点 $\boldsymbol{x}_n$, 其对下界的贡献可以表示为:

$$\begin{aligned}\mathcal{L}_n(\boldsymbol{w},\boldsymbol{\phi}) &= \int q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi}) \ln\left\{\frac{p(\boldsymbol{x}_n \mid \boldsymbol{z}_n,\boldsymbol{w})\, p(\boldsymbol{z}_n)}{q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi})}\right\} \mathrm{d}\boldsymbol{z}_n \\ &= \int q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi}) \ln p(\boldsymbol{x}_n \mid \boldsymbol{z}_n,\boldsymbol{w})\, \mathrm{d}\boldsymbol{z}_n - \mathrm{KL}\left(q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi}) \,\|\, p(\boldsymbol{z}_n)\right).\end{aligned} \tag{19.14}$$

(19.14)右侧的第二项是两个 Gauss 分布之间的 KL 散度, 并且可以解析评估 (见练习 2.27):

$$\mathrm{KL}\left(q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi}) \,\|\, p(\boldsymbol{z}_n)\right) = \frac{1}{2}\sum_{j=1}^{M}\left\{1 + \ln\sigma_j^2(\boldsymbol{x}_n) - \mu_j^2(\boldsymbol{x}_n) - \sigma_j^2(\boldsymbol{x}_n)\right\}. \tag{19.15}$$

注

(重构新增) 原文问题说明:(19.15)的右端按(19.8)的符号约定并不等于左端的 KL 散度, 而是它的相反数. 由练习 19.4的直接计算,

$$\mathrm{KL}\left(\mathcal{N}(\mu,\sigma^2) \,\|\, \mathcal{N}(0,1)\right) = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \ln\sigma^2\right) \geqslant 0,$$

把 $-\mathrm{KL}$ 代入(19.14), 即得

$$-\mathrm{KL}\left(q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi}) \,\|\, p(\boldsymbol{z}_n)\right) = \frac{1}{2}\sum_{j=1}^{M}\left\{1 + \ln\sigma_j^2(\boldsymbol{x}_n) - \mu_j^2(\boldsymbol{x}_n) - \sigma_j^2(\boldsymbol{x}_n)\right\},$$

这正是(19.15)的右端. 因此(19.15)应读作“$-\mathrm{KL}$ 的闭式”, 或把花括号内的符号整体取反; 后文(19.19)正是以 $-\mathrm{KL}$ 的形式使用它的, 故损失函数的写法与标准的 ELBO 一致, 只有(19.15)这一行少了一个负号. 若直接按左端为 KL 使用, 会得到“KL 可以取负值”的矛盾结论, 与(19.9)所依赖的 $\mathrm{KL} \geqslant 0$ 冲突.

对于(19.14), 我们可以尝试用一个简单的 Monte Carlo 估计法来近似潜变量 $\boldsymbol{z}_n$ 上的积分:

$$\int q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi}) \ln p(\boldsymbol{x}_n \mid \boldsymbol{z}_n,\boldsymbol{w})\, \mathrm{d}\boldsymbol{z}_n \approx \frac{1}{L}\sum_{l=1}^{L} \ln p(\boldsymbol{x}_n \mid \boldsymbol{z}_n^{(l)},\boldsymbol{w}), \tag{19.16}$$

其中 $\boldsymbol{z}_n^{(l)}$ 是从编码器分布 $q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi})$ 中采样. 关于 $\boldsymbol{w}$ 求导很容易, 但关于 $\boldsymbol{\phi}$ 求梯度却存在问题, 因为参数 $\boldsymbol{\phi}$ 的变化也会改变采样分布 $q(\boldsymbol{z}_n \mid \boldsymbol{x}_n,\boldsymbol{\phi})$. 然而这些样本是固定值, 所以我们没有办法获取这些样本相对于 $\boldsymbol{\phi}$ 的导数. 从概念上讲, 我们可以认为将潜变量 $\boldsymbol{z}_n$ 固定到特定样本值的过程阻止了误差信号向编码器网络的反向传播, 如图 19.10所示.

我们可以使用重参数化技巧来解决这个问题. 重新设计 Monte Carlo 采样过程, 以便明确计算相对于 $\boldsymbol{\phi}$ 的导

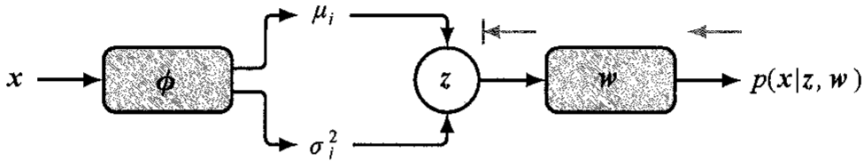


图 19.10: 将潜变量 z 固定到特定采样值的过程阻止了误差信号向编码器网络的反向传播

数.

定义 19.12 (重参数化技巧)

设 $q(z \mid \boldsymbol{x}, \boldsymbol{\phi})$ 是由参数 $\boldsymbol{\phi}$ 指定的一族分布. 若存在与 $\boldsymbol{\phi}$ 无关的随机变量 $\boldsymbol{\varepsilon}$(其分布 $p(\boldsymbol{\varepsilon})$ 与 $\boldsymbol{\phi}$ 无关) 以及关于 $\boldsymbol{\phi}$ 可微的映射 $z = h(\boldsymbol{\varepsilon}; \boldsymbol{\phi})$, 使得当 $\boldsymbol{\varepsilon} \sim p(\boldsymbol{\varepsilon})$ 时 $h(\boldsymbol{\varepsilon}; \boldsymbol{\phi})$ 服从 $q(z \mid \boldsymbol{x}, \boldsymbol{\phi})$, 则称用这种“先采样 $\boldsymbol{\varepsilon}$、再作确定性变换”的方式实现 q 的采样为**重参数化** (reparameterization), 并称之为**重参数化技巧** (reparameterization trick).

♣

注

(重构新增) 定义 19.12的要点是把“随机性”与“参数依赖性”分离: 随机性全部交给与 $\boldsymbol{\phi}$ 无关的 $\boldsymbol{\varepsilon}$, 而 $\boldsymbol{\phi}$ 只出现在确定性的变换 h 中. 这样样本 $\boldsymbol{z}^{(l)} = h(\boldsymbol{\varepsilon}^{(l)}; \boldsymbol{\phi})$ 就是 $\boldsymbol{\phi}$ 的可微函数, 梯度可以沿 h 反传到编码器, 而不必对采样过程本身求导——图 19.10中被切断的反向通路因此重新连通 (见图 19.11).

适用条件是本质性的:h 必须存在且可微. 对连续分布, 这通常可以做到 (如 Gauss 用 $h(\varepsilon; \phi) = \mu + \sigma\varepsilon$); 对离散分布则不存在这样的可微变换, 这正是重参数化技巧“仅限于连续变量上的分布”的原因. 此外,$\boldsymbol{\varepsilon}$ 的分布必须与 $\boldsymbol{\phi}$ 无关, 否则“分离随机性”就无从谈起; 若 q 的支撑随 $\boldsymbol{\phi}$ 变化 (例如均匀分布的区间端点依赖 $\boldsymbol{\phi}$),h 仍可构造, 但需要显式处理端点.

首先请注意, 如果 $\boldsymbol{\varepsilon}$ 是一个均值为零且方差为单位方差的 Gauss 随机变量, 那么:

命题 19.2 (重参数化采样的分布)

设 ε 具有均值为零、方差为单位方差的 Gauss 分布,$\mu \in \mathbb{R}$、$\sigma > 0$. 定义

$$z = \sigma\varepsilon + \mu, \tag{19.17}$$

则 z 具有均值为 μ、方差为 σ^2 的 Gauss 分布 (见练习 19.2).

♠

证明

(重构新增) 记 $\varepsilon \sim \mathcal{N}(0,1)$ 的分布函数为 Φ、密度为 φ. 对任意 $t \in \mathbb{R}$,

$$F_z(t) = \mathbb{P}(z \leqslant t) = \mathbb{P}\Big(\varepsilon \leqslant \frac{t-\mu}{\sigma}\Big) = \Phi\Big(\frac{t-\mu}{\sigma}\Big),$$

其中第二个等号用了 $\sigma > 0$(故不等式方向不变). 对 t 求导, 由链式法则

$$f_z(t) = \frac{1}{\sigma}\varphi\Big(\frac{t-\mu}{\sigma}\Big) = \frac{1}{\sqrt{2\pi}\,\sigma}\exp\Big\{-\frac{(t-\mu)^2}{2\sigma^2}\Big\},$$

这正是 $\mathcal{N}(\mu, \sigma^2)$ 的密度. 又 $\mathbb{E}[z] = \sigma\mathbb{E}[\varepsilon] + \mu = \mu$,$\mathrm{Var}(z) = \sigma^2\,\mathrm{Var}(\varepsilon) = \sigma^2$.

□

注

(重构新增) 命题 19.2是重参数化技巧的算术基础: 它保证“先采 ε 再作线性变换”得到的样本与“直接从 q 采样”同分布, 因此可以放心地把样本写成 $\boldsymbol{\phi}$ 的可微函数. 条件 $\sigma > 0$ 不能去掉: 若 $\sigma = 0$,$z \equiv \mu$ 是退化分布, 分布函数在 $t = \mu$ 处跳跃, 上面的求导不再成立, 此时应按退化分布理解 (采样结果与 $\boldsymbol{\phi}$ 的依赖仍然可微, 只是不再有密度). 多维情形逐分量成立, 因为对角 Gauss 的各分量独立.

原书把它留作练习 19.2, 是为了让读者亲手验证“仿射变换保持 Gaussian 性”这一事实; 更一般的线性变换 $z = \mu + L\varepsilon$ 的协方差是 LL^{T}, 见练习 19.3.

z 将具有均值为 μ、方差为 σ^2 的 Gauss 分布 (见练习 19.2). 将其应用于(19.16)中的采样过程, 其中 μ 和 σ 由编码器网络的输出 $\mu_j(\boldsymbol{x}_n, \boldsymbol{\phi})$ 和 $\sigma_j^2(\boldsymbol{x}_n, \boldsymbol{\phi})$ 确定, 分别对应于(19.13)的均值和方差. 我们不直接从潜变量 $\boldsymbol{z}_n$ 采样, 而是对 $\boldsymbol{\varepsilon}$ 采样, 并使用(19.17)计算对应的 $\boldsymbol{z}_n$ 样本:

$$z_{nj}^{(l)} = \mu_j(\boldsymbol{x}_n, \boldsymbol{\phi})\, \varepsilon_{nj}^{(l)} + \sigma_j^2(\boldsymbol{x}_n, \boldsymbol{\phi}). \tag{19.18}$$

注

(重构新增) 原文问题说明:(19.18)与(19.17)不一致. 按(19.17), 第 j 个分量应为

$$z_{nj}^{(l)} = \mu_j(\boldsymbol{x}_n, \boldsymbol{\phi}) + \sigma_j(\boldsymbol{x}_n, \boldsymbol{\phi})\, \varepsilon_{nj}^{(l)},$$

即 μ_j 与 $\varepsilon_{nj}^{(l)}$ 是相加关系、乘在 ε 上的是标准差 σ_j 而不是方差 σ_j^2.(19.18)若按字面理解, 得到的 z_{nj} 的方差将是 μ_j^2 而非 σ_j^2, 与(19.13)所定义的 q 不符; 把它代入(19.19)也会使 KL 项与重构项不再对应同一个分布. 原书随后在(19.19)下方给出的元素表达式 $z_{nj}^{(l)} = \sigma_{nj}\varepsilon^{(l)} + \mu_{nj}$(其中 $\sigma_{nj} = \sigma_j(\boldsymbol{x}_n, \boldsymbol{\phi})$) 正是上面的正确形式, 可作为内部一致的依据; 算法 19.1 中同一行赋值也应按此更正. 本译本保留原式以供对照, 实际使用时应采用修正形式.

其中 $l = 1, \cdots, L$ 是样本的索引. 该方法显式建立 $\boldsymbol{\phi}$ 与采样值之间的依赖关系, 使梯度计算可行, 如图 19.11所示.

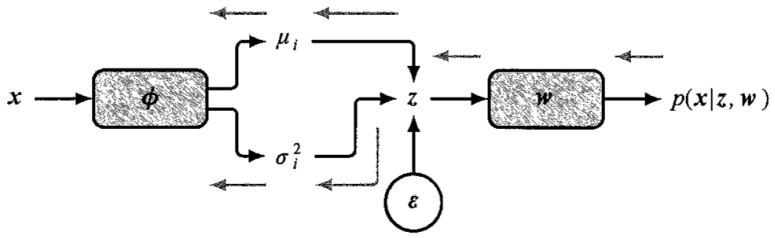


图 19.11: 重参数化技巧通过对一个独立随机变量 $\boldsymbol{\varepsilon}$ 的样本进行计算而不是直接采样 $\boldsymbol{z}$, 从而允许误差信号反向传播到编码器网络. 得到的模型可以使用基于梯度的优化来学习编码器网络和解码器网络的参数

重参数化技巧可以扩展到其他分布, 但仅限于连续变量上的分布. 有的估计器可以直接计算梯度而无须使用重参数化技巧 (Williams,1992), 但这些估计器的方差很大, 因此重参数化技巧也可以看作一种有效的方差降低技术.

命题 19.3 (重参数化梯度估计的无偏性 (重构新增))

设 $q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi}) = \mathcal{N}\big(\boldsymbol{z} \mid \boldsymbol{\mu}(\boldsymbol{x}, \boldsymbol{\phi}), \operatorname{diag}(\boldsymbol{\sigma}^2(\boldsymbol{x}, \boldsymbol{\phi}))\big), \boldsymbol{z} = \boldsymbol{\mu}(\boldsymbol{x}, \boldsymbol{\phi}) + \boldsymbol{\sigma}(\boldsymbol{x}, \boldsymbol{\phi}) \odot \boldsymbol{\varepsilon}, \boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$. 设 $\ln p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$ 关于 $\boldsymbol{z}$ 连续可微, 且存在与 $\boldsymbol{\phi}$ 无关的可积控制函数 $g(\boldsymbol{\varepsilon})$ 使得 $\|\nabla_{\boldsymbol{\phi}} \ln p(\boldsymbol{x} \mid \boldsymbol{\mu}(\boldsymbol{x}, \boldsymbol{\phi}) + \boldsymbol{\sigma}(\boldsymbol{x}, \boldsymbol{\phi}) \odot \boldsymbol{\varepsilon}, \boldsymbol{w})\| \leqslant g(\boldsymbol{\varepsilon})$, 则

$$\nabla_{\boldsymbol{\phi}} \mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x},\boldsymbol{\phi})}\big[\ln p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})\big] = \mathbb{E}_{\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})}\big[\nabla_{\boldsymbol{\phi}} \ln p\big(\boldsymbol{x} \mid \boldsymbol{\mu}(\boldsymbol{x}, \boldsymbol{\phi}) + \boldsymbol{\sigma}(\boldsymbol{x}, \boldsymbol{\phi}) \odot \boldsymbol{\varepsilon}, \boldsymbol{w}\big)\big],$$

即用 $\boldsymbol{\varepsilon}$ 的样本对右端作 Monte Carlo 平均, 得到左端梯度的无偏估计.

证明

(重构新增) 记 $\boldsymbol{z}(\boldsymbol{\varepsilon}; \boldsymbol{\phi}) = \boldsymbol{\mu}(\boldsymbol{x}, \boldsymbol{\phi}) + \boldsymbol{\sigma}(\boldsymbol{x}, \boldsymbol{\phi}) \odot \boldsymbol{\varepsilon}$. 由命题 19.2(逐分量), 当 $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I})$ 时 $\boldsymbol{z}(\boldsymbol{\varepsilon}; \boldsymbol{\phi}) \sim q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi})$, 故对任意可积的 F,

$$\mathbb{E}_{q(\boldsymbol{z}|\boldsymbol{x},\boldsymbol{\phi})}[F(\boldsymbol{z})] = \mathbb{E}_{\boldsymbol{\varepsilon}}[F(\boldsymbol{z}(\boldsymbol{\varepsilon}; \boldsymbol{\phi}))].$$

取 $F(\boldsymbol{z}) = \ln p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$. 右端的被积函数关于 $\boldsymbol{\phi}$ 可微, 且由控制函数条件可在积分号下求导, 于是

$$\begin{aligned}
\nabla_{\boldsymbol{\phi}} \mathbb{E}_q[F] &= \nabla_{\boldsymbol{\phi}} \int F\big(\boldsymbol{z}(\boldsymbol{\varepsilon}; \boldsymbol{\phi})\big) p(\boldsymbol{\varepsilon})\, \mathrm{d}\boldsymbol{\varepsilon} \\
&= \int \nabla_{\boldsymbol{z}} F\big(\boldsymbol{z}(\boldsymbol{\varepsilon}; \boldsymbol{\phi})\big)^{\mathsf{T}} \nabla_{\boldsymbol{\phi}} \boldsymbol{z}(\boldsymbol{\varepsilon}; \boldsymbol{\phi})\, p(\boldsymbol{\varepsilon})\, \mathrm{d}\boldsymbol{\varepsilon} \\
&= \mathbb{E}_{\boldsymbol{\varepsilon}}\big[\nabla_{\boldsymbol{\phi}} \ln p\big(\boldsymbol{x} \mid \boldsymbol{\mu}(\boldsymbol{x}, \boldsymbol{\phi}) + \boldsymbol{\sigma}(\boldsymbol{x}, \boldsymbol{\phi}) \odot \boldsymbol{\varepsilon}, \boldsymbol{w}\big)\big],
\end{aligned}$$

其中第二个等号是链式法则, 第三个等号把对 $\boldsymbol{\phi}$ 的全导数写成 $\nabla_{\boldsymbol{\phi}} \ln p(\boldsymbol{x} \mid \boldsymbol{z}(\boldsymbol{\varepsilon}; \boldsymbol{\phi}), \boldsymbol{w})$. 用 L 个独立样本平均即得无

偏估计, 其偏差为零、方差随 L 增大而减小.

□

注

(重构新增) 命题 19.3中的重参数化梯度, 与得分函数估计器相比用的是“路径导数”: 它把 $\boldsymbol{\phi}$ 的变化如何改变 $\boldsymbol{z}$、进而改变 $\ln p$ 这一整条路径都算了进去, 而不是像练习 19.1那样把被积函数当作与 $\boldsymbol{\phi}$ 无关的 G 再乘上 $\nabla_{\boldsymbol{\phi}} \ln q$. 这就是原书所说“重参数化技巧更有效率、只需要更少的样本”的确切含义: 路径导数利用了 $\ln p$ 关于 $\boldsymbol{z}$ 的梯度信息, 而得分函数估计器完全没有用它, 因而方差通常大得多.

条件“控制函数可积”不能省: 若 $\ln p$ 的梯度随 $\boldsymbol{\varepsilon}$ 增长过快, 积分号下求导可能失败, 此时右端的期望可能不存在, 无偏性也就无从谈起. 实际实现中,$\boldsymbol{\mu}$ 与 $\boldsymbol{\sigma}$ 由神经网络给出, 只要求它们关于 $\boldsymbol{\phi}$ 可微, 对网络本身的形式没有额外要求; 这也是重参数化技巧能与任意深度网络结合的原因.

使用我们特定的建模假设,VAE 的完整损失函数变为

$$\mathcal{L} = \sum_{n} \left\{ \frac{1}{2} \sum_{j=1}^{M} \left\{ 1 + \ln \sigma_{nj}^2 - \mu_{nj}^2 - \sigma_{nj}^2 \right\} + \frac{1}{L} \sum_{l=1}^{L} \ln p(\boldsymbol{x}_n \mid \boldsymbol{z}_n^{(l)}, \boldsymbol{w}) \right\} \tag{19.19}$$

其中 $\boldsymbol{z}_n^{(l)}$ 由元素 $z_{nj}^{(l)} = \sigma_{nj} \varepsilon^{(l)} + \mu_{nj}$ 构成,$\mu_{nj} = \mu_j(\boldsymbol{x}_n, \boldsymbol{\phi})$,$\sigma_{nj} = \sigma_j(\boldsymbol{x}_n, \boldsymbol{\phi})$. 在(19.19)中, 求和操作遍历当前小批量中的所有数据点. 每个数据点 $\boldsymbol{x}_n$ 的样本数 L 通常设置为 1, 即仅使用单个样本进行估计. 尽管这种设置会导致对证据下界的计算结果存在噪声, 但由于随机梯度优化过程本身就具有噪声, 该策略在整体上仍能实现更高效的参数优化.

我们可以将 VAE 训练总结如下. 对于小批量中的每个数据点, 通过编码器网络前向传播来计算近似潜变量分布的均值和方差, 并使用重参数化技巧从该分布中采样, 然后将这些样本通过解码器网络传播以计算证据下界[式(19.19)]. 最后使用自动微分计算相对于 $\boldsymbol{w}$ 和 $\boldsymbol{\phi}$ 的梯度. 算法 19.1 对 VAE 训练做了总结, 为了清晰起见, 我们省略了使用小批量数据点这一细节. 一旦模型训练完成, 编码器网络就被丢弃, 新的数据点将通过从先验分布 $p(\boldsymbol{z})$ 中采样并通过解码器网络前向传播来生成数据空间中的样本.

算法 19.1　VAE 训练

Input: 训练数据集 $\mathcal{D} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$
　编码器网络 $\{\mu_j(\boldsymbol{x}_n, \boldsymbol{\phi}), \sigma_j^2(\boldsymbol{x}_n, \boldsymbol{\phi})\}, j \in \{1, \cdots, M\}$
　解码器网络 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$
　初始权重向量 $\boldsymbol{w}, \boldsymbol{\phi}$
　学习率 η
Output: 最终权重向量 $\boldsymbol{w}, \boldsymbol{\phi}$

---

repeat
　$\mathcal{L} \leftarrow 0$
　for $j \in \{1, \cdots, M\}$ **do**
　　$\varepsilon_{nj} \sim \mathcal{N}(0, 1)$
　　$z_{nj} \leftarrow \mu_j(\boldsymbol{x}_n, \boldsymbol{\phi})\, \varepsilon_{nj} + \sigma_j^2(\boldsymbol{x}_n, \boldsymbol{\phi})$　// 原文如此, 见上文更正
　　$\mathcal{L} \leftarrow \mathcal{L} + \frac{1}{2}\left\{1 + \ln \sigma_{nj}^2 - \mu_{nj}^2 - \sigma_{nj}^2\right\}$
　end for
　$\mathcal{L} \leftarrow \mathcal{L} + \ln p(\boldsymbol{x}_n \mid \boldsymbol{z}_n, \boldsymbol{w})$
　$\boldsymbol{w} \leftarrow \boldsymbol{w} + \eta \nabla_{\boldsymbol{w}} \mathcal{L}$　// 更新解码器权重
　$\boldsymbol{\phi} \leftarrow \boldsymbol{\phi} + \eta \nabla_{\boldsymbol{\phi}} \mathcal{L}$　// 更新编码器权重
until 收敛
return $\boldsymbol{w}, \boldsymbol{\phi}$

训练成功后, 我们想要评估模型表示新测试点 $\hat{\boldsymbol{x}}$ 的效果如何. 由于对数似然是不可计算的, 我们可以使用证据下界 $\mathcal{L}$ 作为近似. 对此, 我们可以从 $q(\boldsymbol{z} \mid \hat{\boldsymbol{x}}, \boldsymbol{\phi})$ 中采样, 因为这比从 $p(\boldsymbol{z})$ 中采样能提供更准确的估计.

VAE 有许多变种. 当应用于图像数据时, 编码器通常基于卷积操作, 而解码器则基于转置卷积 (有时也称反卷积) 操作 (参见小节 10.5.3). 在条件 VAE 中, 编码器和解码器都将一个条件变量 $\boldsymbol{c}$ 作为额外的输入. 例如, 我们可能想要生成物体的图像, 此时 $\boldsymbol{c}$ 代表物体的类别. 潜空间的先验分布 $p(\boldsymbol{z})$ 既可以是一个简单的 Gauss 分布, 也可以扩展成另一个神经网络给出的条件分布 $p(\boldsymbol{z} \mid \boldsymbol{c})$, 训练与测试过程与之前类似.

注意,(19.14)中的第一项鼓励编码器分布 $q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi})$ 接近先验分布 $p(\boldsymbol{z})$, 从而使解码器在生成模式下, 通过从 $p(\boldsymbol{z})$ 采样, 能够产生逼真的输出. 在训练 VAE 时, 可能会出现一个问题, 即变分分布 $q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi})$ 收敛到先验分布 $p(\boldsymbol{z})$, 这意味着变分分布没有从输入 $\boldsymbol{x}$ 中学到有价值的信息. 实际上, 潜在编码被忽略了, 因为它对输入 $\boldsymbol{x}$ 没有区分能力.

定义 19.13 (后验崩塌)

在训练变分自编码器时, 若变分分布 $q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi})$ 收敛到先验分布 $p(\boldsymbol{z})$, 从而不再从输入 $\boldsymbol{x}$ 中提取任何信息, 则称这种现象为**后验崩塌** (posterior collapse). 此时 $\mathrm{KL}(q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi}) \| p(\boldsymbol{z}))$ 接近于零. ♣

注

(重构新增) 定义 19.13的判别信号是“KL 项很小而重构项也不小”, 这与“训练成功”时的“KL 小而重构项很小”容易混淆, 区别在于解码器是否忽略 $\boldsymbol{z}$: 若把 $\boldsymbol{z}$ 换成任意噪声而重建不变, 说明编码已经崩塌. 其原因可以从(19.14)读出:KL 项想要把 q 拉向先验, 重构项想要 q 保留信息, 当解码器本身足够强大 (例如自回归解码器可以独自拟合数据) 时, 重构项对 $\boldsymbol{z}$ 的依赖变弱,KL 项就占了上风, 最优解退化为 $q = p(\boldsymbol{z})$.

这一现象的条件性在于: 它不是优化失败, 而是目标函数的偏好; 因此增加网络容量或训练步数并不能消除它, 需要修改目标 (如 β-VAE 或对 KL 项退火) 或限制解码器. 此时若取一个输入做“编码—解码”, 会得到看起来模糊不清的重建结果, 因为解码器只依赖先验而不再依赖具体输入.

此时, 如果我们取一个输入并编码, 然后解码, 我们将得到一个看起来模糊不清的重建结果. 在这种情况下,KL 散度 $\mathrm{KL}(q(\boldsymbol{z} \mid \boldsymbol{x}, \boldsymbol{\phi}) \| p(\boldsymbol{z}))$ 接近于零.

另一种问题发生在潜在编码没有被压缩的情况下, 具体表现是, 即使能得到高度准确的重建结果, 但从 $p(\boldsymbol{z})$ 采样并通过解码器网络生成的输出质量却很差, 不像训练数据. 在这种情况下,KL 散度相对较大, 并且因为训练系统获得了一个与先验分布完全不同的变分分布, 从先验分布中采样的样本不会生成逼真的输出.

这两个问题都可以通过在(19.14)的第一项前引入一个系数 β 来解决, 以控制 KL 散度的正则化效果, 通常 $\beta > 1$(Higgins et al.,2017). 如果重建结果很差, 则可以增大 β; 而如果生成的样本很差, 则可以减小 β.β 的值也可以根据退火调度来设置, 即它从一个小的值开始, 并在训练期间逐渐增大.

注

(重构新增) 原文问题说明: 这一段对 β 的位置与调节方向的表述有两处需要澄清. 第一, 按(19.14)的书写顺序, 鼓励 q 接近先验的是第二项 (KL 项), 而不是第一项; 原书说“第一项”, 沿用的是把 KL 项写在最前面的等价写法——(19.19)中 KL 项确实排在前面. 第二, 若 β 乘在 KL 项前 (这正是 β-VAE 的标准做法, 也与“控制 KL 散度的正则化效果”相符), 则 β 越大正则化越强、重构越差、生成越接近先验; 因此正确的调节方向应是“重建差则减小 β, 生成差则增大 β”, 与原文相反. 若把 β 理解为乘在重构项前, 则原文的方向是对的, 但“控制 KL 散度的正则化效果”一句就不成立. 两种读法不能同时成立, 读者应以实际实现中 β 所乘的项为准; 本译本保留原文表述以便对照.

最后请注意, 虽然我们在本章中只讨论了一个表示 Gauss 输出分布均值的解码器网络 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w})$, 但我们可以扩展 VAE, 让它包含代表 Gauss 分布方差的输出, 或进一步让它表示其他更复杂分布的参数, 以描绘更广泛多变的数据特征 (参见节 6.5).

19.3 习题

✍ **练习 19.1** (★★) 证明对于任何分布 $q(\boldsymbol{z} \mid \boldsymbol{\phi})$ 和任何函数 $G(\boldsymbol{z})$, 以下关系成立:

$$\nabla_{\boldsymbol{\phi}} \int q(\boldsymbol{z} \mid \boldsymbol{\phi}) G(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z} = \int q(\boldsymbol{z} \mid \boldsymbol{\phi}) G(\boldsymbol{z}) \nabla_{\boldsymbol{\phi}} \ln q(\boldsymbol{z} \mid \boldsymbol{\phi}) \, \mathrm{d}\boldsymbol{z}. \tag{19.20}$$

证明(19.20)左侧的部分可以通过以下 Monte Carlo 估计器来近似:

$$\nabla_{\boldsymbol{\phi}} \int q(\boldsymbol{z} \mid \boldsymbol{\phi}) G(\boldsymbol{z}) \, \mathrm{d}\boldsymbol{z} \approx \sum_i G(\boldsymbol{z}^{(i)}) \nabla_{\boldsymbol{\phi}} \ln q(\boldsymbol{z}^{(i)} \mid \boldsymbol{\phi}), \tag{19.21}$$

其中样本集 $\{\boldsymbol{z}^{(i)}\}$ 是独立地从分布 $q(\boldsymbol{z} \mid \boldsymbol{\phi})$ 中抽取的. 验证这个估计器是无偏的, 即(19.21)右侧部分的平均值在样本分布上取平均后等于(19.21)左侧部分. 原则上, 通过设置 $G(\boldsymbol{z}) = p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$, 这个结果将允许在不使用重参数化技巧的情况下计算(19.14)右侧第二项关于 $\boldsymbol{\phi}$ 的梯度. 另外, 由于这种方法是无偏的, 因此它能够在无限数量的样本极限下给出精确答案. 然而, 重参数化技巧更有效率, 因为可以直接计算 $\boldsymbol{\phi}$ 的改变而导致 $\boldsymbol{z}$ 的改变所引起的 $p(\boldsymbol{x} \mid \boldsymbol{z}, \boldsymbol{w})$ 的变化, 这意味着只需要更少的样本就能获得良好的准确度.

✍ **练习 19.2** (★) 验证如果 ε 具有零均值、单位方差的 Gauss 分布, 则(19.17)中的变量 z 将具有均值为 μ、方差为 σ^2 的 Gauss 分布.

✍ **练习 19.3** (★★) 将具有对角协方差矩阵的 VAE 编码器网络 [式(19.13)] 扩展成一个具有一般协方差矩阵的网络. 考虑一个从简单 Gauss 分布中抽取的 K 维随机向量:

$$\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{0}, \boldsymbol{I}), \tag{19.22}$$

然后使用下面的关系对其进行线性变换:

$$\boldsymbol{z} = \boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{\varepsilon}, \tag{19.23}$$

其中 $\boldsymbol{L}$ 是一个下三角矩阵 (即一个 $K \times K$ 矩阵, 其中主对角线以上所有的元素都是零). 证明 $\boldsymbol{z}$ 具有分布 $\mathcal{N}(\boldsymbol{z} \mid \boldsymbol{\mu}, \boldsymbol{\Sigma})$, 并写出 $\boldsymbol{\Sigma}$ 关于 $\boldsymbol{L}$ 的表达式, 解释为什么 $\boldsymbol{L}$ 的对角元素必须是非负的. 描述 $\boldsymbol{\mu}$ 和 $\boldsymbol{L}$ 如何能够作为神经网络的输出, 并讨论适合的输出单元激活函数.

✍ **练习 19.4** (★★) 计算(19.14)中的 KL 散度项, 并计算这一项相对于 $\boldsymbol{w}$ 和 $\boldsymbol{\phi}$ 的梯度, 以便训练编码器网络和解码器网络.

✍ **练习 19.5** (★)(19.11)给出的证据下界可以写成(19.14)的形式, 证明其也可以写成如下形式:

$$\mathcal{L}_n(\boldsymbol{w}, \boldsymbol{\phi}) = \int q(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{\phi}) \ln \big\{ p(\boldsymbol{x}_n \mid \boldsymbol{z}_n, \boldsymbol{w}) \, p(\boldsymbol{z}_n) \big\} \mathrm{d}\boldsymbol{z}_n - \int q(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{\phi}) \ln q(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{\phi}) \, \mathrm{d}\boldsymbol{z}_n. \tag{19.24}$$

✍ **练习 19.6** (★) 证明(19.11)给出的证据下界可以写成如下形式:

$$\mathcal{L}_n(\boldsymbol{w}, \boldsymbol{\phi}) = \int q(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{\phi}) \ln p(\boldsymbol{z}_n) \, \mathrm{d}\boldsymbol{z}_n + \int q(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{\phi}) \ln \left\{ \frac{p(\boldsymbol{x}_n \mid \boldsymbol{z}_n, \boldsymbol{w})}{q(\boldsymbol{z}_n \mid \boldsymbol{x}_n, \boldsymbol{\phi})} \right\} \mathrm{d}\boldsymbol{z}_n. \tag{19.25}$$

第 20 章 扩散模型

构建丰富的生成式模型的一种行之有效的方法是引入一个潜变量 $\boldsymbol{z}$ 上的分布 $p(\boldsymbol{z})$, 然后使用深度神经网络将 $\boldsymbol{z}$ 转换到数据空间 $\boldsymbol{x}$ 中. 我们仅需要使用一个简单的固定分布来表示 $p(\boldsymbol{z})$, 例如 Gauss 分布 $\mathcal{N}(\boldsymbol{z} \mid \mathbf{0}, \boldsymbol{I})$, 因为神经网络的通用性能够将其转换成一组关于 $\boldsymbol{x}$ 的高度灵活的分布. 在前面的章节 (参见小节 16.4.4) 中, 我们探讨了多种属于这一框架但基于不同方法定义和训练深度神经网络的模型, 包括生成对抗网络、变分自编码器和标准化流.

在本章中, 我们将讨论这一通用框架下的第 4 类模型——**扩散模型** (diffusion model), 也叫去噪扩散概率模型 (Denoising Diffusion Probabilistic Model, DDPM)(Sohl-Dickstein et al., 2015; Ho, Jain, and Abbeel, 2020), 在许多应用领域, 它们已经成为最先进的技术. 尽管这个框架有更广泛的适用性, 但我们仍将以图像数据为例讲述. 这类模型的核心思想是对每一个训练图像, 通过多步骤的噪声来损坏它, 将其转变为 Gauss 分布的样本, 如图 20.1所示. 然后训练一个深度神经网络来逆转这个过程, 一旦训练完成, 该网络就可以基于 Gauss 分布的样本开始生成新的图像.

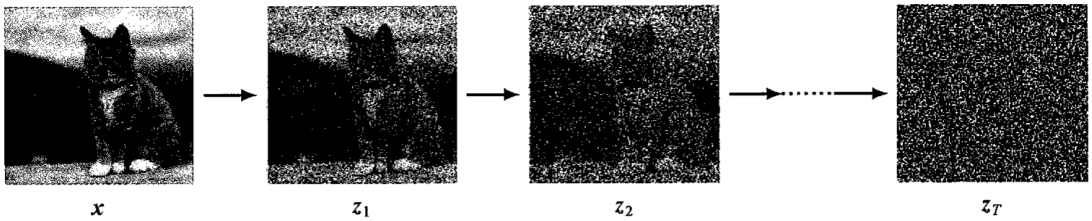


图 20.1: 扩散模型中编码过程的示意图. 一幅图像 $\boldsymbol{x}$ 逐渐被叠加的 Gauss 噪声破坏, 在经过多个阶段后, 得到一系列越来越嘈杂的图像. 最后的结果与从 Gauss 分布中抽取的样本已无法区分. 我们想要训练一个深度神经网络来逆转这个过程

20.1 前向编码器

扩散模型可以视为一种层次化的变分自编码器, 其中编码器分布是固定的, 并由噪声过程定义, 只有生成分布是学习得来的 (Luo, 2022)(参见节 19.2). 它们易于训练, 在并行硬件上具有良好的可扩展性, 在避免对抗训练所带来的挑战和不稳定性的同时, 可以产生与生成对抗网络相当或更优的结果. 然而, 由于需要通过解码器网络进行多次前向传播, 它在生成新样本时会产生高昂的计算成本 (Dhariwal and Nichol, 2021).

从训练集中选取一个图像, 表示为 $\boldsymbol{x}$, 然后将图像中的每个像素独立地与 Gauss 噪声混合, 得到一个被噪声损坏的图像 $\boldsymbol{z}_1$, 其定义如下:

$$\boldsymbol{z}_1 = \sqrt{1-\beta_1}\,\boldsymbol{x} + \sqrt{\beta_1}\,\boldsymbol{\varepsilon}_1. \tag{20.1}$$

其中 $\boldsymbol{\varepsilon}_1 \sim \mathcal{N}(\boldsymbol{\varepsilon}_1 \mid \mathbf{0}, \boldsymbol{I})$ 且 $\beta_1 < 1$ 是噪声分布的方差.(20.1)中的系数 $\sqrt{1-\beta_1}$ 和 $\sqrt{\beta_1}$ 确保了 $\boldsymbol{z}_1$ 的分布均值比 $\boldsymbol{x}$ 的分布均值更接近于零, 且 $\boldsymbol{z}_1$ 的方差比 $\boldsymbol{x}$ 的方差更接近单位矩阵 (见练习 20.1). 我们可以将(20.1)写成以下形式 (参见练习 20.2):

$$q(\boldsymbol{z}_1 \mid \boldsymbol{x}) = \mathcal{N}\left(\boldsymbol{z}_1 \mid \sqrt{1-\beta_1}\,\boldsymbol{x},\, \beta_1 \boldsymbol{I}\right). \tag{20.2}$$

然后重复这个过程, 加入额外的独立 Gauss 噪声, 从而生成一系列越来越多的加噪图像 $\boldsymbol{z}_1, \cdots, \boldsymbol{z}_T$. 注意, 在关于扩散模型的文献中, 这些加噪图像或潜变量有时表示为 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_T$, 而观测变量则表示为 $\boldsymbol{x}_0$. 在本书中, 我们使用 $\boldsymbol{z}$ 来表示潜变量, 并使用 $\boldsymbol{x}$ 来表示观测变量. 每一个后续图像由下式给出:

$$\boldsymbol{z}_t = \sqrt{1-\beta_t}\,\boldsymbol{z}_{t-1} + \sqrt{\beta_t}\,\boldsymbol{\varepsilon}_t, \tag{20.3}$$

其中 $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\boldsymbol{\varepsilon}_t \mid \mathbf{0}, \boldsymbol{I})$. 同样, 我们也可以将(20.3)写成以下形式:

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}) = \mathcal{N}\left(\boldsymbol{z}_t \mid \sqrt{1-\beta_t}\,\boldsymbol{z}_{t-1},\, \beta_t \boldsymbol{I}\right). \tag{20.4}$$

条件分布序列(20.4)形成了一条 Markov 链, 并且可以表示为图 20.2所示的概率图模型 (参见节 11.1). 方差参数 $\beta_t \in (0,1)$, 并且通常是手动设置的, 它们按预定的计划递增, 即 $\beta_1 < \beta_2 < \cdots < \beta_T$.

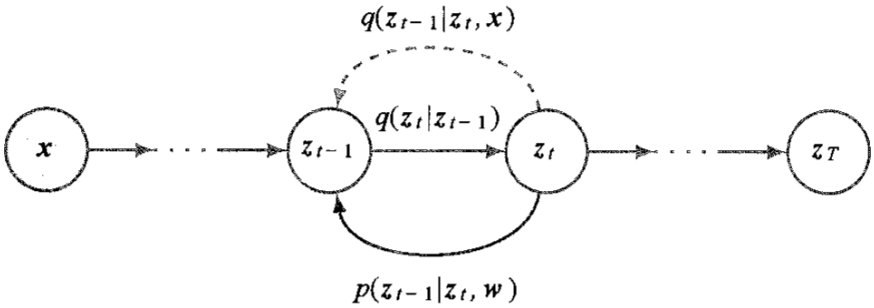


图 20.2: 扩散过程可以表示为一个概率图模型. 原始图像 $\boldsymbol{x}$ 表示为带有底色的节点, 因为它是一个观测变量, 而加噪图像 $z_1, \cdots, z_T$ 则视为潜变量. 加噪过程由前向分布 $q(z_t \mid z_{t-1})$ 定义, 它可以视为一个编码器. 我们的目标是学习一个条件分布 $p(z_{t-1} \mid z_t, \boldsymbol{w})$ 并试图逆转这个过程, 可以把它视为一个解码器. 正如我们稍后将看到的, 条件分布 $q(z_{t-1} \mid z_t, \boldsymbol{x})$ 在定义训练的过程中扮演了重要角色

20.1.1 扩散核

在观测到数据向量 $\boldsymbol{x}$ 的条件下, 潜变量的联合分布由下式给出:

$$q(z_1, \cdots, z_T \mid \boldsymbol{x}) = q(z_1 \mid \boldsymbol{x}) \prod_{t=2}^{T} q(z_t \mid z_{t-1}). \tag{20.5}$$

定义 20.1 (累积噪声系数)

设 $\beta_1, \cdots, \beta_T$ 为噪声调度. 对 $t = 1, \cdots, T$, 定义

$$\alpha_t = \prod_{s=1}^{t} (1 - \beta_s), \tag{20.6}$$

并约定 $\alpha_0 = 1$. 称 α_t 为前 t 步的**累积噪声系数** (cumulative noise coefficient).

♣

注

(重构新增) 引入 α_t 的目的是把“逐步相乘的噪声调度”压缩为一个标量. 由(20.6)立刻得到两个递推关系:$\alpha_t = (1-\beta_t)\alpha_{t-1}$, 以及 $1-\alpha_t$ 随 t 单调增加. 后者正是“总噪声方差”的度量:$\sqrt{\alpha_t}$ 是原始信号在 z_t 中保留的幅度,$\sqrt{1-\alpha_t}$ 是累计注入的噪声幅度, 二者平方和为 1, 这与(20.8)的系数完全一致.

约定 $\alpha_0 = 1$ 不是多余的记号: 它使 $t = 1$ 时的(20.7)退化为(20.2), 从而下面的归纳法可以从 $t = 1$ 统一处理. 由于 $\beta_t \in (0,1)$, 数列 $\{\alpha_t\}$ 严格递减且恒正;α_T 的大小由噪声调度决定, 它衡量前向过程结束时还剩多少信号.

需要提示的是, 扩散模型文献 (例如 DDPM) 常把这里的 α_t 记作 $\bar{\alpha}_t$, 以便与单步系数 $1-\beta_t$ 区分. 本书直接使用 α_t 表示累积量, 阅读其他文献时须注意这一记号差异.

对中间变量 $z_1, \cdots, z_{t-1}$ 进行边缘化, 即可得到**扩散核** (diffusion kernel)(参见练习 20.3):

命题 20.1 (扩散核的闭式边缘分布)

设前向过程由(20.4)给出,α_t 由(20.6)定义. 则对任意 $t = 1, \cdots, T$,

$$q(z_t \mid \boldsymbol{x}) = \mathcal{N}\left(z_t \mid \sqrt{\alpha_t}\boldsymbol{x}, (1-\alpha_t)\boldsymbol{I}\right). \tag{20.7}$$

♠

证明

(重构新增) 对 t 作归纳. 当 $t=1$ 时,$\alpha_1 = 1-\beta_1$, 而(20.2)正是(20.7)在 $t=1$ 时的形式. 设(20.7)对 $t-1$ 成立, 即 $z_{t-1} = \sqrt{\alpha_{t-1}}\,\boldsymbol{x} + \sqrt{1-\alpha_{t-1}}\,\boldsymbol{\eta}$, 其中 $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$ 且与 $\boldsymbol{\varepsilon}_t$ 独立. 代入(20.3)得

$$\begin{aligned} z_t &= \sqrt{1-\beta_t}\left(\sqrt{\alpha_{t-1}}\,\boldsymbol{x} + \sqrt{1-\alpha_{t-1}}\,\boldsymbol{\eta}\right) + \sqrt{\beta_t}\,\boldsymbol{\varepsilon}_t \\ &= \sqrt{(1-\beta_t)\alpha_{t-1}}\,\boldsymbol{x} + \sqrt{(1-\beta_t)(1-\alpha_{t-1})}\,\boldsymbol{\eta} + \sqrt{\beta_t}\,\boldsymbol{\varepsilon}_t. \end{aligned}$$

两个独立 Gauss 变量之和仍为 Gauss 变量, 且均值与协方差分别相加, 故 z_t 的条件分布为 Gauss 分布, 其均值为 $\sqrt{(1-\beta_t)\alpha_{t-1}}\,\boldsymbol{x} = \sqrt{\alpha_t}\,\boldsymbol{x}$, 协方差为

$$(1-\beta_t)(1-\alpha_{t-1})\boldsymbol{I} + \beta_t \boldsymbol{I} = \big(1-\alpha_{t-1}-\beta_t+\beta_t\alpha_{t-1}+\beta_t\big)\boldsymbol{I} = \big(1-(1-\beta_t)\alpha_{t-1}\big)\boldsymbol{I} = (1-\alpha_t)\boldsymbol{I},$$

其中最后一个等号用了 $\alpha_t = (1-\beta_t)\alpha_{t-1}$. 这正是(20.7). 证毕.

□

注

(重构新增) (20.7)是整章的计算基石: 它把“沿着 Markov 链走 t 步”换成了“一次加噪”, 因而训练时可以从任意中间步骤直接采样, 无须逐级展开整条链. 这一点的实际意义在于, 随机梯度下降的每一步只需一次前向传播, 计算量与 t 无关.

条件必须交代清楚: 推导只用到“各步噪声 $\boldsymbol{\varepsilon}_t$ 相互独立且与 $\boldsymbol{x}$ 独立”, 以及 $\beta_t \in (0,1)$ 保证 $\alpha_t > 0$、从而 $\sqrt{\alpha_t}$ 与 $1-\alpha_t$ 都有意义. 若 β_t 允许取 1, 则 $\alpha_t = 0$,z_t 与 $\boldsymbol{x}$ 完全独立, 扩散核退化为先验, 链在该步之后不再携带任何数据信息.

还要注意(20.7)中的 $\boldsymbol{x}$ 被当作固定向量处理: 它是“以 $\boldsymbol{x}$ 为条件”的分布, 而不是对数据分布 $p(\boldsymbol{x})$ 求边缘后的分布. 对 $\boldsymbol{x}$ 再积分得到的是混合分布 (见(20.12)), 一般不再是 Gauss 分布.

可以看到, 每一个中间分布都是一个简单的闭式的 Gauss 分布, 我们可以直接从中采样, 这在训练 DDPM 时非常有用, 因为它允许我们使用 Markov 链中随机选择的中间项进行高效的随机梯度下降, 而不必运行整个 Markov 链. 我们也可以将(20.3)写成以下形式:

$$z_t = \sqrt{\alpha_t}\,\boldsymbol{x} + \sqrt{1-\alpha_t}\,\boldsymbol{\varepsilon}_t, \tag{20.8}$$

其中仍然有 $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\boldsymbol{\varepsilon}_t \mid \mathbf{0}, \boldsymbol{I})$. 请注意, 这里的 ε_t 代表添加到原始图像上的总噪声, 而不是在 Markov 链这一步骤中添加的增量噪声.

在经过多个这样的步骤之后, 图像变得与 Gauss 噪声无法区分. 在 $T \to \infty$ 的情况下, 我们有 (见练习 20.4)

$$q(z_T \mid \boldsymbol{x}) = \mathcal{N}(z_T \mid \mathbf{0}, \boldsymbol{I}), \tag{20.9}$$

因此, 关于原始图像的所有信息都丢失了. 在(20.3)中, 系数 $\sqrt{1-\beta_t}$ 和 $\sqrt{\beta_t}$ 确保了一旦 Markov 链收敛到均值为零、协方差为单位矩阵的分布后, 就不会再受进一步更新的影响了 (见练习 20.5). 由于(20.9)的右侧独立于 $\boldsymbol{x}$, 因此可以得出 z_T 的边缘分布:

$$q(z_T) = \mathcal{N}(z_T \mid \mathbf{0}, \boldsymbol{I}). \tag{20.10}$$

定义 20.2 (前向过程)

设噪声调度 $\beta_1, \cdots, \beta_T$ 满足 $0 < \beta_t < 1$. 由条件分布族(20.4)所确定的、以观测数据 $\boldsymbol{x}$ 为起点的 Markov 链

$$q(z_1, \cdots, z_T \mid \boldsymbol{x}) = q(z_1 \mid \boldsymbol{x}) \prod_{t=2}^{T} q(z_t \mid z_{t-1})$$

称为扩散模型的**前向过程** (forward process), 也称**前向编码器**. 其中 $\boldsymbol{x}$ 为观测变量,$z_1, \cdots, z_T$ 为潜变量. ♣

注

(重构新增) 前向过程在结构上类似于 VAE 中的编码器, 但有一个决定性的差别: 它没有任何可学习参数,β_t 与 $\boldsymbol{x}$ 都事先给定, 因此“编码”是纯随机的加噪, 而不是对后验的推断. 这一差别使扩散模型的训练目标里不存在编码器参数, 也使重参数化技巧变得不必要 (节 19.2中的 VAE 需要它, 是因为其编码器含参数).

术语上要小心一处冲突: 在标准化流的文献中, “前向过程”通常指从潜空间到数据空间的可逆映射 (即生成方向), 而这里的前向过程是从数据到噪声的破坏方向. 两者方向相反, 遇到跨文献的表述时必须按上下文确认, 不能把(20.4)的箭头反过来.

条件的意义:$0 < \beta_t < 1$ 保证每一步都是“部分保留信号 + 部分注入噪声”的压缩映射, 因而(20.7)中 $\sqrt{\alpha_t}$ 与 $\sqrt{1-\alpha_t}$ 同时为正;$\beta_1 < \beta_2 < \cdots < \beta_T$ 则保证噪声水平单调上升, 使链最终到达先验. 若调度不递增, 中间某些步骤可能“变清晰”, 扩散核的单调性被破坏, 后续的逆过程假设也随之失效.

20.1.2 条件分布

我们的目标是学会逆转加噪过程, 因此很自然地要考虑条件分布 $q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1})$ 的逆过程, 可以使用 Bayes 定理把它表示为

$$q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t) = \frac{q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1})\, q(\boldsymbol{z}_{t-1})}{q(\boldsymbol{z}_t)}. \tag{20.11}$$

我们可以将边缘分布 $q(\boldsymbol{z}_{t-1})$ 写成如下形式:

$$q(\boldsymbol{z}_{t-1}) = \int q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})\, p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}, \tag{20.12}$$

其中 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})$ 由条件 Gauss 分布(20.7)给出. 然而, 这个分布是难以计算的, 因为我们必须对未知的数据密度 $p(\boldsymbol{x})$ 进行积分. 如果我们使用训练集中的样本来近似地进行积分, 则会得到一个以 Gauss 函数混合形式表示的复杂分布.

换个思路, 考虑逆向分布的条件版本, 该分布以数据向量 $\boldsymbol{x}$ 为条件, 定义为 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})$, 这实际上是一个简单的 Gauss 分布. 从直观上看这是合理的, 因为我们很难猜测哪个低噪声图像演变为某个具体的加噪图像, 然而如果我们知道原始图像, 那么问题解决起来就会变得容易很多. 我们可以使用 Bayes 定理来计算这个条件分布:

$$q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) = \frac{q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x})\, q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})}{q(\boldsymbol{z}_t \mid \boldsymbol{x})}. \tag{20.13}$$

利用前向过程的 Markov 性质, 可以得到

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x}) = q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}). \tag{20.14}$$

(20.14)的右侧由(20.4)给出. 作为 $\boldsymbol{z}_{t-1}$ 的函数, 这里采用指数的二次型形式. 在(20.13)中, 分子中的 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})$ 是由(20.7)给出的扩散核, 这同样涉及关于 $\boldsymbol{z}_{t-1}$ 的指数的二次型形式. 我们可以忽略(20.13)中的分母, 因为作为 $\boldsymbol{z}_{t-1}$ 的函数, 它是一个常数.(20.13)右侧的形式是一个 Gauss 分布, 我们可以使用“配方法”这一技巧来确定其均值和协方差, 得到以下结论 (见练习 20.6).

命题 20.2 (逆条件分布的 Gauss 形式)

设前向过程由(20.4)给出,α_t 由(20.6)定义,$t \geqslant 2$. 则在给定 $\boldsymbol{x}$ 与 $\boldsymbol{z}_t$ 的条件下, 逆条件分布为 Gauss 分布

$$q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) = \mathcal{N}\big(\boldsymbol{z}_{t-1} \mid \boldsymbol{m}_t(\boldsymbol{x}, \boldsymbol{z}_t),\, \sigma_t^2 \boldsymbol{I}\big), \tag{20.15}$$

其均值和方差分别为

$$\boldsymbol{m}_t(\boldsymbol{x}, \boldsymbol{z}_t) = \frac{(1-\alpha_{t-1})\sqrt{1-\beta_t}\, \boldsymbol{z}_t + \sqrt{\alpha_{t-1}}\, \beta_t\, \boldsymbol{x}}{1-\alpha_t}, \tag{20.16}$$

$$\sigma_t^2 = \frac{\beta_t(1-\alpha_{t-1})}{1-\alpha_t}. \tag{20.17}$$

证明

(重构新增) 记 Z 为(20.13)右端作为 z_{t-1} 的函数所对应的归一化常数 (它不依赖 z_{t-1}). 由(20.13)、(20.14)、(20.4)与(20.7), 有

$$q(z_{t-1}\mid z_t,\boldsymbol{x}) = \frac{1}{Z}\exp\Big\{-\frac{\|z_t-\sqrt{1-\beta_t}\,z_{t-1}\|^2}{2\beta_t} - \frac{\|z_{t-1}-\sqrt{\alpha_{t-1}}\,\boldsymbol{x}\|^2}{2(1-\alpha_{t-1})}\Big\}.$$

把指数中的两个平方项展开, 只保留依赖 z_{t-1} 的部分 (其余并入常数), 得到

$$\ln q(z_{t-1}\mid z_t,\boldsymbol{x}) = -\frac{1}{2}\Big(\frac{1-\beta_t}{\beta_t}+\frac{1}{1-\alpha_{t-1}}\Big)\|z_{t-1}\|^2 + \Big(\frac{\sqrt{1-\beta_t}}{\beta_t}z_t + \frac{\sqrt{\alpha_{t-1}}}{1-\alpha_{t-1}}\boldsymbol{x}\Big)^{\mathsf{T}} z_{t-1} + \text{const.}$$

这是关于 z_{t-1} 的二次型, 故分布为 Gauss 分布. 由二次项系数可知其精度矩阵为

$$\frac{1-\beta_t}{\beta_t}+\frac{1}{1-\alpha_{t-1}} = \frac{(1-\beta_t)(1-\alpha_{t-1})+\beta_t}{\beta_t(1-\alpha_{t-1})} = \frac{1-\alpha_t}{\beta_t(1-\alpha_{t-1})},$$

其中最后一个等号用了 $(1-\beta_t)(1-\alpha_{t-1})+\beta_t = 1-(1-\beta_t)\alpha_{t-1} = 1-\alpha_t$; 求逆即得(20.17). 再令二次型对 z_{t-1} 的梯度为零, 得

$$\begin{aligned}\boldsymbol{m}_t(\boldsymbol{x},z_t) &= \sigma_t^2\Big(\frac{\sqrt{1-\beta_t}}{\beta_t}z_t + \frac{\sqrt{\alpha_{t-1}}}{1-\alpha_{t-1}}\boldsymbol{x}\Big)\\ &= \frac{\beta_t(1-\alpha_{t-1})}{1-\alpha_t}\cdot\frac{\sqrt{1-\beta_t}}{\beta_t}z_t + \frac{\beta_t(1-\alpha_{t-1})}{1-\alpha_t}\cdot\frac{\sqrt{\alpha_{t-1}}}{1-\alpha_{t-1}}\boldsymbol{x}\\ &= \frac{(1-\alpha_{t-1})\sqrt{1-\beta_t}}{1-\alpha_t}z_t + \frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t}\boldsymbol{x},\end{aligned}$$

这正是(20.16). 证毕.

□

注

(重构新增) (20.16)与(20.17)合起来说明: 只要把原始图像 $\boldsymbol{x}$ 也当作条件, 逆向的一步仍然是 Gauss 分布, 而且协方差是各向同性的 $\sigma_t^2\boldsymbol{I}$. 这解释了为什么后文可以用一个只输出均值、方差固定为 β_t 的网络来近似逆过程——逆分布本身的“形状”已经由前向调度决定, 网络要学的只是“均值往哪里走”. 均值(20.16)是 z_t 与 $\boldsymbol{x}$ 的加权平均, 两个权重之和为

$$\frac{(1-\alpha_{t-1})\sqrt{1-\beta_t}+\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t},$$

它一般不等于 1, 因此不能简单理解为“凸组合”; 两个权重分别衡量“当前噪声样本”和“原始图像”在估计 z_{t-1} 时的可信度. 当 t 较大 (α_{t-1} 很小) 时, $\boldsymbol{x}$ 的权重很小而 z_t 的权重接近 1, 这与“噪声大时几乎无法回推一步”的直觉一致.

条件 $t\geqslant 2$ 不能去掉: $t=1$ 时 $\alpha_0=1$, (20.17)的分母 $1-\alpha_1=\beta_1>0$ 尚可, 但(20.16)中 $\sqrt{\alpha_0}\,\beta_1/(1-\alpha_1)=1$ 会使公式退化为 $\boldsymbol{m}_1=\boldsymbol{x}$, 即“已知 $\boldsymbol{x}$ 时 z_0 就是 $\boldsymbol{x}$”, 这需要单独约定 $z_0\triangleq\boldsymbol{x}$. 实际算法对 $t=1$ 单独处理, 见节 20.2中的采样步骤.

20.2 反向解码器

我们已经知道, 前向编码器是由一系列 Gauss 条件分布 $q(z_t\mid z_{t-1})$ 定义的, 但直接逆转这一过程会导致一个难以处理的分布 $q(z_{t-1}\mid z_t)$, 因为需要对所有可能的起始向量 $\boldsymbol{x}$ 的值进行积分, 而 $\boldsymbol{x}$ 的分布就是我们希望建模的未知数据分布 $p(\boldsymbol{x})$. 为了解决这个问题, 我们将通过使用一个由深度神经网络控制的分布 $p(z_{t-1}\mid z_t,\boldsymbol{w})$ 来学习对逆向分布进行近似, 其中 $\boldsymbol{w}$ 表示网络的权重和偏置. 这个逆向步骤类似于变分自编码器中的解码器 (参见节 19.2), 可以通过图 20.2来理解. 一旦网络训练完成, 我们就可以从简单的 Gauss 分布 z_T 中进行采样, 并通过一系列逆向采样步骤将其变换成数据分布 $p(\boldsymbol{x})$ 的样本, 这些步骤是通过反复应用训练好的网络来完成的.

从直观上看, 如果我们保持方差较小, 使得 β_t 远小于 1, 那么潜向量之间的步骤变化将相对较小, 因此学会逆转变换应该更容易. 更具体地说, 如果 β_t 远小于 1, 那么分布 $q(z_{t-1}\mid z_t)$ 将近似于关于 z_{t-1} 的 Gauss 分布. 这一点从(20.11)中可以看出, 因为等式右边是通过 $q(z_t\mid z_{t-1})$ 和 $q(z_{t-1})$ 与 z_{t-1} 相关联的. 如果 $q(z_t\mid z_{t-1})$ 是一个极其

狭窄的 Gauss 分布, 那么在 $q(z_{t-1} \mid z_t)$ 具有显著概率质量的区域,$q(z_{t-1})$ 的变化将非常小, 因此 $q(z_{t-1} \mid z_t)$ 也可以近似认为是 Gauss 分布. 这种直觉可以通过图 20.3和图 20.4所示的简单例子来证实. 然而, 由于每一步的方差都很小, 我们必须使用大量的步骤来确保从前向加噪过程中获得的最终潜变量 z_T 的分布仍然接近 Gauss 分布, 这增加了生成新样本的成本. 在实践中,T 的值可能达到数千.

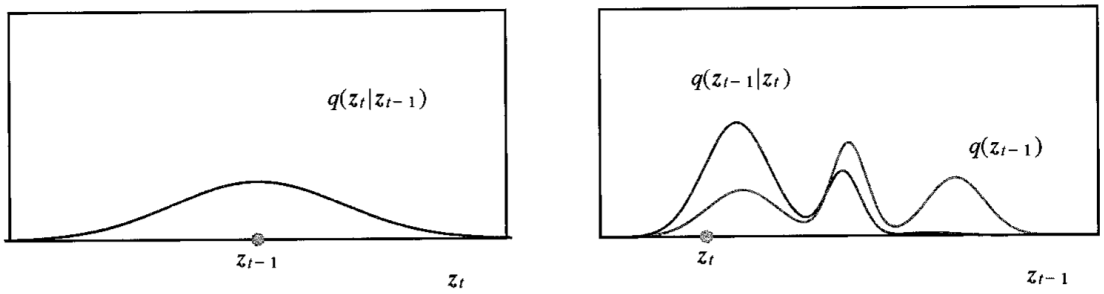


图 20.3: 用于标量变量的 Bayes 定理(20.11)的逆向分布 $q(z_{t-1} \mid z_t)$ 的计算示意图. 右图中的红色曲线展示了使用三个 Gauss 分布混合而成的边缘分布 $q(z_{t-1})$; 左图则展示了以 z_{t-1} 为中心的 Gauss 前向加噪过程 $q(z_t \mid z_{t-1})$. 通过将这些相乘并归一化, 我们得到了特定选择的 z_t 的分布 $q(z_{t-1} \mid z_t)$, 如蓝色曲线所示. 因为左侧的分布相对较宽, 对应于一个较大的方差 β_t, 所以分布 $q(z_{t-1} \mid z_t)$ 具有复杂的多模态结构

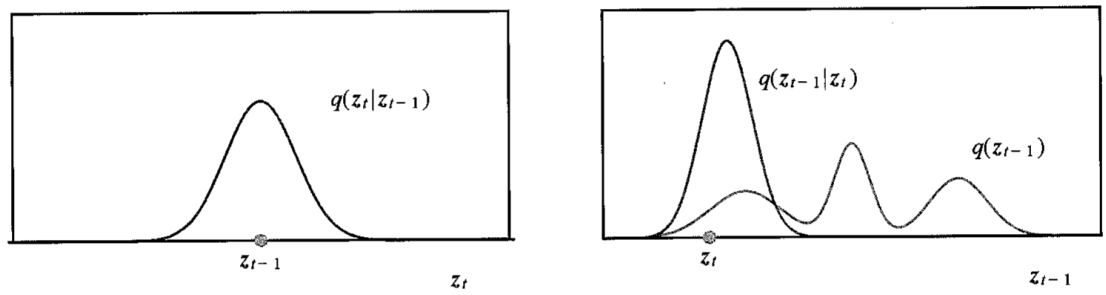


图 20.4: 和图 20.3一样, 但左图中的 Gauss 分布 $q(z_t \mid z_{t-1})$ 具有更小的方差 β_t. 右图中用蓝色显示的相应分布 $q(z_{t-1} \mid z_t)$ 接近于 Gauss 分布, 并且具有与 $q(z_t \mid z_{t-1})$ 类似的方差

例题 20.1 标量变量下的逆分布 取 $D = 1$, 令 $q(z_{t-1})$ 为三个 Gauss 分布混合而成的边缘密度 (图 20.3右图的黑色曲线). 前向核 $q(z_t \mid z_{t-1}) = \mathcal{N}\left(z_t \mid \sqrt{1-\beta_t}\, z_{t-1}, \beta_t \boldsymbol{I}\right)$ 在给定 z_t 时是 z_{t-1} 的 Gauss 函数 (左图). 由 Bayes 定理(20.11), 把二者相乘并归一化, 就得到 z_t 取定值时的逆分布 $q(z_{t-1} \mid z_t)$(右图蓝色曲线): 当 β_t 较大时它是多峰的 (图 20.3), 当 β_t 较小时它退化为单峰并近似为 Gauss 分布 (图 20.4).

注

(重构新增) 这个标量例子把“为什么逆分布近似 Gauss”变成了可视的事实: 逆分布的峰数由前向核的宽度与边缘分布峰间距之比决定.β_t 大时核很宽, 一个 z_t 会同时“看到”边缘分布的多个峰, 乘起来仍是多峰;β_t 小时核很窄, 只有最近的峰有显著权重, 逆分布就被该峰主导, 呈单峰且宽度与核相当.

要注意这是“近似”而不是恒等: 即使 β_t 很小, 逆分布仍带有边缘分布在局部的高阶曲率 (见练习 20.7的 $O(\beta_t^2)$ 修正), 只是这些修正随 $\beta_t \to 0$ 而消失.

例子也提示了后文把协方差固定为 $\beta_t \boldsymbol{I}$ 的合理性: 单峰近似下逆分布的宽度与 $\sqrt{\beta_t}$ 同阶, 恰是前向核的宽度.

我们可以更形象地看到, 通过围绕点 z_t 将 $\ln q(z_{t-1} \mid z_t)$ 对 z_{t-1} 进行 Taylor 级数展开,$q(z_{t-1} \mid z_t)$ 将近似为 Gauss 分布, 并且具有与 $q(z_t \mid z_{t-1})$ 类似的方差. 这也表明对于小方差, 逆向分布 $q(z_{t-1} \mid z_t)$ 的协方差将接近前向加噪过程 $q(z_t \mid z_{t-1})$ 的协方差 $\beta_t \boldsymbol{I}$. 因此, 我们可以使用以下形式的 Gauss 分布来模拟逆向过程 (见练习 20.7).

定义 20.3 (反向过程)

设 $\boldsymbol{\mu}(z_t, \boldsymbol{w}, t)$ 是由参数 $\boldsymbol{w}$ 控制的深度神经网络. 定义条件分布

$$p(z_{t-1} \mid z_t, \boldsymbol{w}) = \mathcal{N}\left(z_{t-1} \mid \boldsymbol{\mu}(z_t, \boldsymbol{w}, t), \beta_t \boldsymbol{I}\right), \tag{20.18}$$

并由它与先验 $p(\boldsymbol{z}_T)=\mathcal{N}(\boldsymbol{z}_T\mid\mathbf{0},\boldsymbol{I})$、观测模型 $p(\boldsymbol{x}\mid\boldsymbol{z}_1,\boldsymbol{w})$ 一起给出联合分布

$$p(\boldsymbol{x},\boldsymbol{z}_1,\cdots,\boldsymbol{z}_T\mid\boldsymbol{w})=p(\boldsymbol{z}_T)\Big\{\prod_{t=2}^{T}p(\boldsymbol{z}_{t-1}\mid\boldsymbol{z}_t,\boldsymbol{w})\Big\}p(\boldsymbol{x}\mid\boldsymbol{z}_1,\boldsymbol{w}). \tag{20.19}$$

称这一 Markov 链为扩散模型的**反向过程** (reverse process), 也称**反向解码器**.

♣

注

(重构新增) 定义中有三处设计需要解释. 第一, 协方差取成 $\beta_t\boldsymbol{I}$ 而不是可学习参数: 本节开头的 Taylor 论证表明, 当 β_t 很小时逆分布的协方差近似等于前向方差 $\beta_t\boldsymbol{I}$, 把它固定下来既减少了参数, 又保证训练目标里的 KL 散度有闭式表达.

第二, 均值网络显式接收步骤索引 t: 链上不同步骤的 β_t 不同, 同一个网络只有知道当前处于哪一步才能给出正确的去噪尺度; 这也使全部步骤可以共享一套参数, 而不必为每一步训练一个独立网络.

第三,$p(\boldsymbol{x}\mid\boldsymbol{z}_1,\boldsymbol{w})$ 被单列为因子, 而不是把 $t=1$ 也塞进连乘:$\boldsymbol{z}_0$ 被约定为 $\boldsymbol{x}$ 本身,$\boldsymbol{z}_1$ 到 $\boldsymbol{x}$ 的映射是“去噪的最后一步”, 其条件分布取 $\mathcal{N}(\boldsymbol{x}\mid\boldsymbol{\mu}(\boldsymbol{z}_1,\boldsymbol{w},1),\beta_1\boldsymbol{I})$. 这正是(20.20)中对 $\boldsymbol{z}_1,\cdots,\boldsymbol{z}_T$ 积分、而对 $\boldsymbol{x}$ 不再积分的原因.

条件方面: 网络输出维数必须与输入 $\boldsymbol{z}_t$ 相同 (否则(20.18)的均值无意义), 这是架构选择的硬约束; 卷积网络中的 U-Net 满足这一约束, 因而成为图像任务的常见选择. 若希望学习协方差, 可以让网络多输出一组分量 (Nichol and Dhariwal, 2021), 代价是 KL 散度不再是简单的平方误差.

其中,$\boldsymbol{\mu}(\boldsymbol{z}_t,\boldsymbol{w},t)$ 是由参数 $\boldsymbol{w}$ 控制的一个深度神经网络. 注意, 该深度神经网络显式地将步骤索引 t 作为输入, 以便考虑到链中不同步骤之间方差 β_t 的变化. 这也使得我们能够使用单个网络来逆转 Markov 链中的所有步骤, 而不必为每个步骤学习一个单独的网络. 此外, 还可以通过在网络中引入更多的输出来学习去噪过程的协方差 (Nichol and Dhariwal, 2021), 从而更好地描述 $\boldsymbol{z}_t$ 邻域内分布 $q(\boldsymbol{z}_{t-1}\mid\boldsymbol{z}_t)$ 的曲率. 用于建模 $\boldsymbol{\mu}(\boldsymbol{z}_t,\boldsymbol{w},t)$ 的神经网络在架构选择上有相当大的灵活性, 只要输出的维度与输入的维度相同即可 (参见小节 10.5.4). 鉴于这一限制,U-Net 架构是图像处理应用的常见选择.

整个逆向去噪过程采用了一种由(20.19)给出的 Markov 链形式, 即从先验 $p(\boldsymbol{z}_T)$ 出发, 依次经过 $p(\boldsymbol{z}_{T-1}\mid\boldsymbol{z}_T,\boldsymbol{w}),\cdots,p(\boldsymbol{z}_1\mid\boldsymbol{z}_2,\boldsymbol{w})$, 最后由 $p(\boldsymbol{x}\mid\boldsymbol{z}_1,\boldsymbol{w})$ 生成数据. 这里假设分布 $p(\boldsymbol{z}_T)$ 与 $q(\boldsymbol{z}_T)$ 相同, 因此也由 $\mathcal{N}(\boldsymbol{z}_T\mid\mathbf{0},\boldsymbol{I})$ 给出. 一旦模型训练好了, 采样就变得简单直接了, 因为我们可以首先从简单的 Gauss 分布 $p(\boldsymbol{z}_T)$ 中采样, 然后依次从每个条件分布 $p(\boldsymbol{z}_{t-1}\mid\boldsymbol{z}_t,\boldsymbol{w})$ 中采样, 最后从 $p(\boldsymbol{x}\mid\boldsymbol{z}_1,\boldsymbol{w})$ 中采样, 从而获得数据空间中的样本 $\boldsymbol{x}$.

20.2.1 训练解码器

接下来, 我们需要决定用来训练神经网络的目标函数. 显而易见的选择是似然函数, 对于数据点 $\boldsymbol{x}$, 似然函数由下式给出:

$$p(\boldsymbol{x}\mid\boldsymbol{w})=\int\cdots\int p(\boldsymbol{x},\boldsymbol{z}_1,\cdots,\boldsymbol{z}_T\mid\boldsymbol{w})\,\mathrm{d}\boldsymbol{z}_1\cdots\mathrm{d}\boldsymbol{z}_T, \tag{20.20}$$

其中 $p(\boldsymbol{x},\boldsymbol{z}_1,\cdots,\boldsymbol{z}_T\mid\boldsymbol{w})$ 由(20.19)定义. 这是一般潜变量模型 (见节 16.2) 的一个实例, 其中潜变量是 $\boldsymbol{z}=(\boldsymbol{z}_1,\cdots,\boldsymbol{z}_T)$, 观测变量是 $\boldsymbol{x}$. 注意, 所有的潜变量与数据空间具有相同的维度, 这与标准化流一样, 但不适用于变分自编码器或生成对抗网络.

从(20.20)中可以看到, 似然函数需要对所有可能生成观测数据点的噪声轨迹进行积分.(20.20)中的积分是难以计算的, 因为涉及对高度复杂的神经网络函数进行积分.

20.2.2 证据下界

鉴于精确的似然函数是难以计算的, 我们可以采取与变分自编码器相似的方法来最大化对数似然的下界, 这个下界称为**证据下界** (Evidence Lower BOund, ELBO). 对于任何选择的分布 $q(\boldsymbol{z})$, 证据下界与 KL 散度分别定义如下:

$$\mathcal{L}(\boldsymbol{w})=\int q(\boldsymbol{z})\ln\left\{\frac{p(\boldsymbol{x},\boldsymbol{z}\mid\boldsymbol{w})}{q(\boldsymbol{z})}\right\}\mathrm{d}\boldsymbol{z}, \tag{20.21}$$

$$\mathrm{KL}\left(f(z)\,\|\,g(z)\right) = -\int f(z)\ln\left\{\frac{g(z)}{f(z)}\right\}\mathrm{d}z, \tag{20.22}$$

其中 $\mathcal{L}$ 是证据下界, 也称变分下界 (variational lower bound),KL 是概率密度 $f(z)$ 和 $g(z)$ 之间的 KL 散度 (参见小节 2.5.5).

根据概率的乘积法则, 可得

$$p(\boldsymbol{x}, z \mid \boldsymbol{w}) = p(z \mid \boldsymbol{x}, \boldsymbol{w})\, p(\boldsymbol{x} \mid \boldsymbol{w}). \tag{20.23}$$

下面的命题说明, 证据下界与对数似然之差恰好是一个 KL 散度.

命题 20.3 (证据下界恒等式)

设 $q(z)$ 为任一概率密度, 且 $\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 有限. 则

$$\ln p(\boldsymbol{x} \mid \boldsymbol{w}) = \mathcal{L}(\boldsymbol{w}) + \mathrm{KL}\left(q(z)\,\|\,p(z \mid \boldsymbol{x}, \boldsymbol{w})\right). \tag{20.24}$$

♠

证明

(重构新增) 将(20.23)代入(20.21), 并利用 $\int q(z)\,\mathrm{d}z = 1$, 得

$$\begin{aligned}
\mathcal{L}(\boldsymbol{w}) &= \int q(z)\ln\left\{\frac{p(z \mid \boldsymbol{x}, \boldsymbol{w})\, p(\boldsymbol{x} \mid \boldsymbol{w})}{q(z)}\right\}\mathrm{d}z \\
&= \int q(z)\ln p(\boldsymbol{x} \mid \boldsymbol{w})\,\mathrm{d}z + \int q(z)\ln\left\{\frac{p(z \mid \boldsymbol{x}, \boldsymbol{w})}{q(z)}\right\}\mathrm{d}z \\
&= \ln p(\boldsymbol{x} \mid \boldsymbol{w}) - \mathrm{KL}\left(q(z)\,\|\,p(z \mid \boldsymbol{x}, \boldsymbol{w})\right),
\end{aligned}$$

其中最后一个等号用了(20.22). 移项即得(20.24). 证毕.

□

注

(重构新增) (20.24)说明 $\mathcal{L}(\boldsymbol{w})$ 与真实对数似然只差一个非负项, 因此最大化 $\mathcal{L}$ 就是在“把 KL 压到零”的同时抬高似然: 当且仅当 $q(z) = p(z \mid \boldsymbol{x}, \boldsymbol{w})$ 时二者相等. 这解释了 ELBO 中 q 的双重角色——它既是求期望的权重, 又是被优化的对象; 在 VAE 里 q 由编码器参数化, 而在扩散模型里 q 被固定为前向链.

两个条件不可省:$\ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 有限, 否则两边都可能是 $-\infty$;q 与 $p(z \mid \boldsymbol{x}, \boldsymbol{w})$ 关于同一测度绝对连续, 否则 KL 为 $+\infty$、恒等式失去意义.q 的支撑集若不覆盖后验的支撑集,KL 会发散, 这是实际使用中必须保证 $q > 0$(至少在后验为正处) 的原因.

还要注意这个恒等式本身不含任何近似: 它是概率乘法法则的直接改写. 扩散模型之所以“近似”, 是因为后面用一个受限的 Gauss 族 $p(z_{t-1} \mid z_t, \boldsymbol{w})$ 代替真实后验, 而不是因为(20.24)有误差.

由于 KL 散度满足 $\mathrm{KL}(\cdot\,\|\,\cdot) \geqslant 0$, 可以推导出

$$\ln p(\boldsymbol{x} \mid \boldsymbol{w}) \geqslant \mathcal{L}(\boldsymbol{w}). \tag{20.25}$$

因为对数似然函数是难以计算的, 所以我们通过最大化下界 $\mathcal{L}(\boldsymbol{w})$ 来训练神经网络.

为了做到这一点, 我们首先为扩散模型的下界推导出一种明确的形式. 在定义下界时, 我们可以自由选择任何形式的 $q(z)$, 只要它是一个有效的概率分布即可, 即它是非负的并且积分值为 1. 在 ELBO 的许多应用 (如变分自编码器) 中, 我们可以选择具有可调参数的 $q(z)$, 这些参数通常以深度神经网络的形式存在. 然后相对于这些参数以及分布 $p(\boldsymbol{x}, z \mid \boldsymbol{w})$ 的参数最大化 ELBO. 优化分布 $q(z)$ 使下界更接近真实值, 从而让 $p(\boldsymbol{x}, z \mid \boldsymbol{w})$ 中参数的优化更接近最大似然估计. 然而在扩散模型中,$q(z)$ 由固定分布 $q(z_1, \cdots, z_T \mid \boldsymbol{x})$ 给出, 该分布由 Markov 链(20.5)定义, 因此可调参数仅存在于用于逆 Markov 链的模型 $p(\boldsymbol{x}, z_1, \cdots, z_T \mid \boldsymbol{w})$ 中. 注意, 我们这里利用 $q(z)$ 在选择上的灵活性, 让它的形式与 $\boldsymbol{x}$ 相关.

因此, 我们可以使用(20.5)替换(20.24)中的 $q(z_1, \cdots, z_T \mid \boldsymbol{x})$. 同样, 我们也可以使用(20.19)替换 $p(\boldsymbol{x}, z_1, \cdots, z_T \mid \boldsymbol{w})$, 从而得到

$$\mathcal{L}(\boldsymbol{w}) = \mathbb{E}_q\left[\ln\frac{p(z_T)\left\{\prod_{t=2}^{T} p(z_{t-1} \mid z_t, \boldsymbol{w})\right\} p(\boldsymbol{x} \mid z_1, \boldsymbol{w})}{q(z_1 \mid \boldsymbol{x})\prod_{t=2}^{T} q(z_t \mid z_{t-1}, \boldsymbol{x})}\right]$$

$$= \mathbb{E}_q\Big[\ln p(\boldsymbol{z}_T) + \sum_{t=2}^{T} \ln \frac{p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})}{q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x})} - \ln q(\boldsymbol{z}_1 \mid \boldsymbol{x}) + \ln p(\boldsymbol{x} \mid \boldsymbol{z}_1, \boldsymbol{w})\Big], \tag{20.26}$$

其中

$$\mathbb{E}_q[\cdot] = \int \cdots \int q(\boldsymbol{z}_1 \mid \boldsymbol{x}) \prod_{t=2}^{T} q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x})\,[\cdot]\,\mathrm{d}\boldsymbol{z}_1 \cdots \mathrm{d}\boldsymbol{z}_T. \tag{20.27}$$

(20.26)右边的第 1 项 $\ln p(\boldsymbol{z}_T)$ 是固定分布 $\mathcal{N}(\boldsymbol{z}_T \mid \boldsymbol{0}, \boldsymbol{I})$, 它没有可训练的参数, 可以从 ELBO 中省略, 因为它代表一个固定的加性常数. 同样,(20.26)右边的第 3 项 $-\ln q(\boldsymbol{z}_1 \mid \boldsymbol{x})$ 与 $\boldsymbol{w}$ 无关, 所以也可以省略.(20.26)右边的第 4 项对应于变分自编码器中的重构项, 它可以从由(20.2)定义的 $\boldsymbol{z}_1$ 的分布中抽取样本来获得蒙特卡洛估计, 从而通过近似期望 $\mathbb{E}_q[\cdot]$ 来评估:

$$\mathbb{E}_q\big[\ln p(\boldsymbol{x} \mid \boldsymbol{z}_1, \boldsymbol{w})\big] \approx \frac{1}{L} \sum_{l=1}^{L} \ln p(\boldsymbol{x} \mid \boldsymbol{z}_1^{(l)}, \boldsymbol{w}), \tag{20.28}$$

其中 $\boldsymbol{z}_1^{(l)} \sim \mathcal{N}(\boldsymbol{z}_1 \mid \sqrt{1-\beta_1}\,\boldsymbol{x}, \beta_1 \boldsymbol{I})$. 与变分自编码器不同, 我们不需要通过抽样值反向传播误差信号, 因为 q 分布是固定的, 所以这里不需要使用重参数化技巧 (参见小节 19.2.2).

现在讨论(20.26)右边的第 2 项, 它包含了一系列子项, 其中每一个子项都依赖于相邻潜变量的值 $\boldsymbol{z}_{t-1}$ 和 $\boldsymbol{z}_t$. 当我们推导扩散核(20.7)时, 我们可以直接从 Gauss 分布 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})$ 中采样, 然后使用(20.4)得到相应的 $\boldsymbol{z}_t$ 的样本, 这也是一个 Gauss 分布. 虽然这在样本量无穷多的极限下是正确的, 但使用成对的采样值会产生非常嘈杂且方差很高的估计, 因此需要大量不必要的样本. 换个思路, 我们也可以重写 ELBO, 使其可以通过对每一项只采样一个值来进行估计.

20.2.3 重写 ELBO

在讨论了变分自编码器的 ELBO 之后, 我们的目标是用 KL 散度来表达 ELBO, 随后可以将其表示为闭式表达式. 神经网络建模的是逆向过程中的分布 $p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})$, 而 q 分布是定义在前向过程 $q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x})$ 上, 因此我们使用 Bayes 定理来反转条件分布:

$$q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x}) = \frac{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})\, q(\boldsymbol{z}_t \mid \boldsymbol{x})}{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})}. \tag{20.29}$$

这样我们就可以把(20.26)中的第二项写为

$$\ln \frac{p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})}{q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x})} = \ln \frac{p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})}{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})} + \ln \frac{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})}{q(\boldsymbol{z}_t \mid \boldsymbol{x})}. \tag{20.30}$$

(20.30)右边的第二项与 $\boldsymbol{w}$ 无关, 因此可以省略. 将(20.30)代入(20.26), 可以得到

$$\mathcal{L}(\boldsymbol{w}) = \mathbb{E}_q\Big[\sum_{t=2}^{T} \ln \frac{p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})}{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})} + \ln p(\boldsymbol{x} \mid \boldsymbol{z}_1, \boldsymbol{w})\Big]. \tag{20.31}$$

定理 20.1 (ELBO 的三重分解)

设前向过程、反向过程与 $\mathcal{L}(\boldsymbol{w})$ 分别由(20.5)、(20.19)与(20.21)给出,$q(\boldsymbol{z}) = q(\boldsymbol{z}_1, \cdots, \boldsymbol{z}_T \mid \boldsymbol{x})$. 则

$$\begin{aligned}\mathcal{L}(\boldsymbol{w}) = {} & \mathbb{E}_{q(\boldsymbol{z}_1 \mid \boldsymbol{x})}\big[\ln p(\boldsymbol{x} \mid \boldsymbol{z}_1, \boldsymbol{w})\big] \\ & - \mathrm{KL}\left(q(\boldsymbol{z}_T \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_T)\right) - \sum_{t=2}^{T} \mathbb{E}_{q(\boldsymbol{z}_t \mid \boldsymbol{x})}\Big[\mathrm{KL}\left(q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})\right)\Big],\end{aligned} \tag{20.32}$$

即 ELBO 等于“重构项”减去“先验 KL 项”, 再减去 $T-1$ 个“一致项”.

证明

(重构新增) 从(20.31)出发. 先处理其中的求和项. 对固定 t, 由 $q(\boldsymbol{z}_{t-1}, \boldsymbol{z}_t \mid \boldsymbol{x}) = q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) q(\boldsymbol{z}_t \mid \boldsymbol{x})$ 以及(20.27)中其余条件分布的积分均为 1, 有

$$\mathbb{E}_q\Big[\ln \frac{p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})}{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})}\Big] = \int q(\boldsymbol{z}_t \mid \boldsymbol{x}) \Big\{ \int q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) \ln \frac{p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})}{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})} \mathrm{d}\boldsymbol{z}_{t-1} \Big\} \mathrm{d}\boldsymbol{z}_t,$$

内层积分正是 $-\mathrm{KL}\left(q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})\right)$, 故

$$\sum_{t=2}^{T} \mathbb{E}_q\Big[\ln \frac{p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})}{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})}\Big] = -\sum_{t=2}^{T} \mathbb{E}_{q(\boldsymbol{z}_t \mid \boldsymbol{x})}\Big[\mathrm{KL}\left(q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})\right)\Big].$$

再处理被省略的两项. 回到(20.26), 由(20.29),$\ln q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1}, \boldsymbol{x}) = \ln q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) + \ln q(\boldsymbol{z}_t \mid \boldsymbol{x}) - \ln q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})$, 于是

$$\begin{aligned} &\mathbb{E}_q\Big[\ln p(\boldsymbol{z}_T) - \ln q(\boldsymbol{z}_1 \mid \boldsymbol{x}) + \sum_{t=2}^{T} \ln \frac{q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x})}{q(\boldsymbol{z}_t \mid \boldsymbol{x})}\Big] \\ &\quad = \mathbb{E}_q\Big[\ln p(\boldsymbol{z}_T) - \ln q(\boldsymbol{z}_T \mid \boldsymbol{x})\Big] = -\mathrm{KL}\left(q(\boldsymbol{z}_T \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_T)\right), \end{aligned}$$

其中第一个等号用了望远镜求和 $\sum_{t=2}^{T}\left[\ln q(\boldsymbol{z}_{t-1} \mid \boldsymbol{x}) - \ln q(\boldsymbol{z}_t \mid \boldsymbol{x})\right] = \ln q(\boldsymbol{z}_1 \mid \boldsymbol{x}) - \ln q(\boldsymbol{z}_T \mid \boldsymbol{x})$, 它与前面的 $-\ln q(\boldsymbol{z}_1 \mid \boldsymbol{x})$ 相消; 第二个等号用了 KL 的定义(20.22). 最后, 重构项只依赖 $\boldsymbol{z}_1$, 故 $\mathbb{E}_q[\ln p(\boldsymbol{x} \mid \boldsymbol{z}_1, \boldsymbol{w})] = \mathbb{E}_{q(\boldsymbol{z}_1 \mid \boldsymbol{x})}[\ln p(\boldsymbol{x} \mid \boldsymbol{z}_1, \boldsymbol{w})]$. 合并三部分即得(20.32). 证毕.

□

注

(重构新增) (20.32)把扩散模型的训练目标拆成了三块可以分别处理的部分: 重构项要求由 $\boldsymbol{z}_1$ 还原 $\boldsymbol{x}$ 的概率大; 先验 KL 项衡量“前向链终点”与“先验”的差距; 一致项则逐对比较“真实的后验一步”与“网络近似的一步”. 三项都在 Gauss 分布之间计算, 因此都能写成闭式或平方误差 (命题 20.4).

这里有一个容易被忽略的细节: 先验 KL 项中的 $q(\boldsymbol{z}_T \mid \boldsymbol{x})$ 依赖 $\boldsymbol{x}$, 而 $p(\boldsymbol{z}_T)$ 不依赖, 所以在有限 T 下它一般不为零; 加噪过程被设计成使 $\boldsymbol{z}_T$ 与 $\boldsymbol{x}$ 几乎独立, 因此当 $\alpha_T \approx 0$ 时它趋于零 (练习 20.10). 在这个意义下把该项并入常数是合理的; 若 α_T 不可忽略, 删掉它会引入 $O(\alpha_T)$ 的偏差.

定理对任意噪声调度都成立, 只要求各条件分布有定义、KL 有限. 它也是后面把“预测 $\boldsymbol{x}$”换成“预测噪声”的出发点: 一致项的形状决定了哪种参数化能让损失变成简单的平方误差.

由于加噪过程使 $q(\boldsymbol{z}_T \mid \boldsymbol{x}) = \mathcal{N}(\boldsymbol{z}_T \mid \boldsymbol{0}, \boldsymbol{I})$ 与 $p(\boldsymbol{z}_T)$ 相同,(20.32)中的先验 KL 项为零, 下界可以写成更常见的形式 (见练习 20.9):

$$\begin{aligned} \mathcal{L}(\boldsymbol{w}) = &\underbrace{\int q(\boldsymbol{z}_1 \mid \boldsymbol{x}) \ln p(\boldsymbol{x} \mid \boldsymbol{z}_1, \boldsymbol{w}) \mathrm{d}\boldsymbol{z}_1}_{\text{重建项}} \\ &\underbrace{- \sum_{t=2}^{T} \int \mathrm{KL}\left(q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})\right) q(\boldsymbol{z}_t \mid \boldsymbol{x}) \mathrm{d}\boldsymbol{z}_t}_{\text{一致项}} . \end{aligned} \tag{20.33}$$

在(20.33)右边的第一项中,$q(\boldsymbol{z}_1, \cdots, \boldsymbol{z}_T \mid \boldsymbol{x})$ 的期望被简化了, 这是因为 $\boldsymbol{z}_1$ 是积分符号内出现的唯一潜变量. 在(20.27)定义的期望中, 所有条件分布的积分值都等于 1, 只留下对 $\boldsymbol{z}_1$ 的积分. 同样, 在(20.33)右边的第二项中, 每个积分仅涉及两个相邻的潜变量 $\boldsymbol{z}_{t-1}$ 和 $\boldsymbol{z}_t$, 所有其余变量都可以通过积分消去.

现在的下界(20.33)与节 19.2中给出的变分自编码器的 ELBO 非常类似, 区别在于这里存在多个编码和解码阶段. 重构项通过提高观测数据样本的概率来优化模型, 并且可以采用与 VAE 中的相应项相同的方式, 通过采样近似(20.28)来训练. 与变分自编码器不同, 我们不需要通过抽样值反向传播误差信号, 因为 q 分布是固定的, 所以这里不需要使用重参数化技巧 (参见小节 19.2.2).

(20.33)中的一致项是在 Gauss 分布对之间定义的, 因此可以用解析形式来表达. 分布 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})$ 由(20.15)给出, 而分布 $p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})$ 由(20.18)给出, 因此 KL 散度变成以下形式 (见练习 20.11).

命题 20.4 (一致项的闭式表达) 设 $q(z_{t-1} \mid z_t, \boldsymbol{x}) = \mathcal{N}\big(z_{t-1} \mid \boldsymbol{m}_t(\boldsymbol{x}, z_t), \sigma_t^2 \boldsymbol{I}\big)$ 由(20.15)给出,$p(z_{t-1} \mid z_t, \boldsymbol{w}) = \mathcal{N}\big(z_{t-1} \mid \boldsymbol{\mu}(z_t, \boldsymbol{w}, t), \beta_t \boldsymbol{I}\big)$ 由(20.18)给出. 则

$$\mathrm{KL}\left(q(z_{t-1} \mid z_t, \boldsymbol{x}) \,\|\, p(z_{t-1} \mid z_t, \boldsymbol{w})\right) = \frac{1}{2\beta_t}\left\|\boldsymbol{m}_t(\boldsymbol{x}, z_t) - \boldsymbol{\mu}(z_t, \boldsymbol{w}, t)\right\|^2 + \text{const}, \tag{20.34}$$

其中 $\boldsymbol{m}_t(\boldsymbol{x}, z_t)$ 由(20.16)定义, 常数 const 与网络参数 $\boldsymbol{w}$ 无关.

证明

(重构新增) 两个 Gauss 分布 $\mathcal{N}(\boldsymbol{m}_1, \boldsymbol{\Sigma}_1)$ 与 $\mathcal{N}(\boldsymbol{m}_2, \boldsymbol{\Sigma}_2)$ 的 KL 散度为

$$\mathrm{KL} = \frac{1}{2}\Big[\operatorname{tr}\left(\boldsymbol{\Sigma}_2^{-1}\boldsymbol{\Sigma}_1\right) + (\boldsymbol{m}_2 - \boldsymbol{m}_1)^{\mathsf{T}}\boldsymbol{\Sigma}_2^{-1}(\boldsymbol{m}_2 - \boldsymbol{m}_1) - D + \ln\frac{\det\boldsymbol{\Sigma}_2}{\det\boldsymbol{\Sigma}_1}\Big],$$

其中 D 是维数. 取 $\boldsymbol{m}_1 = \boldsymbol{m}_t$、$\boldsymbol{\Sigma}_1 = \sigma_t^2\boldsymbol{I}$、$\boldsymbol{m}_2 = \boldsymbol{\mu}$、$\boldsymbol{\Sigma}_2 = \beta_t\boldsymbol{I}$, 则迹项为 $D\sigma_t^2/\beta_t$, 行列式项为 $D\ln(\beta_t/\sigma_t^2)$, 二者都与 $\boldsymbol{w}$ 无关, 可以并入常数; 剩下的二次项为

$$\frac{1}{2}(\boldsymbol{\mu} - \boldsymbol{m}_t)^{\mathsf{T}}(\beta_t\boldsymbol{I})^{-1}(\boldsymbol{\mu} - \boldsymbol{m}_t) = \frac{1}{2\beta_t}\|\boldsymbol{m}_t - \boldsymbol{\mu}\|^2,$$

这正是(20.34). 证毕.

□

注

(重构新增) (20.34)说明一致项对网络参数的依赖只通过 $\|\boldsymbol{m}_t - \boldsymbol{\mu}\|^2$ 出现, 因此“最小化 KL”与“最小化均值的平方误差”完全等价. 这是扩散模型能用简单回归损失训练的根本原因, 也解释了为什么把协方差固定为 $\beta_t\boldsymbol{I}$ 是有利的: 一旦两个协方差不同,KL 中就会出现依赖 $\boldsymbol{w}$ 的迹项与行列式项, 损失不再是纯平方误差.

const 中仍含有 σ_t^2, 而 σ_t^2 依赖噪声调度; 由于它对 $\boldsymbol{w}$ 是常数, 在求梯度时消失, 但在比较不同 t 的损失量级时不可忽略——这正是后文要省略前置因子的原因. 条件 $t \geqslant 2$ 沿用命题 20.2的约定, 对 $t = 1$ 需单独讨论(见(20.41)).

其中 $\boldsymbol{m}_t(\boldsymbol{x}, z_t)$ 由(20.16)定义, 并且任何与网络参数 $\boldsymbol{w}$ 无关的加性项都已经被吸收到常数项 const 中, 在训练中不起作用.(20.33)中的每个一致项对 z_t 的剩余积分 (由 $q(z_t \mid \boldsymbol{x})$ 加权), 可以通过从 $q(z_t \mid \boldsymbol{x})$ 中采样来近似, 这可以通过扩散核(20.7)高效地完成.

观察(20.34)可知,KL 散度表现为简单的平方损失函数的形式. 既然我们通过调整网络参数 $\boldsymbol{w}$ 来最大化(20.33)中的下界, 而 ELBO 中 KL 散度项的前面带有负号, 因此这等价于最小化这个平方误差.

命题 20.5 (最优反向均值) 在命题 20.4的设定下, 对固定的 z_t, 使 $\mathrm{KL}\left(q(z_{t-1} \mid z_t, \boldsymbol{x}) \,\|\, p(z_{t-1} \mid z_t, \boldsymbol{w})\right)$ 关于 $\boldsymbol{\mu}(z_t, \boldsymbol{w}, t)$ 取最小值的解为

$$\boldsymbol{\mu}^\star(z_t, \boldsymbol{w}, t) = \boldsymbol{m}_t(\boldsymbol{x}, z_t), \tag{20.35}$$

最小值为 0; 当协方差固定为 $\beta_t\boldsymbol{I}$ 时, 这也是唯一的最小点.

证明

(重构新增) 由(20.34),KL 关于 $\boldsymbol{\mu}$ 的部分是 $\frac{1}{2\beta_t}\|\boldsymbol{m}_t - \boldsymbol{\mu}\|^2$ 加上与 $\boldsymbol{\mu}$ 无关的常数. 因 $\beta_t > 0$, 该二次函数在 $\boldsymbol{\mu} = \boldsymbol{m}_t$ 处取到最小值 0, 且由正定性知其唯一. 证毕.

□

注

(重构新增) 这条命题给出了“网络究竟该学什么”的答案: 给定 z_t 与 $\boldsymbol{x}$, 最优的反向均值就是真实后验的均值 $\boldsymbol{m}_t$. 它同时说明, 用固定方差 $\beta_t\boldsymbol{I}$ 的 Gauss 族去近似真实后验时,$q(z_{t-1} \mid z_t, \boldsymbol{x})$ 本身就在这个族内 (命题 20.2), 因此该族的近似误差只来自“网络无法精确表示 $\boldsymbol{m}_t$”, 而不来自分布族的选择.

要强调的是,(20.35)中的 $\boldsymbol{m}_t$ 依赖未知的 $\boldsymbol{x}$, 实际训练时无法直接计算, 所以才需要把它改写成只依赖可观测量 z_t 与噪声 $\boldsymbol{\varepsilon}_t$ 的形式 (命题 20.6). 命题只断言“对固定的 z_t 逐点最优”, 并没有说逐点最优的网络能在所有 z_t 上同时达到最优; 这需要网络容量足够, 属于逼近问题而非优化问题.

20.2.4 预测噪声

为了获得更高质量的结果, 我们可以改变神经网络的作用, 使其不是在 Markov 链的每一步都预测去噪后的图像, 而是预测添加到原始图像上以创建该步骤中有噪声图像的总噪声成分 (Ho, Jain and Abbeel, 2020). 对(20.8)重新整理后, 可以得到

$$\boldsymbol{x} = \frac{1}{\sqrt{\alpha_t}} z_t - \frac{\sqrt{1-\alpha_t}}{\sqrt{\alpha_t}} \boldsymbol{\varepsilon}_t. \tag{20.36}$$

将(20.36)代入(20.16), 即可用原始数据向量 $\boldsymbol{x}$ 和噪声 $\boldsymbol{\varepsilon}$ 重写逆条件分布 $q(z_{t-1} \mid z_t, \boldsymbol{x})$ 的均值 $\boldsymbol{m}_t(\boldsymbol{x}, z_t)$, 得到 (见练习 20.12)

$$\boldsymbol{m}_t(\boldsymbol{x}, z_t) = \frac{1}{\sqrt{1-\beta_t}} \left\{ z_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \boldsymbol{\varepsilon}_t \right\}. \tag{20.37}$$

类似地, 我们不再使用神经网络 $\boldsymbol{\mu}(z_t, \boldsymbol{w}, t)$ 来预测去噪后的图像, 而是引入旨在预测添加到 $\boldsymbol{x}$ 上以生成 z_t 的总噪声的神经网络 $\boldsymbol{g}(z_t, \boldsymbol{w}, t)$. 按照推导(20.37)的相同步骤, 我们看到, 这两个神经网络之间的关系如下:

$$\boldsymbol{\mu}(z_t, \boldsymbol{w}, t) = \frac{1}{\sqrt{1-\beta_t}} \left\{ z_t - \frac{\beta_t}{\sqrt{1-\alpha_t}} \boldsymbol{g}(z_t, \boldsymbol{w}, t) \right\}. \tag{20.38}$$

将(20.37)和(20.38)代入(20.34), 可以得到如下结论 (见练习 20.13).

命题 20.6 (噪声预测损失)

设 $\boldsymbol{m}_t$ 与 $\boldsymbol{\mu}$ 分别由(20.37)与(20.38)给出. 则一致项可以写成

$$\begin{aligned} \mathrm{KL}\left(q(z_{t-1} \mid z_t, \boldsymbol{x}) \,\|\, p(z_{t-1} \mid z_t, \boldsymbol{w})\right) &= \frac{\beta_t}{2(1-\alpha_t)(1-\beta_t)} \left\| \boldsymbol{g}(z_t, \boldsymbol{w}, t) - \boldsymbol{\varepsilon}_t \right\|^2 + \text{const} \\ &= \frac{\beta_t}{2(1-\alpha_t)(1-\beta_t)} \left\| \boldsymbol{g}\left(\sqrt{\alpha_t}\,\boldsymbol{x} + \sqrt{1-\alpha_t}\,\boldsymbol{\varepsilon}_t, \boldsymbol{w}, t\right) - \boldsymbol{\varepsilon}_t \right\|^2 + \text{const}. \end{aligned} \tag{20.39}$$

相应地, 重构项可以写成

$$\ln p(\boldsymbol{x} \mid z_1, \boldsymbol{w}) = -\frac{1}{2\beta_1} \left\| \boldsymbol{x} - \boldsymbol{\mu}(z_1, \boldsymbol{w}, 1) \right\|^2 + \text{const}, \tag{20.40}$$

$$= -\frac{1}{2(1-\beta_1)} \left\| \boldsymbol{g}(z_1, \boldsymbol{w}, 1) - \boldsymbol{\varepsilon}_1 \right\|^2 + \text{const}, \tag{20.41}$$

其中 $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I})$, 且(20.39)的第二行用了(20.8)替换 z_t.

♠

证明

(重构新增) 由(20.37)与(20.38),

$$\boldsymbol{m}_t - \boldsymbol{\mu} = \frac{1}{\sqrt{1-\beta_t}} \cdot \frac{\beta_t}{\sqrt{1-\alpha_t}} \left(\boldsymbol{g}(z_t, \boldsymbol{w}, t) - \boldsymbol{\varepsilon}_t \right).$$

代入(20.34)得

$$\mathrm{KL} = \frac{1}{2\beta_t} \cdot \frac{\beta_t^2}{(1-\beta_t)(1-\alpha_t)} \|\boldsymbol{g} - \boldsymbol{\varepsilon}_t\|^2 + \text{const} = \frac{\beta_t}{2(1-\alpha_t)(1-\beta_t)} \|\boldsymbol{g} - \boldsymbol{\varepsilon}_t\|^2 + \text{const},$$

再把 $z_t = \sqrt{\alpha_t}\,\boldsymbol{x} + \sqrt{1-\alpha_t}\,\boldsymbol{\varepsilon}_t$ 代入 $\boldsymbol{g}$ 的自变量即得(20.39). 至于重构项, 由(20.18),$p(\boldsymbol{x} \mid z_1, \boldsymbol{w}) = \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}(z_1, \boldsymbol{w}, 1), \beta_1 \boldsymbol{I})$, 取对数即得(20.40). 在(20.40)中用(20.38)(取 $t=1$) 替换 $\boldsymbol{\mu}$、用(20.2)替换 $\boldsymbol{x}$, 并利用 $\alpha_1 = 1-\beta_1$, 得

$$\boldsymbol{x} - \boldsymbol{\mu}(z_1, \boldsymbol{w}, 1) = \frac{1}{\sqrt{1-\beta_1}} \cdot \frac{\beta_1}{\sqrt{1-\alpha_1}} \left(\boldsymbol{g}(z_1, \boldsymbol{w}, 1) - \boldsymbol{\varepsilon}_1 \right) = \sqrt{\frac{\beta_1}{1-\beta_1}} \left(\boldsymbol{g}(z_1, \boldsymbol{w}, 1) - \boldsymbol{\varepsilon}_1 \right),$$

其中用了 $\sqrt{1-\alpha_1} = \sqrt{\beta_1}$. 代回(20.40)即得(20.41). 证毕.

□

注

(重构新增) (20.39)与(20.41)说明: 在“预测总噪声”的参数化下,ELBO 的每一项都变成“网络输出与真实噪声之差”的平方, 只是前置系数不同. 重构项正好是 $t=1$ 时(20.39)的特例 (把 $\beta_1/(2(1-\alpha_1)(1-\beta_1))$ 在 $\alpha_1=1-\beta_1$ 下化简即得 $1/(2(1-\beta_1))$), 因此重构项与一致项可以合并成对 $t=1,\cdots,T$ 的统一求和.

条件方面: 推导假设 $\boldsymbol{\varepsilon}_t$ 与 $\boldsymbol{x}$ 独立、$\alpha_t<1$(否则 $\sqrt{1-\alpha_t}=0$ 使第二行退化) 以及 $\beta_t<1$. 前置系数 $\beta_t/(2(1-\alpha_t)(1-\beta_t))$ 依赖 t, 在 t 较小时很小、较大时变大, 因此原始 ELBO 会隐式地给不同步骤不同的权重; 这正是下一段“省略系数”做法的由来.

还要注意 $\boldsymbol{g}$ 预测的是“从 $\boldsymbol{x}$ 到 $\boldsymbol{z}_t$ 的累计噪声”, 只有在 $t=1$ 时它才等于第 1 步新加的噪声 $\boldsymbol{\varepsilon}_1$.

这正好与(20.39)在特殊情况 $t=1$ 时的形式相同, 因此重构项和一致项可以合并起来. 有研究 (Ho, Jain, and Abbeel, 2020) 发现, 简单地省略掉(20.39)前面的因子 $\beta_t/\big(2(1-\alpha_t)(1-\beta_t)\big)$,Markov 链中的所有步骤就有了相等的权重, 这可以进一步提高性能. 将简化后的(20.39)代入(20.33), 可以得到

$$\mathcal{L}(\boldsymbol{w})=-\sum_{t=1}^{T}\left\|\boldsymbol{g}\left(\sqrt{\alpha_t}\,\boldsymbol{x}+\sqrt{1-\alpha_t}\,\boldsymbol{\varepsilon}_t,\boldsymbol{w},t\right)-\boldsymbol{\varepsilon}_t\right\|^2, \tag{20.42}$$

其中省略了与 $\boldsymbol{w}$ 无关的常数因子.

(20.42)右侧的平方误差有一个非常简明的解释: 对于 Markov 链中给定的步骤 t 和给定的训练数据点 $\boldsymbol{x}$, 抽取一个噪声向量 $\boldsymbol{\varepsilon}_t$, 并使用这个噪声向量来创建对应步骤 t 的噪声潜变量 $\boldsymbol{z}_t$. 然后损失函数就是预测的噪声和实际噪声的平方差. 注意神经网络 $\boldsymbol{g}(\cdot)$ 预测的是添加到原始数据向量 $\boldsymbol{x}$ 上的总噪声, 而不仅仅是步骤 t 中增加的增量噪声.

当使用随机梯度下降时, 针对从训练集中随机选取的数据点 $\boldsymbol{x}$, 计算损失函数对网络参数的梯度向量. 另外, 对于每个这样的数据点, 随机选择沿着 Markov 链的一个步骤 t, 而不是计算(20.42)中对 t 求和的每一项的误差. 这些梯度在小批量数据样本上累积后, 用来更新网络权重.

另外, 这个损失函数自动地内置了一种数据增强机制, 因为每当特定的训练样本 $\boldsymbol{x}$ 被使用时, 它都会与一个新的噪声样本 $\boldsymbol{\varepsilon}_t$ 相结合. 以上所有的推导都针对来自训练集的单个数据点 $\boldsymbol{x}$. 算法 20.1 展示了相应的梯度计算过程.

算法 20.1: 训练一个去噪扩散概率模型

输入: 训练数据 $\mathcal{D}=\{\boldsymbol{x}_n\}$; 噪声调度 $\{\beta_1,\cdots,\beta_T\}$
输出: 网络参数 $\boldsymbol{w}$

for $t\in\{1,\cdots,T\}$ **do**
  $\alpha_t\leftarrow\prod_{\tau=1}^{t}(1-\beta_\tau)$ // 从 β 计算 α
end for
repeat
  $\boldsymbol{x}\sim\mathcal{D}$ // 采样一个数据点
  $t\sim\{1,\cdots,T\}$ // 沿着 Markov 链采样一个数据点
  $\boldsymbol{\varepsilon}\sim\mathcal{N}(\boldsymbol{\varepsilon}\mid\boldsymbol{0},\boldsymbol{I})$ // 采样一个噪声向量
  $\boldsymbol{z}_t\leftarrow\sqrt{\alpha_t}\,\boldsymbol{x}+\sqrt{1-\alpha_t}\,\boldsymbol{\varepsilon}$ // 评估噪声潜变量
  $\mathcal{L}(\boldsymbol{w})\leftarrow\|\boldsymbol{g}(\boldsymbol{z}_t,\boldsymbol{w},t)-\boldsymbol{\varepsilon}\|^2$ // 计算损失项
  Take optimizer step
until converged
return $\boldsymbol{w}$

20.2.5 生成新的样本

一旦网络训练好了, 我们就可以通过以下步骤生成数据空间中的新样本: 首先从 Gauss 分布 $p(\boldsymbol{z}_T)$ 中采样, 然后逐步通过 Markov 链的每一步去噪. 给定步骤 t 的去噪样本 $\boldsymbol{z}_t$, 我们可以通过三个步骤生成样本 $\boldsymbol{z}_{t-1}$. 首先计算由 $\boldsymbol{g}(\boldsymbol{z}_t,\boldsymbol{w},t)$ 给出的神经网络输出. 根据这个输出, 使用(20.38)计算均值 $\boldsymbol{\mu}(\boldsymbol{z}_t,\boldsymbol{w},t)$. 接下来, 通过在 $p(\boldsymbol{z}_{t-1}\mid\boldsymbol{z}_t,\boldsymbol{w})=\mathcal{N}(\boldsymbol{z}_{t-1}\mid\boldsymbol{\mu}(\boldsymbol{z}_t,\boldsymbol{w},t),\beta_t\boldsymbol{I})$ 中添加经过方差缩放的噪声来生成样本 $\boldsymbol{z}_{t-1}$.

命题 20.7 (采样递推)

设 $\boldsymbol{\mu}(\boldsymbol{z}_t, \boldsymbol{w}, t)$ 由(20.38)给出. 若 $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{\varepsilon} \mid \mathbf{0}, \boldsymbol{I})$ 与 $\boldsymbol{z}_t$ 独立, 则由

$$\boldsymbol{z}_{t-1} = \boldsymbol{\mu}(\boldsymbol{z}_t, \boldsymbol{w}, t) + \sqrt{\beta_t}\,\boldsymbol{\varepsilon} \tag{20.43}$$

定义的 $\boldsymbol{z}_{t-1}$ 服从 $p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w}) = \mathcal{N}(\boldsymbol{z}_{t-1} \mid \boldsymbol{\mu}(\boldsymbol{z}_t, \boldsymbol{w}, t), \beta_t \boldsymbol{I})$.

♠

证明

(重构新增) $\boldsymbol{\mu}$ 在给定 $\boldsymbol{z}_t$ 时是常数向量, 而 $\sqrt{\beta_t}\,\boldsymbol{\varepsilon}$ 是均值为 $\mathbf{0}$、协方差为 $\beta_t \boldsymbol{I}$ 的 Gauss 向量, 且与 $\boldsymbol{z}_t$ 独立. 常数平移不改变协方差、只把均值平移, 故 $\boldsymbol{z}_{t-1}$ 的条件分布为 $\mathcal{N}(\boldsymbol{\mu}, \beta_t \boldsymbol{I})$. 证毕.

□

注

(重构新增) (20.43)就是“祖先采样” (ancestral sampling) 的一步: 从链的末端开始, 每次只用一个已经训练好的网络和一次 Gauss 采样, 就能前进到前一个潜变量. 它的正确性只依赖定义 20.3中“协方差固定为 $\beta_t \boldsymbol{I}$”这一约定; 若网络还输出协方差, 采样时就要用该协方差而不是 $\beta_t \boldsymbol{I}$.

注意 $\boldsymbol{\varepsilon}$ 必须与 $\boldsymbol{z}_t$ 独立地重新抽取, 不能复用训练时的噪声, 否则样本之间会产生虚假的相关性. 递推的初始分布必须精确取为 $p(\boldsymbol{z}_T) = \mathcal{N}(\mathbf{0}, \boldsymbol{I})$: 若从别的分布起步, 前若干步的样本分布会有偏差, 这与 Markov 链的混合时间有关.

最后一步 ($t = 1$) 不添加噪声, 因为目标是产生无噪声的输出 $\boldsymbol{x}$; 这一点在下面的算法里单独写出.

其中 $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{\varepsilon} \mid \mathbf{0}, \boldsymbol{I})$. 注意神经网络 $\boldsymbol{g}(\cdot;\cdot)$ 预测的是生成 $\boldsymbol{z}_t$ 时添加到原始数据向量 $\boldsymbol{x}$ 上的总噪声. 但在采样步骤中, 我们只从 $\boldsymbol{z}_t$ 中减去一部分具有方差 $\beta_t / \sqrt{1-\alpha_t}$ 的噪声, 然后添加具有方差 β_t 的额外噪声来生成 $\boldsymbol{z}_{t-1}$. 在最终计算合成数据样本 $\boldsymbol{x}$ 的最终步骤中, 由于我们的目标是产生一个无噪声输出, 因此我们不会添加额外的噪声. 算法 20.2 对上述采样过程做了总结.

使用扩散模型生成数据的主要缺点是, 需要在训练好的网络中多次顺序执行推理过程, 计算成本非常高. 加快采样过程的一种方法是, 首先将去噪过程转换为连续时间上的微分方程, 然后使用其他有效的离散化方法来高效求解该方程 (参见小节 20.3.4).

在本章中, 我们假设数据和潜变量是连续的, 并且由此我们可以使用 Gauss 噪声模型. 扩散模型也可以定义在离散空间中 (Austin et al., 2021), 例如在生成新的候选药物分子时, 从化学元素子集中选择原子类型.

算法 20.2: 从去噪扩散概率模型中采样

输入: 训练好的去噪网络 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w}, t)$; 噪声调度 $\{\beta_1, \cdots, \beta_T\}$
输出: 数据空间中的样本向量 $\boldsymbol{x}$

$\boldsymbol{z}_T \sim \mathcal{N}(\boldsymbol{z} \mid \mathbf{0}, \boldsymbol{I})$ // 从最终的潜空间中采样
for $t \in \{T, \cdots, 2\}$ **do**
  $\alpha_t \leftarrow \prod_{\tau=1}^{t}(1-\beta_\tau)$ // 计算 α
  // 评估网络输出
  $\boldsymbol{\mu}(\boldsymbol{z}_t, \boldsymbol{w}, t) \leftarrow \frac{1}{\sqrt{1-\beta_t}}\left\{\boldsymbol{z}_t - \frac{\beta_t}{\sqrt{1-\alpha_t}}\boldsymbol{g}(\boldsymbol{z}_t, \boldsymbol{w}, t)\right\}$
  $\boldsymbol{\varepsilon} \sim \mathcal{N}(\boldsymbol{\varepsilon} \mid \mathbf{0}, \boldsymbol{I})$ // 采样一个噪声向量
  $\boldsymbol{z}_{t-1} \leftarrow \boldsymbol{\mu}(\boldsymbol{z}_t, \boldsymbol{w}, t) + \sqrt{\beta_t}\,\boldsymbol{\varepsilon}$ // 添加缩放的噪声
end for
$\boldsymbol{x} = \frac{1}{\sqrt{1-\beta_1}}\left\{\boldsymbol{z}_1 - \frac{\beta_1}{\sqrt{1-\alpha_1}}\boldsymbol{g}(\boldsymbol{z}_1, \boldsymbol{w}, 1)\right\}$ // 最终的步骤
return $\boldsymbol{x}$

我们看到, 由于扩散模型需要依次逆转一个可能包含成百上千个步骤的加噪过程, 计算强度较高. 研究人员 (Song, Meng, and Ermon, 2020) 提出了一种称为**去噪扩散隐式模型** (denoising diffusion implicit models) 的技术, 它放宽了加噪过程的 Markov 假设条件, 同时保留保持训练目标函数不变. 这样就可以在不降低所生成样本质量的情况下, 使采样速度提升一个或两个数量级.

20.3 得分匹配

截至目前, 本章讨论的去噪扩散模型与另一类相对独立发展的深度生成式模型密切相关, 它们都基于**得分匹配** (score matching)(Hyvärinen, 2005; Song and Ermon, 2019). 这些模型利用的得分函数 (或 Stein 得分) 则定义为对数似然函数关于数据向量 $\boldsymbol{x}$ 的梯度:

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}). \tag{20.44}$$

定义 20.4 (得分函数)

设 $p(\boldsymbol{x})$ 关于 $\boldsymbol{x}$ 可微且为正. 称向量值函数

$$\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$$

为分布 p 的**得分函数** (score function), 也称 **Stein 得分**. 若 p 还依赖参数 $\boldsymbol{w}$, 则记 $\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{w})$. ♣

注

(重构新增) 得分函数是“对数密度关于数据的梯度”, 与通常“关于参数 $\boldsymbol{w}$ 的梯度”完全不同: 前者指向“往哪个方向移动 $\boldsymbol{x}$ 能使密度增大”, 后者指向“怎样调整参数能使似然增大”. 这一区分是理解整节的钥匙——训练模型需要后者, 而从模型采样需要前者.

定义要求 $p > 0$ 且可微: 否则 $\ln p$ 或其梯度无定义. 在支撑集边界附近得分函数可能发散, 这是数值实现时要作裁剪或平滑的原因. 另外,$\boldsymbol{s}(\boldsymbol{x})$ 是与 $\boldsymbol{x}$ 同维的向量值函数, 其第 i 个分量 $s_i(\boldsymbol{x}) = \partial \ln p(\boldsymbol{x})/\partial x_i$ 只与 $\boldsymbol{x}$ 的第 i 个分量对应; 若 $\boldsymbol{x}$ 是图像,$\boldsymbol{s}(\boldsymbol{x})$ 也可视为同尺寸的“得分图像”.

得分函数有一个关键性质: 它不依赖归一化常数. 若 $p(\boldsymbol{x}) = \tilde{p}(\boldsymbol{x})/Z$ 且 Z 与 $\boldsymbol{x}$ 无关, 则 $\nabla_{\boldsymbol{x}} \ln p = \nabla_{\boldsymbol{x}} \ln \tilde{p}$. 这正是基于能量的模型可以绕开配分函数的原因 (参见小节 14.3.1).

这里需要强调的是, 梯度是针对数据向量 $\boldsymbol{x}$ 计算的, 而不是针对任何参数向量. 注意 $\boldsymbol{s}(\boldsymbol{x})$ 是一个与 $\boldsymbol{x}$ 维度相同的向量值函数, 并且其中的每个元素 $s_i(\boldsymbol{x}) = \partial \ln p(\boldsymbol{x})/\partial x_i$ 都与 $\boldsymbol{x}$ 的相应元素 x_i 相关联. 例如, 如果 $\boldsymbol{x}$ 是一个图像, 那么 $\boldsymbol{s}(\boldsymbol{x})$ 也可以表示为一个同尺寸的图像, 其中对应的像素是图像的得分.图 20.5显示了二维中的概率密度示例以及相应的得分函数.

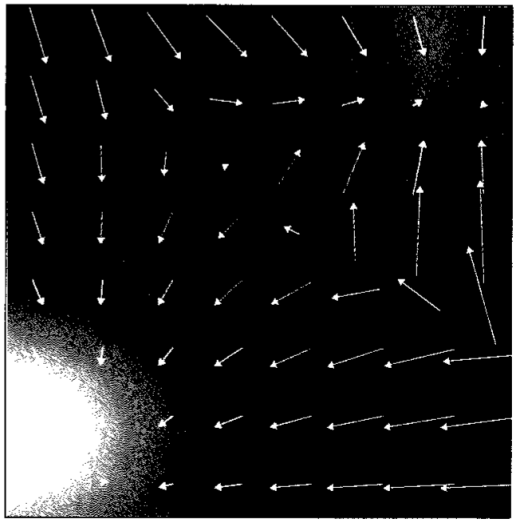


图 20.5: 得分函数的示意图, 图中显示了两种信息的融合: 由 Gauss 混合分布构成的二维分布, 其作为向量绘制在 $\boldsymbol{x}$ 值的规则网格上, 表示为热图; 以及由(20.44)定义的相应的概率密度示例以及相应的得分函数

为了理解得分函数的用处, 考虑两个函数 $q(\boldsymbol{x})$ 和 $p(\boldsymbol{x})$, 它们具有得分相等的属性, 所以对于 $\boldsymbol{x}$ 的所有值,$\nabla \ln q(\boldsymbol{x}) = \nabla \ln p(\boldsymbol{x})$. 我们对这个等式的两边关于 $\boldsymbol{x}$ 积分并取指数, 则可以得到 $q(\boldsymbol{x}) = Kp(\boldsymbol{x})$, 其中 K 是一个独立于 $\boldsymbol{x}$ 的常数. 所以如果我们能学习到一个得分函数的模型 $\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w})$, 就可以重建原始数据分布, 所得结果与

之前的相比, 最多相差一个常数倍数.

命题 20.8 (得分确定密度至多相差常数倍)

设 p, q 是 $\mathbb{R}^D$ 上处处为正的光滑概率密度, 且对一切 $\boldsymbol{x}$ 有 $\nabla_{\boldsymbol{x}} \ln q(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$. 则存在常数 $K > 0$, 使得 $q(\boldsymbol{x}) = Kp(\boldsymbol{x})$ 对一切 $\boldsymbol{x}$ 成立.

♠

证明

(重构新增) 令 $h(\boldsymbol{x}) = \ln q(\boldsymbol{x}) - \ln p(\boldsymbol{x})$. 由假设 $\nabla_{\boldsymbol{x}} h(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln q(\boldsymbol{x}) - \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}) = \boldsymbol{0}$ 对一切 $\boldsymbol{x}$ 成立.$\mathbb{R}^D$ 连通, 梯度处处为零的光滑函数必为常数, 故 $h(\boldsymbol{x}) \equiv \ln K$, 即 $q(\boldsymbol{x}) = Kp(\boldsymbol{x})$. 又 $p, q > 0$, 故 $K > 0$; 若二者都是概率密度, 则积分得 $K = 1$. 证毕.

□

注

(重构新增) 这条命题说明得分函数唯一确定了密度的形状, 而不确定其归一化: 两个得分相同的密度只差一个正常数因子. 这对建模是好事——我们不必计算配分函数就能比较密度的相对高低; 但也有代价: 仅凭得分无法直接得到归一化的概率值, 需要用 Langevin 采样得到样本后再估计归一化常数, 或转而使用不需归一化的采样方法.

证明中的两个条件不可省: 连通性保证“梯度为零”推出“常数”(若定义域分成两个不相连的块, 两块上的常数可以不同); 正性保证 ln 处处有定义. 若 p 或 q 在某个区域为零, 则得分在该处无定义, 命题的表述需要改成“在二者都为正的区域上”.

若 p, q 都是归一化密度, 则 $K = 1$, 即得分相同意味着密度相同. 这一点常被表述为“得分匹配可以识别分布”, 但要注意它依赖“二者都归一化”这一额外信息; 只拟合得分而不加归一化约束时,K 是自由的.

20.3.1 得分损失函数

为了训练这样的模型, 需要定义一个损失函数, 用于将模型得分函数 $\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w})$ 匹配到旨在生成数据的分布 $p(\boldsymbol{x})$ 的得分函数 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$. 这样一个损失函数的例子是计算模型得分和真实得分之间的期望平方误差, 其由下式给出:

$$J(\boldsymbol{w}) = \frac{1}{2} \int \left\| \boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) - \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}) \right\|^2 p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x}. \tag{20.45}$$

正如我们在讨论基于能量的模型时所看到的 (参见小节 14.3.1), 得分函数不要求相关的概率密度被归一化, 因为归一化常数被梯度算子移除了, 所以我们在模型选择上有相当大的灵活性. 使用深度神经网络表示得分函数 $\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w})$ 的方法有两种.$\boldsymbol{s}$ 的每个元素 s_i 都对应于 $\boldsymbol{x}$ 中的一个元素 x_i, 所以第一种方法是使用一个输入数量和输出数量相同的网络. 然而, 得分函数定义为标量函数 (对数概率密度函数) 的梯度, 这是一类受限的函数 (见练习 20.14). 另一种方法是使用一个拥有单一输出的网络 $\phi(\boldsymbol{x}, \boldsymbol{w})$, 然后使用自动微分计算 $\nabla_{\boldsymbol{x}} \phi(\boldsymbol{x}, \boldsymbol{w})$. 然而, 这方法需要进行两次反向传播, 因此计算成本更高. 大多数应用通常采用第一种方法 (见练习 20.15).

20.3.2 修改得分损失

损失函数(20.45)的一个问题是不能直接最小化, 因为我们不知道真实的数据得分 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$. 我们所知的只是有限的数据集 $\mathcal{D} = \{\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N\}$, 从中可以构建如下经验分布:

$$p_{\mathcal{D}}(\boldsymbol{x}) = \frac{1}{N} \sum_{n=1}^{N} \delta(\boldsymbol{x} - \boldsymbol{x}_n). \tag{20.46}$$

这里的 $\delta(\boldsymbol{x})$ 是 Dirac delta 函数, 我们可以将其形象地理解为在 $\boldsymbol{x} = \boldsymbol{0}$ 处无限高的“尖峰”, 它具有以下性质:

$$\delta(\boldsymbol{x}) = 0, \quad \boldsymbol{x} \neq \boldsymbol{0}, \tag{20.47}$$

$$\int \delta(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} = 1. \tag{20.48}$$

由于(20.46)所示的函数不是 $\boldsymbol{x}$ 的可微函数, 我们无法计算它的得分. 但我们可以通过引入噪声模型来“抹平”数据点并给出密度的平滑、可微表示来解决这个问题, 这称为 Parzen 估计器或核密度估计器 (参见小节 3.5.2), 定义如下:

$$q_\sigma(\boldsymbol{z}) = \int q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma)\, p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}, \tag{20.49}$$

其中 $q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma)$ 是噪声核. 常见的核选择是 Gauss 核:

$$q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma) = \mathcal{N}(\boldsymbol{z} \mid \boldsymbol{x}, \sigma^2 \boldsymbol{I}). \tag{20.50}$$

此时, 我们不再最小化损失函数(20.45), 而是使用对应于平滑后的 Parzen 密度的损失函数, 形式为

$$J(\boldsymbol{w}) = \frac{1}{2} \int \left\| \boldsymbol{s}(\boldsymbol{z}, \boldsymbol{w}) - \nabla_{\boldsymbol{z}} \ln q_\sigma(\boldsymbol{z}) \right\|^2 q_\sigma(\boldsymbol{z})\, \mathrm{d}\boldsymbol{z}. \tag{20.51}$$

一个关键结果是, 通过将(20.49)代入(20.51), 我们可以将这个损失函数重写为以下等价形式 (Vincent, 2011)(见练习 20.17).

命题 20.9 (去噪得分匹配)

设 q_σ 由(20.49)与(20.50)给出. 则对任意得分模型 $\boldsymbol{s}(\boldsymbol{z}, \boldsymbol{w})$,

$$J(\boldsymbol{w}) = \frac{1}{2} \iint \left\| \boldsymbol{s}(\boldsymbol{z}, \boldsymbol{w}) - \nabla_{\boldsymbol{z}} \ln q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma) \right\|^2 q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma)\, p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{z}\, \mathrm{d}\boldsymbol{x} + \text{const}, \tag{20.52}$$

其中 const 与网络参数 $\boldsymbol{w}$ 无关.

♠

证明

(重构新增) 把(20.51)中的平方项展开:

$$J(\boldsymbol{w}) = \frac{1}{2} \int \|\boldsymbol{s}\|^2 q_\sigma\, \mathrm{d}\boldsymbol{z} - \int \boldsymbol{s} \cdot \nabla_{\boldsymbol{z}} q_\sigma\, \mathrm{d}\boldsymbol{z} + \frac{1}{2} \int \|\nabla_{\boldsymbol{z}} \ln q_\sigma\|^2 q_\sigma\, \mathrm{d}\boldsymbol{z},$$

其中第二项用了 $\nabla_{\boldsymbol{z}} \ln q_\sigma \cdot q_\sigma = \nabla_{\boldsymbol{z}} q_\sigma$. 第三项不含 $\boldsymbol{w}$, 可以并入常数. 第一项: 由(20.49), $\int \|\boldsymbol{s}\|^2 q_\sigma\, \mathrm{d}\boldsymbol{z} = \iint \|\boldsymbol{s}\|^2 q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma) p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{z}\, \mathrm{d}\boldsymbol{x}$, 与(20.52)中平方项的第一部分相同. 第二项: 对(20.49)求梯度得 $\nabla_{\boldsymbol{z}} q_\sigma(\boldsymbol{z}) = \int \nabla_{\boldsymbol{z}} q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma) p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{x}$, 于是

$$\begin{aligned} \int \boldsymbol{s} \cdot \nabla_{\boldsymbol{z}} q_\sigma\, \mathrm{d}\boldsymbol{z} &= \iint \boldsymbol{s}(\boldsymbol{z}, \boldsymbol{w}) \cdot \nabla_{\boldsymbol{z}} q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma) p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{z}\, \mathrm{d}\boldsymbol{x} \\ &= \iint \boldsymbol{s}(\boldsymbol{z}, \boldsymbol{w}) \cdot \nabla_{\boldsymbol{z}} \ln q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma)\, q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma) p(\boldsymbol{x})\, \mathrm{d}\boldsymbol{z}\, \mathrm{d}\boldsymbol{x}, \end{aligned}$$

这正是(20.52)中交叉项的一半 (带负号). 合并三项即得(20.52). 证毕.

□

注

(重构新增) (20.52)的意义在于把“与未知的 $\nabla \ln q_\sigma$ 比较”换成了“与已知的核得分 $\nabla \ln q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma)$ 比较”, 因而损失可以用样本 $\boldsymbol{x}_n$ 与人工噪声 $\boldsymbol{z}$ 直接计算. 这正是“去噪”一词的来源: 网络要从被噪声污染的 $\boldsymbol{z}$ 中估计出把 $\boldsymbol{z}$ 拉回 $\boldsymbol{x}$ 的方向.

条件方面: 交换积分次序要求 $q(\boldsymbol{z} \mid \boldsymbol{x}, \sigma) p(\boldsymbol{x})$ 可积 (Gauss 核保证这一点); 丢弃的第三项之所以与 $\boldsymbol{w}$ 无关, 是因为它只含真实密度 q_σ. 若两个得分模型在某个 $\boldsymbol{z}$ 上给出相同输出, 这项对二者的相对大小没有影响, 因此不能省掉它去比较不同 σ 下的绝对损失值.

与扩散模型的联系: 若取 $\sigma^2 = 1 - \alpha_t$ 并把核均值改为 $\sqrt{\alpha_t}\,\boldsymbol{x}$, 就得到前向扩散核, 此时(20.52)与(20.39)具有相同的最小值 (相差常数缩放). 这就是“去噪得分匹配”与 DDPM 训练目标等价的含义.

用经验分布(20.46)代替 $p(\boldsymbol{x})$, 可以得到

$$J(\boldsymbol{w}) = \frac{1}{2N} \sum_{n=1}^{N} \int \left\| \boldsymbol{s}(\boldsymbol{z}, \boldsymbol{w}) - \nabla_{\boldsymbol{z}} \ln q(\boldsymbol{z} \mid \boldsymbol{x}_n, \sigma) \right\|^2 q(\boldsymbol{z} \mid \boldsymbol{x}_n, \sigma)\, \mathrm{d}\boldsymbol{z} + \text{const}. \tag{20.53}$$

对于 Gauss 核(20.50), 得分函数变为

$$\nabla_z \ln q(z \mid x, \sigma) = -\frac{1}{\sigma}\varepsilon, \tag{20.54}$$

其中 $\varepsilon = (z - x)/\sigma \sim \mathcal{N}(\varepsilon \mid 0, I)$ 是标准化噪声. 考虑特定的噪声模型(20.7), 可以得到

$$\nabla_z \ln q(z \mid x, \sigma) = -\frac{1}{\sqrt{1-\alpha_t}}\varepsilon. \tag{20.55}$$

由此我们可以看出, 损失函数(20.52)衡量了神经网络预测与噪声 ε 之间的差异. 所以, 该损失函数与去噪扩散模型中使用的损失函数(20.39)拥有相同的最小值, 得分函数 $s(z, w)$ 扮演的角色与噪声预测网络 $g(z, w, t)$ 相同, 但前者做了常数缩放 $-1/\sqrt{1-\alpha_t}$(Song and Ermon, 2019). 最小化(20.52)的过程称为**去噪得分匹配** (denoise score matching), 并且我们看到它与去噪扩散模型有着密切的联系. 我们稍后讨论如何选择噪声方差 σ^2.

训练完基于得分的模型后, 我们需要生成新的样本.Langevin 动力学 (Langevin dynamic) 非常适用于基于得分函数的模型, 因为它以得分函数为基础进行运算, 无须依赖归一化的概率分布, 参见图 20.6.

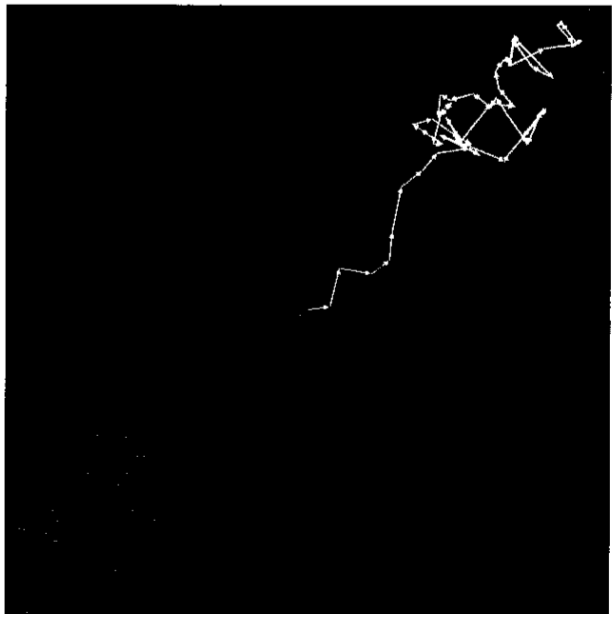


图 20.6: 使用(14.61)定义的 Langevin 动力学得到的采样轨迹, 这里显示了三条都从图中心开始的采样轨迹 (参见节 14.3)

20.3.3 噪声方差

我们已经了解怎么从一组训练数据中学习得分函数, 以及如何使用 Langevin 动力学方法从学习到的分布中进行采样并生成新的样本. 然而, 这种方法存在三个潜在的问题 (Song and Ermon, 2019; Luo, 2022)(参见第 16 章). 第一, 如果数据分布位于比数据空间维度更低的流形上, 那么流形之外的点的概率密度将为零, 此时因为 $\ln p(x)$ 是未定义的, 所以得分函数是未定义的. 第二, 在数据密度低的区域, 得分函数的估计可能不准确, 因为损失函数(20.45)被密度加权了. 一个不准确的得分函数会导致使用 Langevin 动力学方法采样时轨迹不佳. 第三, 即使有一个准确的得分函数, 如果数据分布包括一系列不连续的混合分布, 通过 Langevin 动力学方法也可能无法正确采样 (见练习 20.18).

以上三个问题都可以通过选择核函数(20.50)中使用的噪声方差 σ^2 的足够大值来解决, 因为这会使数据分布平滑. 然而, 过大的方差会导致原始分布显著失真, 导致得分函数建模失准. 一个折中方案是引入方差序列 $\sigma_1^2 < \sigma_2^2 < \cdots < \sigma_L^2$(Song and Ermon, 2019), 其中 σ_1^2 足够小, 以便数据分布得到准确表示; 而 σ_L^2 则足够大, 以避免上述问题. 然后修改得分网络, 使其将方差作为额外的输入 $s(x, w, \sigma^2)$, 并通过使用一个加权和形式的损失函数(20.53)对

其进行训练, 其中的每一项表示相关网络与相应扰动数据集之间的误差. 对于数据向量 $\boldsymbol{x}_n$, 损失函数的形式为

$$\frac{1}{2}\sum_{i=1}^{L}\lambda(i)\int\left\|\boldsymbol{s}(z,\boldsymbol{w},\sigma_i^2)-\nabla_z\ln q(z\mid\boldsymbol{x}_n,\sigma_i)\right\|^2 q(z\mid\boldsymbol{x}_n,\sigma_i)\,\mathrm{d}z, \tag{20.56}$$

其中 $\lambda(i)$ 是权重系数. 这个训练过程精确地对应于用来训练分层去噪网络的机制 (参见小节 20.2.1).

训练完成后, 可以通过依次对 $i=L,L-1,\cdots,2,1$ 的每个模型逐个执行几步 Langevin 动力学方法进行采样并生成样本. 这种技术称为**退火 Langevin 动力学** (annealed Langevin dynamics), 类似于用来从去噪扩散概率模型中采样的算法 20.2.

20.3.4 随机微分方程

在构建扩散模型的加噪过程时, 使用大数量的步骤 (通常为几千步) 是有益处的. 因此, 我们需要考虑在无穷步数的极限情况下会发生什么, 就像我们在引入神经微分方程时为无限深度的神经网络所做的那样 (参见节 18.3). 在这种极限情况下, 我们需要确保每一步的噪声方差 β_t 随着步长变小而同步变小. 这引出了连续时间下扩散模型的理论框架, 即**随机微分方程** (Stochastic Differential Equation, SDE)(Song et al., 2020). 去噪扩散概率模型和得分匹配模型都可以视为连续时间 SDE 的离散化实现.

我们可以将一般的 SDE 写成向量 z 的无穷小更新:

$$\mathrm{d}z=\underbrace{\boldsymbol{f}(z,t)\,\mathrm{d}t}_{\text{漂移项}}+\underbrace{g(t)\,\mathrm{d}\boldsymbol{w}}_{\text{扩散项}}, \tag{20.57}$$

其中漂移项 (drift term) 是确定性的, 类似于常微分方程 (ODE), 但扩散项 (diffusion term) 是随机的. 这里的参数 t 常常称为 “时间”, 类比于物理系统中的时间. 扩散模型的前向加噪过程(20.3), 可以通过采取连续时间极限而写成形如(20.57)的 SDE(见练习 20.19).

定义 20.5 (随机微分方程)

设 $\boldsymbol{f}(z,t)$ 为漂移系数、$g(t)$ 为扩散系数、$\boldsymbol{w}$ 为标准 Wiener 过程. 称形如(20.57)的方程为 **Itô 随机微分方程** (SDE), 其解 $z(t)$ 是一个连续时间的随机过程:$\boldsymbol{f}$ 描述确定性的漂移,$g\,\mathrm{d}\boldsymbol{w}$ 描述随机的扩散. ♣

注

(重构新增) SDE 可以看作 “常微分方程 + 噪声” 的连续时间极限: 把 $\mathrm{d}\boldsymbol{w}$ 理解为均值为零、方差为 $\mathrm{d}t$ 的 Gauss 增量, 就回到了离散的更新式. 它之所以适合描述扩散模型, 是因为前向加噪过程的步数 T 可以趋于无穷, 而每一步的方差 β_t 必须随步长同步缩小, 否则极限不存在.

记号上要注意两点: 其一, 这里写的是 Itô 形式,$g\,\mathrm{d}\boldsymbol{w}$ 的积分按 Itô 定义, 因而普通微积分的链式法则需要换成 Itô 公式; 其二,g 只依赖 t 而不依赖 z, 这是本书的简化约定, 一般的 SDE 允许 $g(z,t)$.

条件:$\boldsymbol{f}$ 关于 z 满足 Lipschitz 条件、g 有界, 才能保证 SDE 有唯一强解; 若这些条件不满足, 解可能在有限时间爆炸, 后面的 Fokker–Planck 论证也随之失效.

对于 SDE(20.57), 存在一个对应的逆向 SDE(Song et al., 2020), 如下所示:

$$\mathrm{d}z=\left\{\boldsymbol{f}(z,t)-g^2(t)\nabla_z\ln p_t(z)\right\}\mathrm{d}t+g(t)\,\mathrm{d}\bar{\boldsymbol{w}}, \tag{20.58}$$

其中 $\nabla_z\ln p_t(z)$ 是得分函数. 对于(20.57)给出的 SDE, 应从 $t=T$ 倒序求解至 $t=0$.

对于所有由 SDE 支配的扩散过程, 都存在一个相应的由 ODE 描述的确定性过程, 其轨迹具有与 SDE 相同的边缘概率密度 $p_t(z)$(Song et al., 2020). 对于形如(20.57)的 SDE, 相应的 ODE 为

$$\frac{\mathrm{d}z}{\mathrm{d}t}=\boldsymbol{f}(z,t)-\frac{1}{2}g^2(t)\nabla_z\ln p_t(z). \tag{20.59}$$

命题 20.10 (逆时 SDE 与概率流 ODE)

设 p_t 是(20.57)的解的密度, 关于 t 连续可微、关于 z 二阶连续可微, 在无穷远处充分快衰减, 并满足 Fokker–

Planck 方程

$$\frac{\partial p_t}{\partial t} = -\nabla_z \cdot \big(\boldsymbol{f}(z,t)p_t\big) + \frac{1}{2}g^2(t)\Delta_z p_t.$$

则:(i) 逆向 SDE(20.58)从 $t = T$ 倒序求解时, 其边缘密度为 p_t;(ii) ODE(20.59)的轨道具有与(20.57)相同的边缘密度 p_t.

♠

证明

(重构新增) 先证 (ii). 令 $\boldsymbol{v}(z,t) = \boldsymbol{f}(z,t) - \frac{1}{2}g^2(t)\nabla_z \ln p_t(z)$, 则 $\boldsymbol{v}p_t = \boldsymbol{f}p_t - \frac{1}{2}g^2\nabla_z p_t$, 取散度得

$$\nabla_z \cdot (\boldsymbol{v}p_t) = \nabla_z \cdot (\boldsymbol{f}p_t) - \frac{1}{2}g^2\Delta_z p_t.$$

而由 Fokker–Planck 方程,$\partial_t p_t = -\nabla_z \cdot (\boldsymbol{f}p_t) + \frac{1}{2}g^2\Delta_z p_t = -\nabla_z \cdot (\boldsymbol{v}p_t)$. 这正是连续性方程, 故 ODE $\mathrm{d}z/\mathrm{d}t = \boldsymbol{v}(z,t)$ 精确输运密度 p_t: 其轨道的边缘密度在任意 t 都等于 p_t. 再证 (i). 令 $\tau = T - t, q_\tau(z) = p_{T-\tau}(z)$. 把(20.58)按反时方向记为 τ 增加方向的 SDE, 其漂移为 $-\{\boldsymbol{f} - g^2\nabla \ln p\}$、扩散系数仍为 g. 由 Fokker–Planck 方程,q_τ 满足

$$\begin{aligned}\frac{\partial q_\tau}{\partial \tau} &= -\nabla_z \cdot \Big(\big(-\boldsymbol{f} + g^2\nabla_z \ln q_\tau\big)q_\tau\Big) + \frac{1}{2}g^2\Delta_z q_\tau \\ &= \nabla_z \cdot (\boldsymbol{f}q_\tau) - g^2\Delta_z q_\tau + \frac{1}{2}g^2\Delta_z q_\tau = \nabla_z \cdot (\boldsymbol{f}q_\tau) - \frac{1}{2}g^2\Delta_z q_\tau.\end{aligned}$$

另一方面, 由 $q_\tau = p_{T-\tau}$ 与 Fokker–Planck 方程,$\partial_\tau q_\tau = -\partial_t p_t|_{T-\tau} = \nabla_z \cdot (\boldsymbol{f}q_\tau) - \frac{1}{2}g^2\Delta_z q_\tau$. 两式相同, 故 q_τ 就是逆时 SDE 的密度, 即逆时 SDE 的边缘密度为 $p_{T-\tau}$; 换回 t 即得 (i). 证毕.

□

注

(重构新增) 这条命题是“连续时间扩散模型”的理论核心: 它把“加噪—去噪”的离散图像换成了“一个 SDE 与其逆时 SDE 具有相同边缘分布”的连续图像. 证明只用到 Fokker–Planck 方程与连续性方程, 两条方程分别刻画“随机过程的密度演化”与“确定性流对密度的输运”, 因此结论是精确的, 而非近似.

ODE(20.59)称为概率流 ODE(probability flow ODE), 它的轨道是确定性的: 给定初值 $\boldsymbol{z}_T$, 整条轨迹就被确定, 因而它定义了从先验到数据的可逆映射, 可以把扩散模型与标准化流联系起来, 用变量替换公式精确计算对数似然 (参见节 18.3).

条件不能省:p_t 的光滑性与衰减性保证 Fokker–Planck 方程的分部积分合法;$\nabla_z \ln p_t$ 必须存在, 否则逆时 SDE 的漂移无定义. 逆时 SDE 中的 $\mathrm{d}\bar{\boldsymbol{w}}$ 是反向时间轴上的 Wiener 增量, 它把“时间的箭头”翻转过来——这正是从 $t = T$ 倒序求解的含义.

要想数值求解 SDE, 我们需要对时间变量进行离散化. 最简单的方法是使用固定且等间距的时间步, 这称为 Euler–Maruyama 求解器. 对于反向 SDE, 我们可以先恢复 Langevin 方程的一种形式 (参见节 14.3), 再采用更复杂的求解器, 这些求解器使用了更灵活的离散化形式 (Kloeden and Platen, 2013).

该 ODE 允许使用高效的自适应步长求解器以最大限度地减少函数评估次数. 此外,ODE 还允许将概率扩散模型与标准化流模型相关联, 后者允许使用变量替换(18.1)来精确计算对数似然 (参见节 18.3).

20.4 有引导的扩散

到目前为止, 我们讨论的扩散模型主要用于表示从一组训练样本 $\boldsymbol{x}_1, \cdots, \boldsymbol{x}_N$ 中学习到的无条件密度 $p(\boldsymbol{x})$, 这些样本是独立地从 $p(\boldsymbol{x})$ 中抽取的. 一旦模型训练好了, 我们就可以从这个分布中生成新的样本. 我们已经看到了一个从深度生成式模型中进行无条件采样的例子, 用于人脸图像, 如图 1.3所示, 该例从一个 GAN 模型中进行采样.

然而在许多应用中, 我们希望从条件分布 $p(\boldsymbol{x} \mid \boldsymbol{c})$ 中进行采样, 其中条件变量 $\boldsymbol{c}$ 可以是一个类标签或是描述目标图像内容的文字. 这也构成了诸如图像超分辨率、图像修复、视频生成等许多其他应用的基础. 实现这一点的最简单方法是将 $\boldsymbol{c}$ 视为去噪神经网络 $\boldsymbol{g}(\boldsymbol{z}, \boldsymbol{w}, t, \boldsymbol{c})$ 的一个额外输入, 然后使用匹配的 $\{\boldsymbol{x}_n, \boldsymbol{c}_n\}$ 对来训练网络. 这

种方法的主要问题是, 网络可能会对条件变量赋予的权重不足, 甚至忽略条件变量, 因此我们需要一种方式来控制给予条件变量多少权重, 并与样本多样性进行权衡. 这种用以匹配条件信息的约束机制称为**引导** (guidance). 根据是否使用单独的分类器, 引导主要有两种.

定义 20.6 (引导与引导尺度)

设条件扩散模型的得分函数为 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{c})$. 称在采样阶段人为放大条件项、以调节“条件信息权重”的机制为**引导** (guidance), 其中用来控制条件项权重的超参数 $\lambda \geqslant 0$ 称为**引导尺度** (guidance scale).

♣

注

(重构新增) 引导要解决的问题是“条件不够强”: 仅把 $\boldsymbol{c}$ 作为额外输入训练时, 网络在训练集上可以完全忽略 $\boldsymbol{c}$ 而仍然取得不差的损失 (因为无条件模型本身就是条件模型的一个特例), 于是采样结果与 $\boldsymbol{c}$ 无关. 引导通过在采样阶段人为放大条件项来补偿这一点, 因此它改变的是采样的分布, 而不是训练的目标.

定义中的 λ 是一个纯粹的采样超参数, 不参与训练: 同一组网络权重可以用不同的 λ 生成不同“贴合度”的样本. 这也意味着引导后的得分一般不再是某个归一化密度的得分 ($\lambda \neq 1$ 时它对应的是退火后的“尖锐化”分布), 这一点在解释 $\lambda > 1$ 的副作用时必须记住.

条件 $\lambda \geqslant 0$ 只是约定; 数学上 $\lambda < 0$ 也有定义, 但会主动远离条件, 实践中没有用处. 下文会看到 $\lambda = 0$ 与 $\lambda = 1$ 是两个有明确概率含义的端点.

20.4.1 有分类器的引导

假设我们有一个训练好的分类器 $p(\boldsymbol{c} \mid \boldsymbol{x})$, 下面从得分函数的角度考虑扩散模型. 使用 Bayes 定理, 我们可以将条件扩散模型的得分函数写成以下形式.

命题 20.11 (分类器引导的得分分解)

设 $p(\boldsymbol{x} \mid \boldsymbol{c})$、$p(\boldsymbol{x})$ 与 $p(\boldsymbol{c} \mid \boldsymbol{x})$ 为正的光滑密度, 且 $p(\boldsymbol{c})$ 不依赖 $\boldsymbol{x}$. 则

$$\begin{aligned} \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{c}) &= \nabla_{\boldsymbol{x}} \ln \left\{ \frac{p(\boldsymbol{c} \mid \boldsymbol{x})\, p(\boldsymbol{x})}{p(\boldsymbol{c})} \right\} \\ &= \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}) + \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{c} \mid \boldsymbol{x}). \end{aligned} \tag{20.60}$$

♠

证明

(重构新增) 由 Bayes 定理,$p(\boldsymbol{x} \mid \boldsymbol{c}) = p(\boldsymbol{c} \mid \boldsymbol{x})p(\boldsymbol{x})/p(\boldsymbol{c})$. 两边取对数得

$$\ln p(\boldsymbol{x} \mid \boldsymbol{c}) = \ln p(\boldsymbol{c} \mid \boldsymbol{x}) + \ln p(\boldsymbol{x}) - \ln p(\boldsymbol{c}),$$

再对 $\boldsymbol{x}$ 求梯度. 由于 $p(\boldsymbol{c})$ 不依赖 $\boldsymbol{x}$,$\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{c}) = \mathbf{0}$, 故(20.60)成立. 证毕.

□

注

(重构新增) (20.60)把“条件生成”拆成“无条件生成 + 分类器梯度”两部分: 第一项是常见的无条件得分函数, 第二项推动去噪过程朝着最大化给定标签 $\boldsymbol{c}$ 在分类器模型下的概率的方向前进 (Dhariwal and Nichol, 2021). 因此只要有任意一个训练好的无条件扩散模型和一个分类器, 就能做条件采样, 无需重新训练扩散模型——这是分类器引导最吸引人的地方.

$p(\boldsymbol{c})$ 不依赖 $\boldsymbol{x}$ 是推导的关键, 也是“分类器只需给出似然比”的体现: 分类器的绝对概率校准不影响引导方向, 只有它对 $\boldsymbol{x}$ 的梯度起作用.$p(\boldsymbol{x} \mid \boldsymbol{c})$ 与 $p(\boldsymbol{c} \mid \boldsymbol{x})$ 都必须为正且可微, 否则对数梯度无定义; 在实践中这意味着分类器要在含噪样本上也有意义, 而后文正是针对这一点提出批评.

这里我们利用了 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{c}) = \mathbf{0}$, 因为 $p(\boldsymbol{c})$ 不依赖于 $\boldsymbol{x}$.(20.60)中的第一项是常见的无条件得分函数, 第二项则推动去噪过程朝着最大化给定标签 $\boldsymbol{c}$ 在分类器模型下的概率的方向前进 (Dhariwal and Nichol, 2021). 我们可以通过引入一个超参数 λ [称为引导尺度 (guidance scale)] 来控制赋予分类器梯度的权重. 用于采样的得分函数随后

变为

$$\text{score}(\boldsymbol{x}, \boldsymbol{c}, \lambda) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}) + \lambda \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{c} \mid \boldsymbol{x}). \tag{20.61}$$

如果 $\lambda = 0$, 则恢复原始的无条件扩散模型; 而如果 $\lambda = 1$, 则得到对应于条件分布 $p(\boldsymbol{x} \mid \boldsymbol{c})$ 的得分. 当 $\lambda > 1$ 时, 模型被强烈引导遵守条件标签, 我们也可能使用 $\lambda \geqslant 1$ 的值, 例如 $\lambda = 10$. 然而, 这是以牺牲样本多样性为代价的, 因为模型更倾向于“简单”的样本, 即分类器能够正确分类的样本.

基于分类器的引导方法的一个问题是必须训练一个单独的分类器. 此外, 这个分类器需要能够对具有不同加噪程度的样本进行分类, 而标准分类器是在干净的样本上进行训练的. 因此, 我们转向另一种无需独立分类器的引导方法, 以避免使用单独的分类器.

20.4.2 无分类器的引导

更进一步, 我们无须分别训练多个网络来对 $p(\boldsymbol{x} \mid \boldsymbol{c})$ 和 $p(\boldsymbol{x})$ 进行建模, 而是可以训练一个单独的条件模型, 并在该模型中将条件变量 $\boldsymbol{c}$ 在训练过程中以一定概率设置为一个空值, 例如 $\boldsymbol{c} = \boldsymbol{0}$, 概率通常设置为 10% ~ 20%. 然后 $p(\boldsymbol{x})$ 由 $p(\boldsymbol{x} \mid \boldsymbol{c} = \boldsymbol{0})$ 表示. 这在某种程度上类似于 dropout,dropout 会将一部分训练向量的所有条件输入都设为零 (参见小节 9.6.1).

训练完成后, 得分函数(20.62)用来强化条件信息的权重. 实践表明, 无分类器的引导相比有分类器的引导可以提供更高的生成质量 (Nichol et al., 2021; Saharia et al., 2022). 原因是分类器 $p(\boldsymbol{c} \mid \boldsymbol{x})$ 可以忽略大部分输入向量 $\boldsymbol{x}$, 只要能对 $\boldsymbol{c}$ 做出良好预测即可; 而无分类器的引导基于条件密度 $p(\boldsymbol{x} \mid \boldsymbol{c})$, 其必须为 $\boldsymbol{x}$ 的所有特征赋予高概率.

用(20.60)替换(20.61)中的 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{c} \mid \boldsymbol{x})$, 即可得到如下结论 (见练习 20.20).

命题 20.12 (无分类器引导 (CFG))

设条件得分与无条件得分分别为 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{c})$ 与 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$, 且二者都存在. 则对任意 λ,

$$\text{score}(\boldsymbol{x}, \boldsymbol{c}, \lambda) = \lambda \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{c}) + (1 - \lambda) \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}). \tag{20.62}$$

♠

证明

(重构新增) 由(20.60)得 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{c} \mid \boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{c}) - \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$. 代入(20.61):

$$\begin{aligned} \text{score} &= \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}) + \lambda\big(\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{c}) - \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})\big) \\ &= \lambda \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{c}) + (1 - \lambda) \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x}), \end{aligned}$$

即(20.62). 证毕.

□

注

(重构新增) (20.62)只用到条件模型自身的两个得分——条件得分与无条件得分——因此不需要任何外部分类器: 训练时以一定概率把 $\boldsymbol{c}$ 置空, 同一个网络就同时学会了二者. 这是“无分类器”一词的确切含义.

对 $0 < \lambda < 1$,(20.62)是条件对数密度 $\ln p(\boldsymbol{x} \mid \boldsymbol{c})$ 与无条件对数密度 $\ln p(\boldsymbol{x})$ 的凸组合, 因而对应一个介于二者之间的“插值”分布;$\lambda = 1$ 给出纯条件模型,$\lambda = 0$ 给出纯无条件模型. 对 $\lambda > 1$, 来自无条件得分项的系数 $1 - \lambda$ 变为负值, 模型主动降低那些忽略条件信息的样本的概率, 而更倾向于生成考虑条件信息的样本.

注意 $\lambda \neq 1$ 时(20.62)一般不是某个归一化密度的得分 (除非该插值恰好可积并归一化), 所以“$\lambda > 1$ 提高贴合度”是以偏离真实条件分布为代价的; 这也解释了它为什么会让样本多样性下降.

对于 $0 < \lambda < 1$, 上式代表了条件对数密度 $\ln p(\boldsymbol{x} \mid \boldsymbol{c})$ 和无条件对数密度 $\ln p(\boldsymbol{x})$ 的凸组合; 对于 $\lambda > 1$, 来自无条件得分项的贡献变为负值, 这意味着模型主动降低了生成那些忽略条件信息的样本的概率, 而更倾向于生成考虑条件信息的样本.

文本引导的扩散模型可以结合大语言模型的技术, 把条件输入扩展为通用文本序列, 即提示词 (prompt), 而非局限于预定义的类别标签 (参见第 12 章). 这允许文本输入以两种方式影响去噪过程: 第一, 将基于 Transformer 的大语言模型的内部表示与去噪网络的输入连接起来; 第二, 允许去噪网络内的交叉注意力层关注文本标记序

列.图 20.7对基于文本提示的无分类器的引导做了说明.

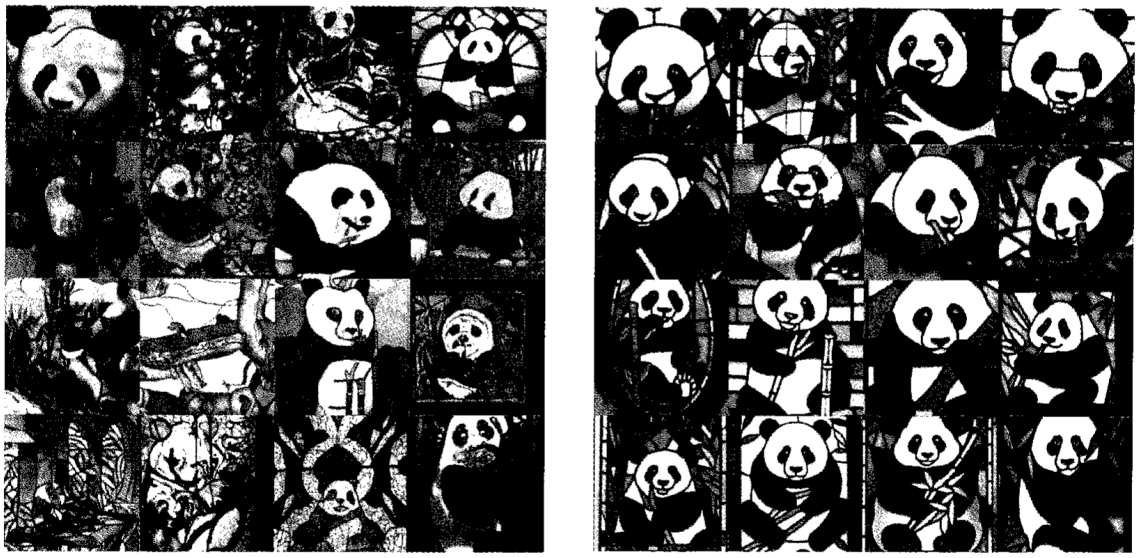


图 20.7: 基于文本提示的无分类器的引导, 扩散模型由一个名为 GLIDE 的模型生成, 使用的条件文本是 “A stained glass window of a panda eating bamboo” (一只熊猫在吃竹子的彩绘玻璃窗). 左侧的样本是用 $\lambda = 0$ 生成的 (无引导, 仅仅是纯条件模型), 而右侧的样本是用 $\lambda = 3$ 生成的 [图片来源于 Nichol et al. (2021), 经授权使用]

条件扩散模型的另一个应用是图像超分辨率, 旨在将低分辨率图像转换为相应的高分辨率图像. 这本质上是一个逆问题, 单个低分辨率图像对应于多个高分辨率图像的解. 实现方法是通过用低分辨率图像作为条件变量, 对由 Gauss 分布采样生成的高分辨率样本进行去噪 (Saharia, Ho, et al., 2021), 参见图 20.8. 这些模型可以级联起来以实现非常高的图像分辨率 (Ho et al., 2021), 例如从 64×64 到 256×256, 然后从 256×256 到 1024×1024. 每个阶段通常用一个 U-Net 架构 (参见小节 10.5.4), 且当前 U-Net 都以前一阶段的最终去噪结果为条件.

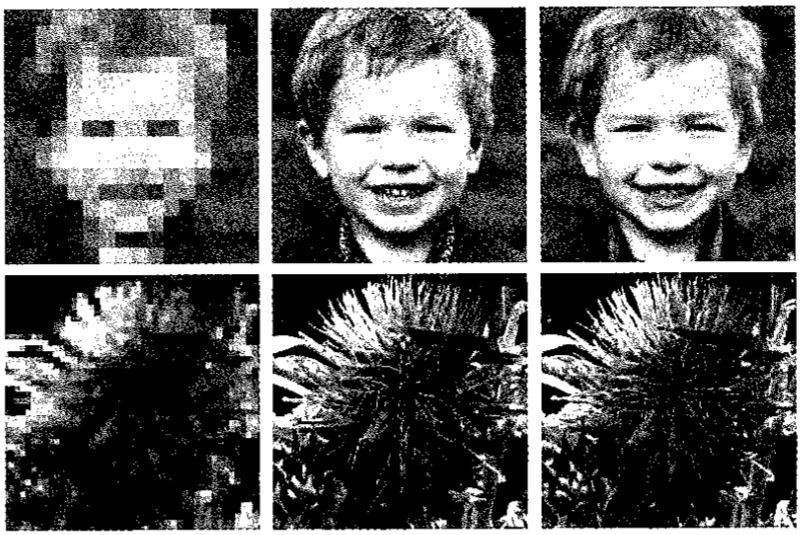


图 20.8: 两个低分辨率图像以及由扩散模型生成的相应高分辨率图像. 上面一行显示了一个 16×16 的输入图像和相应的 128×128 的输出图像, 以及原始图像. 下面一行显示了一个 64×64 的输入图像和一个 256×256 的输出图像, 并与原始图像进行了对比 [图片来源于 Saharia, Ho et al. (2021), 经授权使用]

这种级联方式也适用于图像生成扩散模型, 在这种模型中, 首先在较低分辨率的图像上执行去噪操作, 之后使用另一个独立的网络 (可附加文本提示词输入) 进行上采样, 以生成最终的高分辨率图像 (Nichol et al., 2021;

Saharia et al., 2022). 由于去噪过程可能需要调用去噪网络数百次, 与直接在高维空间中进行计算相比, 级联方法能够大幅减少计算量. 需要指出的是, 这种方法依旧是在图像空间中直接进行操作的, 只是分辨率较低而已.

对于在高分辨率图像空间中直接应用扩散模型所带来的高计算成本问题, 另一种方法是使用**潜扩散模型** (latent diffusion model)(Rombach et al., 2021). 首先在无噪声图像上训练一个自编码器 (autoencoder), 以获得图像的低维表征并使参数固定操作. 接下来训练一个 U-Net 网络, 在低维空间中执行去噪操作, 注意, 这个空间本身并不能直接解释为图像. 最后使用固定自编码器网络的输出部分将去噪表征映射到高分辨率图像空间. 这种方法更有效地利用了低维空间, 该空间可以专注于图像语义, 并让解码器从去噪的低维表示中创建相应的清晰的高分辨率图像.

条件图像生成的其他应用还包括图像修复、去除裁剪、图像恢复、图像变形、图像风格转移、着色、去模糊和视频生成等 (Yang, Srivastava, and Mandt, 2022).图 20.9展示了一个图像修复的例子.

图 20.9: 图像修复的一个例子. 左侧是原始图像, 中间是部分区域被移除的图像, 右侧是经过修复处理的图像 [图片来源于 Saharia, Chan, Chang et al. (2021), 经授权使用]

20.5 习题

✍ **练习 20.1** (★) 使用(20.3), 以 z_{t-1} 的均值和协方差为变量, 写出 z_t 的均值和协方差. 证明当 $0 < \beta_t < 1$ 时,z_t 的均值比 z_{t-1} 的均值更接近于零, 并且 z_t 的协方差比 z_{t-1} 的协方差更接近于单位矩阵 $\boldsymbol{I}$.

✍ **练习 20.2** (★) 证明(20.1)可以等价地表示为(20.2).

✍ **练习 20.3** (★★★) 使用归纳法证明扩散模型前向过程的 z_t 的边缘分布由(20.7)给出, 其中 α_t 由(20.6)定义. 首先验证当 $t = 1$ 时(20.7)成立. 然后假设对于某个特定的 t 值,(20.7)也成立, 并且推导出 $t + 1$ 时的相应结果. 为此, 最简单的方法是使用(20.3)来写前向过程, 并利用 "两个独立 Gauss 随机变量之和仍是 Gauss 分布、且均值与协方差可加" 这一结果.

✍ **练习 20.4** (★) 利用(20.7), 其中 α_t 由(20.6)定义, 证明在极限 $T \to \infty$ 下可以得到(20.9).

✍ **练习 20.5** (★★) 考虑两个独立的随机变量 $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 以及一个固定的标量 λ, 证明

$$\mathrm{Cov}[\boldsymbol{a} + \boldsymbol{b}] = \mathrm{Cov}[\boldsymbol{a}] + \mathrm{Cov}[\boldsymbol{b}], \tag{20.63}$$

$$\mathrm{Cov}[\lambda \boldsymbol{a}] = \lambda^2 \mathrm{Cov}[\boldsymbol{a}]. \tag{20.64}$$

然后进一步证明: 如果 z_{t-1} 的分布具有零均值和单位协方差, 那么无论 β_t 的值如何,(20.3)定义的 z_t 的分布都具有零均值和单位协方差.

✍ **练习 20.6** (★★★) 使用 "配方法" 这一技术从 Bayes 定理(20.13)中推导出(20.15). 首先注意(20.13)右边分子中的两项, 它们分别由(20.4)和(20.7)给出, 并且它们都采用关于 z_{t-1} 的二次函数的指数形式. 因此所需的分布是 Gauss 分布, 我们只需要求出其均值和协方差即可. 为此, 请只考虑指数中依赖于 z_{t-1} 的项, 注意两个指数的乘积等于这两个指数之和的指数形式. 把所有关于 z_{t-1} 的二次项以及关于 z_{t-1} 的一次项收集起来, 然后重新整理成 $(z_{t-1} - \boldsymbol{m}_t)^{\mathsf{T}} \boldsymbol{S}_t^{-1} (z_{t-1} - \boldsymbol{m}_t)$ 的形式. 最后通过观察, 求出 $\boldsymbol{m}_t(\boldsymbol{x}, z_t)$ 和 $\boldsymbol{S}_t$ 的表达式. 注意, 与 z_{t-1} 无关的加法项可以忽略不

计.

✍ **练习 20.7** (★★★) 证明扩散模型中正向加噪过程的条件分布 $q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1})$ 的逆, 在噪声方差较小时可以近似为 Gauss 分布. 考虑由 Bayes 定理(20.11)给出的逆条件分布 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t)$, 而正向分布 $q(\boldsymbol{z}_t \mid \boldsymbol{z}_{t-1})$ 由(20.4)给出. 对(20.11)的两边取对数, 然后以 $\boldsymbol{z}_t$ 为中心对 $q(\boldsymbol{z}_{t-1})$ 进行 Taylor 展开, 证明对于小的噪声方差 β_t, 分布 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t)$ 可以近似为均值为 $\boldsymbol{z}_t$、协方差为 $\beta_t \boldsymbol{I}$ 的 Gauss 分布. 求出关于均值和协方差的最低阶修正项的表达式, 将其表示为 β_t 的幂次展开形式.

✍ **练习 20.8** (★★) 通过将概率的乘积法则(20.23)代入扩散模型的 ELBO 定义(20.21)中, 并利用 KL 散度的定义(20.22), 验证对数似然函数可以写成下界和 KL 散度之和的形式(20.24).

✍ **练习 20.9** (★★) 验证扩散模型的 ELBO [由(20.31)给出] 可以写成(20.33)的形式, 其中 KL 散度由(20.22)定义.

✍ **练习 20.10** (★★) 在推导(20.33)给出的扩散模型的 ELBO 时, 我们省略了(20.26)右边的第一项和第三项, 因为它们与 $\boldsymbol{w}$ 无关. 同样, 我们还省略了(20.30)右边的第二项, 因为它也与 $\boldsymbol{w}$ 无关. 证明如果保留所有这些省略的项,ELBO 中就会增加一个额外的项

$$\mathrm{KL}\left(q(\boldsymbol{z}_T \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_T)\right). \tag{20.65}$$

请注意加噪过程是这样构造的: 设法使分布 $q(\boldsymbol{z}_T \mid \boldsymbol{x})$ 等于 Gauss 分布 $\mathcal{N}(\boldsymbol{z}_T \mid \boldsymbol{0}, \boldsymbol{I})$. 同样, 分布 $p(\boldsymbol{z}_T)$ 定义为等同于 $\mathcal{N}(\boldsymbol{z}_T \mid \boldsymbol{0}, \boldsymbol{I})$, 因此(20.65)中的两个分布相同,KL 散度为 0.

✍ **练习 20.11** (★★) 利用(20.15)所代表的分布 $q(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{x})$ 和(20.18)所代表的分布 $p(\boldsymbol{z}_{t-1} \mid \boldsymbol{z}_t, \boldsymbol{w})$, 证明出现在(20.33)中的一致项中的 KL 散度由(20.34)给出.

✍ **练习 20.12** (★★) 将(20.36)代入(20.16)中, 重写均值 $\boldsymbol{m}_t(\boldsymbol{x}, \boldsymbol{z}_t)$, 使其能够以原始数据向量 $\boldsymbol{x}$ 和噪声 $\boldsymbol{\varepsilon}$ 的形式来表示, 参见(20.37), 其中 $\boldsymbol{\varepsilon}$ 由(20.8)定义.

✍ **练习 20.13** (★★) 证明扩散模型的 ELBO 中的重建项 [(20.40)] 可以写成(20.41)的形式. 考虑用(20.38)替换 $\boldsymbol{\mu}(\boldsymbol{z}_1, \boldsymbol{w}, 1)$, 并用(20.1)替换 $\boldsymbol{x}$, 然后利用 $\alpha_1 = (1 - \beta_1)$, 后者可以由(20.6)得出.

✍ **练习 20.14** (★) 得分函数是由 $\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 定义的, 因此它是一个与输入向量 $\boldsymbol{x}$ 维度相同的向量. 考虑一个矩阵, 其元素由下式给出:

$$M_{ij} = \frac{\partial s_i}{\partial x_j} - \frac{\partial s_j}{\partial x_i}. \tag{20.66}$$

证明如果通过取梯度 $\boldsymbol{s} = \nabla_{\boldsymbol{x}} \phi(\boldsymbol{x})$ 来定义得分函数, 其中 $\phi(\boldsymbol{x})$ 是具有单一输出变量的神经网络的输出, 那么对于所有 i, j, 矩阵元素 M_{ij} 都为 0. 请注意, 如果得分函数 $\boldsymbol{s}(\boldsymbol{x}) = \nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x} \mid \boldsymbol{w})$ 直接用一个深度神经网络来表示, 并且输入与输出的数量相同, 那么只有对角线矩阵元素 $M_{ii} = 0$, 因此网络的输出不一定是某标量函数的梯度.

✍ **练习 20.15** (★★) 考虑使用一个深度神经网络 $\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w})$ 来表示得分函数 [由(20.44)定义], 其中 $\boldsymbol{x}$ 和 $\boldsymbol{s}$ 的维度为 D. 比较以下两种情况在计算得分时的计算复杂度: 一种是具有 D 个输出且直接用来表示得分函数的网络, 另一种是计算单一标量函数 $\phi(\boldsymbol{x}, \boldsymbol{w})$, 并通过自动微分间接计算得分函数的网络. 证明通常后一种方法的计算成本更高.

✍ **练习 20.16** (★★★) 我们无法直接最小化得分函数 [(20.45)], 因为我们不知道真实数据密度 $p(\boldsymbol{x})$ 的函数形式, 因此无法写出得分函数 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{x})$ 的表达式. 然而, 通过使用分部积分 (Hyvärinen, 2005), 我们可以将(20.45)改写为

$$J(\boldsymbol{w}) = \int \left\{ \nabla \cdot \boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) + \frac{1}{2} \left\| \boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w}) \right\|^2 \right\} p(\boldsymbol{x}) \, \mathrm{d}\boldsymbol{x} + \text{const}, \tag{20.67}$$

其中常数项 const 不依赖于网络参数 $\boldsymbol{w}$, 散度 $\nabla \cdot \boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w})$ 定义如下:

$$\nabla \cdot \boldsymbol{s} = \sum_{i=1}^{D} \frac{\partial s_i}{\partial x_i} = \sum_{i=1}^{D} \frac{\partial^2 \ln p(\boldsymbol{x})}{\partial x_i^2}, \tag{20.68}$$

其中 D 是 $\boldsymbol{x}$ 的维度. 首先展开(20.45)中的平方项, 注意涉及 $\|\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w})\|^2$ 的项已经出现在(20.45)中, 而涉及 $\|\boldsymbol{s}_p\|^2$ 的项可以吸收到加法常数中, 这里我们定义 $\boldsymbol{s}_p = \nabla \ln p(\boldsymbol{x})$. 考虑下式对两个函数乘积的导数:

$$\frac{\mathrm{d}}{\mathrm{d}x} \{ p(x) g(x) \} = \frac{\mathrm{d}p(x)}{\mathrm{d}x} g(x) + p(x) \frac{\mathrm{d}g(x)}{\mathrm{d}x}. \tag{20.69}$$

对(20.69)的两边关于 x 进行积分并整理, 得出分部积分公式:

$$\int_{-\infty}^{\infty} \frac{\mathrm{d}p(x)}{\mathrm{d}x} g(x) \, \mathrm{d}x = -\int_{-\infty}^{\infty} \frac{\mathrm{d}g(x)}{\mathrm{d}x} p(x) \, \mathrm{d}x. \tag{20.70}$$

假设 $p(\infty) = p(-\infty) = 0$. 将此结果与定义 $\boldsymbol{s}_p = \nabla \ln p(\boldsymbol{x})$ 一起应用到涉及 $\boldsymbol{s}(\boldsymbol{x}, \boldsymbol{w})^\mathsf{T} \boldsymbol{s}_p$ 的项以完成证明. 请注意, 计算(20.68)中的二阶导数时, 需要对每个导数进行一次反向传播, 因此总的计算成本会随数据空间维度 D 呈二次方增长 (Martens, Sutskever, and Swersky, 2012). 因此, 该损失函数不能直接应用于高维空间, 于是切片得分匹配 (Song et al., 2019) 等技术被开发出来以解决这种低效问题.

✍ **练习 20.17** (★★) 证明: 忽略加性常数, 得分函数损失 [(20.52)] 与(20.51)是等价的. 首先展开(20.51)中的平方项, 然后利用(20.49)证明(20.51)中的 $\boldsymbol{s}^\mathsf{T}\boldsymbol{s}$ 项与通过展开(20.52)中的平方项所得到的相应项是相同的. 另请注意,(20.51)中 $\|\nabla_{\boldsymbol{z}} \ln q_\sigma\|^2$ 这一项与 $\boldsymbol{w}$ 无关, 同样,(20.52)中的相应项也与 $\boldsymbol{w}$ 无关, 因此这些项可以视为损失函数中的加性常数, 它们在训练中不起作用. 最后考虑(20.51)中的交叉项. 通过使用(20.49)替换 $q_\sigma(\boldsymbol{z})$, 证明这等于(20.52)中相应的交叉项, 从而证明这两个损失函数在忽略加性常数的情况下是相等的.

✍ **练习 20.18** (★) 考虑一个概率分布, 它由两个互斥的分布 (换言之, 当其中一个非零时, 另一个必定为零) 混合而成, 形式为

$$p(\boldsymbol{x}) = \lambda p_a(\boldsymbol{x}) + (1 - \lambda) p_b(\boldsymbol{x}). \tag{20.71}$$

证明当针对任何给定点 $\boldsymbol{x}$ 计算由(20.44)定义的得分函数时, 混合系数 λ 不会出现. 由此可知, 由(14.61)定义的 Langevin 动力学方法不会以正确的比例从这两个分布中采样. 如正文所述, 我们可以通过添加来自宽泛分布的噪声来解决这个问题.

✍ **练习 20.19** (★★) 在离散步骤, 扩散模型中的正向加噪过程由(20.3)定义. 这里我们取连续时间极限, 并将其转换为随机微分方程 (SDE). 为此, 首先引入一个连续变化的方差函数 $\beta(t)$, 使得 $\beta_t = \beta(t)\Delta t$. 通过对(20.3)右边第一项中的平方根进行 Taylor 展开, 证明无穷小的更新可以写成如下形式:

$$\mathrm{d}\boldsymbol{z} = -\frac{1}{2}\beta(t)\,\boldsymbol{z}\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{w}, \tag{20.72}$$

这是一般随机微分方程 [(20.57)] 的特殊情况.

✍ **练习 20.20** (★) 用(20.60)替换 $\nabla_{\boldsymbol{x}} \ln p(\boldsymbol{c} \mid \boldsymbol{x})$, 证明(20.61)中的得分函数可以写成(20.62)的形式.

附录 A 线性代数

本附录列出了一些有用的性质以及涉及矩阵和行列式的恒等式. 我们假设读者已经熟悉了基本的线性代数知识, 所以本附录并不是介绍这些概念的教程. 对于某些结论, 我们会说明如何进行证明, 而在更复杂的情况下, 建议感兴趣的读者参考与该主题有关的教科书. 在所有情况下, 我们假设矩阵的逆存在, 并且假设矩阵的维度可以使公式得到正确定义. 关于线性代数的更详细且全面的讨论可以参见 Golub and Van Loan (1996), 并且 Lütkepohl (1996) 给出了大量的矩阵性质,Magnus and Neudecker (1999) 讨论了矩阵导数.

A.1 矩阵恒等式

矩阵 $\boldsymbol{A}$ 由元素 A_{ij} 组成, 其中 i 索引行,j 索引列.

定义 A.1 (恒等矩阵)

我们用 $\boldsymbol{I}_N$ 来表示 $N \times N$ 的恒等矩阵 (也称单位矩阵). 当维度上没有歧义的时候, 我们可以直接用 $\boldsymbol{I}$ 代表单位矩阵.

♣

注

(重构新增) $\boldsymbol{I}_N$ 是矩阵乘法的单位元: 对任意 $N \times N$ 矩阵 $\boldsymbol{A}$ 都有 $\boldsymbol{A}\boldsymbol{I}_N = \boldsymbol{I}_N\boldsymbol{A} = \boldsymbol{A}$, 这正是“恒等”一词的含义. 定义中保留下标 N 并非多余: 在下文的(A.14)中会同时出现 $\boldsymbol{I}_N$ 与 $\boldsymbol{I}_M$, 两者阶数不同, 只有写清下标才能判断运算是否合法; 而在(A.2)这类只涉及单个方阵的场合,$\boldsymbol{A}$ 的阶数已由上下文确定, 于是简写为 $\boldsymbol{I}$.

定义 A.2 (转置)

转置矩阵 $\boldsymbol{A}^{\mathsf{T}}$ 包含元素 $(\boldsymbol{A}^{\mathsf{T}})_{ij} = A_{ji}$.

♣

注

(重构新增) 转置把 $\boldsymbol{A}$ 的第 i 行第 j 列元素搬到第 j 行第 i 列, 即把“行”与“列”的角色互换. 这个定义只要求 $\boldsymbol{A}$ 是任意 (不必方、不必可逆的) 矩阵, 所以转置总是存在且唯一; 它也为定义 A.12提供了表述工具——对称矩阵就是满足 $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}$ 的方阵.

从转置的定义来看, 我们有

命题 A.1 (乘积的转置)

$$(\boldsymbol{A}\boldsymbol{B})^{\mathsf{T}} = \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}} \tag{A.1}$$

♠

证明

(重构新增) 原书只给出“通过写出索引来验证”的提示, 下面补全. 设 $\boldsymbol{A}$ 是 $K \times L$ 矩阵、$\boldsymbol{B}$ 是 $L \times M$ 矩阵, 则 $\boldsymbol{A}\boldsymbol{B}$ 是 $K \times M$ 矩阵, 且

$$(\boldsymbol{A}\boldsymbol{B})_{ij} = \sum_{l=1}^{L} A_{il}B_{lj}.$$

于是 $\left((\boldsymbol{A}\boldsymbol{B})^{\mathsf{T}}\right)_{ij} = (\boldsymbol{A}\boldsymbol{B})_{ji} = \sum_{l} A_{jl}B_{li}$. 另一方面,

$$(\boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}})_{ij} = \sum_{l=1}^{L}(\boldsymbol{B}^{\mathsf{T}})_{il}(\boldsymbol{A}^{\mathsf{T}})_{lj} = \sum_{l=1}^{L} B_{li}A_{jl} = \sum_{l=1}^{L} A_{jl}B_{li},$$

两者对一切 i, j 相等, 故(A.1)成立.

□

注

(重构新增) (A.1)表明转置把乘积的因子顺序颠倒. 条件只有一条:$\boldsymbol{AB}$ 必须可乘, 即 $\boldsymbol{A}$ 的列数等于 $\boldsymbol{B}$ 的行数; 若把因子顺序写反为 $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{B}^{\mathsf{T}}$, 则连乘法本身都可能没有定义. 这条恒等式在命题 A.3与命题 A.24的证明中反复使用.

$\boldsymbol{A}$ 的逆矩阵记为 $\boldsymbol{A}^{-1}$, 满足

定义 A.3 (逆矩阵)

$$\boldsymbol{A}\boldsymbol{A}^{-1}=\boldsymbol{A}^{-1}\boldsymbol{A}=\boldsymbol{I} \tag{A.2}$$

注

(重构新增) 定义要求 $\boldsymbol{A}\boldsymbol{A}^{-1}$ 与 $\boldsymbol{A}^{-1}\boldsymbol{A}$ 同时等于 $\boldsymbol{I}$, 这隐含 $\boldsymbol{A}$ 是方阵: 只有方阵与自身同阶相乘才得到同阶单位矩阵. 因此“矩阵的逆存在”是本附录开篇所约定的前提之一; 由命题 A.9可知, 方阵可逆当且仅当 $|\boldsymbol{A}|\neq 0$. 若 $\boldsymbol{A}^{-1}$ 存在则必唯一: 若有 $\boldsymbol{A}\boldsymbol{X}=\boldsymbol{I}$ 与 $\boldsymbol{Y}\boldsymbol{A}=\boldsymbol{I}$, 则 $\boldsymbol{X}=\boldsymbol{Y}\boldsymbol{A}\boldsymbol{X}=\boldsymbol{Y}$, 这一唯一性在下面几条恒等式的证明中要用到.

因为 $\boldsymbol{A}\boldsymbol{B}\boldsymbol{B}^{-1}\boldsymbol{A}^{-1}=\boldsymbol{I}$, 所以有

命题 A.2 (乘积的逆)

$$(\boldsymbol{A}\boldsymbol{B})^{-1}=\boldsymbol{B}^{-1}\boldsymbol{A}^{-1} \tag{A.3}$$

证明

(重构新增) 原书只写出 $\boldsymbol{A}\boldsymbol{B}\boldsymbol{B}^{-1}\boldsymbol{A}^{-1}=\boldsymbol{I}$ 这一步, 下面补全双向验证. 由结合律,

$$\boldsymbol{A}\boldsymbol{B}\boldsymbol{B}^{-1}\boldsymbol{A}^{-1}=\boldsymbol{A}(\boldsymbol{B}\boldsymbol{B}^{-1})\boldsymbol{A}^{-1}=\boldsymbol{A}\boldsymbol{I}\boldsymbol{A}^{-1}=\boldsymbol{A}\boldsymbol{A}^{-1}=\boldsymbol{I},$$

同理 $\boldsymbol{B}^{-1}\boldsymbol{A}^{-1}\boldsymbol{A}\boldsymbol{B}=\boldsymbol{B}^{-1}(\boldsymbol{A}^{-1}\boldsymbol{A})\boldsymbol{B}=\boldsymbol{B}^{-1}\boldsymbol{B}=\boldsymbol{I}$. 按定义 A.3,$\boldsymbol{B}^{-1}\boldsymbol{A}^{-1}$ 同时是 $\boldsymbol{A}\boldsymbol{B}$ 的左逆与右逆, 再由逆的唯一性即得(A.3).

□

并且有

命题 A.3 (转置的逆)

$$(\boldsymbol{A}^{\mathsf{T}})^{-1}=(\boldsymbol{A}^{-1})^{\mathsf{T}} \tag{A.4}$$

证明

这通过取(A.2)的转置并应用(A.1)可以很容易得证. 补全细节: 对 $\boldsymbol{A}\boldsymbol{A}^{-1}=\boldsymbol{I}$ 两边取转置, 由(A.1)得 $(\boldsymbol{A}^{-1})^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}=\boldsymbol{I}^{\mathsf{T}}=\boldsymbol{I}$; 对 $\boldsymbol{A}^{-1}\boldsymbol{A}=\boldsymbol{I}$ 两边取转置得 $\boldsymbol{A}^{\mathsf{T}}(\boldsymbol{A}^{-1})^{\mathsf{T}}=\boldsymbol{I}$. 于是 $(\boldsymbol{A}^{-1})^{\mathsf{T}}$ 是 $\boldsymbol{A}^{\mathsf{T}}$ 的逆, 即(A.4)成立.

□

注

(重构新增) 命题 A.2 and A.3共同说明: 求逆与转置、求逆与乘法可以按相反的顺序交换. 两条恒等式都只假定所涉及的逆存在; 特别地,(A.3)要求 $\boldsymbol{A}$、$\boldsymbol{B}$ 都可逆, 这比“$\boldsymbol{AB}$ 可乘”更强. 反过来, 两个不可逆方阵的乘积是否可能可逆? 不可能: 由(A.12),$|\boldsymbol{A}\boldsymbol{B}|=|\boldsymbol{A}||\boldsymbol{B}|$, 只要有一个因子行列式为零, 乘积就不可逆.

一个涉及矩阵的逆的有用恒等式如下:

命题 A.4 (矩阵求逆的一个恒等式)

$$(\boldsymbol{P}^{-1}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B})^{-1}\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}=\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}(\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{R})^{-1} \tag{A.5}$$

证明

这很容易通过对(A.5)的两边右乘 $(\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{R})$ 得证. 补全如下: 右边右乘后显然等于 $\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}$; 左边右乘后为

$$(\boldsymbol{P}^{-1}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B})^{-1}\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}(\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{R}).$$

注意 $\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}(\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{R})=\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{B}^{\mathsf{T}}$, 而

$$(\boldsymbol{P}^{-1}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B})\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}=\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}},$$

故括号内的矩阵等于 $(\boldsymbol{P}^{-1}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B})\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}$, 与前面的逆因子相消后, 左边右乘的结果也是 $\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}$. 两边右乘同一个可逆矩阵 $(\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{R})$ 后相等, 故(A.5)成立.

□

注

(重构新增) (A.5)的价值在于“换一个阶数更小的矩阵求逆”. 假设 $\boldsymbol{P}$ 的维度为 $N\times N$、$\boldsymbol{R}$ 的维度为 $M\times M$, 则 $\boldsymbol{B}$ 的维度就是 $M\times N$: 此时左边要对 $N\times N$ 矩阵 $\boldsymbol{P}^{-1}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B}$ 求逆, 右边只对 $M\times M$ 矩阵 $\boldsymbol{B}\boldsymbol{P}\boldsymbol{B}^{\mathsf{T}}+\boldsymbol{R}$ 求逆. 如果 $M\ll N$, 那么计算(A.5)右边的值要比计算其左边的值开销小得多. 恒等式成立的条件是 $\boldsymbol{P}$、$\boldsymbol{R}$ 以及 $\boldsymbol{P}^{-1}+\boldsymbol{B}^{\mathsf{T}}\boldsymbol{R}^{-1}\boldsymbol{B}$ 均可逆; 在应用中 $\boldsymbol{P}$、$\boldsymbol{R}$ 通常是协方差矩阵, 可逆性由它们正定来保证 (参见定义 A.14).

有时也会出现如下特殊情况:

命题 A.5 (推入恒等式)

$$(\boldsymbol{I}+\boldsymbol{A}\boldsymbol{B})^{-1}\boldsymbol{A}=\boldsymbol{A}(\boldsymbol{I}+\boldsymbol{B}\boldsymbol{A})^{-1} \tag{A.6}$$

♠

证明

(重构新增) 原书未给证明. 注意 $(\boldsymbol{I}+\boldsymbol{A}\boldsymbol{B})\boldsymbol{A}=\boldsymbol{A}+\boldsymbol{A}\boldsymbol{B}\boldsymbol{A}=\boldsymbol{A}(\boldsymbol{I}+\boldsymbol{B}\boldsymbol{A})$. 在 $\boldsymbol{I}+\boldsymbol{A}\boldsymbol{B}$ 与 $\boldsymbol{I}+\boldsymbol{B}\boldsymbol{A}$ 均可逆时, 两边左乘 $(\boldsymbol{I}+\boldsymbol{A}\boldsymbol{B})^{-1}$、右乘 $(\boldsymbol{I}+\boldsymbol{B}\boldsymbol{A})^{-1}$, 即得(A.6).

□

注

(重构新增) (A.6)常被称为“推入 (push-through)”恒等式: 它把夹在中间的 $\boldsymbol{A}$ 从左侧“推”到右侧, 代价是把 $\boldsymbol{A}\boldsymbol{B}$ 换成 $\boldsymbol{B}\boldsymbol{A}$. 当 $\boldsymbol{A}$ 是 $N\times M$、$\boldsymbol{B}$ 是 $M\times N$ 时,$\boldsymbol{A}\boldsymbol{B}$ 是 $N\times N$ 而 $\boldsymbol{B}\boldsymbol{A}$ 是 $M\times M$, 因此可以挑选阶数较小的那个求逆. 它只要求两个括号内的矩阵可逆, 不要求 $\boldsymbol{A}$、$\boldsymbol{B}$ 本身可逆或为方阵.

另一个有用的关于矩阵的逆的恒等式如下:

命题 A.6 (Woodbury 恒等式)

$$(\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C})^{-1}=\boldsymbol{A}^{-1}-\boldsymbol{A}^{-1}\boldsymbol{B}(\boldsymbol{D}+\boldsymbol{C}\boldsymbol{A}^{-1}\boldsymbol{B})^{-1}\boldsymbol{C}\boldsymbol{A}^{-1} \tag{A.7}$$

♠

证明

原书指出: 对(A.7)的两边同时乘以 $(\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C})$ 即可得证. 补全如下: 右边乘以 $(\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C})$ 得

$$\boldsymbol{A}^{-1}(\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C})-\boldsymbol{A}^{-1}\boldsymbol{B}(\boldsymbol{D}+\boldsymbol{C}\boldsymbol{A}^{-1}\boldsymbol{B})^{-1}\boldsymbol{C}\boldsymbol{A}^{-1}(\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}).$$

第一项等于 $\boldsymbol{I}+\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}$; 第二项中 $\boldsymbol{C}\boldsymbol{A}^{-1}(\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C})=\boldsymbol{C}+\boldsymbol{C}\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}=(\boldsymbol{D}+\boldsymbol{C}\boldsymbol{A}^{-1}\boldsymbol{B})\boldsymbol{D}^{-1}\boldsymbol{C}$, 故第二项等于 $\boldsymbol{A}^{-1}\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}$. 两项相减即得 $\boldsymbol{I}$, 于是(A.7)右边确是 $(\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C})$ 的逆.

□

注

(重构新增) 这就是 **Woodbury 恒等式**, 它把“先加上低秩修正再求逆”换成“先对小矩阵求逆再作低秩修正”. 它在某些情况下很有用, 例如, 当 $\boldsymbol{A}$ 很大且为对角矩阵时,$\boldsymbol{A}$ 的逆很容易求得, 或者当 $\boldsymbol{B}$ 有很多行但列较少时 ($\boldsymbol{C}$ 则相反),(A.7)右边的计算开销就比左边的计算开销小得多. 成立条件为 $\boldsymbol{A}$、$\boldsymbol{D}$ 以及 $\boldsymbol{D}+\boldsymbol{C}\boldsymbol{A}^{-1}\boldsymbol{B}$ 均可逆, 此时 $\boldsymbol{A}+\boldsymbol{B}\boldsymbol{D}^{-1}\boldsymbol{C}$ 也自动可逆; 若 $\boldsymbol{A}$ 只是可逆而不对称, 并不要求 $\boldsymbol{B}$、$\boldsymbol{C}$ 之间有任何对称关系.

定义 A.4 (线性无关)

如果当且仅当所有 $\alpha_i = 0$ 时,$\sum_i \alpha_i \boldsymbol{a}_i = \boldsymbol{0}$ 才成立, 则称向量组 $\{\boldsymbol{a}_1, \cdots, \boldsymbol{a}_M\}$ 是**线性无关** (linearly independent) 的.

这意味着没有任何一个向量可以表示为其余向量的线性组合.

注

(重构新增) "当且仅当"给出了两个方向: 系数组全为零时组合必为零向量 (这是平凡的), 以及组合为零向量时系数只能全为零 (这是实质条件). 后一方向正是"无法用一个向量去替代其余向量"的定量表述: 若 $\boldsymbol{a}_M = \sum_{i<M} c_i \boldsymbol{a}_i$, 则取 $\alpha_M = -1$、$\alpha_i = c_i$ 就得到一组不全为零的系数使组合为零, 线性无关性随即失效. 因此线性无关刻画的是向量组中"没有冗余成员", 这与后文矩阵的秩直接相关 (参见定义 A.5).

定义 A.5 (矩阵的秩)

矩阵的秩等于线性无关行的最大数量 (也可等价地说, 矩阵的秩等于线性无关列的最大数量).

注

(重构新增) 定义中括号内的等价说法并非显然: 它断言"行秩等于列秩". 这个结论保证秩是矩阵本身的不变量, 而不依赖于我们按行还是按列去数. 定义的关键量是"最大数量", 即从所有线性无关的行子集中取元素个数最多的那一个; 零矩阵的秩为零. 秩在命题 A.20中会与特征值发生联系, 但那里的联系需要额外条件.

A.2 迹和行列式

方阵有迹和行列式.

定义 A.6 (迹)

矩阵 $\boldsymbol{A}$ 的迹 $\mathrm{tr}(\boldsymbol{A})$ 定义为其主对角线上元素的和.

注

(重构新增) 迹只对 $N \times N$ 的方阵有定义, 它把整个矩阵压缩成一个标量, 并且对矩阵的元素是线性的:$\mathrm{tr}(\alpha\boldsymbol{A}+\beta\boldsymbol{B}) = \alpha\,\mathrm{tr}(\boldsymbol{A}) + \beta\,\mathrm{tr}(\boldsymbol{B})$. 正因为是线性的、又只取对角线, 迹在求导运算中格外方便:命题 A.16与命题 A.18都要把矩阵导数化为迹的形式.

通过写出索引, 我们可以得到

命题 A.7 (迹的循环性)

$$\mathrm{tr}(\boldsymbol{AB}) = \mathrm{tr}(\boldsymbol{BA}) \tag{A.8}$$

通过将(A.8)应用于三个矩阵的乘积, 可以得到

$$\mathrm{tr}(\boldsymbol{ABC}) = \mathrm{tr}(\boldsymbol{CAB}) = \mathrm{tr}(\boldsymbol{BCA}) \tag{A.9}$$

这就是迹运算的循环 (cyclic) 性质. 显然, 这一性质可以扩展到任意数量的矩阵的乘积.

证明

(重构新增) 原书只说“通过写出索引”即可, 下面补全. 设 $\boldsymbol{A}$ 为 $K\times L$、$\boldsymbol{B}$ 为 $L\times K$, 则 $\boldsymbol{AB}$ 与 $\boldsymbol{BA}$ 都是方阵, 且

$$\operatorname{tr}(\boldsymbol{AB})=\sum_{k=1}^{K}(\boldsymbol{AB})_{kk}=\sum_{k=1}^{K}\sum_{l=1}^{L}A_{kl}B_{lk}=\sum_{l=1}^{L}\sum_{k=1}^{K}B_{lk}A_{kl}=\sum_{l=1}^{L}(\boldsymbol{BA})_{ll}=\operatorname{tr}(\boldsymbol{BA}),$$

即(A.8). 再把(A.8)用于矩阵 $\boldsymbol{AB}$ 与 $\boldsymbol{C}$, 得 $\operatorname{tr}(\boldsymbol{ABC})=\operatorname{tr}(\boldsymbol{CAB})$; 把(A.8)用于 $\boldsymbol{B}$ 与 $\boldsymbol{CA}$, 得 $\operatorname{tr}(\boldsymbol{BCA})=\operatorname{tr}(\boldsymbol{CAB})$, 于是(A.9)成立.

□

注

(重构新增) (A.8)要求 $\boldsymbol{AB}$ 与 $\boldsymbol{BA}$ 都有定义, 这只要 $\boldsymbol{A}$ 的列数与 $\boldsymbol{B}$ 的行数相配即可, 并不要求二者同阶; 但显然要求两个乘积都是方阵, 否则迹无从谈起. 循环性只允许“轮换”因子顺序, 一般不允许任意置换: 例如 $\operatorname{tr}(\boldsymbol{ABC})\neq\operatorname{tr}(\boldsymbol{ACB})$. 这条性质在命题 A.28中用把 $\boldsymbol{A}$ 对角化的方式计算 $\operatorname{tr}(\boldsymbol{A})$ 时是必需的.

$N\times N$ 矩阵 $\boldsymbol{A}$ 的行列式 $|\boldsymbol{A}|$ 由下式定义:

定义 A.7 (行列式)

$$|\boldsymbol{A}|=\sum(\pm1)A_{1i_1}A_{2i_2}\cdots A_{Ni_N} \tag{A.10}$$

♣

其中, 行列式对每行和每列中元素的乘积求和, 并根据排列 $i_1i_2\cdots i_N$ 为偶数还是奇数来确定系数是 $+1$ 还是 -1.

注

(重构新增) (A.10)中的求和遍历 $1,\cdots,N$ 的全部 $N!$ 个排列 $(i_1,\cdots,i_N)$: 下标 i_k 表示第 k 行所取元素所在的列, 因此每一项恰好从每一行、每一列各取一个元素. 系数 $+1$ 或 -1 由排列的奇偶性 (逆序数的奇偶性) 决定, 这就是定义中“±1”的来源. 定义只对 $N\times N$ 方阵有意义; 由定义立刻可以看出 $|\boldsymbol{I}|=1$(只有恒等排列给出非零项且其系数为 $+1$), 并且对角矩阵的行列式等于其主对角线上元素的乘积 (非恒等排列至少有一个因子取自非对角位置, 取值为零).

注意 $|\boldsymbol{I}|=1$, 且对角矩阵的行列式是由其主对角线上元素的乘积给出的. 因此, 对于 2×2 的矩阵, 其行列式的形式如下:

$$|\boldsymbol{A}|=\begin{vmatrix}a_{11}&a_{12}\\a_{21}&a_{22}\end{vmatrix}=a_{11}a_{22}-a_{12}a_{21} \tag{A.11}$$

两个矩阵乘积的行列式由下式给出:

命题 A.8 (乘积的行列式)

$$|\boldsymbol{AB}|=|\boldsymbol{A}|\,|\boldsymbol{B}| \tag{A.12}$$

♠

证明

原书指出(A.12)可以从(A.10)推导得出. 下面补全思路: 按(A.10)把 $|\boldsymbol{AB}|$ 展开为

$$|\boldsymbol{AB}|=\sum_{i_1,\cdots,i_N}(\pm1)\prod_{k=1}^{N}\Big(\sum_{j_k=1}^{N}A_{kj_k}B_{j_kk}\Big),$$

交换求和次序后, 每一项是某排列 $(j_1,\cdots,j_N)$ 下的 $\prod_k B_{j_kk}$ 与 $\prod_k A_{kj_k}$ 之积; 只有当 $j_1,\cdots,j_N$ 互不相同时乘积才可能非零 (否则 $\prod_k B_{j_kk}$ 必含同一列的两个因子, 这样的项在按列重组后系数相消), 而这些项恰好按排列重组为 $|\boldsymbol{A}|\,|\boldsymbol{B}|$ 的展开式, 符号也相乘一致.

□

同样, 逆矩阵的行列式由下式给出:

命题 A.9 (逆矩阵的行列式)

$$|\boldsymbol{A}^{-1}| = \frac{1}{|\boldsymbol{A}|} \tag{A.13}$$

♠

证明

这可以通过取(A.2)的行列式并应用(A.12)推导得出: 由 $\boldsymbol{A}\boldsymbol{A}^{-1} = \boldsymbol{I}$ 得 $|\boldsymbol{A}||\boldsymbol{A}^{-1}| = |\boldsymbol{I}| = 1$, 故 $|\boldsymbol{A}^{-1}| = 1/|\boldsymbol{A}|$. 这也说明 $\boldsymbol{A}$ 可逆的必要条件是 $|\boldsymbol{A}| \neq 0$.

□

注

(重构新增) 命题 A.8 and A.9说明行列式把乘法变成乘法、把求逆变成取倒数, 因此它是方阵可逆性的判别量:$|\boldsymbol{A}| = 0$ 时 $\boldsymbol{A}$ 不可能有逆 (否则与(A.13)矛盾). 反过来,$|\boldsymbol{A}| \neq 0$ 时 $\boldsymbol{A}$ 必可逆, 这一点本书不证, 属于线性代数基本结论. 注意(A.12)要求两个因子都是同阶方阵, 否则乘积的行列式没有定义.

如果 $\boldsymbol{A}$ 和 $\boldsymbol{B}$ 都是大小为 $N \times M$ 的矩阵, 则有

命题 A.10 (Sylvester 行列式恒等式)

$$|\boldsymbol{I}_N + \boldsymbol{A}\boldsymbol{B}^{\mathsf{T}}| = |\boldsymbol{I}_M + \boldsymbol{A}^{\mathsf{T}}\boldsymbol{B}| \tag{A.14}$$

♠

证明

(重构新增) 原书未给证明. 设 $\boldsymbol{A}, \boldsymbol{B}$ 都是 $N \times M$ 矩阵, 考虑 $(N+M)\times(N+M)$ 的分块矩阵

$$\boldsymbol{M} = \begin{pmatrix} \boldsymbol{I}_N & \boldsymbol{A} \\ -\boldsymbol{B}^{\mathsf{T}} & \boldsymbol{I}_M \end{pmatrix}.$$

用分块下三角矩阵 $\begin{pmatrix} \boldsymbol{I}_N & \boldsymbol{0} \\ \boldsymbol{B}^{\mathsf{T}} & \boldsymbol{I}_M \end{pmatrix}$ 左乘 $\boldsymbol{M}$, 由分块乘法得

$$\begin{pmatrix} \boldsymbol{I}_N & \boldsymbol{0} \\ \boldsymbol{B}^{\mathsf{T}} & \boldsymbol{I}_M \end{pmatrix} \boldsymbol{M} = \begin{pmatrix} \boldsymbol{I}_N & \boldsymbol{A} \\ \boldsymbol{0} & \boldsymbol{I}_M + \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A} \end{pmatrix},$$

右端是分块上三角矩阵, 其行列式等于对角块行列式之积 $|\boldsymbol{I}_M + \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}|$; 左乘因子的行列式为 1, 故 $|\boldsymbol{M}| = |\boldsymbol{I}_M + \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}|$. 同理, 用分块上三角矩阵 $\begin{pmatrix} \boldsymbol{I}_N & \boldsymbol{0} \\ \boldsymbol{B}^{\mathsf{T}} & \boldsymbol{I}_M \end{pmatrix}$ 右乘 $\boldsymbol{M}$ 得

$$\boldsymbol{M} \begin{pmatrix} \boldsymbol{I}_N & \boldsymbol{0} \\ \boldsymbol{B}^{\mathsf{T}} & \boldsymbol{I}_M \end{pmatrix} = \begin{pmatrix} \boldsymbol{I}_N + \boldsymbol{A}\boldsymbol{B}^{\mathsf{T}} & \boldsymbol{A} \\ \boldsymbol{0} & \boldsymbol{I}_M \end{pmatrix},$$

故 $|\boldsymbol{M}| = |\boldsymbol{I}_N + \boldsymbol{A}\boldsymbol{B}^{\mathsf{T}}|$. 于是 $|\boldsymbol{I}_N + \boldsymbol{A}\boldsymbol{B}^{\mathsf{T}}| = |\boldsymbol{I}_M + \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}|$. 最后, 由(A.10)可直接看出转置不改变行列式 (排列 σ 与 σ^{-1} 的项一一对应且符号相同), 故 $|\boldsymbol{I}_M + \boldsymbol{B}^{\mathsf{T}}\boldsymbol{A}| = |\boldsymbol{I}_M + \boldsymbol{A}^{\mathsf{T}}\boldsymbol{B}|$, 即(A.14).

□

注

(重构新增) (A.14)的用途是: 把 $N \times N$ 的行列式换成 $M \times M$ 的行列式, 两边的值相同但计算量可以相差很多, 其思想与命题 A.4完全一致. 条件只有“$\boldsymbol{A}$、$\boldsymbol{B}$ 都是 $N \times M$”: 若两者阶数不同, 或其中一个不是矩阵而是向量, 公式中相应的 $\boldsymbol{I}$ 就必须换阶数 (见命题 A.11).

一种有用的特殊情况如下:

命题 A.11 (秩 1 行列式恒等式)

$$|\boldsymbol{I}_N + \boldsymbol{a}\boldsymbol{b}^{\mathsf{T}}| = 1 + \boldsymbol{a}^{\mathsf{T}}\boldsymbol{b} \tag{A.15}$$

其中 $\boldsymbol{a}$ 和 $\boldsymbol{b}$ 都是 N 维的列向量.

♠

证明

(重构新增) 把(A.14)用于 $M=1$ 的情形: 取 $\boldsymbol{A}=\boldsymbol{a}$、$\boldsymbol{B}=\boldsymbol{b}$(都是 $N\times 1$ 矩阵), 则 $\boldsymbol{A}\boldsymbol{B}^{\mathsf{T}}=\boldsymbol{a}\boldsymbol{b}^{\mathsf{T}}$ 是 $N\times N$ 矩阵, 而 $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{B}=\boldsymbol{a}^{\mathsf{T}}\boldsymbol{b}$ 是 1×1 矩阵. 由(A.14)得 $|\boldsymbol{I}_N+\boldsymbol{a}\boldsymbol{b}^{\mathsf{T}}|=|\boldsymbol{I}_1+\boldsymbol{a}^{\mathsf{T}}\boldsymbol{b}|=1+\boldsymbol{a}^{\mathsf{T}}\boldsymbol{b}$, 即(A.15).

□

注

(重构新增) (A.15)左端是一个 $N\times N$ 行列式, 右端只是一个标量, 因此它对任意大的 N 都给出了闭式结果. 它要求 $\boldsymbol{a}$、$\boldsymbol{b}$ 是同一维数的列向量 (否则 $\boldsymbol{a}\boldsymbol{b}^{\mathsf{T}}$ 与 $\boldsymbol{a}^{\mathsf{T}}\boldsymbol{b}$ 之一无定义). 这个式子在概率模型中用于验证 Gauss 分布的归一化常数——形如 $|\boldsymbol{I}+\boldsymbol{a}\boldsymbol{b}^{\mathsf{T}}|$ 的因子可以立刻化为一元表达式.

A.3 矩阵导数

有时我们需要考虑向量和矩阵关于标量的导数. 向量 $\boldsymbol{a}$ 关于标量 x 的导数也是一个向量, 其分量为

定义 A.8 (向量与矩阵关于标量的导数)

$$\left(\frac{\partial \boldsymbol{a}}{\partial x}\right)_i=\frac{\partial a_i}{\partial x} \tag{A.16}$$

矩阵关于标量的导数也有类似的定义, 即逐元素求导.

♣

注

(重构新增) 这里约定“对矩阵求导就是逐元素求导”, 于是 $\partial\boldsymbol{A}/\partial x$ 与 $\boldsymbol{A}$ 同型. 这条约定使下面的乘积法则能够逐分量地成立, 也使(A.21)这类矩阵恒等式可以通过对恒等式两边整体求导、再按矩阵乘法重组而得到. 若采用“分子布局/分母布局”等其它约定, 转置关系会随之改变, 本附录统一采用上述逐元素约定.

标量 x 关于向量 $\boldsymbol{a}$ 的导数可以定义为

定义 A.9 (标量关于向量的导数)

$$\left(\frac{\partial x}{\partial \boldsymbol{a}}\right)_i=\frac{\partial x}{\partial a_i} \tag{A.17}$$

♣

类似地:

定义 A.10 (向量关于向量的导数)

$$\left(\frac{\partial \boldsymbol{a}}{\partial \boldsymbol{b}}\right)_{ij}=\frac{\partial a_i}{\partial b_j} \tag{A.18}$$

♣

注

(重构新增) 定义 A.9 and A.10给出的是“结果的每个分量对应哪个偏导数”的约定. 注意 $\partial x/\partial\boldsymbol{a}$ 是与 $\boldsymbol{a}$ 同维的向量, 而 $\partial\boldsymbol{a}/\partial\boldsymbol{b}$ 是 i 为行、j 为列的矩阵 (Jacobi 矩阵的形状); 这个形状约定决定了(A.23)中下标 i,j 的排列次序, 也决定了(A.24)右端为何是 $\boldsymbol{B}^{\mathsf{T}}$ 而不是 $\boldsymbol{B}$.

通过写出分量, 易证

命题 A.12 (线性型的导数)

$$\frac{\partial}{\partial \boldsymbol{x}}(\boldsymbol{x}^{\mathsf{T}}\boldsymbol{a})=\frac{\partial}{\partial \boldsymbol{x}}(\boldsymbol{a}^{\mathsf{T}}\boldsymbol{x})=\boldsymbol{a} \tag{A.19}$$

♠

证明

(重构新增) 原书只说“易证”，下面补全. 由于 $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{a} = \boldsymbol{a}^{\mathsf{T}}\boldsymbol{x} = \sum_j a_j x_j$，对第 i 个分量求偏导得 $\partial\big(\sum_j a_j x_j\big)/\partial x_i = a_i$，故梯度向量的第 i 个分量等于 a_i，即(A.19).

□

注

(重构新增) (A.19)说明线性函数的梯度是常向量，这是所有一阶展开的出发点：它表明 $\boldsymbol{x}^{\mathsf{T}}\boldsymbol{a}$ 的导数与 $\boldsymbol{x}$ 无关. 该式对任意维数、任意 $\boldsymbol{a}$ 都成立，不需要任何可逆性或对称性假设.

类似地:

命题 A.13 (矩阵乘积的导数)

$$\frac{\partial}{\partial x}(\boldsymbol{A}\boldsymbol{B}) = \frac{\partial \boldsymbol{A}}{\partial x}\boldsymbol{B} + \boldsymbol{A}\frac{\partial \boldsymbol{B}}{\partial x} \tag{A.20}$$

证明

(重构新增) 对 (i,j) 分量用标量乘积法则:$(\boldsymbol{A}\boldsymbol{B})_{ij} = \sum_k A_{ik}B_{kj}$，故

$$\frac{\partial}{\partial x}(\boldsymbol{A}\boldsymbol{B})_{ij} = \sum_k \Big(\frac{\partial A_{ik}}{\partial x}B_{kj} + A_{ik}\frac{\partial B_{kj}}{\partial x}\Big),$$

右端正是 $\big(\frac{\partial \boldsymbol{A}}{\partial x}\boldsymbol{B} + \boldsymbol{A}\frac{\partial \boldsymbol{B}}{\partial x}\big)_{ij}$，即(A.20).

□

注

(重构新增) (A.20)与标量乘积法则形式相同，但因子顺序不能交换 (矩阵乘法不交换). 它要求 $\boldsymbol{A}\boldsymbol{B}$ 可乘，且 $\boldsymbol{A}$、$\boldsymbol{B}$ 都关于同一个 x 可导. 下面(A.21)的推导正是把这条法则用于恒等式 $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$.

矩阵的逆关于标量 x 的导数可以定义为

命题 A.14 (逆矩阵的导数)

$$\frac{\partial}{\partial x}(\boldsymbol{A}^{-1}) = -\boldsymbol{A}^{-1}\frac{\partial \boldsymbol{A}}{\partial x}\boldsymbol{A}^{-1} \tag{A.21}$$

证明

这可以通过使用(A.20)微分方程 $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$，然后右乘 $\boldsymbol{A}^{-1}$ 得证. 补全如下: 对 $\boldsymbol{A}^{-1}\boldsymbol{A} = \boldsymbol{I}$ 两边关于 x 求导，由(A.20)得

$$\frac{\partial \boldsymbol{A}^{-1}}{\partial x}\boldsymbol{A} + \boldsymbol{A}^{-1}\frac{\partial \boldsymbol{A}}{\partial x} = \boldsymbol{0}$$

(右端 $\boldsymbol{I}$ 与 x 无关，导数为零矩阵)，两边右乘 $\boldsymbol{A}^{-1}$ 即得(A.21).

□

注

(重构新增) (A.21)要求 $\boldsymbol{A}$ 在讨论的 x 处可逆且可导. 它与标量恒等式 $(1/f)' = -f'/f^2$ 同构: 负号与两侧的 $\boldsymbol{A}^{-1}$ 都来自“逆的逆”. 这条式子是命题 A.15与许多最大似然推导中求协方差矩阵导数的关键一步.

类似地:

命题 A.15 (对数行列式关于标量的导数)

$$\frac{\partial}{\partial x}\ln|\boldsymbol{A}| = \operatorname{tr}\Big(\boldsymbol{A}^{-1}\frac{\partial \boldsymbol{A}}{\partial x}\Big) \tag{A.22}$$

我们稍后会证明这一点.

证明

(重构新增) 原书在此处没有给出证明, 只说“稍后会证明”, 并把按提示的验证留到本附录末尾 (见命题 A.28之后的练习). 下面先给出一个只依赖行列式定义(A.10)的直接证明. 对 $|\boldsymbol{I}+\varepsilon\boldsymbol{M}|$ 按(A.10)展开: 只有恒等排列的项含 ε 的零次与一次项 $\prod_k(1+\varepsilon M_{kk}) = 1+\varepsilon\operatorname{tr}(\boldsymbol{M})+O(\varepsilon^2)$, 而任何非恒等排列至少有两个下标不固定, 其项为 $O(\varepsilon^2)$. 于是

$$|\boldsymbol{I}+\varepsilon\boldsymbol{M}| = 1+\varepsilon\operatorname{tr}(\boldsymbol{M})+O(\varepsilon^2).$$

取 $\boldsymbol{M} = \boldsymbol{A}^{-1}\partial\boldsymbol{A}/\partial x$, 则 $\boldsymbol{A}+\varepsilon\partial\boldsymbol{A}/\partial x = \boldsymbol{A}\left(\boldsymbol{I}+\varepsilon\boldsymbol{A}^{-1}\partial\boldsymbol{A}/\partial x\right)$, 由(A.12)得

$$\ln|\boldsymbol{A}+\varepsilon\partial\boldsymbol{A}/\partial x| - \ln|\boldsymbol{A}| = \ln\left(1+\varepsilon\operatorname{tr}\left(\boldsymbol{A}^{-1}\frac{\partial\boldsymbol{A}}{\partial x}\right)+O(\varepsilon^2)\right),$$

两边除以 ε 并令 $\varepsilon\to 0$, 利用 $\ln(1+t) = t+O(t^2)$ 即得(A.22).

□

注

(重构新增) (A.22)把“行列式的对数”的导数化成了迹, 从而把一个 N 阶行列式的问题降为矩阵乘法的问题. 条件是 $\boldsymbol{A}$ 可逆 (否则 $\ln|\boldsymbol{A}|$ 无定义). 原书在本节末尾以练习的形式要求用谱分解再验证一次 (见本节末尾的练习说明), 两条路线给出同一结论, 可以互相校验.

如果我们选择 x 作为 $\boldsymbol{A}$ 的元素之一, 则有

命题 A.16 (迹关于矩阵元素的导数)

$$\frac{\partial}{\partial A_{ij}}\operatorname{tr}(\boldsymbol{A}\boldsymbol{B}) = B_{ji} \tag{A.23}$$

♠

证明

其可以通过使用索引表示法写出矩阵得到. 补全如下:$\operatorname{tr}(\boldsymbol{A}\boldsymbol{B}) = \sum_k(\boldsymbol{A}\boldsymbol{B})_{kk} = \sum_k\sum_l A_{kl}B_{lk}$, 其中只有 $k=i$、$l=j$ 的那一项含 A_{ij}, 且其系数为 B_{ji}, 故(A.23)成立.

□

这个结论也可以写成如下更紧凑的形式:

$$\frac{\partial}{\partial\boldsymbol{A}}\operatorname{tr}(\boldsymbol{A}\boldsymbol{B}) = \boldsymbol{B}^{\mathsf{T}} \tag{A.24}$$

注

(重构新增) (A.23)与(A.24)是同一事实的两种写法: 前者逐元素, 后者按定义 A.10的形状约定把所有偏导数堆成一个与 $\boldsymbol{A}$ 同型的矩阵. 右端出现转置正是“B_{ji} 而非 B_{ij}”的直接后果. 式中要求 $\boldsymbol{A}\boldsymbol{B}$ 为方阵 (以便取迹),$\boldsymbol{A}$ 与 $\boldsymbol{B}$ 本身不必是方阵.

通过使用这种表示法, 我们可以得到以下性质:

命题 A.17 (矩阵导数的若干恒等式)

$$\frac{\partial}{\partial\boldsymbol{A}}\operatorname{tr}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{B}) = \boldsymbol{B} \tag{A.25}$$

$$\frac{\partial}{\partial\boldsymbol{A}}\operatorname{tr}(\boldsymbol{A}) = \boldsymbol{I} \tag{A.26}$$

$$\frac{\partial}{\partial\boldsymbol{A}}\operatorname{tr}(\boldsymbol{A}\boldsymbol{B}\boldsymbol{A}^{\mathsf{T}}) = \boldsymbol{A}(\boldsymbol{B}+\boldsymbol{B}^{\mathsf{T}}) \tag{A.27}$$

♠

证明

这些性质同样可以通过写出矩阵的索引得证. 补全如下:$\mathrm{tr}(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{B}) = \sum_{kl} A_{lk}B_{lk}$, 对 A_{ij} 求偏导只留下 B_{ij}, 得(A.25).$\mathrm{tr}(\boldsymbol{A}) = \sum_{k} A_{kk}$, 对 A_{ij} 求偏导得 δ_{ij}, 按矩阵形式即 $\boldsymbol{I}$, 得(A.26). 最后,$\mathrm{tr}(\boldsymbol{A}\boldsymbol{B}\boldsymbol{A}^{\mathsf{T}}) = \sum_{klm} A_{km}B_{ml}A_{kl}$, 对 A_{ij} 求偏导时 $\boldsymbol{A}$ 出现两次, 得

$$\sum_{l} B_{jl}A_{il} + \sum_{m} A_{im}B_{mj} = (\boldsymbol{A}\boldsymbol{B}^{\mathsf{T}})_{ij} + (\boldsymbol{A}\boldsymbol{B})_{ij},$$

即(A.27).

□

注

(重构新增) 三条恒等式都要求公式中的迹有定义:(A.25)要求 $\boldsymbol{A}$ 与 $\boldsymbol{B}$ 同型,(A.26)要求 $\boldsymbol{A}$ 是方阵,(A.27)要求 $\boldsymbol{A}\boldsymbol{B}\boldsymbol{A}^{\mathsf{T}}$ 为方阵. 注意(A.27)右端的 $\boldsymbol{B}+\boldsymbol{B}^{\mathsf{T}}$ 在 $\boldsymbol{B}$ 对称时化为 $2\boldsymbol{B}$; 若漏掉 $\boldsymbol{B}^{\mathsf{T}}$, 得到的矩阵连对称性都不满足, 可用一维情形 ($A = B = 1$ 时左端导数为 $2AB = 2$) 快速核对.

另外, 我们还可得到

命题 A.18 (对数行列式关于矩阵的导数)

$$\frac{\partial}{\partial \boldsymbol{A}} \ln|\boldsymbol{A}| = (\boldsymbol{A}^{-1})^{\mathsf{T}} \tag{A.28}$$

证明

(A.28)可以由(A.22)和(A.24)推导得出. 取(A.22)中的 $x = A_{ij}$, 则 $\partial\boldsymbol{A}/\partial A_{ij}$ 是仅在 (i,j) 处为 1、其余为 0 的矩阵 $\boldsymbol{E}_{ij}$, 于是 $\mathrm{tr}(\boldsymbol{A}^{-1}\boldsymbol{E}_{ij}) = (\boldsymbol{A}^{-1})_{ji}$. 按矩阵形式堆叠即得(A.28).

□

注

(重构新增) (A.28)要求 $\boldsymbol{A}$ 可逆, 其右端为转置是因为定义 A.10按“行指标在前”堆叠偏导数. 若 $\boldsymbol{A}$ 对称 (如协方差矩阵), 则 $(\boldsymbol{A}^{-1})^{\mathsf{T}} = \boldsymbol{A}^{-1}$, 转置可以省略. 这个式子是推导多元 Gauss 分布最大似然解时对 $\boldsymbol{\Sigma}$ 求导的核心工具.

A.4 特征向量

对于大小为 $M \times M$ 的方阵 $\boldsymbol{A}$, 其特征向量方程定义为

定义 A.11 (特征向量与特征值)

$$\boldsymbol{A}\boldsymbol{u}_i = \lambda_i \boldsymbol{u}_i \tag{A.29}$$

其中 $i = 1, \cdots, M$,$\boldsymbol{u}_i$ 是特征向量,λ_i 是对应的特征值.

注

(重构新增) 特征向量方程要求 $\boldsymbol{A}$ 是方阵, 且 $\boldsymbol{u}_i$ 必须是非零向量 (否则任何 λ 都满足方程, 特征值就没有意义). 它的含义是:$\boldsymbol{A}$ 作用在 $\boldsymbol{u}_i$ 上不改变方向, 只按因子 λ_i 伸缩. 因此定义 A.11刻画的是矩阵所代表的线性变换的“不变方向”, 这使得后文对对称矩阵的谱分解 (定理 A.1) 成为可能.

这可以视为一组 M 个同时满足的齐次线性方程, 有解的条件是

命题 A.19 (特征方程)

$$|\boldsymbol{A} - \lambda_i \boldsymbol{I}| = 0 \tag{A.30}$$

这就是特征方程. 因为这是一个关于 λ_i 的 M 阶多项式, 所以它必须有 M 个解 (尽管这些解不必都是不同的).

证明

(重构新增) 原书未给证明. 把(A.29)移项得 $(\boldsymbol{A}-\lambda_i\boldsymbol{I})\boldsymbol{u}_i=\boldsymbol{0}$, 即 $\boldsymbol{u}_i$ 是齐次线性方程组 $(\boldsymbol{A}-\lambda_i\boldsymbol{I})\boldsymbol{x}=\boldsymbol{0}$ 的非零解. 由线性代数基本结论, 齐次方程有非零解当且仅当系数矩阵奇异, 而方阵奇异当且仅当其行列式为零 (参见命题 A.9), 故(A.30)是存在非零解的必要条件, 也是充分条件. 按(A.10)展开 $|\boldsymbol{A}-\lambda\boldsymbol{I}|$ 得到的是 λ 的 M 次多项式 (特征多项式), 由代数基本定理, 它在复数域内恰有 M 个根 (计重数), 故特征值总共 M 个.

□

注

(重构新增) (A.30)把“求特征值”化归为“求多项式的根”, 因此特征值可能重复、也可能为复数, 这两点在本节后面都要用到: 重复的特征值使特征向量不唯一 (需要另行正交化), 复数特征值则需要命题 A.22来排除. 条件“$\boldsymbol{A}$ 为方阵”不可省, 否则 $|\boldsymbol{A}-\lambda\boldsymbol{I}|$ 无定义.

$\boldsymbol{A}$ 的秩等于非零特征值的数量.

命题 A.20 (秩与非零特征值的个数)

$\boldsymbol{A}$ 的秩等于非零特征值的数量.

♠

注

(重构新增) 原文问题说明: 原书此处直接断言“$\boldsymbol{A}$ 的秩等于非零特征值的数量”, 这一断言对一般方阵并不成立. 反例:$\boldsymbol{A}=\begin{pmatrix}0&1\\0&0\end{pmatrix}$ 的秩为 1(两行不成比例, 且第一行非零), 但它的特征多项式为 $\lambda^2=0$, 两个特征值都是 0, 非零特征值的个数为 $0\neq 1$. 问题的根源在于不可对角化的矩阵会把非零的“能量”放在 Jordan 块的非对角位置上, 而特征值只反映对角部分. 正确的表述是: 若 $\boldsymbol{A}$ 可对角化 (即存在可逆 $\boldsymbol{U}$ 使 $\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{-1}$), 则 $\operatorname{rank}(\boldsymbol{A})=\operatorname{rank}(\boldsymbol{\Lambda})$, 即秩等于非零特征值的个数. 证明: 可逆矩阵相乘不改变秩, 故 $\operatorname{rank}(\boldsymbol{A})=\operatorname{rank}(\boldsymbol{\Lambda})$, 而对角矩阵的秩显然等于其非零对角元的个数. 特别地, 实对称矩阵总可以正交对角化 (定理 A.1), 因此原书的断言在本附录真正使用的对称矩阵情形下是对的.

需要特别讨论的是对称矩阵, 对称矩阵可能是协方差矩阵、核矩阵或 Hessian 矩阵.

定义 A.12 (对称矩阵)

对称矩阵具有性质 $A_{ij}=A_{ji}$, 或等价地有 $\boldsymbol{A}^{\mathsf{T}}=\boldsymbol{A}$.

♣

注

(重构新增) 对称性只对方阵才有意义, 它是“转置不变”这一条件 (参见定义 A.2). 协方差矩阵、核矩阵与 Hessian 矩阵之所以对称, 分别来自协方差的定义、核函数的对称性以及二阶偏导数可交换; 因此本书中大量矩阵求逆与求导的公式都可以在对称情形下简化 (例如(A.28)中的转置可以去掉).

对称矩阵的逆矩阵也是对称矩阵.

命题 A.21 (对称矩阵的逆)

对称矩阵的逆矩阵也是对称矩阵.

♠

证明

这可以通过取 $\boldsymbol{A}^{-1}\boldsymbol{A}=\boldsymbol{I}$ 的转置并利用 $\boldsymbol{A}\boldsymbol{A}^{-1}=\boldsymbol{I}$ 以及 $\boldsymbol{I}$ 的对称性得证. 补全如下: 由(A.2)有 $\boldsymbol{A}^{-1}\boldsymbol{A}=\boldsymbol{I}$, 两边取转置并用(A.1)得 $\boldsymbol{A}^{\mathsf{T}}(\boldsymbol{A}^{-1})^{\mathsf{T}}=\boldsymbol{I}^{\mathsf{T}}=\boldsymbol{I}$; 又 $\boldsymbol{A}^{\mathsf{T}}=\boldsymbol{A}$, 故 $\boldsymbol{A}(\boldsymbol{A}^{-1})^{\mathsf{T}}=\boldsymbol{I}$. 与 $\boldsymbol{A}\boldsymbol{A}^{-1}=\boldsymbol{I}$ 比较并按逆的唯一性得 $(\boldsymbol{A}^{-1})^{\mathsf{T}}=\boldsymbol{A}^{-1}$, 即 $\boldsymbol{A}^{-1}$ 对称.

□

注

(重构新增) 这条结论的条件是 $\boldsymbol{A}$ 对称且可逆; 它保证在对称矩阵的谱分解中,$\boldsymbol{A}^{-1}$ 可以用同一组特征向量展开 (参见(A.46)), 从而 $\boldsymbol{A}$ 与 $\boldsymbol{A}^{-1}$ 的特征值互为倒数.

通常, 矩阵的特征值是复数, 但对于对称矩阵, 其特征值 λ_i 是实数.

命题 A.22 (实对称矩阵的特征值为实数)

实对称矩阵的特征值 λ_i 是实数.

证明

为了证明这一点, 可以首先对(A.29)左乘 $(\boldsymbol{u}_i^*)^{\mathsf{T}}$(其中 $*$ 表示复共轭), 从而得到

$$(\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i = \lambda_i(\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{u}_i \tag{A.31}$$

接下来取(A.29)的复共轭并左乘 $\boldsymbol{u}_i^{\mathsf{T}}$, 从而得到

$$\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i^* = \lambda_i^*\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_i^* \tag{A.32}$$

其中使用了 $\boldsymbol{A}^* = \boldsymbol{A}$, 因为我们仅考虑实矩阵 $\boldsymbol{A}$. 对(A.32)取转置并使用 $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}$, 我们看到两个方程的左边相等, 因此有 $\lambda_i^* = \lambda_i$, 即 λ_i 必须是实数.

(重构新增) 这里补全原书省略的除法. 因 $(\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i$ 是标量, 其转置等于自身. 对(A.32)取转置, 左端为

$$\left(\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i^*\right)^{\mathsf{T}} = (\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{u}_i = (\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i,$$

右端为 $\lambda_i^*\left(\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_i^*\right)^{\mathsf{T}} = \lambda_i^*(\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{u}_i$. 与(A.31)比较得

$$(\lambda_i - \lambda_i^*)(\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{u}_i = 0.$$

而 $(\boldsymbol{u}_i^*)^{\mathsf{T}}\boldsymbol{u}_i = \|\boldsymbol{u}_i\|^2 > 0$(因为 $\boldsymbol{u}_i \neq \boldsymbol{0}$), 故 $\lambda_i = \lambda_i^*$, 即 λ_i 为实数.

□

注

(重构新增) 论证中“实矩阵”与“对称”两个条件各司其职:$\boldsymbol{A}^* = \boldsymbol{A}$ 用到了实性,$\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}$ 用到了对称性; 缺任一条结论都不成立 (例如旋转矩阵 $\begin{pmatrix} 0 & -1 \\ 1 & 0 \end{pmatrix}$ 是实矩阵但特征值为 $\pm\mathrm{i}$). 核心计算是 $(\boldsymbol{u}^*)^{\mathsf{T}}\boldsymbol{u} = \|\boldsymbol{u}\|^2 > 0$, 它保证可以做除法; 这也说明特征向量非零的约定在此处是必需的.

实对称矩阵的特征向量 $\boldsymbol{u}_i$ 可以选择为正交归一化的向量 (即正交的且为单位长度的向量), 从而有

命题 A.23 (特征向量的正交归一性)

$$\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j = I_{ij} \tag{A.33}$$

其中 I_{ij} 是单位矩阵 $\boldsymbol{I}$ 的元素.

证明

为了说明这一点, 首先将(A.29)左乘 $\boldsymbol{u}_j^{\mathsf{T}}$, 从而得到第一个方程

$$\boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i = \lambda_i\boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{u}_i \tag{A.34}$$

然后交换索引, 从而得到第二个方程:

$$\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_j = \lambda_j\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j \tag{A.35}$$

取第二个方程的转置并利用对称性质 $\boldsymbol{A}^{\mathsf{T}} = \boldsymbol{A}$, 最后将这两个方程相减, 从而得到

$$(\lambda_i - \lambda_j)\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j = 0 \tag{A.36}$$

因此, 对于 $\lambda_i \neq \lambda_j$, 我们有 $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j = 0$, 即 $\boldsymbol{u}_i$ 和 $\boldsymbol{u}_j$ 是正交的. 如果这两个特征值相等, 那么任何线性组合 $\alpha\boldsymbol{u}_i + \beta\boldsymbol{u}_j$ 都将是具有相同特征值的特征向量, 因此我们可以任意选择一个线性组合, 然后选择另一个线性组合并使其与第一个线性组合正交 (可以证明退化的特征向量永远不是线性相关的). 因此, 特征向量可以选择为正交向量, 并通过

归一化变为单位长度. 由于有 M 个特征值, 而对应的 M 个正交特征向量构成了一个完备集, 因此任何 M 维向量都可以表示为特征向量的线性组合.

(重构新增) 上面“取转置”一步补全如下: 对(A.35)取转置, 左端为 $(\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_j)^{\mathsf{T}} = \boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{u}_i = \boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i$, 右端为 $\lambda_j\boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{u}_i$(因 $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j$ 是标量). 与(A.34)相减即得(A.36). 当 $\lambda_i \neq \lambda_j$ 时 $\lambda_i - \lambda_j \neq 0$, 故 $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_j = 0$; 再对每个特征向量令 $\boldsymbol{u}_i \leftarrow \boldsymbol{u}_i/\|\boldsymbol{u}_i\|$, 即得(A.33).

□

注

(重构新增) (A.33)是本节后续所有结论的枢纽: 它使特征向量矩阵 $\boldsymbol{U}$ 成为正交矩阵 (定义 A.13), 从而 $\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{U}^{-1}$, 谱分解(A.43)才得以成立. 论证中“特征值互不相同”是正交性的直接来源; 当特征值重复时, 需要原文提到的补充论证——重特征值对应的特征子空间维数等于重数 (对称矩阵可正交对角化), 在子空间内用 Gram-Schmidt 方法重新正交化即可, 这一步本书未展开.

我们可以将特征向量 $\boldsymbol{u}_i$ 作为 $M \times M$ 矩阵 $\boldsymbol{U}$ 的列, 该矩阵由于正交归一性而满足

定义 A.13 (正交矩阵)

$$\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U} = \boldsymbol{I} \tag{A.37}$$

这样的矩阵称为**正交矩阵**.

♣

注

(重构新增) (A.37)说明正交矩阵的列是正交归一的向量组, 因而线性无关、$\boldsymbol{U}$ 必可逆. 由(A.12)可得 $|\boldsymbol{U}|^2 = 1$, 即 $|\boldsymbol{U}| = \pm 1$. 正交矩阵之所以重要, 是因为它保持内积 (命题 A.26), 从而对应坐标系的刚性旋转, 这一点在(A.42)的对角化过程中起决定作用.

有趣的是, 这个矩阵的行也是正交的, 所以有 $\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{I}$.

命题 A.24 (正交矩阵的行正交性与行列式)

设 $\boldsymbol{U}$ 是满足(A.37)的方阵, 则 $\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{I}$, 且 $|\boldsymbol{U}| = \pm 1$.

♠

为了证明这一点, 请注意(A.37)意味着 $\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}\boldsymbol{U}^{-1} = \boldsymbol{U}^{-1} = \boldsymbol{U}^{\mathsf{T}}$, 从而有 $\boldsymbol{U}\boldsymbol{U}^{-1} = \boldsymbol{U}\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{I}$. 使用(A.12), 我们还可以推导出 $|\boldsymbol{U}|^2 = 1$.

证明

(重构新增) 补全如下. 由(A.37)知 $\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U} = \boldsymbol{I}$, 故 $\boldsymbol{U}$ 可逆且 $\boldsymbol{U}^{-1} = \boldsymbol{U}^{\mathsf{T}}$(因为 $\boldsymbol{U}^{\mathsf{T}}$ 同时是左逆与右逆). 于是

$$\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{U}\boldsymbol{U}^{-1} = \boldsymbol{I}.$$

再对(A.37)两边取行列式, 由(A.12)与 $|\boldsymbol{I}| = 1$ 得 $|\boldsymbol{U}^{\mathsf{T}}|\,|\boldsymbol{U}| = 1$, 而由(A.10)可知 $|\boldsymbol{U}^{\mathsf{T}}| = |\boldsymbol{U}|$, 故 $|\boldsymbol{U}|^2 = 1$, 即 $|\boldsymbol{U}| = \pm 1$.

□

注

(重构新增) 原书由 $|\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}| = 1$ 推出 $|\boldsymbol{U}| = 1$, 严格来说只能得到 $|\boldsymbol{U}| = \pm 1$: 反射矩阵 (例如 $\mathrm{diag}(1,-1)$) 也是正交矩阵, 但其行列式为 -1. 只有再限定 $\boldsymbol{U}$ 为“旋转” ($\det\boldsymbol{U} = +1$) 时才有 $|\boldsymbol{U}| = 1$; 本附录的谱分解只用到 $\boldsymbol{U}$ 的正交性, 并不需要这个符号信息, 因此不影响后文结论.$\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}} = \boldsymbol{I}$ 的实质是“行也正交归一”, 它与(A.37)合起来说明正交矩阵的逆就是其转置.

特征向量方程 [(A.29)] 可以用以下关于 $\boldsymbol{U}$ 的形式来表示:

命题 A.25 (特征向量方程的矩阵形式)

$$\boldsymbol{A}\boldsymbol{U} = \boldsymbol{U}\boldsymbol{\Lambda} \tag{A.38}$$

其中 $\boldsymbol{\Lambda}$ 是 $M \times M$ 的对角矩阵, 其对角元素由特征值 λ_i 给出.

♠

证明

(重构新增) 原书未给证明. 把(A.29)对 $i=1,\cdots,M$ 逐个写出并按列拼成矩阵: 左端 $(\boldsymbol{A}\boldsymbol{u}_1,\cdots,\boldsymbol{A}\boldsymbol{u}_M)=\boldsymbol{A}\boldsymbol{U}$, 右端 $(\lambda_1\boldsymbol{u}_1,\cdots,\lambda_M\boldsymbol{u}_M)=\boldsymbol{U}\boldsymbol{\Lambda}$(因为 $\boldsymbol{U}$ 的第 i 列右乘对角元 λ_i 正是 $\lambda_i\boldsymbol{u}_i$), 故(A.38)成立.

□

注

(重构新增) (A.38)把 M 个独立的特征向量方程打包成一个矩阵等式, 从而可以用矩阵运算整体处理. 它要求 $\boldsymbol{U}$ 的列恰好是全部 M 个特征向量 (并按同一顺序排列 λ_i); 若只取部分特征向量, 等式只在相应的列上成立. 下一节的谱分解正是由它两边左乘 $\boldsymbol{U}^{\mathsf{T}}$ 得到的.

假设我们通过使用正交矩阵 $\boldsymbol{U}$ 转换列向量 $\boldsymbol{x}$ 得到了一个新的向量:

命题 A.26 (正交变换保持长度与夹角)

$$\tilde{\boldsymbol{x}}=\boldsymbol{U}\boldsymbol{x} \tag{A.39}$$

则向量的长度保持不变, 因为

$$\tilde{\boldsymbol{x}}^{\mathsf{T}}\tilde{\boldsymbol{x}}=\boldsymbol{x}^{\mathsf{T}}\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}\boldsymbol{x}=\boldsymbol{x}^{\mathsf{T}}\boldsymbol{x} \tag{A.40}$$

同样, 任何两个此类向量之间的夹角也保持不变, 因为

$$\tilde{\boldsymbol{x}}^{\mathsf{T}}\tilde{\boldsymbol{y}}=\boldsymbol{x}^{\mathsf{T}}\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}\boldsymbol{y}=\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y} \tag{A.41}$$

♠

因此, 向量乘以 $\boldsymbol{U}$ 可以解释为坐标系发生刚性旋转.

证明

(重构新增) 原书已给出两行计算, 这里补全“夹角不变”的含义. 由(A.37),$\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}=\boldsymbol{I}$, 故(A.40)与(A.41)成立. 长度是内积的特例 (取 $\boldsymbol{y}=\boldsymbol{x}$), 而夹角由 $\cos\theta=\boldsymbol{x}^{\mathsf{T}}\boldsymbol{y}/(\|\boldsymbol{x}\|\,\|\boldsymbol{y}\|)$ 给出, 分子与分母在变换后都保持不变, 故夹角不变.

□

注

(重构新增) (A.40)与(A.41)说明正交变换是等距变换, 这正是“刚性旋转”的数学内容; 反过来, 保持所有内积的线性变换必为正交变换. 这里的 $\boldsymbol{x}$ 是列向量,$\boldsymbol{U}$ 必须是方阵正交矩阵, 否则 $\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}=\boldsymbol{I}$ 不成立, 长度也不再保持.

从(A.38)可以看出

$$\boldsymbol{U}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{U}=\boldsymbol{\Lambda} \tag{A.42}$$

又因为 $\boldsymbol{\Lambda}$ 是对角矩阵, 我们可以说矩阵 $\boldsymbol{A}$ 被矩阵 $\boldsymbol{U}$ 对角化了.

定理 A.1 (实对称矩阵的谱分解)

设 $\boldsymbol{A}$ 是 $M\times M$ 实对称矩阵,$\boldsymbol{u}_1,\cdots,\boldsymbol{u}_M$ 是它的正交归一特征向量 (命题 A.23),$\boldsymbol{U}$ 以这些特征向量为列,$\boldsymbol{\Lambda}=\operatorname{diag}(\lambda_1,\cdots,\lambda_M)$. 则左乘 $\boldsymbol{U}$ 并右乘 $\boldsymbol{U}^{\mathsf{T}}$, 可以把 $\boldsymbol{A}$ 写成

$$\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}} \tag{A.43}$$

♡

证明

(重构新增) 由(A.42)有 $\boldsymbol{U}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{U}=\boldsymbol{\Lambda}$. 两边左乘 $\boldsymbol{U}$、右乘 $\boldsymbol{U}^{\mathsf{T}}$, 利用 $\boldsymbol{U}\boldsymbol{U}^{\mathsf{T}}=\boldsymbol{I}$(命题 A.24) 即得 $\boldsymbol{A}=\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}}$, 即(A.43).

□

注

(重构新增) 谱分解是本节全部构造的汇合点: 它同时用到(A.33)(保证 $\boldsymbol{U}^{-1}=\boldsymbol{U}^{\mathsf{T}}$)、命题 A.22(保证 $\boldsymbol{\Lambda}$ 是实对角阵)与(A.38)(保证 $\boldsymbol{A}\boldsymbol{U}=\boldsymbol{U}\boldsymbol{\Lambda}$). 条件“实对称”三个字缺一不可: 非对称矩阵未必有完整的特征向量组, 因而不能正交对角化.(A.43)的意义在于把 $\boldsymbol{A}$ 的作用分解为“沿正交方向的独立伸缩”, 因此 $\boldsymbol{A}$ 的幂、逆、行列式与迹都可以逐个特征值计算 (见命题 A.27与命题 A.28).

取(A.43)的逆并利用(A.3)以及 $\boldsymbol{U}^{-1}=\boldsymbol{U}^{\mathsf{T}}$, 可以得到

命题 A.27 (逆矩阵的谱分解与求和形式)

$$\boldsymbol{A}^{-1}=\boldsymbol{U}\boldsymbol{\Lambda}^{-1}\boldsymbol{U}^{\mathsf{T}} \tag{A.44}$$

以上两个方程也可以写成以下形式:

$$\boldsymbol{A}=\sum_{i=1}^{M}\lambda_i\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}} \tag{A.45}$$

$$\boldsymbol{A}^{-1}=\sum_{i=1}^{M}\frac{1}{\lambda_i}\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}} \tag{A.46}$$

♠

证明

(重构新增) 原书未逐条证明, 这里补全. 对(A.43)取逆, 由(A.3)与 $\boldsymbol{U}^{-1}=\boldsymbol{U}^{\mathsf{T}}$ 得

$$\boldsymbol{A}^{-1}=(\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}})^{-1}=(\boldsymbol{U}^{\mathsf{T}})^{-1}\boldsymbol{\Lambda}^{-1}\boldsymbol{U}^{-1}=\boldsymbol{U}\boldsymbol{\Lambda}^{-1}\boldsymbol{U}^{\mathsf{T}},$$

即(A.44)(要求所有 $\lambda_i\neq 0$). 再把 $\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}}$ 按列展开:$(\boldsymbol{U}\boldsymbol{\Lambda})_{:,i}=\lambda_i\boldsymbol{u}_i$, 故 $\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}}=\sum_i\lambda_i\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}}$, 即(A.45); 对 $\boldsymbol{\Lambda}^{-1}$ 重复同一展开得(A.46).

□

注

(重构新增) (A.44)与(A.45)把“求逆”变成“对特征值取倒数再求和”, 因此它们要求所有 $\lambda_i\neq 0$, 这与 $\boldsymbol{A}$ 可逆等价 (参见(A.47)).(A.45)的每一项 $\lambda_i\boldsymbol{u}_i\boldsymbol{u}_i^{\mathsf{T}}$ 都是秩一矩阵, 它表明对称矩阵可以分解为 M 个沿正交方向的秩一成分之和; 这也解释了为什么 $\boldsymbol{A}$ 的二次型只由各方向上的 λ_i 决定 (参见命题 A.29).

如果我们取(A.43)的行列式并使用(A.12), 则可以得到

命题 A.28 (行列式与迹的特征值表示)

$$|\boldsymbol{A}|=\prod_{i=1}^{M}\lambda_i \tag{A.47}$$

类似地, 取(A.43)的迹, 并利用迹运算的循环性质 [(A.9)] 以及 $\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U}=\boldsymbol{I}$, 我们可以得到

$$\operatorname{tr}(\boldsymbol{A})=\sum_{i=1}^{M}\lambda_i \tag{A.48}$$

♠

证明

(重构新增) 补全如下. 对(A.43)取行列式, 由(A.12)得 $|\boldsymbol{A}|=|\boldsymbol{U}||\boldsymbol{\Lambda}||\boldsymbol{U}^{\mathsf{T}}|$, 而 $|\boldsymbol{U}^{\mathsf{T}}|=|\boldsymbol{U}|$ 且 $|\boldsymbol{U}|^2=1$(命题 A.24), 故 $|\boldsymbol{A}|=|\boldsymbol{\Lambda}|=\prod_i\lambda_i$, 即(A.47). 对(A.43)取迹, 由(A.9)得 $\operatorname{tr}(\boldsymbol{A})=\operatorname{tr}(\boldsymbol{U}\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}})=\operatorname{tr}(\boldsymbol{\Lambda}\boldsymbol{U}^{\mathsf{T}}\boldsymbol{U})=\operatorname{tr}(\boldsymbol{\Lambda})=\sum_i\lambda_i$, 即(A.48).

□

注

(重构新增) (A.47)与(A.48)把两个“整体”量 (行列式、迹) 化归为逐个特征值之积与之和, 它们也说明:$\boldsymbol{A}$ 可逆当且仅当所有 $\lambda_i\neq 0$. 两式都只要求 $\boldsymbol{A}$ 可正交对角化 (实对称即可), 并不要求特征值互不相同; 若特征值重复, 按重数计入乘积与求和.

作为一个练习, 请读者利用(A.33)、(A.45)、(A.46)和(A.47)的结论验证(A.22).

注

(重构新增) 下面按原书提示完成这个练习. 设 $\boldsymbol{A}(x)$ 是对称可逆矩阵, 其特征值与特征向量随 x 可微. 由(A.47),$\ln|\boldsymbol{A}|=\sum_i\ln\lambda_i$, 故

$$\frac{\partial}{\partial x}\ln|\boldsymbol{A}|=\sum_{i=1}^{M}\frac{1}{\lambda_i}\frac{\partial\lambda_i}{\partial x}.$$

另一方面, 由(A.46)与(A.33),

$$\operatorname{tr}\left(\boldsymbol{A}^{-1}\frac{\partial \boldsymbol{A}}{\partial x}\right)=\sum_{i=1}^{M}\frac{1}{\lambda_i}\boldsymbol{u}_i^{\mathsf{T}}\frac{\partial \boldsymbol{A}}{\partial x}\boldsymbol{u}_i.$$

还需说明 $\boldsymbol{u}_i^{\mathsf{T}}(\partial \boldsymbol{A}/\partial x)\boldsymbol{u}_i=\partial\lambda_i/\partial x$. 对(A.29)关于 x 求导得

$$\frac{\partial \boldsymbol{A}}{\partial x}\boldsymbol{u}_i+\boldsymbol{A}\frac{\partial \boldsymbol{u}_i}{\partial x}=\frac{\partial \lambda_i}{\partial x}\boldsymbol{u}_i+\lambda_i\frac{\partial \boldsymbol{u}_i}{\partial x},$$

两边左乘 $\boldsymbol{u}_i^{\mathsf{T}}$, 利用 $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{A}=\lambda_i\boldsymbol{u}_i^{\mathsf{T}}$(由 $\boldsymbol{A}^{\mathsf{T}}=\boldsymbol{A}$ 与 $\boldsymbol{A}\boldsymbol{u}_i=\lambda_i\boldsymbol{u}_i$ 得到) 以及 $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{u}_i=1$, 含 $\partial\boldsymbol{u}_i/\partial x$ 的两项相消, 即得所需等式. 代入即得(A.22).

如果对于所有非零向量 $\boldsymbol{w}$, 有 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w}>0$, 则称矩阵 $\boldsymbol{A}$ 为正定矩阵, 记为 $\boldsymbol{A}>0$.

定义 A.14 (正定矩阵)

如果对于所有非零向量 $\boldsymbol{w}$, 有 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w}>0$, 则称矩阵 $\boldsymbol{A}$ 为**正定矩阵**, 记为 $\boldsymbol{A}>0$. ♣

注

(重构新增) 定义中的二次型 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w}$ 只依赖于 $\boldsymbol{A}$ 的对称部分, 因此正定性通常只对对称矩阵讨论; 本附录的语境是协方差矩阵、核矩阵与 Hessian 矩阵, 它们都是对称的. 条件“对所有非零 $\boldsymbol{w}$”是量化条件, 不能只检验个别方向: 只要存在一个方向使二次型非正,$\boldsymbol{A}$ 就不是正定的. 正定性保证 $\boldsymbol{A}$ 可逆且 $\boldsymbol{A}^{-1}$ 也正定, 因此在命题 A.4与命题 A.6中常用它来确保各逆矩阵存在.

等价地, 正定矩阵对于其所有特征值, 有 $\lambda_i>0$(这可以通过将 $\boldsymbol{w}$ 分别设置为每个特征向量来验证, 并注意到任意向量都可以表示为特征向量的线性组合而看出).

命题 A.29 (正定矩阵的特征值判据)

设 $\boldsymbol{A}$ 是实对称矩阵, 则 $\boldsymbol{A}$ 正定当且仅当其全部特征值满足 $\lambda_i>0$. ♠

证明

(重构新增) 原书给出的是要点, 下面补全两向论证. 由定理 A.1, 任意 $\boldsymbol{w}$ 可写成 $\boldsymbol{w}=\sum_i c_i\boldsymbol{u}_i$, 其中 $c_i=\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{w}$, 于是由(A.33),

$$\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w}=\sum_{i,j}c_ic_j\boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i=\sum_{i,j}c_ic_j\lambda_i\boldsymbol{u}_j^{\mathsf{T}}\boldsymbol{u}_i=\sum_{i=1}^{M}\lambda_ic_i^2.$$

若所有 $\lambda_i>0$, 则对 $\boldsymbol{w}\neq\boldsymbol{0}$(即某个 $c_i\neq0$) 有 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w}>0$. 反之, 若 $\boldsymbol{A}$ 正定, 取 $\boldsymbol{w}=\boldsymbol{u}_i\neq\boldsymbol{0}$ 得 $\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{A}\boldsymbol{u}_i=\lambda_i>0$.

□

注

(重构新增) 判据把“对所有方向检验二次型”这样一个不可穷举的条件, 换成“检验 M 个特征值”的可计算条件, 这是谱分解最直接的用途. 它依赖实对称性 (否则特征向量不构成完备正交组, 上面的展开不成立). 注意 $\lambda_i>0$ 是严格不等式: 若某个 $\lambda_i=0$, 则沿该特征方向的二次型为零, 矩阵只是半正定的 (定义 A.15).

注意, 一个矩阵的所有元素均为正并不一定意味着该矩阵就是正定矩阵.

例题 A.1 元素全为正但非正定 例如, 矩阵

$$\begin{pmatrix}1 & 2\\ 3 & 4\end{pmatrix} \tag{A.49}$$

的元素全为正, 但它不是正定矩阵: 它具有特征值 $\lambda_1\approx5.37$ 和 $\lambda_2\approx-0.37$.

解

(重构新增) 补全特征值的计算.$\operatorname{tr}(\boldsymbol{A}) = 1 + 4 = 5, |\boldsymbol{A}| = 1 \cdot 4 - 2 \cdot 3 = -2$, 故特征多项式为 $\lambda^2 - 5\lambda - 2 = 0$, 其根为

$$\lambda = \frac{5 \pm \sqrt{25 + 8}}{2} = \frac{5 \pm \sqrt{33}}{2}.$$

取 $\sqrt{33} \approx 5.745$, 得 $\lambda_1 \approx 5.372$、$\lambda_2 \approx -0.372$. 由于 $\lambda_2 < 0$, 由命题 A.29该矩阵不是正定的.

□

注

(重构新增) 这个反例提醒我们: 正定性是关于二次型 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w}$ 的符号的陈述, 而不是关于矩阵元素符号的陈述. 元素全为正只保证 $\boldsymbol{w}$ 的各分量同号时二次型为正, 一旦 $\boldsymbol{w}$ 的分量异号, 交叉项就可能占优. 矩阵(A.49)本身也不是对称矩阵, 其特征值只按(A.30)计算; 即便如此, 负特征值已足以否定正定性.

如果对于所有的 $\boldsymbol{w}$, 有 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w} \geqslant 0$, 则称这个矩阵为半正定矩阵, 记为 $\boldsymbol{A} \geqslant 0$.

定义 A.15 (半正定矩阵)

如果对于所有的 $\boldsymbol{w}$, 有 $\boldsymbol{w}^{\mathsf{T}}\boldsymbol{A}\boldsymbol{w} \geqslant 0$, 则称这个矩阵为**半正定矩阵**, 记为 $\boldsymbol{A} \geqslant 0$, 等价于 $\lambda_i \geqslant 0$.

♣

注

(重构新增) 把定义 A.14中的严格不等号换成非严格不等号即得半正定性, 相应的特征值判据 (命题 A.29的证明逐字适用) 也由 $\lambda_i > 0$ 变成 $\lambda_i \geqslant 0$. 允许零特征值意味着半正定矩阵可能奇异, 因此 $\boldsymbol{A} \geqslant 0$ 的矩阵未必可逆,(A.46)之类的公式对它没有意义. 协方差矩阵总是半正定的, 而它正定当且仅当数据不落在低维仿射子空间上.

矩阵的条件数由下式给出:

定义 A.16 (条件数)

$$\mathrm{CN} = \left(\frac{\lambda_{\max}}{\lambda_{\min}}\right)^{1/2} \tag{A.50}$$

其中 $\lambda_{\max}$ 是最大的特征值,$\lambda_{\min}$ 是最小的特征值.

♣

注

(重构新增) 条件数衡量的是矩阵所代表的线性变换在不同方向上伸缩幅度的差异:$\mathrm{CN} = 1$ 时各方向伸缩相同, 问题最“良态”;CN 越大, 求逆与解方程对误差越敏感. 定义(A.50)要求 $\boldsymbol{A}$ 对称正定 (否则 $\lambda_{\min}$ 可能为零或为负, 根号与比值失去意义), 此时 $\lambda_{\max}/\lambda_{\min}$ 是 $\boldsymbol{A}$ 的最大与最小伸缩比, 开平方后与“标准差之比”的量纲一致. 本书在讨论病态问题时使用这一指标.

附录 B 变分法

我们可以将函数 $y(x)$ 视为如下运算: 对于任何输入值 x, 返回输出值 y. 同样, 我们也可以将泛函 $F[y]$ 视为接收函数 $y(x)$ 并返回输出值 F 的运算.

定义 B.1 (泛函)

设某个函数类中的每个函数 $y(x)$ 都按确定的规则对应一个数 F. 这样的对应关系称为定义在该函数类上的**泛函** (functional), 记作 $F[y]$: 它接收函数 $y(x)$, 并返回输出值 F. ♣

注

(重构新增) 泛函与普通函数的区别只在“输入是什么”: 函数的输入是一个数 (或一个有限维向量), 泛函的输入是一整条函数 $y(x)$. 定义中“确定的规则”要求同一个函数总给出同一个数, 这与函数定义中“每个 x 只对应一个 y”的要求完全平行; 方括号 $F[y]$ 只是用来提示自变量是函数而非数, 并不是新的运算. 函数类必须事先指定, 因为同一条规则在不同函数类上可能给出不同的值域, 甚至可能没有定义 (例如要求函数可微时, 折线就不能作为输入).

泛函的一个示例是在二维平面上计算绘制的曲线的长度, 其中曲线的路径是由一个函数定义的. 在机器学习中, 一个广泛使用的泛函是连续变量 x 的熵 $H[x]$. 对于任意选择的概率密度函数 $p(x)$, 它都会返回一个标量值以表示该密度下 x 的熵. 因此,$p(x)$ 的熵也可以类似地写作 $H[p]$.

注

(重构新增) 把这两个示例写成积分就可以看清它们为什么是泛函. 若曲线由 $y(x)$ 给出、x 从 a 变到 b, 则其长度为

$$L[y] = \int_a^b \sqrt{1 + (y'(x))^2}\,\mathrm{d}x,$$

输入是整条曲线 y, 输出是一个数; 若密度 p 定义在连续变量 x 上, 则其熵为

$$H[p] = -\int p(x)\ln p(x)\,\mathrm{d}x,$$

输入的是一整个概率密度函数, 输出的同样是一个标量. 这两个例子都说明: 泛函可以在“所有函数”构成的无穷维空间上取值, 因此它的“求极值”问题无法用普通微积分的有限维方法直接处理, 这正是下面要引入变分法的原因.

传统微积分中的一个常见问题, 就是找到一个 x 值以使函数 $y(x)$ 最大化 (或最小化). 类似地, 在变分法中, 我们也要寻求一个函数 $y(x)$, 该函数能使泛函 $F[y]$ 最大化 (或最小化). 也就是说, 在所有可能的函数中, 我们希望找到能使泛函 $F[y]$ 取最大值 (或最小值) 的特定函数. 例如, 我们可以使用变分法来证明两点之间直线最短, 而最大熵分布是 Gauss 分布.

定义 B.2 (变分法)

在给定的函数类中寻找使泛函 $F[y]$ 取极大值或极小值的函数 $y(x)$, 这类问题称为变分问题; 研究变分问题、并给出其解所满足的微分方程的数学分支, 称为**变分法** (calculus of variations). ♣

注

(重构新增) 变分法与普通微积分的形式差别只在“自变量空间”: 普通微积分在实数轴上搜索 x, 变分法在函数类中搜索整条曲线 $y(x)$, 因此后者的候选对象有无穷多个自由度. 也正因为如此, 变分法通常不直接构造极值函数, 而是先导出一个必要条件 (Euler-Lagrange 方程, 见定理 B.1), 把变分问题化为微分方程的边值问题. “两点之间直线最短”正是这一必要条件的解在给定边界下的唯一解, 而“最大熵分布是 Gauss 分布”则是在归一化约束下求解相应的变分问题 (参见附录 C).

如果不熟悉普通微积分的法则, 则可以对变量 x 施加一个小的改变 ε, 然后以 ε 的幂展开来计算传统导数 $\mathrm{d}y/\mathrm{d}x$, 从而有

$$y(x+\varepsilon) = y(x) + \varepsilon\frac{\mathrm{d}y}{\mathrm{d}x} + O(\varepsilon^2) \tag{B.1}$$

最后取极限 $\varepsilon \to 0$. 类似地, 对于具有多个变量的函数 $y(x_1, \cdots, x_D)$, 相应的偏导数由下式定义:

$$y(x_1+\varepsilon_1, \cdots, x_D+\varepsilon_D) = y(x_1, \cdots, x_D) + \sum_{i=1}^{D}\varepsilon_i\frac{\partial y}{\partial x_i} + O(\varepsilon^2) \tag{B.2}$$

当需要考虑对函数 $y(x)$ 进行微小改动 $\varepsilon\eta(x)$ 会使泛函 $F[y]$ 发生多大的改变时, 就可以类似地定义泛函的导数, 其中 $\eta(x)$ 是 x 的任意函数, 如图 B.1所示. 用 $\delta F/\delta y(x)$ 表示 $F[y]$ 相对于 $y(x)$ 的**泛函导数** (functional derivative), 并用以下关系式加以定义:

定义 B.3 (泛函导数)

$$F[y(x)+\varepsilon\eta(x)] = F[y(x)] + \varepsilon\int\frac{\delta F}{\delta y(x)}\eta(x)\,\mathrm{d}x + O(\varepsilon^2) \tag{B.3}$$

♣

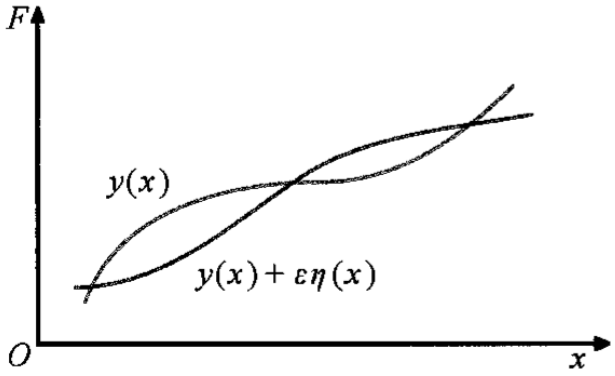


图 B.1: 泛函的导数可以用当 $y(x)$ 变为 $y(x)+\varepsilon\eta(x)$ 时, 泛函 $F[y]$ 的值如何变化来定义, 其中 $\eta(x)$ 是 x 的任意函数.

注

(重构新增) (B.3)与(B.1)的形状完全相同, 区别在于“求导变量”从一个数变成了一个函数: 一阶项中的 ε 系数不再是一个数, 而是 x 的函数 $\delta F/\delta y(x)$, 它必须与扰动 $\eta(x)$ 逐点相乘后再积分. 这里要求 $\eta(x)$ 是任意函数, 正是为了让 $\delta F/\delta y(x)$ 被唯一确定: 若只对某一个固定的 η 检验, 只能得到 $\int(\delta F/\delta y)\eta\,\mathrm{d}x$ 的信息, 而得不到被积函数本身. 定义中的 $O(\varepsilon^2)$ 是对所有这样的 η 一致成立的余项.

式(B.3)可以看作式(B.2)的自然扩展, 其中 $F[y]$ 依赖于一组连续的变量, 即所有点 x 处 y 的值. 如果要求泛函相对于函数 $y(x)$ 的微小变化是平稳的, 则有

$$\int\frac{\delta F}{\delta y(x)}\eta(x)\,\mathrm{d}x = 0 \tag{B.4}$$

由于(B.4)必须对任意选择的 $\eta(x)$ 都成立, 因此对于任意的 x, 泛函的导数都必须为零.

命题 B.1 (变分法的基本引理)

若(B.4)对积分区间内任意 (足够光滑且在边界处为零的) 函数 $\eta(x)$ 都成立, 则对该区间内的每个 x 都有 $\delta F/\delta y(x) = 0$.

♠

证明

为了说明这一点, 选择一个扰动 $\eta(x)$, 其在除了点 $\hat{x}$ 的邻域之外的任意位置都为零. 在这种情况下, 泛函的导数在 $x = \hat{x}$ 处必须为零. 但是, 由于对任意的 $\hat{x}$ 都必须满足此条件, 因此对于任意的 x, 泛函的导数都必须消除.

□

注

(重构新增) 这条引理是变分法的“局部化”机制: 它把“对整个区间的积分恒为零”这样一个全局条件, 转化为“被积函数在每一点为零”这样一个逐点条件. 论证的关键在于 $\eta(x)$ 可以任意选取: 只要在某点 $x = \hat{x}$ 处 $\delta F/\delta y(\hat{x}) \neq 0$, 由连续性就存在 $\hat{x}$ 的一个小邻域使该导数在这一邻域内不变号, 于是取只在这个邻域内非零、且与导数同号的 η,(B.4)中的积分就严格为正, 与假设矛盾. 原书用“消除”一词表达的就是“为零”; 引理只给出必要条件, 因此满足它的函数是驻点而不是 (必然的) 极值点, 这一点与普通微积分中“导数为零只是极值的必要条件”完全一致.

考虑一个定义在函数 $G(y, y', x)$ 的积分之上的泛函, 它同时还依赖于 $y(x)$ 及其导数 $y'(x)$, 并且直接依赖于 x:

定义 B.4 (积分型泛函)

$$F[y] = \int G(y(x), y'(x), x)\,\mathrm{d}x \tag{B.5}$$

其中, 假设 $y(x)$ 的值在积分的边界处 (可能在无穷大处) 为定值.

♣

注

(重构新增) 这类泛函只通过 y、y' 与 x 三个自变量依赖曲线, 因此称为一阶积分型泛函; 被积函数 G 不含 y'' 及更高阶导数, 是下文推导能只用一次分部积分的前提. “$y(x)$ 在边界处为定值”这一条件 (即端点固定) 不可省: 它保证在分部积分时边界项为零, 从而把变分条件集中到积分内部 (证明见定理 B.1). 若端点也允许变动, 就会出现额外的边界项,Euler-Lagrange 方程之外还要加上横截条件.

定理 B.1 (Euler-Lagrange 方程)

设 $y(x)$ 使(B.5)中的泛函在端点固定的函数类中取驻点, 即(B.4)对任意在边界处为零的 $\eta(x)$ 成立, 则 y 满足

$$\frac{\partial G}{\partial y} - \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial G}{\partial y'}\right) = 0 \tag{B.6}$$

♡

证明

考虑函数 $y(x)$ 的变动，我们可以得到

$$F[y(x)+\varepsilon\eta(x)] = F[y(x)] + \varepsilon \int \left\{ \frac{\partial G}{\partial y}\eta(x) + \frac{\partial G}{\partial y'}\eta'(x) \right\} \mathrm{d}x + O(\varepsilon^2) \tag{B.7}$$

(重构新增) 这一步是把 $G(y+\varepsilon\eta, y'+\varepsilon\eta', x)$ 对 ε 作一阶 Taylor 展开，展开点取 $\varepsilon = 0$，其线性项正是 $\dfrac{\partial G}{\partial y}\eta + \dfrac{\partial G}{\partial y'}\eta'$；原书未写出该展开，这里补全. 下面让我们以(B.3)的形式来表示(B.7). 为此，对(B.7)右侧的第二个积分作分部积分，并注意 $\eta(x)$ 必须在积分的边界处消除 (因为 $y(x)$ 在边界处为定值)，从而得到

$$F[y(x)+\varepsilon\eta(x)] = F[y(x)] + \varepsilon \int \left\{ \frac{\partial G}{\partial y} - \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial G}{\partial y'}\right) \right\} \eta(x)\,\mathrm{d}x + O(\varepsilon^2) \tag{B.8}$$

与(B.3)进行比较，即可确定泛函的导数

$$\frac{\delta F}{\delta y(x)} = \frac{\partial G}{\partial y} - \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial G}{\partial y'}\right).$$

由于要求消除泛函的导数，因此由命题 B.1即得(B.6).

(重构新增) 补全分部积分的细节：

$$\int \frac{\partial G}{\partial y'}\eta'(x)\,\mathrm{d}x = \left[\frac{\partial G}{\partial y'}\eta(x)\right]_{\text{边界}} - \int \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial G}{\partial y'}\right)\eta(x)\,\mathrm{d}x,$$

其中边界项因 η 在边界处为零而消失，正是这一步把 η' 换成了 η，使得两个积分可以合并为(B.8)中同一个被积函数. 这一步还要求 $\partial G/\partial y'$ 关于 x 可微，才能对其作 $\dfrac{\mathrm{d}}{\mathrm{d}x}$ 运算.

□

注

(重构新增) Euler-Lagrange 方程把“在函数空间中求驻点”这一无穷维问题化为一个二阶常微分方程 (若 G 只依赖 y 与 y'，方程中最高出现 y'')，从而可以用微分方程的边值理论求解. 推导中三处条件各有用处：G 只含一阶导数使分部积分只做一次；端点固定使边界项为零；η 任意使命题 B.1可以把积分条件化为逐点条件. 原书只给出结论，没有逐行写出 Taylor 展开与分部积分，上面已补全. 方程给出的是必要条件：满足它的解可能是极大值、极小值或鞍点，判断极值类型需要考察二阶变分.

例如，如果

例题 B.1 Euler-Lagrange 方程的一个例子

$$G = y(x)^2 + (y'(x))^2 \tag{B.9}$$

则 Euler-Lagrange 方程的形式变为

$$y(x) - \frac{\mathrm{d}^2 y}{\mathrm{d}x^2} = 0 \tag{B.10}$$

这个二阶微分方程可以利用 $y(x)$ 的边界条件来求解.

解

(重构新增) 补全从(B.9)到(B.10)的计算. 由 $G = y^2 + (y')^2$ 得

$$\frac{\partial G}{\partial y} = 2y, \qquad \frac{\partial G}{\partial y'} = 2y',$$

故 $\dfrac{\mathrm{d}}{\mathrm{d}x}\left(\dfrac{\partial G}{\partial y'}\right) = 2y''$. 代入(B.6)得 $2y - 2y'' = 0$，约去因子 2 即得(B.10). 该方程的通解为 $y(x) = Ae^{x} + Be^{-x}$，其中常数 A, B 由两端的边界值确定；当边界条件给定时，这两个条件恰好确定两个常数.

□

注

(重构新增) 这个例子说明 Euler-Lagrange 方程确实是一个微分方程：G 中对 y' 的平方项经 $\dfrac{\mathrm{d}}{\mathrm{d}x}$ 作用后产生 y''，而对 y 的平方项只产生 y，两者相减于是得到线性的二阶方程. 若把(B.9)换成 $G = \dfrac{1}{2}\dot{y}^2 - V(y)$(力学中的作用量)，Euler-Lagrange 方程就给出 Newton 第二定律；这说明变分原理与经典力学的运动方程是同一件事的两种表述.

考虑由积分定义的泛函，其积分采用 $G(y,x)$ 的形式，并且不依赖于 $y(x)$ 的导数. 在这种情况下，只要 x 的所有值都满足 $\partial G/\partial y(x) = 0$，就能保证解的稳定性.

命题 B.2 (不含导数的泛函的驻点条件)

若(B.5)中的 G 不依赖于 y', 即 $G = G(y, x)$, 则变分问题的驻点由逐点条件

$$\frac{\partial G}{\partial y(x)} = 0$$

给出, 其中 x 取遍积分区间内的所有值.

♠

证明

(重构新增) 原书只给出结论, 这里补全. 若 G 不依赖 y', 则 $\partial G/\partial y' = 0$, 于是(B.6)中的第二项 $\frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial G}{\partial y'}\right)$ 为零, 方程化为 $\partial G/\partial y = 0$; 由命题 B.1, 它对每个 x 都成立.

□

注

(重构新增) 原书用“保证解的稳定性”来描述这一条件, 其数学内容就是“驻点条件在每个 x 处逐点成立”: 因为方程中不含 y', 各点之间不再耦合, 变分问题退化为对每个 x 单独求 $G(\cdot, x)$ 的驻点. 这一退化情形在实际中很常见 (例如最大熵问题中 $G = -p\ln p$ 不含 p'), 它也是把变分问题与逐点最优化联系起来的桥梁. 注意此处仍需 G 关于 y 可微, 否则 $\partial G/\partial y$ 无意义.

如果我们关于概率分布优化一个泛函, 则需要保持概率的归一化约束. 满足该要求最简单的方式就是使用 Lagrange 乘子 (参见附录 C), 其允许进行无约束的优化.

上述结果可以直接扩展到多维变量 x 上. 有关变分法的更全面讨论, 请参阅 Sagan (1969).

附录 C Lagrange 乘子

Lagrange 乘子也称 Lagrange 乘数或未定乘子, 用于在一个或多个约束条件下, 寻找多变量函数的驻点.

考虑在与 x_1, x_2 有关的约束条件下找到函数 $f(x_1, x_2)$ 最大值的问题. 该约束条件可以写成如下形式:

$$g(x_1, x_2) = 0 \tag{C.1}$$

一种方法是求解约束方程 [(C.1)], 从而将 x_2 表示为 x_1 的函数, 形式为 $x_2 = h(x_1)$. 然后将其代入 $f(x_1, x_2)$, 得到形式为 $f(x_1, h(x_1))$ 的仅关于 x_1 的函数. 接下来, 可以通过常用的微分方法找到关于 x_1 的最大值, 得到驻点值 x_1^*, 相应的 x_2 值由 $x_2^* = h(x_1^*)$ 给出.

这种方法存在的一个问题是, 找到约束方程的一个解析解可能会很困难 (该方程使得 x_2 可以表示为 x_1 的显式函数). 此外, 这种方法区别对待 x_1 和 x_2, 因而会破坏这些变量之间的自然对称性.

一种更优雅且通常更简单的方法是引入一个称为 Lagrange 乘子的参数 λ. 我们将从几何角度来说明这一技术. 考虑一个具有 $x_1, \cdots, x_D$ 分量的 D 维向量 $\boldsymbol{x}$. 约束方程 $g(\boldsymbol{x}) = 0$ 表示 $\boldsymbol{x}$ 空间中的一个 $(D-1)$ 维曲面, 如图 C.1所示.

定义 C.1 (约束曲面)

约束方程 $g(\boldsymbol{x}) = 0$ 在 $\boldsymbol{x}$ 空间中表示的那个 $(D-1)$ 维曲面, 称为**约束曲面**; 当 $D = 2$ 时它就是一条约束曲线. ♣

注

(重构新增) "$(D-1)$ 维" 来自一个方程消去一个自由度这一朴素计数, 但它成立需要条件: 若在某点处 $\nabla g(\boldsymbol{x}) \neq \boldsymbol{0}$, 则由隐函数定理, $g(\boldsymbol{x}) = 0$ 在该点附近确实是 $D-1$ 维的光滑曲面; 若 $\nabla g = 0$, 曲面可能退化成一个点 (例如 $g(\boldsymbol{x}) = \|\boldsymbol{x}\|^2 = 0$ 只给出原点), 计数就失效. 本附录后面所有几何论证都默认 $\nabla g \neq \boldsymbol{0}$, 命题 C.1的证明中也显式用到这一点.

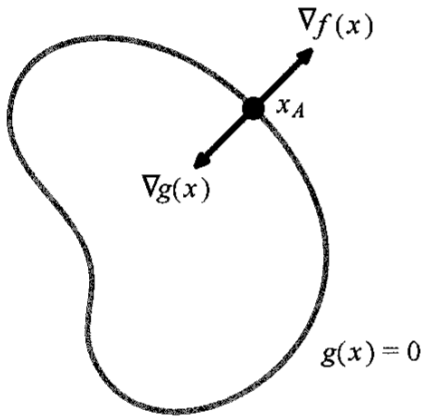


图 C.1: Lagrange 乘子法的几何解释. 我们想要在约束条件 $g(\boldsymbol{x}) = 0$ 下最大化函数 $f(\boldsymbol{x})$. 如果 $\boldsymbol{x}$ 是 D 维的, 则约束条件 $g(\boldsymbol{x}) = 0$ 对应一个维度为 $D-1$ 的子空间, 如红色曲线所示. 这个问题可以通过优化 Lagrange 函数 $L(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) + \lambda g(\boldsymbol{x})$ 来解决.

首先注意, 在该约束曲面上的任何一点, 约束函数的梯度 $\nabla g(\boldsymbol{x})$ 都与曲面正交.

命题 C.1 (梯度与约束曲面正交)

设 g 在点 $\boldsymbol{x}$ 处连续可微, $g(\boldsymbol{x}) = 0$ 且 $\nabla g(\boldsymbol{x}) \neq \boldsymbol{0}$. 则 $\nabla g(\boldsymbol{x})$ 与约束曲面在 $\boldsymbol{x}$ 处的切空间正交: 对曲面在 $\boldsymbol{x}$ 处的任意切向量 $\boldsymbol{\varepsilon}$, 都有 $\boldsymbol{\varepsilon}^{\mathrm{T}} \nabla g(\boldsymbol{x}) = 0$. ♠

证明

为了说明这一点,考虑一个位于该约束曲面上的点 $\boldsymbol{x}$ 以及另一个同样位于该约束曲面上的附近点 $\boldsymbol{x}+\boldsymbol{\varepsilon}$. 如果我们在点 $\boldsymbol{x}$ 附近进行 Taylor 展开,则有

$$g(\boldsymbol{x}+\boldsymbol{\varepsilon}) \approx g(\boldsymbol{x}) + \boldsymbol{\varepsilon}^{\mathsf{T}} \nabla g(\boldsymbol{x}) \tag{C.2}$$

因为点 $\boldsymbol{x}$ 和点 $\boldsymbol{x}+\boldsymbol{\varepsilon}$ 都位于约束曲面上,所以我们有 $g(\boldsymbol{x}) = g(\boldsymbol{x}+\boldsymbol{\varepsilon})$,从而有 $\boldsymbol{\varepsilon}^{\mathsf{T}} \nabla g(\boldsymbol{x}) \approx 0$. 当 $\|\boldsymbol{\varepsilon}\| \to 0$ 时,我们有 $\boldsymbol{\varepsilon}^{\mathsf{T}} \nabla g(\boldsymbol{x}) = 0$,并且因为 $\boldsymbol{\varepsilon}$ 与约束曲面 $g(\boldsymbol{x}) = 0$ 平行,所以我们看到 ∇g 与约束曲面是垂直的.

(重构新增) 上面“≈ 取极限”一步补全如下:取曲面上的光滑曲线 $\boldsymbol{\gamma}(t)$ 使 $\boldsymbol{\gamma}(0) = \boldsymbol{x}$、$\boldsymbol{\gamma}'(0) = \boldsymbol{\varepsilon}$,则 $g(\boldsymbol{\gamma}(t)) \equiv 0$,对 t 求导并取 $t = 0$ 得 $\boldsymbol{\gamma}'(0)^{\mathsf{T}} \nabla g(\boldsymbol{x}) = \boldsymbol{\varepsilon}^{\mathsf{T}} \nabla g(\boldsymbol{x}) = 0$. 可见该等式是精确的,不依赖 $O(\|\boldsymbol{\varepsilon}\|^2)$ 的余项;同时它说明“切向量”就是与 $\nabla g(\boldsymbol{x})$ 正交的向量,故 $\nabla g(\boldsymbol{x})$ 是曲面的法向量.

□

注

(重构新增) 这条命题是 Lagrange 乘子法的几何基础:它把“约束曲面”这个几何对象翻译成“梯度”这个可计算对象,从而可以用向量运算代替曲面上的移动. 条件 $\nabla g(\boldsymbol{x}) \neq \boldsymbol{0}$ 不能省:若梯度为零,曲面在该点可能退化,“法向量”无从定义,后面的“平行”论断也就失去依据. 命题本身只断言 ∇g 与切空间正交,并未断言任何极值性质——极值条件是下一条命题的内容.

接下来我们寻找约束曲面上使得 $f(\boldsymbol{x})$ 最大化的点 $\boldsymbol{x}^*$. 这样的点满足 $\nabla f(\boldsymbol{x})$ 也与约束曲面垂直的性质,如图 C.1所示,否则我们可以通过沿约束曲面移动一小段距离来增大 $f(\boldsymbol{x})$ 的值. 因此,∇f 和 ∇g 是平行 (或反平行) 的,并且必定存在一个参数 λ,使得

命题 C.2 (约束极值点的必要条件)

设 $\boldsymbol{x}^*$ 是 f 在约束曲面 $g(\boldsymbol{x}) = 0$ 上的局部极大值点,f、g 在 $\boldsymbol{x}^*$ 附近连续可微且 $\nabla g(\boldsymbol{x}^*) \neq \boldsymbol{0}$. 则存在实数 λ,使得

$$\nabla f + \lambda \nabla g = 0 \tag{C.3}$$

这里的梯度都在 $\boldsymbol{x}^*$ 处取值.

♠

证明

原书论证如下:在约束曲面上使 f 最大的点 $\boldsymbol{x}^*$ 处,$\nabla f(\boldsymbol{x}^*)$ 必须与约束曲面垂直,否则我们可以沿约束曲面移动一小段距离来增大 f 的值;而由命题 C.1,$\nabla g(\boldsymbol{x}^*)$ 也与曲面垂直. 因此 ∇f 和 ∇g 是平行 (或反平行) 的,必定存在一个参数 λ 使(C.3)成立.

(重构新增) 补全“平行”的理由:由命题 C.1,$\nabla f(\boldsymbol{x}^*)$ 与 $\nabla g(\boldsymbol{x}^*)$ 都正交于曲面在 $\boldsymbol{x}^*$ 处的切空间;当 $\nabla g(\boldsymbol{x}^*) \neq \boldsymbol{0}$ 时该切空间的余维为 1,其正交补是一维的,于是两个非零向量共线,即存在 λ 使 $\nabla f(\boldsymbol{x}^*) = -\lambda \nabla g(\boldsymbol{x}^*)$. 若 $\nabla g(\boldsymbol{x}^*) = \boldsymbol{0}$,余维论断失效,结论可能不成立,这正是本命题需要该条件的原因.

□

注

(重构新增) (C.3)给出的是必要条件而非充分条件:满足它的点可能是极大值、极小值或鞍点,判断类型需要看二阶信息 (或直接在约束曲面上比较函数值). 条件 $\nabla g(\boldsymbol{x}^*) \neq \boldsymbol{0}$ 称为约束规范条件,它保证约束曲面在极值点附近是光滑的;若梯度为零,即使 $\boldsymbol{x}^*$ 是极值点也可能找不到这样的 λ. 此外,λ 的符号不是任意的:在等式约束下它可正可负,而在不等式约束下它会被符号条件固定 (见命题 C.4).

其中 $\lambda \neq 0$ 称为 Lagrange 乘子. 注意,λ 可正可负.

定义 C.2 (Lagrange 乘子)

在(C.3)中出现的参数 λ 称为 **Lagrange 乘子** (Lagrange multiplier),它度量约束对目标函数驻点的影响强度.

♣

注

(重构新增) 乘子 λ 不是待求的“中间量”, 它本身携带信息:$|\lambda|$ 越大, 说明约束 $g(\boldsymbol{x})=0$ 对最优值的影响越强 (在灵敏度分析中,λ 恰是目标函数最优值对约束右端项的导数). 定义中 $\lambda\neq 0$ 是等式约束情形的约定; 在不等式约束下, 非激活约束对应 $\lambda=0$(见定义 C.5), 因此那里不再要求 $\lambda\neq 0$.λ 可正可负, 这一点与命题 C.4中 $\lambda\geqslant 0$ 的符号限制形成对照.

至此, 我们很容易引出下式定义的 Lagrange 函数:

定义 C.3 (Lagrange 函数)

$$L(\boldsymbol{x},\lambda)=f(\boldsymbol{x})+\lambda g(\boldsymbol{x}) \tag{C.4}$$

称为与约束问题(C.1)对应的 **Lagrange 函数** (Lagrangian).

注

(重构新增) 引入 L 的目的是把“带约束的驻点问题”改写成“无约束的驻点问题”:L 把约束 g 以乘子 λ 为权并入目标函数, 于是约束是否被满足不再需要单独处理, 而由 $\partial L/\partial\lambda=g(\boldsymbol{x})=0$ 自动给出 (见命题 C.3). 这一构造的关键是 λ 被当作与 $\boldsymbol{x}$ 平等的自变量: 正因如此,L 的驻点条件才同时包含(C.3)与约束方程两组方程.

可通过设置 $\nabla_{\boldsymbol{x}}L=\boldsymbol{0}$ 得到带约束的驻点条件 [(C.3)]. 此外, 通过 $\partial L/\partial\lambda=0$ 可推出约束条件 $g(\boldsymbol{x})=0$. 为了找到在约束条件 $g(\boldsymbol{x})=0$ 下函数 $f(\boldsymbol{x})$ 的最大值, 首先定义(C.4)所示的 Lagrange 函数, 然后寻找 $L(\boldsymbol{x},\lambda)$ 关于 $\boldsymbol{x}$ 和 λ 的驻点. 对于一个 D 维向量 $\boldsymbol{x}$, 这会给出 $D+1$ 个方程, 用于确定驻点 $\boldsymbol{x}^*$ 以及 λ 的值. 如果只对 $\boldsymbol{x}^*$ 感兴趣, 则可以从驻点方程中消去 λ, 而不需要计算 λ 的值 (“未定乘子”由此得名).

命题 C.3 (Lagrange 乘子法)

设 f、g 连续可微. 点 $\boldsymbol{x}^*$ 满足(C.3)与约束 $g(\boldsymbol{x}^*)=0$, 当且仅当存在 λ^* 使 L 在 $(\boldsymbol{x}^*,\lambda^*)$ 处关于 $\boldsymbol{x}$ 与 λ 的偏导数都为零, 即

$$\nabla_{\boldsymbol{x}}L(\boldsymbol{x}^*,\lambda^*)=\boldsymbol{0},\qquad \frac{\partial L}{\partial\lambda}(\boldsymbol{x}^*,\lambda^*)=0.$$

证明

(重构新增) 原书把这一等价性直接作为方法的描述, 这里补全两向论证. 由(C.4),$\nabla_{\boldsymbol{x}}L=\nabla f+\lambda\nabla g$, 而 $\partial L/\partial\lambda=g(\boldsymbol{x})$. 若 $\boldsymbol{x}^*$ 满足(C.3)与 $g(\boldsymbol{x}^*)=0$, 取 λ^* 为(C.3)中的乘子, 则两个偏导数同时为零. 反之, 若存在 λ^* 使两个偏导数为零, 则 $\partial L/\partial\lambda=0$ 给出 $g(\boldsymbol{x}^*)=0$, 而 $\nabla_{\boldsymbol{x}}L=\boldsymbol{0}$ 给出 $\nabla f(\boldsymbol{x}^*)+\lambda^*\nabla g(\boldsymbol{x}^*)=\boldsymbol{0}$, 即(C.3). 方程个数方面:$\nabla_{\boldsymbol{x}}L=\boldsymbol{0}$ 是 D 个方程,$\partial L/\partial\lambda=0$ 是 1 个方程, 共 $D+1$ 个, 未知量为 D 个分量与 λ, 故一般可确定离散的驻点. □

注

(重构新增) 这条命题说明 Lagrange 乘子法并没有引入新的数学原理, 它只是把(C.3)与约束方程合并成同一个函数 L 的驻点条件, 从而可以套用无约束优化的全部工具. “未定乘子”这个名字来自实践: 若只关心 $\boldsymbol{x}^*$, 可以把 λ 当作中间变量从方程组中消去, 例如从 $\nabla f=-\lambda\nabla g$ 的两个分量式中解出 λ 再令其相等, 就得到只含 $\boldsymbol{x}$ 的方程. 注意该等价性只依赖可微性, 不保证驻点一定是极值点.

举个简单的例子, 假设我们希望找到函数 $f(x_1,x_2)=1-x_1^2-x_2^2$ 在约束条件 $g(x_1,x_2)=x_1+x_2-1=0$ 下的驻点, 如图 C.2所示. 相应的 Lagrange 函数由下式给出:

例题 C.1 二次函数的约束驻点

$$L(\boldsymbol{x},\lambda)=1-x_1^2-x_2^2+\lambda(x_1+x_2-1) \tag{C.5}$$

这个 Lagrange 函数关于 x_1、x_2 和 λ 的驻点条件给出了如下耦合方程:

$$-2x_1+\lambda=0 \tag{C.6}$$

$$-2x_2+\lambda=0 \tag{C.7}$$

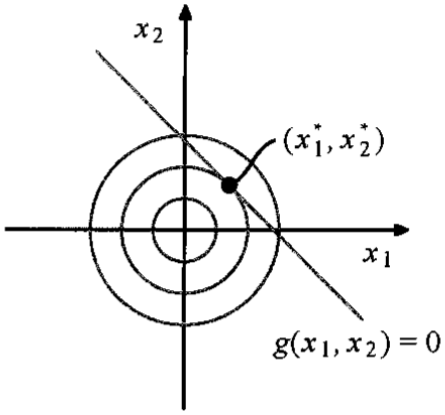


图 C.2: 使用 Lagrange 乘子法的一个简单示例. 在约束条件 $g(x_1, x_2) = 0$ 下最大化函数 $f(x_1, x_2) = 1 - x_1^2 - x_2^2$, 其中 $g(x_1, x_2) = x_1 + x_2 - 1$. 圆圈表示函数 $f(x_1, x_2)$ 的等值线, 对角线则代表约束曲面 $g(x_1, x_2) = 0$.

$$x_1 + x_2 - 1 = 0 \tag{C.8}$$

解这些方程, 可以得到驻点 $(x_1^*, x_2^*) = (1/2, 1/2)$, 相应的 Lagrange 乘子为 $\lambda = 1$.

解

(重构新增) 补全求解过程. 由(C.6)与(C.7)得 $x_1 = x_2 = \lambda/2$; 代入(C.8)得 $\lambda/2+\lambda/2-1 = 0$, 故 $\lambda = 1$, 于是 $x_1^* = x_2^* = 1/2$. 此时 $f(x_1^*, x_2^*) = 1 - 1/4 - 1/4 = 1/2$.

□

注

(重构新增) 这个例子可以纯几何地验证: f 的等值线是以原点为心的圆, 约束是一条直线, 直线上离原点最近的点正是 $(1/2, 1/2)$, 它使 f 最大 (因为 f 随到原点距离增大而减小), 而该点处圆的切线与直线相切, 故 ∇f 与 ∇g 共线——这正是(C.3)的几何内容. 乘子 $\lambda = 1 > 0$ 也符合直觉: 约束 $x_1 + x_2 = 1$ 把最优点从原点推离, 且推得越远 λ 越大.

到目前为止, 我们已经讨论了在形如 $g(\boldsymbol{x}) = 0$ 的等式约束 (equality constraint) 下如何最大化函数的问题. 接下来考虑在形如 $g(\boldsymbol{x}) \geqslant 0$ 的不等式约束 (inequality constraint) 下如何最大化 $f(\boldsymbol{x})$ 的问题, 如图 C.3所示.

定义 C.4 (等式约束与不等式约束)

形如 $g(\boldsymbol{x}) = 0$ 的约束称为**等式约束** (equality constraint); 形如 $g(\boldsymbol{x}) \geqslant 0$ 的约束称为**不等式约束** (inequality constraint).

♣

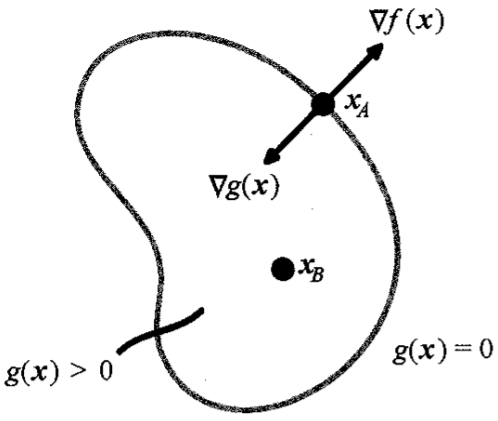


图 C.3: 在不等式约束 $g(\boldsymbol{x}) \geqslant 0$ 下最大化函数 $f(\boldsymbol{x})$ 的问题示意图.

注

(重构新增) 等式约束要求最优点恰好落在曲面上, 可行集是“薄”的 $D-1$ 维集合; 不等式约束只要求 $g \geqslant 0$, 可行集是带内部的 D 维区域, 它的边界才是 $g=0$. 这一差别决定了乘子符号: 等式约束两侧都可移动, 故 λ 可正可负; 不等式约束只能向区域内部移动, 故乘子的符号被固定 (见命题 C.4). 后文所有结论都要求 g 连续可微, 且在最优点处 $\nabla g \neq \mathbf{0}$.

根据带约束的驻点是否位于 $g(\boldsymbol{x})>0$ 的区域内, 这个问题有两种可能的解.

定义 C.5 (激活约束与非激活约束)

如果驻点位于 $g(\boldsymbol{x})>0$ 的区域内, 则约束是**非激活的** (inactive); 而如果驻点位于边界 $g(\boldsymbol{x})=0$ 上, 则约束是**激活的** (active).

注

(重构新增) “激活”是对约束是否起作用的白话描述: 非激活时约束把最优点放过了, 去掉它最优解不变; 激活时最优点被压在边界上, 去掉约束后最优解会跑到区域外. 这个二分法与下一命题的两种情形一一对应, 也是(C.11)(互补松弛条件) 的直观来源:$\lambda g(\boldsymbol{x})=0$ 说的正是“要么约束不起作用 ($\lambda=0$), 要么边界起作用 ($g=0$)”, 两种情形不能同时“半起作用”.

在前一种情况下, 函数 $g(\boldsymbol{x})$ 不起作用, 因此驻点条件是 $\nabla f(\boldsymbol{x})=\mathbf{0}$, 这对应于 $\lambda=0$ 的 Lagrange 函数 [(C.4)] 的驻点. 在后一种情况下, 驻点位于边界上, 类似于前面讨论的等式约束, 对应于 $\lambda \neq 0$ 的 Lagrange 函数 [(C.4)] 的驻点. 可见 Lagrange 乘子的符号是关键, 因为只有当 $f(\boldsymbol{x})$ 的梯度远离 $g(\boldsymbol{x})>0$ 的区域时, 函数 $f(\boldsymbol{x})$ 才会达到最大值, 如图 C.3所示. 因此, 当 $\lambda>0$ 时, 我们有 $\nabla f(\boldsymbol{x})=-\lambda \nabla g(\boldsymbol{x})$.

命题 C.4 (不等式约束下驻点的两种情形)

设 $\boldsymbol{x}^*$ 是 f 在区域 $g(\boldsymbol{x}) \geqslant 0$ 上的局部极大值点,f、g 连续可微, 且在最优点处 $\nabla g(\boldsymbol{x}^*) \neq \mathbf{0}$. 则恰有两种可能:

1. 非激活情形:$g(\boldsymbol{x}^*)>0$, 此时 $\nabla f(\boldsymbol{x}^*)=\mathbf{0}$, 且相应的乘子为零;
2. 激活情形:$g(\boldsymbol{x}^*)=0$, 此时存在 $\lambda>0$ 使 $\nabla f(\boldsymbol{x}^*)=-\lambda \nabla g(\boldsymbol{x}^*)$.

证明

原书论证如下: 在前一种情况下, 函数 $g(\boldsymbol{x})$ 不起作用, 因此驻点条件是 $\nabla f(\boldsymbol{x})=\mathbf{0}$, 这对应于 $\lambda=0$ 的 Lagrange 函数的驻点; 在后一种情况下, 驻点位于边界上, 类似于前面讨论的等式约束, 对应于 $\lambda \neq 0$ 的 Lagrange 函数的驻点. 可见 Lagrange 乘子的符号是关键, 因为只有当 $f(\boldsymbol{x})$ 的梯度远离 $g(\boldsymbol{x})>0$ 的区域时, 函数 $f(\boldsymbol{x})$ 才会达到最大值. 因此, 当 $\lambda>0$ 时, 我们有 $\nabla f(\boldsymbol{x}^*)=-\lambda \nabla g(\boldsymbol{x}^*)$.

(重构新增) 补全符号论证. 非激活情形是区域内部的驻点, 无需约束, 故 $\nabla f(\boldsymbol{x}^*)=\mathbf{0}$. 激活情形中, 任取方向 $\boldsymbol{d}$ 满足 $\nabla g(\boldsymbol{x}^*)^{\mathsf{T}}\boldsymbol{d}>0$(即 $\boldsymbol{d}$ 指向区域 $g>0$ 的内部, 这样的 $\boldsymbol{d}$ 存在是因为 $\nabla g(\boldsymbol{x}^*) \neq \mathbf{0}$), 当 $t>0$ 充分小时点 $\boldsymbol{x}^*+t\boldsymbol{d}$ 仍可行; 由 $\boldsymbol{x}^*$ 是极大值点得一阶必要条件 $\nabla f(\boldsymbol{x}^*)^{\mathsf{T}}\boldsymbol{d} \leqslant 0$. 于是 $\nabla f(\boldsymbol{x}^*)$ 在所有使 $\nabla g(\boldsymbol{x}^*)^{\mathsf{T}}\boldsymbol{d}>0$ 的方向上取非正值, 这迫使 $\nabla f(\boldsymbol{x}^*)$ 与 $\nabla g(\boldsymbol{x}^*)$ 反向共线, 即存在 $\lambda \geqslant 0$ 使 $\nabla f(\boldsymbol{x}^*)=-\lambda \nabla g(\boldsymbol{x}^*)$; 再由 $g(\boldsymbol{x}^*)=0$ 与命题 C.1知 $\nabla f(\boldsymbol{x}^*) \neq \mathbf{0}$, 故 $\lambda>0$.

□

注

(重构新增) 两种情形的分界是“约束是否被触及”, 而 λ 的符号则记录“约束把最优点往哪边推”:$\lambda>0$ 表示目标函数的上升方向被约束挡住. 这一符号条件是不等式约束与等式约束的本质区别: 在等式约束下 λ 可正可负 (见定义 C.2), 而在不等式约束下 $\lambda \geqslant 0$. 命题给出的是必要条件; 由于可行集带有边界, 极大值点还可能出现在不可微的角点处, 此时 ∇g 不存在, 本命题不适用.

对于这两种情况, 乘积 $\lambda g(\boldsymbol{x})=0$. 因此, 在 $g(\boldsymbol{x}) \geqslant 0$ 的不等式约束下最大化 $f(\boldsymbol{x})$ 的问题, 可通过在以下条件下关于 $\boldsymbol{x}$ 和 λ 优化 Lagrange 函数 [(C.4)] 来求解:

定理 C.1 (Karush-Kuhn-Tucker 条件)

在约束 $g(\boldsymbol{x}) \geqslant 0$ 下最大化 $f(\boldsymbol{x})$ 的问题, 其驻点满足

$$g(\boldsymbol{x}) \geqslant 0 \tag{C.9}$$

$$\lambda \geqslant 0 \tag{C.10}$$

$$\lambda g(\boldsymbol{x}) = 0 \tag{C.11}$$

并且 $\nabla_{\boldsymbol{x}} L(\boldsymbol{x}, \lambda) = \boldsymbol{0}$. 我们称它们为 **Karush-Kuhn-Tucker**(KKT) 条件 (Karush, 1939; Kuhn and Tucker, 1951). ♡

证明

(重构新增) 原书由两种情形直接归纳出这三条, 这里补全. 由命题 C.4, 驻点只有两种可能. 非激活情形:$g(\boldsymbol{x}) > 0$, 乘子 $\lambda = 0$, 于是 $\lambda \geqslant 0$ 与 $\lambda g(\boldsymbol{x}) = 0$ 同时成立, 且 $\nabla_{\boldsymbol{x}} L = \nabla f + \lambda \nabla g = \nabla f = \boldsymbol{0}$. 激活情形:$g(\boldsymbol{x}) = 0, \lambda > 0$, 于是 $\lambda \geqslant 0$ 成立且 $\lambda g(\boldsymbol{x}) = 0$, 而 $\nabla_{\boldsymbol{x}} L = \boldsymbol{0}$ 正是(C.3). 两种情形中 $g(\boldsymbol{x}) \geqslant 0$ 都成立, 故(C.9)–(C.11)与 $\nabla_{\boldsymbol{x}} L = \boldsymbol{0}$ 都是必要条件. □

注

(重构新增) 三条条件各司其职:(C.9)是可行性 (最优点必须落在可行区域内),(C.10)是不等式约束的乘子符号,(C.11)称为互补松弛条件, 它把“激活/非激活”这一定性区分写成了一个方程. 互补松弛的含义是:$g(\boldsymbol{x}) > 0$ 时必有 $\lambda = 0, \lambda > 0$ 时必有 $g(\boldsymbol{x}) = 0$, 两者不能同时严格为正. 需要强调 KKT 条件只是必要条件: 满足它们的点可能是极大值、极小值或鞍点; 当 f 为凹函数、g 为凹函数且满足约束规范时,KKT 条件才是充分条件.

注意, 如果我们希望在不等式约束 $g(\boldsymbol{x}) \geqslant 0$ 下最小化 (而非最大化) 函数 $f(\boldsymbol{x})$, 则需要关于 $\boldsymbol{x}$ 最小化 Lagrange 函数 $L(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) - \lambda g(\boldsymbol{x})$, 这同样要求 $\lambda \geqslant 0$.

命题 C.5 (最小化情形的 KKT 条件)

若把定理 C.1中的“最大化 f”改为“最小化 f”, 则相应的 Lagrange 函数改为 $L(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) - \lambda g(\boldsymbol{x})$, 其余三条条件(C.9)–(C.11)不变, 仍要求 $\lambda \geqslant 0$. ♠

证明

(重构新增) 原书只给出结论, 这里补全. 令 $\tilde{f} = -f$, 则“在 $g \geqslant 0$ 下最小化 f”等价于“在 $g \geqslant 0$ 下最大化 $\tilde{f}$”. 对 $\tilde{f}$ 用定理 C.1, 存在 $\lambda \geqslant 0$ 使 $\nabla \tilde{f} + \lambda \nabla g = \boldsymbol{0}$, 即 $-\nabla f + \lambda \nabla g = \boldsymbol{0}$, 等价于 $\nabla(f - \lambda g) = \boldsymbol{0}$. 这正是以 $f - \lambda g$ 为 Lagrange 函数、关于 $\boldsymbol{x}$ 求驻点的条件, 而 $\lambda \geqslant 0$、$g \geqslant 0$、$\lambda g = 0$ 均不变. □

注

(重构新增) 最大化与最小化的差别只在于 L 中 λg 的符号, 这与“把目标函数取负”是同一件事; 乘子的符号条件却保持不变, 因为它是约束的几何性质 (只能向区域内移动) 而不是目标的性质. 使用这条结论时最容易出错的地方是: 最小化时若仍写成 $f + \lambda g$, 则必须配 $\lambda \leqslant 0$, 两种写法不能混用.

最后,Lagrange 乘子法可以很容易地扩展到具有多个等式约束和不等式约束的情况. 假设我们希望在约束条件 $g_j(\boldsymbol{x}) = 0 (j = 1, \cdots, J)$ 和 $h_k(\boldsymbol{x}) \geqslant 0 (k = 1, \cdots, K)$ 下最大化 $f(\boldsymbol{x})$, 则可以引入 Lagrange 乘子 $\{\lambda_j\}$ 和 $\{\mu_k\}$, 并优化如下 Lagrange 函数:

定理 C.2 (多个约束下的 KKT 条件)

设 f、g_j、h_k 连续可微,$\boldsymbol{x}^*$ 是 f 在约束 $g_j(\boldsymbol{x}) = 0 (j = 1, \cdots, J)$ 与 $h_k(\boldsymbol{x}) \geqslant 0 (k = 1, \cdots, K)$ 下的局部极大值点, 且各约束的梯度在 $\boldsymbol{x}^*$ 处线性无关. 则存在乘子 $\{\lambda_j\}$ 与 $\{\mu_k\}$, 使

$$L(\boldsymbol{x}, \{\lambda_j\}, \{\mu_k\}) = f(\boldsymbol{x}) + \sum_{j=1}^{J} \lambda_j g_j(\boldsymbol{x}) + \sum_{k=1}^{K} \mu_k h_k(\boldsymbol{x}) \tag{C.12}$$

在 $\boldsymbol{x}^*$ 处关于 $\boldsymbol{x}$ 的梯度为零, 其中 $\mu_k \geqslant 0$ 且 $\mu_k h_k(\boldsymbol{x}) = 0, k = 1, \cdots, K$. ♡

证明

(重构新增) 原书未给证明, 这里补全思路. 把等式约束视为双侧约束、不等式约束视为单侧约束, 重复命题 C.4的论证: 在 $\boldsymbol{x}^*$ 处, 所有激活的不等式约束 ($h_k(\boldsymbol{x}^*) = 0$) 与等式约束一起把可行方向限制在一个多面锥内, 而局部极大性要求 $\nabla f(\boldsymbol{x}^*)$ 在该锥的所有方向上取非正值; 由 Farkas 引理 (或梯度的线性无关条件), 这等价于 $\nabla f(\boldsymbol{x}^*)$ 可以写成等式约束梯度与激活不等式约束梯度的非负线性组合的相反数, 即存在 λ_j 与 $\mu_k \geqslant 0$ 使

$$\nabla f(\boldsymbol{x}^*) + \sum_{j=1}^{J} \lambda_j \nabla g_j(\boldsymbol{x}^*) + \sum_{k=1}^{K} \mu_k \nabla h_k(\boldsymbol{x}^*) = \boldsymbol{0},$$

其中非激活的 h_k 取 $\mu_k = 0$, 故 $\mu_k h_k(\boldsymbol{x}^*) = 0$ 对每个 k 成立. 这正是(C.12)关于 $\boldsymbol{x}$ 的梯度为零.

□

注

(重构新增) (C.12)只是把单约束的 Lagrange 函数按约束个数线性叠加: 每个等式约束配一个符号自由的乘子 λ_j, 每个不等式约束配一个非负乘子 μ_k, 互补松弛 $\mu_k h_k(\boldsymbol{x}) = 0$ 对每个不等式约束分别成立. 定理中“梯度线性无关”是约束规范条件, 它排除约束曲面在极值点处相互重合或退化导致的乘子不唯一; 若该条件不满足, 乘子可能不存在或有无穷多组,KKT 条件就不再是必要条件. 与单约束情形一样, 这些条件只是必要条件, 充分性需要额外的凸性假设. 关于 Lagrange 乘子法更详细的讨论, 请参阅 Nocedal and Wright (1999).

参考资料

[1] Abramowitz, M., and I. A. Stegun. 1965. *Handbook of Mathematical Functions.* Dover.

[2] Adler, S. L. 1981. "Over-relaxation method for the Monte Carlo evaluation of the partition function for multiquadratic actions." *Physical Review D* 23:2901–2904.

[3] Aghajanyan, Armen, Bernie Huang, Candace Ross, Vladimir Karpukhin, Hu Xu, Naman Goyal, Dmytro Okhonko, et al. 2022. *CM3: A Causal Masked Multimodal Model of the Internet.* Technical report. arXiv:2201.07520.

[4] Aghajanyan, Armen, Luke Zettlemoyer, and Sonal Gupta. 2020. *Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning.* Technical report. arXiv:2012.13255.

[5] Ahn, J. H., and J. H. Oh. 2003. "A constrained EM algorithm for principal component analysis." *Neural Computation* 15 (1): 57–65.

[6] Alayrac, Jean-Baptiste, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, et al. 2022. *Flamingo: a Visual Language Model for Few-Shot Learning.* Technical report. arXiv:2204.14198.

[7] Amari, S., A. Cichocki, and H. H. Yang. 1996. "A new learning algorithm for blind signal separation." In *Advances in Neural Information Processing Systems*, edited by D. S. Touretzky, M. C. Mozer, and M. E. Hasselmo, 8:757–763. MIT Press.

[8] Anderson, J. A., and E. Rosenfeld. 1988. *Neurocomputing: Foundations of Research.* MIT Press.

[9] Anderson, T. W. 1963. "Asymptotic Theory for Principal Component Analysis." *Annals of Mathematical Statistics* 34:122–148.

[10] Arjovsky, M., S. Chintala, and L. Bottou. 2017. *Wasserstein GAN.* Technical report. arXiv:1701.07875.

[11] Attias, H. 1999. "Independent factor analysis." *Neural Computation* 11 (4): 803–851.

[12] Austin, Jacob, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. 2021. "Structured Denoising Diffusion Models in Discrete State-Spaces." In *Advances in Neural Information Processing Systems*, 34:17981–17993.

[13] Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E Hinton. 2016. *Layer Normalization.* Technical report. arXiv:1607.06450.

[14] Bach, F. R., and M. I. Jordan. 2002. "Kernel Independent Component Analysis." *Journal of Machine Learning Research* 3:1–48.

[15] Badrinarayanan, Vijay, Alex Kendall, and Roberto Cipolla. 2015. *SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation.* Technical report. arXiv:1511.00561.

[16] Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. 2014. *Neural Machine Translation by Jointly Learning to Align and Translate.* Technical report. arXiv:1409.0473.

[17] Baldi, P., and K. Hornik. 1989. "Neural networks and principal component analysis: learning from examples without local minima." *Neural Networks* 2 (1): 53–58.

[18] Balduzzi, David, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, and Brian McWilliams. 2017. *The Shattered Gradients Problem: If resnets are the answer, then what is the question?* Technical report. arXiv:1702.08591.

[19] Bartholomew, D J. 1987. *Latent Variable Models and Factor Analysis.* Charles Griffin.

[20] Basilevsky, Alexander. 1994. *Statistical Factor Analysis and Related Methods: Theory and Applications.* Wiley.

[21] Bather, J. 2000. *Decision Theory: An Introduction to Dynamic Programming and Sequential Decisions.* Wiley.

[22] Battaglia, Peter W., Jessica B. Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, et al. 2018. *Relational inductive biases, deep learning, and graph networks.* Technical report. arXiv:1806.01261.

[23] Baydin, A. G., B. A. Pearlmutter, A. A. Radul, and J. M. Siskind. 2018. "Automatic differentiation in machine learning: a survey." *Journal of Machine Learning Research* 18:1–43.

[24] Becker, S., and Y. LeCun. 1989. "Improving the convergence of back-propagation learning with second order methods." In *Proceedings of the 1988 Connectionist Models Summer School*, edited by D. Touretzky, G. E. Hinton, and T. J. Sejnowski, 29–37. Morgan Kaufmann.

[25] Belkin, Mikhail, Daniel Hsu, Siyuan Ma, and Soumik Mandal. 2019. “Reconciling modern machine-learning practice and the classical bias-variance trade-off.” *Proceedings of the National Academy of Sciences* 116 (32): 15849–15854.

[26] Bell, A. J., and T. J. Sejnowski. 1995. “An information maximization approach to blind separation and blind deconvolution.” *Neural Computation* 7 (6): 1129–1159.

[27] Bellman, R. 1961. *Adaptive Control Processes: A Guided Tour.* Princeton University Press.

[28] Bengio, Yoshua, Aaron Courville, and Pascal Vincent. 2012. *Representation Learning: A Review and New Perspectives.* Technical report. arXiv:1206.5538.

[29] Bengio, Yoshua, Nicholas Léonard, and Aaron Courville. 2013. *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation.* Technical report. arXiv:1308.3432.

[30] Berger, J. O. 1985. *Statistical Decision Theory and Bayesian Analysis.* Second. Springer.

[31] Bernardo, J. M., and A. F. M. Smith. 1994. *Bayesian Theory.* Wiley.

[32] Bishop, C. M. 1995a. “Regularization and Complexity Control in Feed-forward Networks.” In *Proceedings International Conference on Artificial Neural Networks ICANN’95*, edited by F. Fogelman-Soulie and P. Gallinari, 1:141–148. EC2 et Cie.

[33] Bishop, Christopher M. 1992. “Exact Calculation of the Hessian Matrix for the Multilayer Perceptron.” *Neural Computation* 4 (4): 494–501.

[34] Bishop, Christopher M. 1994. “Novelty Detection and Neural Network Validation.” *IEE Proceedings: Vision, Image and Signal Processing* 141 (4): 217–222.

[35] Bishop, Christopher M. 1995b. *Neural Networks for Pattern Recognition.* Oxford University Press.

[36] Bishop, Christopher M. 1995c. “Training with noise is equivalent to Tikhonov regularization.” *Neural Computation* 7 (1): 108–116.

[37] Bishop, Christopher M. 2006. *Pattern Recognition and Machine Learning.* Springer.

[38] Bommasani, Rishi, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, et al. 2021. *On the Opportunities and Risks of Foundation Models.* Technical report. arXiv:2108.07258.

[39] Bottou, L. 2010. “Large-scale machine learning with stochastic gradient descent.” In *Proceedings COMPSTAT 2010*, 177–186. Springer.

[40] Bourlard, H., and Y. Kamp. 1988. “Auto-association by multilayer perceptrons and singular value decomposition.” *Biological Cybernetics* 59:291–294.

[41] Breiman, L. 1996. “Bagging predictors.” *Machine Learning* 26:123–140.

[42] Brinker, T. J., A. Hekler, A. H. Enk, C. Berking, S Haferkamp, A. Hauschild, M. Weichenthal, et al. 2019. “Deep neural networks are superior to dermatologists in melanoma image classification.” *European Journal of Cancer* 119:11–17.

[43] Brock, Andrew, Jeff Donahue, and Karen Simonyan. 2018. “Large-Scale GAN Training for High Fidelity Natural Image Synthesis.” In *Proceedings of the International Conference Learning Representations (ICLR).* ArXiv:1809.11096.

[44] Bronstein, Michael M., Joan Bruna, Taco Cohen, and Petar Velickovic. 2021. *Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges.* Technical report. arXiv:2104.13478.

[45] Bronstein, Michael M., Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. 2017. “Geometric Deep Learning: Going Beyond Euclidean Data.” In *IEEE Signal Processing Magazine*, vol. 34. 4. IEEE, July.

[46] Broomhead, D. S., and D. Lowe. 1988. “Multivariable functional interpolation and adaptive networks.” *Complex Systems* 2:321–355.

[47] Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, et al. 2020. *Language Models are Few-Shot Learners.* Technical report. arXiv:2005.14165.

[48] Bubeck, Sébastien, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, et al. 2023. *Sparks of Artificial General Intelligence: Early experiments with GPT-4.* Technical report. arXiv:2303.12712.

[49] Cardoso, J-F. 1998. “Blind signal separation: statistical principles.” *Proceedings of the IEEE* 9 (10): 2009–2025.

[50] Caruana, R. 1997. “Multitask learning.” *Machine Learning* 28:41–75.

[51] Casella, G., and R. L. Berger. 2002. *Statistical Inference.* Second. Duxbury.

[52] Chan, K., T. Lee, and T. J. Sejnowski. 2003. “Variational Bayesian learning of ICA withmissing data.” *Neural Computation* 15 (8): 1991–2011.

[53] Chen, A. M., H. Lu, and R. Hecht-Nielsen. 1993. “On the geometry of feedforward neural network error surfaces.” *Neural Computation* 5 (6): 910–927.

[54] Chen, Mark, Alec Radford, Rewon Child, Jeffrey Wu, Heewoo Jun, David Luan, and Ilya Sutskever. 2020. “Generative Pretraining From Pixels.” *Proceedings of Machine Learning Research* 119:1691–1703.

[55] Chen, R. T. Q., Rubanova Y, J. Bettencourt, and D. Duvenaud. 2018. *Neural Ordinary Differential Equations.* Technical report. arXiv:1806.07366.

[56] Chen, Ting, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. *A Simple Framework for Contrastive Learning of Visual Representations.* Technical report. arXiv:2002.05709.

[57] Cho, Kyunghyun, Bart van Merrienboer, Çaglar Gülçehre, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translations.* Technical report. arXiv:1406.1078.

[58] Choudrey, R. A., and S. J. Roberts. 2003. “Variational mixture of Bayesian independent component analyzers.” *Neural Computation* 15 (1):213–252.

[59] Christiano, Paul, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. *Deep reinforcement learning from human preferences.* Technical report. arXiv:1706.03741.

[60] Collobert, R. 2004. “Large Scale Machine Learning.” PhD diss., Université Paris VI.

[61] Comon, P., C. Jutten, and J. Herault. 1991. “Blind source separation, 2: problems statement.” *Signal Processing* 24 (1): 11–20.

[62] Cover, T., and P. Hart. 1967. “Nearest neighbor pattern classification.” *IEEE Transactions on Information Theory* IT-11:21–27.

[63] Cover, T. M., and J. A. Thomas. 1991. *Elements of Information Theory.* Wiley.

[64] Cox, R. T. 1946. “Probability, frequency and reasonable expectation.” *American Journal of Physics* 14 (1): 1–13.

[65] Cybenko, G. 1989. “Approximation by superpositions of a sigmoidal function.” *Mathematics of Control, Signals and Systems* 2:304–314.

[66] Dawid, A. P. 1979. “Conditional Independence in Statistical Theory (with discussion).” *Journal of the Royal Statistical Society, Series B* 4:1–31.

[67] Dawid, A. P. 1980. “Conditional Independence for Statistical Operations.” *Annals of Statistics* 8:598–617.

[68] Deisenroth, M. P., A. A. Faisal, and C. S. Ong. 2020. *Mathematics for Machine Learning.* Cambridge University Press.

[69] Dempster, A. P., N. M. Laird, and D. B. Rubin. 1977. “Maximum likelihood from incomplete data via the EM algorithm.” *Journal of the Royal Statistical Society, B* 39 (1): 1–38.

[70] Deng, Jia, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. 2009. “ImageNet: A largescale hierarchical image database.” In *IEEE Conference on Computer Vision and Pattern Recognition*.

[71] Devlin, Jacob, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. *BERT: Pretraining of Deep Bidirectional Transformers for Language Understanding.* Technical report. arXiv:1810.04805.

[72] Dhariwal, Prafulla, and Alex Nichol. 2021. *Diffusion Models Beat GANs on Image Synthesis.* Technical report. arXiv:2105.05233.

[73] Dinh, Laurent, David Krueger, and Yoshua Bengio. 2014. *NICE: Nonlinear Independent Components Estimation.* Technical report. arXiv:1410.8516.

[74] Dinh, Laurent, Jascha Sohl-Dickstein, and Samy Bengio. 2016. *Density estimation using Real NVP.* Technical report. arXiv:1605.08803.

[75] Dodge, Samuel, and Lina Karam. 2017. *A Study and Comparison of Human and Deep Learning Recognition Performance Under Visual Distortions.* Technical report. arXiv:1705.02498.

[76] Doersch, C. 2016. *Tutorial on Variational Autoencoders.* Technical report. arXiv:1606.05908.

[77] Dosovitskiy, Alexey, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, et al. 2020. *An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale.* Technical report. arXiv:2010.11929.

[78] Duane, S., A. D. Kennedy, B. J. Pendleton, and D. Roweth. 1987. "Hybrid Monte Carlo." *Physics Letters B* 195 (2): 216–222.

[79] Duchi, J., E. Hazan, and Y. Singer. 2011. "Adaptive Subgradient Methods for Online Learning and Stochastic Optimization." *Journal of Machine Learning Research* 12:2121–2159.

[80] Duda, R. O., and P. E. Hart. 1973. *Pattern Classification and Scene Analysis.* Wiley.

[81] Dufter, Philipp, Martin Schmitt, and Hinrich Schütze. 2021. *Position Information in Transformers: An Overview.* Technical report. arXiv:2102.11090.

[82] Dumoulin, Vincent, and Francesco Visin. 2016. *A guide to convolution arithmetic for deep learning.* Technical report. arXiv:1603.07285.

[83] Elliott, R. J., L. Aggoun, and J. B. Moore. 1995. *Hidden Markov Models: Estimation and Control.* Springer.

[84] Esser, Patrick, Robin Rombach, and Björn Ommer. 2020. *Taming Transformers for High-Resolution Image Synthesis.* Technical report. arXiv:2012.09841.

[85] Esteva, A., B. Kuprel, R. A. Novoa, J. Ko, S. M. Swetter, H. M. Blau, and S. Thrun. 2017. "Dermatologist-level classification of skin cancer with deep neural networks." *Nature* 542:115–118.

[86] Everitt, B. S. 1984. *An Introduction to Latent Variable Models.* Chapman / Hall.

[87] Eykholt, Kevin, Ivan Evtimov, Earlence Fernandes, Bo Li, Amir Rahmati, Chaowei Xiao, Atul Prakash, Tadayoshi Kohno, and Dawn Song. 2018. "Robust Physical-World Attacks on Deep Learning Visual Classification." In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).*

[88] Fawcett, T. 2006. "An introduction to ROC analysis." *Pattern Recognition Letters* 27:861–874.

[89] Feller, W. 1966. *An Introduction to Probability Theory and its Applications.* Second. Vol. 2. Wiley.

[90] Fletcher, R. 1987. *Practical Methods of Optimization.* Second. Wiley.

[91] Forsyth, D. A., and J. Ponce. 2003. *Computer Vision: A Modern Approach.* Prentice Hall.

[92] Freund, Y., and R. E. Schapire. 1996. "Experiments with a new boosting algorithm." In *Thirteenth International Conference on Machine Learning*, edited by L. Saitta, 148–156. Morgan Kaufmann.

[93] Fukushima, K. 1980. "Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position." *Biological Cybernetics* 36:193–202.

[94] Funahashi, K. 1989. "On the approximate realization of continuous mappings by neural networks." *Neural Networks* 2 (3): 183–192.

[95] Fung, R., and K. C. Chang. 1990. "Weighting and Integrating Evidence for Stochastic Simulation in Bayesian Networks." In *Uncertainty in Artificial Intelligence*, edited by P. P. Bonisson,M. Henrion, L. N. Kanal, and J. F. Lemmer, 5:208–219. Elsevier.

[96] Gatys, Leon A., Alexander S. Ecker, and Matthias Bethge. 2015. *A Neural Algorithm of Artistic Style.* Technical report. arXiv:1508.06576.

[97] Geman, S., and D. Geman. 1984. "Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images." *IEEE PAMI* 6 (1): 721–741.

[98] Gemmeke, Jort F, Daniel P. W. Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R. Channing Moore, Manoj Plakal, and Marvin Ritter. 2017. "Audio Set: An ontology and human-labeled dataset for audio events." In *Proc. IEEE ICASSP 2017.* New Orleans, LA.

[99] Germain, Mathieu, Karol Gregor, Iain Murray, and Hugo Larochelle. 2015. *MADE: Masked Autoencoder for Distribution Estimation.* Technical report. arXiv:1502.03509.

[100] Gilks, W. R. 1992. “Derivative-free adaptive rejection sampling for Gibbs sampling.” In *Bayesian Statistics*, edited by J. Bernardo, J. Berger, A. P. Dawid, and A. F. M. Smith, vol. 4. Oxford University Press.

[101] Gilks, W. R., N. G. Best, and K. K. C. Tan. 1995. “Adaptive rejection Metropolis sampling.” *Applied Statistics* 44:455–472.

[102] Gilks, W. R., S. Richardson, and D. J. Spiegelhalter. 1996. *Markov Chain Monte Carlo in Practice.* Chapman / Hall.

[103] Gilks, W. R., and P. Wild. 1992. “Adaptive rejection sampling for Gibbs sampling.” *Applied Statistics* 41:337–348.

[104] Gilmer, Justin, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. 2017. *Neural Message Passing for Quantum Chemistry.* Technical report. arXiv:1704.01212.

[105] Girshick, Ross B. 2015. *Fast R-CNN.* Technical report. arXiv:1504.08083.

[106] Golub, G. H., and C. F. Van Loan. 1996. *Matrix Computations.* Third. John Hopkins University Press.

[107] Gong, Yuan, Yu-An Chung, and James R. Glass. 2021. *AST: Audio Spectrogram Transformer.* Technical report. arXiv:2104.01778.

[108] Goodfellow, Ian, Yoshua Bengio, and Aaron Courville. 2016. *Deep Learning.* MIT Press.

[109] Goodfellow, Ian J., Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. *Generative Adversarial Networks.* Technical report. arXiv:1406.2661.

[110] Goodfellow, Ian J., Jonathon Shlens, and Christian Szegedy. 2014. *Explaining and Harnessing Adversarial Examples.* Technical report. arXiv:1412.6572.

[111] Grathwohl, Will, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. 2018. *FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models.* Technical report. arXiv:1810.01367.

[112] Griewank, A., and A Walther. 2008. *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation.* Second. SIAM.

[113] Grosse, R. 2018. *Automatic Differentiation.* CSC321 Lecture 10. University of Toronto.

[114] Gulrajani, I., F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. 2017. *Improved training of Wasserstein GANs.* Technical report. arXiv:1704.00028.

[115] Gutmann, Michael, and Aapo Hyvärinen. 2010. “Noise-contrastive estimation: A new estimation principle for unnormalized statistical models.” *Journal of Machine Learning Research* 9:297–304.

[116] Hamilton, W. L. 2020. *Graph Representation Learning.* Morgan / Claypool.

[117] Hartley, R., and A. Zisserman. 2004. *Multiple View Geometry in Computer Vision.* Second. Cambridge University Press.

[118] Hassibi, B., and D. G. Stork. 1993. “Second order derivatives for network pruning: optimal brain surgeon.” In *Proceedings International Conference on Neural Information Processing Systems (NeurIPS)*, edited by S. J. Hanson, J. D. Cowan, and C. L. Giles, 5:164–171. Morgan Kaufmann.

[119] Hastie, T., R. Tibshirani, and J. Friedman. 2009. *The Elements of Statistical Learning.* Second. Springer.

[120] Hastings, W. K. 1970. “Monte Carlo sampling methods using Markov chains and their applications.” *Biometrika* 57:97–109.

[121] He, Kaiming, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross B. Girshick. 2021. *Masked Autoencoders Are Scalable Vision Learners.* Technical report. arXiv:2111.06377.

[122] He, Kaiming, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2019. *Momentum Contrast for Unsupervised Visual Representation Learning.* Technical report. arXiv:1911.05722.

[123] He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015a. *Deep Residual Learning for Image Recognition.* Technical report. arXiv:1512.03385.

[124] He, Kaiming, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2015b. *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.* Technical report. arXiv:1502.01852.

[125] Henrion, M. 1988. “Propagation of Uncertainty by Logic Sampling in Bayes’ Networks.” In *Uncertainty in Artificial Intelligence*, edited by J. F. Lemmer and L. N. Kanal, 2:149–164. North Holland.

[126] Higgins, I., L. Matthey, A. Pal, C. Burgess, X. Glorot, M. Botvinik, S. Mohamed, and A. Lerchner. 2017. "β-VAE: learning basic visual concepts with a constrained variational framework." In *Proceedings of the International Conference Learning Representations (ICLR)*.

[127] Hinton, G. E. 2012. *Neural Networks for Machine Learning. Lecture 6.5. Coursera Lectures.*

[128] Hinton, G. E., M. Welling, Y. W. Teh, and S Osindero. 2001. "A new view of ICA." In *Proceedings of the International Conference on Independent Component Analysis and Blind Signal Separation*, vol. 3.

[129] Hinton, Geoffrey, Oriol Vinyals, and Jeff Dean. 2015. *Distilling the Knowledge in a Neural Network.* Technical report. arXiv:1503.02531.

[130] Hinton, Geoffrey E. 2002. "Training products of experts by minimizing contrastive divergence." *Neural Computation* 14:1771–1800.

[131] Ho, Jonathan, Ajay Jain, and Pieter Abbeel. 2020. *Denoising Diffusion Probabilistic Models.* Technical report. arXiv:2006.11239.

[132] Ho, Jonathan, Chitwan Saharia, William Chan, David J. Fleet, Mohammad Norouzi, and Tim Salimans. 2021. *Cascaded Diffusion Models for High Fidelity Image Generation.* Technical report. arXiv:2106.15282.

[133] Hochreiter, S., and J. Schmidhuber. 1997. "Long short-term Memory." *Neural Computation* 9 (8): 1735–1780.

[134] Hojen-Sorensen, P. A., O. Winther, and L. K. Hansen. 2002. "Mean field approaches to independent component analysis." *Neural Computation* 14 (4): 889–918.

[135] Holtzman, Ari, Jan Buys, Maxwell Forbes, and Yejin Choi. 2019. *The Curious Case of Neural Text Degeneration.* Technical report. arXiv:1904.09751.

[136] Hornik, K., M. Stinchcombe, and H. White. 1989. "Multilayer feedforward networks are universal approximators." *Neural Networks* 2 (5):359–366.

[137] Hospedales, Timothy, Antreas Antoniou, Paul Micaelli, and Amos Storkey. 2021. "Meta-learning in neural networks: A survey." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44 (9): 5149–5169.

[138] Hotelling, H. 1933. "Analysis of a complex of statistical variables into principal components." *Journal of Educational Psychology* 24:417–441.

[139] Hotelling, H. 1936. "Relations between two sets of variables." *Biometrika* 28:321–377.

[140] Hu, Anthony, Lloyd Russell, Hudson Yeo, Zak Murez, George Fedoseev, Alex Kendall, Jamie Shotton, and Gianluca Corrado. 2023. *GAIA-1: A Generative World Model for Autonomous Driving.* Technical report. arXiv:2309.17080.

[141] Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. *LoRA: Low-Rank Adaptation of Large Language Models.* Technical report. arXiv:2106.09685.

[142] Hubel, D. H., and T. N. Wiesel. 1959. "Receptive fields of single neurons in the cat's striate cortex." *Journal of Physiology* 148:574–591.

[143] Hyvärinen, A. 2005. "Estimation of Non-Normalized Statistical Models by Score Matching." *Journal of Machine Learning Research* 6:695–709.

[144] Hyvärinen, A., and E. Oja. 1997. "A fast fixed-point algorithm for independent component analysis." *Neural Computation* 9 (7): 1483–1492.

[145] Hyvärinen, Aapo, Jarmo Hurri, and Patrick O. Hoyer. 2009. *Natural Image Statistics: A Probabilistic Approach to Early Computational Vision.* Springer.

[146] Ioffe, S., and C. Szegedy. 2015. "Batch normalization." In *Proceedings of the International Conference on Machine Learning (ICML)*, 448–456.

[147] Jacobs, R. A., M. I. Jordan, S. J. Nowlan, and G. E. Hinton. 1991. "Adaptive mixtures of local experts." *Neural Computation* 3 (1): 79–87.

[148] Jebara, T. 2004. *Machine Learning: Discriminative and Generative.* Kluwer.

[149] Jensen, C., A. Kong, and U. Kjaerulff. 1995. "Blocking Gibbs sampling in very large probabilistic expert systems." *International Journal of Human Computer Studies. Special Issue on Real-World Applications of Uncertain Reasoning.* 42:647–666.

[150] Jolliffe, I. T. 2002. *Principal Component Analysis.* Second. Springer.

[151] Jumper, John, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, and Olaf Ronneberger. 2021. "Highly accurate protein structure prediction with AlphaFold." *Nature* 596:583–589.

[152] Jutten, C., and J. Herault. 1991. "Blind separation of sources, 1: An adaptive algorithm based on neuromimetic architecture." *Signal Processing* 24 (1): 1–10.

[153] Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. *Scaling Laws for Neural Language Models.* Technical report. arXiv:2001.08361.

[154] Karras, Tero, Timo Aila, Samuli Laine, and Jaakko Lehtinen. 2017. *Progressive Growing of GANs for Improved Quality, Stability, and Variation.* Technical report. arXiv:1710.10196.

[155] Karush, W. 1939. "Minima of functions of several variables with inequalities as side constraints." Master's thesis, Department of Mathematics, University of Chicago.

[156] Khosla, Prannay, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. 2020. *Supervised Contrastive Learning.* Technical report. arXiv:2004.11362.

[157] Kingma, D., and J. Ba. 2014. *Adam: A method for stochastic optimization.* Technical report. arXiv:1412.6980.

[158] Kingma, D. P., and M. Welling. 2013. "Auto-encoding variational Bayes." In *Proceedings of the International Conference on Machine Learning (ICML)*. ArXiv:1312.6114.

[159] Kingma, Diederik P., and Max Welling. 2019. *An Introduction to Variational Autoencoders.* Technical report. arXiv:1906.02691.

[160] Kingma, Durk P, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. 2016. "Improved variational inference with inverse autoregressive flow." *Advances in Neural Information Processing Systems* 29.

[161] Kipf, Thomas N., and Max Welling. 2016. *Semi-Supervised Classification with Graph Convolutional Networks.* Technical report. arXiv:1609.02907.

[162] Kloeden, Peter E, and Eckhard Platen. 2013. *Numerical solution of stochastic differential equations.* Vol. 23. Stochastic Modelling and Applied Probability. Springer.

[163] Kobyzev, I., S. J. D. Prince, and M. A. Brubaker. 2019. "Normalizing flows: an introduction and review of current methods." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43 (11): 3964–3979.

[164] Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. 2012. "Imagenet classification with deep convolutional neural networks." In *Advances in Neural Information Processing Systems*, vol. 25.

[165] Kuhn, H. W., and A. W. Tucker. 1951. "Nonlinear programming." In *Proceedings of the 2nd Berkeley Symposium on Mathematical Statistics and Probabilities*, 481–492. University of California Press.

[166] Kullback, S., and R. A. Leibler. 1951. "On information and sufficiency." *Annals of Mathematical Statistics* 22 (1): 79–86.

[167] Kurková, V., and P. C. Kainen. 1994. "Functionally Equivalent Feedforward Neural Networks." *Neural Computation* 6 (3): 543–558.

[168] Lasserre, J., Christopher M. Bishop, and T. Minka. 2006. "Principled hybrids of generative and discriminative models." In *Proceedings 2006 IEEE Conference on Computer Vision and Pattern Recognition, New York.*

[169] Lauritzen, S. L. 1996. *Graphical Models.* Oxford University Press.

[170] Lawley, D. N. 1953. "A Modified Method of Estimation in Factor Analysis and Some Large Sample Results." In *Uppsala Symposium on Psychological Factor Analysis*, 35–42. Number 3 in Nordisk Psykologi Monograph Series. Uppsala: Almqvist / Wiksell.

[171] Lazarsfeld, P. F., and N. W. Henry. 1968. *Latent Structure Analysis.* Houghton Mifflin.

[172] LeCun, Y., B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel. 1989. "Backpropagation Applied to Handwritten ZIP Code Recognition." *Neural Computation* 1 (4): 541–551.

[173] LeCun, Y., L. Bottou, Y. Bengio, and P. Haffner. 1998. "Gradient-Based Learning Applied to Document Recognition." *Proceedings of the IEEE* 86:2278–2324.

[174] LeCun, Y., J. S. Denker, and S. A. Solla. 1990. “Optimal Brain Damage.” In *Proceedings International Conference on Neural Information Processing Systems (NeurIPS)*, edited by D. S. Touretzky, 2:598–605. Morgan Kaufmann.

[175] LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. 2015. “Deep Learning.” *Nature* 512:436–444.

[176] LeCun, Yann, Sumit Chopra, Raia Hadsell, Marc’ Aurelio Ranzato, and Fu-Jie Huang. 2006. “A Tutorial on Energy-Based Learning.” In *Predicting Structured Data*, edited by G. Bakir, T. Hofman, B. Schölkopf, A. Smola, and B. Taskar. MIT Press.

[177] Leen, T. K. 1995. “From data distributions to regularization in invariant learning.” *Neural Computation* 7:974–981.

[178] Leshno, M., V. Y. Lin, A. Pinkus, and S. Schocken. 1993. “Multilayer feedforward networks with a polynomial activation function can approximate any function.” *Neural Networks* 6:861–867.

[179] Li, Hao, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. 2017. *Visualizing the Loss Landscape of Neural Nets.* Technical report. arXiv:1712.09913.

[180] Li, Junnan, Dongxu Li, Caiming Xiong, and Steven Hoi. 2022. *BLIP: Bootstrapping Language-Image Pretraining for Unified Vision-Language Understanding and Generation.* Technical report. arXiv:2201.12086.

[181] Lin, Min, Qiang Chen, and Shuicheng Yan. 2013. *Network in Network.* Technical report. arXiv:1312.4400.

[182] Lin, Tianyang, Yuxin Wang, Xiangyang Liu, and Xipeng Qiu. 2021. *A Survey of Transformers.* Technical report. arXiv:2106.04554.

[183] Lipman, Yaron, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. 2022. *Flow Matching for Generative Modeling.* Technical report arXiv:2210.02747.

[184] Liu, Pengfei, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. 2021. *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing.* Technical report. arXiv:2107.13586.

[185] Lloyd, S. P. 1982. “Least squares quantization in PCM.” *IEEE Transactions on Information Theory* 28 (2): 129–137.

[186] Long, Jonathan, Evan Shelhamer, and Trevor Darrell. 2014. *Fully Convolutional Networks for Semantic Segmentation.* Technical report. arXiv:1411.4038.

[187] Luo, Calvin. 2022. *Understanding Diffusion Models: A Unified Perspective.* Technical report. arXiv:2208.11970.

[188] Lütkepohl, H. 1996. *Handbook of Matrices.* Wiley.

[189] MacKay, D. J. C. 1992. “A Practical Bayesian Framework for Back-propagation Networks.” *Neural Computation* 4 (3): 448–472.

[190] MacKay, D. J. C. 2003. *Information Theory, Inference and Learning Algorithms.* Cambridge University Press.

[191] MacQueen, J. 1967. “Some methods for classification and analysis of multivariate observations.” In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, edited by L. M. LeCam and J. Neyman, 1:281–297. University of California Press.

[192] Magnus, J. R., and H. Neudecker. 1999. *Matrix Differential Calculus with Applications in Statistics and Econometrics.* Wiley.

[193] Mallat, S. 1999. *A Wavelet Tour of Signal Processing.* Second. Academic Press.

[194] Mao, X., Q. Li, H. Xie, R. Lau, Z. Wang, and S. Smolley. 2016. *Least Squares Generative Adversarial Networks.* Technical report. arXiv:1611.04076.

[195] Mardia, K. V., and P. E. Jupp. 2000. *Directional Statistics.* Wiley.

[196] Martens, James, Ilya Sutskever, and Kevin Swersky. 2012. “Estimating the Hessian by Back-propagating Curvature.” In *Proceedings of the International Conference on Machine Learning (ICML)*. ArXiv:1206.6464.

[197] McCullagh, P., and J. A. Nelder. 1989. *Generalized Linear Models.* Second. Chapman / Hall.

[198] McCulloch, W. S., and W. Pitts. 1943. “A Logical Calculus of the Ideas Immanent in Nervous Activity.” Reprinted in Anderson and Rosenfeld (1988), *Bulletin of Mathematical Biophysics* 5:115–133.

[199] McLachlan, G. J., and T. Krishnan. 1997. *The EM Algorithm and its Extensions.* Wiley.

[200] McLachlan, G. J., and D. Peel. 2000. *Finite Mixture Models.* Wiley.

[201] Meng, X. L., and D. B. Rubin. 1993. “Maximum likelihood estimation via the ECM algorithm: a general framework.” *Biometrika* 80:267–278.

[202] Mescheder, L., A. Geiger, and S. Nowozin. 2018. *Which Training Methods for GANs do actually Converge?* Technical report. arXiv:1801.04406.

[203] Metropolis, N., A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller. 1953. “Equation of State Calculations by Fast Computing Machines.” *Journal of Chemical Physics* 21 (6): 1087–1092.

[204] Metropolis, N., and S. Ulam. 1949. “The Monte Carlo method.” *Journal of the American Statistical Association* 44 (247): 335–341.

[205] Mikolov, Tomas, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. *Efficient Estimation of Word Representations in Vector Space.* Technical report. arXiv:1301.3781.

[206] Minsky, M. L., and S. A. Papert. 1969. *Perceptrons.* Expanded edition 1990. MIT Press.

[207] Mirza, M., and S. Osindero. 2014. *Conditional Generative Adversarial Nets.* Technical report. arXiv:1411.1784.

[208] Miskin, J. W., and D. J. C. MacKay. 2001. “Ensemble learning for blind source separation.” In *Independent Component Analysis: Principles and Practice*, edited by S. J. Roberts and R. M. Everson. Cambridge University Press.

[209] Møller, M. 1993. “Efficient Training of Feed-Forward Neural Networks.” PhD diss., Aarhus University, Denmark.

[210] Montúfar, G. F., R. Pascanu, K. Cho, and Y. Bengio. 2014. “On the number of linear regions of deep neural networks.” In *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*. ArXiv:1402.1869.

[211] Mordvintsev, Alexander, Christopher Olah, and Mike Tyka. 2015. *Inceptionism: Going Deeper into Neural Networks.* Google AI blog.

[212] Murphy, Kevin P. 2022. *Probabilistic Machine Learning: An introduction.* MIT Press. probml. ai.

[213] Murphy, Kevin P. 2023. *Probabilistic Machine Learning: Advanced Topics.* MIT Press.

[214] Nakkiran, Preetum, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. 2019. *Deep Double Descent: Where Bigger Models and More Data Hurt.* Technical report. arXiv:1912.02292.

[215] Neal, R. M. 1993. *Probabilistic inference using Markov chain Monte Carlo methods.* Technical report CRG-TR-93-1. Department of Computer Science, University of Toronto, Canada.

[216] Neal, R. M. 1999. “Suppressing random walks in Markov chain Monte Carlo using ordered over-relaxation.” In *Learning in Graphical Models*, edited by Michael I. Jordan, 205–228. MIT Press.

[217] Neal, R. M., and G. E. Hinton. 1999. “A new view of the EM algorithm that justifies incremental and other variants.” In *Learning in Graphical Models*, edited by M. I. Jordan, 355–368. MIT Press.

[218] Nelder, J. A., and R. W. M. Wedderburn. 1972. “Generalized linear models.” *Journal of the Royal Statistical Society, A* 135:370–384.

[219] Nesterov, Y. 2004. *Introductory Lectures on Convex Optimization: A Basic Course.* Kluwer.

[220] Nichol, Alex, and Prafulla Dhariwal. 2021. *Improved Denoising Diffusion Probabilistic Models.* Technical report. arXiv:2102.09672.

[221] Nichol, Alex, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob Mc-Grew, Ilya Sutskever, and Mark Chen. 2021. *GLIDE: Towards Photorealistic Image Generation and Editing with Text-Guided Diffusion Models.* Technical report. arXiv:2112.10741.

[222] Nocedal, J., and S. J. Wright. 1999. *Numerical Optimization.* Springer.

[223] Noh, Hyeonwoo, Seunghoon Hong, and Bohyung Han. 2015. *Learning Deconvolution Network for Semantic Segmentation.* Technical report. arXiv:1505.04366.

[224] Nowlan, S. J., and G. E. Hinton. 1992. “Simplifying neural networks by soft weight sharing.” *Neural Computation* 4 (4): 473–493.

[225] Ogden, R. T. 1997. *Essential Wavelets for Statistical Applications and Data Analysis.* Birkhäuser.

[226] Oord, Aaron van den, Nal Kalchbrenner, and Koray Kavukcuoglu. 2016. *Pixel Recurrent Neural Networks.* Technical report. arXiv:1601.06759.

[227] Oord, Aaron van den, Nal Kalchbrenner, Oriol Vinyals, Lasse Espeholt, Alex Graves, and Koray Kavukcuoglu. 2016. *Conditional Image Generation with PixelCNN Decoders.* Technical report. arXiv:1606.05328.

[228] Oord, Aaron van den, Yazhe Li, and Oriol Vinyals. 2018. *Representation Learning with Contrastive Predictive Coding.* Technical report. arXiv:1807.03748.

[229] Oord, Aaron van den, Oriol Vinyals, and Koray Kavukcuoglu. 2017. *Neural Discrete Representation Learning.* Technical report. arXiv:1711.00937.

[230] OpenAI. 2023. *GPT-4 Technical Report.* Technical report. arXiv:2303.08774.

[231] Opper, M., and O. Winther. 2000. "Gaussian processes and SVM: mean field theory and leave-one-out." In *Advances in Large Margin Classifiers*, edited by A. J. Smola, P. L. Bartlett, B. Schölkopf, and D. Shuurmans, 311–326. MIT Press.

[232] Papamakarios, G., T. Pavlakou, and Iain Murray. 2017. "Masked Autoregressive Flow for Density Estimation." In *Proceedings of the International Conference on Neural Information Processing Systems (NeurIPS)*, vol. 30.

[233] Papamakarios, George, Eric Nalisnick, Danilo Jimenez Rezende, Shakir Mohamed, and Balaji Lakshminarayanan. 2019. *Normalizing Flows for Probabilistic Modeling and Inference.* Technical report. arXiv:1912.02762.

[234] Parisi, Giorgio. 1981. "Correlation functions and computer simulations." *Nuclear Physics B* 180:378–384.

[235] Pearl, J. 1988. *Probabilistic Reasoning in Intelligent Systems.* Morgan Kaufmann.

[236] Pearlmutter, B. A. 1994. "Fast exact multiplication by the Hessian." *Neural Computation* 6 (1): 147–160.

[237] Pearlmutter, B. A., and L. C. Parra. 1997. "Maximum likelihood source separation: a context-sensitive generalization of ICA." In *Advances in Neural Information Processing Systems*, edited by M. C. Mozer, M. I. Jordan, and T. Petsche, 9:613–619. MIT Press.

[238] Pearson, Karl. 1901. "On lines and planes of closest fit to systems of points in space." *The London, Edinburgh and Dublin Philosophical Magazine and Journal of Science, Sixth Series* 2:559–572.

[239] Phuong, Mary, and Marcus Hutter. 2022. *Formal Algorithms for Transformers.* Technical report. arXiv:2207.09238.

[240] Prince, Simon J.D. 2020. *Variational autoencoders.*

[241] Prince, Simon J.D. 2023. *Understanding Deep Learning.* MIT Press.

[242] Radford, A., L. Metz, and S. Chintala. 2015. *Unsupervised representation learning with deep convolutional generative adversarial networks.* Technical report. arXiv:1511.06434.

[243] Radford, Alec, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, et al. 2021. *Learning Transferable Visual Models From Natural Language Supervision.* Technical report. arXiv:2103.00020.

[244] Radford, Alec, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. *Language Models are Unsupervised Multitask Learners.* Technical report. OpenAI.

[245] Rakhimov, Ruslan, Denis Volkhonskiy, Alexey Artemov, Denis Zorin, and Evgeny Burnaev. 2020. *Latent Video Transformer.* Technical report. arXiv:2006.10704.

[246] Ramachandran, P., B. Zoph, and Q. V. Le. 2017. *Searching for Activation Functions.* Technical report. arXiv:1710.05941v2.

[247] Rao, C. R., and S. K. Mitra. 1971. *Generalized Inverse of Matrices and Its Applications.* Wiley.

[248] Redmon, Joseph, Santosh Kumar Divvala, Ross B. Girshick, and Ali Farhadi. 2015. *You Only Look Once: Unified, Real-Time Object Detection.* Technical report. arxiv:1506.02640.

[249] Ren, Shaoqing, Kaiming He, Ross B. Girshick, and Jian Sun. 2015. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks.* Technical report. arxiv:1506.01497.

[250] Rezende, Danilo J, Shakir Mohamed, and Daan Wierstra. 2014. "Stochastic backpropagation and approximate inference in deep generative models." In *Proceedings of the 31st International Conference on Machine Learning (ICML-14)*, 1278–1286.

[251] Ricotti, L. P, S. Ragazzini, and G. Martinelli. 1988. “Learning of word stress in a sub-optimal second order backpropagation neural network.” In *Proceedings of the IEEE International Conference on Neural Networks*, 1:355–361. IEEE.

[252] Robert, C. P., and G. Casella. 1999. *Monte Carlo Statistical Methods.* Springer.

[253] Rombach, Robin, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. 2021. *High-Resolution Image Synthesis with Latent Diffusion Models.* Technical report. arXiv:2112.10752.

[254] Ronneberger, Olaf, Philipp Fischer, and Thomas Brox. 2015. “U-Net: Convolutional Networks for Biomedical Image Segmentation.” In *Medical Image Computing and Computer-Assisted Intervention – MICCAI*, edited by N. Navab, J. Hornegger, W. Wells, and A. Frangi. Springer.

[255] Rosenblatt, F. 1962. *Principles of Neurodynamics: Perceptrons and the Theory of Brain Mechanisms.* Spartan.

[256] Roweis, S. 1998. “EM algorithms for PCA and SPCA.” In *Advances in Neural Information Processing Systems*, edited by M. I. Jordan, M. J. Kearns, and S. A. Solla, 10:626–632. MIT Press.

[257] Roweis, S., and Z. Ghahramani. 1999. “A unifying review of linear Gaussian models.” *Neural Computation* 11 (2): 305–345.

[258] Rubin, D. B., and D. T. Thayer. 1982. “EM algorithms for ML factor analysis.” *Psychometrika* 47 (1): 69–76.

[259] Rumelhart, D. E., G. E. Hinton, and R. J. Williams. 1986. “Learning internal representations by error propagation.” In *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, edited by D. E. Rumelhart, J. L. McClelland, and the PDP Research Group, vol. 1: Foundations, 318–362. Reprinted in Anderson and Rosenfeld (1988). MIT Press.

[260] Ruthotto, L., and E. Haber. 2021. *An introduction to deep generative modeling.* Technical report. arXiv:2103.05180.

[261] Sagan, H. 1969. *Introduction to the Calculus of Variations.* Dover.

[262] Saharia, Chitwan, William Chan, Huiwen Chang, Chris A. Lee, Jonathan Ho, Tim Salimans, David J. Fleet, and Mohammad Norouzi. 2021. *Palette: Image-to-Image Diffusion Models.* Technical report. arXiv:2111.05826.

[263] Saharia, Chitwan, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Seyed Ghasemipour, et al. 2022. *Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding.* Technical report. arXiv:2205.11487.

[264] Saharia, Chitwan, Jonathan Ho, William Chan, Tim Salimans, David J. Fleet, and Mohammad Norouzi. 2021. *Image Super-Resolution via Iterative Refinement.* Technical report. arXiv:2104.07636.

[265] Santurkar, S., D. Tsipras, A. Ilyas, and A. Madry. 2018. *How does batch normalization help optimization?* Technical report. arXiv:1805.11604.

[266] Satorras, Victor Garcia, Emiel Hoogeboom, and Max Welling. 2021. *E(n) Equivariant Graph Neural Networks.* Technical report. arXiv:2102.09844.

[267] Schölkopf, B., and A. J. Smola. 2002. *Learning with Kernels.* MIT Press.

[268] Schuhmann, Christoph, Richard Vencu, Romain Beaumont, Robert Kaczmarczyk, Clayton Mullis, Aarush Katta, Theo Coombes, Jenia Jitsev, and Aran Komatsuzaki. 2021. *LAION-400M: Open Dataset of CLIP-Filtered 400 Million Image-Text Pairs.* Technical report. arXiv:2111.02114.

[269] Schuster, Mike, and Kaisuke Nakajima. 2012. “Japanese and Korean voice search.” In *2012 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 5149–5152.

[270] Selvaraju, Ramprasaath R., Abhishek Das, Ramakrishna Vedantam, Michael Cogswell, Devi Parikh, and Dhruv Batra. 2016. *Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization.* Technical report. arXiv:1610.02391.

[271] Sennrich, Rico, Barry Haddow, and Alexandra Birch. 2015. *Neural Machine Translation of Rare Words with Subword Units.* Technical report. arXiv:1508.07909.

[272] Sermanet, Pierre, David Eigen, Xiang Zhang, Michael Mathieu, Rob Fergus, and Yann LeCun. 2013. *OverFeat: Integrated Recognition, Localization and Detection using Convolutional Networks.* Technical report. arXiv:1312.6229.

[273] Shachter, R. D., and M. Peot. 1990. “Simulation Approaches to General Probabilistic Inference on Belief Networks.” In *Uncertainty in Artificial Intelligence*, edited by P. P. Bonissone, M. Henrion, L. N. Kanal, and J. F. Lemmer, vol. 5. Elsevier.

[274] Shannon, C. E. 1948. “A mathematical theory of communication.” *The Bell System Technical Journal* 27 (3): 379–423 and 623–656.

[275] Shen, Sheng, Zhen Dong, Jiayu Ye, Linjian Ma, Zhewei Yao, Amir Gholami, Michael W. Mahoney, and Kurt Keutzer. 2019. *Q-BERT: Hessian Based Ultra Low Precision Quantization of BERT.* Technical report. arXiv:1909.05840.

[276] Simard, P., B. Victorri, Y. LeCun, and J. Denker. 1992. "Tangent prop – a formalism for specifying selected invariances in an adaptive network." In *Advances in Neural Information Processing Systems*, edited by J. E. Moody, S. J. Hanson, and R. P. Lippmann, 4:895–903. Morgan Kaufmann.

[277] Simard, P. Y., D. Steinkraus, and J. Platt. 2003. "Best practice for convolutional neural networks applied to visual document analysis." In *Proceedings International Conference on Document Analysis and Recognition (ICDAR)*, 958–962. IEEE Computer Society.

[278] Simonyan, Karen, Andrea Vedaldi, and Andrew Zisserman. 2013. "Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps." In *Computer Vision and Pattern Recognition.* ArXiv:1312.6034.

[279] Simonyan, Karen, and Andrew Zisserman. 2014. *Very Deep Convolutional Networks for Large-Scale Image Recognition.* Technical report. arXiv:1409.1556.

[280] Sirovich, L. 1987. "Turbulence and the Dynamics of Coherent Structures." *Quarterly Applied Mathematics* 45 (3): 561–590.

[281] Sohl-Dickstein, Jascha, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. 2015. *Deep Unsupervised Learning using Nonequilibrium Thermodynamics.* Technical report. arXiv:1503.03585.

[282] Sønderby, C., J. Caballero, L. Theis, W. Shi, and F. Huszár. 2016. *Amortised MAP inference for image superresolution.* Technical report. arXiv:1610.04490.

[283] Song, Jiaming, Chenlin Meng, and Stefano Ermon. 2020. *Denoising Diffusion Implicit Models.* Technical report. arXiv:2010.02502.

[284] Song, Yang, and Stefano Ermon. 2019. "Generative Modeling by Estimating Gradients of the Data Distribution." In *Advances in Neural Information Processing Systems*, 11895–11907. ArXiv:1907.05600.

[285] Song, Yang, Sahaj Garg, Jiaxin Shi, and Stefano Ermon. 2019. "Sliced score matching: A scalable approach to density and score estimation." In *Uncertainty in Artificial Intelligence*, 204. ArXiv:1905.07088.

[286] Song, Yang, and Diederik P. Kingma. 2021. *How to Train Your Energy-Based Models.* Technical report. arXiv:2101.03288.

[287] Song, Yang, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. 2020. *Score-Based Generative Modeling through Stochastic Differential Equations.* Technical report. arXiv:2011.13456.

[288] Srivastava, N., G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. 2014. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting." *Journal of Machine Learning Research* 15:1929–1958.

[289] Stone, J. V. 2004. *Independent Component Analysis: A Tutorial Introduction.* MIT Press.

[290] Sutskever, I., J. Martens, G. Dahl, and G. E. Hinton. 2013. "On the importance of initialization and momentum in deep learning." In *Proceedings of the International Conference on Machine Learning (ICML)*.

[291] Sutton, R. 2019. *The Bitter Lesson.* URL: incompleteideas.net/IncIdeas/

[292] Szegedy, Christian, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2013. *Intriguing properties of neural networks*. Technical report. arXiv:1312.6199.

[293] Szeliski, R. 2022. *Computer Vision: Algorithms and Applications*. Second. Springer.

[294] Tarassenko, L. 1995. "Novelty detection for the identification of masses in mammograms." In *Proceedings of the Fourth IEE International Conference on Artificial Neural Networks*, 4:442–447. IEE.

[295] Tay, Yi, Mostafa Dehghani, Dara Bahri, and Donald Metzler. 2020. *Efficient Transformers: A Survey*. Technical report. arXiv:2009.06732.

[296] Tibshirani, R. 1996. "Regression shrinkage and selection via the lasso." *Journal of the Royal Statistical Society, B* 58:267–288.

[297] Tipping, M. E., and Christopher M. Bishop. 1997. *Probabilistic Principal Component Analysis*. Technical report NCRG/97/010. Neural Computing Research Group, Aston University.

[298] Tipping, M. E., and Christopher M. Bishop. 1999. "Probabilistic Principal Component Analysis." *Journal of the Royal Statistical Society, Series B* 21 (3): 611–622.

[299] Vapnik, V. N. 1995. *The nature of statistical learning theory*. Springer.

[300] Vaswani, Ashish, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. *Attention Is All You Need*. Technical report. arXiv:1706.03762.

[301] Veličković, Petar. 2023. *Everything is Connected: Graph Neural Networks*. Technical report. arXiv:2301.08210.

[302] Veličković, Petar, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2017. *Graph Attention Networks*. Technical report. arXiv:1710.10903.

[303] Vidakovic, B. 1999. *Statistical Modelling by Wavelets*. Wiley.

[304] Vig, Jesse, Ali Madani, Lav R. Varshney, Caiming Xiong, Richard Socher, and Nazneen Fatema Rajani. 2020. *BERTology Meets Biology: Interpreting Attention in Protein Language Models*. Technical report. arXiv:2006.15222.

[305] Vincent, P. 2011. "A connection between score matching and denoising autoencoders." *Neural Computation* 23:1661–1674.

[306] Vincent, Pascal, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. "Extracting and Composing Robust Features with Denoising Autoencoders." In *Proceedings of the International Conference on Machine Learning (ICML)*.

[307] Walker, A. M. 1969. "On the asymptotic behaviour of posterior distributions." *Journal of the Royal Statistical Society, B* 31 (1): 80–88.

[308] Wang, Chengyi, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, et al. 2023. *Neural Codec Language Models are Zero-Shot Text to Speech Synthesizers*. Technical report. arXiv:2301.02111.

[309] Weinstein, E. W. 1999. *CRC Concise Encyclopedia of Mathematics*. Chapman / Hall, / CRC.

[310] Welling, Max, and Yee Whye Teh. 2011. "Bayesian Learning via Stochastic Gradient Langevin Dynamics." In *Proceedings of the International Conference on Machine Learning (ICML)*.

[311] Williams, P. M. 1996. "Using neural networks to model conditional multivariate densities." *Neural Computation* 8 (4): 843–854.

[312] Williams, R J. 1992. "Simple statistical gradient-following algorithms for connectionist reinforcement learning." *Machine Learning* 8:229–256.

[313] Winn, J., C. M. Bishop, T. Diethe, J. Guiver, and Y. Zaykov. 2023. *Model-Based Machine Learning*.

[314] Wolpert, D. H. 1996. "The lack of a priori distinctions between learning algorithms." *Neural Computation* 8:1341–1390.

[315] Wu, Zhirong, Yuanjun Xiong, Stella Yu, and Dahua Lin. 2018. *Unsupervised Feature Learning via Non-Parametric Instance-level Discrimination*. Technical report. arXiv:1805.01978.

[316] Wu, Zonghan, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S. Yu. 2019. *A Comprehensive Survey on Graph Neural Networks*. Technical report. arXiv:1901.00596.

[317] Yan, Wilson, Yunzhi Zhang, Pieter Abbeel, and Aravind Srinivas. 2021. *VideoGPT: Video Generation using VQ-VAE and Transformers*. Technical report. arXiv:2104.10157.

[318] Yang, Ruihan, Prakhar Srivastava, and Stephan Mandt. 2022. *Diffusion Probabilistic Modeling for Video Generation*. Technical report. arXiv:2203.09481.

[319] Yilmaz, Fatih Furkan, and Reinhard Heckel. 2022. *Regularization-wise double descent: Why it occurs and how to eliminate it*. Technical report. arXiv:2206.01378.

[320] Yosinski, Jason, Jeff Clune, Anh Mai Nguyen, Thomas J. Fuchs, and Hod Lipson. 2015. *Understanding Neural Networks Through Deep Visualization*. Technical report. arXiv:1506.06579.

[321] Yu, Jiahui, Xin Li, Jing Yu Koh, Han Zhang, Ruoming Pang, James Qin, Alexander Ku, Yuanzhong Xu, Jason Baldridge, and Yonghui Wu. 2021. *Vector-quantized Image Modeling with Improved VQGAN*. Technical report. arXiv:2110.04627.

[322] Yu, Jiahui, Yuanzhong Xu, Jing Yu Koh, Thang Luong, Gunjan Baid, Zirui Wang, Vijay Vasudevan, et al. 2022. *Scaling Autoregressive Models for Content-Rich Text-to-Image Generation*. Technical report. arXiv:2206.10789.

[323] Yu, Lili, Bowen Shi, Ramakanth Pasunuru, Benjamin Muller, Olga Golovneva, Tianlu Wang, Arun Babu, et al. 2023. *Scaling Autoregressive Multi-Modal Models: Pretraining and Instruction Tuning*. Technical report. arXiv:2309.02591.

[324] Zaheer, Manzil, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Ruslan Salakhutdinov, and Alexander Smola. 2017. *Deep Sets*. Technical report. arXiv:1703.06114.

[325] Zarchan, P., and H. Musoff. 2005. *Fundamentals of Kalman Filtering: A Practical Approach*. Second. AIAA.

[326] Zeiler, Matthew D., and Rob Fergus. 2013. *Visualizing and Understanding Convolutional Networks*. Technical report. arXiv:1311.2901.

[327] Zhang, Chiyuan, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. 2016. *Understanding deep learning requires rethinking generalization*. Technical report. arXiv:1611.03530.

[328] Zhao, Wayne Xin, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, et al. 2023. *A Survey of Large Language Models*. Technical report. arXiv:2303.18223.

[329] Zhou, Jie, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2018. *Graph Neural Networks: A Review of Methods and Applications*. Technical report. arXiv:1812.08434.

[330] Zhou, Y., and R. Chellappa. 1988. “Computation of optic flow using a neural network.” In *International Conference on Neural Networks*, 71–78. IEEE.

[331] Zhu, J-Y, T. Park, P. Isola, and A. Efros. 2017. *Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks*. Technical report. arXiv:1703.10593.

索引

1 × 1 convolution 1 × 1 卷积　255
1-of-K coding 1-of-K 编码方案　58, 119, 392
acceptance criterion 接受准则　375, 378, 381
activation 激活　15
activation function 激活函数　15, 137, 156
active constraint 激活的约束　528
AdaGrad AdaGrad　193
Adam optimization Adam 算法　193
adaptive rejection sampling 适应性拒绝采样　370
adjacency matrix 邻接矩阵　349
adjoint sensitivity method 伴随灵敏度　473
adversarial attack 对抗攻击　263
aggregation 聚合　353
aleatoric uncertainty 偶然不确定性　21
AlexNet AlexNet　257
alpha family α 家族　53
amortized inference 摊销推理　487
ancestral sampling 祖先采样　382
anchor 锚点　166
annealed Langevin dynamics 退火朗之万动力学　508
AR model, see autoregressive model AR 模型, 参见“自回归模型”
area under the ROC curve ROC 曲线下面积　130
artificial intelligence 人工智能　1
attention 注意力　306
attention head 注意力头　313
audio data 音频数据　339
auto-associative neural network, see autoencoder 自联想神经网络, 参见“自编码器”
autoencoder 自编码器　164, 479
automatic differentiation 自动微分　19, 201, 211
autoregressive flow 自回归流　470
autoregressive model 自回归模型　5, 299, 323
average pooling 平均池化　255
backpropagation 反向传播　17, 201
backpropagation through time 通过时间反向传播　325
bag of words 词袋　322
bagging 装袋　240
base distribution 基础分布　465
basis function 基函数　98, 138, 150
batch gradient descent 批量梯度下降　185
batch learning 批量学习　102
batch normalization 批量归一化　196
Bayes net 贝叶斯网　280
Bayes’ theorem 贝叶斯定理　26
Bayesian network 贝叶斯网络　280
Bayesian probability 贝叶斯概率　47
beam search 束搜索　329

Bernoulli distribution 伯努利分布 56, 81
Bernoulli mixture model 伯努利混合模型 409
BERT BERT 330
bi-gram model 二元模型 323
bias 偏差 35, 109
bias parameter 偏置参数 98, 116, 156
bias-variance trade-off 偏差-方差权衡 108
BigGAN BigGAN 458
bijective function 双射函数 466
binomial distribution 二项分布 57
bits 比特 41
blind source separation 盲源分离 437
blocked path 阻塞路径 290, 294
boosting 提升 241
bootstrap 自助 239
bottleneck 瓶颈问题 325
bounding box 边界框 265
Box-Muller method Box-Muller 方法 368
byte pair encoding 字节对编码 321
canonical correlation analysis 典型相关分析 426
canonical link function 规范连接函数 143
Cauchy distribution 柯西分布 368
causal attention 因果注意力 328
causality 因果 297
central differences 中心差分法 206
central limit theorem 中心极限定理 60
ChatGPT ChatGPT 335
child node 子节点 216, 281
Cholesky decomposition Cholesky 分解 368
circular normal distribution 圆形正态分布 78
classical probability 经典概率 47
classification 分类 3
CLIP CLIP 167
co-parents 共父节点 298
codebook vector 码本向量 338, 396
collider node 碰撞节点 292
combining models 模型整合 127
committee 委员会 239
complete data set 完整数据集 404
completing the square 完全平方 66
computer vision 计算机视觉 248
concave function 凹函数 46
concentration parameter 聚焦参数 78
condition number 条件数 190
conditional entropy 条件熵 47
conditional expectation 条件期望 31
conditional independence 条件独立性 127, 289
conditional mixture model 条件混合模型 173
conditional probability 条件概率 25

conditional VAE 条件 VAE 491
conditioner 条件函数 469
confusion matrix 混淆矩阵 128
continuous bag of words 连续词袋 320
continuous normalizing flow 连续标准化流 475
contrastive divergence 对比散度 387
contrastive learning 对比学习 165
convex function 凸函数 45
convolution 卷积 249, 275
convolutional network 卷积网络 247
correlation matrix 相关性矩阵 428
cost function 成本函数 124
coupling flow 耦合流 467
coupling function 耦合函数 469
covariance 协方差 32
Cox's axioms Cox 定理 48
cross attention 交叉注意力 332
cross-correlation 互相关 251, 275
cross-entropy error function 交叉熵误差函数 139, 141, 170
cross-validation 交叉验证 13
cumulative distribution function 累积分布函数 29
curse of dimensionality 维度诅咒 150
curve fitting 曲线拟合 7
CycleGAN CycleGAN 459
d-separation d 分离 289, 293, 407
DAG, see directed acyclic graph DAG, 参见“有向无环图”
data augmentation 数据增强 166, 222
data compression 数据压缩 396
DDIM DDIM 505
DDPM DDPM 493
decision 决策 105
decision boundary 决策边界 115, 122
decision region 决策区域 115, 122
decision surface, see decision boundary 决策面, 参见“决策边界”
decision theory 决策理论 105, 121
decoder 解码器 479
deep double descent 双重下降 232
deep learning 深度学习 17
deep neural networks 深度神经网络 17
deep sets 深度集合 356
DeepDream DeepDream 264
degrees of freedom 自由度 421
denoising 去噪 493
denoising autoencoder 去噪自编码器 482
denoising diffusion implicit model 去噪扩散隐式模型 505
denoising diffusion probabilistic model 去噪扩散概率模型 493
denoising score matching 去噪得分匹配 507
density estimation 密度估计 33, 55
dequantization 去量化 448

descendant node 后代节点 292
design matrix 设计矩阵 101
development set 开发集 13
diagonal covariance matrix 对角协方差矩阵 64
differential entropy 微分熵 44
diffusion kernel 扩散核 495
diffusion model 扩散模型 493
Dirac delta function 狄拉克 δ 函数 30
directed acyclic graph 有向无环图 282
directed cycle 有向环 282
directed factorization 有向分解 298
directed graph 有向图 280
directed graphical model 有向图模型 280
discriminant function 判别函数 116, 125
discriminative model 判别模型 126, 137, 297
disentangled representations 解纠缠表示 462
distributed representation 分布式表示 163
dot-product attention 点积注意力 310
double descent 双重下降 232
dropout Dropout 241
E step E 步骤 402, 405
early stopping 早停法 230
earth mover's distance 推土机距离 457
ECM, see expectation conditional maximization ECM, 参见“期望条件最大化”
edge 边 280, 349
edge detection 边缘检测 252
ELBO, see evidence lower bound ELBO, 参见“证据下界”
EM, see expectation maximization EM, 参见“最大期望”
embedding space 嵌入空间 163
embedding vector 嵌入向量 349
encoder 编码器 479
energy function 能量函数 384
energy-based models 基于能量的模型 384
ensemble methods 集成 239
entropy 熵 41
epistemic uncertainty 认知不确定性 21
epoch 训练周期 186
equality constraint 等式约束 527
equivariance 等变性 224, 251, 255, 317, 351
erf function erf 函数 143
error backpropagation, see backpropagation 误差反向传播, 参见“反向传播”
error function 误差函数 8, 48, 169, 182
Euler-Lagrange equations 欧拉-拉格朗日方程 525
evaluation trace 求值轨迹 213
evidence lower bound 证据下界 412, 439, 485, 499
expectation 期望 31
expectation conditional maximization 期望条件最大化 416
expectation maximization 最大期望 400, 404, 440, 442
expectation step, see E step 期望步, 参见“E 步骤”

expectations 期望 366
explaining away 相消解释 293
exploding gradient 梯度爆炸 196, 325
exponential distribution 指数分布 30, 367
exponential family 指数族分布 80, 136, 282
expression swell 表达式膨胀 212
factor analysis 因子分析 436
factor graph 因子图 280
factor loading 因子载荷 436
false negative 假阴性 23
false positive 假阳性 23
fast gradient sign method 快速梯度符号 263
fast R-CNN fast R-CNN 270
feature extraction 特征提取 17, 98
feature map 特征图 251
features 特征 155
feed-forward network 前馈网络 149, 168
feed-forward networks 前馈神经网络 16
few-shot learning 小样本学习 165, 335
filter 滤波器 250
fine-tuning 微调 3, 19, 165, 334
flow matching 流匹配 476
forward kinematics 正运动学 172
forward problem 正问题 172
forward propagation 前向传播 203
foundation model 基础模型 19, 306, 334, 349
frequentist probability 频率学派概率 47
fuel system 燃油系统 292
fully connected graphical model 全连接的图模型 281
fully convolutional network 全卷积网络 272
functional 泛函 524
Gabor filters Gabor 滤波器 259
gamma distribution 伽马分布 370
GAN, see generative adversarial network GAN, 参见“生成对抗网络”
gated recurrent unit 门控循环单元 326
Gaussian 高斯 32, 59
Gaussian mixture 高斯混合 74, 173, 234, 397
GEM, see generalized EM algorithm GEM, 参见“广义 EM 算法”
generalization 泛化 7
generalized EM algorithm 广义 EM 算法 416
generalized linear model 广义线性模型 138, 143
generative adversarial network 生成对抗网络 453
generative AI 生成式 AI 5
generative model 生成式模型 5, 126, 296, 454
generative pre-trained transformer 生成式预训练 Transformer 5, 326
geometric deep learning 几何深度学习 362
Gibbs sampling 吉布斯采样 380
global minimum 全局最小值 183
GNN, see graph neural network GNN, 参见“图神经网络”

GPT, see generative pre-trained transformer GPT, 参见“生成式预训练 Transformer”
GPU, see graphics processing unit GPU, 参见“图形处理单元”
gradient descent 梯度下降 181
graph attention network 图注意力网络 359
graph convolutional network 图卷积网络 353
graph neural network 图神经网络 347
graph representation learning 图表示学习 349
graphical model 图模型 279
graphical model factorization 图模型分解 282
graphics processing unit 图形处理单元 17, 306
group theory 群理论 222
guidance 引导 510
Hadamard product Hadamard 积 468
Hamiltonian Monte Carlo 哈密顿蒙特卡洛 384
handwritten digit 手写数字 427
He initialization He 初始化 188
head-to-head path 头头相接路径 292
head-to-tail path 头尾相接路径 291
Heaviside step function 单位阶跃函数 140
Hessian matrix 黑塞矩阵 183, 209
Hessian outer product approximation 黑塞外积近似法 210
heteroscedastic 异方差 173
hidden Markov model 隐马尔可夫模型 324, 407
hidden unit 隐藏单元 16, 156
hidden variable, see latent variable 隐变量, 参见“潜变量”
hierarchical representation 层次化表示 162
histogram density estimation 直方图密度估计 85
history of machine learning 机器学习历史 14
hold-out set 保留集 13
homogeneous Markov chain 均匀马尔可夫链 377
Hooke’s law 胡克定律 443
Hutchinson’s trace estimator Hutchinson 的迹估计器 474
hybrid Monte Carlo 混合蒙特卡洛 384
hyperparameter 超参数 12
IAF, see inverse autoregressive flow IAF, 参见“逆自回归流”
ICA, see independent component analysis ICA, 参见“独立成分分析”
identifiability 可识别性 400
IID, see independent and identically distributed IID, 参见“独立同分布”
image segmentation 图像分割 270
ImageNet data set ImageNet 数据集 257
importance sampling 重要性采样 371, 383
importance weight 重要性权重 372
improper distribution 反常分布 30
improper prior 反常先验 227
inactive constraint 非激活的约束 528
incomplete data set 不完整的数据集 404
independent and identically distributed 独立同分布 37, 294
independent component analysis 独立成分分析 437
independent factor analysis 独立因子分析 438

independent variables 独立变量 28
inductive bias 归纳偏置 17, 220
inductive learning 归纳学习 349, 358
inequality constraint 不等式约束 527
inference 推理 105, 121, 125, 288
InfoNCE InfoNCE 166
information theory 信息论 46
instance discrimination 个体判别 166
internal covariate shift 内部协变量偏移 197
internal representation 内部表示 265
intersection-over-union 交并比 266
intrinsic dimensionality 内在维度 421
invariance 不变性 222, 255, 351
inverse autoregressive flow 逆自回归流 471
inverse kinematics 逆运动学 172
inverse problem 逆问题 107, 172, 220, 296
Iris data 鸢尾花数据 151
IRLS, see iterative reweighted least squares IRLS, 参见“迭代重加权最小二乘法”
isotropic covariance matrix 各向同性协方差矩阵 64
iterative reweighted least squares 迭代重加权最小二乘法 140
Jacobian matrix 雅可比矩阵 39, 207
Jensen’s inequality 詹森不等式 46
Jensen-Shannon divergence Jensen-Shannon 散度 463
K nearest neighbours K 近邻 89
K-means clustering algorithm K 均值聚类算法 392, 408
Kalman filter 卡尔曼滤波器 302, 439
Karush-Kuhn-Tucker conditions 卡鲁什-库恩-塔克条件 528
kernel density estimator 核密度估计 86, 506
kernel function 核函数 87
kernel image 核图像 250
KKT, see Karush-Kuhn-Tucker conditions KKT, 参见“卡鲁什-库恩-塔克条件”
KL divergence, see Kullback-Leibler divergence KL 散度, 参见“Kullback-Leibler 散度”
Kosambi-Karhunen-Loève transform Kosambi-Karhunen-Loève 变换 422
Kullback-Leibler divergence Kullback-Leibler 散度 45, 463
Lagrange multiplier 拉格朗日乘子 526
Lagrangian 拉格朗日函数 527
Langevin dynamics 朗之万动力学 386
Langevin sampling 朗之万采样 387
language model 语言模型 326
Laplace distribution 拉普拉斯分布 30
large language model 大语言模型 5, 326, 333
lasso lasso 229
latent class analysis 潜在类别分析 409
latent diffusion model 潜扩散模型 512
latent variable 潜变量 64, 288, 391, 421
layer normalization 层归一化 198, 315
LDM, see latent diffusion model LDM, 参见“潜扩散模型”
LDS, see linear dynamical system LDS, 参见“线性动态系统”
leaky ReLU leaky ReLU 161

learning curve 学习曲线 192, 230
learning rate parameter 学习率 192
learning to learn 学会学习 165
least-mean-squares algorithm 最小二乘 103
least-squares GAN 最小二乘 GAN 456
leave-one-out 留一法 13
LeNet convolutional network LeNet 257
Levenberg-Marquardt approximation 列文伯格-马夸尔特近似法 210
likelihood function 似然函数 34, 399
likelihood weighted sampling 似然加权采样 383
linear discriminant 线性判别式 116
linear dynamical system 线性动态系统 439
linear independence 线性无关 518
linear regression 线性回归 6, 97
linear-Gaussian model 线性高斯模型 67, 284
linearly separable 线性可分 115
link, see edge 链接, 参见“边”
link function 连接函数 137, 144
LLM, see large language model LLM, 参见“大语言模型”
LMS, see least-mean-squares algorithm LMS, 参见“平均最小均方算法”
local minimum 局部极小值 182
log odds 对数几率 132
logic sampling 逻辑采样 383
logistic regression 逻辑斯谛回归 138
logistic sigmoid 逻辑斯谛 sigmoid 81, 99, 131, 139
logit function logit 函数 132
long short-term memory 长短时记忆 326
LoRA, see low-rank adaptation LoRA, 参见“低秩自适应”
loss function 损失函数 106, 124
loss matrix 损失矩阵 124
lossless data compression 无损数据压缩 396
lossy data compression 有损数据压缩 396
low-rank adaptation 低秩自适应 335
LSGAN, see least-squares GAN LSGAN, 参见“最小二乘 GAN”
LSTM, see long short-term memory LSTM, 参见“长短时记忆”
M step M 步骤 402, 405
macrostate 宏观状态 42
MAE, see masked autoencoder MAE, 参见“掩蔽自编码器”
MAF, see masked autoregressive flow MAF, 参见“掩蔽自回归流”
Mahalanobis distance 马哈拉诺比斯距离 60
manifold 流形 154, 445
MAP, see maximum a posteriori MAP, 参见“最大后验”
marginal probability 边缘概率 25
Markov blanket 马尔可夫毯 297, 382
Markov boundary, see Markov blanket 马尔可夫边界, 参见“马尔可夫毯”
Markov chain 马尔可夫链 300, 377
Markov chain Monte Carlo 马尔可夫链蒙特卡洛采样 375
Markov model 马尔可夫模型 300
Markov random field 马尔可夫随机场 280

masked attention 遮掩注意力 328
masked autoencoder 掩蔽自编码器 483
masked autoregressive flow 掩蔽自回归流 472
max-pooling 最大池化 255
max-unpooling 最大上采样 272
maximization step, see M step 最大化步骤, 参见“M 步骤”
maximum a posteriori 最大后验 49, 405
maximum likelihood 最大似然 34, 72, 101, 134
MCMC, see Markov chain Monte Carlo MCMC, 参见“马尔可夫链蒙特卡洛采样”
MDN, see mixture density network MDN, 参见“混合密度网络”
mean 平均值 32
mean value theorem 中值定理 43
measure theory 测度论 30
mel spectrogram 梅尔频谱 339
message-passing 消息传递 354
message-passing neural network 消息传递神经网络 354
meta-learning 元学习 165
Metropolis algorithm Metropolis 算法 375
Metropolis-Hastings algorithm Metropolis-Hastings 算法 378
microstate 微观状态 42
mini-batches 小批量 187
minimum risk 最小风险 127
Minkowski loss 闵可夫斯基损失 107
missing at random 随机缺失 406, 442
missing data 缺失数据 442
mixing coefficient 混合系数 75
mixture component 混合情况 75
mixture density network 混合密度网络 172
mixture distribution 混合分布 391
mixture model 混合模型 391
mixture of Gaussians 高斯混合分布 74, 173, 234, 397
MLP, see multilayer perceptron MLP, 参见“多层感知机”
MNIST data MNIST 数据集 421
mode collapse 模式崩溃 456
model averaging 模型平均 239
model comparison 模型比较 9
model selection 模型选择 12
moment 矩 33
momentum 动量 190
Monte Carlo dropout 蒙特卡洛 dropout 242
Monte Carlo sampling 蒙特卡洛采样 365
Moore-Penrose pseudo-inverse, see pseudo-inverse 摩尔-彭若斯伪逆, 参见“伪逆”
MRF, see Markov random field MRF, 参见“马尔可夫随机场”
multi-class logistic regression 多类逻辑斯谛回归 140
multi-head attention 多头注意力 313
multilayer perceptron 多层感知机 16, 149
multimodal transformer 自注意力 309
multimodality 多模态 173
multinomial distribution 多项分布 59, 82

multiplicity 多重数 42
multitask learning 多任务学习 165
mutual information 互信息 47
n-gram model n 元模型 323
naive Bayes model 朴素贝叶斯模型 128, 294, 322
nats 纳特 41
natural language processing 自然语言处理 305
natural parameter 自然参数 81
nearest-neighbours 最近邻 88
neocognitron 新认知机 260
Nesterov momentum Nesterov 动量 191
neural ordinary differential equation 神经常微分方程 472
neuroscience 神经科学 259
NLP, see natural language processing NLP, 参见“自然语言处理”
no free lunch theorem 无免费午餐定理 221
node 节点 280, 349
noise 噪声 21
noiseless coding theorem 无噪编码定理 41
noisy-OR 噪声 OR 303
non-identifiability 不可识别性 437
non-max suppression 非最大抑制 269
nonparametric methods 非参数化方法 56, 85
normal distribution, see Gaussian 正态分布, 参见“高斯分布”
normal equations 正规方程 101
normalized exponential, see softmax function 归一化指数, 参见“softmax 函数”
novelty detection 奇异值检测 126
object detection 目标检测 265
observed variable 观测变量 287
Old Faithful data 老忠实喷泉数据 74
on-hot encoding, see 1-of-K encoding 独热编码, 参见“1-of-K 编码”
one-shot learning 单样本学习 165
one-versus-one classifier 一对一分类器 118
one-versus-the-rest classifier 一对多分类器 117
online gradient descent 在线梯度下降 186
online learning 在线学习 102
ordered over-relaxation 有序过度放松 382
outer product approximation 外积近似法 210
outlier 离群值 120, 126, 143
over-fitting 过拟合 9, 108, 400
over-relaxation 过度放松 382
over-smoothing 过度平滑 362
padding 填充 252
parameter sharing 参数共享 234, 284
parameter shrinkage 参数收缩 103
parameter tying, see parameter sharing 参数捆绑, 参见“参数共享”
parent node 父节点 214, 281
partition function 配分函数 384
Parzen estimator, see kernel density estimator Parzen 估计器, 参见“核密度估计器”
Parzen window Parzen 窗 87

PCA, see principal component analysis PCA, 参见“主成分分析”
perceptron 感知机 15
periodic variables 周期变量 76
permutation matrix 排列矩阵 350
PixelCNN PixelCNN 338
PixelRNN PixelRNN 338
plate 板块 286
polynomial curve fitting 多项式曲线拟合 55
pooling 池化 255
positional encoding 位置编码 317
positive definite covariance 正定协方差矩阵 61
positive definite matrix 正定矩阵 523
posterior collapse 后验崩塌 491
posterior probability 后验概率 28
power method 幂方法 424
pre-activation 预激活 15
pre-processing 预处理 17
pre-training 预训练 164, 334
precision matrix 精度矩阵 65
precision parameter 精度 32
predictive distribution 预测分布 37, 105
prefix prompt 前缀提示 335
principal component analysis 主成分分析 422, 430, 481
principal subspace 主子空间 422
prior 先验 227
prior knowledge 先验知识 17, 220
prior probability 先验概率 28, 126
probabilistic graphical model, see graphical model 概率图模型, 参见“图模型”
probabilistic PCA 概率 PCA 430
probability 概率 23
probability density 概率密度 28
probability theory 概率论 22
probit function probit 函数 142
probit regression probit 回归 141
product rule of probability 概率的乘积法则 24, 25, 280
prompt 提示 335, 511
prompt engineering 提示工程 335
proposal distribution 提议分布 369, 371, 375
pseudo-inverse 伪逆 101, 120
pseudo-random numbers 伪随机数 366
quadratic discriminant 二次判别式 133
radial basis functions 径向基函数 155
random variable 随机变量 24
raster scan 栅格扫描 338
readout layer 读出层 357
real NVP normalizing flow 实数 NVP 标准化流 468
receiver operating characteristic, see ROC curve 受试者工作特征曲线, 参见“ROC 曲线”
receptive field 感受野 250, 355
recurrent neural network 递归神经网络 324

regression 回归 3
regression function 回归函数 106
regularization 正则化 11, 219
regularized least squares 正则化最小二乘法 103
reject option 拒绝选项 125, 127
rejection sampling 拒绝采样 369
relative entropy 相对熵 45
reparameterization trick 重参数化技巧 488
representation learning 表示学习 19, 163
residual block 残差块 237
residual connection 残差连接 19, 236
residual network 残差网络 237
resnet, see residual network ResNet, 参见“残差网络”
responsibility 责任 75, 398
RLHF RLHF 335
RMS error, see root-mean-square error RMS 误差, 参见“均方根误差”
RMSProp RMSProp 193
RNN, see recurrent neural network RNN, 参见“递归神经网络”
robot arm 机械臂 172
robustness 鲁棒性 120
ROC curve ROC 曲线 129
root-mean-square error 均方根误差 9
saliency map 显著性图 262
same convolution 等大卷积 253
sample mean 样本均值 34
sample variance 样本方差 34
sampling 采样 365
sampling-importance-resampling 采样-重要性-重采样 373
scale invariance 尺度不变性 222
scaled self-attention 缩放自注意力 312
scaling hypothesis 缩放假设 306
Schur complement 舒尔补 67
score function 得分函数 386, 505
score matching 得分匹配 505
self-attention 自注意力 309
self-supervised learning 自监督学习 5, 320
semi-supervised learning 半监督学习 358
sequential estimation 序贯估计 73
sequential gradient descent 序贯梯度下降 103
sequential learning 序贯学习 102
SGD, see stochastic gradient descent SGD, 参见“随机梯度下降”
shared parameters, see parameter sharing 共享参数, 参见“参数共享”
shared weights 共享权重 251
shattered gradients 破碎梯度 236
shrinkage 收缩 11
sigmoid, see logistic sigmoid sigmoid, 参见“逻辑斯谛 sigmoid”
singular value decomposition 奇异值分解 102
SIR, see sampling-importance-resampling SIR, 参见“采样-重要性-重采样”
skip-grams 跳字 320

skip-layer connections 跳层连接 237
sliding window 滑动窗口 267
smoothing parameter 平滑参数 95
soft ReLU soft ReLU 161
soft weight sharing 软权重共享 234
softmax function softmax 函数 83, 132, 171, 174, 310
softplus activation function softplus 激活函数 160
sparse autoencoders 稀疏自编码器 482
sparse connections 稀疏连接 251
sparsity 稀疏 229
sphering 球化 429
standard deviation 标准差 32
standardizing 标准化 394, 428
state-space model 状态空间模型 301
statistical bias, see bias 统计偏差, 参见“偏差”
statistical independence, see independent variables 统计独立, 参见“独立变量”
steepest descent 最速下降 185
Stein score, see score function 斯坦因得分, 参见“得分函数”
Stirling’s approximation 斯特林公式 42
stochastic 随机 7
stochastic differential equation 随机微分方程 508
stochastic gradient descent 随机梯度下降 17, 186
stochastic variable 随机变量 26
strided convolution 跨步卷积 253
strides 跨步 267
structured data 结构化数据 247, 365
style transfer 风格迁移 274
sufficient statistics 充分统计量 57, 58, 72, 84
sum rule of probability 概率的加和法则 24, 25, 280
sum-of-squares error 平方和误差函数 8, 37, 119
supervised learning 监督学习 3, 358
support vector machine 支持向量机 156
SVD, see singular value decomposition SVD, 参见“奇异值分解”
SVM, see support vector machine SVM, 参见“支持向量机”
swish activation function swish 激活函数 179
symmetry 对称性 222
symmetry breaking 对称性破坏 188
tail-to-tail path 尾尾相接 290
tangent propagation 切线传播 223
temperature 温度 330
tensor 张量 168, 253
test set 测试集 9, 13
text-to-speech 文本语音转换 340
tied parameters, see parameter sharing 捆绑参数, 参见“参数共享”
token token 308
tokenization 分词 321
training set 训练集 3
transductive 直推式 349, 358
transductive learning 直推式学习 358

transfer learning 迁移学习 3, 164, 188, 331
transformers Transformer 305
transition probability 转移概率 377
translation invariance 平移不变性 222
transpose convolution 转置卷积 272
tri-gram model 三元模型 323
TTS, see text-to-speech TTS, 参见“文本语音转换”
U-net U-Net 273
undetermined multiplier, see Lagrange multiplier 未定乘子, 参见“拉格朗日乘子”
undirected graphical model 无向图模型 280
uniquenesses 唯一性 436
universal approximation theorems 通用近似器 159
unobserved variable, see latent variable 未观测变量, 参见“隐变量”
unsupervised learning 无监督学习 4, 163
utility function 效用函数 124
VAE, see variational autoencoder AE, 参见“变分自编码器”
valid convolution 有效卷积 253
validation set 验证集 13
vanishing gradient 梯度消失 196, 325
variance 方差 31, 32, 109
variational autoencoder 变分自编码器 484
variational inference 变分推断 412
variational lower bound, see evidence lower bound 变分下界, 参见“证据下界”
vector quantization 向量量化 338, 396
vertex, see node 顶点, 参见“点”
vision transformer 视觉 Transformer 336
von Mises distribution 冯 · 米塞斯分布 76
voxel 体素 249
Wasserstein distance Wasserstein 距离 457
Wasserstein GAN Wasserstein GAN 458
wavelets 小波 99
weakly supervised 弱监督 167
weight decay 权重衰减 11, 225
weight parameter 权重参数 15, 156
weight sharing, see parameter sharing 权重共享, 参见“参数共享”
weight vector 权重向量 116
weight-space symmetry 权重空间的对称性 161
WGAN, see Wasserstein GAN WGAN, 参见“Wasserstein GAN”
whitening 白化 429
Woodbury identity 伍德伯里恒等式 518
word embedding 词嵌入 320
word2vec word2vec 320
wrapped distribution 环绕分布 80
Yellowstone National Park 黄石国家公园 74